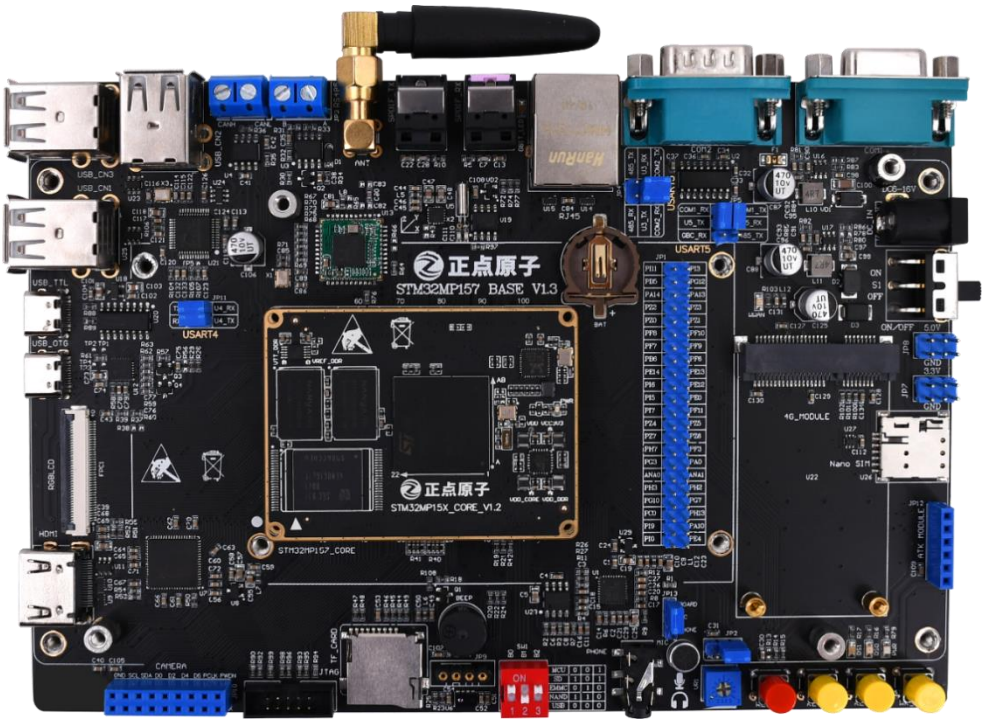
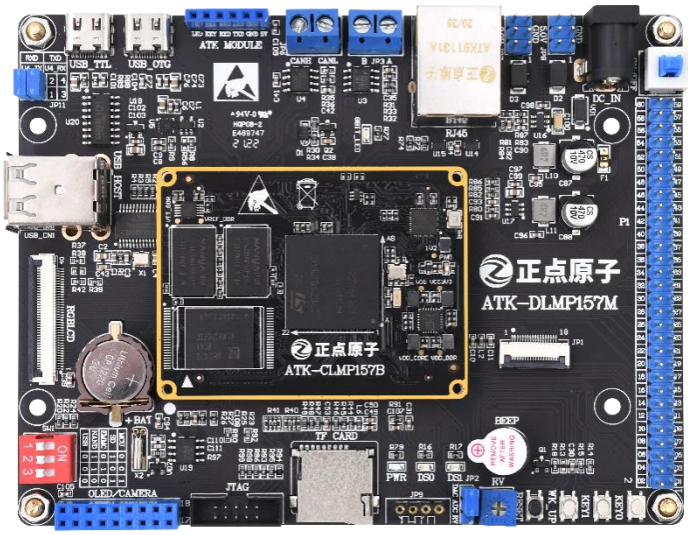
STM32MP157 嵌入式 Linux 驱动开发指南 V2.1

-正点原子 STM32MP157/Mini 开发板教程





正点原子公司名称 ：广州市星翼电子科技有限公司
原子哥在线教学平台：www.yuanzige.com
开源电子网 / 论坛 ：http://www.openedv.com/forum.php
正点原子淘宝店铺 ：https://openedv.taobao.com
正点原子官方网站 ：www.alientek.com
正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。
请下载原子哥 APP，数千讲视频免费学习，更快更流畅。

扫码关注正点原子公众号　　扫码下载“原子哥”APP

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	**初稿：**	梁文聪 左忠凯	左忠凯	2020.11.6
V1.1	**修改：** 1、将第五章顺序调整为第六章 2、修改第四章，添加了 Linux 下相关软件安装 **添加：** 1、添加第五章 STM32MP1 启动详解 2、添加第七章 TF-A 初探 3、添加第九章 TF-A 移植	梁文聪 左忠凯	左忠凯	2020.11.23
V1.2	**添加：** 1、添加第十章 U-boot 使用 2、添加第十三章 U-boot 移植 3、添加第十四章 U-boot 图形化配置及其原理	梁文聪 左忠凯	左忠凯	2020.12.10
V1.3	**添加：** 1、添加第十七章 Linux 内核移植 2、添加第十八章 Busybox 根文件系统构建 3、添加第十九章 Buildroot 根文件系统构建 4、添加第二十章 字符设备驱动开发	梁文聪 左忠凯	左忠凯	2020.12.26
V1.4	**添加：** 1、添加第二十一章 嵌入式 Linux LED 驱动开发实验 2、添加第二十二章 新字符设备驱动实验 3、添加第二十三章 Linux 设备树 4、添加第二十四章 设备树下的 LED 驱动实验	梁文聪 左忠凯	左忠凯	2021.1.30

	5、添加第二十五章 pinctrl 和 gpio 子系统实验 6、添加第二十六章 Linux 蜂鸣器实验 7、添加第二十七章 Linux 并发与竞争			
V1.5	**添加：** 1、添加第二十八章 Linux 并发与竞争实验 2、添加第二十九章 Linux 按键输入实验 3、添加第三十章 Linux 内核定时器实验 4、添加第三十一章 Linux 中断实验 5、添加第三十二章 Linux 阻塞和非阻塞实验 6、添加第三十三章 异步通知实验	梁文聪 左忠凯	左忠凯	2021.3.16
V1.6	**添加：** 1、添加第三十四章 platform 设备驱动实验 2、添加第三十五章 设备树下的 platform 驱动编写实验 3、添加第三十六章 Linux 自带的 LED 灯驱动实验 4、添加第三十七章 Linux MISC 驱动实验 5、添加第三十八章 Linux INPUT 子系统实验 6、添加第三十九章 LCD 驱动实验 7、添加第四十章 Linux I2C 驱动实验	梁文聪 左忠凯	左忠凯	2021.4.30
V1.7	**添加：** 1、添加第四十一章 RGB 转 HDMI 实验 2、添加第四十二章 Linux RTC 驱动实验 3、添加第四十三章 外置 RTC 芯片 PCF8563 实验 4、添加第四十四章 Linux SPI 驱动实验 5、添加第四十五章 Linux RS232/485/GPS 驱动实验 6、添加第四十六章 多点电容触摸屏实验 7、添加第四十七章 Linux 音频驱动实验	梁文聪 左忠凯	左忠凯	2021.7.6

	8、Linux CAN 驱动实验			
V1.8	**添加：** 1、第三十九章 Linux PWM 驱动实验 2、第五十章 Linux USB 驱动实验 3、第五十一章 Linux 块设备驱动实验 4、第五十二章 Linux 网络设备驱动实验 5、第五十三章 Linux WIFI 驱动实验 6、第五十四章 Linu 4G 通信实验 **调整：** 由于将 Linux PWM 驱动实验添加到了第三十九章，因此 V1.7 版本以前第三十九章往后的章节标号都往后推了一章。			
V1.9	**添加：** 1、第五十五章 Regmap API 实验 2、第五十六章 Linux IIO 驱动实验 3、第五十七章 Linux ADC 驱动实验 4、第五十八章 Linux DAC 驱动实验 5、第五十九章 Linux 单总线驱动实验	梁文聪 左忠凯	左忠凯	2021.10.3
V2.0	**添加：** 1、第十一章 U-boot 顶层 Makefile 详解 2、第十二章 U-boot 启动流程详解 3、第十五章 Linux 内核顶层 Makefile 详解 4、第十六章 Linux 内核启动流程	梁文聪 左忠凯	左忠凯	2021.11.1
V2.1	1、修复以前的笔误	梁文聪	左忠凯	2023.2.23

STM32MP157/Mini 开发板教程适配表

正点原子目前有两款 STM32MP157 开发板：STM32MP157 全功能板和 Mini 板，本教程同时适用于这两款开发板。但是，由于 Mini 开发板是 STM32MP157 全功能开发板的精简板，因此会少一些外设，所以本教程有些章节实验就无法在 Mini 板上完成，关于本教程在两款开发板的适配情况请参考下表：

注意：●表示适配，否则表示不适配

章节名称	全功能开发板	Mini 开发板	备注
第一章 Ubuntu 系统安装	●	●	
第二章 Ubuntu 系统入门	●	●	
第三章 Linux C 编程入门	●	●	
第四章 开发环境搭建	●	●	
第五章 STM32MP1 启动详解	●	●	
第六章 TF-A 使用	●	●	
第七章 TF-A 初探	●	●	
第八章 TF-A 启动流程详解	●	●	
第九章 TF-A 移植	●	●	
第十章 U-Boot 使用实验	●	●	
第十一章 U-Boot 顶层 Makefile 详解	●	●	
第十二章 U-Boot 启动流程详解	●	●	
第十三章 U-Boot 移植	●	●	
第十四章 U-Boot 图形化配置及其原理	●	●	
第十五章 Linux 内核顶层 Makefile 详解	●	●	
第十六章 Linux 内核启动流程	●	●	
第十七章 Linux 内核移植	●	●	
第十八章 Busybox 根文件系统构建	●	●	
第十九章 Buildroot 根文件系统构建	●	●	
第二十章 字符设备驱动开发	●	●	
第二十一章 嵌入式 Linux LED 驱动开发实验	●	●	
第二十二章 新字符设备驱动实验	●	●	
第二十三章 Linux 设备树	●	●	
第二十四章 设备树下的 LED 驱动实验	●	●	
第二十五章 pinctrl 和 gpio 子系统实验	●	●	
第二十六章 Linux 蜂鸣器实验	●	●	
第二十七章 Linux 并发与竞争	●	●	
第二十八章 Linux 并发与竞争实验	●	●	
第二十九章 Linux 按键输入实验	●	●	
第三十章 Linux 内核定时器实验	●	●	
第三十一章 Linux 中断实验	●	●	
第三十二章 Linux 阻塞和非阻塞 IO 实验	●	●	
第三十三章 异步通知实验	●	●	

第三十四章 platform 设备驱动实验	●	●	
第三十五章 设备树下的 platform 驱动编写	●	●	
第三十六章 Linux 自带的 LED 灯驱动实验	●	●	
第三十七章 Linux MISC 驱动实验	●	●	
第三十八章 Linux INPUT 子系统实验	●	●	
第三十九章 Linux PWM 驱动实验	●	●	
第四十章 Linux LCD 驱动实验	●	●	
第四十一章 Linux I2C 驱动实验	●		
第四十二章 RGB 转 HDMI 实验	●		
第四十三章 Linux RTC 驱动实验	●	●	
第四十四章 外置 RTC 芯片 PCF8563 实验	●	●	
第四十五章 Linux SPI 驱动实验	●		
第四十六章 Linux RS232/485/GPS 驱动实验	●	●	Mini 板只能完成 RS485 和 GPS
第四十七章 Linux 多点电容触摸屏实验	●	●	
第四十八章 Linux 音频驱动实验	●		
第四十九章 Linux CAN 驱动实验	●	●	
第五十章 Linux USB 驱动实验	●	●	
第五十一章 Linux 块设备驱动实验	●	●	
第五十二章 Linux 网络驱动实验	●	●	
第五十三章 Linux WIFI 驱动实验	●		
第五十四章 Linux 4G 通信实验	●		
第五十五章 Regmap API 实验	●		
第五十六章 Linux IIO 驱动实验	●		
第五十七章 Linux ADC 驱动实验	●	●	
第五十八章 Linux ADC 驱动实验	●	●	
第五十九章 Linux 单总线驱动实验	●	●	

STM32MP157/MINI 开发板教程适配表 7

前言 32

第一篇 UBUNTU 系统入门篇 34

第一章 UBUNTU 系统安装 35

1.1 安装虚拟机软件 VMWARE 36
1.2 创建虚拟机 42
1.3 安装 UBUNTU 操作系统 53
1.3.1 获取 Ubuntu 系统 53
1.3.2 安装 Ubuntu 操作系统 54
1.4 VMWARE TOOLS 安装 65

第二章 UBUNTU 系统入门 68

2.1 UBUNTU 系统初体验 69
2.1.1 Hello Ubuntu 69
2.1.2 系统设置 71
2.1.3 系统关机与锁定 74
2.1.4 中文输入测试 75
2.2 UBUNTU 终端操作 78
2.3 SHELL 操作 79
2.3.1 Shell 简介 79
2.3.2 Shell 基本操作 80
2.2.4 常用 Shell 命令 81
2.4 APT 下载工具 89
2.5 UBUNTU 下文本编辑 93
2.5.1 Gedit 编辑器 93
2.5.2 VI/VIM 编辑器 93
2.6 LINUX 文件系统 98
2.6.1 Linux 文件系统简介以及类型 98
2.6.2 Linux 文件系统结构 100
2.6.2 文件操作命令 103
2.6.3 文件压缩和解压缩 107
2.6.4 文件查询和搜索 112
2.6.5 文件类型 114
2.7 LINUX 用户权限管理 114
2.7.1 Ubuntu 用户系统 114
2.7.2 权限管理 115
2.7.3 权限管理命令 117
2.8 LINUX 磁盘管理 119
2.8.1 Linux 磁盘管理基本概念 119
2.8.2 磁盘管理命令 121

第三章 LINUX C 编程入门 125

3.1 HELLO WORLD！ 126
3.1.1 编写代码 *126*
3.1.2 编译代码 *127*
3.2 GCC 编译器 129
3.2.1 gcc 命令 *129*
3.2.2 编译错误警告 *129*
3.2.3 编译流程 *130*
3.3 MAKEFILE 基础 131
3.3.1 何为 Makefile *131*
3.3.2 Makefile 的引入 *131*
3.4 MAKEFILE 语法 135
3.4.1 Makefile 规则格式 *135*
3.4.2 Makefile 变量 *136*
3.4.3 Makefile 模式规则 *138*
3.4.4 Makefile 自动化变量 *139*
3.4.5 Makefile 伪目标 *140*
3.4.6 Makefile 条件判断 *141*
3.4.7 Makefile 函数使用 *141*

第二篇 系统移植篇 **144**

第四章 开发环境搭建 **145**

4.1 UBUNTU 和 WINDOWS 文件互传 146
4.2 UBUNTU 下 NFS 和 SSH 服务开启 152
4.2.1 NFS 服务开启 *152*
4.2.2 SSH 服务开启 *153*
4.3 UBUNTU 交叉编译工具链安装 153
4.3.1 交叉编译器安装 *153*
4.3.2 安装相关库 *156*
4.3.3 交叉编译器验证 *156*
4.4 VISUAL STUDIO CODE 软件的安装和使用 157
4.4.1 Visual Studio Code 的安装 *157*
4.5.2 Visual Studio Code 插件的安装 *161*
4.5.3 Visual Studio Code 新建工程 *163*
4.5 CH340 串口驱动安装 169
4.6 MOBAXTERM 软件安装和使用 172
4.6.1 MobaXterm 软件安装 *172*
4.6.2 MobaXterm 软件使用 *174*
4.7 WINDOWS 下 ST 官方软件安装 177
4.7.1 ST 官方软件简介 *177*
4.7.2 Java 环境安装 *179*
4.7.3 STM32CubeMX 安装 *180*
4.7.4 STM32CubeIDE 安装 *180*
4.7.5 STM32CubeProgrammer 安装 *184*

4.8 UBUNTU 下 ST 官方软件安装 185
4.8.1 Java 环境安装 185
4.8.2 STM32CubeMX 安装 186
4.8.3 STM32CubeIDE 安装 187
4.8.4 STM32CubeProgrammer 安装 188
4.9 USB DFU 以及 STLINK 驱动安装 189
4.9.1 Windows 下 USB DFU 以及 STLink 驱动安装 189
4.9.2 Ubuntu 下 USB DFU 以及 STLink 驱动安装 190

第五章 STM32MP1 启动详解 193

5.1 STM32MP1 启动模式 194
5.2 STM32MP1 启动流程详解 195
5.2.1 内部 ROM 代码 195
5.2.2 安全启动 196
5.2.3 串行启动 200
5.3 FLASH 设备启动要求 201
5.3.1 从 NAND 启动 202
5.3.2 从 EMMC 启动 202
5.3.3 从 SD 卡启动 203
5.4 STM32MP1 二进制头部信息 203
5.5 STM23MP1 LINUX 系统启动过程 206

第六章 TF-A 使用 209

6.1 你好 TF-A 210
6.1.1 系统源码获取 210
6.1.2 TF-A 源码打补丁 212
6.1.3 创建 VSCode 工程 213
6.2 编译和烧录 TF-A 215
6.2.1 stm32wrapper4dbg 工具安装 215
6.2.2 编译正点原子官方的 TF-A 216
6.2.3 TF-A 烧录到 EMMC 218
6.2.4 TF-A 的运行 226

第七章 TF-A 初探 227

7.1 设备安全怎么保证？ 228
7.2 TF-A 概述 228
7.2.1 TF-A 简介 228
7.2.2 ARMv7 和 ARMv8 权限等级 229
7.2.3 TF-A 不同启动阶段 231
7.3 STM32MP1 中的 TF-A 233
7.3.1 STM32MP1 TF-A 框架 233
7.3.2 STM32MP1 TF-A 镜像存储映射 237

第八章 TF-A 启动流程详解 238

第九章 TF-A 移植 239

9.1 编译 ST 官方 TF-A 源码 240
9.1.1 为什么要编译官方 TF-A 源码 240
9.1.2 编译 TF-A 240
9.1.3 烧写 ST 官方 TF-A 镜像 242
9.2 TF-A 移植 243
9.2.1 新建自己开发板对应的设备树 243
9.2.2 修改设备树电源管理 247
9.2.3 修改 TF 卡和 EMMC 设备树 252
9.2.4 修改 USB OTG 设备树 254
9.3 编译测试 255
9.3.1 编译 255
9.3.2 烧写测试 255
9.3.3 xxx-serialboot.stm32 测试 259

第十章 U-BOOT 使用 261

10.1 U-Boot 简介 262
10.2 U-Boot 初次编译 264
10.2.1 编译 264
10.2.2 烧写 266
10.3 U-Boot 命令使用 268
10.3.1 查询命令 269
10.3.2 环境变量操作命令 271
10.3.3 内存操作命令 272
10.3.4 网络操作命令 275
10.3.5 EMMC 和 SD 卡操作命令 281
10.3.6 EXT 格式文件系统操作命令 285
10.3.7 BOOT 操作命令 288
10.3.8 UMS 命令 292
10.3.9 其他常用命令 293
10.3.10 MII 命令使用说明 294

第十一章 U-BOOT 顶层 MAKEFILE 详解 297

11.1 U-Boot 工程目录分析 298
11.2 U-Boot 顶层 Makefile 分析 306
11.2.1 版本号 306
11.2.2 MAKEFLAGS 变量 306
11.2.3 命令输出 307
11.2.4 静默输出 309
11.2.5 设置编译结果输出目录 310
11.2.6 代码检查 312
11.2.7 模块编译 312
11.2.8 获取主机架构和系统 314

11.2.9 设置目标架构、交叉编译器和配置文件 314
11.2.10 调用 scripts/Kbuild.include 315
11.2.11 交叉编译工具变量设置 316
11.2.12 导出其他变量 316
11.2.13 make xxx_defconfig 过程 320
11.2.14 Makefile.build 脚本分析 325
11.2.15 make 过程 329
第十二章 U-BOOT 启动流程详解 337
12.1 链接脚本 U-BOOT.LDS 详解 338
12.2 U-BOOT 启动流程详解 340
12.2.1 reset 函数源码详解 340
12.2.2 lowlevel_init 函数详解 345
12.2.3 _main 函数详解 348
12.2.4 board_init_f 函数详解 354
12.2.5 relocate_code 函数详解 364
12.2.6 relocate_vectors 函数详解 369
12.2.7 board_init_r 函数详解 370
12.2.8 run_main_loop 函数详解 375
12.2.9 cli_loop 函数详解 380
12.2.10 cmd_process 函数详解 383
12.3 BOOTM 启动 LINUX 内核过程 388
12.3.1 images 全局变量 388
12.3.2 do_bootm 函数 390
12.3.3 do_bootm_states 函数 391
12.3.4 bootm_os_get_boot_func 函数 400
12.3.5 do_bootm_linux 函数 401
第十三章 U-BOOT 移植 405
13.1 ST 官方的 U-BOOT 编译测试 406
13.1.1 ST 官方 uboot 源码打补丁 406
13.1.2 编译 ST 官方 uboot 源码 407
13.1.3 烧写测试 408
13.2 在 U-BOOT 中添加自己的开发板 409
13.2.1 创建默认配置文件 409
13.2.2 创建默认配置设备树 410
13.2.3 修改电源管理设置 411
13.2.4 修改 TF 卡和 EMMC 配置 418
13.2.5 编译 uboot 419
13.2.6 网络驱动修改 421
13.2.7 USB OTG 设备树修改 424
13.2.8 使能 boot 和 bootd 命令 428
13.2.9 LCD 驱动修改 430
13.3 U-BOOT 测试 435

13.3.1 自烧写测试 *435*
13.3.2 从 EMMC 启动 Linux *436*
13.3.3 从网络启动 Linux 系统 *437*
13.3.4 LCD 测试 *439*
13.4 BOOTCMD 和 BOOTARGS 环境变量 442
13.4.1 bootcmd 环境变量 *442*
13.4.2 bootargs 环境变量 *445*
第十四章 U-BOOT 图形化配置及其原理 447
14.1 U-BOOT 图形化配置体验 448
14.2 MENUCONFIG 图形化配置原理 453
14.2.1 make menuconfig 过程分析 *453*
14.2.2 Kconfig 语法简介 *453*
14.3 添加自定义菜单 465
第十五章 LINUX 内核顶层 MAKEFILE 详解 468
15.1 LINUX 内核初次编译 469
15.2 LINUX 工程目录分析 471
15.3 顶层 MAKEFILE 详解 477
15.3.1 make xxx_defconfig 过程 *481*
15.3.2 Makefile.build 脚本分析 *482*
15.3.3 make 过程 *484*
15.3.4 built-in.o 文件编译生成过程 *489*
15.3.5 make zImage 过程 *491*
第十六章 LINUX 内核启动流程 493
16.1 链接脚本 VMLINUX.LDS 494
16.2 LINUX 内核启动流程分析 494
16.2.1 Linux 内核入口 stext *494*
16.2.2 __mmap_switched 函数 *496*
16.2.3 start_kernel 函数 *498*
16.2.4 rest_init 函数 *502*
16.2.5 init 进程 *503*
第十七章 LINUX 内核移植 508
17.1 LINUX 内核初次编译 509
17.1.1 编译正点原子出厂 Linux 源码 *509*
17.1.2 运行测试 *509*
17.2 编译 ST 官方 LINUX 系统 510
17.2.1 ST 官方 Linux 源码打补丁 *510*
17.2.2 编译 ST 官方 Linux 源码 *513*
17.2.3 网络驱动修改 *514*
17.2.4 启动测试 *515*
17.3 在 LINUX 中添加自己的开发板 516

17.3.1 添加开发板对应的默认配置文件 *516*
17.3.2 添加开发板对应的设备树 *516*
17.3.3 关闭内核模块验证 *523*
17.3.4 关闭内核 log 信息时间戳 *525*
17.3.5 编译测试 *526*
17.4 烧写系统镜像到 EMMC 里面 526
17.4.1 系统镜像打包 *527*
17.4.2 烧写到 EMMC *528*
17.5 根文件系统缺失错误 529

第十八章 BUSYBOX 根文件系统构建 531

18.1 根文件系统简介 532
18.2 BUSYBOX 构建根文件系统 534
18.2.1 BusyBox 简介 *534*
18.2.2 编译 BusyBox 构建根文件系统 *535*
18.2.3 向根文件系统添加 lib 库 *546*
18.2.4 创建其他文件夹 *548*
18.3 根文件系统初步测试 548
18.3.1 修改 Ubuntu 的 nfs 版本配置 *548*
18.3.2 bootargs 环境变量设置 *549*
18.3.3 无法进入根文件系统 *550*
18.4 完善根文件系统 551
18.4.1 创建/etc/init.d/rcS 文件 *551*
18.4.2 创建/etc/fstab 文件 *552*
18.4.3 创建/etc/inittab 文件 *553*
18.4.4 使能内核 uevet helper *554*
18.5 根文件系统其他功能测试 555
18.5.1 软件运行测试 *555*
18.5.2 中文字符测试 *557*
18.5.3 开机自启动测试 *557*
18.5.4 外网连接测试 *558*
18.6 烧写根文件系统到 EMMC 中 559
18.6.1 根文件系统打包 *559*
18.6.2 烧写到 EMMC *561*

第十九章 BUILDROOT 根文件系统构建 563

19.1 何为 BUILDROOT？ 564
19.1.1 buildroot 简介 *564*
19.1.2 buildroot 下载 *564*
19.2 BUILDROOT 构建根文件系统 566
19.2.1 配置 buildroot *566*
19.2.2 编译 buildroot *571*
19.2.3 buildroot 根文件系统测试 *573*
19.3 BUILDROOT 下的 BUSYBOX 配置 573

19.3.1 busybox 配置 *573*
19.3.2 busybox 中文字符的支持 *575*
19.3.3 编译 busybox *575*
19.4.3 使用自己的 busybox 源码 *576*
19.4 BUILDROOT 第三方软件和库的配置 577
19.4.1 使能 VSFTPD 服务 *577*
19.4.2 使能 SSH *577*
19.5 BUILDROOT 根文件系统测试 578
19.5.1 depmod 命令测试 *578*
19.5.2 vsftpd 测试 *579*
19.5.3 sshd 测试 *581*
19.5.4 创建自启动文件 *584*
19.5.5 显示路径 *585*
19.5.6 使能 sysfs debug 目录 *587*
19.6 烧写根文件系统到 EMMC 中 588
19.6.1 根文件系统打包 *588*
19.6.2 烧写到 EMMC *588*
19.6.3 EMMC 启动测试 *588*

第三篇 ARM LINUX 驱动开发篇 **591**

第二十章 字符设备驱动开发 **592**

20.1 字符设备驱动简介 593
20.2 字符设备驱动开发步骤 595
20.2.1 驱动模块的加载和卸载 *596*
20.2.2 字符设备注册与注销 *597*
20.2.3 实现设备的具体操作函数 *599*
20.2.4 添加 LICENSE 和作者信息 *601*
20.3 LINUX 设备号 601
20.3.1 设备号的组成 *601*
20.3.2 设备号的分配 *602*
20.4 CHRDEVBASE 字符设备驱动开发实验 603
20.4.1 实验程序编写 *603*
20.4.2 编写测试 APP *610*
20.4.3 编译驱动程序和测试 APP *614*
20.4.4 运行测试 *616*

第二十一章 嵌入式 LINUX LED 驱动开发实验 **620**

21.1 LINUX 下 LED 灯驱动原理 621
21.1.1 地址映射 *621*
21.1.2 I/O 内存访问函数 *623*
21.2 硬件原理图分析 623
21.3 实验程序编写 624
21.3.1 LED 灯驱动程序编写 *624*

21.3.2 编写测试 APP 630
21.4 运行测试 632
21.4.1 编译驱动程序和测试 APP 632
21.4.2 运行测试 632

第二十二章 新字符设备驱动实验 634

22.1 新字符设备驱动原理 635
22.1.1 分配和释放设备号 635
22.1.2 新的字符设备注册方法 636
22.2 自动创建设备节点 637
22.2.1 mdev 机制 637
22.2.1 创建和删除类 638
22.2.2 创建设备 638
22.2.3 参考示例 639
22.3 设置文件私有数据 640
22.4 硬件原理图分析 640
22.5 实验程序编写 640
22.5.1 LED 灯驱动程序编写 641
22.5.2 编写测试 APP 648
22.6 运行测试 648
22.6.1 编译驱动程序和测试 APP 648
22.6.2 运行测试 649

第二十三章 LINUX 设备树 651

23.1 什么是设备树？ 652
23.2 DTS、DTB 和 DTC 653
23.3 DTS 语法 655
23.3.1 .dtsi 头文件 655
23.3.2 设备节点 657
23.3.3 标准属性 659
23.3.4 根节点 compatible 属性 663
23.3.5 向节点追加或修改内容 668
23.4 创建小型模板设备树 671
23.5 设备树在系统中的体现 675
23.6 特殊节点 677
23.6.1 aliases 子节点 677
23.6.2 chosen 子节点 677
23.7 LINUX 内核解析 DTB 文件 680
23.8 绑定信息文档 680
23.9 设备树常用 OF 操作函数 682
23.9.1 查找节点的 OF 函数 683
23.9.2 查找父/子节点的 OF 函数 684
23.9.3 提取属性值的 OF 函数 685
23.9.4 其他常用的 OF 函数 688

第二十四章 设备树下的 LED 驱动实验 692

24.1 设备树 LED 驱动原理 693
24.2 硬件原理图分析 693
24.3 实验程序编写 693
24.3.1 修改设备树文件 693
24.3.2 LED 灯驱动程序编写 694
24.3.3 编写测试 APP 703
24.4.1 编译驱动程序和测试 APP 703
24.4.2 运行测试 703

第二十五章 PINCTRL 和 GPIO 子系统实验 705

25.1 PINCTRL 子系统 706
25.1.1 pinctrl 子系统简介 706
25.1.2 STM32MP1 的 pinctrl 子系统驱动 706
25.1.3 设备树中添加 pinctrl 节点模板 714
25.2 GPIO 子系统 716
25.2.1 gpio 子系统简介 716
25.2.2 STM32MP1 的 gpio 子系统驱动 716
25.2.3 gpio 子系统 API 函数 718
25.2.4 设备树中添加 gpio 节点模板 720
25.2.5 与 gpio 相关的 OF 函数 720
25.3 硬件原理图分析 721
25.4 实验程序编写 721
25.4.1 修改设备树文件 721
25.4.2 LED 灯驱动程序编写 722
25.4.3 编写测试 APP 729
25.5 运行测试 729
25.5.1 编译驱动程序和测试 APP 729
25.5.2 运行测试 729

第二十六章 LINUX 蜂鸣器实验 731

26.1 蜂鸣器驱动原理 732
26.2 硬件原理图分析 732
26.3 实验程序编写 733
26.3.1 修改设备树文件 733
26.3.2 蜂鸣器驱动程序编写 734
26.3.3 编写测试 APP 740
26.4 运行测试 742
26.4.1 编译驱动程序和测试 APP 742
26.4.2 运行测试 742

第二十七章 LINUX 并发与竞争 744

27.1 并发与竞争 745

27.2 原子操作 746
27.2.1 原子操作简介 746
27.2.2 原子整形操作 API 函数 747
27.2.3 原子位操作 API 函数 748
27.3 自旋锁 749
27.3.1 自旋锁简介 749
27.3.2 自旋锁 API 函数 750
27.3.3 其他类型的锁 752
27.3.4 自旋锁使用注意事项 753
27.4 信号量 754
27.4.1 信号量简介 754
27.4.2 信号量 API 函数 755
27.5 互斥体 755
27.5.1 互斥体简介 755
27.5.2 互斥体 API 函数 756

第二十八章 LINUX 并发与竞争实验 757

28.1 原子操作实验 758
28.1.1 实验程序编写 758
28.1.2 运行测试 767
28.2 自旋锁实验 768
28.2.1 实验程序编写 768
28.2.2 运行测试 772
28.3 信号量实验 773
28.3.1 实验程序编写 773
28.3.2 运行测试 776
28.4 互斥体实验 777
28.4.1 实验程序编写 777
28.4.2 运行测试 780

第二十九章 LINUX 按键输入实验 782

29.1 Linux 下按键驱动原理 783
29.2 硬件原理图分析 783
29.3 实验程序编写 783
29.3.1 修改设备树文件 783
29.3.2 按键驱动程序编写 784
29.3.3 编写测试 APP 791
29.4 运行测试 792
29.4.1 编译驱动程序和测试 APP 792
29.4.2 运行测试 793

第三十章 LINUX 内核定时器实验 794

30.1 Linux 时间管理和内核定时器简介 795
30.1.1 内核时间管理简介 795

30.1.2 内核定时器简介 798
30.1.3 Linux 内核短延时函数 800
30.2 硬件原理图分析 801
30.3 实验程序编写 801
30.3.1 修改设备树文件 801
30.3.2 定时器驱动程序编写 801
30.3.3 编写测试 APP 809
30.4 运行测试 811
30.4.1 编译驱动程序和测试 APP 811
30.4.2 运行测试 812

第三十一章 LINUX 中断实验 813

31.1 LINUX 中断简介 814
31.1.1 Linux 中断 API 函数 814
31.1.2 上半部与下半部 816
31.1.3 设备树中断信息节点 822
31.1.4 获取中断号 831
31.2 硬件原理图分析 832
31.3 实验程序编写 832
31.3.1 修改设备树文件 832
31.3.2 按键中断驱动程序编写 833
31.3.2 编写测试 APP 841
31.4 运行测试 843
31.4.1 编译驱动程序和测试 APP 843
31.4.2 运行测试 844

第三十二章 LINUX 阻塞和非阻塞 IO 实验 845

32.1 阻塞和非阻塞 IO 846
32.1.1 阻塞和非阻塞简介 846
32.1.2 等待队列 847
32.1.3 轮询 849
32.1.4 Linux 驱动下的 poll 操作函数 853
32.2 阻塞 IO 实验 854
32.2.1 硬件原理图分析 854
32.2.2 实验程序编写 854
32.2.3 运行测试 857
32.3 非阻塞 IO 实验 859
32.3.1 硬件原理图分析 859
32.3.2 实验程序编写 859
32.3.3 运行测试 863

第三十三章 异步通知实验 865

33.1 异步通知 866
33.1.1 异步通知简介 866

33.1.2 驱动中的信号处理 868
33.1.3 应用程序对异步通知的处理 870
33.2 硬件原理图分析 870
33.3 实验程序编写 870
33.3.1 修改设备树文件 871
33.3.2 程序编写 871
33.3.3 编写测试 APP 873
33.4 运行测试 875
33.4.1 编译驱动程序和测试 APP 875
33.4.2 运行测试 876
第三十四章 PLATFORM 设备驱动实验 877
34.1 LINUX 驱动的分离与分层 878
34.1.1 驱动的分隔与分离 878
34.1.2 驱动的分层 880
34.2 PLATFORM 平台驱动模型简介 880
34.2.1 platform 总线 880
34.2.2 platform 驱动 882
34.2.3 platform 设备 887
34.3 硬件原理图分析 890
34.4 试验程序编写 890
34.4.1 platform 设备与驱动程序编写 890
34.4.2 测试 APP 编写 900
34.5 运行测试 902
34.5.1 编译驱动程序和测试 APP 902
34.4.2 运行测试 903
第三十五章 设备树下的 PLATFORM 驱动编写 905
35.1 设备树下的 PLATFORM 驱动简介 906
35.1.1 修改 pinctrl-stm32.c 文件 906
35.1.2 创建设备的 pinctrl 节点 907
35.1.3 在设备树中创建设备节点 907
35.1.4 编写 platform 驱动的时候要注意兼容属性 908
35.2 检查引脚复用配置 909
35.2.1 检查引脚 pinctrl 配置 909
35.2.2 检查 GPIO 占用 910
35.3 硬件原理图分析 910
35.4 实验程序编写 910
35.4.1 修改设备树文件 911
35.4.2 platform 驱动程序编写 911
35.4.3 编写测试 APP 917
35.5 运行测试 917
35.5.1 编译驱动程序和测试 APP 917
35.5.2 运行测试 918

第三十六章 LINUX 自带的 LED 灯驱动实验 919

36.1 LINUX 内核自带 LED 驱动使能 920
36.2 LINUX 内核自带 LED 驱动简介 921
36.2.1 LED 灯驱动框架分析 921
36.2.2 module_platform_driver 函数简析 922
36.2.3 gpio_led_probe 函数简析 923
36.3 设备树节点编写 926
36.4 运行测试 927

第三十七章 LINUX MISC 驱动实验 929

37.1 MISC 设备驱动简介 930
37.2 硬件原理图分析 931
37.3 实验程序编写 931
37.3.1 修改设备树 931
37.3.2 beep 驱动程序编写 932
37.3.3 编写测试 APP 937
37.4 运行测试 939
37.4.1 编译驱动程序和测试 APP 939
37.4.2 运行测试 939

第三十八章 LINUX INPUT 子系统实验 941

38.1 INPUT 子系统 942
38.1.1 input 子系统简介 942
38.1.2 input 驱动编写流程 942
38.1.3 input_event 结构体 948
38.2 硬件原理图分析 949
38.3 实验程序编写 949
38.3.1 修改设备树文件 950
38.3.2 按键 input 驱动程序编写 950
38.3.3 编写测试 APP 956
38.4 运行测试 958
38.4.1 编译驱动程序和测试 APP 958
38.4.2 运行测试 958
38.5 LINUX 自带按键驱动程序的使用 960
38.5.1 自带按键驱动程序源码简析 960
38.5.2 自带按键驱动程序的使用 965

第三十九章 LINUX PWM 驱动实验 968

39.1 PWM 驱动简析 969
39.1.1 设备树下的 PWM 控制器节点 969
39.1.2 PWM 子系统 972
39.1.3 PWM 驱动源码分析 973
39.2 PWM 驱动编写 979

39.2.1 修改设备树 979
39.2.2 使能 PWM 驱动 982
39.3 PWM 驱动测试 982
第四十章 LINUX LCD 驱动实验 985
40.1 LCD 和 LTDC 简介 986
40.1.1 LCD 简介 986
40.1.2 LTDC 接口 992
40.2 DRM 驱动框架 993
40.2.1 DRM 简介 993
40.2.2 ST 官方的 DRM 驱动框架介绍 994
40.3 RGB LCD 驱动分析(屏的驱动) 1000
40.4 硬件原理图分析 1005
40.5 LCD 驱动程序编写 1006
40.5.1 修改设备树 1006
40.5.2 在 panel-simple.c 文件里面添加屏幕参数 1009
40.5.3 LCD 屏幕背光节点信息 1010
40.6 运行测试 1013
40.6.1 LCD 屏幕的 DRM 基本测试 1013
40.6.2 LCD 屏幕的 FB 基本测试 1016
40.6.3 设置 LCD 作为终端控制台 1019
40.6.4 LCD 背光调节 1021
第四十一章 LINUX I2C 驱动实验 1023
41.1 I2C & AP3216C 简介 1024
41.1.1 I2C 简介 1024
41.1.2 STM32MP1 I2C 简介 1026
41.1.3 AP3216C 简介 1027
41.2 LINUX I2C 总线框架简介 1028
41.2.1 I2C 总线驱动 1029
41.2.2 I2C 总线设备 1032
41.2.3 I2C 设备和驱动匹配过程 1035
41.3 STM32MP1 I2C 适配器驱动分析 1037
41.4 I2C 设备驱动编写流程 1043
41.4.1 I2C 设备信息描述 1043
41.4.2 I2C 设备数据收发处理流程 1044
41.5 硬件原理图分析 1047
41.6 实验程序编写 1048
41.6.1 修改设备树 1048
41.6.2 AP3216C 驱动编写 1050
41.6.3 编写测试 APP 1060
41.7 运行测试 1061
41.7.1 编译驱动程序和测试 APP 1061
41.7.2 运行测试 1062

第四十二章 RGB 转 HDMI 实验 1063

42.1 RGB 转 HMDI 简介 1064
42.2 硬件原理图分析 1064
42.3 实验驱动编写 1065
42.3.1 修改设备树 1065
42.3.2 使能内核自带的 sii902x 驱动 1068
42.4 RGB 转 HDMI 测试 1068

第四十三章 LINUX RTC 驱动实验 1070

43.1 LINUX 内核 RTC 驱动简介 1071
43.2 STM32MP1 内部 RTC 驱动分析 1075
43.3 RTC 时间查看与设置 1079
43.3.1 使能内部 RTC 1079
43.3.2 查看时间 1079

第四十四章 外置 RTC 芯片 PCF8563 实验 1081

44.1 PCF8563 简介 1082
44.1.1 PCF8563 简介 1082
44.1.2 PCF8563 寄存器详解 1083
44.2 硬件原理图分析 1085
44.3 实验驱动编写 1085
44.3.1 修改设备树 1085
44.3.2 PCF8563 驱动使能 1087
44.4 运行测试 1088
44.5 PCF8563 驱动分析 1088

第四十五章 LINUX SPI 驱动实验 1093

45.1 SPI & ICM-20608 简介 1094
45.1.1 SPI 简介 1094
45.1.2 STM32MP1 SPI 简介 1095
45.1.3 ICM-20608 简介 1096
45.2 LINUX 下 SPI 驱动框架 1099
45.2.1 SPI 主机驱动 1099
45.2.2 SPI 设备驱动 1102
45.2.3 SPI 设备和驱动匹配过程 1104
45.3 STM32MP1 SPI 主机驱动分析 1105
45.4 SPI 设备驱动编写流程 1108
45.4.1 SPI 设备信息描述 1108
45.4.2 SPI 从机设备数据收发处理流程 1109
45.5 硬件原理图分析 1112
45.6 实验程序编写 1112
45.6.1 修改设备树 1113
45.6.2 编写 ICM20608 驱动 1114

45.6.3 编写测试 APP 1126
45.7 运行测试 1128
45.7.1 编译驱动程序和测试 APP 1128
45.7.2 运行测试 1130
第四十六章 LINUX RS232/485/GPS 驱动实验 1132
46.1 LINUX 下 UART 驱动框架 1133
46.2 STM32MP1 UART 驱动分析 1136
46.3 硬件原理图分析 1142
46.4 RS232 驱动编写 1143
46.5 移植 MINICOM 1145
46.6 RS232 驱动测试 1147
46.6.1 RS232 连接设置 1147
46.6.2 minicom 设置 1148
46.6.3 RS232 收发测试 1151
46.7 RS485 测试 1152
46.7.1 RS485 连接设置 1152
46.7.2 RS485 收发测试 1153
46.8 GPS 测试 1154
46.8.1 GPS 连接设置 1154
46.8.2 GPS 数据接收测试 1156
第四十七章 多点电容触摸屏实验 1157
47.1 多点电容触摸简介 1158
47.2 LINUX 下电容触摸屏驱动框架简介 1159
47.2.1 多点触摸(MT)协议详解 1159
47.2.2 Type A 触摸点信息上报时序 1161
47.2.3 Type B 触摸点信息上报时序 1163
47.2.4 MT 其他事件的使用 1165
47.2.5 多点触摸所使用到的 API 函数 1165
47.2.6 多点电容触摸驱动框架 1167
47.3 硬件原理图分析 1171
47.4 试验程序编写 1172
47.4.1 添加 FT5426 设备节点 1172
47.4.2 编写多点电容触摸驱动 1173
47.5 运行测试 1181
47.5.1 编译驱动程序 1181
47.5.2 运行测试 1182
47.5.3 将驱动添加到内核中 1183
47.6 TSLIB 移植与使用 1184
47.6.1 tslib 移植 1184
47.6.2 tslib 测试 1185
47.7 使用内核自带的驱动 1187
47.8 4.3 寸屏触摸驱动实验 1189

第四十八章 LINUX 音频驱动实验 1191

48.1 音频接口简介 1192
48.1.1 为何需要音频编解码芯片？ 1192
48.1.2 CS42L51 简介 1192
48.1.3 I2S 总线接口 1193
48.1.4 STM32MP1 SAI 总线接口 1195
48.2 硬件原理图分析 1195
48.3 音频驱动使能 1196
48.3.1 修改设备树 1196
48.3.2 使能或修改内核的 CS42L51 驱动 1202
48.4 ALSA-LIB 和 ALSA-UTILS 移植 1205
48.5 声卡设置与测试 1206
48.5.1 amixer 使用方法 1206
48.5.2 音乐播放测试 1209
48.5.3 MIC 录音测试 1210
48.5.4 PHONE 录音测试 1211
48.6 开机自动配置声卡 1211
48.7 ALSAMIXER 简介 1212

第四十九章 LINUX CAN 驱动实验 1214

49.1 CAN 协议简析 1215
49.1.1 何为 CAN？ 1215
49.1.2 CAN 电气属性 1216
49.1.3 CAN 协议 1217
49.1.4 CAN 速率 1223
49.1.5 CAN FD 简介 1225
49.1.6 STM32MP1 FDCAN 简介 1226
49.2 硬件原理图分析 1227
49.3 实验程序编写 1228
49.3.1 修改设备树 1228
49.3.2 修复 m_can_platform.c 1229
49.3.3 使能 Linux 内核自带的 FDCAN 驱动 1229
49.4 CAN 测试 1231
49.4.1 检查 CAN 网卡设备是否存在 1231
49.4.2 移植 iproute2 和 can-utils 工具 1231
49.4.3 CAN 通信测试 1233
49.4.4 CAN 500K 收发异常处理方法 1235

第五十章 LINUX USB 驱动实验 1236

50.1 USB 接口简介 1237
50.1.1 什么是 USB？ 1237
50.1.2 USB 电气特性 1238
50.1.3 USB 拓扑结构 1239

50.1.4 什么是 USB OTG？ 1240
50.1.5 STM32MP1 USB 接口简介 1241
50.1.6 USB TYPEC 电气属性 1242
50.2 硬件原理图分析 1244
50.2.1 USB HUB 原理图分析 1245
50.2.2 USB OTG 原理图分析 1246
50.3 USB 协议简析 1247
50.3.1 USB 描述符 1247
50.3.2 USB 数据包类型 1250
50.3.3 USB 传输类型 1252
50.3.4 USB 枚举 1253
50.4 STM32MP1 USB HOST 驱动编写 1253
50.5 LINUX 内核自带 HOST 实验 1256
50.5.1 USB 鼠标键盘测试 1256
50.5.2 U 盘实验 1259
50.6 LINUX 内核自带 USB OTG 实验 1261
50.6.1 STUSB1600 设备树编写 1261
50.6.2 FUSB302 驱动移植 1265
50.6.3 OTG 主机实验 1267
50.6.4 OTG 从机实验 1267

第五十一章 LINUX 块设备驱动实验 1272

51.1 什么是块设备 1273
51.2 块设备驱动框架 1273
51.2.1 block_device 结构体 1273
51.2.2 gendisk 结构体 1275
51.2.3 block_device_operations 结构体 1277
51.2.4 块设备 I/O 请求过程 1278
51.3 使用请求队列实验 1285
51.3.1 实验程序编写 1285
51.3.2 运行测试 1293
51.4 不使用请求队列实验 1295
51.4.1 实验程序编写 1295
51.4.2 运行测试 1297

第五十二章 LINUX 网络设备驱动实验 1298

52.1 嵌入式网络简介 1299
52.1.1 嵌入式下的网络硬件接口 1299
52.1.2 MII/RMII、GMII/RGMII 接口 1301
52.1.3 MDIO 接口 1303
52.1.4 RJ45 接口 1303
52.1.5 STM32MP1 GMAC 接口简介 1304
52.2 PHY 芯片详解 1305
52.2.1 PHY 基础知识简介 1305

52.2.2 RTL8211F-CG 详解 1306
52.2.3 YT8511C 详解 1312
52.3 LINUX 内核网络驱动框架 1314
52.3.1 net_device 结构体 1314
52.3.2 net_device_ops 结构体 1320
52.3.3 sk_buff 结构体 1322
52.3.4 网络 NAPI 处理机制 1328
52.4 原理图介绍 1330
52.5 STM32MP1 网络驱动简介 1331
52.5.1 STM32MP1 网络外设设备树 1331
52.5.2 STM32MP1 网络驱动源码简析 1333
52.5.3 stmmac_netdev_ops 操作集 1346
52.5.4 Linux 内核 PHY 子系统与 MDIO 总线简析 1353
52.5.5 RTL8211F PHY 驱动 1360
52.5.6 YT8511 PHY 驱动 1362
52.6 网络驱动实验测试 1364
52.6.1 RTL8211F PHY 驱动测试 1364
52.6.2 YT8511C PHY 驱动测试 1364
52.6.3 千兆以太网网速测试 1365

第五十三章 LINUX WIFI 驱动实验 1368

53.1 硬件原理图分析 1369
53.2 WIFI 驱动添加与编译 1369
53.2.1 向 Linux 内核添加 WIFI 驱动 1370
53.2.2 配置 USB 支持设备 1371
53.2.3 设备树的配置 1373
53.2.4 编译 WIFI 驱动 1375
53.2.5 驱动加载测试 1376
53.3 WIFI 联网测试 1379
53.3.1 buildroot WIFI 工具配置 1379
53.3.2 WIFI 工具测试 1380
53.3.3 RTL8723 SDIO WIFI 联网测试 1382
53.3.4 RTL8188 USB WIFI 联网测试 1384

第五十四章 LINUX 4G 通信实验 1386

54.1 4G 网络连接简介 1387
54.2 高新兴 ME3630 4G 模块实验 1389
54.2.1 ME3630 4G 模块简介 1389
54.2.2 ME3630 4G 模块驱动修改 1390
54.2.3 ME3630 4G 模块 ppp 联网测试 1394
54.2.4 ME3630 4G 模块 ECM 联网测试 1399
54.2.5 ME3630 4G 模块 GNSS 定位测试 1401
54.3 EC20 4G 模块实验 1402
54.3.1 EC20 4G 模块简介 1402

54.3.2 EC20 4G 模块驱动修改 1403
54.3.3 EC20 ppp 拨号上网 1406
54.3.4 移远 GobiNET 驱动移植 1409
54.3.4 EC20 上网测试 1410

第五十五章 REGMAP API 实验 1413

55.1 REGMAP API 简介 1414
55.1.1 什么是 Regmap 1414
55.1.2 Regmap 驱动框架 1414
55.1.3 Regmap 操作函数 1419
55.1.4 regmap_config 掩码设置 1421
55.2 实验程序编写 1422
55.3 运行测试 1426

第五十六章 LINUX IIO 驱动实验 1428

56.1 IIO 子系统简介 1429
56.1.1 iio_dev 1429
56.1.2 iio_info 1432
56.1.3 iio_chan_spec 1434
56.2 IIO 驱动框架创建 1437
56.2.1 基础驱动框架建立 1437
56.2.2 IIO 设备申请与初始化 1439
56.3 实验程序编写 1443
56.3.1 ICM20608 的 IIO 驱动框架搭建 1443
56.3.2 完善 icm20608_read_raw 函数 1455
56.3.3 完善 icm20608_write_raw 函数 1462
56.4 测试应用程序编写 1466
56.4.1 linux 文件流读取 1466
56.4.2 编写测试 APP 1468
56.4.3 运行测试 1473
56.5 IIO 触发缓冲区 1474
56.5.1 IIO 触发器 1474
56.5.2 IIO 缓冲区 1476
56.6 向驱动程序添加触发缓冲功能 1477
56.6.1 修改设备树 1477
56.6.2 驱动程序编写 1477
56.7 触发缓冲测试 1481
56.7.1 缓冲区接口文件 1482
56.7.2 触发器接口文件 1483
56.7.3 缓冲区数据读取 1484
56.7.4 触发缓冲测试 APP 编写测试 1485

第五十七章 LINUX ADC 驱动实验 1492

57.1 ADC 简介 1493

57.1.1 什么是 ADC *1493*
57.1.2 STM32MP157 ADC 简介 *1493*
57.2 ADC 驱动源码简析 1494
57.2.1 设备树下的 ADC 节点 *1494*
57.2.2 ADC 驱动源码分析 *1496*
57.3 硬件原理图分析 1501
57.4 ADC 驱动编写 1501
57.4.1 修改设备树 *1501*
57.4.2 使能 ADC 驱动 *1502*
57.4.3 编写测试 APP *1503*
57.5 运行测试 1507
57.5.1 编译驱动程序和测试 APP *1507*
57.5.2 运行测试 *1507*
第五十八章 LINUX DAC 驱动实验 1509
58.1 DAC 简介 1510
58.2 DAC 驱动源码分析 1511
58.2.1 设备树下的 DAC 节点 *1511*
58.2.2 DAC 驱动源码分析 *1513*
58.3 硬件原理图分析 1515
58.4 DAC 驱动编写 1516
58.4.1 修改设备树 *1516*
58.4.2 使能 DAC 驱动 *1517*
58.4.3 编写测试 APP *1517*
58.5 运行测试 1524
58.5.1 编译驱动程序和测试 APP *1524*
58.5.2 运行测试 *1524*
第五十九章 LINUX 单总线驱动实验 1526
59.1 DS18B20 及工作时序简介 1527
59.1.1 DS18B20 简介 *1527*
59.1.2 DS18B20 时序简介 *1528*
59.2 DHT11 及工作时序简介 1529
59.2.1 DHT11 简介 *1529*
59.2.2 DHT11 工作时序简介 *1530*
59.3 硬件原理图分析 1531
59.4 DS18B20 实验编写 1532
59.4.1 设备树的配置 *1532*
59.4.2 ds18b20 驱动编写 *1533*
59.4.3 编写测试 APP *1541*
59.5 DHT11 实验编写 1542
59.5.1 设备树的配置 *1542*
59.5.2 dht11 驱动编写 *1542*
59.5.3 编写测试 APP *1545*

59.6 DS18B20 运行测试 1546
59.6.1 编译驱动程序和 APP 1546
59.6.2 运行测试 1546
59.7 DHT11 运行测试 1547
59.7.1 编译驱动程序和 APP 1547
59.7.2 运行测试 1547

前言

当今社会是一个电子信息技术飞速发展的年代，小到玩具，家电，大到手机、飞机、潜艇等，都离不开电子信息技术。社会上对于电子信息方面的人才需求也很大，从就业的角度而言电子、计算机、通信以及信息方面专业的毕业生工资都比较高，就业也比较容易，尤其是嵌入式、Linux 和 Android 等相关开发工作。

本书主要讲解嵌入式 Linux 中的驱动开发，相信大部分读者和作者一样，以前都是做单片机开发的工作，比如 51 或者 STM32 等。单片机开发很难接触到更高层次的系统方面的知识，单片机用到的系统都很简单，比如 UCOS、FreeRTOS 等等，这些操作系统都是一个 kernel，如果需要网络、文件系统、GUI 等这些就需要开发者自行移植。而移植又是非常痛苦的一件事情，而且移植完成以后的稳定性也无法保证。即使移植成功以后后续的开发工作也比较繁琐，因为不同的组件其 API 操作函数都不同，没有一个统一的标准，使用起来的话学习成本比较高。这个时候一个功能完善的操作系统显得尤为重要：具有统一的标准；提供完善的多任务管理、存储管理、设备管理、文件管理和网络等。Linux 就是这样一个系统，当然这样的系统还有很多，比如 Windows，MacOS，UNIX 等等。本书我们讲解 Linux，而 Linux 开发可以分为底层驱动开发和应用开发，本书讲解的是 Linux 的驱动开发，主要面向与那些此前使用 STM32 的开发者。平心而论，如果此前只会 51 单片机开发的话我是非常不建议直接上手 Linux 驱动开发的，因为 51 和 Linux 驱动开发的差异太大了！笔者建议在学习嵌入式 Linux 驱动开发之前一定要学一下 STM32 这种 Cortex-M 内核的 MCU，因为 STM32 这样的 MCU 其内部资源基本和可以运行 Linux 的 CPU 差不多，如果会 STM32 的话上手 Linux 驱动开发就会容易很多。笔者就是此前做了 4 年 STM32 开发工作，然后转的 Linux 驱动开发，整个过程比较顺畅。

鉴于当前 STM32 非常火爆，学习者众多，如何帮助 STM32 学习者顺利的转入 Linux 驱动开发是笔者思考了很久的问题。为此笔者本书和相应例程的安排如下：

1、选取合适的 CPU

ST 在 2019 年推出了自己首款 Cortex-A 内核 MPU:STM32MP1 系列，对于学习过 STM32 单片机的朋友来说，STM32MP1 是最适合用来学习嵌入式 Linux，因为大家对 STM32 已经很熟悉了，所以在学习的时候就免去了了解芯片这个过程。另外，Linux 驱动开发入门的 CPU 一定不能复杂!!!比如像三星的 Exynos 4412、Exynos 4418 等这些手机 CPU，这些 CPU 性能很强大，带有 GPU、支持硬件视频解码、可以运行 Andriod。但是正是它们的性能过于强大，功能过于繁杂，所以不适合 Linux 驱动开发入门。一款外设和 STM32H7 这样的 MCU 差不多的 CPU 就非常适合 Linux 入门，STM32MP1 可以看做是换核版的 STM32H7，外设资源大部分和 STM32H7 一样，兼容 STM32 单片机生态，如果此前学习过 STM32 的话会非常容易上手 STM32MP1。而且目前 STM32MP1 正在大量出货，是以前大量使用三星 2440、6410 做的产品更新换代的绝佳之选，学习完 STM32MP1 以后工作就可以直接使用了。本书选取开发平台为正点原子的 STM32MP157 发板，其他厂商的 STM32MP157 开发板也可以参考本书。

2、开发环境讲解

STM32 单片机的开发都是在 Windows 系统下进行的，使用 MDK 或者 IAR 这样的集成 IDE，STM32MP1 系列内部的 M4 内核可以在 Windows 系统下使用 MDK、IAR 这样的集成 IDE 开发。但是 STM32MP1 的 Linux 开发需要的主机是 Linux 平台的，也就是你必须先在自己的电脑上安装 Linux 系统，Linux 系统发行版有 Ubuntu、CentOS、Fedora、Debian 等等，本书我们使用 Ubuntu 操作系统。本书假设大家此前从来没有接触过 Ubuntu 操作系统，因此会有详细的 Ubuntu 操作系统安装、使用教程，帮助大家熟悉开发环境。

3、Uboot、Linux 和根文件系统移植

在进行 Linux 驱动开发之前要先在所使用的开发板平台上移植好 Uboot，Linux 和根文件系统。这是 Linux 驱动开发的第一个拦路虎，因此本书和相应的视频会着重讲解 Uboot/Linux 和根文件系统的移植。

4、嵌入式 Linux 驱动开发

当我们把 Uboot、Linux 和根文件系统都在开发板上移植好了以后就可以开始 Linux 驱动开发了。Linux 驱动有三大类:字符设备驱动、块设备驱动和网络设备驱动，这三大类我们都会详细的讲解，并且配有数十个相应的例程，由简入深，从最简单的点灯，到最后的网络设备驱动。

本书一共分三篇，每篇对应一个不同的阶段：

第一篇：Ubuntu 操作系统入门

本篇主要讲解 Ubuntu 操作系统的使用，本篇不涉及到任何嵌入式方面的知识，全部是在 PC 上完成的，只要安装好 Ubuntu 操作系统即可。

第二篇：Uboot、Linux 和根文件系统移植

本篇讲解如何将 Uboot、Linux 和根文件系统移植到我们的开发板上，为后面的 Linux 驱动开发做准备。

第三篇： Linux 驱动开发

前面做了那么多的工作就是为了本篇，因此本篇注定了将是本书重中之重，大家应该投入精力最多的一篇，望大家做好准备。

通过上面四篇的学习，大家基本掌握了嵌入式 Linux 驱动的开发流程，本书旨在引导大家入门 Linux 驱动开发，更加深入的研究就需要大家自行查阅其他更加专业的书籍了，祝愿大家学习顺利。

第一篇 Ubuntu 系统入门篇

本篇主要讲解 Ubuntu 系统，包括如何在虚拟机上安装 Ubuntu 操作系统，安装好以后 Ubuntu 的设置、基本操作等等。在正式进行嵌入式开发之前肯定是要先学会 Ubuntu 系统如何使用的，由于 Ubuntu 系统功能很庞大，如果要详细讲解的话一本书都讲不完，因此本篇只做 Ubuntu 系统入门讲解，掌握我们进行嵌入式开发所需的技能即可，如果想详细的学习 Ubuntu 系统的话可以参考其他的书籍，比如经典的《鸟哥的 linux 私房菜》，《鸟哥的 linux 私房菜》这本书使用的 CentOS 操作系统，但是 Ubuntu 下完全可以使用。

当 Ubuntu 系统入门以后，我们重点要学的就是如何在 Linux 下进行 C 语言开发，如何使用 gcc 编译器、如何编写 Makefile 文件等等。

如果此前已经使用过 Ubuntu 操作系统，并且从事过 Linux C 编程工作的话本篇就不需要看了，可以直接跳到第二篇，开始 ARM 裸机开发篇的学习。

第一章 Ubuntu 系统安装

Linux 的开发需要在 Linux 系统下进行，这就要求我们的 PC 主机安装 Linux 系统，本书我们选择 Ubuntu 这个 Linux 发行版系统。本章讲解如何安装虚拟机，以及如何在虚拟机中安装 Ubuntu 系统，安装完成以后如何做简单的设置。如果已经对于虚拟机以及 Ubuntu 基础操作已经熟悉的话就可以跳过本章。

1.1 安装虚拟机软件 VMware

不是安装 Ubuntu 吗？怎么要先安装虚拟机呢？虚拟机是个啥？相信大部分第一次安装 Ubuntu 的人都会有这个疑问。我不能直接安装 Ubuntu 吗？能不能不要虚拟机呢？答案是肯定可以的！直接在电脑上安装 Ubuntu 以后你的电脑就是一个真真正正的 Ubuntu 电脑了，你可以再安装一个 Windows 系统，这样你的电脑就是双系统了，在开机的时候可以选择不同的系统启动。但是这样的话会有一个问题，那就是你每次只能选择其中的一个系统启动，要么 Windows 要么 Ubuntu，但是我们再开发的时候很多时候需要在 Windows 和 Ubuntu 下来回切换，Windows 系统下的软件资源要比 Ubuntu 下丰富的多，这个就涉及到两个系统切换问题，显然如果你直接在电脑上安装 Ubuntu 以后就没法做到，因为你每次开机只能在 Windows 和 Ubuntu 下二选一。如果 Ubuntu 系统能作为 Windows 下的一个软件就好了，我们默认启动 Windows 系统，需要用到 Ubuntu 的话直接打开这个软件就行了。这个当然是可以的！这里就要借助虚拟机了，虚拟机顾名思义就是虚拟出一个机器，然后你就可以在这个机器上安装任何你想要的系统，相当于再克隆出一个你的电脑，这样在主机上运行 Windows 系统，当我们需要用到 Ubuntu 的话就打开安装有 Ubuntu 系统的虚拟机。

虚拟机的实现我们可以借助其他软件，比如 Vmware Workstation，Vmware Workstation 是收费软件，免费的虚拟机软件有 Virtualbox。本书我们使用 Vmware Workstation 软件来做虚拟机。Vmware Workstation 软件可以在 Wmware 官网下载，下载地址：https://www.vmware.com/products/workstation-pro/workstation-pro-evaluation.html，当前最新的版本是 Vmware Workstation Pro 15，我们下载 Windows 版本的，如图 1.1.1 所示：

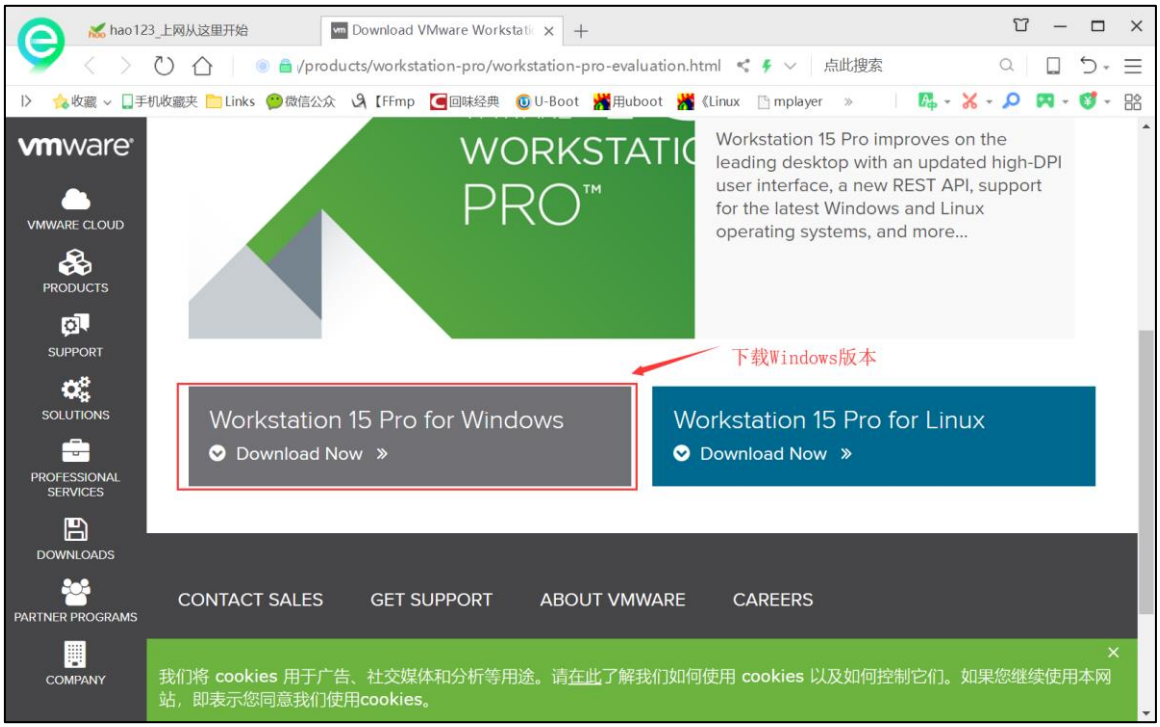


图 1.1.1 Vmware 下载页

我们已经在开发板光盘里面提供了 Vmware Workstation 软件，大家可以直接使用，在光盘目录：开发板光盘→3、软件→VMware-workstation-full-15.5.0-14665864.exe。WMware Workstation 的安装和普通软件安装一样，双击 VMware-workstation-full-15.5.0-14665864.exe 进入安装界面，如图 1.1.2 所示：

图 1.1.1 VMware 安装界面

点击图 1.1.2 中的“下一步”，进入图 1.1.3 所示步骤：

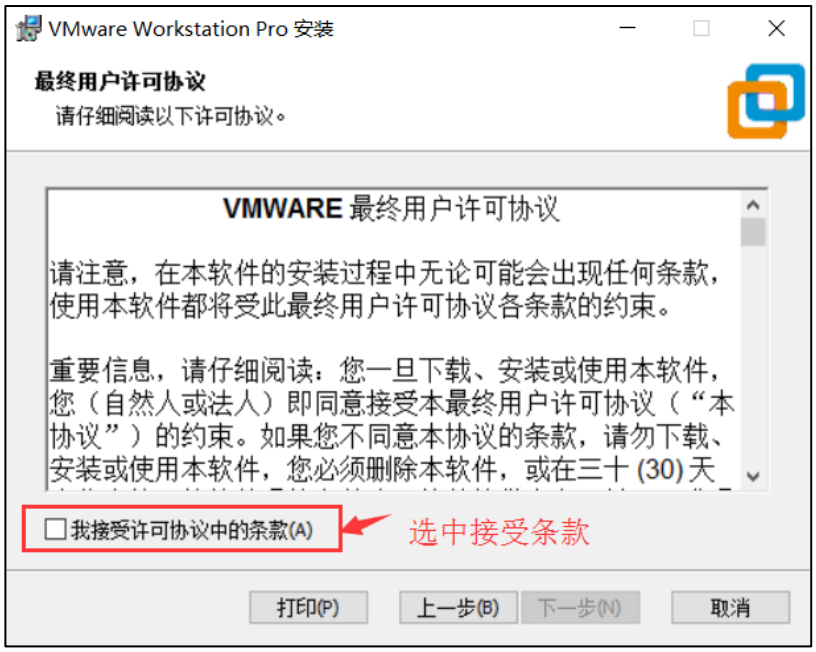


图 1.1.3 VMware 条款

先选择图 1.1.3 中的“我接受许可协议中的条款”，然后在选择“下一步”，进入图 1.1.4 所示步骤：

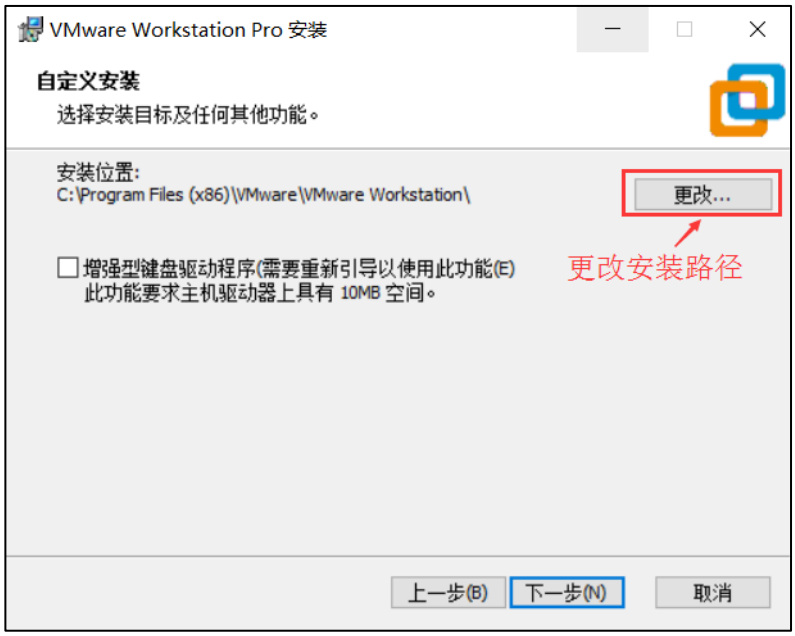


图 1.1.4 选择安装路径

图 1.1.4 中选择软件的安装路径，点击“更改”按钮，然后根据自己的实际需要选择合适路径即可,我的安装路径如图 1.1.5 所示：

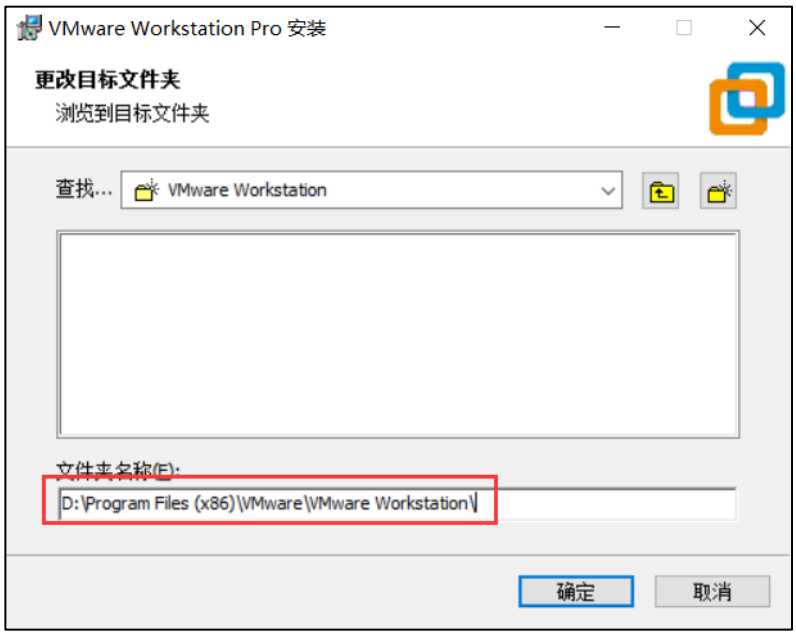


图 1.1.5 安装路径

选择好路径以后点击图 1.1.5 中的“确定”按钮，然后回到图 1.1.4 所示界面，点击图 1.1.4 中的“下一步”，进入图 1.1.6 所示界面：

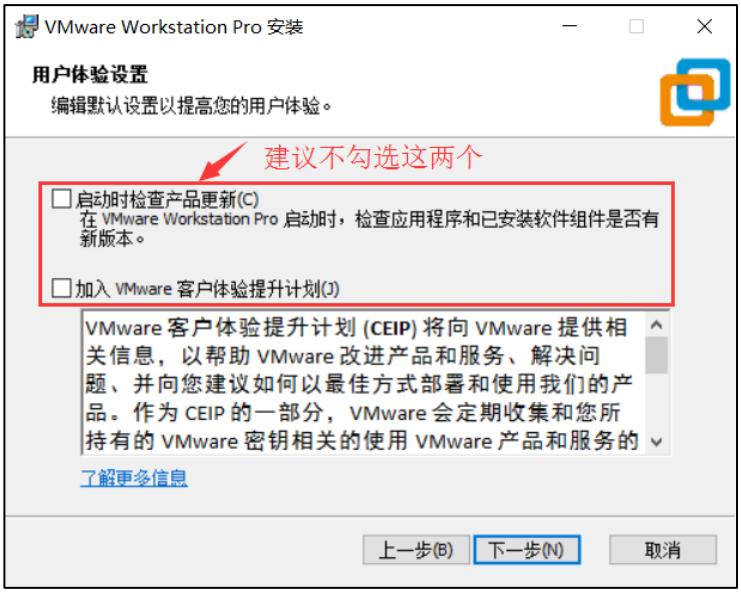


图 1.1.6 检查更新界面

在图 1.1.6 中，会有两个复选框，默认都是选中的，建议不要选中！然后点击图 1.1.6 中的“下一步”按钮，进入图 1.1.7 所示界面：

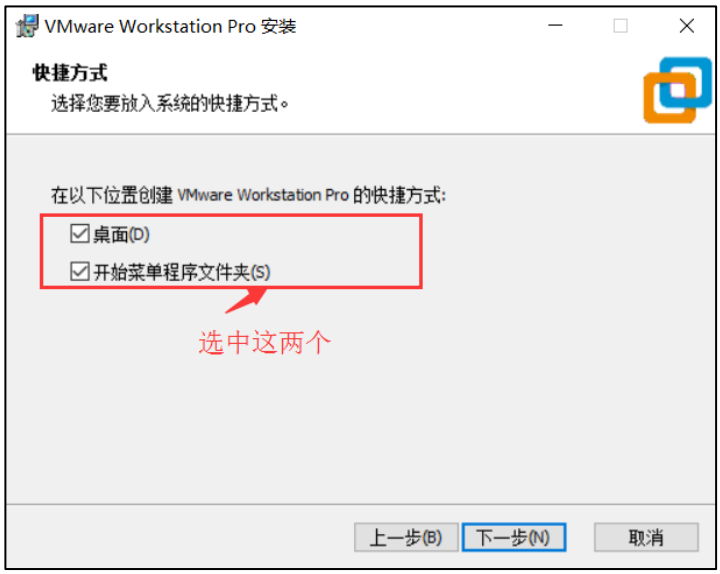


图 1.1.7 快捷方式设置

在图 1.1.7 中有两个选项，我们都选中，这样在安装完成以后就会在开始菜单和桌面上有 VMware 的图标，选中以后点击图 1.1.7 中的“下一步”，进入图 1.1.8 界面：

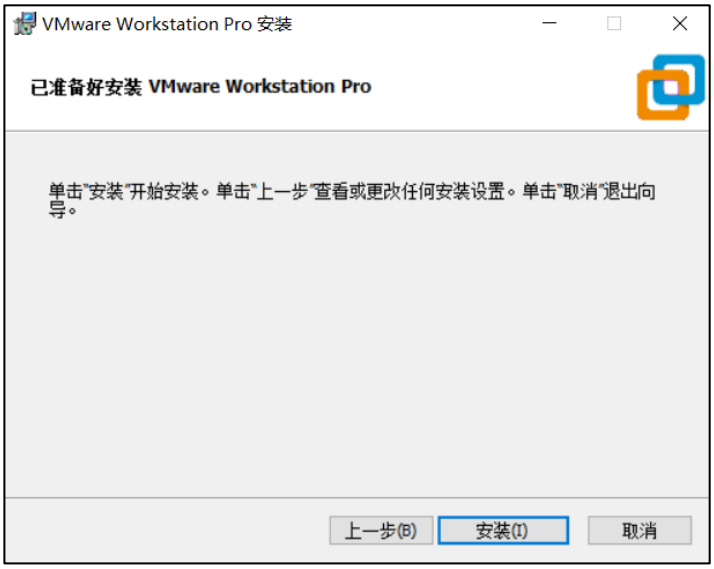


图 1.1.8 安装确定界面

前面几步已经设置好安装参数了，如果不需要修改安装参数的话就点击图 1.1.8 中的“安装”按钮开始安装 VMware，安装过程如图 1.1.9 所示：

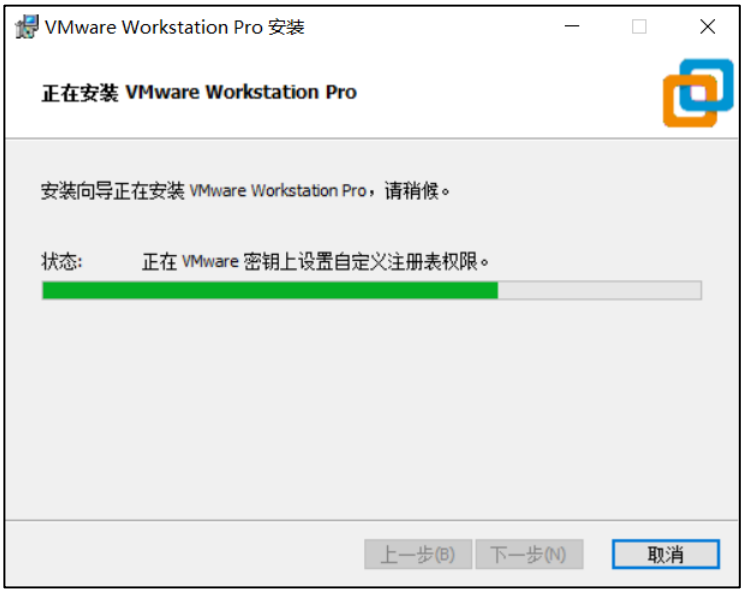


图 1.1.9 安装进行中

图 1.1.9 就是安装过程，耐心等待几分钟，等待安装完成，安装完成以后会有如图 1.1.10 所示提示：

图 1.1.10 安装完成

点击图 1.1.10 中的“完成”按钮，完成 VMware 的安装，安装完成以后就会在桌面上出现 VMware Workstation Pro 的图标，如图 1.1.11 所示：

图 1.1.11 VMware 桌面图标

双击图 1.1.11 中的图标打开 VMware 软件，在第一次打开软件的时候会提示你输入许可证密钥，如图 1.1.12 所示：

图 1.1.12 输入许可证密钥

前面说了 VMware 是付费软件，是需要购买的，如果你购买了 VMware 的话就会有一串许可密钥，如果没有购买的话就选择“我希望试用 VMware Workstation 15 30 天”选项，这样你就可以体验 30 天 VMware。输入密钥以后点击“继续按钮”，如果你的密钥正确的话就会提示你购买成功，如图 1.1.13 所示：

图 1.1.13 购买 VMware 成功

点击图 1.1.13 中的“完成”按钮，VMware 软件正式打开，界面如图 1.1.14 所示：

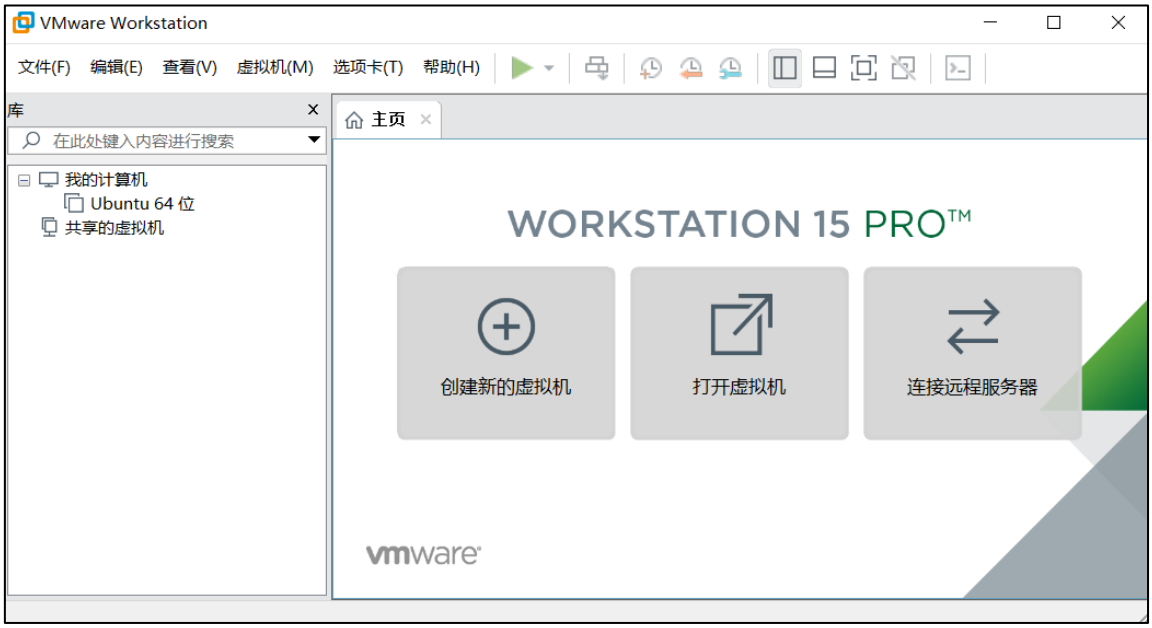


图 1.1.15 VMware Workstation 主界面

至此，虚拟机软件 VMware 安装成功。

1.2 创建虚拟机

安装好 VMware 以后我们就可以在 VMware 上创建一个虚拟机，打开 VMware，选择：文

件->新建虚拟机，如图 1.2.1 所示：

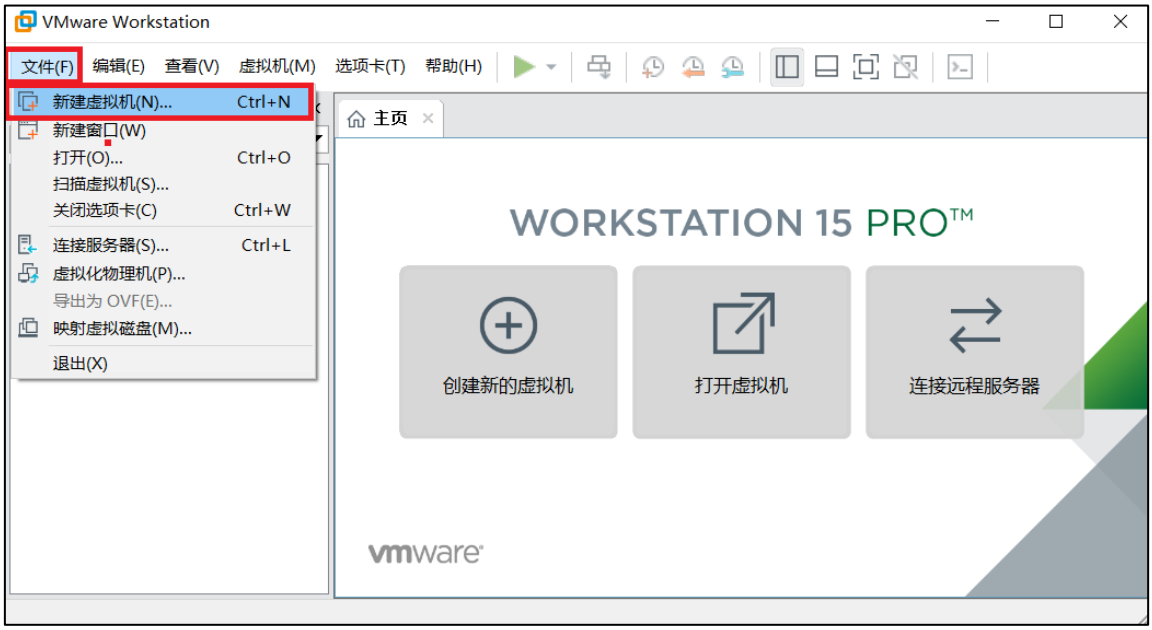


图 1.2.1 新建虚拟机

打开图 1.2.2 所示创建虚拟机向导界面：

图 1.2.2 创建虚拟机向导

选中图 1.2.2 中的“自定义”选项，然后选择“下一步”，进入图 1.2.3 所示硬件兼容性选择界面：

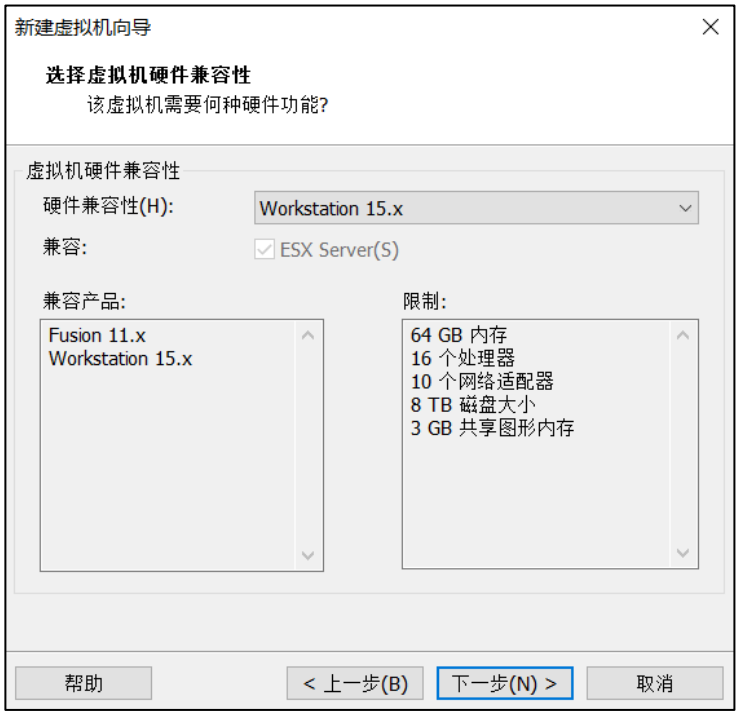


图 1.2.3 硬件兼容性选择

在图 1.2.3 中我们使用默认值就行了，直接点击“下一步”，进入图 1.2.4 所示的操作系统安装界面：

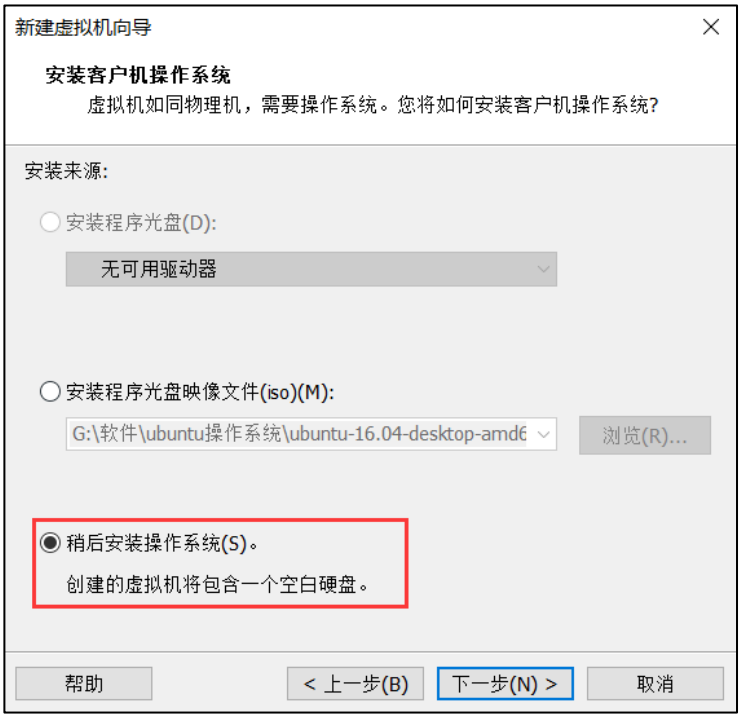


图 1.2.4 安装客户机操作系统

图 1.2.4 就是选择你新创建的虚拟机要安装什么系统？windows 还是 linux，如果你要现在就安装系统的话需要准备好系统文件，一般是.iso 文件。我们现在不安装系统，因此选择“稍后安装操作系统(S)”这个选项，然后选择“下一步”，进入图 1.2.5 所示界面：

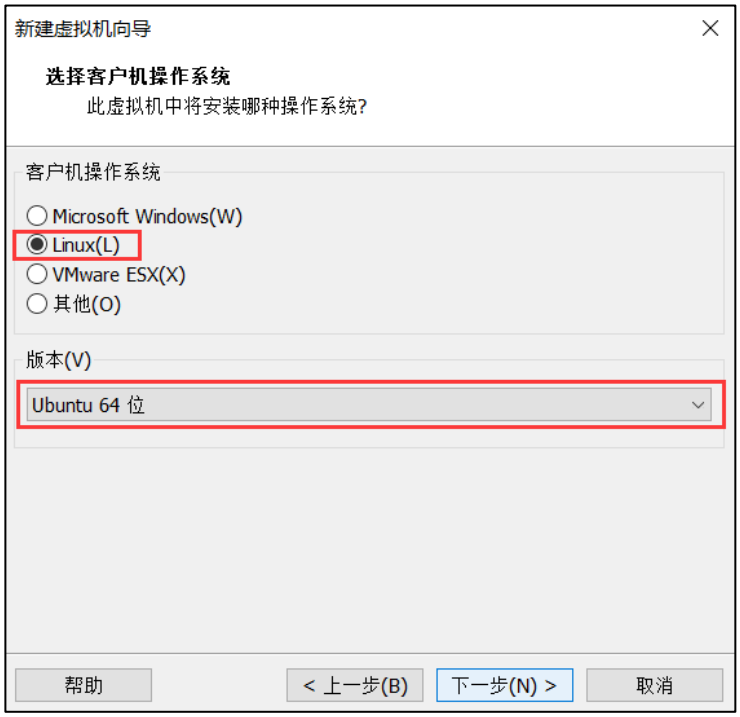


图 1.2.5 客户机操作系统选择

图 1.2.5 中依旧是让你选择你要在虚拟机中装什么系统，图 1.2.5 是和图 1.2.4 配合在一起使用的，在图 1.2.4 中放入系统文件(.iso 文件)，然后在图 1.2.5 中选择你放入的是什么系统，然后 VMware 就会稍后自动安装所设置的系统。在图 1.2.4 中我们没有设置系统文件，因此图 1.2.5 是没用的，不过我们还是在图 1.2.5 中的客户机操作系统一栏选择“Linux”，版本选择 Ubuntu 64 位，然后点击“下一步”，进入图 1.2.6 所示界面：

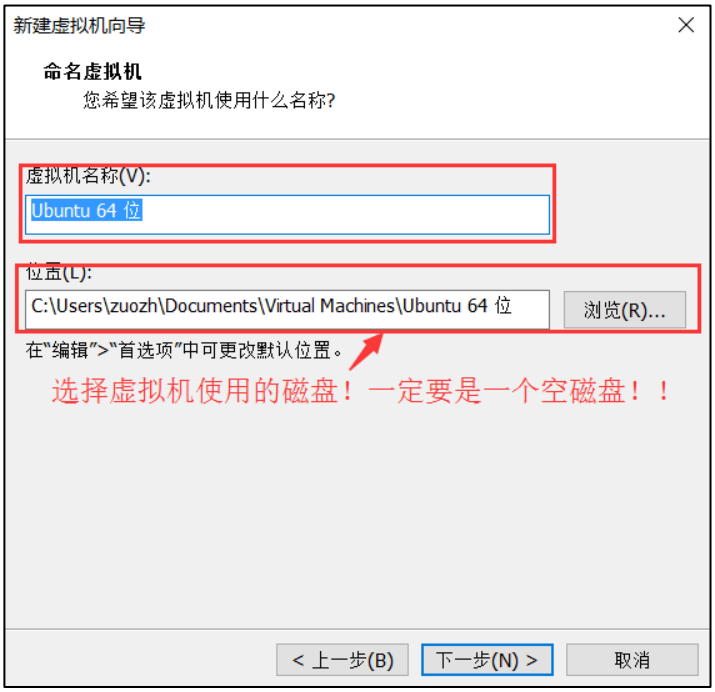


图 1.2.6 命名虚拟机

图 1.2.6 中上面是设置虚拟机名字，大家可以根据自己的使用习惯给虚拟机命名，重点是下

面的虚拟机位置选择！我们要给虚拟机单独清理出一块磁盘，做嵌入式开发建议这块空磁盘的大小不小于 100GB，比如我清理除了一个 196GB 的 I 盘给虚拟机使用，如图 1.2.7 所示：

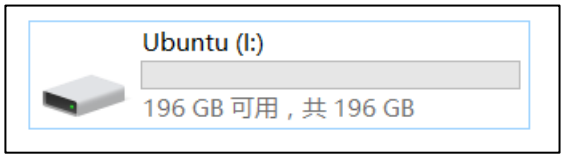


图 1.2.7 虚拟机所使用的磁盘

清理出虚拟机专用的磁盘以后然后就在图 1.2.6 中的位置出选择这个磁盘，比如我的位置选择如图 1.2.8 所示：

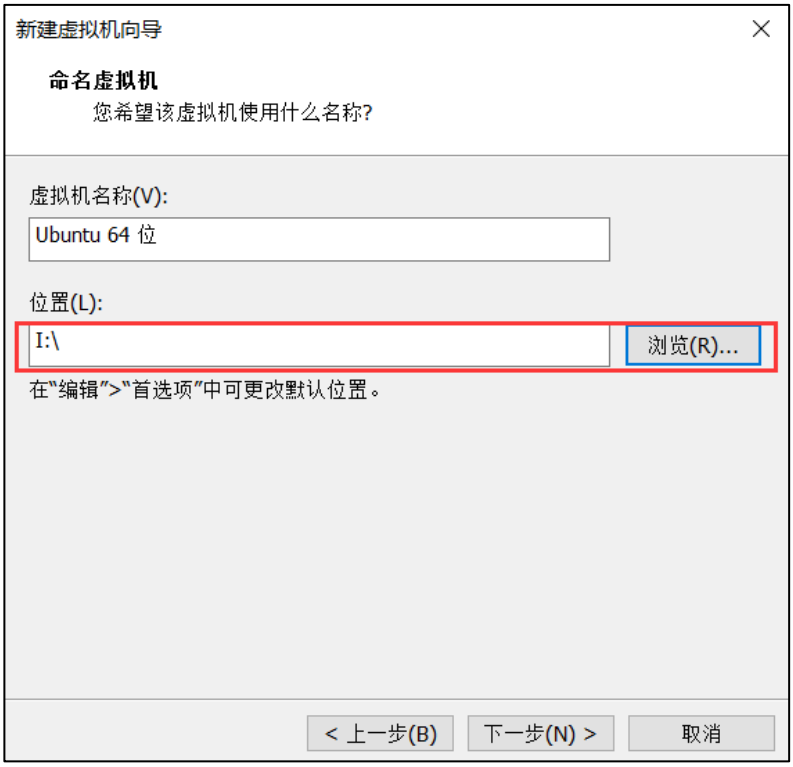


图 1.2.8 选择虚拟机磁盘位置

设置好图 1.2.8 中的虚拟机磁盘位置以后点击“下一步”，进入如图 1.2.9 所示的处理器配置选择界面：

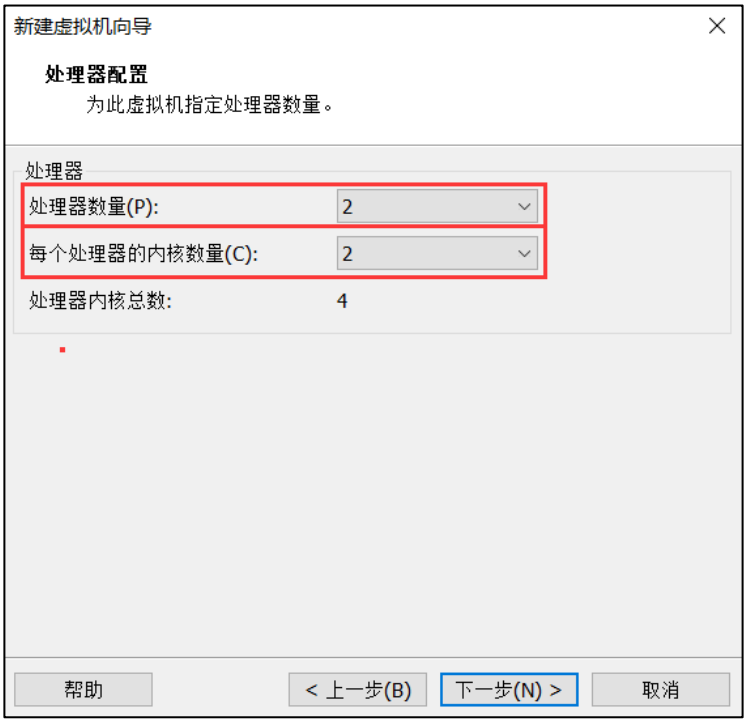


图 1.2.9 处理器配置界面

图 1.2.9 中就是配置你的虚拟机所使用的处理器数量，以及每个处理器的内核数量，这个要根据自己实际使用的电脑 CPU 配置来设置。比如我的电脑 CPU 是 I7-4720HQ，这是个 4 核 8 线程的 CPU，因此我就可以分 2 个核给 VMware，然后 I7-4720HQ 每个物理核有两个逻辑核，因此每个处理器的内核数量就是 2，所以的 VMware 虚拟机配置就如图 1.2.9 所示，大家根据自己的实际电脑 CPU 配置来设置即可，设置好以后点击“下一步”，进入图 1.2.10 所示内存配置界面：

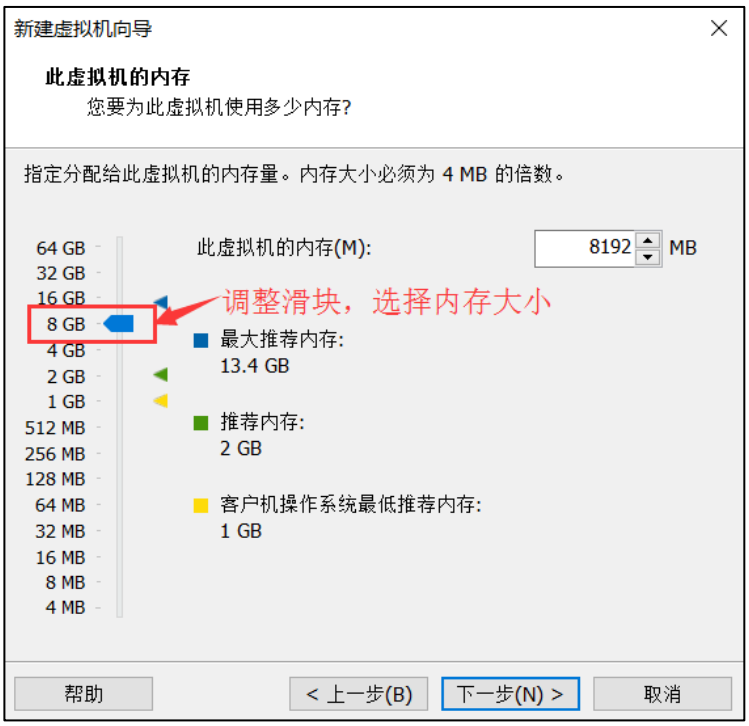


图 1.2.10 内存配置

同样的在图 1.2.10 中根据自己电脑的实际内存配置来设置分给虚拟机的内存大小，比如我的电脑是 16GB 的内存，因此我可以给虚拟机分配 8GB 的内存。配置好虚拟机的内存大小以后点击“下一步”，进入图 1.2.11 所示的网络类型选择界面：

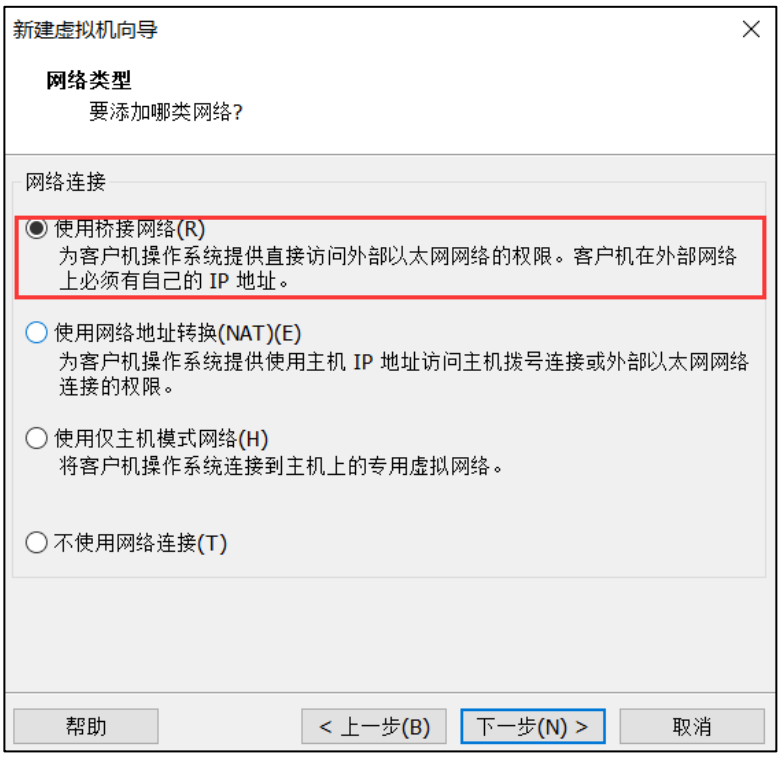


图 1.2.11 网络类型选择界面

在图 1.2.11 中我们选择“使用桥接网络”，然后点击“下一步”，进入图 1.2.12 示的选择 I/O 控制器类型界面：

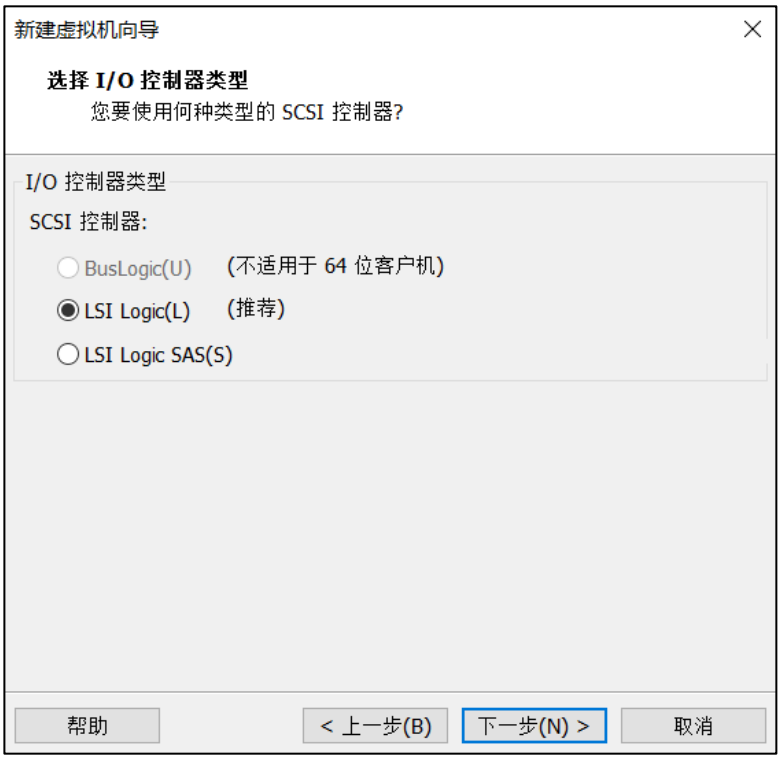


图 1.2.12 I/O 控制器选择

I/O 控制器类型选择默认值就行，也就是“LSI Logic”，然后点击“下一步”，进入磁盘类型选择界面，如图 1.2.13 所示：

图 1.2.13 磁盘类型选择

图 1.2.13 中选择磁盘类型，使用默认值“SCSI”即可，然后点击“下一步”，进入选择磁盘

界面，如图 1.2.14 所示：

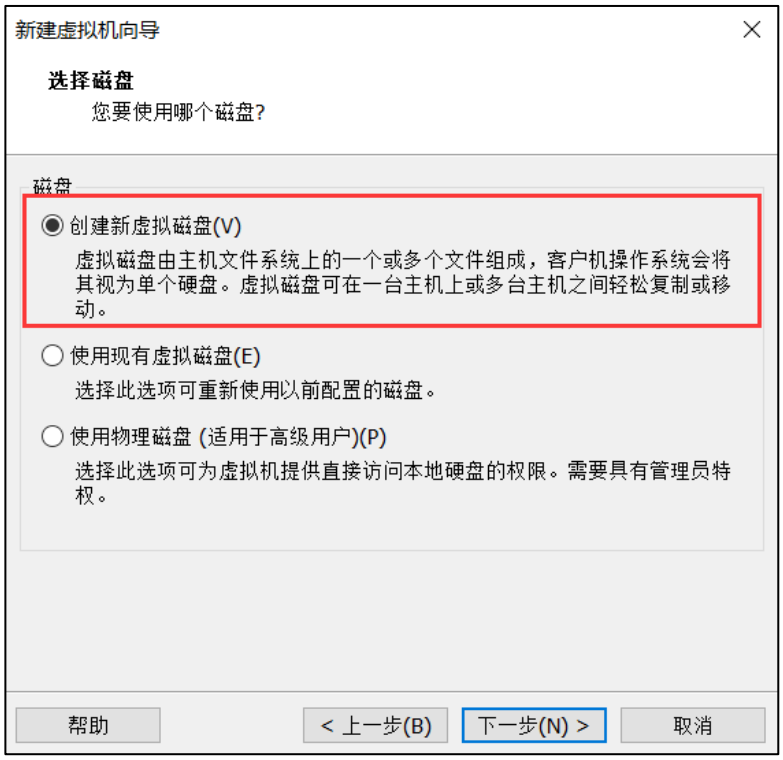


图 1.2.14 磁盘选择

图 1.2.14 中使用默认值，即“创建新虚拟磁盘”，这样我们前面设置好的那个空的磁盘就会被创建为一个新的磁盘，设置要以后点击“下一步”，进入磁盘容量设置界面，如图 1.2.15 所示：

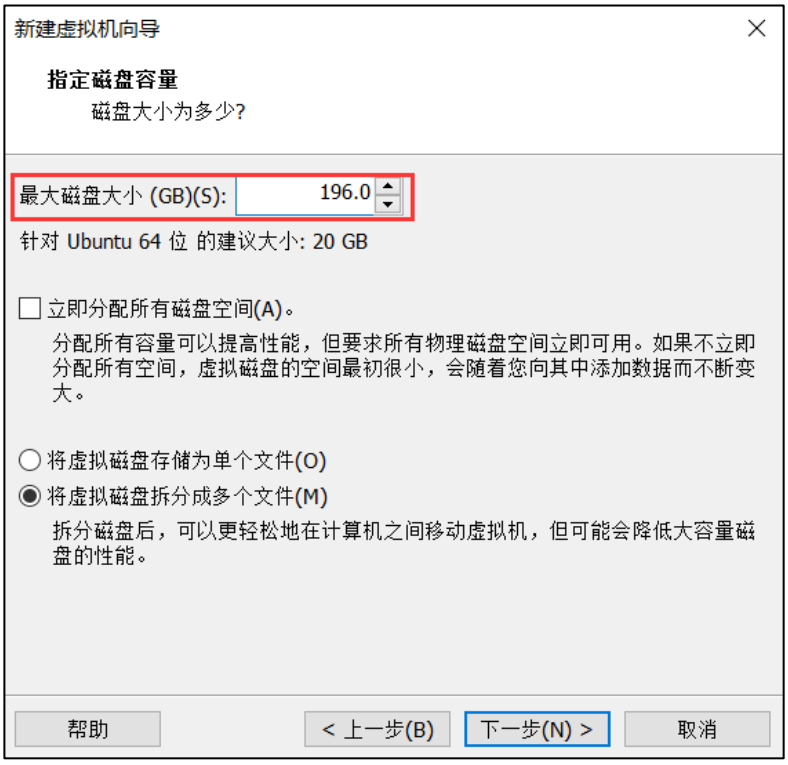


图 1.2.15 磁盘容量设置

图 1.2.15 是用来设置我们清出的空的磁盘多少是给虚拟机用的，我们清出了一个空磁盘肯定是全部给虚拟机用的，因此设置最大磁盘大小为空磁盘的大小，比如图 1.2.7 中我的那个 I 盘是 196GB 的，因此图 1.2.15 中就设置最大磁盘大小为 196GB，然后点击“下一步”，进入图 1.2.16 所示界面指定磁盘文件，

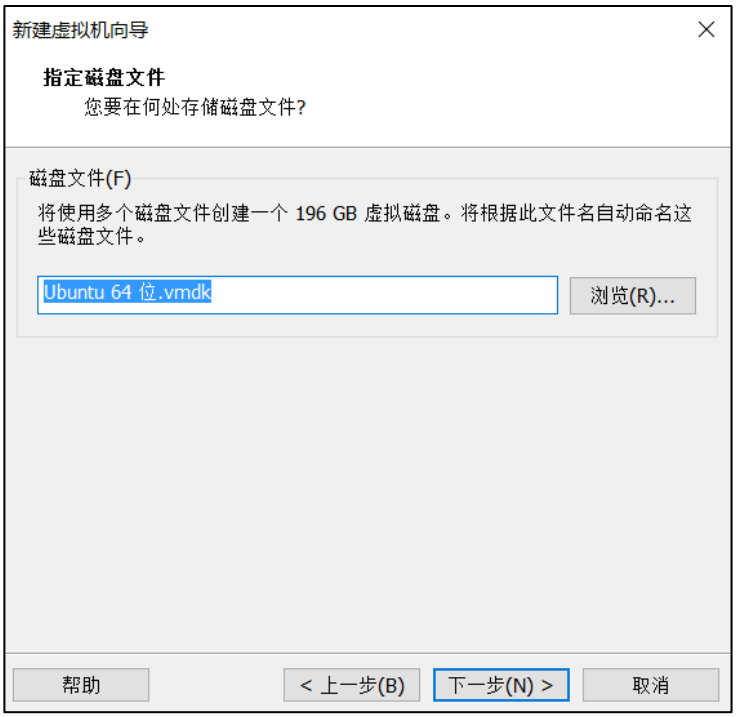


图 1.2.16 指定磁盘文件

图 1.2.16 使用默认设置，不要做任何修改，直接点击“下一步”，进入已准备好创建虚拟机界面，如图 1.2.17 所示：

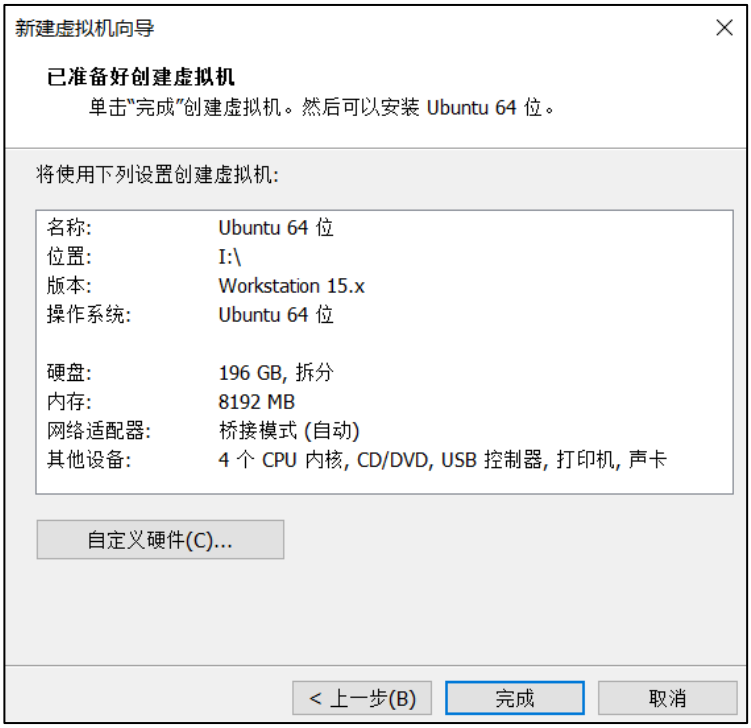


图 1.2.17 准备创建虚拟机

在图 1.2.17 中确认自己的虚拟机配置，如果确认无误就点击“完成”，如果有误的话就返回有误的配置界面做修改，点击“完成”按钮以后就会创建一个虚拟机，如图 1.2.18 所示：

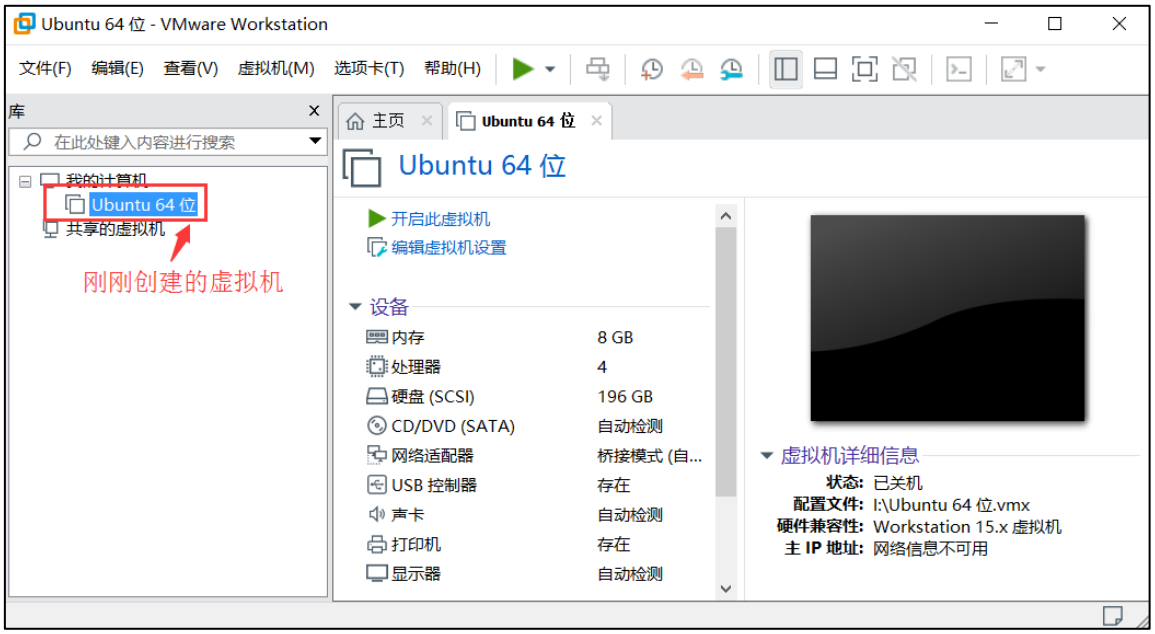


图 1.2.18 新创建的虚拟机

创建虚拟机成功以后就会在右侧的：我的计算机下出现刚刚创建的虚拟机“Ubuntu 64 位”，点击一下就会在右侧打开这个虚拟机的详细信息，如图 1.2.19 所示：

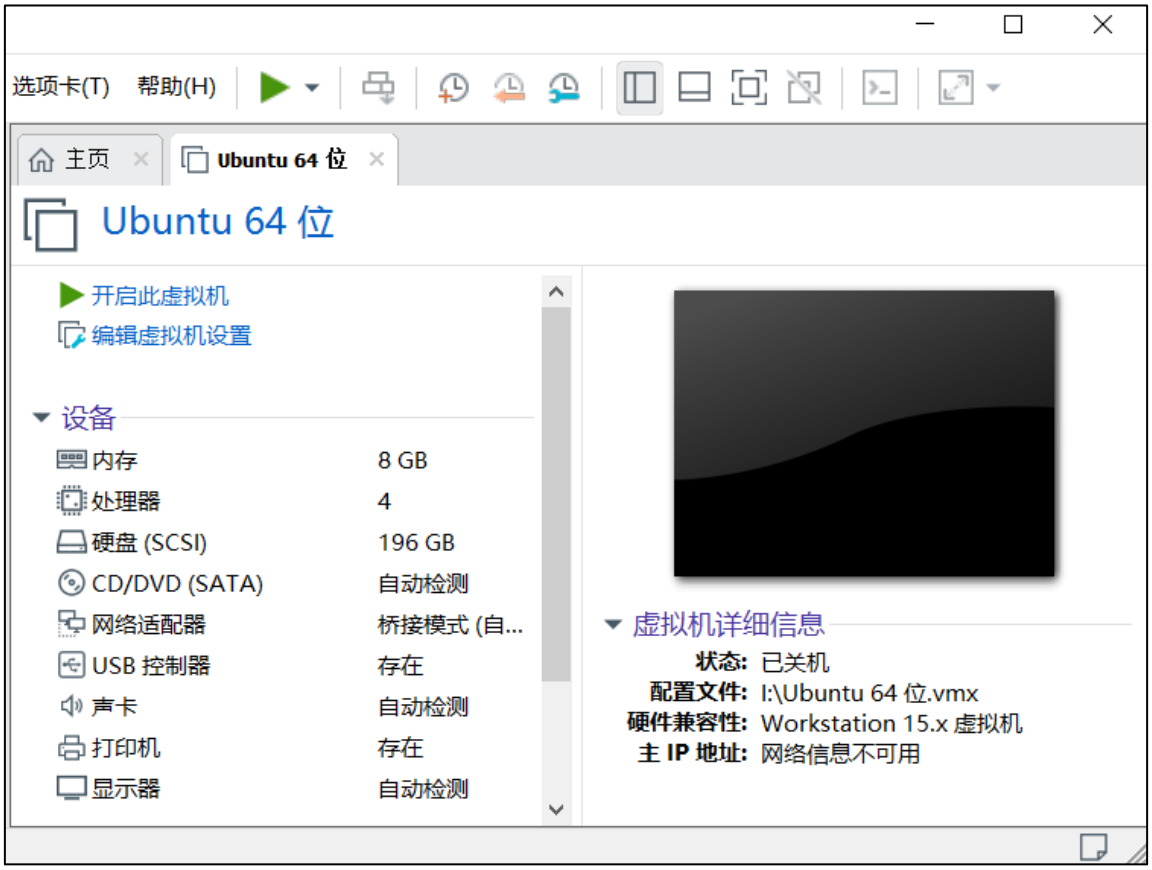


图 1.2.19 新建虚拟机配置信息

在图 1.2.19 中的设备一栏我们可以看到虚拟机详细的配置信息，图 1.2.20 所示示的两个按钮就是虚拟机的开关，

图 1.2.20 虚拟机开关机

图 1.2.20 中的这两个绿色三角按钮都可以打开虚拟机，但是此时虚拟机没有安装任何操作系统，因此没法打开，接下来我们就是要在刚刚新建的这个虚拟机中安装 Ubuntu 操作系统。

1.3 安装 Ubuntu 操作系统

1.3.1 获取 Ubuntu 系统

前面虚拟机已经创建成功了，相当于硬件已经准备好了，接下来就是要在虚拟机中安装 Ubuntu 系统了，首先肯定是获取到 Ubuntu 的系统镜像，Ubuntu 系统镜像肯定是在 Ubuntu 官网获取，下载地址为：https://www.ubuntu.com/download/desktop，如图 1.3.1.1 所示：

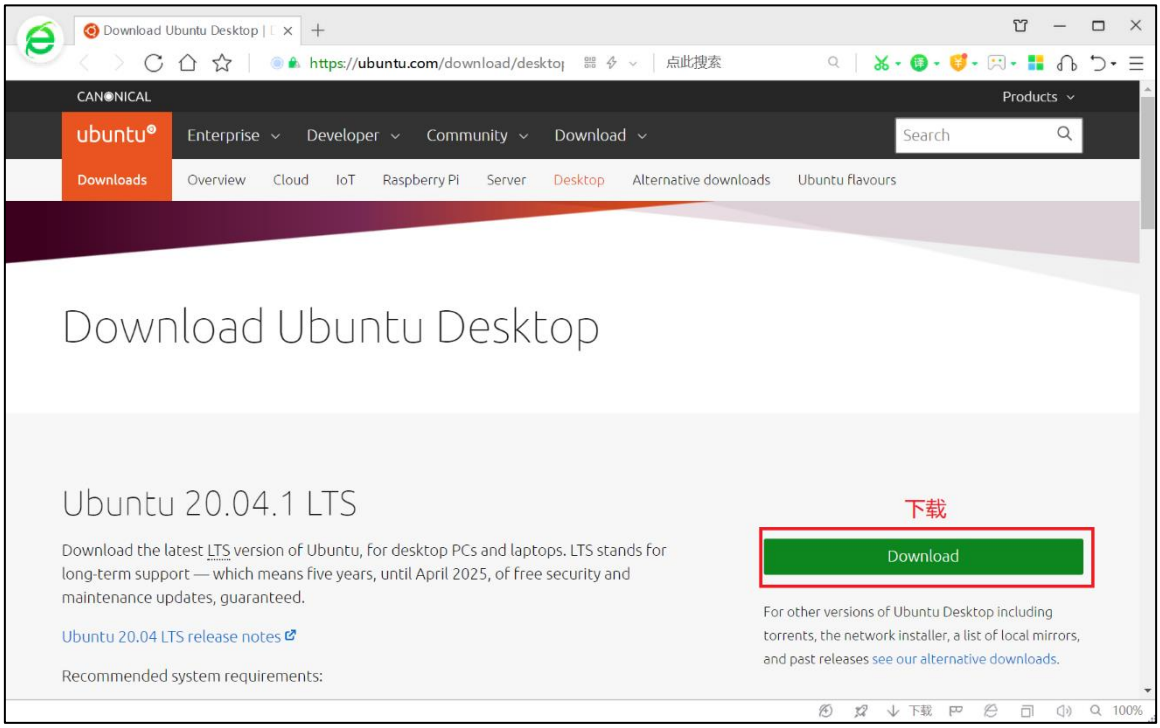


图 1.3.1.1 Ubuntu 最新版系统下载

从图 1.3.1.1 中可以看出，最新版本的 Ubuntu 系统是 20.04，但是在笔者编写本教程的时候

20.04 版刚出来没多久，怕不稳定，因此笔者实际使用的是 18.04 版本的 Ubuntu，后面所有的例程和教程均在 18.04 下完成，包括我们接下来安装的也是 18.04 版本的 Ubuntu。18.04 版本的 Ubuntu 下载地址为：http://releases.ubuntu.com/18.04/，下载“ubuntu-18.04.5-desktop-amd64.iso”这个版本，我已经下载下来放到了开发板光盘中，路径为：开发板光盘→3、软件->ubuntu-18.04.5-desktop-amd64.iso。

1.3.2 安装 Ubuntu 操作系统

Ubuntu 系统获取到以后就可以安装了，打开 VMware 软件，选择：虚拟机->设置，如图 1.3.2.1 所示：

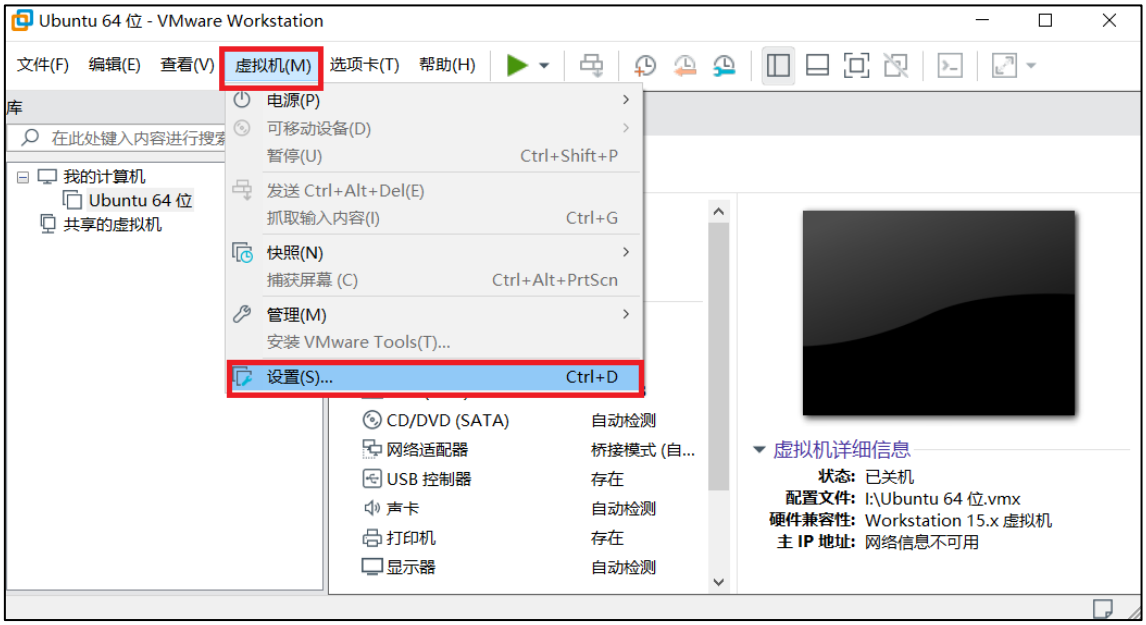


图 1.3.2.1 打开虚拟机设置对话框

打开以后的虚拟机设置对话框如图 1.3.2.2 所示：

图 1.3.2.2 虚拟机对话框

首先设置“USB 控制器”选项，默认 USB 控制器的 USB 兼容性为 USB2.0，这样当你使用 USB3.0 的设备的时候 Ubuntu 可能识别不出来，因此我们需要调整 USB 兼容性为 USB3.0，如图 1.3.2.3 所示：

图 1.3.2.3 USB 兼容性设置

设置要 USB 兼容性以后就开始安装 Ubuntu 系统了，选中虚拟机设置对话框中的“CD/DVD(SATA)”选项，然后在右侧选中“使用 ISO 映像文件”，如图 1.3.2.4 所示：

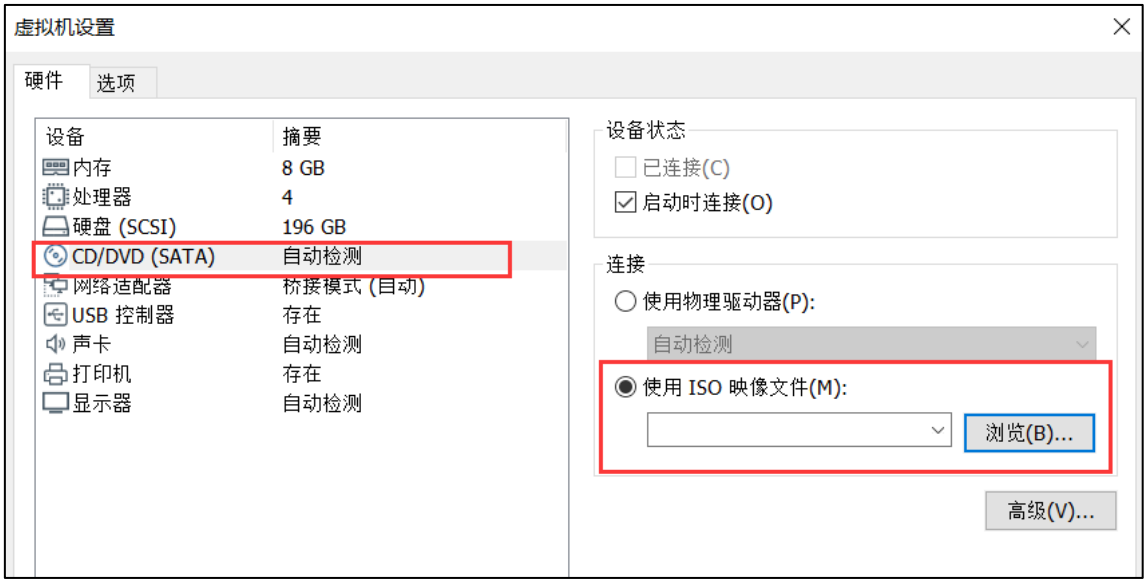


图 1.3.2.4 系统镜像设置

在图 1.3.2.4 中的“使用 ISO 映像文件”里面添加我们刚刚下载到的 Ubuntu 系统镜像，点击“浏览”按钮，选择 Ubuntu 系统镜像，完成以后如图 1.3.2.5 所示：

图 1.3.2.5 Ubuntu 镜像选择

设置好以后点击“确定”按钮退出，退出以后就可以打开虚拟机了，虚拟机就会自动的安装 Ubuntu 系统，如图 1.3.2.6 所示：

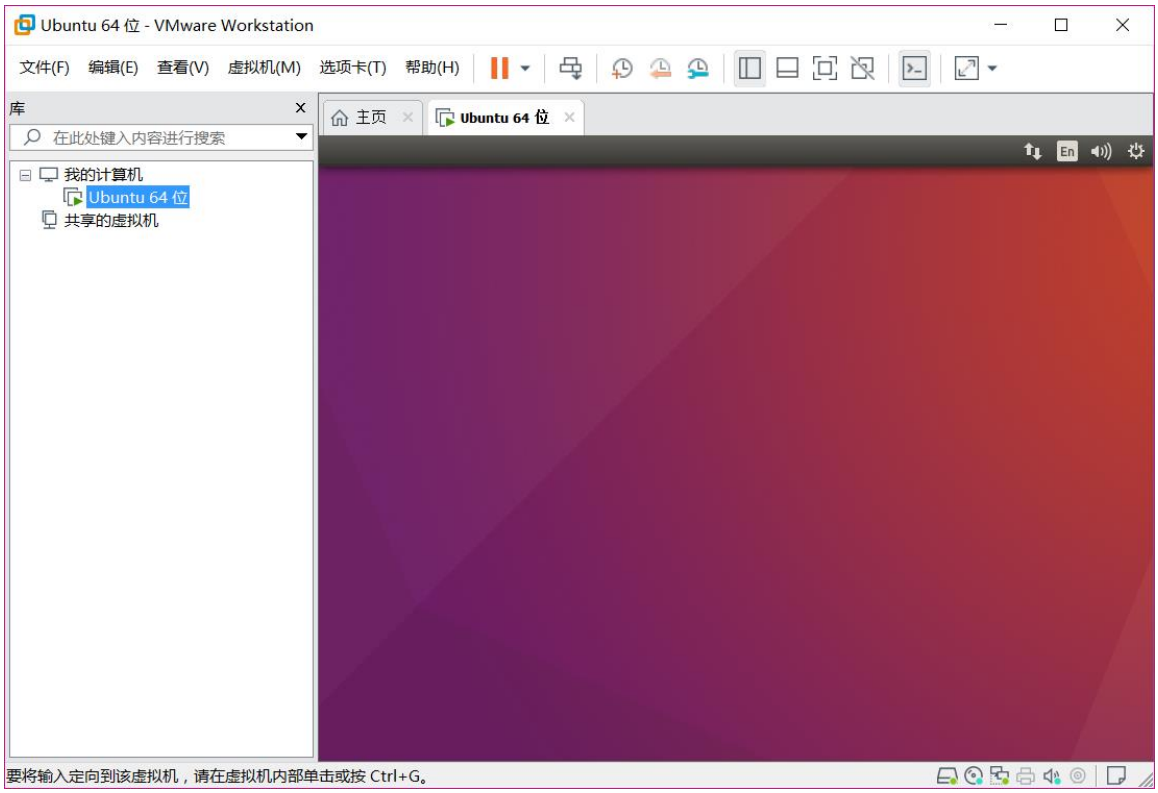


图 1.3.2.6 Ubuntu 安装开始

Ubuntu 开始安装以后首先是语言选择，如图 1.3.2.7 所示：

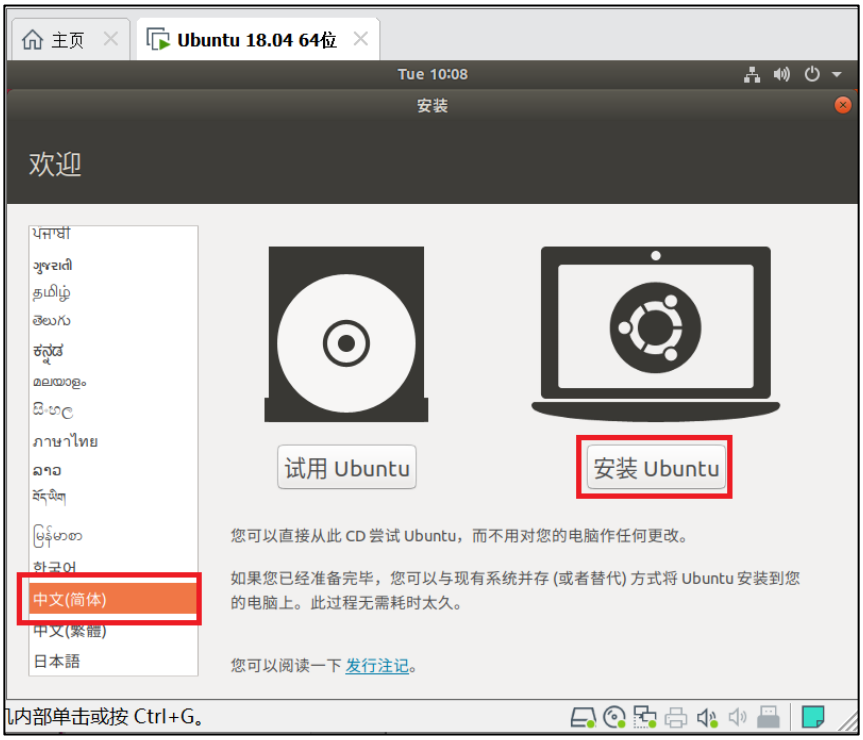


图 1.3.2.7 语言选择与安装

Ubuntu 默认语言是英文，毫无疑问，我们要选择“中文(简体)”，选择好以后点击右侧的“安装 Ubuntu”按钮，进入安装过程。安装一开始会有 7 个配置步骤，第一配置如图 1.3.2.8 所示，让你选择键盘布局：

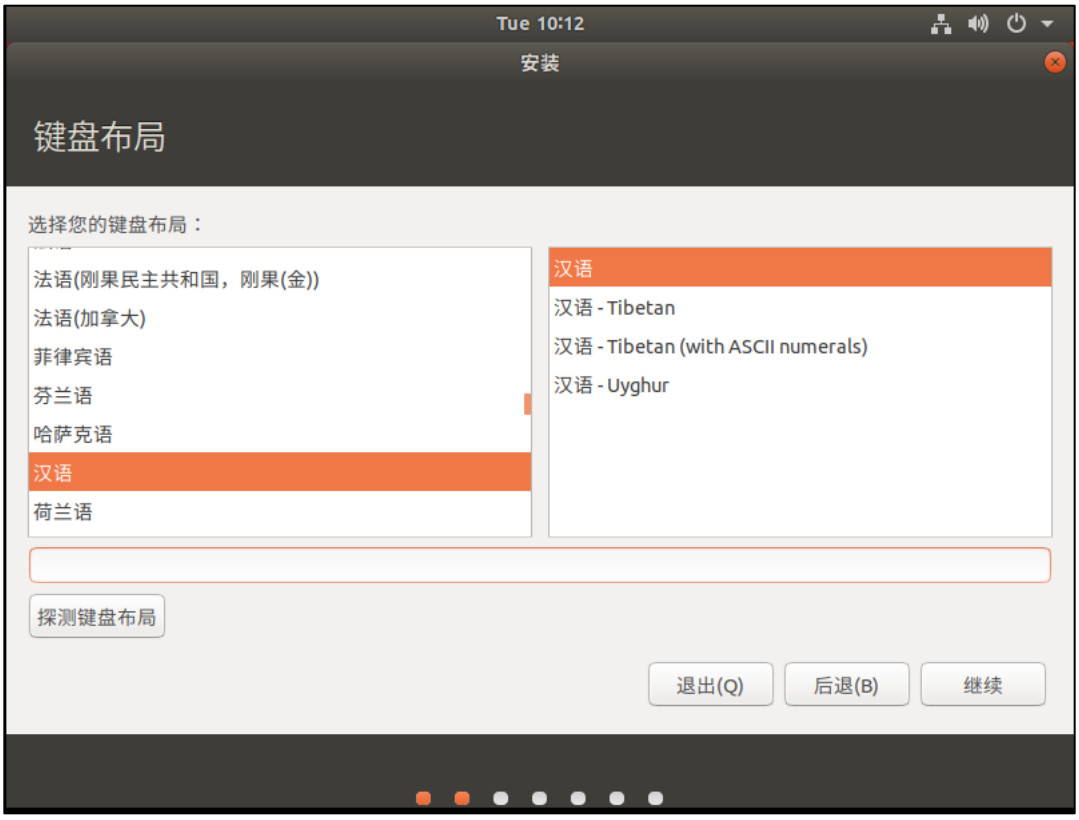


图 1.3.2.8 键盘布局选择

图 1.3.2.8 中键盘布局选择“中文”，然后点击“继续”。接下来就是“更新和其他软件”配置界面，让你选择先安装哪些应用，是否在安装 Ubuntu 时下载更新，以及是否为图形或者无线硬件安装其它第三方软件，配置如图 1.3.2.9 所示：

图 1.3.2.9 是否安装是下载更新

直接点击图 1.3.2.9 中的“继续”按钮，弹出安装类型，使用默认的“清除整个磁盘并安装 Ubuntu”，如图 1.3.2.10 所示：

图 1.3.2.10 安装类型选择

设置好安装类型以后点击“现在安装”按钮，会弹出“将改动写入磁盘吗？”对话框，点击“继续”即可，下一步会让你输入你在哪个位置，输入自己所在的城市即可，比如我在广州就输入“guangzhou”，如图 1.3.2.11 所示：

图 1.3.2.11 输入所在位置

输入地址以后点击“继续”按钮，进入下一步设置用户名和密码，自己设置自己的用户名和密码，比如我的设置如图 1.3.2.12 所示：

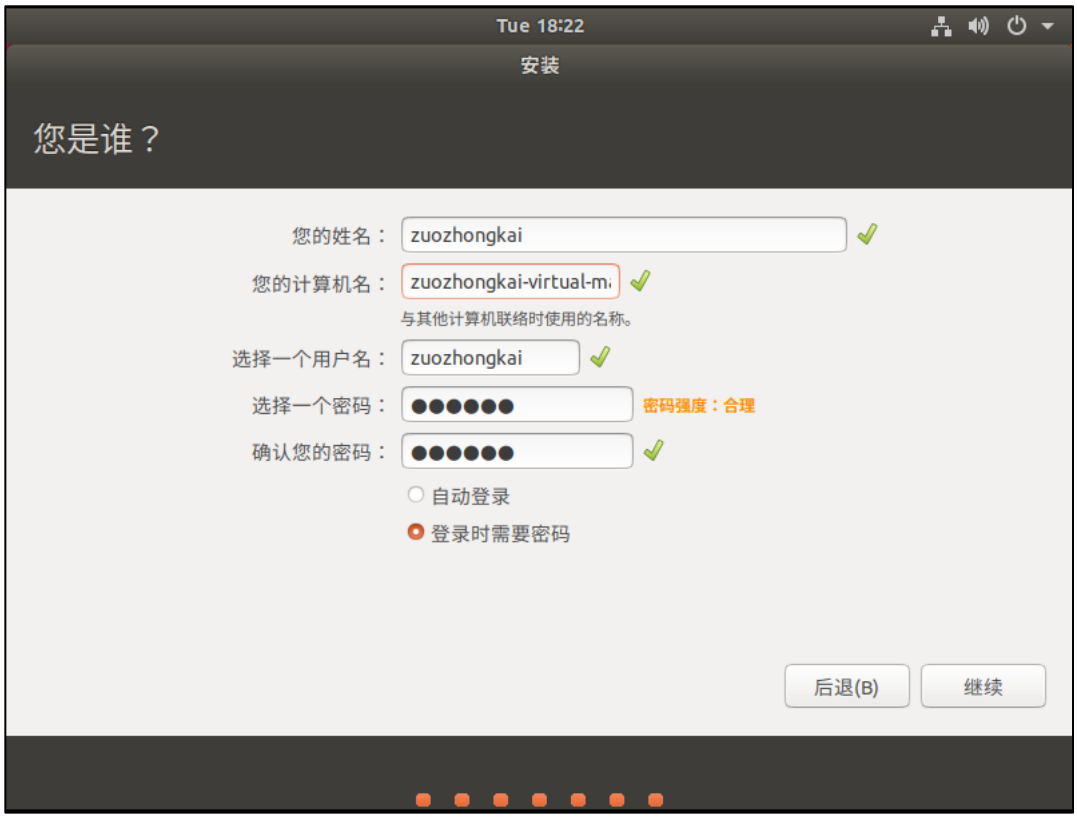


图 1.3.2.12 设置用户名和密码

设置好用户名和密码以后点击“继续”按钮，系统就会开始正式安装，如图 1.3.2.13 所示：

图 1.3.2.13 系统安装中

等待系统安装完成，安装过程中会下载一些文件，所以一定要保证电脑能够正常上网，如果不能正常上网的话可以点击右侧的“skip”按钮来跳过下载文件这个步骤，对于系统的安装没有任何影响，安装完成以后提示重启系统，如图 1.3.2.14 所示：

图 1.3.2.14 安装完成，重启系统

重启的时候需要弹出虚拟机里面的系统镜像！关闭 Ubuntu 操作系统，打开 VMware 的虚拟机设置界面，然后选中“CD/DVD(SATA)”，右侧的“连接”选择“使用物理驱动器”，如图 1.3.2.15 所示。

图 1.3.2.15 弹出 Ubuntu 系统镜像

设置好以后点击“确定”按钮，然后重新打开虚拟机，启动 Ubuntu，系统启动以后就会提示输入密码，如图 1.3.2.16 所示：

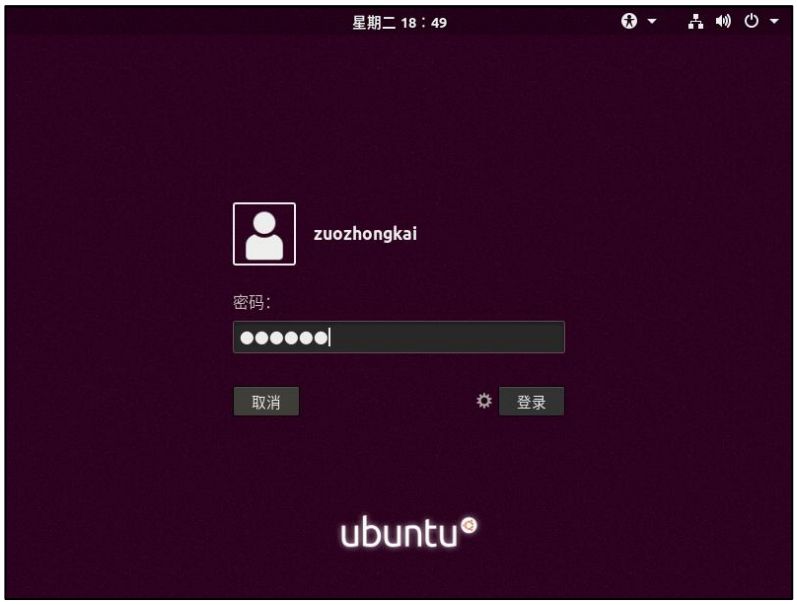


图 1.3.2.16 系统登录界面

在图 1.3.2.16 中输入密码，点击键盘上的回车就会进入系统主界面，系统界面如图 1.3.2.17 所示：

图 1.3.2.17 系统桌面

图 1.3.2.17 就是第一次进入系统的系统桌面，第一次进入系统会有“欢迎使用 Ubuntu 界面”，也就是给 Ubuntu 系统的简易引导指南，点击图 1.3.2.17 右上角的“前进(N)”来一步步观看指南。最终的系统主界面如图 1.3.2.18 所示：

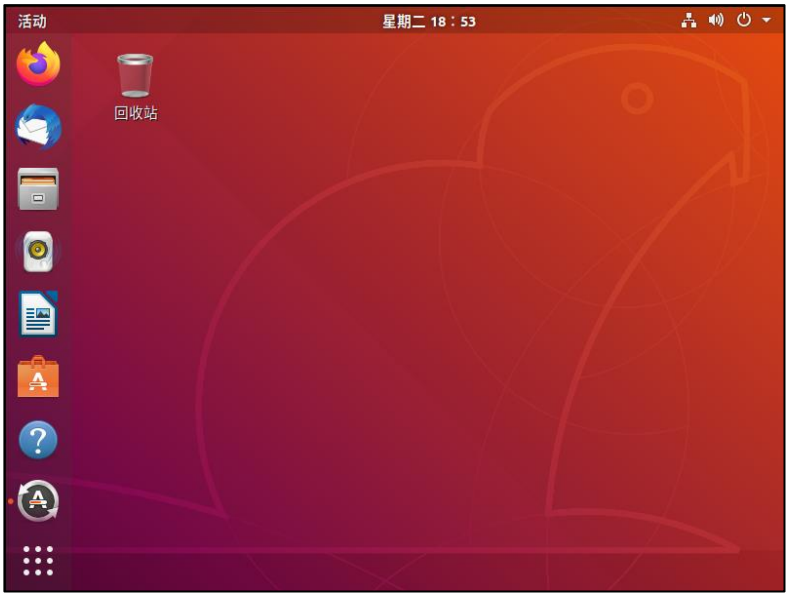


图 1.3.2.18 Ubuntu 系统主界面

要关闭 Ubuntu 系统的话按照图 1.3.2.19 所示操作：

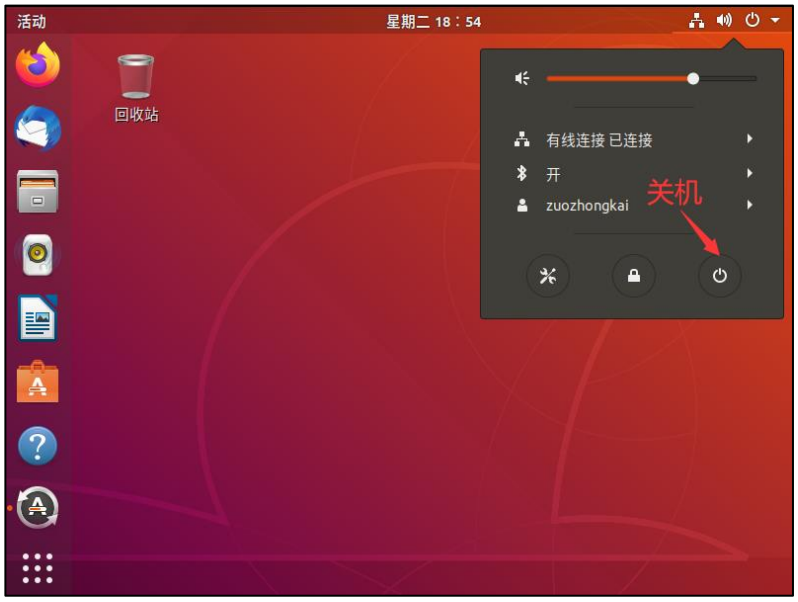


图 1.3.2.19 关闭 Ubuntu

点击图 1.3.2.19 右上角的图标，打开关闭界面，点击关闭界面右下角的关闭按钮，点击以后会弹出如图 1.3.2.20 所以对话框：

图 1.3.2.20 关闭对话框

如果要关机就点击图 1.3.2.20 中的“关机”，如果要重启就点击“重启”。

1.4 VMware tools 安装

本小节需要读者会操作 Ubuntu 系统，会使用命令行，如果对 Ubuntu 完全零基础可以先跳过本小节，先学习后面的 Ubuntu 入门教程，学会以后在来学习本小节。

有时候我们需要在虚拟机中的 Ubuntu 和本地 Windows 之间复制粘贴一些文件，这里就需要安装 VMware Tools。打开虚拟机中的 Ubuntu 系统，然后点击：Vmware 的虚拟机(M)->安装 VMware Tools(T)……，如图 1.4.1 所示：

图 1.4.1 安装“VMware Tools”

点击图 1.4.1 中的“安装 WMware Tools”，此时就会在 Ubuntu 系统中自动下载 VMware Tools 工具，下载完成以后会放到 Ubuntu 桌面上，如图 1.4.2 所示：

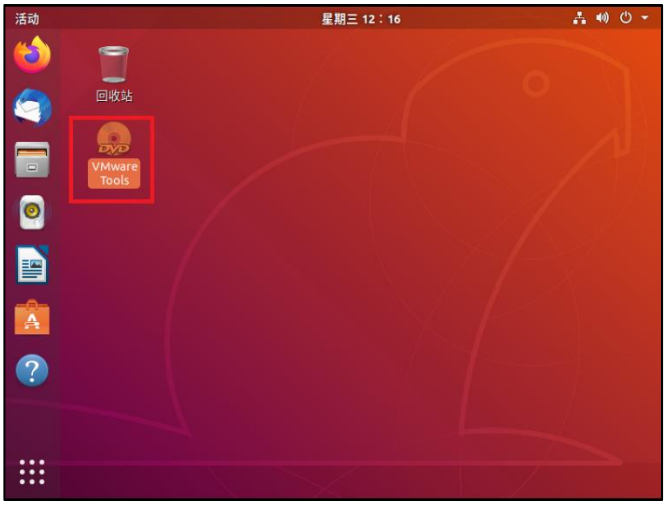


图 1.4.2 下载得到的 VMware Tools

双击图 1.4.2 中的 VMware Tools 安装包，打开以后如图 1.4.3 所示：

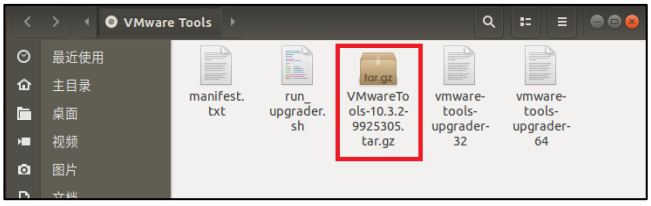


图 1.4.2 VMware Tools 安装包文件

图 1.4.2 中的 VMwareTools-10.3.2-9925305.tar.gz 就是我们要的安装包，将其提出来并放到桌面上，如图 1.4.3 所示：

图 1.4.3 提出出来的安装压缩包

图 1.4.3 就是提取出来的 VMware Tools 安装压缩包，对其进行解压，然后进入到解压后的文件夹，里面的内容如图 1.4.4 所示：

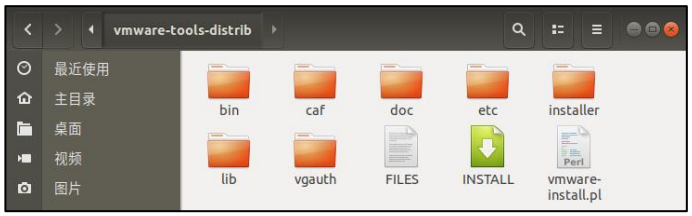


图 1.4.4 解压后的安装文件夹

图 1.4.4 中的 vmware-install.pll 就是我们需要的安装文件，接下来就需要用到命令行运行此安装文件里，打开命令行，输入如下命令进入到图 1.4.4 所在的文件夹：

```
cd 桌面          //进入桌面文件夹
ls               //查看 VMware Tools 是否存在
```

结果如图 1.4.5 所示：

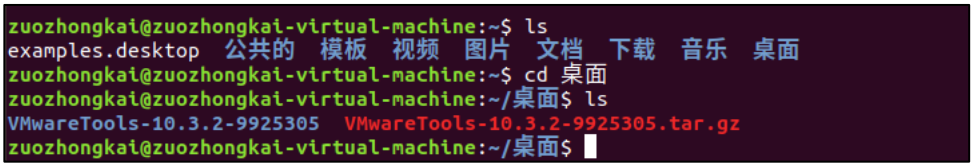


图 1.4.5 终端中查看 VMware Tools

进入到图 1.4.5 中 VMwareTools-10.3.2-9925305 文件夹中，这个就是解压后的 VMware Tools，然后执行 vmware-install.pll 即可，命令如下：

```
cd VMwareTools-10.3.2-9925305/vmware-tools-distrib      //进入安装文件夹
sudo ./vmware-install.pl                                //执行安装软件
```

安装的过程中会弹出一系列的操作提示，如果遇到选择“是”或“否”的这样的问题，输入“yes”表示是，输入“no”或者直接按回车表示“否”。如果是哪些安装路径的问题直接按回车键，使用默认路径即可，其他类型的问题大家根据实际提示做选择。安装完成以后如图 1.4.6 所示：

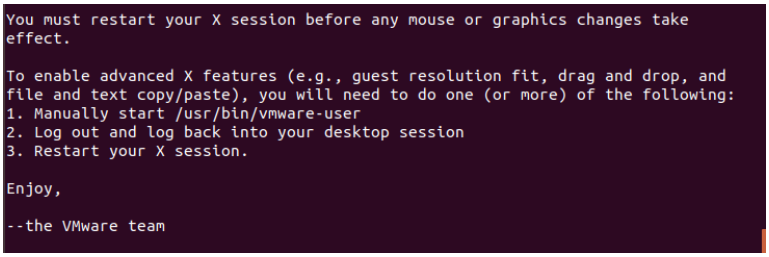


图 1.4.6 VMware Tools 安装完成

VMware Tools 安装完成以后重启 Ubuntu，重启以后就可以直接在虚拟机 Ubuntu 系统和主机 Windows 下进行文字、文件等的复制粘贴。

第二章 Ubuntu 系统入门

在上一章我们已经安装好虚拟机，并且在虚拟机中安装好了 Ubuntu 操作系统了，本章我们就来学习 Ubuntu 系统的基本使用，通过本章的学习为我们以后的开发做准备。Ubuntu 系统是和 Windows 一样的大型桌面操作系统，因此功能非常强大，不是一章就能介绍完的，因此本章叫做《Ubuntu 系统入门》。本章的主要目的是教会读者掌握后续嵌入式开发所需的 Ubuntu 基本技能，比如系统的基本设置、常用的 shell 命令、vim 编辑器的基本操作等等，如果想详细的学习 Ubuntu 操作系统请参考其它更为详实的书籍，本章参考了《Ubuntu Linux 从入门到精通》，这本书不厚，很适合用来作 Ubuntu 入门。

2.1 Ubuntu 系统初体验

2.1.1 Hello Ubuntu

上一章我们已经安装好了 Ubuntu 操作系统，我们再来回顾一下如何开机：

1、打开 VMware 虚拟机软件，打开以后如图 2.1.1.1 所示：

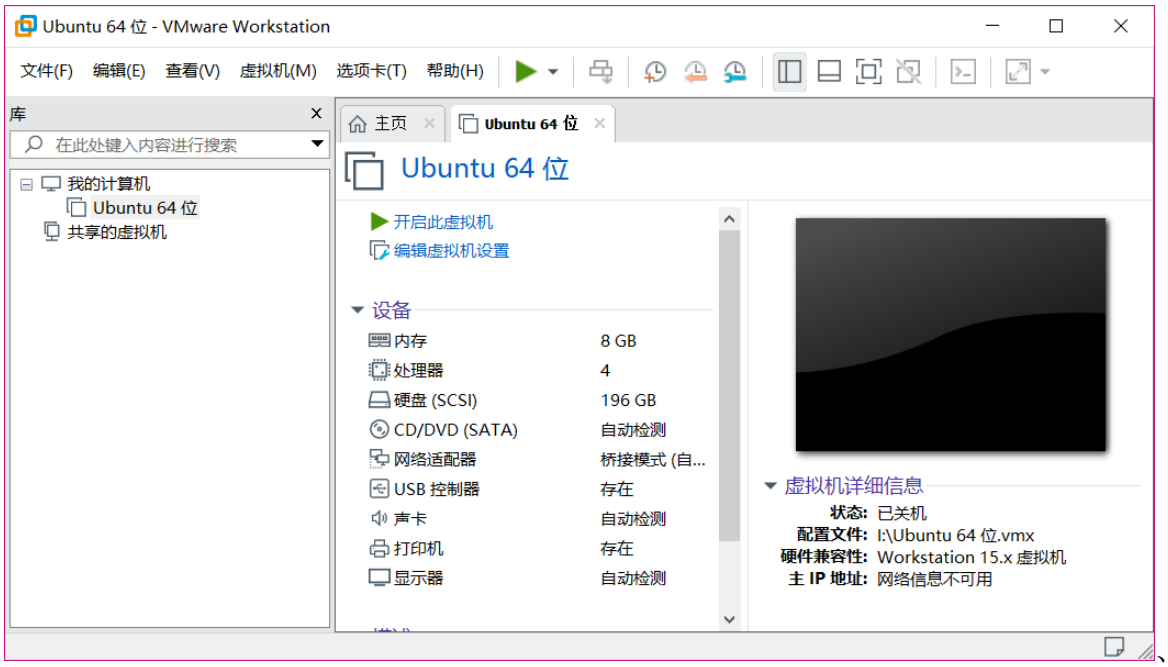


图 2.1.1.1 WMware 主界面

2、打开 VMware 上的开机按钮，打开方式如图 2.1.1.2 所示：

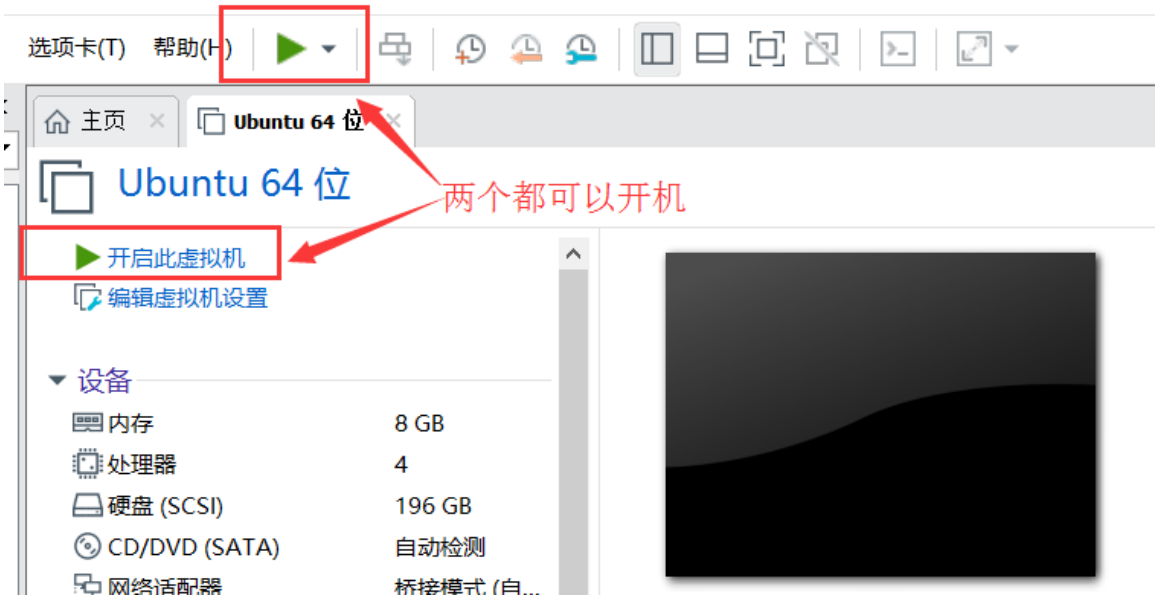


图 2.1.1.2 VMware 开机按钮

3、点击图 2.1.1.2 中两个开机按钮中的任意一个就会打开 Ubuntu 操作系统，首先进入图 2.1.1.3 所示的登陆界面，输入密码即可进入系统。

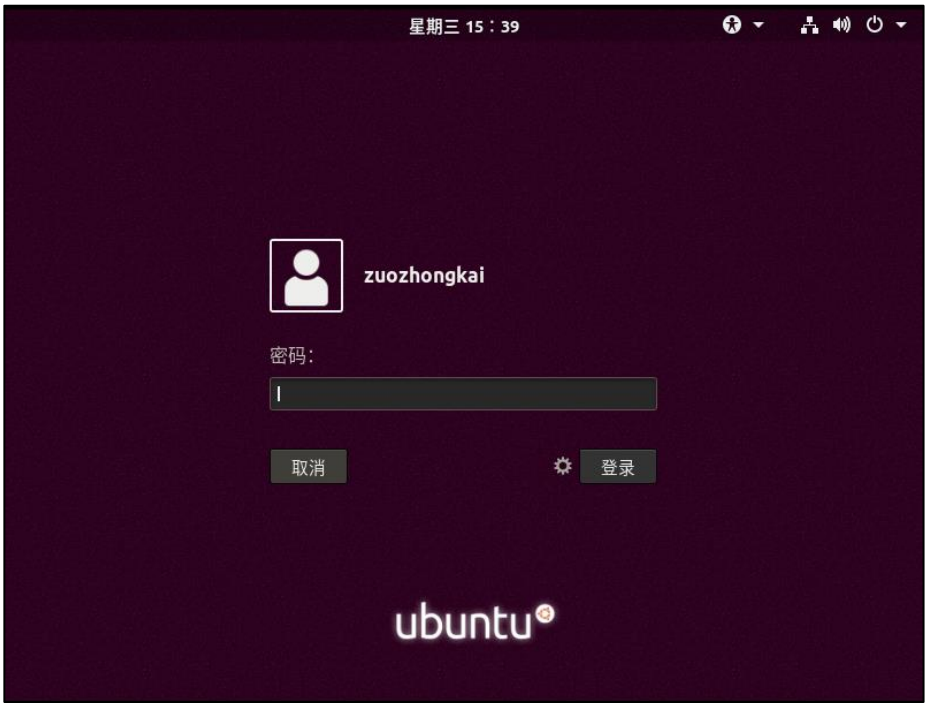


图 2.1.1.3 Ubuntu 登陆界面

在登陆界面输入密码，进入系统主界面，如图 2.1.1.4 所示：

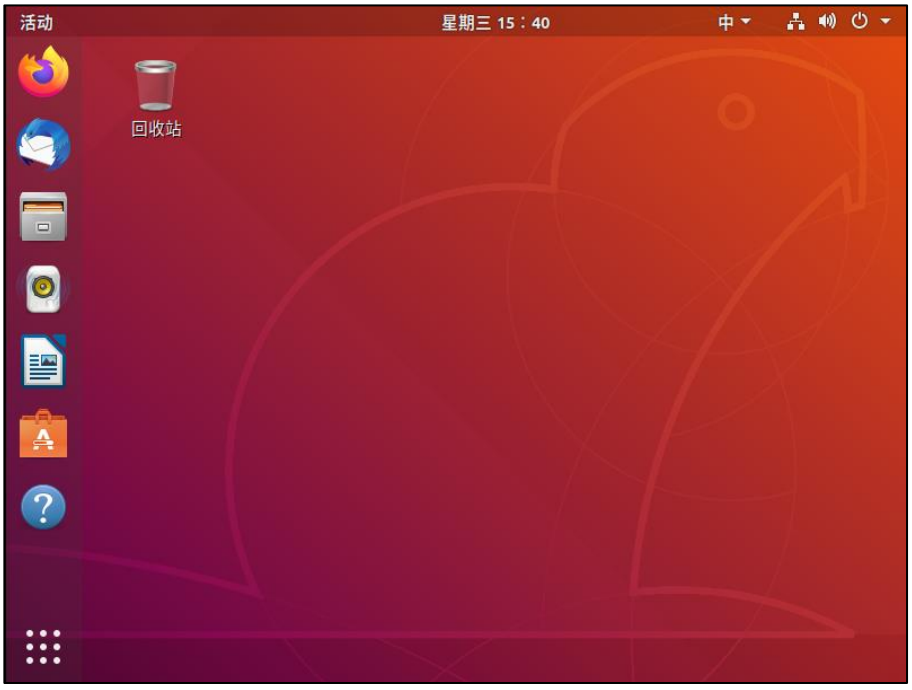


图 2.1.1.4 Ubuntu 主界面

进入主界面以后大家就可以看到和 Windows 基本一样，左侧有一列 APP，第一个是 firefox 浏览器，可以用来上网，比如我们登陆百度网站，如图 2.1.1.5 所示：

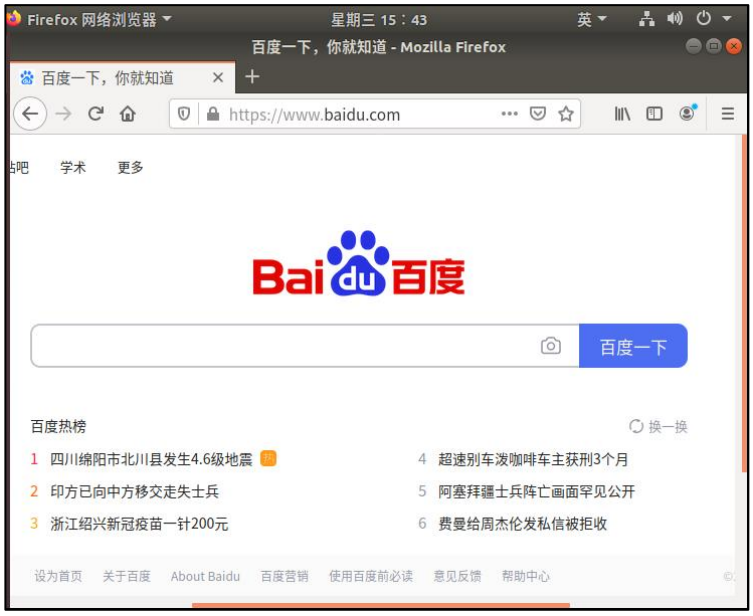


图 2.1.1.5 firefox 浏览器

第二个是 Thunderbird 邮件/新闻，第三个是文件浏览器，打开以后可以浏览 Ubuntu 系统中的文件，打开以后如图 2.1.1.6 所示：

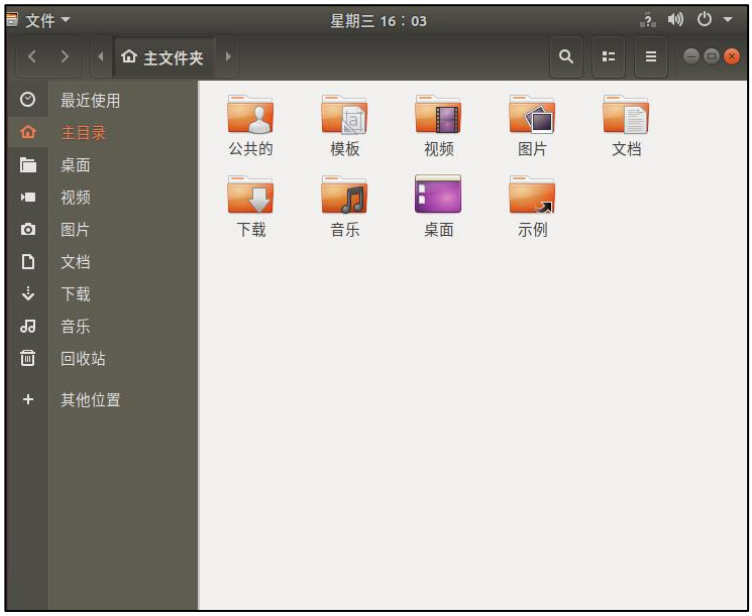


图 2.1.1.6 文件浏览

这里还有其它一些 APP，大家可以自行打开看一下这些 APP 都是干啥的，这里就不一一详细的介绍了。

2.1.2 系统设置

我们会发现，Ubuntu 的默认桌面很小，这是因为 Ubuntu 默认分辨率是 800*600，因此我们首先要设置系统分辨率，调整到合适的大小。打开系统设置界面，打开方式如图 2.1.2.1 所示：

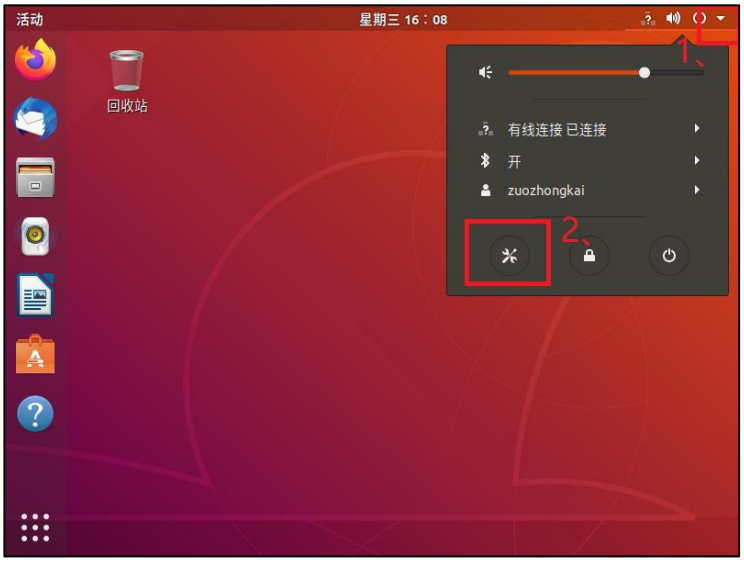


图 2.1.2.1 打开系统设置

打开以后的系统设置界面如图 2.1.2.2 所示：

图 2.1.2.2 系统设置界面

系统设置界面可以完成系统的大部分设置，我们找到“设备”选项并打开，打开以后如图 2.1.2.3 所示：

图 2.1.2.3 设备界面

图 2.1.2.3 就是“设备”界面，选中“显示”这一栏，右边就是显示相关设置，就有分辨率选项，点击“分辨率”选项，如图 2.1.2.4 所示：

图 2.1.2.4 分辨率设置项

从图 2.1.2.4 可以看出，Ubuntu 安装好以后默认分辨率是 800×600 的，点击“分辨率”这一栏，可以对 Ubuntu 分辨率进行设置，由于默认分辨率太低，因此图 2.1.2.4 中看到的图像不完整，不过这不影响我们进行分辨率设置，大家根据自己所使用的显示器进行设置，比如我这里设置 Ubuntu 分辨率为 1920×1200。设置好以后点击右上角的“应用”按钮，这里要注意，由于默认分辨率太小了，导致右上角的“应用”按钮看不到，所以需要拖动设置界面，把右侧的“应用”按钮拖出来，如图 2.1.2.5 所示：

图 2.1.2.5 显示界面的“应用”按钮

点击图 2.1.2.5 中的“应用”按钮以后会弹出来是否保留显示设置对话框，点击“保留更改”，如图 2.1.2.6 所示：

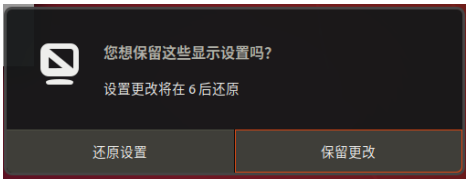


图 2.1.2.6 是否保留显示设置对话框

设置好分辨率以后 Ubuntu 的主界面就大了，看起来也舒服了。通过设置系统分辨率这个例子，我们就知道了如何设置 Ubuntu 系统，如果有需要设置其它东西的话都可以到系统设置里面去进行，这里就不一一详细的介绍了。

2.1.3 系统关机与锁定

当我们不使用 Ubuntu 系统以后就需要将其关机，就和我们使用 Windows 系统一样，千万不要通过直接退出 VMware 软件来关机!!关机很简单，在主界面，点击右上角的关机图标，然后选择关机选项，如图 2.1.3.1 所示：

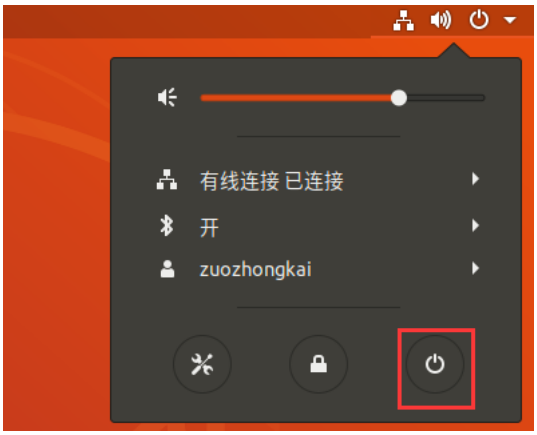


图 2.1.3.1 关机

点击在图 2.1.3.1 中的关机按钮以后，会弹出关机对话框，如图 2.1.3.2 所示：

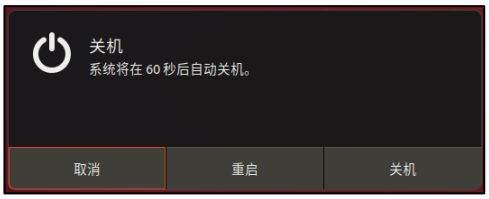


图 2.1.3.2 关机对话框

可以看到有三个选项：取消，重启和关机，想要重启就点击“重启”按钮，想要关机就点击“关机”按钮。

有时候我们需要临时离开，但是既不想关闭 Ubuntu，又不想让别人看我们的 Ubuntu，这个时候就可以锁定系统，点击主界面右上角，打开图 2.1.3.1 所示操作界面，然后点击中间的 图标即可锁定系统。

2.1.4 中文输入测试

我们是中国人，平时用的做多的肯定是中文，那么 Ubuntu 下中文输入是否和 Windows 一样呢？如何在 Ubuntu 下使用中文输入法。本节我们就以创建一个文本为例，介绍如何在 Ubuntu 中使用中文输入法。

Ubuntu 有个奇葩的地方，那就是安装好以后点击鼠标右键没有新建文档选项，这样我们创建文本文档很不方便。我们要做一个简单的设置，点击 Ubuntu 系统主界面上的文件 APP，找到名为“模板”的文件夹，如图 2.1.4.1 所示：

图 2.1.4.1 模板文件夹

打开图 2.1.4.1 中的“模板”文件夹，此文件夹默认是空的，在此文件夹中点击鼠标右键，

选择“在终端打开”，如图 2.1.4.2 所示：

图 2.1.4.2 在终端中打开文件夹

打开终端以后使用 gedit 创建一个文件，输入如下命令：

```
sudo gedit 无标题文档 //使用 gedit 创建一个空文件
```

上述命令需要输入中文“无标题文档”，Ubuntu 和 Windwos 下的输入法切换是一样的，按下“WIN+空格键”组合键即可进行输入法选择。

输入上面的命令以后会用 gedit 创建并打开一个空文件，如图 2.1.4.3 所示：

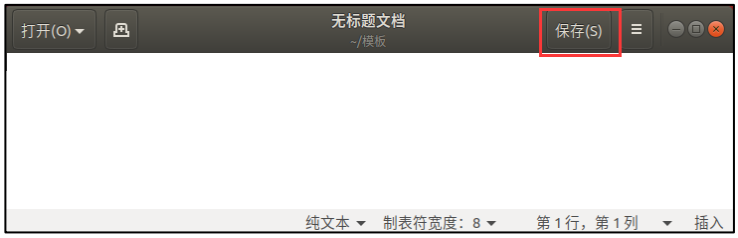


图 2.1.4.3 gedit 创建文件

直接点击图 2.1.4.3 右上角的“保存”按钮，此时就会在图 2.1.4.1 中的“模板”文件夹下创建了一个名为“无标题文档”的空文件，如图 2.1.4.4 所示：

图 2.1.4.4 新建的“无标题文档”文件

重新回到主界面，在桌面上点击鼠标右键，此时就会有“新建文档”选项，如图 2.1.4.5 所示：

图 2.1.4.5 新建文档

点击图 2.1.4.5 中的“无标题文档”，此时就会在桌面上创建一个名为“无标题文档”的文件，如图 2.1.4.6 所示：

图 2.1.4.6 新建的无标题文档

双击打开文档，打开以后如图 2.1.4.7 所示：

图 2.1.4.7 打开文档

打开文档以后，我们可以尝试在里面输入一些英文和数字，输入英文和数字是没有任何问题的，输入中文的话需要切换到 Ubuntu 自带的拼音输入法，前面也讲了，使用“WIN+空格键”，切换到拼音输入法以后就可以输入中文了，如图 2.1.4.8 所示：

图 2.1.4.8 输入中文文本

大家会发现 Ubuntu 下的拼音输入法使用起来跟 Windows 下的输入法差距太大了，没有 Windows 下的输入法好用，没办法，谁让桌面端 Linux 用的少呢，所以也就没有啥公司开发 Linux 下的输入法。

通过上面几个小节中对 Ubuntu 的基本操作来看，基本和 Windows 下的操作差不多，嵌入式开发中不会通过图形界面操作 Ubuntu，而是通过命令行操作的。这也是我们接下来着重要讲的：Ubuntu(Linux)终端操作，会涉及到很多命令，但是常用的命令就那几十个，不需要刻意的去背，使用习惯了就自然记住了。不要看到要记命令就觉得可怕。根据 2080 原则，80%情况下只使用那 20%的命令，实际情况会更少，常用的可能就那 5%~10%的命令。

2.2 Ubuntu 终端操作

本节就是我们学习 Ubuntu 操作系统的重点了，终端操作，也就是俗称的“敲命令”，不管是哪个版本的 Linux 发行版系统，它都会提供终端操作，Linux 下的终端操作类似与 Windows 下的 DOS 操作。要使用终端首先肯定是要打开终端，在主界面上点击鼠标右键，然后选择打开终端，如图 2.2.1 所示：

图 2.2.1 打开终端

打开终端以后如图 2.2.2 所示：

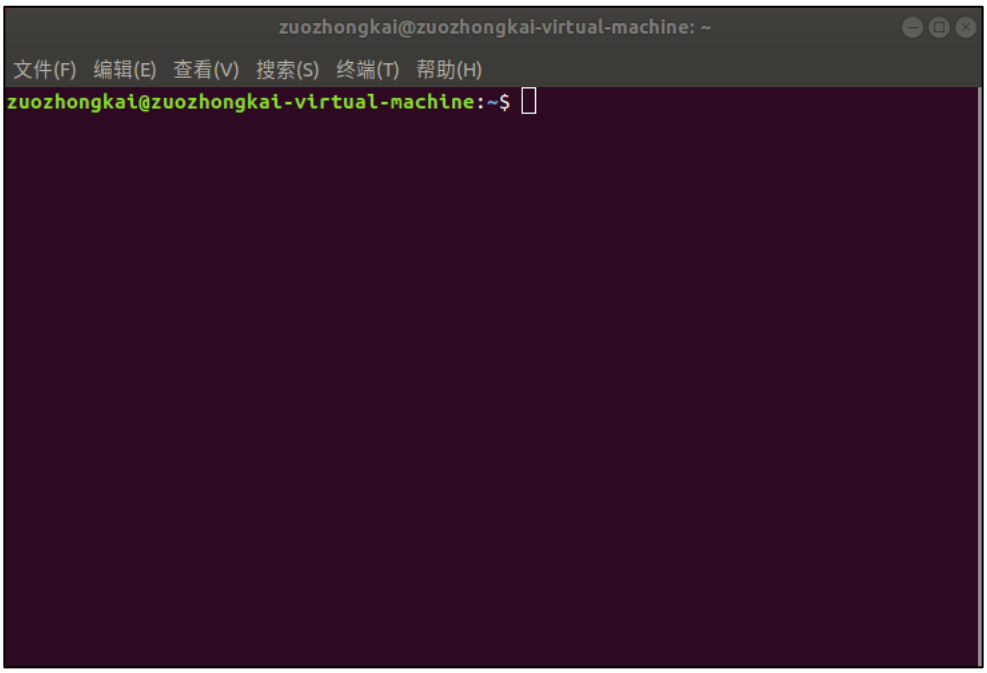


图 2.2.2 终端界面

我们就是在图 2.2.2 所示界面上输入命令的，终端默认会有类似下面一行所示的一串提示符:

```
zuozhongkai@zuozhongkai-virtual-machine: ~$
```

上述字符串中，@前面的“zuozhongkai”是当前的用户名字，@后面的 zuozhongkai-virtual-machine 是我的机器名字。最后面的符号“$”表示当前用户是普通用户，我们可以在提示符后面输入命令，比如输入命令“ls”，命令“ls”是打印出当前所在目录中所有文件和文件夹，如图 2.2.3 所示：

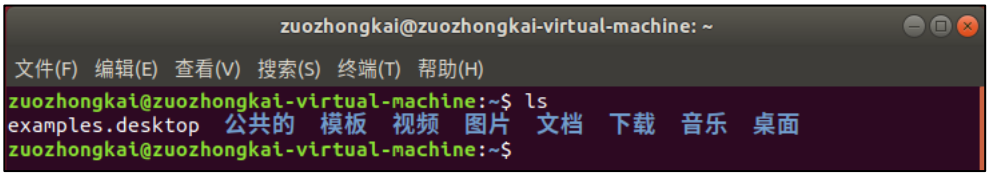


图 2.2.3 ls 命令

在图 2.2.3 中我们输入了“ls”这个命令，然后打印出了当前目录下的所有文件和文件夹，后面我们学习命令的时候就是在终端中输入相应命令的。

2.3 Shell 操作

2.3.1 Shell 简介

学习 linux 的时候会频繁的看到 Shell 这个词语？那么什么是 Shell 呢？网上搜索一下，各种专业的解释一堆，但是对于第一次接触 Linux 的人来说这些专业的词语只会让人更晕。简单的说 Shell 就是敲命令。国内把 Linux 下通过命令行输入命令叫做“敲命令”，国外人玩的比较洋气，人家叫做“Shell”。因此以后看到 Shell 这个词语第一反应就是在终端中敲命令，将多个 Shell 命令按照一定的格式放到一个文本中，那么这个文本就叫做 Shell 脚本。

严格意义上来讲，Shell 是一个应用程序，它负责接收用户输入的命令，然后根据命令做出

相应的动作，Shell 负责将应用层或者用户输入的命令传递给系统内核，由操作系统内核来完成相应的工作，然后将结果反馈给应用层或者用户。

2.3.2 Shell 基本操作

前面我们说 Shell 就是“敲命令”，那么既然是命令，那肯定是有格式的，Shell 命令的格式如下：

```
command      -options      [argument]
```

command: Shell 命令名称。

options：选项，同一种命令可能有不同的选项，不同的选项其实现的功能不同。

argument：Shell 命令是可以带参数的，也可以不带参数运行。

同样以命令“ls”为例，下面“ls”命令的三种不同格式其结果也不同：

```
ls
ls -l
ls /usr
```

这三种命令的运行结果如图 2.3.2.1 所示：

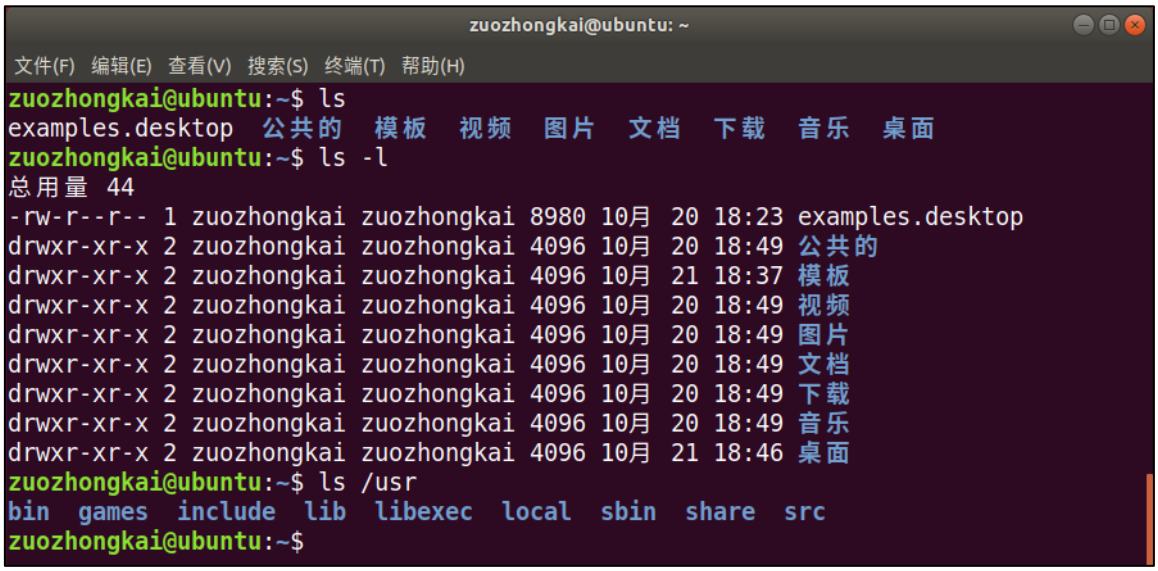


图 2.3.2.1 ls 命令

在图 2.3.2.1 中“ls”命令用来打印出当前目录下的所有文件和文件夹，而“ls -l”同样是打印出当前目录下的所有文件和文件夹，但是此命令会列出所有文件和文件夹的详细信息，比如文件大小、拥有者、创建日期等等。最有一个“ls /usr”是用来打印出目录“/usr”下的所有文件和文件夹。

Shell 命令是支持自动补全功能的，因为 Shell 命令非常多，如果不作自动补全的话就需要用户去记忆这些命令的全部字母。使用自动补全功能以后我们只需要输入命令的前面一部分字母，然后按下 TAB 键，如果只有一个命令匹配的话就会自动补全这个命令剩下的字母。如果有多个命令匹配的话系统就会发出报警声音，此时在按下一次 TAB 键就会列出所有匹配的命令，比如我们输入字母“if”，然后按下 TAB 键，结果如图 2.3.2.2 所示：

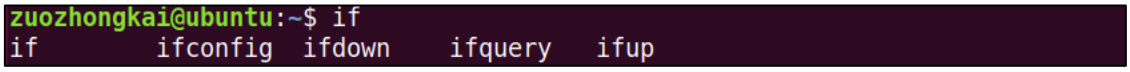


图 2.3.2.2 “if”开始的命令

从图 2.3.2.2 可以看出，以“if”开头的命令有 5 个，我们以“ifconfig”为例(Ubuntu 系统装

好以后默认是没有 ifconfig 命令的，需要手动安装，在终端中输入“sudo apt-get install net-tools”），此命令是用来查看网卡信息的，我们重新输入“ifc”然后在按一下 TAB 键，就会自动补全出“ifconfig”命令，因为以“ifc”开头的命令只有一个，结果如图 2.3.2.3 所示：

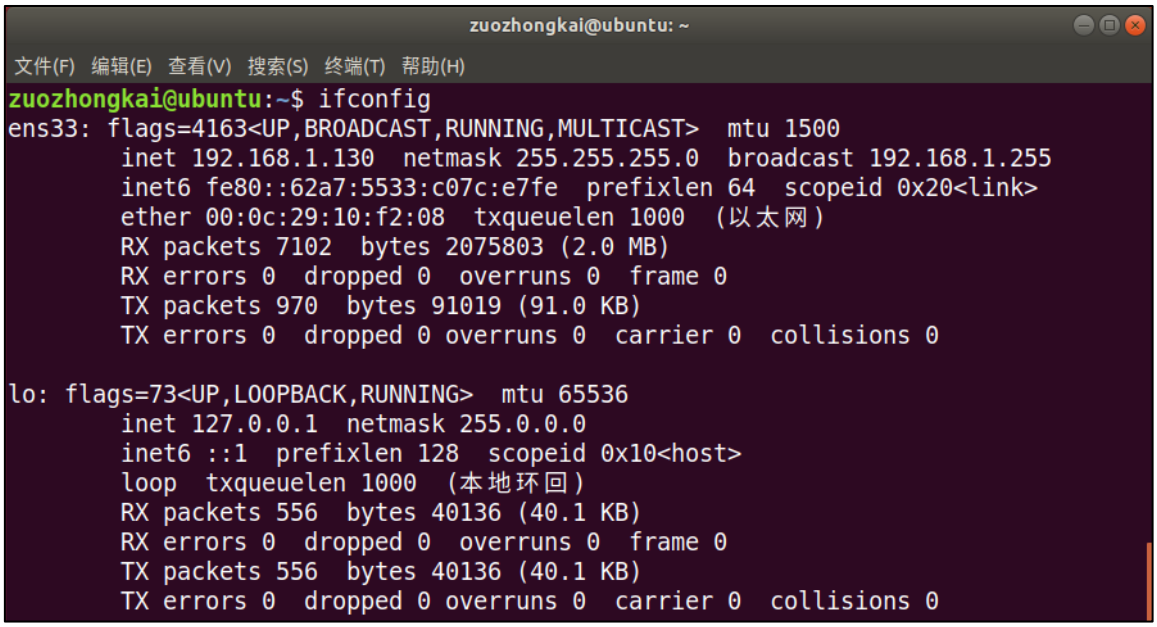


图 2.3.2.3 ifconfig 命令结果

2.2.4 常用 Shell 命令

我们做嵌入式开发用的最多就是 Shell 命令，Shell 命令是所有的 Linux 系统发行版所通用的，并不是说我在 Ubuntu 下学会了 Shell 命令，换另外一个 Linux 发行版操作系统以后就没用了(不同的发行版 Linux 系统可能会自定义一些命令)。本节我们先来介绍一些 Shell 下常用的命令：

1、目录信息查看命令 ls

文件浏览是最基本的操作了，Shell 下文件浏览命令为 ls，格式如下：

```
ls  [选项]  [路径]
```

ls 命令主要用于显示指定目录下的内容，列出指定目录下包含的所有的文件以及子目录，它的主要参数有：

-a ：显示所有的文件以及子目录，包括以“.”开头的隐藏文件。

-l ：显示文件的详细信息，比如文件的形态、权限、所有者、大小等信息。

-t ：将文件按照创建时间排序列出。

-A ：和-a 一样，但是不列出“.”(当前目录)和“..”(父目录)。

-R ：递归列出所有文件，包括子目录中的文件。

Shell 命令里面的参数是可以组合在一起用的，比如组合“-al”就是显示所有文件的详细信息，包括以“.”开头的隐藏文件，ls 命令使用如图 2.2.4.1 所示：

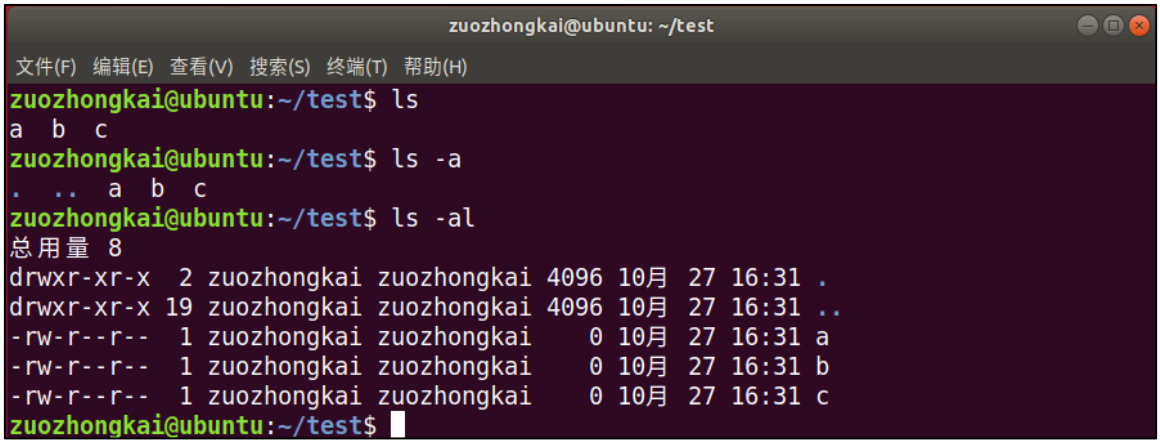


图 2.2.4.1 ls 命令演示

注意图 2.2.4.1 中 tmp 文件夹是我为了演示方便，自己创建的，里面的文件 a，b 和 c 也是我创建的，关于文件夹和文件的创建后面会详细的讲解。

2、目录切换命令 cd

要想在 Shell 中切换到其它的目录，使用的命令是 cd，命令格式如下：

```
cd    [路径]
```

路径就是我们要进入的目录路径，比如下面所示操作：

```
cd   /          //进入到根目录“/”下，Linux 系统的根目录为“/”，
cd   /usr       //进入到目录“/usr”里面。
cd   ..         //进入到上一级目录。
cd   ~          //切换到当前用户主目录
```

比如我们要进入到目录“/usr”下去，并且查看“/usr”下有什么文件，操作如图 2.2.4.2 所示：

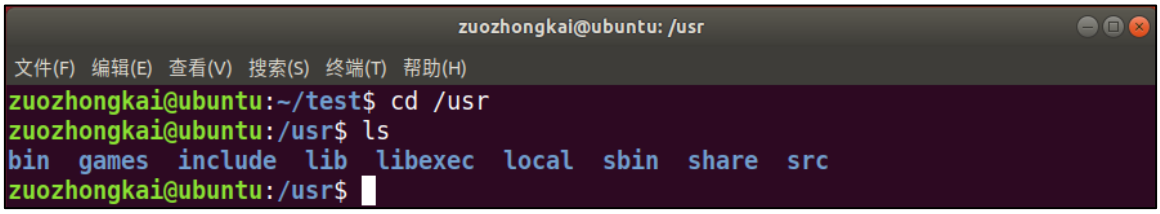


图 2.2.4.2 cd 命令演示

在图 2.2.4.2 中，我们先使用命令“cd /usr”进入到“/usr”目录下，然后使用“ls”命令显示“/usr”目录下的所有文件。仔细观察图 2.2.4.2 可以看到，当我们切换到其它目录以后在符号“$”前面就会以蓝色的字体显示出当前目录名字，如图 2.2.4.3 所示：

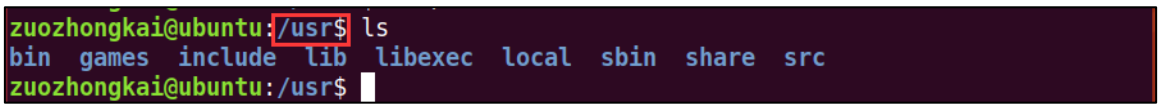


图 2.2.4.3 目录路径显示

3、当前路径显示命令 pwd

图 2.2.4.4 pwd 命令

4、系统信息查看命令 uname

要查看当前系统信息，可以使用命令 uname，命令格式如下：

```
uname [选项]
```

可选的选项参数如下：

-r ：列出当前系统的具体内核版本号。

-s ：列出系统内核名称。

-o ：列出系统信息。

使用如图 2.2.4.5 所示：

```
zuozhongkai@ubuntu:~$ uname
Linux
zuozhongkai@ubuntu:~$ uname -r
5.4.0-52-generic
zuozhongkai@ubuntu:~$ uname -s
Linux
zuozhongkai@ubuntu:~$ uname -o
GNU/Linux
zuozhongkai@ubuntu:~$
```

图 2.2.4.5 uanme 命令操作

5、清屏命令 clear

clear 命令用于清除终端上的所有内容，只留下一行提示符。

6、切换用户执行身份命令 sudo

Ubuntu(Linux)是一个允许多用户的操作系统，其中权限最大的就是超级用户 root，有时候我们执行一些操作的时候是需要用 root 用户身份才能执行，比如安装软件。通过 sudo 命令可以使我们暂时将身份切换到 root 用户。当使用 sudo 命令的时候是需要输入密码的，这里要注意输入密码的时候是没有任何提示的！命令格式如下：

```
sudo [选项] [命令]
```

选项主要参数如下：

-h ：显示帮助信息。

-l ：列出当前用户可执行与不可执行的命令

-p ：改变询问密码的提示符。

假如我们现在要创建一个新的用户 test，创建新用户的命令为“adduser”，创建新用户的权限只有 root 用户才有，我们在装系统的时候创建的那个用户是没有这个权限的，比如我的“zuozhongkai”用户。所以创建新用户的话需要使用“sudo”命令以 root 用户执行“adduser”这个命令，如图 2.2.4.6 所示：

```
zuozhongkai@ubuntu:~$ adduser test
adduser: 只有 root 才能将用户或组添加到系统。
zuozhongkai@ubuntu:~$ sudo adduser test
[sudo] zuozhongkai 的密码：
正在添加用户"test"...
正在添加新组"test" (1001)...
正在添加新用户"test" (1001) 到组"test"...
创建主目录"/home/test"...
正在从"/etc/skel"复制文件...
输入新的 UNIX 密码：
重新输入新的 UNIX 密码：
passwd: 已成功更新密码
正在改变 test 的用户信息
请输入新值，或直接敲回车键以使用默认值
        全名 []:
        房间号码 []:
        工作电话 []:
        家庭电话 []:
        其它 []:
这些信息是否正确？ [Y/n] y
```

图 2.2.4.6 sudo 命令演示

在图 2.2.4.6 中，我们一开始直接使用“adduser test”命令添加用户的时候提示我们“adduser：只有 root 才能将用户或组添加到系统。”所以我们要在前面加上“sudo”命令，表示以 root 用户执行 adduser 操作。

7、添加用户命令 adduser

在讲解 sudo 命令的时候我们已经用过命令“adduser”，此命令需要 root 身份去运行。命令格式如下：

```
adduser [参数] [用户名]
```

常用的参数如下：

-system ：添加一个系统用户

-home DIR ：DIR 表示用户的主目录路径

-uid ID ：ID 表示用户的 uid。

-ingroup GRP ：表示用户所属的组名。

adduser 的使用我们前面已经演示过了，大家可以试着再添加一个用户。

8、删除用户命令 deluser

前面讲了添加用户的命令，那肯定也有删除用户的命令，删除用户使用命令“deluser”，命令参数如下：

```
deluser [参数] [用户名]
```

主要参数有：

-system ：当用户是一个系统用户的时候才能删除。

-remove-home ：删除用户的主目录

-remove-all-files ：删除与用户有关的所有文件。

-backup ：备份用户信息

同样的，命令“deluser”也要使用“sudo”来以 root 用户运行，以删除我们前面创建的用户 test 为例，deluser 使用如图 2.2.4.7 所示：

```
zuozhongkai@ubuntu:~$ sudo deluser test
正在删除用户 'test'...
警告：组"test"没有其他成员了。
完成。
zuozhongkai@ubuntu:~$
```

图 2.2.4.7 命令 deluser 演示

9、切换用户命令 su

前面在讲解命令“sudo”的时候说过，“sudo”是以 root 用户身份执行一个命令，并没有更改当前的用户身份，所有需要 root 身份执行的命令都必须在前面加上“sudo”。命令“su”可以直接将当前用户切换为 root 用户，切换到 root 用户以后就可以尽情的进行任何操作了！因为你已经获得了系统最高权限，在 root 用户下，所有的命令都可以无障碍执行，不需要在前面加上“sudo”，“su”命令格式如下：

```
su [选项] [用户名]
```

常用选项参数如下：

-c –command ：执行指定的命令，执行完毕以后恢复原用户身份。

-login ：改变用户身份，同时改变工作目录和 PATH 环境变量。

-m ：改变用户身份的时候不改变环境变量

-h ：显示帮助信息

以切换到 root 用户为例，使用如图 2.2.4.8 所示：

```
zuozhongkai@ubuntu:~$ sudo su
[sudo] zuozhongkai 的密码：
root@ubuntu:/home/zuozhongkai#
```

图 2.2.4.8 su 命令演示

在图 2.2.4.8 中，先使用命令“sudo su”切换到 root 用户，su 命令不写明用户名的话默认切换到 root 用户。然后输入密码，密码正确的话就会切换到 root 用户，可以看到切换到 root 用户以后提示符的“@”符号前面的用户名变成了“root”，表示当前的用户是 root 用户。并且以“#”结束。

注意!! 由于 root 用户权限太大，稍微不注意就可能删除掉系统文件，导致系统奔溃，因此强烈建议大家，不要以 root 用户运行 Ubuntu。当要用到 root 身份执行某些命令的时候使用“sudo”命令即可。

要切换回原来的用户，使用命令“sudo su 用户名”即可，比如我要从 root 切换回 zuozhongkai 这个用户，操作如图 2.2.4.9 所示：

```
root@ubuntu:/home/zuozhongkai# sudo su zuozhongkai
zuozhongkai@ubuntu:~$
```

图 2.2.4.9 切换回原来用户

10、显示文件内容命令 cat

查看文件内容是最常见的操作了，在 windows 下可以直接使用记事本查看一个文本文件内容，linux 下也有类似记事本的软件，叫做 gedit，找到一个文本文件，双击打开，默认使用的就是 gedit，如图 2.2.4.10 所示：

图 2.2.4.10 gedit 打开文档

我们现在讲解的是 Shell 命令，那么 Shell 下有没有办法读取文件的内容呢？肯定有的，那就是命令“cat”，命令格式如下：

```
cat [选项] [文件]
```

选项主要参数如下：

-n ：由 1 开始对所有输出的行进行编号。

-b ：和-n 类似，但是不对空白行编号。

-s ：当遇到连续两个行以上空白行的话就合并为一个行空白行。

比如我们以查看文件“/etc/environment”的内容为例，结果如图 2.2.4.11 所示：

```
zuozhongkai@ubuntu:~$ cat /etc/environment
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
zuozhongkai@ubuntu:~$ cat /etc/environment -n
     1  PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
```

图 2.2.4.11 命令 cat 演示

11、显示和配置网络属性命令 ifconfig

ubuntu 默认没有 ifconfig 命令，需要自己手动安装，在终端输入如下命令：

```
sudo apt-get install net-tools
```

ifconfig 是一个跟网络属性配置和显示密切相关的命令，通过此命令我们可以查看当前网络属性，也可以通过此命令配置网络属性，比如设置网络 IP 地址等等，此命令格式如下：

```
ifconfig interface options | address
```

主要参数如下：

interface ：网络接口名称，比如 eth0 等。

up ：开启网络设备。

down ：关闭网络设备。

add ：IP 地址，设置网络 IP 地址。

netmask add 子网掩码。

命令 ifconfig 的使用如图 2.2.4.12 所示：

```
zuozhongkai@ubuntu:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.130  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::62a7:5533:c07c:e7fe  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:10:f2:08  txqueuelen 1000  (以太网)
        RX packets 30653  bytes 4231234 (4.2 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1137  bytes 105937 (105.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (本地环回)
        RX packets 1754  bytes 121732 (121.7 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1754  bytes 121732 (121.7 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

zuozhongkai@ubuntu:~$ ifconfig ens33
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.130  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::62a7:5533:c07c:e7fe  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:10:f2:08  txqueuelen 1000  (以太网)
        RX packets 30710  bytes 4236436 (4.2 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1137  bytes 105937 (105.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

图 2.2.4.12 ifconfig 命令演示

在图 2.2.4.12 中有两个网卡：ens33 和 lo，ens33 是我的电脑实际使用的网卡，lo 是回测网卡。可以看出网卡 ens33 的 IP 地址为 192.168.1.130，我们使用命令“ifconfig”将网卡 ens33 的 IP 地址改为 192.168.1.253，操作如图 2.2.4.13 所示：

```
zuozhongkai@ubuntu:~$ ifconfig ens33
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.253  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::62a7:5533:c07c:e7fe  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:10:f2:08  txqueuelen 1000  (以太网)
        RX packets 31271  bytes 4287944 (4.2 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1151  bytes 107944 (107.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

图 2.2.4.13 修改网卡 IP 地址

从图 2.2.4.13 可以看出，我在使用命令“ifconfig”修改网卡 ens33 的 IP 地址的时候使用了“sudo”，说明在 Ubuntu 下修改网卡 IP 地址是需要 root 用户权限的。当修改完以后使用命令“ifconfig ens33”再次查看网卡 ens33 的命令，发现网卡 ens33 的 IP 地址变成了 192.168.1.253

12、系统帮助命令 man

Ubuntu 系统中有很多命令，这些命令都有不同的格式，不同的格式对应不同的功能，要完全记住这些命令和格式几乎是不可能的，必须有一个帮助手册，当我们需要了解一个命令的详细信息的时候查阅这个帮助手册就行了。Ubuntu 提供了一个命令来帮助用户完成这个功能，那就是“man”命令，通过“man”命令可以查看其它命令的语法格式、主要功能、主要参数说明等， “man”命令格式如下：

```
man [命令名]
```

比如我们要查看命令“ifconfig”的说明，输入“man ifconfig”即可，如图 2.2.4.14 所示：

```
zuozhongkai@ubuntu:~$ man ifconfig
```

图 2.2.4.14 man 命令演示

在终端中输入图 2.2.4.14 所示的命令，然后点击回车键就会打开“ifconfig”这个命令的详细说明，如图 2.2.4.15 所示：

```
IFCONFIG(8)          Linux System Administrator's Manual          IFCONFIG(8)

NAME
       ifconfig - configure a network interface

SYNOPSIS
       ifconfig [-v] [-a] [-s] [interface]
       ifconfig [-v] interface [aftype] options | address ...

DESCRIPTION
       Ifconfig  is  used  to configure the kernel-resident network interfaces.
       It is used at boot time to set up interfaces as necessary.  After  that,
       it  is  usually  only  needed  when  debugging  or when system tuning is
       needed.

       If no arguments are given, ifconfig displays the status of the currently
       active interfaces.  If a single interface argument is given, it displays
       the status of the given interface only;  if  a  single  -a  argument  is
       given,  it  displays  the  status of all interfaces, even those that are
       down.  Otherwise, it configures an interface.

Address Families
       If the first argument after the interface name is recognized as the name
 Manual page ifconfig(8) line 1 (press h for help or q to quit)
```

图 2.2.4.15 命令“ifconfig”详细介绍信息

图 2.2.4.15 就是命令“ifconfig”的详细介绍信息，按“q”键退出到终端。

13、 系统重启命令 reboot

通过点击 Ubuntu 主界面右上角的齿轮按钮来选择关机或者重启系统，同样的我们也可以使用 Shell 命令“reboot”来重启系统，直接输入命令“reboot”然后点击回车键接口，如图 2.2.4.16 所示：

```
zuozhongkai@ubuntu:~$ reboot
```

图 2.2.4.16 reboot 命令演示

14、系统关闭命令 poweroff

使用命令“reboot”可以重启系统，使用命令“poweroff”就可以关闭系统，在终端中输入命令“poweroff”然后按下回车键即可关闭 Ubuntu 系统，如图 2.2.4.17 所示：

```
zuozhongkai@ubuntu:~$ poweroff
```

图 2.2.4.17 poweroff 命令演示

15、软件安装命令 install

截至目前，我们都没有讲过 Ubuntu 下如何安装软件，因为 Ubuntu 安装软件不像 Windows 下那样，直接双击.exe 文件就开始安装了。Ubuntu 下很多软件是需要先自行下载源码，下载源码以后自行编译，编译完成以后使用命令“intsall”来安装。当然 Ubuntu 下也有其它的软件安装方法，但是用的最多的就是自行编译源码然后安装，尤其是嵌入式 Linux 开发。命令“install”

格式如下：

```
      install [选项]...  [-T]   源文件        目标文件
或：  install [选项]...         源文件...     目录
或：  install [选项]...    -t   目录          源文件...
或：  install [选项]...    -d   目录...
```

“install”命令是将文件(通常是编译后的文件)复制到目的位置，在前三种形式中，将源文件复制到目标文件或将多个源文件复制到一个已存在的目录中同时设置其所有权和权限模式。在第四种形式会创建指定的目录。命令“install”通常和命令“apt-get”组合在一起使用的，关于“apt-get”命令我们稍后会讲解。

以上就是 Shell 最基本一些命令，还有一些其它的命令我们在后面在讲解，循序渐进嘛。

2.4 APT 下载工具

对于长时间使用 Windows 的我们，下载安装软件非常容易，Windows 下有很多的下载软件，Ubuntu 同样有不少的下载软件，本节我们讲解 Ubuntu 下我们用的最多的下载工具：APT 下载工具，APT 下载工具可以实现软件自动下载、配置、安装二进制或者源码的功能。APT 下载工具和我们前面讲解的“install”命令结合在一起构成了 Ubuntu 下最常用的下载和安装软件方法。它解决了 Linux 平台下一安装软件的一个缺陷，即软件之间相互依赖。

APT 采用的 C/S 模式，也就是客户端/服务器模式，我们的 PC 机作为客户端，当需要下载软件的时候就向服务器请求，因此我们需要知道服务器的地址，也叫做安装源或者更新源。点击 Ubuntu 主界面左上角的“活动”，然后在搜索栏搜索“软件和更新”，如图 2.4.1.1 所示：

图 2.4.1.1 搜索“软件和更新”

打开图 2.4.1.1 中的“软甲和更新”，打开以后如图 2.4.1.2 所示：

图 2.4.1.2 软件和更新设置

在图 2.4.1.2 中的“Ubuntu 软件”选项卡下面的“下载自”就是 APT 工具的安装源，因为我们是在中国，所以需要选择中国的服务器，否则的话可能会导致下载失败！这个也就是网上说的 Ubuntu 安装成功以后要更新源。

在我们使用 APT 工具下载安装或者更新软件的时候，首先会在下载列表中与本机软件对比，看一下需要下载哪些软件，或者升级哪些软件，默认情况下 APT 会下载最新的软件包，被安装的软件包所依赖的其它软件也会被下载安装。说了这么多，APT 下载工具究竟怎么用呢？APT 工具常用的命令如下：

1、更新本地数据库

如果想查看本地哪些软件可以更新的话可以使用如下命令：

```
sudo apt-get update
```

这个命令会访问源地址，并且获取软件列表并保存在本电脑上，过程如图 2.4.1.3 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get update
[sudo] zuozhongkai 的密码：
获取:1 http://security.ubuntu.com/ubuntu bionic-security InRelease [88.7 kB]
命中:2 http://cn.archive.ubuntu.com/ubuntu bionic InRelease
获取:3 http://cn.archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7 kB]
获取:4 http://security.ubuntu.com/ubuntu bionic-security/main i386 Packages [845 kB]
获取:5 http://cn.archive.ubuntu.com/ubuntu bionic-backports InRelease [74.6 kB]
获取:6 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main i386 Packages [1,140 kB]
获取:7 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [1,397 kB]
获取:8 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [1,725 kB]
获取:9 http://security.ubuntu.com/ubuntu bionic-security/main Translation-en [276 kB]
获取:10 http://security.ubuntu.com/ubuntu bionic-security/main amd64 DEP-11 Metadata [49.0 kB]
获取:11 http://security.ubuntu.com/ubuntu bionic-security/universe i386 Packages [964 kB]
获取:12 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main Translation-en [367 kB]
获取:13 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main amd64 DEP-11 Metadata [294 kB]
获取:14 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 Packages [1,078 kB]
获取:15 http://cn.archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [184 kB]
获取:16 http://cn.archive.ubuntu.com/ubuntu bionic-updates/restricted Translation-en [24.6 kB]
获取:17 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe i386 Packages [1,544 kB]
获取:18 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe amd64 Packages [1,680 kB]
获取:19 http://security.ubuntu.com/ubuntu bionic-security/universe Translation-en [241 kB]
获取:20 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 DEP-11 Metadata [58.8 kB]
获取:21 http://security.ubuntu.com/ubuntu bionic-security/universe DEP-11 64x64 Icons [120 kB]
获取:22 http://security.ubuntu.com/ubuntu bionic-security/multiverse amd64 DEP-11 Metadata [2,460 B]
获取:23 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe Translation-en [353 kB]
获取:24 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe amd64 DEP-11 Metadata [287 kB]
获取:25 http://cn.archive.ubuntu.com/ubuntu bionic-updates/multiverse amd64 DEP-11 Metadata [2,464 B]
获取:26 http://cn.archive.ubuntu.com/ubuntu bionic-backports/universe amd64 DEP-11 Metadata [9,288 B]
已下载 12.9 MB，耗时 6秒 (2,163 kB/s)
正在读取软件包列表... 完成
```

图 2.4.1.3 更新本地数据库

2、检查依赖关系

有时候本地某些软件可能存在依赖关系，所谓依赖关系就是 A 软件依赖于 B 软件。通过如下命令可以查看依赖关系，如果存在依赖关系的话 APT 会提出解决方案：

```
sudo apt-get check
```

上述命令的执行结果如图 2.4.1.4 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get check
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
```

图 2.4.1.4 检查依赖关系

3、软件安装

这个是重点了，安装软件，使用如下命令：

```
sudo apt-get install package-name
```

可以看出上述命令是由“apt-get”和“install”组合在一起的，“package-name”就是要安装的软件名字，“apt-get”负责下载软件，“install”负责安装软件。比如我们要安装软件 Ubuntu 下的串口工具“minicom”，我们就可以使用如下命令：

```
sudo apt-get install minicom
```

执行上述命令以后就会自动下载和安装 minicom 软件，如图 2.4.1.5 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get install minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
将会同时安装下列软件:
  lrzsz
下列【新】软件包将被安装:
  lrzsz minicom
升级了 0 个软件包，新安装了 2 个软件包，要卸载 0 个软件包，有 40 个软件包未被升级。
需要下载 313 kB 的归档。
解压缩后会消耗 1,471 kB 的额外空间。
您希望继续执行吗？ [Y/n] y
获取:1 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe amd64 lrzsz amd64 0.12.21-
10~build0.18.04.1 [74.8 kB]
获取:2 http://cn.archive.ubuntu.com/ubuntu bionic/universe amd64 minicom amd64 2.7.1-1 [239 k
B]
已下载 313 kB，耗时 2秒 (153 kB/s)
正在选中未选择的软件包 lrzsz。
(正在读取数据库 ... 系统当前共安装有 167203 个文件和目录。)
正准备解包 .../lrzsz_0.12.21-10~build0.18.04.1_amd64.deb  ...
正在解包 lrzsz (0.12.21-10~build0.18.04.1) ...
正在选中未选择的软件包 minicom。
正准备解包 .../minicom_2.7.1-1_amd64.deb  ...
正在解包 minicom (2.7.1-1) ...
正在设置 minicom (2.7.1-1) ...
正在设置 lrzsz (0.12.21-10~build0.18.04.1) ...
正在处理用于 gnome-menus (3.13.3-11ubuntu1.1) 的触发器 ...
正在处理用于 mime-support (3.60ubuntu1) 的触发器 ...
正在处理用于 desktop-file-utils (0.23-1ubuntu3.18.04.2) 的触发器 ...
正在处理用于 man-db (2.8.3-2ubuntu0.1) 的触发器 ...
```

图 2.4.1.5 安装 minicom 软件

图 2.4.1.5 就是安装 minicom 这个软件的过程，在图 2.4.1.5 中安装的过程中，会有如下所示询问：

```
您希望继续执行吗？[Y/n]
```

如果希望继续执行的话就输入 y，如果不希望继续执行的话就输入 n。安装完成以后我们直接在终端输入如下命令打开 minicom 这个串口软件：

```
minicom -s
```

打开以后的 minicom 软件如图 2.4.1.6 所示：

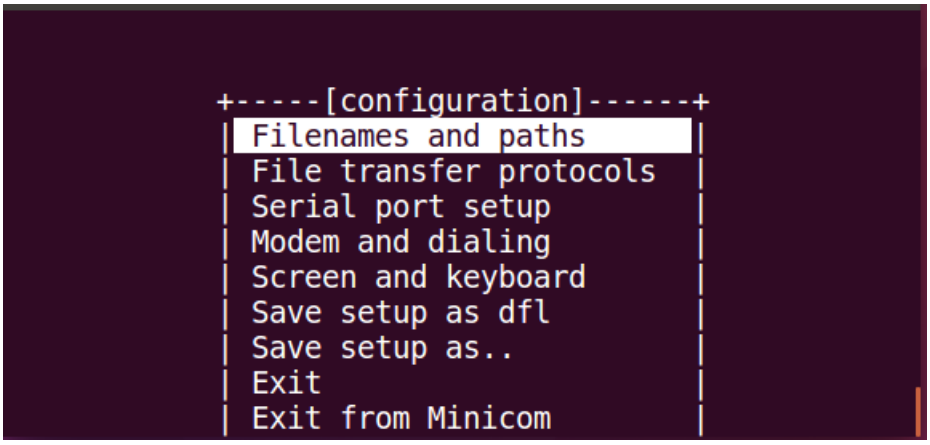


图 2.4.1.6 minicom 软件

关于 minicom 的使用大家可以上网搜索一下，这里就不详细讲解了，要退出 minicom 可以直接按下 ESC 键

4、软件更新

有时候我们需要更新软件，更新软件的话使用命令：

```
sudo apt-get upgrade package-name
```

其中 package-name 为要升级的软件名字，比如我们升级刚刚安装的 minicom 这个软件，如图 2.4.1.7 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get upgrade minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
minicom 已经是最新版 (2.7.1-1)。
```

图 2.4.1.7 更新 minicom 软件

从图 2.4.1.7 可以看出，minicom 已经是最新的了，不用更新，不过有其它软件需要更新，因此会自动更新其它的软件。

5、卸载软件

如果要卸载某个软件的话使用如下命令：

```
sudo apt-get remove package-name
```

其中 package-name 是要卸载的软件，比如卸载前面安装的 minicom 这个软件，操作如图 2.4.1.8 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get remove minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
下列软件包是自动安装的并且现在不需要了:
  lrzsz
使用'sudo apt autoremove'来卸载它(它们)。
下列软件包将被【卸载】:
  minicom
升级了 0 个软件包，新安装了 0 个软件包，要卸载 1 个软件包，有 0 个软件包未被升级。
解压缩后将会空出 940 kB 的空间。
您希望继续执行吗？ [Y/n] y
(正在读取数据库 ... 系统当前共安装有 167303 个文件和目录。)
正在卸载 minicom (2.7.1-1) ...
正在处理用于 desktop-file-utils (0.23-1ubuntu3.18.04.2) 的触发器 ...
正在处理用于 man-db (2.8.3-2ubuntu0.1) 的触发器 ...
正在处理用于 gnome-menus (3.13.3-11ubuntu1.1) 的触发器 ...
正在处理用于 mime-support (3.60ubuntu1) 的触发器 ...
```

图 2.4.1.8 卸载软件

从图 2.4.1.8 中可以看出软件 minicom 被卸载掉了。关于 APT 下载工具就讲解到这里，我们用的最多的就是“sudo apt-get install package-name”来下载和安装软件。有关 Ubuntu 其它的安装软件的方法打开可以自行上网查阅学习，这里就不一一详解了。

2.5 Ubuntu 下文本编辑

2.5.1 Gedit 编辑器

进行文本编辑是最常用的操作，Windows 下我们会使用记事本来完成，或者其它一些优秀的文本编辑器，比如 notepad++，Ubuut 下有一个自带的文本编辑器，那就是 Gedit。Gedit 是一个窗口式的编辑器，关于 Gedit 的使用前面我们已经讲解了。本节我们重点讲解的是另外一个编辑器：VI/VIM 编辑器。

2.5.2 VI/VIM 编辑器

我们如果要在终端模式下进行文本编辑或者修改文件就可以使用 VI/VIM 编辑器，Ubuntu

自带了 VI 编辑器，但是 VI 编辑器对于习惯了 Windows 下进行开发的人来说不方便，比如竟然不能使用键盘上的上下左右键调整光标位置。因此我推荐大家使用 VIM 编辑器，VIM 编辑器是 VI 编辑器升级版本，VI/VIM 编辑器都是一种基于指令式的编辑器，不需要鼠标，也没有菜单，仅仅使用键盘来完成所有的编辑工作。

我们需要先安装 VIM 编辑器，命令如下：

```
sudo apt-get install vim
```

安装完成以后就可以使用 VIM 编辑器了，VIM 编辑器有 3 种工作模式：输入模式、指令模式和底行模式，通过切换不同的模式可以完成不同的功能，我们就以编辑一个文本文档为例讲解 VIM 编辑器的使用。打开终端，输入命令：vi test.txt，如图 2.5.2.1 所示：

```
zuozhongkai@ubuntu:~$ vim test.txt
```

图 2.5.2.1 新建 test.txt 文档

在终端中输入图 2.5.2.1 中所示的命令以后就会创建一个 test.txt 文档，并且用 VIM 打开了，如图 2.5.2.2 所示：

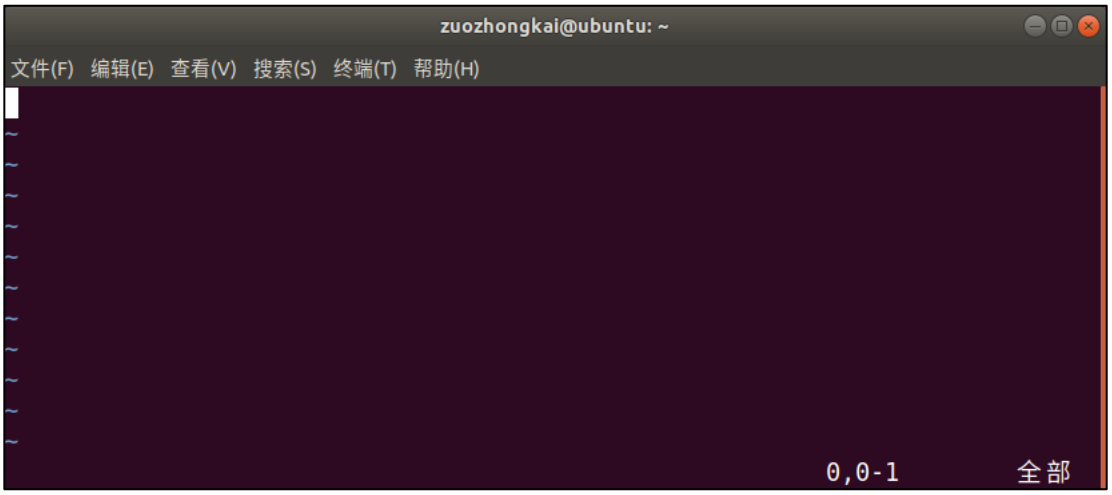


图 2.5.2.2 VIM 打开的 test.txt 文档

我们试着在图 2.5.2.2 中输入数字，发现根本没法输入，这不是因为你的键盘坏了。因为 VIM 默认是以只读模式打开的文档，因此我们要切换到输入模式，切换到输入模式的命令如下：

i ：在当前光标所在字符的前面，转为输入模式。
I ：在当前光标所在行的行首转换为输入模式。
a ：在当前光标所在字符的后面，转为输入模式。
A ：在光标所在行的行尾，转换为输入模式。
o ：在当前光标所在行的下方，新建一行，并转为输入模式。
O ：在当前光标所在行的上方，新建一行，并转为输入模式。
s ：删除光标所在字符。
r ：替换光标处字符。

最常用的就是“a”，我们在图 2.5.2.2 中按下键盘上的“a”键，这时候终端左下角会提示“插入”字样，表示我们进入到了输入模式，如图 2.5.2.3 所示：

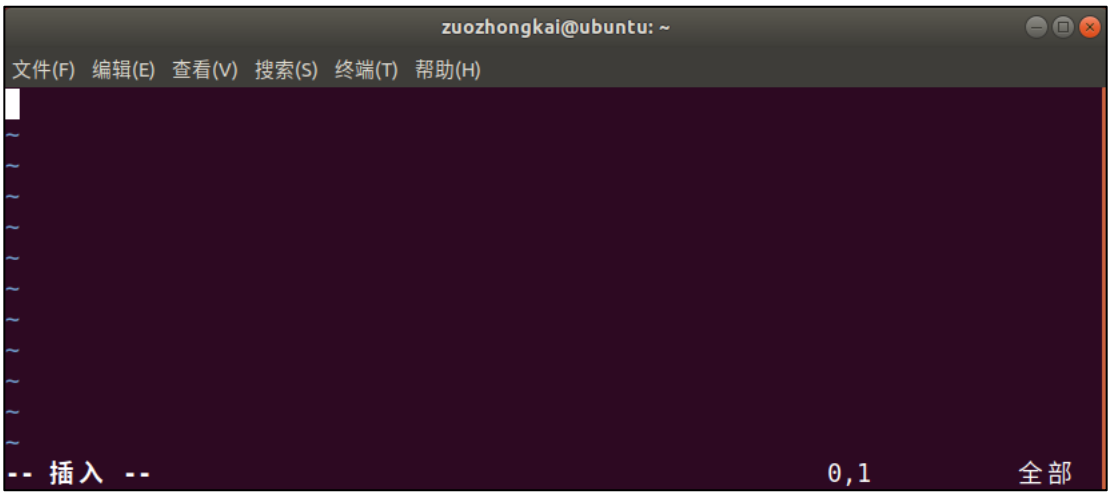


图 2.5.2.3 切换到插入模式

图 2.5.2.3 表明我们可以正常输入文本了，我们可以输入图 2.5.2.4 所示文本：

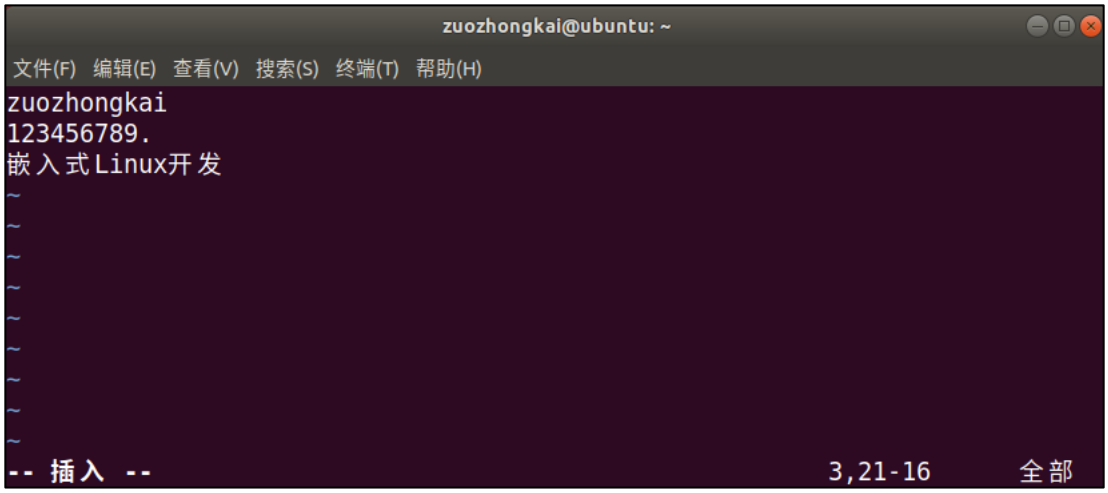


图 2.5.2.4 输入文本

在图 2.5.2.4 中我们在 test.txt 中输入了字母、数字和中文，我们输入完成以后需要保存文本啊，Windows 下的记事本可以使用快捷键 Ctrl+S 来保存，VIM 是否也可以使用 Ctrl+S 来保存呢？你会发现当你按下 Ctrl+S 键以后你的终端不能操作了!!!这是因为在 Ubuntu 下 Ctrl+S 快捷键不是用来完成保存的功能的，而是暂停该终端！所以你一旦在使用终端的时候按下 Ctrl+S 快捷键，那么你的终端肯定不会再有任何反应，如果你按下 Ctrl+S 关闭了当前终端的话可以按下 Ctrl+Q 来重新打开终端。

既然 Ctrl+S 不能保存文本文档，那么有没有其它方法保存文本文档呢？肯定是有的，我们需要从 VIM 现在的输入模式切换到指令模式，方式就是按下键盘的 ESC 键，按下 ESC 键以后终端坐下角的“插入”字样就会消失，此时你就不能在输入任何文本了，如果想再次输入文本的话就按下“a”键重新进入到输入模式。指令模式顾名思义就是输入指令的模式，这些指令是控制文本的指令，我们将这些指令进行分类，如下所示：

1、移动光标指令：

h(或左方向键) ：光标左移一个字符。
l(或右方向键) ：光标右移一个字符。
j(或下方向键) ：光标下移一行。
k(或上方向键) ：光标上移一行。

nG ：光标移动到第 n 行首。
n+ ：光标下移 n 行。
n- ：光标上移 n 行。

2、屏幕翻滚指令

Ctrl+f ：屏幕向下翻一页，相当于下一页。
Ctrl+b ：屏幕向上翻一页，相当于上一页。

3、复制、删除和粘贴指令

cc ：删除整行，并且修改整行内容。
dd ：删除该行，不提供修改功能。
ndd ：删除当前行向下 n 行。
x ：删除光标所在的字符。
X ：删除光标前面的一个字符。
nyy ：复制当前行及其下面 n 行。
p ：粘贴最近复制的内容。

上面就是 VI/VIM 的命令模式下最常用的一些命令，还有一些不常用的我没有列出来，感兴趣的可以自行上网查阅。从上面的命令可以看出，并没有保存文本的命令，那是因为保存文档的命令是在底行模式中，我们要先进入到指令模式，进入底行模式的方式是先进入指令模式下，然后在指令模式下输入“：”进入底行模式，如图 2.5.2.5 所示：

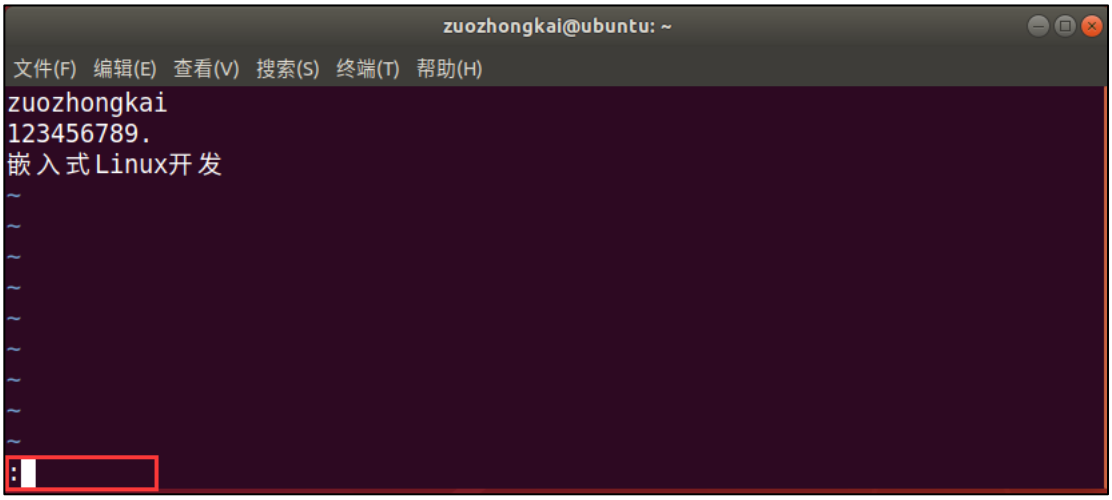


图 2.5.2.5 “:”底行模式

在图 2.5.2.5 中当进入底行模式以后会在终端的左下角就会出现符号“：”，我们可以在“：”后面输入命令，常用的命令如下：

x ：保存当前文档并且退出。
q ：退出。
w ：保存文档。
q! ：退出 VI/VIM，不保存文档。

如果我们要退出并保存文本的话需要在“：”底行模式下输入“wq”，如图 2.5.2.6 所示：

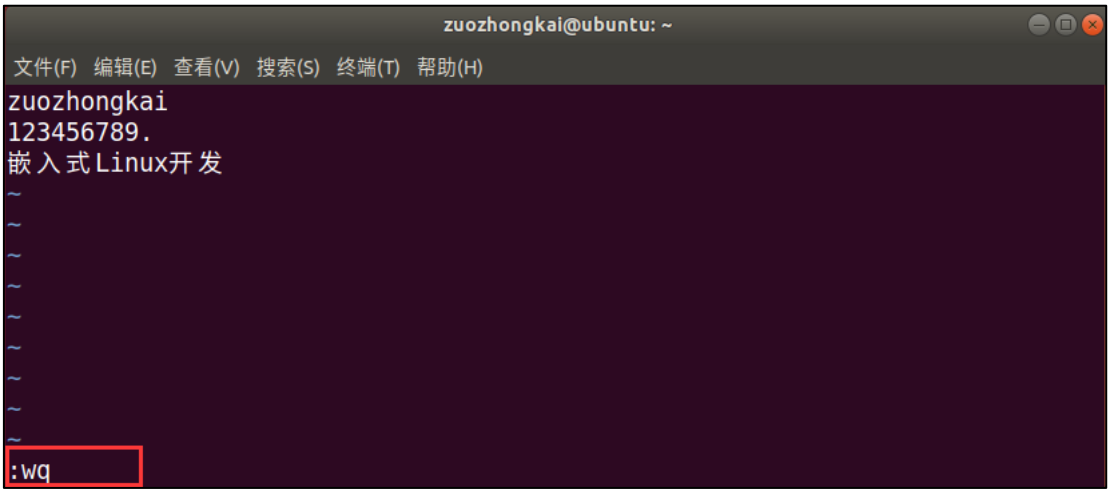


图 2.5.2.6 保存并退出 VIM

在“：”底行模式下输入“wq”以后按下回车键就保存 test.txt 并退出 VI/VIM 编辑器，退出以后我们可以使用命令“cat”来查看刚刚新建的 test.txt 文档的内容，如图 2.5.2.7 所示：

```
zuozhongkai@ubuntu:~$ cat test.txt
zuozhongkai
123456789.
嵌入式Linux开发
```

图 2.5.2.7 查看文档内容

从图 2.5.2.7 中可以看出，test.txt 中的内容就是我们用 VIM 输入的内容，至此我们就完整的进行了一遍 VI/VIM 创建文档、编辑文档和保存文档。

在上面讲解进入 VIM 的底行模式的时候之说了在指令模式下输入“：”的方法，还可以在指令模式下输入“/”进入底行模式，输入“/”以后如图 2.5.2.8 所示。

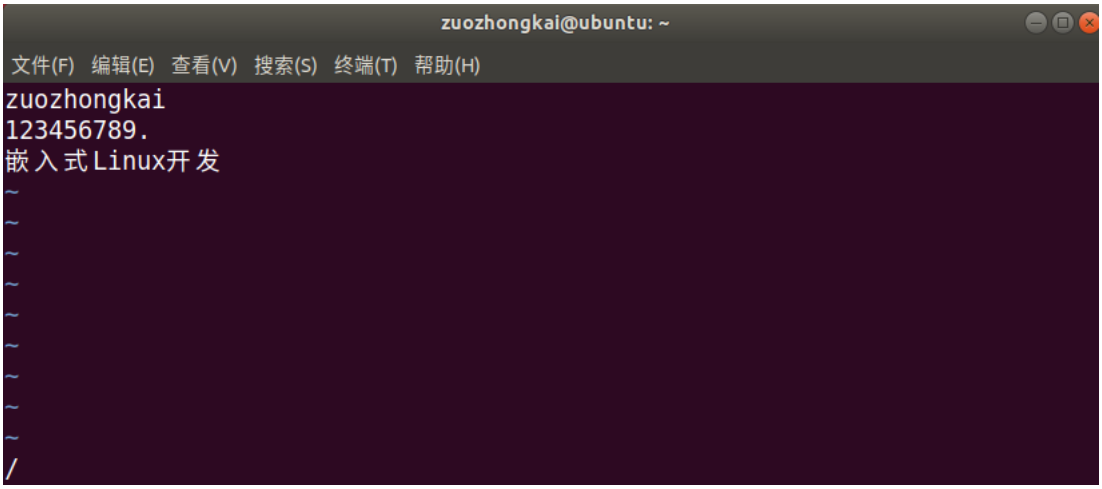


图 2.5.2.8 “/”底行模式

在“/”底行模式下我们可以在文本中搜索指定的内容，比如搜索 test.txt 文件中“嵌入式”三个字，使用方法如图 2.5.2.9 所示：

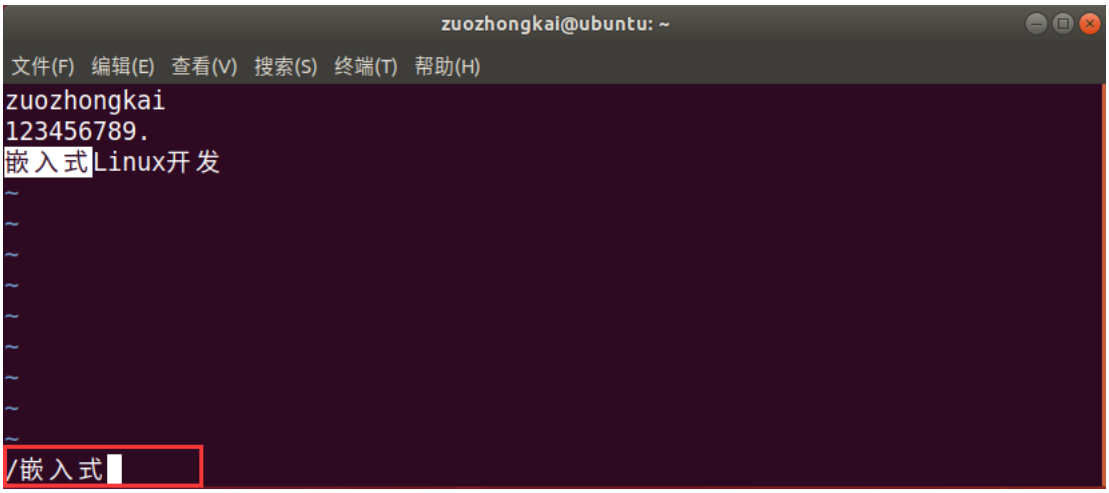


图 2.5.2.9 搜索文本

在“/”后面输入要搜索的内容，然后按下回车键就会在 test.txt 中找到与字符串“嵌入式”匹配的部分，如图 2.5.2.10 所示：

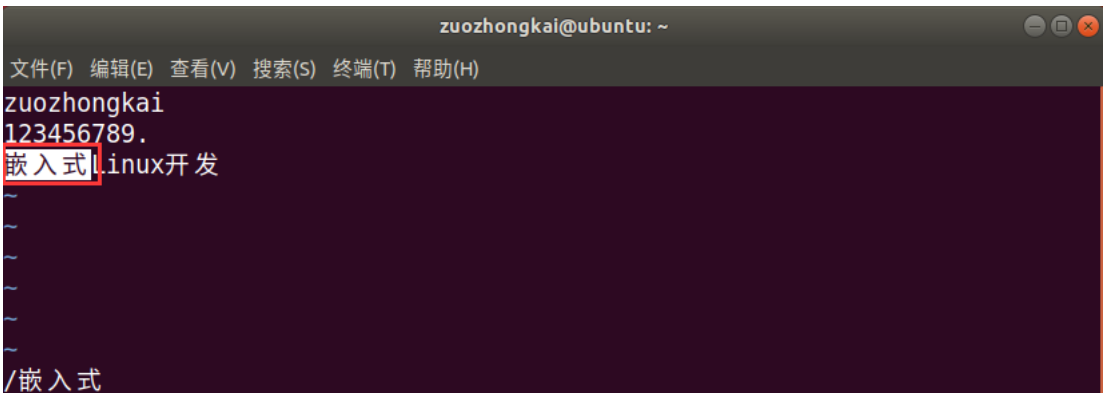


图 2.5.2.10 查找到指定内容

图 2.5.2.10 中可以看出，在 test.txt 中找到了“嵌入式”这个词，并且标记出来位置。我们以后要在一个文档中搜素是否存在某个字符串的时候就可以使用这种方法。有关 VI/VIM 编辑器的讲解就到这里，我们完整的练习了一遍如何使用 VIM 创建文档、编辑文档、保存文档和在文档中搜索字符串。有关更多更详细的 VIM 编辑器的操作大家自行上网查阅相关文档和博客。

2.6 Linux 文件系统

操作系统的基本功能之一就是文件管理，而文件的管理是由文件系统来完成的。Linux 支持多种文件系统，本节我们就来讲解 Linux 下的文件系统、文件系统类型、文件系统结构和文件系统相关 Shell 命令。

2.6.1 Linux 文件系统简介以及类型

1、Linux 文件系统简介

操作系统就是处理各种数据的，这些数据在硬盘上就是二进制，人类肯定不能直接看懂这些二进制数据，要有一个翻译器，将这些二进制的数据还原为人类能看懂的文件形式，这个工作就是由文件系统来完成的，文件系统的目的就是实现数据的查询和存储，由于使用场合、使用环境的不同，Linux 有多种文件系统，不同的文件系统支持不同的体系。文件系统是管理数据

的，而可以存储数据的物理设备有硬盘、U 盘、SD 卡、NAND FLASH、NOR FLASH、网络存储设备等。不同的存储设备其物理结构不同，不同的物理结构就需要不同的文件系统去管理，比如管理 NAND FLASH 的话使用 YAFFS 文件系统，管理硬盘、SD 卡的话就是 ext 文件系统等等。

我们在使用 Windows 的时候新买一个硬盘回来一般肯定是将这个硬盘分为好几个盘，比如 C 盘、D 盘等等。这个叫磁盘的分割，Linux 下也支持磁盘分割，Linux 下常用的磁盘分割工具为：fdisk，fdisk 这个工具我们后面会详细讲解怎么用，因为我们移植 Linux 的时候需要将 SD 卡分为三个分区来存储不同的东西。在 Windows 下我们创建一个新的盘符以后都要做格式化处理，格式化其实就是给这个盘符创建文件系统的过程，我们在 Windows 格式化某个盘的时候都会让你选择文件系统，如图 2.6.1.1 所示：

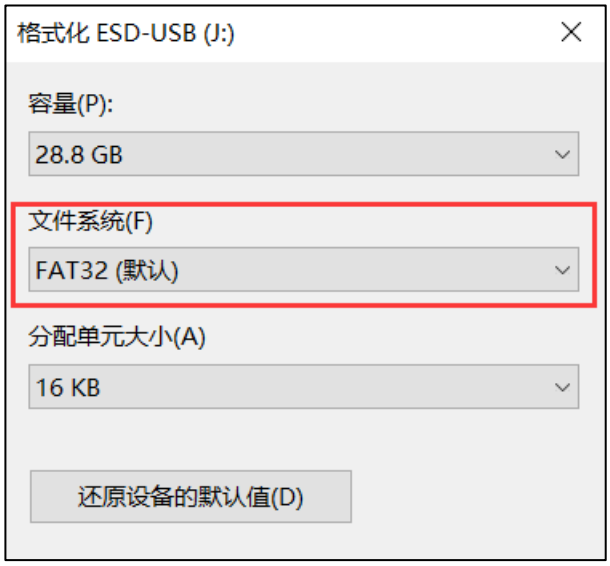


图 2.6.1.1 格式化磁盘

图 2.6.1.1 就是格式化磁盘的时候选择文件系统，Windows 下一般有 FAT、NTFS 和 exFAT 这些文件系统。同样的，在 Linux 下我们使用 fdisk 创建好分区以后也是要先在创建好的分区上面创建文件系统，也就是格式化。

在 Windows 下有磁盘分区的概念，比如 C，D，E 盘等，在 Linux 下没有这个概念，因此 Linux 下你找不到像 C、D、E 盘这样的东西。前面我们说了 Linux 下可以给磁盘分割，但是没有 C、D、E 盘那怎么访问这些分区呢？在 Linux 下创建一个分区并且格式化好以后我们要将其“挂载”到一个目录下才能访问这个分区。Windows 的文件系统挂载过程是其内部完成的，用户是看不到的，Linux 下我们使用 mount 命令来挂载磁盘。挂载磁盘的时候是需要确定挂载点的，也就是你的这个磁盘要挂载到哪个目录下。

2、Linux 文件系统类型

前面我们说了，在 Windows 下有 FAT、NTFS 和 exFAT 这样的文件系统，在 Linux 下又有哪些文件系统呢，Linux 下的文件系统主要有 ext2、ext3、ext4 等文件系统。Linux 还支持其他的 UNIX 文件系统，比如 XFS、JFS、UFS 等，也支持 Windows 的 FAT 文件系统和网络文件系统 NFS 等。这里我们主要讲一下 Linux 自带的 ext2、ext3 和 ext4 文件系统。

ext2 文件系统：

ext2 是 Linux 早期的文件系统，但是随着技术的发展 ext2 文件系统已经不推荐使用了，ext2 是一个非日志文件系统,大多数的 Linux 发行版都不支持 ext2 文件系统了。

ext3 文件系统：

ext3 是在 ext2 的基础上发展起来的文件系统，完全兼容 ext2 文件系统，ext3 是一个日志文件系统，ext3 支持大文件，ext3 文件系统的特点有如下：

高可靠性：使用 ext3 文件系统的话，即使系统非正常关机、发生死机等情况，恢复 ext3 文件系统也只需要数十秒。

数据完整性：ext3 提高了文件系统的完整性，避免意外死机或者关机对文件系统的伤害。

文件系统速度：ext3 的日志功能对磁盘驱动器读写头进行了优化，文件系统速度相对与 ext2 来说没有降低。

数据转换：从 ext2 转换到 ext3 非常容易，只需要两条指令就可以完成转换。用户不需要花时间去备份、恢复、格式化分区等，用 ext3 文件系统提供的工具 tune2fs 即可轻松的将 ext2 文件系统转换为 ext3 日志文件系统。ext3 文件系统不需要经过任何修改，可以直接挂载成 ext2 文件系统。

ext4 文件系统：

ext4 文件系统是在 ext3 上发展起来的，ext4 相比与 ext3 提供了更佳的性能和可靠性，并且功能更丰富，ext4 向下兼容 ext3 和 ext2，因此可以将 ext2 和 ext3 挂载为 ext4。那么我们安装的 Ubuntu 使用的哪个版本的文件系统呢？在终端中输入如下命令来查询当前磁盘挂载的啥文件系统：

```
df  -T  -h
```

结果如图 2.6.1.2 所示：

图 2.6.1.2 Ubuntu 使用的文件系统

在图 2.6.1.2 中，框起来的就是我们安装 Ubuntu 的这个磁盘，在 Linux 下一切皆为文件，“/dev/sda1”就是我们的磁盘分区，可以看出这个磁盘分区类型是 ext4，它的挂载点是“/”，也就是根目录。

2.6.2 Linux 文件系统结构

在 Windows 下直接打开 C 盘，我们进入的就是 C 盘的根目录，打开 D 盘进入的就是 D 盘的根目录，比如 C 盘根目录如下：

图 2.6.2.1 C 盘根目录

在 Linux 下因为没有 C、D 盘之说，因此 Linux 只有一个根目录，没有 C 盘根目录、D 盘根目录之类的。其实如果你的 Windows 只有一个 C 盘的话那么整个系统也就只有一个根目录。Windows 下的 C 盘根目录就是“C:”,在 Linux 下的根目录就是“/”，你没有看错，Linux 根目录就是用“/”来表示的，打开 Ubuntu 的文件浏览器，文件浏览器在左侧的导航栏，图标如图 2.6.2.2 所示：

图 2.6.2.2 文件浏览器

打开以后的文件浏览器如图 2.6.2.3 所示：

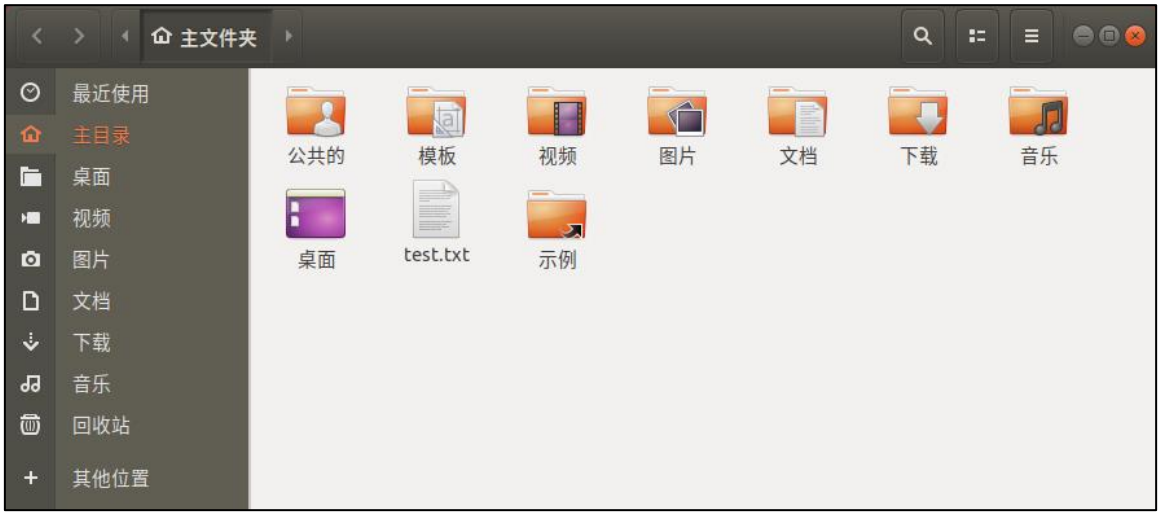


图 2.6.2.3 文件浏览器

直接打开文件浏览器以后，我们默认不是处于根目录中的，不像 Windows，我们直接打开 C 盘就处于 C 盘根目录下。Ubuntu 是支持多用户的，Ubuntu 为每个用户创建了一个根目录，比

如我电脑现在登陆的是“zuozhongkai”这个用户，因此默认进入的是“zuozhongkai”这个用户的根目录。我们点击图 2.6.2.3 中左侧的“其他位置”，选择“计算机”，如图 2.6.2.4 所示：

图 2.6.2.4 打开“计算机”

打开以后如图 2.6.2.5 所示：

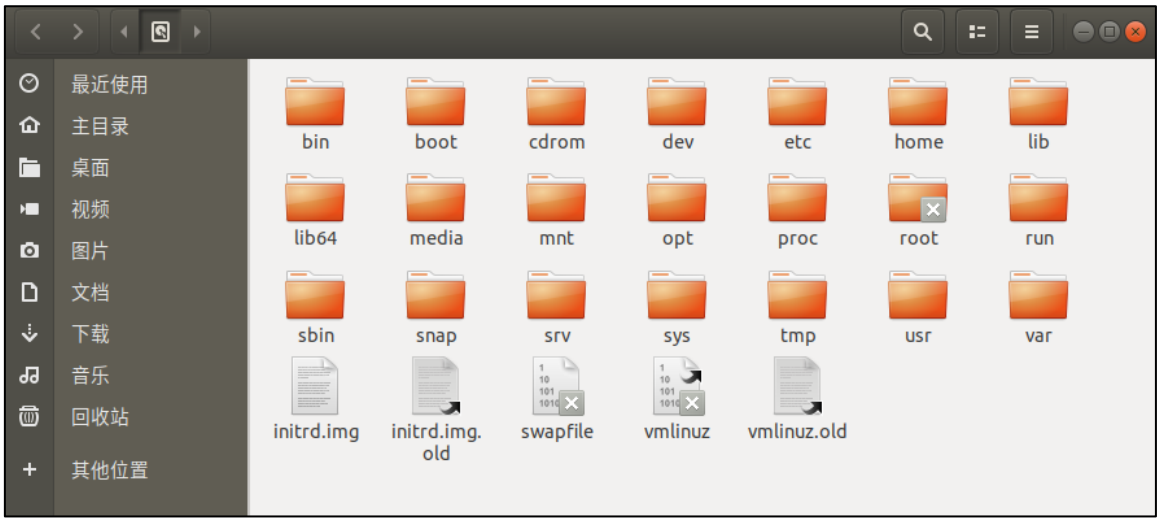


图 2.6.2.5 根目录“/”

图 2.6.2.5 就是 Ubuntu 的根目录“/”，这时候肯定就有人有疑问，刚刚说 Ubuntu 会给每个用户创建一个根目录，那这些用户的根目录在哪里？是不是和根目录“/”是一个地位的？这个所谓的用户根目录其实就是“/”下的一个文件夹，以我的“zuozhongkai”这个用户为例，其用户根目录就是：/home/zuozhongkai。只要你创建了一个用户，那么系统就会在/home 这个目录下创建一个以这个用户名命名的文件夹，这个文件夹就是这个用户的根目录。

用户可以对自己的用户根目录下的文件进行随意的读写操作，但是如果要修改根目录“/”下的文件就会提示没有权限。打开终端以后默认进入的是当前用户根目录，比如我们打开终端以后输入“ls”命令查看当前目录下有什么文件，结果如图 2.6.2.6 所示：

```
zuozhongkai@ubuntu:~$ ls
examples.desktop  test.txt  公共的  模板  视频  图片  文档  下载  音乐  桌面
```

图 2.6.2.6 目录查看

可以看出图 2.6.2.6 中的文件和图 2.6.2.3 中的一模一样，都是“zuozhongkai”这个账户的根目录。我们来看一下根目录“/”下都有哪些文件，在终端中输入如下命令：

```
cd  /    //进入到根目录“/
ls       //查看根目录“/”下的文件以及文件夹
```

执行上述两行命令以后，终端如图 2.6.2.7 所示：

```
zuozhongkai@ubuntu:~$ cd /
zuozhongkai@ubuntu:/$ ls
bin   dev   initrd.img      lib64       mnt   root  snap      sys  var
boot  etc   initrd.img.old  lost+found  opt   run   srv       tmp  vmlinuz
cdrom home  lib             media       proc  sbin  swapfile  usr  vmlinuz.old
```

图 2.6.2.7 查看根目录“/”

图 2.6.2.7 中列举出了根目录“/”下面的所有文件夹，这里我们仔细观察一下，当我们进入到根目录“/”里面以后终端提示符“$”前面的符号“~”变成了“/”，这是因为当我们在终端中切换了目录以后“$”前面就会显示切换以后的目录路径。我们来看一下根目录“/”中的一些重要的文件夹：

/bin ：存储一些二进制可执行命令文件，/usr/bin 也存放了一些基于用户的命令文件。
/sbin ：存储了很多系统命令，/usr/sbin 也存储了许多系统命令。
/root ：超级用户 root 的根目录文件。
/home ：普通用户默认目录，在该目录下，每个用户都有一个以本用户名命名的文件夹。
/boot ：存放 Ubuntu 系统内核和系统启动文件。
/mnt ：通常包括系统引导后被挂载的文件系统的挂载点。
/dev ：存放设备文件，我们后面学习 Linux 驱动主要是跟这个文件夹打交道的。
/etc ：保存系统管理所需的配置文件和目录。
/lib ：保存系统程序运行所需的库文件，/usr/lib 下存放了一些用于普通用户的库文件。
/lost+found ：一般为空，当系统非正常关机以后，此文件夹会保存一些零散文件。
/var ：存储一些不断变化的文件，比如日志文件
/usr ：包括与系统用户直接有关的文件和目录，比如应用程序和所需的库文件。
/media ：存放 Ubuntu 系统自动挂载的设备文件。
/proc ：虚拟目录，不实际存储在磁盘上，通常用来保存系统信息和进程信息。
/tmp ：存储系统和用户的临时文件，该文件夹对所有的用户都提供读写权限。
/opt ：可选文件和程序的存放目录。
/sys ：系统设备和文件层次结构，并向用户程序提供详细的内核数据信息。

2.6.2 文件操作命令

本节我们来学习一下在终端进行文件操作的一些常用命令：

1、创建新文件命令—touch

在前面学习 VIM 的时候我们知道可以用 vi 指令来创建一个文本文档，本节我们就学习一个功能更全面的文件创建命令—touch。touch 不仅仅可以用用来创建文本文档，其它类型的文档也可以创建，命令格式如下：

```
touch  [参数]  [文件名]
```

使用 touch 创建文件的时候，如果[文件名]的文件不存在，那就直接创建一个以[文件名]命名的文件，如果[文件名]文件存在的话就仅仅修改一下此文件的最后修改日期，常用的命令参

数如下：

-a ：只更改存取时间。

-c ：不建立任何文件。

-d<日期> ：使用指定的日期，而并非现在日期。

-t<时间> ：使用指定的时间，而并非现在时间。

进入到用户根目录下，直接使用命令“cd~”即可快速进入用户根目录，进入用户根目录以后使用 touch 命令创建一个名为 test 的文件，创建过程如图 2.6.2.1 所示：

```
zuozhongkai@ubuntu:~$ cd ~   //进入用户根目录
zuozhongkai@ubuntu:~$ ls     //查看当前目录下的文件
examples.desktop  tmp      模板   图片   下载   桌面
test.txt          公共的   视频   文档   音乐
zuozhongkai@ubuntu:~$ touch test //创建test文件
zuozhongkai@ubuntu:~$ ls test    //查看创建的test文件
test
zuozhongkai@ubuntu:~$ ls test -l //查看创建的test文件详细信息
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  22 01:24 test
```

图 2.6.2.1 touch 命令操作

2、文件夹创建命令—mkdir

既然可以创建文件，那么肯定也可以创建文件夹，创建文件夹使用命令“mkdir”，命令格式如下：

```
mkdir   [参数]   [文件夹名目录名]
```

主要参数如下：

-p ：如所要创建的目录其上层目录目前还未创建，那么会一起创建上层目录。

我们在用户根目录下创建两个分别名为“testdir1”和“testdir2”的文件夹，操作如图 2.6.2.2 所示：

```
zuozhongkai@ubuntu:~$ ls  //查看当前目录下的文件
examples.desktop  test.txt  公共的   视频   文档   音乐
test              tmp       模板     图片   下载   桌面
zuozhongkai@ubuntu:~$ mkdir testdir1 //创建testdir1文件夹
zuozhongkai@ubuntu:~$ mkdir testdir2 //创建testdir2文件夹
zuozhongkai@ubuntu:~$ ls             //查看当前目录下的所有文件，看看文件夹创建是否成功
examples.desktop  testdir1  test.txt  公共的   视频   文档   音乐
test              testdir2  tmp       模板     图片   下载   桌面
```

图 2.6.2.2 创建文件夹

在图 2.6.2.2 中，我们使用命令“mkdir”创建了“testdir1”和“testdir2”这两个文件夹。

3、文件及目录删除命令—rm

既然有创建文件的命令，那肯定有删除文件的命令，要删除一个文件或者文件夹可以使用命令“rm”，此命令可以完成删除一个文件或者多个文件及文件夹，它可以实现递归删除。对于链接文件，只删除链接，原文件保持不变，所谓的链接文件，其实就是 Windows 下的快捷方式文件，此命令格式如下：

```
rm  [参数]   [目的文件或文件夹目录名]
```

命令主要参数如下：

-d ：直接把要删除的目录的硬连接数据删成 0，删除该目录。

-f ：强制删除文件和文件夹(目录)。

-i ：删除文件或者文件夹(目录)之前先询问用户。

-r ：递归删除，指定文件夹(目录)下的所有文件和子文件夹全部删除掉。

-v ：显示删除过程。

我们使用 rm 命令来删除前面使用命令“touch”创建的 test 文件，操作过程如图 2.6.2.3 所示：

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop  testdir1  test.txt  公共的  视频  文档  音乐
test              testdir2  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ rm test //删除文件test
zuozhongkai@ubuntu:~$ ls      //查看当前目录，看看test文件是否删除
examples.desktop  testdir2  tmp     模板  图片  下载  桌面
testdir1          test.txt  公共的  视频  文档  音乐
```

图 2.6.2.3 删除文件

命令“rm”也可以直接删除文件夹，我们可以试一下删除前面创建的 testdir1 文件夹，先直接使用命令“rm testdir1”测试一下是否可以删除，结果如图 2.6.2.4 所示：

```
zuozhongkai@ubuntu:~$ rm testdir1
rm: 无法删除'testdir1': 是一个目录
```

图 2.6.2.4 删除文件夹

在图 2.6.2.4 中可以看出，直接使用命令“rm”是无法删除文件夹(目录)的，我们需要加上参数“-rf”，也就是强制递归删除文件夹(目录)，操作结果如图 2.6.2.5 所示：

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop  testdir2  tmp     模板  图片  下载  桌面
testdir1          test.txt  公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ rm testdir1 -rf //使用参数“-rf”强制递归删除文件夹
zuozhongkai@ubuntu:~$ ls              //查看删除文件夹以后的当前目录
examples.desktop  test.txt  公共的  视频  文档  音乐
testdir2          tmp       模板    图片  下载  桌面
```

图 2.6.2.5 带参数删除文件夹

从图 2.6.2.5 可以看出，当在命令“rm”中加入参数“-rf”以后就可以删除掉文件夹“testdir1”了。

4、文件夹(目录)删除命令—rmdir

上面我们讲解了如何使用命令“rm”删除文件夹，那就是要加上参数“-rf”，其实 Linux 提供了直接删除文件夹(目录)的命令—rmdir，它可以不加任何参数的删除掉指定的文件夹(目录)，命令格式如下：

```
rmdir [参数] [文件夹(目录)]
```

命令主要参数如下：

-p 删除指定的文件夹(目录)以后，若上层文件夹(目录)为空文件夹(目录)的话就将其一起删除。

我们使用命令“rmdir”删除掉前面创建的“testdir2”文件夹，操作过程如图 2.6.2.6 所示：

```
zuozhongkai@ubuntu:~$ ls   //查看当前目录下的所有文件
examples.desktop  test.txt  公共的  视频  文档  音乐
testdir2          tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ rmdir testdir2 //删除文件夹testdir2
zuozhongkai@ubuntu:~$ ls             //查看删除文件夹以后的目录文件
examples.desktop  tmp     模板  图片  下载  桌面
test.txt          公共的  视频  文档  音乐
```

图 2.6.2.6 命令 rmdir 删除文件夹

5、文件复制命令—cp

在 Windows 下我们可以通过在文件上点击鼠标右键来进行文件的复制和粘贴，在 Ubuntu 下我们也可以通过点击文件右键进行文件的复制和粘贴。但是本节我们来讲解如何在终端下使用命令来进行文件的复制，Linux 下的复制命令为“cp”，命令描述如下：

```
cp [参数] [源地址] [目的地址]
```

主要参数描述如下：

-a ：此参数和同时指定“-dpR”参数相同

-d ：在复制有符号连接的文件时，保留原始的连接。

-f ：强行复制文件，不管要复制的文件是否已经存在于目标目录。

-I ：覆盖现有文件之前询问用户。

-p ：保留源文件或者目录的属性。

-r 或-R ：递归处理，将指定目录下的文件及子目录一并处理

我们在用户根目录下，使用前面讲解的命令“mkdir”创建两个文件夹：test1 和 test2，过程如图 2.6.2.7 所示。

```
zuozhongkai@ubuntu:~$ ls  //查看当前目录下的文件
examples.desktop  tmp     模板  图片  下载  桌面
test.txt          公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ mkdir test1 test2 //创建test1和test2两个文件夹
zuozhongkai@ubuntu:~$ ls                //查看当前目录下的所有文件
examples.desktop  test2     tmp     模板  图片  下载  桌面
test1             test.txt  公共的  视频  文档  音乐
```

图 2.6.2.7 创建 test1 和 test2 两个文件夹

进入上面创建的 test1 文件夹，然后在 test1 文件夹里面创建一个 a.c 文件，操作过程如图 2.6.2.8 所示：

```
zuozhongkai@ubuntu:~$ cd test1          //进入test1文件夹
zuozhongkai@ubuntu:~/test1$ touch a.c //创建a.c文件
zuozhongkai@ubuntu:~/test1$ ls          //查看当前目录下所有文件
a.c
```

图 2.6.2.8 创建 a.c 文件

我们先将图 2.6.2.8 中的 a.c 这个文件做个备份，也就是复制到同文件夹 test1 里面，新的文件命名为 b.c。然后在将 test1 文件夹中的 a.c 和 b.c 这两个文件都复制到文件夹 test2 中，操作如图 2.6.2.9 所示

```
zuozhongkai@ubuntu:~/test1$ cp a.c b.c      //拷贝a.c到本文件夹中，并重命名为b.c
zuozhongkai@ubuntu:~/test1$ ls              //查看当前目录下的所有文件
a.c  b.c
zuozhongkai@ubuntu:~/test1$ cp *.c ../test2 //拷贝a.c和b.c到文件夹test2中
zuozhongkai@ubuntu:~/test1$ cd ../test2     //进入上级目录中的test2文件夹
zuozhongkai@ubuntu:~/test2$ ls              //查看test2文件夹下的文件
a.c  b.c
```

图 2.6.2.9 拷贝文件

在图 2.6.2.9 中，我们添加了一些高级使用技巧，首先是拷贝 a.c 和 b.c 文件到 test2 文件夹中，我们使用了通配符“*”，“*.c”就表示 test1 下的所有以“.c”结尾的文件，也就是 a.c 和 b.c。“../test2”中的“../”表示上级目录，因此“../test2”就是上级目录下的 test2 文件夹。

上面都是文件复制，我们接下来学习一下文件夹复制，我们将 test2 文件夹复制到同目录下，新拷贝的文件夹命名为 test3，操作如图 2.6.2.10 所示：

```
zuozhongkai@ubuntu:~$ cp -rf test2/ test3/  //复制test2文件夹为test3文件夹
zuozhongkai@ubuntu:~$ ls                    //查看当前目录下的所有文件
examples.desktop  test2  test.txt  公共的  视频  文档  音乐
test1             test3  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ cd test3              //进入test3文件夹
zuozhongkai@ubuntu:~/test3$ ls              //查看test3中的所有文件
a.c  b.c
```

图 2.6.2.10 拷贝文件夹

6、文件移动命令—mv

有时候我们需要将一个文件或者文件夹移动到另外一个地方去，或者给一个文件或者文件夹进行重命名，这个时候我们就可以使用命令“mv”了，此命令格式如下：

```
mv [参数] [源地址] [目的地址]
```

主要参数描述如下：

-b ：如果要覆盖文件的话覆盖前先进行备份。

-f ：若目标文件或目录与现在的文件重复，直接覆盖目的文件或目录。

-I ：在覆盖之前询问用户。

使用上面讲解“cp”命令的时候创建了三个文件夹，在上面创建的 test1 文件夹里面创建一个 c.c 文件，然后将 c.c 这个文件重命名为 d.c。最后将 d.c 这个文件移动到 test2 文件夹里面，操作如图 2.6.2.11 所示：

```
zuozhongkai@ubuntu:~$ cd test1          //进入test1文件夹
zuozhongkai@ubuntu:~/test1$ touch c.c //创建c.c文件
zuozhongkai@ubuntu:~/test1$ ls          //查看当前目录下的所有文件
a.c  b.c  c.c
zuozhongkai@ubuntu:~/test1$ mv c.c d.c //移动c.c到当前目录下，并重命名为d.c
zuozhongkai@ubuntu:~/test1$ ls          //查看当前目录下的所有文件
a.c  b.c  d.c
```

图 2.6.2.11 移动文件操作

我们再将 test1 中的 d.c 文件移动到 test2 文件夹里面，操作如图 2.6.2.12 所示：

```
zuozhongkai@ubuntu:~/test2$ ls      //查看test2下所有文件
a.c  b.c
zuozhongkai@ubuntu:~/test2$ cd ../test1   //进入test1文件夹中
zuozhongkai@ubuntu:~/test1$ ls            //查看test1文件夹所有文件
a.c  b.c  d.c
zuozhongkai@ubuntu:~/test1$ mv d.c ../test2 //将d.c移动到test2中
zuozhongkai@ubuntu:~/test1$ ls ../test2     //查看test2下的所有文件
a.c  b.c  d.c
```

图 2.6.2.12 移动文件操作

2.6.3 文件压缩和解压缩

文件的压缩和解压缩是非常常见的操作，在 Windows 下我们有很多压缩和解压缩的工具，比如 zip、360 压缩等等。在 Ubuntu 下也有压缩工具，本节我们学习 Ubuntu 下图形化以及命令行这两种压缩和解压缩操作。

1、图形化压缩和解压缩

图形化压缩和解压缩和 Windows 下基本一样，在要压缩或者解压的文件上点击鼠标右键，然后选择要进行的操作，我们先讲解一下如何进行文件的压缩。首先找到要压缩的文件，然后

在要压缩的文件上点击鼠标右键，选择“压缩”选项，如图 2.6.3.1 所示：

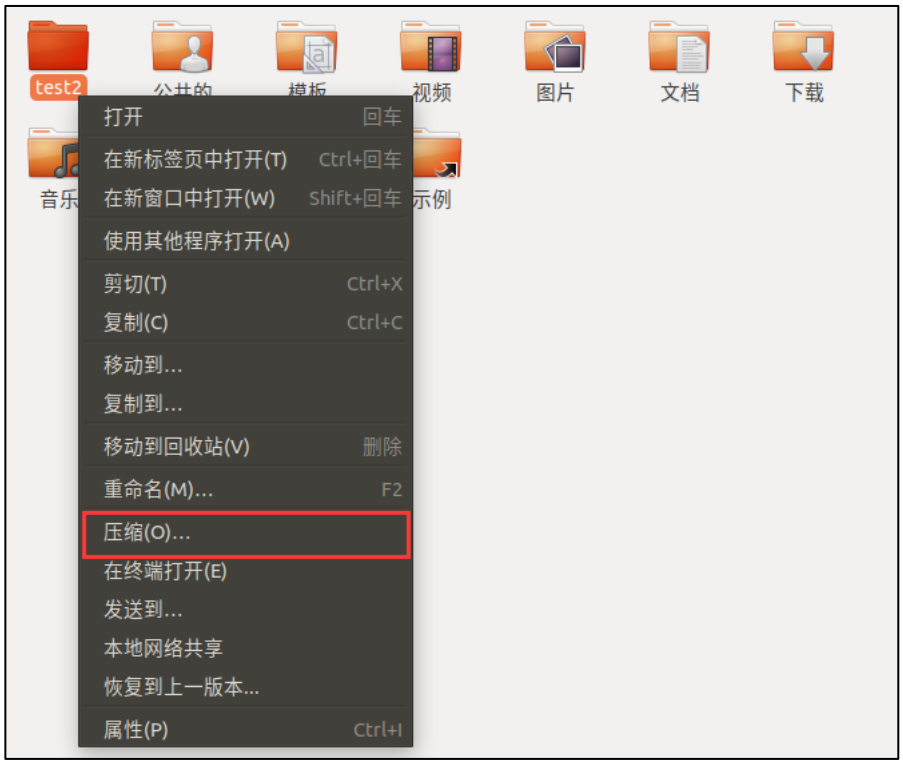


图 2.6.3.1 文件压缩

在图 2.6.3.1 中我们要对 test 这个文件夹进行压缩，点击“压缩”以后会弹出图 2.6.3.2 所示界面让选择压缩后的文件名和压缩格式。

图 2.6.3.2 压缩命名和格式选择

在图 2.6.3.2 中，设置好压缩以后的文件名，然后选择压缩格式，比如我选择的“.zip”这个格式，点击图 2.6.3.2 右上角的“创建”，压缩完成以后如图 2.6.3.3 所示：

图 2.6.3.3 压缩完成的文件

上面就是使用图形化进行文件压缩的过程，我们接下来对刚刚压缩的 test2.zip 进行解压缩，鼠标放到 test2.zip 上然后点击鼠标右键，选择“提取到此处”，如图 2.6.3.4 所示：

图 2.6.3.4 解压缩文件

点击图 2.6.3.4 中的“提取到此处”以后，系统就会自动进行解压缩，上面就是在 Ubuntu 中使用图形化工具进行文件的压缩和解压缩。

2、命令行进行文件的压缩和解压缩

上面我们学习了如何使用图形化工具在 Ubuntu 下进行文件的压缩和解压缩，本节我们学学如何使用命令行进行压缩和解压缩，我们后面的开发中所有涉及到压缩和解压缩的操作都是在命令行下完成的。命令行下进行压缩和解压缩常用的命令有三个：zip、unzip 和 tar，我们依次来学习：

3、命令 zip

zip 命令看名字就知道是针对.zip 文件的，用于将一个或者多个文件压缩成一个.zip 结尾的文件，命令格式如下：

```
zip [参数] [压缩文件名.zip] [被压缩的文件]
```

主要参数函数如下：

-b<工作目录> 指定暂时存放文件的目录。

-d ：从 zip 文件中删除一个文件。

-F ：尝试修复已经损毁的压缩文件。

-g ：将文件压缩入现有的压缩文件中，不需要新建压缩文件。

-h ：帮助。

-j ：只保存文件的名，不保存目录。

-m ：压缩完成以后删除源文件。

-n<字尾符号> ：不压缩特定扩展名的文件。

-q ：不显示压缩命令执行过程。

-r ：递归压缩，将指定目录下的所有文件和子目录一起压缩。

-v ：显示指令执行过程。

-num ：压缩率，为 1~9 的数值。

上面讲解了如何使用图形化压缩工具对文件夹 test2 进行压缩，这里我们使用命令“zip”对 test2 文件夹进行压缩，操作如图 2.6.3.5 所示：

```
zuozhongkai@ubuntu:~$ ls  //显示当前目录下所有文件
examples.desktop  test2  test.txt  公共的  视频  文档  音乐
test1             test3  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ zip -rv test2.zip test2  //使用zip命令进行压缩
  adding: test2/         (in=0) (out=0) (stored 0%)
  adding: test2/d.c      (in=0) (out=0) (stored 0%)
  adding: test2/b.c      (in=0) (out=0) (stored 0%)
  adding: test2/a.c      (in=0) (out=0) (stored 0%)
total bytes=0, compressed=0 -> 0% savings
zuozhongkai@ubuntu:~$ ls                          //显示压缩以后的当前目录下所有文件
examples.desktop  test2      test3     tmp     模板  图片  下载  桌面
test1             test2.zip  test.txt  公共的  视频  文档  音乐
```

图 2.6.3.5 使用 zip 进行文件压缩

图 2.6.3.5 就是使用 zip 命令进行 test2 文件夹的压缩，我们使用的命令如下：

```
zip  -rv  test2.zip  test2
```

上述命令中，-rv 表示递归压缩并且显示压缩命令执行过程。

4、命令 unzip

unzip 命令用于对.zip 格式的压缩包进行解压，命令格式如下：

```
unzip    [参数]   [压缩文件名.zip]
```

主要参数如下：

-l ：显示压缩文件内所包含的文件。

-t ：检查压缩文件是否损坏，但不解压。

-v ：显示命令显示的执行过程。

-Z ：只显示压缩文件的注解。

-C ：压缩文件中的文件名称区分大小写。

-j ：不处理压缩文件中的原有目录路径。

-L ：将压缩文件中的全部文件名改为小写。

-n ：解压缩时不要覆盖原有文件。

-P<密码> ：解压密码。

-q ：静默执行，不显示任何信息。

-x<文件列表> ：指定不要处理.zip 中的哪些文件。

-d<目录> ：把压缩文件解到指定目录下。

对上面压缩的 test2.zip 文件使用 unzip 命令进行解压缩，操作如图 2.6.3.6 所示：

```
zuozhongkai@ubuntu:~$ rm -rf test2 //删除以前的test2文件夹，防止干扰解压过程
zuozhongkai@ubuntu:~$ ls           //显示当前目录下所有文件
examples.desktop  test2.zip  test.txt  公共的  视频  文档  音乐
test1             test3      tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ unzip test2.zip  //解压缩test2.zip文件
Archive:  test2.zip
   creating: test2/
 extracting: test2/d.c
 extracting: test2/b.c
 extracting: test2/a.c
zuozhongkai@ubuntu:~$ ls                //显示当前目录下所有文件
examples.desktop  test2      test3     tmp     模板  图片  下载  桌面
test1             test2.zip  test.txt  公共的  视频  文档  音乐
```

图 2.6.3.6 命令 unzip 使用演示

5、命令 tar

我们前面讲的 zip 和 unzip 这两个是命令只适用于.zip 格式的压缩和解压，其它压缩格式就用不了了，比如 Linux 下最常用的.bz2 和.gz 这两种压缩格式。其它格式的压缩和解压使用命令 tar，tar 将压缩和解压缩集合在一起，使用不同的参数即可，命令格式如下：

```
tar [参数] [压缩文件名] [被压缩文件名]
```

常用参数如下：

-c ：创建新的压缩文件。

-C<目的目录> ：切换到指定的目录。

-f<备份文件> ：指定压缩文件。

-j ：用 tar 生成压缩文件，然后用 bzip2 进行压缩。

-k ：解开备份文件时，不覆盖已有的文件。

-m ：还原文件时，不变更文件的更改时间。

-r ：新增文件到已存在的备份文件的结尾部分。

-t ：列出备份文件内容。

-v ：显示指令执行过程。

-w ：遭遇问题时先询问用户。

-x ：从备份文件中释放文件，也就是解压缩文件。

-z ：用 tar 生成压缩文件，用 gzip 压缩。

-Z ：用 tar 生成压缩文件，用 compress 压缩。

使用 tar 命令来进行.zip 和.gz 格式的文件压缩，操作如图 2.6.3.7 所示：

```
zuozhongkai@ubuntu:~$ ls   //显示当前目录下的所有文件
examples.desktop  test2  test.txt  公共的  视频  文档  音乐
test1             test3  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ tar -vcjf test1.tar.bz2 test1 //压缩为.bz2格式
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls                          //查看压缩后的文件是否存在
examples.desktop  test1.tar.bz2  test3     tmp     模板  图片  下载  桌面
test1             test2          test.txt  公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ tar -vczf test1.tar.gz test1 //压缩为.gz格式
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls                          //查看压缩后的文件是否存在
examples.desktop  test1.tar.bz2  test2  test.txt  公共的  视频  文档  音乐
test1             test1.tar.gz   test3  tmp       模板    图片  下载  桌面
```

图 2.6.3.7 tar 命令进行压缩

在图 2.6.3.7 中，我们使用如下两个命令将 test1 文件夹压缩为.bz2 和.gz 这两个格式：

```
tar -vcjf test1.tar.bz2 test1
tar -vczf test1.tar.gz test1
```

在上面两行命令中，-vcjf 表示创建 bz2 格式的压缩文件，-vczf 表示创建.gz 格式的压缩文件。学习了如何使用 tar 命令来完成压缩，我们再来学习使用 tar 命令完成文件的解压，操作如图 2.6.3.8 所示：

```
zuozhongkai@ubuntu:~$ ls       //显示当前目录下所有文件
examples.desktop  test2.tar.gz  test.txt  公共的  视频  文档  音乐
test1.tar.bz2     test3         tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ tar -vxjf test1.tar.bz2  //解压缩.bz2格式文件
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls                    //检查test1.tar.bz2是否解压缩成功
examples.desktop  test1.tar.bz2  test3     tmp     模板  图片  下载  桌面
test1             test2.tar.gz   test.txt  公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ tar -vxzf test2.tar.gz   //解压缩.gz格式文件
test2/
test2/d.c
test2/b.c
test2/a.c
zuozhongkai@ubuntu:~$ ls                    //检查test2.tar.gz是否解压缩成功
examples.desktop  test2         test.txt  模板  文档  桌面
test1             test2.tar.gz  tmp       视频  下载
test1.tar.bz2     test3         公共的    图片  音乐
```

图 2.6.3.8 tar 解压缩命令

图 2.6.3.8 中我们使用如下所示两行命令完成.bz2 和.gz 格式文件的解压缩：

```
tar  -vxjf    test1.tar.bz2
tar  -vxzf    test2.tar.gz
```

上述两行命令中，-vxjf 用来完成.bz2 格式压缩文件的解压，-vxzf 用来完成.gz 格式压缩文件的解压。关于 Ubuntu 下的命令行压缩和解压缩就讲解到这里，重点是 tar 命令，要熟练掌握使用 tar 命令来完成.bz2 和.gz 格式的文件压缩和解压缩。

2.6.4 文件查询和搜索

文件的查询和搜索也是最常用的操作，在嵌入式 Linux 开发中常常需要在 Linux 源码文件中查询某个文件是否存在，或者搜索哪些文件都调用了某个函数等等。本节我们就讲解两个最常用的文件查询和搜索命令：find 和 grep。

1、命令 find

find 命令用于在目录结构中查找文件，其命令格式如下：

```
find    [路径]  [参数]  [关键字]
```

路径是要查找的目录路径，如果不写的话表示在当前目录下查找，关键字是文件名的一部分，主要参数如下：

-name<filename> ：按照文件名称查找，查找与 filename 匹配的文件，可使用通配符。
-depth ：从指定目录下的最深层的子目录开始查找。
-gid<群组识别码>：查找符合指定的群组识别码的文件或目录。
-group<群组名称>：查找符合指定的群组名称的文件或目录。
-size<文件大小> ：查找符合指定文件大小的文件。
-type<文件类型> ：查找符合指定文件类型的文件。
-user<拥有者名称> ：查找符合指定的拥有者名称的文件或目录。

find 命令的参数有很多，常用的就这些，关于其它的参数大家可以自行上网查找，我们来看一下如何使用 find 命令进行文件搜索，我们搜索目录/etc 中以“vim”开头的文件为例，操作如图 2.6.4.1 所示：

```
zuozhongkai@ubuntu: ~
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
zuozhongkai@ubuntu:~$ find /etc/ -name vim*
/etc/alternatives/vimdiff
/etc/alternatives/vim
find: ‘/etc/polkit-1/localauthority’: 权限不够
find: ‘/etc/ssl/private’: 权限不够
find: ‘/etc/vmware-tools/GuestProxyData/trusted’: 权限不够
find: ‘/etc/cups/ssl’: 权限不够
/etc/vim
/etc/vim/vimrc
/etc/vim/vimrc.tiny
```

图 2.6.4.1 find 命令操作

从图 2.6.4.1 可以看出，在目录/etc 下，包含以“vim*”开头的文件有/etc/vim、/etc/vim/vimrc 等等，就不一一列出了。

2、命令 grep

find 命令用于在目录中搜索文件，我们有时候需要在文件中搜索一串关键字，grep 就是完成这个功能的，grep 命令用于查找包含指定关键字的文件，如果发现某个文件的内容包含所指定的关键字，grep 命令就会把包含指定关键字的这一行标记出来，grep 命令格式如下：

```
grep [参数]  关键字  文件列表
```

grep 命令一次只能查一个关键字，主要参数如下：

-b ：在显示符合关键字的那一列前，标记处该列第 1 个字符的位编号。

-c ：计算符合关键字的列数。

-d<进行动作> ：当指定要查找的是目录而非文件时，必须使用此参数！否则 grep 指令将回报信息并停止搜索。

-i ：忽略字符大小写。

-v ：反转查找，只显示不匹配的行。

-r ：在指定目录中递归查找。

比如我们在目录/usr 下递归查找包含字符“Ubuntu”的文件，操作如图 2.6.4.2 所示：

```
zuozhongkai@ubuntu:~$ grep -ir "Ubuntu" /usr
匹配到二进制文件 /usr/games/sol
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/recordmcount
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/bin2c
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/conmakehash
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/sign-file
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/insert-sys-cert
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/basic/fixdep
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/sumversion.o
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/mk_elfconfig
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/file2alias.o
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/modpost
/usr/src/linux-headers-5.4.0-42-generic/scripts/mod/.empty.o.cmd:cmd_scripts/mod/empty.o := g
cc -Wp,-MD,scripts/mod/.empty.o.d  -nostdinc -isystem /usr/lib/gcc/x86_64-linux-gnu/7/include
  -I/build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/arch/x86/include -I./arch/x86/include/gen
erated -I/build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include -I./include -I/build/linux-h
we-5.4-huXhHV/linux-hwe-5.4-5.4.0/arch/x86/include/uapi -I./arch/x86/include/generated/uapi -
I/build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include/uapi -I./include/generated/uapi -inc
lude /build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include/linux/kconfig.h -Iubuntu/include
  -include /build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include/linux/compiler_types.h -D_
```

图 2.6.4.2 命令 grep 演示

2.6.5 文件类型

这里的文件类型不是说这个文件是音乐文件还是文本文件，在用户根目录下使用命令“ls -l”来查看用户根目录下所有文件的详细信息，如图 2.6.5.1 所示：

```
zuozhongkai@ubuntu:~$  ls -l
总用量 72
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 12月  18 02:08 examples.desktop
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  24 21:56 test1
-rw-rw-r-- 1 zuozhongkai zuozhongkai  194 12月  24 21:58 test1.tar.bz2
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  23 01:00 test2
-rw-rw-r-- 1 zuozhongkai zuozhongkai  171 12月  24 22:09 test2.tar.gz
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  23 00:45 test3
-rw-rw-r-- 1 zuozhongkai zuozhongkai   68 12月  21 01:40 test.txt
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  19 21:41 tmp
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  22 02:08 桌面
```

图 2.6.5.1 文件详细信息

在图 2.6.5.1 中，每个文件的详细信息占一行，每行最前面都是一个符号就标记了当前文件类型，比如 test1 的第一个字符是“d”，test1.tar.bz2 文件第一个字符是“-”。这些字符表示的文件类型如下：

- ：普通文件，一些应用程序创建的，比如文档、图片、音乐等等。

d ：目录文件。

c ：字符设备文件，Linux 驱动里面的字符设备驱动，比如串口设备，音频设备等。

b ：块设备文件，存储设备驱动，比如硬盘，U 盘等。

l ：符号连接文件，相当于 Windwos 下的快捷方式。

s ：套接字文件。

p ：管道文件，主要指 FIFO 文件。

我们后面学习 Linux 驱动开发的时候基本是在和字符设备文件和块设备文件打交道。

2.7 Linux 用户权限管理

2.7.1 Ubuntu 用户系统

Ubuntu 是一个多用户系统，我们可以给不同的使用者创建不同的用户账号，每个用户使用各自的账号登陆，使用用户账号的目的一是方便系统管理员管理，控制不同用户对系统的访问权限，另一方面是为用户提供安全性保护。

我们前面在安装 Ubuntu 系统的时候被要求创建一个账户，当我们创建好账号以后，系统会在目录/home 下以该用户名创建一个文件夹，所有与该用户有关的文件都会被存储在这个文件文件夹中。同样的，创建其它用户账号的时候也会在目录/home 下生成一个文件夹来存储该用户的文件，图 2.7.1.1 就是我的电脑上“zuozhongkai”这个账户的文件夹。

```
zuozhongkai@ubuntu:~$ ls
examples.desktop  公共的  模板  视频  图片  文档  下载  音乐  桌面
```

图 2.7.1.1 用户账号根目录

装系统的时候创建的用户其权限比后面创建的用户大一点，但是没有 root 用户权限大，Ubuntu 下用户类型分为以下 3 类：

- 初次创建的用户，此用户可以完成比普通用户更多的功能。
- root 用户，系统管理员，系统中的玉皇大帝，拥有至高无上的权利。
- 普通用户，安装完操作系统以后被创建的用户。

以上三种用户，每个用户都有一个 ID 号，称为 UID，操作系统通过 UID 来识别是哪个用户，用户相关信息可以在文件/etc/passwd 中查看到，如图 2.7.1.2 所示：

```
zuozhongkai@ubuntu:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
```

图 2.7.1.2 passwd 文件内容

从配置文件 passwd 中可以看到，每个用户名后面都有两个数字，比如用户“zuozhongkai”后面“1000:1000”，第一个数字是用户的 ID，另一个是用户的 GID，也就是用户组 ID。Ubuntu 里面每个用户都属于一个用户组里面，用户组就是一组有相同属性的用户集合。

2.7.2 权限管理

在使用 Windows 的时候我们很少接触到用户权限，最多就是打开某个软件出问题的时候会选择以“管理员身份”打开。Ubuntu 下我们会常跟用户权限打交道，权限就是用户对于系统资源的使用限制情况，root 用户拥有最大的权限，可以为所欲为，装系统的时候创建的用户拥有 root 用户的部分权限，其它普通用户的权限最低。对于我们做嵌入式开发的人一般不关注用户的权限问题，因为嵌入式基本是单用户，做嵌入式开发重点关注的是文件的权限问题。

对于一个文件通常有三种权限：读(r)、写(w)和执行(x)，使用命令“ls -l”可以查看某个目录下所有文件的权限信息，如图 2.7.2.1 所示：

```
zuozhongkai@ubuntu:~$ ls -l
总用量 44
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 10月  20 18:23 examples.desktop
-rw-r--r-- 1 zuozhongkai zuozhongkai    0 10月  28 12:05 test.c
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  21 18:37 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  28 10:47 桌面
```

图 2.7.2.1 文件权限信息

在图 2.7.2.1 中我们以文件 test.c 为例讲解，文件 test.c 文件信息如下：

```
-rw-r--r-- 1 zuozhongkai zuozhongkai    0 10 月  28 12:05 test.c
```

其中“-rw-rw-r--”表示文件权限与用户和用户组之间的关系，第一位表示文件类型，上一小节已经说了。剩下的 9 位以 3 位为一组，分别表示文件拥有者的权限，文件拥有者所在用户组的权限以及其它用户权限。后面的“zuozhongkai zuozhongkai”分别代表文件拥有者(用户)和该用户所在的用户组，因此文件 test.c 的权限情况如下：

①、文件 test.c 的拥有者是用户 zuozhongkai，其对文件 test.c 的权限是“rw-”，也就是对该文件拥有读和写两种权限。

②、用户 zuozhongkai 所在的用户组也叫做 zuozhongkai，其组内用户对于文件 test.c 的权限是“rw-”，也是拥有读和写这两种权限。

③、其它用户对于文件 test.c 的权限是“r--”，也就是只读权限。

对于文件，可读权限表示可以打开查看文件内容，可写权限表示可以对文件进行修改，可执行权限就是可以运行此文件(如果是软件的话)。对于文件夹，拥有可读权限才可以使用命令 ls 查看文件夹中的内容，拥有可执行权限才能进入到文件夹内部。

如果某个用户对某个文件不具有相应的权限的话就不能进行相应的操作，比如根目录“/”下的文件只有 root 用户才有权限进行修改，如果以普通用户去修改的话就会提示没有权限。比如我们要在根目录“/” 创建一个文件 mytest，使用命令“touch mytest”，结果如图 2.7.2.2 所示：

```
zuozhongkai@ubuntu:/$ touch mytest
touch: 无法创建'mytest': 权限不够
```

图 2.7.2.2 创建文件

在图 2.7.2.2 中，我以用户“zuozhongkai”在根目录“/”创建文件 mytest，结果提示我无法创建“mytest”，因为权限不够，因为只有 root 用户才能在根目录“/”下创建文件。我们可以使用命令“sudo”命令暂时切换到 root 用户，这样就可以在根目录“/”下创建文件 mytest 了，如图 2.7.2.3 所示：

```
zuozhongkai@ubuntu:/$ sudo touch mytest
[sudo] zuozhongkai 的密码：
zuozhongkai@ubuntu:/$ ls
bin   dev   initrd.img      lib64       mnt     proc  sbin  swapfile  usr      vmlinuz.old
boot  etc   initrd.img.old  lost+found  mytest  root  snap  sys       var
cdrom home  lib             media       opt     run   srv   tmp       vmlinuz
```

图 2.7.2.3 使用 sudo 命令创建文件

在图 2.7.2.3 中我们使用命令“sudo”以后就可以在根目录“/”创建文件 mytest，在进行其它的操作的时候，遇到提示权限不够的时候都可以使用 sudo 命令暂时以 root 用户身份去执行。

上面我们讲了，文件的权限有三种：读(r)、写(w)和执行(x)，除了用 r、w 和 x 表示以外，我们也可以使用二进制数表示，三种权限就可以使用 3 位二进制数来表示，一种权限对应一个二进制位，如果该位为 1 就表示具备此权限，如果该位为 0 就表示没不具备此权限，如表 2.7.2.1 所示：

字母	二进制	八进制
r	100	4
w	010	2
x	001	1

表 2.7.2.1 文件权限数字表示方法

如果做过单片机开发的话，就会发现和单片机里面的寄存器位一样，将三种权限 r、w 和 x 进行不同的组合，即可得到不同的二进制数和八进制数，3 位权限可以组出 8 种不同的权限组合，如表 2.7.2.2 所示：

权限	二进制数字	八进制数字

---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwx	111	7

表 2.7.2.2 文件所有权限组合

表 2.7.2.2 中权限所对应的八进制数字就是每个权限对应的位相加，比如权限 rwx 就是 4+2+1=7。前面的文件 test.c 其权限为“rw-rw-r--”，因此其十进制表示就是：664。

另外我们也开始使用 a、u、g 和 o 表示文件的归属关系，用=、+和-表示文件权限的变化，如表 2.7.2.3 所示：

字母	意义
r	可读权限
w	可写权限
x	可执行权限
a	所有用户
u	归属用户
g	归属组
o	其它用户
=	具备权限
+	添加某权限
-	去除某权限

表 2.7.2.3 权限修改字母表示方式

对于文件 test.c，我们想要修改其归属用户(zuozhongkai)对其拥有可执行权限，那么就可以使用：u+x。如果希望设置归属用户及其所在的用户组都对其拥有可执行权限就可以使用：gu+x。

2.7.3 权限管理命令

我们也可以使用 Shell 来操作文件的权限管理，主要用到“chmod”和“chown”这两个命令，我们一个一个来看。

1、权限修改命令 chmod

命令“chmod”用于修改文件或者文件夹的权限，权限可以使用前面讲的数字表示也可以使用字母表示，命令格式如下：

```
chmod [参数] [文件名/目录名]
```

主要参数如下：

-c ：效果类似“-v”参数，但仅回显更改的部分。

-f ：不显示错误信息。

-R ：递归处理，指定目录下的所有文件及其子文件目录一起处理。

-v ：显示指令的执行过程。

我们先来学习以下如何使用命令“chmod”修改一个文件的权限，在用户根目录下创建一个文件 test，然后查看其默认权限，操作如图 2.7.3.1 所示：

```
zuozhongkai@ubuntu:~$ touch test
                ubuntu:~$ ls -l
LibreOffice Calc
总用量 48
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 12月  18 02:08 examples.desktop
-rw-rw-r-- 1 zuozhongkai zuozhongkai    0 12月  25 22:24 test
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  19 21:41 tmp
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  22 02:08 桌面
```

图 2.7.3.1 创建文件 test

在图 2.7.3.1 中我们创建了一个文件：test，这个文件的默认权限为“rw-rw-r--”，我们将其权限改为“rwxrw-rw”，对应数字就是 766，操作如下：

```
zuozhongkai@ubuntu:~$ ls -l test    //查看文件test权限
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  25 22:24 test
zuozhongkai@ubuntu:~$ chmod 766 test //修改文件test的权限为766
zuozhongkai@ubuntu:~$ ls -l test     //查看修改以后的文件test权限
-rwxrw-rw- 1 zuozhongkai zuozhongkai 0 12月  25 22:24 test
```

图 2.7.3.2 修改权限

在图 2.7.3.2 中，我们修改文件 test 的权限为 766，修改完成以后的 test 文件权限为“rwxrw-rw-”，和我们设置的一样，说明权限修改成功。

上面我们是通过数字来修改权限的，我们接下来使用字母来修改权限，操作如图 2.7.3.3 所示：

```
zuozhongkai@ubuntu:~$ touch a.c //创建文件a.c
zuozhongkai@ubuntu:~$ ls -l a.c //显示文件a.c的权限
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  25 22:33 a.c
zuozhongkai@ubuntu:~$ chmod u+x a.c  //给文件a.c的归属用户添加可执行权限
zuozhongkai@ubuntu:~$ ls -l a.c      //显示修改权限后的文件a.c
-rwxrw-r-- 1 zuozhongkai zuozhongkai 0 12月  25 22:33 a.c
```

图 2.7.3.3 使用字母修改文件权限

上面两个例子都是修改文件的权限，接下来我们修改文件夹的权限，新建一个 test 文件夹，在文件夹 test 里面创建 a.c、b.c 和 c.c 三个文件，如图 2.7.3.4 所示：

```
zuozhongkai@ubuntu:~$ ls -l test/
总用量 0
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:20 a.c
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:20 b.c
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:20 c.c
```

图 2.7.3.4 test 文件夹

在图 2.7.3.4 中 test 文件夹下的文件 a.c、b.c 和 c.c 的权限均为“rw-rw-r--”，我们将 test 文件夹下的所有文件权限都改为“rwxrwxrwx”，也就是数字 777，操作如图 2.7.3.5 所示：

```
zuozhongkai@ubuntu:~$ chmod -R 777 test/ //递归修改文件权限
zuozhongkai@ubuntu:~$ ls -l test/
总用量 0                                  //查看修改权限以后的文件
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月  26 00:20 a.c
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月  26 00:20 b.c
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月  26 00:20 c.c
```

图 2.7.3.5 递归修改文件夹权限

2、文件归属者修改命令 chown

命令 chown 用来修改某个文件或者目录的归属者用户或者用户组，命令格式如下：

chown [参数] [用户名.<组名>] [文件名/目录]

其中[用户名.<组名>]表示要将文件或者目录改为哪一个用户或者用户组，用户名和组名用“.”隔开，其中用户名和组名中的任何一个都可以省略，命令主要参数如下：

-c ：效果同-v 类似，但仅回报更改的部分。

-f ：不显示错误信息。

-h ：只对符号连接的文件做修改，不改动其它任何相关的文件。

-R ：递归处理，将指定的目录下的所有文件和子目录一起处理。

-v ：显示处理过程。

在用户根目录下创建一个 test 文件，查看其文件夹所属用户和用户组，如图 2.7.3.6 所示：

```
zuozhongkai@ubuntu:~$ touch test
zuozhongkai@ubuntu:~$ ls -l test
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:36 test
```

图 2.7.3.6 test 文件信息查询

从图 2.7.3.6 中可以看出，文件 test 的归属用户为 zuozhongkai，所属的用户组为 zuozhongkai，将文件 test 归属用户改为 root 用户，所属的用户组也改为 root，操作如图 2.7.3.7 所示：

```
zuozhongkai@ubuntu:~$ ls -l test    //显示文件test的归属用户和归属组
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:45 test
zuozhongkai@ubuntu:~$ sudo chown root.root test  //修改文件test的归属用户和归属组
zuozhongkai@ubuntu:~$ ls -l test   //查看修改以后的文件test归属用户和归属组
-rw-rw-r-- 1 root root 0 12月  26 00:45 test
```

图 2.7.3.7 修改文件归属用户和归属组

命令 shown 同样也可以递归处理来修改文件夹的归属用户和用户组，用法和命令 chown 一样，这里就不演示了。

2.8 Linux 磁盘管理

2.8.1 Linux 磁盘管理基本概念

Linux 的磁盘管理体系和 Windows 有很大的区别，在 Windows 下经常会遇到“分区”这个概念，在 Linux 中一般不叫“分区”而叫“挂载点”。 “挂载点”就是将一个硬盘的一部分做成文件夹的形式，这个文件夹的名字就是“挂载点”，不管在哪个发行版的 Linux 中，用户是绝对看到不到 C 盘、D 盘这样的概念的，只能看到以文件夹形式存在的“挂载点”.

文件/etc/fstab 详细的记录了 Ubuntu 中硬盘分区的情况，如图 2.8.1.1 所示：

```
zuozhongkai@ubuntu:~$ cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/sda1 during installation
UUID=6807330c-ef05-4fa4-93ca-a6419ed7b57d /               ext4    errors=remount-ro 0       1
/swapfile                                 none            swap    sw                0       0
```

图 2.8.1.1 文件 fstab

在图 2.8.1.1 中有一行“/ was on /dev/sda1 during installation”，意思是根目录“/”是在/dev/sda1 上的，其中“/”是挂载点，“/dev/sda1”就是我们装 Ubuntu 系统的硬盘。由于我们的系统是安装在虚拟机中的，因此图 2.8.1.1 没有出现实际的硬盘。可以通过如下命令查看当前系统中的磁盘：

```
ls  /dev/sd*
```

上述命令就是打印出所有以/dev/sd 开头的设备文件，如图 2.8.1.2 所示：

```
zuozhongkai@ubuntu:~$ ls /dev/sd*
/dev/sda  /dev/sda1
```

图 2.8.1.2 查看硬盘设备文件

在图 2.8.1.2 中有四个磁盘设备文件，其中 sd 表示是 SATA 硬盘或者其它外部设备，最后面的数字表示该硬盘上的第 n 个分区，比如/dev/sda1 就表示磁盘 sda 上的第一个分区。图 2.8.1.2 中都是以/dev/sda 开头的，说明当前只有一个硬盘。如果再插上 U 盘、SD 卡啥的就可能会出现/dev/sdb，/dev/sdc 等等。如果你的 U 盘有两个分区那么可能就会出现/dev/sdb1、dev/sdb2 这样的设备文件。比如我现在插入我的 U 盘，插入 U 盘会提示 U 盘是接到主机还是虚拟机，如图 2.8.1.3 所示：

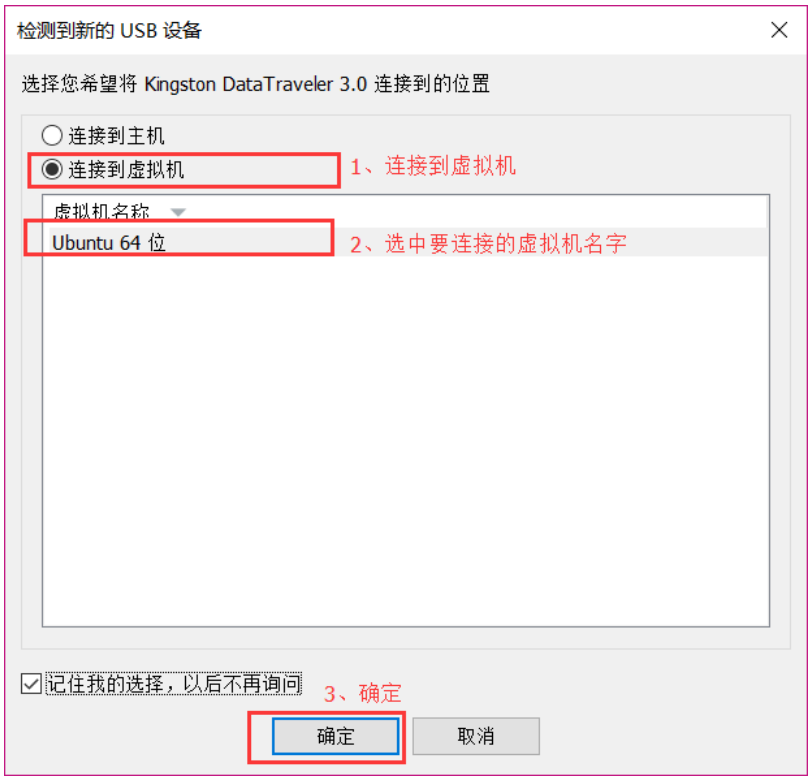


图 2.8.1.3 U 盘连接选择

设置好图 2.8.1.3 以后，点击“确定”按钮 U 盘就会自动连接到虚拟机中，也就是连接到 Ubuntu 系统中，我们再次使用命令“ls /dev/sd*”来查看当前的“/dev/sd*”设备文件，如图 2.8.1.4 所示：

```
zuozhongkai@ubuntu:~$ ls /dev/sd*
/dev/sda  /dev/sda1  /dev/sdb  /dev/sdb1
```

图 2.8.1.4 插入 U 盘后的设备文件

从图 2.8.1.4 可以看出，相比图 2.8.1.2 多了/dev/sdb 和/dev/sdb1 这两个文件，其中/dev/sdb 就是 U 盘文件，/dev/sdb1 表示 U 盘的第一个分区，因为我的 U 盘就一个分区。

2.8.2 磁盘管理命令

本节我们学习以下跟磁盘操作有关的命令，这些命令如下：

1、磁盘分区命令 fdisk

如果要对某个磁盘进行分区，可以使用命令 fdisk，命令格如下：

```
fdisk  [参数]
```

主要参数如下：

-b<分区大小> ：指定每个分区的大小。

-l ：列出指定设备的分区表。

-s<分区编号> ：将指定的分区大小输出到标准的输出上，单位为块。

-u ：搭配“-l”参数，会用分区数目取代柱面数目，来表示每个分区的起始地址。

比如我要对 U 盘进行分区，千万不要对自己装 Ubuntu 系统进行分区!!! 可以使用如下命令：

```
sudo  fdisk  /dev/sdb
```

结果如图 2.8.2.1 所示：

```
zuozhongkai@ubuntu:~$ sudo fdisk /dev/sdb
[sudo] zuozhongkai 的密码：

欢迎使用 fdisk (util-linux 2.31.1)。
更改将停留在内存中，直到您决定将更改写入磁盘。
使用写入命令前请三思。


命令(输入 m 获取帮助)：
```

图 2.8.2.1 U 盘分区界面

在图 2.8.2.1 中提示我们输入“m”可以查看帮助，因为 fdisk 还有一些字命令，通过输入“m”可以查看都有哪些子命令，如图 2.8.2.2 所示：

```
命令(输入 m 获取帮助): m

帮助:

  DOS (MBR)
   a   开关 可启动 标志
   b   编辑嵌套的 BSD 磁盘标签
   c   开关 dos 兼容性标志

  常规
   d   删除分区
   F   列出未分区的空闲区
   l   列出已知分区类型
   n   添加新分区
   p   打印分区表
   t   更改分区类型
   v   检查分区表
   i   打印某个分区的相关信息

  杂项
   m   打印此菜单
   u   更改 显示/记录 单位
   x   更多功能(仅限专业人员)

  脚本
   I   从 sfdisk 脚本文件加载磁盘布局
   O   将磁盘布局转储为 sfdisk 脚本文件

  保存并退出
   w   将分区表写入磁盘并退出
   q   退出而不保存更改

  新建空磁盘标签
   g   新建一份 GPT 分区表
   G   新建一份空 GPT (IRIX) 分区表
   o   新建一份的空 DOS 分区表
   s   新建一份空 Sun 分区表
```

图 2.8.2.2 fdisk 命令的子命令

图 2.8.2.2 中常用的命令如下：

p ：显示现有的分区

n ：建立新分区

t ：更改分区类型

d ：删除现有的分区

a ：更改分区启动标志

w ：对分区的更改写入到硬盘或者存储器中。

q ：不保存退出。

2、格式化命令 mkfs

使用命令 fdisk 创建好一个分区以后，我们需要对其格式化，也就是在这个分区上创建一个文件系统，Linux 下的格式化命令为 mkfs，命令格式如下：

```
mkfs    [参数]   [-t 文件系统类型]    [分区名称]
```

主要参数如下：

fs ：指定建立文件系统时的参数

-V ：显示版本信息和简要的使用方法。

-v ：显示版本信息和详细的使用方法。

比如我们要格式化 U 盘的分区/dev/sdb1 为 FAT 格式，那么就可以使用如下命令：

```
mkfs  -t  vfat  /dev/sdb1
```

3、挂载分区命令 mount

我们创建好分区并且格式化以后肯定是要使用硬盘或者 U 盘的，那么如何访问磁盘呢？比如我的 U 盘就一个分区，为/dev/sdb1，如果直接打开文件/dev/sdb1 会发现根本就不是我们要的结果。我们需要将/dev/sdb1 这个分区挂载到一个文件夹中，然后通过这个文件访问 U 盘，磁盘挂载命令为 mount，命令格式如下：

```
mount  [参数]  -t  [类型]  [设备名称]  [目的文件夹]
```

命令主要参数有：

-V ：显示程序版本。

-h ：显示辅助信息。

-v ：显示执行过程详细信息。

-o ro ：只读模式挂载。

-o rw ：读写模式挂载。

-s-r ：等于-o ro。

-w ：等于-o rw。

挂载点是一个文件夹，因此在挂载之前先要创建一个文件夹，一般我们把挂载点放到“/mnt”目录下，在“/mnt”下创建一个 tmp 文件夹，然后将 U 盘的/dev/sdb1 分区挂载到/mnt/tmp 文件夹里面，操作如图 2.8.2.3 所示：

```
zuozhongkai@ubuntu:~$ ls /mnt  //查看/mnt目录下的所有文件
zuozhongkai@ubuntu:~$
zuozhongkai@ubuntu:~$ sudo mkdir /mnt/tmp  //创建文件夹/mnt/tmp
[sudo] zuozhongkai 的密码：
zuozhongkai@ubuntu:~$ ls /mnt              //查看/mnt目录下的所有文件
tmp
zuozhongkai@ubuntu:~$ sudo mount -t vfat /dev/sdb1 /mnt/tmp //挂载U盘到/mnt/tmp中
zuozhongkai@ubuntu:~$ ls /mnt/tmp //查看/mnt/tmp中的文件，也就是U盘里面的文件
AD                          IMX6UL_ZERO_V1.0.zip        开发板光盘
ARM裸机与嵌入式Linux驱动开发V1.0.docx  System Volume Information
```

图 2.8.2.3 挂载 U 盘

在图 2.8.2.3 中我们将 U 盘以 fat 格式挂载到目录/mnt/tmp 中，然后我们就可以通过访问/mnt/tmp 来访问 U 盘了。

4、卸载命令 umount

当我们不在需要访问已经挂载的 U 盘，可以通过 umount 将其从卸载点卸除，命令格式如下：

```
umount  [参数]  -t  [文件系统类型]  [设备名称]
```

-a ：卸载/etc/mtab 中的所有文件系统。

-h ：显示帮助。

-n ：卸载时不要将信息存入到/etc/mtab 文件中

-r ：如果无法成功卸载，则尝试以只读的方式重新挂载。

-t<文件系统类型> ：仅卸载选项中指定的文件系统。

-v ：显示执行过程。

上面我们将 U 盘挂载到了文件夹/mnt/tmp 里面，这里我们使用命令 umount 将其卸载掉，操作如图 2.8.2.4 所示：

```
zuozhongkai@ubuntu:~$ ls /mnt/tmp //查看卸载之前的文件夹/mnt/tmp
AD                                IMX6UL_ZERO_V1.0.zip        开发板光盘
ARM裸机与嵌入式Linux驱动开发V1.0.docx  System Volume Information
zuozhongkai@ubuntu:~$ sudo umount -t vfat /dev/sdb1 //卸载U盘的/dev/sdb1分区
zuozhongkai@ubuntu:~$ ls /mnt/tmp                   //查看卸载以后的/mnt/tmp文件夹
zuozhongkai@ubuntu:~$
zuozhongkai@ubuntu:~$
```

图 2.8.2.4 卸载 U 盘

在图2.8.2.4中，我们使用命令umount卸载了U盘，卸载以后当我们再去访问文件夹/mnt/tmp的时候发现里面没有任何文件了，说明我们卸载成功了。

第三章 Linux C 编程入门

在 Windows 下我们可是使用各种各样的 IDE 进行编程，比如强大的 Visual Studio。但是在 Ubuntu 下如何进行编程呢？Ubuntu 下也有一些可以进行编程的工具，但是大多都只是编辑器，也就是只能进行代码编辑，如果要编译的话就需要用到 GCC 编译器，使用 GCC 编译器肯定就要接触到 Makefile。本章就讲解如何在 Ubuntu 下进行 C 语言的编辑和编译、GCC 和 Makefile 的使用和编写。通过本章的学习可以掌握 Linux 进行 C 编程的基本方法，为以后的 Linux 驱动学习做准备。

3.1 Hello World！

我们所说的编写代码包括两部分：代码编写和编译，在 Windows 下可以使用 Visual Studio 来完成这两部分，可以在 Visual Studio 下编写代码然后直接点击编译就可以了。但是在 Linux 下这两部分是分开的，比如我们用 VIM 进行代码编写，编写完成以后在使用 GCC 编译器进行编译，其中代码编写工具很多，比如 VIM 编辑器、Emacs 编辑器、VScode 编辑器等等，本章使用 Ubuntu 自带的 VIM 编辑器。先来编写一个最简单的“Hello World”程序，把 Linux 下的 C 编程完整的走一遍。

3.1.1 编写代码

先在用户根目录下创建一个工作文件夹：C_Program，所有的 C 语言练习都保存到这个工作文件夹下，创建过程如图 3.1.1.1 所示：

```
zuozhongkai@ubuntu:~$ mkdir C_Program
zuozhongkai@ubuntu:~$ ls
C_Program  examples.desktop  公共的  模板  视频  图片  文档  下载  音乐  桌面
```

图 3.1.1.1 创建工作目录

进入图 3.1.1.1 创建的 C_Program 工作文件夹，为了方便管理，我们后面每个例程都创建一个文件夹来保存所有与本例程有关的文件，创建一个名为“3.1”的文件夹来保存我们的“Hello World”程序相关的文件，操作如图 3.1.1.2 所示：

```
zuozhongkai@ubuntu:~/C_Program$ mkdir 3.1
zuozhongkai@ubuntu:~/C_Program$ ls
3.1
zuozhongkai@ubuntu:~/C_Program$
```

图 3.1.1.2 创建工程文件夹

前面说了我们使用 VI 编辑器，在使用 VI 编辑器之前我们先做如下设置：

1、设置 TAB 键为 4 字节

VI 编辑器默认 TAB 键为 8 空格，我们改成 4 空格，用 vi 打开文件/etc/vim/vimrc，在此文件最后面输入如下代码：

```
set ts=4
```

添加完成如图 3.1.1.3 所示：

```
" Source a global configuration file if available
if filereadable("/etc/vim/vimrc.local")
  source /etc/vim/vimrc.local
endif

set ts=4
```

图 3.1.1.3 设置 TAB 为四个空格

修改完成以后保存并关闭文件。

2、VIM 编辑器显示行号

VIM 编辑器默认是不显示行号的，不显示行号不利于代码查看，我们设置 VIM 编辑器显示行号，同样是通过在文件/etc/vim/vimrc 中添加代码来实现，在文件最后面加入下面一行代码即可：

```
set nu
```

添加完成以后的/etc/vim/vimrc 文件如图 3.1.1.4 所示：

```
" Source a global configuration file if available
if filereadable("/etc/vim/vimrc.local")
  source /etc/vim/vimrc.local
endif

set ts=4
set nu
```

图 3.1.1.4 设置 VIM 编辑器显示行号

VIM 编辑器可以自行定制，网上有很多的博客讲解如何设置 VIM，感兴趣的可以上网看一下。设置好 VIM 编辑器以后就可以正式开始编写代码了，进入前面创建的“3.1”这个工程文件夹里面，使用 vi 指令创建一个名为“main.c”的文件，然后在里面输入如下代码：

示例代码 3.1.1.1 main.c 文件代码

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[])
4 {
5   printf("Hello World!\n");
6 }
```

编写完成以后保存退出 vi 编辑器，可以使用“cat”命令查看代码是否编写成功，如图 3.1.1.5 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ cat main.c
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello World!\n");
}
```

图 3.1.1.5 查阅程序源码

从图 3.1.1.5 可以看出 main.c 文件是编辑成功的，代码编辑成功以后我们需要对其进行编译。

3.1.2 编译代码

Ubuntu 下的 C 语言编译器是 GCC，Ubuntu18 默认没有安装 GCC 工具，因此需要我们手动安装 gcc、g++、make 等工具，这里我们直接安装 build-essential 软件包即可，build-essential 提供了编译程序所需的所有软件包，输入如下命令：

```
sudo apt-get install build-essential
```

等待安装完成，然后输入如下命令查看 GCC 编译器的版本号：

```
gcc -v
```

在终端中输入上述命令以后终端输出如图 3.1.2.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/7/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 7.5.0-3ubuntu1~18.04' --with-bugurl=fil
e:///usr/share/doc/gcc-7/README.Bugs --enable-languages=c,ada,c++,go,brig,d,fortran,objc,obj-c++ --pre
fix=/usr --with-gcc-major-version-only --program-suffix=-7 --program-prefix=x86_64-linux-gnu- --enable
-shared --enable-linker-build-id --libexecdir=/usr/lib --without-included-gettext --enable-threads=pos
ix --libdir=/usr/lib --enable-nls --enable-bootstrap --enable-clocale=gnu --enable-libstdcxx-debug --e
nable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-gnu-unique-object --disable-vtable-
verify --enable-libmpx --enable-plugin --enable-default-pie --with-system-zlib --with-target-system-zl
ib --enable-objc-gc=auto --enable-multiarch --disable-werror --with-arch-32=i686 --with-abi=m64 --with
-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-offload-targets=nvptx-none
--without-cuda-driver --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --tar
get=x86_64-linux-gnu
Thread model: posix
gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)
```

图 3.1.2.1 gcc 版本查询

如果输入命令“gcc -v”命令以后，你的终端输出类似图 3.1.2.1 中的信息，那么说明你的电脑已经有 GCC 编译器了。最后下面的“gcc version 7.5.0”说明本机的 GCC 编译器版本为 7.5.0 的。注意观察在图 3.1.2.1 中有“Target: x86_64-linux-gnu”一行，这说明这个 GCC 编译器是针对 X86 架构的，因此只能编译在 X86 架构 CPU 上运行的程序。如果想要编译在 ARM 上运行的程序就需要针对 ARM 的 GCC 编译器，也就是交叉编译器！我们是 ARM 开发，因此肯定要安装针对 ARM 架构的 GCC 交叉编译器，当然了，这是后面的事，现在我们不用管这些，只要知道不同的目标架构，其 GCC 编译器是不同的。

如何使用 GCC 编译器来编译 main.c 文件呢？GCC 编译器是命令模式的，因此需要输入命令来使用 gcc 编译器来编译文件，输入如下命令：

```
gcc main.c
```

上述命令的功能就是使用 gcc 编译器来编译 main.c 这个 c 文件，过程如图 3.1.2.2 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ gcc main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$
```

图 3.1.2.2 编译 main.c 文件

在图 3.1.2.2 中可以看到，当编译完成以后会生成一个 a.out 文件，这个 a.out 就是编译生成发的可执行文件，执行此文件看看是否和我们代码的功能一样，执行的方法很简单使用命令：“./+可执行文件”，比如本例程就是命令：./a.out，操作如图 3.1.2.3 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ./a.out
Hello World!
zuozhongkai@ubuntu:~/C_Program/3.1$
```

图 3.1.2.3 执行编译得到的文件

在图 3.1.2.3 中执行 a.out 文件以后终端输出了“Hello World!”，这正是 main.c 要实现的功能，说明我们的程序没有错误。a.out 这个文件的命名是 GCC 编译器自动命名的，那我们能不能决定编译完生成的可执行文件名字呢？肯定可以的，在使用 gcc 命令的时候加上-o 来指定生成的可执行文件名字，比如编译 main.c 以后生成名为“main”的可执行文件，操作如图 3.1.2.4 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ gcc main.c -o main //指定编译生成的可执行文件名字为main
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ./main
Hello World!
zuozhongkai@ubuntu:~/C_Program/3.1$
```

图 3.1.2.4 指定可执行文件名字

在图 3.1.2.4 中，我们使用“gcc main.c –o main”来编译 main.c 文件，使用参数“-o”来指定编译生成的可执行文件名字，至此我们就完成 Linux 下 C 编程和编译的一整套过程。

3.2 GCC 编译器

3.2.1 gcc 命令

在上一小节我们已经使用过 GCC 编译器来编译 C 文件了，我们使用到是 gcc 命令，gcc 命令格式如下：

```
gcc [选项]  [文件名字]
```

主要选项如下：

-c：只编译不链接为可执行文件，编译器将输入的.c 文件编译为.o 的目标文件。

-o：<输出文件名>用来指定编译结束以后的输出文件名，如果使用这个选项的话 GCC 默认编译出来的可执行文件名字为 a.out。

-g：添加调试信息，如果要使用调试工具(如 GDB)的话就必须加入此选项，此选项指示编译的时候生成调试所需的符号信息。

-O：对程序进行优化编译，如果使用此选项的话整个源代码在编译、链接的的时候都会进行优化，这样产生的可执行文件执行效率就高。

-O2：比-O 更幅度更大的优化，生成的可执行效率更高，但是整个编译过程会很慢。

3.2.2 编译错误警告

在 Windows 下不管我们用啥编译器，如果程序有语法错误的话编译的时候都会指示出来，比如开发 STM32 的时候所使用的 MDK 和 IAR，我们可以根据错误信息方便的修改 bug。那 GCC 编译器有没有错误提示呢？肯定是有的，我们可以测试以下，新名为“3.2”的文件夹，使用 vi 在文件夹“3.2”中创建一个 main.c 文件，在文件里面输入如下代码：

示例代码 3.2.2.1 main.c 文件代码

```
1  #include <stdio.h>
2
3  int main(int argc, char *argv[])
4  {
5      int a, b;
6
7      a = 3;
8      b = 4
9      printf("a+b=\n", a + b);
10 }
```

在上述代码中有两处错误：

第 8 行、第一处是“b=4”少写了个一个“;”号。

第 9 行、第二处应该是 printf(“a+b=%d\n”， a + b);

我们编译以下上述代码，看看 GCC 编译器是否能够检查出错误，编译结果如图 3.2.2.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
main.c: In function ‘main’:
main.c:9:2: error: expected ‘;’ before ‘printf’
  printf("a+b=\n", a + b);
  ^~~~~~
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 3.2.2.1 错误提示

从图 3.2.2.1 中可以看出有一个 error，提示在 main.c 文件的第 9 行有错误，错误类型是在 printf 之前没有“;”号，这就是第一处错误，我们在“b = 4”后面加上分号，然后接着编译，结果又提示有一个错误，如图 3.2.2.2 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
main.c: In function ‘main’:
main.c:9:9: warning: too many arguments for format [-Wformat-extra-args]
  printf("a+b=\n", a + b);
         ^~~~~~~~
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 3.2.2.2 错误提示

在图 3.2.2.2 中，提示我们说文件 main.c 的第 9 行：printf(“a+b=\n”, a + b)有 warning，错误是因为太多参数了，我们将其改为：

printf(“a+b=%d\n” , a + b);

修改完成以后接着重新编译一下，结果如图 3.2.2.3 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
zuozhongkai@ubuntu:~/C_Program/3.2$ ls
main  main.c
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 3.2.2.3 编译成功

在图 3.2.2.3 中我们编译成功，生成了可执行文件 main，执行一下 main，看看结果和我们设计的是否一样，如图 3.2.2.4 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ ./main
a+b=7
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 3.2.2.4 执行结果

可以看出，GCC 编译器和其它编译器一样，不仅能够检测出错误类型，而且标记除了错误发生在哪个文件？哪一行？方便我们去修改代码。

3.2.3 编译流程

GCC 编译器的编译流程是：预处理、编译、汇编和链接。预处理就是展开所有的头文件、替换程序中的宏、解析条件编译并添加到文件中。编译是将经过预编译处理的代码编译成汇编代码，也就是我们常说的程序编译。汇编就是将汇编语言文件编译成二进制目标文件。链接就是将汇编出来的多个二进制目标文件链接在一起，形成最终的可执行文件，链接的时候还会涉及到静态库和动态库等问题。

上一小节演示的例程都只有一个文件，而且文件非常简单，因此可以直接使用 gcc 命令生成可执行文件，并没有先将 c 文件编译成.o 文件，然后再链接在一起。

3.3 Makefile 基础

3.3.1 何为 Makefile

上一小节我们讲了如何使用 GCC 编译器在 Linux 进行 C 语言编译，通过在终端执行 gcc 命令来完成 C 文件的编译，如果我们的工程只有一两个 C 文件还好，需要输入的命令不多，当文件有几十、上百甚至上万个的时候用终端输入 GCC 命令的方法显然是不现实的。如果我们能够编写一个文件，这个文件描述了编译哪些源码文件、如何编译那就好了，每次需要编译工程的时只需要使用这个文件就行了。这种问题怎么可能难倒聪明的程序员，为此提出了一个解决大工程编译的工具：make，描述哪些文件需要编译、哪些需要重新编译的文件就叫做 Makefile，Makefile 就跟脚本文件一样，Makefile 里面还可以执行系统命令。使用的时候只需要一个 make 命令即可完成整个工程的自动编译，极大的提高了软件开发的效率。如果大家以前一直使用 IDE 来编写 C 语言的话肯定没有听说过 Makefile 这个东西，其实这些 IDE 是有的，只不过这些 IDE 对其进行了封装，提供给大家的是已经经过封装后的图形界面了，我们在 IDE 中添加要编译的 C 文件，然后点击按钮就完成了编译。在 Linux 下用的最多的是 GCC 编译器，这是个没有 UI 的编译器，因此 Makefile 就需要我们自己来编写了。作为一个专业的程序员，是一定要懂得 Makefile 的，一是因为在 Linux 下你不得不懂 Makefile，再就是通过 Makefile 你就能了解整个工程的处理过程。

由于 Makefile 的知识比较多，完全可以单独写本书，因此本章我们只讲解 Makefile 基础入门，如果想详细的研究 Makefile，推荐大家阅读《跟我一起写 Makefile》这份文档，文档已经放到了开发板光盘->4、参考资料里面了，本章也有很多地方参考了此文档。

3.3.2 Makefile 的引入

我们完成这样一个小工程，通过键盘输入两个整形数字，然后计算他们的和并将结果显示在屏幕上，在这个工程中我们有 main.c、input.c 和 calcu.c 这三个 C 文件和 input.h、calcu.h 这两个头文件。其中 main.c 是主体，input.c 负责接收从键盘输入的数值，calcu.h 进行任意两个数相加，其中 main.c 文件内容如下：

示例代码 3.3.2.1 main.c 文件代码

```
1  #include <stdio.h>
2  #include "input.h"
3  #include "calcu.h"
4
5  int main(int argc, char *argv[])
6  {
7      int a, b, num;
8
9      input_int(&a, &b);
10     num  = calcu(a, b);
11     printf("%d + %d = %d\r\n", a, b, num);
12 }
```

input.c 文件内容如下：

示例代码 3.3.2.2 input.c 文件代码

```
1  #include <stdio.h>
```

```
2 #include "input.h"
3
4 void input_int(int *a, int *b)
5 {
6     printf("input two num:");
7     scanf("%d %d", a, b);
8     printf("\r\n");
9 }
```

calcu.c 文件内容如下：

示例代码 3.3.2.3 calcu.c 文件代码

```
1 #include "calcu.h"
2
3 int calcu(int a, int b)
4 {
5     return (a + b);
6 }
```

文件 input.h 内容如下：

示例代码 3.3.2.4 input.h 文件代码

```
1 #ifndef _INPUT_H
2 #define _INPUT_H
3
4 void input_int(int *a, int *b);
5 #endif
```

文件 calcu.h 内容如下：

示例代码 3.3.2.5 calcu.h 文件代码

```
1 #ifndef _CALCU_H
2 #define _CALCU_H
3
4 int calcu(int a, int b);
5 #endif
```

以上就是我们这个小工程的所有源文件，我们接下来使用 3.1 节讲的方法来对其进行编译，在终端输入如下命令：

```
gcc main.c calcu.c input.c -o main
```

上面命令的意思就是使用 gcc 编译器对 main.c、calcu.c 和 input.c 这三个文件进行编译，编译生成的可执行文件叫做 main。编译完成以后执行 main 这个程序，测试一下软件是否工作正常，结果如图 3.3.2.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcu.c  calcu.h  input.c  input.h  main  main.c
zuozhongkai@ubuntu:~/C_Program/3.3$ ./main
input two num:5 6

5 + 6 = 11
```

图 3.3.2.1 程序测试

可以看出我们的代码按照我们所设想的工作了，使用命令“gcc main.c calcu.c input.c -o main”

看起来很简单是吧，只需要一行就可以完成编译，但是我们这个工程只有三个文件啊！如果几千个文件呢？再就是如果有一个文件被修改了以，使用上面的命令编译的时候所有的文件都会重新编译，如果工程有几万个文件(Linux 源码就有这么多文件！)，想想这几万个文件编译一次所需要的时间就可怕。最好的办法肯定是哪个文件被修改了，只编译这个被修改的文件即可，其它没有修改的文件就不需要再次重新编译了，为此我们改变我们的编译方法，如果第一次编译工程，我们先将工程中的文件都编译一遍，然后后面修改了哪个文件就编译哪个文件，命令如下：

```
gcc -c main.c
gcc -c input.c
gcc -c calcu.c
gcc main.o input.o calcu.o -o main
```

上述命令前三行分别是将 main.c、input.c 和 calcu.c 编译成对应的.o 文件，所以使用了“-c”选项，“-c”选项我们上面说了，是只编译不链接。最后一行命令是将编译出来的所有.o 文件链接成可执行文件 main。假如我们现在修改了 calcu.c 这个文件，只需要将 caclue.c 这一个文件重新编译成.o 文件，然后在将所有的.o 文件链接成可执行文件即，只需要下面两条命令即可：

```
gcc -c calcu.c
gcc main.o input.o calcu.o -o main
```

但是这样就又有一个问题，如果修改的文件一多，我自己可能都不记得哪个文件修改过了，然后忘记编译，然后……，为此我们需要这样一个工具：

1、如果工程没有编译过，那么工程中的所有.c 文件都要被编译并且链接成可执行程序。

2、如果工程中只有个别 C 文件被修改了，那么只编译这些被修改的 C 文件即可。

3、如果工程的头文件被修改了，那么我们需要编译所有引用这个头文件的 C 文件，并且链接成可执行文件。

很明显，能够完成这个功能的就是 Makefile 了，在工程目录下创建名为“Makefile”的文件，文件名一定要叫做“Makefile”!!!区分大小写的哦！如图 3.3.2.2 所示：

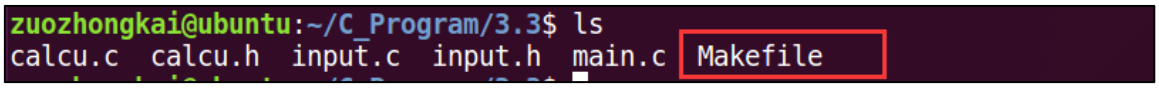


图 3.3.2.2 Makefile 文件

在图 3.3.2.2 中 Makefile 和 C 文件是处于同一个目录的，在 Makefile 文件中输入如下代码：

示例代码 3.3.2.6 Makefile 文件代码

```
1  main: main.o input.o calcu.o
2      gcc -o main  main.o input.o calcu.o
3  main.o: main.c
4      gcc -c main.c
5  input.o: input.c
6      gcc -c input.c
7  calcu.o: calcu.c
8      gcc -c calcu.c
9
10 clean:
11     rm *.o
12     rm main
```

上述代码中所有行首需要空出来的地方一定要使用“TAB”键！不要使用空格键！这是 Makefile 的语法要求，编写好得 Makefile 如图 3.3.2.3 所示：

```
1 main: main.o input.o calcu.o
2     gcc -o main  main.o input.o calcu.o
3 main.o: main.c
4     gcc -c main.c
5 input.o: input.c
6     gcc -c input.c
7 calcu.o: calcu.c
8     gcc -c calcu.c
9
10 clean:
11     rm *.o
12     rm main
13
```

图 3.3.2.3 Makefile 源码

Makefile 编写好以后我们就可以使用 make 命令来编译我们的工程了，直接在命令行中输入“make”即可，make 命令会在当前目录下查找是否存在“Makefile”这个文件，如果存在的话就会按照 Makefile 里面定义的编译方式进行编译，如图 3.3.2.4 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls  //查看目录下所有文件
calcu.c  calcu.h  input.c  input.h  main.c  Makefile
zuozhongkai@ubuntu:~/C_Program/3.3$ make //make命令编译工程
gcc -c main.c
gcc -c input.c
gcc -c calcu.c
gcc -o main  main.o input.o calcu.o
zuozhongkai@ubuntu:~/C_Program/3.3$ ls  //查看编译完成后的文件夹，看看编译是否成功
calcu.c  calcu.o  input.h  main    main.o
calcu.h  input.c  input.o  main.c  Makefile
```

图 3.3.2.4 Make 编译工程

在图 3.3.2.4 中，使用命令“make”编译完成以后就会在当前工程目录下生成各种.o 和可执行文件，说明我们编译成功了。使用 make 命令编译工程的时候可能会提示如图 3.3.2.5 所示错误:

```
zuozhongkai@ubuntu:~/C_Program/3.3$ make
Makefile:2: *** missing separator。  停止。
```

图 3.3.2.5 Make 失败

图 3.3.2.5 中的错误来源一般有两点:

1、Makefile 中命令缩进没有使用 TAB 键！

2、VI/VIM 编辑器使用空格代替了 TAB 键，修改文件/etc/vim/vimrc，在文件最后面加上如下所示代码：

```
set noexpandtab
```

我们修改一下 input.c 文件源码，随便加几行空行就行了，保证 input.c 被修改过即可，修改完成以后再执行一下“make”命令重新编译一下工程，结果如图 3.3.2.6 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ make
gcc -c input.c
gcc -o main  main.o input.o calcu.o
```

图 3.3.2.6 重新编译工程

从图 3.3.2.6 中可以看出因为我们修改了 input.c 这个文件，所以 input.c 和最后的可执行文

件 main 重新编译了，其它没有修改过的文件就没有编译。而且我们只需要输入“make”这个命令即可，非常方便，但是 Makefile 里面的代码都是什么意思呢？这就是接下来我们要讲解的。

3.4 Makefile 语法

3.4.1 Makefile 规则格式

Makefile 里面是由一系列的规则组成的，这些规则格式如下：

```
目标…... :  依赖文件集合……
    命令 1
    命令 2
    ……
```

比如下面这条规则：

```
 main :  main.o  input.o  calcu.o
        gcc  -o  main  main.o  input.o  calcu.o
```

这条规则的目标是 main，main.o、input.o 和 calcu.o 是生成 main 的依赖文件，如果要更新目标 main，就必须先更新它的所有依赖文件，如果依赖文件中的任何一个有更新，那么目标也必须更新，“更新”就是执行一遍规则中的命令列表。

命令列表中的每条命令必须以 TAB 键开始，不能使用空格！

命令列表中的每条命令必须以 TAB 键开始，不能使用空格

命令列表中的每条命令必须以 TAB 键开始，不能使用空格！

make 命令会为 Makefile 中的每个以 TAB 开始的命令创建一个 Shell 进程去执行。

了解了 Makefile 的基本运行规则以后我们再来分析一下 3.3 节中“示例代码 3.3.2.6”中的 Makefile，代码如下：

```
1  main: main.o input.o calcu.o
2      gcc -o main  main.o input.o calcu.o
3  main.o: main.c
4      gcc -c main.c
5  input.o: input.c
6      gcc -c input.c
7  calcu.o: calcu.c
8      gcc -c calcu.c
9
10 clean:
11     rm *.o
12     rm main
```

上述代码中一共有 5 条规则，1~2 行为第一条规则，3~4 行为第二条规则，5~6 行为第三条规则，7~8 行为第四条规则，10~12 为第五条规则，make 命令在执行这个 Makefile 的时候其执行步骤如下：

首先更新第一条规则中的 main，第一条规则的目标成为默认目标，只要默认目标更新了那么就认为 Makefile 的工作，完成了整个 Makefile 就是为了完成这个工作。在第一次编译的时候由于 main 还不存在，因此第一条规则会执行，第一条规则依赖于文件 main.o、input.o 和 calcu.o 这个三个.o 文件，这三个.o 文件目前还都没有，因此必须先更新这三个文件。make 会查找以这三个.o 文件为目标的规则并执行。以 main.o 为例，发现更新 main.o 的是第二条规则，因此会执

行第二条规则，第二条规则里面的命令为“gcc –c main.c”，这行命令很熟悉了吧，就是不链接编译 main.c，生成 main.o，其它两个.o 文件同理。最后一个规则目标是 clean，它没有依赖文件，因此会默认为依赖文件都是最新的，所以其对应的命令不会执行，当我们想要执行 clean 的话可以直接使用命令“make clean”，执行以后就会删除当前目录下所有的.o 文件以及 main，因此 clean 的功能就是完成工程的清理，“make clean”的执行过程如图 3.4.1.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcu.c  calcu.o  input.h  main    main.o
calcu.h  input.c  input.o  main.c  Makefile
zuozhongkai@ubuntu:~/C_Program/3.3$ make clean
rm *.o
rm main
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcu.c  calcu.h  input.c  input.h  main.c  Makefile
```

图 3.4.1.1 make clean 执行过程

从图 3.4.1.1 可以看出，当执行“make clean”命令以后，前面编译出来的.o 和 main 可执行文件都被删除掉了，也就是完成了工程清理工作。

我们在来总结一下 Make 的执行过程：

1、make 命令会在当前目录下查找以 Makefile(makefile 其实也可以)命名的文件。

2、当找到 Makefile 文件以后就会按照 Makefile 中定义的规则去编译生成最终的目标文件。

3、当发现目标文件不存在，或者目标所依赖的文件比目标文件新(也就是最后修改时间比目标文件晚)的话就会执行后面的命令来更新目标。

这就是 make 的执行过程，make 工具就是在 Makefile 中一层一层的查找依赖关系，并执行相应的命令。编译出最终的可执行文件。Makefile 的好处就是“自动化编译”，一旦写好了 Makefile 文件，以后只需要一个 make 命令即可完成整个工程的编译，极大的提高了开发效率。把 make 和 Makefile 和做菜类似，目标都是呈现出一场盛宴，它们之间的对比关系如表 3.4.1.1 所示：

make	做菜	描述
make 工具	厨师	负责将材料加工成“美味”。
gcc 编译器	厨具	加工“美味”的工具。
Makefile	食材	加工美味所需的原材料。
最终的可执行文件	最终的菜肴	最终目的，呈现盛宴

表 3.4.1.1 make 和做菜对比

总结一下，Makefile 中规则用来描述在什么情况下使用什么命令来构建一个特定的文件，这个文件就是规则的“目标”，为了生成这个“目标”而作为材料的其它文件称为“目标”的依赖，规则的命令是用来创建或者更新目标的。

除了 Makefile 的“终极目标”所在的规则以外，其它规则的顺序在 Makefile 中是没有意义的，“终极目标”就是指在使用 make 命令的时候没有指定具体的目标时，make 默认的那个目标，它是 Makefile 文件中第一个规则的目标，如果 Makefile 中的第一个规则有多个目标，那么这些目标中的第一个目标就是 make 的“终极目标”。

3.4.2 Makefile 变量

跟 C 语言一样 Makefile 也支持变量的，先看一下前面的例子：

```
main: main.o input.o calcu.o
    gcc -o main  main.o input.o calcu.o
```

上述 Makefile 语句中，main.o input.o 和 calcue.o 这三个依赖文件，我们输入了两遍，我们

这个 Makefile 比较小，如果 Makefile 复杂的时候这种重复输入的工作就会非常费时间，而且非常容易输错，为了解决这个问题，Makefile 加入了变量支持。不像 C 语言中的变量有 int、char 等各种类型，Makefile 中的变量都是字符串！类似 C 语言中的宏。使用变量将上面的代码修改，修改以后如下所示：

示例代码 3.4.2.1 Makefile 变量使用

```
1 #Makefile 变量的使用
2 objects = main.o input.o calcu.o
3 main: $(objects)
4     gcc -o main $(objects)
```

我们来分析一下“示例代码 3.4.2.1”，第 1 行是注释，Makefile 中可以写注释，注释开头要用符号“#”，不能用 C 语言中的“//”或者“/**/”！第 2 行我们定义了一个变量 objects，并且给这个变量进行了赋值，其值为字符串“main.o input.o calcu.o”，第 3 和 4 行使用到了变量 objects，Makefile 中变量的引用方法是“$(变量名)”，比如本例中的“$(objects)”就是使用变量 objects。

在“示例代码 3.4.2.1”中我们在定义变量 objects 的时候使用“=”对其进行了赋值，Makefile 变量的赋值符还有其它两个“:=”和“?=”，我们来看一下这三种赋值符的区别：

1、赋值符“=”

使用“=”在给变量的赋值的时候，不一定要用已经定义好的值，也可以使用后面定义的值，比如如下代码：

示例代码 3.4.2.1 赋值符"="使用

```
1 name = zzk
2 curname = $(name)
3 name = zuozhongkai
4
5 print:
6     @echo curname: $(curname)
```

我们来分析一下上述代码，第 1 行定义了一个变量 name，变量值为“zzk”，第 2 行也定义了一个变量 curname，curname 的变量值引用了变量 name，按照我们 C 写语言的经验此时 curname 的值就是“zzk”。第 3 行将变量 name 的值改为了“zuozhongkai”，第 5、6 行是输出变量 curname 的值。在 Makefile 要输出一串字符的话使用“echo”，就和 C 语言中的“printf”一样，第 6 行中的“echo”前面加了个“@”符号，因为 Make 在执行的过程中会自动输出命令执行过程，在命令前面加上“@”的话就不会输出命令执行过程，大家可以测试一下不加“@”的效果。使用命令“make print”来执行上述代码，结果如图 3.4.2.1：

```
zuozhongkai@ubuntu:~/C_Program$ make print
curname: zuozhongkai
zuozhongkai@ubuntu:~/C_Program$
```

图 3.4.2.1 make 执行结果

在图 3.4.2.1 中可以看到 curname 的值不是“zzk”，竟然是“zuozhongkai”，也就是变量“name”最后一次赋值的结果，这就是赋值符“=”的神奇之处！借助另外一个变量，可以将变量的真实值推到后面去定义。也就是变量的真实值取决于它所引用的变量的最后一次有效值。

2、赋值符“:=”

在“示例代码 3.4.2.1”上来测试赋值符“:=”，修改“示例代码 3.4.2.1”中的第 2 行，将其中的“=”改为“:=”，修改完成以后的代码如下：

示例代码 3.4.2.2 ":="的使用

```
1 name = zzk
2 curname := $(name)
3 name = zuozhongkai
4
5 print:
6     @echo curname: $(curname)
```

修改完成以后重新执行一下 Makefile，结果如图 3.4.2.2 所示：

```
zuozhongkai@ubuntu:~/C_Program$ make print
curname: zzk
zuozhongkai@ubuntu:~/C_Program$
```

图 3.4.2.2 make 执行结果

从图 3.4.2.2 中可以看到此时的 curname 是 zzk，不是 zuozhongkai 了。这是因为赋值符“:=”不会使用后面定义的变量，只能使用前面已经定义好的，这就是“=”和“:=”两个的区别。

3、赋值符“?=”

“?=”是一个很有用的赋值符，比如下面这行代码：

```
curname ?= zuozhongkai
```

上述代码的意思就是，如果变量 curname 前面没有被赋值，那么此变量就是“zuozhongkai”，如果前面已经赋过值了，那么就使用前面赋的值。

4、变量追加“+=”

Makefile 中的变量是字符串，有时候我们需要给前面已经定义好的变量添加一些字符串进去，此时就要使用到符号“+=”，比如如下所示代码：

```
objects = main.o inpiut.o
objects += calcu.o
```

一开始变量 objects 的值为“main.o input.o”，后面我们给他追加了一个“calcu.o”，因此变量 objects 变成了“main.o input.o calcu.o”，这个就是变量的追加。

3.4.3 Makefile 模式规则

在 3.3.2 小节中我们编写了一个 Makefile 文件用来编译工程，这个 Makefile 的内容如下：

示例代码 3.4.3.1 Makefile 文件代码

```
1  main: main.o input.o calcu.o
2      gcc -o main  main.o input.o calcu.o
3  main.o: main.c
4      gcc -c main.c
5  input.o: input.c
6      gcc -c input.c
7  calcu.o: calcu.c
8      gcc -c calcu.c
9
10 clean:
11     rm *.o
12     rm main
```

上述 Makefile 中第 3~8 行是将对应的.c 源文件编译为.o 文件，每一个 C 文件都要写一个对应的规则，如果工程中 C 文件很多的话显然不能这么做。为此，我们可以使用 Makefile 中的模式规则，通过模式规则我们就可以使用一条规则来将所有的.c 文件编译为对应的.o 文件。

模式规则中，至少在规则的目标定定义中要包涵“%”，否则就是一般规则，目标中的“%”表示对文件名的匹配，“%”表示长度任意的非空字符串，比如“%.c”就是所有的以.c 结尾的文件，类似与通配符，a.%.c 就表示以 a.开头，以.c 结束的所有文件。

当“%”出现在目标中的时候，目标中“%”所代表的值决定了依赖中的“%”值，使用方法如下：

```
%.o : %.c
    命令
```

因此“示例代码 3.4.3.1”中的 Makefile 可以改为如下形式：

示例代码 3.4.3.2 模式规则使用

```
1  objects = main.o input.o calcu.o
2  main: $(objects)
3      gcc -o main $(objects)
4
5  %.o : %.c
6      #命令
7
8  clean:
9      rm *.o
10     rm main
```

“示例代码 3.4.3.2”中第 5、6 这两行代码替代了“示例代码 3.4.3.1”中的 3~8 行代码，修改以后的 Makefile 还不能运行，因为第 6 行的命令我们还没写呢，第 6 行的命令我们需要借助另外一种强大的变量—自动化变量。

3.4.4 Makefile 自动化变量

上面讲的模式规则中，目标和依赖都是一系列的文件，每一次对模式规则进行解析的时候都会是不同的目标和依赖文件，而命令只有一行，如何通过一行命令来从不同的依赖文件中生成对应的目标？自动化变量就是完成这个功能的！所谓自动化变量就是这种变量会把模式中所定义的一系列的文件自动的挨个取出，直至所有的符合模式的文件都取完，自动化变量只应该出现在规则的命令中，常用的自动化变量如表 3.4.4.1：

自动化变量	描述
$@	规则中的目标集合，在模式规则中，如果有多个目标的话，“$@”表示匹配模式中定义的目标集合。
$%	当目标是函数库的时候表示规则中的目标成员名，如果目标不是函数库文件，那么其值为空。
$<	依赖文件集合中的第一个文件，如果依赖文件是以模式(即“%”)定义的，那么“$<”就是符合模式的一系列的文件集合。
$?	所有比目标新的依赖目标集合，以空格分开。
$^	所有依赖文件的集合，使用空格分开，如果在依赖文件中有多个重复的文件，“$^”会去除重复的依赖文件，值保留一份。

$+	和“$^”类似，但是当依赖文件存在重复的话不会去除重复的依赖文件。
$*	这个变量表示目标模式中"%"及其之前的部分，如果目标是 test/a.test.c，目标模式为 a.%.c，那么“$*”就是 test/a.test。

表 3.4.4.1 自动化变量

表 3.4.4.1 中的 7 个自动化变量中，常用的三种：$@、$<和$^，我们使用自动化变量来完成“示例代码 3.4.3.2”中的 Makefile，最终的完整代码如下所示：

示例代码 3.4.4.1 自动化变量

```
1  objects = main.o input.o calcu.o
2  main: $(objects)
3      gcc -o main $(objects)
4
5  %.o : %.c
6      gcc -c $<
7
8  clean:
9      rm *.o
10     rm main
```

上述代码代码就是修改后的完成的 Makefile，可以看出相比 3.3.2 小节中的要精简了很多，核心就在于第 5、6 这两行，第 5 行使用了模式规则，第 6 行使用了自动化变量。

3.4.5 Makefile 伪目标

Makefile 有一种特殊的目标——伪目标，一般的目标名都是要生成的文件，而伪目标不代表真正的目标名，在执行 make 命令的时候通过指定这个伪目标来执行其所在规则的定义的命令。

使用伪目标的主要是为了避免 Makefile 中定义的只执行命令的目标和工作目录下的实际文件出现名字冲突，有时候我们需要编写一个规则用来执行一些命令，但是这个规则不是用来创建文件的，比如在前面的“示例代码 3.4.4.1”中有如下代码用来完成清理工程的功能：

```
clean:
    rm *.o
    rm main
```

上述规则中并没有创建文件 clean 的命令，因此工作目录下永远都不会存在文件 clean，当我们输入“make clean”以后，后面的“rm *.o”和“rm main”总是会执行。可是如果我们“手贱”，在工作目录下创建一个名为“clean”的文件，那就不一样了，当执行“make clean”的时候，规则因为没有依赖文件，所以目标被认为是最新的，因此后面的 rm 命令也就不会执行，我们预先设想的清理工程的功能也就无法完成。为了避免这个问题，我们可以将 clean 声明为伪目标，声明方式如下：

```
.PHONY : clean
```

我们使用伪目标来更改“示例代码 3.4.4.1”，修改完成以后如下：

示例代码 3.4.5.1 伪目标

```
1  objects = main.o input.o calcu.o
2  main: $(objects)
3      gcc -o main $(objects)
4
```

```
5  .PHONY : clean
6
7  %.o : %.c
8      gcc -c $<
9
10 clean:
11     rm *.o
12     rm main
```

上述代码第 5 行声明 clean 为伪目标，声明 clean 为伪目标以后不管当前目录下是否存在名为“clean”的文件，输入“make clean”的话规则后面的 rm 命令都会执行。

3.4.6 Makefile 条件判断

在 C 语言中我们通过条件判断语句来根据不同的情况来执行不同的分支，Makefile 也支持条件判断，语法有两种如下：

```
<条件关键字>
    <条件为真时执行的语句>
endif
```

以及：

```
<条件关键字>
    <条件为真时执行的语句>
else
    <条件为假时执行的语句>
endif
```

其中条件关键字有 4 个：ifeq、ifneq、ifdef 和 ifndef，这四个关键字其实分为两对、ifeq 与 ifneq、ifdef 与 ifndef，先来看一下 ifeq 和 ifneq，ifeq 用来判断是否相等，ifneq 就是判断是否不相等，ifeq 用法如下：

```
ifeq (<参数 1>, <参数 2>)
ifeq ‘<参数 1 >’ , ‘ <参数 2>’
ifeq “<参数 1>” , “<参数 2>”
ifeq “<参数 1>” , ‘<参数 2>’
ifeq ‘<参数 1>’ , “<参数 2>”
```

上述用法中都是用来比较“参数 1”和“参数 2”是否相同，如果相同则为真，“参数 1”和“参数 2”可以为函数返回值。ifneq 的用法类似，只不过 ifneq 是用来了比较“参数 1”和“参数 2”是否不相等，如果不相等的话就为真。

ifdef 和 ifndef 的用法如下：

```
ifdef <变量名>
```

如果“变量名”的值非空，那么表示表达式为真，否则表达式为假。“变量名”同样可以是一个函数的返回值。ifndef 用法类似，但是含义用户 ifdef 相反。

3.4.7 Makefile 函数使用

Makefile 支持函数，类似 C 语言一样，Makefile 中的函数是已经定义好的，我们直接使用，不支持我们自定义函数。make 所支持的函数不多，但是绝对够我们使用了，函数的用法如下：

```
$(函数名 参数集合)
```

或者

```
${函数名 参数集合}
```

可以看出，调用函数和调用普通变量一样，使用符号“$”来标识。参数集合是函数的多个参数，参数之间以逗号“,”隔开，函数名和参数之间以“空格”分隔开，函数的调用以“$”开头。接下来我们介绍几个常用的函数，其它的函数大家可以参考《跟我一起写 Makefile》这份文档。

1、函数 subst

函数 subst 用来完成字符串替换，调用形式如下：

```
$(subst <from>,<to>,<text>)
```

此函数的功能是将字符串<text>中的<from>内容替换为<to>，函数返回被替换以后的字符串，比如如下示例：

```
$(subst    zzk,ZZK,my name is zzk)
```

把字符串“my name is zzk”中的“zzk”替换为“ZZK”，替换完成以后的字符串为“my name is ZZK”。

2、函数 patsubst

函数 patsubst 用来完成模式字符串替换，使用方法如下：

```
$(patsubst <pattern>,<replacement>,<text>)
```

此函数查找字符串<text>中的单词是否符合模式<pattern>，如果匹配就用<replacement>来替换掉，<pattern>可以使用包括通配符“%”，表示任意长度的字符串，函数返回值就是替换后的字符串。如果<replacement>中也包涵“%”，那么<replacement>中的“%”将是<pattern>中的那个“%”所代表的字符串，比如：

```
$(patsubst %.c,%.o,a.c b.c c.c)
```

将字符串“a.c b.c c.c”中的所有符合“%.c”的字符串，替换为“%.o”，替换完成以后的字符串为“a.o b.o c.o”。

3、函数 dir

函数 dir 用来获取目录，使用方法如下：

```
$(dir <names…>)
```

此函数用来从文件名序列<names>中提取出目录部分，返回值是文件名序列<names>的目录部分，比如：

```
$(dir </src/a.c>)
```

提取文件“/src/a.c”的目录部分，也就是“/src”。

4、函数 notdir

函数 notdir 看名字就是知道去除文件中的目录部分，也就是提取文件名，用法如下：

```
$(notdir <names…>)
```

此函数用与从文件名序列<names>中提取出文件名非目录部分，比如：

```
$(notdir </src/a.c>)
```

提取文件“/src/a.c”中的非目录部分，也就是文件名“a.c”。

5、函数 foreach

foreach 函数用来完成循环，用法如下：

```
$(foreach <var>, <list>,<text>)
```

此函数的意思就是把参数<list>中的单词逐一取出来放到参数<var>中，然后再执行<text>所包含的表达式。每次<text>都会返回一个字符串，循环的过程中，<text>中所包含的每个字符串会以空格隔开，最后当整个循环结束时，<text>所返回的每个字符串所组成的整个字符串将会是函数 foreach 函数的返回值。

6、函数 wildcard

通配符“%”只能用在规则中，只有在规则中它才会展开，如果在变量定义和函数使用时，通配符不会自动展开，这个时候就要用到函数 wildcard，使用方法如下：

```
$(wildcard PATTERN…)
```

比如：

```
$(wildcard *.c)
```

上面的代码是用来获取当前目录下所有的.c 文件，类似“%”。

关于 Makefile 的相关内容就讲解到这里，本节只是对 Makefile 做了最基本的讲解，确保大家能够完成后续的学习，Makefile 还有大量的知识没有提到，有兴趣的可以自行参考《跟我一起写 Makefile》这份文档来深入学习 Makefile。

第二篇 系统移植篇

在进行 Linux 驱动开发之前肯定需要先将 Linux 系统移植到开发板上去。如果学习过 UCOS/FreeRTOS 应该知道，UCOS/FreeRTOS 移植就是在官方的 SDK 包里面找一个和自己所使用的芯片一样的工程编译一下，然后下载到开发板就可以了。那么 Linux 的移植是不是也是这样的，下载 Linux 源码，然后找个和我们所使用的芯片一样的工程编译一下就可以了？很明显不是的！Linux 的移植要复杂的多，在移植 Linux 之前我们需要先移植一个 bootloader 代码，这个 bootloader 代码用于启动 Linux 内核，bootloader 有很多，常用的就是 U-Boot。移植好 U-Boot 以后再移植 Linux 内核，移植完 Linux 内核以后 Linux 还不能正常启动，还需要再移植一个根文件系统(rootfs)，根文件系统里面包含了一些最常用的命令和文件。所以 U-Boot、Linux Kernel 和 rootfs 这三者一起构成了一个完整的 Linux 系统，一个可以正常使用、功能完善的 Linux 系统。在本篇我们就来讲解 U-Boot、Linux Kernel 和 rootfs 的移植，与其说是“移植”，倒不如说是“适配”，因为大部分的移植工作都由 ST 完成了，我们这里所谓的“移植”主要是使其能够在正点原子 STM32MP157 开发板上跑起来。

第四章 开发环境搭建

要进行 Linux 驱动开发肯定要先搭建好开发环境，我们在开始学习 STM32 单片机的时候肯定需要安装一堆的软件，比如 MDK、IAR、串口调试助手等等，这个就是 STM32 的开发环境搭建。同样的，要想在 Ubuntu 下进行 Cortex-A(STM32MP157)开发也需要安装一些软件，也就是网上说的开发环境搭建，环境搭建好以后我们就可以进行开发了。环境搭建分为 Ubuntu 和 Windows，因为我们最熟悉 Windows，所以代码编写、查找资料啥的肯定是在 Windows 下进行的。但是 Linux 开发又必须在 Ubuntu 下进行，所以还需要搭建 Ubuntu 下的开发环境，主要是交叉编译器的安装，本章我们就分为 Ubuntu 和 Windows，讲解这两种操作系统下的环境搭建。

4.1 Ubuntu 和 Windows 文件互传

在开发的过程中会频繁的在 Windows 和 Ubuntu 下进行文件传输，比如在 Windwos 下进行代码编写，然后将编写好的代码拿到 Ubuntu 下进行编译。Windows 和 Ubuntu 下的文件互传我们需要使用 FTP 服务，设置方法如下：

1、开启 Ubuntu 下的 FTP 服务

打开 Ubuntu 的终端窗口，然后执行如下命令来安装 FTP 服务：

```
sudo apt-get install vsftpd
```

等待软件自动安装，安装完成以后使用如下 VI 命令打开/etc/vsftpd.conf，命令如下：

```
sudo vi /etc/vsftpd.conf
```

打开以后 vsftpd.conf 文件以后找到如下两行：

```
local_enable=YES
write_enable=YES
```

确保上面两行前面没有“#”，有的话就取消掉，完成以后如图 4.1.1 所示：

```
27 # Uncomment this to allow local users to log in.
28 local_enable=YES
29 #
30 # Uncomment this to enable any form of FTP write command.
31 write_enable=YES
```

图 4.1.1 vsftpd.conf 修改

修改完 vsftpd.conf 以后保存退出，使用如下命令重启 FTP 服务：

```
sudo /etc/init.d/vsftpd restart
```

2、Windows 下 FTP 客户端安装

Windows 下 FTP 客户端我们使用 FileZilla，这是个免费的 FTP 客户端软件，可以在 FileZilla 官网下载，下载地址如下：https://www.filezilla.cn/download，下载界面如图 4.1.2 所示：

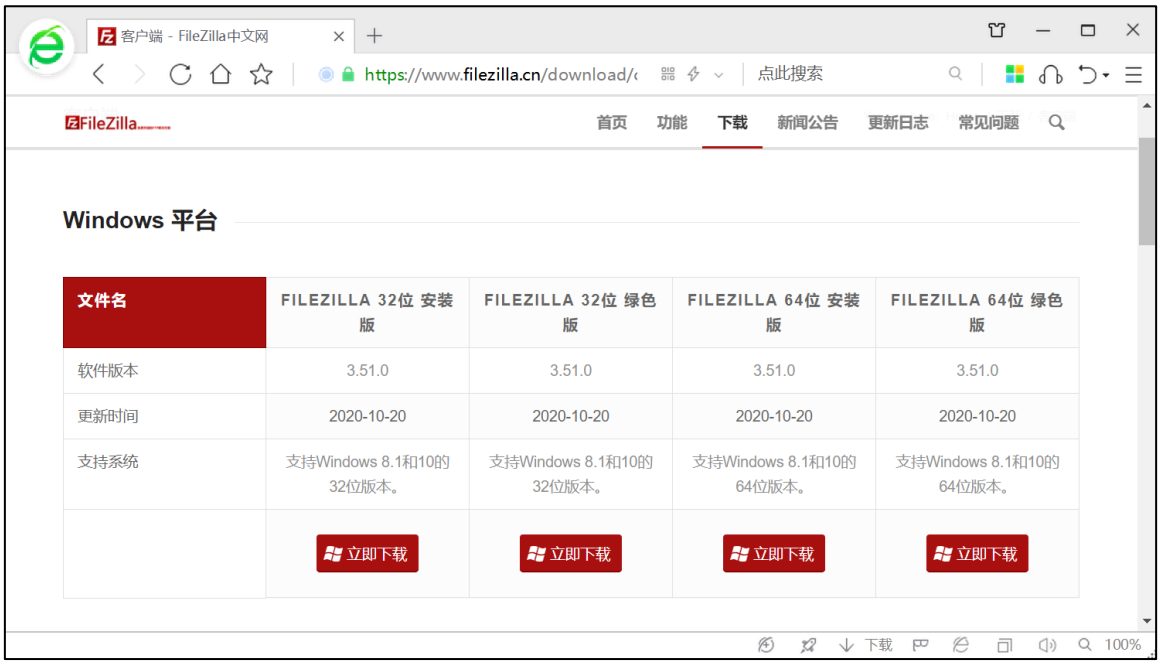


图 4.1.2 FileZilla 软件下载

如果是 32 位电脑就选择 32 位版本，64 位电脑就选择 64 位版本，我们已经下载好了 64 位

版本的 FileZilla 并放到开发板光盘中了，路径为:3、软件-> FileZilla_3.51.0_win64-setup.exe，双击安装即可。安装完成以后找到安装目录，找到图标，然后发送图标快捷方式到桌面，完成以后如图 4.1.3 所示：

图 4.1.3 FileZilla 图标

打开 FileZilla 软件，界面如图 4.1.4 所示：

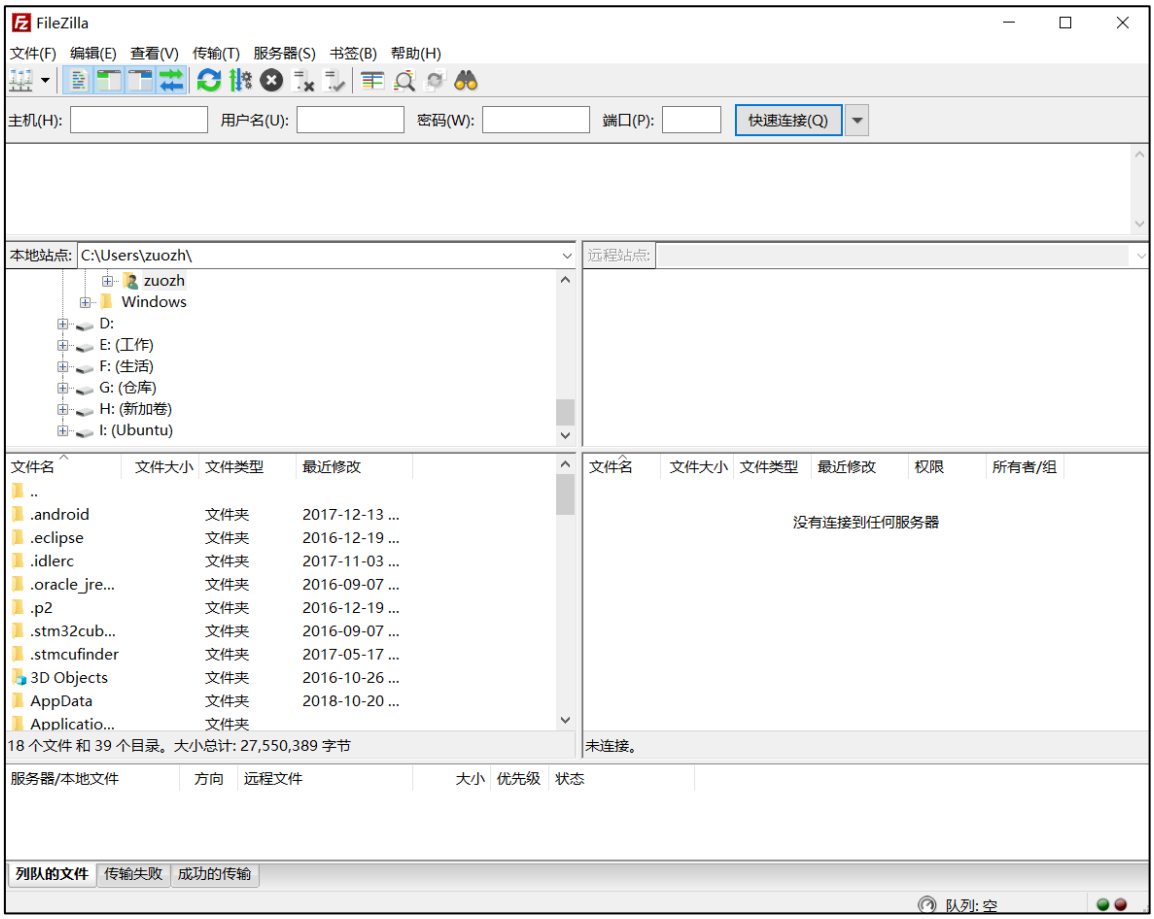


图 4.1.4 FileZilla 软件界面

3、FileZilla 软件设置

Ubuntu 作为 FTP 服务器，FileZilla 作为 FTP 客户端，客户端肯定要连接到服务器上，打开站点管理器，点击：文件->站点管理器，打开以后如图 4.1.5 所示：

图 4.1.5 站点管理器

点击图 4.1.5 中的“新站点(N)”按钮来创建站点，新建站点以后就会在“我的站点”下出现新建的这个站点，站点的名称可以自行修改，比如我将新的站点命名为“Ubuntu”如图 4.1.6 所示：

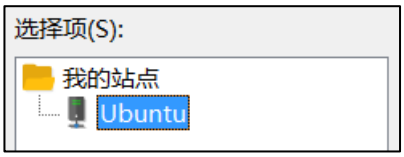


图 4.1.6 新建站点

选中新创建的“Ubuntu”站点，然后对站点的“常规”进行设置，设置如图 4.1.7 所示：

图 4.1.7 站点设置

按照图 4.1.7 中设置好以后，点击“连接”按钮，第一次连接可能会弹出提示是否保存密码的对话框，点击确定即可。连接成功以后如图 4.1.8 所示：

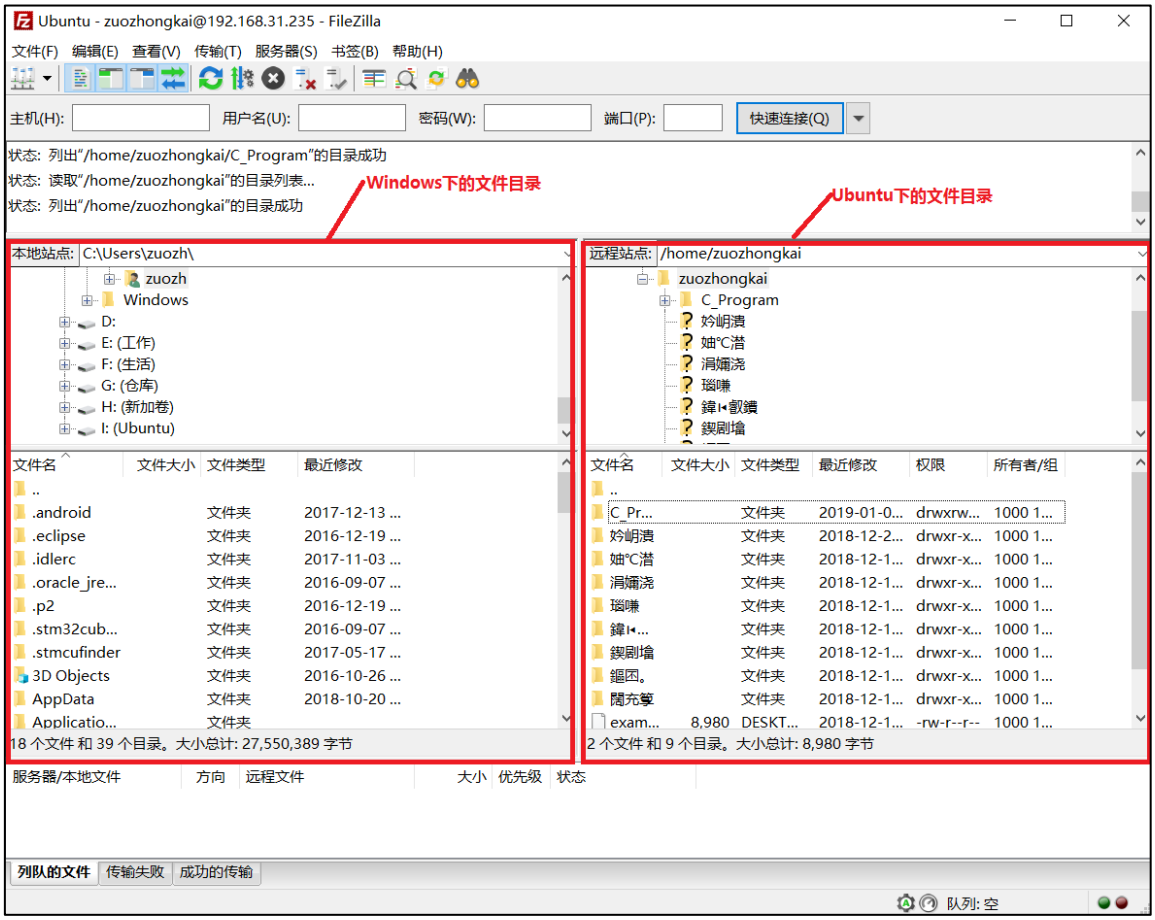


图 4.1.8 连接成功

连接成功以后如图 4.1.8 所示，其中左边就是 Windows 文件目录，右边是 Ubuntu 文件目录，默认进入用户根目录下（比如我电脑的“/home/zuozhongkai”）。但是注意观察在图 4.1.8 中 Ubuntu 文件目录下的中文目录都是乱码的，这是因为编码方式没有选对，先断开连接，点击：服务器(S)->断开连接，然后打开站点管理器，选中要设置的站点“Ubuntu”，选择“字符集”，设置如图 4.1.9 所示：

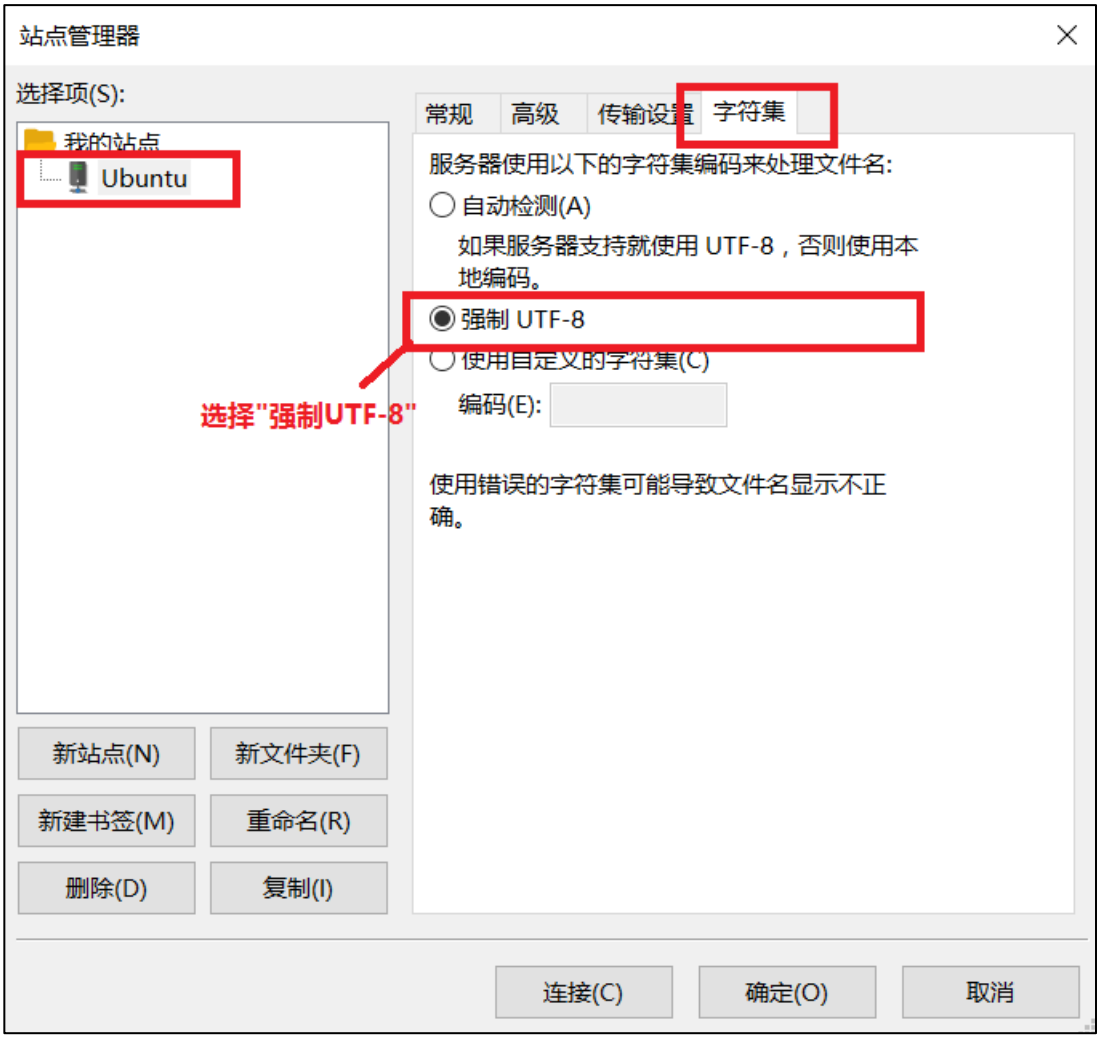


图 4.1.9 设置字符集

按照图 4.1.9 设置好字符集以后重新连接到 FTP 服务器上，重新链接到 FTP 服务器以后 Ubuntu 下的文件目录中文显示就正常了，如图 4.1.10 所示：

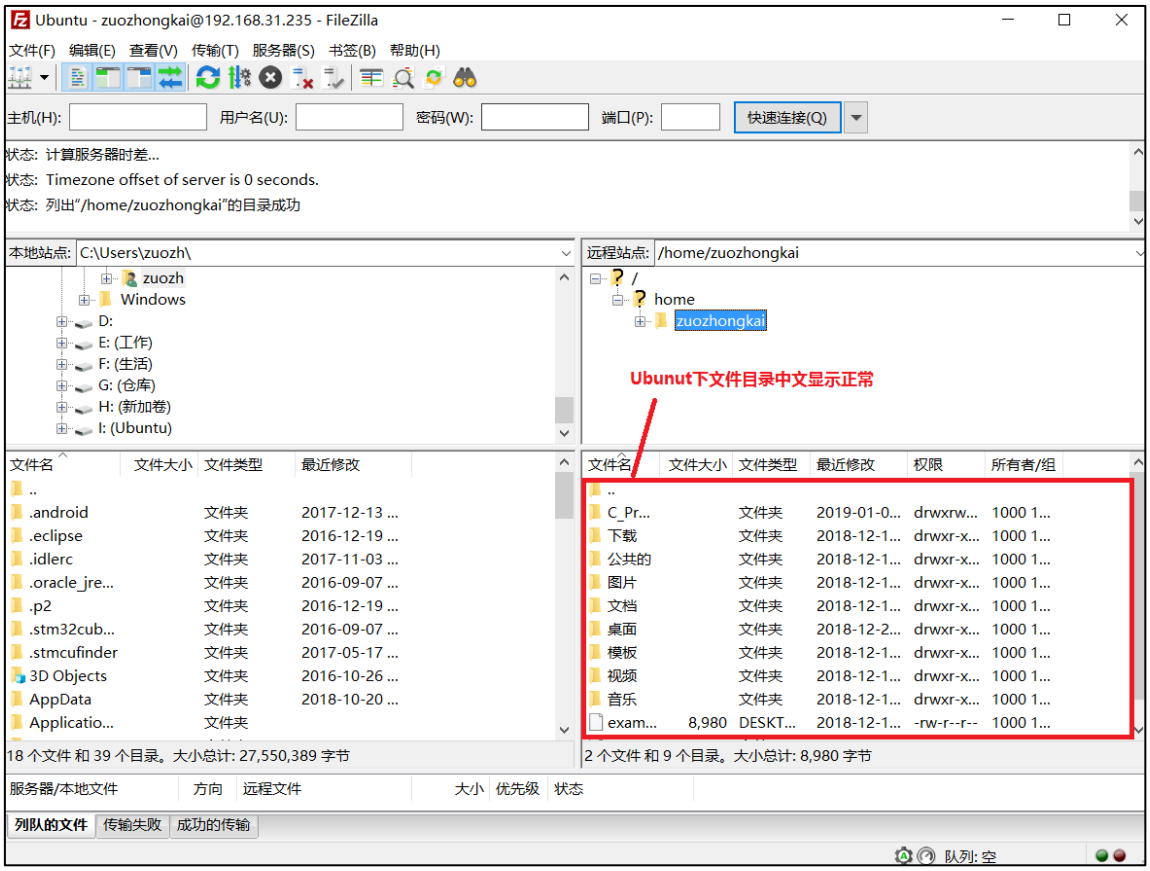


图 4.1.10 Ubuntu 下文件目录中文显示正常

如果要将 Windows 下的文件或文件夹拷贝到 Ubuntu 中，只需要在图 4.1.10 中左侧的 Windows 区域选中要拷贝的文件或者文件夹，然后直接拖到右侧的 Ubuntu 中指定的目录即可。将 Ubuntu 中的文件或者文件夹拷贝到 Windows 中也是直接拖放。

4.2 Ubuntu 下 NFS 和 SSH 服务开启

4.2.1 NFS 服务开启

后面进行 Linux 驱动开发的时候需要 NFS 启动，因此要先安装并开启 Ubuntu 中的 NFS 服务，使用如下命令安装 NFS 服务：

```
sudo apt-get install nfs-kernel-server rpcbind
```

等待安装完成，安装完成以后在用户根目录下创建一个名为“linux”的文件夹，以后所有的东西都放到这个“linux”文件夹里面，在“linux”文件夹里面新建一个名为“nfs”的文件夹，如图 4.2.1 所示：

```
zuozhongkai@ubuntu:~$ ls
C_Program  examples.desktop  linux  公共的  模板  视频  图片  文档  下载  音乐  桌面
zuozhongkai@ubuntu:~$ cd linux/
zuozhongkai@ubuntu:~/linux$ ls
nfs
zuozhongkai@ubuntu:~/linux$
```

图 4.2.1 创建 linux 工作目录

图 4.2.1 中创建的 nfs 文件夹供 nfs 服务器使用，以后我们可以在开发板上通过网络文件系统来访问 nfs 文件夹，要先配置 nfs，使用如下命令打开 nfs 配置文件/etc/exports：

```
sudo vi /etc/exports
```

打开/etc/exports 以后在后面添加如下所示内容：

```
/home/zuozhongkai/linux/nfs *(rw,sync,no_root_squash)
```

添加完成以后的/etc/exports 如图 4.2.2 所示：

```
1 # /etc/exports: the access control list for filesystems which may be exported
2 #       to NFS clients.  See exports(5).
3 #
4 # Example for NFSv2 and NFSv3:
5 # /srv/homes       hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
6 #
7 # Example for NFSv4:
8 # /srv/nfs4        gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
9 # /srv/nfs4/homes  gss/krb5i(rw,sync,no_subtree_check)
10 #
11
12 /home/zuozhongkai/linux/nfs *(rw,sync,no_root_squash)
```

图 4.2.2 修改文件/etc/exports

重启 NFS 服务，使用命令如下：

```
sudo /etc/init.d/nfs-kernel-server restart
```

4.2.2 SSH 服务开启

开启 Ubuntu 的 SSH 服务以后我们就可以在 Windwos 下使用终端软件登陆到 Ubuntu，比如使用 SecureCRT，Ubuntu 下使用如下命令开启 SSH 服务：

```
sudo apt-get install openssh-server
```

上述命令安装 ssh 服务，ssh 的配置文件为/etc/ssh/sshd_config，使用默认配置即可。

4.3 Ubuntu 交叉编译工具链安装

4.3.1 交叉编译器安装

ARM 裸机、Uboot 移植、Linux 移植这些都需要在 Ubuntu 下进行编译，编译就需要编译器，我们在第三章“Linux C 编程入门”里面已经讲解了如何在 Liux 进行 C 语言开发，里面使用 GCC 编译器进行代码编译，但是 Ubuntu 自带的 gcc 编译器是针对 X86 架构的！而我们现在要编译的是 ARM 架构的代码，所以我们需要一个在 X86 架构的 PC 上运行，可以编译 ARM 架构代码的 GCC 编译器，这个编译器就叫做交叉编译器，总结一下交叉编译器就是：

1、它肯定是一个 GCC 编译器。

2、这个 GCC 编译器是运行在 X86 架构的 PC 上的。

3、这个 GCC 编译器是编译 ARM 架构代码的，也就是编译出来的可执行文件是在 ARM 芯片上运行的。

交叉编译器中“交叉”的意思就是在一个架构上编译另外一个架构的代码，相当于两种架构“交叉”起来了。

交叉编译器有很多种，ST 也有推荐的通用交叉编译器，如图 4.3.1 所示：

5.2 ARM cross compiler

A cross compiler [7] must be installed on your Host (X86_64, i686, ...) for the ARM targeted Device architecture. In addition, the $PATH and $CROSS_COMPILE environment variables must be configured in your shell.

You can use gcc for ARM, available in:

- the SDK toolchain (see Cross-compile with OpenSTLinux SDK)
  PATH and CROSS_COMPILE are automatically updated.
- an existing package
  For example, install gcc-arm-linux-gnueabihf on Ubuntu/Debian: (**PC $>** sudo apt-get.
- an existing toolchain:
  - latest gcc toolchain provided by arm (https://developer.arm.com/open-source/gnu-toolchain/gnu-a/downloads/)
  - gcc v7 toolchain provided by linaro: (https://www.linaro.org/downloads/)

For example, to use *gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz* from arm, extract the toolchain in $HOME and update your environment with:

```
PC $> export PATH=$HOME/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf/bin:$PATH
PC $> export CROSS_COMPILE=arm-none-linux-gnueabihf-
```

For example, to use **gcc-linaro-7.2.1-2017.11-x86_64_arm-linux-gnueabi.tar.xz**
from https://releases.linaro.org/components/toolchain/binaries/7.2-2017.11/arm-linux-gnueabi/
Unzip the toolchain in $HOME and update your environment with:

```
PC $> export PATH=$HOME/gcc-linaro-7.2.1-2017.11-x86_64_arm-linux-gnueabi/bin:$PATH
PC $> export CROSS_COMPILE=arm-linux-gnueabi-
```

图 4.3.1 ST 推荐的通用交叉编译器

从图 4.3.1 可以看出，ST 推荐了两款通用交叉编译器，一个是 ARM 官方出品的：gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf，一个是 linaro 出品的：gcc-linaro-7.2.1-2017.11-x86_64_arm-linux-gnueabi.tar.xz，本教程我们使用 ARM 官方出品的交叉编译器。

首先是下载 ARM 官方出品的交叉编译器，图 4.3.1 中也给出了交叉编译器的下载链接：编译器下载地址如下：https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-a/downloads，打开以后如图 4.3.2 所示：

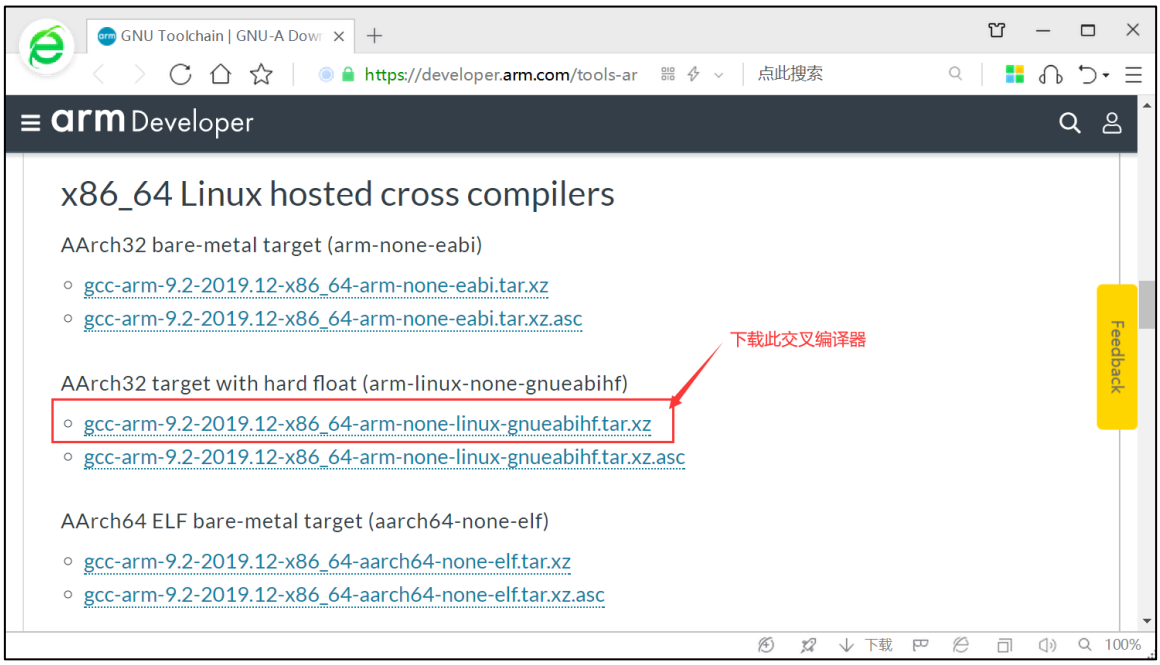


图 4.3.1.2 ARM 官方交叉比编译器下载界面

图 4.3.1.2 中的 gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz 是在 64 位 Ubuntu 下使用的交叉编译器。交叉编译器我们已经下载好放到了开发板光盘中，路径：开发板光盘→5、开发工具→1、交叉编译器→gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz。

注意！直接从 ARM 官方下载的交叉编译器在编译 TF-A 的时候会报错没有 stm32wrapper4dbg 这命令，后面会讲解如何解决这个问题。

我们要先将交叉编译工具拷贝到 Ubuntu 中，在 4.2.1 小节中我们在当前用户根目录下创建了一个名为“linux”的文件夹，在这个 linux 文件夹里面再创建一个名为“tool”的文件夹，用来存放一些开发工具。使用前面已经安装好的 FileZilla 将交叉编译器拷贝到 Ubuntu 中刚刚新建的“tool”文件夹中，操作如图 4.3.1.3 所示：

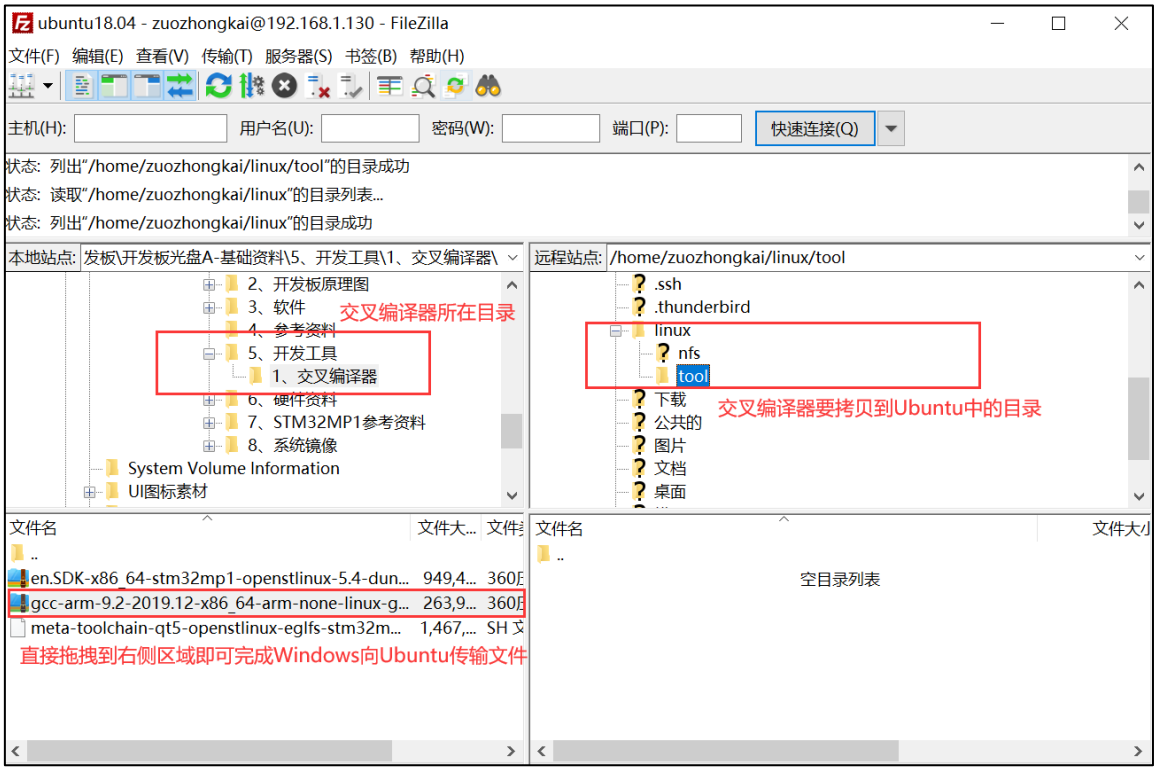


图 4.3.1.3 拷贝交叉编译器

拷贝完成的话 FileZilla 会有提示，如图 4.3.1.4：

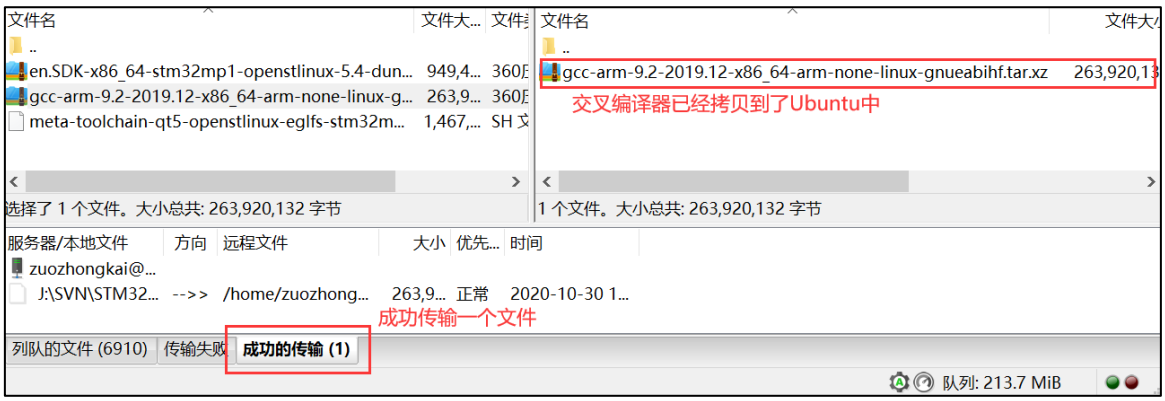


图 4.3.1.4 交叉编译器拷贝完成

在 Ubuntu 中创建目录：/usr/local/arm，命令如下：

```
sudo mkdir /usr/local/arm
```

创建完成以后将刚刚拷贝的交叉编译器复制到/usr/local/arm 这个目录中，在终端使用命令“cd”进入到存放有交叉编译器的目录，比我前面将交叉编译器拷贝到了目录“/home/zuozhongkai/linux/tool”中，然后使用如下命令将交叉编译器复制到/usr/local/arm 中：

```
sudo cp gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz /usr/local/arm/ -f
```

拷贝完成以后在/usr/local/arm 目录中对交叉编译工具进行解压，解压命令如下：

```
sudo tar -vxf gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz
```

等待解压完成，解压完成以后会生成一个名为“gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf”的文件夹，这个文件夹里面就是我们的交叉编译工具链。

修改环境变量，使用打开/etc/profile 文件，命令如下：

```
sudo vi /etc/profile
```

打开/etc/profile 以后，在最后面输入如下所示内容：

```
export PATH=$PATH:/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf/bin
```

添加完成以后的/etc/profile 如图 4.3.1.5 所示：

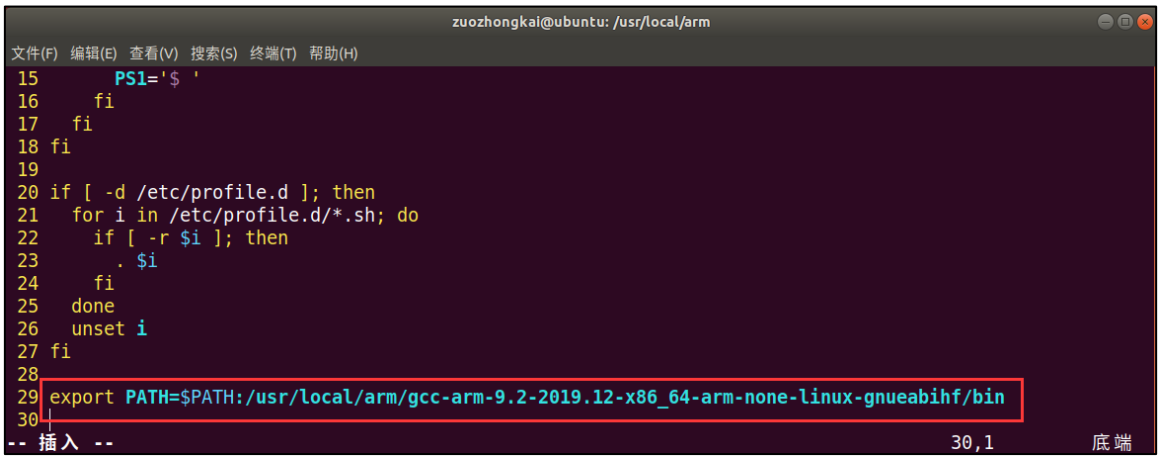


图 4.3.1.5 添加环境变量

修改好以后就保存退出，重启 Ubuntu 系统，交叉编译工具链(编译器)就安装成功了。

4.3.2 安装相关库

在使用交叉编译器之前还需要安装一下其它的库，命令如下：

```
sudo apt-get update                          //先更新，否则安装库可能会出错
sudo apt-get install lsb-core lib32stdc++6   //安装库
```

等待这些库安装完成。

4.3.3 交叉编译器验证

首先查看一下交叉编译工具的版本号，输入如下命令：

```
arm-none-linux-gnueabihf-gcc -v
```

如果交叉编译器安装正确的话就会显示版本号，如图 4.3.3.1 所示：

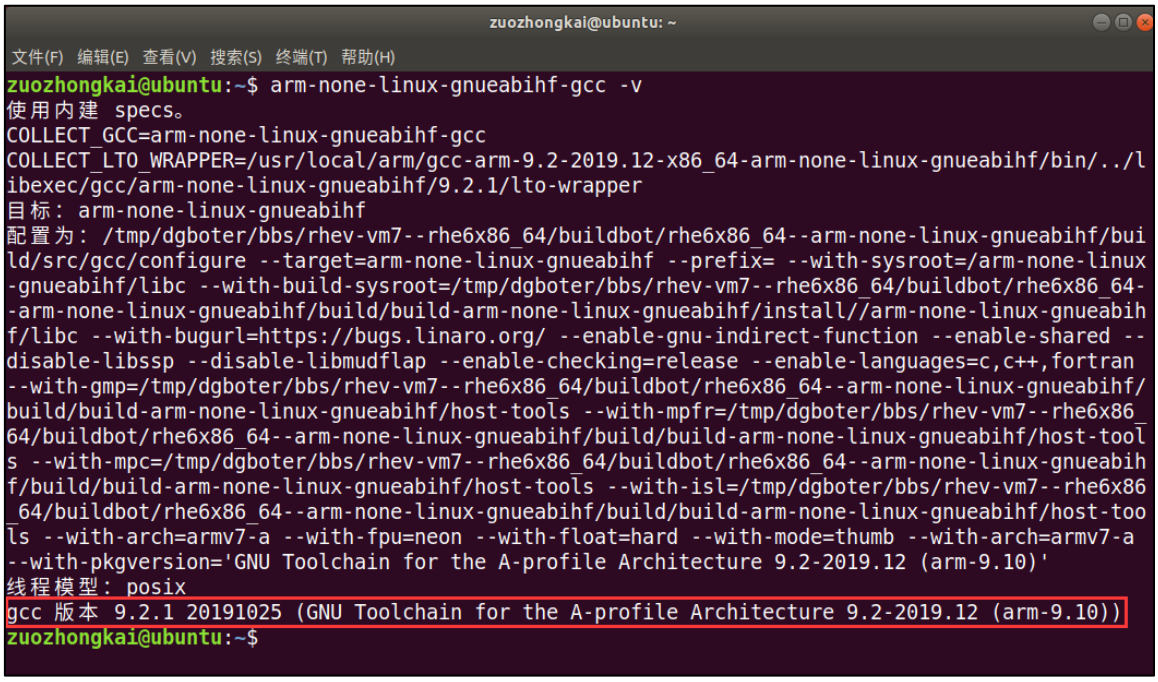


图 4.3.3.1 交叉编译器版本查询

从图 4.3.3.1 中可以看出当前交叉编译器的版本号为 9.2.1，说明交叉编译工具链安装成功。第三章“Linux C 编程入门”中使用 Ubuntu 自带的 GCC 编译器，我们用的是命令“gcc”。要使用刚刚安装的交叉编译器的时候使用的命令是“arm-none-linux-gnueabihf-gcc”，“arm-none-linux-gnueabihf-gcc”的含义如下：

1、arm 表示这是编译 arm 架构代码的编译器。

2、none 表示厂商，一般半导体厂商会修改通用的交叉编译器，此处就是半导体厂商的名字，ARM 自己做的交叉编译这里为 none，表示没有厂商。

3、linux 表示运行在 linux 环境下。

4、gnueabihf 表示嵌入式二进制接口，后面的 hf 是 hard float 的缩写，也就是硬件浮点，说明此交叉编译工具链支持硬件浮点。

5、gcc 表示是 gcc 工具。

4.4 Visual Studio Code 软件的安装和使用

4.4.1 Visual Studio Code 的安装

Visual Stuio Code 是一个编辑器，可以用来编写代码，Visual Studio Sode 本教程以后就简称为 VSCode，VSCode 是微软出的一款免费编辑器。VSCode 有 Windows、Linux 和 macOS 三个版本的，是一个跨平台的编辑器。VSCode 下载地址是：https://code.visualstudio.com/，下载界面如图 4.5.1.1 所示：

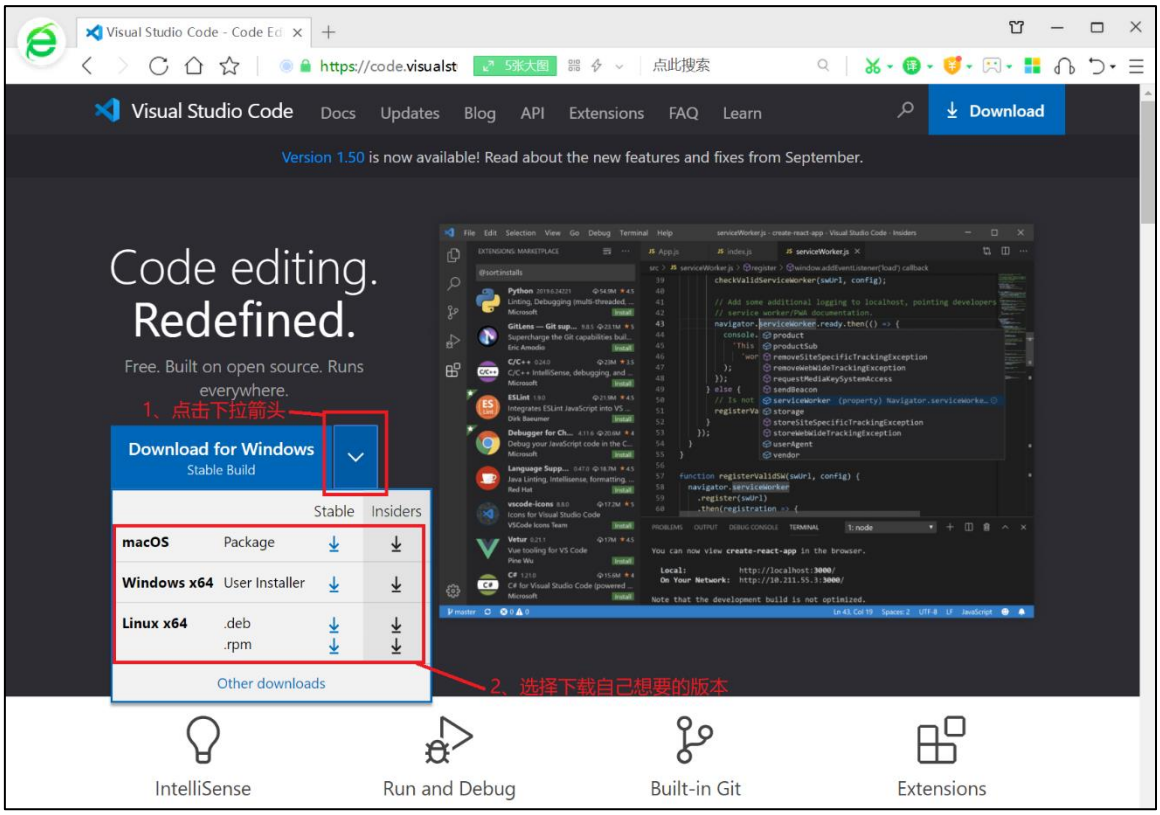


图 4.5.1.1 VSCode 下载界面

在图 4.5.1.1 中下载自己想要的版本，本教程需要 Windows 和 Linux 这两个版本，所以下载这两个即可，我们已经下载好并放入了开发板光盘中，路径为：3、软件->Visual Studio Code。

1、Windows 版本安装

Windows 版本的安装和容易，和其他 Windows 一样，双击.exe 安装包，然后一路“下一步”即可，安装完成以后在桌面上就会有 VSCode 的图标，如图 4.5.1.2 所示：

图 4.5.1.2 VSCode 图标

双击图 4.5.1.2 打开 VSCode，默认界面如图 4.5.1.3 所示：

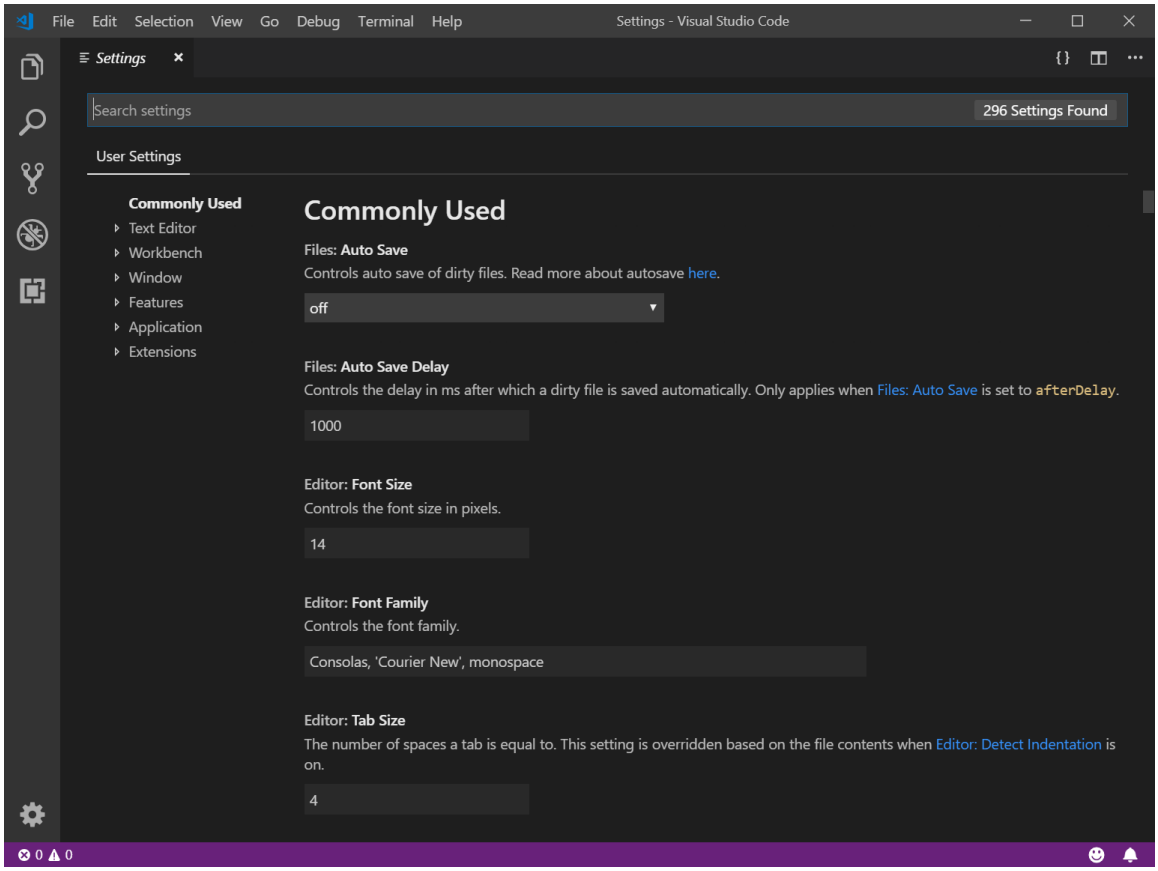


图 4.5.1.3 VSCode 默认界面

2、Linux 版本安装

我们有时候也需要在 Ubuntu 下阅读代码，所以还需要在 Ubuntu 下安装 VSCode。Linux 下的 VSCode 安装包我们也放到了开发板光盘中， 将开发板光盘中的.deb 软件包拷贝到 Ubuntu 系统中，然后使用如下命令安装：

```
sudo dpkg -i code_1.50.1-1602600906_amd64.deb
```

等待安装完成，如图 4.5.1.4 所示：

```
zuozhongkai@ubuntu:~/linux/tool$ sudo dpkg -i code_1.50.1-1602600906_amd64.deb
[sudo] zuozhongkai 的密码：
正在选中未选择的软件包 code。
(正在读取数据库 ... 系统当前共安装有 177197 个文件和目录。)
正准备解包 code_1.50.1-1602600906_amd64.deb  ...
正在解包 code (1.50.1-1602600906) ...
正在设置 code (1.50.1-1602600906) ...
gpg: WARNING: unsafe ownership on homedir '/home/zuozhongkai/.gnupg'
正在处理用于 gnome-menus (3.13.3-11ubuntu1.1) 的触发器 ...
正在处理用于 desktop-file-utils (0.23-1ubuntu3.18.04.2) 的触发器 ...
正在处理用于 mime-support (3.60ubuntu1) 的触发器 ...
正在处理用于 shared-mime-info (1.9-2) 的触发器 ...
```

图 4.5.1.4 VSCode 安装过程

安装完成以后搜索“Visual Studio Code”就可以找到，如图 4.5.1.5 所示：

图 4.5.1.5 Visual Studio Code

每次打开 VSCode 都要搜索，太麻烦了，我们可以将图标添加到 Ubuntu 桌面上，安装的所有软件图标都在目录/usr/share/applications 中，如图 4.5.1.6 所示：

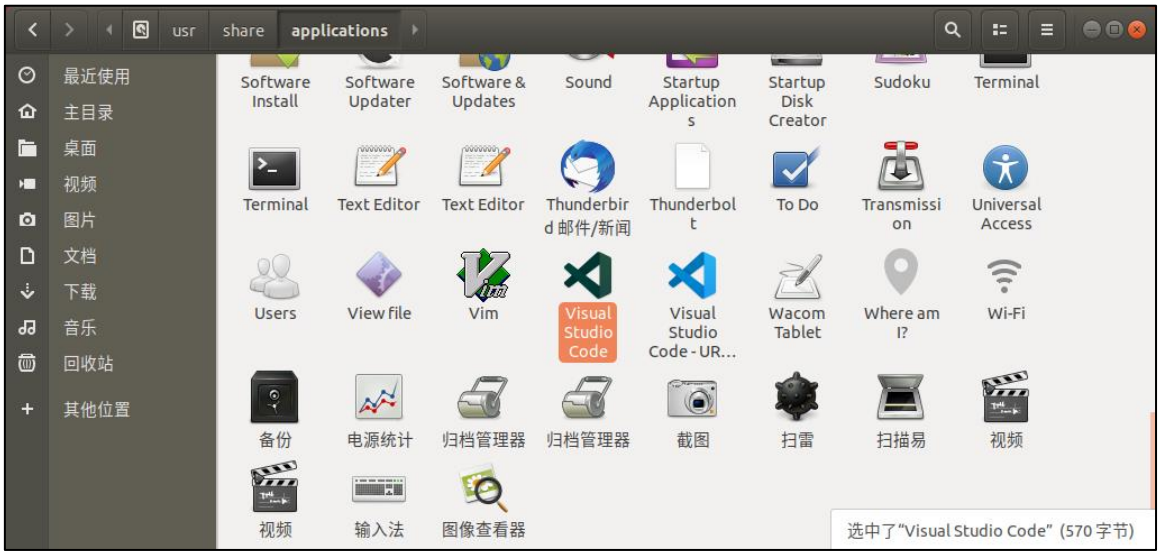


图 4.5.1.6 软件图标

在图 4.5.1.6 中找到 Visual Studio Code 的图标，然后点击鼠标右键，选择复制到->桌面，如图 4.5.1.7 所示：

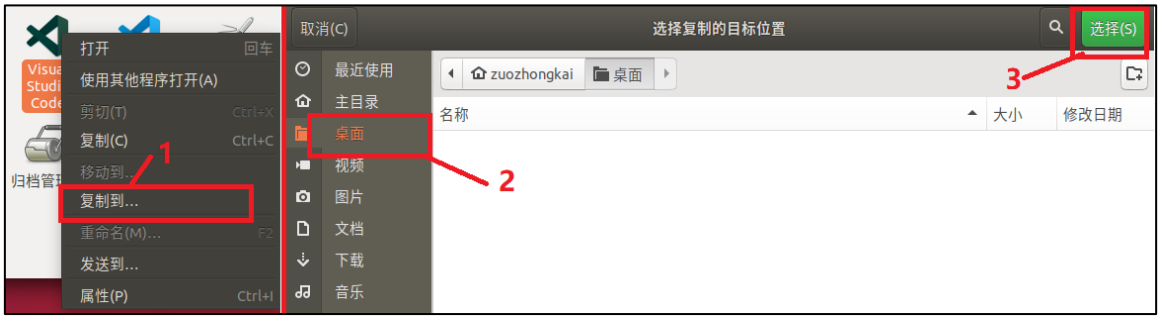


图 4.5.1.7 复制图标到桌面

按照图 4.5.1.7 所示方法将 VSCode 图标复制到桌面，第一次双击的时候会提示“未信任的应用程序启动器”，如图 4.5.1.8 所示：

图 4.5.1.8 “未信任的应用程序启动”对话框

点击图 4.5.1.8 中的“Trust and Launch”，选择信任即可。以后就可以直接双击图标打开 VSCode，Ubuntu 下的 VSCode 打开以后如图 4.5.1.9 所示：

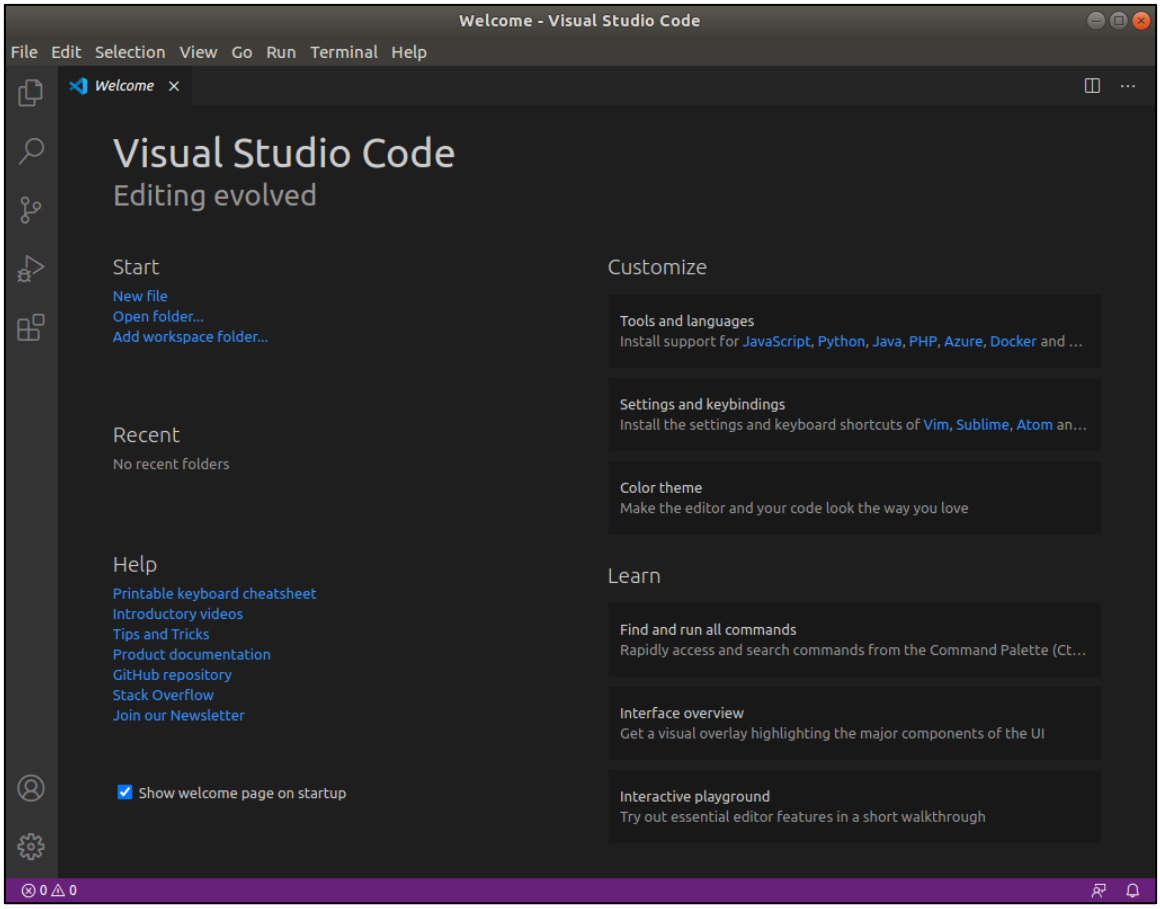


图 4.5.1.9 Linux 下的 VSCode

可以看出 Linux 下的 VSCode 和 Windows 下的基本是一样的，所以使用方法也是一样的。

4.5.2 Visual Studio Code 插件的安装

VSCode 支持多种语言，比如 C/C++、Python、C#等等，本教程我们主要用来编写 C/C++程序的，所以需要安装 C/C++的扩展包，扩展包安装很简单，如图 4.5.2.1 所示：

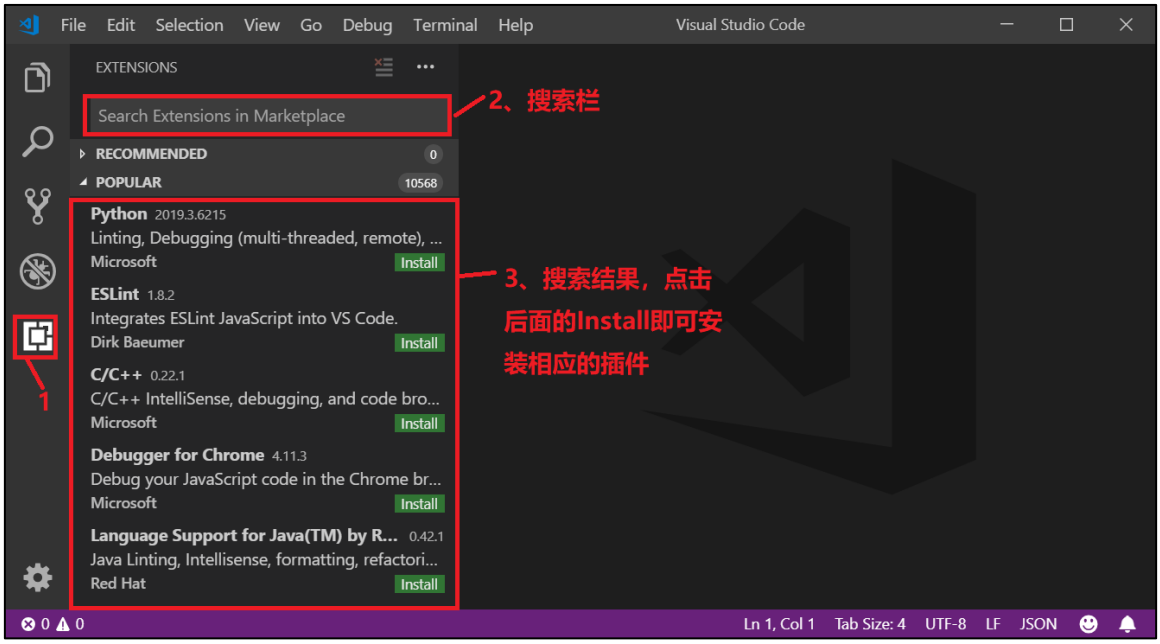


图 4.5.2.1 VSCode 插件安装

我们需要按照的插件有下面几个：

1)、C/C++，这个肯定是必须的。

2)、C/C++ Snippets，即 C/C++重用代码块。

3)、C/C++ Advanced Lint,即 C/C++静态检测 。

4)、Code Runner，即代码运行。

5)、Include AutoComplete，即自动头文件包含。

6)、Rainbow Brackets，彩虹花括号，有助于阅读代码。

7)、One Dark Pro，VSCode 的主题。

8)、GBKtoUTF8，将 GBK 转换为 UTF8。

9)、ARM，即支持 ARM 汇编语法高亮显示。

10)、Chinese(Simplified)，即中文环境。

11)、vscode-icons，VSCode 图标插件，主要是资源管理器下各个文件夹的图标。

12)、compareit，比较插件，可以用于比较两个文件的差异。

13)、DeviceTree，设备树语法插件。

14)、TabNine，一款 AI 自动补全插件，强烈推荐，谁用谁知道！

安装完成以后重新打开 VSCode，如果要查看已经安装好的插件，可以按照图 4.5.2.2 所示方法查看：

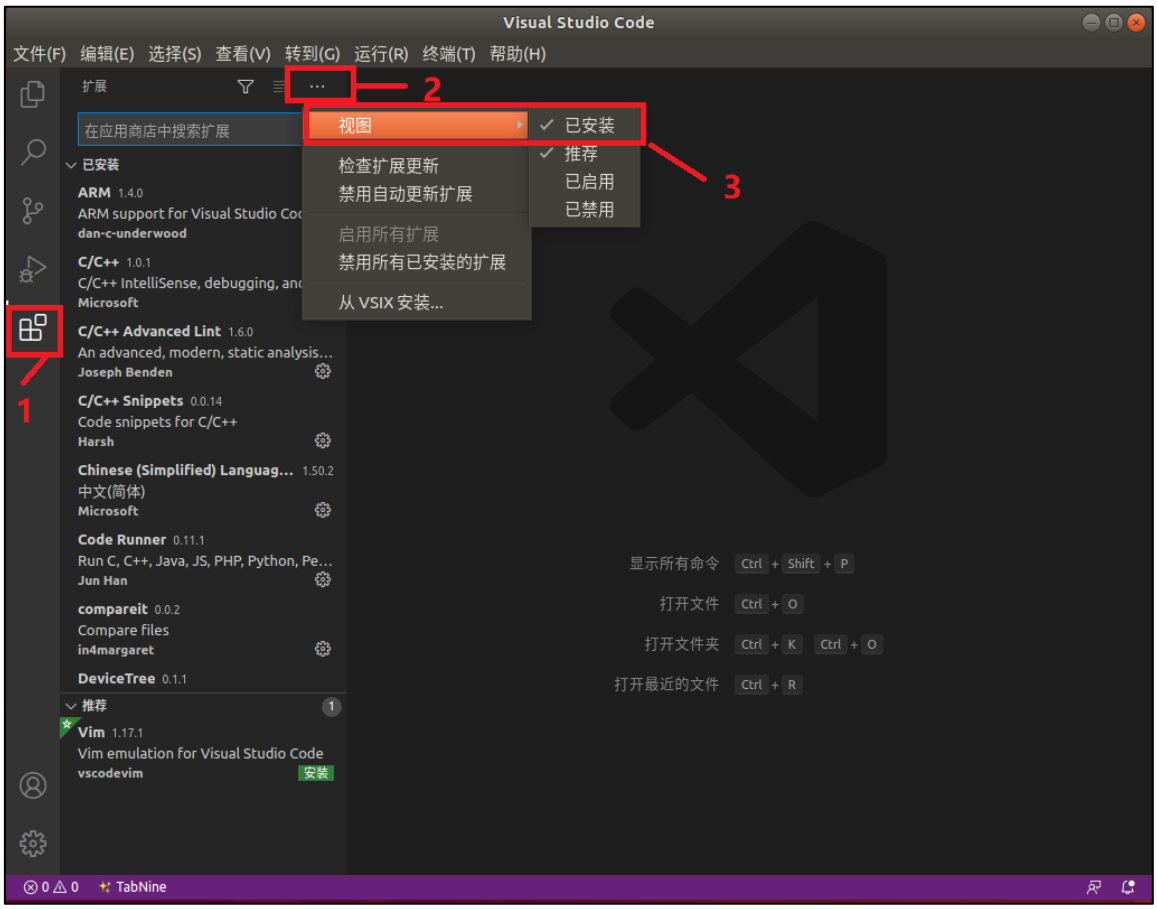


图 4.5.2.2 显示已安装的插件

由于我们已经安装了中文插件，因此重新打开 VSCode 以后就会是中文界面了，如图 4.5.2.3 所示：

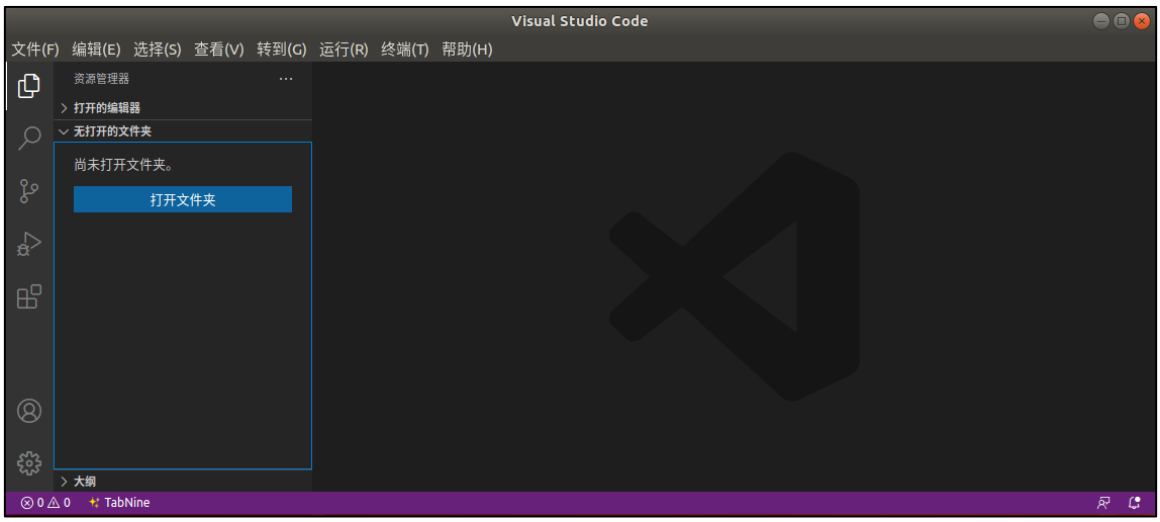


图 4.5.2.3 VSCode 中文环境

4.5.3 Visual Studio Code 新建工程

新建一个文件夹用于存放工程，比如我新建了文件夹目录为 E:\VScode_Program\1_test，路径尽量不要有中文和空格打开 VSCode。然后在 VSCode 上点击文件->打开文件夹...，选刚刚创

建的“1_test”文件夹，打开以后如图 4.5.3.1 所示：

图 4.5.3.1 打开的文件夹

从图 4.5.3.1 可以看出此时的文件夹“1_TEST”是空的，点击文件->将工作区另存为...，打开工作区命名对话框，输入要保存的工作区路径和工作区名字，如图 4.5.3.2 所示：

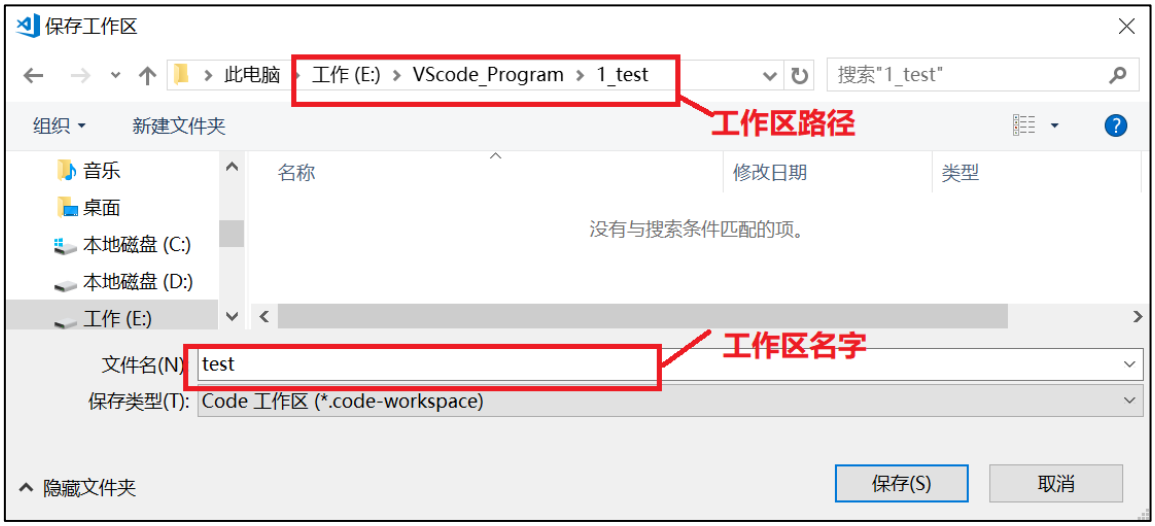


图 4.5.3.2 工作区保存设置

工作区保存成功以后，点击图 4.5.3.1 中的“新建文件”按钮创建 main.c 和 main.h 这两个文件，创建成功以后 VSCode 如图 4.5.3.3 所示：

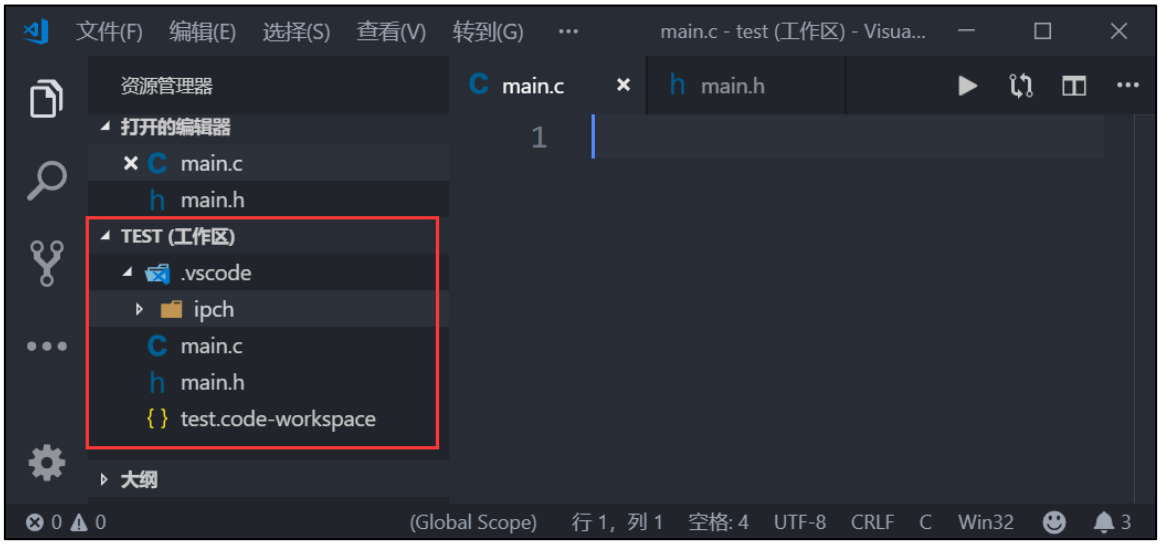


图 4.5.3.2 新建文件以后的 VSCode

从图 4.5.3.2 可以看出，此时“实验 1 TEST”中有.vscode 文件夹、main.c 和 main.h，这三个文件和文件夹同样会出现在“实验 1 test”文件夹中，如图 4.5.3.3 所示：

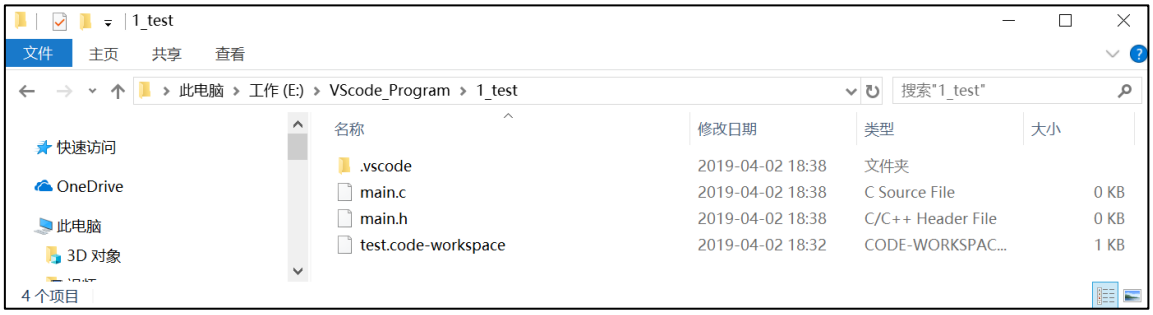


图 4.5.3.3 实验文件夹

在 main.h 中输入如下所示内容：

示例代码 4.5.3.1 main.h 文件代码

```
1 #include <stdio.h>
2
3 int add(int a, int b);
```

在 main.c 中输入如下所示内容：

示例代码 4.5.3.2 main.c 文件代码

```
1  #include <main.h>
2
3  int add(int a, int b)
4  {
5      return (a + b);
6  }
7
8  int main(void)
9  {
10     int value = 0;
11
```

```
12      value = add(5, 6);
13      printf("5 + 6 = %d", value);
14      return 0;
15  }
```

代码编辑完成以后 VSCode 界面如图 4.5.3.4 所示：

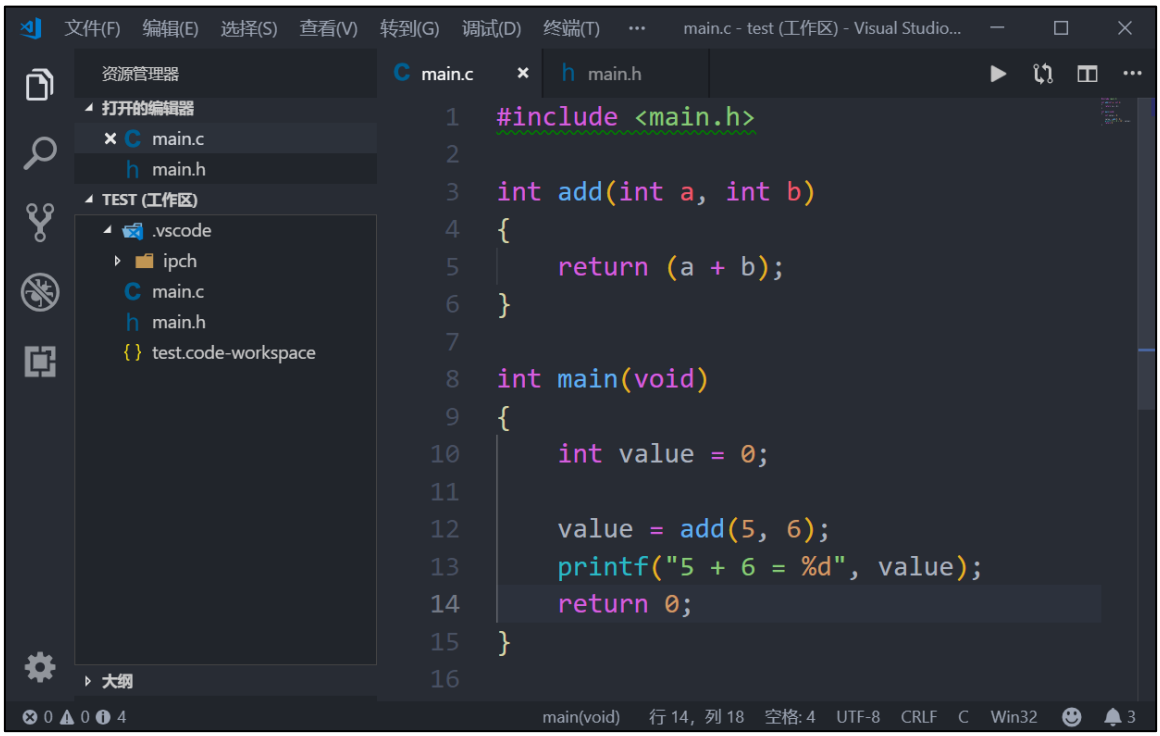


图 4.5.3.4 代码编辑完成以后的界面

从图 4.5.3.4 可以看出，VSCode 的编辑的代码高亮很漂亮，阅读起来很舒服。但是此时提示找不到“stdio.h”这个头文件，如图 4.5.3.5 所示错误提示：

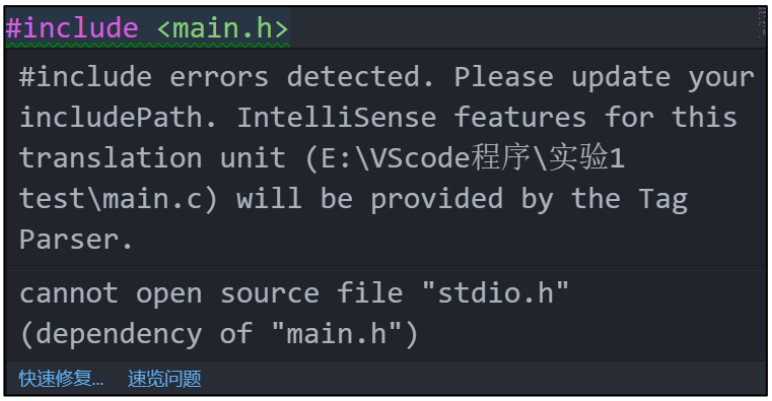


图 4.5.3.5 头文件找不到。

图 4.5.3.5 中提示找不到“main.h”，同样的在 main.h 文件中会提示找不到“stdio.h”。这是因为我们没有添加头文件路径。按下“Ctrl+Shift+P”打开搜索框，然后输入“Edit configurations”，选择“C/C++:Edit configurations…”，如图 4.5.3.6 所示：

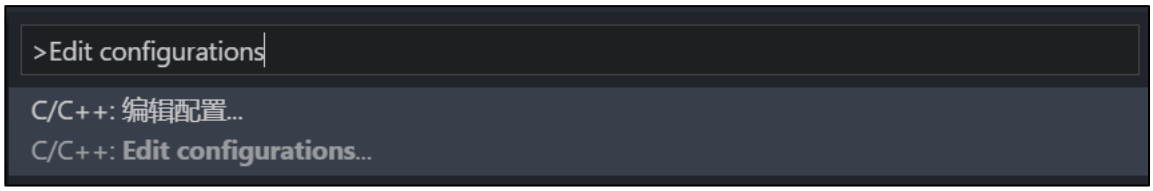


图 4.5.3.6 打开 C/C++编辑配置文件

C/C++的配置文件是个 json 文件，名为：c_cpp_properties.json，此文件默认内容如图 4.5.3.7 所示：

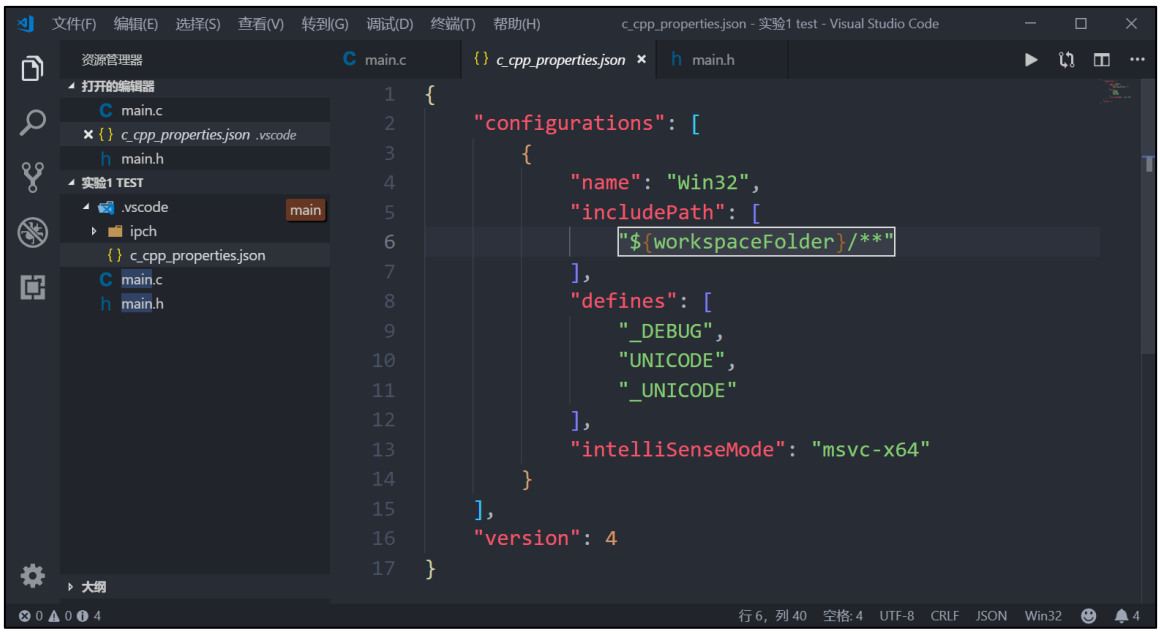


图 4.5.3.7 文件 c_cpp_properties.json 内容

c_cpp_properties.json 中的变量“includePath”用于指定工程中的头文件路径，但是“stdio.h”是 C 语言库文件，而 VSCode 只是个编辑器，没有编译器，所以肯定是没有 stdio.h 的，除非我们自行安装一个编译器，比如 CygWin，然后在 includePath 中添加编译器的头文件。这里我们就不添加了，因为我们不会使用 VSCode 来编译程序，这里主要知道如何指定头文件路径就可以了，后面有实际需要的时候再来讲。

我们在 VScode 上打开一个新文件的话会覆盖掉以前的文件，这是因为 VSCode 默认开启了预览模式，预览模式下单击左侧的文件就会覆盖掉当前的打开的文件。如果不想覆盖的话采用双击打开即可，或者设置 VSCode 关闭预览模式，设置如图 4.5.3.8 所示：

图 4.5.3.8 取消预览

我们在编写代码的时候有时候会在右下角有如图 4.5.3.9 所示的警告提示：

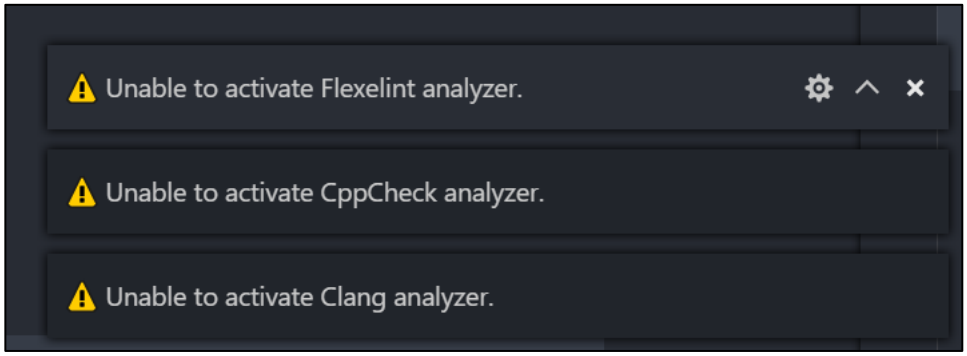


图 4.5.3.9 错误提示

这是因为插件 C/C++ Lint 打开了几个功能，我们将其关闭就可以了，顺便也可以学习一下 VSCode 插件配置方法，如图 4.5.3.10 所示：

图 4.5.3.10 C/C++ Lint 配置界面

在 C/C++ Lint 配置界面上找到 CLang:Enable、Cppcheck:Enable、Flexlint:Enable 这个三个，然后取消掉勾选即可，如图 4.5.3.11 所示：

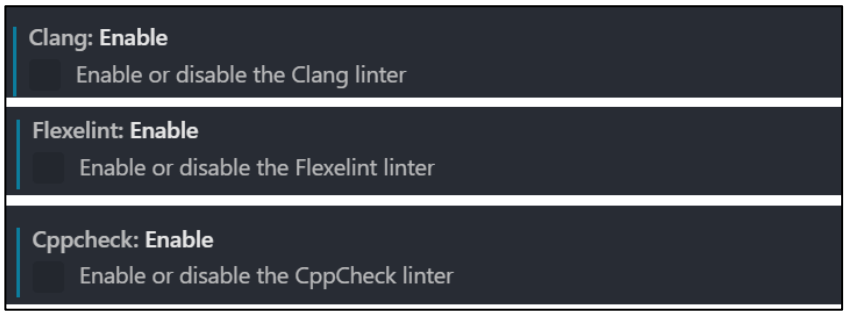


图 4.5.3.11 C/C++ Lint 配置

按照图 4.5.3.11 所示取消这三个有关 C/C++ Lint 的配置以后就不会有图 4.5.3.9 所示的错误提示了。但是关闭 Cppcheck:Enable 以后 VSCode 就不能实时检查错误了，大家根据实际情况选择即可。

4.5 CH340 串口驱动安装

我们一般在 Windwos 下通过串口来调试程序，或者使用串口作为终端，STM32MP157 开发板使用 CH340 这个芯片实现了 USB 转串口功能，CH340 是一枚江苏沁恒生产的国产芯片，稳定性还是很不错的，这里我们要多多支持国产嘛。

先通过 USB 线将开发板的串口和电脑连接起来起来，连接方式如图 4.5.1：

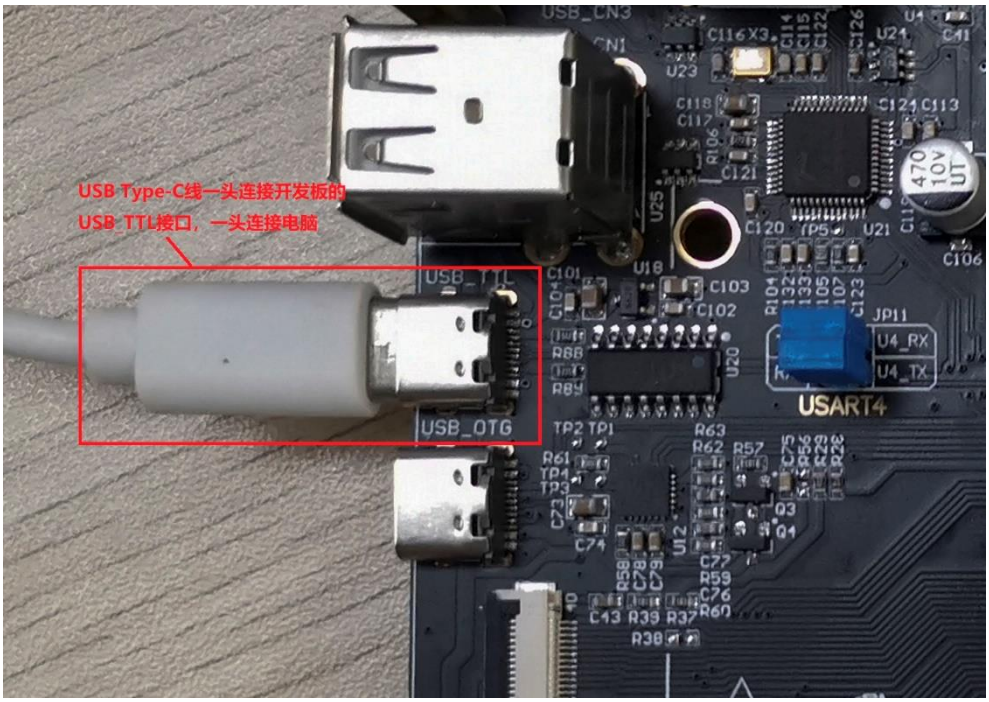


图 4.5.1 开发板串口连接方式

CH340 是需要安装驱动的，驱动我们已经放到了开发板光盘中，路径：开发板光盘->3、软件->CH340 驱动(USB 串口驱动)_XP_WIN7 共用->SETUP.EXE，，双击 SETUP.EXE，打开如图 4.5.2 所示安装界面：

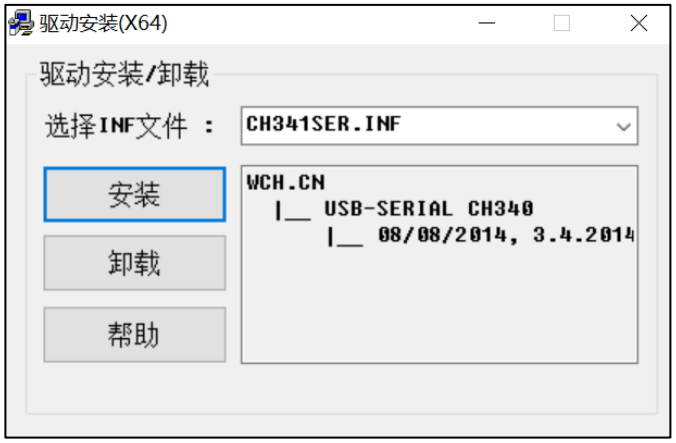


图 4.5.2 CH340 驱动安装

点击图 4.5.2 中的“安装”按钮开始安装驱动，等待驱动安装完成，驱动安装完成以后会有如图 4.5.3 所示提示：

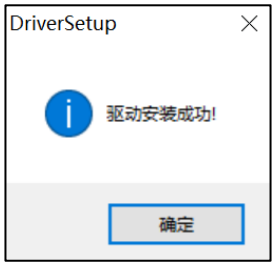


图 4.5.3 驱动安装成功

点击图 4.5.3 中的“确定”按钮退出安装，重新插拔一下串口线。打开设备管理器，打开方式是在 Windows 上的“此电脑”图标上点击鼠标右键，选择“管理”，如图 4.5.4

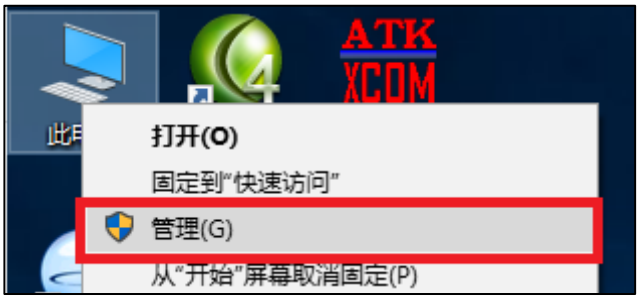


图 4.5.4 打开管理窗口

打开以后的计算机管理器如图 4.5.5 所示：

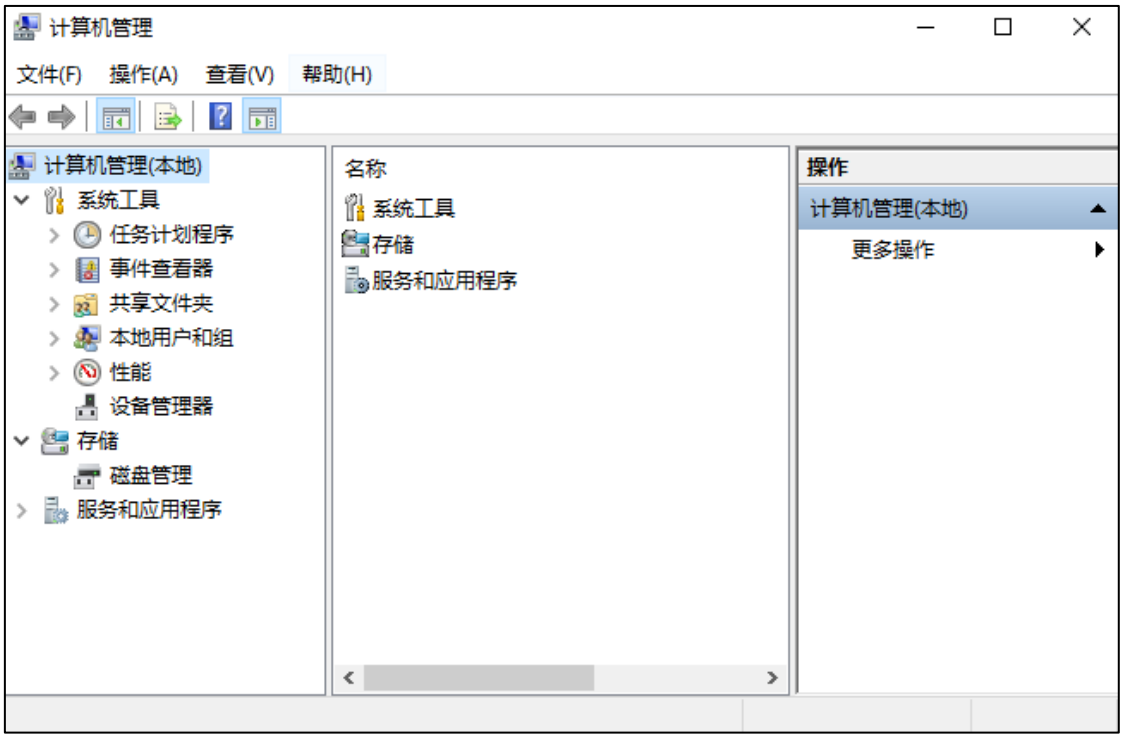


图 4.5.5 计算机管理器

在图 4.5.5 中，点击左侧“计算机管理(本地)”中的“设备管理器”，在右侧选中“端口(COM 和 LPT)”，如图 4.5.6 所示：

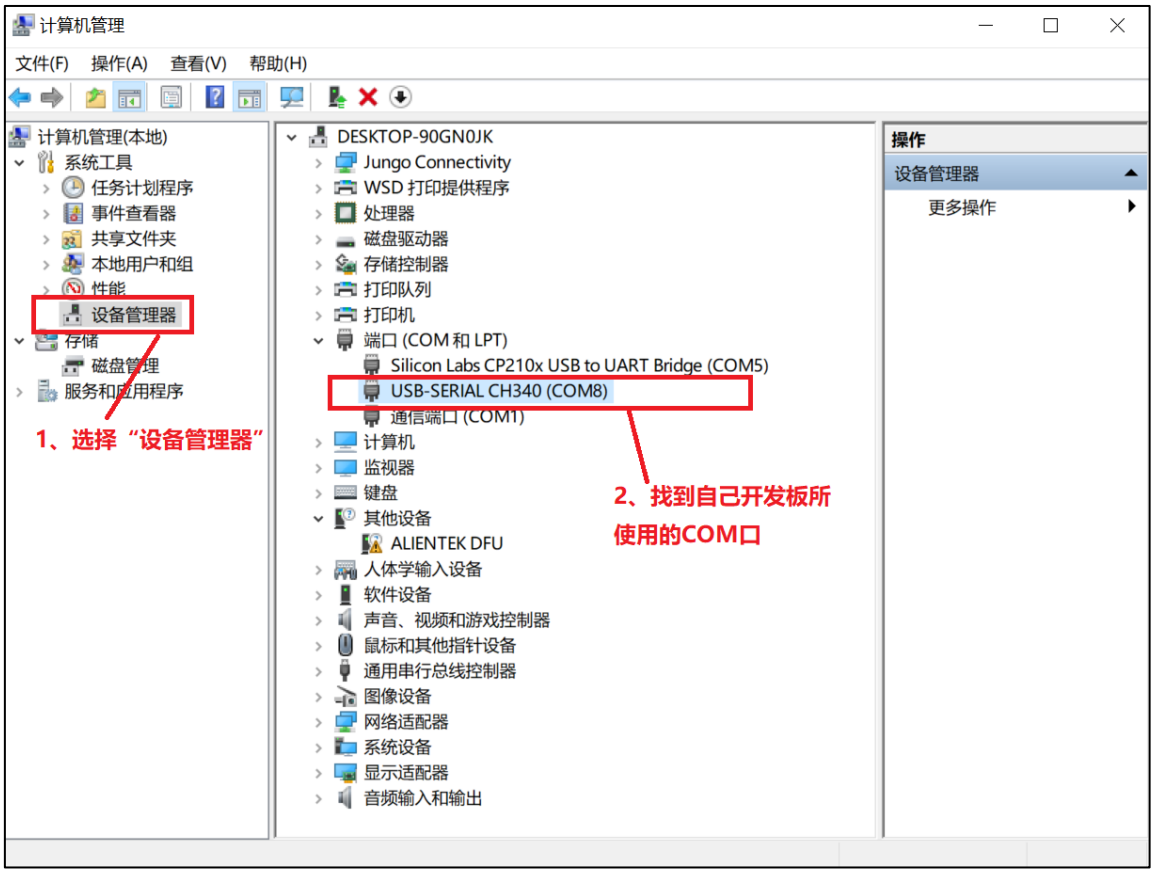


图 4.5.6 设备管理器

如果在图 4.5.6 中找到了有“USB-SERIAL CH340”字样的端口设备就说明 CH340 驱动成功了，一定要用 USB 线将开发板的串口和电脑连接起来！！！！

4.6 MobaXterm 软件安装和使用

4.6.1 MobaXterm 软件安装

MobaXterm 是一款终端软件，功能强大而且免费(也有收费版)！我试用了一下，用起来非常舒服！在这里推荐大家使用此软件作为终端调试软件，MobaXterm 软件在其官网下载即可，地址为 https://mobaxterm.mobatek.net/，如图 4.6.1.1 所示：

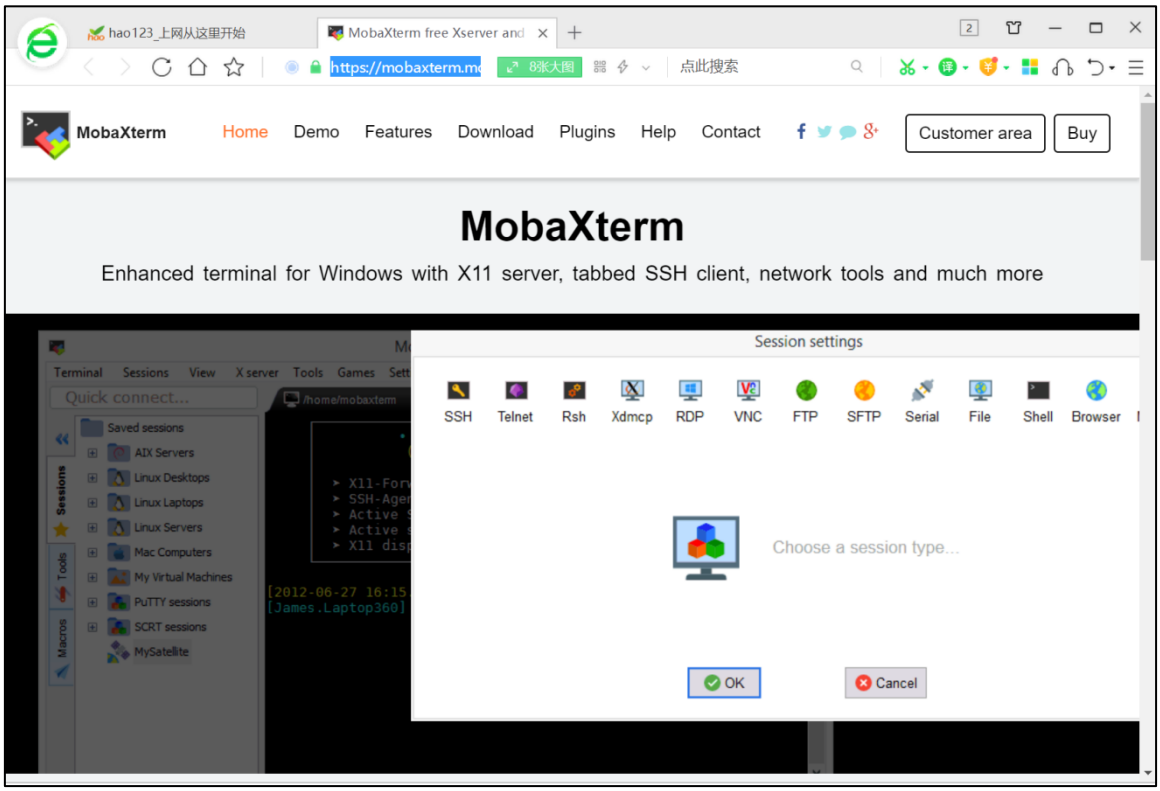


图 4.6.1.1 MobaXterm 官网

点击图 4.6.1.1 中的“Download”按钮即可打开下载界面，如图 4.6.1.2 所示：

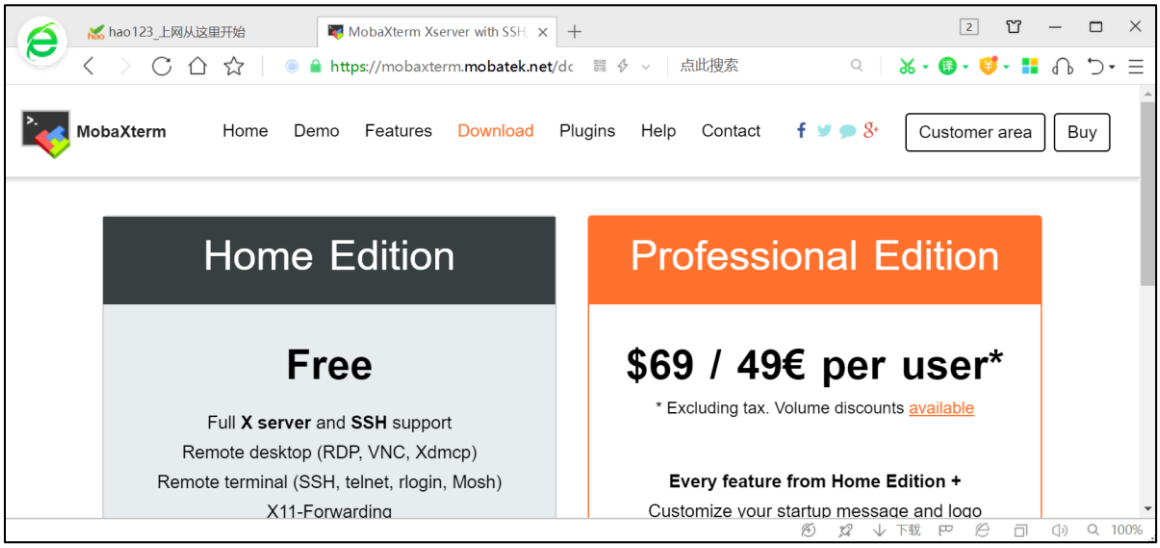


图 4.6.1.2 下载界面

从图 4.6.1.2 可以看出，一共有两个版本，左侧为免费的 Home Edition 版本，右侧为付费的 Professional Edition 版本。毫无疑问，我们肯定选择免费的 Home Edition 版，点击下方的“Download now”，打开下载界面，如图 4.6.1.3 所示：

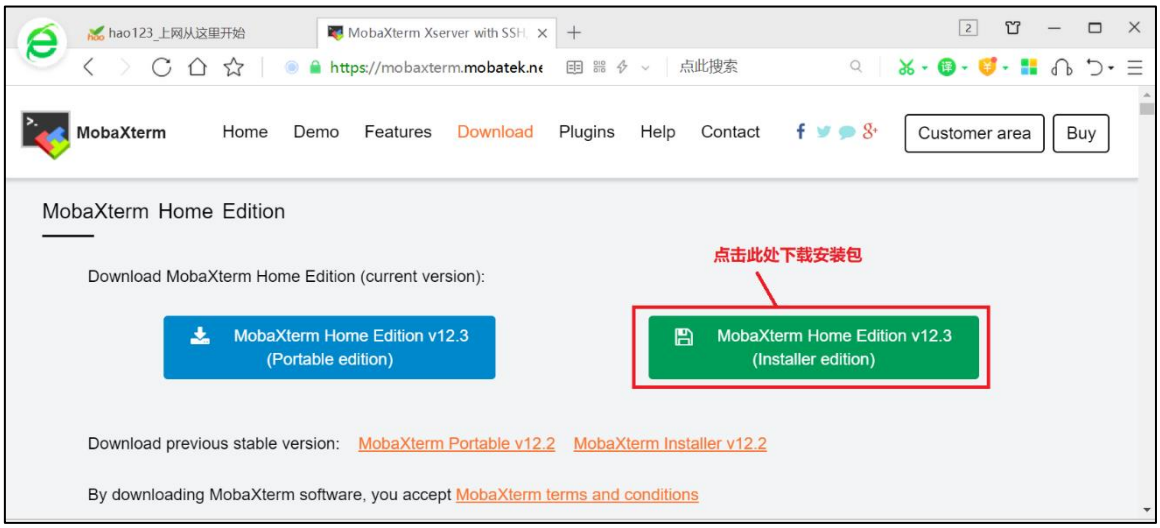


图 4.6.1.3 下载界面

可以看出，当前的版本号为 v12.3，点击右侧按钮下载安装包。安装包已经放到了开发板光盘中，路径为：开发板光盘->3、软件->MobaXterm_Installer_v12.3.zip。打开此压缩包，然后双击 MobaXterm_installer_12.3.msi 进行安装，安装方法很简单，一步一步进行即可。安装完成以后就会在桌面出现 MobaXterm 图标，如图 4.6.1.4 所示，如果桌面没有的话就自行添加。

图 4.6.1.4 MobaXterm 软件图标

4.6.2 MobaXterm 软件使用

双击 MobXterm 图标，打开此软件，软件界面如图 4.6.2.1 所示：

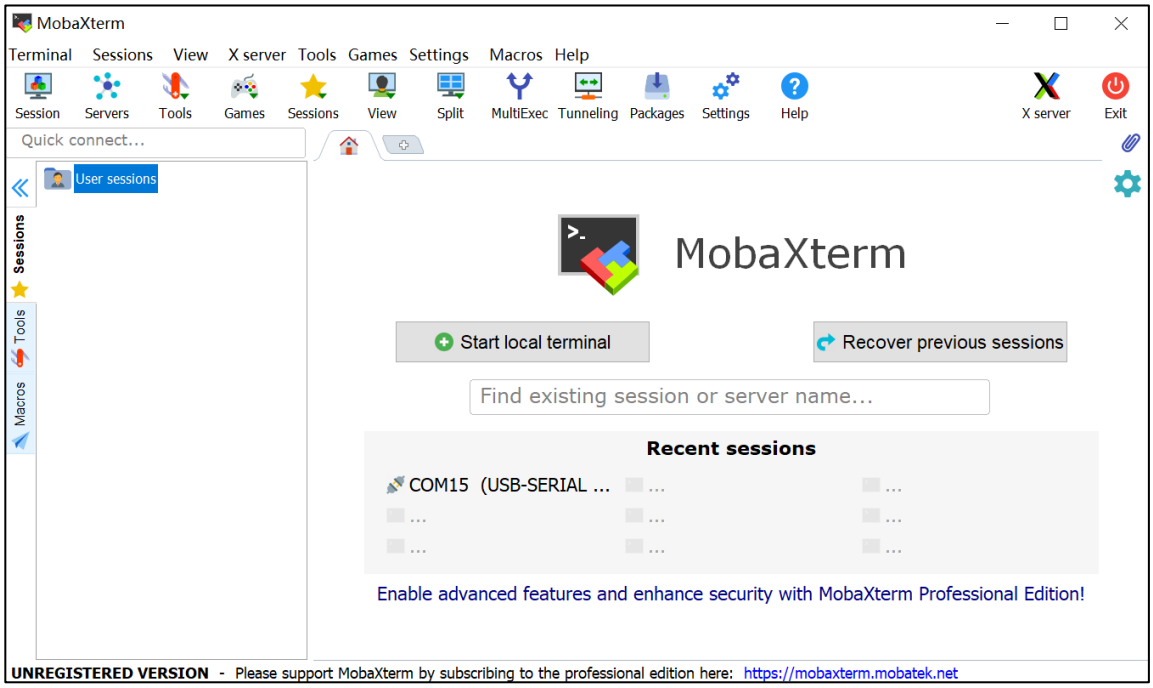


图 4.6.2.1 MobaXterm 软件主界面

点击菜单栏中的“Sessions->New session”按钮，打开新建会话窗口，如图 4.6.2.2 所示：

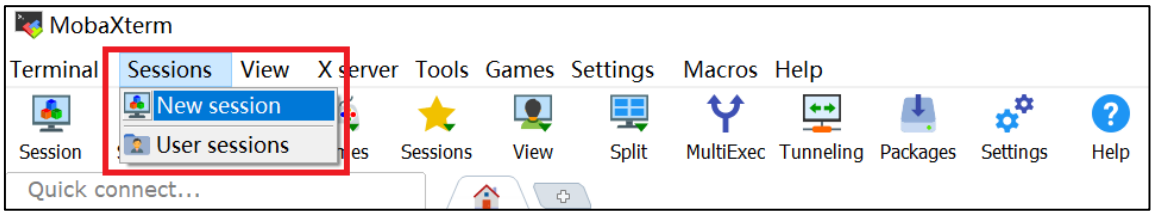


图 4.6.2.2 新建会话

打开以后的新建会话窗口如图 4.6.2.3 所示：

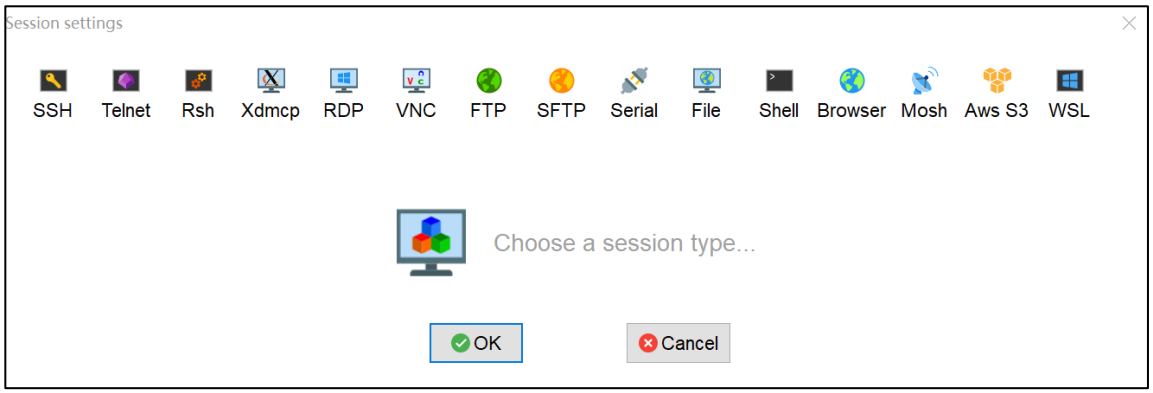


图 4.6.2.3 新建会话窗口

从图 4.6.2.3 可以看出，MobaXterm 软件支持很多种协议，比如 SSH、Telnet、Rsh、Xdmcp、RDP、VNC、FTP、SFTP、Serial 等等，我们现在就讲解一下如何建立 Serial 连接，也就是串口连接，因为我们使用 MobaXterm 的主要目的就是作为串口终端使用。点击图 4.6.2.3 中的“Serial”按钮，打开串口设置界面，如图 4.6.2.4 所示：

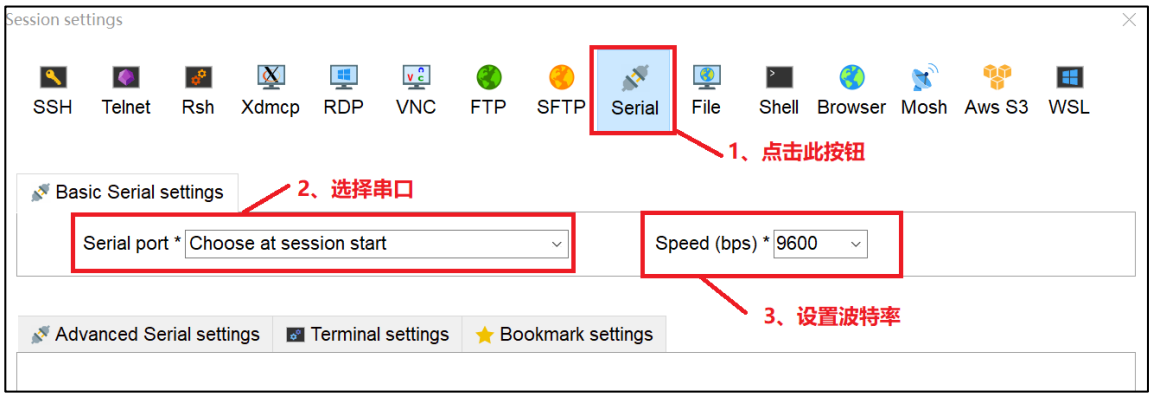


图 4.6.2.4 设置串口

打开串口设置窗口以后先选择要设置的串口号，因此要先用串口线将开发板连接到电脑上上，然后设置波特率为 115200(根据自己实际需要设置)，完成以后如图 4.6.2.5 所示：

图 4.6.2.5 设置串口及其波特率

MobaXterm 软件可以自动识别串口，因此我们直接下拉选择即可，波特率也是同样的设置方式，下拉选择即可。完了以后还要设置串口的其他功能，下方一共有三个设置选项卡，如图 4.6.2.6 所示：

图 4.6.2.6 串口其他设置选项

点击 Advanced Serial settings 选项卡，设置串口的其他功能，比如串口引擎、数据位、停止位、奇偶校验和硬件流控等，按照图 4.6.2.7 所示设置即可：

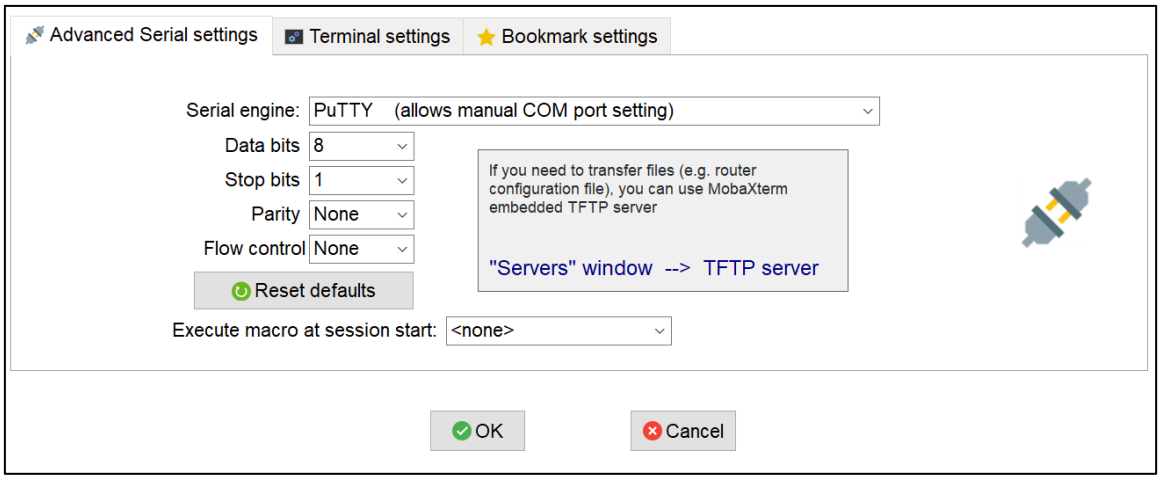


图 4.6.2.7 串口设置

如果要设置终端相关的功能的话点击“Terminal settings”即可，比如终端字体以及字体大小等。设置完成以后点击下方的“OK”按钮即可。串口设置完成以后就会打开对应的终端窗口，如图 4.6.2.8 所示：

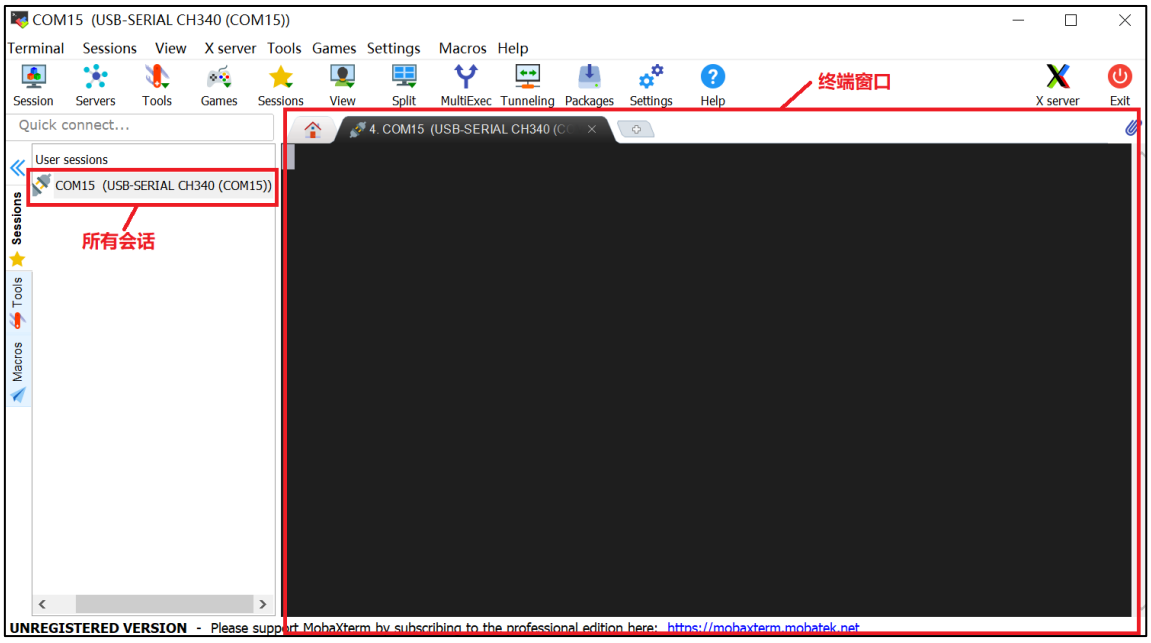


图 4.6.2.8 成功建立的串口终端

如果开发板里面烧写了系统的话就会在终端中打印出系统启动的 log 信息，如图 4.6.2.9 所示：

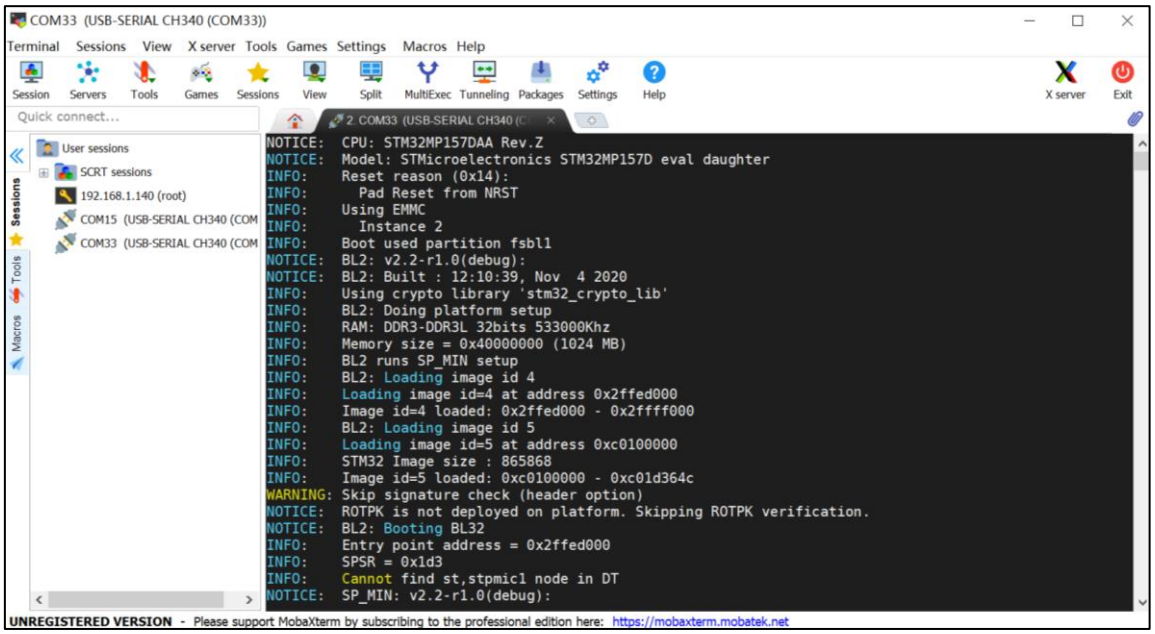


图 4.6.2.9 MobaXterm 作为串口终端

可以看出，MobaXterm 作为串口终端还是非常漂亮的，结合了 SecureCRT 的功能强大与 Putty 的免费。推荐大家使用 MobaXterm 作为串口终端使用，当然了，MobaXterm 也可以作为其他终端软件，这里大家就自行摸索吧。

4.7 Windows 下 ST 官方软件安装

4.7.1 ST 官方软件简介

在开发 STM32MP157 的时候我们还需要用到一些 ST 官方提供的软件，一共有三种：STM32CubeMX、STM32CubeIDE、STM32CubeProgrammer，这三个软件的功能如表 4.7.1.1 所示：

软件	功能
STM32CubeMX	STM32 图形化配置软件
STM32CubeIDE	ST 官方集成 IDE，用于 M4 内核
STM32CubeProgrammer	STM32 烧写软件

表 4.7.1.1 ST 官方软件

1、STM32CubeMX

STM32CubeMX 是近年来开发 STM32 比较流行的一个配置代码工具，目前已经更新了很多个版本，新的版本功能也越来越强大。

STM32CubeMX是图形化配置工具，可以直观的选择MCU/MPU型号，可以动态地配置引脚、配置时钟树、配置中间件、配置内存，可以生成初始化代码、MPU设备树源码，可以进行DDR测试等。STM32CubeMX的好处是具有直观的图形配置界面，而且它把很多东西封装的比较好，例如硬件抽象层、中间层、外围器件和示例代码以及Arm Cortex-A内核部分Linux设备树等等，它可以独立使用或作为 Eclipse 插件使用，例如在STM32CubeIDE上就集成了STM32CubeMX。

STM32CubeMX 开源且跨平台，支持在 Windows、Linux 和 macOS 操作系统上运行（64 位），STM32CubeMX 与 STM32 之间的关系如图 4.7.1.1 所示：

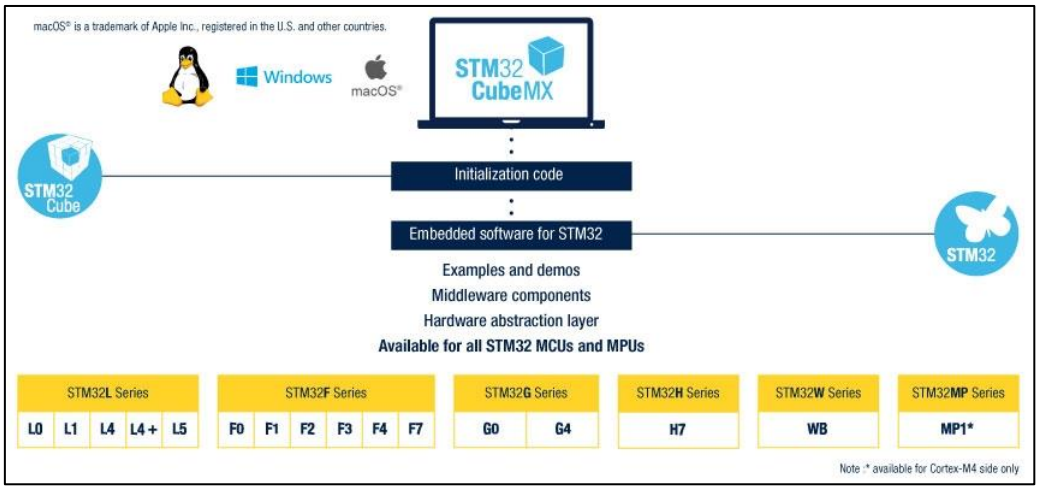


图 4.7.1.1 STM32CubeMX 与 STM32

STM32CubeMX 可以直接在 ST 官网下载到，STM32CubeMX 已经放到了开发板光盘中，路径为：开发板光盘→5、开发工具→2、ST 官方开发工具→en.stm32cubemx_v6-0-1.zip。

2、STM32CubeIDE

STM32CubeIDE 是 ST 于 2019 年新推出的一款多功能的集成开发工具，它集成了 TrueSTUDIO 和 STM32CubeMX 插件，并基于 GDB 进行调试，它允许集成数百个现有插件，这些插件完成 Eclipse 的功能。

TrueSTUDIO 插件是一款建立在 Eclipse CDT、GCC 和 GDB 的 C/C++集成开发工具，其具有项目创建和管理、代码编辑、代码编译以及代码调试等功能。STM32CubeMX 插件具有图形化配置功能，可以直观地选择 MCU/MPU 型号、动态配置引脚和设置时钟树、动态设置外围设备和中间器件的模式，可以自动处理引脚冲突和生成初始化代码。TrueSTUDIO 和 STM32CubeMX 的强强联手，使 STM32CubeIDE 得以和 MDK 进行媲美。STM32CubeIDE 完全开源，并且跨平台，目前支持 Windows、Linux 和 macOS 多个操作系统（64 位版本）。

STM32CubeIDE 可以直接在 ST 官网下载到，STM32CubeIDE 已经放到了开发板光盘中，路径为：开发板光盘→5、开发工具→2、ST 官方开发工具→en.st-stm32cubeide_1.4.0_7511_20200720_0928_x86_64_exe.zip。

3、STM32CubeProgrammer

STM32CubeProgrammer 看名字就知道和编程有关，它属于多合一软件工具，它将 ST Visual Programmer、DFUse Device Firmware Update、Flash Loader 和 ST-Link 等软件功能整合到了一起，为用户提供 STM32 微控制器代码烧写和固件安全安装、更新功能，帮助用户更快地开发特定外部存储器的加载程序。

STM32CubeProgrammer 支持对外部存储器进行编程、擦除和验证，用户烧写 STM32 微控制器既可使用片上 SWD (单线调试)或 JTAG 调试端口，也可以用程序引导装入端口(例如 UART 和 USB)。它支持 Motorola S19、Intel HEX、ELF 和 二进制格式（BIN 文件），支持 ST-LINK 固件更新，支持图形化界面操作也支持命令行操作，在命令行界面，可通过脚本实现自动化编程。目前 STM32CubeProgrammer 支持 Windows、Linux 和 macOS 多个操作系统（64 位版本）。

STM32CubeProgrammer 可以直接在 ST 官网下载到，这里也已经放到了开发板光盘中，路

径为：开发板光盘→5、开发工具→2、ST 官方开发工具→en.stm32cubeprog_v2-5-0.zip。

4.7.2 Java 环境安装

在安装 STM32CbeMX 和 STM32CubeIDE 前我们要先安装 Java 的环境，Java 运行环境版本必须是 V1.7 及以上，否则会导致这两个无法使用。大家可以到 Java 官网 https://www.java.com/zh-CN/download/manual.jsp 查找下载最新的 64 位 Java 软件。

一定要用 64 位的 JAVA 运行环境，否则使用 STM32CubeProgrammer 烧写系统的时候会失败！

我们已经在开发板光盘里面放了 64 位的 JAVA，路径为：开发板光盘→3、软件→Java 安装包→jre-8u271-windows-x64.exe。

对于 Java 运行环境安装，我们这里就不做过多讲解，大家直接双击安装包，根据提示安装即可，安装完成之后提示界面如图 4.7.2.1：

图 4.7.2.1 java 运行环境安装成功

安装完 Java 运行环境之后，为了检测是否正常安装，我们可以打开 Windows 的 cmd 命令输入框，输入如下命令：

```
java -version //命令查询 Java 版本
```

如果安装成功的话就会打印出 Java 的版本号，如图 4.7.2.2 所示：

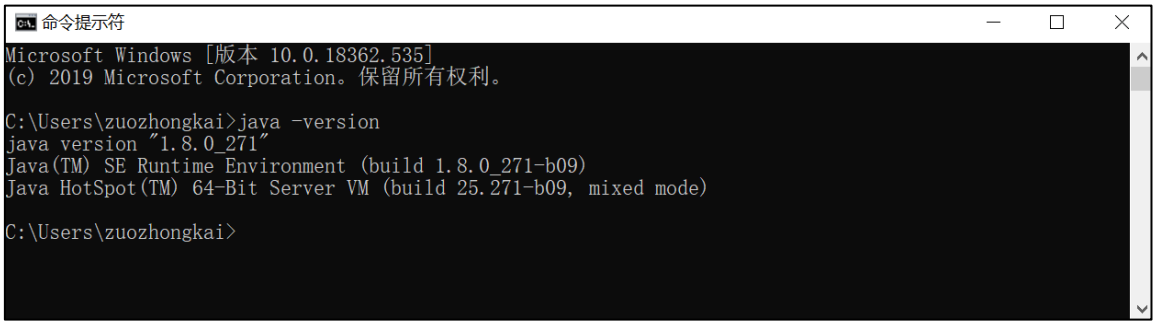


图 4.7.2.2 Java 版本号

从图 4.7.2.2 中可以看出，当前 JAVA 环境版本号为 1.8.0_271，而且是 64 位的。

4.7.3 STM32CubeMX 安装

STM32CubeMX 需要 JAVA 运行环境，确保已经按照 4.7.2 小节安装好了 JAVA 运行环境。

STM32CubeMX 安装方法很简单，解压开发板光盘中的 CubeMX 压缩包，然后双击解压得到的“SetupSTM32CubeMX-6.0.1.exe”进行安装。安装方法很简单，一路“Next”，然后根据提示接受许可学习、设置安装路径即可。安装完成以后就会在桌面生成 CubeMX 的图标，如图 4.7.3.1 所示：

图 7.4.3.1 CubeMX 图标

双击图 7.4.3.1 中的 CubeMX 图标，打开以后如图 7.4.3.2 所示：

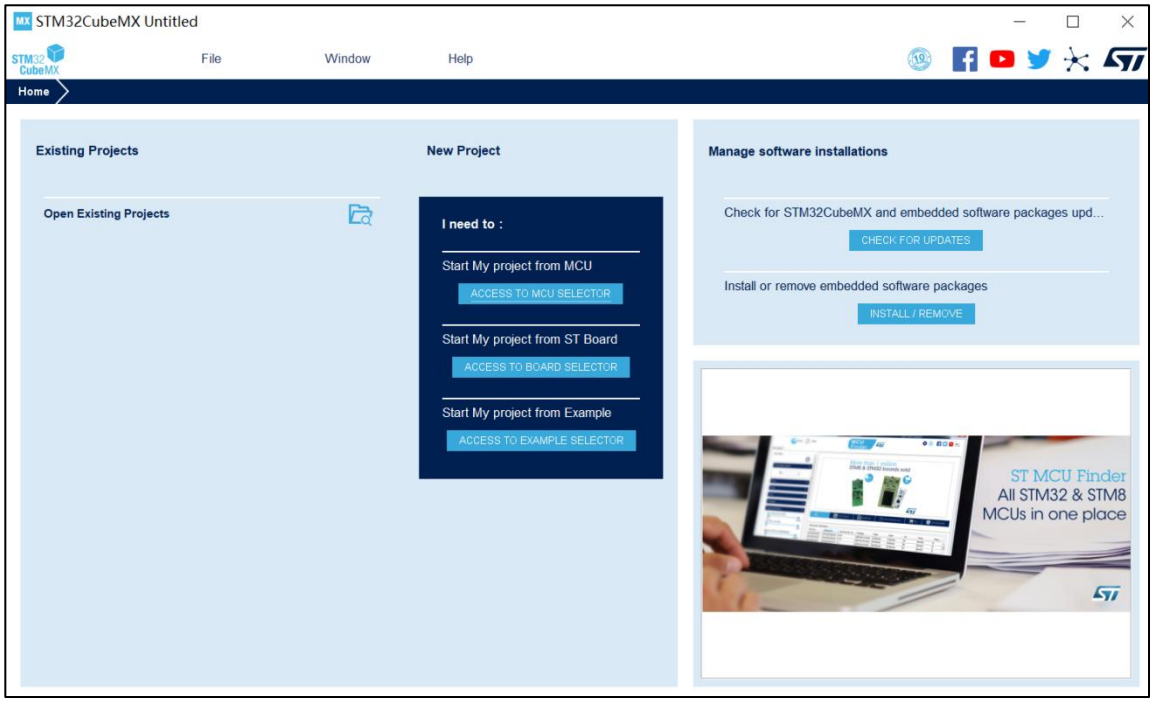


图 7.4.3.2 CubeMX 主界面

我们可以根据具体的 STM32 型号，或者 ST 官方的开发板来构建自己的 CubeMX 工程，正点原子的《STM32MP1 M4 裸机 CubeIDE 开发手册》对 CubeMX 的使用进行了详细的讲解。

4.7.4 STM32CubeIDE 安装

STM32CubeIDE 需要 JAVA 运行环境，确保已经按照 4.7.2 小节安装好了 JAVA 运行环境。

我们选择安装 1.4.0 版本的 STM32CbeIDE，相比于以前的版本其修复了一些 bug 并新增了部分功能，其集成的 STM32CubeMX 是 V6.0.0 版本。

将下载好的 STM32CubeIDE 安装包解压到非中文路径下，如果路径中有中文，安装的时候可能会有如图 4.7.4.1 所示错误。

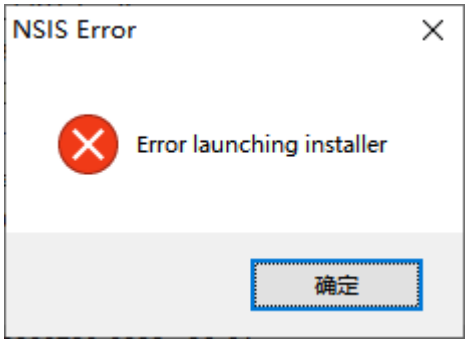


图 4.7.4.1 安装错误提示

双击解压出来的 st-stm32cubeide_1.4.0_7511_20200720_0928_x86_64.exe 可执行文件进行安装，程序进入安装向导界面如图 4.7.4.2 所示：

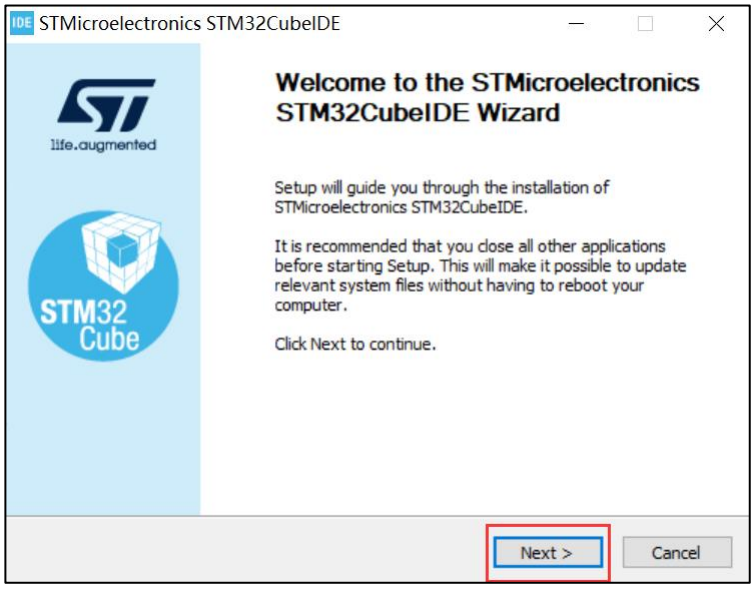


图 4.7.4.2 安装界面

点击图 4.7.4.2 中的“Next”按钮，弹出如图 4.7.4.3 所示的许可协议界面：

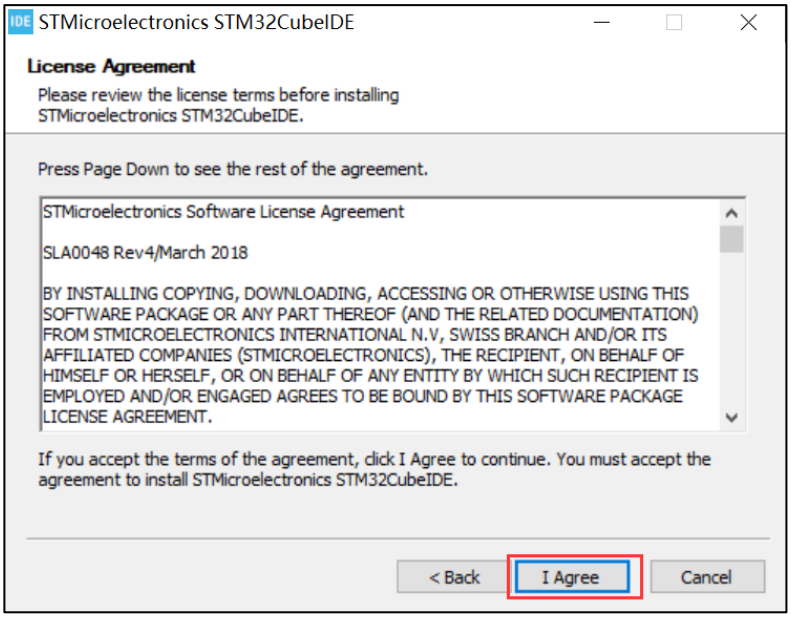


图 4.7.4.3 安装许可协议

点击图 4.7.4.3 中的“I Agree”，然后弹出安装的路径选择界面，默认是安装在 C 盘，可根据个人情况选择安装路径，此处本文选择安装在 D 盘，如图 4.7.4.4 所示：

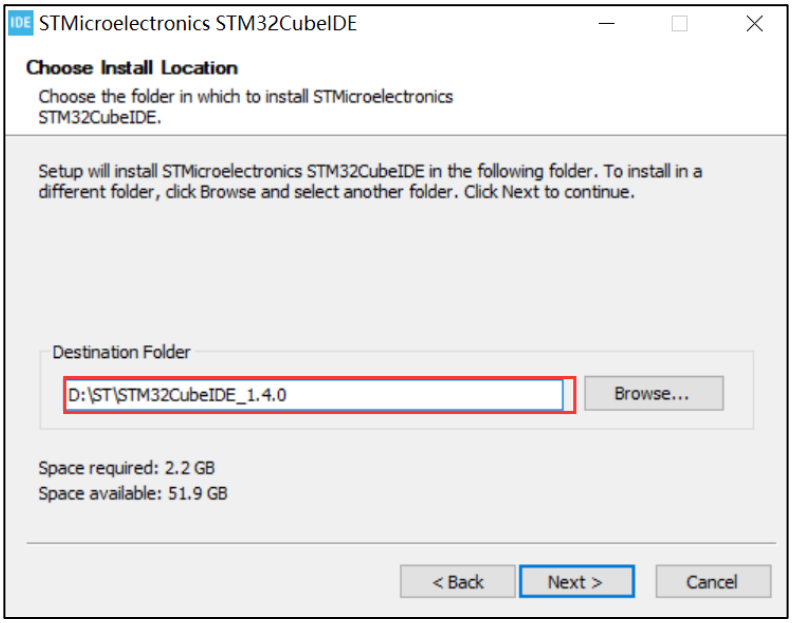


图 4.7.4.4 安装路径选择

注意，安装路径不能有中文，否则会导致 STM32CubeIDE 软件无法安装，即使成功安装以后，也会导致无法正常创建和保存工程，无法正常进行编译。安装路径选择以后点击图 4.7.2.5 中的“Next”按钮，继续下一步。接下来是选择组件对话框选择与 STM32CubeIDE 一起安装的 GDB 服务器组件，默认选择 J-Link 和 ST-Link 的组件，我们选择默认即可，如图 4.7.4.5 所示：

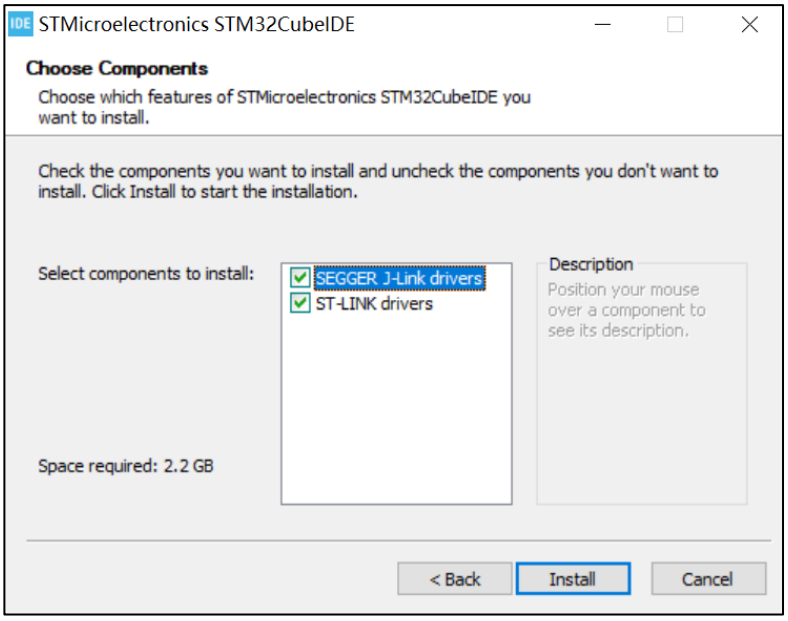


图 4.7.4.5 组件选择

点击图 4.7.2.6 中的“Install”，安装正式进行，待安装程序进度完成。如果弹出需要安装的驱动，选择安装即可。最后点击 Next→Finish 完成安装，如图 4.7.4.6 所示：

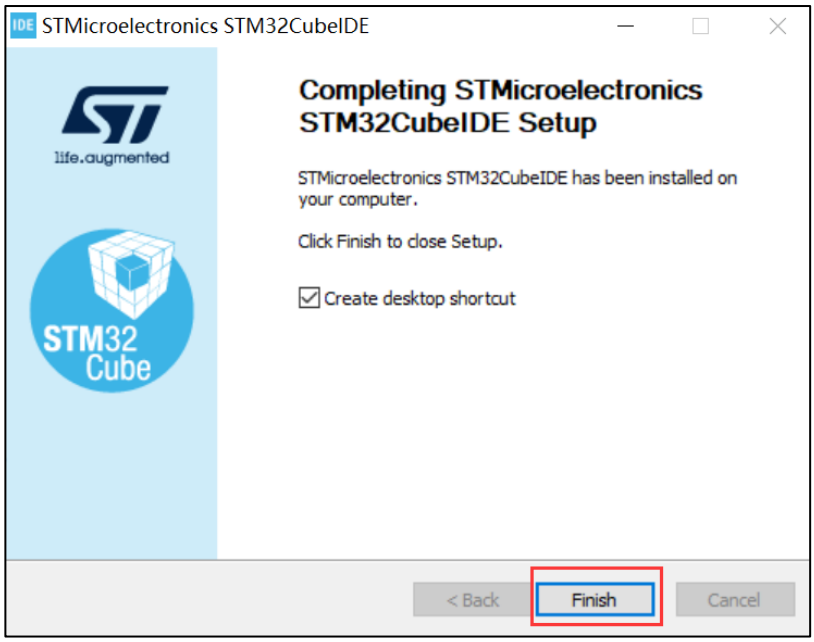


图 4.7.4.6 安装完成

安装完成后桌面有一个快捷启动图标，双击此快捷图标可以启动 STM32CubeIDE，如图 4.7.4.7 所示：

图 4.7.4.7 STM32CubeIDE 图标

STM32CubeIDE 是进行 STM32MP157 的 M4 开发的，不能开发 Linux 部分，因此本教程实际用不到。关于 STM32CubeIDE 的详细用法请参考正点原子的《STM32MP1 M4 裸机 CubeIDE 开发手册》。

4.7.5 STM32CubeProgrammer 安装

STM32CubeProgrammer 需要 JAVA 运行环境，确保已经按照 4.7.2 小节安装好了 JAVA 运行环境。

解压开发板光盘中的 STM32CubeProgrammer 安装压缩包，然后双击解压出来的“SetupSTM32CubeProgrammer-2.5.0.exe”，安装过程很简单，根据提示进行安装即可，安装完成以后的 STM32CubeProgrammer 图标如图 4.7.5.1 所示：

图 4.7.5.1 STM32CubeProgrammer 图标

双击图 4.7.5.1 打开 STM32CubeProgrammer，如图 4.7.5.2 所示：

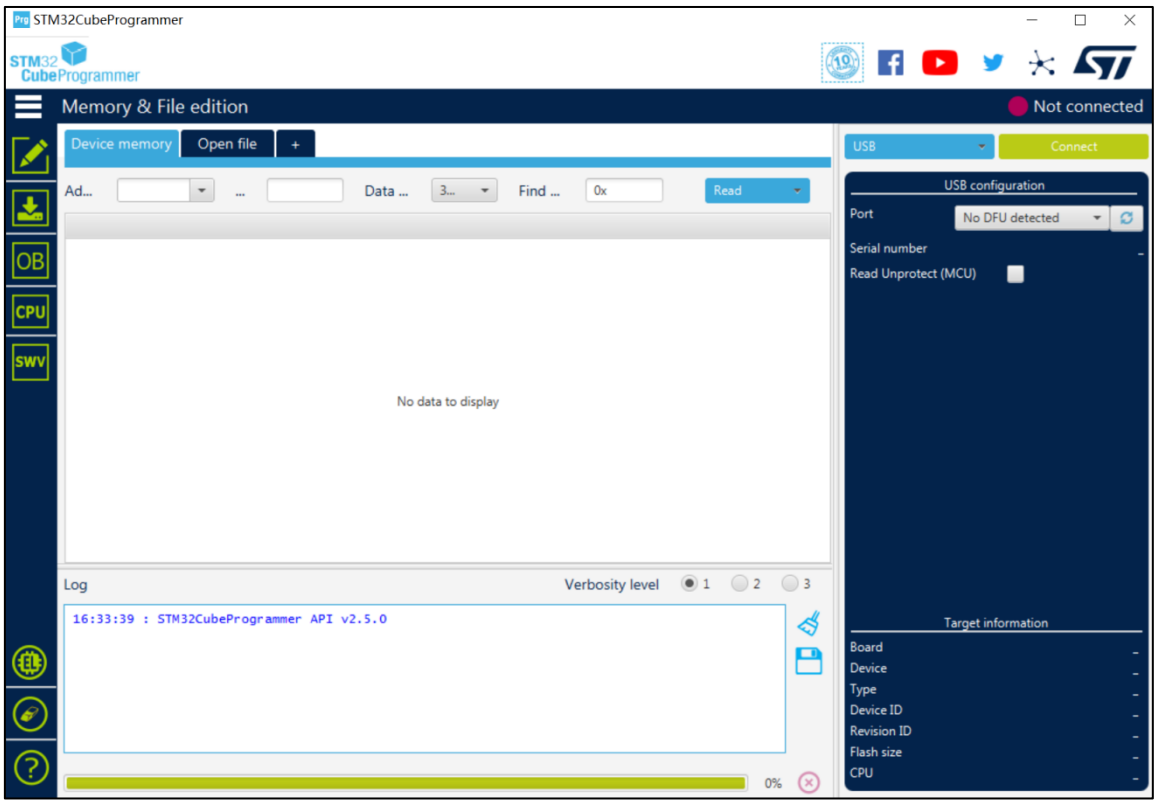


图 4.7.5.2 STM32cubeProgrammer 主界面

ST 为 STM32CubeProgrammer 专门编写了一个使用手册，这个手册已经放到了开发板光盘中，路径：开发板光盘→5、开发工具→2、ST 官方开发工具→STM32CubeProgrammer 软件工具介绍（中文版本）.pdf。

4.8 Ubuntu 下 ST 官方软件安装

上一小节讲解了如何在 Windows 下如何安装 STM32CubeMX、STM32CubeIDE 以及 STM32CubeProgrammer 这三个 ST 官方软件，本节我们就来讲解一下如何在 Ubuntu 下安装这三个软件。

4.8.1 Java 环境安装

在 Ubuntu 下安装这三个软件也是需要 JAVA 运行环境的，Ubuntu 可能会默认安装了 OpenJDK 环境，但是 STM32CubeProgrammer 是用 Oracle 的 JDK 编写的，所以需要先卸载掉默认的 OpeJDK，首先输入“java -version”查看一下当前 Ubuntu 系统下有没有 OpenJDK，有的话会输出如图 4.8.1.1 所示信息：

```
zuozhongkai@ubuntu:~/linux/tool/STM23CUbeProgrammer$ java -version
openjdk version "11.0.9" 2020-10-20
OpenJDK Runtime Environment (build 11.0.9+11-Ubuntu-0ubuntu1.18.04.1)
OpenJDK 64-Bit Server VM (build 11.0.9+11-Ubuntu-0ubuntu1.18.04.1, mixed mode, sharing)
zuozhongkai@ubuntu:~/linux/tool/STM23CUbeProgrammer$
```

图 4.8.1.1 JAVA 运行环境版本号

从图 4.8.1.1 可以看出，当前的 JAVA 是 openjdk 11.0.9，如果没有输出 JAVA 版本信息的话就说明当前 Ubuntu 还没有安装 JAVA 环境。卸载 OpenJDK 的方法很简单，输入如下命令：

```
sudo apt-get remove openjdk*
```

卸载完成以后重新输入“java -version”，如果不能输出 java 版本信息就说明卸载成功了。

接下来我们安装 Oracle 的 JAVA 运行环境，相应的软件包已经放到了开发板光盘中，路径：开发板光盘→3、软件→Java 安装包→jre-8u271-linux-x64.tar.gz，将此压缩包拷贝到 Ubuntu 下，然后解压到 Ubuntu 的/usr/lib/jvm 目录下，输入如下命令：

```
sudo mkdir /usr/local/java                                  //创建目录
sudo tar vzxf jre-8u271-linux-x64.tar.gz -C /usr/local/java  //解压
```

修改/etc/profile，在文件最后面追加如下内容：

```
export CLASSPATH=.:/usr/local/java/jre1.8.0_271/lib
export PATH=$PATH:/usr/local/java/jre1.8.0_271/bin
```

完成以后的/etc/profile 如图 4.8.1.2 所示：

```
29 export PATH=$PATH:/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf/bin
30
31 export CLASSPATH=.:/usr/local/java/jre1.8.0_271/lib
32 export PATH=$PATH:/usr/local/java/jre1.8.0_271/bin
33
```

图 4.8.1.2 追加 JAVA 相关内容

重启电脑，输入“java -version”查看 java 安装是否成功，安装成功的话就会打印出 java 环境的版本号，如图 4.8.1.3 所示：

```
zuozhongkai@ubuntu:~$ java -version
java version "1.8.0_271"
Java(TM) SE Runtime Environment (build 1.8.0_271-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.271-b09, mixed mode)
zuozhongkai@ubuntu:~$
```

图 4.8.1.3 java 版本号

从图 4.8.1.3 可以看出，当前 Ubuntu 下 JAVA 运行环境版本号为 1.8.0_271，JAVA 运行环境安装成功。

4.8.2 STM32CubeMX 安装

在 Ubuntu 下新建一个名为“STM32CubeMX”的文件夹，然后将开发板光盘中的 STM32CubeMX 安装包 en.stm32cubemx_v6-0-1.zip 发送到此文件夹下，然后进行解压，命令如下：

```
unzip en.stm32cubemx_v6-0-1.zip
```

解压完成以后得到的内容如图 4.8.2.1 所示：

```
zuozhongkai@ubuntu:~/linux/tool/STM32CubeMX$ ls
en.stm32cubemx_v6-0-1.zip    SetupSTM32CubeMX-6.0.1.exe
Readme.html                  SetupSTM32CubeMX-6.0.1.linux
SetupSTM32CubeMX-6.0.1.app
zuozhongkai@ubuntu:~/linux/tool/STM32CubeMX$
```

图 4.8.2.1 解压后的文件

图 4.8.2.1 中解压出来的文件一个都不要删除，防止安装的时候出错，在终端里面执行图 4.8.2.1 中的 SetupSTM32CubeMX-6.0.1.linux，输入如下命令：

```
chmod 777 SetupSTM32CubeMX-6.0.1.linux        //给予可执行权限
chmod 777 SetupSTM32CubeMX-6.0.1.exe          //给予可执行权限
./SetupSTM32CubeMX-6.0.1.linux
```

输入上述命令以后打开安装界面，如图 4.8.2.1 所示：

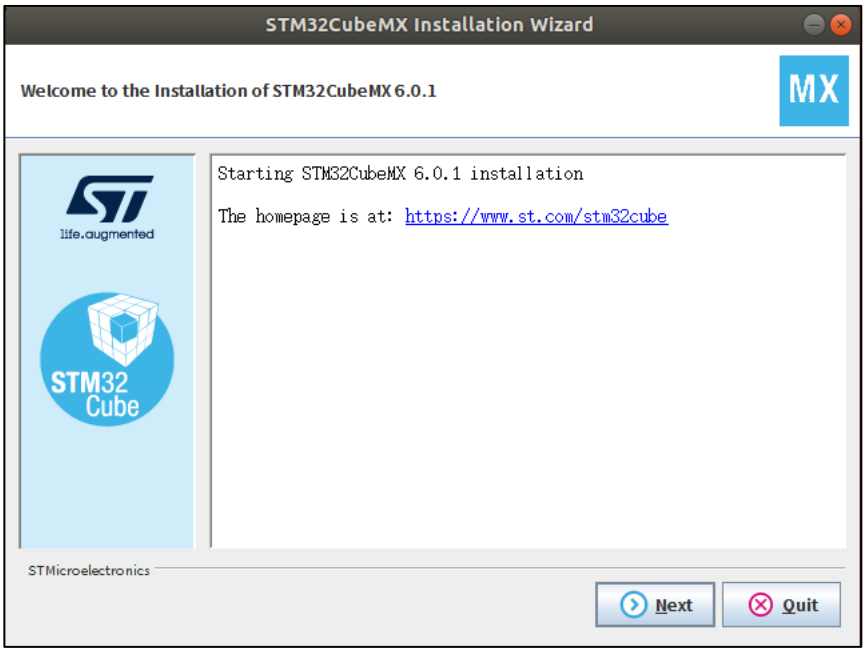


图 4.8.2.1 STM32CubeMX 安装界面

根据提示一步步安装下去，安装默认路径安装，注意，STM32CubeMX 安装完成以后可能在/usr/share/applications 目录下找不到 STM32CubeMX 的图标。所以我们需要直接搜索，点击 Ubuntu 主界面左上角的“活动”按钮，打开搜索条，输入“STM32CubeMX”查找相应的文件，结果如图 4.8.2.2 所示：

图 4.8.2.2 STM32CubeMX 搜索结果

图 4.8.2.2 中第一个搜索结果就是 STM32CubeMX 的安装文件夹，打开以后里面的内容如图 4.8.2.3 所示：

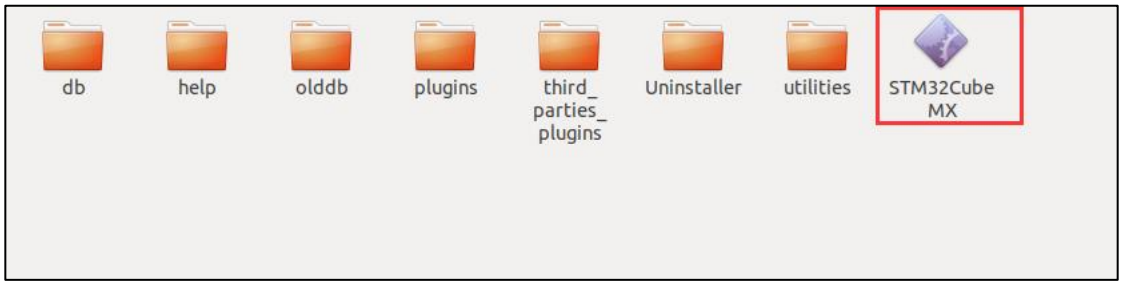


图 4.8.2.3 STM32CubeMX 安装文件

图 4.8.2.3 中的“STM32CubeMX”就是软件图标，双击即可打开 STM32CubeMX。

4.8.3 STM32CubeIDE 安装

注意 STM32CubeIDE 有专门的 Linux 版本，它并不是和 Windows 版放到一起的，我们已经放到了开发板光盘中，路径为：开发板光盘→5、开发工具→2、ST 官方开发工具→en.st-stm32cubeide_1.4.0_7511_20200720_0928_amd64_sh.zip

在 Ubuntu 下新建一个名为“STM32CubeIDE”的文件夹，然后将开发板光盘中的 STM32CubeMX 安装包 en.st-stm32cubeide_1.4.0_7511_20200720_0928_amd64_sh.zip 发送到此文件夹下，然后进行解压，命令如下：

```
unzip en.st-stm32cubeide_1.4.0_7511_20200720_0928_amd64_sh.zip
```

解压完成以后得到的内容如图 4.8.3.1 所示：

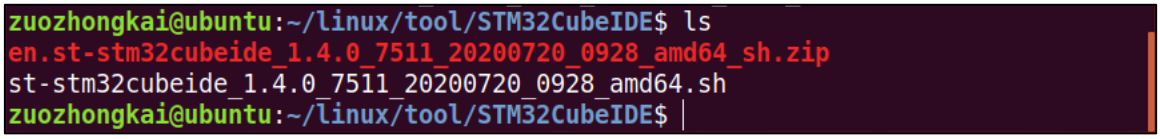


图 4.8.3.1 解压后的文件

在终端里面执行图 4.8.3.1 中的 st-stm32cubeide_1.4.0_7511_20200720_0928_amd64.sh，输入如下命令：

```
chmod 777 st-stm32cubeide_1.4.0_7511_20200720_0928_amd64.sh //给予可执行权限
./st-stm32cubeide_1.4.0_7511_20200720_0928_amd64.sh
```

输入上述命令以后会先解压压缩包，然后打开安装许可，阅读完安装许可，然后输入‘y’表示接受许可，如图 4.8.3.2 所示：

```
THIS SOFTWARE PACKAGE IS PROVIDED BY STMICROELECTRONICS AND CONTRIBUTORS "AS IS" AND
ORY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABIL
PURPOSE AND NON-INFRINGEMENT OF THIRD PARTY INTELLECTUAL PROPERTY RIGHTS ARE DISCLAIM
TTED BY LAW. IN NO EVENT SHALL STMICROELECTRONICS OR CONTRIBUTORS BE LIABLE FOR ANY D
PECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREME
ICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON
ER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING
THIS SOFTWARE PACKAGE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
EXCEPT AS EXPRESSLY PERMITTED HEREUNDER AND SUBJECT TO THE APPLICABLE LICENSING TERMS
INCORPORATED IN THE SOFTWARE PACKAGE AND OPEN SOURCE TERMS AS APPLICABLE, NO LICENSE
SS OR IMPLIED, ARE GRANTED UNDER ANY PATENT OR OTHER INTELLECTUAL PROPERTY RIGHTS OF
D PARTY.

I ACCEPT (y) / I DO NOT ACCEPT (N) [N/y] y
```

图 4.8.3.2 安装许可

在图 4.8.3.2 中输入‘y’，表示接受许可，默认为‘n’，也就是不接受许可。接下来会出现多个询问，根据实际情况填写即可，最后等待安装完成。安装完成以后到/usr/share/applications 目录下将 STM32CubeIDE 的图标复制到桌面上，这样我们就可以通过点击桌面图标打开 STM32CubeIDE 软件了。

4.8.4 STM32CubeProgrammer 安装

在 Ubuntu 下新建一个名为“STM32CubeProgrammer”的文件夹，然后将开发板光盘中的 STM32CubeMX 安装包 en.stm32cubeprog_v2-5-0.zip 发送到此文件夹下，然后进行解压缩，命令如下：

```
unzip en.stm32cubeprog_v2-5-0.zip
```

解压完成以后得到的内容如图 4.8.4.1 所示：

```
zuozhongkai@ubuntu:~/linux/tool/STM23CUbeProgrammer$ ls
en.stm32cubeprog_v2-5-0.zip         SetupSTM32CubeProgrammer-2.5.0.exe
SetupSTM32CubeProgrammer-2.5.0.app  SetupSTM32CubeProgrammer-2.5.0.linux
zuozhongkai@ubuntu:~/linux/tool/STM23CUbeProgrammer$
```

图 4.8.4.1 解压后的文件

图 4.8.4.2 中解压出来的文件一个都不要删除，防止安装的时候出错，在终端里面执行图 4.8.4.2 中的 SetupSTM32CubeProgrammer-2.5.0.linux 即可，输入如下命令：

```
./SetupSTM32CubeProgrammer-2.5.0.linux
```

输入上述命令以后打开安装界面，如图 4.8.4.3 所示：

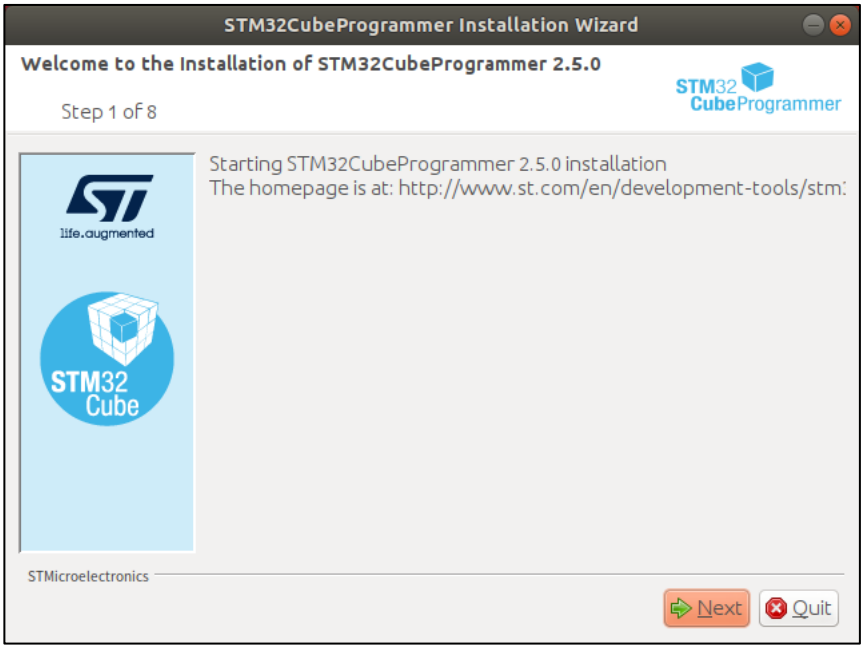


图 4.8.4.3 Ubuntu 下安装界面

和 Windwos 下一样，根据提示一步步安装下去，不要修改安装路径，采用默认路径即可。以后我们就可以直接在 Ubuntu 下通过 STM32CubeProgrammer 来烧写系统了。

最后，在 Ubuntu 中安装 libusb1.0 软件包，输入如下命令：

```
sudo apt-get install libusb-1.0.0-dev
```

4.9 USB DFU 以及 STLink 驱动安装

4.9.1 Windows 下 USB DFU 以及 STLink 驱动安装

在 Windows 下 USB DFU 驱动不需要安装，所以只需要安装 STLink 驱动，STLink 驱动已经放到了开发板光盘中，路径为：开发板光盘→3、软件→ST LINK 驱动及教程→ST-LINK 官方驱动.zip。解压该压缩包得到如图 4.9.1 所示内容：

图 4.9.1 ST LINK 驱动

如果是 64 位的电脑，则双击：dpinst_amd64.exe 进行安装。如果是 32 位的电脑，则双击：dpinst_x86.exe 进行安装。

安装完成后如图 4.9.2 所示：

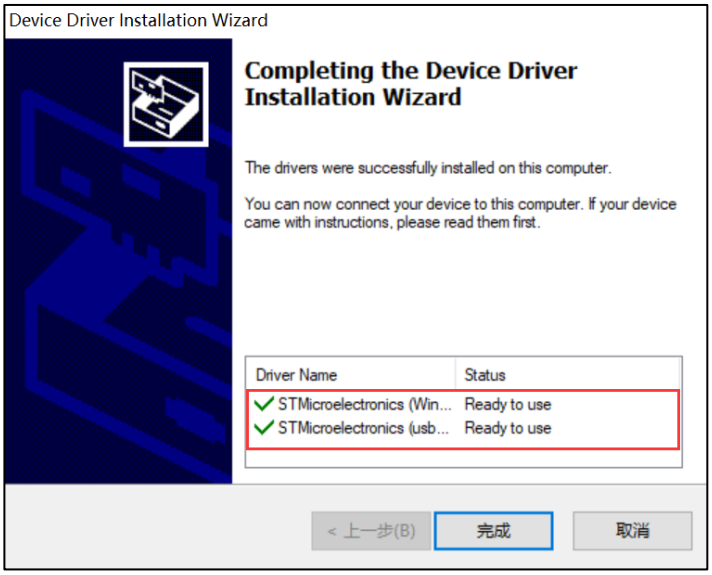


图 4.9.2 ST LINK 驱动安装成功

在 ST LINK 驱动安装完成以后，我们在电脑设备管理器里面可以看到会多出一个设备（此时 ST LINK 必须通过 USB 连接到电脑，ST LINK 红灯常亮），如图 4.9.3 所示：

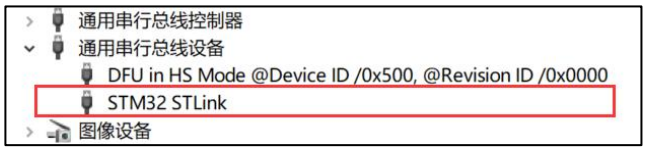


图 4.9.3 设备管理器显示 ST LINK 设备

注意：

①、不同 Windows 版本设备名称和所在设备管理器栏目可能不一样，例如 WIN7 电脑插上 ST LINK 后显示的是：STMicroelectronics STLINK dongle。

②、如果设备名称旁边显示的是黄色的叹号，请直接点击设备名称，然后在弹出的界面点击更新设备驱动。

4.9.2 Ubuntu 下 USB DFU 以及 STLink 驱动安装

如果想要在 Ubuntu 下通过 USB 烧写系统，那么必须要安装 USB DFU 驱动。

必须先在 Ubuntu 下安装 STM32CubeProgrammer，因为 STM32CubeProgrammer 安装过程中会解压出 USB DFU 以及 STLink 相关驱动文件，我们需要用到这些驱动文件！

找到 Ubuntu 里面的 STM32CubeProgrammer 安装路径，如果安装的时候选择的默认路径，那么就会在用户根目录下生成一个名为“STMicroelectronics”的文件夹，比如在我的电脑上的绝对路径就是“/home/zuozhongkai/STMicroelectronics”，如图 4.9.2.1 所示：

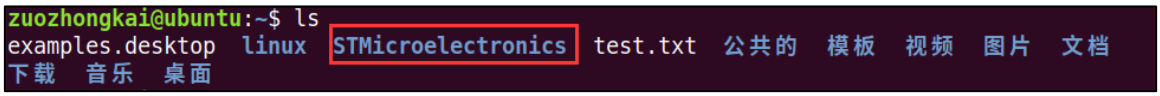


图 4.9.2.1 STM32 软件安装目录

进入路径：/STMicroelectronics/STM32Cube/STM32CubeProgrammer/Drivers/rules，此目录下下的文件内容如图 4.9.2.2 所示：

```
zuozhongkai@ubuntu:~/STMicroelectronics/STM32Cube/STM32CubeProgrammer/Drivers/rules$ ls
49-stlinkv1.rules    49-stlinkv2.rules  50-usb-conf.rules  version.txt
49-stlinkv2-1.rules  49-stlinkv3.rules  Readme.txt
zuozhongkai@ubuntu:~/STMicroelectronics/STM32Cube/STM32CubeProgrammer/Drivers/rules$
```

图 4.9.2.2 USB DFU 以及 STLink 驱动文件

将图 4.9.2.2 中的这些.rules 文件全部拷贝到 Ubuntu 的/etc/udev/rules.d 目录下，命令如下：

```
cd /home/zuozhongkai/STMicroelectronics/STM32Cube/STM32CubeProgrammer/Drivers/rules
sudo cp * /etc/udev/rules.d/
```

拷贝完成以后重启 Ubuntu，用 USB Type-C 线将开发板的 USB OTG 接口与电脑连接起来，默认情况下，拨码开关设置为 000，也就是 USB 启动，然后按下复位按键，此时电脑就会发出 USB 枚举的叮咚声。默认情况下，USB 会连接到 Windwos 下，我们需要将 USB 连接到 Ubuntu，所以需要设置一下 VMware，WMware 右下角会有当前电脑所有连接的 USB 设备，鼠标放上去以后会显示每个 USB 设备的名字，我们找到含有“STMicroelectronics DFU”字样的 USB 设备，如图 4.9.2.3 所示：

图 4.9.2.3 STM32MP157 USB DFU

图 4.9.2.3 中第二个 USB 设备就是 STM32MP157 的 USB DFU 模式，此时图标是灰色的，说明并没有连接到 Ubuntu 下，需要我们进行设置。鼠标放到图 4.9.2.3 中 USB DFU 设备上，比如此时我的电脑就是第二个图标，鼠标放上去以后点击鼠标右键，结果如图 4.9.2.4 所示：

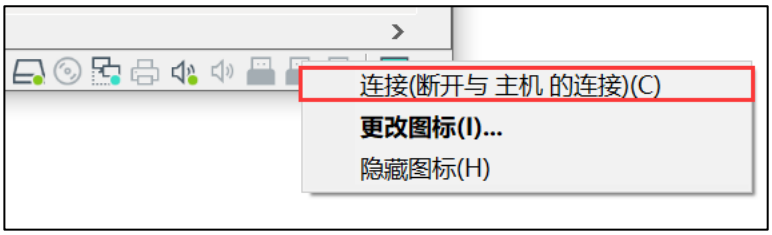


图 4.9.2.4 USB 设备连接到虚拟机

点击图 4.9.2.4 中的“连接”按钮，此时 USB DFU 就会断开与主机(Windows)的连接，从而连接到虚拟机(Ubuntu)上。连接成功以后对应的 USB 图标颜色就会深，如图 4.9.2.5 所示：

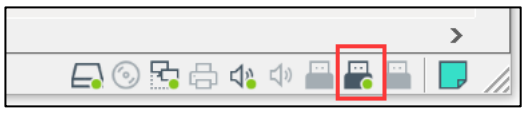


图 4.9.2.5 USB 设备连接到虚拟机

连接成功以后就可以使用 STM32CubeProgrammer 软件进行测试，打开 Ubuntu 下的 STM23CubeProgrammer 软件，选择“USB”字样，查看一下端口是否有“USB1”字样，如图 4.9.2.6 所示：

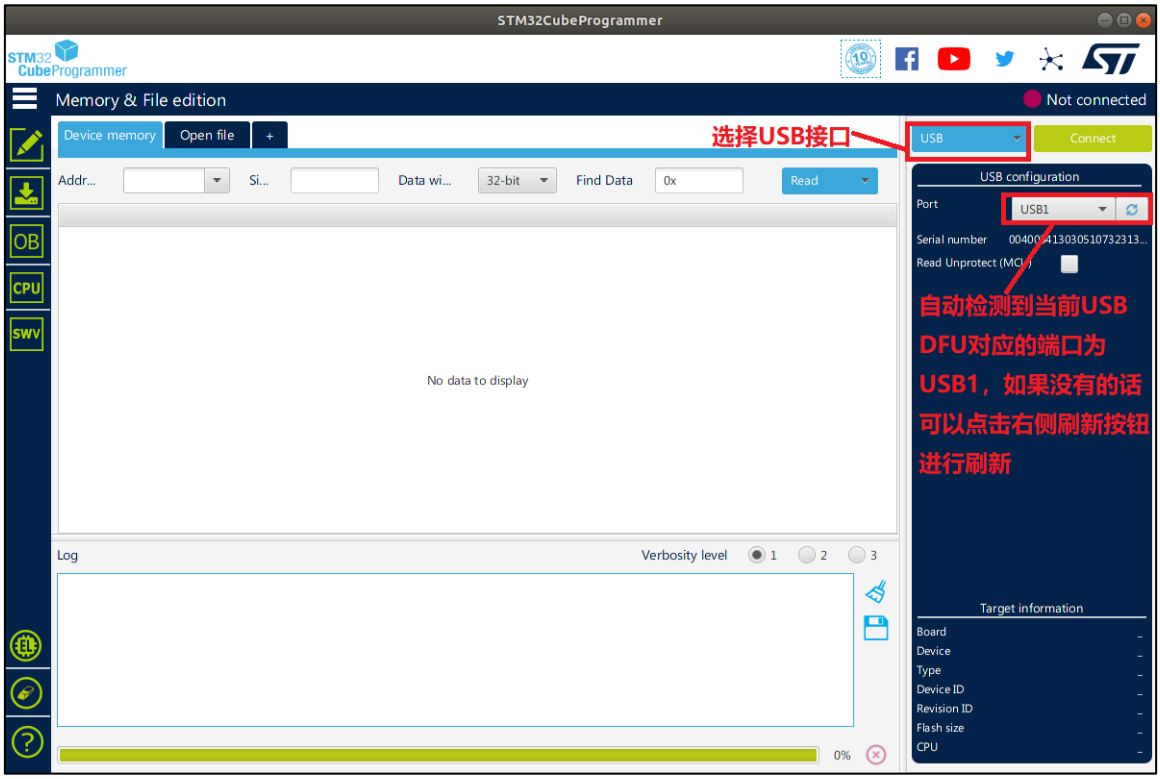


图 4.9.2.6 STM32CubeProgrammer 软件 USB 设置

点击图 4.9.2.6 右上角的“Connect”按钮即可通过 USB 接口连接到开发板上，左下角的 log 区就会输出相应的信息，如图 4.9.2.7 所示：

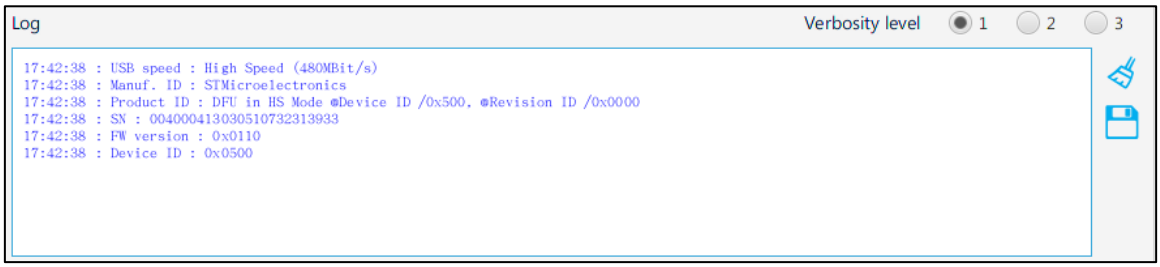


图 4.9.2.7 log 信息

可以看出此时 USB DFU 已经成功工作，说明 Ubuntu 下 STM32 USB DFU 驱动成功。

最后，测试下 STLink，这个测试比较简单，将 STLink 连接到 Ubuntu 下，如果 STLink 工作成功的话就会在/dev 目录下生成相应的设备文件，如图 4.9.2.8 所示：

```
zuozhongkai@ubuntu:~$ ls /dev/stlink*
/dev/stlinkv2_1
zuozhongkai@ubuntu:~$
```

图 4.9.2.8 stlink 设备文件

从图 4.9.2.8 可以看出，当前 STLink 为 V2.1 版本的。

第五章 STM32MP1 启动详解

STM32 单片机是直接将程序下载到内部 Flash 中，上电以后直接运行内部 Flash 中的程序。STM32MP157 内部没有供用户使用的 Flash，系统都是存放在外部 Flash 里面的，比如 EMMC、NAND 等，因此 STM32MP157 上电以后需要从外部 Flash 加载程序到内存中。而且 STM32MP157 支持多种启动方式，这些启动方式都是怎么运行的，这都涉及到 STM32MP1 的启动方式，本章我们就来详细讲解一下 STM32MP1 的启动过程。

5.1 STM32MP1 启动模式

STM32MP1 支持从多种设备启动，比如 EMMC、SD、NAND、NOR、USB、UART 等。STM32MP1 内部有一段 ROM 来存放 ST 自己编写的程序，这段 ROM 空间是不开放给用户使用的，仅供 ST 存放自己的 ROM 代码，ROM 空间如图 5.1.1 所示：

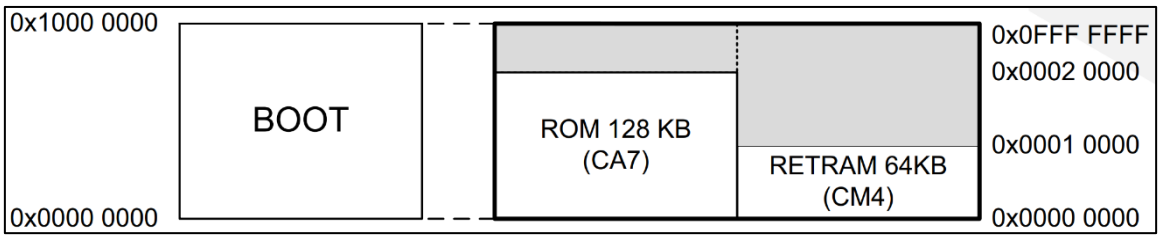


图 5.1.1 内部 ROM 空间

图 5.1.1 中 CA7 是 Cortex-A7 的缩写，可以看出 A7 内核有 128KB 的 ROM 空间，起始地址为 0X00000000，STM32MP1 上电以后会先运行这段 ROM 代码。STM32MP157 有三个 BOOT 引脚：BOOT0~BOOT2，这三个 BOOT 引脚通过拉高/拉低来设置从哪种设备启动，正点原子 STM32MP157 开发板上的拨码开关就是控制这三个 BOOT 引脚的，大家可以打开底板原理图，找到 BOOT 相关原理图，如图 5.1.2 所示：

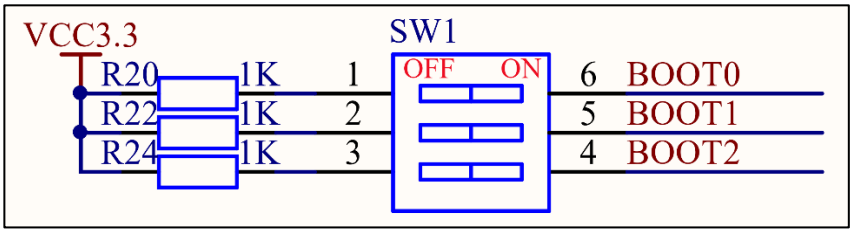


图 5.1.2 BOOT 原理图

从图 5.1.1 可以看出，当 BOOT0~BOOT1 拨到“ON”的时候就会接到 3.3V 上，此时就是逻辑 1，拨到 OFF 的时候 BOOT0~BOOT1 就悬空(也可以外接下拉电阻)，此时就是逻辑 0(悬空或接地都为逻辑 0)，三个引脚不同电平对应的启动模式，如表 5.1.1 所示：

BOOT2	BOOT1	BOOT0	启动模式	描述
0	0	0	UART/USB	- USART2/3/6、UART4/5/6/7/8。 - USB 接口。
0	0	1	串行 NOR	串行 NOR 或者 QUADSPI
0	1	0	EMMC	连接到 SDMMC2 上的 EMMC 设备。
0	1	1	并行 NAND	连接到 FMC 上的并行 SLC NAND
1	0	0	MCU	启动内部的 M4 内核。
1	0	1	SD	连接到 SDMMC1 上的 SD 卡。
1	1	0	UART/USB	和 000 效果一样，从 UART/USB 启动。
1	1	1	串行 NAND	连接到 QUADSPI 上的串行 NAND

表 5.1.1 启动模式

注：STM32MP1 的引脚具有复用功能，因此一个外设有很多不同引脚可以使用，比如 SDMMC2 的 D0 引脚就可以使用 PB14 或 PE6。STM32MP1 内部 ROM 代码肯定会有个默认的引脚，比如内部默认使用 PB14，如果你自己绘制的板子用了 PE6，那么就会出问题。当然了，可以通过修改 OTP 来设置启动设备所使用的引脚，但是笔者强烈不建议用户修改 OTP，因为 OTP 只能改一次，一旦修改错误芯片就废了。所以大家在绘制自己板子的时候启动设备所占用的引脚一定要和 ST 官方的开发板一致！

正点原子 STM32MP157 开发板上通过拨码开关来选择启动模式，开发板上有丝印提示如何选择不同的启动方式，如图 5.1.3 所示：

图 5.1.3 开发板拨码开关设置

大家可以根据开发板上的丝印来选择启动模式，拨上去为 ON，也就是‘1’，拨下来就是 OFF，也就是‘0’。

但是要注意一点，开发板拨码开关 3 个开关从左到右依次对应：BOOT0，BOOT1 和 BOOT2，因此图 5.1.2 中丝印从左到右依次是 BOOT0、BOOT1 和 BOOT2，顺序刚好和表 5.1.1 是相反的，表 5.1.1 中从左到右是 BOOT2、BOOT1 和 BOOT0，所以大家看到开发板上丝印和 ST 数据手册里面不同的时候不要以为是丝印标错了。

5.2 STM32MP1 启动流程详解

上一小节我们说了，STM32MP1 内部有一段 ST 自己编写的 ROM 代码，这段 ROM 代码上电以后就会自动运行，ROM 代码会读取 BOOT0~BOOT2 这三个引脚电平，获取启动模式信息，比如读取到是从 EMMC 启动的，那么 ROM 代码就会从 EMMC 中读取相关程序。说起来很简单，但是实际操作比较复杂，本节我们就来详细学习一下这个启动过程。

5.2.1 内部 ROM 代码

内部 ROM 代码支持如下功能：

①、Secure boot(安全启动)，不管是串行启动还是从 Flash 设备启动。

②、Engineering boot(工程启动？)，当 BOOT2~BOOT0 设置为 100 的时候，我们就可以通过 STLINK 访问 A7 或者 M4 内核。一般是通过此方法来调试 M4 内核代码。

③、Secondary core boot(第二个内核启动)，复位以后，STM32MP157 的每个 A7 内核都会启动，并且运行相同的指令。内部 ROM 代码会分离执行流，只有 Core0 才会运行 ROM 代码，另外一个内核会处于一个死循环状态，等待应用程序发送信号来进行下一步操作。这个信号是由 SGI(软中断)和另外两个 BACKUP 寄存器：MAGIC_NUMBER、BRANCH_ADDRESS 组成的。如果要启动 Core1，运行在 Core0 的应用程序需要：

- 将跳转地址写入 BRANCH_ADDRESS 寄存器。
- 将 0xCA7FACE1 这个值写入到 MAGIC_NUMBER 寄存器。
- 向 Core1 和发送 SGI 中断。

总结一下，只要你不自己开启第二个核，那么由于内部 ROM 代码的作用，此时 STM32MP157 就相当于单核 A7。这样有利于我们编写的 STM32MP157 的 A7 裸机例程，因为无需考虑多核情况。

④、RMA boot，RMA 是 Return Material Authorization 的缩写，笔者没研究过此种启动方式。

⑤、低功耗唤醒。

⑥、提供安全相关服务。

内部 ROM 启动流程如图 5.2.1.1 所示：

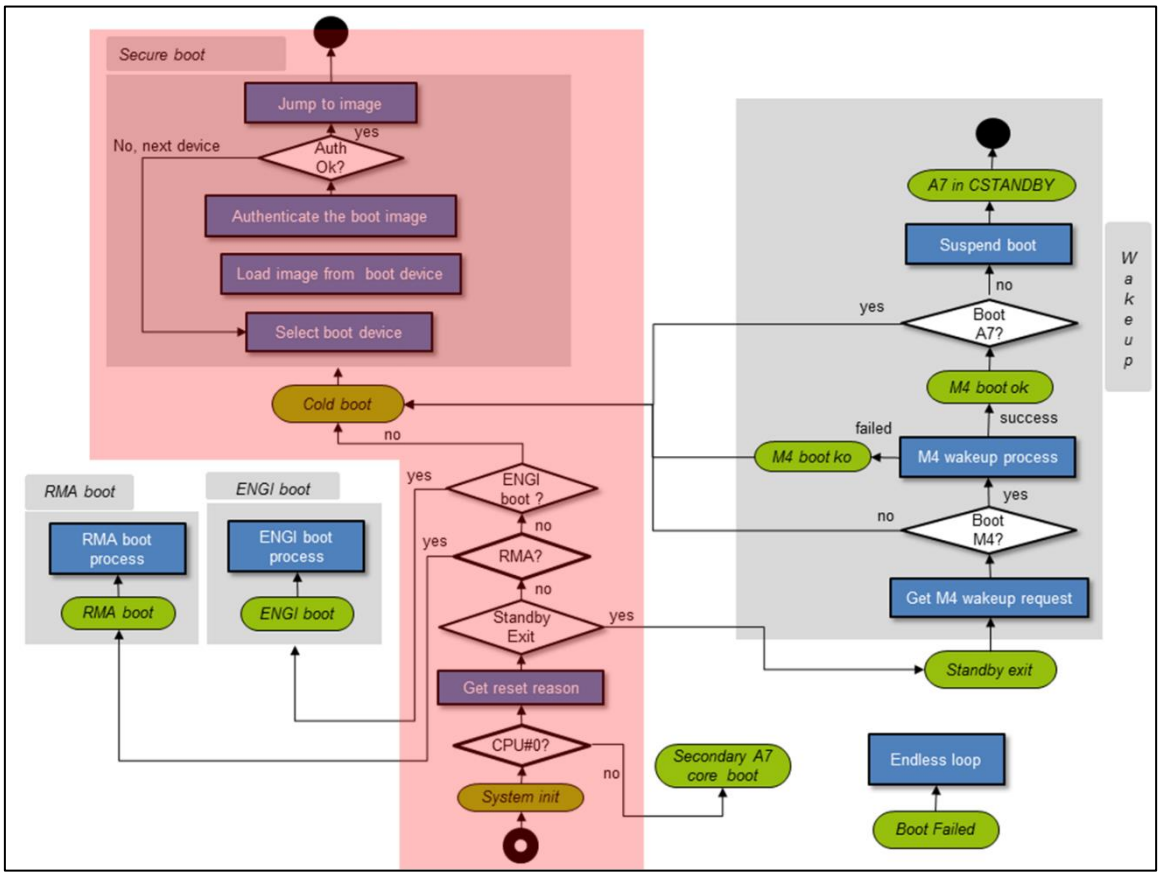


图 5.2.1.1 ROM 流程图

我们只关心图 5.2.1.1 中红色部分，因为这个是最常用的启动流程，也就是上电或复位以后运行流程：

①、首先检查当前是不是 CPU0 运行，如果不是的话就启动 CPU1，正常上电肯定是 CPU0 在运行。

②、如果是 CPU0 在运行，检查复位原因。

③、检查是否为退出 Standby 而导致的复位，如果不是的话就进入 RMA 检查。

③、检查是否为 RMA 启动，不是的话是否为 ENGI 启动。

④、如果不是 ENGI 启动的话直接进去冷启动。

⑤、进入冷启动以后就从 flash 中加载系统，并且进行鉴权，如果鉴权成功的话就运行系统。

5.2.2 安全启动

首先了解两个概念：

FSBL：全称为 First stage boot loader，也就是第一阶段启动文件。

SSBL：全称为 Second stage boot loader，也就是第二阶段启动文件。

当我们设置好 BOOT2~BOOT0，选择从外部 Flash，比如 EMMC、NAND 或 NOR 等启动的时候就会进入安全启动流程。STM32MP157 的安全启动流程比较复杂，这里我们就简单了解一下安全启动的基本流程：

①、首先 ROM 代码从选定的 Flash 设备中加载 FSBL 镜像文件，FSBL 镜像就是 ROM 加载的第一个用户编写的可执行程序，一般是 TF-A 镜像，但是我们可以换成自己编写的程序，比如 A7 裸机代码。当然了，这个 FSBL 镜像是有要求的，不是简单的把 bin 文件丢过来就可

以，而是需要在 bin 文件前面添加一个头部信息，否则内部 ROM 代码不知道如何处理这个 bin 文件，这个头部信息以后稍后讲解。

②、FSBL 镜像加载以后需要对其进行鉴权。

③、如果鉴权成功，那么就会跳转到 FSBL 镜像入口地址，开始运行 FSBL 固件。

内部 ROM 首先从选定的 Flash 设备中读取 FSBL 镜像文件并运行，但是此时 DDR 还没有初始化，那么 FSBL 镜像在哪里运行呢？肯定是内部 RAM 啊，ST32MP1 内部有 256KB 的 SYSRAM，如图 5.2.2.1 所示：

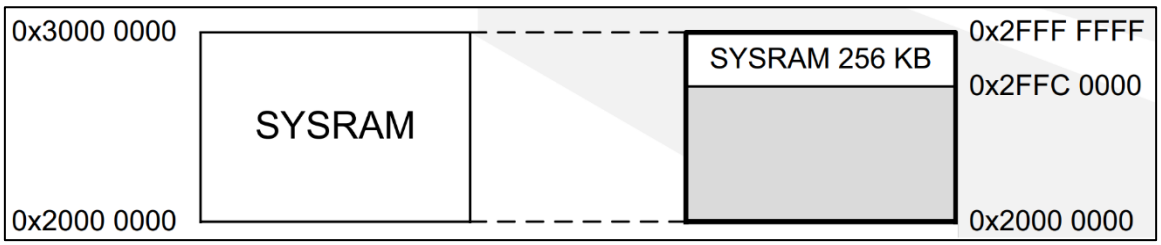


图 5.2.2.1 STM32MP1 内部 SYSRAM

从图 5.2.2.1 中可以看出，SYSRAM 地址范围为：0X2FFC0000~0X2FFFFFFF，一共是 256KB。ROM 代码会将 FSBL 镜像拷贝到 0X2FFC2400 地址，但是要注意，FSBL 镜像的起始地址不是 0X2FFC2400，因为 FSBL 镜像前面还有一个 256(0X100)字节的头部信息，因此 FSBL 镜像的真正起始地址为 0X2FFC2400+0X100=0X2FFC2500。为什么要讲明这一点呢？因为我们可以将 FSBL 镜像换成 A7 裸机例程，我们在编译 A7 裸机例程的时候要指定链接起始地址，这个链接起始地址就是 0X2FFC2500。由此可以计算出整个 FSBL 镜像大小不能超过 0X30000000-0X2FFC2500=252672B=246.75KB，246.75KB 足够我们编写裸机例程了。

FSBL 镜像鉴权成功以后，ROM 代码会 boot 上下文的起始地址保存到 R0 寄存器，然后跳转到 FSBL 镜像的入口地址，这个入口地址会定义到头部里面，其实就是上面讲的 0X2FFC2500。

boot 上下文我们不需要管，但是还要简单讲一下，boot 上下文会保存到 SYSRAM 的前 512 字节里面，boot 上下文包含了 boot 信息，比如选定的 boot 设备，还有一些和安全启动鉴权有关的服务。结构体 boot_api_context_t 定义了 boot 上下文结构，boot_api_context_t 结构体是定义在 TF-A 源码里面的(plat/st/stm32mp1/include/boot_api.h)，内容如下所示：

示例代码 5.2.2.1 boot 上下文结构

```
1  typedef struct {
2      /*
3      * Boot interface used to boot : take values from defines
4      * BOOT_API_CTX_BOOT_INTERFACE_SEL_XXX above
5      */
6      uint16_t boot_interface_selected;
7      uint16_t boot_interface_instance;
8      uint32_t reserved1[12];
9      uint32_t usb_context;
10     uint32_t otp_afmux_values[3];
11     uint32_t reserved[2];
12     /*
13     * Log to boot context, what was the kind of boot action
14     * takes values from defines BOOT_API_BOOT_ACTION_XXX above
15     */
```

```
16      uint32_t boot_action;
17      /*
18       * STANDBY Exit status to be checked by FSBL in case
19      * field 'boot_action' == BOOT_API_CTX_BOOT_ACTION_WAKEUP_STANDBY
20      * take values from defines above
        'BOOT_API_CTX_STBY_EXIT_STATUS_XXX'
21      * depending on encountered situation
22      */
23      uint32_t stby_exit_status;
24      /*
25      * CSTANDBY Exit status to be checked by FSBL in case
26      * boot_action == BOOT_API_CTX_BOOT_ACTION_WAKEUP_CSTANDBY
27      * take values from defines above
           'BOOT_API_CTX_CSTBY_EXIT_STATUS_XXX'
28      * depending on encountered situation
29      */
30      uint32_t cstby_exit_status;
31      uint32_t auth_status;
32
33      /*
34      * Pointers to bootROM External Secure Services
35      * - ECDSA check key
36      * - ECDSA verify signature
37      * - ECDSA verify signature and go
38      */
39      uint32_t (*bootrom_ecdsa_check_key)(uint8_t *pubkey_in,
40                  uint8_t *pubkey_out);
41      uint32_t (*bootrom_ecdsa_verify_signature)(uint8_t *hash_in,
42                     uint8_t *pubkey_in,
43                     uint8_t *signature,
44                     uint32_t ecc_algo);
45      uint32_t (*bootrom_ecdsa_verify_and_go)(uint8_t *hash_in,
46                  uint8_t *pub_key_in,
47                  uint8_t *signature,
48                  uint32_t ecc_algo,
49                  uint32_t *entry_in);
50
51      /*
52      * Information specific to an SD boot
53      * Updated each time an SD boot is at least attempted,
54      * even if not successful
55       * Note : This is useful to understand why an SD boot failed
56      * in particular
```

```
57      */
58      uint32_t sd_err_internal_timeout_cnt;
59      uint32_t sd_err_dcrc_fail_cnt;
60      uint32_t sd_err_dtimeout_cnt;
61      uint32_t sd_err_ctimeout_cnt;
62      uint32_t sd_err_ccrc_fail_cnt;
63      uint32_t sd_overall_retry_cnt;
64      /*
65      * Information specific to an eMMC boot
66      * Updated each time an eMMC boot is at least attempted,
67      * even if not successful
68      * Note : This is useful to understand why an eMMC boot failed
69      * in particular
70      */
71      uint32_t emmc_xfer_status;
72      uint32_t emmc_error_status;
73      uint32_t emmc_nbbytes_rxcopied_tosysram_download_area;
74      uint32_t hse_clock_value_in_hz;
75      /*
76      * Boot partition :
77      * ie FSBL partition on which the boot was successful
78      */
79      uint32_t boot_partition_used_toboot;
80      /*
81      * Address of SSP configuration structure :
82      * given and defined by bootROM
83      * and used by FSBL. The structure is of type
84      * 'boot_api_ssp_config_t'
85      */
86      boot_api_ssp_config_t *p_ssp_config;
87      /*
88      * boot context field containing bootROM updated SSP Status
89      * Values can be of type BOOT_API_CTX_SSP_STATUS_XXX
90      */
91      uint32_t    ssp_status;
92
93      /* Pointer on ROM constant containing ROM information */
94      const boot_api_rom_version_info_t *p_rom_version_info;
95
96  }   __packed boot_api_context_t;
```

boot_api_context_t 结构体目前不需要去研究，后面学习 TF-A 的时候根据实际情况在看是否有必要学习。

5.2.3 串行启动

当我们设置 BOOT2~BOOT0 为串行启动，也就是从 USB 或 UART 启动的时候就会进入此模式。当选择串行启动以后 ROM 代码就会并行扫描所有可以启动的 UART 以及 USB OTG 接口。当扫描到某个活动的串行接口以后，ROM 代码就会使用此串行接口，并且忽略掉其他的串行接口。

1、USB 启动

内部 ROM 代码支持 USB OTG 启动，我们一般使用 STM32CubeProgrammer 软件通过 USB OTG 接口来向 STM32MP1 烧写系统。USB OTG 需要一个 48M 和 60M 的时钟，这两个时钟由 HSE 生成。ROM 代码支持的 HSE 时钟值如下：

8, 10, 12, 14, 16, 20, 24, 25, 26, 28, 32, 36, 40, 48 MHz

正点原子 STM32MP157 开发板使用 24M 有源晶振作为 HSE 时钟源。我们可以通过设置 OTP 来更改 ROM 代码的 HSE 晶振大小，设置如表 5.2.3.1 所示：

OTP WORD 3 值 (2bit)	描述
00	默认模式，ROM 代码自动检测 HSE 频率，HSE 频率必须为 8, 10, 12, 14, 16, 20, 24, 25, 26, 28, 32, 36, 40 或 48MHz，如果没有检测到的话就直接认为是 24MHz
01	HSE=24MHz
10	HSE=25MHz
11	HSE=26MHz

表 5.2.3.1 HSE 时钟 OTP 设置表

从表可以看出，默认情况下 HSE 选择 24MHz，虽然可以通过修改 OTP 来更改 HSE，但是强烈不建议！因为 OTP 只能修改一次，一旦修改错误芯片就废了！所以大家在自己做核心板的时候外部 HSE 时钟最好选择 24MHz，不要特立独行！

2、UART 启动

如果要选择 UART 启动，也就是通过 UART 烧写系统，那么只能使用 USART2、USART3 UART4、UART5、USART6、UART7 或 UART8，此时串口工作模式为：1 位起始位、8 位数据位、偶校验、1 位停止位、波特率 115200。

由于 STM32 的 IO 复用功能，1 个串口可能有多个 IO 可以使用，比如 UART4 的 RX(接收)可以使用 PI10、PH14、PA1、PA11、PB2、PB8、PC11、PD0 或 PD2，一共 9 个 IO 可以用作 UART4_RX 引脚，但是 ROM 代码里面的 UART4_RX 引脚肯定只会使用这个 9 个里面的其中一个，所以我们板子的串口引脚要和 ROM 代码里面的一致，否则就无法使用串口启动。ROM 代码里面串口使用的引脚如表 5.2.3.2 所示：

串口	串口引脚	所使用的 IO	复用编号(AF)
USART2	USART2_RX	PA3	AF07
	USART2_TX	PA2	AF07
USART3	USART3_RX	PB12	AF08
	USART3_TX	PB10	AF07
UART4	UART4_RX	PB2	AF08
	UART4_TX	PG11	AF06
UART5	UART5_RX	PB5	AF12

	UART5_TX	PB13	AF14
USART6	USART6_RX	PC7	AF07
	USART6_TX	PC6	AF07
UART7	UART7_RX	PF6	AF07
	UART7_TX	PF7	AF07
UART8	UART8_RX	PE0	AF08
	UART8_TX	PE1	AF08

表 5.2.3.2 ROM 代码串口默认 IO

大家在做板子的时候，如果要使用串口启动，那么相关串口 IO 一定要参考表 5.2.3.2 中定义的 IO 引脚，比如正点原子开发板 UART4 的 RX 引脚使用 PB2，TX 引脚使用 PG11。

5.3 Flash 设备启动要求

STM32MP1 支持从 SD、EMMC、NAND 或 NOR 等 Flash 设备启动，但是不同的 Flash 设备在启动的时候有不同的要求。linux 系统他不像单片机那样，就一个 bin 文件，烧写进去就可以启动并运行，linux 系统自身编译出来就是一个镜像文件，但是这个镜像文件要运行是需要一大堆的“小弟”来辅助。比如需要 uboot 来启动，启动以后还需要根文件系统(rootfs)，传统的嵌入式 linux 有三巨头：uboot、kernel 和 rootfs，但是对于 STM32MP1 而言，由多了几个“小弟”，比如 TF-A、TEE、vendorfs 等，所有这一大堆构成了最终的系统镜像。系统镜像是要烧写到 Flash 设备中的，这些不同的文件肯定要按照一定的要求，分门别类的烧写，一个萝卜一个坑，TF-A 应该放到哪里、uboot 应该放到哪里等等。

针对 Flash 设备，可以通过创建不同的分区来存放不同的文件，ST 针对 STM32MP1 系列给出了官方分区建议，这些建议包含了 Flash 分区数量、分区最小空间、分区存放的内容等，如表 5.3.1 所示：

尺寸	分区	描述
256KB~512KB	fsbl	第一阶段启动代码，此分区存放 TF-A 或者 uboot 的 SPL 部分，如果写 A7 裸机例程的话此分区也用来存放裸机代码。
2MB	ssbl	第二阶段启动代码，一般是 uboot，如果 uboot 使用设备树的话，设备树添加到后面。
64MB	bootfs	boot 文件分区，可以存放如下内容： • init ram 文件系统，可以将此文件系统拷贝到 RAM 中，在 linux 内核挂载正式根文件系统之前可以使用 init ram 文件系统。 • linux 内核设备树 • linux 内核 • uboot 显示的启动界面 • uboot 发行配置文件 extlinux.conf
16MB	vendorfs	此分区存放第三方的版权信息，确保它们不会受到任何开源许可的污染，比如 GPL V3。
768MB	rootfs	linux 根文件系统。
剩余空间	userfs	用户自行使用的剩余空间

表 5.3.1 ST 官方的 Flash 分区建议

5.3.1 从 NAND 启动

NAND 前几个块(block)里面包含了多份 FSBL，ROM 代码会从第一个块开始扫描，并且加载第一个有效块里面的 FSBL。ROM 代码支持并行 NAND 和串行 NAND，并行 NAND 连接到 FMC 总线上，串行 NAND 连接到 QSPI 上。

ROM 代码支持的并行 NAND 要求如表 5.3.1.1 所示：

块大小(KB)	页大小(KB)	数据宽度	ECC(bit 数和编码)
128	2	8,16	4(bch)，8(bch)，1(hamming)
256	4	8,16	4(bch)，8(bch)，1(hamming)
512	4	8,16	4(bch)，8(bch)，1(hamming)
512	8	8,16	4(bch)，8(bch)，1(hamming)

表 5.3.1.1 并行 NAND 参数要求

ROM 代码支持串行 NAND 的要求如表 5.3.1.2 所示：

块大小(KB)	页大小(KB)
128	2
256	4
512	4
512	8

表 5.3.1.2 串行 NAND 参数要求

所以大家在制作 STM32MP1 硬件的时候，NAND Flash 选型一定要符合表 5.3.1.1 中的参数。

5.3.2 从 EMMC 启动

EMMC 在物理结构上有 boot1、boot2、RPMB(Replay Protected Memory Block)、GPP(General Purpose Partitions，GPP 最多 4 个分区)以及 UDA(User Data Area)这 5 种分区，比如三星的 KLM 系列 EMMC 5.1 的分区结构如图 5.3.2.1 所示：

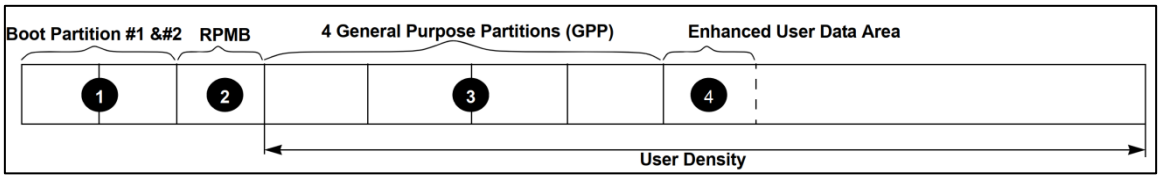


图 5.3.2.1 三星 KLM 系列 EMMC 分区结构

我们一般知道和常用的就是 UDA 分区，也就是用户数据区域，很少会关心 boot1、boot2 这样的分区。boot1、boot2、RPMB 这三个分区大小是固定的，用户不能修改，boot1、boot2 分区存在的意义就是用于引导系统。正点原子 STM32MP157 开发板所使用的 EMMC 型号为 KLM8G1GETF，这是三星的一颗 8GB EMMC 5.1 芯片，boot1、boot2 和 RPMB 分区大小如图 5.3.2.2 所示：

		Boot partition 1	Boot partition 2	RPMB
8GB	Default.	4,096KB	4,096KB	512KB
	Max.	4,096KB	4,096KB	4,096KB
16GB/32GB/64GB	Default.	4,096KB	4,096KB	4,096KB
	Max.	4,096KB	4,096KB	4,096KB

图 5.3.2.2 KLM 系列 EMMC 分区

从图 5.3.2.2 中可以看出，对于三星的 8GB 的 EMMC 而言，boot1 和 boot2 分区默认大小

为 4096KB，RPMB 为 512KB。

ST 会使用 EMMC 的 boot1 和 boot2 这两个分区作为 FSBL，但是同一时间只有一个有效，ROM 代码会加载有效的哪个 FSBL。ROM 代码使用单 bit 模式来操作 EMMC，默认情况下 ROM 代码使用连接到 SDMMC2 上的 EMMC，可以通过 OTP 来修改 EMMC 所使用的 SDMMC 接口，但是这里不建议！

5.3.3 从 SD 卡启动

SD 卡也包含两个 FSBL，但是 SD 卡没有 boot1 和 boot2 这样的物理分区。ROM 代码默认尝试加载第一个 FSBL，如果第一个 FSBL 加载失败，那么 ROM 代码就会加载第二个 FSBL。

ROM 代码首先在 SD 卡上查找 GPT 分区，如果找到的话就查找名字以“fsbl”开始的两个 FSBL 分区。如果没有找到 GPT 分区的话就直接根据物理地址查找两个 FSBL，第一个 FSBL 的起始偏移地址为 LBA34，地址位 34*512=17408=0X4400，所以第一个 FSBL 的起始地址为 0X4400。第二个 FSBL 的起始偏移地址为 LBA546，地址为 546*512=279552=0X44400，所以第二个 FSBL 的起始地址为 0X44400。

ROM 代码默认也是使用单 bit 模式操作 SD 卡，并且默认使用连接到 SDMMC1 接口上的 SD 卡。

5.4 STM32MP1 二进制头部信息

前面讲了 STM32MP1 内部的 ROM 代码会先读取 FSBL 代码，一般是 TF-A 或者 Uboot 的 SPL，也可以是 A7 裸机代码。比如 TF-A 我们之间编译生成二进制 bin 文件，但是这个 bin 文件不能直接拿来用，需要在前面添加一段头部信息，这段头部信息也包含了鉴权内容。加入头部信息以后的 FSBL 代码结构如图 5.4.1 所示：

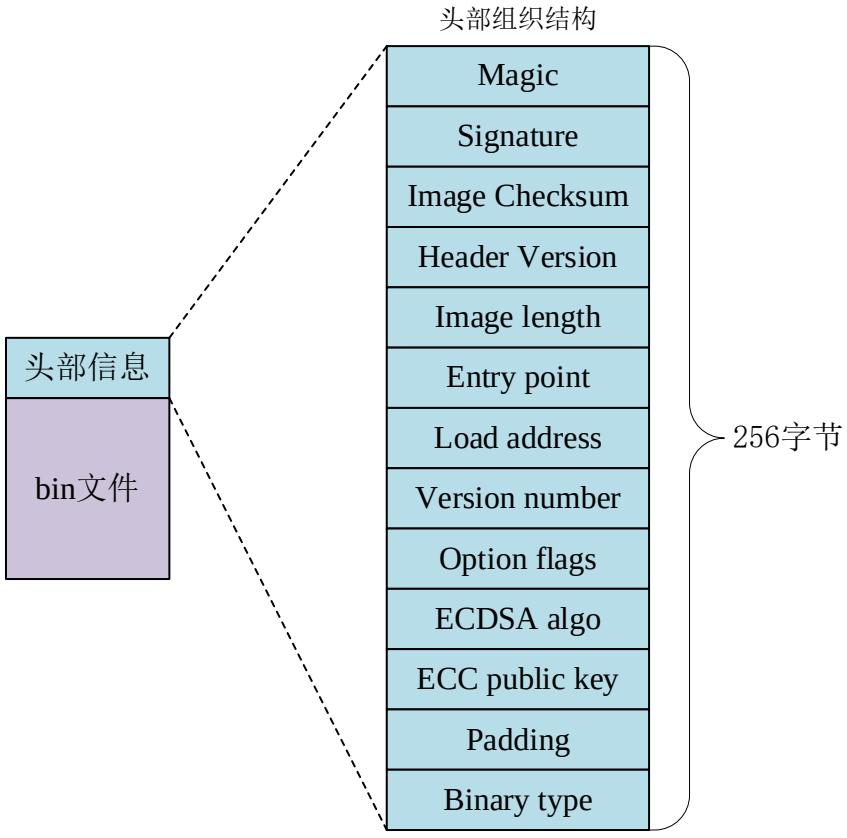


图 5.4.1 FSBL 镜像组织结构

头部信息一共是 256 字节，这 256 个字节的头部信息具体含义如表 5.4.1 所示：

名字	长度	偏移(B)	描述
Magic number	32bits	0	魔术数，4 字节，为‘S’、‘T’、‘M’和 0X32 的组合，固定为 0X53544D32。注意，这 4 个字节是大端模式，也就是高字节存储在低地址处，低字节数据在高地址处。
Image signature	512bits	4	ECDSA 签名，用于镜像鉴权。
Image checksum	32bits	68	镜像校验和。
Header version	32bits	72	头部版本信息，V1.0 的话为 0X00010000，含义： Byte0：保留 Byte1：主版本号为 0X01。 Byte2：次版本号为 0X00。 Byte3：保留
Image length	32bits	76	镜像长度，不包含头部，单位为字节。
Image entry Point	32bits	80	镜像入口地址。
Reserved1	32bits	84	保留。
Load address	32bits	88	镜像加载地址，ROM 代码不使用此地址。
Reserved2	32bits	92	保留。
Version number	32bits	96	镜像版本信息。
Option flags	32bits	100	可选字段，b0=1 的话表示不需要验证签名。
ECDSA algorithm	32bits	104	ECDSA 算法，1：P-256 NIST；2：brainpool 256
ECDSA pubilc key	512bits	108	ECDSA 公共秘钥，签名的时候使用。
Padding	83Bytes	172	保留的填充区域，必须全部为 0
Binary type	1Byte	255	二进制文件类型： 0X00：U-Boot 0X10-0X1F：TF-A 0X20-0X2F：OPTEE 0X30：Copro

表 5.4.1 头部信息含义

头部信息不需要我们自己手动添加，我们在编译 ST 官方提供的 TF-A 或者 Uboot 的时候会自动添加，因为 ST 提供了个名为“stm32image”的工具专门用于在 bin 文件前面添加头部信息。我们已经从 TF-A 源码中提取出来了 stm32image 并放到了开发板光盘中，路径为：开发板光盘→5、开发工具→2、ST 官方开发工具→stm32image→stm32image.c。在编写 A7 裸机的时候需要自己使用 stm32image 工具在 bin 文件前面添加头部信息，stm32image 是在 Ubuntu 下运行的，所以需要先编译，将 stm32image.c 发送到 Ubuntu 下，然后输入如下命令编译：

```
gcc stm32image.c -o stm32image
```

编译成功以后就会生成一个名为 stm32image 的可执行文件，如不 5.4.2 所示：

```
zuozhongkai@ubuntu:~/linux/tool/stm32image$ gcc stm32image.c -o stm32image
zuozhongkai@ubuntu:~/linux/tool/stm32image$ ls
stm32image  stm32image.c
zuozhongkai@ubuntu:~/linux/tool/stm32image$
```

图 5.4.2 stm32image 工具

运行图 5.4.2 编译出来的 stm32image 工具，输入“-s”选项可以查看使用方法，如图 5.4.3

所示：

```
zuozhongkai@ubuntu:~/linux/tool/stm32image$ ./stm32image -s
Usage : ./stm32image [-s srcfile] [-d destfile] [-l loadaddr] [-e entry_point] [-m major] [-n minor]
zuozhongkai@ubuntu:~/linux/tool/stm32image$
```

图 5.4.3 stm32image 使用方法

从图 5.4.3 可以看出，stm32image 在使用的时候需要搭配一系列的参数：

-s：指定源文件。

-d：生成的目标文件。

-l：加载地址。

-e：入口地址。

-m：主版本号。

-n：次版本号。

大家在开发板光盘里面找到正点原子出厂的 tf-a 固件，路径为：开发板光盘→8、系统镜像→2、出厂系统镜像→1、STM32CubeProg 烧录固件包→tf-a→tf-a-stm32mp157d-atk-trusted.stm32，这个就是加入了头部信息的 TF-A 可执行文件。使用 winhex 软件打开 tf-a-stm32mp157d-atk-trusted.stm32，winhex 软件已经放到了开发板光盘中，路径为：开发板光盘→ 3、软件→winhexv19.7.zip，大家自行安装即可。安装完成以后打开 winhex，然后点击：文件->打开，找到 tf-a-stm32mp157d-atk-trusted.stm32 并打开，如图 5.4.4 所示：

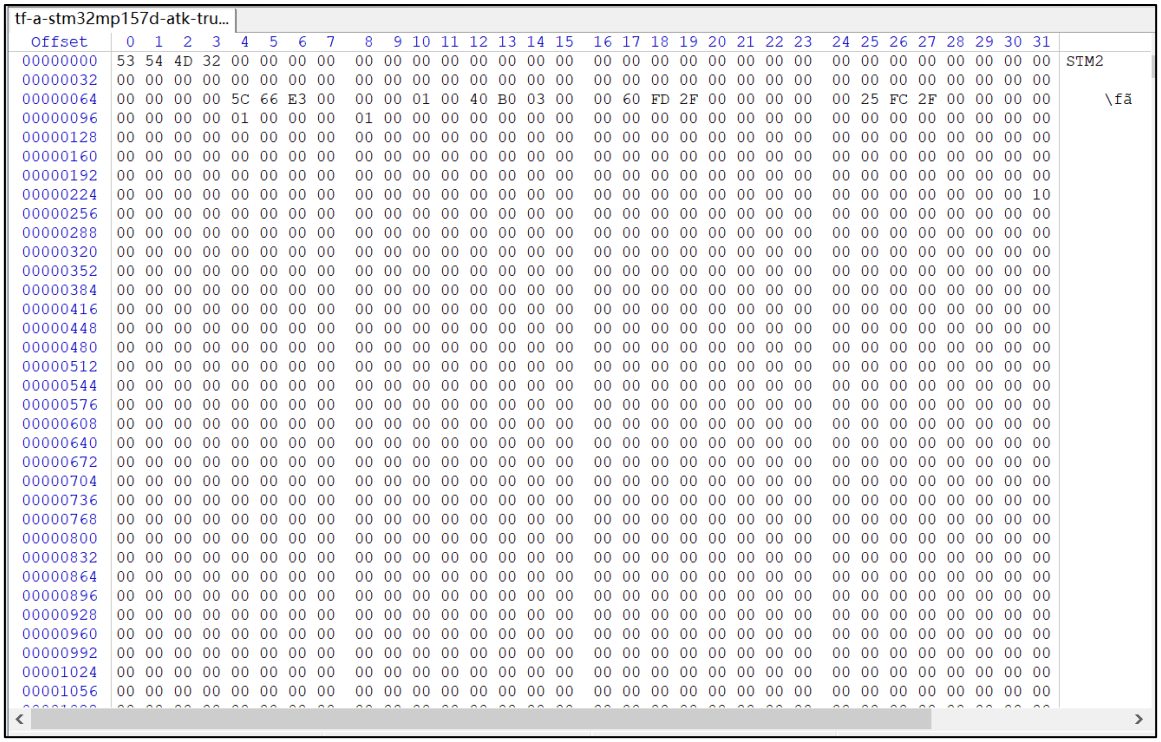


图 5.4.4 tf-a-stm32mp157d-atk-trusted.stm32 文件内容

图 5.4.4 就是 tf-a-stm32mp157d-atk-trusted.stm32 文件原始数据，其中前 256 个字节就是头部信息。这里我们根据图 5.4.4 中的内容，分析一下 tf-a 头部信息中几个比较重要的参数：

Magic number：起始偏移地址为 0，长度为 4 个字节，值依次为：0X53、0X54、0X4D、0X32，合起来就是 0X53544D32，这个就是表 5.4.1 中的魔术数，注意这四个字节的顺序是大端模式。

Header Version：起始偏移地址为 72，长度为 4 个字节，也就是图 5.4.4 中第 72~75 这 4 个字节的数据，分别为：0X00、0X00、0X01 和 0X00，此时有的朋友将这四个字节拼起来发现是

0X00000100，发现并不是表 5.1.1 中的 0X00010000！这不是弄错了？肯定不是的，整个头部信息中，除了 Magic number 采用大端模式存储以外，其他都是小端模式存储，也就是低字节数据存放在底地址处，高字节数据存放在高字节处。因此 0X00、0X00、0X01 和 0X00 这四个字节的数据正确的拼出结果为 0X00010000。

Image length：起始偏移地址为 76，长度为 4 个字节，也就是图 76~79 这 4 个字节的数据，为：0X40、0XB0、0X03 和 0X00，按照小端模式拼起来就是 0X0003B040=241728≈236.1KB，说明此 TF-A 的 bin 镜像大小为 236.1KB。

Image entry Point：起始偏移地址为 80，长度为 4 个字节，也就是图 80~83 这 4 个字节的数据，为：0X00、0X60、0XFD 和 0X2F，按照小端模式拼起来就是 0X2FFD6000，说明入口地址为 0X2FFD6000。

Load address：起始偏移地址为 88，长度为 4 个字节，也就是图 88~91 这 4 个字节的数据，为：0X00、0X25、0XFC 和 0X2F，按照小端模式拼起来就是 0X2FFC2500，说明加载地址为 0X2FFC2500，这个不正是我们前面在 5.2.2 小节中分析的 FSBL 镜像起始地址。

Binary type：起始偏移地址为 255，也就是最后一个字节，为 0X10，表示当前二进制文件是 TF-A。

5.5 STM23MP1 Linux 系统启动过程

前面已经对 STM32MP1 的启动流程做了详细的讲解，STM32MP1 是面向 Linux 领域的，因此所以的这些启动过程都是为了启动 Linux 内核。STM32MP1xil 启动 Linux 内核的流程如图 5.5.1 所示：

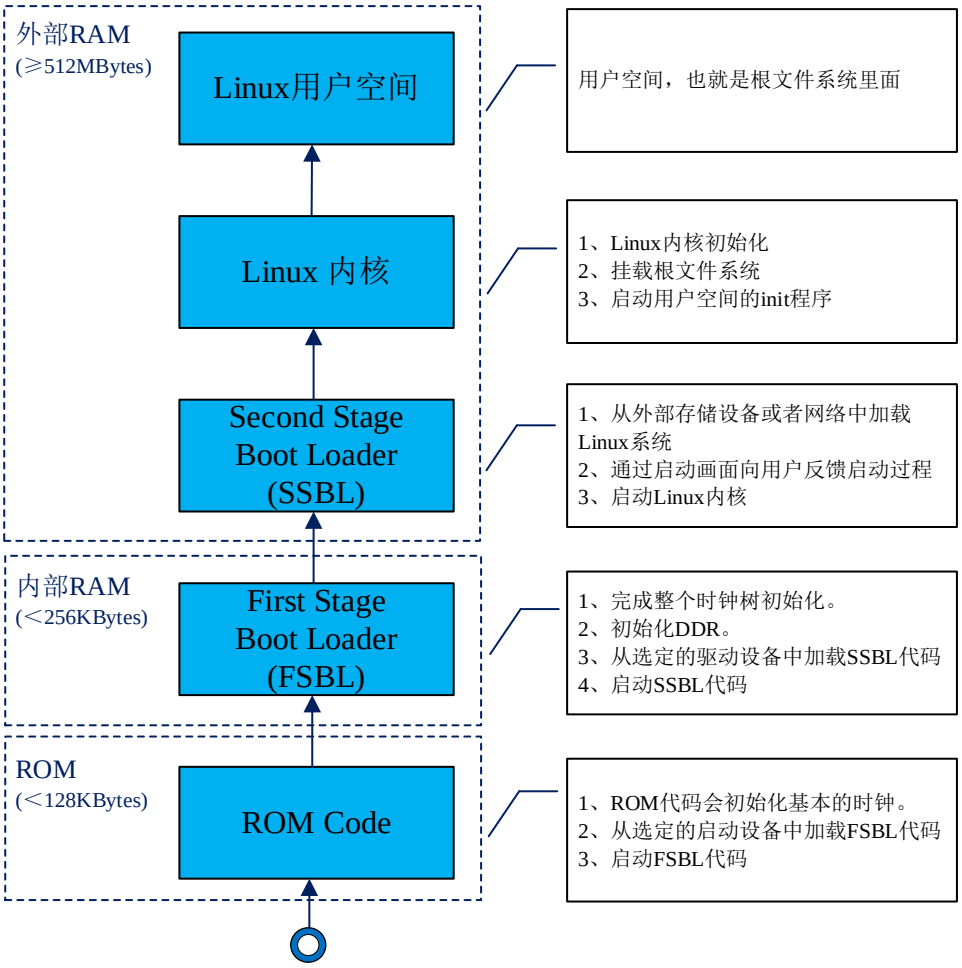


图 5.5.1 MP1 Linux 启动流程

从图 5.5.1 可以看出，STM32MP1 启动 linux 内核一共分为 5 个步骤，我们依次来看一下这五个步骤的内容：

①、ROM 代码

前面说了很多次了，这是 ST 自己编写的代码，在 STM32MP1 出厂的时候就已经烧写进去的，不能被修改的。ROM 代码因为保存在 STM32 内部 ROM 里面，因此也就直接简单明了的叫做“ROM 代码”了。它是处理器上电以后首先执行的程序，ROM 代码的主要工作就是读取 STM32MP1 的 BOOT 引脚电平，然后根据电平判断当前启动设备，最后从选定的启动设备里面读取 FSBL 代码，并将 FSBL 代码放到对应的 RAM 空间。

现在很多产品对设备上运行的应用都提出了安全要求，从图 5.5.1 中可以看出，STM32MP1 启动 Linux 内核的过程是一个链式结构：ROM Code→FSBL→SSBL→Linux kernel→rootfs，系统启动的过程中要保证整个链式结构都是安全的。ROM 代码作为第一链，首先要对 FSBL 代码进行鉴权，同样的，FSBL 以及后面的每一链都要对下一个阶段的镜像进行鉴权，直到设备系统正确启动。

②、FSBL

FSBL 代码初始化时钟树、初始化外部 RAM 控制器，也就是 DDR。最终 FSBL 将 SSBL 加载到 DDR 里面并运行 SSBL 代码。

一般 FSBL 代码是 TF-A 或者 Uboot 的 SPL 代码，前面我们说了，也可以将 FSBL 换成我

们自己编写的 STM32MP1 A7 内核裸机代码。

③、SSBL

由于 SSBL 代码运行在 DDR 里面，无需担心空间不够，因此 SSBL 代码的功能就可以做的很全面，比如使能 USB、网络、显示等等。这样我们就可以在 SSBL 中灵活的加载 linux 内核，比如从 Flash 设备上读取，或者通过网络下载下载等，用户使用起来也非常的友好。SSBL 一般是 Uboot，用来启动 Linux 内核。

③、Linux 内核

SSBL 部分的 Uboot 就一个使命，启动 Linux 内核，Uboot 会将 Linux 内核加载到 DDR 上并运行。Linux 内核启动过程中会初始化板子上的各种外设。

④、Linux 用户空间

系统启动的时候会通过 init 进程切换到用户空间，在这个过程中会初始化根文件系统里面的各种框架以及服务。

第六章 TF-A 使用

在上一章已经学会了如何搭建 STM32MP157 芯片的开发环境，为了方便书写，本教程把 STM32MP157 简称为 MP1。在这个嵌入式的高速发展时代，智能设备在各个领域扮演重要的角色，大量的嵌入式设备使用都是 ARM 为核心的芯片，为了保证安全 ARM 推出了 Arm Trusted Firmware 的可信固件，简称 TF-A。它是一个开源的软件，最早是用在 Armv8-A，ST 也在 MP1 里面使用到了 TF-A。它的作用就是隔离硬件，为硬件提供一个安全环境并且提供安全服务，本章我们先来体验一下 TF-A。

6.1 你好 TF-A

时至今日，智能手机、物联网飞速发展，智能手机里面保存了我们的指纹、Face ID(3D 人脸信息)、虹膜、银行卡密码等重要信息。而物联网更是与我们的生活息息相关，比如家用智能摄像头，一旦安全防线被突破，那么我们的生活就会毫无遮拦的暴露给整个世界。因此智能设备的安全问题就成了半导体行业亟待解决的问题，ARM 为此提供了 TrustZone 解决方案，TrustZone 将 CPU 的工作状态分为了 Secure World(安全世界)和 Normal World(非安全世界)，涉及到安全相关的内容运行在安全世界，比如指纹、密码等，其他的操作都在非安全世界运行，比如应用程序。TrustZone 是一种硬件解决方案，这里就不深究了(TrustZone 相关的安全知识本身就是一门课程)。

6.1.1 系统源码获取

与之对应的，我们需要一套软件来配合 TrustZone，TF-A 应运而生，TF-A 全称是 Arm Trusted Firmware，有些资料也叫做 ATF，一般中文资料叫做 ARM 可信固件。MP1 内部集成了 TrustZone，因此 ST 也提供了 TF-A 相关源码，TF-A 会先初始化 DDR 等外设，把 Uboot 从 Flash(NAND、NOR FLASH、SD、MMC 等)拷贝到 DDR 中。我们不可能直接去官方网站下载 TF-A 的源码，这样的开发难度太大，半导体厂商都会从 TF-A 官网下载源码，然后修改适配自己的芯片，把自家的芯片加进去。我们在实际项目开发中直接使用半导体原厂给提供的 TF-A 即可。ST 官方 TF-A、uboot、kernel 等源码下载链接为：https://my.st.com/content/my_st_com/en/products/embedded-software/mcu-mpu-embedded-software/stm32-embedded-software/stm32-mpu-openstlinux-distribution/stm32mp1dev.html，打开以后如图 6.1.1.1 所示：

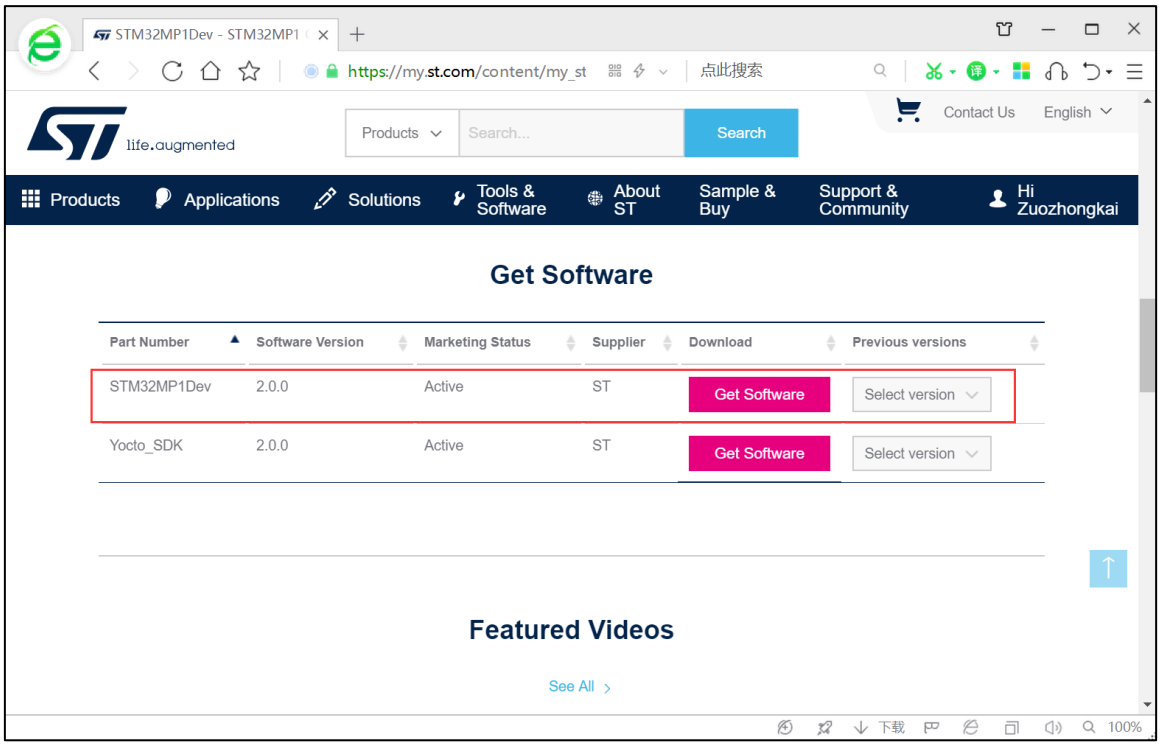


图 6.1.1.1 MP1 系统源码下载

图 6.1.1.1 中的 STM32MP1Dev 就是官方的源码包，里面包括 TF-A，uboot，kernel 等源码，点击“Get Software”下载整个系统源码。这里我们已经下载下来并放到了开发板光盘中，路径为：开发板光盘→1、程序源码→5、ST 官方原版 Linux 源码→en.SOURCES-stm32mp1-openstli

nux-5-4-dunfell-mp1-20-06-24.tar.xz。

在 Ubuntu 中创建一个目录存放源码，然后将 ST 官方系统源码发送到 Ubuntu 中。这里我在前面创建的“linux”目录下新建一个名为“atk-mp1”的目录存放所有源码，命令如下：

```
cd  linux/                  //进入 linux 目录
mkdir  atk-mp1              //创建 atk-mp1 目录
```

用 FileZilla 将 ST 官方源码压缩包 en.SOURCES-stm32mp1-openstlinux-5-4-dunfell-mp1-20-06-24.tar.xz 拷贝到“atk-mp1”目录下。拷贝结果如图.1.1.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1$ ls
en.SOURCES-stm32mp1-openstlinux-5-4-dunfell-mp1-20-06-24.tar.xz
```

图 6.1.1.2 拷贝官方的 SDK

解压图 6.1.1.2 中的压缩包，命令如下：

```
tar -xvf en.SOURCES-stm32mp1-openstlinux-5-4-dunfell-mp1-20-06-24.tar.xz
```

解压完成以后会得到一个名为“stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24”的文件夹，结果如图 6.1.1.3 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1$ ls
en.SOURCES-stm32mp1-openstlinux-5-4-dunfell-mp1-20-06-24.tar.xz
stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24
```

图 6.1.1.3 解压官方的 SDK 包结果

进入 stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi 目录下，里面就是 uboot、optee、tf-a、kernel 源码，如图 6.1.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/
arm-ostl-linux-gnueabi$ ls
linux-stm32mp-5.4.31-r0       tf-a-stm32mp-ssp-2.2.r1-r0
optee-os-stm32mp-3.9.0.r1-r0  u-boot-stm32mp-2020.01-r0
tf-a-stm32mp-2.2.r1-r0
```

图 6.1.1.4 系统源码

图 6.1.1.4 中这 5 个源码文件夹的含义如表 6.1.1.1 所示：

文件夹	描述
u-boot-stm32mp-2020.01-r0	uboot 源码，版本号为 2020.01
linux-stm32mp-5.4.31-r0	linux 源码，版本号为 5.4.31
tf-a-stm32mp-2.2.r1-r0	tf-a 源码，版本号为 2.2，我们用到的
tf-a-stm32mp-ssp-2.2.r1-r0	tf-a 源码，ssp 全称为 secure secret provisioning，安全相关的内容，本教程用不到。
optee-os-stm32mp-3.9.0.r1-r0	optee 系统源码，版本为 3.9.0，本教程用不到

表 6.1.1.1 系统源码

表 6.1.1.1 中共有 5 个源码，本教程就用到前三个：u-boot-stm32mp-2020.01-r0、linux-stm32mp-5.4.31-r0 和 tf-a-stm32mp-2.2.r1-r0。也就是 TF-A、Uboot 和 Linux Kernel。传统的 linux 学习中不需要 TF-A 的，只需要 uboot 和 Linux Kernel，但是 MP1 带有安全硬件，因此加入了 TF-A，加入的 TF-A 对于 linux 学习极其不利。为此我们建议过 ST 修改系统源码，允许客户不使用 TF-A，就像传统的嵌入式 linux 一样，只用 uboot 和 linux kernel。不过在编写本教程的时候并未看到 ST 提供这样的源码，所以我们还是要移植 TF-A 到开发板上的。

首先 TF-A 并不是 ST 自己做的，TF-A 有自己的官方组织，ST 只是将自己的 MP1 芯片移植到了 TF-A 上，表 6.1.1.1 中的 tf-a-stm32mp-2.2.r1-r0 就是 ST 自己修改后的 TF-A 源码，我们作为 MP1 芯片的使用者，直接使用 ST 修改后的 TF-A 源码即可，包括 uboot、linux kernel 都是这样的。

tf-a-stm32mp-2.2.r1-r0 支持 ST 所有的 MP1 芯片，也支持各种启动方式，例如：EMMC、NAND、NOR FLASH 等等。tf-a-stm32mp-2.2.r1-r0 里面包含了 ST 自家所有的 MP1 评估板，正点原子的 STM32MP157 开发板参考了 ST 官方的 STM32MP157C-EV1 开发板，因此，后续的移植都是以 STM32MP157C-EV1 开发板为蓝本，在此基础上进行修改。

tf-a-stm32mp-2.2.r1-r0 目录文件如图 6.1.1.5 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl
-linux-gnueabi/tf-a-stm32mp-2.2.r1-r0$ ls -l
总用量 5064
-rw-r--r-- 1 zuozhongkai zuozhongkai 1281142 6月  17 20:16 0001-st-update-v2.2-r2.0.0.patch
-rw-r--r-- 1 zuozhongkai zuozhongkai    3921 6月  17 20:16 Makefile.sdk
-rw-r--r-- 1 zuozhongkai zuozhongkai    4638 6月  17 20:16 README.HOW_TO.txt
-rw-r--r-- 1 zuozhongkai zuozhongkai      37 6月  17 20:16 series
-rw-r--r-- 1 zuozhongkai zuozhongkai 3886266 6月  17 20:16 tf-a-stm32mp-2.2.r1-r0.tar.gz
```

图 6.1.1.5 tf-a 源码

图 6.1.1.5 中一共有 5 个文件，这五个文件的含义如表 6.1.1.2 所示：

文件/文件夹	描述
0001-st-update-v2.2-r2.0.0.patch	TF-a 补丁文件。
Makefile.sdk	编译 TF-A 用的 Makefile
README.HOW_TO.txt	ST 官方 TF-A 的编译文档，编译方法看此文档即可。
series	存放补丁名字的文件
tf-a-stm32mp-2.2.r1-r0.tar.gz	TF-A 的源码压缩包

表 6.1.1.2 tf-a 源码文件含义

6.1.2 TF-A 源码打补丁

图 6.1.1.5 中的 TF-A 源码还不能直接使用，需要对其打补丁。前面我们说了，ST 也是在 TF-A 官方的源码上将自己的 MP1 系列芯片添加进去的，因此必然涉及到对官方 TF-A 源码的修改，修改的部分就是“补丁”，补丁文件后缀为.patch，表 6.1.1.2 中的 0001-st-update-v2.2-r2.0.0.patch 就是 ST 为 TF-A 官方源码做的补丁文件，补丁文件里面描述了修改源码中的哪些文件，应该添加或删除哪些代码或文件，打完补丁后的 TF-A 才是我们真正需要的。因此，打补丁需要两个文件：

①、补丁文件，在此处就是 0001-st-update-v2.2-r2.0.0.patch。

②、需要打补丁的源文件，在此处就是 tf-a-stm32mp-2.2.r1-r0.tar.gz。

我们需要先解压 tf-a-stm32mp-2.2.r1-r0.tar.gz，如图如下命令：

```
tar -vxf tf-a-stm32mp-2.2.r1-r0.tar.gz
```

解压完就是会在当前目录下生成“tf-a-stm32mp-2.2.r1”目录，如图 6.1.2.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl
-linux-gnueabi/tf-a-stm32mp-2.2.r1-r0$ ls -l
总用量 5068
-rw-r--r--  1 zuozhongkai zuozhongkai 1281142 6月  17 20:16 0001-st-update-v2.2-r2.0.0.patch
-rw-r--r--  1 zuozhongkai zuozhongkai    3921 6月  17 20:16 Makefile.sdk
-rw-r--r--  1 zuozhongkai zuozhongkai    4638 6月  17 20:16 README.HOW_TO.txt
-rw-r--r--  1 zuozhongkai zuozhongkai      37 6月  17 20:16 series
drwxr-xr-x 17 zuozhongkai zuozhongkai    4096 6月  12 00:28 tf-a-stm32mp-2.2.r1
-rw-r--r--  1 zuozhongkai zuozhongkai 3886266 6月  17 20:16 tf-a-stm32mp-2.2.r1-r0.tar.gz
```

图 6.1.2.1 解压 TF-A 的官方源码

接下来就是给 TF-A 源码打补丁，在解压好的 TF-A 的目录下运行以下命令：

```
cd tf-a-stm32mp-2.2.r1                                    //进入 TF-A 源码
```

```
for p in `ls -1 ../*.patch`; do patch -p1 < $p; done          //打补丁
```

上面这条命令的意思是把上一层目录下的所有“.patch”后缀的文件都通过 patch 命令打补丁到 TF-A 的源码目录，在这里就是将 001-st-update-v2.2-r2.0.0.patch 这个补丁打入到 TF-A 源码里面，过程如图 6.1.2.2 所示：

```
patching file plat/st/stm32mp1/stm32mp1_power_config.c
patching file plat/st/stm32mp1/stm32mp1_private.c
patching file plat/st/stm32mp1/stm32mp1_scmi.c
patching file plat/st/stm32mp1/stm32mp1_security.c
patching file plat/st/stm32mp1/stm32mp1_shared_resources.c
patching file plat/st/stm32mp1/stm32mp1_syscfg.c
patching file plat/st/stm32mp1/stm32mp1_usb_desc.c
patching file tools/stm32image/stm32image.c
```

图 6.1.2.2 TF-A 的源码打补丁

由于 ST 官方源码目录太长，为了方便调试，在 atk-mp1 目录下创建一个名为“tf-a”的子目录，然后就打完补丁后的 tf-a-stm32mp-2.2.r1-r0 目录下的所有文件都拷贝到 tf-a 下，命令如下：

```
cd tf-a-stm32mp-2.2.r1-r0/
cp * /home/zuozhongkai/linux/atk-mp1/tf-a/ -rf
```

拷贝完成以后的 tf-a 目录如图 6.1.2.3 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a$ ls -l
总用量 5068
-rw-r--r--  1 zuozhongkai zuozhongkai 1281142 11月  3 17:17 0001-st-update-v2.2-r2.0.0.patch
-rw-r--r--  1 zuozhongkai zuozhongkai    3921 11月  3 17:17 Makefile.sdk
-rw-r--r--  1 zuozhongkai zuozhongkai    4638 11月  3 17:17 README.HOW_TO.txt
-rw-r--r--  1 zuozhongkai zuozhongkai      37 11月  3 17:17 series
drwxr-xr-x 17 zuozhongkai zuozhongkai    4096 11月  3 17:17 tf-a-stm32mp-2.2.r1
-rw-r--r--  1 zuozhongkai zuozhongkai 3886266 11月  3 17:17 tf-a-stm32mp-2.2.r1-r0.tar.gz
```

图 6.1.2.3 tf-a 目录

图 6.1.2.3 中的 tf-a-stm32mp-2.2.r1 就是我们后面要使用的，打完补丁后的 TF-A 源码。

6.1.3 创建 VSCode 工程

为了方便阅读 TF-A 源码，我们可以创建一个 VSCode 工程，然后打开 VSCode，点击：文件->打开文件夹，选中图 6.1.2.3 中打完补丁后的 tf-a-stm32mp-2.2.r1-r0 文件夹，打开以后的 VSCode 如图 6.1.3.1 所示：

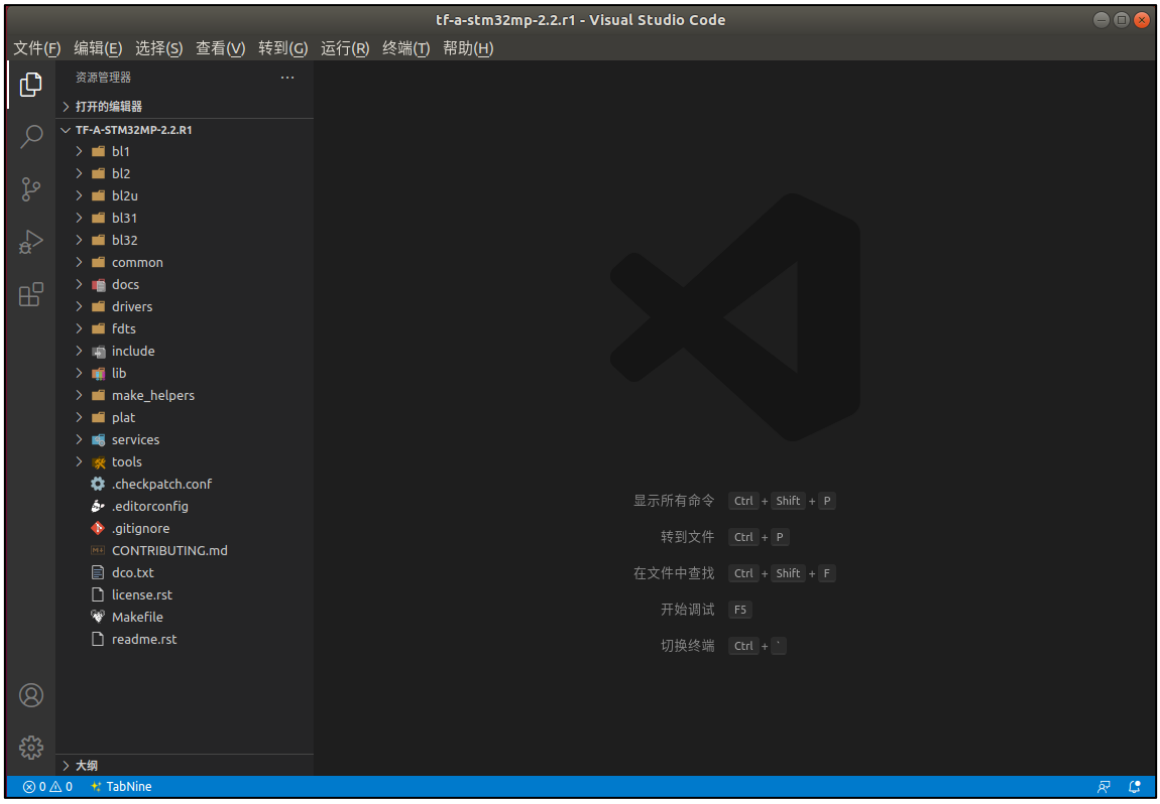


图 6.1.3.1 TF-A 的 VSCode 工程

图 6.1.3.1 中左侧的资源管理器就是 TF-A 的工程目录结构。最后我们保存一下工作区，点击：文件->将工作区另存为…”，打开保存工作区对话框，将工作区保存到 TF-A 源码根目录下，设置文件名为“tf-a”，如图 6.1.3.2 所示：

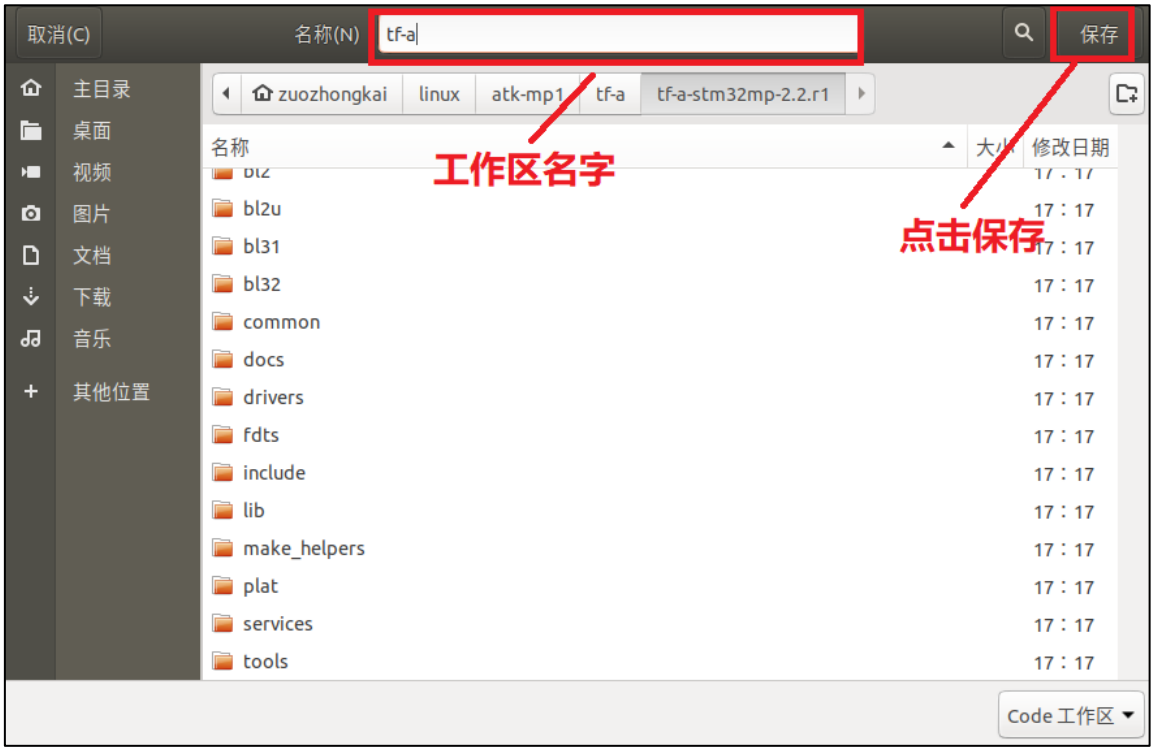


图 6.1.3.2 TF-A 源码工作区创建

保存以后就会在 TF-A 源码根目录下多出一个名为“tf-a.code-workspace”的文件，这样一个完整的 VSCode 工程就建立起来了。

6.2 编译和烧录 TF-A

6.1 小节我们得到的 TF-A 源码是 ST 提供的，这个 TF-A 肯定是不能直接在正点原子的 STM32MP157 开发板上运行的，需要移植和修改。但是在正式移植 TF-A 之前我们要先体验一下如何编译 TF-A，并烧写到正点原子的 STM32MP157 开发板上，这里我们直接编译正点原子已经修改好的 TF-A。

6.2.1 stm32wrapper4dbg 工具安装

我们在编译 TF-A 或者 Uboot 的时候需要用到 stm32wrapper4dbg 这个工具，否则编译会报错。ST 提供了这个工具的源码，我们需要在 Ubuntu 下编译并安装这个源码，源码的下载地址为：https://github.com/STMicroelectronics/stm32wrapper4dbg，这个我们已经下载下来并放到了开发板光盘中，路径为：开发板光盘→5、开发工具→ stm32wrapper4dbg-master.zip。将源码压缩包拷贝到 Ubuntu 下，然后进行解压，命令如下：

```
unzip  stm32wrapper4dbg-master.zip        //解压
```

解压完成以后就会得到一个名为“stm32wrapper4dbg-master”的文件夹，进入到此文件夹里面，然后编译并安装，命令如下：

```
cd  stm32wrapper4dbg-master               //进入到此文件夹
make                                      //编译
```

编译完成以后就会得到一个名为“stm32wrapper4dbg”的工具，如图 6.2.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/tool/stm32wrapper4dbg-master$ ls
Android.bp            LICENSE-GPL2      stm32wrapper4dbg.1
CONTRIBUTING.md       Makefile          stm32wrapper4dbg.c
COPYING               README.md         wrapper_stm32mp15x.inc
LICENSE-BSD-3-CLAUSE  stm32wrapper4dbg  wrapper_stm32mp15x.S
```

图 6.2.1.1 编译得到的工具

图 6.2.1.1 中的“stm32wrapper4dbg”就是我们要的工具，将图 6.2.1.1 中编译出来的 stm32wrapper4dbg 工具拷贝到 Ubuntu 的/usr/bin 目录下，命令如下：

```
sudo cp stm32wrapper4dbg /usr/bin
```

拷贝完成以后就可以直接在终端中使用 stm32wrapper4dbg 这个工具了，输入如下命令查看帮助信息：

```
stm32wrapper4dbg -s
```

如果输出如图 6.2.1.2 所示内容就说明 stm32wrapper4dbg 工具安装成功：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a$ stm32wrapper4dbg -s
stm32wrapper4dbg: option requires an argument -- 's'
Usage: stm32wrapper4dbg -s srcfile -d destfile [-b] [-f]
  Add a debug wrapper to a stm32 image.
  If "-b" is not specified, the wrapper would be placed
  after the last byte of the image.

Options:
  -s srcfile   input image in stm32 file format
  -d destfile  output image in stm32 file format
  -b           place the wrapper before the image
  -f           force re-adding the wrapper
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a$
```

图 6.2.1.2 stm32wrapper4dbg 帮助信息

6.2.2 编译正点原子官方的 TF-A

1、准备正点原子出厂 TF-A 源码

首先安装设备树编译相关命令，输入如下命令：

```
sudo apt-get install device-tree-compiler
```

在 Ubuntu 的 atk-mp1 目录下新建一个名为“alientek_tf-a”的子目录，然后把正点原子修改好 TF-A 源码拷贝到 alientek_tf-a 这个目录下。正点原子修改好的 TF-A 源码已经放到了开发板光盘中，路径为：开发板光盘→1、程序源码→1、正点原子 Linux 出厂系统源码→tf-a-stm32mp-2.2.r1-g463d4d8-v1.0.tar.bz2。拷贝完成以后解压，输入如下命令：

```
tar -xvf tf-a-stm32mp-2.2.r1-g463d4d8-v1.0.tar.bz2
```

解压完成以后 alientek_tf-a 文件夹内容如图 6.2.1.1 示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a$ ls
Makefile.sdk  tf-a-stm32mp-2.2.r1  tf-a-stm32mp-2.2.r1-g463d4d8-v1.0.tar.bz2
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a$
```

图 6.2.1.1 解压好的 TF-A

图 6.2.1.1 中的‘tf-a-stm32mp-2.2.r1’就是正点原子针对自己的 STM32MP157 开发板修改过的 TF-A 源码，Makefile.sdk 是一会编译 TF-A 要用到的 Makefile。

2、修改 Makefile.sdk

首先我们要知道，TF-A 是有自己的 Makefile 文件的，而且真正编译的时候也是要用 TF-A 自己的 Makefile。Makefile.sdk 是 ST 自己编写的，也是我们一会编译 TF-A 的时候要用到的，Makefile.sdk 里面主要定义了一些编译属性，比如要使用的交叉编译器、编译的一些选项等等，Makefile.sdk 最终会调用 TF-A 内部的 Makefile 来编译 TF-A。默认情况下 Makefile.sdk 里面使用的是 ST 官方的交叉编译器(arm-ostl-linux-gnueabi-gcc)，但是我们本教程用的是通用交叉编译器 arm-none-linux-gnueabihf-gcc，因此我们需要 Makefile.sdk，将交叉编译器改为我们目前所使用的。打开 Makefile.sdk，然后找到 CROSS_COMPILE，将其改为“arm-none-linux-gnueabihf-”，如图 6.2.1.2 所示：

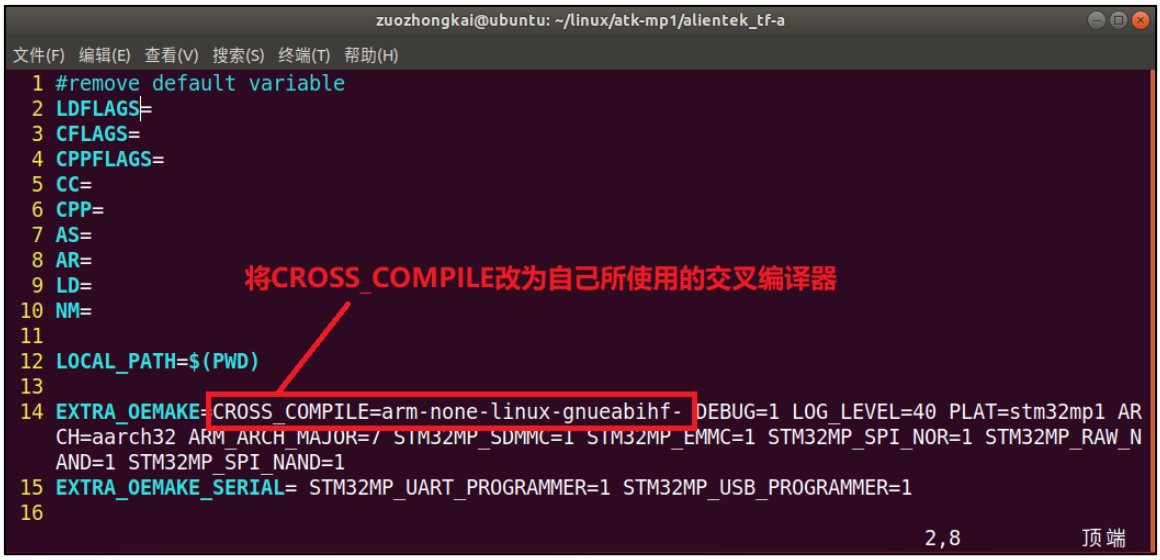


图 6.2.1.2 修改后的 Makefile.sdk

3、编译 TF-A

准备工作都就绪以后就可以编译正点原子出厂 TF-A 了，进入到图 6.2.1.1 中的 tf-a-stm32mp-2.2.r1 目录里面，然后运行执行如下命令编译 TF-A:

```
cd tf-a-stm32mp-2.2.r1/                    //进入到正点原子出厂 TF-A 的源码目录
make -f ../Makefile.sdk all                //编译 TF-A
```

‘-f’的意思是重新指定 Makefile，在这里就是指定图 6.2.1.1 中的 Makefile.sdk，编译成功会出现如图 6.2.1.3 所示：

```
zuozhongkai@ubuntu: ~/linux/atk-mp1/alientek_tf-a/tf-a-stm32mp-2.2.r1
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
Image Type   : ST Microelectronics STM32 V1.0
Image Size   : 241728 bytes
Image Load   : 0x2ffc2500
Entry Point  : 0x2ffd6000
Checksum     : 0x00e3698d
Option      : 0x00000001
Version     : 0x00000000

Building stm32mp1
make[1]: 离开目录“/home/zuozhongkai/linux/atk-mp1/alientek_tf-a/tf-a-stm32mp-2.2.r1”
Image Type   : ST Microelectronics STM32 V1.0
Image Size   : 241784 bytes
Image Load   : 0x2ffc2500
Entry Point  : 0x2fffd541
Checksum     : 0x00e38028
Option       : 0x00000001
Version      : 0x00000000
Halt Address : 0x2ffd6000
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a/tf-a-stm32mp-2.2.r1$
```

图 6.2.1.3 TF-A 编译成功

编译完成以后会在上一层目录，也就是 alientek_tf-a 下生成一个名为“build”的目录，如图 6.2.1.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a$ ls -l
总用量 3424
drwxr-xr-x  5 zuozhongkai zuozhongkai    4096 11月  4 12:11 build
-rw-r--r--  1 zuozhongkai zuozhongkai    3938 11月  4 11:13 Makefile.sdk
drwxr-xr-x 18 zuozhongkai zuozhongkai    4096 11月  3 22:22 tf-a-stm32mp-2.2.r1
-rw-------  1 zuozhongkai zuozhongkai 3489905 11月  3 22:17 tf-a-stm32mp-2.2.r1-g463d4d8-v1
.0.tar.bz2
```

编译结果存放在build里面

图 6.2.1.4 build 目录

进入到图 6.2.1.4 中的 build 目录下，一共有三个子目录：optee、serialboot 和 trusted，如图 6.2.1.5 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a/build$ ls
optee  serialboot  trusted
zuozhongkai@ubuntu:~/linux/atk-mp1/alientek_tf-a/build$
```

图 6.2.1.5 build 子目录

我们只关注 trusted 目录下的文件，此目录下就保存了 MP1 所有型号的 TF-A 固件，包括正点原子开发板所使用的 tf-a-stm32mp157d-atk-trusted.stm32，如图 6.2.1.6 所示：

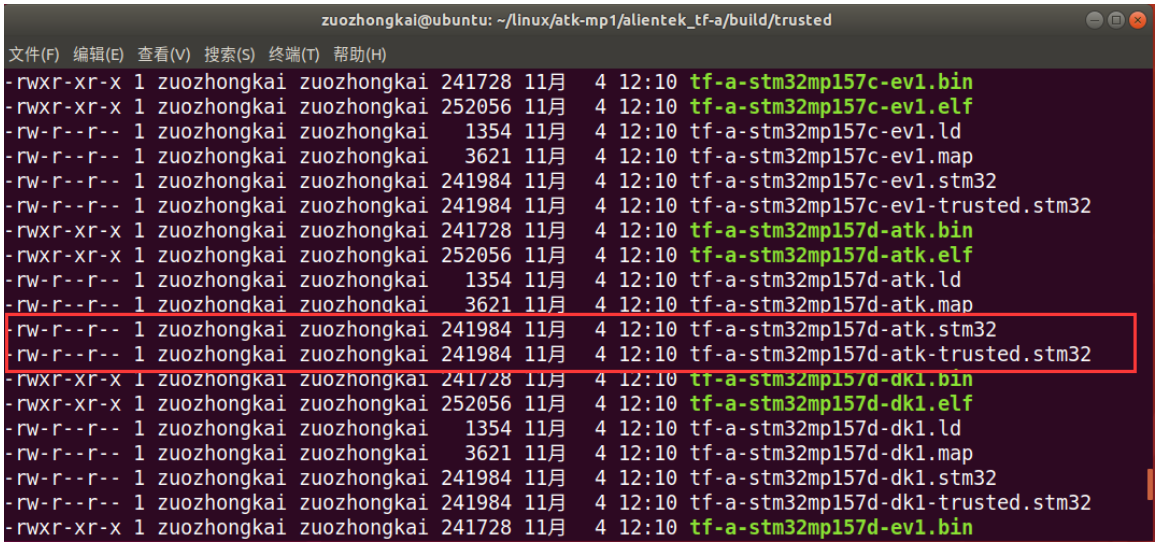


图 6.2.1.6 编译得到的正点原子 157 开发板所使用的 TF-A 固件

6.2.3 TF-A 烧录到 EMMC

使用 STM32CubeProgrammer 将 TF-A 烧写到开发板里面，STM32CubeProgrammer 支持通过 UART、USB、STLINK 来烧写系统，本教程全部采用 USB 烧写，也就是通过开发板上的 USB_OTG 口来烧写系统。

1、准备烧写的材料

我们在 Windows 下通过 STM32CubeProgrammer 来烧写 TF-A，新建一个目录来存放烧写镜像文件，比如我这里创建一个名为“images”的目录。然后将开发板光盘里面的这两个文件先拷贝到 images 目录下：

①、tf-a-stm32mp157d-atk-serialboot.stm32，由正点原子提供。路径：开发板光盘→8、系统镜像→2、出厂系统镜像→ 1、STM32CubeProg 烧录固件包→tf-a→ tf-a-stm32mp157d-atk-serialboot.stm32。

②、u-boot.stm32，由正点原子提供。路径：开发板光盘→8、系统镜像→2、出厂系统镜像→ 1、STM32CubeProg 烧录固件包→uboot→u-boot.stm32 拷贝到 images 文件夹里。

拷贝完成以后 images 如图 6.2.3.1 所示：

图 6.2.3.1 拷贝文件到 images

最后就是前面编译出来，真正要烧写的 tf-a-stm32mp157d-atk-trusted.stm32，通过 FileZilla 将 Ubuntu 里面的 tf-a-stm32mp157d-atk-trusted.stm32 发送到 images 目录下，完成以后结果如图 6.2.3.2 所示：

图 6.2.3.2 最终的材料

大家可能会有疑问，tf-a-stm32mp157d-atk-trusted.stm32 是我们编译出来的 TF-A，这个没啥说的，但是，另外两个是干啥的呢？这个就和 STM32CubeProgrammer 这个软件的设计以及烧写过程有关了。以下内容为作者实际测试研究发现的，ST 并没有给出真正的解释，可能存在错误：

tf-a-stm32mp157d-atk-serialboot.stm32 中间有个“serialboot”，也就是串行 BOOT，说明是和启动有关的。此固件用来初始化 USB、DDR 等外设，DDR 初始化了以后就可以运行 uboot 了，为什么要运行 uboot 呢？因为 uboot 里面会初始化 EMMC、NAND 等外设，而且 uboot 会提供很强大的 EMMC 操作指令。也就是说，启动 uboot 的目的就是为了操作 EMMC、NAND，这样就可以在 uboot 里面通过相关的命令将 tf-a-stm32mp157d-atk-trusted.stm32 写到 EMMC 或者 NAND 里面。这就是需要 tf-a-stm32mp157d-atk-serialboot.stm32 和 u-boot.stm32 的原因，这两个文件存在的目的就是为了将 tf-a-stm32mp157d-atk-trusted.stm32 烧写到 EMMC、NAND、SD 卡里面。

2、准备 FlashLayout

图 6.2.3.2 中有三个文件，STM32CubeProgrammer 如何使用这三个文件，应该放到哪个地方，如果烧写的话要烧写到 EMMC 的哪个地址区域。这些都要通过 STM32CubeProgrammer 脚本文件来定义，STM32CubeProgrammer 脚本文件后缀为.tsv，ST 官方也叫做 FlashLayout。这里我们直接在正点原子提供的 tsv 文件基础上修改，将开发板光盘→8、系统镜像→2、出厂系统镜像→1、STM32CubeProg 烧录固件包→flashlayout→atk_emmc-stm32mp157d-atk-qt.tsv 拷贝到前面的 images 前目录下，然后将其重命名为“tf-a.tsv”，完成以后 images 目录如图 6.2.3.3 所示：

名称	修改日期	类型	大小
tf-a.tsv	2020-10-30 15:51	TSV 文件	1 KB
tf-a-stm32mp157d-atk-serialboot.stm32	2020-10-19 16:14	STM32 文件	237 KB
tf-a-stm32mp157d-atk-trusted.stm32	2020-11-04 16:30	STM32 文件	237 KB
u-boot.stm32	2020-10-19 16:14	STM32 文件	846 KB

图 6.2.3.3 最终的 images 目录

.tsv 是文本格式的，很容易阅读，关于.tsv 语法的详细讲解，请参考：https://wiki.st.com/stm32mpu/wiki/STM32CubeProgrammer_flashlayout。用 Notepad++软件(没有的话自行安装，这是个免费软件，安装很简单)打开图 6.2.3.3 中的 tf-a.tsv，打开以后默认如图 6.2.3.4 所示：

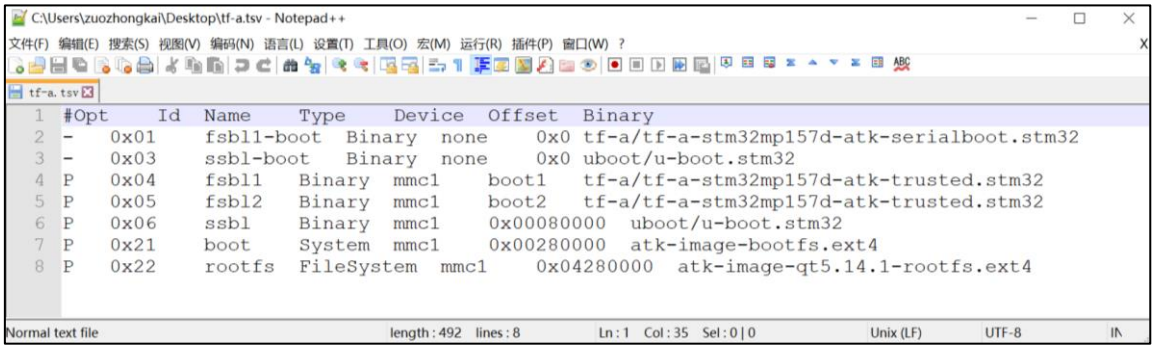


图 6.2.3.4 tf-a.tsv 内容

tsv 对格式有要求，直接看图 6.2.3.4 是看不出来啥的，必须要设置一下 Notepad++软件，点击视图→显示符号→显示空格与制表符，设置好以后的 tf-a.tsv 如图 6.2.3.5 所示：

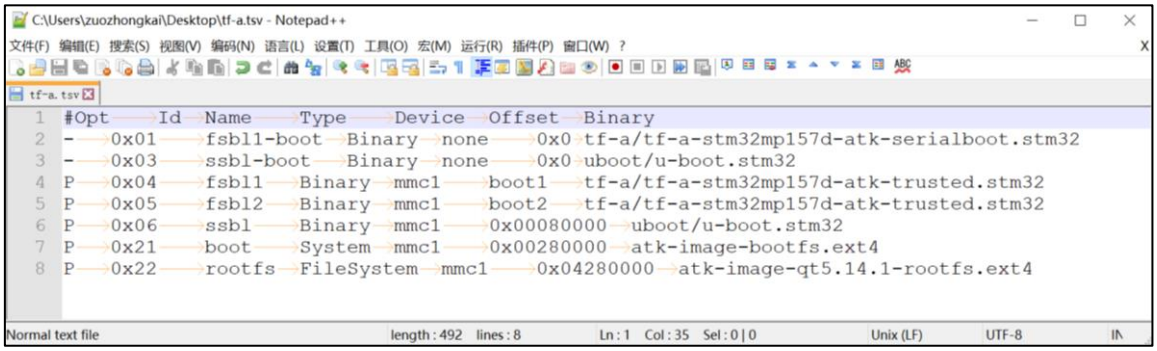


图 6.2.3.5 显示空格与制表符

从图 6.2.3.5 中可以看出，tf-a.tsv 文件就会发现多了黄色的箭头，这些箭头代表是 TAB 键，TAB 键越多黄色箭头就越长，如果用空格键的话就会显示‘.’。tsv 语法要求只能用 TAB 键，不能用空格！以‘#’开头为注释，所以图 6.2.3.5 中第一行为注释。

我们把 tf-a.tsv 文件内容修改成如图 6.2.3.6 所示：

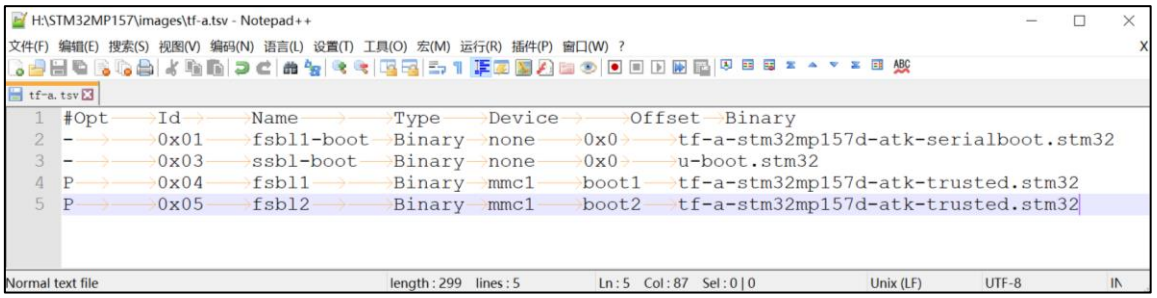


图 6.2.3.6 修改后的 tf-a.tsv 文件

图 6.2.3.6 中一共有 5 行，第 1 行为注释，一共 7 列，这 7 列的含义如表 6.2.3.1 所示：

域	作用
Opt	选项字段，可以设置为“-”、“P”、“D”或“E”
Id	会根据这个 id 来决定烧写分区
Name	分区名字
Type	制定烧写的类型，仅 uboot 使用。
Device	指定烧写的设备类型与编号，比如 emmc0、emmc1、nand0 等，如果 opt 为‘-’,那么此字段就为 none
Offset	分区的起始位置，如果为“boot1”表示 EMMC 的第一个分区，如果为“boot2”就表示 EMMC 第二个分区。如果是数字就表示需要偏移的字节数。
Binary	要烧录的文件

表 6.2.3.1 tf-a.tsv 的配置解析

接下来详细讲解一下表 6.2.3.1 中的这 7 个配置：

①、Opt 域

Opt 是第一个项，此选项通过‘-’、‘P’、‘D’和‘E’这四个字符定义操作方法，首选的是‘-’和‘P’。

- ‘-’：none，也就是空选项，分区或者设备无需修改，如果 Device 域为 none，那么 Opt 强制为‘-’。

- **‘P’**：向分区或者设备烧写固件。

STM32CubeProgrammer 本质是通过 uboot 来烧写系统的，也就是先把 uboot 加载到板子的 DDR 里面并运行，然后使用 uboot 来烧写系统。uboot 会请求需要烧写的二进制文件，然后将其烧写到指定的分区或者 Falsh 设备里面。

针对‘P’选项，还有另外两个可以搭配使用的小伙伴：

- **‘E’**：空分区或设备，表示对应的分区或设备不更新，相关的 Id 项会被跳过。
- **‘D’**：删除分区或设备。

允许的组合选项如下所示：

- **‘-’**：空选型。
- **‘P’**：更新分区或设备，也就是向分区或设备烧写固件。
- **‘PE’**：不更新，也就是指定某个分区或者设备不需要烧写固件，这样我们就可以单独只更新 tf-a、uboot、kernel 或者 rootfs。
- **‘PD’**：删除并更新，也可以写作 DP。
- **‘PDE’**：删除并且保持为空，也可以写作 PED/DPE/DEP/EPD/EDP。

②、Id 域

STM32CubeProgrammer 通过 Id 域来确定烧写方法，会通过 Id 域来识别下一个要烧写到设备里面的二进制文件：

- ROM 或者 FSBL：二进制文件要加载到 RAM 中
- SSBL(uboot)：二进制文件要烧写到 Flash 中。

FlashLayout 支持的 Id 范围如表 6.2.3.2 所示：

范围	分区
0x01~0x0F	带有 STM32 头部信息的 Boot 分区，如 SSBL、FSBL、其他(TEE 或 M4 固件)
0x10~0xF0	不带头部的用户编程分区，如 uimge、dtb、rootfs、vendorfs、userfs

表 6.2.3.2 FlashLayout 支持的 Id 范围

其中 0X01 和 0X03 这两个 ID 是给 FSBL 和 SSBL 留着的，它们会被加载到 RAM 中。一些默认的 Id 含义如表 6.2.3.3 所示：

Id	分区
0x00	内部使用，用户不能使用此 ID！
0x01	FSBL(第一阶段拷贝)，ROM 代码使用，会加载到 RAM 中
0x03	SSBL，FSBL 使用，加载到 RAM 中
0xF1~0xFD	虚拟分区，内部使用
0xF1	命令获取阶段。
0xF2	OTP
0xF3	SSP
0xF4	PMIC NVM
0xFE	操作结束
0xFF	复位

表 6.2.3.3 默认 Id 含义

③、Name 域

Name 域为一段字符串，也就是目标内存段的名字。

④、Type 域

Type 域仅仅用于 uboot，用来选择需要更新的 Flash 区域：

- SD 卡或者 EMMC 设备对应 GPT 分区。
- 原始的 Flash 设备，如 NAND、NOR 等对应 MTD 分区。

SD/EMMC 和 NAND/NOR 所支持的 Type 类型如表 6.2.3.4 所示：

Type	GPT		MTD		
	SD	EMMC	NAND	NOR	RAM
Binary	×	×	×	×	×
Binary(N)			ssbl		
FileSystem	×	×	×	×	dtb
System	×	×	UBI	UBI	kernel
Rawimage	×	user data	×	×	

表 6.2.3.4 Type 域

注：×表示支持。

正点原子 STM32MP157 开发板为 EMMC 类型，所以我们只需要知道 EMMC 下的 Type 域含义：

- **Binary**：原始的二进制文件。
- **FileSystem**：linux 文件系统，为 ext2/ext4/fat 格式。
- **System**：Linux 内核。

⑤、Device 域

Device 域指定 Uboot 设备树定义的设备和索引(从 0 开始)，不同的设备其设备名字和索引不同：

- **mmc+索引**：如 mmc0、mmc1、mmc2 等，对应 SD 卡或 EMMC。比如 SD 卡和 EMMC 分别接到 MP1 的 SDMMC1 和 SDMMC2 接口上，那么 SD 卡和 EMMC 分别为 mmc0 和 mmc1。
- **nor+索引**：如 nor0，对应 NOR 或者 QUADSPI Flash。
- **mmc+索引**：如 nand0，对应连接到 FMC 总线上的并行 NAND Flash。
- **spi-nand+索引**：如 spi-nand0，对应连接到 QSPI 上的串行 NAND Flash。
- **none**：RAM，也就是将固件加载到 RAM 里面，仅允许启动阶段使用，而且 Type 域要为 Binary，Offset 域要为 0，Opt 域为‘-’。
- **ram+索引**：如 ram0，烧写服务讲固件加载到 RAM 中运行。

⑥、Offset 域

Offset 就是偏移，支持的值如下：

- **boot1**：EMMC 的第一个启动区域分区。
- **boot2**：EMMC 的第二个启动区域分区。
- **数字**：具体的偏移值，单位为字节。

⑦、Binary 域

STM32CubeProgrammer 软件要使用的二进制文件。

关于 STM32CubeProgrammer 的 FlashLayout 文件语法就讲解到这里，接下来就是将 TF-A 烧写到开发板的 EMMC 里面。

3、通过 USB 烧写 TF-A

首先设置开发板拨码开关，设置为 000，也就是从 USB 启动，然后复位开发板！

一切准备就绪，现在可以通过 STM32CubeProgrammer 将 TF-A 源码烧写到 EMMC 里面了，

首先通过 USB Type-C 线将开发板的 USB_OTG 和 USB_TTL 连接到电脑上，如图 6.2.3.7 所示：

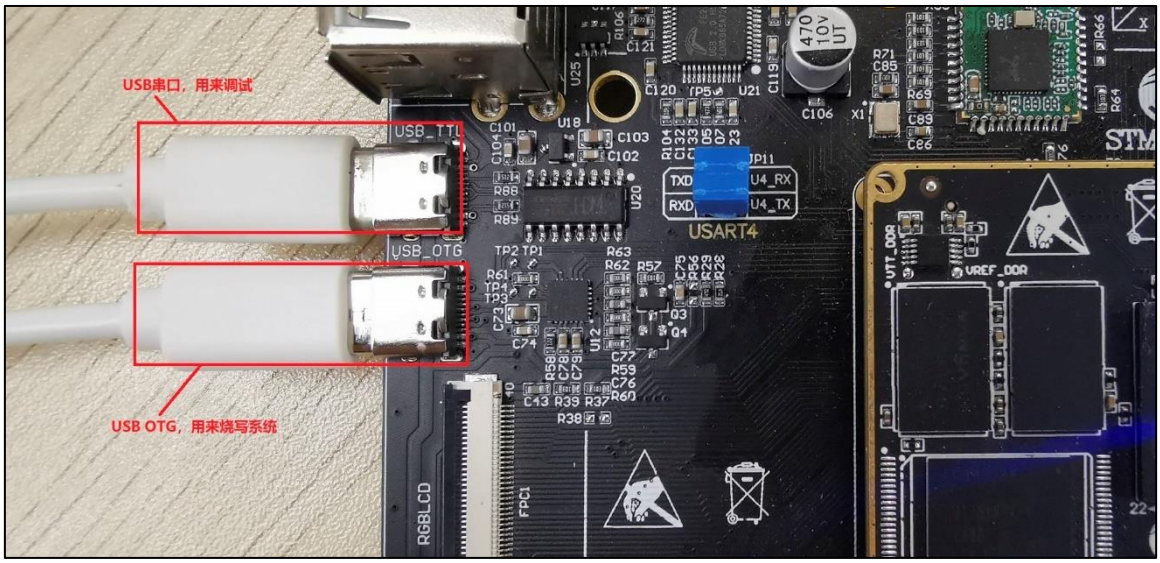


图 6.2.3.7 USB 串口与 USB OTG 连接

按照图 6.2.3.7 将板子的 USB 串口与 USB OTG 连接到电脑以后还要检查一下连接是否成功，打开电脑的设备管理器，查看 CH340 和 DFU 模式是否存在，如图 6.2.3.8 所示：

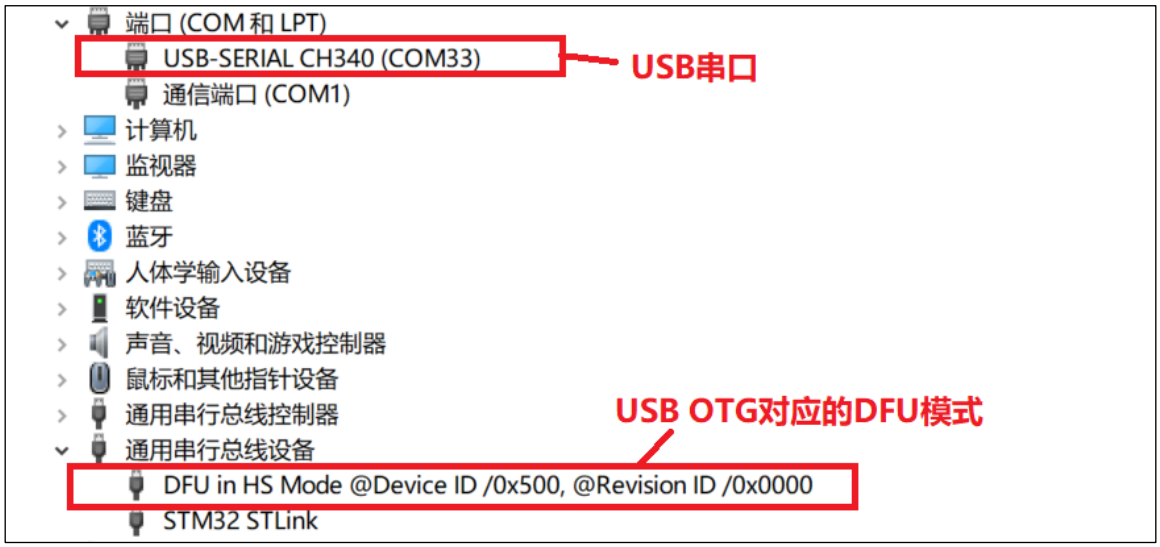


图 6.2.3.8 USB 串口与 USB OTG

连接没问题的话就可以使用 STM32CubeProgrammer 来烧写 TF-A 了，打开 STM32CubeProgrammer，选择 USB 连接方式，Port 选择 USB1，如图 6.2.3.9 所示：

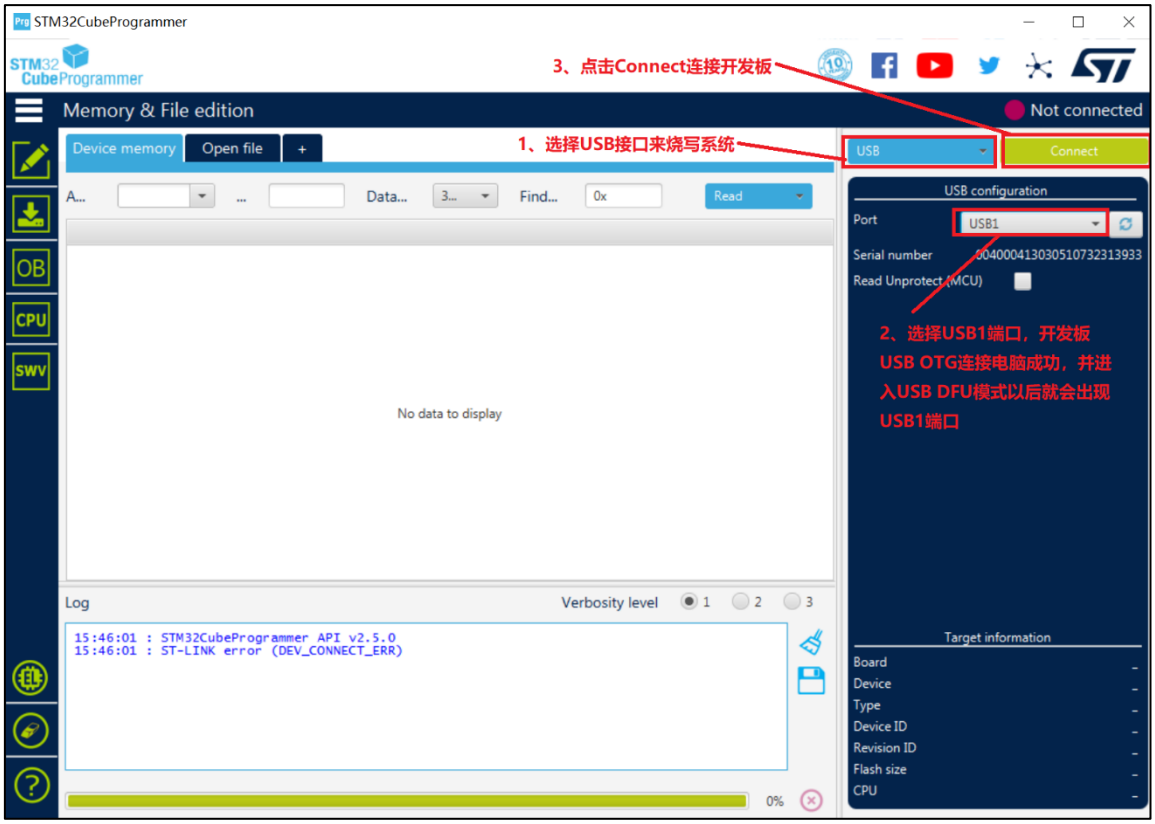


图 6.2.3.9 USB 连接设置

USB 设置好以后点击图 6.2.3.9 右上角的“Connect”来连接开发板，连接成功以后左下角的 log 区域就会输出一些信息，右侧中间的数据区域也会显示开发板默认的分区情况，右下角会显示目标板信息，如图 6.2.3.10 所示：

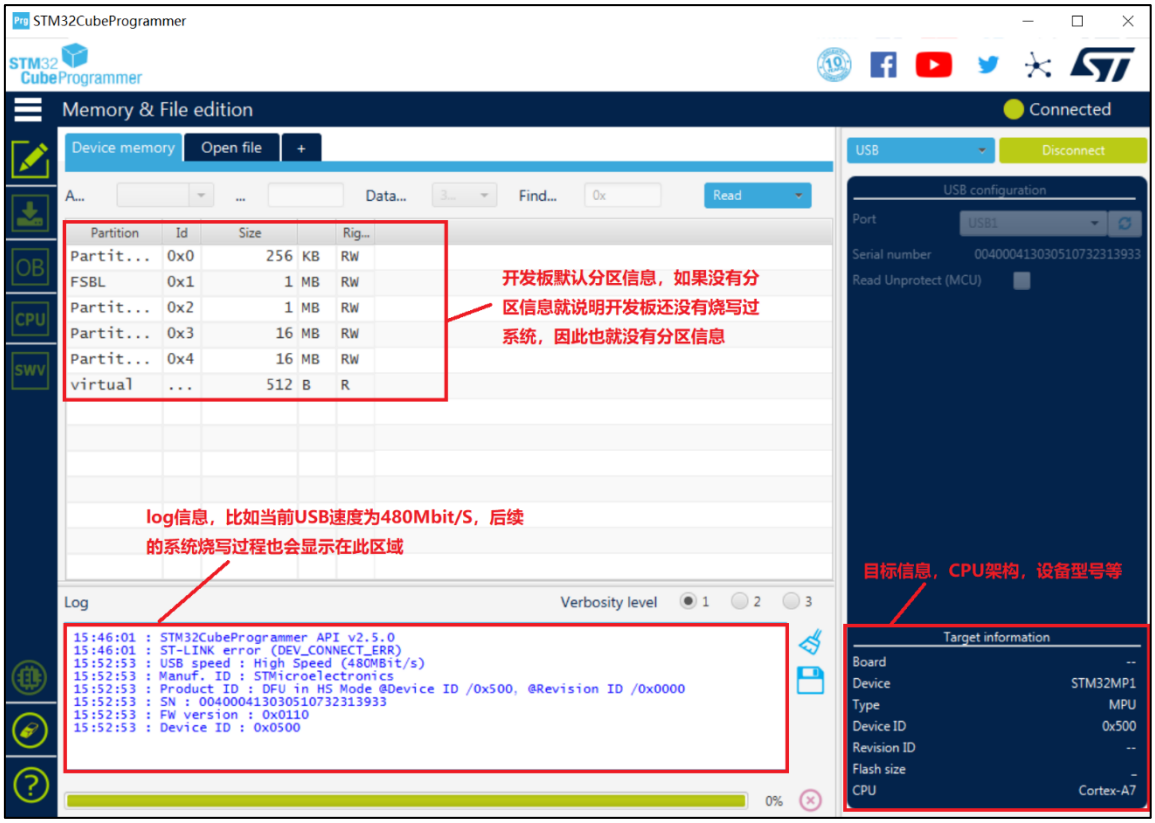


图 6.2.3.10 软件连接信息

STM32CubeProgrammer 要使用 FlashLayout 文件来烧写系统，也就是前面我们创建的 tf-a.tsv。点击图 6.2.3.10 中的“Open File”，打开 tf-a.tsv，打开以后如图 6.2.3.11 所示：

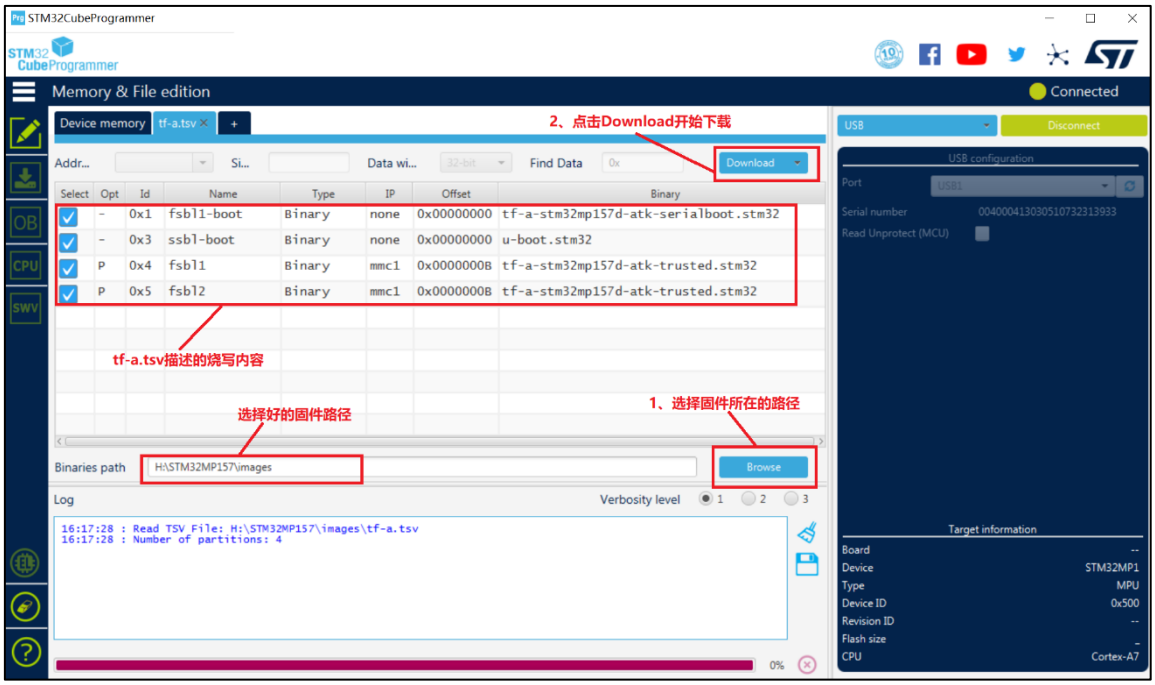


图 6.2.3.11 加载 tf-a.tsv 文件

注意，如果大家没有看到图 6.2.3.11 中的“Browse”和“Download”这两个按钮的话，请把 STM32CubeProgrammer 界面托大一点，尤其是左右托大，因为软件默认界面会把这两个按

钮挡住！

一切准备就绪以后就可以点击图 6.2.3.11 中的“Download”按钮开始下载，下方的 log 区域就会显示烧写过程，烧写完成以后就会有如图 6.2.3.12 所示提示：

图 6.2.3.12 烧写成功

6.2.4 TF-A 的运行

我们自己编译的 TF-A 已经烧写到了开发板中，接下来就是测试一下看能不能运行，打开 MobaXterm 软件，设置好与开发板连接的串口，波特率选择 115200。

设置开发板拨码开关为 010，也就是从 EMMC 启动，然后复位开发板！

注意，由于我们开发板默认已经烧写了整套 Linux 系统，所以直接启动的话会启动整个系统，包括我们刚刚烧写的自己编译的 TF-A，原有的 uboot、Linux 系统等。TF-A 是最先启动的，也就是最前的就是我们自己编译的 TF-A，如图 6.2.4.1 所示：

```
6. COM33 (USB-SERIAL CH340 (CO
NOTICE:  CPU: STM32MP157DAA Rev.Z
NOTICE:  Model: STMicroelectronics STM32MP157D eval daughter
INFO:    Reset reason (0x14):
INFO:      Pad Reset from NRST
INFO:    Using EMMC
INFO:      Instance 2
INFO:    Boot used partition fsbl1
NOTICE:  BL2: v2.2-r1.0(debug):
NOTICE:  BL2: Built : 10:53:02, Nov 11 2020
INFO:    Using crypto library 'stm32_crypto_lib'
INFO:    BL2: Doing platform setup
INFO:    RAM: DDR3-DDR3L 32bits 533000Khz
INFO:    Memory size = 0x40000000 (1024 MB)
INFO:    BL2 runs SP_MIN setup
INFO:    BL2: Loading image id 4
INFO:    Loading image id=4 at address 0x2ffed000
INFO:    Image id=4 loaded: 0x2ffed000 - 0x2ffff000
INFO:    BL2: Loading image id 5
INFO:    Loading image id=5 at address 0xc0100000
INFO:    STM32 Image size : 865868
INFO:    Image id=5 loaded: 0xc0100000 - 0xc01d364c
WARNING: Skip signature check (header option)
NOTICE:  ROTPK is not deployed on platform. Skipping ROTPK verification.
NOTICE:  BL2: Booting BL32
INFO:    Entry point address = 0x2ffed000
INFO:    SPSR = 0x1d3
INFO:    Cannot find st,stpmic1 node in DT
NOTICE:  SP_MIN: v2.2-r1.0(debug):
NOTICE:  SP_MIN: Built : 10:53:03, Nov 11 2020
INFO:    ARM GICv2 driver initialized
INFO:    stm32mp IWDG1 (12): Secure
INFO:    ETZPC: CRYP1 (9) could be non secure
INFO:    SP_MIN: Initializing runtime services
INFO:    SP_MIN: Preparing exit to normal world
```

图 6.2.4.1 TF-A 启动 log 信息

TF-A 启动的时候会打印出编译时间，我们可以通过编译时间来判断是否为我们自己编译的 TF-A。比如图 6.2.4.1 中 TF-A 的编译时间为 2020 年 11 月 11 号 10:53:02，这个时间正是笔者写教程编译的时间，说明目前正在运行的 TF-A 就是笔者自行编译的正点原子出厂 TF-A，也就是 tf-a-stm32mp157d-atk-trusted.stm32。

第七章 TF-A 初探

前面我们简单了解了一下什么是 TF-A，并且也学习了如何编译 TF-A。但是 TF-A 是如何运行的，它的一个运行流程是怎样的我们并没有讲解。TF-A 的详细运行过程是很复杂的，涉及到很多 ARM 处理器底层知识，本章我们主要讲解一下 TF-A 的整个框架，了解一下 TF-A 运行的大致流程。

7.1 设备安全怎么保证?

手机,已经成为人们不可缺少的一部分,相信大部分朋友出门要是忘带手机就会浑身难受,手机给我们带来的便利性自不用多说,娱乐、导航、音乐这些都是最基本的功能。移动支付这一重大创新,颠覆了我们的生活习惯,再也不需要外出的时候携带现金,只需要一部手机即可。我们在享受手机带来的便利的同时,手机安全问题就尤为重要,毕竟手机里面存放着我们的指纹、面部信息、各种银行卡和支付 APP 的密码。

另外,物联网行业现在也蓬勃发展,各种 IOT 设备层出不穷,比如各种智能插座、灯泡、摄像头等。这些 IOT 设备在给我们的生活带来便利性的同时,也为我们的隐私安全埋下了隐患,毕竟这些设备时时刻刻监视着我们的生活环境,一旦被人突破安全防线,那么我们的生活就会暴露无遗,所以 IOT 设备安全也尤为重要。

设备的安全保护涉及到很多方面,本章我们讲解的 TF-A 主要保护的就是设备启动过程,通过各种鉴权,保证设备启动的过程中每个阶段的固件都是安全的,防止被不法分子替换某些启动固件导致安全信息泄露。

对于传统的 ARM 处理而言,Linux 系统的启动流程就是:内部 BootROM→Uboot→kernel→rootfs,整个启动过程是一个链式结构,启动过程其实是没有安全校验的(有些公司会自行定义一些校验方法)。加入 TF-A 固件以后,TF-A 就可以对 uboot、kernel 进行校验,如果还要使用 TEE OS(Trusted Execution Environment,TEE),那么 TF-A 还要完成对 TEE OS 的校验。

在第五章我们已经提到了 STM32MP1 的安全启动(Secure Boot,有些资料也叫做安全引导),安全启动目的是为了保证整个启动过程各个镜像的完整性,防止被不法分子破坏或替换掉。上面我们说了,Linux 启动是一个链式结构,因此安全启动的鉴权(校验)过程也是链式结构的。在系统启动的过程中,会先对下一个要加载运行的镜像进行鉴权,只有鉴权成功此镜像才能运行,并进入到下一阶段,同样要先对下一阶段的镜像进行鉴权,只要其中有一环鉴权失败,那么整个系统就会启动失败。

STM32MP1 使用 ECDSA(Elliptic Curve Digital Signature,椭圆曲线数字签名算法)验证算法来完成鉴权,ECDSA 相比 RSA 效果好,而且秘钥更小,STM32MP1 使用 256 位的 ECDSA 秘钥。一共有两个算法用于计算 ECDSA:

- P-256 NIST
- Brainpool 256

可以在 STM32MP1 头部信息中定义使用哪个算法,这个在 5.4 小节中讲解过了,STM32MP1 整个加密过程比较复杂,本教程不做讲解。STM32MP1 启动的时候,内部 Boot 代码会对 FSBL 进行鉴权,然后 FSBL 会对后续的内容进行鉴权,这个 FSBL 就是 TF-A。

7.2 TF-A 概述

7.2.1 TF-A 简介

TF-A 前面已经说了,是 ARM 可信任固件,是 ARM 官方提供的一个固件代码,它提供了统一的接口标准,方便不同的半导体厂商将自家的芯片添加到 TF-A 里面。ST 就是在 TF-A 官方源码里面添加了 STM32MP1 系列芯片,我们在 6.1.2 小节对 TF-A 源码打过补丁,以支持 STM32MP1 芯片,这个补丁文件就是 STM32MP1 芯片对应的源码补丁文件。

TF-A 官网地址为:https://www.trustedfirmware.org/,此网站不仅仅提供了 TF-A,还有针对 Cortex-M 单片机的 TF-M,以及 OP-TEE 等安全相关软件库。关于 TF-A 官方文档我们已经放到

了开发板光盘中，路径为：开发板光盘→4、参考资料→Trusted Firmware-A(TF-A 用户手册).pdf，感兴趣的可以详细阅读此官方文档。

TF-A 一共分为 5 部分：bl1、bl2、bl2u、bl31、bl32 和 bl33，打开 TF-A 源码目录，可以看到这 5 部分，如图 7.2.1 所示：

图 7.2.1 TF-A 源码

大家可能会有疑问，图 7.2.1 中并没有 bl33 啊，这是因为 bl33 是 TF-A 启动的其他镜像固件，比如 uboot。图 7.2.1 中 bl1、bl2 和 bl31 都属于 TF-A 固件，而 bl32 和 bl33 是 TF-A 要启动的其他第三方固件，比如 TEE OS 和 uboot。

bl1、bl2、bl31、bl32 和 bl33 是 TF-A 的不同启动阶段，TF-A 的启动过程是链式的，不同的阶段完整的功能不同， bl1、bl2、bl31、bl32 和 bl33 全名如下：

bl1：Boot loader stage 1 (BL1)。

bl2：Boot loader stage 2 (BL2)。

bl31：Boot loader stage 3-1 (BL31)。

bl32：Boot loader stage 3-2 (BL32)。

bl33：Boot loader stage 3-3 (BL33)。

7.2.2 ARMv7 和 ARMv8 权限等级

TF-A 一开始是为 ARMv8 准备的，ARMv8 最突出的特点就是支持 64 位指令，但是为了兼容原来的 ARMv7，ARMv8 提供了两种指令集：AAarch64 和 AArch32，根据字面意思就是 64 位和 32 位，其中 AArch32 和 ARMv7 基本一样(会多一些其他操作指令)。STM32MP1 内核为 32 位的 Cortex-A，所以对应 TF-A 中的 AArch32。正是因为 TF-A 一开始是针对 ARMv8 准备的，因此大家在看 TF-A 相关资料的时候会发现很多晦涩难懂的词汇，本来 TF-A 就是非常偏向底层的，需要对 ARM CPU 架构有一定的了解。安全不能仅仅依靠软件来实现，也是需要硬件支持的，比如 ARM 处理器就有不同的运行等级，运行在低等级(非安全模式)的应用就不能访问高等级(安全模式)的资源，以此来保证敏感资源的安全性。

1、ARMv7-A 工作模式

以前的 ARMv7 处理器有 7 中运行模型：User、FIQ、IRQ、Supervisor(SVC)、Abort、Undef 和 System。但新的 ARMv7-A 架构加入了 TrustZone 安全扩展，所以就新加了一种运行模式：Monitor，新的处理器架构还支持虚拟化扩展，因此又加入了另一个运行模式：Hyp，所以 Cortex-A7 处理器有 9 种处理模式，如表 7.2.2.1 所示：

模式	描述
User(USR)	用户模式，非特权模式，大部分程序运行的时候就处于此模式。
FIQ	快速中断模式，进入 FIQ 中断异常
IRQ	一般中断模式。
Supervisor(SVC)	超级管理员模式，特权模式，供操作系统使用。

Monitor(MON)	监视模式？这个模式用于安全扩展模式。
Abort(ABT)	数据访问终止模式，用于虚拟存储以及存储保护。
Hyp(HYP)	超级监视模式？用于虚拟化扩展。
Undef(UND)	未定义指令终止模式。
System(SYS)	系统模式，用于运行特权级的操作系统任务

注：‘？’表示为笔者翻译，不保证翻译的准确性。

表 7.2.2.1 九种运行模式

不同的处理器模式下，CPU 对于硬件的访问权限不同，叫做 Privilege Level(特权等级)，一共有两个特权级别：Privilege(特权级)和 non-privilege(非特权级)。其中只有 User 模式处于 non-privilege，也就是非特权级，剩下的 8 个模式都是 privilege(特权级)。系统启动以后应用软件都是运行在 User 模式，也就是非特权级，这个时候处理器对于敏感资源的访问是受限的，如果要访问这些敏感资源就需要切换到对应的工作模式下。

ARMv7-A 对 Privilege Level 进行了命名：PL0 和 PL1，后来也出现了 PL2，用于虚拟扩展。ARMv7-A 新增的 Monitor 模式就是针对安全扩展的，为了支持 TEE 而引入的。

2、ARMv8 工作模式

ARMv8 没有 Privilege level 的概念，取而代之的是 Exception level(异常级别)，简称为 EL，用于描述特权级别，一共有 4 个级别：EL0、EL1、EL2 和 EL3，数字越大，级别越高，权限越大！这四个 EL 级别对应的应用场合如下：

EL0：一般的应用程序。

EL1：操作系统，比如 Linux。

EL2：虚拟化(Hypervisor)，虚拟机管理器。

EL3：最底层的安全固件(安全监视器)，比如 ARM Trusted Firmware(ARM 安全固件，ATF，也就是 TF-A)。

ARMv8 提供了两种安全状态：Secure 和 Non-secure，也就是安全和非安全，Non-secure 也就是正常世界(Normal World)。我们可以在 Non-secure 运行通用操作系统，比如 Linux，在 Secure 运行可信操作系统，比如 OP-TEE，这两个操作系统可以同时运行，这个需要处理器支持 ARM 的 TrustZone 功能。在 Normal world 和 Secure world 下，ARMv8 个 EL 等级对应的内容如图 7.2.2.1 所示：

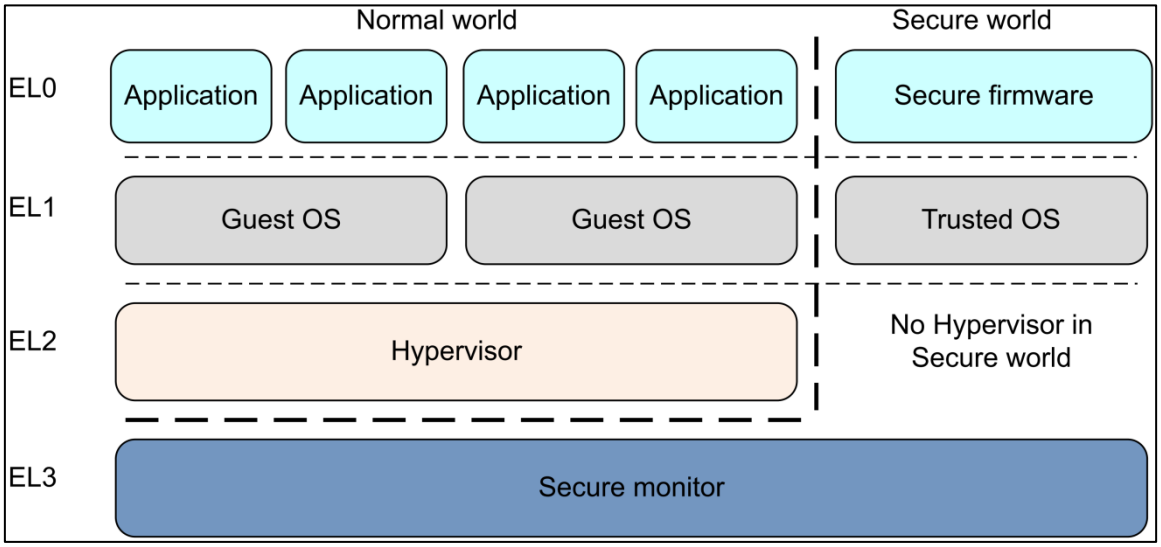


图 7.2.2.1 ARMv8 在 Normal 和 Secure 下 EL 等级对应的不同含义

在 ARMv8 的 AArch32 模式下，处理器模式如图 7.2.2.2 所示：

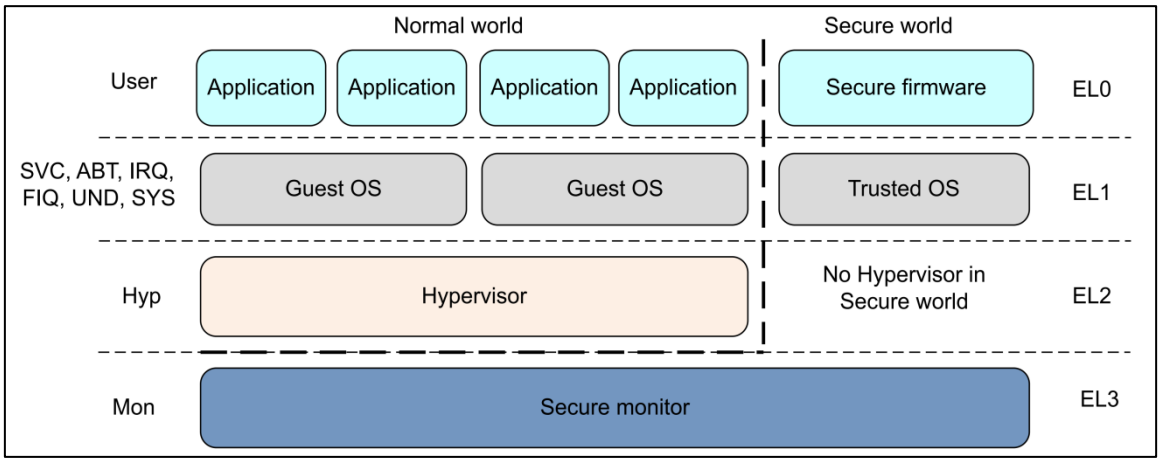


图 7.2.2.2 AArch32 下处理器模式

从图 7.2.2.2 可以看出，在 AArch32 模式下，EL0~LE3 对应 ARMv7 的不同工作模式：

EL0：对应 ARMv7 的 User 工作模式。

EL1：对应 ARMv7 的 SVC、ABT、IRQ、IRQ、UND 和 SYS 这 6 中工作模式。

EL2：对应 ARMv7 的 Hyp 工作模式。

EL3：对应 ARMv7 的 Mon 工作模式。

可以看出，只有 EL3 是用于安全监视器的，所以 TF-A 主要工作在 EL3 下，大家在看 TF-A 源码的时候会看到大量的“EL3”字样的文件或代码。

7.2.3 TF-A 不同启动阶段

前面说了，TF-A 分为不同的启动阶段，按照链式结构依次启动，ATF 代码启动流程如图 7.2.3.1 所示：

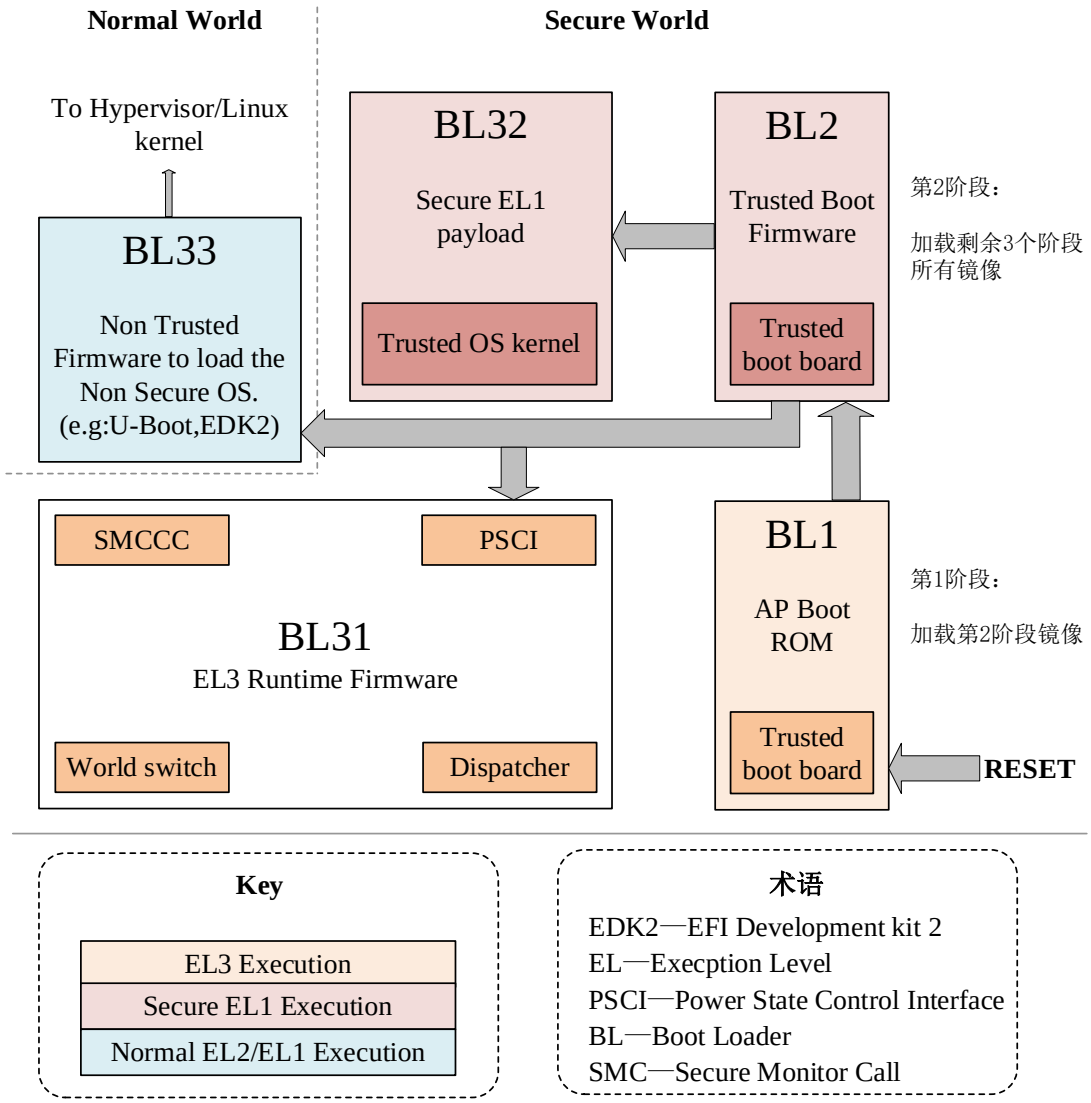


图 7.2.3.1 TF-A 启动流程

从图 7.2.3.1 可以看出，当芯片复位以后首先运行 bl1 代码，bl1 一般是芯片内部的 ROM 代码，bl1 主要工作就是将外置 Flash 中的 bl2 固件加载到指定的 RAM 中，然后跳转到 bl2 部分。

bl2 为安全启动固件，bl 2 会将剩余的三个启动阶段 bl31、bl32 和 bl33 对应的镜像文件加载到指定的内存中。比如 bl32 中的安全操作系统(OP-TEE)，bl31 中的 EL3 运行时固件(Runtime Firware)，bl33 中的 uboot。bl2 将这些固件加载完成以后就会启动相应的固件，也就是进入到第三启动阶段。

TF-A 启动流程就是：bl1→bl2→(bl31/bl32/bl33)。注意，bl31、bl32 和 bl33 对应的镜像不需要全部都有，但是 bl33 一般是必须的，因为 bl33 一般是 uboot，这个是很重要的！

1、bl 1

bl 1 是 TF-A 的第一个启动阶段，芯片复位以后就会运行 bl1 镜像，TF-A 提供了 bl1 源码。但是，实际上 bl1 一般是半导体厂商自己编写的内部 Boot ROM 代码，并没有使用 TF-A 提供的 bl1 镜像，比如 STM32MP1 的内部 ROM 代码就是 bl 1。因此 bl1 部分的实现就千差万别，不同的半导体厂商有不同的实现方法。

一般 bl1 要做的就是初始化 CPU，如果芯片支持不同的启动设备，那么还需要初始化不同

的启动设置，比如 NAND、EMMC、SD、USB 或串口等。然后根据 BOOT 引脚的高低电平来判断当前所选择的启动设备，从对应的启动设备中加载 bl2 镜像，并放到对应的内存中，最后跳转到 bl2 镜像并运行。

2、bl2

bl 2 会进一步的初始化芯片，比如初始化 DDR、MMU、串口等。bl2 会将剩下三个阶段(bl31、bl32 和 bl33)对应的镜像加载到指定的内存中，最后根据实际情况来启动剩下三个阶段的镜像。

3、bl31

在 AArch64 中，bl31 主要是 EL3 的 Runtime 固件。

4、bl32

bl32 一般为安全系统(TEE OS)固件，比如 OP-TEE。TF-A 为 AArch32 提供了 EL3 的 Runtime 软件，这个 Runtime 软件就是 bl32 固件，sp_min 就是这个 Runtime 软件。大家可以打开 TF-A 的 bl32 源码文件夹，其下就有一个名为“sp_min”的子文件夹，这个就是 bl32 的 sp_min 源码，如图 7.2.3.2 所示：

图 7.2.3.2 SP_MIN

讲这个的主要原因是 STM32MP1 的 bl32 部分可以使用 OP-TEE，也可以使用 sp_min，本教程为了方便讲解，并没有使用 OP-TEE，而是采用了 sp_min 作为 bl32 镜像。

5、bl33

bl33 就是 Normal World 下的镜像文件，比如 uboot。

至此，我们对 TF-A 的基本启动流程有了一个大概的了解，我们知道了 TF-A 分为多个阶段，不同的阶段其工作内容不同。但是，在实际的开发中并不一定会用到 TF-A 中所有启动阶段。接下来我们就会以 STM32MP1 为例，看一下 ST 是如何在自家的 STM32MP1 中使用 TF-A 的。

7.3 STM32MP1 中的 TF-A

7.3.1 STM32MP1 TF-A 框架

STM32MP1 支持 TrustZone，所以 ST 提供的软件包包含了安全固件。相比传统 ARM 处理器(如 ARM9，ARM11 等)最常见的 uboot 和 linux kernel，STM32MP1 的软件包还另外提供了 TF-A、OP-TEE 等安全相关的关键软件，因此 STM32MP1 的整体软件框架必然和传统的 ARM 芯片不同，STM32MP1 软件架构如图 7.3.1.1 所示：

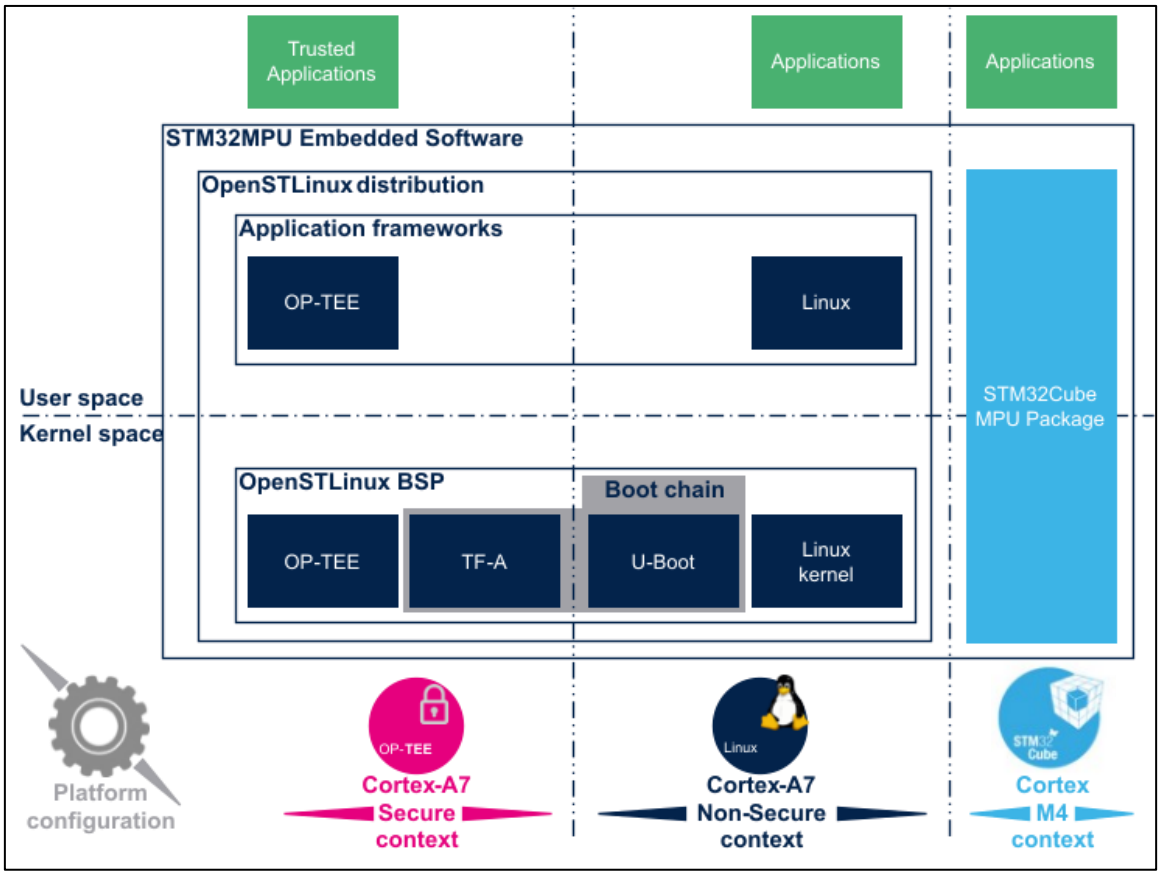


图 7.3.1.1 STM32MP1 软件框架

图 7.3.1.1 从左到右分为三部分：Cortex-A7 Secure、Cortex-A7 Non-Secure 和 Cortex-M4，Cortex-M4 本教程不涉及。所以就剩下了 Cortex-A7 Secure、Cortex-A7 Non-Secure，也就是 A7 的安全和非安全两种情况。

在 Cortex-A7 Secure 下重点是 TF-A 和 OP-TEE，TF-A 是用于完成安全启动的，OP-TEE 是 TEE OS，如果使用 OP-TEE 的话它会和 linux 内核同时运行，OP-TEE 负责可信应用，linux 就是普通的应用程序。在 Cortex-A7 Non-Secure 下就是传统的 ARM 软件框架：uboot、linux kernel 和根文件系统。

前面我们讲了，TF-A 分为了不同阶段：bl1、bl2、bl31、bl32 和 bl33，这个主要是面向 AArch64 的，对于 AArch32 而言只有 4 个阶段：

bl1：第 1 个阶段，一般为芯片内部 ROM 代码。

bl2：第 2 个阶段，可信启动固件。

bl32：EL3 运行时(Runtime)软件。

bl33：非安全固件，比如 uboot。

其中 bl1、bl2 和 bl32 都属于 TF-A 的一部分(如果你使用 TF-A 提供的 bl1 的话)。

1、STM32MP1 下的 bl1

bl1 部分是可选的，在编译 STM32MP1 的 TF-A 的时候可以通过添加 BL2_AT_EL3 编译选项来移除 bl1，默认情况下 ST 提供的 TF-A 源码是有添加 BL2_AT_EL3 编译选项的，在 TF-A 源码里面找到 tf-a-stm32mp-2.2.r1/plat/st/stm32mp1/platform.mk，此文件定义了 STM32MP1 这个平台的编译选项，有如图 7.3.1.2 所示配置项：

```
7  ARM_CORTEX_A7        :=  yes
8  ARM_WITH_NEON        :=  yes
9  BL2_AT_EL3           :=  1
10 USE_COHERENT_MEM     :=  0
```

图 7.3.1.2 STM32MP1 编译配置文件

从图 7.3.1.2 可以看出，platform.mk 文件定了 BL2_AT_EL3 为 1，因此在编译 STM32MP1 平台对应的 TF-A 的时候不会编译 bl1 部分，STM32MP1 内部 ROM 代码完成了 TF-A 中的 bl1 部分的工作，主要就是将外部 Flash 中的 bl2 代码加载到内部 RAM 中并运行。

2、STM32MP1 下的 bl2

bl2 为可信启动固件，在 STM32MP1 中就是 TF-A 的 bl2 部分，bl2 的主要功能就是加载下面几个阶段的固件到内存中，因此 bl2 需要初始化所要用到的外设。

首先是安全部分，STM32MP1 的 bl2 部分会初始化的外设如下：

①、BOOT、安全和 OTP 控制器，也就是 BSEC 外设。

②、扩展的 TrustZone 保护控制器，也就是 ETZPC 外设。

③、TrustZone 针对 DDR 的地址空间保护控制器，也就是 TZC 外设。

由于 bl2 需要从外部 flash 中加载下一阶段的镜像，因此还需要初始化一些外部 flash，比如：

①、SD 卡。

②、EMMC。

③、NAND。

④、NOR。

最后，STM32MP1 的 bl2 部分还要初始化一些其他的外设，比如：

①、DDR 内存。

②、时钟。

③、串口，用于调试以及使用 STM32CubeProgrammer 的时候通过串口下载系统。

④、USB，用 STM32CubeProgrammer 通过 USB 烧写系统的时候需要用到。

bl2 还需要对镜像镜像进行验证和鉴权，鉴权是通过调用内部 ROM 代码的鉴权服务来完成。最后，bl2 会加载 bl32 和 bl33 的固件到指定的内存区域，并跳转到 bl32，bl32 接着运行。

2、STM32MP1 下的 bl32

bl32 提供运行时安全服务，在 TF-A 中默认使用 sp_min，sp_min 已经在前面提过了。sp_min 是一个最小的 AArch32 安全负载(Secure Payload)，整合了 PSCI 库以及 AArch32 的 EL3 运行时软件。sp_min 可以替代可信系统(TEE OS)或者可信执行环境(TEE)，比如 OP-TEE。当然了，STM32MP1 同时支持 sp_min 以及 OP-TEE，用户可以自行选择 bl32 使用哪个软件包，为了方便讲解，本教程选择 sp_min。

bl32 充当安全监控(secure monitor)，因此它向非安全系统(non-secure os，比如 linux)提供了一些安全服务。非安全的应用软件可以通过安全监控调用(secure monitor calls)来使用这些安全服务，这些代码支持标准的服务调用，比如 PSCI。

另外，bl32 也支持 ST32MP1 所特有的一些安全服务，可以访问特有的安全外设，比如 RCC、PWR、RTC 或 BSEC。

3、STM32MP1 下的 bl33

bl33 没什么好说的了，就是传统的 uboot，并不属于 TF-A 本身。

简单总结一下，默认情况下 TF-A 有 bl1、bl2、bl31、bl32 和 bl33 这几个启动阶段。如果 bl32 使用 sp_min 的话那么 bl1、bl2、bl31 和 bl32 都属于 TF-A。但是对于 STM23MP1 而言，因

为其使用的是 AArch32，因此没有 bl31 部分。而 bl1 部分 ST 又没有用 TF-A 提供的，采用的是 STM32MP1 内部 ROM 代码，因此就只剩下了 bl2 和 bl32。所以对于 STM32MP1 而言，TF-A 就两个固件：bl2 和 bl32(sp_min)，TF-A 源码也采用了设备树(device tree)来设备信息，因此对于 STM32MP1 而言，TF-A 一共有三部分：设备树、bl2 和 bl32，这三部分在编译的时候会被合并成一个二进制文件。当然了，还要在最前面加上重要的头部信息，最终这 4 部分就组成了我们烧写到外部 flash 中的 TF-A 镜像，比如我们 6.2.3 小节中烧写到 EMMC 中的 tf-a-stm32mp157d-atk-trusted.stm32，其文件结构如图 7.3.1.3 所示：

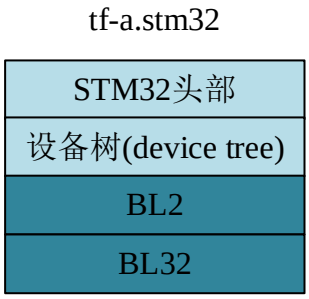


图 7.3.1.3 TF-A 镜像组织结构

STM32MP1 的 TF-A 启动流程如图 7.3.1.4 所示：

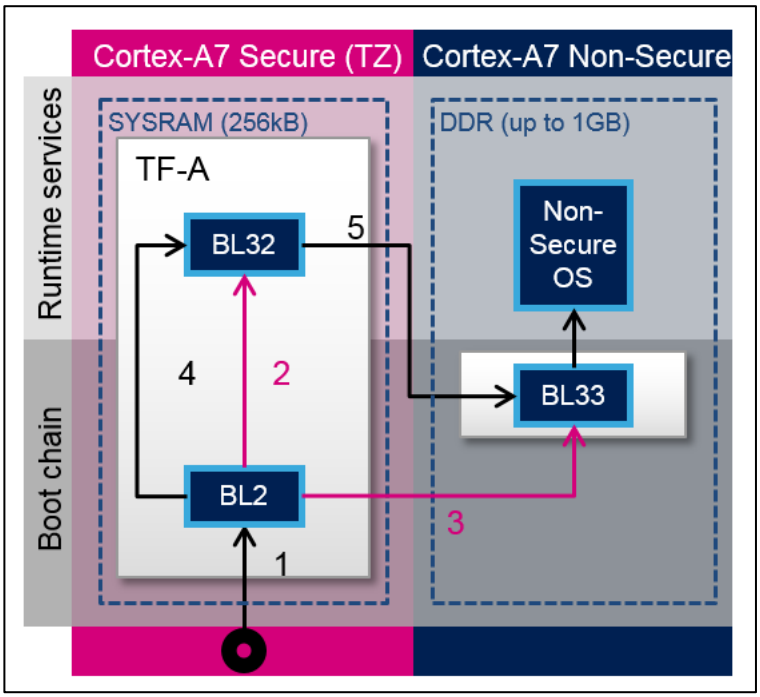


图 7.3.1.4 STM32MP1 TF-A 启动流程

图 7.3.1.4 中 TF-A 启动分了 5 步，这 5 步的含义如下：:

1、复位以后内部 ROM 加载 TF-A 整个镜像，然后运行 bl2 镜像。

2、bl2 将 bl32 镜像加载到指定内存区域。

3、bl2 将 bl33 镜像加载到指定内存区域。

4、bl2 执行完毕以后就会跳转到 bl32 镜像。

5、bl32 镜像执行完以后跳转到 bl33 镜像，也就是 uboot。

最后，uboot 引导非安全系统，也就是 linux 内核。

7.3.2 STM32MP1 TF-A 镜像存储映射

上一小节讲了，我们最终烧写到 STM32MP1 里面的 TF-A 镜像有 4 部分，除去头部信息，还有设备树、bl2 和 bl32。这 3 部分虽然在“肉体”上被打包在了一起，但是“灵魂”上是三部分，比如 bl2 是一个镜像，bl32 是另外一个镜像，其执行顺序都是不一样的！当加载到内存上以后这 3 部分的存储映射如图 7.3.2.1 所示：

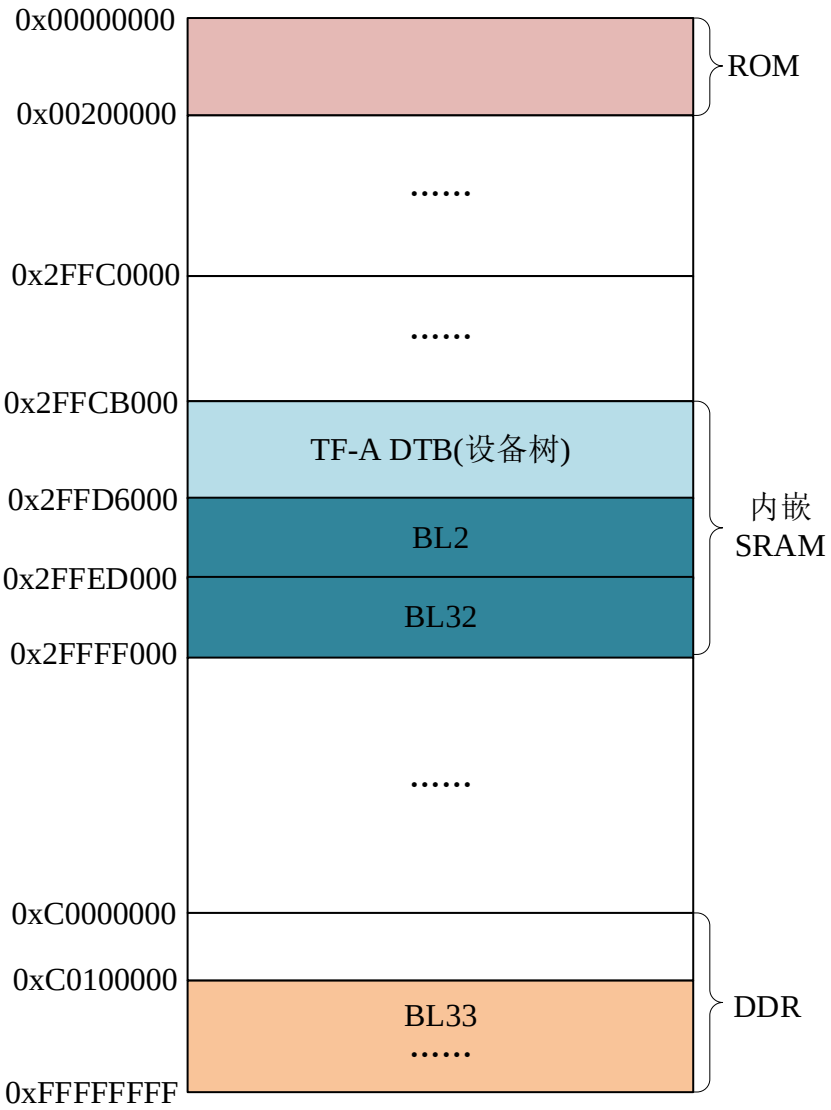


图 7.3.2.1 TF-A 存储分配

图 7.3.2.1 中 TF-A 各部分存储映射不是固定的，编译 TF-A 的时候配置不同，其存储地址也不同。比如图 7.3.2.1 中 BL32 的起始地址为 0x2FFED000，大家在看其他资料的时候可能是别的地址，这个没关系的，重点是其存储映射形式。

关于 TF-A 的基础知识就讲解到这里，大家基本对 TF-A 有了一个初步的了解，也了解了 STM32MP1 中的 TF-A 组织形式，方便我们后续学习。

第八章 TF-A 启动流程详解

后续更新

第九章 TF-A 移植

前面我们已经对 TF-A 进行了详细的讲解，但是当我们实际做产品的时候我们的硬件平台肯定会和 ST 官方的有区别，比如 DDR 容量会改变，自己的硬件没有使用到官方 EVK 开发板所使用的 PMIC 芯片等等。因此这里就涉及到将半导体原厂提供的 TF-A 移植到我们的硬件上，本章我们就来讲解一下如何将 ST 官方提供的 TF-A 源码移植到正点原子的 STM23MP157 开发板上。

9.1 编译 ST 官方 TF-A 源码

9.1.1 为什么要编译官方 TF-A 源码

所谓的移植就是让半导体官方提供的软件在自己的硬件平台上运行起来，准确的说应该是将自己的硬件添加到官方软件包。TF-A 是 ARM 官方出品的一个软件包，半导体厂商会从 ARM 官方下载这个最正宗的 TF-A 软件包，然后将自己公司的 SOC 芯片添加进去，最终打包好提供给 SOC 用户，这个就是所谓的 SDK 包。我们在 6.1.2 小节对 TF-A 源码打的那个补丁就是 ST 编写的针对自家芯片的补丁文件，打完补丁以后的 TF-A 就支持了 ST 公司的芯片，所有使用 ST 芯片的厂商都会使用这个 TF-A 源码。打完补丁以后的 TF-A 能不能正常工作是需要验证的，也就是需要在一个硬件平台上运行测试，所有半导体厂商都会针对自家的芯片出一个或多个官方开发板。ST 官方就有 STM32MP157 EVK 开发板，ST 提供给用户的软件包都是在这个开发板上验证通过的，包括 TF-A、uboot、linux kernel 等。所有使用 STM32MP157 芯片的厂商都会在 ST 官方的开发板硬件基础上进行修改，制作自己的硬件，比如正点原子的 STM32MP157 开发板就参考了 ST 官方的 STM32MP157 EVK 开发板。

既然硬件参考了 ST 官方的开发板，那么软件肯定也是在官方提供的软件包基础上修改的，所以本章 TF-A 的移植就是在 ST 官方提供的 TF-A 源码上进行。首先肯定要先编译一下 ST 官方提供的 TF-A 源码，目的如下：

①、掌握半导体官方软件编译方法，这个非常重要，编译都不会后面怎么可能做下去。

②、验证开发环境搭建是否正确，比如交叉编译器设置是否正确，所依赖的第三方库有没有安装等。

③、观察编译结果，比如编译完成以后的可执行文件保存在哪个目录下，都有哪些可执行文件，其区别是啥等。

④、在自己的开发板上运行编译出来的可执行文件，所谓的移植就是改 bug 的过程，将编译出来的 STM32MP157 官方开发板可执行文件在自己的板子上运行，然后观察运行过程有没有错误，没错误最好(基本上不可能没错，除非完全参考原厂开发板设计的)，有错误就改错误，直到能在自己的开发板上正常运行，这就是移植过程。

9.1.2 编译 TF-A

在 6.1.2 小节我们已经准备好了 ST 官方的 TF-A 源码，我们另外新建一个名为“my-tfa”的目录来存放并编译 ST 官方的 TF-A 源码，如图 9.1.2.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a/my-tfa$ ls
0001-st-update-v2.2-r2.0.0.patch  Makefile.sdk  README.HOW_TO.txt  series  tf-a-stm32mp-2.2.r1
```

图 9.1.2.1 TF-A 源码文件

图 9.1.2.1 中我们重点关注 Makefile.sdk 和 tf-a-stm32mp-2.2.r1，Makefile.sdk 是我们一会编译要用到的 Makefile，其实 Makefile.sdk 主要是配置编译选项，最终是通过调用 tf-a-stm32mp-2.2.r1 下的 Makefile 来完成具体编译过程的。tf-a-stm32mp-2.2.r1 就是打完补丁后的 ST 官方 TF-A 源码文件，也就是我们要编译的。

1、修改 Makefile.sdk

首先要修改图 9.1.2.1 中的 Makefile.sdk，因为此文件定义编译选项，包括交叉编译器，我们要使用自己的交叉编译器，也就是 4.3.1 小节安装的 arm-none-linux-gnueabihf-gcc 编译器。用 VIM 编辑器打开 Makefile.sdk，找到“CROSS_COMPILE”，将其改为“arm-none-linux-gnueabihf-”，修改以后如图 9.1.2.2 所示：

```
1 #remove default variable
2 LDFLAGS=
3 CFLAGS=
4 CPPFLAGS=
5 CC=
6 CPP=
7 AS=
8 AR=
9 LD=
10 NM=
11
12 LOCAL_PATH=$(PWD)
13
14 EXTRA_OEMAKE=CROSS_COMPILE=arm-none-linux-gnueabihf- DEBUG=1 LOG_LEVEL=40 PLAT=stm32mp1 ARCH=aarch32 AR
   M_ARCH_MAJOR=7 STM32MP_SDMMC=1 STM32MP_EMMC=1 STM32MP_SPI_NOR=1 STM32MP_RAW_NAND=1 STM32MP_SPI_NAND=1
15 EXTRA_OEMAKE_SERIAL= STM32MP_UART_PROGRAMMER=1 STM32MP_USB_PROGRAMMER=1
```

图 9.1.2.2 修改后的 Makefile.sdk

修改完成以后保存退出，进入到图 9.1.2.1 中的 tf-a-stm32mp-2.2.r1 目录下，然后进行编译，命令如下：

```
cd tf-a-stm32mp-2.2.r1/          //进入到 tf-a 源码目录下
make -f ../Makefile.sdk all      //编译
```

如果需要加快编译速度，可是使用多线程编译，线程数最好和自己给虚拟机分配的物理核心一致，使用-j 来指定线程数，命令如下：

```
make -f ../Makefile.sdk -j8 all//使用 8 线程编译
```

编译完成以后会在 tf-a-stm32mp-2.2.r1 文件夹的上层目录(也就是 Makefile.sdk 同级目录下)下生成一个名为“build”的文件夹，build 文件夹下就是编译生成的所有可执行文件，如图 9.1.2.3 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a/my-tfa$ ls
0001-st-update-v2.2-r2.0.0.patch  build  Makefile.sdk  README.HOW_TO.txt  series  tf-a-stm32mp-2.2.r1
```

图 9.1.2.3 编译生成的 build 目录

进入到 build 目录下，有三个子目录，如图 9.1.2.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a/my-tfa/build$ ls
optee  serialboot  trusted
```

图 9.1.2.4 编译结果

图 9.1.2.4 中有 3 个子目录：optee、serialboot 和 trusted，说明有 3 个版本的可执行文件，optee 是针对 OP-TEE 的，本教程没使用 OP-TEE，所以 optee 目录和我们没关系。

serialboot 看名字就知道是和串行启动有关的，也就是通过串口或者 USB 烧写系统的时候需要用到 serialboot 下的可执行文件，我们在 6.2.3 小节编写 STM32CubeProgrammer 所使用的 Flashlayout 文件(.tsv)的时候用到了一个名为“tf-a-stm32mp157d-atk-serialboot.stm32”的文件，此文件就是 serialboot 目录下的。当然了，图 9.1.2.4 中目前还没有“tf-a-stm32mp157d-atk-serialboot.stm32”这个文件，这里只是给大家讲解一下 xxx-serialboot.stm32 这个文件的来源。当把 TF-A 移植到自己开发板上以后就会在 serialboot 目录下生成我们板子对应的 xxx-serialboot.stm32 文件，后面会讲解。

trusted 目录下的就是烧写到开发板中的 TF-A 镜像，进入 trusted 目录，如图 9.1.2.5 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a/my-tfa/build/trusted$ ls
bl2                                     tf-a-stm32mp157a-dk1-trusted.stm32  tf-a-stm32mp157d-dk1.map
bl2.bin                                 tf-a-stm32mp157a-ev1.bin            tf-a-stm32mp157d-dk1.stm32
bl32                                    tf-a-stm32mp157a-ev1.elf            tf-a-stm32mp157d-dk1-trusted.stm32
bl32.bin                                tf-a-stm32mp157a-ev1.ld             tf-a-stm32mp157d-ev1.bin
debug-tf-a-stm32mp157a-dk1-trusted.stm32  tf-a-stm32mp157a-ev1.map          tf-a-stm32mp157d-ev1.elf
debug-tf-a-stm32mp157a-ev1-trusted.stm32  tf-a-stm32mp157a-ev1.stm32        tf-a-stm32mp157d-ev1.ld
debug-tf-a-stm32mp157c-dk2-trusted.stm32  tf-a-stm32mp157a-ev1-trusted.stm32  tf-a-stm32mp157d-ev1.map
debug-tf-a-stm32mp157c-ed1-trusted.stm32  tf-a-stm32mp157c-dk2.bin          tf-a-stm32mp157d-ev1.stm32
debug-tf-a-stm32mp157c-ev1-trusted.stm32  tf-a-stm32mp157c-dk2.elf          tf-a-stm32mp157d-ev1-trusted.stm32
debug-tf-a-stm32mp157d-dk1-trusted.stm32  tf-a-stm32mp157c-dk2.ld           tf-a-stm32mp157f-dk2.bin
debug-tf-a-stm32mp157d-ev1-trusted.stm32  tf-a-stm32mp157c-dk2.map          tf-a-stm32mp157f-dk2.elf
debug-tf-a-stm32mp157f-dk2-trusted.stm32  tf-a-stm32mp157c-dk2.stm32        tf-a-stm32mp157f-dk2.ld
debug-tf-a-stm32mp157f-ed1-trusted.stm32  tf-a-stm32mp157c-dk2-trusted.stm32  tf-a-stm32mp157f-dk2.map
debug-tf-a-stm32mp157f-ev1-trusted.stm32  tf-a-stm32mp157c-ed1.bin          tf-a-stm32mp157f-dk2.stm32
fdts                                    tf-a-stm32mp157c-ed1.elf            tf-a-stm32mp157f-dk2-trusted.stm32
lib                                     tf-a-stm32mp157c-ed1.ld             tf-a-stm32mp157f-ed1.bin
libc                                    tf-a-stm32mp157c-ed1.map            tf-a-stm32mp157f-ed1.elf
```

图 9.1.2.5 TF-A 镜像文件

从图 9.1.2.5 可以看出，编译出来了很多后缀为.bin、.elf、.ld、.stm32 这样的文件， 我们以前缀为“tf-a-stm32mp157d-ev1”的文件为例，相关的文件如表 9.1.2.1 所示：

文件	描述
tf-a-stm32mp157d-ev1.bin	使用 STM32MP157D 芯片的 EV1 开发板对应的 TF-A 二进制可执行文件。
tf-a-stm32mp157d-ev1.elf	生成 bin 文件所使用的 elf 文件。
tf-a-stm32mp157d-ev1.ld	连接信息
tf-a-stm32mp157d-ev1.map	内存映射文件。
tf-a-stm32mp157d-ev1.stm32	添加好头部信息的 bin 文件，可以直接烧写到开发板中。
tf-a-stm32mp157d-ev1-trusted.stm32	和 tf-a-stm32mp157d-ev1.stm32 文件一模一样，只是重命名了一下，为了和 optee 以及 serialboot 目录下的 tf-a 固件进行区分。

表 9.1.2.1 使用 STM32MP157D 芯片的 EV1 开发板对应的文件

从表 9.1.2.1 可以看出，这些文件都是针对使用 STM32MP157D 芯片的 EV1 开发板，同理，其他的文件命名也是类似的，我们只需要选择和我们所使用开发板接近的文件就行了。比如正点原子 STM32MP157 开发板使用的主控型号是 STM32MP157DAA1，开发板硬件参考了 ST 官方的 STM32MP157 EVK 开发板，因此我们就需要使用表 9.1.2.1 中的这些文件，在移植的时候需要参考的对象就是 stm32mp157d-ev1。

另外，表 9.1.2.1 中的 tf-a-stm32mp157d-ev1.stm32 和 tf-a-stm32mp157d-ev1-trusted.stm32 这两个文件内容一模一样，后者只是前者的重命名版本，目的是为了和上一级 optee 和 serialboot 下的文件进行区分，我们再烧写的时候使用 tf-a-stm32mp157d-ev1-trusted.stm32 即可。

在 trusted 目录下，还有一些其他的文件，比如单独的 bl2 和 bl32 对应的可执行文件：bl2.bin 和 bl32.bin。bl2 和 bl32 编译过程中所有的.c 源码文件对应的.o 文件也都保存在 bl2 和 bl32 文件夹下，感兴趣的可以去看一下。

9.1.3 烧写 ST 官方 TF-A 镜像

将上一小节编译出来的 tf-a-stm32mp157d-ev1-trusted.stm32 烧写到正点原子开发板 EMMC 里面，烧写方法参考 6.2.3 小节讲解的。烧写完成以后打开串口，设置好串口调试软件 MobaXterm，设置开发板从 EMMC 启动，按下复位按键，观察 MobaXterm 软件接收到的 log 信息，如图 9.1.3.1 所示：

图 9.1.3.1 运行结果

图 9.1.3.1 的意思就是运行崩溃了，说明 ST 官方开发板对应的 TF-A 不能直接在我们的开发板上运行，需要进行修改。

9.2 TF-A 移植

经过测试 ST 官方开发板的 TF-A 镜像并不能直接在正点原子的开发板上运行，所以需要我们对 TF-A 进行修改，也就是移植到正点原子的开发板上。

注意，TF-A 的移植全部是修改设备树，初学 linux 的朋友肯定对设备树(device tree)不了解，而设备树是一个系统的概念，会在后面的 linux 驱动开发里面详解设备树的相关语法！所以初学的朋友直接根据接下来的教程修改就行了，不用深入研究其原理。或者，直接跳过 TF-A 的移植，直接使用正点原子移植好的。

再次“鞭尸”ST，所有的软件全部采用设备树，导致初学者上手难度增大！！！

9.2.1 新建自己开发板对应的设备树

1、创建板子对应的设备树

设备树英文名字叫做 Device tree，用来描述板子硬件信息的，比如你的板子上的 CPU 有几个核、每个 CPU 核主频是多少，IIC、SPI 这些外设的寄存器范围是多少，IIC 接口下都挂了哪些设备等等。设备树文件是一种文本格式的文件，方便阅读与修改，文件后缀为.dts，设备树也有头文件，头文件后缀为.dtsi。和 C 语言一样.dts 可以引用头文件.dtsi，.dts 经过编译以后生成.dtb 文件，.dtb 就是开发板要使用的。类似于我们将 C 语言的.c 文件编译为.bin 文件，然后在开发板上运行，将 dts 编译为 dtb 的工具叫做 DTC，并不是我们使用的 gcc 编译器。

打开 tf-a-stm32mp-2.2.r1\fdts 目录，fdts 目录下保存的就是所有开发板的设备树文件，其中就包括了 STM32MP1 系列的，如图 9.2.1.1 所示：

stm32mp15-ddr3-1x4Gb-1066-binG.dtsi	2020-11-11 9:01	DTSI 文件	4 KB
stm32mp15-ddr3-2x4Gb-1066-binG.dtsi	2020-11-11 9:01	DTSI 文件	4 KB
stm32mp15-pinctrl.dtsi	2020-11-11 9:01	DTSI 文件	8 KB
stm32mp15xa.dtsi	2020-11-11 9:01	DTSI 文件	1 KB
stm32mp15xc.dtsi	2020-11-11 9:01	DTSI 文件	1 KB
stm32mp15xd.dtsi	2020-11-11 9:01	DTSI 文件	1 KB
stm32mp15xf.dtsi	2020-11-11 9:01	DTSI 文件	1 KB
stm32mp15xxaa-pinctrl.dtsi	2020-11-11 9:01	DTSI 文件	2 KB
stm32mp15xxab-pinctrl.dtsi	2020-11-11 9:01	DTSI 文件	2 KB
stm32mp15xxac-pinctrl.dtsi	2020-11-11 9:01	DTSI 文件	2 KB
stm32mp15xxad-pinctrl.dtsi	2020-11-11 9:01	DTSI 文件	2 KB
stm32mp15xx-dkx.dtsi	2020-11-11 9:01	DTSI 文件	9 KB
stm32mp15xx-edx.dtsi	2020-11-11 9:01	DTSI 文件	10 KB
stm32mp15xx-evx.dtsi	2020-11-11 9:01	DTSI 文件	2 KB
stm32mp151.dtsi	2020-11-11 9:01	DTSI 文件	16 KB
stm32mp153.dtsi	2020-11-11 9:01	DTSI 文件	1 KB
stm32mp157.dtsi	2020-11-11 9:01	DTSI 文件	1 KB
stm32mp157a-avenger96.dts	2020-11-11 9:01	媒体文件(.dts)	9 KB
stm32mp157a-dk1.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157a-ed1.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157a-ev1.dts	2020-11-11 9:01	媒体文件(.dts)	1 KB
stm32mp157c-dk2.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157c-ed1.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157c-ev1.dts	2020-11-11 9:01	媒体文件(.dts)	1 KB
stm32mp157d-dk1.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157d-ed1.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157d-ev1.dts	2020-11-11 9:01	媒体文件(.dts)	1 KB
stm32mp157f-dk2.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157f-ed1.dts	2020-11-11 9:01	媒体文件(.dts)	2 KB
stm32mp157f-ev1.dts	2020-11-11 9:01	媒体文件(.dts)	1 KB

图 9.2.1.1 TF-A 设备树文件

图 9.2.1.1 中的 stm32mp157d-ev1.dts 就是 STM32MP157D EVK 开发板所对应的设备树文件，同理还有 STM32MP157 其他型号的芯片，比如 stm32mp157a、stm32mp157c 等。我们以 stm32mp157d-ev1.dts 为蓝本，添加正点原子 STM32MP157 开发板对应的设备树文件。

创建 VScode 工程，然后用 VScode 打开，打开 stm32mp157d-ev1.dts 文件，文件内容如图 9.2.1.2 所示：

```
1  // SPDX-License-Identifier: (GPL-2.0+ OR BSD-3-Clause)
2  /*
3   * Copyright (C) STMicroelectronics 2019 - All Rights Reserved
4   * Author: Alexandre Torgue <alexandre.torgue@st.com> for STMicroelectronics.
5   */
6  /dts-v1/;
7
8  #include "stm32mp157d-ed1.dts"
9  #include "stm32mp15xx-evx.dtsi"
10
11 / {
12     model = "STMicroelectronics STM32MP157D eval daughter on eval mother";
13     compatible = "st,stm32mp157d-ev1", "st,stm32mp157d-ed1", "st,stm32mp157";
14
15     chosen {
16         stdout-path = "serial0:115200n8";
17     };
18
19     aliases {
20         serial1 = &usart3;
21     };
22 };
23
```

图 9.2.1.2 stm32mp157d-ev1.dts 文件内容

图 9.2.1.2 中第 8 行引用了“stm32mp157d-ed1.dts”这样一个 dts 文件，注意，在设备树里面可以直接通过 include 引用另外一个.dts 文件，不仅限于.dtsi 文件，也就是说.dts 文件也可以作为头文件使用。

可以看出 stm32mp157d-ev1.dts 文件非常简洁，主要原因是其引用了 stm32mp157d-ed1.dts，主要工作都由 stm32mp157d-ed1.dts 文件来完成了。既然如此，那我们就把蓝本换为 stm32mp157d-ed1.dts 得了。复制一份 stm32mp157d-ed1.dts，并命名为 stm32mp157d-atk.dts，stm32mp157d-atk.dts 就是我们为正点原子 STM32MP157 开发板准备的设备树，命令如下：

```
cd fdts
cp stm32mp157d-ed1.dts stm32mp157d-atk.dts  //复制
```

复制完成以后打开 stm32mp157d-atk.dts 文件，内容如图 9.2.1.3 所示：

stm32mp157d-atk.dts ×

tf-a-stm32mp-2.2.r1 > fdts > stm32mp157d-atk.dts

```
1  // SPDX-License-Identifier: (GPL-2.0+ OR BSD-3-Clause)
2  /*
3   * Copyright (C) STMicroelectronics 2019 - All Rights Reserved
4   * Author: Alexandre Torgue <alexandre.torgue@st.com> for STMicroelectronics.
5   */
6  /dts-v1/;
7
8  #include "stm32mp157.dtsi"
9  #include "stm32mp15xd.dtsi"
10 #include "stm32mp15-pinctrl.dtsi"
11 #include "stm32mp15xxaa-pinctrl.dtsi"
12 #include "stm32mp15xx-edx.dtsi"
13 #include <dt-bindings/soc/st,stm32-etzpc.h>
14
15 / {
16     model = "STMicroelectronics STM32MP157D eval daughter";
17     compatible = "st,stm32mp157d-ed1", "st,stm32mp157";
18
19     chosen {
20         stdout-path = "serial0:115200n8";
21     };
22
23     aliases {
24         serial0 = &uart4;
25     };
26 };
27
28 &cpu1 {
29     cpu-supply = <&vddcore>;
30 };
31
32 &etzpc {
33     st,decprot = <
34         DECPROT(STM32MP1_ETZPC_USART1_ID, DECPROT_NS_RW, DECPROT_UNLOCK)
35         DECPROT(STM32MP1_ETZPC_SPI6_ID, DECPROT_NS_RW, DECPROT_UNLOCK)
36         DECPROT(STM32MP1_ETZPC_I2C4_ID, DECPROT_NS_RW, DECPROT_UNLOCK)
37         DECPROT(STM32MP1_ETZPC_I2C6_ID, DECPROT_NS_RW, DECPROT_UNLOCK)
38         DECPROT(STM32MP1_ETZPC_RNG1_ID, DECPROT_NS_RW, DECPROT_UNLOCK)
39         DECPROT(STM32MP1_ETZPC_HASH1_ID, DECPROT_NS_RW, DECPROT_UNLOCK)
40         DECPROT(STM32MP1_ETZPC_DDRCTRL_ID, DECPROT_S_RW, DECPROT_LOCK)
41         DECPROT(STM32MP1_ETZPC_DDRPHYC_ID, DECPROT_S_RW, DECPROT_LOCK)
42         DECPROT(STM32MP1_ETZPC_STGENC_ID, DECPROT_S_RW, DECPROT_LOCK)
43         DECPROT(STM32MP1_ETZPC_BKPSRAM_ID, DECPROT_S_RW, DECPROT_LOCK)
44         DECPROT(STM32MP1_ETZPC_IWDG1_ID, DECPROT_S_RW, DECPROT_LOCK)
45     >;
46 };
```

图 9.2.1.3 stm32mp157d-atk.dts 文件内容

第 8~13 行，头文件引用，其中第 8~11 行引用 STM32MP15X 芯片相关的 dtsi 头文件，这些全部保留即可。

第 12 行，非常重要，“stm32mp15xx-edx.dtsi”，看名字就知道，是 edx 系列开发板的通用头文件，适合具体板子有关的，很明显，板子不同其对应的板子头文件也不同。我们同样需要以 stm32mp15xx-edx.dtsi 为蓝本，创建正点原子开发板对应的板子头文件，一会再创建。

第 20 行，stdout-path 表示标准输出，也就是设置 TF-A 信息输出路径，这里设置为 serial0，也就是串行接口 0(注意，不是 STM32MP157 的串口 0)，波特率为 115200。

第 24 行，设置 serial0 对应 STM32MP157 的串口 4，所以 TF-A 会使用 STM32MP157 的串口 4 作为信息输出接口，大家在做板子的时候最好选择串口 4 作为通信接口。

最后，我们上面说了，需要以 stm32mp15xx-edx.dtsi 为蓝本，制作正点原子开发板对应的头文件，方法很简单，直接复制一份 stm32mp15xx-edx.dtsi，然后将其重命名为“stm32mp157d-atk.dtsi”，命令如下：

```
cd fdts
cp stm32mp15xx-edx.dtsi stm32mp157d-atk.dtsi
```

拷贝完成以后就需要修改 stm32mp157d-atk.dts，将第 12 行改为：

```
#include "stm32mp157d-atk.dtsi"
```

修改以后的 stm32mp157d-atk.dts 文件内容如下(有缩减)：

示例代码 9.2.1.1 stm32mp157d-atk.dts 文件内容

```
1  // SPDX-License-Identifier: (GPL-2.0+ OR BSD-3-Clause)
2  /*
3   * Copyright (C) STMicroelectronics 2019 - All Rights Reserved
4   * Author: Alexandre Torgue <alexandre.torgue@st.com> for
        STMicroelectronics.
5   */
6  /dts-v1/;
7
8   #include "stm32mp157.dtsi"
9   #include "stm32mp15xd.dtsi"
10  #include "stm32mp15-pinctrl.dtsi"
11  #include "stm32mp15xxaa-pinctrl.dtsi"
12  #include "stm32mp157d-atk.dtsi"
13  #include <dt-bindings/soc/st,stm32-etzpc.h>
14
15  / {
16      model = "STMicroelectronics STM32MP157D eval daughter";
17      compatible = "st,stm32mp157d-ed1", "st,stm32mp157";
18
19      chosen {
20          stdout-path = "serial0:115200n8";
21      };
22
23  aliases {
24      serial0 = &uart4;
```

```
25      };
26  };
......
```

重点是修改后的第 12 行。

2、编译

设备树创建好以后编译一下，在编译之前要先修改以后 Makefile.sdk，因此我们前面新建的 stm32mp157d-atk.dts 并没有添加到编译列表里面，直接编译的话并不会编译此文件。打开 Makefile.sdk，在 TFA_DEVICETREE 配置项中添加“stm32mp157d-atk”，如图 9.2.1.4 所示：

```
12 LOCAL_PATH=$(PWD)
13
14 EXTRA_OEMAKE=CROSS_COMPILE=arm-none-linux-gnueabihf- DEBUG=1 LOG_LEVEL=40 PLAT=stm32mp1 ARCH=aarch32 ARM_
   ARCH_MAJOR=7 STM32MP_SDMMC=1 STM32MP_EMMC=1 STM32MP_SPI_NOR=1 STM32MP_RAW_NAND=1 STM32MP_SPI_NAND=1
15 EXTRA_OEMAKE_SERIAL= STM32MP_UART_PROGRAMMER=1 STM32MP_USB_PROGRAMMER=1
16
17 # Set default config
18 ELF_DEBUG_ENABLE ?= 1
19 TF_A_CONFIG ?=  trusted  optee  serialboot
20
21 # Set specific OEMAKE config
22 TF_A_CONFIG_OEMAKE =  trusted,"AARCH32_SP=sp_min" optee,"AARCH32_SP=optee" serialboot,"AARCH32_SP=sp_min"
23 # Set default TFA_DEVICETREE
24 TFA_DEVICETREE ?=  stm32mp157d-atk stm32mp157a-dk1 stm32mp157d-dk1 stm32mp157c-dk2 stm32mp157f-dk2  stm
   32mp157c-ed1 stm32mp157f-ed1  stm32mp157a-ev1 stm32mp157c-ev1 stm32mp157d-ev1 stm32mp157f-ev1
25
26 #Set default TF_A_ENABLE_DEBUG_WRAPPER
27 TF_A_ENABLE_DEBUG_WRAPPER ?= 1
```

新创建的设备树名字

图 9.2.1.4 修改后的 TFA_DEVICETREE

修改完成以后保存退出，然后执行如下命令编译一下 TF-A：

```
cd tf-a-stm32mp-2.2.r1/          //进入到 tf-a 源码目录下
make -f ../Makefile.sdk all      //编译
```

如果修改没有错误的话就会编译成功，编译完成以后进入 build/trusted 目录下，此时就会生成以“tf-a-stm32mp157d-atk”开头的相关文件，如图 9.2.1.5 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a/my-tfa/build/trusted$ ls tf-a-stm32mp157d-atk* -l
-rwxr-xr-x 1 zuozhongkai zuozhongkai 241728 11月  21 18:49 tf-a-stm32mp157d-atk.bin
-rwxr-xr-x 1 zuozhongkai zuozhongkai 252056 11月  21 18:49 tf-a-stm32mp157d-atk.elf
-rw-r--r-- 1 zuozhongkai zuozhongkai   1354 11月  21 18:49 tf-a-stm32mp157d-atk.ld
-rw-r--r-- 1 zuozhongkai zuozhongkai   3603 11月  21 18:49 tf-a-stm32mp157d-atk.map
-rw-r--r-- 1 zuozhongkai zuozhongkai 241984 11月  21 18:49 tf-a-stm32mp157d-atk.stm32
-rw-r--r-- 1 zuozhongkai zuozhongkai 241984 11月  21 18:49 tf-a-stm32mp157d-atk-trusted.stm32
```

图 9.2.1.5 编译生成的正点原子开发板 TF-A 固件

图 9.2.1.5 就是我们以后要烧写的 TF-A 固件，其设备树(dtb)已经替换为我们上面新建的 stm32mp157d-atk.dts 了。但是由于我们并没有对 TF-A 源码做什么修改，所以 tf-a-stm32mp157d-atk-trusted.stm32 依旧是不可运行的，错误类型和图 9.1.3.1 中的一样，大家可以自行烧写测试。

9.2.2 修改设备树电源管理

ST 官方 STM32MP157 开发板用到了一颗 PMIC 芯片，型号为 STPMIC1A，PMIC 全称为 Power Management IC，也就是集成电源管理芯片。我们在用单片机的时候，只需要给一个 3.3V 或 5V 电源即可，使用起来很方便。但是随着芯片性能的提升，SOC 对电源的要求越来越多，比如 Cortex-A 系列芯片主电源一般要求 3.3V，但是芯片内核电压可能需要 1.2V，而外扩的 DDR3L 芯片工作电压是 1.35V，DDR3L 芯片的端接电压要求 0.65V，USB 工作电压又要求 5V 等等，也就是说随着 SOC 功能越来越强大，所需要的的电源要求也越来越高，最直观的就是电源种类很多。另外，这些电源的上电顺序也是有要求的，也就是说，哪些电源先启动，那些后

启动等。

为此，SOC 厂商为了简化硬件设计，都会针对自家的 SOC 芯片推出专用的 PMIC 芯片，PMIC 芯片就是一个单进多出的电源芯片，比如输入 5V，然后可以输出很多路电源，输出的这些电源可以配置输出电压。STPMIC1A 就是 ST 专门为 STM32MP1 系列设计的专用 PMIC，此 PMIC 芯片性能非常强大：

- 输入电压 2.8V~5.5V。
- 4 路可调的通用 LDO 输出。
- 1 路 DDR3 端接 LDO 电源。
- 1 路 USB PHY 所使用的 LDO 电源。
- 1 路 DDR 参考电压 LDO 电源。
- 4 路可调的 BUCK 开关电源。
- 1 路 5.2V/1.1A 的 BOOST 开关电源。
- 1 路 500mA 的 USB OTG 电源。
- 1 路 500mA/1000mA 的通用电源。
- 此芯片有一个 IIC 接口，主控通过 IIC 接口来配置电源芯片，设置每一路输出电源的输出电压，开启时间等。

STPMIC1A 是个好东西，但是好东西是要付出代价的，STPMIC1A 在 ST 官网报价 1.9 美金/1K(2020/11/12 日)，也就是说 1000 片起的话，价格是 1.9 美金，换算成人民币就是 12.5 块钱。但是对于 STM32MP157 而言用不了这么多路电源，三四路就够了，所以可以采用分离电源，这样的话三四个电源芯片的价格远小于 12.5 块钱。难点就是分离电源很考验硬件工程师的设计功底，因为处理不同电源上电顺序，好在 ST 已经提供了分离电源设计手册。

总之就是，因为采用 ST 提供的专用 PMIC 成本太高，因此大部分 STM32MP157 核心板都是采用分离电源设计的。STPMIC1A 是一个 IIC 器件，因此就涉及到驱动问题，需要主控通过 IIC 接口来配置各路电源，所以 ST 官方提供的 TF-A、uboot、linux kernel 等都会有 PMIC 驱动。但是正点原子的开发板并没有采用这个 PMIC 芯片，所以 TF-A 默认的电源配置就会出问题，这里就需要我们修改相关的文件，取消掉这个 PMIC 的相关配置。

打开 stm32mp157d-atk.dtsi 文件，找到如下所以代码：

示例代码 9.2.2.1 pmic 节点信息

```
45 &i2c4 {
46     pinctrl-names = "default";
47     pinctrl-0 = <&i2c4_pins_a>;
48     i2c-scl-rising-time-ns = <185>;
49     i2c-scl-falling-time-ns = <20>;
50     clock-frequency = <400000>;
51     status = "okay";
52     secure-status = "okay";
53
54     pmic: stpmic@33 {
55         compatible = "st,stpmic1";
56         reg = <0x33>;
57         interrupts-extended = <&exti_pwr 55 IRQ_TYPE_EDGE_FALLING>;
58         interrupt-controller;
59         #interrupt-cells = <2>;
```

```
60          status = "okay";
61          secure-status = "okay";
62
63          regulators {
64              compatible = "st,stpmic1-regulators";
65              buck1-supply = <&vin>;
66              buck2-supply = <&vin>;
67              buck3-supply = <&vin>;
68              buck4-supply = <&vin>;
69              ldo1-supply = <&v3v3>;
70              ldo2-supply = <&v3v3>;
71              ldo3-supply = <&vdd_ddr>;
72              ldo4-supply = <&vin>;
73              ldo5-supply = <&v3v3>;
74              ldo6-supply = <&v3v3>;
75              vref_ddr-supply = <&vin>;
76              boost-supply = <&vin>;
77              pwr_sw1-supply = <&bst_out>;
78              pwr_sw2-supply = <&bst_out>;
......      /* 省略其他内容 */
271
272             vbus_otg: pwr_sw1 {
273                 regulator-name = "vbus_otg";
274             };
275
276             vbus_sw: pwr_sw2 {
277                 regulator-name = "vbus_sw";
278                 regulator-active-discharge = <1>;
279             };
280         };
281     };
282 };
```

简答给大家说一下上述代码的含义：

第 45 行，“i2c4”表示这段代码是和 I2C4 控制器有关的，在设备树中一个 IIC 接口下的所有设备都“打包”放到一个节点下，在这里就是描述 I2C4 这个 IIC 控制器下的所有 IIC 器件。ST 官方的 STM32MP157 开发板将 STPMIC1A 这个 PMIC 芯片挂到了 I2C4 下，所以 STPMIC 这个芯片的相关信息就会在“i2c4”这个节点下进行描述。

第 54~281 行，这一大段为 STPMIC1A 这颗芯片的描述信息，第 54 行是“pmic: stpmic@33”，即使我们没有学习过设备树，但是也能看出这是和 PMIC 有关的。在这一大段代码中，描述了 STPMIC1A 各输出电源应该如何配置，比如电压应该是多少，每路电源的功能是什么等。

将示例代码 9.2.2.1 中第 54~281 行的内容全部删除掉，也就是删除掉 STPMIC1A 芯片的相关描述，删除以后的 i2c4 节点内容如图 9.2.2.1 所示：

```
45  &i2c4 {
46      pinctrl-names = "default";
47      pinctrl-0 = <&i2c4_pins_a>;
48      i2c-scl-rising-time-ns = <185>;
49      i2c-scl-falling-time-ns = <20>;
50      clock-frequency = <400000>;
51      status = "okay";
52      secure-status = "okay";
53  };
```

图 9.2.2.1 删除 pmic 信息后的 i2c4 节点

由于我们上面把 PMIC 相关的信息都删除了，但是其他设备的电源信息还是要有的，只是不通过 PMIC 来描述了。所以需要我们自己添加一些设备的电源描述信息，大家在 stm32mp157d-atk.dtsi 文件里面找到“vin”节点，如图 9.2.2.2 所示：

```
11  / {
12      memory@c0000000 {
13          device_type = "memory";
14          reg = <0xC0000000 0x40000000>;
15      };
16
17      vin: vin {
18          compatible = "regulator-fixed";
19          regulator-name = "vin";
20          regulator-min-microvolt = <5000000>;
21          regulator-max-microvolt = <5000000>;
22          regulator-always-on;
23      };
24  };
```

图 9.2.2.2 vin 节点内容

将图 9.2.2.2 中的 vin 节点内容全部删除掉，也就是将 17~23 行代码删除掉，替换为下面的代码：

示例代码 9.2.2.2 需要添加的电源节点信息

```
17 vddcore: regulator-vddcore {
18     compatible = "regulator-fixed";
19     regulator-name = "vddcore";
20     regulator-min-microvolt = <1200000>;
21     regulator-max-microvolt = <1350000>;
22     regulator-off-in-suspend;
23     regulator-always-on;
24 };
25
26 v3v3: regulator-3p3v {
27     compatible = "regulator-fixed";
28     regulator-name = "v3v3";
29     regulator-min-microvolt = <3300000>;
30     regulator-max-microvolt = <3300000>;
31     regulator-off-in-suspend;
32     regulator-always-on;
33 };
34
35 vdd: regulator-vdd {
36     compatible = "regulator-fixed";
```

```
37      regulator-name = "vdd";
38      regulator-min-microvolt = <3300000>;
39      regulator-max-microvolt = <3300000>;
40      regulator-off-in-suspend;
41      regulator-always-on;
42  };
43
44  vdd_usb: regulator-vdd-usb {
45      compatible = "regulator-fixed";
46      regulator-name = "vdd_usb";
47      regulator-min-microvolt = <3300000>;
48      regulator-max-microvolt = <3300000>;
49      regulator-off-in-suspend;
50      regulator-always-on;
51  };
```

第 17~24 行，描述 VDDCORE 电源，也就是 STM32MP157 的内核电源，最小为 1.2V，最大为 1.35V。

第 26~33 行，描述 3.3V 电源，最小和最大都是 3.3V

第 35~42 行，描述 VDD 电源，这是一个 3.3V 的电源，所以最小和最大都为 3.3V

第 44~51 行，描述 VDD_USB 电源，为 3.3V，所以最小和最大都为 3.3V。

修改完成以后如图 9.2.2.3 所示：

```
11  / {
12      memory@c0000000 {
13          device_type = "memory";
14          reg = <0xC0000000 0x40000000>;
15      };
16
17      vddcore: regulator-vddcore {
18          compatible = "regulator-fixed";
19          regulator-name = "vddcore";
20          regulator-min-microvolt = <1200000>;
21          regulator-max-microvolt = <1350000>;
22          regulator-off-in-suspend;
23          regulator-always-on;
24      };
25
26      v3v3: regulator-3p3v {
27          compatible = "regulator-fixed";
28          regulator-name = "v3v3";
29          regulator-min-microvolt = <3300000>;
30          regulator-max-microvolt = <3300000>;
31          regulator-off-in-suspend;
32          regulator-always-on;
33      };
34
35      vdd: regulator-vdd {
36          compatible = "regulator-fixed";
37          regulator-name = "vdd";
38          regulator-min-microvolt = <3300000>;
39          regulator-max-microvolt = <3300000>;
40          regulator-off-in-suspend;
41          regulator-always-on;
42      };
43
44      vdd_usb: regulator-vdd-usb {
45          compatible = "regulator-fixed";
46          regulator-name = "vdd_usb";
47          regulator-min-microvolt = <3300000>;
48          regulator-max-microvolt = <3300000>;
49          regulator-off-in-suspend;
50          regulator-always-on;
51      };
52  };
```

图 9.2.2.3 修改后的电源信息

至此，电源管理修改完成了，不要编译，会报错。因为修改了电源设置，而 USB OTG 节点还是引用的以前相关电源设置，所以编译会报错，等修改好 USB OTG 设备节点以后就可以编译了。

9.2.3 修改 TF 卡和 EMMC 设备树

继续操作 stm32mp157d-atk.dtsi 这个文件，找到“sdmmc1”和“sdmmc2”这 2 个节点，节点内容如图 9.2.3.1 所示：

```
225 &sdmmc1 {
226     pinctrl-names = "default";
227     pinctrl-0 = <&sdmmc1_b4_pins_a &sdmmc1_dir_pins_a>;
228     disable-wp;
229     st,sig-dir;
230     st,neg-edge;
231     st,use-ckin;
232     bus-width = <4>;
233     vmmc-supply = <&vdd_sd>;
234     sd-uhs-sdr12;
235     sd-uhs-sdr25;
236     sd-uhs-sdr50;
237     sd-uhs-ddr50;
238     sd-uhs-sdr104;
239     status = "okay";
240 };
241
242 &sdmmc2 {
243     pinctrl-names = "default";
244     pinctrl-0 = <&sdmmc2_b4_pins_a &sdmmc2_d47_pins_a>;
245     non-removable;
246     no-sd;
247     no-sdio;
248     st,neg-edge;
249     bus-width = <8>;
250     vmmc-supply = <&v3v3>;
251     vqmmc-supply = <&vdd>;
252     mmc-ddr-3_3v;
253     status = "okay";
254 };
```

图 9.2.3.1 sdmmc1 和 sdmmc2 原节点内容

将图 9.2.3.1 中的 sdmmc1 和 sdmmc2 节点改为如下所示内容：

示例代码 9.2.3.1 修改后的 sdmmc1 和 sdmmc2 节点内容

```
225 &sdmmc1 {
226     pinctrl-names = "default";
227     pinctrl-0 = <&sdmmc1_b4_pins_a &sdmmc1_dir_pins_a>;
228     st,neg-edge;
229     broken-cd;
230     bus-width = <4>;
231     vmmc-supply = <&v3v3>;
232     status = "okay";
233 };
234
235 &sdmmc2 {
236     pinctrl-names = "default";
237     pinctrl-0 = <&sdmmc2_b4_pins_a &sdmmc2_d47_pins_a>;
238     non-removable;
239     st,neg-edge;
240     bus-width = <8>;
241     vmmc-supply = <&v3v3>;
242     vqmmc-supply = <&v3v3>;
243     status = "okay";
244 };
```

修改完成以后 sdmmc1 和 sdmmc2 如图 9.2.3.2 所示：

```
225 &sdmmc1 {
226     pinctrl-names = "default";
227     pinctrl-0 = <&sdmmc1_b4_pins_a &sdmmc1_dir_pins_a>;
228     st,neg-edge;
229     broken-cd;
230     bus-width = <4>;
231     vmmc-supply = <&v3v3>;
232     status = "okay";
233 };
234
235 &sdmmc2 {
236     pinctrl-names = "default";
237     pinctrl-0 = <&sdmmc2_b4_pins_a &sdmmc2_d47_pins_a>;
238     non-removable;
239     st,neg-edge;
240     bus-width = <8>;
241     vmmc-supply = <&v3v3>;
242     vqmmc-supply = <&v3v3>;
243     status = "okay";
244 };
```

图 9.2.3.2 修改后的 sdmmc1 和 sdmmc2 节点

同样，不要编译，会报错，因为修改了电源，而 USB OTG 节点还是引用的以前相关电源设置，所以编译会报错，等修改好 USB OTG 设备节点以后就可以编译了。

9.2.4 修改 USB OTG 设备树

最后就是修改一下 USB OTG 对应的设备树节点，继续操作 stm32mp157d-atk.dtsi，找到“usbotg_hs”这个节点，节点默认内容如图 9.2.4.1 所示：

```
259 &usbotg_hs {
260     vbus-supply = <&vbus_otg>;
261 };
```

图 9.2.4.1 原来的节点内容

将图 9.2.4.1 中的 usbotg_hs 节点改为如下所示内容：

示例代码 9.2.4.1 修改后的 usbotg_hs 节点内容

```
259 &usbotg_hs {
260     phys = <&usbphyc_port1 0>;
261     phy-names = "usb2-phy";
262     usb-role-switch;
263     status = "okay";
264 };
```

最后还需要修改“usbphyc”节点里面的“status”属性值，看起来就是向 stm32mp157d-atk.dtsi 文件里面添加了一个名为“usbphyc”的节点，内容如下：

示例代码 9.2.4.2 修改后的 usbphyc 节点内容

```
266 &usbphyc {
267     status = "okay";
268 };
```

修改完成以后的 usbotg_hs 和 usbphyc 这两个节点如图 9.2.4.2 所示：

```
259  &usbotg_hs {
260      phys = <&usbphyc_port1 0>;
261      phy-names = "usb2-phy";
262      usb-role-switch;
263      status = "okay";
264  };
265
266  &usbphyc {
267      status = "okay";
268  };
```

图 9.2.4.2 修改后的 usbotg_hs 和 usbphyc

9.3 编译测试

9.3.1 编译

设备树修改完成以后先编译一下 TF-A，但是我们编译的时候默认会编译很多 STM32MP1 开发板的 TF-A 镜像，我们可以修改 Makefile.sdk 固件，只编译正点原子开发板对应的 TF-A 镜像。打开 Makefile.sdk 文件，将 TFA_DEVICETREE 修改为如图 9.3.1.1 所示内容：

```
21 # Set specific OEMAKE config
22 TF_A_CONFIG_OEMAKE =  trusted,"AARCH32_SP=sp_min" optee,"AARCH32_SP=optee" serialboot,"AARCH32_SP=sp_min"
23 # Set default TFA_DEVICETREE
24 #TFA_DEVICETREE ?=   stm32mp157d-atk stm32mp157a-dk1 stm32mp157d-dk1 stm32mp157c-dk2 stm32mp157f-dk2   st
   m32mp157c-ed1 stm32mp157f-ed1   stm32mp157a-ev1 stm32mp157c-ev1 stm32mp157d-ev1 stm32mp157f-ev1
25 TFA_DEVICETREE ?= stm32mp157d-atk        仅编译正点原子STM32MP157开发板对应的TF-A
26
```

图 9.3.1.1 修改后的 TFA_DEVICETREE

图 9.3.1.1 中的 TFA_DEVICETREE 用于指定需要编译的板子对应的设备树，这里改为之编译 stm32mp157d-atk，也就是正点原子开发板对应的设备树，也就是前面我们自己创建的 stm32mp157d-atk.dts 文件。

修改好以后重新编译 TF-A，如图 9.3.1.2 所示，表示编译成功：

```
Built /home/zuozhongkai/linux/atk-mp1/tf-a/my-tfa/tf-a-stm32mp-2.2.r1/../build/serialboot/
tf-a-stm32mp157d-atk.bin successfully


Generate /home/zuozhongkai/linux/atk-mp1/tf-a/my-tfa/tf-a-stm32mp-2.2.r1/../build/serialbo
ot/tf-a-stm32mp157d-atk.stm32
Image Type   : ST Microelectronics STM32 V1.0
Image Size   : 241728 bytes
Image Load   : 0x2ffc2500
Entry Point  : 0x2ffd6000
Checksum     : 0x00e2fa69
Option      : 0x00000001
Version     : 0x00000000
```

图 9.3.1.2 编译成功

图 9.3.1.2 中提示 tf-a-stm32mp157d-atk.bin 编译成功，而且也生成了 tf-a-stm32mp157d-atk.stm32 镜像文件。

9.3.2 烧写测试

使用 STM32CubeProgrammer 软件将上面编译出来的 build/trusted/tf-a-stm32mp157d-atk.stm32 烧写到开发板上的 EMMC 中并运行，运行 log 信息如图 9.3.2.1 所示：

```
6. COM33 (USB-SERIAL CH340 (COM
NOTICE:  CPU: STM32MP157DAA Rev.Z
NOTICE:  Model: STMicroelectronics STM32MP157D eval daughter
INFO:    Reset reason (0x14):
INFO:      Pad Reset from NRST
INFO:    Using EMMC
INFO:      Instance 2
INFO:    Boot used partition fsbl1
NOTICE:  BL2: v2.2-r1.0(debug):
NOTICE:  BL2: Built : 09:18:37, Nov 23 2020
INFO:    Using crypto library 'stm32_crypto_lib'
INFO:    BL2: Doing platform setup
INFO:    RAM: DDR3-DDR3L 32bits 533000Khz
INFO:    Memory size = 0x40000000 (1024 MB)
INFO:    BL2 runs SP_MIN setup
INFO:    BL2: Loading image id 4
INFO:    Loading image id=4 at address 0x2ffed000
INFO:    Image id=4 loaded: 0x2ffed000 - 0x2ffff000
INFO:    BL2: Loading image id 5
INFO:    Loading image id=5 at address 0xc0100000
INFO:    STM32 Image size : 512
INFO:    Image id=5 loaded: 0xc0100000 - 0xc0100200
WARNING: Skip signature check (header option)
NOTICE:  ROTPK is not deployed on platform. Skipping ROTPK verification.
NOTICE:  BL2: Booting BL32
INFO:    Entry point address = 0x2ffed000
INFO:    SPSR = 0x1d3
INFO:    Cannot find st,stpmic1 node in DT
NOTICE:  SP_MIN: v2.2-r1.0(debug):
NOTICE:  SP_MIN: Built : 09:18:48, Nov 23 2020
INFO:    ARM GICv2 driver initialized
INFO:    stm32mp IWDG1 (12): Secure
INFO:    ETZPC: CRYP1 (9) could be non secure
INFO:    SP_MIN: Initializing runtime services
INFO:    SP_MIN: Preparing exit to normal world
```

图 9.3.2.1 TF-A 运行 log 信息

从图 9.3.2.1 中可以看出 TF-A 运行成功，其中 bl2 和 bl32(sp_min)都正常运行，bl2 和 sp_min 的编译时间都为 2020 年 11 月 23 号，其中 bl2 编译时间为 9:18:37，sp_min 的编译时间为 9:18:48，bl2 先编译，SP_MIN 后编译。细心的朋友可能会发现,bl2 其实已经将 uboot 加载到了 0xc0100000 地址处，这是因为大家拿到的开发板默认已经烧写了出厂系统，所以 TF-A 可以加载 uboot(bl33)。

这个时候 uboot 可能会启动，也可能不会启动，这里先不要管 uboot 能不能启动。一会我们会讲解如何把 uboot 镜像也烧写进去，用来测试 TF-A 是否工作正常。

另外，由于正点原子的 STM32MP157 开发板出厂已经烧写系统，所以 bl2 可以加载 uboot 到 DDR 上，但是如果你用的是全新的核心板，EMMC 还没有烧写过任何系统，或者你把 EMMC 格式化过，因为 EMMC 里面没有 uboot 镜像，所以 bl2 在加载 uboot 的时候就会失败，TF-A 运行的 log 信息如图 9.3.2.2 所示：

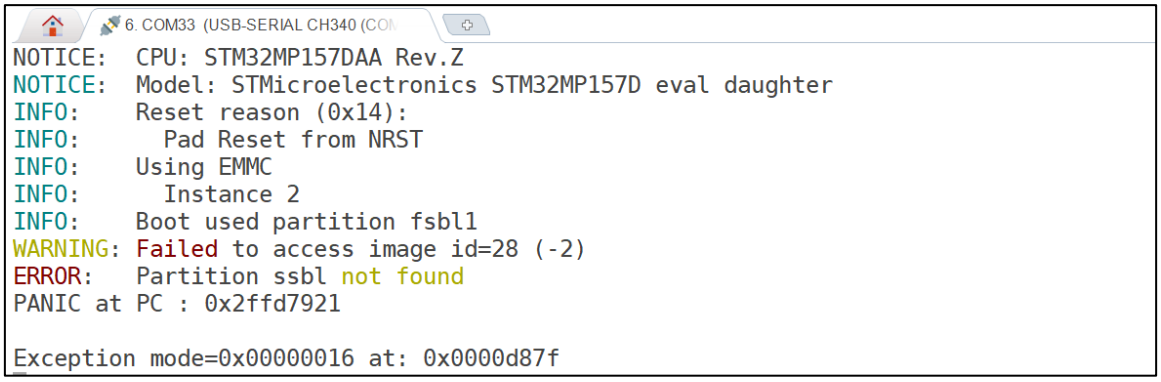


图 9.3.2.2 TF-A 运行错误

从图 9.3.2.2 中可以看出，TF-A 提示没有找到 ssbl 分区(ssbl 分区用于存放 uboot)，因为是全新的 EMMC 核心板，而我们烧写的时候也没有在 FlashLayout 中添加 ssbl 分区信息，所以会提示找不到 ssbl 分区，从而导致 TF-A 启动出问题。解决方法很简单，修改 FlashLayout，然后在里面添加一个 ssbl 分区，但是 ssbl 分区不烧写任何文件，就相当于新建一个名为“ssbl”的分区。打开 6.2.3 小节中我们创建的 FlashLayout 文件 tf-a.tsv，在最后一行添加如下内容：

示例代码 9.3.2.1 向 tf-a.tsv 添加 ssbl 分区信息

```
P      0x06     ssbl         Binary  mmc1     0x00080000
```

上面这一行“PE”表示不更新 ssbl 分区，但是如果此时还没有 ssbl 分区的话会先创建一个 ssbl 分区。其实就是在 EMMC 里面新建一个空的 ssbl 分区而已，添加完成以后的 tf-a.tsv 内容如图 9.3.2.3 所示：

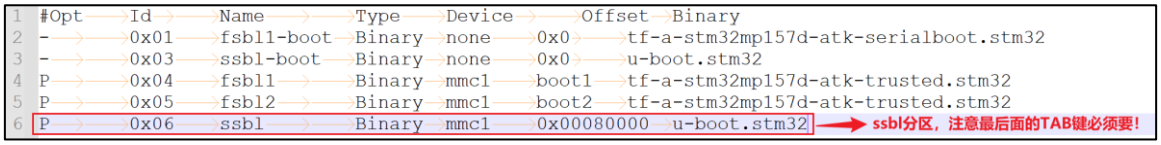


图 9.3.2.3 修改后的 tf-a.tsv 文件

图 9.3.2.3 中最后一行就是 ssbl 分区，注意 0x00080000 后面的箭头表示 TAB 键，这个 TAB 键是必须要的！因为最后面还有 Binary 列的内容，这里我们并没有给出 Binary 列的镜像文件名，表示不烧写任何文件，但是位置得留出来，否则烧写的时候 STM32CubeProgrammer 会 tf-a.tsv 语法错误。

修改完成以后使用新的 tf-a.tsv 来烧写 TF-A 镜像，烧写完成以后 TF-A 运行 log 信息如图 9.3.2.4 所示：

```
6. COM33 (USB-SERIAL CH340 (COM
NOTICE:  CPU: STM32MP157DAA Rev.Z
NOTICE:  Model: STMicroelectronics STM32MP157D eval daughter
INFO:    Reset reason (0x14):
INFO:      Pad Reset from NRST
INFO:    Using EMMC
INFO:      Instance 2
INFO:    Boot used partition fsbl1
NOTICE:  BL2: v2.2-r1.0(debug):
NOTICE:  BL2: Built : 09:18:37, Nov 23 2020
INFO:    Using crypto library 'stm32_crypto_lib'
INFO:    BL2: Doing platform setup
INFO:    RAM: DDR3-DDR3L 32bits 533000Khz
INFO:    Memory size = 0x40000000 (1024 MB)
INFO:    BL2 runs SP_MIN setup
INFO:    BL2: Loading image id 4
INFO:    Loading image id=4 at address 0x2ffed000
INFO:    Image id=4 loaded: 0x2ffed000 - 0x2ffff000
INFO:    BL2: Loading image id 5
INFO:    Loading image id=5 at address 0xc0100000
WARNING: Failed to determine the size of the image id=5 (-12)
ERROR:   BL2: Failed to load image (-12)
```

图 9.3.2.4 TF-A 运行结果

从图 9.3.2.4 可以看出，此时没有再提示 ssbl 分区错误了，但是有另外一个错误，那就是在加载 ID=5 的镜像的时候加载错误，这个就是 uboot 镜像。因为全新的 EMMC 并没有 uboot 镜像，因此肯定加载不到，除非把 uboot 镜像也烧写进去，也就是烧写到上面的 ssbl 分区里面。

修改 tf-a.tsv 文件，在 ssbl 分区添加要烧写的 uboot 镜像即可，这里直接烧写正点原子官方出厂系统的 uboot 镜像即可，也就是 u-boot.stm32，这个文件已经放到了我们烧写目录里面，修改后的 tf-a.tsv 文件如图 9.3.2.5 所示：

```
tf-a.tsv
1  #Opt  Id    Name        Type    Device  Offset      Binary
2  -     0x01  fsbl1-boot  Binary  none    0x0         tf-a-stm32mp157d-atk-serialboot.stm32
3  -     0x03  ssbl-boot   Binary  none    0x0         u-boot.stm32
4  P     0x04  fsbl1       Binary  mmc1    boot1       tf-a-stm32mp157d-atk-trusted.stm32
5  P     0x05  fsbl2       Binary  mmc1    boot2       tf-a-stm32mp157d-atk-trusted.stm32
6  P     0x06  ssbl        Binary  mmc1    0x00080000  u-boot.stm32
```

将uboot镜像烧写到ssbl分区

图 9.3.2.5 修改后的 tf-a.tsv

图 9.3.2.5 中最后一行的 Opt 要改为‘P’，也就是更新 ssbl 分区中的文件，u-boot.stm32 要和 tf-a.tsv 放在同一个目录下。修改完成以后使用新的 tf-a.tsv 烧写 TF-A 和 uboot 镜像到开发板的 EMMC 里面，烧写完成以后运行 TF-A，运行过程的 log 信息如图 9.3.2.6 所示：

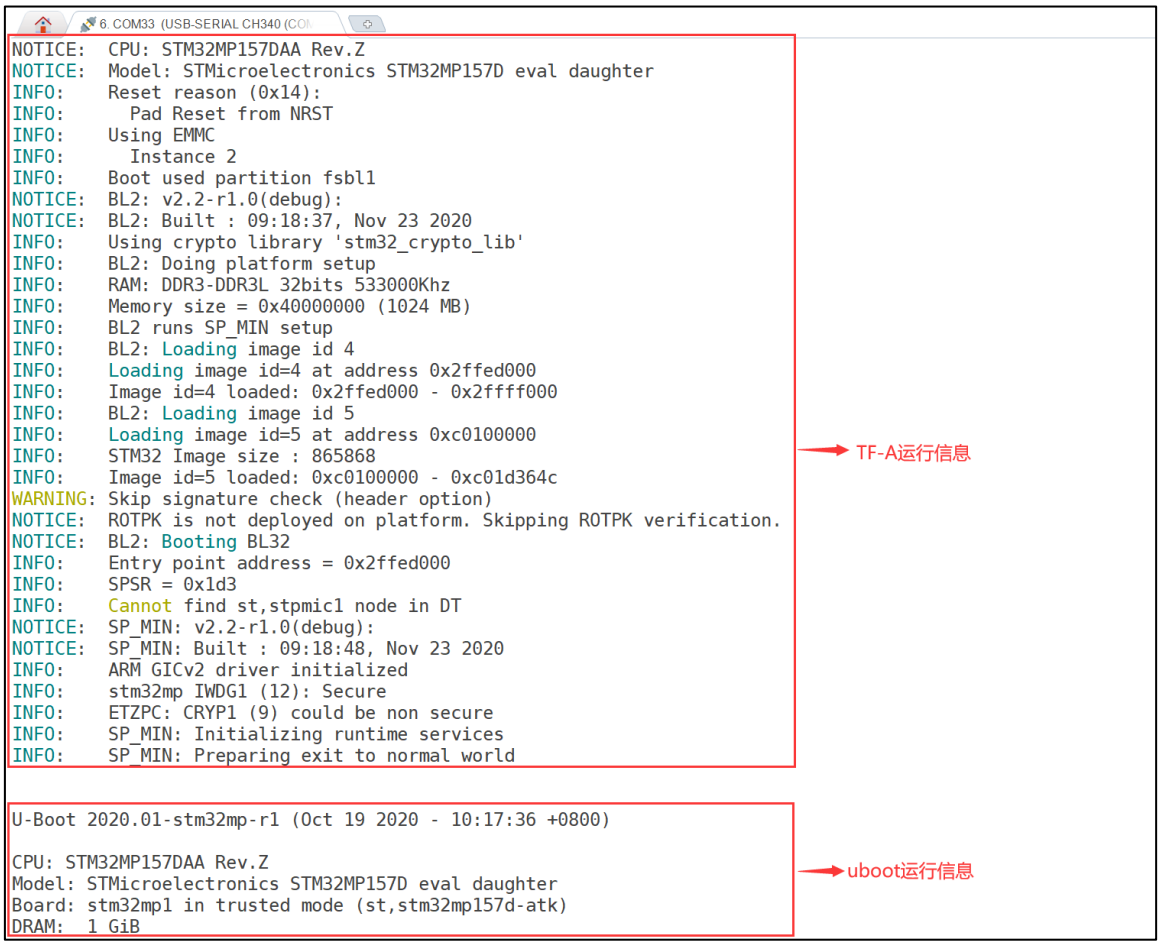


图 9.3.2.6 TF-A 运行信息

从图 9.2.3.6 可以看出，TF-A 运行正常，而且因为我们也烧写了 uboot 镜像，因此 uboot 也启动了，说明 TF-A 运行正常。

9.3.3 xxx-serialboot.stm32 测试

在 6.2.3 小节中我们构建的系统烧写目录中，用到了正点原子出厂镜像中的 stm32mp157d-atk-serialboot.stm32，前面讲了，STM32CubeProgrammer 首先通过 USB 或串口向开发板下载 stm32mp157d-atk-serialboot.stm32，名字里面有个“serialboot”，翻译过来就是串行启动，此镜像主要用于初始化 DDR，并且提供 USB 或串口功能，目的是为了进一步将 uboot 镜像下载到 DDR 的指定位置，最终通过 uboot 来向外部 flash 设备烧写整个系统镜像。

很明显我们也需要掌握自行编译 stm32mp157d-atk-serialboot.stm32 的方法，操作很简单。首先打开 Makefile.sdk，然后将 EXTRA_OEMAKE_SERIAL 改为如下内容：

示例代码 9.3.3.1 EXTRA_OEMAKE_SERIAL 编译选项

```
EXTRA_OEMAKE_SERIAL=$(filter-out STM32MP_SDMMC=1 STM32MP_EMMC=1 STM32MP_SPI_NOR=1 STM32MP_RAW_NAND=1 STM32MP_SPI_NAND=1,$(EXTRA_OEMAKE)) STM32MP_UART_PROGRAMMER=1 STM32MP_USB_PROGRAMMER=1
```

修改完成以后如图 9.3.3.1 所示：

```
14 EXTRA_OEMAKE=CROSS_COMPILE=arm-none-linux-gnueabihf- DEBUG=1 LOG_LEVEL=40 PLAT=stm32mp1 ARCH=aarch32
    ARM_ARCH_MAJOR=7 STM32MP_SDMMC=1 STM32MP_EMMC=1 STM32MP_SPI_NOR=1 STM32MP_RAW_NAND=1 STM32MP_SPI_NA
   ND=1
15 #EXTRA_OEMAKE_SERIAL= STM32MP_UART_PROGRAMMER=1 STM32MP_USB_PROGRAMMER=1
16 EXTRA_OEMAKE_SERIAL=$(filter-out STM32MP_SDMMC=1 STM32MP_EMMC=1 STM32MP_SPI_NOR=1 STM32MP_RAW_NAND=1
    STM32MP_SPI_NAND=1,$(EXTRA_OEMAKE)) STM32MP_UART_PROGRAMMER=1 STM32MP_USB_PROGRAMMER=1
17
```

图 9.3.3.1 修改后的 EXTRA_OEMAKE_SERIAL 配置选项

输入如下命令编译：

```
cd tf-a-stm32mp-2.2.r1/                    //进入到 tf-a 源码目录下
make -f ../Makefile.sdk clean              //清除以前的编译
make -f ../Makefile.sdk TFA_DEVICETREE=stm32mp157d-atk TF_A_CONFIG=serialboot
ELF_DEBUG_ENABLE='1' all                   //编译
```

编译完成以后就会在../build/serialboot 目录下生成 tf-a-stm32mp157d-atk-serialboot.stm32，如图 9.3.3.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a/my-tfa/build/serialboot$ ls
bl2                                      libc                    tf-a-stm32mp157d-atk.bin
bl2.bin                                  libfdt                  tf-a-stm32mp157d-atk.elf
bl32                                     libwrapper              tf-a-stm32mp157d-atk.ld
bl32.bin                                 romlib                  tf-a-stm32mp157d-atk.map
debug-tf-a-stm32mp157d-atk-serialboot.stm32  stm32mp1.o           tf-a-stm32mp157d-atk-serialboot.stm32
fdts                                     tf-a-bl2-serialboot.elf tf-a-stm32mp157d-atk.stm32
lib                                      tf-a-bl32-serialboot.elf
zuozhongkai@ubuntu:~/linux/atk-mp1/tf-a/my-tfa/build/serialboot$
```

图 9.3.3.2 编译生成的 tf-a-stm32mp157d-atk-serialboot.stm32 文件

用图 9.3.3.2 中的 tf-a-stm32mp157d-atk-serialboot.stm32 替换我们以前使用的正点原子出厂系统中的文件，替换完成以后的烧写目录如图 9.3.3.3 所示：

图 9.3.3.3 images 目录

此时图 9.3.3.3 中的 tf-a-stm32mp157d-atk-serialboot.stm32 和 tf-a-stm32mp157d-atk-trusted.stm32 均为我们自行编译的，tf-a-stm32mp157d-atk-trusted.stm32 经过测试运行正常，接下来就要测试一下 tf-a-stm32mp157d-atk-serialboot.stm32 是否能正常烧写系统。

直接用 STM32CubeProgrammer 烧写测试即可，观察是否能正常烧写，烧写完成以后 tf-a-stm32mp157d-atk-trusted.stm32 是否能够正常启动。

注意！按照本小节讲解的方法编译生成 tf-a-stm32mp157d-atk-serialboot.stm32 以后，../build/trusted 和../build/optee 目录就没了，也就是说此本小节的方法只能编译 tf-a-stm32mp157d-atk-serialboot.stm32。一旦编译成功并测试 OK 以后，大家就不要再去修改 stm32mp157d-atk-serialboot.stm32 了。如果想重新编译 tf-a-stm32mp157d-atk-trusted.stm32，那么就按照以前的编译命令即可，不用修改 Makefile.sdk 文件！

至此，TF-A 移植就全部完成。

第十章 U-boot 使用

在移植 U-Boot 之前，我们肯定要先使用一下 U-Boot，得先体验一下 U-Boot 是个什么东西。STM32MP157 开发板光盘资料里面已经提供了一个正点原子团队已经移植好的 U-Boot，本章我们就直接编译这个移植好的 U-Boot，然后烧写到 EMMC 里面启动，启动 U-Boot 以后就可以学习使用 U-Boot 的命令。

10.1 U-Boot 简介

Linux 系统要启动需要通过 bootloader 程序引导，也就说芯片上电以后先运行一段 bootloader 程序。这段 bootloader 程序会先初始化 DDR 等外设，然后将 Linux 内核从 flash(NAND，NOR FLASH，SD，EMMC 等)拷贝到 DDR 中，最后启动 Linux 内核。当然了，bootloader 的实际工作要复杂的多，但是它最主要的工作就是启动 Linux 内核，bootloader 和 Linux 内核的关系就跟 PC 上的 BIOS 和 Windows 的关系一样，bootloader 就相当于 BIOS。所以我们要先搞定 bootloader，很庆幸，有很多现成的 bootloader 软件可以使用，比如 U-Boot、vivi、RedBoot 等等，其中以 U-Boot 使用最为广泛，为了方便书写，本书会将 U-Boot 写为 uboot。

uboot 的全称是 Universal Boot Loader，uboot 是一个遵循 GPL 协议的开源软件，uboot 是一个裸机代码，可以看作是一个裸机综合例程。现在的 uboot 已经支持液晶屏、网络、USB 等高级功能。uboot 官网为 http://www.denx.de/wiki/U-Boot/，如图 10.1.1 所示：

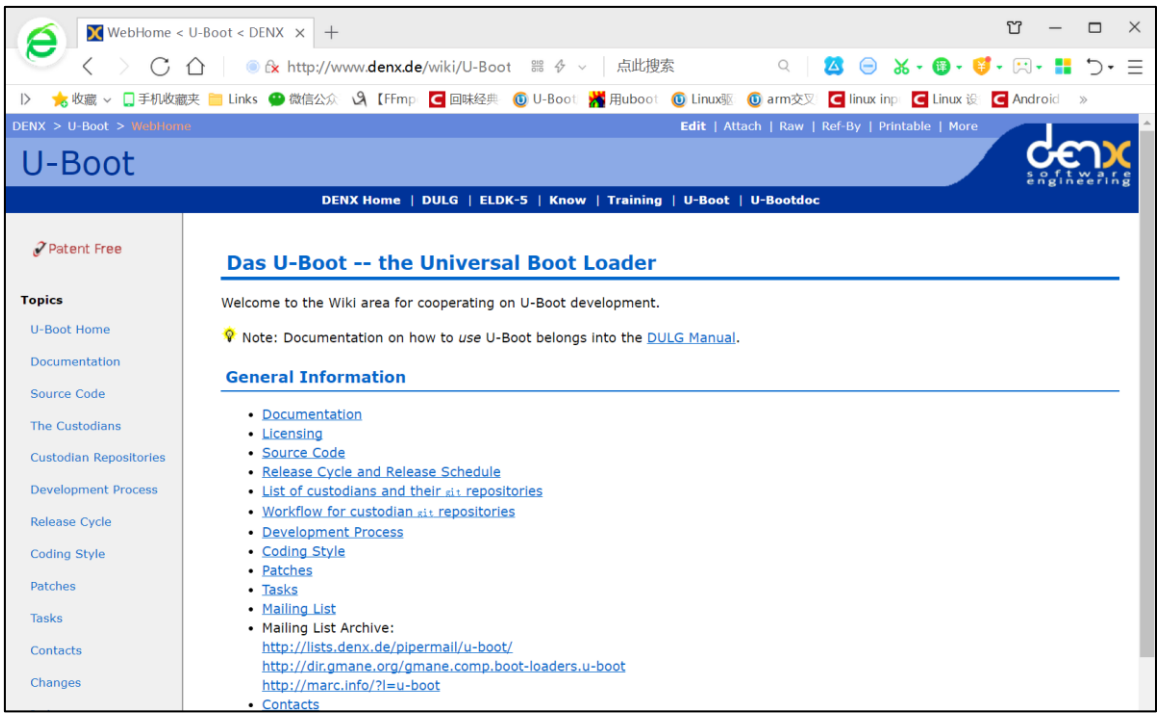


图 10.1.1 uboot 官网

我们可以在 uboot 官网下载 uboot 源码，点击图 10.1.1 中左侧 Topics 中的“Source Code”，打开以后如图 10.1.2 所示：

图 10.1.2 uboot 源码界面

点击图 10.1.2 中的“FTP”，进入其 FTP 服务器即可看到 uboot 源码，如图 10.1.3 所示：

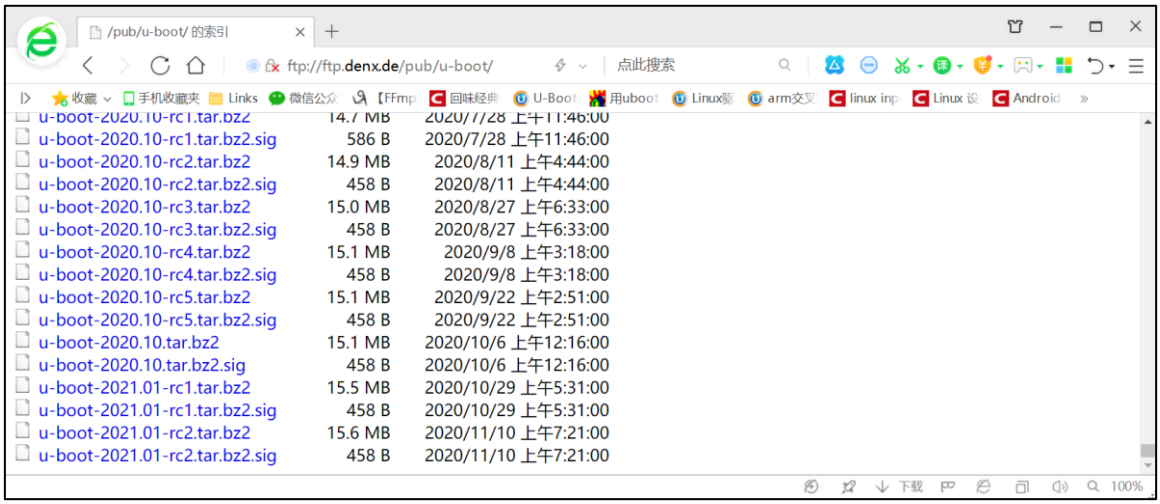


图 10.1.3 uboot 源码

图 10.1.3 中就是 uboot 原汁原味的源码文件，目前最新的版本是 2020.10。但是我们一般不会直接用 uboot 官方的 U-Boot 源码的。uboot 官方的 uboot 源码是给半导体厂商准备的，半导体厂商会下载 uboot 官方的 uboot 源码，然后将自家相应的芯片移植进去。也就是说半导体厂商会自己维护一个版本的 uboot，这个版本的 uboot 相当于是他们定制的。既然是定制的，那么肯定对自家的芯片支持会很全，虽然 uboot 官网的源码中一般也会支持他们的芯片，但是绝对是没有半导体厂商自己维护的 uboot 全面。ST 提供了 2020.01 版本的 uboot，在 6.1.1 小节获取 ST 官方系统源码中我们已经得到了 ST 官方 uboot 源码， 进入到如下目录：

/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi

uboot 源码如图 10.1.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi$ ls
linux-stm32mp-5.4.31-r0       tf-a-stm32mp-2.2.r1-r0      u-boot-stm32mp-2020.01-r0
optee-os-stm32mp-3.9.0.r1-r0  tf-a-stm32mp-ssp-2.2.r1-r0
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi$
```

图 10.1.4 ST 官方 uboot 源码

图 10.1.4 中的“u-boot-stm32mp-2020.01-r0”就是 ST 官方 uboot 源码包，它支持了 STM32MP1 家族全系列芯片(后续 ST 也会一直更新，添加新的 SOC 进去)，而且支持各种启动方式，

比如 EMMC、NAND 等等，这些都是 uboot 官方所不支持的。但是图 10.1.4 中的 uboot 是针对 ST 自家评估板的，如果要在我们自己的板子上跑，那么就需要对其进行修改，使其支持我们自己做的板子，正点原子的 STM32MP157 开发板就是自己做的板子，虽然大部分都参考了 ST 官方的 STM32MP157 EVK 开发板，但是还是有很多不同的地方，所以需要修改 ST 官方的 uboot，使其适配正点原子的 STM32MP157 开发板。所以当我们拿到开发板以后，是有三种 uboot 的，这三种 uboot 的区别如表 10.1.1 所示：

种类	描述
uboot 官方的 uboot 代码	由 uboot 官方维护开发的 uboot 版本，版本更新快，基本包含所有常用的芯片。
半导体厂商的 uboot 代码	半导体厂商维护的一个 uboot，专门针对自家的芯片，在对自家芯片支持上要比 uboot 官方的好。
开发板厂商的 uboot 代码	开发板厂商在半导体厂商提供的 uboot 基础上加入了对自家开发板的支持。

表 10.1.1 三种 uboot 的区别

那么这三种 uboot 该如何选择呢？首先 uboot 官方的基本是不会用的，因为支持太弱了。最常用的就是半导体厂商或者开发板厂商的 uboot，如果你用的半导体厂商的评估板，那么就使用半导体厂商的 uboot，如果你是购买的第三方开发板，比如正点原子的 STM32MP157 开发板，那么就使用正点原子提供的 uboot 源码（也是在半导体厂商的 uboot 上修改的）。当然了，你也可以在购买了第三方开发板以后使用半导体厂商提供的 uboot，只不过有些外设驱动可能不支持，需要自己移植，这个就是我们常说的 uboot 移植。本节是 uboot 的使用，所以就直接使用正点原子已经移植好的 uboot，这个已经放到了开发板光盘中了，路径为：开发板光盘→1、程序源码→1、正点原子 Linux 出厂系统源码→u-boot-stm32mp-2020.01-gdb8d2374-v1.0.tar.bz2。

10.2 U-Boot 初次编译

10.2.1 编译

首先需要在 Ubuntu 中安装一些库，否则编译 uboot 会报错，安装命令如下：

```
sudo apt-get install libncurses5-dev bison flex
```

在 Ubuntu 中创建存放 uboot 的目录，比如我这里新建了一个名为“alientek_uboot”的文件夹用于存放正点原子提供的 uboot 源码。alientek_uboot 文件夹创建成功以后使用 FileZilla 软件将正点原子提供的 uboot 源码拷贝到此目录中，正点原子提供的 uboot 源码已经放到了开发板光盘中，路径为：开发板光盘→1、程序源码→1、正点原子 Linux 出厂系统源码→u-boot-stm32mp-2020.01-xxxxxxxx-v1.0.tar.bz2（“xxxxxxxx”为 uboot 打包时候的版本号，每次打包其版本号都不同！所以大家不要纠结于开发板光盘中的 uboot 源码打包版本号是否和教程里面的一致）。将 stm32mp-2020.01-xxxxxxxx-v1.0.tar.bz2 拷贝到前面新建的 alientek_uboot 文件夹下，完成以后如图 10.2.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ ls
u-boot-stm32mp-2020.01-gdb8d2374-v1.0.tar.bz2
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 10.2.1.1 正点原子出厂 uboot 源码

使用如下命令对其进行解压缩:

```
tar -vxf u-boot-stm32mp-2020.01-gdb8d2374-v1.0.tar.bz2
```

解压完成以后如图 10.2.1.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ ls
api          configs          env        lib          README
arch         CONTRIBUTING.md  examples   Licenses     scripts
board        disk             fs         MAINTAINERS  test
cmd          doc              include    Makefile     tools
common       drivers          Kbuild     net          u-boot-stm32mp-2020.01-gdb8d2374-v1.0.tar.bz2
config.mk    dts              Kconfig    post
```

图 10.2.1.2 解压后的 Uboot

图 10.2.1.2 中除了 u-boot-stm32mp-2020.01-gdb8d2374-v1.0.tar.bz2 这个正点原子提供 uboot 源码压缩包以外，其他的文件和文件夹都是解压出来的 uboot 源码。执行以下命令，编译正点原子提供的 uboot。

```
make distclean
make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- stm32mp157d_atk_defconfig
make V=1 ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- DEVICE_TREE=stm32mp157d-atk all
```

上面命令每次编译的时候都要指定 ARCH、CROSS_COMPILE 和 DEVICE_TREE，这三个含义如下：

ARCH：指定所使用的平台架构，这里肯定是 arm。

CROSS_COMPILE：所使用的交叉编译器前缀，本教程使用的是交叉编译器前缀为 arm-none-linux-gnueabihf-。

DEVICE_TREE：设备树文件，uboot 也支持设备树，所以在编译的时候需要指定设备树文件，不同的硬件其设备树文件肯定不同，这里为 stm32mp157d_atk，也就是正点原子的 STM32MP157 开发板对应的设备树。

编译的时候每次都输入 ARCH 和 CROSS_COMPILE 比较麻烦，为了方便起见，我们可以直接修改 uboot 的 Makefile 文件，在里面直接对 ARCH 和 CROSS_COMPILE 进行赋值，也就是直接将 ARCH 设置为 arm，CROSS_COMPILE 设置为 arm-none-linux-gnueabihf-，修改完成以后如图 10.2.1.3 所示：

```
261 # set default to nothing for native builds
262 ifeq ($(HOSTARCH),$(ARCH))
263 CROSS_COMPILE ?=
264 endif
265
266 ARCH = arm
267 CROSS_COMPILE = arm-none-linux-gnueabihf-
268
```

设置ARCH和CROSS_COMPILE的值

图 10.2.1.3 设置 ARCH 和 CROSS_COMPILE 值

注意！不能在 Makefile 里面对 DEVICE_TREE 进行复制，因为没用，必须在编译的时候手动输入！

设置好 Makefile 里面的 ARCH 和 CROSS_COMPILE 以后就可以将编译命令简化为如下所示：

```
make distclean                              //清除
make stm32mp157d_atk_defconfig              //配置 uboot
make V=1 DEVICE_TREE=stm32mp157d-atk all    //编译
```

上述命令和前面的相比就要简洁很多，最后的“make V=1”是真正的编译命令，V=1 表示编译 uboot 的时候输出详细的编译过程，方便我们观察 uboot 编译过程。直接输入“make”命令的话默认使用单线程编译，编译速度会比较慢，可以通过添加“-j”选项来使用多线程编译，比如使用 8 线程编译，最后的编译命令就是：

```
make V=1 DEVICE_TREE=stm32mp157d-atk all -j8          //8 线程编译
```

uboot 编译完成如图 10.2.1.4 所示：

```
  cat u-boot-nodtb.bin dts/dt.dtb > u-boot-dtb.bin
  cp u-boot-dtb.bin u-boot.bin
  ./tools/mkimage -T stm32image -a 0xC0100000 -e 0xC0100000 -d u-boot.bin u-boot.stm32 >u-boot.s
tm32.log  && cat u-boot.stm32.log
Image Type   : STMicroelectronics STM32 V1.0
Image Size   : 876085 bytes
Image Load   : 0xc0100000
Entry Point  : 0xc0100000
Checksum     : 0x04ae3f76
Option      : 0x00000001
BinaryType : 0x00000000
  ./scripts/check-config.sh u-boot.cfg ./scripts/config_whitelist.txt .
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek uboot$
```

图 10.2.1.4 uboot 编译成功

编译完成以后的 就会在 uboot 源码目录下生成相应的镜像文件，如图 10.2.1.5 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ ls
api          CONTRIBUTING.md  fs          Makefile    tools              u-boot.lds
arch         disk             include     net         u-boot             u-boot.map
board        doc              Kbuild      post        u-boot.bin         u-boot-nodtb.bin
cmd          drivers          Kconfig     README      u-boot.cfg         u-boot.srec
common       dts              lib         scripts     u-boot.cfg.configs u-boot.stm32
config.mk    env              Licenses    System.map  u-boot.dtb         u-boot.stm32.log
configs      examples         MAINTAINERS test        u-boot-dtb.bin     u-boot.sym
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 10.2.1.5 编译后生成的 uboot 可执行文件

可以看出，编译完成以后 uboot 源码多了一些文件，重点是 u-boot.bin 和 u-boot.stm32 这两个文件。u-boot.bin 是 uboot 的二进制可执行文件，u-boot.stm32 是在 u-boot.bin 前面添加了 256 个字节头部信息。STM32MP1 内部 ROM 代码和 TF-A 在运行 uboot 的时候要求前面添加头部信息，所以这就是为什么 uboot 也有这个头部信息的原因。

10.2.2 烧写

使用 STM32CubeProgrammer 将上面编译出来的 u-boot.stm32 镜像烧写到开发板的 EMMC 里面，修改前面创建的 tf-a.tsv 文件，添加 uboot 烧写指令(其实在 9.3.2 小节已经讲过了)，在最后面添加下面这行：

示例代码 10.2.2.1 uboot 烧写指令

```
P       0x06     ssbl         Binary  mmc1     0x00080000   u-boot.stm32
```

修改以后的 tf-a.tsv 如图 10.2.2.1 所示：

```
tf-a.tsv
1  #Opt    Id      Name        Type    Device    Offset      Binary
2  -       0x01    fsbl1-boot  Binary  none      0x0         tf-a-stm32mp157d-atk-serialboot.stm32
3  -       0x03    ssbl-boot   Binary  none      0x0         u-boot.stm32
4  P       0x04    fsbl1       Binary  mmc1      boot1       tf-a-stm32mp157d-atk-trusted.stm32
5  P       0x05    fsbl2       Binary  mmc1      boot2       tf-a-stm32mp157d-atk-trusted.stm32
6  P       0x06    ssbl        Binary  mmc1      0x00080000  u-boot.stm32   <-- uboot镜像烧写命令
```

图 10.2.2.1 修改后的 tf-a.tsv

最后将上一小节编译出来的 u-boot.stm32，拷贝到前面创建的 images 目录下（在做 TF-A 实验的就有 u-boot.stm32 这个文件，我们只要替换就行）。

一切准备就绪以后就可以使用 STM32CubeProgrammer 软件通过 USB OTG 将 uboot 烧写到开发板上的 EMMC 里面，等到烧写完成。完成以后设置开发板上的拨码开关，设置从 EMMC 启动，然后用 USB Type-C 线将开发板上的 USB_TTL 接口与电脑连接起来，因为我们要在串口终端里面输入命令来操作 uboot。

打开 MobaXterm，设置好串口参数，最后复位开发板。在 MobaXterm 上出现“Hit any key tostop autoboot: ”倒计时的时候按下键盘上的回车键，默认是 1 秒倒计时，在 1 秒倒计时结束以后如果没有按下回车键的话 uboot 就会使用默认参数来启动 Linux 内核了（如果内核存在的话，如果 Linux 内核不存在那么就会进入到 uboot 的命令行模式）。如果在 1 秒倒计时结束之前按下回车键，那么就会进入 uboot 的命令行模式，如图 10.2.2.2 所示：

```
U-Boot 2020.01-stm32mp-r1 (Nov 24 2020 - 17:17:20 +0800)

CPU: STM32MP157DAA Rev.Z
Model: STMicroelectronics STM32MP157D eval daughter
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)
DRAM:  1 GiB
Clocks:
- MPU : 800 MHz
- MCU : 208.878 MHz
- AXI : 266.500 MHz
- PER : 24 MHz
- DDR : 533 MHz
WDT:   Started with servicing (32s timeout)
NAND:  0 MiB
MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
Loading Environment from MMC... OK
In:    serial
Out:   serial
Err:   serial
invalid MAC address in OTP 00:00:00:00:00:00
Net:
Error: ethernet@5800a000 address not set.
No ethernet found.

Hit any key to stop autoboot:  0
STM32MP>
```

图 10.2.2.2 uboot 启动 log 信息

从图 10.2.2.2 可以看出，当进入到 uboot 的命令行模式以后，左侧会出现一个“STM32MP=>”标志。uboot 启动的时候会输出一些信息，这些信息如下所示：

示例代码 10.2.2.2 uboot 启动 log 信息

```
1  U-Boot 2020.01-stm32mp-r1 (Nov 24 2020 - 17:17:20 +0800)
2
3  CPU: STM32MP157DAA Rev.Z
4  Model: STMicroelectronics STM32MP157D eval daughter
5  Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)
6  DRAM:  1 GiB
7  Clocks:
8  - MPU : 800 MHz
9  - MCU : 208.878 MHz
10 - AXI : 266.500 MHz
11 - PER : 24 MHz
12 - DDR : 533 MHz
13 WDT:   Started with servicing (32s timeout)
14 NAND:  0 MiB
15 MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
16 Loading Environment from MMC... OK
17 In:    serial
```

```
18 Out:   serial
19 Err:   serial
20 invalid MAC address in OTP 00:00:00:00:00:00
21 Net:
22 Error: ethernet@5800a000 address not set.
23 No ethernet found.
24
25 Hit any key to stop autoboot:  0
26 STM32MP>
```

简单讲解一下 uboot 启动过程的 log 信息：

第 1 行是 uboot 版本号和编译时间，可以看出，当前的 uboot 版本号是 2020.01，编译时间是 2020 年 11 月 24 日 17: 17。

第 3 行是 CPU 的信息，可以看出 CPU 型号为 STM32MP157DAA。

第 4 行是板子信息，当前板子是 ST 公司的 STM32MP157D eval 开发板，这个信息是可以改的，因为正点原子是直接参考 ST 公司的 EVK 开发板移植的 uboot，所以这部分信息也就没改。

第 5 行是板子的一些信息，比如工作在 trusted 模式下。

第 6 行是 DDR 的大小为 1GB。

第 7~12 行它们的频率分别为，MPU 频率、MCU 频率、AXI 总线频率、PER 的频率、DDR 频率。

第 13 行是看门狗信息，喂狗时间为 32s。

第 14 行是 NAND 的大小，因为正点原子的 STM32MP157 开发板没有 NAND，所以这里就是 0MB。

第 15 行是板子上 MMC 设备，一共有两个，SD/MMC0 (SD 卡)和 SD/MMC1 (EMMC)。

第 16 行是从 MMC 里获取环境变量。

第 17~19 行是标准输入、标准输出和标准错误所使用的终端，这里都使用串口(serial)作为终端。

第 20~23 行是网络相关信息，网络的 MAC 地址从 OTP 里获取，因为我们的 OTP 没有设置 MAC 地址，所以就获取失败。这里的网络是可以用的，只是因为没有 MAC 地址所以提示没有找到网络，可以自行添加相关环境变量来设置 MAC 地址，后面会讲如何设置。

第 25 行是倒计时提示，默认倒计时 1 秒，倒计时结束之前按下回车键就会进入 Linux 命令行模式。如果在倒计时结束以后没有按下回车键，那么 Linux 内核就会启动，Linux 内核一旦启动，uboot 就会寿终正寝。

uboot 的主要作用是引导 kernel，我们现在已经进入 uboot 的命令行模式了，进入命令行模式以后就可以给 uboot 发号施令了。当然了，不能随便发号施令，得看看 uboot 支持哪些命令，然后使用这些 uboot 所支持的命令来做一些工作，下一节就讲解 uboot 命令的使用。

10.3 U-Boot 命令使用

进入 uboot 的命令行模式以后输入“help”或者“？”，然后按下回车即可查看当前 uboot 所支持的命令，如图 10.3.1 所示：

```
STM32MP> help
?          - alias for 'help'
adc        - ADC sub-system
base       - print or set address offset
bdinfo     - print Board Info structure
blkcache   - block cache diagnostics and control
bmp        - manipulate BMP image data
bootcount  - bootcount
bootefi    - Boots an EFI payload from memory
bootm      - boot application image from memory
bootp      - boot image via network using BOOTP/TFTP protocol
bootstage  - Boot stage command
bootz      - boot Linux zImage image from memory
chpart     - change active partition
clk        - CLK sub-system
cls        - clear screen
cmp        - memory compare
coninfo    - print console devices and information
cp         - memory copy
crc32      - checksum calculation
date       - get/set/reset date & time
dcache     - enable or disable data cache
dfu        - Device Firmware Upgrade
dhcp       - boot image via network using DHCP/TFTP protocol
dm         - Driver model low level access
dtimg      - manipulate dtb/dtbo Android image
```

图 10.3.1 uboot 的命令列表(部分)

图 10.3.1 中只是 uboot 的一部分命令，具体的命令列表以实际为准。图 10.3.1 中的命令并不是 uboot 所支持的所有命令，说过 uboot 是可配置的，需要什么命令就使能什么命令。所以图 10.3.1 中的命令是正点原子提供的 uboot 中使能的命令，uboot 支持的命令还有很多，而且也可以在 uboot 中自定义命令。这些命令后面都跟有命令说明，用于描述此命令的作用，但是命令具体怎么用呢？我们输入“help(或?) 命令名”既可以查看命令的详细用法，以“bootz”这个命令为例，我们输入如下命令即可查看“bootz”这个命令的用法：

```
? bootz 或 help bootz
```

结果如图 10.3.2 所示：

```
STM32MP> ? bootz
bootz - boot Linux zImage image from memory

Usage:
bootz [addr [initrd[:size]] [fdt]]
    - boot Linux zImage stored in memory
        The argument 'initrd' is optional and specifies the address
        of the initrd in memory. The optional argument ':size' allows
        specifying the size of RAW initrd.
        When booting a Linux kernel which requires a flat device-tree
        a third argument is required which is the address of the
        device-tree blob. To boot that kernel without an initrd image,
        use a '-' for the second argument. If you do not pass a third
        a bd_info struct will be passed instead

STM32MP>
```

图 10.3.2 bootz 命令使用说明

图 10.3.2 列出了“bootz”这个命令的详细说明，其它的命令也可以使用此方法查询具体的使用方法。接下来我们学习一下一些常用的 uboot 命令。

10.3.1 查询命令

常用的和信息查询有关的命令有 3 个：bdinfo、printenv 和 version。先来看一下 bdinfo 命

令，此命令用于查看板子信息，直接输入“bdinfo”即可，结果如图 10.3.1.1 示：

```
STM32MP> bdinfo
arch_number = 0x00000000
boot_params = 0xc0000100
DRAM bank   = 0x00000000
-> start    = 0xc0000000
-> size     = 0x40000000
baudrate    = 115200 bps
TLB addr    = 0xf5ff0000
relocaddr   = 0xf5c46000
reloc off   = 0x35b46000
irq_sp      = 0xf3aedd50
sp start    = 0xf3aedd40
FB base     = 0x00000000
Early malloc usage: 13f4 / 3000
fdt_blob    = 0xf3aedfe0
STM32MP>
```

图 10.3.1.1 bdinfo 命令

从图 10.3.1.1 中可以看出 DRAM 的起始地址和大小、BOOT 参数保存起始地址、波特率、sp(堆栈指针)起始地址等信息。命令“printenv”用于输出环境变量信息，uboot 也支持 TAB 键自动补全功能，输入“print”然后按下 TAB 键就会自动补全命令。直接输入“print”也可以，因为整个 uboot 命令中只有 printenv 的前缀是“print”，所以当输入 print 以后就只有 printenv 命令了。输入“print”，然后按下回车键，环境变量如图 10.3.1.2 所示：

```
STM32MP> print
altbootcmd=run bootcmd
android_mmc_boot=mmc dev ${devnum};run android_mmc_splash;run android_mmc_fdt;run android_mmc_kernel;bootm ${kerne
l_addr_r} - ${fdt_addr_r};
android_mmc_fdt=if part start mmc ${devnum} dt_${suffix} dt_start &&part size mmc ${devnum} dt_${suffix} dt_size;t
hen mmc read ${dtimg_addr} ${dt_start} ${dt_size};dtimg getindex ${dtimg_addr} ${board_id} ${board_rev} dt_index;d
timg start ${dtimg_addr} ${dt_index} fdt_addr_r;fi
android_mmc_kernel=if part start mmc ${devnum} boot_${suffix} boot_start &&part size mmc ${devnum} boot_${suffix}
boot_size;then mmc read ${kernel_addr_r} ${boot_start} ${boot_size};part nb mmc ${devnum} system_${suffix} rootpar
t_nb;env set bootargsroot=/dev/mmcblk${devnum}p${rootpart_nb} androidboot.serialno=${serial#} androidboot.slot_suf
fix=_${suffix};fi
android_mmc_splash=if part start mmc ${devnum} splash splash_start && part size mmc ${devnum} splash splash_size;t
hen mmc read ${splashimage} ${splash_start} ${splash_size};cls; bmp display ${splashimage} m m;fi
arch=arm
autoload=no
baudrate=115200
board=stm32mp1
board_name=stm32mp157d-atk
boot_a_script=load ${devtype} ${devnum}:${distro_bootpart} ${scriptaddr} ${prefix}${script}; source ${scriptaddr}
boot_device=mmc
boot_efi_binary=if fdt addr ${fdt_addr_r}; then bootefi bootmgr ${fdt_addr_r};else bootefi bootmgr ${fdtcontroladd
r};fi;load ${devtype} ${devnum}:${distro_bootpart} ${kernel_addr_r} efi/boot/bootarm.efi; if fdt addr ${fdt_addr_r
}; then bootefi ${kernel_addr_r} ${fdt_addr_r};else bootefi ${kernel_addr_r} ${fdtcontroladdr};fi
boot_extlinux=sysboot ${devtype} ${devnum}:${distro_bootpart} any ${scriptaddr} ${prefix}${boot_syslinux_conf}
```

图 10.3.1.2 printenv 命令部分结果

图 10.3.1.2 只是 printenv 命令的部分内容，STM32MP1 系列的环境变量有很多，比如 baudrate、board、board_name、boot_device、bootcmd、bootdelay 等等。uboot 中的环境变量都是字符串，既然叫做环境变量，那么它的作用就和“变量”一样。比如 bootdelay 这个环境变量就表示 uboot 启动延时时间，默认 bootdelay=1，也就默认延时 1 秒。前面说的 1 秒倒计时就是由 bootdelay 定义的，如果将 bootdelay 改为 5 的话就会倒计时 5s 了。uboot 中的环境变量是可以修改的，有专门的命令来修改环境变量的值，稍后我们会讲解。

命令 version 用于查看 uboot 的版本号，输入“version”，uboot 版本号如图 10.3.1.3 所示：

```
STM32MP> version
U-Boot 2020.01-stm32mp-r1 (Nov 24 2020 - 17:17:20 +0800)

arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 9.2.1 20191025
GNU ld (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 2.33.1.20191209
STM32MP>
```

图 10.3.1.3 version 命令结果

10.3.2 环境变量操作命令

1、修改环境变量

环境变量的操作涉及到两个命令：setenv 和 saveenv，setenv 命令用于设置或者修改环境变量的值。命令 saveenv 用于保存修改后的环境变量，一般环境变量存放在外部 flash 中，uboot 启动的时候会将环境变量从 flash 读取到 DRAM 中。所以使用命令 setenv 修改的是 DRAM 中的环境变量值，修改以后要使用 saveenv 命令将修改后的环境变量保存到 flash 中，否则 uboot 下一次重启会继续使用以前的环境变量值。

命令 saveenv 使用起来很简单，格式为：

```
saveenv
```

比如我们要将环境变量 bootdelay 改为 5，就可以使用如下所示命令：

```
setenv bootdelay 5
saveenv
```

上述命令执行过程如图 10.3.2.1 所示：

```
STM32MP> setenv bootdelay 5
STM32MP> saveenv
Saving Environment to MMC... Writing to redundant MMC(1)... OK
STM32MP>
```

图 10.3.2.1 环境变量修改

在图 10.3.2.1 中，当我们使用命令 saveenv 保存修改后的环境变量会有保存过程提示信息，根据提示可以看出环境变量保存到了 MMC(1)中，也就是 EMMC 中。因为我用 EMMC 启动的，所以会保存到 MMC(1)中。修改 bootdelay 以后，重启开发板，uboot 就是变为 5 秒倒计时，如图 10.3.2.2 所示：

```
U-Boot 2020.01-stm32mp-r1 (Nov 24 2020 - 17:17:20 +0800)

CPU: STM32MP157DAA Rev.Z
Model: STMicroelectronics STM32MP157D eval daughter
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)
DRAM:  1 GiB
Clocks:
- MPU : 800 MHz
- MCU : 208.878 MHz
- AXI : 266.500 MHz
- PER : 24 MHz
- DDR : 533 MHz
WDT:   Started with servicing (32s timeout)
NAND:  0 MiB
MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
Loading Environment from MMC... OK
In:    serial
Out:   serial
Err:   serial
invalid MAC address in OTP 00:00:00:00:00:00
Net:
Error: ethernet@5800a000 address not set.
No ethernet found.

Hit any key to stop autoboot:  5
```

图 10.3.2.2 5 秒倒计时

从图 10.3.2.2 可以看出，此时 uboot 启动倒计时变为了 5 秒。

2、新建环境变量

命令 setenv 也可以用于新建命令，用法和修改环境变量一样，比如我们新建一个环境变量 author，author 的值为‘console=ttySTM0,11520 root=/dev/mmcblk2p2 rootwait rw’，那么就可以使用如下命令：

```
setenv author 'console=ttySTM0,115200 root=/dev/mmcblk2p2 rootwait rw '
saveenv
```

上面命令设置 author 的值为“console=ttySTM0,115200 root=/dev/mmcblk2p2 rootwait rw”，其中“console=ttySTM0,115200”、“root=/dev/mmcblk2p2”、“rootwait”和“rw”相当于四组“值”，这四组“值”之间用空格隔开，所以需要使用单引号‘’将其括起来，表示这四组“值”都属于环境变量 author。

author 命令创建完成以后重启 uboot，然后使用命令 printenv 查看当前环境变量，如图 10.3.2.3 所示：

```
android_mmc_splash=if part start mmc ${devnum} splash splash_start && part size mmc ${devnum} splash splash_size;t
hen mmc read ${splashimage} ${splash_start} ${splash_size};cls; bmp display ${splashimage} m m;fi
arch=arm
author=console=ttySTM0,115200 root=/dev/mmcblk2p2 rootwait rw
autoload=no
baudrate=115200
```

图 10.3.2.3 新建的 author 环境变量值

3、删除环境变量

既然可以新建环境变量，那么就可以删除环境变量，删除环境变量也是使用命令 setenv，要删除一个环境变量只要给这个环境变量赋空值即可，比如我们删除掉上面新建的 author 环境变量，命令如下：

```
setenv author
saveenv
```

上面命令中通过 setenv 给 author 赋空值，也就是什么都不写来删除环境变量 author。重启 uboot 就会发现环境变量 author 没有了。

10.3.3 内存操作命令

内存操作命令就是用于直接对 DRAM 进行读写操作的，常用的内存操作命令有 md、nm、mm、mw、cp 和 cmp，我们依次来看一下这些命令都是做什么的。

1、md 命令

md 命令用于显示内存值，格式如下：

```
md[.b, .w, .l] address [# of objects]
```

命令中的[.b .w .l]对应 byte、word 和 long，也就是分别以 1 个字节、2 个字节、4 个字节来显示内存值。address 就是要查看的内存起始地址，[# of objects]表示要查看的数据长度，这个数据长度单位不是字节，而是跟你所选择的显示格式有关。比如你设置要查看的内存长度为 20(十六进制为 0x14)，如果显示格式为.b 的话那就表示 20 个字节；如果显示格式为.w 的话就表示 20 个 word，也就是 20*2=40 个字节；如果显示格式为.l 的话就表示 20 个 long，也就是 20*4=80 个字节，另外要注意：

uboot 命令中的数字都是十六进制的！不是十进制的！

比如你想查看 0XC0100000 开始的 20 个字节的内存值，显示格式为.b 的话，应该使用如下所示命令：

```
md.b C0100000 14
```

而不是：

```
md.b C0100000 20
```

上面说了，uboot 命令里面的数字都是十六进制的，所以可以不用写“0x”前缀，十进制的 20 对应的十六进制为 0x14，所以命令 md 后面的个数应该是 14，如果写成 20 的话就表示查看 32(十六进制为 0x20)个字节的数据。分析下面三个命令的区别：

```
md.b C0100000 10
md.w C0100000 10
md.l C0100000 10
```

上面这三个命令都是查看以 0XC0100000 为起始地址的内存数据，第一个命令以.b 格式显示，长度为 0x10，也就是 16 个字节；第二个命令以.w 格式显示，长度为 0x10，也就是 16*2=32 个字节；最后一个命令以.l 格式显示，长度也是 0x10，也就是 16*4=64 个字节。这三个命令的执行结果如图 10.3.3.1 所示：

```
STM32MP> md.b c0100000 10
c0100000: b8 00 00 ea 14 f0 9f e5 14 f0 9f e5 14 f0 9f e5    ................
STM32MP> md.w c0100000 10
c0100000: 00b8 ea00 f014 e59f f014 e59f f014 e59f    ................
c0100010: f014 e59f f014 e59f f014 e59f f014 e59f    ................
STM32MP> md.l c0100000 10
c0100000: ea0000b8 e59ff014 e59ff014 e59ff014    ................
c0100010: e59ff014 e59ff014 e59ff014 e59ff014    ................
c0100020: c0100060 c01000c0 c0100120 c0100180    `....... .......
c0100030: c01001e0 c0100240 c01002a0 deadbeef    ....@...........
STM32MP>
```

图 10.3.3.1 md 命令使用示例

2、nm 命令

nm 命令用于修改指定地址的内存值，命令格式如下：

```
nm [.b, .w, .l] address
```

nm 命令同样可以以.b、.w 和.l 来指定操作格式，比如现在以.l 格式修改 0XC0100000 地址的数据为 0x12345678。输入命令：

```
nm.l C0100000
```

输入上述命令以后如图 10.3.3.2 所示：

```
STM32MP> nm.l c0100000
c0100000: ea0000b8 ?
```

图 10.3.3.2 nm 命令

在图 10.3.3.2 中，C0100000 表示现在要修改的内存地址，ea0000b8 表示地址 0xc0100000 现在的数据，‘？’后面就可以输入要修改后的数据 0x12345678，输入完成以后按下回车，然后再输入‘q’即可退出，如图 10.3.3.3 所示：

```
STM32MP> nm.l c0100000
c0100000: ea0000b8 ? 0x12345678
c0100000: 12345678 ? q
STM32MP>
```

图 10.3.3.3 修改内存数据

修改完成以后再使用 md 命令来查看一下有没有修改成功，如图 10.3.3.4 所示：

```
STM32MP> md.l c0100000 1
c0100000: 12345678                                    xV4.
STM32MP>
```

图 10.3.3.4 查看修改后的值

从图 10.3.3.4 可以看出，此时地址 0XC0100000 的值变为了 0X12345678。

3、MM 命令

mm 命令也是修改指定地址内存值的，使用 mm 修改内存值的时候地址会自增，而使用 nm 命令的话地址不会自增。比如以.l 格式修改从地址 0XC0100000 开始的连续 3 个内存块(3*4=12 个字节)的数据为 0X05050505，操作如图 10.3.3.5 所示：

```
STM32MP> mm.l c0100000
c0100000: 12345678 ? 05050505
c0100004: e59ff014 ? 05050505
c0100008: e59ff014 ? 05050505
c010000c: e59ff014 ? q
STM32MP>
```

图 10.3.3.5 命令 mm

从图 10.3.3.5 可以看出，修改了地址 0XC0100000、0XC0100004 和 0XC0100008 的内容为 0x05050505。使用命令 md 查看修改后的值，结果如图 10.3.3.6 所示：

```
STM32MP> md.l c0100000 3
c0100000: 05050505 05050505 05050505                ............
STM32MP>
```

图 10.3.3.6 查看修改后的内存数据

从图 10.3.3.6 可以看出内存数据修改成功。

4、mw 命令

命令 mw 用于使用一个指定的数据填充一段内存，命令格式如下：

```
mw [.b, .w, .l] address value [count]
```

mw 命令同样以.b、.w 和.l 来指定操作格式，address 表示要填充的内存起始地址，value 为要填充的数据，count 是填充的长度。比如使用.l 格式将以 0XC0100000 为起始地址的 0x10 个内存块(0x10 * 4=64 字节)填充为 0X0A0A0A0A，命令如下：

```
mw.l C0100000 0A0A0A0A 10
```

然后使用命令 md 来查看，如图 10.3.3.7 所示：

```
STM32MP> mw.l c0100000 0A0A0A0A 10
STM32MP> md.l c0100000 10
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
STM32MP>
```

图 10.3.3.7 查看修改后的内存数据

从图 10.3.3.7 以看出内存数据修改成功。

5、CP 命令

cp 是数据拷贝命令，用于将 DRAM 中的数据从一段内存拷贝到另一段内存中，或者把 NorFlash 中的数据拷贝到 DRAM 中。命令格式如下：

```
cp [.b, .w, .l] source target count
```

cp 命令同样以.b、.w 和.l 来指定操作格式，source 为源地址，target 为目的地址，count 为拷贝的长度。我们使用.l 格式将 0xC0100000 处的地址拷贝到 0xC0100100 处，长度为 0x10 个内存块(0x10 * 4=64 个字节)，命令如下所示：

```
cp.l c0100000 c0100100 10
```

结果如图 10.3.3.8 所示：

```
STM32MP> md.l c0100000 10
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
STM32MP> md.l c0100100 10
c0100100: e1a0000d fa000622 e320f000 e320f000    ....".... ... .
c0100110: e320f000 e320f000 e320f000 e320f000    .. ... ... ... .
c0100120: e51fd0e8 e58de000 e14fe000 e58de004    .........O.....
c0100130: e3a0d013 e169f00d e1a0e00f e1b0f00e    ......i.........
STM32MP> cp.l c0100000 c0100100 10
STM32MP> md.l c0100000 10
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
STM32MP>
```

图 10.3.3.8 cp 命令操作结果

在图 10.3.3.8 中，先使用 md.l 命令打印出地址 0xC0100000 和 0xC0100100 处的数据，然后使用命令 cp.l 将 0xC0100000 处的数据拷贝到 0xC0100100 处。最后使用命 md.l 查看 0xC0100100 处的数据有没有变化，检查拷贝是否成功。

6、CMP 命令

cmp 是比较命令，用于比较两段内存的数据是否相等，命令格式如下：

```
cmp [.b, .w, .l] addr1 addr2 count
```

cmp 命令同样以.b、.w 和.l 来指定操作格式，addr1 为第一段内存首地址，addr2 为第二段内存首地址，count 为要比较的长度。我们使用.l 格式来比较 0xC0100000 和 0xC0100100 这两个地址数据是否相等，比较长度为 0x10 个内存块(16 * 4=64 个字节)，命令如下所示：

```
cmp.l c0100000 c0100100 10
```

结果如图 10.3.3.9 所示：

```
STM32MP> cmp.l c0100000 c0100100 10
Total of 16 word(s) were the same
STM32MP>
```

图 10.3.3.9 cmp 命令比较结果

从图 10.3.3.9 可以看出两段内存的数据相等。我们再随便挑两段内存比较一下，比如地址 0xC0100000 和 0xC0100200，长度为 0X10，比较结果如图 10.3.3.10 所示：

```
STM32MP> cmp.l c0100000 c0100200 10
word at 0xc0100000 (0xa0a0a0a) != word at 0xc0100200 (0xe24dd048)
Total of 0 word(s) were the same
STM32MP>
```

图 10.3.3.10 cmp 命令比较结果

从图 10.3.3.10 可以看出，0xC0100000 处的数据和 0xC0100200 处的数据就不一样。

10.3.4 网络操作命令

uboot 是支持网络的，我们在移植 uboot 的时候一般都要调通网络功能，因为在移植 linux kernel 的时候需要使用到 uboot 的网络功能做调试。uboot 支持大量的网络相关命令，比如 dhcp、ping、nfs 和 tftpboot，我们接下来依次学习一下这几个和网络有关的命令。

建议开发板和主机 PC 都连接到同一个路由器上！最后设置表 10.3.4.1 所示的几个环境变量：

环境变量	描述
ipaddr	开发板 ip 地址，可以不设置，使用 dhcp 命令来从路由器获取 IP 地址。

ethaddr	开发板的 MAC 地址，一定要设置。
gatewayip	网关地址。
netmask	子网掩码。
serverip	服务器 IP 地址，也就是 Ubuntu 主机 IP 地址，用于调试代码。

表 10.3.4.1 网络相关环境变量

表 10.3.4.1 中环境变量设置命令如下所示：

```
setenv ipaddr 192.168.1.250
setenv ethaddr b8:ae:1d:01:01:00
setenv gatewayip 192.168.1.1
setenv netmask 255.255.255.0
setenv serverip 192.168.1.249
saveenv
```

注意，网络地址环境变量的设置要根据自己的实际情况，确保 Ubuntu 主机和开发板的 IP 地址在同一个网段内，比如我现在的开发板和电脑都在 192.168.1.0 这个网段内，所以设置开发板的 IP 地址为 192.168.1.250，我的 Ubuntu 主机的地址为 192.168.1.249，因此 serverip 就是 192.168.1.249。ethaddr 为网络 MAC 地址，是一个 48bit 的地址，如果在同一个网段内有多个开发板的话一定要保证每个开发板的 ethaddr 是不同的，否则通信会有问题！设置好网络相关的环境变量以后就可以使用网络相关命令了。

1、ping 命令

开发板的网络能否使用，是否可以和服务器(Ubuntu 主机)进行通信，通过 ping 命令就可以验证，直接 ping 服务器的 IP 地址即可，比如我的服务器 IP 地址为 192.168.1.249，命令如下：

```
ping 192.168.1.249
```

结果如图 10.3.4.1 所示：

```
STM32MP> ping 192.168.1.249
ethernet@5800a000 Waiting for PHY auto negotiation to complete..... done
Using ethernet@5800a000 device
host 192.168.1.249 is alive
STM32MP>
```

图 10.3.4.1 ping 命令

从图 10.3.4.1 可以看出，192.168.1.249 这个主机存在，说明 ping 成功，uboot 的网络工作正常。

注意！只能在 uboot 中 ping 其他的机器，其他机器不能 ping uboot，因为 uboot 没有对 ping 命令做处理，如果用其他的机器 ping uboot 的话会失败！

2、dhcp 命令

dhcp 用于从路由器获取 IP 地址，前提是开发板得连接到路由器上的，如果开发板是和电脑直连的，那么 dhcp 命令就会失效。直接输入 dhcp 命令即可通过路由器获取到 IP 地址，如图 10.3.4.2 所示：

```
STM32MP> dhcp
BOOTP broadcast 1
BOOTP broadcast 2
BOOTP broadcast 3
DHCP client bound to address 192.168.1.7 (787 ms)
STM32MP>
```

图 10.3.4.2 dhcp 命令

从图 10.3.4.2 可以看出，开发板通过 dhcp 获取到的 IP 地址为 192.168.1.7。DHCP 不单单是获取 IP 地址，其还会通过 TFTP 来启动 linux 内核，输入“? dhcp”即可查看 dhcp 命令详细的信息，如图 10.3.4.3 所示：

```
STM32MP> ? dhcp
dhcp - boot image via network using DHCP/TFTP protocol

Usage:
dhcp [loadAddress] [[hostIPaddr:]bootfilename]
STM32MP>
```

图 10.3.4.3 dhcp 命令使用查询

3、nfs 命令

nfs(Network File System)网络文件系统，通过 nfs 可以在计算机之间通过网络来分享资源，比如我们将 linux 镜像和设备树文件放到 Ubuntu 中，然后在 uboot 中使用 nfs 命令将 Ubuntu 中的 linux 镜像和设备树下载到开发板的 DRAM 中。这样做的目的是为了方便调试 linux 镜像和设备树，也就是网络调试，网络调试是 Linux 开发中最常用的调试方法。原因是嵌入式 linux 开发不像单片机开发，可以直接通过 JLINK 或 STLink 等仿真器将代码直接烧写到单片机内部的 flash 中，嵌入式 Linux 通常是烧写到 EMMC、NAND Flash、SPI Flash 等外置 flash 中，但是嵌入式 Linux 开发也没有 MDK，IAR 这样的 IDE，更没有烧写算法，因此不可能通过点击一个“download”按钮就将固件烧写到外部 flash 中。虽然半导体厂商一般都会提供一个烧写固件的软件，但是这个软件使用起来比较复杂，这个烧写软件一般用于量产的。其远没有 MDK、IAR 的一键下载方便，在 Linux 内核调试阶段，如果用这个烧写软件的话将会非常浪费时间，而这个时候网络调试的优势就显现出来了，可以通过网络将编译好的 linux 镜像和设备树文件下载到 DRAM 中，然后就可以直接运行。

我们一般使用 uboot 中的 nfs 命令将 Ubuntu 中的文件下载到开发板的 DRAM 中，在使用之前需要开启 Ubuntu 主机的 NFS 服务，并且要新建一个 NFS 使用的目录，以后所有要通过 NFS 访问的文件都需要放到这个 NFS 目录中。Ubuntu 的 NFS 服务开启我们在 4.2.1 小节已经详细讲解过了，包括 NFS 文件目录的创建，如果忘记的话可以去查看一下 4.2.1 小节。我设置的 NFS 文件目录为：/home/zuozhongkai/linux/nfs，uboot 中的 nfs 命令格式如下所示：

```
nfs [loadAddress] [[hostIPaddr:]bootfilename]
```

loadAddress 是要保存的 DRAM 地址，[[hostIPaddr:]bootfilename]是要下载的文件地址。这里我们将正点原子官方编译出来的 Linux 镜像文件 uImage 下载到开发板 DRAM 的 0xC2000000 这个地址处(ST 官方指定的 Linux 内核加载地址)。正点原子编译出来的 uImage 文件已经放到了开发板光盘中，路径为：开发板光盘 A→8、系统镜像→02、出厂系统镜像→linux-mp1-5.4.31-g032f9dcc0-v1.0→uImage。将整个文件通过 FileZilla 发送到 Ubuntu 中的 NFS 目录下：/home/zuozhongkai/linux/nfs，完成以后的 NFS 目录如图 10.3.4.4 所示：

```
zuozhongkai@ubuntu:~/linux/nfs$ ls
uImage
zuozhongkai@ubuntu:~/linux/nfs$
```

图 10.3.4.4 nfs 目录

准备好以后就可以使用 nfs 命令来将 uImage 下载到开发板 DRAM 的 0XC2000000 地址处，命令如下：

```
nfs C2000000 192.168.1.249:/home/zuozhongkai/linux/nfs/uImage
```

命令中的“C2000000”表示 uImage 在 DDR 中的首地址，“192.168.1.249:/home/zuozhongkai/linux/nfs/uImage”表示 uImage 在 192.168.1.249 这个主机中，路径为/home/zuozhongkai/linux/

nfs/uImage。下载过程如图 10.3.4.5 所示：

```
STM32MP> nfs 0xc2000000 192.168.1.249:/home/zuozhongkai/linux/nfs/uImage
Using ethernet@5800a000 device
File transfer via NFS from server 192.168.1.249; our IP address is 192.168.1.250
Filename '/home/zuozhongkai/linux/nfs/uImage'.
Load address: 0xc2000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ######################T ########################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ##############################################################
done
Bytes transferred = 7313888 (6f99e0 hex)
STM32MP>
```

图 10.3.4.5 nfs 命令下载 uImage 过程

图 10.3.4.5 中通过网络下载 uImage 的时候会打印出‘#’表示正在下载，如果网络环境不好，下载有干扰的话就会打印出一个‘T’，这个没事的，只要它能正常下载成功，最终打印出“done”即可。

下载完成以后查看 0xC2000000 地址处的数据，使用命令 md.b 来查看前 0x100(256)个字节的数据，如图 10.3.4.6 所示：

```
STM32MP> md.b c2000000 100
c2000000: 27 05 19 56 77 46 15 2c 5f a3 c3 21 00 6f 99 a0    '..VwF.,_..!.o..
c2000010: c2 00 00 40 c2 00 00 40 14 80 31 03 05 02 02 00    ...@...@..1.....
c2000020: 4c 69 6e 75 78 2d 35 2e 34 2e 33 31 2d 67 30 33    Linux-5.4.31-g03
c2000030: 32 66 39 64 63 63 30 00 00 00 00 00 00 00 00 00    2f9dcc0.........
c2000040: 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1    ................
c2000050: 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1 00 00 a0 e1    ................
c2000060: 05 00 00 ea 18 28 6f 01 00 00 00 00 a0 99 6f 00    .....(o.......o.
c2000070: 01 02 03 04 45 45 45 45 a8 66 00 00 00 90 0f e1    ....EEEE.f......
c2000080: fe 0d 00 eb 01 70 a0 e1 02 80 a0 e1 00 20 0f e1    .....p....... ..
c2000090: 03 00 12 e3 01 00 00 1a 17 00 a0 e3 56 34 12 ef    ............V4..
c20000a0: 00 00 0f e1 1a 00 20 e2 1f 00 10 e3 1f 00 c0 e3    ...... .........
c20000b0: d3 00 80 e3 04 00 00 1a 01 0c 80 e3 0c e0 8f e2    ................
c20000c0: 00 f0 6f e1 0e f3 2e e1 6e 00 60 e1 00 f0 21 e1    ..o.....n.`...!.
c20000d0: 09 f0 6f e1 00 00 00 00 00 00 00 00 00 00 00 00    ..o.............
c20000e0: 0f 40 a0 e1 3e 43 04 e2 02 49 84 e2 0f 00 a0 e1    .@..>C...I......
c20000f0: 04 00 50 e1 6c 02 9f 35 0f 00 80 30 00 00 54 31    ..P.l..5...0..T1
STM32MP>
```

图 10.3.4.6 下载的数据

再使用 winhex 软件来查看刚刚下载的 uImage，检查一下前面的数据是否和图 10.3.4.6 中的一致，结果如图 10.3.4.7 所示：

uImage

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00000000	27	05	19	56	77	46	15	2C	5F	A3	C3	21	00	6F	99	A0
00000010	C2	00	00	40	C2	00	00	40	14	80	31	03	05	02	02	00
00000020	4C	69	6E	75	78	2D	35	2E	34	2E	33	31	2D	67	30	33
00000030	32	66	39	64	63	63	30	00	00	00	00	00	00	00	00	00
00000040	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1
00000050	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1	00	00	A0	E1
00000060	05	00	00	EA	18	28	6F	01	00	00	00	00	A0	99	6F	00
00000070	01	02	03	04	45	45	45	45	A8	66	00	00	00	90	0F	E1
00000080	FE	0D	00	EB	01	70	A0	E1	02	80	A0	E1	00	20	0F	E1
00000090	03	00	12	E3	01	00	00	1A	17	00	A0	E3	56	34	12	EF
000000A0	00	00	0F	E1	1A	00	20	E2	1F	00	10	E3	1F	00	C0	E3
000000B0	D3	00	80	E3	04	00	00	1A	01	0C	80	E3	0C	E0	8F	E2
000000C0	00	F0	6F	E1	0E	F3	2E	E1	6E	00	60	E1	00	F0	21	E1
000000D0	09	F0	6F	E1	00	00	00	00	00	00	00	00	00	00	00	00
000000E0	0F	40	A0	E1	3E	43	04	E2	02	49	84	E2	0F	00	A0	E1
000000F0	04	00	50	E1	6C	02	9F	35	0F	00	80	30	00	00	54	31
00000100	01	40	84	33	9D	00	00	2B	8E	0F	8F	E2	4E	1C	90	E8

图 10.3.4.7 winhex 查看 uImage

可以看出图 10.3.4.和图 10.3.4.7 中的前 0x100 个字节的数据一致，说明 nfs 命令下载到的 uImage 是正确的。

4、tftp 命令

tftp 命令的作用和 nfs 命令一样，都是用于通过网络下载东西到 DRAM 中，只是 tftp 命令使用的是 TFTP 协议，Ubuntu 主机作为 TFTP 服务器。因此需要在 Ubuntu 上搭建 TFTP 服务器，需要安装 tftp-hpa 和 tftpd-hpa，命令如下：

```
sudo apt-get install tftp-hpa tftpd-hpa
sudo apt-get install xinetd
```

和 NFS 一样，TFTP 也需要一个文件夹来存放文件，在用户目录下新建一个目录，命令如下：

```
mkdir /home/zuozhongkai/linux/tftpboot
chmod 777 /home/zuozhongkai/linux/tftpboot
```

这样我就在我的电脑上创建了一个名为 tftpboot 的目录(文件夹)，路径为/home/zuozhongkai/linux/tftpboot。注意！我们要给 tftpboot 文件夹权限，否则的话 uboot 不能从 tftpboot 文件夹里面下载文件。

最后配置 tftp，新建文件/etc/xinetd.d/tftp，如果没有/etc/xinetd.d 目录的话自行创建，然后在里面输入如下内容：

示例代码 10.3.4.1 /etc/xinetd.d/tftp 文件内容

```
1  server tftp
2  {
3      socket_type     = dgram
4      protocol        = udp
5      wait            = yes
6      user            = root
7      server          = /usr/sbin/in.tftpd
8      server_args     = -s /home/zuozhongkai/linux/tftpboot/
9      disable         = no
10     per_source      = 11
```

```
11      cps             = 100 2
12      flags           = IPv4
13  }
```

完了以后启动 tftp 服务，命令如下：

```
sudo service tftpd-hpa start
```

打开/etc/default/tftpd-hpa 文件，将其修改为如下所示内容：

示例代码 10.3.4.2 /etc/default/tftpd-hpa 文件内容

```
1 # /etc/default/tftpd-hpa
2
3 TFTP_USERNAME="tftp"
4 TFTP_DIRECTORY="/home/liangwencong/linux/tftpboot"
5 TFTP_ADDRESS=":69"
6 TFTP_OPTIONS="-l -c -s"
```

TFTP_DIRECTORY 就是我们上面创建的 tftp 文件夹目录，以后我们就将所有需要通过 TFTP 传输的文件都放到这个文件夹里面，并且要给予这些文件相应的权限。

最后输入如下命令， 重启 tftp 服务器：

```
sudo service tftpd-hpa restart
```

tftp 服务器已经搭建好了，接下来就是使用了。将 uImage 镜像文件拷贝到 tftpboot 文件夹中，并且给予 uImage 相应的权限，命令如下：

```
cp uImage /home/zuozhongkai/linux/tftpboot/
cd /home/zuozhongkai/linux/tftpboot/
chmod 777 uImage
```

万事俱备，只剩验证了，uboot 中的 tftp 命令格式如下：

```
tftpboot [loadAddress] [[hostIPaddr:]bootfilename]
```

看起来和 nfs 命令格式一样的，loadAddress 是文件在 DRAM 中的存放地址，[[hostIPaddr:]bootfilename]是要从 Ubuntu 中下载的文件。但是和 nfs 命令的区别在于，tftp 命令不需要输入文件在 Ubuntu 中的完整路径，只需要输入文件名即可。比如我们现在将 tftpboot 文件夹里面的 uImage 文件下载到开发板 DRAM 的 0XC2000000 地址处，命令如下：

```
tftp C2000000 uImage
```

下载过程如图 10.3.4.8 所示：

```
STM32MP> tftp c2000000 uImage
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'uImage'.
Load address: 0xc2000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ##########################################
         2.3 MiB/s
done
Bytes transferred = 7313888 (6f99e0 hex)
STM32MP>
```

图 10.3.4.8 tftp 命令下载过程

从图 10.3.4.8 可以看出，uImage 下载成功了，网速为 2.3MibB/s，文件大小为 7312880 字节。同样的，可以使用 md.b 命令来查看前 0x100 个字节的数据是否和图 10.3.4.7 中的相等。有

时候使用 tftp 命令从 Ubuntu 中下载文件的时候会出现如图 10.3.4.9 所示的错误提示：

```
STM32MP> tftp c2000000 uImage
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'uImage'.
Load address: 0xc2000000
Loading: *
TFTP error: 'Permission denied' (0)
Starting again

STM32MP>
```

图 10.3.4.9 tftp 下载出错

在图 10.3.4.9 中可以看到“TFTP error: 'Permission denied' (0)”这样的错误提示，提示没有权限，出现这个错误一般有两个原因：

①、在 Ubuntu 中创建 tftpboot 目录的时候没有给予 tftboot 相应的权限。

②、tftpboot 目录中要下载的文件没有给予相应的权限。

针对上述两个问题，使用命令“chmod 777 xxx”来给予权限，其中“xxx”就是要给予权限的文件或文件夹。

好了，uboot 中关于网络的命令就讲解到这里，我们最常用的就是 ping、nfs 和 tftp 这三个命令。使用 ping 命令来查看网络的连接状态，使用 nfs 和 tftp 命令来从 Ubuntu 主机中下载文件。

10.3.5 EMMC 和 SD 卡操作命令

uboot 支持 EMMC 和 SD 卡，因此也要提供 EMMC 和 SD 卡的操作命令。一般认为 EMMC 和 SD 卡是同一个东西，所以没有特殊说明，本教程统一使用 MMC 来代指 EMMC 和 SD 卡。uboot 中常用于操作 MMC 设备的命令为“mmc”。

mmc 是一系列的命令，其后可以跟不同的参数，输入“？ mmc”即可查看 mmc 有关的命令，如图 10.3.5.1 所示：

```
STM32MP> ? mmc
mmc - MMC sub system

Usage:
mmc info - display info of the current MMC device
mmc read addr blk# cnt
mmc write addr blk# cnt
mmc erase blk# cnt
mmc rescan
mmc part - lists available partition on current mmc device
mmc dev [dev] [part] - show or set current mmc device [partition]
mmc list - lists available devices
mmc hwpartition [args...] - does hardware partitioning
  arguments (sizes in 512-byte blocks):
    [user [enh start cnt] [wrrel {on|off}]] - sets user data area attributes
    [gp1|gp2|gp3|gp4 cnt [enh] [wrrel {on|off}]] - general purpose partition
    [check|set|complete] - mode, complete set partitioning completed
  WARNING: Partitioning is a write-once setting once it is set to complete.
  Power cycling is required to initialize partitions after set to complete.
mmc bootbus dev boot_bus_width reset_boot_bus_width boot_mode
 - Set the BOOT_BUS_WIDTH field of the specified device
mmc bootpart-resize <dev> <boot part size MB> <RPMB part size MB>
 - Change sizes of boot and RPMB partitions of specified device
mmc partconf dev [boot_ack boot_partition partition_access]
 - Show or change the bits of the PARTITION_CONFIG field of the specified device
mmc rst-function dev value
 - Change the RST_n_FUNCTION field of the specified device
   WARNING: This is a write-once field and 0 / 1 / 2 are the only valid values.
mmc setdsr <value> - set DSR register value

STM32MP>
```

图 10.3.5.1 mmc 命令

从图 10.3.5.1 可以看出，mmc 后面跟不同的参数可以实现不同的功能，如表 10.3.5.1 所示：

命令	描述
mmc info	输出 MMC 设备信息
mmc read	读取 MMC 中的数据。
mmc wirte	向 MMC 设备写入数据。
mmc rescan	扫描 MMC 设备。
mmc part	列出 MMC 设备的分区。
mmc dev	切换 MMC 设备。
mmc list	列出当前有效的所有 MMC 设备。
mmc hwpartition	设置 MMC 设备的分区。
mmc bootbus……	设置指定 MMC 设备的 BOOT_BUS_WIDTH 域的值。
mmc bootpart……	设置指定 MMC 设备的 boot 和 RPMB 分区的大小。
mmc partconf……	设置指定 MMC 设备的 PARTITION_CONFG 域的值。
mmc rst	复位 MMC 设备
mmc setdsr	设置 DSR 寄存器的值。

表 10.3.5.1 mmc 命令

1、mmc info 命令

mmc info 命令用于输出当前选中的 mmc info 设备的信息，输入命令“mmc info”即可，如图 10.3.5.2 所示：

```
STM32MP> mmc info
Device: STM32 SD/MMC
Manufacturer ID: 15
OEM: 100
Name: 8GTF4
Bus Speed: 52000000
Mode: MMC High Speed (52MHz)
Rd Block Len: 512
MMC version 5.1
High Capacity: Yes
Capacity: 7.3 GiB
Bus Width: 8-bit
Erase Group Size: 512 KiB
HC WP Group Size: 8 MiB
User Capacity: 7.3 GiB WRREL
Boot Capacity: 4 MiB ENH
RPMB Capacity: 512 KiB ENH
STM32MP>
```

图 10.3.5.2 mmc info 命令

从图 10.3.5.2 可以看出，当前选中的 MMC 设备是 EMMC，EMMC 芯片版本为 5.1，容量为 7.3GiB(EMMC 为 8GB)，速度为 52000000Hz=52MHz，8 位宽的总线。还有一个与 mmc info 命令相同功能的命令：mmcinfo，“mmc”和“info”之间没有空格。

2、mmc rescan 命令

mmc rescan 命令用于扫描当前开发板上所有的 MMC 设备，包括 EMMC 和 SD 卡，输入“mmc rescan”即可。

3、mmc list 命令

mmc list 命令用于来查看当前开发板一共有几个 MMC 设备，输入“mmc list”，结果如图 10.3.5.3 所示：

```
STM32MP> mmc list
STM32 SD/MMC: 0
STM32 SD/MMC: 1 (eMMC)
STM32MP>
```

图 10.3.5.3 扫描 MMC 设备

可以看出当前开发板有两个 MMC 设备：STM32 SD/MMC:0 和 STM32 SD/MMC:1 (eMMC)，这是因为我现在用的是 EMMC 版本的核心板，加上 SD 卡一共有两个 MMC 设备，STM32 SD/MMC:0 是 SD 卡，STM32 SD/MMC:1 (eMMC)是 EMMC。默认会将 EMMC 设置为当前 MMC 设备，这就是为什么输入“mmc info”查询到的是 EMMC 设备信息，而不是 SD 卡。要想查看 SD 卡信息，就要使用命令“mmc dev”来将 SD 卡设置为当前的 MMC 设备。

4、mmc dev 命令

mmc dev 命令用于切换当前 MMC 设备，命令格式如下：

```
mmc dev [dev] [part]
```

[dev]用来设置要切换的 MMC 设备号，[part]是分区号，如果不写分区号的话默认为分区 0。使用如下命令切换到 SD 卡：

```
mmc dev 0          //切换到 SD 卡，0 为 SD 卡，1 为 eMMC
```

结果如图 10.3.5.4 所示：

```
STM32MP> mmc dev 0
switch to partitions #0, OK
mmc0 is current device
STM32MP>
```

图 10.3.5.4 切换到 SD 卡

从图 30.4.5.4 可以看出，切换到 SD 卡成功，mmc0 为当前的 MMC 设备，输入命令“mmc info”即可查看 SD 卡的信息(要插入 SD 卡)，结果如图 10.3.5.5 所示：

```
STM32MP> mmc info
Device: STM32 SD/MMC
Manufacturer ID: 3
OEM: 5344
Name: SC16G
Bus Speed: 50000000
Mode: SD High Speed (50MHz)
Rd Block Len: 512
SD version 3.0
High Capacity: Yes
Capacity: 14.8 GiB
Bus Width: 4-bit
Erase Group Size: 512 Bytes
STM32MP>
```

图 10.3.5.5 SD 信息

从图 10.3.5.5 可以看出当前 SD 卡版本号为 3.0，容量为 14.8GiB(16GB 的 SD 卡)，4 位宽的总线。

5、mmc part 命令

有时候 SD 卡或者 EMMC 会有多个分区，可以使用命令“mmc part”来查看其分区，比如查看 EMMC 的分区情况，输入如下命令：

```
mmc dev 1      //切换到 EMMC
```

```
mmc part        //查看 EMMC 分区
```

结果如图 10.3.5.6 所示：

```
STM32MP> mmc dev 1
switch to partitions #0, OK
mmc1(part 0) is current device
STM32MP> mmc part

Partition Map for MMC device 1  --   Partition Type: EFI

Part    Start LBA       End LBA         Name
        Attributes
        Type GUID
        Partition GUID
  1     0x00000400      0x00e8fbff      "ssbl"
        attrs:  0x0000000000000000
        type:   8da63339-0007-60c0-c436-083ac8230908
        guid:   c6b6280e-7e2d-4f06-9606-438916e8b7d9
STM32MP>
```

图 10.3.5.6 查看 EMMC 分区

从图 10.3.5.6 中可以看出，此时 EMMC 是 EFI 类型并且有 1 个分区，如果你的核心板 EMMC 烧写过系统的话会有 3 个分区，正点原子出厂开发板都烧写系统，所以一般是 3 个分区，如图 10.3.5.7 所示：

```
Hit any key to stop autoboot:  0
STM32MP> mmc dev 1
switch to partitions #0, OK
mmc1(part 0) is current device
STM32MP> mmc part

Partition Map for MMC device 1  --   Partition Type: EFI

Part    Start LBA       End LBA         Name
        Attributes
        Type GUID
        Partition GUID
  1     0x00000400      0x000013ff      "ssbl"
        attrs:  0x0000000000000000
        type:   8da63339-0007-60c0-c436-083ac8230908
        guid:   3df85fdb-3108-41af-9a8c-ba44821e09b3
  2     0x00001400      0x000213ff      "boot"
        attrs:  0x0000000000000004
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   4a8d95b5-d425-4583-bb7d-e9c41c7b8382
  3     0x00021400      0x00e8fbff      "rootfs"
        attrs:  0x0000000000000000
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   491f6117-415d-4f53-88c9-6e0de54deac6
STM32MP>
```

图 10.3.5.7 烧写过系统的分区

从图 10.3.5.7 中可以看出，有 3 个分区，第一个分区为名字为“ssbl”，用来存放 uboot 镜像，范围为：扇区 0x400~0x13ff。第二个分区名字为“boot”，用来存放 linux 内核镜像，范围为：扇区 0x1400~0x213ff。第三个分区名字为“rootfs”，这个是根文件系统分区，占用了剩余的所有扇区，也就是扇区 0x21400~0xe8fbff。

如果要将 EMMC 的分区 2 设置为当前 MMC 设置分区，可以使用如下命令：

```
mmc dev 1 2
```

结果如图 10.3.5.8 所示：

```
STM32MP> mmc dev 1 2
switch to partitions #2, OK
mmc1(part 2) is current device
STM32MP>
```

图 10.3.5.8 设置 EMMC 分区 2 为当前设备

6、mmc read 命令

mmc read 命令用于读取 mmc 设备的数据，命令格式如下：

```
mmc read addr blk# cnt
```

addr 是数据读取到 DRAM 中的地址，blk 是要读取的块起始地址(十六进制)，一个块是 512 字节，这里的块和扇区是一个意思，在 MMC 设备中我们通常说扇区，cnt 是要读取的块数量(十六进制)。比如从 EMMC 的第 1024(0x400)个块开始，读取 16(0x10)个块的数据到 DRAM 的 0XC0000000 地址处，命令如下：

```
mmc dev 1                    //切换到 EMMC
mmc read c0000000 400 10     //读取数据
```

结果如图 10.3.5.9 所示：

```
STM32MP> mmc dev 1
switch to partitions #0, OK
mmc1(part 0) is current device
STM32MP> mmc read c0000000 400 10

MMC read: dev # 1, block # 1024, count 16 ... 16 blocks read: OK
STM32MP>
```

图 10.3.5.9 mmc read 命令

这里我们还看不出来读取是否正确，通过 md.b 命令查看 0xc0000000 处的数据就行了，查看 16*512=8192(0x2000)个字节的数据，命令如下：

```
md.b c0000000 2000
```

结果如图 10.3.5.10 所示：

```
STM32MP> md.b c0000000 2000
c0000000: 53 54 4d 32 00 00 00 00 00 00 00 00 00 00 00 00    STM2............
c0000010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c0000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c0000030: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c0000040: 00 00 00 00 aa 5a aa 04 00 00 01 00 f4 36 0d 00    .....Z.......6..
c0000050: 00 00 10 c0 00 00 00 00 00 00 10 c0 00 00 00 00    ................
c0000060: 00 00 00 00 01 00 00 00 01 00 00 00 00 00 00 00    ................
c0000070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c0000080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c0000090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c00000a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c00000b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c00000c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c00000d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c00000e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c00000f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
c0000100: b8 00 00 ea 14 f0 9f e5 14 f0 9f e5 14 f0 9f e5    ................
c0000110: 14 f0 9f e5 14 f0 9f e5 14 f0 9f e5 14 f0 9f e5    ................
c0000120: 60 00 10 c0 c0 00 10 c0 20 01 10 c0 80 01 10 c0    `....... .......
```

图 10.3.5.10 读取到的数据(部分截图)

从图 30.4.5.9 可以看出，前面的 265 个字节就是 STM32MP1 的头部信息。

10.3.6 EXT 格式文件系统操作命令

uboot 有 ext2 和 ext4 这两种格式的文件系统的操作命令，STM32MP1 的系统镜像都是 ext4 格式的，所以我们重点讲解一下和 ext4 有关的三个命令：ext4ls、ext4load 和 ext4write。注意，

由于只有 linux 内核、设备树和根文件系统是以 ext4 格式存放在 EMMC 中的，因此在测试 ext4 相关命令的时候要先确保 EMMC 里面烧写了完整的出厂系统！

1、ext4ls

ext4ls 命令用于查询 EXT4 格式设备的目录和文件信息，命令格式如下：

```
ext4ls <interface> [<dev[:part]>] [directory]
```

interface 是要查询的接口，比如 mmc，dev 是要查询的设备号，part 是要查询的分区，directory 是要查询的目录。比如查询 EMMC 分区 2 中的所有的目录和文件，输入命令：

```
ext4ls mmc 1:2
```

结果如图 10.3.6.1 所示：

```
STM32MP> ext4ls mmc 1:2
<DIR>       1024 .
<DIR>       1024 ..
            2943 boot.scr.uimg
<DIR>       1024 lost+found
<DIR>       1024 mmc0_extlinux
<DIR>       1024 mmc1_extlinux
           92670 splash.bmp
           74594 stm32mp157d-atk.dtb
           74014 stm32mp157d-atk-hdmi.dtb
           74510 stm32mp157d-atk-spdif.dtb
         8125872 uImage
         3632241 uInitrd
STM32MP>
```

图 10.3.6.1 EMMC 分区 2 文件查询

从上图可以看出，emmc 的分区 2 中存放了 10 个文件，其中比较重要的就是三个.dtb 设备树文件和 Linux 内核的 uImage 镜像文件。

2、ext4load 命令

extload 命令用于将指定的文件读取到 DRAM 中，命令格式如下：

```
extload <interface> [<dev[:part]> [<addr> [<filename> [bytes [pos]]]]]
```

interface 为接口，比如 mmc，dev 是设备号，part 是分区，addr 是保存在 DRAM 中的起始地址，filename 是要读取的文件名字。bytes 表示读取多少字节的数据，如果 bytes 为 0 或者省略的话表示读取整个文件。pos 是要读的文件相对于文件首地址的偏移，如果为 0 或者省略的话表示从文件首地址开始读取。我们将图 10.3.6.1 中 EMMC 分区 2 中的 uImage 文件读取到 DRAM 中的 0XC2000000 地址处，命令如下：

```
ext4load mmc 1:2 C2000000 uImage
```

操作过程如图 10.3.6.2 所示：

```
STM32MP> ext4load mmc 1:2 c2000000 uImage
8125872 bytes read in 207 ms (37.4 MiB/s)
STM32MP>
```

图 10.3.6.2 读取过程

从上图可以看出在 207ms 内读取了 8125872 个字节的数据，速度为 37.4MiB/s，速度是非常快的，因为这是从 EMMC 里面读取的，而 EMMC 是 8 位的，速度肯定会很快。

3、ext4write

ext4wirte 命令用于将 DRAM 中的数据写入到 MMC 设备中，命令格式如下：

```
ext4write <interface> <dev[:part]> <addr> <absolute filename path> [sizebytes] [file offset]
```

interface 为接口，比如 mmc；dev 是设备号；part 是分区；addr 是要写入的数据在 DRAM 中的起始地址；absolute filename path 是写入的数据文件名字，注意是要带有绝对路径，以‘/’开始；sizebytes 表示要写入多少字节的数据；file offset 为文件偏移。我们可以通过 fatwrite 命令在 uboot 中更新 linux 镜像文件和设备树。我们以更新 linux 镜像文件 uImage 为例，首先将正点原子 STM32MP157 开发板提供的 uImage 镜像文件拷贝到 Ubuntu 中的 tftpboot 目录下，使用命令 tftp 将 uImage 下载到 DRAM 的 0XC0000000 地址处，命令如下：

```
tftp C0000000 uImage
```

下载过程如图 10.3.6.3 所示：

```
STM32MP> tftp c0000000 uImage
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'uImage'.
Load address: 0xc0000000
Loading: ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##########################################
         2.4 MiB/s
done
Bytes transferred = 7313888 (6f99e0 hex)
STM32MP>
```

图 10.3.6.3 uImage 下载过程

uImage 大小为 7313888(0X6F99E0)个字节，接下来使用命令 ext4write 将其写入到 EMMC 的分区 2 中。为了和原有的 uImage 文件区分，我们将要写入的文件命名为 test_uImage，命令如下：

```
ext4write mmc 1:2 c0000000 /test_uImage 0x6f99e0
```

结果如图 10.4.6.4 所示：

```
STM32MP> ext4write mmc 1:2 c0000000 /test_uImage 0x6f99e0
File System is consistent
update journal finished
7313888 bytes written in 335 ms (20.8 MiB/s)
STM32MP>
```

图 10.3.6.4 将 uImage 烧写到 EMMC 扇区 2 中

完成以后使用“ext4ls”命令查看一下 EMMC 分区 2 里面的文件，结果如图 10.3.6.5 所示：

```
STM32MP> ext4ls mmc 1:2
<DIR>       1024 .
<DIR>       1024 ..
            2943 boot.scr.uimg
<DIR>       1024 lost+found
<DIR>       1024 mmc0_extlinux
<DIR>       1024 mmc1_extlinux
           92670 splash.bmp
           74594 stm32mp157d-atk.dtb
           74014 stm32mp157d-atk-hdmi.dtb
           74510 stm32mp157d-atk-spdif.dtb
         8125872 uImage
         3632241 uInitrd
         7313888 test_uImage
STM32MP>
```

图 10.3.6.5 写入的 test_uImage 文件

从图 10.3.6.5 可以看出，test_uImage 写入成功。

10.3.7 BOOT 操作命令

uboot 的本质工作是引导 Linux，所以 uboot 肯定有相关的 boot(引导)命令来启动 Linux。常用的跟 boot 有关的命令有：bootm、bootz 和 boot。

1、bootm 命令

要启动 Linux，需要先将 Linux 镜像文件拷贝到 DRAM 中，如果使用到设备树的话也需要将设备树拷贝到 DRAM 中。可以从 EMMC 或者 NAND 等存储设备中将 Linux 镜像和设备树文件拷贝到 DRAM，也可以通过 nfs 或者 tftp 将 Linux 镜像文件和设备树文件下载到 DRAM 中。不管用那种方法，只要能将 Linux 镜像和设备树文件存到 DRAM 中就行，然后使用 bootm 命令来启动，bootm 命令用于启动 uImage 镜像文件，bootm 命令格式如下：

```
bootm [addr [arg ...]]
```

命令 bootm 主要有三个参数，addr 是 Linux 镜像文件在 DRAM 中的位置，后面的“arg…”表示其他可选的参数，比如要指定 initrd 的话，第二个参数就是 initrd 在 DRAM 中的地址。如果 Linux 内核使用设备树的话还需要第三个参数，用来指定设备树在 DRAM 中的地址，如果不需要 initrd 的话第二个参数就用‘-’来代替。

现在我们使用网络和 EMMC 两种方法来启动 Linux 系统，首先将 STM32MP157 开发板的 Linux 镜像和设备树发送到 Ubuntu 主机中的 tftpboot 文件夹下。Linux 镜像文件前面已经放到了 tftpboot 文件夹中，现在把设备树文件放到 tftpboot 文件夹里面。以 EMMC 核心板为例，将开发板光盘→8、系统镜像→1、教程系统镜像→linux-mp1-5.4.31-g032f9dcc0-v1.0→stm32mp157d-atk.dtb 发送到 Ubuntu 主机中的 tftpboot 文件夹里面，完成以后的 tftpboot 文件夹如图 10.3.7.1 所示：

```
zuozhongkai@ubuntu:~/linux/tftpboot$ ls
stm32mp157d-atk.dtb  uImage
zuozhongkai@ubuntu:~/linux/tftpboot$
```

图 10.3.7.1 tftpboot 文件夹

现在 Linux 镜像文件和设备树都准备好了，我们先学习如何通过网络启动 Linux，使用 tftp 命令将 uImage 下载到 DRAM 的 0XC2000000 地址处，然后将设备树 stm32mp157d-atk.dtb 下载到 DRAM 中的 0XC400000 地址处，最后使用命令 bootm 启动，命令如下：

```
tftp c2000000 uImage
tftp c4000000 stm32mp157d-atk.dtb
bootm c2000000 - c4000000
```

命令运行结果如图 10.3.7.2 所示：

```
STM32MP> tftp c2000000 uImage
ethernet@5800a000 Waiting for PHY auto negotiation to complete..... done
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'uImage'.
Load address: 0xc2000000
Loading: ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##########################################
         2.3 MiB/s
done
Bytes transferred = 7310904 (6f8e38 hex)
STM32MP> tftp c4000000 stm32mp157d-atk.dtb
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'stm32mp157d-atk.dtb'.
Load address: 0xc4000000
Loading: #####
         2.4 MiB/s
done
Bytes transferred = 63833 (f959 hex)
STM32MP> bootm c2000000 - c4000000
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31
   Created:      2020-12-19   9:34:55 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    7310840 Bytes = 7 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffed000, end cffff958 ... OK

Starting kernel ...

Booting Linux on physical CPU 0x0
Linux version 5.4.31 (zuozhongkai@ubuntu) (gcc version 9.2.1 20191025 (GNU Toolchain for the A-profile Archit
ecture 9.2-2019.12 (arm-9.10))) #1 SMP PREEMPT Sat Dec 19 17:31:56 CST 2020
CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
CPU: div instructions available: patching division code
CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
OF: fdt: Machine model: STMicroelectronics STM32MP157D eval daughter
```

1、下载linux镜像uImage

2、下载设备树stm32mp157d-atk.dtb

3、使用命令“bootm”启动linux系统

4、linux启动信息

图 10.3.7.2 通过网络启动 Linux

从图 10.3.7.2 可以看出，此时 Linux 内核已经启动成功，相关的 log 信息已经打印出来了。

注意！只要打印出 Linux 内核启动信息就说明 Linux 系统启动成功，由于没有给定 bootargs 参数，所以 Linux 系统启动最终会失败，因为找不到根文件系统！

如果我们要从 EMMC 中启动 Linux 系统的话只需要使用命令 ext4load 将 uImage 和 stm32mp157d-atk.dtb 从 EMMC 的分区 2 中拷贝到 DRAM 中，然后使用命令 bootm 启动即可。先使用命令 ext4ls 查看要下 EMMC 的分区 2 中有没有 uImage 和 stm32mp157d-atk.dtb，如果没有的话请先使用 STM32CubeProgrammer 将正点原子出厂系统烧写到开发板中。

使用命令 ext4load 将 uImage 和 stm32mp157d-atk.dtb 文件拷贝到 DRAM 中，地址分别为 0XC2000000 和 0XC4000000，最后使用 bootm 启动，命令如下：

```
ext4load mmc 1:2 c2000000 uImage
ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb
bootm c2000000 - c4000000
```

命令运行结果如图 10.3.7.5 所示：

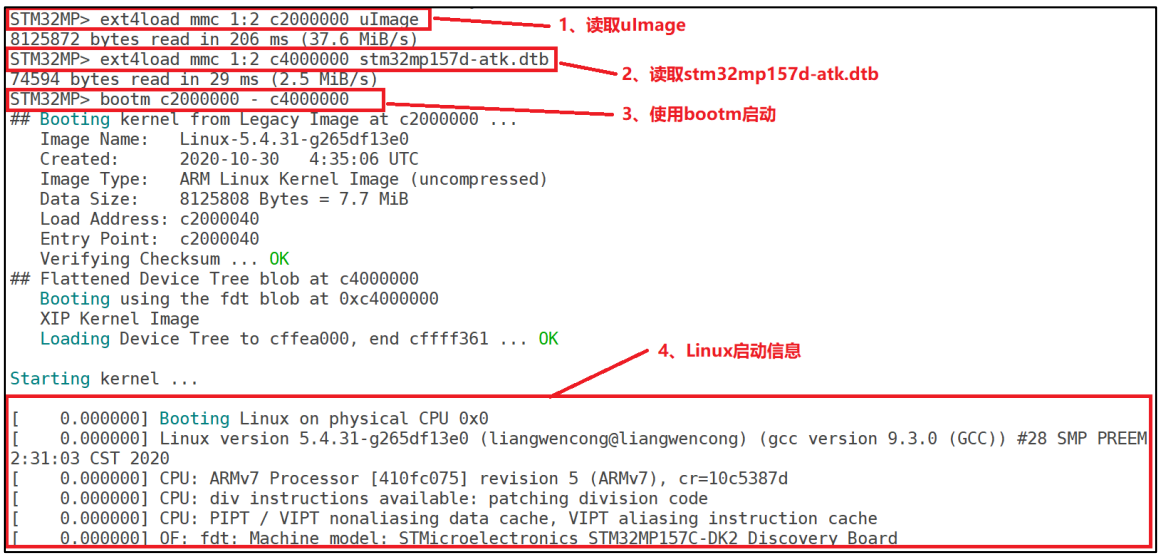


图 10.3.7.5 从 EMMC 中启动 Linux

2、bootz 命令

bootz 和 bootm 功能类似，但是 bootz 用于启动 zImage 镜像文件，bootz 命令格式如下：

```
bootz [addr [initrd[:size]] [fdt]]
```

命令 bootz 有三个参数，addr 是 Linux 镜像文件在 DRAM 中的位置，initrd 是 initrd 文件在 DRAM 中的地址，如果不使用 initrd 的话使用‘-’代替即可，fdt 就是设备树文件在 DRAM 中的地址，使用方法和 bootm 一模一样，只是所引导的 Linux 镜像格式不同，NXP 的 I.MX6ULL 就是使用 bootz 命令来引导 Linux 内核的。

3、boot 和 bootd 命令

注意！ST 官方 uboot 并没有使能 boot 和 bootd 这两个命令，需要自行配置 uboot 来启动这两个命令，正点原子出厂系统已经使能了这两个命令，后面 uboot 移植章节会给大家讲解如何使能这两个命令。

如果你使用别的厂商的开发板，在学习本小节的时候发现其 uboot 下没有使能 boot 和 bootd 命令，请联系对应的厂商，让其提供使能了 boot 和 bootd 命令的 uboot 内核和相应的 uboot 镜像文件，然后使用新的 uboot 镜像完成本实验。

boot 和 bootd 其实是一个命令，它们最终执行的是同一个函数。为了方便起见，后面就统一使用 boot 命令，此命令也是用来启动 Linux 系统的，只是 boot 会读取环境变量 bootcmd 来启动 Linux 系统，bootcmd 是一个很重要的环境变量！其名字分为“boot”和“cmd”，也就是“引导”和“命令”，说明这个环境变量保存着引导命令，其实就是多条启动命令的集合，具体的引导命令内容是可以修改的。比如我们要想使用 tftp 命令从网络启动 Linux 那么就可以设置 bootcmd 为“tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000”，然后使用 saveenv 将 bootcmd 保存起来。然后直接输入 boot 命令即可从网络启动 Linux 系统，命令如下：

```
setenv bootcmd 'tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
saveenv
boot
```

运行结果如图 10.3.7.6 所示：

```
STM32MP> setenv bootcmd 'tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
STM32MP> saveenv
Saving Environment to MMC... Writing to redundant MMC(1)... OK
STM32MP> boot
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'uImage'.
Load address: 0xc2000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ################################
         2.4 MiB/s
done
Bytes transferred = 8125872 (7bfdb0 hex)
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'stm32mp157d-atk.dtb'.
Load address: 0xc4000000
Loading: ######
         2 MiB/s
done
Bytes transferred = 74594 (12362 hex)
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31-g265df13e0
   Created:      2020-10-30   4:35:06 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    8125808 Bytes = 7.7 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffea000, end cffff361 ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 5.4.31-g265df13e0 (liangwencong@liangwencong) (gcc version 9.3.0 (GCC)) #28 SMP
PT Fri Oct 30 12:31:03 CST 2020
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[    0.000000] OF: fdt: Machine model: STMicroelectronics STM32MP157C-DK2 Discovery Board
```

2、使用命令boot启动linux

1、设置环境变量并保存

3、bootcmd环境变量运行，获取内核并启动

图 10.3.7.6 设置 bootcmd 从网络启动 Linux

前面说过 uboot 倒计时结束以后就会启动 Linux 系统，其实就是执行的 bootcmd 中的启动命令。只要不修改 bootcmd 中的内容，以后每次开机 uboot 倒计时结束以后都会使用 tftp 命令从网络下载 uImage 和 stm32mp157d-atk.dtb，然后启动 Linux 内核。

如果想从 EMMC 启动系统，那就设置 bootcmd 环境变量为“ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000”，然后使用 boot 命令启动即可，命令如下：

```
setenv bootcmd 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
saveenv
boot
```

运行结果如图 10.3.7.7 所示：

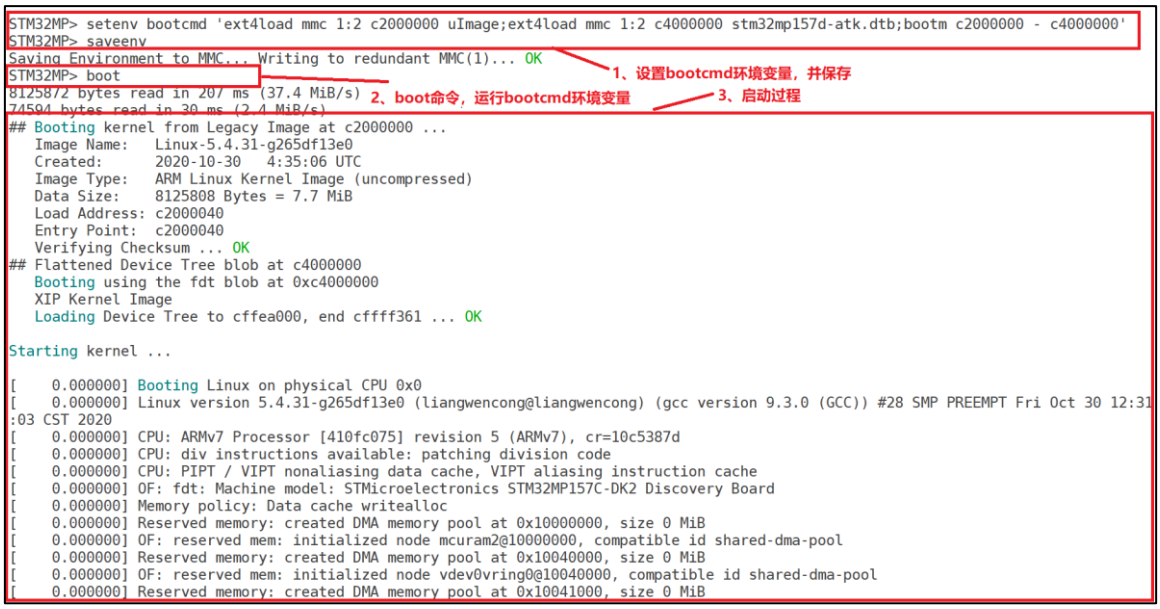


图 10.3.7.7 设置 bootcmd 从 EMMC 启动 Linux

如果不修改 bootcmd 的话，每次开机 uboot 倒计时结束以后都会自动从 EMMC 里面读取 uImage 和 stm32mp157d-atk.dtb，然后启动 Linux。

在启动 Linux 内核的时候可能会遇到如下错误：

“---[end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)]---”

如图 10.3.7.8 所示：

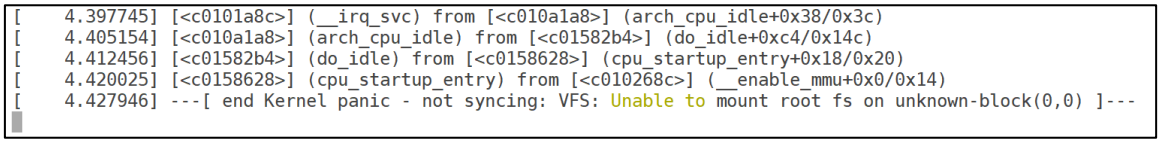


图 10.3.7.8 Linux 内核启动失败

这个错误的原因是 linux 内核没有找到根文件系统，这个很正常，因为没有在 bootargs 环境变量中指定根文件系统路径，关于 bootargs 环境变量后面会讲解！此处我们重点是验证 boot 命令，linux 内核已经成功启动了，说明 boot 命令工作正常。

10.3.8 UMS 命令

在 uboot 下我们可以将开发板虚拟成一个 U 盘，我们可以选择使用哪个 Flash 作为这个 U 盘的存储器，比如将正点原子 STM32MP157 开发板上的 EMMC 或者 SD 卡虚拟成 U 盘。当我们将 EMMC 虚拟成 U 盘以后就可以直接在电脑上向开发板拷贝文件了，比如我们在产品开发阶段，就可以直接在 uboot 下将某个文件拷贝到开发板的根文件系统中，这样就不需要进入系统或者通过网络来替换文件。

uboot 提供的 ums 命令就是来完成此功能的，ums 命令格式如下：

```
ums <USB_controller> [<devtype>] <dev[:part]>
```

其中 USB_controller 是 usb 接口索引，有的开发板有多个 USB SLAVE 接口，具体要使用哪个就可以通过 USB_controller 参数指定。正点原子 STM32MP157 开发板的只有一个 USB_OTG 口可以作为 USB SLAVE，对应的索引为 0。Devtype 是要挂载的设备，默认为 mmc，dev[:part]是要挂载的 Flash 设备，part 是要挂载的分区。

这里我们将开发板的 EMMC 挂载到电脑上，首先使用 USB Type-C 线将开发板的 USB_OTG

口与电脑连接起来，然后用以下命令启动。

```
ums 0 mmc 1
```

运行结果如图 10.3.8.1 所示：

```
STM32MP> ums 0 mmc 1
UMS: LUN 0, dev 1, hwpart 0, sector 0x0, count 0xe90000
\
```

图 10.3.8.1 ums 命令运行结果

挂载成功以后就会在电脑上有多个 U 盘，U 盘数量取决于你当前开发板 EMMC 分区数量，如果烧写过出厂系统的话默认是 3 个 U 盘，如图 10.3.8.2 所示：

图 10.3.8.1 U 盘

注意，在 Windows 下这三个 U 盘是无法操作的，因为这三个 U 盘是 ext4 格式的，而 Windows 是不支持 ext4 格式！所以大家在操作的时候发现 Windows 报出 U 盘识别有问题，让格式化的，千万不要格式化！负责开发板整个 Linux 系统都会被格式化掉！

既然 Windows 识别不了 ext4 格式，那么我们可以将其挂载到 Ubuntu 下，这样就可以正常操作这三个 U 盘了。

如果要结束挂载，在终端下运行先按住 CTRL+C 键就能结束这个挂载。

10.3.9 其他常用命令

uboot 中还有其他一些常用的命令，比如 reset、go、run 和 mtest 等。

1、reset 命令

reset 命令顾名思义就是复位的，输入“reset”即可复位重启，如图 10.3.9.1 所示：

```
STM32MP> reset
resetting ...
INFO:    PSCI Power Domain Map:
INFO:      Domain Node : Level 1, parent_node -1, State ON (0x0)
INFO:      Domain Node : Level 0, parent_node 0, State ON (0x0)
INFO:      CPU Node : MPID 0x0, parent_node 0, State ON (0x0)
INFO:      CPU Node : MPID 0xffffffff, parent_node 0, State OFF (0x2)
NOTICE:  CPU: STM32MP157DAA Rev.Z
NOTICE:  Model: STMicroelectronics STM32MP157D eval daughter
INFO:    Reset reason (0x54):
INFO:      System reset generated by MPU (MPSYSRST)
INFO:    Using EMMC
INFO:      Instance 2
INFO:    Boot used partition fsbl1
NOTICE:  BL2: v2.2-r1.0(debug):463d4d8
NOTICE:  BL2: Built : 02:07:54, Oct 19 2020
INFO:    Using crypto library 'stm32_crypto_lib'
INFO:    BL2: Doing platform setup
INFO:    RAM: DDR3-DDR3L 32bits 533000Khz
```

reset命令

复位以后TF-A重新运行

图 10.3.9.1 reset 命令运行结果

2、go 命令

go 命令用于跳到指定的地址处执行应用，命令格式如下：

```
go addr [arg ...]
```

addr 是应用在 DRAM 中的首地址。

3、run 命令

run 命令用于运行环境变量中定义的命令，比如可以通过“run bootcmd”来运行 bootcmd 中的启动命令，但是 run 命令最大的作用在于运行我们自定义的环境变量。在后面调试 Linux 系统的时候常常要在网络启动和 EMMC 启动之间来回切换，而 bootcmd 只能保存一种启动方式，如果要换另外一种启动方式的话就得重写 bootcmd，会很麻烦。这里我们就可以通过自定义环境变量来实现不同的启动方式，比如定义环境变量 mybootemmc 表示从 emmc 启动，定义 mybootnet 表示从网络启动。如果要切换启动方式的话只需要运行“run mybootxxx(xxx 为 emmc 或 net)”即可。

说干就干，创建环境变量 mybootemmc 和 mybootnet，命令如下：

```
setenv mybootemmc 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000(有空格)
stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
setenv mybootnet 'tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 -
(有空格) c4000000'
saveenv
```

创建环境变量成功以后就可以使用 run 命令来运行 mybootemmc、mybootnet 或 mybootnand 来实现不同的启动：

```
run mybootemmc
```

或

```
run mybootnet
```

4、mtest 命令

mtest 命令是一个简单的内存读写测试命令，可以用来测试自己开发板上的 DDR，命令格式如下：

```
mtest [start [end [pattern [iterations]]]]
```

start 是要测试的 DRAM 开始地址，end 是结束地址，比如我们测试 0XC0000000~0XC0001000 这段内存，输入“mtest C0000000 C0001000”，结果如图 10.3.9.2 所示：

```
  mtd mtdparts mtest
STM32MP> mtest c0000000 c0001000
Testing c0000000 ... c0001000:
Pattern 00000000  Writing..  Reading...Iteration:    527
```

图 10.3.9.2 mtest 命令运行结果

从图 10.3.9.2 可以看出，测试范围为 0XC0000000~0XC0001000，已经测试了 527 次，如果要结束测试就按下键盘上的“Ctrl+C”键。

至此，uboot 常用的命令就讲解完了，如果要使用 uboot 的其他命令，可以查看 uboot 中的帮助信息，或者上网查询一下相应的资料。

10.3.10 MII 命令使用说明

MII 命令是网络相关命令，主要用于读取网络 PHY 芯片寄存器，在 uboot 中调试网络 PHY 芯片的时候非常有用！另外，MDIO 命令也是与网络 PHY 芯片有关的。按道理来讲，uboot 启动以后，如果网络 PHY 芯片工作正常，那么我们可以直接使用 MII 命令来读取核心板上 PHY 芯片的内部寄存器。但是在 STM32MP157 上是不能读取的，原因是 ST 官方提供的 uboot 中，每次网络通信完成以后会关闭 ETHMAC 时钟，所以 MII 命令就会无法工作。解决这个方法很

简单，每次使用 MII 或者 MDIO 命令来操作 PHY 芯片的时候，先使能 ETHMAC 时钟。将 0X50000218 这个寄存器的 bit8~10 置 1 即可。具体步骤如下：

①、先 ping 一下其他网络设备

先在 uboot 中 ping 一下其他设备，注意，此时你的 uboot 网络工作一定要正常！如果 uboot 网络工作不正常就不用 ping 了。ping 一下的目的是让 PHY 芯片与远端设备进行协商，设置好网络状态，方便我们一会使用 mii 命令读取判断。比如我这里 ping 我的电脑，结果如图 10.3.10.1 所示：

```
STM32MP> ping 192.168.1.57
ethernet@5800a000 Waiting for PHY auto negotiation to complete..... done
Using ethernet@5800a000 device
host 192.168.1.57 is alive
STM32MP>
```

图 10.3.10.1 ping 远端设备

②、使能 ETHMAC 时钟

修改寄存器 0X50000218 的 bit8~10 为 1，使用 mm 命令即可，修改过程如图 10.3.10.2 所示：

```
STM32MP> mm 0x50000218
50000218: 00030000 ? 0x00030700
5000021c: 00030700 ? q
STM32MP>
```

图 10.3.10.2 使能 ETHMAC 时钟

重点注意图 10.3.10.2 中红框圈起来的，这个就是修改过程，不要死板的直接照抄！修改之前 0X50000218 寄存器的值为 0X00030000，这个值只是我板子当前的值，可能和你所使用的板子有区别！将 0X00030000 的 bit8~10 置 1 得到的就是 0X00030700，所以我们要写入的就是 0X00030700！一定要根据自己的实际情况来设置！

③、使用 MII 命令获取 PHY 芯片寄存器值

接下来就可以使用 MII 命令读取 PHY 芯片寄存器值，MII 是一系列命令，如图 10.3.10.3 所示：

```
STM32MP> mii
mii - MII utility commands

Usage:
mii device                                - list available devices
mii device <devname>                      - set current device
mii info   <addr>                         - display MII PHY info
mii read   <addr> <reg>                   - read  MII PHY <addr> register <reg>
mii write  <addr> <reg> <data>            - write MII PHY <addr> register <reg>
mii modify <addr> <reg> <data> <mask>     - modify MII PHY <addr> register <reg>
                                            updating bits identified in <mask>
mii dump   <addr> <reg>                   - pretty-print <addr> <reg> (0-5 only)
Addr and/or reg may be ranges, e.g. 2-7.
STM32MP>
```

图 10.3.10.3 mii 系列命令

我们使用“mii info”命令，“mii info”命令格式如下：

```
mii info    <addr>
```

其中 addr 就是 PHY 芯片地址，正点原子 V1.2 版本核心板上的 PHY 芯片为 RTL8211，地址为 0X01。V1.3 版本核心板上的 PHY 芯片为 YT8511，地址为 0X00，大家要根据自己所使用

的核心板版本号来设置“mii info”命令的 addr 参数。我现在用的是 V1.4 核心板，因此命令如下：

```
mii info 0x4
```

结果如图 10.3.10.4 所示：

```
STM32MP> mii info 0x0
PHY 0x00: OUI = 0x0000, Model = 0x10, Rev = 0x0A, 100baseT, FDX
STM32MP>
```

图 10.3.10.4 mii info 命令

大家也可以使用“mii read”命令读取 PHY 芯片的其他寄存器。或者使用“mii write”命令向指定的寄存器写入一个数据。

第十一章 U-boot 顶层 Makefile 详解

在阅读 uboot 源码之前，肯定是要先看一下顶层 Makefile，分析 gcc 版本代码的时候一定是先从顶层 Makefile 开始的，然后再是子 Makefile，这样通过层层分析 Makefile 即可了解整个工程的组织结构。顶层 Makefile 也就是 uboot 根目录下的 Makefile 文件，由于顶层 Makefile 文件内容比较多，所以我们将其分开来看。

11.1 U-Boot 工程目录分析

为了方便，uboot 启动源码分析就在 Windows 下进行，将正点原子提供的 uboot 源码进行解压，解压完成以后的目录如图 11.1.1 所示：

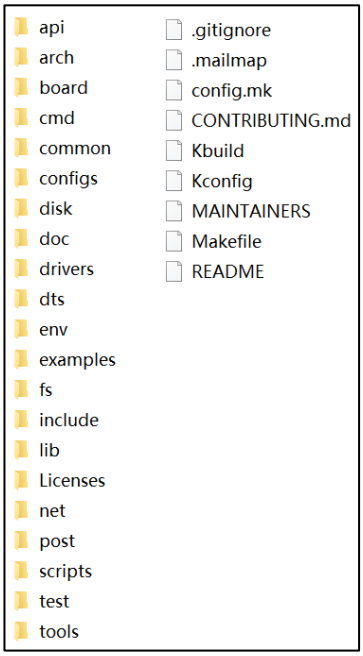


图 11.1.1 未编译的 uboot 目录文件

图 11.1.1 是正点原子提供的未编译的 uboot 源码目录，我们在分析 uboot 源码之前一定要先在 Ubuntu 中编译一下 uboot 源码，因为编译过程会生成一些文件，而生成的这些恰恰是分析 uboot 源码不可或缺的文件。

参考上一章 10.2 小节的方法编译正点原子提供的 uboot 源码，然后将其发送到 Windows 下，编译后的目录如图 11.1.2 所示：

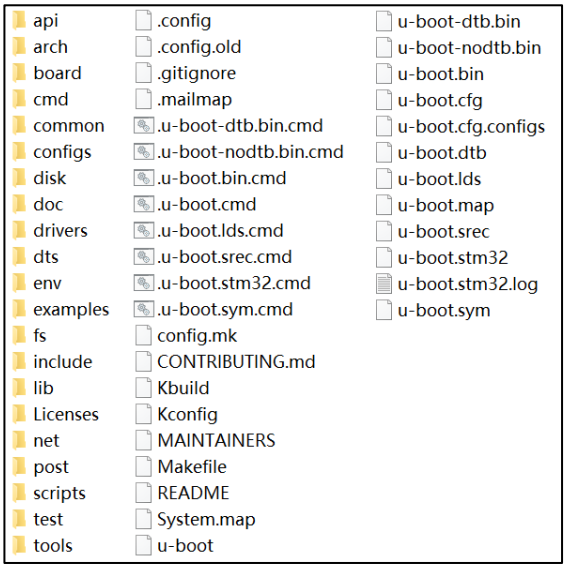


图 11.1.2 编译后的 uboot 源码文件

对比图 11.1.2 和图 11.1.1，可以看出编译后的 uboot 要比没编译之前多了好多文件，这些文件夹或文件的含义见表 11.1.1 所示：

类型	名字	描述	备注
文件夹	api	与硬件无关的 API 函数。	uboot 自带
	arch	与硬件架构体系有关的代码。	
	board	不同板子(开发板)的定制代码。	
	cmd	命令相关代码。	
	common	通用代码。	
	configs	配置文件。	
	disk	磁盘分区相关代码	
	doc	文档。	
	drivers	驱动代码。	
	dts	设备树。	
	examples	示例代码。	
	fs	文件系统。	
	include	头文件。	
	lib	库文件。	
	Licenses	许可证相关文件。	
	net	网络相关代码。	
	post	上电自检程序。	
	scripts	脚本文件。	
	test	测试代码。	
	tools	工具文件夹。	
文件	.config	配置文件，重要的文件。	编译生成的文件
	.config.old	老的配置文件	
	.gitignore	git 工具相关文件。	uboot 自带
	.mailmap	邮件列表。	
	.u-boot.xxx.cmd (一系列)	这是一系列的文件，用于保存着一些命令。	编译生成的文件
	config.mk	某个 Makefile 会调用此文件。	uboot 自带
	Kbuild	用于生成一些和汇编有关的文件。	uboot 自带
	Kconfig	图形配置界面描述文件。	
	MAINTAINERS	维护者联系方式文件。	
	Makefile	主 Makefile，重要文件！	
	README	相当于帮助文档。	uboot 自带
	System.map	系统映射文件	编译出来的文件
	u-boot	编译出来的 u-boot 文件。	
	u-boot.xxx (一系列)	生成的一些 u-boot 相关文件，包括 u-boot.bin、u-boot.stm32.等	

表 11.1.1 uboot 目录列表

表 11.1.1 中的很多文件夹和文件我们都不需要去关心，我们要关注的文件夹或文件如下：

1、arch 文件夹

这个文件夹里面存放着和架构有关的文件，如图 11.1.3 所示：

图 11.1.3 arch 文件夹

从图 11.1.3 可以看出有很多架构，比如 arm、m68k、x86 等，我们现在用的是 ARM 芯片，所以只需要关心 arm 文件夹即可，打开 arm 文件夹里面内容如图 11.1.4 所示：

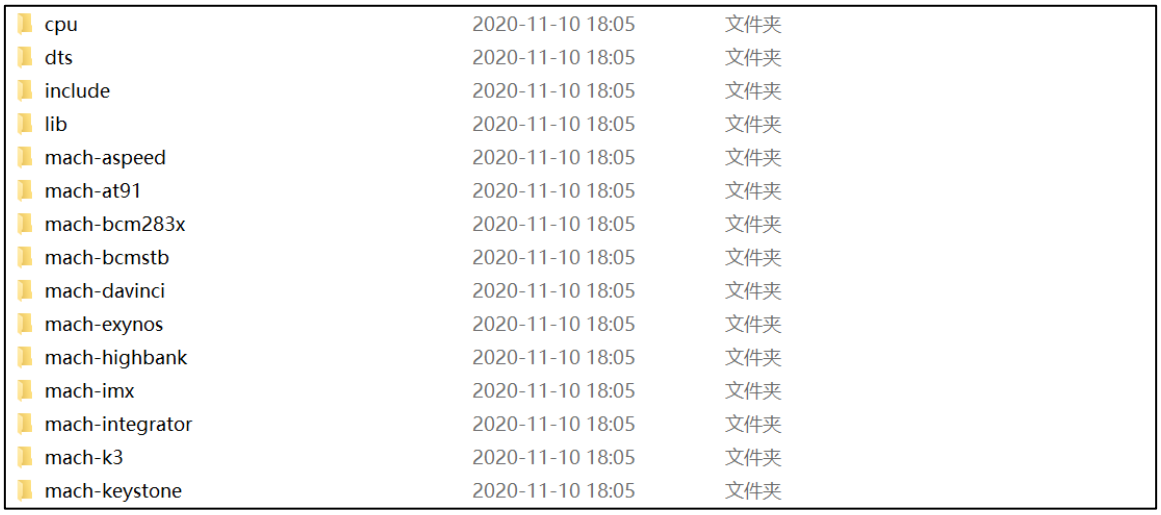


图 11.1.4 arm 文件夹

图 11.1.4 只截取了一部分，还有一部分 mach-xxx 的文件夹。mach 开头的文件夹是跟具体的设备有关的，比如“mach-exynos”就是跟三星的 exyons 系列 CPU 有关的文件。我们使用的是 STM32MP1，所以要关注“mach-stm32mp”这个文件夹。另外“cpu”这个文件夹也是和 cpu 架构有关的，打开以后如图 11.1.5 所示：

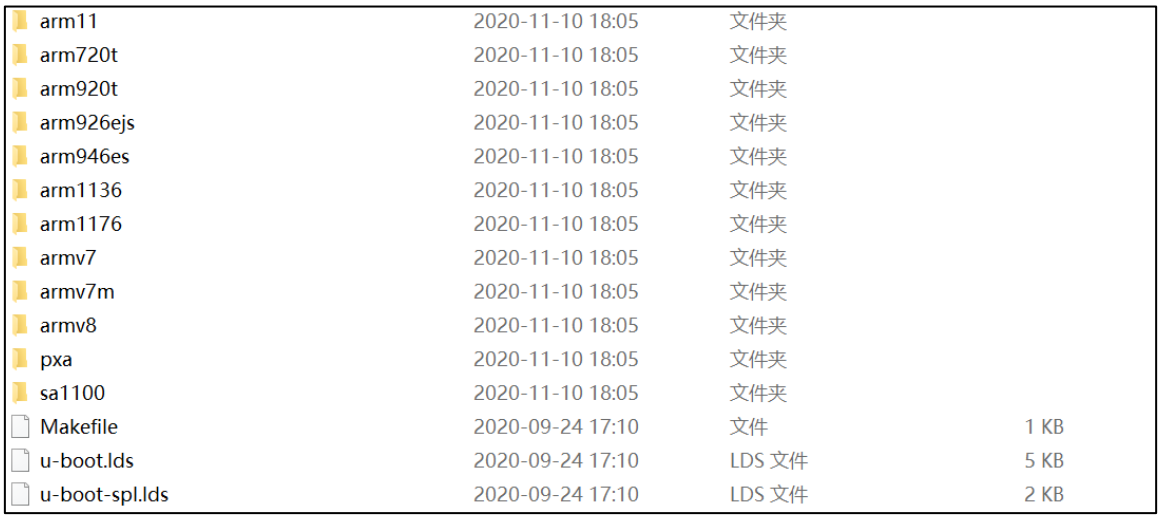


图 11.1.5 cpu 文件夹

从图 11.1.5 可以看出有多种 ARM 架构相关的文件夹，STM32MP1 使用的 Cortex-A7 内核，Cortex-A7 属于 armv7，所以我们要关心“armv7”这个文件夹。cpu 文件夹里面有个名为“u-boot.lds”的链接脚本文件，这个就是 ARM 芯片所使用的 u-boot 链接脚本文件！armv7 这个文件夹里面的文件都是跟 ARMV7 架构有关的，是我们分析 uboot 启动源码的时候需要重点关注的。

2、board 文件夹

board 文件夹就是和具体的板子有关的，打开此文件夹，里面全是不同的板子，毫无疑问正点原子的开发板肯定也在里面(正点原子添加的)，borad 文件夹里面有个名为“st”的文件夹，如图 11.1.6 所示：

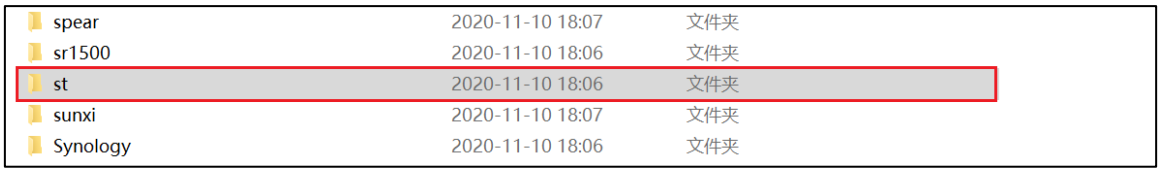


图 11.1.6 freescale 文件夹

所有使用 ST 芯片的板子都放到此文件夹中，打开“st”文件夹，如图 11.1.7 所示：

图 31.1.7 mx6u 相关板子

图 11.1.7 中有 stm32f429-discovery、stm32f469-evaluation、stm32f746-discovery 等目录，对 STM32 熟悉的朋友一眼就看出来，这些是 ST 官方的 STM32 开发板。stm32mp1 这个目录就是针对 STM32MP1 系列芯片对应的板子。

3、configs 文件夹

此文件夹为 uboot 配置文件夹，uboot 是可配置的，但是你要是自己从头开始一个一个项目的配置，那就太麻烦了，因此一般半导体或者开发板厂商都会制作好一个配置文件。我们可以在这个做好的配置文件基础上来添加自己想要的功能，这些半导体厂商或者开发板厂商制作好的配置文件统一命名为“xxx_defconfig”，xxx 表示开发板名字，这些 defconfig 文件都存放在 configs 文件夹，因此，ST 官方开发板和正点原子的开发板配置文件肯定也在这个文件夹中，如图 11.1.8 所示：

图 11.1.8 正点原子开发板配置文件

图 11.1.8 中 stm32mp157d_atk_defconfig 就是正点原子 STM32MP157 开发板对应的默认配置文件。使用“make xxx_defconfig”命令即可配置 uboot，比如：

```
make stm32mp157d_atk_defconfig
```

上述命令就是配置正点原子 STM32MP157 核心板所使用的 uboot。

在编译 uboot 之前一定要使用 defconfig 来配置 uboot！

上一章 10.2 小节编译 uboot 的时候，有下面一行命令：

```
make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- stm32mp157d_atk_defconfig
```

这个就是调用 stm32mp157d_atk_defconfig 来配置 uboot，只是这个命令还带了一些其它参数而已，ARCH 用来指定 CPU 架构为 arm，CROSS_COMPILE 用来指定所使用的交叉编译器为 arm-none-linux-gnueabihf-gcc。

4、.u-boot.xxx_cmd 文件

.u-boot.xxx_cmd 是一系列的文件，这些文件都是编译生成的，都是一些命令文件，比如文件.u-boot.bin.cmd，看名字应该是和 u-boot.bin 有关的，此文件的内容如下：

示例代码 11.1.1 .u-boot.bin.cmd 代码

```
1 cmd_u-boot.bin := cp u-boot-dtb.bin u-boot.bin
```

.u-boot.bin.cmd 里面定义了一个变量：cmd_u-boot.bin，此变量的值为“cp u-boot-dtb.bin u-boot.bin”，也就是拷贝一份 u-boot-dtb.bin 文件，并且重命名为 u-boot.bin，这个就是 u-boot.bin 的来源，来自于文件 u-boot-dtb.bin。

那么 u-boot-dtb.bin 是怎么来的呢？文件.u-boot-dtb.bin.cmd 就是用于生成 u-boot.dtb.bin 的，此文件内容如下：

示例代码 11.1.2 cmd_u-boot-dtb.bin 代码

```
1 cmd_u-boot-dtb.bin := cat u-boot-nodtb.bin dts/dt.dtb > u-boot-
dtb.bin
```

cmd_u-boot-dtb.bin 用于将设备树.dtb 文件和 uboot 的 bin 文件结合起来，STM32MP157 的 uboot 用到了设备树(Device Tree)，因此最终生成的 uboot bin 文件里面要有.dtb 内容。u-boot-nodtb.bin 就是原始的、不含 dtb 的 uboot bin 文件。

u-boot-nodtb.bin 是用.u-boot-nodtb.bin.cmd 文件生成的，内容如下

示例代码 11.1.3 .u-boot-nodtb.bin.cmd 代码

```
1 cmd_u-boot-nodtb.bin := arm-none-linux-gnueabihf-objcopy --gap-fill=0x
ff  -j .text -j .secure_text -j .secure_data -j .rodata -j .hash -j .data
 -j .got -j .got.plt -j .u_boot_list -j .rel.dyn -j .binman_sym_table -j
 .text_rest -j .dtb.init.rodata -j .efi_runtime -j .efi_runtime_rel -O bi
nary   u-boot u-boot-nodtb.bin
```

这里用到了 arm-none-linux-gnueabihf-objcopy，使用 objcopy 将 ELF 格式的 u-boot 文件转换为二进制的 u-boot-nodtb.bin 文件。

文件 u-boot 是 ELF 格式的文件，文件.u-boot.cmd 用于生成 u-boot，文件内容如下：

示例代码 11.1.4 .u-boot.cmd 代码

```
cmd_u-boot := arm-none-linux-gnueabihf-ld.bfd  -pie  --gc-sections -
Bstatic  --no-dynamic-linker -Ttext 0xC0100000 -o u-boot -T u-boot.lds
arch/arm/cpu/armv7/start.o --start-group  arch/arm/cpu/built-in.o
arch/arm/cpu/armv7/built-in.o  arch/arm/lib/built-in.o  arch/arm/mach-
stm32mp/built-in.o  board/st/common/built-in.o  board/st/stm32mp1/built-
in.o  cmd/built-in.o  common/built-in.o  disk/built-in.o  drivers/built-
in.o  drivers/dma/built-in.o  drivers/gpio/built-in.o
drivers/i2c/built-in.o  drivers/net/built-in.o  drivers/net/phy/built-
in.o  drivers/power/built-in.o  drivers/power/battery/built-in.o
drivers/power/domain/built-in.o  drivers/power/fuel_gauge/built-in.o
drivers/power/mfd/built-in.o  drivers/power/pmic/built-in.o
drivers/power/regulator/built-in.o  drivers/serial/built-in.o
drivers/spi/built-in.o  drivers/usb/cdns3/built-in.o
drivers/usb/common/built-in.o  drivers/usb/dwc3/built-in.o
drivers/usb/emul/built-in.o  drivers/usb/eth/built-in.o
drivers/usb/gadget/built-in.o  drivers/usb/gadget/udc/built-in.o
drivers/usb/host/built-in.o  drivers/usb/musb-new/built-in.o
drivers/usb/musb/built-in.o  drivers/usb/phy/built-in.o
drivers/usb/ulpi/built-in.o  env/built-in.o  fs/built-in.o  lib/built-
in.o  net/built-in.o --end-group arch/arm/lib/eabi_compat.o
arch/arm/lib/lib.a -Map u-boot.map;  true
```

.u-boot.cmd 使用到了 arm-none-linux-gnueabihf-ld.bfd，也就是链接工具，使用 ld.bfd 将各个 built-in.o 文件链接在一起就形成了 u-boot 文件。uboot 在编译的时候会将同一个目录中的所有.c 文件都编译在一起，并命名为 built-in.o，相当于将众多的.c 文件对应的.o 文件集合在一起，这个就是 u-boot 文件的来源。

如果我们要向 STM32MP157 内部烧写 uboot，此时烧写的是 u-boot.stm32 文件，而不是 u-boot.bin 文件。u-boot.stm32 是由文件.u-boot.stm32.cmd 来完成的，此文件内容如下：

示例代码 11.1.5 .u-boot.stm32.cmd 代码

```
cmd_u-boot.stm32 := ./tools/mkimage -T stm32image -a 0xC0100000 -e
0xC0100000 -d u-boot.bin u-boot.stm32 >u-boot.stm32.log  && cat u-
boot.stm32.log
```

可以看出，这里用到了工具 tools/mkimage，和 stm32image，通过这两个工具将 u-boot.bin 转换为 u-boot.stm32。

文件.u-boot.lds.cmd 就是用于生成 u-boot.lds 链接脚本的，由于.u-boot.lds.cmd 文件内容太多，这里就不列出来了。uboot 根目录下的 u-boot.lds 链接脚本就是来源于 arch/arm/cpu/u-boot.lds 文件。

还有一些其它的.u-boot.lds.xxx.cmd 文件，大家自行分析一下，关于.u-boot.lds.xxx.cmd 文件就讲解到这里。

5、Makefile 文件

这个是顶层 Makefile 文件，Makefile 是支持嵌套的，也就是顶层 Makefile 可以调用子目录中的 Makefile 文件。Makefile 嵌套在大项目中很常见，一般大项目里面所有的源代码都不会放到同一个目录中，各个功能模块的源代码都是分开的，各自存放在各自的目录中。每个功能模块目录下都有一个 Makefile，这个 Makefile 只处理本模块的编译链接工作，这样所有的编译链接工作就不用全部放到一个 Makefile 中，可以使得 Makefile 变得简洁明了。

uboot 源码根目录下的 Makefile 是顶层 Makefile，他会调用其它的模块的 Makefile 文件，比如 drivers/adc/Makefile。当然了，顶层 Makefile 要做的工作可远不止调用子目录 Makefile 这么简单，关于顶层 Makefile 的内容我们稍后会有详细的讲解。

6、u-boot.xxx 文件

u-boot.xxx 同样也是一系列文件，包括 u-boot、u-boot-dtb.bin、u-boot-nodtb.bin、u-boot.bin、u-boot.cfg、u-boot.dtb、u-boot.lds、u-boot.map、u-boot.srec、u-boot.stm32 和 u-boot.sym，这些文件的含义如下：

u-boot：编译出来的 ELF 格式的 uboot 镜像文件。

u-boot-dtb.bin：编译出来的含有设备树.dtb 的 uboot 镜像文件。

u-boot-nodtb.bin：编译出来的不含有设备树.dtb 的 uboot 镜像文件，和 u-boot.bin 一样。

u-boot.bin：编译出来的二进制格式的 uboot 可执行镜像文件。

u-boot.cfg：uboot 的另外一种配置文件。

u-boot.dtb：uboot 设备树编译后的.dtb 文件。

u-boot.lds：链接脚本。

u-boot.map：uboot 映射文件，通过查看此文件可以知道某个函数被链接到了哪个地址上。

u-boot.srec：S-Record 格式的镜像文件。

u-boot.stm32：最终要写到 STM32MP157 只的 uboot 文件。

u-boot.sym：uboot 符号文件。

7、.config 文件

uboot 配置文件，使用命令“make xxx_defconfig”配置 uboot 以后就会自动生成，.config 内容如下(太长，有省略)：

示例代码 11.1.6 .config 代码

```
1  #
2  # Automatically generated file; DO NOT EDIT.
3  # U-Boot 2020.01-stm32mp-r1 Configuration
4  #
```

```
5  CONFIG_CREATE_ARCH_SYMLINK=y
6  # CONFIG_ARC is not set
7  CONFIG_ARM=y
8  # CONFIG_M68K is not set
9  # CONFIG_MICROBLAZE is not set
10 # CONFIG_MIPS is not set
11 # CONFIG_NDS32 is not set
12 # CONFIG_NIOS2 is not set
13 # CONFIG_PPC is not set
14 # CONFIG_RISCV is not set
15 # CONFIG_SANDBOX is not set
16 # CONFIG_SH is not set
17 # CONFIG_X86 is not set
18 # CONFIG_XTENSA is not set
19 CONFIG_SYS_ARCH="arm"
20 CONFIG_SYS_CPU="armv7"
21 CONFIG_SYS_SOC="stm32mp"
22 CONFIG_SYS_VENDOR="st"
23 CONFIG_SYS_BOARD="stm32mp1"
24 CONFIG_SYS_CONFIG_NAME="stm32mp1"
25 ......
26 #
27 # Boot commands
28 #
29 # CONFIG_CMD_BOOTD is not set
30 CONFIG_CMD_BOOTM=y
31 CONFIG_CMD_BOOTZ=y
32 CONFIG_BOOTM_LINUX=y
33 CONFIG_BOOTM_NETBSD=y
34 ......
35 CONFIG_LIB_DATE=y
36 # CONFIG_UNIT_TEST is not set
```

可以看出.config 文件中都是以“CONFIG_”开始的配置项，这些配置项就是 Makefile 中的变量，因此后面都跟有相应的值，uboot 的顶层 Makefile 或子 Makefile 会调用这些变量值。在.config 中会有大量的变量值为‘y’，这些为‘y’的变量一般用于控制某项功能是否使能，为‘y’的话就表示功能使能，比如：

```
CONFIG_CMD_BOOTM=y
```

如果使能了 bootd 这个命令的话，CONFIG_CMD_BOOTM 就为‘y’。在 cmd/Makefile 中有如下代码：

示例代码 11.1.7 cmd/Makefile 代码

```
1 ifndef CONFIG_SPL_BUILD
2 # core command
3 obj-y += boot.o
```

```
4 obj-$(CONFIG_CMD_BOOTM) += bootm.o
5 obj-y += help.o
6 obj-y += version.o
7
8 ......
9 CFLAGS_ethsw.o := -Wno-enum-conversion
```

在示例代码 11.1.7 中，有如下所示一行代码：

```
obj-$(CONFIG_CMD_BOOTM) += bootm.o
```

CONFIG_CMD_BOOTM=y，将其展开就是：

```
obj-y += bootm.o
```

也就是给 obj-y 追加了一个“bootm.o”，obj-y 包含着所有要编译的文件对应的.o 文件，这里表示需要编译文件 cmd/bootm.c。相当于通过“CONFIG_CMD_BOOTD=y”来使能 bootm 这个命令，进而编译 cmd/bootm.c 这个文件，这个文件实现了命令 bootm。在 uboot 和 Linux 内核中都是采用这种方法来选择使能某个功能，编译对应的源码文件。

8、README

README 文件描述了 uboot 的详细信息，包括 uboot 该如何编译、uboot 中各文件夹的含义、相应的命令等等。建议大家详细的阅读此文件，可以进一步增加对 uboot 的认识。

关于 uboot 根目录中的文件和文件夹的含义就讲解到这里，接下来就要开始分析 uboot 的顶层 Makefile 了。

11.2 U-Boot 顶层 Makefile 分析

用 VScode 打开编译后的 uboot，在阅读 uboot 源码之前，肯定是要先看一下顶层 Makefile，分析 gcc 版本代码的时候一定是先从顶层 Makefile 开始的，然后再是子 Makefile，这样通过层层分析 Makefile 即可了解整个工程的组织结构。顶层 Makefile 也就是 uboot 根目录下的 Makefile 文件，由于顶层 Makefile 文件内容比较多，所以我们将其分开来看。

11.2.1 版本号

顶层 Makefile 一开始是版本号，内容如下(为了方便分析，顶层 Makefile 代码段前段行号采用 Makefile 中的行号，因为 uboot 会更新，因此行号可能会与你所看的顶层 Makefile 有所不同)：

示例代码 11.2.1.1 顶层 Makefile 代码

```
3  VERSION = 2020
4  PATCHLEVEL = 01
5  SUBLEVEL =
6  EXTRAVERSION = -stm32mp-r1
7  NAME =
```

VERSION 是主版本号，PATCHLEVEL 是补丁版本号，SUBLEVEL 是次版本号，这三个一起构成了 uboot 的版本号，比如当前的 uboot 版本号就是“2020.01”。EXTRAVERSION 是附加版本信息，NAME 是和名字有关的。

11.2.2 MAKEFLAGS 变量

make 是支持递归调用的，也就是在 Makefile 中使用“make”命令来执行其他的 Makefile

文件，一般都是子目录中的 Makefile 文件。假如在当前目录下存在一个“subdir”子目录，这个子目录中又有其对应的 Makefile 文件，那么这个工程在编译的时候其主目录中的 Makefile 就可以调用子目录中的 Makefile，以此来完成所有子目录的编译。主目录的 Makefile 可以使用如下代码来编译这个子目录：

```
$(MAKE) -C subdir
```

$(MAKE)就是调用“make”命令，-C 指定子目录。有时候我们需要向子 make 传递变量，这个时候使用“export”来导出要传递给子 make 的变量即可，如果不希望哪个变量传递给子 make 的话就使用“unexport”来声明不导出:

```
export VARIABLE ……          //导出变量给子 make 。
unexport VARIABLE……         //不导出变量给子 make。
```

有两个特殊的变量：“SHELL”和“MAKEFLAGS”，这两个变量除非使用“unexport”声明，否则的话在整个 make 的执行过程中，它们的值始终自动的传递给子 make。在 uboot 的主 Makefile 中有如下代码：

示例代码 11.2.1.2 顶层 Makefile 代码

```
18 MAKEFLAGS += -rR --include-dir=$(CURDIR)
```

上述代码使用“+=”来给变量 MAKEFLAGS 追加了一些值，“-rR”表示禁止使用内置的隐含规则和变量定义，“--include-dir”指明搜索路径，“$(CURDIR)”表示当前目录。

11.2.3 命令输出

uboot 默认编译是不会在终端中显示完整的命令，都是短命令，如图 11.2.3.1 所示：

```
OBJCOPY u-boot-nodtb.bin
SHIPPED dts/dt.dtb
FDTGREP dts/dt-spl.dtb
CAT     u-boot-dtb.bin
COPY    u-boot.bin
MKIMAGE u-boot.stm32
OBJCOPY u-boot.srec
SYM     u-boot.sym
COPY    u-boot.dtb
CFGCHK  u-boot.cfg
```

图 11.2.3.1 终端短命令输出

在终端中输出短命令虽然看起来很清爽，但是不利于分析 uboot 的编译过程。可以通过设置变量“V=1“来实现完整的命令输出，这个在调试 uboot 的时候很有用，结果如图 11.2.3.2 所示：

```
  cat u-boot-nodtb.bin dts/dt.dtb > u-boot-dtb.bin
  cp u-boot-dtb.bin u-boot.bin
  ./tools/mkimage -T stm32image -a 0xC0100000 -e 0xC0100000 -d u-boot.bin u-boot.stm32 >u-boot
.stm32.log  && cat u-boot.stm32.log
Image Type   : STMicroelectronics STM32 V1.0
Image Size   : 876365 bytes
Image Load   : 0xc0100000
Entry Point  : 0xc0100000
Checksum     : 0x04af437f
Option       : 0x00000001
BinaryType : 0x00000000
  arm-none-linux-gnueabihf-objcopy --gap-fill=0xff  -j .text -j .secure_text -j .secure_data -
j .rodata -j .hash -j .data -j .got -j .got.plt -j .u_boot_list -j .rel.dyn -j .binman_sym_tab
le -j .text_rest -j .dtb.init.rodata -j .efi_runtime -j .efi_runtime_rel -O srec u-boot u-boot
.srec
  arm-none-linux-gnueabihf-objdump -t u-boot > u-boot.sym
  ./scripts/check-config.sh u-boot.cfg ./scripts/config_whitelist.txt .
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 11.2.3.2 终端完整命令输出

顶层 Makefile 中控制命令输出的代码如下：

示例代码 11.3.1 顶层 Makefile 代码

```
90  ifeq ("$(origin V)", "command line")
91    KBUILD_VERBOSE = $(V)
92  endif
93  ifndef KBUILD_VERBOSE
94    KBUILD_VERBOSE = 0
95  endif
96
97  ifeq ($(KBUILD_VERBOSE),1)
98    quiet =
99    Q =
100 else
101   quiet=quiet_
102   Q = @
103 endif
```

上述代码中先使用 ifeq 来判断"$(origin V)"和"command line"是否相等。这里用到了 Makefile 中的函数 origin，origin 和其他的函数不一样，它不操作变量的值，origin 用于告诉你变量是哪来的，语法为：

```
$(origin <variable>)
```

variable 是变量名，origin 函数的返回值就是变量来源，因此$(origin V)就是变量 V 的来源。如果变量 V 是在命令行定义的那么它的来源就是"command line"，这样"$(origin V)"和"command line"就相等了。当这两个相等的时候变量 KBUILD_VERBOSE 就等于 V 的值，比如在命令行中输入“V=1“的话那么 KBUILD_VERBOSE=1。如果没有在命令行输入 V 的话 KBUILD_VERBOSE=0。

第 97 行判断 KBUILD_VERBOSE 是否为 1，如果 KBUILD_VERBOSE 为 1 的话变量 quiet 和 Q 都为空，如果 KBUILD_VERBOSE=0 的话变量 quiet 为“quiet_“，变量 Q 为“@”,综上所述：

V=1 的话:

```
KBUILD_VERBOSE=1
```

quiet= 空 。

Q= 空。

V=0 或者命令行不定义 V 的话：

KBUILD_VERBOSE=0

quiet= quiet_。

Q= @。

Makefile 中会用变量 quiet 和 Q 来控制编译的时候是否在终端输出完整的命令，在顶层 Makefile 中有很多如下所示的命令：

```
$(Q)$(MAKE) $(build)=tools
```

如果 V=0 的话上述命令展开就是“@make $(build)=tools”，make 在执行的时候默认会在终端输出命令，但是在命令前面加上“@”就不会在终端输出命令了。当 V=1 的时候 Q 就为空，上述命令就是“make $(build)=tools”，因此在 make 执行的过程，命令会被完整的输出在终端上。

有些命令会有两个版本，比如：

```
quiet_cmd_sym ?= SYM              $@
cmd_sym ?= $(OBJDUMP) -t $< > $@
```

sym 命令分为“quiet_cmd_sym”和“cmd_sym”两个版本，这两个命令的功能都是一样的，区别在于 make 执行的时候输出的命令不同。quiet_cmd_xxx 命令输出信息少，也就是短命令，而 cmd_xxx 命令输出信息多，也就是完整的命令。

如果变量 quiet 为空的话，整个命令都会输出。

如果变量 quiet 为“quiet_”的话，仅输出短版本。

如果变量 quiet 为“silent_”的话，整个命令都不会输出。

11.2.4 静默输出

上一小节讲了，设置 V=0 或者在命令行中不定义 V 的话，编译 uboot 的时候终端中显示的短命令，但是还是会有命令输出，有时候我们在编译 uboot 的时候不需要输出命令，这个时候就可以使用 uboot 的静默输出功能。编译的时候使用“make -s”即可实现静默输出，顶层 Makefile 中相应的代码如下：

示例代码 11.2.4.1 顶层 Makefile 代码

```
105 # If the user is running make -s (silent mode), suppress echoing of
106 # commands
107
108 ifneq ($(filter 4.%,$(MAKE_VERSION)),)  # make-4
109 ifneq ($(filter %s ,$(firstword x$(MAKEFLAGS))),)
110   quiet=silent_
111 endif
112 else                  # make-3.8x
113 ifneq ($(filter s% -s%,$(MAKEFLAGS)),)
114   quiet=silent_
115 endif
116 endif
117
118 export quiet Q KBUILD_VERBOSE
```

第 108 行判断当前正在使用的编译器版本号是否为 4.x，判断 $(filter

4.%,$(MAKE_VERSION))和“ ”(空)是否相等，如果不相等的话就成立，执行里面的语句。也就是说$(filter 4.%,$(MAKE_VERSION))不为空的话条件就成立，这里用到了 Makefile 中的 filter 函数，这是个过滤函数，函数格式如下：

```
$(filter <pattern...>,<text>)
```

filter 函数表示以 pattern 模式过滤 text 字符串中的单词，仅保留符合模式 pattern 的单词，可以有多个模式。函数返回值就是符合 pattern 的字符串。因此$(filter 4.%,$(MAKE_VERSION))的含义就是在字符串“MAKE_VERSION”中找出符合“4.%”的字符(%为通配符)，MAKE_VERSION 是 make 工具的版本号，ubuntu18.04 里面默认自带的 make 工具版本号为 4.1，大家可以输入“make -v”查看。因此$(filter 4.%,$(MAKE_VERSION))不为空，条件成立，执行 109~111 行的语句。

第 109 行也是一个判断语句，如果$(filter %s ,$(firstword x$(MAKEFLAGS)))不为空的话条件成立，变量 quiet 等于“silent_”。这里也用到了函数 filter，在$(firstword x$(MAKEFLAGS)))中过滤出符合“%s”的单词。到了函数 firstword，函数 firstword 是获取首单词，函数格式如下：

```
$(firstword <text>)
```

firstword 函数用于取出 text 字符串中的第一个单词，函数的返回值就是获取到的单词。当使用“make -s”编译的时候，“-s”会作为 MAKEFLAGS 变量的一部分传递给 Makefile。在顶层 Makfile 中添加如图 11.4.1 所示的代码：

```
109 ifneq ($(filter %s ,$(firstword x$(MAKEFLAGS))),)
110   quiet=silent_
111 endif
112 else                    # make-3.8x
113 ifneq ($(filter s% -s%,$(MAKEFLAGS)),)
114   quiet=silent_
115 endif
116 endif
117
118 export quiet Q KBUILD_VERBOSE
119
120 mytest:
121     @echo 'firstword='$(firstword x$(MAKEFLAGS))
122
```

添加测试代码

图 11.2.4.1 顶层 Makefile 添加代码

图 11.2.4.1 中的两行代码用于输出$(firstword x$(MAKEFLAGS))的结果，我们用以下命令去测试静脉输出：

```
make   ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- -s
```

结果如图 11.2.4.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ make  ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- -s
firstword=xrRs
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 11.2.4.2 修改顶层 Makefile 后的执行结果

从图 11.2.4.2 可以看出第一个单词是“xrRs”，将$(filter %s ,$(firstword x$(MAKEFLAGS)))展开就是$(filter %s, xrRs)，而$(filter %s, xrRs)的返回值肯定不为空，条件成立，quiet=silent_。第 118 行使用 export 导出变量 quiet、Q 和 KBUILD_VERBOSE。

11.2.5 设置编译结果输出目录

uboot 可以将编译出来的目标文件输出到单独的目录中，在 make 的时候使用“O”来指定输出目录，比如“make O=out”就是设置目标文件输出到 out 目录中。这么做是为了将源文件和编译产生的文件分开，当然也可以不指定 O 参数，不指定的话源文件和编译产生的文件都在同一个目录内，一般我们不指定 O 参数。顶层 Makefile 中相关的代码如下：

示例代码 11.2.5.1 顶层 Makefile 代码

```
120 # kbuild supports saving output files in a separate directory.
121 # To locate output files in a separate directory two syntaxes are
122 # supported.In both cases the working directory must be the root of
      the kernel src.
123 # 1) O=
124 # Use "make O=dir/to/store/output/files/"
125 #
126 # 2) Set KBUILD_OUTPUT
127 # Set the environment variable KBUILD_OUTPUT to point to the
128 # directory where the output files shall be placed.
129 # export KBUILD_OUTPUT=dir/to/store/output/files/
130 # make
131 #
132 # The O= assignment takes precedence over the KBUILD_OUTPUT
133 # environment variable.
134
135 # KBUILD_SRC is set on invocation of make in OBJ directory
136 # KBUILD_SRC is not intended to be used by the regular
    # user (for now)
137 ifeq ($(KBUILD_SRC),)
138
139 # OK, Make called in directory where kernel src resides
140 # Do we want to locate output files in a separate directory?
141 ifeq ("$(origin O)", "command line")
142  KBUILD_OUTPUT := $(O)
143 endif
144
145 # That's our default target when none is given on the command line
146 PHONY := _all
147 _all:
148
149 # Cancel implicit rules on top Makefile
150 $(CURDIR)/Makefile Makefile: ;
151
152 ifneq ($(KBUILD_OUTPUT),)
153 # Invoke a second make in the output directory, passing relevant
154 # variables check that the output directory actually exists
155 saved-output := $(KBUILD_OUTPUT)
156 KBUILD_OUTPUT := $(shell mkdir -p $(KBUILD_OUTPUT) && cd
                     $(KBUILD_OUTPUT) \
157                          && /bin/pwd)
......
172 endif # ifneq ($(KBUILD_OUTPUT),)
```

```
173 endif # ifeq ($(KBUILD_SRC),)
```

第 142 行判断“O”是否来自于命令行，如果来自命令行的话条件成立，KBUILD_OUTPUT 就为$(O)，因此变量 KBUILD_OUTPUT 就是输出目录。

第 152 行判断 KBUILD_OUTPUT 是否为空。

第 156 行调用 mkdir 命令，创建 KBUILD_OUTPUT 目录，并且将创建成功以后的绝对路径赋值给 KBUILD_OUTPUT。至此，通过 O 指定的输出目录就存在了。

11.2.6 代码检查

uboot 支持代码检查，使用命令“make C=1”使能代码检查，检查那些需要重新编译的文件。“make C=2”用于检查所有的源码文件，顶层 Makefile 中的代码如下：

示例代码 11.2.6.1 顶层 Makefile 代码

```
183 # Call a source code checker (by default, "sparse") as part of the
184 # C compilation.
185 #
186 # Use 'make C=1' to enable checking of only re-compiled files.
187 # Use 'make C=2' to enable checking of *all* source files,
188 # regardless of whether they are re-compiled or not.
189 #
190 # See the file "doc/sparse.txt" for more details, including
191 # where to get the "sparse" utility.
192
193 ifeq ("$(origin C)", "command line")
194   KBUILD_CHECKSRC = $(C)
195 endif
196 ifndef KBUILD_CHECKSRC
197   KBUILD_CHECKSRC = 0
198 endif
```

第 193 行判断 C 是否来源于命令行，如果 C 来源于命令行，那就将 C 赋值给变量 KBUILD_CHECKSRC，如果命令行没有 C 的话 KBUILD_CHECKSRC 就为 0。

11.2.7 模块编译

在 uboot 中允许单独编译某个模块，使用命令“make M=dir”即可，旧语法“make SUBDIRS=dir”也是支持的。顶层 Makefile 中的代码如下：

示例代码 11.2.7.1 顶层 Makefile 代码

```
200 # Use make M=dir to specify directory of external module to build
201 # Old syntax make ... SUBDIRS=$PWD is still supported
202 # Setting the environment variable KBUILD_EXTMOD take precedence
203 ifdef SUBDIRS
204   KBUILD_EXTMOD ?= $(SUBDIRS)
205 endif
206
207 ifeq ("$(origin M)", "command line")
208   KBUILD_EXTMOD := $(M)
```

```
209 endif
210
211 # If building an external module we do not care about the all: rule
212 # but instead _all depend on modules
213 PHONY += all
214 ifeq ($(KBUILD_EXTMOD),)
215 _all: all
216 else
217 _all: modules
218 endif
219
220 ifeq ($(KBUILD_SRC),)
221         # building in the source tree
222         srctree := .
223 else
224         ifeq ($(KBUILD_SRC)/,$(dir $(CURDIR)))
225                 # building in a subdirectory of the source tree
226                 srctree := ..
227         else
228                 srctree := $(KBUILD_SRC)
229         endif
230 endif
231 objtree     := .
232 src     := $(srctree)
233 obj     := $(objtree)
234
235 VPATH       := $(srctree)$(if $(KBUILD_EXTMOD),:$(KBUILD_EXTMOD))
236
237 export srctree objtree VPATH
```

第 203 行判断是否定义了 SUBDIRS，如果定义了 SUBDIRS，变量 KBUILD_EXTMOD=SUBDIRS，这里是为了支持老语法“make SUBIDRS=dir”

第 207 行判断是否在命令行定义了 M，如果定义了的话 KBUILD_EXTMOD=$(M)。

第 214 行判断 KBUILD_EXTMOD 是否为空，如果为空的话目标_all 依赖 all，因此要先编译出 all。否则的话默认目标_all 依赖 modules，要先编译出 modules，也就是编译模块。一般情况下我们不会在 uboot 中编译模块，所以此处会编译 all 这个目标。

第 220 行判断 KBUILD_SRC 是否为空，如果为空的话就设置变量 srctree 为当前目录，即 srctree 为“.”，一般不设置 KBUILD_SRC。

第 231 行设置变量 objtree 为当前目录。

第 232 和 233 行分别设置变量 src 和 obj，都为当前目录。

第 235 行设置 VPATH。

第 237 行导出变量 scrtree、objtree 和 VPATH。

11.2.8 获取主机架构和系统

接下来顶层 Makefile 会获取主机架构和系统，也就是我们电脑的架构和系统，代码如下：

示例代码 11.2.8.1 顶层 Makefile 代码

```
244 HOSTARCH := $(shell uname -m | \
245     sed -e s/i.86/x86/ \
246         -e s/sun4u/sparc64/ \
247         -e s/arm.*/arm/ \
248         -e s/sa110/arm/ \
249         -e s/ppc64/powerpc/ \
250         -e s/ppc/powerpc/ \
251         -e s/macppc/powerpc/\
252         -e s/sh.*/sh/)
253
254 HOSTOS := $(shell uname -s | tr '[:upper:]' '[:lower:]' | \
255         sed -e 's/\(cygwin\).*/cygwin/')
256
257 export  HOSTARCH HOSTOS
```

第 244 行定义了一个变量 HOSTARCH，用于保存主机架构，这里调用 shell 命令“uname -m”获取架构名称，结果如图 11.2.8.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ uname -m
x86_64
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 11.2.8.1 uname -m 命令

从图 11.2.8.1 可以看出当前电脑主机架构为“x86_64”，shell 中的“|”表示管道，意思是将左边的输出作为右边的输入，sed -e 是替换命令，“sed -e s/i.86/x86/”表示将管道输入的字符串中的“i.86”替换为“x86”，其他的“sed -e s”命令同理。对于我的电脑而言，HOSTARCH=x86_64。

第 254 行定义了变量 HOSTOS，此变量用于保存主机 OS 的值，先使用 shell 命令“uname -s”来获取主机 OS，结果如图 11.8.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ uname -s
Linux
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 11.2.8.2 uname -s 命令

从图 11.2.8.2 可以看出此时的主机 OS 为“Linux”，使用管道将“Linux”作为后面“tr '[:upper:]' '[:lower:]'”的输入，“tr '[:upper:]' '[:lower:]'”表示将所有的大写字母替换为小写字母，因此得到“linux”。最后同样使用管道，将“linux”作为“sed -e 's/\(cygwin\).*/cygwin/'”的输入，用于将 cygwin.*替换为 cygwin。因此，HOSTOS=linux。

第 257 行导出 HOSTARCH=x86_64，HOSTOS=linux。

11.2.9 设置目标架构、交叉编译器和配置文件

编译 uboot 的时候需要设置目标板架构和交叉编译器，“make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf-”就是用于设置 ARCH 和 CROSS_COMPILE，在顶层 Makefile 中代码如下：

示例代码 11.2.9.1 顶层 Makefile 代码

```
261 # set default to nothing for native builds
262 ifeq ($(HOSTARCH),$(ARCH))
263 CROSS_COMPILE ?=
264 endif
265
266 KCONFIG_CONFIG  ?= .config
267 export KCONFIG_CONFIG
```

第 262 行判断 HOSTARCH 和 ARCH 这两个变量是否相等，主机架构(变量 HOSTARCH)是 x86_64，而我们编译的是 ARM 版本 uboot，肯定不相等。所以 CROSS_COMPILE= arm-none-linux-gnueabihf-。从示例代码 11.2.9.1 可以看出，每次编译 uboot 的时候都要在 make 命令后面设置 ARCH 和 CROSS_COMPILE，使用起来很麻烦，可以直接修改顶层 Makefile，在里面加入 ARCH 和 CROSS_COMPILE 的定义，如图 11.2.9.1 所示：

```
261  # set default to nothing for native builds
262  ifeq ($(HOSTARCH),$(ARCH))
263  CROSS_COMPILE ?=
264  endif
265
266  ARCH ?= arm
267  CROSS_COMPILE ?= arm-none-linux-gnueabihf-
268
```

图 11.9.1 定义 ARCH 和 CROSS_COMPILE

按照图 11.2.9.1 所示，直接在顶层 Makefile 里面定义 ARCH 和 CROSS_COMPILE，这样就不用每次编译的时候都要在 make 命令后面定义 ARCH 和 CROSS_COMPILE。

继续回到示例代码 11.2.9.1 中，第 266 行定义变量 KCONFIG_CONFIG，uboot 是可以配置的，这里设置配置文件为.config，.config 默认是没有的，需要使用命令“make xxx_defconfig”对 uboot 进行配置，配置完成以后就会在 uboot 根目录下生成.config。默认情况下.config 和 xxx_defconfig 内容是一样的，因为.config 就是从 xxx_defconfig 复制过来的。如果后续自行调整了 uboot 的一些配置参数，那么这些新的配置参数就添加到了.config 中，而不是 xxx_defconfig。相当于 xxx_defconfig 只是一些初始配置，而.config 里面的才是实时有效的配置。

11.2.10 调用 scripts/Kbuild.include

主 Makefile 会调用文件 scripts/Kbuild.include 这个文件，顶层 Makefile 中代码如下：

示例代码 11.2.10.1 顶层 Makefile 代码

```
379 # We need some generic definitions (do not try to remake the file).
380 scripts/Kbuild.include: ;
381 include scripts/Kbuild.include
```

示例代码 11.2.10.1 中使用“include”包含了文件 scripts/Kbuild.include，此文件里面定义了很多变量，如图 11.2.10.1 所示：

alientek_uboot > scripts > Kbuild.include

```
1  ####
2  # kbuild: Generic definitions
3
4  # Convenient variables
5  comma   := ,
6  quote   := "
7  squote  := '
8  empty   :=
9  space   := $(empty) $(empty)
10 pound := \#
11
12 ###
13 # Name of target with a '.' as filename prefix. foo/bar.o => foo/.bar.o
14 dot-target = $(dir $@).$(notdir $@)
15
16 ###
17 # The temporary file to save gcc -MD generated dependencies must not
18 # contain a comma
19 depfile = $(subst $(comma),_,$(dot-target).d)
20
21 ###
22 # filename of target with directory and extension stripped
23 basetarget = $(basename $(notdir $@))
```

图 11.2.10.1 Kbuild.include 文件

在 uboot 的编译过程中会用到 scripts/Kbuild.include 中的这些变量，后面用到的时候再分析。

11.2.11 交叉编译工具变量设置

上面我们只是设置了 CROSS_COMPILE 的名字，但是交叉编译器其他的工具还没有设置，顶层 Makefile 中相关代码如下：

示例代码 11.2.11.1 顶层 Makefile 代码

```
383 # Make variables (CC, etc...)
384
385 AS        = $(CROSS_COMPILE)as
386 # Always use GNU ld
387 ifneq ($(shell $(CROSS_COMPILE)ld.bfd -v 2> /dev/null),)
388 LD        = $(CROSS_COMPILE)ld.bfd
389 else
390 LD        = $(CROSS_COMPILE)ld
391 endif
392 CC        = $(CROSS_COMPILE)gcc
393 CPP       = $(CC) -E
394 AR        = $(CROSS_COMPILE)ar
395 NM        = $(CROSS_COMPILE)nm
396 LDR       = $(CROSS_COMPILE)ldr
397 STRIP     = $(CROSS_COMPILE)strip
398 OBJCOPY   = $(CROSS_COMPILE)objcopy
399 OBJDUMP   = $(CROSS_COMPILE)objdump
```

11.2.12 导出其他变量

接下来在顶层 Makefile 会导出很多变量，代码如下：

示例代码 11.2.12.1 顶层 Makefile 代码

```
429 export VERSION PATCHLEVEL SUBLEVEL UBOOTRELEASE UBOOTVERSION
```

```
430 export ARCH CPU BOARD VENDOR SOC CPUDIR BOARDDIR
431 export CONFIG_SHELL HOSTCC HOSTCFLAGS HOSTLDFLAGS CROSS_COMPILE AS
          LD CC
432 export CPP AR NM LDR STRIP OBJCOPY OBJDUMP
433 export MAKE LEX YACC AWK PERL PYTHON PYTHON2 PYTHON3
434 export HOSTCXX HOSTCXXFLAGS CHECK CHECKFLAGS DTC DTC_FLAGS
435
436 export KBUILD_CPPFLAGS NOSTDINC_FLAGS UBOOTINCLUDE OBJCOPYFLAGS
          LDFLAGS
437 export KBUILD_CFLAGS KBUILD_AFLAGS
```

这些变量中大部分都已经在前面定义了，我们重点来看一下下面这几个变量：

```
ARCH CPU BOARD VENDOR SOC CPUDIR BOARDDIR
```

这 7 个变量在顶层 Makefile 是找不到的，说明这 7 个变量是在其他文件里面定义的，先来看一下这 7 个变量都是什么内容，在顶层 Makefile 中输入如图 11.2.12.1 所示的内容：

```
440 export KBUILD_CPPFLAGS NOSTDINC_FLAGS UBOOTINCLUDE OBJCOPYFLAGS LDFLAGS
441 export KBUILD_CFLAGS KBUILD_AFLAGS
442
443 mytest:
444     @echo 'ARCH='$(ARCH)
445     @echo 'CPU='$(CPU)
446     @echo 'BOARD='$(BOARD)
447     @echo 'SOC='$(SOC)
448     @echo 'CPUDDIR='$(CPUDIR)
449     @echo 'BOARDDIR='$(BOARDDIR)
450
451 # When compiling out-of-tree modules, put MODVERDIR in the module
```

添加这段代码

图 11.2.12.1 输出变量值

修改好顶层 Makefile 以后执行如下命令：

```
make  ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- mytest
```

结果如图 11.2.12.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ make  ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- mytest
ARCH=arm
CPU=armv7
BOARD=stm32mp1
SOC=stm32mp
CPUDDIR=arch/arm/cpu/armv7
BOARDDIR=st/stm32mp1
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 11.2.12.2 变量结果

从图 11.2.12.2 可以看到这 7 个变量的值，这 7 个变量是从哪里来的呢？在 uboot 根目录下有个文件叫做 config.mk，这 7 个变量就是在 config.mk 里面定义的，打开 config.mk 内容如下：

示例代码 11.2.12.2 config.mk 代码

```
1  # SPDX-License-Identifier: GPL-2.0+
2  #
3  # (C) Copyright 2000-2013
4  # Wolfgang Denk, DENX Software Engineering, wd@denx.de.
5  ########################################################################
6
7  # This file is included from ./Makefile and spl/Makefile.
8  # Clean the state to avoid the same flags added twice.
9  #
10 # (Tegra needs different flags for SPL.
```

```
11  #  That's the reason why this file must be included from
12  #  spl/Makefile too.If we did not have Tegra SoCs, build system
       would be much simpler...)
13  PLATFORM_RELFLAGS :=
14  PLATFORM_CPPFLAGS :=
15  PLATFORM_LDFLAGS :=
16  LDFLAGS :=
17  LDFLAGS_FINAL :=
18  LDFLAGS_STANDALONE :=
19  OBJCOPYFLAGS :=
20  # clear VENDOR for tcsh
21  VENDOR :=
22  ###################################################################
23
24  ARCH := $(CONFIG_SYS_ARCH:"%"=%)
25  CPU := $(CONFIG_SYS_CPU:"%"=%)
26  ifdef CONFIG_SPL_BUILD
27  ifdef CONFIG_TEGRA
28  CPU := arm720t
29  endif
30  endif
31  BOARD := $(CONFIG_SYS_BOARD:"%"=%)
32  ifneq ($(CONFIG_SYS_VENDOR),)
33  VENDOR := $(CONFIG_SYS_VENDOR:"%"=%)
34  endif
35  ifneq ($(CONFIG_SYS_SOC),)
36  SOC := $(CONFIG_SYS_SOC:"%"=%)
37  endif
38
39  # Some architecture config.mk files need to know what CPUDIR is set
40  # to,so calculate CPUDIR before including ARCH/SOC/CPU config.mk
41  # files.Check if arch/$ARCH/cpu/$CPU exists, otherwise assume
       arch/$ARCH/cpu contains
42  # CPU-specific code.
43  CPUDIR=arch/$(ARCH)/cpu$(if $(CPU),/$(CPU),)
44
45  sinclude $(srctree)/arch/$(ARCH)/config.mk
46  sinclude $(srctree)/$(CPUDIR)/config.mk
47
48  ifdef   SOC
49  sinclude $(srctree)/$(CPUDIR)/$(SOC)/config.mk
50  endif
51  ifneq ($(BOARD),)
```

```
52  ifdef   VENDOR
53  BOARDDIR = $(VENDOR)/$(BOARD)
54  else
55  BOARDDIR = $(BOARD)
56  endif
57  endif
58  ifdef   BOARD
59  sinclude $(srctree)/board/$(BOARDDIR)/config.mk
60  endif
61
62  ifdef FTRACE
63  PLATFORM_CPPFLAGS += -finstrument-functions -DFTRACE
64  endif
65
66  ########################################################################
67
68  RELFLAGS := $(PLATFORM_RELFLAGS)
69
70  PLATFORM_CPPFLAGS += $(RELFLAGS)
71  PLATFORM_CPPFLAGS += -pipe
72
73  LDFLAGS += $(PLATFORM_LDFLAGS)
74  LDFLAGS_FINAL += -Bstatic
75
76  export PLATFORM_CPPFLAGS
77  export RELFLAGS
78  export LDFLAGS_FINAL
79  export LDFLAGS_STANDALONE
80  export CONFIG_STANDALONE_LOAD_ADDR
```

第 24 行定义变量 ARCH，值为$(CONFIG_SYS_ARCH:"%"=%)，也就是提取 CONFIG_SYS_ARCH 里面双引号“”之间的内容。比如 CONFIG_SYS_ARCH=“arm”的话，ARCH=arm。

第 25 行定义变量 CPU，值为$(CONFIG_SYS_CPU:"%"=%)。

第 31 定义变量 BOARD，值为(CONFIG_SYS_BOARD:"%"=%)。

第 33 定义变量 VENDOR，值为$(CONFIG_SYS_VENDOR:"%"=%)。

第 36 定义变量 SOC，值为$(CONFIG_SYS_SOC:"%"=%)。

第 43 定义变量 CPUDIR，值为 arch/$(ARCH)/cpu$(if $(CPU),/$(CPU),)。

第 45 sinclude 和 include 的功能类似，在 Makefile 中都是读取指定文件内容，这里读取文件$(srctree)/arch/$(ARCH)/config.mk 的内容。sinclude 读取的文件如果不存在的话不会报错。

第 46 读取文件$(srctree)/$(CPUDIR)/config.mk 的内容。

第 49 取文件$(srctree)/$(CPUDIR)/$(SOC)/config.mk 的内容。

第 53 定义变量 BOARDDIR，如果定义了 VENDOR 那么 BOARDDIR=$(VENDOR)/$(BOARD)，否则的 BOARDDIR=$(BOARD)。

第 59 取文件$(srctree)/board/$(BOARDDIR)/config.mk。

接下来需要找到 CONFIG_SYS_ARCH、CONFIG_SYS_CPU、CONFIG_SYS_BOARD、CONFIG_SYS_VENDOR 和 CONFIG_SYS_SOC 这 5 个变量的值。这 5 个变量在 uboot 根目录下的.config 文件中有定义，定义如下：

示例代码 11.2.12.3 .config 文件内容

```
19 CONFIG_SYS_ARCH="arm"
20 CONFIG_SYS_CPU="armv7"
21 CONFIG_SYS_SOC="stm32mp"
22 CONFIG_SYS_VENDOR="st"
23 CONFIG_SYS_BOARD="stm32mp1"
24 CONFIG_SYS_CONFIG_NAME="stm32mp1"
```

根据示例代码 11.2.12.3 可知：

```
ARCH = arm
CPU = armv7
BOARD = stm32mp1
VENDOR = st
SOC = stm32mp
CPUDIR = arch/arm/cpu/armv7
BOARDDIR = st/stm32mp1
```

因此可以推导出 config.mk 中读取的文件有：

```
arch/arm/config.mk
arch/arm/cpu/armv7/config.mk
arch/arm/cpu/armv7/stm32mp/config.mk (此文件不存在)
board/st/stm32mp1/config.mk (此文件不存在)
```

11.2.13 make xxx_defconfig 过程

在编译 uboot 之前要使用“make xxx_defconfig”命令来配置 uboot，那么这个配置过程是如何运行的呢？在顶层 Makefile 中有如下代码：

示例代码 11.2.13.1 顶层 Makefile 代码

```
475 # To make sure we do not include .config for any of the *config
476 # targets catch them early, and hand them over to
477 # scripts/kconfig/Makefile It is allowed to specify more targets
478 # when calling make, including mixing *config targets and build
479 # targets.For example 'make oldconfig all'.
480 # Detect when mixed targets is specified, and make a second
481 # invocation of make so .config is not included in this case either
      (for *config).
482
483 version_h := include/generated/version_autogenerated.h
484 timestamp_h := include/generated/timestamp_autogenerated.h
485 defaultenv_h := include/generated/defaultenv_autogenerated.h
486
487 no-dot-config-targets := clean clobber mrproper distclean \
```

```
488             help %docs check% coccicheck \
489             ubootversion backup tests check qcheck
490
491 config-targets   := 0
492 mixed-targets    := 0
493 dot-config       := 1
494
495 ifneq ($(filter $(no-dot-config-targets), $(MAKECMDGOALS)),)
496    ifeq ($(filter-out $(no-dot-config-targets), $(MAKECMDGOALS)),)
497       dot-config := 0
498    endif
499 endif
500
501 ifeq ($(KBUILD_EXTMOD),)
502       ifneq ($(filter config %config,$(MAKECMDGOALS)),)
503             config-targets := 1
504             ifneq ($(words $(MAKECMDGOALS)),1)
505                   mixed-targets := 1
506             endif
507       endif
508 endif
509
510 ifeq ($(mixed-targets),1)
511 # ==========================================================================
512 # We're called with mixed targets (*config and build targets).
513 # Handle them one by one.
514
515 PHONY += $(MAKECMDGOALS) __build_one_by_one
516
517 $(filter-out __build_one_by_one, $(MAKECMDGOALS)):
__build_one_by_one
518    @:
519
520 __build_one_by_one:
521    $(Q)set -e; \
522    for i in $(MAKECMDGOALS); do \
523       $(MAKE) -f $(srctree)/Makefile $$i; \
524    done
525
526 else
527 ifeq ($(config-targets),1)
528 # ==========================================================================
529 # *config targets only - make sure prerequisites are updated, and
```

```
530 # descend in scripts/kconfig to make the *config target
531
532 KBUILD_DEFCONFIG := sandbox_defconfig
533 export KBUILD_DEFCONFIG KBUILD_KCONFIG
534
535 config: scripts_basic outputmakefile FORCE
536     $(Q)$(MAKE) $(build)=scripts/kconfig $@
537
538 %config: scripts_basic outputmakefile FORCE
539     $(Q)$(MAKE) $(build)=scripts/kconfig $@
540
541 else
542 # ===========================================================================
543 # Build targets only - this includes vmlinux, arch specific
544 # targets, clean targets and others. In general all targets except
     *config targets.
545
546 # Additional helpers built in scripts/
547 # Carefully list dependencies so we do not try to build scripts
548 # twice in parallel
549 PHONY += scripts
550 scripts: scripts_basic include/config/auto.conf
551     $(Q)$(MAKE) $(build)=$(@)
552
553 ifeq ($(dot-config),1)
554 # Read in config
555 -include include/config/auto.conf
......
```

第 483 行定义了变量 version_h，这变量保存版本号文件，此文件是自动生成的。文件 include/generated/version_autogenerated.h 内容如图 11.2.13.1 所示：

```
alientek_uboot > include > generated > h version_autogenerated.h > LD_VERSION_STRING
1 #define PLAIN_VERSION "2020.01-stm32mp-r1"
2 #define U_BOOT_VERSION "U-Boot " PLAIN_VERSION
3 #define CC_VERSION_STRING "arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 9.2.1 20191025"
4 #define LD_VERSION_STRING "GNU ld (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 2.33.1.20191209"
```

图 11.2.13.1 版本号文件

第 484 行定义了变量 timestamp_h，此变量保存时间戳文件，此文件也是自动生成的。文件 include/generated/timestamp_autogenerated.h 内容如图 11.2.13.2 所示：

```
alientek_uboot > include > generated > h timestamp_autogenerated.h
1 #define U_BOOT_DATE "Oct 21 2021"
2 #define U_BOOT_TIME "18:23:50"
3 #define U_BOOT_TZ "+0800"
4 #define U_BOOT_DMI_DATE "10/21/2021"
5 #define U_BOOT_BUILD_DATE 0x20211021
```

图 11.2.13.2 时间戳文件

第 487 行定义了变量 no-dot-config-targets。

第 491 行定义了变量 config-targets，初始值为 0。

第 492 行定义了变量 mixed-targets，初始值为 0。

第 493 行定义了变量 dot-config，初始值为 1。

第 495 行将 MAKECMDGOALS 中不符合 no-dot-config-targets 的部分过滤掉，剩下的如果不为空的话条件就成立。MAKECMDGOALS 是 make 的一个环境变量，这个变量会保存你所指定的终极目标列表，比如执行“make mx6ull_alientek_emmc_defconfig”，那么 MAKECMDGOALS 就为 mx6ull_alientek_emmc_defconfig。很明显过滤后为空，所以条件不成立，变量 dot-config 依旧为 1。

第 501 行判断 KBUILD_EXTMOD 是否为空，如果 KBUILD_EXTMOD 为空的话条件成立，经过前面的分析，我们知道 KBUILD_EXTMOD 为空，所以条件成立。

第 502 行将 MAKECMDGOALS 中不符合“config”和“%config”的部分过滤掉，如果剩下的部分不为空条件就成立，很明显此处条件成立，变量 config-targets=1。

第 504 行统计 MAKECMDGOALS 中的单词个数，如果不为 1 的话条件成立。此处调用 Makefile 中的 words 函数来统计单词个数，words 函数格式如下：

```
$(words <text>)
```

很明显，MAKECMDGOALS 的单词个数是 1 个，所以条件不成立，mixed-targets 继续为 0。综上所述，这些变量值如下：

```
config-targets = 1
mixed-targets = 0
dot-config = 1
```

第 510 行如果变量 mixed-targets 为 1 的话条件成立，很明显，条件不成立。

第 527 行如果变量 config-targets 为 1 的话条件成立，很明显，条件成立，执行这个分支。

第 535 行，没有目标与之匹配，所以不执行。

第 538 行，有目标与之匹配，当输入“make xxx_defconfig”的时候就会匹配到%config 目标，目标“%config”依赖于 scripts_basic、outputmakefile 和 FORCE。FORCE 在顶层 Makefile 后面有如下定义：

示例代码 11.2.13.2 顶层 Makefile 代码

```
2166    PHONY += FORCE
2167    FORCE:
```

可以看出 FORCE 是没有规则和依赖的，所以每次都会重新生成 FORCE。当 FORCE 作为其他目标的依赖时，由于 FORCE 总是被更新过的，因此依赖所在的规则总是会执行的。依赖 scripts_basic 和 outputmakefile 在顶层 Makefile 中的内容如下：

示例代码 11.2.13.3 顶层 Makefile 代码

```
455 # Basic helpers built in scripts/
456 PHONY += scripts_basic
457 scripts_basic:
458     $(Q)$(MAKE) $(build)=scripts/basic
459     $(Q)rm -f .tmp_quiet_recordmcount
460
461 # To avoid any implicit rule to kick in, define an empty command.
462 scripts/basic/%: scripts_basic ;
463
464 PHONY += outputmakefile
```

```
465 # outputmakefile generates a Makefile in the output directory, if
466 # using a separate output directory. This allows convenient use of
467 # make in the output directory.
468 outputmakefile:
469 ifneq ($(KBUILD_SRC),)
470     $(Q)ln -fsn $(srctree) source
471     $(Q)$(CONFIG_SHELL) $(srctree)/scripts/mkmakefile \
472         $(srctree) $(objtree) $(VERSION) $(PATCHLEVEL)
473 endif
```

第 469 行，判断 KBUILD_SRC 是否为空，只有变量 KBUILD_SRC 不为空的时候 outputmakefile 才有意义，经过我们前面的分析 KBUILD_SRC 为空，所以 outputmakefile 无效。只有 scripts_basic 是有效的。

第 457~459 行是 scripts_basic 的规则，其对应的命令用到了变量 Q、MAKE 和 build，其中：

Q=@或为空

MAKE=make

变量 build 是在 scripts/Kbuild.include 文件中有定义，定义如下：

示例代码 11.2.13.4 Kbuild.include 代码段

```
178 ###
179 # Shorthand for $(Q)$(MAKE) -f scripts/Makefile.build obj=
180 # Usage:
181 # $(Q)$(MAKE) $(build)=dir
182 build := -f $(srctree)/scripts/Makefile.build obj
```

从示例代码 11.2.13.4 可以看出 build=-f $(srctree)/scripts/Makefile.build obj，经过前面的分析可知，变量 srctree 为”.”，因此：

build=-f ./scripts/Makefile.build obj

scripts_basic 展开以后如下：

```
scripts_basic:
    @make -f ./scripts/Makefile.build obj=scripts/basic     //也可以没有@，视配置而定
    @rm -f . tmp_quiet_recordmcount                         //也可以没有@
```

scripts_basic 会调用文件./scripts/Makefile.build，这个我们后面再分析。

接着回到示例代码 11.2.13.1 中的%config 处，内容如下：

```
%config: scripts_basic outputmakefile FORCE
    $(Q)$(MAKE) $(build)=scripts/kconfig $@
```

将命令展开就是：

@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig

同样也跟文件./scripts/Makefile.build 有关，我们后面再分析此文件。使用如下命令配置 uboot，并观察其配置过程：

make stm32mp15_atk_defconfig V=1

配置过程如图 11.2.13.1 所示：

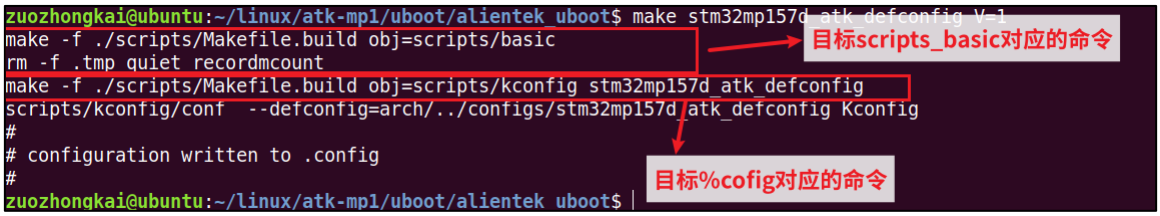


图 11.2.13.1 uboot 配置过程

从图 11.2.13.1 可以看出，我们的分析是正确的，接下来就要结合下面两行命令重点分析一下文件 scripts/Makefile.build。

①、scripts_basic 目标对应的命令

```
@make -f ./scripts/Makefile.build obj=scripts/basic
```

②、%config 目标对应的命令

```
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

11.2.14 Makefile.build 脚本分析

从上一小节可知，“make xxx_defconfig“配置 uboot 的时候如下两行命令会执行脚本 scripts/Makefile.build：

```
@make -f ./scripts/Makefile.build obj=scripts/basic
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

依次来分析一下：

1、scripts_basic 目标对应的命令

scripts_basic 目标对应的命令为：@make -f ./scripts/Makefile.build obj=scripts/basic。打开文件 scripts/Makefile.build，有如下代码：

示例代码 11.2.14.1 Makefile.build 代码段

```
6   # Modified for U-Boot
7   prefix := tpl
8   src := $(patsubst $(prefix)/%,%,$(obj))
9   ifeq ($(obj),$(src))
10  prefix := spl
11  src := $(patsubst $(prefix)/%,%,$(obj))
12  ifeq ($(obj),$(src))
13  prefix := .
14  endif
15  endif
```

第 7 行定义了变量 prefix 值为 tpl。

第 8 行定义了变量 src，这里用到了函数 patsubst，此行代码展开后为：

```
$(patsubst tpl/%,%, scripts/basic)
```

patsubst 是替换函数，格式如下：

```
$(patsubst <pattern>,<replacement>,<text>)
```

此函数用于在 text 中查找符合 pattern 的部分，如果匹配的话就用 replacement 替换掉。pattenr 是可以包含通配符“%”，如果 replacement 中也包含通配符“%”，那么 replacement 中的这个“%”将是 pattern 中的那个“%”所代表的字符串。函数的返回值为替换后的字符串。因

此，第 8 行就是在“scripts/basic”中查找符合“tpl/%”的部分，然后将“tpl/”取消掉，但是“scripts/basic”没有“tpl/”，所以 src= scripts/basic。

第 9 行判断变量 obj 和 src 是否相等，相等的话条件成立，很明显，此处条件成立。

第 10 行和第 7 行一样，只是这里处理的是“spl”，“scripts/basic”里面也没有“spl/”，所以 src 继续为 scripts/basic。

第 12 行因为变量 obj 和 src 相等，所以 prefix=.。

继续分析 scripts/Makefile.build，有如下代码：

示例代码 11.2.14.2 Makefile.build 代码段

```
54  # The filename Kbuild has precedence over Makefile
55  kbuild-dir := $(if $(filter /%,$(src)),$(src),$(srctree)/$(src))
56  kbuild-file := $(if $(wildcard $(kbuild-dir)/Kbuild),$(kbuild-
dir)/Kbuild,$(kbuild-dir)/Makefile)
57  include $(kbuild-file)
```

将 kbuild-dir 展开后为：

```
$(if $(filter /%, scripts/basic),   scripts/basic, ./scripts/basic),
```

因为没有以“/”为开头的单词，所以$(filter /%, scripts/basic)的结果为空，kbuild-dir=./scripts/basic。

将 kbuild-file 展开后为：

```
$(if $(wildcard ./scripts/basic/Kbuild), ./scripts/basic/Kbuild, ./scripts/basic/Makefile)
```

因为 scrpts/basic 目录中没有 Kbuild 这个文件，所以 kbuild-file= ./scripts/basic/Makefile。最后将 57 行展开，即：

```
include ./scripts/basic/Makefile
```

也就是读取 scripts/basic 下面的 Makefile 文件。

继续分析 scripts/Makefile.build，如下代码：

示例代码 11.2.14.3 Makefile.build 代码段

```
114 __build: $(if $(KBUILD_BUILTIN),$(builtin-target) $(lib-target)
$(extra-y)) \
115      $(if $(KBUILD_MODULES),$(obj-m) $(modorder-target)) \
116      $(subdir-ym) $(always)
117     @:
```

__build 是默认目标，因为命令“@make -f ./scripts/Makefile.build obj=scripts/basic”没有指定目标，所以会使用到默认目标：__build。在顶层 Makefile 中，KBUILD_BUILTIN 为 1，KBUILD_MODULES 为 0，因此展开后目标__build 为：

```
__build:$(builtin-target) $(lib-target) $(extra-y)) $(subdir-ym) $(always)
    @:
```

可以看出目标__build 有 5 个依赖：builtin-target、lib-target、extra-y、subdir-ym 和 always。这 5 个依赖的具体内容我们就不通过源码来分析了，直接在 scripts/Makefile.build 中输入图 11.2.14.1 所示内容，将这 5 个变量的值打印出来：

```
114  __build: $(if $(KBUILD_BUILTIN),$(builtin-target) $(lib-target) $(extra-y)) \
115       $(if $(KBUILD_MODULES),$(obj-m) $(modorder-target)) \
116       $(subdir-ym) $(always)
117       @:
118
119       @echo builtin-target = $(builtin-target)
120       @echo lib-target = $(lib-target)
121       @echo extra-y = $(extra-y)
122       @echo subdir-ym = $(subdir-ym)
123       @echo always= $(always)
124
125  # Linus' kernel sanity checking tool
```

添加这段代码

图 11.2.14.1 输出变量

执行如下命令：

```
make stm32mp157d_atk_defconfig V=1
```

结果如图 11.2.14.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ make stm32mp157d_atk_defconfig V=1
make -f ./scripts/Makefile.build obj=scripts/basic
builtin-target =
lib-target =
extra-y =
subdir-ym =
always= scripts/basic/fixdep
rm -f .tmp_quiet_recordmcount
make -f ./scripts/Makefile.build obj=scripts/kconfig stm32mp157d_atk_defconfig
scripts/kconfig/conf  --defconfig=arch/../configs/stm32mp157d_atk_defconfig Kconfig
#
# configuration written to .config
#
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 11.2.14.2 输出结果

从上图可以看出，只有 always 有效，因此__build 最终为：

```
__build: scripts/basic/fixdep
    @:
```

__build 依赖于 scripts/basic/fixdep，所以要先 scripts/basic/fixdep.c 编译，生成 fixdep，前面已经读取了 scripts/basic/Makefile 文件。

综上所述，scripts_basic 目标的作用就是编译出 scripts/basic/fixdep 这个软件。

2、%config 目标对应的命令

%config 目标对应的命令为：@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig，各个变量值如下：

```
src= scripts/kconfig
kbuild-dir = ./scripts/kconfig
kbuild-file = ./scripts/kconfig/Makefile
include ./scripts/kconfig/Makefile
```

可以看出，Makefilke.build 会读取 scripts/kconfig/Makefile 中的内容，此文件有如下所示内容：

示例代码 11.2.14.4 scripts/lconfig/MaKEFILE 代码段

```
127 %_defconfig: $(obj)/conf
128     $(Q)$< $(silent) --defconfig=arch/$(SRCARCH)/configs/$@
$(Kconfig)
129
130 # Added for U-Boot (backward compatibility)
```

```
131 %_config: %_defconfig
132     @:
```

目标%_defconfig 刚好和我们输入的 xxx_defconfig 匹配，所以会执行这条规则。依赖为$(obj)/conf，展开后就是 scripts/kconfig/conf。接下来就是检查并生成依赖 scripts/kconfig/conf。conf 是主机软件。到这里我们就打住，不要纠结 conf 是怎么编译出来的，否则就越陷越深，太绕了，像 conf 这种主机所使用的工具类软件我们一般不关心它是如何编译产生的。如果一定要看是 conf 是怎么生成的，可以输入如下命令重新配置 uboot，在重新配置 uboot 的过程中就会输出 conf 编译信息。

```
make distclean
make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- stm32mp157d_atk_defconfig V=1
```

结果如图 11.2.14.3 所示：

```
CURSES_LOC="<ncurses.h>" -DLOCALE  -Iscripts/kconfig -c -o scripts/kconfig/zconf.tab.o scripts/kconfig/zconf.tab.c
  cc  -o scripts/kconfig/conf scripts/kconfig/conf.o scripts/kconfig/zconf.tab.o
scripts/kconfig/conf  --defconfig=arch/../configs/stm32mp157d_atk_defconfig Kconfig
#
# configuration written to .config
#
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 11.2.14.3 编译过程

得到 scripts/kconfig/conf 以后就要执行目标%_defconfig 的命令：

```
$(Q)$< $(silent) --defconfig=arch/$(SRCARCH)/configs/$@ $(Kconfig)
```

相关的变量值如下：

```
silent=-s 或为空
SRCARCH=..
Kconfig=Kconfig
```

将其展开就是：

```
@ scripts/kconfig/conf --defconfig=arch/../configs/xxx_defconfig Kconfig
```

上述命令用到了 xxx_defconfig 文件，比如 mx6ull_alientek_emmc_defconfig。这里会将 mx6ull_alientek_emmc_defconfig 中的配置输出到.config 文件中，最终生成 uboot 根目录下的.config 文件。

这个就是命令 make xxx_defconfig 执行流程，总结一下如图 11.2.14.4 所示：

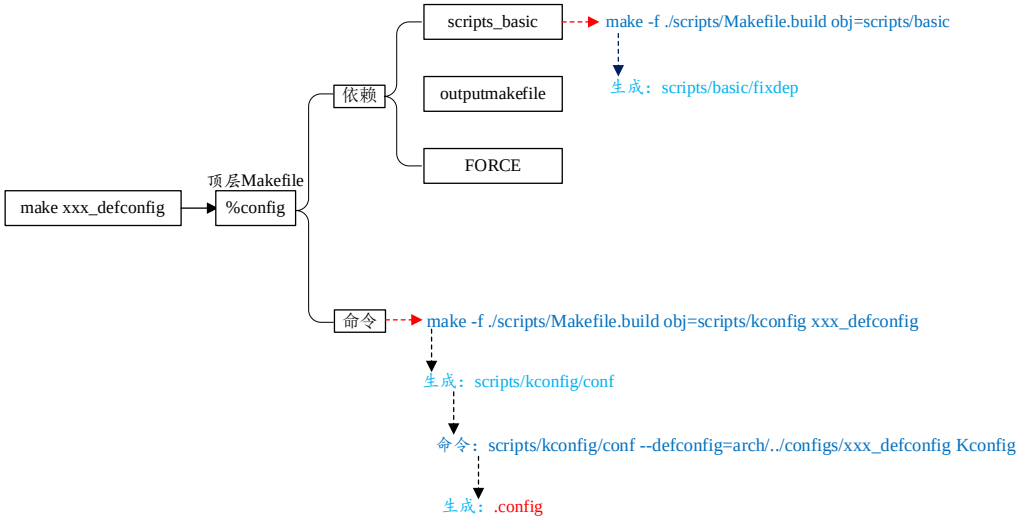


图 11.2.14.4 make xxx_defconfig 执行流程图

至此，make xxx_defconfig 就分析完了，接下来就要分析一下 u-boot.bin 是怎么生成的了。

11.2.15 make 过程

配置好 uboot 以后就可以直接 make 编译了，因为没有指明目标，所以会使用默认目标，主 Makefile 中的默认目标如下：

示例代码 11.2.15.1 顶层 Makefile 代码段

```
145 # That's our default target when none is given on the command line
146 PHONY := _all
147 _all:
```

目标_all 又依赖于 all，如下所示：

示例代码 11.2.15.2 顶层 Makefile 代码段

```
211 # If building an external module we do not care about the all: rule
212 # but instead _all depend on modules
213 PHONY += all
214 ifeq ($(KBUILD_EXTMOD),)
215 _all: all
216 else
217 _all: modules
218 endif
```

如果 KBUILD_EXTMOD 为空的话_all 依赖于 all。这里不编译模块，所以 KBUILD_EXTMOD 肯定为空，_all 的依赖就是 all。在主 Makefile 中 all 目标规则如下：

示例代码 11.2.15.3 顶层 Makefile 代码段

```
960 all:        $(ALL-y)
961 ifeq ($(CONFIG_DEPRECATED),y)
962     $(warning "You have deprecated configuration options enabled in
          your .config! Please check your configuration.")
963 ifeq ($(CONFIG_SPI),y)
964 ifneq ($(CONFIG_DM_SPI)$(CONFIG_OF_CONTROL),yy)
965     $(warning "The relevant config item with associated code will
          remove in v2019.07 release.")
966 endif
967 endif
968 endif
```

从 960 行可以看出，all 目标依赖$(ALL-y)，而在顶层 Makefile 中，ALL-y 如下：

示例代码 11.2.15.4 顶层 Makefile 代码段

```
845 # Always append ALL so that arch config.mk's can add custom ones
846 ALL-y += u-boot.srec u-boot.bin u-boot.sym System.map
             binary_size_check
847
848 ALL-$(CONFIG_ONENAND_U_BOOT) += u-boot-onenand.bin
849 ifeq ($(CONFIG_SPL_FSL_PBL),y)
850 ALL-$(CONFIG_RAMBOOT_PBL) += u-boot-with-spl-pbl.bin
```

```
851 else
852 ifneq ($(CONFIG_SECURE_BOOT), y)
853 # For Secure Boot The Image needs to be signed and Header must also
854 # be included. So The image has to be built explicitly
855 ALL-$(CONFIG_RAMBOOT_PBL) += u-boot.pbl
856 endif
857 endif
858 ALL-$(CONFIG_SPL) += spl/u-boot-spl.bin
859 ifeq ($(CONFIG_MX6)$(CONFIG_IMX_HAB), yy)
860 ALL-$(CONFIG_SPL_FRAMEWORK) += u-boot-ivt.img
861 else
862 ifeq ($(CONFIG_MX7)$(CONFIG_IMX_HAB), yy)
863 ALL-$(CONFIG_SPL_FRAMEWORK) += u-boot-ivt.img
864 else
865 ALL-$(CONFIG_SPL_FRAMEWORK) += u-boot.img
866 endif
867 endif
868 ALL-$(CONFIG_TPL) += tpl/u-boot-tpl.bin
869 ALL-$(CONFIG_OF_SEPARATE) += u-boot.dtb
870 ifeq ($(CONFIG_SPL_FRAMEWORK),y)
871 ALL-$(CONFIG_OF_SEPARATE) += u-boot-dtb.img
872 endif
873 ALL-$(CONFIG_OF_HOSTFILE) += u-boot.dtb
874 ifneq ($(CONFIG_SPL_TARGET),)
875 ALL-$(CONFIG_SPL) += $(CONFIG_SPL_TARGET:"%"=%)
876 endif
877 ALL-$(CONFIG_REMAKE_ELF) += u-boot.elf
878 ALL-$(CONFIG_EFI_APP) += u-boot-app.efi
879 ALL-$(CONFIG_EFI_STUB) += u-boot-payload.efi
880
881 ifneq ($(BUILD_ROM)$(CONFIG_BUILD_ROM),)
882 ALL-$(CONFIG_X86_RESET_VECTOR) += u-boot.rom
883 endif
884
885 # Build a combined spl + u-boot image for sunxi
886 ifeq ($(CONFIG_ARCH_SUNXI)$(CONFIG_SPL),yy)
887 ALL-y += u-boot-sunxi-with-spl.bin
888 endif
```

从示例代码代码 11.2.15.4 可以看出，ALL-y 包含 u-boot.srec、u-boot.bin、u-boot.sym、System.map、u-boot.cfg 和 binary_size_check 这几个文件。根据 uboot 的配置情况也可能包含其他的文件，比如：

```
ALL-$(CONFIG_ONENAND_U_BOOT) += u-boot-onenand.bin
```

CONFIG_ONENAND_U_BOOT 就是 uboot 中跟 ONENAND 配置有关的，如果我们使能了

ONENAND，那么在.config 配置文件中就会有“CONFIG_ONENAND_U_BOOT=y”这一句。相当于 CONFIG_ONENAND_U_BOOT 是个变量，这个变量的值为“y”，所以展开以后就是：

```
ALL-y += u-boot-onenand.bin
```

这个就是.config 里面的配置参数的含义，这些参数其实都是变量，后面跟着变量值，会在顶层 Makefile 或者其他 Makefile 中调用这些变量。

ALL-y 里面有个 u-boot.bin，这个就是我们最终需要的 uboot 二进制可执行文件，所作的所有工作就是为了它。在顶层 Makefile 中找到 u-boot.bin 目标对应的规则，如下所示：

示例代码 11.2.15.5 顶层 Makefile 代码段

```
1092    ifeq ($(CONFIG_MULTI_DTB_FIT),y)
......
1133    else ifeq ($(CONFIG_OF_SEPARATE),y)
1134    u-boot-dtb.bin: u-boot-nodtb.bin dts/dt.dtb FORCE
1135        $(call if_changed,cat)
1136
1137    u-boot.bin: u-boot-dtb.bin FORCE
1138        $(call if_changed,copy)
1139    else
1140    u-boot.bin: u-boot-nodtb.bin FORCE
1141        $(call if_changed,copy)
1142    endif
```

第 1133 行判断 CONFIG_OF_SEPARATE 是否等于 y，如果相等，那条件就成。，在 include/config/auto.conf 文件中 CONFIG_OF_SEPARAT=y，所以条件成立。

第 1134 行就是目标 u-boot-dtb.bin 的规则，目标 u-boot-dtb.bin 依赖于 u-boot-nodtb.bin，命令为$(call if_changed,copy)，第 1137 行就是目标 u-boot.bin 的规则，目标 u-boot.bin 依赖于 u-boot-dtb.bin，他们都是调用了 if_changed，if_changed 是一个函数，这个函数在 scripts/Kbuild.include 中有定义，而顶层 Makefile 中会包含 scripts/Kbuild.include 文件，这个前面已经说过了。

if_changed 在 Kbuild.include 中的定义如下：

示例代码 11.2.15.6 Kbuild.include 代码段

```
227 ###
228 # if_changed      - execute command if any prerequisite is newer
229 #                  than target, or command line has changed
230 # if_changed_dep  - as if_changed, but uses fixdep to reveal
231 #                  dependencies including used config symbols
232 # if_changed_rule - as if_changed but execute rule instead
233 # See Documentation/kbuild/makefiles.txt for more info
234 ......
256 # Execute command if command has changed or prerequisite(s) are
257 # updated.
258 if_changed = $(if $(strip $(any-prereq) $(arg-check)), \
259     @set -e;                                                \
260     $(echo-cmd) $(cmd_$(1));                                  \
261     printf '%s\n' 'cmd_$@ := $(make-cmd)' > $(dot-target).cmd)
```

第 227 行为 if_changed 的描述，根据描述，在一些先决条件比目标新的时候，或者命令行有改变的时候，if_changed 就会执行一些命令。

第 257 行就是函数 if_changed，if_changed 函数引用的变量比较多，也比较绕，我们只需要知道它可以从 u-boot-nodtb.bin 生成 u-boot.bin 就行了。

既然 u-boot.bin 依赖于 u-boot-dtb.bin，u-boot-dtb.bin 又依赖于 u-boot-nodtb.bin，那么肯定要先生成 u-boot-nodtb.bin 文件，顶层 Makefile 中相关代码如下：

示例代码 11.2.15.7 顶层 Makefile 代码段

```
1235    u-boot-nodtb.bin: u-boot FORCE
1236        $(call if_changed,objcopy)
1237        $(call DO_STATIC_RELA,$<,$@,$(CONFIG_SYS_TEXT_BASE))
1238        $(BOARD_SIZE_CHECK)
```

目标 u-boot-nodtb.bin 又依赖于 u-boot，顶层 Makefile 中 u-boot 相关规则如下：

示例代码 11.2.15.8 顶层 Makefile 代码段

```
1670    u-boot: $(u-boot-init) $(u-boot-main) u-boot.lds FORCE
1671        +$(call if_changed,u-boot__)
1672    ifeq ($(CONFIG_KALLSYMS),y)
1673        $(call cmd,smap)
1674        $(call cmd,u-boot__) common/system_map.o
1675    endif
```

目标 u-boot 依赖于 u-boot_init、u-boot-main 和 u-boot.lds，u-boot_init 和 u-boot-main 是两个变量，在顶层 Makefile 中有定义，值如下：

示例代码 11.2.15.9 顶层 Makefile 代码段

```
781 u-boot-init := $(head-y)
782 u-boot-main := $(libs-y)
```

$(head-y)跟 CPU 架构有关，我们使用的是 ARM 芯片，所以 head-y 在 arch/arm/Makefile 中被指定为：

```
head-y := arch/arm/cpu/$(CPU)/start.o
```

根据 11.2.12 小节的分析，我们知道 CPU=armv7，因此 head-y 展开以后就是：

```
head-y := arch/arm/cpu/armv7/start.o
```

因此：

```
u-boot-init= arch/arm/cpu/armv7/start.o
```

$(libs-y)在顶层 Makefile 中被定义为 uboot 所有子目录下 build-in.o 的集合，代码如下：

示例代码 11.2.15.10 顶层 Makefile 代码段

```
724 libs-y += lib/
725 libs-$(HAVE_VENDOR_COMMON_LIB) += board/$(VENDOR)/common/
726 libs-$(CONFIG_OF_EMBED) += dts/
727 libs-y += fs/
728 libs-y += net/
729 libs-y += disk/
730 libs-y += drivers/
731 libs-y += drivers/dma/
732 libs-y += drivers/gpio/
733 libs-y += drivers/i2c/
```

```
734 libs-y += drivers/net/
735 libs-y += drivers/net/phy/
736 libs-y += drivers/power/ \
......
761 libs-y += cmd/
762 libs-y += common/
763 libs-y += env/
764 libs-$(CONFIG_API) += api/
765 libs-$(CONFIG_HAS_POST) += post/
766 libs-$(CONFIG_UNIT_TEST) += test/ test/dm/
767 libs-$(CONFIG_UT_ENV) += test/env/
768 libs-$(CONFIG_UT_OPTEE) += test/optee/
769 libs-$(CONFIG_UT_OVERLAY) += test/overlay/
770
771 libs-y += $(if $(BOARDDIR),board/$(BOARDDIR)/)
772
773 libs-y := $(sort $(libs-y))
774
775 u-boot-dirs := $(patsubst %/,%,$(filter %/, $(libs-y))) tools
                  examples
776
777 u-boot-alldirs  := $(sort $(u-boot-dirs)
                       $(patsubst %/,%,$(filter %/, $(libs-))))
778
779 libs-y      := $(patsubst %/, %/built-in.o, $(libs-y))
```

从上面的代码可以看出，libs-y 都是 uboot 各子目录的集合，最后：

```
libs-y := $(patsubst %/, %/built-in.o, $(libs-y))
```

这里调用了函数 patsubst，将 libs-y 中的“/”替换为”/built-in.o”，比如“drivers/dma/”就变为了“drivers/dma/built-in.o”，相当于将 libs-y 改为所有子目录中 built-in.o 文件的集合。那么 u-boot-main 就等于所有子目录中 built-in.o 的集合。

这个规则就相当于将以 u-boot.lds 为链接脚本，将 arch/arm/cpu/armv7/start.o 和各个子目录下的 built-in.o 链接在一起生成 u-boot。

u-boot.lds 的规则如下：

示例代码 11.2.15.11 顶层 Makefile 代码段

```
1821    u-boot.lds: $(LDSCRIPT) prepare FORCE
1822        $(call if_changed_dep,cpp_lds)
```

接下来的重点就是各子目录下的 built-in.o 是怎么生成的，以 drivers/gpio/built-in.o 为例，在 drivers/gpio/目录下会有个名为.built-in.o.cmd 的文件，此文件内容如下：

示例代码 11.2.15.12 drivers/gpio/.built-in.o.cmd 代码

```
 cmd_drivers/gpio/built-in.o :=  arm-none-linux-gnueabihf-ld.bfd    -r
-o drivers/gpio/built-in.o drivers/gpio/gpio-uclass.o
drivers/gpio/stm32_gpio.o
```

从命令“cmd_drivers/gpio/built-in.o”可以看出，drivers/gpio/built-in.o 这个文件是使用 ld 命令

由文件 drivers/gpio/gpio-uclass.o 和 drivers/gpio/stm32_gpio.o 生成而来的，stm32_gpio.o 是 stm32_gpio.c 编译生成的.o 文件，这个是 ST 的 STM32 系列的 GPIO 驱动文件。这里用到了 ld 的“-r”参数，参数含义如下：

-r –relocateable: 产生可重定向的输出，比如，产生一个输出文件它可再次作为‘ld’的输入，这经常被叫做“部分链接”，当我们需要将几个小的.o 文件链接成为一个.o 文件的时候，需要使用此选项。

最终将各个子目录中的 built-in.o 文件链接在一起就形成了 u-boot，使用如下命令编译 uboot 就可以看到链接的过程：

```
make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- stm32mp157d_atk_defconfig V=1
make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- DEVICE_TREE=stm32mp157d-atk all V=1
```

编译的时候会有如图 11.2.15.1 所示内容输出：

```
  arm-none-linux-gnueabihf-ld.bfd   -pie  --gc-sections -Bstatic  --no-dynamic-linker -Ttext 0xC0100000 -o u-boot -T u-boot.lds arch/arm/cpu/armv7/start.o --start-group  arch/arm/cpu/built-in.o  arch/arm/cpu/armv7/built-in.o  arch/arm/lib/built-in.o  arch/arm/mach-stm32mp/built-in.o  board/st/common/built-in.o  board/st/stm32mp1/built-in.o  cmd/built-in.o  common/built-in.o  disk/built-in.o  drivers/built-in.o  drivers/dma/built-in.o  drivers/gpio/built-in.o  drivers/i2c/built-in.o  drivers/net/built-in.o  drivers/net/phy/built-in.o  drivers/power/built-in.o  drivers/power/battery/built-in.o  drivers/power/domain/built-in.o  drivers/power/fuel_gauge/built-in.o  drivers/power/mfd/built-in.o  drivers/power/pmic/built-in.o  drivers/power/regulator/built-in.o  drivers/serial/built-in.o  drivers/spi/built-in.o  drivers/usb/cdns3/built-in.o  drivers/usb/common/built-in.o  drivers/usb/dwc3/built-in.o  drivers/usb/emul/built-in.o  drivers/usb/eth/built-in.o  drivers/usb/gadget/built-in.o  drivers/usb/gadget/udc/built-in.o  drivers/usb/host/built-in.o  drivers/usb/musb-new/built-in.o  drivers/usb/musb/built-in.o  drivers/usb/phy/built-in.o  drivers/usb/ulpi/built-in.o  env/built-in.o  fs/built-in.o  lib/built-in.o  net/built-in.o --end-group arch/arm/lib/eabi_compat.o  arch/arm/lib/lib.a -Map u-boot.map;  true
  arm-none-linux-gnueabihf-objcopy --gap-fill=0xff  -j .text -j .secure_text -j .secure_data -j .rodata -j .hash -j .data -j .got -j .got.plt -j .u_boot_list -j .rel.dyn -j .binman_sym_table -j .text_rest -j .dtb.init.rodata -j .efi_runtime -j .efi_runtime_rel -O binary   u-boot u-boot-nodtb.bin
make -f ./scripts/Makefile.build obj=dts dtbs
```

图 11.2.15.1 编译内容输出

将其整理一下，内容如下：

```
arm-none-linux-gnueabihf-ld.bfd   -pie   --gc-sections -Bstatic   --no-dynamic-linker -Ttext
0xC0100000 -o u-boot -T u-boot.lds
arch/arm/cpu/armv7/start.o --start-group
arch/arm/cpu/built-in.o
arch/arm/cpu/armv7/built-in.o
arch/arm/lib/built-in.o
arch/arm/mach-stm32mp/built-in.o
board/st/common/built-in.o
board/st/stm32mp1/built-in.o
cmd/built-in.o
common/built-in.o
disk/built-in.o
drivers/built-in.o
drivers/dma/built-in.o
drivers/gpio/built-in.o
drivers/i2c/built-in.o
drivers/net/built-in.o
drivers/net/phy/built-in.o
drivers/power/built-in.o
```

```
drivers/power/battery/built-in.o
drivers/power/domain/built-in.o
drivers/power/fuel_gauge/built-in.o
drivers/power/mfd/built-in.o
drivers/power/pmic/built-in.o
drivers/power/regulator/built-in.o
drivers/serial/built-in.o
drivers/spi/built-in.o
drivers/usb/cdns3/built-in.o
drivers/usb/common/built-in.o
drivers/usb/dwc3/built-in.o
drivers/usb/emul/built-in.o
drivers/usb/eth/built-in.o
drivers/usb/gadget/built-in.o
drivers/usb/gadget/udc/built-in.o
drivers/usb/host/built-in.o
drivers/usb/musb-new/built-in.o
drivers/usb/musb/built-in.o
drivers/usb/phy/built-in.o
drivers/usb/ulpi/built-in.o
env/built-in.o
fs/built-in.o
lib/built-in.o
net/built-in.o --end-group
arch/arm/lib/eabi_compat.o
arch/arm/lib/lib.a
```

可以看出最终是用 arm-none-linux-gnueabihf-ld.bfd 命令将 arch/arm/cpu/armv7/start.o 和其他众多的 built_in.o 链接在一起，形成 u-boot。

目标 all 除了 u-boot.bin 以外还有其他的依赖，比如 u-boot.srec 、u-boot.sym 、System.map、u-boot.cfg 和 binary_size_check 等等，这些依赖的生成方法和 u-boot.bin 很类似，大家自行查看一下顶层 Makefile，我们就不详细的讲解了。

总结一下“make”命令的流程，如图 11.2.15.2 所示：

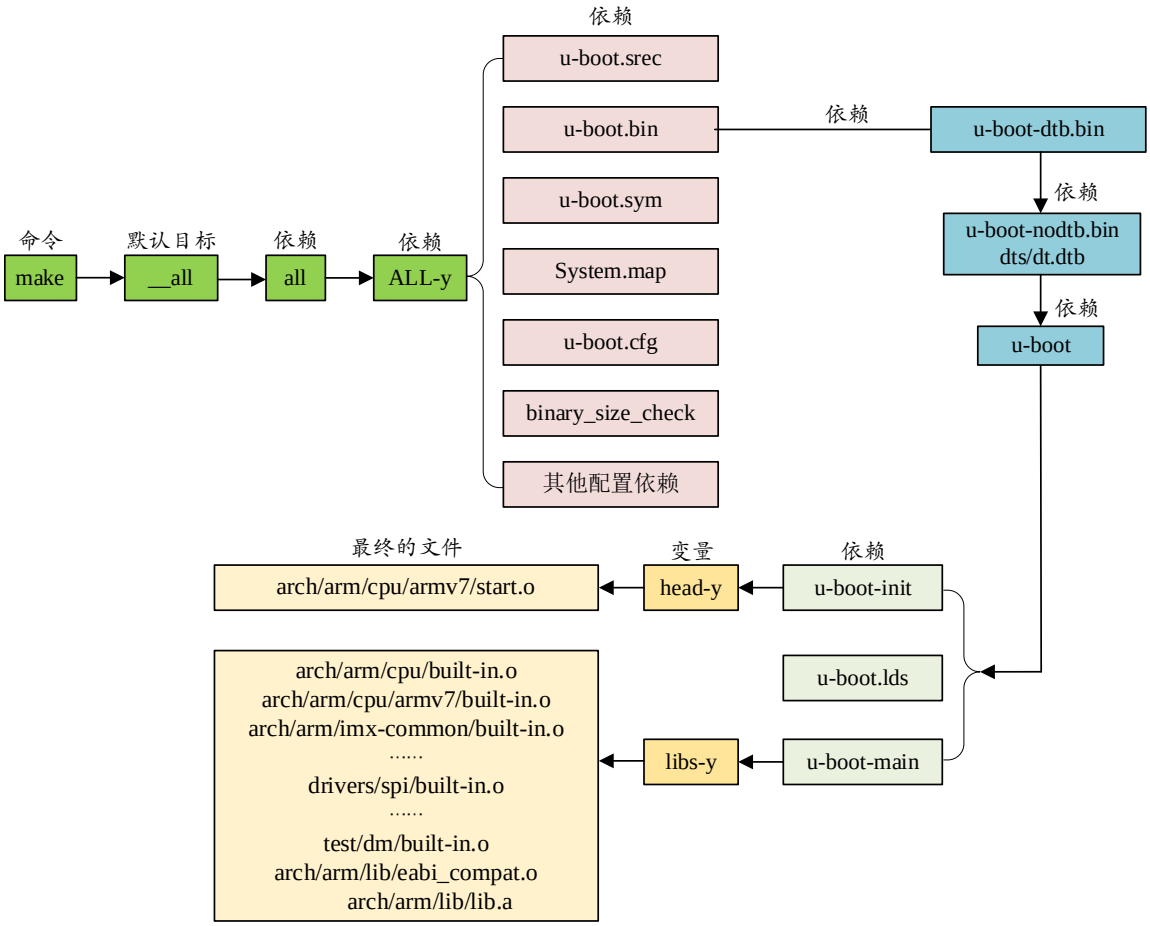


图 11.2.15.2 make 命令流程

图 11.2.15.2 就是“make”命令的执行流程，关于 uboot 的顶层 Makefile 就分析到这里，重点是“make xxx_defconfig”和“make”这两个命令的执行流程：

make xxx_defconfig：用于配置 uboot，这个命令最主要的目的就是生成.config 文件。

make：用于编译 uboot，这个命令的主要工作就是生成二进制的 u-boot.bin 文件和其他的一些与 uboot 有关的文件，比如 u-boot.stm32 等等。

关于 uboot 的顶层 Makefile 就分析到这里，有些内容我们没有详细、深入的去研究，因为我们的重点是使用 uboot，而不是 uboot 的研究者，我们要做的是缕清 uboot 的流程。至于更具体的实现，有兴趣的可以参考一下其他资料。

第十二章 U-boot 启动流程详解

上一章我们详细的分析了 uboot 的顶层 Makefile，理清了 uboot 的编译流程。本章我们来详细的分析一下 uboot 的启动流程，理清 uboot 是如何启动的。通过对 uboot 启动流程的梳理，我们就可以掌握一些外设是在哪里被初始化的，这样当我们需要修改这些外设驱动的时候就会心里有数。另外，通过分析 uboot 的启动流程可以了解 Linux 内核是如何被启动的。

12.1 链接脚本 u-boot.lds 详解

要分析 uboot 的启动流程，首先要找到“入口”，找到第一行程序在哪里。程序的链接是由链接脚本来决定的，所以通过链接脚本可以找到程序的入口。如果没有编译过 uboot 的话链接脚本为 arch/arm/cpu/u-boot.lds。但是这个不是最终使用的链接脚本，最终的链接脚本是在这个链接脚本的基础上生成的。编译一下 uboot，编译完成以后就会在 uboot 根目录下生成 u-boot.lds 文件，如图 12.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$ ls
api          CONTRIBUTING.md  fs           Makefile     tools               u-boot-dtb.bin    u-boot.sym
arch         disk             include      net          u-boot              u-boot.lds
board        doc              Kbuild       post         u-boot.bin          u-boot.map
cmd          drivers          Kconfig      README       u-boot.cfg          u-boot-nodtb.bin
common       dts              lib          scripts      u-boot.cfg.configs  u-boot.srec
config.mk    env              Licenses     System.map   uboot.code-workspace u-boot.stm32
configs      examples         MAINTAINERS  test         u-boot.dtb          u-boot.stm32.log
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/alientek_uboot$
```

图 12.1.1 链接脚本

只有编译 u-boot 以后才会在根目录下出现 u-boot.lds 文件！

打开 u-boot.lds，内容如下(有省略)：

示例代码 12.1.1 u-boot.lds 文件代码

```
1   OUTPUT_FORMAT("elf32-littlearm", "elf32-littlearm", "elf32-
                  littlearm")
2   OUTPUT_ARCH(arm)
3   ENTRY(_start)
4   SECTIONS
5   {
6    . = 0x00000000;
7    . = ALIGN(4);
8    .text :
9    {
10    *(.__image_copy_start)
11    *(.vectors)
12    arch/arm/cpu/armv7/start.o (.text*)
13   }
......
91   .dynsym _image_binary_end : { *(.dynsym) }
92   .dynbss : { *(.dynbss) }
93   .dynstr : { *(.dynstr*) }
94   .dynamic : { *(.dynamic*) }
95   .plt : { *(.plt*) }
96   .interp : { *(.interp*) }
97   .gnu.hash : { *(.gnu.hash) }
98   .gnu : { *(.gnu*) }
99   .ARM.exidx : { *(.ARM.exidx*) }
100  .gnu.linkonce.armexidx : { *(.gnu.linkonce.armexidx.*) }
101 }
```

第 3 行为代码当前入口点：_start， _start 在文件 arch/arm/lib/vectors.S 中有定义，如下示例

代码所示(省略注释)：

示例代码 12.1.2 vectors.S 文件的代码段

```
1  .globl _start
2      .section ".vectors", "ax"
3
4  _start:
5  #ifdef CONFIG_SYS_DV_NOR_BOOT_CFG
6      .word   CONFIG_SYS_DV_NOR_BOOT_CFG
7  #endif
8      ARM_VECTORS
9  #endif /* !defined(CONFIG_ENABLE_ARM_SOC_BOOT0_HOOK) */
10
11     .globl  _reset
12     .globl  _undefined_instruction
13     .globl  _software_interrupt
14     .globl  _prefetch_abort
15     .globl  _data_abort
16     .globl  _not_used
17     .globl  _irq
18     .globl  _fiq
```

图 12.1.2 _start 入口

从示例代码 12.1.2 中可以看出，_start 后面就是中断向量表，第 2 行的“.section ".vectors", "ax”可以得到，此代码存放在.vectors 段里面。

使用如下命令在 uboot 中查找“__image_copy_start”：

```
grep -nR "__image_copy_start"
```

搜索结果如图 12.1.3 所示：

```
u-boot.map:1629: *(.__image_copy_start)
u-boot.map:1630: .__image_copy_start
u-boot.map:1632:                0x00000000c0100000                __image_copy_start
匹配到二进制文件 lib/built-in.o
匹配到二进制文件 lib/fdtdec.o
```

图 12.1.3 查找结果

打开 u-boot.map，找到如图 12.1.4 所示位置：

```
1628 .text           0x00000000c0100000      0x3b8
1629 *(.__image_copy_start)
1630 .__image_copy_start
1631                 0x00000000c0100000        0x0 arch/arm/lib/built-in.o
1632                 0x00000000c0100000                __image_copy_start
1633 *(.vectors)
1634 .vectors         0x00000000c0100000      0x2e8 arch/arm/lib/built-in.o
1635                 0x00000000c0100000                _start
1636                 0x00000000c0100020                _undefined_instruction
1637                 0x00000000c0100024                _software_interrupt
1638                 0x00000000c0100028                _prefetch_abort
1639                 0x00000000c010002c                _data_abort
1640                 0x00000000c0100030                _not_used
1641                 0x00000000c0100034                _irq
1642                 0x00000000c0100038                _fiq
1643                 0x00000000c0100040                IRQ_STACK_START_IN
1644 arch/arm/cpu/armv7/start.o(.text*)
```

图 12.1.4 u-boot.map

u-boot.map 是 uboot 的映射文件，可以从此文件看到某个文件或者函数链接到了哪个地址，从图 12.1.4 的 1629 行可以看到__image_copy_start 为 0Xc0100000，而.text 的起始地址也是 0Xc0100000。

继续回到示例代码 12.1.1 中，第 11 行是 vectors 段，vectors 段保存中断向量表，从图 12.1.2 中我们知道了 vectors.S 的代码是存在 vectors 段中的。从图 12.1.4 可以看出，vectors 段的起始地址也是 0Xc0100000，说明整个 uboot 的起始地址就是 0Xc0100000。

示例代码 12.1.1 中第 12 行将 arch/arm/cpu/armv7/start.s 编译出来的代码放到中断向量表后面。

在 u-boot.lds 中有一些跟地址有关的“变量”需要我们注意一下，后面分析 u-boot 源码的时候会用到，这些变量要最终编译完成才能确定的!!! 比如我编译完成以后这些“变量”的值如表 12.1.1 所示：

变量	数值	描述
__image_copy_start	0xc0100000	uboot 拷贝的首地址
__image_copy_end	0xc01acd98	uboot 拷贝的结束地址
__rel_dyn_start	0xc01acd98	.rel.dyn 段起始地址
__rel_dyn_end	0xc01c2938	.rel.dyn 段结束地址
_image_binary_end	0xc01c2938	镜像结束地址
__bss_start	0xc01acd98	.bss 段起始地址
__bss_end	0xc01b966c	.bss 段结束地址

表 12.1.1 uboot 相关变量表

表 12.1.1 中的“变量”值可以在 u-boot.map 文件中查找，表 12.1.1 中除了__image_copy_start 以外，其他的变量值每次编译的时候可能会变化，如果修改了 uboot 代码、修改了 uboot 配置、选用不同的优化等级等等都会影响到这些值。所以，一切以实际值为准！

12.2 U-Boot 启动流程详解

12.2.1 reset 函数源码详解

从 u-boot.lds 中我们已经知道了入口点是 arch/arm/lib/vectors.S 文件中的_start，回到上面的示例代码 12.1.2。

第 4 行_start 开始的是中断向量表，其中 11~18 行就是中断向量表，和我们裸机例程里面一样。11 行跳转到 reset 函数里面，reset 函数在 arch/arm/cpu/armv7/start.S 里面，代码如下：

示例代码 12.2.1.1 start.S 代码段

```
21  /*************************************************************************
22   *
23   * Startup Code (reset vector)
24   *
25   * Do important init only if we don't start from memory!
26   * Setup memory and board specific bits prior to relocation.
27   * Relocate armboot to ram. Setup stack.
28   *
29   *************************************************************************/
30
```

```
31      .globl  reset
32      .globl  save_boot_params_ret
33      .type   save_boot_params_ret,%function
34  #ifdef CONFIG_ARMV7_LPAE
35      .global switch_to_hypervisor_ret
36  #endif
37
38  reset:
39      /* Allow the board to save important registers */
40      b   save_boot_params
```

第 31 行就是 reset 函数。

第 40 行从 reset 函数跳转到了 save_boot_params 函数，而 save_boot_params 函数同样定义在 start.S 里面，定义如下：

示例代码 12.2.1.2 start.S 代码段

```
110 /*************************************************************************
111  *
112  * void save_boot_params(u32 r0, u32 r1, u32 r2, u32 r3)
113  *  __attribute__((weak));
114  *
115  * Stack pointer is not yet initialized at this moment
116  * Don't save anything to stack even if compiled with -O0
117  *
118  *************************************************************************/
119 ENTRY(save_boot_params)
120     b   save_boot_params_ret        @ back to my caller
```

save_boot_params 函数也是只有一句跳转语句，跳转到 save_boot_params_ret 函数，save_boot_params_ret 函数代码如下：

示例代码 12.2.1.3 start.S 代码段

```
41  save_boot_params_ret:
42  #ifdef CONFIG_ARMV7_LPAE
43  /*
44   * check for Hypervisor support
45   */
46      mrc p15, 0, r0, c0, c1, 1       @ read ID_PFR1
47      and r0, r0, #CPUID_ARM_VIRT_MASK    @ mask virtualization bits
48      cmp r0, #(1 << CPUID_ARM_VIRT_SHIFT)
49      beq switch_to_hypervisor
50  switch_to_hypervisor_ret:
51  #endif
52      /*
53       * disable interrupts (FIQ and IRQ), also set the cpu to SVC32
54       * mode, except if in HYP mode already
55       */
```

```
56      mrs r0, cpsr
57      and r1, r0, #0x1f       @ mask mode bits
58      teq r1, #0x1a           @ test for HYP mode
59      bicne   r0, r0, #0x1f   @ clear all mode bits
60      orrne   r0, r0, #0x13   @ set SVC mode
61      orr r0, r0, #0xc0       @ disable FIQ and IRQ
62      msr cpsr,r0
```

第 42~51 行，CONFIG_ARMV7_LPAE 没有定义所以不会运行。

第 56 行，读取寄存器 cpsr 中的值，并保存到 r0 寄存器中。

第 57 行，将寄存器 r0 中的值与 0X1F 进行与运算，结果保存到 r1 寄存器中，目的就是提取 cpsr 的 bit0~bit4 这 5 位，这 5 位为 M4 M3 M2 M1 M0，M[4:0]这五位用来设置处理器的工作模式，如表 12.2.1.1 所示：

M[4:0]	模式
10000	User(usr)
10001	FIQ(fiq)
10010	IRQ(irq)
10011	Supervisor(svc)
10110	Monitor(mon)
10111	Abort(abt)
11010	Hyp(hyp)
11011	Undefined(und)
11111	System(sys)

表 12.2.1.1 Cortex-A7 工作模式

第 58 行，判断 r1 寄存器的值是否等于 0X1A(0b11010)，也就是判断当前处理器模式是否处于 Hyp 模式。

第 59 行，如果 r1 和 0X1A 不相等，也就是 CPU 不处于 Hyp 模式的话就将 r0 寄存器的 bit0~5 进行清零，其实就是清除模式位

第 60 行，如果处理器不处于 Hyp 模式的话就将 r0 的寄存器的值与 0x13 进行或运算，0x13=0b10011，也就是设置处理器进入 SVC 模式。

第 61 行，r0 寄存器的值再与 0xC0 进行或运算，那么 r0 寄存器此时的值就是 0xD3，cpsr 的 I 为和 F 位分别控制 IRQ 和 FIQ 这两个中断的开关，设置为 1 就关闭了 FIQ 和 IRQ！

第 62 行，将 r0 寄存器写回到 cpsr 寄存器中。完成设置 CPU 处于 SVC32 模式，并且关闭 FIQ 和 IRQ 这两个中断。

继续看下面的代码：

示例代码 12.2.1.4 start.S 代码段

```
64  /*
65   * Setup vector:
66   * (OMAP4 spl TEXT_BASE is not 32 byte aligned.
67   * Continue to use ROM code vector only in OMAP4 spl)
68   */
69  #if !(defined(CONFIG_OMAP44XX) && defined(CONFIG_SPL_BUILD))
70   /* Set V=0 in CP15 SCTLR register - for VBAR to point to vector */
71      mrc p15, 0, r0, c1, c0, 0   @ Read CP15 SCTLR Register
```

```
72      bic r0, #CR_V       @ V = 0
73      mcr p15, 0, r0, c1, c0, 0   @ Write CP15 SCTLR Register
74
75  #ifdef CONFIG_HAS_VBAR
76      /* Set vector address in CP15 VBAR register */
77      ldr r0, =_start
78      mcr p15, 0, r0, c12, c0, 0  @Set VBAR
79  #endif
80  #endif
```

第 69 行，如果没有定义 CONFIG_OMAP44XX 和 CONFIG_SPL_BUILD 的话条件成立，此处条件成立。

第 71 行读取 CP15 中 c1 寄存器的值到 r0 寄存器中，读取 SCTLR 寄存器的值。

第 72 行，CR_V 在 arch/arm/include/asm/system.h 中有如下所示定义：

```
#define CR_V (1 << 13) /* Vectors relocated to 0xffff0000  */
```

因此这一行的目的就是清除 SCTLR 寄存器中的 bit13，SCTLR 寄存器结构如图 12.2.1.1 所示：

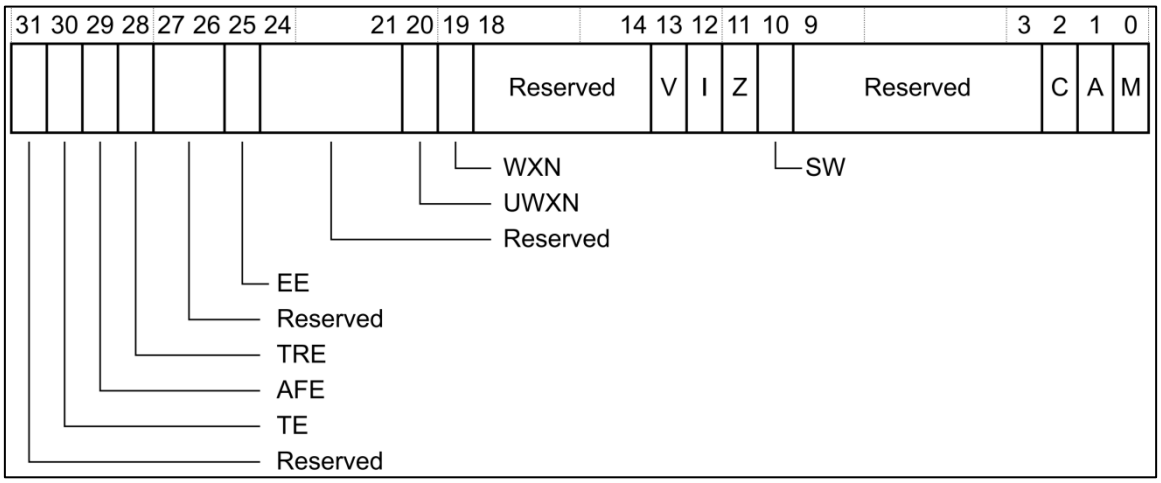


图 12.2.1.1 SCTLR 寄存器结构图

从图 12.2.1.1 可以看出，bit13 为 V 位，此位是向量表控制位，当为 0 的时候向量表基地址为 0X00000000，软件可以重定位向量表。为 1 的时候向量表基地址为 0XFFFF0000，软件不能重定位向量表。这里将 V 清零，目的就是为了接下来的向量表重定位。

第 73 行将 r0 寄存器的值重新写入到寄存器 SCTLR 中。

第 77 行设置 r0 寄存器的值为_start，_start 就是整个 uboot 的入口地址，其值为 0XC0100000，相当于 uboot 的起始地址，因此 0Xc0100000 也是向量表的起始地址。

第 78 行将 r0 寄存器的值(向量表起始地址)写入到 CP15 的 c12 寄存器中，也就是 VBAR 寄存器。因此第 71~78 行就是设置向量表重定位的。

代码继续往下执行：

示例代码 12.2.1.5 start.S 代码段

```
83  #ifndef CONFIG_SKIP_LOWLEVEL_INIT
84  #ifdef CONFIG_CPU_V7A
85      bl  cpu_init_cp15
86  #endif
```

```
87  #ifndef CONFIG_SKIP_LOWLEVEL_INIT_ONLY
88      bl  cpu_init_crit
89  #endif
90  #endif
```

第 83 行如果没有定义 CONFIG_SKIP_LOWLEVEL_INIT 的话条件成立。我们没有定义 CONFIG_SKIP_LOWLEVEL_INIT，因此条件成立，执行下面的语句。

第 84 行，.config 文件里定义 CONFIG_CPU_V7A，所以第 85 行会执行。

第 87 行，没有定义 CONFIG_SKIP_LOWLEVEL_INIT_ONLY，所以第 88 行会执行。

示例代码 12.2.1.5 中的内容比较简单，就是分别调用函数 cpu_init_cp15、cpu_init_crit 和 _main。

函数 cpu_init_cp15 用来设置 CP15 相关的内容，比如关闭 MMU 啥的，此函数同样在 start.S 文件中定义的，代码如下：

示例代码 12.2.1.6 start.S 代码段

```
139 ENTRY(cpu_init_cp15)
140     /*
141      * Invalidate L1 I/D
142      */
143     mov r0, #0          @ set up for MCR
144     mcr p15, 0, r0, c8, c7, 0   @ invalidate TLBs
145     mcr p15, 0, r0, c7, c5, 0   @ invalidate icache
146     mcr p15, 0, r0, c7, c5, 6   @ invalidate BP array
147     mcr     p15, 0, r0, c7, c10, 4  @ DSB
148     mcr     p15, 0, r0, c7, c5, 4   @ ISB
149
150     /*
151      * disable MMU stuff and caches
152      */
153     mrc p15, 0, r0, c1, c0, 0
154     bic r0, r0, #0x00002000 @ clear bits 13 (--V-)
155     bic r0, r0, #0x00000007 @ clear bits 2:0 (-CAM)
156     orr r0, r0, #0x00000002 @ set bit 1 (--A-) Align
157     orr r0, r0, #0x00000800 @ set bit 11 (Z---) BTB
158 #if CONFIG_IS_ENABLED(SYS_ICACHE_OFF)
159     bic r0, r0, #0x00001000 @ clear bit 12 (I) I-cache
160 #else
161     orr r0, r0, #0x00001000 @ set bit 12 (I) I-cache
162 #endif
163     mcr p15, 0, r0, c1, c0, 0
......
320     mov pc, r5          @ back to my caller
321 ENDPROC(cpu_init_cp15)
```

函数 cpu_init_cp15 都是一些和 CP15 有关的内容，我们不用关心，有兴趣的可以详细的看一下。

函数 cpu_init_crit 也在是定义在 start.S 文件中，函数内容如下：

示例代码 12.2.1.7 start.S 代码段

```
333 ENTRY(cpu_init_crit)
334     /*
335      * Jump to board specific initialization...
336      * The Mask ROM will have already initialized
337      * basic memory. Go here to bump up clock rate and handle
338      * wake up conditions.
339      */
340     b   lowlevel_init       @ go setup pll,mux,memory
341 ENDPROC(cpu_init_crit)
```

可以看出函数 cpu_init_crit 内部仅仅是调用了函数 lowlevel_init，接下来就是详细的分析一下 lowlevel_init 和_main 这两个函数。

12.2.2 lowlevel_init 函数详解

函数 lowlevel_init 在文件 arch/arm/cpu/armv7/lowlevel_init.S 中定义，内容如下：

示例代码 12.2.2.1 lowlevel_init.S 代码段

```
13  #include <asm-offsets.h>
14  #include <config.h>
15  #include <linux/linkage.h>
16
17  .pushsection .text.s_init, "ax"
18  WEAK(s_init)
19      bx  lr
20  ENDPROC(s_init)
21  .popsection
22
23  .pushsection .text.lowlevel_init, "ax"
24  WEAK(lowlevel_init)
25      /*
26       * Setup a temporary stack. Global data is not available yet.
27       */
28  #if defined(CONFIG_SPL_BUILD) && defined(CONFIG_SPL_STACK)
29      ldr sp, =CONFIG_SPL_STACK
30  #else
31      ldr sp, =CONFIG_SYS_INIT_SP_ADDR
32  #endif
33      bic sp, sp, #7 /* 8-byte alignment for ABI compliance */
34  #ifdef CONFIG_SPL_DM
35      mov r9, #0
36  #else
37      /*
38       * Set up global data for boards that still need it. This will
```

```
39      * be removed soon.
40      */
41  #ifdef CONFIG_SPL_BUILD
42      ldr r9, =gdata
43  #else
44      sub sp, sp, #GD_SIZE
45      bic sp, sp, #7
46      mov r9, sp
47  #endif
48  #endif
49      /*
50       * Save the old lr(passed in ip) and the current lr to stack
51       */
52      push    {ip, lr}
53
54      /*
55       * Call the very early init function. This should do only the
56       * absolute bare minimum to get started. It should not:
57       *
58       * - set up DRAM
59       * - use global_data
60       * - clear BSS
61       * - try to start a console
62       *
63       * For boards with SPL this should be empty since SPL can do all
64       * of this init in the SPL board_init_f() function which is
65       * called immediately after this.
66       */
67      bl  s_init
68      pop {ip, pc}
69  ENDPROC(lowlevel_init)
```

第 31 行设置 sp 指向 CONFIG_SYS_INIT_SP_ADDR，CONFIG_SYS_INIT_SP_ADDR 在 include/configs/stm32mp1.h 文件中，在 stm32mp1.h 中有如下所示定义：

示例代码 12.2.2.2 stm32mp1.h 代码段

```
23  #define CONFIG_SYS_SDRAM_BASE          STM32_DDR_BASE
24  #define CONFIG_SYS_INIT_SP_ADDR        CONFIG_SYS_TEXT_BASE
```

CONFIG_SYS_TEXT_BASE 的定义在我们.config 文件中定义，如下图所示：

```
154  # CONFIG_TARGET_DURIAN is not set
155  CONFIG_SYS_TEXT_BASE=0xC0100000
156  CONFIG_SYS_MALLOC_F_LEN=0x3000
157  CONFIG_ENV_SIZE=0x2000
158  CONFIG_ENV_OFFSET=0x280000
159  CONFIG_ERR_PTR_OFFSET=0x0
160  CONFIG_NR_DRAM_BANKS=1
161  # CONFIG_SPL is not set
162  CONFIG_SYS_MALLOC_LEN=0x2000000
163  CONFIG_STM32MP15x=y
```

图 12.2.2.1 CONFIG_SYS_TEXT_BASE 定义

从图 12.2.2.1 中可得知 CONFIG_SYS_INIT_SP_ADDR 的值为 0XC0100000。此时 sp 指向 0XC0100000。

继续回到文件 lowlevel_init.S，第 33 行对 sp 指针做 8 字节对齐处理！

第 44 行，sp 指针减去 GD_SIZE，GD_SIZE 的定义在 include/generated/generic-asm-offsets.h 文件中，如示例代码 12.2.2.3 所示：

示例代码 12.2.2.3 generic-asm-offsets.h 代码段

```
9   #define GENERATED_GBL_DATA_SIZE     240
10  #define GENERATED_BD_INFO_SIZE      80
11  #define GD_SIZE                     240
12  #define GD_BD                       0
13  #define GD_MALLOC_BASE              164
14  #define GD_RELOCADDR                64
15  #define GD_RELOC_OFF                84
16  #define GD_START_ADDR_SP            80
17  #define GD_NEW_GD                   88
```

第 11 行，GD_SIZE 大小为 240。

继续回到文件 lowlevel_init.S，第 45 行对 sp 做 8 个字节对齐，此时 sp 的地址为 0XC0100000-240=0XC00FFDC0。

第 46 行将 sp 地址保存在 r9 寄存器中。

第 52 行将 ip 和 lr 压栈

第 67 行调用函数 s_init，此函数没啥作用。

第 88 行将第 36 行入栈的 ip 和 lr 进行出栈，并将 lr 赋给 pc。

lowlevel_init 运行完后，就返回函数 cpu_init_crit，函数 cpu_init_crit 也执行完成了，最终返回到 save_boot_params_ret，函数调用路径如图 12.2.2.2 所示：

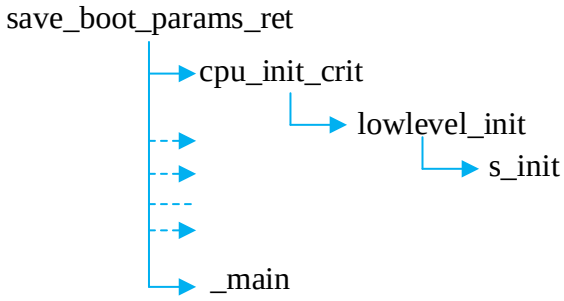


图 12.2.2.2 uboot 函数调用路径

从图 12.2.2.2 可知，接下来要执行的是 save_boot_params_ret 中的_main 函数，接下来分析_main 函数。

12.2.3 _main 函数详解

_main 函数定义在文件 arch/arm/lib/crt0.S 中，函数内容如下：

示例代码代码 12.2.3.1 crt0.S 代码段

```
91  ENTRY(_main)
92
93  /*
94   * Set up initial C runtime environment and call board_init_f(0).
95   */
96
97  #if defined(CONFIG_TPL_BUILD) &&
              defined(CONFIG_TPL_NEEDS_SEPARATE_STACK)
98      ldr r0, =(CONFIG_TPL_STACK)
99  #elif defined(CONFIG_SPL_BUILD) && defined(CONFIG_SPL_STACK)
100     ldr r0, =(CONFIG_SPL_STACK)
101 #else
102     ldr r0, =(CONFIG_SYS_INIT_SP_ADDR)
103 #endif
104     bic r0, r0, #7  /* 8-byte alignment for ABI compliance */
105     mov sp, r0
106     bl  board_init_f_alloc_reserve
107     mov sp, r0
108     /* set up gd here, outside any C code */
109     mov r9, r0
110     bl  board_init_f_init_reserve
111
112 #if defined(CONFIG_SPL_EARLY_BSS)
113     SPL_CLEAR_BSS
114 #endif
115
116     mov r0, #0
117     bl  board_init_f
118
119 #if ! defined(CONFIG_SPL_BUILD)
120
121 /*
122  * Set up intermediate environment (new sp and gd) and call
123  * relocate_code(addr_moni). Trick here is that we'll return
124  * 'here' but relocated.
125  */
126
```

```
127     ldr r0, [r9, #GD_START_ADDR_SP] /* sp = gd->start_addr_sp */
128     bic r0, r0, #7  /* 8-byte alignment for ABI compliance */
129     mov sp, r0
130     ldr r9, [r9, #GD_NEW_GD]        /* r9 <- gd->new_gd */
131
132     adr lr, here
133     ldr r0, [r9, #GD_RELOC_OFF]     /* r0 = gd->reloc_off */
134     add lr, lr, r0
135 #if defined(CONFIG_CPU_V7M)
136     orr lr, #1              /* As required by Thumb-only */
137 #endif
138     ldr r0, [r9, #GD_RELOCADDR]     /* r0 = gd->relocaddr */
139     b   relocate_code
140 here:
141 /*
142  * now relocate vectors
143  */
144
145     bl  relocate_vectors
146
147 /* Set up final (full) environment */
148
149     bl  c_runtime_cpu_setup /* we still call old routine here */
150 #endif
151 #if !defined(CONFIG_SPL_BUILD) || CONFIG_IS_ENABLED(FRAMEWORK)
152
153 #if !defined(CONFIG_SPL_EARLY_BSS)
154     SPL_CLEAR_BSS
155 #endif
156
157 # ifdef CONFIG_SPL_BUILD
158     /* Use a DRAM stack for the rest of SPL, if requested */
159     bl  spl_relocate_stack_gd
160     cmp r0, #0
161     movne   sp, r0
162     movne   r9, r0
163 # endif
164
165 #if ! defined(CONFIG_SPL_BUILD)
166     bl coloured_LED_init
167     bl red_led_on
168 #endif
169     /* call board_init_r(gd_t *id, ulong dest_addr) */
```

```
170     mov     r0, r9                  /* gd_t */
171     ldr r1, [r9, #GD_RELOCADDR] /* dest_addr */
172     /* call board_init_r */
173 #if CONFIG_IS_ENABLED(SYS_THUMB_BUILD)
174     ldr lr, =board_init_r   /* this is auto-relocated! */
175     bx  lr
176 #else
177     ldr pc, =board_init_r   /* this is auto-relocated! */
178 #endif
179     /* we should not return here. */
180 #endif
181
182 ENDPROC(_main)
183
```

第 102 行，设置 sp 指针为 CONFIG_SYS_INIT_SP_ADDR，也就是 sp 指向 0XC0100000。

第 104 行，sp 做 8 字节对齐。

第 105 行，读取 sp 到寄存器 r0 里面，此时 r0=0XC0100000。

第 106 行，调用函数 board_init_f_alloc_reserve，此函数有一个参数，参数为 r0 中的值，也就是 0XC0100000，此函数定义在文件 common/init/board_init.c 中，内容如下：

示例代码 12.2.3.2 board_init.c 代码段

```
62  ulong board_init_f_alloc_reserve(ulong top)
63  {
64      /* Reserve early malloc arena */
65  #if CONFIG_VAL(SYS_MALLOC_F_LEN)
66      top -= CONFIG_VAL(SYS_MALLOC_F_LEN);
67  #endif
68      /* LAST : reserve GD (rounded up to a multiple of 16 bytes) */
69      top = rounddown(top-sizeof(struct global_data), 16);
70
71      return top;
72  }
```

函数 board_init_f_alloc_reserve 主要是留出早期的 malloc 内存区域和 gd 内存区域，其中 CONFIG_SYS_MALLOC_F_LEN=0X3000(在文件 include/generated/autoconf.h 中定义)，另外 sizeof(struct global_data)=240，也就是 GD_SIZE 值。内存结构如图 12.2.3.1 所示：

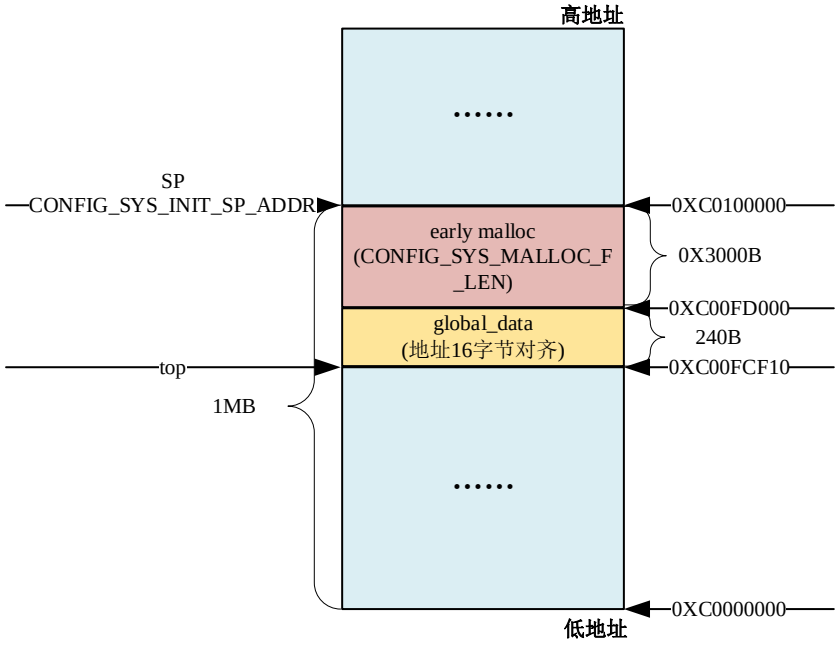


图 12.2.3.1 内存结构图

函数 board_init_f_alloc_reserve 是有返回值的，返回值为新的 top 值，此时 top=0XC00FCF10（地址刚好是 16 字节对齐）。

继续回到示例代码 12.2.3.1 中，第 107 行，将 r0 写入到 sp 里面，r0 保存着函数 board_init_f_alloc_reserve 的返回值，所以这一句也就是设置 sp=0XC00FCF10。

第 109 行，将 r0 寄存器的值写到寄存器 r9 里面，因为 r9 寄存器存放着全局变量 gd 的地址，在文件 arch/arm/include/asm/global_data.h 中有如图 12.2.3.2 所示宏定义：

```
109 #ifdef CONFIG_ARM64
110 #define DECLARE_GLOBAL_DATA_PTR     register volatile gd_t *gd asm ("x18")
111 #else
112 #define DECLARE_GLOBAL_DATA_PTR     register volatile gd_t *gd asm ("r9")
113 #endif
```

图 12.2.3.2 DECLARE_GLOBAL_DATA_PTR 宏定义

从图 12.2.3.2 可以看出，uboot 中定义了一个指向 gd_t 结构体类型的指针：gd，gd 存放在寄存器 r9 里面的，因此 gd 是个全局变量。gd_t 是个结构体，在 include/asm-generic/global_data.h 里面有定义，gd_定义如下：

示例代码 12.2.3.3 global_data 结构体

```
27 typedef struct global_data {
28     bd_t *bd;
29     unsigned long flags;
30     unsigned int baudrate;
31     unsigned long cpu_clk;      /* CPU clock in Hz!     */
32     unsigned long bus_clk;
33     /* We cannot bracket this with CONFIG_PCI due to mpc5xxx */
34     unsigned long pci_clk;
35     unsigned long mem_clk;
36 #if defined(CONFIG_LCD) || defined(CONFIG_VIDEO) ||
                    defined(CONFIG_DM_VIDEO)
```

```
37      unsigned long fb_base;      /* Base address of framebuffer mem */
38  #endif
......
137 #if defined(CONFIG_TRANSLATION_OFFSET)
138     fdt_addr_t translation_offset; /* optional translation offset */
139 #endif
140 #if CONFIG_IS_ENABLED(WDT)
141     struct udevice *watchdog_dev;
142 #endif
143 } gd_t;
```

因此这一行代码就是设置 gd 所指向的位置，也就是 gd 指向 0XC00FCF10。

继续回到示例代码 12.2.3.1 中，第 110 行调用函数 board_init_f_init_reserve，此函数在文件 common/init/board_init.c 中有定义，函数内容如下：

示例代码 12.2.3.4 board_init_f_init_reserve 函数

```
116 void board_init_f_init_reserve(ulong base)
117 {
118     struct global_data *gd_ptr;
119
120     /*
121      * clear GD entirely and set it up.
122      * Use gd_ptr, as gd may not be properly set yet.
123      */
124
125     gd_ptr = (struct global_data *)base;
126     /* zero the area */
127     memset(gd_ptr, '\0', sizeof(*gd));
128     /* set GD unless architecture did it already */
129 #if !defined(CONFIG_ARM)
130     arch_setup_gd(gd_ptr);
131 #endif
132     /* next alloc will be higher by one GD plus 16-byte alignment */
133     base += roundup(sizeof(struct global_data), 16);
134
135     /*
136      * record early malloc arena start.
137      * Use gd as it is now properly set for all architectures.
138      */
139
140 #if CONFIG_VAL(SYS_MALLOC_F_LEN)
141     /* go down one 'early malloc arena' */
142     gd->malloc_base = base;
143     /* next alloc will be higher by one 'early malloc arena' size */
144     base += CONFIG_VAL(SYS_MALLOC_F_LEN);
```

```
145 #endif
146
147     if (CONFIG_IS_ENABLED(SYS_REPORT_STACK_F_USAGE))
148         board_init_f_init_stack_protection();
149 }
```

可以看出，此函数用于初始化 gd，其实就是清零处理。另外，此函数还设置了 gd->malloc_base 为 gd 基地址+gd 大小=0XC00FCF10+240=0XC00FD000，这个也就是 early malloc 的起始地址。

继续回到示例代码 12.2.3.1 中，第 116 行设置 R0 为 0。

第 117 行，调用 board_init_f 函数，此函数定义在文件 common/board_f.c 中！主要用来初始化平台相关的 API，定时器，完成代码拷贝等等，此函数我们后面在详细的分析。

第 127 行，重新设置环境(sp 和 gd)，获取 gd->start_addr_sp 的值赋给 sp，在函数 board_init_f 中会初始化 gd 的所有成员变量，最终 gd->start_addr_sp=0XF3AEDD20，所以这里相当于设置 sp=gd->start_addr_sp=0XF3AEDD20。GD_START_ADDR_SP=80，参考示例代码 12.2.2.3。

第 128 行，sp 做 8 字节对齐。

第 130 行，获取 gd->new_gd 的地址赋给 r9，此时 r9 存放的是老的 gd，这里通过获取 gd->new_gd 的地址来计算出新的 gd 的位置。GD_BD=0，参考示例代码 12.2.2.3。

第 132 行，设置 lr 寄存器为 here，这样后面执行其他函数返回的时候就返回到了第 122 行的 here 位置处。

第 133 行，读取 gd->reloc_off 的值复制给 r0 寄存器，GD_RELOC_OFF=84，参考示例代码 12.2.2.3。

第 134 行，lr 寄存器的值加上 r0 寄存器的值，重新赋值给 lr 寄存器。因为接下来要重定位代码，也就是把代码拷贝到新的地方去(现在的 uboot 存放的起始地址为 0XC0100000，下面要将 uboot 拷贝到 DDR 最后面的地址空间出，将 0XC0100000 开始的内存空出来)，其中就包括 here，因此 lr 中的 here 要使用重定位后的位置。

第 138 行，读取 gd->relocaddr 的值赋给 r0 寄存器，此时 r0 寄存器就保存着 uboot 要拷贝的目的地址，为 0XF5C46000。GD_RELOCADDR=40，参考示例代码 12.2.2.3。

第 139 行，调用函数 relocate_code，也就是代码重定位函数，此函数负责将 uboot 拷贝到新的地方去，此函数定义在文件 arch/arm/lib/relocate.S 中稍后会详细分析此函数。

继续回到示例代码 12.2.3.1 第 145 行，调用函数 relocate_vectors，对中断向量表做重定位，此函数定义在文件 arch/arm/lib/relocate.S 中，稍后会详细分析此函数。

继续回到示例代码 12.2.3.1 第 149 行，调用函数 c_runtime_cpu_setup，此函数定义在文件 arch/arm/cpu/armv7/start.S 中，函数内容如下：

示例代码 12.2.3.5 start.S 代码段

```
96  ENTRY(c_runtime_cpu_setup)
97  /*
98   * If I-cache is enabled invalidate it
99   */
100 #if !CONFIG_IS_ENABLED(SYS_ICACHE_OFF)
101     mcr p15, 0, r0, c7, c5, 0   @ invalidate icache
102     mcr     p15, 0, r0, c7, c10, 4  @ DSB
103     mcr     p15, 0, r0, c7, c5, 4   @ ISB
104 #endif
```

```
105
106     bx  lr
107
108 ENDPROC(c_runtime_cpu_setup)
```

第 170 行，设置函数 board_init_r 的两个参数，函数 board_init_r 声明如下：

```
board_init_r(gd_t *id, ulong dest_addr)
```

第一个参数是 gd，因此读取 r9 保存到 r0 里面。

第 171 行，设置函数 board_init_r 的第二个参数是目的地址，因此 r1= gd->relocaddr。

第 177 行、调用函数 board_init_r，此函数定义在文件 common/board_r.c 中，稍后会详细的分析此函数。

这个就是_main 函数的运行流程，在_main 函数里面调用了 board_init_f、relocate_code、relocate_vectors 和 board_init_r 这 4 个函数，接下来依次看一下这 4 个函数都是干啥的。

12.2.4 board_init_f 函数详解

_main 中会调用 board_init_f 函数，board_init_f 函数主要有两个工作：

①、初始化一系列外设，比如串口、定时器，或者打印一些消息等。

②、初始化 gd 的各个成员变量，uboot 会将自己重定位到 DRAM 最后面的地址区域，也就是将自己拷贝到 DRAM 最后面的内存区域中。这么做的目的是给 Linux 腾出空间，防止 Linux kernel 覆盖掉 uboot，将 DRAM 前面的区域完整的空出来。在拷贝之前肯定要给 uboot 各部分分配好内存位置和大小，比如 gd 应该存放到哪个位置，malloc 内存池应该存放到哪个位置等等。这些信息都保存在 gd 的成员变量中，因此要对 gd 的这些成员变量做初始化。最终形成一个完整的内存“分配图”，在后面重定位 uboot 的时候就会用到这个内存“分配图”。

board_init_f 函数定义在文件 common/board_f.c 中定义，代码如下：

示例代码 12.2.4.1 board_f.c 代码段

```
1021    void board_init_f(ulong boot_flags)
1022    {
1023        gd->flags = boot_flags;
1024        gd->have_console = 0;
1025
1026        if (initcall_run_list(init_sequence_f))
1027            hang();
1028
1029    #if !defined(CONFIG_ARM) && !defined(CONFIG_SANDBOX) && \
1030        !defined(CONFIG_EFI_APP) && !CONFIG_IS_ENABLED(X86_64) && \
1031        !defined(CONFIG_ARC)
1032        /* NOTREACHED - jump_to_copy() does not return */
1033        hang();
1034    #endif
1035    }
```

第 1023 行，初始化 gd->flags=boot_flags。

第 1024 行，设置 gd->have_console=0。

重点在第 1026 行！通过函数 initcall_run_list 来运行初始化序列 init_sequence_f 里面的一些列函数，init_sequence_f 里面包含了一系列的初始化函数，init_sequence_f 也定义在文件

common/board_f.c 中，由于 init_sequence_f 的内容比较长，里面有大量的条件编译代码，这里为了缩小篇幅，将条件编译部分删除掉了，去掉条件编译以后的 init_sequence_f 定义如下：

示例代码 12.2.4.2 board_f.c 代码段

```
/*****************去掉条件编译语句后的 init_sequence_f***************/
1   static const init_fnc_t init_sequence_f[] = {
2       setup_mon_len,
3       fdtdec_setup,
4       trace_early_init,
5
6       initf_malloc,
7       log_init,
8       initf_bootstage,    /* uses its own timer, so does not need DM */
9       bloblist_init,
10      setup_spl_handoff,
11      initf_console_record,
12      arch_fsp_init,
13      arch_cpu_init,      /* basic arch cpu dependent setup */
14      mach_cpu_init,      /* SoC/machine dependent CPU setup */
15      initf_dm,
16      arch_cpu_init_dm,
17      board_early_init_f,
18      get_clocks,     /* get CPU and bus clocks (etc.) */
19      timer_init,     /* initialize timer */
20      board_postclk_init,
21      env_init,       /* initialize environment */
22      init_baud_rate,     /* initialze baudrate settings */
23      serial_init,        /* serial communications setup */
24      console_init_f,     /* stage 1 init of console */
25      display_options,    /* say that we are here */
26      display_text_info,  /* show debugging info if required */
27      checkcpu,
28      print_resetinfo,
29      print_cpuinfo,      /* display cpu info (and speed) */
30      embedded_dtb_select,
31      show_board_info,
32      INIT_FUNC_WATCHDOG_INIT
33      misc_init_f,
34      INIT_FUNC_WATCHDOG_RESET
35      init_func_i2c,
36      init_func_vid,
37      announce_dram_init,
38      dram_init,      /* configure available RAM banks */
39      post_init_f,
```

```
40      INIT_FUNC_WATCHDOG_RESET
41      testdram,
42      INIT_FUNC_WATCHDOG_RESET
43      init_post,
44      INIT_FUNC_WATCHDOG_RESET
45      /*
46       * Now that we have DRAM mapped and working, we can
47       * relocate the code and continue running from DRAM.
48       *
49       * Reserve memory at end of RAM for (top down in that order):
50       *  - area that won't get touched by U-Boot and Linux (optional)
51       *  - kernel log buffer
52       *  - protected RAM
53       *  - LCD framebuffer
54       *  - monitor code
55       *  - board info struct
56       */
57      setup_dest_addr,
58      reserve_pram,
59      reserve_round_4k,
60      reserve_mmu,
61      reserve_video,
62      reserve_trace,
63      reserve_uboot,
64      reserve_malloc,
65      reserve_board,
66      setup_machine,
67      reserve_global_data,
68      reserve_fdt,
69      reserve_bootstage,
70      reserve_bloblist,
71      reserve_arch,
72      reserve_stacks,
73      dram_init_banksize,
74      show_dram_config,
75      setup_board_part1,
76      INIT_FUNC_WATCHDOG_RESET
77      setup_board_part2,
78      display_new_sp,
79      fix_fdt,
80      INIT_FUNC_WATCHDOG_RESET
81      reloc_fdt,
82      reloc_bootstage,
```

```
83      reloc_bloblist,
84      setup_reloc,
85      copy_uboot_to_ram,
86      do_elf_reloc_fixups,
87      clear_bss,
88      clear_bss,
89      jump_to_copy,
90      NULL,
91  };
```

接下来分析以上函数执行完以后的结果：

第 2 行，setup_mon_len 函数设置 gd 的 mon_len 成员变量，此处为__bss_end -_start，也就是整个代码的长度。0XC01B966C-0XC0100000=0XB966C，这个就是代码长度

第 3 行，fdtdec_setup 函数设置 gd 的 fdt_dlob 指针变量，fdt_blob 保存着设备树(.dtb)文件地址。此处为_image_binary_end，也就是我们的设备地址为 0XC01C2938。

第 4 行，CONFIG_TRACE_EARLY 宏没有定义不会运行此函数。

第 6 行，initf_malloc 函数初始化 gd 中跟 malloc 有关的成员变量，比如 malloc_limit，此函数会设置 gd->malloc_limit = CONFIG_SYS_MALLOC_F_LEN=0X3000。malloc_limit 表示 malloc 内存池大小。

第 7 行，log_init 函数将 struct log_driver 结构体加入到 gd->log_head 的循环链表中，并初始化 gd->default_log_level。

第 8 行，initf_bootstage 函数为 gd->bootstage 分配空间，并初始化 gd->bootstage。

第 9 行，CONFIG_BLOBLIST 宏没有定义，此函数不执行。

第 10 行，setup_spl_handoff 函数和 SPL 相关，STM32MP1 是使用 TF-A 引导 U-BOOT，此函数啥也没有做。

第 11 行，initf_console_record，如果定义了宏 CONFIG_CONSOLE_RECORD 和宏 CONFIG_SYS_MALLOC_F_LEN，此函数就会调用函数 console_record_init，但是 STM32MP1 的 uboot 没有定义宏 CONFIG_CONSOLE_RECORD，所以此函数直接返回 0。

第 12 行，如果定义 CONFIG_HAVE_FSP 宏，就会调用 arch_fsp_init 函数。STM32MP1 没有定义此宏。

第 13 行，arch_cpu_init 函数，初始化架构相关的内容，CPU 级别的操作。

第 14 行，mach_cpu_init 函数，初始化 SOC 相关的内容，CPU 级别的操作。

第 15 行，initf_dm 函数，驱动模型有关的初始化操作，如果定义了 CONFIG_DM，则调用 dm_init_and_scan 初始化并扫描系统所有的 device。如果定义了 CONFIG_TIMER_EARLY，调用 dm_timer_init 初始化 driver model 所需的 timer 。STM32MP1 定义了 CONFIG_DM，会扫描所有的 device。

第 16 行，arch_cpu_init_dm 函数为空函数。

第 17 行，如果定义 CONFIG_BOARD_EARLY_INIT_F，则调用 board_early_init_f 接口，执行板级的 early 初始化。这边 STM32MP1 没有定义。

第 18 行，用于获取一些时钟值。

第 19 行，timer_init，初始化定时器。主要用来初始化 gd->arch.timer_rate_hz，设置定时器频率。

第 20 行，CONFIG_BOARD_POSTCLK_INIT 宏没有定义，所以函数不会调用。

第 21 行，env_init 函数是和环境变量有关的，设置 gd 的成员变量 env_addr，也就是环境变

量的保存地址。

第 22 行，init_baud_rate 函数用于初始化波特率，根据环境变量 baudrate 来初始化 gd->baudrate。

第 23 行，serial_init，初始化串口，设置 gd 的 flags 成员变量为 GD_FLG_SERIAL_READY，开启当前选中的串口。

第 24 行，console_init_f，设置 gd->have_console 为 1，表示有个控制台，此函数也将前面暂存在缓冲区中的数据通过控制台打印出来。

第 25 行、display_options，通过串口输出一些信息，比如 uboot 版本号，编译时间等，如图 12.2.4.1 所示：

```
U-Boot 2020.01-stm32mp-r1 (Oct 23 2021 - 11:47:47 +0800)
```

图 12.2.4.1 串口信息输出

第 26 行，display_text_info，打印一些文本信息，如果开启 UBOOT 的 DEBUG 功能的话就会输出 text_base、bss_start、bss_end。开启 DEBUG 方法很简单，在对应的板子的配置头文件里面定义 DEBUG 即可，比如在 stm32mp1.h 文件中添加下面一行：

```
#define DEBUG          //使能 DEBUG
```

使能以后重新编译 uboot，然后使用新的 uboot 启动，这个时候会输出大量的 debug 信息，如图 12.2.4.2 所示：

```
U-Boot 2020.01-stm32mp-r1 (Oct 23 2021 - 12:21:33 +0800)

initcall: c0117825
U-Boot code: C0100000 -> C01C66E8  BSS: -> C01D57B4
initcall: c0117939
initcall: c01020c9
```

图 12.2.4.2 文本信息

图 12.2.4.2 中“U-Boot code: C0100000 -> C01C66E8 BSS: -> C01D57B4”就是输出的一条 debug 信息，包含了 uboot 代码起始地址，BSS 段起始地址。还有其他很多 debug 信息，不建议大家打开 DEBUG，因为 debug 信息太多了，严重影响了我们正常开发！

第 27 行，PPC、X86、SH 架构的芯片会执行，ARM 架构不会执行。

第 28 行，如果使能了 CONFIG_SYSRESET 宏的话，会打印一些系统复位信息。

第 29 行，print_cpuinfo 函数用于打印 CPU 信息，结果如图 12.2.4.3 所示：

```
U-Boot 2020.01-stm32mp-r1 (Oct 23 2021 - 12:31:33 +0800)

CPU: STM32MP157DAA Rev.Z
Model: STMicroelectronics STM32MP157D eval daughter
```

图 12.2.4.3 CPU 信息

第 30 行，CONFIG_DTB_RESELECT 宏没有定义所以此函数不执行。

第 31 行，show_board_info 函数用于打印板子信息，如果在 uboot 中使用了设备树，那么此函数会先从设备树里面获取到 model 属性信息并且打印出出来。然后会调用 checkboard 函数获取板子信息并打印出来，结果如图 12.2.4.4 所示：

```
CPU: STM32MP157DAA Rev.Z
Model: STMicroelectronics STM32MP157D eval daughter
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)
DRAM:  1 GiB
Clocks:
```

图 12.2.4.4 板子信息

第 32 行，INIT_FUNC_WATCHDOG_INIT，初始化看门狗，对于 STM32MP1 来说是空函数。

第 33 行，CONFIG_MISC_INIT_F 宏没有定义，所以此函数不会执行。

第 34 行，INIT_FUNC_WATCHDOG_RESET，复位看门狗，对于 STM32MP1 来说是空函数。

第 35 行，CONFIG_SYS_I2C 宏没有定义，所以此函数不会执行。

第 36 行，init_func_vid 是一个空函数，返回值为 0。

第 37 行，announce_dram_init，此函数很简单，就是输出字符串“DRAM:”

第 38 行，dram_init，并非真正的初始化 DDR，只是设置 gd->ram_size 的值，对于正点原子 STM32MP1 开发板核来说，DDR 大小为 1G，所以 gd->ram_size 的值为 0X40000000。

第 39 行，CONFIG_POST 宏没有定义，此函数不执行。

第 41 行，testdram，测试 DRAM，空函数。

第 43 行，CONFIG_POST 宏没有定义，此函数不执行。

第 57 行，setup_dest_addr 函数，设置目的地址，设置 gd->ram_base 、gd->ram_size、gd->ram_top、gd->relocaddr 这三个的值。接下来我们会遇到很多跟数值有关的设置，如果直接看代码分析的话就太费时间了，我可以修改 uboot 代码，直接将这些值通过串口打印出来，比如这里我们打开文件 common/board_f.c，找到 setup_dest_addr 函数，在函数输入如图 12.2.4.5 所示内容：

```
351     debug("Ram top: %08lX\n", (ulong)gd->ram_top);
352
353     printf("\ngd->ram_base = %#X\r\n", gd->ram_base);
354     printf("gd->ram_size = %#X\r\n", gd->ram_size);
355     printf("gd->ram_top = %#X\r\n", gd->ram_top);
356     printf("gd->relocaddr = %#X\r\n", gd->relocaddr);
357 #if defined(CONFIG_MP) && (defined(CONFIG_MPC86xx) || defined(CONFIG_E500))
```

图 12.2.4.5 添加 printf 函数打印成员变量值

设置好以后重新编译 uboot，然后烧写到 SD 卡中，选择 SD 卡启动，重启开发板，打开 MobaXterm，uboot 会输出如图 12.2.5.6 所示信息：

```
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)
DRAM:
gd->ram_base = 0XC0000000
gd->ram_size = 0X40000000
gd->ram_top = 0XF6000000
gd->relocaddr = 0XF6000000
1 GiB
```

图 12.2.4.6 信息输出

从图 12.2.4.6 可以看出：

```
gd->ram_base = 0XC0000000      //DDR 起始地址 0XC0000000
gd->ram_size  = 0X40000000     //DDR 大小为 0X40000000=1G
gd->ram_top   = 0XF6000000     //DDR 最高地址为 0XF6000000
gd->relocaddr = 0XF6000000     //重定位后最高地址为 0XF6000000
```

大家可能会疑问，为啥 ram_top (也就是 DDR)最高地址是 0XF6000000，不应该是 0XC0000000+0X40000000 -1=0XFFFFFFFF。这是因为 0XF6000000~0XFFFFFFFF 这段内存分配给了 GPU 和 OPTEE，打开设备树 arch/arm/dts/stm32mp157d-atk.dts 文件，找到如下内容：

```
28      reserved-memory {
29          gpu_reserved: gpu@f6000000 {
30              reg = <0xf6000000 0x8000000>;
31              no-map;
32          };
33
34          optee_memory: optee@fe000000 {
35              reg = <0xfe000000 0x02000000>;
36              no-map;
37          };
38      };
```

图 12.2.4.7 内存保留配置

结合上面的图 12.2.4.6 和 7 得知，我们的 rom 最高为 0XF600000，剩余的 0XA000000 预留给 GPU 和 optee 使用。其中 0XF6000000~0XFDFFFFFF 分配给了 GPU，0XFE000000~0XFFFFFFFF 分配给了 OPTEE。

第 59 行，reserve_round_4k 函数用于对 gd->relocaddr 做 4KB 对齐，因为 gd->relocaddr=0XF6000000，已经是 4K 对齐了，所以调整后不变。

第 60 行，reserve_mmu，留出 MMU 的 TLB 表的位置，分配 MMU 的 TLB 表内存以后会对 gd->relocaddr 做 64K 字节对齐。完成以后 gd->arch.tlb_size、gd->arch.tlb_addr 和 gd->relocaddr 如图 12.2.4.8 所示：

```
DRAM:
gd->arch.tlb_size = 0X4000
gd->arch.tlb_addr = 0XF5FF0000
gd->relocaddr = 0XF5FF0000
1 GiB
```

图 12.2.4.8 信息输出

从图 12.2.4.8 可以看出：

```
gd->arch.tlb_size   = 0X4000        //MMU 的 TLB 表大小
gd->arch.tlb_addr   = 0XF5FF0000    //MMU 的 TLB 表起始地址，64KB 对齐以后
gd->relocaddr       = 0XF5FF0000    /relocaddr 地址
```

第 61 行，reserve_video 函数，留出显示的内存，分配显示内存后 gd->relocaddr 如图 12.2.4.9 所示：

```
DRAM:
gd->video_top = 0XF5FF0000
gd->video_bottom  = 0XF5D00000
gd->relocaddr = 0XF5D00000
1 GiB
```

图 12.2.4.9 信息输出

从图 12.2.4.9 可以知道，留出显示内存以后，gd->relocaddr=0XF5D00000，显示相关内存大小为 0X2F0000。

第 62 行，reserve_trace 函数，留出跟踪调试的内存，STM32MP1 没有用到最终功能，因此无需留出对应的内存区域。

第 63 行，reserve_uboot， 留出重定位后的 uboot 所占用的内存区域，uboot 所占用大小由 gd->mon_len 所指定，留出 uboot 的空间以后还要对 gd->relocaddr 做 4K 字节对齐，并且重新设置 gd->start_addr_sp，结果如图 12.2.4.10 所示：

```
DRAM:
gd->mon_len = 0XB98EC
gd->relocaddr = 0XF5C46000
gd->start_addr_sp = 0XF5C46000
1 GiB
```

图 12.2.4.10 信息输出

从图 12.2.4.10 可以看出：

```
gd->mon_len = 0XB98EC
gd->start_addr_sp = 0XF5C46000
gd->relocaddr = 0XF5C46000
```

gd->mon_len 是当前 uboot 大小，只要你在 uboot 里面添加了其他代码，那么这个编译出来的大小是会变的，所以大家如果发现本书其他地方的 mon_len 不等于 0XB98EC 不要以为写错了。

第 64 行，reserve_malloc，留出 malloc 区域以及 MMU 相关内存。 malloc 区域由宏 TOTAL_MALLOC_LEN 定义，宏定义如下：

```
#define TOTAL_MALLOC_LEN (CONFIG_SYS_MALLOC_LEN+CONFIG_ENV_SIZE)
```

.config 文件中定义宏 CONFIG_SYS_MALLOC_LEN 为 32MB=0X2000000，宏 CONFIG_ENV_SIZE=8KB=0X2000，因此 TOTAL_MALLOC_LEN=0X2002000。此时的 gd->start_addr_sp=0XF5C46000 - 0X2002000=0XF3C44000。

reserve_malloc 还通过 reserve_noncached 函数来预留出 MMU 相关内存。经过各种调整以后 gd->start_addr_sp 如图 12.2.4.11 所示：

```
DRAM:
gd->start_addr_sp = 0XF3B00000
1 GiB
```

图 12.2.4.11 信息输出

从图 12.2.4.11 可以看出，最终预留出的 malloc 和 MMU 相关内存范围为 0XF5C46000~0XF3AFFFF，此时 start_addr_sp 变成了 0XF3B00000

第 65 行，reserve_board 函数，留出板子 bd 所占的内存区，bd 是结构体 bd_t，bd_t 大小为 80 字节，结果如图 12.2.4.12 所示：

```
DRAM:
gd->bd= 0XF3AFFFB0
gd->start_addr_sp = 0XF3AFFFB0
1 GiB
```

图 12.2.4.12 信息输出

从图 12.2.4.12 可以看出：

```
gd->bd=0XF3AFFFB0
gd->start_addr_sp=0XF3AFFFB0
```

第 66 行，setup_machine，设置机器 ID，linux 启动的时候会和这个机器 ID 匹配，如果匹配的话 linux 就会启动正常。但是!! STM32MP1 不用这种方式了，这是以前老版本的 uboot 和 linux 使用的，新版本使用设备树了，因此此函数无效。

第 67 行，reserve_global_data 函数，保留出新的 gd_t 的内存区域，gd_t 结构体大小为 240B，结果如图 12.2.4.13 所示：

```
DRAM:
gd->new_gd= 0XF3AFFEC0
gd->start_addr_sp = 0XF3AFFEC0
1 GiB
```

图 12.2.4.13 信息输出

从图 12.2.4.13 可以看出，此时新的 gd 地址为 0XF3AFFEC0

```
gd->start_addr_sp=0XF3AFFEC0      //0XF3AFFFB0-240=0XF3AFFEC0
gd->new_gd=0XF3AFFEC0
```

第 68 行，reserve_fdt 函数，留出设备树相关的内存区域，首先计算出设备树的空间大小，计算公式如下：

```
gd->fdt_size = ALIGN(fdt_totalsize(gd->fdt_blob) + 0x1000, 32);
```

fdt_totalsize 函数负责计算出设备树的大小，gd->fdt_blob 是代码重定位前设备树地址。旧的地址大小加 4KB，在以 32 字节对齐。reserve_fdt 函数运行后的结果如图 12.2.4.14 所示：

```
DRAM:
gd->fdt_blob = 0XC01C2C50
gd->fdt_size = 0X11F00
gd->new_fdt = 0XF3AEDFC0
gd->start_addr_sp = 0XF3AEDFC0
1 GiB
```

图 12.2.4.14 信息输出

从图 12.2.4.14 可以看出：

```
gd->fdt_blob = 0XC01C2C50      //设备树的旧位置
gd->fdt_size= 0X11F00          //设备树.dtb 大小为 0X11F00
gd->new_fdt= 0XF3AEDFC0        //0XF3AFFEC0 - 0X11F00 = 0XF3AEDFC0
gd->start_addr_sp = 0XF3AEDFC0
```

第 69 行，reserve_bootstage 函数，保留 bootstage 的内存区域。结果如图所示：

```
DRAM:
gd->new_bootstage = 0XF3AEDD40
gd->start_addr_sp = 0XF3AEDD40
1 GiB
```

图 12.2.4.15 信息输出

从图 12.2.4.15 可以看出，新的 bootstage 地址为 0XF3AEDD40。

第 70 行，此函数不执行。

第 71 行，reserve_arch 是个空函数。

第 72 行，reserve_stacks，留出栈空间。如果使能 IRQ 的话还要留出 IRQ 相应的内存，具体工作是由 arch/arm/lib/stack.c 文件中的函数 arch_reserve_stacks 完成。结果如图 12.2.4.16 所示：

```
DRAM:
gd->start_addr_sp = 0XF3AEDD20
1 GiB
```

图 12.2.4.16 信息输出

至此，uboot 内存重定位完成。

第 73 行，dram_init_banksize 函数设置 dram 信息，就是设置 gd->bd->bi_dram[0].start 和 gd->bd->bi_dram[0].size，后面会传递给 linux 内核，告诉 linux DRAM 的起始地址和大小。结果如图 12.2.4.17 所示：

```
DRAM:
gd->bd->bi_dram[0].start = 0XC0000000
gd->bd->bi_dram[0].size = 0X40000000
1 GiB
```

图 12.2.4.17 信息输出

从图 12.2.4.17 可以看出，DRAM 的起始地址为 0XC0000000，大小为 0X40000000(1G)。

第 74 行，show_dram_config 函数，用于显示 DRAM 的配置，如图 12.2.4.18 所示：

```
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)
DRAM:  1 GiB
Clocks:
- MPU : 800 MHz
```

图 12.2.4.18 信息输出

第 78 行，display_new_sp 函数，显示新的 sp 位置，也就是 gd->start_addr_sp，不过要定义宏 DEBUG，结果如图 12.2.4.19 所示:

```
New Stack Pointer is: f3aedd20
```

图 12.2.4.19 信息输出

图 12.2.4.19 中的 gd->start_addr_sp 值和我们前面分析的最后一次修改的值一致。

第 81 行，reloc_fdt 函数用于重定位 fdt，使用 memcpy 函数将原来存储在 gd->fdt_blob 中的设备树文件拷贝到 gd->new_fdt 处，然后将 gd->new_fdt 赋值给 gd->fdt_blob。经过这一步设备树拷贝到了新地址，而且 gd->fdt_blob 也保存了新的设备树地址。

第 82 行，reloc_bootstage 函数用于 bootstage 重定位，使用 memcpy 函数将 gd->bootstage 中原来的 bootstage 拷贝到新的 gd->new_bootstage 处。然后将 gd->new_bootstage 赋值给 gd->bootstage。

第 84 行，setup_reloc，设置 gd 的其他一些成员变量，供后面重定位的时候使用，并且将以前的 gd 拷贝到 gd->new_gd 处。需要使能 DEBUG 才能看到相应的信息输出，如图 12.2.4.20 所示：

```
Relocation Offset is: 35b46000
Relocating to f5c46000, new gd at f3affec0, sp at f3aedd20
```

图 12.2.4.20 信息输出

从图 12.2.4.20 可以看出，uboot 重定位后的偏移为 0X35b46000，重定位后的新地址为 0XF5C46000，新的 gd 首地址为 0XF3AFFEC0，最终的 sp 为 0XF3AEDD20。

至此，board_init_f 函数就执行完成了，最终的内存分配如图 12.2.4.21 所示：

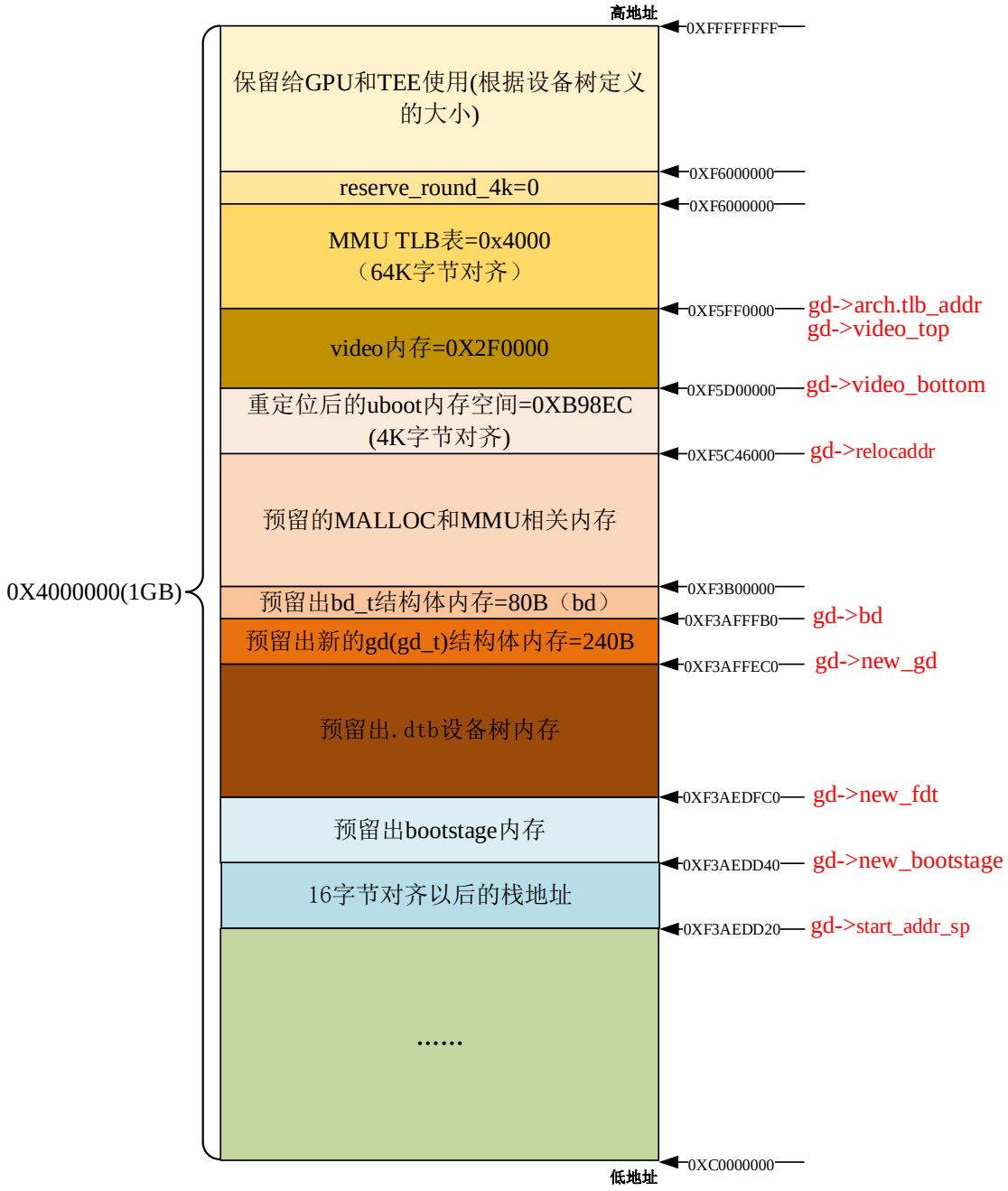


图 12.2.4.21 最终的内存分配图

12.2.5 relocate_code 函数详解

relocate_code 函数是用于代码拷贝的，此函数定义在文件 arch/arm/lib/relocate.S 中，代码如下：

示例代码 12.2.5.1 relocate.S 代码段

```
80  ENTRY(relocate_code)
81      ldr r1, =__image_copy_start /* r1 <- SRC &__image_copy_start */
82      subs    r4, r0, r1      /* r4 <- relocation offset   */
83      beq relocate_done       /* skip relocation           */
84      ldr r2, =__image_copy_end   /* r2 <- SRC &__image_copy_end */
```

```
85
86  copy_loop:
87      ldmia   r1!, {r10-r11}   /* copy from source address [r1]    */
88      stmia   r0!, {r10-r11}   /* copy to   target address [r0]    */
89      cmp r1, r2               /* until source end address [r2]    */
90      blo copy_loop
91
92      /*
93       * fix .rel.dyn relocations
94       */
95      ldr r2, =__rel_dyn_start /* r2 <- SRC &__rel_dyn_start   */
96      ldr r3, =__rel_dyn_end   /* r3 <- SRC &__rel_dyn_end     */
97  fixloop:
98      ldmia   r2!, {r0-r1}     /* (r0,r1) <- (SRC location,fixup) */
99      and r1, r1, #0xff
100     cmp r1, #R_ARM_RELATIVE
101     bne fixnext
102
103     /* relative fix: increase location by offset */
104     add r0, r0, r4
105     ldr r1, [r0]
106     add r1, r1, r4
107     str r1, [r0]
108 fixnext:
109     cmp r2, r3
110     blo fixloop
111
112 relocate_done:
113
114 #ifdef __XSCALE__
115     /*
116      * On xscale, icache must be invalidated and write buffers
117      * drained,even with cache disabled - 4.2.7 of xscale core
118      * developer's manual /
119     mcr p15, 0, r0, c7, c7, 0   /* invalidate icache     */
120     mcr p15, 0, r0, c7, c10, 4  /* drain write buffer    */
121 #endif
122
123   /* ARMv4- don't know bx lr but the assembler fails to see that */
124
125 #ifdef __ARM_ARCH_4__
126     mov pc, lr
127 #else
```

```
128     bx  lr
129 #endif
130
131 ENDPROC(relocate_code)
```

第 81 行，r1=__image_copy_start，也就是 r1 寄存器保存源地址，由表 12.1.1 可知，__image_copy_start=0XC0100000。

第 82 行，r0=0XF5C46000，这个地址就是 uboot 拷贝的目标首地址。r4=r0-r1=0XF5C46000-0XC0100000=0X35B46000，因此 r4 保存偏移量。

第 83 行，如果在第 81 中，r0-r1 等于 0，说明 r0 和 r1 相等，也就是源地址和目的地址是一样的，那肯定就不需要拷贝了！执行 relocate_done 函数

第 84 行，r2=__image_copy_end，r2 中保存拷贝之前的代码结束地址，由表 12.1.1 可知，__image_copy_end =0XC01ACD98。

第 85 行，函数 copy_loop 完成代码拷贝工作！从 r1，也就是__image_copy_start 开始，读取 uboot 代码保存到 r10 和 r11 中，一次就只拷贝这 2 个 32 位的数据。拷贝完成以后 r1 的值会更新，保存下一个要拷贝的数据地址。

第 88 行，将 r10 和 r11 的数据写到 r0 开始的地方，也就是目的地址。写完以后 r0 的值会更新，更新为下一个要写入的数据地址。

第 89 行，比较 r1 是否和 r2 相等，也就是检查是否拷贝完成，如果不相等的话说明没有拷贝完成，没有拷贝完成的话就跳转到 copy_loop 接着拷贝，直至拷贝完成。

接下来的第 95 行~110 行是重定位.rel.dyn 段， .rel.dyn 是.relocation.dynamic 的简写，也就是动态重定位的意，.rel.dyn 段是存放.text 段中需要重定位地址的集合。重定位就是 uboot 将自身拷贝到 DRAM 的另一个地放去继续运行(DRAM 的高地址处)。我们知道，一个可执行的 bin 文件，其链接地址和运行地址要相等，也就是链接到哪个地址，在运行之前就要拷贝到哪个地址去。现在我们重定位以后，运行地址就和链接地址不同了，这样寻址的时候不会出问题吗？为了分析这个问题，我们需要在 board/st/stm32mp1/stm32mp1.c 中输入如下所示内容：

示例代码 12.2.5.1 stm32mp1.c 新添代码段

```
1 static int rel_a = 0;
2
3 void rel_test(void)
4 {
5   rel_a += 100;
6   printf("rel_a = %d\r\n", rel_a);
7 }
```

最后还需要在 stm32mp1.c 文件中的 board_init 函数里面调用 rel_test 函数，否则 rel_reset 不会被编译进 uboot。修改完成后的 stm32mp1.c 如图 12.2.5.1 所示：

```
707  static int rel_a = 0;
708
709  void rel_test(void)
710  {
711      rel_a += 100;
712      printf("rel_a = %d\r\n", rel_a);
713  }
714
715  /* board dependent setup after realloc */
716  int board_init(void)
717  {
718      rel_test();
```

图 12.2.5.1 加入 rel 测试相关代码

board_init 函数会调用 rel_test，rel_test 会调用全局变量 rel_a。

重新编译 Uboot，编译完成以后，使用 arm-none-linux-gnueabihf-objdump 将编译出来的 u-boot 进行反汇编，得到 u-boot.dis 这个汇编文件，命令如下：

```
arm-none-linux-gnueabihf-objdump -D -m arm u-boot > u-boot.dis
```

在 u-boot.dis 文件中找到 rel_a、rel_rest 和 board_init，相关内容如下所示：

示例代码 12.2.5.2 汇编文件代码段

```
1  c010619c <rel_test>:
2  c010619c:    4b03    ldr r3, [pc, #12]   ; (c01061ac <rel_test+0x10>)
3  c010619e:    4804    ldr r0, [pc, #16]   ; (c01061b0 <rel_test+0x14>)
4  c01061a0:    6819    ldr r1, [r3, #0]
5  c01061a2:    3164    adds    r1, #100    ; 0x64
6  c01061a4:    6019    str r1, [r3, #0]
7  c01061a6:    f06d bbe6    b.w c0173976 <printf>
8  c01061aa:    bf00         nop
9  c01061ac:    c01ad378     andsgt  sp, sl, r8, ror r3
10 c01061b0:    c018de67     andsgt  sp, r8, r7, ror #28
11
12 c01061b4 <board_init>:
13 c01061b4:    b530         push    {r4, r5, lr}
14 c01061b6:    b089         sub sp, #36 ; 0x24
15 c01061b8:    f7ff fff0    bl  c010619c <rel_test>
16
17 ......
18
19 c01ad378 <rel_a>:
20 c01ad378:    00000000     andeq   r0, r0, r0
```

第 15 行是 borad_init 调用 rel_test 函数，用到了 bl 指令，而 bl 指令是位置无关指令，bl 指令是相对寻址的(pc+offset)，因此 uboot 中函数调用是与绝对位置无关的。

再来看一下函数 rel_test 对于全局变量 rel_a 的调用，第 2 行设置 r3 的值为 pc+12 地址处的值，根据后面的提示，这个地址就是 rel_test+0X10，也就是 0XC010619C+0X10=0XC01061AC。第 9 行就是 0XC01061AC 这个地址，0XC01061AC 处的值为 0XC01AD378。根据第 19 知，0XC01AD378 正是变量 rel_a 的地址，最终 r3=0XC01AD378。

第 4 行，读取 r3 地址出的值，保存到 r1 寄存器里面，也就是读取 rel_a 的值保存到 r1 里

面。

第 5 行，给 r1 里面的值加 100，也就是给 rel_a 的值加 100。

第 6 行，将 r1 内的值写到 r3 地址处，这不就是示例代码代码 12.2.5.1 中的第 5 行：rel_a += 100。

总结一下 rel_a+=100 的汇编执行过程：

①、在函数 rel_test 末尾处有一个地址为 0XC01061AC 的内存空间(示例代码 12.2.5.2 第 7 行)，此内存空间保存着变量 rel_a 的地址。

②、函数 rel_test 要想访问变量 rel_a，首先访问末尾的 0XC01061AC 来获取变量 rel_a 的地址，而访问 0XC01061AC 是通过偏移来访问的，很明显是个位置无关的操作。

③、通过 0XC01061AC 获取到变量 rel_a 的地址，对变量 rel_a 进行操作。

④、可以看出，函数 rel_test 对变量 rel_a 的访问没有直接进行，而是使用了一个第三方偏移地址 0XC01061AC，专业术语叫做 Label。这个第三方偏移地址就是实现重定位后运行不会出错的重要原因！

uboot 重定位后偏移为 0X35B46000，那么重定位后函数 rel_test 的首地址就是 0X C010619C +0X35B46000=0XF5C4C19C 。保存变量 rel_a 地址的 Label 就是 0XF5C4C19+0X10=0XF5C4C1AC(既：0XC01061AC +0X35B46000)。变量 rel_a 的地址就为 0XC01AD378+0X35B46000=0XF5CF3378。重定位后函数 rel_test 要想正常访问变量 rel_a 就得设置 0XF5C4C1AC (重定位后的 Label)地址出的值为 0XF5CF3378 (重定位后的变量 rel_a 地址)。这样就解决了重定位后链接地址和运行地址不一致的问题。

可以看出，uboot 对于重定位后链接地址和运行地址不一致的解决方法就是采用位置无关码，在使用 ld 进行链接的时候使用选项“-pie”生成位置无关的可执行文件。在文件 arch/arm/config.mk 下有如下代码：

示例代码 12.2.5.3 汇编文件代码段

```
102 # needed for relocation
103 LDFLAGS_u-boot += -pie
```

第 103 行就是设置 uboot 链接选项，加入了“-pie”选项。编译链接 uboot 的时候会使用到“-pie”选项，如下图所示：

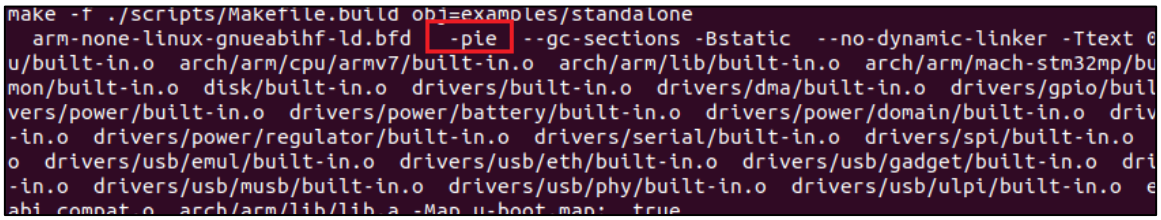


图 12.2.5.2 链接命令

使用“-pie”选项以后会生成一个.rel.dyn 段，uboot 就是靠这个.rel.dyn 来解决重定位问题的，在 u-bot.dis 的.rel.dyn 段中有如下所示内容：

示例代码 12.2.5.4 rel.dyn 代码段

```
1  Disassembly of section .rel.dyn:
2
3  c01ad2d8 <__efi_runtime_rel_stop>:
4  c01ad2d8:   c0100020    andsgt  r0, r0, r0, lsr #32
5  c01ad2dc:   00000017    andeq   r0, r0, r7, lsl r0
6  ......
```

```
7  c01ade08:    c01061ac    andsgt  r6, r0, ip, lsr #3
8  c01ade0c:    00000017    andeq   r0, r0, r7, lsl r0
9  c01ade10:    c01061b0            ; <UNDEFINED> instruction: 0xc01061b0
10 c01ade14:    00000017    andeq   r0, r0, r7, lsl r0
```

先来看一下.rel.dyn 段的格式，类似第 7 行和第 8 行这样的是一组，也就是两个 4 字节数据为一组。高 4 字节（第 7 行的 0XC01ADE0C）是 Label 地址标识，值为 0X17。低 4 字节(第 7 行的 0XC01ADE08)就是 Label 的地址。首先判断 Label 地址标识是否正确，也就是判断高 4 字节是否为 0X17，如果是的话低 4 字节就是 Label 地址值。第 7 行值为 0XC01061AC，第 8 行为 0X00000017，说明第 7 行的 0XC01061AC 是个 Label。

继续返回示例代码 12.2.5.1 中。

第 95 行，r2=__rel_dyn_start，也就是.rel.dyn 段的起始地址。

第 96 行，r3=__rel_dyn_end，也就是.rel.dyn 段的终止地址。

第 97 行，从.rel.dyn 段起始地址开始，每次读取两个 4 字节的数据存放到 r0 和 r1 寄存器中，r0 存放低 4 字节的数据，也就是 Label 地址；r1 存放高 4 字节的数据，也就是 Label 标志。

第 99 行，r1 中给的值与 0xff 进行与运算，其实就是取 r1 的低 8 位。

第 100 行，判断 r1 中的值是否等于 23(0X17)，R_ARM_RELATIVE 经过编译以后为 23。

第 101 行，如果 r1 不等于 23 的话就说明不是描述 Label 的，执行函数 fixnext，否则的话继续执行下面的代码。

第 104 行，r0 保存着 Label 值，r4 保存着重定位后的地址偏移，r0+r4 就得到了重定位后的 Label 值。此时 r0 保存着重定位后的 Label 值，相当于 0XC01006F0+0X35B46000=0XF5C466F0。

第 105 行，读取重定位后 Label 所保存的变量地址，此时这个变量地址还是重定位前的(相当于 rel_a 重定位前的地址 0XC01006F0)，将得到的值放到 r1 寄存器中。

第 106 行，r1+r4 即可得到重定位后的变量地址，相当于 rel_a 重定位后的 0XC01061AC +0X35B46000=0XF5C4C1AC。

第 107 行，重定位后的变量地址写入到重定位后的 Label 中，相等于设置地址 0XC01061AC 处的值为 0XF5C4C1AC。

第 109 行，比较 r2 和 r3，查看.rel.dyn 段重定位是否完成。

第 110 行，如果 r2 和 r3 不相等，说明.rel.dyn 重定位还未完成，因此跳到 fixloop 继续重定位.rel.dyn 段。

可以看出，uboot 中对.rel.dyn 段的重定位方法和我们猜想的一致。.rel.dyn 段的重定位比较复杂一点，有点绕，因为涉及到链接地址和运行地址的问题。

12.2.6 relocate_vectors 函数详解

函数 relocate_vectors 用于重定位向量表，此函数定义在文件 relocate.S 中，函数源码如下：

示例代码 12.2.6.1 relocate.S 代码段

```
28 ENTRY(relocate_vectors)
29
30 #ifdef CONFIG_CPU_V7M
31     /*
32      * On ARMv7-M we only have to write the new vector address
33      * to VTOR register.
34      */
35     ldr r0, [r9, #GD_RELOCADDR] /* r0 = gd->relocaddr */
```

```
36      ldr r1, =V7M_SCB_BASE
37      str r0, [r1, V7M_SCB_VTOR]
38  #else
39  #ifdef CONFIG_HAS_VBAR
40      /*
41       * If the ARM processor has the security extensions,
42       * use VBAR to relocate the exception vectors.
43       */
44      ldr r0, [r9, #GD_RELOCADDR] /* r0 = gd->relocaddr */
45      mcr     p15, 0, r0, c12, c0, 0  /* Set VBAR */
46  #else
47      /*
48       * Copy the relocated exception vectors to the
49       * correct address
50       * CP15 c1 V bit gives us the location of the vectors:
51       * 0x00000000 or 0xFFFF0000.
52       */
53      ldr r0, [r9, #GD_RELOCADDR] /* r0 = gd->relocaddr */
54      mrc p15, 0, r2, c1, c0, 0   /* V bit (bit[13]) in CP15 c1 */
55      ands    r2, r2, #(1 << 13)
56      ldreq   r1, =0x00000000     /* If V=0 */
57      ldrne   r1, =0xFFFF0000     /* If V=1 */
58      ldmia   r0!, {r2-r8,r10}
59      stmia   r1!, {r2-r8,r10}
60      ldmia   r0!, {r2-r8,r10}
61      stmia   r1!, {r2-r8,r10}
62  #endif
63  #endif
64      bx  lr
65
66  ENDPROC(relocate_vectors)
```

第 30 行，如果定义了 CONFIG_CPU_V7M 的话就执行第 30~36 行的代码，这是 Cortex-M 内核单片机执行的语句，因此对于 STM32MP157 来说是无效的。

第 39 行，如果定义了 CONFIG_HAS_VBAR 的话就执行此语句，这个是向量表偏移，Cortex-A7 是支持向量表偏移的。而且，在.config 里面定义了 CONFIG_HAS_VBAR，因此会执行这个分支。

第 44 行，r0=gd->relocaddr，也就是重定位后 uboot 的首地址，向量表肯定是从这个地址开始存放的。

第 45 行，将 r0 的值写入到 CP15 的 VBAR 寄存器中，也就是将新的向量表首地址写入到寄存器 VBAR 中，设置向量表偏移。

12.2.7 board_init_r 函数详解

第 12.2.4 小节讲解了 board_init_f 函数，在此函数里面会调用一系列的函数来初始化一些外

设和 gd 的成员变量。但是 board_init_f 并没有初始化所有的外设，还需要做一些后续工作，这些后续工作就是由函数 board_init_r 来完成的，board_init_r 函数定义在文件 common/board_r.c 中，代码如下：

示例代码 12.2.7.1 board_init_r 代码段

```
844 void board_init_r(gd_t *new_gd, ulong dest_addr)
845 {
......
869
870     if (initcall_run_list(init_sequence_r))
871         hang();
872
873     /* NOTREACHED - run_main_loop() does not return */
874     hang();
875 }
```

第 870 行调用 initcall_run_list 函数来执行初始化序列 init_sequence_r，init_sequence_r 是一个函数集合，init_sequence_r 也定义在文件 common/board_r.c 中，由于 init_sequence_f 的内容比较长，里面有大量的条件编译代码，这里为了缩小篇幅，将条件编译部分删除掉了，去掉条件编译以后的 init_sequence_r 定义如下：

示例代码 12.2.7.2 init_sequence_r 结构体

```
1   static init_fnc_t init_sequence_r[] = {
2       initr_trace,
3       initr_reloc,
4       initr_caches,
5       initr_reloc_global_data,
6       initr_unlock_ram_in_cache,
7       initr_barrier,
8       initr_malloc,
9       log_init,
10      initr_bootstage,    /* Needs malloc() but has its own timer */
11      initr_console_record,
12      initr_noncached,
13      initr_of_live,
14      initr_dm,
15      board_init, /* Setup chipselects */
16      set_cpu_clk_info, /* Setup clock information */
17      efi_memory_init,
18      stdio_init_tables,
19      initr_serial,
20      initr_announce,
21      initr_watchdog,
22      INIT_FUNC_WATCHDOG_RESET
23      initr_manual_reloc_cmdtable,
24      initr_trap,
```

```
25      initr_addr_map,
26      board_early_init_r,
27      INIT_FUNC_WATCHDOG_RESET
28      initr_post_backlog,
29      INIT_FUNC_WATCHDOG_RESET
30      initr_pci,
31      arch_early_init_r,
32      power_init_board,
33      initr_flash,
34      INIT_FUNC_WATCHDOG_RESET
35      cpu_init_r,
36      initr_nand,
37      initr_onenand,
38      initr_mmc,
39      initr_env,
40      initr_malloc_bootparams,
41      INIT_FUNC_WATCHDOG_RESET
42      initr_secondary_cpu,
43      mac_read_from_eeprom,
44      INIT_FUNC_WATCHDOG_RESET
45      initr_pci,
46      stdio_add_devices,
47      initr_jumptable,
48      initr_api,
49      console_init_r,     /* fully init console as a device */
50      console_announce_r,
51      show_board_info,
52      arch_misc_init,     /* miscellaneous arch-dependent init */
53      misc_init_r,        /* miscellaneous platform-dependent init */
54      INIT_FUNC_WATCHDOG_RESET
55      initr_kgdb,
56      interrupt_init,
57      initr_enable_interrupts,
58      timer_init,     /* initialize timer */
59      initr_status_led,
60      initr_ethaddr,
61      gpio_hog_probe_all,
62      board_late_init,
63      INIT_FUNC_WATCHDOG_RESET
64      initr_scsi,
65      initr_bbmii,
66      INIT_FUNC_WATCHDOG_RESET
67      initr_net,
```

```
68      initr_post,
69      initr_ide,
70      INIT_FUNC_WATCHDOG_RESET
71      last_stage_init,
72      INIT_FUNC_WATCHDOG_RESET
73      initr_bedbug,
74      initr_mem,
75      run_main_loop,
76  };
```

第 2 行，initr_trace 函数，如果定义了宏 CONFIG_TRACE 的话就会调用函数 trace_init，初始化和调试跟踪有关的内容。

第 3 行，initr_reloc 函数用于设置 gd->flags，标记重定位完成。

第 4 行，initr_caches 函数用于初始化 cache，使能 cache。

第 5 行，initr_reloc_global_data 函数，初始化重定位后 gd 的一些成员变量。

第 6 行，CONFIG_SYS_INIT_RAM_LOCK 宏没有定义，因此函数不执行。

第 7 行，CONFIG_PPC 宏没有定义，因此函数不执行。

第 8 行，initr_malloc 函数，初始化 malloc。

第 9 行，空函数。

第 10 行，初始化 bootstage 相关的内容。

第 11 行，初始化控制台相关内容。

第 12 行，保留 MMU 相关的 1M 内存。

第 13 行，CONFIG_OF_LIVE 宏没有定义，此函数不会运行。

第 14 行，初始化设备树，主要是解析设备树，创建一个树形结构，根节点挂载在 gd->dm_root。

第 15 行，board_init 函数，板级初始化，包括时钟、电压。这里执行的是 board/st/stm32mp1/stm32mp1.c 文件中的 board_init 函数。

第 16 行，CONFIG_CLOCKS 宏没有定义，此函数不执行。

第 17 行，efi 相关初始化。

第 18 行，stdio_init_tables 函数，stdio 相关初始化。

第 19 行，initr_serial 函数，初始化串口。

第 20 行，initr_announce 函数，与调试有关，通知已经在 RAM 中运行。

第 21 行，初始化看门狗，串口输出如图 12.2.7.1 所示：

```
WDT:   Started with servicing (32s timeout)
```

图 12.2.7.1 看门狗信息输出

第 23 行，CONFIG_NEEDS_MANUAL_RELOC 宏没有定义，此函数不执行。

第 24 行，此函数不执行。

第 25 行，CONFIG_ADDR_MAP 没有定义，此函数不执行。

第 26 行，CONFIG_BOARD_EARLY_INIT_R 宏没有定义，此函数不执行。

第 28 行，CONFIG_POST 宏没有定义，此函数不执行。

第 30 行，CONFIG_PCI 宏没有定义，此函数不执行。

第 31 行，CONFIG_ARCH_EARLY_INIT_R 宏没有定义，此函数不执行。

第 32 行，初始化电源，此函数为空函数。

第 33 行，CONFIG_MTD_NOR_FLASH 宏没有定义，此函数不执行。

第 35 行，此函数不执行。

第 36 行，initr_nand 函数，初始化 NAND Flash，STM32MP1 支持 NAND，并且也定义了 CONFIG_CMD_NAND 宏，因此此函数会执行。正点原子 STM32MP1 开发板没有使用 NAND ，因此识别出来 NAND 为 0MiB，如图 12.2.7.2 所示：

```
NAND:  0 MiB
```

图 12.2.7.2 NAND 信息

第 37 行，CONFIG_CMD_ONENAND 宏没有定义，此函数不执行。

第 38 行，initr_mmc 函数，初始化 EMMC，串口输出如图 12.2.7.1 所示信息：

```
MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
```

图 12.2.7.3 EMMC 信息输出

从图 12.2.7.3 可以看出，此时有两个 EMCM 设备，MMC:0 和 MMC:1。

第 39 行，initr_env 函数，初始化环境变量，如果在 EMMC 中保存了环境变量，那么就会提示从 EMMC 中读取环境变量，如图 12.2.7.4 所示：

```
Loading Environment from MMC... OK
```

图 12.2.7.4 从 EMMC 中读取环境变量

如果 EMMC 中没有保存环境变量，那么就会输出“bad CRC”类的信息。

第 40 行，CONFIG_SYS_BOOTPARAMS_LEN 宏没有定义，此函数不执行。

第 42 行，initr_secondary_cpu 函数，初始化其他 CPU 核。

第 43 行，CONFIG_ID_EEPROM 宏没有定义，此函数不执行。

第 46 行，stdio_add_devices 函数，各种输入输出设备的初始化。

第 47 行，initr_jumptable 函数，初始化跳转表。

第 48 行，CONFIG_API 宏没有定义，此函数不执行。

第 49 行，console_init_r 函数，控制台初始化，初始化完成以后此函数会调用 stdio_print_current_devices 函数来打印出当前的控制台设备，如图 12.2.7.5 所示：

```
In:    serial
Out:   serial
Err:   serial
```

图 12.2.7.5 控制台信息

第 50~51 行，CONFIG_DISPLAY_BOARDINFO_LATE 宏没有定义，这两个函数不执行。

第 52 行，arch_misc_init 函数，设置一些其他的东西，函数的路径为：arch/arm/mach-stm32mp/cpu.c。这里重点是从 STM32MP1 的 OTP 里面读取 MAC 地址来设置“ethaddr”环境变量，信息如下图所示：

```
invalid MAC address in OTP 00:00:00:00:00:00
```

图 12.2.7.6 从 OTP 获取 MAC 地址

从图 12.2.7.6 可以看出，从 OTP 读取到的 MAC 地址是错误的，因为我们没有在 OTP 里面设置 MAC 地址。

第 53 行，CONFIG_MISC_INIT_R 宏没有定义，此函数不执行。

第 55 行，CONFIG_CMD_KGDB 宏没有定义，此函数不执行。

第 56 行，interrupt_init 函数，初始化中断。

第 57 行，initr_enable_interrupts 函数，使能中断。

第 58~59 行，函数不执行。

第 60 行，根据 bd->bi_enetaddr 变量成员，去设置 ethaddr 的 MAC 地址。

第 61 行，gpio_hog_orobe_all 函数，初始化 uboot 下的 gpio 驱动框架。

第 62 行，board_late_init 函数，板子后续初始化，此函数定义在文件 board/st/stm32mp1/stm32mp1.c 中。

第 64~65 行，此函数不执行。

第 67 行，网络的初始化，输出的信息如图 12.2.7.7 所示：

```
Net:
Error: ethernet@5800a000 address not set.
No ethernet found.
```

图 12.2.7.7 网络初始化

图 12.2.7.7 中由于没有设置网络相关地址信息，因此网络初始化失败，提示“No ethernet found”。如果设置了网络地址环境变量，那么就会初始化成功网络，输出信息如下：

```
Net:   eth0: ethernet@5800a000
```

图 12.2.7.8 网络初始化成功

第 68~74 行，这里的函数都不执行。

第 75 行，run_main_loop 行，主循环，处理命令。

12.2.8 run_main_loop 函数详解

uboot 启动以后会进入 N(N=1.2.3…)秒倒计时，如果在 N 秒倒计时结束之前按下回车键，那么就会进入 uboot 的命令模式，如果倒计时结束以后都没有按下回车键，那么就会自动启动 Linux 内核，这个功能就是由 run_main_loop 函数来完成的。run_main_loop 函数定义在文件 common/board_r.c 中，函数内容如下：

示例代码 12.2.8.1 run_main_loop 函数

```
645 static int run_main_loop(void)
646 {
647 #ifdef CONFIG_SANDBOX
648     sandbox_main_loop_init();
649 #endif
650     /* main_loop() can return to retry autoboot, if so just run it
        again */
651     for (;;)
652         main_loop();
653     return 0;
654 }
```

第 651 行和第 652 行是个死循环，“for(;;)”和“while(1)”功能一样，死循环里面就一个 main_loop 函数，main_loop 函数定义在文件 common/main.c 里面，代码如下：

示例代码 12.2.8.2 main_loop 函数

```
41  void main_loop(void)
42  {
43      const char *s;
44
45      bootstage_mark_name(BOOTSTAGE_ID_MAIN_LOOP, "main_loop");
```

```
46
47      if (IS_ENABLED(CONFIG_VERSION_VARIABLE))
48          env_set("ver", version_string);  /* set version variable */
49
50      cli_init();
51
52      if (IS_ENABLED(CONFIG_USE_PREBOOT))
53          run_preboot_environment_command();
54
55      if (IS_ENABLED(CONFIG_UPDATE_TFTP))
56          update_tftp(0UL, NULL, NULL);
57
58      s = bootdelay_process();
59      if (cli_process_fdt(&s))
60          cli_secure_boot_cmd(s);
61
62      autoboot_command(s);
63
64      cli_loop();
65      panic("No CLI available");
66  }
```

第 45 行，调用 bootstage_mark_name 函数，打印出启动进度。

第 47 行，如果定义了宏 CONFIG_VERSION_VARIABLE 的话就会执行函数 setenv，设置环境变量 ver 的值为 version_string，也就是设置版本号环境变量。version_string 定义在文件 cmd/version.c 中，定义如下：

```
const char __weak version_string[] = U_BOOT_VERSION_STRING;
```

U_BOOT_VERSION_STRING 是个宏，定义在文件 include/version.h，如下：

```
#define U_BOOT_VERSION_STRING U_BOOT_VERSION " (" U_BOOT_DATE " - " \
    U_BOOT_TIME " " U_BOOT_TZ ")" CONFIG_IDENT_STRING
```

U_BOOT_VERSION 定义在文件 include/generated/version_autogenerated.h 中，文件 version_autogenerated.h 内如如下：

示例代码 12.2.8.3 version_autogenerated.h 文件代码

```
1  #define PLAIN_VERSION "2020.01-stm32mp-r1"
2  #define U_BOOT_VERSION "U-Boot " PLAIN_VERSION
3  #define CC_VERSION_STRING "arm-none-linux-gnueabihf-gcc (GNU Toolcha
in for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 9.2.1 20191025"
4  #define LD_VERSION_STRING "GNU ld (GNU Toolchain for the A-profile Ar
chitecture 9.2-2019.12 (arm-9.10)) 2.33.1.20191209"
```

可以看出，U_BOOT_VERSION 为“U-boot 2020.01-stm32mp-r1”，U_BOOT_DATE、U_BOOT_TIME 和 U_BOOT_TZ 这定义在文件 include/generated/timestamp_autogenerated.h 中，如下所示：

示例代码 12.2.8.4 timestamp_autogenerated.h 文件代码

```
1  #define U_BOOT_DATE "Oct 25 2021"
```

```
2   #define U_BOOT_TIME "16:07:12"
3   #define U_BOOT_TZ "+0800"
4   #define U_BOOT_DMI_DATE "10/25/2021"
5   #define U_BOOT_BUILD_DATE 0x20211025
```

宏 CONFIG_IDENT_STRING 为空，所以 U_BOOT_VERSION_STRING 为“U-Boot 2020.01-stm32mp-r1 (Jun 24 2021 - 18:05:23 +0800)”，进入 uboot 命令模式，输入命令“version”查看版本号，如图 12.2.8.1 所示：

```
STM32MP> version
U-Boot 2020.01-stm32mp-r1 (Oct 25 2021 - 16:07:12 +0800)

arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 9.2.1 20191025
GNU ld (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 2.33.1.20191209
STM32MP>
```

图 12.2.8.1 版本查询

图 12.2.8.1 中的第一行就是 uboot 版本号，和我们分析的一致。

接着回到示例代码 12.2.8.2 中，第 50 行，cli_init 函数，跟命令初始化有关，初始化 hush shell 相关的变量。

第 53 行，run_preboot_environment_command 函数，获取环境变量 perboot 的内容，perboot 是一些预启动命令，一般不使用这个环境变量。

第 58 行，bootdelay_process 函数，此函数会读取环境变量 bootdelay 和 bootcmd 的内容，然后将 bootdelay 的值赋值给全局变量 stored_bootdelay，返回值为环境变量 bootcmd 的值。

第 59 行，如果定义了 CONFIG_OF_CONTROL 的话函数 cli_process_fdt 执行。

第 62 行，autoboot_command 函数，此函数就是检查倒计时是否结束？倒计时结束之前有没有被打断？此函数定义在文件 common/autoboot.c 中，内容如下：

示例代码 12.2.8.5 autoboot_command 函数

```
359 void autoboot_command(const char *s)
360 {
361     debug("### main_loop: bootcmd=\"%s\"\n", s ? s : "<UNDEFINED>");
362
363     if (stored_bootdelay != -1 && s && !abortboot(stored_bootdelay)) {
364         bool lock;
365         int prev;
366
367         lock = IS_ENABLED(CONFIG_AUTOBOOT_KEYED) &&
368             !IS_ENABLED(CONFIG_AUTOBOOT_KEYED_CTRLC);
369         if (lock)
370             prev = disable_ctrlc(1); /* disable Ctrl-C checking */
371
372         run_command_list(s, -1, 0);
373
374         if (lock)
375             disable_ctrlc(prev);    /* restore Ctrl-C checking */
376     }
377
378     if (IS_ENABLED(CONFIG_USE_AUTOBOOT_MENUKEY) &&
```

```
379         menukey == AUTOBOOT_MENUKEY) {
380         s = env_get("menucmd");
381         if (s)
382             run_command_list(s, -1, 0);
383     }
384 }
```

可以看出，autoboot_command 函数里面有很多条件编译，条件编译一多就不利于我们阅读程序。宏 CONFIG_AUTOBOOT_KEYED 、 CONFIG_AUTOBOOT_KEYED_CTRLC 和 CONFIG_MENUKEY 这三个宏在 STM32MP1 里面没有定义，所以讲示例代码 12.2.8.6 进行精简，得到如下代码：

示例代码 12.2.8.6 autoboot_command 函数精简版本

```
1 void autoboot_command(const char *s)
2 {
3   if (stored_bootdelay != -1 && s && !abortboot(stored_bootdelay)) {
4       run_command_list(s, -1, 0);
5   }
6 }
```

当一下三条全部成立的话，就会执行函数 run_command_list。

①、stored_bootdelay 不等于-1。

②、s 不为空。

③、函数 abortboot 返回值为 0。

stored_bootdelay 等于环境变量 bootdelay 的值；s 是环境变量 bootcmd 的值，一般不为空，因此前两个成立，就剩下了函数 abortboot 的返回值，abortboot 函数也定义在文件 common/autoboot.c 中，内容如下：

示例代码 12.2.8.7 abortboot 函数

```
288 static int abortboot(int bootdelay)
289 {
290     int abort = 0;
291
292     if (bootdelay >= 0) {
293         if (IS_ENABLED(CONFIG_AUTOBOOT_KEYED))
294             abort = abortboot_key_sequence(bootdelay);
295         else
296             abort = abortboot_single_key(bootdelay);
297     }
298
299     if (IS_ENABLED(CONFIG_SILENT_CONSOLE) && abort)
300         gd->flags &= ~GD_FLG_SILENT;
301
302     return abort;
303 }
```

因为宏 CONFIG_AUTOBOOT_KEYED 未定义，因此执行函数 abortboot_single_key。接着来看函数 abortboot_single_key，此函数也定义在文件 common/autoboot.c 中，内容如下：

示例代码 12.2.8.8 abortboot_single_key 函数

```
246 static int abortboot_single_key(int bootdelay)
247 {
248     int abort = 0;
249     unsigned long ts;
250
251     printf("Hit any key to stop autoboot: %2d ", bootdelay);
252
253     /*
254      * Check if key already pressed
255      */
256     if (tstc()) {   /* we got a key press   */
257         (void) getc();  /* consume input    */
258         puts("\b\b\b 0");
259         abort = 1;  /* don't auto boot  */
260     }
261
262     while ((bootdelay > 0) && (!abort)) {
263         --bootdelay;
264         /* delay 1000 ms */
265         ts = get_timer(0);
266         do {
267             if (tstc()) {   /* we got a key press   */
268                 int key;
269
270                 abort  = 1; /* don't auto boot  */
271                 bootdelay = 0;  /* no more delay    */
272                 key = getc(); /* consume input  */
273                 if (IS_ENABLED(CONFIG_USE_AUTOBOOT_MENUKEY))
274                     menukey = key;
275                 break;
276             }
277             udelay(10000);
278         } while (!abort && get_timer(ts) < 1000);
279
280         printf("\b\b\b%2d ", bootdelay);
281     }
282
283     putc('\n');
284
285     return abort;
```

第 248 行的变量 abort 是函数 abortboot_normal 的返回值，默认值为 0。

第 251 行通过串口输出“Hit any key to stop autoboot”字样，如图 12.2.8.2 所示：

```
Hit any key to stop autoboot:  0
```

图 12.2.8.2 倒计时

第 262~278 行就是倒计时的具体实现。

第 267 行判断键盘是否有按下，也就是是否打断了倒计时，如果键盘按下的话就执行相应的分支。比如设置 abort 为 1，设置 bootdelay 为 0 等，最后跳出倒计时循环。

第 285 行，返回 abort 的值，如果倒计时自然结束，没有被打断 abort 就为 0，否则的话 abort 的值就为 1。

回到示例代码 12.2.8.6 的 autoboot_command 函数中，如果倒计时自然结束那么就执行函数 run_command_list，此函数会执行参数 s 指定的一系列命令，也就是环境变量 bootcmd 的命令，bootcmd 里面保存着默认的启动命令，因此 linux 内核启动！这个就是 uboot 中倒计时结束以后自动启动 linux 内核的原理。如果倒计时结束之前按下了键盘上的按键，那么 run_command_list 函数就不会执行，相当于 autoboot_command 是个空函数。

回到“遥远”的示例代码 12.2.8.2 中的 main_loop 函数中，如果倒计时结束之前按下按键，那么就会执行第 64 行的 cli_loop 函数，这个就是命令处理函数，负责接收好处理输入的命令。此函数定义在文件 common/cli.c 中，具体的内容我们就不看了。

12.2.9 cli_loop 函数详解

cli_loop 函数是 uboot 的命令行处理函数，我们在 uboot 中输入各种命令，进行各种操作就是有 cli_loop 来处理的，此函数定义在文件 common/cli.c 中，函数内容如下：

示例代码 12.2.9.1 cli.c 文件代码段

```
216 void cli_loop(void)
217 {
218     bootstage_mark(BOOTSTAGE_ID_ENTER_CLI_LOOP);
219 #ifdef CONFIG_HUSH_PARSER
220     parse_file_outer();
221     /* This point is never reached */
222     for (;;);
223 #elif defined(CONFIG_CMDLINE)
224     cli_simple_loop();
225 #else
226     printf("## U-Boot command line is disabled. Please enable
CONFIG_CMDLINE\n");
227 #endif /*CONFIG_HUSH_PARSER*/
228
```

第 220 行，STM32MP157 定义了宏 CONFIG_SYS_HUSH_PARSER。因此会调用函数 parse_file_outer。

第 222 行是个死循环，永远不会执行到这里。

函数 parse_file_outer 定义在文件 common/cli_hush.c 中，去掉条件编译内容以后的函数内容如下：

示例代码 12.2.9.2 parse_file_outer 函数精简

```
1 int parse_file_outer(void)
2 {
```

```
3       int rcode;
4       struct in_str input;
5
6       setup_file_in_str(&input);
7       rcode = parse_stream_outer(&input, FLAG_PARSE_SEMICOLON);
8       return rcode;
9   }
```

第 6 行调用函数 setup_file_in_str 初始化变量 input 的成员变量。

第 7 行调用函数 parse_stream_outer，这个函数就是 hush shell 的命令解释器，负责接收命令行输入，然后解析并执行相应的命令，函数 parse_stream_outer 定义在文件 common/cli_hush.c 中，精简版的函数内容如下：

示例代码 12.2.9.3 parse_stream_outer 函数精简

```
1   static int parse_stream_outer(struct in_str *inp, int flag)
2   {
3
4       struct p_context ctx;
5       o_string temp=NULL_O_STRING;
6       int rcode;
7       int code = 1;
8
9       do {
10          ......
11          rcode = parse_stream(&temp, &ctx, inp,
12                  flag & FLAG_CONT_ON_NEWLINE ? -1 : '\n');
13          ......
14          if (rcode != 1 && ctx.old_flag == 0) {
15              ......
16              code = run_list(ctx.list_head);
17              ......
18          } else {
19              ......
20          }
21          b_free(&temp);
22      /* loop on syntax errors, return on EOF */
23      } while (rcode != -1 && !(flag & FLAG_EXIT_FROM_LOOP) &&
24      (   inp->peek != static_peek || b_peek(inp)));
25
26      return (code != 0) ? 1 : 0;
27  }
```

第 9~23 行中的 do-while 循环就是处理输入命令的。

第 11 行调用函数 parse_stream 进行命令解析。

第 16 行调用 run_list 函数来执行解析出来的命令。

函数 run_list 会经过一系列的函数调用，最终通过调用 cmd_process 函数来处理命令，过程如下：

示例代码 12.2.9.4 run_list 执行流程

```
1  static int run_list(struct pipe *pi)
2  {
3      int rcode=0;
4
5      rcode = run_list_real(pi);
6      ......
7      return rcode;
8  }
9
10 static int run_list_real(struct pipe *pi)
11 {
12     char *save_name = NULL;
13     ......
14     int rcode=0, flag_skip=1;
15     ......
16     rcode = run_pipe_real(pi);
17     ......
18     return rcode;
19 }
20
21 static int run_pipe_real(struct pipe *pi)
22 {
23     int i;
24     ......
25     int nextin;
26     int flag = do_repeat ? CMD_FLAG_REPEAT : 0;
27     struct child_prog *child;
28     char *p;
29     ......
30     if (pi->num_progs == 1) child = & (pi->progs[0]);
31         ......
32         return rcode;
33     } else if (pi->num_progs == 1 && pi->progs[0].argv != NULL) {
34         ......
35         /* Process the command */
36         return cmd_process(flag, child->argc, child->argv,
37                 &flag_repeat, NULL);
38
39  }
40
```

```
41      return -1;
42  }
```

第 5 行，run_list 调用 run_list_real 函数。

第 16 行，run_list_real 函数调用 run_pipe_real 函数。

第 36 行，run_pipe_real 函数调用 cmd_process 函数。

最终通过函数 cmd_process 来处理命令，接下来就是分析 cmd_process 函数。

12.2.10 cmd_process 函数详解

在学习 cmd_process 之前先看一下 uboot 中命令是如何定义的。uboot 使用宏 U_BOOT_CMD 来定义命令，宏 U_BOOT_CMD 定义在文件 include/command.h 中，定义如下：

示例代码 12.2.10.1 U_BOOT_CMD 宏定义

```
#define U_BOOT_CMD(_name, _maxargs, _rep, _cmd, _usage, _help)      \
   U_BOOT_CMD_COMPLETE(_name, _maxargs, _rep, _cmd, _usage, _help, NULL)
```

可以看出 U_BOOT_CMD 是 U_BOOT_CMD_COMPLETE 的特例，将 U_BOOT_CMD_COMPLETE 的最后一个参数设置成 NULL 就是 U_BOOT_CMD。宏 U_BOOT_CMD_COMPLETE 如下：

示例代码 12.2.10.2 U_BOOT_CMD_COMPLETE 宏定义

```
#define U_BOOT_CMD_COMPLETE(_name, _maxargs, _rep, _cmd, _usage, _help,
_comp) \
    ll_entry_declare(cmd_tbl_t, _name, cmd) =          \
       U_BOOT_CMD_MKENT_COMPLETE(_name, _maxargs, _rep, _cmd,  \
                     _usage, _help, _comp);
```

宏 U_BOOT_CMD_COMPLETE 又用到了 ll_entry_declare 和 U_BOOT_CMD_MKENT_COMPLETE。ll_entry_declar 定义在文件 include/linker_lists.h 中，定义如下：

示例代码 12.2.10.3 ll_entry_declare 宏定义

```
#define ll_entry_declare(_type, _name, _list)              \
    _type _u_boot_list_2_##_list##_2_##_name __aligned(4)      \
          __attribute__((unused,              \
          section(".u_boot_list_2_"#_list"_2_"#_name)))
```

_type 为 cmd_tbl_t，因此 ll_entry_declare 就是定义了一个 cmd_tbl_t 变量，这里用到了 C 语言中的“##”连接符符。其中的“##_list”表示用_list 的值来替换，“##_name”就是用_name 的值来替换。

宏 U_BOOT_CMD_MKENT_COMPLETE 定义在文件 include/command.h 中，内容如下：

示例代码 12.2.10.4 U_BOOT_CMD_MKENT_COMPLETE 宏定义

```
#define U_BOOT_CMD_MKENT_COMPLETE(_name, _maxargs, _rep, _cmd,      \
              _usage, _help, _comp)          \
       { #_name, _maxargs,                \
        _rep ? cmd_always_repeatable : cmd_never_repeatable,   \
        _cmd, _usage, _CMD_HELP(_help) _CMD_COMPLETE(_comp) }
```

上述代码中的“#”表示将_name 传递过来的值字符串化，U_BOOT_CMD_MKENT_COMPLETE 又用到了宏_CMD_HELP 和_CMD_COMPLETE，这两个宏的定义如下：

示例代码 12.2.10.5 _CMD_HELP 和_CMD_COMPLETE 宏定义

```
1  #ifdef CONFIG_AUTO_COMPLETE
2  # define _CMD_COMPLETE(x) x,
3  #else
4  # define _CMD_COMPLETE(x)
5  #endif
6  #ifdef CONFIG_SYS_LONGHELP
7  # define _CMD_HELP(x) x,
8  #else
9  # define _CMD_HELP(x)
10 #endif
```

可以看出，如果定义了宏 CONFIG_AUTO_COMPLETE 和 CONFIG_SYS_LONGHELP 的话，_CMD_COMPLETE 和_CMD_HELP 就是取自身的值，然后在加上一个‘,’。CONFIG_AUTO_COMPLETE 和 CONFIG_SYS_LONGHELP 这两个宏都有定义。

U_BOOT_CMD 宏的流程我们已经清楚了(一个 U_BOOT_CMD 宏就如此的绕来绕去的!)，我们就以一个具体的命令为例，来看一下 U_BOOT_CMD 经过展开以后究竟是个什么模样的。以命令 dhcp 为例，dhcp 命令定义在 cmd/net.c 文件中，内容如下：

示例代码 12.2.10.6 dhcp 命令宏定义

```
U_BOOT_CMD(
    dhcp,   3,  1,  do_dhcp,
    "boot image via network using DHCP/TFTP protocol",
    "[loadAddress] [[hostIPaddr:]bootfilename]"
);
```

将其展开，结果如下：

示例代码 12.2.10.7 dhcp 命令展开

```
U_BOOT_CMD(
    dhcp,   3,  1,  do_dhcp,
    "boot image via network using DHCP/TFTP protocol",
    "[loadAddress] [[hostIPaddr:]bootfilename]"
);

1、将 U_BOOT_CMD 展开后为:
U_BOOT_CMD_COMPLETE(dhcp, 3, 1, do_dhcp,
                    "boot image via network using DHCP/TFTP protocol",
                    "[loadAddress] [[hostIPaddr:]bootfilename]",
                    NULL)

2、将 U_BOOT_CMD_COMPLETE 展开后为:
ll_entry_declare(cmd_tbl_t, dhcp, cmd) =          \
U_BOOT_CMD_MKENT_COMPLETE(dhcp, 3, 1, do_dhcp,  \
                "boot image via network using DHCP/TFTP protocol", \
                "[loadAddress] [[hostIPaddr:]bootfilename]", \
                NULL);
```

```
3、将 ll_entry_declare 和 U_BOOT_CMD_MKENT_COMPLETE 展开后为:
cmd_tbl_t _u_boot_list_2_cmd_2_dhcp __aligned(4)      \
          __attribute__((unused,section(.u_boot_list_2_cmd_2_dhcp))) \
      { "dhcp", 3, 1, do_dhcp,  \
        "boot image via network using DHCP/TFTP protocol", \
        "[loadAddress] [[hostIPaddr:]bootfilename]",\
        NULL}
```

从示例代码 12.2.10.7 可以看出，dhcp 命令最终展开结果为：

示例代码 12.2.10.8 dhcp 命令最终结果

```
1 cmd_tbl_t _u_boot_list_2_cmd_2_dhcp __aligned(4)      \
2         __attribute__((unused,section(.u_boot_list_2_cmd_2_dhcp))) \
3       { "dhcp", 3, 1, do_dhcp,  \
4       "boot image via network using DHCP/TFTP protocol", \
5       "[loadAddress] [[hostIPaddr:]bootfilename]",\
6       NULL}
```

第 1 行定义了一个 cmd_tbl_t 类型的变量，变量名为_u_boot_list_2_cmd_2_dhcp，此变量 4 字节对齐。

第 2 行，使用__attribute__关键字设置变量_u_boot_list_2_cmd_2_dhcp 存储在.u_boot_list_2_cmd_2_dhcp 段中。u-boot.lds 链接脚本中有一个名为“.u_boot_list”的段，所有.u_boot_list 开头的段都存放到.u_boot.list 中，如图 12.2.10.1 所示：

```
35   . = ALIGN(4);
36   . = .;
37   . = ALIGN(4);
38   .u_boot_list : {
39    KEEP(*(SORT(.u_boot_list*)));
40   }
41   . = ALIGN(4);
```

图 12.2.10.1 u-boot.lds 中的.u_boot_list 段

因此，第 2 行就是设置变量_u_boot_list_2_cmd_2_dhcp 的存储位置。

第 3~6 行，cmd_tbl_t 是个结构体，因此第 3-6 行是初始化 cmd_tbl_t 这个结构体的各个成员变量。cmd_tbl_t 结构体定义在文件 include/command.h 中，内容如下：

示例代码 12.2.10.9 cmd_tbl_t 结构体

```
30 struct cmd_tbl_s {
31      char    *name;          /* Command Name         */
32      int     maxargs;        /* maximum number of arguments  */
......
41      int     (*cmd_rep)(struct cmd_tbl_s *cmd, int flags, int argc,
42                char * const argv[], int *repeatable);
43                              /* Implementation function  */
44      int     (*cmd)(struct cmd_tbl_s *, int, int, char * const []);
45      char        *usage;     /* Usage message    (short) */
46      #ifdef   CONFIG_SYS_LONGHELP
47      char        *help;      /* Help  message    (long)  */
```

```
48      #endif
49      #ifdef CONFIG_AUTO_COMPLETE
50      /* do auto completion on the arguments */
51      int     (*complete)(int argc, char * const argv[],
                    char last_char, int maxv, char *cmdv[]);
52      #endif
53  };
54
55  typedef struct cmd_tbl_s cmd_tbl_t;
```

结合实例代码 12.2.10.8，可以得出变量_u_boot_list_2_cmd_2_dhcp 的各个成员的值如下所示：

```
_u_boot_list_2_cmd_2_dhcp.name = "dhcp"
_u_boot_list_2_cmd_2_dhcp.maxargs = 3
_u_boot_list_2_cmd_2_dhcp. cmd_rep = cmd_always_repeatable
_u_boot_list_2_cmd_2_dhcp.cmd = do_dhcp
_u_boot_list_2_cmd_2_dhcp.usage = "boot image via network using DHCP/TFTP protocol"
_u_boot_list_2_cmd_2_dhcp.help =    "[loadAddress] [[hostIPaddr:]bootfilename]"
_u_boot_list_2_cmd_2_dhcp.complete = NULL
```

当我们在 uboot 的命令行中输入“dhcp”这个命令的时候，最终执行的是 do_dhcp 这个函数。总结一下，uboot 中使用 U_BOOT_CMD 来定义一个命令，最终的目的就是为了定义一个 cmd_tbl_t 类型的变量，并初始化这个变量的各个成员。uboot 中的每个命令都存储在.u_boot_list 段中，每个命令都有一个名为 do_xxx(xxx 为具体的命令名)的函数，这个 do_xxx 函数就是具体的命令处理函数。

了解了 uboot 中命令的组成以后，再来看一下 cmd_process 函数的处理过程，cmd_process 函数定义在文件 common/command.c 中，函数内容如下：

示例代码 12.2.10.10 command.c 文件代码段

```
582 enum command_ret_t cmd_process(int flag, int argc,
583       char * const argv[], int *repeatable, ulong *ticks)
584 {
585     enum command_ret_t rc = CMD_RET_SUCCESS;
586     cmd_tbl_t *cmdtp;
......
602     /* Look up command in command table */
603     cmdtp = find_cmd(argv[0]);
604     if (cmdtp == NULL) {
605         printf("Unknown command '%s' - try 'help'\n", argv[0]);
606         return 1;
607     }
608
609     /* found - check max args */
610     if (argc > cmdtp->maxargs)
611         rc = CMD_RET_USAGE;
612
```

```
613 #if defined(CONFIG_CMD_BOOTD)
614     /* avoid "bootd" recursion */
615     else if (cmdtp->cmd == do_bootd) {
616         if (flag & CMD_FLAG_BOOTD) {
617             puts("'bootd' recursion detected\n");
618             rc = CMD_RET_FAILURE;
619         } else {
620             flag |= CMD_FLAG_BOOTD;
621         }
622     }
623 #endif
624
625     /* If OK so far, then do the command */
626     if (!rc) {
627         int newrep;
628
629         if (ticks)
630             *ticks = get_timer(0);
631         rc = cmd_call(cmdtp, flag, argc, argv, &newrep);
632         if (ticks)
633             *ticks = get_timer(*ticks);
634         *repeatable &= newrep;
635     }
636     if (rc == CMD_RET_USAGE)
637         rc = cmd_usage(cmdtp);
638     return rc;
639 }
```

第 603 行，调用函数 find_cmd 在命令表中找到指定的命令，find_cmd 函数内容如下：

示例代码 12.2.10.11 command.c 文件代码段

```
120 cmd_tbl_t *find_cmd(const char *cmd)
121 {
123     cmd_tbl_t *start = ll_entry_start(cmd_tbl_t, cmd);
124     const int len = ll_entry_count(cmd_tbl_t, cmd);
125     return find_cmd_tbl(cmd, start, len);
126 }
```

参数 cmd 就是所查找的命令名字，uboot 中的命令表其实就是 cmd_tbl_t 结构体数组，通过函数 ll_entry_start 得到数组的第一个元素，也就是命令表起始地址。通过函数 ll_entry_count 得到数组长度，也就是命令表的长度。最终通过函数 find_cmd_tbl 在命令表中找到所需的命令，每个命令都有一个 name 成员，所以将参数 cmd 与命令表中每个成员的 name 字段都对比一下，如果相等的话就说明找到了这个命令，找到以后就返回这个命令。

回到示例代码 12.2.10.10 的 cmd_process 函数中，找到命令以后肯定就要执行这个命令了，第 631 行调用函数 cmd_call 来执行具体的命令，cmd_call 函数内容如下：

示例代码 12.2.10.12 command.c 文件代码段

```
571 static int cmd_call(cmd_tbl_t *cmdtp, int flag, int argc,
572           char * const argv[], int *repeatable)
573 {
574     int result;
575
576     result = cmdtp->cmd_rep(cmdtp, flag, argc, argv, repeatable);
577     if (result)
578         debug("Command failed, result=%d\n", result);
579     return result;
580 }
```

第 576 行，调用 cmd_tbl_t 的 cmd_rep 成员变量，也就是 cmd_always_repeatable，此函数定义在 common/command.c 文件里面，内容如下：

示例代码 12.2.10.13 cmd_always_repeatable 函数

```
535 int cmd_always_repeatable(cmd_tbl_t *cmdtp, int flag, int argc,
536             char * const argv[], int *repeatable)
537 {
538     *repeatable = 1;
539
540     return cmdtp->cmd(cmdtp, flag, argc, argv);
541 }
```

在前面的分析中我们知道，cmd_tbl_t 的 cmd 成员就是具体的命令处理函数，所以第 540 行调用 cmdtp 的 cmd 成员来处理具体的命令，返回值为命令的执行结果。

cmd_process 中会检测 cmd_tbl 的返回值，如果返回值为 CMD_RET_USAGE 的话就会调用 cmd_usage 函数输出命令的用法，其实就是输出 cmd_tbl_t 的 usage 成员变量。

12.3 bootm 启动 Linux 内核过程

12.3.1 images 全局变量

不管是 bootm 还是 bootz 命令，在启动 Linux 内核的时候都会用到一个重要的全局变量：images，images 在文件 include/image.h 中有如下定义：

```
414 extern bootm_headers_t images;
```

images 是 bootm_headers_t 类型的全局变量，bootm_headers_t 是个 boot 头结构体，在文件 include/image.h 中的定义如下(删除了一些条件编译代码)：

示例代码 12.3.1.1 bootm_headers 结构体

```
348 typedef struct bootm_headers {
349     /*
350      * Legacy os image header, if it is a multi component image
351      * then boot_get_ramdisk() and get_fdt() will attempt to get
352      * data from second and third component accordingly.
353      */
354     image_header_t    *legacy_hdr_os;     /* image header pointer */
355     image_header_t    legacy_hdr_os_copy; /* header copy */
356     ulong             legacy_hdr_valid;
```

```
......
#ifndef USE_HOSTCC
image_info_t os;                /* OS 镜像信息              */
   ulong       ep;              /* OS 入口点                */

   ulong      rd_start, rd_end; /* ramdisk 开始和结束位置   */

   char       *ft_addr;         /* 设备树地址               */
   ulong      ft_len;           /* 设备树长度               */

   ulong      initrd_start;     /* initrd 开始位置          */
   ulong      initrd_end;       /* initrd 结束位置          */
   ulong      cmdline_start;    /* cmdline 开始位置         */
   ulong      cmdline_end;      /* cmdline 结束位置         */
   bd_t       *kbd;
#endif

    int        verify;     /* env_get("verify")[0] != 'n' */

#define BOOTM_STATE_START       (0x00000001)
#define BOOTM_STATE_FINDOS      (0x00000002)
#define BOOTM_STATE_FINDOTHER   (0x00000004)
#define BOOTM_STATE_LOADOS      (0x00000008)
#define BOOTM_STATE_RAMDISK     (0x00000010)
#define BOOTM_STATE_FDT         (0x00000020)
#define BOOTM_STATE_OS_CMDLINE  (0x00000040)
#define BOOTM_STATE_OS_BD_T     (0x00000080)
#define BOOTM_STATE_OS_PREP     (0x00000100)
#define BOOTM_STATE_OS_FAKE_GO  (0x00000200)
#define BOOTM_STATE_OS_GO       (0x00000400)
    int     state;

#ifdef CONFIG_LMB
    struct lmb  lmb;        /* for memory mgmt */
#endif
} bootm_headers_t;
```

第 379 行的 os 成员变量是 image_info_t 类型的，为系统镜像信息。

第 396~406 行这 11 个宏定义表示 BOOT 的不同阶段。

接下来看一下结构体 image_info_t，也就是系统镜像信息结构体，此结构体在文件 include/image.h 中的定义如下：

示例代码 12.3.1.2 image_info_t 结构体

```
typedef struct image_info {
    ulong   start, end;             /* blob 开始和结束位置      */
```

```
338     ulong   image_start, image_len; /* 镜像起始地址(包括 blob)和长度*/
339     ulong   load;                   /* 系统镜像加载地址          */
340     uint8_t comp, type, os;         /* 镜像压缩、类型，OS 类型   */
341     uint8_t     arch;               /* CPU 架构                  */
342 } image_info_t;
```

全局变量 images 会在 bootm 命令的执行中频繁使用到，相当于 Linux 内核启动的“灵魂”。

12.3.2 do_bootm 函数

bootm 命令的执行函数为 do_bootm，在文件 cmd/bootm.c 中有如下定义(省略条件判断)：

示例代码 12.3.2.1 do_bootm 函数

```
92  int do_bootm(cmd_tbl_t *cmdtp, int flag, int argc,
                 char * const argv[])
93  {
.....
108     /* determine if we have a sub command */
109     argc--; argv++;
110     if (argc > 0) {
111         char *endp;
112
113         simple_strtoul(argv[0], &endp, 16);
114         /* endp pointing to NULL means that argv[0] was just a
115          * valid number, pass it along to the normal bootm processing
116          *
117          * If endp is ':' or '#' assume a FIT identifier so pass
118          * along for normal processing.
119          *
120          * Right now we assume the first arg should never be '-'
121          */
122         if ((*endp != 0) && (*endp != ':') && (*endp != '#'))
123             return do_bootm_subcommand(cmdtp, flag, argc, argv);
124     }
125
126     return do_bootm_states(cmdtp, flag, argc, argv,
127         BOOTM_STATE_START | BOOTM_STATE_FINDOS |
128         BOOTM_STATE_FINDOTHER | BOOTM_STATE_LOADOS |
129 #ifdef CONFIG_SYS_BOOT_RAMDISK_HIGH
130         BOOTM_STATE_RAMDISK |
131 #endif
132 #if defined(CONFIG_PPC) || defined(CONFIG_MIPS)
133         BOOTM_STATE_OS_CMDLINE |
134 #endif
135         BOOTM_STATE_OS_PREP | BOOTM_STATE_OS_FAKE_GO |
136         BOOTM_STATE_OS_GO, &images, 1);
```

```
137 }
```

第 109~124 行，主要作用是解析 bootm 的命令参数和检查有没有子命令，在第 113 行，simple_strtoul 函数将字符串转换成 unsigend long 型数据，endp 指向尾字符。第 122 行，判断是否有子命令，一般不会有，所以这里也是不会运行的。

第 126 行，调用函数 do_bootm_states 来执行不同的 BOOT 阶段，这里要执行的 BOOT 阶段有：BOOTM_STATE_START、BOOTM_STATE_FINDOS、BOOTM_STATE_FINDOTHER、BOOTM_STATE_LOADOS 、 BOOTM_STATE_RAMDISK 、 BOOTM_STATE_OS_PREP 、BOOTM_STATE_OS_FAKE_GO 和 BOOTM_STATE_OS_GO。

12.3.3 do_bootm_states 函数

do_bootm 最后调用的就是函数 do_bootm_states，此函数定义在文件 common/bootm.c 中，函数代码如下：

示例代码 12.3.3.1 do_bootm_states 函数

```
521 int do_bootm_states(cmd_tbl_t *cmdtp, int flag, int argc,
              char * const argv[], int states, bootm_headers_t *images,
522           int boot_progress)
523 {
524     boot_os_fn *boot_fn;
525     ulong iflag = 0;
526     int ret = 0, need_boot_fn;
527
528     images->state |= states;
529
530     /*
531      * Work through the states and see how far we get. We stop on
532      * any error.
533      */
534     if (states & BOOTM_STATE_START)
535         ret = bootm_start(cmdtp, flag, argc, argv);
536
537     if (!ret && (states & BOOTM_STATE_FINDOS))
538         ret = bootm_find_os(cmdtp, flag, argc, argv);
539
540     if (!ret && (states & BOOTM_STATE_FINDOTHER))
541         ret = bootm_find_other(cmdtp, flag, argc, argv);
542
543     /* Load the OS */
544     if (!ret && (states & BOOTM_STATE_LOADOS)) {
545         iflag = bootm_disable_interrupts();
546         ret = bootm_load_os(images, 0);
547         if (ret && ret != BOOTM_ERR_OVERLAP)
548             goto err;
549         else if (ret == BOOTM_ERR_OVERLAP)
```

```
550             ret = 0;
551     }
552
553     /* Relocate the ramdisk */
554 #ifdef CONFIG_SYS_BOOT_RAMDISK_HIGH
555     if (!ret && (states & BOOTM_STATE_RAMDISK)) {
556         ulong rd_len = images->rd_end - images->rd_start;
557
558         ret = boot_ramdisk_high(&images->lmb, images->rd_start,
559             rd_len, &images->initrd_start, &images->initrd_end);
560         if (!ret) {
561             env_set_hex("initrd_start", images->initrd_start);
562             env_set_hex("initrd_end", images->initrd_end);
563         }
564     }
565 #endif
566 #if IMAGE_ENABLE_OF_LIBFDT && defined(CONFIG_LMB)
567     if (!ret && (states & BOOTM_STATE_FDT)) {
568         boot_fdt_add_mem_rsv_regions(&images->lmb, images->ft_addr);
569         ret = boot_relocate_fdt(&images->lmb, &images->ft_addr,
570                     &images->ft_len);
571     }
572 #endif
573
574     /* From now on, we need the OS boot function */
575     if (ret)
576         return ret;
577     boot_fn = bootm_os_get_boot_func(images->os.os);
578     need_boot_fn = states & (BOOTM_STATE_OS_CMDLINE |
579             BOOTM_STATE_OS_BD_T | BOOTM_STATE_OS_PREP |
580             BOOTM_STATE_OS_FAKE_GO | BOOTM_STATE_OS_GO);
581     if (boot_fn == NULL && need_boot_fn) {
582         if (iflag)
583             enable_interrupts();
584         printf("ERROR: booting os '%s' (%d) is not supported\n",
585                genimg_get_os_name(images->os.os), images->os.os);
586         bootstage_error(BOOTSTAGE_ID_CHECK_BOOT_OS);
587         return 1;
588     }
589
590
591     /* Call various other states that are not generally used */
592     if (!ret && (states & BOOTM_STATE_OS_CMDLINE))
```

```
593         ret = boot_fn(BOOTM_STATE_OS_CMDLINE, argc, argv, images);
594     if (!ret && (states & BOOTM_STATE_OS_BD_T))
595         ret = boot_fn(BOOTM_STATE_OS_BD_T, argc, argv, images);
596     if (!ret && (states & BOOTM_STATE_OS_PREP)) {
597 #if defined(CONFIG_SILENT_CONSOLE)
          && !defined(CONFIG_SILENT_U_BOOT_ONLY)
598         if (images->os.os == IH_OS_LINUX)
599             fixup_silent_linux();
600 #endif
601         ret = boot_fn(BOOTM_STATE_OS_PREP, argc, argv, images);
602     }
603
604 #ifdef CONFIG_TRACE
605     /* Pretend to run the OS, then run a user command */
606     if (!ret && (states & BOOTM_STATE_OS_FAKE_GO)) {
607         char *cmd_list = env_get("fakegocmd");
608
609         ret = boot_selected_os(argc, argv, BOOTM_STATE_OS_FAKE_GO,
610                images, boot_fn);
611         if (!ret && cmd_list)
612             ret = run_command_list(cmd_list, -1, flag);
613     }
614 #endif
615
616     /* Check for unsupported subcommand. */
617     if (ret) {
618         puts("subcommand not supported\n");
619         return ret;
620     }
621
622     /* Now run the OS! We hope this doesn't return */
623     if (!ret && (states & BOOTM_STATE_OS_GO))
624         ret = boot_selected_os(argc, argv, BOOTM_STATE_OS_GO,
625                images, boot_fn);
626
627     /* Deal with any fallout */
628 err:
629     if (iflag)
630         enable_interrupts();
631
632     if (ret == BOOTM_ERR_UNIMPLEMENTED)
633         bootstage_error(BOOTSTAGE_ID_DECOMP_UNIMPL);
634     else if (ret == BOOTM_ERR_RESET)
```

```
635         do_reset(cmdtp, flag, argc, argv);
636
637     return ret;
638 }
```

函数 do_bootm_states 根据不同的 BOOT 状态执行不同的代码段，通过如下代码来判断 BOOT 状态：

```
states & BOOTM_STATE_XXX
```

根据示例代码 12.3.2.1 里的 126 行，有 8 个阶段，分别为：

```
BOOTM_STATE_START
BOOTM_STATE_FINDOS
BOOTM_STATE_FINDOTHER
BOOTM_STATE_LOADOS
BOOTM_STATE_RAMDISK
BOOTM_STATE_OS_PREP
BOOTM_STATE_OS_FAKE_GO
BOOTM_STATE_OS_GO。
```

1、第一阶段 BOOTM_STATE_START

示例代码 12.3.3.1 里 534~535 行，就是第一阶段 BOOTM_STATE_START，运行 bootm_start 函数，函数在 common/bootm.c 文件里，打开此文件找到如下代码：

示例代码 12.3.3.2 bootm_start 函数

```
68  static int bootm_start(cmd_tbl_t *cmdtp, int flag, int argc,
69                 char * const argv[])
70  {
71      memset((void *)&images, 0, sizeof(images));
72      images.verify = env_get_yesno("verify");
73
74      boot_start_lmb(&images);
75
76      bootstage_mark_name(BOOTSTAGE_ID_BOOTM_START, "bootm_start");
77      images.state = BOOTM_STATE_START;
78
79      return 0;
80  }
```

第 71 行，清空 images 结构体。

第 72 行，获取 uboot 的环境变量 verify 的值，并赋值给 images 的 verify 成员。

第 74 行，执行 boot_start_lmb 函数，如果定义了宏 CONFIG_LMB，此函数就会初始化 images 的 lmb 成员，没有定义的话此函数为空函数。STM32MP157 没有定义宏 CONFIG_LMB，因此此函数为空函数。

第 76 行，执行 bootstage_mark_name 函数，主要是记录启动阶段的名字。

第 77 行，设置 images 的阶段为 BOOTM_STATE_START。

2、第二阶段 BOOTM_STATE_FINDOS

示例代码 12.3.3.1 里 537~538 行，就是第二阶段 BOOTM_STATE_FINDOS，运行

bootm_find_os 函数，函数在 common/bootm.c 文件里，打开此文件找到如下代码(省略了条件相关的判断代码)：

示例代码 12.3.3.3 bootm_find_os 函数

```
1  static int bootm_find_os(cmd_tbl_t *cmdtp, int flag, int argc,
2              char * const argv[])
3  {
4      const void *os_hdr;
5      bool ep_found = false;
6      int ret;
7
8      /* get kernel image header, start address and length */
9      os_hdr = boot_get_kernel(cmdtp, flag, argc, argv,
10             &images, &images.os.image_start, &images.os.image_len);
11     if (images.os.image_len == 0) {
12         puts("ERROR: can't get kernel image!\n");
13         return 1;
14     }
15
16     /* get image parameters */
17     switch (genimg_get_format(os_hdr)) {
18     case IMAGE_FORMAT_LEGACY:
19         images.os.type = image_get_type(os_hdr);
20         images.os.comp = image_get_comp(os_hdr);
21         images.os.os = image_get_os(os_hdr);
22
23         images.os.end = image_get_image_end(os_hdr);
24         images.os.load = image_get_load(os_hdr);
25         images.os.arch = image_get_arch(os_hdr);
26         break;
27     default:
28         puts("ERROR: unknown image format type!\n");
29         return 1;
30     }
31
32     if (images.legacy_hdr_valid) {
33         images.ep = image_get_ep(&images.legacy_hdr_os_copy);
34     }
35
36
37     images.os.start = map_to_sysmem(os_hdr);
38
39     return 0;
40 }
```

第 9 行，boot_get_kernel 函数很复杂，这里就不具体分析了。它的主要作用是：首先会根据 bootm 传过来的参数去获取 uImage(镜像)的存储地址，如果 bootm 没有参数就使用全局变量 load_addr。如果使能了宏 LEGACY_IMAGE_FORMAT，那么就会按照老的格式获取 Linux 镜像。STM32MP157 使能了宏 LEGACY_IMAGE_FORMAT，所以会按照老的格式得到镜像，并且会输出相应的提示信息，如图 12.3.3.1 所示：

```
## Booting kernel from Legacy Image at c2000000 ...
```

图 12.3.3.1 kernel 使用老格式

boot_get_kernel 会调用 image_get_kernel 函数进行 kernel 格式校验，校验老格式 kernel 的完整性并且返回 kernel 的镜像头。校验过程中会输出 kernel 相关信息以及校验结果，如图 12.3.3.2 所示：

```
Image Name:   Linux-5.4.31
Created:      2021-10-02   8:41:13 UTC
Image Type:   ARM Linux Kernel Image (uncompressed)
Data Size:    7643528 Bytes = 7.3 MiB
Load Address: c2000040
Entry Point:  c2000040
Verifying Checksum ... OK
```

图 12.3.3.2 kernel 信息以及校验结果

获取到内核镜像以后，将镜像的起始地址赋值给参数 os_data，镜像的长度赋值给参数 os_len。最后将镜像头部拷贝到 images 的 legacy_hdr_os_copy 成员中，并且设置 legacy_hdr_valid 成员为 1。

第 17 行，使用 genimg_get_format 函数来检查镜像的类型，STM32MP157 的镜像为老的格式类型(legacy)，所以 genimg_get_format 函数会返回 IMAGE_FORMAT_LEGACY，表示这是一个老的类型。

第 19~25 行，使用 image_get_XXXX 相关的函数去获取 image 的 type(内核的类型)、comp（内核压缩方式）、os(内核是什么操作系统)、end、load（加载地址）和 arch（芯片架构），然后将其这些信息赋值给 images 的对应成员变量。

第 37 行，把内核的起始地址赋值给全局变量 images.os.start。

到这里 bootm_fimd_os 函数结束，用一句话概括就是：主要工作是解析 image，获取一些基本参数信息赋值到 images 全局结构体中。

3、第三阶段 BOOTM_STATE_FINDOTHER

示例代码 12.3.3.1 里 540~541 行，就是第三阶段 BOOTM_STATE_FINDOTHER，运行 bootm_find_other 函数，函数在 common/bootm.c 文件里，打开此文件找到如下代码：

示例代码 12.3.3.4 bootm_find_other 函数

```
286 static int bootm_find_other(cmd_tbl_t *cmdtp, int flag, int argc,
287                 char * const argv[])
288 {
289     if (((images.os.type == IH_TYPE_KERNEL) ||
290          (images.os.type == IH_TYPE_KERNEL_NOLOAD) ||
291          (images.os.type == IH_TYPE_MULTI)) &&
292         (images.os.os == IH_OS_LINUX ||
293          images.os.os == IH_OS_VXWORKS))
294         return bootm_find_images(flag, argc, argv);
```

```
295
296     return 0;
297 }
```

第 294 行，调用 bootm_find_images 函数来获取 ramdisk 或者设备树信息，STM32MP157 没用 ramdisk，因此 bootm_find_images 主要用来获取设备树，bootm_find_images 函数内容如下(省略掉条件编译部分代码)：

示例代码 12.3.3.5 bootm_find_images 函数

```
1  int bootm_find_images(int flag, int argc, char * const argv[])
2  {
3      int ret;
4
5      /* find ramdisk */
6      ret = boot_get_ramdisk(argc, argv, &images, IH_INITRD_ARCH,
7                  &images.rd_start, &images.rd_end);
8      if (ret) {
9          puts("Ramdisk image is corrupt or invalid\n");
10         return 1;
11     }
12
13
14     /* find flattened device tree */
15     ret = boot_get_fdt(flag, argc, argv, IH_ARCH_DEFAULT, &images,
16           &images.ft_addr, &images.ft_len);
17     if (ret) {
18         puts("Could not find a valid device tree\n");
19         return 1;
20     }
21     if (CONFIG_IS_ENABLED(CMD_FDT))
22         set_working_fdt_addr(map_to_sysmem(images.ft_addr));
23
24     return 0;
25 }
```

第 6 行，调用 boot_get_ramdisk 函数来获取 ramdisk，这里没有用到 ramdisk，所以这段代码无效。

第 15 行，调用 boot_get_fdt 函数来获取设备树，并且将设备树首地址赋值给 images 的 ft_addr 成员。

4、第四阶段 BOOTM_STATE_LOADOS

示例代码 12.3.3.1 里 544~551 行，就是第四阶段 BOOTM_STATE_LOADOS，第 545 行先使用 bootm_disable_interrupts 函数禁用中断，再运行 bootm_load_os 函数，函数在 common/bootm.c 文件里，打开此文件找到如下代码：

示例代码 12.3.3.6 bootm_load_os 函数

```
341 static int bootm_load_os(bootm_headers_t *images,
                               int boot_progress)
```

```
342 {
343     image_info_t os = images->os;
344     ulong load = os.load;
345     ulong load_end;
346     ulong blob_start = os.start;
347     ulong blob_end = os.end;
348     ulong image_start = os.image_start;
349     ulong image_len = os.image_len;
350     ulong flush_start = ALIGN_DOWN(load, ARCH_DMA_MINALIGN);
351     bool no_overlap;
352     void *load_buf, *image_buf;
353     int err;
354
355     load_buf = map_sysmem(load, 0);
356     image_buf = map_sysmem(os.image_start, image_len);
357     err = image_decomp(os.comp, load, os.image_start, os.type,
358             load_buf, image_buf, image_len,
359             CONFIG_SYS_BOOTM_LEN, &load_end);
360     if (err) {
361         err = handle_decomp_error(os.comp, load_end - load, err);
362         bootstage_error(BOOTSTAGE_ID_DECOMP_IMAGE);
363         return err;
364     }
365
366     flush_cache(flush_start, ALIGN(load_end, ARCH_DMA_MINALIGN) -
                 flush_start);
367
368     debug("   kernel loaded at 0x%08lx, end = 0x%08lx\n", load,
           load_end);
369     bootstage_mark(BOOTSTAGE_ID_KERNEL_LOADED);
370
371     no_overlap = (os.comp == IH_COMP_NONE && load == image_start);
372
373     if (!no_overlap && load < blob_end && load_end > blob_start) {
374         debug("images.os.start = 0x%lX, images.os.end = 0x%lx\n",
375               blob_start, blob_end);
376         debug("images.os.load = 0x%lx, load_end = 0x%lx\n", load,
377               load_end);
378
379         /* Check what type of image this is. */
380         if (images->legacy_hdr_valid) {
381             if (image_get_type(&images->legacy_hdr_os_copy)
382                     == IH_TYPE_MULTI)
```

```
383                 puts("WARNING: legacy format multi component image
                        overwritten\n");
384             return BOOTM_ERR_OVERLAP;
385         } else {
386             puts("ERROR: new format image overwritten - must RESET
                    the board to recover\n");
387             bootstage_error(BOOTSTAGE_ID_OVERWRITTEN);
388             return BOOTM_ERR_RESET;
389         }
390     }
391
392     lmb_reserve(&images->lmb, images->os.load, (load_end -
393                         images->os.load));
394     return 0;
395 }
```

第 357 行，调用 image_decomp 函数来解压内核。函数的参数都是在第二阶段里获取到参数 os.image_start = 0Xc2000000 和 load_buf=0xc2000040，os.image_start 是内核未解压时所在的地址，load_buf 是内核的启动地址也就是解压后内核所在的地址。

5、第五阶段 BOOTM_STATE_RAMDISK

示例代码 12.3.3.1 里 554~564 行，就是第五阶段 BOOTM_STATE_RAMDISK，这里没有用 ramdisk。

示例代码 12.3.3.1 里 577 行，非常重要！通过函数 bootm_os_get_boot_func 来查找系统启动函数，参数 images->os.os 就是系统类型，根据这个系统类型来选择对应的启动函数。示例代码 12.3.3.3 中的 bootm_find_os 函数会根据系统类型设置 images->os.os 的值为 IH_OS_LINUX。数 bootm_os_get_boot_func 函数返回值就是找到的系统启动函数，这里找到的 Linux 系统启动函数为 do_bootm_linux。关于 bootm_os_get_boot_func 函数查找系统启动函数的过程请参考 12.3.4 小节。因此 boot_fn=do_bootm_linux，后面执行 boot_fn 函数的地方实际上是执行的 do_bootm_linux 函数。

6、第六阶段 BOOTM_STATE_PREP

示例代码 12.3.3.1 里 596~602 行，就是第六阶段 BOOTM_STATE_PREP 主要是解析 bootargs 环境变量和设置内核的启动参数。

第 599 行，fixup_silent_linux 函数应该是重新设置 bootagrs 的环境变量，不知道有啥作用。

第 601 行，调用 boot_fn 函数，前面已经说了，boot_fn 为 do_bootm_linux，所以这里也就是运行 do_bootm_linux 函数。此处 do_bootm_linux 函数主要是调用 boot_prep_linux 函数去解析 bootargs 变量。

7、第七阶段 BOOTM_STATE_FAKE_GO

示例代码 12.3.3.1 里 606~613 行，就是第七阶段 BOOTM_STATE_FAKE_GO，代码不执行。

8、第八阶段 BOOTM_STATE_OS_GO

示例代码 12.3.3.1 里 623~625 行，就是第八阶段 BOOTM_STATE_OS_GO，第 624 行，运行 bootm_selected_os 函数，函数在 common/ bootm_os.c 文件里，打开此文件找到如下代码：

示例代码 12.3.3.7 boot_selected_os 函数

```
551 int boot_selected_os(int argc, char * const argv[], int state,
552             bootm_headers_t *images, boot_os_fn *boot_fn)
553 {
554     arch_preboot_os();
555     board_preboot_os();
556     boot_fn(state, argc, argv, images);
557 
558     /* Stand-alone may return when 'autostart' is 'no' */
559     if (images->os.type == IH_TYPE_STANDALONE ||
560         IS_ENABLED(CONFIG_SANDBOX) ||
561         state == BOOTM_STATE_OS_FAKE_GO) /* We expect to return */
562         return 0;
563     bootstage_error(BOOTSTAGE_ID_BOOT_OS_RETURNED);
564     debug("\n## Control returned to monitor - resetting...\n");
565 
566     return BOOTM_ERR_RESET;
567 }
```

第 554~555 行，这两个函数都是空函数，不执行。

重点是第 556 行，执行 boot_fn，也就是 do_bootm_linux 函数，注意 boot_fn 函数的第一个参数 state 为 BOOTM_STATE_OS_GO。稍后我们会详细讲解 do_bootm_linux 函数。

12.3.4 bootm_os_get_boot_func 函数

do_bootm_states 会调用 bootm_os_get_boot_func 来查找对应系统的启动函数，此函数定义在文件 common/bootm_os.c 中，函数内容如下：

示例代码 12.3.4.1 bootm_os_get_boot_func 函数

```
569 boot_os_fn *bootm_os_get_boot_func(int os)
570 {
571 #ifdef CONFIG_NEEDS_MANUAL_RELOC
572     static bool relocated;
573 
574     if (!relocated) {
575         int i;
576 
577         /* relocate boot function table */
578         for (i = 0; i < ARRAY_SIZE(boot_os); i++)
579             if (boot_os[i] != NULL)
580                 boot_os[i] += gd->reloc_off;
581 
582         relocated = true;
583     }
584 #endif
585     return boot_os[os];
586 }
```

第 571~584 行，这是一个宏定义，CONFIG_NEEDS_MANUAL_RELOC 宏没有定义，这里的代码就不会编译。

第 585 行，boot_os 是一个全局函数数组，数组里面存放着不同的系统对应的启动函数，boot_os 也定义在文件 common/bootm_os.c 中，如下所示：

示例代码 12.3.4.2 boot_os 数组

```
501 static boot_os_fn *boot_os[] = {
502     [IH_OS_U_BOOT] = do_bootm_standalone,
503 #ifdef CONFIG_BOOTM_LINUX
504     [IH_OS_LINUX] = do_bootm_linux,
505 #endif
......
534 #ifdef CONFIG_BOOTM_OPTEE
535     [IH_OS_TEE] = do_bootm_tee,
536 #endif
537 };
```

第 504 行，do_bootm_linux 就 Linux 系统对应的启动函数，可以看出查找方法很简单，就是读取数组中指定元素。

12.3.5 do_bootm_linux 函数

经过前面的分析，我们知道了 do_bootm_linux 就是最终启动 Linux 内核的函数，此函数定义在文件 arch/arm/lib/bootm.c，函数内容如下：

示例代码 12.3.5.1 do_bootm_linux 函数

```
421 int do_bootm_linux(int flag, int argc, char * const argv[],
422             bootm_headers_t *images)
423 {
424     /* No need for those on ARM */
425     if (flag & BOOTM_STATE_OS_BD_T || flag & BOOTM_STATE_OS_CMDLINE)
426         return -1;
427
428     if (flag & BOOTM_STATE_OS_PREP) {
429         boot_prep_linux(images);
430         return 0;
431     }
432
433     if (flag & (BOOTM_STATE_OS_GO | BOOTM_STATE_OS_FAKE_GO)) {
434         boot_jump_linux(images, flag);
435         return 0;
436     }
437
438     boot_prep_linux(images);
439     boot_jump_linux(images, flag);
440     return 0;
441 }
```

在我们八个阶段里，其中在第六阶段和第八阶段都回调 do_bootm_linux 函数，示例代码 12.3.3.1 中第 601 行和第 624 行。第六阶段传进的 flag 为 BOOTM_STATE_PREP，就会运行示例代码 12.3.3.10 里 429 行，boot_prep_linux 函数就是负责解析 bootargs 变量。第八阶段传进的 fiag 为 BOOTM_STATE_OS_GO，就会运行 434 行的 boot_jump_linux 函数，此函数定义在文件 arch/arm/lib/bootm.c 中，函数内容如下：

示例代码 12.3.5.2 boot_jump_linux 函数

```
331 static void boot_jump_linux(bootm_headers_t *images, int flag)
332 {
333 #ifdef CONFIG_ARM64
......
371 #else
372     unsigned long machid = gd->bd->bi_arch_number;
373     char *s;
374     void (*kernel_entry)(int zero, int arch, uint params);
375     unsigned long r2;
376     int fake = (flag & BOOTM_STATE_OS_FAKE_GO);
377
378     kernel_entry = (void (*)(int, int, uint))images->ep;
379 #ifdef CONFIG_CPU_V7M
380     ulong addr = (ulong)kernel_entry | 1;
381     kernel_entry = (void *)addr;
382 #endif
383     s = env_get("machid");
384     if (s) {
385         if (strict_strtoul(s, 16, &machid) < 0) {
386             debug("strict_strtoul failed!\n");
387             return;
388         }
389         printf("Using machid 0x%lx from environment\n", machid);
390     }
391
392     debug("## Transferring control to Linux (at address %08lx)" \
393         "...\n", (ulong) kernel_entry);
394     bootstage_mark(BOOTSTAGE_ID_RUN_OS);
395     announce_and_cleanup(fake);
396
397     if (IMAGE_ENABLE_OF_LIBFDT && images->ft_len)
398         r2 = (unsigned long)images->ft_addr;
399     else
400         r2 = gd->bd->bi_boot_params;
......
410             kernel_entry(0, machid, r2);
411     }
```

```
412 #endif
```

第 333~371 行是 64 位 ARM 芯片对应的代码，Cortex-A7 是 32 位芯片，因此用不到。

第 372 行，变量 machid 保存机器 ID，如果不使用设备树的话这个机器 ID 会被传递给 Linux 内核，Linux 内核会在自己的机器 ID 列表里面查找是否存在与 uboot 传递进来的 machid 匹配的项目，如果存在就说明 Linux 内核支持这个机器，那么 Linux 就会启动！如果使用设备树的话这个 machid 就无效了，设备树存有一个“兼容性”这个属性，Linux 内核会比较“兼容性”属性的值(字符串)来查看是否支持这个机器。

第 374 行，函数 kernel_entry，看名字“内核_进入”，说明此函数是进入 Linux 内核的，也就是最终的大 boos!!此函数有三个参数：zero，arch，params，第一个参数 zero 要为 0；第二个参数为机器 ID；第三个参数 ATAGS 或者设备树(DTB)首地址，ATAGS 是传统的方法，用于传递一些命令行信息啥的，如果使用设备树的话就要传递设备树(DTB)。

第 378 行，获取 kernel_entry 函数，函数 kernel_entry 并不是 uboot 定义的，而是 Linux 内核定义的，Linux 内核镜像文件的第一行代码就是函数 kernel_entry，而 images->ep 保存着 Linux 内核镜像的起始地址，起始地址保存的正是 Linux 内核第一行代码！

第 395 行，调用函数 announce_and_cleanup 来打印一些信息并做一些清理工作，此函数定义在文件 arch/arm/lib/bootm.c 中，函数内容如下：

示例代码 12.3.5.3 announce_end_cleanup 函数

```
94  static void announce_and_cleanup(int fake)
95  {
96      bootstage_mark_name(BOOTSTAGE_ID_BOOTM_HANDOFF, "start_kernel");
......
108     board_quiesce_devices();
109
110     printf("\nStarting kernel ...%s\n\n", fake ?
111         "(fake run for tracing)" : "");
112     /*
113      * Call remove function of all devices with a removal flag set.
114      * This may be useful for last-stage operations, like cancelling
115      * of DMA operation or releasing device internal buffers.
116      */
117     dm_remove_devices_flags(DM_REMOVE_ACTIVE_ALL);
118
119     cleanup_before_linux();
120 }
```

第 110 行，在启动 Linux 之前输出“Starting kernel ...”信息，如图 12.3.5.1 所示：

```
Starting kernel ...
```

图 12.3.5.1 系统启动提示信息

第 119 行调用 cleanup_before_linux 函数做一些清理工作。

继续回到示例代码 12.3.5.2 的函数 boot_jump_linux，第 397~400 行是设置寄存器 r2 的值。为什么要设置 r2 的值呢？Linux 内核一开始是汇编代码，因此函数 kernel_entry 就是个汇编函数。向汇编函数传递参数要使用 r0、r1 和 r2(参数数量不超过 3 个的时候)，所以 r2 寄存器就是函数 kernel_entry 的第三个参数。

第 398 行，如果使用设备树的话，r2 应该是设备树的起始地址，而设备树地址保存在 images 的 ftd_addr 成员变量中。

第 400 行，如果不使用设备树的话，r2 应该是 uboot 传递给 Linux 的参数起始地址，也就是环境变量 bootargs 的值，

第 410 行，调用 kernel_entry 函数进入 Linux 内核，此行将一去不复返，uboot 的使命也就完成了，它可以安息了！

第十三章 U-boot 移植

uboot 的移植并不是说我们完完全全的从零开始将 uboot 移植到我们现在所使用的开发板或者开发平台上。这个对于我们来说基本是不可能的，这个工作一般是半导体厂商做的，半导体厂商负责将 uboot 移植到他们的芯片上，因此半导体厂商都会自己做一个开发板，这个开发板就叫做原厂开发板，比如大家学习 STM32 的时候听说过的 discover 开发板就是 ST 自己做的。半导体厂商将 uboot 移植到自己的原厂开发板上，测试好以后就会将这个 uboot 发布出去，这就是大家常说的原厂 BSP 包。我们一般做产品的时候就会参考原厂的开发板做硬件，然后在原厂提供的 BSP 包上做修改，将 uboot 或者 linux kernel 移植到我们的硬件上。这就是 uboot 移植的一般流程：

1、在 uboot 中找到参考的开发平台，一般是原厂的开发板。

2、参考原厂开发板移植 uboot 到我们所使用的开发板上。

正点原子 STM32MP157 开发板参考的是 ST 官方的 STM32MP157 EVK 开发板做的硬件，因此我们在移植 uboot 的时候就可以以 ST 官方的 STM32MP157 EVK 开发板为蓝本。

13.1 ST 官方的 U-boot 编译测试

13.1.1 ST 官方 uboot 源码打补丁

1、获取到 ST 官方 uboot 源码

首先肯定要获取到 ST 官方的 uboot 源码，这个已经在 6.1.1 小节获取到了，进入到对应的 uboot 源码目录，命令如下：

```
cd /home/zuozhongkai/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources
/arm-ostl-linux-gnueabi/u-boot-stm32mp-2020.01-r0          //进入 ST 官方 uboot 源码
```

ST 官方 uboot 源码如图 13.1.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm
-ostl-linux-gnueabi/u-boot-stm32mp-2020.01-r0$ ls
0001-ARM-v2020.01-stm32mp-r1-MACHINE.patch
0002-ARM-v2020.01-stm32mp-r1-BOARD.patch
0003-ARM-v2020.01-stm32mp-r1-MISC-DRIVERS.patch
0004-ARM-v2020.01-stm32mp-r1-DEVICETREE.patch
0005-ARM-v2020.01-stm32mp-r1-CONFIG.patch
0099-Add-external-var-to-allow-build-of-new-devicetree-fi.patch
Makefile.sdk
README.HOW_TO.txt
series
u-boot-stm32mp-2020.01-r0.tar.gz
```

图 13.1.1.1 ST 官方 uboot 源码

从图 13.1.1.1 可以看出，ST 官方的 uboot 源码跟 TF-A 的源码包文件夹内容格式基本一样，包括了补丁文件.patch、Makefile.sdk 和 uboot 源码压缩包，只是 uboot 有多个.patch 补丁文件，打补丁的时候这些.patch 都要用到。

下一步肯定是解压图 13.1.1.1 中 u-boot-stm32mp-2020.01-r0.tar.gz 这个真正的源码包压缩包，解压命令如下：

```
tar -vxf u-boot-stm32mp-2020.01-r0.tar.gz
```

解压完成以后就会得到一个名为“u-boot-stm32mp-2020.01”的 uboot 源码文件夹，如图 13.1.1.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm
-ostl-linux-gnueabi/u-boot-stm32mp-2020.01-r0$ ls
0001-ARM-v2020.01-stm32mp-r1-MACHINE.patch
0002-ARM-v2020.01-stm32mp-r1-BOARD.patch
0003-ARM-v2020.01-stm32mp-r1-MISC-DRIVERS.patch
0004-ARM-v2020.01-stm32mp-r1-DEVICETREE.patch
0005-ARM-v2020.01-stm32mp-r1-CONFIG.patch
0099-Add-external-var-to-allow-build-of-new-devicetree-fi.patch
Makefile.sdk
README.HOW_TO.txt                    解压得到的uboot源码文件夹
series
u-boot-stm32mp-2020.01
u-boot-stm32mp-2020.01-r0.tar.gz
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm
```

图 13.1.1.2 解压得到 uboot 源码

2、打补丁

上面已经解压出来了 uboot 的源码文件，接下来就要对其打补丁，进入到上面解压出来的 u-boot-stm32mp-2020.01 目录，然后执行相应的打补丁命令：

```
cd u-boot-stm32mp-2020.01/                    //进入 uboot 源码目录
for p in `ls -1 ../*.patch`;do patch -p1 < $p;done    //打补丁
```

打补丁结果如图 13.1.1.3 所示:

```
patching file arch/arm/dts/stm32mp15xxaa-pinctrl.dtsi
patching file arch/arm/dts/stm32mp15xxab-pinctrl.dtsi
patching file arch/arm/dts/stm32mp15xxac-pinctrl.dtsi
patching file arch/arm/dts/stm32mp15xxad-pinctrl.dtsi
patching file configs/stm32mp15_basic_defconfig
patching file configs/stm32mp15_dhcom_basic_defconfig
patching file configs/stm32mp15_optee_defconfig
patching file configs/stm32mp15_trusted_defconfig
patching file arch/arm/dts/Makefile
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm
-ostl-linux-gnueabi/u-boot-stm32mp-2020.01-r0/u-boot-stm32mp-2020.01$
```

图 13.1.1.3 打补丁结果

打完补丁以后的 u-boot-stm32mp-2020.01 目录就是我们要移植的 uboot 源码，但是图 13.1.1.3 中的 u-boot-stm32mp-2020.01 目录路径有点长，不适合阅读和编译。所以我们新建一个名为“my_uboot”的目录来保存我们要移植的 uboot 源码，然后将 ST 官方的 uboot 源码拷贝到“my_uboot”目录下，命令如下：

```
cd /home/zuozhongkai/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources
/arm-ostl-linux-gnueabi/u-boot-stm32mp-2020.01-r0/u-boot-stm32mp-2020.01
cp * /home/zuozhongkai/linux/atk-mp1/uboot/my_uboot/ -rf      //拷贝到 my_uboot 目录下
```

拷贝完成以后的 my_uboot 目录如图 13.1.1.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/my_uboot$ ls
api     common          disk     env       Kbuild    MAINTAINERS  README
arch    config.mk       doc      examples  Kconfig   Makefile     scripts
board   configs         drivers  fs        lib       net          test
cmd     CONTRIBUTING.md dts      include   Licenses  post         tools
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/my_uboot$
```

图 13.1.1.4 拷贝过来的 uboot 源码

最后就是创建 vscode 工程，方便我们移植和阅读。

13.1.2 编译 ST 官方 uboot 源码

上一小节已经准备好了 ST 官方 uboot 源码，存放到了 my_uboot 目录下，本小节我们来编译一下这个 uboot 源码。

1、修改 Makefile

和 10.2.1 小节一样，首先修改一下 uboot 源码的 Makefile 文件，也可以不修改，但是在编译的时候需要多输入一些参数，为了偷懒，还是修改一下。修改方法已经在 10.2.1 小节讲解过了，就是在 Makefile 文件里面添加 ARCH 和 CROSS_COMPILE 这两个变量的值，如图 10.1.2.1 所示：

```
265
266  ARCH = arm
267  CROSS_COMPILE = arm-none-linux-gnueabihf-
268
269  KCONFIG_CONFIG  ?= .config
270  export KCONFIG_CONFIG
```

设置ARCH和CROSS_COMPILE的值

图 10.1.2.1 设置 ARCH 和 CROSS_COMPILE 变量值

ST 官方 uboot 肯定适配了官方的 STM32MP1 EVK 开发板，我们就编译 EVK 开发板对应的 uboot，编译完成以后将得到的 uboot 可执行文件烧写到正点原子的 STM32MP1 开发板中，看看能不能运行，不能的话就要修改 uboot 相应的文件，这就是 uboot 的移植。编译命令如下：

```
make stm32mp15_trusted_defconfig
make DEVICE_TREE=stm32mp157d-ev1 all -j8
```

编译完成以后如图 13.1.2.2 所示：

```
  DTC     arch/arm/dts/stm32mp157c-ev1.dtb
  DTC     arch/arm/dts/stm32mp157d-dk1.dtb
  DTC     arch/arm/dts/stm32mp157d-ed1.dtb
  DTC     arch/arm/dts/stm32mp157d-ev1.dtb
  DTC     arch/arm/dts/stm32mp157f-dk2.dtb
  DTC     arch/arm/dts/stm32mp157f-ed1.dtb
  DTC     arch/arm/dts/stm32mp157f-ev1.dtb
  DTC     arch/arm/dts/stm32mp15xx-dhcom-pdk2.dtb
  SHIPPED dts/dt.dtb
  FDTGREP dts/dt-spl.dtb
  CAT     u-boot-dtb.bin
  COPY    u-boot.dtb
  COPY    u-boot.bin
  MKIMAGE u-boot.stm32
  CFGCHK  u-boot.cfg
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/my_uboot$
```

图 13.1.2.2 uboot 编译成功

编译完成以后就会得到 uboot 可执行文件，如图 13.1.2.3 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/my_uboot$ ls
api              doc       lib                   System.map          u-boot.lds
arch             drivers   Licenses              test                u-boot.map
board            dts       MAINTAINERS           tools               u-boot-nodtb.bin
cmd              env       Makefile              u-boot              u-boot.srec
common           examples  myuboot.code-workspace u-boot.bin        u-boot.stm32
config.mk        fs        net                   u-boot.cfg          u-boot.stm32.log
configs          include   post                  u-boot.cfg.configs  u-boot.sym
CONTRIBUTING.md  Kbuild    README                u-boot.dtb
disk             Kconfig   scripts               u-boot-dtb.bin
zuozhongkai@ubuntu:~/linux/atk-mp1/uboot/my_uboot$
```

图 13.1.2.3 uboot 可执行文件

从图 13.1.2.3 可以看出，uboot 编译成功，生成了 u-boot.bin 和 u-boot.stm32，u-boot.bin 包含了设备树(dtb)，也就是将 uboot 镜像和设备树打包在了一起。其中 u-boot.stm32 是在 u-boot.bin 前面添加了 256 字节头部信息的可执行文件，是要烧写到开发板里面的。

13.1.3 烧写测试

上一小节我们编译出了 ST 官方 EVK 开发板对应的 u-boot.stm32，本节我们将其烧写到我们的开发板中，看看能不能启动。这里烧写就要注意一下了，不能直接用图 13.1.2.3 小节中的 u-boot.stm32 文件替换以前 images 目录下的 u-boot.stm32。前面说了 STM23CubeProgrammer 烧写原理就是先向开发板的 DDR 里面下载一个完整的 uboot 进去，然后用这个 uboot 来烧写系统。所以我们必须保证这个下载到 DDR 里面的 uboot 是工作正常的，但是我们刚刚编译出来的 uboot 可执行文件肯定是有问题的，所以下载到 DDR 中的 uboot 必须用正点原子提供的，烧写到 EMMC 里面的是我们刚刚编译的，这两个 uboot 要区分开。

将图 13.1.2.3 中的 u-boot.stm32 重命名为 my-u-boot.stm32，一定要重命名！然后将重命名以后的 my-u-boot.stm32 放到 images 目录下。我们还需要修改一下 Flashlayouts 文件，修改一下烧写到 EMMC 里面的 uboot 名字，如图 13.1.3.1 所示：

tf-a.tsv

```
#Opt    Id      Name        Type    Device      Offset  Binary
-       0x01    fsbl1-boot  Binary  none        0x0     tf-a-stm32mp157d-atk-serialboot.stm32
-       0x03    ssbl-boot   Binary  none        0x0     u-boot.stm32
P       0x04    fsbl1       Binary  mmc1        boot1   tf-a-stm32mp157d-atk-trusted.stm32
P       0x05    fsbl2       Binary  mmc1        boot2   tf-a-stm32mp157d-atk-trusted.stm32
P       0x06    ssbl        Binary  mmc1        0x00080000  my-u-boot.stm32
```

图 13.1.3.1 修改后的 Flashlayouts 文件

一切准备好以后就可以烧写了，烧写完成以后从 EMMC 启动，此时会发现，开发板一直在重启！其中 uboot 的启动过程如图 13.1.3.2 所示：

```
U-Boot 2020.01-stm32mp-r1 (Dec 01 2020 - 10:33:15 +0800)

CPU: STM32MP157DAA Rev.Z
Model: STMicroelectronics STM32MP157D eval daughter on eval mother
Board: stm32mp1 in trusted mode (st,stm32mp157d-ev1)
DRAM:  1 GiB
Clocks:
- MPU : 800 MHz
- MCU : 208.878 MHz
- AXI : 266.500 MHz
- PER : 24 MHz
- DDR : 533 MHz
STM32Programmer key pressed, entering download mode...
stpmic1_read: failed to read register 0x23 : -5stpmic1_read: failed to read register 0x23 : -5stpmic1_r
ead: failed to read register 0x23 : -5stpmic1_read: failed to read register 0x23 : -5vdd enable failed:
 -5
stpmic1_read: failed to read register 0x21 : -5stpmic1_read: failed to read register 0x22 : -5stpmic1_r
ead: failed to read register 0x23 : -5stpmic1_read: failed to read register 0x29 : -5stpmic1_read: fail
ed to read register 0x24 : -5stpmic1_read: failed to read register 0x24 : -5WDT:   Started with servici
ng (32s timeout)
NAND:  0 MiB
MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
Loading Environment from MMC... stpmic1_read: failed to read register 0x23 : -5stpmic1_read: failed to
read register 0x23 : -5stpmic1_read: failed to read register 0x23 : -5stpmic1_read: failed to read regi
ster 0x23 : -5Error enabling VMMC supply
*** Warning - No block device, using default environment
```

图 13.1.3.2 uboot 启动过程

从图 13.1.3.2 可以看出一下几个重要的信息：

①、uboot 能运行，也就是说 ST 官方 EVK 开发板的 uboot 可以直接在正点原子的开发板上运行，但是运行会出错！

②、uboot 版本为 2020.01，编译日期为 2020 年 12 月 1 日，10:33:15，说明就是刚刚我们自己编译的 uboot。

②、出现了“stpmic1_read: failed to read register”错误，前面讲解 TF-A 的时候已经说了，ST 官方 EVK 开发板使用了电源管理芯片 STPMIC1A，所以 uboot 运行的时候会初始化这个 PMIC 芯片，但是正点原子开发板并没有使用这个 PMIC 芯片，所以就会报 STPMIC 错误！

接下来就是一步步的修改 uboot，至到其正常工作，也就是所谓的 uboot 移植。

13.2 在 U-boot 中添加自己的开发板

13.2.1 创建默认配置文件

首先创建自己所使用开发板对应的默认配置文件，我们在第一次编译 uboot 的时候先执行：

```
make stm32mp15_trusted_defconfig
```

上面命令的意思就是先使用默认配置文件配置一下 uboot，stm32mp15_trusted_defconfig 这个文件保存了默认配置选项。默认配置文件创建方法很简单，既然正点原子开发板参考了 ST 官方的 EVK 开发板，那么默认配置文件也可以直接参考官方的 EVK 开发板。在 uboot 的源码目录下，运行以下命令：

```
cd configs                //进入 uboot 的 configs 目录
cp stm32mp15_trusted_defconfig stm32mp15_atk_trusted_defconfig //拷贝
```

此时在 uboot 的 configs 目录下就存在一个名为“stm32mp15_atk_trusted_defconfig”的默认配置文件，这个默认配置文件就是给我们开发板使用的。

13.2.2 创建默认配置设备树

另外我们还要创建自己所使用开发板对应的设备树，方法也很简单，直接复制 ST 官方 EVK 开发板对应的设备树，在 uboot 的源码目录下，运行以下命令：

```
cd arch/arm/dts/                                    //进入 uboot 设备树目录
cp stm32mp157d-ed1.dts stm32mp157d-atk.dts    //复制.dts
cp stm32mp15xx-edx.dtsi stm32mp157d-atk.dtsi //复制.dtsi
cp stm32mp157a-ed1-u-boot.dtsi stm32mp157d-atk-u-boot.dtsi  //复制.dtsi
```

打开 stm32mp157d-atk.dts 文件，要修改一下其中的一个头文件引用，stm32mp157d-atk.dts 默认头文件引用如下：

示例代码 13.2.2.1 stm32mp157d-atk.dts 文件

```
1  // SPDX-License-Identifier: (GPL-2.0+ OR BSD-3-Clause)
2  /*
3   * Copyright (C) STMicroelectronics 2019 - All Rights Reserved
4   * Author: Alexandre Torgue <alexandre.torgue@st.com> for
      STMicroelectronics.
5   */
6  /dts-v1/;
7
8  #include "stm32mp157.dtsi"
9  #include "stm32mp15xd.dtsi"
10 #include "stm32mp15-pinctrl.dtsi"
11 #include "stm32mp15xxaa-pinctrl.dtsi"
12 #include "stm32mp157-m4-srm.dtsi"
13 #include "stm32mp157-m4-srm-pinctrl.dtsi"
14 #include "stm32mp15xx-edx.dtsi"
```

注意，第 14 行引用的是 stm32mp15xx-edx.dtsi 这个设备树头文件，我们要将其改为上面创建的 stm32mp15d-atk.dtsi，修改以后如图 13.2.2.1 所示：

```
6  /dts-v1/;
7
8  #include "stm32mp157.dtsi"
9  #include "stm32mp15xd.dtsi"
10 #include "stm32mp15-pinctrl.dtsi"
11 #include "stm32mp15xxaa-pinctrl.dtsi"
12 #include "stm32mp157-m4-srm.dtsi"
13 #include "stm32mp157-m4-srm-pinctrl.dtsi"
14 #include "stm32mp157d-atk.dtsi"
15
16 / {
17     model = "STMicroelectronics STM32MP157D eval daughter";
18     compatible = "st,stm32mp157d-ed1", "st,stm32mp157";
```

图 13.2.2.1 修改后的 stm32mp157d-atk.dts

13.2.3 修改电源管理设置

打开 13.2.2 小节创建的 stm32mp157d-atk-u-boot.dtsi 这个文件。前 53 行的内容如下所示：

示例代码 13.2.3.1 stm32mp157d-atk-u-boot.dtsi 文件

```
1  // SPDX-License-Identifier: GPL-2.0+ OR BSD-3-Clause
2  /*
3   * Copyright : STMicroelectronics 2018
4   */
5
6  #include <dt-bindings/clock/stm32mp1-clksrc.h>
7  #include "stm32mp15-u-boot.dtsi"
8  #include "stm32mp15-ddr3-2x4Gb-1066-binG.dtsi"
9
10 / {
11  aliases {
12      i2c3 = &i2c4;
13      mmc0 = &sdmmc1;
14      mmc1 = &sdmmc2;
15  };
16
17  config {
18      u-boot,boot-led = "heartbeat";
19      u-boot,error-led = "error";
20      u-boot,mmc-env-partition = "ssbl";
21      st,fastboot-gpios = <&gpioa 13 GPIO_ACTIVE_LOW>;
22      st,stm32prog-gpios = <&gpioa 14 GPIO_ACTIVE_LOW>;
23  };
24
25  led {
26      red {
27          label = "error";
28          gpios = <&gpioa 13 GPIO_ACTIVE_LOW>;
29          default-state = "off";
30          status = "okay";
31          };
32      };
33 };
34
35 #ifndef CONFIG_STM32MP1_TRUSTED
36 &clk_hse {
37      st,digbypass;
38 };
39
```

```
40 &i2c4 {
41     u-boot,dm-pre-reloc;
42 };
43
44 &i2c4_pins_a {
45     u-boot,dm-pre-reloc;
46     pins {
47         u-boot,dm-pre-reloc;
48     };
49 };
50
51 &pmic {
52     u-boot,dm-pre-reloc;
53 };
```

把示例代码 13.2.3.1 中的的 21~22 行、26~31 行和 51~53 行都删除，删除以后如图 13.2.3.1 所示：

```
 6 #include <dt-bindings/clock/stm32mp1-clksrc.h>
 7 #include "stm32mp15-u-boot.dtsi"
 8 #include "stm32mp15-ddr3-2x4Gb-1066-binG.dtsi"
 9
10 / {
11     aliases {
12         i2c3 = &i2c4;
13         mmc0 = &sdmmc1;
14         mmc1 = &sdmmc2;
15     };
16
17     config {
18         u-boot,boot-led = "heartbeat";
19         u-boot,error-led = "error";
20         u-boot,mmc-env-partition = "ssbl";
21     };
22
23     led {
24     };
25 };
26
27 #ifndef CONFIG_STM32MP1_TRUSTED
28 &clk_hse {
29     st,digbypass;
30 };
31
32 &i2c4 {
33     u-boot,dm-pre-reloc;
34 };
35
36 &i2c4_pins_a {
37     u-boot,dm-pre-reloc;
38     pins {
39         u-boot,dm-pre-reloc;
40     };
41 };
```

图 13.2.3.1 修改以后的 stm32mp157d-atk-u-boot.dtsi 文件

接着修改 stm32mp157d-atk.dtsi 文件，找到如下所示代码：

示例代码 13.2.3.2 stm32mp157d-atk.dtsi 代码段

```
90  &adc {
91 /* ANA0, ANA1 are dedicated pins and don't need pinctrl:
   only in6. */
92      pinctrl-0 = <&adc1_in6_pins_a>;
93      pinctrl-names = "default";
94      vdd-supply = <&vdd>;
95      vdda-supply = <&vdda>;
96      vref-supply = <&vdda>;
97      status = "disabled";
98      adc1: adc@0 {
99          st,adc-channels = <0 1 6>;
100         /* 16.5 ck_cycles sampling time */
101         st,min-sample-time-nsecs = <400>;
102         status = "okay";
103     };
104 };
105
106 &cpu0{
107     cpu-supply = <&vddcore>;
108 };
109
110 &crc1 {
111     status = "okay";
112 };
113
114 &dac {
115     pinctrl-names = "default";
116     pinctrl-0 = <&dac_ch1_pins_a &dac_ch2_pins_a>;
117     vref-supply = <&vdda>;
118     status = "disabled";
119     dac1: dac@1 {
120         status = "okay";
121     };
122     dac2: dac@2 {
123         status = "okay";
124     };
125 };
126
127 &dma1 {
128     sram = <&dma_pool>;
129 };
```

```
......
142
143 &i2c4 {
144     pinctrl-names = "default", "sleep";
145     pinctrl-0 = <&i2c4_pins_a>;
146     pinctrl-1 = <&i2c4_pins_sleep_a>;
147     i2c-scl-rising-time-ns = <185>;
148     i2c-scl-falling-time-ns = <20>;
149     clock-frequency = <400000>;
150     status = "okay";
151     /* spare dmas for other usage */
152     /delete-property/dmas;
153     /delete-property/dma-names;
154
155     pmic: stpmic@33 {
156         compatible = "st,stpmic1";
157         reg = <0x33>;
158         interrupts-extended = <&exti_pwr 55 IRQ_TYPE_EDGE_FALLING>;
159         interrupt-controller;
160         #interrupt-cells = <2>;
161         status = "okay";
162
163         regulators {
164             compatible = "st,stpmic1-regulators";
165             buck1-supply = <&vin>;
166             buck2-supply = <&vin>;
167             buck3-supply = <&vin>;
168             buck4-supply = <&vin>;
169             ldo1-supply = <&v3v3>;
170             ldo2-supply = <&v3v3>;
171             ldo3-supply = <&vdd_ddr>;
172             ldo4-supply = <&vin>;
173             ldo5-supply = <&v3v3>;
174             ldo6-supply = <&v3v3>;
175             vref_ddr-supply = <&vin>;
176             boost-supply = <&vin>;
177             pwr_sw1-supply = <&bst_out>;
178             pwr_sw2-supply = <&bst_out>;
179
180             vddcore: buck1 {
181                 regulator-name = "vddcore";
182                 regulator-min-microvolt = <1200000>;
183                 regulator-max-microvolt = <1350000>;
```

```
184                 regulator-always-on;
185                 regulator-initial-mode = <0>;
186                 regulator-over-current-protection;
187             };
......
278              vbus_sw: pwr_sw2 {
279                 regulator-name = "vbus_sw";
280                 interrupts = <IT_OCP_SWOUT 0>;
281                 regulator-active-discharge = <1>;
282              };
283         };
284
285         onkey {
286             compatible = "st,stpmic1-onkey";
287             interrupts = <IT_PONKEY_F 0>, <IT_PONKEY_R 0>;
288             interrupt-names = "onkey-falling", "onkey-rising";
289             power-off-time-sec = <10>;
290             status = "okay";
291         };
292
293         watchdog {
294             compatible = "st,stpmic1-wdt";
295             status = "disabled";
296         };
297     };
298 };
```

将示例代码 13.2.3.2 中 90~104 行的 adc 节点、114~125 行的 dac 节点以及 143~298 行的 i2c4 节点全部删除掉，删除以后如图 13.2.3.2 所示：

```
90  &cpu0{
91      cpu-supply = <&vddcore>;
92  };
93
94  &crc1 {
95      status = "okay";
96  };
97
98  &dma1 {
99      sram = <&dma_pool>;
100 };
101
102 &dma2 {
103     sram = <&dma_pool>;
104 };
105
106 &dts {
107     status = "okay";
108 };
109
110 &hash1 {
111     status = "okay";
112 };
113
114 &ipcc {
115     status = "okay";
116 };
```

图 13.2.3.2 修改后的 stm32mp157d-atk.dtsi 代码段

继续修改 stm32mp157d-atk.dtsi 文件，找到如下所示代码：

示例代码 13.2.3.3 stm32mp157d-atk.dtsi 代码段

```
58 led {
59     compatible = "gpio-leds";
60     blue {
61         label = "heartbeat";
62         gpios = <&gpiod 9 GPIO_ACTIVE_HIGH>;
63         linux,default-trigger = "heartbeat";
64         default-state = "off";
65     };
66 };
67
68 sd_switch: regulator-sd_switch {
69     compatible = "regulator-gpio";
70     regulator-name = "sd_switch";
71     regulator-min-microvolt = <1800000>;
72     regulator-max-microvolt = <2900000>;
73     regulator-type = "voltage";
74     regulator-always-on;
75
76     gpios = <&gpiof 14 GPIO_ACTIVE_HIGH>;
77     gpios-states = <0>;
```

```
78      states = <1800000 0x1 2900000 0x0>;
79  };
```

上述代码是 led 和 sd_switch 节点信息，将这两个节点都删除掉，这两个节点删除以后如图 13.2.3.3 所示：

```
51          retram: retram@38000000 {
52              compatible = "shared-dma-pool";
53              reg = <0x38000000 0x10000>;
54              no-map;
55          };
56      };
57
58
59      vin: vin {
60          compatible = "regulator-fixed";
61          regulator-name = "vin";
62          regulator-min-microvolt = <5000000>;
63          regulator-max-microvolt = <5000000>;
64          regulator-always-on;
65      };
66  };
```

图 13.2.3.3 删除以后的 stm32mp157d-atk.dtsi 代码段

最后我们需要向 stm32mp157d-atk.dtsi 文件的根节点‘/’下添加自己的电源管理配置，将下面的代码添加到图 13.2.3.3 中的 58 行处：

示例代码 13.2.3.4 要添加的电源管理配置节点

```
1  vddcore: regulator-vddcore {
2      compatible = "regulator-fixed";
3      regulator-name = "vddcore";
4      regulator-min-microvolt = <1200000>;
5      regulator-max-microvolt = <1350000>;
6      regulator-always-on;
7      regulator-boot-on;
8  };
9
10 v3v3: regulator-3p3v {
11     compatible = "regulator-fixed";
12     regulator-name = "v3v3";
13     regulator-min-microvolt = <3300000>;
14     regulator-max-microvolt = <3300000>;
15     regulator-always-on;
16     regulator-boot-on;
17 };
18
19 v1v8_audio: regulator-v1v8-audio {
20     compatible = "regulator-fixed";
21     regulator-name = "v1v8_audio";
22     regulator-min-microvolt = <1800000>;
```

```
23      regulator-max-microvolt = <1800000>;
24      regulator-always-on;
25      regulator-boot-on;
26  };
27
28  vdd: regulator-vdd {
29      compatible = "regulator-fixed";
30      regulator-name = "vdd";
31      regulator-min-microvolt = <3300000>;
32      regulator-max-microvolt = <3300000>;
33      regulator-always-on;
34      regulator-boot-on;
35  };
36
37  vdd_usb: regulator-vdd-usb {
38      compatible = "regulator-fixed";
39      regulator-name = "vdd_usb";
40      regulator-min-microvolt = <3300000>;
41      regulator-max-microvolt = <3300000>;
42      regulator-always-on;
43      regulator-boot-on;
44  };
```

上述代码定义了 5 个电源节点，分别为：vddcore、v3v3、v1v8_audio、vdd 和 vdd_usb，至此，电源管理相关配置已经设置好了。

13.2.4 修改 TF 卡和 EMMC 配置

继续修改 stm32mp157d-atk.dtsi 文件，找到 sdmmc1 和 sdmmc2 这两个节点，将这两个节点改为如下所示内容：

示例代码 13.2.4.1 修改后的 sdmmc1 和 sdmmc2 节点

```
1  &sdmmc1 {
2      pinctrl-names = "default", "opendrain", "sleep";
3      pinctrl-0 = <&sdmmc1_b4_pins_a>;
4      pinctrl-1 = <&sdmmc1_b4_od_pins_a>;
5      pinctrl-2 = <&sdmmc1_b4_sleep_pins_a>;
6      st,neg-edge;
7      broken-cd;
8      bus-width = <4>;
9      vmmc-supply = <&v3v3>;
10     status = "okay";
11 };
12
13 &sdmmc2 {
14     pinctrl-names = "default", "opendrain", "sleep";
```

```
15      pinctrl-0 = <&sdmmc2_b4_pins_a &sdmmc2_d47_pins_a>;
16      pinctrl-1 = <&sdmmc2_b4_od_pins_a &sdmmc2_d47_pins_a>;
17      pinctrl-2 = <&sdmmc2_b4_sleep_pins_a &sdmmc2_d47_sleep_pins_a>;
18      non-removable;
19      st,neg-edge;
20      bus-width = <8>;
21      vmmc-supply = <&v3v3>;
22      keep-power-in-suspend;
23      status = "okay";
24  };
```

修改以后如图 13.2.4.1 所示：

```
169  &sdmmc1 {
170      pinctrl-names = "default", "opendrain", "sleep";
171      pinctrl-0 = <&sdmmc1_b4_pins_a>;
172      pinctrl-1 = <&sdmmc1_b4_od_pins_a>;
173      pinctrl-2 = <&sdmmc1_b4_sleep_pins_a>;
174      st,neg-edge;
175      broken-cd;
176      bus-width = <4>;
177      vmmc-supply = <&v3v3>;
178      status = "okay";
179  };
180
181  &sdmmc2 {
182      pinctrl-names = "default", "opendrain", "sleep";
183      pinctrl-0 = <&sdmmc2_b4_pins_a &sdmmc2_d47_pins_a>;
184      pinctrl-1 = <&sdmmc2_b4_od_pins_a &sdmmc2_d47_pins_a>;
185      pinctrl-2 = <&sdmmc2_b4_sleep_pins_a &sdmmc2_d47_sleep_pins_a>;
186      non-removable;
187      st,neg-edge;
188      bus-width = <8>;
189      vmmc-supply = <&v3v3>;
190      keep-power-in-suspend;
191      status = "okay";
192  };
```

图 13.2.4.1 修改后的 sdmmc1 和 sdmmc2

13.2.5 编译 uboot

接下来我们先编译一下 uboot，在编译之前先在 stm32mp157d-atk.dtsi 文件中找到 usbotg_hs 节点，此节点默认内容如下所示：

示例代码 13.2.5.1 usbotg_hs 节点

```
222 &usbotg_hs {
223     vbus-supply = <&vbus_otg>;
224 };
```

此节点定义了 USB_OTG 的电源配置，ST 官方开发板的 USB_OTG 电源也是通过 PMIC 配置的，所以 USB_OTG 电源配置也要修改，但是这里我们要先测试一下当前修改后的 uboot 能不能运行，因此先不改 USB_OTG 电源，所以先将示例代码 13.2.5.1 中的 usbotg_hs 节点屏蔽掉，防止编译报错，屏蔽以后如图 13.2.5.1 所示：

```
222  /*&usbotg_hs {
223      vbus-supply = <&vbus_otg>;
224  };*/
225
226  &usbphyc_port0 {
227      phy-supply = <&vdd_usb>;
228  };
```

图 13.2.5.1 屏蔽 usbotg_hs 节点

在编译 uboot 之前要先让编译器知道我们要编译哪个设备树文件，打开 arch/arm/dts/Makefile 文件，找到“dtb-$(CONFIG_STM32MP15x)”配置项，然后在此配置项中加入“stm32mp157d-atk.dtb”，添加完以后如图 13.2.5.2 所示：

```
832  dtb-$(CONFIG_STM32MP15x) += \
833      stm32mp157a-avenger96.dtb \
834      stm32mp157a-dk1.dtb \
835      stm32mp157a-ed1.dtb \
836      stm32mp157a-ev1.dtb \
837      stm32mp157c-dk2.dtb \
838      stm32mp157c-ed1.dtb \
839      stm32mp157c-ev1.dtb \
840      stm32mp157d-dk1.dtb \
841      stm32mp157d-ed1.dtb \
842      stm32mp157d-ev1.dtb \
843      stm32mp157f-dk2.dtb \
844      stm32mp157f-ed1.dtb \
845      stm32mp157f-ev1.dtb \
846      stm32mp15xx-dhcom-pdk2.dtb \
847      stm32mp157d-atk.dtb
848
```

添加stm32mp157d-atk.dtb

图 13.2.5.2 添加 stm32mp157d-atk.dtb

图 13.2.5.2 中第 847 行里就是告诉“Makefile”把我们刚新建的 stm32mp157d-atk.bts 编译成对应的 dtb 文件。以前编译 uboot 都是自己输入一条一条命令编译，我们可以创建一个 shell 脚本，将所有的编译命令都写到这个 shell 脚本里面，然后每次的时候只需要执行一下这个 shell 脚本即可。在 uboot 源码根目录下新建一个名为 stm32mp157d_alientek.sh 的 shell 脚本，在这个 shell 脚本里面输入如下内容：

示例代码 13.2.5.2 stm32mp157d_alientek.sh 文件

```
1  #!/bin/bash
2
3  make distclean
4  make stm32mp15_atk_trusted_defconfig
5  make DEVICE_TREE=stm32mp157d-atk all -j12
```

第 3 行清零 uboot。

第 4 行使用 stm32mp15_atk_trusted_defconfig 来配置 uboot。

第 5 行编译 uboot，设备树为 stm32mp157d-atk.dts。

给予 stm32mp157d_alientek.sh 可执行权限，然后运行脚本来完成编译，命令如下：

```
chmod 777 stm32mp157d_alientek.sh      //给予可执行权限，一次即可
./stm32mp157d_alientek.sh              //运行脚本编译 uboot
```

编译成功会出现如图 13.2.5.3 所示信息：

图 13.2.5.3 uboot 编译成功

从图 13.2.5.3 可以看出，stm32mp157d-atk.dts 已经编译成功，生成了对应的 stm32mp157d-atk.dtb 文件，u-boot.stm32 就是我们一会要烧写测试的，我们要先将其重命名为 my-u-boot.stm32，然后再烧写进去。

烧写完成以后 uboot 启动过程如图 13.2.5.4 所示：

```
U-Boot 2020.01-stm32mp-r1 (Dec 01 2020 - 18:32:30 +0800)

CPU: STM32MP157DAA Rev.Z
Model: STMicroelectronics STM32MP157D eval daughter
Board: stm32mp1 in trusted mode (st,stm32mp157d-ed1)
DRAM:  1 GiB
Clocks:
- MPU : 800 MHz
- MCU : 208.878 MHz
- AXI : 266.500 MHz
- PER : 24 MHz
- DDR : 533 MHz
WDT:   Started with servicing (32s timeout)
NAND:  0 MiB
MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
Loading Environment from MMC... OK
In:    serial
Out:   serial
Err:   serial
invalid MAC address in OTP 00:00:00:00:00:00
Net:   No ethernet found.
Hit any key to stop autoboot:  0
Boot over mmc1!
*** Warning: old environment U-Boot 2020.01-stm32mp-r1 (Oct 19 2020 - 10:17:36 +0800)
* set default: env default -a; env save; reset
* update current: env set env_ver ${ver}; env save
switch to partitions #0, OK
mmc1(part 0) is current device
** Unrecognized filesystem type **
STM32MP>
```

图 13.2.5.4 uboot 启动过程

从图 13.2.5.4 可以看出，uboot 启动成功，并且进入了命令行模式，并且命令行也可以正常操作，那么是不是说明 uboot 已经移植成功了呢？不一定，uboot 有没有移植成功取决于当前 uboot 能不能满足我们的要求，如果不满足就说明还没移植成功。

当前 uboot 的网络、USB_OTG 也不能正常工作，所以我们还需要接着修改。

13.2.6 网络驱动修改

1、网络设备树的修改

从图 13.5.2.4 可以看到“Net: No ethernet found.”这一行，这样行的意思就是没有找到网络，说明 uboot 下的网路驱动有问题。原因是当前设备树并没有网络相关节点，我们只需要将网络相关节点加进去就可以了。打开 stm32mp157d-atk.dtsi 文件，将如下所示的 ethernet0 节点加添加到最后面：

示例代码 13.2.6.1 ethernet0 节点

```
1  &ethernet0 {
2          status = "okay";
3          pinctrl-0 = <&ethernet0_rgmii_pins_a>;
4          pinctrl-1 = <&ethernet0_rgmii_pins_sleep_a>;
5          pinctrl-names = "default", "sleep";
6          phy-mode = "rgmii-id";
7          max-speed = <1000>;
8          phy-handle = <&phy0>;
9
10         mdio0 {
11                 #address-cells = <1>;
12                 #size-cells = <0>;
13                 compatible = "snps,dwmac-mdio";
14                 phy0: ethernet-phy@0 {
15                         reg = <0>;
16                 };
17         };
18 };
```

添加完成以后如图 13.2.6.1 所示：

```
230  &usbphyc_port1 {
231      phy-supply = <&vdd_usb>;
232  };
233
234  &ethernet0 {
235          status = "okay";
236          pinctrl-0 = <&ethernet0_rgmii_pins_a>;
237          pinctrl-1 = <&ethernet0_rgmii_pins_sleep_a>;
238          pinctrl-names = "default", "sleep";
239          phy-mode = "rgmii-id";
240          max-speed = <1000>;
241          phy-handle = <&phy0>;
242
243          mdio0 {
244                  #address-cells = <1>;
245                  #size-cells = <0>;
246                  compatible = "snps,dwmac-mdio";
247                  phy0: ethernet-phy@0 {
248                          reg = <0>;
249                  };
250          };
251  };
252
```

图 13.2.6.1 添加进来的 ethernet0 节点

支持，网络节点已经添加到设备树里面了。

2、网络驱动修改

正点原子 V1.3 及以后版本的核心板开始将网络 PHY 芯片更换为了国产裕太电子的 YT8511，所以需要修改一下驱动。YT8511 驱动源码以及修改方法已经放到了开发板光盘中，路径为：1、程序源码→8、模块驱动源码→1、YT8511 驱动源码→uboot 下修改方法→phy.c。phy.c 是正点原

子修改后支持 YT8511，此文件同时支持 RTL8211 以及 YT8511。用修改后的 phy.c 文件替换掉 uboot 下的/drivers/net/phy/phy.c 文件即可，设备树无需做任何修改，直接编译即可。

注意，正点原子 V1.2 版本核心板上使用的 RTL8211，其 PHY 地址为 0X01。V1.3 以后的核心板使用 YT8511，其 PHY 地址为 0X00，因此在设备树里面需要修改 PHY 地址为 0X00。但是这样修改以后会导致 YT8511 和 RTL8211 的设备树不兼容，需要多编写一份设备树，导致出厂设备树过多。因此为了一个设备树兼容 YT8511 和 RTL8211，正点原子团队修改了 phy.c 文件中的 phy_connect 函数，直接给定 YT8511 的地址为 0X00，也就是不受设备树控制，如图 13.2.6.2 所示：

```
1114    /***********zuozhongkai add 2021/4/23****************/
1115    if (!phydev) /* 如果还没有获取到phy_device，尝试YT8511 */
1116    {
1117        addr = 0;
1118        mask = (addr >= 0) ? (1 << addr) : 0xffffffff;
1119        phydev = phy_find_by_mask(bus, mask, interface);
1120    }
1121    /******************end add*****************************/
```

图 13.2.6.2 直接读取 YT8511

这种做法虽然不合规则，但是也是为了一个设备树兼容两个地址不同的 PHY 芯片而不得已采取的方法。

3、编译测试

重新编译 uboot 并烧写，uboot 启动过程如图 13.2.6.3 所示：

```
U-Boot 2020.01-stm32mp-r1 (Dec 01 2020 - 20:13:48 +0800)

CPU: STM32MP157DAA Rev.Z
Model: STMicroelectronics STM32MP157D eval daughter
Board: stm32mp1 in trusted mode (st,stm32mp157d-ed1)
DRAM:  1 GiB
Clocks:
- MPU : 800 MHz
- MCU : 208.878 MHz
- AXI : 266.500 MHz
- PER : 24 MHz
- DDR : 533 MHz
WDT:   Started with servicing (32s timeout)
NAND:  0 MiB
MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
Loading Environment from MMC... OK
In:    serial
Out:   serial
Err:   serial
invalid MAC address in OTP 00:00:00:00:00:00
Net:
Error: ethernet@5800a000 address not set.
No ethernet found.

Hit any key to stop autoboot:  0
Boot over mmc1!
*** Warning: old environment U-Boot 2020.01-stm32mp-r1 (Oct 19 2020 - 10:17:36 +0800)
* set default: env default -a; env save; reset
* update current: env set env_ver ${ver}; env save
switch to partitions #0, OK
mmc1(part 0) is current device
** Unrecognized filesystem type **
STM32MP>
```

图 13.2.6.3 uboot 启动信息

从图 13.2.6.3 中可以看到 “ethernet@5800a00 address not set.”字样，说明已经找到了网络外设并且网络外设已经启动，但是还是会报“No ethernet found.”错误，这是因为 uboot 启动的时候获取不到 MAC 地址，我们只需要设置一些地址相关的环境变量即可，前面讲解 uboot 的

网络相关命令的时候已经讲过了，命令如下：

```
setenv ipaddr 192.168.1.250          //开发板 IP 地址
setenv ethaddr 00:04:9f:04:d2:35     //开发板网卡 MAC 地址
setenv gatewayip 192.168.1.1         //开发板默认网关
setenv netmask 255.255.255.0         //开发板子网掩码
setenv serverip 192.168.1.249        //服务器地址，也就是 Ubuntu 地址
saveenv
```

设置好地址相关环境变量以后就可以在 uboot 中使用网络了，用网线将开发板上的网络接口与电脑或者路由器连接起来，保证开发板和电脑在同一个网段内，通过 ping 命令来测试一下网络连接，命令如下：

```
ping 192.168.1.249
```

结果如图 13.2.6.4 所示：

```
STM32MP> ping 192.168.1.249
ethernet@5800a000 Waiting for PHY auto negotiation to complete..... done
Using ethernet@5800a000 device
host 192.168.1.249 is alive
STM32MP>
```

图 13.2.6.3 ping 命令测试

从图 13.2.6.4 中可以看到“host 192.168.1.249 is alive”这句，表明 ping 主机成功，说明网络工作正常。

13.2.7 USB OTG 设备树修改

1、添加 usb_phy_tuning 子节点

在这一小节里，我们就给 uboot 添加 USB_OTG 功能，操作 stm32mp157d-atk.dtsi 这个文件，在根节点“/”下添加名为“usb_phy_tuning”的子节点，节点内容如下：

示例代码 13.2.7.1 usb_phy_tuning 节点

```
1  usb_phy_tuning: usb-phy-tuning {
2        st,hs-dc-level = <2>;
3        st,fs-rftime-tuning;
4        st,hs-rftime-reduction;
5        st,hs-current-trim = <15>;
6        st,hs-impedance-trim = <1>;
7        st,squelch-level = <3>;
8        st,hs-rx-offset = <2>;
9       st,no-lsfs-sc;
10 };
```

添加完成以后如图 13.2.7.1 所示：

```
51          retram: retram@38000000 {
52              compatible = "shared-dma-pool";
53              reg = <0x38000000 0x10000>;
54              no-map;
55          };
56      };
57
58      usb_phy_tuning: usb-phy-tuning {
59          st,hs-dc-level = <2>;
60          st,fs-rftime-tuning;
61          st,hs-rftime-reduction;
62          st,hs-current-trim = <15>;
63          st,hs-impedance-trim = <1>;
64          st,squelch-level = <3>;
65          st,hs-rx-offset = <2>;
66          st,no-lsfs-sc;
67      };
68
69      vddcore: regulator-vddcore {
70          compatible = "regulator-fixed";
71          regulator-name = "vddcore";
72          regulator-min-microvolt = <1200000>;
73          regulator-max-microvolt = <1350000>;
74          regulator-always-on;
75          regulator-boot-on;
76      };
```

在“/”节点下新添加的usb_phy_tuning子节点

图 13.2.7.1 新添加的 usb_phy_tuning 子节点

2、添加 STUSB1600 I2C 子节点

正点原子 STM32MP157 开发板上的 USB OTG 接口类型为 Type-C，使用 STUSB1600 芯片来实现此接口功能，STUSB1600 有一个 I2C 接口，此 I2C 接口用来配置芯片，因此我们还需要在设备树中添加 STUSB1600 相关的 I2C 节点内容。将如下内容添加到 stm32mp157d-atk.dtsi 的最后面：

示例代码 13.2.7.2 stusb1600 芯片 i2c 节点

```
1  &i2c1 {
2      pinctrl-names = "default", "sleep";
3      pinctrl-0 = <&i2c1_pins_a>;
4      pinctrl-1 = <&i2c1_pins_sleep_a>;
5      i2c-scl-rising-time-ns = <100>;
6      i2c-scl-falling-time-ns = <7>;
7      status = "okay";
8      /delete-property/dmas;
9      /delete-property/dma-names;
10
11     stusb1600@28 {
12         compatible = "st,stusb1600";
13         reg = <0x28>;
14         interrupts = <2 IRQ_TYPE_EDGE_FALLING>;
15         interrupt-parent = <&gpiog>;
16         pinctrl-names = "default";
17         pinctrl-0 = <&stusb1600_pins_a>;
18         status = "okay";
19         vdd-supply = <&vin>;
```

```
20
21          connector {
22              compatible = "usb-c-connector";
23              label = "USB-C";
24              power-role = "dual";
25              power-opmode = "default";
26
27              port {
28                  con_usbotg_hs_ep: endpoint {
29                      remote-endpoint = <&usbotg_hs_ep>;
30                  };
31              };
32          };
33      };
34  };
```

由于 STUSB1600 是挂在 STM32MP157 的 I2C1 接口下，因此示例代码 13.2.7.2 是向 I2C1 节点追加内容，完成以后如图 13.7.2.2 所示：

```
264  &i2c1 {
265      pinctrl-names = "default", "sleep";
266      pinctrl-0 = <&i2c1_pins_a>;
267      pinctrl-1 = <&i2c1_pins_sleep_a>;
268      i2c-scl-rising-time-ns = <100>;
269      i2c-scl-falling-time-ns = <7>;
270      status = "okay";
271      /delete-property/dmas;
272      /delete-property/dma-names;
273
274      stusb1600@28 {
275          compatible = "st,stusb1600";
276          reg = <0x28>;
277          interrupts = <2 IRQ_TYPE_EDGE_FALLING>;
278          interrupt-parent = <&gpiog>;
279          pinctrl-names = "default";
280          pinctrl-0 = <&stusb1600_pins_a>;
281          status = "okay";
282          vdd-supply = <&vin>;
283
284          connector {
285              compatible = "usb-c-connector";
286              label = "USB-C";
287              power-role = "dual";
288              power-opmode = "default";
289
290              port {
291                  con_usbotg_hs_ep: endpoint {
292                      remote-endpoint = <&usbotg_hs_ep>;
293                  };
294              };
295          };
296      };
297  };
```

图 13.7.2.2 stusb1600 节点

3、添加 usb 接口相关节点

继续向 stm32mp157d-atk.dtsi 文件添加 USB 接口相关节点内容，内容如下：

示例代码 13.2.7.3 usb 接口节点

```
1  &usbh_ehci {
2      phys = <&usbphyc_port0>;
3      status = "okay";
4  };
5
6  &usbotg_hs {
7      phys = <&usbphyc_port1 0>;
8      phy-names = "usb2-phy";
9      usb-role-switch;
10     status = "okay";
11
12     port {
13         usbotg_hs_ep: endpoint {
14             remote-endpoint = <&con_usbotg_hs_ep>;
15         };
16     };
17 };
18
19 &usbphyc {
20     status = "okay";
21 };
```

上述代码中有三个节点 usbh_ehci、usbotg_hs 和 usbphyc，其中 usbotg_hs 默认就有，我们在前面将其屏蔽掉了，完成以后如图 13.2.7.3 所示：

```
233 &usbh_ehci {
234     phys = <&usbphyc_port0>;
235     status = "okay";
236 };
237
238 &usbotg_hs {
239     phys = <&usbphyc_port1 0>;
240     phy-names = "usb2-phy";
241     usb-role-switch;
242     status = "okay";
243
244     port {
245         usbotg_hs_ep: endpoint {
246             remote-endpoint = <&con_usbotg_hs_ep>;
247         };
248     };
249 };
250
251 &usbphyc {
252     status = "okay";
253 };
254
255 &usbphyc_port0 {
256     phy-supply = <&vdd_usb>;
257 };
```

新添加的3个USB接口相关节点

图 13.2.7.3 USB 接口节点

4、在 stm32mp157d-atk-u-boot.dtsi 文件中添加 usbotg_hs 节点

最后，我们需要在 stm32mp157d-atk-u-boot.dtsi 文件里面添加 usbotg_hs 节点，节点内容如下所示：

示例代码 13.2.7.4 usbotg_hs 节点

```
1 &usbotg_hs {
2     u-boot,force-b-session-valid;
3     hnp-srp-disable;
4     /* TEMP: force peripheral for USB OTG */
5     dr_mode = "peripheral";
6 };
```

结果如图 13.2.7.4 所示：

```
196
197 &usbotg_hs {
198     u-boot,force-b-session-valid;
199     hnp-srp-disable;
200     /* TEMP: force peripheral for USB OTG */
201     dr_mode = "peripheral";
202 };
203
```

图 13.2.7.4 stm32mp157d-atk-u-boot.dtsi 下的 usbotg_hs 节点

至此，uboot 下的 USB OTG 就已经修改完成，重新编译 uboot 并烧写，然后使用 ums 命令测试，看看能不能将 EMMC 模拟成 U 盘，挂载到电脑上，命令如下：

做到这里，保存修改的文件，编译和拷贝。把 STM32MP157D-ATK 开发板的 USB_OTG 接口连接到电脑里，重启开发板，接着运行以下命令：

```
ums 0 mmc 1
```

如果电脑上出现如图 13.2.7.5 所示的磁盘，说明 USB OTG 工作成功。

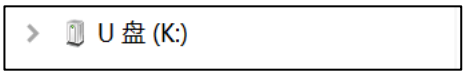


图 13.2.7.5 uboot 下开发板模拟 U 盘

由于笔者在编写本章节教程的时候使用的全新的核心板，因此 EMMC 上只有一个分区，所以在电脑上就只有一个 U 盘，如果 EMMC 上有多个分区的话就会出现多个 U 盘，一切以实际情况为准。

13.2.8 使能 boot 和 bootd 命令

ST 官方 uboot 默认并没有使能 boot 和 bootd 这两个命令，这两个命令的实现源文件为 cmd/bootm.c，bootm.c 下有如图 13.2.8.1 所示内容：

```
199  /*************************************************************************/
200  /* bootd - boot default image */
201  /*************************************************************************/
202  #if defined(CONFIG_CMD_BOOTD)
203  int do_bootd(cmd_tbl_t *cmdtp, int flag, int argc, char * const argv[])
204  {
205      return run_command(env_get("bootcmd"), flag);
206  }
207
208  U_BOOT_CMD(
209      boot,   1,  1,  do_bootd,
210      "boot default, i.e., run 'bootcmd'",
211      ""
212  );
213
214  /* keep old command name "bootd" for backward compatibility */
215  U_BOOT_CMD(
216      bootd, 1,   1,  do_bootd,
217      "boot default, i.e., run 'bootcmd'",
218      ""
219  );
220
221  #endif
```

图 13.2.8.1 boot 和 bootd 命令具体实现

从图 13.2.8.1 可以看出，如果要使能 boot 和 bootd 这两个命令，必须定义宏 CONFIG_CMD_BOOT。打开 include/configs/stm32mp1.h，然后在后面添加如下宏定义：

示例代码 13.2.8.1 CONFIG_CMD_BOOTD 宏

```
#define CONFIG_CMD_BOOTD    /* 使能 boot 和 bootd 命令 */
```

如图 13.2.8.2 所示：

```
257  #endif /* ifdef CONFIG_DISTRO_DEFAULTS*/
258
259  #define CONFIG_CMD_BOOTD /* boot & bootd CMD */
260
261  #endif /* __CONFIG_H */
262
```

图 13.2.8.3 添加 CONFIG_CMD_BOOTD 宏

重新编译 uboot 并烧写启动，输入命令：

```
? boot
```

或者

```
? bootd
```

如果 boot 和 bootd 使能的话就会打印出相应的命令使用方法，如图 13.2.8.4 所示：

```
STM32MP> ? boot
boot - boot default, i.e., run 'bootcmd'

Usage:
boot
STM32MP> ? bootd
bootd - boot default, i.e., run 'bootcmd'

Usage:
bootd
STM32MP>
```

图 13.2.8.4 boot 和 bootd 帮助信息

13.2.9 LCD 驱动修改

uboot 也是支持 LCD 显示的，但是要进行相应的设置，主要是设置屏幕背光、屏幕时序参数这些，这些直接在设备树里面修改即可。打开 stm32mp157d-atk.dts 文件，在里面添加 LCD 相关节点信息，首先在根节点“/”下添加 panel_backlight 和 panel_rgb 这两个节点，节点内容如下：

示例代码 13.2.9.1 新添加节点信息

```
1  panel_backlight: panel-backlight {
2      compatible = "gpio-backlight";
3      gpios = <&gpiod 13 GPIO_ACTIVE_HIGH>;
4      default-on;
5      status = "okay";
6   };
7
8  panel_rgb: panel-rgb {
9      compatible = "simple-panel";
10     pinctrl-names = "default", "sleep";
11     pinctrl-0 = <&ltdc_pins_b>;
12     pinctrl-1 = <&ltdc_pins_sleep_b>;
13     backlight = <&panel_backlight>;
14     status = "okay";
15
16      port {
17          panel_in_rgb: endpoint {
18              remote-endpoint = <&ltdc_ep0_out>;
19          };
20      };
21
22      display-timings {
23          native-mode = <&timing0>; /* 时序信息 */
24          timing0: timing0 {                  /* 7 寸 1024*600 分辨率 */
25              clock-frequency = <51200000>;   /* LCD 像素时钟，单位 Hz */
```

```
26              hactive = <1024>;                /* LCD X 轴像素个数 */
27              vactive = <600>;                 /* LCD Y 轴像素个数 */
28              hfront-porch = <160>;            /* LCD hfp 参数 */
29              hback-porch = <140>;             /* LCD hbp 参数 */
30              hsync-len = <20>;                /* LCD hspw 参数 */
31              vback-porch = <20>;              /* LCD vbp 参数 */
32              vfront-porch = <12>;             /* LCD vfp 参数 */
33              vsync-len = <3>;                 /* LCD vspw 参数 */
34          };
35      };
36  };
```

第 1~6 行，panel_backlight 为 LCD 的背光控制节点，主要指定 LCD 背光 IO 所使用的引脚，正点原子的 STM32MP157 开发板 LCD 背光引脚为 PD13。

第 8~38 行，panel_rgb 为 RGB LCD 节点，指定了 LTDC 接口所使用的 IO、屏幕时序参数等。第 11~12 行指定 LTDC 接口的 IO，ltdc_pins_b 和 ltdc_pins_sleep_b 定义在 stm32mp15-pinctrl.dtsi 文件中。第 22~35 行的 display-timings 是非常重要的 LCD 时序参数，不同的屏幕其时序参数不同，这里演示的是正点原子 7 寸 1024*600 分辨率的。正点原子有 4 款 RGB 接口屏幕，分别为 4.3 寸 480*272 和 800*480 分辨率、7 寸 800*480 和 1024*600 分辨率，这款屏幕时序参数如表 13.2.9.1 所示：

屏幕型号	参数	值	单位
ATK4342	水平显示区域	480	tCLK
	HSPW(thp)	1	tCLK
	HBP(thb)	40	tCLK
	HFP(thf)	5	tCLK
	垂直显示区域	272	th
	VSPW(tvp)	1	th
	VBP(tvb)	8	th
	VFP(tvf)	8	th
	像素时钟	9	MHz
ATK4384	水平显示区域	800	tCLK
	HSPW(thp)	48	tCLK
	HBP(thb)	88	tCLK
	HFP(thf)	40	tCLK
	垂直显示区域	480	th
	VSPW(tvp)	3	th
	VBP(tvb)	32	th
	VFP(tvf)	13	th
	像素时钟	31	MHz
ATK7084	水平显示区域	800	tCLK
	HSPW(thp)	1	tCLK
	HBP(thb)	46	tCLK
	HFP(thf)	210	tCLK

	垂直显示区域	480	th
	VSPW(tvp)	1	th
	VBP(tvb)	23	th
	VFP(tvf)	22	th
	像素时钟	33.3	MHz
ATK7016	水平显示区域	1024	tCLK
	HSPW(thp)	20	tCLK
	HBP(thb)	140	tCLK
	HFP(thf)	160	tCLK
	垂直显示区域	600	th
	VSPW(tvp)	3	th
	VBP(tvb)	20	th
	VFP(tvf)	12	th
	像素时钟	51.2	MHz

表 13.2.9.1 RGB LCD 屏幕时间参数

正点原子其他 3 款 RGB 屏幕的时序参数如下所示：

示例代码 13.2.9.2 其他 3 款 LCD 时序参数

```
/* 4.3 寸 480*272 分辨率 */
display-timings {
  native-mode = <&timing0>;           /* 时序信息 */
  timing0: timing0 {
       clock-frequency = <9200000>; /* LCD 像素时钟，单位 Hz */
       hactive = <480>;              /* LCD X 轴像素个数 */
       vactive = <272>;              /* LCD Y 轴像素个数 */
       hfront-porch = <8>;           /* LCD hfp 参数 */
       hback-porch = <4>;            /* LCD hbp 参数 */
       hsync-len = <41>;             /* LCD hspw 参数 */
       vback-porch = <2>;            /* LCD vbp 参数 */
       vfront-porch = <4>;           /* LCD vfp 参数 */
       vsync-len = <10>;             /* LCD vspw 参数 */
 };
}

/* 4.3 寸 800*480 分辨率 */
display-timings {
   native-mode = <&timing0>;         /* 时序信息 */
   timing0: timing0 {
        clock-frequency = <31000000>; /* LCD 像素时钟，单位 Hz */
        hactive = <800>;             /* LCD X 轴像素个数 */
        vactive = <480>;             /* LCD Y 轴像素个数 */
        hfront-porch = <40>;         /* LCD hfp 参数 */
        hback-porch = <88>;          /* LCD hbp 参数 */
        hsync-len = <48>;            /* LCD hspw 参数 */
```

```
27          vback-porch = <32>;             /* LCD vbp 参数 */
28          vfront-porch = <13>;            /* LCD vfp 参数 */
29          vsync-len = <3>;                /* LCD vspw 参数 */
30      };
31  }
32  /* 7 寸 800*480 分辨率 */
33  display-timings {
34      native-mode = <&timing0>;           /* 时序信息 */
35      timing0: timing0 {
36          clock-frequency = <51200000>; /* LCD 像素时钟，单位 Hz */
37          hactive = <800>;                /* LCD X 轴像素个数 */
38          vactive = <480>;                /* LCD Y 轴像素个数 */
39          hfront-porch = <210>;           /* LCD hfp 参数 */
40          hback-porch = <46>;             /* LCD hbp 参数 */
41          hsync-len = <1>;                /* LCD hspw 参数 */
42          vback-porch = <23>;             /* LCD vbp 参数 */
43          vfront-porch = <22>;            /* LCD vfp 参数 */
44          vsync-len = <1>;                /* LCD vspw 参数 */
45      };
46  }
```

第 2~15 行为 4.3 寸 480*272 分辨率的 LCD 时序参数，第 18~31 行为 4.3 寸 800*480 分辨率的 LCD 时序参数，第 33~46 行为 7 寸 800*480 分辨率的 LCD 时序参数。大家根据自己所使用的屏幕选择合适的屏幕时序参数。

添加完成以后如图 13.2.9.1 所示：

```
28      reserved-memory {
29          gpu_reserved: gpu@f6000000 {
30              reg = <0xf6000000 0x8000000>;
31              no-map;
32          };
33
34          optee_memory: optee@fe000000 {
35              reg = <0xfe000000 0x02000000>;
36              no-map;
37          };
38      };
39
40      panel_backlight: panel-backlight {
41          compatible = "gpio-backlight";
42          gpios = <&gpiod 13 GPIO_ACTIVE_HIGH>;
43          default-on;
44          status = "okay";
45      };
46
47      panel_rgb: panel-rgb {
48          compatible = "simple-panel";
49          pinctrl-names = "default", "sleep";
50          pinctrl-0 = <&ltdc_pins_b>;
51          pinctrl-1 = <&ltdc_pins_sleep_b>;
52          backlight = <&panel_backlight>;
53          status = "okay";
54
55          port {
56              panel_in_rgb: endpoint {
57                  remote-endpoint = <&ltdc_ep0_out>;
58              };
59          };
60
61          display-timings {
62              native-mode = <&timing0>;    /* 时序信息 */
63              timing0: timing0 {           /* 7寸1024*600分辨率 */
64                  clock-frequency = <51200000>; /* LCD 像素时钟，单位 Hz */
65                  hactive = <1024>;        /* LCD X 轴像素个数 */
66                  vactive = <600>;         /* LCD Y 轴像素个数 */
67                  hfront-porch = <160>;    /* LCD hfp 参数 */
68                  hback-porch = <140>;     /* LCD hbp 参数 */
69                  hsync-len = <20>;        /* LCD hspw 参数 */
70                  vback-porch = <20>;      /* LCD vbp 参数 */
71                  vfront-porch = <12>;     /* LCD vfp 参数 */
72                  vsync-len = <3>;         /* LCD vspw 参数 */
73              };
74          };
75      };
76  };
77
```

图 13.2.9.1 panel_backlight 和 panel_rgb 节点

最后还需要在 stm32mp157d-atk.dts 文件里面向 ltdc 节点追加一些内容，内容如下：

示例代码 13.2.9.3 ltdc 节点

```
1  &ltdc {
2      status = "okay";
3      pinctrl-names = "default";
4      port {
5          #address-cells = <1>;
6          #size-cells = <0>;
7
8          ltdc_ep0_out: endpoint@0 {
```

```
9                reg = <0>;
10               remote-endpoint = <&panel_in_rgb>;
11           };
12       };
13   };
```

添加完成以后如图 13.2.9.2 所示：

```
77
78  &ltdc {
79      status = "okay";
80      pinctrl-names = "default";
81      port {
82          #address-cells = <1>;
83          #size-cells = <0>;
84
85          ltdc_ep0_out: endpoint@0 {
86              reg = <0>;
87              remote-endpoint = <&panel_in_rgb>;
88          };
89      };
90  };
91
92  &cpu1{
93      cpu-supply = <&vddcore>;
94  };
```

图 13.2.9.3 ltdc 节点

修改完成以后编译 uboot，得到新的 uImage 和 stm32mp157d-atk.dtb 设备树，关于 uboot 下 LCD 的测试方法稍后会讲解。

13.3 U-boot 测试

13.3.1 自烧写测试

前面说了很多次 STM32CubeProgrammer 是如何烧写系统的，核心就是先向 DDR 里面下载一个 uboot 镜像，然后启动此 uboot，使用 uboot 里面的相关命令进行烧写。本章前面都是使用正点原子出厂 u-boot.stm32 来烧写我们移植的 uboot，本小节就用我们移植好的 u-boot 来烧写自身，也就是自烧写测试。这里就再也不需要 my-u-boot.stm32 了，直接使用我们前面编译得到的 u-boot.stm32 替换掉 images 目录下的正点原子出厂 u-boot.stm32。然后修改 FlashLayout，如图 13.3.1.1 所示：

tf-a.tsv

```
1  #Opt    Id      Name        Type    Device  Offset      Binary
2  -       0x01    fsbl1-boot  Binary  none    0x0         tf-a-stm32mp157d-atk-serialboot.stm32
3  -       0x03    ssbl-boot   Binary  none    0x0         u-boot.stm32
4  P       0x04    fsbl1       Binary  mmc1    boot1       tf-a-stm32mp157d-atk-trusted.stm32
5  P       0x05    fsbl2       Binary  mmc1    boot2       tf-a-stm32mp157d-atk-trusted.stm32
6  P       0x06    ssbl        Binary  mmc1    0x00080000  u-boot.stm32
```

图 13.3.1.1 修改后的 FlashLayout

修改完成以后重新烧写测试，此时首先下载到 DDR 中的 u-boot.stm32 镜像也是我们自己编译得到了，也就是用我们自己编译的 uboot 来烧写自身。检测方法很简单，就是看能不能正常烧写。

13.3.2 从 EMMC 启动 Linux

从EMMC启动也就是将编译出来的Linux镜像文件uImage和.dtb设备树文件保存在EMMC中，uboot从EMMC中读取这两个文件并启动，这个是我们产品最终的启动方式。首先EMMC里面要先存放Linux镜像文件uImage和.dtb设备树，但是我们前面新建的FlashLayout文件并没有烧写uImage和.dtb，所以我们需要修改一下FlashLayout文件，加入uImage和.dtb烧写命令。由于我们还没有移植Linux系统，所以uImage和.dtb就先使用正点原子出厂系统提供的，路径为：开发板光盘→8、系统镜像→2、出厂系统镜像→1、STM32CubeProg烧录固件包→atk-image-bootfs.ext4。atk-image-bootfs.ext4是ext4格式的打包文件，因为STM32CubeProgrammer软件要求将uImage和.dtb打包在一起，格式为ext4。atk-image-bootfs.ext4里面的文件如图13.3.2.1所示：

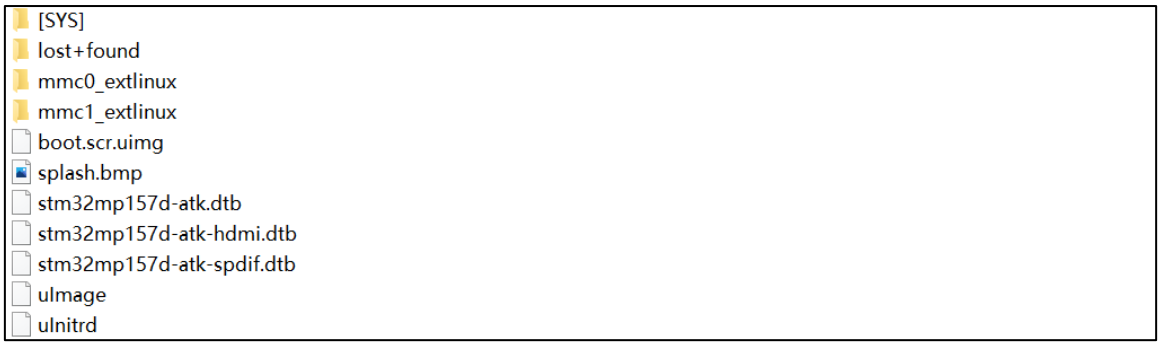


图 13.3.2.1 atk-image-bootfs.ext4 文件

从图13.3.2.1可以看出，正点原子出厂的atk-image-bootfs.ext4里面还是有不少文件，其中我们最关心的就是uImage和.dtb设备树。后面讲解Linux系统移植的时候会讲解如何将uImage和.dtb打包成ext4格式。

复制一份atk-image-bootfs.ext4到images文件夹，然后打开FlashLayout文件(tf-a.tsv)，添加下面一行：

示例代码 13.3.2.1 linux 系统烧写命令

```
P    0x21    boot    System  mmc1   0x00280000  atk-image-bootfs.ext4
```

修改完成以后如图13.3.2.2所示：

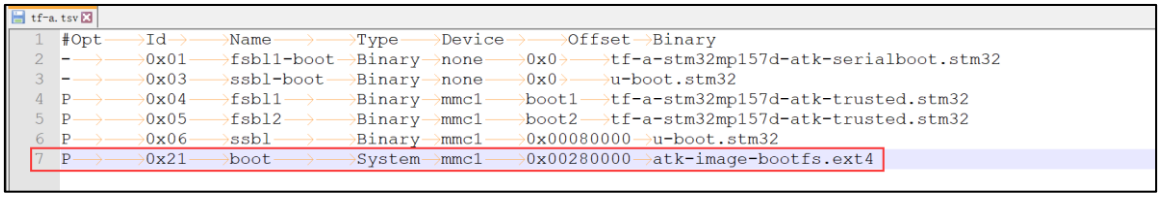


图 13.3.2.2 修改后的 FlashLayout

修改完成以后烧写到开发板中并重启，首先使用ext4ls命令查看一下EMMC的分区2里面有没有uImage和.dtb文件，命令如下：

```
ext4ls mmc 1:2
```

结果如图13.3.2.3所示：

```
STM32MP> ext4ls mmc 1:2
<DIR>       1024 .
<DIR>       1024 ..
            2943 boot.scr.uimg
<DIR>       1024 lost+found
<DIR>       1024 mmc0_extlinux
<DIR>       1024 mmc1_extlinux
           92670 splash.bmp
           74594 stm32mp157d-atk.dtb
           74014 stm32mp157d-atk-hdmi.dtb
           74510 stm32mp157d-atk-spdif.dtb
         8125872 uImage
         3632241 uInitrd
STM32MP>
```

图 13.3.2.3 EMMC 分区 2 文件

这里我们只需要用到图 13.3.2.3 中的 uImage 和 stm32mp157d-atk.dtb 这两个文件，设置 bootcmd 环境变量从 EMMC 里面读取系统文件，然后再启动，这个在前面讲解 BOOT 命令的时候已经讲过了如何操作，命令如下：

```
setenv bootcmd 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
saveenv
boot
```

设置好以后 uboot 就会按照 bootcmd 环境变量的内容加载系统并启动，如图 13.3.2.4 所示：

```
STM32MP> setenv bootcmd 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb
;bootm c2000000 - c4000000'
STM32MP> saveenv
Saving Environment to MMC... Writing to MMC(1)... OK
STM32MP> boot
8125872 bytes read in 206 ms (37.6 MiB/s)
74594 bytes read in 29 ms (2.5 MiB/s)
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31-g265df13e0
   Created:      2020-10-30   4:35:06 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    8125808 Bytes = 7.7 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffea000, end cffff361 ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 5.4.31-g265df13e0 (liangwencong@liangwencong) (gcc version 9.3.0 (GCC)) #2
8 SMP PREEMPT Fri Oct 30 12:31:03 CST 2020
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[    0.000000] OF: fdt: Machine model: STMicroelectronics STM32MP157C-DK2 Discovery Board
```

图 13.3.2.4 系统启动流程

注意！只有出现图 13.3.2.4 中的“Booting Linux on physical CPU 0x0”这一行就说明 uboot 引导 Linux 内核成功！

13.3.3 从网络启动 Linux 系统

从网络启动 Linux 系统的唯一目的就是为了调试！不管是为了调试 Linux 系统还是 Linux 下的驱动。每次修改 Linux 系统文件或者 Linux 下的某个驱动以后都要将其烧写到 EMMC 中去

测试，这样太麻烦了。我们可以设置 Linux 从网络启动，也就是将 Linux 镜像文件和根文件系统都放到 Ubuntu 下某个指定的文件夹中，这样每次重新编译 Linux 内核或者某个 Linux 驱动以后只需要使用 cp 命令将其拷贝到这个指定的文件夹中即可，这样就不用需要频繁的烧写 EMMC，这样就加快了开发速度。我们可以通过 nfs 或者 tftp 从 Ubuntu 中下载 uImage 和设备树文件，根文件系统的话也可以通过 nfs 挂载，不过本小节我们不讲解如何通过 nfs 挂载根文件系统，这个在讲解根文件系统移植的时候再讲解。这里我们使用 tftp 从 Ubuntu 中下载 uImage 和设备树文件，前提是要将 uImage 和设备树文件放到 Ubuntu 下的 tftp 目录中。

设置 bootcmd 环境变量，设置如下：

```
setenv bootcmd 'tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 -
(有空格) c4000000'
saveenv
boot
```

设置好以后，uboot 先从 tftp 服务器下载 uImage 和 stm32mp157d-atk.dtb 这两个文件，然后启动，如图 13.3.3.1 所示：

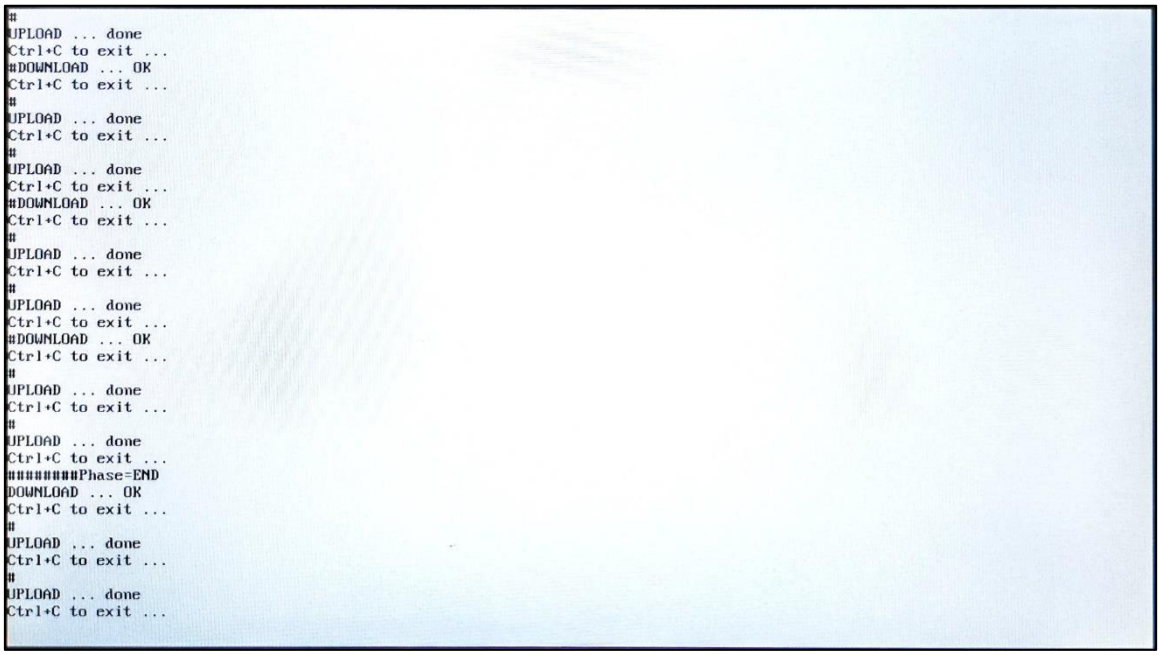
```
STM32MP> setenv bootcmd 'tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c40000
00'
STM32MP> saveenv
Saving Environment to MMC... Writing to redundant MMC(1)... OK
STM32MP> boot
ethernet@5800a000 Waiting for PHY auto negotiation to complete...... done
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'uImage'.
Load address: 0xc2000000
Loading: ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ################################
         2.1 MiB/s
done
Bytes transferred = 8125872 (7bfdb0 hex)
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'stm32mp157d-atk.dtb'.
Load address: 0xc4000000
Loading: ######
         1.5 MiB/s
done
Bytes transferred = 74594 (12362 hex)
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31-g265df13e0
   Created:      2020-10-30   4:35:06 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    8125808 Bytes = 7.7 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffea000, end cffff361 ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 5.4.31-g265df13e0 (liangwencong@liangwencong) (gcc version 9.3.0 (GCC)) #2
8 SMP PREEMPT Fri Oct 30 12:31:03 CST 2020
```

图 13.3.3.1 uboot 网络启动 linux 成功

13.3.4 LCD 测试

1、烧写过程显示测试

uboot 下 LCD 测试有两种方法，第一种大家应该已经看到了，那就是用 STM32CubeProgrammer 烧写系统的时候会在 LCD 上显示烧写过程，如图 13.3.4.1 所示：



13.3.4.1 uboot 烧写过程

2、bmp 命令显示测试

另外我们可以使用 uboot 下的 bmp 命令在 LCD 上显示一张 bmp 图片，bmp 命令如图 13.3.4.2 所示：

```
STM32MP> bmp
bmp - manipulate BMP image data

Usage:
bmp info <imageAddr>           - display image info
bmp display <imageAddr> [x y] - display image at x,y
STM32MP>
```

图 13.3.4.2 bmp 命令

从图 13.3.4.2 可以看出，有 bmp info 和 bmp display 两个命令：

①、bmp info 命令

命令格式如下：

```
bmp info <imageAddr>
```

bmp info 命令用于显示 BMP 图片信息，imageAddr 就是 BMP 图片在 RAM 中的起始地址。

②、bmp display 命令

bmp display 命令用于显示 bmp 图片，imageAddr 是要显示的 BMP 图片在 RAM 中的起始地址，[x,y]用于指定 BMP 图片左上角在屏幕上的显示坐标。

大家准备一个 BMP 格式的图片，注意这个 BMP 图片是有要求的，不是随便拿个图片过来

就能显示的：

①、图片的分辨率不能超过当前所使用的 LCD 分辨率，比如我现在使用的 LCD 分辨率为 1024*600，因此要显示的图片分辨率就不能超过 1024*600。

②、图片的色深应该和 LCD 驱动格式一致，比如 STM32MP1 的 uboot 中 LCD 驱动默认为 16BPP 色深的 RGB565 格式，因此 BMP 图片必须是 16 位的 RGB565 格式，否则显示会出问题！一般的图片都是 32 位色深的，所以我们需要借助其他软件将其改为 RGB565 格式，这里以 Photoshop 为例(自行安装 Photoshop CC 软件)，给大家简单讲一下如何将 32 位 BMP 图片转为 RGB565 格式的 16 位图片。首先使用 PS 打开需要转换格式的 BMP 图片，如图 13.3.4.3 所示：

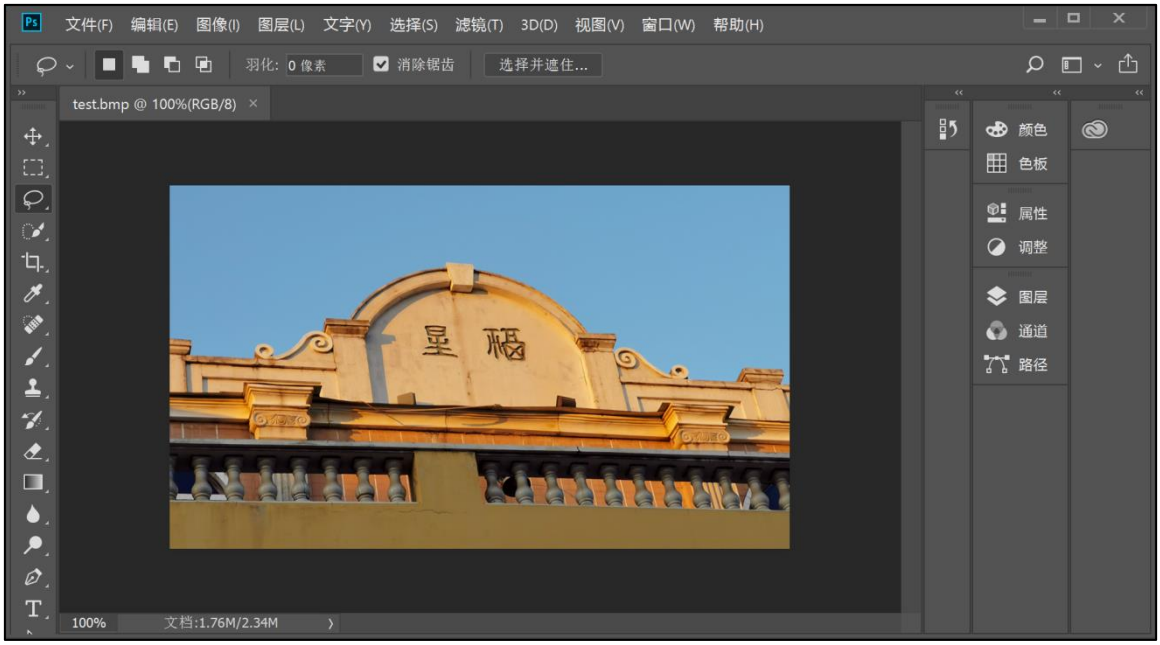


图 13.3.4.3 PS 打开图片

点击“文件->存储为(A)……”,打开如图 13.3.4.4 所示界面：

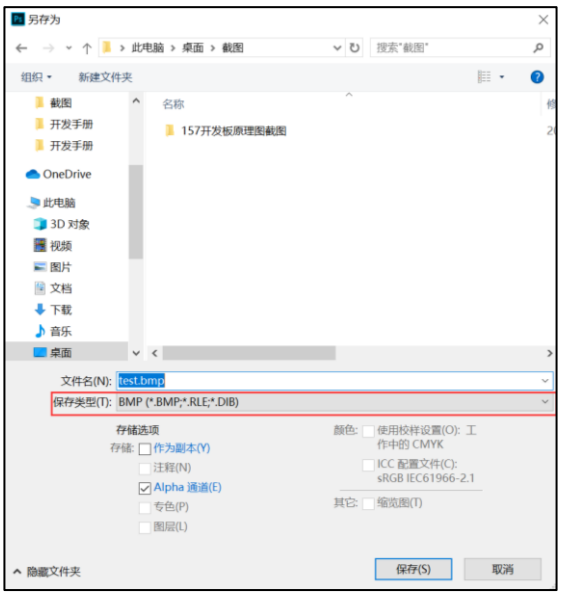


图 13.3.4.4 保存设置

在图 13.3.4.4 中设置好图片名字，保存类型一定要选择 BMP，设置好以后点击右下角的“保

存”按钮，然后打开保存设置对话框，如图 13.3.4.5 所示：

图 13.3.4.5 BMP 图片选项设置

点击图 13.3.4.5 中的“高级模式”，打开以后如图 13.3.4.6 所示：

图 13.3.4.6 BMP 高级模式

在图 13.3.4.6 中选择 16 位“R5 G6 B5”这个模式，也就是 RGB565，然后点击右上角的“确定”按钮即可，此时就会在我们前面设置的文件夹里面生成一个 16 位 RGB565 格式的 BMP 图片。

16 位 RGB565 格式的 BMP 图片准备好以后就可以显示了，首先将要显示的 BMP 图片放到 ubuntu 的 TFTP 服务器目录下，我们通过网络将 BMP 图片下载到板子的 DDR 中，然后再用 bmp 命令显示。命令如下：

```
tftp c0000000 test.bmp        //下载 bmp 图片
bmp info c0000000             //显示图片信息
```

图片信息如图 13.3.4.7 所示：

```
STM32MP> bmp info c0000000
Image size    : 1024 x 600
Bits per pixel: 16
Compression   : 3
```

图 13.3.4.7 图片信息

从图 13.3.4.7 可以看出，当前 BMP 图片分辨率为 1024*600，每个像素 16 位，也就是 16BPP，当前 LCD 分辨率为 1024*600，输入如下命令在 LCD 上显示图片：

```
bmp display c0000000 0 0    //显示 bmp 图片
```

结果如图 13.3.4.8 所示：

图 13.3.4.8 bmp 图片显示

13.4 bootcmd 和 bootargs 环境变量

13.4.1 bootcmd 环境变量

bootcmd 在前面已经说了很多次了，bootcmd 保存着 uboot 默认命令，uboot 倒计时结束以后就会执行 bootcmd 中的命令。这些命令一般都是用来启动 Linux 内核的，比如读取 EMMC 或者 NAND Flash 中的 Linux 内核镜像文件和设备树文件到 DRAM 中，然后启动 Linux 内核。可以在 uboot 启动以后进入命令行设置 bootcmd 环境变量的值。如果 EMMC 或者 NAND 中没有保存 bootcmd 的值，那么 uboot 就会使用默认的值，板子第一次运行 uboot 的时候都会使用默认值来设置 bootcmd 环境变量。打开文件 include/env_default.h，在此文件中有如下所示内容：

示例代码 13.4.1.1 默认环境变量

```
13 env_t embedded_environment __UBOOT_ENV_SECTION__(environment) = {
14     ENV_CRC,    /* CRC Sum */
15 #ifdef CONFIG_SYS_REDUNDAND_ENVIRONMENT
16     1,      /* Flags: valid */
17 #endif
18     {
19 #elif defined(DEFAULT_ENV_INSTANCE_STATIC)
20 static char default_environment[] = {
21 #else
22 const uchar default_environment[] = {
```

```
#endif
#ifndef CONFIG_USE_DEFAULT_ENV_FILE
#ifdef  CONFIG_ENV_CALLBACK_LIST_DEFAULT
    ENV_CALLBACK_VAR "=" CONFIG_ENV_CALLBACK_LIST_DEFAULT "\0"
#endif
#ifdef  CONFIG_ENV_FLAGS_LIST_DEFAULT
    ENV_FLAGS_VAR "=" CONFIG_ENV_FLAGS_LIST_DEFAULT "\0"
#endif
#ifdef  CONFIG_USE_BOOTARGS
    "bootargs=" CONFIG_BOOTARGS        "\0"
#endif
#ifdef  CONFIG_BOOTCOMMAND
    "bootcmd="  CONFIG_BOOTCOMMAND     "\0"
#endif
#ifdef  CONFIG_RAMBOOTCOMMAND
    "ramboot="  CONFIG_RAMBOOTCOMMAND      "\0"
#endif
#ifdef  CONFIG_NFSBOOTCOMMAND
    "nfsboot="  CONFIG_NFSBOOTCOMMAND      "\0"
#endif
#if defined(CONFIG_BOOTDELAY)
    "bootdelay="    __stringify(CONFIG_BOOTDELAY)   "\0"
#endif
#if defined(CONFIG_BAUDRATE) && (CONFIG_BAUDRATE >= 0)
    "baudrate=" __stringify(CONFIG_BAUDRATE)    "\0"
#endif
#ifdef  CONFIG_LOADS_ECHO
    "loads_echo="   __stringify(CONFIG_LOADS_ECHO)  "\0"
#endif
#ifdef  CONFIG_ETHPRIME
    "ethprime=" CONFIG_ETHPRIME        "\0"
#endif
#ifdef  CONFIG_IPADDR
    "ipaddr="   __stringify(CONFIG_IPADDR)  "\0"
#endif
#ifdef  CONFIG_SERVERIP
    "serverip=" __stringify(CONFIG_SERVERIP)    "\0"
#endif
#ifdef  CONFIG_SYS_AUTOLOAD
    "autoload=" CONFIG_SYS_AUTOLOAD     "\0"
#endif
#ifdef  CONFIG_PREBOOT
    "preboot="  CONFIG_PREBOOT         "\0"
```

```
66  #endif
67  #ifdef  CONFIG_ROOTPATH
68      "rootpath=" CONFIG_ROOTPATH         "\0"
69  #endif
70  #ifdef  CONFIG_GATEWAYIP
71      "gatewayip="    __stringify(CONFIG_GATEWAYIP)   "\0"
72  #endif
73  #ifdef  CONFIG_NETMASK
74      "netmask="  __stringify(CONFIG_NETMASK) "\0"
75  #endif
76  #ifdef  CONFIG_HOSTNAME
77      "hostname=" CONFIG_HOSTNAME "\0"
78  #endif
79  #ifdef  CONFIG_BOOTFILE
80      "bootfile=" CONFIG_BOOTFILE         "\0"
81  #endif
82  #ifdef  CONFIG_LOADADDR
83      "loadaddr=" __stringify(CONFIG_LOADADDR)    "\0"
84  #endif
85  #ifdef  CONFIG_CLOCKS_IN_MHZ
86      "clocks_in_mhz=1\0"
87  #endif
88  #if defined(CONFIG_PCI_BOOTDELAY) && (CONFIG_PCI_BOOTDELAY > 0)
89      "pcidelay=" __stringify(CONFIG_PCI_BOOTDELAY)"\0"
90  #endif
91  #ifdef  CONFIG_ENV_VARS_UBOOT_CONFIG
92      "arch="     CONFIG_SYS_ARCH         "\0"
93  #ifdef CONFIG_SYS_CPU
94      "cpu="      CONFIG_SYS_CPU          "\0"
95  #endif
96  #ifdef CONFIG_SYS_BOARD
97      "board="    CONFIG_SYS_BOARD        "\0"
98      "board_name="   CONFIG_SYS_BOARD        "\0"
99  #endif
100 #ifdef CONFIG_SYS_VENDOR
101     "vendor="   CONFIG_SYS_VENDOR       "\0"
102 #endif
103 #ifdef CONFIG_SYS_SOC
104     "soc="      CONFIG_SYS_SOC          "\0"
105 #endif
106 #endif
107 #if defined(CONFIG_BOOTCOUNT_BOOTLIMIT) &&
(CONFIG_BOOTCOUNT_BOOTLIMIT > 0)
```

```
108     "bootlimit="    __stringify(CONFIG_BOOTCOUNT_BOOTLIMIT)"\0"
109 #endif
110 #ifdef  CONFIG_EXTRA_ENV_SETTINGS
111     CONFIG_EXTRA_ENV_SETTINGS
112 #endif
113     "\0"
114 #else /* CONFIG_USE_DEFAULT_ENV_FILE */
115 #include "generated/defaultenv_autogenerated.h"
116 #endif
117 #ifdef DEFAULT_ENV_INSTANCE_EMBEDDED
118     }
119 #endif
120 };
```

从示例代码 13.3.4.1 的第 13 行可以看出，embedded_environment 是个 env_t 类型的变量，保存在 environment 段里面，env_t 类型如下：

示例代码 13.3.4.2 env_t 结构体

```
102 typedef struct environment_s {
103     uint32_t    crc;          /* CRC32 over data bytes    */
104 #ifdef CONFIG_SYS_REDUNDAND_ENVIRONMENT
105     unsigned char   flags;   /* active/obsolete flags ENVF_REDUND_ */
106 #endif
107     unsigned char   data[ENV_SIZE]; /* Environment data     */
108 } env_t;
```

env_t 结构体中的 crc 为 CRC 值，flags 是标志位，data 数组就是环境变量值。示代码 13.3.4.1 中指定了很多环境变量的默认值，比如 bootcmd 的默认值就是 CONFIG_BOOTCOMMAND，bootargs 的默认值就是 CONFIG_BOOTARGS。我们可以直接在 stm32mp1.h 文件中通过设置宏 CONFIG_BOOTCOMMAND 来设置 bootcmd 的默认值。

13.4.2 bootargs 环境变量

bootargs 保存着 uboot 传递给 Linux 内核的参数，比如指定 Linux 内核所使用的 console、指定根文件系统所在的分区等，如下面 bootargs 环境变量值：

```
console=ttySTM0,115200 root=/dev/mmcblk2p3 rootwait rw
```

1、console

console 用来设置 linux 终端(或者叫控制台)，也就是通过什么设备来和 Linux 进行交互，是串口还是 LCD 屏幕？如果是串口的话应该是串口几等等。一般设置串口作为 Linux 终端，这样我们就可以在电脑上通过 MobaXterm 来和 linux 交互了。这里设置 console 为 ttySTM0，因为 linux 启动以后 STM32MP1 的串口 4 在 linux 下的设备文件就是/dev/ttySTM0，在 Linux 下，一切皆文件。

ttySTM0 后面有个“,115200”，这是设置串口的波特率，console=ttySTM0,115200 综合起来就是设置 ttySTM0（也就是串口 4）作为 Linux 的终端，并且串口波特率设置为 115200。

2、root

root 用来设置根文件系统的位置，root=/dev/mmcblk2p3 用于指明根文件系统存放在

mmcblk2 设备的分区 3 中。正点原子的 STM32MP1 核心板启动 linux 以后会存在/dev/mmcblk1、/dev/mmcblk2 、 /dev/mmcblk1p1 、 /dev/mmcblk1p2 、 /dev/mmcblk2p1 、 /dev/mmcblk2p2 和 /dev/mmcblk2p3 这样的文件，其中/dev/mmcblkx(x=0~n)表示 mmc 设备，而/dev/mmcblkxpy(x=0~n,y=1~n)表示 mmc 设备 x 的分区 y。在 STM32MP1 开发板中/dev/mmcblk2 表示 EMMC，而/dev/mmcblk2p3 表示 EMMC 的分区 3。

root 后面有“rootwait rw”，rootwait 表示等待 mmc 设备初始化完成以后再挂载，否则的话 mmc 设备还没初始化完成就挂载根文件系统会出错的。rw 表示根文件系统是可以读写的，不加 rw 的话可能无法在根文件系统中进行写操作，只能进行读操作。

3、rootfstype

此选项一般配合 root 一起使用，rootfstype 用于指定根文件系统类型，如果根文件系统为 ext 格式的话此选项无所谓。如果根文件系统是 yaffs、jffs 或 ubifs 的话就需要设置此选项，指定根文件系统的类型。

bootargs 常设置的选项就这三个，后面遇到其他选项的话再讲解。

第十四章 U-boot 图形化配置及其原理

在前两章中我们知道 uboot 可以通过 stm32mp15_trusted_defconfig 来配置，或者通过文件 stm32mp1.h 来配置 uboot。还有另外一种配置 uboot 的方法，就是图形化配置，以前的 uboot 是不支持图形化配置，只有 Linux 内核才支持图形化配置。不过不知道从什么时候开始，uboot 也支持图形化配置了，本章我们就来学习一下如何通过图形化配置 uboot，并且学习一下图形化配置的原理，因为后面学习 Linux 驱动开发的时候可能要修改图形配置文件。

14.1 U-Boot 图形化配置体验

uboot 或 Linux 内核可以通过输入“make menuconfig”来打开图形化配置界面，menuconfig 是一套图形化的配置工具，需要 ncurses 库支持。ncurses 库提供了一系列的 API 函数供调用者生成基于文本的图形界面，因此需要先在 Ubuntu 中安装 ncurses 库，命令如下：

```
sudo apt-get install build-essential
sudo apt-get install libncurses5-dev
```

menuconfig 重点会用到两个文件：.config 和 Kconfig，.config 文件前面已经说了，这个文件保存着 uboot 的配置项，使用 menuconfig 配置完 uboot 以后肯定要更新.config 文件。Kconfig 文件是图形界面的描述文件，也就是描述界面应该有什么内容，很多目录下都会有 Kconfig 文件。

在打开图形化配置界面之前，要先使用“make xxx_defconfig”对 uboot 进行一次默认配置，只需要一次即可。如果使用“make clean”清理了工程的话就那就需要重新使用“make xxx_defconfig”再对 uboot 进行一次配置。进入 uboot 根目录，输入如下命令：

```
make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- stm32mp15_atk_trusted_defconfig
make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- menuconfig
```

如果已经在 uboot 的顶层 Makefile 中定义了 ARCH 和 CROSS_COMPILE 的值，那么上述命令可以简化为：

```
make stm32mp15_atk_trusted_defconfig
make menuconfig
```

打开后的图形化界面如图 14.1.1 所示：

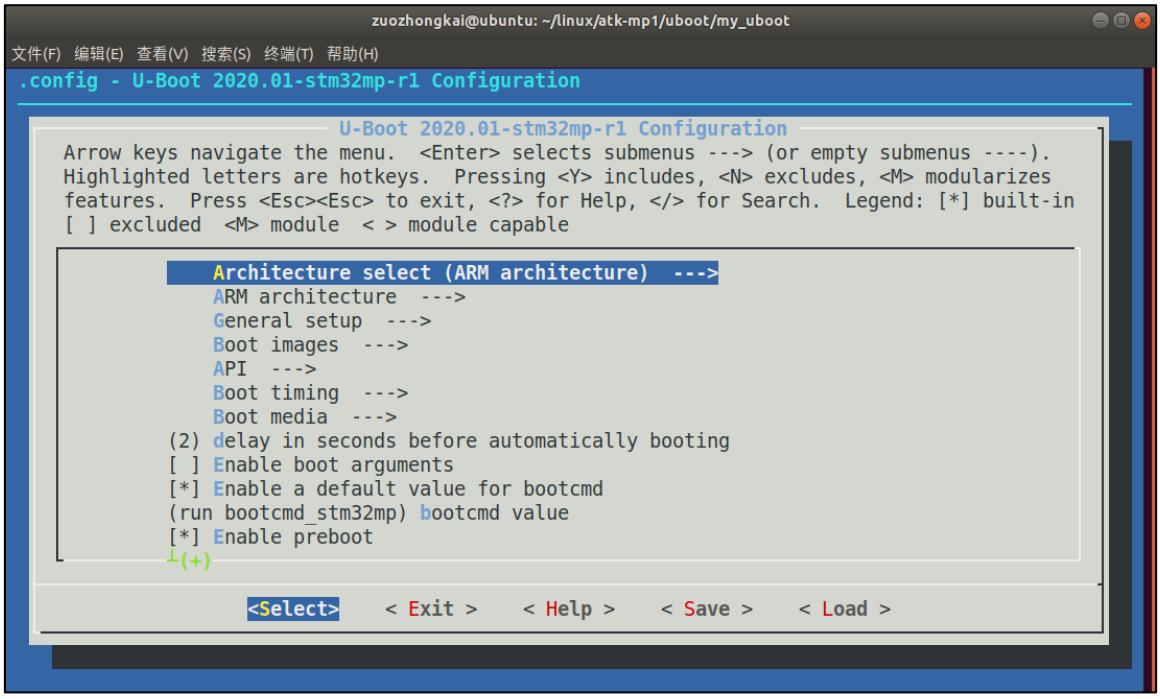


图 14.1.1 uboot 图形化配置界面

图 14.1.1 就是主界面，主界面上方的英文就是简单的操作说明，操作方法如下：

通过键盘上的“↑”和“↓”键来选择要配置的菜单，按下“Enter”键进入子菜单。菜单中高亮的字母就是此菜单的热键，在键盘上按下此高亮字母对应的键可以快速选中对应的菜单。

选中子菜单以后按下“Y”键就会将相应的代码编译进 Uboot 中，菜单前面变为“[*]。按下“N”键不编译相应的代码，按下“M”键就会将相应的代码编译为模块，菜单前面变为“[M]”。按两下“Esc”键退出，也就是返回到上一级，按下“?”键查看此菜单的帮助信息，按下“/”键打开搜索框，可以在搜索框输入要搜索的内容。

在配置界面下方会有五个按钮，这五个按钮的功能如下：

<Select>：选中按钮，和“Enter”键的功能相同，负责选中并进入某个菜单。

<Exit>：退出按钮，和按两下“Esc”键功能相同，退出当前菜单，返回到上一级。

<Help>：帮助按钮，查看选中菜单的帮助信息。

<Save>：保存按钮，保存修改后的配置文件。

<Load>：加载按钮，加载指定的配置文件。

在图 14.1.1 中有很多不同配置的主配置项，通过键盘上的上下键调节配置项。后面跟着“-->”表示此配置项是有子配置项的，按下回车键就可以进入子配置项。

我们就以如何使能 dns 命令为例，讲解一下如何通过图形化界面来配置 uboot。进入“Command line interface --->”这个配置项，此配置项用于配置 uboot 的命令，进入以后如图 14.1.2 所示：

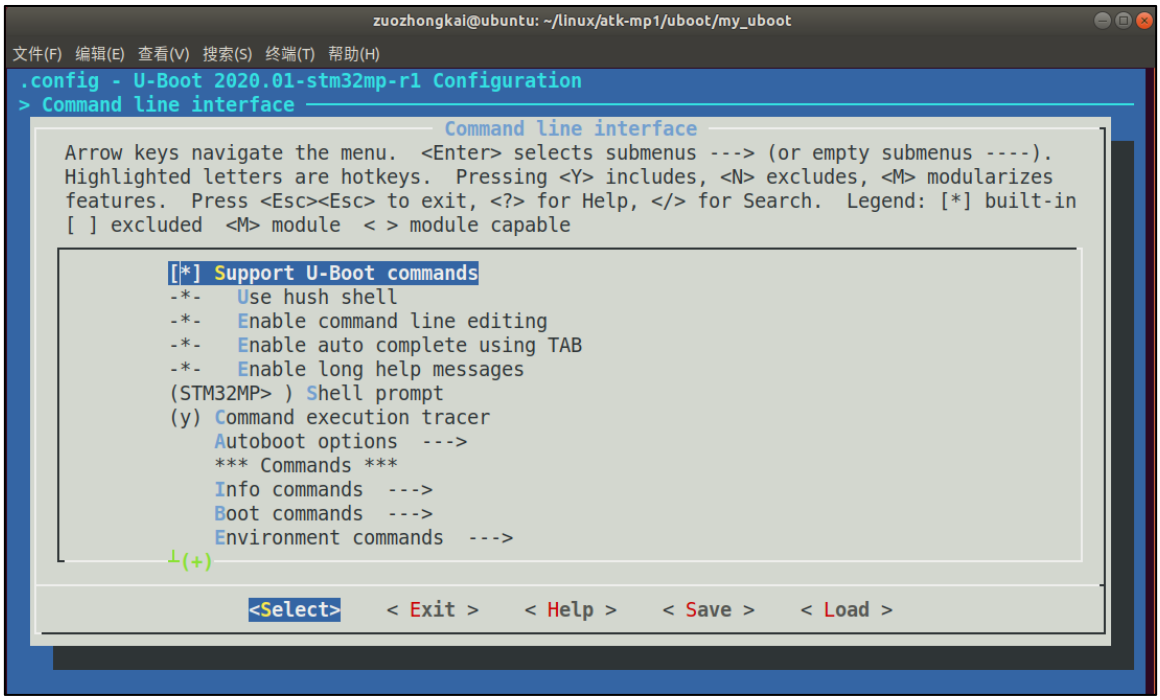


图 14.1.2 Command line interface 配置项

从图 14.1.2 可以看出，有很多配置项，这些配置项也有子配置项，选择“Network commands --->”，进入网络相关命令配置项，如图 14.1.3 所示：

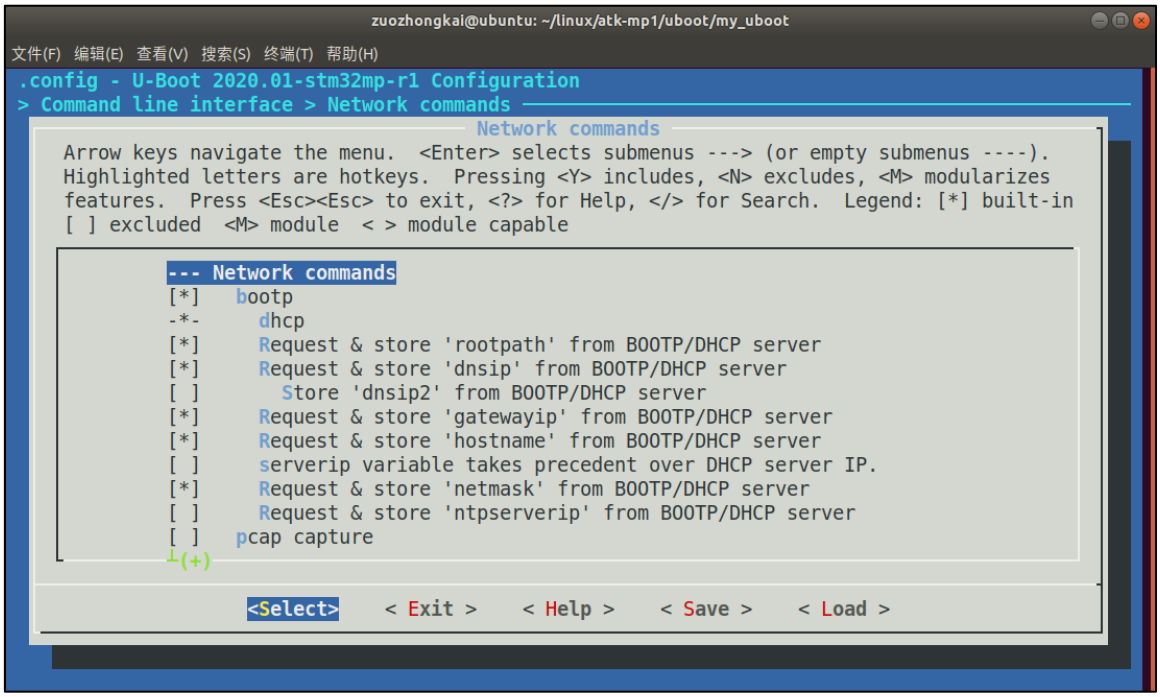


图 14.1.3 Network commands 配置项

从图 14.1.3 可以看出，uboot 中有很多和网络有关的命令，比如 bootp、dhcp、tftpboot 等等。选中 dns，然后按下键盘上的“Y”键，此时 dns 前面的“[ ]”变成了“[*]”，如图 34.1.4 所示：

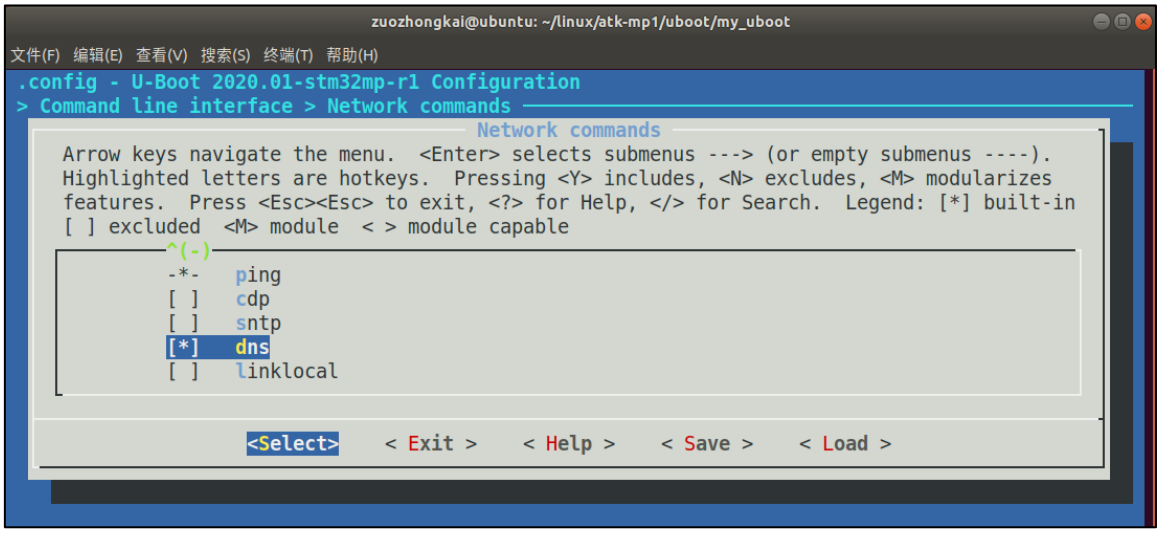


图 14.1.4 选中 dns 命令

每个配置项有 3 种编译选项：编译进 uboot 中(也就是编译进 u-boot.bin 中)、取消编译(也就是不编译这个功能模块)、编译为模块。按下“Y”键表示编译进 uboot 中，此时“[ ]”变成了“[*]”；按下“N”表示不编译，“[ ]”默认表示不编译；有些功能模块是支持编译为模块的，这个一般在 Linux 内核里面很常用，uboot 下面不使用，如果要将某个功能编译为模块，那就按下“M”，此时“[ ]”就会变为“< M >”。

选中 dns，然后按下“H”或者“?”键可以打开 dns 命令的提示信息，如图 14.1.5 所示：

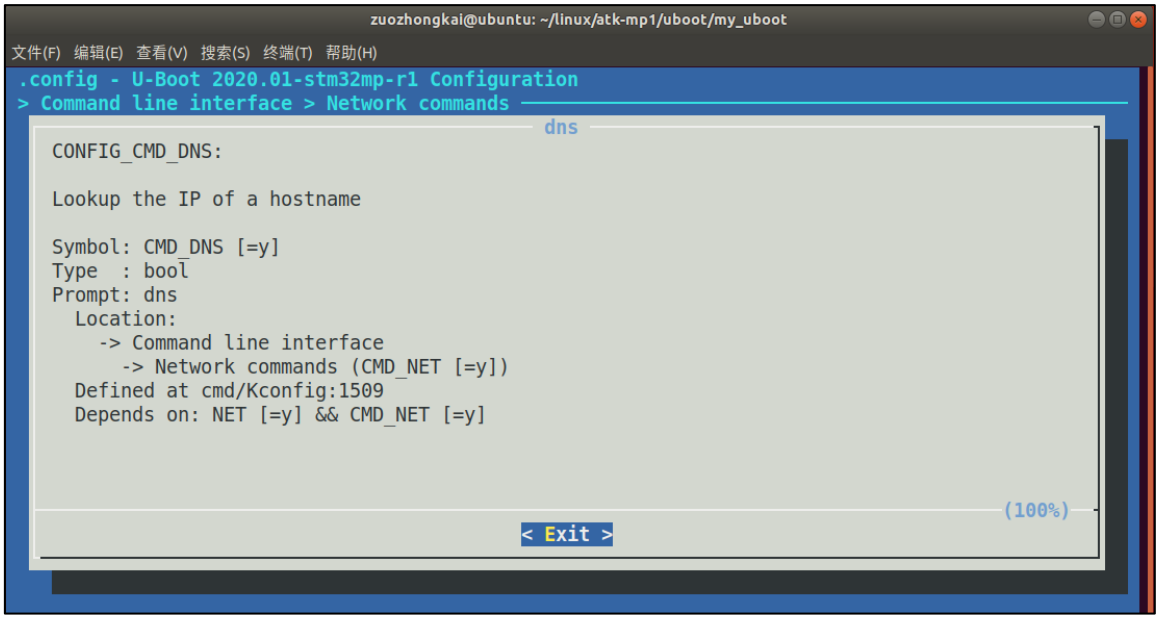


图 14.1.5 dns 命令提示信息

按两下 ESC 键即可退出提示界面，相当于返回上一层。选择 dns 命令以后，按两下 ESC 键（按两下 ESC 键相当于返回上一层），退出当前配置项，进入到上一层配置项。如果没有要修改的就按两下 ESC 键，退出到主配界面，如果也没有其他要修改的，那就再次按两下 ESC 键退出 menuconfig 配置界面。如果修改过配置的话，在退出主界面的时候会有如图 14.1.6 所示提示：

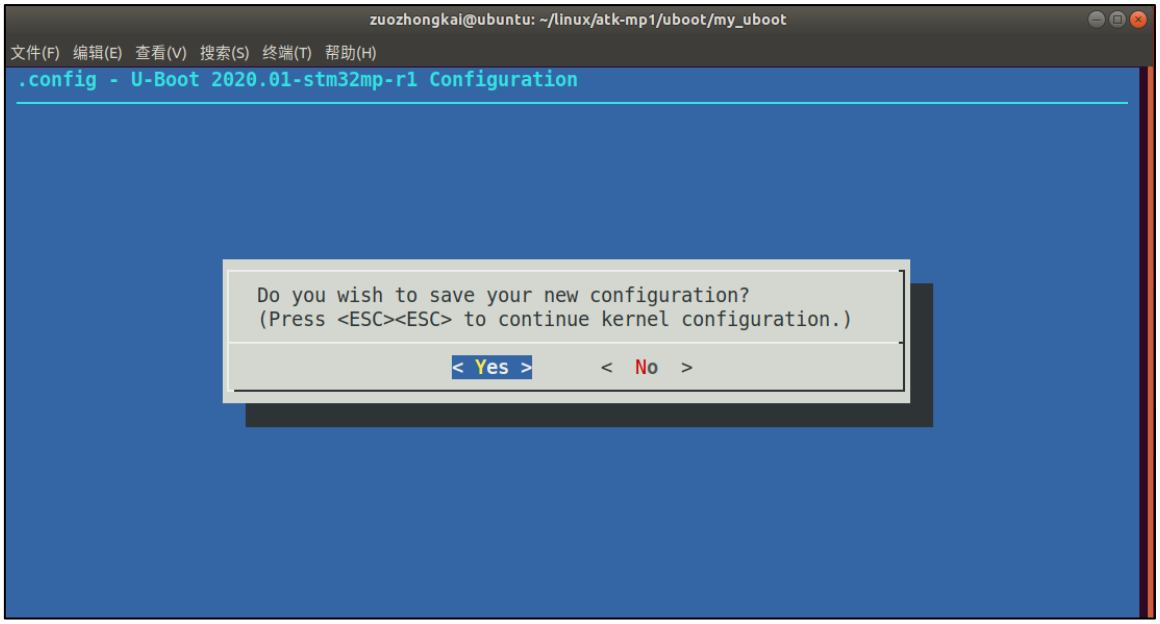


图 14.1.6 是否保存新的配置文件对话框

图 14.1.6 询问是否保存新的配置文件，通过键盘的“←”或“→”键来选择“Yes”项，然后按下键盘上的回车键确认保存。至此，我们就完成了通过图形界面使能了 uboot 的 dns 命令，打开.config 文件，会发现多了“CONFIG_CMD_DNS=y”这一行，如图 14.1.7 中的 323 行所示：

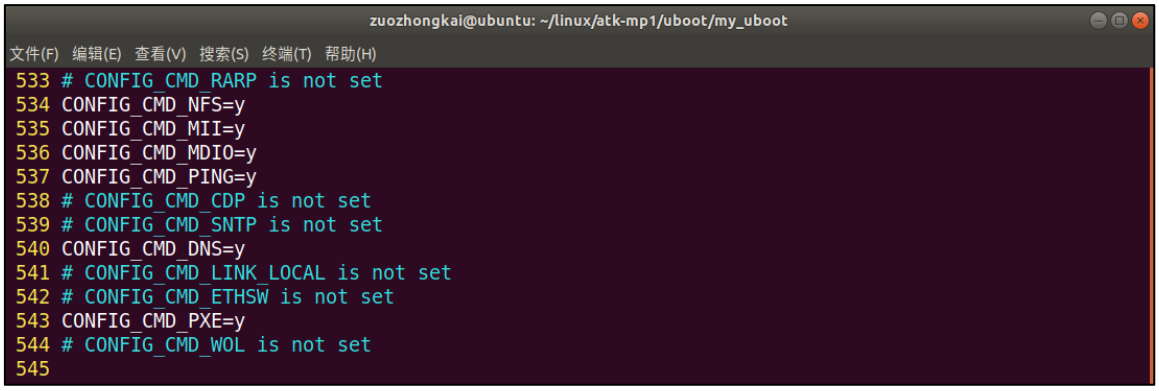


图 14.1.7 .config 文件

使用如下命令编译 uboot：

```
make DEVICE_TREE=stm32mp157d-atk all -j12
```

千万不能使用如下命令：

```
./stm32mp157d_alientek.sh
```

因为 stm32mp157d_alientek.sh 在编译之前会清理工程，会删除掉.config 文件！通过图形化界面配置所有配置项都会被删除，结果就是竹篮打水一场空。

编译完成以后烧写到 SD 卡中，重启开发板进入 uboot 命令模式，输入“?”查看是否有“dns”命令，如果 dns 命令存在的话就会输出相应的使用信息，如图 14.1.8 所示：

```
STM32MP> ? dns
dns - lookup the IP of a hostname

Usage:
dns hostname [envvar]
STM32MP>
```

图 14.1.8 dns 帮助信息

测试一下 dns 命令工作是否正常，使用 dns 命令来查看一下百度官网“www.baidu.com”的 IP 地址。

首先用 dhcp 命令从路由器动态获取一个 IP 地址，防止我们自己设置的静态 IP 地址和网段内其他设备冲突，导致 dns 命令失败，直接输入“dhcp”即可，结果如图 14.1.9 所示：

```
STM32MP> dhcp
BOOTP broadcast 1
BOOTP broadcast 2
DHCP client bound to address 192.168.1.200 (294 ms)
```

图 14.1.9 hdcp 获取 IP 地址

从图 14.1.9 可以看出，此时开发板从路由器获取到的 IP 地址为 192.168.1.200，接下来要设置一下 dns 服务器的 IP 地址，也就是设置环境变量 dnsip 的值，命令如下：

```
setenv dnsip 114.114.114.114
saveenv
```

设置好以后就可以使用 dns 命令查看百度官网的 IP 地址了，输入命令：

```
dns www.baidu.com
```

结果如图 14.1.10 所示：

```
STM32MP> dns www.baidu.com
14.215.177.38
STM32MP>
```

图 14.1.10 dns 命令

从图 14.1.10 可以看出，“www.baidu.com 地址为 14.215.177.38，说明 dns 命令工作正常。这个就是通过图形化命令来配置 uboot，一般用来使能一些命令还是很方便的，这样就不需要到处找命令的配置宏是什么，然后在到配置文件里面去定义。

14.2 menuconfig 图形化配置原理

14.2.1 make menuconfig 过程分析

当输入 make menuconfig 以后会匹配到顶层 Makefile 的如下代码：

示例代码 14.2.1.1 顶层 Makefile 代码段

```
541 %config: scripts_basic outputmakefile FORCE
542     $(Q)$(MAKE) $(build)=scripts/kconfig $@
```

这个在前面已经详细的讲解过了，其中 build=-f ./scripts/Makefile.build obj，将 542 行的规则展开就是：

```
@ make -f ./scripts/Makefile.build obj=scripts/kconfig menuconfig
```

Makefile.build 会读取 scripts/kconfig/Makefile 中的内容，在 scripts/kconfig/Makefile 中可以找到如下代码：

示例代码 14.2.1.2 scripts/kconfig/Makefile 代码段

```
34 menuconfig: $(obj)/mconf
35     $< $(silent) $(Kconfig)
```

其中 obj= scripts/kconfig，silent 是设置静默编译的，在这里可以忽略不计，Kconfig=Kconfig，因此扩展以后就是：

```
menuconfig: scripts/kconfig/mconf
  scripts/kconfig/mconf Kconfig
```

目标 menuconfig 依赖 scripts/kconfig/mconf，因此 scripts/kconfig/mconf.c 这个文件会被编译，生成 mconf 这个可执行文件。目标 menuconfig 对应的规则为 scripts/kconfig/mconf Kconfig，也就是说 mconf 会调用 uboot 根目录下的 Kconfig 文件开始构建图形配置界面。

14.2.2 Kconfig 语法简介

上一小节我们已经知道了 scripts/kconfig/mconf 会调用 uboot 根目录下的 Kconfig 文件开始构建图形化配置界面，接下来简单学习一下 Kconfig 的语法。因为后面学习 Linux 驱动开发的时候可能会涉及到修改 Kconfig，对于 Kconfig 语法我们不需要太深入的去研究，关于 Kconfig 的详细语法介绍，可以参考 linux 内核源码(不知为何 uboot 源码中没有这个文件)中的文件 Documentation/kbuild/kconfig-language.txt，本节我们大概了解其原理即可。打开 uboot 根目录下的 Kconfig，这个 Kconfig 文件就是顶层 Kconfig，我们就以这个文件为例来简单学习一下 Kconfig 语法。

1、mainmenu

顾名思义 mainmenu 就是主菜单，也就是输入“make menuconfig”以后打开的默认界面，在顶层 Kconfig 中有如下代码：

示例代码 14.2.2.1 顶层 Kconfig 代码段

```
5 mainmenu "U-Boot $UBOOTVERSION Configuration"
```

上述代码就是定义了一个名为“U-Boot $UBOOTVERSION Configuration”的主菜单，其中 UBOOTVERSION=2020.01-stm32mp-r1，因此主菜单名为“U-Boot 2020.01-stm32mp-r1 Configuration”，如图 14.2.2.1 所示：

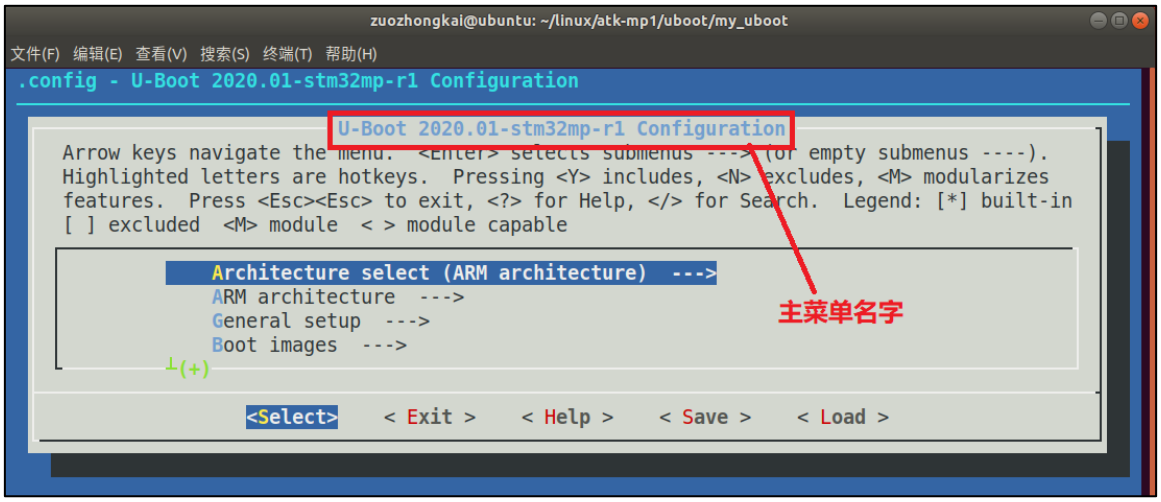


图 14.2.2.1 主菜单名字

2、调用其他目录下的 Kconfig 文件

和 makefile 一样，Kconfig 也可以调用其他子目录中的 Kconfig 文件，调用方法如下：

```
source "xxx/Kconfig"    //xxx 为具体的目录名，相对路径
```

在顶层 Kconfig 中有如下代码：

示例代码 14.2.2.2 顶层 Kconfig 代码段

```
13  source "arch/Kconfig"
......
576
577 source "api/Kconfig"
578
579 source "common/Kconfig"
580
581 source "cmd/Kconfig"
582
583 source "disk/Kconfig"
584
585 source "dts/Kconfig"
586
587 source "env/Kconfig"
588
589 source "net/Kconfig"
590
591 source "drivers/Kconfig"
592
593 source "fs/Kconfig"
```

```
594
595 source "lib/Kconfig"
596
597 source "test/Kconfig"
```

从示例代码 14.2.2.2 中可以看出，顶层 Kconfig 文件调用了很多其他子目录下的 Kcofig 文件，这些子目录下的 Kconfig 文件在主菜单中生成各自的菜单项。

3、menu/endmenu 条目

menu 用于生成菜单，endmenu 就是菜单结束标志，这两个一般是成对出现的。在顶层 Kconfig 中有如下代码：

示例代码 14.2.2.3 顶层 Kconfig 代码段

```
15  menu "General setup"
16
17  config BROKEN
18      bool
19      help
20        This option cannot be enabled. It is used as dependency
21        for broken and incomplete features.
......
298
299 endmenu     # General setup
300
301 menu "Boot images"
302
303 config ANDROID_BOOT_IMAGE
304     bool "Enable support for Android Boot Images"
305     default y if FASTBOOT
306     help
307       This enables support for booting images which use the Android
308       image format header.
......
574
575 endmenu     # Boot images
```

示例代码 14.2.2.3 中有两个 menu/endmenu 代码块，这两个代码块就是两个子菜单，第 15 行的“menu "General setup"”表示子菜单“General setup”。第 301 行的“menu "Boot images"”表示子菜单“Boot images”。体现在主菜单界面中就如图 14.2.2.2 所示：

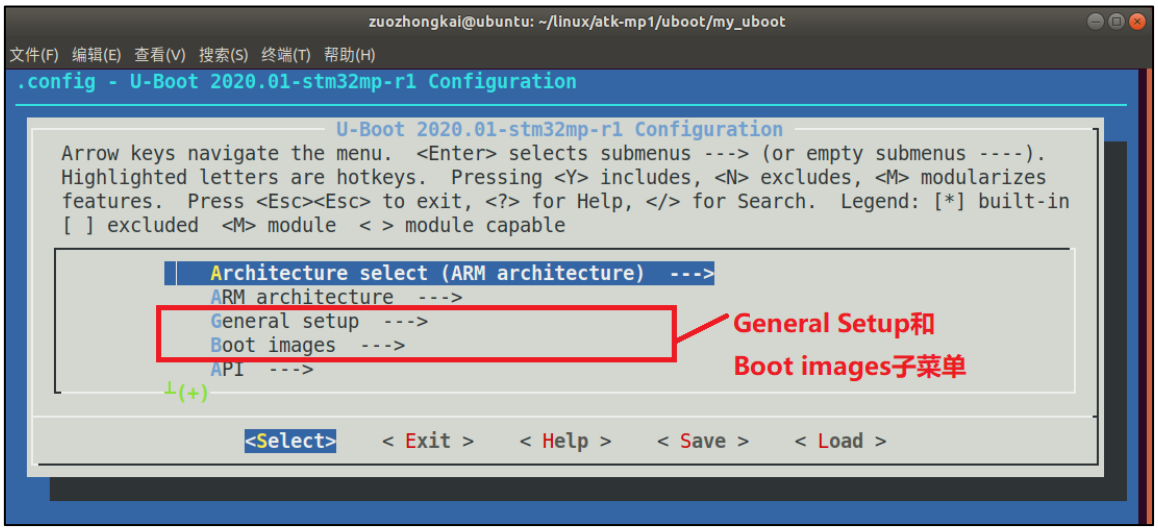


图 14.2.2.2 子菜单

在“General setup”菜单上面还有 “Architecture select (ARM architecture)”和“ARM architecture”这两个子菜单，但是在顶层 Kconfig 中并没有看到这两个子菜单对应的 menu/endmenu 代码块，那这两个子菜单是怎么来的呢？这两个子菜单就是 arch/Kconfig 文件生成的。包括主界面中的“API”、“Boot timing”等等这些子菜单，都是分别由顶层 Kconfig 所调用的 api/Kconfig、common/Kconfig 等这些子 Kconfig 文件来创建的。

3、config 条目

顶层 Kconfig 中的“General setup”子菜单内容如下：

示例代码 14.2.2.4 顶层 Kconfig 代码段

```
15  menu "General setup"
16
17  config BROKEN
18      bool
19      help
20        This option cannot be enabled. It is used as dependency
21        for broken and incomplete features.
22
23  config DEPRECATED
24      bool
25      help
......
29
30  config LOCALVERSION
31      string "Local version - append to U-Boot release"
32      help
......
39
40  config LOCALVERSION_AUTO
41      bool "Automatically append version information to the version
string"
```

```
42      default y
43      help
......
59
60  config CC_OPTIMIZE_FOR_SIZE
61      bool "Optimize for size"
62      default y
63      help
......
68
69  config CC_COVERAGE
70      bool "Enable code coverage analysis"
71      depends on SANDBOX
72      help
......
75
76  config DISTRO_DEFAULTS
77      bool "Select defaults suitable for booting general purpose Linux
distributions"
78      select AUTO_COMPLETE
......
102
103 config ENV_VARS_UBOOT_CONFIG
104     bool "Add arch, board, vendor and soc variables to default
environment"
105     help
......
285 config ERR_PTR_OFFSET
286     hex
287     default 0x0
288     help
......
299 endmenu     # General setup
```

可以看出，在 menu/endmenu 代码块中有大量的“config xxxx”的代码块，这就是 config 条目。config 条目就是“General setup”菜单的具体配置项，如图 14.2.2.3 所示：

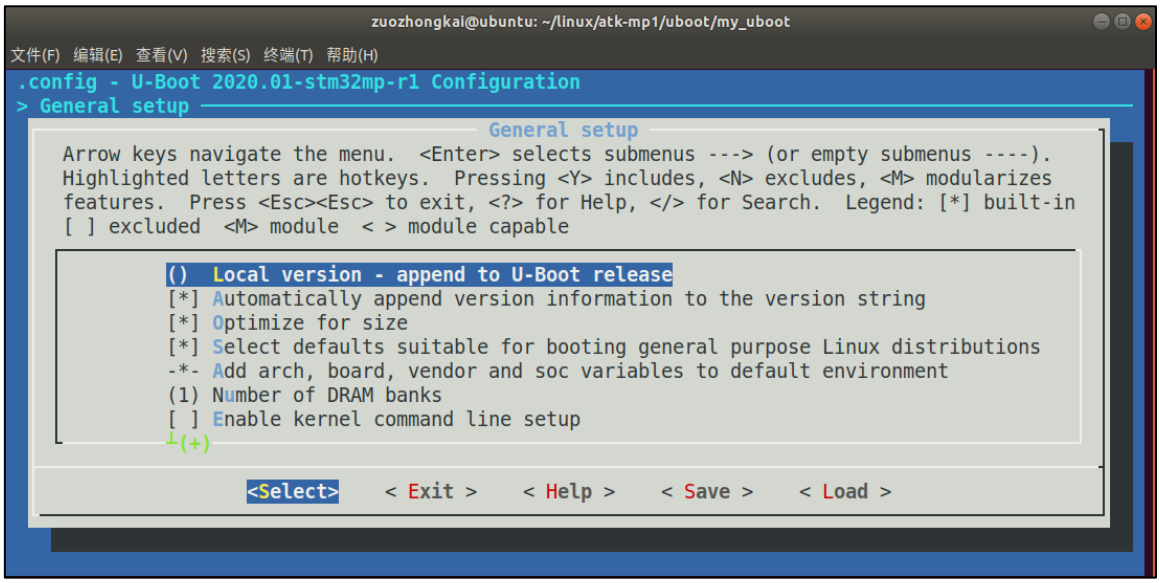


图 14.2.2.3 General setup 配置项

示例代码 14.2.2.4 中第 30 行的“config LOCALVERSION”对应着第一个配置项，“config LOCALVERSION_AUTO”对应着第二个配置项，以此类推。信心的朋友可能会发现，示例代码最前面还有“config BROKEN”和“config DEPRECATED”这两个配置项，但是图 14.2.2.3 中并没有这两个配置项，因为这两个配置项现在不能使能，所以也就看不到。

我们以“config LOCALVERSION”和“config LOCALVERSION_AUTO”这两个为例来分析一下 config 配置项的语法：

示例代码 14.2.2.5 顶层 Kconfig 代码段

```
30 config LOCALVERSION
31  string "Local version - append to U-Boot release"
32  help
33    Append an extra string to the end of your U-Boot version.
34    This will show up in your boot log, for example.
......
38    be a maximum of 64 characters.
39
40 config LOCALVERSION_AUTO
41  bool "Automatically append version information to the version
string"
42  default y
43  help
44    This will try to automatically determine if the current tree is a
45    release tree by looking for Git tags that belong to the current
46    top of tree revision.
......
57
58    which is done within the script "scripts/setlocalversion".)
```

第 30 和 40 行，这两行都以 config 关键字开头，后面跟着 LOCALVERSION 和 LOCALVERSION_AUTO，这两个就是配置项名字。假如我们使能了 LOCALVERSION_AUTO

这个功能，那么就会在.config 文件中生成 CONFIG_LOCALVERSION_AUTO，这个在上一小节讲解如何使能 dns 命令的时候讲过了。由此可知，.config 文件中的“CONFIG_xxx” (xxx 就是具体的配置项名字)就是 Kconfig 文件中 config 关键字后面的配置项名字加上“CONFIG_”前缀。

config 关键字下面的这几行是配置项属性，31~38 行是 LOCALVERSION 的属性，41~58 行是 LOCALVERSION_AUTO 的属性。属性里面描述了配置项的类型、输入提示、依赖关系、帮助信息和默认值等。

第 31 行的 string 是变量类型，也就是“CONFIG_ LOCALVERSION”的变量类型。可以为：bool、tristate、string、hex 和 int，一共 5 种。最常用的是 bool、tristate 和 string 这三种，bool 类型有两种值：y 和 n，当为 y 的时候表示使能这个配置项，当为 n 的时候就禁止这个配置项。tristate 类型有三种值：y、m 和 n，其中 y 和 n 的涵义与 bool 类型一样，m 表示将这个配置项编译为模块。string 为字符串类型，所以 LOCALVERSION 是个字符串变量，用来存储本地字符串，选中以后即可输入用户定义的本地版本号，如图 14.2.2.4 所示：

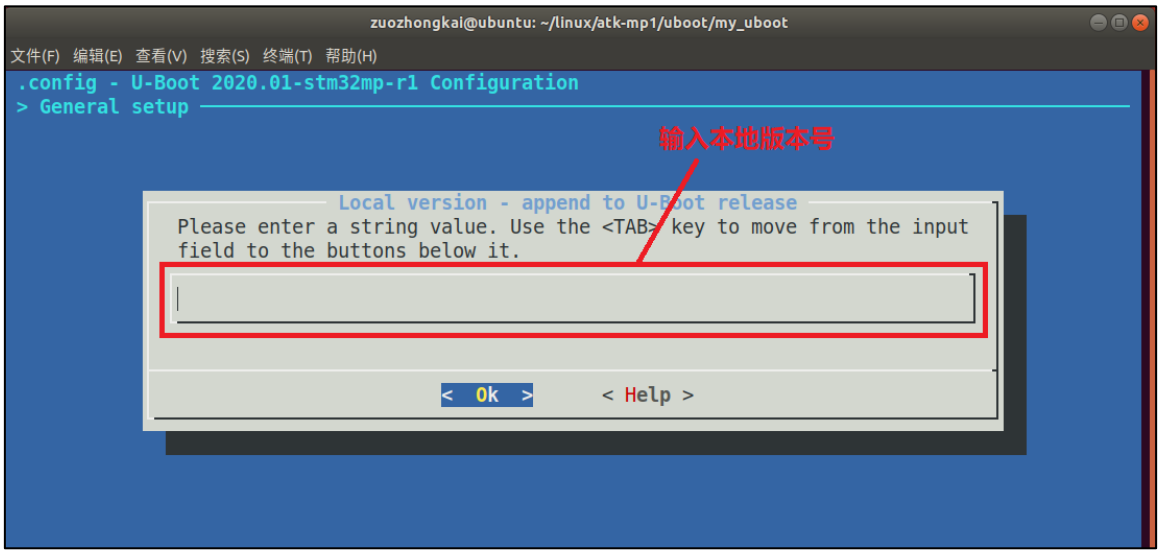


图 14.2.2.4 本地版本号配置

string 后面的“Local version - append to U-Boot release”就是这个配置项在图形界面上的显示出来的标题。

第 32 行，help 表示帮助信息，告诉我们配置项的含义，当我们按下“h”或“?”弹出来的帮助界面就是 help 的内容。

第 41 行，说明“CONFIG_LOCALVERSION_AUTO”是个 bool 类型，可以通过按下 Y 或 N 键来使能或者禁止 CONFIG_LOCALVERSION_AUTO。

第 42 行，“default y”表示 CONFIG_LOCALVERSION_AUTO 的默认值就是 y，所以这一行默认会被选中。

4、depends on 和 select

打开顶层 Kconfig 文件，在里面有这如下代码：

示例代码 14.2.2.6 arch/Kconfig 代码段

```
430 config SPL_FIT_SIGNATURE
431     bool "Enable signature verification of FIT firmware within SPL"
432     depends on SPL_DM
433     select SPL_FIT
```

```
434     select SPL_RSA
```

第 432 行，“depends on”说明“SPL_FIT_SIGNATURE”项依赖于“SPL_DM”,也就是说“SPL_DM”被选中以后“SPL_FIT_SIGNATURE”才能被选中。

第 433~434 行，“select”表示反向依赖，当选中“SPL_FIT_SIGNATURE”以后，“SPL_FIT”和“SPL_RSA”这两个也会被选中。

4、choice/endchoice

在 arch/Kconfig 文件中有如下代码：

示例代码 14.2.2.7 arch/Kconfig 代码段

```
7   choice
8       prompt "Architecture select"
9       default SANDBOX
10
11  config ARC
12      bool "ARC architecture"
......
19
20  config ARM
21      bool "ARM architecture"
......
25
26  config M68K
27      bool "M68000 architecture"
......
32
33  config MICROBLAZE
34      bool "MicroBlaze architecture"
......
37
38  config MIPS
39      bool "MIPS architecture"
......
43
44  config NDS32
45      bool "NDS32 architecture"
46      select SUPPORT_OF_CONTROL
47
48  config NIOS2
49      bool "Nios II architecture"
......
55
56  config PPC
57      bool "PowerPC architecture"
......
```

```
62
63  config RISCV
64      bool "RISC-V architecture"
......
215
216 config XTENSA
217     bool "Xtensa architecture"
218     select CREATE_ARCH_SYMLINK
219     select SUPPORT_OF_CONTROL
220
221 endchoice
```

choice/endchoice 代码段定义了一组可选择项，将多个类似的配置项组合在一起，供用户单选或者多选。示例代码 14.2.2.7 就是选择处理器架构，可以从 ARC、ARM、M68K 等这些架构中选择，这里是单选。在 uboot 图形配置界面上选择“Architecture select”，进入以后如图 14.2.2.5 所示：

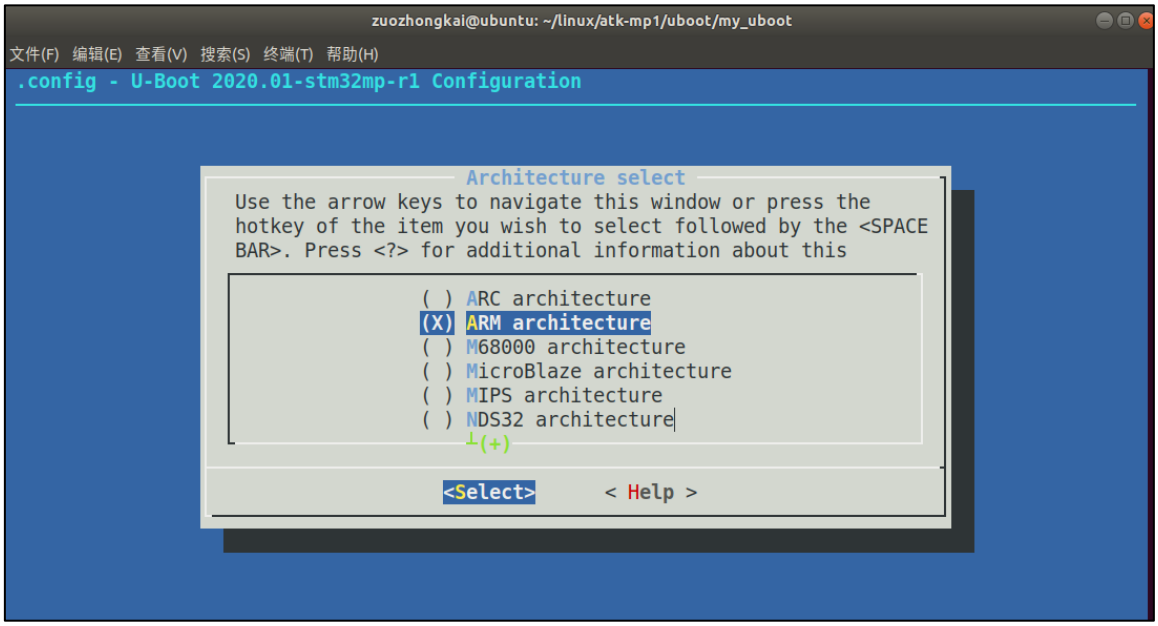


图 14.2.2.5 架构选择界面

可以在图 14.2.2.5 中通过移动光标来选择所使用的 CPU 架构。第 8 行的 prompt 给出这个 choice/endchoice 段的提示信息为“Architecture select”。

5、menuconfig

menuconfig 和 menu 很类似，但是 menuconfig 是个带选项的菜单，其一般用法为：

示例代码 14.2.2.8 menuconfig 用法

```
1 menuconfig MODULES
2     bool "菜单"
3 if MODULES
4 ...
5 endif # MODULES
```

第 1 行，定义了一个可选的菜单 MODULES，只有选中了 MODULES 第 3~5 行 if 到 endif 之间的内容才会显示。在顶层 Kconfig 中有如下代码：

示例代码 14.2.2.9 顶层 Kconfig 代码段

```
15  menu "General setup"
......
187 menuconfig EXPERT
188     bool "Configure standard U-Boot features (expert users)"
189     default y
190     help
191       This option allows certain base U-Boot options and settings
192       to be disabled or tweaked. This is for specialized
193       environments which can tolerate a "non-standard" U-Boot.
194       Use this only if you really know what you are doing.
195
196 if EXPERT
197     config SYS_MALLOC_CLEAR_ON_INIT
198     bool "Init with zeros the memory reserved for malloc (slow)"
199     default y
200     help
201       This setting is enabled by default. The reserved malloc
......
211       should be replaced by calloc - if one expects zeroed memory.
212
213 config TOOLS_DEBUG
214     bool "Enable debug information for tools"
215     help
216       Enable generation of debug information for tools such as
......
220
221 endif # EXPERT
......
298
299 endmenu     # General setup
```

第 187~221 行使用 menuconfig 实现了一个菜单，路径如下：

```
General setup
  -> Configure standard U-Boot features (expert users)  --->
```

如图 14.2.2.6 所示：

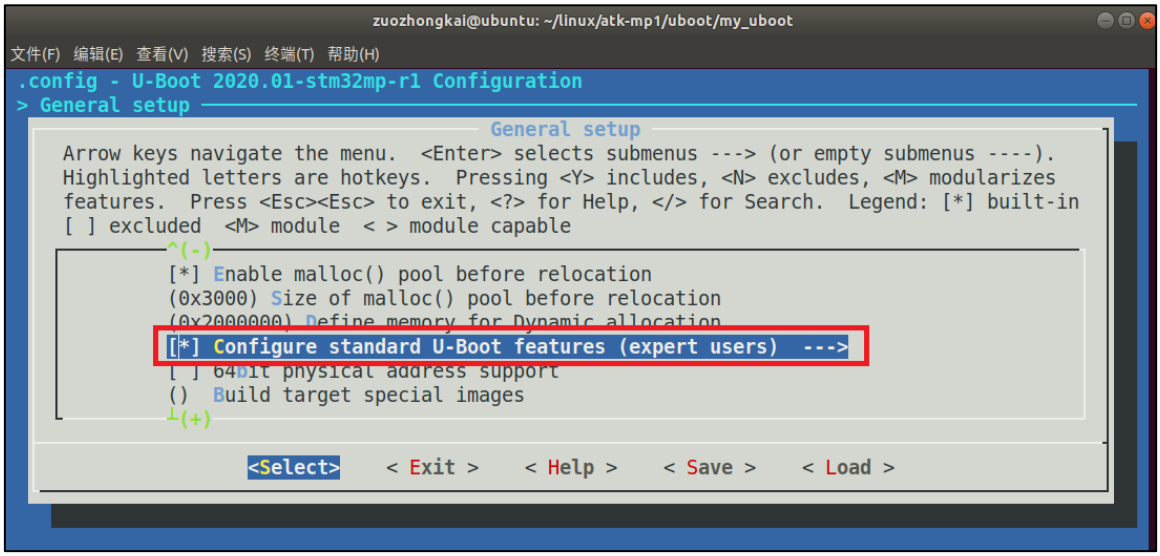


图 14.2.2.6 菜单 Configure standard U-Boot features (expert users)

从图 14.2.2.6 可以看到，前面有“[ ]”说明这个菜单是可选的，当选中这个菜单以后就可以进入到子选项中，也就是示例代码 14.2.2.9 中的第 196~221 行所描述的菜单，如图 14.2.2.7 所示：

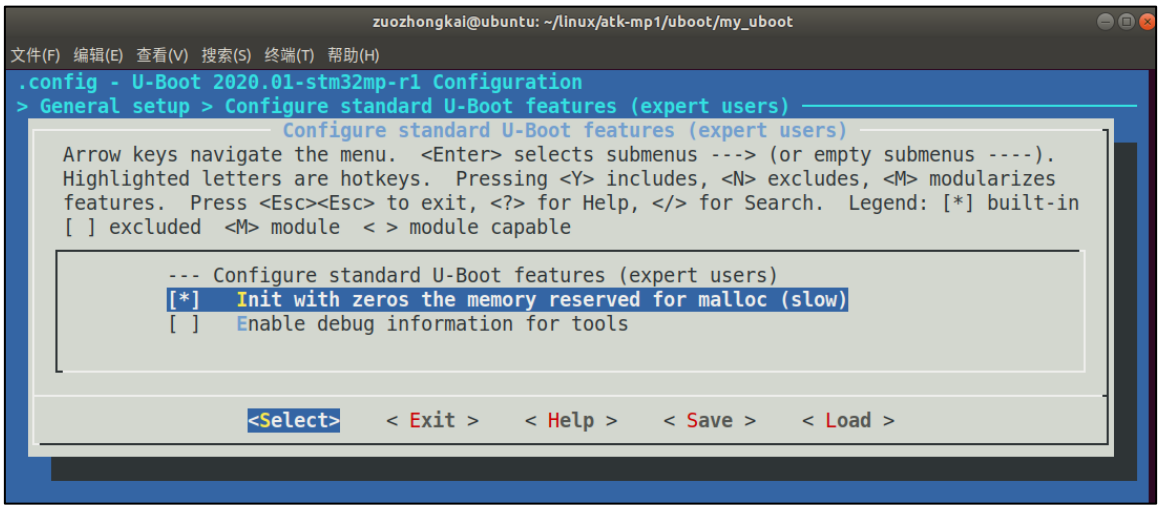


图 14.2.2.7 菜单 Init with zeros the memory reserved for malloc (slow)

如果不选择“Configure standard U-Boot features (expert users)”，那么示例代码 14.2.2.9 中的第 196~221 行所描述的菜单就不会显示出来，进去以后是空白的。

6、comment

comment 用于注释，也就是在图形化界面中显示一行注释，打开文件 drivers/mtd/nand/raw/Kconfig，有如下所示代码：

示例代码 14.2.2.10 drivers/mtd/nand/raw/Kconfig 代码段

```
295 config NAND_STM32_FMC2
296     bool "Support for NAND controller on STM32MP SoCs"
297     depends on ARCH_STM32MP
298     select SYS_NAND_SELF_INIT
299     imply CMD_NAND
```

```
300     help
301       Enables support for NAND Flash chips on SoCs containing the
302       FMC2 NAND controller. This controller is found on STM32MP
303       SoCs. The controller supports a maximum 8k page size and
304       supports a maximum 8-bit correction error per sector of 512
          bytes.
305
306 comment "Generic NAND options"
```

第 306 行使用 comment 标注了一行注释，注释内容为：“Generic NAND options”，这行注释在配置项 NAND_STM32_FMC2 的下面。在图形化配置界面中按照如下路径打开：

```
-> Device Drivers
    -> MTD Support
        -> Raw NAND Device Support
```

结果如图 14.2.2.8 所示：

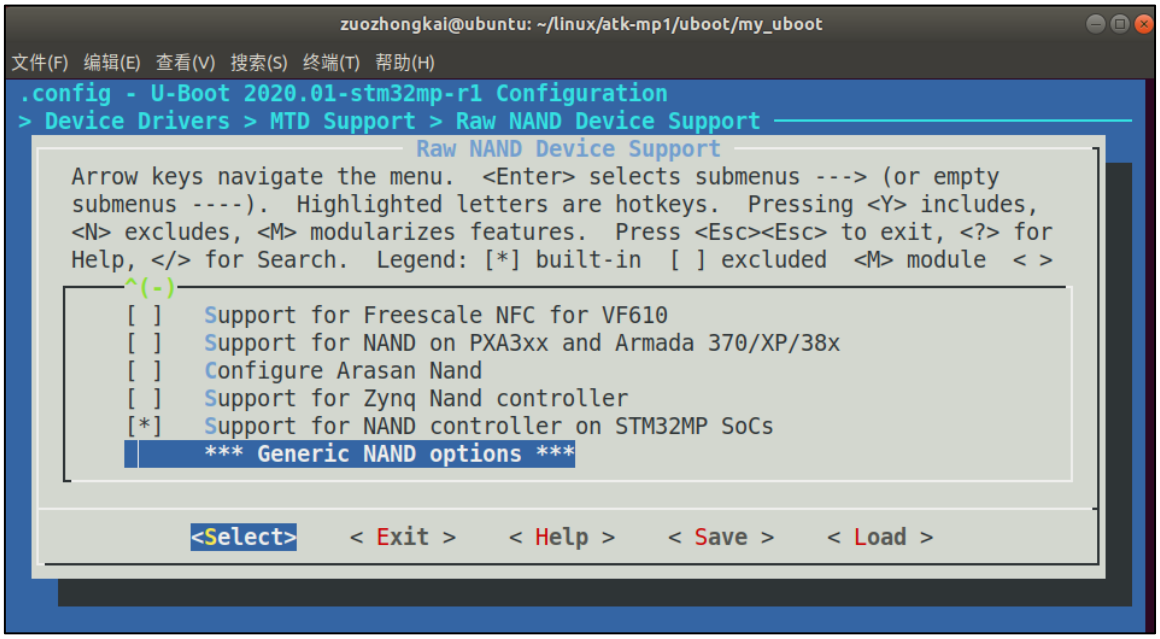


图 14.2.2.8 注释“Generic NAND options”

从图 14.2.2.8 可以看出，在配置项“Support for NAND controller on STM32MP SoCs”下面有一行注释，注释内容为“*** Generic NAND options ***”。

7、source

source 用于读取另一个 Kconfig，比如：

```
source "arch/Kconfig"
```

这个在前面已经讲过了。

Kconfig 语法就讲解到这里，基本上常用的语法就是这些，因为 uboot 相比 Linux 内核要小很多，所以配置项也要少很多，所以建议大家使用 uboot 来学习 Kconfig。一般不会修改 uboot 中的 Kconfig 文件，甚至都不会使用 uboot 的图形化界面配置工具，本小节学习 Kconfig 的目的主要还是为了 Linux 内核作准备。

14.3 添加自定义菜单

图形化配置工具的主要工作就是在.config 下面生成前缀为“CONFIG_”的变量，这些变量一般都有值，为 y，m 或 n，在 uboot 源码里面会根据这些变量来决定编译哪个文件。本小节我们就来学习一下如何添加自己的自定义菜单，自定义菜单要求如下：

①、在主界面中添加一个名为“My test menu”，此菜单内部有一个配置项。

②、配置项为“MY_TESTCONFIG”，此配置项处于菜单“My test menu”中。

③、配置项的为变量类型为 bool，默认值为 y。

④、配置项菜单名字为“This is my test config”。

⑤、配置项的帮助内容为“This is a empty config, just for tset!”。

打开顶层 Kconfig，在最后面加入如下代码：

示例代码 14.3.1 自定义菜单

```
1 menu "My test menu"
2
3 config MY_TESTCONFIG
4   bool "This is my test config"
5   default y
6   help
7     This is a empty config, just for test!
8
9 endmenu       # my test menu
```

添加完成以后打开图形化配置界面，如图 14.3.1 所示：

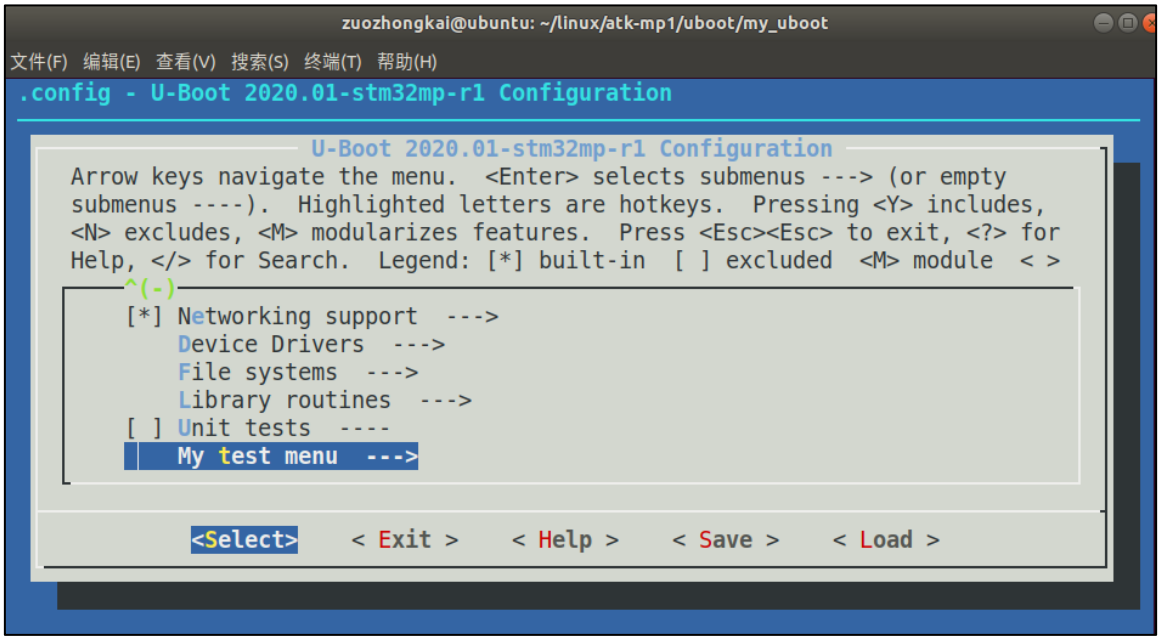


图 14.3.1 主界面

从图 14.3.1 可以看出，主菜单最后面出现了一个名为“My test menu”的子菜单，这个就是我们上面添加进来的子菜单。进入此子菜单，如图 14.3.2 所示：

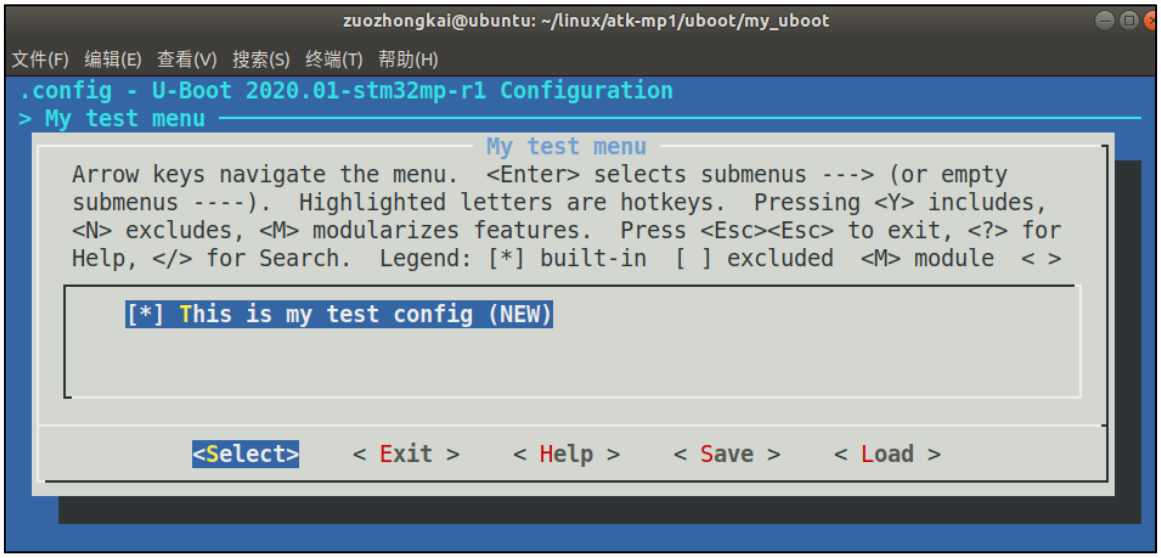


图 14.3.2 “My test menu”子菜单

从图 14.3.2 可以看出，配置项添加成功，选中“This is my test config”配置项，然后按下“H”键打开帮助文档，如图 14.3.3 所示：

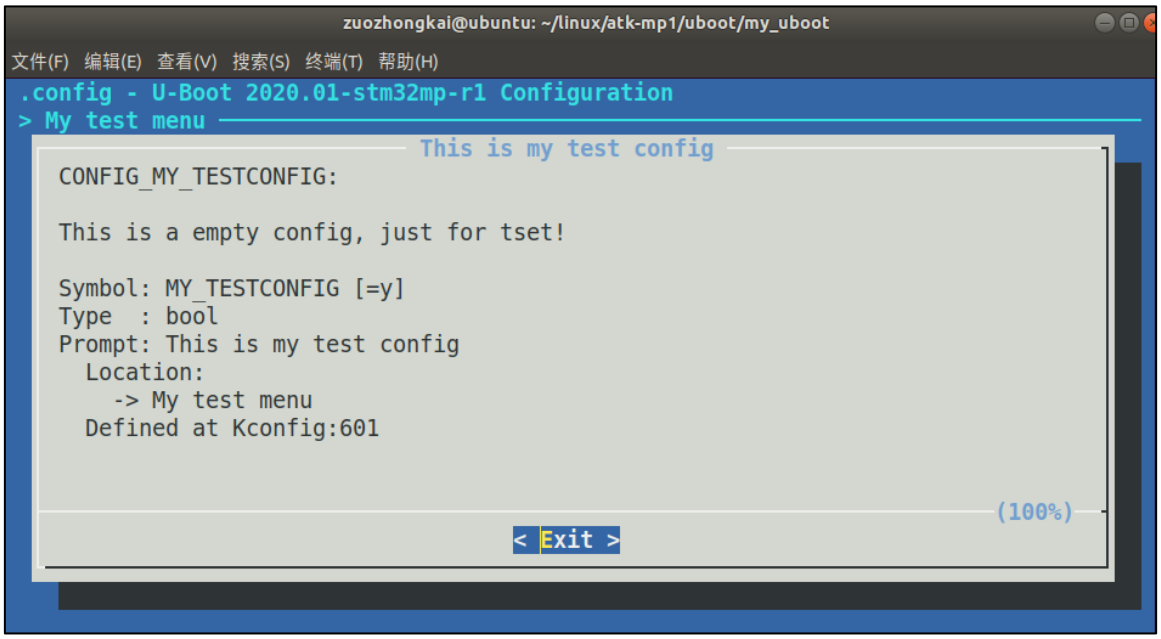


图 14.3.3 帮助信息

从图 14.3.3 可以看出，帮助信息也正确。配置项 MY_TESTCONFIG 默认也是被选中的，因此在.config 文件中肯定会有“CONFIG_MY_TESTCONFIG=y”这一行，如图 14.3.4 所示：

图 14.3.4 .config 文件

至此，我们在主菜单添加自己的自定义菜单就成功了，以后大家如果去半导体原厂工作的话，如果要编写 Linux 驱动，那么很有可能需要你来修改甚至编写 Kconfig 文件。Kconfig 语法

其实不难，重要的点就是 14.2.2 小节中的那几个，最主要的是记住：Kconfig 文件的最终目的就是在.config 文件中生成以“CONFIG_”开头的变量。

第十五章 Linux 内核顶层 Makefile 详解

前几章我们重点讲解了如何移植 uboot 到 STM32MP157 开发板上，从本章开始我们就开始学习如何移植 Linux 内核。同 uboot 一样，在具体移植之前，我们先来学习一下 Linux 内核的顶层 Makefile 文件，因为顶层 Makefile 控制着 Linux 内核的编译流程。

15.1 Linux 内核初次编译

我们先编译一下正点原子 STM32MP157 开发板的出厂 Linux 内核。

1、第三方库安装

编译内核之前需要先在 ubuntu 上安装 lzop 库，否则内核编译会失败！命令如下：

```
sudo apt-get update                    //先更新在安装，防止安装的时候报错
sudo apt-get install lzop
sudo apt-get install libssl-dev
```

2、mkimage 工具安装

STM32MP1 编译出来的 Linux 内核镜像文件为 uImage，这是 uboot 所使用的内核镜像格式，通过在 zImage 镜像的前面添加 0X40 个字节的头部来得到 uImage，这个需要 mkimage 工具来完成此工作。所以我们需要在 Ubuntu 下安装 mkimage 工具，输入如下命令：

```
sudo apt-get install u-boot-tools
```

3、编译 Linux 系统

首先编译一下正点原子出厂的 Linux 源码，在 Ubuntu 下新建一个名为“alientek_linux”的目录存放正点原子出厂 Linux 源码，正点原子出厂 Linux 系统源码已经放到了开发板光盘中，路径为：开发板光盘→1、程序源码→1、正点原子 Linux 出厂系统源码→linux-5.4.31-gb8d3ec3ac-v1.1.tar.bz2，将正点原子出厂 linux 系统源码拷贝到前面在 Ubuntu 下新建的“alientek_linux”目录下，拷贝完成以后使用如下命令解压缩：

```
cd alientek_linux                      //进入到 alientek_linux 目录
tar -vxjf linux-5.4.31-gb8d3ec3ac-v1.1.tar.bz2    //解压缩
```

解压完成以后的如图 15.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$ ls
arch            crypto          ipc       linux-5.4.31-gb8d3ec3ac-v1.1.tar.bz2  samples   virt
block           Documentation   Kbuild    MAINTAINERS                           scripts
certs           drivers         Kconfig   Makefile                              security
CONTRIBUTING.md fs              kernel    mm                                    sound
COPYING         include         lib       net                                   tools
CREDITS         init            LICENSES  README                                usr
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$
```

图 15.1.1 正点原子提供的 Linux 源码根目录

在图 15.1.1 中 Linux 源码根目录下新建名为“stm32mp157d_atk.sh”的 shell 脚本，然后在这个 shell 脚本里面输入如下所示内容：

示例代码 15.1.1 stm32mp157d_atk.sh 文件内容

```
1 #!/bin/sh
2 make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- distclean
3 make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf-
stm32mp1_atk_defconfig
4 make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- menuconfig
5 make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabihf- uImage dtbs
LOADADDR=0XC2000040 -j16
```

第 2 行，执行“make distclean”，清理工程，所以 stm32mp157d_atk.sh 每次都会清理一下工程。如果通过图形界面配置了 Linux，但是还没保存新的配置文件，那么就要慎重使用 stm32mp157d_atk.sh 编译脚本了，因为它会把你的配置信息都删除掉！

第 3 行，执行“make xxx_defconfig”，配置工程，这里使用的配置文件为

stm32mp1_atk_defconfig

第 4 行，执行“make menuconfig”，打开图形配置界面，对 Linux 进行配置，如果不想每次编译都打开图形配置界面的话可以将这一行删除掉。

第 5 行，编译 Linux 内核，后面的“uImage”表示编译 uImage 格式的 Linux 内核，“dtbs”表示编译设备树，LOADADDR 表示 Linux 内核在 DDR 中的加载地址为 0XC2000040。

stm32mp157d_atk.sh 编译脚本每次执行都是先清理整个工程，然后在重新全编译，这个过程很费时间，如果电脑配置差的话要编译很久。除了第一次编译 Linux 源码，我们很少清理工程全编译的，后续驱动开发很少用“make distclean”来清理工程，都是直接“make uImage LOADADDR=0XC2000040”编译内核或者“make dtbs”编译设备树。

使用 chmod 给予 stm32mp157d_atk.sh 可执行权限，然后运行此 shell 脚本，命令如下：

```
chmod 777 stm32mp157d_atk.sh        //给予可执行权限
./stm32mp157d_atk.sh                //执行编译脚本
```

编译的时候会弹出 Linux 图形配置界面，如图 15.1.2 所示：

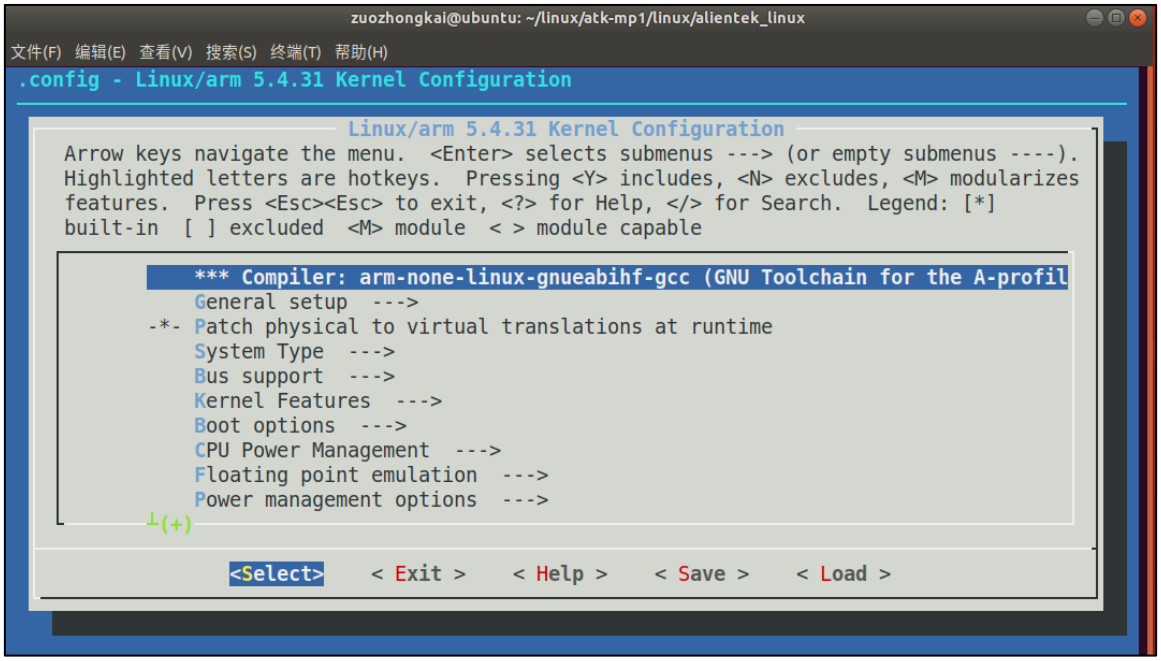


图 15.1.2 Linux 图形配置界面

Linux 的图行界面配置和 uboot 是一样的，这里我们不需要做任何的配置，直接按两下 ESC 键退出，退出图形界面以后会自动开始编译 Linux，等待编译完成。编译完成以后如图 15.1.3 所示：

```
  UIMAGE  arch/arm/boot/uImage
Image Name:   Linux-5.4.31
Created:      Fri Dec 11 15:09:55 2020
Image Type:   ARM Linux Kernel Image (uncompressed)
Data Size:    8133176 Bytes = 7942.55 KiB = 7.76 MiB
Load Address: c2000040
Entry Point:  c2000040
  Kernel: arch/arm/boot/uImage is ready
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$
```

图 15.1.3 Linux 内核编译成功

编译完成以后就会在 arch/arm/boot 这个目录下生成一个叫做 uImage 的文件，uImage 就是我们要用的 Linux 镜像文件，如图 15.1.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux/arch/arm/boot$ ls
bootp       deflate_xip_data.sh  Image       Makefile  zImage
compressed  dts                  install.sh  uImage
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux/arch/arm/boot$
```

图 15.1.4 编译得到 uImage

另外也会在 arch/arm/boot/dts 下生成很多.dtb 文件，这些.dtb 就是设备树文件，我们需要的是 stm32mp157d-atk.dtb 这个文件，如图 15.1.5 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux/arch/arm/boot/dts$ ls stm32m
p157d*
stm32mp157d-atk.dtb       stm32mp157d-atk-spdif.dtb   stm32mp157d-ed1.dtb
stm32mp157d-atk.dts       stm32mp157d-atk-spdif.dts   stm32mp157d-ed1.dts
stm32mp157d-atk.dtsi      stm32mp157d-atk-spdif.dtsi  stm32mp157d-ev1.dtb
stm32mp157d-atk-hdmi.dtb  stm32mp157d-dk1.dtb         stm32mp157d-ev1.dts
stm32mp157d-atk-hdmi.dts  stm32mp157d-dk1.dts
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux/arch/arm/boot/dts$
```

图 15.1.5 编译出来的 dtb 文件

在编译脚本 stm32mp157d_atk.sh 里面，每条编译命令我们都指定 ARCH 和 CROSS_COMPILE 这两个变量的值，如果自己单独运行编译命令的话输入起来会很麻烦，我们可以直接在 Linux 内核源码的 Makefile 文件里面指定 ARCH 和 CROSS_COMPILE 这两个变量的值，打开 Linux 内核源码目录下的主 Makefile，按照如图 15.1.6 所示设置 ARCH 和 CROSS_COMPILE：

```
360 ARCH        ?= $(SUBARCH)
361
362 ARCH = arm
363 CROSS_COMPILE = arm-none-linux-gnueabihf-
364
365 # Architecture as present in compile.h
366 UTS_MACHINE     := $(ARCH)
367 SRCARCH     := $(ARCH)
```

指定ARCH和CROSS_COMPILE的值

图 15.1.6 变量 ARCH 和 CROSS_COMPILE

在主 Makefile 中添加 ARCH 和 CROSS_COMPILE 以后，stm32mp157d_atk.sh 就可以简化为：

示例代码 15.1.2 简化后的 stm32mp157d_atk.sh 文件内容

```
1 #!/bin/sh
2 make distclean
3 make stm32mp1_atk_defconfig
4 make menuconfig
5 make uImage dtbs LOADADDR=0XC2000040 -j16
```

如果要单独编译 uImage 的话可以使用如下命令：

```
make uImage LOADADDR=0XC2000040
```

单独编译设备树的话使用如下命令：

```
make dtbs
```

可以看出，Linux 的编译过程基本和 uboot 一样，都要先执行“make xxx_defconfig”来配置一下，然后在执行“make uImage”进行编译吗，如果需要使用图形界面配置的话就执行“make menuconfig”。

15.2 Linux 工程目录分析

将正点原子提供的 Linux 源码进行解压，解压完成以后的目录如图 15.2.1 所示：

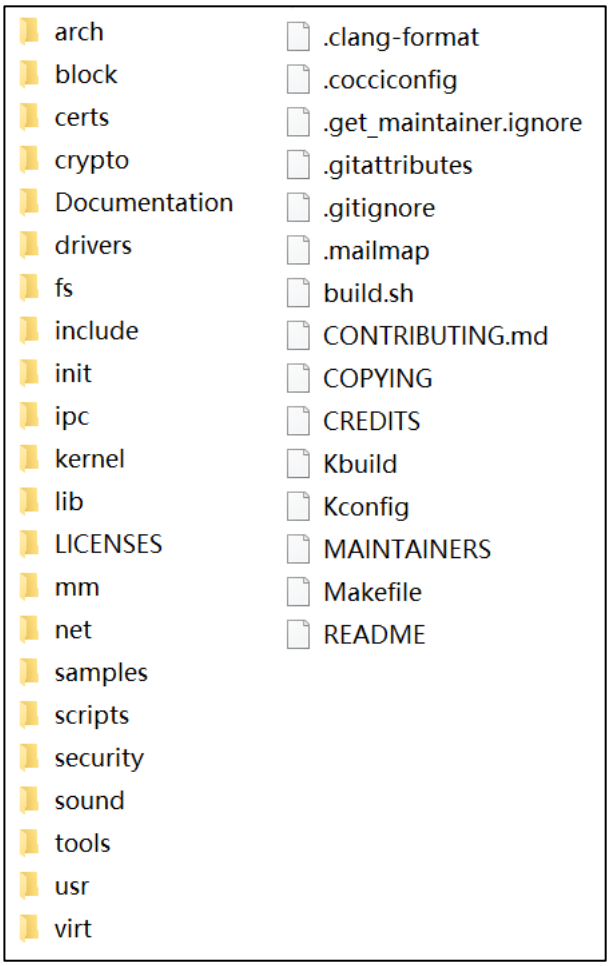


图 15.2.1 未编译的 Linux 源码目录

图 15.2.1 就是正点原子提供的未编译的 Linux 源码目录文件，我们在分析 Linux 之前一定要先在 Ubuntu 中编译一下 Linux，因为编译过程会生成一些文件，而生成的这些恰恰是分析 Linux 不可或缺的文件。编译完成以后使用 tar 压缩命令对其进行压缩并使用 Filezilla 软件将压缩后的 uboot 源码拷贝到 Windows 下。

编译后的 Linux 目录如图 15.2.2 所示：

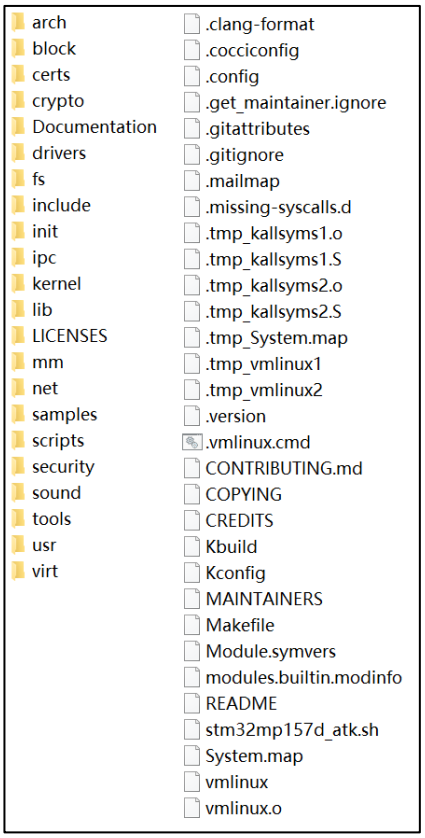


图 15.2.2 编译后的 Linux 目录

图 15.2.2 中重要的文件夹或文件的含义见表 15.2.1 所示：

类型	名字	描述	备注
文件夹	arch	架构相关目录。	Linux 自带
	block	块设备相关目录。	
	crypto	加密相关目录。	
	Documentation	文档相关目录。	
	drivers	驱动相关目录。	
	fs	文件系统相关目录。	
	include	头文件相关目录。	
	init	初始化相关目录。	
	ipc	进程间通信相关目录。	
	kernel	内核相关目录。	
	lib	库相关目录。	
	LICENSES	许可相关目录	
	mm	内存管理相关目录。	
	net	网络相关目录。	
	samples	例程相关目录。	
	scripts	脚本相关目录。	
	security	安全相关目录。	
	sound	音频处理相关目录。	

	tools	工具相关目录。	
	usr	与 initramfs 相关的目录，用于生成 initramfs。	
	virt	提供虚拟机技术(KVM)。	
文件	.config	Linux 最终使用的配置文件。	编译生成的文件。
	.gitignore	git 工具相关文件。	Linux 自带
	.mailmap	邮件列表。	
	.version	好像和版本有关。	
	.vmlinux.cmd	cmd 文件，用于生成 vmlinux。	
	COPYING	版权声明。	Linux 自带
	CREDITS	Linux 贡献者。	
	Kbuild	Makefile 会读取此文件。	
	Kconfig	图形化配置界面的配置文件。。	
	MAINTAINERS	维护者名单。	
	Makefile	Linux 顶层 Makefile	
	Module.xx modules.xx	一系列文件，和模块有关。	编译生成的文件
	README	Linux 描述文件。	Linux 自带
	stm32mp157d_atk.sh	正点原子提供的，Linux 编译脚本。	正点原子提供
	System.map	符号表。	编译生成的文件
	vmlinux	编译出来的、未压缩的 ELF 格式 Linux 文件	
	vmlinux.o	编译出来的 vmlinux.o 文件。	

表 15.2.1 Linux 目录

表 15.2.1 中的很多文件夹和文件我们都不需要去关心，我们要关注的文件夹或文件如下：

1、arch 目录

这个目录是和架构有关的目录，比如 arm、arm64、avr32、x86 等等架构。每种架构都对应一个目录，在这些目录中又有很多子目录，比如 boot、common、configs 等等，以 arch/arm 为例，其子目录如图 15.2.3 所示：

图 15.2.3 arch/arm 子目录

图 15.2.3 是 arch/arm 的一部分子目录，这些子目录用于控制系统引导、系统调用、动态调频、主频设置等。arch/arm/configs 目录是不同平台的默认配置文件：xxx_defconfig，如图 15.2.4 所示：

am200epdkit_defconfig	2021-09-14 18:03	文件	3 KB
aspeed_g4_defconfig	2021-09-14 18:03	文件	7 KB
aspeed_g5_defconfig	2021-09-14 18:03	文件	8 KB
assabet_defconfig	2021-09-14 18:03	文件	2 KB
at91_dt_defconfig	2021-09-14 18:03	文件	6 KB
axm55xx_defconfig	2021-09-14 18:03	文件	6 KB
badge4_defconfig	2021-09-14 18:03	文件	3 KB
bcm2835_defconfig	2021-09-14 18:03	文件	5 KB

图 15.2.4 配置文件

在 arch/arm/configs 中就包含有 STM32MP157 开发板的默认配置文件：imx_v7_defconfig,执行“make stm32mp1_atk_defconfig”即可完成配置。arch/arm/boot/dts 目录里面是对应开发平台的设备树文件，正点原子 STM32MP157 开发板对应的设备树文件如图 15.2.5 所示：

spear13xx_defconfig	2021-09-14 18:03	文件	3 KB
spitz_defconfig	2021-09-14 18:03	文件	6 KB
stm32_defconfig	2021-09-14 18:03	文件	3 KB
stm32mp1_atk_defconfig	2021-09-14 18:03	文件	24 KB
sunxi_defconfig	2021-09-14 18:03	文件	4 KB
tango4_defconfig	2021-09-14 18:03	文件	3 KB
tct_hammer_defconfig	2021-09-14 18:03	文件	2 KB

图 15.2.5 正点原子 STM32MP157 开发板对应的设备树

arch/arm/boot 目录下会保存编译出来的 Image 和 uImage 镜像文件，而 uImage 就是我们要用的 linux 镜像文件。

arch/arm/mach-xxx 目录分别为相应平台的驱动和初始化文件，比如 mach-stm32 目录里面就是 STM32 系列 CPU 的驱动和初始化文件。

2、block 目录

block 是 Linux 下块设备目录，像 SD 卡、EMMC、NAND、硬盘等存储设备就属于块设备，block 目录中存放着管理块设备的相关文件。

3、crypto 目录

crypto 目录里面存放着加密文件，比如常见的 crc、crc32、md4、md5、hash 等加密算法。

4、Documentation 目录

此目录里面存放着 Linux 相关的文档，如果要想了解 Linux 某个功能模块或驱动架构的功能，就可以在 Documentation 目录中查找有没有对应的文档。

5、drivers 目录

驱动目录文件，此目录根据驱动类型的不同，分门别类进行整理，比如 drivers/i2c 就是 I2C 相关驱动目录，drivers/gpio 就是 GPIO 相关的驱动目录，这是我们学习的重点。

6、fs 目录

此目录存放文件系统，比如 fs/ext2、fs/ext4、fs/f2fs 等，分别是 ext2、ext4 和 f2fs 等文件系统。

7、include 目录

头文件目录。

8、init 目录

此目录存放 Linux 内核启动的时候初始化代码。

9、ipc 目录

IPC 为进程间通信，ipc 目录是进程间通信的具体实现代码。

10、kernel 目录

Linux 内核代码。

11、lib 目录

lib 是库的意思，lib 目录都是一些公用的库函。

12、mm 目录

此目录存放内存管理相关代码。

13、net 目录

此目录存放网络相关代码。

14、samples 目录

此目录存放一些示例代码文件。

15、scripts 目录

脚本目录，Linux 编译的时候会用到很多脚本文件，这些脚本文件就保存在此目录中。

16、security 目录

此目录存放安全相关的文件。

17、sound 目录

此目录存放音频相关驱动文件，音频驱动文件并没有存放到 drivers 目录中，而是单独的目录。

18、tools 目录

此目录存放一些编译的时候使用到的工具。

19、usr 目录

此目录存放与 initramfs 有关的代码。

20、virt 目录

此目录存放虚拟机相关文件。

21、.config 文件

跟 uboot 一样，.config 保存着 Linux 最终的配置信息，编译 Linux 的时候会读取此文件中的配置信息。最终根据配置信息来选择编译 Linux 哪些模块，哪些功能。

22、Kbuild 文件

有些 Makefile 会读取此文件。

23、Kconfig 文件

图形化配置界面的配置文件。

24、Makefile 文件

Linux 顶层 Makefile 文件，建议好好阅读一下此文件。

25、README 文件

此文件详细讲解了如何编译 Linux 源码，以及 Linux 源码的目录信息，建议仔细阅读一下此文件。

关于 Linux 源码目录就分析到这里，接下来分析一下 Linux 的顶层 Makefile。

15.3 顶层 Makefile 详解

创建 VScode 的 Linux 内核工程。

Linux 的顶层 Makefile 和 uboot 的顶层 Makefile 非常相似，因为 uboot 参考了 Linux，前 602 行几乎一样，所以前面部分我们大致看一下就行了。

1、版本号

顶层 Makefile 一开始就是 Linux 内核的版本号，如下所示：

示例代码 15.3.1 顶层 Makefile 代码段

```
1 VERSION = 5
2 PATCHLEVEL = 4
3 SUBLEVEL = 31
4 EXTRAVERSION =
```

可以看出，Linux 内核版本号为 5.4.31。

2、MAKEFLAGS 变量

MAKEFLAGS 变量设置如下所示：

示例代码 15.3.2 顶层 Makefile 代码段

```
38 MAKEFLAGS += -rR
```

3、命令输出

Linux 编译的时候也可以通过“V=1”来输出完整的命令，这个和 uboot 一样，相关代码如下所示：

示例代码 15.3.3 顶层 Makefile 代码段

```
75 ifeq ("$(origin V)", "command line")
76   KBUILD_VERBOSE = $(V)
77 endif
78 ifndef KBUILD_VERBOSE
79   KBUILD_VERBOSE = 0
80 endif
81
82 ifeq ($(KBUILD_VERBOSE),1)
83   quiet =
84   Q =
85 else
86   quiet=quiet_
87   Q = @
88 endif
```

4、静默输出

Linux 编译的时候使用“make -s”就可实现静默编译，编译的时候就不会打印任何的信息，同 uboot 一样，相关代码如下：

示例代码 15.3.4 顶层 Makefile 代码段

```
93 ifneq ($(findstring s,$(filter-out --%,$(MAKEFLAGS))),)
94   quiet=silent_
95 endif
96
97 export quiet Q KBUILD_VERBOSE
```

5、设置编译结果输出目录

Linux 编译的时候使用“O=xxx”即可将编译产生的过程文件输出到指定的目录中，相关代码如下：

示例代码 15.3.5 顶层 Makefile 代码段

```
119 # Do we want to change the working directory?
120 ifeq ("$(origin O)", "command line")
121   KBUILD_OUTPUT := $(O)
122 endif
```

6、代码检查

Linux 也支持代码检查，使用命令“make C=1”使能代码检查，检查那些需要重新编译的文件。“make C=2”用于检查所有的源码文件，顶层 Makefile 中的代码如下：

示例代码 15.3.6 顶层 Makefile 代码段

```
202 ifeq ("$(origin C)", "command line")
203   KBUILD_CHECKSRC = $(C)
204 endif
205 ifndef KBUILD_CHECKSRC
206   KBUILD_CHECKSRC = 0
207 endif
```

7、模块编译

Linux 允许单独编译某个模块，使用命令“make M=dir”即可，旧语法“make SUBDIRS=dir”也是支持的。顶层 Makefile 中的代码如下：

示例代码 15.3.7 顶层 Makefile 代码段

```
209 # Use make M=dir or set the environment variable KBUILD_EXTMOD to
210 # specify the directory of external module to build. Setting
    # M= takes precedence.
211 ifeq ("$(origin M)", "command line")
212   KBUILD_EXTMOD := $(M)
213 endif
214
215 export KBUILD_CHECKSRC KBUILD_EXTMOD
216
217 extmod-prefix = $(if $(KBUILD_EXTMOD),$(KBUILD_EXTMOD)/)
218
219 ifeq ($(abs_srctree),$(abs_objtree))
220         # building in the source tree
221         srctree := .
```

```
222     building_out_of_srctree :=
223 else
224         ifeq ($(abs_srctree)/,$(dir $(abs_objtree)))
225                 # building in a subdirectory of the source tree
226                 srctree := ..
227         else
228                 srctree := $(abs_srctree)
229         endif
230     building_out_of_srctree := 1
231 endif
232
233 ifneq ($(KBUILD_ABS_SRCTREE),)
234 srctree := $(abs_srctree)
235 endif
236
237 objtree     := .
238 VPATH       := $(srctree)
239
240 export building_out_of_srctree srctree objtree VPATH
```

外部模块编译过程和 uboot 基本类似，最终导出 srctree、objtree 和 VPATH 这三个变量的值，其中 srctree=.，也就是当前目录，objtree 同样为“.”。

8、设置目标架构和交叉编译器

同 uboot 一样，Linux 编译的时候需要设置目标板架构 ARCH 和交叉编译器 CROSS_COMPILE，在顶层 Makefile 中代码如下：

示例代码 15.3.8 顶层 Makefile 代码段

```
360 ARCH            ?= $(SUBARCH)
```

这里只定义了 ARCH，CROSS_COMPILE 需要编译的时候手动输入值。为了方便，一般直接在顶层 Makefile 中设置 ARCH 和 CROSS_COMPILE，直接将其设置为对应的架构和编译器，比如本教程将 ARCH 设置为为 arm，CROSS_COMPILE 设置为 arm-none-linux-gnueabihf-，如下所示：

示例代码 15.3.9 顶层 Makefile 代码段

```
360 ARCH            ?= arm
361 CROSS_COMPILE   ?= arm-none-linux-gnueabihf-
```

设置好以后我们就可以使用如下命令编译 Linux 了：

```
make xxx_defconfig          //使用默认配置文件配置 Linux
make menuconfig             //启动图形化配置界面
make -j16                   //编译 Linux
```

9、调用 scripts/Kbuild.include 文件

同 uboot 一样，Linux 顶层 Makefile 也会调用文件 scripts/Kbuild.include，顶层 Makefile 相应代码如下：

示例代码 15.3.10 顶层 Makefile 代码段

```
348 # We need some generic definitions (do not try to remake the file).
```

```
333 include scripts/Kbuild.include
```

10、交叉编译工具变量设置

顶层 Makefile 中其他和交叉编译器有关的变量设置如下：

示例代码 15.3.11 顶层 Makefile 代码段

```
407 AS        = $(CROSS_COMPILE)as
408 LD        = $(CROSS_COMPILE)ld
409 CC        = $(CROSS_COMPILE)gcc
410 CPP       = $(CC) -E
411 AR        = $(CROSS_COMPILE)ar
412 NM        = $(CROSS_COMPILE)nm
413 STRIP     = $(CROSS_COMPILE)strip
414 OBJCOPY   = $(CROSS_COMPILE)objcopy
415 OBJDUMP   = $(CROSS_COMPILE)objdump
416 OBJSIZE   = $(CROSS_COMPILE)size
```

LA、LD、CC 等这些都是交叉编译器所使用的工具。

11、头文件路径变量

顶层 Makefile 定义了两个变量保存头文件路径：USERINCLUDE 和 LINUXINCLUDE，相关代码如下：

示例代码 15.3.12 顶层 Makefile 代码段

```
442 USERINCLUDE    := \
443         -I$(srctree)/arch/$(SRCARCH)/include/uapi \
444         -I$(objtree)/arch/$(SRCARCH)/include/generated/uapi \
445         -I$(srctree)/include/uapi \
446         -I$(objtree)/include/generated/uapi \
447                 -include $(srctree)/include/linux/kconfig.h
448
449 # Use LINUXINCLUDE when you must reference the include/ directory.
450 # Needed to be compatible with the O= option
451 LINUXINCLUDE    := \
452         -I$(srctree)/arch/$(SRCARCH)/include \
453         -I$(objtree)/arch/$(SRCARCH)/include/generated \
454         $(if $(building_out_of_srctree),-I$(srctree)/include) \
455         -I$(objtree)/include \
456         $(USERINCLUDE)
```

第 442~447 行的 USERINCLUDE 是 UAPI 相关的头文件路径，第 451~456 行的 LINUXINCLUDE 是 Linux 内核源码的头文件路径。重点来看一下 LINUXINCLUDE，其中 srctree=.，objtree=.，SRACARCH=arm。因此，将 USERINCLUDE 和 LINUXINCLUDE 展开以后为：

```
USERINCLUDE     := \
        -I./arch/arm/include/uapi \
        -Iarch/arm/include/generated/uapi \
        -I./include/uapi \
```

```
            -Iinclude/generated/uapi \
            -include ./include/linux/kconfig.h

LINUXINCLUDE    := \
            -I./arch/arm/include \
            -I./arch/arm/include/generated \
            -I./include \
            -I./arch/arm/include/uapi \
            -Iarch/arm/include/generated/uapi \
            -I./include/uapi \
            -Iinclude/generated/uapi \
            -include ./include/linux/kconfig.h
```

12、导出变量

顶层 Makefile 会导出很多变量给子 Makefile 使用，导出的这些变量如下：

示例代码 15.3.13 顶层 Makefile 代码段

```
474 export ARCH SRCARCH CONFIG_SHELL BASH HOSTCC KBUILD_HOSTCFLAGS
          CROSS_COMPILE AS LD CC
475 export CPP AR NM STRIP OBJCOPY OBJDUMP OBJSIZE PAHOLE LEX YACC AWK
          INSTALLKERNEL
476 export PERL PYTHON PYTHON2 PYTHON3 CHECK CHECKFLAGS MAKE
          UTS_MACHINE HOSTCXX
477 export KBUILD_HOSTCXXFLAGS KBUILD_HOSTLDFLAGS KBUILD_HOSTLDLIBS
          LDFLAGS_MODULE
478
479 export KBUILD_CPPFLAGS NOSTDINC_FLAGS LINUXINCLUDE OBJCOPYFLAGS
          KBUILD_LDFLAGS
480 export KBUILD_CFLAGS CFLAGS_KERNEL CFLAGS_MODULE
481 export CFLAGS_KASAN CFLAGS_KASAN_NOSANITIZE CFLAGS_UBSAN
482 export KBUILD_AFLAGS AFLAGS_KERNEL AFLAGS_MODULE
483 export KBUILD_AFLAGS_MODULE KBUILD_CFLAGS_MODULE
          KBUILD_LDFLAGS_MODULE
484 export KBUILD_AFLAGS_KERNEL KBUILD_CFLAGS_KERNEL
```

15.3.1 make xxx_defconfig 过程

第一次编译 Linux 之前都要先使用“make xxx_defconfig”配置 Linux 内核，在顶层 Makefile 中有“%config”这个目标，如下所示：

示例代码 15.3.1.1 顶层 Makefile 代码段

```
560 include arch/$(SRCARCH)/Makefile
561 export KBUILD_DEFCONFIG KBUILD_KCONFIG CC_VERSION_TEXT
562
563 config: outputmakefile scripts_basic FORCE
564     $(Q)$(MAKE) $(build)=scripts/kconfig $@
```

```
565
566 %config: outputmakefile scripts_basic FORCE
567     $(Q)$(MAKE) $(build)=scripts/kconfig $@
```

第 560 行引用 arch/arm/Makefile 这个文件，这个文件很重要，因为 zImage、uImage 等这些文件就是由 arch/arm/Makefile 来生成的。

第 561 行导出变量 KBUILD_DEFCONFIG、KBUILD_KCONFIG、CC_VERSION_TEXT。

第 563 行，没有目标与之匹配，因此不执行。

第 566 行，“make xxx_defconfig”与目标“%config”匹配，因此执行。“%config”依赖 scripts_basic、outputmakefile 和 FORCE，“%config”真正有意义的依赖就只有 scripts_basic，scripts_basic 的规则如下：

示例代码 15.3.1.2 顶层 Makefile 代码段

```
499 scripts_basic:
500     $(Q)$(MAKE) $(build)=scripts/basic
501     $(Q)rm -f .tmp_quiet_recordmcount
```

build 定义在文件 scripts/Kbuild.include 中，值为 build := -f $(srctree)/scripts/Makefile.build obj，因此将示例代码 15.3.1.2 展开就是：

```
scripts_basic:
        @make -f ./scripts/Makefile.build obj=scripts/basic     //也可以没有@，视配置而定
        @rm -f . tmp_quiet_recordmcount                         //也可以没有@
```

接着回到示例代码 15.3.1.1 的目标“%config”处，内容如下：

```
%config: scripts_basic outputmakefile FORCE
    $(Q)$(MAKE) $(build)=scripts/kconfig $@
```

将命令展开就是：

```
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

15.3.2 Makefile.build 脚本分析

从上一小节可知，“make xxx_defconfig”配置 Linux 的时候如下两行命令会执行脚本 scripts/Makefile.build：

```
@make -f ./scripts/Makefile.build obj=scripts/basic
@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig
```

我们依次来分析一下：

1、scripts_basic 目标对应的命令

scripts_basic 目标对应的命令为：@make -f ./scripts/Makefile.build obj=scripts/basic。打开文件 scripts/Makefile.build，有如下代码：

示例代码 15.3.2.1 Makefile.build 代码段

```
40 kbuild-dir := $(if $(filter /%,$(src)),$(src),$(srctree)/$(src))
41 kbuild-file := $(if $(wildcard $(kbuild-dir)/Kbuild),$(kbuild-dir)/Kb
uild,$(kbuild-dir)/Makefile)
42 include $(kbuild-file)
```

将 kbuild-dir 展开后为：

```
kbuild-dir=./scripts/basic
```

将 kbuild-file 展开后为：

kbuild-file= ./scripts/basic/Makefile

最后将 42 行展开，即：

include ./scripts/basic/Makefile

继续分析 scripts/Makefile.build，如下代码：

示例代码 15.3.2.2 Makefile.build 代码段

```
488 __build: $(if $(KBUILD_BUILTIN),$(builtin-target) $(lib-target)
$(extra-y)) \
489      $(if $(KBUILD_MODULES),$(obj-m) $(modorder-target)) \
490      $(subdir-ym) $(always)
491     @:
```

__build 是默认目标，因为命令“@make -f ./scripts/Makefile.build obj=scripts/basic”没有指定目标，所以会使用到默认目标__build。在顶层 Makefile 中，KBUILD_BUILTIN 为 1，KBUILD_MODULES 为空，因此展开后目标__build 为：

```
__build:$(builtin-target) $(lib-target) $(extra-y)) $(subdir-ym) $(always)
    @:
```

可以看出目标__build 有 5 个依赖：builtin-target、lib-target、extra-y、subdir-ym 和 always。这 5 个依赖的具体内容如下：

```
builtin-target =
lib-target =
extra-y =
subdir-ym =
always = scripts/basic/fixdep scripts/basic/bin2c
```

只有 always 有效，因此__build 最终为：

```
__build: scripts/basic/fixdep scripts/basic/bin2c
    @:
```

__build 依赖于 scripts/basic/fixdep 和 scripts/basic/bin2c，所以要先将 scripts/basic/fixdep 和 scripts/basic/bin2c.c 这两个文件编译成 fixdep 和 bin2c。

综上所述，scripts_basic 目标的作用就是编译出 scripts/basic/fixdep 和 scripts/basic/bin2c 这两个软件。

2、 %config 目标对应的命令

%config 目标对应的命令为：@make -f ./scripts/Makefile.build obj=scripts/kconfig xxx_defconfig。Makefile.build 会读取 scripts/kconfig/Makefile 中的内容，此文件有如下所示内容：

示例代码 15.3.2.3 scripts/kconfig/Makefile 代码段

```
89 %_defconfig: $(obj)/conf
90     $(Q)$< $(silent) --defconfig=arch/$(SRCARCH)/configs/$@
$(Kconfig)
```

目标%_defconfig 与 xxx_defconfig 匹配，所以会执行这条规则，将其展开就是：

```
%_defconfig: scripts/kconfig/conf
    @ scripts/kconfig/conf  --defconfig=arch/arm/configs/%_defconfig  Kconfig
```

%_defconfig 依赖 scripts/kconfig/conf，所以会编译 scripts/kconfig/conf.c 生成 conf 这个软件。此软件就会将%_defconfig 中的配置输出到.config 文件中，最终生成 Linux kernel 根目录下的.config 文件。

15.3.3 make 过程

使用命令“make xxx_defconfig”配置好 Linux 内核以后就可以使用“make”或者“make all”命令进行编译。顶层 Makefile 有如下代码：

示例代码 15.3.3.1 顶层 Makefile 代码段

```
15 PHONY := _all
16 _all:
......
576 PHONY += all
577 ifeq ($(KBUILD_EXTMOD),)
578 _all: all
579 else
580 _all: modules
581 endif
......
631 all: vmlinux
```

第 15 行，_all 是默认目标，如果使用命令“make”编译 Linux 的话此目标就会被匹配。

第 577 行，如果 KBUILD_EXTMOD 为空的话 578 行的代码成立。

第 578 行，默认目标_all 依赖 all。

第 631 行，目标 all 依赖 vmlinux，所以接下来的重点就是 vmlinux！

顶层 Makefile 中有如下代码：

示例代码 15.3.3.2 顶层 Makefile 代码段

```
1037 export KBUILD_VMLINUX_OBJS := $(head-y) $(init-y) $(core-y) \
1038              $(libs-y2)$(drivers-y) $(net-y) $(virt-y)
1039 export KBUILD_VMLINUX_LIBS := $(libs-y1)
1040 export KBUILD_LDS          := arch/$(SRCARCH)/kernel/vmlinux.lds
1041 export LDFLAGS_vmlinux
1042 # used by scripts/Makefile.package
1043 export KBUILD_ALLDIRS := $(sort $(filter-out arch/%,$(vmlinux-
          alldirs)) LICENSES arch include scripts tools)
1044
1045 vmlinux-deps := $(KBUILD_LDS) $(KBUILD_VMLINUX_OBJS)
                    $(KBUILD_VMLINUX_LIBS)
1046
1047 # Recurse until adjust_autoksyms.sh is satisfied
1048 PHONY += autoksyms_recursive
1049 ifdef CONFIG_TRIM_UNUSED_KSYMS
1050 autoksyms_recursive: descend modules.order
1051   $(Q)$(CONFIG_SHELL) $(srctree)/scripts/adjust_autoksyms.sh \
1052     "$(MAKE) -f $(srctree)/Makefile vmlinux"
1053 endif
......
1075 vmlinux: scripts/link-vmlinux.sh autoksyms_recursive $(vmlinux-
```

```
            deps) FORCE
1076        +$(call if_changed,link-vmlinux)
```

从第 1075 行可以看出目标 vmlinux 依赖 scripts/link-vmlinux.sh、autoksyms_recursive、$(vmlinux-deps) 和 FORCE。第 1045 行定义了 vmlinux-deps，值为：

```
vmlinux-deps := $(KBUILD_LDS) $(KBUILD_VMLINUX_OBJS) $(KBUILD_VMLINUX_LIBS)
```

第 1037 行，KBUILD_VMLINUX_OBJS := $(head-y) $(init-y) $(core-y) $(libs-y2) $(drivers-y) $(net-y) $(virt-y)。

第 1039 行，KBUILD_VMLINUX_LIBS := $(libs-y1)。

第 1040 行，KBUILD_LDS:=arch/$(SRCARCH)/kernel/vmlinux.lds，其中 SRCARCH=arm，因此 KBUILD_LDS= arch/arm/kernel/vmlinux.lds。

第 1049 行，我们没有定义宏 CONFIG_TRIM_UNUSED_KSYMS，所以 autoksyms_recursive 为空的。

综上所述，vmlinux 的依赖为：scripts/link-vmlinux.sh、$(head-y) 、$(init-y)、$(core-y) 、$(libs-y2) 、$(drivers-y) 、$(net-y)、$(virt-y)、$(libs-y1)、arch/arm/kernel/vmlinux.lds 和 FORCE。

第 1076 行的命令用于链接生成 vmlinux。

重点来看一下$(head-y) 、$(init-y)、$(core-y) 、$(libs-y2) 、$(drivers-y) 、$(net-y)、$(virt-y)和$(libs-y1)这八个变量的值。

1、head-y

head-y 定义在文件 arch/arm/Makefile 中，内容如下：

示例代码 15.3.3.3 arch/arm/Makefile 代码段

```
144 head-y      := arch/arm/kernel/head$(MMUEXT).o
```

当不使能 MMU 的话 MMUEXT=-nommu，如果使能 MMU 的话为空，因此 head-y 最终的值为：

```
head-y = arch/arm/kernel/head.o
```

2、init-y、drivers-y 和 net-y

在顶层 Makefile 中有如下代码：

示例代码 15.3.3.4 顶层 Makefile 代码段

```
618 init-y    := init/
619 drivers-y := drivers/ sound/
620 drivers-$(CONFIG_SAMPLES) += samples/
621 net-y     := net/
......
1028 init-y   := $(patsubst %/, %/built-in.a, $(init-y))
1030 drivers-y:= $(patsubst %/, %/built-in.a, $(drivers-y))
1031 net-y    := $(patsubst %/, %/built-in.a, $(net-y))
```

从示例代码 15.3.3.4 可知，init-y、drivers-y 和 net-y 最终的值为：

```
init-y    = init/built-in.o
drivers-y = drivers/built-in.a sound/built-in.a samples/built-in.a
net-y     = net/built-in.o
```

3、libs-y1 和 libs-y2

libs-y 基本和 init-y 一样，在顶层 Makefile 中存在如下代码：

示例代码 15.3.3.5 顶层 Makefile 代码段

```
622  libs-y    := lib/
......
1032 libs-y1   := $(patsubst %/, %/lib.a, $(libs-y))
1033 libs-y2   := $(patsubst %/, %/built-in.a, $(filter-out %.a, $(libs-
y)))
```

根据示例代码 15.3.3.5 可知，libs-y1=lib/lib.a，libs-y2=lib/built-in.o”，这个只正确了一部分！因为在 arch/arm/Makefile 中会向 libs-y 中追加一些值，代码如下：

示例代码 15.3.3.6 arch/arm/Makefile 代码段

```
297 libs-y     := arch/arm/lib/ $(libs-y)
```

arch/arm/Makefile 将 libs-y 的值改为了：arch/arm/lib $(libs-y)，展开以后为：

```
libs-y = arch/arm/lib lib/
```

因此根据示例代码 15.3.3.5 的第 1032 和 1033 行可知，libs-y1 和 libs-y2 最终应该为：

```
libs-y1 = arch/arm/lib/lib.a   lib/lib.a
libs-y2 = arch/arm/lib/built-in.o   lib/built-in.o
```

4、virt-y

virt-y 在顶层 Makefile 中有如下代码：

示例代码 15.3.3.7 顶层 Makefile 代码段

```
624  virt-y    := virt/
......
1034 virt-y    := $(patsubst %/, %/built-in.a, $(virt-y))
```

从示例代码 15.3.3.7 可知，virt-y 最终的值为：

```
virt-y= virt/built-in.a
```

5、core-y

core-y 和 init-y 也一样，在顶层 Makefile 中有如下代码：

示例代码 15.3.3.8 顶层 Makefile 代码段

```
623  core-y    := usr/
......
1015 core-y    += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ block/
```

但是在 arch/arm/Makefile 中会对 core-y 进行追加，代码如下：

示例代码 15.3.3.9 arch/arm/Makefile 代码段

```
276 core-$(CONFIG_FPE_NWFPE)    += arch/arm/nwfpe/
277 # Put arch/arm/fastfpe/ to use this.
278 core-$(CONFIG_FPE_FASTFPE)  += $(patsubst $(srctree)/%,%,$(wildcard
                                  $(srctree)/arch/arm/fastfpe/))
279 core-$(CONFIG_VFP)      += arch/arm/vfp/
280 core-$(CONFIG_XEN)      += arch/arm/xen/
281 core-$(CONFIG_KVM_ARM_HOST)     += arch/arm/kvm/
282 core-$(CONFIG_VDSO)     += arch/arm/vdso/
283
284 # If we have a machine-specific directory, then include it in the
      build.
285 core-y            += arch/arm/kernel/ arch/arm/mm/ arch/arm/common/
```

```
286 core-y              += arch/arm/probes/
287 core-y              += arch/arm/net/
288 core-y              += arch/arm/crypto/
289 core-y              += $(machdirs) $(platdirs)
```

第 276~282 行根据不同的配置向 core-y 追加不同的值，比如使能 VFP 的话就会在.config 中有 CONFIG_VFP=y 这一行，那么 core-y 就会追加“arch/arm/vfp/”。

第 285~289 行就是对 core-y 直接追加的值。

在顶层 Makefile 中有如下一行：

示例代码 15.3.3.10 顶层 Makefile 代码段

```
1029 core-y     := $(patsubst %/, %/built-in.a, $(core-y))
```

经过上述代码的转换，最终 core-y 的值为：

```
core-y =  usr/built-in.a                      arch/arm/vfp/built-in.a
          arch/arm/vdso/built-in.a            arch/arm/kernel/built-in.a
          arch/arm/mm/built-in.a              arch/arm/common/built-in.a
          arch/arm/probes/built-in.a          arch/arm/net/built-in.a
          arch/arm/crypto/built-in.a          arch/arm/mach-aspeed/built-in.a
          arch/arm/mach-milbeaut/built-in.a   arch/arm/mach-stm32/built-in.a
          kernel/built-in.a                   certs/built-in.a
          mm/built-in.a                       fs/built-in.a
          ipc/built-in.a                      security/built-in.a
          crypto/built-in.a                   block/built-in.a
```

关于 head-y 、init-y、core-y、libs-y2、drivers-y、net-y、virt-y 和 libs-y1 这 8 个变量就讲解到这里。这些变量都是一些 built-in.o 或.a 等文件，这个和 uboot 一样，都是将相应目录中的源码文件进行编译，然后在各自目录下生成 built-in.o 文件，有些生成了.a 库文件。最终将这些 built-in.o 和.a 文件进行链接即可形成 ELF 格式的可执行文件，也就是 vmlinux！但是链接是需要链接脚本的，vmlinux 的依赖 arch/arm/kernel/vmlinux.lds 就是整个 Linux 的链接脚本。

示例代码 15.3.3.2 第 1076 行的命令“+$(call if_changed,link-vmlinux)”表示将$(call if_changed,link-vmlinux)的结果作为最终生成 vmlinux 的命令，前面的“+”表示该命令结果不可忽略。$(call if_changed,link-vmlinux)是调用函数 if_changed，link-vmlinux 是函数 if_changed 的参数，函数 if_changed 定义在文件 scripts/Kbuild.include 中，如下所示：

示例代码 15.3.3.11 scripts/Kbuild.include 代码段

```
217 if_changed = $(if $(any-prereq)$(cmd-check),                \
218     $(cmd);                                                  \
219     printf '%s\n' 'cmd_$@ := $(make-cmd)' > $(dot-target).cmd, @:)
```

any-prereq 用于检查依赖文件是否有变化，如果依赖文件有变化那么 any-prereq 就不为空，否则就为空。cmd-check 用于检查命令是否有变化，如果没有变化那么 cmd-check 就为空。当命令或者依赖有变化的话就执行命令

第 219 行，打印命令执行过程。比如示例代码 15.3.3.2 中第 1076 行“(call if_changed,link-vmlinux)”，这行命令用于链接 vmlinux。在这里$@就是“link-vmlinux”，因此 cmd_$@表示执行 cmd_link-vmlinux 的内容。cmd_link-vmlinux 在顶层 Makefile 中有如下所示定义：

示例代码 15.3.3.12 顶层 Makefile 代码段

```
1071 cmd_link-vmlinux =                                          \
1072   $(CONFIG_SHELL) $< $(LD) $(KBUILD_LDFLAGS) $(LDFLAGS_vmlinux);\
```

```
1073    $(if $(ARCH_POSTLINK), $(MAKE) -f $(ARCH_POSTLINK) $@, true)
```

第 1071 行就是 cmd_link-vmlinux 的值，其中 CONFIG_SHELL=sh，$<表示目标 vmlinux 的第一个依赖文件，根据示例代码 15.3.3.2 可知，这个文件为 scripts/link-vmlinux.sh。LD= arm-none-linux-gnueabihf-ld -EL，KBUILD_LDFLAGS= -EL。LDFLAGS_vmlinux 的值由顶层 Makefile 和 arch/arm/Makefile 这两个文件共同决定，最终 LDFLAGS_vmlinux=--no-undefined -X --pic-veneer --build-id。因此 cmd_link-vmlinux 最终的值为：

```
cmd_link-vmlinux = sh scripts/link-vmlinux.sh arm-none-linux-gnueabihf-ld -EL--no-undefined -X --pic-veneer --build-id
```

cmd_link-vmlinux 会调用 scripts/link-vmlinux.sh 这个脚本来链接出 vmlinux！在 link-vmlinux.sh 中有如下所示代码：

示例代码 15.3.3.13 scripts/link-vmlinux.sh 代码段

```
58  # Link of vmlinux
59  # ${1} - output file
60  # ${2}, ${3}, ... - optional extra .o files
61  vmlinux_link()
62  {
63      local lds="${objtree}/${KBUILD_LDS}"
64      local output=${1}
65      local objects
66
67      info LD ${output}
68
69      # skip output file argument
70      shift
71
72      if [ "${SRCARCH}" != "um" ]; then
73          objects="--whole-archive          \
74              ${KBUILD_VMLINUX_OBJS}        \
75              --no-whole-archive        \
76              --start-group             \
77              ${KBUILD_VMLINUX_LIBS}        \
78              --end-group           \
79              ${@}"
80
81          ${LD} ${KBUILD_LDFLAGS} ${LDFLAGS_vmlinux}  \
82              -o ${output}              \
83              -T ${lds} ${objects}
84      else
85          objects="-Wl,--whole-archive          \
86              ${KBUILD_VMLINUX_OBJS}        \
87              -Wl,--no-whole-archive        \
88              -Wl,--start-group         \
89              ${KBUILD_VMLINUX_LIBS}        \
```

```
90              -Wl,--end-group                 \
91              ${@}"
92
93          ${CC} ${CFLAGS_vmlinux}                 \
94              -o ${output}                    \
95              -Wl,-T,${lds}                   \
96              ${objects}                  \
97              -lutil -lrt -lpthread
98          rm -f linux
99      fi
100 }
......
305 vmlinux_link vmlinux "${kallsymso}" ${btf_vmlinux_bin_o}
```

vmliux_link 就是最终链接出 vmlinux 的函数，第 72 行判断 SRCARCH 是否等于“um”，如果不相等的话就执 73~83 行的代码。因为 SRCARCH=arm，因此条件成立，执行 73~83 行的代码。重点看一下 81~83 行，这三行代码就应该很熟悉了！就是普通的链接操作，连接脚本为 lds=./arch/arm/kernel/vmlinux.lds。

第 305 行调用 vmlinux_link 函数来链接出 vmlinux。

使用命令“make V=1”编译 Linux，会有如图 15.3.3.1 所示的编译信息：

```
+ arm-none-linux-gnueabihf-ld -EL --no-undefined -X --pic-veneer --build-id -o vmlinux -T ./arch/arm/kernel/vmlinux.lds --whole-archive arch/arm/kernel/head.o init/built-in.a usr/built-in.a arch/arm/vfp/built-in.a arch/arm/vdso/built-in.a arch/arm/kernel/built-in.a arch/arm/mm/built-in.a arch/arm/common/built-in.a arch/arm/probes/built-in.a arch/arm/net/built-in.a arch/arm/crypto/built-in.a arch/arm/mach-aspeed/built-in.a arch/arm/mach-milbeaut/built-in.a arch/arm/mach-stm32/built-in.a kernel/built-in.a certs/built-in.a mm/built-in.a fs/built-in.a ipc/built-in.a security/built-in.a crypto/built-in.a block/built-in.a arch/arm/lib/built-in.a lib/built-in.a drivers/built-in.a sound/built-in.a samples/built-in.a net/built-in.a virt/built-in.a --no-whole-archive --start-group arch/arm/lib/lib.a lib/lib.a --end-group .tmp_kallsyms2.o
```

图 15.3.3.1 link-vmlinux.sh 链接 vmlinux 过程

至此我们基本理清了 make 的过程，重点就是将各个子目录下的 built-in.o、.a 等文件链接在一起，最终生成 vmlinux 这个 ELF 格式的可执行文件。链接脚本为 arch/arm/kernel/vmlinux.lds，链接过程是由 shell 脚本 scripts/link-vmlinux.s 来完成的。接下来的问题就是这些子目录下的 built-in.o、.a 等文件又是如何编译出来的呢？

15.3.4 built-in.o 文件编译生成过程

根据示例代码 15.3.3.2 第 1075 行可知，vmliux 依赖 vmlinux-deps，而 vmlinux-deps= $(KBUILD_LDS) $(KBUILD_VMLINUX_OBJS) $(KBUILD_VMLINUX_LIBS)，KBUILD_LDS 是链接脚本，这里不考虑，剩下的 KBUILD_VMLINUX_OBJS 和 KBUILD_VMLINUX_LIBS 就是各个子目录下的 built-in.o、.a 等文件。最终 vmlinux-deps 的值如下：

```
vmlinux-deps =  arch/arm/kernel/vmlinux.lds     arch/arm/kernel/head.o
                init/built-in.a                 usr/built-in.a
                arch/arm/vfp/built-in.a         arch/arm/vdso/built-in.a
                arch/arm/kernel/built-in.a      arch/arm/mm/built-in.a
                arch/arm/common/built-in.a      arch/arm/probes/built-in.a
                arch/arm/net/built-in.a         arch/arm/crypto/built-in.a
                arch/arm/mach-aspeed/built-in.a arch/arm/mach-milbeaut/built-in.a
                arch/arm/mach-stm32/built-in.a  kernel/built-in.a
```

```
                    certs/built-in.a        mm/built-in.a
                    fs/built-in.a           ipc/built-in.a
                    security/built-in.a     crypto/built-in.a
                    block/built-in.a        arch/arm/lib/built-in.a
                    lib/built-in.a          drivers/built-in.a
                    sound/built-in.a        samples/built-in.a
                    net/built-in.a          virt/built-in.a
                    arch/arm/lib/lib.a      lib/lib.a
```

除了 arch/arm/kernel/vmlinux.lds 以外，其他都是要编译链接生成的。在顶层 Makefile 中有如下代码：

示例代码 15.3.4.1 顶层 Makefile 代码段

```
1082 $(sort $(vmlinux-deps)): descend ;
```

sort 是排序函数，用于对 vmlinux-deps 的字符串列表进行排序，并且去掉重复的单词。可以看出 vmlinux-deps 依赖 descend，descend 也定义在顶层 Makefile 中，定义如下：

示例代码 15.3.4.2 顶层 Makefile 代码段

```
1025 build-dirs   := $(vmlinux-dirs)
......
1688 descend: $(build-dirs)
1689 $(build-dirs): prepare
1690    $(Q)$(MAKE) $(build)=$@ \
1691    single-build=$(if $(filter-out $@/, $(single-no-ko)),1) \
1692    need-builtin=1 need-modorder=1
```

可以看出 descend 又依赖于 build-dirs，从 1025 行可以看出，build-dirs 就是 vmlinux-dirs。vmlinux-dirs 看名字就知道和目录有关，此变量保存着生成 vmlinux 所需源码文件的目录，值如下::

```
vmlinux-dirs =  init                    usr                     arch/arm/vfp
                arch/arm/vdso           arch/arm/kernel         arch/arm/mm
                arch/arm/common         arch/arm/probes         arch/arm/net
                arch/arm/crypto         arch/arm/mach-aspeed    arch/arm/mach-milbeaut
                arch/arm/mach-stm32     kernel                  certs
                mm                      fs                      ipc
                security                crypto                  block
                drivers                 sound                   samples
                net                     arch/arm/lib            lib
                virt
```

示例代码 15.3.4.2 中第 1689 行，目标 build-dirs 依赖 prepare，这个依赖不去浪费时间了，重点看一下第 1690 行的命令。build 前面已经说了，值为“-f ./scripts/Makefile.build obj”，因此将 1690 行的命令展开就是：

```
@ make -f ./scripts/Makefile.build obj=$@
```

$@表示目标文件，也就是 build-dirs 的值，而 build-dirs 就是 vmlinux-dirs，将 vmlinux-dirs 中的这些目录全部带入到命令中，结果如下：

```
@ make -f ./scripts/Makefile.build obj=init
@ make -f ./scripts/Makefile.build obj=usr
```

```
@ make -f ./scripts/Makefile.build obj=arch/arm/vfp
@ make -f ./scripts/Makefile.build obj=arch/arm/vdso
@ make -f ./scripts/Makefile.build obj=arch/arm/kernel
@ make -f ./scripts/Makefile.build obj=arch/arm/mm
@ make -f ./scripts/Makefile.build obj=arch/arm/common
@ make -f ./scripts/Makefile.build obj=arch/arm/probes
@ make -f ./scripts/Makefile.build obj=arch/arm/net
……
@ make -f ./scripts/Makefile.build obj=virt
```

这些命令运行过程其实都是一样的，我们就以“@ make -f ./scripts/Makefile.build obj=init”这个命令为例，讲解一下详细的运行过程。这里又要用到 Makefile.build 这个脚本了，此脚本默认目标为__build，这个在 15.3.2 小节已经讲过了，我们再来看一下，__build 目标对应的规则如下：

示例代码 15.3.4.4 scripts/Makefile.build 代码段

```
488 __build: $(if $(KBUILD_BUILTIN),$(builtin-target) $(lib-target)
$(extra-y)) \
489   $(if $(KBUILD_MODULES),$(obj-m) $(modorder-target)) \
490   $(subdir-ym) $(always)
491  @:
```

当只编译 Linux 内核镜像文件，也就是使用“make uImage”编译的时候，KBUILD_BUILTIN=1，KBUILD_MODULES 为空。“make”命令是会编译所有的东西，包括 Linux 内核镜像文件和一些模块文件。如果只编译 Linux 内核镜像的话，__build 目标简化为：

```
__build: $(builtin-target) $(lib-target) $(extra-y)) $(subdir-ym) $(always)
  @:
```

重点来看一下 builtin-target 这个依赖，builtin-target 同样定义在文件 scripts/Makefile.build 中，定义如下：

示例代码 15.3.4.5 scripts/Makefile.build 代码段

```
69 ifneq ($(strip $(real-obj-y) $(need-builtin)),)
70 builtin-target := $(obj)/built-in.a
71 endif
```

第 70 行就是 builtin-target 变量的值，为“$(obj)/built-in.o”，这就是这些 built-in.o 的来源了。要生成 built-in.o，要求 real-obj-y 和 need-builtin 这些变量不能全部为空。

15.3.5 make zImage 过程

1、vmlinux、Image，zImage、uImage 的区别

前面几小节重点是讲 vmlinux 是如何编译出来的，vmlinux 是 ELF 格式的文件，但是在实际中我们不会使用 vmlinux，而是使用 zImage 或 uImage 这样的 Linux 内核镜像文件。那么 vmlinux、zImage、uImage 他们之间有什么区别呢？

①、vmlinux 是编译出来的最原始的内核文件，是未压缩的，比如正点原子提供的 Linux 源码编译出来的 vmlinux 差不多有 260 多 MB，如图 15.3.5.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$ ls vmlinux -l
-rwxrwxr-x 1 zuozhongkai zuozhongkai 261346772 10月  28 18:47 vmlinux
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$
```

图 15.3.5.1 vmlinux 信息

②、Image 是 Linux 内核镜像文件，但是 Image 仅包含可执行的二进制数据。Image 就是使用 objcopy 取消掉 vmlinux 中的一些其他信息，比如符号表什么的。但是 Image 是没有压缩过的，Image 保存在 arch/arm/boot 目录下，其大小大概在 19MB 左右，如图 15.3.5.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$ ls arch/arm/boot/Image  -l
-rwxrwxr-x 1 zuozhongkai zuozhongkai 19835216 10月  28 18:48 arch/arm/boot/Image
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$
```

图 15.3.5.2 Image 镜像信息

相比 vmlinux 的 260MB，Image 缩小到了 19MB。

③、zImage 是经过 gzip 压缩后的 Image，经过压缩以后其大小大概在 8MB 左右，如图 15.3.5.3 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$ ls arch/arm/boot/zImage -l
-rwxrwxr-x 1 zuozhongkai zuozhongkai 8133168 10月  28 18:48 arch/arm/boot/zImage
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$
```

图 15.3.5.3 zImage 镜像信息

④、uImage 是老版本 uboot 专用的镜像文件，uImag 是在 zImage 前面加了一个长度为 64 字节的“头”，这个头信息描述了该镜像文件的类型、加载位置、生成时间、大小等信息。uImage 大小如图 15.3.5.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$ ls arch/arm/boot/uImage -l
-rw-r--r-- 1 zuozhongkai zuozhongkai 8133232 10月  28 18:48 arch/arm/boot/uImage
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/alientek_linux$
```

图 15.3.5.4 uImage 镜像大小

图 15.3.5.3 中 zImage 大小为 8133168 字节，图 15.3.5.4 中 uImage 大小为 8133232 字节，差距就是 8133232-8133168=64 字节，这 64 字节就是头部信息。

使用“make”、“make all”、“make uImage”这些命令就可以编译出 uImage 镜像，在 arch/arm/Makefile 中有如下代码：

示例代码 15.3.5.1 顶层 Makefile 代码段

```
333 BOOT_TARGETS    = zImage Image xipImage bootpImage uImage
334 INSTALL_TARGETS = zinstall uinstall install
335
336 PHONY += bzImage $(BOOT_TARGETS) $(INSTALL_TARGETS)
337
338 bootpImage uImage: zImage
339 zImage: Image
340
341 $(BOOT_TARGETS): vmlinux
342     $(Q)$(MAKE) $(build)=$(boot) MACHINE=$(MACHINE) $(boot)/$@
343     @$(kecho) '  Kernel: $(boot)/$@ is ready'
```

第 333 行，变量 BOOT_TARGETS 包含 zImage，Image，xipImage，bootpImage 和 uImage。

第 341 行，BOOT_TARGETS 依赖 vmlinux，因此如果使用“make uImage”编译的 Linux 内核的话，首先肯定要先编译出 vmlinux。

第 342 行，具体的命令，比如要编译 uImage，那么命令展开以后如下所示：

```
@ make -f ./scripts/Makefile.build obj=arch/arm/boot MACHINE=arch/arm/boot/uImage
```

看来又是使用 scripts/Makefile.build 文件来完成 vmlinux 到 uImage 的转换。

关于 Linux 顶层 Makefile 就讲解到这里，基本和 uboot 的顶层 Makefile 一样，重点在于 vmlinux 的生成。最后将 vmlinux 压缩成我们最常用的 uImage 或 zImage 等文件。

第十六章 Linux 内核启动流程

看完 Linux 内核的顶层 Makefile 以后再来看 Linux 内核的大致启动流程，Linux 内核的启动流程要比 uboot 复杂的多，涉及到的内容也更多，因此本章我们就大致的了解一下 Linux 内核的启动流程。

16.1 链接脚本 vmlinux.lds

要分析 Linux 启动流程，同样需要先编译一下 Linux 源码，因为有很多文件是需要编译才会生成的。首先分析 Linux 内核的连接脚本文件 arch/arm/kernel/vmlinux.lds，通过链接脚本可以找到 Linux 内核的第一行程序是从哪里执行的。vmlinux.lds 中有如下代码(有省略)：

示例代码 16.1.1 vmlinux.lds 代码段

```
1  OUTPUT_ARCH(arm)
2  ENTRY(stext)
3  jiffies = jiffies_64;
4  SECTIONS
5  {
6       /DISCARD/ : {
7       *(.ARM.exidx.exit.text) *(.ARM.extab.exit.text) *(.ARM.exidx.tex
t.exit) *(.ARM.extab.text.exit) *(.exitcall.exit) *(.discard) *(.discar
d.*)
8       }
9       . = ((0xC0000000)) + 0x00008000;
10      .head.text : {
11      _text = .;
12      KEEP(*(.head.text))
13      }
......
73 }
```

第 2 行的 ENTRY 指明了了 Linux 内核入口，入口为 stext，stext 定义在文件 arch/arm/kernel/head.S 中，因此要分析 Linux 内核的启动流程，就得先从文件 arch/arm/kernel/head.S 的 stext 处开始分析。

16.2 Linux 内核启动流程分析

16.2.1 Linux 内核入口 stext

stext 是 Linux 内核的入口地址，在文件 arch/arm/kernel/head.S 中有如下所示提示内容：

示例代码 16.2.1.1 arch/arm/kernel/head.S 代码段

```
/*
 * Kernel startup entry point.
 * ---------------------------
 *
 * This is normally called from the decompressor code.  The requirements
 * are: MMU = off, D-cache = off, I-cache = dont care, r0 = 0,
 * r1 = machine nr, r2 = atags or dtb pointer.
 .....
 */
```

根据示例代码 16.2.1.1 中的注释，Linux 内核启动之前要求如下：

①、关闭 MMU。

②、关闭 D-cache。

③、I-Cache 无所谓。

④、r0=0。

⑤、r1=machine nr(也就是机器 ID)。

⑥、r2=atags 或者设备树(dtb)首地址。

Linux 内核的入口点 stext 其实相当于内核的入口函数，stext 函数内容如下：

示例代码 16.2.1.2 arch/arm/kernel/head.S 代码段

```
77  ENTRY(stext)
......
88      @ ensure svc mode and all interrupts masked
89      safe_svcmode_maskall r9
90
91      mrc p15, 0, r9, c0, c0      @ get processor id
92      bl  __lookup_processor_type @ r5=procinfo r9=cpuid
93      movs    r10, r5             @ invalid processor (r5=0)?
94   THUMB( it  eq )            @ force fixup-able long branch encoding
95      beq __error_p           @ yes, error 'p'
......
105 #ifndef CONFIG_XIP_KERNEL
......
110 #else
111     ldr r8, =PLAT_PHYS_OFFSET       @ always constant in this case
112 #endif
113
114     /*
115      * r1 = machine no, r2 = atags or dtb,
116      * r8 = phys_offset, r9 = cpuid, r10 = procinfo
117      */
118     bl  __vet_atags
......
125     bl  __create_page_tables
......
146     ldr r13, =__mmap_switched       @ address to jump to after
147                     @ mmu has been enabled
148     badr    lr, 1f              @ return (PIC) address
149 #ifdef CONFIG_ARM_LPAE
150     mov r5, #0              @ high TTBR0
151     mov r8, r4, lsr #12         @ TTBR1 is swapper_pg_dir pfn
152 #else
153     mov r8, r4              @ set TTBR1 to swapper_pg_dir
154 #endif
155     ldr r12, [r10, #PROCINFO_INITFUNC]
156     add r12, r12, r10
```

```
157     ret r12
158 1:  b   __enable_mmu
159 ENDPROC(stext)
```

第 89 行，调用函数 safe_svcmode_maskall 确保 CPU 处于 SVC 模式，并且关闭了所有的中断。safe_svcmode_maskall 定义在文件 arch/arm/include/asm/assembler.h 中。

第 91 行，读处理器 ID，ID 值保存在 r9 寄存器中。

第 92 行，调用函数__lookup_processor_type 检查当前系统是否支持此 CPU，如果支持的就获取 procinfo 信息。procinfo 是 proc_info_list 类型的结构体， proc_info_list 在文件 arch/arm/include/asm/procinfo.h 中的定义如下：

示例代码 16.2.1.3 proc_info_list 结构体

```
26 struct proc_info_list {
27     unsigned int          cpu_val;
28     unsigned int          cpu_mask;
29     unsigned long         __cpu_mm_mmu_flags;  /* used by head.S */
30     unsigned long         __cpu_io_mmu_flags;  /* used by head.S */
31     unsigned long         __cpu_flush;         /* used by head.S */
32     const char            *arch_name;
33     const char            *elf_name;
34     unsigned int          elf_hwcap;
35     const char            *cpu_name;
36     struct processor      *proc;
37     struct cpu_tlb_fns    *tlb;
38     struct cpu_user_fns   *user;
39     struct cpu_cache_fns *cache;
40 };
```

Linux 内核将每种处理器都抽象为一个 proc_info_list 结构体，每种处理器都对应一个 procinfo。因此可以通过处理器 ID 来找到对应的 procinfo 结构，__lookup_processor_type 函数找到对应处理器的 procinfo 以后会将其保存到 r5 寄存器中。

继续回到示例代码 16.2.1.2 中，第 118 行，调用函数__vet_atags 验证 atags 或设备树(dtb)的合法性。函数__vet_atags 定义在文件 arch/arm/kernel/head-common.S 中。

第 120 行，STM32MP157 是双核 A7 处理器，支持 MP，因此定了宏 CONFIG_SMP_ON_UP，所以此处__fixup_smp 函数会执行，处理多核。

第 125 行，调用函数__create_page_tables 创建页表。

第 146 行，将函数__mmap_switched 的地址保存到 r13 寄存器中。__mmap_switched 定义在文件 arch/arm/kernel/head-common.S，__mmap_switched 最终会调用 start_kernel 函数。

第 158 行，调用__enable_mmu 函数使能 MMU，__enable_mmu 定义在文件 arch/arm/kernel/head.S 中。__enable_mmu 最终会通过调用__turn_mmu_on 来打开 MMU，__turn_mmu_on 最后会执行 r13 里面保存的__mmap_switched 函数。

16.2.2 __mmap_switched 函数

__mmap_switched 函数定义在文件 arch/arm/kernel/head-common.S 中，函数代码如下：

示例代码 16.2.2.1 head-common.S 代码段

```
77 __mmap_switched:
```

```
78
79      mov r7, r1
80      mov r8, r2
81      mov r10, r0
82
83      adr r4, __mmap_switched_data
84      mov fp, #0
85
86  #if defined(CONFIG_XIP_DEFLATED_DATA)
87    ARM( ldr sp, [r4], #4 )
88   THUMB( ldr sp, [r4] )
89   THUMB( add r4, #4 )
90      bl  __inflate_kernel_data      @ decompress .data to RAM
91      teq r0, #0
92      bne __error
93  #elif defined(CONFIG_XIP_KERNEL)
94    ARM( ldmia   r4!, {r0, r1, r2, sp} )
95   THUMB( ldmia   r4!, {r0, r1, r2, r3} )
96   THUMB( mov sp, r3 )
97      sub r2, r2, r1
98      bl  memcpy            @ copy .data to RAM
99  #endif
100
101   ARM( ldmia   r4!, {r0, r1, sp} )
102  THUMB( ldmia   r4!, {r0, r1, r3} )
103  THUMB( mov sp, r3 )
104     sub r2, r1, r0
105     mov r1, #0
106     bl  memset            @ clear .bss
107
108     ldmia   r4, {r0, r1, r2, r3}
109     str r9, [r0]          @ Save processor ID
110     str r7, [r1]          @ Save machine type
111     str r8, [r2]          @ Save atags pointer
112     cmp r3, #0
113     strne   r10, [r3]         @ Save control register values
114     mov lr, #0
115     b   start_kernel
116 ENDPROC(__mmap_switched)
```

第 115 行最终调用 start_kernel 来启动 Linux 内核，start_kernel 函数定义在文件 init/main.c 中。

16.2.3 start_kernel 函数

start_kernel 通过调用众多的子函数来完成 Linux 启动之前的一些初始化工作，由于 start_kernel 函数里面调用的子函数太多，而这些子函数又很复杂，因此我们简单的来看一下一些重要的子函数。精简并添加注释后的 start_kernel 函数内容如下：

示例代码 16.2.3.1 start_kernel 函数

```
asmlinkage __visible void __init start_kernel(void)
{
    char *command_line;
    char *after_dashes;

    set_task_stack_end_magic(&init_task);/* 设置任务栈结束魔术数，用于栈溢出检
                                            测 */
    smp_setup_processor_id();   /* 跟 SMP 有关(多核处理器)，设置处理器 ID。
                                 * 有很多资料说 ARM 架构下此函数为空函数，那是因
                                 * 为他们用的老版本 Linux，而那时候 ARM 还没有多
                                 * 核处理器。
                                 */
    debug_objects_early_init();     /* 做一些和 debug 有关的初始化 */
    cgroup_init_early();    /* cgroup 初始化，cgroup 用于控制 Linux 系统资源*/
    local_irq_disable();    /* 关闭当前 CPU 中断 */
    early_boot_irqs_disabled = true;

    /*
     * 中断关闭期间做一些重要的操作，然后打开中断
     */
    boot_cpu_init();              /* 跟 CPU 有关的初始化 */
    page_address_init();          /* 页地址相关的初始化 */
    pr_notice("%s", linux_banner);/* 打印 Linux 版本号、编译时间等信息 */
    setup_arch(&command_line);  /* 架构相关的初始化，此函数会解析传递进来的
                                 * ATAGS 或者设备树(DTB)文件。会根据设备树里面
                                 * 的 model 和 compatible 这两个属性值来查找
                                 * Linux 是否支持这个单板。此函数也会获取设备树
                                 * 中 chosen 节点下的 bootargs 属性值来得到命令
                                 * 行参数，也就是 uboot 中的 bootargs 环境变量的
                                 * 值，获取到的命令行参数会保存到
                                 *command_line 中。
                                 */
    mm_init_cpumask(&init_mm);  /* 看名字，应该是和内存有关的初始化 */
    setup_command_line(command_line);   /* 好像是存储命令行参数 */
    setup_nr_cpu_ids();         /* 如果只是 SMP(多核 CPU)的话，此函数用于获取
                                 * CPU 核心数量，CPU 数量保存在变量
                                 * nr_cpu_ids 中。
```

```
                                */
    setup_per_cpu_areas();      /* 在 SMP 系统中有用，设置每个 CPU 的 per-cpu 数
                                   据 */
    boot_cpu_state_init();
    smp_prepare_boot_cpu();

    build_all_zonelists(NULL, NULL);    /* 建立系统内存页区(zone)链表 */
    page_alloc_init();                  /* 处理用于热插拔 CPU 的页 */

    /* 打印命令行信息 */
    pr_notice("Kernel command line: %s\n", boot_command_line);
    parse_early_param();                /* 解析命令行中的 console 参数 */
    after_dashes = parse_args("Booting kernel",
                static_command_line, __start___param,
                __stop___param - __start___param,
                -1, -1, &unknown_bootoption);
    if (!IS_ERR_OR_NULL(after_dashes))
        parse_args("Setting init args", after_dashes, NULL, 0, -1, -1,
              set_init_arg);

    jump_label_init();

    setup_log_buf(0);     /* 设置 log 使用的缓冲区*/
    pidhash_init();       /* 构建 PID 哈希表，Linux 中每个进程都有一个 ID，
                          * 这个 ID 叫做 PID。通过构建哈希表可以快速搜索进程
                          * 信息结构体。
                          */
    vfs_caches_init_early();  /* 预先初始化 vfs(虚拟文件系统)的目录项和索引
                                 节点缓存*/
    sort_main_extable();      /* 定义内核异常列表 */
    trap_init();              /* 完成对系统保留中断向量的初始化 */
    mm_init();                /* 内存管理初始化 */

    sched_init();             /* 初始化调度器，主要是初始化一些结构体 */
    preempt_disable();        /* 关闭优先级抢占 */
    if (WARN(!irqs_disabled(),/* 检查中断是否关闭，如果没有的话就关闭中断 */
         "Interrupts were enabled *very* early, fixing it\n"))
        local_irq_disable();
    radix_tree_init();
    workqueue_init_early(); /*允许及早创建工作队列和工作项排队/取消。工作项的
                             * 执行取决于 kthread，并在 workqueue_init（）之后
                             开始。*/
    rcu_init();     /* 初始化 RCU，RCU 全称为 Read Copy Update(读-拷贝修改) */
```

```
    trace_init();   /* 跟踪调试相关初始化 */

    context_tracking_init();
    radix_tree_init();    /* 基数树相关数据结构初始化 */
    early_irq_init();     /* 初始中断相关初始化,主要是注册 irq_desc 结构体变
                           * 量，因为 Linux 内核使用 irq_desc 来描述一个中断。
                           */
    init_IRQ();           /* 中断初始化              */
    tick_init();          /* tick 初始化             */
    rcu_init_nohz();
    init_timers();        /* 初始化定时器            */
    hrtimers_init();      /* 初始化高精度定时器      */
    softirq_init();       /* 软中断初始化            */
    timekeeping_init();
    time_init();          /* 初始化系统时间          */
    sched_clock_postinit();
    printk_safe_init();
    perf_event_init();
    profile_init();
    call_function_init();
    WARN(!irqs_disabled(), "Interrupts were enabled early\n");
    early_boot_irqs_disabled = false;
    local_irq_enable();       /* 使能中断 */

    kmem_cache_init_late();  /* slab 初始化，slab 是 Linux 内存分配器  */
    console_init();          /* 初始化控制台，之前 printk 打印的信息都存放
                              * 缓冲区中，并没有打印出来。只有调用此函数
                              * 初始化控制台以后才能在控制台上打印信息。
                              */
    if (panic_later)
        panic("Too many boot %s vars at `%s'", panic_later,
            panic_param);

    lockdep_info();/* 如果定义了宏 CONFIG_LOCKDEP，那么此函数打印一些信息。*/

    locking_selftest();   /* 锁自测 */
    ......
    page_ext_init();
    kmemleak_init();      /* kmemleak 初始化，kmemleak 用于检查内存泄漏 */
    debug_objects_mem_init();
    setup_per_cpu_pageset();
    numa_policy_init();
    if (late_time_init)
```

```
    late_time_init();
    calibrate_delay(); /* 测定 BogoMIPS 值，可以通过 BogoMIPS 来判断 CPU 的性能
                        * BogoMIPS 设置越大，说明 CPU 性能越好。
                        */
    pidmap_init();      /* PID 位图初始化 */
    anon_vma_init();      /* 生成 anon_vma slab 缓存 */
    acpi_early_init();
    ......
    thread_stack_cache_init();
    cred_init();          /* 为对象的每个用于赋予资格(凭证)    */
    fork_init();          /* 初始化一些结构体以使用 fork 函数  */
    proc_caches_init();   /* 给各种资源管理结构分配缓存        */
    buffer_init();        /* 初始化缓冲缓存                   */
    key_init();           /* 初始化密钥                       */
    security_init();      /* 安全相关初始化                   */
    dbg_late_init();
    vfs_caches_init(totalram_pages);    /* 为 VFS 创建缓存   */
    signals_init();                     /* 初始化信号        */

    proc_root_init();                   /* 注册并挂载 proc 文件系统 */
    nsfs_init();
    cpuset_init();  /* 初始化 cpuset，cpuset 是将 CPU 和内存资源以逻辑性
                     * 和层次性集成的一种机制，是 cgroup 使用的子系统之一
                     */
    cgroup_init();              /* 初始化 cgroup */
    taskstats_init_early();     /* 进程状态初始化 */
    delayacct_init();

    check_bugs();                /* 检查写缓冲一致性 */

    acpi_subsystem_init();
    sfi_init_late();

    if (efi_enabled(EFI_RUNTIME_SERVICES)) {
        efi_free_boot_services();
    }

    rest_init();                /* rest_init 函数 */
}
```

start_kernel 里面调用了大量的函数，每一个函数都是一个庞大的知识点，如果想要学习 Linux 内核，那么这些函数就需要去详细的研究。本教程注重于嵌入式 Linux 入门，因此不会去讲太多关于 Linux 内核的知识。start_kernel 函数最后调用了 rest_init，接下来简单看一下 rest_init 函数。

16.2.4 rest_init 函数

rest_init 函数定义在文件 init/main.c 中，函数内容如下：

示例代码 16.2.4.1 rest_init 函数

```
406 noinline void __ref rest_init(void)
407 {
408     struct task_struct *tsk;
409     int pid;
410 
411     rcu_scheduler_starting();
412     /*
413      * We need to spawn init first so that it obtains pid 1, however
414      * the init task will end up wanting to create kthreads, which,
415      * if we schedule it before we create kthreadd, will OOPS.
416      */
417     pid = kernel_thread(kernel_init, NULL, CLONE_FS);
418     /*
419      * Pin init on the boot CPU. Task migration is not properly
420      * working until sched_init_smp() has been run. It will set the
421      * allowed CPUs for init to the non isolated CPUs.
422      */
423     rcu_read_lock();
424     tsk = find_task_by_pid_ns(pid, &init_pid_ns);
425     set_cpus_allowed_ptr(tsk, cpumask_of(smp_processor_id()));
426     rcu_read_unlock();
427 
428     numa_default_policy();
429     pid = kernel_thread(kthreadd, NULL, CLONE_FS | CLONE_FILES);
430     rcu_read_lock();
431     kthreadd_task = find_task_by_pid_ns(pid, &init_pid_ns);
432     rcu_read_unlock();
433 
434     /*
435      * Enable might_sleep() and smp_processor_id() checks.
436      * They cannot be enabled earlier because with
        CONFIG_PREEMPTION=y
437      * kernel_thread() would trigger might_sleep() splats. With
438      * CONFIG_PREEMPT_VOLUNTARY=y the init task might have scheduled
439      * already, but it's stuck on the kthreadd_done completion.
440      */
441     system_state = SYSTEM_SCHEDULING;
442 
443     complete(&kthreadd_done);
```

```
444
445     /*
446      * The boot idle thread must execute schedule()
447      * at least once to get things moving:
448      */
449     schedule_preempt_disabled();
450     /* Call into cpu_idle with preempt disabled */
451     cpu_startup_entry(CPUHP_ONLINE);
452 }
```

第 411 行，调用函数 rcu_scheduler_starting，启动 RCU 锁调度器

第 417 行，调用函数 kernel_thread 创建 kernel_init 进程，也就是大名鼎鼎的 init 内核进程。init 进程的 PID 为 1。init 进程一开始是内核进程(也就是运行在内核态)，后面 init 进程会在根文件系统中查找名为“init”这个程序，这个“init”程序处于用户态，通过运行这个“init”程序，init 进程就会实现从内核态到用户态的转变。

第 429 行，调用函数 kernel_thread 创建 kthreadd 内核进程，此内核进程的 PID 为 2。kthreadd 进程负责所有内核进程的调度和管理。

第 451 行，最后调用函数 cpu_startup_entry 来进入 idle 进程，cpu_startup_entry 会调用 cpu_idle_loop，cpu_idle_loop 是个 while 循环，也就是 idle 进程代码。idle 进程的 PID 为 0，idle 进程叫做空闲进程，如果学过 FreeRTOS 或者 UCOS 的话应该听说过空闲任务。idle 空闲进程就和空闲任务一样，当 CPU 没有事情做的时候就在 idle 空闲进程里面“瞎逛游”，反正就是给 CPU 找点事做。当其他进程要工作的时候就会抢占 idle 进程，从而夺取 CPU 使用权。其实大家应该可以看到 idle 进程并没有使用 kernel_thread 或者 fork 函数来创建，因为它是有主进程演变而来的。

在 Linux 终端中输入“ps -A”就可以打印出当前系统中的所有进程，其中就能看到 init 进程和 kthreadd 进程，如图 16.2.4.1 所示：

```
[root@ATK-stm32mp1]:~$ ps -A
PID   USER     COMMAND
    1 root     init
    2 root     [kthreadd]
    3 root     [rcu_gp]
    4 root     [rcu_par_gp]
    5 root     [kworker/0:0-mm_]
    6 root     [kworker/0:0H-mm]
    8 root     [mm_percpu_wq]
```

图 16.2.4.1 Linux 系统当前进程

从图 16.2.4.1 可以看出，init 进程的 PID 为 1，kthreadd 进程的 PID 为 2。之所以图 16.2.4.1 中没有显示 PID 为 0 的 idle 进程，那是因为 idle 进程是内核进程。我们接下来重点看一下 init 进程，kernel_init 就是 init 进程的进程函数。

16.2.5 init 进程

kernel_init 函数就是 init 进程具体做的工作，定义在文件 init/main.c 中，函数内容如下：

示例代码 16.2.5.1 kernel_init 函数

```
1106    static int __ref kernel_init(void *unused)
1107    {
```

```
1108        int ret;
1109
1110        kernel_init_freeable(); /* init 进程的一些其他初始化工作 */
1111        /* need to finish all async __init code before freeing the
               memory */
1112        async_synchronize_full();   /* 等待所有的异步调用执行完成    */
1113        ftrace_free_init_mem();     /* 释放 init 段内存              */
1114        free_initmem();
1115        mark_readonly();
1116
1117        /*
1118         * Kernel mappings are now finalized - update the userspace
1119         * page-table to finalize PTI.
1120         */
1121        pti_finalize();
1122
1123        system_state = SYSTEM_RUNNING; /* 标记系统正在运行      */
1124        numa_default_policy();
1125
1126        rcu_end_inkernel_boot();
1127
1128        if (ramdisk_execute_command) {
1129            ret = run_init_process(ramdisk_execute_command);
1130            if (!ret)
1131                return 0;
1132            pr_err("Failed to execute %s (error %d)\n",
1133                   ramdisk_execute_command, ret);
1134        }
1135
1136        /*
1137         * We try each of these until one succeeds.
1138         *
1139         * The Bourne shell can be used instead of init if we are
1140         * trying to recover a really broken machine.
1141         */
1142        if (execute_command) {
1143            ret = run_init_process(execute_command);
1144            if (!ret)
1145                return 0;
1146            panic("Requested init %s failed (error %d).",
1147                  execute_command, ret);
1148        }
1149        if (!try_to_run_init_process("/sbin/init") ||
```

```
1150            !try_to_run_init_process("/etc/init") ||
1151            !try_to_run_init_process("/bin/init") ||
1152            !try_to_run_init_process("/bin/sh"))
1153            return 0;
1154
1155        panic("No working init found.  Try passing init= option to
                 kernel. "
1156              "See Linux Documentation/admin-guide/init.rst for
                 guidance.");
1157    }
```

第 1110 行，kernel_init_freeable 函数用于完成 init 进程的一些其他初始化工作，稍后再来具体看一下此函数。

第 1128 行，ramdisk_execute_command 是一个全局的 char 指针变量，此变量值为“/init”，也就是根目录下的 init 程序。ramdisk_execute_command 也可以通过 uboot 传递，在 bootargs 中使用“rdinit=xxx”即可，xxx 为具体的 init 程序名字。

第 1129 行，如果存在“/init”程序的话就通过函数 run_init_process 来运行此程序。

第 1142 行，如果 ramdisk_execute_command 为空的话就看 execute_command 是否为空，反正不管如何一定要在根文件系统中找到一个可运行的 init 程序。execute_command 的值是通过 uboot 传递，在 bootargs 中使用“init=xxxx”就可以了，比如“init=/linuxrc”表示根文件系统中的 linuxrc 就是要执行的用户空间 init 程序。

第 1149~1153 行，如果 ramdisk_execute_command 和 execute_command 都为空，那么就依次查找“/sbin/init”、“/etc/init”、“/bin/init”和“/bin/sh”，这四个相当于备用 init 程序，如果这四个也不存在，那么 Linux 启动失败！

第 1155 行，如果以上步骤都没有找到用户空间的 init 程序，那么就提示错误发生！

最后来简单看一下 kernel_init_freeable 函数，前面说了，kernel_init 会调用此函数来做一些 init 进程初始化工作。kernel_init_freeable 定义在文件 init/main.c 中，缩减后的函数内容如下：

示例代码 162.5.2 kernel_init_freeable 函数

```
1159    static noinline void __init kernel_init_freeable(void)
1160    {
1161        /*
1162         * Wait until kthreadd is all set-up.
1163         */
1164        wait_for_completion(&kthreadd_done);/* 等待 kthreadd 准备就绪 */
......
1185        smp_init();             /* SMP 初始化              */
1186        sched_init_smp();       /* 多核(SMP)调度初始化    */
1187
1188        page_alloc_init_late(); /* 设备初始化都在此函数中完成 */
1189        /* Initialize page ext after all struct pages are
              initialized. */
1190        page_ext_init();
1191
1192        do_basic_setup();
```

```
1193
1194        /* Open the /dev/console on the rootfs, this should never
             fail */
1195        if (ksys_open((const char __user *) "/dev/console",
              O_RDWR, 0) < 0)
1196            pr_err("Warning: unable to open an initial console.\n");
1197
1198        (void) ksys_dup(0);
1199        (void) ksys_dup(0);
1200        /*
1201         * check if there is an early userspace init.  If yes, let it
1202         * do all the work
1203         */
1204
1205        if (!ramdisk_execute_command)
1206            ramdisk_execute_command = "/init";
1207
1208        if (ksys_access((const char __user *)
1209                ramdisk_execute_command, 0) != 0) {
1210            ramdisk_execute_command = NULL;
1211            prepare_namespace();
1212        }
1213
1214        /*
1215         * Ok, we have completed the initial bootup, and
1216         * we're essentially up and running. Get rid of the
1217         * initmem segments and start the user-mode stuff..
1218         *
1219         * rootfs is available now, try loading the public keys
1220         * and default modules
1221         */
1222
1223        integrity_load_keys();
1224    }
```

第 1192 行，do_basic_setup 函数用于完成 Linux 下设备驱动初始化工作！非常重要。do_basic_setup 会调用 driver_init 函数完成 Linux 下驱动模型子系统的初始化。

第 1195 行，打开设备“/dev/console”，在 Linux 中一切皆为文件！因此“/dev/console”也是一个文件，此文件为控制台设备。每个文件都有一个文件描述符，此处打开的“/dev/console”文件描述符为 0，作为标准输入(0)。

第 1198 和 1199 行，sys_dup 函数将标准输入(0)的文件描述符复制了 2 次，一个作为标准输出(1)，一个作为标准错误(2)。这样标准输入、输出、错误都是/dev/console 了。console 通过 uboot 的 bootargs 环境变量设置，“console=ttySTM0,115200”表示将/dev/ttySTM0 设置为 console，

也就是 STM32MP1 的串口 4。当然，也可以设置其他的设备为 console，比如虚拟控制台 tty1，设置 tty1 为 console 就可以在 LCD 屏幕上看到系统的提示信息。

第 1211 行，调用函数 prepare_namespace 来挂载根文件系统。根文件系统也是由命令行参数指定的，也就是 uboot 的 bootargs 环境变量。比如“root=/dev/mmcblk1p3 rootwait rw”就表示根文件系统在/dev/mmcblk1p3 中，也就是 EMMC 的分区 3 中。

Linux 内核启动流程就分析到这里，Linux 内核最终是需要和根文件系统打交道的，需要挂载根文件系统，并且执行根文件系统中的 init 程序，以此来进去用户态。这里就正式引出了根文件系统，根文件系统也是我们系统移植的最后一片拼图。Linux 移植三巨头：uboot、Linux kernel、rootfs(根文件系统)。关于根文件系统后面章节会详细的讲解，这里我们只需要知道 Linux 内核移植完成以后还需要构建根文件系统即可。

第十七章 Linux 内核移植

前两章我们简单了解了一下 Linux 内核顶层 Makefile 和 Linux 内核的启动流程，本章我们就来学习一下如何将 ST 官方提供的 Linux 内核移植到正点原子的 STM32MP157 开发板上。通过本章的学习，我们将掌握如何将半导体厂商提供的 Linux BSP 包移植到我们自己的平台上。

17.1 Linux 内核初次编译

17.1.1 编译正点原子出厂 Linux 源码

参考 15.1 小节编译正点原子 STM32MP157 开发板出厂 Linux 内核。

17.1.2 运行测试

成功编译正点原子出厂 Linux 系统以后，会得到 uImage 和对应的 stm32mp157d-atk.dtb 设备树，本节我们就测试一下两个文件能不能正常启动，将 uImage 和 stm32mp157d-atk.dtb 这两个文件发送到 Ubuntu 的 TFTP 服务器目录下。启动开发板，uboot 通过 tftp 命令从 Ubuntu 中 uImage 和 stm32mp157d-atk.dtb 并启动。为了不用每次手动输入命令，我们可以直接设置 uboot 中的 bootcmd 环境变量为：

```
setenv bootcmd 'tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 - (有空格) c4000000'
saveenv
```

启动以后 Linux 输出如图 17.1.2.1 所示信息：

```
ethernet@5800a000 Waiting for PHY auto negotiation to complete...... done
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'uImage'.
Load address: 0xc2000000
Loading: #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ################################
         953.1 KiB/s
done
Bytes transferred = 8133232 (7c1a70 hex)
Using ethernet@5800a000 device
TFTP from server 192.168.1.249; our IP address is 192.168.1.250
Filename 'stm32mp157d-atk.dtb'.
Load address: 0xc4000000
Loading: ######
         2.7 MiB/s
done
Bytes transferred = 74594 (12362 hex)
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31
   Created:      2020-12-11   8:55:55 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    8133168 Bytes = 7.8 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffea000, end cffff361 ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 5.4.31 (zuozhongkai@ubuntu) (gcc version 9.2.1 20191025 (GNU Toolchain for
 the A-profile Architecture 9.2-2019.12 (arm-9.10))) #1 SMP PREEMPT Fri Dec 11 16:52:12 CST 2020
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[    0.000000] OF: fdt: Machine model: STMicroelectronics STM32MP157C-DK2 Discovery Board
[    0.000000] Memory policy: Data cache writealloc
```

内核编译时间

图 17.1.2.1 Linux 内核启动 log 信息

从图 17.1.2.1 可以看出，Linux 内核已经运行成功，并且当前 Linux 内核编译时间为 2020 年 12 月 11 日 16:52:12。

注意！如果出现如图 17.1.2.2 所示“end Kernel panic”错误为正常现象！

```
[    4.593457] ---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
]---
```

图 17.1.2.2 根文件系统缺失错误

图 17.1.2.2 中的错误是由于根文件系统缺失导致的，也就是我们没有指定根文件系统，根文件系统的制作我们后面会详细讲解。

17.2 编译 ST 官方 Linux 系统

上一节我们编译了正点原子提供的出厂系统，主要是学习一下 Linux 系统的编译流程，我们最终只要将 ST 原厂提供的 Linux 系统移植到正点原子的 STM32MP157 开发板上，因此我们肯定要先编译一下 ST 原厂的系统。

17.2.1 ST 官方 Linux 源码打补丁

1、获取到 ST 官方 Linux 源码

首先肯定要获取到 ST 官方的 Linux 源码，这个已经在 6.1.1 小节获取到了，进入到对应的 Linux 源码目录，命令如下：

```
cd /home/zuozhongkai/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi/linux-stm32mp-5.4.31-r0 //进入 ST 官方 Linux 源码
```

ST 官方 Linux 源码如图 17.2.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi/linux-stm32mp-5.4.31-r0$ ls
0001-ARM-stm32mp1-r1-MACHINE.patch                0016-ARM-stm32mp1-r1-PHY-USB.patch
0002-ARM-stm32mp1-r1-CPUFREQ.patch                0017-ARM-stm32mp1-r1-PINCTRL-REGULATOR-SPI-PWM.patch
0003-ARM-stm32mp1-r1-CRYPTO.patch                 0018-ARM-stm32mp1-r1-SOUND.patch
0004-ARM-stm32mp1-r1-RNG-DEBUG-NVMEM.patch        0019-ARM-stm32mp1-r1-MISC.patch
0005-ARM-stm32mp1-r1-CLOCK.patch                  0020-ARM-stm32mp1-r1-DEVICETREE.patch
0006-ARM-stm32mp1-r1-DMA.patch                    0021-ARM-stm32mp1-r1-CONFIG.patch
0007-ARM-stm32mp1-r1-DRM.patch                    0022-ARM-stm32mp1-r1-POWER.patch
0008-ARM-stm32mp1-r1-HWSPINLOCK.patch             0023-ARM-stm32mp1-r1-PERF.patch
0009-ARM-stm32mp1-r1-I2C-IIO-IRQCHIP.patch        fragment-03-systemd.config
0010-ARM-stm32mp1-r1-MAILBOX-REMOTEPROC-RPMSG.patch fragment-04-optee.config
0011-ARM-stm32mp1-r1-RESET-RTC-WATCHDOG.patch     fragment-05-modules.config
0012-ARM-stm32mp1-r1-MEDIA-SOC-THERMAL.patch      fragment-06-signature.config
0013-ARM-stm32mp1-r1-MFD.patch                    linux-5.4.31.tar.xz
```

图 17.2.1.1 ST 官方 Linux 源码

从图 17.2.1.1 可以看出，ST 官方的 Linux 源码包有各种.patch 补丁文件以及 Linux 源码压缩包，注意细心的朋友可能观察到了，图 17.1.1.1 中有 4 个以“fragment”开头的.config 文件，这些文件是默认配置文件的补丁文件，用来生成默认配置文件，稍后会讲解怎么操作。

首先解压图 17.2.1.1 中“linux-5.4.31.tar.xz”这个真正的源码包压缩包，解压命令如下：

```
tar -vxf linux-5.4.31.tar.xz
```

解压完成以后就会得到一个名为“linux-5.4.31”的 Linux 源码文件夹，如图 17.2.1.2 所示：

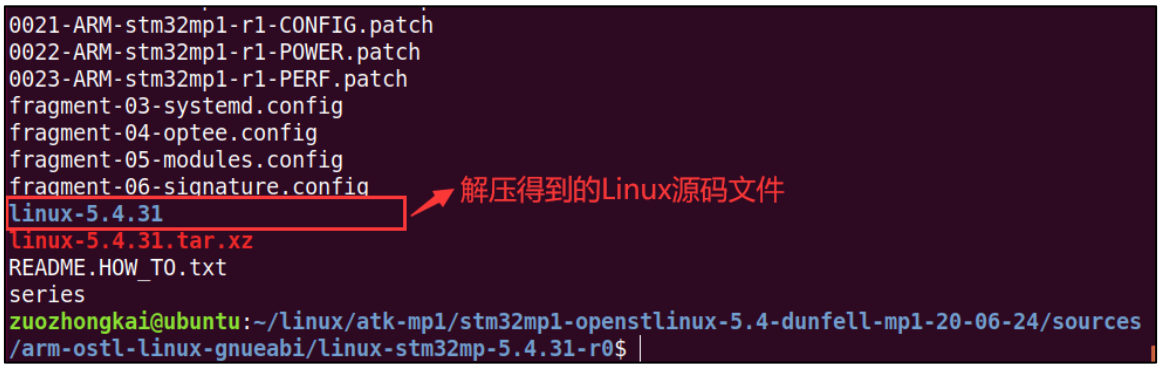


图 17.2.1.2 解压得到 Linux 源码

2、内核打补丁

上面已经解压出来了 Linux 的源码文件，接下来就要对其打补丁，进入到上面解压出来的 linux-5.4.31 目录，然后执行相应的打补丁命令：

```
cd linux-5.4.31/                                    //进入 Linux 源码目录
for p in `ls -1 ../*.patch`; do patch -p1 < $p; done //打补丁
```

打补丁结果如图 17.2.1.3 所示:

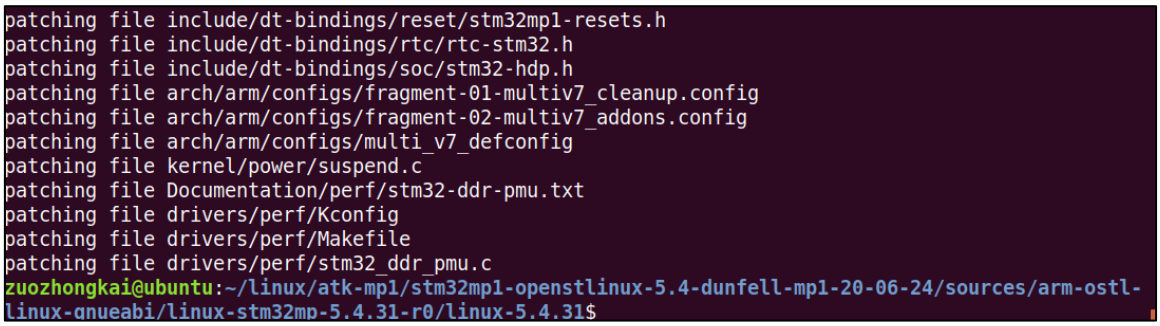


图 17.2.1.3 打补丁结果

3、生成默认配置文件

编译 linux 内核的时候也需要使用“make xxx_defconfig”来对其进行默认配置，比如编译正点原子出厂 Linux 系统的时候我们使用了“make stm32mp1_atk_defconfig”，其中 stm32mp1_atk_defconfig 就是正点原子开发板使用的默认配置文件。但是 ST 官原厂 Linux 内核需要先生成默认配置文件，并且对其进行打补丁，进入 Linux 内核源码根目录下，然后执行如下命令：

```
cd linux-5.4.31/                                    //进入到 linux 内核
make ARCH=arm multi_v7_defconfig "fragment*.config"  //生成默认配置文件
```

完成以后结果如图 17.2.1.4 所示：

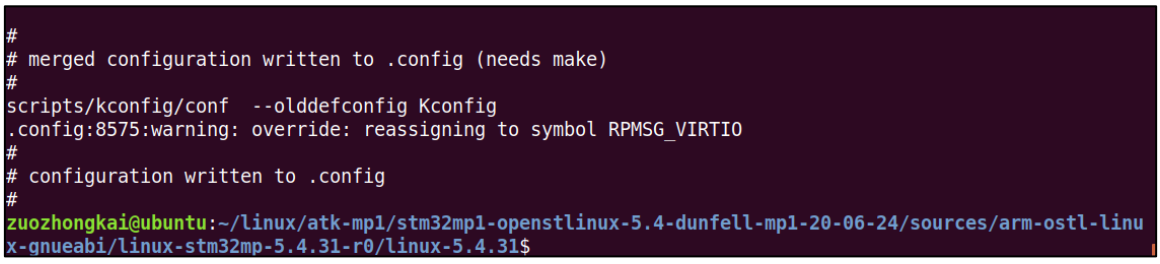


图 17.2.1.4 生成.config 文件

上面我们只是在 Linux 源码根目录下生成了.config 配置文件，如图 17.2.1.5 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linu
x-gnueabi/linux-stm32mp-5.4.31-r0/linux-5.4.31$ ls -a
.               .cocciconfig     crypto                .gitignore  kernel       mm        sound
..              .config          Documentation         include     lib          net       tools
arch            .config.old      drivers               init        LICENSES     README    usr
block           CONTRIBUTING.md  fs                    ipc         .mailmap     samples   virt
certs           COPYING          .get_maintainer.ignore  Kbuild    MAINTAINERS  scripts
.clang-format   CREDITS          .gitattributes        Kconfig     Makefile     security
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linu
x-gnueabi/linux-stm32mp-5.4.31-r0/linux-5.4.31$
```

图 17.2.1.5 .config 文件

.config 文件非常重要，Linux 内核的所有配置项最终都保存在.config 文件里面，最终编译 Linux 内核的时候需要读取.config 里面的配置项！此时我们只是生成了.config，还并没有将图 17.1.1.1 中的这些 fragment config 补丁文件打进去，执行如入两条命令打补丁：

```
for f in `ls -1 ../fragment*.config`; do scripts/kconfig/merge_config.sh -m -r .config $f; done
yes '' | make ARCH=arm oldconfig
```

第 1 条命令是将图 17.1.2.1 中的 fragment config 补丁打到.config 文件里面。

第 2 条命令是对 oldconfig 的所有选项都选择 yes，这些选项最终都会写入到.config 里面。注意，**这条命令 yes 后面是两个 ‘’ ,而不是一个 “"”，不要输入错了!!!**

完成以后如图 17.2.1.6 所示：

```
  Character device in Userspace support (CUSE) [N/m/y/?] (NEW)
  Virtio Filesystem (VIRTIO_FS) [N/m/y/?] (NEW)
Overlay filesystem support (OVERLAY_FS) [Y/n/m/?] y
  Overlayfs: turn on redirect directory feature by default (OVERLAY_FS_REDIRECT_DIR) [N/y/?] n
  Overlayfs: follow redirects even if redirects are turned off (OVERLAY_FS_REDIRECT_ALWAYS_FOLLOW) [Y/
n/?] y
  Overlayfs: turn on inodes index feature by default (OVERLAY_FS_INDEX) [N/y/?] n
  Overlayfs: auto enable inode number mapping (OVERLAY_FS_XINO_AUTO) [N/y/?] n
  Overlayfs: turn on metadata only copy up feature by default (OVERLAY_FS_METACOPY) [N/y/?] n
UTF-8 normalization and casefolding support (UNICODE) [N/y/?] n
#
# configuration written to .config
#
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linu
x-gnueabi/linux-stm32mp-5.4.31-r0/linux-5.4.31$
```

图 17.2.1.6 配置完成的.config

至此，Linux 源码根目录下的.config 文件就已经保存了所有的配置项，所以只需要复制一份.config 作为我们的默认配置文件即可，复制命令如下：

```
cp .config ./arch/arm/configs/stm32mp1_atk_defconfig
```

此时在 Linux 内核的/arch/arm/configs 目录下存在一个名为 stm32mp1_atk_defconfig 的默认配置文件，如图 17.2.1.7 所示：

```
x-gnueabi/linux-stm32mp-5.4.31-r0/linux-5.4.31/arch/arm/configs$ ls stm32*
stm32_defconfig  stm32mp1_atk_defconfig
zuozhongkai@ubuntu:~/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linu
x-gnueabi/linux-stm32mp-5.4.31-r0/linux-5.4.31/arch/arm/configs$
```

图 17.2.1.7 默认配置文件

至此，Linux 内核全部打完补丁，linux-5.4.31 目录就是我们要移植的 Linux 源码，但是图 17.2.1.3 中的 linux-5.4.31 目录路径有点长，不适合阅读和编译。所以我们新建一个名为“my_linux”的目录来保存我们要移植的 linux 源码，然后将打完补丁的 linux 源码 linux-5.4.31 拷贝到“my_linux”目录下，命令如下：

```
cd /home/zuozhongkai/linux/atk-mp1/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources
/arm-ostl-linux-gnueabi/linux-stm32mp-5.4.31-r0
cp linux-5.4.31 /home/zuozhongkai/linux/atk-mp1/linux/my_linux/ -rf      //拷贝
```

拷贝完成以后的 my_linux 目录如图 17.2.1.8 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux$ ls
linux-5.4.31
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux$
```

图 17.2.1.8 拷贝过来的 linux 源码

最后就是创建 vscode 工程，方便我们移植和阅读。

17.2.2 编译 ST 官方 Linux 源码

上一小节已经准备好了 ST 官方 Linux 源码，存放到了 my_ulinux 目录下，本小节我们来编译一下这个 Linux 源码。

1、修改 Makefile

首先修改一下 Linux 源码的 Makefile 文件，也可以不修改，但是在编译的时候需要多输入一些参数，为了偷懒，还是修改一下。修改方法已经在 17.1.1 小节讲解过了，就是在 Makefile 文件里面添加 ARCH 和 CROSS_COMPILE 这两个变量的值，如图 17.2.2.1 所示：

```
360  ARCH         ?= $(SUBARCH)
361
362  ARCH  = arm
363  CROSS_COMPILE = arm-none-linux-gnueabihf-
364
```

设置ARCH和CROSS_COMPILE的值

图 17.2.2.1 设置 ARCH 和 CROSS_COMPILE 变量值

ST 官方 Linux 肯定适配了官方的 STM32MP1 EVK 开发板，我们就编译 EVK 开发板对应的 Linux 内核和设备树，编译完成以后在正点原子的 STM32MP1 开发板启动，看看能不能运行，不能的话就要修改相应的文件，这就是 Linux 的移植。创建一个名为“stm32mp157d_atk.sh”的编译脚本，脚本内容如下：

示例代码 17.1.2.1 stm32mp157d_atk.sh 文件内容

```
1 #!/bin/sh
2 make distclean
3 make stm32mp1_atk_defconfig
4 make menuconfig
5 make uImage dtbs LOADADDR=0XC2000040 -j16
```

给予 stm32mp157d_atk.sh 可执行文件，然后运行此编译脚本，命令如下：

```
chmod 777 stm32mp157d_atk.sh      //给予可执行权限，仅第一次
./stm32mp157d_atk.sh              //运行编译脚本
```

等待编译完成，编译完成以后的到 uImage 镜像文件和设备树，其中 STM32MP157 系列的设备树有很多，如图 17.2.2.3 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31/arch/arm/boot/dts$ ls stm32mp*.dtb
stm32mp157a-avenger96.dtb        stm32mp157c-ed1.dtb              stm32mp157f-dk2-a7-examples.dtb
stm32mp157a-dk1.dtb              stm32mp157c-ev1-a7-examples.dtb  stm32mp157f-dk2.dtb
stm32mp157a-ed1.dtb              stm32mp157c-ev1.dtb              stm32mp157f-dk2-m4-examples.dtb
stm32mp157a-ev1.dtb              stm32mp157c-ev1-m4-examples.dtb  stm32mp157f-ed1.dtb
stm32mp157c-dk2-a7-examples.dtb  stm32mp157d-dk1.dtb              stm32mp157f-ev1-a7-examples.dtb
stm32mp157c-dk2.dtb              stm32mp157d-ed1.dtb              stm32mp157f-ev1.dtb
stm32mp157c-dk2-m4-examples.dtb  stm32mp157d-ev1.dtb              stm32mp157f-ev1-m4-examples.dtb
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31/arch/arm/boot/dts$
```

图 17.2.2.3 编译得到的.dtb 文件

从图 17.2.2.3 可以看出，此时有很多.dtb 文件，我们这里使用 stm32mp157d-ed1.dtb 这个文

件。

17.2.3 网络驱动修改

V1.2 版本核心板网络 PHY 芯片采用 RTL8211，ST 官方源码默认已经支持了 RTL8211，所以不需要进行任何修改。但是 V1.3 以后的核心板网络 PHY 芯片改为了裕太电子的 YT8511，ST 官方源码默认没有支持 YT8511，因此需要我们自行移植相关的网络驱动。相关驱动以及修改方法已经放到了开发板光盘中，路径为：1、程序源码→8、模块驱动源码→1、YT8511 驱动源码→linux 内核下修改方法→linux。

将 motorcomm.c 和 motorcomm_phy.h 分别拷贝到 Linux 源码下的 drivers/net/phy 和 include/linux 目录下。拷贝完成以后修改 drivers/net/phy/Makefile 文件，加上下面这句：

```
obj-$(CONFIG_MOTORCOMM_PHY)     += motorcomm.o
```

修改以后的 Makefile 如图 17.2.3.1 所示：

```
95  obj-$(CONFIG_VITESSE_PHY)    += vitesse.o
96  obj-$(CONFIG_XILINX_GMII2RGMII) += xilinx_gmii2rgmii.o
97  obj-$(CONFIG_MOTORCOMM_PHY) += motorcomm.o
```

图 17.2.3.1 修改后的 Makefile

另外还需要修改 drivers/net/phy/Kconfig 文件，加入如下内容：

示例代码 17.2.3.1 Kconfig 添加的内容

```
1 config MOTORCOMM_PHY
2   tristate "Motorcomm PHYs"
3   ---help---
4     Supports the YT8010, YT8510, YT8511, YT8512 PHYs.
```

添加完成以后的 Kconfig 文件如图 17.2.3.2 所示：

```
487  config XILINX_GMII2RGMII
488      tristate "Xilinx GMII2RGMII converter driver"
489      ---help---
490        This driver support xilinx GMII to RGMII IP core it provides
491        the Reduced Gigabit Media Independent Interface(RGMII) between
492        Ethernet physical media devices and the Gigabit Ethernet controller.
493
494  config MOTORCOMM_PHY
495      tristate "Motorcomm PHYs"
496      ---help---
497        Supports the YT8010, YT8510, YT8511, YT8512 PHYs.
498
499  endif # PHYLIB
500
501  config MICREL_KS8995MA
502      tristate "Micrel KS8995MA 5-ports 10/100 managed Ethernet switch"
503      depends on SPI
504
```

图 17.2.3.2 修改后的 Kconfig

最后输入“make menuconfig”，打开 linux 内核配置界面，使能 YT8511 驱动，配置路径如下：

```
-> Device Drivers
    -> Network device support (NETDEVICES [=y])
        -> PHY Device support and infrastructure (PHYLIB [=y])
            -> <*> Motorcomm PHYs      //将 YT8511 驱动编译进内核
```

如图 17.2.3.3 所示：

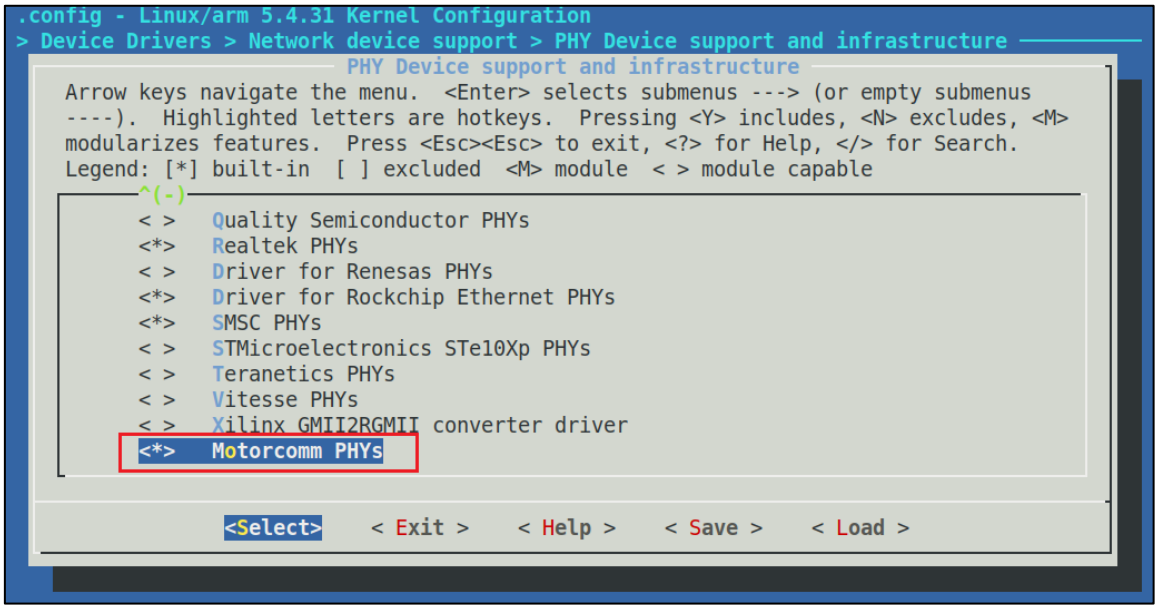


图 17.2.3.3 使能 YT8511 驱动

网络驱动修改完成以后按照上一小节的方法重新编译 linux 内核即可。

17.2.4 启动测试

上一小节我们已经编译出来了 ST 官方开发板对应的 uImage 和 stm32mp157d-ed1.dtb 设备树，本章我们直接在正点原子的 STM32MP157 开发板上运行，看看 ST 官方系统能不能运行下去。讲 uImage 和 stm32mp157d-ed1.dtb 发送到 Ubuntu 下的 tftp 服务器目录下，然后通过 uboot 的 tftp 命令下载并启动，Linux 系统运行 log 信息如图 17.2.4.1 所示：

```
STM32MP> bootm c2000000 - c4000000
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31
   Created:      2020-12-12   8:41:08 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    7313864 Bytes = 7 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffec000, end cffff6ec ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 5.4.31 (zuozhongkai@ubuntu) (gcc version 9.2.1 20191025 (GNU Toolchain for
 the A-profile Architecture 9.2-2019.12 (arm-9.10))) #1 SMP PREEMPT Sat Dec 12 16:38:01 CST 2020
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[    0.000000] OF: fdt: Machine model: STMicroelectronics STM32MP157D eval daughter
```

图 17.2.4.1 ST 官方 Linux 系统运行结果

从图 17.2.4.1 可以看出，ST 官方开发板的 Linux 系统在正点原子 STM32MP157 开发板上启动成功，所以后续的移植就要简单的多(实际不需要移植，直接在 ST 官方开发板相应文件上修改即可，但是为了学习，我们还是学习一下如何在 Linux 源码里面添加我们自己的开发板)，只需要参考 ST 官方开发板创建一个设备树即可。

17.3 在 Linux 中添加自己的开发板

17.3.1 添加开发板对应的默认配置文件

首先就是添加开发板对应的默认配置文件，这里直接使用 17.2.1 小节制作出来的 stm32mp1_atk_defconfig 文件即可。

17.3.2 添加开发板对应的设备树

1、新建相应的设备树文件

上面我们使用 ST 官方开发板的设备树已经成功启动了 Linux 系统，但是我们在实际开发中肯定要添加一份自己开发板所对应的设备树。正点原子 STM32MP157 开发板是以 ST 官方的开发板为蓝本制作的，所以设备树也是参考 ST 官方开发板的。在 arch/arm/boot/dts/目录下新建名为“stm32mp157d-atk.dtsi”的设备树头文件，然后将 stm32mp15xx-edx.dtsi 文件里面的内容全部复制到 stm32mp157d-atk.dtsi 里面。

再新建一个名为“stm32mp157d-atk.dts”的文件，将 stm32mp157d-ed1.dts 文件里面的内容都拷贝到 stm32mp157d-atk.dts 里面，如图 17.3.2.1 所示:

```
6  /dts-v1/;
7
8  #include "stm32mp157.dtsi"
9  #include "stm32mp15xd.dtsi"
10 #include "stm32mp15-pinctrl.dtsi"
11 #include "stm32mp15xxaa-pinctrl.dtsi"
12 #include "stm32mp157-m4-srm.dtsi"
13 #include "stm32mp157-m4-srm-pinctrl.dtsi"
14 #include "stm32mp157d-atk.dtsi"
15
16 / {
17     model = "STMicroelectronics STM32MP157D eval daughter";
18     compatible = "st,stm32mp157d-ed1", "st,stm32mp157";
19
20     chosen {
21         stdout-path = "serial0:115200n8";
22     };
23
24     aliases {
25         serial0 = &uart4;
26     };
27
28     reserved-memory {
29         gpu_reserved: gpu@f6000000 {
30             reg = <0xf6000000 0x8000000>;
31             no-map;
32         };
33
34         optee_memory: optee@fe000000 {
35             reg = <0xfe000000 0x02000000>;
36             no-map;
37         };
38     };
39 };
40
41 &cpu1{
42     cpu-supply = <&vddcore>;
43 };
44
45 &gpu {
46     contiguous-area = <&gpu_reserved>;
47     status = "okay";
48 };
49
50 &optee {
51     status = "okay";
52 };
```

头文件改为"stm32mp157d-atk.dtsi"

图 17.3.2.1 stm32mp157d-atk.dts 文件内容

记得一定要将图 17.3.2.1 中 14 行原来的“stm32mp15xx-edx.dtsi”头文件改为“stm32mp15d-atk.dtsi”！

2、修改 stm32mp157d-atk.dtsi 文件

stm32mp157d-atk.dtsi 里面的内容是直接复制的 ST 官方开发板的 stm32mp15xx-edx.dtsi，所以里面的很多配置是针对 ST 官方开发板的，比如 PMIC 芯片，这些配置是不需要的，要删除

掉。正点原子开发板没有用到集成 PMIC 芯片，因此还需要在设备树里面添加一些电源节点信息，修改完成以后的 stm32mp157d-atk.dtsi 文件内容如下所示：

示例代码 17.3.2.1 stm32mp157d_atk.dtsi 文件内容

```
/*
* Copyright (C) STMicroelectronics 2017 - All Rights Reserved
* Author: Ludovic Barre <ludovic.barre@st.com> for ST.
*/
#include "stm32mp157-m4-srm.dtsi"
#include "stm32mp157-m4-srm-pinctrl.dtsi"
#include <dt-bindings/gpio/gpio.h>
#include <dt-bindings/mfd/st,stpmic1.h>

/ {
    memory@c0000000 {
        device_type = "memory";
        reg = <0xC0000000 0x40000000>;
    };

    reserved-memory {
        #address-cells = <1>;
        #size-cells = <1>;
        ranges;

        mcuram2: mcuram2@10000000 {
            compatible = "shared-dma-pool";
            reg = <0x10000000 0x40000>;
            no-map;
        };

        vdev0vring0: vdev0vring0@10040000 {
            compatible = "shared-dma-pool";
            reg = <0x10040000 0x1000>;
            no-map;
        };

        vdev0vring1: vdev0vring1@10041000 {
            compatible = "shared-dma-pool";
            reg = <0x10041000 0x1000>;
            no-map;
        };

        vdev0buffer: vdev0buffer@10042000 {
            compatible = "shared-dma-pool";
```

```
40              reg = <0x10042000 0x4000>;
41              no-map;
42          };
43
44          mcuram: mcuram@30000000 {
45              compatible = "shared-dma-pool";
46              reg = <0x30000000 0x40000>;
47              no-map;
48          };
49
50          retram: retram@38000000 {
51              compatible = "shared-dma-pool";
52              reg = <0x38000000 0x10000>;
53              no-map;
54          };
55      };
56
57      vddcore: buck1 {
58          compatible = "regulator-fixed";
59          regulator-name = "vddcore";
60          regulator-min-microvolt = <1200000>;
61          regulator-max-microvolt = <1350000>;
62          regulator-always-on;
63          regulator-boot-on;
64      };
65
66      v3v3: regulator-3p3v {
67          compatible = "regulator-fixed";
68          regulator-name = "v3v3";
69          regulator-min-microvolt = <3300000>;
70          regulator-max-microvolt = <3300000>;
71          regulator-always-on;
72          regulator-boot-on;
73      };
74  };
75
76  &cpu0{
77      cpu-supply = <&vddcore>;
78  };
79
80  &crc1 {
81      status = "okay";
82  };
```

```
83
84  &dma1 {
85      sram = <&dma_pool>;
86  };
87
88  &dma2 {
89      sram = <&dma_pool>;
90  };
91
92  &dts {
93      status = "okay";
94  };
95
96  &ethernet0 {
97      status = "okay";
98      pinctrl-0 = <&ethernet0_rgmii_pins_a>;
99      pinctrl-1 = <&ethernet0_rgmii_pins_sleep_a>;
100     pinctrl-names = "default", "sleep";
101     phy-mode = "rgmii-id";
102     max-speed = <1000>;
103     phy-handle = <&phy0>;
104
105     mdio0 {
106         #address-cells = <1>;
107         #size-cells = <0>;
108         compatible = "snps,dwmac-mdio";
109         phy0: ethernet-phy@0 {
110             reg = <0>;
111         };
112     };
113 };
114
115 &hash1 {
116     status = "okay";
117 };
118
119 &ipcc {
120     status = "okay";
121 };
122
123 &iwdg2 {
124     timeout-sec = <32>;
125     status = "okay";
```

```
126 };
127
128 &rng1 {
129     status = "okay";
130 };
131
132 &rtc {
133     status = "okay";
134 };
135
136 &sdmmc1 {
137     pinctrl-names = "default", "opendrain", "sleep";
138     pinctrl-0 = <&sdmmc1_b4_pins_a>;
139     pinctrl-1 = <&sdmmc1_b4_od_pins_a>;
140     pinctrl-2 = <&sdmmc1_b4_sleep_pins_a>;
141     broken-cd;
142     st,neg-edge;
143     bus-width = <4>;
144     vmmc-supply = <&v3v3>;
145     status = "okay";
146 };
147
148 &sdmmc2 {
149     pinctrl-names = "default", "opendrain", "sleep";
150     pinctrl-0 = <&sdmmc2_b4_pins_a>;
151     pinctrl-1 = <&sdmmc2_b4_od_pins_a>;
152     pinctrl-2 = <&sdmmc2_b4_sleep_pins_a>;
153     non-removable;
154     st,neg-edge;
155     bus-width = <8>;
156     vmmc-supply = <&v3v3>;
157     keep-power-in-suspend;
158     status = "okay";
159 };
160
161 &sram {
162     dma_pool: dma_pool@0 {
163         reg = <0x50000 0x10000>;
164         pool;
165     };
166 };
167
168 &uart4 {
```

```
169     pinctrl-names = "default", "sleep", "idle";
170     pinctrl-0 = <&uart4_pins_a>;
171     pinctrl-1 = <&uart4_sleep_pins_a>;
172     pinctrl-2 = <&uart4_idle_pins_a>;
173     pinctrl-3 = <&uart4_pins_a>;
174     /delete-property/dmas;
175     /delete-property/dma-names;
176     status = "okay";
177 };
```

3、编译 stm32mp157d-atk.dts 设备树

最后我们肯定要编译 stm32mp157d-atk.dts，得到对应的.dtb 文件，打开 arch/arm/boot/dts/Makefile，到“dtb-$(CONFIG_ARCH_STM32)”配置项，在此配置项中加入“stm32mp157d-atk.dtb”，完成以后如图 17.3.2.2 所示：

```
 981 dtb-$(CONFIG_ARCH_STM32) += \
 982     stm32f429-disco.dtb \
 983     stm32f469-disco.dtb \
 984     stm32f746-disco.dtb \
 985     stm32f769-disco.dtb \
 986     stm32429i-eval.dtb \
 987     stm32746g-eval.dtb \
 988     stm32h743i-eval.dtb \
 989     stm32h743i-disco.dtb \
 990     stm32mp157a-avenger96.dtb \
 991     stm32mp157a-dk1.dtb \
 992     stm32mp157d-dk1.dtb \
 993     stm32mp157c-dk2.dtb \
 994     stm32mp157f-dk2.dtb \
 995     stm32mp157c-dk2-a7-examples.dtb \
 996     stm32mp157c-dk2-m4-examples.dtb \
 997     stm32mp157f-dk2-a7-examples.dtb \
 998     stm32mp157f-dk2-m4-examples.dtb \
 999     stm32mp157a-ed1.dtb \
1000     stm32mp157c-ed1.dtb \
1001     stm32mp157d-ed1.dtb \
1002     stm32mp157f-ed1.dtb \
1003     stm32mp157a-ev1.dtb \
1004     stm32mp157c-ev1.dtb \
1005     stm32mp157d-ev1.dtb \
1006     stm32mp157f-ev1.dtb \
1007     stm32mp157c-ev1-a7-examples.dtb \
1008     stm32mp157c-ev1-m4-examples.dtb \
1009     stm32mp157f-ev1-a7-examples.dtb \
1010     stm32mp157f-ev1-m4-examples.dtb \
1011     stm32mp157d-atk.dtb
1012
```

图 17.3.2.2 Makefile 文件

图 17.3.2.2 中第 1011 行添加了“stm32mp157d-atk.dtb”，表示编译的时候也将 stm32mp157d-atk.dts 编译为 stm32mp157d-atk.dtb。

17.3.3 关闭内核模块验证

后续做 Linux 驱动实验的时候我们都是编译驱动模块，然后在系统里面加载，加载的时候系统会验证模块，有时候会验证出错。比如板子运行的系统和编译驱动模块所用的系统不一致的时候，这样为我们的学习带来了很大的不便，为了方便开发，我们可以关闭内核模块验证。打开默认配置文件 stm32mp1_atk_defconfig，里面有如下所示配置项目：

```
CONFIG_MODULE_SIG=y
CONFIG_MODULE_SIG_ALL=y
CONFIG_MODULE_SIG_SHA256=y
CONFIG_MODULE_SIG_HASH="sha256"
```

将上面 4 个配置项屏蔽掉，如图 17.3.3.1 所示：

```
669 CONFIG_MODVERSIONS=y
670 # CONFIG_MODULE_SRCVERSION_ALL is not set
671 # CONFIG_MODULE_SIG=y
672 # CONFIG_MODULE_SIG_FORCE is not set
673 # CONFIG_MODULE_SIG_ALL=y
674 # CONFIG_MODULE_SIG_SHA1 is not set
675 # CONFIG_MODULE_SIG_SHA224 is not set
676 # CONFIG_MODULE_SIG_SHA256=y
677 # CONFIG_MODULE_SIG_SHA384 is not set
678 # CONFIG_MODULE_SIG_SHA512 is not set
679 # CONFIG_MODULE_SIG_HASH="sha256"
680 # CONFIG_MODULE_COMPRESS is not set
681 # CONFIG_MODULE_ALLOW_MISSING_NAMESPACE_IMPORTS is not set
682 # CONFIG_UNUSED_SYMBOLS is not set
683 # CONFIG_TRIM_UNUSED_KSYMS is not set
```

图 17.3.3.1 关闭内核模块验证功能

另外，我们也可以直接在 Linux 的图形化配置界面上关闭掉内核模块验证，输入如下命令打开 Linux 内核图形化配置界面：

```
make menuconfig
```

配置路径如下：

```
-> Enable loadable module support (MODULES [=y])
    ->Module signature verification
```

取消对“Module signature verification”选项的选中，如图 17.3.3.2 所示：

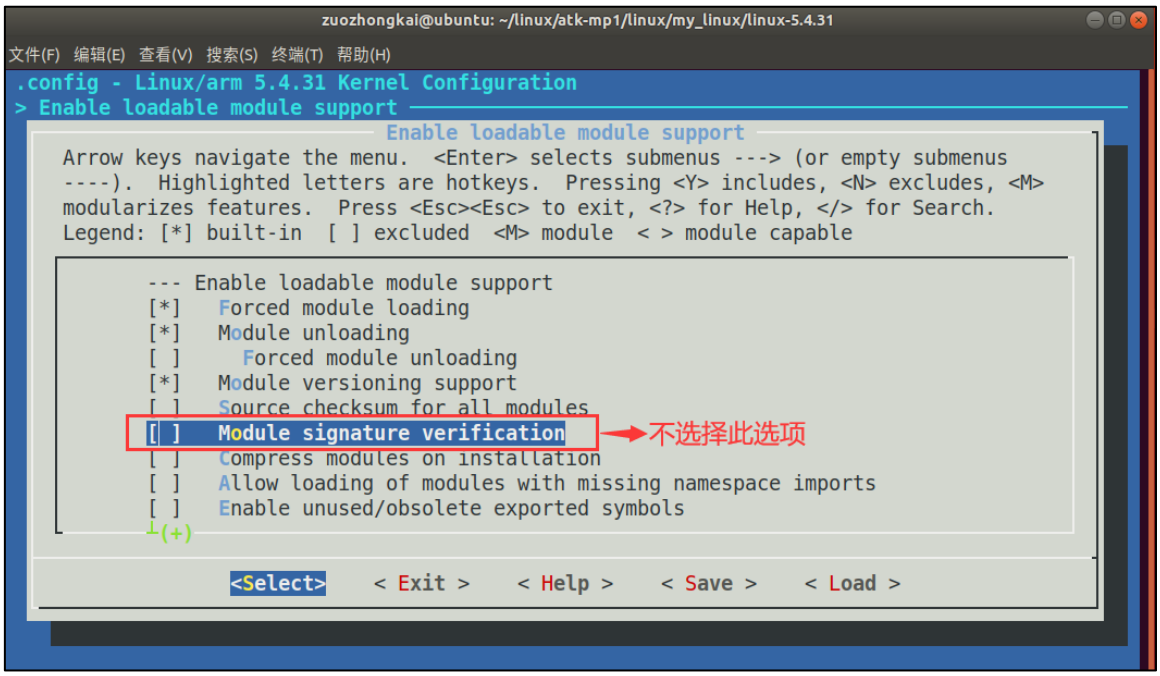


图 17.3.3.2 取消内核模块验证

配置完成以后退出，打开.config 文件，查看还有没有 CONFIG_MODULE_SIG 配置项，如图 17.3.3.3 所示：

```
667 # CONFIG_MODULE_FORCE_UNLOAD is not set
668 CONFIG_MODVERSIONS=y
669 # CONFIG_MODULE_SRCVERSION_ALL is not set
670 # CONFIG_MODULE_SIG is not set
671 # CONFIG_MODULE_COMPRESS is not set
672 # CONFIG_MODULE_ALLOW_MISSING_NAMESPACE_IMPORTS is not set
673 # CONFIG_UNUSED_SYMBOLS is not set
674 # CONFIG_TRIM_UNUSED_KSYMS is not set
```

图 17.3.3.3 .config 文件

从图 17.3.3.3 可以看出，COFIG_MODULE_SIG 选项没有设置，同样的 CONFIG_MODULE_SIG_ALL、CONFIG_MODULE_SIG_SHA256 和 CONFIG_MODULE_SIG_HASH 也一起消失了。

最后我们需要将配置项保存到 stm32mp1_atk_defconfig 里面，打开图形化配置界面，选择<Save>项目，将修改后的所有配置项保存到 stm32mp1_atk_defconfig 文件里面，如图 17.3.3.4 所示：

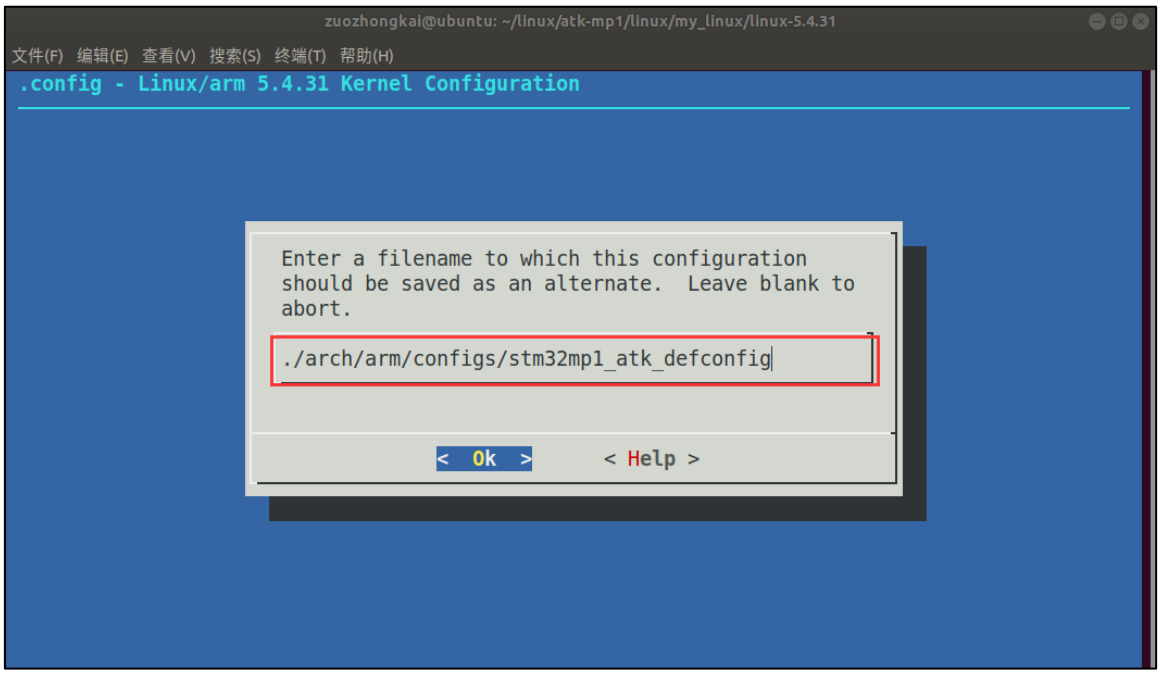


图 17.3.3.4 保存默认配置

后续试验中，只要通过图形化界面修改了 Linux 内核配置，最好及时将其保存到 stm32mp1_atk_defconfig 文件。因为图形化界面修改的配置只是暂时保存到.config 文件里面，一旦使用“make clean”清理工程，那么.config 文件就会被删除掉，所有的配置也就丢失了！

17.3.4 关闭内核 log 信息时间戳

大家可以仔细观察会发现 Linux 系统启动的时候每行信息前面都会打印出相应的时间，如图 17.3.4.1 所示：

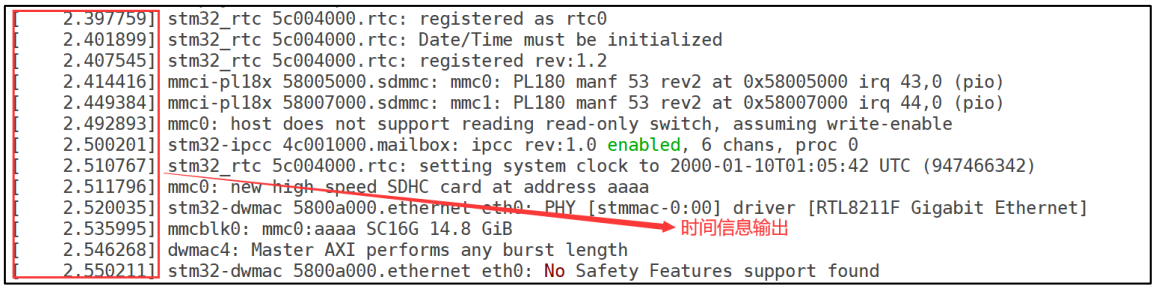


图 17.3.4.1 log 时间信息输出

从图 17.3.4.1 可以看出，每行 log 信息前面都有时间信息，在调试的时候这个时间信息看起来比较“烦人”我们可以通过配置内核将其关闭，配置路径如下：

```
-> Kernel hacking
    -> printk and dmesg options
        ->Show timing information on printks        //取消选中
```

如图 17.3.4.1 所示：

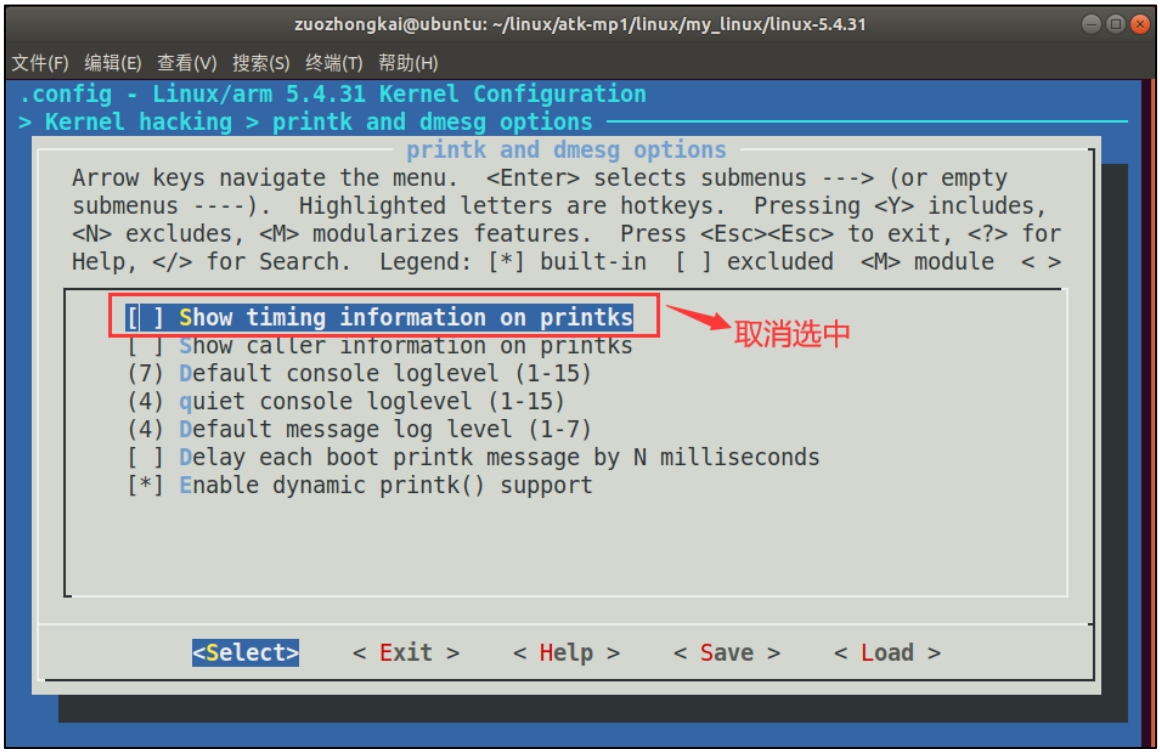


图 17.3.4.1 取消 printk 时间戳

17.3.5 编译测试

设备树修改好以后就可以编译了，因为我们只是修改了设备树，所以编译脚本直接用 17.1.2 小节创建的 stm32mp157d_atk.sh 编译脚本即可！

编译完成以后在 arch/arm/boot 目录下生成 uImage 镜像，在 arch/arm/boot/dts 目录下生成 stm32mp157d-atk.dtb 文件，将这两个文件拷贝到 tftp 服务器目录下，然后在 uboot 中使用 tftp 命令下载并运行，命令如下：

```
tftp c2000000 uImage
tftp c4000000 stm32mp157d-atk.dtb
bootm c2000000 - c4000000
```

当出现如图 17.3.5.1 所示的 log 信息以后就说明 Linux 启动运行成功：

```
Booting Linux on physical CPU 0x0
Linux version 5.4.31 (zuozhongkai@ubuntu) (gcc version 9.2.1 20191025 (GNU Toolchain for the A-profile Archit
ecture 9.2-2019.12 (arm-9.10))) #1 SMP PREEMPT Sat Dec 19 17:31:56 CST 2020
CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
CPU: div instructions available: patching division code
CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
OF: fdt: Machine model: STMicroelectronics STM32MP157D eval daughter
Memory policy: Data cache writealloc
Reserved memory: created DMA memory pool at 0x10000000, size 0 MiB
OF: reserved mem: initialized node mcuram2@10000000, compatible id shared-dma-pool
Reserved memory: created DMA memory pool at 0x10040000, size 0 MiB
```

图 17.3.5.1 Linux 运行 log 信息

17.4 烧写系统镜像到 EMMC 里面

上面已经测试过了 Linux 系统镜像和设备树，但是我们是通过 tftp 命令从网络上下载测试的，实际产品开发中最终是要将系统烧写到外部 Flash 中的，比如 EMMC。本节我们来学习一下如何将 uIamge 和 stm32mp157d-atk.dtb 打包成 ext4 格式，然后通过 STM32CubeProgrammer

烧写到 EMMC 里面，最终启动 EMMC 里面的 Linux 系统。

17.4.1 系统镜像打包

首先就是将 uImage 和 stm32mp157d-atk.dtb 打包在一起，格式为 ext4 格式。当然了，你也可以向这个 ext4 格式打包文件里面添加其他的内容，比如图片等，本节我们只用到 uImage 和 stm32mp157d-atk.dtb。

打包过程是在 Ubuntu 下完成的，首先新建一个名为“bootfs”的文件夹，然后将 uImage 和 stm32mp157d-atk.dtb 这两个文件放到 bootfs 文件夹下，如图 17.4.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/bootfs$ ls
stm32mp157d-atk.dtb  uImage
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/bootfs$
```

图 17.4.1.1 bootfs 文件夹

1、新建 ext4 格式磁盘

首先新建一个 ext4 格式的磁盘，然后挂载这个 ext4 格式的磁盘，将 stm32mp157d-atk.dtb 和 uImage 拷贝到这个 ext4 磁盘即可。

进入到 bootfs 文件夹，然后输入如下命令创建 ext4 磁盘：

示例代码 17.4.1.1 ext4 磁盘创建命令

```
1 cd bootfs
2 dd if=/dev/zero of=bootfs.ext4 bs=1M count=10
3 mkfs.ext4 -L bootfs bootfs.ext4
```

第 1 行，进入到图 17.4.1.1 中的 bootfs 目录。

第 2 行，使用 dd 命令创建一个名为 bootfs.ext4 的磁盘，of 指定磁盘名字为“bootfs.ext4”；bs 指定磁盘输入/输出块大小为 1MB；count 指定磁盘的块数量为 10 个。因此 bootfs.ext4 只能存放不超过 10MB 的文件，如果要存放的文件总大小超过 10MB，那么就要适当调整 count 参数的大小。

第 3 行，使用 mkfs.ext4 将 bootfs.ext4 磁盘格式化为 ext4 格式。

完成以后就会生成名为“bootfs.ext4”的磁盘，如图 17.4.1.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/bootfs$ ls
bootfs.ext4  stm32mp157d-atk.dtb  uImage
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/bootfs$
```

图 17.4.1.2 bootfs.ext4 磁盘

2、将系统镜像拷贝到 ext4 磁盘中

首先创建一个目录用来挂载前面制作制作出来的 bootfs.ext4，比如我这里创建目录/mnt/bootfs，命令如下：

```
sudo mkdir /mnt/bootfs
```

接下来使用 mount 命令将 bootfs.ext4 挂载到/mnt/bootfs 目录下，命令如下：

```
cd /home/zuozhongkai/linux/atk-mp1/linux/bootfs
sudo mount bootfs.ext4 /mnt/bootfs/
```

挂载成功以后就将图 17.4.1.1 中的 uImage 和 stm32mp157d-atk.dtb 拷贝到/mnt/bootfs 目录下，命令如下：

```
cd /home/zuozhongkai/linux/atk-mp1/linux/bootfs
sudo cp uImage stm32mp157d-atk.dtb /mnt/bootfs/
```

拷贝完成以后使用 umount 卸载/mnt/bootfs 即可，命令如下：

```
sudo umount /mnt/bootfs
```

至此，uImage 和 stm32mp157d-atk.dtb 就已经打包到了图 17.4.1.2 中的 bootfs.ext4 中，稍后使用 STM32CubeProgrammer 软件将其烧写到 EMMC 里面。烧写之前最好在 Windows 下打开 bootfs.ext4 看一下，看看是否已经将 uImage 和 stm32mp157d-atk.dtb 打包进去，如图 17.4.1.3 所示：

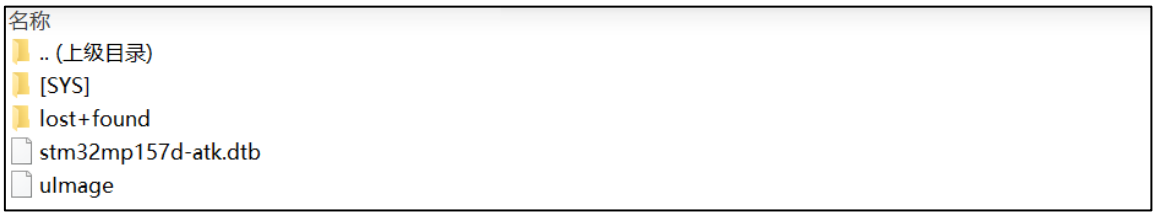


图 17.4.1.3 bootfs.ext4

17.4.2 烧写到 EMMC

接下来就是将上一小节打包好的 ext4 格式的 bootfs.ext4 烧写到开发板的 EMMC 里面，使用 STM32CubeProgrammer 软件完成此操作。将 bootfs.ext4 拷贝到以前创建的 images 目录下，如图 17.4.2.1 所示：

图 17.4.2.1 images 目录

修改 flashlayout 文件 tf-a.tsv，在后面加入 bootfs.ext4 的烧写脚本，如图 17.4.2.2 所示：

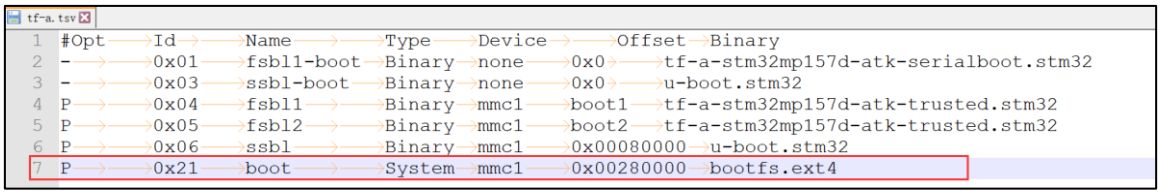


图 17.4.2.2 修改后的 flashlayout

图 17.4.2.2 中第 7 行就是 bootfs.ext4 的烧写脚本，设置好以后就可以使用 STM32CubeProgrammer 烧写系统了，烧写完成以后设置拨码开关从 EMMC 启动，启动以后进入 uboot 的命令行，输入如下命令查看 EMMC 分区 2 里面是否正确烧写了 uImage 和 stm32mp157d-atk.dtb：

```
ext4ls mmc 1:2
```

结果如图 17.4.2.3 所示：

```
STM32MP> ext4ls mmc 1:2
<DIR>       1024 .
<DIR>       1024 ..
<DIR>      12288 lost+found
         7310632 uImage
           63833 stm32mp157d-atk.dtb
STM32MP>
```

图 17.4.2.3 EMMC 分区 2

从图 17.4.2.3 可以看出，此时 EMMC 的分区 2 存在 uImage 和 stm32mp157d-atk.dtb 这两个文件，设置 bootcmd 环境变量，从 EMMC 里面读取系统镜像和设备树并启动，命令如下：

```
setenv bootcmd 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
saveenv
boot
```

如果烧写正常的话就会从 EMMC 的分区 2 中加载系统并运行，如图 17.4.2.4 所示：

```
7310632 bytes read in 190 ms (36.7 MiB/s)
63833 bytes read in 30 ms (2 MiB/s)
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31
   Created:      2020-12-15   8:36:47 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    7310568 Bytes = 7 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffed000, end cffff958 ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 5.4.31 (zuozhongkai@ubuntu) (gcc version 9.2.1 20191025 (GNU Toolchain for
 the A-profile Architecture 9.2-2019.12 (arm-9.10))) #1 SMP PREEMPT Tue Dec 15 16:32:11 CST 2020
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[    0.000000] OF: fdt: Machine model: STMicroelectronics STM32MP157D eval daughter
[    0.000000] Memory policy: Data cache writealloc
```

图 17.4.2.4 EMMC 启动系统

可以看出，将系统镜像烧写到 EMMC 里面还是很繁琐的，因此一般都是在产品最终开发完成，出厂的时候才会烧写到 EMMC 里面。在调试阶段都不会烧写到 EMMC 里面，而是在 uboot 里面使用 tftp 或 nfs 命令通过网络下载系统镜像并运行，这样当我们修改了系统以后只需要将系统镜像复制到 tftp 或 nfs 服务器目录即可，极大的简化开发方式。

17.5 根文件系统缺失错误

Linux 内核启动以后是需要根文件系统的，根文件系统存在哪里是由 uboot 的 bootargs 环境变量指定的，它会传递给 Linux 内核作为命令行(command line)参数。比如 13.4.2 小节设置 root=/dev/mmcblk2p3，也就是说根文件系统存储在/dev/mmcblk2p3 中，也就是 EMMC 的分区 3 中。如果我们不设置根文件系统路径，或者说根文件系统路径设置错误的话会出现什么问题？这个问题是很常见的，我们在实际的工作中开发一个产品，这个产品的第一版硬件出来以后我们是没有对应的根文件系统可用的，必须要自己做根文件系统。在没有根文件系统的情况下，Linux 内核启动的时候就会有如图 17.5.1 所示的错误：

```
[    2.925753] Exception stack(0xc1101f40 to 0xc1101f88)
[    2.930810] 1f40: 00000000 00000968 e7667074 c011b7c0 00000000 c1100000 c1104ca8 c1104ce8
[    2.939018] 1f60: 00000001 c105ca40 e7fffe40 00000000 c11efc18 c1101f90 c010a1c8 c010a1cc
[    2.947210] 1f80: 60000113 ffffffff
[    2.950694] [<c0101a8c>] (__irq_svc) from [<c010a1cc>] (arch_cpu_idle+0x38/0x3c)
[    2.958122] [<c010a1cc>] (arch_cpu_idle) from [<c01584f0>] (do_idle+0xc4/0x14c)
[    2.965443] [<c01584f0>] (do_idle) from [<c0158864>] (cpu_startup_entry+0x18/0x1c)
[    2.973031] [<c0158864>] (cpu_startup_entry) from [<c1000f10>] (start_kernel+0x4bc/0x4f4)
[    2.981230] [<c1000f10>] (start_kernel) from [<00000000>] (0x0)
[    2.987162] ---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
]---
```

图 17.5.1 根文件系统缺失错误

在图 17.5.1 中最后一行为：

end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)

也就是提示内核崩溃，VFS(虚拟文件系统)不能挂载根文件系统，因为根文件系统目录不存在。解决方法就是制作根文件系统，并且设置 uboot 的 bootargs 环境变量，指定根文件系统所在的目录。

第十八章 Busybox 根文件系统构建

前面我们已经移植了 TF-A、Uboot 和 Linux kernel，就剩最后一个 rootfs(根文件系统)了，本章我们就来学习一下根文件系统的组成以及如何构建根文件系统。这是 Linux 系统移植的最后一步，根文件系统构建好以后就意味着我们已经拥有了一个完整的、可以运行的最小系统。以后我们就在这个最小系统上编写、测试 Linux 驱动，移植一些第三方组件，逐步的完善这个最小系统。最终得到一个功能完善、驱动齐全、相对完善的操作系统。

18.1 根文件系统简介

根文件系统一般也叫做 rootfs，那么什么叫根文件系统？看到“文件系统”这四个字，很多人，包括我第一反应就是 FATFS、FAT、EXT4、YAFFS 和 NTFS 等这样的文件系统。在这里，根文件系统并不是 FATFS、EXT4 这样的文件系统代码，我们在学单片机的时候听到的 FATFS 是具体的文件系统代码，这个是属于 Linux 内核的一部分。Linux 中的根文件系统更像是一个文件夹或者叫做目录(在我看来就是一个文件夹，只不过是特殊的文件夹)，在这个目录里面会有很多的子目录。根目录和子目录中会有很多的文件，这些文件是 Linux 运行所必须的，比如库、常用的软件和命令、设备文件、配置文件等等。以后我们说到文件系统，如果不特别指明，统一表示根文件系统。对于根文件系统专业的解释，百度百科上是这么说的：

根文件系统首先是内核启动时所 mount(挂载)的第一个文件系统，内核代码映像文件保存在根文件系统中，而系统引导启动程序会在根文件系统挂载之后从中把一些基本的初始化脚本和服务等加载到内存中去运行。

百度百科上说内核代码镜像文件保存在根文件系统中，但是我们嵌入式 Linux 并没有将内核代码镜像保存在根文件系统中，而是保存到了其他地方。比如 NAND Flash 的指定存储地址、EMMC 专用分区中。根文件系统是 Linux 内核启动以后挂载(mount)的第一个文件系统，然后从根文件系统中读取初始化脚本，比如 rcS，inittab 等。根文件系统和 Linux 内核是分开的，单独的 Linux 内核是没法正常工作的，必须要搭配根文件系统。如果不提供根文件系统，Linux 内核在启动的时候就会提示内核崩溃(Kernel panic)的提示，这个在 17.5 小节已经说过了。

根文件系统的这个“根”字就说明了这个文件系统的重要性，它是其他文件系统的根，没有这个“根”，其他的文件系统或者软件就别想工作。比如我们常用的 ls、mv、ifconfig 等命令其实就是一个个小软件，只是这些软件没有图形界面，而且需要通过输入命令的形式来运行。这些小软件就保存在根文件系统中，这些小软件是怎么来的呢？这个就是我们本章教程的目的，教大家来构建自己的根文件系统，这个根文件系统是满足 Linux 运行的最小根文件系统，后续我们可以根据自己的实际工作需求不断的去填充这个最小根文件系统，最终使其成为一个相对完善的根文件系统。

在构建根文件系统之前，我们先来看一下根文件系统里面大概都有些什么内容，以 Ubuntu 为例，根文件系统的目录名字为‘/’，没看错就是一个斜杠，所以输入如下命令就可以进入根目录中：

```
cd /            //进入根目录
```

进入根目录以后输入“ls”命令查看根目录下的内容都有哪些，结果如图 18.1.1 所示：

```
zuozhongkai@ubuntu:/$ ls
bin    dev   initrd.img      lib32       media  proc  sbin  swapfile  usr      vmlinuz.old
boot   etc   initrd.img.old  lib64       mnt    root  snap  sys       var
cdrom  home  lib             lost+found  opt    run   srv   tmp       vmlinuz
zuozhongkai@ubuntu:/$
```

图 18.1.1 Ubuntu 根目录

图 18.1.1 中根目录下子目录和文件不少，但是这些都是 Ubuntu 所需要的，其中有很多子目录和文件我们嵌入式 Linux 是用不到的，所以这里就讲解一些常用的子目录：

1、/bin 目录

看到“bin”大家应该能想到 bin 文件，bin 文件就是可执行文件。所以此目录下存放着系统需要的可执行文件，一般都是一些命令，比如 ls、mv 等命令。此目录下的命令所有的客户都可以使用。

2、/dev 目录

dev 是 device 的缩写，所以此目录下的文件都是和设备有关的，此目录下的文件都是设备文件。在 Linux 下一切皆文件，即使是硬件设备，也是以文件的形式存在的，比如/dev/ttySTM0(STM32MP1 根目录会有此文件)就表示 STM32MP157 的串口 4，我们要想通过串口 4 发送或者接收数据就要操作文件/dev/ttySTM0，通过对文件/dev/ttySTM0 的读写操作来实现串口 4 的数据收发。

3、/etc 目录

此目录下存放着各种配置文件，大家可以进入 Ubuntu 的 etc 目录看一下，里面的配置文件非常多！但是在嵌入式 Linux 下此目录会很简洁。

4、/lib 目录

lib 是 library 的简称，也就是库的意思，因此此目录下存放着 Linux 所必须的库文件。这些库文件是共享库，命令和用户编写的应用程序要使用这些库文件。

5、/mnt 目录

临时挂载目录，一般是空目录，可以在此目录下创建空的子目录，比如/mnt/sd、/mnt/usb，这样就可以将 SD 卡或者 U 盘挂载到/mnt/sd 或者/mnt/usb 目录中。

6、/proc 目录

此目录一般是空的，当 Linux 系统启动以后会将此目录作为 proc 文件系统的挂载点，proc 是个虚拟文件系统，没有实际的存储设备。proc 里面的文件都是临时存在的，一般用来存储系统运行信息文件。

7、/usr 目录

要注意，usr 不是 user 的缩写，而是 Unix Software Resource 的缩写，也就是 Unix 操作系统软件资源目录。这里有个小知识点，那就是 Linux 一般被成为类 Unix 操作系统，苹果的 MacOS 也是类 Unix 操作系统。关于 Linux 和 Unix 操作系统的渊源大家可以直接在网上找 Linux 的发展历史来看。既然是软件资源目录，因此/usr 目录下也存放着很多软件，一般系统安装完成以后此目录占用的空间最多。

8、/var 目录

此目录存放一些可以改变的数据。

9、/sbin 目录

此目录也用于存放一些可执行文件，但是此目录下的文件或者说命令只有管理员才能使用，主要用户系统管理。

10、/sys 目录

系统启动以后此目录作为 sysfs 文件系统的挂载点，sysfs 是一个类似于 proc 文件系统的特殊文件系统，sysfs 也是基于 ram 的文件系统，也就是说它也没有实际的存储设备。此目录是系统设备管理的重要目录，此目录通过一定的组织结构向用户提供详细的内核数据结构信息。

11、/opt

可选的文件、软件存放区，由用户选择将哪些文件或软件放到此目录中。

关于 Linux 的根目录就介绍到这里，接下来的构建根文件系统就是研究如何创建上面这些子目录以及子目录中的文件。

18.2 BusyBox 构建根文件系统

18.2.1 BusyBox 简介

上一小节说了，根文件系统里面就是一堆的可执行文件和其他文件组成的？难道我们得一个一个的从网上去下载这些文件？显然这是不现实的！那么有没有人或者组织专门干这个事呢？他们负责“收集”这些文件，然后将其打包，像我们这样的开发者可以直接拿来用。答案是有的，它就叫做 BusyBox！其名字分为“Busy”和“Box”，也就是忙碌的盒子。盒子是用来放东西的，忙碌的是因为它要提供根文件系统所需的文件，所以忙碌。BusyBox 是一个集成了大量的 Linux 命令和工具的软件，像 ls、mv、ifconfig 等命令 BusyBox 都会提供。BusyBox 就是一个大的工具箱，这个工具箱里面集成了 Linux 的许多工具和命令。一般下载 BusyBox 的源码，然后配置 BusyBox，选择自己想要的功能，最后编译即可。

BusyBox 可以在其官网下载到，官网地址为：https://busybox.net/，官网比较简陋，如图 18.2.1.1 所示：

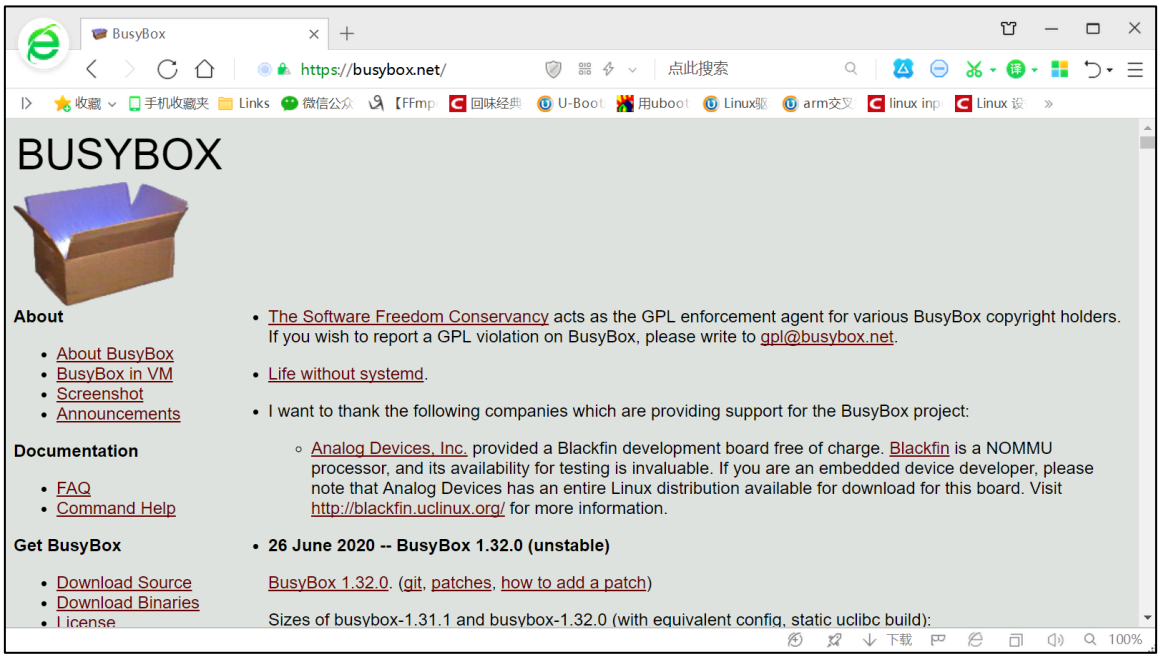


图 18.2.1.1 BusyBox 官网

在官网左侧的“Get BusyBox”栏有一行“Download Source”，点击“Download Source”即可打开 BusyBox 的下载页，如图 18.2.1.2 所示：

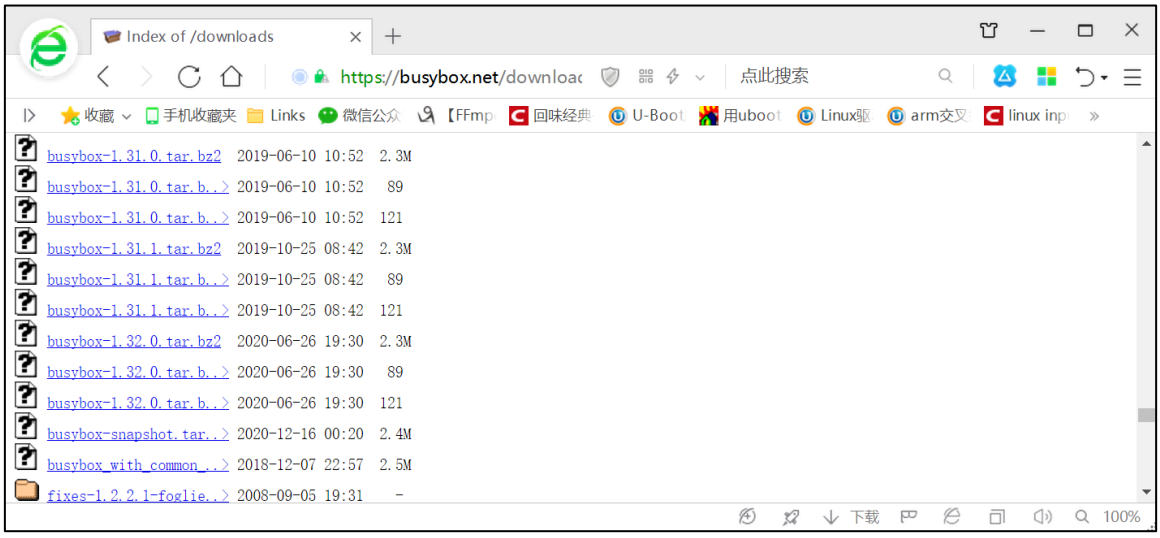


图 18.2.1.2 BusyBox 下载页

从图 18.2.1.2 可以看出，目前最新的 BusyBox 版本是 1.32.0，大家下载“busybox-1.32.0.tar.bz2”这个压缩包即可，我们已经将其放到了开发板光盘中，路径为：1、例程源码->6、BusyBox 源码->busybox-1.32.0.tar.bz2，BusyBox 准备好以后就可以构建根文件系统了。

18.2.2 编译 BusyBox 构建根文件系统

一般我们在 Linux 驱动开发的时候都是通过 nfs 挂载根文件系统的，当产品最终上市开卖的时候才会将根文件系统烧写到 EMMC 或者 NAND 中。所以要在 4.2.1 小节中设置的 nfs 服务器目录中创建一个名为 rootfs 的子目录(名字大家可以随意起，为了方便就用了 rootfs)，比如我的电脑中“/home/zuozhongkai/linux/nfs”就是我设置的 NFS 服务器目录，使用如下命令创建名为 rootfs 的子目录：

```
cd /home/zuozhongkai/linux/nfs
mkdir rootfs
```

创建好的 rootfs 子目录就用来存放我们的根文件系统了。

将 busybox-1.32.0.tar.bz2 发送到 Ubuntu 中，存放位置大家随便选择。然后使用如下命令将其解压：

```
tar -vxjf busybox-1.32.0.tar.bz2
```

解压完成以后进入到 busybox-1.32.0 目录中，此目录中的文件和文件夹如图 18.2.2.1 所示：

```
zuozhongkai@ubuntu:~/linux/busybox$ cd busybox-1.32.0/
zuozhongkai@ubuntu:~/linux/busybox/busybox-1.32.0$ ls
applets         editors      Makefile               qemu_multiarch_testing
applets_sh      examples     Makefile.custom        README
arch            findutils    Makefile.flags         runit
archival        include      Makefile.help          scripts
AUTHORS         init         make_single_applets.sh selinux
Config.in       INSTALL      miscutils              shell
configs         klibc-utils  modutils               size_single_applets.sh
console-tools   libbb        networking             sysklogd
coreutils       libpwdgrp    NOFORK_NOEXEC.lst      testsuite
debianutils     LICENSE      NOFORK_NOEXEC.sh       TODO
docs            loginutils   printutils             TODO_unicode
e2fsprogs       mailutils    procps                 util-linux
zuozhongkai@ubuntu:~/linux/busybox/busybox-1.32.0$
```

图 18.2.2.1 busybox-1.32.0 目录内容

1、修改 Makefile，添加编译器

同 Uboot 和 Linux 移植一样，打开 busybox 的顶层 Makefile，添加 ARCH 和 CROSS_COMPILE 的值，如下所示：

示例代码 18.2.2.1 Makefile 代码段

```
164 CROSS_COMPILE ?= /usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-
none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-
......
190 ARCH ?= arm
```

在示例代码 18.2.2.1 中 CORSS_COMPILE 使用了绝对路径！主要是为了防止编译出错。

2、busybox 中文字符支持

如果默认直接编译 busybox 的话，在使用 SecureCRT 的时候中文字符是显示不正常的，中文字符会显示为“?”，比如你的中文目录，中文文件都显示为“?”。不知道从哪个版本开始 busybox 中的 shell 命令对中文输入即显示做了限制，即使内核支持中文但在 shell 下也依然无法正确显示。

所以我们需要修改 busybox 源码，取消 busybox 对中文显示的限制，打开文件 busybox-1.32.0/libbb/printable_string.c，找到函数 printable_string2，缩减后的函数内容如下：

示例代码 18.2.2.2 libbb/printable_string.c 代码段

```
12 const char* FAST_FUNC printable_string2(uni_stat_t *stats,
                                           const char *str)
13 {
14  char *dst;
15  const char *s;
16
17  s = str;
18  while (1) {
19      unsigned char c = *s;
20      if (c == '\0') {
21          /* 99+% of inputs do not need conversion */
22          if (stats) {
23              stats->byte_count = (s - str);
24              stats->unicode_count = (s - str);
25              stats->unicode_width = (s - str);
26          }
27          return str;
28      }
29      if (c < ' ')
30          break;
31      if (c >= 0x7f)
32          break;
33      s++;
34  }
35
```

```
36 #if ENABLE_UNICODE_SUPPORT
37  dst = unicode_conv_to_printable(stats, str);
38 #else
39  {
40      char *d = dst = xstrdup(str);
41      while (1) {
42          unsigned char c = *d;
43          if (c == '\0')
44              break;
45          if (c < ' ' || c >= 0x7f)
46              *d = '?';
47          d++;
48      }
49      if (stats) {
50          stats->byte_count = (d - dst);
51          stats->unicode_count = (d - dst);
52          stats->unicode_width = (d - dst);
53      }
54  }
55 #endif
56  return auto_string(dst);
57 }
```

第 31 和 32 行，当字符大于 0X7F 以后就跳出去了。

第 45 和 46 行，如果支持 UNICODE 码的话，当字符大于 0X7F 就直接输出‘?’。

所以我们需要对这 4 行代码进行修改，修改以后如下所示：

示例代码 18.2.2.3 libbb/printable_string.c 代码段

```
12 const char* FAST_FUNC printable_string2(uni_stat_t *stats, const
char *str)
13 {
14  char *dst;
15  const char *s;
16
17  s = str;
18  while (1) {
19      unsigned char c = *s;
20      if (c == '\0') {
21          /* 99+% of inputs do not need conversion */
22          if (stats) {
23              stats->byte_count = (s - str);
24              stats->unicode_count = (s - str);
25              stats->unicode_width = (s - str);
26          }
27          return str;
```

```
28      }
29      if (c < ' ')
30          break;
31      /* 注释掉下面这个两行代码 */
32      /* if (c >= 0x7f)
33          break; */
34      s++;
35  }
36
37  #if ENABLE_UNICODE_SUPPORT
38   dst = unicode_conv_to_printable(stats, str);
39  #else
40   {
41      char *d = dst = xstrdup(str);
42      while (1) {
43          unsigned char c = *d;
44          if (c == '\0')
45              break;
46          /* 修改下面代码 */
47          /* if (c < ' ' || c >= 0x7f) */
48          if( c < ' ')
49              *d = '?';
50          d++;
51      }
52      if (stats) {
53          stats->byte_count = (d - dst);
54          stats->unicode_count = (d - dst);
55          stats->unicode_width = (d - dst);
56      }
57   }
58  #endif
59   return auto_string(dst);
60  }
```

示例代码 18.2.2.3 中红色部分的代码就是被修改以后的，主要就是禁止字符大于 0X7F 以后 break 和输出‘?’。

接着打开文件 busybox-1.32.0/libbb/unicode.c，找到如下内容：

示例代码 18.2.2.4 libbb/unicode.c 代码段

```
1009 static char* FAST_FUNC unicode_conv_to_printable2(uni_stat_t
*stats, const char *src, unsigned width, int flags)
1010 {
1011    char *dst;
1012    unsigned dst_len;
1013    unsigned uni_count;
```

```
1014    unsigned uni_width;
1015
1016    if (unicode_status != UNICODE_ON) {
1017        char *d;
1018        if (flags & UNI_FLAG_PAD) {
1019            d = dst = xmalloc(width + 1);
1020            while ((int)--width >= 0) {
1021                unsigned char c = *src;
1022                if (c == '\0') {
1023                    do
1024                        *d++ = ' ';
1025                    while ((int)--width >= 0);
1026                    break;
1027                }
1028                *d++ = (c >= ' ' && c < 0x7f) ? c : '?';
1029                src++;
1030            }
1031            *d = '\0';
1032        } else {
1033            d = dst = xstrndup(src, width);
1034            while (*d) {
1035                unsigned char c = *d;
1036                if (c < ' ' || c >= 0x7f)
1037                    *d = '?';
1038                d++;
1039            }
1040        }
1041        if (stats) {
1042            stats->byte_count = (d - dst);
1043            stats->unicode_count = (d - dst);
1044            stats->unicode_width = (d - dst);
1045        }
1046        return dst;
1047    }
......
1139    return dst;
1140 }
```

第 1028 行，当字符大于 0X7F 以后，*d++就为‘?’。

第 1036 和 1037 行，当字符大于 0X7F 以后，*d 也为‘?’。

修改示例代码 18.2.2.4，修改后内容如下所示：

示例代码 18.2.2.5 libbb/unicode.c 代码段

```
1009 static char* FAST_FUNC unicode_conv_to_printable2(uni_stat_t
*stats, const char *src, unsigned width, int flags)
```

```
1010 {
1011     char *dst;
1012     unsigned dst_len;
1013     unsigned uni_count;
1014     unsigned uni_width;
1015 
1016     if (unicode_status != UNICODE_ON) {
1017         char *d;
1018         if (flags & UNI_FLAG_PAD) {
1019             d = dst = xmalloc(width + 1);
1020             while ((int)--width >= 0) {
1021                 unsigned char c = *src;
1022                 if (c == '\0') {
1023                     do
1024                         *d++ = ' ';
1025                     while ((int)--width >= 0);
1026                     break;
1027                 }
1028                 /* 修改下面一行代码 */
1029                 /* *d++ = (c >= ' ' && c < 0x7f) ? c : '?'; */
1030                 *d++ = (c >= ' ') ? c : '?';
1031                 src++;
1032             }
1033             *d = '\0';
1034         } else {
1035             d = dst = xstrndup(src, width);
1036             while (*d) {
1037                 unsigned char c = *d;
1038                 /* 修改下面一行代码 */
1039                 /* if (c < ' ' || c >= 0x7f) */
1040                 if(c < ' ')
1041                     *d = '?';
1042                 d++;
1043             }
1044         }
1045         if (stats) {
1046             stats->byte_count = (d - dst);
1047             stats->unicode_count = (d - dst);
1048             stats->unicode_width = (d - dst);
1049         }
1050         return dst;
1051     }
......
```

```
1143    return dst;
1144 }
```

示例代码 18.2.2.5 中红色部分的代码就是被修改以后的，同样主要是禁止字符大于 0X7F 的时候设置为‘?’。busybox 中文字符支持跟代码修改有关的就改好了，最后还需要配置 busybox 来使能 unicode 码，这个稍后我们配置 busybox 的时候在设置。

3、配置 busybox

和我们编译 Uboot、Linux kernel 一样，我们要先对 busybox 进行默认的配置，有以下几种配置选项：

①、defconfig，缺省配置，也就是默认配置选项。

②、allyesconfig，全选配置，也就是选中 busybox 的所有功能。

③、allnoconfig，最小配置。

我们一般使用默认配置即可，因此使用如下命令先使用默认配置来配置一下 busybox：

```
make defconfig
```

busybox 也支持图形化配置，通过图形化配置我们可以进一步选择自己想要的功能，输入如下命令打开图形化配置界面：

```
make menuconfig
```

打开以后如图 18.2.2.2 所示：

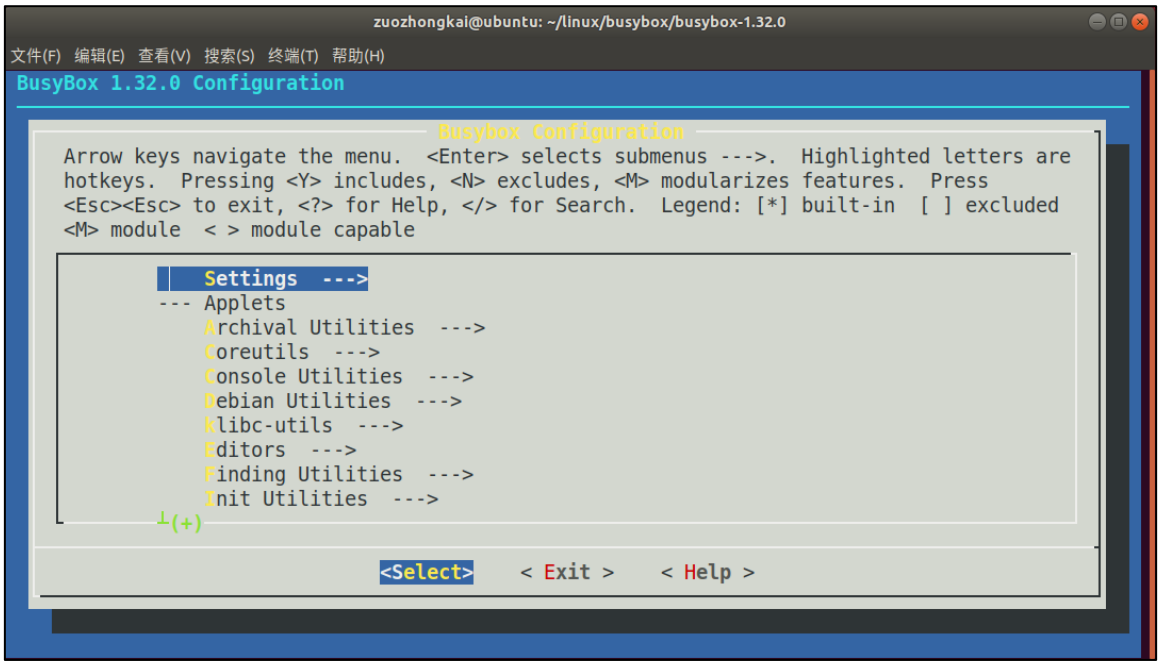


图 18.2.2.2 busybox 图形化配置界面

配置路径如下：

```
Location:
    -> Settings
        -> Build static binary (no shared libs)
```

选项“Build static binary (no shared libs)”用来决定是静态编译是动态编译，静态编译的话就不需要库文件，但是编译出来的根文件系统会很大。动态编译的话要求根文件系统中有库文件，但是编译出来的 busybox 会小很多。这里我们不能采用静态编译！因为采用静态编译的话 DNS 会出问题！无法进行域名解析，配置如图 18.2.2.3 所示：

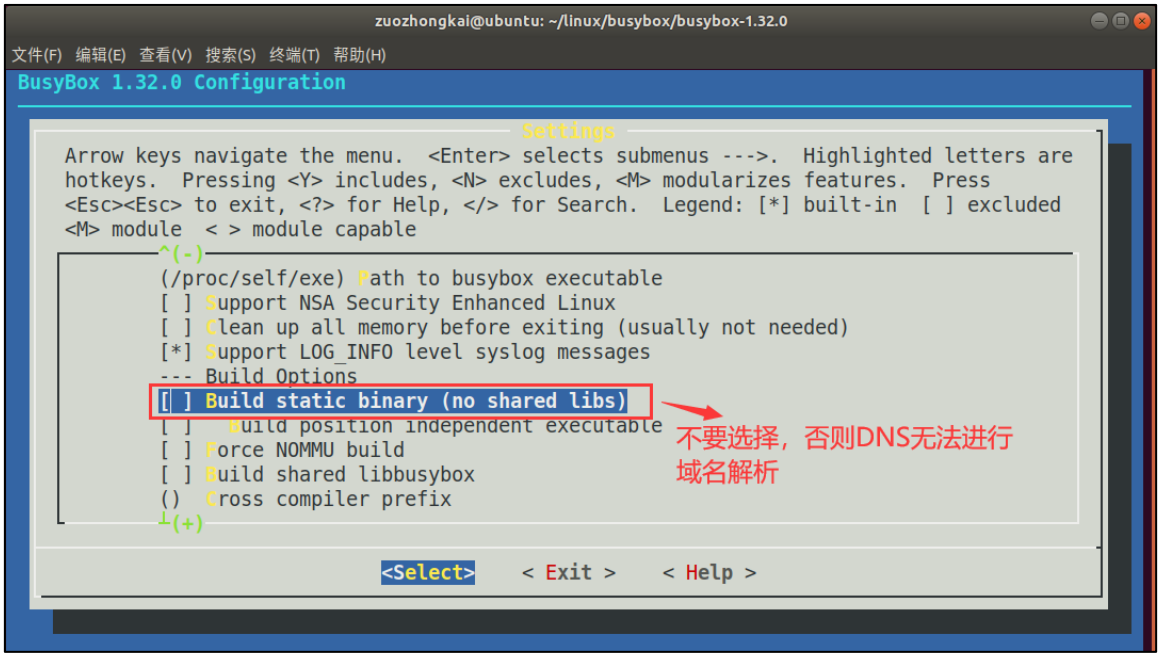


图 18.2.2.3 不选择“Build static binary (no shared libs)”

继续配置如下路径配置项：

Location:
-> Settings
-> vi-style line editing commands

结果如图 18.2.2.4 所示：

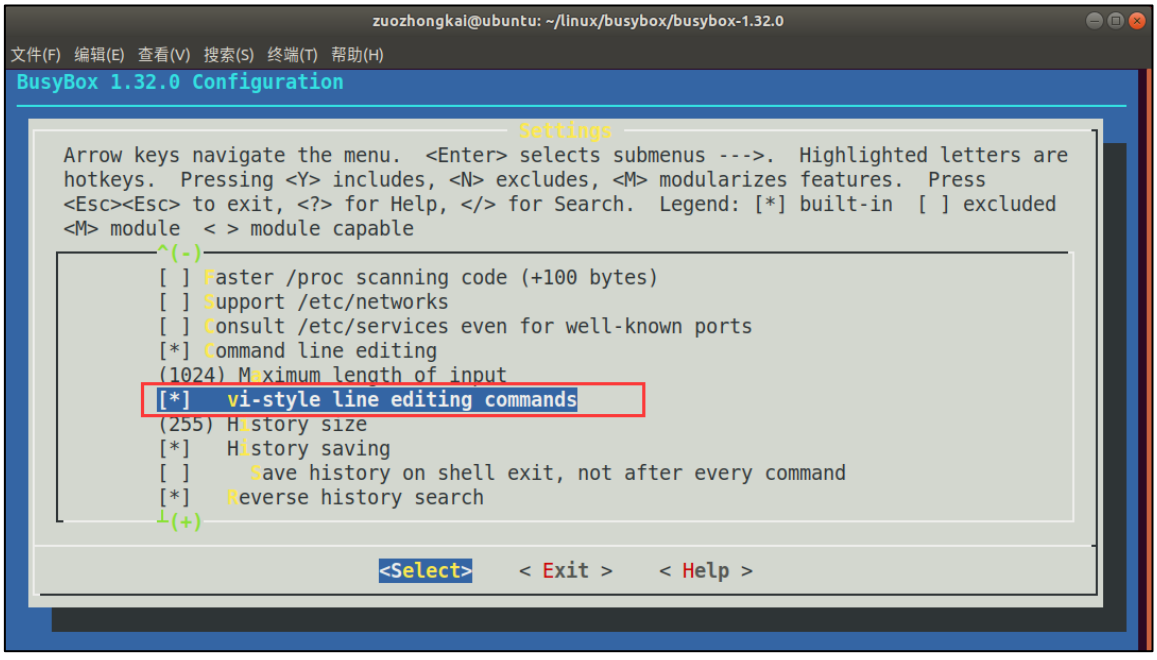


图 18.2.2.4 选择“vi-style line editing commands ”

继续配置如下路径配置项：

Location:
-> Linux Module Utilities
-> Simplified modutils

默认会选中“Simplified modutils”，这里我们要取消勾选!!结果如图 18.2.2.5 所示：

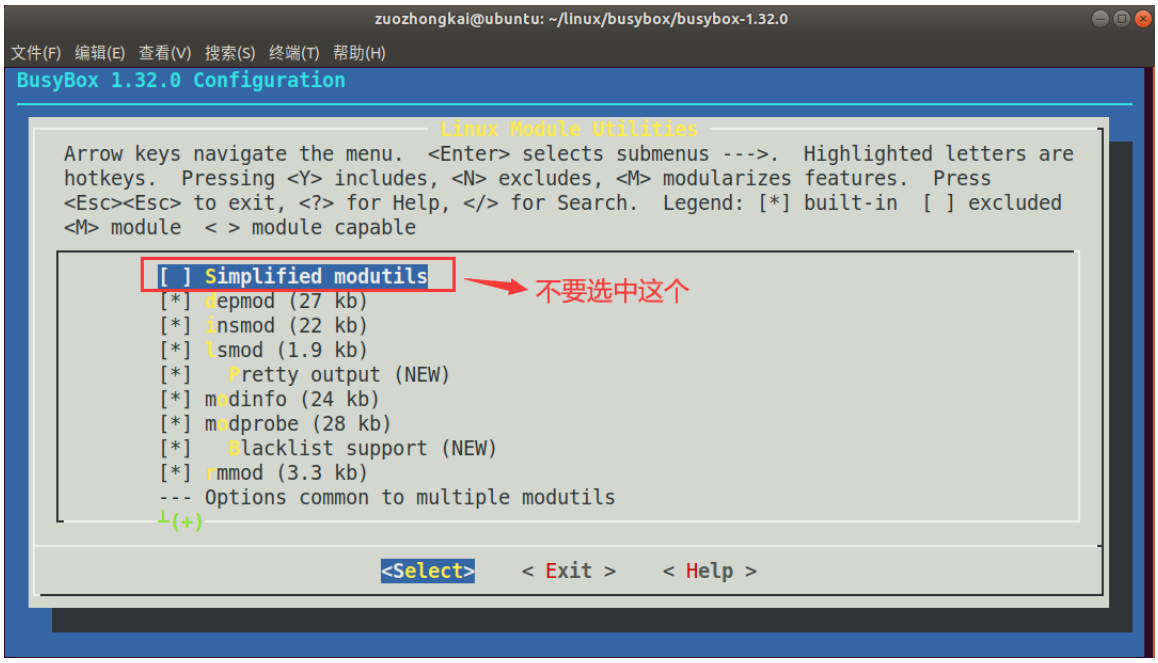


图 18.2.2.5 取消选中“Simplified modutils”

继续配置如下路径配置项：

Location:
-> Linux System Utilities
-> mdev (17 kb) //确保下面的全部选中，默认都是选中的

结果如图 18.2.2.6 所示：

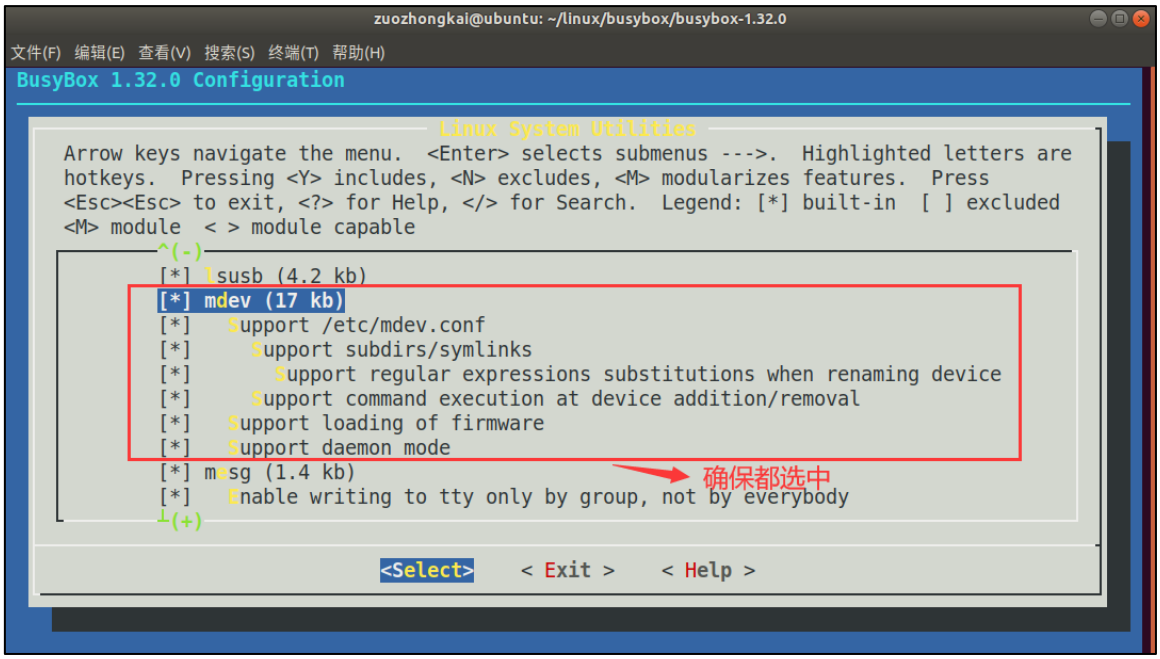


图 18.2.2.6 “mdev”配置项

最后就是使能 busybox 的 unicode 编码以支持中文，配置路径如下：

Location:
-> Settings

-> Support Unicode //选中
-> Check $LC_ALL, $LC_CTYPE and $LANG environment variables //选中

结果如图 18.2.2.7 所示：

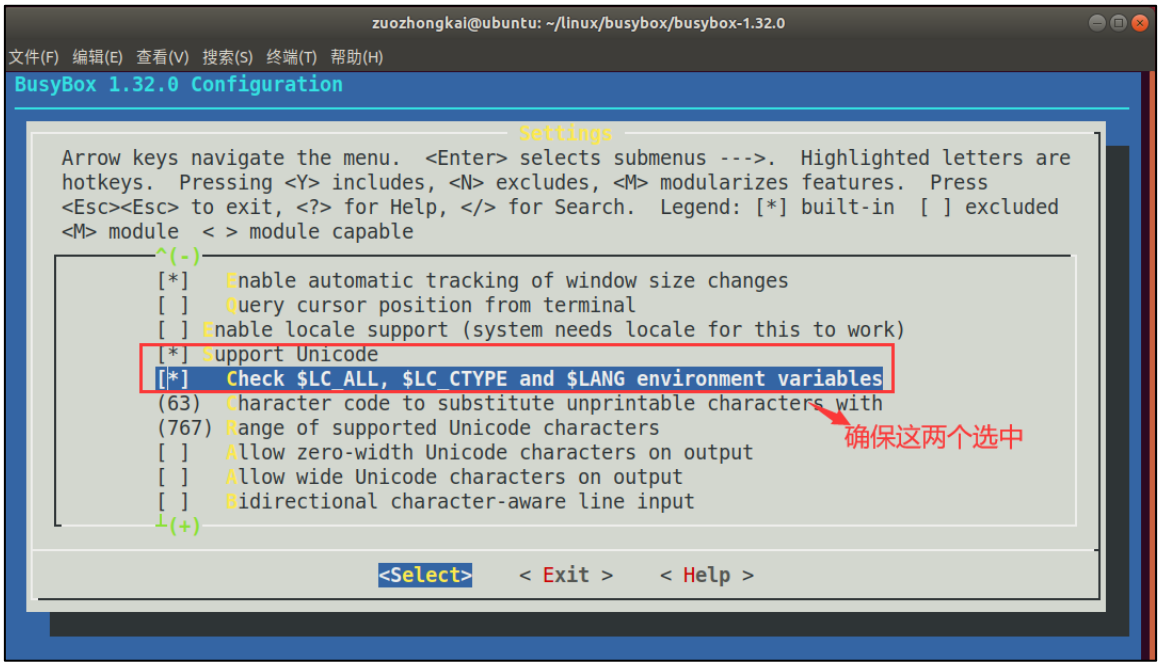


图 18.2.2.7 中文支持

busybox 的配置就到此结束了，大家也可以根据自己的实际需求选择配置其他的选项，不过对于初学者笔者不建议再做其他的修改，可能会出现编译出错的情况发生。

我们使用图形化修改过 Busybox 的配置以后最好保存一下配置文件，以防“make clean”以后删除掉以前的配置，在配置主界面上，选中“Save Configuration to an Alternate File”，如图 18.2.2.8 所示：

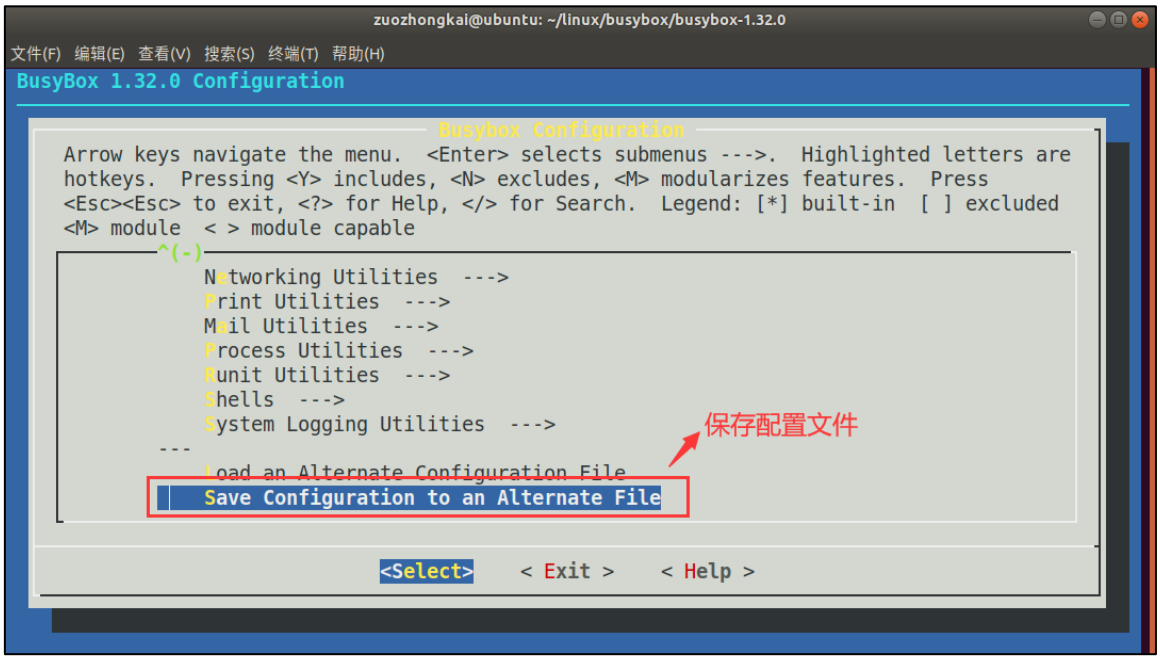


图 18.2.2.8 保存配置文件

选中以后会让你输入配置文件名，Busybox 的默认配置文件都是保存到 configs 目录下，配

置文件的名字后缀必须是“_defconfig”， 这里设置配置文件名字为“stm32mp1_atk_defconfig”，如图 18.2.2.9 所示：

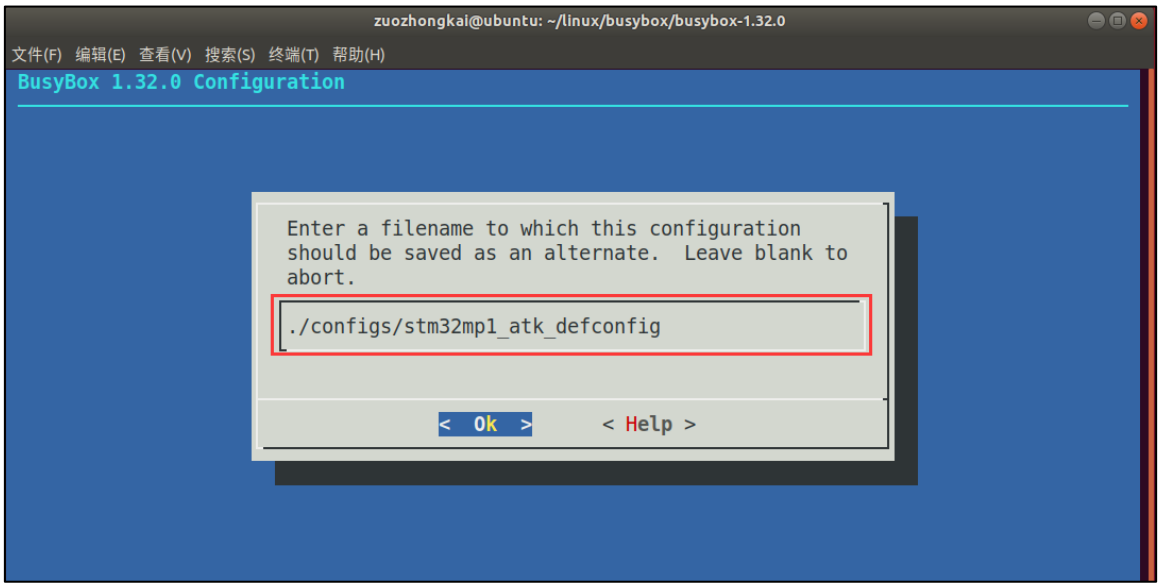


图 18.2.2.9 设置配置文件名字

点击图 18.2.2.9 中的“Ok”按钮即可完成保存，以后要想使用我们的自己的配置文件，直接输入：

```
make stm32mp1_atk_defconfig
```

4、编译 busybox

配置好 busybox 以后就可以编译了，我们可以指定编译结果的存放目录，我们肯定要将编译结果存放到前面创建的 rootfs 目录中，输入如下命令：

```
make
make install CONFIG_PREFIX=/home/zuozhongkai/linux/nfs/rootfs
```

COFIG_PREFIX 指定编译结果的存放目录，比如我存放到“/home/zuozhongkai/linux/nfs/rootfs”目录中，等待编译完成。编译完成以后如图 18.2.2.8 所示：

```
  /home/zuozhongkai/linux/rootfs//usr/sbin/ubirsvol -> ../../bin/busybox
  /home/zuozhongkai/linux/rootfs//usr/sbin/ubiupdatevol -> ../../bin/busybox
  /home/zuozhongkai/linux/rootfs//usr/sbin/udhcpd -> ../../bin/busybox


----------------------------------------------------
You will probably need to make your busybox binary
setuid root to ensure all configured applets will
work properly.
----------------------------------------------------

zuozhongkai@ubuntu:~/linux/busybox/busybox-1.32.0$
```

图 18.2.2.8 busybox 编译完成

编译完成以后会在 busybox 的所有工具和文件就会被安装到 rootfs 目录中，rootfs 目录内容如图 18.2.2.9 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs$ ls
bin  linuxrc  sbin  usr
zuozhongkai@ubuntu:~/linux/nfs/rootfs$
```

图 18.2.2.9 rootfs 目录

从图 18.2.2.9 可以看出，rootfs 目录下有 bin、sbin 和 usr 这三个目录，以及 linuxrc 这个文

件。前面说过 Linux 内核 init 进程最后会查找用户空间的 init 程序，找到以后就会运行这个用户空间的 init 程序，从而切换到用户态。如果 bootargs 设置 init=/linuxrc，那么 linuxrc 就是可以作为用户空间的 init 程序，所以用户态空间的 init 程序是 busybox 来生成的。

busybox 的工作就完成了，但是此时的根文件系统还不能使用，还需要一些其他的文件，我们继续来完善 rootfs。

18.2.3 向根文件系统添加 lib 库

1、向 rootfs 的“/lib”目录添加库文件

Linux 中的应用程序一般都是需要动态库的，当然你也可以编译成静态的，但是静态的可执行文件会很大。如果编译为动态的话就需要动态库，所以我们需要先根文件系统中添加动态库。在 rootfs 中创建一个名为“lib”的文件夹，命令如下：

```
mkdir lib
```

lib 文件夹创建好了，库文件从哪里来呢？lib 库文件从交叉编译器中获取，前面我们搭建交叉编译环境的时候将交叉编译器存放到了“/usr/local/arm/”目录中。交叉编译器里面有很多的库文件，这些库文件具体是做什么的我们作为初学者肯定不知道，既然我不知道那就简单粗暴的把所有的库文件都放到我们的根文件系统中。这样做出来的根文件系统肯定比较大，而且有很多我们可能用不到的库文件，但是我们现在是学习阶段，还做不了裁剪。

进入如下路径对应的目录：

```
/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf/arm-none-linux-gnueabihf/libc/lib
```

此目录下有很多的*so*(*是通配符)文件，这些就是库文件，将此目录下所有的*so*文件都拷贝到 rootfs/lib 目录中，拷贝命令如下：

```
cp *so* /home/zuozhongkai/linux/nfs/rootfs/lib/ -d
```

后面的“-d”表示拷贝符号链接，这里有个比较特殊的库文件：ld-linux-armhf.so.3，此库文件也是个符号链接，相当于 Windows 下的快捷方式。会链接到库 ld-2.30.so 上，输入命令“ls ld-linux-armhf.so.3 -l”查看此文件详细信息，如图 18.2.3.1 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$ ls ld-linux-armhf.so.3 -l
lrwxrwxrwx 1 zuozhongkai zuozhongkai 10 12月  17 10:40 ld-linux-armhf.so.3 -> ld-2.30.so
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$
```

图 18.2.3.1 文件 ld-linux-armhf.so.3

从图 18.2.3.1 可以看出，ld-linux-armhf.so.3 后面有个“->”，表示其是个软连接文件，链接到文件 ld-2.30.so，因为是一个“快捷方式”，因此大小只有 10B。但是，ld-linux-armhf.so.3 不能作为符号链接，否则的话在根文件系统中执行程序无法执行！所以我们需要 ld-linux-armhf.so.3 完成逆袭，由“快捷方式”变为“本尊”，方法很简单，那就是重新复制 ld-linux-armhf.so.3，只是不复制软链接即可，先将 rootfs/lib 中的 ld-linux-armhf.so.3 文件删除掉，命令如下：

```
rm ld-linux-armhf.so.3
```

然后重新进入到/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf/arm-none-linux-gnueabihf/libc/lib 目录中，重新拷贝 ld-linux-armhf.so.3，命令如下：

```
cp ld-linux-armhf.so.3 /home/zuozhongkai/linux/nfs/rootfs/lib/
```

拷贝完成以后再到 rootfs/lib 目录下查看 ld-linux-armhf.so.3 文件详细信息，如图 18.2.3.2 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$ ls ld-linux-armhf.so.3 -l
-rwxr-xr-x 1 zuozhongkai zuozhongkai 1279392 12月  17 10:48 ld-linux-armhf.so.3
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$
```

图 18.2.3.2 文件 ld-linux-armhf.so.3

从图 18.2.3.2 可以看出，此时 ld-linux-armhf.so.3 已经不是软连接了，而是实实在在的一个库文件，而且文件大小为 1279392B。

继续进入如下目录中：

```
/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf/arm-none-linux-gnueabihf/lib
```

此目录下也有很多的的*so*和.a 库文件，我们将其也拷贝到 rootfs/lib 目录中，命令如下：

```
cp *so* *.a /home/zuozhongkai/linux/nfs/rootfs/lib/ -d
```

rootfs/lib 目录的库文件就这些了，完成以后的 rootfs/lib 目录如图 18.2.3.3 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$ ls
ld-2.30.so              libdl-2.30.so         libm.so.6             librt-2.30.so
ld-linux-armhf.so.3     libdl.so.2            libnsl-2.30.so        librt.so.1
libanl-2.30.so          libgcc_s.so           libnsl.so.1           libSegFault.so
libanl.so.1             libgcc_s.so.1         libnss_compat-2.30.so libstdc++.a
libasan.a               libgfortran.a         libnss_compat.so.2    libstdc++fs.a
libasan.so              libgfortran.so        libnss_db-2.30.so     libstdc++.so
libasan.so.5            libgfortran.so.5      libnss_db.so.2        libstdc++.so.6
libasan.so.5.0.0        libgfortran.so.5.0.0  libnss_dns-2.30.so    libstdc++.so.6.0.28
libatomic.a             libgomp.a             libnss_dns.so.2       libstdc++.so.6.0.28-gdb.py
libatomic.so            libgomp.so            libnss_files-2.30.so  libsupc++.a
libatomic.so.1          libgomp.so.1          libnss_files.so.2     libthread_db-1.0.so
libatomic.so.1.2.0      libgomp.so.1.0.0      libnss_hesiod-2.30.so libthread_db.so.1
libBrokenLocale-2.30.so libitm.a              libnss_hesiod.so.2    libubsan.a
libBrokenLocale.so.1    libitm.so             libpcprofile.so       libubsan.so
libc-2.30.so            libitm.so.1           libpthread-2.30.so    libubsan.so.1
libcrypt-2.30.so        libitm.so.1.0.0       libpthread.so.0       libubsan.so.1.0.0
libcrypt.so.1           libm-2.30.so          libresolv-2.30.so     libutil-2.30.so
libc.so.6               libmemusage.so        libresolv.so.2        libutil.so.1
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib$
```

图 18.2.3.3 lib 目录

2、向 rootfs 的“usr/lib”目录添加库文件

在 rootfs 的 usr 目录下创建一个名为 lib 的目录，将如下目录中的库文件拷贝到 rootfs/usr/lib 目录下：

```
/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf/arm-none-linux-gnueabihf/libc/usr/lib
```

将此目录下的 so 和.a 库文件都拷贝到 rootfs/usr/lib 目录中，命令如下：

```
cp *so* *.a /home/zuozhongkai/linux/nfs/rootfs/usr/lib/ -d
```

完成以后的 rootfs/usr/lib 目录如图 18.2.3.4 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/usr/lib$ ls
libanl.a            libc.so               libitm.so.1.0.0   librt.a
libanl.so           libdl.a               libm.a            librt.so
libasan.so          libdl.so              libmcheck.a       libstdc++.so
libasan.so.5        libg.a                libm.so           libstdc++.so.6
libasan.so.5.0.0    libgcc_s.so           libnss_compat.so  libstdc++.so.6.0.28
libatomic.so        libgcc_s.so.1         libnss_db.so      libstdc++.so.6.0.28-gdb.py
libatomic.so.1      libgfortran.so        libnss_dns.so     libthread_db.so
libatomic.so.1.2.0  libgfortran.so.5      libnss_files.so   libubsan.so
libBrokenLocale.a   libgfortran.so.5.0.0  libnss_hesiod.so  libubsan.so.1
libBrokenLocale.so  libgomp.so            libpthread.a      libubsan.so.1.0.0
libc.a              libgomp.so.1          libpthread.so     libutil.a
libc_nonshared.a    libgomp.so.1.0.0      libresolv.a       libutil.so
libcrypt.a          libitm.so             libresolv.so
libcrypt.so         libitm.so.1           librpcsvc.a
zuozhongkai@ubuntu:~/linux/nfs/rootfs/usr/lib$
```

图 18.2.3.4 rootfs/usr/lib 目录

至此，根文件系统的库文件就全部添加好了，可以使用“du”命令来查看一下 rootfs/lib 和 rootfs/usr/lib 这两个目录的大小，命令如下：

```
cd rootfs                  //进入根文件系统目录
du ./lib ./usr/lib/ -sh    //查看 lib 和 usr/lib 这两个目录的大小
```

结果如图 18.2.3.5 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs$ du ./lib/ ./usr/lib/ -sh
158M    ./lib/
89M     ./usr/lib/
zuozhongkai@ubuntu:~/linux/nfs/rootfs$
```

图 18.2.3.5 lib 和 usr/lib 目录大小

可以看出 lib 和 usr/lib 这两个文件的大小分别为 158MB 和 89MB，加起来就是 158+89=247MB，还是挺大的，但是正点原子 STM23MP157 开发板板载 8GB 的 EMMC，大家不用担心存储不够用的问题。。

18.2.4 创建其他文件夹

在根文件系统中创建其他文件夹，如 dev、proc、mnt、sys、tmp、etc 和 root 等，创建完成以后如图 18.2.4.1 所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs$ ls
bin  dev  etc  lib  linuxrc  mnt  proc  root  sbin  sys  tmp  usr
zuozhongkai@ubuntu:~/linux/nfs/rootfs$
```

图 18.2.4.1 创建好其他文件夹以后的 rootfs

目前来看，这个根文件系统好像已经准备好了，究竟有没有准备好，直接测一下就知道了！

18.3 根文件系统初步测试

18.3.1 修改 Ubuntu 的 nfs 版本配置

我们测试根文件系统的时候不是直接烧写到 EMMC 里面，这样测试效率太低了，Ubuntu 的 rootfs 目录已经保存了根文件系统，我们只需要在开发板上通过 nfs 挂载 Ubuntu 下的 rootfs 目录即可。也就是说，根文件系统一直在 Ubuntu 下，开发板通过网络在使用这个根文件系统，这样方便我们开发调试。

但是，Ubuntu18 的 nfs 默认只支持 3 和 4 版本的 nfs，uboot 默认使用的是版本 2，所以直接需要修改 Ubuntu18 的 nfs 配置，否则 nfs 根文件系统会报如图 18.3.1.1 所示错误，导致无法挂载。

```
[  104.177011] VFS: Unable to mount root fs via NFS, trying floppy.
[  104.182384] VFS: Cannot open root device "nfs" or unknown-block(2,0): error -6
```

图 18.3.1.1 nfs 挂载错误

解决方法很简单，打开 Ubuntu 下的/etc/default/nfs-kernel-server 文件，然后在最后面添加下面这一行：

```
RPCNFSDOPTS="--nfs-version 2,3,4 --debug --syslog"
```

如图 18.3.1.2 所示：

```
18 # Options for rpc.svcgssd.
19 RPCSVCGSSDOPTS=""
20
21 RPCNFSDOPTS="--nfs-version 2,3,4 --debug --syslog"
~
~
```

图 18.3.1.2 修改 nfs 配置

添加完成以后保存退出，输入如下命令重启 NFS 服务即可：

```
sudo /etc/init.d/nfs-kernel-server restart
```

18.3.2 bootargs 环境变量设置

接下来我们要设置 uboot 下的 bootargs 环境变量，主要是设置里面的“root”值，我们将 root 的值改为 NFS 挂载即可。在 Linux 内核源码里面有相应的文档讲解如何设置，文档为 Documentation/filesystems/nfs/ nfsroot.txt，格式如下：

```
root=/dev/nfs nfsroot=[<server-ip>:]<root-dir>[,<nfs-options>] ip=<client-ip>:<server-ip>:<gw-ip>:<netmask>:<hostname>:<device>:<autoconf>:<dns0-ip>:<dns1-ip>
```

<server-ip>：服务器 IP 地址，也就是存放根文件系统主机的 IP 地址，那就是 Ubuntu 的 IP 地址，比如我的 Ubuntu 主机 IP 地址为 192.168.1.249。

<root-dir>：根文件系统的存放路径，比如我的就是/home/zuozhongkai/linux/nfs/rootfs。

<nfs-options>：NFS 的其他可选选项，一般不设置。

<client-ip>：客户端 IP 地址，也就是我们开发板的 IP 地址，Linux 内核启动以后就会使用此 IP 地址来配置开发板。此地址一定要和 Ubuntu 主机在同一个网段内，并且没有被其他的设备使用，在 Ubuntu 中使用 ping 命令 ping 一下就知道要设置的 IP 地址有没有被使用，如果不能 ping 通就说明没有被使用，那么就可以设置为开发板的 IP 地址，比如我就可以设置为 192.168.1.250。

<server-ip>：服务器 IP 地址，前面已经说了。

<gw-ip>：网关地址，我的就是 192.168.1.1。

<netmask>：子网掩码，我的就是 255.255.255.0。

<hostname>：客户机的名字，一般不设置，此值可以空着。

<device>：设备名，也就是网卡名，一般是 eth0，eth1….，正点原子 STM32MP157 开发板只有一个网口，名字为 eth0。

<autoconf>：自动配置，一般不使用，所以设置为 off。

<dns0-ip>：DNS0 服务器 IP 地址，不使用。

<dns1-ip>：DNS1 服务器 IP 地址，不使用。

根据上面的格式 bootargs 环境变量的 root 值如下：

```
root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw ip=192.168.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off
```

“proto=tcp”表示使用 TCP 协议，“rw”表示 nfs 挂载的根文件系统为可读可写。启动开发板，进入 uboot 命令行模式，然后重新设置 bootargs 环境变量，命令如下：

```
setenv bootargs 'console=ttySTM0,115200 root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw ip=192.168.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off'    //设置 bootargs
saveenv                 //保存环境变量
```

设置好以后使用“boot”命令启动 Linux 内核，结果如图 18.3.2.1 所示：

```
[    7.771962] stm32-dwmac 5800a000.ethernet eth0: Link is Up - 1Gbps/Full - flow control rx/tx
[    7.818791] IP-Config: Complete:
[    7.820572]      device=eth0, hwaddr=00:04:9f:04:d2:35, ipaddr=192.168.1.250, mask=255.255.255.0, gw
=192.168.1.1
[    7.830794]      host=192.168.1.250, domain=, nis-domain=(none)
[    7.836677]      bootserver=192.168.1.249, rootserver=192.168.1.249, rootpath=
[    7.845134] ALSA device list:
[    7.846867]   No soundcards found.
[    7.868203] VFS: Mounted root (nfs filesystem) on device 0:16.
[    7.873863] devtmpfs: mounted
[    7.877409] Freeing unused kernel memory: 1024K
[    7.919082] Run /sbin/init as init process
can't run '/etc/init.d/rcS': No such file or directory

Please press Enter to activate this console.
/ #
/ #
```

图 18.3.2.1 进入根文件系统

大家要注意，由于是通过网络挂载根文件系统，因此 Linux 系统在启动的时候需要初始化网络，因此会卡一下，大概在初始化完 EMMC 以后会卡几秒钟，这是正常现象。

我们可以输入“ls”命令测试一下，结果如图 18.3.2.2 所示：

```
/ # ls
bin      lib      mnt      root     sys      usr
dev      linuxrc  proc     sbin     tmp
/ #
```

图 18.3.2.2 ls 命令测试

可以看出 ls 命令工作正常！那么是不是说明我们的 rootfs 就制作成功了呢？大家注意，在进入根文件系统的时候会有下面这一行错误提示：

```
can't run '/etc/init.d/rcS': No such file or directory
```

提示很简单，说是无法运行“/etc/init.d/rcS”这个文件，因为这个文件不存在。如图 18.3.3 所示

```
[    7.919082] Run /sbin/init as init process
can't run '/etc/init.d/rcS': No such file or directory

Please press Enter to activate this console.
```

图 39.3.3 “/etc/init.d/rcS”不存在

看来我们的 rootfs 还是缺文件啊，没什么说的，一步一步的完善吧。

18.3.3 无法进入根文件系统

如果大家使用 nfs 挂载根文件系统的时候发现并没有进入根文件系统，那么可能的原因有一下几点。

1、网络硬件原因

可能是开发板网络硬件有问题，首先检查一下网线是否接好，网络硬件是否有损坏。最简单的检测方法就是烧写正点原子出厂系统，然后在出厂系统里面测试网络是否能正常工作，如果可以正常工作的话就说明不是硬件问题。

2、bootargs 环境变量设置错误

bootargs 环境变量设置错误是最常见的，典型的就是单词写错！少个字母或者多个字母，直接进入 uboot 里面，输入：

```
print bootargs
```

输入上述命令以后就会打印出 bootargs 环境变量的值，如图 18.3.3.1 所示：

```
STM32MP> print bootargs
bootargs=root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw ip=192.168
.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off
STM32MP>
```

图 18.3.3.1 bootargs 环境变量值

3、bootargs 环境变量没有传递给内核

这个问题不常见，但却是最容易忽略的！bootargs 环境变量的值会传递给内核作为命令行(command line)参数，Linux 内核启动的时候会打印出命令行参数，如图 18.3.3.2 所示：

```
[    0.000000] percpu: Embedded 20 pages/cpu s50124 r8192 d23604 u81920
[    0.000000] Built 1 zonelists, mobility grouping on.  Total pages: 227647
[    0.000000] Kernel command line: root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs/rootfs,pr
oto=tcp rw ip=192.168.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off
[    0.000000] Dentry cache hash table entries: 131072 (order: 7, 524288 bytes, linear)
[    0.000000] Inode-cache hash table entries: 65536 (order: 6, 262144 bytes, linear)
```

图 18.3.3.2 命令行参数

从图 18.3.3.2 可以看出，有“Kernel command line:”这一行，这一行就是命令行参数，可以看到命令行参数和 bootargs 环境变量一模一样，如果不一致的话就说明 bootargs 没有传递给内核。

没有传递进来的原因无非两点：

①、bootargs 环境变量设置错误，比如设置的时候将 bootargs 打错，写成了“botargs”、“bottargs”等等。导致无法正确识别为“bootargs”。

②、bootargs 环境变量被覆写，也就是说 bootargs 环境变量设置正确，但是就是传递不进来，或者传递给命令行参数是错误的，这是因为 uboot 在启动 linux 内核的时候将 bootargs 覆写了。比如 ST 官方 uboot 里面的 bootcmd 是一个脚本文件，此脚本文件比较复杂，会经过一系列的计算得到具体的 bootcmd 命令以及 bootargs 参数，在这个过程中 bootargs 环境变量就会被覆写掉，解决方法就是在 uboot 命令行先输入“eraseenv”命令来擦除默认环境变量内容，然后再手动设置 bootcmd 和 bootargs 这两个环境变量的值，其他的网络 IP 地址、MAC 地址能也需要重新设置。

18.4 完善根文件系统

18.4.1 创建/etc/init.d/rcS 文件

rcS 是个 shell 脚本，Linux 内核启动以后需要启动一些服务，而 rcS 就是规定启动哪些文件的脚本文件。在 rootfs 中创建/etc/init.d/rcS 文件，然后在 rcS 中输入如下所示内容：

示例代码 18.4.1.1 /etc/init.d/rcS 文件

```
1  #!/bin/sh
2
3  PATH=/sbin:/bin:/usr/sbin:/usr/bin:$PATH
4  LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/lib:/usr/lib
5  export PATH LD_LIBRARY_PATH
6
7  mount -a
8  mkdir /dev/pts
9  mount -t devpts devpts /dev/pts
10
11 echo /sbin/mdev > /proc/sys/kernel/hotplug
```

```
12 mdev -s
```

第 1 行，表示这是一个 shell 脚本。

第 3 行，PATH 环境变量保存着可执行文件可能存在的目录，这样我们在执行一些命令或者可执行文件的时候就不会提示找不到文件这样的错误。

第 4 行，LD_LIBRARY_PATH 环境变量保存着库文件所在的目录。

第 5 行，使用 export 来导出上面这些环境变量，相当于声明一些“全局变量”。

第 7 行，使用 mount 命令来挂载所有的文件系统，这些文件系统由文件/etc/fstab 来指定，所以我们一会还要创建/etc/fstab 文件。

第 8 和 9 行，创建目录/dev/pts，然后将 devpts 挂载到/dev/pts 目录中。

第 11 和 12 行，使用 mdev 来管理热插拔设备，通过这两行，Linux 内核就可以在/dev 目录下自动创建设备节点。关于 mdev 的详细内容可以参考 busybox 中的 docs/mdev.txt 文档。

示例代码 18.4.1.1 中的 rcS 文件内容是最精简的，大家如果去看 Ubuntu 或者其他大型 Linux 操作系统中的 rcS 文件，就会发现其非常复杂。因为我们是初次学习，所以不用搞这么复杂的，而且这么复杂的 rcS 文件也是借助其他工具创建的，比如 buildroot 等。

创建好文件/etc/init.d/rcS 以后一定要给其可执行权限！

使用如下命令给予/ec/init.d/rcS 可执行权限：

```
chmod 777 rcS
```

设置好以后就重新启动 Linux 内核，启动以后如图 18.4.1.1 所示：

```
mount: can't read '/etc/fstab': No such file or directory
/etc/init.d/rcS: line 11: can't create /proc/sys/kernel/hotplug: nonexistent directory
mdev: /sys/dev: No such file or directory

Please press Enter to activate this console.
/ #
```

图 18.4.1.1 Linux 启动过程

从图 18.4.1.1 可以看到，提示找不到/etc/fstab 文件，还有一些其他的错误，我们先把/etc/fstab 这个错误解决了。说不定把这个问题解决以后其他的错误也就解决了。前面我们说了“mount -a”挂载所有根文件系统的时候需要读取/etc/fstab，因为/etc/fstab 里面定义了该挂载哪些文件，好了，接下来就是创建/etc/fstab 文件。

18.4.2 创建/etc/fstab 文件

在 rootfs 中创建/etc/fstab 文件，fstab 在 Linux 开机以后自动配置哪些需要自动挂载的分区，格式如下：

```
<file system>  <mount point>  <type>  <options>  <dump>  <pass>
```

<file system>：要挂载的特殊的设备，也可以是块设备，比如/dev/sda 等等。

<mount point>：挂载点。

<type>：文件系统类型，比如 ext2、ext3、proc、romfs、tmpfs 等等。

<options>：挂载选项，在 Ubuntu 中输入“man mount”命令可以查看具体的选项。一般使用 defaults，也就是默认选项，defaults 包含了 rw、suid、 dev、 exec、 auto、 nouser 和 async。

<dump>：为 1 的话表示允许备份，为 0 不备份，一般不备份，因此设置为 0。

<pass>：磁盘检查设置，为 0 表示不检查。根目录‘/’设置为 1，其他的都不能设置为 1，其他的分区从 2 开始。一般不在 fstab 中挂载根目录，因此这里一般设置为 0。

按照上述格式，在 fstab 文件中输入如下内容：

示例代码 18.4.2.1 /etc/fstab 文件

```
1 #<file system> <mount point>    <type>    <options>   <dump>  <pass>
```

```
2 proc            /proc           proc       defaults    0       0
3 tmpfs           /tmp            tmpfs      defaults    0       0
4 sysfs           /sys            sysfs      defaults    0       0
```

fstab 文件创建完成以后重新启动 Linux，结果如图 18.4.2.1 所示：

```
[    7.989052] Run /sbin/init as init process
/etc/init.d/rcS: line 11: can't create /proc/sys/kernel/hotplug: nonexistent directory

Please press Enter to activate this console.
/ #
```

图 18.4.2.1 Linux 启动过程

从图 18.4.2.1 可以看出，在运行/etc/init.d/rcS 脚本第 11 行出现问题，提示不能创建“/proc/sys/kernel/hotplug”，这个是 Linux 内核配置问题，最后在讲解怎么处理。我们接下来还需要创建一个文件/etc/inittab。

18.4.3 创建/etc/inittab 文件

inittab 的详细内容可以参考 busybox 下的文件 examples/inittab。init 程序会读取/etc/inittab 这个文件，inittab 由若干条指令组成。每条指令的结构都是一样的，由以“:”分隔的 4 个段组成，格式如下：

```
<id>:<runlevels>:<action>:<process>
```

<id>：每个指令的标识符，不能重复。但是对于 busybox 的 init 来说，<id>有着特殊意义。对于 busybox 而言<id>用来指定启动进程的控制 tty，一般我们将串口或者 LCD 屏幕设置为控制 tty。

<runlevels>：对 busybox 来说此项完全没用，所以空着。

<action>：动作，用于指定<process>可能用到的动作。busybox 支持的动作如表 18.4.3.1 所示：

动作	描述
sysinit	在系统初始化的时候 process 才会执行一次。
respawn	当 process 终止以后马上启动一个新的。
askfirst	和 respawn 类似，在运行 process 之前在控制台上显示“Please press Enter to activate this console.”。只要用户按下“Enter”键以后才会执行 process。
wait	告诉 init，要等待相应的进程执行完以后才能继续执行。
once	仅执行一次，而且不会等待 process 执行完成。
restart	当 init 重启的时候才会执行 procee。
ctrlaltdel	当按下 ctrl+alt+del 组合键才会执行 process。
shutdown	关机的时候执行 process。

表 18.4.3.1 动作

<process>：具体的动作，比如程序、脚本或命令等。

参考 busybox 的 examples/inittab 文件，我们也创建一个/etc/inittab，在里面输入如下内容：

示例代码 18.4.3.1 /etc/inittab 文件

```
1 #etc/inittab
2 ::sysinit:/etc/init.d/rcS
3 console::askfirst:-/bin/sh
4 ::restart:/sbin/init
5 ::ctrlaltdel:/sbin/reboot
6 ::shutdown:/bin/umount -a -r
```

```
7 ::shutdown:/sbin/swapoff -a
```

第 2 行，系统启动以后运行/etc/init.d/rcS 这个脚本文件。

第 3 行，将 console 作为控制台终端，也就是 ttySTM0。

第 4 行，重启的话运行/sbin/init。

第 5 行，按下 ctrl+alt+del 组合键的话就运行/sbin/reboot，看来 ctrl+alt+del 组合键用于重启系统。

第 6 行，关机的时候执行/bin/umount，也就是卸载各个文件系统。

第 7 行，关机的时候执行/sbin/swapoff，也就是关闭交换分区。

/etc/inittab 文件创建好以后就可以重启开发板即可，至此！根文件系统要创建的文件就已经全部完成了。

18.4.4 使能内核 uevet helper

根文件系统要创建的文件已经全部完成了，但还是会存在如图 18.4.4.1 所示问题：

```
[    7.909115] Run /sbin/init as init process
/etc/init.d/rcS: line 11: can't create /proc/sys/kernel/hotplug: nonexistent directory

Please press Enter to activate this console.
/ #
```

图 18.4.4.1 hotplug 错误

前面已经说了，这是 Linux 内核配置问题，只要配置一下内核就行了，路径如下：

```
Location:
        -> Device Drivers
          -> Generic Driver Options
                ->Support for uevent helper          //选中
```

配置如图 18.4.4.2 所示：

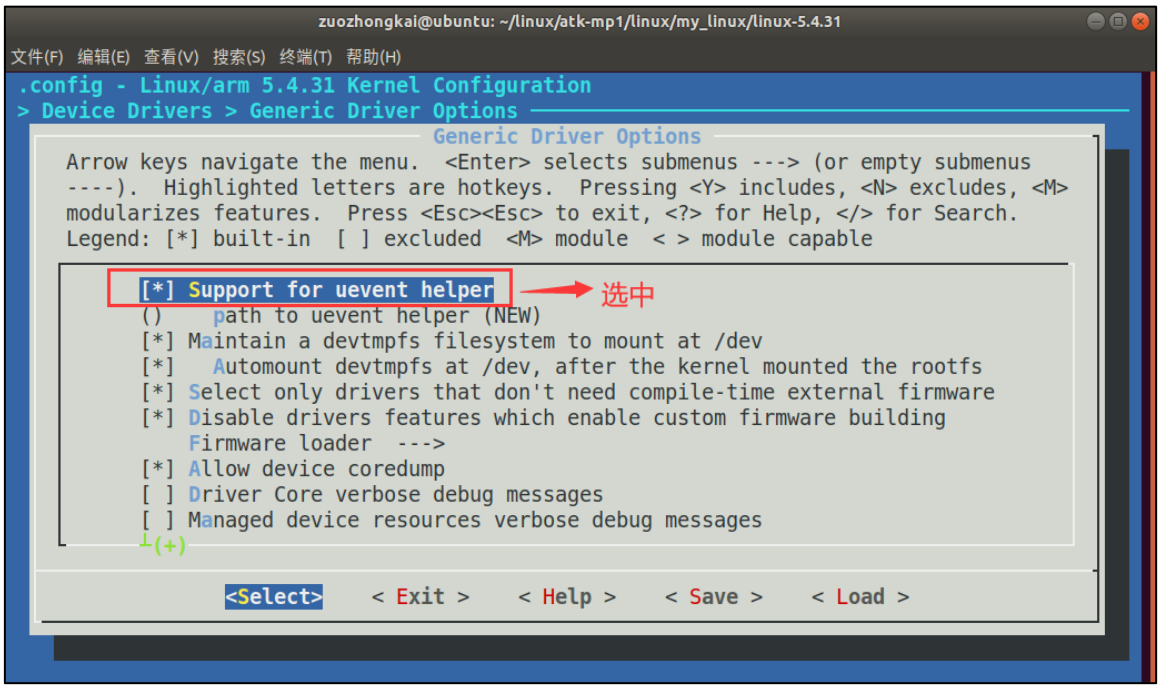


图 18.4.4.2 内核配置

选中图 18.4.4.2 中的“Support for uevent helper”选项，选中以后就会在.config 中存在：

```
CONFIG_UEVENT_HELPER=y
```

配置好以后重新编译内核，然后使用新的内核启动，如图 18.4.4.3 所示：

```
[    7.948734] Freeing unused kernel memory: 1024K
[    7.988791] Run /sbin/init as init process

Please press Enter to activate this console.
/ #
/ #
```

图 18.4.4.3 根文件系统运行结果

从图 18.4.4.3 可以看出，没有任何错误提示，说明根文件系统工作已经正常了。接下来就要对根文件系统进行其他的测试，比如我们自己编写的软件运行是否正常、是否支持软件开机自启动、中文支持是否正常以及能不能链接等。

18.5 根文件系统其他功能测试

18.5.1 软件运行测试

我们使用 Linux 系统的目的就是运行我们自己的软件，我们编译的应用软件一般都使用动态库，使用动态库的话应用软件体积就很小，但是得提供库文件，库文件我们已经添加到了根文件系统中。我们编写一个小小的测试软件来测试一下库文件是否工作正常，在根文件系统下创建一个名为“drivers”的文件夹，以后我们学习 Linux 驱动的时候就把所有的实验文件放到这个文件夹里面。

在 Ubuntu 下使用 vim 编辑器新建一个 hello.c 文件，在 hello.c 里面输入如下内容：

示例代码 18.5.1.1 hello.c 文件

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      while(1) {
6          printf("hello world!\r\n");
7          sleep(2);
8      }
9      return 0;
10 }
```

hello.c 内容很简单，就是循环输出“hello world”，sleep 相当于 Linux 的延时函数，单位为秒，所以 sleep(2)就是延时 2 秒。编写好以后就是编译，因为我们是要在 ARM 芯片上运行的，所以要用交叉编译器去编译，也就是使用 arm-linux-gnueabihf-gcc 编译，命令如下：

```
arm-none-linux-gnueabihf-gcc hello.c -o hello
```

使用 arm-none-linux-gnueabihf-gcc 将 hello.c 编译为 hello 可执行文件。这个 hello 可执行文件究竟是不是 ARM 使用的呢？使用“file”命令查看文件类型以及编码格式：

```
file hello      //查看 hello 的文件类型以及编码格式
```

结果如图 18.5.1.1 所示：

```
zuozhongkai@ubuntu:~/linux$ file hello
hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpre
ter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, with debug_info, not stripped
zuozhongkai@ubuntu:~/linux$
```

图 18.5.1.1 查看 hello 编码格式

从图 18.5.1.1 可以看出，输入“file hello”输入了如下所示信息：

```
hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked……
```

hello 是个 32 位的 LSB 可执行文件，ARM 架构的，并且是动态链接的。所以我们编译出来的 hello 文件没有问题。将其拷贝到开发板的 rootfs/drivers 目录下，在开发板中输入如下命令来执行这个可执行文件：

```
cd /drivers     //进入 drivers 目录
./hello         //执行 hello
```

结果如图 18.5.1.2 所示：

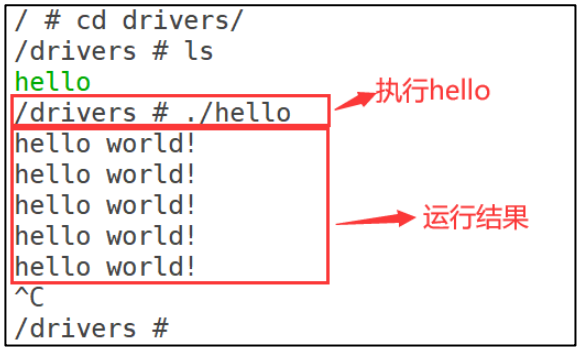


图 18.5.1.2 hello 运行结果

可以看出，hello 这个软件运行正常，说明我们的根文件系统中的共享库是没问题的，要想终止 hello 的运行，按下“ctrl+c”组合键即可。此时大家应该能感觉到，hello 执行的时候终端是没法用的，除非使用“ctrl+c”来关闭 hello，那么有没有办法既能让 hello 正常运行，而且终端能够正常使用？那肯定是有的，让 hello 进入后台运行就行了，让一个软件进入后台的方法很简单，运行软件的时候加上“&”即可，比如“./hello &”就是让 hello 在后台运行。在后台运行的软件可以使用“kill -9 pid(进程 ID)”命令来关闭掉，首先使用“ps”命令查看要关闭的软件 PID 是多少，ps 命令用于查看所有当前正在运行的进程，并且会给出进程的 PID。输入“ps”命令，结果如图 18.5.1.3 所示：

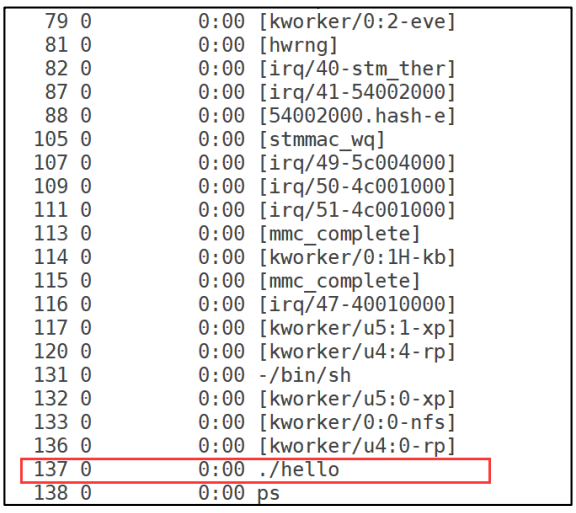


图 18.5.1.3 ps 命令结果

从图 18.5.1.3 可以看出 hello 对应的 PID 为 137，因此我们使用如下命令关闭在后台运行的 hello 软件：

```
kill -9 137
```

因为 hello 在不断的输出“hello world”所以我们的输入看起来会被打断，其实是没有的，因为我们是输入，而 hello 是输出。在数据流上是没有打断的，只是显示在 SecureCRT 上就好像被打断了，所以只管输入“kill -9 137”即可。hello 被 kill 以后会有提示，如图 18.5.1.4 所示：

```
/drivers #
[1]+  Killed                    ./hello
/drivers #
```

图 18.5.1.4 提示 hello 被 kill 掉。

再去用 ps 命令查看一下当前的进程，发现没有 hello 了。这个就是 Linux 下的软件后台运行以及如何关闭软件的方法，重点就是 3 个操作：软件后面加“&”、使用 ps 查看要关闭的软件 PID、使用“kill -9 pid”来关闭指定的软件。

18.5.2 中文字符测试

在 ubuntu 中向在 rootfs 目录新建一个名为“中文测试”的文件夹，然后在 MobaXterm 下查看中文名能不能显示正确。输入“ls”命令，结果如图 18.5.2.1 所示：

```
/ # ls
bin         etc         mnt         sbin        usr
dev         lib         proc        sys         中文测试
drivers     linuxrc     root        tmp
/ #
```

图 18.5.2.1 中文文件夹测试

可以看出“中文测试”这个文件夹显示正常，接着“touch”命令在“中文测试”文件夹中新建一个名为“测试文档.txt”的文件，并且使用 vim 编辑器在其中输入“这是一个中文测试文件”，借此来测试一下中文文件名和中文内容显示是否正常。在 MobaXterm 中使用“cat”命令来查看“测试文档.txt”中的内容，结果如图 18.5.2.2 所示：

```
/ # cd 中文测试
/中文测试 # ls
测试文档.txt
/中文测试 # cat 测试文档.txt
这是一个中文测试文件
/中文测试 #
```

使用cat命令查看"测试文档.txt"中的内容

图 18.5.2.2 中文文档内容显示

从图 18.5.2.2 可以看出，“测试文档.txt”的中文内容显示正确，而且中文路径也完全正常，说明我们的根文件系统已经完美支持中文了！

18.5.3 开机自启动测试

在 18.5.1 小节测试 hello 软件的时候都是等 Linux 启动进入根文件系统以后手动输入命令“./hello”来完成的。我们一般做好产品以后都是需要开机自动启动相应的软件，本节我们就以 hello 这个软件为例，讲解一下如何实现开机自启动。前面我们说过了，进入根文件系统的时候会运行/etc/init.d/rcS 这个 shell 脚本，因此我们可以在这个脚本里面添加自启动相关内容。添加完成以后的/etc/init.d/rcS 文件内容如下：

示例代码 18.5.3.1 rcS 文件代码

```
1  #!/bin/sh
2  PATH=/sbin:/bin:/usr/sbin:/usr/bin
3  LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/lib:/usr/lib
4  runlevel=S
```

```
5  umask 022
6  export PATH LD_LIBRARY_PATH runlevel
7
8  mount -a
9  mkdir /dev/pts
10 mount -t devpts devpts /dev/pts
11
12 echo /sbin/mdev > /proc/sys/kernel/hotplug
13 mdev -s
14
15 #开机自启动
16 cd /drivers
17 ./hello &
18 cd /
```

第 16 行，进入 drivers 目录，因为要启动的软件存放在 drivers 目录下。

第 17 行，以后台方式执行 hello 这个软件。

第 18 行，退出 drivers 目录，进入到根目录下。

自启动代码添加完成以后就可以重启开发板，看看 hello 这个软件会不会自动运行。结果如图 18.5.3.1 所示：

```
[    7.978890] Run /sbin/init as init process

Please press Enter to activate this console. hello world!
hello world!
hello world!
hello world!
```

hello软件开始运行

图 18.5.3.1 hello 开机自启动

从图 18.5.3.1 可以看出，hello 开机自动运行了，说明开机自启动成功。

18.5.4 外网连接测试

这里说的外网不是外国哪些 404 网站的连接测试，而是百度、淘宝等这些网站的测试。也就是说看看我们的开发板能不能上网，能不能和我们的局域网外的这些网站进行通信。测试方法很简单，就是通过 ping 命令来 ping 一下百度的官网：www.baidu.com。输入如下命令：

```
ping www.baidu.com
```

结果如图 18.5.4.1 所示：

```
/ # ping www.baidu.com
ping: bad address 'www.baidu.com'
/ #
```

图 18.5.4.1 ping 测试结果

可以看出，测试失败，提示 www.baidu.com 是个“bad address”，也就是地址不对，显然我们的地址是正确的。之所以出现这个错误提示是因为 www.baidu.com 的地址解析失败了，并没有解析出其对应的 IP 地址。我们需要配置域名解析服务器的 IP 地址，一般域名解析地址可以设置为所处网络的网关地址，比如 192.168.1.1。也可以设置为 114.114.1144.114，这个是运营商的域名解析服务器地址。

在 rootfs 中新建文件/etc/resolv.conf，然后在里面输入如下内容：

示例代码 18.5.4.1 resolv.conf 文件内容

```
1 nameserver 114.114.114.114
2 nameserver 192.168.1.1
```

设置很简单，nameserver 表示这是个域名服务器，设置了两个域名服务器地址：114.114.114.114 和 192.168.1.1，大家也可以改为其他的域名服务器试试。如果使用“udhcpc”命令自动获取 IP 地址，“udhcpc”命令会修改 nameserver 的值，一般是将其设置为对应的网关地址。修改好以后保存退出，重启开发板！重启以后重新 ping 一下百度官网，结果如图 18.5.4.2 所示：

```
/ # ping www.baidu.com
PING www.baidu.com (14.215.177.39): 56 data bytes
64 bytes from 14.215.177.39: seq=0 ttl=56 time=6.872 ms
64 bytes from 14.215.177.39: seq=1 ttl=56 time=6.350 ms
64 bytes from 14.215.177.39: seq=2 ttl=56 time=6.722 ms
^C
--- www.baidu.com ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 6.350/6.648/6.872 ms
/ #
```

图 18.5.4.2 ping 百度官网结果

可以看出 ping 百度官网成功了！域名也成功的解析了，至此！我们的根文件系统就彻底的制作完成，这个根文件系统最好打包保存一下，防止以后做实验不小心破坏了根文件系统而功亏一篑，又得从头制作根文件系统。uboot、Linux kernel、rootfs 这三个共同构成了一个完整的 Linux 系统，现在的系统至少是一个可以正常运行的系统，后面我们就可以在这个系统上完成 Linux 驱动开发的学习。

18.6 烧写根文件系统到 EMMC 中

一个最小的根文件系统已经制作好了，而且通过 nfs 测试正常，接下来就学习一下如何将其烧写到开发板 EMMC 中。根文件系统烧写进去以后，一个完成的嵌入式 Linux 开发环境就准备好了，我们以后实际产品开发也是用的这套流程。

18.6.1 根文件系统打包

根文件系统打包和 17.4.1 小节中对 uImage 和 stm32mp157d-atk.dtb 打包方法一样，也是制作 ext4 格式的根文件系统包，这里我们需要对/home/zuozhongkai/linux/nfs/rootfs 目录进行打包，这里我们再讲解一遍。

1、新建 ext4 格式磁盘

首先新建一个 ext4 格式的磁盘，然后挂载这个 ext4 格式的磁盘，将/home/zuozhongkai/linux/nfs/rootfs 这个目录下的文件拷贝到这个 ext4 磁盘即可。

首选创建一个名为 rootfs 文件夹，注意不要和/home/zuozhongkai/linux/nfs/rootfs 这个目录重复了！比如我就在/home/zuozhongkai/linux/目录下创建一个名为“rootfs”目录，然后输入如下命令创建 ext4 磁盘：

示例代码 18.6.1.1 ext4 磁盘创建命令

```
1 cd rootfs
2 dd if=/dev/zero of=rootfs.ext4 bs=1M count=1024
3 mkfs.ext4 -L rootfs rootfs.ext4
```

第 1 行，进入 rootfs 目录。

第 2 行，使用 dd 命令创建一个名为 rootfs.ext4 的磁盘，由于根文件系统比较大，因此 count 设置为 1024，也就是根文件系统总空间为 1GB，这个是可以调整的，大家要根据自己的实际根文件系统大小调整 count 的值，只要空间不超过开发板所使用的 8GB EMMC 即可。最好不要设置太大，否则烧写会很慢！

第 3 行，使用 mkfs.ext4 将 rootfs.ext4 磁盘格式化为 ext4 格式。

完成以后就会生成名为“rootfs.ext4”的磁盘，如图 18.6.1.2 所示：

```
zuozhongkai@ubuntu:~/linux/rootfs$ ls
rootfs.ext4
zuozhongkai@ubuntu:~/linux/rootfs$
```

图 18.6.1.2 rootfs.ext4 磁盘

2、将系统镜像拷贝到 ext4 磁盘中

首先创建一个目录用来挂载前面制作制作出来的 rootfs.ext4，比如我这里创建目录/mnt/rootfs，命令如下：

```
sudo mkdir /mnt/rootfs
```

接下来使用 mount 命令将 rootfs.ext4 挂载到/mnt/rootfs 目录下，命令如下：

```
cd /home/zuozhongkai/linux/atk-mp1/linux/rootfs
sudo mount rootfs.ext4 /mnt/rootfs/
```

挂载成功以后就将/home/zuozhongkai/linux/nfs/rootfs 目录下的所有根文件系统文件拷贝到/mnt/bootfs 目录下，命令如下：

```
cd /home/zuozhongkai/linux/nfs/rootfs/
sudo cp * /mnt/rootfs/ -drf
```

拷贝完成以后使用 umount 卸载/mnt/rootfs 即可，命令如下：

```
sudo umount /mnt/rootfs
```

至此，根文件系统就已经打包到了图 18.6.1.2 中的 rootfs.ext4 中，稍后使用 STM32CubeProgrammer 软件将其烧写到 EMMC 里面。烧写之前最好在 Windows 下打开 rootfs.ext4 看一下，看看是否已经将根文件系统打包进去，如图 18.6.1.3 所示：

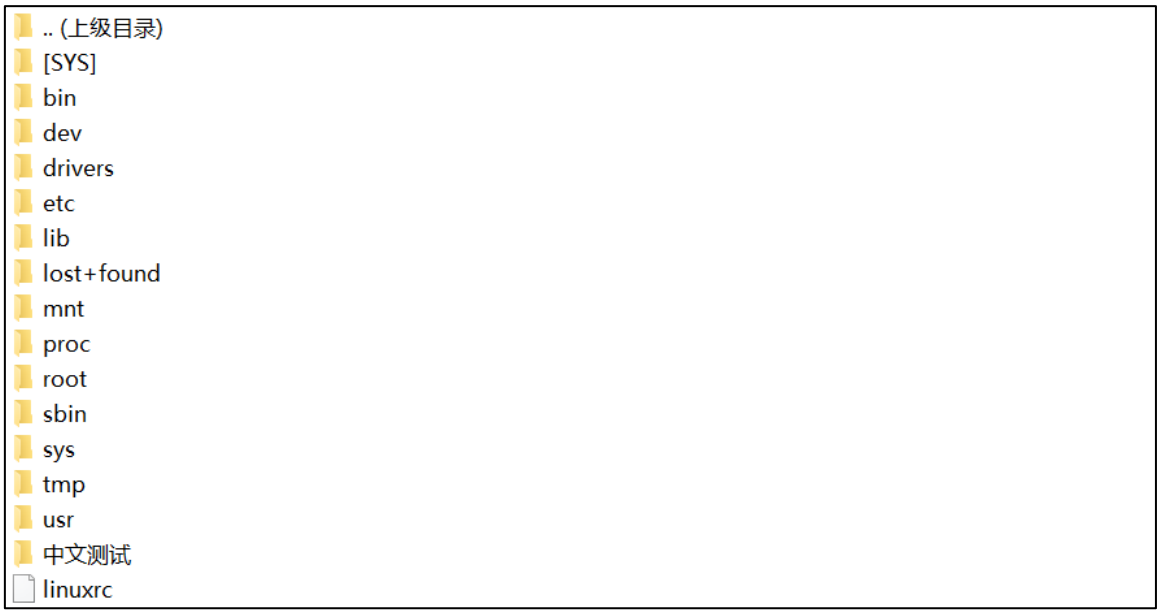


图 18.6.1.3 bootfs.ext4

18.6.2 烧写到 EMMC

接下来就是将上一小节打包好的 ext4 格式的根文件系统 rootfs.ext4 烧写到开发板的 EMMC 里面，使用 STM32CubeProgrammer 软件完成此操作。将 rootfs.ext4 拷贝到以前创建的 images 目录下，如图 18.6.2.1 所示：

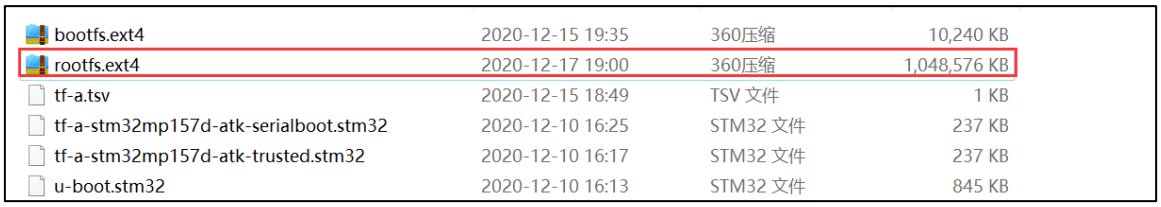


图 18.6.2.1 images 目录

修改 flashlayout 文件 tf-a.tsv，在后面加入 rootfs.ext4 的烧写脚本，如图 18.6.2.2 所示：

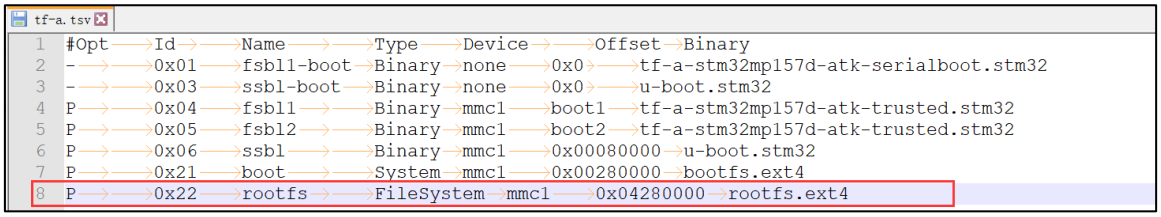


图 18.6.2.2 修改后的 flashlayout

图 18.6.2.2 中第 8 行就是根文件系统 rootfs.ext4 的烧写脚本，设置好以后就可以使用 STM32CubeProgrammer 烧写系统了，烧写完成以后设置拨码开关从 EMMC 启动。启动以后进入 uboot 的命令行，设置 bootcmd 和 bootargs 这两个环境变量，命令如下：

```
setenv bootcmd 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
setenv bootargs 'console=ttySTM0,115200 root=/dev/mmcblk1p3 rootwait rw'
saveenv
boot
```

注意 bootargs 环境变量里面 root 的值为/dev/mmcblk1p3，说明根文件系统存放在 mmcblk1 的第 3 个分区。细心的朋友可能会有疑问，正点原子出厂系统的根文件系统是存放在 mmcblk2 的第 3 个分区，而这里设置的是 mmcblk1，为什么会不同呢？这是因为正点原子出厂系统使能了 SDIO WIFI，而 SDIO WIFI 也是 SDIO 接口，如果使能了 SDIO WIFI 那么 SDIO WIFI 所使用的那个 SDIO 接口就变成了 mmcblk1，而 EMMC 所使用的 SDIO 接口就会变为 mmcblk2。

设置以后启动系统，此时就会从 EMMC 里面加载 Linux 系统镜像，并且正确读取到根系统，如图 18.6.2.3 所示：

```
[    2.698405] Run /sbin/init as init process

Please press Enter to activate this console.
/ #
/ #
/ #
```

图 18.6.2.3 EMMC 启动系统

从图 18.6.2.3 可以看出，正常进入根文件系统。至此，Linux 系统移植就全部完成，包括 tf-a、uboot、Linux kernel 和 rootfs。

第十九章 Buildroot 根文件系统构建

上一小节我们学习了如何使用 busybox 来构建根文件系统，但是 busybox 构建的根文件系统不齐全，很多东西需要我们自行添加，比如 lib 库文件。在我们后面的驱动开发中很多第三方软件也需要我们自己去移植，这些第三方软件有很多又依赖其他的库文件，导致移植过程非常的繁琐。本章我们来学习一下另外一种更加实用的根文件系统构建方法，那就是使用 buildroot 来构建根文件系统。

19.1 何为 buildroot?

19.1.1 buildroot 简介

在上一章我们讲解了如何使用 busybox 构建文件系统，busybox 仅仅只是帮我们构建好了一些常用的命令和文件，像 lib 库、/etc 目录下的一些文件都需要我们自己手动创建，而且 busybox 构建的根文件系统默认没有用户名和密码设置。在后续的实验中，我们还要自己去移植一些第三方软件和库，比如 alsa、iperf、mplayer 等等。那么有没有一种傻瓜式的方法或软件，它不仅包含了 busybox 的功能，而且里面还集成了各种软件，需要什么软件就选择什么软件，不需要我们去移植。答案肯定是有的，buildroot 就是这样一种工具，buildroot 比 busybox 更上一层楼，buildroot 不仅集成了 busybox，而且还集成了各种常见的第三方库和软件，需要什么就选择什么，就跟我们去吃自助餐一样，想吃什么就拿什么。buildroot 极大的方便了我们嵌入式 Linux 开发人员构建实用的根文件系统。

从 busybox 开始一步一步的构建根文件系统适合学习、了解根文件系统的组成，但是不适合做产品(主要是自己构建的话会有很多不完善、没有注意到的细节)。buildroot 会帮我们处理好各种细节，根文件系统也会更加的合理、有效。因此在做产品的时候推荐大家使用 buildroot 来构建自己的根文件系统，当然了，类似 buildroot 的软件还有很多，比如 yocto，一般半导体厂商会选择 yocto 来制作系统包，但是由于国内的网络环境，yocto 编译起来会有很多问题！

buildroot 和 uboot、kernel 很类似，我们需要到其官网上下载源码，然后对其进行配置，比如设置交叉编译器、设置目标 CPU 参数等，最主要的就是选择所需要的第三方库或软件。一切配置好以后就可以进行编译，编译完成了以后就会在一个文件夹里面存放好编译结果，也就是根文件系统。

注意！buildroot 制作的根文件系统也是放到 ubuntu 的 nfs 目录下的 rootfs 文件夹，因此如果 rootfs 文件夹已经存放了前面 busybox 制作的根文件系统，那么请大家对其做一份备份，然后清空 rootfs 文件夹。

19.1.2 buildroot 下载

buildroot 源码肯定是要从 buildroot 官网下载，官网地址为 https://buildroot.org/，打开以后的官网界面如图 19.1.2.1 所示：

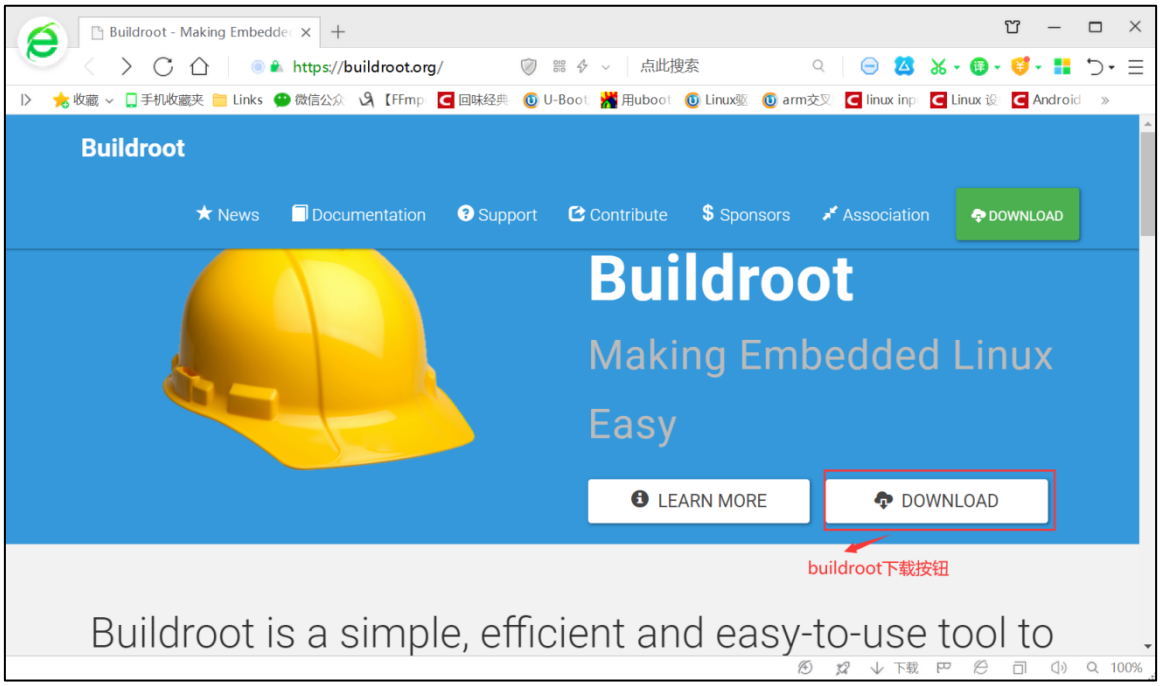


图 19.1.2.1 buildroot 官网界面

点击图 19.1.2.1 中的“DOWNLOAD”按钮即可打开 buildroot 的下载界面，如图 19.1.2.2 所示：

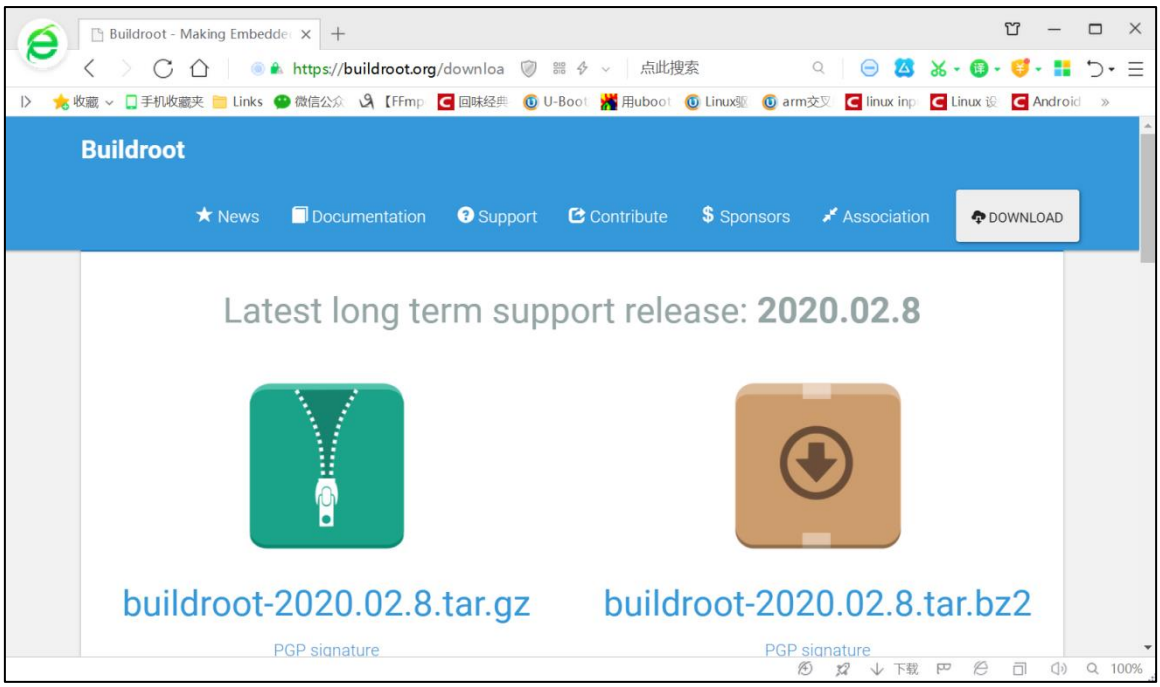


图 19.1.2.2 buildroot 下载界面

可以看出，在写本教程的时候最新的 LTS(长期支持版)版 buildroot 为 2020.02.8，分为.gz 和.bz2 两种压缩格式，这里我就使用右侧的.bz2 压缩格式的源码，选中以后下载即可。但是笔者在此前测试的都是 2020.02.6 版本的，为了保险起见，我们就使用 2020.02.6 版本的。我们已经将其放到了开发板光盘中，路径为：1、例程源码-》7、buildroot 源码-》buildroot-2020.02.6.tar.bz2，一切准备好以后就可以使用 buildroot 构建根文件系统了。

19.2 buildroot 构建根文件系统

19.2.1 配置 buildroot

将 buildroot 源码 buildroot-2020.02.6.tar.bz2 拷贝到 ubuntu 中，拷贝完成以后对其进行解压，命令如下：

```
tar -vxjf buildroot-2020.02.6.tar.bz2
```

解压完成以后就会得到一个名为“buildroot-2020.02.6”的目录，此目录就是我们解压得到的 buildroot 源码，进入到此目录中，此目录下的文件如图 19.2.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/tool$ cd buildroot-2020.02.6/
zuozhongkai@ubuntu:~/linux/tool/buildroot-2020.02.6$ ls
arch    CHANGES           configs     docs    Makefile         README    toolchain
board   Config.in         COPYING     fs      Makefile.legacy  support   utils
boot    Config.in.legacy  DEVELOPERS  linux   package          system
zuozhongkai@ubuntu:~/linux/tool/buildroot-2020.02.6$
```

图 19.2.1.1 buildroot 源码文件

buildroot 和 uboot、Linux kernel 一样也支持图形化配置，输入如下命令即可打开图形化配置界：

```
make menuconfig
```

打开以后的图形化配置界面如图 19.2.1.2 所示：

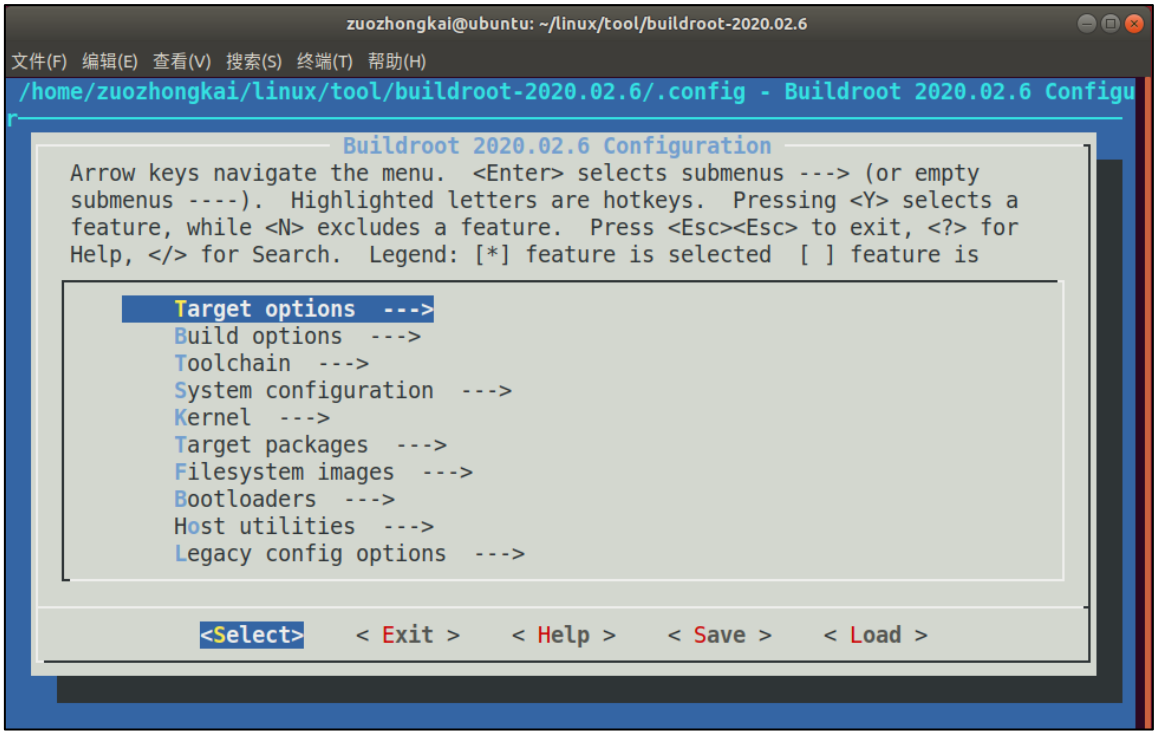


图 19.2.1.2 buildroot 图形化配置界面

接下来我们就依次配置 buildroot，配置完成以后就可以进行编译了。

1、配置 Target options

首先配置 Target options 选项，需要配置的项目和其对应的内容如下(“=”号后面是配置项要选择的内容！)：

```
Target options
```

```
        -> Target Architecture                  = ARM (little endian)
        -> Target Binary Format                 = ELF
        -> Target Architecture Variant          = cortex-A7
        -> Target ABI                           = EABIhf
        -> Floating point strategy              = NEON/VFPv4
        -> ARM instruction set                  = ARM
```

配置完成以后如图 19.2.1.3 所示：

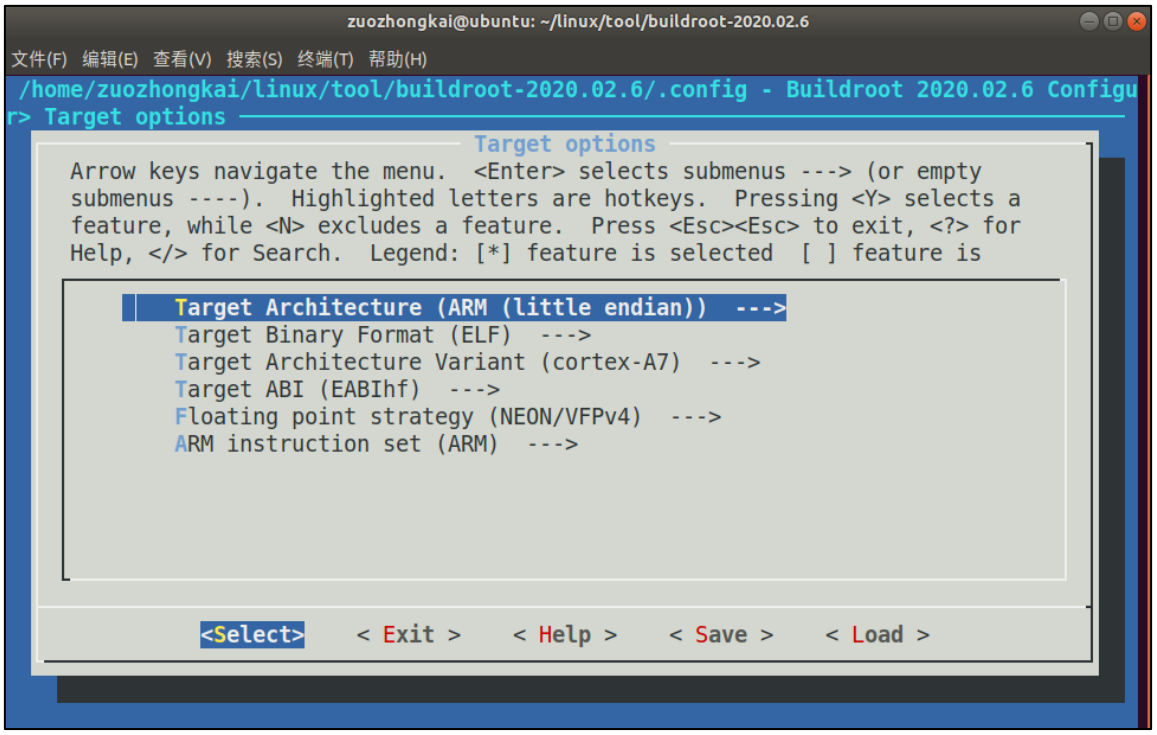


图 19.2.1.3 配置好的 Target options 选项

2、配置 Toolchain

此配置项用于配置交叉编译工具链，也就是交叉编译器，这里设置为我们自己所使用的交叉编译器即可。buildroot 其实是可以自动下载交叉编译器的，但是都是从国外服务器下载的，鉴于国内的网络环境，强烈推荐大家设置成自己所使用的交叉编译器。需要配置的项目和其对应的内容如下：

```
Toolchain
        -> Toolchain type    = External toolchain
        -> Toolchain         = Custom toolchain              //用户自己的交叉编译器
        -> Toolchain origin = Pre-installed toolchain       //预装的编译器
        -> Toolchain path    =          /usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-
gnueabihf
        -> Toolchain prefix       = $(ARCH) -none-linux-gnueabihf      //前缀
        -> External toolchain gcc version                = 9.x
        -> External toolchain kernel headers series   = 4.20.x        //交叉编译器的 linux 版本号
        -> External toolchain C library                  = glibc/eglibc
        -> [*] Toolchain has SSP support? (NEW)    //选中
        -> [*] Toolchain has RPC support? (NEW)    //选中
```

```
    -> [*] Toolchain has C++ support?          //选中
    -> [*] Enable MMU support (NEW)            //选中
```

Toolchain 下几个比较重要的选项需要说明一下，如下所示：

Toolchain：设置为 Custom toolchain，表示使用用户自己的交叉编译器。

Toolchain origin：设置为 Pre-installed toolchain，表示使用预装的交叉编译器。

Toolchain path：设置自己安装的交叉编译器绝对路径！buildroot 要用到。

Toolchain prefix：设置交叉编译器前缀，要根据自己实际所使用的交叉编译器来设置，比如我们使用的是 arm-none-linux-gnueabihf-gcc，因此前缀就是$(ARCH)-none-linux-gnueabihf，其中 ARCH 我们前面已经设置为了 arm。

External toolchain kernel headers series:这个设置的是交叉编译器所对应的 linux 内核版本号，gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf 所对应的 linux 内核版本号为 4.20，因此这里需要设置为 4.20.x！大家一定要根据自己所使用的交叉编译器来设置，否则编译的时候会报版本不一致的错误。

3、配置 System configuration

此选项用于设置一些系统配置，比如开发板名字、欢迎语、用户名、密码等。需要配置的项目和其对应的内容如下：

```
System configuration
    -> System hostname          = ATK-stm32mp1              //平台名字，自行设置
    -> System banner            = Welcome to alientek STM32MP157      //欢迎语
    -> Init system              = BusyBox                   //使用 busybox
    -> /dev management          = Dynamic using devtmpfs + mdev     //使用 mdev
    -> [*] Enable root login with password (NEW)            //使能登录密码
        -> Root password        = 123456                    //登录密码为 123456
```

在 System configuration 选项中可以配置平台名字，登录密码等信息。可以看出 buildroot 里面可以设置登录密码，但是作为实验，不建议大家设置登录密码，否则开发板每次重启都要输入密码，不方便开发。

4、配置 Filesystem images

此选项配置我们最终制作的根文件系统为什么格式的，配置如下：

```
-> Filesystem images
    -> [*] ext2/3/4 root filesystem          //如果是 EMMC 或 SD 卡的话就用 ext3/ext4
        -> ext2/3/4 variant     = ext4      //选择 ext4 格式
        -> exact size           =1G         //ext4 格式根文件系统 1GB(根据实际情况修改)
    -> [*] ubi image containing an ubifs root filesystem   //如果使用 NAND 的话就用 ubifs
```

可以看出，buildroot 可以直接制作出 ext4 格式的根文件系统，但是一般我们会自行往根文件系统里面添加很多其他的文件，所以产品开发完成以后需要自行打包根文件系统，然后烧写到开发板里面。不管是针对 EMMC 的 ext4 格式的根文件系统还是针对 NAND 的 ubi 格式的根文件系统，都要设置相应的大小，比如这里我们设置的 ex4 格式根文件系统大小为 1GB。

5、禁止编译 Linux 内核和 uboot

buildroot 不仅仅能构建根文件系统，也可以编译 linux 内核和 uboot。当配置 buildroot，使能 linux 内核和 uboot 以后 buildroot 就会自动下载最新的 linux 内核和 uboot 源码并编译。但是我们一般都不会使用 buildroot 下载的 linux 内核和 uboot，因为 buildroot 下载的 linux 和 uboot 官方源码，里面会缺少很多驱动文件，而且最新的 linux 内核和 uboot 会对编译器版本号有要

求，可能导致编译失败。因此我们需要配置 buildroot，关闭 linux 内核和 uboot 的编译，只使用 buildroot 来构建根文件系统，首先是禁止 Linux 内核的编译，配置如下：

```
-> Kernel
    -> [ ] Linux Kernel     //不要选择编译 Linux Kernel 选项！
```

如图 19.2.1.4 所示：

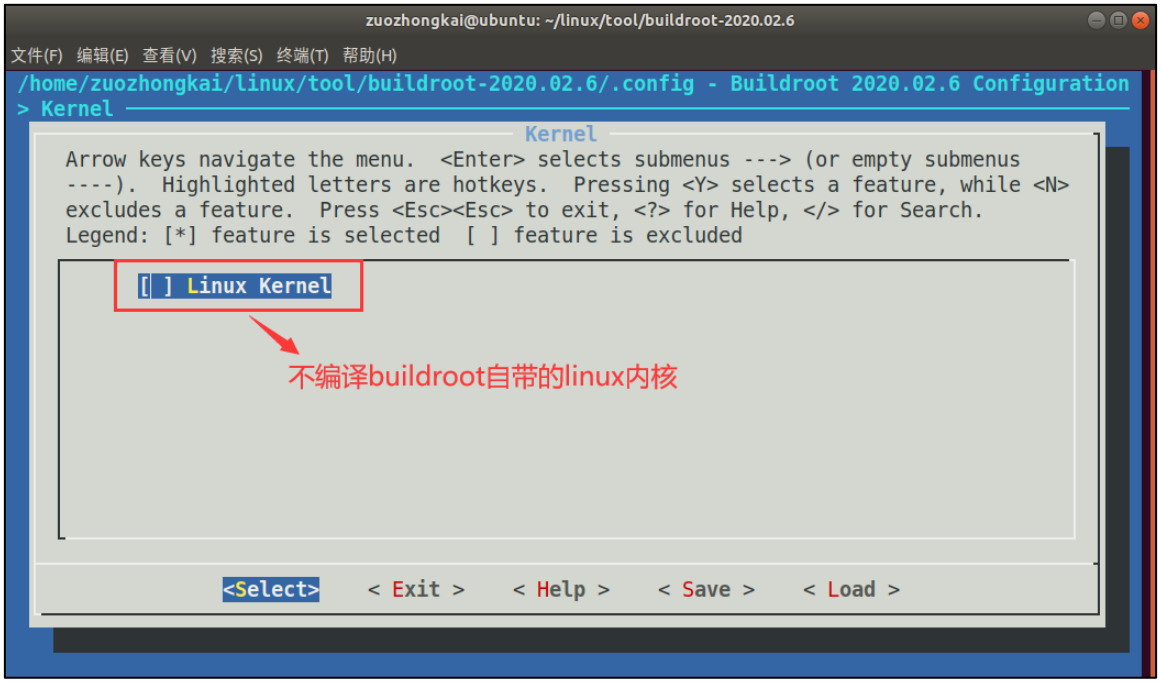


图 19.2.1.4 不编译 Linux 内核

接着禁止编译 Uboot，配置如下：

```
-> Bootloaders
    -> [ ] U-Boot     //不要选择编译 U-Boot 选项！
```

如图 19.2.1.5 所示：

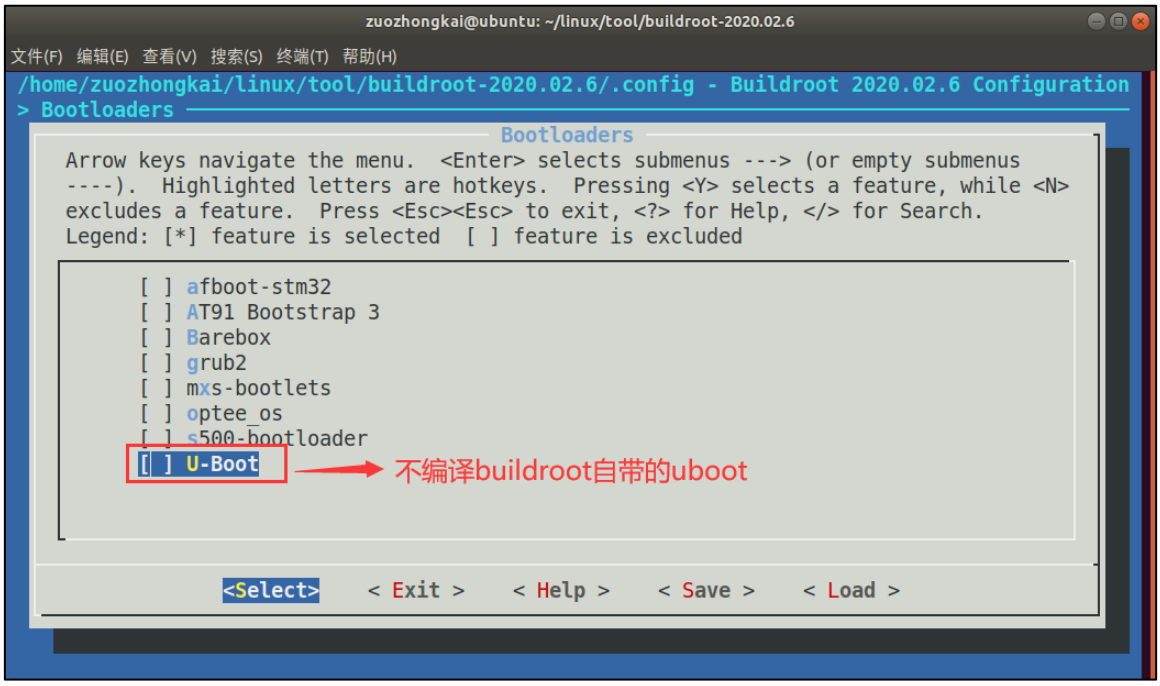


图 19.2.1.5 不编译 U-Boot

6、配置 Target packages

此选项用于配置要选择的第三方库或软件、比如 alsa-utils、ffmpeg、iperf 等工具，这里我们先只选择内核的模块加载相关软件，配置如下：

```
-> Target packages
    -> System tools
        -> [*] kmod          //使能内核模块相关命令
```

上面的配置是使能内核模块相关的操作命令，比如 depmod 等，如图 19.2.1.6 所示：

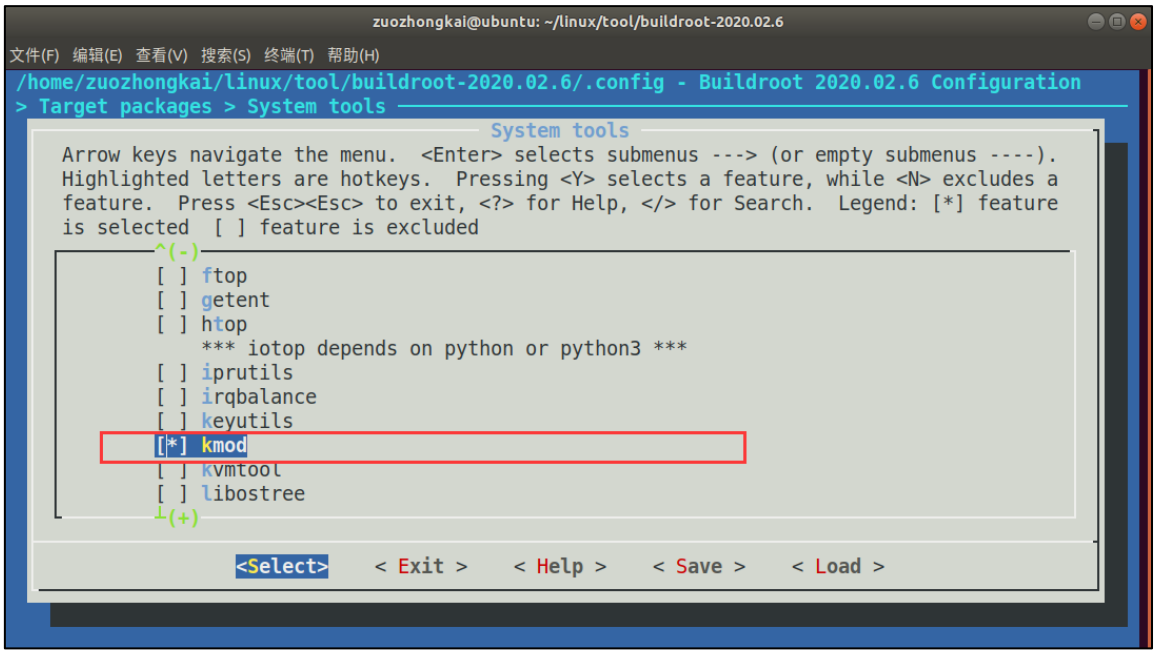


图 19.2.1.6 使能模块操作相关命令

至于其他的第三方库这里就先不选择了，先编译一下最基本的根文件系统，如果没有问题的话再重新配置选择第三方库和软件。否则一口吃太多会容易撑着的，编译出问题的时候都不知道怎么找问题。

7、保存配置项

和 uboot、kernel 一样，通过图形化界面配置好 buildroot 以后最好保存一下配置项，防止清除工程以后将配置项给删除掉。buildroot 的默认配置项都保存在 configs 目录下，配置完成以后选择<Save>，然后输入要设置的配置项名字，如图 19.2.1.7 所示：

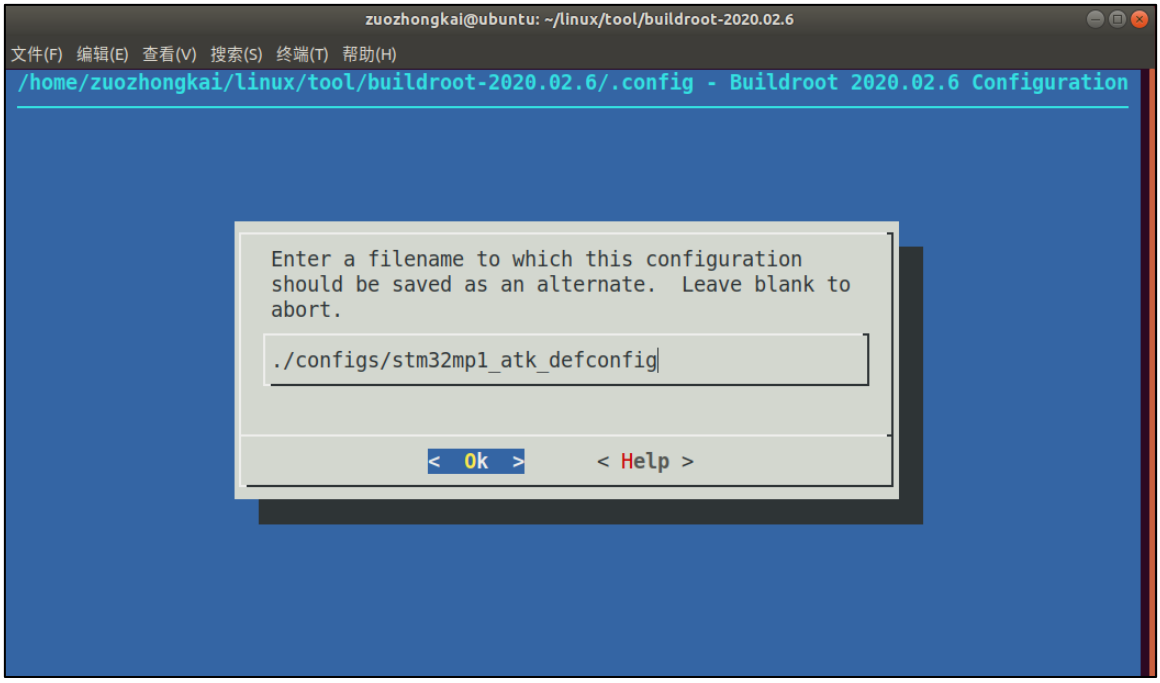


图 19.2.1.7 保存配置项

我们将配置项命名为“stm32mp1_atk_defconfig”，以后要重新配置 buildroot 的话就可以直接输入：

```
make stm32mp1_atk_defconfig
```

保存完成以后就会在 buildroot 的 configs 目录下看到“stm32mp1_atk_defconfig”，如图 19.2.1.8 所示：

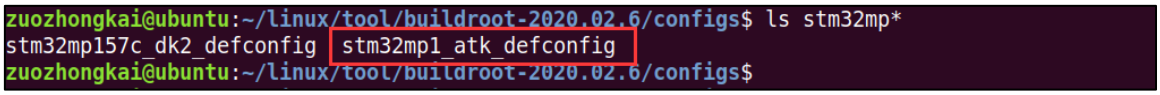


图 19.2.1.8 保存好的 buildroot 配置文件

19.2.2 编译 buildroot

配置完成以后就可以编译 buildroot 了，编译完成以后 buildroot 就会生成编译出来的根文件系统压缩包，我们可以直接使用。输入如下命令开始编译：

```
make -j8   //多线程编译
```

buildroot 编译的时候会先从网上下载所需的软件源码，有些软件源码可能下载不下来，这个时候就需要我们自行处理，这个后面会详细的讲解。

buildroot 编译过程会很耗时，因为所需要的所有软件要先从网上下载源码，很多软件都是从外网下载的，速度比较慢，在加上电脑配置的不同，整个编译过程可能需要几十分钟甚至数小时，请耐心等待！

buildroot 因为要从网上下载源码，因此可能存在有些源码无法下载或下载很慢的情况，比如如图 19.2.2.1 所示：

图 19.2.2.1 下载 cmake-3.8.2.tar.gz

可以看出图 19.2.2.1 中正在下载 cmake-3.8.2.tar.gz 这个压缩包，大小是 7.2MB，当前下载网速是 1.6KB/S，需要用时 71 分钟，显然这是无法忍受的！我们可以自行到 https://cmake.org/files/v3.8/cmake-3.8.2.tar.gz 这个网站上去将 cmake-3.8.2.tar.gz 这个源码下载下来，然后拷贝到 buildroot 源码目录下的 dl 文件夹中，dl 文件夹专用用于存放下载下来的源码。拷贝进去以后重新编译 buildroot，此时就不会在从网上下载 cmake-3.8.2.tar.gz 了，而是自己使用 dl 目录下已经存在的 cmake-3.8.2.tar.gz 压缩包，加快编译速度！

等待编译完成，编译完成以后就会在 buildroot-2020.02.6/output/images 下生成根文件系统，如图 19.2.2.2 所示：

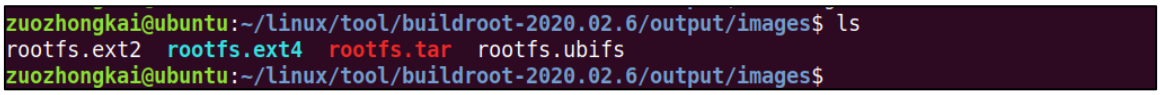


图 19.2.2.2 编译生成的根文件系统

从图 19.2.2.2 可以看出，编译出来了多种格式的 rootfs，比如 ext2、ext4、ubi 等。rootfs.ext4 就是可以直接使用 STM32CubeProgrammer 烧写到开发板中的。但是我们后面的开发都是通过 nfs 挂载的方式，所以我们就使用 rootfs.tar 进行测试。将 rootfs.tar 拷贝到在 nfs 目录下的 rootfs 文件夹中并解压，命令如下：

```
cd /home/zuozhongkai/linux/tool/buildroot-2020.02.6/output/images
cp rootfs.tar /home/zuozhongkai/linux/nfs/rootfs/ -f        //拷贝 rootfs.tar
cd /home/zuozhongkai/linux/nfs/rootfs/                      //进入到 rootfs 目录下
tar -vxf rootfs.tar                                         //解压缩 rootfs.tar
rm rootfs.tar                                               //删除 rootfs.tar
```

解压缩完成以后的 rootfs 目录如图 19.2.2.3 所示：

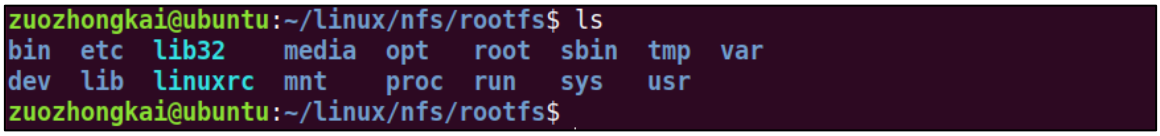


图 19.2.2.3 使用 buildroot 编译出来的根文件系统

图 19.2.2.3 就是使用 buildroot 编译出来的根文件系统，我们可以通过 nfs 挂载到开发板上，然后对其进行测试。

19.2.3 buildroot 根文件系统测试

buildroot 制作出来的根文件系统已经准备好了，接下来就是对其进行测试。测试方法也是通过 nfs 挂载的方式，启动 uboot，修改 bootargs 环境变量，设置 nfsroot 目录为 Ubuntu 中的 rootfs 目录，命令如下：

```
setenv bootargs console=ttySTM0,115200 root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw ip=192.168.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off"
```

设置好以后启动系统，进入根文件系统以后如图 19.2.3.1 所示：

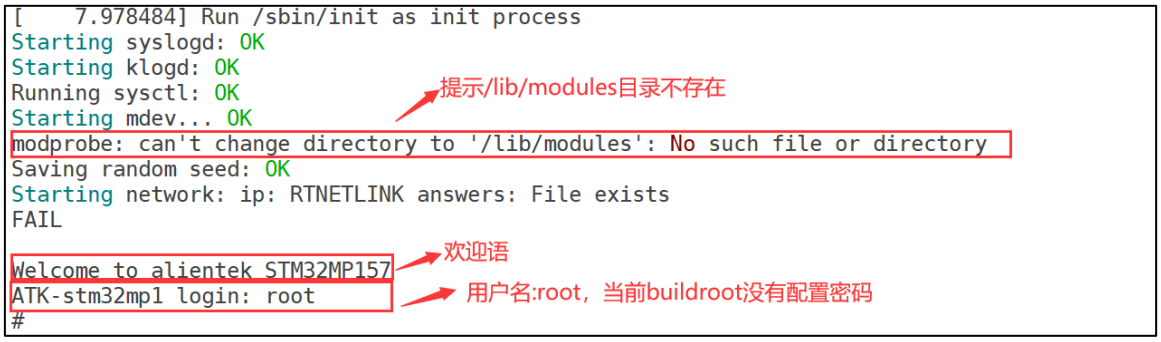


图 19.2.3.1 buildroot 根文件系统

在图 19.2.3.1 中，首先要进入到/lib/modules 目录，但是默认没有，因此需要我们自行创建此目录。buildroot 构建的根文件系统启动以后会输出我们前面设置的欢迎语“Welcome to alientek STM32MP157”。然后需要输入用户名和密码，用户名是“root”，如果配置 buildroot 的时候设置了密码，那么就需要输入密码，这里我们没有设置密码，因此输入用户名就行了，如图 19.2.3.2 所示：

```
Welcome to alientek STM32MP157
ATK-stm32mp1 login: root
#
# ls
# ls /
bin         lib32       opt         rootfs.tar  tmp
dev         linuxrc     proc        run         usr
etc         media       root        sbin        var
lib         mnt         rootfs.ext4 sys
#
```

图 19.2.3.1 buildroot 根文件系统文件

可以看出的 buildroot 构建的根文件系统运行基本没有问题，但是这个根文件系统是最简单的，我们并没有在 buildroot 里面配置任何第三方的库和软件，接下来我们就配置 buildroot，使能一些常见的第三方软件。

19.3 buildroot 下的 busybox 配置

19.3.1 busybox 配置

buildroot 在构建根文件系统的时候也是要用到 busybox 的，既然用到了 busybox 那么就涉及到 busybox 的配置。buildroot 会自动下载 busybox 压缩包，buildroot 下载的源码压缩包都存

放在/dl 目录下，在 dl 目录下就有一个叫做“busybox”的文件夹，此目录下保存着 busybox 压缩包，如图 19.3.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/tool/buildroot-2020.02.6/dl/busybox$ ls
busybox-1.31.1.tar.bz2
zuozhongkai@ubuntu:~/linux/tool/buildroot-2020.02.6/dl/busybox$
```

图 19.3.1.1 busybox 源码

可以看出，buildroot 下载的 busybox 版本为 1.31.1。要想编译 busybox，必须对图 19.3.1.1 中的压缩包进行解压缩，buildroot 会对其进行解压！buildroot 将所有解压缩后的软件保存在/output/build 软件中，我们可以在找到/output/build/busybox-1.31.1 这个文件夹，此文件夹就是解压后的 busybox 源码，文件内容如图 19.3.1.2 所示：

```
zuozhongkai@ubuntu:~/linux/tool/buildroot-2020.02.6/output/build/busybox-1.31.1$ ls
applets                 debianutils  mailutils              README
applets_sh              docs         Makefile               runit
arch                    e2fsprogs    Makefile.custom        scripts
archival                editors      Makefile.flags         selinux
AUTHORS                 examples     Makefile.help          shell
busybox                 findutils    make_single_applets.sh size_single_applets.sh
busybox.links           include      miscutils              sysklogd
busybox_unstripped      init         modutils               testsuite
busybox_unstripped.map  INSTALL      networking             TODO
busybox_unstripped.out  klibc-utils  NOFORK_NOEXEC.lst      TODO_unicode
Config.in               libbb        NOFORK_NOEXEC.sh       util-linux
configs                 libpwdgrp    printutils
console-tools           LICENSE      procps
coreutils               loginutils   qemu_multiarch_testing
zuozhongkai@ubuntu:~/linux/tool/buildroot-2020.02.6/output/build/busybox-1.31.1$
```

图 19.3.1.2 busybox 源码

如果大家想要修改 busybox 源码的话就直接在图 19.3.1.2 中找到相应的文件，然后修改即可。我们现在是要配置 buildroot 下的 busybox，因此肯定要打开 busybox 的配置界面，在 buildroot 源码根目录下输入如下命令：

```
make busybox-menuconfig
```

输入以后就会打开 buildroot 下的 busybox 配置界面，如图 19.3.1.3 所示：

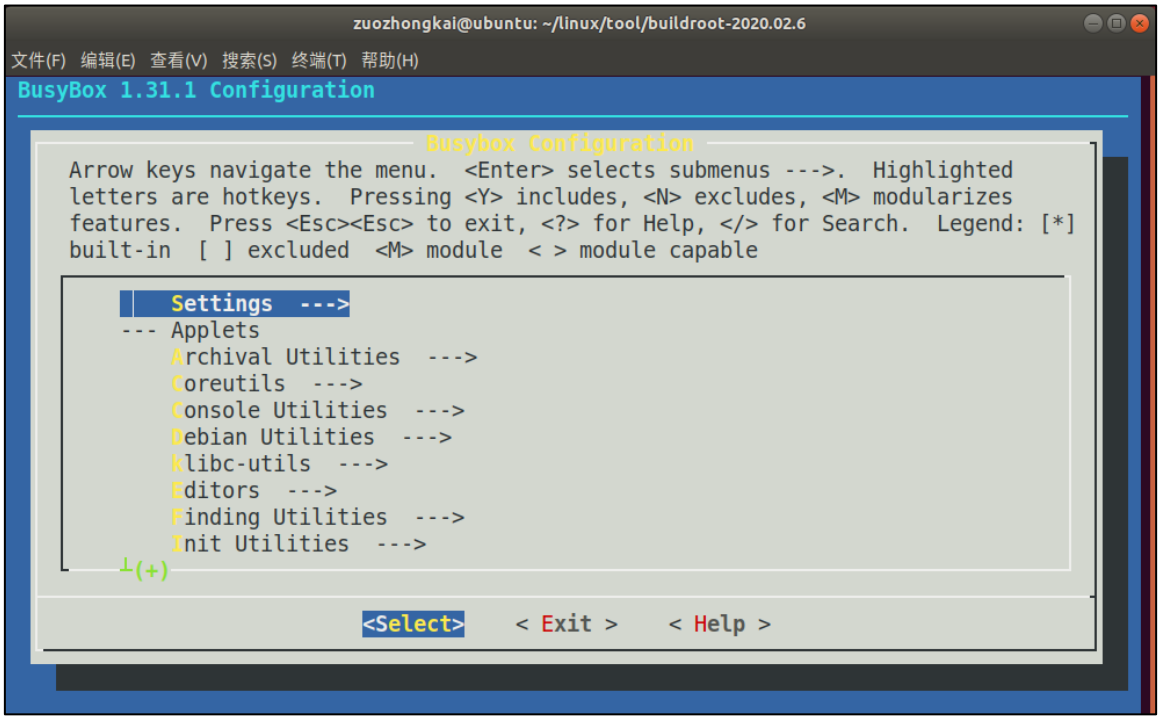


图 19.3.1.3 buildroot 下的 busybox 配置界面

图 19.3.1.3 就是我们最熟悉的 busybox 配置界面了，大家想要配置什么就直接操作就行了，或者参考第十八章即可。

19.3.2 busybox 中文字符的支持

在第十八章我们讲过了，busybox 对中文字符显示做了限制，因此必须要修改相关的文件，具体修改过程参考 18.2.2 小节即可，这里就不再赘述了。

19.3.3 编译 busybox

配置好以后就可以重新编译 buildroot 下的 busybox，进入到 buildroot 源码目录下，输入如下命令查看当前 buildroot 所有配置了的目标软件包，也就是 packages：

```
make show-targets
```

结果如图 19.3.3.1 所示：

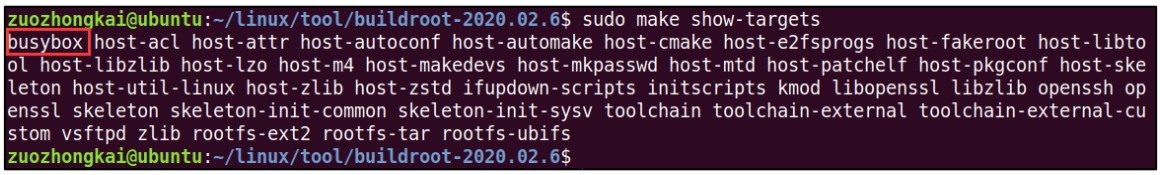


图 19.3.3.1 当前 buildroot 下的所有 packages

图 19.3.3.1 中列出了当前 buildroot 中所有使能了的 packages 包，其中就包括 busybox，如果我们想单独编译并安装 busybox 的话执行下面命令即可：

```
make busybox
```

上述命令就会重新编译 busybox。编译完成以后重新编译 buildroot，主要是对其进行打包，输入如下命令：

```
make -j8
```

重新编译完成以后查看一下 output/images 目录下 rootfs.tar 的创建时间是否为刚刚编译的，如果不是的话就删除掉 rootfs.tar，然后重新执行“sudo make”重新编译一下即可。最后我们使用新的 rootfs.tar 启动 Linux 系统。

19.4.3 使用自己的 busybox 源码

buildroot 会自行下载 busybox 并编译，但是有时候我们不想要 buildroot 自带的 busybox，我们想要使用自己配置好的 busybox，比如第十八章我们已经配置好的 busybox。这个可以在 buildroot 中实现，只要进行相应的配置，指定要使用的 busybox 源码目录即可。这个要自己编写 buildroot 的 local.mk 文件，在此文件里面指定 busybox 目录，在 buildroot 的 configs 目录下新建一个名为“local.mk”的文件，然后在里面输入如下格式内容：

```
XXXXXX_OVERRIDE_SRCDIR=‘具体路径’
```

比如指定 busybox 路径的时候就使用：

```
BUSYBOX_OVERRIDE_SRCDIR=/home/zuozhongkai/linux/busybox/busybox-1.32.0
```

local.mk 准备好以后就可以配置 buildroot，使用 local.mk，打开 buildroot 的配置界面需要配置的项目和其对应的内容如下(“=”号后面是配置项要选择的内容！)：

```
-> Build options
    -> location of a package override file    = ./configs/local.mk  指定 local.mk 所在目录
```

如图 19.4.3.1 所示：

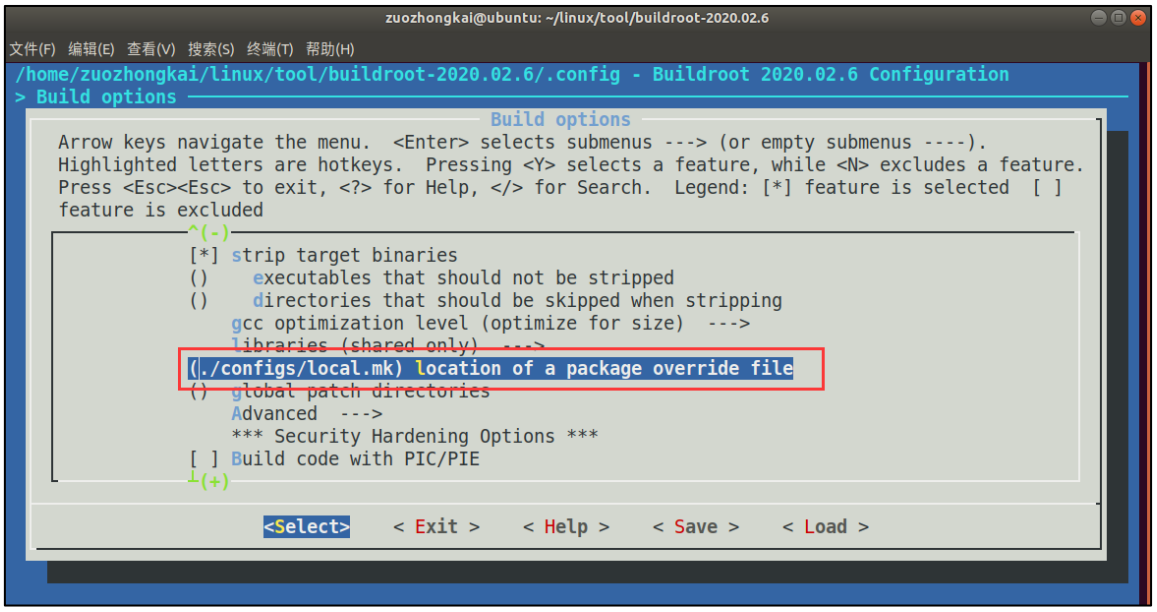


图 19.4.3.1 设置 local.mk

图 19.4.3.1 中“location of a package override file”选项就是设置 local.mk 路径的，这里设置为“./configs/local.mk”。重新编译 buildroot，此时 buildroot 会将 BUSYBOX_OVERRIDE_SCRDIR 指定的 busybox 源码拷贝到 buildroot 的 output/build/busybox-custom 目录下。编译完成以后使用新的根文件系统启动，输入“busybox”命令查看 busybox 版本号是否为 1.32.0，如图 19.4.3.2 所示：

```
# busybox
BusyBox v1.32.0 (2020-12-22 12:36:23 CST) multi-call binary.
BusyBox is copyrighted by many authors between 1998-2015.
Licensed under GPLv2. See source distribution for detailed
copyright notices.
```

图 19.4.3.2 使用自定义 busybox 源码

从图 19.4.3.2 可以看出，此时 buildroot 中的 busybox 版本号变为了 1.32.0。为了方便开发，建议大家使用 buildroot 自带的 busybox，也就是 1.31.0 版本的。

19.4 buildroot 第三方软件和库的配置

上一小节我们通过 buildroot 制作了一个最基本的根文件系统，本节我们来学习一下如何配置 buildroot 使能一些第三放软件或库，比如 FTP 和 SSH 服务等。

19.4.1 使能 VSFTPD 服务

我们可以在开发板上搭建一个 FTP 服务器，这样我们就可以使用 FileZilla 软件直接向开发板拷贝文件，或者将开发板中的文件拷贝到电脑中。这里我们通过使能 buildroot 中的 vsftpd 软件来完成 FTP 服务器的搭建，配置路径如下：

```
-> Target packages
    -> Networking applications
        -> [*] vsftpd                    //使能 vsftpd
```

如图 19.4.1.1 所示：

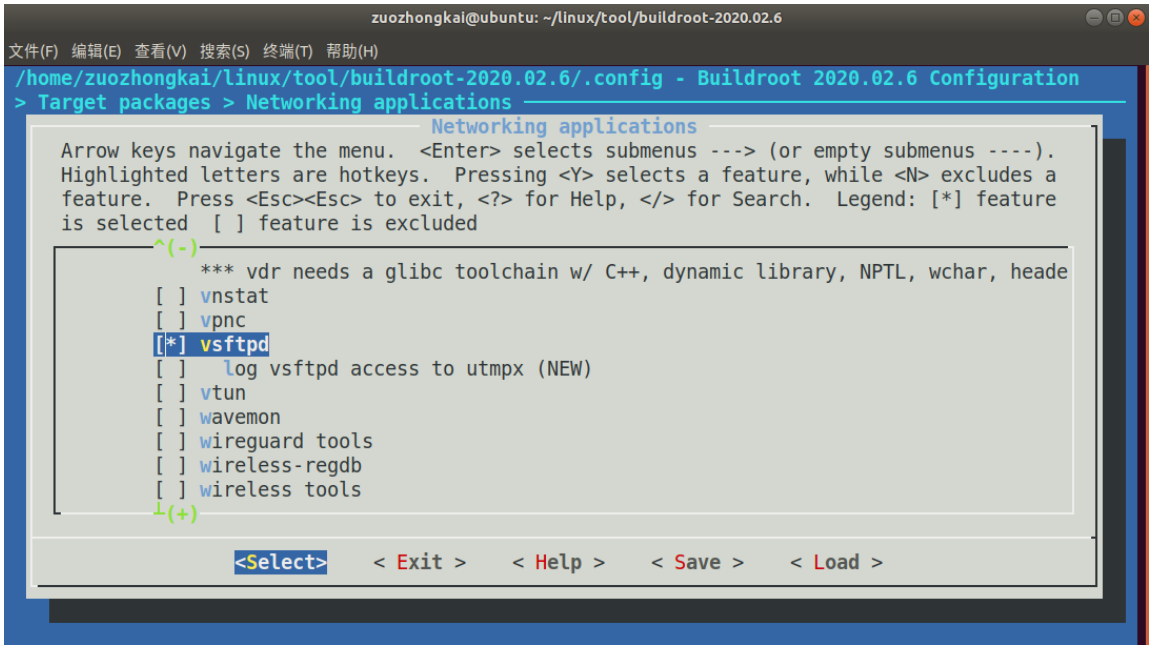


图 19.4.1.1 使能 vsftpd

19.4.2 使能 SSH

有时候我们需要远程登录开发板，这个时候就可以通过网络登录，要用到 SSH 服务，OpenSSH 是 SSH 的开源免费版本。直接使能 buildroot 中的 OpenSSH 即可，配置路径如下：

```
-> Target packages
    -> Networking applications
        -> [*] openssh                   //使能 openssh
```

如图 19.4.2.1 所示：

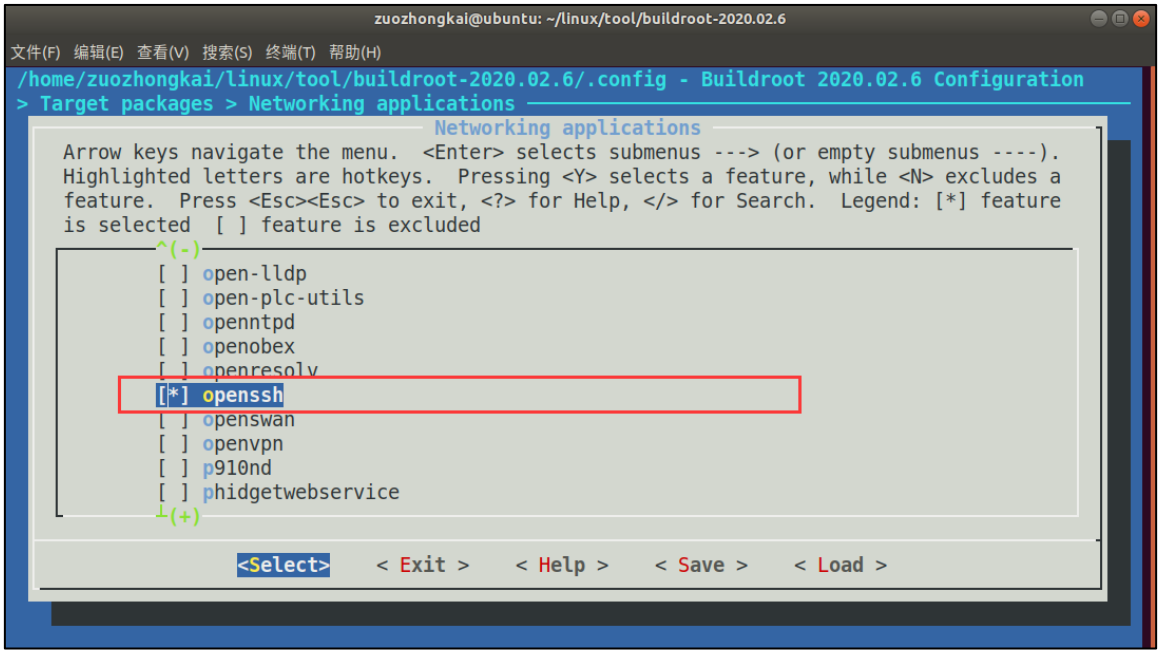


图 19.4.2.1 使能 openssh

等待编译完成就可以使用新的根文件系统进行测试了，将 ouput/images/rootfs.tar 拷贝到 nfs 目录下的 rootfs 目录中，然后重新解压。注意，以前自己添加的文件并不会被删除掉的，解压命令如下：

```
tar -vxf rootfs.tar
```

解压完成以后就可以使用 FTP 和 SSH 等相关的软件了，由于 FTP 和 SSH 都是通过网络进行数据传输的，因此需要先配置网络，如果是通过 nfs 挂载的根文件系统，那么网络已经初始化完成了，因此可以直接使用。如果是烧写到 EMMC 里面的，那么就需要先配置网络相关功能。

关于 VSFTPD 和 SSH 的测试稍后再讲解。

19.5 buildroot 根文件系统测试

buildroot 的根文件系统制作好以后就是测试工作了，基本测试内容和 18.5 小节一样，这里就不赘述了，我们重点讲解一下其他的测试内容。

19.5.1 depmod 命令测试

depmod 命令非常重要，我们后面学习 Linux 驱动的时候需要使用此命令分析模块的依赖性，此命令需要在 busybox 中使能，路径如下：

```
-> Linux Module Utilities
    -> [*] depmod              //使能 depmod 命令
```

如图 19.5.1.1 所示：

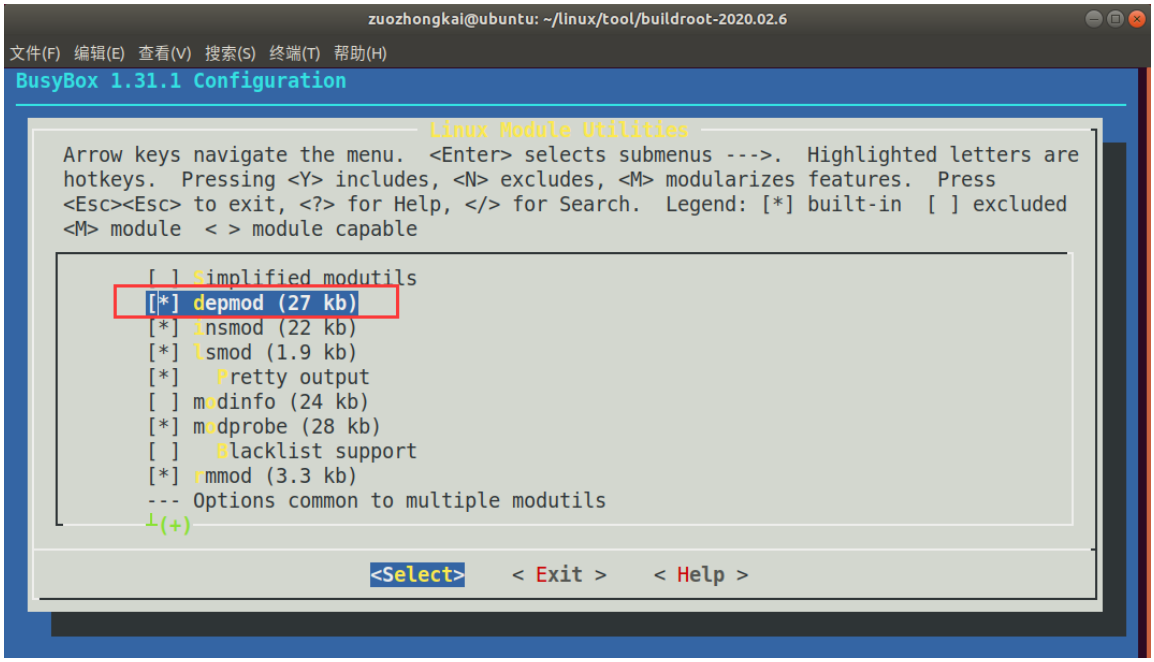


图 19.5.1.1 使能 depmod 命令

默认情况下 depmod 命令是使能了的，但是如果大家输入 depmod 命令以后发现不存在，那么就自行重新配置一下 buildroot 下的 busybox，按照上面的方法使能 depmod 命令并编译一次即可，最后使用新的 buildroot 根文件系统启动，输入 depmod 命令，一般会有如图 19.5.1.2 所示错误提示：

```
# depmod
depmod: can't change directory to 'lib/modules/5.4.31': No such file or directory
#
```

图 19.5.1.2 depmod 命令错误提示

从图 19.5.1.2 可以看出，depmod 命令提示没有找到“lib/modules/5.4.31”目录(后面的 5.4.31 是和具体的 Linux 内核版本相关的，我们本教程使用的 Linux 内核版本为 5.4.31。如果你使用的其他版本的 Linux 内核，那么这里就是相应的内核版本名字，比如 4.1.15)。我们一般将驱动模块放到 lib/modules/5.4.31 目录下，既然当前根文件系统不存在这个目录，那么就手动创建此目录，命令如下：

```
mkdir /lib/modules/5.4.31 -p
```

创建好以后在执行 depmod 命名就不会报错了，并且 depmod 命令会在 lib/modules/5.4.31 目录下生成三个文件：modules.alias、modules.dep、modules.symbols，如图 19.5.1.3 所示：

```
# ls
modules.alias    modules.dep      modules.symbols
#
```

图 19.5.1.3 depmod 命令产生的三个文件

图 19.5.1.3 中这三个文件不需要我们去操作，系统会自己使用这三个文件。

19.5.2 vsftpd 测试

接下来测试一下用 vsftpd 搭建的 FTP 服务器，在测试之前要先进行一些设置，首先需要对 vsftpd 进行配置，打开/etc/vsftpd.conf 文件，将下面两行前面的“#”去掉：

```
local_enable=YES        //取消掉前面的‘#’
```

write_enable=YES //取消掉前面的‘#’

接下来需要修改/etc/vsftpd.conf 文件的所属用户，默认为 sshd 用户，我们要将其改为 root 用户，输入如下命令：

chown root:root /etc/vsftpd.conf //修改 vsftpd.conf 文件所属用户

必须修改 vsftpd.conf 的所有用户以及所属用户组，否则 vsftpd 运行的时候会报错！修改以后的 vsftpd.conf 所属用户以及用户组就都是 root 了，如图 19.5.2.1 所示：

```
# ls /etc/vsftpd.conf -l
-rw-r--r--    1 root     root          4591 Dec 21  2020 /etc/vsftpd.conf
#
```

图 19.5.2.1 修改后的 vsftpd.conf 所属用户和用户组

最后在开发板上使用 adduser 命令新建一个用户要来完成 FTP 登录，如图 19.5.2.2 所示：

```
# adduser zuozhongkai
Changing password for zuozhongkai
New password:
Bad password: too weak
Retype password:
passwd: password for zuozhongkai changed by root
#
```

新建名为“zuozhongkai”的用户

图 19.5.2.2 新建“zuozhongkai”用户

创建完成以后就会在/home 目录下存在“zuozhongkai”用户目录，如图 19.5.2.3 所示：

```
# ls /home/
ftp          zuozhongkai
#
```

图 19.4.3.3 zuozhongkai 用户目录

设置完成以后重启开发板，vsftpd 服务默认会使能！启动 log 信息如图 19.5.2.4 所示：

```
modprobe: can't change directory to '/lib/modules': No such file or directory
Initializing random number generator: OK
Saving random seed: OK
Starting network: ip: RTNETLINK answers: File exists
FAIL
Starting sshd: OK
Starting vsftpd: OK
```

图 19.5.2.4 vsftpd 开机自启动

从图 19.5.2.4 可以看出，开机以后 vsftpd 默认已经启动了，大家可以输入“ps”命令查看一下，结果如图 19.5.2.5 所示：

```
  155 root     /sbin/mdev -df
  184 root     /usr/sbin/sshd
  190 root     vsftpd
  191 root     -sh
  193 root     ps
```

图 19.5.2.5 后台运行的 vsftpd 软件

至此 vsftpd 服务已经正常运行了，我们可以使用 FileZilla 来登录开发板，用户名使用前面创建的“zuozhongkai”即可，如图 19.5.2.6 所示：

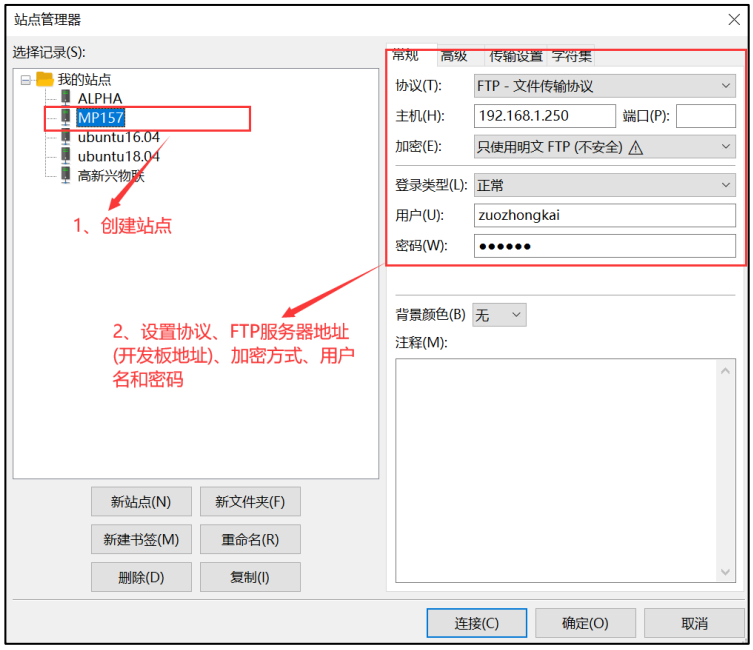


图 19.5.2.6 设置 FileZilla

设置好以后就可以连接了，连接成功以后如图 19.5.2.7 所示：

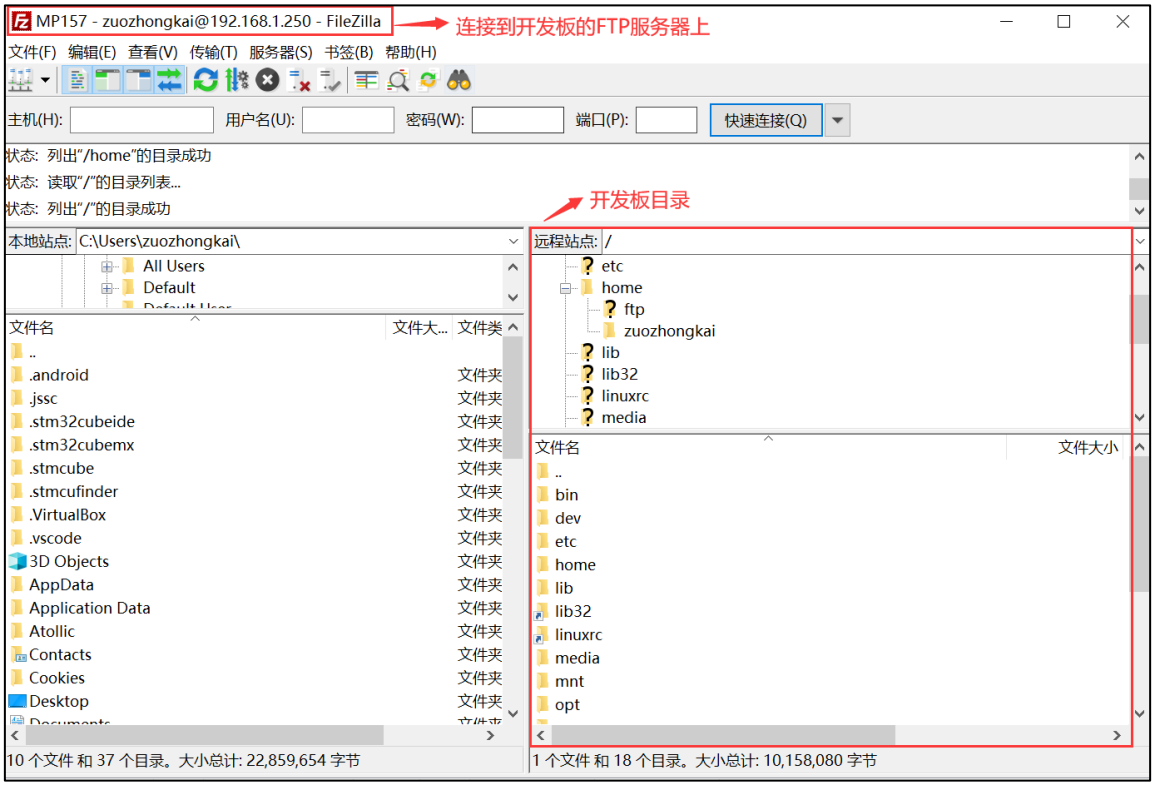


图 19.5.2.7 FileZilla 连接开发板

至此，开发板 FTP 服务就搭建完成了。

19.5.3 sshd 测试

SSHD 不仅包含 SSH 服务，也包含 scp 指令，SSHD 不需要进行配置，只需要创建一个登陆用户即可，前面在 FTP 设置中已经创建了一个名为“zuozhongkai”的用户，所以这里就不用

再创建用户了，直接使用“zuozhongkai”这个用户即可。SSHD 默认会启动开启，然后在后台运行，开发板 linux 系统启动的时候会输出 SSHD 开启信息，如图 19.5.3.1 所示：

```
Starting network: ip: RTNETLINK answers: File exists
FAIL
Starting sshd: OK
Starting vsftpd: OK
```

图 19.5.3.1 SSHD 服务自启动

从图 19.5.3.1 可以看出，SSHD 服务开机自启动了，如果 SSHD 启动失败并且提示“sshd: /var/empty must be owned by root and not group or world-writable.”，此时我们需要修改/var/empty 目录所属用户以及用户组，输入如下命令：

```
chown root:root /var/empty
```

完成以后重启开发板即可，进入系统以后通过“ps”命令来查看 sshd 后台服务，如图 19.5.3.2 所示:

```
  155 root      /sbin/mdev -df
  184 root      /usr/sbin/sshd
  190 root      vsftpd
  191 root      -sh
  193 root      ps
```

图 19.5.3.2 后台运行的 sshd

sshd 测试方法很简单，使用 MobaXterm 软件通过 SSH 服务登录即可，打开 MobaXterm 软件，点击“Session”按钮，在弹出的“Session settings”界面上选择“SSH”。打开以后在“Remote Host”栏填写开发板的 IP 地址，勾选后面的“Specify username”，并且填写登录用户名，比如：zuozhongkai，配置完成以后如图 19.5.3.3 所示：

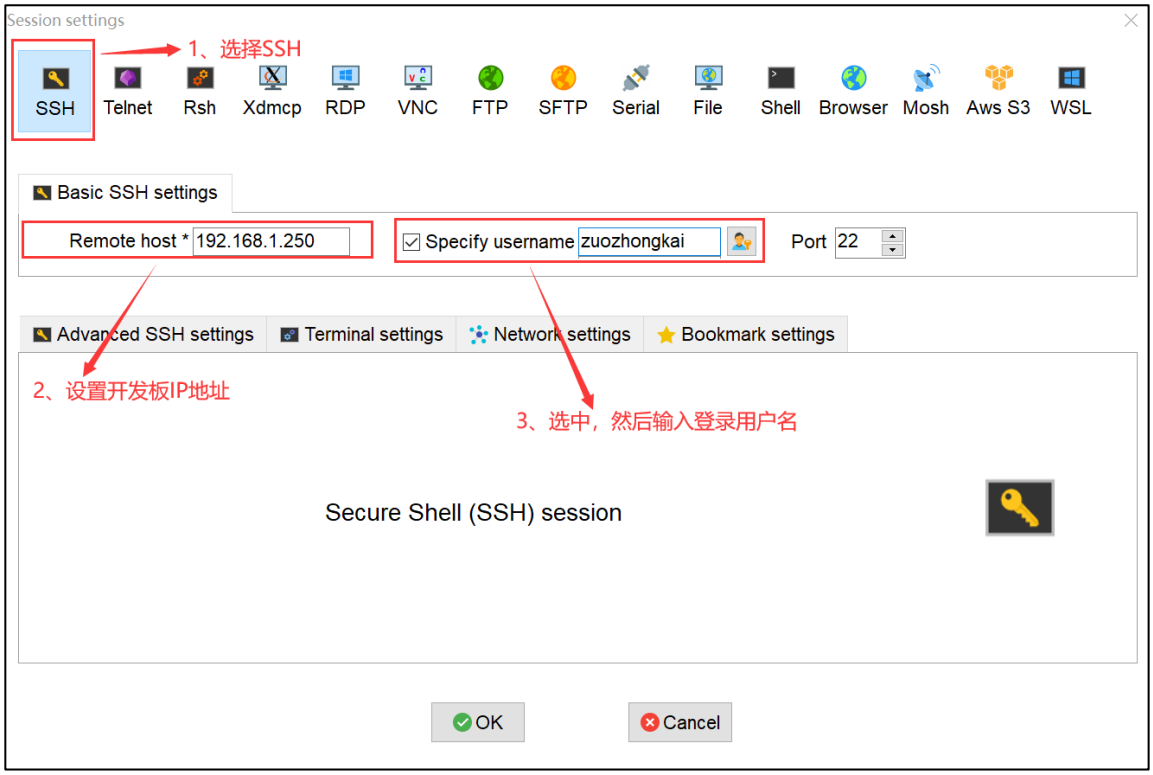


图 19.5.3.3 SSH 配置

配置好以后点击“OK”按钮，此时就会打开 ssh 会话框，并且要求你输入用户密码，如图 19.5.3.4 所示：

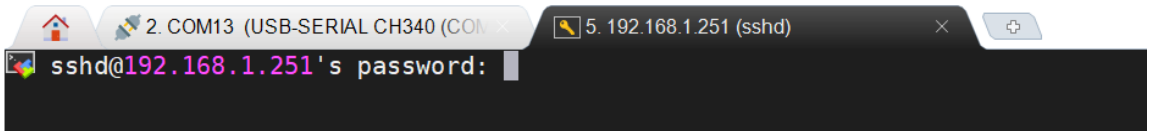


图 19.5.3.4 SSH 登录

输入 sshd 用户密码，可能会弹出是否保存密码对话框，选择保存即可。如果密码正确的话就会登录到开发板上，如图 19.5.3.5 所示：

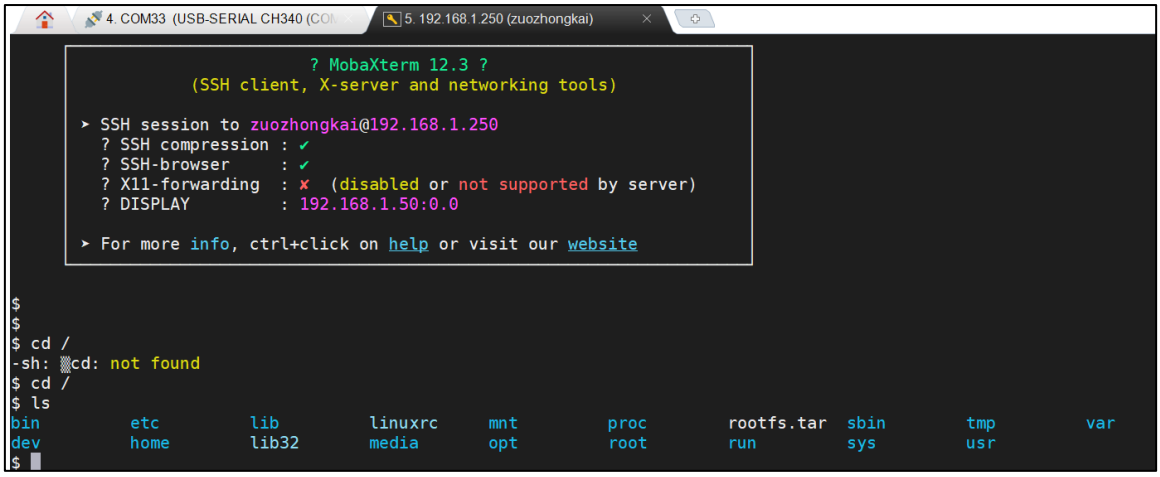


图 19.5.3.5 SSH 登录成功

同样的，我们也可以在 ubuntu 下通过 ssh 命令登录开发板，输入如下命令：

```
ssh zuozhongkai@192.168.1.251
```

其中“zuozhongkai”为登录账户名字，192.168.1.250 是开发板的 IP 地址。用户名和开发板 IP 地址之间用“@”符号链接起来。第一次与开发板建立连接的时候会让你进行确认，输入“yes”就行了，如图 19.5.3.6 所示：

```
zuozhongkai@ubuntu:~$ ssh zuozhongkai@192.168.1.250
The authenticity of host '192.168.1.250 (192.168.1.250)' can't be established.
ECDSA key fingerprint is SHA256:SQ1eU4SqI6cR1BVhrclDmVO+mT8OEm+mew1ayLdedxU.
Are you sure you want to continue connecting (yes/no)? yes
```

图 19.5.3.6 确认建立连接

输入“yes”以后就会让你输入“zuozhongkai”用户密码，如图 19.5.3.7 所示：

```
zuozhongkai@ubuntu:~$ ssh zuozhongkai@192.168.1.250
The authenticity of host '192.168.1.250 (192.168.1.250)' can't be established.
ECDSA key fingerprint is SHA256:SQ1eU4SqI6cR1BVhrclDmVO+mT8OEm+mew1ayLdedxU.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.1.250' (ECDSA) to the list of known hosts.
zuozhongkai@192.168.1.250's password:
```

图 19.5.3.7 输入 zuozhongkai 用户密码

如果密码正确的话就会登录进开发板，可以对开发板进行各种操作，如图 19.5.3.8 所示：

```
zuozhongkai@192.168.1.250's password:
$ cd /
$ ls
bin         home        linuxrc     opt         rootfs.tar  sys         var
dev         lib         media       proc        run         tmp
etc         lib32       mnt         root        sbin        usr
$
```

图 19.5.3.8 通过 SSH 对开发板进行操作

输入“exit”命令即可退出 SSH 会话。

19.5.4 创建自启动文件

前面在测试 busybox 的根文件系统的时候都是直接在/etc/init.d/rcS 里面添加自启动相关命令的，但是 buildroot 构建的根文件系统中就不需要直接在/etc/init.d/rcS 中添加自启动命令了，默认情况下 buildroot 构建的根文件系统中 rcS 文件内容如图 19.5.4.1 所示：

```
#!/bin/sh


# Start all init scripts in /etc/init.d
# executing them in numerical order.
#
for i in /etc/init.d/S??* ;do

     # Ignore dangling symlinks (if any).
     [ ! -f "$i" ] && continue

     case "$i" in
        *.sh)
            # Source shell script for speed.
            (
                trap - INT QUIT TSTP
                set start
                . $i
            )
            ;;
        *)
            # No sh extension, so fork subprocess.
            $i start
            ;;
    esac
done
```

图 19.5.4.1 rcS 默认内容

从图 19.5.4.1 中可以看出，rcS 默认会在/etc/init.d 目录下查找所有以‘S’开头的脚本，然后依次执行这些脚本。所以我们可以自己创建一个以‘S’开头的自启动脚本文件，比如我创建一个名为 Sautorun 的自启动文件，命令如下：

```
cd /etc/init.d/                 //进入/etc/init.d 目录
touch Sautorun                  //使用 touch 命令创建 Sautorun 脚本
chmod 777 Sautorun              //给予 Aautorun 脚本可执行权限
```

最后在 Sautorun 脚本里面输入要执行的命令，比如要开机自启动 test 这软件，那么 Sautorun 脚本内容如图 19.5.4.2 所示：

```
#/bin/sh

cd /
./test &
cd /
```

图 19.5.4.2 Sautorun 脚本内容

设置好以后重启开发板，此时 Sautorun 脚本就会被 rcS 调用，进而运行 test 软件。

19.5.5 显示路径

这里我们重点说一下另外一个问题，我们使用 buildroot 构建的根文件系统启动以后会发现，输入命令的时候命令行前面一直都是“#”，如果我们进入到某个目录的话前面并不会显示当前目录路径，如图 19.5.5.1 所示：

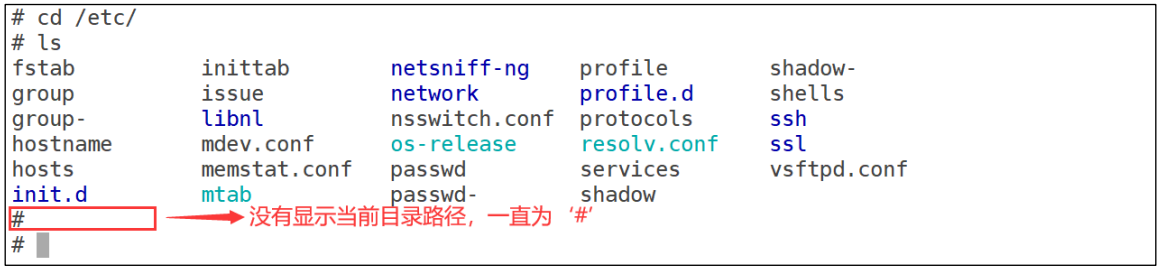


图 19.5.5.1 目录路径未显示

从图 19.5.5.1 可以看出，我们当前所处的目录是/etc，但是前面的提示符一直是“#”，这样不利于我们查看自己当前所处的路径。最好能像 Ubuntu 一样，可以指出当前登录的用户名，主机名以及所处的目录，如图 19.5.5.2 所示：

图 19.5.5.2 Ubuntu shell 命令前缀信息

我们现在就来设置，实现图 19.5.5.2 中所示的效果。我们要先了解一下“PS1”这个环境变量，PS1 用于设置命令提示符格式，格式如下：

PS1 = ‘命令列表’

命令列表中可选的参数如下：

\!：显示该命令的历史记录编号。
\#：显示当前命令的命令编号。
\$：显示$符作为提示符，如果用户是 root 的话，则显示#号。
\\：显示反斜杠。
\d：显示当前日期。
\h：显示主机名。
\n：打印新行。
\nnn：显示 nnn 的八进制值。
\s：显示当前运行的 shell 的名字。
\t：显示当前时间。
\u：显示当前用户的用户名。
\W：显示当前工作目录的名字。
\w：显示当前工作目录的路径

我们打开/etc/profile 文件，文件内容如下所示：

示例代码 19.5.5.1 /etc/profile 文件要屏蔽的内容

```
1 export PATH="/bin:/sbin:/usr/bin:/usr/sbin"
2
3 if [ "$PS1" ]; then
```

```
4         if [ "`id -u`" -eq 0 ]; then
5                 export PS1='# '
6         else
7                 export PS1='$ '
8         fi
9  fi
10
11 export EDITOR='/bin/vi'
12
13 # Source configuration files from /etc/profile.d
14 for i in /etc/profile.d/*.sh ; do
15         if [ -r "$i" ]; then
16                 . $i
17         fi
18 done
19 unset i
```

第 3~9 行就是设置 PS1 环境变量的值，我们可以直接修改这部分代码，但是不建议大家这么做，因为我们及时修改正常了，但是后续如果重新编译 buildroot 并解压以后，/etc/profile 文件又会被重新替换掉，我们又得修改/etc/profile。

第 14~17 行，从这里可以看出，/etc/profile 文件执行的时候会遍历/etc/profile.d 目录下的所示.sh 脚本文件，然后执行这些.sh 脚本文件。所以我们可以在/etc/profile.d 目录下创建一个自定义的.sh 脚本文件，然后在此脚本文件里面添加 PS1 初始化代码就行了，这样即使后面重新编译了 buildroot 也不用担心此.sh 脚本会被替换掉。

在/etc/profile.d 目录下新建一个名为“myprofile.sh”的 shell 脚本文件，并且给予此文件可执行权限，命令如下：

```
cd /etc/profile.d/                  //进入/etc/profile.d 目录
touch myprofile.sh                  //创建 myprofile.sh 文件
chmod 777 myprofile.sh              //给予 myprofile.sh 可执行权限
```

最后在 myprofile.sh 里面添加如下所示内容：

示例代码 19.5.5.2 /etc/profile 添加的内容

```
1 #!/bin/sh
2
3 PS1='[\u@\h]:\w$ '
4 export PS1
```

重点是第 3 行，也就是设置 PS1 环境变量，格式就是：

```
[user@hostname]:currentpath$:
```

user：用户名。

hostname：主机名。

currentpath：当前所处目录绝对路径。

设置好以后的 myprofile.sh 文件内容如图 19.5.5.3 所示：

```
2. COM33 (USB-SERIAL CH340 (COM
#!/bin/sh

PS1='[\u@\h]:\w$ '
export PS1
~
```

图 19.5.5.3 修改后的 myprofile.sh 文件

myprofile.sh 文件创建完成以后重启开发板，这个时候我们就如到某个目录的时候命令行就会有提示，如图 19.5.5.4 所示：

```
[root@ATK-stm32mp1]:~$ cd /etc/
[root@ATK-stm32mp1]:/etc$
[root@ATK-stm32mp1]:/etc$
[root@ATK-stm32mp1]:/etc$
```

命令提示符

图 19.5.5.4 命令提示符

从图 19.5.4 可以看出，命令提示符显示正常了，完整的显示除了用户名、主机名和当前路径。至此，buildroot 构建根文件系统就已经全部完成了，当然了，很多第三方软件本章并没有使能，大家可以自行根据实际需求选择对应的第三方软件和库。

19.5.6 使能 sysfs debug 目录

后续调试驱动的时候我们可能要用到/sys/kernel/debug 目录，默认我们没有挂载 debugfs 文件系统，所以/sys/kernel/debug 目录下没有任何文件，挂载方法很简单。在 19.5.4 小节中创建的 Sautorun 文件中添加如下代码：

```
mount -t debugfs none /sys/kernel/debug
```

如图 19.5.6.1 所示：

```
3. COM6 (USB-SERIAL CH340 (COM
#/bin/sh

mount -t debugfs none /sys/kernel/debug
```

图 19.5.6.1 挂载 debugfs

上述命令是挂载/sys/kernel/debug 目录，文件系统为 debugfs。修改完成以后重启开发板，进入到/sys/kernel/debug 目录，此时此目录下就有很多子目录，如图 19.5.6.2 所示：

```
[root@ATK-stm32mp1]:/sys/kernel/debug$ ls
49000000.usb-otg    fault_around_bytes  pwm
asoc                gpio                ras
bdi                 hid                 regmap
block               ieee80211           regulator
cec                 iio                 remoteproc
clear_warn_once     memblock            sleep_time
clk                 memcg_slabinfo      stmmaceth
device_component    mmc0                suspend_stats
devices_deferred    mmc1                tracing
dma_buf             mmc2                ubi
dmaengine           mtd                 ubifs
dri                 opp                 usb
dynamic_debug       pinctrl             wakeup_sources
edt_ft5x06          pm_genpd
extfrag             pm_qos
```

图 19.5.6.2 debug 目录

从图 19.5.6.2 可以看出，debug 目录下有很多子目录和文件，分别对应不同的设备。

19.6 烧写根文件系统到 EMMC 中

至此，一个最基本的 buildroot 根文件系统就制作好了，我们可以将其打包烧写到开发板中。虽然编译 buildroot 的时候已经生成了 rootfs.ext4 格式的根文件系统，但是！我们实际上还在自行创建了一些文件，因此 buildroot 编译出来的 rootfs.ext4 不实用，所以还是需要我们对上面测试过的根文件系统打包，然后再烧写。

19.6.1 根文件系统打包

首先对/home/zuozhongkai/linux/nfs/rootfs 目录下的根文件系统打包，打包方法和 18.6.1 小节一模一样，大家参考 18.6.1 小节打包即可，最终得到一个名为“rootfs.ext4”的根文件系统包。

19.6.2 烧写到 EMMC

打包完成以后就可以使用 STM32CubeProgrammer 软件烧写到开发板 EMMC 中，烧写方法在 18.6.2 小节详细讲解了。

19.6.3 EMMC 启动测试

烧写完成以后就可以从 EMMC 启动测试了，需要设置 uboot 下面的 bootcmd 和 bootargs 这两个环境变量，命令如下：

```
setenv bootcmd 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
setenv bootargs 'console=ttySTM0,115200 root=/dev/mmcblk1p3 rootwait rw'
saveenv
boot
```

启动以后如图 19.6.3.1 所示：

```
EXT4-fs (mmcblk1p3): re-mounted. Opts: (null)
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Starting mdev... OK
Initializing random number generator: OK
Saving random seed: OK
Starting network: OK
Starting sshd: OK
Starting vsftpd: OK

Welcome to alientek STM32MP157
ATK-stm32mp1 login: root
[root@ATK-stm32mp1]:~$
[root@ATK-stm32mp1]:~$
[root@ATK-stm32mp1]:~$
[root@ATK-stm32mp1]:~$
```

图 19.6.3.1 从 EMMC 启动

从 EMMC 启动的话默认是不开启网络的，需要我们自行开启，输入“ifconfig -a”命令查看当前开发板所有网卡信息，如图 19.6.3.2 所示：

```
[root@ATK-stm32mp1]:~$ ifconfig -a
eth0      Link encap:Ethernet  HWaddr 52:52:95:B2:0E:8E
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:45 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[root@ATK-stm32mp1]:~$
```

图 19.6.3.3 当前开发板所有网卡

从图 19.6.3.3 可以看出，当前开发板有两个网卡:eth0 和 lo，其中 eth0 就是开发板的千 M 有线网卡，lo 是回测用的。但是 eth0 这个网卡默认是没有启用的，大家输入“ifconfig”命令可以查看当前正在工作的网卡，如图 19.6.3.4 所示：

```
[root@ATK-stm32mp1]:~$ ifconfig
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[root@ATK-stm32mp1]:~$
```

图 19.6.3.4 当前系统启用的网卡

从图 19.6.3.4 可以看出，当前系统就只启用了 lo 网卡，eth0 网卡并没有启用，所以我们需要手动打开 eth0 网卡，输入如下命令：

```
ifconfig eth0 up                //打开 eth0 网卡
```

输入上述命令以后就会打开 eth0 网卡，如图 19.6.3.5 所示：

```
[root@ATK-stm32mp1]:~$ ifconfig eth0 up
stm32-dwmac 5800a000.ethernet eth0: PHY [stmmac-0:00] driver [RTL8211F Gigabit Ethernet]
dwmac4: Master AXI performs any burst length
stm32-dwmac 5800a000.ethernet eth0: No Safety Features support found
stm32-dwmac 5800a000.ethernet eth0: IEEE 1588-2008 Advanced Timestamp supported
stm32-dwmac 5800a000.ethernet eth0: registered PTP clock
stm32-dwmac 5800a000.ethernet eth0: configuring for phy/rgmii-id link mode
[root@ATK-stm32mp1]:~$ stm32-dwmac 5800a000.ethernet eth0: Link is Up - 1Gbps/Full - flow control rx/tx

[root@ATK-stm32mp1]:~$
```

图 19.6.3.5 eth0 网卡启动

从图 19.6.3.5 可以看出，eth0 网络连接成功，网速为 1Gbps。注意！如果你的开发板连接的路由器或交换机是百 M 的，那么网速可能为 100Mbps！

再次输入“ifconfig”命令就可以看到 eth0 网卡打开了，如图 19.6.3.6 所示：

```
[root@ATK-stm32mp1]:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 52:52:95:B2:0E:8E
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4439 errors:0 dropped:120 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:352663 (344.3 KiB)  TX bytes:0 (0.0 B)
          Interrupt:45 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[root@ATK-stm32mp1]:~$
```

图 19.6.3.6 eth0 网卡打开

从图 19.6.3.6 可以看出，此时 eth0 网卡已经打开了，但是此时开发板网络还不能使用，因为我们还没有配置 eth0 网卡地址信息，如果开发板没有连接路由器，而是直接连接的交换机或者电脑，那么只能手动设置 eth0 网卡地址信息，命令如下：

```
ifconfig eth0 192.168.1.250 netmask 255.255.255.0
```

如果需要开机自动设置静态 IP 地址，那么就可以在/etc/init.d/Sautorun 中添加如下两行：

```
ifconfig eth0 up                                    //打开 eth0
ifconfig eth0 192.168.1.250 netmask 255.255.255.0   //设置 IP 地址
```

修改完 Sautorun 文件以后执行一次 sync 命令，确保修改的内容写入到 Sautorun 文件里面，而不是写入到缓存中。最后重启开发板，此时系统启动的时候就会自动打开 eth0 网卡，并且设置静态 IP 地址。

如果开发板连接路由器的话就更简单了，不需要我们手动设置静态 IP 地址，直接使用“udhcpc”命令从路由器获取动态 IP 地址即可，如图 19.6.3.7 所示：

```
[root@ATK-stm32mp1]:/$ udhcpc
udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending select for 192.168.1.157
udhcpc: lease of 192.168.1.157 obtained, lease time 86400
deleting routers
adding dns 114.114.114.114
[root@ATK-stm32mp1]:/$
```

图 19.6.3.7 udhcpc 动态获取 IP 地址

从图 19.6.3.7 可以看出，开发板通过 udhcpc 命令获取到了 192.168.1.157 这个 IP 地址。

第三篇 ARM Linux 驱动开发篇

前面 2 篇，我们学习 Ubuntu 操作系统已经系统移植，其目的就是为了本篇做准备。本篇应该是大家最期待的内容了，毕竟大部分学习者的最终目的就是学习 Linux 驱动开发。本篇我们将会详细讲解 Linux 中的三大类驱动：字符设备驱动、块设备驱动和网络设备驱动。其中字符设备驱动是占用篇幅最大的一类驱动，因为字符设备最多，从最简单的点灯到 I2C、SPI、音频等都属于字符设备驱动的类型。块设备和网络设备驱动要比字符设备驱动复杂，就是因为其复杂所以半导体厂商一般都给我们编写好了，大多数情况下都是直接可以使用的。所谓的块设备驱动就是存储器设备的驱动，比如 EMMC、NAND、SD 卡和 U 盘等存储设备，因为这些存储设备的特点是以存储块为基础，因此叫做块设备。网络设备驱动就更好理解了，就是网络驱动，不管是有线的还是无线的，都属于网络设备驱动的范畴。一个设备可以属于多种设备驱动类型，比如 USB WIFI，其使用 USB 接口，所以属于字符设备，但是其又能上网，所以也属于网络设备驱动。本篇我们就围绕着三大设备驱动类型展开，尽可能详细的讲解每种设备驱动的开发方式。

本书使用的 Linux 内核版本为 5.4.31，其支持设备树(Device tree)，所以本篇所有例程均采用设备树。设备树将是本篇的重点！从设备树的基本原理到设备树驱动的开发方式，从最简单的点灯到复杂的网络驱动开发，本篇均有详细的讲解，是学习设备树的不二之选。

第二十章 字符设备驱动开发

本章我们从 Linux 驱动开发中最基础的字符设备驱动开始，重点学习 Linux 下字符设备驱动开发框架。本章会以一个虚拟的设备为例，讲解如何进行字符设备驱动开发，以及如何编写测试 APP 来测试驱动工作是否正常，为以后的学习打下坚实的基础。

20.1 字符设备驱动简介

字符设备是 Linux 驱动中最基本的一类设备驱动，字符设备就是一个一个字节，按照字节流进行读写操作的设备，读写数据是分先后顺序的。比如我们最常见的点灯、按键、IIC、SPI，LCD 等等都是字符设备，这些设备的驱动就叫做字符设备驱动。

在详细的学习字符设备驱动架构之前，我们先来简单的了解一下 Linux 下的应用程序是如何调用驱动程序的，Linux 应用程序对驱动程序的调用如图 20.1.1 所示：

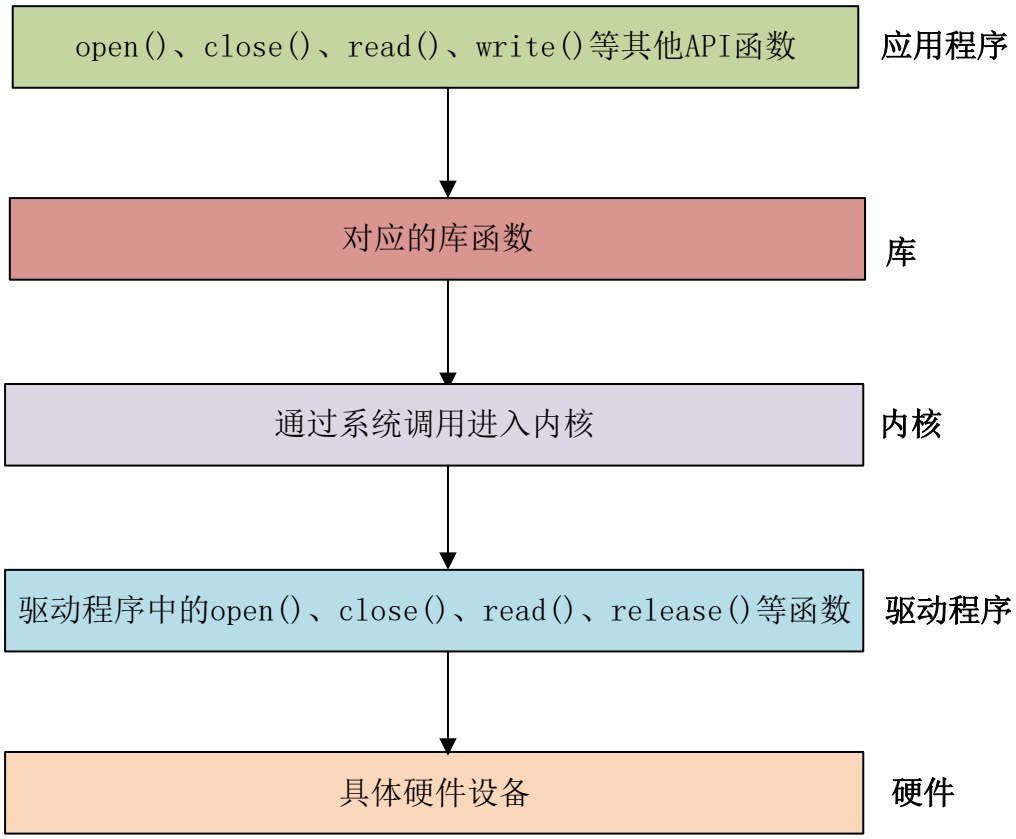


图 20.1.1 Linux 应用程序对驱动程序的调用流程

在 Linux 中一切皆为文件，驱动加载成功以后会在“/dev”目录下生成一个相应的文件，应用程序通过对这个名为“/dev/xxx”(xxx 是具体的驱动文件名字)的文件进行相应的操作即可实现对硬件的操作。比如现在有个叫做/dev/led 的驱动文件，此文件是 led 灯的驱动文件。应用程序使用 open 函数来打开文件/dev/led，使用完成以后使用 close 函数关闭/dev/led 这个文件。open 和 close 就是打开和关闭 led 驱动的函数，如果要点亮或关闭 led，那么就使用 write 函数来操作，也就是向此驱动写入数据，这个数据就是要关闭还是要打开 led 的控制参数。如果要获取 led 灯的状态，就用 read 函数从驱动中读取相应的状态。

应用程序运行在用户空间，而 Linux 驱动属于内核的一部分，因此驱动运行于内核空间。当我们在用户空间想要实现对内核的操作，比如使用 open 函数打开/dev/led 这个驱动，因为用户空间不能直接对内核进行操作，因此必须使用一个叫做“系统调用”的方法来实现从用户空间“陷入”到内核空间，这样才能实现对底层驱动的操作。open、close、write 和 read 等这些函数是由 C 库提供的，在 Linux 系统中，系统调用作为 C 库的一部分。当我们调用 open 函数的时候流程如图 20.1.2 所示：

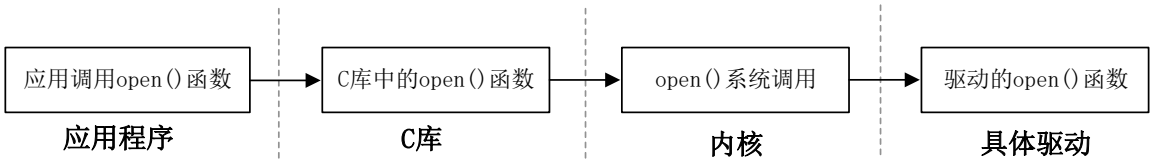


图 20.1.2 open 函数调用流程

其中关于 C 库以及如何通过系统调用“陷入”到内核空间这个我们不用去管，我们重点关注的是应用程序和具体的驱动，应用程序使用到的函数在具体驱动程序中都有与之对应的函数，比如应用程序中调用了 open 这个函数，那么在驱动程序中也得有一个名为 open 的函数。每一个系统调用，在驱动中都有与之对应的一个驱动函数，在 Linux 内核文件 include/linux/fs.h 中有个叫做 file_operations 的结构体，此结构体就是 Linux 内核驱动操作函数集合，内容如下所示：

示例代码 20.1.1 file_operations 结构体

```
1822  struct file_operations {
1823      struct module *owner;
1824      loff_t (*llseek) (struct file *, loff_t, int);
1825      ssize_t (*read) (struct file *, char __user *, size_t,
                          loff_t *);
1826      ssize_t (*write) (struct file *, const char __user *,
                          size_t, loff_t *);
1827      ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
1828      ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
1829      int (*iopoll)(struct kiocb *kiocb, bool spin);
1830      int (*iterate) (struct file *, struct dir_context *);
1831      int (*iterate_shared) (struct file *, struct dir_context *);
1832      __poll_t (*poll) (struct file *, struct poll_table_struct *);
1833      long (*unlocked_ioctl) (struct file *, unsigned int,
                                unsigned long);
1834      long (*compat_ioctl) (struct file *, unsigned int,
                               unsigned long);
1835      int (*mmap) (struct file *, struct vm_area_struct *);
1836      unsigned long mmap_supported_flags;
1837      int (*open) (struct inode *, struct file *);
1838      int (*flush) (struct file *, fl_owner_t id);
1839      int (*release) (struct inode *, struct file *);
1840      int (*fsync) (struct file *, loff_t, loff_t, int datasync);
1841      int (*fasync) (int, struct file *, int);
1842      int (*lock) (struct file *, int, struct file_lock *);
1843      ssize_t (*sendpage) (struct file *, struct page *,
                             int, size_t, loff_t *, int);
1844      unsigned long (*get_unmapped_area)(struct file *,
           unsigned long, unsigned long, unsigned long, unsigned long);
1845      int (*check_flags)(int);
1846      int (*flock) (struct file *, int, struct file_lock *);
```

```
1847        ssize_t (*splice_write)(struct pipe_inode_info *,
                    struct file *, loff_t *, size_t, unsigned int);
1848        ssize_t (*splice_read)(struct file *, loff_t *,
                    struct pipe_inode_info *, size_t, unsigned int);
1849        int (*setlease)(struct file *, long, struct file_lock **,
                        void **);
1850        long (*fallocate)(struct file *file, int mode, loff_t offset,
1851                    loff_t len);
1852        void (*show_fdinfo)(struct seq_file *m, struct file *f);
1853    #ifndef CONFIG_MMU
1854        unsigned (*mmap_capabilities)(struct file *);
1855    #endif
1856        ssize_t (*copy_file_range)(struct file *, loff_t,
1857                struct file *, loff_t, size_t, unsigned int);
1858        loff_t (*remap_file_range)(struct file *file_in,
                                    loff_t pos_in,
1859                    struct file *file_out, loff_t pos_out,
1860                    loff_t len, unsigned int remap_flags);
1861        int (*fadvise)(struct file *, loff_t, loff_t, int);
1862    } __randomize_layout;
```

简单介绍一下 file_operation 结构体中比较重要的、常用的函数：

第 1823 行，owner 拥有该结构体的模块的指针，一般设置为 THIS_MODULE。

第 1824 行，llseek 函数用于修改文件当前的读写位置。

第 1825 行，read 函数用于读取设备文件。

第 1826 行，write 函数用于向设备文件写入(发送)数据。

第 1832 行，poll 是个轮询函数，用于查询设备是否可以进行非阻塞的读写。

第 1833 行，unlocked_ioctl 函数提供对于设备的控制功能，与应用程序中的 ioctl 函数对应。

第 1834 行，compat_ioctl 函数与 unlocked_ioctl 函数功能一样，区别在于在 64 位系统上，32 位的应用程序调用将会使用此函数。在 32 位的系统上运行 32 位的应用程序调用的是 unlocked_ioctl。

第 1835 行，mmap 函数用于将将设备的内存映射到进程空间中(也就是用户空间)，一般帧缓冲设备会使用此函数，比如 LCD 驱动的显存，将帧缓冲(LCD 显存)映射到用户空间中以后应用程序就可以直接操作显存了，这样就不用在用户空间和内核空间之间来回复制。

第 1837 行，open 函数用于打开设备文件。

第 1839 行，release 函数用于释放(关闭)设备文件，与应用程序中的 close 函数对应。

第 1841 行，fasync 函数用于刷新待处理的数据，用于将缓冲区中的数据刷新到磁盘中。

在字符设备驱动开发中最常用的就是上面这些函数，关于其他的函数大家可以查阅相关文档。我们在字符设备驱动开发中最主要的工作就是实现上面这些函数，不一定全部都要实现，但是像 open、release、write、read 等都是需要实现的，当然了，具体需要实现哪些函数还是要看具体的驱动要求。

20.2 字符设备驱动开发步骤

上一小节我们简单的介绍了一下字符设备驱动，那么字符设备驱动开发都有哪些步骤呢？

我们在学习裸机或者 STM32 的时候关于驱动的开发就是初始化相应的外设寄存器，在 Linux 驱动开发中肯定也是要初始化相应的外设寄存器，这个是毫无疑问的。只是在 Linux 驱动开发中我们需要按照其规定的框架来编写驱动，所以说学 Linux 驱动开发重点是学习其驱动框架。

20.2.1 驱动模块的加载和卸载

Linux 驱动有两种运行方式，第一种就是将驱动编译进 Linux 内核中，这样当 Linux 内核启动的时候就会自动运行驱动程序。第二种就是将驱动编译成模块(Linux 下模块扩展名为.ko)，在 Linux 内核启动以后使用“modprobe”或者“insmod”命令加载驱动模块，本教程我们统一使用“modprobe”命令。在调试驱动的时候一般都选择将其编译为模块，这样我们修改驱动以后只需要编译一下驱动代码即可，不需要编译整个 Linux 代码。而且在调试的时候只需要加载或者卸载驱动模块即可，不需要重启整个系统。总之，将驱动编译为模块最大的好处就是方便开发，当驱动开发完成，确定没有问题以后就可以将驱动编译进 Linux 内核中，当然也可以不编译进 Linux 内核中，具体看自己的需求。

模块有加载和卸载两种操作，我们在编写驱动的时候需要注册这两种操作函数，模块的加载和卸载注册函数如下：

```
module_init(xxx_init);      //注册模块加载函数
module_exit(xxx_exit);      //注册模块卸载函数
```

module_init 函数用来向 Linux 内核注册一个模块加载函数，参数 xxx_init 就是需要注册的具体函数，当使用“modprobe”命令加载驱动的时候，xxx_init 这个函数就会被调用。module_exit 函数用来向 Linux 内核注册一个模块卸载函数，参数 xxx_exit 就是需要注销的具体函数，当使用“rmmod”命令卸载具体驱动的时候 xxx_exit 函数就会被调用。字符设备驱动模块加载和卸载模板如下所示：

示例代码 20.2.1.1 字符设备驱动模块加载和卸载函数模板

```
1  /* 驱动入口函数 */
2  static int __init xxx_init(void)
3  {
4      /* 入口函数具体内容 */
5      return 0;
6  }
7
8  /* 驱动出口函数 */
9  static void __exit xxx_exit(void)
10 {
11     /* 出口函数具体内容 */
12 }
13
14 /* 将上面两个函数指定为驱动的入口和出口函数 */
15 module_init(xxx_init);
16 module_exit(xxx_exit);
```

第 2 行，定义了个名为 xxx_init 的驱动入口函数，并且使用了“__init”来修饰。

第 9 行，定义了个名为 xxx_exit 的驱动出口函数，并且使用了“__exit”来修饰。

第 15 行，调用函数 module_init 来声明 xxx_init 为驱动入口函数，当加载驱动的时候 xxx_init 函数就会被调用。

第 16 行，调用函数 module_exit 来声明 xxx_exit 为驱动出口函数，当卸载驱动的时候 xxx_exit 函数就会被调用。

驱动编译完成以后扩展名为.ko，前面说了，有两种命令可以加载驱动模块：insmod 和 modprobe，insmod 是最简单的模块加载命令，此命令用于加载指定的.ko 模块，比如加载 drv.ko 这个驱动模块，命令如下：

```
insmod drv.ko
```

insmod 命令不能解决模块的依赖关系，比如 drv.ko 依赖 first.ko 这个模块，就必须先使用 insmod 命令加载 first.ko 这个模块，然后再加载 drv.ko 这个模块。但是 modprobe 就不会存在这个问题，modprobe 会分析模块的依赖关系，然后会将所有的依赖模块都加载到内核中，因此 modprobe 命令相比 insmod 要智能一些。modprobe 命令主要智能在提供了模块的依赖性分析、错误检查、错误报告等功能，推荐使用 modprobe 命令来加载驱动。modprobe 命令默认会去/lib/modules/<kernel-version>目录中查找模块，比如本书使用的 Linux kernel 的版本号为 5.4.31，因此 modprobe 命令默认会到/lib/modules/5.4.31 这个目录中查找相应的驱动模块，一般自己制作的根文件系统中是不会有这个目录的，所以需要自己手动创建，这个我们在讲解 buildroot 构建根文件系统的时候已经说过了。

驱动模块的卸载使用命令“rmmod”即可，比如要卸载 drv.ko，使用如下命令即可：

```
rmmod drv.ko
```

也可以使用“modprobe -r”命令卸载驱动，比如要卸载 drv.ko，命令如下：

```
modprobe -r drv
```

使用 modprobe 命令可以卸载掉驱动模块所依赖的其他模块，前提是这些依赖模块已经没有被其他模块所使用，否则就不能使用 modprobe 来卸载驱动模块。所以对于模块的卸载，还是推荐使用 rmmod 命令。

20.2.2 字符设备注册与注销

对于字符设备驱动而言，当驱动模块加载成功以后需要注册字符设备，同样，卸载驱动模块的时候也需要注销掉字符设备。字符设备的注册和注销函数原型如下所示:

```
static inline int register_chrdev(unsigned int                major,
                                  const char                  *name,
                                  const struct file_operations *fops)

static inline void unregister_chrdev(unsigned int  major,
                                     const char    *name)
```

register_chrdev 函数用于注册字符设备，此函数一共有三个参数，这三个参数的含义如下：

major：主设备号，Linux 下每个设备都有一个设备号，设备号分为主设备号和次设备号两部分，关于设备号后面会详细讲解。

name：设备名字，指向一串字符串。

fops：结构体 file_operations 类型指针，指向设备的操作函数集合变量。

unregister_chrdev 函数用户注销字符设备，此函数有两个参数，这两个参数含义如下：

major：要注销的设备对应的主设备号。

name：要注销的设备对应的设备名。

一般字符设备的注册在驱动模块的入口函数 xxx_init 中进行，字符设备的注销在驱动模块的出口函数 xxx_exit 中进行。在示例代码 20.2.1.1 中字符设备的注册和注销，内容如下所示：

示例代码 20.2.2.1 加入字符设备注册和注销

```
1  static struct file_operations test_fops;
2
3  /* 驱动入口函数 */
4  static int __init xxx_init(void)
5  {
6      /* 入口函数具体内容 */
7      int retvalue = 0;
8
9      /* 注册字符设备驱动 */
10     retvalue = register_chrdev(200, "chrtest", &test_fops);
11     if(retvalue < 0){
12         /* 字符设备注册失败,自行处理 */
13     }
14     return 0;
15 }
16
17 /* 驱动出口函数 */
18 static void __exit xxx_exit(void)
19 {
20     /* 注销字符设备驱动 */
21     unregister_chrdev(200, "chrtest");
22 }
23
24 /* 将上面两个函数指定为驱动的入口和出口函数 */
25 module_init(xxx_init);
26 module_exit(xxx_exit);
```

第 1 行，定义了一个 file_operations 结构体变量 test_fops，test_fops 就是设备的操作函数集合，只是此时我们还没有初始化 test_fops 中的 open、release 等这些成员变量，所以这个操作函数集合还是空的。

第 10 行，调用函数 register_chrdev 注册字符设备，主设备号为 200，设备名字为“chrtest”，设备操作函数集合就是第 1 行定义的 test_fops。要注意的一点就是，选择没有被使用的主设备号，输入命令“cat /proc/devices”可以查看当前已经被使用掉的设备号，如图 20.2.2.1 所示(限于篇幅原因，只展示一部分)：

```
[root@ATK-stm32mp1]:/$ cat /proc/devices
Character devices:
  1 mem
  2 pty
  3 ttyp
  4 /dev/vc/0
  4 tty
  5 /dev/tty
  5 /dev/console
  5 /dev/ptmx
  5 ttyRPMSG
```

图 20.2.2.1 查看当前设备

在图 20.2.2.1 中可以列出当前系统中所有的字符设备和块设备，其中第 1 列就是设备对应的主设备号。200 这个主设备号在我的开发板中并没有被使用，所以我这里就用了 200 这个主设备号。

第 21 行，调用函数 unregister_chrdev 注销主设备号为 200 的这个设备。

20.2.3 实现设备的具体操作函数

file_operations 结构体就是设备的具体操作函数，在示例代码 20.2.2.1 中我们定义了 file_operations 结构体类型的变量 test_fops，但是还没对其进行初始化，也就是初始化其中的 open、release、read 和 write 等具体的设备操作函数。本节我们就完成变量 test_fops 的初始化，设置好针对 chrtest 设备的操作函数。在初始化 test_fops 之前我们要分析一下需求，也就是要对 chrtest 这个设备进行哪些操作，只有确定了需求以后才知道我们应该实现哪些操作函数。假设对 chrtest 这个设备有如下两个要求：

1、能够对 chrtest 进行打开和关闭操作

设备打开和关闭是最基本的要求，几乎所有的设备都得提供打开和关闭的功能。因此我们需要实现 file_operations 中的 open 和 release 这两个函数。

2、对 chrtest 进行读写操作

假设 chrtest 这个设备控制着一段缓冲区(内存)，应用程序需要通过 read 和 write 这两个函数对 chrtest 的缓冲区进行读写操作。所以需要实现 file_operations 中的 read 和 write 这两个函数。

需求很清晰了，修改示例代码 20.2.2.1，在其中加入 test_fops 这个结构体变量的初始化操作，完成以后的内容如下所示：

示例代码 20.2.3.1 加入设备操作函数

```
1  /* 打开设备 */
2  static int chrtest_open(struct inode *inode, struct file *filp)
3  {
4      /* 用户实现具体功能 */
5      return 0;
6  }
7
8  /* 从设备读取 */
9  static ssize_t chrtest_read(struct file *filp, char __user *buf,
                               size_t cnt, loff_t *offt)
10 {
11     /* 用户实现具体功能 */
12     return 0;
13 }
14
15 /* 向设备写数据 */
16 static ssize_t chrtest_write(struct file *filp,
                               const char __user *buf,
                               size_t cnt, loff_t *offt)
17 {
```

```
18      /* 用户实现具体功能 */
19      return 0;
20  }
21
22  /* 关闭/释放设备 */
23  static int chrtest_release(struct inode *inode, struct file *filp)
24  {
25      /* 用户实现具体功能 */
26      return 0;
27  }
28
29  static struct file_operations test_fops = {
30      .owner = THIS_MODULE,
31      .open = chrtest_open,
32      .read = chrtest_read,
33      .write = chrtest_write,
34      .release = chrtest_release,
35  };
36
37  /* 驱动入口函数 */
38  static int __init xxx_init(void)
39  {
40      /* 入口函数具体内容 */
41      int retvalue = 0;
42
43      /* 注册字符设备驱动 */
44      retvalue = register_chrdev(200, "chrtest", &test_fops);
45      if(retvalue < 0){
46          /* 字符设备注册失败,自行处理 */
47      }
48      return 0;
49  }
50
51  /* 驱动出口函数 */
52  static void __exit xxx_exit(void)
53  {
54      /* 注销字符设备驱动 */
55      unregister_chrdev(200, "chrtest");
56  }
57
58  /* 将上面两个函数指定为驱动的入口和出口函数 */
59  module_init(xxx_init);
60  module_exit(xxx_exit);
```

在示例代码 20.2.3.1 中我们一开始编写了四个函数：chrtest_open、chrtest_read、chrtest_write 和 chrtest_release。这四个函数就是 chrtest 设备的 open、read、write 和 release 操作函数。第 29 行~35 行初始化 test_fops 的 open、read、write 和 release 这四个成员变量。

20.2.4 添加 LICENSE 和作者信息

最后我们需要在驱动中加入 LICENSE 信息和作者信息，其中 LICENSE 是必须添加的，否则的话编译的时候会报错，作者信息可以添加也可以不添加。LICENSE 和作者信息的添加使用如下两个函数：

```
MODULE_LICENSE()      //添加模块 LICENSE 信息
MODULE_AUTHOR()       //添加模块作者信息
```

最后给示例代码 20.2.3.1 加入 LICENSE 和作者信息，完成以后的内容如下：

示例代码 20.2.4.1 字符设备驱动最终的模板

```
1  /* 打开设备 */
2  static int chrtest_open(struct inode *inode, struct file *filp)
3  {
4      /* 用户实现具体功能 */
5      return 0;
6  }
......
57
58 /* 将上面两个函数指定为驱动的入口和出口函数 */
59 module_init(xxx_init);
60 module_exit(xxx_exit);
61
62 MODULE_LICENSE("GPL");
63 MODULE_AUTHOR("zuozhongkai");
```

第 62 行，LICENSE 采用 GPL 协议。

第 63 行，添加作者名字。

至此，字符设备驱动开发的完整步骤就讲解完了，而且也编写好了一个完整的字符设备驱动模板，以后字符设备驱动开发都可以在此模板上进行。

20.3 Linux 设备号

20.3.1 设备号的组成

为了方便管理，Linux 中每个设备都有一个设备号，设备号由主设备号和次设备号两部分组成，主设备号表示某一个具体的驱动，次设备号表示使用这个驱动的各个设备。Linux 提供了一个名为 dev_t 的数据类型表示设备号，dev_t 定义在文件 include/linux/types.h 里面，定义如下：

示例代码 20.3.1.1 设备号 dev_t

```
13 typedef __u32 __kernel_dev_t;
......
16 typedef __kernel_dev_t dev_t;
```

可以看出 dev_t 是__u32 类型的，而__u32 定义在文件 include/uapi/asm-generic/int-ll64.h 里面，定义如下：

示例代码 20.3.1.2 __u32 类型

```
27 typedef unsigned int __u32;
```

综上所述，dev_t 其实就是 unsigned int 类型，是一个 32 位的数据类型。这 32 位的数据构成了主设备号和次设备号两部分，其中高 12 位为主设备号，低 20 位为次设备号。因此 Linux 系统中主设备号范围为 0~4095，所以大家在选择主设备号的时候一定不要超过这个范围。在文件 include/linux/kdev_t.h 中提供了几个关于设备号的操作函数(本质是宏)，如下所示：

示例代码 20.3.1.3 设备号操作函数

```
7  #define MINORBITS    20
8  #define MINORMASK    ((1U << MINORBITS) - 1)
9
10 #define MAJOR(dev)   ((unsigned int) ((dev) >> MINORBITS))
11 #define MINOR(dev)   ((unsigned int) ((dev) & MINORMASK))
12 #define MKDEV(ma,mi) (((ma) << MINORBITS) | (mi))
```

第 7 行，宏 MINORBITS 表示次设备号位数，一共是 20 位。

第 8 行，宏 MINORMASK 表示次设备号掩码。

第 10 行，宏 MAJOR 用于从 dev_t 中获取主设备号，将 dev_t 右移 20 位即可。

第 11 行，宏 MINOR 用于从 dev_t 中获取次设备号，取 dev_t 的低 20 位的值即可。

第 12 行，宏 MKDEV 用于将给定的主设备号和次设备号的值组合成 dev_t 类型的设备号。

20.3.2 设备号的分配

1、静态分配设备号

本小节讲的设备号分配主要是主设备号的分配。前面讲解字符设备驱动的时候说过了，注册字符设备的时候需要给设备指定一个设备号，这个设备号可以是驱动开发者静态指定的一个设备号，比如 200 这个主设备号。有一些常用的设备号已经被 Linux 内核开发者给分配掉了，具体分配的内容可以查看文档 Documentation/devices.txt。并不是说内核开发者已经分配掉的主设备号我们就不能用了，具体能不能用还得看我们的硬件平台运行过程中有没有使用这个主设备号，使用“cat /proc/devices”命令即可查看当前系统中所有已经使用了的设备号。

2、动态分配设备号

静态分配设备号需要我们检查当前系统中所有被使用了的设备号，然后挑选一个没有使用的。而且静态分配设备号很容易带来冲突问题，Linux 社区推荐使用动态分配设备号，在注册字符设备之前先申请一个设备号，系统会自动给你一个没有被使用的设备号，这样就避免了冲突。卸载驱动的时候释放掉这个设备号即可，设备号的申请函数如下：

```
int alloc_chrdev_region(dev_t *dev, unsigned baseminor, unsigned count, const char *name)
```

函数 alloc_chrdev_region 用于申请设备号，此函数有 4 个参数：

dev：保存申请到的设备号。

baseminor：次设备号起始地址，alloc_chrdev_region 可以申请一段连续的多个设备号，这些设备号的主设备号一样，但是次设备号不同，次设备号以 baseminor 为起始地址地址开始递增。一般 baseminor 为 0，也就是说次设备号从 0 开始。

count：要申请的设备号数量。

name：设备名字。

注销字符设备之后要释放掉设备号，设备号释放函数如下：

```
void unregister_chrdev_region(dev_t from, unsigned count)
```

此函数有两个参数：

from：要释放的设备号。

count：表示从 from 开始，要释放的设备号数量。

20.4 chrdevbase 字符设备驱动开发实验

字符设备驱动开发的基本步骤我们已经了解了，本节我们就以 chrdevbase 这个虚拟设备为例，完整的编写一个字符设备驱动模块。chrdevbase 不是实际存在的一个设备，是笔者为了方便讲解字符设备的开发而引入的一个虚拟设备。chrdevbase 设备有两个缓冲区，一个读缓冲区，一个写缓冲区，这两个缓冲区的大小都为 100 字节。在应用程序中可以向 chrdevbase 设备的写缓冲区中写入数据，从读缓冲区中读取数据。chrdevbase 这个虚拟设备的功能很简单，但是它包含了字符设备的最基本功能。

20.4.1 实验程序编写

本实验对应的例程路径为：开发板光盘-> 1、Linux 驱动例程-> 1_chrdevbase。

应用程序调用 open 函数打开 chrdevbase 这个设备，打开以后可以使用 write 函数向 chrdevbase 的写缓冲区 writebuf 中写入数据(不超过 100 个字节)，也可以使用 read 函数读取读缓冲区 readbuf 中的数据操作，操作完成以后应用程序使用 close 函数关闭 chrdevbase 设备。

1、创建 VSCode 工程

在 Ubuntu 中创建一个目录用来存放 Linux 驱动程序，比如我创建了一个名为 Linux_Drivers 的目录来存放所有的 Linux 驱动。在 Linux_Drivers 目录下新建一个名为 1_chrdevbase 的子目录来存放本实验所有文件，如图 20.4.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers$ ls
1_chrdevbase
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers$
```

图 20.4.1.1 Linux 实验程序目录

在 1_chrdevbase 目录中新建 VSCode 工程，并且新建 chrdevbase.c 文件，完成以后 1_chrdevbase 目录中的文件如图 20.4.1.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase$ ls -a
.  ..  1_chrdevbase.code-workspace  chrdevbase.c
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase$
```

图 20.4.1.2 1_chrdevbase 目录文件

2、添加头文件路径

因为是编写 Linux 驱动，因此会用到 Linux 源码中的函数。我们需要在 VSCode 中添加 Linux 源码中的头文件路径。打开 VSCode，按下“Crtl+Shift+P”打开 VSCode 的控制台，然后输入“C/C++: Edit configurations(JSON) ”，打开 C/C++编辑配置文件，如图 20.4.1.3 所示：

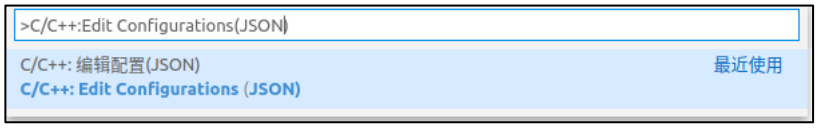


图 20.4.1.3 C/C++编辑配置文件

打开以后会自动在.vscode 目录下生成一个名为 c_cpp_properties.json 的文件，此文件默认内容如下所示：

示例代码 20.4.1.1 c_cpp_properties.json 文件原内容

```
1  {
```

```
2     "configurations": [
3         {
4             "name": "Linux",
5             "includePath": [
6                 "${workspaceFolder}/**",
7                 ],
8             "defines": [],
9             "compilerPath": "/usr/bin/clang",
10            "cStandard": "c11",
11            "cppStandard": "c++17",
12            "intelliSenseMode": "clang-x64"
13        }
14    ],
15    "version": 4
16 }
```

第 5 行的 includePath 表示头文件路径，需要将 Linux 源码里面的头文件路径添加进来，也就是我们前面移植的 Linux 源码中的头文件路径。添加头文件路径以后的 c_cpp_properties.json 的文件内容如下所示：

示例代码 20.4.1.2 添加头文件路径后的 c_cpp_properties.json

```
1  {
2     "configurations": [
3         {
4             "name": "Linux",
5             "includePath": [
6                 "${workspaceFolder}/**",
7                 "/home/zuozhongkai/linux/atk-mp1/linux/my_linux/
                   linux-5.4.31/arch/arm/include",
8                 "/home/zuozhongkai/linux/atk-mp1/linux/my_linux/
                   linux-5.4.31/include",
9                 "/home/zuozhongkai/linux/atk-mp1/linux/my_linux/
                   linux-5.4.31/arch/arm/include/generated"
10            ],
11            "defines": [],
12            "compilerPath": "/usr/bin/gcc",
13            "cStandard": "gnu11",
14            "cppStandard": "gnu++14",
15            "intelliSenseMode": "gcc-x64"
16        }
17    ],
18    "version": 4
19 }
```

第 7~9 行就是添加好的 Linux 头文件路径。分别是开发板所使用的 Linux 源码下的 include、arch/arm/include 和 arch/arm/include/generated 这三个目录的路径，注意，这里使用了绝对路径。

3、编写实验程序

工程建立好以后就可以开始编写驱动程序了，新建 chrdevbase.c，然后在里面输入如下内容：

示例代码 20.4.1.3 chrdevbase.c 文件

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  /***************************************************************
8  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
9  文件名   : chrdevbase.c
10 作者     : 正点原子
11 版本     : V1.0
12 描述     : chrdevbase 驱动文件。
13 其他     : 无
14 论坛     : www.openedv.com
15 日志     : 初版 V1.0 2020/12/26 正点原子团队创建
16 ***************************************************************/
17
18 #define CHRDEVBASE_MAJOR    200                 /* 主设备号     */
19 #define CHRDEVBASE_NAME     "chrdevbase"        /* 设备名       */
20
21 static char readbuf[100];                       /* 读缓冲区     */
22 static char writebuf[100];                      /* 写缓冲区     */
23 static char kerneldata[] = {"kernel data!"};
24
25 /*
26  * @description   : 打开设备
27  * @param - inode : 传递给驱动的 inode
28  * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
29  *                  一般在 open 的时候将 private_data 指向设备结构体。
30  * @return        : 0 成功;其他 失败
31  */
32 static int chrdevbase_open(struct inode *inode, struct file *filp)
33 {
34     //printk("chrdevbase open!\r\n");
35     return 0;
36 }
37
38 /*
39  * @description   : 从设备读取数据
40  * @param - filp  : 要打开的设备文件(文件描述符)
```

```
41   * @param - buf   : 返回给用户空间的数据缓冲区
42   * @param - cnt   : 要读取的数据长度
43   * @param - offt  : 相对于文件首地址的偏移
44   * @return        : 读取的字节数，如果为负值，表示读取失败
45   */
46  static ssize_t chrdevbase_read(struct file *filp, char __user *buf,
                                   size_t cnt, loff_t *offt)
47  {
48      int retvalue = 0;
49
50      /* 向用户空间发送数据 */
51      memcpy(readbuf, kerneldata, sizeof(kerneldata));
52      retvalue = copy_to_user(buf, readbuf, cnt);
53      if(retvalue == 0){
54          printk("kernel senddata ok!\r\n");
55      }else{
56          printk("kernel senddata failed!\r\n");
57      }
58
59      //printk("chrdevbase read!\r\n");
60      return 0;
61  }
62
63  /*
64   * @description   : 向设备写数据
65   * @param - filp  : 设备文件，表示打开的文件描述符
66   * @param - buf   : 要写给设备写入的数据
67   * @param - cnt   : 要写入的数据长度
68   * @param - offt  : 相对于文件首地址的偏移
69   * @return        : 写入的字节数，如果为负值，表示写入失败
70   */
71  static ssize_t chrdevbase_write(struct file *filp,
                                    const char __user *buf,
                                    size_t cnt, loff_t *offt)
72  {
73      int retvalue = 0;
74      /* 接收用户空间传递给内核的数据并且打印出来 */
75      retvalue = copy_from_user(writebuf, buf, cnt);
76      if(retvalue == 0){
77          printk("kernel recevdata:%s\r\n", writebuf);
78      }else{
79          printk("kernel recevdata failed!\r\n");
80      }
```

```
81
82      //printk("chrdevbase write!\r\n");
83      return 0;
84  }
85
86  /*
87   * @description   : 关闭/释放设备
88   * @param - filp  : 要关闭的设备文件(文件描述符)
89   * @return        : 0 成功;其他 失败
90   */
91  static int chrdevbase_release(struct inode *inode,
                                  struct file *filp)
92  {
93      //printk("chrdevbase release! \r\n");
94      return 0;
95  }
96
97  /*
98   * 设备操作函数结构体
99   */
100 static struct file_operations chrdevbase_fops = {
101     .owner = THIS_MODULE,
102     .open = chrdevbase_open,
103     .read = chrdevbase_read,
104     .write = chrdevbase_write,
105     .release = chrdevbase_release,
106 };
107
108 /*
109  * @description   : 驱动入口函数
110  * @param         : 无
111  * @return        : 0 成功;其他 失败
112  */
113 static int __init chrdevbase_init(void)
114 {
115     int retvalue = 0;
116
117     /* 注册字符设备驱动 */
118     retvalue = register_chrdev(CHRDEVBASE_MAJOR, CHRDEVBASE_NAME,
                                   &chrdevbase_fops);
119     if(retvalue < 0){
120         printk("chrdevbase driver register failed\r\n");
121     }
```

```
122     printk("chrdevbase_init()\r\n");
123     return 0;
124 }
125
126 /*
127  * @description   : 驱动出口函数
128  * @param         : 无
129  * @return        : 无
130  */
131 static void __exit chrdevbase_exit(void)
132 {
133     /* 注销字符设备驱动 */
134     unregister_chrdev(CHRDEVBASE_MAJOR, CHRDEVBASE_NAME);
135     printk("chrdevbase_exit()\r\n");
136 }
137
138 /*
139  * 将上面两个函数指定为驱动的入口和出口函数
140  */
141 module_init(chrdevbase_init);
142 module_exit(chrdevbase_exit);
143
144 /*
145  * LICENSE 和作者信息
146  */
147 MODULE_LICENSE("GPL");
148 MODULE_AUTHOR("ALIENTEK");
149 MODULE_INFO(intree, "Y");
```

第 32~36 行，chrdevbase_open 函数，当应用程序调用 open 函数的时候此函数就会调用。本例程中我们没有做任何工作，只是输出一串字符，用于调试。这里使用了 printk 来输出信息，而不是 printf！因为在 Linux 内核中没有 printf 这个函数。printk 相当于 printf 的孪生兄妹，printf 运行在用户态，printk 运行在内核态。在内核中想要向控制台输出或显示一些内容，必须使用 printk 这个函数。不同之处在于，printk 可以根据日志级别对消息进行分类，一共有 8 个消息级别，这 8 个消息级别定义在文件 include/linux/kern_levels.h 里面，定义如下：

```
#define KERN_SOH        "\001"
#define KERN_EMERG      KERN_SOH "0"    /* 紧急事件，一般是内核崩溃          */
#define KERN_ALERT      KERN_SOH "1"    /* 必须立即采取行动                  */
#define KERN_CRIT       KERN_SOH "2"    /* 临界条件，比如严重的软件或硬件错误*/
#define KERN_ERR        KERN_SOH "3"    /* 错误状态，一般设备驱动程序中使用
                                           KERN_ERR 报告硬件错误             */
#define KERN_WARNING    KERN_SOH "4"    /* 警告信息，不会对系统造成严重影响  */
#define KERN_NOTICE     KERN_SOH "5"    /* 有必要进行提示的一些信息          */
#define KERN_INFO       KERN_SOH "6"    /* 提示性的信息                      */
```

```
#define KERN_DEBUG      KERN_SOH "7"    /* 调试信息                    */
```

一共定义了 8 个级别，其中 0 的优先级最高，7 的优先级最低。如果要设置消息级别，参考如下示例：

```
printk(KERN_EMERG "gsmi: Log Shutdown Reason\n");
```

上述代码就是设置“gsmi: Log Shutdown Reason\n”这行消息的级别为 KERN_EMERG。在具体的消息前面加上 KERN_EMERG 就可以将这条消息的级别设置为 KERN_EMERG。如果使用 printk 的时候不显式的设置消息级别，那么 printk 将会采用默认级别 MESSAGE_LOGLEVEL_DEFAULT。

在 include/linux/printk.h 中有个宏 CONSOLE_LOGLEVEL_DEFAULT，定义如下：

```
#define CONSOLE_LOGLEVEL_DEFAULT    CONFIG_CONSOLE_LOGLEVEL_DEFAULT
```

MESSAGE_LOGLEVEL_DEFAULT 和 CONFIG_CONSOLE_LOGLEVEL_DEFAULT 是通过内核图形化界面配置的，配置路径如下：

```
-> Kernel hacking
    -> printk and dmesg options
        -> (7) Default console loglevel (1-15)          //设置默认终端消息级别
        -> (4) Default message log level (1-7)          //设置默认消息级别
```

“Default console loglevel”就是用来设置 CONSOLE_LOGLEVEL_DEFAULT 的值，“Default message log level”设置 CONFIG_MESSAGE_LOGLEVEL_DEFAULT 的值。默认如图 20.4.1.1 所示：

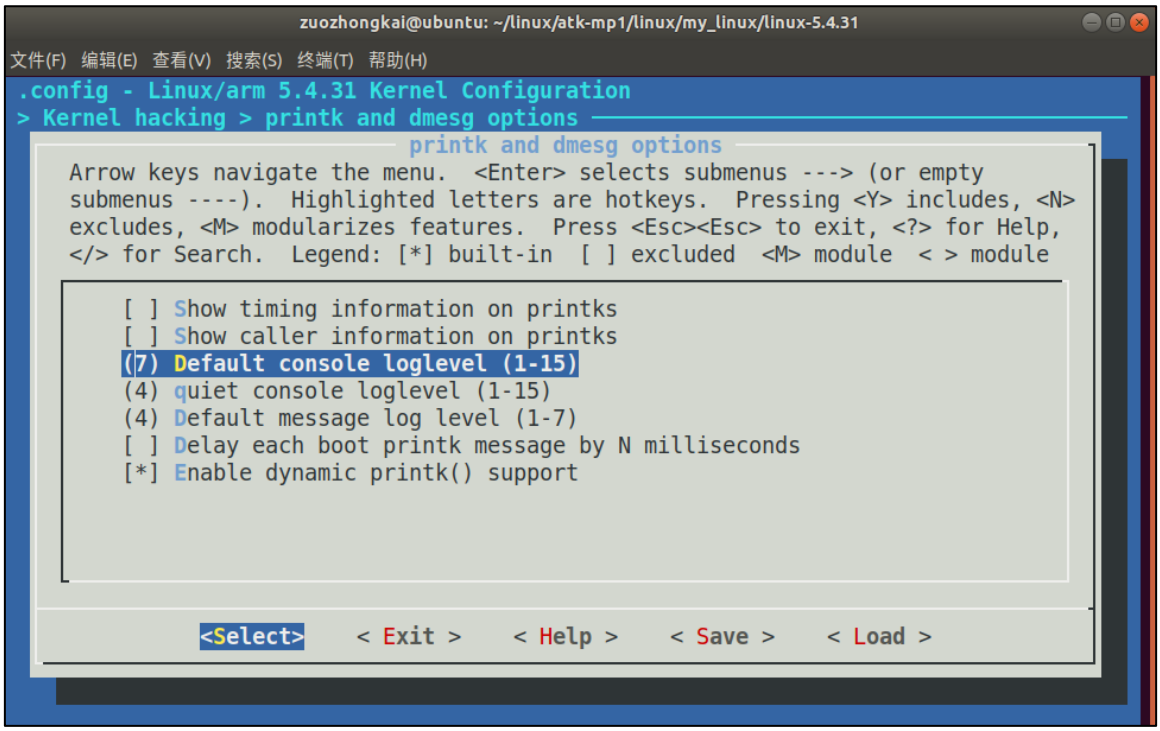


图 20.4.1.1 内核消息配置

图 20.4.1.1 可以看出，默认为 CONSOLE_LOGLEVEL_DEFAULT 默认为 7。所以红 CONSOLE_LOGLEVEL_DEFAULT 的值默认也为 7，MESSAGE_LOGLEVEL_DEFAULT 默认值为 4。CONSOLE_LOGLEVEL_DEFAULT 控制着哪些级别的消息可以显示在控制台上，此宏默认为 7，意味着只有优先级高于 7 的消息才能显示在控制台上。

这个就是 printk 和 printf 的最大区别，可以通过消息级别来决定哪些消息可以显示在控制台上。默认消息级别为 4，4 的级别比 7 高，所示直接使用 printk 输出的信息是可以显示在控制台上的。

参数 filp 有个叫做 private_data 的成员变量，private_data 是个 void 指针，一般在驱动中将 private_data 指向设备结构体，设备结构体会存放设备的一些属性。

第 46~61 行，chrdevbase_read 函数，应用程序调用 read 函数从设备中读取数据的时候此函数会执行。参数 buf 是用户空间的内存，读取到的数据存储在 buf 中，参数 cnt 是要读取的字节数，参数 offt 是相对于文件首地址的偏移。kerneldata 里面保存着用户空间要读取的数据，第 51 行先将 kerneldata 数组中的数据拷贝到读缓冲区 readbuf 中，第 52 行通过函数 copy_to_user 将 readbuf 中的数据复制到参数 buf 中。因为内核空间不能直接操作用户空间的内存，因此需要借助 copy_to_user 函数来完成内核空间的数据到用户空间的复制。copy_to_user 函数原型如下：

```
static inline long copy_to_user(void __user *to, const void *from, unsigned long n)
```

参数 to 表示目的，参数 from 表示源，参数 n 表示要复制的数据长度。如果复制成功，返回值为 0，如果复制失败则返回负数。

第 71~84 行，chrdevbase_write 函数，应用程序调用 write 函数向设备写数据的时候此函数就会执行。参数 buf 就是应用程序要写入设备的数据，也是用户空间的内存，参数 cnt 是要写入的数据长度，参数 offt 是相对文件首地址的偏移。第 75 行通过函数 copy_from_user 将 buf 中的数据复制到写缓冲区 writebuf 中，因为用户空间内存不能直接访问内核空间的内存，所以需要借助函数 copy_from_user 将用户空间的数据复制到 writebuf 这个内核空间中。

第 91~95 行，chrdevbase_release 函数，应用程序调用 close 关闭设备文件的时候此函数会执行，一般会在此函数里面执行一些释放操作。如果在 open 函数中设置了 filp 的 private_data 成员变量指向设备结构体，那么在 release 函数最终就要释放掉。

第 100~106 行，新建 chrdevbase 的设备文件操作结构体 chrdevbase_fops，初始化 chrdevbase_fops。

第 113~124 行，驱动入口函数 chrdevbase_init，第 118 行调用函数 register_chrdev 来注册字符设备。

第 131~136 行，驱动出口函数 chrdevbase_exit，第 134 行调用函数 unregister_chrdev 来注销字符设备。

第 141~142 行，通过 module_init 和 module_exit 这两个函数来指定驱动的入口和出口函数。

第 147~148 行，添加 LICENSE 和作者信息。

第 149 行是为了欺骗内核，给本驱动添加 intree 标记，如果不加就会有“loading out-of-tree module taints kernel.”这个警告

20.4.2 编写测试 APP

1、C 库文件操作基本函数

编写测试 APP 就是编写 Linux 应用，需要用到 C 库里面和文件操作有关的一些函数，比如 open、read、write 和 close 这四个函数。

①、open 函数

open 函数原型如下：

```
int open(const char *pathname, int flags)
```

open 函数参数含义如下：

pathname：要打开的设备或者文件名。

flags：文件打开模式，以下三种模式必选其一：

O_RDONLY 只读模式
O_WRONLY 只写模式
O_RDWR 读写模式

因为我们要对 chrdevbase 这个设备进行读写操作，所以选择 O_RDWR。除了上述三种模式以外还有其他的可选模式，通过逻辑或来选择多种模式：

O_APPEND 每次写操作都写入文件的末尾
O_CREAT 如果指定文件不存在，则创建这个文件
O_EXCL 如果要创建的文件已存在，则返回 -1，并且修改 errno 的值
O_TRUNC 如果文件存在，并且以只写/读写方式打开，则清空文件全部内容
O_NOCTTY 如果路径名指向终端设备，不要把这个设备用作控制终端。
O_NONBLOCK 如果路径名指向 FIFO/块文件/字符文件，则把文件的打开和后继 I/O 设置为非阻塞
O_DSYNC 等待物理 I/O 结束后再 write。在不影响读取新写入的数据的前提下，不等待文件属性更新。
O_RSYNC read 等待所有写入同一区域的写操作完成后再进行。
O_SYNC 等待物理 I/O 结束后再 write，包括更新文件属性的 I/O。

返回值：如果文件打开成功的话返回文件的文件描述符。

在 Ubuntu 中输入“man 2 open”即可查看 open 函数的详细内容，如图 20.4.2.1 所示：

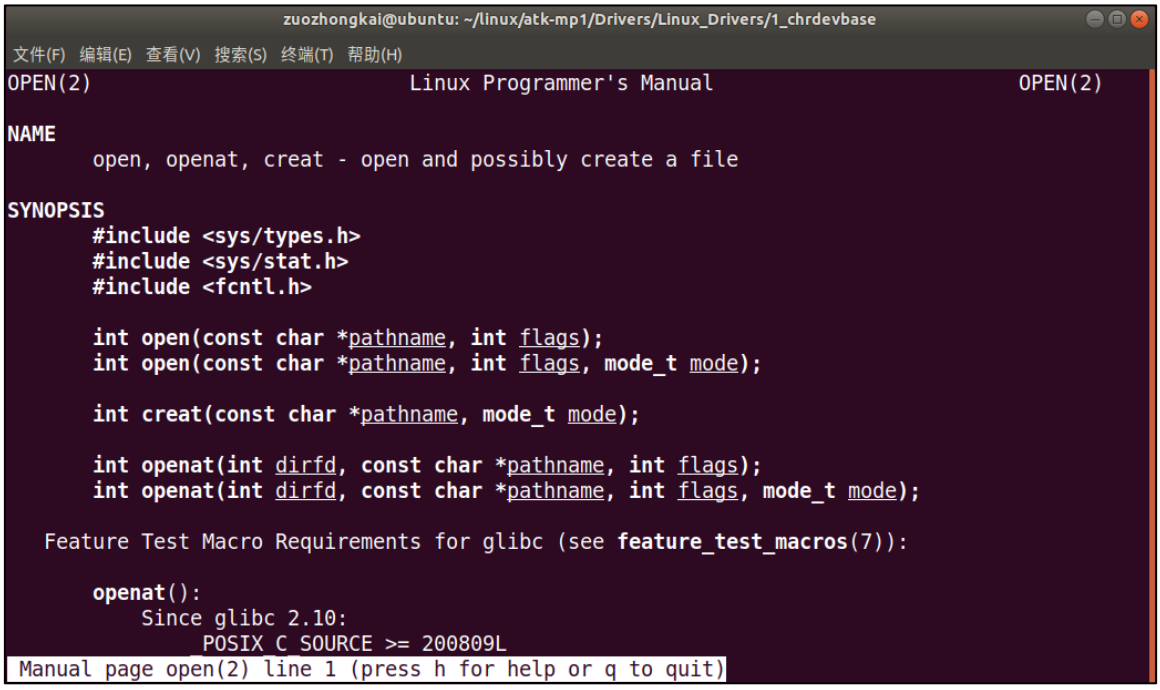


图 20.4.2.1 open 函数帮助信息

②、read 函数

read 函数原型如下：

```
ssize_t read(int fd, void *buf, size_t count)
```

read 函数参数含义如下：

fd：要读取的文件描述符，读取文件之前要先用 open 函数打开文件，open 函数打开文件成功以后会得到文件描述符。

buf：数据读取到此 buf 中。

count：要读取的数据长度，也就是字节数。

返回值：读取成功的话返回读取到的字节数；如果返回 0 表示读取到了文件末尾；如果返回负值，表示读取失败。在 Ubuntu 中输入“man 2 read”命令即可查看 read 函数的详细内容。

③、write 函数

write 函数原型如下：

```
ssize_t write(int fd, const void *buf, size_t count);
```

write 函数参数含义如下：

fd：要进行写操作的文件描述符，写文件之前要先用 open 函数打开文件，open 函数打开文件成功以后会得到文件描述符。

buf：要写入的数据。

count：要写入的数据长度，也就是字节数。

返回值：写入成功的话返回写入的字节数；如果返回 0 表示没有写入任何数据；如果返回负值，表示写入失败。在 Ubuntu 中输入“man 2 write”命令即可查看 write 函数的详细内容。

④、close 函数

close 函数原型如下：

```
int close(int fd);
```

close 函数参数含义如下：

fd：要关闭的文件描述符。

返回值：0 表示关闭成功，负值表示关闭失败。在 Ubuntu 中输入“man 2 close”命令即可查看 close 函数的详细内容。

2、编写测试 APP 程序

驱动编写好以后是需要测试的，一般编写一个简单的测试 APP，测试 APP 运行在用户空间。测试 APP 很简单通过输入相应的指令来对 chrdevbase 设备执行读或者写操作。在 1_chrdevbase 目录中新建 chrdevbaseApp.c 文件，在此文件中输入如下内容：

示例代码 20.4.2.1 chrdevbaseApp.c 文件

```
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "fcntl.h"
6  #include "stdlib.h"
7  #include "string.h"
8  /***************************************************************
9  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
10 文件名     : chrdevbaseApp.c
11 作者       : 正点原子
12 版本       : V1.0
13 描述       : chrdevbase 驱测试 APP。
14 其他       : 使用方法：./chrdevbaseApp /dev/chrdevbase <1>|<2>
15             argv[2] 1:读文件
16             argv[2] 2:写文件
17 论坛       : www.openedv.com
```

```
18 日志        : 初版 V1.0 2019/1/30 正点原子团队创建
19 ***************************************************************/
20
21 static char usrdata[] = {"usr data!"};
22
23 /*
24  * @description    : main 主程序
25  * @param - argc   : argv 数组元素个数
26  * @param - argv   : 具体参数
27  * @return         : 0 成功;其他 失败
28  */
29 int main(int argc, char *argv[])
30 {
31     int fd, retvalue;
32     char *filename;
33     char readbuf[100], writebuf[100];
34
35     if(argc != 3){
36         printf("Error Usage!\r\n");
37         return -1;
38     }
39
40     filename = argv[1];
41
42     /* 打开驱动文件 */
43     fd  = open(filename, O_RDWR);
44     if(fd < 0){
45         printf("Can't open file %s\r\n", filename);
46         return -1;
47     }
48
49     if(atoi(argv[2]) == 1){ /* 从驱动文件读取数据 */
50         retvalue = read(fd, readbuf, 50);
51         if(retvalue < 0){
52             printf("read file %s failed!\r\n", filename);
53         }else{
54             /*  读取成功，打印出读取成功的数据 */
55             printf("read data:%s\r\n",readbuf);
56         }
57     }
58
59     if(atoi(argv[2]) == 2){
60         /* 向设备驱动写数据 */
```

```
61          memcpy(writebuf, usrdata, sizeof(usrdata));
62          retvalue = write(fd, writebuf, 50);
63          if(retvalue < 0){
64              printf("write file %s failed!\r\n", filename);
65          }
66      }
67
68      /* 关闭设备 */
69      retvalue = close(fd);
70      if(retvalue < 0){
71          printf("Can't close file %s\r\n", filename);
72          return -1;
73      }
74
75      return 0;
76  }
```

第 21 行，数组 usrdata 是测试 APP 要向 chrdevbase 设备写入的数据。

第 35 行，判断运行测试 APP 的时候输入的参数是不是为 3 个，main 函数的 argc 参数表示参数数量，argv[]保存着具体的参数，如果参数不为 3 个的话就表示测试 APP 用法错误。比如，现在要从 chrdevbase 设备中读取数据，需要输入如下命令：

```
./chrdevbaseApp /dev/chrdevbase 1
```

上述命令一共有三个参数“./chrdevbaseApp”、“/dev/chrdevbase”和“1”，这三个参数分别对应 argv[0]、argv[1]和 argv[2]。第一个参数表示运行 chrdevbaseAPP 这个软件，第二个参数表示测试 APP 要打开/dev/chrdevbase 这个设备。第三个参数就是要执行的操作，1 表示从 chrdevbase 中读取数据，2 表示向 chrdevbase 写数据。

第 40 行，获取要打开的设备文件名字，argv[1]保存着设备名字。

第 43 行，调用 C 库中的 open 函数打开设备文件：/dev/chrdevbase。

第 49 行，判断 argv[2]参数的值是 1 还是 2，因为输入命令的时候其参数都是字符串格式的，因此需要借助 atoi 函数将字符串格式的数字转换为真实的数字。

第 50 行，当 argv[2]为 1 的时候表示要从 chrdevbase 设备中读取数据，一共读取 50 字节的数据，读取到的数据保存在 readbuf 中，读取成功以后就在终端上打印出读取到的数据。

第 59 行，当 argv[2]为 2 的时候表示要向 chrdevbase 设备写数据。

第 69 行，对 chrdevbase 设备操作完成以后就关闭设备。

chrdevbaseApp.c 内容还是很简单的，就是最普通的文件打开、关闭和读写操作。

20.4.3 编译驱动程序和测试 APP

1、编译驱动程序

首先编译驱动程序，也就是 chrdevbase.c 这个文件，我们需要将其编译为.ko 模块，创建 Makefile 文件，然后在其中输入如下内容：

示例代码 20.4.3.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
2  CURRENT_PATH := $(shell pwd)
```

```
3  obj-m := chrdevbase.o
4
5  build: kernel_modules
6
7  kernel_modules:
8      $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) modules
9  clean:
10     $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 1 行，KERNELDIR 表示开发板所使用的 Linux 内核源码目录，使用绝对路径，大家根据自己的实际情况填写即可。

第 2 行，CURRENT_PATH 表示当前路径，直接通过运行“pwd”命令来获取当前所处路径。

第 3 行，obj-m 表示将 chrdevbase.c 这个文件编译为 chrdevbase.ko 模块。

第 8 行，具体的编译命令，后面的 modules 表示编译模块，-C 表示将当前的工作目录切换到指定目录中，也就是 KERNERLDIR 目录。M 表示模块源码目录，“make modules”命令中加入 M=dir 以后程序会自动到指定的 dir 目录中读取模块的源码并将其编译为.ko 文件。

Makefile 编写好以后输入“make”命令编译驱动模块，编译过程如图 20.4.3.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase$ make
make -C /home/zuozhongkai/linux/atk-mp1/linux/my_linux/linux-5.4.31 M=/home/zuozhongkai/linux
/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase modules
make[1]: 进入目录“/home/zuozhongkai/linux/atk-mp1/linux/my_linux/linux-5.4.31”

  CC [M]  /home/zuozhongkai/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase/chrdevbase.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC [M]  /home/zuozhongkai/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase/chrdevbase.mod.o
  LD [M]  /home/zuozhongkai/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase/chrdevbase.ko
make[1]: 离开目录“/home/zuozhongkai/linux/atk-mp1/linux/my_linux/linux-5.4.31”
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase$
```

图 20.4.3.1 驱动模块编译过程

编译成功以后就会生成一个叫做 chrdevbaes.ko 的文件，此文件就是 chrdevbase 设备的驱动模块。至此，chrdevbase 设备的驱动就编译成功。

2、编译测试 APP

测试 APP 比较简单，只有一个文件，因此就不需要编写 Makefile 了，直接输入命令编译。因为测试 APP 是要在 ARM 开发板上运行的，所以需要使用 arm-linux-gnueabihf-gcc 来编译，输入如下命令：

```
arm-none-linux-gnueabihf-gcc chrdevbaseApp.c -o chrdevbaseApp
```

编译完成以后会生成一个叫做 chrdevbaseApp 的可执行程序，输入如下命令查看 chrdevbaseAPP 这个程序的文件信息：

```
file chrdevbaseApp
```

结果如图 20.4.3.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase$ file chrdevbaseApp
chrdevbaseApp: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, in
terpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, with debug_info, not stripped
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/1_chrdevbase$
```

图 20.4.3.2 chrdevbaseAPP 文件信息

从图 20.4.3.2 可以看出，chrdevbaseAPP 这个可执行文件是 32 位 LSB 格式，ARM 版本的，因此 chrdevbaseAPP 只能在 ARM 芯片下运行。

20.4.4 运行测试

1、加载驱动模块

驱动模块 chrdevbase.ko 和测试软件 chrdevbaseAPP 都已经准备好了，接下来就是运行测试。为了方便测试，Linux 系统选择通过 TFTP 从网络启动，并且使用 NFS 挂载网络根文件系统。确保 uboot 中 bootcmd 环境变量的值为：

```
tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000
```

bootargs 环境变量的值为：

```
console=ttySTM0,115200 root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw ip=192.168.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off
```

设置好以后启动 Linux 系统，检查开发板根文件系统中有没有“/lib/modules/5.4.31”这个目录，如果没有的话自行创建。因为是通过 NFS 将 Ubuntu 中的 rootfs(第十八章制作好的 buildroot 根文件系统)目录挂载为根文件系统，所以可以很方便的将 chrdevbase.ko 和 chrdevbaseAPP 复制到 rootfs/lib/modules/5.4.31 目录中，命令如下：

```
sudo cp chrdevbase.ko chrdevbaseApp /home/zuozhongkai/linux/nfs/rootfs/lib/modules/4.1.15/ -f
```

拷贝完成以后就会在开发板的/lib/modules/5.4.31 目录下存在 chrdevbase.ko 和 chrdevbaseAPP 这两个文件，如图 20.4.4.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls
chrdevbase.ko  chrdevbaseApp
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.1 驱动和测试文件

输入如下命令加载 chrdevbase.ko 驱动文件：

```
modprobe chrdevbase.ko
```

或

```
insmod chrdevbase.ko
```

如果使用 modprobe 加载驱动的话，可能会出现如图 20.4.4.2 或图 20.4.4.3 所示的提示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe  chrdevbase.ko
modprobe: can't open 'modules.dep': No such file or directory
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.2 找不到“modules.dep”文件

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe chrdevbase.ko
modprobe: module chrdevbase.ko not found in modules.dep
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.3 找不到“chrdevbase.ko”文件

图 20.4.4.2 和图 20.4.4.3 分别提示找不到“moudules.dep”和“chrdevbase.ko”，产生这两个错误的原因都是一样的。modprobe 命令会在“/lib/modules/5.4.31”目录下解析 modules.dep 文件，modules.dep 文件里面保存了要加载的.ko 模块，我们不用手动创建 modules.dep 这个文件，直接输入 depmod 命令即可自动生成 modules.dep，有些根文件系统可能没有 depmod 这个命令，如果没有这个命令就只能重新配置 busybox，使能此命令，然后重新编译 busybox。输入“depmod”命令以后会自动生成 modules.alias、modules.symbols 和 modules.dep 等等一些 modprobe 所需的文件，如图 20.4.4.4 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls
chrdevbase.ko     modules.alias     modules.symbols
chrdevbaseApp     modules.dep
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.4 depmod 命令执行结果

重新使用 modprobe 加载 chrdevbase.ko，结果如图 20.4.4.5 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe chrdevbase
chrdevbase init!
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.5 驱动加载成功

从图 20.4.4.5 可以看到“chrdevbase init!”这一行，这一行正是 chrdevbase.c 中模块入口函数 chrdevbase_init 输出的信息，说明模块加载成功！

注意：用 modprobe 这个命令加载模块不用加‘.ko’，加了的话可能报会图 20.4.4.3 所示错误，所以建议大家在使用 modprobe 命令加载驱动模块的时候不用加模块名字后面的“.ko”后缀。

输入“lsmod”命令即可查看当前系统中存在的模块，结果如图 20.4.4.6 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ lsmod
Module                  Size  Used by    Not tainted
chrdevbase             16384  0
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.6 当前系统中的模块

从图 20.4.4.6 可以看出，当前系统只有“chrdevbase”这一个模块。输入如下命令查看当前系统中有没有 chrdevbase 这个设备：

```
cat /proc/devices
```

结果如图 20.4.4.7 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ cat /proc/devices
Character devices:
  1 mem
  2 pty
  3 ttyp
  4 /dev/vc/0
  4 tty
  5 /dev/tty
  5 /dev/console
  5 /dev/ptmx
  5 ttyRPMSG
  7 vcs
 10 misc
 13 input
 21 sg
 29 fb
 89 i2c
 90 mtd
116 alsa
128 ptm
136 pts
153 spi
166 ttyACM
180 usb
189 usb_device
200 chrdevbase
226 drm
```

查看当前系统中的所有设备

主设备号为200的chrdevbase设备

图 20.4.4.7 当前系统设备

从图 20.4.4.7 可以看出，当前系统存在 chrdevbase 这个设备，主设备号为 200，跟我们设置

的主设备号一致。

2、创建设备节点文件

驱动加载成功需要在/dev 目录下创建一个与之对应的设备节点文件，应用程序就是通过操作这个设备节点文件来完成对具体设备的操作。输入如下命令创建/dev/chrdevbase 这个设备节点文件：

```
mknod /dev/chrdevbase c 200 0
```

其中“mknod”是创建节点命令，“/dev/chrdevbase”是要创建的节点文件，“c”表示这是个字符设备，“200”是设备的主设备号，“0”是设备的次设备号。创建完成以后就会存在/dev/chrdevbase 这个文件，可以使用“ls /dev/chrdevbase -l”命令查看，结果如图 20.4.4.8 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls /dev/chrdevbase -l
crw-r--r--    1 root     root      200,   0 Jan  3 01:50 /dev/chrdevbase
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.8 /dev/chrdevbase 文件

如果 chrdevbaseAPP 想要读写 chrdevbase 设备，直接对/dev/chrdevbase 进行读写操作即可。相当于/dev/chrdevbase 这个文件是 chrdevbase 设备在用户空间中的实现。前面一直说 Linux 下一切皆文件，包括设备也是文件，现在大家应该是有这个概念了吧？

3、chrdevbase 设备操作测试

一切准备就绪，接下来就是“大考”的时刻了。使用 chrdevbaseApp 软件操作 chrdevbase 这个设备，看看读写是否正常，首先进行读操作，输入如下命令：

```
./chrdevbaseApp /dev/chrdevbase 1
```

结果如图 20.4.4.9 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./chrdevbaseApp /dev/chrdevbase 1
kernel senddata ok!
read data:kernel data!
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.9 读操作结果

从图 20.4.4.9 可以看出，首先输出“kernel senddata ok!”这一行信息，这是驱动程序中 chrdevbase_read 函数输出的信息，因为 chrdevbaseAPP 使用 read 函数从 chrdevbase 设备读取数据，因此 chrdevbase_read 函数就会执行。chrdevbase_read 函数向 chrdevbaseAPP 发送“kernel data!”数据，chrdevbaseAPP 接收到以后就打印出来，“read data:kernel data!”就是 chrdevbaseAPP 打印出来的接收到的数据。说明对 chrdevbase 的读操作正常，接下来测试对 chrdevbase 设备的写操作，输入如下命令：

```
./chrdevbaseApp /dev/chrdevbase 2
```

结果如图 20.4.4.10 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./chrdevbaseApp /dev/chrdevbase 2
kernel recevdata:usr data!
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.10 写操作结果

只有一行“kernel recevdata:usr data!”，这个是驱动程序中的 chrdevbase_write 函数输出的。chrdevbaseAPP 使用 write 函数向 chrdevbase 设备写入数据“usr data!”。chrdevbase_write 函数接收到以后将其打印出来。说明对 chrdevbase 的写操作正常，既然读写都没问题，说明我们编写的 chrdevbase 驱动是没有问题的。

4、卸载驱动模块

如果不再使用某个设备的话可以将其驱动卸载掉，比如输入如下命令卸载掉 chrdevbase 这个设备：

```
rmmod chrdevbase
```

卸载以后使用 lsmod 命令查看 chrdevbase 这个模块还存不存在，结果如图 20.4.4.11 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ lsmod
Module                  Size  Used by    Not tainted
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 20.4.4.11 系统中当前模块

从图 20.4.4.11 可以看出，此时系统已经没有任何模块了，chrdevbase 这个模块也不存在了，说明模块卸载成功。

至此，chrdevbase 这个设备的整个驱动就验证完成了，驱动工作正常。本章我们详细的讲解了字符设备驱动的开发步骤，并且以一个虚拟的 chrdevbase 设备为例，带领大家完成了第一个字符设备驱动的开发，掌握了字符设备驱动的开发框架以及测试方法，以后的字符设备驱动实验基本都以此为蓝本。

第二十一章 嵌入式 Linux LED 驱动开发实验

上一章我们详细的讲解了字符设备驱动开发步骤，并且用一个虚拟的 chrdevbase 设备为例带领大家完成了第一个字符设备驱动的开发。本章我们就开始编写第一个真正的 Linux 字符设备驱动。在正点原子 STM32MP157 开发板上有一个 LED 灯，本章我们就来学习一下如何编写 Linux 下的 LED 灯驱动。

21.1 Linux 下 LED 灯驱动原理

Linux 下的任何外设驱动，最终都是要配置相应的硬件寄存器。所以本章的 LED 灯驱动最终也是对 STM32MP157 的 IO 口进行配置，与裸机实验不同的是，在 Linux 下编写驱动要符合 Linux 的驱动框架。开发板上的 LED0 连接到 STM32MP157 的 PI0 这个引脚上，因此本章实验的重点就是编写 Linux 下 STM32MP157 引脚控制驱动。

21.1.1 地址映射

在编写驱动之前，我们需要先简单了解一下 MMU 这个神器，MMU 全称叫做 Memory Manage Unit，也就是内存管理单元。在老版本的 Linux 中要求处理器必须有 MMU，但是现在 Linux 内核已经支持无 MMU 的处理器了。MMU 主要完成的功能如下：

①、完成虚拟空间到物理空间的映射。

②、内存保护，设置存储器的访问权限，设置虚拟存储空间的缓冲特性。

我们重点来看一下第①点，也就是虚拟空间到物理空间的映射，也叫做地址映射。首先了解两个地址概念：虚拟地址(VA,Virtual Address)、物理地址(PA，Physcical Address)。对于 32 位的处理器来说，虚拟地址范围是 2^32=4GB，我们的开发板上有 1GB 的 DDR3，这 1GB 的内存就是物理内存，经过 MMU 可以将其映射到整个 4GB 的虚拟空间，如图 21.1.2 所示：

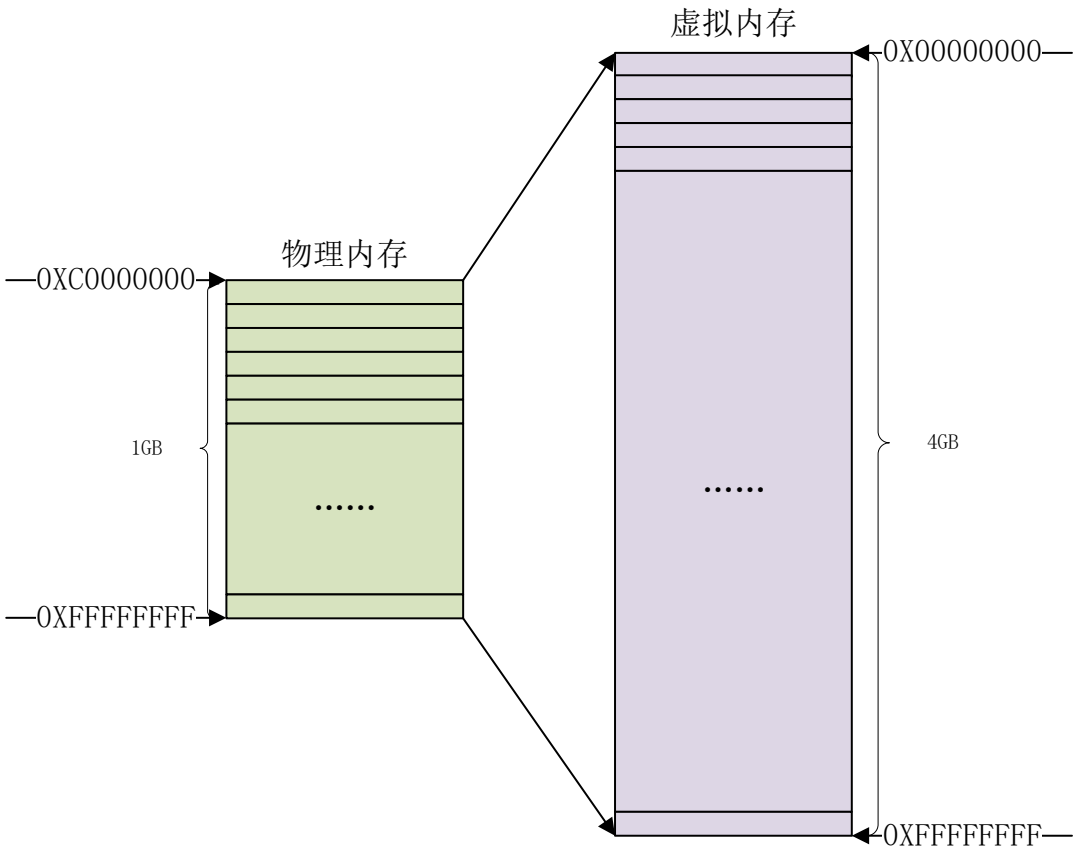


图 21.1.2 内存映射

物理内存只有 1GB，虚拟内存有 4GB，那么肯定存在多个虚拟地址映射到同一个物理地址上去，虚拟地址范围比物理地址范围大的问题处理器自会处理，这里我们不要去深究，因为 MMU 是很复杂的一个东西，后续有时间的话正点原子 Linux 团队会专门做 MMU 专题教程。

Linux 内核启动的时候会初始化 MMU，设置好内存映射，设置好以后 CPU 访问的都是虚拟地址。比如 STM32MP157 的 PI0 引脚的端口模式寄存器 GPIOI_MODER 物理地址为 0x5000A000。如果没有开启 MMU 的话直接向 0x5000A000 这个寄存器地址写入数据就可以配置 PI0 的引脚功能(输入、输出、复用或模拟等)。现在开启了 MMU，并且设置了内存映射，因此就不能直接向 0x5000A000 这个地址写入数据了。我们必须得到 0x5000A000 这个物理地址在 Linux 系统里面对应的虚拟地址，这里就涉及到了物理内存和虚拟内存之间的转换，需要用到两个函数：ioremap 和 iounmap。

1、ioremap 函数

ioremap 函数用于获取指定物理地址空间对应的虚拟地址空间，定义在 arch/arm/include/asm/io.h 文件中，定义如下：

示例代码 21.1.1.1 ioremap 函数声明

```
393 void __iomem *ioremap(resource_size_t res_cookie, size_t size);
```

函数的实现是在 arch/arm/mm/ioremap.c 文件中，实现如下：

示例代码 21.1.1.2 ioremap 函数实现

```
377 void __iomem *ioremap(resource_size_t res_cookie, size_t size)
378 {
379     return arch_ioremap_caller(res_cookie, size, MT_DEVICE,
380                 __builtin_return_address(0));
381 }
```

ioremap 有两个参数：res_cookie 和 size，真正起作用的是函数 arch_ioremap_caller。ioremap 函数有两个参数和一个返回值，这些参数和返回值的含义如下：

res_cookie：要映射的物理起始地址。

size：要映射的内存空间大小。

返回值：__iomem 类型的指针，指向映射后的虚拟空间首地址。

假如我们要获取 STM32MP157-ATK 的 GPIOI_MODER 寄存器对应的虚拟地址，使用如下代码即可：

```
#define GPIOI_MODER                 (0X5000A000)
static void __iomem*                GPIO_MODER_PI;
GPIO_MODER_PI = ioremap(GPIOI_MODER, 4);
```

宏 GPIOI_MODER 是寄存器物理地址，GPIO_MODER_PI 是映射后的虚拟地址。对于 STMP32MP157 来说一个寄存器是 4 字节(32 位)，因此映射的内存长度为 4。映射完成以后直接对 GPIO_MODER_PI 进行读写操作即可。

2、iounmap 函数

卸载驱动的时候需要使用 iounmap 函数释放掉 ioremap 函数所做的映射，iounmap 函数原型如下：

示例代码 41.1.1.3 iounmap 函数原型

```
413 void iounmap (volatile void __iomem *addr)
```

iounmap 只有一个参数 addr，此参数就是要取消映射的虚拟地址空间首地址。假如我们现在要取消掉 GPIO_MODER_PI 寄存器的地址映射，使用如下代码即可：

```
iounmap(GPIO_MODER_PI);
```

21.1.2 I/O 内存访问函数

这里说的 I/O 是输入/输出的意思，并不是我们学习单片机的时候讲的 GPIO 引脚。这里涉及到两个概念：I/O 端口和 I/O 内存。当外部寄存器或内存映射到 IO 空间时，称为 I/O 端口。当外部寄存器或内存映射到内存空间时，称为 I/O 内存。但是对于 ARM 来说没有 I/O 空间这个概念，因此 ARM 体系下只有 I/O 内存(可以直接理解为内存)。使用 ioremap 函数将寄存器的物理地址映射到虚拟地址以后，我们就可以直接通过指针访问这些地址，但是 Linux 内核不建议这么做，而是推荐使用一组操作函数来对映射后的内存进行读写操作。

1、读操作函数

读操作函数有如下几个：

示例代码 21.1.2.1 读操作函数

```
1 u8  readb(const volatile void __iomem *addr)
2 u16 readw(const volatile void __iomem *addr)
3 u32 readl(const volatile void __iomem *addr)
```

readb、readw 和 readl 这三个函数分别对应 8bit、16bit 和 32bit 读操作，参数 addr 就是要读取写内存地址，返回值就是读取到的数据。

2、写操作函数

写操作函数有如下几个：

示例代码 21.1.2.2 写操作函数

```
1 void writeb(u8 value,  volatile void __iomem *addr)
2 void writew(u16 value, volatile void __iomem *addr)
3 void writel(u32 value, volatile void __iomem *addr)
```

writeb、writew 和 writel 这三个函数分别对应 8bit、16bit 和 32bit 写操作，参数 value 是要写入的数值，addr 是要写入的地址。

21.2 硬件原理图分析

我们先进行 LED 的硬件原理分析，打开开发板底板原理图，底板原理图和核心板原理图都放到了开发板光盘中，路径为:开发板光盘→2、开发板原理图→STM32MP15x 底板原理图 V1.X。开发板上有一个 LED0，原理如图 21.1.1 所示：

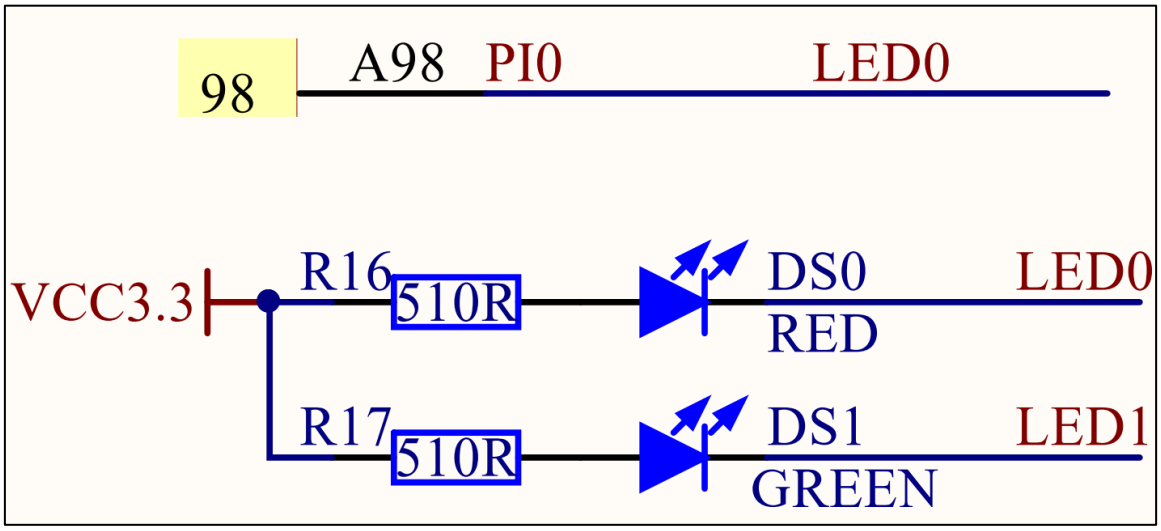


图 21.2.1 LED 原理图

从图 21.2.1 可以看出，LED0 接到了 PI0 上，PI0 就是 GPIOI 组的第 0 个引脚，当 PI0 输出低电平(0)的时候发光二极管 LED0 就会导通点亮，当 PI0 输出高电平(1)的时候发光二极管 LED0 不会导通，因此 LED0 也就不会点亮。所以 LED0 的亮灭取决于 PI0 的输出电平，输出 0 就亮，输出 1 就灭。

21.3 实验程序编写

本实验对应的例程路径为：开发板光盘-> 1、Linux 驱动例程-> 2_led。

本章实验编写 Linux 下的 LED 灯驱动，可以通过应用程序对开发板上的 LED0 进行开关操作。

21.3.1 LED 灯驱动程序编写

新建名为“2_led”文件夹，然后在 2_led 文件夹里面创建 VSCode 工程，工作区命名为“led”。工程创建好以后新建 led.c 文件，此文件就是 led 的驱动文件，在 led.c 里面输入如下内容：

示例代码 21.3.1.1 led.c 驱动文件代码

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <asm/mach/map.h>
10 #include <asm/uaccess.h>
11 #include <asm/io.h>
12 /***************************************************************
13 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
14 文件名     : led.c
15 作者       : 正点原子
16 版本       : V1.0
17 描述       : LED 驱动文件。
18 其他       : 无
19 论坛       : www.openedv.com
20 日志       : 初版 V1.0 2021/1/8 正点原子团队创建
21 ***************************************************************/
22 #define LED_MAJOR       200      /* 主设备号 */
23 #define LED_NAME        "led"    /* 设备名字 */
24
25 #define LEDOFF  0                /* 关灯     */
26 #define LEDON   1                /* 开灯     */
27
28 /* 寄存器物理地址 */
```

```
29  #define PERIPH_BASE             (0x40000000)
30  #define MPU_AHB4_PERIPH_BASE    (PERIPH_BASE + 0x10000000)
31  #define RCC_BASE                (MPU_AHB4_PERIPH_BASE + 0x0000)
32  #define RCC_MP_AHB4ENSETR       (RCC_BASE + 0XA28)
33  #define GPIOI_BASE              (MPU_AHB4_PERIPH_BASE + 0xA000)
34  #define GPIOI_MODER             (GPIOI_BASE + 0x0000)
35  #define GPIOI_OTYPER            (GPIOI_BASE + 0x0004)
36  #define GPIOI_OSPEEDR           (GPIOI_BASE + 0x0008)
37  #define GPIOI_PUPDR             (GPIOI_BASE + 0x000C)
38  #define GPIOI_BSRR              (GPIOI_BASE + 0x0018)
39
40
41  /* 映射后的寄存器虚拟地址指针 */
42  static void __iomem *MPU_AHB4_PERIPH_RCC_PI;
43  static void __iomem *GPIOI_MODER_PI;
44  static void __iomem *GPIOI_OTYPER_PI;
45  static void __iomem *GPIOI_OSPEEDR_PI;
46  static void __iomem *GPIOI_PUPDR_PI;
47  static void __iomem *GPIOI_BSRR_PI;
48
49  /*
50   * @description   : LED 打开/关闭
51   * @param - sta   : LEDON(0) 打开 LED，LEDOFF(1) 关闭 LED
52   * @return        : 无
53   */
54  void led_switch(u8 sta)
55  {
56      u32 val = 0;
57      if(sta == LEDON) {
58          val = readl(GPIOI_BSRR_PI);
59          val |= (1 << 16);
60          writel(val, GPIOI_BSRR_PI);
61      }else if(sta == LEDOFF) {
62          val = readl(GPIOI_BSRR_PI);
63          val|= (1 << 0);
64          writel(val, GPIOI_BSRR_PI);
65      }
66  }
67
68  /*
69   * @description   : 取消映射
70   * @return        : 无
71   */
```

```
72  void led_unmap(void)
73  {
74      /* 取消映射 */
75      iounmap(MPU_AHB4_PERIPH_RCC_PI);
76      iounmap(GPIOI_MODER_PI);
77      iounmap(GPIOI_OTYPER_PI);
78      iounmap(GPIOI_OSPEEDR_PI);
79      iounmap(GPIOI_PUPDR_PI);
80      iounmap(GPIOI_BSRR_PI);
81  }
82
83  /*
84   * @description   : 打开设备
85   * @param - inod  : 传递给驱动的 inode
86   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
87   *                  一般在 open 的时候将 private_data 指向设备结构体。
88   * @return        : 0 成功;其他 失败
89   */
90  static int led_open(struct inode *inode, struct file *filp)
91  {
92      return 0;
93  }
94
95  /*
96   * @description   : 从设备读取数据
97   * @param - filp  : 要打开的设备文件(文件描述符)
98   * @param - buf   : 返回给用户空间的数据缓冲区
99   * @param - cnt   : 要读取的数据长度
100  * @param - offt  : 相对于文件首地址的偏移
101  * @return        : 读取的字节数，如果为负值，表示读取失败
102  */
103 static ssize_t led_read(struct file *filp, char __user *buf,
                            size_t cnt, loff_t *offt)
104 {
105     return 0;
106 }
107
108 /*
109  * @description   : 向设备写数据
110  * @param - filp  : 设备文件，表示打开的文件描述符
111  * @param - buf   : 要写给设备写入的数据
112  * @param - cnt   : 要写入的数据长度
113  * @param - offt  : 相对于文件首地址的偏移
```

```
114  * @return         : 写入的字节数，如果为负值，表示写入失败
115  */
116 static ssize_t led_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
117 {
118     int retvalue;
119     unsigned char databuf[1];
120     unsigned char ledstat;
121 
122     retvalue = copy_from_user(databuf, buf, cnt);
123     if(retvalue < 0) {
124         printk("kernel write failed!\r\n");
125         return -EFAULT;
126     }
127 
128     ledstat = databuf[0];       /* 获取状态值   */
129 
130     if(ledstat == LEDON) {
131         led_switch(LEDON);      /* 打开 LED 灯   */
132     } else if(ledstat == LEDOFF) {
133         led_switch(LEDOFF);     /* 关闭 LED 灯   */
134     }
135     return 0;
136 }
137 
138 /*
139  * @description   : 关闭/释放设备
140  * @param - filp  : 要关闭的设备文件(文件描述符)
141  * @return        : 0 成功;其他 失败
142  */
143 static int led_release(struct inode *inode, struct file *filp)
144 {
145     return 0;
146 }
147 
148 /* 设备操作函数 */
149 static struct file_operations led_fops = {
150     .owner    = THIS_MODULE,
151     .open     = led_open,
152     .read     = led_read,
153     .write    = led_write,
154     .release  =  led_release,
155 };
```

```
156
157 /*
158  * @description   : 驱动出口函数
159  * @param         : 无
160  * @return        : 无
161  */
162 static int __init led_init(void)
163 {
164     int retvalue = 0;
165     u32 val = 0;
166
167     /* 初始化 LED           */
168     /* 1、寄存器地址映射     */
169     MPU_AHB4_PERIPH_RCC_PI = ioremap(RCC_MP_AHB4ENSETR, 4);
170     GPIOI_MODER_PI = ioremap(GPIOI_MODER, 4);
171     GPIOI_OTYPER_PI = ioremap(GPIOI_OTYPER, 4);
172     GPIOI_OSPEEDR_PI = ioremap(GPIOI_OSPEEDR, 4);
173     GPIOI_PUPDR_PI = ioremap(GPIOI_PUPDR, 4);
174     GPIOI_BSRR_PI = ioremap(GPIOI_BSRR, 4);
175
176     /* 2、使能 PI 时钟 */
177     val = readl(MPU_AHB4_PERIPH_RCC_PI);
178     val &= ~(0X1 << 8);  /* 清除以前的设置    */
179     val |= (0X1 << 8);   /* 设置新值          */
180     writel(val, MPU_AHB4_PERIPH_RCC_PI);
181
182     /* 3、设置 PI0 通用的输出模式。*/
183     val = readl(GPIOI_MODER_PI);
184     val &= ~(0X3 << 0);  /* bit0:1 清零       */
185     val |= (0X1 << 0);   /* bit0:1 设置 01    */
186     writel(val, GPIOI_MODER_PI);
187
188     /* 3、设置 PI0 为推挽模式。*/
189     val = readl(GPIOI_OTYPER_PI);
190     val &= ~(0X1 << 0); /* bit0 清零，设置为上拉*/
191     writel(val, GPIOI_OTYPER_PI);
192
193     /* 4、设置 PI0 为高速。*/
194     val = readl(GPIOI_OSPEEDR_PI);
195     val &= ~(0X3 << 0);  /* bit0:1 清零         */
196     val |= (0x2 << 0);   /* bit0:1 设置为 10    */
197     writel(val, GPIOI_OSPEEDR_PI);
198
```

```
199     /* 5、设置 PI0 为上拉。*/
200     val = readl(GPIOI_PUPDR_PI);
201     val &= ~(0X3 << 0);  /* bit0:1 清零     */
202     val |= (0x1 << 0);   /*bit0:1 设置为 01  */
203     writel(val,GPIOI_PUPDR_PI);
204
205     /* 6、默认关闭 LED */
206     val = readl(GPIOI_BSRR_PI);
207     val |= (0x1 << 0);
208     writel(val, GPIOI_BSRR_PI);
209
210     /* 6、注册字符设备驱动 */
211     retvalue = register_chrdev(LED_MAJOR, LED_NAME, &led_fops);
212     if(retvalue < 0){
213         printk("register chrdev failed!\r\n");
214         goto fail_map;
215     }
216     return 0;
217
218 fail_map:
219     led_unmap();
220     return -EIO;
221 }
222
223 /*
224  * @description : 驱动出口函数
225  * @param       : 无
226  * @return      : 无
227  */
228 static void __exit led_exit(void)
229 {
230     /* 取消映射 */
231     led_unmap();
232
233     /* 注销字符设备驱动 */
234     unregister_chrdev(LED_MAJOR, LED_NAME);
235 }
236
237 module_init(led_init);
238 module_exit(led_exit);
239 MODULE_LICENSE("GPL");
240 MODULE_AUTHOR("ALIENTEK");
241 MODULE_INFO(intree, "Y");
```

第 22~26 行，定义了一些宏，包括主设备号、设备名字、LED 开/关宏。

第 29~38 行，本实验要用到的寄存器宏定义。

第 42~47 行，经过内存映射以后的寄存器地址指针。

第 54~66 行，led_switch 函数，用于控制开发板上的 LED 灯亮灭，当参数 sta 为 LEDON(0)的时候打开 LED 灯，sta 为 LEDOFF(1)的时候关闭 LED 灯。

第 72~81 行，led_unmap 函数，取消所有物理寄存器映射，回收对应的资源。当程序出错退出或者卸载驱动模块的时候需要调用此函数，用来取消此前所做的寄存器映射。

第 90~93 行，led_open 函数，为空函数，可以自行在此函数中添加相关内容，一般在此函数中将设备结构体作为参数 filp 的私有数据(filp->private_data)，后面实验会讲解如何添加私有数据。

第 103~106 行，led_read 函数，为空函数，如果想在应用程序中读取 LED 的状态，那么就可以在此函数中添加相应的代码。

第 116~136 行，led_write 函数，实现对 LED 灯的开关操作，当应用程序调用 write 函数向 led 设备写数据的时候此函数就会执行。首先通过函数 copy_from_user 获取应用程序发送过来的操作信息(打开还是关闭 LED)，最后根据应用程序的操作信息来打开或关闭 LED 灯。

第 143~146 行，led_release 函数，为空函数，可以自行在此函数中添加相关内容，一般关闭设备的时候会释放掉 led_open 函数中添加的私有数据。

第 149~155 行，设备文件操作结构体 led_fops 的定义和初始化。

第 162~221 行，驱动入口函数 led_init，此函数实现了 LED 的初始化工作，169~174 行通过 ioremap 函数获取物理寄存器地址映射后的虚拟地址，得到寄存器对应的虚拟地址以后就可以完成相关初始化工作了。比如使能 GPIOI 时钟、设置 PI0 复用功能、配置 PI0 的属性等等。最后，最重要的一步！使用 register_chrdev 函数注册 led 这个字符设备。

第 218~220 行，如果在第 6 步注册字符设备失败，就要回收以前注册成功的资源。

第 228~235 行，驱动出口函数 led_exit，首先使用函数 iounmap 取消内存映射，最后使用函数 unregister_chrdev 注销 led 这个字符设备。

第 237~238 行，使用 module_init 和 module_exit 这两个函数指定 led 设备驱动加载和卸载函数。

第 239~240 行，添加 LICENSE 和作者信息。

第 241 行，告诉内核这个驱动也是 intree 模块驱动。

21.3.2 编写测试 APP

编写测试 APP，led 驱动加载成功以后手动创建/dev/led 节点，应用程序(APP)通过操作/dev/led 文件来完成对 LED 设备的控制。向/dev/led 文件写 0 表示关闭 LED 灯，写 1 表示打开 LED 灯。新建 ledApp.c 文件，在里面输入如下内容：

示例代码 21.3.2.1 ledApp.c 文件代码

```
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "fcntl.h"
6  #include "stdlib.h"
7  #include "string.h"
8  /*****************************************************************
```

```
9   Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
10  文件名      : ledApp.c
11  作者        : 正点原子
12  版本        : V1.0
13  描述        : led 测试 APP。
14  其他        : 无
15  使用方法    : ./ledtest /dev/led  0 关闭 LED
16                ./ledtest /dev/led  1 打开 LED
17  论坛        : www.openedv.com
18  日志        : 初版 V1.0 2021/11/23 正点原子团队创建
19  ***************************************************************/
20
21  #define LEDOFF  0
22  #define LEDON   1
23
24  /*
25   * @description       : main 主程序
26   * @param - argc      : argv 数组元素个数
27   * @param - argv      : 具体参数
28   * @return            : 0 成功;其他 失败
29   */
30  int main(int argc, char *argv[])
31  {
32      int fd, retvalue;
33      char *filename;
34      unsigned char databuf[1];
35
36      if(argc != 3){
37          printf("Error Usage!\r\n");
38          return -1;
39      }
40
41      filename = argv[1];
42
43      /* 打开 led 驱动 */
44      fd = open(filename, O_RDWR);
45      if(fd < 0){
46          printf("file %s open failed!\r\n", argv[1]);
47          return -1;
48      }
49
50      databuf[0] = atoi(argv[2]); /* 要执行的操作：打开或关闭 */
51
```

```
52      /* 向/dev/led 文件写入数据 */
53      retvalue = write(fd, databuf, sizeof(databuf));
54      if(retvalue < 0){
55          printf("LED Control Failed!\r\n");
56          close(fd);
57          return -1;
58      }
59
60      retvalue = close(fd); /* 关闭文件 */
61      if(retvalue < 0){
62          printf("file %s close failed!\r\n", argv[1]);
63          return -1;
64      }
65      return 0;
66  }
```

ledApp.c 的内容还是很简单的，就是对 led 的驱动文件进行最基本的打开、关闭、写操作等。

21.4 运行测试

21.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第二十章实验基本一样，只是将 obj-m 变量的值改为 led.o，Makefile 内容如下所示：

示例代码 21.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := led.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 led.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“led.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 ledApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc ledApp.c -o ledApp
```

编译成功以后就会生成 ledApp 这个应用程序。

21.4.2 运行测试

将上一小节编译出来的 led.ko 和 ledApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，

重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 led.ko 驱动模块：

```
depmod          //第一次加载驱动的时候需要运行此命令
modprobe led    //加载驱动
```

驱动加载成功以后创建“/dev/led”设备节点，命令如下：

```
mknod /dev/led c 200 0
```

驱动节点创建成功以后就可以使用 ledApp 软件来测试驱动是否工作正常，输入如下命令打开 LED 灯：

```
./ledApp /dev/led 1     //打开 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯，也就是 LED0 是否点亮，如果点亮的话说明驱动工作正常。在输入如下命令关闭 LED 灯：

```
./ledApp /dev/led 0     //关闭 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯是否熄灭，如果熄灭的话说明我们编写的 LED 驱动工作完全正常！至此，我们成功编写了第一个真正的 Linux 驱动设备程序。

如果要卸载驱动的话输入如下命令即可：

```
rmmod led.ko
```

第二十二章 新字符设备驱动实验

经过前两章实验的实战操作，我们已经掌握了 Linux 字符设备驱动开发的基本步骤，字符设备驱动开发重点是使用 register_chrdev 函数注册字符设备，当不再使用设备的时候就使用 unregister_chrdev 函数注销字符设备，驱动模块加载成功以后还需要手动使用 mknod 命令创建设备节点。register_chrdev 和 unregister_chrdev 这两个函数是老版本驱动使用的函数，现在新的字符设备驱动已经不再使用这两个函数，而是使用 Linux 内核推荐的新字符设备驱动 API 函数。本节我们就来学习一下如何编写新字符设备驱动，并且在驱动模块加载的时候自动创建设备节点文件。

22.1 新字符设备驱动原理

22.1.1 分配和释放设备号

使用 register_chrdev 函数注册字符设备的时候只需要给定一个主设备号即可，但是这样会带来两个问题：

①、需要我们事先确定好哪些主设备号没有使用。

②、会将一个主设备号下的所有次设备号都使用掉，比如现在设置 LED 这个主设备号为 200，那么 0~1048575(2^20-1)这个区间的次设备号就全部都被 LED 一个设备分走了。这样太浪费次设备号了！一个 LED 设备肯定只能有一个主设备号，一个次设备号。

解决这两个问题最好的方法就是在使用设备号的时候向 Linux 内核申请，需要几个就申请几个，由 Linux 内核分配设备可以使用的设备号。这个就是我们在 20.3.2 小节讲解的设备号的分配，如果没有指定设备号的话就使用如下函数来申请设备号：

```
int alloc_chrdev_region(dev_t *dev, unsigned baseminor, unsigned count, const char *name)
```

如果给定了设备的主设备号和次设备号就使用如下所示函数来注册设备号即可：

```
int register_chrdev_region(dev_t from, unsigned count, const char *name)
```

参数 from 是要申请的起始设备号，也就是给定的设备号；参数 count 是要申请的数量，一般都是一个；参数 name 是设备名字。

注销字符设备之后要释放掉设备号，不管是通过 alloc_chrdev_region 函数还是 register_chrdev_region 函数申请的设备号，统一使用如下释放函数：

```
void unregister_chrdev_region(dev_t from, unsigned count)
```

新字符设备驱动下，设备号分配示例代码如下：

示例代码 22.1.1.1 新字符设备驱动下设备号分配

```
1  int major;                          /* 主设备号                  */
2  int minor;                          /* 次设备号                  */
3  dev_t devid;                        /* 设备号                    */
4
5  if (major) {                        /* 定义了主设备号            */
6      devid = MKDEV(major, 0);        /* 大部分驱动次设备号都选择 0*/
7      register_chrdev_region(devid, 1, "test");
8  } else {                            /* 没有定义设备号            */
9      alloc_chrdev_region(&devid, 0, 1, "test");  /* 申请设备号 */
10     major = MAJOR(devid);           /* 获取分配号的主设备号      */
11     minor = MINOR(devid);           /* 获取分配号的次设备号      */
12 }
```

第 1~3 行，定义了主/次设备号变量 major 和 minor，以及设备号变量 devid。

第 5 行，判断主设备号 major 是否有效，在 Linux 驱动中一般给出主设备号的话就表示这个设备的设备号已经确定了，因为次设备号基本上都选择 0，这个算 Linux 驱动开发中约定俗成的一种规定了。

第 6 行，如果 major 有效的话就使用 MKDEV 来构建设备号，次设备号选择 0。

第 7 行，使用 register_chrdev_region 函数来注册设备号。

第 9~11 行，如果 major 无效，那就表示没有给定设备号。此时就要使用 alloc_chrdev_region 函数来申请设备号。设备号申请成功以后使用 MAJOR 和 MINOR 来提取出主设备号和次设备

号，当然了，第 10 和 11 行提取主设备号和次设备号的代码可以不要。

如果要注销设备号的话，使用如下代码即可：

示例代码 22.1.1.2 cdev 结构体

```
1 unregister_chrdev_region(devid, 1);          /* 注销设备号 */
```

注销设备号的代码很简单。

22.1.2 新的字符设备注册方法

1、字符设备结构

在 Linux 中使用 cdev 结构体表示一个字符设备，cdev 结构体在 include/linux/cdev.h 文件中的定义如下：

示例代码 22.1.2.1 cdev 结构体

```
1 struct cdev {
2      struct kobject    kobj;
3      struct module     *owner;
4      const struct file_operations *ops;
5      struct list_head  list;
6      dev_t             dev;
7      unsigned int      count;
8 } __randomize_layout;
```

在 cdev 中有两个重要的成员变量：ops 和 dev，这两个就是字符设备文件操作函数集合 file_operations 以及设备号 dev_t。编写字符设备驱动之前需要定义一个 cdev 结构体变量，这个变量就表示一个字符设备，如下所示：

```
struct cdev test_cdev;
```

2、cdev_init 函数

定义好 cdev 变量以后就要使用 cdev_init 函数对其进行初始化，cdev_init 函数原型如下：

```
void cdev_init(struct cdev *cdev, const struct file_operations *fops)
```

参数 cdev 就是要初始化的 cdev 结构体变量，参数 fops 就是字符设备文件操作函数集合。使用 cdev_init 函数初始化 cdev 变量的示例代码如下：

示例代码 22.1.2.2 cdev_init 函数使用示例代码

```
1  struct cdev testcdev;
2
3  /* 设备操作函数 */
4  static struct file_operations test_fops = {
5      .owner = THIS_MODULE,
6      /* 其他具体的初始项 */
7  };
8
9  testcdev.owner = THIS_MODULE;
10 cdev_init(&testcdev, &test_fops); /* 初始化 cdev 结构体变量 */
```

3、cdev_add 函数

cdev_add 函数用于向 Linux 系统添加字符设备(cdev 结构体变量)，首先使用 cdev_init 函数

完成对 cdev 结构体变量的初始化，然后使用 cdev_add 函数向 Linux 系统添加这个字符设备。cdev_add 函数原型如下：

```
int cdev_add(struct cdev *p, dev_t dev, unsigned count)
```

参数 p 指向要添加的字符设备(cdev 结构体变量)，参数 dev 就是设备所使用的设备号，参数 count 是要添加的设备数量。完善示例代码 22.1.2.2，加入 cdev_add 函数，内容如下所示：

示例代码 22.1.2.2 cdev_add 函数使用示例

```
1  struct cdev testcdev;
2
3  /* 设备操作函数 */
4  static struct file_operations test_fops = {
5      .owner = THIS_MODULE,
6      /* 其他具体的初始项 */
7  };
8
9  testcdev.owner = THIS_MODULE;
10 cdev_init(&testcdev, &test_fops);       /* 初始化 cdev 结构体变量     */
11 cdev_add(&testcdev, devid, 1);          /* 添加字符设备               */
```

示例代码 22.1.2.2 就是新的注册字符设备代码段，Linux 内核中大量的字符设备驱动都是采用这种方法向 Linux 内核添加字符设备。如果在加上示例代码 22.1.1.1 中分配设备号的程序，那么它们就一起实现的就是函数 register_chrdev 的功能。

3、cdev_del 函数

卸载驱动的时候一定要使用 cdev_del 函数从 Linux 内核中删除相应的字符设备，cdev_del 函数原型如下：

```
void cdev_del(struct cdev *p)
```

参数 p 就是要删除的字符设备。如果要删除字符设备，参考如下代码：

示例代码 22.1.2.3 cdev_del 函数使用示例

```
1 cdev_del(&testcdev);   /*  删除 cdev */
```

cdev_del 和 unregister_chrdev_region 这两个函数合起来的功能相当于 unregister_chrdev 函数。

22.2 自动创建设备节点

在前面的 Linux 驱动实验中，当我们使用 modprobe 加载驱动程序以后还需要使用命令“mknod”手动创建设备节点。本节就来讲解一下如何实现自动创建设备节点，在驱动中实现自动创建设备节点的功能以后，使用 modprobe 加载驱动模块成功的话就会自动在/dev 目录下创建对应的设备文件。

22.2.1 mdev 机制

udev 是一个用户程序，在 Linux 下通过 udev 来实现设备文件的创建与删除，udev 可以检测系统中硬件设备状态，可以根据系统中硬件设备状态来创建或者删除设备文件。比如使用 modprobe 命令成功加载驱动模块以后就自动在/dev 目录下创建对应的设备节点文件,使用 rmmod 命令卸载驱动模块以后就删除掉/dev 目录下的设备节点文件。我们在使用 buildroot 构建根文件系统的时候选择了 udev 的简化版本—mdev，所以在嵌入式 Linux 中我们使用 mdev 来实

现设备节点文件的自动创建与删除，Linux 系统中的热插拔事件也由 mdev 管理，如果使用 busybox 构建根文件系统，会在/etc/init.d/rcS 文件中如下语句：

```
echo /sbin/mdev > /proc/sys/kernel/hotplug
```

上述命令设置热插拔事件由 mdev 来管理。buildroot 构建的根文件系统已经帮我们全部处理好 mdev 了，不需要我们在修改什么文件。buildroot 构建的根文件系统再启动的时候会执行/etc/init.d/S10mdev 脚本来启动 mdev，如图 22.2.1.1 所示：

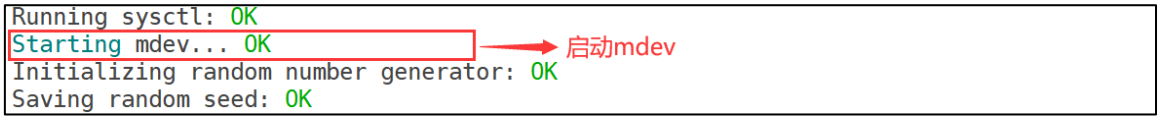


图 22.2.1.1 启动 mdev

关于 udev 或 mdev 更加详细的工作原理这里就不详细探讨了，我们重点来学习一下如何通过 mdev 来实现设备文件节点的自动创建与删除。

22.2.1 创建和删除类

自动创建设备节点的工作是在驱动程序的入口函数中完成的，一般在 cdev_add 函数后面添加自动创建设备节点相关代码。首先要创建一个 class 类，class 是个结构体，定义在文件 include/linux/device.h 里面。class_create 是类创建函数，class_create 是个宏定义，内容如下：

示例代码 22.2.1.1 class_create 函数

```
771 extern struct class * __must_check __class_create(struct module
      *owner,
772                       const char *name,
773                       struct lock_class_key *key);
774 extern void class_destroy(struct class *cls);
775
776 /* This is a #define to keep the compiler from merging different
777  * instances of the __key variable */
778 #define class_create(owner, name)       \
779 ({                      \
780     static struct lock_class_key __key; \
781     __class_create(owner, name, &__key);    \
782 })
```

根据上述代码，将宏 class_create 展开以后内容如下：

```
struct class *class_create (struct module *owner, const char *name)
```

class_create 一共有两个参数，参数 owner 一般为 THIS_MODULE，参数 name 是类名字。返回值是个指向结构体 class 的指针，也就是创建的类。

卸载驱动程序的时候需要删除掉类，类删除函数为 class_destroy，函数原型如下：

```
void class_destroy(struct class *cls);
```

参数 cls 就是要删除的类。

22.2.2 创建设备

上一小节创建好类以后还不能实现自动创建设备节点，我们还需要在这个类下创建一个设备。使用 device_create 函数在类下面创建设备，device_create 函数原型如下：

```
struct device *device_create(struct class      *cls,
```

```
                                struct device    *parent,
                                dev_t            devt,
                                void             *drvdata,
                                const char       *fmt, ...)
```

device_create 是个可变参数函数，参数 cls 就是设备要创建哪个类下面；参数 parent 是父设备，一般为 NULL，也就是没有父设备；参数 devt 是设备号；参数 drvdata 是设备可能会使用的一些数据，一般为 NULL；参数 fmt 是设备名字，如果设置 fmt=xxx 的话，就会生成/dev/xxx 这个设备文件。返回值就是创建好的设备。

同样的，卸载驱动的时候需要删除掉创建的设备，设备删除函数为 device_destroy，函数原型如下：

```
void device_destroy(struct class *cls, dev_t devt)
```

参数 classs 是要删除的设备所处的类，参数 devt 是要删除的设备号。

22.2.3 参考示例

在驱动入口函数里面创建类和设备，在驱动出口函数里面删除类和设备，参考示例如下：

示例代码 22.2.3.1 创建/删除类/设备参考代码

```
1  struct class *class;      /* 类       */
2  struct device *device;    /* 设备     */
3  dev_t devid;              /* 设备号   */
4
5  /* 驱动入口函数 */
6  static int __init xxx_init(void)
7  {
8      /* 创建类   */
9      class = class_create(THIS_MODULE, "xxx");
10     /* 创建设备 */
11     device = device_create(class, NULL, devid, NULL, "xxx");
12     return 0;
13 }
14
15 /* 驱动出口函数 */
16 static void __exit led_exit(void)
17 {
18     /* 删除设备 */
19     device_destroy(newchrled.class, newchrled.devid);
20     /* 删除类   */
21     class_destroy(newchrled.class);
22 }
23
24 module_init(led_init);
25 module_exit(led_exit);
```

22.3 设置文件私有数据

每个硬件设备都有一些属性，比如主设备号(dev_t)，类(class)、设备(device)、开关状态(state)等等，在编写驱动的时候你可以将这些属性全部写成变量的形式，如下所示：

示例代码 22.3.1 变量形式的设备属性

```
dev_t devid;              /* 设备号   */
struct cdev cdev;         /* cdev     */
struct class *class;      /* 类       */
struct device *device;    /* 设备     */
int major;                /* 主设备号 */
int minor;                /* 次设备号 */
```

这样写肯定没有问题，但是这样写不专业！对于一个设备的所有属性信息我们最好将其做成一个结构体。编写驱动 open 函数的时候将设备结构体作为私有数据添加到设备文件中，如下所示：

示例代码 22.3.2 设备结构体作为私有数据

```
/* 设备结构体 */
struct test_dev{
    dev_t devid;              /* 设备号       */
    struct cdev cdev;         /* cdev         */
    struct class *class;      /* 类           */
    struct device *device;    /* 设备         */
    int major;                /* 主设备号     */
    int minor;                /* 次设备号     */
};

struct test_dev testdev;

/* open 函数 */
static int test_open(struct inode *inode, struct file *filp)
{
    filp->private_data = &testdev; /* 设置私有数据 */
    return 0;
}
```

在 open 函数里面设置好私有数据以后，在 write、read、close 等函数中直接读取 private_data 即可得到设备结构体。

22.4 硬件原理图分析

本实验的硬件原理参考 21.2 小节即可。

22.5 实验程序编写

本实验对应的例程路径为：开发板光盘-> 1、Linux 驱动例程-> 3_newchrled。

本章实验在上一章实验的基础上完成，重点是使用了新的字符设备驱动、设置了文件私有数据、添加了自动创建设备节点相关内容。

22.5.1 LED 灯驱动程序编写

新建名为“3_newchrled”文件夹，然后在 3_newchrled 文件夹里面创建 vscode 工程，工作区命名为“newchrled”。工程创建好以后新建 newchrled.c 文件，在 newchrled.c 里面输入如下内容：

示例代码 22.5.1.1 newchrled.c 文件

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <asm/mach/map.h>
12 #include <asm/uaccess.h>
13 #include <asm/io.h>
14
15 /***************************************************************
16 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
17 文件名      : newchrled.c
18 作者        : 正点原子
19 版本        : V1.0
20 描述        : LED 驱动文件。
21 其他        : 无
22 论坛        : www.openedv.com
23 日志        : 初版 V1.0 2020/11/24 正点原子团队创建
24 ***************************************************************/
25 #define NEWCHRLED_CNT        1              /* 设备号个数   */
26 #define NEWCHRLED_NAME       "newchrled"    /* 名字         */
27 #define LEDOFF               0              /* 关灯         */
28 #define LEDON                1              /* 开灯         */
29
30 /* 寄存器物理地址 */
31 #define PERIPH_BASE             (0x40000000)
32 #define MPU_AHB4_PERIPH_BASE    (PERIPH_BASE + 0x10000000)
33 #define RCC_BASE                (MPU_AHB4_PERIPH_BASE + 0x0000)
34 #define RCC_MP_AHB4ENSETR       (RCC_BASE + 0XA28)
35 #define GPIOI_BASE              (MPU_AHB4_PERIPH_BASE + 0xA000)
36 #define GPIOI_MODER             (GPIOI_BASE + 0x0000)
37 #define GPIOI_OTYPER            (GPIOI_BASE + 0x0004)
```

```
38  #define GPIOI_OSPEEDR           (GPIOI_BASE + 0x0008)
39  #define GPIOI_PUPDR             (GPIOI_BASE + 0x000C)
40  #define GPIOI_BSRR              (GPIOI_BASE + 0x0018)
41
42  /* 映射后的寄存器虚拟地址指针 */
43  static void __iomem *MPU_AHB4_PERIPH_RCC_PI;
44  static void __iomem *GPIOI_MODER_PI;
45  static void __iomem *GPIOI_OTYPER_PI;
46  static void __iomem *GPIOI_OSPEEDR_PI;
47  static void __iomem *GPIOI_PUPDR_PI;
48  static void __iomem *GPIOI_BSRR_PI;
49
50  /* newchrled 设备结构体 */
51  struct newchrled_dev{
52      dev_t devid;                /* 设备号       */
53      struct cdev cdev;           /* cdev         */
54      struct class *class;        /* 类           */
55      struct device *device;      /* 设备         */
56      int major;                  /* 主设备号     */
57      int minor;                  /* 次设备号     */
58  };
59
60  struct newchrled_dev newchrled; /* led 设备 */
61
62  /*
63   * @description   : LED 打开/关闭
64   * @param - sta   : LEDON(0) 打开 LED，LEDOFF(1) 关闭 LED
65   * @return        : 无
66   */
67  void led_switch(u8 sta)
68  {
69      u32 val = 0;
70      if(sta == LEDON) {
71          val = readl(GPIOI_BSRR_PI);
72          val |= (1 << 16);
73          writel(val, GPIOI_BSRR_PI);
74      }else if(sta == LEDOFF) {
75          val = readl(GPIOI_BSRR_PI);
76          val|= (1 << 0);
77          writel(val, GPIOI_BSRR_PI);
78      }
79  }
80
```

```
81  /*
82   * @description   : 取消映射
83   * @return        : 无
84   */
85  void led_unmap(void)
86  {
87          /* 取消映射 */
88      iounmap(MPU_AHB4_PERIPH_RCC_PI);
89      iounmap(GPIOI_MODER_PI);
90      iounmap(GPIOI_OTYPER_PI);
91      iounmap(GPIOI_OSPEEDR_PI);
92      iounmap(GPIOI_PUPDR_PI);
93      iounmap(GPIOI_BSRR_PI);
94  }
95
96  /*
97   * @description   : 打开设备
98   * @param - inode : 传递给驱动的 inode
99   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
100  *                  一般在 open 的时候将 private_data 指向设备结构体。
101  * @return        : 0 成功;其他 失败
102  */
103 static int led_open(struct inode *inode, struct file *filp)
104 {
105     filp->private_data = &newchrled; /* 设置私有数据 */
106     return 0;
107 }
108
109 /*
110  * @description   : 从设备读取数据
111  * @param - filp  : 要打开的设备文件(文件描述符)
112  * @param - buf   : 返回给用户空间的数据缓冲区
113  * @param - cnt   : 要读取的数据长度
114  * @param - offt  : 相对于文件首地址的偏移
115  * @return        : 读取的字节数，如果为负值，表示读取失败
116  */
117 static ssize_t led_read(struct file *filp, char __user *buf,
                            size_t cnt, loff_t *offt)
118 {
119     return 0;
120 }
121
122 /*
```

```
123  * @description   : 向设备写数据
124  * @param - filp  : 设备文件，表示打开的文件描述符
125  * @param - buf   : 要写给设备写入的数据
126  * @param - cnt   : 要写入的数据长度
127  * @param - offt  : 相对于文件首地址的偏移
128  * @return        : 写入的字节数，如果为负值，表示写入失败
129  */
130 static ssize_t led_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
131 {
132     int retvalue;
133     unsigned char databuf[1];
134     unsigned char ledstat;
135 
136     retvalue = copy_from_user(databuf, buf, cnt);
137     if(retvalue < 0) {
138         printk("kernel write failed!\r\n");
139         return -EFAULT;
140     }
141 
142     ledstat = databuf[0];       /* 获取状态值   */
143 
144     if(ledstat == LEDON) {
145         led_switch(LEDON);      /* 打开 LED 灯   */
146     } else if(ledstat == LEDOFF) {
147         led_switch(LEDOFF);     /* 关闭 LED 灯   */
148     }
149     return 0;
150 }
151 
152 /*
153  * @description   : 关闭/释放设备
154  * @param - filp  : 要关闭的设备文件(文件描述符)
155  * @return        : 0 成功;其他 失败
156  */
157 static int led_release(struct inode *inode, struct file *filp)
158 {
159     return 0;
160 }
161 
162 /* 设备操作函数 */
163 static struct file_operations newchrled_fops = {
164     .owner = THIS_MODULE,
```

```
165     .open = led_open,
166     .read = led_read,
167     .write = led_write,
168     .release =  led_release,
169 };
170
171 /*
172  * @description   : 驱动出口函数
173  * @param         : 无
174  * @return        : 无
175  */
176 static int __init led_init(void)
177 {
178     u32 val = 0;
179     int ret;
180
181     /* 初始化 LED */
182     /* 1、寄存器地址映射 */
183     MPU_AHB4_PERIPH_RCC_PI = ioremap(RCC_MP_AHB4ENSETR, 4);
184     GPIOI_MODER_PI = ioremap(GPIOI_MODER, 4);
185     GPIOI_OTYPER_PI = ioremap(GPIOI_OTYPER, 4);
186     GPIOI_OSPEEDR_PI = ioremap(GPIOI_OTYPER, 4);
187     GPIOI_PUPDR_PI = ioremap(GPIOI_PUPDR, 4);
188     GPIOI_BSRR_PI = ioremap(GPIOI_BSRR, 4);
189
190     /* 2、使能 PI 时钟 */
191     val = readl(MPU_AHB4_PERIPH_RCC_PI);
192     val &= ~(0X1 << 8);  /* 清除以前的设置    */
193     val |= (0X1 << 8);   /* 设置新值          */
194     writel(val, MPU_AHB4_PERIPH_RCC_PI);
195
196     /* 3、设置 PI0 通用的输出模式。*/
197     val = readl(GPIOI_MODER_PI);
198     val &= ~(0X3 << 0);  /* bit0:1 清零      */
199     val |= (0X1 << 0);   /* bit0:1 设置 01   */
200     writel(val, GPIOI_MODER_PI);
201
202     /* 3、设置 PI0 为推挽模式。*/
203     val = readl(GPIOI_OTYPER_PI);
204     val &= ~(0X1 << 0); /* bit0 清零，设置为上拉*/
205     writel(val, GPIOI_OTYPER_PI);
206
207     /* 4、设置 PI0 为高速。*/
```

```
208     val = readl(GPIOI_OSPEEDR_PI);
209     val &= ~(0X3 << 0);  /* bit0:1 清零         */
210     val |= (0x2 << 0);   /* bit0:1 设置为 10    */
211     writel(val, GPIOI_OSPEEDR_PI);
212
213     /* 5、设置 PI0 为上拉。*/
214     val = readl(GPIOI_PUPDR_PI);
215     val &= ~(0X3 << 0);  /* bit0:1 清零         */
216     val |= (0x1 << 0);   /*bit0:1 设置为 01     */
217     writel(val,GPIOI_PUPDR_PI);
218
219     /* 6、默认关闭 LED */
220     val = readl(GPIOI_BSRR_PI);
221     val |= (0x1 << 0);
222     writel(val, GPIOI_BSRR_PI);
223
224
225     /* 注册字符设备驱动 */
226     /* 1、创建设备号 */
227     if (newchrled.major) {      /*  定义了设备号 */
228         newchrled.devid = MKDEV(newchrled.major, 0);
229         ret = register_chrdev_region(newchrled.devid, NEWCHRLED_CNT,
                                         NEWCHRLED_NAME);
230         if(ret < 0) {
231             pr_err("cannot register %s char driver
[ret=%d]\n",NEWCHRLED_NAME, NEWCHRLED_CNT);
232             goto fail_map;
233         }
234     } else {                        /* 没有定义设备号 */
235         ret = alloc_chrdev_region(&newchrled.devid, 0, NEWCHRLED_CNT,
                                  NEWCHRLED_NAME);  /* 申请设备号 */
236         if(ret < 0) {
237             pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                       NEWCHRLED_NAME, ret);
238             goto fail_map;
239         }
240         newchrled.major = MAJOR(newchrled.devid);   /* 获取主设备号 */
241         newchrled.minor = MINOR(newchrled.devid);   /* 获取次设备号 */
242     }
243     printk("newcheled major=%d,minor=%d\r\n",newchrled.major,
              newchrled.minor);
244
245     /* 2、初始化 cdev */
```

```
246     newchrled.cdev.owner = THIS_MODULE;
247     cdev_init(&newchrled.cdev, &newchrled_fops);
248 
249     /* 3、添加一个 cdev */
250     ret = cdev_add(&newchrled.cdev, newchrled.devid, NEWCHRLED_CNT);
251     if(ret < 0)
252         goto del_unregister;
253 
254     /* 4、创建类 */
255     newchrled.class = class_create(THIS_MODULE, NEWCHRLED_NAME);
256     if (IS_ERR(newchrled.class)) {
257         goto del_cdev;
258     }
259 
260     /* 5、创建设备 */
261     newchrled.device = device_create(newchrled.class, NULL,
                                newchrled.devid, NULL, NEWCHRLED_NAME);
262     if (IS_ERR(newchrled.device)) {
263         goto destroy_class;
264     }
265 
266     return 0;
267 
268 destroy_class:
269     class_destroy(newchrled.class);
270 del_cdev:
271     cdev_del(&newchrled.cdev);
272 del_unregister:
273     unregister_chrdev_region(newchrled.devid, NEWCHRLED_CNT);
274 fail_map:
275     led_unmap();
276     return -EIO;
277 
278 }
279 
280 /*
281  * @description   : 驱动出口函数
282  * @param         : 无
283  * @return        : 无
284  */
285 static void __exit led_exit(void)
286 {
287     /* 取消映射 */
```

```
288     led_unmap();
289
290     /* 注销字符设备驱动 */
291     cdev_del(&newchrled.cdev);/*  删除 cdev */
292     unregister_chrdev_region(newchrled.devid, NEWCHRLED_CNT);
293
294     device_destroy(newchrled.class, newchrled.devid);
295     class_destroy(newchrled.class);
296 }
297
298 module_init(led_init);
299 module_exit(led_exit);
300 MODULE_LICENSE("GPL");
301 MODULE_AUTHOR("ALIENTEK");
302 MODULE_INFO(intree, "Y");
```

第 25 行，宏 NEWCHRLED_CNT 表示设备数量，在申请设备号或者向 Linux 内核添加字符设备的时候需要设置设备数量，一般我们一个驱动一个设备，所以这个宏为 1。

第 26 行，宏 NEWCHRLED_NAME 表示设备名字，本实验的设备名为“newchrdev”，为了方便管理，所有使用到设备名字的地方统一使用此宏，当驱动加载成功以后就生成/dev/newchrled 这个设备文件。

第 51~58 行，创建设备结构体 newchrled_dev。

第 60 行，定义一个设备结构体变量 newchrdev，此变量表示 led 设备。

第 103~107 行，在 led_open 函数中设置文件的私有数据 private_data 指向 newchrdev。

第 227~264 行，根据前面讲解的方法在驱动入口函数 led_init 中申请设备号、添加字符设备、创建类和设备。本实验我们采用动态申请设备号的方法，第 243 行使用 printk 在终端上显示出申请到的主设备号和次设备号。

第 291~295 行，根据前面讲解的方法，在驱动出口函数 led_exit 中注销字符新设备、删除类和设备。

总体来说 newchrled.c 文件中的内容不复杂，LED 灯驱动部分的程序和上一章一样。重点就是使用了新的字符设备驱动方法。

22.5.2 编写测试 APP

本章直接使用上一章的测试 APP，将上一章的 ledApp.c 文件复制到本章实验工程下即可。

22.6 运行测试

22.6.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第二十章实验基本一样，只是将 obj-m 变量的值改为 newchrled.o，Makefile 内容如下所示：

示例代码 22.6.1.1 Makefile 文件

```
1 KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
```

```
......
4  obj-m := newchrled.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 newchrled.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“newchrled.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 ledApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc ledApp.c -o ledApp
```

编译成功以后就会生成 ledApp 这个应用程序。

22.6.2 运行测试

将上一小节编译出来的 newchrled.ko 和 ledApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 newchrled.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe newchrled      //加载驱动
```

驱动加载成功以后会输出申请到的主设备号和次设备号，如图 22.6.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe newchrled
newcheled major=241,minor=0
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 22.6.2.1 申请到的主设备号和次设备号

从图 22.6.2.1 可以看出，申请到的主设备号为 241，次设备号为 0。驱动加载成功以后会自动在/dev 目录下创建设备节点文件/dev/newchrdev，输入如下命令查看/dev/newchrdev 这个设备节点文件是否存在：

```
ls /dev/newchrled -l
```

结果如图 22.6.2.2 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls /dev/newchrled -l
crw-rw----    1 root     root      241,   0 Jan 16 02:38 /dev/newchrled
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 22.6.2.2 /dev/newchrled 设备节点文件

从图 22.6.2.2 中可以看出，/dev/newchrled 这个设备文件存在，而且主设备号为 241，此设备号为 0，说明设备节点文件创建成功。

驱动节点创建成功以后就可以使用 ledApp 软件来测试驱动是否工作正常，输入如下命令打开 LED 灯：

```
./ledApp /dev/newchrled 1       //打开 LED 灯
```

输入上述命令以后观察 STM32MP157D-ATK 开发板上的红色 LED 灯是否点亮，如果点亮的话说明驱动工作正常。在输入如下命令关闭 LED 灯：

```
./ledApp   /dev/newchrled 0     //关闭 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯是否熄灭。如果要卸载驱动的话输入如下命

令即可：

```
rmmod newchrled
```

第二十三章 Linux 设备树

在前面章节中我们多次提到“设备树”这个概念和创建自己的设备树。但是并没有在 TF-A 和 uboot 里说设备树的原理，主要是一开始就讲解设备树不利于嵌入式 Linux 入门，对于大多数嵌入式开发人员来说设备树是个全新的概念。

本章我们就来详细的谈一谈设备树。掌握设备树是 Linux 驱动开发人员必备的技能！因为在新版本的 Linux 中，ARM 相关的驱动全部采用了设备树(也有支持老式驱动的，比较少)，最新出 CPU 在系统启动的时候就支持设备树，比如我们的 MP1 系列、NXP 的 I.MX8 系列等。我们所使用的 Linux 版本为 5.4.31，其支持设备树，所以正点原子 MP1 开发板的所有 Linux 驱动都是基于设备树的。本章我们就来了解一下设备树的起源、重点学习一下设备树语法。

23.1 什么是设备树？

设备树(Device Tree)，将这个词分开就是“设备”和“树”，描述设备树的文件叫做 DTS(Device Tree Source)，这个 DTS 文件采用树形结构描述板级设备，也就是开发板上的设备信息，比如 CPU 数量、 内存基地址、IIC 接口上接了哪些设备、SPI 接口上接了哪些设备等等，如图 23.1.1 所示：

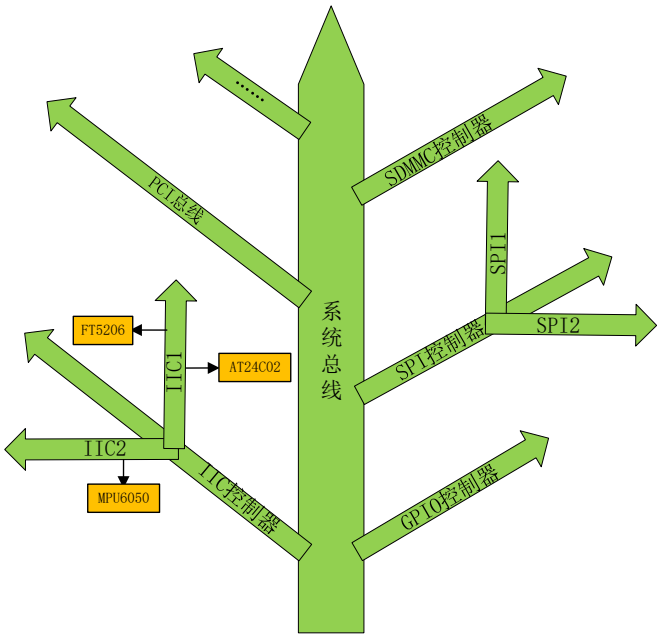


图 23.1.1 设备树结构示意图

在图 23.1.1 中，树的主干就是系统总线，IIC 控制器、GPIO 控制器、SPI 控制器等都是接到系统主线上的分支。IIC 控制器有分为 IIC1 和 IIC2 两种，其中 IIC1 上接了 FT5206 和 AT24C02 这两个 IIC 设备，IIC2 上只接了 MPU6050 这个设备。DTS 文件的主要功能就是按照图 23.1.1 所示的结构来描述板子上的设备信息，DTS 文件描述设备信息是有相应的语法规则要求的，稍后我们会详细的讲解 DTS 语法规则。

在 3.x 版本(具体哪个版本笔者也无从考证)以前的 Linux 内核中 ARM 架构并没有采用设备树。在没有设备树的时候 Linux 是如何描述 ARM 架构中的板级信息呢？在 Linux 内核源码中大量的 arch/arm/mach-xxx 和 arch/arm/plat-xxx 文件夹，这些文件夹里面的文件就是对应平台下的板级信息。比如在 arch/arm/mach-s3c24xx/mach-smdk2440.c 中有如下内容(有缩减)：

示例代码 23.1.1 mach-smdk2440.c 文件代码段

```
90  static struct s3c2410fb_display smdk2440_lcd_cfg __initdata = {
91
92      .lcdcon5    = S3C2410_LCDCON5_FRM565 |
93              S3C2410_LCDCON5_INVVLINE |
94              S3C2410_LCDCON5_INVVFRAME |
95              S3C2410_LCDCON5_PWREN |
96              S3C2410_LCDCON5_HWSWP,
.......
113 };
114
```

```
115 static struct s3c2410fb_mach_info smdk2440_fb_info __initdata = {
116     .displays   = &smdk2440_lcd_cfg,
117     .num_displays   = 1,
118     .default_display = 0,
......
133 };
134
135 static struct platform_device *smdk2440_devices[] __initdata = {
136     &s3c_device_ohci,
137     &s3c_device_lcd,
138     &s3c_device_wdt,
139     &s3c_device_i2c0,
140     &s3c_device_iis,
141 };
```

上述代码中的结构体变量 smdk2440_fb_info 就是描述 SMDK2440 这个开发板上的 LCD 信息的，结构体指针数组 smdk2440_devices 描述的 SMDK2440 这个开发板上的所有平台相关信息。这个仅仅是使用 2440 这个芯片的 SMDK2440 开发板下的 LCD 信息，SMDK2440 开发板还有很多的其他外设硬件和平台硬件信息。使用 2440 这个芯片的板子有很多，每个板子都有描述相应板级信息的文件，这仅仅只是一个 2440。随着智能手机的发展，每年新出的 ARM 架构芯片少说都在数十、甚至数百款，Linux 内核下板级信息文件将会成指数级增长！这些板级信息文件都是.c 或.h 文件，都会被硬编码进 Linux 内核中，导致 Linux 内核“虚胖”。就好比你喜欢吃自助餐，然后花了 100 多块钱到一家宣传看着很不错的自助餐厅，结果你想吃的牛排、海鲜、烤肉基本没多少，全都是一些凉菜、炒面、西瓜、饮料等小吃，相信你此时肯定会脱口而出一句“F*k!”、“骗子！”。同样的，当 Linux 之父 linus 看到 ARM 社区向 Linux 内核添加了大量“无用”、冗余的板级信息文件，不禁的发出了一句“This whole ARM thing is a f*cking pain in the ass”。从此以后 ARM 社区就引入了 PowerPC 等架构已经采用的设备树(Flattened Device Tree)，将这些描述板级硬件信息的内容都从 Linux 内中分离开来，用一个专属的文件格式来描述，这个专属的文件就叫做设备树，文件扩展名为.dts。一个 SOC 可以作出很多不同的板子，这些不同的板子肯定是有共同的信息，将这些共同的信息提取出来作为一个通用的文件，其他的.dts 文件直接引用这个通用文件即可，这个通用文件就是.dtsi 文件，类似于 C 语言中的头文件。一般.dts 描述板级信息(也就是开发板上有哪些 IIC 设备、SPI 设备等)，.dtsi 描述 SOC 级信息(也就是 SOC 有几个 CPU、主频是多少、各个外设控制器信息等)。

这个就是设备树的由来，简而言之就是，Linux 内核中 ARM 架构下有太多的冗余的垃圾板级信息文件，导致 linus 震怒，然后 ARM 社区引入了设备树。

23.2 DTS、DTB 和 DTC

上一小节说了，设备树源文件扩展名为.dts，但是我们在前面移植 Linux 的时候却一直在使用.dtb 文件，那么 DTS 和 DTB 这两个文件是什么关系呢？DTS 是设备树源码文件，DTB 是将 DTS 编译以后得到的二进制文件。将.c 文件编译为.o 需要用到 gcc 编译器，那么将.dts 编译为.dtb 需要用到 DTC 工具！DTC 工具源码在 Linux 内核的 scripts/dtc 目录下，scripts/dtc/Makefile 文件内容如下：

示例代码 23.2.1 scripts/dtc/Makefile 文件代码段

```
4  hostprogs-$(CONFIG_DTC) := dtc
```

```
5  always      := $(hostprogs-y)
6
7  dtc-objs := dtc.o flattree.o fstree.o data.o livetree.o treesource.o
8          srcpos.o checks.o util.o
9  dtc-objs += dtc-lexer.lex.o dtc-parser.tab.o
......
```

可以看出，DTC 工具依赖于 dtc.c、flattree.c、fstree.c 等文件，最终编译并链接出 DTC 这个主机文件。如果要编译 DTS 文件的话只需要进入到 Linux 源码根目录下，然后执行如下命令：

```
make all
```

或者：

```
make dtbs
```

“make all”命令是编译 Linux 源码中的所有东西，包括 uImage，.ko 驱动模块以及设备树，如果只是编译设备树的话建议使用“make dtbs”命令，“make dtbs”会编译选中的所有设备树文件。如果只要编译指定的某个设备树，比如 ST 官方编写的“stm32mp157d-ed1.dts”，可以输入如下命令：

```
make stm32mp157d-ed1.dtb
```

结果如图 23.2.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31$ make stm32mp157d-ed1.dtb
  DTC     arch/arm/boot/dts/stm32mp157d-ed1.dtb
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31$
```

图 23.2.1 编译单独的设备树

基于 ARM 架构的 SOC 有很多种，一种 SOC 又可以制作出很多款板子，每个板子都有一个对应的 DTS 文件，那么如何确定编译哪一个 DTS 文件呢？我们就以 STM32MP1 这款芯片对应的板子为例来看一下，打开 arch/arm/boot/dts/Makefile，有如下内容：

示例代码 23.2.2 arch/arm/boot/dts/Makefile 文件代码段

```
981 dtb-$(CONFIG_ARCH_STM32) += \
982     stm32f429-disco.dtb \
983     stm32f469-disco.dtb \
984     stm32f746-disco.dtb \
985     stm32f769-disco.dtb \
986     stm32429i-eval.dtb \
987     stm32746g-eval.dtb \
988     stm32h743i-eval.dtb \
989     stm32h743i-disco.dtb \
990     stm32mp157a-avenger96.dtb \
991     stm32mp157a-dk1.dtb \
992     stm32mp157d-dk1.dtb \
993     stm32mp157c-dk2.dtb \
994     stm32mp157f-dk2.dtb \
995     stm32mp157c-dk2-a7-examples.dtb \
996     stm32mp157c-dk2-m4-examples.dtb \
997     stm32mp157f-dk2-a7-examples.dtb \
998     stm32mp157f-dk2-m4-examples.dtb \
```

```
999     stm32mp157a-ed1.dtb \
1000    stm32mp157c-ed1.dtb \
1001    stm32mp157d-ed1.dtb \
1002    stm32mp157f-ed1.dtb \
1003    stm32mp157a-ev1.dtb \
1004    stm32mp157c-ev1.dtb \
1005    stm32mp157d-ev1.dtb \
1006    stm32mp157f-ev1.dtb \
1007    stm32mp157d-atk.dtb \
1008    stm32mp157c-ev1-a7-examples.dtb \
1009    stm32mp157c-ev1-m4-examples.dtb \
1010    stm32mp157f-ev1-a7-examples.dtb \
1011    stm32mp157f-ev1-m4-examples.dtb
```

可以看出，当选中 STM32MP1 这个 SOC 以后(CONFIG_ARCH_STM32=y)，所有使用到 STM32MP1 这个 SOC 的板子对应的.dts 文件都会被编译为.dtb。如果我们使用 STM32MP1 新做了一个板子，只需要新建一个此板子对应的.dts 文件，然后将对应的.dtb 文件名添加到 dtb-$(CONFIG_ARCH_STM32)下，这样在编译设备树的时候就会将对应的.dts 编译为二进制的.dtb 文件。

示例代码 23.2.2 中 1007 行就是我们在给正点原子的开发板移植 Linux 系统的时候添加的设备树。关于.dtb 文件怎么使用这里就不多说了，前面讲解 TF-A 移植、Uboot 移植、Linux 内核移植的时候已经无数次的提到如何使用.dtb 文件了(uboot 中使用 bootz 或 bootm 命令向 Linux 内核传递二进制设备树文件(.dtb))。

23.3 DTS 语法

虽然我们基本上不会从头到尾重写一个.dts 文件，大多时候是直接在 SOC 厂商提供的.dts 文件上进行修改。但是 DTS 文件语法我们还是需要详细的学习一遍，因为后续工作中肯定需要修改.dts 文件。大家不要看到要学习新的语法就觉得会很复杂，DTS 语法非常的人性化，是一种 ASCII 文本文件，不管是阅读还是修改都很方便。

本节我们就以 stm32mp157d-atk.dts 这个文件为例来讲解一下 DTS 语法。关于设备树详细的语法规则请参考《Devicetree SpecificationV0.2.pdf》和《Power_ePAPR_APPROVED_v1.12.pdf》这两份文档，此两份文档已经放到了开发板光盘中，路径为：4、参考资料->Devicetree SpecificationV0.2.pdf，4、参考资料-> Power_ePAPR_APPROVED_v1.12.pdf

23.3.1 .dtsi 头文件

和 C 语言一样，设备树也支持头文件，设备树的头文件扩展名为.dtsi。在 stm32mp157d-atk.dts 中有如下所示内容：

示例代码 23.3.1.1 stm32mp157d-atk.dts 文件代码段

```
8  #include "stm32mp157.dtsi"
9  #include "stm32mp15xd.dtsi"
10 #include "stm32mp15-pinctrl.dtsi"
11 #include "stm32mp15xxaa-pinctrl.dtsi"
12 #include "stm32mp157-m4-srm.dtsi"
13 #include "stm32mp157-m4-srm-pinctrl.dtsi"
```

```
14 #include "stm32mp157d-atk.dtsi"
```

第 8~14 行，使用“#include”来引用“stm32mp15*.dtsi”这些.dtsi 头文件。

设备树里面除了可以通过“#include”来引用.dtsi 文件，也可以引用.h 文件头文件，大家打开 stm32mp157f-dk2.dts 这个文件，找到如下代码：

示例代码 23.3.1.2 stm32mp157f-dk2.dts 文件代码段

```
14 #include <dt-bindings/rtc/rtc-stm32.h>
```

.dts 文件不仅可以应用 C 语言里面的.h 头文件，甚至也可以引用.dts 文件，打开 stm32mp157c-ev1.dts 这个文件，此文件中有如下内容：

示例代码 23.3.1.3 stm32mp157c-ev1.dts 文件代码段

```
8 #include "stm32mp157c-ed1.dts"
```

可以看出，示例代码 23.3.1.3 中直接引用了.dts 文件，因此在.dts 设备树文件中，可以通过“#include”来引用.h、.dtsi 和.dts 文件。只是，我们在编写设备树头文件的时候最好选择.dtsi 后缀。

一般.dtsi 文件用于描述 SOC 的内部外设信息，比如 CPU 架构、主频、外设寄存器地址范围，比如 UART、IIC 等等。如果一个系列里有多个 SOC 就会把相同内部外设信息提炼到一个.dtsi 文件里，这样为了减少代码的冗余。目前 STM32MP1 系列里有 stm32mp151、stm32mp153 和 stm32mp157 这三款 SOC，其中 151 是外设最少的，153 和 157 的外设是在 151 的基础上逐渐增加的。因此 151 就相当于“基类”，153 和 157 是在 151 基础上得到的“派生类”。因此 ST 就把最基本的外设资源都写在 stm32mp151.dtsi 文件里。stm32mp151.dtsi 就是描述 151、153 和 157 共有的外设信息的，内容如下：

示例代码 23.3.1.4 stm32mp151.dtsi 文件代码段

```
6   #include <dt-bindings/interrupt-controller/arm-gic.h>
7   #include <dt-bindings/clock/stm32mp1-clks.h>
8   #include <dt-bindings/gpio/gpio.h>
9   #include <dt-bindings/reset/stm32mp1-resets.h>
10  #include <dt-bindings/thermal/thermal.h>
11
12
13  / {
14      #address-cells = <1>;
15      #size-cells = <1>;
16
17      cpus {
18          #address-cells = <1>;
19          #size-cells = <0>;
20
21          cpu0: cpu@0 {
22              compatible = "arm,cortex-a7";
......
29              nvmem-cell-names = "part_number";
30              #cooling-cells = <2>;
31          };
32      };
```

```
33
34      cpu0_opp_table: cpu0-opp-table {
35          compatible = "operating-points-v2";
36          opp-shared;
37      };
38  };
......
469         spi2: spi@4000b000 {
470             #address-cells = <1>;
471             #size-cells = <0>;
472             compatible = "st,stm32h7-spi";
473             reg = <0x4000b000 0x400>;
474             interrupts = <GIC_SPI 36 IRQ_TYPE_LEVEL_HIGH>;
475             clocks = <&rcc SPI2_K>;
476             resets = <&rcc SPI2_R>;
477             dmas = <&dmamux1 39 0x400 0x01>,
478                    <&dmamux1 40 0x400 0x01>;
479             dma-names = "rx", "tx";
480             power-domains = <&pd_core>;
481             status = "disabled";
482         };
```

示例代码 23.3.1.4 中第 17~33 行就是 CPU0 这个设备节点信息，这个节点信息描述了 STM32MP151 这颗 SOC 所使用的 CPU 信息，比如架构是 cortex-A7,CPU 的时钟等等,stm32mp151.dtsi 文件中不仅仅描述了 CPU0 这个节点信息，STM32MP151 这颗 SOC 所有的外设都描述的清清楚楚，比如 SAI1~4, U(S)ART1~8，SDIO1~3 等等，关于这些设备节点信息的具体内容我们后面具体章节里面再详细的讲解。

23.3.2 设备节点

设备树是采用树形结构来描述板子上的设备信息的文件，每个设备都是一个节点，叫做设备节点，每个节点都通过一些属性信息来描述节点信息，属性就是键—值对。以下文件是结合 ST 官方的设备树缩减出来设备树文件内容：

示例代码 23.3.2.1 设备树模板

```
1   / {
2       #address-cells = <1>;
3       #size-cells = <1>;
4
5       aliases {
6           serial0 = &uart4;
7       };
8
9       cpus {
10          #address-cells = <1>;
11          #size-cells = <0>;
```

```
12
13          cpu0: cpu@0 {
14              compatible = "arm,cortex-a7";
15              device_type = "cpu";
16              reg = <0>;
17              clocks = <&scmi0_clk CK_SCMI0_MPU>;
18              clock-names = "cpu";
19              operating-points-v2 = <&cpu0_opp_table>;
20              nvmem-cells = <&part_number_otp>;
21              nvmem-cell-names = "part_number";
22              #cooling-cells = <2>;
23          };
24      };
25
26      soc {
27          compatible = "simple-bus";
28          #address-cells = <1>;
29          #size-cells = <1>;
30          interrupt-parent = <&intc>;
31          ranges;
32
33          sram: sram@10000000 {
34              compatible = "mmio-sram";
35              reg = <0x10000000 0x60000>;
36              #address-cells = <1>;
37              #size-cells = <1>;
38              ranges = <0 0x10000000 0x60000>;
39          };
40      };
41  };
```

第 1 行，“/”是根节点，每个设备树文件只有一个根节点。细心的同学应该会发现，在内核移植设备树的时候，我们新建的 stm32mp157d-atk.dts 和 stm32mp157d-atk.dtsi 这两个文件都有一个“/”根节点，这样不会出错吗？不会的，因为这两个“/”根节点的内容会合并成一个根节点。

第 5、9 和 26 行，aliases、cpus 和 soc 是三个子节点，33 行 sram 是 soc 的子节点。在设备树中节点命名格式如下：

```
node-name@unit-address
```

其中“node-name”是节点名字，为 ASCII 字符串，节点名字应该能够清晰的描述出节点的功能，比如“uart1”就表示这个节点是 UART1 外设。“unit-address”一般表示设备的地址或寄存器首地址，如果某个节点没有地址或者寄存器的话“unit-address”可以不要，比如“cpu@0”、“sram@10000000”、“soc”。

但是我们在示例代码 23.3.2.1 中我们看到的节点命名却如下所示：

```
cpu0:cpu@0
```

上述命令并不是“node-name@unit-address”这样的格式，而是用“：”隔开成了两部分，“：”前面是节点标签(label)，“：”后面的才是节点名字，格式如下所示：

```
label: node-name@unit-address
```

引入 label 的目的就是为了方便访问节点，可以直接通过&label 来访问这个节点，比如通过&cpu0 就可以访问“cpu@0”这个节点，而不需要输入完整的节点名字。再比如节点 “sram: sram@10000000”，节点 label 是 sram，而节点名字就很长了，为“sram@10000000”。很明显通过&sram 来访问“sram@10000000”这个节点要方便很多！

第 13 行，cpu0 也是一个节点，只是 cpu0 是 cpus 的子节点。

每个节点都有不同属性，不同的属性又有不同的内容，属性都是键值对，值可以为空或任意的字节流。设备树源码中常用的几种数据形式如下所示：

1、字符串

```
compatible = "arm,cortex-a7";
```

上述代码设置 compatible 属性的值为字符串“arm,cortex-a7”。

2、32 位无符号整数

```
reg = <0>;
```

上述代码设置 reg 属性的值为 0，reg 的值也可以设置为一组值，比如：

```
reg = <0 0x123456 100>;
```

3、字符串列表

属性值也可以为字符串列表，字符串和字符串之间采用“,”隔开，如下所示：

```
compatible = "st,stm32mp157d-atk", "st,stm32mp157";
```

上述代码设置属性 compatible 的值为“st,stm32mp157d-atk”和“st,stm32mp157”。

23.3.3 标准属性

节点是由一堆的属性组成，节点都是具体的设备，不同的设备需要的属性不同，用户可以自定义属性。除了用户自定义属性，有很多属性是标准属性，Linux 下的很多外设驱动都会使用这些标准属性，本节我们就来学习一下几个常用的标准属性。

1、compatible 属性

compatible 属性也叫做“兼容性”属性，这是非常重要的一个属性！compatible 属性的值是一个字符串列表，compatible 属性用于将设备和驱动绑定起来。字符串列表用于选择设备所要使用的驱动程序，compatible 属性的值格式如下所示：

```
"manufacturer,model"
```

其中 manufacturer 表示厂商，model 一般是模块对应的驱动名字。比如 stm32mp15xx-dkx.dtsi 中有一个音频设备节点，这个节点的音频芯片采用的 Cirrus Logic 公司出品的 cs42l51，compatible 属性值如下：

```
compatible = "cirrus,cs42l51";
```

属性值为“cirrus,cs42l51”，其中‘cirrus’表示厂商是 Cirrus Logic，“cs42l51”表示驱动模块名字。compatible 也可以多个属性值。比如：

```
compatible =   "cirrus,my_cs42l51","cirrus,cs42l51";
```

这样我们的设备就有两个属性值，这个设备首先使用第一个兼容值在 Linux 内核里面查找，看看能不能找到与之匹配的驱动文件，如果没有找到的话就使用第二个兼容值查，以此类推，直到查找完 compatible 属性中的所有值。

一般驱动程序文件都会有一个 OF 匹配表，此 OF 匹配表保存着一些 compatible 值，如果设备节点的 compatible 属性值和 OF 匹配表中的任何一个值相等，那么就表示设备可以使用这个驱动。比如在文件 cs42l51.c 中有如下内容：

示例代码 23.3.3.1 cs42l51.c 文件代码段

```
799 const struct of_device_id cs42l51_of_match[] = {
800     { .compatible = "cirrus,cs42l51", },
801     { }
802 };
```

数组 cs42l51_of_match 就是 cs42l51.c 这个驱动文件的匹配表，此匹配表只有一个匹配值“cirrus,cs42l51”。如果在设备树中有哪个节点的 compatible 属性值与此相等，那么这个节点就会使用此驱动文件。

2、model 属性

model 属性值也是一个字符串，一般 model 属性描述开发板的名字或者设备模块信息，比如名字什么的，比如：

```
model = "STMicroelectronics STM32MP157C-DK2 Discovery Board";
```

3、status 属性

status 属性看名字就知道是和设备状态有关的，status 属性值也是字符串，字符串是设备的状态信息，可选的状态如表 23.3.3.1 所示：

值	描述
“okay”	表明设备是可操作的。
“disabled”	表明设备当前是不可操作的，但是在未来可以变为可操作的，比如热插拔设备插入以后。至于 disabled 的具体含义还要看设备的绑定文档。
“fail”	表明设备不可操作，设备检测到了一系列的错误，而且设备也不大可能变得可操作。
“fail-sss”	含义和“fail”相同，后面的 sss 部分是检测到的错误内容。

表 23.3.3.1 status 属性值表

4、#address-cells 和#size-cells 属性

这两个属性的值都是无符号 32 位整形，#address-cells 和#size-cells 这两个属性可以用在任何拥有子节点的设备中，用于描述子节点的地址信息。#address-cells 属性值决定了子节点 reg 属性中地址信息所占用的字长(32 位)，#size-cells 属性值决定了子节点 reg 属性中长度信息所占的字长(32 位)。#address-cells 和#size-cells 表明了子节点应该如何编写 reg 属性值，一般 reg 属性都是和地址有关的内容，和地址相关的信息有两种：起始地址和地址长度，reg 属性的格式为：

```
reg = <address1 length1 address2 length2 address3 length3……>
```

每个“address length”组合表示一个地址范围，其中 address 是起始地址，length 是地址长度，#address-cells 表明 address 这个数据所占用的字长，#size-cells 表明 length 这个数据所占用的字长，比如:

示例代码 23.3.3.2 #address-cells 和#size-cells 属性

```
1   cpus {
2       #address-cells = <1>;
3       #size-cells = <0>;
4
```

```
5       cpu0: cpu@0 {
6           compatible = "arm,cortex-a7";
7           device_type = "cpu";
8           reg = <0>;
9           clocks = <&scmi0_clk CK_SCMI0_MPU>;
10          clock-names = "cpu";
11          operating-points-v2 = <&cpu0_opp_table>;
12          nvmem-cells = <&part_number_otp>;
13          nvmem-cell-names = "part_number";
14          #cooling-cells = <2>;
15      };
16  };
17
18  scmi_sram: sram@2ffff000 {
19      compatible = "mmio-sram";
20      reg = <0x2ffff000 0x1000>;
21      #address-cells = <1>;
22      #size-cells = <1>;
23      ranges = <0 0x2ffff000 0x1000>;
24
25      scmi0_shm: scmi_shm@0 {
26          reg = <0 0x80>;
27      };
28  };
```

第 2，3 行，节点 cpus 的#address-cells = <1>，#size-cells = <0>，说明 cpus 的子节点 reg 属性中起始地址所占用的字长为 1，地址长度所占用的字长为 0。

第 8 行，子节点 cpu0: cpu@0 的 reg 属性值为 <0>，因为父节点设置了#address-cells = <1>，#size-cells = <0>，因此 addres=0，没有 length 的值，相当于设置了起始地址，而没有设置地址长度。

第 21，22 行，设置 scmi_sram: sram@02ffff000 节点#address-cells = <1>，#size-cells = <1>，说明 scmi_sram: sram@02ffff000 节点起始地址长度所占用的字长为 1，地址长度所占用的字长也为 1。

第 26 行，子节点 scmi0_shm: scmi_shm@0 的 reg 属性值为<0x0 0x80>，因为父节点设置了#address-cells = <1>，#size-cells = <1>，address= 0x0，length= 0x80，相当于设置了起始地址为 0x0，地址长度为 0x80。

5、reg 属性

reg 属性前面已经提到过了，reg 属性的值一般是(address，length)对。reg 属性一般用于描述设备地址空间资源信息或者设备地址信息，比如某个外设的寄存器地址范围信息，或者 IIC 器件的设备地址等，比如在 stm32mp151.dtsi 中有如下内容：

示例代码 23.3.3.3 uart5 节点信息

```
576         uart5: serial@40011000 {
577             compatible = "st,stm32h7-uart";
578             reg = <0x40011000 0x400>;
```

```
579                interrupts-extended = <&exti 31 IRQ_TYPE_LEVEL_HIGH>;
580                clocks = <&rcc UART5_K>;
581                resets = <&rcc UART5_R>;
582                wakeup-source;
583                power-domains = <&pd_core>;
584                dmas = <&dmamux1 65 0x400 0x5>,
585                       <&dmamux1 66 0x400 0x1>;
586                dma-names = "rx", "tx";
587                status = "disabled";
588            };
```

uart5 节点描述了 stm32mp1 系列芯片的 UART5 相关信息，重点是第 578 行的 reg 属性。由于 uart5 的父节点“soc”设置了#address-cells = <1>、#size-cells = <1>，因此 reg 属性中 address=0x40011000，length=0x400。查阅《STM32MP157 参考手册》可知，stm32mp157 芯片的 UART5 寄存器首地址为 0x40011000，但是 UART5 的地址长度(范围)并没有 0x400 这么多，这里我们重点是获取 UART5 寄存器首地址。

6、ranges 属性

ranges 属性值可以为空或者按照(child-bus-address,parent-bus-address,length)格式编写的数字矩阵，ranges 是一个地址映射/转换表，ranges 属性每个项目由子地址、父地址和地址空间长度这三部分组成：

child-bus-address：子总线地址空间的物理地址，由父节点的#address-cells 确定此物理地址所占用的字长。

parent-bus-address：父总线地址空间的物理地址，同样由父节点的#address-cells 确定此物理地址所占用的字长。

length：子地址空间的长度，由父节点的#size-cells 确定此地址长度所占用的字长。

如果 ranges 属性值为空值，说明子地址空间和父地址空间完全相同，不需要进行地址转换，对于我们所使用的 stm32mp157 来说，子地址空间和父地址空间完全相同，因此会在 stm32mp151.dtsi 中找到大量的值为空的 ranges 属性，如下所示：

示例代码 23.3.3.4 stm32mp151.dtsi 文件代码段

```
192        soc {
193            compatible = "simple-bus";
194            #address-cells = <1>;
195            #size-cells = <1>;
196            interrupt-parent = <&intc>;
197            ranges;
......
1968       };
```

第 197 行定义了 ranges 属性，但是 ranges 属性值为空。

ranges 属性不为空的示例代码如下所示：

示例代码 23.3.3.5 ranges 属性不为空

```
1   soc {
2       compatible = "simple-bus";
3       #address-cells = <1>;
4       #size-cells = <1>;
```

```
5       interrupt-parent = <&intc>;
6       ranges = <0 0x10000000 0x100000>;
7
8       sram: sram@10000000 {
9           compatible = "mmio-sram";
10          reg = <0x0 0x60000>;
11          #address-cells = <1>;
12          #size-cells = <1>;
13          ranges = <0 0x10000000 0x60000>;
14      };
15  };
```

第 6 行，节点 soc 定义的 ranges 属性，值为<0 0x10000000 0x100000>，此属性值指定了一个 1024KB(0x100000)的地址范围，子地址空间的物理起始地址为 0，父地址空间的物理起始地址为 0x10000000。

第 8 行，sram 是 STM32MP1 内部 RAM 节点，reg 属性定义了 sram 设备寄存器的起始地址为 0，寄存器长度为 0x60000。经过地址转换，sram 设备可以从 0x10000000 开始进行读写操作，0x10000000=0x0 + 0x10000000。

7、name 属性

name 属性值为字符串，name 属性用于记录节点名字，name 属性已经被弃用，不推荐使用 name 属性，一些老的设备树文件可能会使用此属性。

8、device_type 属性

device_type 属性值为字符串，IEEE 1275 会用到此属性，用于描述设备的 FCode，但是设备树没有 FCode，所以此属性也被抛弃了。此属性只能用于 cpu 节点或者 memory 节点。Stm32mp151.dtsi 的 cpu0 节点用到了此属性，内容如下所示：

示例代码 23.3.3.6 stm32mp151.dtsi 文件代码段

```
21          cpu0: cpu@0 {
22              compatible = "arm,cortex-a7";
23              device_type = "cpu";
......
32          };
```

关于标准属性就讲解这么多，其他的比如中断、IIC、SPI 等使用的标准属性等到具体的例程再讲解。

23.3.4 根节点 compatible 属性

每个节点都有 compatible 属性，根节点“/”也不例外，在我们新建的 stm32mp157d-atk.dts 文件中根节点的 compatible 属性内容如下所示：

示例代码 23.3.4.1 stm32mp157d-atk.dts 根节点 compatible 属性

```
16  / {
17      model = "STMicroelectronics STM32MP157C-DK2 Discovery Board";
18      compatible = "st,stm32mp157d-atk", "st,stm32mp157";
......
41  };
```

可以看出，compatible 有两个值：“st,stm32mp157d-atk”和“st,stm32mp157”。前面我们说了，设备节点的 compatible 属性值是为了匹配 Linux 内核中的驱动程序，那么根节点中的 compatible 属性是为了做什么工作的？ 通过根节点的 compatible 属性可以知道我们所使用的设备，一般第一个值描述了所使用的硬件设备名字，比如这里使用的是“stm32mp157d-atk”这个设备，第二个值描述了设备所使用的 SOC，比如这里使用的是“stm32mp157”这颗 SOC。Linux 内核会通过根节点的 compoatible 属性查看是否支持此设备，如果支持的话设备就会启动 Linux 内核。接下来我们就来学习一下 Linux 内核在使用设备树前后是如何判断是否支持某款设备的。

1、 使用设备树之前设备匹配方法

在没有使用设备树以前，uboot 会向 Linux 内核传递一个叫做 machine id 的值，machine id 也就是设备 ID，告诉 Linux 内核自己是个什么设备，看看 Linux 内核是否支持。Linux 内核是支持很多设备的，针对每一个设备(板子)，Linux 内核都用 MACHINE_START 和 MACHINE_END 来定义一个 machine_desc 结构体来描述这个设备，比如在文件 arch/arm/mach-imx/mach-mx35_3ds.c 中有如下定义：

示例代码 23.3.4.2 MX35_3DS 设备

```
506 MACHINE_START(MX35_3DS, "Freescale MX35PDK")
507     /* Maintainer: Freescale Semiconductor, Inc */
508     .atag_offset = 0x100,
509     .map_io = mx35_map_io,
510     .init_early = imx35_init_early,
511     .init_irq = mx35_init_irq,
512     .init_time  = mx35pdk_timer_init,
513     .init_machine = mx35_3ds_init,
514     .reserve = mx35_3ds_reserve,
515     .restart    = mxc_restart,
516 MACHINE_END
```

上述代码就是定义了“Freescale MX35PDK”这个设备，其中 MACHINE_START 和 MACHINE_END 定义在文件 arch/arm/include/asm/mach/arch.h 中，内容如下：

示例代码 23.3.4.3 MACHINE_START 和 MACHINE_END 宏定义

```
81 #define MACHINE_START(_type,_name)          \
82 static const struct machine_desc __mach_desc_##_type \
83  __used                          \
84  __attribute__((__section__(".arch.info.init"))) = { \
85  .nr     = MACH_TYPE_##_type,        \
86  .name       = _name,
87
88 #define MACHINE_END             \
89
```

根据 MACHINE_START 和 MACHINE_END 的宏定义，将示例代码 23.3.4.2 展开后如下所示：

示例代码 23.3.4.3 展开以后

```
1  static const struct machine_desc __mach_desc_MX35_3DS     \
2       __used                                                \
3       __attribute__((__section__(".arch.info.init"))) = {
```

```
4       .nr     = MACH_TYPE_MX35_3DS,
5       .name       = "Freescale MX35PDK",
6       /* Maintainer: Freescale Semiconductor, Inc */
7       .atag_offset = 0x100,
8       .map_io = mx35_map_io,
9       .init_early = imx35_init_early,
10      .init_irq = mx35_init_irq,
11      .init_time  = mx35pdk_timer_init,
12      .init_machine = mx35_3ds_init,
13      .reserve = mx35_3ds_reserve,
14      .restart    = mxc_restart,
15  };
```

从示例代码 23.3.4.3 中可以看出，这里定义了一个 machine_desc 类型的结构体变量__mach_desc_MX35_3DS，这个变量存储在“.arch.info.init”段中。第 4 行的 MACH_TYPE_MX35_3DS 就是“Freescale MX35PDK”这个板子的 machine id。MACH_TYPE_MX35_3DS 定义在文件 include/generated/mach-types.h 中，此文件定义了大量的 machine id，内容如下所示：

示例代码 23.3.4.3 mach-types.h 文件中的 machine id

```
10    #define MACH_TYPE_EBSA110             0
11    #define MACH_TYPE_RISCPC              1
12    #define MACH_TYPE_NEXUSPCI            3
......
1629  #define MACH_TYPE_MX35_3DS            1645
......
5053  #define MACH_TYPE_OMAP3_MRC3D         5114
```

第 1629 行就是 MACH_TYPE_MX35_3DS 的值，为 1645。

前面说了，uboot 会给 Linux 内核传递 machine id 这个参数，Linux 内核会检查这个 machine id，其实就是将 machine id 与示例代码 23.3.4.3 中的这些 MACH_TYPE_XXX 宏进行对比，看看有没有相等的，如果相等的话就表示 Linux 内核支持这个设备，如果不支持的话那么这个设备就没法启动 Linux 内核。

2、使用设备树以后的设备匹配方法

当 Linux 内核引入设备树以后就不再使用 MACHINE_START 了，而是换为了 DT_MACHINE_START。DT_MACHINE_START 也定义在文件 arch/arm/include/asm/mach/arch.h 里面，定义如下：

示例代码 23.3.4.4 DT_MACHINE_START 宏

```
91  #define DT_MACHINE_START(_name, _namestr)       \
92  static const struct machine_desc __mach_desc_##_name \
93   __used                          \
94   __attribute__((__section__(".arch.info.init"))) = { \
95   .nr     = ~0,                  \
96   .name       = _namestr,
97
98  #endif
```

可以看出，DT_MACHINE_START 和 MACHINE_START 基本相同，只是.nr 的设置不同，在 DT_MACHINE_START 里面直接将.nr 设置为~0。说明引入设备树以后不会再根据 machine id 来检查 Linux 内核是否支持某个设备了。

打开文件 arch/arm/mach-stm32/board-dt.c，有如下所示内容：

示例代码 23.3.4.5 stm32mp157 设备

```
14 static const char *const stm32_compat[] __initconst = {
15     "st,stm32f429",
16     "st,stm32f469",
17     "st,stm32f746",
18     "st,stm32f769",
19     "st,stm32h743",
20     "st,stm32mp151",
21     "st,stm32mp153",
22     "st,stm32mp157",
23     NULL
24 };
25
26 DT_MACHINE_START(STM32DT, "STM32 (Device Tree Support)")
27     .dt_compat = stm32_compat,
28 #ifdef CONFIG_ARM_SINGLE_ARMV7M
29     .restart = armv7m_restart,
30 #endif
31 MACHINE_END
```

machine_desc 结构体中有个.dt_compat 成员变量，此成员变量保存着本设备兼容属性，示例代码 23.3.4.5 中设置.dt_compat 为 stm32_compat，此表里面含有 Linux 内核所支持的 soc 兼容值。只要某个设备(板子)根节点“/”的 compatible 属性值与 stm32_compat 表中的任何一个值相等，那么就表示 Linux 内核支持此设备。stm32mp157d-atk.dts 中根节点的 compatible 属性值如下：

```
compatible = "st,stm32mp157d-atk", "st,stm32mp157";
```

其中“st,stm32mp157”与 stm32_compat 中的“st,stm32mp157”匹配，因此正点原子 STM32MP157 开发板可以正常启动 Linux 内核。

接下来我们简单看一下 Linux 内核是如何根据设备树根节点的 compatible 属性来匹配出对应的 machine_desc，Linux 内核调用 start_kernel 函数来启动内核，start_kernel 函数会调用 setup_arch 函数来匹配 machine_desc，setup_arch 函数定义在文件 arch/arm/kernel/setup.c 中，函数内容如下(有缩减)：

示例代码 23.3.4.6 setup_arch 函数内容

```
1076 void __init setup_arch(char **cmdline_p)
1077 {
1078     const struct machine_desc *mdesc;
1079
1080     setup_processor();
1081     mdesc = setup_machine_fdt(__atags_pointer);
1082     if (!mdesc)
```

```
1083          mdesc = setup_machine_tags(__atags_pointer,
                                         __machine_arch_type);
......
1094        machine_desc = mdesc;
1095        machine_name = mdesc->name;
......
1174    }
```

第 1081 行，调用 setup_machine_fdt 函数来获取匹配的 machine_desc，参数就是 atags 的首地址，也就是 uboot 传递给 Linux 内核的 dtb 文件首地址，setup_machine_fdt 函数的返回值就是找到的最匹配的 machine_desc。

函数 setup_machine_fdt 定义在文件 arch/arm/kernel/devtree.c 中，内容如下(有缩减)：

示例代码 23.3.4.7 setup_machine_fdt 函数内容

```
211 const struct machine_desc * __init setup_machine_fdt(unsigned int
dt_phys)
212 {
213     const struct machine_desc *mdesc, *mdesc_best = NULL;
......
224     if (!dt_phys || !early_init_dt_verify(phys_to_virt(dt_phys)))
225         return NULL;
226
227     mdesc = of_flat_dt_match_machine(mdesc_best,
                                         arch_get_next_mach);
......
256     __machine_arch_type = mdesc->nr;
257
258     return mdesc;
259 }
```

第 227 行，调用函数 of_flat_dt_match_machine 来获取匹配的 machine_desc，参数 mdesc_best 是默认的 machine_desc，参数 arch_get_next_mach 是个函数，此函数定义在定义在 arch/arm/kernel/devtree.c 文件中。找到匹配的 machine_desc 的过程就是用设备树根节点的 compatible 属性值和 Linux 内核中保存的所有 machine_desc 结构的. dt_compat 中的值比较，看看哪个相等，如果相等的话就表示找到匹配的 machine_desc，arch_get_next_mach 函数的工作就是获取 Linux 内核中下一个 machine_desc 结构体。

最后再来看一下 of_flat_dt_match_machine 函数，此函数定义在文件 drivers/of/fdt.c 中，内容如下(有缩减)：

示例代码 23.3.4.8 of_flat_dt_match_machine 函数内容

```
815     const void * __init of_flat_dt_match_machine(const void
*default_match,
816             const void * (*get_next_compat)(const char * const**))
817     {
818         const void *data = NULL;
819         const void *best_data = default_match;
820         const char *const *compat;
```

```
821         unsigned long dt_root;
822         unsigned int best_score = ~1, score = 0;
823
824         dt_root = of_get_flat_dt_root();
825         while ((data = get_next_compat(&compat))) {
826             score = of_flat_dt_match(dt_root, compat);
827             if (score > 0 && score < best_score) {
828                 best_data = data;
829                 best_score = score;
830             }
831         }
.....
850         pr_info("Machine model: %s\n",
of_flat_dt_get_machine_name());
851
852         return best_data;
853     }
```

第 824 行，通过函数 of_get_flat_dt_root 获取设备树根节点。

第 825~831 行，此循环就是查找匹配的 machine_desc 过程，第 826 行的 of_flat_dt_match 函数会将根节点 compatible 属性的值和每个 machine_desc 结构体中. dt_compat 的值进行比较，直至找到匹配的那个 machine_desc。

总结一下，Linux 内核通过根节点 compatible 属性找到对应的设备的函数调用过程，如图 23.3.4.2 所示：

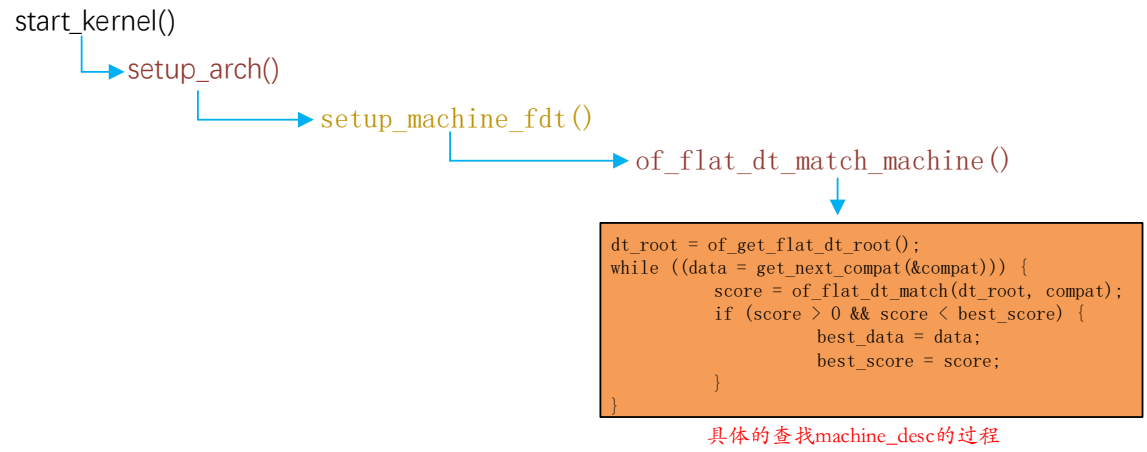


图 23.3.4.2 查找匹配设备的过程

23.3.5 向节点追加或修改内容

产品开发过程中可能面临着频繁的需求更改，比如第一版硬件上有一个 IIC 接口的六轴芯片 MPU6050，第二版硬件又要把这个 MPU6050 更换为 MPU9250 等。一旦硬件修改了，我们就要同步的修改设备树文件，毕竟设备树是描述板子硬件信息的文件。假设现在有个六轴芯片 fxls8471，fxls8471 要接到 STM32MP157D-ATK 开发板的 I2C1 接口上，那么相当于需要在 i2c1 这个节点上添加一个 fxls8471 子节点。先看一下 I2C1 接口对应的节点，打开文件 stm32mp151.dtsi 文件，找到如下所示内容：

示例代码 23.3.5.1 i2c1 节点

```
590     i2c1: i2c@40012000 {
591         compatible = "st,stm32mp15-i2c";
592         reg = <0x40012000 0x400>;
593         interrupt-names = "event", "error";
594         interrupts-extended = <&exti 21 IRQ_TYPE_LEVEL_HIGH>,
595                       <&intc GIC_SPI 32 IRQ_TYPE_LEVEL_HIGH>;
596         clocks = <&rcc I2C1_K>;
597         resets = <&rcc I2C1_R>;
598         #address-cells = <1>;
599         #size-cells = <0>;
600         dmas = <&dmamux1 33 0x400 0x80000001>,
601                <&dmamux1 34 0x400 0x80000001>;
602         dma-names = "rx", "tx";
603         power-domains = <&pd_core>;
604         st,syscfg-fmp = <&syscfg 0x4 0x1>;
605         wakeup-source;
606         status = "disabled";
607     };
```

示例代码 23.3.5.1 就是 STM32MP157 的 I2C1 节点，现在要在 i2c1 节点下创建一个子节点，这个子节点就是 fxls8471，最简单的方法就是在 i2c1 下直接添加一个名为 fxls8471 的子节点，如下所示：

示例代码 23.3.5.2 添加 fxls8471 子节点

```
590     i2c1: i2c@40012000 {
591         compatible = "st,stm32mp15-i2c";
592         reg = <0x40012000 0x400>;
593         interrupt-names = "event", "error";
594         interrupts-extended = <&exti 21 IRQ_TYPE_LEVEL_HIGH>,
595                       <&intc GIC_SPI 32 IRQ_TYPE_LEVEL_HIGH>;
596         clocks = <&rcc I2C1_K>;
597         resets = <&rcc I2C1_R>;
598         #address-cells = <1>;
599         #size-cells = <0>;
600         dmas = <&dmamux1 33 0x400 0x80000001>,
601                <&dmamux1 34 0x400 0x80000001>;
602         dma-names = "rx", "tx";
603         power-domains = <&pd_core>;
604         st,syscfg-fmp = <&syscfg 0x4 0x1>;
605         wakeup-source;
606         status = "disabled";
607         //fxls8471 子节点
608         fxls8471@1e {
609             compatible = "fsl,fxls8471";
```

```
610                 reg = <0x1e>;
611             };
612     };
```

第 608~611 行就是添加的 fxls8471 这个芯片对应的子节点。但是这样会有个问题！i2c1 节点是定义在 stm32mp151.dtsi 文件中的，而 stm32mp151.dtsi 是共有的设备树头文件，其他所有使用到 stm32mp1 这颗 SOC 的板子都会引用 stm32mp151.dtsi 这个文件。直接在 i2c1 节点中添加 fxls8471 就相当于在其他的所有板子上都添加了 fxls8471 这个设备，但是其他的板子并没有这个设备啊！因此，按照示例代码 23.3.5.2 这样写肯定是不行的。

这里就要引入另外一个内容，那就是如何向节点追加数据，我们现在要解决的就是如何向 i2c1 节点追加一个名为 fxls8471 的子节点，而且不能影响到其他使用到 stm32mp1 的板子。STM32MP157D-ATK 开发板使用的设备树文件为 stm32mp157d-atk.dts 和 stm32mp157d-atk.dtsi，因此我们需要在 stm32mp157d-atk.dts 文件中完成数据追加的内容，方式如下：

示例代码 23.3.5.3 节点追加数据方法

```
1 &i2c1 {
2   /* 要追加或修改的内容 */
3 };
```

第 1 行，&i2c1 表示要访问 i2c1 这个 label 所对应的节点，也就是 stm32mp151.dtsi 中的“i2c1: i2c@40012000”。

第 2 行，花括号内就是要向 i2c1 这个节点添加的内容，包括修改某些属性的值。

打开 stm32mp157d-atk.dts，在根节点后添加以下代码：

示例代码 23.3.5.4 向 i2c1 节点追加数据

```
1   &i2c1 {
2       pinctrl-names = "default", "sleep";
3       pinctrl-0 = <&i2c1_pins_b>;
4       pinctrl-1 = <&i2c1_pins_sleep_b>;
5       status = "okay";
6       clock-frequency = <100000>;
7
8       fxls8471@1e {
9           compatible = "fsl,fxls8471";
10          reg = <0x1e>;
11          position = <0>;
12          interrupt-parent = <&gpioh>;
13          interrupts = <6 IRQ_TYPE_EDGE_FALLING>;
14      };
15  };
```

示例代码 23.3.5.4 就是向 i2c1 节点添加/修改数据，比如第 5 行将 status 属性的值由原来的 disabled 改为 okay。第 6 行的属性“clock-frequency”表示 i2c1 时钟为 100KHz。“clock-frequency”就是新添加的属性。

第 8~14 行，i2c1 子节点 fxls8471，表示 I2C1 上连接的 fxls8471，“fxls8471”子节点里面描述了 fxls8471 这颗芯片的相关信息。

因为示例代码 23.3.5.4 中的内容是 stm32mp157d-atk.dts 这个文件内的，所以不会对使用 STM32MP157 这颗 SOC 的其他板子造成任何影响。这个就是向节点追加或修改内容，重点就是通过&label 来访问节点，然后直接在里面编写要追加或者修改的内容。

23.4 创建小型模板设备树

上一节已经对 DTS 的语法做了比较详细的讲解，本节我们就根据前面讲解的语法，从头到尾编写一个小型的设备树文件。当然了，这个小型设备树没有实际的意义，做这个的目的是为了掌握设备树的语法。在实际产品开发中，我们是不需要完完全全的重写一个.dts 设备树文件，一般都是使用 SOC 厂商提供好的.dts 文件，我们只需要在上面根据自己的实际情况做相应的修改即可。在编写设备树之前要先定义一个设备，我们就以 STM32MP157 这个 SOC 为例，我们需要在设备树里面描述的内容如下：

①、这个芯片是由两个 Cortex-A7 架构的 32 位 CPU 和 Cortex-M4 组成。

②、STM32MP157 内部 sram，起始地址为 0x10000000，大小为 384KB(0x60000)。

③、STM32MP157 内部 timers6，起始地址为 0x40004000，大小为 25.6KB(0x400)。

④、STM32MP157 内部 spi2，起始地址为 0x4000b000，大小为 25.6KB(0x400)。

⑤、STM32MP157 内部 usart2，起始地址为 0x4000e000，大小为 25.6KB(0x400)。

⑥、STM32MP157 内部 i2c1，起始地址为 0x40012000，大小为 25.6KB(0x400)。

为了简单起见，我们在设备树里面就实现这些内容即可，首先，搭建一个仅含有根节点“/”的基础的框架，新建一个名为 myfirst.dts 文件，在里面输入如下所示内容：

示例代码 23.4.1 设备树基础框架

```
1  / {
2      compatible = "st,stm32mp157d-atk", "st,stm32mp157";
3  };
```

设备树框架很简单，就一个根节点“/”，根节点里面只有一个 compatible 属性。我们就在这个基础框架上面将上面列出的内容一点点添加进来。

1、添加 cpus 节点

首先添加 CPU 节点，STM32MP157 采用 Cortex-A7 架构，先添加一个 cpus 节点，在 cpus 节点下添加 cpu0 子节点和 cpu1 子节点，完成以后如下所示：

示例代码 23.4.2 添加 cpus 节点

```
1  / {
2      compatible = "st,stm32mp157d-atk", "st,stm32mp157";
3      /* cpus 节点 */
4      cpus {
5          #address-cells = <1>;
6          #size-cells = <0>;
7
8          /* CPU0 节点 */
9          cpu0: cpu@0 {
10             compatible = "arm,cortex-a7";
11             device_type = "cpu";
12             reg = <0>;
13         };
```

```
14          /* CPU1 节点 */
15          cpu1: cpu@1 {
16              compatible = "arm,cortex-a7";
17              device_type = "cpu";
18              reg = <1>;
19          };
20      };
21  };
```

第 4~20 行，cpus 节点，此节点用于描述 SOC 内部的所有 CPU，因为 STM32MP157 有两个 CPU，所以在 cpus 下添加两个子节点分别为 cpu0 和 cpu1。

2、添加 soc 节点

像 uart，iic 控制器等等这些都属于 SOC 内部外设，因此一般会创建一个叫做 soc 的父节点来管理这些 SOC 内部外设的子节点，添加 soc 节点以后的 myfirst.dts 文件内容如下所示：

示例代码 23.4.3 添加 soc 节点

```
1   / {
2       compatible = "st,stm32mp157d-atk", "st,stm32mp157";
3       /* cpus 节点  */
4       cpus {
5           #address-cells = <1>;
6           #size-cells = <0>;
7
8           /* CPU0 节点 */
9           cpu0: cpu@0 {
10              compatible = "arm,cortex-a7";
11              device_type = "cpu";
12              reg = <0>;
13          };
14          /* CPU1 节点 */
15          cpu1: cpu@1 {
16              compatible = "arm,cortex-a7";
17              device_type = "cpu";
18              reg = <1>;
19          };
20      };
21      /* soc 节点 */
22      soc {
23          compatible = "simple-bus";
24          #address-cells = <1>;
25          #size-cells = <1>;
26          ranges;
27      };
28  };
```

第 22~27 行，soc 节点，soc 节点设置#address-cells=<1>，#size-cells=<1>，这样 soc 子节点的 reg 属性中起始地占用一个字长，地址空间长度也占用一个字长。

第 26 行，ranges 属性，ranges 属性为空，说明子空间和父空间地址范围相同。

3、添加 sram 节点

我们继续添加 sram 节点，sram 是 STM32MP157 内部 RAM，M4 内核会用到 SRAM4。sram 是 soc 节点的子节点。sram 起始地址为 0x10000000，大小为 384KB，添加 sram 节点以后 myfirst.dts 文件内容如下所示：

示例代码 23.4.4 添加 sram 节点

```
1  / {
2      compatible = "st,stm32mp157d-atk", "st,stm32mp157";
3      /* cpus 节点 */
4      cpus {
5          #address-cells = <1>;
6          #size-cells = <0>;
7
8          /* CPU0 节点 */
9          cpu0: cpu@0 {
10             compatible = "arm,cortex-a7";
11             device_type = "cpu";
12             reg = <0>;
13         };
14         /* CPU1 节点 */
15         cpu1: cpu@1 {
16             compatible = "arm,cortex-a7";
17             device_type = "cpu";
18             reg = <1>;
19         };
20     };
21     /* soc 节点  */
22     soc {
23         compatible = "simple-bus";
24         #address-cells = <1>;
25         #size-cells = <1>;
26         ranges;
27         /* sram 节点  */
28         sram: sram@10000000 {
29             compatible = "mmio-sram";
30             reg = <0x10000000 0x60000>;
31             ranges = <0 0x10000000 0x60000>;
32         };
33     };
34 };
```

第 28~32 行，sram 节点，第 28 行节点名字@后面的 0x10000000 就是 sram 的起始地址。第 30 行的 reg 属性也指明了 sram 内存的起始地址为 0x10000000，大小为 0x60000。

4、添加 timers6、spi2、usart2 和 i2c1 这四个子节点

最后我们在 myfirst.dts 文件中加入 timers6、spi2、usart2 和 i2c1 这四个外设控制器的节点。最终的 myfirst.dts 文件内容如下：

示例代码 23.4.5 最终的 myfirst.dts 文件

```
1  / {
2      compatible = "st,stm32mp157d-atk", "st,stm32mp157";
3      /* cpus 节点 */
4      cpus {
5          #address-cells = <1>;
6          #size-cells = <0>;
7
8          /* CPU0 节点 */
9          cpu0: cpu@0 {
10             compatible = "arm,cortex-a7";
11             device_type = "cpu";
12             reg = <0>;
13         };
14         /* CPU1 节点 */
15         cpu1: cpu@1 {
16             compatible = "arm,cortex-a7";
17             device_type = "cpu";
18             reg = <1>;
19         };
20     };
21     /* soc 节点 */
22     soc {
23         compatible = "simple-bus";
24         #address-cells = <1>;
25         #size-cells = <1>;
26         ranges;
27         /* sram 节点 */
28         sram: sram@10000000 {
29             compatible = "mmio-sram";
30             reg = <0x10000000 0x60000>;
31             #address-cells = <1>;
32             #size-cells = <1>;
33             ranges = <0 0x10000000 0x60000>;
34         };
35         /* timers6 节点 */
36         timers6: timer@40004000 {
```

```
37              #address-cells = <1>;
38              #size-cells = <0>;
39              compatible = "st,stm32-timers";
40              reg = <0x40004000 0x400>;
41          };
42          /* spi2 节点  */
43          spi2: spi@4000b000 {
44              #address-cells = <1>;
45              #size-cells = <0>;
46              compatible = "st,stm32h7-spi";
47              reg = <0x4000b000 0x400>;
48          };
49          /* usart2 节点 */
50          usart2: serial@4000e000 {
51              compatible = "st,stm32h7-uart";
52              reg = <0x4000e000 0x400>;
53          };
54          /* i2c1 节点  */
55          i2c1: i2c@40012000 {
56              compatible = "st,stm32mp15-i2c";
57              reg = <0x40012000 0x400>;
58          };
59
60      };
61  };
```

第 36~41 行，timers6 外设控制器节点。

第 43~48 行，spi2 外设控制器节点。

第 50~53 行，usart2 外设控制器节点。

第 55~58 行，i2c1 外设控制器节点。

到这里，我们的 myfirst.dts 小型的设备树模板就编辑好了，基本和 stm32mp151.dtsi 很像，可以看做是 stm32mp151.dtsi 的缩小版。在 myfirst.dts 里面我们仅仅是编写了 STM32MP1 的外设控制器节点，像 SAI 接口，LTDC 接口下所连接具体设备我们并没有写，因为具体的设备其设备树属性内容不同，这个等到具体的实验在详细讲解。

23.5 设备树在系统中的体现

Linux 内核启动的时候会解析设备树中各个节点的信息，并且在根文件系统的/proc/device-tree 目录下根据节点名字创建不同文件夹，如图 23.5.1 所示：

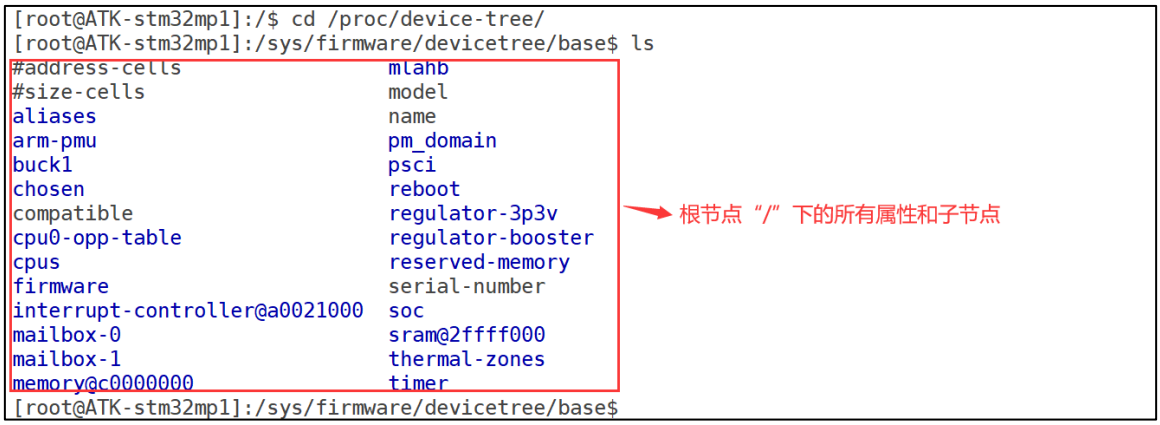


图 23.5.1 根节点“/”的属性以及子节点

图 23.5.1 就是目录/proc/device-tree 目录下的内容，/proc/device-tree 目录下是根节点“/”的所有属性和子节点，我们依次来看一下这些属性和子节点。

1、根节点“/”各个属性

在图 23.5.1 中，根节点属性属性表现为一个个的文件，比如图 23.5.1 中的“#address-cells”、“#size-cells”、“compatible”、“model”和“name”这 5 个文件，它们在设备树中就是根节点的 5 个属性。既然是文件那么肯定可以查看其内容，输入 cat 命令来查看 model 和 compatible 这两个文件的内容，结果如图 23.5.2 所示：

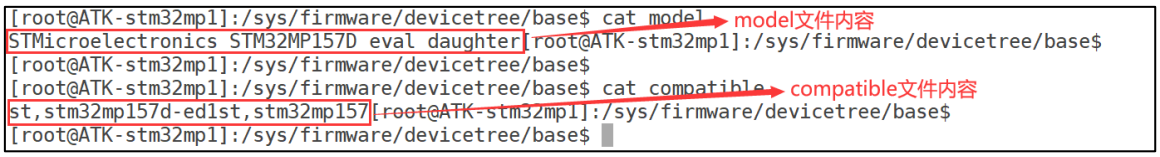


图 23.5.2 model 和 compatible 文件内容

从图 23.5.2 可以看出，文件 model 的内容是“STMicroelectronics STM32MP157D eval daughter”，文件 compatible 的内容为“st，stm32mp157d-ed1st,stm32mp157”。打开文件 stm32mp157d-atk.dts 查看一下，这不正是根节点“/”的 model 和 compatible 属性值吗！

2、根节点“/”各子节点

图 23.5.1 中各个文件夹就是根节点“/”的各个子节点，比如“aliases”、“reboot”、“chosen”和“cpus”等等。大家可以查看一下 stm32mp157d-atk.dts 和 stm32mp157d-atk.dts 所引用的.dtsi 文件，看看根节点的子节点都有哪些，是否和图 23.5.1 中的一致。

/proc/device-tree 目录就是设备树在根文件系统中的体现，同样是按照树形结构组织的，进入/proc/device-tree/soc 目录中就可以看到 soc 节点的所有子节点，如图 23.5.3 所示：

```
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base$ cd soc/
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base/soc$ ls
#address-cells                  sai@4400c000
#size-cells                     sai@50027000
adc@48003000                    sdmmc@48004000
audio-controller@4000b000       sdmmc@58005000
audio-controller@4000c000       sdmmc@58007000
audio-controller@4000d000       serial@4000e000
audio-controller@44004000       serial@4000f000
can@4400e000                    serial@40010000
can@4400f000                    serial@40011000
cec@40016000                    serial@40018000
compatible                      serial@40019000
crc@58009000                    serial@44003000
dac@40017000                    serial@5c000000
```

图 23.5.3 soc 部分节点

和根节点“/”一样，图 23.5.3 中的所有文件分别为 soc 节点的属性文件和子节点文件夹。大家可以自行查看一下这些属性文件的内容是否和 stm32mp151.dtsi 中 soc 节点的属性值相同。

23.6 特殊节点

在根节点“/”中有两个特殊的子节点：aliases 和 chosen，我们接下来看一下这两个特殊的子节点。

23.6.1 aliases 子节点

打开 stm32mp157d-atk.dts 文件，aliases 节点内容如下所示：

示例代码 23.6.1.1 aliases 子节点

```
24      aliases {
25          serial0 = &uart4;
26      };
```

单词 aliases 的意思是“别名”，因此 aliases 节点的主要功能就是定义别名，定义别名的目的就是为了方便访问节点。不过我们一般会在节点命名的时候会加上 label，然后通过&label 来访问节点，这样也很方便，而且设备树里面大量的使用&label 的形式来访问节点。

23.6.2 chosen 子节点

chosen 并不是一个真实的设备，chosen 节点主要是为了 uboot 向 Linux 内核传递数据，重点是 bootargs 参数。一般.dts 文件中 chosen 节点通常为空或者内容很少，stm32mp157d-atk.dts 中 chosen 节点内容如下所示：

示例代码 23.6.2.1 chosen 子节点

```
20      chosen {
21          stdout-path = "serial0:115200n8";
22      };
```

从示例代码 23.6.2.1 中可以看出，chosen 节点仅仅设置了属性“stdout-path”，表示标准输出使用 serial0，而 aliases 已经设置了 serial0 为 uart4，所以开发板启动以后使用 UART4 作为默认终端。但是当我们进入到/proc/device-tree/chosen 目录里面，会发现多了 bootargs 这个属性，如图 23.6.2.1 所示：

```
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base/chosen$ ls
bootargs    name        stdout-path
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base/chosen$
```

图 23.6.2.1 chosen 节点目录

输入 cat 命令查看 bootargs 这个文件的内容，结果如图 23.6.2.2 所示：

```
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base/chosen$ cat bootargs
console=ttySTM0,115200 root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs/rootfs,proto=tcp rw
ip=192.168.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off[root@ATK-stm32mp1]:/sys/firmware/devicet
ree/base/chosen$
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base/chosen$
```

图 23.6.2.2 bootargs 文件内容

从图 23.6.2.2 可以看出，bootargs 这个文件的内容为“console=ttySTM0,115200……”，这个就是我们在 uboot 中设置的 bootargs 环境变量，现在有两个疑点：

①、我们并没有在设备树中设置 chosen 节点的 bootargs 属性，那么图 23.6.2.1 中 bootargs 这个属性是怎么产生的？

②、为何图 23.6.2.1 中 bootargs 文件的内容和 uboot 中 bootargs 环境变量的值一样？它们之间有什么关系？

前面讲解 uboot 的时候说过，uboot 在启动 Linux 内核的时候会将 bootargs 的值传递给 Linux 内核，bootargs 会作为 Linux 内核的命令行参数，Linux 内核启动的时候会打印出命令行参数(也就是 uboot 传递进来的 bootargs 的值)，如图 23.6.2.3 所示：

```
psci: SMC Calling Convention v1.0
percpu: Embedded 20 pages/cpu s50124 r8192 d23604 u81920
Built 1 zonelists, mobility grouping on.  Total pages: 227647
Kernel command line: console=ttySTM0,115200 root=/dev/nfs nfsroot=192.168.1.249:/home/zuozhongkai/linux/nfs
/rootfs,proto=tcp rw ip=192.168.1.250:192.168.1.249:192.168.1.1:255.255.255.0::eth0:off
```

命令行参数

图 23.6.2.3 命令行参数

既然 chosen 节点的 bootargs 属性不是我们在设备树里面设置的，那么只有一种可能，那就是 uboot 自己在 chosen 节点里面添加了 bootargs 属性！并且设置 bootargs 属性的值为 bootargs 环境变量的值。因为在启动 Linux 内核之前，只有 uboot 知道 bootargs 环境变量的值，并且 uboot 也知道.dtb 设备树文件在 DRAM 中的位置，因此 uboot 的“作案”嫌疑最大。在 uboot 源码中全局搜索“chosen”这个字符串，看看能不能找到一些蛛丝马迹。果然不出所料，在 common/fdt_support.c 文件中发现了“chosen”的身影，fdt_support.c 文件中有个 fdt_chosen 函数，此函数内容如下所示：

示例代码 23.6.2.2 uboot 源码中的 fdt_chosen 函数

```
275 int fdt_chosen(void *fdt)
276 {
277     int   nodeoffset;
278     int   err;
279     char  *str;     /* used to set string properties */
280
281     err = fdt_check_header(fdt);
282     if (err < 0) {
283         printf("fdt_chosen: %s\n", fdt_strerror(err));
284         return err;
285     }
286
287     /* find or create "/chosen" node. */
```

```
288     nodeoffset = fdt_find_or_add_subnode(fdt, 0, "chosen");
289     if (nodeoffset < 0)
290         return nodeoffset;
291
292     str = getenv("bootargs");
293     if (str) {
294         err = fdt_setprop(fdt, nodeoffset, "bootargs", str,
295                 strlen(str) + 1);
296         if (err < 0) {
297             printf("WARNING: could not set bootargs %s.\n",
298                   fdt_strerror(err));
299             return err;
300         }
301     }
302
303     return fdt_fixup_stdout(fdt, nodeoffset);
304 }
```

第 288 行，调用函数 fdt_find_or_add_subnode 从设备树(.dtb)中找到 chosen 节点，如果没有找到的话就会自己创建一个 chosen 节点。

第 292 行，读取 uboot 中 bootargs 环境变量的内容。

第 294 行，调用函数 fdt_setprop 向 chosen 节点添加 bootargs 属性，并且 bootargs 属性的值就是环境变量 bootargs 的内容。

证据“实锤”了，就是 uboot 中的 fdt_chosen 函数在设备树的 chosen 节点中加入了 bootargs 属性，并且还设置了 bootargs 属性值。接下来我们顺着 fdt_chosen 函数一点点的抽丝剥茧，看看都有哪些函数调用了 fdt_chosen，一直找到最终的源头。这里我就不卖关子了，直接告诉大家整个流程是怎么样的，见图 23.6.2.4：

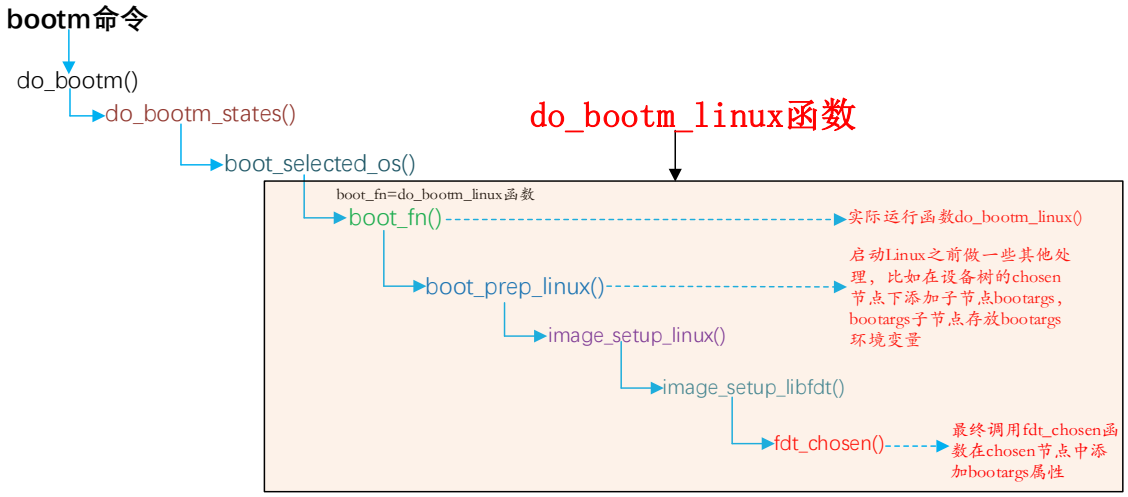


图 23.6.2.4 fdt_chosen 函数调用流程

图 23.6.2.4 中框起来的部分就是函数 do_bootm_linux 函数的执行流程，也就是说 do_bootm_linux 函数会通过一系列复杂的调用，最终通过 fdt_chosen 函数在 chosen 节点中加入了 bootargs 属性。而我们通过 bootz 命令启动 Linux 内核的时候会运行 do_bootm_linux 函数，至此，真相大白，一切事情的源头都源于如下命令：

```
bootm c2000000 - c4000000
```

当我们输入上述命令并执行以后，do_bootm 函数就会执行，然后一切就按照图 23.6.2.4 中所示的流程开始运行。

23.7 Linux 内核解析 DTB 文件

Linux 内核在启动的时候会解析 DTB 文件，然后在/proc/device-tree 目录下生成相应的设备树节点文件。接下来我们简单分析一下 Linux 内核是如何解析 DTB 文件的，流程如图 23.7.1 所示：

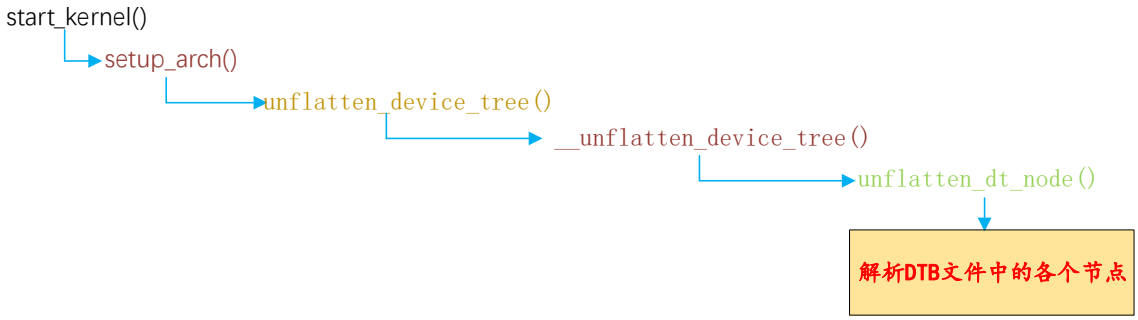


图 23.7.1 设备树节点解析流程。

从图 23.7.1 中可以看出，在 start_kernel 函数中完成了设备树节点解析的工作，最终实际工作的函数为 unflatten_dt_node。

23.8 绑定信息文档

设备树是用来描述板子上的设备信息的，不同的设备其信息不同，反映到设备树中就是属性不同。那么我们在设备树中添加一个硬件对应的节点的时候从哪里查阅相关的说明呢？在 Linux 内核源码中有详细的 TXT 文档描述了如何添加节点，这些 TXT 文档叫做绑定文档，路径为：Linux 源码目录/Documentation/devicetree/bindings，如图 23.8.1 所示：

图 23.8.1 绑定文档

比如我们现在要想在 STM32MP157 这颗 SOC 的 I2C 下添加一个节点，那么就可以查看 Documentation/devicetree/bindings/i2c/i2c-stm32.txt，此文档详细的描述了 STM32MP1 系列的 SOC 如何在设备树中添加 I2C 设备节点，文档内容如下所示：

```
* I2C controller embedded in STMicroelectronics STM32 I2C platform

Required properties:
- compatible: Must be one of the following
  - "st,stm32f4-i2c"
```

```
  - "st,stm32f7-i2c"
  - "st,stm32mp15-i2c"
- reg: Offset and length of the register set for the device
- interrupts: Must contain the interrupt id for I2C event and then the
  interrupt id for I2C error.
- resets: Must contain the phandle to the reset controller.
- clocks: Must contain the input clock of the I2C instance.
- A pinctrl state named "default" must be defined to set pins in mode
  of operation for I2C transfer. An optional pinctrl state named "sleep"
  has to be defined as well as to put I2C in low power mode in suspend
  mode.
- #address-cells = <1>;
- #size-cells = <0>;

Optional properties:
- clock-frequency: Desired I2C bus clock frequency in Hz. If not
specified,
  the default 100 kHz frequency will be used.
  For STM32F4 SoC Standard-mode and Fast-mode are supported, possible
values are
  100000 and 400000.
  For STM32F7, STM32H7 and STM32MP1 SoCs, Standard-mode, Fast-mode and
Fast-mode
  Plus are supported, possible values are 100000, 400000 and 1000000.
- dmas: List of phandles to rx and tx DMA channels. Refer to stm32-
dma.txt.
- dma-names: List of dma names. Valid names are: "rx" and "tx".
- i2c-scl-rising-time-ns: I2C SCL Rising time for the board (default:
25)
  For STM32F7, STM32H7 and STM32MP1 only.
- i2c-scl-falling-time-ns: I2C SCL Falling time for the board (default:
10)
  For STM32F7, STM32H7 and STM32MP1 only.
  I2C Timings are derived from these 2 values
- st,syscfg-fmp: Use to set Fast Mode Plus bit within SYSCFG when Fast
Mode
  Plus speed is selected by slave.
    1st cell: phandle to syscfg
    2nd cell: register offset within SYSCFG
    3rd cell: register bitmask for FMP bit
  For STM32F7, STM32H7 and STM32MP1 only.
- st,smbus-alert: enable the SMBus Alert feature
- st,smbus-host-notify: enable the SMBus Host-Notify feature
```

```
- wakeup-source: Enable the possibility to use the I2C as a wakeup-
source
  For STM32H7 and STM32MP1 only.

Example:

    i2c@40005400 {
        compatible = "st,stm32f4-i2c";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x40005400 0x400>;
        interrupts = <31>,
                <32>;
        resets = <&rcc 277>;
        clocks = <&rcc 0 149>;
        pinctrl-0 = <&i2c1_sda_pin>, <&i2c1_scl_pin>;
        pinctrl-names = "default";
    };
    i2c@40005400 {
        compatible = "st,stm32f7-i2c";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x40005400 0x400>;
        interrupts = <31>,
                <32>;
        resets = <&rcc STM32F7_APB1_RESET(I2C1)>;
        clocks = <&rcc 1 CLK_I2C1>;
        pinctrl-0 = <&i2c1_sda_pin>, <&i2c1_scl_pin>;
        pinctrl-1 =  <&i2c1_sda_pin_sleep>, <&i2c1_scl_pin_sleep>;
        pinctrl-names = "default", "sleep";
        st,syscfg-fmp = <&syscfg 0x4 0x1>;
        st,syscfg-fmp-clr = <&syscfg 0x44 0x1>;
    };
```

有时候使用的一些芯片在 Documentation/devicetree/bindings 目录下找不到对应的文档，这个时候就要咨询芯片的提供商，让他们给你提供参考的设备树文件。

23.9 设备树常用 OF 操作函数

设备树描述了设备的详细信息，这些信息包括数字类型的、字符串类型的、数组类型的，我们在编写驱动的时候需要获取到这些信息。比如设备树使用 reg 属性描述了某个外设的寄存器地址为 0X02005482，长度为 0X400，我们在编写驱动的时候需要获取到 reg 属性的 0X02005482 和 0X400 这两个值，然后初始化外设。Linux 内核给我们提供了一系列的函数来获取设备树中的节点或者属性信息，这一系列的函数都有一个统一的前缀“of_”，所以在很多资料里面也被叫做 OF 函数。这些 OF 函数原型都定义在 include/linux/of.h 文件中。

23.9.1 查找节点的 OF 函数

设备都是以节点的形式“挂”到设备树上的，因此要想获取这个设备的其他属性信息，必须先获取到这个设备的节点。Linux 内核使用 device_node 结构体来描述一个节点，此结构体定义在文件 include/linux/of.h 中，定义如下：

示例代码 23.3.9.1 device_node 节点

```
51 struct device_node {
52     const char *name;                /*节点名字          */
53     phandle phandle;
54     const char *full_name;           /*节点全名          */
55     struct fwnode_handle fwnode;
56
57     struct  property *properties;    /*属性              */
58     struct  property *deadprops;     /*removed 属性      */
59     struct  device_node *parent;     /*父节点            */
60     struct  device_node *child;      /*子节点            */
61     struct  device_node *sibling;
62 #if defined(CONFIG_OF_KOBJ)
63     struct  kobject kobj;
64 #endif
65     unsigned long _flags;
66     void    *data;
67 #if defined(CONFIG_SPARC)
68     unsigned int unique_id;
69     struct of_irq_controller *irq_trans;
70 #endif
71 };
```

与查找节点有关的 OF 函数有 5 个，我们依次来看一下。

1、of_find_node_by_name 函数

of_find_node_by_name 函数通过节点名字查找指定的节点，函数原型如下：

```
struct device_node *of_find_node_by_name(struct device_node    *from,
                                         const char            *name);
```

函数参数和返回值含义如下：

from：开始查找的节点，如果为 NULL 表示从根节点开始查找整个设备树。

name：要查找的节点名字。

返回值：找到的节点，如果为 NULL 表示查找失败。

2、of_find_node_by_type 函数

of_find_node_by_type 函数通过 device_type 属性查找指定的节点，函数原型如下：

```
struct device_node *of_find_node_by_type(struct device_node *from, const char *type)
```

函数参数和返回值含义如下：

from：开始查找的节点，如果为 NULL 表示从根节点开始查找整个设备树。

type：要查找的节点对应的 type 字符串，也就是 device_type 属性值。

返回值：找到的节点，如果为 NULL 表示查找失败。

3、of_find_compatible_node 函数

of_find_compatible_node 函数根据 device_type 和 compatible 这两个属性查找指定的节点，函数原型如下：

```
struct device_node *of_find_compatible_node(struct device_node  *from,
                                            const char          *type,
                                            const char          *compatible)
```

函数参数和返回值含义如下：

from：开始查找的节点，如果为 NULL 表示从根节点开始查找整个设备树。

type：要查找的节点对应的 type 字符串，也就是 device_type 属性值，可以为 NULL，表示忽略掉 device_type 属性。

compatible：要查找的节点所对应的 compatible 属性列表。

返回值：找到的节点，如果为 NULL 表示查找失败

4、of_find_matching_node_and_match 函数

of_find_matching_node_and_match 函数通过 of_device_id 匹配表来查找指定的节点，函数原型如下：

```
struct device_node *of_find_matching_node_and_match(struct device_node          *from,
                                                    const struct of_device_id   *matches,
                                                    const struct of_device_id   **match)
```

函数参数和返回值含义如下：

from：开始查找的节点，如果为 NULL 表示从根节点开始查找整个设备树。

matches：of_device_id 匹配表，也就是在此匹配表里面查找节点。

match：找到的匹配的 of_device_id。

返回值：找到的节点，如果为 NULL 表示查找失败

5、of_find_node_by_path 函数

of_find_node_by_path 函数通过路径来查找指定的节点，函数原型如下：

```
inline struct device_node *of_find_node_by_path(const char *path)
```

函数参数和返回值含义如下：

path：带有全路径的节点名，可以使用节点的别名，比如“/backlight”就是 backlight 这个节点的全路径。

返回值：找到的节点，如果为 NULL 表示查找失败

23.9.2 查找父/子节点的 OF 函数

Linux 内核提供了几个查找节点对应的父节点或子节点的 OF 函数，我们依次来看一下。

1、of_get_parent 函数

of_get_parent 函数用于获取指定节点的父节点(如果有父节点的话)，函数原型如下：

```
struct device_node *of_get_parent(const struct device_node *node)
```

函数参数和返回值含义如下：

node：要查找的父节点的节点。

返回值：找到的父节点。

2、of_get_next_child 函数

of_get_next_child 函数用迭代的查找子节点，函数原型如下：

```
struct device_node *of_get_next_child(const struct device_node  *node,
                                      struct device_node        *prev)
```

函数参数和返回值含义如下：

node：父节点。

prev：前一个子节点，也就是从哪一个子节点开始迭代的查找下一个子节点。可以设置为 NULL，表示从第一个子节点开始。

返回值：找到的下一个子节点。

23.9.3 提取属性值的 OF 函数

节点的属性信息里面保存了驱动所需要的内容，因此对于属性值的提取非常重要，Linux 内核中使用结构体 property 表示属性，此结构体同样定义在文件 include/linux/of.h 中，内容如下：

示例代码 23.9.3.1 property 结构体

```
31 struct property {
32     char    *name;                  /* 属性名字          */
33     int     length;                 /* 属性长度          */
34     void    *value;                 /* 属性值            */
35     struct property *next;          /* 下一个属性        */
36 #if defined(CONFIG_OF_DYNAMIC) || defined(CONFIG_SPARC)
37     unsigned long _flags;
38 #endif
39 #if defined(CONFIG_OF_PROMTREE)
40     unsigned int unique_id;
41 #endif
42 #if defined(CONFIG_OF_KOBJ)
43     struct bin_attribute attr;
44 #endif
45 };
```

Linux 内核也提供了提取属性值的 OF 函数，我们依次来看一下。

1、of_find_property 函数

of_find_property 函数用于查找指定的属性，函数原型如下：

```
property *of_find_property(const struct device_node  *np,
                           const char                 *name,
                           int                        *lenp)
```

函数参数和返回值含义如下：

np：设备节点。

name： 属性名字。

lenp：属性值的字节数

返回值：找到的属性。

2、of_property_count_elems_of_size 函数

of_property_count_elems_of_size 函数用于获取属性中元素的数量，比如 reg 属性值是一个数组，那么使用此函数可以获取到这个数组的大小，此函数原型如下：

```
int of_property_count_elems_of_size(const struct device_node   *np,
                                    const char                  *propname,
                                    int                         elem_size)
```

函数参数和返回值含义如下：

np：设备节点。

proname： 需要统计元素数量的属性名字。

elem_size：元素长度。

返回值：得到的属性元素数量。

3、of_property_read_u32_index 函数

of_property_read_u32_index 函数用于从属性中获取指定标号的 u32 类型数据值(无符号 32 位)，比如某个属性有多个 u32 类型的值，那么就可以使用此函数来获取指定标号的数据值，此函数原型如下：

```
int of_property_read_u32_index(const struct device_node   *np,
                               const char                 *propname,
                               u32                        index,
                               u32                        *out_value)
```

函数参数和返回值含义如下：

np：设备节点。

proname： 要读取的属性名字。

index：要读取的值标号。

out_value：读取到的值

返回值：0 读取成功，负值，读取失败，-EINVAL 表示属性不存在，-ENODATA 表示没有要读取的数据，-EOVERFLOW 表示属性值列表太小。

4、 of_property_read_u8_array 函数
of_property_read_u16_array 函数
of_property_read_u32_array 函数
of_property_read_u64_array 函数

这 4 个函数分别是读取属性中 u8、u16、u32 和 u64 类型的数组数据，比如大多数的 reg 属性都是数组数据，可以使用这 4 个函数一次读取出 reg 属性中的所有数据。这四个函数的原型如下：

```
int of_property_read_u8_array(const struct device_node   *np,
                              const char                 *propname,
                              u8                         *out_values,
                              size_t                     sz)
int of_property_read_u16_array(const struct device_node  *np,
                               const char                *propname,
                               u16                       *out_values,
                               size_t                    sz)
int of_property_read_u32_array(const struct device_node  *np,
                               const char                *propname,
                               u32                       *out_values,
                               size_t                    sz)
int of_property_read_u64_array(const struct device_node  *np,
```

```
                                const char          *propname,
                                u64                 *out_values,
                                size_t              sz)
```

函数参数和返回值含义如下：

np：设备节点。

proname： 要读取的属性名字。

out_value：读取到的数组值，分别为 u8、u16、u32 和 u64。

sz：要读取的数组元素数量。

返回值：0，读取成功，负值，读取失败，-EINVAL 表示属性不存在，-ENODATA 表示没有要读取的数据，-EOVERFLOW 表示属性值列表太小。

5、of_property_read_u8 函数

of_property_read_u16 函数

of_property_read_u32 函数

of_property_read_u64 函数

有些属性只有一个整形值，这四个函数就是用于读取这种只有一个整形值的属性，分别用于读取 u8、u16、u32 和 u64 类型属性值，函数原型如下：

```
int of_property_read_u8(const struct device_node    *np,
                        const char                  *propname,
                        u8                          *out_value)
int of_property_read_u16(const struct device_node   *np,
                        const char                  *propname,
                        u16                         *out_value)
int of_property_read_u32(const struct device_node   *np,
                        const char                  *propname,
                        u32                         *out_value)
int of_property_read_u64(const struct device_node   *np,
                        const char                  *propname,
                        u64                         *out_value)
```

函数参数和返回值含义如下：

np：设备节点。

proname： 要读取的属性名字。

out_value：读取到的数组值。

返回值：0，读取成功，负值，读取失败，-EINVAL 表示属性不存在，-ENODATA 表示没有要读取的数据，-EOVERFLOW 表示属性值列表太小。

6、of_property_read_string 函数

of_property_read_string 函数用于读取属性中字符串值，函数原型如下：

```
int of_property_read_string(struct device_node  *np,
                            const char          *propname,
                            const char          **out_string)
```

函数参数和返回值含义如下：

np：设备节点。

proname： 要读取的属性名字。

out_string：读取到的字符串值。

返回值：0，读取成功，负值，读取失败。

7、of_n_addr_cells 函数

of_n_addr_cells 函数用于获取#address-cells 属性值，函数原型如下：

```
int of_n_addr_cells(struct device_node *np)
```

函数参数和返回值含义如下：

np：设备节点。

返回值：获取到的#address-cells 属性值。

8、of_n_size_cells 函数

of_n_size_cells 函数用于获取#size-cells 属性值，函数原型如下：

```
int of_n_size_cells(struct device_node *np)
```

函数参数和返回值含义如下：

np：设备节点。

返回值：获取到的#size-cells 属性值。

23.9.4 其他常用的 OF 函数

1、of_device_is_compatible 函数

of_device_is_compatible 函数用于查看节点的 compatible 属性是否有包含 compat 指定的字符串，也就是检查设备节点的兼容性，函数原型如下：

```
int of_device_is_compatible(const struct device_node *device,
                            const char               *compat)
```

函数参数和返回值含义如下：

device：设备节点。

compat：要查看的字符串。

返回值：0，节点的 compatible 属性中不包含 compat 指定的字符串；正数，节点的 compatible 属性中包含 compat 指定的字符串。

2、of_get_address 函数

of_get_address 函数用于获取地址相关属性，主要是“reg”或者“assigned-addresses”属性值，函数属性如下：

```
const __be32 *of_get_address(struct device_node *dev,
                             int                index,
                             u64                *size,
                             unsigned int       *flags)
```

函数参数和返回值含义如下：

dev：设备节点。

index：要读取的地址标号。

size：地址长度。

flags：参数，比如 IORESOURCE_IO、IORESOURCE_MEM 等

返回值：读取到的地址数据首地址，为 NULL 的话表示读取失败。

3、of_translate_address 函数

of_translate_address 函数负责将从设备树读取到的地址转换为物理地址，函数原型如下：

```
u64 of_translate_address(struct device_node     *dev,
                         const __be32           *addr)
```

函数参数和返回值含义如下：

dev：设备节点。

in_addr：要转换的地址。

返回值：得到的物理地址，如果为 OF_BAD_ADDR 的话表示转换失败。

4、of_address_to_resource 函数

IIC、SPI、GPIO 等这些外设都有对应的寄存器，这些寄存器其实就是一组内存空间，Linux 内核使用 resource 结构体来描述一段内存空间，“resource”翻译出来就是“资源”，因此用 resource 结构体描述的都是设备资源信息，resource 结构体定义在文件 include/linux/ioport.h 中，定义如下：

示例代码 23.9.4.1 resource 结构体

```
20 struct resource {
21     resource_size_t start;
22     resource_size_t end;
23     const char *name;
24     unsigned long flags;
25     unsigned long desc;
26     struct resource *parent, *sibling, *child;
27 };
```

对于 32 位的 SOC 来说，resource_size_t 是 u32 类型的。其中 start 表示开始地址，end 表示结束地址，name 是这个资源的名字，flags 是资源标志位，一般表示资源类型，可选的资源标志定义在文件 include/linux/ioport.h 中，如下所示：

示例代码 23.9.4.2 资源标志

```
1  #define IORESOURCE_BITS          0x000000ff
2  #define IORESOURCE_TYPE_BITS     0x00001f00
3  #define IORESOURCE_IO            0x00000100
4  #define IORESOURCE_MEM           0x00000200
5  #define IORESOURCE_REG           0x00000300
6  #define IORESOURCE_IRQ           0x00000400
7  #define IORESOURCE_DMA           0x00000800
8  #define IORESOURCE_BUS           0x00001000
9  #define IORESOURCE_PREFETCH      0x00002000
10 #define IORESOURCE_READONLY      0x00004000
11 #define IORESOURCE_CACHEABLE     0x00008000
12 #define IORESOURCE_RANGELENGTH   0x00010000
13 #define IORESOURCE_SHADOWABLE    0x00020000
14 #define IORESOURCE_SIZEALIGN     0x00040000
15 #define IORESOURCE_STARTALIGN    0x00080000
16 #define IORESOURCE_MEM_64        0x00100000
17 #define IORESOURCE_WINDOW        0x00200000
18 #define IORESOURCE_MUXED         0x00400000
19 #define IORESOURCE_EXCLUSIVE     0x08000000
```

```
20 #define IORESOURCE_DISABLED     0x10000000
21 #define IORESOURCE_UNSET        0x20000000
22 #define IORESOURCE_AUTO         0x40000000
23 #define IORESOURCE_BUSY         0x80000000
```

大家一般最常见的资源标志就是 IORESOURCE_MEM、IORESOURCE_REG 和 IORESOURCE_IRQ 等。接下来我们回到 of_address_to_resource 函数，此函数看名字像是从设备树里面提取资源值，但是本质上就是将 reg 属性值，然后将其转换为 resource 结构体类型，函数原型如下所示

```
int of_address_to_resource(struct device_node    *dev,
                           int                   index,
                           struct resource       *r)
```

函数参数和返回值含义如下：

dev：设备节点。

index：地址资源标号。

r：得到的 resource 类型的资源值。

返回值：0，成功；负值，失败。

5、of_iomap 函数

of_iomap 函数用于直接内存映射，以前我们会通过 ioremap 函数来完成物理地址到虚拟地址的映射，采用设备树以后就可以直接通过 of_iomap 函数来获取内存地址所对应的虚拟地址，不需要使用 ioremap 函数了。当然了，你也可以使用 ioremap 函数来完成物理地址到虚拟地址的内存映射，只是在采用设备树以后，大部分的驱动都使用 of_iomap 函数了。of_iomap 函数本质上也是将 reg 属性中地址信息转换为虚拟地址，如果 reg 属性有多段的话，可以通过 index 参数指定要完成内存映射的是哪一段，of_iomap 函数原型如下：

```
void __iomem *of_iomap(struct device_node    *np,
                       int                   index)
```

函数参数和返回值含义如下：

np：设备节点。

index：reg 属性中要完成内存映射的段，如果 reg 属性只有一段的话 index 就设置为 0。

返回值：经过内存映射后的虚拟内存首地址，如果为 NULL 的话表示内存映射失败。

关于设备树常用的 OF 函数就先讲解到这里，Linux 内核中关于设备树的 OF 函数不仅仅只有前面讲的这几个，还有很多 OF 函数我们并没有讲解，这些没有讲解的 OF 函数要结合具体的驱动，比如获取中断号的 OF 函数、获取 GPIO 的 OF 函数等等，这些 OF 函数我们在后面的驱动实验中再详细的讲解。

关于设备树就讲解到这里，关于设备树我们重点要了解一下几点内容：

①、DTS、DTB 和 DTC 之间的区别，如何将.dts 文件编译为.dtb 文件。

②、设备树语法，这个是重点，因为在实际工作中我们是需要修改设备树的。

③、设备树的几个特殊子节点。

④、关于设备树的 OF 操作函数，也是重点，因为设备树最终是被驱动文件所使用的，而驱动文件必须要读取设备树中的属性信息，比如内存信息、GPIO 信息、中断信息等等。要想在驱动中读取设备树的属性值，那么就必须使用 Linux 内核提供的众多的 OF 函数。

从下一章开始所以的 Linux 驱动实验都将采用设备树，从最基本的点灯，到复杂的音频、网络或块设备等驱动。将会带领大家由简入深，深度剖析设备树，最终掌握基于设备树的驱动开发技能。

第二十四章 设备树下的 LED 驱动实验

上一章我们详细的讲解了设备树语法以及在驱动开发中常用的 OF 函数，本章我们就开始第一个基于设备树的 Linux 驱动实验。本章在第二十二章实验的基础上完成，只是将其驱动开发改为设备树形式而已。

24.1 设备树 LED 驱动原理

在《第二十二章 新字符设备驱动实验》中，我们直接在驱动文件 newchrled.c 中定义有关寄存器物理地址，然后使用 io_remap 函数进行内存映射，得到对应的虚拟地址，最后操作寄存器对应的虚拟地址完成对 GPIO 的初始化。本章我们在第四十二章实验基础上完成，本章我们使用设备树来向 Linux 内核传递相关的寄存器物理地址，Linux 驱动文件使用上一章讲解的 OF 函数从设备树中获取所需的属性值，然后使用获取到的属性值来初始化相关的 IO。本章实验还是比较简单的，本章实验重点内容如下：

①、在 stm32mp157d-atk.dts 文件中创建相应的设备节点。

②、编写驱动程序(在第二十二章实验基础上完成)，获取设备树中的相关属性值。

③、使用获取到的有关属性值来初始化 LED 所使用的 GPIO。

24.2 硬件原理图分析

本实验的硬件原理参考 21.2 小节即可。

24.3 实验程序编写

24.3.1 修改设备树文件

在根节点“/”下创建一个名为“stm32mp1_led”的子节点，打开 stm32mp157d-atk.dts 文件，在根节点“/”最后面输入如下所示内容：

示例代码 24.3.1.1 stm32mp1_led 节点

```
1   stm32mp1_led {
2       compatible = "atkstm32mp1-led";
3       status = "okay";
4       reg = <0X50000A28 0X04      /* RCC_MP_AHB4ENSETR     */
5              0X5000A000 0X04      /* GPIOI_MODER           */
6              0X5000A004 0X04      /* GPIOI_OTYPER          */
7              0X5000A008 0X04      /* GPIOI_OSPEEDR         */
8              0X5000A00C 0X04      /* GPIOI_PUPDR           */
9              0X5000A018 0X04 >;   /* GPIOI_BSRR            */
10  };
```

第 2 行，属性 compatible 设置 stm32mp1_led 节点兼容为“atkstm32mp1-led”。

第 3 行，属性 status 设置状态为“okay”。

第 4~9 行，reg 属性，非常重要！reg 属性设置了驱动里面所要使用的寄存器物理地址，比如第 4 行的“0X50000A28 0X04”表示 STM32MP1 的 RCC_MP_AHB4ENSETR 寄存器，其中寄存器地址为 0X50000A28，长度为 4 个字节。

设备树修改完成以后输入如下命令重新编译一下 stm32mp157d-atk.dts：

```
make dtbs
```

编译完成以后得到 stm32mp157d-atk.dtb，使用新的 stm32mp157d-atk.dtb 启动 Linux 内核。Linux 启动成功以后进入到/proc/device-tree/目录中查看是否有“stm32mp1_led”这个节点，结果如图 24.3.1.1 所示：

```
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base$ ls
#address-cells                  model
#size-cells                     name
aliases                         pm_domain
arm-pmu                         psci
buck1                           reboot
chosen                          regulator-3p3v
compatible                      regulator-booster
cpu0-opp-table                  reserved-memory
cpus                            serial-number
firmware                        soc
interrupt-controller@a0021000   sram@2ffff000
mailbox-0                       stm32mp1_led
mailbox-1                       thermal-zones
memory@c0000000                 timer
mlahb
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base$
```

图 24.3.1.1 stm32mp1_led 节点

如果没有“stm32mp1_led”节点的话请重点检查下面两点：

①、检查设备树修改是否成功，也就是 stm32mp1_led 节点是否为根节点“/”的子节点。

②、检查是否使用新的设备树启动的 Linux 内核。

可以进入到图 24.3.1.1 中的 stm32mpl_led 目录中，查看一下都有哪些属性文件，结果如图 24.3.1.2 所示：

```
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base$ cd stm32mp1_led/
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base/stm32mp1_led$ ls
compatible  name        reg         status
[root@ATK-stm32mp1]:/sys/firmware/devicetree/base/stm32mp1_led$
```

图 24.3.1.2 stm32mp1_led 节点文件

大家可以用 cat 命令查看一下 compatible、status 等属性值是否和我们设置的一致。

24.3.2 LED 灯驱动程序编写

设备树准备好以后就可以编写驱动程序了，本章实验在第二十二章实验驱动文件 newchrled.c 的基础上修改而来。新建名为“4_dtsled”文件夹，然后在 4_dtsled 文件夹里面创建 vscode 工程，工作区命名为“dtsled”。工程创建好以后新建 dtsled.c 文件，在 dtsled.c 里面输入如下内容：

示例代码 24.3.2.1 dtsled.c 文件内容

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <linux/of.h>
12 #include <linux/of_address.h>
13 #include <asm/mach/map.h>
14 #include <asm/uaccess.h>
```

```
15  #include <asm/io.h>
16
17  /***************************************************************
18  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
19  文件名      : dtsled.c
20  作者        : 正点原子 Linux 团队
21  版本        : V1.0
22  描述        : LED 驱动文件。
23  其他        : 无
24  论坛        : www.openedv.com
25  日志        : 初版 V1.0 2020/12/19 正点原子 Linux 团队创建
26  ***************************************************************/
27  #define DTSLED_CNT      1           /* 设备号个数    */
28  #define DTSLED_NAME     "dtsled"    /* 名字          */
29  #define LEDOFF          0           /* 关灯          */
30  #define LEDON           1           /* 开灯          */
31
32  /* 映射后的寄存器虚拟地址指针 */
33  static void __iomem *MPU_AHB4_PERIPH_RCC_PI;
34  static void __iomem *GPIOI_MODER_PI;
35  static void __iomem *GPIOI_OTYPER_PI;
36  static void __iomem *GPIOI_OSPEEDR_PI;
37  static void __iomem *GPIOI_PUPDR_PI;
38  static void __iomem *GPIOI_BSRR_PI;
39
40  /* dtsled 设备结构体 */
41  struct dtsled_dev{
42      dev_t devid;              /* 设备号       */
43      struct cdev cdev;         /* cdev         */
44      struct class *class;      /* 类           */
45      struct device *device;    /* 设备         */
46      int major;                /* 主设备号     */
47      int minor;                /* 次设备号     */
48      struct device_node  *nd; /* 设备节点     */
49  };
50
51  struct dtsled_dev dtsled;    /* led 设备     */
52
53  /*
54   * @description    : LED 打开/关闭
55   * @param - sta    : LEDON(0) 打开 LED，LEDOFF(1) 关闭 LED
56   * @return         : 无
57   */
```

```
58  void led_switch(u8 sta)
59  {
60      u32 val = 0;
61      if(sta == LEDON) {
62          val = readl(GPIOI_BSRR_PI);
63          val |= (1 << 16);
64          writel(val, GPIOI_BSRR_PI);
65      }else if(sta == LEDOFF) {
66          val = readl(GPIOI_BSRR_PI);
67          val|= (1 << 0);
68          writel(val, GPIOI_BSRR_PI);
69      }
70  }
71
72  /*
73   * @description   : 取消映射
74   * @return        : 无
75   */
76  void led_unmap(void)
77  {
78      /* 取消映射 */
79      iounmap(MPU_AHB4_PERIPH_RCC_PI);
80      iounmap(GPIOI_MODER_PI);
81      iounmap(GPIOI_OTYPER_PI);
82      iounmap(GPIOI_OSPEEDR_PI);
83      iounmap(GPIOI_PUPDR_PI);
84      iounmap(GPIOI_BSRR_PI);
85  }
86
87  /*
88   * @description   : 打开设备
89   * @param - inode : 传递给驱动的 inode
90   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
91   *                  一般在 open 的时候将 private_data 指向设备结构体。
92   * @return        : 0 成功;其他 失败
93   */
94  static int led_open(struct inode *inode, struct file *filp)
95  {
96      filp->private_data = &dtsled; /* 设置私有数据 */
97      return 0;
98  }
99
100 /*
```

```
101  * @description   : 从设备读取数据
102  * @param - filp  : 要打开的设备文件(文件描述符)
103  * @param - buf   : 返回给用户空间的数据缓冲区
104  * @param - cnt   : 要读取的数据长度
105  * @param - offt  : 相对于文件首地址的偏移
106  * @return        : 读取的字节数，如果为负值，表示读取失败
107  */
108 static ssize_t led_read(struct file *filp, char __user *buf,
                            size_t cnt, loff_t *offt)
109 {
110     return 0;
111 }
112
113 /*
114  * @description   : 向设备写数据
115  * @param - filp  : 设备文件，表示打开的文件描述符
116  * @param - buf   : 要写给设备写入的数据
117  * @param - cnt   : 要写入的数据长度
118  * @param - offt  : 相对于文件首地址的偏移
119  * @return        : 写入的字节数，如果为负值，表示写入失败
120  */
121 static ssize_t led_write(struct file *filp, const char __user *buf,
                            size_t cnt, loff_t *offt)
122 {
123     int retvalue;
124     unsigned char databuf[1];
125     unsigned char ledstat;
126
127     retvalue = copy_from_user(databuf, buf, cnt);
128     if(retvalue < 0) {
129         printk("kernel write failed!\r\n");
130         return -EFAULT;
131     }
132
133     ledstat = databuf[0];       /* 获取状态值         */
134
135     if(ledstat == LEDON) {
136         led_switch(LEDON);        /* 打开 LED 灯      */
137     } else if(ledstat == LEDOFF) {
138         led_switch(LEDOFF);       /* 关闭 LED 灯      */
139     }
140     return 0;
141 }
```

```
142
143 /*
144  * @description   : 关闭/释放设备
145  * @param - filp  : 要关闭的设备文件(文件描述符)
146  * @return        : 0 成功;其他 失败
147  */
148 static int led_release(struct inode *inode, struct file *filp)
149 {
150     return 0;
151 }
152
153 /* 设备操作函数 */
154 static struct file_operations dtsled_fops = {
155     .owner = THIS_MODULE,
156     .open = led_open,
157     .read = led_read,
158     .write = led_write,
159     .release =  led_release,
160 };
161
162 /*
163  * @description   : 驱动出口函数
164  * @param         : 无
165  * @return        : 无
166  */
167 static int __init led_init(void)
168 {
169     u32 val = 0;
170     int ret;
171     u32 regdata[12];
172     const char *str;
173     struct property *proper;
174
175     /* 获取设备树中的属性数据           */
176     /* 1、获取设备节点: stm32mp1_led  */
177     dtsled.nd = of_find_node_by_path("/stm32mp1_led");
178     if(dtsled.nd == NULL) {
179         printk("stm32mp1_led node nost find!\r\n");
180         return -EINVAL;
181     } else {
182         printk("stm32mp1_lcd node find!\r\n");
183     }
184
```

```
185     /* 2、获取 compatible 属性内容 */
186     proper = of_find_property(dtsled.nd, "compatible", NULL);
187     if(proper == NULL) {
188         printk("compatible property find failed\r\n");
189     } else {
190         printk("compatible = %s\r\n", (char*)proper->value);
191     }
192
193     /* 3、获取 status 属性内容 */
194     ret = of_property_read_string(dtsled.nd, "status", &str);
195     if(ret < 0){
196         printk("status read failed!\r\n");
197     } else {
198         printk("status = %s\r\n",str);
199     }
200
201     /* 4、获取 reg 属性内容 */
202     ret = of_property_read_u32_array(dtsled.nd, "reg", regdata, 12);
203     if(ret < 0) {
204         printk("reg property read failed!\r\n");
205     } else {
206         u8 i = 0;
207         printk("reg data:\r\n");
208         for(i = 0; i < 12; i++)
209             printk("%#X ", regdata[i]);
210         printk("\r\n");
211     }
212
213     /* 初始化 LED           */
214     /* 1、寄存器地址映射    */
215     MPU_AHB4_PERIPH_RCC_PI = of_iomap(dtsled.nd, 0);
216     GPIOI_MODER_PI = of_iomap(dtsled.nd, 1);
217     GPIOI_OTYPER_PI = of_iomap(dtsled.nd, 2);
218     GPIOI_OSPEEDR_PI = of_iomap(dtsled.nd, 3);
219     GPIOI_PUPDR_PI = of_iomap(dtsled.nd, 4);
220     GPIOI_BSRR_PI = of_iomap(dtsled.nd, 5);
221
222     /* 2、使能 PI 时钟 */
223     val = readl(MPU_AHB4_PERIPH_RCC_PI);
224     val &= ~(0X1 << 8);  /* 清除以前的设置    */
225     val |= (0X1 << 8);   /* 设置新值          */
226     writel(val, MPU_AHB4_PERIPH_RCC_PI);
227
```

```
228     /* 3、设置 PI0 通用的输出模式。*/
229     val = readl(GPIOI_MODER_PI);
230     val &= ~(0X3 << 0);      /* bit0:1 清零      */
231     val |= (0X1 << 0);       /* bit0:1 设置 01   */
232     writel(val, GPIOI_MODER_PI);
233
234     /* 3、设置 PI0 为推挽模式。*/
235     val = readl(GPIOI_OTYPER_PI);
236     val &= ~(0X1 << 0);      /* bit0 清零，设置为上拉*/
237     writel(val, GPIOI_OTYPER_PI);
238
239     /* 4、设置 PI0 为高速。*/
240     val = readl(GPIOI_OSPEEDR_PI);
241     val &= ~(0X3 << 0);      /* bit0:1 清零         */
242     val |= (0x2 << 0);       /* bit0:1 设置为 10    */
243     writel(val, GPIOI_OSPEEDR_PI);
244
245     /* 5、设置 PI0 为上拉。*/
246     val = readl(GPIOI_PUPDR_PI);
247     val &= ~(0X3 << 0);      /* bit0:1 清零         */
248     val |= (0x1 << 0);       /* bit0:1 设置为 01    */
249     writel(val,GPIOI_PUPDR_PI);
250
251     /* 6、默认关闭 LED */
252     val = readl(GPIOI_BSRR_PI);
253     val |= (0x1 << 0);
254     writel(val, GPIOI_BSRR_PI);
255
256     /* 注册字符设备驱动 */
257     /* 1、创建设备号 */
258     if (dtsled.major) {      /*  定义了设备号        */
259         dtsled.devid = MKDEV(dtsled.major, 0);
260         ret = register_chrdev_region(dtsled.devid, DTSLED_CNT,
DTSLED_NAME);
261         if(ret < 0) {
262             pr_err("cannot register %s char driver
[ret=%d]\n",DTSLED_NAME, DTSLED_CNT);
263             goto fail_map;
264         }
265     } else {                 /* 没有定义设备号       */
266         ret = alloc_chrdev_region(&dtsled.devid, 0, DTSLED_CNT,
                                DTSLED_NAME);   /* 申请设备号 */
267         if(ret < 0) {
```

```
268             pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                        DTSLED_NAME, ret);
269             goto fail_map;
270         }
271         dtsled.major = MAJOR(dtsled.devid); /* 获取分配号的主设备号 */
272         dtsled.minor = MINOR(dtsled.devid); /* 获取分配号的次设备号 */
273 
274     }
275     printk("dtsled major=%d,minor=%d\r\n",dtsled.major,
              dtsled.minor);
276 
277     /* 2、初始化 cdev */
278     dtsled.cdev.owner = THIS_MODULE;
279     cdev_init(&dtsled.cdev, &dtsled_fops);
280 
281     /* 3、添加一个 cdev */
282     ret = cdev_add(&dtsled.cdev, dtsled.devid, DTSLED_CNT);
283     if(ret < 0)
284         goto del_unregister;
285 
286     /* 4、创建类        */
287     dtsled.class = class_create(THIS_MODULE, DTSLED_NAME);
288     if (IS_ERR(dtsled.class)) {
289         goto del_cdev;
290     }
291 
292     /* 5、创建设备      */
293     dtsled.device = device_create(dtsled.class, NULL, dtsled.devid,
                                      NULL, DTSLED_NAME);
294     if (IS_ERR(dtsled.device)) {
295         goto destroy_class;
296     }
297 
298     return 0;
299 
300 destroy_class:
301     class_destroy(dtsled.class);
302 del_cdev:
303     cdev_del(&dtsled.cdev);
304 del_unregister:
305     unregister_chrdev_region(dtsled.devid, DTSLED_CNT);
306 fail_map:
307     led_unmap();
```

```
308     return -EIO;
309 }
310
311 /*
312  * @description   : 驱动出口函数
313  * @param         : 无
314  * @return        : 无
315  */
316 static void __exit led_exit(void)
317 {
318     /* 取消映射         */
319     led_unmap();
320
321     /* 注销字符设备驱动 */
322     cdev_del(&dtsled.cdev);  /*  删除 cdev */
323     unregister_chrdev_region(dtsled.devid, DTSLED_CNT); /*注销*/
324
325     device_destroy(dtsled.class, dtsled.devid);
326     class_destroy(dtsled.class);
327 }
328
329 module_init(led_init);
330 module_exit(led_exit);
331 MODULE_LICENSE("GPL");
332 MODULE_AUTHOR("ALIENTEK");
333 MODULE_INFO(intree, "Y");
```

dtsled.c 文件中的内容和第二十二章的 newchrled.c 文件中的内容基本一样，只是 dtsled.c 中包含了处理设备树的代码，我们重点来看一下这部分代码。

第 48 行，在设备结构体 dtsled_dev 中添加了成员变量 nd，nd 是 device_node 结构体类型指针，表示设备节点。如果我们要读取设备树某个节点的属性值，首先要先得到这个节点，一般在设备结构体中添加 device_node 指针变量来存放这个节点。

第 177~183 行，通过 of_find_node_by_path 函数得到 stm32mp1_led 节点，后续其他的 OF 函数要使用 device_node。

第 186~191 行，通过 of_find_property 函数获取 stm32mp1_led 节点的 compatible 属性，返回值为 property 结构体类型指针变量，property 的成员变量 value 表示属性值。

第 194~199 行，通过 of_property_read_string 函数获取 stm32mp1_led 节点的 status 属性值。

第 202~211 行，通过 of_property_read_u32_array 函数获取 stm32mp1_led 节点的 reg 属性所有值，并且将获取到的值都存放到 regdata 数组中。第 209 行将获取到的 reg 属性值依次输出到终端上。

第 215~220 行，使用 of_iomap 函数一次性完成读取 reg 属性以及内存映射，of_iomap 函数是设备树推荐使用的 OF 函数。

24.3.3 编写测试 APP

本章直接使用第二十二章的测试 APP，将上一章的 ledApp.c 文件复制到本章实验工程下即可。

24.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第二十章实验基本一样，只是将 obj-m 变量的值改为 dtsled.o，Makefile 内容如下所示：

示例代码 24.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := dtsled.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 dtsled.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“dtsled.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 ledApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc ledApp.c -o ledApp
```

编译成功以后就会生成 ledApp 这个应用程序。

24.4.2 运行测试

将上一小节编译出来的 dtsled.ko 和 ledApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 dtsled.ko 驱动模块：

```
depmod          //第一次加载驱动的时候需要运行此命令
modprobe dtsled //加载驱动
```

驱动加载成功以后会在终端中输出一些信息，如图 24.4.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe dtsled.ko
stm32mp1_lcd node find!
compatible = atkstm32mp1-led
status = okay
reg data:
0X50000A28
0X4
0X5000A000
0X4
0X5000A004
0X4
0X5000A008
0X4
0X5000A00C
0X4
0X5000A018
0X4

dtsled major=241,minor=0
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 24.4.2.1 驱动加载成功以后输出的信息

从图 24.4.2.1 可以看出，stm32mp1_led 这个节点找到了，并且 compatible 属性值为“atkstm32mp1-led”，status 属性值为“okay”，reg 属性的值为“0X50000A28 0X4 0X5000A000 0X4 0X5000A004 0X4 0X5000A008 0X4 0X5000A00C 0X4 0X5000A018 0X4”，这些都和我们设置的设备树一致。

驱动加载成功以后就可以使用 ledApp 软件来测试驱动是否工作正常，输入如下命令打开 LED 灯：

```
./ledApp /dev/dtsled 1          //打开 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯是否点亮，如果点亮的话说明驱动工作正常。在输入如下命令关闭 LED 灯：

```
./ledApp  /dev/dtsled 0         //关闭 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯是否熄灭。如果要卸载驱动的话输入如下命令即可：

```
rmmod dtsled.ko
```

第二十五章 pinctrl 和 gpio 子系统实验

上一章我们编写了基于设备树的 LED 驱动，但是驱动的本质还是没变，都是配置 LED 灯所使用的 GPIO 寄存器，驱动开发方式和裸机基本没啥区别。Linux 是一个庞大而完善的系统，尤其是驱动框架，像 GPIO 这种最基本的驱动不可能采用“原始”的裸机驱动开发方式，否则就相当于你买了一辆车，结果每天推着车去上班。Linux 内核提供了 pinctrl 和 gpio 子系统用于 GPIO 驱动，本章我们就来学习一下如何借助 pinctrl 和 gpio 子系统来简化 GPIO 驱动开发。

25.1 pinctrl 子系统

25.1.1 pinctrl 子系统简介

Linux 驱动讲究驱动分离与分层，pinctrl 和 gpio 子系统就是驱动分离与分层思想下的产物，驱动分离与分层其实就是按照面向对象编程的设计思想而设计的设备驱动框架，关于驱动的分离与分层我们后面会讲。本来 pinctrl 和 gpio 子系统应该放到驱动分离与分层章节后面讲解，但是不管什么外设驱动，GPIO 驱动基本都是必须的，而 pinctrl 和 gpio 子系统又是 GPIO 驱动必须使用的，所以就将 pintrcl 和 gpio 子系统这一章节提前了。

我们先来回顾一下上一章是怎么初始化 LED 灯所使用的 GPIO，步骤如下：

①、修改设备树，添加相应的节点，节点里面重点是设置 reg 属性，reg 属性包括了 GPIO 相关寄存器。

②、 获取 reg 属性中 GPIOI_MODER、GPIOI_OTYPER、GPIOI_OSPEEDR、GPIOI_PUPDR 和 GPIOI_BSRR 这些寄存器的地址，并且初始化它们，这些寄存器用于设置 PI0 这个 PIN 的复用功能、上下拉、速度等。

③、 在②里面将 PI0 这个 PIN 设置为通用输出功能，因此需要设置 PI0 这个 GPIO 相关的寄存器，也就是设置 GPIOI_MODER 寄存器。

④、在②里面将 PI0 这个 PIN 设置为高速、上拉和推挽模式，就要需要设置 PI0 的 GPIOI_OTYPER、GPIOI_OSPEEDR 和 GPIOI_PUPDR 这些寄存器。

总结一下，②中完成对 PI0 这个 PIN 的获取相关的寄存器地址，③和④设置这个 PIN 的复用功能、上下拉等，比如将 GPIOI_0 这个 PIN 设置为通用输出功能。如果使用过 STM32 单片机的话应该都记得，STM32 单片机也是要先设置某个 PIN 的复用功能、速度、上下拉等，然后再设置 PIN 所对应的 GPIO，STM32MP1 是从 STM32 单片机发展而来的，设置 GPIO 是一样。其实对于大多数的 32 位 SOC 而言，引脚的设置基本都是这两方面，因此 Linux 内核针对 PIN 的复用配置推出了 pinctrl 子系统，对于 GPIO 的电气属性配置推出了 gpio 子系统。本小节我们来学习 pinctrl 子系统，下一小节再学习 gpio 子系统。

大多数 SOC 的 pin 都是支持复用的，比如 STM32MP1 的 PI0 既可以作为普通的 GPIO 使用，也可以作为 SPI2 的 NSS 引脚、TIM5 的 CH4 引脚等等。此外我们还需要配置 pin 的电气特性，比如上/下拉、速度、驱动能力等等。传统的配置 pin 的方式就是直接操作相应的寄存器，但是这种配置方式比较繁琐、而且容易出问题(比如 pin 功能冲突)。pinctrl 子系统就是为了解决这个问题而引入的，pinctrl 子系统主要工作内容如下：

①、获取设备树中 pin 信息。

②、根据获取到的 pin 信息来设置 pin 的复用功能

③、根据获取到的 pin 信息来设置 pin 的电气特性，比如上/下拉、速度、驱动能力等。

对于我们使用者来讲，只需要在设备树里面设置好某个 pin 的相关属性即可，其他的初始化工作均由 pinctrl 子系统来完成，pinctrl 子系统源码目录为 drivers/pinctrl。

25.1.2 STM32MP1 的 pinctrl 子系统驱动

1、PIN 配置信息详解

要使用 pinctrl 子系统，我们需要在设备树里面设置 PIN 的配置信息，毕竟 pinctrl 子系统要根据你提供的信息来配置 PIN 功能，一般会在设备树里面创建一个节点来描述 PIN 的配置信息。打开 stm32mp151.dtsi 文件，找到一个叫做 pinctrl 的节点，如下所示：

示例代码 25.1.2.1 pinctrl 节点内容 1

```
1814        pinctrl: pin-controller@50002000 {
1815            #address-cells = <1>;
1816            #size-cells = <1>;
1817            compatible = "st,stm32mp157-pinctrl";
1818            ranges = <0 0x50002000 0xa400>;
1819            interrupt-parent = <&exti>;
1820            st,syscfg = <&exti 0x60 0xff>;
1821            hwlocks = <&hsem 0 1>;
1822            pins-are-numbered;
......
1968        };
```

第 1815~1816 行，#address-cells 属性值为 1 和#size-cells 属性值为 1，也就是说 pinctrl 下的所有子节点的 reg 第一位是起始地址，第二位为长度。

第 1818 行，ranges 属性表示 STM32MP1 的 GPIO 相关寄存器起始地址，STM32MP1 系列芯片最多拥有 176 个通用 GPIO，分为 12 组，分别为：PA0~PA15、PB0~PB15、PC0~PC15、PD0~PD15、PE0~PE15、PF0~PF15、PG0~PG15、PH0~PH15、PI0~PI15、PJ0~PJ15、PK0~PK7、PZ0~PZ7。

其中 PA~PK 这 11 组 GPIO 的寄存器都在一起，起始地址为 0X50002000，终止地址为 0X5000C3FF。这个可以在《STM32MP157 参考手册》里面找到。PZ 组寄存器起始地址为 0X54004000，终止地址为 0X540043FF，所以 stm32mp151.dtsi 文件里面还有个名为“pinctrl_z”的子节点来描述 PZ 组 IO。pinctrl 节点用来描述 PA~PK 这 11 组 IO，因此 ranges 属性中的 0x50002000 表示起始地址，0xa400 表示寄存器地址范围。

第 1819 行，interrupt-parent 属性值为“&exti”，父中断为 exti。

后面的 gpiox 子节点先不分析，这些子节点都是 gpio 子系统的内容，到后面在去分析。这个节点看起来，根本没有 PIN 相关的配置，别急！先打开 stm32mp15-pinctrl.dtsi 文件，你们能找到如下内容：

示例代码 25.1.2.2 pinctrl 节点内容 2

```
1   &pinctrl {
......
534     m_can1_pins_a: m-can1-0 {
535         pins1 {
536             pinmux = <STM32_PINMUX('H', 13, AF9)>;     /* CAN1_TX */
537             slew-rate = <1>;
538             drive-push-pull;
539             bias-disable;
540         };
541         pins2 {
542             pinmux = <STM32_PINMUX('I', 9, AF9)>;  /* CAN1_RX */
543             bias-disable;
544         };
545     };
......
```

```
554     pwm1_pins_a: pwm1-0 {
555         pins {
556             pinmux = <STM32_PINMUX('E', 9, AF1)>, /* TIM1_CH1 */
557                 <STM32_PINMUX('E', 11, AF1)>,     /* TIM1_CH2 */
558                 <STM32_PINMUX('E', 14, AF1)>;     /* TIM1_CH4 */
559             bias-pull-down;
560             drive-push-pull;
561             slew-rate = <0>;
562         };
563     };
......
1411};
```

示例代码 25.1.2.2 就是向 pinctrl 节点追加数据，不同的外设使用的 PIN 不同、其配置也不同，因此一个萝卜一个坑，将某个外设所使用的所有 PIN 都组织在一个子节点里面。示例代码 25.1.2.2 中 m_can1_pins_a 子节点就是 CAN1 的所有相关 IO 的 PIN 集合，pwm1_pins_a 子节点就是 PWM1 相关 IO 的 PIN 集合。绑定文档 Documentation/devicetree/bindings/pinctrl/st,stm32-pinctrl.yaml 描述了如何在设备树中设置 STM32 的 PIN 信息：

每个 pincrtl 节点必须至少包含一个子节点来存放 pincrtl 相关信息，也就是 pinctrl 集，这个集合里面存放当前外设用到哪些引脚(PIN)、这些引脚应该怎么配置、复用相关的配置、上下拉、默认输出高电平还是低电平。一般这个存放 pincrtl 集的子节点名字是“pins”，如果某个外设用到多种配置不同的引脚那么就需要多个 pins 子节点，比如示例代码 25.1.2.2 中第 535 行和 541 行的 pins1 和 pins2 这两个子节点分别描述 PH13 和 PI9 的配置方法，由于 PH13 和 PI9 这两个 IO 的配置不同，因此需要两个 pins 子节点来分别描述。第 555~562 行的 pins 子节点是描述 PWM1 的相关引脚，包括 PE9、PE11、PE14，由于这三个引脚的配置是一模一样的，因此只需要一个 pins 子节点就可以了。

上面讲了，在 pins 子节点里面存放外设的引脚描述信息，这些信息包括：

1、pinmux 属性

此属性用来存放外设所要使用的所有 IO，示例代码 25.1.2.2 中第 536 行的 pinmux 属性内容如下：

```
pinmux = <STM32_PINMUX('H', 13, AF9)>;
```

可以看出，这里使用 STM32_PINMUX 这宏来配置引脚和引脚的复用功能，此宏定义在 include/dt-bindings/pinctrl/stm32-pinfunc.h 文件里面，内容如下：

示例代码 25.1.2.3 STM32_PINMUX 宏定义

```
32 #define PIN_NO(port, line)   (((port) - 'A') * 0x10 + (line))
33
34 #define STM32_PINMUX(port, line, mode) (((PIN_NO(port, line)) << 8)
                                          | (mode))
```

可以看出，STM32_PINMUX 宏有三个参数，这三个参数含义如下所示：

port：表示用那一组 GPIO(例：H 表示为 GPIO 第 H 组，也就是 GPIOH)。

line：表示这组 GPIO 的第几个引脚(例：13 表示为 GPIOH_13，也就是 PH13)。

mode：表示当前引脚要做那种复用功能(例：AF9 表示为用第 9 个复用功能)，这个需要查阅 STM32MP1 数据手册来确定使用哪个复用功能。打开《STM32MP157A&D 数据手册》，一个 IO 最大有 16 种复用方法：AF0~AF15，打开第 4 章“Pinouts, pin description and alternate functions”，

其中“Table 8. Alternate function AF0 to AF7”描述的是 AF0~AF7 这 8 种复用方式，“Table 9. Alternate function AF8 to AF15”描述的是 AF8~AF15 这 8 中复用方式。找到图 25.1.2.1 所示内容：

Table 9. Alternate function AF8 to AF15(1) (continued)

Port		AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SPI6/SAI2/ USART3/ UART4/5/8/ SDMMC1/2/ SPDIFRX	FDCAN1/2/ TIM13/14/ QUADSPI/ SDMMC2/3/ LCD/SPDIFRX	SAI2/4/ QUADSPI/ FMC/ SDMMC2/3/ OTG_FS/ OTG_HS	DFSDM1/ QUADSPI/ SDMMC1/ MDIOS/ETH1/ DSI	SAI4/UART5/ FMC/SDMMC1/ MDIOS	UART7/DCMI/ LCD/DSI/RNG	UART5/LCD	SYS
Port H	PH9	-	-	-	-	-	DCMI_D0	LCD_R3	EVENTOUT
	PH10	-	-	-	-	-	DCMI_D1	LCD_R4	EVENTOUT
	PH11	-	-	-	-	-	DCMI_D2	LCD_R5	EVENTOUT
	PH12	-	-	-	-	-	DCMI_D3	LCD_R6	EVENTOUT
	PH13	UART4_TX	FDCAN1_TX	-	-	-	-	LCD_G2	EVENTOUT
	PH14	UART4_RX	FDCAN1_RX	-	-	-	DCMI_D4	LCD_G3	EVENTOUT

图 25.1.2.1 PH13 引脚复用功能

可以看出，PH13 要设置为 FDCAN1 的 TX 引脚，就要使用 AF9 这个复用配置，这个就是如何在 pinmux 属性中添加一个引脚的复用配置。从图 25.1.2.1 中可以看出，如果 PH13 也可以复用为 UART4 的 TX 引脚，只要设置为 AF8 即可，此时相关的引脚配置如下所示：

示例代码 25.1.2.3 myuart4 pinctrl 节点

```
1  &pinctrl {
2
3      myuart4_pins_a: myuart4-0 {
4          pins{
5              pinmux = <STM32_PINMUX('H', 13, AF8)>;
6          };
7      };
8  };
```

这时候大家可能会奇怪？PH13 做了 FDCAN1 的 TX 功能，还能做 UART4 的 TX 功能，不是冲突了吗？这是不会冲突的，因为 pinctrl 驱动只会把设备树 pinctrl 节点解析出来的数据存储到一个链表里，只有当外设调用这个 pinctrl 节点的时候才会真的使用。但是，如果你要同时使用 FDCAN1 和 UART4 的话就会出问题，因此 PH13 同一时间只能用于一个外设，所以为了方便开发，还是建议大家一个 PIN 最好只能被一个外设使用。

stm32-pinfunc.h 文件里面定义了一个 PIN 的所有配置项 AF0~AF9，如下所示：

示例代码 25.1.2.4 stm32-pinfunc.h 文件

```
10 /*  define PIN modes */
11 #define GPIO      0x0
12 #define AF0       0x1
13 #define AF1       0x2
14 #define AF2       0x3
15 #define AF3       0x4
16 #define AF4       0x5
17 #define AF5       0x6
18 #define AF6       0x7
19 #define AF7       0x8
```

```
20  #define AF8       0x9
21  #define AF9       0xa
22  #define AF10      0xb
23  #define AF11      0xc
24  #define AF12      0xd
25  #define AF13      0xe
26  #define AF14      0xf
27  #define AF15      0x10
28  #define ANALOG    0x11
29  #define RSVD      0x12
```

可以看出，除了 A0~AF15，还有 GPIO、ANALOG 这两个，如果一个 PIN 只是作为最基本的 GPIO 功能，那么就是用“GPIO”；如果这个引脚要用作模拟功能，比如 ADC 采集引脚，那么就设置为“ANALOG”。

2、电气属性配置

接下来了解一下 PIN 的电气特性如何设置，电气特性在 pinctrl 子系统里不是必须的，可以不配置，但是 pinmux 属性是必须要设置的。stm32-pinctrl.yaml 文件里面也描述了如何设置 STM32 的电气属性，如表 25.1.2.1 所示：

电气特性属性	类型	作用
bias-disable	bootlean	禁止使用内部偏置电压
bias-pull-down	bootlean	内部下拉
bias-pull-up	bootlean	内部上拉
drive-push-pull	bootlean	推挽输出
drive-open-drain	bootlean	开漏输出
output-low	bootlean	输出低电平
output-high	bootlean	输出高电平
slew-rate	enum	引脚的速度，可设置：0~3， 0 最慢，3 最高。

表 25.1.2.1 pinctrl 的电气特性

表 25.1.2.1 里的 bootlean 类型表示了在 pinctrl 子系统只要定义这个电气属性就行了，例如：我要禁用内部电压，只要在 PIN 的配置集里添加“bias-disable”即可，这个时候 bias-pull-down 和 bias-pull-up 都不能使用了，因为已经禁用了内部电压，所以不能配置上下拉。enum 类型使用方法更简单跟 C 语言的一样，比如要设置 PIN 速度为最低就可以使用“slew-rate=<0>”。在示例代码 25.1.2.3 里添加引脚的电气特性组合成，如示例代码 25.1.2.6 所示：

示例代码 25.1.2.6 添加电气特性的 myuart4-0 节点

```
1  &pinctrl {
2
3      myuart4_pins_a: myuart4-0 {
4          pins1{
5              pinmux = <STM32_PINMUX('H', 13, AF8)>;
6              bias-pull-up;
7              drive-push-pull;
8            };
9      };
```

```
10  };
```

在第 6~7 行里给 myuart4-0 添加了两个电气属性分别为内部上拉和推挽输出，这样我们就设置好一个 PIN 配置了。

2、PIN 驱动程序讲解

本小节会涉及到 Linux 驱动分层与分离、平台设备驱动等还未讲解的知识，所以本小节教程可以不用看，不会影响后续的实验。如果对 Linux 内核的 pinctrl 子系统实现原理感兴趣的话可以看本小节。

所有的东西都已经准备好了，包括寄存器地址和寄存器值，Linux 内核相应的驱动文件就会根据这些值来做相应的初始化。接下来就找一下哪个驱动文件来做这一件事情，pinctrl 节点中 compatible 属性的值为“st,stm32mp157-pinctrl”，在 Linux 内核中全局搜索“pinctrl”字符串就会找到对应的驱动文件。在文件 drivers/pinctrl/stm32/pinctrl-stm32mp157.c 中有如下内容：

示例代码 25.1.2.7 pinctrl-stm32mp157.c 文件代码段

```
2323  static struct stm32_pinctrl_match_data stm32mp157_match_data = {
2324      .pins = stm32mp157_pins,
2325      .npins = ARRAY_SIZE(stm32mp157_pins),
2326  };
2327
2328  static struct stm32_pinctrl_match_data stm32mp157_z_match_data =
{
2329      .pins = stm32mp157_z_pins,
2330      .npins = ARRAY_SIZE(stm32mp157_z_pins),
2331      .pin_base_shift = STM32MP157_Z_BASE_SHIFT,
2332  };
2333
2334  static const struct of_device_id stm32mp157_pctrl_match[] = {
2335      {
2336          .compatible = "st,stm32mp157-pinctrl",
2337          .data = &stm32mp157_match_data,
2338      },
2339      {
2340          .compatible = "st,stm32mp157-z-pinctrl",
2341          .data = &stm32mp157_z_match_data,
2342      },
2343      { }
2344  };
2345
2346  static const struct dev_pm_ops stm32_pinctrl_dev_pm_ops = {
2347       SET_LATE_SYSTEM_SLEEP_PM_OPS(NULL, stm32_pinctrl_resume)
2348  };
2349
2350  static struct platform_driver stm32mp157_pinctrl_driver = {
2351      .probe = stm32_pctl_probe,
2352      .driver = {
```

```
2353            .name = "stm32mp157-pinctrl",
2354            .of_match_table = stm32mp157_pctrl_match,
2355            .pm = &stm32_pinctrl_dev_pm_ops,
2356        },
2357    };
2358
2359    static int __init stm32mp157_pinctrl_init(void)
2360    {
2361        return platform_driver_register(&stm32mp157_pinctrl_driver);
2362    }
2363    arch_initcall(stm32mp157_pinctrl_init);
2364
```

第 2334~2344 行，of_device_id 结构体类型的数组，在第二十三章讲解设备树的时候说过，of_device_id 里面保存着这个驱动文件的兼容性值，设备树中的 compatible 属性值会和 of_device_id 中的所有兼容性字符串比较，查看是否可以使用此驱动。stm32mp157_pctrl_match 结构体数组一共有两个兼容，分别为“st,stm32mp157-pinctrl”和“st,stm32mp157-z-pinctrl”，设备树也定义了这两个兼容性值，因此 pinctrl 和 pinctrl_z 节点都会和此驱动匹配，所以 pinctrl-stm32mp157.c 会完成 STM32MP1 的 PIN 配置工作。

第 2350~2357 行，platform_driver 是平台设备驱动，这个是我们后面章节要讲解的内容，platform_driver 是个结构体，有个 probe 成员变量。在这里大家只需要知道，当设备和驱动匹配成功以后 platform_driver 的 probe 成员变量所代表的函数就会执行，在 2351 行设置 probe 成员变量为 stm32_pctl_probe 函数，因此在本章实验中 stm32_pctl_probe 这个函数就会执行，可以认为 stm32_pctl_probe 函数就是 STM32MP157 这个 SOC 的 PIN 配置入口函数。

第 2359~2362 行，就是一个简单的驱动入口函数，platform_driver_register 函数是一个标准的平台设备驱动注册函数，用于向 Linux 内核注册一个 platform_driver，这里就是将 stm32mp157_pinctrl_driver 注册到 Linux 内核总线，关于平台设备驱动后面章节会详细讲解。

我们重点来分析一下 stm32_pctl_probe 函数，函数定义在 drivers/pinctrl/stm32/pinctrl-stm32.c 里面，函数内容如下所示：

示例代码 25.1.2.8 stm32_pctl_probe 代码段

```
1452    int stm32_pctl_probe(struct platform_device *pdev)
1453    {
......
1458        struct stm32_pinctrl *pctl;
......
1530
1531        pctl->pctl_desc.name = dev_name(&pdev->dev);
1532        pctl->pctl_desc.owner = THIS_MODULE;
1533        pctl->pctl_desc.pins = pins;
1534        pctl->pctl_desc.npins = pctl->npins;
1535        pctl->pctl_desc.link_consumers = true;
1536        pctl->pctl_desc.confops = &stm32_pconf_ops;
1537        pctl->pctl_desc.pctlops = &stm32_pctrl_ops;
1538        pctl->pctl_desc.pmxops = &stm32_pmx_ops;
```

```
1539        pctl->dev = &pdev->dev;
1540        pctl->pin_base_shift = pctl->match_data->pin_base_shift;
1541
1542        pctl->pctl_dev = devm_pinctrl_register(&pdev->dev,
1543                        &pctl->pctl_desc, pctl);
......
1600};
```

第 1458 行，看它的结构体的名字就知道是 ST 官方自定义的一个结构体类型，用于存放 STM32 相关 PIN 属性集合。第 22.5.1 小节我们驱动代码也添加了自己的结构体，它们都有自己的结构体这样做有啥好处？可以实现一个驱动代码“通杀”多个设备。要想驱动能通用，就要用结构体来保存数据和驱动里面尽量不要使用全局变量(在 pinctrl 驱动里就没有一个全局变量，全部使用结构体来描述一个物体，物体的所有属性都作为结构体成员变量)。接着我们去看下 stm32_pinctrl 结构体是如何定义的，如示例代码 25.1.2.9 所示：

示例代码 25.1.2.9 stm32_pinctrl 结构体代码段

```
100 struct stm32_pinctrl {
......
103     struct pinctrl_desc pctl_desc;
......
107     struct stm32_gpio_bank *banks;
......
120 };
```

第 103 行，pinctrl_desc 结构体用来描述 PIN 控制器，PIN 控制器，PIN 控制器用于配置 SOC 的 PIN 复用功能和电气特性。

第 107 行，这个 stm32_gpio_bank 结构体，是用来注册 GPIO 驱动。到后面 GPIO 子系统再说。

pinctrl_desc 结构体内容如下所示：

示例代码 25.1.2.10 pinctrl_desc 结构体

```
130 struct pinctrl_desc {
131     const char *name;
132     const struct pinctrl_pin_desc *pins;
133     unsigned int npins;
134     const struct pinctrl_ops *pctlops;
135     const struct pinmux_ops *pmxops;
136     const struct pinconf_ops *confops;
137     struct module *owner;
138 #ifdef CONFIG_GENERIC_PINCONF
139     unsigned int num_custom_params;
140     const struct pinconf_generic_params *custom_params;
141     const struct pin_config_item *custom_conf_items;
142 #endif
143     bool link_consumers;
144 };
```

第 134~136 行，这三个“_ops”结构体指针非常重要！因为这三个结构体就是 PIN 控制器的“工具”，这三个结构体里面包含了很多操作函数，Linux 内核初始化 PIN 最终使用的就是这些操作函数。因此编写一个 SOC 的 PIN 控制器驱动的核心就是实现 pinctrl_desc 里面的 pctlops、pmxops 和 confops，pinctrl_desc 结构体需要由用户提供，结构体里面的成员变量也是用户编写的。但是这个用户并不是我们这些使用芯片的程序员，而是半导体厂商，半导体厂商发布的 Linux 内核源码中已经把这些工作做完了。

示例代码 25.1.2.8 里，第 1536~1538 行，给这三个结构体赋值分别对应 stm32_pconf_ops、stm32_prtrl_ops 和 stm32_pmx_ops，三个结构体如下：

示例代码 25.1.2.11 stm32_pconf_ops、stm32_prtrl_ops 和 stm32_pmx_ops 结构体

```
714  static const struct pinctrl_ops stm32_pctrl_ops = {
715      .dt_node_to_map      = stm32_pctrl_dt_node_to_map,
716      .dt_free_map         = pinctrl_utils_free_map,
717      .get_groups_count    = stm32_pctrl_get_groups_count,
718      .get_group_name      = stm32_pctrl_get_group_name,
719      .get_group_pins      = stm32_pctrl_get_group_pins,
720  };
......
865  static const struct pinmux_ops stm32_pmx_ops = {
866      .get_functions_count = stm32_pmx_get_funcs_cnt,
867      .get_function_name   = stm32_pmx_get_func_name,
868      .get_function_groups = stm32_pmx_get_func_groups,
869      .set_mux             = stm32_pmx_set_mux,
870      .gpio_set_direction  = stm32_pmx_gpio_set_direction,
871      .strict              = true,
872  };
......
1238 static const struct pinconf_ops stm32_pconf_ops = {
1239     .pin_config_group_get    = stm32_pconf_group_get,
1240     .pin_config_group_set    = stm32_pconf_group_set,
1241     .pin_config_dbg_show     = stm32_pconf_dbg_show,
1242     .pin_config_set          = stm32_pconf_set,
1243 };
```

pinctrl_desc 结构体初始化完成以后，需要调用 pinctrl_register 或者 devm_pinctrl_register 函数就能够向 Linux 内核注册一个 PIN 控制器，示例代码 25.1.2.8 中的 1542 行就是向 Linux 内核注册 PIN 控制器。

总结一下，pinctrl 驱动流程如下：

1、定义 pinctrl_desc 结构体。

2、初始化结构体，重点是 pinconf_ops、pinmux_ops 和 pinctrl_ops 这三个结构体成员变量，但是这部分半导体厂商帮我们搞定。

3、调用 devm_pinctrl_register 函数完成 PIN 控制器注册。

25.1.3 设备树中添加 pinctrl 节点模板

我们已经对 pinctrl 有了比较深入的了解，接下来我们学习一下如何在设备树中添加某个外

设的 PIN 信息。比如我们需要将 PG11 这个 PIN 复用为 UART4_TX 引脚，pinctrl 节点添加过程如下：

1、创建对应的节点

在 pinctrl 节点下添加一个“uart4_pins”节点：

示例代码 25.1.3.1 uart4_pins 设备节点

```
&pinctrl {
    uart4_pins: uart4-0 {
    /* 具体的 PIN 信息 */
    };
};
```

2、添加“pins”属性

添加一个“pins”子节点，这个子节点是真正用来描述 PIN 配置信，要注意，同一个 pins 子节点下的所有 PIN 电气属性要一样。如果某个外设所用的 PIN 可能有不同的配置，那么就需要多个 pins 子节点，例如 UART4 的 TX 和 RX 引脚配置不同，因此就有 pins1 和 pins2 两个子节点。这里我们只添加 UART4 的 TX 引脚，所以添加完 pins1 子节点以后如下所示：

示例代码 25.1.3.2 uart4_pins 设备节点下的 pins1 属性

```
&pinctrl {
    uart4_pins: uart4-0 {
        pins1{
            /* UART4 TX 引脚的 PIN 配置信息 */
        };
    };
};
```

3、在“pins”节点中添加 PIN 配置信息

最后在“pins”节点中添加具体的 PIN 配置信息，完成以后如下所示：

示例代码 25.1.3.3 完整的 uart4_pins 设备 pinctrl 子节点

```
&pinctrl {
    uart4_pins: uart4-0 {
        pins1{
            pinmux = <STM32_PINMUX('G', 11, AF6)>; /* UART4_TX */
            bias-disable;
            drive-push-pull;
        };
    };
};
```

按道理来讲，当我们将一个 IO 用作 GPIO 功能的时候也需要创建对应的 pinctrl 节点，并且将所用的 IO 复用为 GPIO 功能，比如将 PI0 复用为 GPIO 的时候就需要设置 pinmux 属性为：<STM32_PINMUX('I', 0, GPIO)>，但是！对于 STM32MP1 而言，如果一个 IO 用作 GPIO 功能的时候不需要创建对应的 pinctrl 节点！

25.2 gpio 子系统

25.2.1 gpio 子系统简介

上一小节讲解了 pinctrl 子系统，pinctrl 子系统重点是设置 PIN(有的 SOC 叫做 PAD)的复用和电气属性，如果 pinctrl 子系统将一个 PIN 复用为 GPIO 的话，那么接下来就要用到 gpio 子系统了。gpio 子系统顾名思义，就是用于初始化 GPIO 并且提供相应的 API 函数，比如设置 GPIO 为输入输出，读取 GPIO 的值等。gpio 子系统的主要目的就是方便驱动开发者使用 gpio，驱动开发者在设备树中添加 gpio 相关信息，然后就可以在驱动程序中使用 gpio 子系统提供的 API 函数来操作 GPIO，Linux 内核向驱动开发者屏蔽掉了 GPIO 的设置过程，极大的方便了驱动开发者使用 GPIO。

25.2.2 STM32MP1 的 gpio 子系统驱动

1、设备树中的 gpio 信息

首先肯定是 GPIO 控制器的节点信息，以 PI0 这个引脚所在的 GPIOI 为例，打开 stm32mp151.dtsi，在里面找到如下所示内容：

示例代码 25.2.2.1 gpioi 控制器节点

```
1814    pinctrl: pin-controller@50002000 {
1815        #address-cells = <1>;
1816        #size-cells = <1>;
1817        compatible = "st,stm32mp157-pinctrl";
......
1912        gpioi: gpio@5000a000 {
1913            gpio-controller;
1914            #gpio-cells = <2>;
1915            interrupt-controller;
1916            #interrupt-cells = <2>;
1917            reg = <0x8000 0x400>;
1918            clocks = <&rcc GPIOI>;
1919            st,bank-name = "GPIOI";
1920            status = "disabled";
1921        };
1944    };
```

第 1912~1921 行就是 GPIOI 的控制器信息，属于 pincrtl 的子节点，因此对于 STM32MP1 而言，pinctrl 和 gpio 这两个子系统的驱动文件是一样的，都为 pinctrl-stm32mp157.c，所以在注册 pinctrl 驱动顺便会把 gpio 驱动程序一起注册。绑定文档 Documentation/devicetree/bindings/gpio/gpio.txt 详细描述了 gpio 控制器节点各个属性信息。

第 1913 行，“gpio-controller”表示 gpioi 节点是个 GPIO 控制器，每个 GPIO 控制器节点必须包含“gpio-controller”属性。

第 1914 行，“#gpio-cells”属性和“#address-cells”类似，#gpio-cells 应该为 2，表示一共有两个 cell，第一个 cell 为 GPIO 编号，比如“&gpioi 0”就表示 PI0。第二个 cell 表示 GPIO 极性，如果为 0(GPIO_ACTIVE_HIGH)的话表示高电平有效，如果为 1(GPIO_ACTIVE_LOW)的话表示低电平有效。

第 1917 行，reg 属性设置了 GPIOI 控制器的寄存器基地址偏移为 0X800，因此 GPIOI 寄存器地址为 0X50002000+0X800=0X5000A000，大家可以打开《STM32MP157 参考手册》，找到“Table 9. Register boundary addresses”章节的 2.5.2 Memory map and register boundary addresses 小节，如图 25.2.2.1 所示：

0x5000A400 - 0x5000AFFF	3 KB	Reserved	-
0x5000A000 - 0x5000A3FF	1 KB	GPIOI	*GPIO registers*
0x50009400 - 0x50009FFF	3 KB	Reserved	-
0x50009000 - 0x500093FF	1 KB	GPIOH	*GPIO registers*
0x50008400 - 0x50008FFF	3 KB	Reserved	-
0x50008000 - 0x500083FF	1 KB	GPIOG	*GPIO registers*

图 25.2.2.1 GPIOI 寄存器表

从图 25.2.2.1 可以看出，GPIOI 控制器的基地址就是 0X5000A000，这个地址是基于 pinctrl 的地址 0X50002000+0x8000 = 0X5000A000。

第 1918 行，clocks 属性指定这个 GPIOI 控制器的时钟。

示例代码 25.2.2.1 中的是 GPIOI 控制器节点，当某个具体的引脚作为 GPIO 使用的时候还需要进一步设置。ST 官方 EVK 开发板将 PG1 用作 SD 卡的检测(CD)引脚，PG1 复用为 GPIO 功能，通过读取这个 GPIO 的高低电平就可以知道 SD 卡有没有插入。但是，SD 卡驱动程序怎么知道 CD 引脚连接的 PG1 呢？这里肯定需要设备树来告诉驱动，在设备树中的 SD 卡节点下添加一个属性来描述 SD 卡的 CD 引脚就行了，SD 卡驱动直接读取这个属性值就知道 SD 卡的 CD 引脚使用的是哪个 GPIO 了。ST 官方 EVK 开饭的 SD 卡连接在 STM32MP157 的 sdmmc1 接口上，在 stm32mp15xx-edx.dtsi 中找到名为“sdmmc1”的节点，这个节点就是 SD 卡设备节点，如下所示：

示例代码 25.2.2.2 设备树中 SD 卡节点

```
333 &sdmmc1 {
334     pinctrl-names = "default", "opendrain", "sleep";
335     pinctrl-0 = <&sdmmc1_b4_pins_a &sdmmc1_dir_pins_a>;
336     pinctrl-1 = <&sdmmc1_b4_od_pins_a &sdmmc1_dir_pins_a>;
337     pinctrl-2 = <&sdmmc1_b4_sleep_pins_a &sdmmc1_dir_sleep_pins_a>;
338     cd-gpios = <&gpiog 1 (GPIO_ACTIVE_LOW | GPIO_PULL_UP)>;
339     disable-wp;
......
351     status = "okay";
352 };
```

第 338 行，属性“cd-gpios”描述了 SD 卡的 CD 引脚使用的哪个 IO。属性值一共有三个，我们来看一下这三个属性值的含义，“&gpiog”表示 CD 引脚所使用的 IO 属于 GPIOG 组，“1”表示 GPIOG 组的第 1 号 IO，通过这两个值 SD 卡驱动程序就知道 CD 引脚使用了 PG1 这 GPIO。最后一个是“GPIO_ACTIVE_LOW | GPIO_PULL_UP”，Linux 内核定义在 include/linux/gpio/machine.h 文件中定义了枚举类型 gpio_lookup_flags，内容如下：

示例代码 25.2.2.3 gpio_lookup_flag 枚举类型

```
8  enum gpio_lookup_flags {
9       GPIO_ACTIVE_HIGH     = (0 << 0),
10      GPIO_ACTIVE_LOW      = (1 << 0),
```

```
11      GPIO_OPEN_DRAIN       = (1 << 1),
12      GPIO_OPEN_SOURCE      = (1 << 2),
13      GPIO_PERSISTENT       = (0 << 3),
14      GPIO_TRANSITORY       = (1 << 3),
15      GPIO_PULL_UP          = (1 << 4),
16      GPIO_PULL_DOWN        = (1 << 5),
17      GPIO_LOOKUP_FLAGS_DEFAULT   = GPIO_ACTIVE_HIGH | GPIO_PERSISTENT,
18 };
```

我们可以通过或运算组合不同的配置内容，示例代码 25.2.2.2 中的 338 行，“GPIO_ACTIVE_LOW”表示低电平有效，“GPIO_PULL_UP”表示上拉，所以 PG1 引脚默认上拉，而且低电平有效(当 PG1 被拉低的时候表示 SD 卡插入)。

这里也可以看出，把 PG1 用作 GPIO 的时候不需要添加其对应的 pincrl 节点！

2、GPIO 驱动程序简介

在 25.1.2 小节里已经分析过 pinctrl 驱动代码了，前面一小节说过了 STM32MP1 的 pinctrl 驱动和 gpio 驱动是同一个驱动文件，都为 pinctrl-stm32mp157.c，所以他们的入口函数都是 stm32_pctl_probe，找到如下代码所示：

示例代码 25.2.2.3 stm32_pctl_probe 代码段

```
1452    int stm32_pctl_probe(struct platform_device *pdev)
1453    {
......
1585    for_each_available_child_of_node(np, child) {
1586        if (of_property_read_bool(child, "gpio-controller")) {
1587            ret = stm32_gpiolib_register_bank(pctl, child);
1588            if (ret) {
1589                of_node_put(child);
1590                return ret;
1591            }
1592        }
1593    }
......
1600}
```

第 1586 行，判断设备树节点，是否有 gpio-controller。如果存在，那么这个节点就是一个 GPIO 控制器节点。

第 1587 行，stm32_gpiolib_register_bank 函数用来注册 GPIO 驱动，包括生成回调函数，注册的过程是跟 pinctrl 驱动注册是一样的。都是创建自己的结构体，然后初始化结构体，调用内核的注册函数，这样把自己的结构体注册到内核。这边就不去分析了，大家可以试下分析 GPIO 驱动代码。

25.2.3 gpio 子系统 API 函数

对于驱动开发人员，设置好设备树以后就可以使用 gpio 子系统提供的 API 函数来操作指定的 GPIO，gpio 子系统向驱动开发人员屏蔽了具体的读写寄存器过程。这就是驱动分层与分离

的好处，大家各司其职，做好自己的本职工作即可。gpio 子系统提供的常用的 API 函数有下面几个：

1、gpio_request 函数

gpio_request 函数用于申请一个 GPIO 管脚，在使用一个 GPIO 之前一定要使用 gpio_request 进行申请，函数原型如下：

```
int gpio_request(unsigned gpio,    const char *label)
```

函数参数和返回值含义如下：

gpio：要申请的 gpio 标号，使用 of_get_named_gpio 函数从设备树获取指定 GPIO 属性信息，此函数会返回这个 GPIO 的标号。

label：给 gpio 设置个名字。

返回值：0，申请成功；其他值，申请失败。

2、gpio_free 函数

如果不使用某个 GPIO 了，那么就可以调用 gpio_free 函数进行释放。函数原型如下：

```
void gpio_free(unsigned gpio)
```

函数参数和返回值含义如下：

gpio：要释放的 gpio 标号。

返回值：无。

3、gpio_direction_input 函数

此函数用于设置某个 GPIO 为输入，函数原型如下所示：

```
int gpio_direction_input(unsigned gpio)
```

函数参数和返回值含义如下：

gpio：要设置为输入的 GPIO 标号。

返回值：0，设置成功；负值，设置失败。

4、gpio_direction_output 函数

此函数用于设置某个 GPIO 为输出，并且设置默认输出值，函数原型如下：

```
int gpio_direction_output(unsigned gpio, int value)
```

函数参数和返回值含义如下：

gpio：要设置为输出的 GPIO 标号。

value：GPIO 默认输出值。

返回值：0，设置成功；负值，设置失败。

5、gpio_get_value 函数

此函数用于获取某个 GPIO 的值(0 或 1)，此函数是个宏，定义所示：

```
#define gpio_get_value    __gpio_get_value
int __gpio_get_value(unsigned gpio)
```

函数参数和返回值含义如下：

gpio：要获取的 GPIO 标号。

返回值：非负值，得到的 GPIO 值；负值，获取失败。

6、gpio_set_value 函数

此函数用于设置某个 GPIO 的值，此函数是个宏，定义如下

```
#define gpio_set_value    __gpio_set_value
```

```
void __gpio_set_value(unsigned gpio, int value)
```

函数参数和返回值含义如下：

gpio：要设置的 GPIO 标号。

value：要设置的值。

返回值：无

关于 gpio 子系统常用的 API 函数就讲这些，这些是我们用的最多的。

25.2.4 设备树中添加 gpio 节点模板

本节我们以正点原子 STM32MP157 开发板上的 LED0 为例，学习一下如何创建 GPIO 节点。LED0 连接到了 PI0 引脚上，首先创建一个“led”设备节点。

1、创建 led 设备节点

在根节点“/”下创建 test 设备子节点，如下所示：

示例代码 25.2.4.1 led 设备节点

```
1 led {
2   /* 节点内容 */
3 };
```

2、添加 GPIO 属性信息

在 led 节点中添加 GPIO 属性信息，表明 test 所使用的 GPIO 是哪个引脚，添加完成以后如下所示：

示例代码 25.2.4.2 向 led 节点添加 gpio 属性

```
1 led {
2   compatible = "atk,led";
3   gpio = <&gpioi 0 GPIO_ACTIVE_LOW>;
4   status = "okay";
5 };
```

第 3 行，led 设备所使用的 gpio。

关于 pinctrl 子系统和 gpio 子系统就讲解到这里，接下来就使用 pinctrl 和 gpio 子系统来驱动 STM32MP1 开发板上的 LED 灯。

25.2.5 与 gpio 相关的 OF 函数

在示例代码 25.2.4.2 中，我们定义了一个名为“gpio”的属性，gpio 属性描述了 led 这个设备所使用的 GPIO。在驱动程序中需要读取 gpio 属性内容，Linux 内核提供了几个与 GPIO 有关的 OF 函数，常用的几个 OF 函数如下所示：

1、of_gpio_named_count 函数

of_gpio_named_count 函数用于获取设备树某个属性里面定义了几个 GPIO 信息，要注意的是空的 GPIO 信息也会被统计到，比如：

```
gpios = <0
        &gpio1 1 2
        0
        &gpio2 3 4>;
```

上述代码的“gpios”节点一共定义了 4 个 GPIO，但是有 2 个是空的，没有实际的含义。通过 of_gpio_named_count 函数统计出来的 GPIO 数量就是 4 个，此函数原型如下：

```
int of_gpio_named_count(struct device_node *np, const char    *propname)
```

函数参数和返回值含义如下：

np：设备节点。

propname：要统计的 GPIO 属性。

返回值：正值，统计到的 GPIO 数量；负值，失败。

2、of_gpio_count 函数

和 of_gpio_named_count 函数一样，但是不同的地方在于，此函数统计的是“gpios”这个属性的 GPIO 数量，而 of_gpio_named_count 函数可以统计任意属性的 GPIO 信息，函数原型如下所示：

```
int of_gpio_count(struct device_node *np)
```

函数参数和返回值含义如下：

np：设备节点。

返回值：正值，统计到的 GPIO 数量；负值，失败。

3、of_get_named_gpio 函数

此函数获取 GPIO 编号，因为 Linux 内核中关于 GPIO 的 API 函数都要使用 GPIO 编号，此函数会将设备树中类似<&gpioi 0(GPIO_ACTIVE_LOW | GPIO_PULL_UP)>的属性信息转换为对应的 GPIO 编号，此函数在驱动中使用很频繁！函数原型如下：

```
int of_get_named_gpio(struct device_node    *np,
                      const char            *propname,
                      int                   index)
```

函数参数和返回值含义如下：

np：设备节点。

propname：包含要获取 GPIO 信息的属性名。

index：GPIO 索引，因为一个属性里面可能包含多个 GPIO，此参数指定要获取哪个 GPIO 的编号，如果只有一个 GPIO 信息的话此参数为 0。

返回值：正值，获取到的 GPIO 编号；负值，失败。

25.3 硬件原理图分析

本实验的硬件原理参考 21.2 小节即可。

25.4 实验程序编写

本实验对应的例程路径为：开发板光盘→ 1、程序源码→2、Linux 驱动例程→5_gpioled。

本章实验我们继续研究 LED 灯，在第二十四章实验中我们通过设备树向 dtsled.c 文件传递相应的寄存器物理地址，然后在驱动文件中配置寄存器。本章实验我们使用 gpio 子系统来完成 LED 灯驱动。

25.4.1 修改设备树文件

在 stm32mp157d-atk.dts 文件的根节点“/”下创建 LED 灯节点，节点名为“gpioled”，节点内容如下：

示例代码 25.4.1.1 创建 LED 灯节点

```
1  gpioled {
2      compatible = "alientek,led";
3      status = "okay";
4      led-gpio = <&gpioi 0 GPIO_ACTIVE_LOW>;
5  };
```

第 4 行，led-gpio 属性指定了 LED 灯所使用的 GPIO，在这里就是 GPIOI 的 0 号，低电平有效。稍后编写驱动程序的时候会获取 led-gpio 属性的内容来得到 GPIO 编号，因为 gpio 子系统的 API 操作函数需要 GPIO 编号。设备树编写完成以后使用“make dtbs”命令重新编译设备树，然后使用新编译出来的 stm32mp157d-atk.dtb 文件启动 Linux 系统。启动成功以后进入“/proc/device-tree”目录中查看“gpioled”节点是否存在，如果存在的话就说明设备树基本修改成功(具体还要驱动验证)，结果如图 25.4.1.1 所示：

```
[root@ATK-stm32mp1]:~$ ls /proc/device-tree/
#address-cells                  mlahb
#size-cells                     model
aliases                         name
arm-pmu                         pm_domain
buck1                           psci
chosen                          reboot
compatible                      regulator-3p3v
cpu0-opp-table                  regulator-booster
cpus                            reserved-memory
firmware                        serial-number
gpioled                         soc
interrupt-controller@a0021000   sram@2ffff000
mailbox-0                       stm32mp1_led
mailbox-1                       thermal-zones
memory@c0000000                 timer
```

gpioled子节点

图 25.4.1.1 gpioled 子节点

25.4.2 LED 灯驱动程序编写

设备树准备好以后就可以编写驱动程序了，本章实验在第四十四章实验驱动文件 dtsled.c 的基础上修改而来。新建名为“5_gpioled”文件夹，然后在 5_gpioled 文件夹里面创建 vscode 工程，工作区命名为“gpioled”。工程创建好以后新建 gpioled.c 文件，在 gpioled.c 里面输入如下内容：

示例代码 25.4.1.2 gpioled.c 驱动文件代码

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <linux/of.h>
```

```
12  #include <linux/of_address.h>
13  #include <linux/of_gpio.h>
14  #include <asm/mach/map.h>
15  #include <asm/uaccess.h>
16  #include <asm/io.h>
17  /***************************************************************
18  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
19  文件名      : gpioled.c
20  作者        : 正点原子 Linux 团队
21  版本        : V1.0
22  描述        : gpio 子系统驱动 LED 灯。
23  其他        : 无
24  论坛        : www.openedv.com
25  日志        : 初版 V1.0 2020/12/30 正点原子 Linux 团队创建
26  ***************************************************************/
27  #define GPIOLED_CNT      1              /* 设备号个数   */
28  #define GPIOLED_NAME     "gpioled"      /* 名字         */
29  #define LEDOFF           0              /* 关灯         */
30  #define LEDON            1              /* 开灯         */
31
32  /* gpioled 设备结构体 */
33  struct gpioled_dev{
34      dev_t devid;                /* 设备号   */
35      struct cdev cdev;           /* cdev     */
36      struct class *class;        /* 类       */
37      struct device *device;      /* 设备     */
38      int major;                  /* 主设备号 */
39      int minor;                  /* 次设备号 */
40      struct device_node  *nd;    /* 设备节点 */
41      int led_gpio;               /* led 所使用的 GPIO 编号    */
42  };
43
44  struct gpioled_dev gpioled;     /* led 设备 */
45
46  /*
47   * @description   : 打开设备
48   * @param - inode : 传递给驱动的 inode
49   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
50   *                  一般在 open 的时候将 private_data 指向设备结构体。
51   * @return        : 0 成功;其他 失败
52   */
53  static int led_open(struct inode *inode, struct file *filp)
54  {
```

```
55      filp->private_data = &gpioled; /* 设置私有数据 */
56      return 0;
57  }
58
59  /*
60   * @description   : 从设备读取数据
61   * @param - filp  : 要打开的设备文件(文件描述符)
62   * @param - buf   : 返回给用户空间的数据缓冲区
63   * @param - cnt   : 要读取的数据长度
64   * @param - offt  : 相对于文件首地址的偏移
65   * @return        : 读取的字节数，如果为负值，表示读取失败
66   */
67  static ssize_t led_read(struct file *filp, char __user *buf,
                            size_t cnt, loff_t *offt)
68  {
69      return 0;
70  }
71
72  /*
73   * @description   : 向设备写数据
74   * @param - filp  : 设备文件，表示打开的文件描述符
75   * @param - buf   : 要写给设备写入的数据
76   * @param - cnt   : 要写入的数据长度
77   * @param - offt  : 相对于文件首地址的偏移
78   * @return        : 写入的字节数，如果为负值，表示写入失败
79   */
80  static ssize_t led_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
81  {
82      int retvalue;
83      unsigned char databuf[1];
84      unsigned char ledstat;
85      struct gpioled_dev *dev = filp->private_data;
86
87      retvalue = copy_from_user(databuf, buf, cnt);
88      if(retvalue < 0) {
89          printk("kernel write failed!\r\n");
90          return -EFAULT;
91      }
92
93      ledstat = databuf[0];       /* 获取状态值 */
94
95      if(ledstat == LEDON) {
```

```
96          gpio_set_value(dev->led_gpio, 0);   /* 打开 LED 灯 */
97      } else if(ledstat == LEDOFF) {
98          gpio_set_value(dev->led_gpio, 1);   /* 关闭 LED 灯 */
99      }
100     return 0;
101 }
102
103 /*
104  * @description   : 关闭/释放设备
105  * @param - filp  : 要关闭的设备文件(文件描述符)
106  * @return        : 0 成功;其他 失败
107  */
108 static int led_release(struct inode *inode, struct file *filp)
109 {
110     return 0;
111 }
112
113 /* 设备操作函数 */
114 static struct file_operations gpioled_fops = {
115     .owner = THIS_MODULE,
116     .open = led_open,
117     .read = led_read,
118     .write = led_write,
119     .release =  led_release,
120 };
121
122 /*
123  * @description   : 驱动出口函数
124  * @param         : 无
125  * @return        : 无
126  */
127 static int __init led_init(void)
128 {
129     int ret = 0;
130     const char *str;
131
132     /* 设置 LED 所使用的 GPIO        */
133     /* 1、获取设备节点: gpioled    */
134     gpioled.nd = of_find_node_by_path("/gpioled");
135     if(gpioled.nd == NULL) {
136         printk("gpioled node not find!\r\n");
137         return -EINVAL;
138     }
```

```
139
140     /* 2.读取 status 属性 */
141     ret = of_property_read_string(gpioled.nd, "status", &str);
142     if(ret < 0)
143         return -EINVAL;
144
145     if (strcmp(str, "okay"))
146         return -EINVAL;
147
148     /* 3、获取 compatible 属性值并进行匹配 */
149     ret = of_property_read_string(gpioled.nd, "compatible", &str);
150     if(ret < 0) {
151         printk("gpioled: Failed to get compatible property\n");
152         return -EINVAL;
153     }
154
155     if (strcmp(str, "alientek,led")) {
156         printk("gpioled: Compatible match failed\n");
157         return -EINVAL;
158     }
159
160     /* 4、 获取设备树中的 gpio 属性，得到 LED 所使用的 LED 编号 */
161     gpioled.led_gpio = of_get_named_gpio(gpioled.nd, "led-gpio", 0);
162     if(gpioled.led_gpio < 0) {
163         printk("can't get led-gpio");
164         return -EINVAL;
165     }
166     printk("led-gpio num = %d\r\n", gpioled.led_gpio);
167
168     /* 5.向 gpio 子系统申请使用 GPIO */
169     ret = gpio_request(gpioled.led_gpio, "LED-GPIO");
170     if (ret) {
171         printk(KERN_ERR "gpioled: Failed to request led-gpio\n");
172         return ret;
173     }
174
175     /* 6、设置 PI0 为输出，并且输出高电平，默认关闭 LED 灯 */
176     ret = gpio_direction_output(gpioled.led_gpio, 1);
177     if(ret < 0) {
178         printk("can't set gpio!\r\n");
179     }
180
181     /* 注册字符设备驱动 */
```

```
182     /* 1、创建设备号 */
183     if (gpioled.major) {        /*  定义了设备号 */
184         gpioled.devid = MKDEV(gpioled.major, 0);
185         ret = register_chrdev_region(gpioled.devid, GPIOLED_CNT,
                                         GPIOLED_NAME);
186         if(ret < 0) {
187             pr_err("cannot register %s char driver [ret=%d]\n",
                                  GPIOLED_NAME, GPIOLED_CNT);
188             goto free_gpio;
189         }
190     } else {                        /* 没有定义设备号 */
191         ret = alloc_chrdev_region(&gpioled.devid, 0, GPIOLED_CNT,
                                  GPIOLED_NAME);    /* 申请设备号 */
192         if(ret < 0) {
193             pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                         GPIOLED_NAME, ret);
194             goto free_gpio;
195         }
196         gpioled.major = MAJOR(gpioled.devid); /* 获取分配好的主设备号 */
197         gpioled.minor = MINOR(gpioled.devid); /* 获取分配好的次设备号 */
198     }
199     printk("gpioled major=%d,minor=%d\r\n",gpioled.major,
                    gpioled.minor);
200
201     /* 2、初始化 cdev */
202     gpioled.cdev.owner = THIS_MODULE;
203     cdev_init(&gpioled.cdev, &gpioled_fops);
204
205     /* 3、添加一个 cdev */
206     cdev_add(&gpioled.cdev, gpioled.devid, GPIOLED_CNT);
207     if(ret < 0)
208         goto del_unregister;
209
210     /* 4、创建类 */
211     gpioled.class = class_create(THIS_MODULE, GPIOLED_NAME);
212     if (IS_ERR(gpioled.class)) {
213         goto del_cdev;
214     }
215
216     /* 5、创建设备 */
217     gpioled.device = device_create(gpioled.class, NULL,
                    gpioled.devid, NULL, GPIOLED_NAME);
218     if (IS_ERR(gpioled.device)) {
```

```
219         goto destroy_class;
220     }
221     return 0;
222 
223 destroy_class
224     class_destroy(gpioled.class);
225 del_cdev:
226     cdev_del(&gpioled.cdev);
227 del_unregister:
228     unregister_chrdev_region(gpioled.devid, GPIOLED_CNT);
229 free_gpio:
230     gpio_free(gpioled.led_gpio);
231     return -EIO;
232 }
233 
234 /*
235  * @description   : 驱动出口函数
236  * @param         : 无
237  * @return        : 无
238  */
239 static void __exit led_exit(void)
240 {
241     /* 注销字符设备驱动 */
242     cdev_del(&gpioled.cdev);         /*  删除 cdev   */
243     unregister_chrdev_region(gpioled.devid, GPIOLED_CNT);
244     device_destroy(gpioled.class, gpioled.devid);
245     class_destroy(gpioled.class);   /* 注销类       */
246     gpio_free(gpioled.led_gpio);    /* 释放 GPIO    */
247 }
248 
249 module_init(led_init);
250 module_exit(led_exit);
251 MODULE_LICENSE("GPL");
252 MODULE_AUTHOR("ALIENTEK");
253 MODULE_INFO(intree, "Y");
```

第 41 行，在设备结构体 gpioled_dev 中加入 led_gpio 这个成员变量，此成员变量保存 LED 等所使用的 GPIO 编号。

第 55 行，将设备结构体变量 gpioled 设置为 filp 的私有数据 private_data。

第 85 行，通过读取 filp 的 private_data 成员变量来得到设备结构体变量，也就是 gpioled。这种将设备结构体设置为 filp 私有数据的方法在 Linux 内核驱动里面非常常见。

第 96、98 行，直接调用 gpio_set_value 函数来向 GPIO 写入数据，实现开/关 LED 的效果。不需要我们直接操作相应的寄存器。

第 134 行，获取节点“/gpioled”。

第 141~146 行，获取“status”属性的值，判断属性是否“okay”。

第 149~153 行，获取 compatible 属性值并进行匹配。

第 161 行，通过函数 of_get_named_gpio 函数获取 LED 所使用的 LED 编号。相当于将 gpioled 节点中的“led-gpio”属性值转换为对应的 LED 编号。

第 169 行，通过函数 gpio_request 向 GPIO 子系统申请使用 PI0。

第 176 行，调用函数 gpio_direction_output 设置 PI0 这个 GPIO 为输出，并且默认高电平，这样默认就会关闭 LED 灯。

可以看出 gpioled.c 文件中的内容和第二十四章的 dtsled.c 差不多，只是取消掉了配置寄存器的过程，改为使用 Linux 内核提供的 API 函数。在 GPIO 操作上更加的规范化，符合 Linux 代码框架，而且也简化了 GPIO 驱动开发的难度，以后我们所有例程用到 GPIO 的地方都采用此方法。

25.4.3 编写测试 APP

本章直接使用第四十二章的测试 APP，将上一章的 ledApp.c 文件复制到本章实验工程下即可。

25.5 运行测试

25.5.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第二十章实验基本一样，只是将 obj-m 变量的值改为 gpioled.o，Makefile 内容如下所示：

示例代码 25.5.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := gpioled.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 gpioled.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“gpioled.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 ledApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc ledApp.c -o ledApp
```

编译成功以后就会生成 ledApp 这个应用程序。

25.5.2 运行测试

将上一小节编译出来的 gpioled.ko 和 ledApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 gpioled.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe gpioled        //加载驱动
```

驱动加载成功以后会在终端中输出一些信息，如图 25.5.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe gpioled.ko
led-gpio num = 128
gpioled major=241,minor=0
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 25.5.2.1 驱动加载成功以后输出的信息

从图 25.5.2.1 可以看出，gpioled 这个节点找到了，并且 PI0 这个 GPIO 的编号为 128。驱动加载成功以后就可以使用 ledApp 软件来测试驱动是否工作正常，输入如下命令打开 LED 灯：

```
./ledApp /dev/gpioled 1         //打开 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯是否点亮，如果点亮的话说明驱动工作正常。在输入如下命令关闭 LED 灯：

```
./ledApp    /dev/gpioled 0      //关闭 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯是否熄灭。如果要卸载驱动的话输入如下命令即可：

```
rmmod gpioled.ko
```

第二十六章 Linux 蜂鸣器实验

上一章实验中我们借助 gpio 子系统编写了 LED 灯驱动，STM32MP1 开发板上还有一个蜂鸣器，从软件的角度考虑，蜂鸣器驱动和 LED 灯驱动其实是一摸一样的，都是控制 IO 输出高低电平。本章我们就来学习编写蜂鸣器的 Linux 驱动，也算是对上一章讲解的 gpio 子系统的巩固。

26.1 蜂鸣器驱动原理

蜂鸣器常用于计算机、打印机、报警器、电子玩具等电子产品中，常用的蜂鸣器有两种：有源蜂鸣器和无源蜂鸣器，这里的有“源”不是电源，而是震荡源，有源蜂鸣器内部带有震荡源，所以有源蜂鸣器只要通电就会叫。无源蜂鸣器内部不带震荡源，直接用直流电是驱动不起来的，需要 2K-5K 的方波去驱动。正点原子 STM32MP1 开发板使用的是有源蜂鸣器，因此只要给其供电就会工作，开发板所使用的有源蜂鸣器如下图所示：

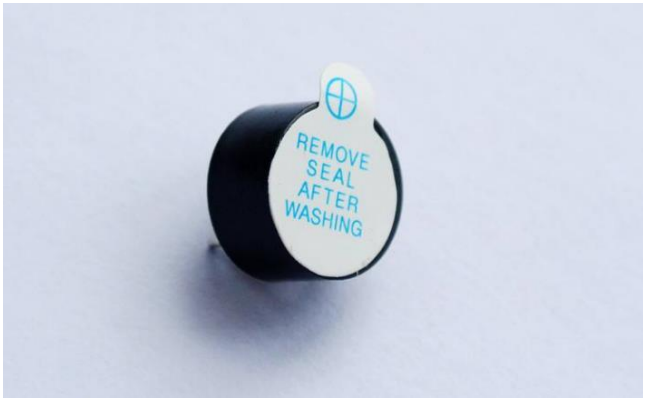


图 26.1.1 有源蜂鸣器

有源蜂鸣器只要通电就会叫，所以我们可以做一个供电电路，这个供电电路通过一个 IO 来控制其通断，一般使用三极管来搭建这个电路。为什么我们不能像控制 LED 灯一样，直接将 GPIO 接到蜂鸣器的负极，通过 IO 输出高低来控制蜂鸣器的通断。这事因为蜂鸣器工作的电流比 LED 灯要大，直接将蜂鸣器接到开发板的 GPIO 上有可能会烧毁 IO，所以我们需要通过一个三极管来间接的控制蜂鸣器的通断，相当于加了一层隔离。本章我们就驱动开发板上的有源蜂鸣器，然后编写简单的测试 APP，通过 APP 来控制蜂鸣器鸣叫或关闭。

本节我们来看一下如果在 Linux 下编写蜂鸣器驱动需要做哪些工作：

①、在设备树中创建蜂鸣器节点，在蜂鸣器节点中加入 GPIO 信息。

②、编写驱动程序和测试 APP，和第二十五章的 LED 驱动程序和测试 APP 基本一样。

接下来我们就根据上面这两步来编写蜂鸣器 Linux 驱动。

26.2 硬件原理图分析

蜂鸣器的硬件原理图如图 26.2.1 所示：

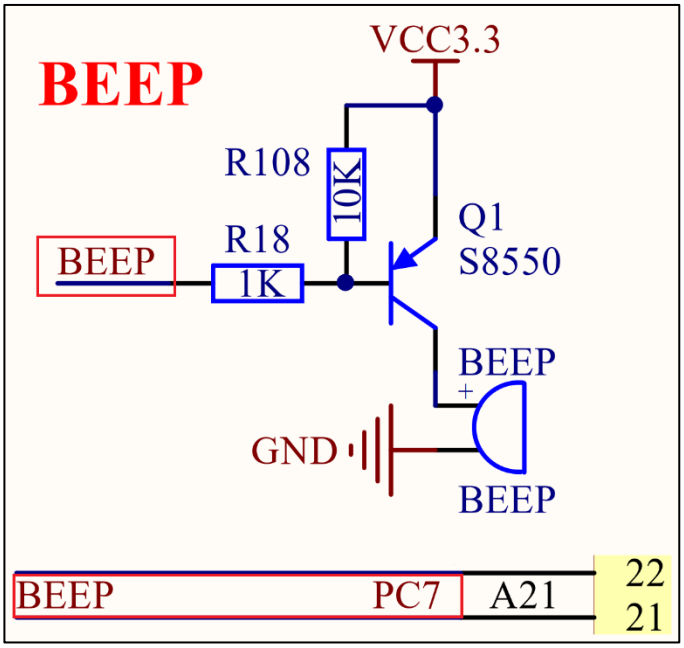


图 26.2.1 蜂鸣器原理图

图 26.2.1 中通过一个 PNP 型的三极管 8550 来驱动蜂鸣器，通过 PC7 这个 IO 来控制三极管 Q1 的导通，当 BEEP 输出低电平的时候 Q1 导通，相当于蜂鸣器的正极连接到 3.3V 电源，蜂鸣器形成一个通路，因此蜂鸣器会鸣叫。同理，当 BEEP 输出高电平的时候 Q1 不导通，那么蜂鸣器就没有形成一个通路，因此蜂鸣器也就不会鸣叫。

26.3 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→6_beep。

本章实验在二十五章实验的基础上完成，重点是将驱动改为基于设备树的.

26.3.1 修改设备树文件

在根节点“/”下创建 BEEP 节点，节点名为“beep”，节点内容如下：

示例代码 26.3.1.1 创建 BEEP 蜂鸣器节点

```
1  beep {
2      compatible = "alientek,beep";
3      status = "okay";
4      beep-gpio = <&gpioc 7 GPIO_ACTIVE_HIGH>;
5  };
```

第 4 行，beep-gpio 属性指定了蜂鸣器所使用的 GPIO。

设备树编写完成以后使用“make dtbs”命令重新编译设备树，然后使用新编译出来的 stm32mp157d-atk.dtb 文件启动 Linux 系统。启动成功以后进入“/proc/device-tree”目录中查看“beep”节点是否存在，如果存在的话就说明设备树基本修改成功(具体还要驱动验证)，结果如图 26.3.1.1 所示：

```
[root@ATK-stm32mp1]:~$ ls /proc/device-tree/
#address-cells                  mlahb
#size-cells                     model
aliases                         name
arm-pmu                         pm_domain
beep                            psci
buck1                           reboot
chosen                          regulator-3p3v
compatible                      regulator-booster
cpu0-opp-table                  reserved-memory
cpus                            serial-number
```

beep节点

图 26.3.1.1 beep 节点

26.3.2 蜂鸣器驱动程序编写

设备树准备好以后就可以编写驱动程序了，本章实验在第四十五章实验驱动文件 gpioled.c 的基础上修改而来。新建名为“6_beep”的文件夹，然后在 6_beep 文件夹里面创建 vscode 工程，工作区命名为“beep”。工程创建好以后新建 beep.c 文件，在 beep.c 里面输入如下内容：

示例代码 26.3.2.1 beep.c 文件代码段

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <linux/of.h>
12 #include <linux/of_address.h>
13 #include <linux/of_gpio.h>
14 #include <asm/mach/map.h>
15 #include <asm/uaccess.h>
16 #include <asm/io.h>
17 /***************************************************************
18 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
19 文件名      : beep.c
20 作者        : 正点原子 Linux 团队
21 版本        : V1.0
22 描述        : 蜂鸣器驱动程序。
23 其他        : 无
24 论坛        : www.openedv.com
25 日志        : 初版 V1.0 2020/12/31 正点原子 Linux 团队创建
26 ***************************************************************/
27 #define BEEP_CNT         1       /* 设备号个数    */
```

```
28  #define BEEP_NAME       "beep" /* 名字         */
29  #define BEEPOFF         0      /* 关蜂鸣器     */
30  #define BEEPON          1      /* 开蜂鸣器     */
31
32  /* beep 设备结构体 */
33  struct beep_dev{
34      dev_t devid;                /* 设备号       */
35      struct cdev cdev;           /* cdev         */
36      struct class *class;        /* 类           */
37      struct device *device;      /* 设备         */
38      int major;                  /* 主设备号     */
39      int minor;                  /* 次设备号     */
40      struct device_node  *nd;    /* 设备节点     */
41      int beep_gpio;              /* beep 所使用的 GPIO 编号 */
42  };
43
44  struct beep_dev beep;   /* beep 设备 */
45
46  /*
47   * @description   : 打开设备
48   * @param - inode : 传递给驱动的 inode
49   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
50   *                  一般在 open 的时候将 private_data 指向设备结构体。
51   * @return        : 0 成功;其他 失败
52   */
53  static int led_open(struct inode *inode, struct file *filp)
54  {
55      filp->private_data = &beep; /* 设置私有数据 */
56      return 0;
57  }
58
59  /*
60   * @description   : 从设备读取数据
61   * @param - filp  : 要打开的设备文件(文件描述符)
62   * @param - buf   : 返回给用户空间的数据缓冲区
63   * @param - cnt   : 要读取的数据长度
64   * @param - offt  : 相对于文件首地址的偏移
65   * @return        : 读取的字节数，如果为负值，表示读取失败
66   */
67  static ssize_t led_read(struct file *filp, char __user *buf,
                            size_t cnt, loff_t *offt)
68  {
69      return 0;
```

```
70  }
71
72  /*
73   * @description   : 向设备写数据
74   * @param - filp  : 设备文件，表示打开的文件描述符
75   * @param - buf   : 要写给设备写入的数据
76   * @param - cnt   : 要写入的数据长度
77   * @param - offt  : 相对于文件首地址的偏移
78   * @return        : 写入的字节数，如果为负值，表示写入失败
79   */
80  static ssize_t led_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
81  {
82      int retvalue;
83      unsigned char databuf[1];
84      unsigned char ledstat;
85      struct beep_dev *dev = filp->private_data;
86
87      retvalue = copy_from_user(databuf, buf, cnt);
88      if(retvalue < 0) {
89          printk("kernel write failed!\r\n");
90          return -EFAULT;
91      }
92
93      ledstat = databuf[0];                   /* 获取状态值 */
94
95      if(ledstat == BEEPON) {
96          gpio_set_value(dev->beep_gpio, 0);  /* 打开蜂鸣器 */
97      } else if(ledstat == BEEPOFF) {
98          gpio_set_value(dev->beep_gpio, 1);  /* 关闭蜂鸣器 */
99      }
100     return 0;
101 }
102
103 /*
104  * @description   : 关闭/释放设备
105  * @param - filp  : 要关闭的设备文件(文件描述符)
106  * @return        : 0 成功;其他 失败
107  */
108 static int led_release(struct inode *inode, struct file *filp)
109 {
110     return 0;
111 }
```

```
112
113 /* 设备操作函数 */
114 static struct file_operations beep_fops = {
115     .owner = THIS_MODULE,
116     .open = led_open,
117     .read = led_read,
118     .write = led_write,
119     .release =  led_release,
120 };
121
122 /*
123  * @description   : 驱动出口函数
124  * @param         : 无
125  * @return        : 无
126  */
127 static int __init led_init(void)
128 {
129     int ret = 0;
130     const char *str;
131
132     /* 设置 LED 所使用的 GPIO    */
133     /* 1、获取设备节点：beep    */
134     beep.nd = of_find_node_by_path("/beep");
135     if(beep.nd == NULL) {
136         printk("beep node not find!\r\n");
137         return -EINVAL;
138     }
139
140     /* 2.读取 status 属性 */
141     ret = of_property_read_string(beep.nd, "status", &str);
142     if(ret < 0)
143         return -EINVAL;
144
145     if (strcmp(str, "okay"))
146         return -EINVAL;
147
148     /* 3、获取 compatible 属性值并进行匹配 */
149     ret = of_property_read_string(beep.nd, "compatible", &str);
150     if(ret < 0) {
151         printk("beep: Failed to get compatible property\n");
152         return -EINVAL;
153     }
154
```

```
155     if (strcmp(str, "alientek,beep")) {
156         printk("beep: Compatible match failed\n");
157         return -EINVAL;
158     }
159 
160     /* 4、 获取设备树中的 gpio 属性，得到 LED 所使用的 LED 编号 */
161     beep.beep_gpio = of_get_named_gpio(beep.nd, "beep-gpio", 0);
162     if(beep.beep_gpio < 0) {
163         printk("can't get led-gpio");
164         return -EINVAL;
165     }
166     printk("beep-gpio num = %d\r\n", beep.beep_gpio);
167 
168     /* 5.向 gpio 子系统申请使用 GPIO */
169     ret = gpio_request(beep.beep_gpio, "BEEP-GPIO");
170     if (ret) {
171         printk(KERN_ERR "beep: Failed to request beep-gpio\n");
172         return ret;
173     }
174 
175     /* 6、设置 PC7 为输出，并且输出高电平，默认关闭 BEEP */
176     ret = gpio_direction_output(beep.beep_gpio, 1);
177     if(ret < 0) {
178         printk("can't set gpio!\r\n");
179     }
180 
181     /* 注册字符设备驱动 */
182     /* 1、创建设备号    */
183     if (beep.major) {       /*  定义了设备号 */
184         beep.devid = MKDEV(beep.major, 0);
185         ret = register_chrdev_region(beep.devid, BEEP_CNT,
                                         BEEP_NAME);
186         if(ret < 0) {
187             pr_err("cannot register %s char driver [ret=%d]\n",
                       BEEP_NAME, BEEP_CNT);
188             goto free_gpio;
189         }
190     } else {                        /* 没有定义设备号 */
191         ret = alloc_chrdev_region(&beep.devid, 0, BEEP_CNT,
                                      BEEP_NAME); /* 申请设备号 */
192         if(ret < 0) {
193             pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                       BEEP_NAME, ret);
```

```
194             goto free_gpio;
195         }
196         beep.major = MAJOR(beep.devid); /* 获取分配号的主设备号 */
197         beep.minor = MINOR(beep.devid); /* 获取分配号的次设备号 */
198     }
199     printk("beep major=%d,minor=%d\r\n",beep.major, beep.minor);
200
201     /* 2、初始化 cdev */
202     beep.cdev.owner = THIS_MODULE;
203     cdev_init(&beep.cdev, &beep_fops);
204
205     /* 3、添加一个 cdev */
206     cdev_add(&beep.cdev, beep.devid, BEEP_CNT);
207     if(ret < 0)
208         goto del_unregister;
209
210     /* 4、创建类 */
211     beep.class = class_create(THIS_MODULE, BEEP_NAME);
212     if (IS_ERR(beep.class)) {
213         goto del_cdev;
214     }
215
216     /* 5、创建设备 */
217     beep.device = device_create(beep.class, NULL, beep.devid, NULL,
                                    BEEP_NAME);
218     if (IS_ERR(beep.device)) {
219         goto destroy_class;
220     }
221     return 0;
222
223 destroy_class:
224     class_destroy(beep.class);
225 del_cdev:
226     cdev_del(&beep.cdev);
227 del_unregister:
228     unregister_chrdev_region(beep.devid, BEEP_CNT);
229 free_gpio:
230     gpio_free(beep.beep_gpio);
231     return -EIO;
232 }
233
234 /*
235  * @description   : 驱动出口函数
```

```
236  * @param         : 无
237  * @return        : 无
238  */
239 static void __exit led_exit(void)
240 {
241     /* 注销字符设备驱动 */
242     cdev_del(&beep.cdev);                       /*  删除 cdev */
243     unregister_chrdev_region(beep.devid, BEEP_CNT); /* 注销设备号 */
244     device_destroy(beep.class, beep.devid);     /* 注销设备 */
245     class_destroy(beep.class);                  /* 注销类 */
246     gpio_free(beep.beep_gpio);                  /* 释放 GPIO */
247 }
248
249 module_init(led_init);
250 module_exit(led_exit);
251 MODULE_LICENSE("GPL");
252 MODULE_AUTHOR("ALIENTEK");
253 MODULE_INFO(intree, "Y");
```

beep.c 中的内容和上一章的 gpioled.c 中的内容基本一样，只是换为了初始化 BEEP 这个 PIN，这里就不详细的讲解了。

26.3.3 编写测试 APP

测试 APP 在上一章实验的 ledApp.c 文件的基础上完成，新建名为 beepApp.c 的文件，然后输入如下所示内容：

示例代码 26.3.3.1 beepApp.c 文件

```
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "fcntl.h"
6  #include "stdlib.h"
7  #include "string.h"
8  /***************************************************************
9  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
10 文件名      : beepApp.c
11 作者        : 正点原子 Linux 团队
12 版本        : V1.0
13 描述        : beep 测试 APP。
14 其他        : 无
15 使用方法    : ./beepApp /dev/beep  0 关闭蜂鸣器
16              ./beepApp /dev/beep  1 打开蜂鸣器
17 论坛        : www.openedv.com
18 日志        : 初版 V1.0 2020/12/31 正点原子 Linux 团队创建
```

```
19  ****************************************************************/
20
21  #define BEEPOFF     0
22  #define BEEPON  1
23
24  /*
25   * @description   : main 主程序
26   * @param - argc  : argv 数组元素个数
27   * @param - argv  : 具体参数
28   * @return        : 0 成功;其他 失败
29   */
30  int main(int argc, char *argv[])
31  {
32      int fd, retvalue;
33      char *filename;
34      unsigned char databuf[1];
35
36      if(argc != 3){
37          printf("Error Usage!\r\n");
38          return -1;
39      }
40
41      filename = argv[1];
42
43      /* 打开 beep 驱动 */
44      fd = open(filename, O_RDWR);
45      if(fd < 0){
46          printf("file %s open failed!\r\n", argv[1]);
47          return -1;
48      }
49
50      databuf[0] = atoi(argv[2]); /* 要执行的操作：打开或关闭 */
51
52      /* 向/dev/beep 文件写入数据 */
53      retvalue = write(fd, databuf, sizeof(databuf));
54      if(retvalue < 0){
55          printf("BEEP Control Failed!\r\n");
56          close(fd);
57          return -1;
58      }
59
60      retvalue = close(fd); /* 关闭文件 */
61      if(retvalue < 0){
```

```
62          printf("file %s close failed!\r\n", argv[1]);
63          return -1;
64      }
65      return 0;
66  }
67
```

beepApp.c 的文件内容和 ledApp.c 文件内容基本一样，要是对文件进行打开、写、关闭等操作。

26.4 运行测试

26.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第二十章实验基本一样，只是将 obj-m 变量的值改为 beep.o，Makefile 内容如下所示：

示例代码 26.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := beep.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 beep.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“beep.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 beepApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc beepApp.c -o beepApp
```

编译成功以后就会生成 beepApp 这个应用程序。

26.4.2 运行测试

将上一小节编译出来的 beep.ko 和 beepApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 beep.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe beep           //加载驱动
```

驱动加载成功以后会在终端中输出一些信息，如图 26.4.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ depmod
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe beep
beep-gpio num = 39
beep major=241,minor=0
```

图 26.4.2.1 驱动加载成功以后输出的信息

从图 26.4.2.1 可以看出，beep 这个节点找到了，并且 PC7 这个 GPIO 的编号为 39。使用

beepApp 软件来测试驱动是否工作正常，输入如下命令打开蜂鸣器：

```
./beepApp  /dev/beep 1      //打开蜂鸣器
```

输入上述命令，观察开发板上的蜂鸣器是否有鸣叫，如果鸣叫的话说明驱动工作正常。在输入如下命令关闭蜂鸣器：

```
./beepApp  /dev/beep 0      //关闭蜂鸣器
```

输入上述命令以后观察开发板上的蜂鸣器是否停止鸣叫。如果要卸载驱动的话输入如下命令即可：

```
rmmod beep.ko
```

第二十七章 Linux 并发与竞争

Linux 是一个多任务操作系统，肯定会存在多个任务共同操作同一段内存或者设备的情况，多个任务甚至中断都能访问的资源叫做共享资源，就和共享单车一样。在驱动开发中要注意对共享资源的保护，也就是要处理对共享资源的并发访问。比如共享单车，大家按照谁扫谁骑走的原则来共用这个单车，如果没有这个并发访问共享单车的原则存在，只怕到时候为了一辆单车要打起来了。在 Linux 驱动编写过程中对于并发控制的管理非常重要，本章我们就来学习一下如何在 Linux 驱动中处理并发。

27.1 并发与竞争

1、并发与竞争简介

并发就是多个“用户”同时访问同一个共享资源，比如你们公司有一台打印机，你们公司的所有人都可以使用。现在小李和小王要同时使用这一台打印机，都要打印一份文件。小李要打印的文件内容如下：

示例代码 27.1.1 小李要打印的内容

```
我叫小李
电话：123456
工号：16
```

小王要打印的内容如下：

示例代码 27.1.2 小王要打印的内容

```
我叫小王
电话：678910
工号：20
```

这两份文档肯定是各自打印出来的，不能相互影响。当两个人同时打印的时候，如果打印机不做处理，可能会出现小李的文档打印了一行，然后开始打印小王的文档，这样打印出来的文档就错乱了，可能会出现如下的错误文档内容：

示例代码 27.1.3 小王打印出来的错误文档

```
我叫小王
电话：123456
工号：20
```

可以看出，小王打印出来的文档中电话号码错误了，变成小李的了，这是绝对不允许的。如果有多人同时向打印机发送了多份文档，打印机必须保证一次只能打印一份文档，只有打印完成以后才能打印其他的文档。

Linux 系统是个多任务操作系统，会存在多个任务同时访问同一片内存区域，这些任务可能会相互覆盖这段内存中的数据，造成内存数据混乱。针对这个问题必须要做处理，严重的话可能会导致系统崩溃。现在的 Linux 系统并发产生的原因很复杂，总结一下有下面几个主要原因：

①、多线程并发访问，Linux 是多任务(线程)的系统，所以多线程访问是最基本的原因。

②、抢占式并发访问，从 2.6 版本内核开始，Linux 内核支持抢占，也就是说调度程序可以在任意时刻抢占正在运行的线程，从而运行其他的线程。

③、中断程序并发访问，这个无需多说，学过 STM32 的同学应该知道，硬件中断的权利可是很大的。

④、SMP(多核)核间并发访问，现在 ARM 架构的多核 SOC 很常见，多核 CPU 存在核间并发访问。

并发访问带来的问题就是竞争，学过 FreeRTOS 或 UCOS 这类 RTOS 的同学应该知道临界区这个概念，所谓的临界区就是共享数据段，对于临界区必须保证一次只有一个线程访问，也就是要保证临界区是原子访问的，注意这里的“原子”不是正点原子的“原子”。我们都知道，原子是化学反应不可再分的基本微粒，这里的原子访问就表示这一个访问是一个步骤，不能再进行拆分。如果多个线程同时操作临界区就表示存在竞争，我们在编写驱动的时候一定要注意避免并发和防止竞争访问。很多 Linux 驱动初学者往往不注意这一点，在驱动程序中埋下了隐

患，这类问题往往又很不容易查找，导致驱动调试难度加大、费时费力。所以我们一般在编写驱动的时候就要考虑到并发与竞争，而不是驱动都编写完了然后再处理并发与竞争。

2、保护内容是什么

前面一直说要防止并发访问共享资源，换句话说就是要保护共享资源，防止进行并发访问。那么问题来了，什么是共享资源？现实生活中的公共电话、共享单车这些是共享资源，我们都很容易理解，那么在程序中什么是共享资源？也就是保护的内容是什么？我们保护的不是代码，而是数据！某个线程的局部变量不需要保护，我们要保护的是多个线程都会访问的共享数据。一个整形的全局变量 a 是数据，一份要打印的文档也是数据，虽然我们知道了要对共享数据进行保护，那么怎么判断哪些共享数据要保护呢？找到要保护的数据才是重点，而这个也是难点，因为驱动程序各不相同，那么数据也千变万化，一般像全局变量，设备结构体这些肯定是要保护的，至于其他的数据就要根据实际的驱动程序而定了。

当我们发现驱动程序中存在并发和竞争的时候一定要处理掉，接下来我们依次来学习一下 Linux 内核提供的几种并发和竞争的处理方法。

27.2 原子操作

27.2.1 原子操作简介

首先看一下原子操作，原子操作就是指不能再进一步分割的操作，一般原子操作用于变量或者位操作。假如现在要对无符号整形变量 a 赋值，值为 3，对于 C 语言来讲很简单，直接就是：

```
a = 3
```

但是 C 语言要先编译为成汇编指令，ARM 架构不支持直接对寄存器(内存)进行读写操作，要借助寄存器 R0、R1 等来完成赋值操作。假设变量 a 的地址为 0X3000000，“a=3”这一行 C 语言可能会被编译为如下所示的汇编代码：

```
                                示例代码 27.2.1.1 汇编示例代码
1 ldr r0, =0X30000000           /* 变量 a 地址                */
2 ldr r1, = 3                   /* 要写入的值                 */
3 str r1, [r0]                  /* 将 3 写入到 a 变量中       */
```

示例代码 27.2.1.1 只是一个简单的举例说明，实际的结果要比示例代码复杂的多。从上述代码可以看出，C 语言里面简简单单的一句“a=3”，编译成汇编文件以后变成了 3 句，那么程序在执行的时候肯定是按照示例代码 27.2.1.1 中的汇编语句一条一条的执行。假设现在线程 A 要向 a 变量写入 10 这个值，而线程 B 也要向 a 变量写入 20 这个值，我们理想中的执行顺序如图 27.2.1.1 所示：

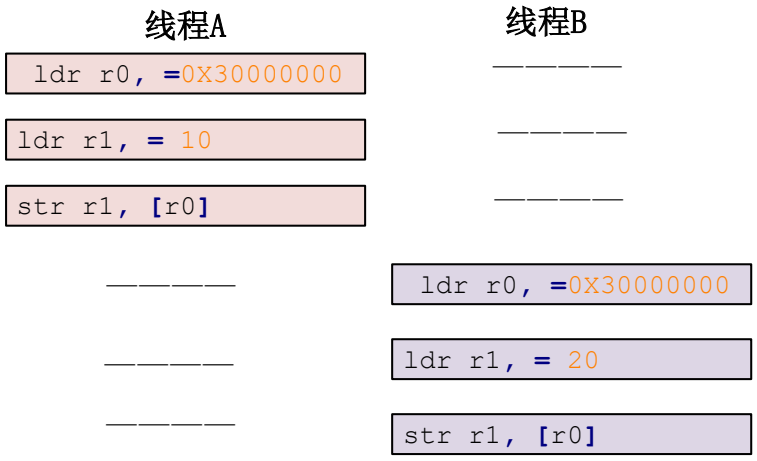


图 27.2.1.1 理想的执行流程

按照图 27.2.1.1 所示的流程，确实可以实现线程 A 将 a 变量设置为 10，线程 B 将 a 变量设置为 20。但是实际上的执行流程可能如图 27.2.1.2 所示：

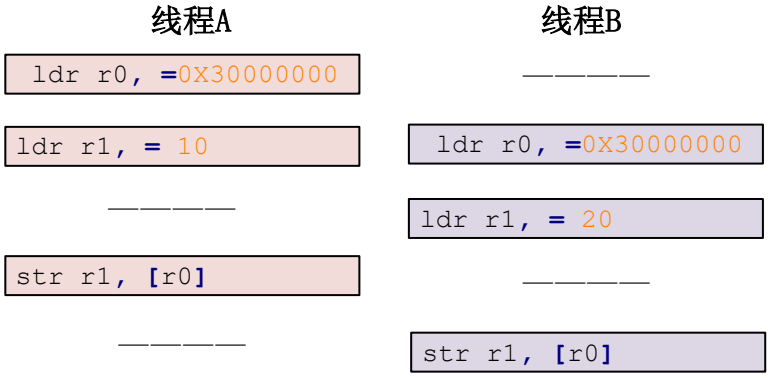


图 27.2.1.2 可能的执行流程

按照图 27.2.1.2 所示的流程，线程 A 最终将变量 a 设置为了 20，而并不是要求的 10！线程 B 没有问题。这就是一个最简单的设置变量值的并发与竞争的例子，要解决这个问题就要保证示例代码 27.2.1.1 中的三行汇编指令作为一个整体运行，也就是作为一个原子存在。Linux 内核提供了一组原子操作 API 函数来完成此功能，Linux 内核提供了两组原子操作 API 函数，一组是对整形变量进行操作的，一组是对位进行操作的，我们接下来看一下这些 API 函数。

27.2.2 原子整形操作 API 函数

Linux 内核定义了叫做 atomic_t 的结构体来完成整形数据的原子操作，在使用中用原子变量来代替整形变量，此结构体定义在 include/linux/types.h 文件中，定义如下：

示例代码 27.2.2.1 atomic_t 结构体

```
171 typedef struct {
172     int counter;
173 } atomic_t;
```

如果要使用原子操作 API 函数，首先要先定义一个 atomic_t 的变量，如下所示：

```
atomic_t  a;  //定义 a
```

也可以在定义原子变量的时候给原子变量赋初值，如下所示：

```
atomic_t b = ATOMIC_INIT(0);  //定义原子变量 b 并赋初值为 0
```

可以通过宏 ATOMIC_INIT 向原子变量赋初值。

原子变量有了，接下来就是对原子变量进行操作，比如读、写、增加、减少等等，Linux 内核提供了大量的原子操作 API 函数，如表 27.2.2.1 所示：

函数	描述
ATOMIC_INIT(int i)	定义原子变量的时候对其初始化。
int atomic_read(atomic_t *v)	读取 v 的值，并且返回。
void atomic_set(atomic_t *v, int i)	向 v 写入 i 值。
void atomic_add(int i, atomic_t *v)	给 v 加上 i 值。
void atomic_sub(int i, atomic_t *v)	从 v 减去 i 值。
void atomic_inc(atomic_t *v)	给 v 加 1，也就是自增。
void atomic_dec(atomic_t *v)	从 v 减 1，也就是自减
int atomic_dec_return(atomic_t *v)	从 v 减 1，并且返回 v 的值。
int atomic_inc_return(atomic_t *v)	给 v 加 1，并且返回 v 的值。
int atomic_sub_and_test(int i, atomic_t *v)	从 v 减 i，如果结果为 0 就返回真，否则返回假
int atomic_dec_and_test(atomic_t *v)	从 v 减 1，如果结果为 0 就返回真，否则返回假
int atomic_inc_and_test(atomic_t *v)	给 v 加 1，如果结果为 0 就返回真，否则返回假
int atomic_add_negative(int i, atomic_t *v)	给 v 加 i，如果结果为负就返回真，否则返回假

表 27.2.2.1 原子整形操作 API 函数表

如果使用 64 位的 SOC 的话，就要用到 64 位的原子变量，Linux 内核也定义了 64 位原子结构体，如下所示：

示例代码 27.2.2.2 atomic64_t 结构体

```
176 typedef struct {
177     s64 counter;
178 } atomic64_t;
```

可以看出，counter 变为了 s64 类型，s64 本质是 long long 类型，相应的也提供了 64 位原子变量的操作 API 函数，这里我们就不详细讲解了，和表 27.2.1.1 中的 API 函数有用法一样，只是将“atomic_”前缀换为“atomic64_”，将 int 换为 long long。如果使用的是 64 位的 SOC，那么就要使用 64 位的原子操作函数。Cortex-A7 是 32 位的架构，所以本书中只使用表 27.2.2.1 中的 32 位原子操作函数。原子变量和相应的 API 函数使用起来很简单，参考如下示例：

示例代码 27.2.2.2 原子变量和 API 函数使用

```
atomic_t v = ATOMIC_INIT(0);    /* 定义并初始化原子变零 v=0  */

atomic_set(&v, 10);             /* 设置 v=10                */
atomic_read(&v);                /* 读取 v 的值，肯定是 10    */
atomic_inc(&v);                 /* v 的值加 1，v=11          */
```

27.2.3 原子位操作 API 函数

位操作也是很常用的操作，Linux 内核也提供了一系列的原子位操作 API 函数，只不过原子位操作不像原子整形变量那样有个 atomic_t 的数据结构，原子位操作是直接对内存进行操作，API 函数如表 27.2.3.1 所示：

函数	描述
void set_bit(int nr, void *p)	将 p 地址的第 nr 位置 1。

void clear_bit(int nr,void *p)	将 p 地址的第 nr 位清零。
void change_bit(int nr, void *p)	将 p 地址的第 nr 位进行翻转。
int test_bit(int nr, void *p)	获取 p 地址的第 nr 位的值。
int test_and_set_bit(int nr, void *p)	将 p 地址的第 nr 位置 1，并且返回 nr 位原来的值。
int test_and_clear_bit(int nr, void *p)	将 p 地址的第 nr 位清零，并且返回 nr 位原来的值。
int test_and_change_bit(int nr, void *p)	将 p 地址的第 nr 位翻转，并且返回 nr 位原来的值。

表 27.2.3.1 原子位操作函数表

27.3 自旋锁

27.3.1 自旋锁简介

原子操作只能对整形变量或者位进行保护，但是，在实际的使用环境中怎么可能只有整形变量或位这么简单的临界区。举个最简单的例子，设备结构体变量就不是整型变量，我们对于结构体中成员变量的操作也要保证原子性，在线程 A 对结构体变量使用期间，应该禁止其他的线程来访问此结构体变量，这些工作原子操作都不能胜任，需要本节要讲的锁机制，在 Linux 内核中就是自旋锁。

当一个线程要访问某个共享资源的时候首先要先获取相应的锁，锁只能被一个线程持有，只要此线程不释放持有的锁，那么其他的线程就不能获取此锁。对于自旋锁而言，如果自旋锁正在被线程 A 持有，线程 B 想要获取自旋锁，那么线程 B 就会处于忙循环-旋转-等待状态，线程 B 不会进入休眠状态或者说去做其他的处理，而是会一直傻傻的在那里“转圈圈”的等待锁可用。比如现在有个公用电话亭，一次肯定只能进去一个人打电话，现在电话亭里面有人正在打电话，相当于获得了自旋锁。此时你到了电话亭门口，因为里面有人，所以你不能进去打电话，相当于没有获取自旋锁，这个时候你肯定是站在原地等待，你可能因为无聊的等待而转圈圈消遣时光，反正就是哪里也不能去，要一直等到里面的人打完电话出来。终于，里面的人打完电话出来了，相当于释放了自旋锁，这个时候你就可以使用电话亭打电话了，相当于获取到了自旋锁。

自旋锁的“自旋”也就是“原地打转”的意思，“原地打转”的目的是为了等待自旋锁可以用，可以访问共享资源。把自旋锁比作一个变量 a，变量 a=1 的时候表示共享资源可用，当 a=0 的时候表示共享资源不可用。现在线程 A 要访问共享资源，发现 a=0(自旋锁被其他线程持有)，那么线程 A 就会不断的查询 a 的值，直到 a=1。从这里我们可以看到自旋锁的一个缺点：那就等待自旋锁的线程会一直处于自旋状态，这样会浪费处理器时间，降低系统性能，所以自旋锁的持有时间不能太长。自旋锁适用于短时期的轻量级加锁，如果遇到需要长时间持有锁的场景那就需要换其他的方法了，这个我们后面会讲解。

Linux 内核使用结构体 spinlock_t 表示自旋锁，结构体定义如下所示：

示例代码 27.3.1.1 spinlock_t 结构体

```
61 typedef struct spinlock {
62     union {
63         struct raw_spinlock rlock;
64
65 #ifdef CONFIG_DEBUG_LOCK_ALLOC
66 # define LOCK_PADSIZE (offsetof(struct raw_spinlock, dep_map))
67         struct {
68             u8 __padding[LOCK_PADSIZE];
```

```
69              struct lockdep_map dep_map;
70          };
71  #endif
72      };
73  } spinlock_t;
```

在使用自旋锁之前，肯定要先定义一个自旋锁变量，定义方法如下所示：

```
spinlock_t    lock;  //定义自旋锁
```

定义好自旋锁变量以后就可以使用相应的 API 函数来操作自旋锁。

27.3.2 自旋锁 API 函数

最基本的自旋锁 API 函数如表 27.3.2.1 所示：

函数	描述
DEFINE_SPINLOCK(spinlock_t lock)	定义并初始化一个自选变量。
int spin_lock_init(spinlock_t *lock)	初始化自旋锁。
void spin_lock(spinlock_t *lock)	获取指定的自旋锁，也叫做加锁。
void spin_unlock(spinlock_t *lock)	释放指定的自旋锁。
int spin_trylock(spinlock_t *lock)	尝试获取指定的自旋锁，如果没有获取到就返回 0
int spin_is_locked(spinlock_t *lock)	检查指定的自旋锁是否被获取，如果没有被获取就返回非 0，否则返回 0。

表 27.3.2.1 自旋锁基本 API 函数表

表 27.3.2.1 中的自旋锁 API 函数适用于 SMP 或支持抢占的单 CPU 下线程之间的并发访问，也就是用于线程与线程之间，被自旋锁保护的临界区一定不能调用任何能够引起睡眠和阻塞的 API 函数，否则的话会可能会导致死锁现象的发生。自旋锁会自动禁止抢占，也就说当线程 A 得到锁以后会暂时禁止内核抢占。如果线程 A 在持有锁期间进入了休眠状态，那么线程 A 会自动放弃 CPU 使用权。线程 B 开始运行，线程 B 也想要获取锁，但是此时锁被 A 线程持有，而且内核抢占还被禁止了！线程 B 无法被调度出去，那么线程 A 就无法运行，锁也就无法释放，好了，死锁发生了！

表 27.3.2.1 中的 API 函数用于线程之间的并发访问，如果此时中断也要插一脚，中断也想访问共享资源，那该怎么办呢？首先可以肯定的是，中断里面可以使用自旋锁，但是在中断里面使用自旋锁的时候，在获取锁之前一定要先禁止本地中断(也就是本 CPU 中断，对于多核 SOC 来说会有多个 CPU 核)，否则可能导致锁死现象的发生，如图 27.3.2.1 所示：

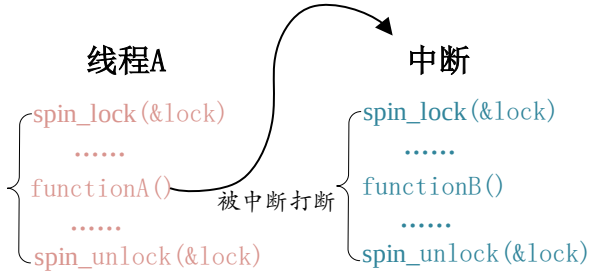


图 27.3.2.1 中断打断线程

在图 27.3.2.1 中，线程 A 先运行，并且获取到了 lock 这个锁，当线程 A 运行 functionA 函数的时候中断发生了，中断抢走了 CPU 使用权。右边的中断服务函数也要获取 lock 这个锁，但是这个锁被线程 A 占有着，中断就会一直自旋，等待锁有效。但是在中断服务函数执行完之

前，线程 A 是不可能执行的，线程 A 说“你先放手”，中断说“你先放手”，场面就这么僵持着，死锁发生！

最好的解决方法就是获取锁之前关闭本地中断，Linux 内核提供了相应的 API 函数，如表 27.3.2.2 所示：

函数	描述
void spin_lock_irq(spinlock_t *lock)	禁止本地中断，并获取自旋锁。
void spin_unlock_irq(spinlock_t *lock)	激活本地中断，并释放自旋锁。
void spin_lock_irqsave(spinlock_t *lock, unsigned long flags)	保存中断状态，禁止本地中断，并获取自旋锁。
void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放自旋锁。

表 27.3.2.2 线程与中断并发访问处理 API 函数

使用 spin_lock_irq/spin_unlock_irq 的时候需要用户能够确定加锁之前的中断状态，但实际上内核很庞大，运行也是“千变万化”，我们是很难确定某个时刻的中断状态，因此不推荐使用 spin_lock_irq/spin_unlock_irq。建议使用 spin_lock_irqsave/ spin_unlock_irqrestore，因为这一组函数会保存中断状态，在释放锁的时候会恢复中断状态。一般在线程中使用 spin_lock_irqsave/ spin_unlock_irqrestore，在中断中使用 spin_lock/spin_unlock，示例代码如下所示：

示例代码 27.3.2.1 自旋锁使用示例

```
1  DEFINE_SPINLOCK(lock)                          /* 定义并初始化一个锁    */
2
3  /* 线程 A */
4  void functionA (){
5      unsigned long flags;                       /* 中断状态              */
6      spin_lock_irqsave(&lock, flags)            /* 获取锁                */
7      /* 临界区 */
8      spin_unlock_irqrestore(&lock, flags)       /* 释放锁                */
9  }
10
11 /* 中断服务函数 */
12 void irq() {
13     spin_lock(&lock)                           /* 获取锁                */
14     /* 临界区 */
15     spin_unlock(&lock)                         /* 释放锁                */
16 }
```

下半部(BH)也会竞争共享资源，有些资料也会将下半部叫做底半部。关于下半部后面的章节会讲解，如果要在下半部里面使用自旋锁，可以使用表 27.3.2.3 中的 API 函数：

函数	描述
void spin_lock_bh(spinlock_t *lock)	关闭下半部，并获取自旋锁。
void spin_unlock_bh(spinlock_t *lock)	打开下半部，并释放自旋锁。

表 27.3.2.3 下半部竞争处理函数

27.3.3 其他类型的锁

在自旋锁的基础上还衍生出了其他特定场合使用的锁，这些锁在驱动中其实用的不多，更多的是在 Linux 内核中使用，本节我们简单来了解一下这些衍生出来的锁。

1、读写自旋锁

现在有个学生信息表，此表存放着学生的年龄、家庭住址、班级等信息，此表可以随时被修改和读取。此表肯定是数据，那么必须要对其进行保护，如果我们现在使用自旋锁对其进行保护。每次只能一个读操作或者写操作，但是，实际上此表是可以并发读取的。只需要保证在修改此表的时候没人读取，或者在其他人读取此表的时候没有人修改此表就行了。也就是此表的读和写不能同时进行，但是可以多人并发的读取此表。像这样，当某个数据结构符合读/写或生产者/消费者模型的时候就可以使用读写自旋锁。

读写自旋锁为读和写操作提供了不同的锁，一次只能允许一个写操作，也就是只能一个线程持有写锁，而且不能进行读操作。但是当没有写操作的时候允许一个或多个线程持有读锁，可以进行并发的读操作。Linux 内核使用 rwlock_t 结构体表示读写锁，结构体定义如下(删除了条件编译)：

示例代码 27.3.3.1 rwlock_t 结构体

```
typedef struct {
    arch_rwlock_t raw_lock;
} rwlock_t;
```

读写锁操作 API 函数分为两部分，一个是给读使用的，一个是给写使用的，这些 API 函数如表 27.3.3.1 所示：

函数	描述
DEFINE_RWLOCK(rwlock_t lock)	定义并初始化读写锁
void rwlock_init(rwlock_t *lock)	初始化读写锁。
读锁	
void read_lock(rwlock_t *lock)	获取读锁。
void read_unlock(rwlock_t *lock)	释放读锁。
void read_lock_irq(rwlock_t *lock)	禁止本地中断，并且获取读锁。
void read_unlock_irq(rwlock_t *lock)	打开本地中断，并且释放读锁。
void read_lock_irqsave(rwlock_t *lock, unsigned long flags)	保存中断状态，禁止本地中断，并获取读锁。
void read_unlock_irqrestore(rwlock_t *lock, unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放读锁。
void read_lock_bh(rwlock_t *lock)	关闭下半部，并获取读锁。
void read_unlock_bh(rwlock_t *lock)	打开下半部，并释放读锁。
写锁	
void write_lock(rwlock_t *lock)	获取写锁。
void write_unlock(rwlock_t *lock)	释放写锁。
void write_lock_irq(rwlock_t *lock)	禁止本地中断，并且获取写锁。
void write_unlock_irq(rwlock_t *lock)	打开本地中断，并且释放写锁。
void write_lock_irqsave(rwlock_t *lock, unsigned long flags)	保存中断状态，禁止本地中断，并获取写锁。

void write_unlock_irqrestore(rwlock_t *lock, unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放读锁。
void write_lock_bh(rwlock_t *lock)	关闭下半部，并获取读锁。
void write_unlock_bh(rwlock_t *lock)	打开下半部，并释放读锁。

表 27.3.3.1 读写锁 API 函数

2、顺序锁

顺序锁在读写锁的基础上衍生而来的，使用读写锁的时候读操作和写操作不能同时进行。使用顺序锁的话可以允许在写的时候进行读操作，也就是实现同时读写，但是不允许同时进行并发的写操作。虽然顺序锁的读和写操作可以同时进行，但是如果在读的过程中发生了写操作，最好重新进行读取，保证数据完整性。顺序锁保护的资源不能是指针，因为如果在写操作的时候可能会导致指针无效，而这个时候恰巧有读操作访问指针的话就可能导致意外发生，比如读取野指针导致系统崩溃。Linux 内核使用 seqlock_t 结构体表示顺序锁，结构体定义如下：

示例代码 27.3.3.2 seqlock_t 结构体

```
404 typedef struct {
405   struct seqcount seqcount;
406   spinlock_t lock;
407 } seqlock_t;
```

关于顺序锁的 API 函数如表 27.3.3.2 所示：

函数	描述
DEFINE_SEQLOCK(seqlock_t sl)	定义并初始化顺序锁
void seqlock_ini seqlock_t *sl)	初始化顺序锁。
顺序锁写操作	
void write_seqlock(seqlock_t *sl)	获取写顺序锁。
void write_sequnlock(seqlock_t *sl)	释放写顺序锁。
void write_seqlock_irq(seqlock_t *sl)	禁止本地中断，并且获取写顺序锁
void write_sequnlock_irq(seqlock_t *sl)	打开本地中断，并且释放写顺序锁。
void write_seqlock_irqsave(seqlock_t *sl, unsigned long flags)	保存中断状态，禁止本地中断，并获取写顺序锁。
void write_sequnlock_irqrestore(seqlock_t *sl, unsigned long flags)	将中断状态恢复到以前的状态，并且激活本地中断，释放写顺序锁。
void write_seqlock_bh(seqlock_t *sl)	关闭下半部，并获取写读锁。
void write_sequnlock_bh(seqlock_t *sl)	打开下半部，并释放写读锁。
顺序锁读操作	
unsigned read_seqbegin(const seqlock_t *sl)	读单元访问共享资源的时候调用此函数，此函数会返回顺序锁的顺序号。
unsigned read_seqretry(const seqlock_t *sl, unsigned start)	读结束以后调用此函数检查在读的过程中有没有对资源进行写操作，如果有的话就要重读

表 27.3.3.2 顺序锁 API 函数表

27.3.4 自旋锁使用注意事项

综合前面关于自旋锁的信息，我们需要在使用自旋锁的时候要注意一下几点：

①、因为在等待自旋锁的时候处于“自旋”状态，因此锁的持有时间不能太长，一定要短，否则的话会降低系统性能。如果临界区比较大，运行时间比较长的话要选择其他的并发处理方式，比如稍后要讲的信号量和互斥体。

②、自旋锁保护的临界区内不能调用任何可能导致线程休眠的 API 函数，否则的话可能导致死锁。

③、不能递归申请自旋锁，因为一旦通过递归的方式申请一个你正在持有的锁，那么你就必须“自旋”，等待锁被释放，然而你正处于“自旋”状态，根本没法释放锁。结果就是自己把自己锁死了！

④、在编写驱动程序的时候我们必须考虑到驱动的可移植性，因此不管你用的是单核的还是多核的 SOC，都将其当做多核 SOC 来编写驱动程序。

27.4 信号量

27.4.1 信号量简介

大家如果有学习过 FreeRTOS 或者 UCOS 的话就应该对信号量很熟悉，因为信号量是同步的一种方式。Linux 内核也提供了信号量机制，信号量常常用于控制对共享资源的访问。举一个很常见的例子，某个停车场有 100 个停车位，这 100 个停车位大家都可以用，对于大家来说这 100 个停车位就是共享资源。假设现在这个停车场正常运行，你要把车停到这个这个停车场肯定要先看一下现在停了多少车了？还有没有停车位？当前停车数量就是一个信号量，具体的停车数量就是这个信号量值，当这个值到 100 的时候说明停车场满了。停车场满的时你可以等一会看看有没有其他的车开出停车场，当有车开出停车场的时候停车数量就会减一，也就是说信号量减一，此时你就可以把车停进去了，你把车停进去以后停车数量就会加一，也就是信号量加一。这就是一个典型的使用信号量进行共享资源管理的案例，在这个案例中使用的就是计数型信号量。

相比于自旋锁，信号量可以使线程进入休眠状态，比如 A 与 B、C 合租了一套房子，这个房子只有一个厕所，一次只能一个人使用。某一天早上 A 去上厕所了，过了一会 B 也想用厕所，因为 A 在厕所里面，所以 B 只能等到 A 用来了才能进去。B 要么就一直在厕所门口等着，等 A 出来，这个时候就相当于自旋锁。B 也可以告诉 A，让 A 出来以后通知他一下，然后 B 继续回房间睡觉，这个时候相当于信号量。可以看出，使用信号量会提高处理器的使用效率，毕竟不用一直傻乎乎的在那里“自旋”等待。但是，信号量的开销要比自旋锁大，因为信号量使线程进入休眠状态以后会切换线程，切换线程就会有开销。总结一下信号量的特点：

①、因为信号量可以使等待资源线程进入休眠状态，因此适用于那些占用资源比较久的场合。

②、因此信号量不能用于中断中，因为信号量会引起休眠，中断不能休眠。

③、如果共享资源的持有时间比较短，那就不适合使用信号量了，因为频繁的休眠、切换线程引起的开销要远大于信号量带来的那点优势。

信号量有一个信号量值，相当于一个房子有 10 把钥匙，这 10 把钥匙就相当于信号量值为 10。因此，可以通过信号量来控制访问共享资源的访问数量，如果要想进房间，那就要先获取一把钥匙，信号量值减 1，直到 10 把钥匙都被拿走，信号量值为 0，这个时候就不允许任何人进入房间了，因为没钥匙了。如果有人从房间出来，那他要归还他所持有的那把钥匙，信号量值加 1，此时有 1 把钥匙了，那么可以允许进去一个人。相当于通过信号量控制访问资源的线程数，在初始化的时候将信号量值设置的大于 1，那么这个信号量就是计数型信号量，计数型信号量不能用于互斥访问，因为它允许多个线程同时访问共享资源。如果要互斥的访问共享资

源那么信号量的值就不能大于 1，此时的信号量就是一个二值信号量。

27.4.2 信号量 API 函数

Linux 内核使用 semaphore 结构体表示信号量，结构体内容如下所示：

示例代码 27.4.2.1 semaphore 结构体

```
15 struct semaphore {
16    raw_spinlock_t      lock;
17    unsigned int        count;
18    struct list_head    wait_list;
19 };
```

要想使用信号量就得先定义，然后初始化信号量。有关信号量的 API 函数如表 27.4.2.1 所示：

函数	描述
DEFINE_SEAMPHORE(name)	定义一个信号量，并且设置信号量的值为 1。
void sema_init(struct semaphore *sem, int val)	初始化信号量 sem，设置信号量值为 val。
void down(struct semaphore *sem)	获取信号量，因为会导致休眠，因此不能在中断中使用。
int down_trylock(struct semaphore *sem);	尝试获取信号量，如果能获取到信号量就获取，并且返回 0。如果不能就返回非 0，并且不会进入休眠。
int down_interruptible(struct semaphore *sem)	获取信号量，和 down 类似，只是使用 down 进入休眠状态的线程不能被信号打断。而使用此函数进入休眠以后是可以被信号打断的。
void up(struct semaphore *sem)	释放信号量

表 27.4.2.1 信号量 API 函数

信号量的使用如下所示：

示例代码 27.4.2.2 信号量使用示例

```
struct semaphore sem;    /* 定义信号量    */

sema_init(&sem, 1);      /* 初始化信号量 */

down(&sem);              /* 申请信号量    */
/* 临界区 */
up(&sem);                /* 释放信号量    */
```

27.5 互斥体

27.5.1 互斥体简介

在 FreeRTOS 和 UCOS 中也有互斥体，将信号量的值设置为 1 就可以使用信号量进行互斥访问了，虽然可以通过信号量实现互斥，但是 Linux 提供了一个比信号量更专业的机制来进行互斥，它就是互斥体—mutex。互斥访问表示一次只有一个线程可以访问共享资源，不能递归申请互斥体。在我们编写 Linux 驱动的时候遇到需要互斥访问的地方建议使用 mutex。Linux 内核

使用 mutex 结构体表示互斥体，定义如下(省略条件编译部分)：

示例代码 27.5.1.1 mutex 结构体

```
struct mutex {
    atomic_long_t      owner;
    spinlock_t         wait_lock;
};
```

在使用 mutex 之前要先定义一个 mutex 变量。在使用 mutex 的时候要注意如下几点：

①、mutex 可以导致休眠，因此不能在中断中使用 mutex，中断中只能使用自旋锁。

②、和信号量一样，mutex 保护的临界区可以调用引起阻塞的 API 函数。

③、因为一次只有一个线程可以持有 mutex，因此，必须由 mutex 的持有者释放 mutex。并且 mutex 不能递归上锁和解锁。

27.5.2 互斥体 API 函数

有关互斥体的 API 函数如表 27.5.2.1 所示：

函数	描述
DEFINE_MUTEX(name)	定义并初始化一个 mutex 变量。
void mutex_init(mutex *lock)	初始化 mutex。
void mutex_lock(struct mutex *lock)	获取 mutex，也就是给 mutex 上锁。如果获取不到就进休眠。
void mutex_unlock(struct mutex *lock)	释放 mutex，也就给 mutex 解锁。
int mutex_trylock(struct mutex *lock)	尝试获取 mutex，如果成功就返回 1，如果失败就返回 0。
int mutex_is_locked(struct mutex *lock)	判断 mutex 是否被获取，如果是的话就返回 1，否则返回 0。
int mutex_lock_interruptible(struct mutex *lock)	使用此函数获取信号量失败进入休眠以后可以被信号打断。

表 27.5.2.1 互斥体 API 函数

互斥体的使用如下所示：

示例代码 27.5.2.1 互斥体使用示例

```
1 struct mutex lock;      /* 定义一个互斥体    */
2 mutex_init(&lock);      /* 初始化互斥体      */
3
4 mutex_lock(&lock);      /* 上锁              */
5 /* 临界区 */
6 mutex_unlock(&lock);    /* 解锁              */
```

关于 Linux 中的并发和竞争就讲解到这里，Linux 内核还有很多其他的处理并发和竞争的机制，本章我们主要讲解了常用的原子操作、自旋锁、信号量和互斥体。以后我们在编写 Linux 驱动的时候就会频繁的使用到这几种机制，希望大家能够深入理解这几个常用的机制。

第二十八章 Linux 并发与竞争实验

在上一章中我们学习了 Linux 下的并发与竞争，并且学习了四种常用的处理并发和竞争的机制：原子操作、自旋锁、信号量和互斥体。本章我们就通过四个实验来学习如何在驱动中使用这四种机制。

28.1 原子操作实验

本实验对应的例程路径为：开发板光盘→ 1、程序源码→2、Linux 驱动例程→7_atomic。

本例程我们在第二十五章的 gpioled.c 文件基础上完成。在本节使用中我们使用原子操作来实现对 LED 这个设备的互斥访问，也就是一次只允许一个应用程序可以使用 LED 灯。

28.1.1 实验程序编写

1、修改设备树文件

因为本章实验是在第二十五章实验的基础上完成的，因此不需要对设备树做任何的修改。

2、LED 驱动修改

本节实验在第二十五章实验驱动文件 gpioled.c 的基础上修改而来。新建名为“7_atomic”的文件夹，然后在 7_atomic 文件夹里面创建 vscode 工程，工作区命名为“atomic”。将 5_gpioled 实验中的 gpioled.c 复制到 7_atomic 文件夹中，并且重命名为 atomic.c。本节实验重点就是使用 atomic 来实现一次只能允许一个应用访问 LED，所以我们只需要在 atomic.c 文件源码的基础上加上添加 atomic 相关代码即可，完成以后的 atomic.c 文件内容如下所示：

示例代码 28.1.1.1 atomic.c 文件代码段

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <linux/of.h>
12 #include <linux/of_address.h>
13 #include <linux/of_gpio.h>
14 #include <asm/mach/map.h>
15 #include <asm/uaccess.h>
16 #include <asm/io.h>
17 /***************************************************************
18 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
19 文件名      : gpioled.c
20 作者        : 正点原子 Linux 团队
21 版本        : V1.0
22 描述        : 原子操作实验，使用原子变量来实现对实现设备的互斥访问。
23 其他        : 无
24 论坛        : www.openedv.com
25 日志        : 初版 V1.0 2021/1/4 正点原子 Linux 团队创建
26 ***************************************************************/
```

```
27  #define GPIOLED_CNT      1              /* 设备号个数   */
28  #define GPIOLED_NAME     "gpioled"      /* 名字         */
29  #define LEDOFF           0              /* 关灯         */
30  #define LEDON            1              /* 开灯         */
31
32  /* gpioled 设备结构体 */
33  struct gpioled_dev{
34      dev_t devid;              /* 设备号       */
35      struct cdev cdev;         /* cdev         */
36      struct class *class;      /* 类           */
37      struct device *device;    /* 设备         */
38      int major;                /* 主设备号     */
39      int minor;                /* 次设备号     */
40      struct device_node  *nd; /* 设备节点     */
41      int led_gpio;             /* led 所使用的 GPIO 编号        */
42      atomic_t lock;            /* 原子变量     */
43  };
44
45  static struct gpioled_dev gpioled;  /* led 设备 */
46
47
48  /*
49   * @description   : 打开设备
50   * @param - inode : 传递给驱动的 inode
51   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
52   *                  一般在 open 的时候将 private_data 指向设备结构体。
53   * @return        : 0 成功;其他 失败
54   */
55  static int led_open(struct inode *inode, struct file *filp)
56  {
57      /* 通过判断原子变量的值来检查 LED 有没有被别的应用使用 */
58      if (!atomic_dec_and_test(&gpioled.lock)) {
59          atomic_inc(&gpioled.lock);/* 小于 0 的话就加 1,使其原子变量等于 0 */
60          return -EBUSY;             /* LED 被使用，返回忙 */
61      }
62
63      filp->private_data = &gpioled; /* 设置私有数据 */
64      return 0;
65  }
66
67  /*
68   * @description   : 从设备读取数据
69   * @param - filp  : 要打开的设备文件(文件描述符)
```

```
70   * @param - buf   : 返回给用户空间的数据缓冲区
71   * @param - cnt   : 要读取的数据长度
72   * @param - offt  : 相对于文件首地址的偏移
73   * @return        : 读取的字节数，如果为负值，表示读取失败
74   */
75  static ssize_t led_read(struct file *filp, char __user *buf,
                            size_t cnt, loff_t *offt)
76  {
77      return 0;
78  }
79
80  /*
81   * @description   : 向设备写数据
82   * @param - filp  : 设备文件，表示打开的文件描述符
83   * @param - buf   : 要写给设备写入的数据
84   * @param - cnt   : 要写入的数据长度
85   * @param - offt  : 相对于文件首地址的偏移
86   * @return        : 写入的字节数，如果为负值，表示写入失败
87   */
88  static ssize_t led_write(struct file *filp, const char __user *buf,
                            size_t cnt, loff_t *offt)
89  {
90      int retvalue;
91      unsigned char databuf[1];
92      unsigned char ledstat;
93      struct gpioled_dev *dev = filp->private_data;
94
95      retvalue = copy_from_user(databuf, buf, cnt);
96      if(retvalue < 0) {
97          printk("kernel write failed!\r\n");
98          return -EFAULT;
99      }
100
101     ledstat = databuf[0];       /* 获取状态值 */
102
103     if(ledstat == LEDON) {
104         gpio_set_value(dev->led_gpio, 0);   /* 打开 LED 灯 */
105     } else if(ledstat == LEDOFF) {
106         gpio_set_value(dev->led_gpio, 1);   /* 关闭 LED 灯 */
107     }
108     return 0;
109 }
110
```

```
111 /*
112  * @description   : 关闭/释放设备
113  * @param - filp  : 要关闭的设备文件(文件描述符)
114  * @return        : 0 成功;其他 失败
115  */
116 static int led_release(struct inode *inode, struct file *filp)
117 {
118     struct gpioled_dev *dev = filp->private_data;
119 
120     /* 关闭驱动文件的时候释放原子变量 */
121     atomic_inc(&dev->lock);
122 
123     return 0;
124 }
125 
126 /* 设备操作函数 */
127 static struct file_operations gpioled_fops = {
128     .owner = THIS_MODULE,
129     .open = led_open,
130     .read = led_read,
131     .write = led_write,
132     .release =  led_release,
133 };
134 
135 /*
136  * @description   : 驱动出口函数
137  * @param         : 无
138  * @return        : 无
139  */
140 static int __init led_init(void)
141 {
142     int ret = 0;
143     const char *str;
144 
145     /* 1、初始化原子变量    */
146     gpioled.lock = (atomic_t)ATOMIC_INIT(0);
147 
148     /* 2、原子变量初始值为 1 */
149     atomic_set(&gpioled.lock, 1);
150 
151     /* 设置 LED 所使用的 GPIO */
152     /* 1、获取设备节点：gpioled */
153     gpioled.nd = of_find_node_by_path("/gpioled");
```

```
154     if(gpioled.nd == NULL) {
155         printk("gpioled node not find!\r\n");
156         return -EINVAL;
157     }
158 
159     /* 2.读取 status 属性 */
160     ret = of_property_read_string(gpioled.nd, "status", &str);
161     if(ret < 0)
162         return -EINVAL;
163 
164     if (strcmp(str, "okay"))
165         return -EINVAL;
166 
167     /* 3、获取 compatible 属性值并进行匹配 */
168     ret = of_property_read_string(gpioled.nd, "compatible", &str);
169     if(ret < 0) {
170         printk("gpioled: Failed to get compatible property\n");
171         return -EINVAL;
172     }
173 
174     if (strcmp(str, "alientek,led")) {
175         printk("gpioled: Compatible match failed\n");
176         return -EINVAL;
177     }
178 
179     /* 4、 获取设备树中的 gpio 属性，得到 LED 所使用的 LED 编号 */
180     gpioled.led_gpio = of_get_named_gpio(gpioled.nd, "led-gpio", 0);
181     if(gpioled.led_gpio < 0) {
182         printk("can't get led-gpio");
183         return -EINVAL;
184     }
185     printk("led-gpio num = %d\r\n", gpioled.led_gpio);
186 
187     /* 5.向 gpio 子系统申请使用 GPIO */
188     ret = gpio_request(gpioled.led_gpio, "LED-GPIO");
189     if (ret) {
190         printk(KERN_ERR "gpioled: Failed to request led-gpio\n");
191         return ret;
192     }
193 
194     /* 6、设置 PI0 为输出，并且输出高电平，默认关闭 LED 灯 */
195     ret = gpio_direction_output(gpioled.led_gpio, 1);
196     if(ret < 0) {
```

```
197         printk("can't set gpio!\r\n");
198     }
199
200     /* 注册字符设备驱动 */
201     /* 1、创建设备号 */
202     if (gpioled.major) {        /*  定义了设备号 */
203         gpioled.devid = MKDEV(gpioled.major, 0);
204         ret = register_chrdev_region(gpioled.devid, GPIOLED_CNT,
                                         GPIOLED_NAME);
205         if(ret < 0) {
206             pr_err("cannot register %s char driver [ret=%d]\n",
                       GPIOLED_NAME, GPIOLED_CNT);
207             goto free_gpio;
208         }
209     } else {                        /* 没有定义设备号 */
210         ret = alloc_chrdev_region(&gpioled.devid, 0, GPIOLED_CNT,
                                      GPIOLED_NAME);  /* 申请设备号 */
211         if(ret < 0) {
212             pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                       GPIOLED_NAME, ret);
213             goto free_gpio;
214         }
215         gpioled.major = MAJOR(gpioled.devid); /* 获取分配号的主设备号 */
216         gpioled.minor = MINOR(gpioled.devid);  /* 获取分配号的次设备号 */
217     }
218     printk("gpioled major=%d,minor=%d\r\n",gpioled.major,
                                               gpioled.minor);
219
220     /* 2、初始化 cdev */
221     gpioled.cdev.owner = THIS_MODULE;
222     cdev_init(&gpioled.cdev, &gpioled_fops);
223
224     /* 3、添加一个 cdev */
225     cdev_add(&gpioled.cdev, gpioled.devid, GPIOLED_CNT);
226     if(ret < 0)
227         goto del_unregister;
228
229     /* 4、创建类 */
230     gpioled.class = class_create(THIS_MODULE, GPIOLED_NAME);
231     if (IS_ERR(gpioled.class)) {
232         goto del_cdev;
233     }
234
```

```
235     /* 5、创建设备 */
236     gpioled.device = device_create(gpioled.class, NULL,
                                   gpioled.devid, NULL, GPIOLED_NAME);
237     if (IS_ERR(gpioled.device)) {
238         goto destroy_class;
239     }
240     return 0;
241
242 destroy_class:
243     device_destroy(gpioled.class, gpioled.devid);
244 del_cdev:
245     cdev_del(&gpioled.cdev);
246 del_unregister:
247     unregister_chrdev_region(gpioled.devid, GPIOLED_CNT);
248 free_gpio:
249     gpio_free(gpioled.led_gpio);
250     return -EIO;
251 }
252
253 /*
254  * @description   : 驱动出口函数
255  * @param         : 无
256  * @return        : 无
257  */
258 static void __exit led_exit(void)
259 {
260     /* 注销字符设备驱动 */
261     cdev_del(&gpioled.cdev);         /*  删除 cdev */
262     unregister_chrdev_region(gpioled.devid, GPIOLED_CNT);/* 注销*/
263     device_destroy(gpioled.class, gpioled.devid);/* 注销设备 */
264     class_destroy(gpioled.class);   /* 注销类       */
265     gpio_free(gpioled.led_gpio);    /* 释放 GPIO    */
266 }
267
268 module_init(led_init);
269 module_exit(led_exit);
270 MODULE_LICENSE("GPL");
271 MODULE_AUTHOR("ALIENTEK");
272 MODULE_INFO(intree, "Y");
```

第 42 行，原子变量 lock，用来实现一次只能允许一个应用访问 LED 灯，led_init 驱动入口函数会将 lock 的值设置为 1。

第 57~61 行，每次调用 open 函数打开驱动设备的时候先申请 lock，如果申请成功的话就表示 LED 灯还没有被其他的应用使用，如果申请失败就表示 LED 灯正在被其他的应用程序使用。

每次打开驱动设备的时候先使用 atomic_dec_and_test 函数将 lock 减 1，如果 atomic_dec_and_test 函数返回值为真就表示 lock 当前值为 0，说明设备可以使用。如果 atomic_dec_and_test 函数返回值为假，就表示 lock 当前值为负数(lock 值默认是 1)，lock 值为负数的可能性只有一个，那就是其他设备正在使用 LED。其他设备正在使用 LED 灯，那么就只能退出了，在退出之前调用函数 atomic_inc 将 lock 加 1，因为此时 lock 的值被减成了负数，必须要对其加 1，将 lock 的值变为 0。

第 120 行，LED 灯使用完毕，应用程序调用 close 函数关闭的驱动文件，led_release 函数执行，调用 atomic_inc 释放 lcok，也就是将 lock 加 1。

第 146 行，初始化原子变量 lock，初始值设置为 0。

第 149 行，原子变量 lock 设置为 1，这样每次就只允许一个应用使用 LED 灯。

3、编写测试 APP

示例代码 28.1.1.2 atomicApp.c 文件代码

```
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "fcntl.h"
6  #include "stdlib.h"
7  #include "string.h"
8  /***************************************************************
9  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
10 文件名      : atomicApp.c
11 作者        : 正点原子 Linux 团队
12 版本        : V1.0
13 描述        : 原子变量测试 APP，测试原子变量能不能实现一次
14             只允许一个应用程序使用 LED。
15 其他        : 无
16 使用方法    : ./atomicApp /dev/gpioled  0 关闭 LED 灯
17             ./atomicApp /dev/gpioled  1 打开 LED 灯
18 论坛        : www.openedv.com
19 日志        : 初版 V1.0 2021/1/04 正点原子 Linux 团队创建
20 ***************************************************************/
21
22 #define LEDOFF  0
23 #define LEDON   1
24
25 /*
26  * @description   : main 主程序
27  * @param - argc  : argv 数组元素个数
28  * @param - argv  : 具体参数
29  * @return        : 0 成功;其他 失败
30  */
31 int main(int argc, char *argv[])
```

```
32  {
33      int fd, retvalue;
34      char *filename;
35      unsigned char cnt = 0;
36      unsigned char databuf[1];
37
38      if(argc != 3){
39          printf("Error Usage!\r\n");
40          return -1;
41      }
42
43      filename = argv[1];
44
45      /* 打开 beep 驱动 */
46      fd = open(filename, O_RDWR);
47      if(fd < 0){
48          printf("file %s open failed!\r\n", argv[1]);
49          return -1;
50      }
51
52      databuf[0] = atoi(argv[2]); /* 要执行的操作：打开或关闭 */
53
54      /* 向/dev/gpioled 文件写入数据 */
55      retvalue = write(fd, databuf, sizeof(databuf));
56      if(retvalue < 0){
57          printf("LED Control Failed!\r\n");
58          close(fd);
59          return -1;
60      }
61
62      /* 模拟占用 25S LED */
63      while(1) {
64          sleep(5);
65          cnt++;
66          printf("App running times:%d\r\n", cnt);
67          if(cnt >= 5) break;
68      }
69
70      printf("App running finished!");
71      retvalue = close(fd); /* 关闭文件 */
72      if(retvalue < 0){
73          printf("file %s close failed!\r\n", argv[1]);
74          return -1;
```

```
75      }
76      return 0;
77  }
78
```

28.1.2 运行测试

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第二十章实验基本一样，只是将 obj-m 变量的值改为 atomic.o，Makefile 内容如下所示：

示例代码 28.1.2.1 Makefile 文件

```
1  KERNELDIR := home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := atomic.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 atomic.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“atomic.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 atomicApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc atomicApp.c -o atomicApp
```

编译成功以后就会生成 atomicApp 这个应用程序。

3、运行测试

将上一小节编译出来的 atomic.ko 和 atomicApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 atomic.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe atomic.ko      //加载驱动
```

驱动加载成功以后就可以使用 atomicApp 软件来测试驱动是否工作正常，输入如下命令以后台运行模式打开 LED 灯，“&”表示在后台运行 atomicApp 这个软件：

```
./atomicApp /dev/gpioled 1 &      //打开 LED 灯
```

输入上述命令以后观察开发板上的红色 LED 灯是否点亮，然后每隔 5 秒都会输出一行“App running times ”，如图 28.1.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./atomicApp /dev/gpioled  1&
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ App running times:1
App running times:2
App running times:3
App running times:4
```

图 28.1.2.1 打开 LED 灯

从图 28.1.2.1 可以看出，atomicApp 运行正常，输出了“App running times:1”和“App running

times:2”等字符串，这就是模拟 25S 占用，说明 atomicApp 这个软件正在使用 LED 灯。此时再输入如下命令关闭 LED 灯：

```
./atomicApp  /dev/gpioled 0      //关闭 LED 灯
```

输入上述命令以后会发现如图 28.1.2.2 所示输入信息：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./atomicApp /dev/gpioled  0
file /dev/gpioled open failed!
```

图 28.1.2.2 关闭 LED 灯

从图 28.1.2.2 可以看出，打开/dev/gpioled 失败！原因是在图 28.1.2.1 中运行的 atomicAPP 软件正在占用/dev/gpioled，如果再次运行 atomicApp 软件去操作/dev/gpioled 肯定会失败。必须等待图28.1.2.1中的atomicApp运行结束，也就是25S结束以后其他软件才能去操作/dev/gpioled。这个就是采用原子变量实现一次只能有一个应用程序访问 LED 灯。

如果要卸载驱动的话输入如下命令即可：

```
rmmod atomic.ko
```

28.2 自旋锁实验

本实验对应的例程路径为：开发板光盘→ 1、程序源码→2、Linux 驱动例程→8_spinlock。

上一节我们使用原子变量实现了一次只能有一个应用程序访问 LED 灯，本节我们使用自旋锁来实现此功能。在使用自旋锁之前，先回顾一下自旋锁的使用注意事项：

①、自旋锁保护的临界区要尽可能的短，因此在 open 函数中申请自旋锁，然后在 release 函数中释放自旋锁的方法就不可取。我们可以使用一个变量来表示设备的使用情况，如果设备被使用了那么变量就加一，设备被释放以后变量就减 1，我们只需要使用自旋锁保护这个变量即可。

②、考虑驱动的兼容性，合理的选择 API 函数。

综上所述，在本节例程中，我们通过定义一个变量 dev_stats 表示设备的使用情况，dev_stats 为 0 的时候表示设备没有被使用，dev_stats 大于 0 的时候表示设备被使用。驱动 open 函数中先判断 dev_stats 是否为 0，也就是判断设备是否可用，如果为 0 的话就使用设备，并且将 dev_stats 加 1，表示设备被使用了。使用完以后在 release 函数中将 dev_stats 减 1，表示设备没有被使用了。因此真正实现设备互斥访问的是变量 dev_stats，但是我们要使用自旋锁对 dev_stats 来做保护。

28.2.1 实验程序编写

1、修改设备树文件

本章实验是在上一节实验的基础上完成的，同样不需要对设备树做任何的修改。

2、LED 驱动修改

本节实验在第上一节实验驱动文件 atomic.c 的基础上修改而来。新建名为“8_spinlock”的文件夹，然后在 8_spinlock 文件夹里面创建 vscode 工程，工作区命名为“spinlock”。将 7_atomic 实验中的 atomic.c 复制到 8_spinlock 文件夹中，并且重命名为 spinlock.c。将原来使用 atomic 的地方换为 spinlock 即可，其他代码不需要修改，完成以后的 spinlock.c 文件内容如下所示(有省略)：

示例代码 28.2.1.1 spinlock.c 文件代码

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
```

```
3   #include <linux/delay.h>
.....
17  /***************************************************************
18  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
19  文件名   : gpioled.c
20  作者     : 正点原子 Linux 团队
21  版本     : V1.0
22  描述     : 自旋锁实验，使用自旋锁来实现对实现设备的互斥访问。
23  其他     : 无
24  论坛     : www.openedv.com
25  日志     : 初版 V1.0 2021/1/4 正点原子 Linux 团队创建
26  ***************************************************************/
27  #define GPIOLED_CNT      1              /* 设备号个数 */
28  #define GPIOLED_NAME     "gpioled"      /* 名字     */
29  #define LEDOFF           0              /* 关灯     */
30  #define LEDON            1              /* 开灯     */
31
32  /* gpioled 设备结构体 */
33  struct gpioled_dev{
34      dev_t devid;              /* 设备号          */
35      struct cdev cdev;         /* cdev            */
36      struct class *class;      /* 类              */
37      struct device *device;    /* 设备            */
38      int major;                /* 主设备号        */
39      int minor;                /* 次设备号        */
40      struct device_node  *nd; /* 设备节点         */
41      int led_gpio;             /* led 所使用的 GPIO 编号       */
42      int dev_stats;            /*使用状态，0，设备未使用;>0,设备已经被使用 */
43      spinlock_t lock;          /* 自旋锁          */
44  };
45
46  static struct gpioled_dev gpioled;  /* led 设备 */
47
48
49  /*
50   * @description   : 打开设备
51   * @param - inode : 传递给驱动的 inode
52   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
53   *                  一般在 open 的时候将 private_data 指向设备结构体。
54   * @return        : 0 成功;其他 失败
55   */
56  static int led_open(struct inode *inode, struct file *filp)
57  {
```

```
58      unsigned long flags;
59      filp->private_data = &gpioled;              /* 设置私有数据 */
60
61      spin_lock_irqsave(&gpioled.lock, flags);  /* 上锁         */
62      if (gpioled.dev_stats) {                    /* 如果设备被使用了 */
63          spin_unlock_irqrestore(&gpioled.lock, flags);/* 解锁 */
64          return -EBUSY;
65      }
66      gpioled.dev_stats++;    /* 如果设备没有打开，那么就标记已经打开了 */
67      spin_unlock_irqrestore(&gpioled.lock, flags);/* 解锁 */
68
69      return 0;
70  }
.......
116 /*
117  * @description   : 关闭/释放设备
118  * @param - filp  : 要关闭的设备文件(文件描述符)
119  * @return        : 0 成功;其他 失败
120  */
121 static int led_release(struct inode *inode, struct file *filp)
122 {
123     unsigned long flags;
124     struct gpioled_dev *dev = filp->private_data;
125
126     /* 关闭驱动文件的时候将 dev_stats 减 1 */
127     spin_lock_irqsave(&dev->lock, flags);      /* 上锁 */
128     if (dev->dev_stats) {
129         dev->dev_stats--;
130     }
131     spin_unlock_irqrestore(&dev->lock, flags);/* 解锁 */
132
133     return 0;
134 }
135
136 /* 设备操作函数 */
137 static struct file_operations gpioled_fops = {
138     .owner = THIS_MODULE,
139     .open = led_open,
140     .read = led_read,
141     .write = led_write,
142     .release =  led_release,
143 };
144
```

```
145 /*
146  * @description   : 驱动出口函数
147  * @param         : 无
148  * @return        : 无
149  */
150 static int __init led_init(void)
151 {
152     int ret = 0;
153     const char *str;
154 
155     /*  初始化自旋锁 */
156     spin_lock_init(&gpioled.lock);
157 
......
258 }
259 
260 /*
261  * @description   : 驱动出口函数
262  * @param         : 无
263  * @return        : 无
264  */
265 static void __exit led_exit(void)
266 {
267     /* 注销字符设备驱动 */
268     cdev_del(&gpioled.cdev);     /*  删除 cdev */
269     unregister_chrdev_region(gpioled.devid, GPIOLED_CNT);
270     device_destroy(gpioled.class, gpioled.devid);/* 注销设备 */
271     class_destroy(gpioled.class);/* 注销类 */
272     gpio_free(gpioled.led_gpio); /* 释放 GPIO */
273 }
274 
275 module_init(led_init);
276 module_exit(led_exit);
277 MODULE_LICENSE("GPL");
278 MODULE_AUTHOR("ALIENTEK");
279 MODULE_INFO(intree, "Y");
```

第 42 行，dev_stats 表示设备状态，如果为 0 的话表示设备还没有被使用，如果大于 0 的话就表示设备已经被使用了。

第 43 行，定义自旋锁变量 lock。

第 60~67 行，使用自旋锁实现对设备的互斥访问，第 61 行调用 spin_lock_irqsave 函数获取锁，为了考虑到驱动兼容性，这里并没有使用 spin_lock 函数来获取锁。第 62 行判断 dev_stats 是否大于 0，如果是的话表示设备已经被使用了，那么就调用 spin_unlock_irqrestore 函数释放锁，并且返回-EBUSY。如果设备没有被使用的话就在第 66 行将 dev_stats 加 1，表

示设备要被使用了，然后调用 spin_unlock_irqrestore 函数释放锁。自旋锁的工作就是保护 dev_stats 变量，真正实现对设备互斥访问的是 dev_stats。

第 126~131 行，在 release 函数中将 dev_stats 减 1，表示设备被释放了，可以被其他的应用程序使用。将 dev_stats 减 1 的时候需要自旋锁对其进行保护。

第 156 行，在驱动入口函数 led_init 中调用 spin_lock_init 函数初始化自旋锁。

3、编写测试 APP

测试 APP 使用 28.1.1 小节中的 atomicApp.c 即可，将 7_atomic 中的 atomicApp.c 文件到本例程中，并将 atomicApp.c 重命名为 spinlockApp.c 即可。

28.2.2 运行测试

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 spinlock.o，Makefile 内容如下所示：

示例代码 28.2.2.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
.......
4  obj-m := spinlock.o
.......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 spinlock.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“spinlock.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 atomicApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc spinlockApp.c -o spinlockApp
```

编译成功以后就会生成 spinlockApp 这个应用程序。

3、运行测试

将上一小节编译出来的 spinlock.ko 和 spinlockApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 spinlock.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe spinlock.ko    //加载驱动
```

驱动加载成功以后就可以使用 spinlockApp 软件测试驱动是否工作正常，测试方法和 28.1.2 小节中一样，先输入如下命令让 spinlockAPP 软件模拟占用 25S 的 LED 灯：

```
./spinlockApp /dev/gpioled 1&          //打开 LED 灯
```

紧接着再输入如下命令关闭 LED 灯：

```
./spinlockApp   /dev/gpioled 0         //关闭 LED 灯
```

看一下能不能关闭 LED 灯，驱动正常工作的话并不会马上关闭 LED 灯，会提示你“file /dev/gpioled open failed!”，必须等待第一个 spinlockApp 软件运行完成(25S 计时结束)才可以再

次操作 LED 灯。

如果要卸载驱动的话输入如下命令即可：

```
rmmod spinlock.ko
```

28.3 信号量实验

本节我们使用信号量来实现一次只能有一个应用程序访问 LED 灯，信号量可以导致休眠，因此信号量保护的临界区没有运行时间限制，可以在驱动的 open 函数申请信号量，然后在 release 函数中释放信号量。但是信号量不能用在中断中，本节实验我们不会在中断中使用信号量。

28.3.1 实验程序编写

1、修改设备树文件

本章实验是在上一节实验的基础上完成的，同样不需要对设备树做任何的修改。

2、LED 驱动修改

本节实验在第上一节实验驱动文件 spinlock.c 的基础上修改而来。新建名为“9_semaphore”的文件夹，然后在 9_semaphore 文件夹里面创建 vscode 工程，工作区命名为“semaphore”。将 8_spinlock 实验中的 spinlock.c 复制到 9_semaphore 文件夹中，并且重命名为 semaphore.c。将原来使用到自旋锁的地方换为信号量即可，其他的内容基本不变，完成以后的 semaphore.c 文件内容如下所示(有省略)：

示例代码 28.3.1.1 semaphore.c 文件代码

```
1  #include <linux/types.h>
.....
14 #include <linux/semaphore.h>
15 #include <asm/mach/map.h>
16 #include <asm/uaccess.h>
17 #include <asm/io.h>
18 /***************************************************************
19 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
20 文件名      : gpioled.c
21 作者        : 正点原子 Linux 团队
22 版本        : V1.0
23 描述        : 信号量实验，使用信号量来实现对实现设备的互斥访问。
24 其他        : 无
25 论坛        : www.openedv.com
26 日志        : 初版 V1.0 2021/1/4 正点原子 Linux 团队创建
27 ***************************************************************/
28 #define GPIOLED_CNT          1          /* 设备号个数    */
29 #define GPIOLED_NAME         "gpioled"  /* 名字          */
30 #define LEDOFF               0          /* 关灯          */
31 #define LEDON                1          /* 开灯          */
32
33 /* gpioled 设备结构体 */
```

```
34  struct gpioled_dev{
35      dev_t devid;            /* 设备号          */
36      struct cdev cdev;       /* cdev            */
37      struct class *class;    /* 类              */
38      struct device *device;  /* 设备            */
39      int major;              /* 主设备号        */
40      int minor;              /* 次设备号        */
41      struct device_node  *nd; /* 设备节点       */
42      int led_gpio;           /* led 所使用的 GPIO 编号      */
43      struct semaphore sem;   /* 信号量          */
44  };
45
46  static struct gpioled_dev gpioled;  /* led 设备 */
47
48
49  /*
50   * @description   : 打开设备
51   * @param - inode : 传递给驱动的 inode
52   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
53   *                  一般在 open 的时候将 private_data 指向设备结构体。
54   * @return        : 0 成功;其他 失败
55   */
56  static int led_open(struct inode *inode, struct file *filp)
57  {
58      filp->private_data = &gpioled; /* 设置私有数据 */
59
60      /* 获取信号量 */
61      if (down_interruptible(&gpioled.sem)) { /* 获取信号量,进入休眠状态的
                          进程可以被信号打断，这时 count 就为 0 */
62          return -ERESTARTSYS;
63      }
64  #if 0
65      down(&gpioled.sem);     /* 不能被信号打断 */
66  #endif
67
68      return 0;
69  }
.....
120 static int led_release(struct inode *inode, struct file *filp)
121 {
122     struct gpioled_dev *dev = filp->private_data;
123
124     up(&dev->sem);  /* 释放信号量，信号量 count 值加 1 */
```

```
125     return 0;
126 }
127
128 /* 设备操作函数 */
129 static struct file_operations gpioled_fops = {
130     .owner = THIS_MODULE,
131     .open = led_open,
132     .read = led_read,
133     .write = led_write,
134     .release =  led_release,
135 };
136
137 /*
138  * @description   : 驱动出口函数
139  * @param         : 无
140  * @return        : 无
141  */
142 static int __init led_init(void)
143 {
144     int ret = 0;
145     const char *str;
146
147     /* 初始化信号量 */
148     sema_init(&gpioled.sem, 1);
.......
250 }
251
252 /*
253  * @description   : 驱动出口函数
254  * @param         : 无
255  * @return        : 无
256  */
257 static void __exit led_exit(void)
258 {
259     /* 注销字符设备驱动 */
260     cdev_del(&gpioled.cdev);        /*  删除 cdev */
261     unregister_chrdev_region(gpioled.devid, GPIOLED_CNT);
262     device_destroy(gpioled.class, gpioled.devid);/* 注销设备 */
263     class_destroy(gpioled.class);   /* 注销类 */
264     gpio_free(gpioled.led_gpio);    /* 释放 GPIO */
265 }
266
267 module_init(led_init);
```

```
268 module_exit(led_exit);
269 MODULE_LICENSE("GPL");
270 MODULE_AUTHOR("ALIENTEK");
271 MODULE_INFO(intree, "Y");
```

第 14 行，要使用信号量必须添加<linux/semaphore.h>头文件。

第 43 行，在设备结构体中添加一个信号量成员变量 sem。

第 60~66 行，在 open 函数中申请信号量，可以使用 down 函数，也可以使用 down_interruptible 函数。如果信号量值大于等于 1 就表示可用，那么应用程序就会开始使用 LED 灯。如果信号量值为 0 就表示应用程序不能使用 LED 灯，此时应用程序就会进入到休眠状态。等到信号量值大于 1 的时候应用程序就会唤醒，申请信号量，获取 LED 灯使用权。

第 124 行，在 release 函数中调用 up 函数释放信号量，这样其他因为没有得到信号量而进入休眠状态的应用程序就会唤醒，获取信号量。

第 148 行，在驱动入口函数中调用 sema_init 函数初始化信号量 sem 的值为 1，相当于 sem 是个二值信号量。

总结一下，当信号量 sem 为 1 的时候表示 LED 灯还没有被使用，如果应用程序 A 要使用 LED 灯，先调用 open 函数打开/dev/gpioled，这个时候会获取信号量 sem，获取成功以后 sem 的值减 1 变为 0。如果此时应用程序 B 也要使用 LED 灯，调用 open 函数打开/dev/gpioled 就会因为信号量无效(值为 0)而进入休眠状态。当应用程序 A 运行完毕，调用 close 函数关闭/dev/gpioled 的时候就会释放信号量 sem，此时信号量 sem 的值就会加 1，变为 1。信号量 sem 再次有效，表示其他应用程序可以使用 LED 灯了，此时在休眠状态的应用程序 A 就会获取到信号量 sem，获取成功以后就开始使用 LED 灯。

3、编写测试 APP

测试 APP 使用 28.1.1 小节中的 atomicApp.c 即可，将 7_atomic 中的 atomicApp.c 文件到本例程中，并将 atomicApp.c 重命名为 semaApp.c 即可。

28.3.2 运行测试

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 semaphore.o，Makefile 内容如下所示：

示例代码 28.1.2.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := semaphore.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 semaphore.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“semaphore.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 semaApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc semaApp.c -o semaApp
```

编译成功以后就会生成 semaApp 这个应用程序。

3、运行测试

将上一小节编译出来的 semaphore.ko 和 semaApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 semaphore.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe semaphore.ko//加载驱动
```

驱动加载成功以后就可以使用 semaApp 软件测试驱动是否工作正常，测试方法和 28.1.2 小节中一样，先输入如下命令让 semaApp 软件模拟占用 25S 的 LED 灯：

```
./ semaApp /dev/gpioled 1&      //打开 LED 灯
```

紧接着再输入如下命令关闭 LED 灯：

```
./ semaApp /dev/gpioled 0&      //关闭 LED 灯
```

注意两个命令都是运行在后台，第一条命令先获取到信号量，因此可以操作 LED 灯，将 LED 灯打开，并且占有 25S。第二条命令因为获取信号量失败而进入休眠状态，等待第一条命令运行完毕并释放信号量以后才拥有 LED 灯使用权，将 LED 灯关闭，运行结果如图 28.3.2.1 所示：

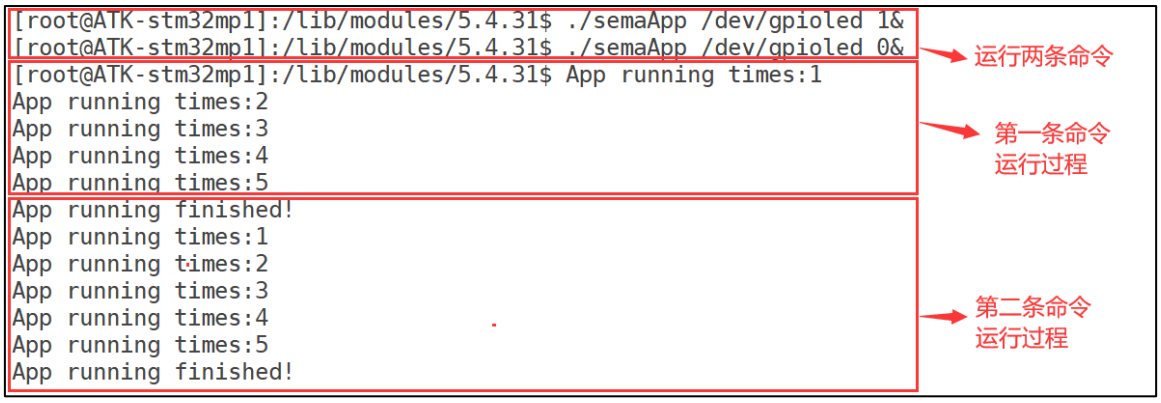


图 28.3.2.1 命令运行过程

如果要卸载驱动的话输入如下命令即可：

```
rmmod semaphore.ko
```

28.4 互斥体实验

前面我们使用原子操作、自旋锁和信号量实现了对 LED 灯的互斥访问，但是最适合互斥的就是互斥体 mutex 了。本节我们来学习一下如何使用 mutex 实现对 LED 灯的互斥访问。

28.4.1 实验程序编写

1、修改设备树文件

本章实验是在上一节实验的基础上完成的，同样不需要对设备树做任何的修改。

2、LED 驱动修改

本节实验在第上一节实验驱动文件 semaphore.c 的基础上修改而来。新建名为“10_mutex”

的文件夹，然后在 10_mutex 文件夹里面创建 vscode 工程，工作区命名为“mutex”。将 9_semaphore 实验中的 semaphore.c 复制到 10_mutex 文件夹中，并且重命名为 mutex.c。将原来使用到信号量的地方换为 mutex 即可，其他的内容基本不变，完成以后的 mutex.c 文件内容如下所示(有省略)：

示例代码 28.4.1.1 mutex.c 文件代码

```
1  #include <linux/types.h>
.....
18 /***************************************************************
19 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
20 文件名   : gpioled.c
21 作者     : 正点原子 Linux 团队
22 版本     : V1.0
23 描述     : 互斥体实验，使用互斥体来实现对实现设备的互斥访问。
24 其他     : 无
25 论坛     : www.openedv.com
26 日志     : 初版 V1.0 2021/1/4 正点原子 Linux 团队创建
27 ***************************************************************/
28 #define GPIOLED_CNT      1          /* 设备号个数    */
29 #define GPIOLED_NAME     "gpioled"  /* 名字          */
30 #define LEDOFF           0          /* 关灯          */
31 #define LEDON            1          /* 开灯          */
32
33 /* gpioled 设备结构体 */
34 struct gpioled_dev{
35     dev_t devid;                    /* 设备号        */
36     struct cdev cdev;               /* cdev          */
37     struct class *class;            /* 类            */
38     struct device *device;          /* 设备          */
39     int major;                      /* 主设备号      */
40     int minor;                      /* 次设备号      */
41     struct device_node  *nd;        /* 设备节点      */
42     int led_gpio;                   /* led 所使用的 GPIO 编号        */
43     struct mutex lock;              /* 互斥体        */
44 };
45
46 static struct gpioled_dev gpioled;  /* led 设备 */
47
48
49 /*
50  * @description   : 打开设备
51  * @param - inode : 传递给驱动的 inode
52  * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
53  *                  一般在 open 的时候将 private_data 指向设备结构体。
54  * @return        : 0 成功;其他 失败
```

```
55  */
56  static int led_open(struct inode *inode, struct file *filp)
57  {
58      filp->private_data = &gpioled; /* 设置私有数据 */
59
60      /* 获取互斥体,可以被信号打断 */
61      if (mutex_lock_interruptible(&gpioled.lock)) {
62          return -ERESTARTSYS;
63      }
64  #if 0
65      mutex_lock(&gpioled.lock);  /* 不能被信号打断 */
66  #endif
67
68      return 0;
69  }
......
120 static int led_release(struct inode *inode, struct file *filp)
121 {
122     struct gpioled_dev *dev = filp->private_data;
123
124     /* 释放互斥锁 */
125     mutex_unlock(&dev->lock);
126     return 0;
127 }
128
129 /* 设备操作函数 */
130 static struct file_operations gpioled_fops = {
131     .owner = THIS_MODULE,
132     .open = led_open,
133     .read = led_read,
134     .write = led_write,
135     .release =  led_release,
136 };
137
138 /*
139  * @description   : 驱动出口函数
140  * @param         : 无
141  * @return        : 无
142  */
143 static int __init led_init(void)
144 {
145     int ret = 0;
146     const char *str;
```

```
147
148     /* 初始化互斥体 */
149     mutex_init(&gpioled.lock);
......
251 }
252
253 /*
254  * @description   : 驱动出口函数
255  * @param         : 无
256  * @return        : 无
257  */
258 static void __exit led_exit(void)
259 {
260     /* 注销字符设备驱动 */
261     cdev_del(&gpioled.cdev);        /*  删除 cdev */
262     unregister_chrdev_region(gpioled.devid, GPIOLED_CNT);
263     device_destroy(gpioled.class, gpioled.devid);/* 注销设备 */
264     class_destroy(gpioled.class);   /* 注销类 */
265     gpio_free(gpioled.led_gpio);    /* 释放 GPIO */
266 }
267
268 module_init(led_init);
269 module_exit(led_exit);
270 MODULE_LICENSE("GPL");
271 MODULE_AUTHOR("ALIENTEK");
272 MODULE_INFO(intree, "Y");
```

第 43 行，定义互斥体 lock。

第 60~66 行，在 open 函数中调用 mutex_lock_interruptible 或者 mutex_lock 获取 mutex，成功的话就表示可以使用 LED 灯，失败的话就会进入休眠状态，和信号量一样。

第 125 行，在 release 函数中调用 mutex_unlock 函数释放 mutex，这样其他应用程序就可以获取 mutex 了。

第 149 行，在驱动入口函数中调用 mutex_init 初始化 mutex。

互斥体和二值信号量类似，只不过互斥体是专门用于互斥访问的。

3、编写测试 APP

测试 APP 使用 28.1.1 小节中的 atomicApp.c 即可，将 7_atomic 中的 atomicApp.c 文件到本例程中，并将 atomicApp.c 重命名为 mutexApp.c 即可。

28.4.2 运行测试

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 mutex.o，Makefile 内容如下所示：

示例代码 28.4.2.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31

......
4  obj-m := mutex.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 mutex.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“mutex.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 mutexApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc mutexApp.c -o mutexApp
```

编译成功以后就会生成 mutexApp 这个应用程序。

3、运行测试

将上一小节编译出来的 mutex.ko 和 mutexApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 mutex.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe mutex.ko       //加载驱动
```

驱动加载成功以后就可以使用 mutexApp 软件测试驱动是否工作正常，测试方法和 28.3.2 中测试信号量的方法一样。

如果要卸载驱动的话输入如下命令即可：

```
rmmod mutex.ko
```

第二十九章 Linux 按键输入实验

在前几章我们都是使用的 GPIO 输出功能，还没有用过 GPIO 输入功能，本章我们就来学习一下如果在 Linux 下编写 GPIO 输入驱动程序。正点原子的 STM32MP1 开发板上有三个按键，我们就使用这些按键来完成 GPIO 输入驱动程序，同时利用第二十七章讲的原子操作来对按键值进行保护。

29.1 Linux 下按键驱动原理

按键驱动和 LED 驱动原理上来讲基本都是一样的，都是操作 GPIO，只不过一个是读取 GPIO 的高低电平，一个是从 GPIO 输出高低电平。本章我们实现按键输入，在驱动程序中使用一个整形变量来表示按键值，应用程序通过 read 函数来读取按键值，判断按键有没有按下。在这里，这个保存按键值的变量就是个共享资源，驱动程序要向其写入按键值，应用程序要读取按键值。所以我们要对其进行保护，对于整形变量而言我们首选的就是原子操作，使用原子操作对变量进行赋值以及读取。Linux 下的按键驱动原理很简单，接下来开始编写驱动。

注意，本章例程只是为了演示 Linux 下 GPIO 输入驱动的编写，实际中的按键驱动并不会采用本章中所讲解的方法，Linux 下的 input 子系统专门用于输入设备！

29.2 硬件原理图分析

开发板上有三个按键:KEY0、KEY1 和 WK_UP，原理图如图 29.2.1 所示：

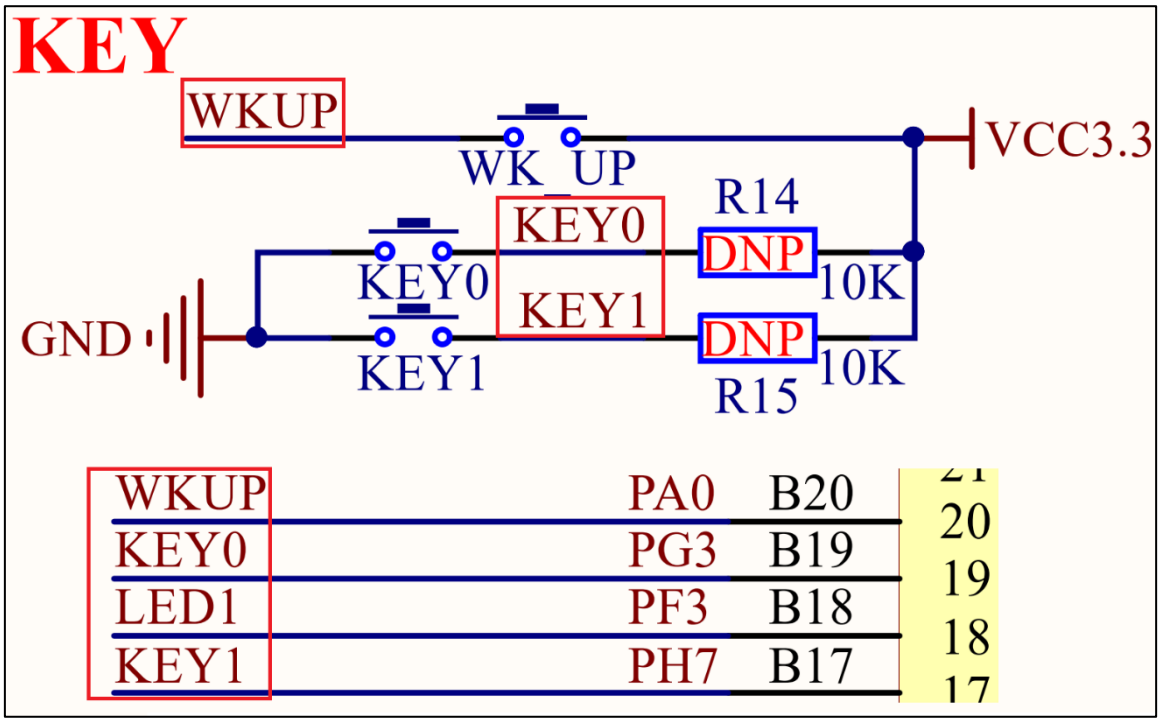


图 29.2.1 按键原理图

从图 29.2.1 可以看出，按键 KEY0 、KEY1 和 WK_UP 这三个按键分别连接到正点原子 STM32MP1 开发板的 PG3、PH7 和 PA0 这三个 IO 上。本节我们只用到 KEY0 这个按键，从图 29.2.1 可以看出，KEY0 接了一个 10K 的上拉电阻，因此 KEY0 没有按下的时候 PG3 应该是高电平，当 KEY0 按下以后 PG3 就是低电平。

29.3 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→11_key

29.3.1 修改设备树文件

在根节点“/”下创建 KEY 节点，节点名为“key”，节点内容如下：

示例代码 29.3.1.1 创建 KEY 节点

```
1  key {
2      compatible = "alientek,key";
3      status = "okay";
4      key-gpio = <&gpiog 3 GPIO_ACTIVE_LOW>;
5  };
```

第 4 行，key-gpio 属性指定了 KEY 所使用的 GPIO。

29.3.2 按键驱动程序编写

设备树准备好以后就可以编写驱动程序了，新建名为“11_key”的文件夹，然后在 11_key 文件夹里面创建 vscode 工程，工作区命名为“key”。工程创建好以后新建 key.c 文件，在 key.c 里面输入如下内容：

示例代码 29.3.2.1 key.c 文件代码

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <linux/of.h>
12 #include <linux/of_address.h>
13 #include <linux/of_gpio.h>
14 #include <linux/semaphore.h>
15 #include <asm/mach/map.h>
16 #include <asm/uaccess.h>
17 #include <asm/io.h>
18 /***************************************************************
19 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
20 文件名      : key.c
21 作者        : 正点原子 Linux 团队
22 版本        : V1.0
23 描述        : Linux 按键输入驱动实验
24 其他        : 无
25 论坛        : www.openedv.com
26 日志        : 初版 V1.0 2021/01/5 正点原子 Linux 团队创建
27 ***************************************************************/
28 #define KEY_CNT      1      /* 设备号个数   */
29 #define KEY_NAME     "key"  /* 名字         */
30
```

```
31  /* 定义按键值 */
32  #define KEY0VALUE    0XF0    /* 按键值        */
33  #define INVAKEY      0X00    /* 无效的按键值  */
34
35  /* key 设备结构体 */
36  struct key_dev{
37      dev_t devid;             /* 设备号       */
38      struct cdev cdev;        /* cdev         */
39      struct class *class;     /* 类           */
40      struct device *device;   /* 设备         */
41      int major;               /* 主设备号     */
42      int minor;               /* 次设备号     */
43      struct device_node  *nd; /* 设备节点     */
44      int key_gpio;            /* key 所使用的 GPIO 编号        */
45      atomic_t keyvalue;       /* 按键值       */
46  };
47
48  static struct key_dev keydev;/* key 设备 */
49
50  /*
51   * @description   : 初始化按键 IO，open 函数打开驱动的时候
52   *                  初始化按键所使用的 GPIO 引脚。
53   * @param         : 无
54   * @return        : 无
55   */
56  static int keyio_init(void)
57  {
58      int ret;
59      const char *str;
60
61      /* 设置 LED 所使用的 GPIO */
62      /* 1、获取设备节点：keydev */
63      keydev.nd = of_find_node_by_path("/key");
64      if(keydev.nd == NULL) {
65          printk("keydev node not find!\r\n");
66          return -EINVAL;
67      }
68
69      /* 2.读取 status 属性 */
70      ret = of_property_read_string(keydev.nd, "status", &str);
71      if(ret < 0)
72          return -EINVAL;
73
```

```
74      if (strcmp(str, "okay"))
75          return -EINVAL;
76
77      /* 3、获取 compatible 属性值并进行匹配 */
78      ret = of_property_read_string(keydev.nd, "compatible", &str);
79      if(ret < 0) {
80          printk("keydev: Failed to get compatible property\n");
81          return -EINVAL;
82      }
83
84      if (strcmp(str, "alientek,key")) {
85          printk("keydev: Compatible match failed\n");
86          return -EINVAL;
87      }
88
89      /* 4、 获取设备树中的 gpio 属性，得到 KEY0 所使用的 KYE 编号 */
90      keydev.key_gpio = of_get_named_gpio(keydev.nd, "key-gpio", 0);
91      if(keydev.key_gpio < 0) {
92          printk("can't get key-gpio");
93          return -EINVAL;
94      }
95      printk("key-gpio num = %d\r\n", keydev.key_gpio);
96
97      /* 5.向 gpio 子系统申请使用 GPIO */
98      ret = gpio_request(keydev.key_gpio, "KEY0");
99      if (ret) {
100         printk(KERN_ERR "keydev: Failed to request key-gpio\n");
101         return ret;
102     }
103
104     /* 6、设置 PG3 输入模式 */
105     ret = gpio_direction_input(keydev.key_gpio);
106     if(ret < 0) {
107         printk("can't set gpio!\r\n");
108         return ret;
109     }
110     return 0;
111 }
112
113 /*
114  * @description   : 打开设备
115  * @param - inode : 传递给驱动的 inode
116  * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
```

```
117  *                  一般在 open 的时候将 private_data 指向设备结构体。
118  * @return         : 0 成功;其他 失败
119  */
120 static int key_open(struct inode *inode, struct file *filp)
121 {
122     int ret = 0;
123     filp->private_data = &keydev;   /* 设置私有数据 */
124 
125     ret = keyio_init();             /* 初始化按键 IO */
126     if (ret < 0) {
127         return ret;
128     }
129 
130     return 0;
131 }
132 
133 /*
134  * @description   : 从设备读取数据
135  * @param - filp  : 要打开的设备文件(文件描述符)
136  * @param - buf   : 返回给用户空间的数据缓冲区
137  * @param - cnt   : 要读取的数据长度
138  * @param - offt  : 相对于文件首地址的偏移
139  * @return        : 读取的字节数，如果为负值，表示读取失败
140  */
141 static ssize_t key_read(struct file *filp, char __user *buf,
                            size_t cnt, loff_t *offt)
142 {
143     int ret = 0;
144     int value;
145     struct key_dev *dev = filp->private_data;
146 
147     if (gpio_get_value(dev->key_gpio) == 0) {     /* key0 按下    */
148         while(!gpio_get_value(dev->key_gpio));    /* 等待按键释放 */
149         atomic_set(&dev->keyvalue, KEY0VALUE);
150     } else {
151         atomic_set(&dev->keyvalue, INVAKEY);      /* 无效的按键值 */
152     }
153 
154     value = atomic_read(&dev->keyvalue);
155     ret = copy_to_user(buf, &value, sizeof(value));
156     return ret;
157 }
158 
```

```
159 /*
160  * @description   : 向设备写数据
161  * @param - filp  : 设备文件，表示打开的文件描述符
162  * @param - buf   : 要写给设备写入的数据
163  * @param - cnt   : 要写入的数据长度
164  * @param - offt  : 相对于文件首地址的偏移
165  * @return        : 写入的字节数，如果为负值，表示写入失败
166  */
167 static ssize_t key_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
168 {
169     return 0;
170 }
171
172 /*
173  * @description   : 关闭/释放设备
174  * @param - filp  : 要关闭的设备文件(文件描述符)
175  * @return        : 0 成功;其他 失败
176  */
177 static int key_release(struct inode *inode, struct file *filp)
178 {
179     struct key_dev *dev = filp->private_data;
180     gpio_free(dev->key_gpio);
181
182     return 0;
183 }
184
185 /* 设备操作函数 */
186 static struct file_operations key_fops = {
187     .owner = THIS_MODULE,
188     .open = key_open,
189     .read = key_read,
190     .write = key_write,
191     .release =  key_release,
192 };
193
194 /*
195  * @description   : 驱动入口函数
196  * @param         : 无
197  * @return        : 无
198  */
199 static int __init mykey_init(void)
200 {
```

```
201     int ret;
202     /* 1、初始化原子变量    */
203     keydev.keyvalue= (atomic_t)ATOMIC_INIT(0);
204
205     /* 2、原子变量初始值为 INVAKEY */
206     atomic_set(&keydev.keyvalue, INVAKEY);
207
208     /* 注册字符设备驱动 */
209     /* 1、创建设备号 */
210     if (keydev.major) {          /*  定义了设备号 */
211         keydev.devid = MKDEV(keydev.major, 0);
212         ret = register_chrdev_region(keydev.devid, KEY_CNT,
                                         KEY_NAME);
213         if(ret < 0) {
214             pr_err("cannot register %s char driver [ret=%d]\n",
KEY_NAME, KEY_CNT);
215             return -EIO;
216         }
217     } else {                        /* 没有定义设备号 */
218         ret = alloc_chrdev_region(&keydev.devid, 0, KEY_CNT,
                                      KEY_NAME); /* 申请设备号 */
219         if(ret < 0) {
220             pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                       KEY_NAME, ret);
221             return -EIO;
222         }
223         keydev.major = MAJOR(keydev.devid); /* 获取分配号的主设备号 */
224         keydev.minor = MINOR(keydev.devid); /* 获取分配号的次设备号 */
225     }
226     printk("keydev major=%d,minor=%d\r\n",keydev.major,
              keydev.minor);
227
228     /* 2、初始化 cdev */
229     keydev.cdev.owner = THIS_MODULE;
230     cdev_init(&keydev.cdev, &key_fops);
231
232     /* 3、添加一个 cdev */
233     cdev_add(&keydev.cdev, keydev.devid, KEY_CNT);
234     if(ret < 0)
235         goto del_unregister;
236
237     /* 4、创建类 */
238     keydev.class = class_create(THIS_MODULE, KEY_NAME);
```

```
239     if (IS_ERR(keydev.class)) {
240         goto del_cdev;
241     }
242 
243     /* 5、创建设备 */
244     keydev.device = device_create(keydev.class, NULL, keydev.devid,
                                      NULL, KEY_NAME);
245     if (IS_ERR(keydev.device)) {
246         goto destroy_class;
247     }
248     return 0;
249 
250 destroy_class:
251     device_destroy(keydev.class, keydev.devid);
252 del_cdev:
253     cdev_del(&keydev.cdev);
254 del_unregister:
255     unregister_chrdev_region(keydev.devid, KEY_CNT);
256     return -EIO;
257 }
258 
259 /*
260  * @description   : 驱动出口函数
261  * @param         : 无
262  * @return        : 无
263  */
264 static void __exit mykey_exit(void)
265 {
266     /* 注销字符设备驱动 */
267     cdev_del(&keydev.cdev);                              /*  删除 cdev */
268     unregister_chrdev_region(keydev.devid, KEY_CNT); /* 注销设备号 */
269 
270     device_destroy(keydev.class, keydev.devid);
271     class_destroy(keydev.class);
272 }
273 
274 module_init(mykey_init);
275 module_exit(mykey_exit);
276 MODULE_LICENSE("GPL");
277 MODULE_AUTHOR("ALIENTEK");
278 MODULE_INFO(intree, "Y");
```

第 36~46 行，结构体 key_dev 为按键的设备结构体，第 45 行的原子变量 keyvalue 用于记录按键值。

第 56~111 行，函数 keyio_init 用于初始化按键，从设备树中获取按键的 gpio 信息，然后设置为输入。这里将按键的初始化代码提取出来，将其作为独立的一个函数有利于提高程序的模块化设计。

第 120~131 行，key_open 函数通过调用 keyio_init 函数来始化按键所使用的 IO，应用程序每次打开按键驱动文件的时候都会初始化一次按键 IO。

第 141~157 行，key_read 函数，应用程序通过 read 函数读取按键值的时候此函数就会执行。第 147 行读取按键 IO 的电平，如果为 0 的话就表示按键按下了，如果按键按下的话第 148 等待按键释放。按键释放以后标记按键值为 0XF0。

第 199~257 行，驱动入口函数，第 206 行调用 atomic_set 函数初始化原子变量默认为无效值。

第 264~272 行，驱动出口函数。

key.c 文件代码很简单，重点就是 key_read 函数读取按键值，要对 keyvalue 进行保护。

29.3.3 编写测试 APP

新建名为 keyApp.c 的文件，然后输入如下所示内容：

示例代码 29.3.3.1 keyApp.c 文件代码

```
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "fcntl.h"
6  #include "stdlib.h"
7  #include "string.h"
8  /***************************************************************
9  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
10 文件名       : keyApp.c
11 作者         : 正点原子 Linux 团队
12 版本         : V1.0
13 描述         : 按键输入测试应用程序
14 其他         : 无
15 使用方法     : ./keyApp /dev/key
16 论坛         : www.openedv.com
17 日志         : 初版 V1.0 2021/01/5 正点原子 Linux 团队创建
18 ***************************************************************/
19
20 /* 定义按键值 */
21 #define KEY0VALUE     0XF0
22 #define INVAKEY       0X00
23
24 /*
25  * @description    : main 主程序
26  * @param - argc   : argv 数组元素个数
27  * @param - argv   : 具体参数
```

```
28  * @return          : 0 成功;其他 失败
29  */
30 int main(int argc, char *argv[])
31 {
32     int fd, ret;
33     char *filename;
34     int keyvalue;
35
36     if(argc != 2){
37         printf("Error Usage!\r\n");
38         return -1;
39     }
40
41     filename = argv[1];
42
43     /* 打开 key 驱动 */
44     fd = open(filename, O_RDWR);
45     if(fd < 0){
46         printf("file %s open failed!\r\n", argv[1]);
47         return -1;
48     }
49
50     /* 循环读取按键值数据！ */
51     while(1) {
52         read(fd, &keyvalue, sizeof(keyvalue));
53         if (keyvalue == KEY0VALUE) {    /* KEY0 */
54             printf("KEY0 Press, value = %#X\r\n", keyvalue);/* 按下 */
55         }
56     }
57
58     ret= close(fd); /* 关闭文件 */
59     if(ret < 0){
60         printf("file %s close failed!\r\n", argv[1]);
61         return -1;
62     }
63     return 0;
64 }
```

第 51~56 行，循环读取/dev/key 文件，也就是循环读取按键值，并且将按键值打印出来。

29.4 运行测试

29.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 key.o，Makefile 内容如下所示：

示例代码 29.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := key.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 key.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“key.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 keyApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc keyApp.c -o keyApp
```

编译成功以后就会生成 keyApp 这个应用程序。

29.4.2 运行测试

将上一小节编译出来的 key.ko 和 keyApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 key.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe key.ko         //加载驱动
```

驱动加载成功以后如下命令来测试：

```
./keyApp  /dev/key
```

输入上述命令以后终端显示如图 29.4.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./keyApp /dev/key
key-gpio num = 99
```

图 29.4.2.1 测试 APP 运行界面

按下开发板上的 KEY0 按键，keyApp 就会获取并且输出按键信息，如图 29.4.2.2 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./keyApp /dev/key
key-gpio num = 99
KEY0 Press, value = 0XF0
KEY0 Press, value = 0XF0
KEY0 Press, value = 0XF0
KEY0 Press, value = 0XF0
KEY0 Press, value = 0XF0
```

图 29.4.2.2 按键运行结果

从图 29.4.2.2 可以看出，当我们按下 KEY0 以后就会打印出“KEY0 Press, value = 0XF0”，表示按键按下。但是大家可能会发现，有时候按下一次 KEY0 但是会输出好几行“KEY0 Press, value = 0XF0”，这是因为我们的代码没有做按键消抖处理。

如果要卸载驱动的话输入如下命令即可：

```
rmmod key.ko
```

第三十章 Linux 内核定时器实验

定时器是我们最常用到的功能，一般用来完成定时功能，本章我们就来学习一下 Linux 内核提供的定时器 API 函数，通过这些定时器 API 函数我们可以完成很多要求定时的应用。Linux 内核也提供了短延时函数，比如微秒、纳秒、毫秒延时函数，本章我们就来学习一下这些和时间有关的功能。

30.1 Linux 时间管理和内核定时器简介

30.1.1 内核时间管理简介

学习过 UCOS 或 FreeRTOS 的同学应该知道，UCOS 或 FreeRTOS 是需要一个硬件定时器提供系统时钟，一般使用 Systick 作为系统时钟源。同理，Linux 要运行，也是需要一个系统时钟的，至于这个系统时钟是由哪个定时器提供的，笔者没有去研究过 Linux 内核，但是在 Cortex-A7 内核中有个通用定时器，在《Cortex-A7 Technical ReferenceManua.pdf》的“9:Generic Timer”章节有简单的讲解，关于这个通用定时器的详细内容，可以参考《ARM ArchitectureReference Manual ARMv7-A and ARMv7-R edition.pdf》的“chapter B8 The Generic Timer”章节。这个通用定时器是可选的，按照笔者学习 FreeRTOS 和 STM32 的经验，猜测 Linux 会将这个通用定时器作为 Linux 系统时钟源(前提是 SOC 得选配这个通用定时器)。具体是怎么做的笔者没有深入研究过，这里仅仅是猜测！不过对于我们 Linux 驱动编写者来说，不需要深入研究这些具体的实现，只需要掌握相应的 API 函数即可，除非你是内核编写者或者内核爱好者。

Linux 内核中有大量的函数需要时间管理，比如周期性的调度程序、延时程序、对于我们驱动编写者来说最常用的定时器。硬件定时器提供时钟源，时钟源的频率可以设置， 设置好以后就周期性的产生定时中断，系统使用定时中断来计时。中断周期性产生的频率就是系统频率，也叫做节拍率(tick rate)(有的资料也叫系统频率)，比如 100Hz、1000Hz 等等说的就是系统节拍率。系统节拍率是可以设置的，单位是 Hz，我们在编译 Linux 内核的时候可以通过图形化界面设置系统节拍率，按照如下路径打开配置界面：

```
-> Kernel Features
    -> Timer frequency (<choice> [=y])
```

选中“Timer frequency”，打开以后如图 30.1.1.1 所示：

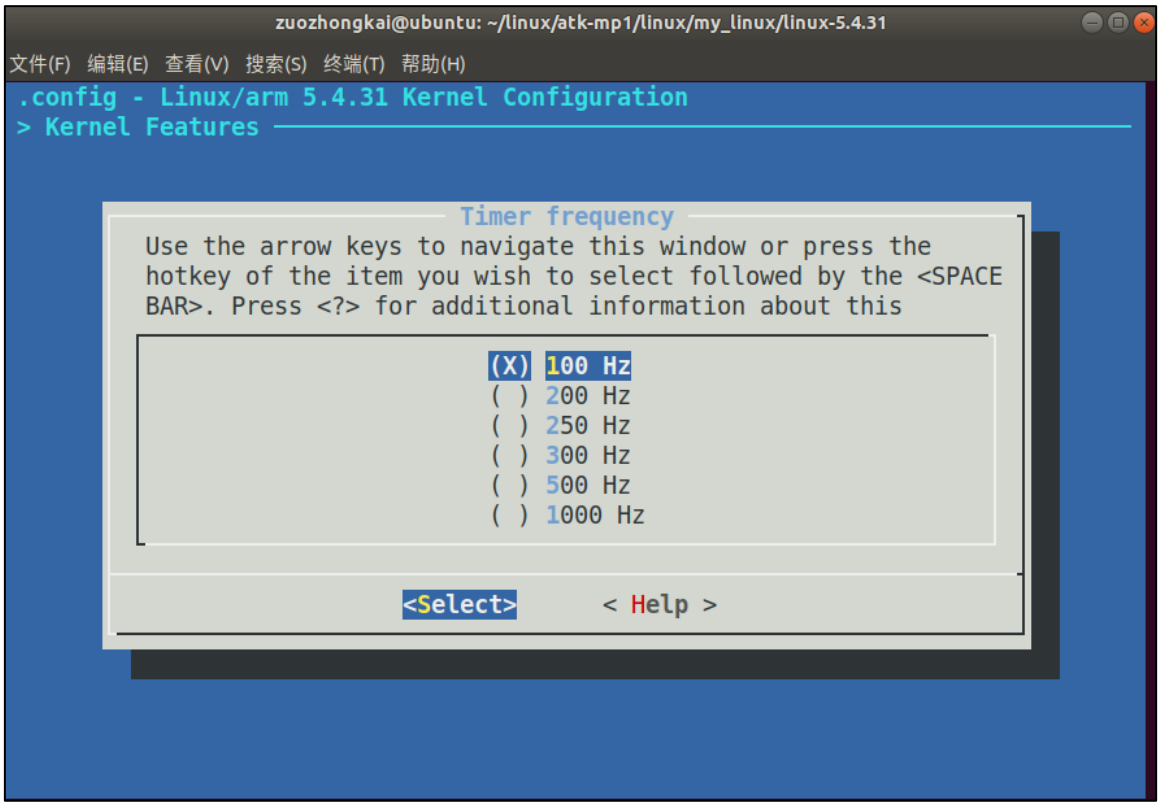


图 30.1.1.1 系统节拍率设置

从图 30.1.1.1 可以看出，可选的系统节拍率为 100Hz、200Hz、250Hz、300Hz、500Hz 和 1000Hz，默认情况下选择 100Hz。设置好以后打开 Linux 内核源码根目录下的.config 文件，在此文件中有如图 30.1.1.2 所示定义：

图 30.1.1.2 系统节拍率

图 30.1.1.2 中的 CONFIG_HZ 为 100，Linux 内核会使用 CONFIG_HZ 来设置自己的系统时钟。打开文件 include/asm-generic/param.h，有如下内容：

示例代码 30.1.1.1 include/asm-generic/param.h 文件代码段

```
6 # undef HZ
7 # define HZ              CONFIG_HZ
8 # define USER_HZ         100
9 # define CLOCKS_PER_SEC  (USER_HZ)
```

第 7 行定义了一个宏 HZ，宏 HZ 就是 CONFIG_HZ，因此 HZ=100，我们后面编写 Linux 驱动的时候会常常用到 HZ，因为 HZ 表示一秒的节拍数，也就是频率。

大多数初学者看到系统节拍率默认为 100Hz 的时候都会有疑问，怎么这么小？100Hz 是可选的节拍率里面最小的。为什么不选择大一点的呢？这里就引出了一个问题：高节拍率和低节拍率的优缺点：

①、高节拍率会提高系统时间精度，如果采用 100Hz 的节拍率，时间精度就是 10ms，采用 1000Hz 的话时间精度就是 1ms，精度提高了 10 倍。高精度时钟的好处有很多，对于那些对时间要求严格的函数来说，能够以更高的精度运行，时间测量也更加准确。

②、高节拍率会导致中断的产生更加频繁，频繁的中断会加剧系统的负担，1000Hz 和 100Hz 的系统节拍率相比，系统要花费 10 倍的“精力”去处理中断。中断服务函数占用处理器的时间增加，但是现在的处理器性能都很强大，所以采用 1000Hz 的系统节拍率并不会增加太大的负载压力。根据自己的实际情况，选择合适的系统节拍率，本教程我们全部采用默认的 100Hz 系统节拍率。

Linux 内核使用全局变量 jiffies 来记录系统从启动以来的系统节拍数，系统启动的时候会将 jiffies 初始化为 0，jiffies 定义在文件 include/linux/jiffies.h 中，定义如下：

示例代码 30.1.1.2 include/jiffies.h 文件代码段

```
80 extern u64 __cacheline_aligned_in_smp jiffies_64;
81 extern unsigned long volatile __cacheline_aligned_in_smp
__jiffy_arch_data jiffies;
```

第 80 行，定义了一个 64 位的 jiffies_64。

第 81 行，定义了一个 unsigned long 类型的 32 位的 jiffies。

jiffies_64 和 jiffies 其实是同一个东西，jiffies_64 用于 64 位系统，而 jiffies 用于 32 位系统。为了兼容不同的硬件，jiffies 其实就是 jiffies_64 的低 32 位，jiffies_64 和 jiffies 的结构如图 30.1.1.3 所示：

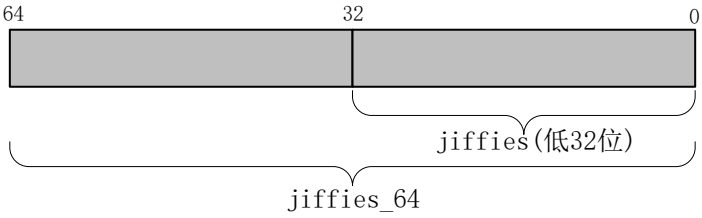


图 30.1.1.3 jiffies_64 和 jiffies 结构图

当我们访问 jiffies 的时候其实访问的是 jiffies_64 的低 32 位，使用 get_jiffies_64 这个函数可以获取 jiffies_64 的值。在 32 位的系统上读取的是 jiffies，在 64 位的系统上 jiffes 和 jiffies_64 表示同一个变量，因此也可以直接读取 jiffies 的值。所以不管是 32 位的系统还是 64 位系统，都可以使用 jiffies。

前面说了 HZ 表示每秒的节拍数，jiffies 表示系统运行的 jiffies 节拍数，所以 jiffies/HZ 就是系统运行时间，单位为秒。不管是 32 位还是 64 位的 jiffies，都有溢出的风险，溢出以后会重新从 0 开始计数，相当于绕回来了，因此有些资料也将这个现象也叫做绕回。假如 HZ 为最大值 1000 的时候，32 位的 jiffies 只需要 49.7 天就发生了绕回，对于 64 位的 jiffies 来说大概需要 5.8 亿年才能绕回，因此 jiffies_64 的绕回忽略不计。处理 32 位 jiffies 的绕回显得尤为重要，Linux 内核提供了如表 30.1.1.1 所示的几个 API 函数来处理绕回。

函数	描述
time_after(unkown, known)	unkown 通常为 jiffies，known 通常是需要对比的值。
time_before(unkown, known)	
time_after_eq(unkown, known)	
time_before_eq(unkown, known)	

表 30.1.1.1 处理绕回的 API 函数

如果 unkown 超过 known 的话，time_after 函数返回真，否则返回假。如果 unkown 没有超过 known 的话 time_before 函数返回真，否则返回假。time_after_eq 函数和 time_after 函数类似，只是多了判断等于这个条件。同理，time_before_eq 函数和 time_before 函数也类似。比如我们要判断某段代码执行时间有没有超时，此时就可以使用如下所示代码：

示例代码 30.1.1.3 使用 jiffies 判断超时

```
1  unsigned long timeout;
2  timeout = jiffies + (2 * HZ);    /* 超时的时间点 */
3
4  /************************************
5    具体的代码
6   ************************************/
7
8  /* 判断有没有超时 */
9  if(time_before(jiffies, timeout)) {
10     /* 超时未发生 */
11 } else {
12     /* 超时发生 */
13 }
```

timeout 就是超时时间点，比如我们要判断代码执行时间是不是超过了 2 秒，那么超时时间点就是 jiffies+(2*HZ)，如果 jiffies 大于 timeout 那就表示超时了，否则就是没有超时。第 4~6 行

就是具体的代码段。第 9 行通过函数 time_before 来判断 jiffies 是否小于 timeout，如果小于的话就表示没有超时。

为了方便开发，Linux 内核提供了几个 jiffies 和 ms、us、ns 之间的转换函数，如表 30.1.1.2 所示：

函数	描述
int jiffies_to_msecs(const unsigned long j)	将 jiffies 类型的参数 j 分别转换为对应的毫秒、微秒、纳秒。
int jiffies_to_usecs(const unsigned long j)	
u64 jiffies_to_nsecs(const unsigned long j)	
long msecs_to_jiffies(const unsigned int m)	将毫秒、微秒、纳秒转换为 jiffies 类型。
long usecs_to_jiffies(const unsigned int u)	
unsigned long nsecs_to_jiffies(u64 n)	

表 30.1.1.2 jiffies 和 ms、us、ns 之间的转换函数

30.1.2 内核定时器简介

定时器是一个很常用的功能，需要周期性处理的工作都要用到定时器，Linux 内核定时器采用系统时钟来实现。Linux 内核定时器使用很简单，只需要提供超时时间(相当于定时值)和定时处理函数即可，当超时时间到了以后设置的定时处理函数就会执行，和我们使用硬件定时器的套路一样，只是使用内核定时器不需要做一大堆的寄存器初始化工作。在使用内核定时器的时候要注意一点，内核定时器并不是周期性运行的，超时以后就会自动关闭，因此如果想要实现周期性定时，那么就需要在定时处理函数中重新开启定时器。Linux 内核使用 timer_list 结构体表示内核定时器，timer_list 定义在文件 include/linux/timer.h 中，定义如下：

示例代码 30.1.2.1 timer_list 结构体

```
11 struct timer_list {
12  /*
13   * All fields that change during normal runtime grouped to the
14   * same cacheline
15   */
16  struct hlist_node entry;
17  unsigned long     expires;      /* 定时器超时时间，单位是节拍数      */
18  void              (*function)(struct timer_list *);/* 定时处理函数*/
19  u32               flags;        /* 标志位                           */
20
21 #ifdef CONFIG_LOCKDEP
22     struct lockdep_map  lockdep_map;
23 #endif
24 };
```

要使用内核定时器首先要先定义一个 timer_list 变量，表示定时器，tiemr_list 结构体的 expires 成员变量表示超时时间，单位为节拍数。比如我们现在需要定义一个周期为 2 秒的定时器，那么这个定时器的超时时间就是 jiffies+(2*HZ)，因此 expires=jiffies+(2*HZ)。function 就是定时器超时以后的定时处理函数，我们要做的工作就放到这个函数里面，需要我们编写这个定时处理函数，function 函数的形参就是我们定义的 timer_list 变量。

定义好定时器以后还需要通过一系列的 API 函数来初始化此定时器，这些函数如下：

1、timer_setup 函数

timer_setup 函数负责初始化 timer_list 类型变量，当我们定义了一个 timer_list 变量以后一定要先用 timer_setup 初始化一下。timer_setup 函数原型如下：

```
void timer_setup(struct timer_list *timer, void (*func)(struct timer_list *), unsigned int flags)
```

函数参数和返回值含义如下：

timer：要初始化定时器。

func：定时器的回调函数，此函数的形参是当前定时器的变量。

flags：标志位，直接给 0 就行。

返回值：没有返回值。

2、add_timer 函数

add_timer 函数用于向 Linux 内核注册定时器，使用 add_timer 函数向内核注册定时器以后，定时器就会开始运行，函数原型如下：

```
void add_timer(struct timer_list *timer)
```

函数参数和返回值含义如下：

timer：要注册的定时器。

返回值：没有返回值。

3、del_timer 函数

del_timer 函数用于删除一个定时器，不管定时器有没有被激活，都可以使用此函数删除。在多处理器系统上，定时器可能会在其他的处理器上运行，因此在调用 del_timer 函数删除定时器之前要先等待其他处理器的定时处理器函数退出。del_timer 函数原型如下：

```
int del_timer(struct timer_list * timer)
```

函数参数和返回值含义如下：

timer：要删除的定时器。

返回值：0，定时器还没被激活；1，定时器已经激活。

4、del_timer_sync 函数

del_timer_sync 函数是 del_timer 函数的同步版，会等待其他处理器使用完定时器再删除，del_timer_sync 不能使用在中断上下文中。del_timer_sync 函数原型如下所示：

```
int del_timer_sync(struct timer_list *timer)
```

函数参数和返回值含义如下：

timer：要删除的定时器。

返回值：0，定时器还没被激活；1，定时器已经激活。

5、mod_timer 函数

mod_timer 函数用于修改定时值，如果定时器还没有激活的话，mod_timer 函数会激活定时器！函数原型如下：

```
int mod_timer(struct timer_list *timer, unsigned long expires)
```

函数参数和返回值含义如下：

timer：要修改超时时间(定时值)的定时器。

expires：修改后的超时时间。

返回值：0，调用 mod_timer 函数前定时器未被激活；1，调用 mod_timer 函数前定时器已被激活。

关于内核定时器常用的 API 函数就讲这些，内核定时器一般的使用流程如下所示：

示例代码 30.1.2.2 内核定时器使用方法演示

```
1  struct timer_list timer;    /* 定义定时器  */
2
3  /* 定时器回调函数 */
4  void function(struct timer_list *arg)
5  {
6      /*
7       * 定时器处理代码
8       */
9
10     /* 如果需要定时器周期性运行的话就使用 mod_timer
11      * 函数重新设置超时值并且启动定时器。
12      */
13     mod_timer(&dev->timertest, jiffies + msecs_to_jiffies(2000));
14     }
15
16     /* 初始化函数 */
17 void init(void)
18 {
19     timer_setup(&timerdev.timer, timer_function, 0); /* 初始化定时器 */
20     timer.expires=jffies + msecs_to_jiffies(2000);/* 超时时间 2 秒 */
21     add_timer(&timer);                              /* 启动定时器   */
22 }
23
24 /* 退出函数 */
25 void exit(void)
26 {
27     del_timer(&timer);  /* 删除定时器 */
28     /* 或者使用 */
29     del_timer_sync(&timer);
30 }
```

30.1.3 Linux 内核短延时函数

有时候我们需要在内核中实现短延时，尤其是在 Linux 驱动中。Linux 内核提供了毫秒、微秒和纳秒延时函数，这三个函数如表 30.1.3.1 所示：

函数	描述
void ndelay(unsigned long nsecs)	纳秒、微秒和毫秒延时函数。
void udelay(unsigned long usecs)	
void mdelay(unsigned long mseces)	

表 30.1.3.1 内核短延时函数

30.2 硬件原理图分析

本章使用通过设置一个定时器来实现周期性的闪烁 LED 灯，因此本章例程就使用到了一个 LED 灯，本实验的硬件原理参考 21.2 小节即可。

30.3 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→12_timer

本章实验我们使用内核定时器周期性的点亮和熄灭开发板上的 LED 灯，LED 灯的闪烁周期由内核定时器来设置，测试应用程序可以控制内核定时器周期。

30.3.1 修改设备树文件

本章实验使用到了 LED 灯，LED 灯的设备树节点信息使用 25.4.1 小节创建的即可。

30.3.2 定时器驱动程序编写

新建名为“12_timer”的文件夹，然后在 12_timer 文件夹里面创建 vscode 工程，工作区命名为“timer”。工程创建好以后新建 timer.c 文件，在 timer.c 里面输入如下内容：

示例代码 30.3.2.1 timer.c 文件代码段

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <linux/of.h>
12 #include <linux/of_address.h>
13 #include <linux/of_gpio.h>
14 #include <linux/semaphore.h>
15 #include <linux/timer.h>
16 #include <asm/mach/map.h>
17 #include <asm/uaccess.h>
18 #include <asm/io.h>
19 /***************************************************************
20 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
21 文件名   : timer.c
22 作者     : 正点原子 Linux 团队
23 版本     : V1.0
24 描述     : Linux 内核定时器实验
25 其他     : 无
```

```
26  论坛     : www.openedv.com
27  日志     : 初版 V1.0 2021/01/5 正点原子 Linux 团队创建
28  ***************************************************************/
29  #define TIMER_CNT        1                  /* 设备号个数       */
30  #define TIMER_NAME       "timer"            /* 名字             */
31  #define CLOSE_CMD        (_IO(0XEF, 0x1))   /* 关闭定时器       */
32  #define OPEN_CMD         (_IO(0XEF, 0x2))   /* 打开定时器       */
33  #define SETPERIOD_CMD    (_IO(0XEF, 0x3))   /* 设置定时器周期命令*/
34  #define LEDON            1                  /* 开灯             */
35  #define LEDOFF           0                  /* 关灯             */
36
37  /* timer 设备结构体 */
38  struct timer_dev{
39      dev_t devid;              /* 设备号              */
40      struct cdev cdev;         /* cdev                */
41      struct class *class;      /* 类                  */
42      struct device *device;    /* 设备                */
43      int major;                /* 主设备号            */
44      int minor;                /* 次设备号            */
45      struct device_node  *nd; /* 设备节点             */
46      int led_gpio;             /* key 所使用的 GPIO 编号       */
47      int timeperiod;           /* 定时周期,单位为 ms    */
48      struct timer_list timer; /* 定义一个定时器       */
49      spinlock_t lock;          /* 定义自旋锁          */
50  };
51
52  struct timer_dev timerdev;  /* timer 设备 */
53
54  /*
55   * @description    : 初始化 LED 灯 IO，open 函数打开驱动的时候
56   *                   初始化 LED 灯所使用的 GPIO 引脚。
57   * @param          : 无
58   * @return         : 无
59   */
60  static int led_init(void)
61  {
62      int ret;
63      const char *str;
64
65      /* 设置 LED 所使用的 GPIO */
66      /* 1、获取设备节点：timerdev */
67      timerdev.nd = of_find_node_by_path("/gpioled");
68      if(timerdev.nd == NULL) {
```

```
69          printk("timerdev node not find!\r\n");
70          return -EINVAL;
71      }
72
73      /* 2.读取 status 属性 */
74      ret = of_property_read_string(timerdev.nd, "status", &str);
75      if(ret < 0)
76          return -EINVAL;
77
78      if (strcmp(str, "okay"))
79          return -EINVAL;
80
81      /* 3、获取 compatible 属性值并进行匹配 */
82      ret = of_property_read_string(timerdev.nd, "compatible", &str);
83      if(ret < 0) {
84          printk("timerdev: Failed to get compatible property\n");
85          return -EINVAL;
86      }
87
88      if (strcmp(str, "alientek,led")) {
89          printk("timerdev: Compatible match failed\n");
90          return -EINVAL;
91      }
92
93      /* 4、 获取设备树中的 gpio 属性，得到 led-gpio 所使用的 led 编号 */
94      timerdev.led_gpio = of_get_named_gpio(timerdev.nd,
                                              "led-gpio", 0);
95      if(timerdev.led_gpio < 0) {
96          printk("can't get led-gpio");
97          return -EINVAL;
98      }
99      printk("led-gpio num = %d\r\n", timerdev.led_gpio);
100
101     /* 5.向 gpio 子系统申请使用 GPIO */
102     ret = gpio_request(timerdev.led_gpio, "led");
103     if (ret) {
104         printk(KERN_ERR "timerdev: Failed to request led-gpio\n");
105         return ret;
106     }
107
108         /* 6、设置 PIO 为输出，并且输出高电平，默认关闭 LED 灯 */
109     ret = gpio_direction_output(timerdev.led_gpio, 1);
110     if(ret < 0) {
```

```
111         printk("can't set gpio!\r\n");
112         return ret;
113     }
114     return 0;
115 }
116 
117 /*
118  * @description   : 打开设备
119  * @param - inode : 传递给驱动的 inode
120  * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
121  *                  一般在 open 的时候将 private_data 指向设备结构体。
122  * @return        : 0 成功;其他 失败
123  */
124 static int timer_open(struct inode *inode, struct file *filp)
125 {
126     int ret = 0;
127     filp->private_data = &timerdev;     /* 设置私有数据      */
128 
129     timerdev.timeperiod = 1000;         /* 默认周期为 1s     */
130     ret = led_init();                   /* 初始化 LED IO     */
131     if (ret < 0) {
132         return ret;
133     }
134 
135     return 0;
136 }
137 
138 /*
139  * @description   : ioctl 函数,
140  * @param - filp  : 要打开的设备文件(文件描述符)
141  * @param - cmd   : 应用程序发送过来的命令
142  * @param - arg   : 参数
143  * @return        : 0 成功;其他 失败
144  */
145 static long timer_unlocked_ioctl(struct file *filp,
                                 unsigned int cmd, unsigned long arg)
146 {
147     struct timer_dev *dev =  (struct timer_dev *)filp->private_data;
148     int timerperiod;
149     unsigned long flags;
150 
151     switch (cmd) {
152         case CLOSE_CMD:     /* 关闭定时器 */
```

```
153             del_timer_sync(&dev->timer);
154             break;
155         case OPEN_CMD:      /* 打开定时器 */
156             spin_lock_irqsave(&dev->lock, flags);
157             timerperiod = dev->timeperiod;
158             spin_unlock_irqrestore(&dev->lock, flags);
159             mod_timer(&dev->timer, jiffies +
                          msecs_to_jiffies(timerperiod));
160             break;
161         case SETPERIOD_CMD: /* 设置定时器周期 */
162             spin_lock_irqsave(&dev->lock, flags);
163             dev->timeperiod = arg;
164             spin_unlock_irqrestore(&dev->lock, flags);
165             mod_timer(&dev->timer, jiffies + msecs_to_jiffies(arg));
166             break;
167         default:
168             break;
169     }
170     return 0;
171 }
172
173 /*
174  * @description   : 关闭/释放设备
175  * @param - filp  : 要关闭的设备文件(文件描述符)
176  * @return        : 0 成功;其他 失败
177  */
178 static int led_release(struct inode *inode, struct file *filp)
179 {
180     struct timer_dev *dev = filp->private_data;
181     gpio_set_value(dev->led_gpio, 1);  /* APP 结束的时候关闭 LED    */
182     gpio_free(dev->led_gpio);          /* 释放 LED                  */
183     del_timer_sync(&dev->timer);       /* 关闭定时器                */
184
185     return 0;
186 }
187
188 /* 设备操作函数 */
189 static struct file_operations timer_fops = {
190     .owner = THIS_MODULE,
191     .open = timer_open,
192     .unlocked_ioctl = timer_unlocked_ioctl,
193     .release =  led_release,
194 };
```

```
195
196 /* 定时器回调函数 */
197 void timer_function(struct timer_list *arg)
198 {
199     /*  from_timer 是个宏，可以根据结构体的成员地址，获取到这个结构体的首地址。
200         第一个参数表示结构体，第二个参数表示第一个参数里的一个成员，第三个参数表
        示第二个参数的类型,得到第一个参数的首地址。
201     */
202     struct timer_dev *dev = from_timer(dev, arg, timer);
203     static int sta = 1;
204     int timerperiod;
205     unsigned long flags;
206
207     sta = !sta;     /* 每次都取反，实现 LED 灯反转 */
208     gpio_set_value(dev->led_gpio, sta);
209
210     /* 重启定时器 */
211     spin_lock_irqsave(&dev->lock, flags);
212     timerperiod = dev->timeperiod;
213     spin_unlock_irqrestore(&dev->lock, flags);
214     mod_timer(&dev->timer, jiffies +
                  msecs_to_jiffies(dev->timeperiod));
215  }
216
217 /*
218  * @description   : 驱动入口函数
219  * @param         : 无
220  * @return        : 无
221  */
222 static int __init timer_init(void)
223 {
224     int ret;
225
226     /* 初始化自旋锁 */
227     spin_lock_init(&timerdev.lock);
228
229     /* 注册字符设备驱动 */
230     /* 1、创建设备号 */
231     if (timerdev.major) {      /*  定义了设备号 */
232         timerdev.devid = MKDEV(timerdev.major, 0);
233         ret = register_chrdev_region(timerdev.devid, TIMER_CNT,
                                         TIMER_NAME);
234         if(ret < 0) {
```

```
235             pr_err("cannot register %s char driver [ret=%d]\n",
                        TIMER_NAME, TIMER_CNT);
236             return -EIO;
237         }
238     } else {                                        /* 没有定义设备号 */
239         ret = alloc_chrdev_region(&timerdev.devid, 0, TIMER_CNT,
                                      TIMER_NAME);      /* 申请设备号 */
240         if(ret < 0) {
241             pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                       TIMER_NAME, ret);
242             return -EIO;
243         }
244         timerdev.major = MAJOR(timerdev.devid); /* 获取主设备号 */
245         timerdev.minor = MINOR(timerdev.devid); /* 获取次设备号 */
246     }
247     printk("timerdev major=%d,minor=%d\r\n",timerdev.major,
              timerdev.minor);
248
249     /* 2、初始化 cdev */
250     timerdev.cdev.owner = THIS_MODULE;
251     cdev_init(&timerdev.cdev, &timer_fops);
252
253     /* 3、添加一个 cdev */
254     cdev_add(&timerdev.cdev, timerdev.devid, TIMER_CNT);
255     if(ret < 0)
256         goto del_unregister;
257
258     /* 4、创建类 */
259     timerdev.class = class_create(THIS_MODULE, TIMER_NAME);
260     if (IS_ERR(timerdev.class)) {
261         goto del_cdev;
262     }
263
264     /* 5、创建设备 */
265     timerdev.device = device_create(timerdev.class, NULL,
                          timerdev.devid, NULL, TIMER_NAME);
266     if (IS_ERR(timerdev.device)) {
267         goto destroy_class;
268     }
269
270     /* 6、初始化 timer，设置定时器处理函数,还未设置周期，所有不会激活定时器 */
271     timer_setup(&timerdev.timer, timer_function, 0);
272
```

```
273     return 0;
274
275 destroy_class:
276     device_destroy(timerdev.class, timerdev.devid);
277 del_cdev:
278     cdev_del(&timerdev.cdev);
279 del_unregister:
280     unregister_chrdev_region(timerdev.devid, TIMER_CNT);
281     return -EIO;
282 }
283
284 /*
285  * @description   : 驱动出口函数
286  * @param         : 无
287  * @return        : 无
288  */
289 static void __exit timer_exit(void)
290 {
291     del_timer_sync(&timerdev.timer);        /* 删除 timer */
292 #if 0
293     del_timer(&timerdev.tiemr);
294 #endif
295
296     /* 注销字符设备驱动 */
297     cdev_del(&timerdev.cdev);               /*  删除 cdev */
298     unregister_chrdev_region(timerdev.devid, TIMER_CNT);
299
300     device_destroy(timerdev.class, timerdev.devid);
301     class_destroy(timerdev.class);
302 }
303
304 module_init(timer_init);
305 module_exit(timer_exit);
306 MODULE_LICENSE("GPL");
307 MODULE_AUTHOR("ALIENTEK");
308 MODULE_INFO(intree, "Y");
```

第 38~50 行，定时器设备结构体，在 48 行定义了一个定时器成员变量 timer。

第 60~115 行，LED 灯初始化函数，从设备树中获取 LED 灯信息，然后初始化相应的 IO。

第 124~136 行，函数 timer_open，对应应用程序的 open 函数，应用程序调用 open 函数打开/dev/timer 驱动文件的时候此函数就会执行。此函数设置文件私有数据为 timerdev，并且初始化定时周期默认为 1 秒，最后调用 led_init 函数初始化 LED 所使用的 IO。

第 145~171 行，函数 timer_unlocked_ioctl，对应应用程序的 ioctl 函数，应用程序调用 ioctl 函数向驱动发送控制信息，此函数响应并执行。此函数有三个参数：filp，cmd 和 arg，其中 filp

是对应的设备文件，cmd 是应用程序发送过来的命令信息，arg 是应用程序发送过来的参数，在本章例程中 arg 参数表示定时周期。

一共有三种命令 CLOSE_CMD，OPEN_CMD 和 SETPERIOD_CMD，这三个命令分别为关闭定时器、打开定时器、设置定时周期。这三个命令的左右如下：

CLOSE_CMD：关闭定时器命令，调用 del_timer_sync 函数关闭定时器。

OPEN_CMD：打开定时器命令，调用 mod_timer 函数打开定时器，定时周期为 timerdev 的 timeperiod 成员变量，定时周期默认是 1 秒。

SETPERIOD_CMD：设置定时器周期命令，参数 arg 就是新的定时周期，设置 timerdev 的 timeperiod 成员变量为 arg 所表示定时周期值。并且使用 mod_timer 重新打开定时器，使定时器以新的周期运行。

第 178~186 行，函数 timer_release，对应应用程序的 close 函数。退出的时候，关闭 LED 和关闭定时器。

第 189~194 行，定时器驱动操作函数集 timer_fops。

第 197~215 行，函数 timer_function，定时器服务函数。重点来看一下第 202 行，这一行需要得到第 52 行定义的 timerdev 变量。这里使用 from_timer 函数来根据 arg 参数反推出 timerdev 变量地址，from_timer 函数其实是一个宏，是对 container_of 函数的一个简单封装。container_of 函数的作用就是：给定结构体中的某个成员变量的地址、该结构体类型和该成员的名字来得到这个成员所在结构体变量的首地址。比如 timerdev 这个结构体变量，类型为 timer_dev，而 timer_dev 中有个成员变量 timer，timer 是 timer_list 类型。因此当我们知道了 timer 这个成员变量的具体地址以后，就可以根据 container_of 函数来反推出 timer 这个成员变量所属的 timer_dev 结构体变量首地址，在这里就是得到 timerdev 地址。最后在 timer_function 函数中将 LED 灯的状态取反，实现 LED 灯闪烁的效果。因为内核定时器不是循环的定时器，执行一次以后就结束了，因此在 214 行又调用了 mod_timer 函数重新开启定时器。

第 222~282 行，函数 timer_init，驱动入口函数。在第 271 行初始化定时器，设置定时器的定时处理函数为 timer_function，标志位为 0。在此函数中并没有调用 timer_add 函数来开启定时器，因此定时器默认是关闭的，除非应用程序发送打开命令。

第 289~302 行，驱动出口函数。第 291 行调用 del_timer_sync 函数删除定时器，也可以使用 del_timer 函数。

30.3.3 编写测试 APP

测试 APP 我们要实现的内容如下：

①、运行 APP 以后提示我们输入要测试的命令，输入 1 表示关闭定时器、输入 2 表示打开定时器，输入 3 设置定时器周期。

②、如果要设置定时器周期的话，需要让用户输入要设置的周期值，单位为毫秒。

新建名为 timerApp.c 的文件，然后输入如下所示内容：

示例代码 50.3.2.2 timerApp.c 文件代码段

```
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "fcntl.h"
6  #include "stdlib.h"
7  #include "string.h"
```

```
8   #include <sys/ioctl.h>
9   /***************************************************************
10  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
11  文件名      : timerApp.c
12  作者        : 正点原子 Linux 团队
13  版本        : V1.0
14  描述        : 定时器测试应用程序
15  其他        : 无
16  使用方法    : ./timertest /dev/timer 打开测试 App
17  论坛        : www.openedv.com
18  日志        : 初版 V1.0 2021/01/5 正点原子 Linux 团队创建
19  ***************************************************************/
20
21  /* 命令值 */
22  #define CLOSE_CMD        (_IO(0XEF, 0x1))    /* 关闭定时器          */
23  #define OPEN_CMD         (_IO(0XEF, 0x2))    /* 打开定时器          */
24  #define SETPERIOD_CMD    (_IO(0XEF, 0x3))    /* 设置定时器周期命令  */
25
26  /*
27   * @description   : main 主程序
28   * @param - argc  : argv 数组元素个数
29   * @param - argv  : 具体参数
30   * @return        : 0 成功;其他 失败
31   */
32  int main(int argc, char *argv[])
33  {
34      int fd, ret;
35      char *filename;
36      unsigned int cmd;
37      unsigned int arg;
38      unsigned char str[100];
39
40      if (argc != 2) {
41          printf("Error Usage!\r\n");
42          return -1;
43      }
44
45      filename = argv[1];
46
47      fd = open(filename, O_RDWR);
48      if (fd < 0) {
49          printf("Can't open file %s\r\n", filename);
50          return -1;
```

```
51      }
52
53      while (1) {
54          printf("Input CMD:");
55          ret = scanf("%d", &cmd);
56          if (ret != 1) {                 /* 参数输入错误 */
57              fgets(str, sizeof(str), stdin);             /* 防止卡死 */
58          }
59          if(4 == cmd)                    /* 退出 APP     */
60              goto out;
61          if(cmd == 1)                    /* 关闭 LED 灯 */
62              cmd = CLOSE_CMD;
63          else if(cmd == 2)               /* 打开 LED 灯 */
64              cmd = OPEN_CMD;
65          else if(cmd == 3) {
66              cmd = SETPERIOD_CMD;        /* 设置周期值 */
67              printf("Input Timer Period:");
68              ret = scanf("%d", &arg);
69              if (ret != 1) {             /* 参数输入错误 */
70                  fgets(str, sizeof(str), stdin);         /* 防止卡死 */
71              }
72          }
73          ioctl(fd, cmd, arg);            /* 控制定时器的打开和关闭 */
74      }
75
76  out:
77      close(fd);
78  }
```

第 22~24 行，命令值。

第 53~73 行，while(1)循环，让用户输入要测试的命令，然后通过第 73 行的 ioctl 函数发送给驱动程序。如果是设置定时器周期命令 SETPERIOD_CMD，那么 ioctl 函数的 arg 参数就是用户输入的周期值。

第 59~60 行，输入 4 就能退出 APP。

30.4 运行测试

30.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 timer.o，Makefile 内容如下所示：

示例代码 30.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
```

```
4  obj-m := timer.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 timer.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“timer.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 timerApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc timerApp.c -o timerApp
```

编译成功以后就会生成 timerApp 这个应用程序。

30.4.2 运行测试

将上一小节编译出来的 timer.ko 和 timerApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 timer.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe timer.ko       //加载驱动
```

驱动加载成功以后如下命令来测试：

```
./timerApp  /dev/timer
```

输入上述命令以后终端提示输入命令，如图 30.4.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./timerApp /dev/timer
led-gpio num = 128
Input CMD:
```

图 30.4.2.1 输入命令

输入“2”，打开定时器，此时 LED0 就会以默认的 1 秒周期开始闪烁。在输入“3”来设置定时周期，根据提示输入要设置的周期值，如图 30.4.2.2 所示：

```
Input CMD:3
Input Timer Period:
```

图 30.4.2.2 设置周期值

输入“500”，表示设置定时器周期值为 500ms，设置好以后 LED 灯就会以 500ms 为间隔，开始闪烁。最后可以通过输入“1”来关闭定时器，如果要卸载驱动的话输入如下命令即可：

```
rmmod timer.ko
```

第三十一章 Linux 中断实验

不管是单片机裸机实验还是 Linux 下的驱动实验，中断都是频繁使用的功能，在裸机中使用中断我们需要做一大堆的工作，比如配置寄存器，使能 IRQ 等等。但是 Linux 内核提供了完善的中断框架，我们只需要申请中断，然后注册中断处理函数即可，使用非常方便，不需要一系列复杂的寄存器配置。本章我们就来学习一下如何在 Linux 下使用中断。

31.1 Linux 中断简介

31.1.1 Linux 中断 API 函数

先来回顾一下裸机实验里面中断的处理方法：

①、使能中断，初始化相应的寄存器。

②、注册中断服务函数，也就是向 irqTable 数组的指定标号处写入中断服务函数

②、中断发生以后进入 IRQ 中断服务函数，在 IRQ 中断服务函数在数组 irqTable 里面查找具体的中断处理函数，找到以后执行相应的中断处理函数。

在 Linux 内核中也提供了大量的中断相关的 API 函数，我们来看一下这些跟中断有关的 API 函数：

1、中断号

每个中断都有一个中断号，通过中断号即可区分不同的中断，有的资料也把中断号叫做中断线。在 Linux 内核中使用一个 int 变量表示中断号，关于中断号我们已经在第十七章讲解过了。

2、request_irq 函数

在 Linux 内核中要想使用某个中断是需要申请的，request_irq 函数用于申请中断，request_irq 函数可能会导致睡眠，因此不能在中断上下文或者其他禁止睡眠的代码段中使用 request_irq 函数。request_irq 函数会激活(使能)中断，所以不需要我们手动去使能中断，request_irq 函数原型如下：

```
int request_irq(unsigned int      irq,
                irq_handler_t     handler,
                unsigned long     flags,
                const char        *name,
                void              *dev)
```

函数参数和返回值含义如下：

irq：要申请中断的中断号。

handler：中断处理函数，当中断发生以后就会执行此中断处理函数。

flags：中断标志，可以在文件 include/linux/interrupt.h 里面查看所有的中断标志，这里我们介绍几个常用的中断标志，如表 31.1.1.1 所示：

标志	描述
IRQF_SHARED	多个设备共享一个中断线，共享的所有中断都必须指定此标志。如果使用共享中断的话，request_irq 函数的 dev 参数就是唯一区分他们的标志。
IRQF_ONESHOT	单次中断，中断执行一次就结束。
IRQF_TRIGGER_NONE	无触发。
IRQF_TRIGGER_RISING	上升沿触发。
IRQF_TRIGGER_FALLING	下降沿触发。
IRQF_TRIGGER_HIGH	高电平触发。
IRQF_TRIGGER_LOW	低电平触发。

表 31.1.1.1 常用的中断标志

比如 STM32MP157 开发板上的 KEY0 使用 PG3，按下 KEY0 以后为低电平，因此可以设置为下降沿触发，也就是将 flags 设置为 IRQF_TRIGGER_FALLING。表 31.1.1.1 中的这些标志可以通过“|”来实现多种组合。

name：中断名字，设置以后可以在/proc/interrupts 文件中看到对应的中断名字。

dev：如果将 flags 设置为 IRQF_SHARED 的话，dev 用来区分不同的中断，一般情况下将 dev 设置为设备结构体，dev 会传递给中断处理函数 irq_handler_t 的第二个参数。

返回值：0 中断申请成功，其他负值 中断申请失败，如果返回-EBUSY 的话表示中断已经被申请了。

3、free_irq 函数

使用中断的时候需要通过 request_irq 函数申请，使用完成以后就要通过 free_irq 函数释放掉相应的中断。如果中断不是共享的，那么 free_irq 会删除中断处理函数并且禁止中断。free_irq 函数原型如下所示：

```
void free_irq(unsigned int     irq,
              void             *dev)
```

函数参数和返回值含义如下：

irq：要释放的中断。

dev：如果中断设置为共享(IRQF_SHARED)的话，此参数用来区分具体的中断。共享中断只有在释放最后中断处理函数的时候才会被禁止掉。

返回值：无。

4、中断处理函数

使用 request_irq 函数申请中断的时候需要设置中断处理函数，中断处理函数格式如下所示：

```
irqreturn_t (*irq_handler_t) (int, void *)
```

第一个参数是要中断处理函数要相应的中断号。第二个参数是一个指向 void 的指针，也就是个通用指针，需要与 request_irq 函数的 dev 参数保持一致。用于区分共享中断的不同设备，dev 也可以指向设备数据结构。中断处理函数的返回值为 irqreturn_t 类型，irqreturn_t 类型定义如下所示：

示例代码 31.1.1.1 irqreturn_t 结构

```
11 enum irqreturn {
12     IRQ_NONE          = (0 << 0),
13     IRQ_HANDLED       = (1 << 0),
14     IRQ_WAKE_THREAD   = (1 << 1),
15 };
16
17 typedef enum irqreturn irqreturn_t;
```

可以看出 irqreturn_t 是个枚举类型，一共有三种返回值。一般中断服务函数返回值使用如下形式：

```
return IRQ_RETVAL(IRQ_HANDLED)
```

5、中断使能与禁止函数

常用的中断使用和禁止函数如下所示：

```
void enable_irq(unsigned int irq)
void disable_irq(unsigned int irq)
```

enable_irq 和 disable_irq 用于使能和禁止指定的中断，irq 就是要禁止的中断号。disable_irq

函数要等到当前正在执行的中断处理函数执行完才返回，因此使用者需要保证不会产生新的中断，并且确保所有已经开始执行的中断处理程序已经全部退出。在这种情况下，可以使用另外一个中断禁止函数：

```
void disable_irq_nosync(unsigned int irq)
```

disable_irq_nosync 函数调用以后立即返回，不会等待当前中断处理程序执行完毕。上面三个函数都是使能或者禁止某一个中断，有时候我们需要关闭当前处理器的整个中断系统，也就是在学习 STM32 的时候常说的关闭全局中断，这个时候可以使用如下两个函数：

```
local_irq_enable()
local_irq_disable()
```

local_irq_enable 用于使能当前处理器中断系统，local_irq_disable 用于禁止当前处理器中断系统。假如 A 任务调用 local_irq_disable 关闭全局中断 10S，当关闭了 2S 的时候 B 任务开始运行，B 任务也调用 local_irq_disable 关闭全局中断 3S，3 秒以后 B 任务调用 local_irq_enable 函数将全局中断打开了。此时才过去 2+3=5 秒的时间，然后全局中断就被打开了，此时 A 任务要关闭 10S 全局中断的愿望就破灭了，然后 A 任务就“生气了”，结果很严重，可能系统都要被 A 任务整崩溃。为了解决这个问题，B 任务不能直接简单粗暴的通过 local_irq_enable 函数来打开全局中断，而是将中断状态恢复到以前的状态，要考虑到别的任务的感受，此时就要用到下面两个函数：

```
local_irq_save(flags)
local_irq_restore(flags)
```

这两个函数是一对，local_irq_save 函数用于禁止中断，并且将中断状态保存在 flags 中。local_irq_restore 用于恢复中断，将中断到 flags 状态。

31.1.2 上半部与下半部

在有些资料中也将上半部和下半部称为顶半部和底半部，都是一个意思。我们在使用 request_irq 申请中断的时候注册的中断服务函数属于中断处理的上半部，只要中断触发，那么中断处理函数就会执行。我们都知道中断处理函数一定要快点执行完毕，越短越好，但是现实往往是残酷的，有些中断处理过程就是比较费时间，我们必须要对其进行处理，缩小中断处理函数的执行时间。比如电容触摸屏通过中断通知 SOC 有触摸事件发生，SOC 响应中断，然后通过 IIC 接口读取触摸坐标值并将其上报给系统。但是我们都知道 IIC 的速度最高也只有 400Kbit/S，所以在中断中通过 IIC 读取数据就会浪费时间。我们可以将通过 IIC 读取触摸数据的操作暂后执行，中断处理函数仅仅相应中断，然后清除中断标志位即可。这个时候中断处理过程就分为了两部分：

上半部：上半部就是中断处理函数，那些处理过程比较快，不会占用很长时间的处理就可以放在上半部完成。

下半部：如果中断处理过程比较耗时，那么就将这些比较耗时的代码提出来，交给下半部去执行，这样中断处理函数就会快进快出。

因此，Linux 内核将中断分为上半部和下半部的主要目的就是实现中断处理函数的快进快出，那些对时间敏感、执行速度快的操作可以放到中断处理函数中，也就是上半部。剩下的所有工作都可以放到下半部去执行，比如在上半部将数据拷贝到内存中，关于数据的具体处理就可以放到下半部去执行。至于哪些代码属于上半部，哪些代码属于下半部并没有明确的规定，一切根据实际使用情况去判断，这个就很考验驱动编写人员的功底了。这里有一些可以借鉴的参考点：

①、如果要处理的内容不希望被其他中断打断，那么可以放到上半部。

②、如果要处理的任务对时间敏感，可以放到上半部。

③、如果要处理的任务与硬件有关，可以放到上半部

④、除了上述三点以外的其他任务，优先考虑放到下半部。

上半部处理很简单，直接编写中断处理函数就行了，关键是下半部该怎么做呢？Linux 内核提供了多种下半部机制，接下来我们来学习一下这些下半部机制。

1、软中断

一开始 Linux 内核提供了“bottom half”机制来实现下半部，简称“BH”。后面引入了软中断和 tasklet 来替代“BH”机制，完全可以使用软中断和 tasklet 来替代 BH，从 2.5 版本的 Linux 内核开始 BH 已经被抛弃了。Linux 内核使用结构体 softirq_action 表示软中断， softirq_action 结构体定义在文件 include/linux/interrupt.h 中，内容如下：

示例代码 31.1.2.1 softirq_action 结构体

```
541 struct softirq_action
542 {
543     void    (*action)(struct softirq_action *);
544 };
```

在 kernel/softirq.c 文件中一共定义了 10 个软中断，如下所示：

示例代码 31.1.2.2 softirq_vec 数组

```
static struct softirq_action softirq_vec[NR_SOFTIRQS];
```

NR_SOFTIRQS 是枚举类型，定义在文件 include/linux/interrupt.h 中，定义如下：

示例代码 31.1.2.3 softirq_vec 数组

```
enum
{
    HI_SOFTIRQ=0,               /* 高优先级软中断        */
    TIMER_SOFTIRQ,              /* 定时器软中断          */
    NET_TX_SOFTIRQ,             /* 网络数据发送软中断    */
    NET_RX_SOFTIRQ,             /* 网络数据接收软中断    */
    BLOCK_SOFTIRQ,
    IRQ_POLL_SOFTIRQ,
    TASKLET_SOFTIRQ,            /* tasklet 软中断        */
    SCHED_SOFTIRQ,              /* 调度软中断            */
    HRTIMER_SOFTIRQ,            /* 高精度定时器软中断    */
    RCU_SOFTIRQ,                /* RCU 软中断            */

    NR_SOFTIRQS
};
```

可以看出，一共有 10 个软中断，因此 NR_SOFTIRQS 为 10，因此数组 softirq_vec 有 10 个元素。softirq_action 结构体中的 action 成员变量就是软中断的服务函数，数组 softirq_vec 是个全局数组，因此所有的 CPU(对于 SMP 系统而言)都可以访问到，每个 CPU 都有自己的触发和控制机制，并且只执行自己所触发的软中断。但是各个 CPU 所执行的软中断服务函数确是相同的，都是数组 softirq_vec 中定义的 action 函数。要使用软中断，必须先使用 open_softirq 函数注册对应的软中断处理函数，open_softirq 函数原型如下：

```
void open_softirq(int nr,    void (*action)(struct softirq_action *))
```

函数参数和返回值含义如下：

nr：要开启的软中断，在示例代码 31.1.2.3 中选择要开启的软中断。

action：软中断对应的处理函数。

返回值：没有返回值。

注册好软中断以后需要通过 raise_softirq 函数触发，raise_softirq 函数原型如下：

```
void raise_softirq(unsigned int nr)
```

函数参数和返回值含义如下：

nr：要触发的软中断，在示例代码 31.1.2.3 中选择要注册的软中断。

返回值：没有返回值。

软中断必须在编译的时候静态注册！Linux 内核使用 softirq_init 函数初始化软中断，softirq_init 函数定义在 kernel/softirq.c 文件里面，函数内容如下：

示例代码 31.1.2.4 softirq_init 函数内容

```
575 void __init softirq_init(void)
576 {
577     int cpu;
578
579     for_each_possible_cpu(cpu) {
580         per_cpu(tasklet_vec, cpu).tail =
581             &per_cpu(tasklet_vec, cpu).head;
582         per_cpu(tasklet_hi_vec, cpu).tail =
583             &per_cpu(tasklet_hi_vec, cpu).head;
584     }
585
586     open_softirq(TASKLET_SOFTIRQ, tasklet_action);
587     open_softirq(HI_SOFTIRQ, tasklet_hi_action);
588 }
```

从示例代码 31.1.2.4 可以看出，softirq_init 函数默认会打开 TASKLET_SOFTIRQ 和 HI_SOFTIRQ。

2、tasklet

tasklet 是利用软中断来实现的另外一种下半部机制，在软中断和 tasklet 之间，建议大家使用 tasklet。tasklet_struct 结构体如下所示：

示例代码 31.1.2.5 tasklet_struct 结构体

```
592 struct tasklet_struct
593 {
594     struct tasklet_struct *next;    /* 下一个 tasklet          */
595     unsigned long state;            /* tasklet 状态            */
596     atomic_t count;                 /* 计数器，记录对 tasklet 的引用数 */
597     void (*func)(unsigned long);    /* tasklet 执行的函数      */
598     unsigned long data;             /* 函数 func 的参数        */
599 };
```

第 597 行的 func 函数就是 tasklet 要执行的处理函数，用户实现具体的函数内容，相当于中断处理函数。如果要使用 tasklet，必须先定义一个 tasklet_struct 变量，然后使用 tasklet_init 函数对其进行初始化，taskled_init 函数原型如下：

```
void tasklet_init(struct tasklet_struct     *t,
```

```
                    void (*func)(unsigned long),
                    unsigned long       data);
```

函数参数和返回值含义如下：

t：要初始化的 tasklet

func：tasklet 的处理函数。

data：要传递给 func 函数的参数

返回值：没有返回值。

也可以使用宏 DECLARE_TASKLET 来一次性完成 tasklet 的定义和初始化，DECLARE_TASKLET 定义在 include/linux/interrupt.h 文件中，定义如下:

```
DECLARE_TASKLET(name, func, data)
```

其中 name 为要定义的 tasklet 名字，其实就是 tasklet_struct 类型的变量名，func 就是 tasklet 的处理函数，data 是传递给 func 函数的参数。

在上半部，也就是中断处理函数中调用 tasklet_schedule 函数就能使 tasklet 在合适的时间运行，tasklet_schedule 函数原型如下：

```
void tasklet_schedule(struct tasklet_struct *t)
```

函数参数和返回值含义如下：

t：要调度的 tasklet，也就是 DECLARE_TASKLET 宏里面的 name。

返回值：没有返回值。

关于 tasklet 的参考使用示例如下所示：

示例代码 31.1.2.7 tasklet 使用示例

```
/* 定义 taselet                */
struct tasklet_struct testtasklet;

/* tasklet 处理函数            */
void testtasklet_func(unsigned long data)
{
    /* tasklet 具体处理内容 */
}

/* 中断处理函数 */
irqreturn_t test_handler(int irq, void *dev_id)
{
    ......
    /* 调度 tasklet            */
    tasklet_schedule(&testtasklet);
    ......
}

/* 驱动入口函数                */
static int __init xxxx_init(void)
{
    ......
    /* 初始化 tasklet          */
```

```
    tasklet_init(&testtasklet, testtasklet_func, data);
    /* 注册中断处理函数      */
    request_irq(xxx_irq, test_handler, 0, "xxx", &xxx_dev);
    ......
}
```

2、工作队列

工作队列是另外一种下半部执行方式，工作队列在进程上下文执行，工作队列将要推后的工作交给一个内核线程去执行，因为工作队列工作在进程上下文，因此工作队列允许睡眠或重新调度。因此如果你要推后的工作可以睡眠那么就可以选择工作队列，否则的话就只能选择软中断或 tasklet。

Linux 内核使用 work_struct 结构体表示一个工作，内容如下(省略掉条件编译)：

示例代码 31.1.2.8 work_struct 结构体

```
struct work_struct {
    atomic_long_t data;
    struct list_head entry;
    work_func_t func;         /* 工作队列处理函数  */
};
```

这些工作组织成工作队列，工作队列使用 workqueue_struct 结构体表示，内容如下(省略掉条件编译)：

示例代码 31.1.2.9 workqueue_struct 结构体

```
struct workqueue_struct {
    struct list_head     pwqs;
    struct list_head     list;
    struct mutex         mutex;
    int                  work_color;
    int                  flush_color;
    atomic_t             nr_pwqs_to_flush;
    struct wq_flusher    *first_flusher;
    struct list_head     flusher_queue;
    struct list_head     flusher_overflow;
    struct list_head     maydays;
    struct worker        *rescuer;
    int                  nr_drainers;
    int                  saved_max_active;
    struct workqueue_attrs  *unbound_attrs;
    struct pool_workqueue   *dfl_pwq;
    char                 name[WQ_NAME_LEN];
    struct rcu_head      rcu;
    unsigned int         flags ____cacheline_aligned;
    struct pool_workqueue __percpu *cpu_pwqs;
    struct pool_workqueue __rcu *numa_pwq_tbl[];
};
```

Linux 内核使用工作者线程(worker thread)来处理工作队列中的各个工作，Linux 内核使用 worker 结构体表示工作者线程，worker 结构体内容如下：

示例代码 31.1.2.10 worker 结构体

```
struct worker {
    union {
        struct list_head      entry;
        struct hlist_node     hentry;
    };

    struct work_struct        *current_work;
    work_func_t               current_func;
    struct pool_workqueue     *current_pwq;
    struct list_head          scheduled;
    struct task_struct        *task;
    struct worker_pool        *pool;
    struct list_head          node;
    unsigned long             last_active;
    unsigned int              flags;
    int                       id;
    int                       sleeping;
    char                      desc[WORKER_DESC_LEN];
    struct workqueue_struct   *rescue_wq;
    work_func_t               last_func;
};
```

从示例代码 31.1.2.10 可以看出，每个 worker 都有一个工作队列，工作者线程处理自己工作队列中的所有工作。在实际的驱动开发中，我们只需要定义工作(work_struct)即可，关于工作队列和工作者线程我们基本不用去管。简单创建工作很简单，直接定义一个 work_struct 结构体变量即可，然后使用 INIT_WORK 宏来初始化工作，INIT_WORK 宏定义如下：

```
#define INIT_WORK(_work, _func)
```

_work 表示要初始化的工作，_func 是工作对应的处理函数。

也可以使用 DECLARE_WORK 宏一次性完成工作的创建和初始化，宏定义如下：

```
#define DECLARE_WORK(n, f)
```

n 表示定义的工作(work_struct)，f 表示工作对应的处理函数。

和 tasklet 一样，工作也是需要调度才能运行的，工作的调度函数为 schedule_work，函数原型如下所示：

```
bool schedule_work(struct work_struct *work)
```

函数参数和返回值含义如下：

work：要调度的工作。

返回值：0 成功，其他值 失败。

关于工作队列的参考使用示例如下所示：

示例代码 31.1.2.11 工作队列使用示例

```
/* 定义工作(work)                */
struct  work_struct testwork;
```

```
/* work 处理函数            */
void testwork_func_t(struct work_struct *work);
{
    /* work 具体处理内容     */
}

/* 中断处理函数             */
irqreturn_t test_handler(int irq, void *dev_id)
{
    ......
    /* 调度 work            */
    schedule_work(&testwork);
    ......
}

/* 驱动入口函数             */
static int __init xxxx_init(void)
{
    ......
    /* 初始化 work          */
    INIT_WORK(&testwork, testwork_func_t);
    /* 注册中断处理函数      */
    request_irq(xxx_irq, test_handler, 0, "xxx", &xxx_dev);
    ......
}
```

31.1.3 设备树中断信息节点

1、GIC 中断控制器

STM32MP1 有三个与中断有关的控制器：GIC、EXTI 和 NVIC，其中 NVIC 是 Cortex-M4 内核的中断控制器，本教程只讲解 Cortex-A7 内核，因此就只剩下了 GIC 和 EXTI。首先是 GIC，全称为：Generic Interrupt Controller，关于 GIC 的详细内容可以查看文档《ARM Generic Interrupt Controller(ARM GIC 控制器)V2.0》，此文档已经放到了开发板光盘中，路径为：开发板光盘→4、参考资料→ ARM Generic Interrupt Controller(ARM GIC 控制器)V2.0.pdf。

GIC 是 ARM 公司给 Cortex-A/R 内核提供的一个中断控制器，类似 Cortex-M 内核中的 NVIC。目前 GIC 有 4 个版本:V1~V4，V1 是最老的版本，已经被废弃了。V2~V4 目前正在大量的使用。GIC V2 是给 ARMv7-A 架构使用的，比如 Cortex-A7、Cortex-A9、Cortex-A15 等，V3 和 V4 是给 ARMv8-A/R 架构使用的，也就是 64 位芯片使用的。STM32MP1 是 Cortex-A7 内核，因此我们主要讲解 GIC V2。GIC V2 最多支持 8 个核。ARM 会根据 GIC 版本的不同研发出不同的 IP 核，那些半导体厂商直接购买对应的 IP 核即可，比如 ARM 针对 GIC V2 就开发出了 GIC400 这个中断控制器 IP 核。当 GIC 接收到外部中断信号以后就会报给 ARM 内核，但是

ARM 内核只提供了四个信号给 GIC 来汇报中断情况：VFIQ、VIRQ、FIQ 和 IRQ，他们之间的关系如图 31.1.3.1 所示：

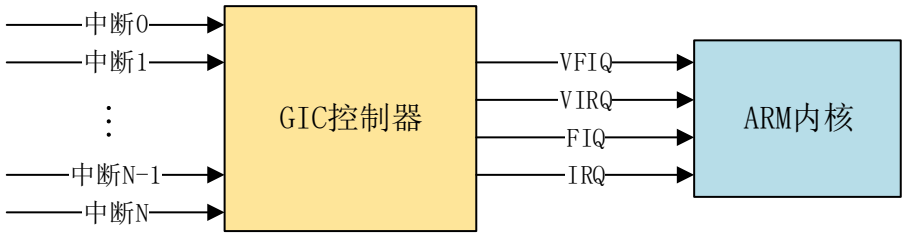


图 31.1.3.1 中断示意图

在图 31.1.3.1 中，GIC 接收众多的外部中断，然后对其进行处理，最终就只通过四个信号报给 ARM 内核，这四个信号的含义如下：

VFIQ:虚拟快速 FIQ。

VIRQ:虚拟快速 IRQ。

FIQ:快速中断 IRQ。

IRQ:外部中断 IRQ。

VFIQ 和 VIRQ 是针对虚拟化的，我们讨论不虚拟化，剩下的就是 FIQ 和 IRQ 了，本教程我们只使用 IRQ。所以相当于 GIC 最终向 ARM 内核就上报一个 IRQ 信号。那么 GIC 是如何完成这个工作的呢？GIC V2 的逻辑图如图 31.1.3.2 所示：

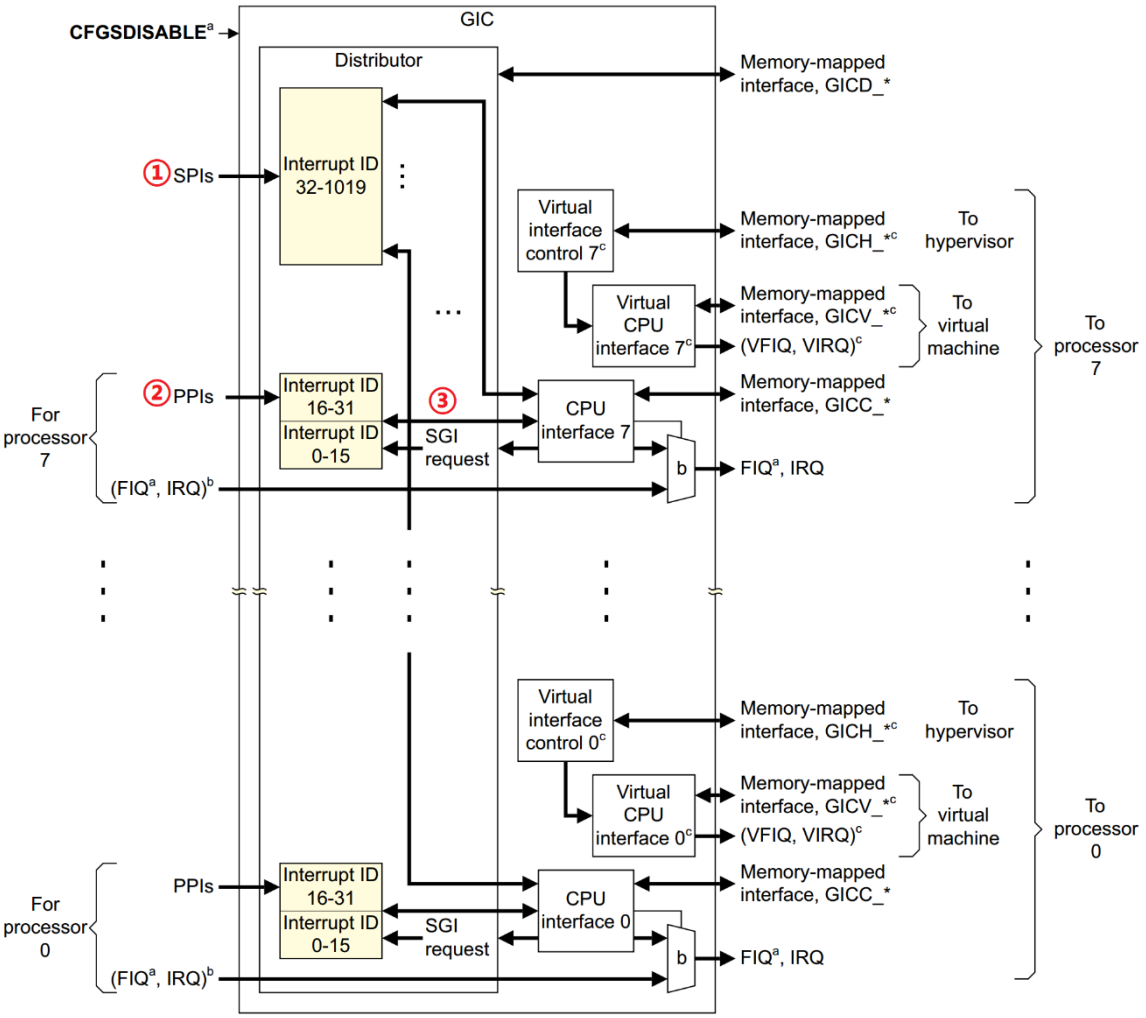


图 31.1.3.2 GIC V2 总体框图

图 31.1.3.2 中左侧部分就是中断源，中间部分就是 GIC 控制器，最右侧就是中断控制器向处理器内核发送中断信息。我们重点要看的肯定是中间的 GIC 部分，GIC 将众多的中断源分为分为三类：

①、SPI(Shared Peripheral Interrupt),共享中断，顾名思义，所有 Core 共享的中断，这个是最常见的，那些外部中断都属于 SPI 中断(注意！不是 SPI 总线那个中断) 。比如 GPIO 中断、串口中断等等，这些中断所有的 Core 都可以处理，不限定特定 Core。

②、PPI(Private Peripheral Interrupt)，私有中断，我们说了 GIC 是支持多核的，每个核肯定有自己独有的中断。这些独有的中断肯定是要指定的核心处理，因此这些中断就叫做私有中断。

③、SGI(Software-generated Interrupt)，软件中断，由软件触发引起的中断，通过向寄存器 GICD_SGIR 写入数据来触发，系统会使用 SGI 中断来完成多核之间的通信。

2、中断 ID

中断源有很多，为了区分这些不同的中断源肯定要给他们分配一个唯一 ID，这些 ID 就是中断 ID。每一个 CPU 最多支持 1020 个中断 ID，中断 ID 号为 ID0~ID1019。这 1020 个 ID 包含了 PPI、SPI 和 SGI，那么这三类中断是如何分配这 1020 个中断 ID 的呢？这 1020 个 ID 分配如下：

ID0~ID15：这 16 个 ID 分配给 SGI。

ID16~ID31：这 16 个 ID 分配给 PPI。

ID32~ID1019：这 988 个 ID 分配给 SPI，像 GPIO 中断、串口中断等这些外部中断 ，至于具体到某个 ID 对应哪个中断那就由半导体厂商根据实际情况去定义了。比如 STM32MP1 系列总共分配了 265 个中断 ID(有很多并未使用，只是保留着)，加上前面属于 PPI 和 SGI 的 32 个 ID，STM32MP1 的中断源共有 256+32=288 个，这 288 个中断 ID 对应的中断源可以在《STM32MP157 参考手册》中找到详细的解释。找到“21.2 GIC Interrupts”小节的“Table 117. STM32MP157 interrupt mapping for Cortex®-A7 GIC (continued)”，我们重点关注的是从 ID32 开始的 SPI 中断，因为这些才是 STM32MP1 的外设中断，如图 31.1.3.3 所示(由于表太大，这里只是截取其中一部分)：

0	32	WWDG1_IT	Window watchdog 1 early wakeup interrupt	-
1	33	PVD_AVD	PVD & AVD detector through EXTI	16
2	34	TAMP	Tamper interrupt (include LSECSS interrupts)	(18)
3	35	RTC_WKUP_ALARM	RTC wakeup timer and alarms (A and B) interrupt	(19)
4	36	TZC_IT	TrustZone DDR address space controller	-
5	37	RCC	RCC global interrupt	-
6	38	EXTI0	EXTI line 0 interrupt	0
7	39	EXTI1	EXTI line 1 interrupt	1
8	40	EXTI2	EXTI line 2 interrupt	2
9	41	EXTI3	EXTI line 3 interrupt	3
10	42	EXTI4	EXTI line 4 interrupt	4
11	43	DMA1_STR0	DMA1 stream0 global interrupt	-
12	44	DMA1_STR1	DMA1 stream1 global interrupt	-
13	45	DMA1_STR2	DMA1 stream2 global interrupt	-
.....	.....	.....	.....	.....
215	247	-	Reserved	-
216	248	nCTIIRQ0	Cortex®-A7 core#0 CTI interrupt	-
217	249	nCTIIRQ1	Cortex®-A7 core#1 CTI interrupt	-
210	242	-	Reserved	-
211	243	-	Reserved	-
212 to 255	244 to 287	-	Reserved	-

图 31.1.3.3 STM32MP1 中断源

关于 GIC 就先讲到这里，我们接下来讲解一下 EXTI。

3、EXTI- 外部中断和事件控制器

EXTI 是什么东西呢？全称是：Extended interrupt and event controller，中文一般叫做外部中断和事件控制器。EXTI 是 ST 自己设计的，用来辅助 GIC 管理 STM32MP1 相应中断的。EXTI 通过可配置的事件输入和直接事件输入来管理唤醒。它可以针对电源控制提供唤醒请求、针对 CPU 事件输入生成事件。EXTI 唤醒请求可让系统从停止模式唤醒，以及让 CPU 从 CSTOP 和 CSTANDBY 模式唤醒。此外，EXTI 还可以在运行模式下生成中断请求和事件请求，这个非常重要，因为在实际使用中 EXTI 主要是为 STM32 的 GPIO 中断服务的。

EXTI 主要特性如下：

- 支持 76 个输入事件
- 两个 CPU 内核都支持。

• 所有事件输入均可让 CPU 唤醒。

EXTI 的异步输入事件可以分为 2 组：

①、可配置事件（来自能够生成脉冲的 I/O 或外设的信号），这类事件具有以下特性：

- 可选择的有效触发边沿。
- 中断挂起状态寄存器位。
- 单独的中断和事件生成屏蔽。
- 支持软件触发。

②、直接事件（来自其他外设的中断和唤醒源，需要在外设中清除），这类事件具有以下特性：

- 固定上升沿有效触发。
- EXTI 中无中断挂起状态寄存器位（中断挂起状态由生成事件的外设提供）。
- 单独的中断和事件生成屏蔽。
- 不支持软件触发。

对于 GPIO 中断来说，就是可配置事件，EXTI 和 GIC 的关系如图 31.1.3.4 所示：

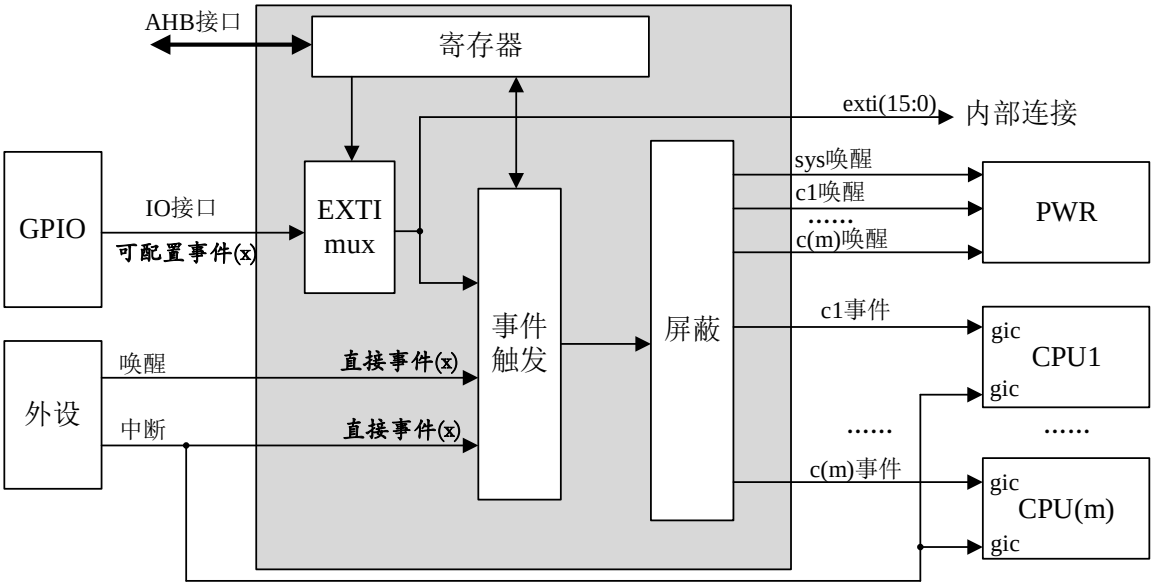


图 31.1.3.4 EXTI 框架

从图 31.1.3.4 可以看出 STM32MP1 的中断处理方式有 5 种：

①、外设直接产生中断到 GIC，然后由 GIC 通知 CPU 内核。

②、GPIO 或外设产生中断到 EXTI，EXTI 将信号提交给 GIC，最终再将中断信号提交给 CPU。

③、GPIO 或外设产生中断到 EXTI，EXTI 直接将中断信号提交给 CPU。

Linux 系统会用到这三种中断方式，一个外设最多可以有两种中断方式，为啥是两种而不是三种后面会说。

STM32MP1 的所有 GPIO 都有中断功能，而 GPIO 中断是我们最常用的功能。STM32 每一组 GPIO 最多有 16 个 IO，比如 PA0~PA15，因此每组 GPIO 就有 16 个中断，这 16 个 GPIO 事件输入对应 EXTI0~15，其中 PA0、PB0 等都对应 EXTI0，如图 31.1.3.5 所示：

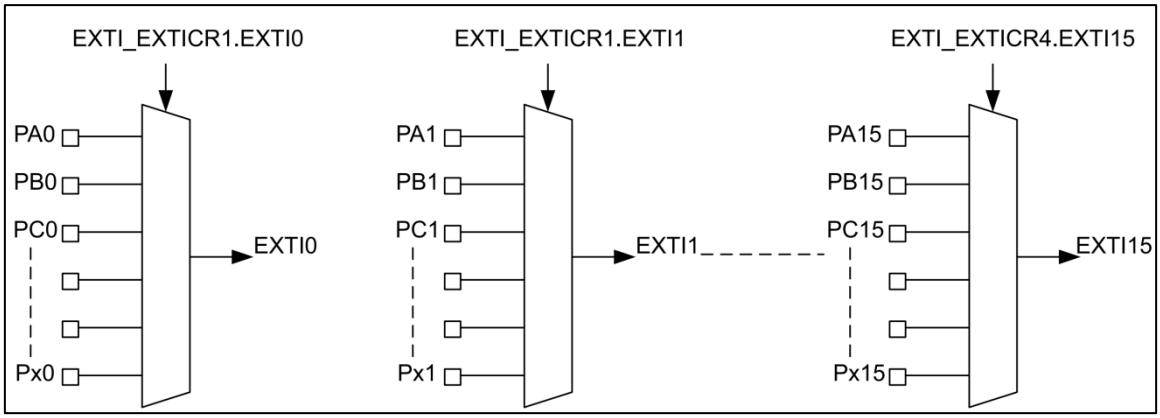


图 31.1.3.5 EXTI0~15 对应的 GPIO

如果要在 Linux 系统中使用中断，那么就需要在设备树中设置好中断属性信息，Linux 内核通过读取设备树中的中断属性信息来配置中断，GIC 控制器的设备树绑定信息参考文档 Documentation/devicetree/bindings/ interrupt-controller/arm,gic.yaml， EXTI 控制器的设备树绑定信息参考文档 Documentation/devicetree/bindings/interrupt-controller/st,stm32-exti.txt。

4、GIC 控制器节点

打开 stm32mp151.dtsi 文件，其中的 intc 节点就是 GIC 的中断控制器节点，节点内容如下所示：

示例代码 31.1.3.1 中断控制器 intc 节点

```
1  intc: interrupt-controller@a0021000 {
2      compatible = "arm,cortex-a7-gic";
3      #interrupt-cells = <3>;
4      interrupt-controller;
5      reg =  <0xa0021000 0x1000>,
6             <0xa0022000 0x2000>;
7  };
```

第 2 行，compatible 属性值为“arm,cortex-a7-gic”在 Linux 内核源码中搜索“arm,cortex-a7-gic”即可找到 GIC 中断控制器驱动文件。

第 3 行，#interrupt-cells 和#address-cells、#size-cells 一样。表示此中断控制器下设备的 cells 大小，对于设备而言，会使用 interrupts 属性描述中断信息，#interrupt-cells 描述了 interrupts 属性的 cells 大小，也就是一条信息有几个 cells。每个 cells 都是 32 位整形值，对于 ARM 处理的 GIC 来说，一共有 3 个 cells，这三个 cells 的含义如下：

第一个 cells：中断类型，0 表示 SPI 中断，1 表示 PPI 中断。

第二个 cells：中断号，对于 SPI 中断来说中断号的范围为 32~287(256 个)，对于 PPI 中断来说中断号的范围为 16~31，但是该 cell 描述的中断号是从 0 开始。

第三个 cells：标志，bit[3:0]表示中断触发类型，为 1 的时候表示上升沿触发，为 2 的时候表示下降沿触发，为 4 的时候表示高电平触发，为 8 的时候表示低电平触发。bit[15:8]为 PPI 中断的 CPU 掩码。

第 4 行，interrupt-controller 节点为空，表示当前节点是中断控制器。

我们来看一下 STM32MP1 的 SPI6 是如何在设备树节点中描述中断信息的，首先是查阅《STM32MP157 参考手册》第“21.2 GIC Interrupts”小节中的表 117，也就是图 31.1.3.3。找到 SPI6 对应的中断号，如图 31.1.3.6 所示：

Num	ID	Acronym	Description	EXTI event (1)
85	117	SPI5	SPI5 global interrupt	(40)
86	118	SPI6	SPI6 global interrupt	(41)

图 31.1.3.6 SPI6 中断

从图 31.1.3.6 可以看出，第一列的“Num”就是 SPI6 的中断号：86，注意这里并没有算前面 32 个中断号，如果加上前面 32 个中断号的话就是第二列“ID”，为：86+32=118。

打开 stm32mp151.dtsi，找到 SPI6 节点内容，如下所示：

示例代码 31.1.3.2 SPI6 节点

```
1  spi6: spi@5c001000 {
2      #address-cells = <1>;
3      #size-cells = <0>;
4      compatible = "st,stm32h7-spi";
5      reg = <0x5c001000 0x400>;
6      interrupts = <GIC_SPI 86 IRQ_TYPE_LEVEL_HIGH>;
7      clocks = <&scmi0_clk CK_SCMI0_SPI6>;
8      resets = <&scmi0_reset RST_SCMI0_SPI6>;
9      dmas = <&mdma1 34 0x0 0x40008 0x0 0x0 0x0>,
10           <&mdma1 35 0x0 0x40002 0x0 0x0 0x0>;
11     dma-names = "rx", "tx";
12     power-domains = <&pd_core>;
13     status = "disabled";
14 };
```

第 6 行，interrupts 描述中断中断源的信息，第一个表示中断类型，为 GIC_SPI，也就是共享中断。第二个表示中断号为 86，来源就是图 31.1.3.6。第三个表示中断触发类型是高电平触发。

2、EXTI 控制器节点

对于 GPIO 中断而言，要用到 EXTI，所以我们接下来看一下 EXTI 设备节点。打开 stm32mp151.dtsi 文件，其中的 exti 节点就是 EXTI 的中断控制器节点，节点内容如下所示：

示例代码 51.1.3.3 中断控制器 exti 节点

```
1  exti: interrupt-controller@5000d000 {
2      compatible = "st,stm32mp1-exti", "syscon";
3      interrupt-controller;
4      #interrupt-cells = <2>;
5      reg = <0x5000d000 0x400>;
6      hwlocks = <&hsem 1 1>;
7  };
```

第 3 行，表明 exti 节点是个中断控制器。

第 4 行，interrupt-cells=2，表明 exti 的子节点里面第一个 cell 表示为中断号，也可以叫 EXTI 事件编号，第二个 cell 表示中断标志位。其它的设备树属性和 GIC 控制器是一样的。

前面说了，GPIO 用到了 EXTI，因此 GPIO 节点信息里面的 EXTI 相关内容，在 stm32mp151.dtsi 文件中找到如下所示内容：

示例代码 31.1.3.4 GPIO 中断控制器节点

```
1   pinctrl: pin-controller@50002000 {
2       #address-cells = <1>;
3       #size-cells = <1>;
4       compatible = "st,stm32mp157-pinctrl";
5       ranges = <0 0x50002000 0xa400>;
6       interrupt-parent = <&exti>;
7       st,syscfg = <&exti 0x60 0xff>;
8       hwlocks = <&hsem 0 1>;
9       pins-are-numbered;
10
11      gpioa: gpio@50002000 {
12          gpio-controller;
13          #gpio-cells = <2>;
14          interrupt-controller;
15          #interrupt-cells = <2>;
16          reg = <0x0 0x400>;
17          clocks = <&rcc GPIOA>;
18          st,bank-name = "GPIOA";
19          status = "disabled";
20      };
......
120
121     gpiok: gpio@5000c000 {
122         gpio-controller;
123         #gpio-cells = <2>;
124         interrupt-controller;
125         #interrupt-cells = <2>;
126         reg = <0xa000 0x400>;
127         clocks = <&rcc GPIOK>;
128         st,bank-name = "GPIOK";
129         status = "disabled";
130     };
131 };
132
133 pinctrl_z: pin-controller-z@54004000 {
134     #address-cells = <1>;
135     #size-cells = <1>;
136     compatible = "st,stm32mp157-z-pinctrl";
137     ranges = <0 0x54004000 0x400>;
138     pins-are-numbered;
139     interrupt-parent = <&exti>;
140     st,syscfg = <&exti 0x60 0xff>;
```

```
141     hwlocks = <&hsem 0 1>;
142
143     gpioz: gpio@54004000 {
144         gpio-controller;
145         #gpio-cells = <2>;
146         interrupt-controller;
147         #interrupt-cells = <2>;
148         reg = <0 0x400>;
149         clocks = <&scmi0_clk CK_SCMI0_GPIOZ>;
150         st,bank-name = "GPIOZ";
151         st,bank-ioport = <11>;
152         status = "disabled";
153     };
154 };
```

第 1~131 行，pinctrl 节点，此节点有 11 个子节点，gpioa~gpiok，分别对应 GPIOA~GPIOK 这 11 组 IO。第 8 行，通过 interrupt-parent 属性指定 pinctrl 所有子节点的中断父节点为 exti，这样 GPIO 的中断就和 EXTI 联系起来了。第 11~20 行为 gpioa 节点，第 14 行表明 gpioa 节点也是个中断控制器，第 15 行设置 interrupt-cells 为 2，那么对于具体的 GPIO 而言，interrupts 属性第一个 cell 为某个 IO 在所处组的编号，第二个 cell 表示中断触发方式。比如现在要设置 PA1 这个引脚为下降沿触发，那么就要 interrupts = <1 IRQ_TYPE_EDGE_FALLING>。

第 133~154 行，pinctrl_z 节点，由于 GPIOZ 这一组对应的寄存器地址和 GPIOA~GPIOK 不是连续的，所以单独使用 pinctrl_z 来描述 GPIOZ，含义和 pinctrl 一样。

我们来看一个具体的应用，打开 stm32mp15xx-dkx.dtsi 文件，找到如下所示内容：

示例代码 31.1.3.6 hdmi 节点信息

```
1   hdmi-transmitter@39 {
2       compatible = "sil,sii9022";
3       reg = <0x39>;
4       iovcc-supply = <&v3v3_hdmi>;
5       cvcc12-supply = <&v1v2_hdmi>;
6       reset-gpios = <&gpioa 10 GPIO_ACTIVE_LOW>;
7       interrupts = <1 IRQ_TYPE_EDGE_FALLING>;
8       interrupt-parent = <&gpiog>;
9       #sound-dai-cells = <0>;
10      status = "okay";
11  };
```

sii9022 是 ST 官方在开发板上的一个 HDMI 芯片，上述代码就是 sii9022 的节点信息，sii9022a 芯片有一个中断，此引脚链接到了 STM32MP1 的 PG1 上，此中断是下降沿触发。

第 7 行，interrupts 设置中断信息，1 表示本组内第一个 IO，在这里就是 PG1。IRQ_TYPE_EDGE_FALLING 表示下降沿触发。

第 8 行，interrupt-parent 属性设置中断控制器，这里是有 gpiog 作为中断控制器。结合上面的 interrupts 属性，这两行的目的就是设置 PG1 为下降沿出触发。可以看出使用起来是非常的简单，在我们实际编写代码的时候，只需要通过 interrupt-parent 和 interrupts 这两个属性即可设置某个 GPIO 的中断功能。

简单总结一下与中断有关的设备树属性信息：

①、#interrupt-cells，指定中断源的信息 cells 个数。

②、interrupt-controller，表示当前节点为中断控制器。

③、interrupts，指定中断号，触发方式等。

④、interrupt-parent，指定父中断，也就是中断控制器。

⑤、interrupts-extended，指定中断控制器、中断号、中断类型和触发方式，这个属性比较特殊，是新加入的。前面说了，要通过 interrupts 和 interrupt-parent 一起设置某个 IO 的中断方式。这里我们也可以只使用 interrupts-extended 属性一次性指定中断父节点，IO 编号，中断方式等。打开 stm32mp157f-ev1-a7-examples.dts 文件，里面有如下所示代码：

示例代码 31.1.3.7 test_keys 节点

```
1  test_keys {
2      compatible = "gpio-keys";
3      #address-cells = <1>;
4      #size-cells = <0>;
5      autorepeat;
6      status = "okay";
7      /* gpio needs vdd core in retention for wakeup */
8      power-domains = <&pd_core_ret>;
9
10     button@1 {
11         label = "PA13";
12         linux,code = <BTN_1>;
13         interrupts-extended = <&gpioa 13 IRQ_TYPE_EDGE_FALLING>;
14         status = "okay";
15         wakeup-source;
16     };
17 };
```

很明显，上述代码用于描述一个按键，此按键采用中断方式，这个按键使用到了 PA13 这个引脚。第 13 行直接通过 interrupts-extended 属性描述了所有的中断信息，第一个参数为 gpioa，第二个参数为 13，第三个参数表示下降沿触发。如果使用 interrupts 和 interrupt-parent 来描述的话就是：

```
interrupt-parent = <&gpioa>;
interrupts = <13 IRQ_TYPE_EDGE_FALLING>;
```

大家根据自己的喜好来选择使用哪种形式描述即可。

31.1.4 获取中断号

编写驱动的时候需要用到中断号，我们用到的中断号，中断信息已经写到了设备树里面，因此可以通过 irq_of_parse_and_map 函数从 interupts 属性中提取到对应的设备号，函数原型如下：

```
unsigned int irq_of_parse_and_map(struct device_node *dev, int index)
```

函数参数和返回值含义如下：

dev：设备节点。

index：索引号，interrupts 属性可能包含多条中断信息，通过 index 指定要获取的信息。

返回值：中断号。

如果使用 GPIO 的话，可以使用 gpio_to_irq 函数来获取 gpio 对应的中断号，函数原型如下：

```
int gpio_to_irq(unsigned int gpio)
```

函数参数和返回值含义如下：

gpio：要获取的 GPIO 编号。

返回值：GPIO 对应的中断号。

31.2 硬件原理图分析

本章实验硬件原理图参考 29.2 小节即可。

31.3 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→13_irq。

本章实验我们驱动正点原子的 STM32MP157 开发板上的 KEY0 按键，不过我们采用中断的方式，并且采用定时器来实现按键消抖，应用程序读取按键值并且通过终端打印出来。通过本章我们可以学习到 Linux 内核中断的使用方法，以及对 Linux 内核定时器的回顾。

31.3.1 修改设备树文件

本章实验使用到了按键 KEY0，按键 KEY0 使用中断模式，因此需要在“key”节点下添加中断相关属性，添加完成以后的“key”节点内容如下所示：

示例代码 31.3.1.1 key 节点信息

```
1  key {
2      compatible = "alientek,key";
3      status = "okay";
4      key-gpio = <&gpiog 3 GPIO_ACTIVE_LOW>;
5      interrupt-parent = <&gpiog>;
6      interrupts = <3 IRQ_TYPE_EDGE_BOTH>;
7  };
```

第 5 行，设置 interrupt-parent 属性值为“gpiog”，因为 KEY0 所使用的 GPIO 为 PG3，所以要设置 KEY0 的中断控制器为 gpiog。

第 6 行，设置 interrupts 属性，也就是设置中断源，第一个 cells 的 3 表示 GPIOG 组的 3 号 IO。IRQ_TYPE_EDGE_BOTH 定义在文件 include/linux/irq.h 中，定义如下：

示例代码 31.3.1.2 中断状态

```
75 enum {
76     IRQ_TYPE_NONE           = 0x00000000,
77     IRQ_TYPE_EDGE_RISING    = 0x00000001,
78     IRQ_TYPE_EDGE_FALLING   = 0x00000002,
79     IRQ_TYPE_EDGE_BOTH      = (IRQ_TYPE_EDGE_FALLING |
                                  IRQ_TYPE_EDGE_RISING),
80     IRQ_TYPE_LEVEL_HIGH     = 0x00000004,
81     IRQ_TYPE_LEVEL_LOW      = 0x00000008,
82     IRQ_TYPE_LEVEL_MASK     = (IRQ_TYPE_LEVEL_LOW |
```

```
                                    IRQ_TYPE_LEVEL_HIGH),
......
100 };
```

从示例代码 31.3.1.2 中可以看出，IRQ_TYPE_EDGE_BOTH 表示上升沿和下降沿同时有效，相当于 KEY0 按下和释放都会触发中断。

设备树编写完成以后使用“make dtbs”命令重新编译设备树，然后使用新编译出来的 stm32mp157d-atk.dtb 文件启动 Linux 系统。

31.3.2 按键中断驱动程序编写

新建名为“13_irq”的文件夹，然后在 13_irq 文件夹里面创建 vscode 工程，工作区命名为“keyirq”。工程创建好以后新建 keyirq.c 文件，在 keyirq.c 里面输入如下内容：

示例代码 31.3.2.1 keyirq.c 文件代码

```
11  #include <linux/types.h>
12  #include <linux/kernel.h>
13  #include <linux/delay.h>
14  #include <linux/ide.h>
15  #include <linux/init.h>
16  #include <linux/module.h>
17  #include <linux/errno.h>
18  #include <linux/gpio.h>
19  #include <linux/cdev.h>
20  #include <linux/device.h>
21  #include <linux/of.h>
22  #include <linux/of_address.h>
23  #include <linux/of_gpio.h>
24  #include <linux/semaphore.h>
25  #include <linux/of_irq.h>
26  #include <linux/irq.h>
27  #include <asm/mach/map.h>
28  #include <asm/uaccess.h>
29  #include <asm/io.h>
30
31  #define KEY_CNT         1         /* 设备号个数    */
32  #define KEY_NAME        "key"     /* 名字          */
33
34  /* 定义按键状态 */
35  enum key_status {
36      KEY_PRESS = 0,          /*按键按下       */
37      KEY_RELEASE,            /*按键松开       */
38      KEY_KEEP,               /* 按键状态保持  */
39  };
40
41  /* key 设备结构体 */
```

```
42  struct key_dev{
43      dev_t devid;              /* 设备号       */
44      struct cdev cdev;         /* cdev         */
45      struct class *class;      /* 类           */
46      struct device *device;    /* 设备         */
47      struct device_node  *nd; /* 设备节点      */
48      int key_gpio;             /* key 所使用的 GPIO 编号*/
49      struct timer_list timer; /* 按键值       */
50      int irq_num;              /* 中断号       */
51      spinlock_t spinlock;      /* 自旋锁       */
52  };
53
54  static struct key_dev key;          /* 按键设备 */
55  static int status = KEY_KEEP;       /* 按键状态 */
56
57  static irqreturn_t key_interrupt(int irq, void *dev_id)
58  {
59      /* 按键防抖处理，开启定时器延时 15ms */
60      mod_timer(&key.timer, jiffies + msecs_to_jiffies(15));
61      return IRQ_HANDLED;
62  }
63
64  /*
65   * @description   : 初始化按键 IO，open 函数打开驱动的时候
66   *                  初始化按键所使用的 GPIO 引脚。
67   * @param         : 无
68   * @return        : 无
69   */
70  static int key_parse_dt(void)
71  {
72      int ret;
73      const char *str;
74
75      /* 设置 LED 所使用的 GPIO    */
76      /* 1、获取设备节点：key *   /
77      key.nd = of_find_node_by_path("/key");
78      if(key.nd == NULL) {
79          printk("key node not find!\r\n");
80          return -EINVAL;
81      }
82
83      /* 2.读取 status 属性 */
84      ret = of_property_read_string(key.nd, "status", &str);
```

```
85      if(ret < 0)
86          return -EINVAL;
87
88      if (strcmp(str, "okay"))
89          return -EINVAL;
90
91      /* 3、获取 compatible 属性值并进行匹配 */
92      ret = of_property_read_string(key.nd, "compatible", &str);
93      if(ret < 0) {
94          printk("key: Failed to get compatible property\n");
95          return -EINVAL;
96      }
97
98      if (strcmp(str, "alientek,key")) {
99          printk("key: Compatible match failed\n");
100         return -EINVAL;
101     }
102
103     /* 4、 获取设备树中的 gpio 属性，得到 KEY0 所使用的 KYE 编号 */
104     key.key_gpio = of_get_named_gpio(key.nd, "key-gpio", 0);
105     if(key.key_gpio < 0) {
106         printk("can't get key-gpio");
107         return -EINVAL;
108     }
109
110     /* 5 、获取 GPIO 对应的中断号 */
111     key.irq_num = irq_of_parse_and_map(key.nd, 0);
112     if(!key.irq_num){
113         return -EINVAL;
114     }
115
116     printk("key-gpio num = %d\r\n", key.key_gpio);
117     return 0;
118 }
119
120 static int key_gpio_init(void)
121 {
122     int ret;
123     unsigned long irq_flags;
124
125     ret = gpio_request(key.key_gpio, "KEY0");
126     if (ret) {
127         printk(KERN_ERR "key: Failed to request key-gpio\n");
```

```
128         return ret;
129     }
130 
131     /* 将 GPIO 设置为输入模式           */
132     gpio_direction_input(key.key_gpio);
133 
134     /* 获取设备树中指定的中断触发类型    */
135     irq_flags = irq_get_trigger_type(key.irq_num);
136     if (IRQF_TRIGGER_NONE == irq_flags)
137         irq_flags = IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING;
138 
139     /* 申请中断 */
140     ret = request_irq(key.irq_num, key_interrupt, irq_flags,
                          "Key0_IRQ", NULL);
141     if (ret) {
142         gpio_free(key.key_gpio);
143         return ret;
144     }
145 
146     return 0;
147 }
148 
149 static void key_timer_function(struct timer_list *arg)
150 {
151     static int last_val = 1;
152     unsigned long flags;
153     int current_val;
154 
155     /* 自旋锁上锁 */
156     spin_lock_irqsave(&key.spinlock, flags);
157 
158     /* 读取按键值并判断按键当前状态 */
159     current_val = gpio_get_value(key.key_gpio);
160     if (0 == current_val && last_val)  /* 按下     */
161         status = KEY_PRESS;
162     else if (1 == current_val && !last_val)
163         status = KEY_RELEASE;          /* 松开     */
164     else
165         status = KEY_KEEP;             /*状态保持  */
166 
167     last_val = current_val;
168 
169     /* 自旋锁解锁 */
```

```
170     spin_unlock_irqrestore(&key.spinlock, flags);
171 }
172
173 /*
174  * @description   : 打开设备
175  * @param - inode : 传递给驱动的 inode
176  * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
177  *                  一般在 open 的时候将 private_data 指向设备结构体。
178  * @return        : 0 成功;其他 失败
179  */
180 static int key_open(struct inode *inode, struct file *filp)
181 {
182     return 0;
183 }
184
185 /*
186  * @description   : 从设备读取数据
187  * @param - filp  : 要打开的设备文件(文件描述符)
188  * @param - buf   : 返回给用户空间的数据缓冲区
189  * @param - cnt   : 要读取的数据长度
190  * @param - offt  : 相对于文件首地址的偏移
191  * @return        : 读取的字节数，如果为负值，表示读取失败
192  */
193 static ssize_t key_read(struct file *filp, char __user *buf,
194             size_t cnt, loff_t *offt)
195 {
196     unsigned long flags;
197     int ret;
198
199     /* 自旋锁上锁   */
200     spin_lock_irqsave(&key.spinlock, flags);
201
202     /* 将按键状态信息发送给应用程序 */
203     ret = copy_to_user(buf, &status, sizeof(int));
204
205     /* 状态重置     */
206     status = KEY_KEEP;
207
208     /* 自旋锁解锁   */
209     spin_unlock_irqrestore(&key.spinlock, flags);
210
211     return ret;
212 }
```

```
213
214 /*
215  * @description   : 向设备写数据
216  * @param - filp  : 设备文件，表示打开的文件描述符
217  * @param - buf   : 要写给设备写入的数据
218  * @param - cnt   : 要写入的数据长度
219  * @param - offt  : 相对于文件首地址的偏移
220  * @return        : 写入的字节数，如果为负值，表示写入失败
221  */
222 static ssize_t key_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
223 {
224     return 0;
225 }
226
227 /*
228  * @description   : 关闭/释放设备
229  * @param - filp  : 要关闭的设备文件(文件描述符)
230  * @return        : 0 成功;其他 失败
231  */
232 static int key_release(struct inode *inode, struct file *filp)
233 {
234     return 0;
235 }
236
237 /* 设备操作函数 */
238 static struct file_operations key_fops = {
239     .owner = THIS_MODULE,
240     .open = key_open,
241     .read = key_read,
242     .write = key_write,
243     .release =  key_release,
244 };
245
246 /*
247  * @description   : 驱动入口函数
248  * @param         : 无
249  * @return        : 无
250  */
251 static int __init mykey_init(void)
252 {
253     int ret;
254
```

```
255     /* 初始化自旋锁     */
256     spin_lock_init(&key.spinlock);
257
258     /* 设备树解析       */
259     ret = key_parse_dt();
260     if(ret)
261         return ret;
262
263     /* GPIO 中断初始化 */
264     ret = key_gpio_init();
265     if(ret)
266         return ret;
267
268     /* 注册字符设备驱动 */
269     /* 1、创建设备号    */
270     ret = alloc_chrdev_region(&key.devid, 0, KEY_CNT, KEY_NAME);
271     if(ret < 0) {
272         pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                    KEY_NAME, ret);
273         goto free_gpio;
274     }
275
276     /* 2、初始化 cdev   */
277     key.cdev.owner = THIS_MODULE;
278     cdev_init(&key.cdev, &key_fops);
279
280     /* 3、添加一个 cdev */
281     cdev_add(&key.cdev, key.devid, KEY_CNT);
282     if(ret < 0)
283         goto del_unregister;
284
285     /* 4、创建类        */
286     key.class = class_create(THIS_MODULE, KEY_NAME);
287     if (IS_ERR(key.class)) {
288         goto del_cdev;
289     }
290
291     /* 5、创建设备      */
292     key.device = device_create(key.class, NULL, key.devid, NULL,
                                   KEY_NAME);
293     if (IS_ERR(key.device)) {
294         goto destroy_class;
295     }
```

```
296
297     /* 6、初始化 timer，设置定时器处理函数,还未设置周期，所有不会激活定时器 */
298     timer_setup(&key.timer, key_timer_function, 0);
299
300     return 0;
301
302 destroy_class:
303     device_destroy(key.class, key.devid);
304 del_cdev:
305     cdev_del(&key.cdev);
306 del_unregister:
307     unregister_chrdev_region(key.devid, KEY_CNT);
308 free_gpio:
309     free_irq(key.irq_num, NULL);
310     gpio_free(key.key_gpio);
311     return -EIO;
312 }
313
314 /*
315  * @description   : 驱动出口函数
316  * @param         : 无
317  * @return        : 无
318  */
319 static void __exit mykey_exit(void)
320 {
321     /* 注销字符设备驱动 */
322     cdev_del(&key.cdev);                    /* 删除 cdev    */
323     unregister_chrdev_region(key.devid, KEY_CNT); /* 注销设备号 */
324     del_timer_sync(&key.timer);             /* 删除 timer   */
325     device_destroy(key.class, key.devid); /*注销设备       */
326     class_destroy(key.class);               /* 注销类       */
327     free_irq(key.irq_num, NULL);            /* 释放中断     */
328     gpio_free(key.key_gpio);                /* 释放 IO      */
329 }
330
331 module_init(mykey_init);
332 module_exit(mykey_exit);
333 MODULE_LICENSE("GPL");
334 MODULE_AUTHOR("ALIENTEK");
335 MODULE_INFO(intree, "Y");
```

第 35~39 行，定义了一个枚举类型，包含 3 个常量 KEY_PRESS、KEY_RELEASE、KEY_KEEP，分别用来表示按键的 3 种不同的状态，即按键按下、按键松开以及按键状态保持。

第 42~52 行，结构体 key_dev 为按键设备所对应的结构体，key_gpio 为按键 GPIO 编号，

irq_num 为按键 IO 对应的中断号；除此之外，结构体当中还定义了一个定时器用于实现按键的去抖操作，还定义了一个自旋锁用于实现对关键代码的保护操作。

第 54 行，定义一个按键设备 key。

第 55 行，定义一个 int 类型的静态全局变量 status 用来表示按键的状态。

第 57~62 行，key_interrupt 函数是按键 KEY0 中断处理函数，参数 dev_id 是一个 void 类型的指针，本驱动程序并没使用到这个参数；这个中断处理函数很简单直接开启定时器，延时 15 毫秒，用于实现按键的软件防抖。

第 70~118 行，key_parse_dt 函数中主要是对设备树中的属性进行了解析，获取设备树中的 key 节点，通过 of_get_named_gpio 函数得到按键的 GPIO 编号，通过 irq_of_parse_and_map 函数获取按键的中断号，irq_of_parse_and_map 函数会解析 key 节点中的 interrupt-parent 和 interrupts 属性然后得到一个中断号，后面就可以使用这个中断号去申请以及释放中断了。

第 120~147 行，key_gpio_init 函数中主要对 GPIO 以及中断进行了相关的初始化。使用 gpio_request 函数申请 GPIO 使用权，通过 gpio_direction_input 将 GPIO 设置为输出模式；irq_get_trigger_type 函数可以获取到 key 节点中定义的中断触发类型，最后使用 request_irq 申请中断，并设置 key_interrupt 函数作为我们的按键中断处理函数，当按键中断发生之后便会跳转到该函数执行；request_irq 函数会默认使能中断，所以不需要 enable_irq 来使能中断，当然我们也可以在申请成功之后先使用 disable_irq 函数禁用中断，等所有工作完成之后再来使能中断，这样会比较安全，建议大家这样使用。

第 149~171 行，key_timer_function 函数为定时器定时处理函数，它的参数 arg 在本驱动程序中我们并没有使用到；该函数中定义了一个静态局部变量 last_val 用来保存按键上一次读取到的值，变量 current_val 用来存放当前按键读取到的值；第 159~165 行，通过读取到的按键值以及上一次读取到的值来判断按键当前所属的状态，如果本次读取的值为 0，而上一次读取的值 1，则表示按键按下；如果本次读取的值为 1，而上一次读取的值 0，则表示按键松开；如果本次读取的值为 0，而上一次读取的值也是 0，则表示按键一直被按着；如果本次读取的值为 1，而上一次读取的值也是 1，则表示没有触碰按键。第 167 行，当状态判断完成之后，会将 current_val 的值赋值给 last_val。本函数中也使用自旋锁对全局变量 status 进行加锁保护！

第 193~212 行，key_read 函数，对应应用程序的 read 函数。此函数向应用程序返回按键状态信息数据；这个函数其实很简单，使用 copy_to_user 函数直接将 statue 数据发送给应用程序，status 变量保存了按键当前的状态，发送完成之后再将按键状态重置即可！需要注意的是，该函数中使用了自旋锁进行保护。

第 238~244 行，按键设备的 file_operations 结构体。

第 251~312 行，mykey_init 是驱动入口函数，第 256 行调用 spin_lock_init 初始化自旋锁变量，298 行对定时器进行初始化并将 key_timer_function 函数绑定为定时器定时处理函数，当定时时间到了之后便会跳转到该函数执行。

第 319~329 行，mykey_exit 驱动出口函数，第 324 行调用 del_timer_sync 函数删除定时器，代码中已经注释得非常详细了，这里便不再多说！

31.3.2 编写测试 APP

测试 APP 要实现的内容很简单，通过不断的读取/dev/key 设备文件来获取按键值来判断当前按键的状态，从按键驱动上传到应用程序的数据可以有 3 个值，分别为 0、1、2；0 表示按键按下时的这个状态，1 表示按键松开时对应的状态，而 2 表示按键一直被按住或者松开；搞懂数据代表的意思之后，我们开始编写测试程序，在 13_irq 目录下新建名为 keyirqApp.c 的文件，然后输入如下所示内容：

示例代码 31.3.3.1 keyrqApp.c 文件代码

```
1  /***************************************************************
2  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
3  文件名       : keyApp.c
4  作者         : 正点原子 Linux 团队
5  版本         : V1.0
6  描述         : Linux 中断驱动实验
7  其他         : 无
8  使用方法     : ./keyirqApp /dev/key
9  论坛         : www.openedv.com
10 日志         : 初版 V1.0 2021/1/30 正点原子 Linux 团队创建
11 ***************************************************************/
12
13 #include <stdio.h>
14 #include <unistd.h>
15 #include <sys/types.h>
16 #include <sys/stat.h>
17 #include <fcntl.h>
18 #include <stdlib.h>
19 #include <string.h>
20
21 /*
22  * @description   : main 主程序
23  * @param - argc  : argv 数组元素个数
24  * @param - argv  : 具体参数
25  * @return        : 0 成功;其他 失败
26  */
27 int main(int argc, char *argv[])
28 {
29     int fd, ret;
30     int key_val;
31
32     /* 判断传参个数是否正确 */
33     if(2 != argc) {
34         printf("Usage:\n"
35             "\t./keyApp /dev/key\n"
36            );
37         return -1;
38     }
39
40     /* 打开设备 */
41     fd = open(argv[1], O_RDONLY);
42     if(0 > fd) {
```

```
43          printf("ERROR: %s file open failed!\n", argv[1]);
44          return -1;
45      }
46
47      /* 循环读取按键数据 */
48      for ( ; ; ) {
49
50          read(fd, &key_val, sizeof(int));
51          if (0 == key_val)
52              printf("Key Press\n");
53          else if (1 == key_val)
54              printf("Key Release\n");
55      }
56
57      /* 关闭设备 */
58      close(fd);
59      return 0;
60  }
```

第 48~55 行使用 for 循环不断的读取按键值，如果读取到的值是 0 则打印“Key Press”字符串，而过读取到的值是 1 则打印“Key Release”字符串。

31.4 运行测试

31.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 keyirq.o，Makefile 内容如下所示：

示例代码 31.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := keyirq.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 keyirq.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“keyirq.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 keyirqApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc keyirqApp.c -o keyirqApp
```

编译成功以后就会生成 keyirqApp 这个应用程序。

31.4.2 运行测试

将上一小节编译出来 keyirq.ko 和 keyirqApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 keyirq.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe keyirq.ko      //加载驱动
```

驱动加载成功以后可以通过查看/proc/interrupts 文件来检查一下对应的中断有没有被注册上，输入如下命令：

```
cat /proc/interrupts
```

结果如图 51.4.2.1 所示：

```
 47:        803          0  stm32-exti-h-direct  30 Level     40010000.serial
 48:          0          0  stm32-exti-h-direct  70 Level     eth0
 49:          0          0  stm32-exti-h-direct  19 Level     5c004000.rtc
 50:          0          0  stm32-exti-h-direct  61 Edge      4c001000.mailbox
 51:          0          0     GIC-0 133 Level     4c001000.mailbox
 52:          0          0  stm32gpio   3 Edge      Key0_IRQ
IPI0:         0          0  CPU wakeup interrupts
IPI1:         0          0  Timer broadcast interrupts
```

图 31.4.2.1 proc/interrupts 文件内容

从图 31.4.2.1 可以看出 keyirq.c 驱动文件里面的 KEY0 中断已经存在了，触发方式为跳边沿(Edge)。

接下来使用如下命令来测试中断：

```
./keyirqApp /dev/key
```

按下开发板上的 KEY0 键，终端就会输出按键值，如图 31.4.2.2 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./keyirqApp /dev/key
Key Press
Key Release
Key Press
Key Release
Key Press
Key Release
Key Press
Key Release
```

图 31.4.2.2 读取到的按键值

从图 31.4.2.2 可以看出，按键值获取成功，并且不会有按键抖动导致的误判发生，说明按键消抖工作正常。如果要卸载驱动的话输入如下命令即可：

```
rmmod keyirq.ko
```

第三十二章 Linux 阻塞和非阻塞 IO 实验

阻塞和非阻塞 IO 是 Linux 驱动开发里面很常见的两种设备访问模式，在编写驱动的时候一定要考虑到阻塞和非阻塞。本章我们就来学习一下阻塞和非阻塞 IO，以及如何在驱动程序中处理阻塞与非阻塞，如何在驱动程序使用等待队列和 poll 机制。

32.1 阻塞和非阻塞 IO

32.1.1 阻塞和非阻塞简介

这里的“IO”并不是我们学习单片机的时候所说的“GPIO”(也就是引脚)。这里的 IO 指的是 Input/Output，也就是输入/输出，是应用程序对驱动设备的输入/输出操作。当应用程序对设备驱动进行操作的时候，如果不能获取到设备资源，那么阻塞式 IO 就会将应用程序对应的线程挂起，直到设备资源可以获取为止。对于非阻塞 IO，应用程序对应的线程不会挂起，它要么一直轮询等待，直到设备资源可以使用，要么就直接放弃。阻塞式 IO 如图 32.1.1.1 所示：

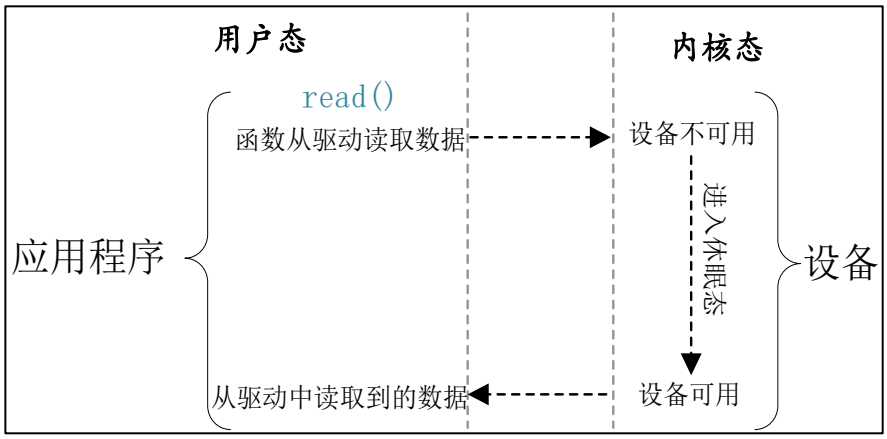


图 32.1.1.1 阻塞 IO 访问示意图。

图 32.1.1.1 中应用程序调用 read 函数从设备中读取数据，当设备不可用或数据未准备好的时候就会进入到休眠态。等设备可用的时候就会从休眠态唤醒，然后从设备中读取数据返回给应用程序。非阻塞 IO 如图 32.1.2 所示：

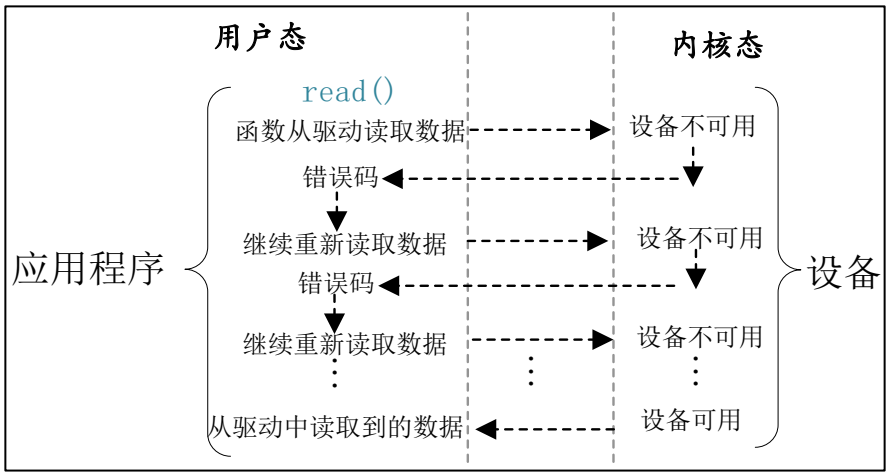


图 32.1.1.2 非阻塞 IO 访问示意图

从图 32.1.1.2 可以看出，应用程序使用非阻塞访问方式从设备读取数据，当设备不可用或数据未准备好的时候会立即向内核返回一个错误码，表示数据读取失败。应用程序会再次重新读取数据，这样一直往复循环，直到数据读取成功。

应用程序可以使用如下所示示例代码来实现阻塞访问：

示例代码 32.1.1.1 应用程序阻塞读取数据

```
1 int fd;
2 int data = 0;
```

```
3
4 fd = open("/dev/xxx_dev", O_RDWR);       /* 阻塞方式打开 */
5 ret = read(fd, &data, sizeof(data));     /* 读取数据     */
```

从示例代码 32.1.1.1 可以看出，对于设备驱动文件的默认读取方式就是阻塞式的，所以我们前面所有的例程测试 APP 都是采用阻塞 IO。

如果应用程序要采用非阻塞的方式来访问驱动设备文件，可以使用如下所示代码：

示例代码 32.1.1.2 应用程序非阻塞读取数据

```
1 int fd;
2 int data = 0;
3
4 fd = open("/dev/xxx_dev", O_RDWR | O_NONBLOCK);    /* 非阻塞方式打开 */
5 ret = read(fd, &data, sizeof(data));               /* 读取数据 */
```

第 4 行使用 open 函数打开“/dev/xxx_dev”设备文件的时候添加了参数“O_NONBLOCK”，表示以非阻塞方式打开设备，这样从设备中读取数据的时候就是非阻塞方式的了。

32.1.2 等待队列

1、等待队列头

阻塞访问最大的好处就是当设备文件不可操作的时候进程可以进入休眠态，这样可以将 CPU 资源让出来。但是，当设备文件可以操作的时候就必须唤醒进程，一般在中断函数里面完成唤醒工作。Linux 内核提供了等待队列(wait queue)来实现阻塞进程的唤醒工作，如果我们要在驱动中使用等待队列，必须创建并初始化一个等待队列头，等待队列头使用结构体 wait_queue_head 表示，wait_queue_head 结构体定义在文件 include/linux/wait.h 中，结构体内容如下所示：

示例代码 32.1.2.1 wait_queue_head 结构体

```
34 struct wait_queue_head {
35  spinlock_t              lock;
36  struct list_head        head;
37 };
38 typedef struct wait_queue_head wait_queue_head_t;
```

有些资料里面用 wait_queue_head_t 表示等待队列头，从 38 行可以看出，wait_queue_head_t 是 wait_queue_head 的别名，这么做的目的是为了兼容老版本代码。最新版本的系统都用 wait_queue_head 表示等待队列头，如果你的代码要考虑移植到老版本 linux 内核中，那么最好使用 wait_queue_head_t 表示等待队列头。

定义好等待队列头以后需要初始化，使用 init_waitqueue_head 函数初始化等待队列头，函数原型如下：

```
void init_waitqueue_head(struct wait_queue_head *wq_head)
```

参数 wq_head 就是要初始化的等待队列头。

也可以使用宏 DECLARE_WAIT_QUEUE_HEAD 来一次性完成等待队列头的定义的初始化。

2、等待队列项

等待队列头就是一个等待队列的头部，每个访问设备的进程都是一个队列项，当设备不可用的时候就要将这些进程对应的队列项添加到等待队列里面。在以前的 linux 版本中使用结构

体 wait_queue_t 表示等待队列项，在 5.4 版本的 linux 内核中已经删除了 wait_queue_t，取而代之的是 wait_queue_entry 结构体(其实只是换了个名字，结构体内的成员变量还是一样的)，wait_queue_entry 结构体内容如下：

示例代码 32.1.2.2 wait_queue_entry 结构体

```
27 struct wait_queue_entry {
28  unsigned int          flags;
29  void                  *private;
30  wait_queue_func_t     func;
31  struct list_head      entry;
32 };
```

使用宏 DECLARE_WAITQUEUE 定义并初始化一个等待队列项，宏的内容如下：

```
DECLARE_WAITQUEUE(name, tsk)
```

name 就是等待队列项的名字，tsk 表示这个等待队列项属于哪个任务(进程)，一般设置为 current，在 Linux 内核中 current 相当于一个全局变量，表示当前进程。因此宏 DECLARE_WAITQUEUE 就是给当前正在运行的进程创建并初始化了一个等待队列项。

3、将队列项添加/移除等待队列头

当设备不可访问的时候就需要将进程对应的等待队列项添加到前面创建的等待队列头中，只有添加到等待队列头中以后进程才能进入休眠态。当设备可以访问以后再将进程对应的等待队列项从等待队列头中移除即可，等待队列项添加 API 函数如下：

```
void add_wait_queue(struct wait_queue_head   *wq_head,
                    struct wait_queue_entry   *wq_entry)
```

函数参数和返回值含义如下：

wq_head：等待队列项要加入的等待队列头。

wq_entry：要加入的等待队列项。

返回值：无。

等待队列项移除 API 函数如下：

```
void remove_wait_queue(struct wait_queue_head     *wq_head,
                       struct wait_queue_entry    *wq_entry)
```

函数参数和返回值含义如下：

wq_head：要删除的等待队列项所处的等待队列头。

wq_entry：要删除的等待队列项。

返回值：无。

4、等待唤醒

当设备可以使用的时候就要唤醒进入休眠态的进程，唤醒可以使用如下两个函数：

```
void wake_up(struct wait_queue_head   *wq_head)
void wake_up_interruptible(struct wait_queue_head *wq_head)
```

参数 wq_head 就是要唤醒的等待队列头，这两个函数会将这个等待队列头中的所有进程都唤醒。wake_up 函数可以唤醒处于 TASK_INTERRUPTIBLE 和 TASK_UNINTERRUPTIBLE 状态的进程，而 wake_up_interruptible 函数只能唤醒处于 TASK_INTERRUPTIBLE 状态的进程。

5、等待事件

除了主动唤醒以外，也可以设置等待队列等待某个事件，当这个事件满足以后就自动唤醒等待队列中的进程，和等待事件有关的 API 函数如表 32.1.2.1 所示：

函数	描述
wait_event(wq_head, condition)	等待以 wq_head 为等待队列头的等待队列被唤醒，前提是 condition 条件必须满足(为真)，否则一直阻塞。此函数会将进程设置为 TASK_UNINTERRUPTIBLE 状态
wait_event_timeout(wq_head, condition, timeout)	功能和 wait_event 类似，但是此函数可以添加超时时间，以 jiffies 为单位。此函数有返回值，如果返回 0 的话表示超时时间到，而且 condition 为假。为 1 的话表示 condition 为真，也就是条件满足了。
wait_event_interruptible(wq_head, condition)	与 wait_event 函数类似，但是此函数将进程设置为 TASK_INTERRUPTIBLE，就是可以被信号打断。
wait_event_interruptible_timeout(wq_head, condition, timeout)	与 wait_event_timeout 函数类似，此函数也将进程设置为 TASK_INTERRUPTIBLE，可以被信号打断。

表 32.1.2.1 等待事件 API 函数

32.1.3 轮询

如果用户应用程序以非阻塞的方式访问设备，设备驱动程序就要提供非阻塞的处理方式，也就是轮询。poll、epoll 和 select 可以用于处理轮询，应用程序通过 select、epoll 或 poll 函数来查询设备是否可以操作，如果可以操作的话就从设备读取或者向设备写入数据。当应用程序调用 select、epoll 或 poll 函数的时候设备驱动程序中的 poll 函数就会执行，因此需要在设备驱动程序中编写 poll 函数。我们先来看一下应用程序中使用的 select、poll 和 epoll 这三个函数。

1、select 函数

select 函数原型如下：

```
int select(int             nfds,
           fd_set          *readfds,
           fd_set          *writefds,
           fd_set          *exceptfds,
           struct timeval  *timeout)
```

函数参数和返回值含义如下：

nfds：所要监视的这三类文件描述集合中，最大文件描述符加 1。

readfds、writefds 和 exceptfds：这三个指针指向描述符集合，这三个参数指明了关心哪些描述符、需要满足哪些条件等等，这三个参数都是 fd_set 类型的，fd_set 类型变量的每一个位都代表了一个文件描述符。readfds 用于监视指定描述符集的读变化，也就是监视这些文件是否可以读取，只要这些集合里面有一个文件可以读取那么 seclect 就会返回一个大于 0 的值表示文件可以读取。如果没有文件可以读取，那么就会根据 timeout 参数来判断是否超时。可以将 readfs 设置为 NULL，表示不关心任何文件的读变化。writefds 和 readfs 类似，只是 writefs 用于监视这些文件是否可以进行写操作。exceptfds 用于监视这些文件的异常。

比如我们现在要从一个设备文件中读取数据，那么就可以定义一个 fd_set 变量，这个变量要传递给参数 readfds。当我们定义好一个 fd_set 变量以后可以使用如下所示几个宏进行操作：

```
void FD_ZERO(fd_set *set)
void FD_SET(int fd, fd_set *set)
void FD_CLR(int fd, fd_set *set)
int   FD_ISSET(int fd, fd_set *set)
```

FD_ZERO 用于将 fd_set 变量的所有位都清零，FD_SET 用于将 fd_set 变量的某个位置 1，也就是向 fd_set 添加一个文件描述符，参数 fd 就是要加入的文件描述符。FD_CLR 用户将 fd_set 变量的某个位清零，也就是将一个文件描述符从 fd_set 中删除，参数 fd 就是要删除的文件描述符。FD_ISSET 用于测试一个文件是否属于某个集合，参数 fd 就是要判断的文件描述符。

timeout:超时时间，当我们调用 select 函数等待某些文件描述符可以设置超时时间，超时时间使用结构体 timeval 表示，结构体定义如下所示：

```
struct timeval {
    long    tv_sec;         /* 秒     */
    long    tv_usec;        /* 微妙    */
};
```

当 timeout 为 NULL 的时候就表示无限期的等待。

返回值：0，表示的话就表示超时发生，但是没有任何文件描述符可以进行操作；-1，发生错误；其他值，可以进行操作的文件描述符个数。

使用 select 函数对某个设备驱动文件进行读非阻塞访问的操作示例如下所示：

示例代码 32.1.3.1 select 函数非阻塞读访问示例

```
1  void main(void)
2  {
3      int ret, fd;                /* 要监视的文件描述符          */
4      fd_set readfds;             /* 读操作文件描述符集          */
5      struct timeval timeout;     /* 超时结构体                  */
6
7      fd = open("dev_xxx", O_RDWR | O_NONBLOCK);    /* 非阻塞式访问 */
8
9      FD_ZERO(&readfds);          /* 清除 readfds                */
10     FD_SET(fd, &readfds);       /* 将 fd 添加到 readfds 里面   */
11
12     /* 构造超时时间 */
13     timeout.tv_sec = 0;
14     timeout.tv_usec = 500000;   /* 500ms   */
15
16     ret = select(fd + 1, &readfds, NULL, NULL, &timeout);
17     switch (ret) {
18         case 0:                 /* 超时         */
19             printf("timeout!\r\n");
20             break;
21         case -1:                /* 错误         */
22             printf("error!\r\n");
```

```
23              break;
24          default:        /* 可以读取数据 */
25              if(FD_ISSET(fd, &readfds)) {  /* 判断是否为 fd 文件描述符 */
26                  /* 使用 read 函数读取数据 */
27              }
28               break;
29      }
30  }
```

2、poll 函数

在单个线程中，select 函数能够监视的文件描述符数量有最大的限制，一般为 1024，可以修改内核将监视的文件描述符数量改大，但是这样会降低效率！这个时候就可以使用 poll 函数，poll 函数本质上和 select 没有太大的差别，但是 poll 函数没有最大文件描述符限制，Linux 应用程序中 poll 函数原型如下所示：

```
int poll(struct pollfd       *fds,
         nfds_t              nfds,
         int                 timeout)
```

函数参数和返回值含义如下：

fds：要监视的文件描述符集合以及要监视的事件,为一个数组，数组元素都是结构体 pollfd 类型的，pollfd 结构体如下所示：

```
struct pollfd {
    int     fd;         /* 文件描述符     */
    short events;       /* 请求的事件     */
    short revents;      /* 返回的事件     */
};
```

fd 是要监视的文件描述符，如果 fd 无效的话那么 events 监视事件也就无效，并且 revents 返回 0。events 是要监视的事件，可监视的事件类型如下所示：

```
POLLIN          有数据可以读取。
POLLPRI         有紧急的数据需要读取。
POLLOUT         可以写数据。
POLLERR         指定的文件描述符发生错误。
POLLHUP         指定的文件描述符挂起。
POLLNVAL        无效的请求。
POLLRDNORM      等同于 POLLIN
```

revents 是返回参数，也就是返回的事件，由 Linux 内核设置具体的返回事件。

nfds：poll 函数要监视的文件描述符数量。

timeout：超时时间，单位为 ms。

返回值：返回 revents 域中不为 0 的 pollfd 结构体个数，也就是发生事件或错误的文件描述符数量；0，超时；-1，发生错误，并且设置 errno 为错误类型。

使用 poll 函数对某个设备驱动文件进行读非阻塞访问的操作示例如下所示：

示例代码 32.1.3.2 poll 函数读非阻塞访问示例

```
1  void main(void)
2  {
3      int ret;
```

```
4       int fd;                     /* 要监视的文件描述符       */
5       struct pollfd fds;
6
7       fd = open(filename, O_RDWR | O_NONBLOCK); /* 非阻塞式访问 */
8
9       /* 构造结构体    */
10      fds.fd = fd;
11      fds.events = POLLIN;    /* 监视数据是否可以读取 */
12
13      ret = poll(&fds, 1, 500);    /* 轮询文件是否可操作，超时 500ms */
14      if (ret) {                   /* 数据有效      */
15          ......
16          /* 读取数据 */
17          ......
18      } else if (ret == 0) {       /* 超时          */
19          ......
20      } else if (ret < 0) {        /* 错误          */
21          ......
22      }
23  }
```

3、epoll 函数

传统的 selcet 和 poll 函数都会随着所监听的 fd 数量的增加，出现效率低下的问题，而且 poll 函数每次必须遍历所有的描述符来检查就绪的描述符，这个过程很浪费时间。为此，epoll 应运而生，epoll 就是为处理大并发而准备的，一般常常在网络编程中使用 epoll 函数。应用程序需要先使用 epoll_create 函数创建一个 epoll 句柄，epoll_create 函数原型如下：

```
int epoll_create(int size)
```

函数参数和返回值含义如下：

size：从 Linux2.6.8 开始此参数已经没有意义了，随便填写一个大于 0 的值就可以。

返回值：epoll 句柄，如果为-1 的话表示创建失败。

epoll 句柄创建成功以后使用 epoll_ctl 函数向其中添加要监视的文件描述符以及监视的事件，epoll_ctl 函数原型如下所示：

```
int epoll_ctl(int                   epfd,
              int                   op,
              int                   fd,
              struct epoll_event    *event)
```

函数参数和返回值含义如下：

epfd：要操作的 epoll 句柄，也就是使用 epoll_create 函数创建的 epoll 句柄。

op：表示要对 epfd(epoll 句柄)进行的操作，可以设置为：

```
EPOLL_CTL_ADD     向 epfd 添加文件参数 fd 表示的描述符。
EPOLL_CTL_MOD     修改参数 fd 的 event 事件。
EPOLL_CTL_DEL     从 epfd 中删除 fd 描述符。
```

fd：要监视的文件描述符。

event：要监视的事件类型，为 epoll_event 结构体类型指针，epoll_event 结构体类型如下所示：

```
struct epoll_event {
    uint32_t     events;     /* epoll 事件     */
    epoll_data_t data;       /*  用户数据      */
};
```

结构体 epoll_event 的 events 成员变量表示要监视的事件，可选的事件如下所示：

```
EPOLLIN      有数据可以读取。
EPOLLOUT     可以写数据。
EPOLLPRI     有紧急的数据需要读取。
EPOLLERR     指定的文件描述符发生错误。
EPOLLHUP     指定的文件描述符挂起。
EPOLLET      设置 epoll 为边沿触发，默认触发模式为水平触发。
EPOLLONESHOT 一次性的监视，当监视完成以后还需要再次监视某个 fd，那么就需要将 fd 重新添加到 epoll 里面。
```

上面这些事件可以进行“或”操作，也就是说可以设置监视多个事件。

返回值：0，成功；-1，失败，并且设置 errno 的值为相应的错误码。

一切都设置好以后应用程序就可以通过 epoll_wait 函数来等待事件的发生，类似 select 函数。epoll_wait 函数原型如下所示：

```
int epoll_wait(int                   epfd,
               struct epoll_event    *events,
               int                   maxevents,
               int                   timeout)
```

函数参数和返回值含义如下：

epfd：要等待的 epoll。

events：指向 epoll_event 结构体的数组，当有事件发生的时候 Linux 内核会填写 events，调用者可以根据 events 判断发生了哪些事件。

maxevents：events 数组大小，必须大于 0。

timeout：超时时间，单位为 ms。

返回值：0，超时；-1，错误；其他值，准备就绪的文件描述符数量。

epoll 更多的是用在大规模的并发服务器上，因为在这种场合下 select 和 poll 并不适合。当设计到的文件描述符(fd)比较少的时候就适合用 selcet 和 poll，本章我们就使用 sellect 和 poll 这两个函数。

32.1.4 Linux 驱动下的 poll 操作函数

当应用程序调用 select 或 poll 函数来对驱动程序进行非阻塞访问的时候，驱动程序 file_operations 操作集中的 poll 函数就会执行。所以驱动程序的编写者需要提供对应的 poll 函数，poll 函数原型如下所示：

```
unsigned int (*poll) (struct file *filp, struct poll_table_struct *wait)
```

函数参数和返回值含义如下：

filp：要打开的设备文件(文件描述符)。

wait：结构体 poll_table_struct 类型指针，由应用程序传递进来的。一般将此参数传递给 poll_wait 函数。

返回值:向应用程序返回设备或者资源状态，可以返回的资源状态如下：

POLLIN	有数据可以读取。
POLLPRI	有紧急的数据需要读取。
POLLOUT	可以写数据。
POLLERR	指定的文件描述符发生错误。
POLLHUP	指定的文件描述符挂起。
POLLNVAL	无效的请求。
POLLRDNORM	等同于 POLLIN，普通数据可读

我们需要在驱动程序的 poll 函数中调用 poll_wait 函数，poll_wait 函数不会引起阻塞，只是将应用程序添加到 poll_table 中，poll_wait 函数原型如下：

```
void poll_wait(struct file * filp, wait_queue_head_t * wait_address, poll_table *p)
```

参数 wait_address 是要添加到 poll_table 中的等待队列头，参数 p 就是 poll_table，就是 file_operations 中 poll 函数的 wait 参数。

32.2 阻塞 IO 实验

在上一章 Linux 中断实验中，我们直接在应用程序中通过 read 函数不断的读取按键状态，当按键有效的时候就打印出按键值。这种方法有个缺点，那就是 keyirqApp 这个测试应用程序拥有很高的 CPU 占用率，大家可以在开发板中加载上一章的驱动程序模块 keyirq.ko，然后以后台运行模式打开 keyirqApp 这个测试软件，命令如下：

```
./keyirqApp /dev/keyirq &
```

测试驱动是否正常工作，如果驱动工作正常的话输入“top”命令查看 keyirqApp 这个应用程序的 CPU 使用率，结果如图 32.2.1 所示：

```
Mem: 40492K used, 850996K free, 48K shrd, 0K buff, 5944K cached
CPU:   2% usr  51% sys   0% nic  46% idle   0% io   0% irq   0% sirq
Load average: 0.66 0.27 0.14 3/62 309
  PID  PPID USER     STAT   VSZ %VSZ %CPU COMMAND
  307   191 root     R     1380   0%  50% ./keyirqApp /dev/key
   21     2 root     IW       0   0%   2% [kworker/0:1-eve]
  309   191 root     R     2344   0%   1% top
```

图 32.2.1 CPU 使用率

从图 32.2.1 可以看出，keyirqApp 这个应用程序的 CPU 使用率竟然高达 50%，这仅仅是一个读取按键值的应用程序，这么高的 CPU 使用率显然是有问题的！原因就在于我们是直接在 while 循环中通过 read 函数读取按键值，因此 keyirqApp 这个软件会一直运行，一直读取按键值，CPU 使用率肯定就会很高。最好的方法就是在没有有效的按键事件发生的时候，keyirqApp 这个应用程序应该处于休眠状态，当有按键事件发生以后 keyirqApp 这个应用程序才运行，打印出按键值，这样就会降低 CPU 使用率，本小节我们就使用阻塞 IO 来实现此能。

32.2.1 硬件原理图分析

本章实验硬件原理图参考 29.2 小节即可。

32.2.2 实验程序编写

1、驱动程序编写

本实验对应的例程路径为：开发板光盘-> 2、Linux 驱动例程-> 14_blockio。

本章实验我们在上一章的“13_irq”实验的基础上完成，主要是对其添加阻塞访问相关的代码。新建名为“14_blockio”的文件夹，然后在 14_blockio 文件夹里面创建 vscode 工程，工作区命名为“blockio”。将“13_irq”实验中的 keyirq.c 复制到 14_blockio 文件夹中，并重命名为 blockio.c。接下来我们就修改 blockio.c 这个文件，在其中添加阻塞相关的代码，完成以后的 blockio.c 内容如下所示(因为是在上一章实验的 keyirq.c 文件的基础上修改的，为了减少篇幅，下面的代码有省略)：

示例代码 32.2.2.1 blockio.c 文件代码(有省略)

```
41  /* key 设备结构体 */
42  struct key_dev{
43      dev_t devid;                /* 设备号                   */
44      struct cdev cdev;           /* cdev                     */
45      struct class *class;        /* 类                       */
46      struct device *device;      /* 设备                     */
47      struct device_node  *nd;    /* 设备节点                 */
48      int key_gpio;               /* key 所使用的 GPIO 编号   */
49      struct timer_list timer;    /* 按键值                   */
50      int irq_num;                /* 中断号                   */
51
52      atomic_t status;            /* 按键状态                 */
53      wait_queue_head_t r_wait;   /* 读等待队列头             */
54  };
......
150 static void key_timer_function(struct timer_list *arg)
151 {
152     static int last_val = 1;
153     int current_val;
154
155     /* 读取按键值并判断按键当前状态 */
156     current_val = gpio_get_value(key.key_gpio);
157     if (0 == current_val && last_val){
158         atomic_set(&key.status, KEY_PRESS);    /* 按下     */
159         wake_up_interruptible(&key.r_wait);    /* 唤醒     */
160     }
161     else if (1 == current_val && !last_val) {
162         atomic_set(&key.status, KEY_RELEASE); /* 松开     */
163         wake_up_interruptible(&key.r_wait);    /* 唤醒     */
164     }
165     else
166         atomic_set(&key.status, KEY_KEEP);     /* 状态保持 */
167
168     last_val = current_val;
169 }
```

```
183 /*
184  * @description   : 从设备读取数据
185  * @param - filp  : 要打开的设备文件(文件描述符)
186  * @param - buf   : 返回给用户空间的数据缓冲区
187  * @param - cnt   : 要读取的数据长度
188  * @param - offt  : 相对于文件首地址的偏移
189  * @return        : 读取的字节数，如果为负值，表示读取失败
190  */
191 static ssize_t key_read(struct file *filp, char __user *buf,
192                         size_t cnt, loff_t *offt)
193 {
194     int ret;
195 
196     /* 加入等待队列，当有按键按下或松开动作发生时，才会被唤醒 */
197     ret = wait_event_interruptible(key.r_wait, KEY_KEEP !=
                                       atomic_read(&key.status));
198     if(ret)
199         return ret;
200 
201     /* 将按键状态信息发送给应用程序 */
202     ret = copy_to_user(buf, &key.status, sizeof(int));
203 
204     /* 状态重置                    */
205     atomic_set(&key.status, KEY_KEEP);
206 
207     return ret;
208 }
......
242 /*
243  * @description   : 驱动入口函数
244  * @param         : 无
245  * @return        : 无
246  */
247 static int __init mykey_init(void)
248 {
249     int ret;
250 
251     /* 初始化等待队列头 */
252     init_waitqueue_head(&key.r_wait);
253 
254     /* 初始化按键状态 */
255     atomic_set(&key.status, KEY_KEEP);
......
```

```
311 }
......
```

第 42~54 行，我们删除了设备结构体 struct key_dev 中的自旋锁变量，本章驱动代码我们不用自旋锁，而改成第 52 行的原子变量来实现对相应变量的保护操作。第 53 行添加了一个等待队列头 r_wait，因为在 Linux 驱动中处理阻塞 IO 需要用到等待队列，而等待队列会使进程进入休眠状态，所以不能使用自旋锁！

第 150~169 行，定时器定时处理函数 key_timer_function，对按键状态进行判断，如果是按下动作或是松开动作则会使用 wake_up_interruptible 函数唤醒等待队列，并且将 status 变量设置为 KEY_PRESS 或 KEY_RELEASE；这样在 key_read 函数中阻塞的进程就会解除阻塞继续进行下面的操作。

第 191~208 行，key_read 函数，在这个函数中我们会判断按键是否有按下或松开动作发生时，如果没有则调用 wait_event_interruptible 把它加入等待队列当中，进行阻塞。如果等待队列被唤醒并且条件“KEY_KEEP != atomic_read(&status)”成立，则解除阻塞，继续下面的操作，也就是读取按键状态数据将其发送给应用程序。因为采用了 wait_event_interruptible 函数，因此进入休眠态的进程可以被信号打断。在该函数中我们读取 status 原子变量必须要使用 atomic_read 函数进行操作。

第 252 行，在驱动入口函数中我们会调用 init_waitqueue_head 初始化等待队列头；第 255 行使用原子操作 atomic_set 设置按键的初始状态 status 为 KEY_KEEP。

使用等待队列实现阻塞访问重点注意两点：

①、将任务或者进程加入到等待队列头，

②、在合适的点唤醒等待队列，一般是中断处理函数里面。

2、编写测试 APP

本节实验的测试 APP 直接使用 13_irq 实验目录下的 keyirqApp.c 测试程序，将 keyirqApp.c 复制到本实验目录 14_blockio 中，不需要修改任何内容。

32.2.3 运行测试

1、编译驱动程序和测试 APP

①、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 blockio.o，Makefile 内容如下所示：

示例代码 32.2.3.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := blockio.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 blockio.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“blockio.ko”的驱动模块文件。

②、编译测试 APP

输入如下命令编译测试 blockioApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc blockioApp.c -o blockioApp
```

编译成功以后就会生成 blcokioApp 这个应用程序。

2、运行测试

将上一小节编译出来 blockio.ko 和 blockioApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 blockio.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe blockio.ko     //加载驱动
```

驱动加载成功以后使用如下命令打开 blockioApp 这个测试 APP，并且以后台模式运行：

```
./blockioApp /dev/key &
```

按下开发板上的 KEY0 按键，结果如图 32.2.3.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./blockioApp /dev/key &
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ Key Press
Key Release
Key Press
Key Release
Key Press
Key Release
```

图 32.2.3.1 测试 APP 运行测试

当按下 KEY0 按键以后 blockioApp 这个测试 APP 就会打印出按键状态。输入“top”命令，查看 blockioAPP 这个应用 APP 的 CPU 使用率，如图 32.2.3.2 所示：

```
Mem: 41296K used, 850192K free, 48K shrd, 0K buff, 6628K cached
CPU:   0% usr   1% sys   0% nic  98% idle   0% io   0% irq   0% sirq
Load average: 0.13 0.49 0.79 1/64 340
  PID  PPID USER     STAT   VSZ %VSZ %CPU COMMAND
  324     2 root     IW       0   0%   1% [kworker/0:0-pm]
  340   191 root     R     2344   0%   0% top
  183     1 root     S     5512   1%   0% /usr/sbin/sshd
  189     1 root     S     4520   1%   0% vsftpd
  151     1 root     S     2604   0%   0% /sbin/mdev -df
  191     1 root     S     2344   0%   0% -sh
    1     0 root     S     2224   0%   0% init
  135     1 root     S     2224   0%   0% /sbin/syslogd -n
  139     1 root     S     2224   0%   0% /sbin/klogd -n
  338   191 root     S     1380   0%   0% ./blockioApp /dev/key
   21     2 root     IW       0   0%   0% [kworker/0:1-eve]
```

图 32.2.3.2 应用程序 CPU 使用率

从图 32.2.3.2 可以看出，当我们在按键驱动程序里面加入阻塞访问以后，blockioApp 这个应用程序的 CPU 使用率从图 32.2.1 中的 48%降低到了 0%。大家注意，这里的 0%并不是说 blockioApp 这个应用程序不使用 CPU 了，只是因为使用率太小了，CPU 使用率可能为 0.00001%，但是图 32.2.3.2 是没有小数点显示的，因此就显示成了 0%。

我们可以使用“kill”命令关闭后台运行的应用程序，比如我们关闭掉 blockioApp 这个后台运行的应用程序。首先输出“Ctrl+C”关闭 top 命令界面，进入到命令行模式。然后使用“ps”命令查看一下 blockioApp 这个应用程序的 PID，如图 32.2.3.3 所示：

```
  333 root      [kworker/u4:4-rp]
  337 root      [kworker/1:2-eve]
  338 root      ./blockioApp /dev/key
  339 root      [kworker/0:2-nfs]
```

图 32.2.3.3 当前系统所有进程的 ID

从图图 32.2.3.3 可以看出，blockioApp 这个应用程序的 PID 为 338，使用“kill -9 PID”即可“杀死”指定 PID 的进程，比如我们现在要“杀死”PID 为 338 的 blockioApp 应用程序，可是使用如下命令：

```
kill -9 338
```

输入上述命令以后终端显示如图 32.2.3.4 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ kill -9 338
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
[1]+  Killed                  ./blockioApp /dev/key
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 32.2.3.4 kill 命令输出结果

从图 32.2.3.4 可以看出，“./blockioApp /dev/key”这个应用程序已经被“杀掉”了，在此输入“ps”命令查看当前系统运行的进程，会发现 blockioApp 已经不见了。这个就是使用 kill 命令“杀掉”指定进程的方法。

32.3 非阻塞 IO 实验

32.3.1 硬件原理图分析

本章实验硬件原理图参考 29.2 小节即可。

32.3.2 实验程序编写

1、驱动程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→15_noblockio。

本章实验我们在 32.2 小节中的“14_blockio”实验的基础上完成，上一小节实验我们已经在驱动中添加了阻塞 IO 的代码，本小节我们继续完善驱动，加入非阻塞 IO 驱动代码。新建名为“15_noblockio”的文件夹，然后在 15_noblockio 文件夹里面创建 vscode 工程，工作区命名为“noblockio”。将“14_blockio”实验中的 blockio.c 复制到 15_noblockio 文件夹中，并重命名为 noblockio.c。接下来我们就修改 noblockio.c 这个文件，在其中添加非阻塞相关的代码，完成以后的 noblockio.c 内容如下所示(因为是在上一小节实验的 blockio.c 文件的基础上修改的，为了减少篇幅，下面的代码有省略)：

示例代码 32.3.3.1 noblockio.c 文件(有省略)

```
......
27  #include <linux/wait.h>
28  #include <linux/poll.h>
29  #include <asm/mach/map.h>
30  #include <asm/uaccess.h>
31  #include <asm/io.h>
32
```

```
......
/*
 * @description   : 从设备读取数据
 * @param - filp  : 要打开的设备文件(文件描述符)
 * @param - buf   : 返回给用户空间的数据缓冲区
 * @param - cnt   : 要读取的数据长度
 * @param - offt  : 相对于文件首地址的偏移
 * @return        : 读取的字节数，如果为负值，表示读取失败
 */
static ssize_t key_read(struct file *filp, char __user *buf,
                        size_t cnt, loff_t *offt)
{
    int ret;

    if (filp->f_flags & O_NONBLOCK) {   /* 非阻塞方式访问 */
        if(KEY_KEEP == atomic_read(&key.status))
            return -EAGAIN;
    } else {                                /* 阻塞方式访问 */
    /* 加入等待队列，当有按键按下或松开动作发生时，才会被唤醒 */
    ret = wait_event_interruptible(key.r_wait, KEY_KEEP !=
                                   atomic_read(&key.status));
    if(ret)
        return ret;
    }
    /* 将按键状态信息发送给应用程序  */
    ret = copy_to_user(buf, &key.status, sizeof(int));

    /* 状态重置                     */
    atomic_set(&key.status, KEY_KEEP);

    return ret;
}
......
/*
 * @description   : poll 函数，用于处理非阻塞访问
 * @param - filp  : 要打开的设备文件(文件描述符)
 * @param - wait  : 等待列表(poll_table)
 * @return        : 设备或者资源状态，
 */
static unsigned int key_poll(struct file *filp,
                             struct poll_table_struct *wait)
{
    unsigned int mask = 0;
```

```
248
249     poll_wait(filp, &key.r_wait, wait);
250
251     if(KEY_KEEP != atomic_read(&key.status))/* 按键按下或松开动作发生 */
252         mask = POLLIN | POLLRDNORM;             /* 返回 PLLIN */
253
254     return mask;
255 }
256
257 /* 设备操作函数 */
258 static struct file_operations key_fops = {
259     .owner = THIS_MODULE,
260     .open = key_open,
261     .read = key_read,
262     .write = key_write,
263     .release =  key_release,
264     .poll = key_poll,
265 };
......
```

第 28 行，使用 include 将内核源码目录 include/linux/poll.h 头文件包含进来。

第 193~214 行，key_read 函数中判断是否为非阻塞式读取访问，如果是的话就判断按键状态是否有效，也就是判断是否产生了按下或松开这样的动作，如果没有的话就返回-EAGAIN。

第 245~255 行，key_poll 函数就是 file_operations 驱动操作集中的 poll 函数，当应用程序调用 select 或者 poll 函数的时候 key_poll 函数就会执行。第 249 行调用 poll_wait 函数将等待队列头添加到 poll_table 中，第 251~252 行判断按键是否有效，如果按键有效的话就向应用程序返回 POLLIN 这个事件，表示有数据可以读取。

第 264 行，设置 file_operations 的 poll 成员变量为 key_poll。

2、编写测试 APP

新建名为 noblockioApp.c 测试 APP 文件，然后在其中输入如下所示内容：

示例代码 32.3.3.2 noblockioApp.c 文件代码

```
12  #include <stdio.h>
13  #include <unistd.h>
14  #include <sys/types.h>
15  #include <sys/stat.h>
16  #include <fcntl.h>
17  #include <stdlib.h>
18  #include <string.h>
19  #include <poll.h>
20
21  /*
22   * @description   : main 主程序
23   * @param - argc  : argv 数组元素个数
24   * @param - argv  : 具体参数
```

```
25   * @return         : 0 成功;其他 失败
26   */
27  int main(int argc, char *argv[])
28  {
29      fd_set readfds;
30      int key_val;
31      int fd;
32      int ret;
33
34      /* 判断传参个数是否正确 */
35      if(2 != argc) {
36          printf("Usage:\n"
37                 "\t./keyApp /dev/key\n"
38                );
39          return -1;
40      }
41
42      /* 打开设备 */
43      fd = open(argv[1], O_RDONLY | O_NONBLOCK);
44      if(0 > fd) {
45          printf("ERROR: %s file open failed!\n", argv[1]);
46          return -1;
47      }
48
49      FD_ZERO(&readfds);
50      FD_SET(fd, &readfds);
51
52      /* 循环轮训读取按键数据 */
53      for ( ; ; ) {
54
55          ret = select(fd + 1, &readfds, NULL, NULL, NULL);
56          switch (ret) {
57
58          case 0:     /* 超时 */
59              /* 用户自定义超时处理 */
60              break;
61
62          case -1:    /* 错误 */
63              /* 用户自定义错误处理 */
64              break;
65
66          default:
67              if(FD_ISSET(fd, &readfds)) {
```

```
68                 read(fd, &key_val, sizeof(int));
69                 if (0 == key_val)
70                     printf("Key Press\n");
71                 else if (1 == key_val)
72                     printf("Key Release\n");
73             }
74
75             break;
76         }
77     }
78
79     /* 关闭设备 */
80     close(fd);
81     return 0;
82 }
```

第 53~77 行，在本测试程序中我们使用 select 函数来实现非阻塞访问，在 for 循环中使用 select 函数不断的轮询，检查驱动程序是否有数据可以读取，如果可以读取的话就调用 read 函数读取按键数据。大家也可以试试使用 poll 函数来实现！

32.3.3 运行测试

1、编译驱动程序和测试 APP

①、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 noblockio.o，Makefile 内容如下所示：

示例代码 32.3.3.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := noblockio.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 noblockio.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“noblockio.ko”的驱动模块文件。

②、编译测试 APP

输入如下命令编译测试 noblockioApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc noblockioApp.c -o noblockioApp
```

编译成功以后就会生成 noblcokioApp 这个应用程序。

2、运行测试

将上一小节编译出来 noblockio.ko 和 noblockioApp 这两个文件拷贝到

rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 blockio.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe noblockio.ko  //加载驱动
```

驱动加载成功以后使用如下命令打开 noblockioApp 这个测试 APP，并且以后台模式运行：

```
./noblockioApp /dev/key &
```

按下开发板上的 KEY0 按键，结果如图 32.3.3.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./noblockioApp /dev/key &
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ Key Press
Key Release
Key Press
Key Release
Key Press
Key Release
```

图 32.3.3.1 测试 APP 运行测试

当按下 KEY0 按键以后 noblockioApp 这个测试 APP 就会打印出按键信息。输入“top”命令，查看 noblockioAPP 这个应用 APP 的 CPU 使用率，如图 32.3.3.2 所示：

```
Mem: 41296K used, 850192K free, 48K shrd, 0K buff, 6864K cached
CPU:   0% usr   4% sys   0% nic  95% idle   0% io   0% irq   0% sirq
Load average: 0.08 0.11 0.13 1/62 367
  PID  PPID USER     STAT   VSZ %VSZ %CPU COMMAND
  367   191 root     R     2224   0%   5% top
  183     1 root     S     5512   1%   0% /usr/sbin/sshd
  189     1 root     S     4520   1%   0% vsftpd
  151     1 root     S     2604   0%   0% /sbin/mdev -df
  191     1 root     S     2344   0%   0% -sh
    1     0 root     S     2224   0%   0% init
  135     1 root     S     2224   0%   0% /sbin/syslogd -n
  139     1 root     S     2224   0%   0% /sbin/klogd -n
  366   191 root     S     1380   0%   0% ./noblockioApp /dev/key
  324     2 root     IW       0   0%   0% [kworker/0:0-eve]
   10     2 root     IW       0   0%   0% [rcu_preempt]
```

图 32.3.3.2 应用程序 CPU 使用率

从图 32.3.3.2 可以看出，采用非阻塞方式读处理以后，noblockioApp 的 CPU 占用率也低至 0%，和图 32.2.3.2 中的 blockioApp 一样，这里的 0%并不是说 noblockioApp 这个应用程序不使用 CPU 了，只是因为使用率太小了。

如果要“杀掉”处于后台运行模式的 noblockioApp 这个应用程序，可以参考 32.2.3 小节讲解的方法。

第三十三章 异步通知实验

在前面使用阻塞或者非阻塞的方式来读取驱动中按键值都是应用程序主动读取的，对于非阻塞方式来说还需要应用程序通过 poll 函数不断的轮询。最好的方式就是驱动程序能主动向应用程序发出通知，报告自己可以访问，然后应用程序再从驱动程序中读取或写入数据，类似于中断。Linux 提供了异步通知这个机制来完成此功能，本章我们就来学习一下异步通知以及如何在驱动中添加异步通知相关处理代码。

33.1 异步通知

33.1.1 异步通知简介

我们首先来回顾一下“中断”，中断是处理器提供的一种异步机制，我们配置好中断以后就可以让处理器去处理其他的事情了，当中断发生以后会触发我们事先设置好的中断服务函数，在中断服务函数中做具体的处理。比如我们以前学习单片机的时候用到的 GPIO 按键中断，我们通过按键去开关蜂鸣器，采用中断以后处理器就不需要时刻的去查看按键有没有被按下，因为按键按下以后会自动触发中断。同样的，Linux 应用程序可以通过阻塞或者非阻塞这两种方式来访问驱动设备，通过阻塞方式访问的话应用程序会处于休眠态，等待驱动设备可以使用。非阻塞方式的话会通过 poll 函数来不断的轮询，查看驱动设备文件是否可以使用。这两种方式都需要应用程序主动的去查询设备的使用情况，如果能提供一种类似中断的机制，当驱动程序可以访问的时候主动告诉应用程序那就最好了。

“信号”为此应运而生，信号类似于我们硬件上使用的“中断”，只不过信号是软件层次上的。算是在软件层次上对中断的一种模拟，驱动可以通过主动向应用程序发送信号的方式来报告自己可以访问了，应用程序获取到信号以后就可以从驱动设备中读取或者写入数据了。整个过程就相当于应用程序收到了驱动发送过来了的一个中断，然后应用程序去响应这个中断，在整个处理过程中应用程序并没有去查询驱动设备是否可以访问，一切都是由驱动设备自己告诉给应用程序的。

阻塞、非阻塞、异步通知，这三种是针对不同的场合提出来的不同的解决方法，没有优劣之分，在实际的工作和学习中，根据自己的实际需求选择合适的处理方法即可。

异步通知的核心就是信号，在 arch/xtensa/include/uapi/asm/signal.h 文件中定义了 Linux 所支持的所有信号，这些信号如下所示：

示例代码 33.1.1.1 Linux 信号

```
18 #define SIGHUP      1     /* 终端挂起或控制进程终止          */
19 #define SIGINT      2     /* 终端中断(Ctrl+C 组合键)         */
20 #define SIGQUIT     3     /* 终端退出(Ctrl+\组合键)          */
21 #define SIGILL      4     /* 非法指令                        */
22 #define SIGTRAP     5     /* debug 使用，有断点指令产生      */
23 #define SIGABRT     6     /* 由 abort(3)发出的退出指令       */
24 #define SIGIOT      6     /* IOT 指令                        */
25 #define SIGBUS      7     /* 总线错误                        */
26 #define SIGFPE      8     /* 浮点运算错误                    */
27 #define SIGKILL     9     /* 杀死、终止进程                  */
28 #define SIGUSR1     10    /* 用户自定义信号 1                */
30 #define SIGSEGV     11    /* 段违例(无效的内存段)            */
31 #define SIGUSR2     12    /* 用户自定义信号 2                */
32 #define SIGPIPE     13    /* 向非读管道写入数据              */
33 #define SIGALRM     14    /* 闹钟                            */
34 #define SIGTERM     15    /* 软件终止                        */
35 #define SIGSTKFLT   16    /* 栈异常                          */
36 #define SIGCHLD     17    /* 子进程结束                      */
37 #define SIGCONT     18    /* 进程继续                        */
38 #define SIGSTOP     19    /* 停止进程的执行，只是暂停        */
```

```
39 #define SIGTSTP       20      /* 停止进程的运行(Ctrl+Z 组合键)     */
40 #define SIGTTIN       21      /* 后台进程需要从终端读取数据         */
41 #define SIGTTOU       22      /* 后台进程需要向终端写数据       */
42 #define SIGURG        23      /* 有"紧急"数据                   */
43 #define SIGXCPU       24      /* 超过 CPU 资源限制              */
44 #define SIGXFSZ       25      /* 文件大小超额                   */
45 #define SIGVTALRM     26      /* 虚拟时钟信号                   */
46 #define SIGPROF       27      /* 时钟信号描述                   */
47 #define SIGWINCH      28      /* 窗口大小改变                   */
48 #define SIGIO         29      /* 可以进行输入/输出操作          */
49 #define SIGPOLL       SIGIO
50 /* #define SIGLOS    29 */
51 #define SIGPWR        30      /* 断点重启                       */
52 #define SIGSYS        31      /* 非法的系统调用                 */
53 #define  SIGUNUSED    31      /* 未使用信号                     */
```

在示例代码 33.1.1.1 中的这些信号中，除了 SIGKILL(9)和 SIGSTOP(19)这两个信号不能被忽略外，其他的信号都可以忽略。这些信号就相当于中断号，不同的中断号代表了不同的中断，不同的中断所做的处理不同，因此，驱动程序可以通过向应用程序发送不同的信号来实现不同的功能。

我们使用中断的时候需要设置中断处理函数，同样的，如果要在应用程序中使用信号，那么就必须设置信号所使用的信号处理函数，在应用程序中使用 signal 函数来设置指定信号的处理函数，signal 函数原型如下所示：

```
sighandler_t signal(int signum, sighandler_t handler)
```

函数参数和返回值含义如下：

signum：要设置处理函数的信号。

handler：信号的处理函数。

返回值：设置成功的话返回信号的前一个处理函数，设置失败的话返回 SIG_ERR。

信号处理函数原型如下所示：

```
typedef void (*sighandler_t)(int)
```

我们前面讲解的使用“kill -9 PID”杀死指定进程的方法就是向指定的进程(PID)发送 SIGKILL 这个信号。当按下键盘上的 CTRL+C 组合键以后会向当前正在占用终端的应用程序发出 SIGINT 信号，SIGINT 信号默认的动作是关闭当前应用程序。这里我们修改一下 SIGINT 信号的默认处理函数，当按下 CTRL+C 组合键以后先在终端上打印出“SIGINT signal!”这行字符串，然后再关闭当前应用程序。新建 signaltest.c 文件，然后输入如下所示内容：

示例代码 33.1.1.2 信号测试

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <signal.h>
4
5 void sigint_handler(int num)
6 {
7     printf("\r\nSIGINT signal!\r\n");
8     exit(0);
```

```
9  }
10
11 int main(void)
12 {
13     signal(SIGINT, sigint_handler);
14     while(1);
15     return 0;
16 }
```

在示例代码 33.1.1.2 中我们设置 SIGINT 信号的处理函数为 sigint_handler，当按下 CTRL+C 向 signaltest 发送 SIGINT 信号以后 sigint_handler 函数就会执行，此函数先输出一行“SIGINT signal!”字符串，然后调用 exit 函数关闭 signaltest 应用程序。

使用如下命令编译 signaltest.c：

```
gcc signaltest.c -o signaltest
```

然后输入“./signaltest”命令打开 signaltest 这个应用程序，然后按下键盘上的 CTRL+C 组合键，结果如图 33.1.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1$ ./signaltest
^C
SIGINT signal!
zuozhongkai@ubuntu:~/linux/atk-mp1$
```

图 33.1.1.1 signaltest 软件运行结果

从图 33.1.1.1 可以看出，当按下 CTRL+C 组合键以后 sigint_handler 这个 SIGINT 信号处理函数执行了，并且输出了“SIGINT signal!”这行字符串。

33.1.2 驱动中的信号处理

1、fasync_struct 结构体

首先我们需要在驱动程序中定义一个 fasync_struct 结构体指针变量，fasync_struct 结构体内容如下：

示例代码 33.1.2.1 fasync_struct 发结构体

```
struct fasync_struct {
    spinlock_t              fa_lock;
    int                     magic;
    int                     fa_fd;
    struct fasync_struct    *fa_next;
    struct file             *fa_file;
    struct rcu_head         fa_rcu;
};
```

一般将 fasync_struct 结构体指针变量定义到设备结构体中，比如在上一章节的 key_dev 结构体中添加一个 fasync_struct 结构体指针变量，结果如下所示：

示例代码 33.1.2.2 在设备结构体中添加 fasync_struct 类型变量指针

```
1  struct key_dev{
2      dev_t          devid;
3      struct cdev    cdev;
4      struct class   *class;
```

```
......
12      struct fasync_struct *async_queue;  /* fasync_struct 结构体 */
13  };
```

第 12 行就是在 imx6uirq_dev 中添加了一个 fasync_struct 结构体指针变量。

2、fasync 函数

如果要使用异步通知，需要在设备驱动中实现 file_operations 操作集中的 fasync 函数，此函数格式如下所示：

```
int (*fasync) (int fd, struct file *filp, int on)
```

fasync 函数里面一般通过调用 fasync_helper 函数来初始化前面定义的 fasync_struct 结构体指针，fasync_helper 函数原型如下：

```
int fasync_helper(int fd, struct file * filp, int on, struct fasync_struct **fapp)
```

fasync_helper 函数的前三个参数就是 fasync 函数的那三个参数，第四个参数就是要初始化的 fasync_struct 结构体指针变量。当应用程序通过“fcntl(fd, F_SETFL, flags | FASYNC)”改变 fasync 标记的时候，驱动程序 file_operations 操作集中的 fasync 函数就会执行。

驱动程序中的 fasync 函数参考示例如下：

示例代码 33.1.2.3 驱动中 fasync 函数参考示例

```
1  struct xxx_dev {
2      ......
3      struct fasync_struct *async_queue;  /* 异步相关结构体 */
4  };
5
6  static int xxx_fasync(int fd, struct file *filp, int on)
7  {
8      struct xxx_dev *dev = (xxx_dev)filp->private_data;
9
10     if (fasync_helper(fd, filp, on, &dev->async_queue) < 0)
11         return -EIO;
12     return 0;
13 }
14
15 static struct file_operations xxx_ops = {
16     ......
17     .fasync = xxx_fasync,
18     ......
19 };
```

在关闭驱动文件的时候需要在 file_operations 操作集中的 release 函数中释放 fasync_struct，fasync_struct 的释放函数同样为 fasync_helper，release 函数参数参考实例如下：

示例代码 33.1.2.4 释放 fasync_struct 参考示例

```
1 static int xxx_release(struct inode *inode, struct file *filp)
2 {
3     return xxx_fasync(-1, filp, 0); /* 删除异步通知 */
4 }
5
```

```
6 static struct file_operations xxx_ops = {
7    ......
8       .release = xxx_release,
9 };
```

第 3 行通过调用示例代码 33.1.2.3 中的 xxx_fasync 函数来完成 fasync_struct 的释放工作，但是，其最终还是通过 fasync_helper 函数完成释放工作。

1、kill_fasync 函数

当设备可以访问的时候，驱动程序需要向应用程序发出信号，相当于产生“中断”。kill_fasync 函数负责发送指定的信号，kill_fasync 函数原型如下所示：

```
void kill_fasync(struct fasync_struct **fp, int sig, int band)
```

函数参数和返回值含义如下：

fp：要操作的 fasync_struct。

sig：要发送的信号。

band：可读时设置为 POLL_IN，可写时设置为 POLL_OUT。

返回值：无。

33.1.3 应用程序对异步通知的处理

应用程序对异步通知的处理包括以下三步：

1、注册信号处理函数

应用程序根据驱动程序所使用的信号来设置信号的处理函数，应用程序使用 signal 函数来设置信号的处理函数。前面已经详细的讲过了，这里就不细讲了。

2、将本应用程序的进程号告诉给内核

使用 fcntl(fd, F_SETOWN, getpid())将本应用程序的进程号告诉给内核。

3、开启异步通知

使用如下两行程序开启异步通知：

```
flags = fcntl(fd, F_GETFL);             /* 获取当前的进程状态          */
fcntl(fd, F_SETFL, flags | FASYNC);  /* 开启当前进程异步通知功能  */
```

重点就是通过 fcntl 函数设置进程状态为 FASYNC，经过这一步，驱动程序中的 fasync 函数就会执行。

33.2 硬件原理图分析

本章实验硬件原理图参考 29.2 小节即可。

33.3 实验程序编写

本实验对应的例程路径为：开发板光盘-> 2、Linux 驱动例程-> 16_asyncnoti。

本章实验我们在上一章实验“15_noblockio”的基础上完成，在其中加入异步通知相关内容即可，当按键按下以后驱动程序向应用程序发送 SIGIO 信号，应用程序获取到 SIGIO 信号以后读取并且打印出按键值。

33.3.1 修改设备树文件

因为是在实验“15_noblockio”的基础上完成的，因此不需要修改设备树。

33.3.2 程序编写

新建名为“16_asyncnoti”的文件夹，然后在 16_asyncnoti 文件夹里面创建 vscode 工程，工作区命名为“asyncnoti”。将“15_noblockio”实验中的 noblockio.c 复制到 16_asyncnoti 文件夹中，并重命名为 asyncnoti.c。接下来我们就修改 asyncnoti.c 这个文件，在其中添加异步通知关的代码，完成以后的 asyncnoti.c 内容如下所示(因为是在上一章实验的 noblockio.c 文件的基础上修改的，因为了减少篇幅，下面的代码有省略)：

示例代码 33.3.2.1 asyncnoti.c 文件代码段

```
32  #include <linux/fcntl.h>
33
34  #define KEY_CNT         1         /* 设备号个数    */
35  #define KEY_NAME        "key"     /* 名字          */
36
37  /* 定义按键状态 */
38  enum key_status {
39      KEY_PRESS = 0,                /* 按键按下      */
40      KEY_RELEASE,                  /* 按键松开      */
41      KEY_KEEP,                     /* 按键状态保持  */
42  };
43
44  /* key 设备结构体 */
45  struct key_dev{
46      dev_t devid;                  /* 设备号        */
47      struct cdev cdev;             /* cdev          */
48      struct class *class;          /* 类            */
49      struct device *device;        /* 设备          */
50      struct device_node  *nd;      /* 设备节点      */
51      int key_gpio;                 /* key 所使用的 GPIO 编号 */
52      struct timer_list timer;      /* 按键值        */
53      int irq_num;                  /* 中断号        */
54      atomic_t status;              /* 按键状态      */
55      wait_queue_head_t r_wait;     /* 读等待队列头  */
56      struct fasync_struct *async_queue;  /* fasync_struct 结构体 */
57  };
......
153 static void key_timer_function(struct timer_list *arg)
154 {
155     static int last_val = 1;
156     int current_val;
157
```

```
158     /* 读取按键值并判断按键当前状态 */
159     current_val = gpio_get_value(key.key_gpio);
160     if (0 == current_val && last_val){
161         atomic_set(&key.status, KEY_PRESS);         /* 按下 */
162         wake_up_interruptible(&key.r_wait);         /* 唤醒 */
163         if(key.async_queue)
164             kill_fasync(&key.async_queue, SIGIO, POLL_IN);
165     }
166     else if (1 == current_val && !last_val) {
167         atomic_set(&key.status, KEY_RELEASE);       /* 松开 */
168         wake_up_interruptible(&key.r_wait);         /* 唤醒 */
169         if(key.async_queue)
170             kill_fasync(&key.async_queue, SIGIO, POLL_IN);
171     }
172     else
173         atomic_set(&key.status, KEY_KEEP);          /* 状态保持 */
174
175     last_val = current_val;
176 }
......
221 /*
222  * @description   : fasync 函数，用于处理异步通知
223  * @param - fd    : 文件描述符
224  * @param - filp  : 要打开的设备文件(文件描述符)
225  * @param - on    : 模式
226  * @return        : 负数表示函数执行失败
227  */
228 static int key_fasync(int fd, struct file *filp, int on)
229 {
230     return fasync_helper(fd, filp, on, &key.async_queue);
231 }
......
246 /*
247  * @description   : 关闭/释放设备
248  * @param - filp  : 要关闭的设备文件(文件描述符)
249  * @return        : 0 成功;其他 失败
250  */
251 static int key_release(struct inode *inode, struct file *filp)
252 {
253     return key_fasync(-1, filp, 0);
254 }
......
274 /* 设备操作函数 */
```

```
275 static struct file_operations key_fops = {
276     .owner = THIS_MODULE,
277     .open = key_open,
278     .read = key_read,
279     .write = key_write,
280     .release =  key_release,
281     .poll = key_poll,
282     .fasync = key_fasync,
283 };
......
373 module_init(mykey_init);
374 module_exit(mykey_exit);
375 MODULE_LICENSE("GPL");
376 MODULE_AUTHOR("ALIENTEK");
377 MODULE_INFO(intree, "Y");
```

第 32 行，添加 fcntl.h 头文件，因为要用到相关的 API 函数。

第 56 行，在设备结构体 key_dev 中添加 fasync_struct 指针变量。

第 153~176 行，在 key_timer_function 函数中，当按键按下或松开动作发生时调用 kill_fasync 函数向应用程序发送 SIGIO 信号，通知应用程序按键数据可以进行读取了。

第 228~231 行，设备操作函数集 file_operations 结构体中的 fasync 函数 key_fasync，该函数中直接调用 fasync_helper 函数进行相关处理。

第 253 行，在 key_release 函数中也调用 key_fasync 函数释放 fasync_struct 指针变量。

第 282 行，将 key_fasync 函数绑定到 key_fops 变量的 fasync 函数指针中。

33.3.3 编写测试 APP

测试 APP 要实现的内容很简单，设置 SIGIO 信号的处理函数为 sigio_signal_func，当驱动程序向应用程序发送 SIGIO 信号以后 sigio_signal_func 函数就会执行。sigio_signal_func 函数内容很简单，就是通过 read 函数读取按键值。新建名为 asyncnotiApp.c 的文件，然后输入如下所示内容：

示例代码 33.3.3.2 asyncnotiApp.c 文件代码段

```
13 #include <stdio.h>
14 #include <unistd.h>
15 #include <sys/types.h>
16 #include <sys/stat.h>
17 #include <fcntl.h>
18 #include <stdlib.h>
19 #include <string.h>
20 #include <signal.h>
21
22 static int fd;
23
24 /*
25  * SIGIO 信号处理函数
```

```
26   * @param - signum    : 信号值
27   * @return            : 无
28   */
29  static void sigio_signal_func(int signum)
30  {
31      unsigned int key_val = 0;
32
33      read(fd, &key_val, sizeof(unsigned int));
34      if (0 == key_val)
35          printf("Key Press\n");
36      else if (1 == key_val)
37          printf("Key Release\n");
38  }
39
40
41  /*
42   * @description   : main 主程序
43   * @param - argc  : argv 数组元素个数
44   * @param - argv  : 具体参数
45   * @return        : 0 成功;其他 失败
46   */
47  int main(int argc, char *argv[])
48  {
49      int flags = 0;
50
51      /* 判断传参个数是否正确 */
52      if(2 != argc) {
53          printf("Usage:\n"
54                 "\t./asyncKeyApp /dev/key\n"
55                );
56          return -1;
57      }
58
59      /* 打开设备 */
60      fd = open(argv[1], O_RDONLY | O_NONBLOCK);
61      if(0 > fd) {
62          printf("ERROR: %s file open failed!\n", argv[1]);
63          return -1;
64      }
65
66      /* 设置信号 SIGIO 的处理函数 */
67      signal(SIGIO, sigio_signal_func);
68      fcntl(fd, F_SETOWN, getpid());      /* 将当前进程的进程号告诉给内核 */
```

```
69      flags = fcntl(fd, F_GETFD);          /*获取当前的进程状态  */
70      fcntl(fd, F_SETFL, flags | FASYNC);/* 设置进程启用异步通知功能 */
71
72
73      /* 循环轮询读取按键数据 */
74      for ( ; ; ) {
75
76          sleep(2);
77      }
78
79      /* 关闭设备 */
80      close(fd);
81      return 0;
82  }
```

第 29~38 行，sigio_signal_func 函数，SIGIO 信号的处理函数，当驱动程序有效按键按下以后就会发送 SIGIO 信号，此函数就会执行。此函数通过 read 函数读取按键状态数据，然后通过 printf 函数打印在终端上。

第 67 行，通过 signal 函数设置 SIGIO 信号的处理函数为 sigio_signal_func。

第 68~70 行，设置当前进程的状态，开启异步通知的功能。

第 74~77 行，for 循环，等待信号产生。

33.4 运行测试

33.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为 asyncnoti.o，Makefile 内容如下所示：

示例代码 33.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := asyncnoti.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为 asyncnoti.o。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“asyncnoti.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 asyncnotiApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc asyncnotiApp.c -o asyncnotiApp
```

编译成功以后就会生成 asyncnotiApp 这个应用程序。

33.4.2 运行测试

将上一小节编译出来 asyncnoti.ko 和 asyncnotiApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 asyncnoti.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe asyncnoti.ko   //加载驱动
```

驱动加载成功以后使用如下命令来测试中断：

```
./asyncnotiApp /dev/key
```

按下开发板上的 KEY0 键，终端就会输出按键值，如图 33.4.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./asyncnotiApp /dev/key
Key Press
Key Release
Key Press
Key Release
```

图 33.4.2.1 读取到的按键值

从图 33.4.2.1 可以看出，捕获到 SIGIO 信号，并且按键值获取成功，大家可以自行以后台模式运行 asyncnotiApp，查看一下这个应用程序的 CPU 使用率。如果要卸载驱动的话输入如下命令即可：

```
rmmod asyncnoti.ko
```

第三十四章 platform 设备驱动实验

我们在前面几章编写的设备驱动都非常的简单，都是对 GPIO 进行最简单的读写操作。像 I2C、SPI、LCD 等这些复杂外设的驱动就不能这么去写了，Linux 系统要考虑到驱动的可重用性，因此提出了驱动的分离与分层这样的软件思路，在这个思路下诞生了我们将来最常打交道的 platform 设备驱动，也叫做平台设备驱动。本章我们就来学习一下 Linux 下的驱动分离与分层，以及 platform 框架下的设备驱动该如何编写。

34.1 Linux 驱动的分离与分层

34.1.1 驱动的分隔与分离

对于 Linux 这样一个成熟、庞大、复杂的操作系统，代码的重用性非常重要，否则的话就会在 Linux 内核中存在大量无意义的重复代码。尤其是驱动程序，因为驱动程序占用了 Linux 内核代码量的大头，如果不对驱动程序加以管理，任由重复的代码肆意增加，那么用不了多久 Linux 内核的文件数量就庞大到无法接受的地步。

假如现在有三个平台 A、B 和 C，这三个平台(这里的平台说的是 SOC)上都有 MPU6050 这个 I2C 接口的六轴传感器，按照我们写裸机 I2C 驱动的时候的思路，每个平台都有一个 MPU6050 的驱动，因此编写出来的最简单的驱动框架如图 34.1.1.1 所示：

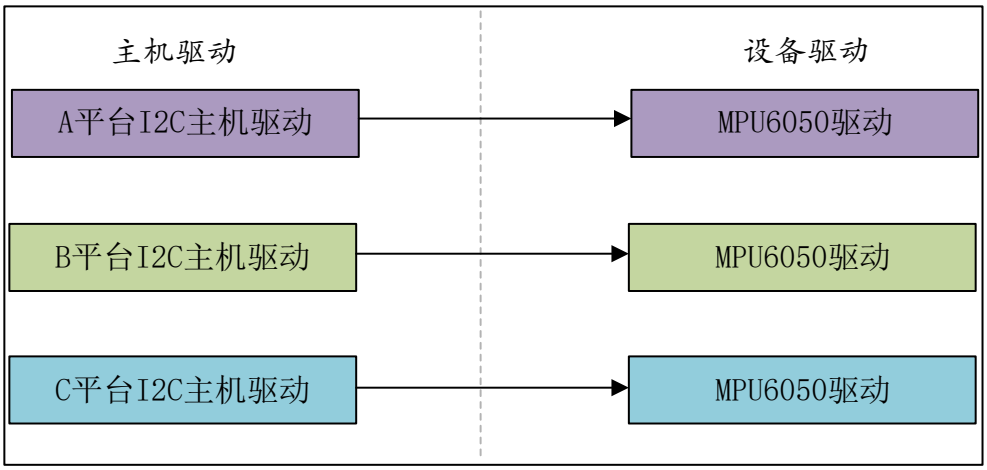


图 34.1.1.1 传统的 I2C 设备驱动

从图 34.1.1.1 可以看出，每种平台下都有一个主机驱动和设备驱动，主机驱动肯定是必须要的，毕竟不同的平台其 I2C 控制器不同。但是右侧的设备驱动就没必要每个平台都写一个，因为不管对于那个 SOC 来说，MPU6050 都是一样，通过 I2C 接口读写数据就行了，只需要一个 MPU6050 的驱动程序即可。如果再来几个 I2C 设备，比如 AT24C02、FT5206(电容触摸屏)等，如果按照图 34.1.1.1 中的写法，那么设备端的驱动将会重复的编写好几次。显然在 Linux 驱动程序中这种写法是不推荐的，最好的做法就是每个平台的 I2C 控制器都提供一个统一的接口(也叫做主机驱动)，每个设备的话也只提供一个驱动程序(设备驱动)，每个设备通过统一的 I2C 接口驱动来访问，这样就可以大大简化驱动文件，比如 34.1.1.1 中三种平台下的 MPU6050 驱动框架就可以简化为图 34.1.1.2 所示：

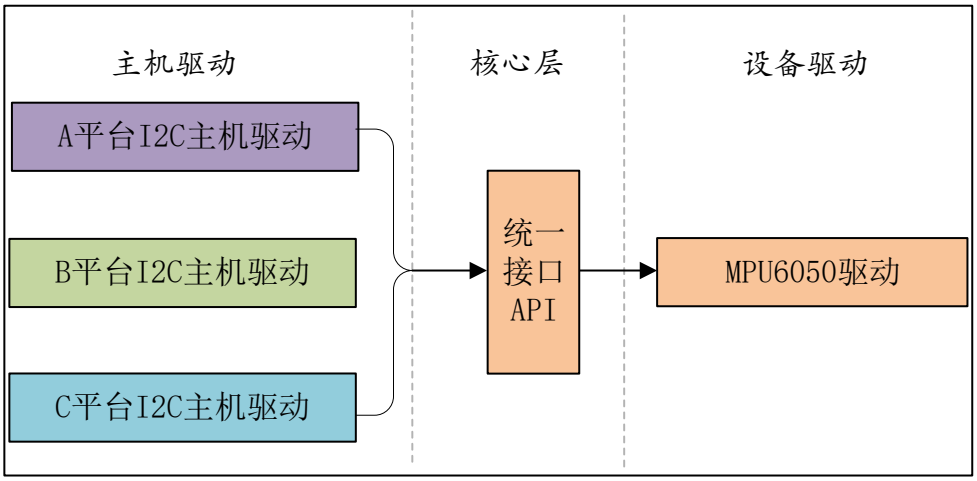


图 34.1.1.2 改进后的设备驱动

实际的 I2C 驱动设备肯定有很多种，不止 MPU6050 这一个，那么实际的驱动架构如图 34.1.1.3 所示：

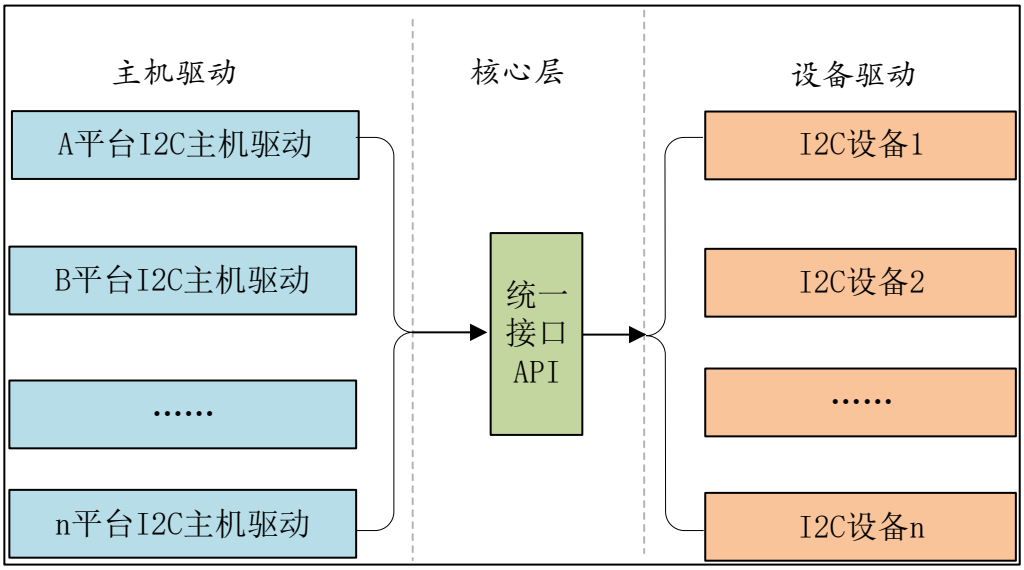


图 34.1.1.3 分隔后的驱动框架

这个就是驱动的分隔，也就是将主机驱动和设备驱动分隔开来，比如 I2C、SPI 等等都会采用驱动分隔的方式来简化驱动的开发。在实际的驱动开发中，一般 I2C 主机控制器驱动已经由半导体厂家编写好了，而设备驱动一般也由设备器件的厂家编写好了，我们只需要提供设备信息即可，比如 I2C 设备的话提供设备连接到了哪个 I2C 接口上，I2C 的速度是多少等等。相当于将设备信息从设备驱动中剥离开来，驱动使用标准方法去获取到设备信息(比如从设备树中获取到设备信息)，然后根据获取到的设备信息来初始化设备。 这样就相当于驱动只负责驱动，设备只负责设备，想办法将两者进行匹配即可。这个就是 Linux 中的总线(bus)、驱动(driver)和设备(device)模型，也就是常说的驱动分离。总线就是驱动和设备信息的月老，负责给两者牵线搭桥，如图 34.1.1.4 所示：

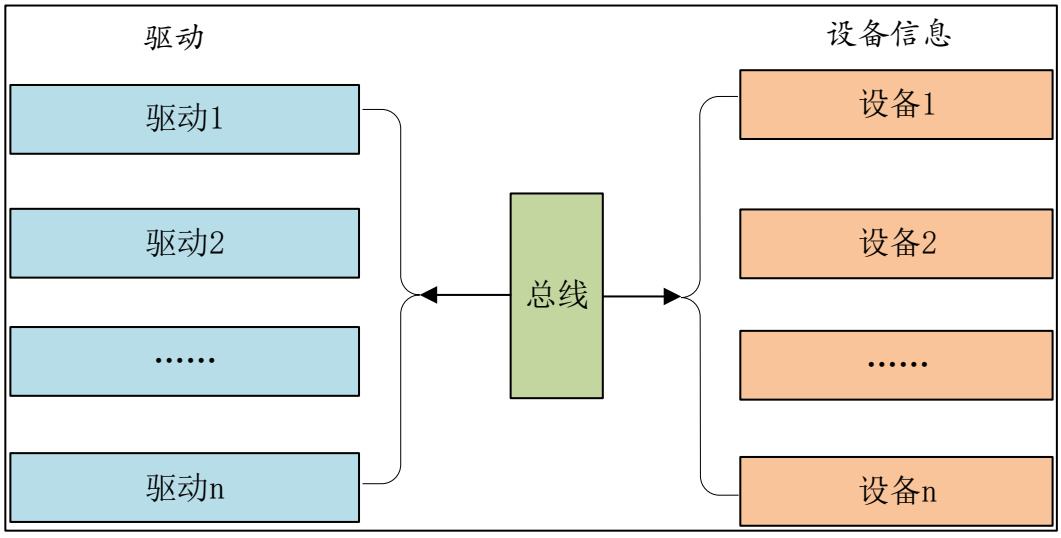


图 34.1.1.4 Linux 总线、驱动和设备模式

当我们向系统注册一个驱动的时候，总线就会在右侧的设备中查找，看看有没有与之匹配的设备，如果有的话就将两者联系起来。同样的，当向系统中注册一个设备的时候，总线就会在左侧的驱动中查找看有没有与之匹配的设备，有的话也联系起来。Linux 内核中大量的驱动程序都采用总线、驱动和设备模式，我们一会要重点讲解的 platform 驱动就是这一思想下的产物。

34.1.2 驱动的分层

上一小节讲了驱动的分隔与分离，本节我们来简单看一下驱动的分层，大家应该听说过网络的 7 层模型，不同的层负责不同的内容。同样的，Linux 下的驱动往往也是分层的，分层的目的也是为了在不同的层处理不同的内容。以其他书籍或者资料常常使用到的 input(输入子系统，后面会有专门的章节详细的讲解)为例，简单介绍一下驱动的分层。input 子系统负责管理所有跟输入有关的驱动，包括键盘、鼠标、触摸等，最底层的就是设备原始驱动，负责获取输入设备的原始值，获取到的输入事件上报给 input 核心层。input 核心层会处理各种 IO 模型，并且提供 file_operations 操作集合。我们在编写输入设备驱动的时候只需要处理好输入事件的上报即可，至于如何处理这些上报的输入事件那是上层去考虑的，我们不用管。可以看出借助分层模型可以极大的简化我们的驱动编写，对于驱动编写来说非常的友好。

34.2 platform 平台驱动模型简介

前面我们讲了设备驱动的分离，并且引出了总线(bus)、驱动(driver)和设备(device)模型，比如 I2C、SPI、USB 等总线。在 SOC 中有些外设是没有总线这个概念的，但是又要使用总线、驱动和设备模型该怎么办呢？为了解决此问题，Linux 提出了 platform 这个虚拟总线，相应的就有 platform_driver 和 platform_device。

34.2.1 platform 总线

Linux 系统内核使用 bus_type 结构体表示总线，此结构体定义在文件 include/linux/device.h，bus_type 结构体内容如下：

示例代码 34.2.1.1 bus_type 结构体代码段

```
1 struct bus_type {
```

```
2   const char      *name;
3   const char      *dev_name;
4   struct device       *dev_root;
5   const struct attribute_group **bus_groups;
6   const struct attribute_group **dev_groups;
7   const struct attribute_group **drv_groups;
8   int (*match)(struct device *dev, struct device_driver *drv);
9   int (*uevent)(struct device *dev, struct kobj_uevent_env *env);
10  int (*probe)(struct device *dev);
11  int (*remove)(struct device *dev);
12  void (*shutdown)(struct device *dev);
13  int (*online)(struct device *dev);
14  int (*offline)(struct device *dev);
15  int (*suspend)(struct device *dev, pm_message_t state);
16  int (*resume)(struct device *dev);
17  int (*num_vf)(struct device *dev);
18  int (*dma_configure)(struct device *dev);
19  const struct dev_pm_ops *pm;
20  const struct iommu_ops *iommu_ops;
21  struct subsys_private *p;
22  struct lock_class_key lock_key;
23  bool need_parent_lock;
24 };
```

第 8 行，match 函数，此函数很重要，单词 match 的意思就是“匹配、相配”，因此此函数就是完成设备和驱动之间匹配的，总线就是使用 match 函数来根据注册的设备来查找对应的驱动，或者根据注册的驱动来查找相应的设备，因此每一条总线都必须实现此函数。match 函数有两个参数：dev 和 drv，这两个参数分别为 device 和 device_driver 类型，也就是设备和驱动。

platform 总线是 bus_type 的一个具体实例，定义在文件 drivers/base/platform.c，platform 总线定义如下：

示例代码 34.2.1.2 platform 总线实例

```
1  struct bus_type platform_bus_type = {
2      .name           = "platform",
3      .dev_groups     = platform_dev_groups,
4      .match          = platform_match,
5      .uevent         = platform_uevent,
6      .dma_configure  = platform_dma_configure,
7      .pm             = &platform_dev_pm_ops,
8  };
```

platform_bus_type 就是 platform 平台总线，其中 platform_match 就是匹配函数。我们来看一下驱动和设备是如何匹配的，platform_match 函数定义在文件 drivers/base/platform.c 中，函数内容如下所示：

示例代码 34.2.1.3 platform 总线实例

```
1  static int platform_match(struct device *dev,
```

```
                                struct device_driver *drv)
2  {
3      struct platform_device *pdev = to_platform_device(dev);
4      struct platform_driver *pdrv = to_platform_driver(drv);
5
6      /*When driver_override is set,only bind to the matching driver*/
7      if (pdev->driver_override)
8      return !strcmp(pdev->driver_override, drv->name);
9
10     /* Attempt an OF style match first */
11     if (of_driver_match_device(dev, drv))
12         return 1;
13
14     /* Then try ACPI style match */
15     if (acpi_driver_match_device(dev, drv))
16         return 1;
17
18     /* Then try to match against the id table */
19     if (pdrv->id_table)
20         return platform_match_id(pdrv->id_table, pdev) != NULL;
21
22     /* fall-back to driver name match */
23     return (strcmp(pdev->name, drv->name) == 0);
24 }
```

驱动和设备的匹配有四种方法，我们依次来看一下：

第 11~12 行，第一种匹配方式， OF 类型的匹配，也就是设备树采用的匹配方式，of_driver_match_device 函数定义在文件 include/linux/of_device.h 中。device_driver 结构体(表示设备驱动)中有个名为of_match_table 的成员变量，此成员变量保存着驱动的 compatible 匹配表，设备树中的每个设备节点的 compatible 属性会和 of_match_table 表中的所有成员比较，查看是否有相同的条目，如果有的话就表示设备和此驱动匹配，设备和驱动匹配成功以后 probe 函数就会执行。

第 15~16 行，第二种匹配方式，ACPI 匹配方式。

第 19~20 行，第三种匹配方式，id_table 匹配，每个 platform_driver 结构体有一个 id_table 成员变量，顾名思义，保存了很多 id 信息。这些 id 信息存放着这个 platformd 驱动所支持的驱动类型。

第 23 行，第四种匹配方式，如果第三种匹配方式的 id_table 不存在的话就直接比较驱动和设备的 name 字段，看看是不是相等，如果相等的话就匹配成功。

对于支持设备树的 Linux 版本号，一般设备驱动为了兼容性都支持设备树和无设备树两种匹配方式。也就是第一种匹配方式一般都会存在，第三种和第四种只要存在一种就可以，一般用的最多的还是第四种，也就是直接比较驱动和设备的 name 字段，毕竟这种方式最简单了。

34.2.2 platform 驱动

platform_driver 结构体表示 platform 驱动，此结构体定义在文件

include/linux/platform_device.h 中，内容如下：

示例代码 34.2.2.1 platform_driver 结构体

```
1  struct platform_driver {
2      int (*probe)(struct platform_device *);
3      int (*remove)(struct platform_device *);
4      void (*shutdown)(struct platform_device *);
5      int (*suspend)(struct platform_device *, pm_message_t state);
6      int (*resume)(struct platform_device *);
7      struct device_driver driver;
8      const struct platform_device_id *id_table;
9      bool prevent_deferred_probe;
10 };
```

第 2 行，probe 函数，当驱动与设备匹配成功以后 probe 函数就会执行，非常重要的函数!!一般驱动的提供者会编写，如果自己要编写一个全新的驱动，那么 probe 就需要自行实现。

第 7 行，driver 成员，为 device_driver 结构体变量，Linux 内核里面大量使用到了面向对象的思维，device_driver 相当于基类，提供了最基础的驱动框架。plaform_driver 继承了这个基类，然后在此基础上又添加了一些特有的成员变量。

第 8 行，id_table 表，也就是我们上一小节讲解 platform 总线匹配驱动和设备的时候采用的第三种方法，id_table 是个表(也就是数组)，每个元素的类型为 platform_device_id，platform_device_id 结构体内容如下：

示例代码 34.2.2.2 platform_device_id 结构体

```
1 struct platform_device_id {
2      char name[PLATFORM_NAME_SIZE];
3      kernel_ulong_t driver_data;
4 };
```

device_driver 结构体定义在 include/linux/device.h，device_driver 结构体内容如下：

示例代码 34.2.2.3 device_driver 结构体

```
1  struct device_driver {
2   const char        *name;
3   struct bus_type   *bus;
4   struct module     *owner;
5   const char        *mod_name;     /* used for built-in modules */
6   bool suppress_bind_attrs;       /* disables bind/unbind via sysfs */
7   enum probe_type probe_type;
8   const struct of_device_id   *of_match_table;
9   const struct acpi_device_id *acpi_match_table;
10  int (*probe) (struct device *dev);
11  int (*remove) (struct device *dev);
12  void (*shutdown) (struct device *dev);
13  int (*suspend) (struct device *dev, pm_message_t state);
14  int (*resume) (struct device *dev);
15  const struct attribute_group **groups;
16  const struct attribute_group **dev_groups;
```

```
17  const struct dev_pm_ops *pm;
18  void (*coredump) (struct device *dev);
19  struct driver_private *p;
20 };
```

第 8 行，of_match_table 就是采用设备树的时候驱动使用的匹配表，同样是数组，每个匹配项都为 of_device_id 结构体类型，此结构体定义在文件 include/linux/mod_devicetable.h 中，内容如下：

示例代码 34.2.2.4 of_device_id 结构体

```
1 struct of_device_id {
2   char            name[32];
3   char            type[32];
4   char            compatible[128];
5   const void      *data;
6 };
```

第 4 行的 compatible 非常重要，因为对于设备树而言，就是通过设备节点的 compatible 属性值和 of_match_table 中每个项目的 compatible 成员变量进行比较，如果有相等的就表示设备和此驱动匹配成功。

在编写 platform 驱动的时候，首先定义一个 platform_driver 结构体变量，然后实现结构体中的各个成员变量，重点是实现匹配方法以及 probe 函数。当驱动和设备匹配成功以后 probe 函数就会执行，具体的驱动程序在 probe 函数里面编写，比如字符设备驱动等等。

当我们定义并初始化好 platform_driver 结构体变量以后，需要在驱动入口函数里面调用 platform_driver_register 函数向 Linux 内核注册一个 platform 驱动，platform_driver_register 函数原型如下所示：

```
int platform_driver_register (struct platform_driver      *driver)
```

函数参数和返回值含义如下：

driver：要注册的 platform 驱动。

返回值：负数，失败；0，成功。

还需要在驱动卸载函数中通过 platform_driver_unregister 函数卸载 platform 驱动，platform_driver_unregister 函数原型如下：

```
void platform_driver_unregister(struct platform_driver *drv)
```

函数参数和返回值含义如下：

drv：要卸载的 platform 驱动。

返回值：无。

platform 驱动框架如下所示：

示例代码 34.2.2.5 platform 驱动框架

```
/* 设备结构体 */
1  struct xxx_dev{
2      struct cdev cdev;
3      /* 设备结构体其他具体内容 */
4  };
5
6  struct xxx_dev xxxdev;   /* 定义个设备结构体变量 */
7
```

```
8  static int xxx_open(struct inode *inode, struct file *filp)
9  {
10     /* 函数具体内容 */
11     return 0;
12 }
13
14 static ssize_t xxx_write(struct file *filp, const char __user *buf,
                            size_t cnt, loff_t *offt)
15 {
16     /* 函数具体内容 */
17     return 0;
18 }
19
20 /*
21  * 字符设备驱动操作集
22  */
23 static struct file_operations xxx_fops = {
24     .owner = THIS_MODULE,
25     .open = xxx_open,
26     .write = xxx_write,
27 };
28
29 /*
30  * platform 驱动的 probe 函数
31  * 驱动与设备匹配成功以后此函数就会执行
32  */
33 static int xxx_probe(struct platform_device *dev)
34 {
35     ......
36     cdev_init(&xxxdev.cdev, &xxx_fops); /* 注册字符设备驱动 */
37     /* 函数具体内容 */
38     return 0;
39 }
40
41 static int xxx_remove(struct platform_device *dev)
42 {
43     ......
44     cdev_del(&xxxdev.cdev);/*  删除 cdev */
45     /* 函数具体内容 */
46     return 0;
47 }
48
49 /* 匹配列表 */
```

```
50 static const struct of_device_id xxx_of_match[] = {
51     { .compatible = "xxx-gpio" },
52     { /* Sentinel */ }
53 };
54
55 /*
56  * platform 平台驱动结构体
57  */
58 static struct platform_driver xxx_driver = {
59     .driver = {
60         .name       = "xxx",
61         .of_match_table = xxx_of_match,
62     },
63     .probe      = xxx_probe,
64     .remove     = xxx_remove,
65 };
66
67 /* 驱动模块加载 */
68 static int __init xxxdriver_init(void)
69 {
70     return platform_driver_register(&xxx_driver);
71 }
72
73 /* 驱动模块卸载 */
74 static void __exit xxxdriver_exit(void)
75 {
76         platform_driver_unregister(&xxx_driver);
77 }
78
79 module_init(xxxdriver_init);
80 module_exit(xxxdriver_exit);
81 MODULE_LICENSE("GPL");
82 MODULE_AUTHOR("zuozhongkai");
```

第 1~27 行，传统的字符设备驱动，所谓的 platform 驱动并不是独立于字符设备驱动、块设备驱动和网络设备驱动之外的其他种类的驱动。platform 只是为了驱动的分离与分层而提出来的一种框架，其驱动的具体实现还是需要字符设备驱动、块设备驱动或网络设备驱动。

第 33~39 行，xxx_probe 函数，当驱动和设备匹配成功以后此函数就会执行，以前在驱动入口 init 函数里面编写的字符设备驱动程序就全部放到此 probe 函数里面。比如注册字符设备驱动、添加 cdev、创建类等等。

第 41~47 行，xxx_remove 函数，platform_driver 结构体中的 remove 成员变量，当关闭 platfor 备驱动的时候此函数就会执行，以前在驱动卸载 exit 函数里面要做的事情就放到此函数中来。比如，使用 iounmap 释放内存、删除 cdev，注销设备号等等。

第 50~53 行，xxx_of_match 匹配表，如果使用设备树的话将通过此匹配表进行驱动和设备的匹配。第 51 行设置了一个匹配项，此匹配项的 compatible 值为“xxx-gpio”，因此当设备树中设备节点的 compatible 属性值为“xxx-gpio”的时候此设备就会与此驱动匹配。第 52 行是一个标记，of_device_id 表最后一个匹配项必须是空的。

第 58~65 行，定义一个 platform_driver 结构体变量 xxx_driver，表示 platform 驱动，第 59~62 行设置 paltform_driver 中的 device_driver 成员变量的 name 和 of_match_table 这两个属性。其中 name 属性用于传统的驱动与设备匹配，也就是检查驱动和设备的 name 字段是不是相同。of_match_table 属性就是用于设备树下的驱动与设备检查。对于一个完整的驱动程序，必须提供有设备树和无设备树两种匹配方法。最后 63 和 64 这两行设置 probe 和 remove 这两成员变量。

第 68~71 行，驱动入口函数，调用 platform_driver_register 函数向 Linux 内核注册一个 platform 驱动，也就是上面定义的 xxx_driver 结构体变量。

第 74~77 行，驱动出口函数，调用 platform_driver_unregister 函数卸载前面注册的 platform 驱动。

总体来说，platform 驱动还是传统的字符设备驱动、块设备驱动或网络设备驱动，只是套上了一张“platform”的皮，目的是为了使用总线、驱动和设备这个驱动模型来实现驱动的分离与分层。

34.2.3 platform 设备

platform 驱动已经准备好了，我们还需要 platform 设备，否则的话单单一个驱动也做不了什么。platform_device 这个结构体表示 platform 设备，这里我们要注意，如果内核支持设备树的话就不要再使用 platform_device 来描述设备了，因为改用设备树去描述了。当然了，你如果一定要用 platform_device 来描述设备信息的话也是可以的。platform_device 结构体定义在文件 include/linux/platform_device.h 中，结构体内容如下：

示例代码 34.2.3.1 platform_device 结构体代码段

```
1  struct platform_device {
2      const char  *name;
3      int     id;
4      bool        id_auto;
5      struct device   dev;
6      u64     platform_dma_mask;
7      u32     num_resources;
8      struct resource *resource;
9
10     const struct platform_device_id *id_entry;
11     char *driver_override; /* Driver name to force a match */
12
13     /* MFD cell pointer */
14     struct mfd_cell *mfd_cell;
15
16     /* arch specific additions */
17     struct pdev_archdata    archdata;
18 };
```

第 2 行，name 表示设备名字，要和所使用的 platform 驱动的 name 字段相同，否则的话设备就无法匹配到对应的驱动。比如对应的 platform 驱动的 name 字段为“xxx-gpio”，那么此 name 字段也要设置为“xxx-gpio”。

第 7 行，num_resources 表示资源数量，一般为第 8 行 resource 资源的大小。

第 8 行，resource 表示资源，也就是设备信息，比如外设寄存器等。Linux 内核使用 resource 结构体表示资源，resource 结构体定义在 include/linux/ioport.h 文件里面，内容为：

示例代码 54.2.3.2 resource 结构体代码段

```
1 struct resource {
2   resource_size_t start;
3   resource_size_t end;
4   const char *name;
5   unsigned long flags;
6   unsigned long desc;
7   struct resource *parent, *sibling, *child;
8 };
```

start 和 end 分别表示资源的起始和终止信息，对于内存类的资源，就表示内存起始和终止地址，name 表示资源名字，flags 表示资源类型，可选的资源类型都定义在了文件 include/linux/ioport.h 里面，如下所示：

示例代码 34.2.3.3 资源类型

```
1   #define IORESOURCE_BITS         0x000000ff  /* Bus-specific bits */
2
3   #define IORESOURCE_TYPE_BITS    0x00001f00  /* Resource type    */
4   #define IORESOURCE_IO           0x00000100  /* 表示 IO 口的资源 */
5   #define IORESOURCE_MEM          0x00000200  /* 表示内存地址 */
6   #define IORESOURCE_REG          0x00000300  /* Register offsets */
7   #define IORESOURCE_IRQ          0x00000400  /* 中断号 */
8   #define IORESOURCE_DMA          0x00000800  /* DMA 通道号 */
9   #define IORESOURCE_BUS          0x00001000      /* 总线号 */
......
84 #define IORESOURCE_PCI_FIXED    (1<<4)  /* Do not move resource */
```

在以前不支持设备树的 Linux 版本中，用户需要编写 platform_device 变量来描述设备信息，然后使用 platform_device_register 函数将设备信息注册到 Linux 内核中，此函数原型如下所示：

```
int platform_device_register(struct platform_device *pdev)
```

函数参数和返回值含义如下：

pdev：要注册的 platform 设备。

返回值：负数，失败；0，成功。

如果不再使用 platform 的话可以通过 platform_device_unregister 函数注销掉相应的 platform 设备，platform_device_unregister 函数原型如下：

```
void platform_device_unregister(struct platform_device *pdev)
```

函数参数和返回值含义如下：

pdev：要注销的 platform 设备。

返回值：无。

platform 设备信息框架如下所示：

示例代码 54.2.3.4 platform 设备框架

```
1  /* 寄存器地址定义*/
2  #define PERIPH1_REGISTER_BASE    (0X20000000) /* 外设 1 寄存器首地址 */
3  #define PERIPH2_REGISTER_BASE    (0X020E0068) /* 外设 2 寄存器首地址 */
4  #define REGISTER_LENGTH          4
5
6  /* 资源 */
7  static struct resource xxx_resources[] = {
8      [0] = {
9          .start  = PERIPH1_REGISTER_BASE,
10         .end    = (PERIPH1_REGISTER_BASE + REGISTER_LENGTH - 1),
11         .flags  = IORESOURCE_MEM,
12     },
13     [1] = {
14         .start  = PERIPH2_REGISTER_BASE,
15         .end    = (PERIPH2_REGISTER_BASE + REGISTER_LENGTH - 1),
16         .flags  = IORESOURCE_MEM,
17     },
18 };
19
20 /* platform 设备结构体 */
21 static struct platform_device xxxdevice = {
22     .name = "xxx-gpio",
23     .id = -1,
24     .num_resources = ARRAY_SIZE(xxx_resources),
25     .resource = xxx_resources,
26 };
27
28 /* 设备模块加载 */
29 static int __init xxxdevice_init(void)
30 {
31     return platform_device_register(&xxxdevice);
32 }
33
34 /* 设备模块注销 */
35 static void __exit xxx_resourcesdevice_exit(void)
36 {
37     platform_device_unregister(&xxxdevice);
38 }
39
40 module_init(xxxdevice_init);
41 module_exit(xxxdevice_exit);
42 MODULE_LICENSE("GPL");
```

```
43 MODULE_AUTHOR("zuozhongkai");
```

第 7~18 行，数组 xxx_resources 表示设备资源，一共有两个资源，分别为设备外设 1 和外设 2 的寄存器信息。因此 flags 都为 IORESOURCE_MEM，表示资源为内存类型的。

第 21~26 行，platform 设备结构体变量，注意 name 字段要和所使用的驱动中的 name 字段一致，否则驱动和设备无法匹配成功。num_resources 表示资源大小，其实就是数组 xxx_resources 的元素数量，这里用 ARRAY_SIZE 来测量一个数组的元素个数。

第 29~32 行，设备模块加载函数，在此函数中调用 platform_device_register 向 Linux 内核注册 platform 设备。

第 35~38 行，设备模块卸载函数，在此函数中调用 platform_device_unregister 从 Linux 内核中卸载 platform 设备。

示例代码 34.2.3.4 主要是在不支持设备树的 Linux 版本中使用的，当 Linux 内核支持了设备树以后就不需要用户手动去注册 platform 设备了。因为设备信息都放到了设备树中去描述，Linux 内核启动的时候会从设备树中读取设备信息，然后将其组织成 platform_device 形式，至于设备树到 platform_device 的具体过程就不去详细的追究了，感兴趣的可以去看一下，网上也有很多博客详细的讲解了整个过程。

关于 platform 下的总线、驱动和设备就讲解到这里，我们接下来就使用 platform 驱动框架来编写一个 LED 灯驱动，本章我们不使用设备树来描述设备信息，我们采用自定义 platform_device 这种“古老”方式来编写 LED 的设备信息。下一章我们来编写设备树下的 platform 驱动，这样我们就掌握了无设备树和有设备树这两种 platform 驱动的开发方式。

34.3 硬件原理图分析

本章实验我们只使用到正点原子的 STM32MP1 开发板上的 LED 灯，因此实验硬件原理图参考 21.2 小节即可。

34.4 试验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→17_platform。

本章实验我们需要编写一个驱动模块和一个设备模块，其中驱动模块是 platform 驱动程序，设备模块是 platform 的设备信息。当这两个模块都加载成功以后就会匹配成功，然后 platform 驱动模块中的 probe 函数就会执行，probe 函数中就是传统的字符设备驱动那一套。

34.4.1 platform 设备与驱动程序编写

新建名为“17_platform”的文件夹，然后在 17_platform 文件夹里面创建 vscode 工程，工作区命名为“platform”。新建名为 leddevice.c 和 leddriver.c 这两个文件，这两个文件分别为 LED 灯的 platform 设备文件和 LED 灯的 platform 的驱动文件。在 leddevice.c 中输入如下所示内容：

示例代码 34.4.1.1 leddevice.c 文件代码段

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
```

```
8   #include <linux/gpio.h>
9   #include <linux/cdev.h>
10  #include <linux/device.h>
11  #include <linux/of_gpio.h>
12  #include <linux/semaphore.h>
13  #include <linux/timer.h>
14  #include <linux/irq.h>
15  #include <linux/wait.h>
16  #include <linux/poll.h>
17  #include <linux/fs.h>
18  #include <linux/fcntl.h>
19  #include <linux/platform_device.h>
20  #include <asm/mach/map.h>
21  #include <asm/uaccess.h>
22  #include <asm/io.h>
23
24  /* 寄存器物理地址 */
25  #define PERIPH_BASE             (0x40000000)
26  #define MPU_AHB4_PERIPH_BASE    (PERIPH_BASE + 0x10000000)
27  #define RCC_BASE                (MPU_AHB4_PERIPH_BASE + 0x0000)
28  #define RCC_MP_AHB4ENSETR       (RCC_BASE + 0XA28)
29  #define GPIOI_BASE              (MPU_AHB4_PERIPH_BASE + 0xA000)
30  #define GPIOI_MODER             (GPIOI_BASE + 0x0000)
31  #define GPIOI_OTYPER            (GPIOI_BASE + 0x0004)
32  #define GPIOI_OSPEEDR           (GPIOI_BASE + 0x0008)
33  #define GPIOI_PUPDR             (GPIOI_BASE + 0x000C)
34  #define GPIOI_BSRR              (GPIOI_BASE + 0x0018)
35  #define REGISTER_LENGTH         4
36
37  /* @description   : 释放 flatform 设备模块的时候此函数会执行
38   * @param - dev   : 要释放的设备
39   * @return        : 无
40   */
41  static void led_release(struct device *dev)
42  {
43      printk("led device released!\r\n");
44  }
45
46  /*
47   * 设备资源信息，也就是 LED0 所使用的所有寄存器
48   */
49  static struct resource led_resources[] = {
50      [0] = {
```

```
51          .start = RCC_MP_AHB4ENSETR,
52          .end   = (RCC_MP_AHB4ENSETR + REGISTER_LENGTH - 1),
53          .flags = IORESOURCE_MEM,
54      },
55      [1] = {
56          .start = GPIOI_MODER,
57          .end   = (GPIOI_MODER + REGISTER_LENGTH - 1),
58          .flags = IORESOURCE_MEM,
59      },
60      [2] = {
61          .start = GPIOI_OTYPER,
62          .end   = (GPIOI_OTYPER + REGISTER_LENGTH - 1),
63          .flags = IORESOURCE_MEM,
64      },
65      [3] = {
66          .start = GPIOI_OSPEEDR,
67          .end   = (GPIOI_OSPEEDR + REGISTER_LENGTH - 1),
68          .flags = IORESOURCE_MEM,
69      },
70      [4] = {
71          .start = GPIOI_PUPDR,
72          .end   = (GPIOI_PUPDR + REGISTER_LENGTH - 1),
73          .flags = IORESOURCE_MEM,
74      },
75      [5] = {
76          .start = GPIOI_BSRR,
77          .end   = (GPIOI_BSRR + REGISTER_LENGTH - 1),
78          .flags = IORESOURCE_MEM,
79      },
80  };
81
82  /*
83   * platform 设备结构体
84   */
85  static struct platform_device leddevice = {
86      .name  = "stm32mp1-led",
87      .id    = -1,
88      .dev   = {
89          .release = &led_release,
90      },
91      .num_resources = ARRAY_SIZE(led_resources),
92      .resource = led_resources,
93  };
```

```
94
95  /*
96   * @description   : 设备模块加载
97   * @param         : 无
98   * @return        : 无
99   */
100 static int __init leddevice_init(void)
101 {
102     return platform_device_register(&leddevice);
103 }
104
105 /*
106  * @description   : 设备模块注销
107  * @param         : 无
108  * @return        : 无
109  */
110 static void __exit leddevice_exit(void)
111 {
112     platform_device_unregister(&leddevice);
113 }
114
115 module_init(leddevice_init);
116 module_exit(leddevice_exit);
117 MODULE_LICENSE("GPL");
118 MODULE_AUTHOR("ALIENTEK");
119 MODULE_INFO(intree, "Y");
```

leddevice.c 文件内容就是按照示例代码 34.2.3.4 的 platform 设备模板编写的。

第 49~80 行，led_resources 数组，也就是设备资源，描述了 LED 所要使用到的寄存器信息，也就是 IORESOURCE_MEM 资源。

第 85~93，platform 设备结构体变量 leddevice，这里要注意 name 字段为“stm32mp1-led”，所以稍后编写 platform 驱动中的 name 字段也要为“stm32mp1-led”，否则设备和驱动匹配失败。

第 100~103 行，设备模块加载函数，在此函数里面通过 platform_device_register 向 Linux 内核注册 leddevice 这个 platform 设备。

第 110~113 行，设备模块卸载函数，在此函数里面通过 platform_device_unregister 从 Linux 内核中删除掉 leddevice 这个 platform 设备。

leddevice.c 文件编写完成以后就编写 leddriver.c 这个 platform 驱动文件，在 leddriver.c 里面输入如下内容：

示例代码 54.4.1.2 leddriver.c 文件代码段

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
```

```
6   #include <linux/module.h>
7   #include <linux/errno.h>
8   #include <linux/gpio.h>
9   #include <linux/cdev.h>
10  #include <linux/device.h>
11  #include <linux/of_gpio.h>
12  #include <linux/semaphore.h>
13  #include <linux/timer.h>
14  #include <linux/irq.h>
15  #include <linux/wait.h>
16  #include <linux/poll.h>
17  #include <linux/fs.h>
18  #include <linux/fcntl.h>
19  #include <linux/platform_device.h>
20  #include <asm/mach/map.h>
21  #include <asm/uaccess.h>
22  #include <asm/io.h>
23
24  #define LEDDEV_CNT      1             /* 设备号长度    */
25  #define LEDDEV_NAME     "platled"     /* 设备名字      */
26  #define LEDOFF          0
27  #define LEDON           1
28
29  /* 映射后的寄存器虚拟地址指针 */
30  static void __iomem *MPU_AHB4_PERIPH_RCC_PI;
31  static void __iomem *GPIOI_MODER_PI;
32  static void __iomem *GPIOI_OTYPER_PI;
33  static void __iomem *GPIOI_OSPEEDR_PI;
34  static void __iomem *GPIOI_PUPDR_PI;
35  static void __iomem *GPIOI_BSRR_PI;
36
37  /* leddev 设备结构体 */
38  struct leddev_dev{
39      dev_t devid;              /* 设备号       */
40      struct cdev cdev;         /* cdev         */
41      struct class *class;      /* 类           */
42      struct device *device;    /* 设备         */
43  };
44
45  struct leddev_dev leddev;     /* led 设备     */
46
47  /*
48   * @description   : LED 打开/关闭
```

```
49   * @param - sta   : LEDON(0) 打开 LED，LEDOFF(1) 关闭 LED
50   * @return        : 无
51   */
52  void led_switch(u8 sta)
53  {
54      u32 val = 0;
55      if(sta == LEDON) {
56          val = readl(GPIOI_BSRR_PI);
57          val |= (1 << 16);
58          writel(val, GPIOI_BSRR_PI);
59      }else if(sta == LEDOFF) {
60          val = readl(GPIOI_BSRR_PI);
61          val|= (1 << 0);
62          writel(val, GPIOI_BSRR_PI);
63      }
64  }
65
66  /*
67   * @description   : 取消映射
68   * @return        : 无
69   */
70  void led_unmap(void)
71  {
72      /* 取消映射 */
73      iounmap(MPU_AHB4_PERIPH_RCC_PI);
74      iounmap(GPIOI_MODER_PI);
75      iounmap(GPIOI_OTYPER_PI);
76      iounmap(GPIOI_OSPEEDR_PI);
77      iounmap(GPIOI_PUPDR_PI);
78      iounmap(GPIOI_BSRR_PI);
79  }
80
81  /*
82   * @description   : 打开设备
83   * @param - inode : 传递给驱动的 inode
84   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
85   *                  一般在 open 的时候将 private_data 指向设备结构体。
86   * @return        : 0 成功;其他 失败
87   */
88  static int led_open(struct inode *inode, struct file *filp)
89  {
90      return 0;
91  }
```

```
92
93  /*
94   * @description   : 向设备写数据
95   * @param - filp  : 设备文件，表示打开的文件描述符
96   * @param - buf   : 要写给设备写入的数据
97   * @param - cnt   : 要写入的数据长度
98   * @param - offt  : 相对于文件首地址的偏移
99   * @return        : 写入的字节数，如果为负值，表示写入失败
100  */
101 static ssize_t led_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
102 {
103     int retvalue;
104     unsigned char databuf[1];
105     unsigned char ledstat;
106
107     retvalue = copy_from_user(databuf, buf, cnt);
108     if(retvalue < 0) {
109         printk("kernel write failed!\r\n");
110         return -EFAULT;
111     }
112
113     ledstat = databuf[0];       /* 获取状态值   */
114     if(ledstat == LEDON) {
115         led_switch(LEDON);      /* 打开 LED 灯   */
116     }else if(ledstat == LEDOFF) {
117         led_switch(LEDOFF);     /* 关闭 LED 灯   */
118     }
119
120     return 0;
121 }
122
123 /* 设备操作函数 */
124 static struct file_operations led_fops = {
125     .owner = THIS_MODULE,
126     .open = led_open,
127     .write = led_write,
128 };
129
130 /*
131  * @description   : flatform 驱动的 probe 函数
132  * @param - dev   : platform 设备
133  * @return        : 0，成功;其他负值,失败
```

```
134 */
135 static int led_probe(struct platform_device *dev)
136 {
137     int i = 0, ret;
138     int ressize[6];
139     u32 val = 0;
140     struct resource *ledsource[6];
141 
142     printk("led driver and device has matched!\r\n");
143     /* 1、获取资源 */
144     for (i = 0; i < 6; i++) {
145         ledsource[i] = platform_get_resource(dev, IORESOURCE_MEM, i);
146         if (!ledsource[i]) {
147             dev_err(&dev->dev, "No MEM resource for always on\n");
148             return -ENXIO;
149         }
150         ressize[i] = resource_size(ledsource[i]);
151     }
152 
153     /* 2、初始化 LED */
154     /* 寄存器地址映射 */
155     MPU_AHB4_PERIPH_RCC_PI = ioremap(ledsource[0]->start,
                                         ressize[0]);
156     GPIOI_MODER_PI = ioremap(ledsource[1]->start, ressize[1]);
157     GPIOI_OTYPER_PI = ioremap(ledsource[2]->start, ressize[2]);
158     GPIOI_OSPEEDR_PI = ioremap(ledsource[3]->start, ressize[3]);
159     GPIOI_PUPDR_PI = ioremap(ledsource[4]->start, ressize[4]);
160     GPIOI_BSRR_PI = ioremap(ledsource[5]->start, ressize[5]);
161 
162     /* 3、使能 PI 时钟 */
163     val = readl(MPU_AHB4_PERIPH_RCC_PI);
164     val &= ~(0X1 << 8);  /* 清除以前的设置    */
165     val |= (0X1 << 8);   /* 设置新值          */
166     writel(val, MPU_AHB4_PERIPH_RCC_PI);
167 
168     /* 4、设置 PI0 通用的输出模式。*/
169     val = readl(GPIOI_MODER_PI);
170     val &= ~(0X3 << 0);  /* bit0:1 清零      */
171     val |= (0X1 << 0);   /* bit0:1 设置 01   */
172     writel(val, GPIOI_MODER_PI);
173 
174     /* 5、设置 PI0 为推挽模式。*/
175     val = readl(GPIOI_OTYPER_PI);
```

```
176     val &= ~(0X1 << 0);  /* bit0 清零，设置为上拉*/
177     writel(val, GPIOI_OTYPER_PI);
178
179     /* 6、设置 PI0 为高速。*/
180     val = readl(GPIOI_OSPEEDR_PI);
181     val &= ~(0X3 << 0);  /* bit0:1 清零          */
182     val |= (0x2 << 0);   /* bit0:1 设置为 10     */
183     writel(val, GPIOI_OSPEEDR_PI);
184
185     /* 7、设置 PI0 为上拉。*/
186     val = readl(GPIOI_PUPDR_PI);
187     val &= ~(0X3 << 0);  /* bit0:1 清零          */
188     val |= (0x1 << 0);   /*bit0:1 设置为 01      */
189     writel(val,GPIOI_PUPDR_PI);
190
191     /* 8、默认关闭 LED */
192     val = readl(GPIOI_BSRR_PI);
193     val |= (0x1 << 0);
194     writel(val, GPIOI_BSRR_PI);
195
196     /* 注册字符设备驱动 */
197     /* 1、申请设备号 */
198     ret = alloc_chrdev_region(&leddev.devid, 0, LEDDEV_CNT,
                                  LEDDEV_NAME);
199     if(ret < 0)
200         goto fail_map;
201
202     /* 2、初始化 cdev */
203     leddev.cdev.owner = THIS_MODULE;
204     cdev_init(&leddev.cdev, &led_fops);
205
206     /* 3、添加一个 cdev */
207     ret = cdev_add(&leddev.cdev, leddev.devid, LEDDEV_CNT);
208     if(ret < 0)
209         goto del_unregister;
210
211     /* 4、创建类 */
212     leddev.class = class_create(THIS_MODULE, LEDDEV_NAME);
213     if (IS_ERR(leddev.class)) {
214         goto del_cdev;
215     }
216
217     /* 5、创建设备 */
```

```
218     leddev.device = device_create(leddev.class, NULL, leddev.devid,
                                      NULL, LEDDEV_NAME);
219     if (IS_ERR(leddev.device)) {
220         goto destroy_class;
221     }
222     return 0;
223
224 destroy_class:
225     class_destroy(leddev.class);
226 del_cdev:
227     cdev_del(&leddev.cdev);
228 del_unregister:
229     unregister_chrdev_region(leddev.devid, LEDDEV_CNT);
230 fail_map:
231     led_unmap();
232     return -EIO;
233 }
234
235 /*
236  * @description   : platform 驱动的 remove 函数
237  * @param - dev   : platform 设备
238  * @return        : 0，成功;其他负值,失败
239  */
240 static int led_remove(struct platform_device *dev)
241 {
242     led_unmap();                /* 取消映射     */
243     cdev_del(&leddev.cdev);     /* 删除 cdev    */
244     unregister_chrdev_region(leddev.devid, LEDDEV_CNT); /* 注销设备号
*/
245     device_destroy(leddev.class, leddev.devid);   /* 注销设备 */
246     class_destroy(leddev.class);                  /* 注销类 */
247     return 0;
248 }
249
250 /* platform 驱动结构体 */
251 static struct platform_driver led_driver = {
252     .driver     = {
253         .name   = "stm32mp1-led",    /* 驱动名字，用于和设备匹配 */
254     },
255     .probe      = led_probe,
256     .remove     = led_remove,
257 };
258
```

```
259 /*
260  * @description   : 驱动模块加载函数
261  * @param         : 无
262  * @return        : 无
263  */
264 static int __init leddriver_init(void)
265 {
266     return platform_driver_register(&led_driver);
267 }
268
269 /*
270  * @description   : 驱动模块卸载函数
271  * @param         : 无
272  * @return        : 无
273  */
274 static void __exit leddriver_exit(void)
275 {
276     platform_driver_unregister(&led_driver);
277 }
278
279 module_init(leddriver_init);
280 module_exit(leddriver_exit);
281 MODULE_LICENSE("GPL");
282 MODULE_AUTHOR("ALIENTEK");
283 MODULE_INFO(intree, "Y");
```

leddriver.c 文件内容就是按照示例代码 34.2.2.5 的 platform 驱动模板编写的。

第 88~128 行，传统的字符设备驱动。

第 135~233 行，probe 函数，当设备和驱动匹配以后此函数就会执行，当匹配成功以后会在终端上输出“led driver and device has matched!”。在 probe 函数里面初始化 LED、注册字符设备驱动。也就是将原来在驱动加载函数里面做的工作全部放到 probe 函数里面完成。

第 240~248 行，remove 函数，当卸载 platform 驱动的时候此函数就会执行。在此函数里面释放内存、注销字符设备等。也就是将原来驱动卸载函数里面的工作全部都放到 remove 函数中完成。

第 251~257 行，platform_driver 驱动结构体，注意 name 字段为"stm32mp1-led"，和我们在 leddevice.c 文件里面设置的设备 name 字段一致。

第 264~267 行，驱动模块加载函数，在此函数里面通过 platform_driver_register 向 Linux 内核注册 led_driver 驱动。

第 274~277 行，驱动模块卸载函数，在此函数里面通过 platform_driver_unregister 从 Linux 内核卸载 led_driver 驱动。

34.4.2 测试 APP 编写

测试 APP 的内容很简单，就是打开和关闭 LED 灯，新建 ledApp.c 这个文件，然后在里面输入如下内容：

示例代码 34.4.2.1 ledApp.c 文件代码段

```
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "fcntl.h"
6  #include "stdlib.h"
7  #include "string.h"
8  /***************************************************************
9  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
10 文件名       : ledApp.c
11 作者         : 正点原子 Linux 团队
12 版本         : V1.0
13 描述         : platform 驱动驱测试 APP。
14 其他         : 无
15 使用方法     : ./ledApp /dev/platled  0 关闭 LED
16               ./ledApp /dev/platled  1 打开 LED
17 论坛         : www.openedv.com
18 日志         : 初版 V1.0 2019/8/16 正点原子 Linux 团队创建
19 ***************************************************************/
20 #define LEDOFF   0
21 #define LEDON    1
22
23 /*
24  * @description    : main 主程序
25  * @param - argc   : argv 数组元素个数
26  * @param - argv   : 具体参数
27  * @return         : 0 成功;其他 失败
28  */
29 int main(int argc, char *argv[])
30 {
31     int fd, retvalue;
32     char *filename;
33     unsigned char databuf[1];
34
35     if(argc != 3){
36         printf("Error Usage!\r\n");
37         return -1;
38     }
39
40     filename = argv[1];
41     /* 打开 led 驱动 */
42     fd = open(filename, O_RDWR);
```

```
43      if(fd < 0){
44          printf("file %s open failed!\r\n", argv[1]);
45          return -1;
46      }
47
48      databuf[0] = atoi(argv[2]); /* 要执行的操作：打开或关闭 */
49      retvalue = write(fd, databuf, sizeof(databuf));
50      if(retvalue < 0){
51          printf("LED Control Failed!\r\n");
52          close(fd);
53          return -1;
54      }
55
56      retvalue = close(fd); /* 关闭文件 */
57      if(retvalue < 0){
58          printf("file %s close failed!\r\n", argv[1]);
59          return -1;
60      }
61      return 0;
62  }
```

ledApp.c 文件内容很简单，就是控制 LED 灯的亮灭，和第四十一章的测试 APP 基本一致，这里就不重复讲解了。

34.5 运行测试

34.5.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为“leddevice.o leddriver.o”，Makefile 内容如下所示：

示例代码 34.5.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := leddevice.o
5  obj-m += leddriver.o
......
12 clean:
13  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4，5 行，设置 obj-m 变量的值为“leddevice.o leddriver.o”。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“leddevice.ko leddriver.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 ledApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc ledApp.c -o ledApp
```

编译成功以后就会生成 ledApp 这个应用程序。

34.4.2 运行测试

将上一小节编译出来 leddevice.ko、leddriver.ko 和 ledApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 leddevice.ko 设备模块和 leddriver.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe leddevice.ko   //加载设备模块
modprobe leddriver.ko   //加载驱动模块
```

根文件系统中/sys/bus/platform/目录下保存着当前板子 platform 总线下的设备和驱动，其中 devices 子目录为 platform 设备，drivers 子目录为 plartofm 驱动。进入/sys/bus/platform/devices/目录，查看我们的设备是否存在，我们在 leddevice.c 中设置设备的 name 字段为“stm32mp1-led”，因此肯定在/sys/bus/platform/devices/目录下存在一个名字“stm32mp1-led”的文件，否则说明我们的设备模块加载失败，结果如图 34.4.2.1 所示：

```
5800a000.ethernet                soc
59000000.gpu                     soc:pin-controller-z@54004000
5a002000.watchdog                soc:pin-controller@50002000
5a007000.perf                    stm32-cpufreq
5c004000.rtc                     stm32mp1-led        <-- 设备
5c005000.nvmem                   timer
[root@ATK-stm32mp1]:/sys/bus/platform/devices$
```

图 34.4.2.1 stm32mp1-led 设备

同理，查看/sys/bus/platform/drivers/目录，看一下驱动是否存在，我们在 leddriver.c 中设置 name 字段为“stm32mp1-led”，因此会在/sys/bus/platform/drivers/目录下存在名为“stm32mp1-led”这个文件，结果如图 34.4.2.2 所示：

```
bcm6368_nand        rcar-lvds           stm32_hwspinlock
bcma-host-soc       reg-dummy           stm32_pwr
bdc                 reg-fixed-voltage   stm32_rtc
brcm-kona-reset     restart-gpio        stm32f7-i2c          <-- 驱动
brcmstb_nand        rfkill_gpio         stm32mp1-led
cdns-spi            rk808-regulator     stm32mp157-pinctrl
chipidea-usb2       rn5t618-regulator   stm32mp1_rcc
```

图 34.4.2.2 stm32mp1-led 驱动

驱动模块和设备模块加载成功以后 platform 总线就会进行匹配，当驱动和设备匹配成功以后就会输出如图 34.4.2.3 所示一行语句：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe leddevice.ko
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe leddriver.ko
led driver and device has matched!      <-- 驱动和设备匹配成功
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 34.4.2.3 驱动和设备匹配成功

驱动和设备匹配成功以后就可以测试 LED 灯驱动了，输入如下命令打开 LED 灯：

```
./ledApp /dev/platled 1   //打开 LED 灯
```

在输入如下命令关闭 LED 灯：

```
./ledApp /dev/platled 0   //关闭 LED 灯
```

观察一下 LED 灯能否打开和关闭，如果可以的话就说明驱动工作正常，如果要卸载驱动的话输入如下命令即可：

```
rmmod leddevice.ko
rmmod leddriver.ko
```

第三十五章 设备树下的 platform 驱动编写

上一章我们详细的讲解了 Linux 下的驱动分离与分层，以及总线、设备和驱动这样的驱动框架。基于总线、设备和驱动这样的驱动框架，Linux 内核提出来 platform 这个虚拟总线，相应的也有 platform 设备和 platform 驱动。上一章我们讲解了传统的、未采用设备树的 platform 设备和驱动编写方法。最新的 Linux 内核已经支持了设备树，因此在设备树下如何编写 platform 驱动就显得尤为重要，本章我们就来学习一下如何在设备树下编写 platform 驱动。

35.1 设备树下的 platform 驱动简介

platform 驱动框架分为总线、设备和驱动，其中总线不需要我们这些驱动程序员去管理，这个是 Linux 内核提供的，我们在编写驱动的时候只要关注于设备和驱动的具体实现即可。在没有设备树的 Linux 内核下，我们需要分别编写并注册 platform_device 和 platform_driver，分别代表设备和驱动。在使用设备树的时候，设备的描述被放到了设备树中，因此 platform_device 就不需要我们去编写了，我们只需要实现 platform_driver 即可。

35.1.1 修改 pinctrl-stm32.c 文件

在前面第 25 章的时候我们详细的讲解了 pinctrl，但是在后续的实验中却一直没有使用 pinctrl。按道理来讲，在我们使用某个引脚的时候需要先配置其电气属性，比如复用、输入还是输入、默认上下拉等！但是在前面的实验中均没有配置引脚的电气属性，也就是引脚的 pinctrl 配置。这是因为 ST 针对 STM32MP1 提供的 Linux 系统中，其 pinctrl 配置的电气属性只能在 platform 平台下被引用，前面的实验都没用到 platform，所以 pinctrl 配置是不起作用的！

笔者在使用 NXP 的 I.MX6ULL 芯片的时候，Linux 系统启动运行过程中会自动解析设备树下的 pinctrl 配置，然后初始化引脚的电气属性，不需要 platform 驱动框架。所以 pinctrl 什么时候有效，不同的芯片厂商有不同的处理方法，一切以实际所使用的芯片为准！

对于 STM32MP1 来说，在使用 pinctrl 的时候需要修改一下 pinctrl-stm32.c 这个文件，否则当某个引脚用作 GPIO 的时候会提示此引脚无法申请到，如图 35.1.1.1 所示：

```
stm32mp157-pinctrl soc:pin-controller@50002000: pin PI0 already requested by gpioled; cannot claim
 for GPIOI:128
stm32mp157-pinctrl soc:pin-controller@50002000: pin-128 (GPIOI:128) status -22
led: Failed to request led-gpio
stm32mp1-led: probe of gpioled failed with error -22
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 35.1.1.1 IO 申请失败

从图 35.1.1.1 可以看出，提示 PI0 这个 IO 已经被其他外设申请走了，不能再申请。打开 pinctrl-stm32.c 这个文件，找到如下所示代码：

示例代码 35.1.1.1 stm32_pmx_ops 结构体

```
1 static const struct pinmux_ops stm32_pmx_ops = {
2       .get_functions_count    = stm32_pmx_get_funcs_cnt,
3       .get_function_name  = stm32_pmx_get_func_name,
4       .get_function_groups    = stm32_pmx_get_func_groups,
5       .set_mux        = stm32_pmx_set_mux,
6       .gpio_set_direction = stm32_pmx_gpio_set_direction,
7       .strict         = true,
8 };
```

第 7 行的 strict 成员变量默认为 true，我们需要将其改为 false，改完以后如图 35.1.1.2 所示：

```
865  static const struct pinmux_ops stm32_pmx_ops = {
866      .get_functions_count    = stm32_pmx_get_funcs_cnt,
867      .get_function_name  = stm32_pmx_get_func_name,
868      .get_function_groups    = stm32_pmx_get_func_groups,
869      .set_mux        = stm32_pmx_set_mux,
870      .gpio_set_direction = stm32_pmx_gpio_set_direction,
871      .strict         = false,        改为false
872  };
```

图 35.1.1.2 修改后的 stm32_pmx_ops 结构体

修改完成以后使用如下命令重新编译 Linux 内核：

```
make uImage LOADADDR=0XC2000040 -j16  //编译内核
```

编译完成以后使用新的 uImage 启动即可。

35.1.2 创建设备的 pinctrl 节点

上面已经说了，在 platform 驱动框架下必须使用 pinctrl 来配置引脚复用功能。我们以本章实验需要用到的 LED0 为例，编写 LED0 引脚的 pinctrl 配置。打开 stm32mp15-pinctrl.dtsi 文件，STM32MP1 的所有引脚 pinctrl 配置都是在这个文件里面完成的，在 pinctrl 节点下添加如下所示内容：

示例代码 35.1.2.1 GPIO 的 pinctrl 配置

```
1 led_pins_a: gpioled-0 {
2     pins {
3         pinmux = <STM32_PINMUX('I', 0, GPIO)>;
4         drive-push-pull;
5         bias-pull-up;
6         output-high;
7         slew-rate = <0>;
8     };
9 };
```

示例代码 35.1.2.1 中的 led_pins_a 节点就是 LED 的 pinctrl 配置，把 PI0 端口复用为 GPIO 功能，同时设置 PI0 的电气特性。我们已经在 25.1.1 小节详细的讲解过如何配置 STM32MP1 的电气属性，大家可以回过头去看一下，这里我们就简单介绍一下 LED0 的配置：

第 3 行，设置 PI0 复用为 GPIO 功能。

第 4 行，设置 PI0 为推挽输出。

第 5 行，设置 PI0 内部上拉。

第 6 行，设置 PI0 默认输出高电平。

第 7 行，设置 PI0 的速度为 0 档，也就是最慢。

添加完成以后如图 35.1.2.1 所示：

```
8  &pinctrl {
9      /* gpioled zuozhongkai */
10     led_pins_a: gpioled-0 {
11         pins {
12             pinmux = <STM32_PINMUX('I', 0, GPIO)>;
13             drive-push-pull;
14             bias-pull-up;
15             output-high;
16             slew-rate = <0>;
17         };
18     };
19
```

led的pinctrl节点

图 35.1.2.1 led 对应的 pinctrl 节点

35.1.3 在设备树中创建设备节点

接下来要在设备树中创建设备节点来描述设备信息，重点是要设置好 compatible 属性的值，因为 platform 总线需要通过设备节点的 compatible 属性值来匹配驱动!这点要切记。修改 25.4.1.2

小节中我们创建的 gpioled 节点，修改以后的内容如下：

示例代码 35.1.3.1 gpioled 设备节点

```
1 gpioled {
2       compatible = "alientek,led";
3       pinctrl-names = "default";
4       status = "okay";
5       pinctrl-0 = <&led_pins_a>;
6       led-gpio = <&gpioi 0 GPIO_ACTIVE_LOW>;
7 };
```

第 2 行的 compatible 属性值为“alientek,led”，因此一会在编写 platform 驱动的时候 of_match_table 属性表中要有“alientek,led”。

第 5 行里，pinctrl-0 属性设置 LED 的 PIN 对应的 pinctrl 节点，也就是我们在示例代码 35.1.1 中编写的 led_pins_a。

35.1.4 编写 platform 驱动的时候要注意兼容属性

上一章已经详细的讲解过了，在使用设备树的时候 platform 驱动会通过 of_match_table 来保存兼容性值，也就是表明此驱动兼容哪些设备。所以，of_match_table 将会尤为重要，比如本例程的 platform 驱动中 platform_driver 就可以按照如下所示设置：

示例代码 35.1.4.1 of_match_table 匹配表的设置

```
1  static const struct of_device_id led_of_match[] = {
2      { .compatible = "alientek,led" },  /* 兼容属性 */
3      { /* Sentinel */ }
4  };
5
6  MODULE_DEVICE_TABLE(of, led_of_match);
7
8  static struct platform_driver led_platform_driver = {
9      .driver = {
10         .name       = "stm32mp1-led",
11         .of_match_table = led_of_match,
12     },
13     .probe          = led_probe,
14     .remove         = led_remove,
15 };
```

第 1~4 行，of_device_id 表，也就是驱动的兼容表，是一个数组，每个数组元素为 of_device_id 类型。每个数组元素都是一个兼容属性，表示兼容的设备，一个驱动可以跟多个设备匹配。这里我们仅仅匹配了一个设备，那就是示例代码 35.1.2 中创建的 gpioled 这个设备。第 2 行的 compatible 值为“alientek,led”，驱动中的 compatible 属性和设备中的 compatible 属性相匹配，因此驱动中对应的 probe 函数就会执行。注意第 3 行是一个空元素，在编写 of_device_id 的时候最后一个元素一定要为空！

第 6 行，通过 MODULE_DEVICE_TABLE 声明一下 led_of_match 这个设备匹配表。

第 11 行，设置 platform_driver 中的 of_match_table 匹配表为上面创建的 leds_of_match，至此我们就设置好了 platform 驱动的匹配表了。

最后就是编写驱动程序，基于设备树的 platform 驱动和上一章无设备树的 platform 驱动基本一样，都是当驱动和设备匹配成功以后先根据设备树里的 pinctrl 属性设置 PIN 的电气特性再去执行 probe 函数。我们需要在 probe 函数里面执行字符设备驱动那一套，当注销驱动模块的时候 remove 函数就会执行，都是大同小异的。

35.2 检查引脚复用配置

35.2.1 检查引脚 pinctrl 配置

STM32MP1 的一个引脚可以复用为多种功能，比如 PI0 可以作为 GPIO、TIM5_CH4、SPI2_NSS、DCMI_D13、LCD_G5 等。我们在做 STM32 单片机开发的时候，一个 IO 可以被多个外设使用，比如 PI0 同时作为 TIM5_CH4、LCD_G5，但是同一时刻只能用做一个功能，比如做 LCD_G5 的时候就不能做 TIM5_CH4！在嵌入式 Linux 下，我们要严格按照一个引脚对应一个功能来设计硬件，比如 PI0 现在要用作 GPIO 来驱动 LED 灯，那么就不能将 PI0 作为其他功能，比如你在设计硬件的时候就不能再将 PI0 作为 LCD_G5。

正点原子 STM32MP1 开发板上将 PI0 连接到了 LED0 上，也就是将其用作普通的 GPIO，对应的 pinctrl 配置就是示例代码 35.1.2.1。但是 stm32mp15-pinctrl.dtsi 是 ST 根据自己官方 EVK 开发板编写的，因此 PI0 就可能被 ST 官方用作其他功能，大家在 stm32mp15-pinctrl.dtsi 里面找到如下所示代码：

```
407     ltdc_pins_a: ltdc-a-0 {
408         pins {
409             pinmux = <STM32_PINMUX('G',  7, AF14)>, /* LCD_CLK */
410                      <STM32_PINMUX('I', 10, AF14)>, /* LCD_HSYNC */
411                      <STM32_PINMUX('I',  9, AF14)>, /* LCD_VSYNC */
412                      <STM32_PINMUX('F', 10, AF14)>, /* LCD_DE */
413                      <STM32_PINMUX('H',  2, AF14)>, /* LCD_R0 */
414                      <STM32_PINMUX('H',  3, AF14)>, /* LCD_R1 */
415                      <STM32_PINMUX('H',  8, AF14)>, /* LCD_R2 */
416                      <STM32_PINMUX('H',  9, AF14)>, /* LCD_R3 */
417                      <STM32_PINMUX('H', 10, AF14)>, /* LCD_R4 */
418                      <STM32_PINMUX('C',  0, AF14)>, /* LCD_R5 */
419                      <STM32_PINMUX('H', 12, AF14)>, /* LCD_R6 */
420                      <STM32_PINMUX('E', 15, AF14)>, /* LCD_R7 */
421                      <STM32_PINMUX('E',  5, AF14)>, /* LCD_G0 */
422                      <STM32_PINMUX('E',  6, AF14)>, /* LCD_G1 */
423                      <STM32_PINMUX('H', 13, AF14)>, /* LCD_G2 */
424                      <STM32_PINMUX('H', 14, AF14)>, /* LCD_G3 */
425                      <STM32_PINMUX('H', 15, AF14)>, /* LCD_G4 */
426                      <STM32_PINMUX('I',  0, AF14)>, /* LCD_G5 */
427                      <STM32_PINMUX('I',  1, AF14)>, /* LCD_G6 */
428                      <STM32_PINMUX('I',  2, AF14)>, /* LCD_G7 */
429                      <STM32_PINMUX('D',  9, AF14)>, /* LCD_B0 */
430                      <STM32_PINMUX('G', 12, AF14)>, /* LCD_B1 */
431                      <STM32_PINMUX('G', 10, AF14)>, /* LCD_B2 */
432                      <STM32_PINMUX('D', 10, AF14)>, /* LCD_B3 */
433                      <STM32_PINMUX('I',  4, AF14)>, /* LCD_B4 */
434                      <STM32_PINMUX('A',  3, AF14)>, /* LCD_B5 */
435                      <STM32_PINMUX('B',  8, AF14)>, /* LCD_B6 */
436                      <STM32_PINMUX('D',  8, AF14)>; /* LCD_B7 */
437             bias-disable;
438             drive-push-pull;
439             slew-rate = <1>;
440         };
441     };
```

默认作为LCD_G5

图 35.2.1.1 ltdc_pins_a 节点

从图 35.2.1.1 可以看出，ST 官方默认将 PI0 复用为 LCD_G5，前面说了，一个 IO 只能复用为一个功能，因此我们需要将图 35.2.1.1 中的“<STM32_PINMUX('I', 0, AF14)>”屏蔽掉，因为我们现在要将 PI0 用作 GPIO。同样的，继续在 stm32mp15-pinctrl.dtsi 文件里面查找，会发现如图 35.2.1.2 所示的地方也将 PI0 复用为了 LCD_G5：

```
443     ltdc_pins_sleep_a: ltdc-a-1 {
444         pins {
445             pinmux = <STM32_PINMUX('G',  7, ANALOG)>, /* LCD_CLK */
446                      <STM32_PINMUX('I', 10, ANALOG)>, /* LCD_HSYNC */
447                      <STM32_PINMUX('I',  9, ANALOG)>, /* LCD_VSYNC */
448                      <STM32_PINMUX('F', 10, ANALOG)>, /* LCD_DE */
449                      <STM32_PINMUX('H',  2, ANALOG)>, /* LCD_R0 */
450                      <STM32_PINMUX('H',  3, ANALOG)>, /* LCD_R1 */
451                      <STM32_PINMUX('H',  8, ANALOG)>, /* LCD_R2 */
452                      <STM32_PINMUX('H',  9, ANALOG)>, /* LCD_R3 */
453                      <STM32_PINMUX('H', 10, ANALOG)>, /* LCD_R4 */
454                      <STM32_PINMUX('C',  0, ANALOG)>, /* LCD_R5 */
455                      <STM32_PINMUX('H', 12, ANALOG)>, /* LCD_R6 */
456                      <STM32_PINMUX('E', 15, ANALOG)>, /* LCD_R7 */
457                      <STM32_PINMUX('E',  5, ANALOG)>, /* LCD_G0 */
458                      <STM32_PINMUX('E',  6, ANALOG)>, /* LCD_G1 */
459                      <STM32_PINMUX('H', 13, ANALOG)>, /* LCD_G2 */
460                      <STM32_PINMUX('H', 14, ANALOG)>, /* LCD_G3 */
461                      <STM32_PINMUX('H', 15, ANALOG)>, /* LCD_G4 */
462                      <STM32_PINMUX('I',  0, ANALOG)>, /* LCD_G5 */
463                      <STM32_PINMUX('I',  1, ANALOG)>, /* LCD_G6 */
464                      <STM32_PINMUX('I',  2, ANALOG)>, /* LCD_G7 */
465                      <STM32_PINMUX('D',  9, ANALOG)>, /* LCD_B0 */
466                      <STM32_PINMUX('G', 12, ANALOG)>, /* LCD_B1 */
467                      <STM32_PINMUX('G', 10, ANALOG)>, /* LCD_B2 */
468                      <STM32_PINMUX('D', 10, ANALOG)>, /* LCD_B3 */
469                      <STM32_PINMUX('I',  4, ANALOG)>, /* LCD_B4 */
470                      <STM32_PINMUX('A',  3, ANALOG)>, /* LCD_B5 */
471                      <STM32_PINMUX('B',  8, ANALOG)>, /* LCD_B6 */
472                      <STM32_PINMUX('D',  8, ANALOG)>; /* LCD_B7 */
473         };
474     };
```

图 35.2.1.2 ltdc_pins_sleep_a 节点

图 35.2.1.2 中的 ltdc_pins_sleep_a 节点也将 PI0 复用为 LCD_G5，将这行代码也屏蔽掉。确保所使用的设备树中，一个引脚只复用为一个功能！

35.2.2 检查 GPIO 占用

上一小节只是检查了一下，PI0 这个引脚有没有被复用为多个设备，本节我们将 PI0 复用为 GPIO。因为我们是在 ST 官方提供的设备树上修改的，因此还要检查一下当 PI0 作为 GPIO 的时候，ST 官方有没有将这个 GPIO 分配给其他设备。其实对于 PI0 这个引脚来说不会的，因为 ST 官方将其复用为了 LCD_G5，所以也就不存在说将其在作为 GPIO 分配给其他设备。但是我们在实际开发中要考虑到这一点，说不定其他的引脚就会被分配给某个设备做 GPIO，而我们没有检查，导致两个设备都用这一个 GPIO，那么肯定有一个因为申请不到 GPIO 而导致驱动无法工作。

所以当我们将一个引脚用作 GPIO 的时候，一定要检查一下当前设备树里面是否有其他设备也使用到了这个 GPIO，保证设备树中只有一个设备树在使用这个 GPIO。

35.3 硬件原理图分析

本实验的硬件原理参考 21.2 小节即可。

35.4 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→18_dtsplatform。

本章实验我们编写基于设备树的 platform 驱动，所以需要在设备树中添加设备节点，然后我们只需要编写 platform 驱动即可。

35.4.1 修改设备树文件

首先修改设备树文件，加上我们需要的设备信息，本章我们就使用到一个 LED0。需要创建 LED0 引脚的 pinctrl 节点，这个直接使用示例代码 35.1.2.1 中的 led_pins_a 节点。另外也要创建一个 LED0 设备节点，这个直接使用示例代码 35.1.3.1 中的 gpioled 设备节点。

35.4.2 platform 驱动程序编写

设备已经准备好了，接下来就要编写相应的 platform 驱动了，新建名为“18_dtsplatform”的文件夹，然后在 18_dtsplatform 文件夹里面创建 vscode 工程，工作区命名为“dtsplatform”。新建名为 leddriver.c 的驱动文件，在 leddriver.c 中输入如下所示内容：

示例代码 35.4.2.1 leddriver.c 文件代码段

```
1  #include <linux/types.h>
2  #include <linux/kernel.h>
3  #include <linux/delay.h>
4  #include <linux/ide.h>
5  #include <linux/init.h>
6  #include <linux/module.h>
7  #include <linux/errno.h>
8  #include <linux/gpio.h>
9  #include <linux/cdev.h>
10 #include <linux/device.h>
11 #include <linux/of_gpio.h>
12 #include <linux/semaphore.h>
13 #include <linux/timer.h>
14 #include <linux/irq.h>
15 #include <linux/wait.h>
16 #include <linux/poll.h>
17 #include <linux/fs.h>
18 #include <linux/fcntl.h>
19 #include <linux/platform_device.h>
20 #include <asm/mach/map.h>
21 #include <asm/uaccess.h>
22 #include <asm/io.h>
23
24 #define LEDDEV_CNT   1                 /* 设备号长度    */
25 #define LEDDEV_NAME  "dtsplatled"      /* 设备名字      */
26 #define LEDOFF       0
27 #define LEDON        1
28
29 /* leddev 设备结构体 */
30 struct leddev_dev{
31     dev_t devid;                       /* 设备号          */
32     struct cdev cdev;                  /* cdev            */
```

```
33      struct class *class;                    /* 类               */
34      struct device *device;                  /* 设备             */
35      struct device_node *node;               /* LED 设备节点     */
36      int gpio_led;                           /* LED 灯 GPIO 标号 */
37  };
38
39  struct leddev_dev leddev;                   /* led 设备         */
40
41  /*
42   * @description   : LED 打开/关闭
43   * @param - sta   : LEDON(0) 打开 LED，LEDOFF(1) 关闭 LED
44   * @return        : 无
45   */
46  void led_switch(u8 sta)
47  {
48      if (sta == LEDON )
49          gpio_set_value(leddev.gpio_led, 0);
50      else if (sta == LEDOFF)
51          gpio_set_value(leddev.gpio_led, 1);
52  }
53
54  static int led_gpio_init(struct device_node *nd)
55  {
56      int ret;
57
58      /* 从设备树中获取 GPIO */
59      leddev.gpio_led = of_get_named_gpio(nd, "led-gpio", 0);
60      if(!gpio_is_valid(leddev.gpio_led)) {
61          printk(KERN_ERR "leddev: Failed to get led-gpio\n");
62          return -EINVAL;
63      }
64
65      /* 申请使用 GPIO */
66      ret = gpio_request(leddev.gpio_led, "LED0");
67      if (ret) {
68          printk(KERN_ERR "led: Failed to request led-gpio\n");
69          return ret;
70      }
71
72      /* 将 GPIO 设置为输出模式并设置 GPIO 初始电平状态 */
73      gpio_direction_output(leddev.gpio_led,1);
74
75      return 0;
```

```
76  }
77
78  /*
79   * @description   : 打开设备
80   * @param - inode : 传递给驱动的 inode
81   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
82   *                  一般在 open 的时候将 private_data 指向设备结构体。
83   * @return        : 0 成功;其他 失败
84   */
85  static int led_open(struct inode *inode, struct file *filp)
86  {
87      return 0;
88  }
89
90  /*
91   * @description   : 向设备写数据
92   * @param - filp  : 设备文件，表示打开的文件描述符
93   * @param - buf   : 要写给设备写入的数据
94   * @param - cnt   : 要写入的数据长度
95   * @param - offt  : 相对于文件首地址的偏移
96   * @return        : 写入的字节数，如果为负值，表示写入失败
97   */
98  static ssize_t led_write(struct file *filp, const char __user *buf,
                             size_t cnt, loff_t *offt)
99  {
100     int retvalue;
101     unsigned char databuf[1];
102     unsigned char ledstat;
103
104     retvalue = copy_from_user(databuf, buf, cnt);
105     if(retvalue < 0) {
106         printk("kernel write failed!\r\n");
107         return -EFAULT;
108     }
109
110     ledstat = databuf[0];
111     if (ledstat == LEDON) {
112         led_switch(LEDON);
113     } else if (ledstat == LEDOFF) {
114         led_switch(LEDOFF);
115     }
116     return 0;
117 }
```

```
118
119 /* 设备操作函数 */
120 static struct file_operations led_fops = {
121     .owner = THIS_MODULE,
122     .open = led_open,
123     .write = led_write,
124 };
125
126 /*
127  * @description   : flatform 驱动的 probe 函数，当驱动与设备匹配以后此函数
128  *                  就会执行
129  * @param - dev   : platform 设备
130  * @return        : 0，成功;其他负值,失败
131  */
132 static int led_probe(struct platform_device *pdev)
133 {
134     int ret;
135
136     printk("led driver and device was matched!\r\n");
137
138     /* 初始化 LED */
139     ret = led_gpio_init(pdev->dev.of_node);
140     if(ret < 0)
141         return ret;
142
143     /* 1、设置设备号 */
144     ret = alloc_chrdev_region(&leddev.devid, 0, LEDDEV_CNT,
                                  LEDDEV_NAME);
145     if(ret < 0) {
146         pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                 LEDDEV_NAME, ret);
147         goto free_gpio;
148     }
149
150     /* 2、初始化 cdev  */
151     leddev.cdev.owner = THIS_MODULE;
152     cdev_init(&leddev.cdev, &led_fops);
153
154     /* 3、添加一个 cdev */
155     ret = cdev_add(&leddev.cdev, leddev.devid, LEDDEV_CNT);
156     if(ret < 0)
157         goto del_unregister;
158
```

```
159     /* 4、创建类      */
160     leddev.class = class_create(THIS_MODULE, LEDDEV_NAME);
161     if (IS_ERR(leddev.class)) {
162         goto del_cdev;
163     }
164 
165     /* 5、创建设备 */
166     leddev.device = device_create(leddev.class, NULL, leddev.devid,
                                      NULL, LEDDEV_NAME);
167     if (IS_ERR(leddev.device)) {
168         goto destroy_class;
169     }
170 
171     return 0;
172 destroy_class:
173     class_destroy(leddev.class);
174 del_cdev:
175     cdev_del(&leddev.cdev);
176 del_unregister:
177     unregister_chrdev_region(leddev.devid, LEDDEV_CNT);
178 free_gpio:
179     gpio_free(leddev.gpio_led);
180     return -EIO;
181 }
182 
183 /*
184  * @description   : platform 驱动的 remove 函数
185  * @param - dev   : platform 设备
186  * @return        : 0，成功;其他负值,失败
187  */
188 static int led_remove(struct platform_device *dev)
189 {
190     gpio_set_value(leddev.gpio_led, 1);     /* 卸载驱动的时候关闭 LED */
191     gpio_free(leddev.gpio_led);             /* 注销 GPIO */
192     cdev_del(&leddev.cdev);                 /*  删除 cdev */
193     unregister_chrdev_region(leddev.devid, LEDDEV_CNT);
194     device_destroy(leddev.class, leddev.devid);   /* 注销设备 */
195     class_destroy(leddev.class);                  /* 注销类   */
196     return 0;
197 }
198 
199 /* 匹配列表 */
200 static const struct of_device_id led_of_match[] = {
```

```
201     { .compatible = "alientek,led" },
202     { /* Sentinel */ }
203 };
204
205 MODULE_DEVICE_TABLE(of, led_of_match);
206
207 /* platform 驱动结构体 */
208 static struct platform_driver led_driver = {
209     .driver     = {
210         .name   = "stm32mp1-led",           /* 驱动名字，用于和设备匹配 */
211         .of_match_table = led_of_match,     /* 设备树匹配表         */
212     },
213     .probe      = led_probe,
214     .remove     = led_remove,
215 };
216
217 /*
218  * @description   : 驱动模块加载函数
219  * @param         : 无
220  * @return        : 无
221  */
222 static int __init leddriver_init(void)
223 {
224     return platform_driver_register(&led_driver);
225 }
226
227 /*
228  * @description   : 驱动模块卸载函数
229  * @param         : 无
230  * @return        : 无
231  */
232 static void __exit leddriver_exit(void)
233 {
234     platform_driver_unregister(&led_driver);
235 }
236
237 module_init(leddriver_init);
238 module_exit(leddriver_exit);
239 MODULE_LICENSE("GPL");
240 MODULE_AUTHOR("ALIENTEK");
241 MODULE_INFO(intree, "Y");
```

代码中以前讲过的知识点这里就不再重述了！

第 54~76 行，自定义函数 led_gpio_init，该函数的参数是 struct device_node 类型的指针，

也就是 led 对应的设备节点，当调用函数的时候传递进来。

第 132~181 行，platform 下的 probe 函数：led_probe，当设备树中的设备节点与驱动之间匹配**成功会先去初始化 pinctrl 里面配置的 IO，也就是根据示例代码 35.1.2.1 中的属性进行配置，然后再执行 probe 函数**，第 139 行调用 led_gpio_init 函数时，将 pdev->dev.of_node 作为参数传递到函数中，platform_device 结构体中内置了一个 device 结构体类型的成员变量 dev。在 device 结构体中定义了一个 device_node 类型的指针变量 of_node，使用设备树的情况下，当匹配成功之后，of_node 会指向设备树中定义的节点，所以在这里我们不需要通过调用 of_find_node_by_path("/gpioled")函数得到 led 的节点。我们原来在驱动加载函数里面做的工作现在全部放到 probe 函数里面完成。

第 188~197 行，platform 下的 remove 函数：led_remove，当 platform 驱动模块被卸载时此函数就会执行。在此函数里面释放内存、注销字符设备等，也就是将原来驱动卸载函数里面的工作全部都放到 remove 函数中完成。

第 200~203 行，匹配表，描述了此驱动都和什么样的设备匹配，第 201 行添加了一条值为"alientek,led"的 compatible 属性值，当设备树中某个设备节点的 compatible 属性值也为“alientek,led”的时候就会与此驱动匹配。

第 208~215 行，platform_driver 驱动结构体变量 led_driver，210 行设置这个 platform 驱动的名字为“stm32mp1-led”，因此，当驱动加载成功以后就会在/sys/bus/platform/drivers/目录下存在一个名为“stm32mp1-led”的文件。第 211 行绑定 platform 驱动的 of_match_table 表。

第 222~225 行，platform 驱动模块入口函数，在此函数里面通过 platform_driver_register 向 Linux 内核注册一个 platform 驱动 led_driver。

第 232~235 行，platform 驱动驱动模块出口函数，在此函数里面通过 platform_driver_unregister 从 Linux 内核卸载一个 platform 驱动 led_driver。

35.4.3 编写测试 APP

测试 APP 就直接使用上一章 34.4.2 小节编写的 ledApp.c 即可。

35.5 运行测试

35.5.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为“leddriver.o”，Makefile 内容如下所示：

示例代码 35.5.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := leddriver.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“leddriver.o”。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“leddriver.o”的驱动模块文件。

2、编译测试 APP

测试 APP 直接使用上一章的 ledApp 这个测试软件即可。

35.5.2 运行测试

将上一小节编译出来 leddriver.ko 拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 leddriver.ko 这个驱动模块。

```
depmod          //第一次加载驱动的时候需要运行此命令
modprobe leddriver.ko  //加载驱动模块
```

驱动模块加载完成以后到/sys/bus/platform/drivers/目录下查看驱动是否存在，我们在 leddriver.c 中设置 name 字段为“stm32mp1-led”，因此会在/sys/bus/platform/drivers/目录下存在名为“stm32mp1-led”这个文件，结果如图 35.5.2.1 所示：

```
bcma-host-soc          reg-dummy          stm32_pwr
bdc                    reg-fixed-voltage  stm32_rtc
brcm-kona-reset        restart-gpio       stm32f7-i2c
brcmstb_nand           rfkill_gpio        stm32mp1-led   → 驱动
cdns-spi               rk808-regulator    stm32mp157-pinctrl
```

图 35.5.2.1 stm32mp1-led 驱动

同理，在/sys/bus/platform/devices/目录下也存在 led 的设备文件，也就是设备树中 gpioled 这个节点，如图 35.5.2.2 所示：

```
4c001000.mailbox          cpufreq-dt
50000000.rcc              firmware:scmi-0
50000a28.stm32mp1_led     gpioled   → 设备
50001000.pwr              key
50001014.pwr_mcu          mailbox-0
```

图 35.5.2.2 gpioled 设备

驱动和模块都存在，当驱动和设备匹配成功以后就会输出如图 35.5.2.3 所示一行语句：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe leddriver.ko
led driver and device was matched!   → 驱动和设备匹配成功
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls /sys/bus/platform/drivers
```

图 35.5.2.3 驱动和设备匹配成功

驱动和设备匹配成功以后就可以测试 LED 灯驱动了，输入如下命令打开 LED 灯：

```
./ledApp /dev/dtsplatled 1    //打开 LED 灯
```

在输入如下命令关闭 LED 灯：

```
./ledApp /dev/dtsplatled 0    //关闭 LED 灯
```

观察一下 LED 灯能否打开和关闭，如果可以的话就说明驱动工作正常，如果要卸载驱动的话输入如下命令即可：

```
rmmod leddriver.ko
```

第三十六章 Linux 自带的 LED 灯驱动实验

前面我们都是自己编写 LED 灯驱动，其实像 LED 灯这样非常基础的设备驱动，Linux 内核已经集成了。Linux 内核的 LED 灯驱动采用 platform 框架，因此我们只需要按照要求在设备树文件中添加相应的 LED 节点即可，本章我们就来学习如何使用 Linux 内核自带的 LED 驱动来驱动正点原子的 STM32MP1 开发板上的 LED0 和 LED1 这两个 LED 灯。

36.1 Linux 内核自带 LED 驱动使能

上一章节我们编写基于设备树的 platform LED 灯驱动，其实 Linux 内核已经自带了 LED 灯驱动，要使用 Linux 内核自带的 LED 灯驱动首先得先配置 Linux 内核，使能自带的 LED 灯驱动，输入如下命令打开 Linux 配置菜单：

```
make menuconfig
```

按照如下路径打开 LED 驱动配置项：

```
→ Device Drivers
    → LED Support (NEW_LEDS [=y])
        →LED Support for GPIO connected LEDs
```

按照上述路径，选择“LED Support for GPIO connected LEDs”，将其编译进 Linux 内核，也即是在此选项上按下“Y”键，使此选项前面变为“<*>”，如图 36.1.1 所示：

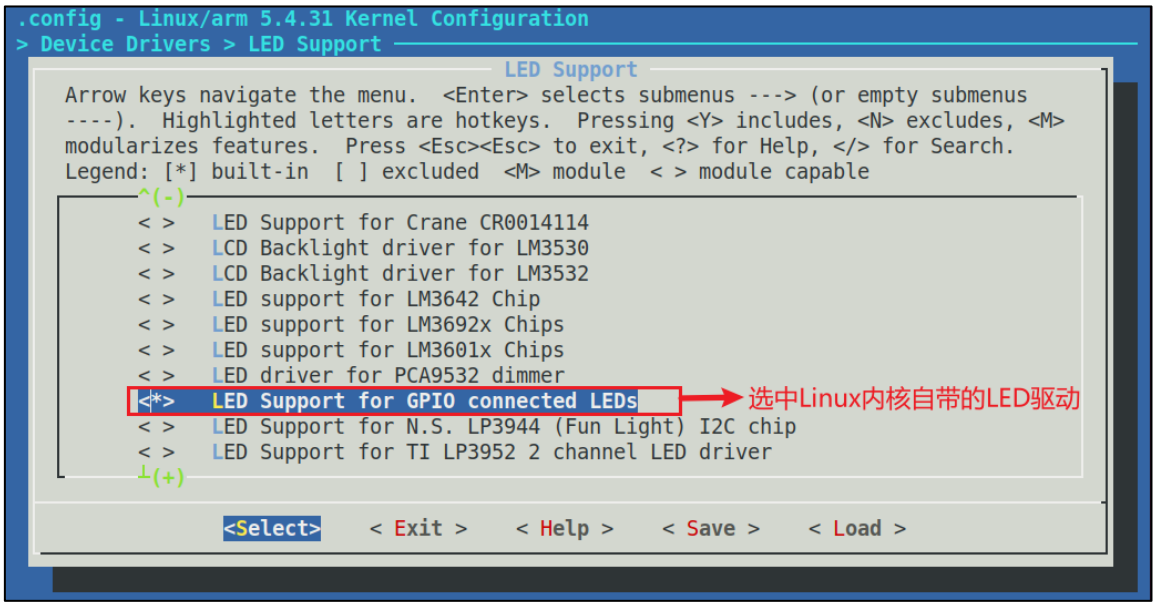


图 36.1.1 使能 LED 灯驱动

在“LED Support for GPIO connected LEDs”上按下“?”健可以打开此选项的帮助信息，如图 36.1.2 所示：

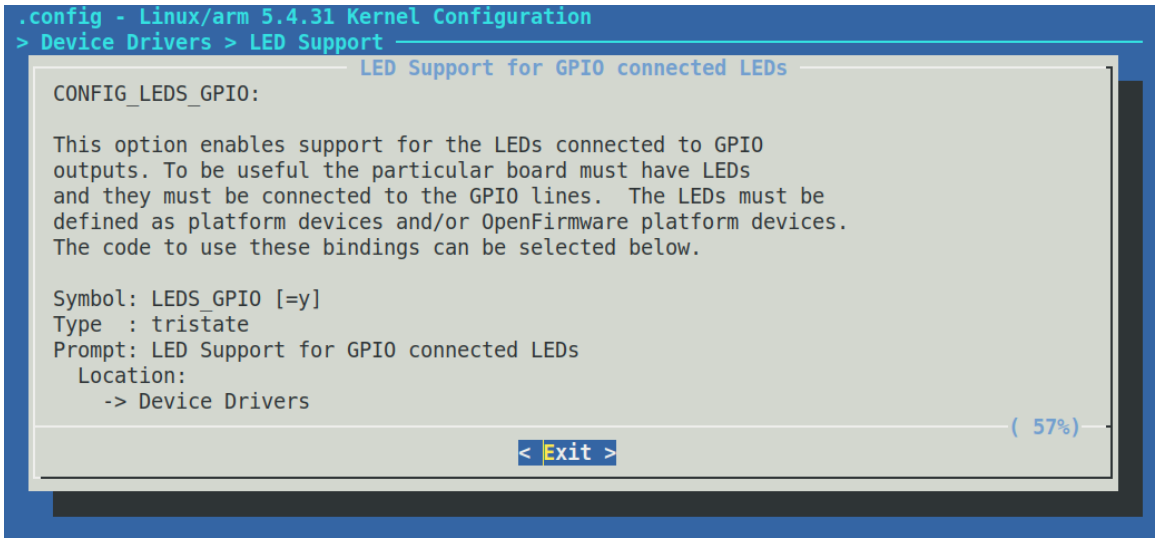


图 36.1.2 内部 LED 灯驱动帮助信息

从图 36.1.2 可以看出，把 Linux 内部自带的 LED 灯驱动编译进内核以后，CONFIG_LEDS_GPIO 就会等于‘y’，Linux 会根据 CONFIG_LEDS_GPIO 的值来选择如何编译 LED 灯驱动，如果为‘y’就将其编译进 Linux 内核。

配置好 Linux 内核以后退出配置界面，打开.config 文件，会找到“CONFIG_LEDS_GPIO=y”这一行，如图 36.1.3 所示：

```
4510 # CONFIG_LEDS_LM3692X is not set
4511 # CONFIG_LEDS_LM3601X is not set
4512 # CONFIG_LEDS_PCA9532 is not set
4513 CONFIG_LEDS_GPIO=y
4514 # CONFIG_LEDS_LP3944 is not set
4515 # CONFIG_LEDS_LP3952 is not set
4516 # CONFIG_LEDS_LP5521 is not set
4517 # CONFIG_LEDS_LP5523 is not set
4518 # CONFIG_LEDS_LP5562 is not set
4519 # CONFIG_LEDS_LP8501 is not set
```

图 36.1.3 .config 文件内容

重新编译 Linux 内核，然后使用新编译出来的 zImage 镜像启动开发板。

36.2 Linux 内核自带 LED 驱动简介

36.2.1 LED 灯驱动框架分析

LED 灯驱动文件为/drivers/leds/leds-gpio.c，大家可以打开/drivers/leds/Makefile 这个文件，找到如下所示内容：

示例代码 36.2.1.1 /drivers/leds/Makefile 文件代码段

```
1   # SPDX-License-Identifier: GPL-2.0
2
3   # LED Core
4   obj-$(CONFIG_NEW_LEDS)              += led-core.o
......
31  obj-$(CONFIG_LEDS_PCA9532)          += leds-pca9532.o
32  obj-$(CONFIG_LEDS_GPIO_REGISTER)    += leds-gpio-register.o
33  obj-$(CONFIG_LEDS_GPIO)             += leds-gpio.o
34  obj-$(CONFIG_LEDS_LP3944)           += leds-lp3944.o
......
```

第 33 行，如果定义了 CONFIG_LEDS_GPIO 的话就会编译 leds-gpio.c 这个文件，在上一小节我们选择将 LED 驱动编译进 Linux 内核，在.config 文件中就会有“CONFIG_LEDS_GPIO=y”这一行，因此 leds-gpio.c 驱动文件就会被编译。

接下来我们看一下 leds-gpio.c 这个驱动文件，找到如下所示内容：

示例代码 36.2.1.2 leds-gpio.c 文件代码段

```
203 static const struct of_device_id of_gpio_leds_match[] = {
204     { .compatible = "gpio-leds", },
205     {},
206 };
207
......
```

```
316 static struct platform_driver gpio_led_driver = {
317     .probe      = gpio_led_probe,
318     .shutdown   = gpio_led_shutdown,
319     .driver     = {
320         .name   = "leds-gpio",
321         .of_match_table = of_gpio_leds_match,
322     },
323 };
324
325 module_platform_driver(gpio_led_driver);
```

第 203~206 行，LED 驱动的匹配表，此表只有一个匹配项，compatible 内容为“gpio-leds”，因此设备树中的 LED 灯设备节点的 compatible 属性值也要为“gpio-leds”，否则设备和驱动匹配不成功，驱动就没法工作。

第 316~323 行，platform_driver 驱动结构体变量，可以看出，Linux 内核自带的 LED 驱动采用了 platform 框架。第 317 行可以看出 probe 函数为 gpio_led_probe，因此当驱动和设备匹配成功以后 gpio_led_probe 函数就会执行。从 320 行可以看出，驱动名字为“leds-gpio”，因此会在/sys/bus/platform/drivers 目录下存在一个名为“leds-gpio”的文件，如图 36.2.1.1 所示：

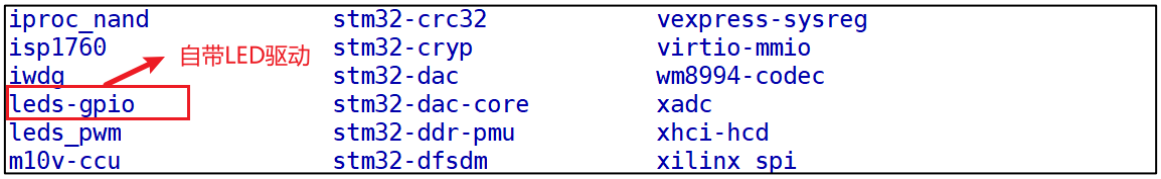


图 36.2.1.1 leds-gpio 驱动文件

第 326 行通过 module_platform_driver 函数向 Linux 内核注册 gpio_led_driver 这个 platform 驱动。

36.2.2 module_platform_driver 函数简析

在上一小节中我们知道 LED 驱动会采用 module_platform_driver 函数向 Linux 内核注册 platform 驱动，其实在 Linux 内核中会大量采用 module_platform_driver 来完成向 Linux 内核注册 platform 驱动的操作。module_platform_driver 定义在 include/linux/platform_device.h 文件中，为一个宏，定义如下：

示例代码 36.2.2.1 module_platform_driver 函数

```
237 #define module_platform_driver(__platform_driver) \
238     module_driver(__platform_driver, platform_driver_register, \
239             platform_driver_unregister)
```

可以看出，module_platform_driver 依赖 module_driver，module_driver 也是一个宏，定义在 include/linux/device.h 文件中，内容如下：

示例代码 36.2.2.2 module_driver 函数

```
1898 #define module_driver(__driver, __register, __unregister, ...) \
1899 static int __init __driver##_init(void) \
1900 { \
1901     return __register(&(__driver) , ##__VA_ARGS__); \
1902 } \
```

```
1903 module_init(__driver##_init); \
1904 static void __exit __driver##_exit(void) \
1905 { \
1906     __unregister(&(__driver) , ##__VA_ARGS__); \
1907 } \
1908 module_exit(__driver##_exit);
```

借助示例代码 36.2.2.1 和示例代码 36.2.2.2，将：

```
module_platform_driver(gpio_led_driver)
```

展开以后就是：

```
static int __init gpio_led_driver_init(void)
{
    return platform_driver_register (&(gpio_led_driver));
}
module_init(gpio_led_driver_init);

static void __exit gpio_led_driver_exit(void)
{
    platform_driver_unregister (&(gpio_led_driver) );
}
module_exit(gpio_led_driver_exit);
```

上面的代码不就是标准的注册和删除 platform 驱动吗？因此 module_platform_driver 函数的功能就是完成 platform 驱动的注册和删除。

36.2.3 gpio_led_probe 函数简析

当驱动和设备匹配以后 gpio_led_probe 函数就会执行，此函数主要是从设备树中获取 LED 灯的 GPIO 信息，缩减后的函数内容如下所示：

示例代码 36.2.3.1 gpio_led_probe 函数

```
256 static int gpio_led_probe(struct platform_device *pdev)
257 {
258     struct gpio_led_platform_data *pdata =
                            dev_get_platdata(&pdev->dev);
259     struct gpio_leds_priv *priv;
260     int i, ret = 0;
261
262     if (pdata && pdata->num_leds) {     /* 非设备树方式      */
        /* 获取 platform_device 信息 */
.....
292     } else {                            /* 采用设备树        */
293         priv = gpio_leds_create(pdev);
294         if (IS_ERR(priv))
295             return PTR_ERR(priv);
296     }
297
```

```
298     platform_set_drvdata(pdev, priv);
299
300     return 0;
301 }
```

第 293~295 行，如果使用设备树的话，使用 gpio_leds_create 函数从设备树中提取设备信息，获取到的 LED 灯 GPIO 信息保存在返回值中，gpio_leds_create 函数内容如下：

示例代码 36.2.3.2 gpio_leds_create 函数

```
134 static struct gpio_leds_priv *gpio_leds_create(struct
                                        platform_device *pdev)
135 {
136     struct device *dev = &pdev->dev;
137     struct fwnode_handle *child;
138     struct gpio_leds_priv *priv;
139     int count, ret;
140
141     count = device_get_child_node_count(dev);
142     if (!count)
143         return ERR_PTR(-ENODEV);
144
145     priv = devm_kzalloc(dev, sizeof_gpio_leds_priv(count),
                            GFP_KERNEL);
146     if (!priv)
147         return ERR_PTR(-ENOMEM);
148
149     device_for_each_child_node(dev, child) {
150         struct gpio_led_data *led_dat = &priv->leds[priv->num_leds];
151         struct gpio_led led = {};
152         const char *state = NULL;
153
154         /*
155          * Acquire gpiod from DT with uninitialized label, which
156          * will be updated after LED class device is registered,
157          * Only then the final LED name is known.
158          */
159         led.gpiod = devm_fwnode_get_gpiod_from_child(dev, NULL,
                                child,
160                                GPIOD_ASIS,
161                                NULL);
162         if (IS_ERR(led.gpiod)) {
163             fwnode_handle_put(child);
164             return ERR_CAST(led.gpiod);
165         }
166
```

```
167         led_dat->gpiod = led.gpiod;
168
169         fwnode_property_read_string(child, "linux,default-trigger",
170                     &led.default_trigger);
171
172         if (!fwnode_property_read_string(child, "default-state",
173                     &state)) {
174             if (!strcmp(state, "keep"))
175                 led.default_state = LEDS_GPIO_DEFSTATE_KEEP;
176             else if (!strcmp(state, "on"))
177                 led.default_state = LEDS_GPIO_DEFSTATE_ON;
178             else
179                 led.default_state = LEDS_GPIO_DEFSTATE_OFF;
180         }
181
182         if (fwnode_property_present(child, "retain-state-suspended"))
183             led.retain_state_suspended = 1;
184         if (fwnode_property_present(child, "retain-state-shutdown"))
185             led.retain_state_shutdown = 1;
186         if (fwnode_property_present(child, "panic-indicator"))
187             led.panic_indicator = 1;
188
189         ret = create_gpio_led(&led, led_dat, dev, child, NULL);
190         if (ret < 0) {
191             fwnode_handle_put(child);
192             return ERR_PTR(ret);
193         }
194         /* Set gpiod label to match the corresponding LED name. */
195         gpiod_set_consumer_name(led_dat->gpiod,
196                     led_dat->cdev.dev->kobj.name);
197         priv->num_leds++;
198     }
199
200     return priv;
201 }
```

第 141 行，调用 device_get_child_node_count 函数统计子节点数量，一般在设备树中创建一个节点表示 LED 灯，然后在这个节点下面为每个 LED 灯创建一个子节点。因此子节点数量也是 LED 灯的数量。

第 149 行，遍历每个子节点，获取每个子节点的信息。

第 159 行，获取 LED 灯所使用的 GPIO 信息。

第 169~170 行，获取“linux,default-trigger”属性值，可以通过此属性设置某个 LED 灯在 Linux 系统中的默认功能，比如作为系统心跳指示灯等等。

第 172~173 行，获取“default-state”属性值，也就是 LED 灯的默认状态属性。

第 189 行，调用 create_gpio_led 函数创建 LED 相关的 io，其实就是设置 LED 所使用的 io 为输出之类的。create_gpio_led 函数主要是初始化 led_dat 这个 gpio_led_data 结构体类型变量，led_dat 保存了 LED 的操作函数等内容。

第 195~196 行，使用 label 属性作为 LED 的名字，led_dat->cdev.dev->kobj.name 指向设备树里的 LED 灯节点下的 label 属性。

关于 gpio_led_probe 函数就分析到这里，gpio_led_probe 函数主要功能就是获取 LED 灯的设备信息，然后根据这些信息来初始化对应的 IO，设置为输出等。

36.3 设备树节点编写

打开文档 Documentation/devicetree/bindings/leds/leds-gpio.txt，此文档详细的讲解了 Linux 自带驱动对应的设备树节点该如何编写，我们在编写设备节点的时候要注意以下几点：

①、创建一个节点表示 LED 灯设备，比如 dtsleds，如果板子上有多个 LED 灯的话每个 LED 灯都作为 dtsleds 的子节点。

②、dtsleds 节点的 compatible 属性值一定要为“gpio-leds”。

③、设置 label 属性，此属性为可选，每个子节点都有一个 label 属性，label 属性一般表示 LED 灯的名字，比如以颜色区分的话就是 red、green 等等。

④、每个子节点必须要设置 gpios 属性值，表示此 LED 所使用的 GPIO 引脚！

⑤、可以设置“linux,default-trigger”属性值，也就是设置 LED 灯的默认功能，查阅 Documentation/devicetree/bindings/leds/common.txt 这个文档来查看可选功能，比如：

backlight：LED 灯作为背光。

default-on：LED 灯打开。

heartbeat：LED 灯作为心跳指示灯，可以作为系统运行提示灯。

disk-activity：LED 灯作为磁盘活动指示灯。

ide-disk：LED 灯作为硬盘活动指示灯。

timer：LED 灯周期性闪烁，由定时器驱动，闪烁频率可以修改。

⑥、可以设置“default-state”属性值，可以设置为 on、off 或 keep，为 on 的时候 LED 灯默认打开，为 off 的话 LED 灯默认关闭，为 keep 的话 LED 灯保持当前模式。

另外还有一些其他的可选属性，比如 led-sources、color、function 等属性，这些属性的用法在 Documentation/devicetree/bindings/leds/common.txt 里面有详细的讲解，大家自行查阅。

本节实验我们把 STM32MP1 开发板上的 2 个 LED 灯都用上，其中 LED0(红色)连接到 PI0 引脚上，LED1(绿色)连接到 PF3。首先是创建这两个 LED 灯对应的 pinctrl 节点，直接在示例代码 35.1.2.1 上修改即可，修改完成以后如下所示：

示例代码 36.3.1 pinctrl 子节点

```
1  led_pins_a: gpioled-0 {
2      pins {
3          pinmux = <STM32_PINMUX('I', 0, GPIO)>,  /* LED0   */
4                   <STM32_PINMUX('F', 3, GPIO)>;  /* LED1   */
5          drive-push-pull;
6          bias-pull-up;
7          output-high;
8          slew-rate = <0>;
9      };
10 };
```

上述代码仅仅只是在示例代码 35.1.2.1 只添加了第 4 行，也就是 LED1 对应的 PF3 引脚，其他的属性配置没变。

最后，根据前面的绑定文档要求添加 LED 设备子节点，打开 stm32mp157d-atk.dts，在“/”根节点下添加如下所示 LED 灯设备子节点：

示例代码 36.3.2 dtsleds 设备节点

```
1  dtsleds {
2      compatible = "gpio-leds";
3      pinctrl-0 = <&led_pins_a>;
4
5       led0 {
6           label = "red";
7           gpios = <&gpioi 0 GPIO_ACTIVE_LOW>;
8           default-state = "off";
9       };
10
11      led1 {
12          label = "green";
13          gpios = <&gpiof 3 GPIO_ACTIVE_LOW>;
14          default-state = "off";
15      };
16 };
```

第 3 行，设置 LED 的 pinctrl 节点为 led_pins_a，也就是示例代码 36.3.1。第 4~8 行是开发板上的 LED0，第 10~13 行是开发板上的 LED1。修改完成以后保存并重新编译设备树，然后用新的设备树启动开发板。

36.4 运行测试

用新的 uImage 和 stm32mp157d-atk.dtb 启动开发板，启动以后查看/sys/bus/platform/devices/dtsleds 这个目录是否存在，如图 36.4.1 所示：

```
4c000000.hwspinlock          buck1
4c001000.mailbox             cpufreq-dt
50000000.rcc                 dtsleds        → dtsleds设备
50000a28.stm32mp1_led        firmware:scmi-0
50001000.pwr                 gpioled
```

图 36.4.1 dtsleds 目录

进入到 dtsleds/leds 目录中，此目录中的内容如图 36.4.2 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/dtsleds/leds$ ls
green  red                   → 两个LED灯
[root@ATK-stm32mp1]:/sys/devices/platform/dtsleds/leds$
```

图 36.4.2 leds 目录内容

从图 36.4.2 可以看出，在 leds 目录下有两个子目录，分别为：green 和 red，其中 green 就是 LED1，red 就是 LED0，这两个子目录的名字就是我们在示例代码 36.3.2 中第 5 行和第 11 行设置的 label 属性值。

我们的设置究竟有没有用，最终是要通过测试才能知道的，首先查看一下系统中有没有“sys/class/leds/red/brightness”和“sys/class/leds/green/brightness”这两个文件，这两个文件分别

对应 LED0 和 LED1，通过操作这两个文件即可实现 LED0 和 LED1 的打开和关闭。

果有的话就输入如下命令打开 LED0(RED)和 LED1(GREEN)这两个 LED 灯：

```
echo 1 > /sys/class/leds/red/brightness        //打开 LED0
echo 1 > /sys/class/leds/green/brightness      //打开 LED1
```

关闭这两个 LED 灯的命令如下：

```
echo 0 > /sys/class/leds/red/brightness        //关闭 LED0
echo 0 > /sys/class/leds/green/brightness      //关闭 LED1
```

如果能正常的打开和关闭两个 LED 灯话就说明我们 Linux 内核自带的 LED 灯驱动工作正常。我们一般会将一个 LED 灯作为系统指示灯，系统运行正常的话这个 LED 指示灯就会一闪一闪的。里我们设置 LED0 作为系统指示灯，在 dtsleds/led0 这个设备节点中加入“linux,default-trigger”属性信息即可，属性值为“heartbeat”，修改完以后的 dtsleds 节点内容如下：

示例代码 36.4.1 dtsleds 设备节点

```
1  dtsleds {
2      compatible = "gpio-leds";
3      pinctrl-0 = <&led_pins_a>;
4
5      led0 {
6          label = "red";
7          gpios = <&gpioi 0 GPIO_ACTIVE_LOW>;
8          linux,default-trigger = "heartbeat";
9          default-state = "on";
10     };
11
12     led1 {
13         label = "green";
14         gpios = <&gpiof 3 GPIO_ACTIVE_LOW>;
15         default-state = "off";
16     };
17 };
```

第 8 行，设置 LED0 为 heartbeat。

第 9 行，默认打开 LED0。

重新编译设备树并且使用新的设备树启动 Linux 系统，启动以后 LED0 就会闪烁，作为系统心跳指示灯，表示系统正在运行。

第三十七章 Linux MISC 驱动实验

misc 的意思是混合、杂项的，因此 MISC 驱动也叫做杂项驱动，也就是当我们板子上的某些外设无法进行分类的时候就可以使用 MISC 驱动。MISC 驱动其实就是最简单的字符设备驱动，通常嵌套在 platform 总线驱动中，实现复杂的驱动，本章我们就来学习一下 MISC 驱动的编写。

37.1 MISC 设备驱动简介

所有的 MISC 设备驱动的主设备号都为 10，不同的设备使用不同的从设备号。随着 Linux 字符设备驱动的不断增加，设备号变得越来越紧张，尤其是主设备号，MISC 设备驱动就用于解决此问题。MISC 设备会自动创建 cdev，不需要像我们以前那样手动创建，因此采用 MISC 设备驱动可以简化字符设备驱动的编写。我们需要向 Linux 注册一个 miscdevice 设备，miscdevice 是一个结构体，定义在文件 include/linux/miscdevice.h 中，内容如下：

示例代码 37.1.1 miscdevice 结构体代码

```
66 struct miscdevice  {
67     int minor;                              /* 子设备号      */
68     const char *name;                       /* 设备名字      */
69     const struct file_operations *fops;     /* 设备操作集    */
70     struct list_head list;
71     struct device *parent;
72     struct device *this_device;
73     const struct attribute_group **groups;
74     const char *nodename;
75     umode_t mode;
76 };
```

定义一个 MISC 设备(miscdevice 类型)以后我们需要设置 minor、name 和 fops 这三个成员变量。minor 表示子设备号，MISC 设备的主设备号为 10，这个是固定的，需要用户指定子设备号，Linux 系统已经预定义了一些 MISC 设备的子设备号，这些预定义的子设备号定义在 include/linux/miscdevice.h 文件中，如下所示：

示例代码 37.1.2 预定义的 MISC 设备子设备号

```
15 #define PSMOUSE_MINOR           1
16 #define MS_BUSMOUSE_MINOR       2   /* unused */
17 #define ATIXL_BUSMOUSE_MINOR    3   /* unused */
18 /*#define AMIGAMOUSE_MINOR      4   FIXME OBSOLETE */
19 #define ATARIMOUSE_MINOR        5   /* unused */
20 #define SUN_MOUSE_MINOR         6   /* unused */
......
61 #define MISC_DYNAMIC_MINOR      255
```

我们在使用的时候可以从这些预定义的子设备号中挑选一个，当然也可以自己定义，只要这个子设备号没有被其他设备使用接口。

name 就是此 MISC 设备名字，当此设备注册成功以后就会在/dev 目录下生成一个名为 name 的设备文件。fops 就是字符设备的操作集合，MISC 设备驱动最终是需要使用用户提供的 fops 操作集合。

当设置好 miscdevice 以后就需要使用 misc_register 函数向系统中注册一个 MISC 设备，此函数原型如下：

```
int misc_register(struct miscdevice * misc)
```

函数参数和返回值含义如下：

misc：要注册的 MISC 设备。

返回值：负数，失败；0，成功。

以前我们需要自己调用一堆的函数去创建设备，比如在以前的字符设备驱动中我们会使用如下几个函数完成设备创建过程：

示例代码 37.1.3 传统的创建设备过程

```
1 alloc_chrdev_region();      /* 申请设备号    */
2 cdev_init();                /* 初始化 cdev   */
3 cdev_add();                 /* 添加 cdev     */
4 class_create();             /* 创建类        */
5 device_create();            /* 创建设备      */
```

现在我们可以直接使用 misc_register 一个函数来完成示例代码 37.1.3 中的这些步骤。当我们卸载设备驱动模块的时候需要调用 misc_deregister 函数来注销掉 MISC 设备，函数原型如下：

```
int misc_deregister(struct miscdevice *misc)
```

函数参数和返回值含义如下：

misc：要注销的 MISC 设备。

返回值：负数，失败；0，成功。

以前注销设备驱动的时候，我们需要调用一堆的函数去删除此前创建的 cdev、设备等等内容，如下所示：

示例代码 37.1.4 传统的删除设备的过程

```
1 cdev_del();                    /*  删除 cdev    */
2 unregister_chrdev_region();    /* 注销设备号    */
3 device_destroy();              /* 删除设备      */
4 class_destroy();               /* 删除类        */
```

现在我们只需要一个 misc_deregister 函数即可完成示例代码 37.1.4 中的这些工作。关于 MISC 设备驱动就讲解到这里，接下来我们就使用 platform 加 MISC 驱动框架来编写 beep 蜂鸣器驱动。

37.2 硬件原理图分析

本章实验我们只使用到正点原子的 STM32MP1 开发板上的 BEEP 蜂鸣器，因此实验硬件原理图参考 26.2 小节即可。

37.3 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→19_miscbeep。

本章实验我们采用 platform 加 misc 的方式编写 beep 驱动，这也是实际的 Linux 驱动中很常用的方法。采用 platform 来实现总线、设备和驱动，misc 主要负责完成字符设备的创建。

37.3.1 修改设备树

本章实验我们需要用到蜂鸣器，因此需要在 stm32mp157d-atk.dts 文件中创建蜂鸣器设备节点。

1、添加 pinctrl 节点

在 STM32MP157 开发板上的蜂鸣器使用了 PC7 这个引脚，打开 stm32mp15-pinctrl.dtsi 文件，在 pinctrl 节点下添加一个‘beep_pins_a’子节点，这个节点就是蜂鸣器的 pinctrl 配置。

示例代码 37.3.1.1 蜂鸣器的 pinctrl 节点

```
1 beep_pins_a: beep_pins {
```

```
2      pins {
3          pinmux = <STM32_PINMUX('C', 7, GPIO)>; /* BEEP */
4          drive-push-pull;
5          bias-pull-up;
6          output-high;
7          slew-rate = <0>;
8      };
9  };
```

2、添加 beep 设备节点

在 26.3.1 小节我们创建了一个名为“beep”的子节点，本章我们就在这个子节点里面稍微修改即可，修改完成以后的 beep 子节点内容如下：

示例代码 37.3.1.2 蜂鸣器节点

```
1  beep {
2      compatible = "alientek,beep";
3      status = "okay";
4      pinctrl-names = "default";
5      pinctrl-0 = <&beep_pins_a>;
6      beep-gpio = <&gpioc 7 GPIO_ACTIVE_HIGH>;
7  };
```

相比 26.3.1 小节里面的 beep 节点，这里仅仅添加了第 4~5 行，这两行指定 BEEP 所使用的引脚 pinctrl 子节点。

37.3.2 beep 驱动程序编写

新建名为“19_miscbeep”的文件夹，然后在 19_miscbeep 文件夹里面创建 vscode 工程，工作区命名为“miscbeep。新建名为 miscbeep.c 的驱动文件，在 miscbeep.c 中输入如下所示内容：

示例代码 37.3.2.1 miscbeep.c 文件代码段

```
1   #include <linux/types.h>
2   #include <linux/kernel.h>
3   #include <linux/delay.h>
4   #include <linux/ide.h>
5   #include <linux/init.h>
6   #include <linux/module.h>
7   #include <linux/errno.h>
8   #include <linux/gpio.h>
9   #include <linux/cdev.h>
10  #include <linux/device.h>
11  #include <linux/of.h>
12  #include <linux/of_address.h>
13  #include <linux/of_gpio.h>
14  #include <linux/platform_device.h>
15  #include <linux/miscdevice.h>
16  #include <asm/mach/map.h>
```

```
17  #include <asm/uaccess.h>
18  #include <asm/io.h>
19
20  #define MISCBEEP_NAME    "miscbeep"     /* 名字     */
21  #define MISCBEEP_MINOR   144            /* 子设备号 */
22  #define BEEPOFF          0              /* 关蜂鸣器 */
23  #define BEEPON           1              /* 开蜂鸣器 */
24
25  /* miscbeep 设备结构体 */
26  struct miscbeep_dev{
27      dev_t devid;              /* 设备号   */
28      struct cdev cdev;         /* cdev     */
29      struct class *class;      /* 类       */
30      struct device *device;    /* 设备     */
31      int beep_gpio;            /* beep 所使用的 GPIO 编号       */
32  };
33
34  struct miscbeep_dev miscbeep;       /* beep 设备 */
35
36  /*
37   * @description   : beep 相关初始化操作
38   * @param - pdev  : struct platform_device 指针，也就是 platform 设备指针
39   * @return        : 成功返回 0，失败返回负数
40   */
41  static int beep_gpio_init(struct device_node *nd)
42  {
43      int ret;
44
45      /* 从设备树中获取 GPIO */
46      miscbeep.beep_gpio = of_get_named_gpio(nd, "beep-gpio", 0);
47      if(!gpio_is_valid(miscbeep.beep_gpio)) {
48          printk("miscbeep: Failed to get beep-gpio\n");
49          return -EINVAL;
50      }
51
52      /* 申请使用 GPIO */
53      ret = gpio_request(miscbeep.beep_gpio, "beep");
54      if(ret) {
55          printk("beep: Failed to request beep-gpio\n");
56          return ret;
57      }
58
59      /* 将 GPIO 设置为输出模式并设置 GPIO 初始化电平状态 */
```

```
60      gpio_direction_output(miscbeep.beep_gpio, 1);
61
62      return 0;
63  }
64
65  /*
66   * @description   : 打开设备
67   * @param - inode : 传递给驱动的 inode
68   * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
69   *                  一般在 open 的时候将 private_data 指向设备结构体。
70   * @return        : 0 成功;其他 失败
71   */
72  static int miscbeep_open(struct inode *inode, struct file *filp)
73  {
74      return 0;
75  }
76
77  /*
78   * @description   : 向设备写数据
79   * @param - filp  : 设备文件，表示打开的文件描述符
80   * @param - buf   : 要写给设备写入的数据
81   * @param - cnt   : 要写入的数据长度
82   * @param - offt  : 相对于文件首地址的偏移
83   * @return        : 写入的字节数，如果为负值，表示写入失败
84   */
85  static ssize_t miscbeep_write(struct file *filp,
                    const char __user *buf, size_t cnt, loff_t *offt)
86  {
87      int retvalue;
88      unsigned char databuf[1];
89      unsigned char beepstat;
90
91      retvalue = copy_from_user(databuf, buf, cnt);
92      if(retvalue < 0) {
93          printk("kernel write failed!\r\n");
94          return -EFAULT;
95      }
96
97      beepstat = databuf[0];                          /* 获取状态值 */
98      if(beepstat == BEEPON) {
99          gpio_set_value(miscbeep.beep_gpio, 0);      /* 打开蜂鸣器 */
100     } else if(beepstat == BEEPOFF) {
101         gpio_set_value(miscbeep.beep_gpio, 1);      /* 关闭蜂鸣器 */
```

```
102     }
103     return 0;
104 }
105 
106 /* 设备操作函数 */
107 static struct file_operations miscbeep_fops = {
108     .owner = THIS_MODULE,
109     .open = miscbeep_open,
110     .write = miscbeep_write,
111 };
112 
113 /* MISC 设备结构体 */
114 static struct miscdevice beep_miscdev = {
115     .minor = MISCBEEP_MINOR,
116     .name = MISCBEEP_NAME,
117     .fops = &miscbeep_fops,
118 };
119 
120  /*
121   * @description  : flatform 驱动的 probe 函数，当驱动与
122   *                 设备匹配以后此函数就会执行
123   * @param - dev  : platform 设备
124   * @return       : 0，成功;其他负值,失败
125   */
126 static int miscbeep_probe(struct platform_device *pdev)
127 {
128     int ret = 0;
129 
130     printk("beep driver and device was matched!\r\n");
131 
132     /* 初始化 BEEP */
133     ret = beep_gpio_init(pdev->dev.of_node);
134     if(ret < 0)
135         return ret;
136 
137     /* 一般情况下会注册对应的字符设备，但是这里我们使用 MISC 设备
138      * 所以我们不需要自己注册字符设备驱动，只需要注册 misc 设备驱动即可
139      */
140     ret = misc_register(&beep_miscdev);
141     if(ret < 0){
142         printk("misc device register failed!\r\n");
143         goto free_gpio;
144     }
```

```
145
146     return 0;
147
148 free_gpio:
149     gpio_free(miscbeep.beep_gpio);
150     return -EINVAL;
151 }
152
153 /*
154  * @description   : platform 驱动的 remove 函数
155  * @param - dev   : platform 设备
156  * @return        : 0，成功;其他负值,失败
157  */
158 static int miscbeep_remove(struct platform_device *dev)
159 {
160     /* 注销设备的时候关闭 LED 灯 */
161     gpio_set_value(miscbeep.beep_gpio, 1);
162
163     /* 释放 BEEP */
164     gpio_free(miscbeep.beep_gpio);
165
166     /* 注销 misc 设备 */
167     misc_deregister(&beep_miscdev);
168     return 0;
169 }
170
171 /* 匹配列表 */
172 static const struct of_device_id beep_of_match[] = {
173     { .compatible = "alientek,beep" },
174     { /* Sentinel */ }
175 };
176
177 /* platform 驱动结构体 */
178 static struct platform_driver beep_driver = {
179     .driver     = {
180         .name   = "stm32mp1-beep",          /* 驱动名字，用于和设备匹配 */
181         .of_match_table = beep_of_match, /* 设备树匹配表             */
182     },
183     .probe      = miscbeep_probe,
184     .remove     = miscbeep_remove,
185 };
186
187 /*
```

```
188  * @description   : 驱动出口函数
189  * @param         : 无
190  * @return        : 无
191  */
192 static int __init miscbeep_init(void)
193 {
194     return platform_driver_register(&beep_driver);
195 }
196
197 /*
198  * @description   : 驱动出口函数
199  * @param         : 无
200  * @return        : 无
201  */
202 static void __exit miscbeep_exit(void)
203 {
204     platform_driver_unregister(&beep_driver);
205 }
206
207 module_init(miscbeep_init);
208 module_exit(miscbeep_exit);
209 MODULE_LICENSE("GPL");
210 MODULE_AUTHOR("ALIENTEK");
211 MODULE_INFO(intree, "Y");
```

第 72~111 行，标准的字符设备驱动。

第 114~118 行，MISC 设备 beep_miscdev，第 115 行设置子设备号为 144，第 116 行设置设备名字为“miscbeep”，这样当系统启动以后就会在/dev/目录下存在一个名为“miscbeep”的设备文件。第 117 行，设置 MISC 设备的操作函数集合，为 file_operations 类型。

第 126~151 行，platform 框架的 probe 函数，当驱动与设备匹配以后此函数就会执行，首先在此函数中初始化 BEEP 所使用的 IO。最后在 140 行通过 misc_register 函数向 Linux 内核注册 MISC 设备，也就是前面定义的 beep_miscdev。

第 158~168 行，platform 框架的 remove 函数，在此函数中调用 misc_deregister 函数来注销 MISC 设备。

第 192~205，标准的 platform 驱动。

37.3.3 编写测试 APP

新建 miscbeepApp.c 文件，然后在里面输入如下所示内容：

示例代码 37.3.2.2 miscbeepApp.c 文件代码段

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
```

```
6  #include <stdlib.h>
7  #include <string.h>
8
9  #define BEEPOFF    0
10 #define BEEPON     1
11
12 /*
13  * @description    : main 主程序
14  * @param - argc   : argv 数组元素个数
15  * @param - argv   : 具体参数
16  * @return         : 0 成功;其他 失败
17  */
18 int main(int argc, char *argv[])
19 {
20     int fd, retvalue;
21     char *filename;
22     unsigned char databuf[1];
23
24     if(argc != 3){
25         printf("Error Usage!\r\n");
26         return -1;
27     }
28
29     filename = argv[1];
30     fd = open(filename, O_RDWR);     /* 打开 beep 驱动 */
31     if(fd < 0){
32         printf("file %s open failed!\r\n", argv[1]);
33         return -1;
34     }
35
36     databuf[0] = atoi(argv[2]);      /* 要执行的操作：打开或关闭 */
37     retvalue = write(fd, databuf, sizeof(databuf));
38     if(retvalue < 0){
39         printf("BEEP Control Failed!\r\n");
40         close(fd);
41         return -1;
42     }
43
44     retvalue = close(fd);            /* 关闭文件 */
45     if(retvalue < 0){
46         printf("file %s close failed!\r\n", argv[1]);
47         return -1;
48     }
```

```
49      return 0;
50  }
```

miscbeepApp.c 文件内容和其他例程的测试 APP 基本一致，很简单，这里就不讲解了。

37.4 运行测试

37.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为“miscbeep.o”，Makefile 内容如下所示：

示例代码 37.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := miscbeep.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“miscbeep.o”。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“miscbeep.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 miscbeepApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc miscbeepApp.c -o miscbeepApp
```

编译成功以后就会生成 miscbeepApp 这个应用程序。

37.4.2 运行测试

将上一小节编译出来 miscbeep.ko 和 miscbeepApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 miscbeep.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe miscbeep.ko    //加载设备模块
```

当驱动模块加载成功以后我们可以在/sys/class/misc 这个目录下看到一个名为“miscbeep”的子目录，如图 37.4.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls /sys/class/misc/
autofs           hw_random        rfkill
cpu_dma_latency  loop-control     ubi_ctrl
fuse             miscbeep         watchdog
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 37.4.2.1 misbeep 子目录

所有的 misc 设备都属于同一个类，/sys/class/misc 目录下就是 misc 这个类的所有设备，每个设备对应一个子目录。

驱动与设备匹配成功以后就会生成/dev/miscbeep 这个设备驱动文件，输入如下命令查看这

个文件的主次设备号：

```
ls /dev/miscbeep -l
```

结果如图 37.4.2.2 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls /dev/miscbeep -l
crw-rw----    1 root     root       10, 144 Jan 19 07:19 /dev/miscbeep
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

次设备号

图 37.4.2.2 /dev/miscbeep 设备文件

从图 37.4.2.2 可以看出，/dev/miscbeep 这个设备的主设备号为 10，次设备号为 144，和我们驱动程序里面设置的一致。

输入如下命令打开 BEEP：

```
./miscbeepApp /dev/miscbeep 1    //打开 BEEP
```

在输入如下命令关闭 LED 灯：

```
./miscbeepApp /dev/miscbeep 0    //关闭 BEEP
```

观察一下 BEEP 能否打开和关闭，如果可以的话就说明驱动工作正常，如果要卸载驱动的话输入如下命令即可：

```
rmmod miscbeep.ko
```

第三十八章 Linux INPUT 子系统实验

按键、鼠标、键盘、触摸屏等都属于输入(input)设备，Linux 内核为此专门做了一个叫做 input 子系统的框架来处理输入事件。输入设备本质上还是字符设备，只是在此基础上套上了 input 框架，用户只需要负责上报输入事件，比如按键值、坐标等信息，input 核心层负责处理这些事件。本章我们就来学习一下 Linux 内核中的 input 子系统。

38.1 input 子系统

38.1.1 input 子系统简介

input 就是输入的意思，因此 input 子系统就是管理输入的子系统，和 pinctrl、gpio 子系统一样，都是 Linux 内核针对某一类设备而创建的框架。比如按键输入、键盘、鼠标、触摸屏等等这些都属于输入设备，不同的输入设备所代表的含义不同，按键和键盘就是代表按键信息，鼠标和触摸屏代表坐标信息，因此在应用层的处理就不同，对于驱动编写者而言不需要去关心应用层的事情，我们只需要按照要求上报这些输入事件即可。为此 input 子系统分为 input 驱动层、input 核心层、input 事件处理层，最终给用户空间提供可访问的设备节点，input 子系统框架如图 38.1.1.1 所示：

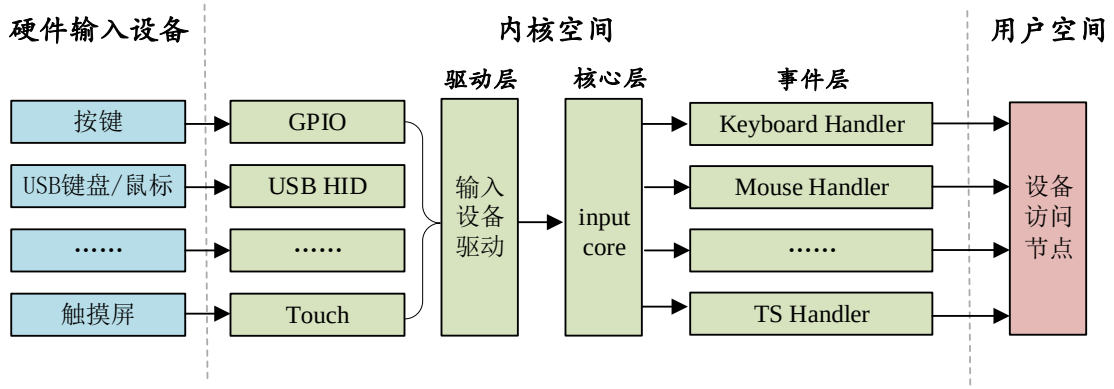


图 38.1.1.1 input 子系统结构图

图 38.1.1.1 中左边就是最底层的具体设备，比如按键、USB 键盘/鼠标等，中间部分属于 Linux 内核空间，分为驱动层、核心层和事件层，最右边的就是用户空间，所有的输入设备以文件的形式供用户应用程序使用。可以看出 input 子系统用到了我们前面讲解的驱动分层模型，我们编写驱动程序的时候只需要关注中间的驱动层、核心层和事件层，这三个层的分工如下：

驱动层：输入设备的具体驱动程序，比如按键驱动程序，向内核层报告输入内容。

核心层：承上启下，为驱动层提供输入设备注册和操作接口。通知事件层对输入事件进行处理。

事件层：主要和用户空间进行交互。

38.1.2 input 驱动编写流程

input 核心层会向 Linux 内核注册一个字符设备，大家找到 drivers/input/input.c 这个文件，input.c 就是 input 输入子系统的核心层，此文件里面有如下所示代码：

示例代码 38.1.2.1 input 核心层创建字符设备过程

```
1797    struct class input_class = {
1798        .name       = "input",
1799        .devnode    = input_devnode,
1800    };
......
2494    static int __init input_init(void)
2495    {
2496        int err;
```

```
2497
2498        err = class_register(&input_class);
2499        if (err) {
2500            pr_err("unable to register input_dev class\n");
2501            return err;
2502        }
2503
2504        err = input_proc_init();
2505        if (err)
2506            goto fail1;
2507
2508        err = register_chrdev_region(MKDEV(INPUT_MAJOR, 0),
2509                     INPUT_MAX_CHAR_DEVICES, "input");
2510        if (err) {
2511            pr_err("unable to register char major %d", INPUT_MAJOR);
2512            goto fail2;
2513        }
2514
2515        return 0;
2516
2517     fail2: input_proc_exit();
2518     fail1: class_unregister(&input_class);
2519        return err;
2520    }
```

第 2498 行，注册一个 input 类，这样系统启动以后就会在/sys/class 目录下有一个 input 子目录，如图 38.1.2.1 所示：

```
backlight     i2c-adapter   pps           sound         vc
bdi           i2c-dev       ptp           spi_master    vtconsole
block         input         pwm           spidev        wakeup
bsg           leds          regulator     tee           watchdog
devcoredump   mdio_bus      remoteproc    thermal
```

图 38.1.2.1 input 类

第 2508~2509 行，注册一个字符设备，主设备号为 INPUT_MAJOR，INPUT_MAJOR 定义在 include/uapi/linux/major.h 文件中，定义如下：

```
#define INPUT_MAJOR          13
```

因此，input 子系统的所有设备主设备号都为 13，我们在使用 input 子系统处理输入设备的时候就不需要去注册字符设备了，我们只需要向系统注册一个 input_device 即可。

1、注册 input_dev

在使用 input 子系统的时候我们只需要注册一个 input 设备即可，input_dev 结构体表示 input 设备，此结构体定义在 include/linux/input.h 文件中，定义如下(有省略)：

示例代码 38.1.2.2 input_dev 结构体

```
131 struct input_dev {
132     const char *name;
133     const char *phys;
```

```
134     const char *uniq;
135     struct input_id id;
136
137     unsigned long propbit[BITS_TO_LONGS(INPUT_PROP_CNT)];
138
139     unsigned long evbit[BITS_TO_LONGS(EV_CNT)];     /* 事件类型的位图 */
140     unsigned long keybit[BITS_TO_LONGS(KEY_CNT)];   /* 按键值的位图 */
141     unsigned long relbit[BITS_TO_LONGS(REL_CNT)];   /* 相对坐标的位图*/
142     unsigned long absbit[BITS_TO_LONGS(ABS_CNT)];   /* 绝对坐标的位图 */
143     unsigned long mscbit[BITS_TO_LONGS(MSC_CNT)];   /* 杂项事件的位图 */
144     unsigned long ledbit[BITS_TO_LONGS(LED_CNT)];   /*LED 相关的位图 */
145     unsigned long sndbit[BITS_TO_LONGS(SND_CNT)];   /* sound 有关的位*/
146     unsigned long ffbit[BITS_TO_LONGS(FF_CNT)];     /* 压力反馈的位图 */
147     unsigned long swbit[BITS_TO_LONGS(SW_CNT)];     /*开关状态的位图 */
148
......
201     bool devres_managed;
202
203     ktime_t timestamp[INPUT_CLK_MAX];
204 };
```

第 139 行，evbit 表示输入事件类型，可选的事件类型定义在 include/uapi/linux/input.h 文件中，事件类型如下：

示例代码 38.1.2.3 事件类型

```
38 #define EV_SYN              0x00    /* 同步事件          */
39 #define EV_KEY              0x01    /* 按键事件          */
40 #define EV_REL              0x02    /* 相对坐标事件      */
41 #define EV_ABS              0x03    /* 绝对坐标事件      */
42 #define EV_MSC              0x04    /* 杂项(其他)事件    */
43 #define EV_SW               0x05    /* 开关事件          */
44 #define EV_LED              0x11    /* LED               */
45 #define EV_SND              0x12    /* sound(声音)       */
46 #define EV_REP              0x14    /* 重复事件          */
47 #define EV_FF               0x15    /* 压力事件          */
48 #define EV_PWR              0x16    /* 电源事件          */
49 #define EV_FF_STATUS        0x17    /* 压力状态事件      */
```

比如本章我们要使用到按键，那么就需要注册 EV_KEY 事件，如果要使用连按功能的话还需要注册 EV_REP 事件。

继续回到示例代码 38.1.2.2 中，第 139 行~147 行的 evbit、keybit、relbit 等等都是存放不同事件对应的值。比如我们本章要使用按键事件，因此要用到 keybit，keybit 就是按键事件使用的位图，Linux 内核定义了很多按键值，这些按键值定义在 include/uapi/linux/input-event-codes.h 文件中，按键值如下：

示例代码 38.1.2.4 按键值

```
75  #define KEY_RESERVED    0
```

```
76  #define KEY_ESC          1
77  #define KEY_1            2
78  #define KEY_2            3
79  #define KEY_3            4
80  #define KEY_4            5
81  #define KEY_5            6
82  #define KEY_6            7
83  #define KEY_7            8
84  #define KEY_8            9
85  #define KEY_9            10
86  #define KEY_0            11
.....
692 #define BTN_TRIGGER_HAPPY39     0x2e6
693 #define BTN_TRIGGER_HAPPY40     0x2e7
```

我们可以将开发板上的按键值设置为示例代码 38.1.2.4 中的任意一个，比如我们本章实验会将 STM32MP1 开发板上的 KEY0 按键值设置为 KEY_0。

在编写 input 设备驱动的时候我们需要先申请一个 input_dev 结构体变量，使用 input_allocate_device 函数来申请一个 input_dev，此函数原型如下所示：

```
struct input_dev *input_allocate_device(void)
```

函数参数和返回值含义如下：

参数：无。

返回值：申请到的 input_dev。

如果要注销的 input 设备的话需要使用 input_free_device 函数来释放掉前面申请到的 input_dev，input_free_device 函数原型如下：

```
void input_free_device(struct input_dev *dev)
```

函数参数和返回值含义如下：

dev：需要释放的 input_dev。

返回值：无。

申请好一个 input_dev 以后就需要初始化这个 input_dev，需要初始化的内容主要为事件类型(evbit)和事件值(keybit)这两种。input_dev 初始化完成以后就需要向 Linux 内核注册 input_dev 了，需要用到 input_register_device 函数，此函数原型如下：

```
int input_register_device(struct input_dev *dev)
```

函数参数和返回值含义如下：

dev：要注册的 input_dev 。

返回值：0，input_dev 注册成功；负值，input_dev 注册失败。

同样的，注销 input 驱动的时候也需要使用 input_unregister_device 函数来注销掉前面注册的 input_dev，input_unregister_device 函数原型如下：

```
void input_unregister_device(struct input_dev *dev)
```

函数参数和返回值含义如下：

dev：要注销的 input_dev 。

返回值：无。

综上所述，input_dev 注册过程如下：

①、使用 input_allocate_device 函数申请一个 input_dev。

②、初始化 input_dev 的事件类型以及事件值。

③、使用 input_register_device 函数向 Linux 系统注册前面初始化好的 input_dev。

④、卸载 input 驱动的时候需要先使用 input_unregister_device 函数注销掉注册的 input_dev，然后使用 input_free_device 函数释放掉前面申请的 input_dev。input_dev 注册过程示例代码如下所示：

示例代码 38.1.2.5 input_dev 注册流程

```
1  struct input_dev *inputdev;      /* input 结构体变量 */
2
3  /* 驱动入口函数 */
4  static int __init xxx_init(void)
5  {
6      ......
7      inputdev = input_allocate_device();    /* 申请 input_dev      */
8      inputdev->name = "test_inputdev";      /* 设置 input_dev 名字  */
9
10     /*********第一种设置事件和事件值的方法***********/
11     __set_bit(EV_KEY, inputdev->evbit);    /* 设置产生按键事件      */
12     __set_bit(EV_REP, inputdev->evbit);    /* 重复事件             */
13     __set_bit(KEY_0, inputdev->keybit);    /*设置产生哪些按键值     */
14     /**********************************************/
15
16     /*********第二种设置事件和事件值的方法***********/
17     keyinputdev.inputdev->evbit[0] = BIT_MASK(EV_KEY) |
                                        BIT_MASK(EV_REP);
18     keyinputdev.inputdev->keybit[BIT_WORD(KEY_0)] |=
                                        BIT_MASK(KEY_0);
19     /**********************************************/
20
21     /*********第三种设置事件和事件值的方法***********/
22     keyinputdev.inputdev->evbit[0] = BIT_MASK(EV_KEY) |
                                      BIT_MASK(EV_REP);
23     input_set_capability(keyinputdev.inputdev, EV_KEY, KEY_0);
24     /**********************************************/
25
26     /* 注册 input_dev */
27     input_register_device(inputdev);
28     ......
29     return 0;
30 }
31
32 /* 驱动出口函数 */
33 static void __exit xxx_exit(void)
34 {
```

```
35      input_unregister_device(inputdev);      /* 注销 input_dev  */
36      input_free_device(inputdev);            /* 删除 input_dev  */
37  }
```

第 1 行，定义一个 input_dev 结构体指针变量。

第 4~30 行，驱动入口函数，在此函数中完成 input_dev 的申请、设置、注册等工作。第 7 行调用 input_allocate_device 函数申请一个 input_dev。第 10~23 行都是设置 input 设备事件和按键值，这里用了三种方法来设置事件和按键值。第 27 行调用 input_register_device 函数向 Linux 内核注册 inputdev。

第 33~37 行，驱动出口函数，第 35 行调用 input_unregister_device 函数注销前面注册的 input_dev，第 36 行调用 input_free_device 函数删除前面申请的 input_dev。

2、上报输入事件

当我们向 Linux 内核注册好 input_dev 以后还不能高枕无忧的使用 input 设备，input 设备都是具有输入功能的，但是具体是什么样的输入值 Linux 内核是不知道的，我们需要获取到具体的输入值，或者说是输入事件，然后将输入事件上报给 Linux 内核。比如按键，我们需要在按键中断处理函数，或者消抖定时器中断函数中将按键值上报给 Linux 内核，这样 Linux 内核才能获取到正确的输入值。不同的事件，其上报事件的 API 函数不同，我们依次来看一下一些常用的事件上报 API 函数。

首先是 input_event 函数，此函数用于上报指定的事件以及对应的值，函数原型如下：

```
void input_event(struct input_dev    *dev,
                 unsigned int        type,
                 unsigned int        code,
                 int                 value)
```

函数参数和返回值含义如下：

dev：需要上报的 input_dev。

type：上报的事件类型，比如 EV_KEY。

code：事件码，也就是我们注册的按键值，比如 KEY_0、KEY_1 等等。

value：事件值，比如 1 表示按键按下，0 表示按键松开。

返回值：无。

input_event 函数可以上报所有的事件类型和事件值，Linux 内核也提供了其他的针对具体事件的上报函数，这些函数其实都用到了 input_event 函数。比如上报按键所使用的 input_report_key 函数，此函数内容如下：

例代码 38.1.2.6 input_report_key 函数

```
static inline void input_report_key(struct input_dev *dev,
                                    unsigned int code, int value)
{
    input_event(dev, EV_KEY, code, !!value);
}
```

从示例代码 38.1.2.6 可以看出，input_report_key 函数的本质就是 input_event 函数，如果要上报按键事件的话还是建议大家使用 input_report_key 函数。

同样的还有一些其他的事件上报函数，这些函数如下所示：

```
void input_report_rel(struct input_dev *dev, unsigned int code, int value)
void input_report_abs(struct input_dev *dev, unsigned int code, int value)
void input_report_ff_status(struct input_dev *dev, unsigned int code, int value)
```

```
void input_report_switch(struct input_dev *dev, unsigned int code, int value)
void input_mt_sync(struct input_dev *dev)
```

当我们上报事件以后还需要使用 input_sync 函数来告诉 Linux 内核 input 子系统上报结束，input_sync 函数本质是上报一个同步事件，此函数原型如下所示：

```
void input_sync(struct input_dev *dev)
```

函数参数和返回值含义如下：

dev：需要上报同步事件的 input_dev。

返回值：无。

综上所述，按键的上报事件的参考代码如下所示：

示例代码 38.1.2.7 事件上报参考代码

```
1  /* 用于按键消抖的定时器服务函数 */
2  void timer_function(unsigned long arg)
3  {
4      unsigned char value;
5
6      value = gpio_get_value(keydesc->gpio);     /* 读取 IO 值     */
7      if(value == 0){                            /* 按下按键       */
8          /* 上报按键值 */
9          input_report_key(inputdev, KEY_0, 1); /* 最后一个参数 1，按下 */
10         input_sync(inputdev);                  /*  同步事件      */
11     } else {                                   /* 按键松开       */
12         input_report_key(inputdev, KEY_0, 0); /* 最后一个参数 0，松开 */
13         input_sync(inputdev);                  /*  同步事件      */
14     }
15 }
```

第 6 行，获取按键值，判断按键是否按下。

第 9~10 行，如果按键值为 0 那么表示按键被按下了，如果按键按下的话就要使用 input_report_key 函数向 Linux 系统上报按键值，比如向 Linux 系统通知 KEY_0 这个按键按下了。

第 12~13 行，如果按键值为 1 的话就表示按键没有按下，是松开的。向 Linux 系统通知 KEY_0 这个按键没有按下或松开了。

38.1.3 input_event 结构体

Linux 内核使用 input_event 这个结构体来表示所有的输入事件，input_envent 结构体定义在 include/uapi/linux/input.h 文件中，结构体内容如下：

示例代码 38.1.3.1 input_event 结构体

```
28 struct input_event {
29 #if (__BITS_PER_LONG != 32 || !defined(__USE_TIME_BITS64))
       && !defined(__KERNEL__)
30     struct timeval time;
31 #define input_event_sec time.tv_sec
32 #define input_event_usec time.tv_usec
33 #else
```

```
34      __kernel_ulong_t __sec;
35  #if defined(__sparc__) && defined(__arch64__)
36      unsigned int __usec;
37      unsigned int __pad;
38  #else
39      __kernel_ulong_t __usec;
40  #endif
41  #define input_event_sec  __sec
42  #define input_event_usec __usec
43  #endif
44      __u16 type;
45      __u16 code;
46      __s32 value;
47  };
```

我们依次来看一下 input_event 结构体中的各个成员变量：

time：时间，也就是此事件发生的时间，为 timeval 结构体类型，timeval 结构体定义如下：

示例代码 38.1.3.2 timeval 结构体

```
1 typedef long                __kernel_long_t;
2 typedef __kernel_long_t     __kernel_time_t;
3 typedef __kernel_long_t     __kernel_suseconds_t;
4
5 struct timeval {
6       __kernel_time_t           tv_sec;     /* 秒     */
7       __kernel_suseconds_t      tv_usec;    /* 微秒      */
8 };
```

从示例代码 38.1.3.2 可以看出，tv_sec 和 tv_usec 这两个成员变量都为 long 类型，也就是 32 位，这个一定要记住，后面我们分析 event 事件上报数据的时候要用到。

type：事件类型，比如 EV_KEY，表示此次事件为按键事件，此成员变量为 16 位。

code：事件码，比如在 EV_KEY 事件中 code 就表示具体的按键码，如：KEY_0、KEY_1 等等这些按键。此成员变量为 16 位。

value：值，比如 EV_KEY 事件中 value 就是按键值，表示按键有没有被按下，如果为 1 的话说明按键按下，如果为 0 的话说明按键没有被按下或者按键松开了。

input_envent 这个结构体非常重要，因为所有的输入设备最终都是按照 input_event 结构体呈现给用户的，用户应用程序可以通过 input_event 来获取到具体的输入事件或相关的值，比如按键值等。关于 input 子系统就讲解到这里，接下来我们就以开发板上的 KEY0 按键为例，讲解一下如何编写 input 驱动。

38.2 硬件原理图分析

本章实验硬件原理图参考 29.2 小节即可。

38.3 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→20_input。

38.3.1 修改设备树文件

1、创建按键所使用的引脚 pinctrl 子节点

首先在 stm32mp15-pinctrl.dtsi 文件中创建按键对应的 pinctrl 子节点，内容如下：

示例代码 38.3.1.1 key_pins_a 节点

```
1  key_pins_a: key_pins-0 {
2      pins1 {
3        pinmux = <STM32_PINMUX('G', 3, GPIO)>,   /* KEY0 */
4                 <STM32_PINMUX('H', 7, GPIO)>;   /* KEY1 */
5        bias-pull-up;
6        slew-rate = <0>;
7      };
8
9      pins2 {
10       pinmux = <STM32_PINMUX('A', 0, GPIO)>;    /* WK_UP */
11       bias-pull-down;
12       slew-rate = <0>;
13     };
14 };
```

这里我们将 STM32MP1 开发板上的三个按键：KEY0、KEY1 和 WK_UP 都设置了，虽然本小节我们只使用到 KEY0。由于 KEY0 和 KEY1 这两个按键是低电平有效(按下以后为低电平)，WK_UP 是高电平有效(按下以后为高电平)。所以这里分开初始化，其中第 2~7 行初始化 KEY0 和 KEY1 使用的 PG3 和 PH7 这两个引脚，第 9~13 行初始化 WK_UP 使用的 PA0 引脚。

2、创建按键所使用的引脚 pinctrl 子节点

接下就是创建按键设备节点，我们直接在 29.3.1 小节创建的 key 节点上修改即可，修改完成以后如下所示：

示例代码 38.3.1.2 key 节点

```
1 key {
2      compatible = "alientek,key";
3      status = "okay";
4      pinctrl-names = "default";
5      pinctrl-0 = <&key_pins_a>;
6      key-gpio = <&gpiog 3 GPIO_ACTIVE_LOW>;
7      interrupt-parent = <&gpiog>;
8      interrupts = <3 IRQ_TYPE_EDGE_BOTH>;
9 };
```

38.3.2 按键 input 驱动程序编写

新建名为“20_input”的文件夹，然后在 20_input 文件夹里面创建 vscode 工程，工作区命名为“keyinput”。工程创建好以后新建 keyinput.c 文件，在 keyinput.c 里面输入如下内容：

示例代码 38.3.2.1 keyinput.c 文件代码段

```
1  #include <linux/module.h>
```

```
2  #include <linux/errno.h>
3  #include <linux/of.h>
4  #include <linux/platform_device.h>
5  #include <linux/of_gpio.h>
6  #include <linux/input.h>
7  #include <linux/timer.h>
8  #include <linux/of_irq.h>
9  #include <linux/interrupt.h>
10
11 #define KEYINPUT_NAME       "keyinput"  /* 名字       */
12
13 /* key 设备结构体 */
14 struct key_dev{
15     struct input_dev *idev;  /* 按键对应的 input_dev 指针*/
16     struct timer_list timer; /* 消抖定时器               */
17     int gpio_key;            /* 按键对应的 GPIO 编号      */
18     int irq_key;             /* 按键对应的中断号          */
19 };
20
21 static struct key_dev key;  /* 按键设备                  */
22
23 /*
24  * @description   : 按键中断服务函数
25  * @param - irq   : 触发该中断事件对应的中断号
26  * @param - arg   : arg 参数可以在申请中断的时候进行配置
27  * @return        : 中断执行结果
28  */
29 static irqreturn_t key_interrupt(int irq, void *dev_id)
30 {
31     if(key.irq_key != irq)
32         return IRQ_NONE;
33
34     /* 按键防抖处理，开启定时器延时 15ms */
35     disable_irq_nosync(irq);     /* 禁止按键中断 */
36     mod_timer(&key.timer, jiffies + msecs_to_jiffies(15));
37
38     return IRQ_HANDLED;
39 }
40
41 /*
42  * @description   : 按键初始化函数
43  * @param - nd    : device_node 设备指针
44  * @return        : 成功返回 0，失败返回负数
```

```
45  */
46  static int key_gpio_init(struct device_node *nd)
47  {
48      int ret;
49      unsigned long irq_flags;
50
51      /* 从设备树中获取 GPIO */
52      key.gpio_key = of_get_named_gpio(nd, "key-gpio", 0);
53      if(!gpio_is_valid(key.gpio_key)) {
54          printk("key: Failed to get key-gpio\n");
55          return -EINVAL;
56      }
57
58      /* 申请使用 GPIO */
59      ret = gpio_request(key.gpio_key, "KEY0");
60      if (ret) {
61          printk(KERN_ERR "key: Failed to request key-gpio\n");
62          return ret;
63      }
64
65      /* 将 GPIO 设置为输入模式 */
66      gpio_direction_input(key.gpio_key);
67
68      /* 获取 GPIO 对应的中断号 */
69      key.irq_key = irq_of_parse_and_map(nd, 0);
70      if(!key.irq_key){
71          return -EINVAL;
72      }
73
74      /* 获取设备树中指定的中断触发类型 */
75      irq_flags = irq_get_trigger_type(key.irq_key);
76      if (IRQF_TRIGGER_NONE == irq_flags)
77          irq_flags = IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING;
78
79      /* 申请中断 */
80      ret = request_irq(key.irq_key, key_interrupt, irq_flags,
                          "Key0_IRQ", NULL);
81      if (ret) {
82          gpio_free(key.gpio_key);
83          return ret;
84      }
85
86      return 0;
```

```
87  }
88
89  /*
90   * @description   : 定时器服务函数，用于按键消抖，定时时间到了以后
91   *                  再读取按键值，根据按键的状态上报相应的事件
92   * @param - arg   : arg 参数就是定时器的结构体
93   * @return        : 无
94   */
95  static void key_timer_function(struct timer_list *arg)
96  {
97      int val;
98
99      /* 读取按键值并上报按键事件 */
100     val = gpio_get_value(key.gpio_key);
101     input_report_key(key.idev, KEY_0, !val);
102     input_sync(key.idev);
103
104     enable_irq(key.irq_key);
105 }
106
107 /*
108  * @description   : platform 驱动的 probe 函数，当驱动与设备匹配成功
109  *                  以后此函数会被执行
110  * @param - pdev  : platform 设备指针
111  * @return        : 0，成功;其他负值,失败
112  */
113 static int atk_key_probe(struct platform_device *pdev)
114 {
115     int ret;
116
117     /* 初始化 GPIO */
118     ret = key_gpio_init(pdev->dev.of_node);
119     if(ret < 0)
120         return ret;
121
122     /* 初始化定时器 */
123     timer_setup(&key.timer, key_timer_function, 0);
124
125     /* 申请 input_dev */
126     key.idev = input_allocate_device();
127     key.idev->name = KEYINPUT_NAME;
128
129 #if 0
```

```
130     /* 初始化 input_dev，设置产生哪些事件 */
131     __set_bit(EV_KEY, key.idev->evbit); /* 设置产生按键事件 */
132     __set_bit(EV_REP, key.idev->evbit); /* 重复事件，比如按下去不放开，
                                               就会一直输出信息 */
133
134     /* 初始化 input_dev，设置产生哪些按键 */
135     __set_bit(KEY_0, key.idev->keybit);
136 #endif
137
138 #if 0
139     key.idev->evbit[0] = BIT_MASK(EV_KEY) | BIT_MASK(EV_REP);
140     key.idev->keybit[BIT_WORD(KEY_0)] |= BIT_MASK(KEY_0);
141 #endif
142
143     key.idev->evbit[0] = BIT_MASK(EV_KEY) | BIT_MASK(EV_REP);
144     input_set_capability(key.idev, EV_KEY, KEY_0);
145
146     /* 注册输入设备 */
147     ret = input_register_device(key.idev);
148     if (ret) {
149         printk("register input device failed!\r\n");
150         goto free_gpio;
151     }
152
153     return 0;
154 free_gpio:
155     free_irq(key.irq_key,NULL);
156     gpio_free(key.gpio_key);
157     del_timer_sync(&key.timer);
158     return -EIO;
159
160 }
161
162 /*
163  * @description   : platform 驱动的 remove 函数，当 platform 驱动模块卸载
164  *                  时此函数会被执行
165  * @param - dev   : platform 设备指针
166  * @return        : 0，成功;其他负值,失败
167  */
168 static int atk_key_remove(struct platform_device *pdev)
169 {
170     free_irq(key.irq_key,NULL);         /* 释放中断号       */
171     gpio_free(key.gpio_key);            /* 释放 GPIO        */
```

```
172     del_timer_sync(&key.timer);          /* 删除 timer      */
173     input_unregister_device(key.idev); /* 释放 input_dev   */
174
175     return 0;
176 }
177
178 static const struct of_device_id key_of_match[] = {
179     {.compatible = "alientek,key"},
180     {/* Sentinel */}
181 };
182
183 static struct platform_driver atk_key_driver = {
184     .driver = {
185         .name = "stm32mp1-key",
186         .of_match_table = key_of_match,
187     },
188     .probe  = atk_key_probe,
189     .remove = atk_key_remove,
190 };
191
192 module_platform_driver(atk_key_driver);
193
194 MODULE_LICENSE("GPL");
195 MODULE_AUTHOR("ALIENTEK");
196 MODULE_INFO(intree, "Y");
```

在本程序中用到了定时器和中断相关的 API 函数，这些内容在前面章节都已经给大家介绍过了，而本章的重点知识点是 linux 下的 input 子系统，那下面我们将对 keyinput.c 代码进行讲解。

第 14~19 行，自定义的按键设备结构体 struct key_dev，用于描述一个按键设备，其中的成员变量包括一个 input_dev 指针变量，定时器 timer、GPIO 以及中断号。

第 29~39 行，按键中断处理函数 key_interrupt，当按键按下或松开的时候都会触发，也就是上升沿和下降沿都会触发此中断。key_interrupt 函数中的操作也很简单，调用 disable_irq_nosync 函数先禁止中断，然后使用 mod_timer 打开定时器，定时时长为 15ms。

第 46~87 行，按键的 GPIO 初始化。

第 95~105 行，定时器服务函数 key_timer_function，使用到定时器的目的主要是为了使用软件的方式进行按键消抖处理，在 key_timer_function 函数中，我们使用 gpio_get_value 获取按键 GPIO 的电平状态，使用 input_report_key 函数上报按键事件。在 val 变量前加了一个取反，因为按键按下的情况下，获取到的 val 为 0，按键松开的情况下，获取到的 val 为 1。但是 input 子系统框架规定按键按下上报 1，按键松开上报 0，所以这里需要进行取反操作。事件上报完成之后使用 input_sync 函数同步事件，表示此事件已上报完成，input 子系统核心层就会进行相关的处理。第 104 行 enable_irq 函数使能中断，因为在按键中断发生的时候我们会关闭中断，等事件处理完成之后再打开。

第 113~160 行，platform 驱动的 probe 函数 atk_key_probe，其中第 136~161 行，使用

input_allocate_device 函数申请 input_dev，然后设置相应的事件以及事件码(也就是 KEY 模拟成那个按键，这里我们设置为 KEY_0)。最后使用 input_register_device 函数向 Linux 内核注册 input_dev。

第 168~176 行，platform 驱动的 remove 函数 mykey_remove，在该函数中先释放 GPIO 在使用 del_timer_sync 删除定时器并且调用 input_unregister_device 卸载按键设备。

38.3.3 编写测试 APP

新建 keyinputApp.c 文件，然后在里面输入如下所示内容：

示例代码 38.3.3.1 keyinputApp.c 文件代码段

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  #include <sys/stat.h>
5  #include <fcntl.h>
6  #include <stdlib.h>
7  #include <string.h>
8  #include <linux/input.h>
9
10 /*
11  * @description    : main 主程序
12  * @param - argc   : argv 数组元素个数
13  * @param - argv   : 具体参数
14  * @return         : 0 成功;其他 失败
15  */
16 int main(int argc, char *argv[])
17 {
18     int fd, ret;
19     struct input_event ev;
20
21     if(2 != argc) {
22         printf("Usage:\n"
23             "\t./keyinputApp /dev/input/eventX    @ Open Key\n"
24         );
25         return -1;
26     }
27
28     /* 打开设备 */
29     fd = open(argv[1], O_RDWR);
30     if(0 > fd) {
31         printf("Error: file %s open failed!\r\n", argv[1]);
32         return -1;
33     }
34
```

```
35      /* 读取按键数据 */
36      for ( ; ; ) {
37
38          ret = read(fd, &ev, sizeof(struct input_event));
39          if (ret) {
40              switch (ev.type) {
41              case EV_KEY:                        /* 按键事件              */
42                  if (KEY_0 == ev.code) {         /* 判断是不是 KEY_0 按键 */
43                      if (ev.value)               /* 按键按下              */
44                          printf("Key0 Press\n");
45                      else                        /* 按键松开              */
46                          printf("Key0 Release\n");
47                  }
48                  break;
49
50              /* 其他类型的事件，自行处理 */
51              case EV_REL:
52                  break;
53              case EV_ABS:
54                  break;
55              case EV_MSC:
56                  break;
57              case EV_SW:
58                  break;
59              };
60          }
61          else {
62              printf("Error: file %s read failed!\r\n", argv[1]);
63              goto out;
64          }
65      }
66
67  out:
68      /* 关闭设备 */
69      close(fd);
70      return 0;
71  }
```

第 38.1.3 小节已经说过了，Linux 内核会使用 input_event 结构体来表示输入事件，所以我们要获取按键输入信息，那么必须借助于 input_event 结构体。第 19 行定义了一个 input_event 类型变量 ev。

第 36~65 行，当我们向 Linux 内核成功注册 input_dev 设备以后，会在/dev/input 目录下生成一个名为“eventX(X=0….n)”的文件，这个/dev/input/eventX 就是对应的 input 设备文件。我们读取这个文件就可以获取到输入事件信息，比如按键值什么的。使用 read 函数读取输入设备

文件，也就是/dev/input/eventX，读取到的数据按照 input_event 结构体组织起来。获取到输入事件以后(input_event 结构体类型)使用 switch case 语句来判断事件类型，本章实验我们设置的事件类型为 EV_KEY，因此只需要处理 EV_KEY 事件即可。比如获取按键编号(KEY_0 的编号为 11)、获取按键状态，按下还是松开的？

38.4 运行测试

38.4.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为“keyinput.o”，Makefile 内容如下所示：

示例代码 38.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := keyinput.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“keyinput.o”。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“keyinput.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 keyinputApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc keyinputApp.c -o keyinputApp
```

编译成功以后就会生成 keyinputApp 这个应用程序。

38.4.2 运行测试

将上一小节编译出来 keyinput.ko 和 keyinputApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中。在加载 keyinput.ko 驱动模块之前，先看一下/dev/input 目录下都有哪些文件，结果如图 38.4.2.1 所示：

```
[root@ATK-stm32mp1]:~$ ls /dev/input
ls: /dev/input: No such file or directory
[root@ATK-stm32mp1]:~$
```

图 38.4.2.1 /dev/input 目录

从图 38.4.2.1 可以看出，在/dev 目录下根本就不存在 input 文件夹，原因在于此时系统中并没有加载过 input 输入类设备，所以这个文件夹不存在。接下来输入如下命令加载 keyinput.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe keyinput.ko    //加载驱动模块
```

当驱动模块加载成功以后再来看一下/dev/input 目录下有哪些文件，结果如图 38.4.2.2 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls /dev/input/ -l
total 0
crw-r-----    1 root     root       13,  64 Jan 20 01:05 event0
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 38.4.2.2 加载驱动以后的/dev/input 目录

从图 38.4.2.2 可以看出，在/dev/input 目录下生成了一个 event0 文件，这其实就是我们注册的驱动所对应的设备文件。keyinputApp 就是通过读取/dev/input/event0 这个文件来获取输入事件信息的，输入如下测试命令：

```
./keyinputApp /dev/input/event0
```

然后按下开发板上的 KEY 按键，结果如图 38.4.2.3 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./keyinputApp /dev/input/event0
Key0 Press
Key0 Release
Key0 Press
Key0 Release
Key0 Press
Key0 Release
```

图 38.4.2.3 测试结果

从图 38.4.2.3 可以看出，当我们按下或者释放开发板上的按键以后都会在终端上输出相应的内容，提示我们哪个按键按下或释放了，在 Linux 内核中 KEY_0 为 11。

另外，我们也可以不用 keyinputApp 来测试驱动，可以直接使用 hexdump 命令来查看/dev/input/event0 文件内容，输入如下命令：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ hexdump /dev/input/event0
0000000 5ffe 3886 7a70 0006 0001 000b 0001 0000
0000010 5ffe 3886 7a70 0006 0000 0000 0000 0000
0000020 5ffe 3886 fcd4 0009 0001 000b 0000 0000
0000030 5ffe 3886 fcd4 0009 0000 0000 0000 0000
0000040 5fff 3886 f7fa 0002 0001 000b 0001 0000
0000050 5fff 3886 f7fa 0002 0000 0000 0000 0000
```

图 38.4.2.4 原始数据值

图 38.4.2.4 就是 input_event 类型的原始事件数据值，采用十六进制表示，这些原始数据的含义如下：

示例代码 38.4.2.1 input_event 类型的原始事件值

```
/*************************input_event 类型*****************************/
/* 编号 */ /* tv_sec */ /* tv_usec */ /* type */  /* code */  /* value */
0000000    5ffe 3886     7a70 0006     0001        000b       0001 0000
0000010    5ffe 3886     7a70 0006     0000        0000       0000 0000
0000020    5ffe 3886     fcd4 0009     0001        000b       0000 0000
0000030    5ffe 3886     fcd4 0009     0000        0000       0000 0000
```

type 为事件类型，查看示例代码 38.1.2.3 可知，EV_KEY 事件值为 1，EV_SYN 事件值为 0。因此第 1 行表示 EV_KEY 事件，第 2 行表示 EV_SYN 事件。code 为事件编码，也就是按键号，查看示例代码 38.1.2.4 可以，KEY_0 这个按键编号为 11，对应的十六进制为 0xb，因此第 1 行表示 KEY_0 这个按键事件，最后的 value 就是按键值，为 1 表示按下，为 0 的话表示松开。综上所述，示例代码 38.4.2.1 中的原始事件值含义如下：

第 1 行，按键(KEY_0)按下事件。

第 2 行，EV_SYN 同步事件，因为每次上报按键事件以后都要同步的上报一个 EV_SYN 事

件。

第 3 行，按键(KEY_0)松开事件。

第 4 行，EV_SYN 同步事件，和第 2 行一样。

38.5 Linux 自带按键驱动程序的使用

38.5.1 自带按键驱动程序源码简析

Linux 内核也自带了 KEY 驱动，如果要使用内核自带的 KEY 驱动的话需要配置 Linux 内核，不过 Linux 内核一般默认已经使能了 KEY 驱动，但是我们还是要检查一下。按照如下路径找到相应的配置选项：

```
→ Device Drivers
    → Input device support
        → Generic input layer (needed for keyboard, mouse, ...) (INPUT [=y])
            → Keyboards (INPUT_KEYBOARD [=y])
                →GPIO Buttons
```

选中“GPIO Buttons”选项，将其编译进 Linux 内核中，如图 38.5.1.1 所示：

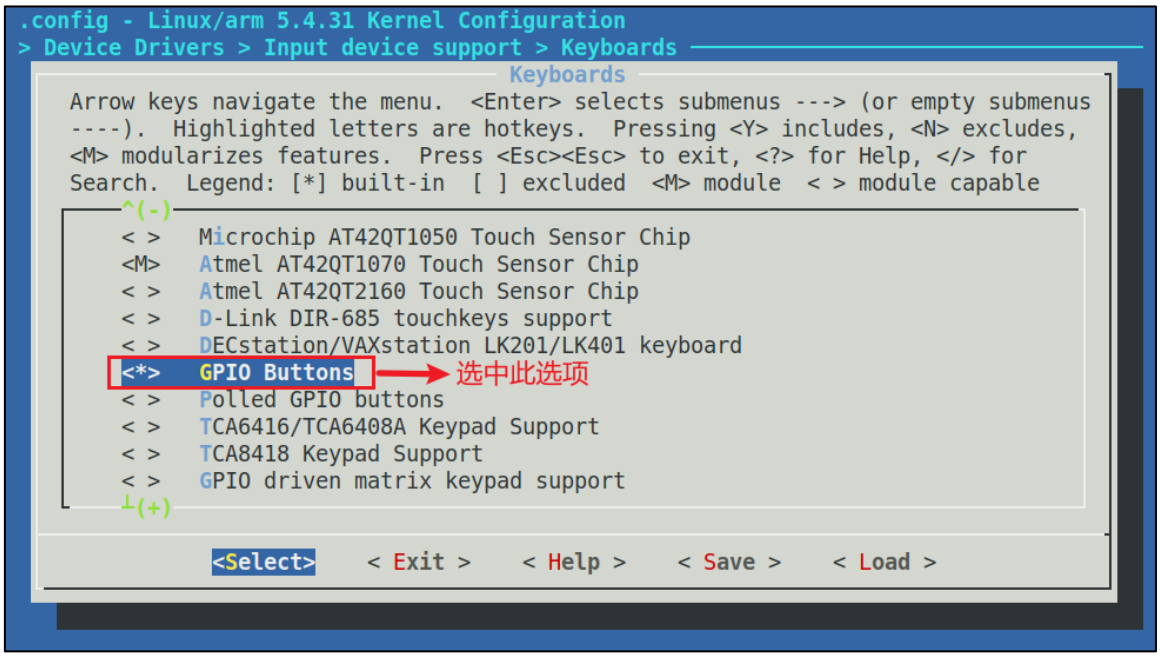


图 38.5.1.1 内核自带 KEY 驱动使能选项

选中以后就会在.config 文件中出现“CONFIG_KEYBOARD_GPIO=y”这一行，Linux 内核就会根据这一行来将 KEY 驱动文件编译进 Linux 内核。Linux 内核自带的 KEY 驱动文件为 drivers/input/keyboard/gpio_keys.c，gpio_keys.c 采用了 platform 驱动框架，在 KEY 驱动上使用了 input 子系统实现。在 gpio_keys.c 文件中找到如下所示内容：

示例代码 38.5.1.1 gpio_keys 文件代码段

```
758 static const struct of_device_id gpio_keys_of_match[] = {
759     { .compatible = "gpio-keys", },
760     { },
761 };
......
```

```
1012    static struct platform_driver gpio_keys_device_driver = {
1013        .probe      = gpio_keys_probe,
1014        .shutdown   = gpio_keys_shutdown,
1015        .driver     = {
1016            .name   = "gpio-keys",
1017            .pm = &gpio_keys_pm_ops,
1018            .of_match_table = gpio_keys_of_match,
1019            .dev_groups = gpio_keys_groups,
1020        }
1021    };
1022
1023    static int __init gpio_keys_init(void)
1024    {
1025        return platform_driver_register(&gpio_keys_device_driver);
1026    }
1027
1028    static void __exit gpio_keys_exit(void)
1029    {
1030        platform_driver_unregister(&gpio_keys_device_driver);
1031    }
```

从示例代码 38.5.1.1 可以看出，这就是一个标准的 platform 驱动框架，如果要使用设备树来描述 KEY 设备信息的话，设备节点的 compatible 属性值要设置为“gpio-keys”。当设备和驱动匹配以后 gpio_keys_probe 函数就会执行，gpio_keys_probe 函数内容如下(为了篇幅有缩减)：

示例代码 38.5.1.2 gpio_keys_probe 函数代码段

```
764     static int gpio_keys_probe(struct platform_device *pdev)
765     {
766         struct device *dev = &pdev->dev;
767         const struct gpio_keys_platform_data *pdata =
                                            dev_get_platdata(dev);
768         struct fwnode_handle *child = NULL;
769         struct gpio_keys_drvdata *ddata;
770         struct input_dev *input;
771         int i, error;
772         int wakeup = 0;
773
774         if (!pdata) {
775             pdata = gpio_keys_get_devtree_pdata(dev);
776             if (IS_ERR(pdata))
777                 return PTR_ERR(pdata);
778         }
......
793         input = devm_input_allocate_device(dev);
794         if (!input) {
```

```
795             dev_err(dev, "failed to allocate input device\n");
796             return -ENOMEM;
797         }
.....
806         input->name = pdata->name ? : pdev->name;
807         input->phys = "gpio-keys/input0";
808         input->dev.parent = dev;
809         input->open = gpio_keys_open;
810         input->close = gpio_keys_close;
811
812         input->id.bustype = BUS_HOST;
813         input->id.vendor = 0x0001;
814         input->id.product = 0x0001;
815         input->id.version = 0x0100;
.....
821         /* Enable auto repeat feature of Linux input subsystem */
822         if (pdata->rep)
823             __set_bit(EV_REP, input->evbit);
824
825         for (i = 0; i < pdata->nbuttons; i++) {
826             const struct gpio_keys_button *button =
                                    &pdata->buttons[i];
827
828             if (!dev_get_platdata(dev)) {
829                 child = device_get_next_child_node(dev, child);
830                 if (!child) {
831                     dev_err(dev,
832                         "missing child device node for entry %d\n",
833                         i);
834                     return -EINVAL;
835                 }
836             }
837
838             error = gpio_keys_setup_key(pdev, input, ddata,
839                                 button, i, child);
840             if (error) {
841                 fwnode_handle_put(child);
842                 return error;
843             }
844
845             if (button->wakeup)
846                 wakeup = 1;
847         }
```

```
848
849         fwnode_handle_put(child);
850
851         error = input_register_device(input);
852         if (error) {
853             dev_err(dev, "Unable to register input device,
error: %d\n",
854                 error);
855             return error;
856         }
857
858         device_init_wakeup(dev, wakeup);
859
860         return 0;
861     }
```

第 775 行，调用 gpio_keys_get_devtree_pdata 函数从设备树中获取到 KEY 相关的设备节点信息。

第 793 行，使用 devm_input_allocate_device 函数申请 input_dev。

第 806~815，初始化 input_dev。

第 823 行，设置 input_dev 事件，这里设置了 EV_REP 事件。

第 838 行，调用 gpio_keys_setup_key 函数继续设置 KEY，此函数会设置 input_dev 的 EV_KEY 事件以及事件码(也就是 KEY 模拟哪个按键)。

第 851 行，调用 input_register_device 函数向 Linux 系统注册 input_dev。

我们接下来再来看一下 gpio_keys_setup_key 函数，此函数内容如下：

示例代码 38.5.1.3 gpio_keys_setup_key 函数代码段

```
477     static int gpio_keys_setup_key(struct platform_device *pdev,
478                    struct input_dev *input,
479                    struct gpio_keys_drvdata *ddata,
480                    const struct gpio_keys_button *button,
481                    int idx,
482                    struct fwnode_handle *child)
483     {
484         const char *desc = button->desc ? button->desc : "gpio_keys";
485         struct device *dev = &pdev->dev;
486         struct gpio_button_data *bdata = &ddata->data[idx];
487         irq_handler_t isr;
488         unsigned long irqflags;
489         int irq;
490         int error;
491
492         bdata->input = input;
493         bdata->button = button;
494         spin_lock_init(&bdata->lock);
```

```
495
.....
612         bdata->code = &ddata->keymap[idx];
613         *bdata->code = button->code;
614         input_set_capability(input, button->type ?: EV_KEY,
*bdata->code);
615
.....
641
642         return 0;
643     }
```

第 614 行，调用 input_set_capability 函数设置 EV_KEY 事件以及 KEY 的按键类型，也就是 KEY 作为哪个按键？我们会在设备树里面设置指定的 KEY 作为哪个按键。

一切都准备就绪以后剩下的就是等待按键按下，然后向 Linux 内核上报事件，事件上报是在 gpio_keys_irq_isr 函数中完成的，此函数内容如下：

示例代码 38.5.1.4 gpio_keys_irq_isr 函数代码段

```
433     static irqreturn_t gpio_keys_irq_isr(int irq, void *dev_id)
434     {
435         struct gpio_button_data *bdata = dev_id;
436         struct input_dev *input = bdata->input;
437         unsigned long flags;
438
439         BUG_ON(irq != bdata->irq);
440
441         spin_lock_irqsave(&bdata->lock, flags);
442
443         if (!bdata->key_pressed) {
444             if (bdata->button->wakeup)
445                 pm_wakeup_event(bdata->input->dev.parent, 0);
446
447             input_event(input, EV_KEY, *bdata->code, 1);
448             input_sync(input);
449
450             if (!bdata->release_delay) {
451                 input_event(input, EV_KEY, *bdata->code, 0);
452                 input_sync(input);
453                 goto out;
454             }
455
456             bdata->key_pressed = true;
457         }
458
459         if (bdata->release_delay)
```

```
460             mod_timer(&bdata->release_timer,
461                 jiffies + msecs_to_jiffies(bdata->release_delay));
462     out:
463         spin_unlock_irqrestore(&bdata->lock, flags);
464         return IRQ_HANDLED;
465     }
```

gpio_keys_irq_isr 是按键中断处理函数，第 447 行向 Linux 系统上报 EV_KEY 事件，表示按键按下。第 448 行使用 input_sync 函数向系统上报 EV_REP 同步事件。

综上所述，Linux 内核自带的 gpio_keys.c 驱动文件思路和我们前面编写的 keyinput.c 驱动文件基本一致。都是申请和初始化 input_dev，设置事件，向 Linux 内核注册 input_dev。最终在按键中断服务函数或者消抖定时器中断服务函数中上报事件和按键值。

38.5.2 自带按键驱动程序的使用

要使用 Linux 内核自带的按键驱动程序很简单，只需要根据 Documentation/devicetree/bindings/input/gpio-keys.txt 这个文件在设备树中添加指定的设备节点即可，节点要求如下：

①、节点名字为“gpio-keys”。

②、gpio-keys 节点的 compatible 属性值一定要设置为“gpio-keys”。

③、所有的 KEY 都是 gpio-keys 的子节点，每个子节点可以用如下属性描述自己：

gpios：KEY 所连接的 GPIO 信息。

interrupts：KEY 所使用 GPIO 中断信息，不是必须的，可以不写。

label：KEY 名字

linux,code：KEY 要模拟的按键，也就是示例代码 38.1.2.4 中的这些按键。

④、如果按键要支持连按的话要加入 autorepeat。

这里我们将开发板上的三个按键都用起来，KEY0、KEY1 和 WKUP 分别模拟为键盘上的：L、S 和 Enter(回车)健。

打开 stm32mp157d-atk.dts，先添加一个头文件“dt-bindings/input/input.h”此文件就是“linux,code”属性的按键宏定义，如图 38.5.2.1 所示：

```
13  #include "stm32mp157-m4-srm-pinctrl.dtsi"
14  #include "stm32mp157d-atk.dtsi"
15  #include "stm32mp157d-atk.dtsi"
16  #include "dt-bindings/input/input.h"
17
```

添加此头文件

图 38.5.2.1 添加头文件

头文件添加完成以后再根据上面的要求创建对应的设备节点，设备节点内容如下所示：

示例代码 38.5.2.1 gpio-keys 节点内容

```
1  gpio-keys {
2      compatible = "gpio-keys";
3      pinctrl-names = "default";
4      pinctrl-0 = <&key_pins_a>;
5      autorepeat;
6
7      key0 {
8          label = "GPIO Key L";
```

```
9           linux,code = <KEY_L>;
10          gpios = <&gpiog 3 GPIO_ACTIVE_LOW>;
11      };
12
13      key1 {
14          label = "GPIO Key S";
15          linux,code = <KEY_S>;
16          gpios = <&gpioh 7 GPIO_ACTIVE_LOW>;
17      };
18
19      wkup {
20          label = "GPIO Key Enter";
21          linux,code = <KEY_ENTER>;
22          gpios = <&gpioa 0 GPIO_ACTIVE_HIGH>;
23          gpio-key,wakeup;
24      };
25  };
```

第 5 行，autorepeat 表示按键支持连按。

第 7~11 行，STM32MP1 开发板 KEY0 按键信息，名字设置为“GPIO Key L”，这里我们将开发板上的 KEY0 按键设置为“EKY_L”这个按键，也就是‘L’键，效果和键盘上的‘L’键一样。

第 13~17 行，KEY1 按键设置，模拟键盘上的‘S’键。

第 19~24 行，WKUP 按键，模拟键盘上的‘回车’键，注意，WKUP 按键连接的 PA0 引脚，查看原理图可以知道，WKUP 按下去以后是高电平，因此这里设置高电平有效。

最后一定要检查一下设备树看看这些引脚有没有被用到其他外设上，如果有的话要删除掉相关代码！

重新编译设备树，然后用新编译出来的 stm32mp157d-atk.dtb 启动 Linux 系统，系统启动以后查看/dev/input 目录，看看都有哪些文件，结果如图 38.5.2.1 所示：

```
[root@ATK-stm32mp1]:~$ ls /dev/input/ -l
total 0
crw-r-----    1 root     root       13,  64 Jan  1  1970 event0
[root@ATK-stm32mp1]:~$
```

图 38.5.2.1 /dev/input 目录文件

从图 38.5.2.1 可以看出存在 event0 这个文件，这个文件就是 KEY 对应的设备文件，使用 hexdump 命令来查看/dev/input/event0 文件，输入如下命令：

```
hexdump /dev/input/event0
```

然后按下 STM32MP1 开发板上的按键，终端输出图 38.5.2.2 所示内容：

```
[root@ATK-stm32mp1]:~$ hexdump /dev/input/event0
0000000 f396 3887 f8d6 000d 0001 0026 0001 0000
0000010 f396 3887 f8d6 000d 0000 0000 0000 0000
0000020 f397 3887 34c7 0000 0001 0026 0000 0000
0000030 f397 3887 34c7 0000 0000 0000 0000 0000
0000040 f397 3887 1edf 000a 0001 001f 0001 0000
0000050 f397 3887 1edf 000a 0000 0000 0000 0000
0000060 f397 3887 f26a 000b 0001 001f 0000 0000
0000070 f397 3887 f26a 000b 0000 0000 0000 0000
0000080 f398 3887 e218 0003 0001 001c 0001 0000
0000090 f398 3887 e218 0003 0000 0000 0000 0000
00000a0 f398 3887 3ec7 0005 0001 001c 0000 0000
00000b0 f398 3887 3ec7 0005 0000 0000 0000 0000
```

图 38.5.2.2 按键信息

如果按下 KEY 按键以后会在终端上输出图 38.5.2.2 所示的信息那么就表示 Linux 内核的按键驱动工作正常。至于图 38.5.2.2 中内容的含义大家就自行分析，这个已经在 38.4.2 小节详细的分析过了，这里就不再讲解了。

大家如果发现按下 KEY 按键以后没有反应，那么请检查一下三方面：

①、是否使能 Linux 内核 KEY 驱动。

②、设备树中 gpio-keys 节点是否创建成功。

③、在设备树中是否有其他外设也使用了 KEY 按键对应的 GPIO，但是我们并没有删除掉这些外设信息。检查 Linux 启动 log 信息，看看是否有类似下面这条信息：

```
gpio-keys gpio-keys: failed to get gpio: -16
```

上述信息表示 GPIO 申请失败，失败的原因就是有其他的外设正在使用此 GPIO。

第三十九章 Linux PWM 驱动实验

PWM 是很常用到功能，我们可以通过 PWM 来控制电机速度，也可以使用 PWM 来控制 LCD 的背光亮度。本章我们就来学习一下如何在 Linux 下进行 PWM 驱动开发。

39.1 PWM 驱动简析

PWM 全称是 Pulse Width Modulation，也就是脉冲宽度调制，PWM 信号如图 39.1.1 所示：

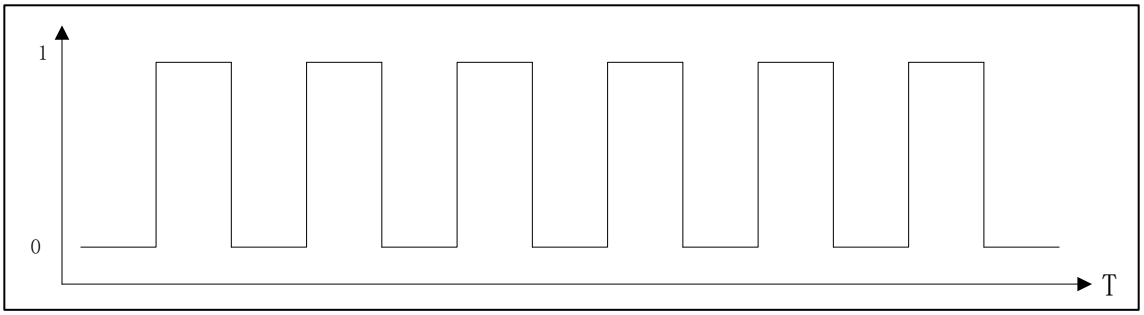


图 39.1.1 PWM 信号

PWM 信号有两个关键的术语：频率和占空比，频率就是开关速度，把一次开关算作一个周期，那么频率就是 1 秒内进行了多少次开关。占空比就是一个周期内高电平时间和低电平时间的比例，一个周期内高电平时间越长占空比就越大，反之占空比就越小。占空比用百分之表示，如果一个周期内全是低电平那么占空比就是 0%，如果一个周期内全是高电平那么占空比就是 100%。

我们给 LCD 的背光引脚输入一个 PWM 信号，这样就可以通过调整占空比的方式来调整 LCD 背光亮度了。提高占空比就会提高背光亮度，降低占空比就会降低背光亮度，重点就在于 PWM 信号的产生和占空比的控制。

39.1.1 设备树下的 PWM 控制器节点

1、定时器节点

STM32MP157 有很多路 PWM，这些 PWM 都是由定时器产生的：

TIM1/TIM8：这 2 个是 16 位高级定时器，主要用于电机控制。这两个定时器支持 PWM 型号，每个定时器支持 4 通道 PWM 信号。

TIM2/TIM3/TIM4/TIM5：这 4 个是通用定时器，TIM3/TIM4 是 16 位定时器，TIM2/TIM5 是 32 位定时器。这 4 个定时器也支持 PWM 输出，每个定时器支持 4 通道 PWM 信号。

TIM12/TIM13/TIM14：这 3 个都是 16 位的通用定时器，TIM12 支持 2 通道的 PWM 信号，TIM13/TIM14 这两个定时器每个只支持 1 个通道的 PWM 信号。

TIM15/TIM16/TIM17：这 3 个也都是 16 位的通用定时器，TIM15 支持 2 通道的 PWM 信号，TIM16/TIM17 每个定时器支持 1 通道的 PWM 信号。

可以看出，STM32MP157 的 PWM 通道非常多，不同的 PWM 通道功能也不同，我们可以根据实际情况选择合适的 PWM 通道。本节我们使用 PA10 这个引脚来实现 PWM 功能，注意！PA10 这个引脚被用作 USB 的 ID 引脚，如果你所使用的开发板使用了 PA10 作为 USB OTG 的 ID 引脚，那么在做本实验的时候开发板的 USB OTG 接口不能连接到电脑上！正点原子 STM32MP157 开发板的 USB 接口采用了 TypeC 接口，因此没有用到 PA10 作为 ID 引脚。

打开 STM32MP157 的数据手册，可以看到 PA10 可以作为 TIM1 的通道 3，如图 39.1.1.1 所示：

U16	T16	Y17	Y17	PA10	I/O	FT_u	-	TIM1_CH3, SPI3_NSS/I2S3_WS, USART1_RX, MDIOS_MDIO, SAI4_FS_B, DCMI_D1, LCD_B1, EVENTOUT	OTG_FS_ID, OTG_HS_ID

图 39.1.1.1 PA10 引脚共功能

2、TIM1 简介

TIM1 通道 3 这一路 PWM 属于 TIM1 定时器，我们先简单了解一下 TIM1 定时器的。TIM1 叫做高级定时器，因为 TIM1 的功能非常强大，主要用于电机驱动。TIM1 主要特性如下：

①、16 位的向上、向下自动加载计数器。

②、16 位可编程的预分频器。

③、6 个独立的通道，这些通道的功能如下：

— 输入捕获(只有通道 5 和 6 支持)。

— 输出比较

— PWM 波形生成(边缘和中间对齐模式)。

— 单脉冲模式。

④、带有死区的可编程互补输出。

⑤、以下事件可以生成中断或者 DMA：

— 更新事件，计数器溢出。

— 触发事件，计数器开始、停止、初始化等。

— 输入捕获。

— 输出比较。

3、TIM1 设备节点

接下来看一下 TIM1 的设备树，STM32 定时器设备树绑定信息文档为：Documentation/devicetree/bindings/mfd/stm32-timers.txt，我们简单总结一下定时器节点信息。

①、必须的参数：

compatible：必须是“st,stm32-timers”。

reg：定时器控制器物理寄存器基地址，对于 TIM1 来说，这个地址为 0x44000000，这个可以在 STM32MP157 的参考手册上找到。

clock-names：时钟源名字，设置为“int”。

clocks：时钟源。

②、可选的参数：

resets：复位句柄，用来复位定时器控制器，可以参考文档 reset/st,stm32-rcc.txt。

dmas：DMA 通道，最多 7 通道的 DMA。

dma-names：DMA 名字列表，必须和“dmas”属性匹配，可选的名字有：“ch1”、“ch2”、“ch3”、“ch4”、“up”、“trig”、“com”。

③、可选的子节点：

STM32 定时器有多种功能，比如及时、PWM、计数器等，不同的功能需要用不同的子节点来表示，可选子节点有三种，分别对应不同的功能：

pwm: pwm 子节点描述定时器的 PWM 功能，关于 PWM 的详细信息请参考绑定文档 pwm/pwm-stm32.txt，稍微我们会讲解。

timer: timer 子节点描述定时器的定时功能，定时相关信息请参考绑定文档 iio/timer/stm32-timer-trigger.txt。

counter: counter 子节点描述定时器的计数功能，相关信息请参考绑定文档 counter/stm32-timer-cnt.txt。

了解完定时器的绑定文档以后，我们来看一下 STM32MP157 实际的定时器节点，打开

stm32mp151.dtsi，找到名为“timers1”的设备节点，这个就是 TIM1 定时器节点，内容如下：

示例代码 39.1.1.1 TIM1 节点

```
1  timers1: timer@44000000 {
2       #address-cells = <1>;
3       #size-cells = <0>;
4       compatible = "st,stm32-timers";
5       reg = <0x44000000 0x400>;
6       clocks = <&rcc TIM1_K>;
7       clock-names = "int";
8       dmas = <&dmamux1 11 0x400 0x80000001>,
9              <&dmamux1 12 0x400 0x80000001>,
10             <&dmamux1 13 0x400 0x80000001>,
11             <&dmamux1 14 0x400 0x80000001>,
12             <&dmamux1 15 0x400 0x80000001>,
13             <&dmamux1 16 0x400 0x80000001>,
14             <&dmamux1 17 0x400 0x80000001>;
15      dma-names = "ch1", "ch2", "ch3", "ch4",
16                  "up", "trig", "com";
17      status = "disabled";
18
19      pwm {
20          compatible = "st,stm32-pwm";
21          #pwm-cells = <3>;
22          status = "disabled";
23      };
24
25      timer@0 {
26          compatible = "st,stm32h7-timer-trigger";
27          reg = <0>;
28          status = "disabled";
29      };
30
31      counter {
32          compatible = "st,stm32-timer-counter";
33          status = "disabled";
34      };
35 };
```

第 19~23 行，TIM1 的 pwm 功能子节点，这个是我们本小节重点关注的。

4、PWM 设备子节点

通过上面对定时器绑定文档的讲解，我们知道 PWM 作为定时器的子节点，这里我们就来看一下 PWM 子节点绑定文档：Documentation/devicetree/bindings/pwm/pwm-stm32.txt，我们简单总结一下 PWM 子节点属性信息：

compatible：必须为“st,stm32-pwm”。

pinctrl-names：设置为“default”，也可以添加“sleep”，这样当进入低功耗的时候 PWM 引脚引入 sleep 模式。

pinctrl-n：PWM 引脚 pinctrl 句柄，用来指定 PWM 信号输出引脚。

#pwm-cells：应该设置为 3。

STM32MP157 的 PWM 节点的 compatible 属性为“st,stm32-pwm”，我们可以在 linux 内核源码中搜索这个字符串找到 PWM 驱动文件，这个文件为：drivers/pwm/pwm-stm32.c。

39.1.2 PWM 子系统

Linux 内核提供了个 PWM 子系统框架，编写 PWM 驱动的时候一定要符合这个框架。PWM 子系统的核心是 pwm_chip 结构体，定义在文件 include/linux/pwm.h 中，定义如下：

示例代码 39.1.2.1 pwm_chip 结构体

```
290 struct pwm_chip {
291     struct device        *dev;
292     const struct pwm_ops *ops;
293     int                  base;
294     unsigned int         npwm;
295
296     struct pwm_device * (*of_xlate)(struct pwm_chip *pc,
297                 const struct of_phandle_args *args);
298     unsigned int of_pwm_n_cells;
299
300     /* only used internally by the PWM framework */
301     struct list_head list;
302     struct pwm_device *pwms;
303 };
```

第 292 行，pwm_ops 结构体就是 PWM 外设的各种操作函数集合，编写 PWM 外设驱动的时候需要开发人员实现。pwm_ops 结构体也定义在 pwm.h 头文件中，定义如下：

示例代码 39.1.2.2 pwm_ops 结构体

```
259 struct pwm_ops {
260     int (*request)(struct pwm_chip *chip,      /* 请求 PWM */
                   struct pwm_device *pwm);
261     void (*free)(struct pwm_chip *chip,        /* 释放 PWM */
                   struct pwm_device *pwm);
262     int (*capture)(struct pwm_chip *chip,      /* 捕获 PWM 信号 */
                  struct pwm_device *pwm,
263                 struct pwm_capture *result, unsigned long timeout);
264     int (*apply)(struct pwm_chip *chip, /* 新的 PWM 配置方法，配置 PWM 周
                                           期和占空比 */
                  struct pwm_device *pwm,
265                 const struct pwm_state *state);
266     void (*get_state)(struct pwm_chip *chip,
                      struct pwm_device *pwm,
267                     struct pwm_state *state);
```

```
268     struct module *owner;
269
270     /* Only used by legacy drivers */
271     int (*config)(struct pwm_chip *chip,   /* 配置 PWM 周期和占空比 */
                  struct pwm_device *pwm,
272               int duty_ns, int period_ns);
273     int (*set_polarity)(struct pwm_chip *chip,    /*  设置 PWM 极性 */
274           struct pwm_device *pwm,enum pwm_polarity polarity);
275     int (*enable)(struct pwm_chip *chip,          /* 使能 PWM */
                    struct pwm_device *pwm);
276     void (*disable)(struct pwm_chip *chip,        /* 关闭 PWM */
                    struct pwm_device *pwm);
277 };
```

pwm_ops 中的这些函数不一定全部实现，但是配置 PWM 的函数必须实现，比如 apply 或者 config。第 264 行的 applay 函数是最新的 PWM 配置函数，通过此函数来配置 PWM 的周期以及占空比，老的内核里面会使用第 271 行的 config 函数来配置 PWM。其中第 271~276 行的 config、set_polarity、enable 和 disable 都是老版本内核所使用的函数。

PWM 子系统驱动的核心初始化 pwm_chip 结构体，然后向内核注册初始化完成以后的 pwm_chip。这里就要用到 pwmchip_add 函数，此函数定义在 drivers/pwm/core.c 文件中，函数原型如下：

```
int pwmchip_add(struct pwm_chip      *chip)
```

函数参数和返回值含义如下：

chip：要向内核注册的 pwm_chip。

返回值：0 成功；负数 失败。

卸载 PWM 驱动的时候需要将前面注册的 pwm_chip 从内核移除掉，这里要用到 pwmchip_remove 函数，函数原型如下：

```
int pwmchip_remove(struct pwm_chip *chip)
```

函数参数和返回值含义如下：

chip：要移除的 pwm_chip。

返回值：0 成功；负数 失败。

39.1.3 PWM 驱动源码分析

我们简单分析一下 Linux 内核自带的 STM32MP157 PWM 驱动，驱动文件前面都说了，是 pwm-stm32.c 这个文件。打开这个文件，可以看到，这是一个标准的平台设备驱动文件，有如下所示：

示例代码 39.1.3.1 STM32MP157 PWM 平台驱动

```
1 static const struct of_device_id stm32_pwm_of_match[] = {
2     { .compatible = "st,stm32-pwm", },
3     { /* end node */ },
4 };
5 MODULE_DEVICE_TABLE(of, stm32_pwm_of_match);
6
7 static struct platform_driver stm32_pwm_driver = {
```

```
8       .probe      = stm32_pwm_probe,
9       .remove     = stm32_pwm_remove,
10      .driver     = {
11          .name = "stm32-pwm",
12          .of_match_table = stm32_pwm_of_match,
13          .pm = &stm32_pwm_pm_ops,
14      },
15  };
```

第 2 行，当设备树 PWM 节点的 compatible 属性值为“st,stm32-pwm”的话就会匹配此驱动。

第 14 行，当设备树节点和驱动匹配以后 stm32_pwm_probe 函数就会执行。

在看 stm32_pwm_probe 函数之前先来看下 stm32_pwm 结构体，这个结构体是 ST 官方创建的 STM32 PWM 结构体，这个结构体会贯穿整个 PWM 驱动，起到灵魂的作用。stm32_pwm 结构体定义在 pwm-stm32.c 文件中，结构体内容如下：

示例代码 39.1.3.2 stm32_pwm 结构体

```
1  struct stm32_pwm {
2       struct pwm_chip chip;
......
10      u32 capture[4] ____cacheline_aligned; /* DMA'able buffer */
11  };
```

重点看一下第 2 行，这是一个 pwm_chip 结构体成员变量 chip，前面说了，PWM 子系统的核心就是 pwm_chip。

stm32_pwm_probe 函数如下(有缩减)：

示例代码 39.1.3.3 stm32_pwm_probe 函数

```
608 static int stm32_pwm_probe(struct platform_device *pdev)
609 {
610     struct device *dev = &pdev->dev;
611     struct device_node *np = dev->of_node;
612     struct stm32_timers *ddata = dev_get_drvdata(pdev->dev.parent);
613     struct stm32_pwm *priv;
614     int ret;
615
616     priv = devm_kzalloc(dev, sizeof(*priv), GFP_KERNEL);
617     if (!priv)
618         return -ENOMEM;
619
620     mutex_init(&priv->lock);
621     priv->regmap = ddata->regmap;
622     priv->clk = ddata->clk;
623     priv->max_arr = ddata->max_arr;
624     priv->chip.of_xlate = of_pwm_xlate_with_flags;
625     priv->chip.of_pwm_n_cells = 3;
626
```

```
627     if (!priv->regmap || !priv->clk)
628         return -EINVAL;
629 
630     ret = stm32_pwm_probe_breakinputs(priv, np);
631     if (ret)
632         return ret;
633 
634     stm32_pwm_detect_complementary(priv);
635 
636     priv->chip.base = -1;
637     priv->chip.dev = dev;
638     priv->chip.ops = &stm32pwm_ops;
639     priv->chip.npwm = stm32_pwm_detect_channels(priv);
640 
641     ret = pwmchip_add(&priv->chip);
642     if (ret < 0)
643         return ret;
644 
645     platform_set_drvdata(pdev, priv);
646 
647     return 0;
648 
```

第 616 行，priv 是一个 stm32_pwm 类型的结构体指针变量，这里为其申请内存。stm32_pwm 结构体有个重要的成员变量 chip，chip 是 pwm_chip 类型的。所以这一行就引出了 PWM 子系统核心部件 pwm_chip，稍后的重点就是初始化 chip。

第 621~625 行，初始化 priv 的各个成员变量，第 624 和 625 还初始化了 pwm_chip 的 of_xlate 和 of_pwm_n_cells 这两个成员变量。

第 630 行，调用 stm32_pwm_probe_breakinputs 函数来读取“st,breakinput”属性，设置 break 输入，本章例程用不到。

第 634 行，调用 stm32_pwm_detect_complementary 函数来检测是否使能 TIM1 的互补输出功能。

第 636~639 行，重点，初始化 pwm_chip 的各个成员变量，第 638 行设置 pwm_chip 的 ops 函数为 stm32pwm_ops，stm32pwm_ops 里面包含了 PWM 的具体操作，稍后重点分析。第 639 行设置 pwm_chip 的 npwm，也就是设置当前打开多少路 PWM。这里直接使用 stm32_pwm_detect_channels 函数来读取 TIM1 的 CCER 寄存器，CCER 寄存器的 CC1E(bit0)、CC2E(bit4)、CC3E(bit8)和 CC4E(bit12)这 4 个位用于开启 TIM1 的 4 通道 PWM，如果为 1 就表示对应的 PWM 通道打开。所以 stm32_pwm_detect_channels 函数就会直接读取这 4 个位来判断对应的 PWM 通道是否打开。

我们重点来看一下 stm32pwm_ops，定义如下：

示例代码 39.1.3.4 imx_pwm_ops 操作集合

```
485 static const struct pwm_ops stm32pwm_ops = {
486     .owner = THIS_MODULE,
487     .apply = stm32_pwm_apply_locked,
```

```
488     .capture = IS_ENABLED(CONFIG_DMA_ENGINE) ?
                   stm32_pwm_capture : NULL,
489 };
```

第 487 行 stm32_pwm_apply_locked 就是最终的 PWM 设置函数，我们在应用中设置的 PWM 频率和占空比最终就是由 stm32_pwm_apply_locked 函数来完成的，此函数会最终操作 STM32 相关的寄存器。

stm32_pwm_apply_locked 函数源码如下：

示例代码 39.1.3.5 stm32_pwm_apply_locked 函数

```
471 static int stm32_pwm_apply_locked(struct pwm_chip *chip,
                                       struct pwm_device *pwm,
472                                    const struct pwm_state *state)
473 {
474     struct stm32_pwm *priv = to_stm32_pwm_dev(chip);
475     int ret;
476
477     /* protect common prescaler for all active channels */
478     mutex_lock(&priv->lock);
479     ret = stm32_pwm_apply(chip, pwm, state);
480     mutex_unlock(&priv->lock);
481
482     return ret;
483 }
```

第 478 行，加互斥锁，防止竞争的产生。一次只有一个应用可以设置 PWM。

第 479 行，调用 stm32_pwm_apply 函数来设置 PWM。

stm32_pwm_apply 函数内容如下：

示例代码 39.1.3.6 stm32_pwm_apply 函数

```
443 static int stm32_pwm_apply(struct pwm_chip *chip,
                                struct pwm_device *pwm,
444                             const struct pwm_state *state)
445 {
446     bool enabled;
447     struct stm32_pwm *priv = to_stm32_pwm_dev(chip);
448     int ret;
449
450     enabled = pwm->state.enabled;
451
452     if (enabled && !state->enabled) {
453         stm32_pwm_disable(priv, pwm->hwpwm);
454         return 0;
455     }
456
457     if (state->polarity != pwm->state.polarity)
458         stm32_pwm_set_polarity(priv, pwm->hwpwm, state->polarity);
```

```
459
460     ret = stm32_pwm_config(priv, pwm->hwpwm,
461                            state->duty_cycle, state->period);
462     if (ret)
463         return ret;
464
465     if (!enabled && state->enabled)
466         ret = stm32_pwm_enable(priv, pwm->hwpwm);
467
468     return ret;
469 }
```

第 453 行，在设置 PWM 之前，先调用 stm32_pwm_disable 函数关闭 PWM。

第 458 行，调用 stm32_pwm_set_polarity 函数设置指定 PWM 通道的极性。

第 460 行，调用 stm32_pwm_config 来设置 PWM 的频率以及占空比。

第 465 行，PWM 设置完成以后调用 stm32_pwm_enable 函数使能 PWM。

stm32_pwm_config 函数内容如下：

示例代码 39.1.3.7 stm32_pwm_config 函数

```
322 static int stm32_pwm_config(struct stm32_pwm *priv, int ch,
323                              int duty_ns, int period_ns)
324 {
325     unsigned long long prd, div, dty;
326     unsigned int prescaler = 0;
327     u32 ccmr, mask, shift;
328
329     /* Period and prescaler values depends on clock rate */
330     div = (unsigned long long)clk_get_rate(priv->clk) * period_ns;
331
332     do_div(div, NSEC_PER_SEC);
333     prd = div;
334
335     while (div > priv->max_arr) {
336         prescaler++;
337         div = prd;
338         do_div(div, prescaler + 1);
339     }
340
341     prd = div;
342
343     if (prescaler > MAX_TIM_PSC)
344         return -EINVAL;
345
346     /*
347      * All channels share the same prescaler and counter so when two
```

```
348     * channels are active at the same time we can't change them
349     */
350    if (active_channels(priv) & ~(1 << ch * 4)) {
351        u32 psc, arr;
352
353        regmap_read(priv->regmap, TIM_PSC, &psc);
354        regmap_read(priv->regmap, TIM_ARR, &arr);
355
356        if ((psc != prescaler) || (arr != prd - 1))
357            return -EBUSY;
358    }
359
360    regmap_write(priv->regmap, TIM_PSC, prescaler);
361    regmap_write(priv->regmap, TIM_ARR, prd - 1);
362    regmap_update_bits(priv->regmap, TIM_CR1, TIM_CR1_ARPE,
                       TIM_CR1_ARPE);
363
364    /* Calculate the duty cycles */
365    dty = prd * duty_ns;
366    do_div(dty, period_ns);
367
368    write_ccrx(priv, ch, dty);
369
370    /* Configure output mode */
371    shift = (ch & 0x1) * CCMR_CHANNEL_SHIFT;
372    ccmr = (TIM_CCMR_PE | TIM_CCMR_M1) << shift;
373    mask = CCMR_CHANNEL_MASK << shift;
374
375    if (ch < 2)
376        regmap_update_bits(priv->regmap, TIM_CCMR1, mask, ccmr);
377    else
378        regmap_update_bits(priv->regmap, TIM_CCMR2, mask, ccmr);
379
380    regmap_update_bits(priv->regmap, TIM_BDTR, TIM_BDTR_MOE,
                       TIM_BDTR_MOE);
381
382    return 0;
383 }
```

PWM 的设置主要就是两方面：频率和占空比。第 330~362 行都是设置 PWM 频率的，函数参数 period_ns 为周期值，也就是 PWM 的频率。TIM 的 PSC 寄存器用来设置定时器分频值，当 TIM 时钟源确定以后，设置 PSC 分频值即可得到 TIM 最终的时钟频率。TIM 的 ARR 寄存器是自动加载寄存器，将 TIM 设置为向下计数器，定时器开启以后每个时钟周期，计数器减一，直到计数器减为 0。这个时候再将 ARR 里面的值加载到计数器里面，计数器就会重新开始倒计

时，如此一直重复。因此 PSC 和 ARR 这两个寄存器就决定了 PWM 的周期值。要注意！由于一个定时器有 4 通道的 PWM，而这 4 路 PWM 只能设置成同一个周期，如果想要多路周期不同的 PWM 信号，那就要使用多个不同的 TIM！

第 365~380 行，设置 PWM 的占空比，参数 duty_ns 表示占空比。一个定时器下的 4 路 PWM 可以设置不同的占空比，相当于一个定时器下的 4 路 PWM 信号，周期是觉得一样的，但是占空比可以不同。占空比的设计原理比较简单，前面我们已经知道当定时器时钟频率确定以后(PSC 分频值不变)，ARR 寄存器里面的值就决定了 PWM 周期，比如当 ARR 为 100 的时候，当 100 减为 0，那么一个周期就结束了。假如 PWM 信号默认是高电平，这个时候如果我们让计数器里面的值从 100 减少到 50 的时候，输出的波形改变一下方向，也就是从高电平变为低电平，当计数器减少到 0 的时候重新回到原来的高电平，这样 PWM 波形不就出来了！而占空比就是 50%，如果我们让计数器减少到 70 的时候改变电平方向，此时占空比不就是 30%了，这个数值专业数据就叫比较值，改变比较值就可以改变 PWM 的占空比。STM32MP157 一个定时器有 4 路 PWM 通道，每个通道都有个用来存放比较值的寄存器，因此一共有 4 个寄存器 CCR1~CCR4，这 4 个寄存器就叫做比较寄存器。所以第 365、366 行就是根据参数 duty_ns 算出对应的 CCRx(x=1~4)寄存器对应的值，然后在 368 行通过 write_ccrx 函数将相应的值写入到对应的 CCRx 寄存器里面。

第 371~378 行是设置 PWM 输出模式，通道 1 和通道 2 使用 CCMR1 寄存器，通道 3 和通道 4 使用 CCMR2 寄存器。最后的 380 行设置 BDTR 寄存器，这个寄存器是 break 和死区控制相关的，本章我们用不到。

至此，STM32MP157 的 PWM 驱动我们就分析完了。

39.2 PWM 驱动编写

39.2.1 修改设备树

PWM 驱动就不需要我们再编写了，ST 已经写好了，前面我们也已经详细的分析过这个驱动源码了。我们在实际使用的时候只需要修改设备树即可，STM32MP157 开发板上的 JP1 排针引出了 PA10 这个引脚，如图 39.2.1.1 所示：

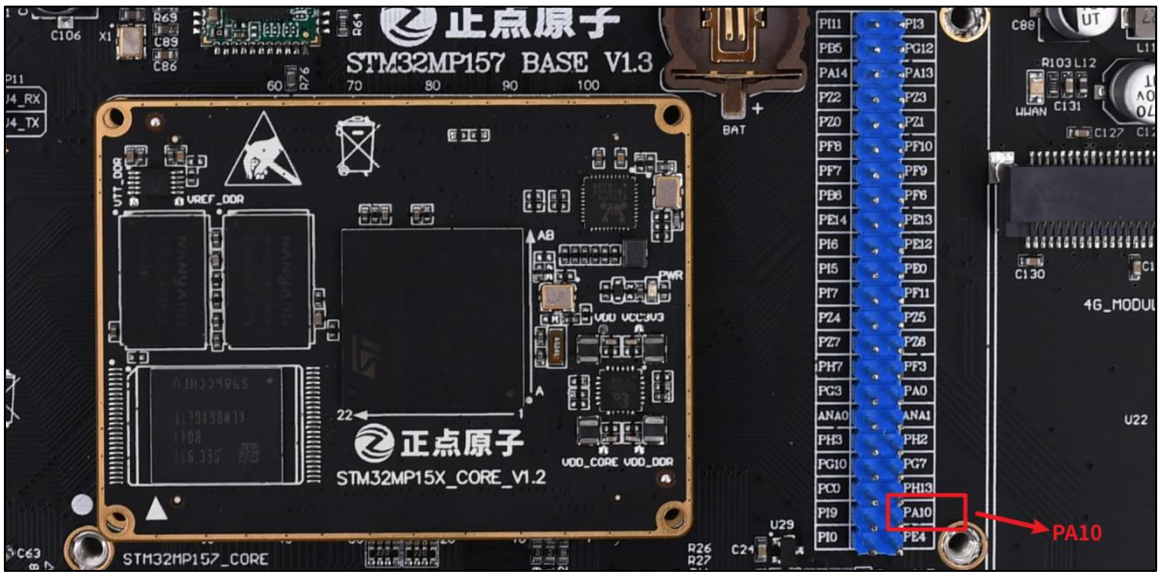


图 39.2.1.1 PA10 引脚

如果用的是 STM32MP157-Mini 开发板的话，PA10 引脚如图 39.2.1.2 所示：

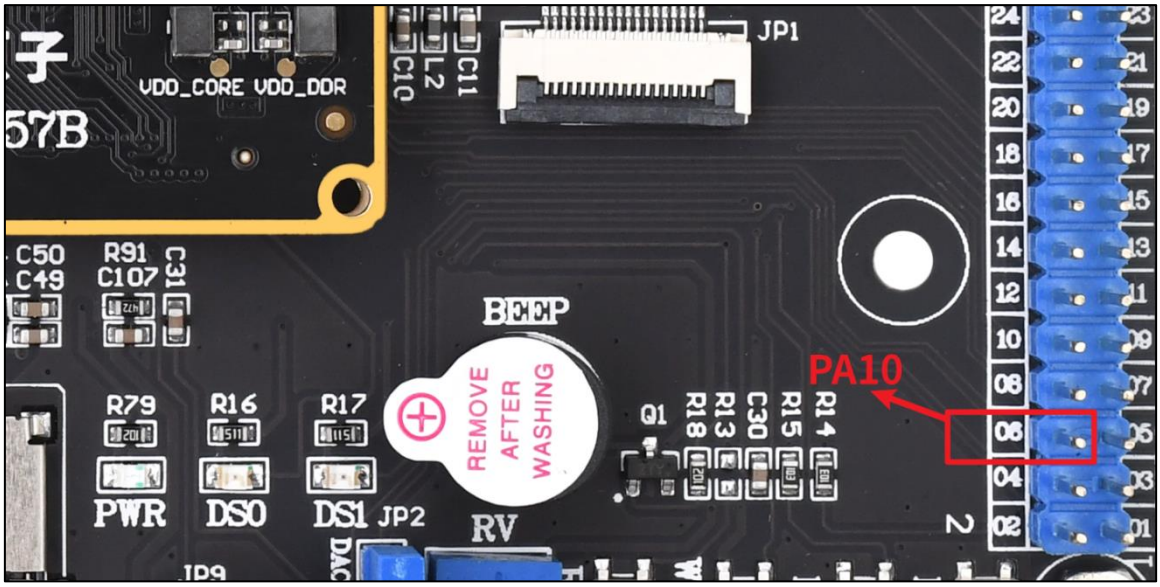


图 39.2.1.1 STM32MP157-Mini 开发板 PA10 引脚

PA10 可以作为 TIM1 的通道 3 的 PWM 输出引脚，所以我们需要在设备树里面添加 PA10 的引脚信息以及 TIM1 通道 3 的 PWM 信息。

1、添加 PA10 引脚信息

打开 stm32mp15-pinctrl.dtsi 文件，在 iomuxc 节点下添加 GPIO1_IO04 的引脚信息，如下所示：

示例代码 39.2.1.1 TIM1 PWM 引脚信息

```
1  pwm1_pins_a: pwm1-0 {
2      pins {
3          pinmux = <STM32_PINMUX('E', 9, AF1)>, /* TIM1_CH1 */
4              <STM32_PINMUX('E', 11, AF1)>, /* TIM1_CH2 */
5              <STM32_PINMUX('E', 14, AF1)>; /* TIM1_CH4 */
6          bias-pull-down;
7          drive-push-pull;
8          slew-rate = <0>;
9      };
10 };
11
12 pwm1_sleep_pins_a: pwm1-sleep-0 {
13     pins {
14         pinmux = <STM32_PINMUX('E', 9, ANALOG)>, /* TIM1_CH1 */
15             <STM32_PINMUX('E', 11, ANALOG)>, /* TIM1_CH2 */
16             <STM32_PINMUX('E', 14, ANALOG)>; /* TIM1_CH4 */
17     };
18 };
```

可以看出 ST 官方已经设置好了 TIM1 的 CH1、CH2 和 CH4 这三个通道的引脚配置，但是这里我们只需要 CH3，因此将示例代码 29.2.1.1 改成如下所示：

示例代码 39.2.1.2 PA10 引脚配置

```
1  pwm1_pins_a: pwm1-0 {
2      pins {
3          pinmux = <STM32_PINMUX('A', 10, AF1)>; /* TIM1_CH3 */
4          bias-pull-down;
5          drive-push-pull;
6          slew-rate = <0>;
7      };
8  };
9
10 pwm1_sleep_pins_a: pwm1-sleep-0 {
11     pins {
12         pinmux = <STM32_PINMUX('A', 10, ANALOG)>; /* TIM1_CH3 */
13     };
14 };
```

示例代码 39.2.1.2 中仅仅将 PA10 复用为 TIM1 的 CH3，大家一定要根据自己所使用的板子硬件来配置引脚。

2、向 timers1 节点追加信息

前面已经讲过了，stm32mp151.dtsi 文件中已经有了“timers1”节点，但是这个节点默认是 disable 的，而且还不能直接使用。需要在 stm32mp157d-atk.dts 文件中向 timers1 节点追加一些内容，在 stm32mp157d-atk.dts 文件中加入如下所示内容：

示例代码 39.2.1.3 向 timers1 添加的内容

```
1  &timers1 {
2   status = "okay";
3  /* spare all DMA channels since they are not needed for PWM output */
4   /delete-property/dmas;
5   /delete-property/dma-names;
6      pwm1: pwm {
7          pinctrl-0 = <&pwm1_pins_a>;
8          pinctrl-1 = <&pwm1_sleep_pins_a>;
9          pinctrl-names = "default", "sleep";
10         #pwm-cells = <2>;
11         status = "okay";
12     };
13 };
```

第 4、5 行，关闭 DMA 功能，因为 PWM 输出不需要 DMA。

第 7 行，pinctrl-0 属性指定 TIM1 的 CH3 所使用的输出引脚对应的 pinctrl 节点，这里设置为示例代码 39.2.1.2 中的 pwm1_pins_a。

3、屏蔽掉其他复用的 IO

检查一下设备树中有没有其他外设用到 PA10，如果有的话需要屏蔽掉！注意，不能只屏蔽掉 PA10 的 pinctrl 配置信息，也要搜索一下“gpioa 10”，看看有没有哪里用到，用到的话也要

屏蔽掉。

设备树修改完成以后重新编译设备树，然后使用新的设备树启动系统。

39.2.2 使能 PWM 驱动

ST 官方的 Linux 内核已经默认使能了 PWM 驱动，所以不需要我们修改，但是为了学习，我们还是需要知道怎么使能。打开 Linux 内核配置界面，按照如下路径找到配置项：

```
-> Device Drivers
    -> Pulse-Width Modulation (PWM) Support
        -> <*> STMicroelectronics STM32 PWM          //选中
```

配置如图 39.2.2.1 所示：

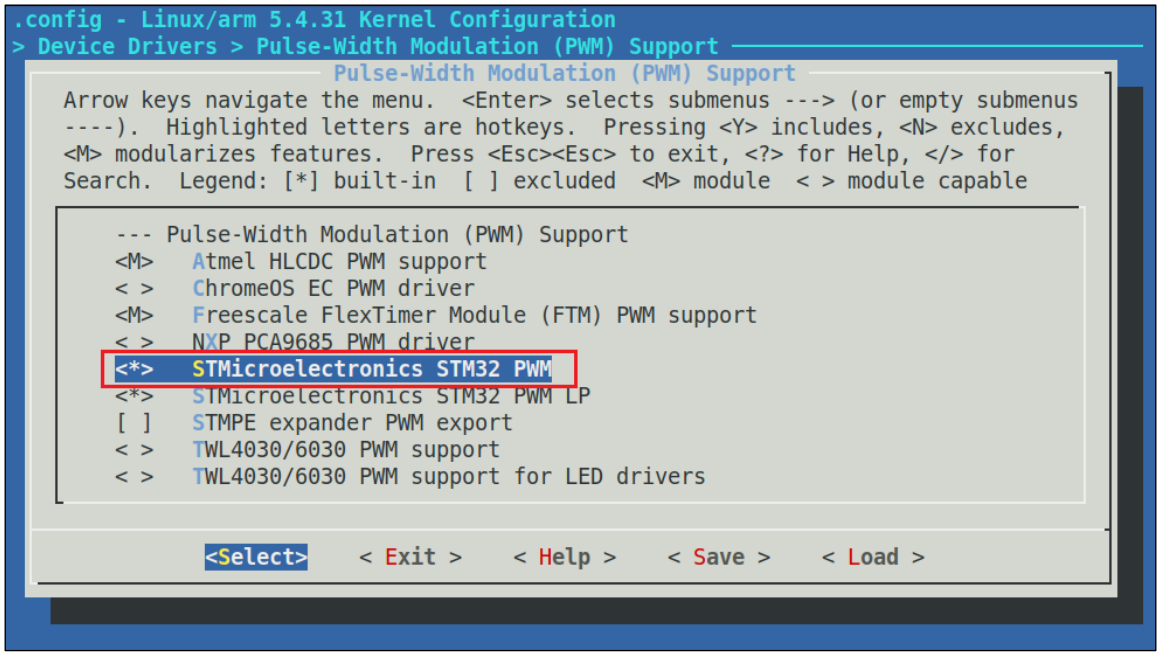


图 39.2.2.1 PWM 配置项

39.3 PWM 驱动测试

1、确定 TIM1 对应的 pwmchipX 文件

使用新的设备树启动系统，然后将开发板上的 PA10 引脚连接到示波器上，通过示波器来查看 PWM 波形图。我们可以直接在用户层来配置 PWM，进入目录/sys/class/pwm 中，如图 39.3.1 所示：

```
[root@ATK-stm32mp1]:/sys/class/pwm$ ls
pwmchip0
[root@ATK-stm32mp1]:/sys/class/pwm$
```

图 39.3.1 TIM1 对应的 PWM

注意！图 39.3.1 中有个 pwmchip0，但是我们并不知道这个 pwmchip0 是否为 TIM1 对应的文件。我们可以通过查看 pwmchip0 对应的地址是否和 TIM1 定时器寄存器起始地址是否一致来确定其是否属于 TIM1。输入如下命令进入到 pwmchip0 目录：

```
cd pwmchip0    //进入
```

进入到 pwmchip0 目录以后会打印出其路劲，如图 39.3.2 所示：

```
[root@ATK-stm32mp1]:/sys/class/pwm$ cd pwmchip0/
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44000000.timer/44000000.timer:pwm/pwm/pwmchip0$
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44000000.timer/44000000.timer:pwm/pwm/pwmchip0$
```

图 39.3.2 pwmchip0 路径名称

从图 39.3.2 可以看出 pwmchip0 对应的定时器寄存器起始地址为 0X44000000，根据示例代码 39.1.1.1 中的 timers1 节点，可以知道 TIM1 这个定时器的寄存器起始地址就是 0X44000000。因此，pwmchip0 就是 TIM1 对应的文件。

为什么要用这么复杂的方式来确定定时器对应的 pwmchip 文件呢？因为当 STM32MP157 开启多个定时器的 PWM 功能以后，其 pwmchip 文件就会变！ 这个在下一章讲解 LCD 的背光调节实验中就会体现出来，LCD 背光使用的 TIM4_CH2，当同时开始 TIM1 和 TIM4 的 PWM 功能以后，此时 pwmchip0 就不是 TIM1 了！而是 TIM4。按照我们的惯性思维，肯定会一直操作 pwmchip0 来控制 TIM1_CH3 这路 PWM，但是会发现一直提示设备忙，无法操作，这个笔者在做实验的时候被困了很久！

2、调出 pwmchip0 的 pwm2 子目录

pwmchip0 是整个 TIM1 的总目录，而 TIM1 有 4 路 PWM，每路都可以独立打开或关闭。CH1~CH4 对应的编号为 0~3，因此打开 TIM1 的 CH3 输入如下命令：

```
echo 2 > /sys/class/pwm/pwmchip0/export
```

上述命令中 2 就是 TIM1_CH3，如果要打开 TIM1 的 CH1，那就是 0。执行完成会在 pwmchip0 目录下生成一个名为“pwm2”的子目录，如图 39.3.3 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44000000.timer/44000000.timer:pw
device      npwm        pwm2        uevent      通道3，后续操作要用到
export      power       subsystem   unexport
```

图 39.3.3 新生成的 pwm2 子目录

2、设置 PWM 的频率

注意，这里设置的是周期值，单位为 ns，比如 20KHz 频率的周期就是 50000ns，输入如下命令：

```
echo 50000 > /sys/class/pwm/pwmchip0/pwm2/period
```

3、设置 PWM 的占空比

这里不能直接设置占空比，而是设置的一个周期的 ON 时间，也就是高电平时间，比如 20KHz 频率下 20%占空比的 ON 时间就是 10000，输入如下命令：

```
echo 10000 > /sys/class/pwm/pwmchip0/pwm2/duty_cycle
```

4、使能 TIM1 的通道 3

一定要先设置频率和波特率，最后在开启定时器，否则会提示参数错误！输入如下命令使能 TIM1 的通道 3 这路 PWM：

```
echo 1 > /sys/class/pwm/pwmchip0/pwm2/enable
```

设置完成使用示波器查看波形是否正确，正确的话如图 39.3.4 所示：

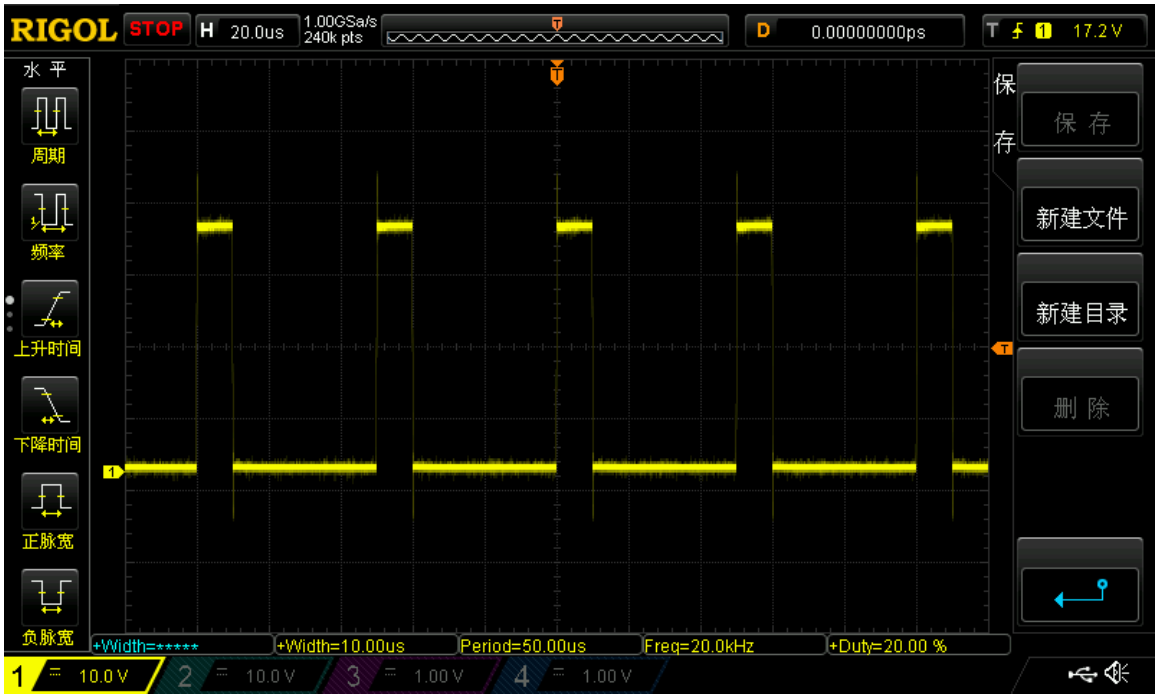


图 39.3.4 PWM 波形图

从图 39.3.4 可以看出，此时 PWM 频率为 20KHz，占空比为 20%，与我们设置的一致。如果要修改频率或者占空比的话一定要注意这两者时间值，比如 20KHz 频率的周期值为 50000ns，那么你在调整占空比的时候 ON 时间就不能设置大于 50000，否则就会提示你参数无效。

5、极性反转

前面我们也可以修改 PWM 的极性，上面我们设置的 PWM 占空比为 20%，我们只需要修改极性就可以将占空比变为 80%。向/pwmchip0/pwm2/polarity 文件写入“inversed”即可反转极性，命令如下：

```
echo "inversed" > /sys/class/pwm/pwmchip0/pwm2/polarity
```

极性反转以后占空比就变为了 80%，如果要恢复回原来的极性，向/pwmchip0/pwm2/polarity 文件写入“normal”即可，命令如下：

```
echo "normal" > /sys/class/pwm/pwmchip0/pwm2/polarity
```

第四十章 Linux LCD 驱动实验

LCD 是很常用的一个外，通过 LCD 可以显示绚丽的图片、界面等，提交人机交互的效率。STM32MP1 提供了一个 LTDC 接口用于连接 RGB 接口的液晶屏。本章我们就来学校一下如何在 Linux 下驱动 LCD 屏。

40.1 LCD 和 LTDC 简介

40.1.1 LCD 简介

LCD 全称是 Liquid Crystal Display，也就是液晶显示器，是现在最常用到的显示器。手机、电脑、各种人机交互设备等基本都用到了 LCD，最常见就是手机和电脑显示器了。由于笔者不是 LCD 从业人员，对于 LCD 的具体原理不了解，网上对于 LCD 的原理解释如下：

LCD 的构造是在两片平行的玻璃基板当中放置液晶盒，下基板玻璃上设置 TFT（薄膜晶体管），上基板玻璃上设置彩色滤光片，通过 TFT 上的信号与电压改变来控制液晶分子的转动方向，从而达到控制每个像素点偏振光出射与否而达到显示目的。

我们现在要在 STM32MP1 开发板上使用 LCD，所以不需要去研究 LCD 的具体实现原理，我们只需要从使用的角度去关注 LCD 的几个重要点：

1、分辨率

提起 LCD 显示器，我们都会听到 720P、1080P、2K 或 4K 这样的字眼，这个就是 LCD 显示器分辨率。LCD 显示器都是由一个个的像素点组成，像素点就类似一个灯(在 OLED 显示器中，像素点就是一个小灯)，这个小灯是 RGB 灯，也就是由 R(红色)、G(绿色)和 B(蓝色)这三种颜色组成的，而 RGB 就是光的三原色。1080P 的意思就是 LCD 屏幕上的像素数量是 1920*1080 个，也就是这个屏幕一列 1080 个像素点，一共 1920 列，如图 40.1.1.1 所示：

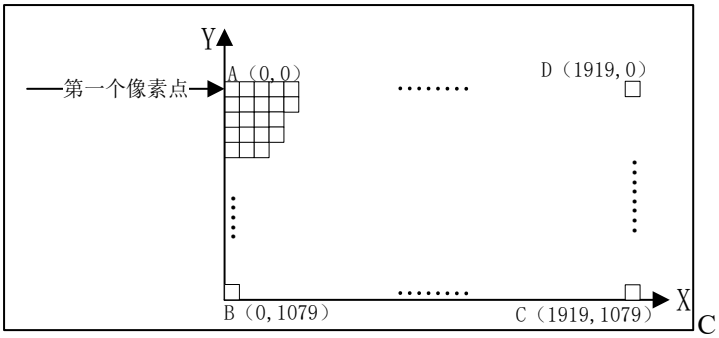


图 40.1.1.1 LCD 像素点排布

图 40.1.1.1 就是 1080P 显示器的像素示意图，X 轴就是 LCD 显示器的横轴，Y 轴就是显示器的竖轴。图中的小方块就是像素点，一共有 1920*1080=2073600 个像素点。左上角的 A 点是第一个像素点，右下角的 C 点就是最后一个像素点。2K 就是 2560*1440 个像素点，4K 是 3840*2160 个像素点。很明显，在 LCD 尺寸不变的情况下，分辨率越高越清晰。同样的，分辨率不变的情况下，LCD 尺寸越小越清晰。比如我们常用的 24 寸显示器基本都是 1080P 的，而我们现在使用的 5 寸的手机基本也是 1080P 的，但是手机显示细腻程度就要比 24 寸的显示器要好很多！

由此可见，LCD 显示器的分辨率是一个很重要的参数，但是并不是分辨率越高的 LCD 就越好。衡量一款 LCD 的好坏，分辨率只是其中的一个参数，还有色彩还原程度、色彩偏离、亮度、可视角度、屏幕刷新率等其他参数。

2、像素格式

上面讲了，一个像素点就相当于一个 RGB 小灯，通过控制 R、G、B 这三种颜色的亮度就可以显示出各种各样的色彩。那该如何控制 R、G、B 这三种颜色的显示亮度呢？一般一个像素点的 R、G、B 这三部分分别使用 8bit 的数据，那么一个像素点就是 8bit*3=24bit，也就是说一

个像素点 3 个字节，这种像素格式称为 RGB888。如果再加入 8bit 的 Alpha(透明)通道的话一个像素点就是 32bit，也就是 4 个字节，这种像素格式称为 ARGB8888。如果学习过 STM32 的话应该还听过 RGB565 这种像素格式，在本章实验中我们使用 ARGB8888 这种像素格式，一个像素占用 4 个字节的内存，这四个字节每个位的分配如图 40.1.1.2 所示：

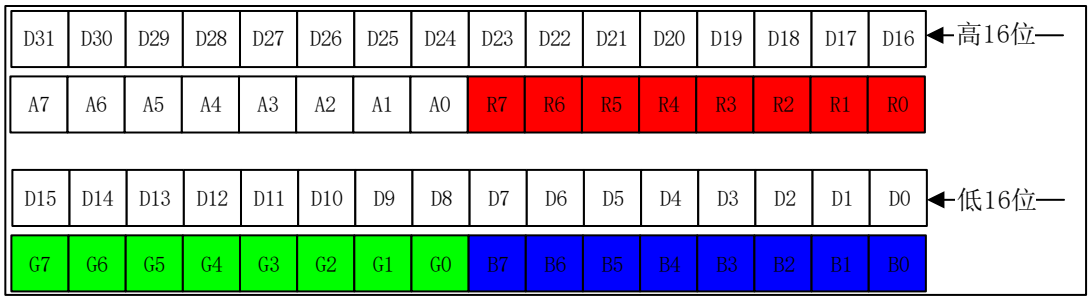


图 40.1.1.2 ARGB8888 数据格式

在图 40.1.1.2 中，一个像素点是 4 个字节，其中 bit31~bit24 是 Alpha 通道，bit23~bit16 是 RED 通道，bit15~bit14 是 GREEN 通道，bit7~bit0 是 BLUE 通道。所以红色对应的值就是 0X00FF0000，蓝色对应的值就是 0X000000FF，绿色对应的值为 0X0000FF00。通过调节 R、G、B 的比例可以产生其它的颜色，比如 0X00FFFF00 就是黄色，0X00000000 就是黑色，0X00FFFFFF 就是白色。大家可以打开电脑的“画图”工具，在里面使用调色板即可获取到想要的颜色对应的数值，如图 40.1.1.3 所示：

图 40.1.1.3 颜色选取

3、LCD 屏幕接口

LCD 屏幕或者说显示器有很多种接口，比如在显示器上常见的 VGA、HDMI、DP 等等，但是 STM32MP1 开发板不支持这些接口。STM32MP1 支持 RGB 接口的 LCD，RGBLCD 接口的信号线如表 40.1.1.1 所示：

信号线	描述
R[7:0]	8 根红色数据线。
G[7:0]	8 根绿色数据线。

B[7:0]	8 根蓝色数据线。
DE	数据使能线。
VSYNC	垂直同步信号线。
HSYNC	水平同步信号线。
PCLK	像素时钟信号线。

表 40.1.1.1 RGB 数据线

表 40.1.1.1 就是 RGBLCD 的信号线，R[7:0]、G[7:0]和 B[7:0]这 24 根是数据线，DE、VSYNC、HSYNC 和 PCLK 这四根是控制信号线。RGB LCD 一般有两种驱动模式：DE 模式和 HV 模式，这两个模式的区别是 DE 模式需要用到 DE 信号线，而 HV 模式不需要用到 DE 信号线，在 DE 模式下是可以不需要 HSYNC 信号线的，即使不接 HSYNC 信号线 LCD 也可以正常工作。

ALIENTEK 一共有四款 RGB LCD 屏幕，型号分别为：ATK-4342(4.3 寸，480*272)、ATK-4842(4.3 寸，800*480)、ATK-7084(7 寸，800*480)和 ATK-7016(7 寸，1024*600)，本教程就以 ATK-7016 这款屏幕为例讲解，ATK-7016 的屏幕接口原理图如图 40.1.1.4 所示：

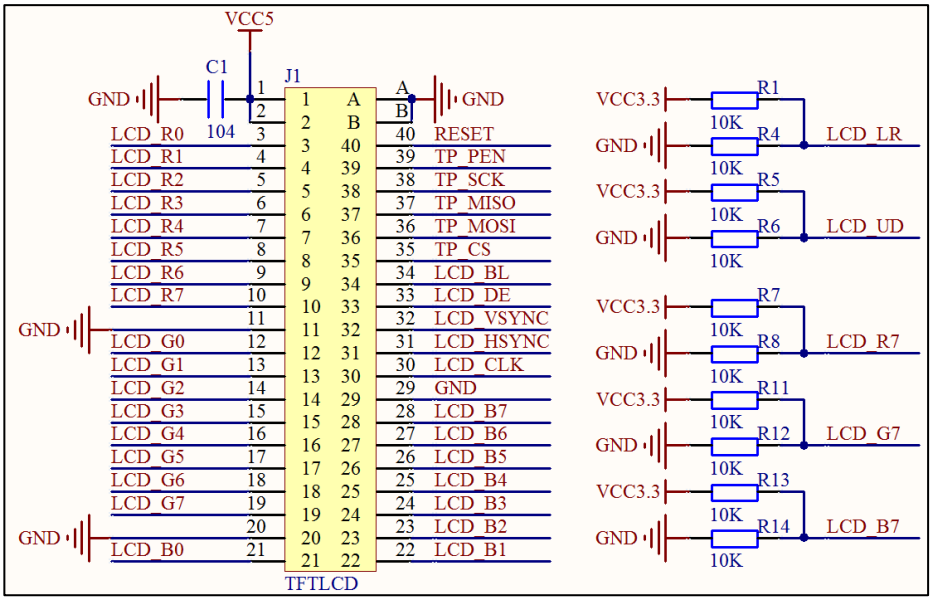


图 40.1.1.4 RGB LCD 液晶屏屏幕接口

图中 J1 就是对外接口，是一个 40PIN 的 FPC 座（0.5mm 间距），通过 FPC 线，可以连接到 STM32MP1 开发板上面。该接口十分完善，采用 RGB888 格式，并支持 DE&HV 模式，还支持触摸屏和背光控制。右侧的几个电阻，并不是都焊接的，用户可以根据自己时间需要而选择是否焊接(正点原子出厂屏幕不能做修改!)。默认情况，R1 和 R6 焊接，设置 LCD_LR 和 LCD_UD，控制 LCD 的扫描方向，是从左到右，从上到下（横屏看）。而 LCD_R7/G7/B7 则用来设置 LCD 的 ID，由于 RGBLCD 没有读写寄存器，也就没有所谓的 ID，这里我们通过在模块上面，控制 R7/G7/B7 的上/下拉，来自定义 LCD 模块的 ID，帮助 SOC 判断当前 LCD 面板的分辨率和相关参数，以提高程序兼容性。这几个位的设置关系如表 40.1.1.2 所示：

M2 LCD_G 7	M1 LCD_ G7	M0 LCD_ R7	LCD ID	说明
0	0	0	4342	ATK-4342 RGBLCD 模块，分辨率：480*272
0	0	1	7084	ATK-7084 RGBLCD 模块，分辨率：800*480

0	1	0	7016	ATK-7016，RGBLCD 模块，分辨率：1024*600
1	0	0	4384	ATK-4384，RGBLCD 模块，分辨率：800*480
X	X	X	NC	暂时未用到

表 40.1.1.2 ALIENTEK RGBLCD 模块 ID 对应关系

ATK-7016 模块，就设置 M2:M0=010 即可。这样，我们在程序里面，读取 LCD_R7/G7/B7，得到 M0:M2 的值，从而判断 RGBLCD 模块的型号，并执行不同的配置，即可实现不同 LCD 模块的兼容。

4、LCD 时间参数

如果将 LCD 显示一帧图像的过程想象成绘画，那么在显示的过程中就是用一根“笔”在不同的像素点画上不同的颜色。这根笔按照从左至右、从上到下的顺序扫描每个像素点，并且在像素画上对应的颜色，当画到最后一个像素点的时候一幅图像就绘制好了。假如一个 LCD 的分辨率为 1024*600，那么其扫描如图 40.1.1.5 所示：

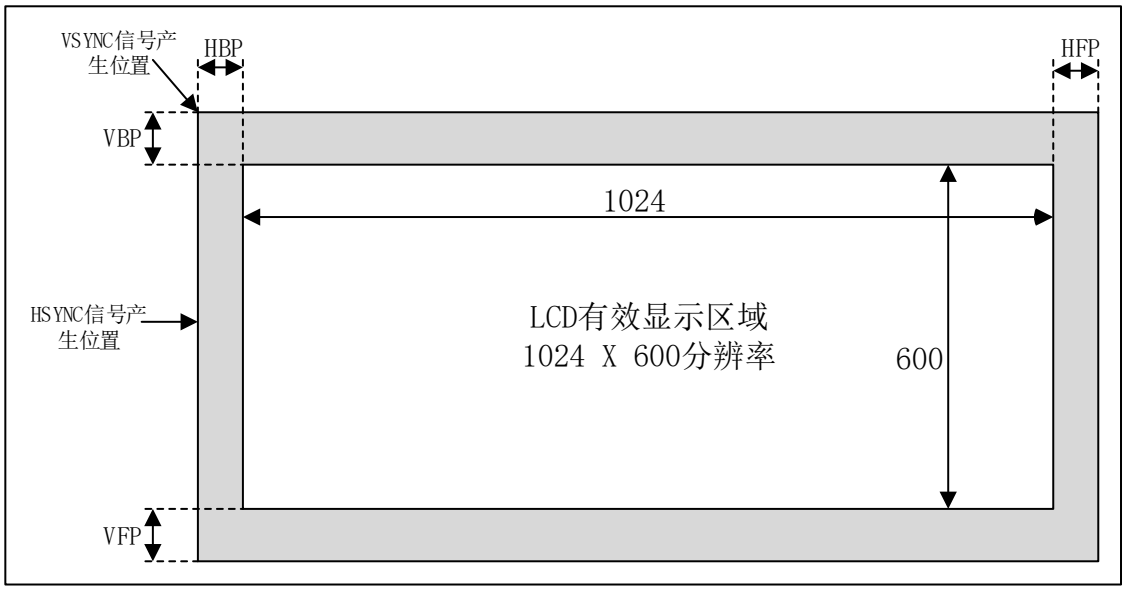


图 40.1.1.5 LCD 一帧图像扫描图

结合图 40.1.1.4，我们来看一下 LCD 是怎么扫描显示一帧图像的。一帧图像也是由一行一行组成的。HSYNC 是水平同步信号，也叫做行同步信号，当产生此信号的话就表示开始显示新的一行了，所以此信号都是在图 40.1.1.5 的最左边。VSYNC 信号是垂直同步信号，也叫做帧同步信号，当产生此信号的话就表示开始显示新的一帧图像了，所以此信号在图 40.1.1.4 的左上角。

在图 40.1.1.5 可以看到有一圈“黑边”，真正有效的显示区域是中间的白色部分。那这一圈“黑边”是什么东西呢？这就要从显示器的“祖先”CRT 显示器开始说起了，CRT 显示器就是以前很常见的那种大屁股显示器，在 2019 年应该很少见了，如果在农村应该还是可以见到的。CRT 显示器屁股后面是个电子枪，这个电子枪就是我们上面说的“画笔”，电子枪打出的电子撞击到屏幕上的荧光物质使其发光。只要控制电子枪从左到右扫完一行(也就是扫描一行)，然后从上到下扫描完所有行，这样一帧图像就显示出来了。也就是说，显示一帧图像电子枪是按照‘Z’形在运动，当扫描速度很快的时候看起来就是一幅完成的画面了。

当显示完一行以后会发出 HSYNC 信号，此时电子枪就会关闭，然后迅速的移动到屏幕的

左边，当 HSYNC 信号结束以后就可以显示新的一行数据了，电子枪就会重新打开。在 HSYNC 信号结束到电子枪重新打开之间会插入一段延时，这段延时就图 40.1.1.5 中的 HBP。当显示完一行以后就会关闭电子枪等待 HSYNC 信号产生，关闭电子枪到 HSYNC 信号产生之间会插入一段延时，这段延时就是图 40.1.1.5 中的 HFP 信号。同理，当显示完一帧图像以后电子枪也会关闭，然后等到 VSYNC 信号产生，期间也会加入一段延时，这段延时就是图 40.1.1.5 中的 VFP。VSYNC 信号产生，电子枪移动到左上角，当 VSYNC 信号结束以后电子枪重新打开，中间也会加入一段延时，这段延时就是图 40.1.1.5 中的 VBP。

HBP、HFP、VBP 和 VFP 就是导致图 40.1.1.5 中黑边的原因，但是这是 CRT 显示器存在黑边的原因，现在是 LCD 显示器，不需要电子枪了，那么为何还会有黑边呢？这是因为 RGB LCD 屏幕内部是有一个 IC 的，发送一行或者一帧数据给 IC，IC 是需要反应时间的。通过这段反应时间可以让 IC 识别到一行数据扫描完了，要换行了，或者一帧图像扫描完了，要开始下一帧图像显示了。因此，在 LCD 屏幕中继续存在 HBP、HFP、VPB 和 VFP 这四个参数的主要目的是为了锁定有效的像素数据。这四个时间是 LCD 重要的时间参数，后面编写 LCD 驱动的时候要用到的，至于这四个时间参数具体值是多少，那要需要去查看所使用的 LCD 数据手册了。

5、RGB LCD 屏幕时序

上面讲了行显示和帧显示，我们来看一下行显示对应的时序图，如图 40.1.1.6 所示：

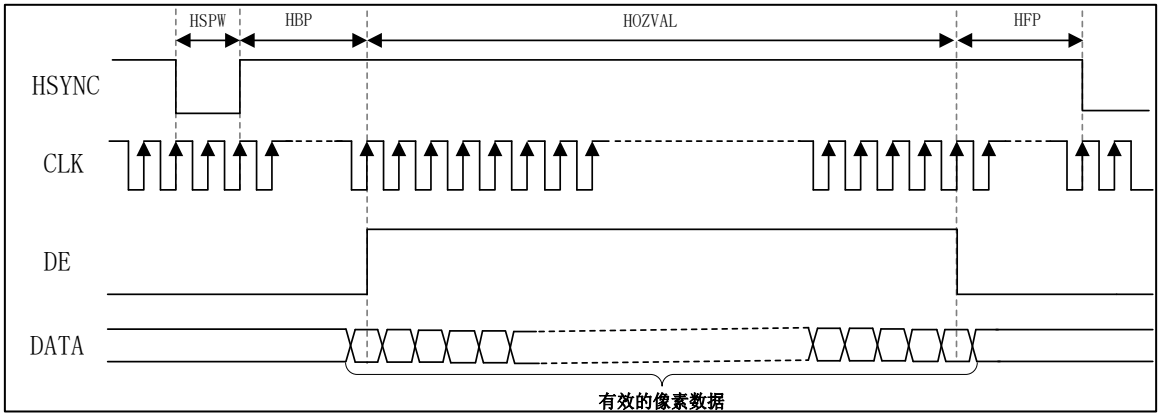


图 40.1.1.6 行显示时序

图 40.1.1.6 就是 RGB LCD 的行显示时序，我们来分析一下其中重要几个的参数：

HSYNC：行同步信号，当此信号有效的话就表示开始显示新的一行数据，查阅所使用的 LCD 数据手册可以知道此信号是低电平有效还是高电平有效，假设此时是低电平有效。

HSPW：有些地方也叫做 thp，是 HSYNC 信号宽度，也就是 HSYNC 信号持续时间。HSYNC 信号不是一个脉冲，而是需要持续一段时间才是有效的，单位为 CLK。

HBP：有些地方叫做 thb，前面已经讲过了，术语叫做行同步信号后肩，单位是 CLK。

HOZVAL：有些地方叫做 thd，显示一行数据所需的时间，假如屏幕分辨率为 1024*600，那么 HOZVAL 就是 1024，单位为 CLK。

HFP：有些地方叫做 thf，前面已经讲过了，术语叫做行同步信号前肩，单位是 CLK。

当 HSYNC 信号发出以后，需要等待 HSPW+HBP 个 CLK 时间才会接收到真正有效的像素数据。当显示完一行数据以后需要等待 HFP 个 CLK 时间才能发出下一个 HSYNC 信号，所以显示一行所需要的时间就是：HSPW + HBP + HOZVAL + HFP。

一帧图像就是由很多个行组成的，RGB LCD 的帧显示时序如图 40.1.1.7 所示：

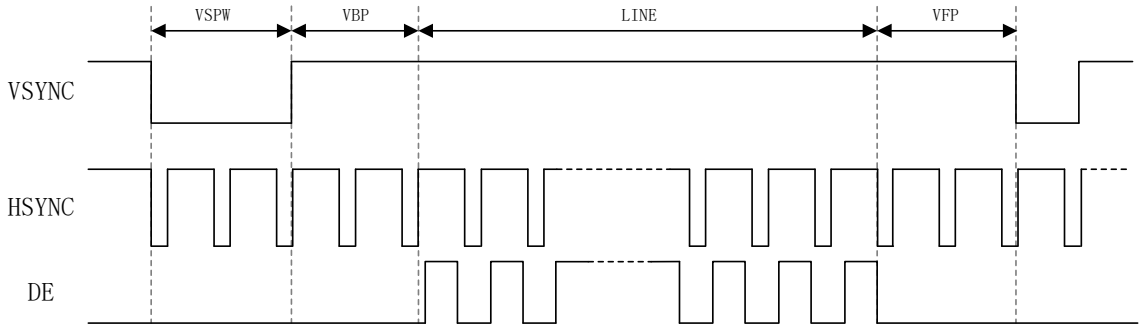


图 40.1.1.7 帧显示时序图

图 40.1.1.7 就是 RGB LCD 的帧显示时序，我们来分析一下其中重要的几个参数：

VSYNC：帧同步信号，当此信号有效的话就表示开始显示新的一帧数据，查阅所使用的 LCD 数据手册可以知道此信号是低电平有效还是高电平有效，假设此时是低电平有效。

VSPW：些地方也叫做 tvp，是 VSYNC 信号宽度，也就是 VSYNC 信号持续时间，单位为 1 行的时间。

VBP：有些地方叫做 tvb，前面已经讲过了，术语叫做帧同步信号后肩，单位为 1 行的时间。

LINE：有些地方叫做 tvd，显示一帧有效数据所需的时间，假如屏幕分辨率为 1024*600，那么 LINE 就是 600 行的时间。

VFP：有些地方叫做 tvf，前面已经讲过了，术语叫做帧同步信号前肩，单位为 1 行的时间。

显示一帧所需要的时间就是：VSPW+VBP+LINE+VFP 个行时间，最终的计算公式：

T = (VSPW+VBP+LINE+VFP) * (HSPW + HBP + HOZVAL + HFP)

因此我们在配置一款 RGB LCD 的时候需要知道这几个参数：HOZVAL(屏幕有效宽度)、LINE(屏幕有效高度)、HBP、HSPW、HFP、VSPW、VBP 和 VFP。ALIENTEK 三款 RGB LCD 屏幕的参数如表 40.1.1.3 所示：

屏幕型号	参数	值	单位
ATK4342	水平显示区域	480	tCLK
	HSPW(thp)	1	tCLK
	HBP(thb)	40	tCLK
	HFP(thf)	5	tCLK
	垂直显示区域	272	th
	VSPW(tvp)	1	th
	VBP(tvb)	8	th
	VFP(tvf)	8	th
	像素时钟	9	MHz
ATK4384	水平显示区域	800	tCLK
	HSPW(thp)	48	tCLK
	HBP(thb)	88	tCLK
	HFP(thf)	40	tCLK
	垂直显示区域	480	th
	VSPW(tvp)	3	th
	VBP(tvb)	32	th
	VFP(tvf)	13	th

	像素时钟	31	MHz
ATK7084	水平显示区域	800	tCLK
	HSPW(thp)	1	tCLK
	HBP(thb)	46	tCLK
	HFP(thf)	210	tCLK
	垂直显示区域	480	th
	VSPW(tvp)	1	th
	VBP(tvb)	23	th
	VFP(tvf)	22	th
	像素时钟	33.3	MHz
ATK7016	水平显示区域	1024	tCLK
	HSPW(thp)	20	tCLK
	HBP(thb)	140	tCLK
	HFP(thf)	160	tCLK
	垂直显示区域	600	th
	VSPW(tvp)	3	th
	VBP(tvb)	20	th
	VFP(tvf)	12	th
	像素时钟	51.2	MHz

表 40.1.1.3 RGB LCD 屏幕时间参数

6、像素时钟

像素时钟就是 RGB LCD 的时钟信号，以 ATK7016 这款屏幕为例，显示一帧图像所需要的时钟数就是：

= (VSPW+VBP+LINE+VFP) * (HSPW + HBP + HOZVAL + HFP)

= (3 + 20 + 600 + 12) * (20 + 140 + 1024 + 160)

= 635 * 1344

= 853440。

显示一帧图像需要 853440 个时钟数，那么显示 60 帧就是：853440 * 60 = 51206400≈51.2M，所以像素时钟就是 51.2MHz。

7、显存

在讲像素格式的时候就已经说过了，如果采用 ARGB8888 格式的话一个像素需要 4 个字节的内存来存放像素数据，那么 1024*600 分辨率就需要 1024*600*4=2457600B≈2.4MB 内存。但是 RGB LCD 内部是没有内存的，所以就需要在开发板上的 DDR3 中分出一段内存作为 RGB LCD 屏幕的显存，我们如果要在屏幕上显示什么图像的话直接操作这部分显存即可。

40.1.2 LTDC 接口

LTDC 是 STM32MP1 自带的液晶屏幕接口，用于连接 RGB LCD 接口的屏幕，LTDC 接口特性如下：

①、24 位 RGB 并行像素输出；每像素 8 位(RGB888)

②、2 个带有专用 FIFO 的显示层(FIFO 深度 64x64 位)

③、查色表(CLUT)，每个图层最高 256 种颜色(256x24)位

④、可针对不同显示面板编程时序

⑤、每层有多达 8 个输入颜色格式可供选择，分别为:ARGB8888、RGB888、RGB565、ARGB1555、ARGB4444、L8、AL44、AL88。

图 40.1.2.1 为 LTDC 功能框架图：

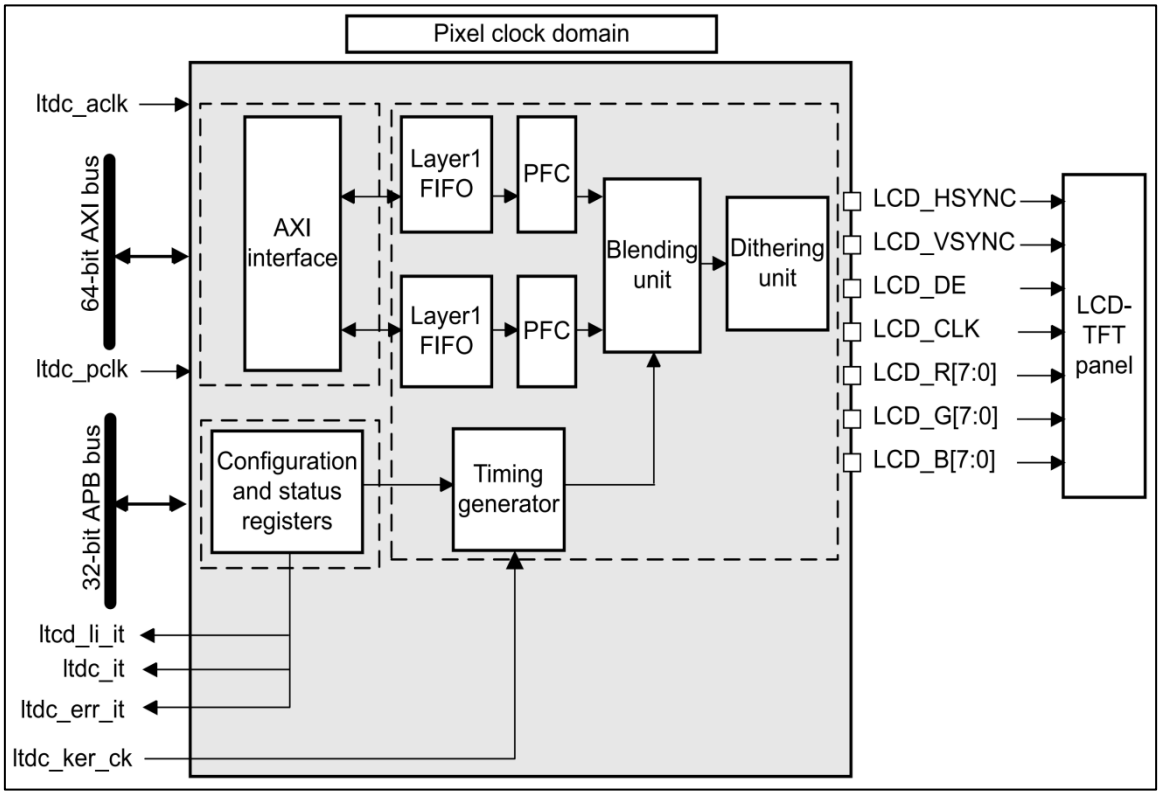


图 40.1.2.1 LTDC 功能框架图

从图可以看出 LTDC 的信号可以分为两类：4 个控制信号(LCD_CLK 像素时钟、LCD_HSYNC 水平同步、LCD_VSYNC 垂直同步、LCD_DE 数据有效)和 3 个 RGB 数据信号(8bit x 3)。

40.2 DRM 驱动框架

40.2.1 DRM 简介

在 Linux 系统中，主流的显示框架有两种：DRM(Direct Rendering Module)框架和 FB(FrameBuffer)框架。FB 框架不能处理基于 3D 加速 GPU 显卡，DRM 是可以统一管理 GPU 显示，所以 DRM 相对于 FB 更能适应新的显示硬件。比如 DRM 支持多层合成、支持 VSYNC、支持 DMA-BUF、支持 fence 机制等等。

下图就是一个 DRM 驱动框架包括两部分：DRM core 和 DRM driver。DRM core 提供了一个基本的 DRM 框架，DRM driver 就可以注册进 DRM 框架，同时为用户空间提供一组 ioctl。libdrm 对底层接口(DRM driver 提供的 ioctl)进行封装，向上层提供统一的 API 接口。DRM driver 包含了 GEM 模块和 KMS 模块，这两模块也分为好几个小模块。

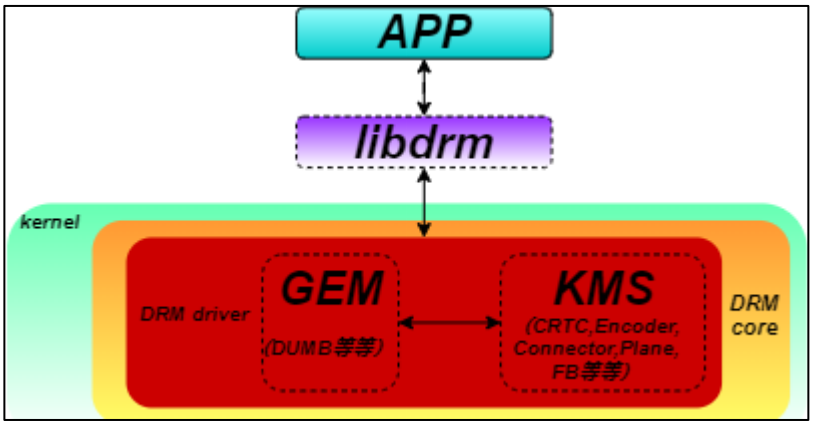


图 40.1.2.1 DRM 驱动框架

先介绍图 40.1.2.1 中的部分名词意思和作用：

图形执行管理器(GEM)：全称 Graphics Execution Manager，这是一个内存管理器，主要负责内存的分配和释放，可以调用 GPU。

DUMB：这是一个 dumb 缓冲区，主要负责一些简单的 buffer 显示，可以通过 CPU 直接渲染 dumb，GPU 不会使用 dumb。

内核显示模式设置(KMS)：全称 Kernel Mode Setting，主要负责显示的控制，包括屏幕分辨率、屏幕刷新率和颜色深度等等。

CRTC：就是指显示控制器，在 DRM 里有多个显存，就可以通过操作 CRTC 来控制要显示那个显存。

Encoder：负责从 CRTC 里输出的 timing 时序转换成外部设备所需要的信号的模块，同时也负责控制 LCD 的显示。

Connector：连接物理显示设备的连接器，比如 DSI、HDMI 等等。

Plane：负责获取显存，在输出到 CRTC 里，说明 CRTC 必须要有一个 Plane。

帧缓冲(FB)：能够显示图层的 buffer。

接着我们看下 DRM driver 里的两大模块 GEM 和 KMS 是如何连接显示器的，如下图所示：

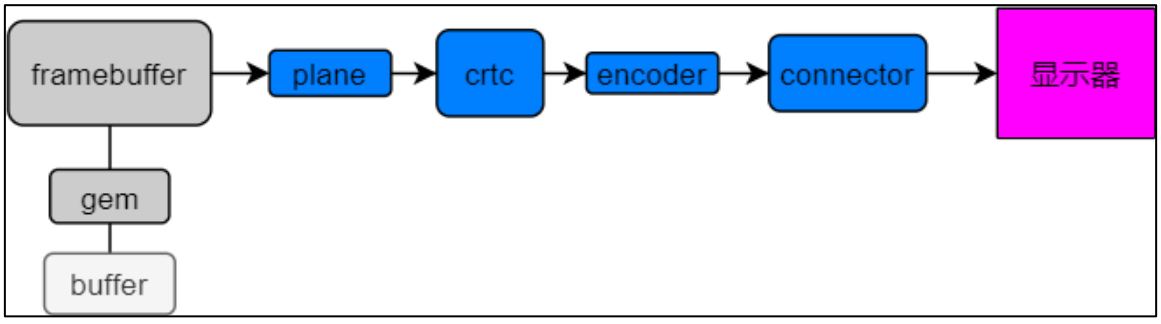


图 40.1.2.2 DRM driver 模块关系图

在图 40.1.2.2 里，蓝色框表示 KMS 里的模块。plane 是连接 crtc 和 framebuffer 的纽带，而 encoder 是连接 crtc 和 connector 的纽带。GEM 是负责和物理的 buffer 打交道不是 framebuffer。plane 把获取到显存输出到 crtc 里，crtc 通过 connector 接口输出到显示器。

40.2.2 ST 官方的 DRM 驱动框架介绍

在 Linux 系统中，DRM 驱动的核心主要就一个 drm_driver 结构体，驱动程序要初始化 drm_driver 结构体，然后调用 drm_dev_init 函数，将其注册到 DRM core。

我们简单分析 ST 官方编写的在 Linux 下的 DRM 驱动,打开 stm32mp151.dtsi,然后找到 ltdc 节点内容，如下所示：

示例代码 40.2.2.1 ltdc 节点

```
1652    ltdc: display-controller@5a001000 {
1653        compatible = "st,stm32-ltdc";
1654        reg = <0x5a001000 0x400>;
1655        interrupts = <GIC_SPI 88 IRQ_TYPE_LEVEL_HIGH>,
1656                <GIC_SPI 89 IRQ_TYPE_LEVEL_HIGH>;
1657        clocks = <&rcc LTDC_PX>;
1658        clock-names = "lcd";
1659        resets = <&rcc LTDC_R>;
1660        status = "disabled";
1661    };
```

示例代码 40.2.2.1 中的 ltdc 节点信息是所有使用 STM32MP1 芯片的板子所共有的，并不是完整的 ltdc 节点信息。可以看出 ltdc 节点的 compatible 属性值为“st,stm32-ltdc”，因此在 Linux 源码中搜索这个字符串就可以找到 STM32MP1 的 DRM 驱动文件，这个文件为“drivers/gpu/drm/stm/drv.c”文件，打开此文件找的如下内容所示：

示例代码 40.2.2.2 platform 下的 DRM 驱动

```
236 static const struct of_device_id drv_dt_ids[] = {
237     { .compatible = "st,stm32-ltdc"},
238     { /* end node */ },
239 };
240 MODULE_DEVICE_TABLE(of, drv_dt_ids);
241
242 static struct platform_driver stm_drm_platform_driver = {
243     .probe = stm_drm_platform_probe,
244     .remove = stm_drm_platform_remove,
245     .driver = {
246         .name = "stm32-display",
247         .of_match_table = drv_dt_ids,
248         .pm = &drv_pm_ops,
249     },
250 };
```

从示例代码 40.2.2.2 可以看出，这是一个标准的 platform 驱动，当驱动和设备匹配以后 stm_drm_platform_probe 函数就会执行。

和其他设备驱动一样，DRM 也分为 DRM 设备和 DRM 驱动，drm_device 结构体为 DRM 设备，drm_driver 为 DRM 驱动，我们依次来看一下这两个结构体。

1、drm_device 结构体

drm_device 结构体定义在 include/drm/drm_device.h 文件里，内容如下(有省略)：

示例代码 40.2.2.3 drm_device 结构体

```
1  struct drm_device {
2      struct list_head legacy_dev_list;
3      int if_version;
```

```
4       struct kref ref;
......
30      u32 max_vblank_count;
31      struct list_head vblank_event_list;
32      spinlock_t event_lock;
33      struct drm_agp_head *agp;
34      struct pci_dev *pdev;
35      unsigned int num_crtcs;
36      struct drm_mode_config mode_config;
37      struct mutex object_name_lock;
38      struct idr object_name_idr;
39      struct drm_vma_offset_manager *vma_offset_manager;
40      struct drm_vram_mm *vram_mm;
41      enum switch_power_state switch_power_state;
42      struct drm_fb_helper *fb_helper;
43  };
```

我们在编写 DRM 驱动的时候需要自行申请 drm_device 内存并且使用初始化，这个可以直接通过 drm_dev_alloc 函数来完成，函数原型如下：

```
struct drm_device *drm_dev_alloc(struct drm_driver *driver, struct device *parent)
```

此函数会先调用 kzalloc 为 drm_device 分配内存，然后调用 drm_dev_init 初始化 drm_device，函数参数和返回值含义如下：

driver：drm_driver 结构体指针，也就是 DRM 设备对应的 DRM 驱动。

parent：父设备。

返回值：返回分配成功的新 DRM 设备，如果返回 ERR_PTR 的话就表示 drm_device 申请失败。

drm_device 分配成功以后还需要使用 drm_dev_register 函数向内核注册，函数原型如下：

```
int drm_dev_register(struct drm_device *dev,      unsigned long flags)
```

函数参数和返回值含义如下：

dev：需要注册到内核的 drm_device。

flags：传递给驱动.load 函数的标志。

返回值：0，成功；负数，失败。

2、drm_driver 结构体

Linux 内核为 DRM 驱动提供一个叫做 drm_driver 的结构体，drm_driver 结构体包含了 DRM 驱动的完整属性和操作集合，因此每一个 DRM 驱动都必须有一个 drm_driver。drm_driver 结构体定义在 include/drm/drm_drv.h 文件里，内容如下(有省略):

示例代码 40.2.2.4 drm_driver 结构体

```
1  struct drm_driver {
2   int (*load) (struct drm_device *, unsigned long flags);
3   int (*open) (struct drm_device *, struct drm_file *);
......
56
57  int (*dumb_create)(struct drm_file *file_priv,
58            struct drm_device *dev,
```

```
59          struct drm_mode_create_dumb *args);
60  int (*dumb_map_offset)(struct drm_file *file_priv,
61              struct drm_device *dev, uint32_t handle,
62              uint64_t *offset);
63  int (*dumb_destroy)(struct drm_file *file_priv,
64            struct drm_device *dev,
65            uint32_t handle);
66  const struct vm_operations_struct *gem_vm_ops;
67
68  int major;            /* 驱动主设备号 */
69  int minor;            /* 驱动次设备号 */
70  int patchlevel;       /* 驱动补丁等级 */
71  char *name;           /* 驱动名字     */
72  char *desc;           /* 驱动描述     */
73  char *date;           /* 驱动日期     */
74  u32 driver_features; /* 驱动特性     */
75  const struct drm_ioctl_desc *ioctls;
76  int num_ioctls;
77  const struct file_operations *fops;
78
79  struct list_head legacy_dev_list;
80  int (*firstopen) (struct drm_device *);
81  void (*preclose) (struct drm_device *, struct drm_file *file_priv);
82  int (*dma_ioctl) (struct drm_device *dev, void *data,
                    struct drm_file *file_priv);
83  int (*dma_quiescent) (struct drm_device *);
84  int (*context_dtor) (struct drm_device *dev, int context);
85  int dev_priv_size;
86 };
```

drm_driver 结构体的成员变量很多，我们重点关注 driver_features、fops 和 dumb_create。

第 57 行，dumb_create 是一个回调函数，用于创建 gem 对象，并分配物理 buffer。

第 74 行，driver_features 用来描述驱动特性，枚举类型 drm_driver_feature 定义了可以选择的驱动特性：

DRIVER_GEM：驱动使用 GEM 内存管理，此特性必须选中！

DRIVER_MODESET：驱动支持模式设置接口(KMS)。

DRIVER_RENDER：驱动支持专用渲染节点。

DRIVER_ATOMIC：驱动提供完整的原子操作，以供用户空间 API 函数操作。

DRIVER_SYNCOBJ：驱动支持 SYNCOBJ，用于命令提交的显式同步。

DRIVER_SYNCOBJ_TIMELINE：驱动支持 SYNCOBJ 时间线。

DRIVER_USE_AGP：驱动程序使用 AGP 接口，DRM 核心将管理 AGP 资源。

DRIVER_LEGACY：表明这是一个使用影子附着的旧驱动程序，不使用。

DRIVER_PCI_DMA：驱动支持 PCI DMA。

DRIVER_SG：驱动可以提供 scatter/gather DMA 功能。

DRIVER_HAVE_DMA：驱动支持 DMA。

DRIVER_HAVE_IRQ：驱动支持 IRQ，旧驱动使用。

DRIVER_KMS_LEGACY_CONTEXT：仅供 nouveau 使用！

第 77 行，fops 这个相信大家都很熟悉了吧，就是一个简单的字符设备接口结构体。

前面我们说了，STM32MP1 的 LTDC 是个 platform 驱动，当设备和驱动匹配成功以后 stm_drm_platform_probe 函数就会执行，函数内容如下：

示例代码 40.2.2.5 stm_drm_platform_probe 函数

```
191 static int stm_drm_platform_probe(struct platform_device *pdev)
192 {
193     struct device *dev = &pdev->dev;
194     struct drm_device *ddev;
195     int ret;
196
197     DRM_DEBUG("%s\n", __func__);
198
199     dma_set_coherent_mask(dev, DMA_BIT_MASK(32));
200
201     ddev = drm_dev_alloc(&drv_driver, dev);
202     if (IS_ERR(ddev))
203         return PTR_ERR(ddev);
204
205     ret = drv_load(ddev);
206     if (ret)
207         goto err_put;
208
209     ret = drm_dev_register(ddev, 0);
210     if (ret)
211         goto err_put;
212
213     drm_fbdev_generic_setup(ddev, 16);
214
215     return 0;
216
217 err_put:
218     drm_dev_put(ddev);
219
220     return ret;
221 }
```

第 201 行，调用 drm_dev_alloc 函数，此函数主要完成三个功能：

①、给 drm_device 分配内存。

②、通过 drm_dev_init 函数初始化 drm_device。

drm_dev_alloc 会通过调用 drm_dev_init 函数将 drm_driver 和 drm_device 联系起来，drm_device 结构体里面有个 drvier 指针成员变量，此成员变量指向 DRM 设备对应的 DRM 驱动

的。因此，drm_dev_init 函数会通过将 drm_device 下的 driver 成员变量指向 drm_driver 来实现两者相连。

第 205 行，drv_load 这个函数就是初始化 KMS，放到后面分析。

第 209 行，注册 drm_device 对象进 DRM core。

接下来看下，drv_load 函数，内容如下：

示例代码 40.2.2.6 drv_load 函数

```
29  #define STM_MAX_FB_WIDTH    2048
30  #define STM_MAX_FB_HEIGHT   2048
31
32  static const struct drm_mode_config_funcs drv_mode_config_funcs = {
33      .fb_create = drm_gem_fb_create,
34      .atomic_check = drm_atomic_helper_check,
35      .atomic_commit = drm_atomic_helper_commit,
36  };
......
79  static int drv_load(struct drm_device *ddev)
80  {
81      struct platform_device *pdev = to_platform_device(ddev->dev);
82      struct ltdc_device *ldev;
83      int ret;
84
85      DRM_DEBUG("%s\n", __func__);
86
87      ldev = devm_kzalloc(ddev->dev, sizeof(*ldev), GFP_KERNEL);
88      if (!ldev)
89          return -ENOMEM;
90
91      ddev->dev_private = (void *)ldev;
92
93      drm_mode_config_init(ddev);
94
95      /*
96       * set max width and height as default value.
97       * this value would be used to check framebuffer size limitation
98       * at drm_mode_addfb().
99       */
100     ddev->mode_config.min_width = 0;
101     ddev->mode_config.min_height = 0;
102     ddev->mode_config.max_width = STM_MAX_FB_WIDTH;
103     ddev->mode_config.max_height = STM_MAX_FB_HEIGHT;
104     ddev->mode_config.funcs = &drv_mode_config_funcs;
105
106     ret = ltdc_load(ddev);
```

```
107     if (ret)
108         goto err;
109
110     drm_mode_config_reset(ddev);
111     drm_kms_helper_poll_init(ddev);
112
113     platform_set_drvdata(pdev, ddev);
114
115     return 0;
116 err:
117     drm_mode_config_cleanup(ddev);
118     return ret;
119 }
```

第 102 行，设置 DRM 驱动 X 轴(宽度)最大支持 2048 个像素。

第 103 行，设置 DRM 驱动 Y 轴(宽度)最大支持 2048 个像素，结合上一行，可以看出驱动里面设置的最大分辨率支持 2048*2048。但是根据 STM32MP157 手册所描述，最大支持 1366*768 分辨率的屏幕。

第 104 行，设置 framebuffer 的回调函数结构体。

第 106 行，这里我们要引入 drm_panel 结构体，此结构体作用是提供一堆控制回调函数。比如屏幕参数回调函数，背光控制函数等等。ltdc_load 函数是负责初始化 ltdc 接口(同时 connector 和 encoder 一起初始化)。在 connector 初始化的时候，就会调用 drm_panel 结构体里的获取屏幕参数函数(所以我们只需要提供一个屏的驱动就能正常显示了)。通常 encoder 和 connector 是放在同一个驱动初始化的，目的是为了方便驱动程序设计。

重点来了！要完成整个 DRM 驱动的正常初始化，前面的 GEM 和 KMS 这些模块 ST 官方已经提供给我们了，只需提供一个 drm_panel 对象就行了。打开“include/drm/drm_panel.h”文件，找到如下内容所示：

示例代码 40.2.2.7 drm_panel 结构体

```
1  struct drm_panel {
2      struct drm_device *drm;                 /* drm_device 对象     */
3      struct drm_connector *connector;        /* connector 对象      */
4      struct device *dev;                     /* 设备节点 dev        */
5      const struct drm_panel_funcs *funcs;    /* 回调函数的结构体    */
6      struct list_head list;
7  };
```

这个结构体是比较简单的相信各位能够看懂。主要是第 5 行，funcs 对象成员。在这里我们的 DRM 驱动分析基本完成了。

40.3 RGB LCD 驱动分析(屏的驱动)

上一小节说了，drm_panel 就是 DRM 驱动的核心结构体，我们需要实现此结构体。我们先来看一下 panel_simple 结构体。这里用到了面向对象的思维，drm_panel 结构体是基类，panel_simple 在 drm_panel 基础上增加了一些成员变量，相当于继承类。

我们使用的是 RGB LCD 屏，所以我们的 LCD 驱动文件为 drivers/gpu/drm/panel/panel-simple.c，打文件找到如下内容所示：

示例代码 40.3.1 panel_simple 结构体

```
99  struct panel_simple {
100     struct drm_panel base;
101     bool prepared;
102     bool enabled;
103     bool no_hpd;
104
105     const struct panel_desc *desc;
106
107     struct backlight_device *backlight;
108     struct regulator *supply;
109     struct i2c_adapter *ddc;
110
111     struct gpio_desc *enable_gpio;
112
113     struct drm_display_mode override_mode;
114 };
```

panel_simple 结构体用来管理 RGB LCD 设备。

第 100 行，base 成员变量，为 drm_panel 结构体类型。可以看出 panel_simple 就是在 drm_panel 的基础上发展而来的，在 DRM 驱动注册的时候就会回调 base->funcs。

第 105 行，desc 属性就是 RGB 屏参数结构体。

第 107 行，屏的背光结构体。

接着我们看下如何跟设备树匹配的，找到如下所示内容：

示例代码 40.3.2 LCD 驱动的 platform_driver

```
3491    static struct platform_driver panel_simple_platform_driver = {
3492        .driver = {
3493            .name = "panel-simple",
3494            .of_match_table = platform_of_match,
3495        },
3496        .probe = panel_simple_platform_probe,
3497        .remove = panel_simple_platform_remove,
3498        .shutdown = panel_simple_platform_shutdown,
3499    };
```

示例代码 40.3.1 中，这是一个标准的 platform 驱动框架，第 3494 行就是匹配表。platform_of_match 内容如下所示(有省略)：

示例代码 40.3.3 platform_of_match 结构体

```
3133    static const struct of_device_id platform_of_match[] = {
3134        {
3135            .compatible = "ampire,am-480272h3tmqw-t01h",
3136            .data = &ampire_am_480272h3tmqw_t01h,
3137        }, {
3138            .compatible = "ampire,am800480r3tmqwa1h",
3139            .data = &ampire_am800480r3tmqwa1h,
```

```
3140        }, {
3141        }
3142    };
```

可以看出，platform_of_match 里面有大量的匹配项，分别针对不同的屏幕，比如：

第 3135 行，这就是一个匹配项，compatible 内容为“ampire,am-480272h3tmqw-t01h”。

第 3136 行，在 platform 框架里有个 data 成员变量，这个是一个 void 类型的指针，这里指向 ampire_am_480272h3tmqw_t01h，内容如下所示：

示例代码 40.3.4 ampire_am_480272h3tmqw_t01h

```
515 static const struct drm_display_mode
                        ampire_am_480272h3tmqw_t01h_mode = {
516     .clock = 9000,                  /* LCD 像素时钟，单位 KHz    */
517     .hdisplay = 480,                /* LCD X 轴像素个数          */
518     .hsync_start = 480 + 2,         /* LCD X 轴+hbp 的像素个数   */
519     .hsync_end = 480 + 2 + 41,      /* LCD X 轴+hbp+hspw 的像素个数 */
520     .htotal = 480 + 2 + 41 + 2,     /* LCD X 轴+hbp+hspw+hfp 的像素个数 */
521     .vdisplay = 272,                /* LCD Y 轴像素个数          */
522     .vsync_start = 272 + 2,         /* LCD Y 轴+vbp 的像素个数   */
523     .vsync_end = 272 + 2 + 10,      /* LCD Y 轴+vbp+vspw 的像素个数 */
524     .vtotal = 272 + 2 + 10 + 2,     /* LCD Y 轴+vbp+vspw+vfp 的像素个数 */
525     .vrefresh = 60,                 /* LCD 的刷新频率为 60HZ     */
526     .flags = DRM_MODE_FLAG_PHSYNC | DRM_MODE_FLAG_PVSYNC,
527 };
528
529 static const struct panel_desc ampire_am_480272h3tmqw_t01h = {
530     .modes = &ampire_am_480272h3tmqw_t01h_mode,
531     .num_modes = 1,
532     .bpc = 8,
533     .size = {
534         .width = 105,
535         .height = 67,
536     },
537     .bus_format = MEDIA_BUS_FMT_RGB888_1X24,
538 };
```

第 516~525 行，drm_display_mode 结构体就是用来设置屏幕参数。

第 529 行，定义一个 panel_desc 结构体对象。

第 530 行，modes 变量设置为 ampire_am_480272h3tmqw_t01h_mode。

第 531 行，设置 modes 的数量。

第 532 行，设置屏幕为 8bit。

第 533~536 行，设置屏幕实际显示区域的物理宽度，单位为毫米，此屏幕尺寸为 105mm×67mm。

第 537 行， bus_format 属性设置总线模式，include/uapi/linux/media-bus-format.h 里面定义了所有可选的总线类型：

示例代码 40.3.5 总线配置项

```
1   #define MEDIA_BUS_FMT_FIXED                     0x0001
2
3   /* RGB - next is    0x101d */
4   #define MEDIA_BUS_FMT_RGB444_1X12               0x1016
5   #define MEDIA_BUS_FMT_RGB444_2X8_PADHI_BE       0x1001
6   #define MEDIA_BUS_FMT_RGB444_2X8_PADHI_LE       0x1002
7   #define MEDIA_BUS_FMT_RGB555_2X8_PADHI_BE       0x1003
8   #define MEDIA_BUS_FMT_RGB555_2X8_PADHI_LE       0x1004
9   #define MEDIA_BUS_FMT_RGB565_1X16               0x1017
10  #define MEDIA_BUS_FMT_BGR565_2X8_BE             0x1005
11  #define MEDIA_BUS_FMT_BGR565_2X8_LE             0x1006
12  #define MEDIA_BUS_FMT_RGB565_2X8_BE             0x1007
13  #define MEDIA_BUS_FMT_RGB565_2X8_LE             0x1008
14  #define MEDIA_BUS_FMT_RGB666_1X18               0x1009
15  #define MEDIA_BUS_FMT_RBG888_1X24               0x100e
16  #define MEDIA_BUS_FMT_RGB666_1X24_CPADHI        0x1015
17  #define MEDIA_BUS_FMT_RGB666_1X7X3_SPWG         0x1010
18  #define MEDIA_BUS_FMT_BGR888_1X24               0x1013
19  #define MEDIA_BUS_FMT_BGR888_3X8                0x101b
20  #define MEDIA_BUS_FMT_GBR888_1X24               0x1014
21  #define MEDIA_BUS_FMT_RGB888_1X24               0x100a
22  #define MEDIA_BUS_FMT_RGB888_2X12_BE            0x100b
23  #define MEDIA_BUS_FMT_RGB888_2X12_LE            0x100c
24  #define MEDIA_BUS_FMT_RGB888_3X8                0x101c
25  #define MEDIA_BUS_FMT_RGB888_1X7X4_SPWG         0x1011
26  #define MEDIA_BUS_FMT_RGB888_1X7X4_JEIDA        0x1012
27  #define MEDIA_BUS_FMT_ARGB8888_1X32             0x100d
......
120
121 /* HSV - next is    0x6002 */
122 #define MEDIA_BUS_FMT_AHSV8888_1X32             0x6001
```

可以看出，Linux 内核支持很多种不同的总线格式，比如 RGB、YUV、Bayer 等。我们以 MEDIA_BUS_FMT_RGB888_1X24 为例，看看这个总线格式的含义：

①、从“RGB888”可以看出，这是一个 RGB888 格式的。

②、后面的“1X24”表示一个像素点使用 24bit，如果是 2X8 就表示一个像素点使用 2 个 8bit 表示。

③、有的右面也会有“BE”或“LE”，BE 表示最高位先传输，LE 表示最低位先传输。

假设设备树里有个设备节点的 compatible 属性为“ampire,am-480272h3tmqw-t01h”，那么就会和驱动匹配成功，然后运行 panel_simple_platform_probe 函数，此函数的内容如下所示：

示例代码 40.3.6 LCD 驱动的 probe

```
3470    static int panel_simple_platform_probe(struct platform_device
*pdev)
3471    {
```

```
3472        const struct of_device_id *id;
3473
3474        id = of_match_node(platform_of_match, pdev->dev.of_node);
3475        if (!id)
3476            return -ENODEV;
3477
3478        return panel_simple_probe(&pdev->dev, id->data);
3479    }
```

第 3474 行，使用 of_match_node 函数查找匹配的设备 ID。

第 3478 行，当我们得到匹配的设备 ID(of_device_id)以后就可以通过提取 data 成员变量得到屏幕参数信息，比如此处 id->data 就是 ampire_am_480272h3tmqw_t01h。最后调用 panel_simple_probe 函数将其注册到内核，panel_simple_probe 函数也定义在 drivers/gpu/drm/panel/panel-simple.c 文件中，函数内容如下所示：

示例代码 40.3.7 panel_simple_probe 函数

```
414 static int panel_simple_probe(struct device *dev,
                                   const struct panel_desc *desc)
415 {
416     struct device_node *backlight, *ddc;
417     struct panel_simple *panel;
418     struct display_timing dt;
419     int err;
420
421     panel = devm_kzalloc(dev, sizeof(*panel), GFP_KERNEL);
422     if (!panel)
423         return -ENOMEM;
424
425     panel->enabled = false;
426     panel->prepared = false;
427     panel->desc = desc;
428
......
444     backlight = of_parse_phandle(dev->of_node, "backlight", 0);
445     if (backlight) {
446         panel->backlight = of_find_backlight_by_node(backlight);
447         of_node_put(backlight);
448
449         if (!panel->backlight)
450             return -EPROBE_DEFER;
451     }
452
......
467     drm_panel_init(&panel->base);
468     panel->base.dev = dev;
```

```
469     panel->base.funcs = &panel_simple_funcs;
470
471     err = drm_panel_add(&panel->base);
472     if (err < 0)
473         goto free_ddc;
474
475     dev_set_drvdata(dev, panel);
476
477     return 0;
478
479 free_ddc:
480     if (panel->ddc)
481         put_device(&panel->ddc->dev);
482 free_backlight:
483     if (panel->backlight)
484         put_device(&panel->backlight->dev);
485
486     return err;
487 }
```

第 427 行，设置屏幕参数。

第 444 行，从设备树里获取背光节点，所以我们的设备树要提供“backlight”属性。

第 467 行，用 drm_panel_init 函数初始化屏幕。

第 469 行，设置 panel_simple 的回调函数，为 DRM 驱动注册的时候提供屏的参数。

第 471 行，把屏幕注册到内核。

LCD 屏的驱动分析结束，我们总结一下添加自己的屏要做那些操作：

①、在根节点下提供一个 LCD 设备树，包含背光的节点和引用 ltdc 节点。

②、在 panel-simple.c 文件里的 platform_of_match 结构体里添加一组设备 ID，此设备 ID 对应你所使用的屏幕，重点是屏幕参数 panel_desc 结构体。

40.4 硬件原理图分析

在 STM32MP1 开发板里，有一个 RGB LCD 接口，其原理如下所示：

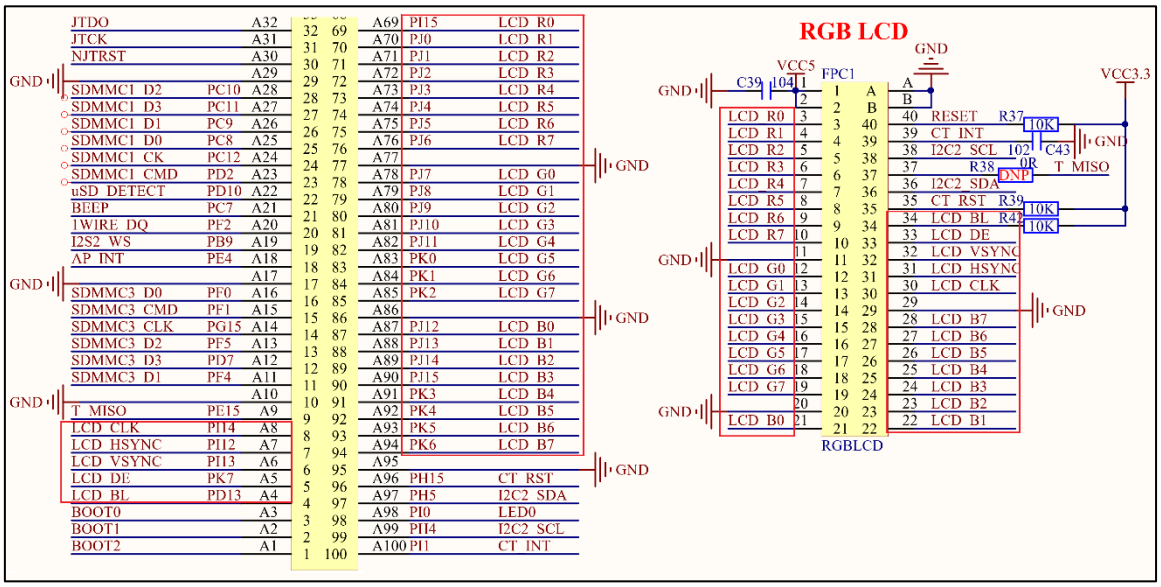


图 40.4.1 RGB LCD 接口硬件原理图

图 40.4.1 左边对应的是 LCD 接口引脚图，右边为 STM32MP1 开发板 RGB LCD 接口的原理图。

40.5 LCD 驱动程序编写

40.5.1 修改设备树

首先就是修改设备树，重点要注意三个地方：

①、LCD 所使用的 IO 配置，由于 STM32MP1 的 IO 支持复用，所以大家实际所使用的 LCD 引脚可能不一样。因此首先要根据实际硬件设计，修改 LCD 所有使用的 IO 配置。

②、LDTC 接口节点修改，修改相应的属性值，告诉内核要指定那个输出接口，比如输出到 RGB LCD 屏、MIPI 屏等。本实验用到是正点原子 ATK7016 屏，也就是 RGB LCD 屏。

③、输出接口节点的编写，比如本实验用到的 RGB LCD 屏需要在根节点下添加 RGB LCD 节点。

④、LCD 背光节点信息修改，要根据实际所使用的背光 IO 来修改相应的设备节点信息。

接下来我们依次来看一下上面这三个节点如何去修改。

1、LCD 屏幕使用的 IO 配置

首先要检查一下设备树中 LCD 所使用的 IO 配置，这个其实 ST 都已经给我们写好了，需要修改，不过我们还是要看一下。打开 arch/arm/boot/dts/stm32mp15-pinctrl.dtsi 文件，在 pinctrl 节点中找到如下内容：

示例代码 40.5.1.1 ltdc 的 pinmux 节点

```
1  ltdc_pins_b: ltdc-b-0 {
2      pins {
3          pinmux = <STM32_PINMUX('I', 14, AF14)>,    /* LCD_CLK     */
4               <STM32_PINMUX('I', 12, AF14)>, /* LCD_HSYNC   */
5               <STM32_PINMUX('I', 13, AF14)>, /* LCD_VSYNC   */
6               <STM32_PINMUX('K',  7, AF14)>, /* LCD_DE      */
7               <STM32_PINMUX('I', 15, AF14)>, /* LCD_R0      */
8               <STM32_PINMUX('J',  0, AF14)>, /* LCD_R1      */
```

```
9               <STM32_PINMUX('J',  1, AF14)>, /* LCD_R2     */
10              <STM32_PINMUX('J',  2, AF14)>, /* LCD_R3     */
11              <STM32_PINMUX('J',  3, AF14)>, /* LCD_R4     */
12              <STM32_PINMUX('J',  4, AF14)>, /* LCD_R5     */
13              <STM32_PINMUX('J',  5, AF14)>, /* LCD_R6     */
14              <STM32_PINMUX('J',  6, AF14)>, /* LCD_R7     */
15              <STM32_PINMUX('J',  7, AF14)>, /* LCD_G0     */
16              <STM32_PINMUX('J',  8, AF14)>, /* LCD_G1     */
17              <STM32_PINMUX('J',  9, AF14)>, /* LCD_G2     */
18              <STM32_PINMUX('J', 10, AF14)>, /* LCD_G3     */
19              <STM32_PINMUX('J', 11, AF14)>, /* LCD_G4     */
20              <STM32_PINMUX('K',  0, AF14)>, /* LCD_G5     */
21              <STM32_PINMUX('K',  1, AF14)>, /* LCD_G6     */
22              <STM32_PINMUX('K',  2, AF14)>, /* LCD_G7     */
23              <STM32_PINMUX('J', 12, AF14)>, /* LCD_B0     */
24              <STM32_PINMUX('J', 13, AF14)>, /* LCD_B1     */
25              <STM32_PINMUX('J', 14, AF14)>, /* LCD_B2     */
26              <STM32_PINMUX('J', 15, AF14)>, /* LCD_B3     */
27              <STM32_PINMUX('K',  3, AF14)>, /* LCD_B4     */
28              <STM32_PINMUX('K',  4, AF14)>, /* LCD_B5     */
29              <STM32_PINMUX('K',  5, AF14)>, /* LCD_B6     */
30              <STM32_PINMUX('K',  6, AF14)>; /* LCD_B7     */
31          bias-disable;
32          drive-push-pull;
33          slew-rate = <1>;
34      };
35  };
36
37  ltdc_pins_sleep_b: ltdc-b-1 {
38      pins {
39          pinmux = <STM32_PINMUX('I', 14, ANALOG)>, /* LCD_CLK */
40              <STM32_PINMUX('I', 12, ANALOG)>,   /* LCD_HSYNC */
41              <STM32_PINMUX('I', 13, ANALOG)>,   /* LCD_VSYNC */
42              <STM32_PINMUX('K',  7, ANALOG)>,   /* LCD_DE  */
43              <STM32_PINMUX('I', 15, ANALOG)>,   /* LCD_R0  */
44              <STM32_PINMUX('J',  0, ANALOG)>,   /* LCD_R1  */
45              <STM32_PINMUX('J',  1, ANALOG)>,   /* LCD_R2  */
46              <STM32_PINMUX('J',  2, ANALOG)>,   /* LCD_R3  */
47              <STM32_PINMUX('J',  3, ANALOG)>,   /* LCD_R4  */
48              <STM32_PINMUX('J',  4, ANALOG)>,   /* LCD_R5  */
49              <STM32_PINMUX('J',  5, ANALOG)>,   /* LCD_R6  */
50              <STM32_PINMUX('J',  6, ANALOG)>,   /* LCD_R7  */
51              <STM32_PINMUX('J',  7, ANALOG)>,   /* LCD_G0  */
```

```
52              <STM32_PINMUX('J',  8, ANALOG)>,   /* LCD_G1  */
53              <STM32_PINMUX('J',  9, ANALOG)>,   /* LCD_G2  */
54              <STM32_PINMUX('J', 10, ANALOG)>,   /* LCD_G3  */
55              <STM32_PINMUX('J', 11, ANALOG)>,   /* LCD_G4  */
56              <STM32_PINMUX('K',  0, ANALOG)>,   /* LCD_G5  */
57              <STM32_PINMUX('K',  1, ANALOG)>,   /* LCD_G6  */
58              <STM32_PINMUX('K',  2, ANALOG)>,   /* LCD_G7  */
59              <STM32_PINMUX('J', 12, ANALOG)>,   /* LCD_B0  */
60              <STM32_PINMUX('J', 13, ANALOG)>,   /* LCD_B1  */
61              <STM32_PINMUX('J', 14, ANALOG)>,   /* LCD_B2  */
62              <STM32_PINMUX('J', 15, ANALOG)>,   /* LCD_B3  */
63              <STM32_PINMUX('K',  3, ANALOG)>,   /* LCD_B4  */
64              <STM32_PINMUX('K',  4, ANALOG)>,   /* LCD_B5  */
65              <STM32_PINMUX('K',  5, ANALOG)>,   /* LCD_B6  */
66              <STM32_PINMUX('K',  6, ANALOG)>;   /* LCD_B7  */
67      };
68  };
```

第 1~35 行，子节点 ltdc_pins_b，为 RGB LCD 的 24 根数据线配置项和 4 根控制线配置项。

第 37~67 行，子节点 ltdc_pins_sleep_b，同样也是 RGB LCD 的 24 根数据线配置项和 4 根控制线配置项。

可以看出，这里有两个 pinmux 分别为：ltdc_pins_b 和 ltdc_pins_sleep_b，其中 ltdc_pins_b 是在默认模式下的 RGB LCD pinmux 配置，ltdc_pins_sleep_b 是在 sleep 模式下 RGB LCD 的 pinmux 配置。如果我们想要 LCD 屏进入睡眠模式就切换为 sleep 模式。

正点原子 STM32MP1 开发板 RGB LCD 屏幕所使用的引脚和 ST 官方开发板一致，因此示例代码 40.5.1 不需要做任何修改。如果你所使用的开发板其 LCD 引脚和正点原子 STM32MP1 开发板不一致，一定要根据自己的实际硬件修改示例代码 40.5.1.1。

2、LTDC 接口节点修改

LTDC 节点在 stm32mp151.dtsi 里已经写好一部分了，我们只需要告诉 LTDC 节点输出到 RGB LCD 屏里就行。在 stm32mp157d-atk.dts 文件，添加如下内容所示：

示例代码 40.5.5.2 ltdc 节点

```
1   &ltdc {
2       pinctrl-names = "default", "sleep";
3       pinctrl-0 = <&ltdc_pins_b>;
4       pinctrl-1 = <&ltdc_pins_sleep_b>;
5       status = "okay";
6
7       port {
8           #address-cells = <1>;
9           #size-cells = <0>;
10
11          ltdc_ep0_out: endpoint@0 {
12              reg = <0>;
13              remote-endpoint = <&rgb_panel_in>;
```

```
14          };
15      };
16  };
```

第 2~4 行，给 LTDC 设置了两个 pinmux 模式，pinctrl-0 为 default 模式，pinctrl-1 为 sleep 模式，系统默认使用 default 模式。

第 8~9 行，设置 port 下子节点的 reg 属性的地址信息描述。

第 11~14 行，在 port 下添加了一个子节点为 ltdc_ep0_out。在第 12 行里，reg 属性值为 0。在第 13 行里，remote-endpoint 属性是用来告诉 ltdc 节点输出到那里，我们是用 RGB LCD 屏做实验，所以输出到 rgb_panel_in 接口。

3、输出接口的编写

我们还需要添加一个 LCD 设备树节点，在 stm32mp157d-atk.dts 文件的根节点“/”下添加如下所示内容：

示例代码 40.5.5.3 LCD 屏的设备节点

```
1   panel_rgb: panel-rgb {
2       compatible = "alientek,lcd-rgb";
3       backlight = <&backlight>;
4       status = "okay";
5
6       port {
7           rgb_panel_in: endpoint {
8               remote-endpoint = <&ltdc_ep0_out>;
9           };
10      };
11  };
```

第 2 行，设置 compatible 属性值为“alientek,lcd-rgb”，所以我们稍后要在 panel-simple.c 文件里的 platform_of_match 数组增加一个 of_device_id 结构体，此结构体的 compatible 成员属性值为“alientek,lcd-rgb”。

第 3 行，设置 backlight 属性值为“&backlight”，此属性值为引用背光节点，稍后给大家讲解如何编写。

第 6~9 行，告诉 LCD 驱动，要从 LTDC 节点里获取显示数据。第 8 行就是引用 ltdc 节点。

40.5.2 在 panel-simple.c 文件里面添加屏幕参数

接着我们就要在 panel-simple.c 文件里面添加屏幕参数，打开此文件，找到 platform_of_match 数组，添加如下内容：

示例代码 40.5.2.1 屏的匹配属性

```
1 .compatible = "alientek,lcd-rgb",
2 .data = &alientek_desc,
```

添加完成以后如图 40.5.2.1 所示：

```
3438            .data = &winstar_wf35ltiacd,
3439        },
3440        {
3441            .compatible = "alientek,lcd-rgb",
3442            .data = &alientek_desc,
3443        }, {
3444            /* sentinel */
3445        }
3446    };
3447    MODULE_DEVICE_TABLE(of, platform_of_match);
```

图 40.5.2.1 新添加的 LCD 屏幕参数

在图 40.5.2.1 里的 alientek_desc 保存参数，我们还需要继续在 panel-simple.c 里面实现 alientek_desc，添加如下所示代码：

示例代码 40.5.2.2 alientek_desc 结构体

```
1   static const struct drm_display_mode ATK7016_mode = {
2       .clock = 51200,                 /* LCD 像素时钟，单位 KHz           */
3       .hdisplay = 1024,               /* LCD X 轴像素个数                 */
4       .hsync_start = 1024 + 140,      /* LCD X 轴+hbp 的像素个数          */
5       .hsync_end = 1024 + 140 + 20,    /* LCD X 轴+hbp+hspw 的像素个数*/
6       .htotal = 1024 + 140 + 20 + 160,/* LCD X 轴+hbp+hspw+hfp     */
7       .vdisplay = 600,                /* LCD Y 轴像素个数                 */
8       .vsync_start = 600 + 20,        /* LCD Y 轴+vbp 的像素个数          */
9       .vsync_end = 600 + 20 + 3,      /* LCD Y 轴+vbp+vspw 的像素个数     */
10      .vtotal = 600 + 20 + 3 + 12,/* LCD Y 轴+vbp+vspw+vfp          */
11      .vrefresh = 60,                 /* LCD 的刷新频率为 60HZ            */
12  };
13
14  static const struct panel_desc alientek_desc = {
15      .modes = &ATK7016_mode,
16      .num_modes = 1,
17      .bus_format = MEDIA_BUS_FMT_RGB888_1X24,
18  };
```

在示例代码 40.5.2.2 就是 ATK7016 屏的参数，并且设置为 RGB888 模式。如果使用的其他屏幕，请按照屏幕手册对应的时序参数设置。

第 2~11 行，ATK7016 屏幕时序参数，根据自己所使用的屏幕修改即可。

40.5.3 LCD 屏幕背光节点信息

1、背光 PWM 节点设置

LCD 背光使用 PWM 来控制，通过 PWM 波形来调节屏幕亮度。关于 PWM 已经在《第三十九章 Linux PWM 驱动实验》进行了详细的讲解。

正点原子的 LCD 接口背光控制 IO 连接到了 STM32MP1 的 PD13 引脚上，我们需要将 PD13 复用为 TIM4_CH2，然后配置 TIM4 的 CH2 输出 PWM 信号，然后通过此 PWM 信号来控制 LCD 屏幕背光的亮度，接着我们来看一下如何在设备树中添加背光节点信息。

首先是 PD13 这个 pinmux 的配置，在 stm32mp15-pinctrl.dtsi 中找到如下内容：

示例代码 40.5.3.1 PD13 的 pinmux 配置

```
1   pwm4_pins_b: pwm4-1 {
2       pins {
3           pinmux = <STM32_PINMUX('D', 13, AF2)>; /* TIM4_CH2 */
4           bias-pull-down;
5           drive-push-pull;
6           slew-rate = <0>;
7       };
8   };
9
10  pwm4_sleep_pins_b: pwm4-sleep-1 {
11      pins {
12          pinmux = <STM32_PINMUX('D', 13, ANALOG)>; /* TIM4_CH2 */
13      };
14  };
```

示例代码 40.5.3.1 默认设置了 PD13 引脚的两种 pinmux 配置，从第 3 行可以看出，设置 PD13 复用为 TIM4_CH2，TIM4_CH2 这里的意思为 TIM4 里的第二个 PWM 通道，并且设置电气属性为内部下拉和推挽输出。这是因为 ST 官方开发板就使用了 PD13 作为 LCD 的背光控制引脚，正点原子 STM32MP1 开发板也使用 PD13 作为 LCD 背光控制引脚，所以不需要修改。如果你使用其他引脚作为 LCD 的背光控制引脚，那么就需要进行修改。

继续在 stm32mp157d-atk.dts 文件中向 timers4 追加内容，如下所示：

示例代码 40.5.3.2 向 tim4 节点追加内容

```
1   &timers4 {
2       status = "okay";
3       /* spare dmas for other usage */
4       /delete-property/dmas;
5       /delete-property/dma-names;
6       pwm4: pwm {
7           pinctrl-0 = <&pwm4_pins_b>;
8           pinctrl-1 = <&pwm4_sleep_pins_b>;
9           pinctrl-names = "default", "sleep";
10          #pwm-cells = <2>;
11          status = "okay";
12      };
13  };
```

第 2 行，把 status 设置为 okay。

第 4~5 行，设置此节点不用 dma。

第 6 行，pwm4 是我们为 pwm 设置的一个别名。

第 7~9 行，设置 PWM 所使用的 IO 配置。

第 10 行，此参数是用来规定 pwms 属性的参数。比如：#pwm-cells =<2>，表示 pwms 属性有 2 个参数，如下所示：

```
pwms= <&pwm4 1 5000000>
```

其中 pwm4 表示使用 PWM4，后面两个是参数，其中 1 表示使用 PWM4 的通道 2(通道从 1 开始)；5000000 表示为 200Hz。

如果背光用的其他 pwm 通道，比如 pwm2，那么就需要仿照示例代码 40.5.3.2，向 timers2 节点追加相应的内容。比如：如果我们要设置 TIM2_CH4，那么 timers2 里的 pwm 节点下的 pinmux 配置就是 TIM2_CH4。

2、backlight 节点设置

到这里，PWM 和相关的 IO 已经准备好了，但是 Linux 系统怎么知道 TIM4_CH2 就是控制 LCD 背光的呢？因此我们还需要一个节点来将 LCD 背光和 TIM4_CH2 连接起来。这个节点就是 backlight，backlight 节点描述可以参考 Documentation/devicetree/bindings/leds/backlight/pwm-backlight.txt 这个文档，此文档详细讲解了 backlight 节点该如何去创建，这里大概总结一下：

①、节点名称要为“backlight”。

②、节点的 compatible 属性值要为“pwm-backlight”，因此可以通过在 Linux 内核中搜索“pwm-backlight”来查找 PWM 背光控制驱动程序，这个驱动程序文件为 drivers/video/backlight/pwm_bl.c，感兴趣的可以去看一下这个驱动程序。

③、pwms 属性用于描述背光所使用的 PWM 的通道以及 PWM 频率，比如本章我们要使用的 pwm4 的第二个通道，pwm 频率设置为 200Hz。

④、brightness-levels 属性描述亮度级别，范围为 0~255，0 表示 PWM 占空比为 0%，也就是亮度最低，255 表示 100%占空比，也就是亮度最高。至于设置几级亮度，大家可以自行填写此属性。

⑤、default-brightness-level 属性为默认亮度级别。

根据上述 5 点设置 backlight 节点，我们在根节点创建一个 backlight 节点，在 stm32mp157d-atk.dts 文件中新建内容如下：

示例代码 40.5.3.3 backlight 节点

```
1   backlight: backlight {
2       compatible = "pwm-backlight";
3       pwms = <&pwm4 1 5000000>;
4       brightness-levels = <0 4 8 16 32 64 128 255>;
5       power-supply = <&v3v3>;
6       default-brightness-level = <7>;
7       status = "okay";
8   };
```

第 3 行，设置背光使用 pwm4 的第二个通道，PWM 频率为 200Hz。

第 4 行，设置 8 级背光(0~7)，分别为 0、4、8、16、32、64、128、255，对应占空比为 0%、1.57%、3.13%、6.27%、12.55%、25.1%、50.19%、100%，如果嫌少的话可以自行添加一些其他的背光等级值。

第 5 行，设置默认背光等级为 7，也就是 100%的亮度。

注意：背光的驱动代码有点 bug，如果设置 0 级屏不会灭屏，打开 pwm_bl.c 文件，找到 123 行，如图 40.5.3.1 所示：

```
122
123     if (brightness > 0) {
124         pwm_get_state(pb->pwm, &state);
125         state.duty_cycle = compute_duty_cycle(pb, brightness);
126         pwm_apply_state(pb->pwm, &state);
127         pwm_backlight_power_on(pb);
128     } else
```

将'>'改为'>='

图 40.5.3.1 修改背光驱动 bug

将图 40.5.3.1 中 123 行的‘>’改为 ‘>=’即可。

关于背光的设备树节点信息就讲到这里，整个的 LCD 设备树节点内容我们就讲完了，按照这些节点内容配置自己的开发板即可。

40.6 运行测试

40.6.1 LCD 屏幕的 DRM 基本测试

1、编译新的内核和设备树

输入如下命令重新编译 Linux 内核和设备树：

```
make uImage dtbs LOADADDR=0xC2000040 -j16
```

编译完成以后一会要使用新的设备树和内核启动 Linux 内核。

2、配置内核

ST 官方的默认配置已经使能了 DRM 驱动，还是要告诉各位如何配置内核，打开 Linux 内核图形化配置界面，按下路径找到对应的配置项：

```
-> Device Drivers
    -> Graphics support
        [*] Direct Rendering Manager (XFree86 4.1.0 and higher DRI support)    //选中
        [*] DRM Support for STMicroelectronics SoC Series                      //选中
        -> Display Panels
            [*] support for simple panels                                      //选中
        -> Backlight & LCD device support
            [*] Generic PWM based Backlight Driver                             //选中
```

用新的 uImage 镜像和 stm32mp157d-atk.dtb 设备树用来启动内核，如果设置正确那么在文件系统/sys/class/drm/路径下，有如图 40.6.1.1 所示内容：

```
[root@ATK-stm32mp1]:~$ ls /sys/class/drm/
card0        card0-DPI-1  version
[root@ATK-stm32mp1]:~$
```

图 40.6.1.1 屏配置成功的结果

如果没有出现图 40.6.1.1 的内容，就要检查一下设备树和 panel-simple.c 文件是否设置正确。因为 Linux 一切皆文件，所以 DRM 驱动肯定会提供一个接口给用户使用，接口为“/dev/dri/card0”。可以通过此接口来设置 LCD 的显示。

2、文件系统使能 libdrm 库

在前面 40.2 小节里，没有 libdrm 库是不能调用 drm 驱动的，所以我们要在文件系统使能 libdrm 库，跳转到 buildroot-2020.02.6 的目录下，打开 buildroot 的图形化配置界面，根据如下配置去使能 libdrm 库。

```
Location:
  -> Target packages
    -> Libraries
      -> Graphics
        ->[*]libdrm                    //选中
          ->[*]Install test programs   //选中
```

如图 40.6.1.2 所示：

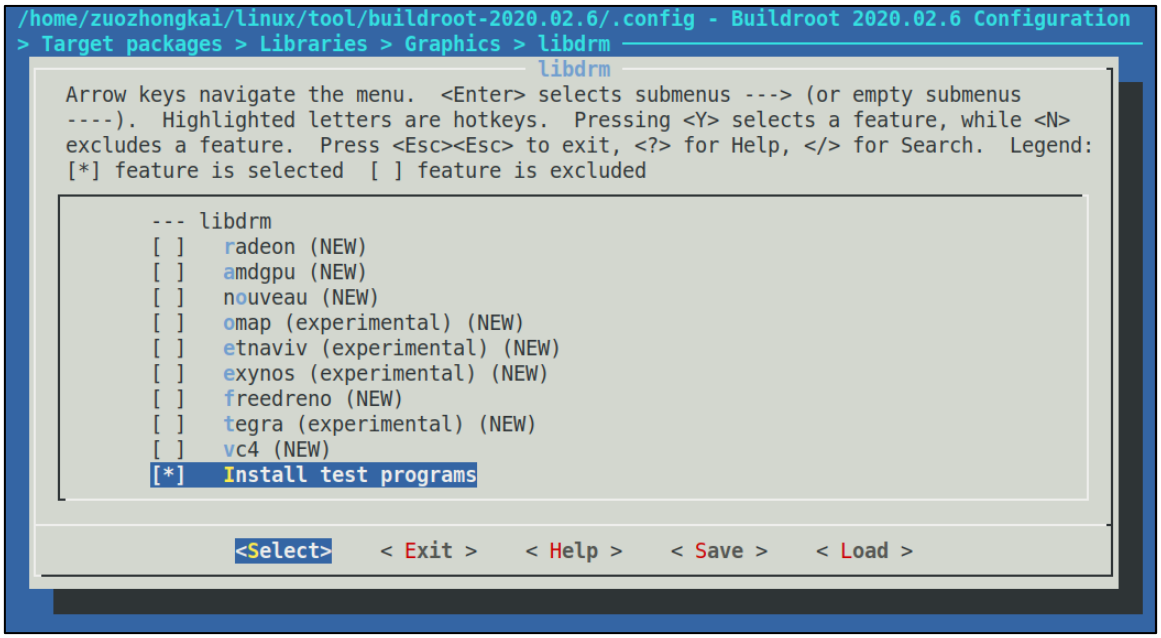


图 40.6.1.2 配置 libdrm 库

图 40.6.1.2 中的 Install test programs 配置会生成 modetest 命令，此命令是用来测试 DRM 驱动。保存并重新编译 buildroot 的文件系统，然后直接将新得到的根文件系统解压到开发板正在使用的根文件系统中，命令如下：

```
sudo tar -axvf rootfs.tar
```

3、测试

重新启动开发板，我们使用 modetest 命令进行测试，输入如下命令可以查看 modetest 命令的使用方法：

```
modetest --help
```

帮助信息如图 40.6.1.3 所示：

```
[root@ATK-stm32mp1]:~$ modetest --help
usage: modetest [-acDdefMPpsCvw]

 Query options:

        -c      list connectors
        -e      list encoders
        -f      list framebuffers
        -p      list CRTCs and planes (pipes)

 Test options:

        -P <plane_id>@<crtc_id>:<w>x<h>[+<x>+<y>][*<scale>][@<format>]  set a plane
        -s <connector_id>[,<connector_id>][@<crtc_id>]:<mode>[-<vrefresh>][@<format>]   set a mode
        -C      test hw cursor
        -v      test vsynced page flipping
        -w <obj_id>:<prop_name>:<value> set property
        -a      use atomic API
        -F pattern1,pattern2    specify fill patterns

 Generic options:

        -d      drop master after mode set
        -M module       use the given driver
        -D device       use the given device

       Default is to dump all info.
[root@ATK-stm32mp1]:~$
```

图 40.6.1.3 modetest 帮助信息

先输入如下命令查看一下设备信息：

```
modetest -M stm
```

-M：指定模块，这里我们查看“stm”这个设备。

输入命令以后就会打印出 stm 这个设备的详细信息，比较长，图 40.6.1.3 是一些重要信息：

```
[root@ATK-stm32mp1]:~$ modetest -M stm
Encoders:
id      crtc    type    possible crtcs  possible clones
31      35      DPI     0x00000001      0x00000000

Connectors:
id      encoder status          name            size (mm)       modes   encoders
32      31      connected       DPI-1           0x0             1       31
  modes:
        name refresh (Hz) hdisp hss hse htot vdisp vss vse vtot)
  1024x600 60 1024 1164 1184 1344 600 620 623 635 51200 flags: ; type: preferred, driver
  props:
        1 EDID:
                flags: immutable blob
                blobs:

                value:
        2 DPMS:
                flags: enum
                enums: On=0 Standby=1 Suspend=2 Off=3
                value: 0
        5 link-status:
                flags: enum
                enums: Good=0 Bad=1
                value: 0
        6 non-desktop:
                flags: immutable range
                values: 0 1
                value: 0
        4 TILE:
                flags: immutable blob
                blobs:

                value:
        20 CRTC_ID:
                flags: object
                value: 35

CRTCs:
id      fb      pos     size
35      0       (0,0)   (1024x600)
  1024x600 60 1024 1164 1184 1344 600 620 623 635 51200 flags: ; type: preferred, driver
  props:
        22 ACTIVE:
                flags: range
                values: 0 1
                value: 1
```

图 40.6.1.3 stm 设备信息

从图 40.6.1.3 可以看出，Connectors 的 id 为 32，CRTC 的 id 为 35，这两个 ID 很重要，一会测试要用到，大家要根据自己的实际情况填写。

输入如下命令测试 DRM 驱动：

```
modetest -M stm -s 32@35:1024x600
```

命令参数介绍如下：

-M：指定 stm 模块。

-s：32 表示 connectors 的 ID，35 表示 CRTC 的 ID ,1024x600 表示显示的模式。

显示结果如图 40.6.1.4 所示：

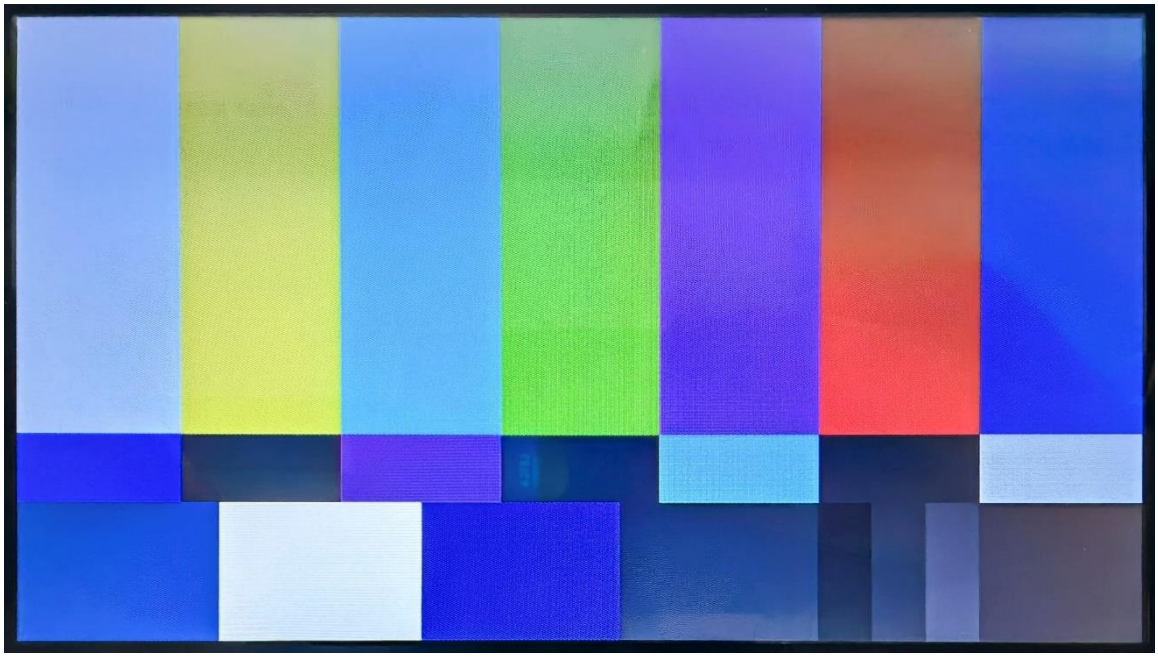


图 40.6.1.4 DRM 显示测试

40.6.2 LCD 屏幕的 FB 基本测试

在 40.2 小节里我们说了，KMS 包含了 FB 框架。DRM 驱动默认为 CRTC 用来控制，CRTC 是可以模仿 FB 框架，实现使用 FB 接口。示例代码 40.2.2.5 中的第 213 行就是负责初始化基于 CRTC 的 FB 接口。我们只需在 Linux 内核图形化配置界面里配置以下选项。

1、使能 DRM 驱动的 FB

配置路径如下：

```
> Device Drivers
    -> Graphics support
        -> Direct Rendering Manager (XFree86 4.1.0 and higher DRI support)
            -> [*]Enable legacy fbdev support for your modesetting driver    //选中
```

如图 40.6.2.1 所示：

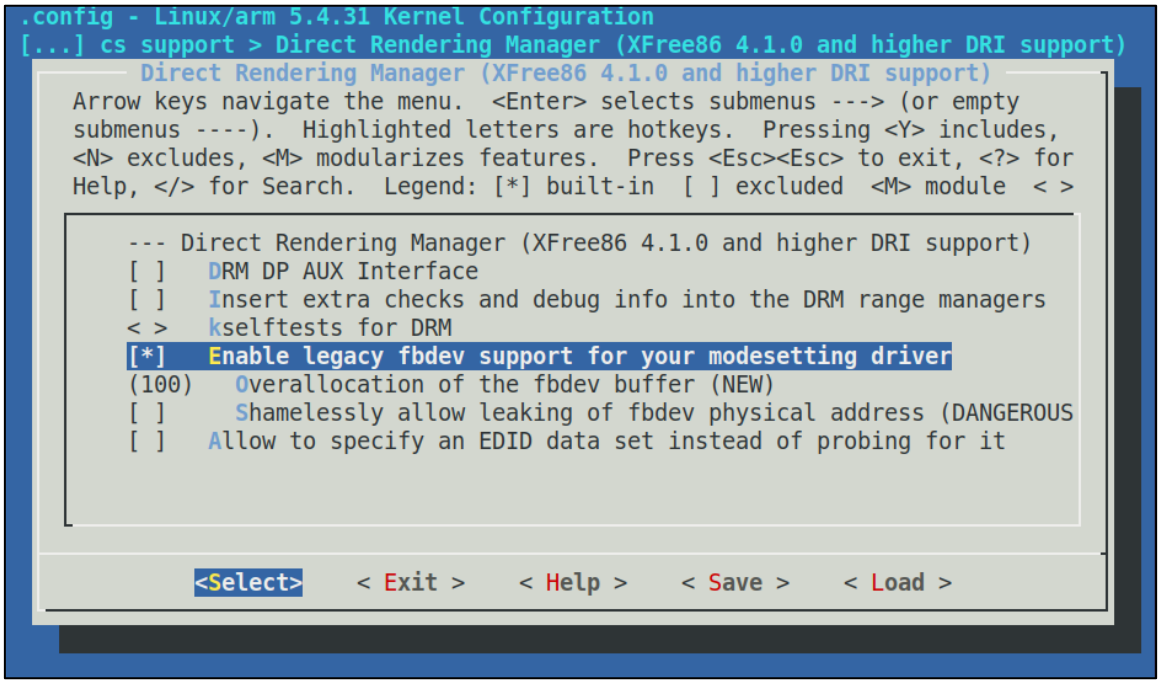


图 40.6.2.1 使能 DRM 的 FB 支持

2、使能 PL110

```
-> Device Drivers
    -> Graphics support
        -> Frame buffer Devices
            -> Support for frame buffer devices
                -> <*>ARM PrimeCell PL110 support //选中，支持/dev/fb0
```

配置如图 40.6.2.2 所示：

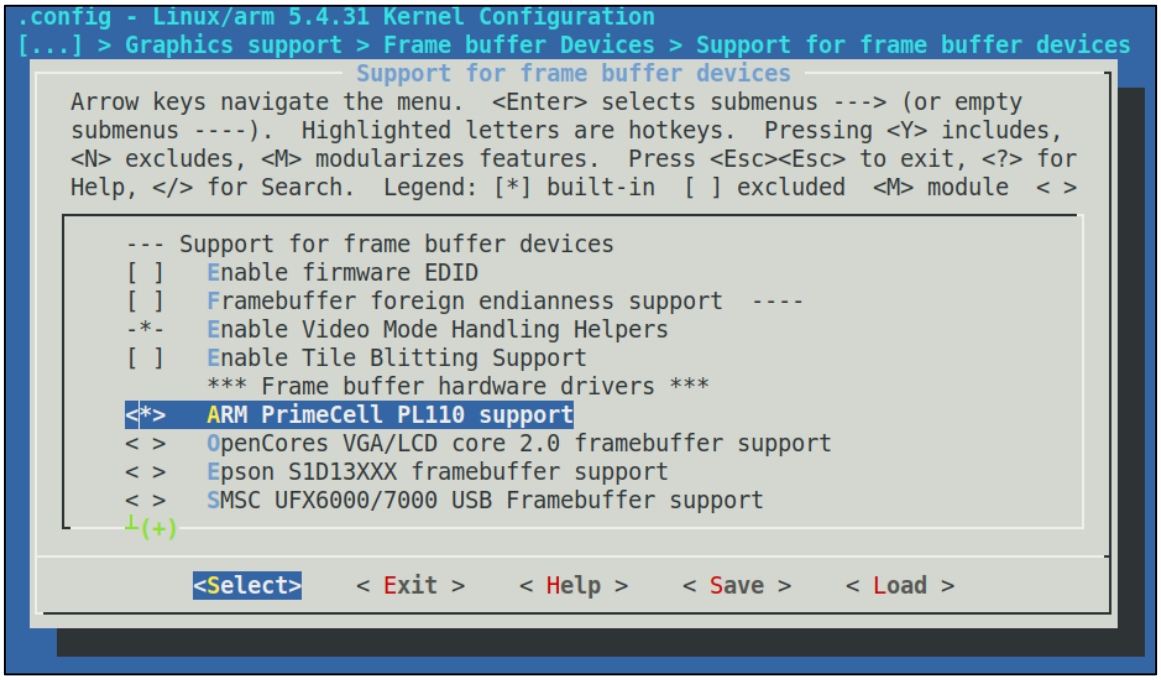


图 40.6.2.2 使能 PL100

3、使能 Linux logo 显示

Linux 内核启动的时候可以选择显示小企鹅 logo。打开 Linux 内核图形化配置界面，按下路径找到对应的配置项：

```
-> Device Drivers
    -> Graphics support
      -> [*] Bootup logo                                    //选中
          -> [*] Standard black and white Linux logo (NEW)  //选中
          -> [*] Standard 16-color Linux logo (NEW)         //选中
          -> [*] Standard 224-color Linux logo (NEW)        //选中
```

如图 40.6.2.3 所示：

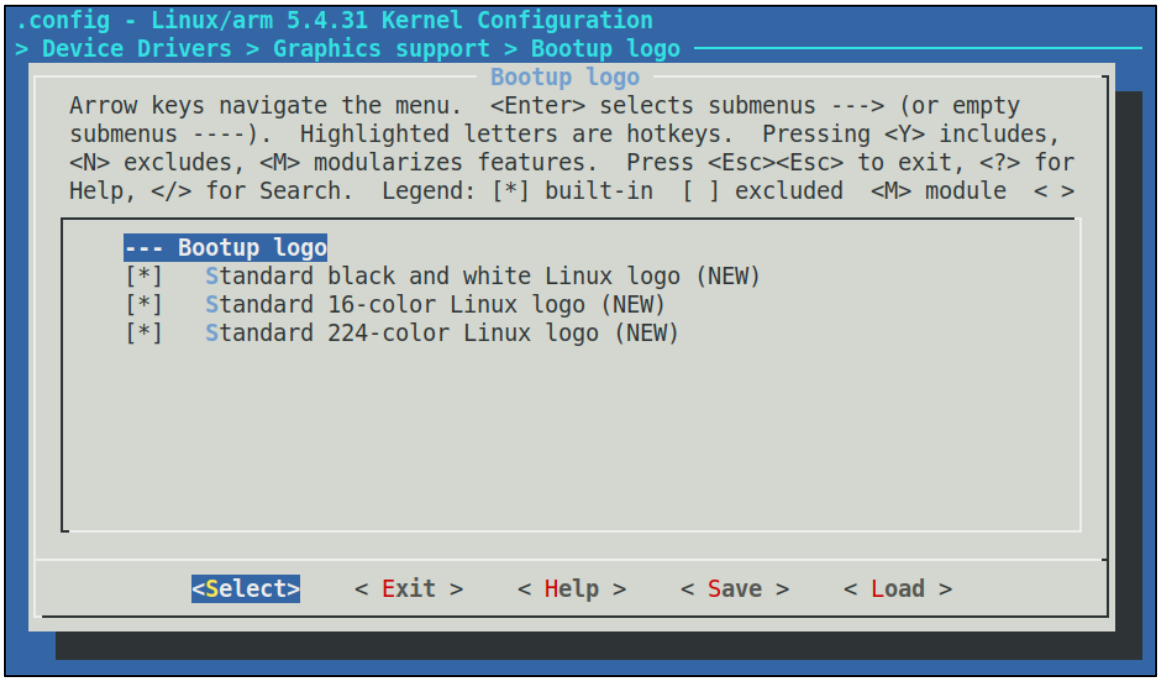


图 40.6.2.3 logo 配置项

图 40.6.2.3 中这三个选项分别对应黑白、16 位、24 位色彩格式的 logo，我们把这三个都选中，都编译进 Linux 内核里面。设置好以后保存退出，重新编译 Linux 内核，编译完成以后使用新编译出来 uImage 镜像启动系统，在 LCD 屏幕左上角出现两个彩色的小企鹅 logo(企鹅的个数和 CPU 的个数相同)，屏幕背景色为黑色，如图 40.6.2.3 所示：

图 40.6.2.3 Linux 启动 logo 显示

40.6.3 设置 LCD 作为终端控制台

LCD 作为终端控制台前提条件要实现 FB 接口。我们一直使用 MobaXterm 作为 Linux 开发板终端，开发板通过串口和 MobaXterm 进行通信。现在我们已经驱动起来 LCD 并且提供了 FB 接口，所以可以设置 LCD 作为终端，也就是开发板使用自己的显示设备作为自己的终端，接上键盘就可以直接在开发板上敲命令了，将 LCD 设置为终端控制台的方法如下：

1、设置 uboot 中的 bootargs

重启开发板，进入 Linux 命令行，重新设置 bootargs 参数的 console 内容，命令如下所示：

```
setenv bootargs 'console=tty1 console=ttySTM0,115200 root=/dev/nfs nfsroot=192.168.1.21:/home/liangwencong/linux/nfs/rootfs,proto=tcp rw ip=192.168.1.246:192.168.1.21:192.168.1.1:255.255.255.0::eth0:off'
```

注意红色字体部分设置 console，这里我们设置了两遍 console，第一次设置 console=tty1，也就是设置 LCD 屏幕为控制台，第二遍又设置 console=ttymxc0,115200，也就是设置串口也作为控制台。相当于我们打开了两个 console，一个是 LCD，一个是串口，大家重启开发板就会发现 LCD 和串口都会显示 Linux 启动 log 信息。但是此时我们还不能使用 LCD 作为终端进行交互，因为我们的设置还未完成。

2、修改/etc/inittab 文件

打开开发板根文件系统中的/etc/inittab 文件，在里面加入下面这一行：

```
tty1::askfirst:-/bin/sh
```

添加完成以后的/etc/inittab 文件内容如图 40.6.3.1 所示：

```
# Put a getty on the serial port
console::respawn:/sbin/getty -L  console 0 vt100 # GENERIC_SERIAL
tty1::askfirst:-/bin/sh
# Stuff to do for the 3-finger salute
#::ctrlaltdel:/sbin/reboot
```

打开tty1，设置LCD作为终端

图 40.6.3.1 修改后的/etc/inittab 文件

修改完成以后保存/etc/inittab 并退出，然后重启开发板，重启以后开发板 LCD 屏幕最后一行会显示下面一行语句：Please press Enter to activate this console.上述提示语句说的是：按下回车键使能当前终端。至此，我们就拥有了两套终端，一个是基于串口的 MobaXterm，一个就是我们开发板的 LCD 屏幕，但是为了方便调试，我们以后还是以 MobaXterm 为主。我们可以通过下面这一行命令向 LCD 屏幕输出“hello linux!”

```
echo hello linux! > /dev/tty1
```

屏幕显示如图 40.6.3.2 所示：

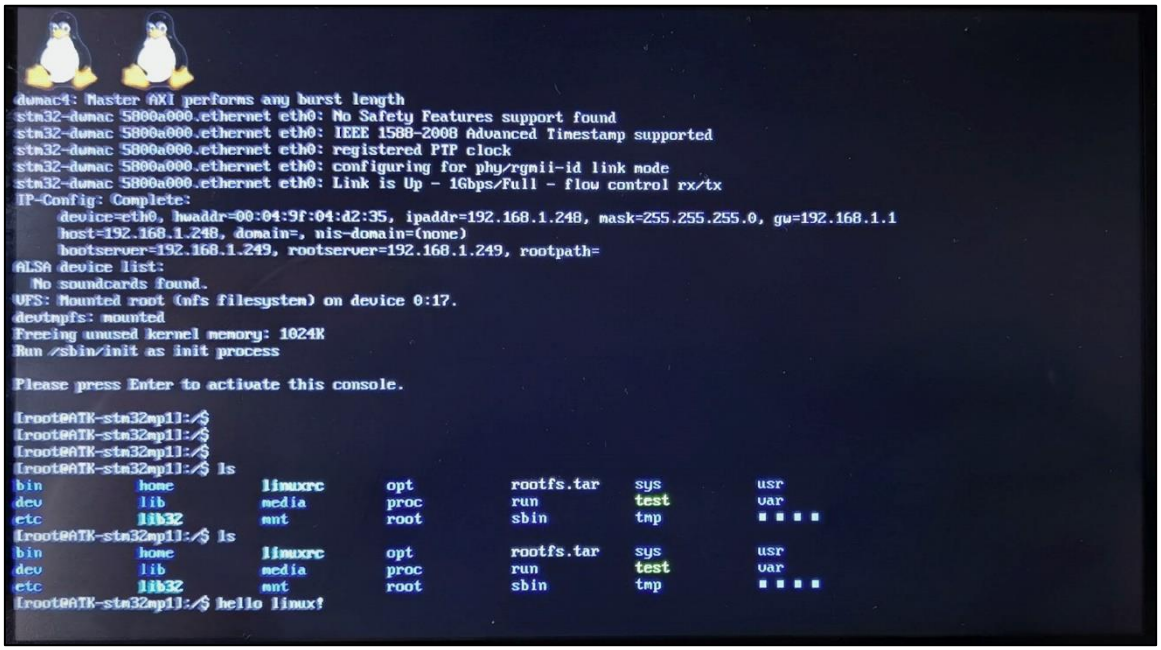


图 40.6.3.2 LCD 终端显示

40.6.4 LCD 背光调节

背光调节不用像上一章 PWM 实验那样，直接操作 pwmchipX(X=0~N)目录里面的文件，但是我们还是需要看一下。因为本章我们开启了 TIM4_CH2 这路 PWM，上一章开启了 TIM1_CH3 这路 PWM，相当于我们开启了两个不同的定时器对应的 PWM 通道。因此会在/sys/class/pwm 目录下存在两个 pwmchipX 目录，如下图所示：

```
[root@ATK-stm32mp1]:/sys/class/pwm$ ls
pwmchip0  pwmchip4
[root@ATK-stm32mp1]:/sys/class/pwm$
```

图 40.6.4.1 pwm 文件

从图 40.6.4.1 可以看出此时有 pwmchip0 和 pwmchip4，上一章 PWM 实验中 pwmchip0 对应 TIM1，那么本章是不是呢？进入 pwmchip0 目录，查看一下路径中的寄存器首地址，如图 40.6.4.2 所示：

```
[root@ATK-stm32mp1]:/sys/class/pwm$ cd pwmchip0/
[root@ATK-stm32mp1]:/sys/devices/platform/soc/40002000.timer/40002000.timer:pwm/pwm/pwmchip0$
```

图 40.6.4.2 pwmchip0 路径

从图 40.6.4.2 此时 pwmchip0 对应的定时器寄存器首地址为 0X40002000，这个正是 TIM4 的寄存器收地址！所以在本章 pwmchip0 对应的是 TIM4。同样的方法查看一下 pwmchip4 的地址，如图 40.6.4.3 所示：

```
[root@ATK-stm32mp1]:/sys/class/pwm$ cd pwmchip4/
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44000000.timer/44000000.timer:pwm/pwm/pwmchip4$
```

图 40.6.4.3 pwmchip4 路径

从图 40.6.4.3 可以看出，pwmchip4 对应的定时器寄存器首地址为 0X4000000，这个是 TIM1 定时器寄存器地址。所以在本章 pwmchip0 对应的是 TIM4，pwmchip4 对应的是 TIM1！

大家在开启多路 PWM 以后，一定要使用这个的方法来确定 TIM 对应的 pwmchip 文件！！

第 40.5 小节已经讲过了，背光设备树节点设置了 8 个等级的背光调节，可以设置为 0~7，我们可以通过设置背光等级来实现 LCD 背光亮度的调节，进入如下目录：

```
/sys/devices/platform/backlight/backlight/backlight
```

此目录下的文件如图 40.6.4.4 所示：

```
[root@ATK-stm32mp1]:~$ cd /sys/devices/platform/backlight/backlight/backlight/
[root@ATK-stm32mp1]:/sys/devices/platform/backlight/backlight/backlight$ ls
actual_brightness  device          scale       uevent
bl_power           max_brightness  subsystem
brightness         power           type
[root@ATK-stm32mp1]:/sys/devices/platform/backlight/backlight/backlight$
```

图 40.6.4.4 目录下的文件和子目录

图 40.6.4.4 中的 brightness 表示当前亮度等级，max_bgigntness 表示最大亮度等级。当前这两个文件内容如图 40.6.4.5 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/backlight/backlight/backlight$ cat max
_brightness
7
[root@ATK-stm32mp1]:/sys/devices/platform/backlight/backlight/backlight$ cat bri
ghtness
7
[root@ATK-stm32mp1]:/sys/devices/platform/backlight/backlight/backlight$
```

图 40.6.4.5 brightness 和 max_brightness 文件内容

从图 40.6.4.5 可以看出，当前屏幕亮度等级为 7，根据前面的分析可以，这个是 100%亮度。屏幕最大亮度等级为 7。如果我们要修改屏幕亮度，只需要向 brightness 写入需要设置的屏幕亮度等级即可。比如设置屏幕亮度等级为 6，那么可以使用如下命令：

```
echo 6 > brightness
```

输入上述命令以后就会发现屏幕亮度变暗了，如果设置 brightness 为 0 的话就会关闭 LCD 背光，屏幕就会熄灭。

第四十一章 Linux I2C 驱动实验

对于 I2C 我相信大家都很熟悉，基本上做过单片机开发的朋友都接触过，在电子产品硬件设计当中，I2C 是一种很常见的同步、串行、低速、近距离通信接口，用于连接各种 IC、传感器等器件，它们都会提供 I2C 接口与 SoC 主控相连，比如陀螺仪、加速度计、触摸屏等，其最大优势在于可以在总线上扩展多个外围设备的支持。

Linux 内核开发者为了让驱动开发工程师在内核中方便的添加自己的 I2C 设备驱动程序，更容易的在 linux 下驱动自己的 I2C 接口硬件，进而引入了 I2C 总线框架。与 Linux 下的 platform 虚拟总线不同的是，I2C 是实际的物理总线，所以 I2C 总线框架也是 Linux 下总线、设备、驱动模型的产物。

本章我们来学习一下如何在 Linux 下的 I2C 总线框架，以及如何使用 I2C 总线框架编写一个 I2C 接口的外设驱动程序；本章重点是学习 Linux 下的 I2C 总线框架。

41.1 I2C & AP3216C 简介

41.1.1 I2C 简介

I2C 是很常见的一种总线协议，I2C 是 NXP 公司设计的，I2C 使用两条线在主控制器和从机之间进行数据通信。一条是 SCL(串行时钟线)，另外一条是 SDA(串行数据线)，这两条数据线需要接上拉电阻，总线空闲的时候 SCL 和 SDA 处于高电平。I2C 总线标准模式下速度可以达到 100Kb/S，快速模式下可以达到 400Kb/S。I2C 总线工作是按照一定的协议来运行的，接下来就看一下 I2C 协议。

I2C 是支持多从机的，也就是一个 I2C 控制器下可以挂多个 I2C 从设备，这些不同的 I2C 从设备有不同的器件地址，这样 I2C 主控制器就可以通过 I2C 设备的器件地址访问指定的 I2C 设备了，一个 I2C 总线连接多个 I2C 设备如图 41.1.1.1 所示：

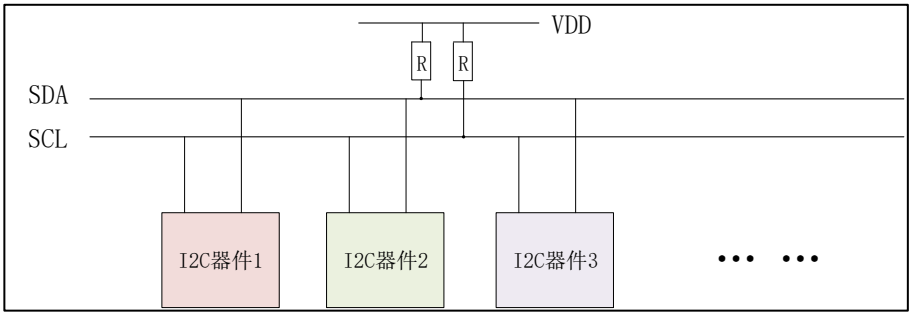


图 41.1.1.1 I2C 多个设备连接结构图

图 41.1.1.1 中 SDA 和 SCL 这两根线必须要接一个上拉电阻，一般是 4.7K。其余的 I2C 从器件都挂接到 SDA 和 SCL 这两根线上，这样就可以通过 SDA 和 SCL 这两根线来访问多个 I2C 设备。

接下来看一下 I2C 协议有关的术语：

1、起始位(钟高启停)

顾名思义，也就是 I2C 通信起始标志，通过这个起始位就可以告诉 I2C 从机，“我”要开始进行 I2C 通信了。在 SCL 为高电平的时候，SDA 出现下降沿就表示为起始位，如图 41.1.1.2 所示：

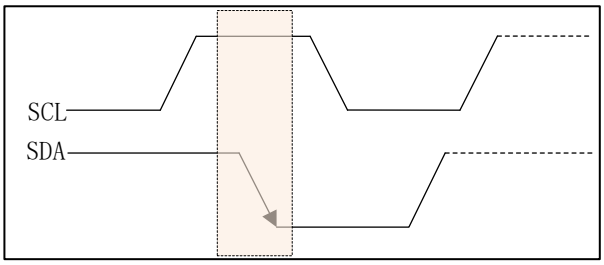


图 41.1.1.2 I2C 通信起始位

2、停止位

停止位就是停止 I2C 通信的标志位，和起始位的功能相反。在 SCL 位高电平的时候，SDA 出现上升沿就表示为停止位，如图 41.1.1.3 所示：

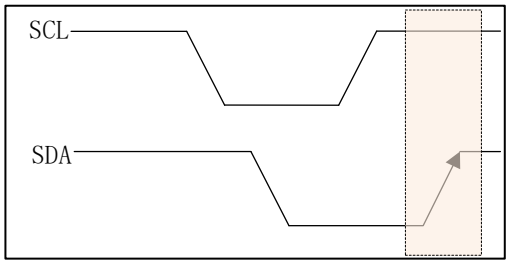


图 41.1.1.3 I2C 通信停止位

3、数据传输

I2C 总线在数据传输的时候要保证在 SCL 高电平期间，SDA 上的数据稳定，因此 SDA 上的数据变化只能在 SCL 低电平期间发生，如图 41.1.1.4 所示：

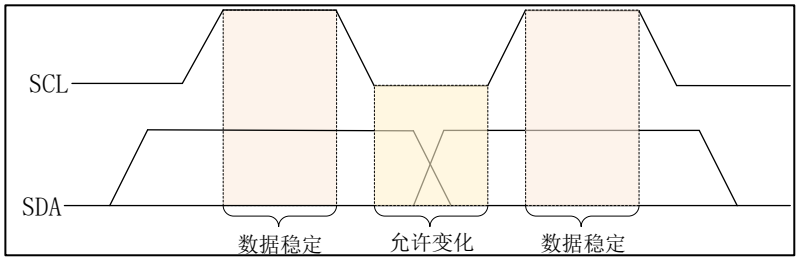


图 41.1.1.4 I2C 数据传输

4、应答信号(数低应答)

当 I2C 主机发送完 8 位数据以后会将 SDA 设置为输入状态，等待 I2C 从机应答，也就是等待 I2C 从机告诉主机它接收到数据了。应答信号是由从机发出的，主机需要提供应答信号所需的时钟，主机发送完 8 位数据以后紧跟着的一个时钟信号就是给应答信号使用的。从机通过将 SDA 拉低来表示发出应答信号，表示通信成功，否则表示通信失败。

5、I2C 写时序

主机通过 I2C 总线与从机之间进行通信不外乎两个操作：写和读，I2C 总线单字节写时序如图 41.1.1.5 所示：

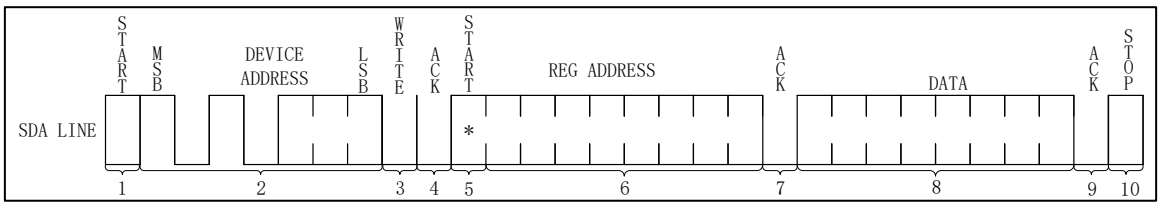


图 41.1.1.5 I2C 写时序

图 41.1.1.5 就是 I2C 写时序，我们来看一下写时序的具体步骤：

1)、开始信号。

2)、发送 I2C 设备地址，每个 I2C 器件都有一个设备地址，通过发送具体的设备地址来决定访问哪个 I2C 器件。这是一个 8 位的数据，其中高 7 位是设备地址，最后 1 位是读写位，为 1 的话表示这是一个读操作，为 0 的话表示这是一个写操作。

3)、 I2C 器件地址后面跟着一个读写位，为 0 表示写操作，为 1 表示读操作。

4)、从机发送的 ACK 应答信号。

5)、重新发送开始信号。

6)、发送要写写入数据的寄存器地址。

7)、从机发送的 ACK 应答信号。

8)、发送要写入寄存器的数据。

9)、从机发送的 ACK 应答信号。

10)、停止信号。

6、I2C 读时序

I2C 总线单字节读时序如图 41.1.1.6 所示：

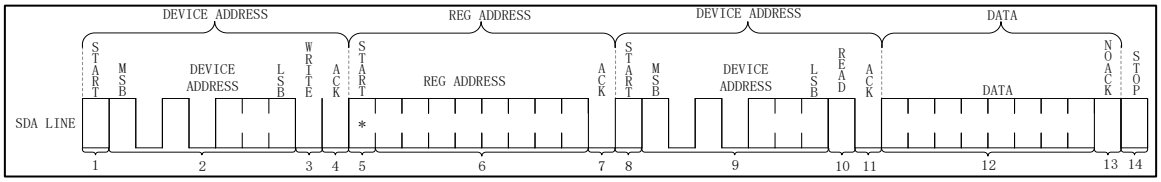


图 41.1.1.6 I2C 单字节读时序

I2C 单字节读时序比写时序要复杂一点，读时序分为 4 大步，第一步是发送设备地址，第二步是发送要读取的寄存器地址，第三步重新发送设备地址，最后一步就是 I2C 从器件输出要读取的寄存器值，我们具体来看一下这步。

1)、主机发送起始信号。

2)、主机发送要读取的 I2C 从设备地址。

3)、读写控制位，因为是向 I2C 从设备发送数据，因此是写信号。

4)、从机发送的 ACK 应答信号。

5)、重新发送 START 信号。

6)、主机发送要读取的寄存器地址。

7)、从机发送的 ACK 应答信号。

8)、重新发送 START 信号。

9)、重新发送要读取的 I2C 从设备地址。

10)、读写控制位，这里是读信号，表示接下来是从 I2C 从设备里面读取数据。

11)、从机发送的 ACK 应答信号。

12)、从 I2C 器件里面读取到的数据。

13)、主机发出 NO ACK 信号，表示读取完成，不需要从机再发送 ACK 信号了。

14)、主机发出 STOP 信号，停止 I2C 通信。

7、I2C 多字节读写时序

有时候我们需要读写多个字节，多字节读写时序和单字节的基本一致，只是在读写数据的时候可以连续发送多个自己的数据，其他的控制时序都是和单字节一样的。

41.1.2 STM32MP1 I2C 简介

STM32MP157D 有 6 个 I2C 接口，其中 I2C4 和 I2C6 可以在 A7 安全模式或者 A7 非安全模式下使用，M4 无法使用，STM32MP157 的 I2C 部分特性如下：

①、兼容 I2C 总线规范第 03 版。

②、支持从模式和主模式，支持多主模式功能。

③、支持标准模式 (Sm)、快速模式 (Fm) 和超快速模式 (Fm+)，其中，标准模式 100kHz，快速模式 400 kHz，超快速模式可以到 1 MHz。

④、7 位和 10 位寻址模式。

⑤、多个 7 位从地址，所有 7 位地址应答模式。

⑥、软件复位。
⑦、带 DMA 功能的 1 字节缓冲。
⑧、广播呼叫。
关于 STM32M157 IIC 更多详细的介绍，请参考《STM32MP157 参考手册》相关章节。

41.1.3 AP3216C 简介

STM32MP1 开发板上通过 I2C5 连接了一个三合一环境传感器：AP3216C，AP3216C 是由敦南科技推出的一款传感器，其支持环境光强度(ALS)、接近距离(PS)和红外线强度(IR)这三个环境参数检测。该芯片可以通过 IIC 接口与主控制相连，并且支持中断，AP3216C 的特点如下：
①、I2C 接口，快速模式下波特率可以到 400Kbit/S
②、多种工作模式选择：ALS、PS+IR、ALS+PS+IR、PD 等等。
③、内建温度补偿电路。
④、宽工作温度范围(-30° C ~ +80° C)。
⑤、超小封装，4.1mm x 2.4mm x 1.35mm
⑥、环境光传感器具有 16 位分辨率。
⑦、接近传感器和红外传感器具有 10 位分辨率。
AP3216C 常被用于手机、平板、导航设备等，其内置的接近传感器可以用于检测是否有物体接近，比如手机上用来检测耳朵是否接触听筒，如果检测到的话就表示正在打电话，手机就会关闭手机屏幕以省电。也可以使用环境光传感器检测光照强度，可以实现自动背光亮度调节。AP3216C 结构如图 41.1.3.1 所示：

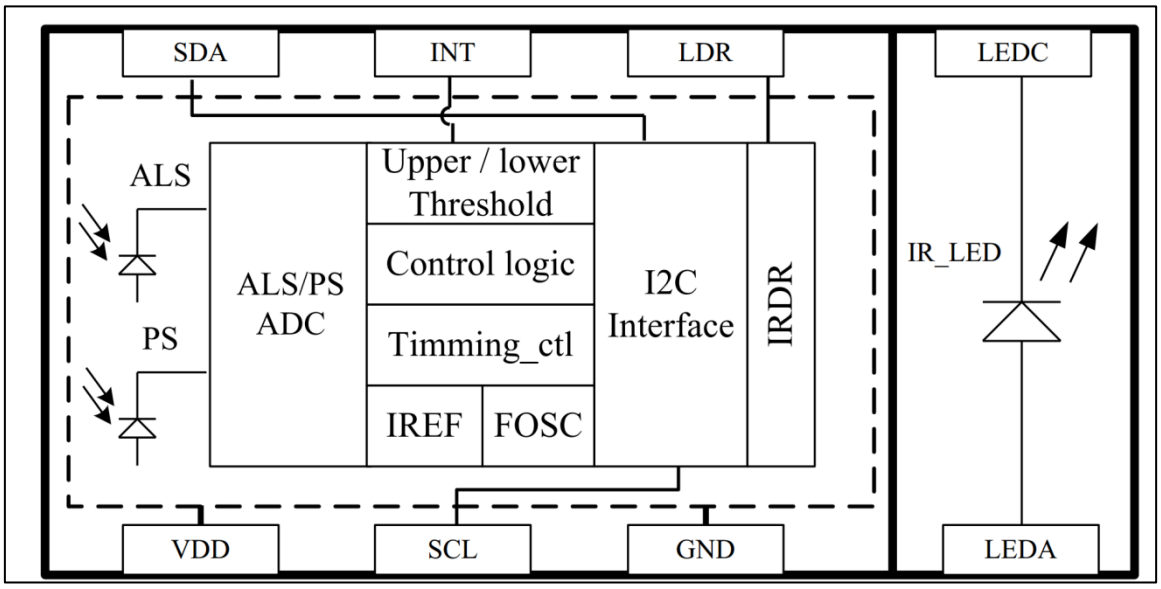


图 41.1.3.1 AP3216C 结构图

AP3216 的设备地址为 0X1E，同几乎所有的 I2C 从器件一样，AP3216C 内部也有一些寄存器，通过这些寄存器我们可以配置 AP3216C 的工作模式，并且读取相应的数据。AP3216C 我们用的寄存器如表 41.1.3.1 所示：

寄存器地址	位	寄存器功能	描述
0X00	2:0	系统模式	000：掉电模式(默认)。 001：使能 ALS。 010：使能 PS+IR。 011：使能 ALS+PS+IR。

			100：软复位。 101：ALS 单次模式。 110：PS+IR 单次模式。 111：ALS+PS+IR 单次模式。
0X0A	7	IR 低位数据	0：IR&PS 数据有效，1:无效
	1:0		IR 最低 2 位数据。
0X0B	7:0	IR 高位数据	IR 高 8 位数据。
0X0C	7:0	ALS 低位数据	ALS 低 8 位数据。
0X0D	7:0	ALS 高位数据	ALS 高 8 位数据。
0X0E	7	PS 低位数据	0，物体在远离；1，物体在接近。
	6		0，IR&PS 数据有效；1，IR&PS 数据无效
	3:0		PS 最低 4 位数据。
0X0F	7	PS 高位数据	0，物体在远离；1，物体在接近。
	6		0，IR&PS 数据有效；1，IR&PS 数据无效
	5:0		PS 最低 6 位数据。

表 41.1.3.1 本章使用的 AP3216C 寄存器表

在表 41.1.3.1 中，0X00 这个寄存器是模式控制寄存器，用来设置 AP3216C 的工作模式，一般开始先将其设置为 0X04，也就是先软件复位一次 AP3216C。接下来根据实际使用情况选择合适的工作模式，比如设置为 0X03，也就是开启 ALS+PS+IR。0X0A~0X0F 这 6 个寄存器就是数据寄存器，保存着 ALS、PS 和 IR 这三个传感器获取到的数据值。如果同时打开 ALS、PS 和 IR 的读取间隔最少要 112.5ms，因为 AP3216C 完成一次转换需要 112.5ms。关于 AP3216C 的介绍就到这里，如果要想详细的研究此芯片的话，请大家自行查阅其数据手册。

41.2 Linux I2C 总线框架简介

使用裸机的方式编写一个 I2C 器件的驱动程序，我们一般需要实现两部分：

①、I2C 主机驱动。

②、I2C 设备驱动。

I2C 主机驱动也就是 SoC 的 I2C 控制器对应的驱动程序，I2C 设备驱动其实就是挂在 I2C 总线下的具体设备对应的驱动程序，例如 eeprom、触摸屏 IC、传感器 IC 等；对于主机驱动来说，一旦编写完成就不需要再做修改，其他的 I2C 设备直接调用主机驱动提供的 API 函数完成读写操作即可。这个正好符合 Linux 的驱动分离与分层的思想，因此 Linux 内核也将 I2C 驱动分为两部分。

Linux 内核开发者为了让驱动开发工程师在内核中方便的添加自己的 I2C 设备驱动程序，方便大家更容易的在 linux 下驱动自己的 I2C 接口硬件，进而引入了 I2C 总线框架，我们一般也叫作 I2C 子系统，Linux 下 I2C 子系统总体框架如下所示：

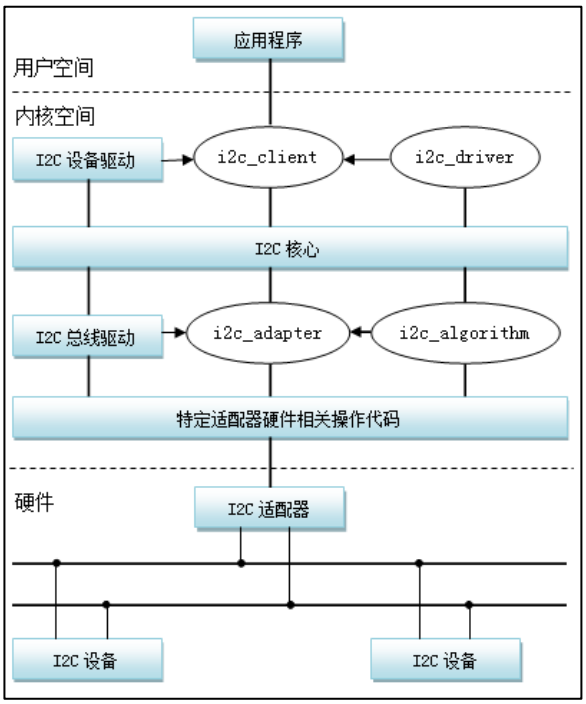


图 41.2.3.1 I2C 子系统框架图

从图 41.2.3.1 可以知道，I2C 子系统分为三大组成部分：

1、I2C 核心(I2C-core)

I2C 核心提供了 I2C 总线驱动（适配器）和设备驱动的注册、注销方法，I2C 通信方法(algorithm)与具体硬件无关的代码，以及探测设备地址的上层代码等；

2、I2C 总线驱动(I2C adapter)

I2C 总线驱动是 I2C 适配器的软件实现，提供 I2C 适配器与从设备间完成数据通信的能力。I2C 总线驱动由 i2c_adapter 和 i2c_algorithm 来描述。I2C 适配器是 SoC 中内置 i2c 控制器的软件抽象，可以理解为他所代表的是一个 I2C 主机；

3、I2C 设备驱动(I2C client driver)

包括两部分：设备的注册和驱动的注册。

I2C 子系统帮助内核统一管理 I2C 设备，让驱动开发工程师在内核中可以更加容易地添加自己的 I2C 设备驱动程序。

41.2.1 I2C 总线驱动

首先来看一下 I2C 总线，在讲 platform 的时候就说过，platform 是虚拟出来的一条总线，目的是为了实现总线、设备、驱动框架。对于 I2C 而言，不需要虚拟出一条总线，直接使用 I2C 总线即可。I2C 总线驱动重点是 I2C 适配器(也就是 SoC 的 I2C 接口控制器)驱动，这里要用到两个重要的数据结构：i2c_adapter 和 i2c_algorithm，I2C 子系统将 SoC 的 I2C 适配器(控制器)抽象成一个 i2c_adapter 结构体，i2c_adapter 结构体定义在 include/linux/i2c.h 文件中，结构体内容如下：

示例代码 41.2.1 i2c_adapter 结构体

```
685 struct i2c_adapter {
686     struct module *owner;
```

```
687     unsigned int class;
688     const struct i2c_algorithm *algo;
689     void *algo_data;
690
691     /* data fields that are valid for all devices   */
692     const struct i2c_lock_operations *lock_ops;
693     struct rt_mutex bus_lock;
694     struct rt_mutex mux_lock;
695
696     int timeout;                /* in jiffies */
697     int retries;
698     struct device dev;          /* the adapter device */
699     unsigned long locked_flags; /* owned by the I2C core */
700 #define I2C_ALF_IS_SUSPENDED          0
701 #define I2C_ALF_SUSPEND_REPORTED      1
702
703     int nr;
704     char name[48];
705     struct completion dev_released;
706
707     struct mutex userspace_clients_lock;
708     struct list_head userspace_clients;
709
710     struct i2c_bus_recovery_info *bus_recovery_info;
711     const struct i2c_adapter_quirks *quirks;
712
713     struct irq_domain *host_notify_domain;
714 };
```

第 688 行，i2c_algorithm 类型的指针变量 algo，对于一个 I2C 适配器，肯定要对外提供读写 API 函数，设备驱动程序可以使用这些 API 函数来完成读写操作。i2c_algorithm 就是 I2C 适配器与 IIC 设备进行通信的方法。

i2c_algorithm 结构体定义在 include/linux/i2c.h 文件中，内容如下：

示例代码 41.2.1.2 i2c_algorithm 结构体

```
526 struct i2c_algorithm {
527     /*
528      * If an adapter algorithm can't do I2C-level access, set
529      * master_xfer to NULL. If an adapter algorithm can do SMBus
530      * access, set smbus_xfer. If set to NULL, the SMBus protocol is
531      * simulated using common I2C messages.
532      *
533      * master_xfer should return the number of messages successfully
534      * processed, or a negative value on error
535      */
```

```
536     int (*master_xfer)(struct i2c_adapter *adap,
                           struct i2c_msg *msgs,
537                int num);
538     int (*master_xfer_atomic)(struct i2c_adapter *adap,
539                   struct i2c_msg *msgs, int num);
540     int (*smbus_xfer)(struct i2c_adapter *adap, u16 addr,
541               unsigned short flags, char read_write,
542               u8 command, int size, union i2c_smbus_data *data);
543     int (*smbus_xfer_atomic)(struct i2c_adapter *adap, u16 addr,
544                 unsigned short flags, char read_write,
545                 u8 command, int size, union i2c_smbus_data *data);
546
547     /* To determine what the adapter supports */
548     u32 (*functionality)(struct i2c_adapter *adap);
549
550 #if IS_ENABLED(CONFIG_I2C_SLAVE)
551     int (*reg_slave)(struct i2c_client *client);
552     int (*unreg_slave)(struct i2c_client *client);
553 #endif
554 };
```

第 536 行，master_xfer 就是 I2C 适配器的传输函数，可以通过此函数来完成与 IIC 设备之间的通信。

第 540 行，smbus_xfer 就是 SMBUS 总线的传输函数。smbus 协议是从 I2C 协议的基础上发展而来的，他们之间有很大的相似度，SMBus 与 I2C 总线之间在时序特性上存在一些差别，应用于移动 PC 和桌面 PC 系统中的低速率通讯。

综上所述，I2C 总线驱动，或者说 I2C 适配器驱动的主要工作就是初始化 i2c_adapter 结构体变量，然后设置 i2c_algorithm 中的 master_xfer 函数。完成以后通过 i2c_add_numbered_adapter 或 i2c_add_adapter 这两个函数向 I2C 子系统注册设置好的 i2c_adapter，这两个函数的原型如下：

```
int i2c_add_adapter(struct i2c_adapter *adapter)
int i2c_add_numbered_adapter(struct i2c_adapter *adap)
```

这两个函数的区别在于 i2c_add_adapter 会动态分配一个总线编号，而 i2c_add_numbered_adapter 函数则指定一个静态的总线编号。函数参数和返回值含义如下：

adapter 或 adap：要添加到 Linux 内核中的 i2c_adapter，也就是 I2C 适配器。

返回值：0，成功；负值，失败。

如果要删除 I2C 适配器的话使用 i2c_del_adapter 函数即可，函数原型如下：

```
void i2c_del_adapter(struct i2c_adapter * adap)
```

函数参数和返回值含义如下：

adap：要删除的 I2C 适配器。

返回值：无。

关于 I2C 的总线(控制器或适配器)驱动就讲解到这里，一般 SoC 的 I2C 总线驱动都是由半导体厂商编写的，比如 STM32MP1 的 I2C 适配器驱动 ST 官方已经编写好了，这个不需要用户去编写。因此 I2C 总线驱动对我们这些 SoC 使用者来说是被屏蔽掉的，我们只要专注于 I2C 设备驱动即可，除非你是在半导体公司上班，工作内容就是写 I2C 适配器驱动。

41.2.2 I2C 总线设备

I2C 设备驱动重点关注两个数据结构：i2c_client 和 i2c_driver，根据总线、设备和驱动模型，I2C 总线上一小节已经讲了。还剩下设备和驱动，i2c_client 用于描述 I2C 总线下的设备，i2c_driver 则用于描述 I2C 总线下的设备驱动，类似于 platform 总线下的 platform_device 和 platform_driver。

1、i2c_client 结构体

i2c_client 结构体定义在 include/linux/i2c.h 文件中，内容如下：

示例代码 41.2.2.1 i2c_client 结构体

```
313    struct i2c_client {
314        unsigned short flags;           /* div., see below      */
......
328        struct i2c_adapter *adapter;    /* the adapter we sit on */
329        struct device dev;              /* the device structure  */
330        int init_irq;           /* irq set at initialization    */
331        int irq;                /* irq issued by device         */
332        struct list_head detected;
333    #if IS_ENABLED(CONFIG_I2C_SLAVE)
334        i2c_slave_cb_t slave_cb;    /* callback for slave mode   */
335    #endif
336    };
```

一个 I2C 设备对应一个 i2c_client 结构体变量，系统每检测到一个 I2C 从设备就会给这个设备分配一个 i2c_client。

2、i2c_driver 结构体

i2c_driver 类似 platform_driver，是我们编写 I2C 设备驱动重点要处理的内容，i2c_driver 结构体定义在 include/linux/i2c.h 文件中，内容如下：

示例代码 41.2.2.2 i2c_driver 结构体

```
253    struct i2c_driver {
254        unsigned int class;
255
256        /* Standard driver model interfaces */
257        int (*probe)(struct i2c_client *client,
                    const struct i2c_device_id *id);
258        int (*remove)(struct i2c_client *client);
259
260        /* New driver model interface to aid the seamless removal of
261         * the current probe()'s, more commonly unused than used
262         second parameter.*/
263        int (*probe_new)(struct i2c_client *client);
264
265      /* driver model interfaces that don't relate to enumeration  */
266        void (*shutdown)(struct i2c_client *client);
```

```
267
268         /* Alert callback, for example for the SMBus alert protocol.
269          * The format and meaning of the data value depends on the
270          * protocol. For the SMBus alert protocol, there is a single
271          * bit of data passed as the alert response's low bit ("event
272          * flag"). For the SMBus Host Notify protocol, the data
273          * corresponds to the 16-bit payload data reported by the
274          slave device acting as master.*/
275         void (*alert)(struct i2c_client *client,
                          enum i2c_alert_protocol protocol,
276                       unsigned int data);
277
278         /* a ioctl like command that can be used to perform specific
279          * functions with the device.
280          */
281         int (*command)(struct i2c_client *client, unsigned int cmd,
                          void *arg);
282
283         struct device_driver driver;
284         const struct i2c_device_id *id_table;
285
286         /* Device detection callback for automatic device creation */
287         int (*detect)(struct i2c_client *client,
                          struct i2c_board_info *info);
288         const unsigned short *address_list;
289         struct list_head clients;
290
291         bool disable_i2c_core_irq_mapping;
292     };
```

第 257 行，当 I2C 设备和驱动匹配成功以后 probe 函数就会执行，和 platform 驱动一样。

第 283 行，device_driver 驱动结构体，如果使用设备树的话，需要设置 device_driver 的 of_match_table 成员变量，也就是驱动的兼容(compatible)属性。

第 284 行，id_table 是传统的、未使用设备树的设备匹配 ID 表。

对于我们 I2C 设备驱动编写人来说，重点工作就是构建 i2c_driver，构建完成以后需要向 I2C 子系统注册这个 i2c_driver。i2c_driver 注册函数为 int i2c_register_driver，此函数原型如下：

```
int i2c_register_driver(struct module      *owner,
                        struct i2c_driver  *driver)
```

函数参数和返回值含义如下：

owner：一般为 THIS_MODULE。

driver：要注册的 i2c_driver。

返回值：0，成功；负值，失败。

另外 i2c_add_driver 也常常用于注册 i2c_driver，i2c_add_driver 是一个宏，定义如下：

示例代码 41.2.2.3 i2c_add_driver 宏

```
844     #define i2c_add_driver(driver) \
845         i2c_register_driver(THIS_MODULE, driver)
```

i2c_add_driver 就是对 i2c_register_driver 做了一个简单的封装，只有一个参数，就是要注册的 i2c_driver。

注销 I2C 设备驱动的时候需要将前面注册的 i2c_driver 从 I2C 子系统中注销掉，需要用到 i2c_del_driver 函数，此函数原型如下：

```
void i2c_del_driver(struct i2c_driver    *driver)
```

函数参数和返回值含义如下：

driver：要注销的 i2c_driver。

返回值：无。

i2c_driver 的注册示例代码如下：

示例代码 41.2.2.4 i2c_driver 注册流程

```
1  /* i2c 驱动的 probe 函数 */
2  static int xxx_probe(struct i2c_client *client,
                        const struct i2c_device_id *id)
3  {
4      /* 函数具体程序 */
5      return 0;
6  }
7
8  /* i2c 驱动的 remove 函数 */
9  static int ap3216c_remove(struct i2c_client *client)
10 {
11     /* 函数具体程序 */
12     return 0;
13 }
14
15 /* 传统匹配方式 ID 列表 */
16 static const struct i2c_device_id xxx_id[] = {
17     {"xxx", 0},
18     {}
19 };
20
21 /* 设备树匹配列表 */
22 static const struct of_device_id xxx_of_match[] = {
23     { .compatible = "xxx" },
24     { /* Sentinel */ }
25 };
26
27 /* i2c 驱动结构体 */
28 static struct i2c_driver xxx_driver = {
29     .probe = xxx_probe,
30     .remove = xxx_remove,
```

```
31      .driver = {
32          .owner = THIS_MODULE,
33          .name = "xxx",
34          .of_match_table = xxx_of_match,
35          },
36          .id_table = xxx_id,
37      };
38
39  /* 驱动入口函数 */
40  static int __init xxx_init(void)
41  {
42      int ret = 0;
43
44      ret = i2c_add_driver(&xxx_driver);
45      return ret;
46  }
47
48  /* 驱动出口函数 */
49  static void __exit xxx_exit(void)
50  {
51      i2c_del_driver(&xxx_driver);
52  }
53
54  module_init(xxx_init);
55  module_exit(xxx_exit);
```

第 16~19 行，i2c_device_id，无设备树的时候匹配 ID 表。

第 22~25 行，of_device_id，设备树所使用的匹配表。

第 28~37 行，i2c_driver，当 I2C 设备和 I2C 驱动匹配成功以后 probe 函数就会执行，这些和 platform 驱动一样，probe 函数里面基本就是标准的字符设备驱动那一套了。

41.2.3 I2C 设备和驱动匹配过程

I2C 设备和驱动的匹配过程是由 I2C 子系统核心层来完成的，drivers/i2c/i2c-core-base.c 就是 I2C 的核心部分，I2C 核心提供了一些与具体硬件无关的 API 函数，比如前面讲过的：

1、i2c_adapter 注册/注销函数

```
int i2c_add_adapter(struct i2c_adapter *adapter)
int i2c_add_numbered_adapter(struct i2c_adapter *adap)
void i2c_del_adapter(struct i2c_adapter * adap)
```

2、i2c_driver 注册/注销函数

```
int i2c_register_driver(struct module *owner, struct i2c_driver *driver)
int i2c_add_driver (struct i2c_driver *driver)
void i2c_del_driver(struct i2c_driver *driver)
```

设备和驱动的匹配过程也是由核心层完成的，I2C 总线的数据结构为 i2c_bus_type，定义在

drivers/i2c/i2c-core-base.c 文件，i2c_bus_type 内容如下：

示例代码 41.2.3.1 i2c_bus_type 结构体

```
492 struct bus_type i2c_bus_type = {
493     .name       = "i2c",
494     .match      = i2c_device_match,
495     .probe      = i2c_device_probe,
496     .remove     = i2c_device_remove,
497     .shutdown   = i2c_device_shutdown,
498 };
```

.match 就是 I2C 总线的设备和驱动匹配函数，在这里就是 i2c_device_match 这个函数，此函数内容如下：

示例代码 41.2.3.2 i2c_device_match 函数

```
93  static int i2c_device_match(struct device *dev,
                                struct device_driver *drv)
94  {
95      struct i2c_client   *client = i2c_verify_client(dev);
96      struct i2c_driver   *driver;
97
98
99      /* Attempt an OF style match */
100     if (i2c_of_match_device(drv->of_match_table, client))
101         return 1;
102
103     /* Then ACPI style match */
104     if (acpi_driver_match_device(dev, drv))
105         return 1;
106
107     driver = to_i2c_driver(drv);
108
109     /* Finally an I2C match */
110     if (i2c_match_id(driver->id_table, client))
111         return 1;
112
113     return 0;
114 }
```

第 100 行，i2c_of_match_device 函数用于完成设备树中定义的设备与驱动匹配过程。比较 I2C 设备节点的 compatible 属性和 of_device_id 中的 compatible 属性是否相等，如果相当的话就表示 I2C 设备和驱动匹配。

第 104 行，acpi_driver_match_device 函数用于 ACPI 形式的匹配。

第 110 行，i2c_match_id 函数用于传统的、无设备树的 I2C 设备和驱动匹配过程。比较 I2C 设备名字和 i2c_device_id 的 name 字段是否相等，相等的话就说明 I2C 设备和驱动匹配成功。

41.3 STM32MP1 I2C 适配器驱动分析

上一小节我们讲解了 Linux 下的 I2C 子系统，重点分为 I2C 适配器驱动和 I2C 设备驱动，其中 I2C 适配器驱动就是 SoC 的 I2C 控制器驱动。I2C 设备驱动是需要用户根据不同的 I2C 从设备去编写，而 I2C 适配器驱动一般都是 SoC 厂商去编写的，比如 ST 就已经提供了 STM3MP21 的 I2C 适配器驱动程序。在内核源码 arch/arm/boot/dts/stm32mp151.dtsi 设备树文件中找到 STM32MP1 的 I2C 控制器节点，节点内容如下所示：

示例代码 41.3.1 I2C1 控制器节点

```
590     i2c1: i2c@40012000 {
591         compatible = "st,stm32mp15-i2c";
592         reg = <0x40012000 0x400>;
593         interrupt-names = "event", "error";
594         interrupts-extended = <&exti 21 IRQ_TYPE_LEVEL_HIGH>,
595                       <&intc GIC_SPI 32 IRQ_TYPE_LEVEL_HIGH>;
596         clocks = <&rcc I2C1_K>;
597         resets = <&rcc I2C1_R>;
598         #address-cells = <1>;
599         #size-cells = <0>;
600         dmas = <&dmamux1 33 0x400 0x80000001>,
601                <&dmamux1 34 0x400 0x80000001>;
602         dma-names = "rx", "tx";
603         power-domains = <&pd_core>;
604         st,syscfg-fmp = <&syscfg 0x4 0x1>;
605         wakeup-source;
606         status = "disabled";
607     };
```

重点关注 i2c1 节点的 compatible 属性值，因为通过 compatible 属性值可以在 Linux 源码里面找到对应的驱动文件。这里 i2c1 节点的 compatible 属性值“st,stm32mp15-i2c”，在 Linux 源码中搜索这个字符串即可找到对应的驱动文件。STM32MP1 的 I2C 适配器驱动驱动文件为 drivers/i2c/busses/i2c-stm32f7.c，在此文件中有如下内容：

示例代码 41.3.2 i2c-stm32f7.c 文件代码段

```
2520    static const struct of_device_id stm32f7_i2c_match[] = {
2521     { .compatible = "st,stm32f7-i2c", .data = &stm32f7_setup},
2522     { .compatible = "st,stm32mp15-i2c", .data = &stm32mp15_setup},
2523     {},
2524    };
2525    MODULE_DEVICE_TABLE(of, stm32f7_i2c_match);
2526
2527    static struct platform_driver stm32f7_i2c_driver = {
2528        .driver = {
2529            .name = "stm32f7-i2c",
2530            .of_match_table = stm32f7_i2c_match,
2531            .pm = &stm32f7_i2c_pm_ops,
```

```
2532        },
2533        .probe = stm32f7_i2c_probe,
2534        .remove = stm32f7_i2c_remove,
2535    };
2536
2537    module_platform_driver(stm32f7_i2c_driver);
```

从示例代码 41.3.2 可以看出，STM32MP1 的 I2C 适配器驱动是个标准的 platform 驱动，由此可以看出，虽然 I2C 总线为别的设备提供了一种总线驱动框架，但是 I2C 适配器却是 platform 驱动。就像你的部门老大是你的领导，你是他的下属，但是放到整个公司，你的部门老大却也是老板的下属。

第 2529 行，“st,stm32mp15-i2c”属性值，设备树中 i2c1 节点的 compatible 属性值就是与此匹配上的。因此 i2c-stm32f7.c 文件就是 STM32MP1 的 I2C 适配器驱动文件。

第 2533 行，当设备和驱动匹配成功以后 stm32f7_i2c_probe 函数就会执行，stm32f7_i2c_probe 函数就会完成 I2C 适配器初始化工作。

stm32f7_i2c_probe 函数内容如下所示(有省略)：

示例代码 41.3.3 stm32f7_i2c_probe 函数代码段

```
2106    static int stm32f7_i2c_probe(struct platform_device *pdev)
2107    {
2108        struct stm32f7_i2c_dev *i2c_dev;
2109        const struct stm32f7_i2c_setup *setup;
2110        struct resource *res;
2111        u32 rise_time, fall_time;
2112        struct i2c_adapter *adap;
2113        struct reset_control *rst;
2114        dma_addr_t phy_addr;
2115        int irq_error, ret;
2116
2117        i2c_dev = devm_kzalloc(&pdev->dev, sizeof(*i2c_dev),
                                   GFP_KERNEL);
2118        if (!i2c_dev)
2119            return -ENOMEM;
2120
2121        res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
2122        i2c_dev->base = devm_ioremap_resource(&pdev->dev, res);
2123        if (IS_ERR(i2c_dev->base))
2124            return PTR_ERR(i2c_dev->base);
2125        phy_addr = (dma_addr_t)res->start;
2126
2127        i2c_dev->irq_event = platform_get_irq(pdev, 0);
2128        if (i2c_dev->irq_event <= 0) {
2129            if (i2c_dev->irq_event != -EPROBE_DEFER)
2130                dev_err(&pdev->dev, "Failed to get IRQ event: %d\n",
2131                    i2c_dev->irq_event);
```

```
2132            return i2c_dev->irq_event ? : -ENOENT;
2133        }
2134
2135        irq_error = platform_get_irq(pdev, 1);
2136        if (irq_error <= 0) {
2137            if (irq_error != -EPROBE_DEFER)
2138                dev_err(&pdev->dev, "Failed to get IRQ error: %d\n",
2139                    irq_error);
2140            return irq_error ? : -ENOENT;
2141        }
......
2159        ret = device_property_read_u32(&pdev->dev, "clock-frequency",
2160                           &i2c_dev->bus_rate);
2161        if (ret)
2162            i2c_dev->bus_rate = I2C_STD_RATE;
2163
2164        if (i2c_dev->bus_rate > I2C_FASTPLUS_RATE) {
2165            dev_err(&pdev->dev, "Invalid bus speed (%i>%i)\n",
2166                i2c_dev->bus_rate, I2C_FASTPLUS_RATE);
2167            return -EINVAL;
2168        }
......
2183
2184        ret = devm_request_threaded_irq(&pdev->dev,
                                            i2c_dev->irq_event,
2185                        stm32f7_i2c_isr_event,
2186                        stm32f7_i2c_isr_event_thread,
2187                        IRQF_ONESHOT,
2188                        pdev->name, i2c_dev);
2189        if (ret) {
2190            dev_err(&pdev->dev, "Failed to request irq event %i\n",
2191                i2c_dev->irq_event);
2192            goto clk_free;
2193        }
2194
2195        ret = devm_request_irq(&pdev->dev, irq_error,
                                   stm32f7_i2c_isr_error, 0,
2196                       pdev->name, i2c_dev);
2197        if (ret) {
2198            dev_err(&pdev->dev, "Failed to request irq error %i\n",
2199                irq_error);
2200            goto clk_free;
2201        }
```

```
2226        if (i2c_dev->bus_rate > I2C_FAST_RATE) {
2227            ret = stm32f7_i2c_setup_fm_plus_bits(pdev, i2c_dev);
2228            if (ret)
2229                goto clk_free;
2230        }
2231
2232        adap = &i2c_dev->adap;
2233        i2c_set_adapdata(adap, i2c_dev);
2234        snprintf(adap->name, sizeof(adap->name), "STM32F7 I2C(%pa)",
2235             &res->start);
2236        adap->owner = THIS_MODULE;
2237        adap->timeout = 2 * HZ;
2238        adap->retries = 3;
2239        adap->algo = &stm32f7_i2c_algo;
2240        adap->dev.parent = &pdev->dev;
2241        adap->dev.of_node = pdev->dev.of_node;
2242
2243        init_completion(&i2c_dev->complete);
2244
2245        /* Init DMA config if supported */
2246        i2c_dev->dma = stm32_i2c_dma_request(i2c_dev->dev, phy_addr,
2247                             STM32F7_I2C_TXDR,
2248                             STM32F7_I2C_RXDR);
2249        if (PTR_ERR(i2c_dev->dma) == -ENODEV)
2250            i2c_dev->dma = NULL;
2251        else if (IS_ERR(i2c_dev->dma)) {
2252            ret = PTR_ERR(i2c_dev->dma);
2253            goto fmp_clear;
2254        }
......
2276        stm32f7_i2c_hw_config(i2c_dev);
2277
2278        ret = i2c_add_adapter(adap);
2279        if (ret)
2280            goto pm_disable;
......
2307        return 0;
......
2340    }
```

第 2117 行，ST 使用 stm32f7_i2c_dev 结构体来表示 STM32MP1 系列 SOC 的 I2C 控制器，这里使用 devm_kzalloc 函数来申请内存。

第 2121~2122 行，调用 platform_get_resource 函数从设备树中获取 I2C1 控制器寄存器物理

基地址，也就是 0x40012000。获取到寄存器基地址以后使用 devm_ioremap_resource 函数对其进行内存映射，得到可以在 Linux 中使用的虚拟地址。

第 2127 行和第 2135 行，调用 platform_get_irq 函数获取中断号。

第 2159~2160 行，设置 I2C 频率默认为 I2C_STD_RATE=100KHz，如果设备树节点设置了“clock-frequency”属性的话 I2C 频率就使用 clock-frequency 属性值。

第 2184~2196 行，注册 I2C 控制器的两个中断。

第 2232~2241 行，stm32f7_i2c_dev 结构体有个 adap 的成员变量，adap 就是 i2c_adapter，这里初始化 i2c_adapter。第 2239 行设置 i2c_adapter 的 algo 成员变量为 stm32f7_i2c_algo，也就是设置 i2c_algorithm。

第 2246 行，申请 DMA，看来 STM32MP1 的 I2C 适配器驱动是可以采用 DMA 方式。

第 2276 行，调用 stm32f7_i2c_hw_config 函数初始化 I2C1 控制器的相关硬件寄存器。

第 2278 行，调用 i2c_add_adapter 函数向 Linux 内核注册 i2c_adapter。

stm32f7_i2c_probe 函数主要的工作就是一下两点：

①、初始化 i2c_adapter，设置 i2c_algorithm 为 stm32f7_i2c_algo，最后向 Linux 内核注册 i2c_adapter。

②、初始化 I2C1 控制器的相关寄存器。stm32f7_i2c_algo 包含 I2C1 适配器与 I2C 设备的通信函数 master_xfer，stm32f7_i2c_algo 结构体定义如下：

示例代码 41.3.4 stm32f7_i2c_algo 结构体

```
2098    static const struct i2c_algorithm stm32f7_i2c_algo = {
2099        .master_xfer = stm32f7_i2c_xfer,
2100        .smbus_xfer = stm32f7_i2c_smbus_xfer,
2101        .functionality = stm32f7_i2c_func,
2102        .reg_slave = stm32f7_i2c_reg_slave,
2103        .unreg_slave = stm32f7_i2c_unreg_slave,
2104    };
```

我们先来看一下. functionality，functionality 用于返回此 I2C 适配器支持什么样的通信协议，在这里 functionality 就是 stm32f7_i2c_func 函数，stm32f7_i2c_func 函数内容如下：

示例代码 41.3.5stm32f7_i2c_func 函数

```
2088    static u32 stm32f7_i2c_func(struct i2c_adapter *adap)
2089    {
2090        return I2C_FUNC_I2C | I2C_FUNC_10BIT_ADDR | I2C_FUNC_SLAVE |
2091            I2C_FUNC_SMBUS_QUICK | I2C_FUNC_SMBUS_BYTE |
......
2094            I2C_FUNC_SMBUS_PROC_CALL | I2C_FUNC_SMBUS_PEC |
2095            I2C_FUNC_SMBUS_I2C_BLOCK | I2C_FUNC_SMBUS_HOST_NOTIFY;
2096    }
```

重点来看一下 stm32f7_i2c_xfer 函数，因为最终就是通过此函数来完成与 I2C 设备通信的，此函数内容如下：

示例代码 41.3.6stm32f7_i2c_xfer 函数

```
1657    static int stm32f7_i2c_xfer(struct i2c_adapter *i2c_adap,
1658                    struct i2c_msg msgs[], int num)
1659    {
1660        struct stm32f7_i2c_dev *i2c_dev = i2c_get_adapdata(i2c_adap);
```

```
1661        struct stm32f7_i2c_msg *f7_msg = &i2c_dev->f7_msg;
1662        struct stm32_i2c_dma *dma = i2c_dev->dma;
1663        unsigned long time_left;
1664        int ret;
1665
1666        i2c_dev->msg = msgs;
1667        i2c_dev->msg_num = num;
1668        i2c_dev->msg_id = 0;
1669        f7_msg->smbus = false;
1670
1671        ret = pm_runtime_get_sync(i2c_dev->dev);
1672        if (ret < 0)
1673            return ret;
1674
1675        ret = stm32f7_i2c_wait_free_bus(i2c_dev);
1676        if (ret)
1677            goto pm_free;
1678
1679        stm32f7_i2c_xfer_msg(i2c_dev, msgs);
1680
1681        time_left = wait_for_completion_timeout(&i2c_dev->complete,
1682                                                i2c_dev->adap.timeout);
1683        ret = f7_msg->result;
1684
1685        if (!time_left) {
1686            dev_dbg(i2c_dev->dev, "Access to slave 0x%x timed out\n",
1687                i2c_dev->msg->addr);
1688            if (i2c_dev->use_dma)
1689                dmaengine_terminate_all(dma->chan_using);
1690            ret = -ETIMEDOUT;
1691        }
1692
1693    pm_free:
1694        pm_runtime_mark_last_busy(i2c_dev->dev);
1695        pm_runtime_put_autosuspend(i2c_dev->dev);
1696
1697        return (ret < 0) ? ret : num;
1698    }
```

第 1675 行，调用 stm32f7_i2c_wait_free_bus 函数等待 I2C 总线空闲，也就是读取 I2C 控制的 ISR 寄存器的 bit15(BUSY)位，此位用来标记 I2C 控制器是否忙。

第 1679 行，调用 stm32f7_i2c_xfer_msg 函数发送数据，此函数也是操作 I2C 控制器硬件寄存器的，具体内容这里就不分析了，大家感兴趣可以自己阅读代码。

41.4 I2C 设备驱动编写流程

I2C 适配器驱动 SOC 厂商已经替我们编写好了，我们需要做的就是编写具体的设备驱动，本小节我们就来学习一下 I2C 设备驱动的详细编写流程。

41.4.1 I2C 设备信息描述

1、未使用设备树的时候

首先肯定要描述 I2C 设备节点信息，先来看一下没有使用设备树的时候是如何在 BSP 里面描述 I2C 设备信息的，在未使用设备树的时候需要在 BSP 里面使用 i2c_board_info 结构体来描述一个具体的 I2C 设备。i2c_board_info 结构体如下：

示例代码 41.4.1.1 i2c_board_info 结构体

```
406 struct i2c_board_info {
407     char        type[I2C_NAME_SIZE];
408     unsigned short  flags;
409     unsigned short  addr;
410     const char  *dev_name;
411     void        *platform_data;
412     struct device_node *of_node;
413     struct fwnode_handle *fwnode;
414     const struct property_entry *properties;
415     const struct resource *resources;
416     unsigned int    num_resources;
417     int     irq;
418 };
```

type 和 addr 这两个成员变量是必须要设置的，一个是 I2C 设备的名字，一个是 I2C 设备的器件地址。举个例子，打开 arch/arm/mach-imx/mach-armadillo5x0.c 文件，此文件中有关于 s35390a 这个 I2C 器件对应的设备描述信息：

示例代码 41.4.1.2 s35390a 的 I2C 设备信息

```
246 static struct i2c_board_info armadillo5x0_i2c_rtc = {
247     I2C_BOARD_INFO("s35390a", 0x30),
248 };
```

示例代码 41.4.1.2 中使用 I2C_BOARD_INFO 来完成 armadillo5x0_i2c_rtc 的初始化工作，I2C_BOARD_INFO 是一个宏，定义如下：

示例代码 41.4.1.3 I2C_BOARD_INFO 宏

```
433 #define I2C_BOARD_INFO(dev_type, dev_addr) \
434     .type = dev_type, .addr = (dev_addr)
```

可以看出，I2C_BOARD_INFO 宏其实就是设置 i2c_board_info 的 type 和 addr 这两个成员变量，因此示例代码 41.4.1.2 的主要工作就是设置 I2C 设备名字为 s35390a，器件地址为 0X30。

大家可以在 Linux 源码里面全局搜索 i2c_board_info，会找到大量以 i2c_board_info 定义的 I2C 设备信息，这些就是未使用设备树的时候 I2C 设备的描述方式，当采用了设备树以后就不会再使用 i2c_board_info 来描述 I2C 设备了。

2、使用设备树的时候

使用设备树的时候 I2C 设备信息通过创建相应的节点就行了，比如在我们的 STM32MP1 的开发板上有一个 I2C 器件 AP3216C，这是三合一的环境传感器，并且该器件挂在 STM32MP1 I2C5 总线接口上，因此必须在 i2c5 节点下创建一个子节点来描述 AP3216C 设备，节点示例如下所示：

示例代码 41.4.1.4 i2c 从设备节点示例

```
1  &i2c5 {
2      pinctrl-names = "default", "sleep";
3      pinctrl-0 = <&i2c5_pins_a>;
4      pinctrl-1 = <&i2c5_pins_sleep_a>;
5      status = "okay";
6
7      ap3216c@1e {
8          compatible = "alientek,ap3216c";
9          reg = <0x1e>;
10     };
11 };
```

第 2~4 行，设置了 i2c5 的 pinmux 的配置。

第 7~10 行，向 i2c5 添加 ap3216c 子节点，第 7 行“ap3216c@1e”是子节点名字，“@”后面的“1e”就是 ap3216c 的 I2C 器件地址。第 8 行设置 compatible 属性值为“alientek,ap3216c”。第 9 行的 reg 属性也是设置 ap3216c 的器件地址的，因此值为 0x1e。I2C 设备节点的创建重点是 compatible 属性和 reg 属性的设置，一个用于匹配驱动，一个用于设置器件地址。

41.4.2 I2C 设备数据收发处理流程

在 41.2.2 小节已经说过了，I2C 设备驱动首先要做的就是初始化 i2c_driver 并向 Linux 内核注册。当设备和驱动匹配以后 i2c_driver 里面的 probe 函数就会执行，probe 函数里面所做的就是字符设备驱动那一套了。一般需要在 probe 函数里面初始化 I2C 设备，要初始化 I2C 设备就必须能够对 I2C 设备寄存器进行读写操作，这里就要用到 i2c_transfer 函数了。i2c_transfer 函数最终会调用 I2C 适配器中 i2c_algorithm 里面的 master_xfer 函数，对于 STM32MP1 而言就是 stm32f7_i2c_xfer 这个函数。i2c_transfer 函数原型如下：

```
int i2c_transfer(struct i2c_adapter     *adap,
                 struct i2c_msg         *msgs,
                 int                    num)
```

函数参数和返回值含义如下：

adap：所使用的 I2C 适配器，i2c_client 会保存其对应的 i2c_adapter。

msgs：I2C 要发送的一个或多个消息。

num：消息数量，也就是 msgs 的数量。

返回值：负值，失败，其他非负值，发送的 msgs 数量。

我们重点来看一下 msgs 这个参数，这是一个 i2c_msg 类型的指针参数，I2C 进行数据收发说白了就是消息的传递，Linux 内核使用 i2c_msg 结构体来描述一个消息。i2c_msg 结构体定义在 include/uapi/linux/i2c.h 文件中，结构体内容如下：

示例代码 41.4.2.1 i2c_msg 结构体

```
69 struct i2c_msg {
70     __u16 addr;                  /* 从机地址          */
```

```
71      __u16 flags;                /* 标志             */
72      #define I2C_M_TEN           0x0010
73      #define I2C_M_RD            0x0001
74      #define I2C_M_STOP          0x8000
75      #define I2C_M_NOSTART       0x4000
76      #define I2C_M_REV_DIR_ADDR  0x2000
77      #define I2C_M_IGNORE_NAK    0x1000
78      #define I2C_M_NO_RD_ACK     0x0800
79      #define I2C_M_RECV_LEN      0x0400
80      __u16 len;                  /* 消息(本 msg)长度 */
81      __u8 *buf;                  /* 消息数据         */
82  };
```

使用 i2c_transfer 函数发送数据之前要先构建好 i2c_msg，使用 i2c_transfer 进行 I2C 数据收发的示例代码如下：

示例代码 41.4.2.2 I2C 设备多寄存器数据读写

```
1  /* 设备结构体 */
2  struct xxx_dev {
3      ......
4      void *private_data; /* 私有数据，一般会设置为 i2c_client */
5  };
6
7  /*
8   * @description    : 读取 I2C 设备多个寄存器数据
9   * @param - dev    : I2C 设备
10  * @param - reg    : 要读取的寄存器首地址
11  * @param - val    : 读取到的数据
12  * @param - len    : 要读取的数据长度
13  * @return         : 操作结果
14  */
15 static int xxx_read_regs(struct xxx_dev *dev, u8 reg, void *val,
                           int len)
16 {
17     int ret;
18     struct i2c_msg msg[2];
19     struct i2c_client *client = (struct i2c_client *)
                                   dev->private_data;
20
21     /* msg[0]，第一条写消息，发送要读取的寄存器首地址 */
22     msg[0].addr = client->addr;         /* I2C 器件地址     */
23     msg[0].flags = 0;                   /* 标记为发送数据   */
24     msg[0].buf = &reg;                  /* 读取的首地址     */
25     msg[0].len = 1;                     /* reg 长度         */
26
```

```
27      /* msg[1]，第二条读消息，读取寄存器数据 */
28      msg[1].addr = client->addr;         /* I2C 器件地址      */
29      msg[1].flags = I2C_M_RD;            /* 标记为读取数据    */
30      msg[1].buf = val;                   /* 读取数据缓冲区    */
31      msg[1].len = len;                   /* 要读取的数据长度  */
32
33      ret = i2c_transfer(client->adapter, msg, 2);
34      if(ret == 2) {
35          ret = 0;
36      } else {
37          ret = -EREMOTEIO;
38      }
39      return ret;
40  }
41
42  /*
43   * @description     : 向 I2C 设备多个寄存器写入数据
44   * @param - dev     : 要写入的设备结构体
45   * @param - reg     : 要写入的寄存器首地址
46   * @param - val     : 要写入的数据缓冲区
47   * @param - len     : 要写入的数据长度
48   * @return          : 操作结果
49   */
50  static s32 xxx_write_regs(struct xxx_dev *dev, u8 reg, u8 *buf,
                              u8 len)
51  {
52      u8 b[256];
53      struct i2c_msg msg;
54      struct i2c_client *client = (struct i2c_client *)
                                    dev->private_data;
55
56      b[0] = reg;                     /* 寄存器首地址                    */
57      memcpy(&b[1],buf,len);          /* 将要发送的数据拷贝到数组 b 里面  */
58
59      msg.addr = client->addr;        /* I2C 器件地址                    */
60      msg.flags = 0;                  /* 标记为写数据                    */
61
62      msg.buf = b;                    /* 要发送的数据缓冲区              */
63      msg.len = len + 1;              /* 要发送的数据长度                */
64
65      return i2c_transfer(client->adapter, &msg, 1);
66  }
```

第 2~5 行，设备结构体，在设备结构体里面添加一个执行 void 的指针成员变量 private_data，此成员变量用于保存设备的私有数据。在 I2C 设备驱动中我们一般将其指向 I2C 设备对应的 i2c_client。

第 15~40 行，xxx_read_regs 函数用于读取 I2C 设备多个寄存器数据。第 18 行定义了一个 i2c_msg 数组，2 个数组元素，因为 I2C 读取数据的时候要先发送要读取的寄存器地址，然后再读取数据，所以需要准备两个 i2c_msg。一个用于发送寄存器地址，一个用于读取寄存器值。对于 msg[0]，将 flags 设置为 0，表示写数据。msg[0]的 addr 是 I2C 设备的器件地址，msg[0]的 buf 成员变量就是要读取的寄存器地址。对于 msg[1]，将 flags 设置为 I2C_M_RD，表示读取数据。msg[1]的 buf 成员变量用于保存读取到的数据，len 成员变量就是要读取的数据长度。调用 i2c_transfer 函数完成 I2C 数据读操作。

第 50~66 行，xxx_write_regs 函数用于向 I2C 设备多个寄存器写数据，I2C 写操作要比读操作简单一点，因此一个 i2c_msg 即可。数组 b 用于存放寄存器首地址和要发送的数据，第 59 行设置 msg 的 addr 为 I2C 器件地址。第 60 行设置 msg 的 flags 为 0，也就是写数据。第 62 行设置要发送的数据，也就是数组 b。第 63 行设置 msg 的 len 为 len+1，因为要加上一个字节的寄存器地址。最后通过 i2c_transfer 函数完成向 I2C 设备的写操作。

另外还有两个 API 函数分别用于 I2C 数据的收发操作，这两个函数最终都会调用 i2c_transfer。首先来看一下 I2C 数据发送函数 i2c_master_send，函数原型如下：

```
int i2c_master_send(const struct i2c_client  *client,
                    const char               *buf,
                    int                      count)
```

函数参数和返回值含义如下：

client：I2C 设备对应的 i2c_client。

buf：要发送的数据。

count：要发送的数据字节数，要小于 64KB，以为 i2c_msg 的 len 成员变量是一个 u16(无符号 16 位)类型的数据。

返回值：负值，失败，其他非负值，发送的字节数。

I2C 数据接收函数为 i2c_master_recv，函数原型如下：

```
int i2c_master_recv(const struct i2c_client  *client,
                    char                     *buf,
                    int                      count)
```

函数参数和返回值含义如下：

client：I2C 设备对应的 i2c_client。

buf：要接收的数据。

count：要接收的数据字节数，要小于 64KB，以为 i2c_msg 的 len 成员变量是一个 u16(无符号 16 位)类型的数据。

返回值：负值，失败，其他非负值，发送的字节数。

关于 Linux 下 I2C 设备驱动的编写流程就讲解到这里，重点就是 i2c_msg 的构建和 i2c_transfer 函数的调用，接下来我们就编写 AP3216C 这个 I2C 设备的 Linux 驱动。

41.5 硬件原理图分析

AP3612C 原理图如图 41.5.1 所示：

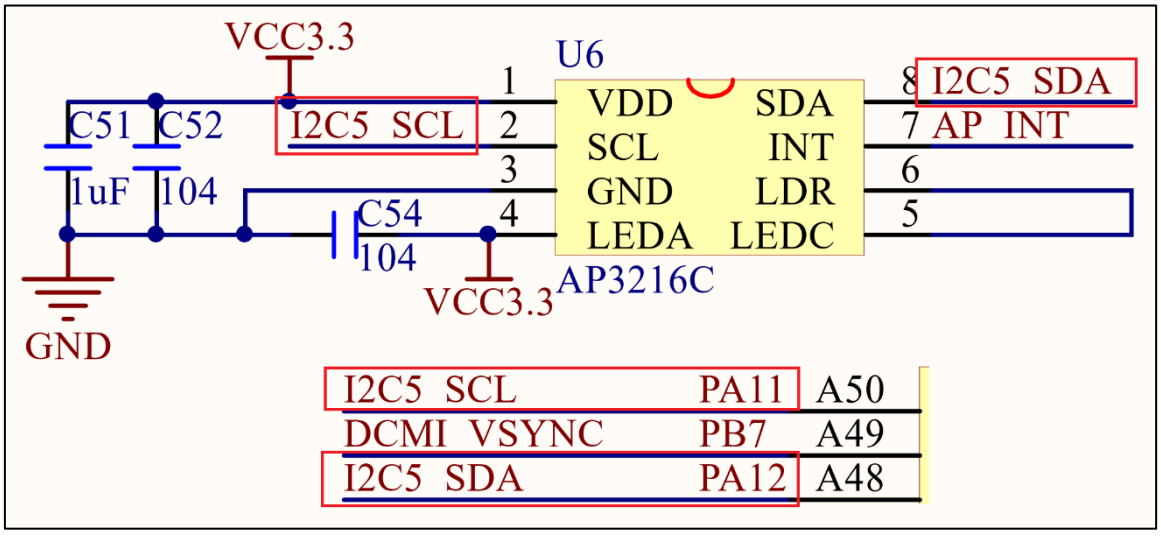


图 41.5.1 AP3216C 原理图

从图 41.5.1 可以看出 AP3216C 使用的是 I2C5，其中 I2C5_SCL 使用的是 PA11 这个 IO，I2C_SDA 使用的是 PA12 这个 IO。AP3216C 还有个中断引脚，这里我们没有用到中断功能。

41.6 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→21_iic。

41.6.1 修改设备树

1、IO 修改或添加

AP3216C 用到了 I2C5 接口。因为 I2C5 所使用的 IO 分别为 PA11 和 PA12，所以我们要根据数据手册设置 I2C5 的 pinmux 的配置。如果要用到 AP3216C 的中断功能的话还需要初始化 AP_INT 对应的 PE4 这个 IO，本章实验我们不使用中断功能。因此只需要设置 PA11 和 PA12 这个两个 IO 复用为 AF4 功能，ST 其实已经将这个两个 IO 设置好了，打开 stm32mp15-pinctrl.dtsi，然后找到如下内容：

示例代码 41.6.1.1 I2C5 的 pinmux 配置

```
1  i2c5_pins_a: i2c5-0 {
2      pins {
3          pinmux = <STM32_PINMUX('A', 11, AF4)>, /* I2C5_SCL   */
4              <STM32_PINMUX('A', 12, AF4)>;      /* I2C5_SDA   */
5          bias-disable;
6          drive-open-drain;
7          slew-rate = <0>;
8      };
9  };
10
11 i2c5_pins_sleep_a: i2c5-1 {
12     pins {
13         pinmux = <STM32_PINMUX('A', 11, ANALOG)>, /* I2C5_SCL */
14             <STM32_PINMUX('A', 12, ANALOG)>; /* I2C5_SDA */
```

```
15
16      };
17  };
```

示例代码 41.6.1.1 中，定义了 I2C5 接口的两个 pinmux 配置分别为：i2c5_pins_a 和 i2c5_pins_sleep_a。第一个默认的状态下使用，第二个是在 sleep 状态下使用。

2、在 i2c5 节点追加 ap3216c 子节点

接着我们打开 stm32mp157d-atk.dts 文件，通过节点内容追加的方式，向 i2c5 节点中添加“ap3216c@1e”子节点，节点如下所示：

示例代码 41.6.1.2 向 i2c5 追加 ap3216c 子节点

```
1   &i2c5 {
2       pinctrl-names = "default", "sleep";
3       pinctrl-0 = <&i2c5_pins_a>;
4       pinctrl-1 = <&i2c5_pins_sleep_a>;
5       status = "okay";
6
7       ap3216c@1e {
8           compatible = "alientek,ap3216c";
9           reg = <0x1e>;
10      };
11  };
```

第 2~4 行，给 I2C5 节点设置了 pinmux 配置。

第 7 行，ap3216c 子节点，@后面的“1e”是 ap3216c 的器件地址。

第 8 行，设置 compatible 值为“alientek,ap3216c”。

第 9 行，reg 属性也是设置 ap3216c 器件地址的，因此 reg 设置为 0x1e。

设备树修改完成以后使用“make dtbs”重新编译一下，然后使用新的设备树启动 Linux 内核。/sys/bus/i2c/devices 目录下存放着所有 I2C 设备，如果设备树修改正确的话，会在/sys/bus/i2c/devices 目录下看到一个名为“0-001e”的子目录，如图 41.6.1.1 所示：

```
[root@ATK-stm32mp1]:~$ cd /sys/bus/i2c/devices/
[root@ATK-stm32mp1]:/sys/bus/i2c/devices$ ls
0-001e  i2c-0
[root@ATK-stm32mp1]:/sys/bus/i2c/devices$
```

图 41.6.1.1 当前系统 I2C 设备

图 41.6.1.1 中的“0-001e”就是 ap3216c 的设备目录，“1e”就是 ap3216c 器件地址。进入 0-001e 目录，可以看到“name”文件，name 文件保存着此设备名字，在这里就是“ap3216c”，如图 41.6.1.2 所示：

```
[root@ATK-stm32mp1]:/sys/bus/i2c/devices$ cat 0-001e/name
ap3216c
[root@ATK-stm32mp1]:/sys/bus/i2c/devices$
```

图 41.6.1.2 ap3216c 器件名字

41.6.2 AP3216C 驱动编写

新建名为“21_iic”的文件夹，然后在 21_iic 文件夹里面创建 vscode 工程，工作区命名为“iic”。工程创建好以后新建 ap3216c.c 和 ap3216creg.h 这两个文件，ap3216c.c 为 AP3216C 的驱动代码，ap3216creg.h 是 AP3216C 寄存器头文件。先在 ap3216creg.h 中定义好 AP3216C 的寄存器，输入如下内容，

示例代码 41.6.2.1 ap3216creg.h 文件代码段

```
1  #ifndef AP3216C_H
2  #define AP3216C_H
3  /***************************************************************
4  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
5  文件名    : ap3216creg.h
6  作者      : 正点原子 Linux 团队
7  版本      : V1.0
8  描述      : AP3216C 寄存器地址描述头文件
9  其他      : 无
10 论坛      : www.openedv.com
11 日志      : 初版 V1.0 2021/03/19 正点原子 Linux 团队创建
12 ***************************************************************/
13
14 #define AP3216C_ADDR        0X1E       /* AP3216C 器件地址     */
15
16 /* AP3316C 寄存器 */
17 #define AP3216C_SYSTEMCONG  0x00    /* 配置寄存器           */
18 #define AP3216C_INTSTATUS   0X01    /* 中断状态寄存器       */
19 #define AP3216C_INTCLEAR    0X02    /* 中断清除寄存器       */
20 #define AP3216C_IRDATALOW   0x0A    /* IR 数据低字节        */
21 #define AP3216C_IRDATAHIGH  0x0B    /* IR 数据高字节        */
22 #define AP3216C_ALSDATALOW  0x0C    /* ALS 数据低字节       */
23 #define AP3216C_ALSDATAHIGH 0X0D    /* ALS 数据高字节       */
24 #define AP3216C_PSDATALOW   0X0E    /* PS 数据低字节        */
25 #define AP3216C_PSDATAHIGH  0X0F    /* PS 数据高字节        */
26
27 #endif
```

ap3216creg.h 没什么好讲的，就是一些寄存器宏定义。然后在 ap3216c.c 输入如下内容：

示例代码 41.6.2.2 ap3216c.c 文件代码段

```
1  /***************************************************************
2  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
3  文件名    : ap3216c.c
4  作者      : 正点原子 Linux 团队
5  版本      : V1.0
6  描述      : AP3216C 驱动程序
7  其他      : 无
```

```
8  论坛      : www.openedv.com
9  日志      : 初版 V1.0 2021/03/19 正点原子 Linux 团队创建
10 ***************************************************************/
11 #include <linux/types.h>
12 #include <linux/kernel.h>
13 #include <linux/delay.h>
14 #include <linux/ide.h>
15 #include <linux/init.h>
16 #include <linux/module.h>
17 #include <linux/errno.h>
18 #include <linux/gpio.h>
19 #include <linux/cdev.h>
20 #include <linux/device.h>
21 #include <linux/of_gpio.h>
22 #include <linux/semaphore.h>
23 #include <linux/timer.h>
24 #include <linux/i2c.h>
25 #include <asm/mach/map.h>
26 #include <asm/uaccess.h>
27 #include <asm/io.h>
28 #include "ap3216creg.h"
29
30 #define AP3216C_CNT 1
31 #define AP3216C_NAME    "ap3216c"
32
33 struct ap3216c_dev {
34     struct i2c_client *client;  /* i2c 设备        */
35     dev_t devid;                /* 设备号          */
36     struct cdev cdev;           /* cdev            */
37     struct class *class;        /* 类              */
38     struct device *device;      /* 设备            */
39     struct device_node  *nd;    /* 设备节点        */
40     unsigned short ir, als, ps; /* 三个光传感器数据  */
41 };
42
43 /*
44  * @description   :   从 ap3216c 读取多个寄存器数据
45  * @param - dev   :   ap3216c 设备
46  * @param - reg   :   要读取的寄存器首地址
47  * @param - val   :   读取到的数据
48  * @param - len   :   要读取的数据长度
49  * @return        :   操作结果
50  */
```

```
51  static int ap3216c_read_regs(struct ap3216c_dev *dev, u8 reg,
                                 void *val, int len)
52  {
53      int ret;
54      struct i2c_msg msg[2];
55      struct i2c_client *client = (struct i2c_client *)dev->client;
56
57      /* msg[0]为发送要读取的首地址 */
58      msg[0].addr = client->addr;         /* ap3216c 地址     */
59      msg[0].flags = 0;                   /* 标记为发送数据    */
60      msg[0].buf = &reg;                  /* 读取的首地址      */
61      msg[0].len = 1;                     /* reg 长度         */
62
63      /* msg[1]读取数据 */
64      msg[1].addr = client->addr;         /* ap3216c 地址     */
65      msg[1].flags = I2C_M_RD;            /* 标记为读取数据    */
66      msg[1].buf = val;                   /* 读取数据缓冲区    */
67      msg[1].len = len;                   /* 要读取的数据长度  */
68
69      ret = i2c_transfer(client->adapter, msg, 2);
70      if(ret == 2) {
71          ret = 0;
72      } else {
73          printk("i2c rd failed=%d reg=%06x len=%d\n",ret, reg, len);
74          ret = -EREMOTEIO;
75      }
76      return ret;
77  }
78
79  /*
80   * @description: 向 ap3216c 多个寄存器写入数据
81   * @param - dev: ap3216c 设备
82   * @param - reg: 要写入的寄存器首地址
83   * @param - val: 要写入的数据缓冲区
84   * @param - len: 要写入的数据长度
85   * @return     : 操作结果
86   */
87  static s32 ap3216c_write_regs(struct ap3216c_dev *dev, u8 reg,
                                  u8 *buf, u8 len)
88  {
89      u8 b[256];
90      struct i2c_msg msg;
91      struct i2c_client *client = (struct i2c_client *)dev->client;
```

```
92
93      b[0] = reg;                     /* 寄存器首地址                    */
94      memcpy(&b[1],buf,len);          /* 将要写入的数据拷贝到数组 b 里面  */
95
96      msg.addr = client->addr;        /* ap3216c 地址                    */
97      msg.flags = 0;                  /* 标记为写数据                    */
98
99      msg.buf = b;                    /* 要写入的数据缓冲区              */
100     msg.len = len + 1;              /* 要写入的数据长度                */
101
102     return i2c_transfer(client->adapter, &msg, 1);
103 }
104
105 /*
106  * @description: 读取 ap3216c 指定寄存器值，读取一个寄存器
107  * @param - dev: ap3216c 设备
108  * @param - reg: 要读取的寄存器
109  * @return     : 读取到的寄存器值
110  */
111 static unsigned char ap3216c_read_reg(struct ap3216c_dev *dev,
                                          u8 reg)
112 {
113     u8 data = 0;
114
115     ap3216c_read_regs(dev, reg, &data, 1);
116     return data;
117 }
118
119 /*
120  * @description: 向 ap3216c 指定寄存器写入指定的值，写一个寄存器
121  * @param - dev: ap3216c 设备
122  * @param - reg: 要写的寄存器
123  * @param - data:要写入的值
124  * @return     : 无
125  */
126 static void ap3216c_write_reg(struct ap3216c_dev *dev, u8 reg,
                                  u8 data)
127 {
128     u8 buf = 0;
129     buf = data;
130     ap3216c_write_regs(dev, reg, &buf, 1);
131 }
132
```

```
133 /*
134  * @description   : 读取 AP3216C 的数据，包括 ALS,PS 和 IR, 注意！如果同时
135  *                 :打开 ALS,IR+PS 两次数据读取的时间间隔要大于 112.5ms
136  * @param - ir    : ir 数据
137  * @param - ps    : ps 数据
138  * @param - ps    : als 数据
139  * @return        : 无。
140  */
141 void ap3216c_readdata(struct ap3216c_dev *dev)
142 {
143     unsigned char i =0;
144     unsigned char buf[6];
145
146     /* 循环读取所有传感器数据 */
147     for(i = 0; i < 6; i++) {
148         buf[i] = ap3216c_read_reg(dev, AP3216C_IRDATALOW + i);
149     }
150
151     if(buf[0] & 0X80)     /* IR_OF 位为 1,则数据无效   */
152         dev->ir = 0;
153     else                  /* 读取 IR 传感器的数据       */
154         dev->ir = ((unsigned short)buf[1] << 2) | (buf[0] & 0X03);
155
156     dev->als = ((unsigned short)buf[3] << 8) | buf[2];
157
158     if(buf[4] & 0x40)     /* IR_OF 位为 1,则数据无效   */
159         dev->ps = 0;
160     else                  /* 读取 PS 传感器的数据       */
161         dev->ps = ((unsigned short)(buf[5] & 0X3F) << 4) | (buf[4] &
0X0F);
162 }
163
164 /*
165  * @description   : 打开设备
166  * @param - inode : 传递给驱动的 inode
167  * @param - filp  : 设备文件，file 结构体有个叫做 private_data 的成员变量
168  *                  一般在 open 的时候将 private_data 指向设备结构体。
169  * @return        : 0 成功;其他 失败
170  */
171 static int ap3216c_open(struct inode *inode, struct file *filp)
172 {
173     /* 从 file 结构体获取 cdev 指针，再根据 cdev 获取 ap3216c_dev 首地址 */
174     struct cdev *cdev = filp->f_path.dentry->d_inode->i_cdev;
```

```
175     struct ap3216c_dev *ap3216cdev = container_of(cdev,
                                struct ap3216c_dev, cdev);
176
177     /* 初始化 AP3216C */
178     ap3216c_write_reg(ap3216cdev, AP3216C_SYSTEMCONG, 0x04);
179     mdelay(50);
180     ap3216c_write_reg(ap3216cdev, AP3216C_SYSTEMCONG, 0X03);
181     return 0;
182 }
183
184 /*
185  * @description   : 从设备读取数据
186  * @param - filp  : 要打开的设备文件(文件描述符)
187  * @param - buf   : 返回给用户空间的数据缓冲区
188  * @param - cnt   : 要读取的数据长度
189  * @param - offt  : 相对于文件首地址的偏移
190  * @return        : 读取的字节数，如果为负值，表示读取失败
191  */
192 static ssize_t ap3216c_read(struct file *filp, char __user *buf,
                                size_t cnt, loff_t *off)
193 {
194     short data[3];
195     long err = 0;
196
197     struct cdev *cdev = filp->f_path.dentry->d_inode->i_cdev;
198     struct ap3216c_dev *dev = container_of(cdev, struct ap3216c_dev,
                                                cdev);
199
200     ap3216c_readdata(dev);
201
202     data[0] = dev->ir;
203     data[1] = dev->als;
204     data[2] = dev->ps;
205     err = copy_to_user(buf, data, sizeof(data));
206     return 0;
207 }
208
209 /*
210  * @description   : 关闭/释放设备
211  * @param - filp  : 要关闭的设备文件(文件描述符)
212  * @return        : 0 成功;其他 失败
213  */
214 static int ap3216c_release(struct inode *inode, struct file *filp)
```

```
215 {
216     return 0;
217 }
218 
219 /* AP3216C 操作函数 */
220 static const struct file_operations ap3216c_ops = {
221     .owner = THIS_MODULE,
222     .open = ap3216c_open,
223     .read = ap3216c_read,
224     .release = ap3216c_release,
225 };
226 
227  /*
228   * @description  : i2c 驱动的 probe 函数，当驱动与
229   *                 设备匹配以后此函数就会执行
230   * @param - client  : i2c 设备
231   * @param - id      : i2c 设备 ID
232   * @return          : 0，成功;其他负值,失败
233   */
234 static int ap3216c_probe(struct i2c_client *client,
                             const struct i2c_device_id *id)
235 {
236     int ret;
237     struct ap3216c_dev *ap3216cdev;
238 
239 
240     ap3216cdev = devm_kzalloc(&client->dev, sizeof(*ap3216cdev),
                                  GFP_KERNEL);
241     if(!ap3216cdev)
242         return -ENOMEM;
243 
244     /* 注册字符设备驱动 */
245     /* 1、创建设备号 */
246     ret = alloc_chrdev_region(&ap3216cdev->devid, 0, AP3216C_CNT,
                                  AP3216C_NAME);
247     if(ret < 0) {
248         pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                   AP3216C_NAME, ret);
249         return -ENOMEM;
250     }
251 
252     /* 2、初始化 cdev */
253     ap3216cdev->cdev.owner = THIS_MODULE;
```

```
254     cdev_init(&ap3216cdev->cdev, &ap3216c_ops);
255
256     /* 3、添加一个 cdev */
257     ret = cdev_add(&ap3216cdev->cdev, ap3216cdev->devid,
                   AP3216C_CNT);
258     if(ret < 0) {
259         goto del_unregister;
260     }
261
262     /* 4、创建类 */
263     ap3216cdev->class = class_create(THIS_MODULE, AP3216C_NAME);
264     if (IS_ERR(ap3216cdev->class)) {
265         goto del_cdev;
266     }
267
268     /* 5、创建设备 */
269     ap3216cdev->device = device_create(ap3216cdev->class, NULL,
                          ap3216cdev->devid, NULL, AP3216C_NAME);
270     if (IS_ERR(ap3216cdev->device)) {
271         goto destroy_class;
272     }
273     ap3216cdev->client = client;
274     /* 保存 ap3216cdev 结构体 */
275     i2c_set_clientdata(client,ap3216cdev);
276
277     return 0;
278 destroy_class:
279     device_destroy(ap3216cdev->class, ap3216cdev->devid);
280 del_cdev:
281     cdev_del(&ap3216cdev->cdev);
282 del_unregister:
283     unregister_chrdev_region(ap3216cdev->devid, AP3216C_CNT);
284     return -EIO;
285 }
286
287 /*
288  * @description  : i2c 驱动的 remove 函数，移除 i2c 驱动的时候此函数会执行
289  * @param - client   : i2c 设备
290  * @return           : 0，成功;其他负值,失败
291  */
292 static int ap3216c_remove(struct i2c_client *client)
293 {
294     struct ap3216c_dev *ap3216cdev = i2c_get_clientdata(client);
```

```
295     /* 注销字符设备驱动 */
296     /* 1、删除 cdev */
297     cdev_del(&ap3216cdev->cdev);
298     /* 2、注销设备号 */
299     unregister_chrdev_region(ap3216cdev->devid, AP3216C_CNT);
300     /* 3、注销设备 */
301     device_destroy(ap3216cdev->class, ap3216cdev->devid);
302     /* 4、注销类 */
303     class_destroy(ap3216cdev->class);
304     return 0;
305 }
306
307 /* 传统匹配方式 ID 列表 */
308 static const struct i2c_device_id ap3216c_id[] = {
309     {"alientek,ap3216c", 0},
310     {}
311 };
312
313 /* 设备树匹配列表 */
314 static const struct of_device_id ap3216c_of_match[] = {
315     { .compatible = "alientek,ap3216c" },
316     { /* Sentinel */ }
317 };
318
319 /* i2c 驱动结构体 */
320 static struct i2c_driver ap3216c_driver = {
321     .probe = ap3216c_probe,
322     .remove = ap3216c_remove,
323     .driver = {
324             .owner = THIS_MODULE,
325             .name = "ap3216c",
326             .of_match_table = ap3216c_of_match,
327           },
328     .id_table = ap3216c_id,
329 };
330
331 /*
332  * @description   : 驱动入口函数
333  * @param         : 无
334  * @return        : 无
335  */
336 static int __init ap3216c_init(void)
337 {
```

```
338     int ret = 0;
339
340     ret = i2c_add_driver(&ap3216c_driver);
341     return ret;
342 }
343
344 /*
345  * @description   : 驱动出口函数
346  * @param         : 无
347  * @return        : 无
348  */
349 static void __exit ap3216c_exit(void)
350 {
351     i2c_del_driver(&ap3216c_driver);
352 }
353
354 /* module_i2c_driver(ap3216c_driver) */
355
356 module_init(ap3216c_init);
357 module_exit(ap3216c_exit);
358 MODULE_LICENSE("GPL");
359 MODULE_AUTHOR("ALIENTEK");
360 MODULE_INFO(intree, "Y");
```

在示例代码 41.6.2.2 里，没有定义一个全局变量，那是因为 linux 内核不推荐使用全局变量，要使用内存的就用 devm_kzalloc 之类的函数去申请空间。

第 33~41 行，自定义一个 ap3216c_dev 结构体。第 34 行的 client 成员变量用来存储从设备树提供的 i2c_client 结构体。第 40 行的 ir、als 和 ps 分别存储 AP3216C 的 IR、ALS 和 PS 数据。

第 51~77 行，ap3216c_read_regs 函数实现多字节读取，但是 AP3216C 好像不支持连续多字节读取，此函数在测试其他 I2C 设备的时候可以实现多给字节连续读取，但是在 AP3216C 上不能连续读取多个字节，不过读取一个字节没有问题的。

第 87~103 行，ap3216c_write_regs 函数实现连续多字节写操作。

第 111~117 行，ap3216c_read_reg 函数用于读取 AP3216C 的指定寄存器数据，用于一个寄存器的数据读取。

第 126~131 行，ap3216c_write_reg 函数用于向 AP3216C 的指定寄存器写入数据，用于一个寄存器的数据写操作。

第 141~162 行，读取 AP3216C 的 PS、ALS 和 IR 等传感器原始数据值。

第 171~225 行，标准的字符设备驱动框架。ap3216c_dev 结构体里有一个 cdev 的变量成员，第 174 行就是获取 ap3216c_dev 里的 cdev 这个变量的地址，在第 175 行使用 container_of 宏获取 ap3216c_dev 的首地址。

第 234~285 行，ap3216c_probe 函数，当 I2C 设备和驱动匹配成功以后此函数就会执行，和 platform 驱动框架一样。此函数前面都是标准的字符设备注册代码，第 275 行，调用

i2c_set_clientdata 函数将 ap3216cdev 变量的地址绑定到 client，进行绑定之后，可以通过 i2c_get_clientdata 来获取 ap3216cdev 变量指针。

第 292~305 行，ap3216c_remove 函数，当 I2C 驱动模块卸载时会执行此函数。第 294 行通过调用 i2c_get_clientdata 函数来得到 ap3216cdev 变量的地址，后面执行的一系列卸载、注销操作都是前面讲到过的标准字符设备。

第 308~311 行，ap3216c_id 匹配表，i2c_device_id 类型。用于传统的设备和驱动匹配，也就是没有使用设备树的时候。

第 314~317 行，ap3216c_of_match 匹配表，of_device_id 类型，用于设备树设备和驱动匹配。这里只写了一个 compatible 属性，值为“alientek,ap3216c”。

第 320~329 行，ap3216c_driver 结构体变量，i2c_driver 类型。

第 336~342 行，驱动入口函数 ap3216c_init，此函数通过调用 i2c_add_driver 来向 Linux 内核注册 i2c_driver，也就是 ap3216c_driver。

第 349~352 行，驱动出口函数 ap3216c_exit，此函数通过调用 i2c_del_driver 来注销掉前面注册的 ap3216c_driver。

41.6.3 编写测试 APP

新建 ap3216cApp.c 文件，然后在里面输入如下所示内容：

示例代码 41.6.3.1 测试 APP

```
12  #include "stdio.h"
13  #include "unistd.h"
14  #include "sys/types.h"
15  #include "sys/stat.h"
16  #include "sys/ioctl.h"
17  #include "fcntl.h"
18  #include "stdlib.h"
19  #include "string.h"
20  #include <poll.h>
21  #include <sys/select.h>
22  #include <sys/time.h>
23  #include <signal.h>
24  #include <fcntl.h>
25  /*
26   * @description   : main 主程序
27   * @param - argc  : argv 数组元素个数
28   * @param - argv  : 具体参数
29   * @return        : 0 成功;其他 失败
30   */
31  int main(int argc, char *argv[])
32  {
33      int fd;
34      char *filename;
35      unsigned short databuf[3];
36      unsigned short ir, als, ps;
```

```
37      int ret = 0;
38
39      if (argc != 2) {
40          printf("Error Usage!\r\n");
41          return -1;
42      }
43
44      filename = argv[1];
45      fd = open(filename, O_RDWR);
46      if(fd < 0) {
47          printf("can't open file %s\r\n", filename);
48          return -1;
49      }
50
51      while (1) {
52          ret = read(fd, databuf, sizeof(databuf));
53          if(ret == 0) {              /* 数据读取成功 */
54              ir =  databuf[0];       /* ir 传感器数据 */
55              als = databuf[1];       /* als 传感器数据 */
56              ps =  databuf[2];       /* ps 传感器数据 */
57              printf("ir = %d, als = %d, ps = %d\r\n", ir, als, ps);
58          }
59          usleep(200000);   /*100ms */
60      }
61      close(fd);              /* 关闭文件 */
62      return 0;
63  }
```

ap3216cApp.c 文件内容很简单，就是在 while 循环中不断的读取 AP3216C 的设备文件，从而得到 ir、als 和 ps 这三个数据值，然后将其输出到终端上。

41.7 运行测试

41.7.1 编译驱动程序和测试 APP

1、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为“ap3216c.o”，Makefile 内容如下所示：

示例代码 41.7.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := ap3216c.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“ap3216c.o”。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“ap3216c.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译 ap3216cApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc ap3216cApp.c -o ap3216cApp
```

编译成功以后就会生成 ap3216cApp 这个应用程序。

41.7.2 运行测试

将上一小节编译出来 ap3216c.ko 和 ap3216cApp 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中。输入如下命令加载 ap3216c.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe ap3216c        //加载驱动模块
```

当驱动模块加载成功以后使用 ap3216cApp 来测试，输入如下命令：

```
./ap3216cApp /dev/ap3216c
```

测试 APP 会不断的从 AP3216C 中读取数据，然后输出到终端上，如图 41.7.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./ap3216cApp  /dev/ap3216c
ir = 0, als = 0, ps = 0
ir = 0, als = 636, ps = 6
ir = 0, als = 635, ps = 0
ir = 0, als = 633, ps = 3
ir = 1, als = 633, ps = 5
ir = 4, als = 638, ps = 0
ir = 0, als = 640, ps = 0
```

图 41.7.2.1 获取到的 AP3216C 数据

大家可以用手电筒照一下 AP3216C，或者手指靠近 AP3216C 来观察传感器数据有没有变化。

第四十二章 RGB 转 HDMI 实验

目前大多数的显示器都提供了 HDMI 接口，HDMI 的应用范围也越来越广，但是 STM32MP157 这颗芯片原生并不支持 HDMI 显示。我们可以通过 RGB 转 HDMI 芯片将 RGB 信号转为 HDMI 信号，这样就可以连接 HDMI 显示器了。本章我们就来学习一下如何在正点原子的 STM32MP1 开发板上实现 RGB 转 HDMI。

42.1 RGB 转 HMDI 简介

STM32MP157 这颗 SOC 没有 HDMI 外设，只有 RGB 屏幕接口，因此只能通过 RGB 转 HDMI 的芯片来实现 HDMI 连接。效果肯定是没法和原生支持 HDMI 接口的 SOC 比，当个玩具来玩一下还是可以的。因此本质上来讲还是 RGB 驱动，并非原生的 HDM 驱动。

正点原子的 STM32MP1 开发板提供了 RGB 转 HDM 模块，如图 42.1.1 所示：

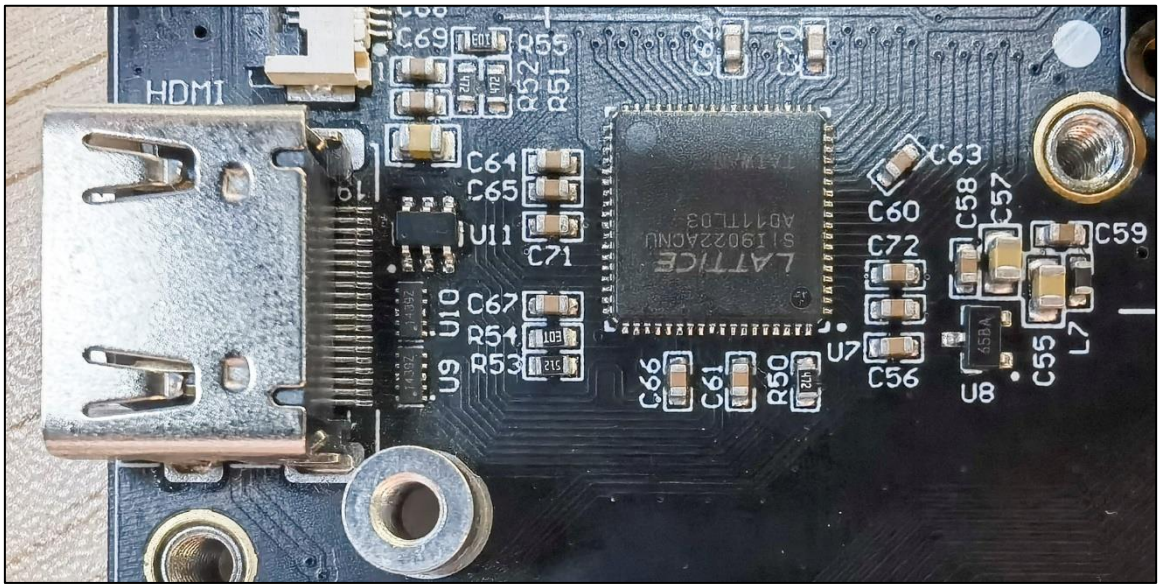


图 42.1.1 Sii9022A 芯片

这里我们使用 Sii9022A 这颗芯片来完成 RGB 转 HDMI，Sii9022A 以前是 Silicon Image 公司出品的，但是 Silicon Image 后来被 Lattice 收购了。

SiI9022A 是一款 HDMI 传输芯片，适用于高清便携相机、数字相机和个人移动设备，可以灵活的将其他音视频接口转换为 HDMI 或者 DVI 格式。SiI9022A 支持预编程 HDCP 键码，可以完全自动进行 HDCP 检测和鉴定。SiI9022A 是一个视频转换芯片，支持输入视频格式有：xvYCC、BTA-T1004、ITU-R.656，内置 DE 发生器支持 SYNC 格式(RGB 格式)。输出格式支持：HDMI、HDCP 和 DVI、最高支持 1080P 视频输出、支持 HDMI A、HDMI C 和 Micro-D 连接器。SiI9022A 功能非常多，具体使用什么功能需要进行配置，因此 SiI9022A 提供了一个 I2C 接口用于配置。

42.2 硬件原理图分析

我们先进行 SiI9022A 的硬件原理分析，打开开发板底版原理图，原理如图 42.2.1 所示：

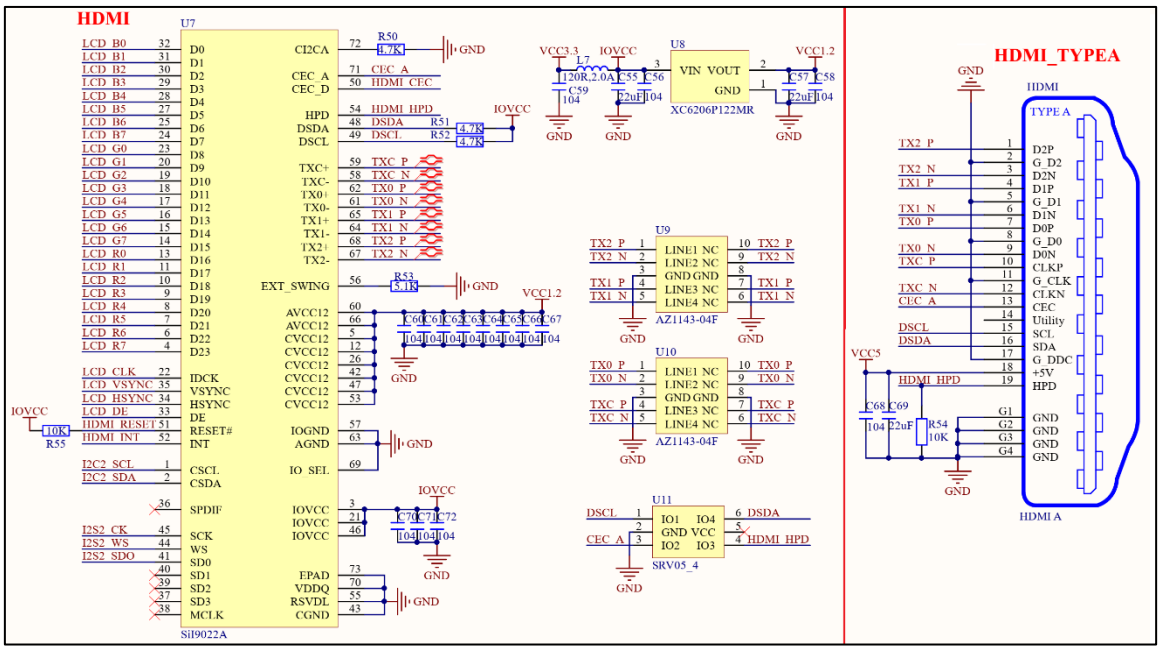


图 42.2.1 SiI9022A 的原理图

上图就是板载的 HDMI 接口，在图中可以看出 HDMI 模块分为 4 部分：RGB 接口、I2C2 接口、I2S2 音频接口和 HDMI_CEC 接口。主要还是使用 RGB 接口引脚获取显示数据，SiI9022A 使用 I2C 来进行配置，这里使用 STM32MP1 的 I2C2 接口，使用到了到 PH4 和 PH5 这两个引脚。另外还有一个中断 INT 一个复位 HDMI_RESET 分别连接到 PH6 和 PA3 引脚上。在本章节里主要是实现 HDMI 的显示功能，其它接口就不用管它。

42.3 实验驱动编写

42.3.1 修改设备树

1、设置 I2C2 的 pinmux

如果要实现 HDMI 显示，就要给 SiI9022A 提供 RGB 接口和 I2C2 接口，RGB 接口在《第四十章 LCD 驱动》的 40.5 小节讲解了如何修改，这里就不再讲解了。我们主要看一下 I2C2 接口的 pinmux 配置，打开 stm32mp15-pinctrl.dtsi 文件，然后找到如下内容：

示例代码 42.3.1.1 I2C2 的 pinmux

```
1  i2c2_pins_a: i2c2-0 {
2      pins {
3          pinmux = <STM32_PINMUX('H', 4, AF4)>,  /* I2C2_SCL */
4                   <STM32_PINMUX('H', 5, AF4)>;  /* I2C2_SDA */
5          bias-disable;
6          drive-open-drain;
7          slew-rate = <0>;
8      };
9  };
10
11 i2c2_pins_sleep_a: i2c2-1 {
12     pins {
```

```
13          pinmux = <STM32_PINMUX('H', 4, ANALOG)>,  /* I2C2_SCL */
14                   <STM32_PINMUX('H', 5, ANALOG)>;  /* I2C2_SDA */
15      };
16  };
```

示例代码 42.3.1.1 中，定义了 I2C2 接口的两个 pinmux 配置分别为：i2c2_pins_a 和 i2c2_pins_sleep_a。第一个默认的状态下使用，第二个是在 sleep 状态下使用。

2、添加 HDMI 的电源节点

Sii9022A 需要一个 1.2V 电压，这个开发板上已经提供了，图 42.2.1 中原理图上的 XC6206P122MR 芯片就是 1.2V 电源芯片。我们还需要在设备树中添加 1.2V 电压节点，供 Sii9022A 驱动程序使用。打开 stm32mp157d-atk.dts 文件，在根节点下添加如下内容所示：

示例代码 42.3.1.2 HDMI 的电源节点

```
1   v1v2_hdmi: regulator-v1v2-hdmi {
2       compatible = "regulator-fixed";
3       regulator-name = "v1v2_hdmi";
4       regulator-min-microvolt = <1200000>;
5       regulator-max-microvolt = <1200000>;
6       regulator-always-on;
7       regulator-boot-on;
8   };
```

这个没啥解析的就是提供一个简单的电压。

3、在 i2c2 节点追加 HDMI 子节点

在讲解 LCD 驱动就告诉了各位，RGB 接口是不用再次修改，我们只需要提供一个接口用来接收 LTDC 数据，HDMI 就是用来接收 LTDC 接口的。在 stm32mp157d-atk.dts 文件下，用追加节点方式，把 i2c2 节点追加如下内容：

示例代码 42.3.1.3 追加的 I2C2 节点内容

```
1   &i2c2 {
2       pinctrl-names = "default", "sleep";
3       pinctrl-0 = <&i2c2_pins_a>;
4       pinctrl-1 = <&i2c2_pins_sleep_a>;
5       status = "okay";
6
7       hdmi: hdmi-transmitter@39 {
8           compatible = "sil,sii9022";
9           reg = <0x39>;
10          iovcc-supply = <&v3v3>;
11          cvcc12-supply = <&v1v2_hdmi>;
12          reset-gpios = <&gpioa 3 GPIO_ACTIVE_LOW>;
13          interrupts = <6 IRQ_TYPE_EDGE_FALLING>;
14          interrupt-parent = <&gpioh>;
15          #sound-dai-cells = <1>;
16          status = "okay";
17
```

```
18          ports {
19              #address-cells = <1>;
20              #size-cells = <0>;
21
22              port@0 {
23                  reg = <0>;
24                  sii9022_in: endpoint {
25                      remote-endpoint = <&ltdc_ep0_out>;
26                  };
27              };
28          };
29      };
30  };
```

第 2~4 行，配置了 I2C2 的两个 pinmux 设置。

第 7~29 行，就是 I2C2 下的 HDMI 子节点，接着我们就分析这个节点比较重要的属性的作用。第 12 行，设置复位引脚为 PA3，低电压有效。第 13~14 行，设置中断引脚为 PH6，下降沿有效。第 22~27 行，port 节点就是用来接收 LTDC 数据的接口，第 25 行就是引用了 ltdc_ep0_out 节点。

这里只是告诉了 HDMI 从 LTDC 接口获取数据，还需要告诉 LTDC 节点我们数据输出到 HDMI，我们需要修改 ltdc 节点，修改完成以后的 ltdc 节点如下所示：

示例代码 42.3.1.4 基于 HDMI 接口的 LTDC

```
1   &ltdc {
2       pinctrl-names = "default", "sleep";
3       pinctrl-0 = <&ltdc_pins_b>;
4       pinctrl-1 = <&ltdc_pins_sleep_b>;
5       status = "okay";
6
7       port {
8           #address-cells = <1>;
9           #size-cells = <0>;
10
11          ltdc_ep0_out: endpoint@0 {
12              reg = <0>;
13              remote-endpoint = <&sii9022_in>;
14          };
15      };
16  };
```

第 13 行，就是告诉 LTDC 接口，数据输出到 HDMI。

注意，我们需要将 panel_rgb 节点屏蔽掉，否则编译设备树的时候会如下所示警告：

```
Warning (graph_endpoint): /panel-rgb/port/endpoint: graph connection to node '/soc/display-controller@5a001000/port/endpoint@0' is not bidirectional
```

最后执行“make dtbs”命令，重新编译设备树。

42.3.2 使能内核自带的 sii902x 驱动

ST 提供的 linux 内核已经集成了 sii902x 驱动了同已经使能了，我们还是需要教大家如何使能这个配置，配置路径如下：

```
-> Device Drivers
    -> Graphics support
        -> Display Interface Bridges
            -> <*> Silicon Image sii902x RGB/HDMI bridge      //选中
```

将 sii902x 驱动编译进 Linux 内核中，如图 42.3.2.1 所示：

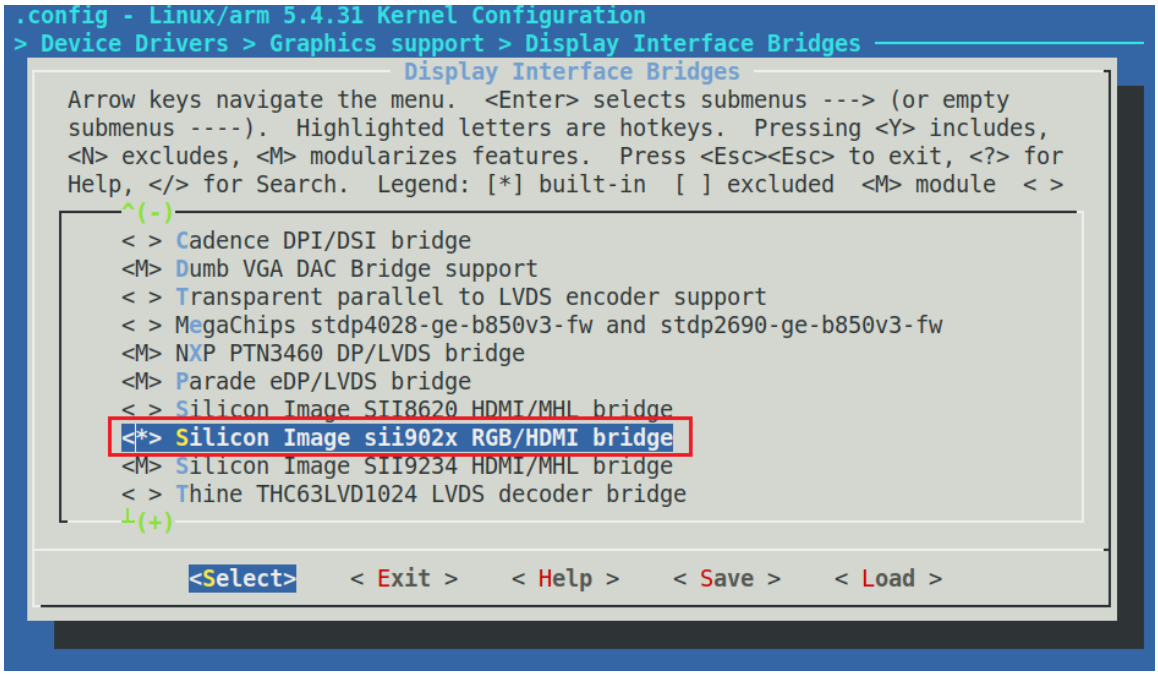


图 42.3.2.1 使能 sii902x 驱动

内核默认已经使能了 sii902x 驱动，如果没使能的话就按照上述方法使能，然后重新编译内核，得到新的内核和设备树。

42.4 RGB 转 HDMI 测试

使用 HDMI 线将开发板和显示器连接起来，然后使用新编译得到的内核和设备树启动开发板，如果驱动工作正常，那么就会在 HDMI 显示器上显示信息，如图 42.4.1 所示(屏幕截图)：

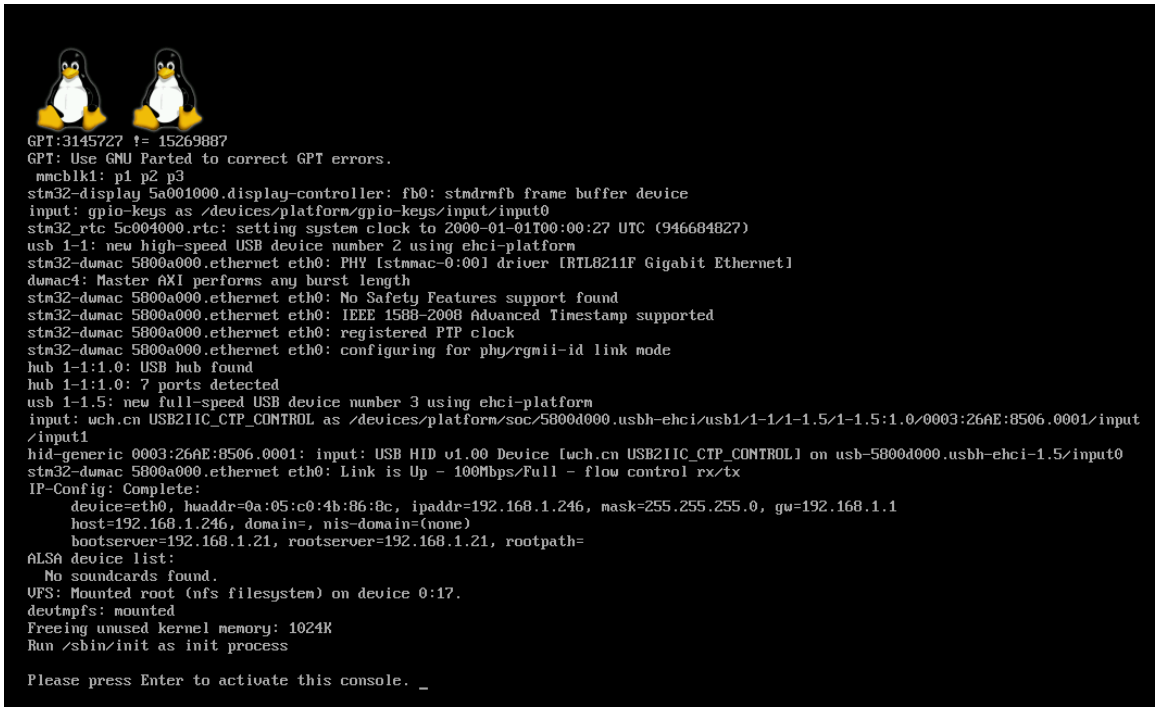


图 42.4.1 显示器显示

至此 HDMI 驱动工作正常，我们可以使用 HDMI 显示器来进行 Linux 下的图形开发，但是本教程后续依旧会使用 LCD 屏幕，大家根据自己的喜好选择 HDMI 显示器还是 LCD 屏幕即可。

第四十三章 Linux RTC 驱动实验

RTC 也就是实时时钟，用于记录当前系统时间，对于 Linux 系统而言时间是非常重要的，就和我们使用 Windows 电脑或手机查看时间一样，我们在使用 Linux 设备的时候也需要查看时间。本章我们就来学习一下如何编写 Linux 下的 RTC 驱动程序。

43.1 Linux 内核 RTC 驱动简介

RTC 设备驱动是一个标准的字符设备驱动，应用程序通过 open、release、read、write 和 ioctl 等函数完成对 RTC 设备的操作，本章我们主要学习如何使用 STM32MP1 内部自带的 RTC 外设。

Linux 内核将 RTC 设备抽象为 rtc_device 结构体，因此 RTC 设备驱动就是申请并初始化 rtc_device，最后将 rtc_device 注册到 Linux 内核里面，这样 Linux 内核就有一个 RTC 设备的。至于 RTC 设备的操作肯定是用一个操作集合(结构体)来表示的，我们先来看一下 rtc_device 构体，此结构体定义在 include/linux/rtc.h 文件中，结构体内容如下(删除条件编译)：

示例代码 43.1.1 rtc_device 结构体

```
100 struct rtc_device {
101     struct device dev;                  /*  设备            */
102     struct module *owner;
103
104     int id;                             /* ID               */
105
106     const struct rtc_class_ops *ops;    /* RTC 设备底层操作函数 */
107     struct mutex ops_lock;
108
109     struct cdev char_dev;               /* 字符设备         */
110     unsigned long flags;
111
112     unsigned long irq_data;
113     spinlock_t irq_lock;
114     wait_queue_head_t irq_queue;
115     struct fasync_struct *async_queue;
116
117     int irq_freq;
118     int max_user_freq;
119
120     struct timerqueue_head timerqueue;
121     struct rtc_timer aie_timer;
122     struct rtc_timer uie_rtctimer;
123     struct hrtimer pie_timer; /* sub second exp, so needs hrtimer */
124     int pie_enabled;
125     struct work_struct irqwork;
126     /* Some hardware can't support UIE mode */
127     int uie_unsupported;
......
159 };
```

我们需要重点关注的是 ops 成员变量，这是一个 rtc_class_ops 类型的指针变量，rtc_class_ops 为 RTC 设备的最底层操作函数集合，包括从 RTC 设备中读取时间、向 RTC 设备写入新的时间值等。因此，rtc_class_ops 是需要用户根据所使用的 RTC 设备编写的，此结构体定义在

include/linux/rtc.h 文件中，内容如下：

示例代码 43.1.2 rtc_class_ops 结构体

```
75 struct rtc_class_ops {
76     int (*ioctl)(struct device *, unsigned int, unsigned long);
77     int (*read_time)(struct device *, struct rtc_time *);
78     int (*set_time)(struct device *, struct rtc_time *);
79     int (*read_alarm)(struct device *, struct rtc_wkalrm *);
80     int (*set_alarm)(struct device *, struct rtc_wkalrm *);
81     int (*proc)(struct device *, struct seq_file *);
82     int (*alarm_irq_enable)(struct device *, unsigned int enabled);
83     int (*read_offset)(struct device *, long *offset);
84     int (*set_offset)(struct device *, long offset);
85 };
```

看名字就知道 rtc_class_ops 操作集合中的这些函数是做什么的了，但是我们要注意，rtc_class_ops 中的这些函数只是最底层的 RTC 设备操作函数，并不是提供给应用层的 file_operations 函数操作集。RTC 是个字符设备，那么肯定有字符设备的 file_operations 函数操作集，Linux 内核提供了一个 RTC 通用字符设备驱动文件，文件名为 drivers/rtc/dev.c，dev.c 文件提供了所有 RTC 设备共用的 file_operations 函数操作集，如下所示：

示例代码 43.1.3 RTC 通用 file_operations 操作集

```
431 static const struct file_operations rtc_dev_fops = {
432     .owner      = THIS_MODULE,
433     .llseek     = no_llseek,
434     .read       = rtc_dev_read,
435     .poll       = rtc_dev_poll,
436     .unlocked_ioctl = rtc_dev_ioctl,
437     .open       = rtc_dev_open,
438     .release    = rtc_dev_release,
439     .fasync     = rtc_dev_fasync,
440 };
```

看到示例代码 43.1.3 是不是很熟悉了，标准的字符设备操作集。应用程序可以通过 ioctl 函数来设置/读取时间、设置/读取闹钟的操作，对应的 rtc_dev_ioctl 函数就会执行。 rtc_dev_ioctl 最终会通过操作 rtc_class_ops 中的 read_time、set_time 等函数来对具体 RTC 设备的读写操作。我们简单来看一下 rtc_dev_ioctl 函数，函数内容如下(有省略)：

示例代码 43.1.4 rtc_dev_ioctl 函数代码段

```
202 static long rtc_dev_ioctl(struct file *file,
203             unsigned int cmd, unsigned long arg)
204 {
205     int err = 0;
206     struct rtc_device *rtc = file->private_data;
207     const struct rtc_class_ops *ops = rtc->ops;
208     struct rtc_time tm;
209     struct rtc_wkalrm alarm;
210     void __user *uarg = (void __user *)arg;
```

```
211
212     err = mutex_lock_interruptible(&rtc->ops_lock);
213     if (err)
214         return err;
......
253     switch (cmd) {
......
317     case RTC_RD_TIME:       /* 读取时间 */
318         mutex_unlock(&rtc->ops_lock);
319
320         err = rtc_read_time(rtc, &tm);
321         if (err < 0)
322             return err;
323
324         if (copy_to_user(uarg, &tm, sizeof(tm)))
325             err = -EFAULT;
326         return err;
327
328     case RTC_SET_TIME:      /* 设置时间 */
329         mutex_unlock(&rtc->ops_lock);
330
331         if (copy_from_user(&tm, uarg, sizeof(tm)))
332             return -EFAULT;
333
334         return rtc_set_time(rtc, &tm);
.....
385     default:
386         /* Finally try the driver's ioctl interface */
387         if (ops->ioctl) {
388             err = ops->ioctl(rtc->dev.parent, cmd, arg);
389             if (err == -ENOIOCTLCMD)
390                 err = -ENOTTY;
391         } else {
392             err = -ENOTTY;
393         }
394         break;
395     }
396
397 done:
398     mutex_unlock(&rtc->ops_lock);
399     return err;
400 }
```

第 317 行，RTC_RD_TIME 为时间读取命令。

第 320 行，如果是读取时间命令的话就调用 rtc_read_time 函数获取当前 RTC 时钟，rtc_read_time 会调用__rtc_read_time 函数，__rtc_read_time 函数内容如下：

示例代码 43.1.5 __rtc_read_time 函数代码段

```
84 static int __rtc_read_time(struct rtc_device *rtc, struct rtc_time *tm)
85 {
86     int err;
87
88     if (!rtc->ops) {
89         err = -ENODEV;
90     } else if (!rtc->ops->read_time) {
91         err = -EINVAL;
92     } else {
93         memset(tm, 0, sizeof(struct rtc_time));
94         err = rtc->ops->read_time(rtc->dev.parent, tm);
95         if (err < 0) {
96             dev_dbg(&rtc->dev, "read_time: fail to read: %d\n",
97                 err);
98             return err;
99         }
100
101         rtc_add_offset(rtc, tm);
102
103         err = rtc_valid_tm(tm);
104         if (err < 0)
105             dev_dbg(&rtc->dev, "read_time: rtc_time isn't valid\n");
106     }
107     return err;
108 }
```

从第 94 行可以看出，__rtc_read_time 函数会通过调用 rtc_class_ops 中的 read_time 成员变量来从 RTC 设备中获取当前时间。rtc_dev_ioctl 函数对其他的命令处理都是类似的，比如 RTC_ALM_READ 命令会通过 rtc_read_alarm 函数获取到闹钟值，而 rtc_read_alarm 函数经过层层调用，最终会调用 rtc_class_ops 中的 read_alarm 函数来获取闹钟值。

至此，Linux 内核中 RTC 驱动调用流程就很清晰了，如图 42.1.1 所示：

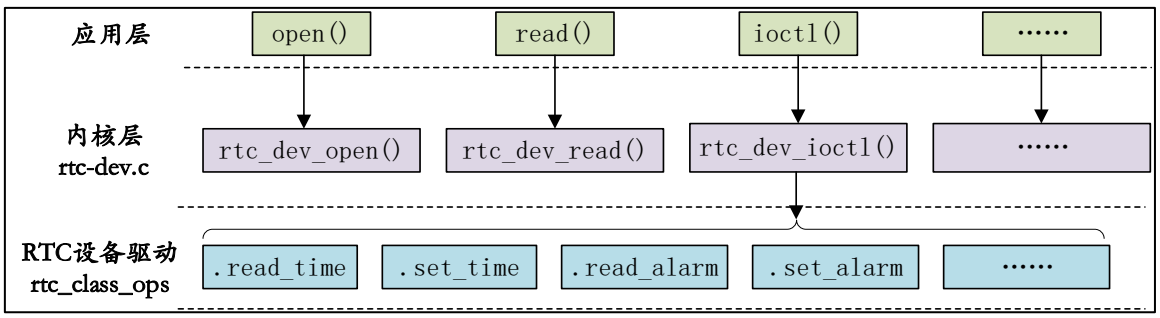


图 43.1.1 Linux RTC 驱动调用流程

当 rtc_class_ops 准备好以后需要将其注册到 Linux 内核中，这里我们可以使用

rtc_device_register 函数完成注册工作。此函数会申请一个 rtc_device 并且初始化这个 rtc_device，最后向调用者返回这个 rtc_device，此函数原型如下：

```
struct rtc_device *rtc_device_register(const char                *name,
                                       struct device             *dev,
                                       const struct rtc_class_ops *ops,
                                       struct module             *owner)
```

函数参数和返回值含义如下：

name：设备名字。

dev：设备。

ops：RTC 底层驱动函数集。

owner：驱动模块拥有者。

返回值：注册成功的话就返回 rtc_device，错误的话会返回一个负值。

当卸载 RTC 驱动的时候需要调用 rtc_device_unregister 函数来注销注册的 rtc_device，函数原型如下：

```
void rtc_device_unregister(struct rtc_device    *rtc)
```

函数参数和返回值含义如下：

rtc：要删除的 rtc_device。

返回值：无。

还有另外一对 rtc_device 注册函数 devm_rtc_device_register 和 devm_rtc_device_unregister，分别为注册和注销 rtc_device。

43.2 STM32MP1 内部 RTC 驱动分析

先直接告诉大家，STM32MP1 的 RTC 驱动我们不用自己编写，因为 ST 已经写好了。其实对于大多数的 SOC 来讲，内部 RTC 驱动都不需要我们去编写，半导体厂商会编写好。但是这不代表我们就偷懒了，虽然不用编写 RTC 驱动，但是我们得看一下这些原厂是怎么编写 RTC 驱动的。

分析驱动，先从设备树入手，打开 stm32mp151.dtsi，在里面找到如下 rtc 设备节点，节点内容如下所示：

示例代码 43.2.1 stm32mp151.dtsi 文件 rtc 节点

```
1746    rtc: rtc@5c004000 {
1747        compatible = "st,stm32mp1-rtc";
1748        reg = <0x5c004000 0x400>;
1749        clocks = <&scmi0_clk CK_SCMI0_RTCAPB>,
1750                 <&scmi0_clk CK_SCMI0_RTC>;
1751        clock-names = "pclk", "rtc_ck";
1752        interrupts-extended = <&exti 19 IRQ_TYPE_LEVEL_HIGH>;
1753        status = "disabled";
1754    };
```

第 1747 行设置兼容属性 compatible 的值为“st,stm32mp1-rtc”，因此在 Linux 内核源码中搜索此字符串即可找到对应的驱动文件，此文件为 drivers/rtc/rtc-stm32.c，在 rtc-stm32.c 文件中找到如下所示内容：

示例代码 43.2.2 设备 platform 驱动框架

```
719    static const struct of_device_id stm32_rtc_of_match[] = {
```

```
720         { .compatible = "st,stm32-rtc", .data = &stm32_rtc_data },
721         { .compatible = "st,stm32h7-rtc", .data = &stm32h7_rtc_data },
722         { .compatible = "st,stm32mp1-rtc", .data = &stm32mp1_data },
723         {}
724     };
725     MODULE_DEVICE_TABLE(of, stm32_rtc_of_match);
726
......
1020    static struct platform_driver stm32_rtc_driver = {
1021        .probe      = stm32_rtc_probe,
1022        .remove     = stm32_rtc_remove,
1023        .driver     = {
1024            .name   = DRIVER_NAME,
1025            .pm = &stm32_rtc_pm_ops,
1026            .of_match_table = stm32_rtc_of_match,
1027        },
1028    };
1029
1030    module_platform_driver(stm32_rtc_driver);
```

第 719~723 行，设备树 ID 表。第 722 行，刚好有一个 compatible 属性和设备树的 rtc 的 compatible 属性值一样，所以 rtc 设备节点会和此驱动匹配。

第 1020~1028 行，标准的 platform 驱动框架，当设备和驱动匹配成功以后 stm32_rtc_probe 函数就会执行，我们来看一下 stm32_rtc_probe 函数，函数内容如下(有省略)：

示例代码 43.2.3 stm32_rtc_probe 函数代码段

```
789 static int stm32_rtc_probe(struct platform_device *pdev)
790 {
791     struct stm32_rtc *rtc;
792     const struct stm32_rtc_registers *regs;
793     struct resource *res;
794     int ret;
795
796     rtc = devm_kzalloc(&pdev->dev, sizeof(*rtc), GFP_KERNEL);
797     if (!rtc)
798         return -ENOMEM;
799
800     res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
801     rtc->base = devm_ioremap_resource(&pdev->dev, res);
......
856     ret = clk_prepare_enable(rtc->rtc_ck);
857     if (ret)
858         goto err;
......
872     ret = stm32_rtc_init(pdev, rtc);
```

```
873     if (ret)
874         goto err;
875
876     rtc->irq_alarm = platform_get_irq(pdev, 0);
877     if (rtc->irq_alarm <= 0) {
878         ret = rtc->irq_alarm;
879         goto err;
880     }
......
892     rtc->rtc_dev = devm_rtc_device_register(&pdev->dev, pdev->name,
893                         &stm32_rtc_ops, THIS_MODULE);
894     if (IS_ERR(rtc->rtc_dev)) {
895         ret = PTR_ERR(rtc->rtc_dev);
896         dev_err(&pdev->dev, "rtc device registration failed, err=%d\n",
897             ret);
898         goto err;
899     }
900
901     /* Handle RTC alarm interrupts */
902     ret = devm_request_threaded_irq(&pdev->dev, rtc->irq_alarm, NULL,
903                     stm32_rtc_alarm_irq, IRQF_ONESHOT,
904                     pdev->name, rtc);
905     if (ret) {
906                 dev_err(&pdev->dev, "IRQ%d (alarm interrupt) already
claimed\n",
907             rtc->irq_alarm);
908         goto err;
909     }
......
940
941     return 0;
......
954 }
```

第 796 行，调用 devm_kzalloc 申请 rtc 大小的空间，返回申请空间的首地址。

第 800 行，调用 platform_get_resource 函数从设备树中获取到 RTC 外设寄存器基地址。

第 801 行，调用函数 devm_ioremap_resource 完成内存映射，得到 RTC 外设寄存器物理基地址对应的虚拟地址。

第 856 行，调用 clk_prepare_enable 函数使能时钟。

第 872 行，初始化 STM32MP1 rtc 的寄存器。

第 876 行，获取设备树的中断号。

第 892 行，调用 devm_rtc_device_register 函数向系统注册 rtc_devcie，RTC 底层驱动集为 stm32_rtc_ops。stm32_rtc_ops 操作集包含了读取/设置 RTC 时间，读取/设置闹钟等函数。

第 902 行，调用 devm_request_threaded_irq 函数请求 RTC 中断，中断服务函数为

stm32_rtc_alarm_irq，用于 RTC 闹钟中断。

stm32_rtc_ops 内容如下所示：

示例代码 43.2.4 rtc_class_ops 操作集

```
623 static const struct rtc_class_ops stm32_rtc_ops = {
624     .read_time     = stm32_rtc_read_time,
625     .set_time      = stm32_rtc_set_time,
626     .read_alarm    = stm32_rtc_read_alarm,
627     .set_alarm     = stm32_rtc_set_alarm,
628     .alarm_irq_enable = stm32_rtc_alarm_irq_enable,
629 };
```

我们就以第 624 行的 stm32_rtc_read_time 函数为例讲解一下 rtc_class_ops 的各个 RTC 底层操作函数该如何去编写。stm32_rtc_read_time 函数用于读取 RTC 时间值，此函数内容如下所示：

示例代码 43.2.5 stm32_rtc_read_time 代码段

```
364 static int stm32_rtc_read_time(struct device *dev,
                                    struct rtc_time *tm)
365 {
366     struct stm32_rtc *rtc = dev_get_drvdata(dev);
367     const struct stm32_rtc_registers *regs = &rtc->data->regs;
368     unsigned int tr, dr;
369
370     /* Time and Date in BCD format */
371     tr = readl_relaxed(rtc->base + regs->tr);
372     dr = readl_relaxed(rtc->base + regs->dr);
373
374     tm->tm_sec = (tr & STM32_RTC_TR_SEC) >> STM32_RTC_TR_SEC_SHIFT;
375     tm->tm_min = (tr & STM32_RTC_TR_MIN) >> STM32_RTC_TR_MIN_SHIFT;
376     tm->tm_hour = (tr & STM32_RTC_TR_HOUR) >> STM32_RTC_TR_HOUR_SHIFT;
377
378     tm->tm_mday = (dr & STM32_RTC_DR_DATE) >> STM32_RTC_DR_DATE_SHIFT;
379     tm->tm_mon = (dr & STM32_RTC_DR_MONTH) >> STM32_RTC_DR_MONTH_SHIFT;
380     tm->tm_year = (dr & STM32_RTC_DR_YEAR) >> STM32_RTC_DR_YEAR_SHIFT;
381     tm->tm_wday = (dr & STM32_RTC_DR_WDAY) >> STM32_RTC_DR_WDAY_SHIFT;
382
383     /* We don't report tm_yday and tm_isdst */
384
385     bcd2tm(tm);
386
387     return 0;
388 }
```

第 371~372 行，调用 readl_relaxed 读取 STM32MP1 的 RTC_TR 和 RTC_DR 这两个寄存器的值，其中 TR 寄存器为 RTC 时间寄存器，保存着时、分、秒信息；DR 为 RTC 的日期寄存器，保存着年、月、日信息。通过这两个寄存器我们就可以得到 RTC 时间

第 374~381 行，前两行获取到了 TR 和 DR 这两个寄存器的值，这里需要从这两个寄存器

值中提取出具体的年、月、日和时、分、秒信息。

第 385 行，上面得到的时间信息为 BCD 格式的，这里通过 bcd2tm 函数将 BCD 格式转换为 rtc_time 格式， rtc_time 结构体定义如下：

示例代码 43.2.6 rtc_time 结构体类型

```
20 struct rtc_time {
21     int tm_sec;
22     int tm_min;
23     int tm_hour;
24     int tm_mday;
25     int tm_mon;
26     int tm_year;
27     int tm_wday;
28     int tm_yday;
29     int tm_isdst;
30 };
```

43.3 RTC 时间查看与设置

43.3.1 使能内部 RTC

在 Linux 内核移植的时候，设备树是经过精简的，就没有启动 RTC 功能。打开 stm32mp157d-atk.dts 文件，添加如下代码所示：

示例代码 43.3.1.1 rtc 节点信息

```
1 &rtc {
2     status = "okay";
3 };
```

追加的 RTC 节点内容很简单，就是把 status 属性改为“okay”。接着我们重新编译设备树，然后使用新编译的 stm32mp157d-atk.dtb 文件启动开发板。

43.3.2 查看时间

RTC 是用来记时的，因此最基本的就是查看时间，Linux 内核启动的时候可以看到系统时钟设置信息，如图 43.3.2.1 所示：

```
stm32_rtc 5c004000.rtc: registered as rtc0
stm32_rtc 5c004000.rtc: Date/Time must be initialized
stm32_rtc 5c004000.rtc: registered rev:1.2
```

图 43.3.2.1 Linux 启动 log 信息

从图 43.3.2.1 中可以看出，Linux 内核在启动的时候将 rtc 设置为 rtc0，大家的启动信息可能会和图 43.3.2.1 中不同，但是基本上都是一样的。

如果要查看时间的话输入“date”命令即可，结果如图 43.3.2.2 所示：

```
[root@ATK-stm32mp1]:~$ date
Sat Jan  1 03:30:29 UTC 2000
[root@ATK-stm32mp1]:~$
```

图 43.3.2.2 当前时间值

从图 43.3.2.2 可以看出，当前时间为 2000 年 1 月 1 日 03:30:29，很明显时间不对，我们需

要重新设置 RTC 时间。

RTC 时间设置也是使用的 date 命令，输入“date --help”命令即可查看 date 命令如何设置系统时间，结果如图 43.3.2.3 所示：

```
[root@ATK-stm32mp1]:~$ date --help
BusyBox v1.31.1 (2020-12-22 16:23:11 CST) multi-call binary.

Usage: date [OPTIONS] [+FMT] [TIME]

Display time (using +FMT), or set time

        [-s,--set] TIME Set time to TIME
        -u,--utc        Work in UTC (don't convert to local time)
        -R,--rfc-2822   Output RFC-2822 compliant date string
        -I[SPEC]        Output ISO-8601 compliant date string
                        SPEC='date' (default) for date only,
                        'hours', 'minutes', or 'seconds' for date and
                        time to the indicated precision
        -r,--reference FILE     Display last modification time of FILE
        -d,--date TIME  Display TIME, not 'now'
        -D FMT          Use FMT (strptime format) for -d TIME conversion

Recognized TIME formats:
        hh:mm[:ss]
        [YYYY.]MM.DD-hh:mm[:ss]
        YYYY-MM-DD hh:mm[:ss]
        [[[[[YY]YY]MM]DD]hh]mm[.ss]
        'date TIME' form accepts MMDDhhmm[[YY]YY][.ss] instead
[root@ATK-stm32mp1]:~$
```

图 43.3.2.3 date 命令帮助信息

比如现在设置当前时间为 2021 年 5 月 2 日 18:53:00，因此输入如下命令：

```
date -s "2021-05-02 18:53:00"
```

设置完成以后再次使用 date 命令查看一下当前时间就会发现时间改过来了，如图 43.3.2.4 所示：

```
[root@ATK-stm32mp1]:~$ date
Sun May  2 18:53:23 UTC 2021
[root@ATK-stm32mp1]:~$
```

图 43.3.2.4 当前时间

大家注意我们使用“date -s”命令仅仅是修改了当前时间，此时间还没有写入到 STM32MP1 内部 RTC 里面或其他的 RTC 芯片里面，因此系统重启以后时间又会丢失。我们需要将当前的时间写入到 RTC 里面，这里要用到 hwclock 命令，输入如下命令将系统时间写入到 RTC 里面：

```
hwclock -w          //将当前系统时间写入到 RTC 里面
```

时间写入到 RTC 里面以后就不怕系统重启以后时间丢失了，如果 STM32MP1 开发板底板接了纽扣电池，那么开发板即使断电了时间也不会丢失。大家可以尝试一下不断电重启和断电重启这两种情况下开发板时间会不会丢失。

第四十四章 外置 RTC 芯片 PCF8563 实验

上一章我们学习了 STM32MP1 内置 RTC 外设，了解了 Linux 系统下 RTC 驱动框架。一般的应用场合使用 SOC 内置的 RTC 就可以了，而且成本也低，但是在一些对于时间精度要求比较高的场合，SOC 内置的 RTC 就不适用了。这个时候我们需要根据自己的应用要求选择合适的外置 RTC 芯片，正点原子 STM32MP1 开发板上板载了一个 RTC 芯片：PCF8563，这是一个 IIC 接口的外置 RTC 芯片，本章我们就来学习一下如何驱动外置 RTC 芯片。

44.1 PCF8563 简介

44.1.1 PCF8563 简介

PCF8563 是一个 CMOS RTC 芯片，支持时间和日历功能，支持可编程的时钟输出、中断输出以及低电压检测。PCF8563 提供了两线 IIC 接口来传输时间信息，最大传输速度为 400Kbit/S，在读写寄存器的时候地址自增，PCF8563 相关特性如下：

①、提供年、月、日、星期，时、分、秒计时，使用外置 32.768Khz 晶振。

②、低后备电流：0.25uA ，VDD=3.0V，温度 25℃。

③、IIC 接口，速度最高 400KHz。

④、可编程时钟输出，可以供其他设备使用，可输出的时钟频率有 32.768kHz、1.024kHz、32Hz 和 1Hz。

⑤、支持闹钟和定时功能。

⑥、IIC 读地址为 0XA3，写地址为 0XA2，也就是 IIC 器件地址为：0X51。

⑦，有一个开漏输出的中断引脚。

PCF8563 框图如图 44.1.1.1 所示：

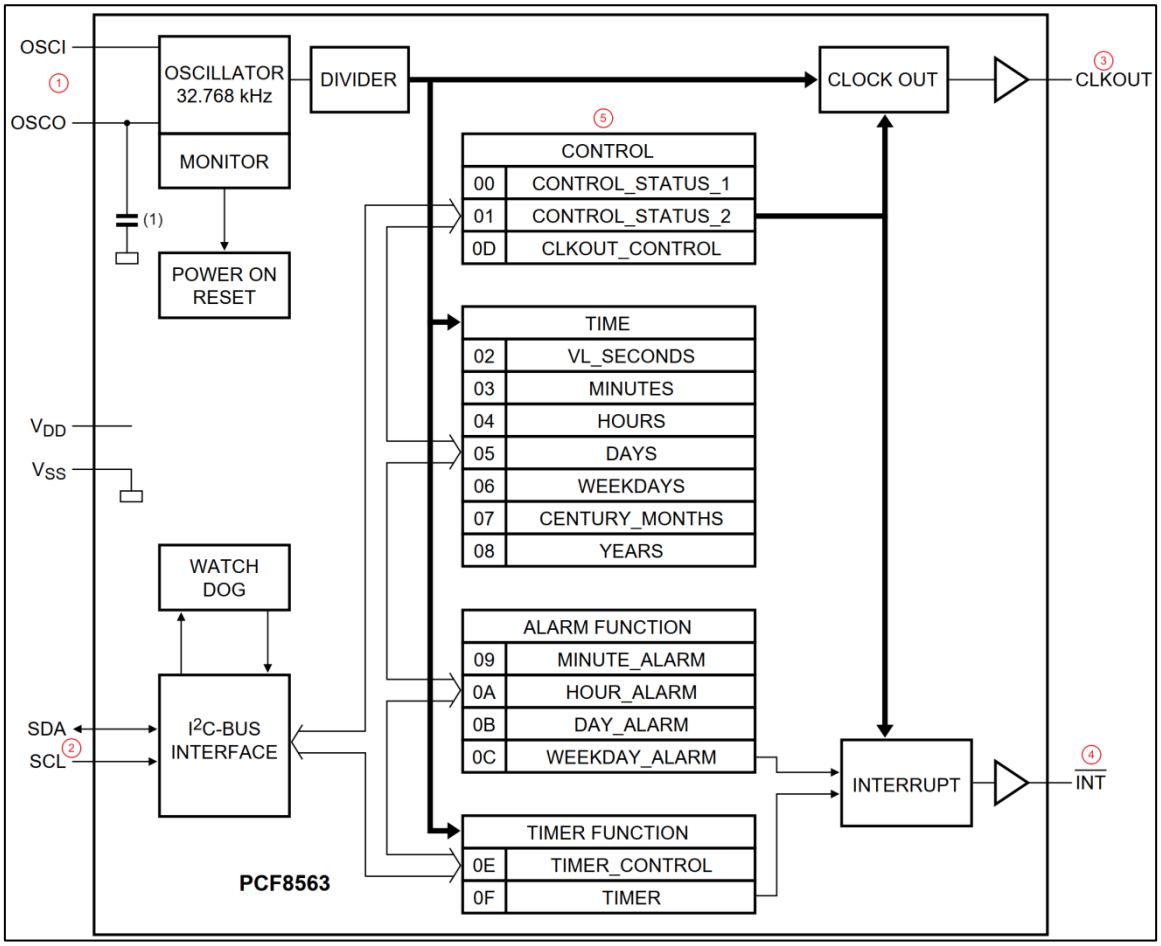


图 44.1.1.1 PCF8563 框图

简单分析一下图 44.1.1.1 中的框图：

①、这是 PCF8563 的 32.768kHz 晶振引脚，PCF8563 必选要外接 32.768kHz 晶振。

②、这是 PCF8563 的 IIC 引脚，PCF8563 通过 IIC 接口与主控进行通信，因此 PCF8563 本质是个 IIC 器件。

③、时钟输出引脚。

④、中断引脚。

⑤、前面说了，PCF8563 是个 IIC 器件，因此内部就有很多寄存器来实现 RTC 功能，比如配置芯片，读取时间信息等。这部分就是 PCF8563 的内部寄存器，稍后我们会详细分析 PCF8563 内部寄存器。

44.1.2 PCF8563 寄存器详解

PCF8563 有 16 个内部寄存器，这些寄存器都是 8 位的。前两个寄存器(0x00 和 0x01)为控制/状态寄存器。0X02~0X08 为时间和日期寄存器，这些寄存器保存着秒、分、时、日、星期、月和年信息。0X09~0X0C 为闹钟寄存器，保存闹钟信息。0X0D 为时钟输出频率寄存器，0X0E 和 0X0F 这两个寄存器时钟控制寄存器。注意、时分秒、年月日、闹钟等时间信息为 BCD 格式。

接下来我们看一下这些寄存器如何使用：

1、控制状态寄存器 1(0X00)

首先是状态寄存器，寄存器结构如图 44.1.2.1 所示：

Address	Register name	Bit							
		7	6	5	4	3	2	1	0
00h	Control_status_1	TEST1	N	STOP	N	TESTC	N	N	N

图 44.1.2.1 控制状态寄存器 1

图 44.1.2.1 是控制状态寄存器 1，相应的位含义如下：

TEST1(bit7)：0，正常模式；1，测试模式。

N(bit6,bit4,bit2~0)：未使用。

STOP(bit5)：0，RTC 时钟运行；1，RTC 时钟停止。

TESTC(bit3)：0，正常模式，关闭 POR 覆写；1，使能 POR 覆写。

2、控制状态寄存器 2(0X01)

接下来看一下控制状态寄存器 2，寄存器结构如图 44.1.2.2 所示：

Address	Register name	Bit							
		7	6	5	4	3	2	1	0
01h	Control_status_2	N	N	N	TI_TP	AF	TF	AIE	TIE

图 44.1.2.2 控制状态寄存器 2

图 44.1.2.2 是控制状态寄存器 2，相应的位含义如下：

N(bit7~5)：未使用。

TI_TP(bit4)：为 0 的时候 INT 引脚取决于 TF 位，为 1 的时候 INT 引脚输出指定频率的脉冲。

AF(bit3)：闹钟标志位，为 1 的话表示闹钟发生，写 0 清除，写 1 无效。

TF(bit2)：定时器标志位，为 1 的话表示定时发生，写 0 清除，写 1 无效。

AIE(bit1)：闹钟中断使能位，0，关闭闹钟中断；1，使能闹钟中断。

TIE(bit0)：定时器中断使能位，0，关闭定时器中断；1，使能定时器中断。

3、时间和日期寄存器(0X02~0X08)

接下来看一下时间和日期相关寄存器，一共 7 个寄存器，结构如图 44.1.2.3 所示：

Address	Register name	Bit							
		7	6	5	4	3	2	1	0
Time and date registers									
02h	VL_seconds	VL	SECONDS (0 to 59)						
03h	Minutes	x	MINUTES (0 to 59)						
04h	Hours	x	x	HOURS (0 to 23)					
05h	Days	x	x	DAYS (1 to 31)					
06h	Weekdays	x	x	x	x	x	WEEKDAYS (0 to 6)		
07h	Century_months	C	x	x	MONTHS (1 to 12)				
08h	Years	YEARS (0 to 99)							

图 44.1.2.3 时间和日期寄存器

我们依次来看一下图 44.1.2.3 中的这些寄存器：

0X02：此寄存器为秒钟寄存器，PCF8563 是有低电压检测的，当 VDD 电压低于最小允许电压的时候 VL(bit)位就会置 1，表示时钟异常，如果电压正常的话就为 0。 SECONDS(bit6~0)这 7 位表示具体的秒数，范围 0~59，为 BCD 格式。

0X03：此寄存器为分钟寄存器，MINUTES(bit6~0)这 7 位有效，表示具体的分钟数，范围 0~59，为 BCD 格式。

0X04：此寄存器为小时寄存器，HOURS(bit5~0)这 6 位有效，表示具体的小时数，范围 0~23，为 BCD 格式。

0X05：此寄存器为日期寄存器，DAYS(bit5~0)这 6 位有效，表示具体的小时数，范围 1~31，为 BCD 格式。

0X06：此寄存器为星期寄存器，WEEKDAYS(bit2~0)这 3 位有效，表示具体的星期，范围 0~6，为 BCD 格式。

0X06：此寄存器为星期寄存器，WEEKDAYS(bit2~0)这 3 位有效，表示具体的星期，范围 0~6，为 BCD 格式。0 为星期日，1 为星期一，以此类推，6 就是星期六。

0X07：此寄存器为月份寄存器，其中 C(bit7)为世纪标志位，如果为 1 的话表示 20xx 年，为 0 的话表示 19xx 年。MONTHS(bit4~0)这 5 位有效，表示具体的月份，范围 1~12，分别为 1~12 月，为 BCD 格式。

0X08：此寄存器为年寄存器，YEARS(bit7~0)这 8 位有效，表示具体的年份，范围 0~99。

4、闹钟寄存器(0X09~0X0C)

接下来看一下闹钟相关寄存器，一共 4 个寄存器，结构如图 44.1.2.4 所示：

Address	Register name	Bit							
		7	6	5	4	3	2	1	0
09h	Minute_alarm	AE_M	MINUTE_ALARM (0 to 59)						
0Ah	Hour_alarm	AE_H	x	HOUR_ALARM (0 to 23)					
0Bh	Day_alarm	AE_D	x	DAY_ALARM (1 to 31)					
0Ch	Weekday_alarm	AE_W	x	x	x	x	WEEKDAY_ALARM (0 to 6)		

图 44.1.2.4 闹钟寄存器

我们依次来看一下图 44.1.2.4 中的这些寄存器：

0X09：此寄存器为闹钟分钟寄存器，AE_M(bit7)为分钟闹钟使能位，为 0 的话使能分钟闹钟，为 1 的话关闭。 MINUTE_ALARM(bit6~0)这 7 位表示具体的闹钟分钟，范围 0~59，为 BCD 格式。

0X0A：此寄存器为闹钟小时寄存器，含义和 0X09 寄存器类似。

0X0B：此寄存器为闹钟日期寄存器，含义和 0X09 寄存器类似。

0X0C：此寄存器为闹钟日期寄存器，含义和 0X09 寄存器类似。

另外还有时钟输出寄存(0X0D)以及定时器寄存器(0X0E 和 0X0F)，这里我们不用 PFC8563 的时钟输出和定时器功能，这里就不讲解了，感兴趣的可以参考 PCF8563 数据手册。

总体来说，PCF8563 还是很简单的，这是一个 IIC 接口的 RTC 芯片，因此在 Linux 系统下就涉及到两类驱动：

①、IIC 驱动，需要 IIC 驱动框架来读写 PCF8563 芯片。

②、RTC 驱动，因为这是一个 RTC 芯片，因此要用到 RTC 驱动框架。

如果要用到中断功能的话，还需要用到 Linux 系统中的中断子系统，这些我们前面都有相应的实验讲解。所以 PCF8563 的 Linux 驱动并不复杂，而且重点是 Linux 系统默认就已经集成了 PCF8563 驱动，我们使用起来非常简单，直接修改设备树，添加 PCF8563 节点信息，然后使能内核的 PCF8563 驱动即可。

44.2 硬件原理图分析

PCF8563 原理图如图 44.2.1 所示：

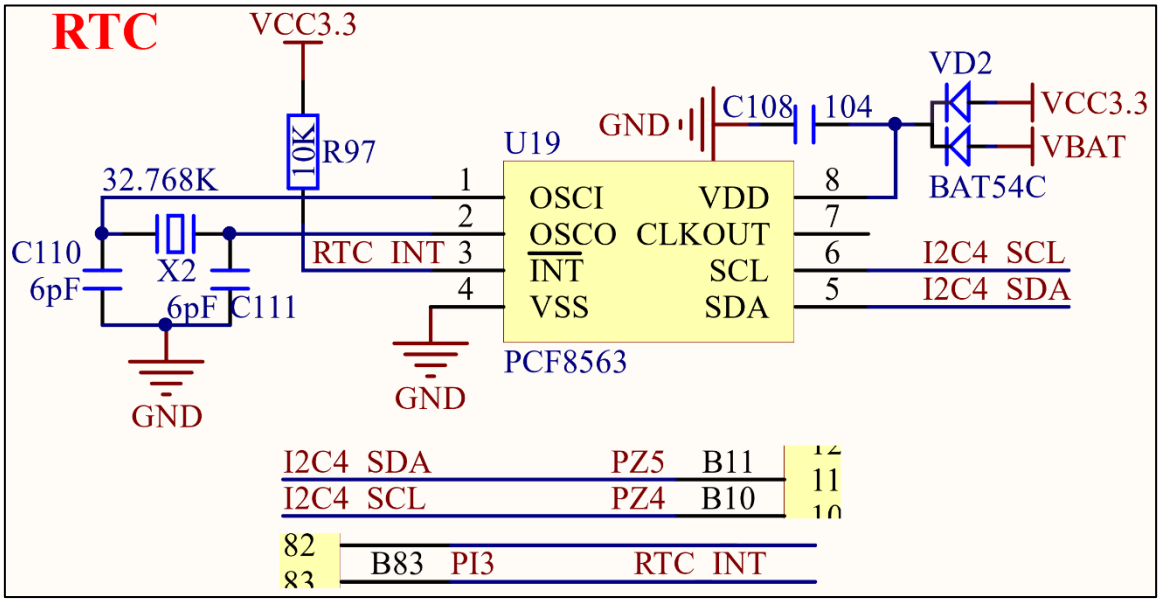


图 44.2.1 PCF8563 原理图

从图 44.2.1 可以看出，PCF8563 连接到了 STM32MP157 的 I2C4 接口上，引脚为 PZ5、PZ4。另外，PCF8563 的 INT 引脚连接到了 STM32MP157 的 PI3 引脚上。

44.3 实验驱动编写

44.3.1 修改设备树

1、添加或者查找 PCF8563 所使用的 IO 的 pinmux 配置

前面说了，PCF8563 的 IIC 接口连接到了 STM32MP157 的 I2C4 上，对应的引脚为 PZ4 和 PZ5。另外还有一个中断引脚 PI3，我们首先需要在设备树中添加这 3 个引脚对应的配置信息。首先添加 PZ4 和 PZ5，打开 stm32mp15-pincrtl.dtsi 文件，查找一下有没有 I2C4 的引脚配置信息，默认是有的，内容如下：

示例代码 44.3.1.1 i2c4 引脚节点

```
1  i2c4_pins_a: i2c4-0 {
2      pins {
3          pinmux = <STM32_PINMUX('Z', 4, AF6)>,      /* I2C4_SCL   */
4                   <STM32_PINMUX('Z', 5, AF6)>;  /* I2C4_SDA   */
5          bias-disable;
6          drive-open-drain;
7          slew-rate = <0>;
8      };
9  };
10
11 i2c4_pins_sleep_a: i2c4-1 {
12     pins {
13         pinmux = <STM32_PINMUX('Z', 4, ANALOG)>,  /* I2C4_SCL   */
14                  <STM32_PINMUX('Z', 5, ANALOG)>;  /* I2C4_SDA   */
15     };
16 };
```

从第 3、4 行可以看出，I2C4 默认引脚就是 PZ4 和 PZ5，和我们本实验一样，所以 I2C4 的引脚不需要我们修改，直接使用 i2c4_pins_a 即可。接下来还需要定义中断引脚 PI3 的引脚信息，前面讲过了，如果一个引脚作为 GPIO 功能的话可以不用添加此引脚 pinctrl 信息。

2、在 I2C4 节点下添加 pinmux 并追加 pcf8563 子节点

前面说了 linux 内核内部已经集成了 PCF8563 驱动，所以肯定有文档描述如何使用这个驱动。打开 Documentation/devicetree/bindings/rtc/pcf8563.txt，此文档描述了如何使用 Linux 内核自带的 pcf8563 驱动，也给出了参考设备节点，大家参考此文档即可。

在 stm32mp157d-atk.dts 文件，追加 I2C4 节点，追加如下所示内容：

示例代码 44.3.1.3 追加 pcf8563 节点

```
1  &i2c4 {
2      pinctrl-names = "default", "sleep";
3      pinctrl-0 = <&i2c4_pins_a>;
4      pinctrl-1 = <&i2c4_pins_sleep_a>;
5      status = "okay";
6
7      pcf8563@51{
8          compatible = "nxp,pcf8563";
9          irq_gpio = <&gpioi 3 IRQ_TYPE_EDGE_FALLING>;
10         reg = <0x51>;
11     };
12 };
```

第 2~4 行，设置 IO 要使用的 pinmux 配置。

第 7~10 行，pcf8563 设备子节点，第 8 行设置 compatible 为“nxp,pcf8563”，这个是必须的，否则无法匹配 linux 内核自带的 pcf8563 驱动。从第 9 行设置 pcf8563 中断引脚为 PI3，下降沿触发。pcf8563 的I2C 地址为0X51，因此reg 为0X51。(这里的51不是很理解,好像52也行)

44.3.2 PCF8563 驱动使能

1、关闭 STM32MP157 内置 RTC 驱动

上一个实验我们使能了 STM32MP157 内部 RTC，为了防止干扰，所以要先关闭内部 RTC！配置路径为：

```
-> Device Drivers
    -> Real Time Clock
        -> STM32 RTC        //取消选中
```

如图 44.3.2.1 所示：

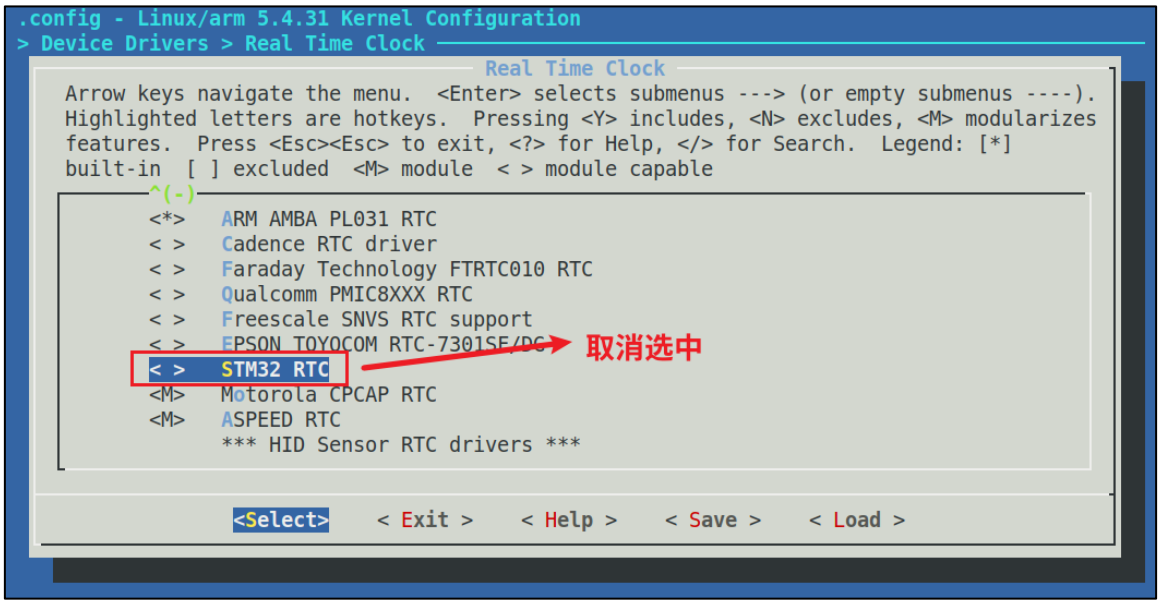


图 44.3.2.1 关闭 STM32MP157 内部 RTC

2、使能 Linux 内核自带的 PCF8563 驱动

接下来需要使能 linux 内核自带的 PCF8563 驱动，配置路径如下：

```
-> Device Drivers
    -> Real Time Clock
        -> <*> Philips PCF8563/Epson RTC8564        //选中 PCF8563
```

如图 44.3.2.2 所示：

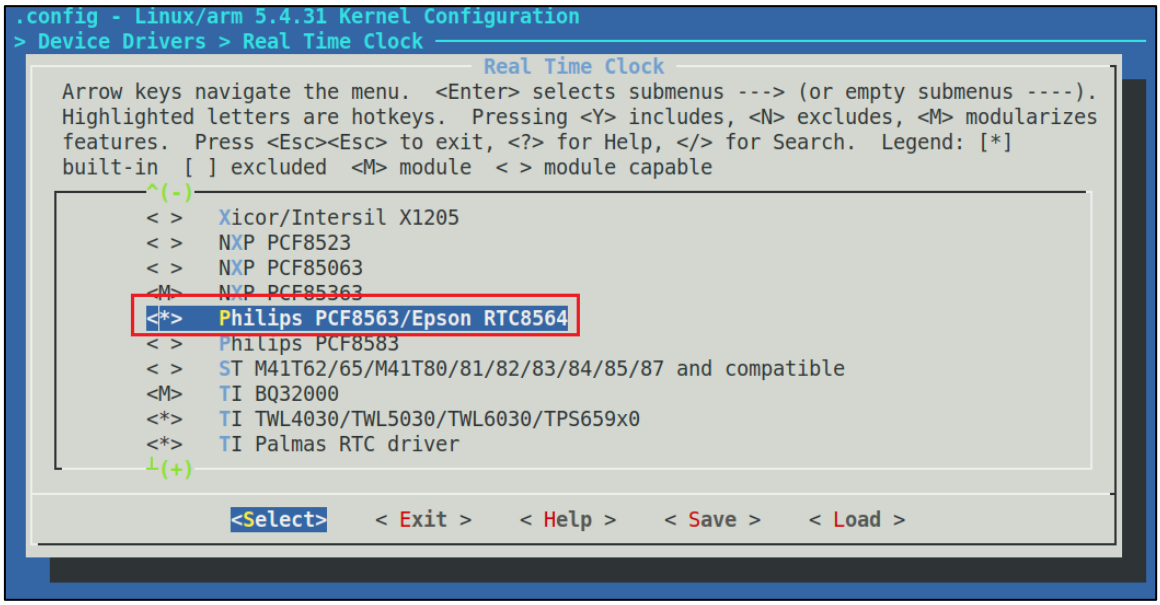


图 44.3.2.2 使能 PCF8563 驱动

配置完成以后重新编译内核和设备树，得到新的 uImage 和 stm32mp157d-atk.dtb。

44.4 运行测试

使用上面编译得到的内核和设备树启动开发板。当系统第一次启动，我们没有设置 PCF8563 时间的时候，启动过程会提示如图 44.4.1 所示信息：

```
input: gpio-keys as /devices/platform/gpio-keys/input/input0
rtc-pcf8563 3-0051: low voltage detected, date/time is not reliable.
rtc-pcf8563 3-0051: hctosys: unable to read the hardware clock
stm32-dwmac 5800a000.ethernet eth0: PHY [stmmac-0:00] driver [YT8511 Gigabit Ethernet]
```

图 44.4.1 PCF8563 启动过程

从图 44.4.1 可以看出，系统已经识别出了 PCF8563，说明驱动没问题。但是，这里提示检测到低电压，日期和时间无效。这是因为我们没有设置时间，等系统启动成功，然后参考上一章的方法设置 RTC 时间，比如我这里设置时间为 2021 年 5 月 21 号，下午 15:52:00，输入如下命令：

```
date -s "2021-05-21 15:52:00"      //设置时间
hwclock -w                         //保存
```

时间设置好以后重启系统，此时系统 log 信息如图 44.4.2 所示：

```
 mmcblk1: p1 p2 p3
input: gpio-keys as /devices/platform/gpio-keys/input/input0
rtc-pcf8563 3-0051: setting system clock to 2021-05-21T15:53:29 UTC (1621612409)
stm32-dwmac 5800a000.ethernet eth0: PHY [stmmac-0:00] driver [YT8511 Gigabit Ethernet]
```

图 44.4.2 PCF8563 启动信息

从图 44.4.2 可以看出，此时 PCF8563 再没有提示电压低的错误，而且正确的读出了时间信息，整个开发板掉电以后 PCF8563 也会继续计时，因为有一个纽扣电池供电。

44.5 PCF8563 驱动分析

上一小节我们已经测试了 PCF8563，本小节我们来简单看一下 PCF8563 驱动源码，根据示例代码 44.3.1.3 中的第 8 行的 compatible 属性值可以找到对应到驱动文件，在 linux 源码中搜索

字符串“nxp,pcf8563”即可找到对应的驱动文件，驱动文件为 drivers/rtc/rtc-pcf8563.c。

PCF8563 是个 I2C 器件，因此基础驱动框架是 I2C，在 rtc-pcf8563.c 文件中找到如下所示内容：

示例代码 44.5.1 pcf8563 I2C 驱动框架

```
1  static const struct i2c_device_id pcf8563_id[] = {
2      { "pcf8563", 0 },
3      { "rtc8564", 0 },
4      { }
5  };
6  MODULE_DEVICE_TABLE(i2c, pcf8563_id);
7
8  #ifdef CONFIG_OF
9  static const struct of_device_id pcf8563_of_match[] = {
10     { .compatible = "nxp,pcf8563" },
11     { .compatible = "epson,rtc8564" },
12     { .compatible = "microcrystal,rv8564" },
13     {}
14 };
15 MODULE_DEVICE_TABLE(of, pcf8563_of_match);
16 #endif
17
18 static struct i2c_driver pcf8563_driver = {
19     .driver     = {
20         .name   = "rtc-pcf8563",
21         .of_match_table = of_match_ptr(pcf8563_of_match),
22     },
23     .probe      = pcf8563_probe,
24     .id_table   = pcf8563_id,
25 };
26
27 module_i2c_driver(pcf8563_driver);
```

上述示例代码就是个标准的 I2C 驱动框架，第 9~14 行的 pcf8563_of_match 结构体数组就是设备树匹配数组，第 10 行的 compatible 属性为“nxp,pcf8563”，和我们的设备树向匹配。匹配以后第 23 行的 pcf8563_probe 函数就会执行。

接下来看一下 pcf8563_probe 函数，函数源码如下(有缩略)：

示例代码 44.5.2 pcf8563_probe 函数

```
1  static int pcf8563_probe(struct i2c_client *client,
2              const struct i2c_device_id *id)
3  {
4   struct pcf8563 *pcf8563;
5   int err;
6   unsigned char buf;
......
```

```
13  pcf8563 = devm_kzalloc(&client->dev, sizeof(struct pcf8563),
14            GFP_KERNEL);
15  if (!pcf8563)
16      return -ENOMEM;
17
18  i2c_set_clientdata(client, pcf8563);
19  pcf8563->client = client;
20  device_set_wakeup_capable(&client->dev, 1);
21
22  /* Set timer to lowest frequency to save power */
23  buf = PCF8563_TMRC_1_60;
24  err = pcf8563_write_block_data(client, PCF8563_REG_TMRC, 1, &buf);
25  if (err < 0) {
26      dev_err(&client->dev, "%s: write error\n", __func__);
27      return err;
28  }
29
30  /* Clear flags and disable interrupts */
31  buf = 0;
32  err = pcf8563_write_block_data(client, PCF8563_REG_ST2, 1, &buf);
33  if (err < 0) {
34      dev_err(&client->dev, "%s: write error\n", __func__);
35      return err;
36  }
37
38  pcf8563->rtc = devm_rtc_allocate_device(&client->dev);
39  if (IS_ERR(pcf8563->rtc))
40      return PTR_ERR(pcf8563->rtc);
41
42  pcf8563->rtc->ops = &pcf8563_rtc_ops;
43  /* the pcf8563 alarm only supports a minute accuracy */
44  pcf8563->rtc->uie_unsupported = 1;
45  pcf8563->rtc->range_min = RTC_TIMESTAMP_BEGIN_2000;
46  pcf8563->rtc->range_max = RTC_TIMESTAMP_END_2099;
47  pcf8563->rtc->set_start_time = true;
48
49  if (client->irq > 0) {
50      err = devm_request_threaded_irq(&client->dev, client->irq,
51              NULL, pcf8563_irq,
52              IRQF_SHARED | IRQF_ONESHOT | IRQF_TRIGGER_LOW,
53              pcf8563_driver.driver.name, client);
54      if (err) {
55          dev_err(&client->dev, "unable to request IRQ %d\n",
```

```
56                              client->irq);
57          return err;
58      }
59  }
60
61  err = rtc_register_device(pcf8563->rtc);
62  if (err)
63      return err;
......
70  return 0;
71 }
```

第 13 行，申请内存内存，rtc-pcf8563.c 定义了一个 pcf8563 结构体来描述 PCF8563 芯片，所以这里就是申请一个 pcf8563 实例。

第 23~36 行，初始化 PCF8563。

第 38 行，pcf8563 结构体里面有个 rtc 成员变量，此成员变量是个 rtc_device 结构体指针。看到这里，大家应该很熟悉了，这个就是上一章讲解的 RTC 驱动框架最核心的 rtc_device。这里需要对这个 rtc 指针分配内存。

第 42 行，设置 rtc_device 的 ops 成员变量为 pcf8563_rtc_ops，pcf8563_rtc_ops 包含了 PCF8563 的具体操作，包括设置时间、读取时间、设置闹钟等。

第 44~47 行，继续初始化 rtc 的其他成员变量。

第 49~59 行，中断初始化，PCF8563 有个中断引脚 INT，因此可以使用中断功能。这里使用 devm_request_threaded_irq 函数完成中断申请已经初始化，中断函数为 pcf8563_irq。

第 61 行，调用 rtc_register_device 函数向系统注册 rtc_device，也就是 pcf8563。

总结一下，pcf8563_probe 函数的核心就是初始化 PCF8563，然后使用上一章讲的 RTC 驱动框架来设置 PCF8563，然后向内核注册。

接下来我们看一下 PCF8563 的核心：pcf8563_rtc_ops，内容如下：

示例代码 44.5.3 pcf8563_rtc_ops

```
1 static const struct rtc_class_ops pcf8563_rtc_ops = {
2       .ioctl        = pcf8563_rtc_ioctl,
3       .read_time    = pcf8563_rtc_read_time,
4       .set_time     = pcf8563_rtc_set_time,
5       .read_alarm   = pcf8563_rtc_read_alarm,
6       .set_alarm    = pcf8563_rtc_set_alarm,
7       .alarm_irq_enable = pcf8563_irq_enable,
8 };
```

pcf8563_rtc_ops 提供了 PCF8563 的时间以及闹钟读写操作函数，应用程序对 PCF8563 的所有操作最终都是通过这些函数来完成的。我们以读时间为例，当应用程序读取 PCF8563 当前时间的时候，.read_time 就会执行，在这里就是 pcf8563_rtc_read_time，函数源码如下(有省略)：

示例代码 44.5.4 pcf8563_rtc_read_time 函数

```
1  static int pcf8563_rtc_read_time(struct device *dev,
                                    struct rtc_time *tm)
2  {
3       struct i2c_client *client = to_i2c_client(dev);
```

```
4       struct pcf8563 *pcf8563 = i2c_get_clientdata(client);
5       unsigned char buf[9];
6       int err;
7
8       err = pcf8563_read_block_data(client, PCF8563_REG_ST1, 9, buf);
9       if (err)
10          return err;
11
12      if (buf[PCF8563_REG_SC] & PCF8563_SC_LV) {
13          pcf8563->voltage_low = 1;
14          dev_err(&client->dev,
15              "low voltage detected, date/time is not reliable.\n");
16          return -EINVAL;
17      }
......
28      tm->tm_sec = bcd2bin(buf[PCF8563_REG_SC] & 0x7F);
29      tm->tm_min = bcd2bin(buf[PCF8563_REG_MN] & 0x7F);
30      tm->tm_hour = bcd2bin(buf[PCF8563_REG_HR] & 0x3F);
31      tm->tm_mday = bcd2bin(buf[PCF8563_REG_DM] & 0x3F);
32      tm->tm_wday = buf[PCF8563_REG_DW] & 0x07;
33      tm->tm_mon = bcd2bin(buf[PCF8563_REG_MO] & 0x1F) - 1; 、
34      tm->tm_year = bcd2bin(buf[PCF8563_REG_YR]) + 100;
35      /* detect the polarity heuristically. see note above. */
36      pcf8563->c_polarity = (buf[PCF8563_REG_MO] & PCF8563_MO_C) ?
37          (tm->tm_year >= 100) : (tm->tm_year < 100);
......
45      return 0;
46  }
```

第 8 行，使用 pcf8563_read_block_data 函数从 PCF8563_REG_ST1 寄存器(地址为 0X00)开始，连续读取 9 个寄存器的数据。这样就可以得到 PCF8563 的控制与状态寄存器 1 和 2，以及事件与日期寄存器的值。

第 12 行，判断 PCF8563 的 0X02 寄存器 VL 位是否为 1，也就是检查 PCF8563 是否处于低电压模式，事件和日期是否有效。

第 28~34 行，依次获取 PCF8563 中的时间和日期值，这里使用 bcd2bin 函数将原始的 BCD 值转换为时间值。将获取到的时间和日期打包到参数 tm 中，tm 是个 rtc_time 结构体指针变量。

第 36 行，判断 0X07 寄存器的 C 位(bit7)的值，此位为 1 的话表示 20xx 年，为 0 的话就是 19xx 年。

可以看出 pcf8563_rtc_read_time 函数很简单，就是读取 PCF8563 内部的时间和日期值，然后将其打包进 rtc_time 里面。其他的函数大同小异，大家可以自行分析一下，这里就不讲解了。

至此，PCF8563 驱动就简单分析完成了，其他 IIC 接口的 RTC 芯片驱动基本都是类似的，大家可以在实际项目开发中选择合适的 RTC 芯片。

第四十五章 Linux SPI 驱动实验

到目前为止，我们已经给大家介绍了 Linux 下的 platform 总线框架、I2C 总线框架，本章将向大家介绍 Linux 下的 SPI 总线框架。与 I2C 总线一样，SPI 是物理总线，也是一种很常用的串行通信协议；本章我们就来学习如何在 Linux 下编写 SPI 总线接口的设备驱动。本章实验的最终目的就是驱动 STM32MP1 开发板上的 ICM-20608 这个 SPI 接口的六轴传感器，可以在应用程序中读取 ICM-20608 的原始传感器数据。

45.1 SPI & ICM-20608 简介

45.1.1 SPI 简介

上一章我们讲解了 I2C，I2C 是串行通信的一种，只需要两根线就可以完成主机和从机之间的通信，但是 I2C 的速度最高只能到 400KHz，如果对于访问速度要求比价高的话 I2C 就不适合了。本章我们就来学习一下另外一个和 I2C 一样广泛使用的串行通信：SPI，SPI 全称是 Serial Perripheral Interface，也就是串行外围设备接口。SPI 是 Motorola 公司推出的一种同步串行接口技术，是一种高速、全双工的同步通信总线，SPI 时钟频率相比 I2C 要高很多，最高可以工作在上百 MHz。SPI 以主从方式工作，通常是有一个主设备和一个或多个从设备，一般 SPI 需要 4 根线，但是也可以使用三根线(单向传输)，本章我们讲解标准的 4 线 SPI，这四根线如下：

①、CS/SS，Slave Select/Chip Select，这个是片选信号线，用于选择需要进行通信的从设备。I2C 主机是通过发送从机设备地址来选择需要进行通信的从机设备的，SPI 主机不需要发送从机设备，直接将相应的从机设备片选信号拉低即可。

②、SCK，Serial Clock，串行时钟，和 I2C 的 SCL 一样，为 SPI 通信提供时钟。

③、MOSI/SDO，Master Out Slave In/Serial Data Output，简称主出从入信号线，这根数据线只能用于主机向从机发送数据，也就是主机输出，从机输入。

④、MISO/SDI，Master In Slave Out/Serial Data Input，简称主入从出信号线，这根数据线只能用户从机向主机发送数据，也就是主机输入，从机输出。

SPI 通信都是由主机发起的，主机需要提供通信的时钟信号。主机通过 SPI 线连接多个从设备的结构如图 45.1.1.1 所示：

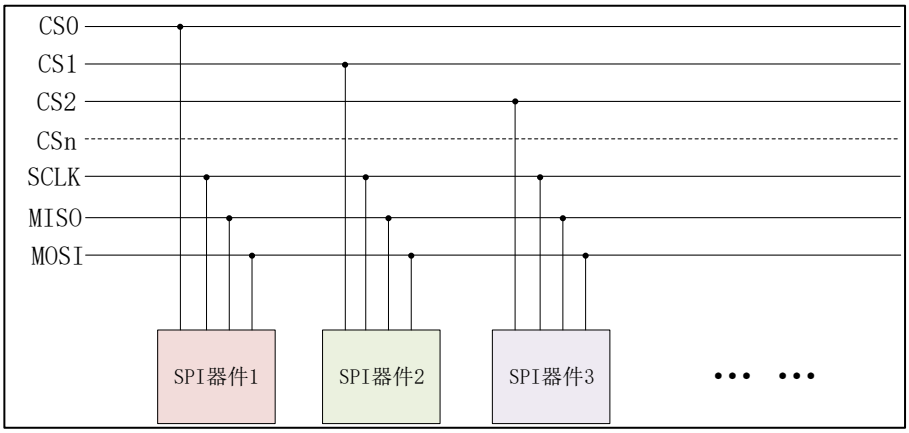


图 45.1.1.1 SPI 设备连接图

SPI 有四种工作模式，通过串行时钟极性(CPOL)和相位(CPHA)的搭配来得到四种工作模式：

①、CPOL=0，串行时钟空闲状态为低电平。

②、CPOL=1，串行时钟空闲状态为高电平，此时可以通过配置时钟相位(CPHA)来选择具体的传输协议。

③、CPHA=0，串行时钟的第一个跳变沿(上升沿或下降沿)采集数据。

④、CPHA=1，串行时钟的第二个跳变沿(上升沿或下降沿)采集数据。

这四种工作模式如图 45.1.1.2 所示：

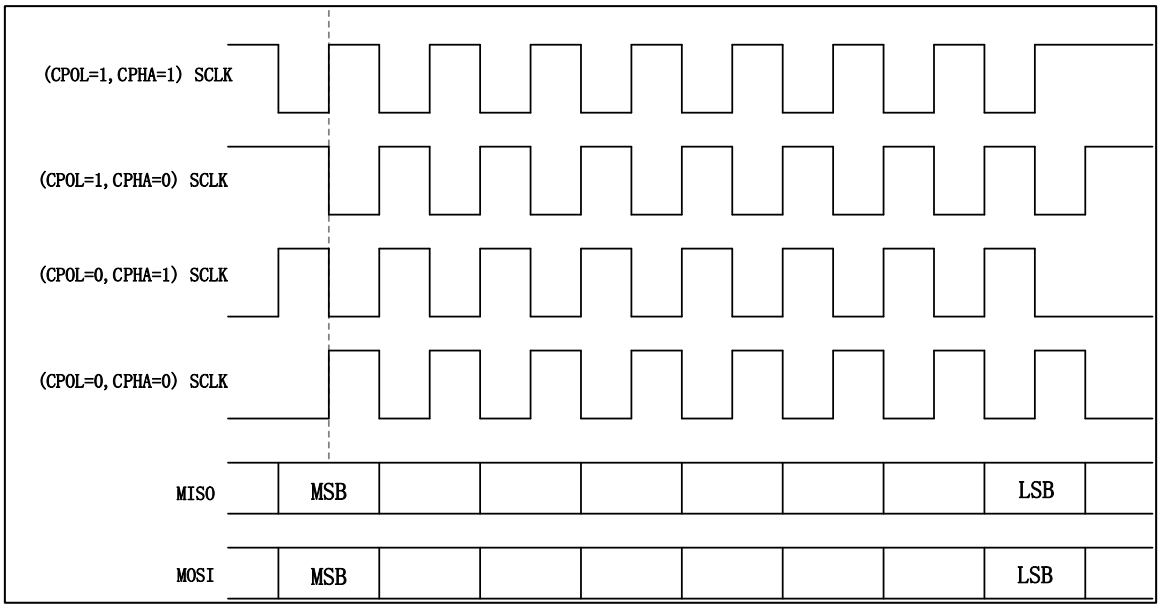


图 45.1.1.2 SPI 四种工作模式

和 I2C 一样，SPI 也是有时序图的，以 CPOL=0，CPHA=0 这个工作模式为例，SPI 进行全双工通信的时序如图 45.1.1.3 所示：

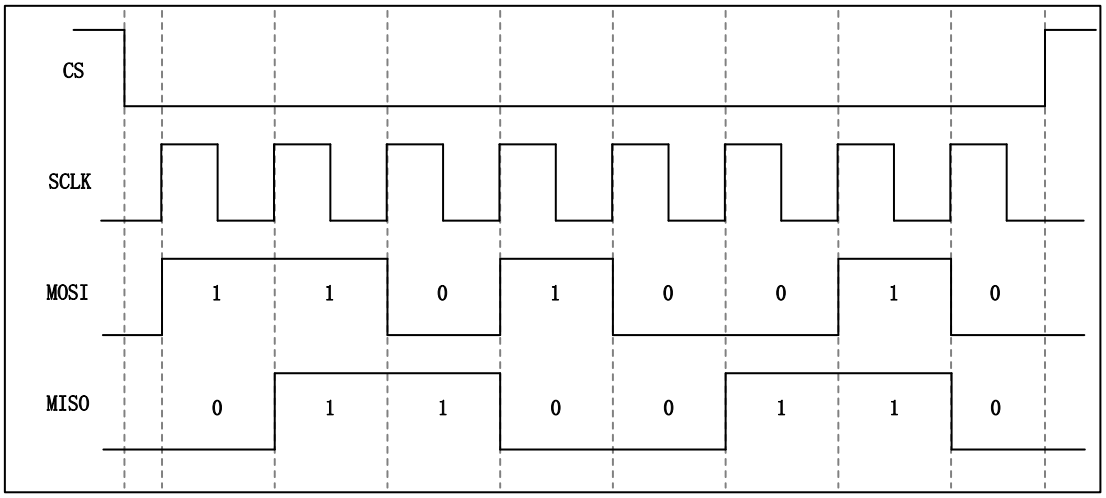


图 45.1.1.3 SPI 时序图

从图 45.1.1.3 可以看出，SPI 的时序图很简单，不像 I2C 那样还要分为读时序和写时序，因为 SPI 是全双工的，所以读写时序可以一起完成。图 45.1.1.3 中，CS 片选信号先拉低，选中要通信的从设备，然后通过 MOSI 和 MISO 这两根数据线进行收发数据，MOSI 数据线发出了 0XD2 这个数据给从设备，同时从设备也通过 MISO 线给主设备返回了 0X66 这个数据。这个就是 SPI 时序图。

关于 SPI 就讲解到这里，接下来我们看一下 STM32MP1 自带的 SPI 外设。

45.1.2 STM32MP1 SPI 简介

STM32MP1 自带的 SPI 全称为：Serial peripheral interface。SPI 特性如下：

①、全双工同步串口接口。

②、半双工模式。

③、可配置的主/从模式。

④、支持 I2S 协议。

⑤、在达到 FIFO 阈值、超时、操作完成以及发生访问错误时产生中断。

⑥、允许 16 位，24 位或者 32 位数据长度。

⑦、支持软件片选和硬件片选。

STM32MP1 的 SPI 可以工作在主模或从模式，本章我们使用主模式，此芯片有 6 个 SPI，其中 SPI1~3 是支持 I2S 协议。在主模式下，可以选择硬件片选和软件片选，如果使用了硬件片选，那么每一个 SPI 只支持一个外设，软件片选就可以支持无数个外设，本章实验我们不使用硬件片选信号，因为硬件片选信号只能使用指定的片选 IO，软件片选的话可以使用任意的 IO。

45.1.3 ICM-20608 简介

ICM-20608 是 InvenSense 出品的一款 6 轴 MEMS 传感器，包括 3 轴加速度和 3 轴陀螺仪。ICM-20608 尺寸非常小，只有 3x3x0.75mm，采用 16P 的 LGA 封装。ICM-20608 内部有一个 512 字节的 FIFO。陀螺仪的量程范围可以编程设置，可选择±250，±500，±1000 和±2000°/s，加速度的量程范围也可以编程设置，可选择±2g，±4g，±4g，±8g 和±16g。陀螺仪和加速度计都是 16 位的 ADC，并且支持 I2C 和 SPI 两种协议，使用 I2C 接口的话通信速度最高可以达到 400KHz，使用 SPI 接口的话通信速度最高可达到 8MHz。开发板上的 ICM-20608 通过 SPI 接口和 STM32MP157 连接在一起。ICM-20608 特性如下：

①、陀螺仪支持 X,Y 和 Z 三轴输出，内部集成 16 位 ADC，测量范围可设置：±250，±500，±1000 和±2000°/s。

②、加速度计支持 X,Y 和 Z 轴输出，内部集成 16 位 ADC，测量范围可设置：±2g，±4g，±4g，±8g 和±16g。

③、用户可编程中断。

④、内部包含 512 字节的 FIFO。

⑤、内部包含一个数字温度传感器。

⑥、耐 10000g 的冲击。

⑦、支持快速 I2C，速度可达 400KHz。

⑧、支持 SPI，速度可达 8MHz。

ICM-20608 的 3 轴方向如图 45.1.3.1 所示：

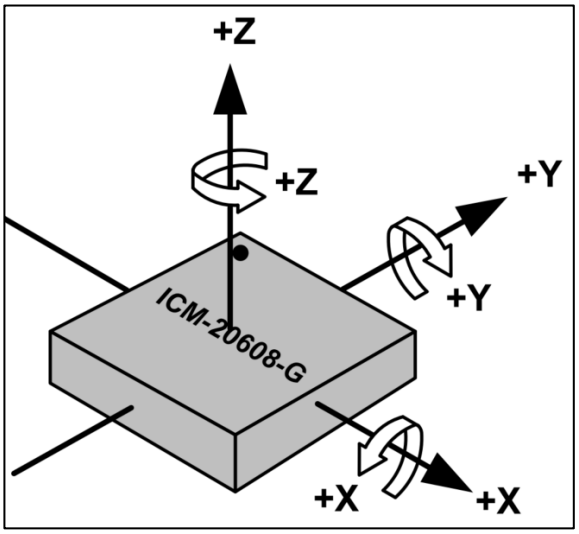


图 45.1.3.1 ICM-20608 检测轴方向和极性

ICM-20608 的结构框图如图 45.1.3.2 所示：

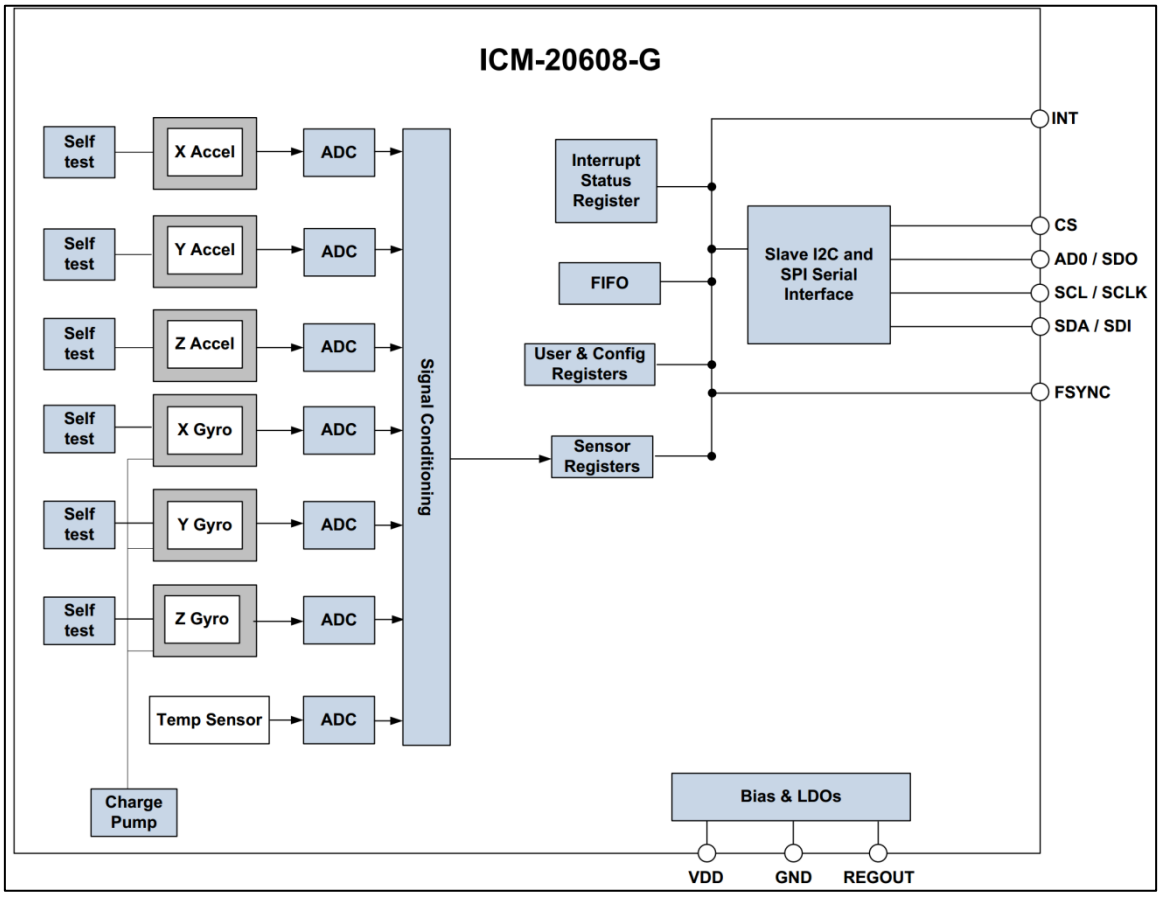


图 45.1.3.2 ICM-20608 框图

如果使用 IIC 接口的话，ICM-20608 的 AD0 引脚决定 I2C 设备从地址的最后一位，如果 AD0 为 0 的话 ICM-20608 从设备地址是 0X68，如果 AD0 为 1 的话 ICM-20608 从设备地址为 0X69。本章我们使用 SPI 接口，跟上一章使用 AP3216C 一样，ICM-20608 也是通过读写寄存器来配置和读取传感器数据，使用 SPI 接口读写寄存器需要 16 个时钟或者更多(如果读写操作包括多个字节的话)，第一个字节包含要读写的寄存器地址，寄存器地址最高位是读写标志位，如果是读的话寄存器地址最高位要为 1，如果是写的话寄存器地址最高位要为 0，剩下的 7 位才是实际的寄存器地址，寄存器地址后面跟着的就是读写的数据。表 45.1.3.1 列出了本章实验用到的一些寄存器和位，关于 ICM-20608 的详细寄存器和位的介绍请参考 ICM-20608 的寄存器手册：

寄存器地址	位	寄存器功能	描述
0X19	SMLPRT_DIV[7:0]	输出速率设置	设置输出速率，输出速率计算公式如下：SAMPLE_RATE=INTERNAL_SAMPLE_RATE/(1 + SMPLRT_DIV)
0X1A	DLPF_CFG[2:0]	芯片配置	设置陀螺仪低通滤波。可设置 0~7。
0X1B	FS_SEL[1:0]	陀螺仪量程设置	0：±250dps；1：±500dps；2：±1000dps 3：±2000dps
0X1C	ACC_FS_SEL[1:0]	加速度计量程设置	0：±2g；1：±4g；2：±8g；3：±16g
0X1D	A_DLPF_CFG[2:0]	加速度计低通	设置加速度计的低通滤波，可设置 0~7。

		滤波设置	
0X1E	GYRO_CYCLE[7]	陀螺仪低功耗使能	0：关闭陀螺仪的低功耗功能。 1：使能陀螺仪的低功耗功能。
0X23	TEMP_FIFO_EN[7]	FIFO 使能控制	1：使能温度传感器 FIFO。 0：关闭温度传感器 FIFO。
	XG_FIFO_EN[6]		1：使能陀螺仪 X 轴 FIFO。 0：关闭陀螺仪 X 轴 FIFO。
	YG_FIFO_EN[5]		1：使能陀螺仪 Y 轴 FIFO。 0：关闭陀螺仪 Y 轴 FIFO。
	ZG_FIFO_EN[4]		1：使能陀螺仪 Z 轴 FIFO。 0：关闭陀螺仪 Z 轴 FIFO。
	ACCEL_FIFO_EN[3]		1：使能加速度计 FIFO。 0：关闭加速度计 FIFO。
0X3B	ACCEL_XOUT_H[7:0]	数据寄存器	加速度 X 轴数据高 8 位
0X3C	ACCEL_XOUT_L[7:0]		加速度 X 轴数据低 8 位
0X3D	ACCEL_YOUT_H[7:0]		加速度 Y 轴数据高 8 位
0X3E	ACCEL_YOUT_L[7:0]		加速度 Y 轴数据低 8 位
0X3F	ACCEL_ZOUT_H[7:0]		加速度 Z 轴数据高 8 位
0X40	ACCEL_ZOUT_L[7:0]		加速度 Z 轴数据低 8 位
0X41	TEMP_OUT_H[7:0]		温度数据高 8 位
0X42	TEMP_OUT_L[7:0]		温度数据低 8 位
0X43	GYRO_XOUT_H[7:0]		加速度计 X 轴数据高 8 位
0X44	GYRO_XOUT_L[7:0]		加速度计 X 轴数据低 8 位
0X45	GYRO_YOUT_H[7:0]		加速度计 Y 轴数据高 8 位
0X46	GYRO_YOUT_L[7:0]		加速度计 Y 轴数据低 8 位
0X47	GYRO_ZOUT_H[7:0]		加速度计 Z 轴数据高 8 位
0X48	GYRO_ZOUT_L[7:0]		加速度计 Z 轴数据低 8 位
0X6B	DEVICE_RESET[7]	电源管理寄存器 1	1：复位 ICM-20608。
	SLEEP[6]		0：退出休眠模式；1，进入休眠模式
0X6C	STBY_XA[5]	电源管理寄存器 2	0：使能加速度计 X 轴。 1：关闭加速度计 X 轴。
	STBY_YA[4]		0：使能加速度计 Y 轴。 1：关闭加速度计 Y 轴。
	STBY_ZA[3]		0：使能加速度计 Z 轴。 1：关闭加速度计 Z 轴。
	STBY_XG[2]		0：使能陀螺仪 X 轴。 1：关闭陀螺仪 X 轴。
	STBY_YG[1]		0：使能陀螺仪 Y 轴。 1：关闭陀螺仪 Y 轴。
	STBY_ZG[0]		0：使能陀螺仪 Z 轴。 1：关闭陀螺仪 Z 轴。
0X75	WHOAMI[7:0]		ID 寄存器，ICM-20608G 的 ID 为 0XAF，

			ICM-20608D 的 ID 为 0XAE。

表 45.1.3.1 ICM-20608 寄存器表

ICM-20608 的介绍就到这里，关于 ICM-20608 的详细介绍请参考 ICM-20608 的数据手册和寄存器手册。

45.2 Linux 下 SPI 驱动框架

SPI 总线框架和 I2C 总线框架很类似，都采用了主机控制器驱动和设备驱动分离的思想；主机控制器也就是 SoC 的 SPI 控制器，例如 STM32MP1 的 SPI 控制器；而设备驱动对应的则是挂在 SPI 总线下的从机设备驱动程序。主机控制器针对具体的 SOC 平台，例如 STM32MP1，对于同一个 SOC 平台来说，SPI 控制器驱动程序是不用动的，不管外接的是什么 SPI 从机设备，对应的控制器驱动程序都一样，所以我们的重点就落在了种类繁多的 SPI 从机设备驱动开发了。SPI 控制器驱动程序一般是不需要驱动开发工程师自己编写，SOC 厂商会提供相应的主机驱动程序。

在 Linux 内核当中，与 I2C 总线框架一样，SPI 总线框架（也可以叫做 SPI 子系统）也可以分为三个部分：

SPI 核心层：SPI 核心层是 Linux 的 SPI 子系统的核心代码部分，提供了核心数据结构的定义、SPI 控制器驱动和设备驱动的注册、注销、管理等 API。其为硬件平台无关层，向下屏蔽了物理总线控制器的差异，定义了统一的访问策略和接口；其向上提供了统一的接口，以便 SPI 设备驱动通过总线控制器进行数据收发。在 Linux 系统中，SPI 核心层的代码位于 drivers/spi/spi.c。

SPI 控制器驱动层：每种处理器平台都有自己的 SPI 控制器驱动程序，它的职责是为系统中的 SPI 总线实现相应的读写方法。例如 STM32MP1 就有六个 SPI，那么就有六个 SPI 控制器，每个控制器都有一条特定的 SPI 总线的读写。SPI 子系统使用 struct spi_master 数据结构体来描述 SPI 控制器。在内核源码 drivers/spi 目录下有很多以 spi-xxxx.c 命名的源文件，如图 45.2.1：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31/drivers/spi$ ls
atmel-quadspi.c       spidev.o            spi-mpc52xx.c       spi-sh-hspi.c
built-in.a            spi-dln2.c          spi-mpc52xx-psc.c   spi-sh-msiof.c
internals.h           spi-dw.c            spi-mt65xx.c        spi-sh-sci.c
Kconfig               spi-dw.h            spi-mt7621.c        spi-sifive.c
Makefile              spi-dw-mid.c        spi-mxic.c          spi-sirf.c
modules.builtin       spi-dw-mmio.c       spi-mxs.c           spi-slave-mt27xx.c
spi-altera.c          spi-dw-pci.c        spi-npcm-fiu.c      spi-slave-system-control.c
spi-armada-3700.c     spi-efm32.c         spi-npcm-pspi.c     spi-slave-time.c
spi-at91-usart.c      spi-ep93xx.c        spi-nxp-fspi.c      spi-sprd-adi.c
```

图 45.2.1 SPI 控制器驱动源码

这些文件就是具体平台对应的 SPI 控制器驱动程序，使用 SPI 核心层提供的接口向 SPI 子系统注册 SPI 控制器。

SPI 设备驱动层：SPI 从设备对应的驱动程序，比如一些 SPI 接口的芯片器件对应的驱动程序。接下来我们详细的聊聊 SPI 子系统。

45.2.1 SPI 主机驱动

SPI 主机驱动就是 SoC 的 SPI 控制器驱动，类似 I2C 总线的适配器驱动。SPI 子系统使用 spi_master 结构体来描述 SPI 控制器，其实 spi_master 是一个宏，这个宏定义在 include/linux/spi/spi.h 文件中，如下所示：

```
#define spi_master              spi_controller
```

所以由此可以知道，spi_master 就是 spi_controller 结构体，该结构体定义在

include/linux/spi/spi.h 文件中，如下所示：

示例代码 45.2.1 spi_controller 结构体

```
424 struct spi_controller {
425     struct device   dev;        /* device 对象 */
426
427     struct list_head list;
......
435     s16         bus_num;        /* SPI 总线编号 */
......
440     u16         num_chipselect; /* 片选        */
441
442     /* some SPI controllers pose alignment requirements on DMAable
443      * buffers; let protocol drivers know about these requirements.
444      */
445     u16         dma_alignment;
446
447     /* spi_device.mode flags understood by this controller driver */
448     u32         mode_bits;      /* 模式位 */
......
455     /* limits on transfer speed */
456     u32         min_speed_hz;   /* SPI 控制器支持的最小传输速率 */
457     u32         max_speed_hz;   /* SPI 控制器支持的最大传输速率 */
458
459     /* other constraints relevant to this driver */
460     u16         flags;          /* 传输类型标志 */
......
468
469     /* flag indicating this is an SPI slave controller */
470     bool            slave;      /* 标志该控制器是否为 SPI 从设备存在 */
471
472     /*
473      * on some hardware transfer / message size may be constrained
474      * the limit may depend on device transfer settings
475      */
476     size_t  (*max_transfer_size)(struct spi_device *spi);
477     size_t  (*max_message_size)(struct spi_device *spi);
478
479     /* I/O mutex */
480     struct mutex        io_mutex;
481
482     /* lock and mutex for SPI bus locking */
483     spinlock_t      bus_lock_spinlock;
484     struct mutex        bus_lock_mutex;
```

```
485
486    /* flag indicating that the SPI bus is locked for exclusive use */
487     bool            bus_lock_flag;
......
495     int         (*setup)(struct spi_device *spi);
......
527     int         (*transfer)(struct spi_device *spi,
528                             struct spi_message *mesg);
......
567     int (*prepare_transfer_hardware)(struct spi_controller *ctlr);
568     int (*transfer_one_message)(struct spi_controller *ctlr,
569                                 struct spi_message *mesg);
......
607 };
```

第 495 行，SPI 控制器的 setup 函数，类似于初始化函数。

第 527 行，SPI 控制器的 transfer 函数，和 i2c_algorithm 中的 master_xfer 函数一样，控制器数据传输函数。

第 568 行，transfer_one_message 函数，也用于 SPI 数据发送，用于发送一个 spi_message，SPI 的数据会打包成 spi_message，然后以队列方式发送出去。

SPI 主机端最终会通过 transfer 函数与 SPI 设备进行通信，因此对于 SPI 主机控制器的驱动编写者而言 transfer 函数是需要实现的，因为不同的 SOC 其 SPI 控制器不同，寄存器都不一样。和 I2C 适配器驱动一样，SPI 主机驱动一般都是 SOC 厂商去编写的，所以我们作为 SOC 的使用者，这一部分的驱动就不用操心了，除非你是在 SOC 原厂工作，内容就是写 SPI 主机驱动。

SPI 主机驱动的核心就是申请 spi_master，然后初始化 spi_master，最后向 Linux 内核注册 spi_master。

1、spi_master 申请与释放

spi_alloc_master 函数用于申请 spi_master，函数原型如下：

```
struct spi_controller *spi_alloc_master(struct device    *host,
                                        unsigned int         size)
```

函数参数和返回值含义如下：

host：设备，一般是 platform_device 中的 dev 成员变量。

size：私有数据大小，可以通过 spi_master_get_devdata 函数获取到这些私有数据。

返回值：申请到的 spi_controller，也就是 spi_master。

spi_master 的释放通过 spi_master_put 函数来完成，当我们删除一个 SPI 主机驱动的时候就需要释放掉前面申请的 spi_master，spi_master_put 本质上是个宏：

```
#define spi_master_put(_ctlr)       spi_controller_put(_ctlr)
```

spi_master_put 函数最终通过调用 spi_controller_put 函数来完成 spi_master 释放，原型如下：

```
void spi_master_put(struct spi_controller *ctlr)
```

函数参数和返回值含义如下：

ctlr：要释放的 spi_master。

返回值：无。

2、spi_master 的注册与注销

当 spi_master 初始化完成以后就需要将其注册到 Linux 内核，spi_master 注册函数为 spi_register_master，函数原型如下：

```
int spi_register_master(struct spi_controller *ctlr)
```

函数参数和返回值含义如下：

ctlr：要注册的 spi_master。

返回值：0，成功；负值，失败。

如果要注销 spi_master 的话可以使用 spi_unregister_master 函数，此函数原型为：

```
void spi_unregister_master(struct spi_controller *ctlr)
```

函数参数和返回值含义如下：

ctlr：要注销的 spi_master。

返回值：无。

45.2.2 SPI 设备驱动

spi 设备驱动和 i2c 设备驱动也很类似，Linux 内核使用 spi_driver 结构体来表示 spi 设备驱动，我们在编写 SPI 设备驱动的时候需要实现 spi_driver。spi_driver 结构体定义在 include/linux/spi/spi.h 文件中，结构体内容如下：

示例代码 45.2.2.1 spi_driver 结构体

```
259 struct spi_driver {
260     const struct spi_device_id *id_table;
261     int         (*probe)(struct spi_device *spi);
262     int         (*remove)(struct spi_device *spi);
263     void        (*shutdown)(struct spi_device *spi);
264     struct device_driver    driver;
265 };
```

可以看出，spi_driver 和 i2c_driver、platform_driver 基本一样，当 SPI 设备和驱动匹配成功以后 probe 函数就会执行。

同样的，spi_driver 初始化完成以后需要向 Linux 内核注册，spi_driver 注册函数为 spi_register_driver，函数原型如下：

```
int spi_register_driver(struct spi_driver *sdrv)
```

函数参数和返回值含义如下：

sdrv：要注册的 spi_driver。

返回值：0，注册成功；赋值，注册失败。

注销 SPI 设备驱动以后也需要注销掉前面注册的 spi_driver，使用 spi_unregister_driver 函数完成 spi_driver 的注销，函数原型如下：

```
void spi_unregister_driver(struct spi_driver *sdrv)
```

函数参数和返回值含义如下：

sdrv：要注销的 spi_driver。

返回值：无。

spi_driver 注册示例程序如下：

示例代码 45.2.2.2 spi_driver 注册示例程序

```
1  /* probe 函数 */
```

```
2  static int xxx_probe(struct spi_device *spi)
3  {
4      /* 具体函数内容 */
5      return 0;
6  }
7
8  /* remove 函数 */
9  static int xxx_remove(struct spi_device *spi)
10 {
11     /* 具体函数内容 */
12     return 0;
13 }
14 /* 传统匹配方式 ID 列表 */
15 static const struct spi_device_id xxx_id[] = {
16     {"xxx", 0},
17     {}
18 };
19
20 /* 设备树匹配列表 */
21 static const struct of_device_id xxx_of_match[] = {
22     { .compatible = "xxx" },
23     { /* Sentinel */ }
24 };
25
26 /* SPI 驱动结构体 */
27 static struct spi_driver xxx_driver = {
28     .probe = xxx_probe,
29     .remove = xxx_remove,
30     .driver = {
31         .owner = THIS_MODULE,
32         .name = "xxx",
33         .of_match_table = xxx_of_match,
34         },
35     .id_table = xxx_id,
36 };
37
38 /* 驱动入口函数 */
39 static int __init xxx_init(void)
40 {
41     return spi_register_driver(&xxx_driver);
42 }
43
44 /* 驱动出口函数 */
```

```
45 static void __exit xxx_exit(void)
46 {
47     spi_unregister_driver(&xxx_driver);
48 }
49
50 module_init(xxx_init);
51 module_exit(xxx_exit);
```

第 1~36 行，spi_driver 结构体，需要 SPI 设备驱动人员编写，包括匹配表、probe 函数等。和 i2c_driver、platform_driver 一样，就不详细讲解了。

第 39~42 行，在驱动入口函数中调用 spi_register_driver 来注册 spi_driver。

第 45~48 行，在驱动出口函数中调用 spi_unregister_driver 来注销 spi_driver。

45.2.3 SPI 设备和驱动匹配过程

SPI 设备和驱动的匹配过程是由 SPI 总线来完成的，这点和 platform、I2C 等驱动一样，SPI 总线为 spi_bus_type，定义在 drivers/spi/spi.c 文件中，内容如下：

示例代码 45.2.3.1 spi_bus_type 结构体

```
377 struct bus_type spi_bus_type = {
378     .name           = "spi",
379     .dev_groups     = spi_dev_groups,
380     .match          = spi_match_device,
381     .uevent         = spi_uevent,
382 };
```

可以看出，SPI 设备和驱动的匹配函数为 spi_match_device，函数内容如下：

示例代码 45.2.3.2 spi_match_device 函数

```
342 static int spi_match_device(struct device *dev,
                                 struct device_driver *drv)
343 {
344     const struct spi_device *spi = to_spi_device(dev);
345     const struct spi_driver *sdrv = to_spi_driver(drv);
346
347     /* Check override first, and if set, only use the named driver */
348     if (spi->driver_override)
349         return strcmp(spi->driver_override, drv->name) == 0;
350
351     /* Attempt an OF style match */
352     if (of_driver_match_device(dev, drv))
353         return 1;
354
355     /* Then try ACPI */
356     if (acpi_driver_match_device(dev, drv))
357         return 1;
358
359     if (sdrv->id_table)
```

```
360         return !!spi_match_id(sdrv->id_table, spi);
361
362     return strcmp(spi->modalias, drv->name) == 0;
363 }
```

spi_match_device 函数和 i2c_match_device 函数的对于设备和驱动的匹配过程基本一样。

第 352 行，of_driver_match_device 函数用于完成设备树设备和驱动匹配。比较 SPI 设备节点的 compatible 属性和 of_device_id 中的 compatible 属性是否相等，如果相当的话就表示 SPI 设备和驱动匹配。

第 356 行，acpi_driver_match_device 函数用于 ACPI 形式的匹配。

第 360 行，spi_match_id 函数用于传统的、无设备树的 SPI 设备和驱动匹配过程。比较 SPI 设备名字和 spi_device_id 的 name 字段是否相等，相等的话就说明 SPI 设备和驱动匹配。

第 362 行，比较 spi_device 中 modalias 成员变量和 device_driver 中的 name 成员变量是否相等。

45.3 STM32MP1 SPI 主机驱动分析

和 I2C 的适配器驱动一样，SPI 主机驱动一般都由 SOC 厂商编写好了，打开 stm32mp151.dtsi 文件，找到如下所示内容：

示例代码 45.3.1 stm32mp151.dtsi 文件中的 spi1 节点内容

```
1  spi1: spi@44004000 {
2      #address-cells = <1>;
3      #size-cells = <0>;
4      compatible = "st,stm32h7-spi";
5      reg = <0x44004000 0x400>;
6      interrupts = <GIC_SPI 35 IRQ_TYPE_LEVEL_HIGH>;
7      clocks = <&rcc SPI1_K>;
8      resets = <&rcc SPI1_R>;
9      dmas = <&dmamux1 37 0x400 0x01>,
10          <&dmamux1 38 0x400 0x01>;
11     dma-names = "rx", "tx";
12     power-domains = <&pd_core>;
13     status = "disabled";
14 };
```

重点来看一下第 4 行的 compatible 属性值，compatible 属性为“st,stm32h7-spi”，在 Linux 内核源码中搜索这两个属性值即可找到 STM32MP1 对应的 SPI 主机驱动。STM32MP1 的 SPI 主机驱动文件为 drivers/spi/spi-stm32.c，在此文件中找到如下内容：

示例代码 45.3.2 stm32_spi_driver 结构体

```
1859  static const struct of_device_id stm32_spi_of_match[] = {
1860      {   .compatible = "st,stm32h7-spi",
              .data = (void *)&stm32h7_spi_cfg },
1861      {   .compatible = "st,stm32f4-spi",
              .data = (void *)&stm32f4_spi_cfg },
1862      {},
1863  };
```

```
......
2154    static struct platform_driver stm32_spi_driver = {
2155        .probe = stm32_spi_probe,
2156        .remove = stm32_spi_remove,
2157        .driver = {
2158            .name = DRIVER_NAME,
2159            .pm = &stm32_spi_pm_ops,
2160            .of_match_table = stm32_spi_of_match,
2161        },
2162    };
2163
2164    module_platform_driver(stm32_spi_driver);
```

第 1860 行，“st,stm32h7-spi”匹配项，因此可知 STM32MP1 主机驱动就是 spi-stm32.c 这个文件。

第 2154~2164 行，从这里可以知道，该主机驱动程序是基于 platform 总线框架编写，platform_driver 结构体变量为 stm32_spi_driver，当 platform 总线下设备和设备驱动匹配成功之后就会执行 stm32_spi_probe 函数，同样当驱动模块卸载的时候就会执行 stm32_spi_remove 函数。

接下来我们重点来看下 stm32_spi_probe 函数做了些什么，函数如下所示：

示例代码 45.3.3 stm32_spi_probe 函数

```
1866    static int stm32_spi_probe(struct platform_device *pdev)
1867    {
1868        struct spi_master *master;
1869        struct stm32_spi *spi;
1870        struct resource *res;
1871        struct reset_control *rst;
1872        int i, ret, num_cs, cs_gpio;
1873
1874        master = spi_alloc_master(&pdev->dev,
                          sizeof(struct stm32_spi));
1875        if (!master) {
1876            dev_err(&pdev->dev, "spi master allocation failed\n");
1877            return -ENOMEM;
1878        }
.....
1892        res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
1893        spi->base = devm_ioremap_resource(&pdev->dev, res);
1894        if (IS_ERR(spi->base)) {
1895            ret = PTR_ERR(spi->base);
1896            goto err_master_put;
1897        }
1898
1899        spi->phys_addr = (dma_addr_t)res->start;
```

```
1900
1901        spi->irq = platform_get_irq(pdev, 0);
1902        if (spi->irq <= 0) {
1903            ret = spi->irq;
1904            if (ret != -EPROBE_DEFER)
1905                dev_err(&pdev->dev, "failed to get irq: %d\n", ret);
1906            goto err_master_put;
1907        }
1908        ret = devm_request_threaded_irq(&pdev->dev, spi->irq,
1909                        spi->cfg->irq_handler_event,
1910                        spi->cfg->irq_handler_thread,
1911                        IRQF_ONESHOT, pdev->name, master);
......
1963        master->dev.of_node = pdev->dev.of_node;
1964        master->auto_runtime_pm = true;
1965        master->bus_num = pdev->id;
1966        master->mode_bits = SPI_CPHA | SPI_CPOL | SPI_CS_HIGH |
1967                            SPI_LSB_FIRST | SPI_3WIRE;
1968        master->bits_per_word_mask = spi->cfg->get_bpw_mask(spi);
1969        master->max_speed_hz =   spi->clk_rate /
                                    spi->cfg->baud_rate_div_min;
1970        master->min_speed_hz = spi->clk_rate /
                                  spi->cfg->baud_rate_div_max;
1971        master->setup = stm32_spi_setup;
1972        master->prepare_message = stm32_spi_prepare_msg;
1973        master->transfer_one = stm32_spi_transfer_one;
1974        master->unprepare_message = stm32_spi_unprepare_msg;
......
2026        ret = spi_register_master(master);
......

2050    }
```

第 1874 行，通过调用 spi_alloc_master 函数为 master 指针申请内存，也就是实例化 master。

第 1901~1911 行，获取中断号和注册中断函数。

第 1963~1974 行，对 master 变量进行初始化和赋值，从结果可以看到，master 结构体中并没有设置 transfer 和 transfer_one_message 这两个用于 SPI 数据传输的函数，而是使用了 transfer_one 作为 SPI 数据传输的函数，对应的函数为 stm32_spi_transfer_one，也就是该函数是 STM32MP1 SPI 数据传输的函数。

第 2026 行，调用 spi_register_master 函数向 SPI 子系统注册一个 master。

这里简单第看看 stm32_spi_transfer_one 函数的内容，如下所示(有省略)：

示例代码 45.3.4 stm32_spi_transferZ_one 函数

```
1683    static int stm32_spi_transfer_one(struct spi_master *master,
1684                    struct spi_device *spi_dev,
```

```
1685                    struct spi_transfer *transfer)
1686    {
1687        struct stm32_spi *spi = spi_master_get_devdata(master);
1688        u32 xfer_time, midi_delay_ns;
1689        unsigned long timeout;
1690        int ret;
1691
1692        spi->tx_buf = transfer->tx_buf;
1693        spi->rx_buf = transfer->rx_buf;
1694        spi->tx_len = spi->tx_buf ? transfer->len : 0;
1695        spi->rx_len = spi->rx_buf ? transfer->len : 0;
1696
1697        spi->cur_usedma = (master->can_dma &&
1698                master->can_dma(master, spi_dev, transfer));
1699
1700        ret = stm32_spi_transfer_one_setup(spi, spi_dev, transfer);
1701        if (ret) {
1702            dev_err(spi->dev, "SPI transfer setup failed\n");
1703            return ret;
1704        }
1705
1706        reinit_completion(&spi->xfer_completion);
1707        spi->xfer_status = 0;
1708
1709        if (spi->cur_usedma)
1710            ret = stm32_spi_transfer_one_dma(spi, transfer);
1711        else
1712            ret = spi->cfg->transfer_one_irq(spi);
1713
1714        if (ret)
1715            return ret;
......
1739    }
```

第 1709~1715 行，如果启动了 DMA 那么就使用 stm32_spi_transfer_one_dma 来进行传输数据，没有用 DMA 的话就调用 transfer_one_irq 函数。这两个函数也就是控制寄存器来进行收发数据。

关于 STM32MP1 SPI 主机程序的分析就到这里了。

45.4 SPI 设备驱动编写流程

45.4.1 SPI 设备信息描述

1、IO 的 pinctrl 子节点创建与修改

首先肯定是根据所使用的 IO 来创建或者修改 pinctrl 子节点，这个没有什么好说的。唯独要注意的是检查相应的 IO 有没有被其它的设备所使用，如果有多个 pinctrl 配置相同的 IO 是没有关系的，只要保证没有被设备调用就行。

2、SPI 设备节点的创建与修改

采用设备树方式的情况下，SPI 从机设备信息描述就通过创建相应的设备子节点来完成，我们可以打开 stm32mp157d-atk.dts 这个设备树文件，然后在里边创建一个 SPI 从机设备节点，描述该设备的相关信息，我们后面再创建。

45.4.2 SPI 从机设备数据收发处理流程

SPI 设备驱动的核心是 spi_driver，这个我们已经在 45.2.2 小节讲过了。当我们向 Linux 内核注册成功 spi_driver 以后就可以使用 SPI 核心层提供的 API 函数来对设备进行读写操作了。首先是 spi_transfer 结构体，此结构体用于描述 SPI 传输信息，结构体内容如下：

示例代码 45.4.2.1 spi_transfer 结构体

```
811 struct spi_transfer {
......
817     const void  *tx_buf;
818     void        *rx_buf;
819     unsigned    len;
820
821     dma_addr_t  tx_dma;
822     dma_addr_t  rx_dma;
823     struct sg_table tx_sg;
824     struct sg_table rx_sg;
825
826     unsigned    cs_change:1;
827     unsigned    tx_nbits:3;
828     unsigned    rx_nbits:3;
829 #define SPI_NBITS_SINGLE    0x01 /* 1bit transfer */
830 #define SPI_NBITS_DUAL      0x02 /* 2bits transfer */
831 #define SPI_NBITS_QUAD      0x04 /* 4bits transfer */
832     u8      bits_per_word;
833     u8      word_delay_usecs;
834     u16     delay_usecs;
835     u16     cs_change_delay;
836     u8      cs_change_delay_unit;
837 #define SPI_DELAY_UNIT_USECS    0
838 #define SPI_DELAY_UNIT_NSECS    1
839 #define SPI_DELAY_UNIT_SCK  2
840     u32     speed_hz;
841     u16     word_delay;
842
843     u32     effective_speed_hz;
```

```
844
845     struct list_head transfer_list;
846 };
```

第 817 行，tx_buf 保存着要发送的数据。

第 818 行，rx_buf 用于保存接收到的数据。

第 819 行，len 是要进行传输的数据长度，SPI 是全双工通信，因此在一次通信中发送和接收的字节数都是一样的，所以 spi_transfer 中也就没有发送长度和接收长度之分。

spi_transfer 需要组织成 spi_message，spi_message 也是一个结构体，内容如下：

示例代码 45.4.2.2 spi_message 结构体

```
878 struct spi_message {
879     struct list_head    transfers;
880
881     struct spi_device   *spi;
882
883     unsigned        is_dma_mapped:1;
......
897     void            (*complete)(void *context);
898     void            *context;
899     unsigned        frame_length;
900     unsigned        actual_length;
901     int         status;
......
907     struct list_head    queue;
908     void            *state;
909
910     /* list of spi_res reources when the spi message is processed */
911     struct list_head        resources;
912 };
```

在使用 spi_message 之前需要对其进行初始化，spi_message 初始化函数为 spi_message_init，函数原型如下：

```
void spi_message_init(struct spi_message *m)
```

函数参数和返回值含义如下：

m：要初始化的 spi_message。

返回值：无。

spi_message 初始化完成以后需要将 spi_transfer 添加到 spi_message 队列中，这里我们要用到 spi_message_add_tail 函数，此函数原型如下：

```
void spi_message_add_tail(struct spi_transfer *t, struct spi_message *m)
```

函数参数和返回值含义如下：

t：要添加到队列中的 spi_transfer。

m：spi_transfer 要加入的 spi_message。

返回值：无。

spi_message 准备好以后就可以进行数据传输了，数据传输分为同步传输和异步传输，同步传输会阻塞的等待 SPI 数据传输完成，同步传输函数为 spi_sync，函数原型如下：

int spi_sync(struct spi_device *spi, struct spi_message *message)

函数参数和返回值含义如下：

spi：要进行数据传输的 spi_device。

message：要传输的 spi_message。

返回值：无。

异步传输不会阻塞的等到 SPI 数据传输完成，异步传输需要设置 spi_message 中的 complete 成员变量，complete 是一个回调函数，当 SPI 异步传输完成以后此函数就会被调用。SPI 异步传输函数为 spi_async，函数原型如下：

int spi_async(struct spi_device *spi, struct spi_message *message)

函数参数和返回值含义如下：

spi：要进行数据传输的 spi_device。

message：要传输的 spi_message。

返回值：无。

在本章实验中，我们采用同步传输方式来完成 SPI 数据的传输工作，也就是 spi_sync 函数。

综上所述，SPI 数据传输步骤如下：

①、申请并初始化 spi_transfer，设置 spi_transfer 的 tx_buf 成员变量，tx_buf 为要发送的数据。然后设置 rx_buf 成员变量，rx_buf 保存着接收到的数据。最后设置 len 成员变量，也就是要进行数据通信的长度。

②、使用 spi_message_init 函数初始化 spi_message。

③、使用 spi_message_add_tail 函数将前面设置好的 spi_transfer 添加到 spi_message 队列中。

④、使用 spi_sync 函数完成 SPI 数据同步传输。

通过 SPI 进行 n 个字节的数据发送和接收的示例代码如下所示：

示例代码 45.4.2.3 SPI 数据读写操作

```
/* SPI 多字节发送 */
static int spi_send(struct spi_device *spi, u8 *buf, int len)
{
    int ret;
    struct spi_message m;

    struct spi_transfer t = {
        .tx_buf = buf,
        .len = len,
    };

    spi_message_init(&m);          /* 初始化 spi_message */
    spi_message_add_tail(t, &m);/* 将 spi_transfer 添加到 spi_message 队列 */
    ret = spi_sync(spi, &m);       /* 同步传输 */
    return ret;
}

/* SPI 多字节接收 */
static int spi_receive(struct spi_device *spi, u8 *buf, int len)
{
```

```
    int ret;
    struct spi_message m;

    struct spi_transfer t = {
        .rx_buf = buf,
        .len = len,
    };

    spi_message_init(&m);          /* 初始化 spi_message */
    spi_message_add_tail(t, &m);/* 将 spi_transfer 添加到 spi_message 队列 */
    ret = spi_sync(spi, &m);       /* 同步传输 */
    return ret;
};
```

45.5 硬件原理图分析

STM32MP1 开发板上 ICM20608 原理如图 45.5.1 所示：

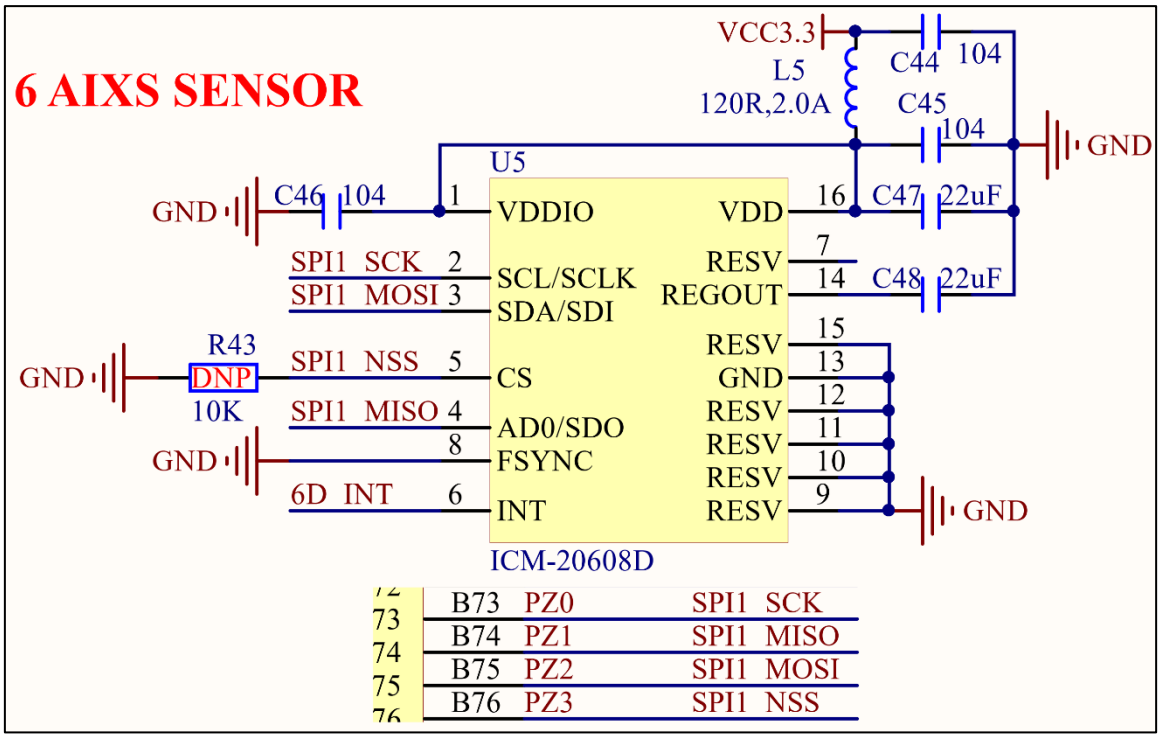


图 45.5.1 ICM-20608 原理图

图 45.5.1 这是 ICM-20608 的硬件原理图。正点原子 STM32MP1 开发板的 PZ0~3 分别连接到 ICM-20608 的 SCK、SDA、AD0 和 CS。其中 6D_INT 为 ICM20608 的中断引脚，连接到 PA14 引脚上，在本章实验上没有用到 ICM20608 的中断引脚。

45.6 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→22_spi。

45.6.1 修改设备树

1、添加或者查找 ICM20608 所使用的 IO 的 pinmux 配置

首先在 stm32mp15-pinctrl.dtsi 文件中添加 IO 配置信息，ICM20608 连接到了 STM32MP157 的 SPI1 接口，因此先在 stm32mp15-pinctrl.dtsi 里面搜索一下，看看有没有 SPI1 接口引脚配置(在本教程中，默认是有的)。如果没有的话就自行添加，有的话就检查一下 SPI1 接口的引脚配置是否和自己所使用的硬件一致，不一致的话就要修改。修改后的引脚信息如下所示：

示例代码 45.6.1.1 spi1 的 pinmux 配置

```
1  spi1_pins_a: spi1-0 {
2   pins1 {
3      pinmux =   <STM32_PINMUX('Z', 0, AF5)>,    /* SPI1_SCK   */
4                 <STM32_PINMUX('Z', 2, AF5)>;    /* SPI1_MOSI  */
5      bias-disable;
6      drive-push-pull;
7      slew-rate = <1>;
8   };
9
10  pins2 {
11     pinmux = <STM32_PINMUX('Z', 1, AF5)>;      /* SPI1_MISO  */
12     bias-disable;
13  };
14
15  pins3 {
16     pinmux = <STM32_PINMUX('Z', 3, GPIO)>;     /* SPI1_NSS   */
17        drive-push-pull;
18        bias-pull-up;
19     output-high;
20        slew-rate = <0>;
21  };
22 };
23
24 spi1_sleep_pins_a: spi1-sleep-0 {
25  pins {
26     pinmux = <STM32_PINMUX('Z', 0, ANALOG)>,   /* SPI1_SCK   */
27         <STM32_PINMUX('Z', 1, ANALOG)>,        /* SPI1_MISO  */
28         <STM32_PINMUX('Z', 2, ANALOG)>,        /* SPI1_MOSI  */
29         <STM32_PINMUX('Z', 3, ANALOG)>;        /* SPI1_NSS   */
30  };
31 };
```

示例代码 45.6.3.1 里 15~21 行是设置 ICM20608 的片选信号，直接复用为 GPIO，也就是使用软件片选。

2、在 SPI1 节点下添加 pinmux 并追加 icm20608 子节点

在 stm32mp157d-atk.dts 文件，追加 SPI1 节点，追加如下所示内容：

示例代码 45.6.1.2 追加内容的 spi1 节点

```
1  &spi1 {
2      pinctrl-names = "default", "sleep";
3      pinctrl-0 = <&spi1_pins_a>;
4      pinctrl-1 = <&spi1_sleep_pins_a>;
5      cs-gpios = <&gpioz 3 GPIO_ACTIVE_LOW>;
6      status = "okay";
7
8      spidev: icm20608@0 {
9          compatible = "alientek,icm20608";
10         reg = <0>; /* CS #0 */
11         spi-max-frequency = <8000000>;
12     };
13 };
```

第 2~4 行，设置 IO 要使用的 pinmux 配置。

第 5 行，“cs-gpios”属性是用来设置 SPI 的片选引脚。SPI 主机驱动就会根据此属性去控制设备的片选引脚，本实验我们使用 PZ3 作为片选引脚。关于 cs-gpios 属性的详细描述可以参考绑定文档：Documentation/devicetree/bindings/spi/spi-controller.yaml。如果一个 SPI 接口下连接了多个 SPI 芯片，那么 cs-gpios 属性里面就要添加所有 SPI 芯片的片选信号，比如：

```
cs-gpios = <&gpio1 0 0>, <&gpio1 1 0>, <&gpio1 2 0>, <&gpio1 3 0>;
```

上述描述说明此时 SPI 节点下有 4 个 SPI 芯片，第一个 SPI 芯片的片选引脚为 gpio1_0，一以此类推。

第 8~12 行，icm20608 设备子节点，从第 5 行的 cs-gpios 节点可以看出，此时 SPI 接口下只有一个 ICM20608，而且 ICM20608 的片选索引为 0，因此@后面为 0。注意，@后面的数字就是对应 SPI 芯片片选信号在 cs-gpios 中的索引值。第 9 行设置节点属性兼容值为“alientek,icm20608”，第 10 行 reg 属性表示 icm20608 所使用的片选，和第 8 行@后面的数字含义相同，这里也设置为 0，也就是 cs-gpios 属性中的第一个片选信号。第 11 行设置 SPI 最大时钟频率为 8MHz，这是 ICM20608 的 SPI 接口所能支持的最大的时钟频率。

45.6.2 编写 ICM20608 驱动

新建名为“22_spi”的文件夹，然后在 22_spi 文件夹里面创建 vscode 工程，工作区命名为“spi”。工程创建好以后新建 icm20608.c 和 icm20608reg.h 这两个文件，icm20608.c 为 ICM20608 的驱动代码，icm20608reg.h 是 ICM20608 寄存器头文件。先在 icm20608reg.h 中定义好 ICM20608 的寄存器，输入如下内容(有省略，完整的内容请参考例程)：

示例代码 45.6.2.1 icm20608reg.h 文件内容

```
1  #ifndef ICM20608_H
2  #define ICM20608_H
3  /***************************************************************
4  Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
5  文件名    : icm20608reg.h
6  作者      : 左忠凯
7  版本      : V1.0
```

```
8  描述      : ICM20608 寄存器地址描述头文件
9  其他      : 无
10 论坛      : www.openedv.com
11 日志      : 初版 V1.0 2019/9/2 左忠凯创建
12 ***************************************************************/
13 #define ICM20608G_ID        0XAF    /* ID 值 */
14 #define ICM20608D_ID        0XAE    /* ID 值 */
15
16 /* ICM20608 寄存器
17  *复位后所有寄存器地址都为 0，除了
18  *Register 107(0X6B) Power Management 1    = 0x40
19  *Register 117(0X75) WHO_AM_I              = 0xAF 或 0xAE
20  */
21 /* 陀螺仪和加速度自测(出产时设置，用于与用户的自检输出值比较） */
22 #define  ICM20_SELF_TEST_X_GYRO     0x00
23 #define  ICM20_SELF_TEST_Y_GYRO     0x01
24 #define  ICM20_SELF_TEST_Z_GYRO     0x02
25 #define  ICM20_SELF_TEST_X_ACCEL    0x0D
26 #define  ICM20_SELF_TEST_Y_ACCEL    0x0E
27 #define  ICM20_SELF_TEST_Z_ACCEL    0x0F
......
88 #endif
```

接下来继续编写 icm20608.c 文件，因为 icm20608.c 文件内容比较长，因此这里就将其分开来讲解。

1、icm20608 设备结构体创建

首先创建一个 icm20608 设备结构体，如下所示：

示例代码 45.6.2.2 icm20608 设备结构体创建

```
1  #include <linux/spi/spi.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
......
14
15 #define ICM20608_CNT    1
16 #define ICM20608_NAME   "icm20608"
17
18 struct icm20608_dev {
19     struct spi_device *spi;      /* spi 设备            */
20     dev_t devid;                 /* 设备号              */
21     struct cdev cdev;            /* cdev                */
22     struct class *class;         /* 类                  */
23     struct device *device;       /* 设备                */
24     struct device_node  *nd;     /* 设备节点            */
25     signed int gyro_x_adc;       /* 陀螺仪 x 轴原始值    */
```

```
26      signed int gyro_y_adc;        /* 陀螺仪 Y 轴原始值      */
27      signed int gyro_z_adc;        /* 陀螺仪 Z 轴原始值      */
28      signed int accel_x_adc;       /* 加速度计 X 轴原始值    */
29      signed int accel_y_adc;       /* 加速度计 Y 轴原始值    */
30      signed int accel_z_adc;       /* 加速度计 Z 轴原始值    */
31      signed int temp_adc;          /* 温度原始值             */
32  };
```

icm20608 的设备结构体 icm20608_dev 没什么好讲的，重点看一下第 19 行的 spi，对于 SPI 设备驱动来讲最核心的就是 spi_device。probe 函数会向驱动提供当前 SPI 设备对应的 spi_device，因此把这个结构体赋值到 spi 成员里，我们可以在字符串设备里操作 spi 成员就可以操作对应的 SPI 设备。

2、icm20608 的 spi_driver 注册与注销

对于 SPI 设备驱动，首先就是要初始化并向系统注册 spi_driver，icm20608 的 spi_driver 初始化、注册与注销代码如下：

示例代码 45.6.2.3 icm20608 的 spi_driver 初始化、注册与注销

```
1   /* 传统匹配方式 ID 列表 */
2   static const struct spi_device_id icm20608_id[] = {
3       {"alientek,icm20608", 0},
4       {}
5   };
6
7   /* 设备树匹配列表 */
8   static const struct of_device_id icm20608_of_match[] = {
9       { .compatible = "alientek,icm20608" },
10      { /* Sentinel */ }
11  };
12
13  /* SPI 驱动结构体 */
14  static struct spi_driver icm20608_driver = {
15      .probe = icm20608_probe,
16      .remove = icm20608_remove,
17      .driver = {
18              .owner = THIS_MODULE,
19              .name = "icm20608",
20              .of_match_table = icm20608_of_match,
21             },
22      .id_table = icm20608_id,
23  };
24
25  /*
26   * @description   : 驱动入口函数
27   * @param         : 无
28   * @return        : 无
```

```
29   */
30  static int __init icm20608_init(void)
31  {
32      return spi_register_driver(&icm20608_driver);
33  }
34
35  /*
36   * @description   : 驱动出口函数
37   * @param         : 无
38   * @return        : 无
39   */
40  static void __exit icm20608_exit(void)
41  {
42      spi_unregister_driver(&icm20608_driver);
43  }
44
45  module_init(icm20608_init);
46  module_exit(icm20608_exit);
47  MODULE_LICENSE("GPL");
48  MODULE_AUTHOR("ALIENTEK");
49  MODULE_INFO(intree, "Y");
```

第 2~5 行，传统的设备和驱动匹配表。

第 8~11 行，设备树的设备与驱动匹配表，这里只有一个匹配项：“alientek,icm20608”。

第 14~23 行，icm20608 的 spi_driver 结构体变量，当 icm20608 设备和此驱动匹配成功以后第 15 行的 icm20608_probe 函数就会执行。同样的，当注销此驱动的时候 icm20608_remove 函数会执行。

第 30~33 行，icm20608_init 函数为 icm20608 的驱动入口函数，在此函数中使用 spi_register_driver 向 Linux 系统注册上面定义的 icm20608_driver。

第 40~43 行，icm20608_exit 函数为 icm20608 的驱动出口函数，在此函数中使用 spi_unregister_driver 注销掉前面注册的 icm20608_driver。

3、probe&remove 函数

icm20608_driver 中的 probe 和 remove 函数内容如下所示：

示例代码 45.6.3.4 probe 和 remove 函数

```
1  /*
2   * @description   : spi 驱动的 probe 函数，当驱动与设备匹配以后此函数
3   *                  就会执行
4   * @param - spi   : spi 设备
5   */
6  static int icm20608_probe(struct spi_device *spi)
7  {
8      int ret;
9      struct icm20608_dev *icm20608dev;
10
```

```
11      /* 分配 icm20608dev 对象的空间 */
12      icm20608dev = devm_kzalloc(&spi->dev, sizeof(*icm20608dev),
                                   GFP_KERNEL);
13      if(!icm20608dev)
14          return -ENOMEM;
15
16      /* 注册字符设备驱动 */
17      /* 1、创建设备号 */
18      ret = alloc_chrdev_region(&icm20608dev->devid, 0, ICM20608_CNT,
                                  ICM20608_NAME);
19      if(ret < 0) {
20          pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                ICM20608_NAME, ret);
21          return 0;
22      }
23
24      /* 2、初始化 cdev */
25      icm20608dev->cdev.owner = THIS_MODULE;
26      cdev_init(&icm20608dev->cdev, &icm20608_ops);
27
28      /* 3、添加一个 cdev */
29      ret = cdev_add(&icm20608dev->cdev, icm20608dev->devid,
                       ICM20608_CNT);
30      if(ret < 0) {
31          goto del_unregister;
32      }
33
34      /* 4、创建类 */
35      icm20608dev->class = class_create(THIS_MODULE, ICM20608_NAME);
36      if (IS_ERR(icm20608dev->class)) {
37          goto del_cdev;
38      }
39
40      /* 5、创建设备 */
41      icm20608dev->device = device_create(icm20608dev->class, NULL,
                          icm20608dev->devid, NULL, ICM20608_NAME);
42      if (IS_ERR(icm20608dev->device)) {
43          goto destroy_class;
44      }
45      icm20608dev->spi = spi;
46
47      /*初始化 spi_device */
48      spi->mode = SPI_MODE_0; /*MODE0, CPOL=0, CPHA=0*/
```

```
49      spi_setup(spi);
50
51      /* 初始化 ICM20608 内部寄存器 */
52      icm20608_reginit(icm20608dev);
53      /* 保存 icm20608dev 结构体 */
54      spi_set_drvdata(spi, icm20608dev);
55
56      return 0;
57  destroy_class:
58      device_destroy(icm20608dev->class, icm20608dev->devid);
59  del_cdev:
60      cdev_del(&icm20608dev->cdev);
61  del_unregister:
62      unregister_chrdev_region(icm20608dev->devid, ICM20608_CNT);
63      return -EIO;
64  }
65
66  /*
67   * @description     : spi 驱动的 remove 函数，移除 spi 驱动的时候此函数会执行
68   * @param - spi     : spi 设备
69   * @return          : 0，成功;其他负值,失败
70   */
71  static int icm20608_remove(struct spi_device *spi)
72  {
73      struct icm20608_dev *icm20608dev = spi_get_drvdata(spi);
74      /* 注销字符设备驱动 */
75      /* 1、删除 cdev */
76      cdev_del(&icm20608dev->cdev);
77      /* 2、注销设备号 */
78      unregister_chrdev_region(icm20608dev->devid, ICM20608_CNT);
79      /* 3、注销设备 */
80      device_destroy(icm20608dev->class, icm20608dev->devid);
81      /* 4、注销类 */
82      class_destroy(icm20608dev->class);
83      return 0;
84  }
```

第 6~64 行，probe 函数，当设备与驱动匹配成功以后此函数就会执行。第 12 行就是给自定义的结构体申请空间。第 25~44 行都是标准的注册字符设备驱动。注意我们这边驱动没有去读取“cs-gpios”属性，那是因为 SPI 的核心会自动控制的，我们不用关心它。

第 45 行，将 probe 函数的 spi_device 参数赋值给我们自定义的 spi 成员变量，也就是保存 spi_device 结构体对象。

第 48 行，设置 SPI 为模式 0，也就是 CPOL=0，CPHA=0。

第 49 行，设置好 spi_device 以后需要使用 spi_setup 配置一下。

第 52 行，调用 icm20608_reginit 函数初始化 ICM20608，主要是初始化 ICM20608 指定寄存器。

第 54 行，icm20608dev 变量是在 probe 函数里申请的内存空间，在 remove 函数需要释放掉。这里使用 spi_set_drvdata 函数将 icm20608dev 地址保存起来，后面再 remove 函数里面可以通过 spi_get_drvdata 函数获取到 icm20608dev 地址。

第 71~84 行，icm20608_remove 函数，注销驱动的时候此函数就会执行。我们可以使用 spi_get_drvdata 函数来获取 icm20608dev 的地址。

4、icm20608 寄存器读写与初始化

SPI 驱动最终是通过读写 icm20608 的寄存器来实现的，因此需要编写相应的寄存器读写函数，并且使用这些读写函数来完成对 icm20608 的初始化。icm20608 的寄存器读写以及初始化代码如下：

示例代码 45.6.2.5 icm20608 寄存器读写以及初始化

```
1  /*
2   * @description   : 从 icm20608 读取多个寄存器数据
3   * @param - dev   : icm20608 设备
4   * @param - reg   : 要读取的寄存器首地址
5   * @param - val   : 读取到的数据
6   * @param - len   : 要读取的数据长度
7   * @return        : 操作结果
8   */
9  static int icm20608_read_regs(struct icm20608_dev *dev, u8 reg,
                                 void *buf, int len)
10 {
11
12     int ret = -1;
13     unsigned char txdata[1];
14     unsigned char * rxdata;
15     struct spi_message m;
16     struct spi_transfer *t;
17     struct spi_device *spi = (struct spi_device *)dev->spi;
18
19     t = kzalloc(sizeof(struct spi_transfer), GFP_KERNEL);/* 申请内存*/
20     if(!t) {
21         return -ENOMEM;
22     }
23     rxdata = kzalloc(sizeof(char) * len, GFP_KERNEL);   /* 申请内存 */
24     if(!rxdata) {
25         goto out1;
26     }
27     /* 一共发送 len+1 个字节的数据，第一个字节为
28     寄存器首地址，一共要读取 len 个字节长度的数据，*/
29     txdata[0] = reg | 0x80;  /* 写数据的时候首寄存器地址 bit8 要置 1    */
30     t->tx_buf = txdata;      /* 要发送的数据                            */
```

```
31      t->rx_buf = rxdata;         /* 要读取的数据                          */
32      t->len = len+1;             /* t->len=发送的长度+读取的长度          */
33      spi_message_init(&m);       /* 初始化 spi_message                    */
34      spi_message_add_tail(t, &m);/* 将 spi_transfer 添加到 spi_message*/
35      ret = spi_sync(spi, &m); /* 同步发送                                 */
36      if(ret) {
37          goto out2;
38      }
39
40      memcpy(buf , rxdata+1, len);    /* 只需要读取的数据  */
41
42  out2:
43      kfree(rxdata);                  /* 释放内存          */
44  out1:
45      kfree(t);                       /* 释放内存          */
46
47      return ret;
48  }
49
50  /*
51   * @description : 向 icm20608 多个寄存器写入数据
52   * @param - dev:  icm20608 设备
53   * @param - reg:  要写入的寄存器首地址
54   * @param - val:  要写入的数据缓冲区
55   * @param - len:  要写入的数据长度
56   * @return    :   操作结果
57   */
58  static s32 icm20608_write_regs(struct icm20608_dev *dev, u8 reg,
                                   u8 *buf, u8 len)
59  {
60      int ret = -1;
61      unsigned char *txdata;
62      struct spi_message m;
63      struct spi_transfer *t;
64      struct spi_device *spi = (struct spi_device *)dev->spi;
65
66      t = kzalloc(sizeof(struct spi_transfer), GFP_KERNEL);/* 申请内存*/
67      if(!t) {
68          return -ENOMEM;
69      }
70
71      txdata = kzalloc(sizeof(char)+len, GFP_KERNEL);
72      if(!txdata) {
```

```
73          goto out1;
74      }
75
76      /* 一共发送 len+1 个字节的数据，第一个字节为
77      寄存器首地址，len 为要写入的寄存器的集合，*/
78      *txdata = reg & ~0x80;          /* 写数据的时候首寄存器地址 bit8 要清零 */
79      memcpy(txdata+1, buf, len); /* 把 len 个数据拷贝到 txdata 里 */
80      t->tx_buf = txdata;             /* 要发送的数据                  */
81      t->len = len+1;                 /* t->len=发送的长度+读取的长度*/
82      spi_message_init(&m);           /* 初始化 spi_message            */
83      spi_message_add_tail(t, &m);/*添加到 spi_message 队列       */
84      ret = spi_sync(spi, &m);        /* 同步发送                      */
85      if(ret) {
86          goto out2;
87      }
88
89  out2:
90      kfree(txdata);                  /* 释放内存 */
91  out1:
92      kfree(t);                       /* 释放内存 */
93      return ret;
94
95  }
96
97  /*
98   * @description   : 读取 icm20608 指定寄存器值，读取一个寄存器
99   * @param - dev   : icm20608 设备
100  * @param - reg   : 要读取的寄存器
101  * @return        : 读取到的寄存器值
102  */
103 static unsigned char icm20608_read_onereg(struct icm20608_dev *dev,
                                              u8 reg)
104 {
105     u8 data = 0;
106     icm20608_read_regs(dev, reg, &data, 1);
107     return data;
108 }
109
110 /*
111  * @description   : 向 icm20608 指定寄存器写入指定的值，写一个寄存器
112  * @param - dev   : icm20608 设备
113  * @param - reg   : 要写的寄存器
114  * @param - data  : 要写入的值
```

```
115  * @return         : 无
116  */
117
118 static void icm20608_write_onereg(struct icm20608_dev *dev, u8 reg,
                                      u8 value)
119 {
120     u8 buf = value;
121     icm20608_write_regs(dev, reg, &buf, 1);
122 }
123
124 /*
125  * @description   : 读取 ICM20608 的数据，读取原始数据，包括三轴陀螺仪、
126  *                : 三轴加速度计和内部温度。
127  * @param - dev   : ICM20608 设备
128  * @return        : 无。
129  */
130 void icm20608_readdata(struct icm20608_dev *dev)
131 {
132     unsigned char data[14];
133     icm20608_read_regs(dev, ICM20_ACCEL_XOUT_H, data, 14);
134
135     dev->accel_x_adc = (signed short)((data[0] << 8) | data[1]);
136     dev->accel_y_adc = (signed short)((data[2] << 8) | data[3]);
137     dev->accel_z_adc = (signed short)((data[4] << 8) | data[5]);
138     dev->temp_adc    = (signed short)((data[6] << 8) | data[7]);
139     dev->gyro_x_adc  = (signed short)((data[8] << 8) | data[9]);
140     dev->gyro_y_adc  = (signed short)((data[10] << 8) | data[11]);
141     dev->gyro_z_adc  = (signed short)((data[12] << 8) | data[13]);
142 }
143
144 /*
145  * ICM20608 内部寄存器初始化函数
146  * @param - spi   : 要操作的设备
147  * @return        : 无
148  */
149 void icm20608_reginit(struct icm20608_dev *dev)
150 {
151     u8 value = 0;
152
153     icm20608_write_onereg(dev, ICM20_PWR_MGMT_1, 0x80);
154     mdelay(50);
155     icm20608_write_onereg(dev, ICM20_PWR_MGMT_1, 0x01);
156     mdelay(50);
```

```
157
158     value = icm20608_read_onereg(dev, ICM20_WHO_AM_I);
159     printk("ICM20608 ID = %#X\r\n", value);
160
161     icm20608_write_onereg(dev, ICM20_SMPLRT_DIV, 0x00);
162     icm20608_write_onereg(dev, ICM20_GYRO_CONFIG, 0x18);
163     icm20608_write_onereg(dev, ICM20_ACCEL_CONFIG, 0x18);
164     icm20608_write_onereg(dev, ICM20_CONFIG, 0x04);
165     icm20608_write_onereg(dev, ICM20_ACCEL_CONFIG2, 0x04);
166     icm20608_write_onereg(dev, ICM20_PWR_MGMT_2, 0x00);
167     icm20608_write_onereg(dev, ICM20_LP_MODE_CFG, 0x00);
168     icm20608_write_onereg(dev, ICM20_FIFO_EN, 0x00);
169 }
```

第 9~48 行，icm20608_read_regs 函数，从 icm20608 中读取连续多个寄存器数据；注意：在本实验中，SPI 为全双工通讯没有所谓的发送和接收长度之分。要读取或者发送 N 个字节就要封装 N+1 个字节，第 1 个字节是告诉设备我们要进行读还是写，后面的 N 个字节才是我们要读或者发送的数据。

第 58~95 行，icm20608_write_rege 函数，向 icm20608 连续写入多个寄存器数据。此函数和 icm20608_read_regs 函数区别不大。

第 103~108 行，icm20608_read_onereg 函数，读取 icm20608 指定一个寄存器数据。

第 118~122 行，icm20608_write_onereg 函数，向 icm20608 指定一个寄存器写入数据。

第 130~142 行，icm20608_readdata 函数，读取 icm20608 六轴传感器和温度传感器原始数据值，应用程序读取 icm20608 的时候这些传感器原始数据就会上报给应用程序。

第 149~169 行，icm20608_reginit 函数，初始化 icm20608。

5、字符设备驱动框架

icm20608 的字符设备驱动框架如下：

示例代码 45.6.2.6 icm20608 字符设备驱动

```
1   /*
2    * @description   : 打开设备
3    * @param - inode : 传递给驱动的 inode
4    * @param - filp  : 要打开的设备文件(文件描述符)
5    * @return        : 0 成功;其他 失败
6    */
7   static int icm20608_open(struct inode *inode, struct file *filp)
8   {
9       return 0;
10  }
11
12  /*
13   * @description   : 从设备读取数据
14   * @param - filp  : 要打开的设备文件(文件描述符)
15   * @param - buf   : 返回给用户空间的数据缓冲区
16   * @param - cnt   : 要读取的数据长度
```

```
17   * @param - offt  : 相对于文件首地址的偏移
18   * @return        : 读取的字节数，如果为负值，表示读取失败
19   */
20  static ssize_t icm20608_read(struct file *filp, char __user *buf,
                                 size_t cnt, loff_t *off)
21  {
22      signed int data[7];
23      long err = 0;
24      struct cdev *cdev = filp->f_path.dentry->d_inode->i_cdev;
25      struct icm20608_dev *dev = container_of(cdev, struct icm20608_dev,
                                                cdev);
26
27      icm20608_readdata(dev);
28      data[0] = dev->gyro_x_adc;
29      data[1] = dev->gyro_y_adc;
30      data[2] = dev->gyro_z_adc;
31      data[3] = dev->accel_x_adc;
32      data[4] = dev->accel_y_adc;
33      data[5] = dev->accel_z_adc;
34      data[6] = dev->temp_adc;
35      err = copy_to_user(buf, data, sizeof(data));
36      return 0;
37  }
38
39  /*
40   * @description   : 关闭/释放设备
41   * @param - filp  : 要关闭的设备文件(文件描述符)
42   * @return        : 0 成功;其他 失败
43   */
44  static int icm20608_release(struct inode *inode, struct file *filp)
45  {
46      return 0;
47  }
48
49  /* icm20608 操作函数 */
50  static const struct file_operations icm20608_ops = {
51      .owner = THIS_MODULE,
52      .open = icm20608_open,
53      .read = icm20608_read,
54      .release = icm20608_release,
55  };
```

字符设备驱动框架没什么好说的，重点是第 20~37 行的 icm20608_read 函数，当应用程序调用 read 函数读取 icm20608 设备文件的时候此函数就会执行。其中的 24 和 25 行里，通过

probe 注册的 cdev 变量，获取到我们自定义的 icm20608_dev 结构体的首地址。此函数调用上面编写好的 icm20608_readdata 函数读取 icm20608 的原始数据并将其上报给应用程序。大家注意，在内核中尽量不要使用浮点运算，所以不要在驱动将 icm20608 的原始值转换为对应的实际值，因为会涉及到浮点计算。

这个驱动例程还不是很完美，在 icm20608_read 没有加锁。如果多个程序去读取这个驱动的时候就会出现读取数据出错，有能力的可以把这一点补充完整。

45.6.3 编写测试 APP

新建 icm20608App.c 文件，然后在里面输入如下所示内容：

示例代码 45.6.3.1 icm20608App.c 文件代码

```
12  #include "stdio.h"
13  #include "unistd.h"
14  #include "sys/types.h"
15  #include "sys/stat.h"
16  #include "sys/ioctl.h"
17  #include "fcntl.h"
18  #include "stdlib.h"
19  #include "string.h"
20  #include <poll.h>
21  #include <sys/select.h>
22  #include <sys/time.h>
23  #include <signal.h>
24  #include <fcntl.h>
25  /*
26   * @description   : main 主程序
27   * @param - argc  : argv 数组元素个数
28   * @param - argv  : 具体参数
29   * @return        : 0 成功;其他 失败
30   */
31  int main(int argc, char *argv[])
32  {
33      int fd;
34      char *filename;
35      signed int databuf[7];
36      unsigned char data[14];
37      signed int gyro_x_adc, gyro_y_adc, gyro_z_adc;
38      signed int accel_x_adc, accel_y_adc, accel_z_adc;
39      signed int temp_adc;
40
41      float gyro_x_act, gyro_y_act, gyro_z_act;
42      float accel_x_act, accel_y_act, accel_z_act;
43      float temp_act;
44
```

```
45      int ret = 0;
46
47      if (argc != 2) {
48          printf("Error Usage!\r\n");
49          return -1;
50      }
51
52      filename = argv[1];
53      fd = open(filename, O_RDWR);
54      if(fd < 0) {
55          printf("can't open file %s\r\n", filename);
56          return -1;
57      }
58
59      while (1) {
60          ret = read(fd, databuf, sizeof(databuf));
61          if(ret == 0) {          /* 数据读取成功 */
62              gyro_x_adc = databuf[0];
63              gyro_y_adc = databuf[1];
64              gyro_z_adc = databuf[2];
65              accel_x_adc = databuf[3];
66              accel_y_adc = databuf[4];
67              accel_z_adc = databuf[5];
68              temp_adc = databuf[6];
69
70              /* 计算实际值 */
71              gyro_x_act = (float)(gyro_x_adc)  / 16.4;
72              gyro_y_act = (float)(gyro_y_adc)  / 16.4;
73              gyro_z_act = (float)(gyro_z_adc)  / 16.4;
74              accel_x_act = (float)(accel_x_adc) / 2048;
75              accel_y_act = (float)(accel_y_adc) / 2048;
76              accel_z_act = (float)(accel_z_adc) / 2048;
77              temp_act = ((float)(temp_adc) - 25 ) / 326.8 + 25;
78
79
80              printf("\r\n 原始值:\r\n");
81              printf("gx = %d, gy = %d, gz = %d\r\n", gyro_x_adc,
                        gyro_y_adc, gyro_z_adc);
82              printf("ax = %d, ay = %d, az = %d\r\n", accel_x_adc,
                        accel_y_adc, accel_z_adc);
83              printf("temp = %d\r\n", temp_adc);
84              printf("实际值:");
85              printf("act gx = %.2f°/S, act gy = %.2f°/S, act gz
```

```
                = %.2f°/S\r\n", gyro_x_act, gyro_y_act, gyro_z_act);
86              printf("act ax = %.2fg, act ay = %.2fg, act az = %.2fg\r\n",
                       accel_x_act, accel_y_act, accel_z_act);
87              printf("act temp = %.2f°C\r\n", temp_act);
88          }
89          usleep(100000);   /*100ms    */
90      }
91      close(fd);            /* 关闭文件 */
92      return 0;
93  }
```

第 60~91 行，在 while 循环中每隔 100ms 从 icm20608 中读取一次数据，读取到 icm20608 原始数据以后将其转换为实际值，比如陀螺仪就是角速度、加速度计就是 g 值。注意，我们在 icm20608 驱动中将陀螺仪和加速度计的测量范围全部设置到了最大，分别为±2000 和±16g。因此，在计算实际值的时候陀螺仪使用 16.4，加速度计使用 2048。最终将传感器原始数据和得到的实际值显示在终端上。

45.7 运行测试

45.7.1 编译驱动程序和测试 APP

1、内核使能 SPI 控制器

ST 官方系统把 SPI 控制器的驱动编译成模块，我们需要把 SPI 控制器驱动编译进内核，这样就可以在启动 linux 内核的时候自动加载 SPI 控制器驱动，无需我们手动加载，方便我们使用。打开 Linux 内核图形化配置界面，按下路径找到对应的配置项：

```
→ Device Drivers
  → SPI support (SPI [=y])
    →<*> STMicroelectronics STM32 SPI controller      //编译进内核
```

如图 45.7.1.1 所示：

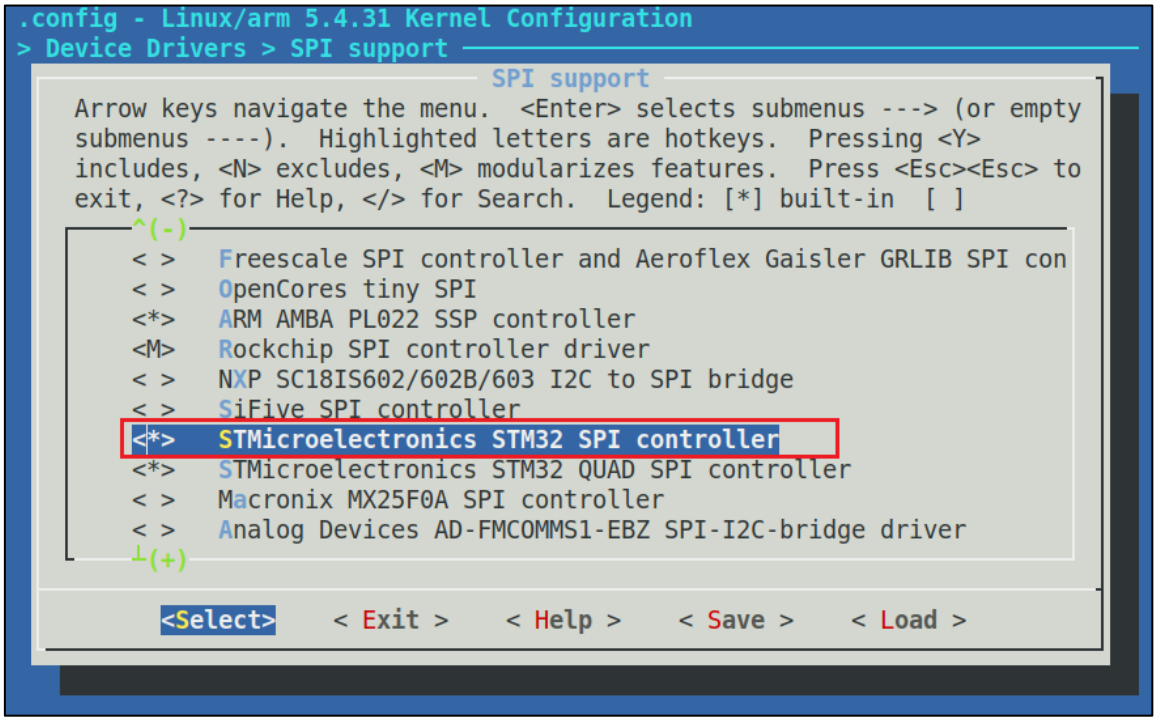


图 45.7.1.1 SPI 控制器配置项

图 45.7.1.1 本来是选择为“M”,我们要改为“*”，也就是编译进内核。接着我们重新编译设备树和内核，运行以下命令进行编译：

```
make dtbs uImage LOADADDR=0XC2000040 -j32
```

使用新编译好的 stm32mp157d-atk.dtb 和 uImage 镜像启动系统，如果 SPI 控制器驱动工作正常就会有图 45.7.1.2 所示提示信息：

```
printk: console [ttySTM0] enabled
panel-simple panel-rgb: panel-rgb supply power not found, using dummy regulator
spi_stm32 44004000.spi: driver initialized
stm32-dwmac 5800a000.ethernet: PTP uses main clock
stm32-dwmac 5800a000.ethernet: no reset control found
stm32-dwmac 5800a000.ethernet: No phy clock provided...
stm32-dwmac 5800a000.ethernet: User ID: 0x40, Synopsys ID: 0x42
```

SPI控制器初始化

图 45.7.1.2 SPI 控制器初始化

如果没有输出图 45.7.1.2 中“spi_stm32 44004000.spi: driver initialized”这句话，那就要检查一下设备树和内配配置是否有问题，通过查看/sys/bus/spi/devices/下有没有 spi 相关的设备，就能够知道设备树配置是否正确，比如本例程如图 45.7.1.3 所示：

```
[root@ATK-stm32mp1]:/sys/bus/spi/devices$ ls /sys/bus/spi/devices/
spi0.0
[root@ATK-stm32mp1]:/sys/bus/spi/devices$
```

图 45.7.1.3 SPI 设备

2、编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十章实验基本一样，只是将 obj-m 变量的值改为“icm20608.o”，Makefile 内容如下所示：

示例代码 45.7.1.1 Makefile 文件

```
1 KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
```

```
4  obj-m := icm20608.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“icm20608.o”。

输入如下命令编译出驱动模块文件：

```
make -j16
```

编译成功以后就会生成一个名为“icm20608.ko”的驱动模块文件。

3、编译测试 APP

在 icm20608App.c 这个测试 APP 中我们用到了浮点计算，而 STM32MP1 是支持硬件浮点的，因此我们在编译 icm20608App.c 的时候就可以使能硬件浮点，这样可以加速浮点计算。使能硬件浮点很简单，在编译的时候加入如下参数即可：

```
-march-armv7-a -mfpu-neon -mfloat=hard
```

输入如下命令使能硬件浮点编译 icm20608App.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc -march=armv7-a -mfpu=neon -mfloat-abi=hard icm20608App.c -o icm20608App
```

编译成功以后就会生成 icm20608App 这个应用程序，那么究竟有没有使用硬件浮点呢？使用 arm-linux-gnueabihf-readelf 查看一下编译出来的 icm20608App 就知道了，输入如下命令：

```
arm-none-linux-gnueabihf-readelf -A icm20608App
```

结果如图 45.7.1.4 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/22_spi$ arm-none-linux-gnueabihf-readelf -A icm20608App
Attribute Section: aeabi
File Attributes
  Tag_CPU_name: "7-A"
  Tag_CPU_arch: v7
  Tag_CPU_arch_profile: Application
  Tag_ARM_ISA_use: Yes
  Tag_THUMB_ISA_use: Thumb-2
  Tag_FP_arch: VFPv3
  Tag_Advanced_SIMD_arch: NEONv1
  Tag_ABI_PCS_wchar_t: 4
  Tag_ABI_FP_rounding: Needed
  Tag_ABI_FP_denormal: Needed
  Tag_ABI_FP_exceptions: Needed
  Tag_ABI_FP_number_model: IEEE 754
  Tag_ABI_align_needed: 8-byte
  Tag_ABI_align_preserved: 8-byte, except leaf SP
  Tag_ABI_enum_size: int
  Tag_ABI_VFP_args: VFP registers
  Tag_CPU_unaligned_access: v6
zuozhongkai@ubuntu:~/linux/atk-mp1/Drivers/Linux_Drivers/22_spi$
```

图 45.7.1.4 icm20608App 文件信息

从图 45.7.1.4 可以看出 FPU 架构为 VFPv3，SIMD 使用了 NEON，说明 icm20608App 这个应用程序使用了硬件浮点。

45.7.2 运行测试

将上一小节编译出来 icm20608.ko 和 icm20608App 这两个文件拷贝到 rootfs/lib/modules/5.3.41 目录中，重启开发板，进入到目录 lib/modules/5.3.41 中。输入如下命令加载 icm20608.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe icm20608.ko  //加载驱动模块
```

当驱动模块加载成功以后使用 icm20608App 来测试，输入如下命令：

```
./icm20608App /dev/icm20608
```

测试 APP 会不断的从 ICM20608 中读取数据，然后输出到终端上，如图 45.7.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./icm20608App /dev/icm20608

原始值:
gx = -19, gy = -26, gz = 5
ax = -685, ay = -1025, az = 1996
temp = 4766
实际值:act gx = -1.16°/S, act gy = -1.59°/S, act gz = 0.30°/S
act ax = -0.33g, act ay = -0.50g, act az = 0.97g
act temp = 39.51°C
```

图 45.7.2.1 获取到的 ICM20608 数据

可以看出，开发板静止状态下，Z 轴方向的加速度为 0.97g，这个就是重力加速度。对于陀螺仪来讲，静止状态下三轴的角速度应该在 0°/S 左右。ICM20608 内温度传感器采集到的温度在 39.51 度，大家可以晃动一下开发板，这个时候陀螺仪和加速度计的值就会有变化。

第四十六章 Linux RS232/485/GPS 驱动实验

串口是很常用的一个外设，在 Linux 下通常通过串口和其他设备或传感器进行通信，根据电平的不同，串口分为 TTL 和 RS232。不管是什么样的接口电平，其驱动程序都是一样的，通过外接 RS485 这样的芯片就可以将串口转换为 RS485 信号，正点原子的 STM32MP1 开发板就是这么做的。对于正点原子的 STM32MP1 开发板而言有 8 个串口，四个同步串口(USART1、USART2、USART3 和 USART6)，四个异步串口(UART4、UART5、UART7 和 UART8)。

RS232 和 RS485 接口连接到了 STM32MP1 的 USART3 接口上，通过跳线帽选择 USART3 作为 RS232 还是 RS485。GPS 模块是连接到 UART5 接口上，因此这些外设最终都归结为 USART3 和 UART5 的串口驱动。本章我们就来学习一下如何驱动 STM32MP1 开发板上的 USART3 串口和 UART5，进而实现 RS232、RS485 以及 GPS 驱动。

46.1 Linux 下 UART 驱动框架

1、uart_driver 注册与注销

同 I2C、SPI 一样，Linux 也提供了串口驱动框架，我们只需要按照相应的串口框架编写驱动程序即可。串口驱动没有什么主机端和设备端之分，就只有一个串口驱动，而且这个驱动也已经由 ST 官方编写好了，我们真正要做的就是在设备树中添加所要使用的串口节点信息。当系统启动以后串口驱动和设备匹配成功，相应的串口就会被驱动起来，生成/dev/ttySTMX(X=0….n)文件。

虽然串口驱动不需要我们去写，但是串口驱动框架我们还是需要了解的，uart_driver 结构体表示 UART 驱动，uart_driver 定义在 include/linux/serial_core.h 文件中，内容如下：

示例代码 46.1.1 uart_driver 结构体

```
303 struct uart_driver {
304     struct module     *owner;           /* 模块所属者    */
305     const char        *driver_name;     /* 驱动名字      */
306     const char        *dev_name;        /* 设备名字      */
307     int               major;            /* 主设备号      */
308     int               minor;            /* 次设备号      */
309     int               nr;               /* 设备数        */
310     struct console    *cons;            /* 控制台        */
311
312      /*
313      * these are private; the low level driver should not
314      * touch these; they should be initialised to NULL
315      */
316     struct uart_state   *state;
317     struct tty_driver   *tty_driver;
318 };
```

每个串口驱动都需要定义一个 uart_driver，加载驱动的时候通过 uart_register_driver 函数向系统注册这个 uart_driver，此函数原型如下：

```
int uart_register_driver(struct uart_driver *uart)
```

函数参数和返回值含义如下：

uart：要注册的 uart_driver。

返回值：0，成功；负值，失败。

注销驱动的时候也需要注销掉前面注册的 uart_driver，需要用到 uart_unregister_driver 函数，函数原型如下：

```
void uart_unregister_driver(struct uart_driver *uart)
```

函数参数和返回值含义如下：

uart：要注销的 uart_driver。

返回值：无。

2、uart_port 的添加与移除

uart_port 表示一个具体的 port，uart_port 定义在 include/linux/serial_core.h 文件，内容如下(有省略)：

示例代码 46.1.2 uart_port 结构体

```
105 struct uart_port {
106     spinlock_t          lock;                   /* port lock      */
107     unsigned long       iobase;                 /* in/out[bwl]    */
108     unsigned char       __iomem   *membase;     /* read/write[bwl] */
......
240     const struct        uart_ops    *ops;
241     unsigned int        custom_divisor;
242     unsigned int        line;                   /* port index     */
243     unsigned int        minor;
244     resource_size_t     mapbase;                /* for ioremap    */
245     resource_size_t     mapsize;
246     struct device       *dev;                   /* parent device  */
......
256 };
```

uart_port 中最主要的就是第 240 行的 ops，ops 包含了串口的具体驱动函数，这个我们稍后再看。每个 UART 都有一个 uart_port，那么 uart_port 是怎么和 uart_driver 结合起来的呢？这里要用到 uart_add_one_port 函数，函数原型如下：

```
int uart_add_one_port(struct uart_driver      *reg,
                      struct uart_port        *port)
```

函数参数和返回值含义如下：

reg：此 port 对应的 uart_driver。

port：要添加到 uart_driver 中的 port。

返回值：0，成功；负值，失败。

卸载 UART 驱动的时候也需要将 uart_port 从相应的 uart_driver 中移除，需要用到 uart_remove_one_port 函数，函数原型如下：

```
int uart_remove_one_port(struct uart_driver *reg, struct uart_port *port)
```

函数参数和返回值含义如下：

reg：要卸载的 port 所对应的 uart_driver。

port：要卸载的 uart_port。

返回值：0，成功；负值，失败。

3、uart_ops 实现

在上面讲解 uart_port 的时候说过，uart_port 中的 ops 成员变量很重要，因为 ops 包含了针对 UART 具体的驱动函数，Linux 系统收发数据最终调用的都是 ops 中的函数。ops 是 uart_ops 类型的结构体指针变量，uart_ops 定义在 include/linux/serial_core.h 文件中，内容如下：

示例代码 46.1.3 uart_port 结构体

```
37  struct uart_ops {
38      unsigned int    (*tx_empty)(struct uart_port *);
39      void            (*set_mctrl)(struct uart_port *, unsigned int mctrl);
40      unsigned int    (*get_mctrl)(struct uart_port *);
41      void        (*stop_tx)(struct uart_port *);
42      void        (*start_tx)(struct uart_port *);
43      void        (*throttle)(struct uart_port *);
```

```
44      void        (*unthrottle)(struct uart_port *);
45      void        (*send_xchar)(struct uart_port *, char ch);
46      void        (*stop_rx)(struct uart_port *);
47      void        (*enable_ms)(struct uart_port *);
48      void        (*break_ctl)(struct uart_port *, int ctl);
49      int         (*startup)(struct uart_port *);
50      void        (*shutdown)(struct uart_port *);
51      void        (*flush_buffer)(struct uart_port *);
52      void       (*set_termios)(struct uart_port *, struct ktermios *new,
53                      struct ktermios *old);
54      void        (*set_ldisc)(struct uart_port *, struct ktermios *);
55      void        (*pm)(struct uart_port *, unsigned int state,
56                     unsigned int oldstate);
57
58      /*
59       * Return a string describing the type of the port
60       */
61      const char  *(*type)(struct uart_port *);
62
63      /*
64       * Release IO and memory resources used by the port.
65       * This includes iounmap if necessary.
66       */
67      void        (*release_port)(struct uart_port *);
68
69      /*
70       * Request IO and memory resources used by the port.
71       * This includes iomapping the port if necessary.
72       */
73      int     (*request_port)(struct uart_port *);
74      void    (*config_port)(struct uart_port *, int);
75      int     (*verify_port)(struct uart_port *, struct serial_struct *);
76      int     (*ioctl)(struct uart_port *, unsigned int, unsigned long);
77  #ifdef CONFIG_CONSOLE_POLL
78      int     (*poll_init)(struct uart_port *);
79      void    (*poll_put_char)(struct uart_port *, unsigned char);
80      int     (*poll_get_char)(struct uart_port *);
81  #endif
82  };
```

UART 驱动编写人员需要实现 uart_ops，因为 uart_ops 是最底层的 UART 驱动接口，是实实在在的和 UART 寄存器打交道的。关于 uart_ops 结构体中的这些函数的具体含义请参考 Documentation/serial/driver 这个文档。

UART 驱动框架大概就是这些，接下来我们理论联系实际，看一下 ST 官方的 UART 驱动

文件是如何编写的。

46.2 STM32MP1 UART 驱动分析

1、UART 的 platform 驱动框架

打开 stm32mp151.dtsi 文件，找到 USART3 对应的子节点，子节点内容如下所示：

示例代码 46.2.1 usart3 设备节点

```
1  usart3: serial@4000f000 {
2      compatible = "st,stm32h7-uart";
3      reg = <0x4000f000 0x400>;
4      interrupts-extended = <&exti 28 IRQ_TYPE_LEVEL_HIGH>;
5      clocks = <&rcc USART3_K>;
6      resets = <&rcc USART3_R>;
7      wakeup-source;
8      power-domains = <&pd_core>;
9      dmas = <&dmamux1 45 0x400 0x5>,
10           <&dmamux1 46 0x400 0x1>;
11     dma-names = "rx", "tx";
12     status = "disabled";
13 };
```

重点看一下第 2 行的 compatible 属性值为“st,stm32h7-uart”。在 linux 源码中搜索这个值即可找到对应的 UART 驱动文件，此文件为 drivers/tty/serial/stm32-usart.c，在此文件中可以找到如下内容：

示例代码 46.2.2 UART platform 驱动框架

```
1218   static const struct of_device_id stm32_match[] = {
1219       { .compatible = "st,stm32-uart", .data = &stm32f4_info},
1220       { .compatible = "st,stm32f7-uart", .data = &stm32f7_info},
1221       { .compatible = "st,stm32h7-uart", .data = &stm32h7_info},
1222       {},
1223   };
......
1668   static struct platform_driver stm32_serial_driver = {
1669       .probe      = stm32_usart_serial_probe,
1670       .remove     = stm32_usart_serial_remove,
1671       .driver = {
1672           .name   = DRIVER_NAME,
1673           .pm = &stm32_serial_pm_ops,
1674           .of_match_table = of_match_ptr(stm32_match),
1675       },
1676   };
1677
1678   static int __init stm32_usart_init(void)
1679   {
```

```
1680        static char banner[] __initdata = "STM32 USART driver
                                              initialized";
1681        int ret;
1682
1683        pr_info("%s\n", banner);
1684
1685        ret = uart_register_driver(&stm32_usart_driver);
1686        if (ret)
1687            return ret;
1688
1689        ret = platform_driver_register(&stm32_serial_driver);
1690        if (ret)
1691            uart_unregister_driver(&stm32_usart_driver);
1692
1693        return ret;
1694    }
1695
1696    static void __exit stm32_usart_exit(void)
1697    {
1698        platform_driver_unregister(&stm32_serial_driver);
1699        uart_unregister_driver(&stm32_usart_driver);
1700    }
1701
1702    module_init(stm32_usart_init);
1703    module_exit(stm32_usart_exit);
```

可以看出 STM32MP1 的 UART 本质上是一个 platform 驱动，第 1218~1223 行，设备树所使用的匹配表，第 1221 行的 compatible 属性值为“st,stm32h7-uart”。

第 1668~1676 行，platform 驱动框架结构体 stm32_serial_driver。

第 1678~1694 行，驱动入口函数，第 1685 行调用 uart_register_driver 函数向 Linux 内核注册 uart_driver，在这里就是 stm32_usart_driver。

第 1696~1700 行，强度出口函数，第 1699 行调用 uart_unregister_driver 函数注销掉前面注册的 uart_driver，也就是 stm32_usart_driver。

2、uart_driver 初始化

在 stm32_usart_init 函数中向 Linux 内核注册了 stm32_usart_driver，stm32_usart_driver 就是 uart_driver 类型的结构体变量，stm32_usart_driver 定义如下：

示例代码 46.2.3 stm32_usart_driver 结构体

```
1550    static struct uart_driver stm32_usart_driver = {
1551        .driver_name  = DRIVER_NAME,
1552        .dev_name     = STM32_SERIAL_NAME,
1553        .major        = 0,
1554        .minor        = 0,
1555        .nr           = STM32_MAX_PORTS,
1556        .cons         = STM32_SERIAL_CONSOLE,
```

```
1557    };
```

3、uart_port 初始化与添加

当 UART 设备和驱动匹配成功以后 stm32_usart_serial_probe 函数就会执行，此函数的重点工作就是初始化 uart_port，然后将其添加到对应的 uart_driver 中。在看 stm32_usart_serial_probe 函数之前先来看一下 stm32_port 结构体，stm32_port 是 ST 为 STM32MP1 系列 SOC 定义的一个设备结构体，此结构体内部就包含了 uart_port 成员变量，stm32_port 结构体内容如下所示(有缩减)：

示例代码 46.2.4 stm32_port 结构体

```
251 #define STM32_MAX_PORTS 8
......
257 struct stm32_port {
258     struct uart_port port;
259     struct clk *clk;
260     struct stm32_usart_info *info;
......
274     int rdr_mask;       /* receive data register mask */
275     struct dma_tx_state state;
276     enum dma_status status;
277 };
......
279 static struct stm32_port stm32_ports[STM32_MAX_PORTS];
```

第 258 行，uart_port 结构体成员变量：port。

第 279 行，这里定义了一个数组为 stm32_ports，数组的类型为 stm32_port 结构体，数组的长度为 8。这是因为 STM32MP157 最多只有 8 个串口，一个串口对应一个 stm32_port，因此数组长度就是 8。

接下来看一下 stm32_usart_serial_probe 函数，函数内容如下：

示例代码 46.2.5 stm32_usart.c 文件代码段

```
1306    static int stm32_usart_serial_probe(struct platform_device *pdev)
1307    {
1308        const struct of_device_id *match;
1309        struct stm32_port *stm32port;
1310        int ret;
1311
1312        stm32port = stm32_usart_of_get_port(pdev);
1313        if (!stm32port)
1314            return -ENODEV;
1315
1316        match = of_match_device(stm32_match, &pdev->dev);
1317        if (match && match->data)
1318            stm32port->info = (struct stm32_usart_info *)match->data;
1319        else
1320            return -EINVAL;
1321
```

```
1322        ret = stm32_usart_init_port(stm32port, pdev);
1323        if (ret)
1324            return ret;
......
1373
1374        ret = uart_add_one_port(&stm32_usart_driver,
                                    &stm32port->port);
1375        if (ret)
1376            goto err_port;
1377
1378        pm_runtime_put_sync(&pdev->dev);
1379
1380        return 0;
.....
1410    }
```

第 1312 行，调用 stm32_usart_of_get_port 函数，它主要是负责配置 stm32_ports 数组。

第 1322 行，调用 stm32_usart_init_port 函数，它主要是负责获取 SOC UART 外设首地址、中断号、注册中断函数同时还设置 uart_ops 为 stm32_uart_ops，stm32_uart_ops 就是 STM32MP1 最底层的驱动函数集合。

第 1374 行，使用 uart_add_one_port 向 uart_driver 添加 uart_port，在这个就是向 stm32_usart_driver 添加 stm32port->port。

我们接下来看一下 stm32_usart_of_get_port 函数，因为 stm32_usart_serial_probe 函数会调用此函数来获取串口信息，这些串口信息会放到示例代码 46.2.4 中的 stm32_ports 数组里面。stm32_usart_of_get_port 函数源码如下：

示例代码 46.2.6 stm32_usart_of_get_port 函数

```
1189 static struct stm32_port *stm32_usart_of_get_port(
                                struct platform_device *pdev)
1190 {
1191    struct device_node *np = pdev->dev.of_node;
1192    int id;
1193
1194    if (!np)
1195        return NULL;
1196
1197    id = of_alias_get_id(np, "serial");
1198    if (id < 0) {
1199        dev_err(&pdev->dev, "failed to get alias id, errno %d\n",
                                id);
1200        return NULL;
1201    }
1202
1203    if (WARN_ON(id >= STM32_MAX_PORTS))
1204        return NULL;
```

```
1205
1206    stm32_ports[id].hw_flow_control =
1207                of_property_read_bool(np, "uart-has-rtscts");
1208    stm32_ports[id].port.line = id;
1209    stm32_ports[id].cr1_irq = USART_CR1_RXNEIE;
1210    stm32_ports[id].cr3_irq = 0;
1211    stm32_ports[id].last_res = RX_BUF_L;
1212    stm32_ports[id].rx_dma_buf = 0;
1213    stm32_ports[id].tx_dma_buf = 0;
1214    return &stm32_ports[id];
1215 }
```

第 1197 行，通过 of_alias_get_id 函数从设备树的 aliases 节点中获取“serial”相关的 ID。打开 stm32mp157d-atk.dts 文件，当前此文件里面的 aliases 节点内容如图 46.2.1 所示：

```
25      aliases {
26          serial0 = &uart4;
27      };
```

图 46.2.1 aliases 节点

从图 46.2.1 可以看出，此时 alases 节点里面只有一个 serial0，对应 STM32MP157 的 uart4。所以 stm32_usart_of_get_port 函数只能得到 uart4 这一个串口的信息，如果要使用其他的串口，那就必须向 alases 节点里面按照如下格式添加对应的串口信息：

```
serialX=&串口名字;
```

这个 X 表示 0~7，那是因为 STM32MP1 的串口只有 8 个。&后面的串口名字一定要对应设备树中具体的串口名，比如 usart3、uart5 等。

第 1206~1213 行，获取对应的串口信息，然后保存到 stm32_ports 数组中，获取到的串口 ID 就是串口在数组中的索引。

第 1214 行，返回得到的串口信息。

接下来我们再来看一下 stm32_usart_init_port 函数，stm32_usart_serial_probe 函数会调用此函数来初始化串口！函数源码如下所示：

示例代码 46.2.7 stm32_usart_init_port 函数

```
1136 static int stm32_usart_init_port(struct stm32_port *stm32port,
1137                struct platform_device *pdev)
1138 {
1139    struct uart_port *port = &stm32port->port;
1140    struct resource *res;
1141    int ret;
1142
1143    ret = platform_get_irq(pdev, 0);
1144    if (ret <= 0)
1145        return ret ? : -ENODEV;
1146
1147    port->iotype    = UPIO_MEM;
1148    port->flags = UPF_BOOT_AUTOCONF;
1149    port->ops   = &stm32_uart_ops;
```

```
1150    port->dev   = &pdev->dev;
1151    port->fifosize  = stm32port->info->cfg.fifosize;
1152    port->irq = ret;
1153    port->rs485_config = stm32_usart_config_rs485;
1154
1155    stm32_usart_init_rs485(port, pdev);
1156
1157    if (stm32port->info->cfg.has_wakeup)
1158        stm32port->wakeup_src =
                of_property_read_bool(pdev->dev.of_node,
1159                              "wakeup-source");
1160
1161    stm32port->fifoen = stm32port->info->cfg.has_fifo;
1162
1163    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
1164    port->membase = devm_ioremap_resource(&pdev->dev, res);
1165    if (IS_ERR(port->membase))
1166        return PTR_ERR(port->membase);
1167    port->mapbase = res->start;
......
1186    return ret;
1187 }
```

stm32_usart_init_port 函数主要是负责获取 SOC UART 外设首地址、中断号、注册中断函数。重点是 1149 行设置 uart_ops 为 stm32_uart_ops，stm32_uart_ops 就是 STM32MP1 最底层的驱动函数集合。

4、stm32_uart_ops 结构体变量

stm32_uart_ops 就是 uart_ops 类型的结构体变量，保存了 STM32MP1 串口最底层的操作函数，stm32_uart_ops 定义如下：

示例代码 46.2.8 stm32_uart_ops

```
1109    static const struct uart_ops stm32_uart_ops = {
1110        .tx_empty     = stm32_usart_tx_empty,
1111        .set_mctrl    = stm32_usart_set_mctrl,
1112        .get_mctrl    = stm32_usart_get_mctrl,
1113        .stop_tx      = stm32_usart_stop_tx,
1114        .start_tx     = stm32_usart_start_tx,
1115        .throttle     = stm32_usart_throttle,
1116        .unthrottle   = stm32_usart_unthrottle,
1117        .stop_rx      = stm32_usart_stop_rx,
1118        .break_ctl    = stm32_usart_break_ctl,
1119        .startup      = stm32_usart_startup,
1120        .shutdown     = stm32_usart_shutdown,
1121        .flush_buffer = stm32_usart_flush_buffer,
1122        .set_termios  = stm32_usart_set_termios,
```

```
1123        .pm             = stm32_usart_pm,
1124        .type           = stm32_usart_type,
1125        .release_port = stm32_usart_release_port,
1126        .request_port = stm32_usart_request_port,
1127        .config_port   = stm32_usart_config_port,
1128        .verify_port   = stm32_usart_verify_port,
1129    };
```

stm32_uart_ops 中的函数基本都是和 STM32MP1 的 UART 寄存器打交道的，这里就不去详细的分析了。简单的了解了 STM32MP1 的 UART 驱动以后我们再来学习一下，如何驱动正点原子 STM32MP1 开发板上的 USART3 接口和 UART5 接口。

46.3 硬件原理图分析

本实验要用到的 STM32MP1 的 USART3 接口和 UART5 接口，USART3 连接 RS485 和 RS232 的公头，UART5 连接 GPS 和 RS232 的母头。我们依次来看一下这个两个串口的硬件原理图。

1、RS232 原理图

RS232 原理图如图 46.3.1 所示：

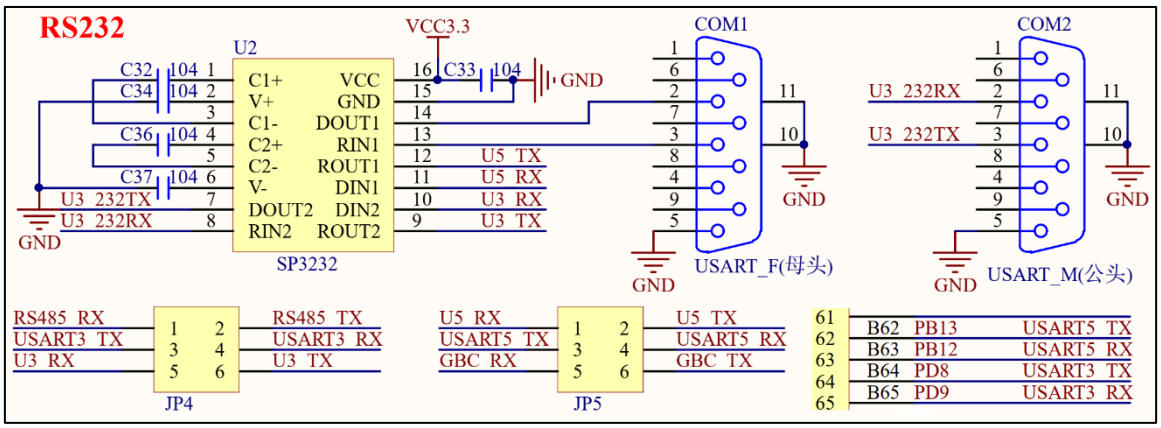


图 46.3.1 RS232 原理图

正点原子 STM32MP157 开发板一共有 2 个 RS232 串口，图 46.3.1 中 COM1 是母头，COM2 为公头，这两个 RS232 串口都是通过 SP3232 这个芯片来实现。

COM1 母头连接到 STM32MP1 的 UART5 接口上，COM1 和正点原子的 ATK 模块共用 USART5，把 JP5 的 1-3 和 2-4 连接起来以后 SP3232 就和 URAT5 连接到一起了。UART5_TX 和 UART5_RX 分别接到了 PB13 和 PB12 这两个引脚上。

COM2 公头连接到了 STM32MP1 的 USART3 接口上，COM2 和 RS485 共用 USART3，把 JP4 的 3-5 和 4-6 连接起来以后 SP3232 就和 USRAT3 连接到一起了。USART3_TX 和 USART3_RX 分别接到了 PD8 和 PD9 这两个引脚上。

2、RS485 原理图

上面已经说了，RS485 和 COM2 共用 USART3，将图 46.3.1 中 JP4 的 3-5 和 4-6 连接起来，这时候 RS485 就连接到了 USART3 上。RS485 原理图如图 46.3.2 所示：

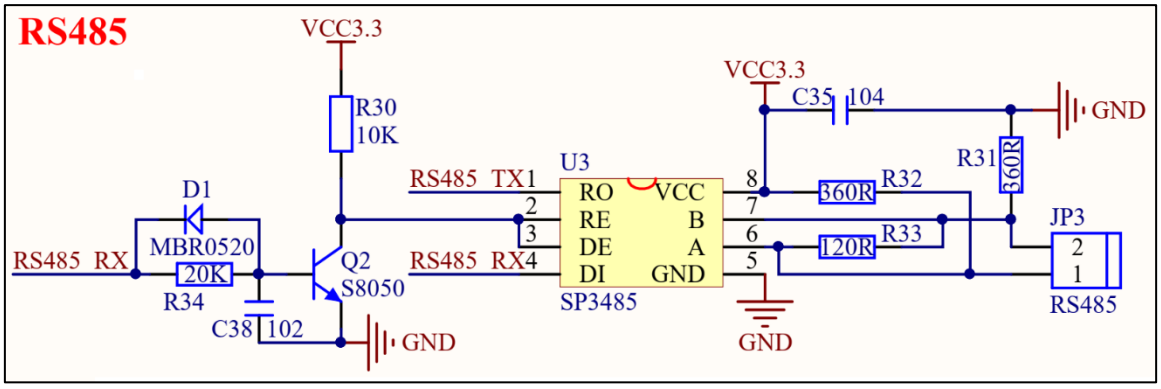


图 46.3.2 RS485 原理图

RS485 采用 SP3485 这颗芯片来实现，RO 为数据输出端，RI 为数据输入端，RE 是接收使能信号(低电平有效)，DE 是发送使能信号(高电平有效)。在图 46.3.2 中 RE 和 DE 经过一系列的电路，最终通过 RS485_RX 来控制，这样我们可以省掉一个 RS485 收发控制 IO，将 RS485 完全当作一个串口来使用，方便我们写驱动。

3、GPS 原理图

正点原子有一款 GPS+北斗定位模块，型号为 ATK1218-BD，STM32MP157 开发板留出了这款 GPS 定位模块的接口，接口原理图如图 46.3.3 所示：

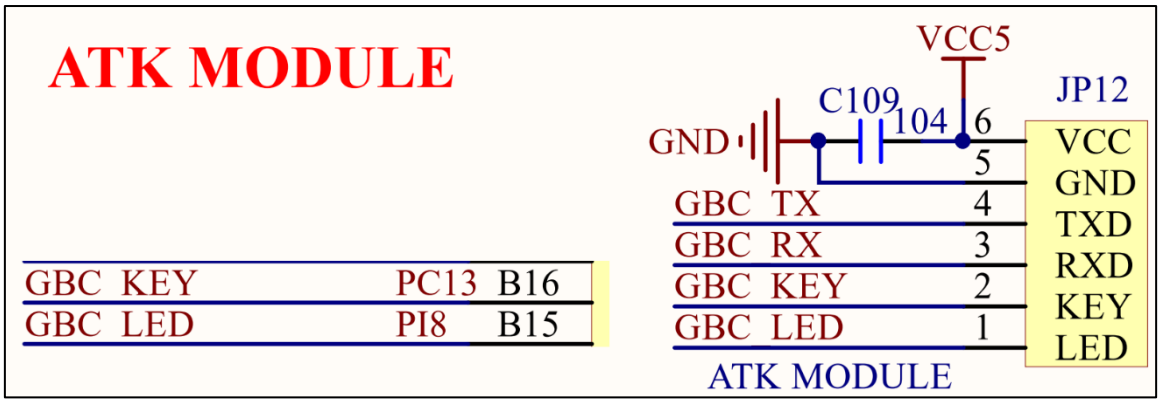


图 46.3.3 ATK MODULE 模块。

前面讲解 RS232 原理图的时候说了，COM1 和正点原子的 ATK 模块共用 USART5 接口，正点原子的 ATK1218-BD 这个模块用的就是 ATK 模块接口。如果要使用 GPS 模块，就要将图 46.3.1 中 JP5 的 3-5 和 4-6 连接起来。这样 GPS 模块就连接到了 USART5 上，USART5 驱动成功以后就可以直接读取 GPS 模块数据了。从图 46.3.3 可以看出，ATK 模块还有两个引脚 GBC_KEY 和 GBC_LED 分别连接到了 STM32MP157 的 PC13 和 PI8 上，这两个引脚是给其他模块准备的，GPS 模块并没有用到。

46.4 RS232 驱动编写

前面我们已经说过了，STM32MP1 的 UART 驱动 ST 已经编写好了，所以不需要我们编写。我们要做的就是在设备树中添加 USART3 和 UART5 对应的设备节点即可。打开 stm32mp157d-atk.dts 文件，因为 usart3 和 uasrt5 的节点在 stm32mp151.dtsi 已经存在了，我们只要在 stm32mp157d-atk.dts 文件里面向这些节点追加一些内容即可，追加步骤如下：

1、添加 usart3 和 uart5 的引脚信息

我们先在 stm32mp15-pinctrl.dtsi 文件看下没有 usart3 和 uart5 的引脚配置，以及引脚配置是否是我们开发板对应的。默认情况下 stm32mp15-pinctrl.dtsi 里面是有 usart3 的引脚配置，但是不是我们开发板所使用的 PD8 和 PD9，所以不能使用。我们直接在 stm32mp15-pinctrl.dtsi 里面添加 usart3 和 uart5 这两个串口对应的引脚信息，内容如下

示例代码 46.4.1 要追加的 pinmux 配置

```
1    usart3_pins_c: usart3-2 {
2        pins1 {
3            pinmux = <STM32_PINMUX('D', 8, AF7)>; /* USART3_TX */
4            bias-disable;
5            drive-push-pull;
6            slew-rate = <0>;
7        };
8        pins2 {
9            pinmux = <STM32_PINMUX('D', 9, AF7)>; /* USARTS_RX */
10           bias-disable;
11       };
12   };
13
14   uart5_pins_a: uart5-0 {
15       pins1 {
16           pinmux = <STM32_PINMUX('B', 13, AF14)>; /* UART5_TX */
17           bias-disable;
18           drive-push-pull;
19           slew-rate = <0>;
20       };
21       pins2 {
22           pinmux = <STM32_PINMUX('B', 12, AF14)>; /* UART5_RX */
23           bias-disable;
24       };
25   };
```

示例代码 46.4.1 里配置了两个 pinmux 分别为 usart3_pins_c 和 uart5_pins_a。稍后我们向 usart3 和 uart5 中追加内容的时候就会用到这两个节点。

2、向 usart3 和 uart5 节点追加内容

还是在 stm32mp157d-atk.dts 文件中，在不是根节点下追加如下代码：

示例代码 46.4.2 串口的节点

```
1  &usart3 {
2      pinctrl-names = "default";
3      pinctrl-0 = <&usart3_pins_c>;
4      status = "okay";
5  };
6
7  &uart5 {
8      pinctrl-names = "default";
```

```
9       pinctrl-0 = <&uart5_pins_a>;
10      status = "okay";
11  };
```

这里我们追加了两个串口，分别为 uart5 和 usart3，追加的内容很简单都是使用了我们刚刚添加的 pinmux 配置。把 status 属性原来为“disabled”改为“okay”。

3、设置串口的别名

在 46.2 小节 UART 驱动分析已经知道，驱动会读取 aliases 节点，添加的别名如下所示：

示例代码 46.4.3 串口的别名

```
1   aliases {
2       serial0 = &uart4;
3       serial1 = &uart5;
4       serial2 = &usart3;
5   };
```

serial0 是 uart4 的别名，表示在系统启动生成一个名为“/dev/ttySTM0”的设备文件，serial1 就会生成“/dev/ttySTM1”如此类推，最多 8 个。**serial0 就是我们的调试串口。**

修改完成以后重新编译设备树并使用新的设备树启动 Linux，如果设备树修改成功的话，系统启动以后就会有如 46.4.1 所示设备文件：

```
[root@ATK-stm32mp1]:~$ ls /dev/ttySTM*
/dev/ttySTM0  /dev/ttySTM1  /dev/ttySTM2
[root@ATK-stm32mp1]:~$
```

图 46.4.1 串口的设备文件

ttySTM0 为 serial0，对应 uart4；ttySTM1 为 serial1，对应 uart5；ttySTM2 为 serial2，对应 usart3。

46.5 移植 minicom

minicom 类似我们常用的串口调试助手，是 Linux 下很常用的一个串口工具，将 minicom 移植到我们的开发板中，这样我们就可以借助 minicom 对串口进行读写操作。

buildroot 已经集成了 minicom，所以我们只需要重新配置 buildroot，使能 minicom 即可。首先我们跳转到 buildroot 的源码目录下，打开 buildroot 的图形化配置界面里配置以下选项：

```
→ Target packages
    → Hardware handling
        [*] minicom
```

配置如图 46.5.1 所示：

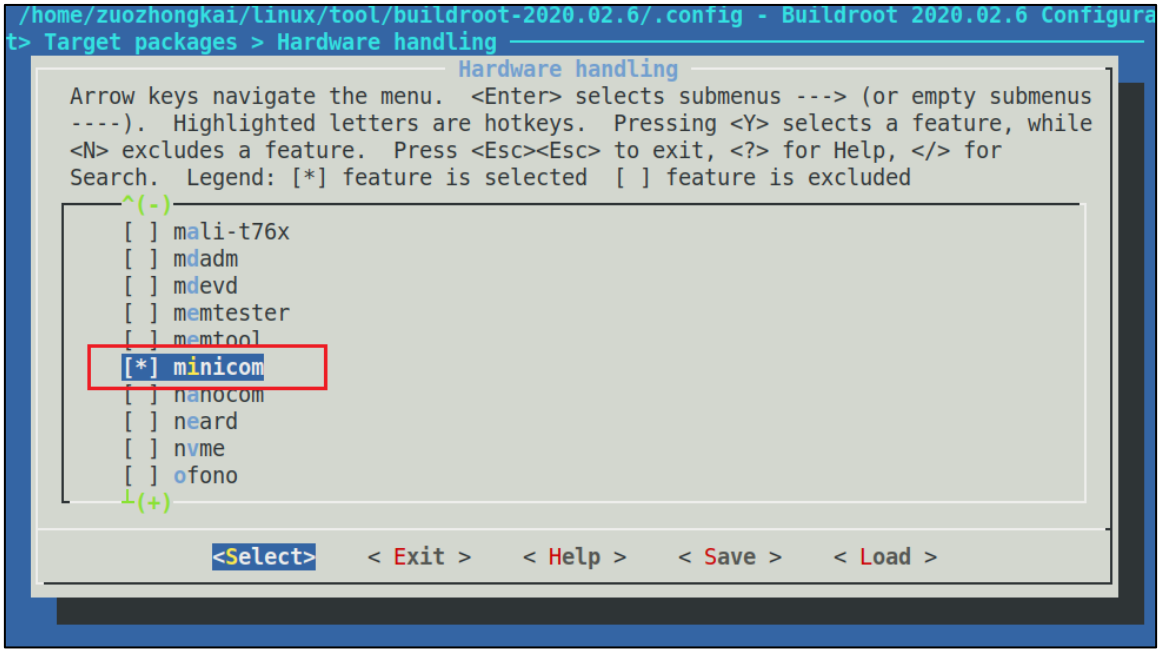


图 46.5.1 使能 minicom

我们保存 buildroot 的配置文件，输入命令“sudo make”重新编译文件系统。编译的时候要联网，因为 buildroot 在编译的时候需要从网上下载 minicom 源码。当编译完成后，进入 output/images 目录，运行以下命令把文件系统替换进去：

```
cd output/images/                                          //进入到 output/images 目录
sudo tar -axvf rootfs.tar -C /home/zuozhongkai/linux/nfs/rootfs  //解压到 nfsroot 目录
```

上述命令将 buildroot 中 output/images/rootfs.tar 这个压缩包解压到/home/zuozhongkai/linux/nfs/rootfs 这个目录中，这个目录就是我当前 nfsroot 目录，大家根据自己的实际情况解压到对应的目录文件中。

完成以后重启开发板！

重启以后在开发板中输入“minicom -v”来查看 minicom 工作是否正常，结果如图 46.5.2 所示：

```
[root@ATK-stm32mp1]:~$ minicom -v
minicom version 2.7.90 (compiled Jun 17 2021)
Copyright (C) Miquel van Smoorenburg.

This program is free software; you can redistribute it and/or
modify it under the terms of the GNU General Public License
as published by the Free Software Foundation; either version
2 of the License, or (at your option) any later version.

[root@ATK-stm32mp1]:~$
```

图 46.5.2 minicom 版本号

从图 46.5.2 可以看出，此时 minicom 版本号为 2.7.90，minicom 版本号查看正常。输入如下命令打开 minicom 配置界面：

```
minicom -s
```

此时 minicom 配置界面就可以打开了，如图 46.5.3 所示：

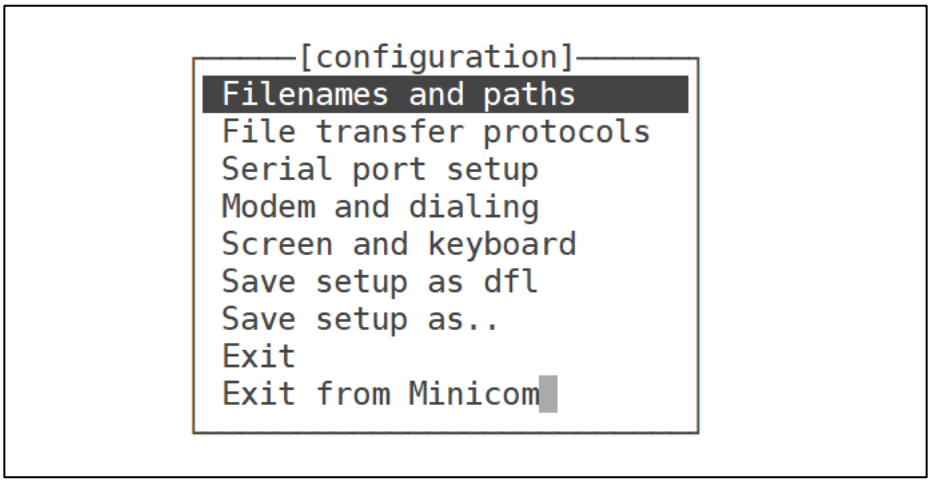


图 46.5.3 mincom 配置界面

如果能出现图 46.5.3 所示界面，那么就说明 mincom 工作正常了。

46.6 RS232 驱动测试

46.6.1 RS232 连接设置

在测试之前要先将 STM32MP1 开发板的 RS232 接口与电脑连接起来，正点原子 STM32MP1 开发板上两个 RS232 接口如图 46.6.1.1 所示：

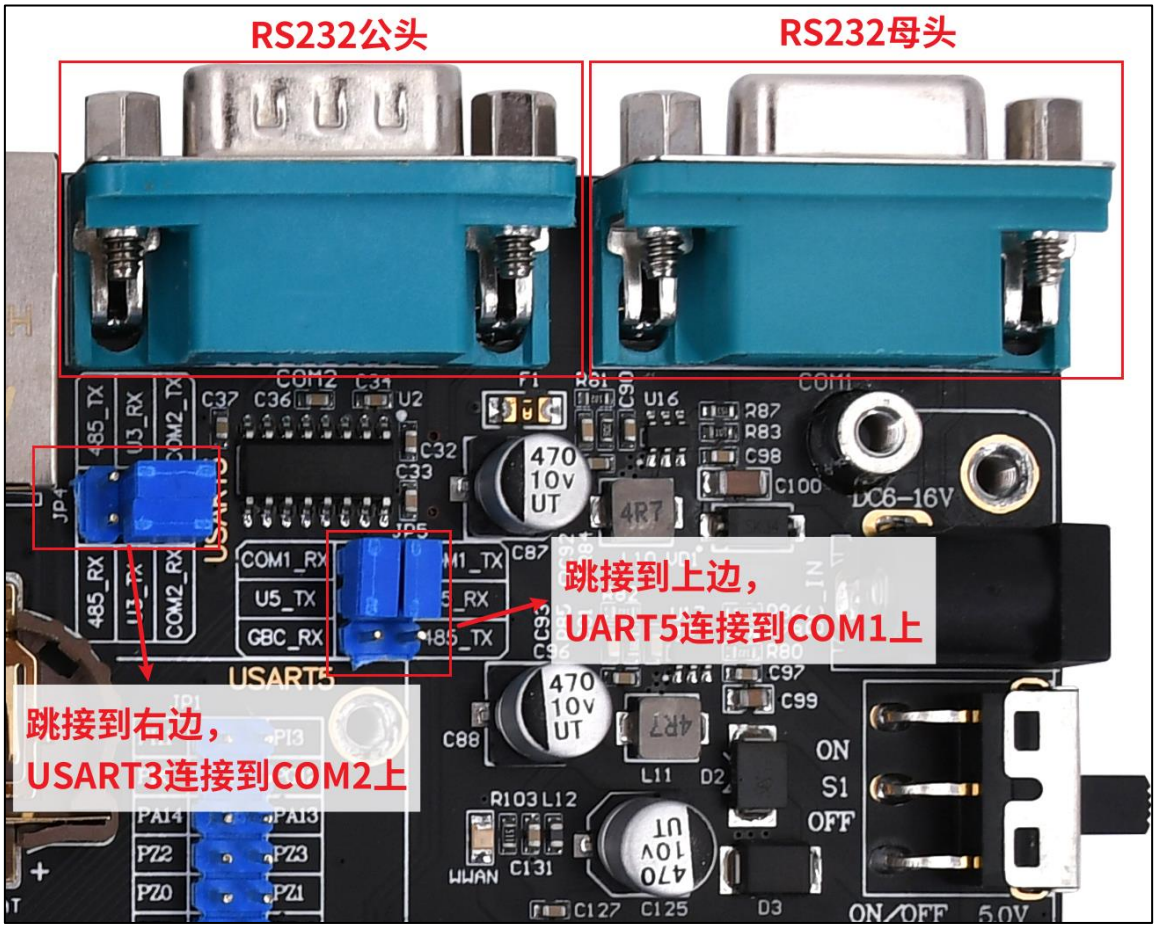


图 46.6.1.1 RS232 接口

从图 46.6.1.1 中可以看出，正点原子开发板上有 2 个 RS232 接口，这个在 46.2 小节讲解硬件原理图的时候都已经说过了。这里我们要注意的是这两个 RS232 接口一个为公头，一个为母头，方便大家外接自己的设备。图 46.6.1.1 中左边的 COM2 为公头，可以通过 JP4 跳接到 USART3 上。右边的 COM1 为母头，可以通过 JP5 跳接到 UART5 上。本实验我们使用右边的 COM1，所以我们需要将 JP5 的两个跳线帽接到上方，也就是将 UART5 与 COM1 连接起来。

跳线帽设置好以后使用 RS232 线将开发板与电脑连接起来，这里建议使用 USB 转 DB9(RS232)数据线，比如正点原子售卖的 CH340 方案的 USB 转 DB9 数据线，如图 46.6.1.2 所示：

图 46.6.1.2 USB 转 DB9 数据线

图 46.6.1.2 中所示的数据线是带有 CH340 芯片的，因此当连接到电脑以后就会出现一个 COM 口，这个 COM 口就是我们要使用的 COM 口。比如在我的电脑上就是 COM11，在 MobaXterm 上新建一个连接，串口为 COM11，波特率为 115200。

46.6.2 minicom 设置

在开发板中输入“minicom -s”，打开 minicom 配置界面，然后选中“Serial port setup”，如图 46.6.2.1 所示：

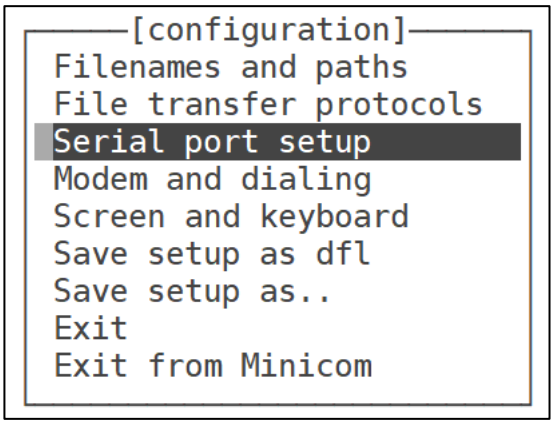


图 46.6.2.1 选中串口设置项

选中“Serial port setup”以后点击回车，进入设置菜单，如图 46.6.2.2 所示：

```
A -    Serial Device      : /dev/ttyS1
B - Lockfile Location     : /var/lock
C -   Callin Program      :
D -  Callout Program      :
E -    Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : Yes
G - Software Flow Control : No
H -     RS485 Enable      : No
I -   RS485 Rts On Send   : No
J -  RS485 Rts After Send : No
K -  RS485 Rx During Tx   : No
L -  RS485 Terminate Bus  : No
M - RS485 Delay Rts Before: 0
N - RS485 Delay Rts After : 0

   Change which setting?
```

图 46.6.2.2 串口设置项

图 46.6.2.2 中有 14 个设置项目，分别对应 A、B……N，比如第一个是选中串口，UART5 的串口文件为/dev/ttySTM1(因为我们的设备别名 serial1=&UART5)，因此串口设置要设置为/dev/ttySTM1。设置方法就是按下键盘上的‘A’，然后输入“/dev/ttySTM1”即可，如图 46.6.2.3 所示：

```
A -    Serial Device      : /dev/ttySTM1
B - Lockfile Location     : /var/lock
C -   Callin Program      :
D -  Callout Program      :
E -    Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : Yes
G - Software Flow Control : No
H -     RS485 Enable      : No
I -   RS485 Rts On Send   : No
J -  RS485 Rts After Send : No
K -  RS485 Rx During Tx   : No
L -  RS485 Terminate Bus  : No
M - RS485 Delay Rts Before: 0
N - RS485 Delay Rts After : 0

   Change which setting?
```

UART5对应的设备文件

图 46.6.2.3 串口设备文件设置

设置完以后按下回车键确认，确认完以后就可以设置其他的配置项。比如 E 设置波特率、数据位和停止位的、F 设置硬件流控的，设置方法都一样，设置完以后如图 46.6.2.4 所示：

```
A -    Serial Device      : /dev/ttySTM1
B - Lockfile Location     : /var/lock
C -   Callin Program      :
D -  Callout Program      :
E -    Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : No
G - Software Flow Control : No
H -     RS485 Enable      : No
I -   RS485 Rts On Send   : No
J -  RS485 Rts After Send : No
K -  RS485 Rx During Tx   : No
L -  RS485 Terminate Bus  : No
M - RS485 Delay Rts Before: 0
N - RS485 Delay Rts After : 0

   Change which setting?
```

波特率设置115200、8位数据位、1位停止位，关闭硬件和软件流控

图 46.6.2.4 UART5 设置

都设置完成以后按下回车键确认并退出，这时候会退回到如图 46.6.2.1 所示的界面，按下 ESC 键退出图 46.6.2.1 所示的配置界面，退出以后如图 46.6.2.5 所示：

```
Welcome to minicom 2.7.90

OPTIONS:
Compiled on Jun 17 2021, 17:39:35.
Port /dev/ttySTM1, 00:00:02

Press CTRL-A Z for help on special keys
```

图 46.6.2.5 minicom 串口界面

图 46.6.2.2 就是我们的串口调试界面，可以看出当前的串口文件为/dev/ttySTM1，按下 CTRL-A，然后再按下 Z 就可以打开 minicom 帮助信息界面，如图 46.6.2.6 所示：

```
                    Minicom Command Summary

                Commands can be called by CTRL-A <key>

                 Main Functions                  Other Functions

 Dialing directory..D  run script (Go)....G | Clear Screen.......C
 Send files.........S  Receive files......R | cOnfigure Minicom..O
 comm Parameters....P  Add linefeed.......A | Suspend minicom....J
 Capture on/off.....L  Hangup.............H | eXit and reset.....X
 send break.........F  initialize Modem...M | Quit with no reset.Q
 Terminal settings..T  run Kermit.........K | Cursor key mode....I
 lineWrap on/off....W  local Echo on/off..E | Help screen........Z
 Paste file.........Y  Timestamp toggle...N | scroll Back........B
 Add Carriage Ret...U

              Select function or press Enter for none.
```

图 46.6.2.6 minicom 帮助信息界面

从图 46.6.2.6 可以看出，minicom 有很多快捷键，本实验我们打开 minicom 的回显功能，回显功能配置项为“local Echo on/off..E”，因此按下 E 即可打开/关闭回显功能。

46.6.3 RS232 收发测试

1、发送测试

首先测试开发板通过 UART5 向电脑发送数据的功能，需要打开 minicom 的回显功能(不打开也可以，但是在 minicom 中看不到自己输入的内容)，回显功能打开以后输入“AAAA”，如图 46.6.3.1 所示：

图 46.6.3.1 通过 UART5 向电脑发送“AAAA”

图 46.6.3.1 中的“AAAA”就是开发板通过 UART5 向电脑发送的数据，那么电脑的 COM11 就会接收到“AAAA”，MobaXterm 中 COM11 收到的数据如图 46.6.3.2 所示：

图 46.6.3.2 电脑接收到开发板发送过来的数据

可以看出，开发板通过 UART3 向电脑发送数据正常，那么接下来就测试开发板数据接收功能。

2、接收测试

接下来测试开发板的 UART5 接收功能，同样的，要先打开 MobaXterm 上 COM11 的本地回显，由于笔者没有找到如何打开 MobaXterm 的回显，所以当我们在 COM11 上输入字符以后并不会显示出来，不利于我们观察，但是开发板是可以接收到我们在 COM11 上输入的字符。比如，这里我们输入‘123456’，此时开发板接收到的数据如图 46.6.3.3 所示：

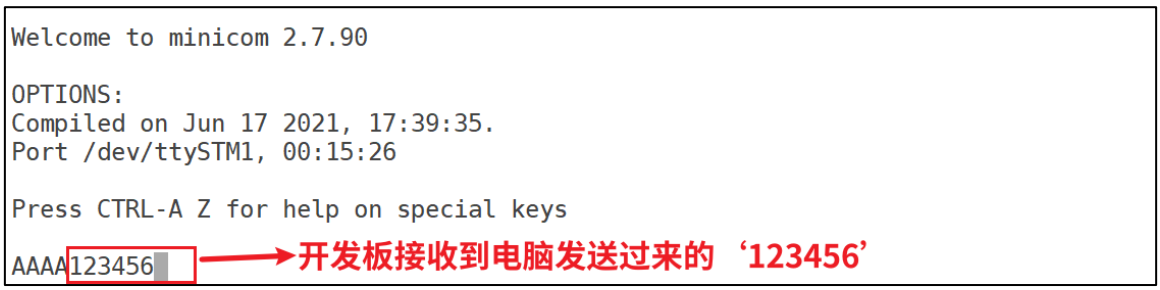


图 46.6.3.3 开发板接收到发送过来的数据

UART5 收发测试都没有问题，说明我们的 UART5 驱动工作正常。如果要退出 minicom 的话，在 minicom 通信界面按下 CRTL+A，然后按下 X 来关闭 minicom。关于 minicom 的使用我们这里讲的很简单，大家可以在网上查找更加详细的 minicom 使用教程。

46.7 RS485 测试

前面已经说过了，STM32MP1 开发板上的 RS485 接口连接到了 USART3 上，因此本质上就是个串口。RS232 实验我们已经将 USART3 的驱动编写好了，所以 RS485 实验就不需要编写任何驱动程序，可以直接使用 minicom 来进行测试。

46.7.1 RS485 连接设置

首先是设置 JP4 跳线帽，将 1-3、2-4 连接起来，RS485 接口如图 46.7.1.1 所示：

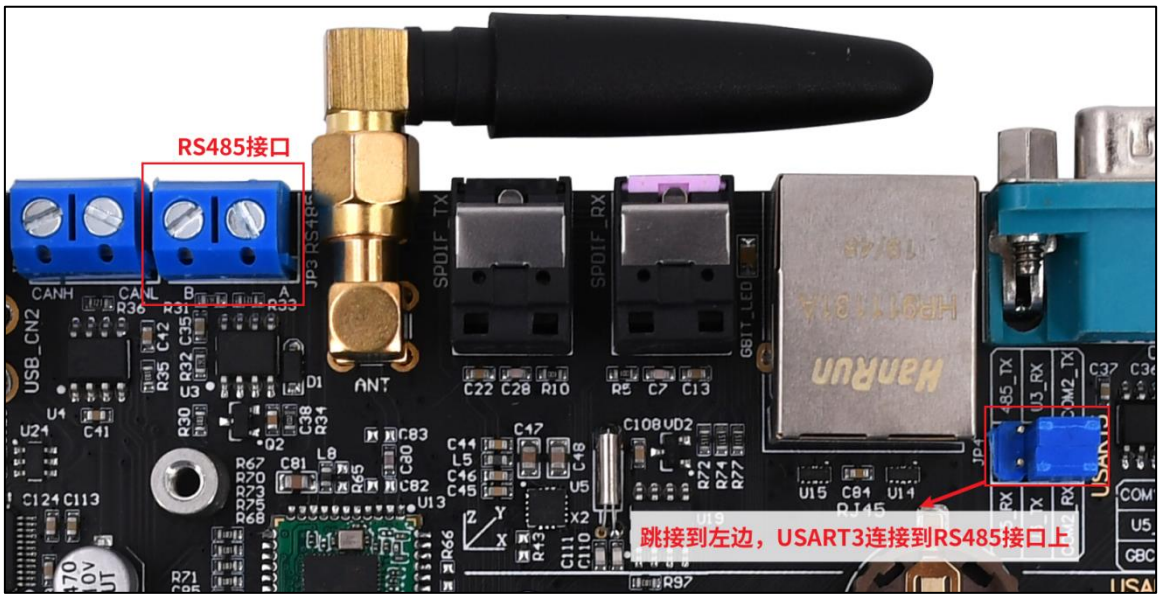


图 46.7.1.1 RS485 接口设置

一个板子是不能进行 RS485 通信测试的，还需要另一个 RS485 设备，比如另外一块 STM32MP1 开发板。这里推荐大家使用正点原子出品的 USB 三合一串口转换器，支持 USB 转 TTL、RS232 和 RS485，如图 46.7.1.2 所示：

图 46.7.1.2 正点原子 USB 三合一串口转换器

使用杜邦线将 USB 串口转换器的 RS485 接口和 STM32MP157 开发板的 RS485 连接起来，A 接 A，B 接 B，不能接错了！连接完成以后如图 46.7.1.3 所示：

图 46.7.1.3 串口转换器和开发板 RS485 连接示意图

串口转换器通过 USB 线连接到电脑上，我用的是 CH340 版本的，因此就不需要安装驱动的，如果使用的是 FT232 版本的就需要安装相应的驱动。连接成功以后电脑就会有相应的 COM 口，比如我的电脑上就是 COM6，接下来就是测试。

46.7.2 RS485 收发测试

RS485 的测试和 RS232 一模一样！我电脑上 USB 多合一转换器对应 COM12。因为 MobaXterm 没有找到回显设置，因此这里为了方便观察，USB 多合一转换器使用 SecureCRT 这个终端软件。使用 SecureCRT 创建一个 COM12 的连接，开发板使用 USART3，对应的串口设备文件为/dev/ttySTM2，因此开发板使用 minicom 创建一个/dev/ttySTM2 的串口连接。串口波特率都选择 115200，8 位数据位，1 位停止位，关闭硬件和软件流控。

1、RS485 发送测试

首先测试开发板通过 RS485 发送数据，设置好 minicom 以后，同样输入“AAAA”，也就是通过 RS485 向电脑发送一串“AAAA”。如果 RS485 驱动工作正常的话，那么电脑就会介绍到开发板发送过来的“AAAA”，如图 46.7.2.1 所示：

图 46.7.2.1 RS485 数据发送测试

从图 46.7.2.1 可以看出开发板通过 RS485 向电脑发送“AAAA”成功，说明 RS485 数据数

据发送正常。

2、RS485 接收测试

接下来测试一下 RS485 数据接收，电脑通过 RS485 向开发板发送“BBBB”，然后观察 minicom 是否能接收到“BBBB”。结果如图 46.7.2.2 所示：

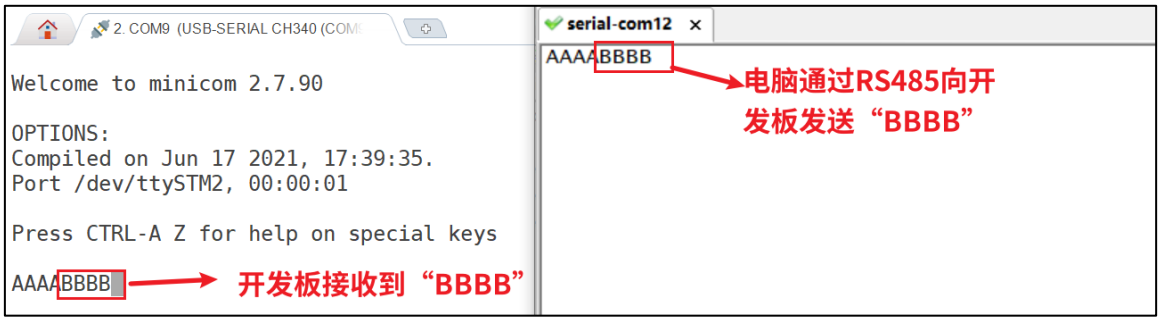


图 46.7.2.2 RS485 数据接收测试

从图 46.7.2.1 可以看出开发板接收到电脑通过 RS485 发送过来的“BBBB”，说明 RS485 数据接收也正常。

46.8 GPS 测试

46.8.1 GPS 连接设置

GPS 模块大部分都是串口输出的，这里以正点原子出品的 ATK1218-BD 模块为例，这是一款 GSP+北斗的定位模块，模块如图 46.8.1.1 所示：

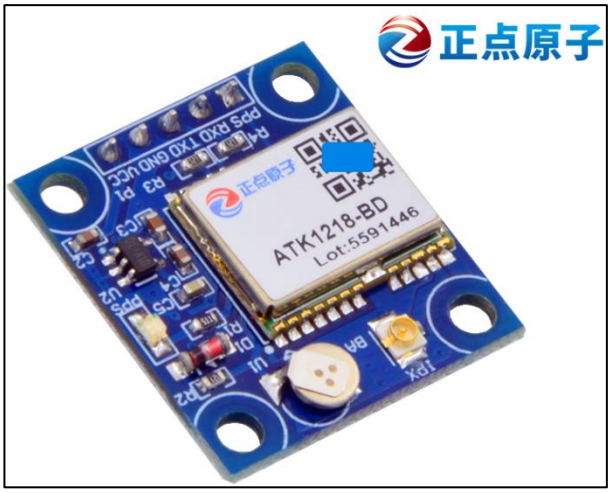


图 46.8.1.1 正点原子 ATK1218-BD 定位模块

首先要设置 STM32MP1 开发板上的 JP5 跳线帽，将 UART5 与 ATK 模块接口上的串口连接起来，如图 46.8.1.2 所示：

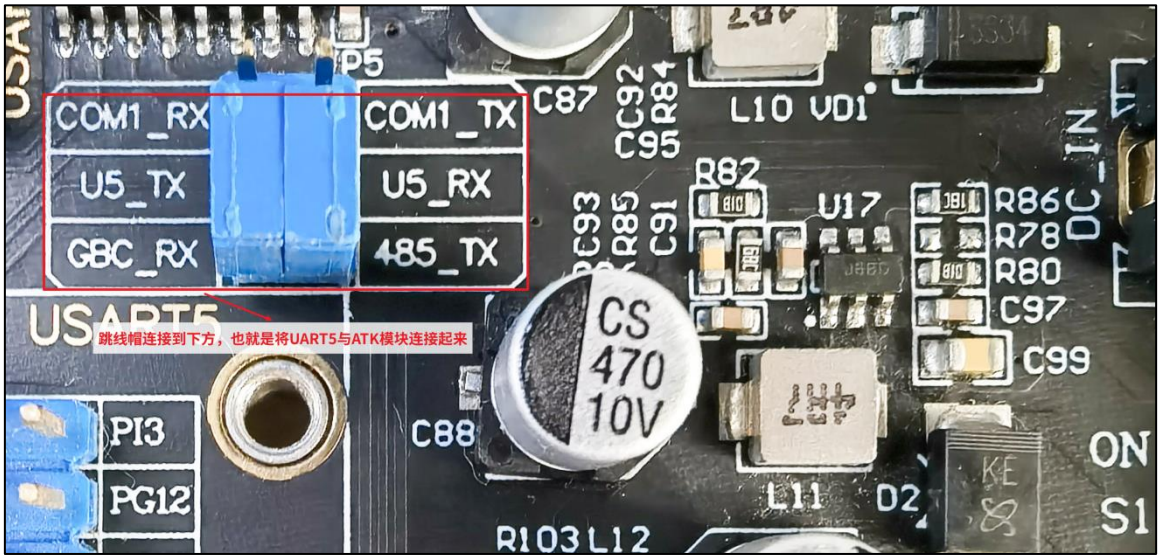


图 46.8.1.2 UART5 跳线帽接到 GPS 上

此时 UART5_TX 和 UART5_RX 已经连接到了开发板上的 ATK MODULE 上，直接将 ATK1218-BD 模块插到开发板上的 ATK MODULE 接口即可，开发板上的 ATK MODULE 接口是 6 脚的，而 ATK1218-BD 模块是 5 脚的，因此需要靠下插（VCC 对应 5V）！然后 GPS 需要接上天线，天线的接收头一定要放到户外，因此室内一般是没有 GPS 信号的。连接完成以后如图 46.8.1.3 所示：

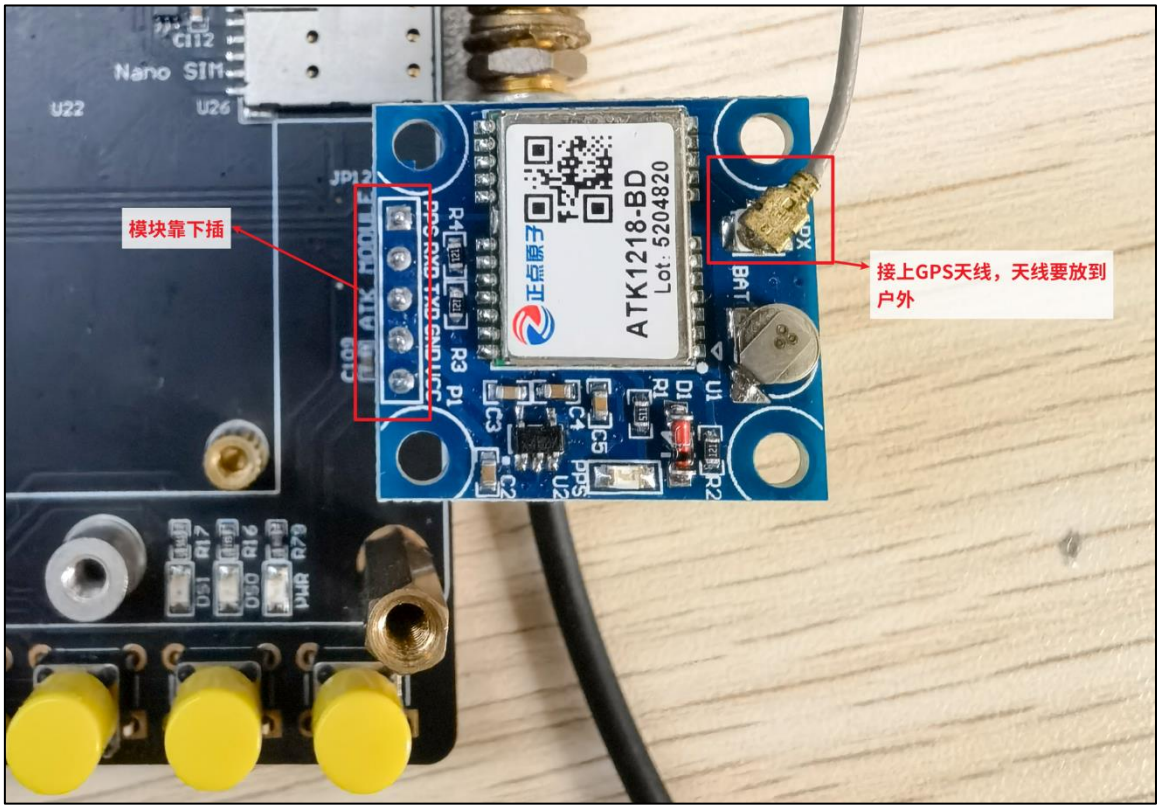


图 46.8.1.3 GPS 模块连接示意图

46.8.2 GPS 数据接收测试

GPS 都是被动接收定位数据的，模块接收定位卫星数据，然后计算出位置信息通过串口输出。所以我们要先设置 minicom，UART5 对应/dev/ttySTM1，串口设置要求如下：

①、波特率设置为 38400，因为正点原子的 ATK1218-BD 模块默认波特率就是 38400。

②、8 位数据位，1 位停止位。

③、关闭硬件和软件流控。

设置好以后如图 46.8.2.1 所示：

```
A -    Serial Device      : /dev/ttySTM1
B - Lockfile Location     : /var/lock
C -   Callin Program      :
D -  Callout Program      :
E -    Bps/Par/Bits       : 38400 8N1
F - Hardware Flow Control : No
G - Software Flow Control : No
H -     RS485 Enable      : No
I -   RS485 Rts On Send   : No
J -  RS485 Rts After Send : No
K -  RS485 Rx During Tx   : No
L -  RS485 Terminate Bus  : No
M - RS485 Delay Rts Before: 0
N - RS485 Delay Rts After : 0

   Change which setting?
```

图 46.8.2.1 串口设置

设置好以后就可以静静的等待 GPS 数据输出，GPS 模块第一次启动可能需要几分钟搜星，等搜到卫星以后才会有定位数据输出。搜到卫星以后 GPS 模块输出的定位数据如图 46.8.2.2 所示：

```
Welcome to minicom 2.7.90

OPTIONS:
Compiled on Jun 17 2021, 17:39:35.
Port /dev/ttySTM1, 00:00:02

Press CTRL-A Z for help on special keys

L▒b▒▒b▒▒bbbbbb▒r▒b▒r▒b▒r▒R▒▒
                              $GNGSA,A,3,210,212,224,240,,,,,,,,,3.2,2.7,1.8*21
$GPGSV,3,1,10,14,78,355,38,28,59,336,39,17,45,339,44,03,41,078,*75
$GPGSV,3,2,10,06,34,241,,30,33,211,,19,31,308,38,01,25,036,31*75
$GPGSV,3,3,10,22,25,053,40,21,07,044,26*7D
$BDGSV,3,1,10,208,80,253,,238,77,239,,210,70,341,33,240,69,029,36*69
$BDGSV,3,2,10,244,50,331,19,224,47,287,38,212,38,297,34,245,10,162,30*62
$BDGSV,3,3,10,256,00,000,25,257,00,000,40*6B
$GNRMC,035305.400,A,2318.12982,N,11319.74238,E,000.0,000.0,190621,,,A*70
$GNVTG,000.0,T,,M,000.0,N,000.0,K,A*13
$GNZDA,035305.400,19,06,2021,00,00*43
$GNGGA,035305.600,2318.12982,N,11319.74238,E,1,11,2.7,32.6,M,-5.4,M,,0000*63
$GNRMC,035305.600,A,2318.12982,N,11319.74238,E,000.0,000.0,190621,,,A*72
$GNGGA,035305.800,2318.12982,N,11319.74238,E,1,11,2.7,32.6,M,-5.4,M,,0000*6D
$GNGGA,035323.600,2318.12982,N,11319.74238,E,1,11,2.7,32.6,M,-5.4,M,,0000*67
```

GPS模块输出定位数据

图 46.8.2.2 GPS 数据

第四十七章 多点电容触摸屏实验

随着智能手机的发展，电容触摸屏也得到了飞速的发展。相比电阻触摸屏，电容触摸屏有很多的优势，比如支持多点触控、不需要按压，只需要轻轻触摸就有反应。ALIENTEK 的三款 RGB LCD 屏幕都支持多点电容触摸，本章就以 ATK7016 这款 RGB LCD 屏幕为例讲解一下如何驱动电容触摸屏，并获取对应的触摸坐标值。

47.1 多点电容触摸简介

触摸屏很早就有了，一开始是电阻触摸屏，电阻触摸屏只能单点触摸，在以前的学习机、功能机时代被广泛使用。2007 年 1 月 9 日苹果发布了划时代的第一代 Iphone，也就是 Iphone 2G，Iphone 2G 上使用了多点电容触摸屏，而当时的手机基本都是使用的电阻触摸屏。电容触摸屏优秀的触摸品质和手感瞬间征服了消费者，带来了手机触摸屏的大变革，后面新出的手机也都采用了多点电容触摸屏。和电阻触摸屏相比，电容触摸屏最大的优点是支持多点触摸(后面的电阻屏也支持多点触摸，但是为时已晚)，电容屏只需要手指轻触即可，而电阻屏是需要手指给予一定的压力才有反应，而且电容屏不需要校准。如今多点电容触摸屏已经得到了广泛的应用，手机、平板、电脑、广告机等等，如果要做人机交互设备的开发，多点电容触摸屏基本是不可能绕过去的。所以本章我们就来学习一下如何使用多点触摸屏，如何获取到多点触摸值。关于电容屏的物理原理我们就不去研究了，毕竟我们不是开发电容屏的，而是电容屏的使用者，我们只需要关注如何使用电容屏，如何得到其多点触摸坐标值即可。ALIENTEK 的三款 RGB LCD 屏幕都是支持 5 点电容触摸屏的，本章我们同样以 ATK-7016 这款屏幕为例来讲解如何使用多点电容触摸屏。

ATK-7016 这款屏幕其实是由 TFT LCD+触摸屏组合起来的。底下是 LCD 面板，上面是触摸面板，将两个封装到一起就成了带有触摸屏的 LCD 屏幕。电容触摸屏也是需要一个驱动 IC 的，驱动 IC 一般会提供一个 I2C 接口给主控制器，主控制器可以通过 I2C 接口来读取驱动 IC 里面的触摸坐标数据。ATK-7016、ATK-7084 这两款屏幕使用的触摸控制 IC 是 FT5426，ATK-4342 使用的驱动 IC 是 GT9147。这三个电容屏触摸 IC 都是 I2C 接口的，使用方法基本一样。

FT5426 这款驱动 IC 采用 15*28 的驱动结构，也就是 15 个感应通道，28 个驱动通道，最多支持 5 点电容触摸。ATK-7016 的电容触摸屏部分有 4 个 IO 用于连接主控制器：SCL、SDA、RST 和 INT，SCL 和 SDA 是 I2C 引脚，RST 是复位引脚，INT 是中断引脚。一般通过 INT 引脚来通知主控制器有触摸点按下，然后在 INT 中断服务函数中读取触摸数据。也可以不使用中断功能，采用轮询的方式不断查询是否有触摸点按下，本章实验我们使用中断方式来获取触摸数据。

跟所有的 I2C 器件一样，FT5426 也是通过读写寄存器来完成初始化和触摸坐标数据读取的，STM32MP1 的 I2C 我们已经在第四十章做了详细的讲解，所以本章的主要工作就是读写 FT5426 的寄存器。FT5426 的 I2C 设备地址为 0X38，FT5426 的寄存器有很多，本章我们只用到了其中的一部分，如表 47.1.1 所示：

寄存器地址	位	寄存器功能	描述
0X00	[6:4]	模式寄存器	设置 FT5426 的工作模式： 000：正常模式。 001：系统信息模式 100：测试模式。
0X02	[3:0]	触摸状态寄存器	记录有多少个触摸点， 有效值为 1~5。
0X03	[7:6]	第一个触摸点 X 坐标高位数据	事件标志： 00：按下。 01：抬起 10：接触 11：保留
	[3:0]		X 轴坐标值高 4 位。

0X04	[7:0]	第一个触摸点 X 坐标低位数据	X 轴坐标值低 8 位
0X05	[7:4]	第一个触摸点 Y 坐标高位数据	触摸点的 ID。
	[3:0]		Y 轴坐标高 4 位
0X06	[7:0]	第一个触摸点 Y 坐标低位数据	Y 轴坐标低 8 位
0X09	[7:6]	第二个触摸点 X 坐标高位数据	与寄存器 0X03 含义相同。
	[3:0]		
0X0A	[7:0]	第二个触摸点 X 坐标低位数据	与寄存器 0X04 含义相同。
0X0B	[7:4]	第二个触摸点 Y 坐标高位数据	与寄存器 0X05 含义相同。
	[3:0]		
0X0C	[7:0]	第二个触摸点 Y 坐标低位数据	与寄存器 0X06 含义相同
0X0F	[7:6]	第三个触摸点 X 坐标高位数据	与寄存器 0X03 含义相同。
	[3:0]		
0X10	[7:0]	第三个触摸点 X 坐标低位数据	与寄存器 0X04 含义相同。
0X11	[7:4]	第三个触摸点 Y 坐标高位数据	与寄存器 0X05 含义相同。
	[3:0]		
0X12	[7:0]	第三个触摸点 Y 坐标低位数据	与寄存器 0X06 含义相同
0X15	[7:6]	第四个触摸点 X 坐标高位数据	与寄存器 0X03 含义相同。
	[3:0]		
0X16	[7:0]	第四个触摸点 X 坐标低位数据	与寄存器 0X04 含义相同。
0X17	[7:4]	第四个触摸点 Y 坐标高位数据	与寄存器 0X05 含义相同。
	[3:0]		
0X18	[7:0]	第四个触摸点 Y 坐标低位数据	与寄存器 0X06 含义相同
0X1B	[7:6]	第五个触摸点 X 坐标高位数据	与寄存器 0X03 含义相同。
	[3:0]		
0X1C	[7:0]	第五个触摸点 X 坐标低位数据	与寄存器 0X04 含义相同。
0X1D	[7:4]	第五个触摸点 Y 坐标高位数据	与寄存器 0X05 含义相同。
	[3:0]		
0X1E	[7:0]	第五个触摸点 Y 坐标低位数据	与寄存器 0X06 含义相同
0XA1	[7:0]	版本寄存器	版本高字节
0XA2	[7:0]		版本低字节
0XA4	[7:0]	中断模式寄存器	用于设置中断模式： 0：轮询模式 1：触发模式

表 47.1.1 FT5426 使用到的寄存器表

表 47.1.1 中就是本章实验我们会使用到的寄存器。关于触摸屏和 FT5426 的知识就讲解到这里。

47.2 Linux 下电容触摸屏驱动框架简介

47.2.1 多点触摸(MT)协议详解

电容触摸驱动的基本原理进行了详细的讲解，回顾一下几个重要的知识点：

①、电容触摸屏是 IIC 接口的，需要触摸 IC，以正点原子的 ATK7016 为例，其所使用的触

摸屏控制 IC 为 FT5426，因此所谓的电容触摸驱动就是 IIC 设备驱动。

②、触摸 IC 提供了中断信号引脚(INT)，可以通过中断来获取触摸信息。

③、电容触摸屏得到的是触摸位置绝对信息以及触摸屏是否有按下。

④、电容触摸屏不需要校准，当然了，这只是理论上的，如果电容触摸屏质量比较差，或者触摸玻璃和 TFT 之间没有完全对齐，那么也是需要校准的。

根据以上几个知识点，我们可以得出电容触摸屏驱动其实就是以下几种 linux 驱动框架的组合：

①、IIC 设备驱动，因为电容触摸 IC 基本都是 IIC 接口的，因此大框架就是 IIC 设备驱动。

②、通过中断引脚(INT)向 linux 内核上报触摸信息，因此需要用到 linux 中断驱动框架。坐标的上报在中断服务函数中完成。

③、触摸屏的坐标信息、屏幕按下和抬起信息都属于 linux 的 input 子系统，因此向 linux 内核上报触摸屏坐标信息就得使用 input 子系统。只是，我们得按照 linux 内核规定的规则来上报坐标信息。

经过简单的分析，我们发现 IIC 驱动、中断驱动、input 子系统我们都已经在前面学过了，唯独没学过的就是 input 子系统下的多点电容触摸协议，这个才是我们本章学习的重点。linux 内核中有一份文档详细的讲解了多点电容触摸屏协议，文档路径为：Documentation/input/multi-touch-protocol.txt。

老版本的 linux 内核是不支持多点电容触摸的(Multi-touch，简称 MT)，MT 协议是后面加入的，因此如果使用 2.x 版本 linux 内核的话可能找不到 MT 协议。MT 协议被分为两种类型，Type A 和 TypeB，这两种类型的区别如下：

Type A：适用于触摸点不能被区分或者追踪，此类型的设备上报原始数据(此类型在实际使用中非常少！)。

Type B：适用于有硬件追踪并能区分触摸点的触摸设备，此类型设备通过 slot 更新某一个触摸点的信息，FT5426 就属于此类型，一般的多点电容触摸屏 IC 都有此能力。

触摸点的信息通过一系列的 ABS_MT 事件(有的资料也叫消息)上报给 linux 内核，只有 ABS_MT 事件是用于多点触摸的，ABS_MT 事件定义在文件 include/uapi/linux/input-event-codes.h 中，相关事件如下所示：

示例代码 47.2.1.1 ABS_MT 事件

```
769 #define ABS_MT_SLOT         0x2f /* MT slot being modified */
770 #define ABS_MT_TOUCH_MAJOR  0x30 /*Major axis of touching ellipse*/
771 #define ABS_MT_TOUCH_MINOR  0x31 /* Minor axis (omit if circular) */
772 #define ABS_MT_WIDTH_MAJOR  0x32 //Major axis of approaching ellipse
773 #define ABS_MT_WIDTH_MINOR  0x33 /* Minor axis (omit if circular) */
774 #define ABS_MT_ORIENTATION  0x34 /* Ellipse orientation */
775 #define ABS_MT_POSITION_X   0x35 /* Center X touch position */
776 #define ABS_MT_POSITION_Y   0x36 /* Center Y touch position */
777 #define ABS_MT_TOOL_TYPE    0x37 /* Type of touching device */
778 #define ABS_MT_BLOB_ID      0x38 /*Group a set of packets as a blob*/
779 #define ABS_MT_TRACKING_ID  0x39 /*Unique ID of initiated contact */
780 #define ABS_MT_PRESSURE     0x3a /* Pressure on contact area */
781 #define ABS_MT_DISTANCE     0x3b /* Contact hover distance */
782 #define ABS_MT_TOOL_X       0x3c /* Center X tool position */
783 #define ABS_MT_TOOL_Y       0x3d /* Center Y tool position */
```

在上面这些众多的 ABS_MT 事件中，我们最常用的就是 ABS_MT_SLOT、ABS_MT_POSITION_X、ABS_MT_POSITION_Y 和 ABS_MT_TRACKING_ID。其中 ABS_MT_POSITION_X 和 ABS_MT_POSITION_Y 用来上报触摸点的(X,Y)坐标信息，ABS_MT_SLOT 用来上报触摸点 ID，对于 Type B 类型的设备，需要用到 ABS_MT_TRACKING_ID 事件来区分触摸点。

对于 Type A 类型的设备，通过 input_mt_sync()函数来隔离不同的触摸点数据信息，此函数原型如下所示：

```
void input_mt_sync(struct input_dev *dev)
```

此函数只要一个参数，类型为 input_dev，用于指定具体的 input_dev 设备。input_mt_sync()函数会触发 SYN_MT_REPORT 事件，此事件会通知接收者获取当前触摸数据，并且准备接收下一个触摸点数据。

对于 Type B 类型的设备，上报触摸点信息的时候需要通过 input_mt_slot()函数区分是哪一个触摸点，input_mt_slot()函数原型如下所示：

```
void input_mt_slot(struct input_dev *dev, int slot)
```

此函数有两个参数，第一个参数是 input_dev 设备，第二个参数 slot 用于指定当前上报的是哪个触摸点信息。input_mt_slot()函数会触发 ABS_MT_SLOT 事件，此事件会告诉接收者当前正在更新的是哪个触摸点(slot)的数据。

不管是哪个类型的设备，最终都要调用 input_sync()函数来标识多点触摸信息传输完成，告诉接收者处理之前累计的所有消息，并且准备好下一次接收。Type B 和 Type A 相比最大的区别就是 Type B 可以区分出触摸点， 因此可以减少发送到用户空间的数据。Type B 使用 slot 协议区分具体的触摸点，slot 需要用到 ABS_MT_TRACKING_ID 消息，这个 ID 需要硬件提供，或者通过原始数据计算出来。对于 Type A 设备，内核驱动需要一次性将触摸屏上所有的触摸点信息全部上报，每个触摸点的信息在本次上报事件流中的顺序不重要，因为事件的过滤和手指(触摸点)跟踪是在内核空间处理的。

Type B 设备驱动需要给每个识别出来的触摸点分配一个 slot，后面使用这个 slot 来上报触摸点信息。可以通过 slot 的 ABS_MT_TRACKING_ID 来新增、替换或删除触摸点。一个非负数的 ID 表示一个有效的触摸点，-1 这个 ID 表示未使用 slot。一个以前不存在的 ID 表示这是一个新加的触摸点，一个 ID 如果再也不存在了就表示删除了。

有些设备识别或追踪的触摸点信息要比他上报的多，这些设备驱动应该给硬件上报的每个触摸点分配一个 Type B 的 slot。一旦检测到某一个 slot 关联的触摸点 ID 发生了变化，驱动就应该改变这个 slot 的 ABS_MT_TRACKING_ID，使这个 slot 失效。如果硬件设备追踪到了比他正在上报的还要多的触摸点，那么驱动程序应该发送 BTN_TOOL_*TAP 消息，并且调用 input_mt_report_pointer_emulation()函数，将此函数的第二个参数 use_count 设置为 false。

47.2.2 Type A 触摸点信息上报时序

对于 Type A 类型的设备，发送触摸点信息的时序如下所示，这里以 2 个触摸点为例：

示例代码 47.2.2.1 Type A 触摸点数据上报时序

```
1 ABS_MT_POSITION_X x[0]
2 ABS_MT_POSITION_Y y[0]
3 SYN_MT_REPORT
4 ABS_MT_POSITION_X x[1]
5 ABS_MT_POSITION_Y y[1]
6 SYN_MT_REPORT
```

```
7 SYN_REPORT
```

第 1 行，通过 ABS_MT_POSITION_X 事件上报第一个触摸点的 X 坐标数据，通过 input_report_abs 函数实现，下面同理。

第 2 行，通过 ABS_MT_POSITION_Y 事件上报第一个触摸点的 Y 坐标数据。

第 3 行，上报 SYN_MT_REPORT 事件，通过调用 input_mt_sync 函数来实现。

第 4 行，通过 ABS_MT_POSITION_X 事件上报第二个触摸点的 X 坐标数据。

第 5 行，通过 ABS_MT_POSITION_Y 事件上报第二个触摸点的 Y 坐标数据。

第 6 行，上报 SYN_MT_REPORT 事件，通过调用 input_mt_sync 函数来实现。

第 7 行，上报 SYN_REPORT 事件，通过调用 input_sync 函数实现。

我们在编写 Type A 类型的多点触摸驱动的时候就需要按照示例代码 47.2.2.1 中的时序上报坐标信息。Linux 内核里面也有 Type A 类型的多点触摸驱动，找到 st1232.c 这个驱动文件，路径为 drivers/input/touchscreen/st1232.c，找到 st1232_ts_irq_handler 函数，此函数里面就是上报触摸点坐标信息的。

示例代码 47.2.2.2 st1232_ts_irq_handler 函数代码段

```
103 static irqreturn_t st1232_ts_irq_handler(int irq, void *dev_id)
104 {
......
111     ret = st1232_ts_read_data(ts);
112     if (ret < 0)
113         goto end;
114
115     /* multi touch protocol */
116     for (i = 0; i < MAX_FINGERS; i++) {
117         if (!finger[i].is_valid)
118             continue;
119
120         input_report_abs(input_dev, ABS_MT_TOUCH_MAJOR, finger[i].t);
121         input_report_abs(input_dev, ABS_MT_POSITION_X, finger[i].x);
122         input_report_abs(input_dev, ABS_MT_POSITION_Y, finger[i].y);
123         input_mt_sync(input_dev);
124         count++;
125     }
......
140
141     /* SYN_REPORT */
142     input_sync(input_dev);
143
144 end:
145     return IRQ_HANDLED;
146 }
```

第 111 行，获取所有触摸点信息。

第 116~125 行，按照 Type A 类型轮流上报所有的触摸点坐标信息，第 121 和 122 行分别上报触摸点的(X,Y)轴坐标，也就是 ABS_MT_POSITION_X 和 ABS_MT_POSITION_Y 事件。每

上报完一个触摸点坐标，都要在第 123 行调用 input_mt_sync 函数上报一个 SYN_MT_REPORT 信息。

第 142 行，每上报完一轮触摸点信息就调用一次 input_sync 函数，也就是发送一个 SYN_REPORT 事件

47.2.3 Type B 触摸点信息上报时序

对于 Type B 类型的设备，发送触摸点信息的时序如下所示，这里以 2 个触摸点为例：

示例代码 47.2.3.1 Type B 触摸点数据上报时序

```
1 ABS_MT_SLOT 0
2 ABS_MT_TRACKING_ID 45
3 ABS_MT_POSITION_X x[0]
4 ABS_MT_POSITION_Y y[0]
5 ABS_MT_SLOT 1
6 ABS_MT_TRACKING_ID 46
7 ABS_MT_POSITION_X x[1]
8 ABS_MT_POSITION_Y y[1]
9 SYN_REPORT
```

第 1 行，上报 ABS_MT_SLOT 事件，也就是触摸点对应的 SLOT。每次上报一个触摸点坐标之前要先使用 input_mt_slot 函数上报当前触摸点 SLOT，触摸点的 SLOT 其实就是触摸点 ID，需要由触摸 IC 提供。

第 2 行，根据 Type B 的要求，每个 SLOT 必须关联一个 ABS_MT_TRACKING_ID，通过修改 SLOT 关联的 ABS_MT_TRACKING_ID 来完成对触摸点的添加、替换或删除。具体用到的函数就是 input_mt_report_slot_state，如果是添加一个新的触摸点，那么此函数的第三个参数 active 要设置为 true，linux 内核会自动分配一个 ABS_MT_TRACKING_ID 值，不需要用户去指定具体的 ABS_MT_TRACKING_ID 值。

第 3 行，上报触摸点 0 的 X 轴坐标，使用函数 input_report_abs 来完成。

第 4 行，上报触摸点 0 的 Y 轴坐标，使用函数 input_report_abs 来完成。

第5~8 行，和第1~4 行类似，只是换成了上报触摸点1的(X,Y)坐标信息

第 9 行，当所有的触摸点坐标都上传完毕以后就得发送 SYN_REPORT 事件，使用 input_sync 函数来完成。

当一个触摸点移除以后，同样需要通过 SLOT 关联的 ABS_MT_TRACKING_ID 来处理，时序如下所示：

示例代码 47.2.3.2 Type B 触摸点移除时序

```
1 ABS_MT_TRACKING_ID -1
2 SYN_REPORT
```

第 1 行，当一个触摸点(SLOT)移除以后，需要通过 ABS_MT_TRACKING_ID 事件发送一个-1 给内核。方法很简单，同样使用 input_mt_report_slot_state 函数来完成，只需要将此函数的第三个参数 active 设置为 false 即可，不需要用户手动去设置-1。

第 2 行，当所有的触摸点坐标都上传完毕以后就得发送 SYN_REPORT 事件。

当要编写 Type B 类型的多点触摸驱动的时候就需要按照示例代码 47.2.3.1 中的时序上报坐标信息。Linux 内核里面有大量的 Type B 类型的多点触摸驱动程序，我们可以参考这些现成的驱动程序来编写自己的驱动代码。这里就以 ili210x 这个触摸驱动 IC 为例，看看是 Type B 类型是如何上报触摸点坐标信息的。找到 ili210x.c 这个驱动文件，路径为

drivers/input/touchscreen/ili210x.c，找到 ili210x_report_events 函数，此函数就是用于上报 ili210x 触摸坐标信息的，函数内容如下所示：

示例代码 47.2.3.3 ili210x_report_events 函数代码段

```
138 static bool ili210x_report_events(struct ili210x *priv,
                                       u8 *touchdata)
146 {
154     struct input_dev *input = priv->input;
162     int i;
170     bool contact = false, touch = false;
178     unsigned int x = 0, y = 0;
186
194     for (i = 0; i < priv->max_touches; i++) {
202         if (priv->model == MODEL_ILI210X) {
210             touch = ili210x_touchdata_to_coords(priv, touchdata,
218                                 i, &x, &y);
226         } else if (priv->model == MODEL_ILI251X) {
234             touch = ili251x_touchdata_to_coords(priv, touchdata,
242                                 i, &x, &y);
250             if (touch)
258                 contact = true;
266         }
274
282         input_mt_slot(input, i);
290         input_mt_report_slot_state(input, MT_TOOL_FINGER, touch);
298         if (!touch)
306             continue;
314         touchscreen_report_pos(input, &priv->prop, x, y,
322                        true);
330     }
338
346     input_mt_report_pointer_emulation(input, false);
354     input_sync(input);
362
370     if (priv->model == MODEL_ILI210X)
378         contact = touchdata[0] & 0xf3;
386
394     return contact;
402 }
```

第 194~330 行，使用 for 循环实现上报所有的触摸点坐标，第 202~266 行从触摸芯片中读取触摸坐标数据。第 282 行调用 input_mt_slot 函数上报 ABS_MT_SLOT 事件。第 290 行调用 input_mt_report_slot_state 函数上报 ABS_MT_TRACKING_ID 事件，也就是给 SLOT 关联一个 ABS_MT_TRACKING_ID。第 314 行使用 touchscreen_report_pos 函数上报触摸点对应的(X,Y)坐标值，touchscreen_report_pos 函数定义在 of_touchscreen.c 文件中，此函数最终通过

input_report_abs 来上报坐标值。

第 354 行，使用 input_sync 函数上报 SYN_REPORT 事件。

47.2.4 MT 其他事件的使用

在示例代码 47.2.1.1 中给出了 Linux 所支持的所有 ABS_MT 事件，大家可以根据实际需求将这些事件组成各种事件组合。最简单的组合就是 ABS_MT_POSITION_X 和 ABS_MT_POSITION_Y，可以通过在这两个事件上报触摸点，如果设备支持的话，还可以使用 ABS_MT_TOUCH_MAJOR 和 ABS_MT_WIDTH_MAJOR 这两个消息上报触摸面积信息，关于其他 ABS_MT 事件的具体含义大家可以查看 Linux 内核中的 multi-touch-protocol.txt 文档，这里我们重点补充一下 ABS_MT_TOOL_TYPE 事件。

ABS_MT_TOOL_TYPE 事件用于上报触摸工具类型，很多内核驱动都不能区分出触摸设备类型是手指还是触摸笔？这种情况下，这个事件可以忽略掉。目前的协议支持 MT_TOOL_FINGER(手指)、MT_TOOL_PEN(笔)和 MT_TOOL_PALM(手掌)这三种触摸设备类型，于 Type B 类型，此事件由 input 子系统内核处理。如果驱动程序需要上报 ABS_MT_TOOL_TYPE 事件，那么可以使用 input_mt_report_slot_state 函数来完成此工作。

关于 Linux 系统下的多点触摸(MT)协议就讲解到这里，简单总结一下，MT 协议隶属于 linux 的 input 子系统，驱动通过大量的 ABS_MT 事件向 linux 内核上报多点触摸坐标数据。根据触摸 IC 的不同，分为 Type A 和 Type B 两种类型，不同的类型其上报时序不同，目前使用最多的是 Type B 类型。接下来我们就根据前面学习过的 MT 协议来编写一个多点电容触摸驱动程序，本章节所使用的触摸屏是正点原子的 ATK7084(7 寸 800*480)和 ATK7016(7 寸 1024*600)这两款触摸屏，这两款触摸屏都使用 FT5426 这款触摸 IC，因此驱动程序是完全通用的。

47.2.5 多点触摸所使用到的 API 函数

根据前面的讲解，我们知道 linux 下的多点触摸协议其实就是通过不同的事件来上报触摸点坐标信息，这些事件都是通过 Linux 内核提供的对应 API 函数实现的，本小节我们来看一下一些常见的 API 函数。

1、input_mt_init_slots 函数

input_mt_init_slots 函数用于初始化 MT 的输入 slots，编写 MT 驱动的时候必须先调用此函数初始化 slots，此函数定义在文件 drivers/input/input-mt.c 中，函数原型如下所示：

```
int input_mt_init_slots( struct input_dev    *dev,
                         unsigned int        num_slots,
                         unsigned int        flags)
```

函数参数和返回值含义如下：

dev： MT 设备对应的 input_dev，因为 MT 设备隶属于 input_dev。

num_slots：设备要使用的 SLOT 数量，也就是触摸点的数量。

flags：其他一些 flags 信息，可设置的 flags 如下所示：

```
#define INPUT_MT_POINTER        0x0001  /* pointer device, e.g. trackpad */
#define INPUT_MT_DIRECT         0x0002  /* direct device, e.g. touchscreen */
#define INPUT_MT_DROP_UNUSED    0x0004  /* drop contacts not seen in frame */
#define INPUT_MT_TRACK          0x0008  /* use in-kernel tracking */
#define INPUT_MT_SEMI_MT        0x0010  /* semi-mt device, finger count handled manually */
```

可以采用‘|’运算来同时设置多个 flags 标识。

返回值：0，成功；负值，失败。

2、input_mt_slot 函数

此函数用于 Type B 类型，产生 ABS_MT_SLOT 事件，告诉内核当前上报的是哪个触摸点的坐标数据，此函数定义在文件 include/linux/input/mt.h 中，函数原型如下所示：

```
void input_mt_slot(struct input_dev    *dev,
                   int                 slot)
```

函数参数和返回值含义如下：

dev： MT 设备对应的 input_dev。

slot：当前发送的是哪个 slot 的坐标信息，也就是哪个触摸点。

返回值：无。

3、input_mt_report_slot_state 函数

此函数用于 Type B 类型，用于产生 ABS_MT_TRACKING_ID 和 ABS_MT_TOOL_TYPE 事件，ABS_MT_TRACKING_ID 事件给 slot 关联一个 ABS_MT_TRACKING_ID，ABS_MT_TOOL_TYPE 事件指定触摸类型（是笔还是手指等）。此函数定义在文件 drivers/input/input-mt.c 中，此函数原型如下所示：

```
bool input_mt_report_slot_state(  struct input_dev    *dev,
                                  unsigned int        tool_type,
                                  bool                active)
```

函数参数和返回值含义如下：

dev： MT 设备对应的 input_dev。

tool_type：触摸类型，可以选择 MT_TOOL_FINGER(手指)、MT_TOOL_PEN(笔)或 MT_TOOL_PALM(手掌)，对于多点电容触摸屏来说一般都是手指。

active：true，连续触摸，input 子系统内核会自动分配一个 ABS_MT_TRACKING_ID 给 slot。false，触摸点抬起，表示某个触摸点无效了，input 子系统内核会分配一个-1 给 slot，表示触摸点溢出。

返回值：触摸有效的话返回 true，否则返回 false。

4、input_report_abs 函数

Type A 和 Type B 类型都使用此函数上报触摸点坐标信息，通过 ABS_MT_POSITION_X 和 ABS_MT_POSITION_Y 事件实现 X 和 Y 轴坐标信息上报。此函数定义在文件 include/linux/input.h 中，函数原型如下所示：

```
void input_report_abs(  struct input_dev    *dev,
                        unsigned int        code,
                        int                 value)
```

函数参数和返回值含义如下：

dev： MT 设备对应的 input_dev。

code：要上报的是什么数据，可以设置为 ABS_MT_POSITION_X 或 ABS_MT_POSITION_Y，也就是 X 轴或者 Y 轴坐标数据。

value：具体的 X 轴或 Y 轴坐标数据值。

返回值：无。

5、input_mt_report_pointer_emulation 函数

如果追踪到的触摸点数量多于当前上报的数量，驱动程序使用 BTN_TOOL_TAP 事件来通

知用户空间当前追踪到的触摸点总数量，然后调用 input_mt_report_pointer_emulation 函数将 use_count 参数设置为 false。否则的话将 use_count 参数设置为 true，表示当前的触摸点数量(此函数会获取到具体的触摸点数量，不需要用户给出)，此函数定义在文件 drivers/input/input-mt.c 中，函数原型如下：

```
void input_mt_report_pointer_emulation(struct input_dev  *dev,
                                       bool              use_count)
```

函数参数和返回值含义如下：

dev： MT 设备对应的 input_dev。

use_count：true，有效的触摸点数量；false，追踪到的触摸点数量多于当前上报的数量。

返回值：无。

47.2.6 多点电容触摸驱动框架

前面几小节已经详细的讲解了 linux 下多点触摸屏驱动原理，本小节我们来梳理一下 linux 下多点电容触摸驱动的编写框架和步骤。首先确定驱动需要用到哪些知识点，哪些框架？根据前面的分析，我们在编写驱动的时候需要注意一下几点：

①、多点电容触摸芯片的接口，一般都为 I2C 接口，因此驱动主框架肯定是 I2C。

②、linux 里面一般都是通过中断来上报触摸点坐标信息，因此需要用到中断框架。

③、多点电容触摸属于 input 子系统，因此还要用到 input 子系统框架。

④、在中断处理程序中按照 linux 的 MT 协议上报坐标信息。

根据上面的分析，多点电容触摸驱动编写框架以及步骤如下：

1、I2C 驱动框架

驱动总体采用 I2C 框架，参考框架代码如下所示：

示例代码 47.2.6.1 多点电容触摸驱动 I2C 驱动框架

```
1  /* 设备树匹配表 */
2  static const struct i2c_device_id xxx_ts_id[] = {
3      { "xxx", 0, },
4      { /* sentinel */ }
5  };
6
7  /* 设备树匹配表 */
8  static const struct of_device_id xxx_of_match[] = {
9      { .compatible = "xxx", },
10     { /* sentinel */ }
11 };
12
13 /* i2c 驱动结构体 */
14 static struct i2c_driver ft5x06_ts_driver = {
15     .driver = {
16         .owner = THIS_MODULE,
17         .name = "edt_ft5x06",
18         .of_match_table = of_match_ptr(xxx_of_match),
19     },
```

```
20      .id_table = xxx_ts_id,
21      .probe    = xxx_ts_probe,
22      .remove   = xxx_ts_remove,
23  };
24
25  /*
26   * @description  : 驱动入口函数
27   * @param        : 无
28   * @return       : 无
29   */
30  static int __init xxx_init(void)
31  {
32      int ret = 0;
33
34      ret = i2c_add_driver(&xxx_ts_driver);
35
36      return ret;
37  }
38
39  /*
40   * @description  : 驱动出口函数
41   * @param        : 无
42   * @return       : 无
43   */
44  static void __exit xxx_exit(void)
45  {
46      i2c_del_driver(&ft5x06_ts_driver);
47  }
48
49  module_init(xxx_init);
50  module_exit(xxx_exit);
51  MODULE_LICENSE("GPL");
52  MODULE_AUTHOR("zuozhongkai");
```

I2C 驱动框架已经在四十章进行了详细的讲解，这里就不再赘述了。当设备树中触摸 IC 的设备节点和驱动匹配以后，示例代码 47.2.6.1 中第 21 行的 xxx_ts_probe 函数就会执行，我们可以在此函数中初始化触摸 IC，中断和 input 子系统等。

2、初始化触摸 IC、中断和 input 子系统

初始化操作都是在 xxx_ts_probe 函数中完成，参考框架如下所示(以下代码中步骤顺序可以自行调整，不一定按照示例框架来)：

示例代码 47.2.6.2 xxx_ts_probe 驱动框架

```
1   static int xxx_ts_probe(struct i2c_client *client, const struct
i2c_device_id *id)
2   {
```

```
3       struct input_dev *input;
4
5       /* 1、初始化 I2C              */
6       ......
7
8       /* 2，申请中断，  */
9       devm_request_threaded_irq(&client->dev, client->irq, NULL,
10              xxx_handler, IRQF_TRIGGER_FALLING | IRQF_ONESHOT,
11              client->name, &xxx);
12      ......
13
14      /* 3，input 设备申请与初始化    */
15      input = devm_input_allocate_device(&client->dev);
16
17      input->name = client->name;
18      input->id.bustype = BUS_I2C;
19      input->dev.parent = &client->dev;
20      ......
21
22      /* 4，初始化 input 和 MT          */
23      __set_bit(EV_ABS, input->evbit);
24      __set_bit(BTN_TOUCH, input->keybit);
25
26      input_set_abs_params(input, ABS_X, 0, width, 0, 0);
27      input_set_abs_params(input, ABS_Y, 0, height, 0, 0);
28      input_set_abs_params(input, ABS_MT_POSITION_X,0, width, 0, 0);
29      input_set_abs_params(input, ABS_MT_POSITION_Y,0, height, 0, 0);
30      input_mt_init_slots(input, MAX_SUPPORT_POINTS, 0);
31      ......
32
33      /* 5，注册 input_dev           */
34      input_register_device(input);
35      ......
36  }
```

第 5~7 行，首先肯定是初始化触摸芯片，包括芯片的相关 IO，比如复位、中断等 IO 引脚，然后就是芯片本身的初始化，也就是配置触摸芯片的相关寄存器。

第 9 行，因为一般触摸芯片都是通过中断来向系统上报触摸点坐标信息的，因此我们需要初始化中断，这里又和第三十一章内容结合起来了。大家可能会发现第 9 行并没有使用 request_irq 函数申请中断，而是采用了 devm_request_threaded_irq 这个函数，为什么使用这个函数呢？是不是 request_irq 函数不能使用？答案肯定不是的，这里用 request_irq 函数是绝对没问题的。那为何要用 devm_request_threaded_irq 呢？这里我们就简单的介绍一下这个 API 函数，devm_request_threaded_irq 函数特点如下：

①、用于申请中断，作用和 request_irq 函数类似。

②、此函数的作用是中断线程化，大家如果直接在网上搜索“devm_request_threaded_irq”会发现相关解释很少。但是大家去搜索 request_threaded_irq 函数就会有很多讲解的博客和帖子，这两个函数在名字上的差别就是前者比后者多了个“devm_”前缀，“devm_”前缀稍后讲解。大家应该注意到了“request_threaded_irq”相比“request_irq”多了个 threaded 函数，也就是线程的意思。那么为什么要中断线程化呢？我们都是知道硬件中断具有最高优先级，不论什么时候只要硬件中断发生，那么内核都会终止当前正在执行的操作，转而去执行中断处理程序(不考虑关闭中断和中断优先级的情况)，如果中断非常频繁的话那么内核将会频繁的执行中断处理程序，导致任务得不到及时的处理。中断线程化以后中断将作为内核线程运行，而且也可以被赋予不同的优先级，任务的优先级可能比中断线程的优先级高，这样做的目的就是保证高优先级的任务能被优先处理。大家可能会疑问，前面不是说可以将比较耗时的中断放到下半部(bottom half)处理吗？虽然下半部可以被延迟处理，但是依旧先于线程执行，中断线程化可以让这些比较耗时的下半部与进程进行公平竞争。

要注意，并不是所有的中断都可以被线程化，重要的中断就不能这么操作。对于触摸屏而言只要手指放到屏幕上，它可能就会一直产生中断(视具体芯片而定，FT5426 是这样的)，中断处理程序里面需要通过 I2C 读取触摸信息并上报给内核，I2C 的速度最大只有 400KHz，算是低速外设。不断的产生中断、读取触摸信息、上报信息会导致处理器在触摸中断上花费大量的时间，但是触摸相对来说不是那么重要的事件，因此可以将触摸中断线程化。如果你觉得触摸中断很重要，那么就可以不将其进行线程化处理。总之，要不要将一个中断进行线程化处理是需要自己根据实际情况去衡量的。linux 内核自带的 goodix.c(汇顶科技)、mms114.c(MELFAS 公司)、zforce_ts.c(zForce 公司)等多点电容触摸 IC 驱动程序都采用了中断线程化，当然也有一些驱动没有采用中断线程化。

③、最后来看一下“devm_”前缀，在 linux 内核中有很多的申请资源类的 API 函数都有对应的“devm_”前缀版本。比如 devm_request_irq 和 request_irq 这两个函数，这两个函数都是申请中断的，我们使用 request_irq 函数申请中断的时候，如果驱动初始化失败的话就要调用 free_irq 函数对申请成功的 irq 进行释放，卸载驱动的时候也需要我们手动调用 free_irq 来释放 irq。假如我们的驱动里面申请了很多资源，比如：gpio、irq、input_dev，那么就需要添加很多 goto 语句对其做处理，当这样的标签多了以后代码看起来就不整洁了。“devm_”函数就是为了处理这种情况而诞生的，“devm_”函数最大的作用就是：

使用“devm_”前缀的函数申请到的资源可以由系统自动释放，不需要我们手动处理。

如果我们使用 devm_request_threaded_irq 函数来申请中断，那么就不需要我们再调用 free_irq 函数对其进行释放。大家可以注意一下，带有“devm_”前缀的都是一些和设备资源管理有关的函数。关于“devm_”函数的实现原理这里就不做详细的讲解了，我们的重点在于学会如何使用这些 API 函数，感兴趣的可以查阅一些其他文档或者帖子来看一下“devm_”函数的实现原理。

第 15 行，接下来就是申请 input_dev，因为多点电容触摸属于 input 子系统。这里同样使用 devm_input_allocate_device 函数来申请 input_dev，也就是我们前面讲解的 input_allocate_device 函数加“devm_”前缀版本。申请到 input_dev 以后还需要对其进行初始化操作。

第 23~24 行，设置 input_dev 需要上报的事件为 EV_ABS 和 BTN_TOUCH，因为多点电容屏的触摸坐标为绝对值，因此需要上报 EV_ABS 事件。触摸屏有按下和抬起之分，因此需要上报 BTN_TOUCH 按键。

第 26~29 行，调用 input_set_abs_params 函数设置 EV_ABS 事件需要上报 ABS_X、ABS_Y、ABS_MT_POSITION_X 和 ABS_MT_POSITION_Y。单点触摸需要上报 ABS_X 和 ABS_Y，对于多点触摸需要上报 ABS_MT_POSITION_X 和 ABS_MT_POSITION_Y。

第 30 行，调用 input_mt_init_slots 函数初始化多点电容触摸的 slots。

第 34 行，调用 input_register_device 函数系统注册前面申请到的 input_dev。

3、上报坐标信息

最后就是在中断服务程序中上报读取到的坐标信息，根据所使用的多点电容触摸设备类型选择使用 Type A 还是 Type B 时序。由于大多数的设备都是 Type B 类型，因此这里就以 Type B 类型为例讲解一下上报过程，参考驱动框架如下所示：

示例代码 47.2.6.3 xxx_handler 中断处理程序

```
1  static irqreturn_t xxx_handler(int irq, void *dev_id)
2  {
3
4      int num;                /* 触摸点数量 */
5      int x[n], y[n];         /* 保存坐标值 */
6
7      /* 1、从触摸芯片获取各个触摸点坐标值 */
8      ......
9
10     /* 2、上报每一个触摸点坐标 */
11     for (i = 0; i < num; i++) {
12         input_mt_slot(input, id);
13         input_mt_report_slot_state(input, MT_TOOL_FINGER, true);
14         input_report_abs(input, ABS_MT_POSITION_X, x[i]);
15         input_report_abs(input, ABS_MT_POSITION_Y, y[i]);
16     }
17     ......
18
19     input_sync(input);
20     ......
21
22     return IRQ_HANDLED;
23 }
```

进入中断处理程序以后首先肯定是从触摸 IC 里面读取触摸坐标以及触摸点数量，假设触摸点数量保存到 num 变量，触摸点坐标存放到 x，y 数组里面。

第 11~16 行，循环上报每一个触摸点坐标，一定要按照 Type B 类型的时序进行，这个已经在 47.2.3 小节进行详细的讲解，这里就不再赘述了。

第 19 行，每一轮触摸点坐标上报完毕以后就调用一次 input_sync 函数发送一个 SYN_REPORT 事件。

关于多点电容触摸驱动框架就讲解到这里，接下来我们就实际编写一个多点电容触摸驱动程序。

47.3 硬件原理图分析

触摸屏原理图一共涉及到 4 个引脚，如下所示：

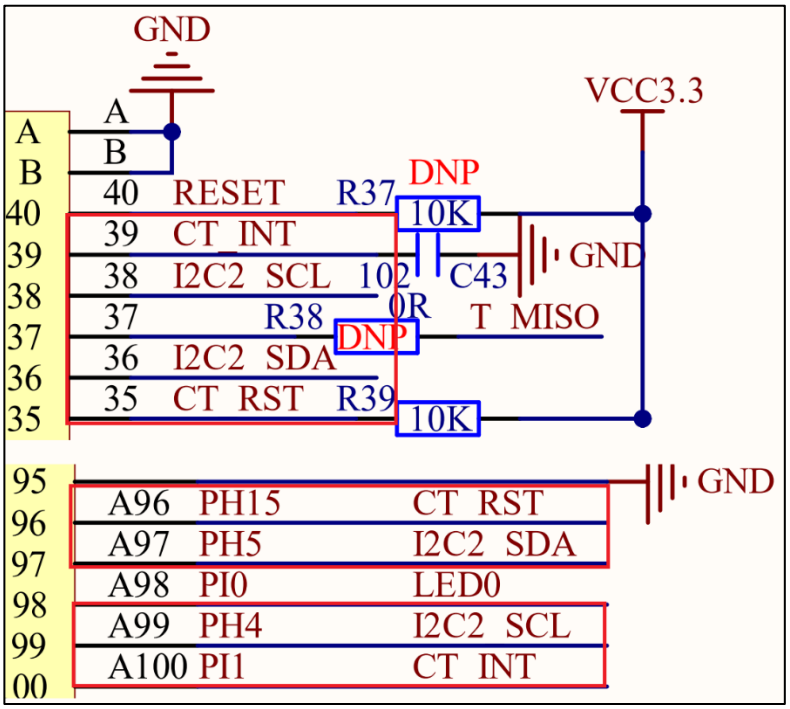


图 47.3.1 触摸原理图

I2C 接口引脚 I2C2_SCL 和 I2C2_SDA，触摸 IC 复位引脚 CT_RST 以及触摸 IC 中断引脚 CT_INT。I2C2_SCL 和 I2C2_SDA 对应的引脚为 PH4 和 PH5。CT_INT 对应的引脚为 PI1，CT_RST 对应的引脚为 PH15。

对于中断触发模式，当 FT5426 检测到有触摸事件发生时，会将中断信号拉低，所以对主机控制器来说，它是一个下降沿中断触发方式；当一直按着触摸屏不松开，则会一直触发中断，现在大部分的触摸 IC 都是这样设计的。相对于以前老式的触摸 IC，它们只有检测到按下这个动作发生时才会触发一次中断，一直按着不放是不会再次触发中断的。这样的设计对我们编写驱动来说在软件处理上会比较麻烦，一般要专门使用一个定时器来检测、处理触摸屏长按的情况，但对于 FT5426 这类 IC 来说，不需要这样做。

47.4 试验程序编写

本试验以正点原子的 ATK7084(7 寸 800*480 分辨率)和 ATK7016(7 寸 1024*600 分辨率)这两款屏幕所使用的 FT5426 触摸芯片为例，讲解如何编写多点电容触摸驱动。前面也已经对 FT5426 这颗触摸芯片进行了比较详细的讲解。本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→ 23_multitouch。

47.4.1 添加 FT5426 设备节点

1、添加引脚节点

FT5426 用到了 I2C2 接口以及 PI1 和 PH15 这两个 IO，I2C2 的引脚前面实验已经修改好了，本实验就不需要再修改。本实验重点只需要配置 PI1 以及 PH15 这两个引脚，打开 stm32mp15-pinctrl.dtsi 文件，添加如下引脚配置信息：

示例代码 47.4.1.1 ft5426 中断以及复位引脚配置

```
1 ft5426int_reset_pins_a: ft5426int_reset_pins-0 {
2       pins1 {
```

```
3       pinmux = <STM32_PINMUX('I', 1, GPIO)>,    /* ft5426 INT     */
4                <STM32_PINMUX('H', 15, GPIO)>;   /* ft5426 RESET   */
5       bias-pull-up;
6       slew-rate = <0>;
7      };
8 };
```

2、FT5426 节点配置

FT5426 这个触摸 IC 挂载在 STM32MP1 的 I2C2 总线接口上，因此需要向 I2C2 节点下添加一个子节点，此子节点用于描述 FT5426，添加完成以后的 I2C2 节点内容如下所示(省略掉其他挂载到 I2C2 下的设备)：

示例代码 47.4.1.2 ft5426 节点信息

```
1  &i2c2 {
2      pinctrl-names = "default", "sleep";
3      pinctrl-0 = <&i2c2_pins_a>;
4      pinctrl-1 = <&i2c2_pins_sleep_a>;
5      status = "okay";
6
7       ft5426: ft5426@38 {
8          compatible = "edt,edt-ft5426";
9          pinctrl-0 = <&ft5426int_reset_pins_a>;
10         reg = <0x38>;
11         irq-gpios = <&gpioi 1 GPIO_ACTIVE_LOW>;
12         reset-gpios = <&gpioh 15 GPIO_ACTIVE_LOW>;
13         status = "okay";
14     };
15 };
```

第 7 行，触摸屏所使用的 FT5426 芯片节点，挂载 I2C2 节点下，FT5426 的器件地址为 0X38。

第 10 行，reg 属性描述 FT5426 的器件地址为 0x38。

第 11 行，irq-gpios 属性描述中断 IO 对应的 GPIO 为 PI1。

第 12 行，reset-gpios 属性描述复位 IO 对应的 GPIO 为 PH15。

47.4.2 编写多点电容触摸驱动

新建名为“23_multitouch”的文件夹，然后在 23_multitouch 文件夹里面创建 vscode 工程，工作区命名为“multitouch”。工程创建好以后新建 ft5x06.c 这个驱动文件，在里面输入如下所示内容：

示例代码 47.4.2.1 ft5x06.c 文件内容

```
11 #include <linux/module.h>
12 #include <linux/i2c.h>
13 #include <linux/input/mt.h>
14 #include <linux/of_gpio.h>
15 #include <linux/delay.h>
```

```
16  #include <linux/interrupt.h>
17
18  /* FT5426 寄存器相关宏定义 */
19  #define FT5426_DEVIDE_MODE_REG  0x00    /* 模式寄存器                */
20  #define FT5426_TD_STATUS_REG    0x02    /* 状态寄存器                */
21  #define FT5426_TOUCH_DATA_REG   0x03    /* 触摸数据读取的起始寄存器  */
22  #define FT5426_ID_G_MODE_REG    0xA4    /* 中断模式寄存器            */
23
24  #define MAX_SUPPORT_POINTS      5       /* ft5426 最大支持 5 点触摸  */
25
26  #define TOUCH_EVENT_DOWN        0x00    /* 按下 */
27  #define TOUCH_EVENT_UP          0x01    /* 抬起 */
28  #define TOUCH_EVENT_ON          0x02    /* 接触 */
29  #define TOUCH_EVENT_RESERVED    0x03    /* 保留 */
30
31
32  struct edt_ft5426_dev {
33      struct i2c_client *client;
34      struct input_dev *input;
35      int reset_gpio;
36      int irq_gpio;
37  };
38
39  static int edt_ft5426_ts_write(struct edt_ft5426_dev *ft5426,
40            u8 addr, u8 *buf, u16 len)
41  {
42      struct i2c_client *client = ft5426->client;
43      struct i2c_msg msg;
44      u8 send_buf[6] = {0};
45      int ret;
46
47      send_buf[0] = addr;
48      memcpy(&send_buf[1], buf, len);
49
50      msg.flags = 0;                      /* i2c 写 */
51      msg.addr = client->addr;
52      msg.buf = send_buf;
53      msg.len = len + 1;
54
55      ret = i2c_transfer(client->adapter, &msg, 1);
56      if (1 == ret)
57          return 0;
58      else {
```

```
59          dev_err(&client->dev, "%s: write error, addr=0x%x len=%d.\n",
60                      __func__, addr, len);
61          return -1;
62      }
63  }
64
65  static int edt_ft5426_ts_read(struct edt_ft5426_dev *ft5426,
66              u8 addr, u8 *buf, u16 len)
67  {
68      struct i2c_client *client = ft5426->client;
69      struct i2c_msg msg[2];
70      int ret;
71
72      msg[0].flags = 0;               /* i2c写   */
73      msg[0].addr = client->addr;
74      msg[0].buf = &addr;
75      msg[0].len = 1;                 /* 1个字节 */
76
77      msg[1].flags = I2C_M_RD;        /* i2c读   */
78      msg[1].addr = client->addr;
79      msg[1].buf = buf;
80      msg[1].len = len;
81
82      ret = i2c_transfer(client->adapter, msg, 2);
83      if (2 == ret)
84          return 0;
85      else {
86          dev_err(&client->dev, "%s: read error, addr=0x%x len=%d.\n",
87                      __func__, addr, len);
88          return -1;
89      }
90  }
91
92  static int edt_ft5426_ts_reset(struct edt_ft5426_dev *ft5426)
93  {
94      struct i2c_client *client = ft5426->client;
95      int ret;
96
97      /* 从设备树中获取复位管脚 */
98          ft5426->reset_gpio  =  of_get_named_gpio(client->dev.of_node,
"reset-gpios", 0);
99      if (!gpio_is_valid(ft5426->reset_gpio)) {
100         dev_err(&client->dev, "Failed to get ts reset gpio\n");
```

```
101         return ft5426->reset_gpio;
102     }
103
104     /* 申请使用管脚 */
105     ret = devm_gpio_request_one(&client->dev, ft5426->reset_gpio,
106                 GPIOF_OUT_INIT_HIGH, "ft5426 reset");
107     if (ret < 0)
108         return ret;
109
110     msleep(20);
111     gpio_set_value_cansleep(ft5426->reset_gpio, 0);  /* 拉低复位引脚 */
112     msleep(5);
113     gpio_set_value_cansleep(ft5426->reset_gpio, 1);  /* 结束复位 */
114
115     return 0;
116 }
117
118 static irqreturn_t edt_ft5426_ts_isr(int irq, void *dev_id)
119 {
120     struct edt_ft5426_dev *ft5426 = dev_id;
121     u8 rdbuf[30] = {0};
122     int i, type, x, y, id;
123     bool down;
124     int ret;
125
126     /* 读取 FT5426 触摸点坐标从 0x02 寄存器开始，连续读取 29 个寄存器 */
127     ret = edt_ft5426_ts_read(ft5426, FT5426_TD_STATUS_REG, rdbuf, 29);
128     if (ret)
129         goto out;
130
131     for (i = 0; i < MAX_SUPPORT_POINTS; i++) {
132
133         u8 *buf = &rdbuf[i * 6 + 1];
134
135         /* 以第一个触摸点为例，寄存器 TOUCH1_XH(地址 0x03)，各 bit 位描述如下:
136          * bit7:6  Event flag  0:按下 1:释放 2:接触 3:没有事件
137          * bit5:4  保留
138          * bit3:0  X 轴触摸点的 11~8 位
139          */
140         type = buf[0] >> 6;     /* 获取触摸点的 Event Flag */
141         if (type == TOUCH_EVENT_RESERVED)
142             continue;
143
```

```
144         /* 我们所使用的触摸屏和 FT5426 是反过来的 */
145         x = ((buf[2] << 8) | buf[3]) & 0x0fff;
146         y = ((buf[0] << 8) | buf[1]) & 0x0fff;
147 
148         /* 以第一个触摸点为例，寄存器 TOUCH1_YH(地址 0x05)，各 bit 位描述如下:
149          * bit7:4  Touch ID  触摸 ID，表示是哪个触摸点
150          * bit3:0  Y 轴触摸点的 11~8 位。
151          */
152         id = (buf[2] >> 4) & 0x0f;
153         down = type != TOUCH_EVENT_UP;
154 
155         input_mt_slot(ft5426->input, id);
156         input_mt_report_slot_state(ft5426->input, MT_TOOL_FINGER,
                                       down);
157 
158         if (!down)
159             continue;
160 
161         input_report_abs(ft5426->input, ABS_MT_POSITION_X, x);
162         input_report_abs(ft5426->input, ABS_MT_POSITION_Y, y);
163     }
164 
165     input_mt_report_pointer_emulation(ft5426->input, true);
166     input_sync(ft5426->input);
167 
168 out:
169     return IRQ_HANDLED;
170 }
171 
172 static int edt_ft5426_ts_irq(struct edt_ft5426_dev *ft5426)
173 {
174     struct i2c_client *client = ft5426->client;
175     int ret;
176 
177     /* 从设备树中获取中断管脚 */
178     ft5426->irq_gpio = of_get_named_gpio(client->dev.of_node,
                                             " irq-gpios", 0);
179     if (!gpio_is_valid(ft5426->irq_gpio)) {
180         dev_err(&client->dev, "Failed to get ts interrupt gpio\n");
181         return ft5426->irq_gpio;
182     }
183 
184     /* 申请使用管脚 */
```

```
185     ret = devm_gpio_request_one(&client->dev, ft5426->irq_gpio,
186               GPIOF_IN, "ft5426 interrupt");
187     if (ret < 0)
188         return ret;
189 
190     /* 注册中断服务函数 */
191     ret = devm_request_threaded_irq(&client->dev,
                                gpio_to_irq(ft5426->irq_gpio),
192                                 NULL, edt_ft5426_ts_isr,
                                 IRQF_TRIGGER_FALLING | IRQF_ONESHOT,
193                                  client->name, ft5426);
194     if (ret) {
195         dev_err(&client->dev, "Failed to request touchscreen IRQ.\n");
196         return ret;
197     }
198 
199     return 0;
200 }
201 
202 static int edt_ft5426_ts_probe(struct i2c_client *client,
203           const struct i2c_device_id *id)
204 {
205     struct edt_ft5426_dev *ft5426;
206     struct input_dev *input;
207     u8 data;
208     int ret;
209 
210     /* 实例化一个 struct edt_ft5426_dev 对象 */
211     ft5426 = devm_kzalloc(&client->dev, sizeof(struct edt_ft5426_dev),
                            GFP_KERNEL);
212     if (!ft5426) {
213     dev_err(&client->dev, "Failed to allocate ft5426 driver data.\n");
214         return -ENOMEM;
215     }
216 
217     ft5426->client = client;
218 
219     /* 复位 FT5426 触摸芯片 */
220     ret = edt_ft5426_ts_reset(ft5426);
221     if (ret)
222         return ret;
223 
224     msleep(5);
```

```
225
226     /* 初始化 FT5426 */
227     data = 0;
228     edt_ft5426_ts_write(ft5426, FT5426_DEVIDE_MODE_REG, &data, 1);
229     data = 1;
230     edt_ft5426_ts_write(ft5426, FT5426_ID_G_MODE_REG, &data, 1);
231
232     /* 申请、注册中断服务函数 */
233     ret = edt_ft5426_ts_irq(ft5426);
234     if (ret)
235         return ret;
236
237     /* 注册 input 设备 */
238     input = devm_input_allocate_device(&client->dev);
239     if (!input) {
240         dev_err(&client->dev, "Failed to allocate input device.\n");
241         return -ENOMEM;
242     }
243
244     ft5426->input = input;
245     input->name = "FocalTech FT5426 TouchScreen";
246     input->id.bustype = BUS_I2C;
247
248     input_set_abs_params(input, ABS_MT_POSITION_X,
249                  0, 1024, 0, 0);
250     input_set_abs_params(input, ABS_MT_POSITION_Y,
251                  0, 600, 0, 0);
252
253     ret = input_mt_init_slots(input, MAX_SUPPORT_POINTS,
                                  INPUT_MT_DIRECT);
254     if (ret) {
255         dev_err(&client->dev, "Failed to init MT slots.\n");
256         return ret;
257     }
258
259     ret = input_register_device(input);
260     if (ret)
261         return ret;
262
263     i2c_set_clientdata(client, ft5426);
264     return 0;
265 }
266
```

```
267 static int edt_ft5426_ts_remove(struct i2c_client *client)
268 {
269     struct edt_ft5426_dev *ft5426 = i2c_get_clientdata(client);
270     input_unregister_device(ft5426->input);
271     return 0;
272 }
273
274 static const struct of_device_id edt_ft5426_of_match[] = {
275     { .compatible = "edt,edt-ft5426", },
276     { /* sentinel */ }
277 };
278
279 static struct i2c_driver edt_ft5426_ts_driver = {
280     .driver = {
281         .owner          = THIS_MODULE,
282         .name           = "edt_ft5426",
283         .of_match_table = of_match_ptr(edt_ft5426_of_match),
284     },
285     .probe    = edt_ft5426_ts_probe,
286     .remove   = edt_ft5426_ts_remove,
287 };
288
289 module_i2c_driver(edt_ft5426_ts_driver);
290
291 MODULE_LICENSE("GPL");
292 MODULE_AUTHOR("ALIENTEK");
293 MODULE_INFO(intree, "Y");
```

第 19~29 行，FT5426 触摸屏相关宏定义。

第 32~37 行，自定义结构体 edt_ft5426_dev，用于描述 FT5426 设备，其中包括 I2C 从机设备指针 client、input 输入设备指针 input、一个复位引脚 reset_gpio 以及一个中断引脚 irq_gpio。

第 39~63 行，自定义函数 edt_ft5426_ts_write，用于向 FT5426 内部寄存器连续写入数据，可指定写入数据字节长度。

第 65~90 行，自定义函数 edt_ft5426_ts_read，用于连续读取 FT5426 内部寄存器中的数据，可指定读取数据字节长度。

第 92~116 行，自定义函数 edt_ft5426_ts_reset，用于复位 FT5426 触摸芯片，FT5426 的复位 IO 为低电平有效。FT5426 的复位过程没有什么特殊要求，我们直接将复位 IO 拉低，延时 5ms 然后再将其拉高即可！拉高之后表示结束复位状态，但是复位结束之后不能立即读写寄存器，需要先延时一段时间再进行读写寄存器操作。

第 118~170 行，edt_ft5426_ts_isr 函数，触摸屏中断服务函数，触摸点坐标的上报就是在此函数中完成的。第 127 行通过 edt_ft5426_ts_read 函数读取 FT5426 的所有触摸点信息寄存器数据，从 0x02 这个地址开始，一共 29 个寄存器。第 131~ 163 行的 for 循环就是一个一个的上报触摸点坐标数据，使用 Type B 时序，这个我们已经在前面说了很多次了。最后在 166 行通过 input_sync 函数上报 SYN_REPORT 事件。如果理解了前面讲解的 Type B 时序，那么此函数就

很好看懂。

第 172~200 行，自定义函数 edt_ft5426_ts_irq，用于初始化 FT5426 中断 IO，第 185 行使用 devm_gpio_request_one 函数申请中断 IO。第 192 行使用函数 devm_request_threaded_irq 申请并注册中断，中断处理函数为 edt_ft5426_ts_isr；IRQF_TRIGGER_FALLING 表示中断触发类型为下降沿触发。

第 202~265 行，I2C 总线的 probe 函数 edt_ft5426_ts_probe，当 I2C 从机设备与驱动匹配成功之后就会执行这个函数，一般在此函数中完成一些初始化工作。我们重点来看一下 238~261 行，首先第 238 行调用 devm_input_allocate_device 函数实例化一个 input_dev 结构体对象，248 和 250 行设置 input_dev 需要上报的事件为 ABS_MT_POSITION_X 和 ABS_MT_POSITION_Y，对于多点触摸设备来说，这两个事件是必须要设置的，1024 和 600 表示触摸屏的 X 和 Y 轴分辨率，这里笔者是以 7 寸 1024x600 触摸屏为例，如果大家使用的是 7 寸 800x480 的话，则需要将这里修改为 800 和 480。此外还有 EV_KEY、EV_ABS 以及 BTN_TOUCH 这些事件对于触摸屏来说都是必须事件，但是大家会发现在我们的代码中并没有设置 input_dev 支持上报这些事件，其实这些事件在 input_mt_init_slots 函数中会自动帮我们设置，大家可以自己去看一下这个函数的代码。EV_KEY 是按键事件，用于上报触摸屏是否被按下，相当于把触摸屏当做一个按键。EV_ABS 是触摸点坐标数据，BTN_TOUCH 表示将触摸屏的按下和抬起用作 BTN_TOUCH 按键。直接调用 input_set_abs_params 函数即可设置。

第 253 行调用 input_mt_init_slots 函数初始化 slots，第二个参数表示触摸屏设置支持的最大触摸点数量，FT5426 是个 5 点电容触摸芯片，因此一共 5 个 slot；最后一个参数是一个标志位，表示设备类型，INPUT_MT_DIRECT 表示为触摸屏设备类型。

最后在 259 行调用 input_register_device 函数向 input 子系统注册一个 input 设备。

第 267~272 行，当驱动模块被卸载时 edt_ft5426_ts_remove 函数就会执行，因为前面很多资源我们都是用“devm_”前缀函数来申请的，因此不需要手动释放。此函数只需要调用 input_unregister_device 来释放掉前面添加到内核中的 input_dev。

驱动代码到这里就给大家讲解完毕了，其实也非常简单没什么太多难点，理解了前面所说的知识点，这段代码理解起来是很容易的，关于 4.3 寸屏和 10 寸屏的驱动程序就不给大家讲解了，驱动代码也给大家提供了，路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→23_multitouch→gt9147.c

47.5 运行测试

47.5.1 编译驱动程序

编写 Makefile 文件，本章实验的 Makefile 文件和第四十三章实验基本一样，只是将 obj-m 变量的值改为“ft5x06.o”，Makefile 内容如下所示：

示例代码 47.5.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
4  obj-m := ft5x06.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“ft5x06.o”。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“ft5x06.ko”的驱动模块文件。

47.5.2 运行测试

编译设备树，然后使用新的设备树启动 linux 内核。

多点电容触摸屏测试不需要编写专门的 APP，将上一小节编译出来 ft5x06.ko 拷贝到 rootfs/lib/modules/5.3.41 目录中，启动开发板，进入到目录 lib/modules/5.3.41 中，输入如下命令加载 ft5x06.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe ft5x06.ko      //加载驱动模块
```

当驱动模块加载成功以后会有如图 47.5.2.1 所示信息输入：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe ft5x06.ko
input: FocalTech FT5426 TouchScreen as /devices/platform/soc/40013000.i2c/i2c-0/0-0038/input/input1
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 47.5.2.1 驱动加载过程

驱动加载成功以后就会生成/dev/input/eventX(X=1,2,3…)，比如本实验的多点电容触摸驱动就会在我所使用的 STM32MP1 开发板平台下就会生成/dev/input/event1 这个文件，如图 47.5.2.2 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls /dev/input/*
/dev/input/event0  /dev/input/event1  → 电容触摸屏
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 47.5.2.2 电容屏对应的 event 设备

不同的平台 event 序号不同，也可能是 event3，event4 等，一切以实际情况为准！输入如下命令查看 event1，也就是多点电容触摸屏上报的原始数据：

```
hexdump /dev/input/event1
```

现在用一根手指触摸屏幕的右上角，然后再抬起，理论坐标值为(1023,0)，但是由于触摸误差的原因，大概率不会是绝对的(1023,0)，应该是在此值附近的一个触摸坐标值，实际的上报数据如图 47.5.2.3 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ hexdump /dev/input/event1
0000000 04d5 0000 5e1e 0005 0003 0039 0000 0000
0000010 04d5 0000 5e1e 0005 0003 0035 03fc 0000
0000020 04d5 0000 5e1e 0005 0003 0036 0010 0000
0000030 04d5 0000 5e1e 0005 0001 014a 0001 0000
0000040 04d5 0000 5e1e 0005 0003 0000 03fc 0000
0000050 04d5 0000 5e1e 0005 0003 0001 0010 0000
0000060 04d5 0000 5e1e 0005 0000 0000 0000 0000
0000070 04d5 0000 1dd9 0007 0003 0039 ffff ffff
0000080 04d5 0000 1dd9 0007 0001 014a 0000 0000
0000090 04d5 0000 1dd9 0007 0000 0000 0000 0000
```

图 47.5.2.3 上报的原始数据

图 47.5.2.3 上报的信息是按照 input_event 类型呈现的，这个同样在第 38.4.2 小节做了详细的介绍，这里我们重点来分析一下，在多点电容触摸屏上其所代表的具体含义，将图 46.5.2.3 中的数据进行整理，结果如下所示：

示例代码 47.5.2.1 Makefile 文件

```
/*************************input_event 类型*************************/
/* 编号 */ /* tv_sec */ /* tv_usec */ /* type */ /* code */ /* value */
```

```
0000000    04d5 0000    5e1e 0005    0003    0039    0000 0000
0000010    04d5 0000    5e1e 0005    0003    0035    03fc 0000
0000020    04d5 0000    5e1e 0005    0003    0036    0010 0000
0000030    04d5 0000    5e1e 0005    0001    014a    0001 0000
0000040    04d5 0000    5e1e 0005    0003    0000    03fc 0000
0000050    04d5 0000    5e1e 0005    0003    0001    0010 0000
0000060    04d5 0000    5e1e 0005    0000    0000    0000 0000
0000070    04d5 0000    1dd9 0007    0003    0039    ffff ffff
0000080    04d5 0000    1dd9 0007    0001    014a    0000 0000
0000090    04d5 0000    1dd9 0007    0000    0000    0000 0000
```

第 1 行，type 为 0x3，说明是一个 EV_ABS 事件，code 为 0x39，为 ABS_MT_TRACKING_ID，因此这一行就是 input_mt_slot 函数上报的 ABS_MT_TRACKING_ID 事件。value=0，说明屏被按下。

第 2 行，type 为 0x3，是一个 EV_ABS 事件，code 为 0x35，为 ABS_MT_POSITION_X，这一行就是 input_report_abs 函数上报的 ABS_MT_POSITION_X 事件，也就是触摸点的 X 轴坐标。value=0x3fc=1020，说明触摸点 X 轴坐标为 1020，属于屏幕右边区域。

第 3 行，type 为 0x3，是一个 EV_ABS 事件，code 为 0x36，为 ABS_MT_POSITION_Y，这一行就是 input_mt_report_slot_state 函数上报的 ABS_MT_POSITION_Y 事件，也就是触摸点的 Y 轴坐标。value=0x10=16，说明 Y 轴坐标为 16，由此可以看出本次触摸的坐标为(1020, 16)，处于屏幕右上角区域。

第 4 行，type 为 0x1，是一个 EV_KEY 事件，code=0x14a，为 BTN_TOUCH，value=0x1 表示触摸屏被按下。

第 5 行，type 为 0x3，是一个 EV_ABS 事件，code 为 0x0，为 ABS_X，用于单点触摸的时候上报 X 轴坐标。在这里和 ABS_MT_POSITION_X 相同，value 也为 0x3fc=1020。ABS_X 是由 input_mt_report_pointer_emulation 函数上报的。

第 6 行，type 为 0x3，是一个 EV_ABS 事件，code 为 0x1，为 ABS_Y，用于单点触摸的时候上报 Y 轴坐标。在这里和 ABS_MT_POSITION_Y 相同，value 也为 0x10=16。ABS_Y 是由 input_mt_report_pointer_emulation 函数上报的。

第 7 行，type 为 0x0，是一个 EV_SYN 事件，由 input_sync 函数上报。

第 8 行，type 为 0x3，是一个 EV_ABS 事件，code 为 0x39，也就是 ABS_MT_TRACKING_ID，value=0xffffffff=-1，说明触摸点离开了屏幕。

第 9 行，type 为 0x1，是一个 EV_KEY 事件，code=0x14a，为 BTN_TOUCH，value=0x0 表示手指离开触摸屏，也就是触摸屏没有被按下了。

第 10 行，type 为 0x0，是一个 EV_SYN 事件，由 input_sync 函数上报。

以上就是一个触摸点的坐标上报过程，和我们前面讲解的 Type B 类型设备一致。

47.5.3 将驱动添加到内核中

前面我们一直将触摸驱动编译为模块，每次系统启动以后在手动加载驱动模块，这样很不方便。当我们把驱动调试成功以后一般都会将其编译到内核中，这样内核启动以后就会自动加载驱动，不需要我们再手动 modprobe 了。本节我们就来学习一下如何将 ft5x06.c 添加到 linux 内核里面，步骤如下所示：

1、将驱动文件放到合适的位置

首先肯定是在内核源码中找个合适的位置将 ft5x06.c 放进去，ft5x06.c 是个触摸屏驱动，因此我们需要查找一下 linux 内核里面触摸屏驱动放到了哪个目录下。linux 内核里面将触摸屏驱动放到了 drivers/input/touchscreen 目录下，因此我们要将 ft5x06.c 拷贝到此目录下，命令如下：

```
cp ft5x06.c (内核源码目录)/drivers/input/touchscreen/ -f
```

2、修改对应的 Makefile

修改 drivers/input/touchscreen 目录下的 Makefile，在最下面添加下面一行：

示例代码 47.5.3.1 drivers/input/touchscreen/Makefile 添加的内容

```
obj-y   += ft5x06.o
```

完成以后如图 47.5.3.1 所示：

```
111  obj-$(CONFIG_TOUCHSCREEN_RASPBERRYPI_FW)    += raspberrypi-ts.o
112  obj-$(CONFIG_TOUCHSCREEN_IQS5XX)    += iqs5xx.o
113  obj-y += ft5x06.o
114
```

图 47.5.3.1 修改后的 Makefile

修改完成以后重新编译 linux 内核，然后用新的 uImage 启动开发板。如果驱动添加成功的话系统启动的时候就会输出如图 47.5.3.2 所示的信息：

```
i2c i2c-0: Added multiplexed i2c bus 1
input: FocalTech FT5426 TouchScreen as /devices/platform/soc/40013000.i2c/i2c-0/0-0038/input/input0
stm32f7-i2c 40013000.i2c: STM32F7 I2C-0 bus adapter
stm32f7-i2c 40015000.i2c: using dma2chan0 (tx) and dma2chan1 (rx) for DMA transfers
```

图 47.5.3.2 启动信息

从图 47.5.3.2 可以看出，触摸屏驱动已经启动了，这个时候就会自动生成/dev/input/evenvtX。在本实验中将触摸屏驱动添加到 linux 内核里面以后触摸屏对应的是 event0，而不是前面编译为模块对应的 event1，这一点一定要注意！输入如下命令，查看驱动工作是否正常：

```
hexdump   /dev/input/event0      //查看触摸屏原始数据上报信息
```

结果如图 47.5.3.3 所示：

```
[root@ATK-stm32mp1]:~$ hexdump /dev/input/event0
0000000 005f 0000 1f92 0007 0003 0039 0000 0000
0000010 005f 0000 1f92 0007 0003 0035 03f1 0000
0000020 005f 0000 1f92 0007 0003 0036 0031 0000
0000030 005f 0000 1f92 0007 0001 014a 0001 0000
0000040 005f 0000 1f92 0007 0003 0000 03f1 0000
0000050 005f 0000 1f92 0007 0003 0001 0031 0000
0000060 005f 0000 1f92 0007 0000 0000 0000 0000
0000070 005f 0000 b641 0008 0003 0039 ffff ffff
0000080 005f 0000 b641 0008 0001 014a 0000 0000
0000090 005f 0000 b641 0008 0000 0000 0000 0000
```

图 47.5.3.3 上报的原始数据

可以看出，坐标数据上报正常，说明驱动工作没问题。

47.6 tslib 移植与使用

47.6.1 tslib 移植

tslib 是一个开源的第三方库，用于触摸屏性能调试，使用电阻屏的时候一般使用 tslib 进行校准。虽然电容屏不需要校准，但是由于电容屏加工的原因，有的时候其不一定精准，因此有时候也需要进行校准。最主要的是 tslib 提供了一些其他软件，我们可以通过这些软件来测试触

摸屏工作是否正常。最新版本的 tslib 已经支持了多点电容触摸屏，因此可以通过 tslib 来直观的测试多点电容触摸屏驱动，这个要比观看 eventX 原始数据方便的多。

tslib 的移植很简单，我们使用的文件系统为 buildroot，只要打开图形化配置界面即可 tilib 的配置，进入 buildroot 的源码目录下，运行以下命令：

```
make menuconfig
```

配置路径如下：

```
-> Target packages
    -> Libraries
        -> Hardware handling
            ->  [*] tslib
```

配置如图 47.6.1.1 所示：

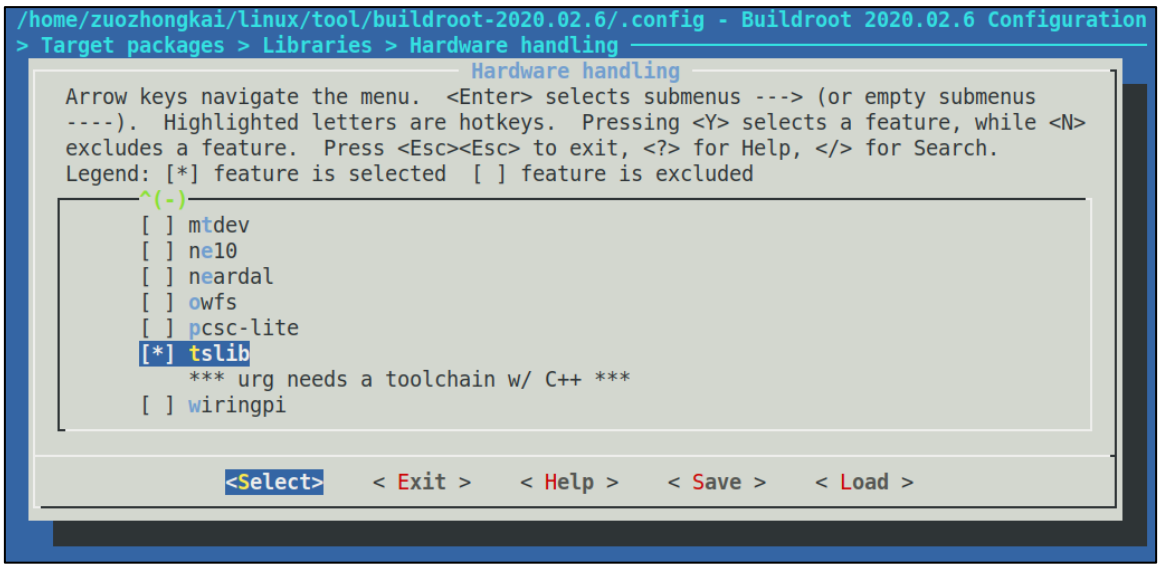


图 47.6.1.1 tslib 库的配置

保存配置在重新编译文件系统记得 ubuntu 要连接网络哦。编译完成后，进入 output/images 目录，运行以下命令把文件系统替换进去：

```
cd output/images/                                          //进入到 output/images 目录
sudo tar -axvf rootfs.tar -C /home/zuozhongkai/linux/nfs/rootfs  //解压到 nfsroot 目录
```

上述命令将 buildroot 中 output/images/rootfs.tar 这个压缩包解压到/home/zuozhongkai/linux/nfs/rootfs 这个目录中，这个目录就是我当前 nfsroot 目录，大家根据自己的实际情况解压到对应的目录文件中。

完成以后重启开发板！然后就可以进行测试了。

47.6.2 tslib 测试

电容屏可以不用校准，如果是电阻屏就要先进行校准！校准的话输入如下命令：

```
ts_calibrate
```

校准完成以后如果不满意，或者不小心对电容屏做了校准，那么直接删除掉/etc/pointercal 文件即可。

最后我们使用 ts_test_mt 这个软件来测试触摸屏工作是否正常，以及多点触摸是否有效，执行如下所示命令：

```
ts_test_mt
```

此命令会打开一个触摸测试界面，如图 47.6.2.1 所示：

图 47.6.2.1 ts_test_mt 界面

在图 47.6.2.1 上有三个按钮“Drag”、“Draw”和“Quit”，这三个按钮的功能如下：

Drag：拖拽按钮，默认就是此功能，大家可以看到屏幕中间有一个十字光标，我们可以通过触摸屏幕来拖拽此光标。一个触摸点一个十字光标，对于 5 点电容触摸屏，如果 5 个手指都放到屏幕上，那么就有 5 个光标，一个手指一个。

Draw：绘制按钮，按下此按钮我们就可以在屏幕上进行简单的绘制，可以通过此功能检测多点触摸工作是否正常。

Quit：退出按钮，退出 ts_test_mt 测试软件。

点击“Draw”按钮，使用绘制功能，5 个手指一起划过屏幕，如果多点电容屏工作正常的话就会在屏幕上留下 5 条线，如图 47.6.2.2 所示：

图 47.6.2.2 5 点触摸测试

从图 47.6.2.2 可以看出，屏幕上有 5 条线，说明 5 点电容触摸工作正常。

47.7 使用内核自带的驱动

Linux 内核已经集成了很多电容触摸 IC 的驱动文件，比如本章实验我们所使用 FT5426，本节我们就来学习一下，如何使用 Linux 内核自带的多点电容触摸驱动。在使用之前要先将前面我们自己添加到内核的 ft5x06.c 这个文件从内核中去除掉，只需要修改 drivers/input/touchscreen/Makefile 这个文件即可，将下面这一行删除掉：

```
obj-y       += ft5x06.o
```

内核自带的 FT5426 的驱动文件为 drivers/input/touchscreen/edt-ft5x06.c，此驱动文件不仅仅能够驱动 FT5426，FT5206、FT5406 这些都可以驱动。按照如下步骤来操作，学习如何使用此驱动。

1、使能内核自带的 FT5X06 驱动

edt-ft5x06.c 这个驱动默认是使能的，我们还是要教一下如何配置 Linux 内核，使能此驱动，通过图形化配置界面即可完成配置。进入 linux 内核源码目录，输入如下所示命令打开图形化配置界面：

```
make menuconfig
```

配置路径如下：

```
-> Device Drivers
    -> Input device support
        -> Generic input layer (needed for keyboard, mouse, ...) (INPUT [=y])
            -> Touchscreens (INPUT_TOUCHSCREEN [=y])
                -> <*>      EDT FocalTech FT5x06 I2C Touchscreen support
```

配置如图 47.7.1 所示：

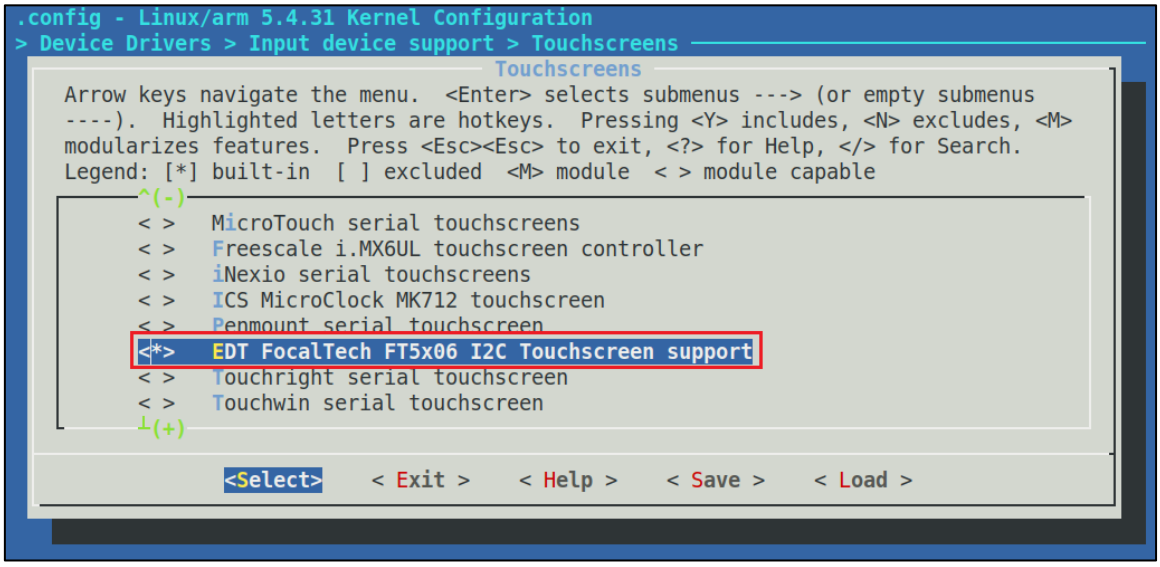


图 47.7.1 使能内核自带的 FT5X06 驱动

可以看出来 ST 官方已经把 FT5X06 的驱动编译进内核了，我们不用修改。要修改设备树和此驱动进行匹配。

2、修改设备树

修改我们在 47.4.1 中编写的 ft5426 这个设备节点，需要在里面添加 compatible 属性，添加的内容就要参考 edt-ft5x06.c 文件了，edt-ft5x06.c 所支持的 compatible 属性列表如下所示：

示例代码 47.7.1 edt-ft5x06.c 的 compatible 兼容属性列表

```
static const struct of_device_id edt_ft5x06_of_match[] = {
    { .compatible = "edt,edt-ft5206", .data = &edt_ft5x06_data },
    { .compatible = "edt,edt-ft5306", .data = &edt_ft5x06_data },
    { .compatible = "edt,edt-ft5406", .data = &edt_ft5x06_data },
    { .compatible = "edt,edt-ft5506", .data = &edt_ft5506_data },
    { .compatible = "evervision,ev-ft5726", .data = &edt_ft5506_data },
    /* Note focaltech vendor prefix for compatibility with ft6236.c */
    { .compatible = "focaltech,ft6236", .data = &edt_ft6236_data },
    { /* sentinel */ }
};
```

可以看出，edt-ft5x06.c 文件默认支持的 compatible 属性只要六个“edt,edt-ft5206”、“edt,edt-ft5306”和“edt,edt-ft5406”等等。因此我们需要修改设备树中的 ft5426 节点，修改以后的节点内容如下所示：

示例代码 47.7.2 ft5426 节点内容

```
1   ft5246: ft5426@38 {
2       compatible = "edt,edt-ft5406";
3       pinctrl-0 = <&ft5426int_reset_pins_a>;
4       reg = <0x38>;
5       interrupt-parent = <&gpioi>;
6       interrupts = <1 IRQ_TYPE_EDGE_RISING>;
8       reset-gpios = <&gpioh 15 GPIO_ACTIVE_LOW>;
9       status = "okay";
```

```
10      };
```

第 2 行，添加一条“edt,edt-ft5406”兼容性值。

修改完成以后重新编译设备树，然后使用新得到的.dtb 启动 linux 内核。如果一切正常的话系统启动的时候就会输出如图 47.7.2 所示信息：

```
stm32f7-i2c 40013000.i2c: using dma1chan6 (tx) and dma1chan7 (rx) for DMA transfers
i2c i2c-0: Added multiplexed i2c bus 1
input: generic ft5x06 (82) as /devices/platform/soc/40013000.i2c/i2c-0/0-0038/input/input0
stm32f7-i2c 40013000.i2c: STM32F7 I2C-0 bus adapter
stm32f7-i2c 40015000.i2c: using dma2chan0 (tx) and dma2chan1 (rx) for DMA transfers
```

图 47.7.2 触摸屏 log 信息

直接运行 ts_test_mt 来测试触摸屏是否可以使用。如果触摸点不对可运行 ts_calibrate 进行校准。至此，关于 Linux 下的多点电容触摸驱动就结束了，重点就是掌握 linux 下的触摸屏上报时序，大多数都是 Type B 类型。

47.8 4.3 寸屏触摸驱动实验

正点原子有两款 4.3 寸电容触摸屏，分辨率分别为 800*480 和 480*272，这两款电容触摸屏的触摸驱动 IC 都是 GT9147，因此本质上就是编写 GT9147 驱动。原理和方法基本和前面讲的 7 寸屏所使用的 FT5426 一样，这里我们只简单讲解一下 GT9147 的驱动编写步骤。

1、在设备树的 i2c2 节点下添加 gt9147 子节点

第一肯定是修改设备树，在 i2c2 的节点下添加一个 gt9147 的子节点。添加内容如下：

示例代码 47.8.1 gt9147 子节点内容

```
1   gt9147: gt9147@14 {
2       compatible = "atk-gt9147";
3       reg = <0x14>;
4       interrupt-parent = <&gpioi>;
5       interrupts = <1 IRQ_TYPE_EDGE_RISING>;
6       interrupt-gpios = <&gpioi 1 GPIO_ACTIVE_LOW>;
7       reset-gpios = <&gpioh 15 GPIO_ACTIVE_LOW>;
8       status = "okay";
9   };
```

在我们使用的 ST 官方的内核配置，已经把 gt9147 这个驱动编译进内核了，所以我们的 compatible 的属性值为“atk-gt9147”，免得使用内核的 gt9147 驱动代码。

2、添加屏的参数

不同的屏幕其配置参数也不同，因此需要在 drivers/gpu/drm/panel/panel-simple.c 文件下，添加对应屏幕的参数，4.3 寸 800*480 和 480*272 这两款屏幕参数如下所示：

示例代码 47.8.2 添加屏的参数

```
1   static const struct drm_display_mode ATK4384_mode = {
2       .clock = 33300,
3       .hdisplay = 800,
4       .hsync_start = 800 + 88,
5       .hsync_end = 800 + 88 + 48,
6       .htotal = 800 + 88 + 48 + 40,
7       .vdisplay = 480,
8       .vsync_start = 480 + 32,
```

```
9       .vsync_end = 480 + 32 + 3,
10      .vtotal = 480 + 32 + 3 + 13,
11      .vrefresh = 60,
12  };
13  static const struct drm_display_mode ATK4342_mode = {
14      .clock = 9000,
15      .hdisplay = 480,
16      .hsync_start = 480 + 40,
17      .hsync_end = 480 + 40 + 1,
18      .htotal = 480 + 40 + 1 + 5,
19      .vdisplay = 272,
20      .vsync_start = 272 + 8,
21      .vsync_end = 272 + 8 + 1,
22      .vtotal = 272 + 8 + 1 + 8,
23      .vrefresh = 60,
24  };
25  static const struct panel_desc alientek_desc = {
26      .modes = &ATK4342_mode,
27      .num_modes = 1,
28      .bus_format = MEDIA_BUS_FMT_RGB888_1X24,
29  };
```

第 1-12 行和 13-24 行分别为 800*480 和 480*272 这两个屏的参数，第 26 行，我们本来的 modes 属性为“ATK7016_mode”，如果我的屏为 480*272 就把 modes 属性修改为 ATK4342_mode 即可。

3、编译 GT9147 驱动文件

GT9147 的驱动编写这里就不再详细的讲解了，基本和 FT5426 一样，除了芯片配置的不同。GT9147 驱动已经写好并放到了本章教程的驱动文件夹中，驱动文件名字为：23_multitouch->gt9147.c，大家自行编译即可。编译完成以后加载得到的 gt9147.ko 驱动文件，然后使用 tslib 进行测试。如果大家在使用 tslib 的时候提示如图 47.8.1 所示错误：

```
[root@ATK-stm32mp1]:/etc$ ts_test_mt
ts_setup: No such file or directory
```

图 47.8.1 tslib 执行报错

上图中我们使用 ts_test_mt 来测试多点电容触摸，但是此时提示 ts_setup 错误。这个时候大家先用 hexdump 查看一下原始触摸数据有没有。如果有的话就是 tslib 配置错误，没有指定对触摸设备。打开/etc/profile 文件，输入如下这行即可：

```
export TSLIB_TSDEVICE=/dev/input/event1
```

其中 LIB_TSDEVICE 表示触摸屏对应的设备，这里设置为/dev/input/event1，大家要根据自己的实际情况来设置。

注意，gt9147.c 里面的驱动是单点触摸的，因为 GT9147 没有硬件检测每个触摸点的按下和抬起，因此在上报数据的时候不好处理。尝试过一些其他的处理方法，但是效果都不理想，因此改为了单点触摸。如果想直接使用 Linux 自带的 gt9147 驱动，直接修改设备树的 compatible 属性值为“goodix,gt9147”即可。

第四十八章 Linux 音频驱动实验

音频是我们最常用到的功能，音频也是 linux 和安卓的重点应用场合。STM32MP1 带有 SAI 接口，正点原子的 STM32MP1 开发板通过此接口外接了一个 CS42L51 音频 DAC 芯片，本章我们就来学习一下如何使能 CS42L51 驱动，并且 CS42L51 通过芯片来完成音乐播放与录音。

48.1 音频接口简介

48.1.1 为何需要音频编解码芯片？

处理器要想“听到”外界的声音必须要把外界的声音转化为自己能够理解的“语言”，处理器能理解的就是 0 和 1，也就是二进制数据。所以我们需要先把外界的声音转换为处理器能理解的 0 和 1，在信号处理领域，外界的声音是模拟信号，处理器能理解的是数字信号，因此这里就涉及到一个模拟信号转换为数字信号的过程，而完成这个功能的就是 ADC 芯片。

同理，如果处理器要向外界传达自己的“心声”，也就是放音，那么就涉及到将处理器能理解的 0 和 1 转化为外界能理解的连续变化的声音，这个过程就是将数字信号转化为模拟信号，而完成这个功能的是 DAC 芯片。

现在我们知道了，处理器如果既想“听到”外界的声音，又想向外界传达自己的“心声”，那么就需要同时用到 DAC 和 ADC 这两款芯片。那是不是买两颗 DAC 和 ADC 芯片就行了呢？答案肯定是可以的，但是音频不单单是能出声、能听到就行。我们往往需要听到的声音动听、录进去的语音贴近真实、可以调节音效、对声音能够进行一些处理(需要 DSP 单元)、拥有统一的标准接口，方便开发等等。将这些针对声音的各种要求全部叠加到 DAC 和 ADC 芯片上，那么就会得到一个专门用于音频的芯片，也就是音频编解码芯片，英文名字就是 Audio CODEC，所以我们在手机或者电脑的介绍中看到“CODEC”这个词语，一般说的都是音频编解码。

既然音频 CODEC 的本质是 ADC 和 DAC，那么采样率和采样位数就是衡量一款音频 CODEC 最重要的指标。比如常见音频采样率有 8K、44.1K、48K、192K 甚至 384K 和 768K，采样位数常见的有 8 位、16 位、24 位、32 位。采样率和采样位数越高，那么音频 CODEC 越能真实的还原声音，也就是大家说的 HIFI。因此大家会看到高端的音频播放器都会有很高的采样率和采样位数，同样的价格也会越高。当然了，实际的效果还与其他部分有关，采样率和采样位数只是其中重要的指标之一。

48.1.2 CS42L51 简介

前面我们已经分析了为何需要音频编解码芯片，那是因为专用的音频编解码芯片提供了很多针对音频的特性。我们就以正点原子 STM32MP157 开发板所使用的 CS42L51 这颗芯片为例，来看一下专用的音频编解码芯片都有哪些特性。

CS42L51 是一颗由 Cirrus 公司出品的音频编解码芯片，是一颗低功耗、高质量的立体声音频 CODEC。CS42L51 的最主要特点在于它只需要 1.8V 的低电压供电，但是却可以输出立体声，可以为 16Ω 耳机提供 46mW 的输出能量。在 2.5V 的时候可以 16Ω 耳机提供 88mW 的输出能量，CS42L51 主要特性如下所示：

①、24 位转换器。

②、支持 4kHz~96kHz 的采样率。

③、多位 Delta Sigma 架构。

④、低功耗，立体声播放的时候 1.8V 电压下功耗为 12.93mW。录音与播放一起工作时为 20.18mW。

⑤、数字信号处理引擎。

⑥、3:1 立体声输入选择器。

⑦、带有溢漫的自动水平监控。

……

CS42L51 整体框架如图 48.1.2.1 所示：

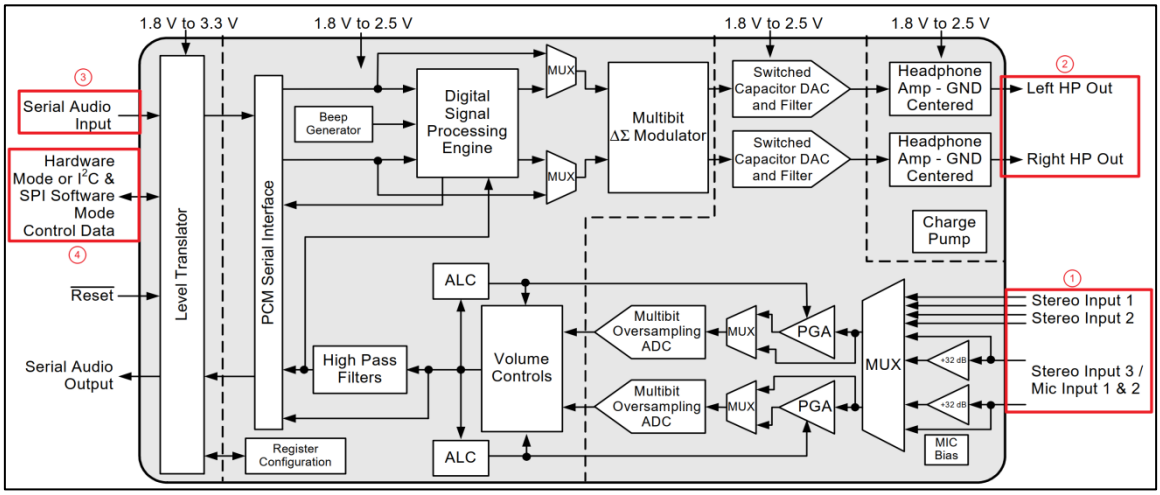


图 48.1.2.1 CS42L51 框图

依次来看一下图 48.1.2.1 中这四部分接口都是什么功能：

①、此部分是 CS42L51 提供的输入接口，作为立体声音频输入源，一共提供了三路，分别为 Stereo Input1、Stereo Input2、Stereo Input3/Mic INPUT1&2。麦克风或线路输入就连接到那个接口上，这部分是需要硬件工程师重点关心的，因为音频选择从哪一路进入需要在画 PCB 的时候就应该定好。

②、此部分是 CS42L51 提供的输出接口，只支持耳机输出接口。如果要实现功放就要外接其他电路。

③、此部分是数字音频接口，用于和主控器连接，使用的接口为 I2S。

④、此部分为控制接口，是一个标准的 I2C 接口， CS42L51 要想工作必须对其进行配置，这个 I2C 接口就是用于配置 CS42L51 的。

48.1.3 I2S 总线接口

I2S(Inter-IC Sound)总线有时候也写作 IIS，I2S 是飞利浦公司提出的一种用于数字音频设备之间进行音频数据传输的总线。和 I2C、SPI 这些常见的通信协议一样，I2S 总线用于主控制器和音频 CODEC 芯片之间传输音频数据。因此，要想使用 I2S 协议，主控制器和音频 CODEC 都得支持 I2S 协议，STM32MP1 的 SAI 外设就支持 I2S 协议，CS42L51 同样也支持 I2S，所以本章实验就是使用 I2S 协议来完成的。I2S 接口需要 3 根信号线(如果需要实现收和发，那么就要 4 根信号线，收和发分别使用一根信号线)：

SCK：串行时钟信号，也叫做位时钟(BCLK)，音频数据的每一位数据都对应一个 SCK，立体声都是双声道的，因此 SCK=2×采样率×采样位数。比如采样率为 44.1KHz、16 位的立体声音频，那么 SCK=2×44100×16=1411200Hz=1.4112MHz。

WS：字段(声道)选择信号，也叫做 LRCK，也叫做帧时钟，用于切换左右声道数据，WS 为“1”表示正在传输左声道的数据，WS 为“0”表示正在传输右声道的数据。WS 的频率等于采样率，比如采样率为 44.1KHz 的音频，WS=44.1KHz。

SD：串行数据信号，也就是我们实际的音频数据，如果要同时实现放音和录音，那么就需要 2 根数据线，比如 CS42L51 的 SDOUT 和 SDIN，就是分别用于放音和录音。不管音频数据是多少位的，数据的最高位都是最先传输的。数据的最高位总是出现在一帧开始后(LRCK 变化)的第 2 个 SCK 脉冲处。

另外，有时候为了使音频 CODEC 芯片与主控制器之间能够更好的同步，会引入另外一个

叫做 MCLK 的信号，也叫做主时钟或系统时钟，一般是采样率的 256 倍或 384 倍。

图 48.1.3.1 就是一帧立体声音频时序图：

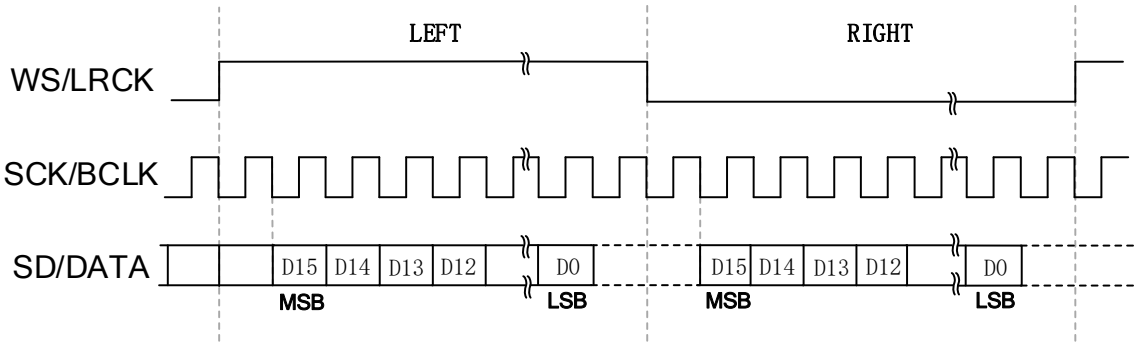


图 48.1.3.1 I2S 时序图

图 48.1.3.2 就是笔者采用逻辑分析仪抓取到的一帧真实的音频时序图：

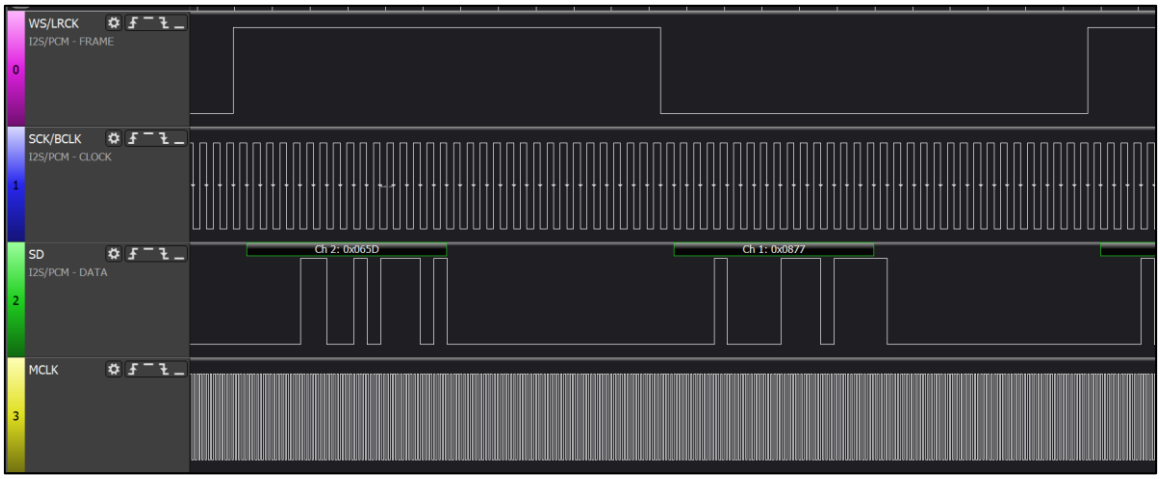


图 48.1.3.2 真实的 I2S 时序图

图 48.1.3.2 中通道 0 是 LRCK 时钟，通道 1 为 BCLK，通道 2 是 DACDATA，通道 3 是 MCLK。随着技术的发展，在统一的 I2S 接口下，出现了不同的数据格式，根据 DATA 数据相对于 LRCK 和 SCLK 位置的不同，出现了 Left Justified(左对齐)和 Right Justified(右对齐)两种格式，这两种格式的时序图如图 48.1.3.3 所示：

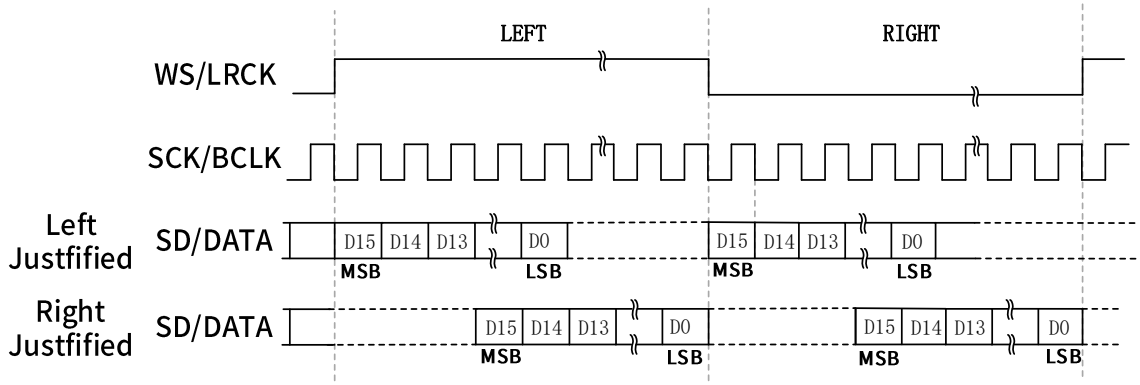


图 48.1.3.3 I2S 左对齐和右对齐数据格式

48.1.4 STM32MP1 SAI 总线接口

SAI 接口(串行音频接口)灵活性高、配置多样，可支持多种音频协议。该接口适用许多立体声或单声道应用。支持 I2S 标准、LSB 或 MSB 对齐、PCM/DSP、TDM 等等协议。全称为 Synchronous Audio Interface，SAI 的特性：

①、有两个独立的音频子模块，子模块可以用作接收和发送功能，并且带有自身的 FIFO。
②、每个音频子模块集成多达 8 个字，每个字 32 位的 FIFO。
③、两个音频子模块间可以是同步或异步模式。
④、多个 SAI 之间看实现同步。
⑤、两个音频子模块的主/从配置相互独立。
⑥、数据大小可配置：8 位、10 位、16 位、20 位、24 位或 32 位。
……

SAI 框架图如 48.1.4.1 所示：

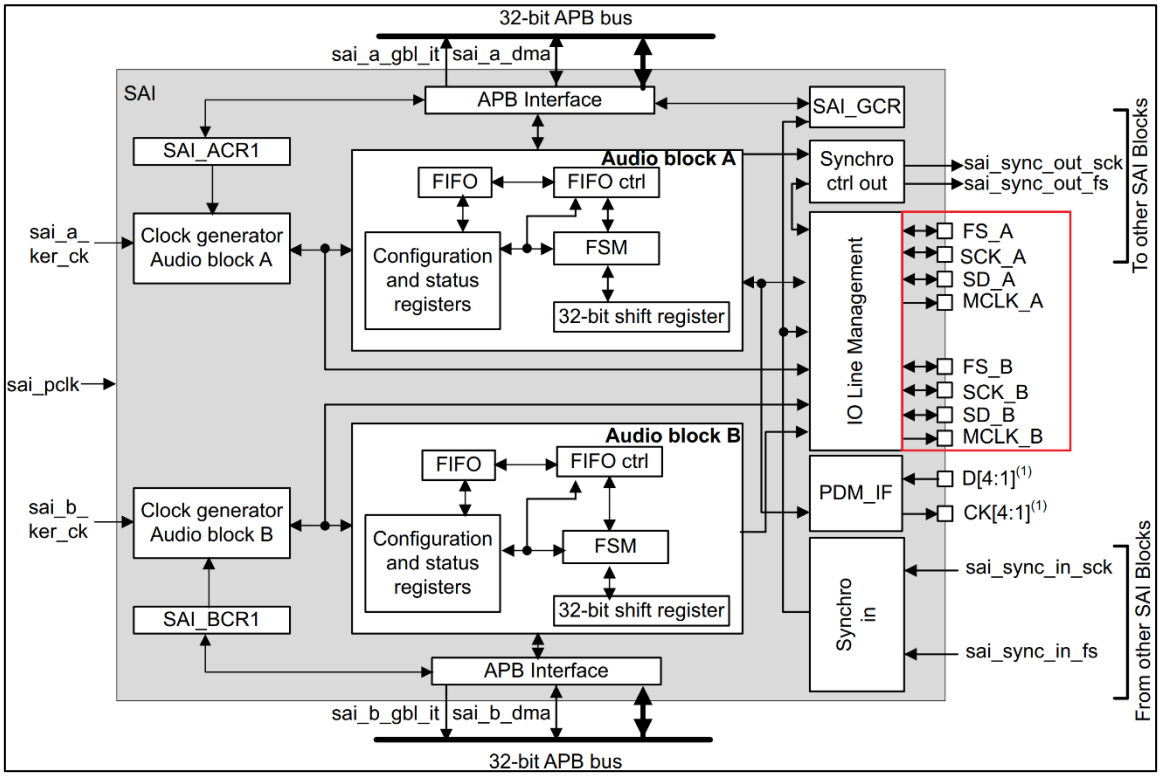


图 48.1.4.1 SAI 框架图

从图中可以看出 SAI 是由两个各自带有时钟发生器的音频子模块组成。在红色框架里标明了每个子模块都有 4 个专用引脚控制(SD、SCK、FS、MCLK)。如果将两个子模块声明为同步模块，那么 SCK、FS 和 MCLK 就可以共用引脚，不相同的只有数据 SD 引脚了。

48.2 硬件原理图分析

正点原子 STM32MP1 开发板音频原理图如图 48.2.1 所示：

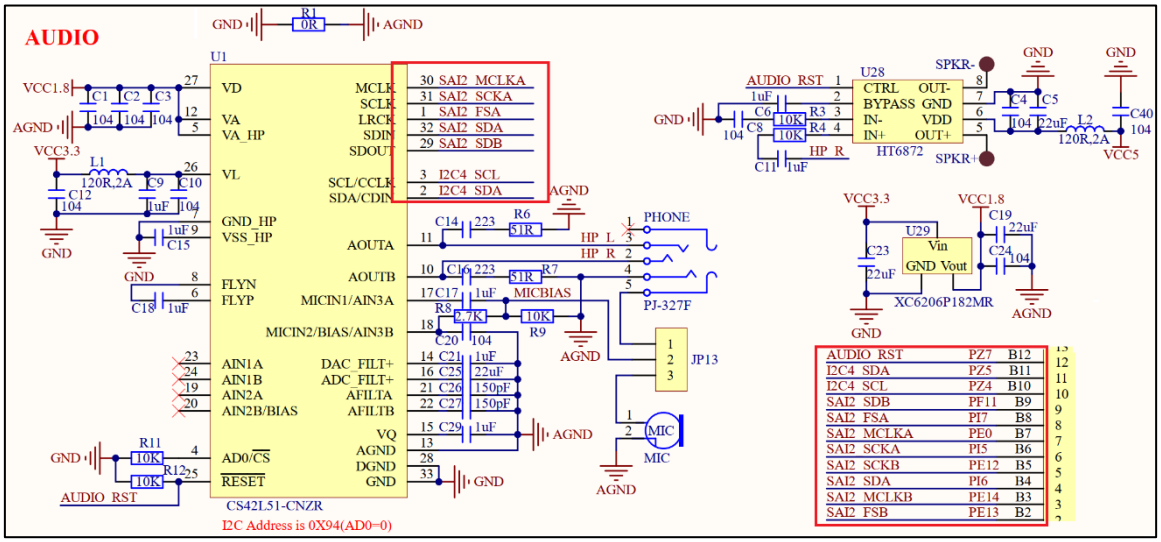


图 48.2.1 音频原理图

图 48.2.1 中我们重点关注有三方面 SAI、I2C 和 HT6872，我们依次来看一下这个三方面：

①、SAI 接口一共用到了 5 根数据线，这个 5 根数据线用于 STM32MP1 与 CS42L51 之间的音频数据收发。

②、CS42L51 在使用的时候需要进行配置，配置接口为 I2C，连接到 STM32MP1 的 I2C4 上。

③、CS42L51 是没有功放接口的，我们就要提供一个 HT6872 把 HP_R 耳机数据放大。

最后音频的耳机和 MIC 录音功能是有 JP13 来控制的，如果 1-2 连接就使用耳机进行录音，2-3 连接就使用板载 MIC 录音。

48.3 音频驱动使能

ST 官方已经写好了 CS42L51 驱动，因此我们直接配置内核使能 CS42L51 驱动即可，按照如下所示步骤使能 CS42L51 驱动。

48.3.1 修改设备树

前面分析原理图的时候已经说过了，CS42L51 与 STM32MP1 之间有两个通信接口：I2C 和 SAI，因此设备树中会涉及到 I2C 和 SAI 两个设备节点。其中 I2C 用于配置 CS42L51，SAI 接口用于音频数据传输，我们依次来配置一下这两个接口。

1、CS42L51 i2c 接口设备树

首先配置一下 I2C 接口，根据原理图我们知道 CS42L51 连接到了 STM32MP1 的 I2C4 接口上，因此在设备树中的“i2c4”节点下需要添加 cs41l51 信息。打开 Documentation/devicetree/bindings/sound/cs42l51.txt，此文件仅仅用于描述如何在 I2C 节点下添加 cs41l51 相关信息，此文档适用于所有的主控，不局限于 STM32MP1。

相关属性如下：

①、**Compatible**：兼容属性，属性值要设置为“cirrus,cs42l51”。大家在 Linux 内核全局搜索“cirrus,cs42l51”的话就会找到 cs42l51 的 I2C 驱动为，此文件为：sound/soc/codecs/cs42l51.c。

②、**reg**：设置 cs42l51 的地址，在正点原子的 STM32MP1 开发板中 cs42l51 的 I2C 地址为 0x4a。

③、**clocks**：时钟。

④、**clock-names**：比如为“MCLK”。

⑤、**XX-supply**：这些都是电源相关的属性。指定芯片的电压，这里一般给 1.8v。所以我们还需要添加一个 1.8v 的电源节点。

⑥、**reset-gpios**：指定芯片的复位引脚，根据原理图可知复位引脚连接在 PZ7 上。

绑定文档给出的参考节点内容如下所示：

示例代码 48.3.1.1 CS42L51 节点参考设置

```
1  cs42l51: cs42l51@4a {
2      compatible = "cirrus,cs42l51";
3      reg = <0x4a>;
4      clocks = <&mclk_prov>;
5      clock-names = "MCLK";
6      VL-supply = <&reg_audio>;
7      VD-supply = <&reg_audio>;
8      VA-supply = <&reg_audio>;
9      VAHP-supply = <&reg_audio>;
10     reset-gpios = <&gpiog 9 GPIO_ACTIVE_LOW>;
11 };
```

根据 cs42l51.txt 这份绑定文档我们就可以在任意一个主控的 I2C 节点下添加 cs42l51 相关信息了，ST 官方 STM32M157C-DK2 开发板使用的也是 CS42L51，因此我们可以直接拿来使用，要注意官方的 cs42l51 是使用 I2C1 来配置的，我们的开发板是使用 I2C4 来配置的。打开 stm32mp157d-atk.dts 文件，先里面创建 I2C4 节点，然后将 arch/arm/boot/dts/stm32mp15xx-dkx.dtsi 文件中 ST 为官方 DK2 开发板编写的 CS42L51 节点复制到 I2C4 节点下。完成以后的 IC4 和 CS42L51 节点内容如下所示：

示例代码 48.3.1.2 添加 cs42l51 节点后的 i2c4 节点

```
1  &i2c4 {
2      pinctrl-names = "default", "sleep";
3      pinctrl-0 = <&i2c4_pins_a>;
4      pinctrl-1 = <&i2c4_pins_sleep_a>;
5      status = "okay";
6      /delete-property/dmas;
7      /delete-property/dma-names;
8      cs42l51: cs42l51@4a {
9          compatible = "cirrus,cs42l51";
10         reg = <0x4a>;
11         #sound-dai-cells = <0>;
12         VL-supply = <&v3v3>;
13         VD-supply = <&v1v8_audio>;
14         VA-supply = <&v1v8_audio>;
15         VAHP-supply = <&v1v8_audio>;
16         reset-gpios = <&gpioz 7 GPIO_ACTIVE_LOW>;
17         clocks = <&sai2a>;
18         clock-names = "MCLK";
19         status = "okay";
```

```
20
21          cs42l51_port: port {
22              #address-cells = <1>;
23              #size-cells = <0>;
24
25              cs42l51_tx_endpoint: endpoint@0 {
26                  reg = <0>;
27                  remote-endpoint = <&sai2a_endpoint>;
28                  frame-master;
29                  bitclock-master;
30              };
31
32              cs42l51_rx_endpoint: endpoint@1 {
33                  reg = <1>;
34                  remote-endpoint = <&sai2b_endpoint>;
35                  frame-master;
36                  bitclock-master;
37              };
38          };
39      };
40  };
```

示例代码 48.3.1.2 里，我们新追加了一个 I2C4 节点和添加了 i2c4 的一个 cs42l51 子节点。

第 3、4 行，设置 I2C4 的引脚，要根据自己的实际硬件情况设计，正点原子 STM32MP157 开发板上 I2C4_SDA 和 I2C4_SCL 分别为 PZ5 和 PZ4。stm32mp15-pinctrl.dtsi 文件中 I2C4 引脚默认配置的就是 PZ5 和 PZ4，因此无需修改。

第 8~39 行，就是 cs42l51 子节点。

第 8~19 行，就是跟我们的 cs42l51.txt 文档一样，只是将复位引脚改为了 PZ7，这里就不多说了。重点是第 13~15 行，多了个 1.8v 的电源管理，所以我们需要添加一个 1.8v 的节点，稍后添加。

第 21~38 行，cs42l51_port 子节点用来设置 CS42L51 的数据输入/输出端口信息，第 25~30 行 cs42l51_tx_endpoint 端口用来接收 CPU 上哪个 SAI 的数据，第 27 行就是告诉 CS42L51 从 sai2a_endpoint 获取音频数据，此接口是用来播放音频的通道。第 32~37 行 cs42l51_rx_endpoint 端口用来将获取到的音频数据发送到 CPU 的哪个 SAI 接口上，第 34 行就是告诉 CS42L51 从外面获取的音频数据传输到 sai2b_endpoint，此接口是用来录音的通道。

最后我们还需要在根节点里添加一个 v1v8_audio 的电源管理。如下示例代码所示：

示例代码 48.3.1.3 追加的 v1v8_audio 节点内容

```
1   v1v8_audio: regulator-v1v8-audio {
2       compatible = "regulator-fixed";
3       regulator-name = "v1v8_audio";
4       regulator-min-microvolt = <1800000>;
5       regulator-max-microvolt = <1800000>;
6       regulator-always-on;
7       regulator-boot-on;
```

```
8    };
```

至此，关于 cs42l51 的 I2C 配置接口设备树就已经添加好了。

2、STM32MP1 SAI 音频接口设备树

接下来就是 STM32MP1 的 SAI 音频接口设备树相关内容的修改了，同样的，先查阅一下相应的绑定文档：devicetree/bindings/sound/st,stm32-sai.txt。和我们前面讲过的 IIC 接口、SPI 等接口一样，在 stm32mp151.dtsi 文件中会有关于 SAI 相关接口的描述，这部分是 ST 原厂编写的，我们不需要做任何修改，SAI2 的设备子节点内容如下所示：

示例代码 48.3.1.4 STM32MP1 SAI 接口子节点

```
1      sai2: sai@4400b000 {
2          compatible = "st,stm32h7-sai";
3          #address-cells = <1>;
4          #size-cells = <1>;
5          ranges = <0 0x4400b000 0x400>;
6          reg = <0x4400b000 0x4>, <0x4400b3f0 0x10>;
7          interrupts = <GIC_SPI 91 IRQ_TYPE_LEVEL_HIGH>;
8          resets = <&rcc SAI2_R>;
9          status = "disabled";
10
11         sai2a: audio-controller@4400b004 {
12             #sound-dai-cells = <0>;
13             compatible = "st,stm32-sai-sub-a";
14             reg = <0x4 0x1c>;
15             clocks = <&rcc SAI2_K>;
16             clock-names = "sai_ck";
17             dmas = <&dmamux1 89 0x400 0x01>;
18             status = "disabled";
19         };
20
21         sai2b: audio-controller@4400b024 {
22             #sound-dai-cells = <0>;
23             compatible = "st,stm32-sai-sub-b";
24             reg = <0x24 0x1c>;
25             clocks = <&rcc SAI2_K>;
26             clock-names = "sai_ck";
27             dmas = <&dmamux1 90 0x400 0x01>;
28             status = "disabled";
29         };
30     };
```

在示例代码 48.3.1.4 中 sai2 节点有两个子节点，分别为 sai2a 和 sai2b，刚好对应我们的两个子模块。直接搜索“st,stm32h7-sai”兼容值，那么你就会找的 STM32MP1 的 SAI 接口驱动文件，路径为 sound/soc/stm/stm32_sai.c，此驱动文件不需要我们去研究，除非你在 ST 上班，而且你的工作恰好是给 ST 的 STM32MP1 系列芯片编写 SAI 驱动的。从第 9 行可以看出，SAI2 默认是关闭的，因此我们需要将其打开，也就是设置 status 属性的值为“okay”，这个工作肯定是

在具体板子对应的.dts 文件中完成的，其实就是向 sai2 节点以及对应子节点里面追加或者修改一些属性值。

前面说了，ST 官方的 DK2 开发板也使用了 CS42L51，因此 DK2 开发板的设备树文件 stm32mp15xx-dkx.dtsi 里面肯定有具体的 SAI2 节点信息。我们可以直接将其复制到 stm32mp157d-atk.dts 文件里面，内容如下：

示例代码 48.3.1.5 拷贝的 SAI2 节点内容

```
1  &sai2 {
2      clocks = <&rcc SAI2>, <&rcc PLL3_Q>, <&rcc PLL3_R>;
3      clock-names = "pclk", "x8k", "x11k";
4      pinctrl-names = "default", "sleep";
5      pinctrl-0 = <&sai2a_pins_a>, <&sai2b_pins_b>;
6      pinctrl-1 = <&sai2a_sleep_pins_a>, <&sai2b_sleep_pins_b>;
7      status = "okay";
8
9      sai2a: audio-controller@4400b004 {
10         #clock-cells = <0>;
11         dma-names = "tx";
12         clocks = <&rcc SAI2_K>;
13         clock-names = "sai_ck";
14         status = "okay";
15
16         sai2a_port: port {
17             sai2a_endpoint: endpoint {
18                 remote-endpoint = <&cs42l51_tx_endpoint>;
19                 format = "i2s";
20                 mclk-fs = <256>;
21                 dai-tdm-slot-num = <2>;
22                 dai-tdm-slot-width = <32>;
23             };
24         };
25     };
26
27     sai2b: audio-controller@4400b024 {
28         dma-names = "rx";
29         st,sync = <&sai2a 2>;
30         clocks = <&rcc SAI2_K>, <&sai2a>;
31         clock-names = "sai_ck", "MCLK";
32         status = "okay";
33
34         sai2b_port: port {
35             sai2b_endpoint: endpoint {
36                 remote-endpoint = <&cs42l51_rx_endpoint>;
37                 format = "i2s";
```

```
38                mclk-fs = <256>;
39                dai-tdm-slot-num = <2>;
40                dai-tdm-slot-width = <32>;
41            };
42        };
43    };
44 };
```

示例代码 48.3.1.5 中的内容，主要是对 sai2 节点做了四个方面描述：SAI2 接口引脚的 pinctrl 设置、时钟配置、修改 status 为“okay”以及指定收发接口。我们重点来看一下 pinctrl 的设置，因为关系到 SAI2 接口的 IO 配置，从 pinctrl-0 属性可以看出这里一共有两组 IO：sai2a_pins_a 和 sai2b_pins_b，打开 stm32mp15-pinctrl.dtsi 文件，这两组 IO 内容如下：

示例代码 48.3.1.6 sai2 引脚配置

```
1    sai2a_pins_a: sai2a-0 {
2        pins {
3            pinmux = <STM32_PINMUX('I', 5, AF10)>,   /* SAI2_SCK_A */
4                <STM32_PINMUX('I', 6, AF10)>,        /* SAI2_SD_A  */
5                <STM32_PINMUX('I', 7, AF10)>,        /* SAI2_FS_A  */
6                <STM32_PINMUX('E', 0, AF10)>;        /* SAI2_MCLK_A */
7            slew-rate = <0>;
8            drive-push-pull;
9            bias-disable;
10       };
11   };
12   sai2b_pins_b: sai2b-2 {
13       pins {
14           pinmux = <STM32_PINMUX('F', 11, AF10)>; /* SAI2_SD_B */
15           bias-disable;
16       };
17   };
```

sai2a_pins_a 和 sai2b_pins_b 就是描述的是 SAI2 接口的 IO 配置，这个要根据自己板子的实际硬件情况修改，正点原子的 STM32MP1 开发板上 SAI2 所使用的引脚和 ST 官方的 DK2 开发板一样，因此这里不需要做任何修改。

对于正点原子的 STM32MP1 开发板，SAI 部分的设备树信息不需要做任何修改，直接使用 ST 官方写好的即可。

3、STM32MP1 sound 节点

最后我们需要在根节点“/”下创建一个名为“sound”的子节点，此节点用作控制音频数据的收发。我们可以参考文档：Documentation/devicetree/bindings/sound/audio-graph-card.txt。此文档有很多种配置方法，还要根据你的音频选择对应的配置方法。ST 官方已经针对 DK2 开发板编写了 sound 节点，我们直接拷贝过来即可。在 stm32mp15xx-dkx.dtsi 文件中找到 sound 节点，然后将其拷贝到 stm32mp157d-atk.dts 根节点下，拷贝完成以后的 sound 节点内容如下所示：

示例代码 48.3.1.7 sound 节点内容

```
1    sound: sound {
2        compatible = "audio-graph-card";
```

```
3         label = "STM32MP1-DK";
4         routing =
5             "Playback" , "MCLK",
6             "Capture" , "MCLK",
7             "MICL" , "Mic Bias";
8         dais = <&sai2a_port &sai2b_port>;
9         status = "okay";
10    };
```

简单看一下 sound 节点中几个重要的属性：

compatible：非常重要，用于匹配相应的驱动文件，在整个 linux 内核源码中搜索这“audio-graph-card”属性值即可找到对应的驱动文件，这里找到的驱动文件为：sound/soc/generic/audio-graph-card.c。

model：最终用户看到的此声卡名字，这里设置为“STM32MP1-DK”。

routing：音频器件一系列的连接设置，每个条目都是一对字符串，第一个字符串是连接的 sink，第二个是连接的 source(源)。

dais：用来指定我们的音频有多少个接收和发送，sai2a_port 和 sai2b_port 就是 CS42L51 发送和接收的接口。

48.3.2 使能或修改内核的 CS42L51 驱动

设备树配置完成以后就可以使能内核自带的 CS42L51 驱动了，直接通过图形化界面配置即可，输入如下命令打开 linux 内核的图形化配置界面：

```
make menuconfig
```

ST 官方默认已经使能了 CS42L51 驱动，这里还是要教大家如何配置。

1、使能 CS42L51 驱动

配置路径如下：

```
-> Device Drivers
    -> Sound card support
        -> Advanced Linux Sound Architecture
            -> ALSA for SoC audio support
                -> CODEC drivers
                    -> <*> Cirrus Logic CS42L51 CODEC (I2C)(选中)
```

结果如图 48.3.2.1 所示：

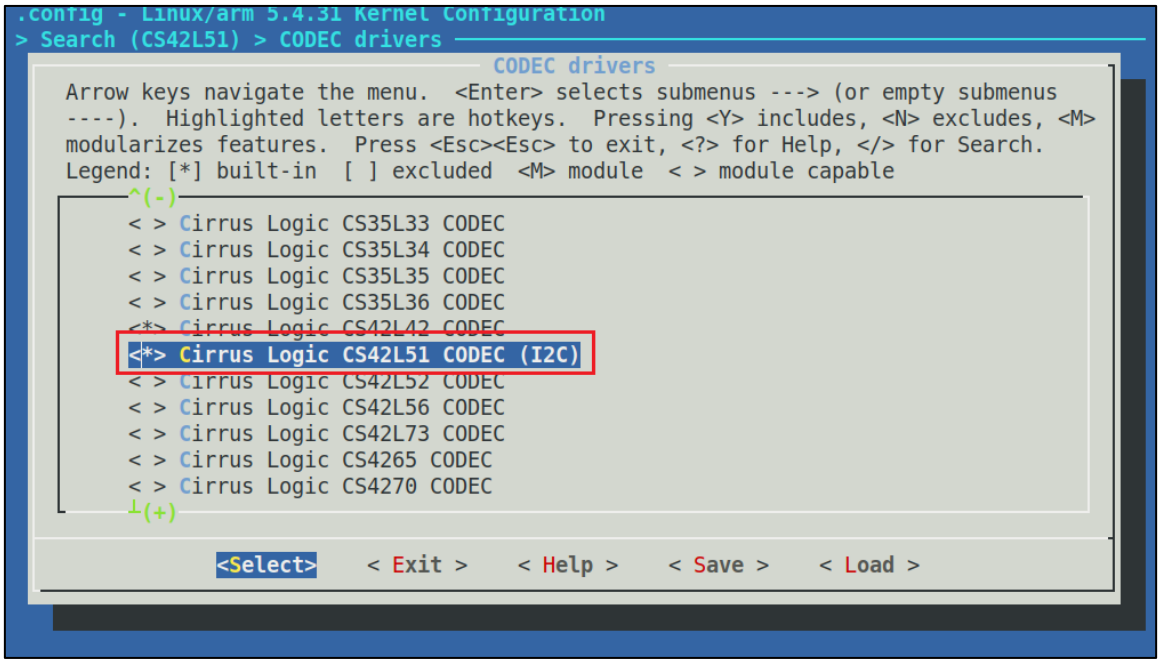


图 48.3.2.1 使能 CS42L51 驱动

2、使能 sound 驱动

接下来我们使能 sound，配置的路径如下所示：

-> Device Drivers
-> Sound card support
-> Advanced Linux Sound Architecture
-> ALSA for SoC audio support
-> <*> ASoC Audio Graph sound card support(选中)

结果如图 48.3.2.2 所示：

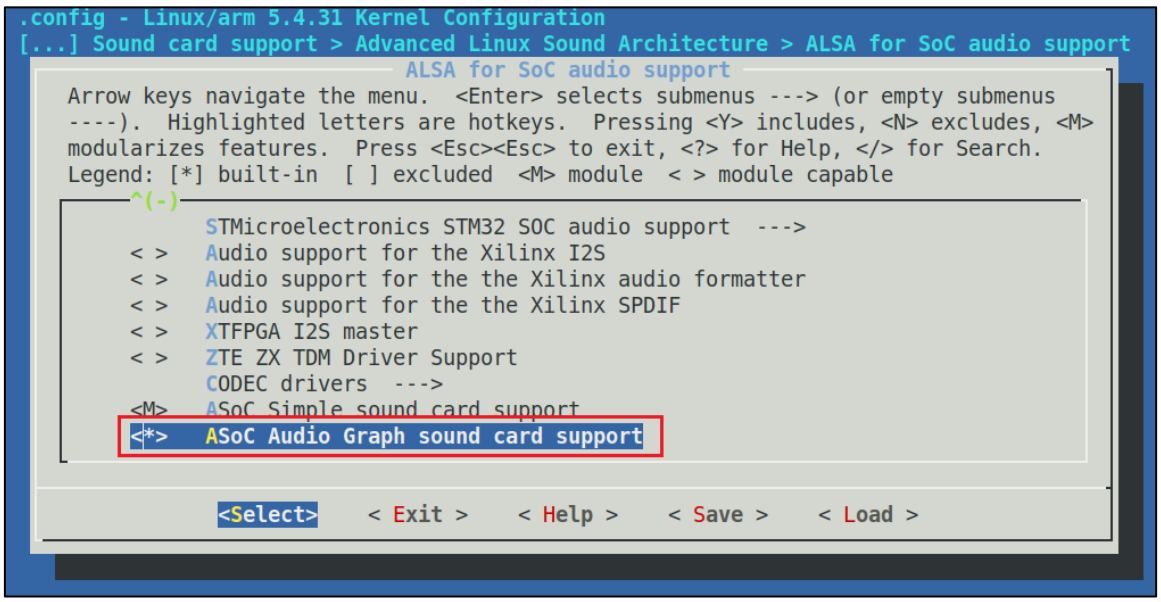


图 48.3.2.2 使能 sound 驱动

3、使能 SAI 驱动

最后我们去使能 SAI 接口：

```
-> Device Drivers
    -> Sound card support (SOUND [=y])
        -> Advanced Linux Sound Architecture (SND [=y])
            -> ALSA for SoC audio support (SND_SOC [=y])
                -> STMicroelectronics STM32 SOC audio support
                    -> <*> STM32 SAI interface (Serial Audio Interface) support（选中）
                    -> <*> STM32 I2S interface (SPI/I2S block) support （选中）
                    -> <*> STM32 S/PDIF receiver (SPDIFRX) support （选中）
                    -> <*> SoC Audio support for STM32 DFSDM （选中）
```

结果如图 48.3.2.3 所示：

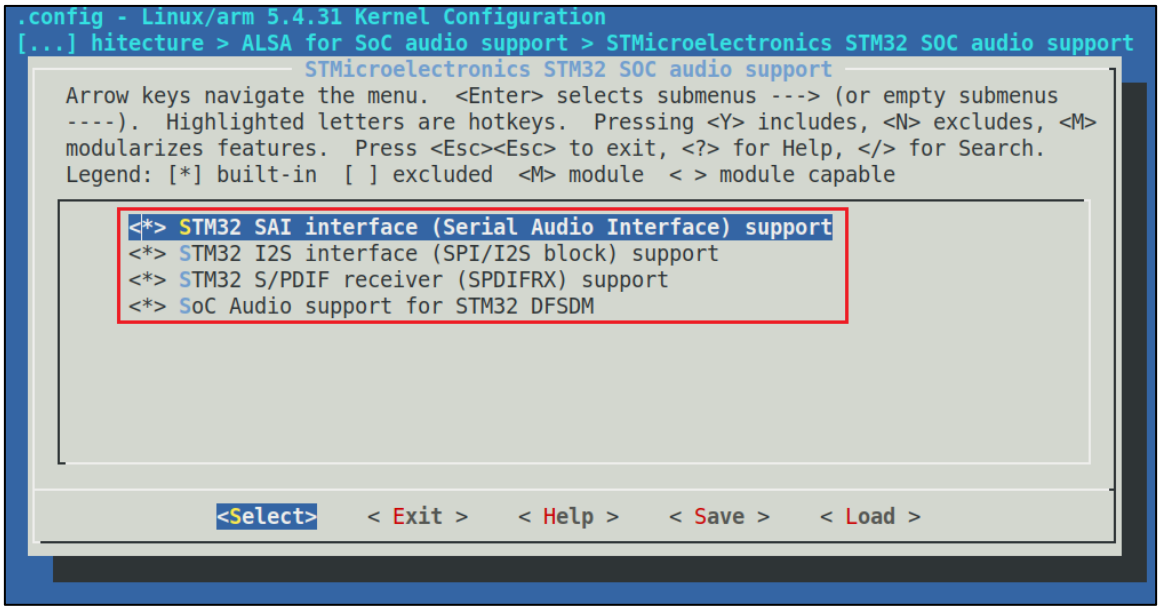


图 48.3.2.3 使能 SAI 驱动

4、修改 CS42L51 驱动

CS42L51 默认驱动的录音为单通道，所以我们要修改驱动来实现双通道录音。打开 sound/soc/codecs/cs42l51.c 文件，找到 cs42l51_hw_params 函数，将如下代码添加到第 477 行：

```
intf_ctl |= (1 << 0);
```

结果如图 48.3.2.4 所示：

```
473
474     if (ratios[i].mclk)
475         power_ctl |= CS42L51_MIC_POWER_CTL_MCLK_DIV2;
476
477     intf_ctl |= (1 << 0);    /* CTL寄存器bit0置1 */
478
479     ret = snd_soc_component_write(component, CS42L51_INTF_CTL, intf_ctl);
480     if (ret < 0)
481         return ret;
482
```

图 48.3.2.4 修改 CS42L51 驱动代码结果

图 48.3.2.4 中添加的内容是将 CS42L51 的 0X04 寄存器的 bit0 置 1。驱动使能以后重新编译 linux 内核，编译完成以后使用新的 uImage 和.dtb 文件启动，如果设备树和驱动都使能的话系统启动过程中就会如图 48.3.2.5 所示的 log 信息：

```
mmc0: new high speed SDHC card at address aaaa
mmcblk0: mmc0:aaaa SC16G 14.8 GiB
cs42l51 3-004a: Cirrus Logic CS42L51, Revision: 00
 mmcblk0: p1 p2
asoc-audio-graph-card sound: cs42l51-hifi <-> 4400b004.audio-controller mapping ok
asoc-audio-graph-card sound: cs42l51-hifi <-> 4400b024.audio-controller mapping ok
mmc1: new high speed MMC card at address 0001
```

图 48.3.2.5 CS42L51 信息

系统最终启动以后会打印出 ALSA 设备列表，现在的音频 CODEC 驱动基本都是 ALSA 架构的，本章的 CS42L51 驱动也是根据 ALSA 架构编写的。因此在 ALSA 设备列表中就会找到“STM32MP1-DK”这个声卡，如图 48.3.2.6 所示：

```
ALSA device list:
  #0: STM32MP1-DK
VFS: Mounted root (nfs filesystem) on device 0:17.
devtmpfs: mounted
```

图 48.3.2.6 CS42L51 声卡

进入系统以后查看一下/dev/snd 目录，看看有没有如图 48.3.2.7 所示文件：

```
[root@ATK-stm32mp1]:~$ cd /dev/snd/
[root@ATK-stm32mp1]:/dev/snd$ ls
controlC0  pcmC0D0p   pcmC0D1c   timer
[root@ATK-stm32mp1]:/dev/snd$
```

图 48.3.2.7 alsa 驱动设备文件

图 48.3.2.7 中的这些文件就是 ALSA 音频驱动框架对应的设备文件，这些文件的作用如下：

controlC0：用于声卡控制，C0 表示声卡 0。

pcmC0D0p：用于播放的 pcm 设备，最后面的“p”是 playback 的缩写，表示放音。

pcmC0D1c：用于录音的 pcm 设备，最后面的“c”是 capture 的缩写，表示录音。

timer：定时器。

音频驱动使能以后还不能直接播放音乐或录音，我们还需要移植 alsa-lib 和 alsa-utils 这两个东西。

48.4 alsa-lib 和 alsa-utils 移植

alsa-lib 这个库在 buildroot 已经默认编译进去我们可以不用管，我们只需要使能 alsa-utils 就行了，还是在 buildroot 的源码目录下，运行以下命令进入图形化界面配置：

```
make menuconfig
```

按照以下路径进入配置我们的 alsa-utils：

```
-> Target packages
    -> Audio and video applications
        -> [*] alsa-utils  (选中)
            -> [*] alsactl(选中)
            ……
            -> [*] speaker-test(选中)
```

把 alsa-utils 下的软件全部选中，结果如图 48.4.1 所示：

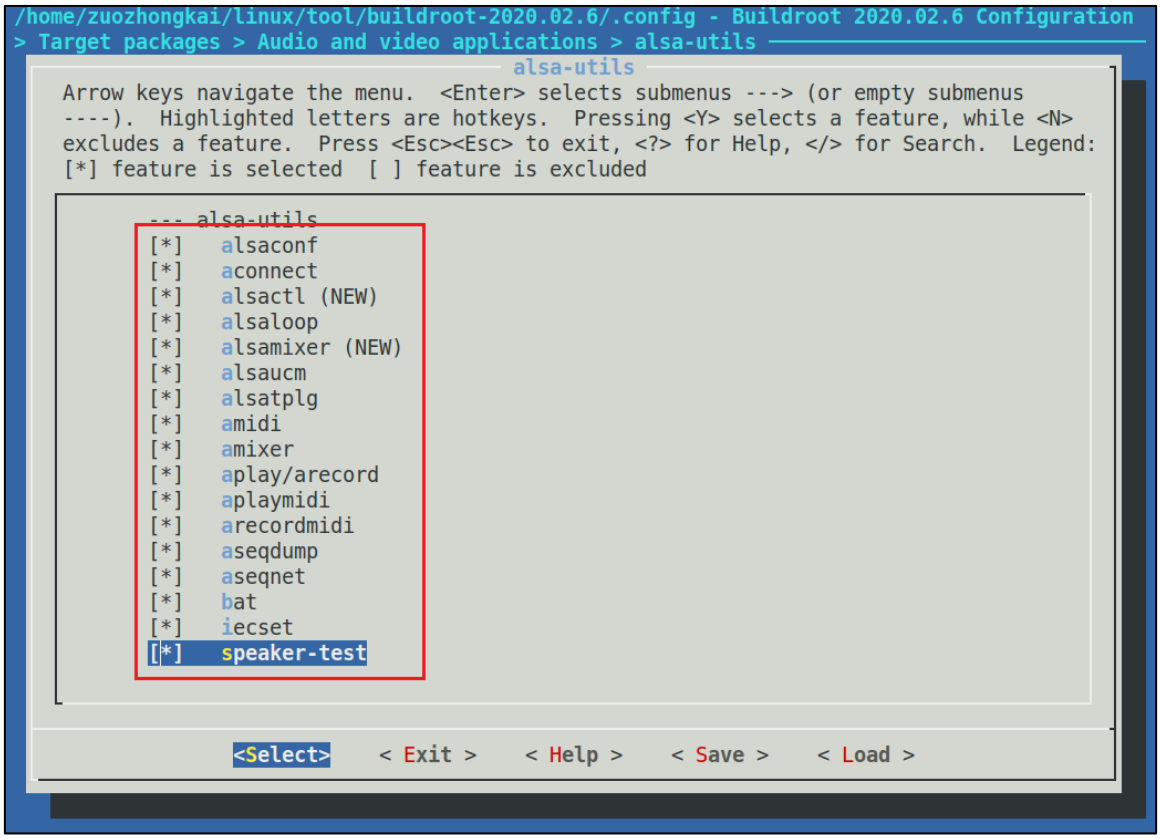


图 48.4.1 要配置的 alsa-utils

保存和退出配置选项，然后运行以下命令重新编译 buildroot：

```
sudo make
```

编译完成后，进入 output/images 目录，运行以下命令把文件系统替换进去：

```
cd output/images/                                          //进入到 output/images 目录
sudo tar -axvf rootfs.tar -C /home/zuozhongkai/linux/nfs/rootfs  //解压到 nfsroot 目录
```

上述命令将 buildroot 中 output/images/rootfs.tar 这个压缩包解压到/home/zuozhongkai/linux/nfs/rootfs 这个目录中，这个目录就是我当前 nfsroot 目录，大家根据自己的实际情况解压到对应的目录文件中。

重启开发板。

48.5 声卡设置与测试

48.5.1 amixer 使用方法

1、查看帮助信息

声卡相关选型默认都是关闭的，比如耳机和喇叭的左右声道输出等。因此我们在使用之前一定要先设置好声卡，alsa-utils 自带了 amixer 这个声卡设置工具。输入如下命令即可查看 amixer 的帮助信息：

```
amixer --help                    //查看 amixer 帮助信息
```

结果如图 48.5.1.1 所示：

```
[root@ATK-stm32mp1]:/$ amixer --help
Usage: amixer <options> [command]

Available options:
  -h,--help       this help
  -c,--card N     select the card
  -D,--device N   select the device, default 'default'
  -d,--debug      debug mode
  -n,--nocheck    do not perform range checking
  -v,--version    print version of this program
  -q,--quiet      be quiet
  -i,--inactive   show also inactive controls
  -a,--abstract L select abstraction level (none or basic)
  -s,--stdin      Read and execute commands from stdin sequentially
  -R,--raw-volume Use the raw value (default)
  -M,--mapped-volume Use the mapped volume

Available commands:
  scontrols       show all mixer simple controls
  scontents       show contents of all mixer simple controls (default command)
  sset sID P      set contents for one mixer simple control
  sget sID        get contents for one mixer simple control
  controls        show all controls for given card
  contents        show contents of all controls for given card
  cset cID P      set control contents for one control
  cget cID        get control contents for one control
[root@ATK-stm32mp1]:/$
```

图 48.5.1.1 amixer 帮助信息

从图 48.5.1.1 可以看出，amixer 软件命令分为两组，scontrols、scontents、sset 和 sget 为一一组。controls、contents、cset 和 cget 为另一组。这两组的基本功能都是一样的，只不过“s”开头的是 simple(简单)组，这一组命令是简化版，本教程最终使用“s”开头的命令设置声卡，因为少输入很多字符。

2、查看设置项

我们要先看一下都有哪些设置项，先来看一下 scontrols 对应的设置项，输入如下命令：

```
amixer scontrols                //查看所有设置项
```

结果如图 48.5.1.2 所示：

```
[root@ATK-stm32mp1]:/$ amixer scontrols
Simple mixer control 'Bass',0
Simple mixer control 'Treble',0
Simple mixer control 'PCM',0
Simple mixer control 'PCM channel mixer',0
Simple mixer control 'Mic Boost',0
Simple mixer control 'Playback Deemphasis',0
Simple mixer control 'ADC Boost',0
Simple mixer control 'ADC Mixer',0
Simple mixer control 'Analog',0
Simple mixer control 'Auto-Mute',0
Simple mixer control 'DAC Mux',0
Simple mixer control 'PGA-ADC Mux Left',0
Simple mixer control 'PGA-ADC Mux Right',0
Simple mixer control 'Soft Ramp',0
[root@ATK-stm32mp1]:/$
```

图 48.5.1.2 scontrols 命令对应的设置项

再来看一下 controls 对应的设置项，输入如下命令：

```
amixer controls                    //查看所有设置项
```

结果如图 48.5.1.3 所示：

```
[root@ATK-stm32mp1]:/$ amixer controls
numid=14,iface=MIXER,name='PCM channel mixer'
numid=2,iface=MIXER,name='PCM Playback Switch'
numid=1,iface=MIXER,name='PCM Playback Volume'
numid=10,iface=MIXER,name='Mic Boost Volume'
numid=11,iface=MIXER,name='ADC Boost Volume'
numid=5,iface=MIXER,name='ADC Mixer Switch'
numid=4,iface=MIXER,name='ADC Mixer Volume'
numid=6,iface=MIXER,name='Playback Deemphasis Switch'
numid=3,iface=MIXER,name='Analog Playback Volume'
numid=7,iface=MIXER,name='Auto-Mute Switch'
numid=12,iface=MIXER,name='Bass Volume'
numid=15,iface=MIXER,name='DAC Mux'
numid=16,iface=MIXER,name='PGA-ADC Mux Left'
numid=17,iface=MIXER,name='PGA-ADC Mux Right'
numid=8,iface=MIXER,name='Soft Ramp Switch'
numid=13,iface=MIXER,name='Treble Volume'
numid=9,iface=MIXER,name='Zero Cross Switch'
[root@ATK-stm32mp1]:/$
```

图 48.5.1.3 controls 命令对应的设置项

图 48.5.1.2 和 48.5.1.3 整体设置项目还是比较多的，很多设置项目我们都不知道是啥意思，毕竟不是从事音频专业的。这里我们只关注一些最常用的设置即可，比如设置耳机和喇叭音量、设置左右声道音量、设置输入音量等等。

3、查看设置值

不同的设置项对应的设置值类型不同，先查看一下 scontents 对应的设置值，输入如下命令：

```
amixer scontents    //查看设置值
```

结果如图 48.5.1.4 所示：

```
[root@ATK-stm32mp1]:/$ amixer scontents
Simple mixer control 'Bass',0
  Capabilities: volume volume-joined
  Playback channels: Mono
  Capture channels: Mono
  Limits: 0 - 15
  Mono: 7 [47%] [0.00dB]
Simple mixer control 'Treble',0
  Capabilities: volume volume-joined
  Playback channels: Mono
  Capture channels: Mono
  Limits: 0 - 15
  Mono: 7 [47%] [0.00dB]
Simple mixer control 'PCM',0
  Capabilities: pvolume pswitch
  Playback channels: Front Left - Front Right
  Limits: Playback 0 - 127
  Mono:
  Front Left: Playback 103 [81%] [0.00dB] [off]
  Front Right: Playback 103 [81%] [0.00dB] [off]
Simple mixer control 'PCM channel mixer',0
  Capabilities: enum
  Items: 'L R' 'L+R' 'R L'
  Item0: 'L R'
```

图 48.5.1.4 scontrols 命令对应的设置项

从图 48.5.1.4 可以看出“PCM”项目就是设置耳机音量的，音量范围为 0-127，当前音量为 103。有些设置项是 bool 类型，只有 on 和 off 两种状态。关于 controls 对应的设置值大家自行输入“amixer controls”命令查看即可。

4、设置声卡

知道了设置项和设置值，那么设置声卡就很简单了，直接使用下面命令即可：

```
amixer sset 设置项目 设置值
```

或：

```
amixer cset 设置项目 设置值
```

5、获取声卡设置值

如果要读取当前声卡某项设置值的话使用如下命令：

```
amixer sget 设置项目
```

或：

```
amixer cget 设置项目
```

48.5.2 音乐播放测试

1、使用 amixer 设置声卡

第一次使用声卡之前一定要先使用 amixer 设置声卡，打开耳机或者喇叭，并且设置喇叭或者耳机音量，还有就是开启左右声道，输入如下命令：

```
amixer cset name='PCM Playback Switch' 'on','on'
amixer cset name='PCM Playback Volume' '63','63'
amixer cset name='Analog Playback Volume' '204','204'
```

```
amixer cset name='PCM channel mixer' 'L R'
```

2、使用 aplay 播放 WAV 格式音乐

声卡设置好以后就可以使用 aplay 软件播放 wav 格式的音乐测试一下，aplay 也是 alsa-utils 提供的。buildroot 编译 alsa-utils 的时候已经在/usr/share/sounds/alsa/路径下存放了一些音乐了，我们可以直接拿来测试，测试命令如下:

```
aplay /usr/share/sounds/alsa/Front_Right.wav                    //播放歌曲
```

如果一切设置正常的话就会开始播放音乐。

注意：由于 CS42L51 只有耳机播放的输出接口，没有喇叭接口。正点原子 STM32MP1 开发板上的喇叭直接接到了耳机右声道上，所以耳机与喇叭不能自动切换，播放音乐的时候会同时有声音。

48.5.3 MIC 录音测试

正点原子的 STM32MP1 开发板上有一个板载麦克风，如图 48.5.3.1 所示：

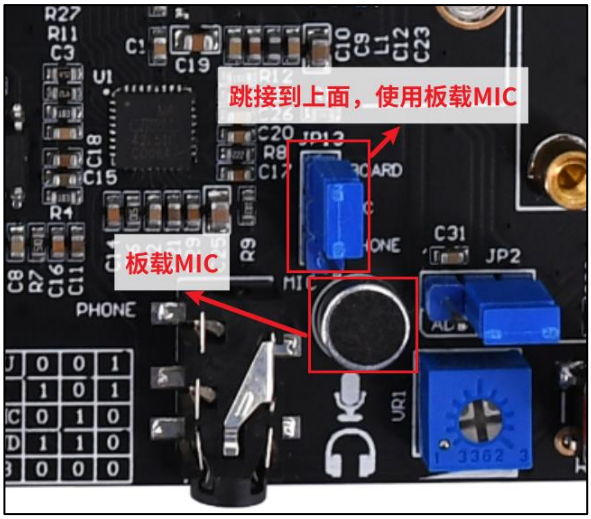


图 48.5.3.1 板载麦克风

将图 48.5.3.1 中的 JP13 跳接到上方，也就是使用板载 MIC 录音。如果跳接到下方就是使用耳机自带的话筒。

1、使用 amixer 设置声卡

同样的，第一次使用声卡录音之前要先使用 amixer 设置一下声卡，设置声卡的命令如下所示：

```
amixer cset name='PGA-ADC Mux Left' '3'
amixer cset name='PGA-ADC Mux Right' '3'
amixer cset name='Mic Boost Volume' '1','1'
```

2、使用 arecord 录制音频

使用 arecord 来录制一段 10 秒中的音频，arecord 也是 alsa-utils 编译出来的，输入如下命令：

```
arecord -f S16_LE   -d 10 -D hw:0,1 record.wav
```

-f 是用来设置录音的数据以 S16_LE 格式进行采样。-d 是指定录音时间，单位为 s，-D 是指定硬件声卡，0 是卡数，1 是在设备数量。这条指令就是录制一段以 S16_LE 格式 10s 的 wav

音频，音频名字为 record.wav。录制的时候大家就可以对着开发板上的 MIC 说话，直到录制完成。

录制完成以后使用 aplay 播放刚刚录制的音频，看看录制是否成功，命令如下：

```
aplay record.wav
```

48.5.4 PHONE 录音测试

最后进行一下 phone 录音测试，也就是使用耳机上的 MIC 录音，前提是你的耳机支持录音。正点原子的 STM32MP1 开发板上 phone 接口如图 48.5.4.1 所示：

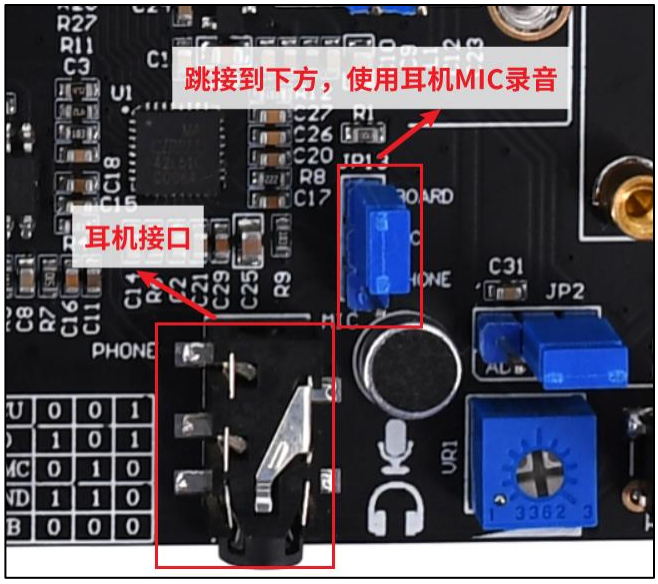


图 48.5.4.1 PHONE 接口

使用一根 3.5mm 的接口耳机，耳机要有麦。录音的方法跟 MIC 录音一样参考 48.5.3 即可。

48.6 开机自动配置声卡

大家在使用的时候应该应该会发现开发板重启以后声卡的所有设置都会消失，必须重新设置声卡。也就是说我们对声卡的设置不能保存，本小节我们就来学习一下如何保存声卡的设置。

1、使用 alsactl 保存声卡设置

声卡设置的保存通过 alsactl 工具来完成，此工具也是 alsa-utils 编译出来的。alsactl 默认将声卡配置文件保存在/var/lib/alsa 目录下。

首先使用 amixer 设置声卡，然后输入如下命令保存声卡设置

```
alsactl -f /var/lib/alsa/asound.state store          //保存声卡设置
```

-f 指定声卡配置文件，store 表示保存。关于 alsactl 的详细使用方法，输入“alsactl -h”即可。保存成功以后就会生成/var/lib/alsa/asound.state 这个文件，asound.state 里面就是关于声卡的各种设置信息，大家可以打开此文件查看一下里面的内容，如图 48.6.1 所示：

```
state.STM32MP1DK {
        control.1 {
                iface MIXER
                name 'PCM Playback Volume'
                value.0 63
                value.1 63
                comment {
                        access 'read write'
                        type INTEGER
                        count 2
                        range '0 - 127'
                        dbmin -5150
                        dbmax 1200
                        dbvalue.0 -2000
                        dbvalue.1 -2000
                }
        }
```

图 48.6.1 asound.state 文件部分内容

如果要使用 asound.state 中的配置信息来配置声卡，执行如下命令即可：

```
alsactl -f /var/lib/alsa/asound.state restore
```

最后面的参数改为 restore 即可，也就是恢复的意思。

在 19.5.4 小节中我们创建了个/etc/init.d/Sautorun 自启动文件，如果没有此文件的话自行创建，或者使用/etc/init.d/rcS 文件，在此文件中添加如下内容：

示例代码 4.6.1 /etc/init.d/Sautorun 追加内容

```
1 if [ -f "/var/lib/alsa/asound.state" ]; then
2         echo "ALSA: Restoring mixer setting......"
3         /usr/sbin/alsactl -f /var/lib/alsa/asound.state restore &
4 fi
```

第 1 行判断/var/lib/alsa/asound.state 这个文件是否存在，存在的话就执行下面的。首先输出一行提示符：“ALSA: Restoring mixer setting.....”，表示设置声卡，最后调用/usr/sbin/alsactl 来执行声卡设置工作。

设置完成以后重启开发板，开发板开机就会自动设置声卡，输出如图 48.6.2 所示内容：

```
FAIL
Starting sshd: OK
Starting vsftpd: OK
ALSA: Restoring mixer setting......

Welcome to alientek STM32MP157
ATK-stm32mp1 login: root
[root@ATK-stm32mp1]:~$
```

图 48.6.2 声卡开机自动设置

直接使用 aplay 播放音乐测试声卡开机自动配置是否正确。

48.7 alsamixer 简介

alsamixer 是基于图形化的，直接输入“alsamixer”命令即可打开声卡配置界面，如图 48.7.1 所示：

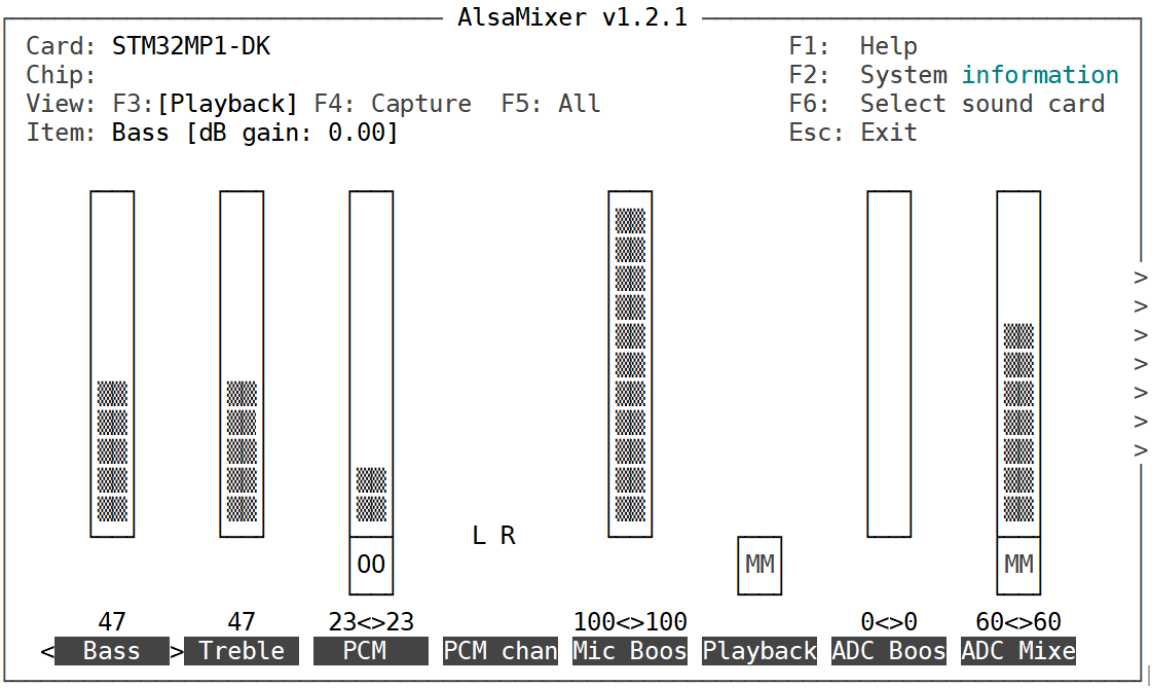


图 48.7.1 alsamixer 配置界面

F1 键：查看帮助信息。

F2 键：查看系统信息。

F3 键：播放设置。

F4 键：录音设置。

F6 键：选择声卡，多声卡情况下。

Item：设置项全名。

图 48.7.1 最下面一行就是具体的设置项，比如“Bass”、“PCM”等等，通过键盘上左右键选择设置项。按下上下键来调整大小，比如设置耳机音量大小等。有些项目会显示“MM”，表示静音，按下“M”键修改为“OO”状态打开，“M”键用于修改打开或关闭某些项目。

关于 alsamixer 的介绍就到这里，用起来还是很简单的。

第四十九章 Linux CAN 驱动实验

CAN 是目前应用非常广泛的现场总线之一，主要应用于汽车电子和工业领域，尤其是汽车领域，汽车上大量的传感器与模块都是通过 CAN 总线连接起来的。CAN 总线目前是自动化领域发展的热点技术之一，由于其高可靠性，CAN 总线目前广泛的应用于工业自动化、船舶、汽车、医疗和工业设备等方面。STM32MP1 自带了 CAN 外设，因此可以开发 CAN 相关的设备，本章我们就来学习一下如何驱动正点原子的 STM32MP1 开发板上的 CAN 接口。

49.1 CAN 协议简析

有关 CAN 协议详细内容请参考开发板资料里面由瑞萨电子编写的《CAN 入门教程》，路径为：4、参考资料->CAN 入门教程.pdf，本小节参考自此教程。

49.1.1 何为 CAN？

CAN 的全称为 Controller Area Network，也就是控制局域网络，简称为 CAN。CAN 最早是由德国 BOSCH(博世)开发的，目前已经是国际标准(ISO 11898)，是当前应用最广泛的现场总线之一。BOSCH 主要是做汽车电子的，因此 CAN 一开始主要是为汽车电子准备的，事实也是如此，CAN 协议目前已经是汽车网络的标准协议。当然了，CAN 不仅仅应用于汽车电子，经过几十年的发展，CAN 协议的高性能和高可靠性已经得到了业界的认可，目前除了汽车电子以外也广泛应用于工业自动化、医疗、工业和船舶等领域。

以汽车电子为例，汽车上有空调、车门、发动机、大量传感器等，这些部件都是通过 CAN 总线连在一起形成一个网络，车载网络结构如图 49.1.1.1 所示：

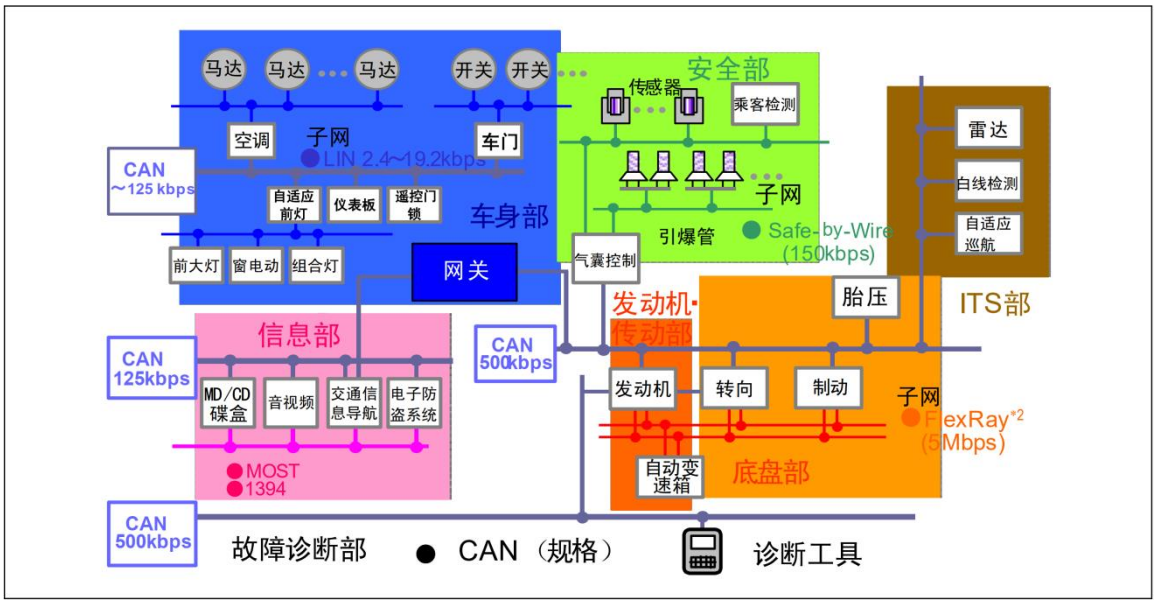


图 49.1.1.1 车载网络示意图

图 49.1.1.1 中各个单元通过 CAN 总线连接在一起，每个单元都是独立的 CAN 节点。同一个 CAN 网络中所有单元的通信速度必须一致，不同的网络之间通信速度可以不同。比如图 49.1.1.1 中 125Kbps 的 CAN 网络下所有的节点速度都是 125Kbps 的，整个网络由一个网关与其他的网络连接。

CAN 的特点主要有一下几点：

1、多主控制

在总线空闲时，所有单元都可以发送消息（多主控制），而两个以上的单元同时开始发送消息时，根据标识符（Identifier 以下称为 ID）决定优先级。ID 并不是表示发送的目的地址，而是表示访问总线的消息的优先级。两个以上的单元同时开始发送消息时，对各消息 ID 的每个位进行逐个仲裁比较。仲裁获胜（被判定为优先级最高）的单元可继续发送消息，仲裁失利的单元则立刻停止发送而进行接收工作。

2、系统的柔软性

与总线相连的单元没有类似于“地址”的信息。因此在总线上增加单元时，连接在总线上的其它单元的软硬件及应用层都不需要改变。

3、通信速度快，距离远

最高 1Mbps（距离小于 40M），最远可达 10KM（速率低于 5Kbps），最新的 CAN FD 速度可以到 5Mbps。

4、具有错误检测、错误通知和错误恢复功能

所有单元都可以检测错误（错误检测功能），检测出错误的单元会立即同时通知其他所有单元（错误通知功能），正在发送消息的单元一旦检测出错误，会强制结束当前的发送。强制结束发送的单元会不断反复地重新发送此消息直到成功发送为止（错误恢复功能）。

5、故障封闭功能

CAN 可以判断出错误的类型是总线上暂时的数据错误（如外部噪声等）还是持续的数据错误（如单元内部故障、驱动器故障、断线等）。由此功能，当总线上发生持续数据错误时，可将引起此故障的单元从总线上隔离出去。

6、连接节点多

CAN 总线是可同时连接多个单元的总线。可连接的单元总数理论上是没有限制的。但实际上可连接的单元数受总线上的时间延迟及电气负载的限制。降低通信速度，可连接的单元数增加；提高通信速度，则可连接的单元数减少。

49.1.2 CAN 电气属性

CAN 总线使用两根线来连接各个单元：CAN_H 和 CAN_L，CAN 控制器通过判断这两根线上的电位差来得到总线电平，CAN 总线电平分为显性电平和隐性电平两种。显性电平表示逻辑“0”，此时 CAN_H 电平比 CAN_L 高，分别为 3.5V 和 1.5V，电位差为 2V。隐形电平表示逻辑“1”，此时 CAN_H 和 CAN_L 电压都为 2.5V 左右，电位差为 0V。CAN 总线就通过显性和隐形电平的变化来将具体的数据发送出去，如图 49.1.2.1 所示：

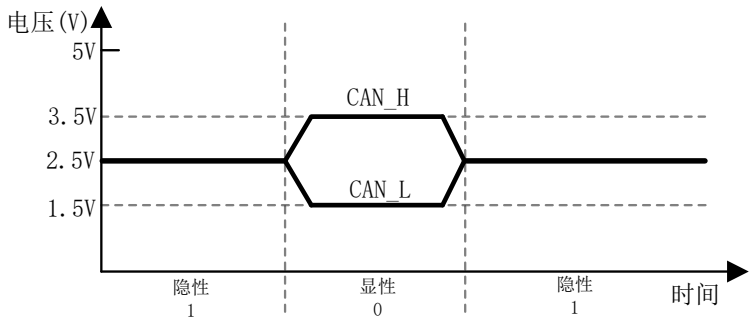


图 49.1.2.1 CAN 电平

CAN 总线上没有节点传输数据的时候一直处于隐性状态，也就是说总线空闲状态的时候一直处于隐性。CAN 网络中的所有单元都通过 CAN_H 和 CAN_L 这两根线连接在一起，如图 49.1.2.2 所示：

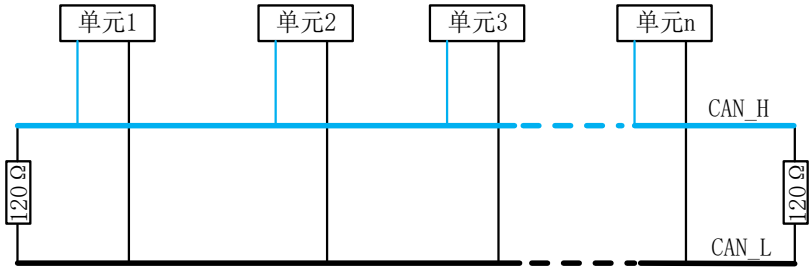


图 49.1.2.2 CAN 网络示意图

图中所有的 CAN 节点单元都采用 CAN_H 和 CAN_L 这两根线连接在一起，CAN_H 接 CAN_H、CAN_L 接 CAN_L，CAN 总线两端要各接一个 120Ω 的端接电阻，用于匹配总线阻抗，吸收信号反射及回拨，提高数据通信的抗干扰能力以及可靠性。

CAN 总线传输速度可达 1Mbps/S，最新的 CAN-FD 最高速度可达 5Mbps/S，甚至更高，CAN-FD 不在本章讨论范围，感兴趣的可以自行查阅相关资料。CAN 传输速度和总线距离有关，总线距离越短，传输速度越快。

49.1.3 CAN 协议

通过 CAN 总线传输数据是需要按照一定协议进行的，CAN 协议提供了 5 种帧格式来传输数据：数据帧、遥控帧、错误帧、过载帧和帧间隔。其中数据帧和遥控帧有标准格式和扩展格式两种，标准格式有 11 位标识符(ID)，扩展格式有 29 个标识符(ID)。这 5 中帧的用途见表 49.1.3.1：

帧类型	帧用途
数据帧	用于 CAN 节点单元之间进行数据传输的帧
遥控帧	用于接收单元向具有相同 ID 的发送单元请求数据的帧
错误帧	用于当检测出错误时向其它单元通知错误的帧
过载帧	用于接收单元通知其尚未做好接收准备的帧
间隔帧	用于将数据帧及遥控帧与前面的帧分离开来的帧

表 49.1.3.1 帧用途

1、数据帧

数据帧由 7 段组成：

①、帧起始，表示数据帧开始的段。

②、仲裁段，表示该帧优先级的段。

③、控制段，表示数据的字节数及保留位的段。

④、数据段，数据的内容，一帧可发送 0~8 个字节的数据。

⑤、CRC 段，检查帧的传输错误的段。

⑥、ACK 段，表示确认正常接收的段。

⑦、帧结束，表示数据帧结束的段。

数据帧结构如图 49.1.3.1 所示：

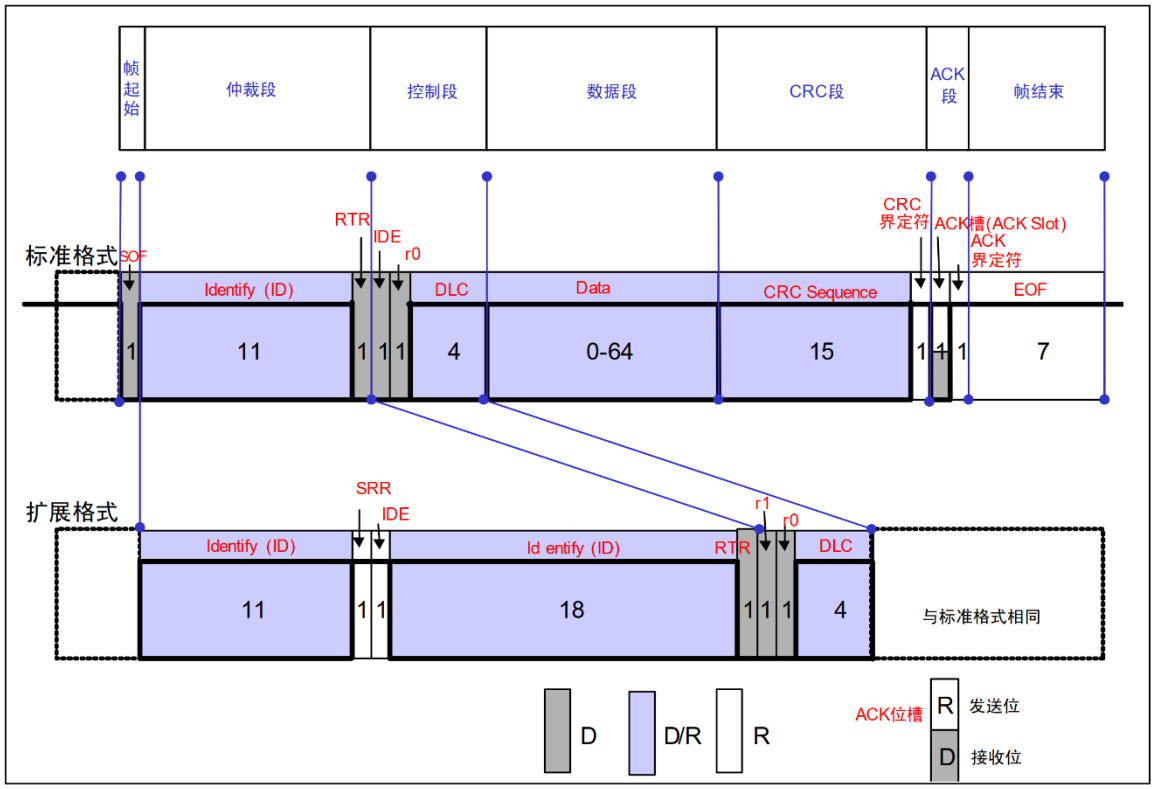


图 49.1.3.1 数据帧结构

图 49.1.3.1 给出了数据帧标准格式和扩展格式两种帧结构，图中 D 表示显性电平 0、R 表示隐性电平 1，D/R 表示显性或隐性，也就是 0 或 1，我们来简单分析一下数据帧的这 7 个段。

①、帧起始

帧起始很简单，标准格式和扩展格式都是由一个位的显性电平 0 来表示帧起始。

②、仲裁段

仲裁段表示帧优先级，仲裁段结构如图 49.1.3.2 所示：

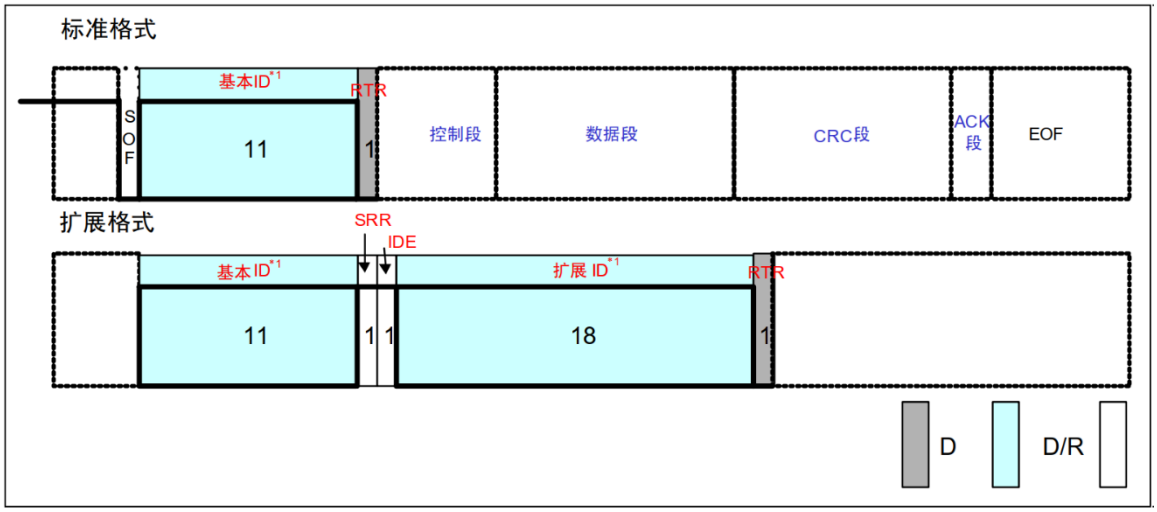


图 49.1.3.2 仲裁段结构

标准格式和扩展格式的仲裁段不同，从图 49.1.3.2 可以看出，标准格式的 ID 为 11 位，发送顺序是从 ID10 到 ID0，最高 7 位 ID10~ID4 不能全为隐性(1)，也就是禁止 0X1111111XXXXX

这样的 ID。扩展格式的 ID 为 29 位，基本 ID 从 ID28 到 ID18，扩展 ID 由 ID17 到 ID0，基本 ID 与标准格式一样，禁止最高 7 位都为隐性。

③、控制段

控制段由 6 个位构成，表示数据段的字节数，标准格式和扩展格式的控制段略有不同，如图 49.1.3.3 所示：

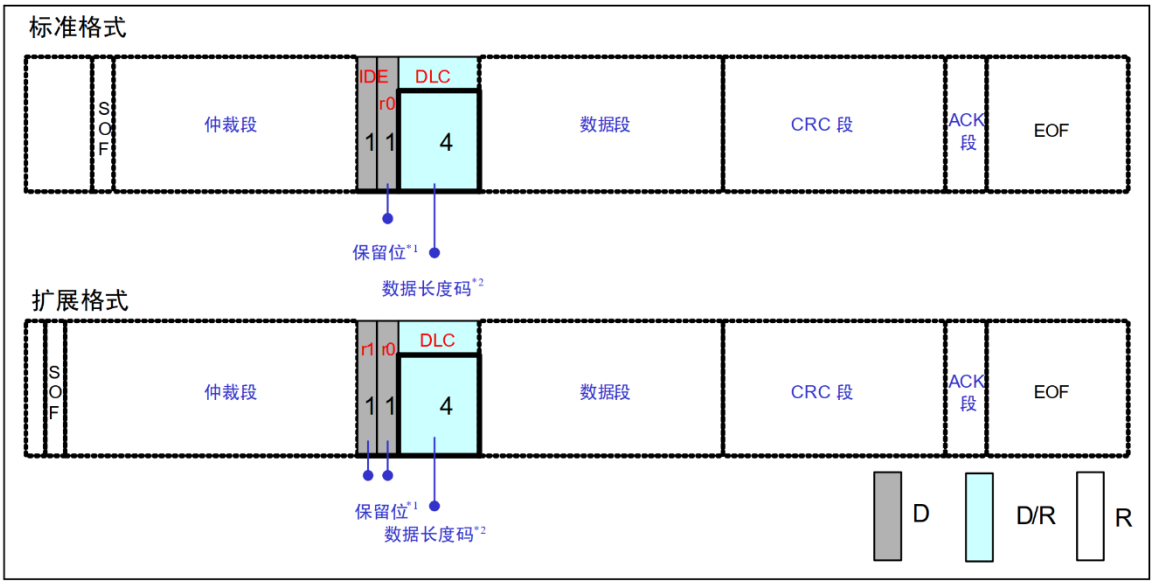


图 49.1.3.3 控制段结构

图 49.1.3.3 中 r1 和 r0 为保留位，保留位必须以显性电平发送。DLC 为数据长度，高位在前，DLC 段有效值范围为 0~8。

④、数据段

数据段也就是帧的有效数据，标准格式和扩展格式相同，可以包含 0~8 个字节的数据，从最高位(MSB)开始发送，结构如图 49.1.3.4 所示：

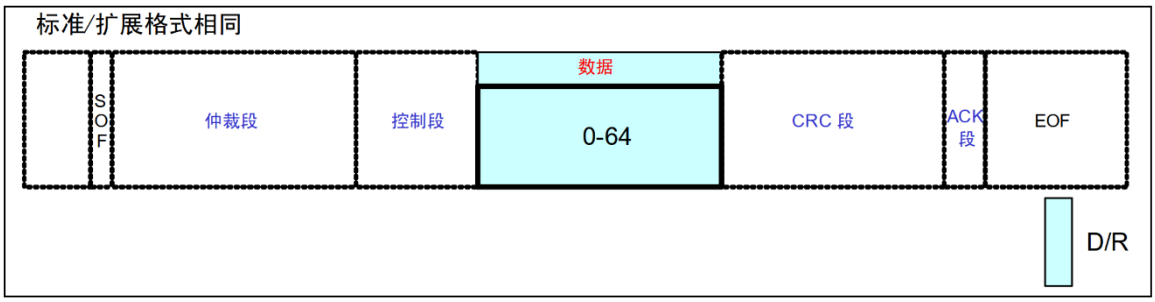


图 49.1.3.4 数据段

注意，图 49.1.3.4 中数据段的 0~64 为 bit，对应到字节就是 0~8 字节。

⑤、CRC 段

CRC 段保存 CRC 校准值，用于检查帧传输错误，标准格式和扩展格式相同，CRC 段结构如图 49.1.3.5 所示：

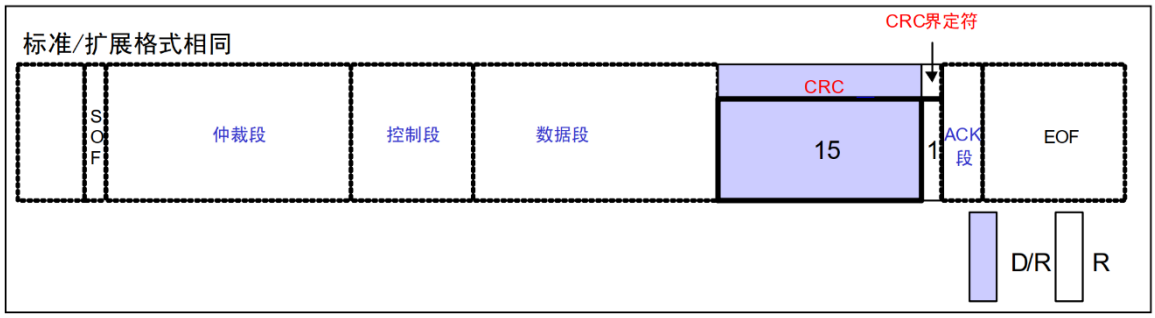


图 49.1.3.5 CRC 段结构

从图 49.1.3.5 可以看出，CRC 段由 15 位的 CRC 值与 1 位的 CRC 界定符组成。CRC 值的计算范围包括：帧起始、仲裁段、控制段、数据段，接收方以同样的算法进行计算，然后用计算得到的 CRC 值与此 CRC 段进行比较，如果不一致的话就会报错。

⑥、ACK 段

ACK 段用来确认接收是否正常，标准格式和扩展格式相同，ACK 段结构如图 49.1.3.6 所示：

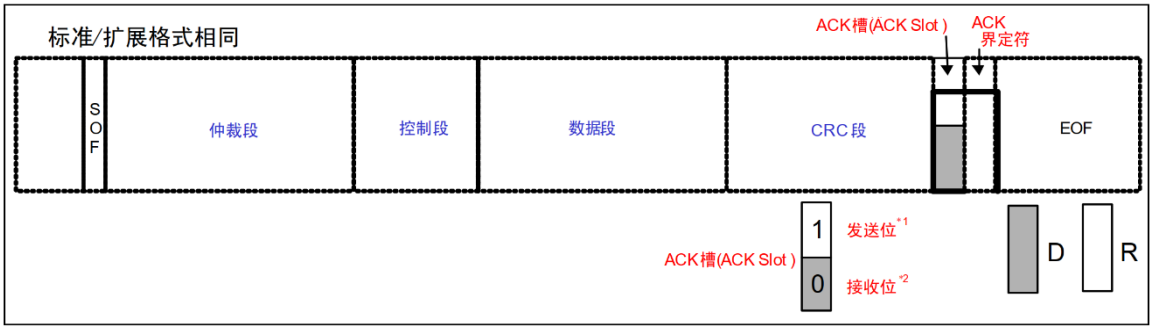


图 49.1.3.6 ACK 段结构

从图 49.1.3.7 可以看出，ACK 段由 ACK 槽(ACK Slot)和 ACK 界定符两部分组成。发送单元的 ACK，发送 2 个隐性位，而接收到正确消息的单元在 ACK 槽（ACK Slot）发送显性位，通知发送单元正常接收结束，这个过程叫发送 ACK/返回 ACK。发送 ACK 的是所有接收单元中接收到正常消息的单元，所谓正常消息是指不含填充错误、格式错误、CRC 错误的消息，这些接收单元既不处于总线关闭态也不处于休眠态的所有接收单元中。

⑦、帧结束

最后就是帧结束段，标准格式和扩展格式相同，帧结束段结构如图 49.1.3.7 所示：

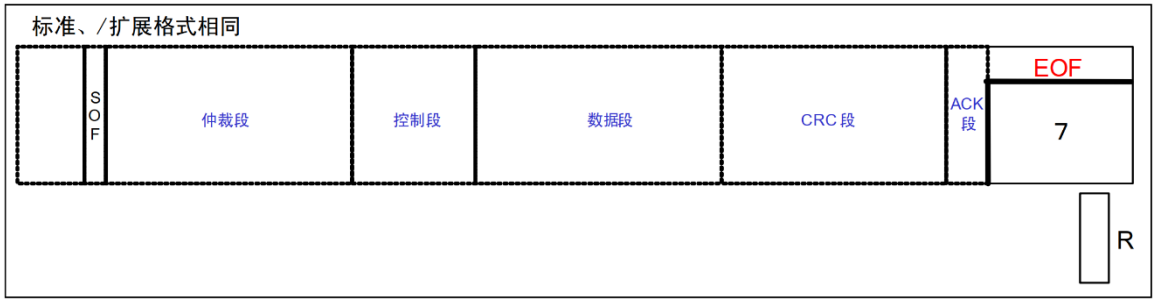


图 49.1.3.7 帧结束段结构

从图 49.1.3.7 可以看出，帧结束段很简单，由 7 位隐性位构成。

2、遥控帧

接收单元向发送单元请求数据的时候就用遥控帧，遥控帧由 6 个段组成：

①、帧起始，表示数据帧开始的段。

②、仲裁段，表示该帧优先级的段。

③、控制段，表示数据的字节数及保留位的段。

④、CRC 段，检查帧的传输错误的段。

⑤、ACK 段，表示确认正常接收的段。

⑥、帧结束，表示数据帧结束的段。

遥控帧结构如图 49.1.3.8 所示：

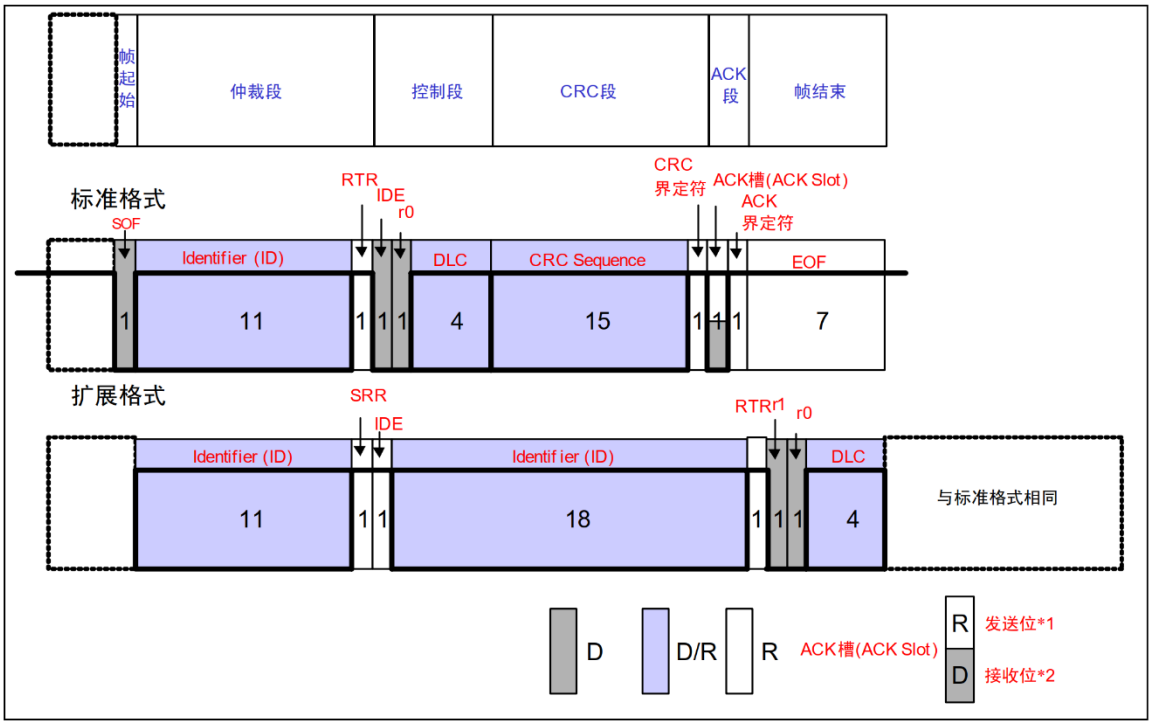


图 49.1.3.8 遥控帧结构

从图 49.1.3.8 可以看出，遥控帧结构基本和数据帧一样，最主要的区别就是遥控帧没有数据段。遥控帧的 RTR 位为隐性的，数据帧的 RTR 位为显性，因此可以通过 RTR 位来区分遥控帧和没有数据的数据帧。遥控帧没有数据，因此 DLC 表示的是所请求的数据帧数据长度，遥控帧的其他段参考数据帧的描述即可。

3、错误帧

当接收或发送消息出错的时候使用错误帧来通知，错误帧由错误标志和错误界定符两部分组成，错误帧结构如图 49.1.3.9 所示：

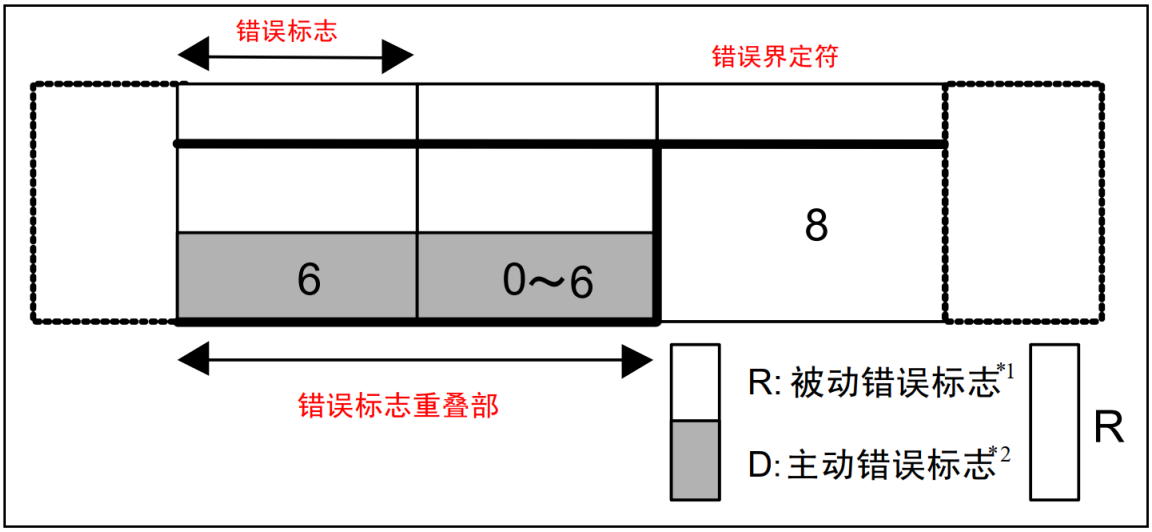


图 49.1.3.9 错误帧结构

错误标志有主动错误标志和被动错误标志两种，主动错误标志是 6 个显性位，被动错误标志是 6 个隐性位，错误界定符由 8 个隐性位组成。

4、过载帧

接收单元尚未完成接收准备的话就会发送过载帧，过载帧由过载标志和过载界定符构成，过载帧结构如图 49.1.3.10 所示：

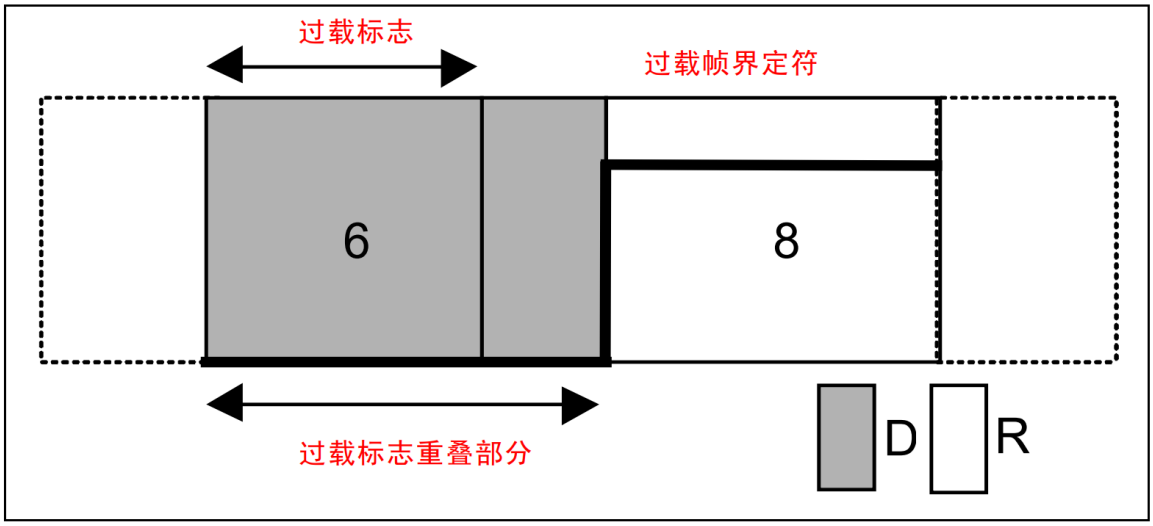


图 49.1.3.10 过载帧结构

过载标志由 6 个显性位组成，与主动错误标志相同，过载界定符由 8 个隐性位组成，与错误帧中的错误界定符构成相同。

5、帧间隔

帧间隔用于分隔数据帧和遥控帧，数据帧和遥控帧可以通过插入帧间隔来将本帧与前面的任何帧隔开，过载帧和错误帧前不能插入帧间隔，帧间隔结构如图 49.1.3.11 所示：

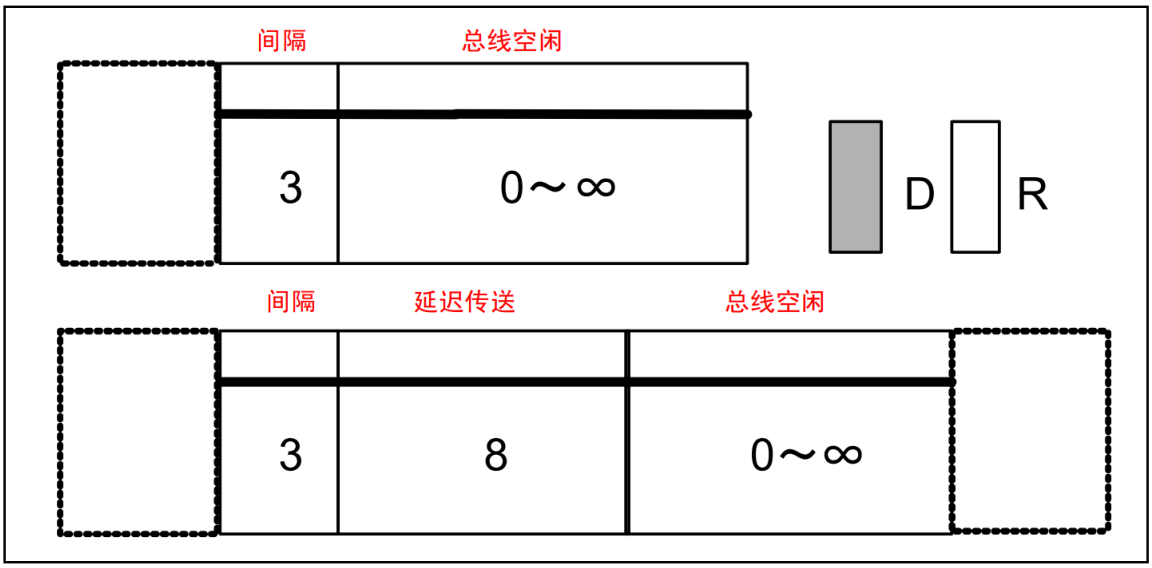


图 49.1.3.11 帧间隔结构

图 49.1.3.11 中间隔由 3 个隐性位构成，总线空闲为隐性电平，长度没有限制，本状态下表示总线空闲，发送单元可以访问总线。延迟发送由 8 个隐性位构成，处于被动错误状态的单元发送一个消息后的帧间隔中才会有延迟发送。

49.1.4 CAN 速率

CAN 总线以帧的形式发送数据，但是最终到总线上的就是“0”和“1”这样的二进制数据，这里就涉及到了通信速率，也就是每秒钟发送多少位数据，前面说了 CAN2.0 最高速度为 1Mbps/S。对于 CAN 总线，一个位分为 4 段：

①、同步段(SS)

②、传播时间段(PTS)

③、相位缓冲段 1(PBS1)

④、相位缓冲段 2(PBS2)

这些段由 Tq(Time Quantum)组成，Tq 是 CAN 总线的最小时间单位。帧由位构成，一个位由 4 个段构成，每个段又由若干个 Tq 组成，这个就是位时序。1 位由多少个 Tq 构成、每个段又由多少个 Tq 构成等，可以任意设定位时序。通过设定位时序，多个单元可同时采样，也可任意设定采样点。各段的作用和 Tq 数如图 49.1.4.1 所示：

段名称	段的作用	Tq 数	
同步段 （SS: Synchronization Segment）	多个连接在总线上的单元通过此段实现时序调整，同步进行接收和发送的工作。由隐性电平到显性电平的边沿或由显性电平到隐性电平边沿最好出现在此段中。	1Tq	8～25Tq
传播时间段 （PTS: Propagation Time Segment）	用于吸收网络上的物理延迟的段。 所谓的网络的物理延迟指发送单元的输出延迟、总线上信号的传播延迟、接收单元的输入延迟。 这个段的时间为以上各延迟时间的和的两倍。	1～8Tq	
相位缓冲段 1 (PBS1: Phase Buffer Segment 1)	当信号边沿不能被包含于 SS 段中时，可在此段进行补偿。 由于各单元以各自独立的时钟工作，细微的时钟误差会累积起来，PBS 段可用于吸收此误差。 通过对相位缓冲段加减 SJW 吸收误差。（请参照图 34）。SJW 加大后允许误差加大，但通信速度下降。	1～8Tq	
相位缓冲段 2 (PBS2: Phase Buffer Segment 2)		2～8Tq	
再同步补偿宽度 (SJW: reSynchronization Jump Width)	因时钟频率偏差、传送延迟等，各单元有同步误差。SJW 为补偿此误差的最大值。	1～4Tq	

图 49.1.4.1 一个位各段及其作用

1 个位的构成如图 49.1.4.2 所示：

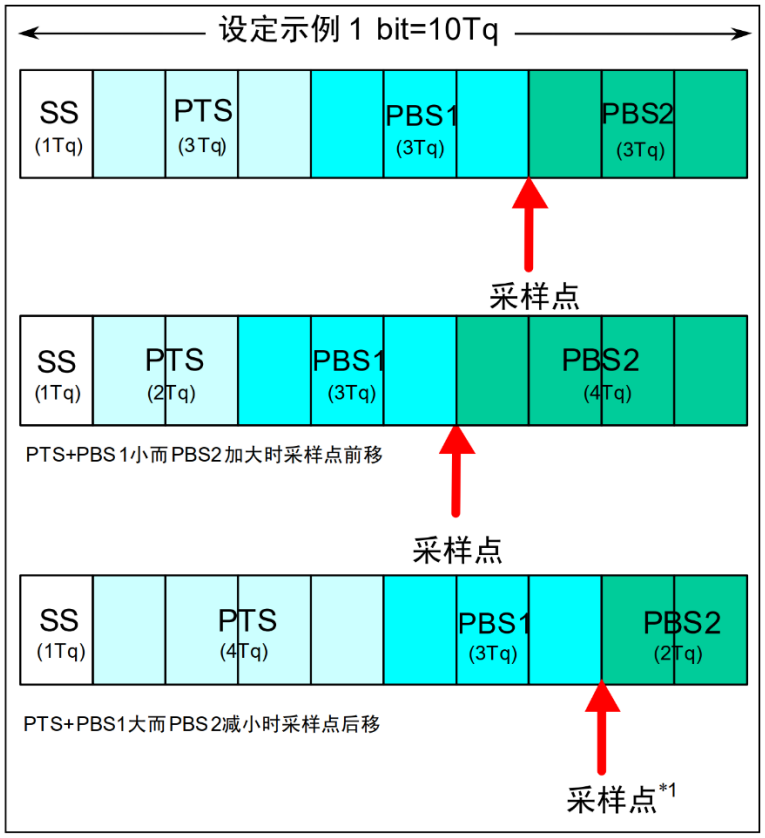


图 49.1.4.2 一个位的构成

图 49.1.4.2 中的采样点是指读取总线电平，并将读到的电平作为位值的点。位置在 PBS1 结束处。根据这个位时序，我们就可以计算 CAN 通信的波特率了。具体计算方法，我们等下再介绍，前面提到的 CAN 协议具有仲裁功能，下面我们来看看是如何实现的。

在总线空闲态，最先开始发送消息的单元获得发送权。

当多个单元同时开始发送时，各发送单元从仲裁段的第一位开始进行仲裁。连续输出显性电平最多的单元可继续发送。实现过程，如图 49.1.4.3 所示：

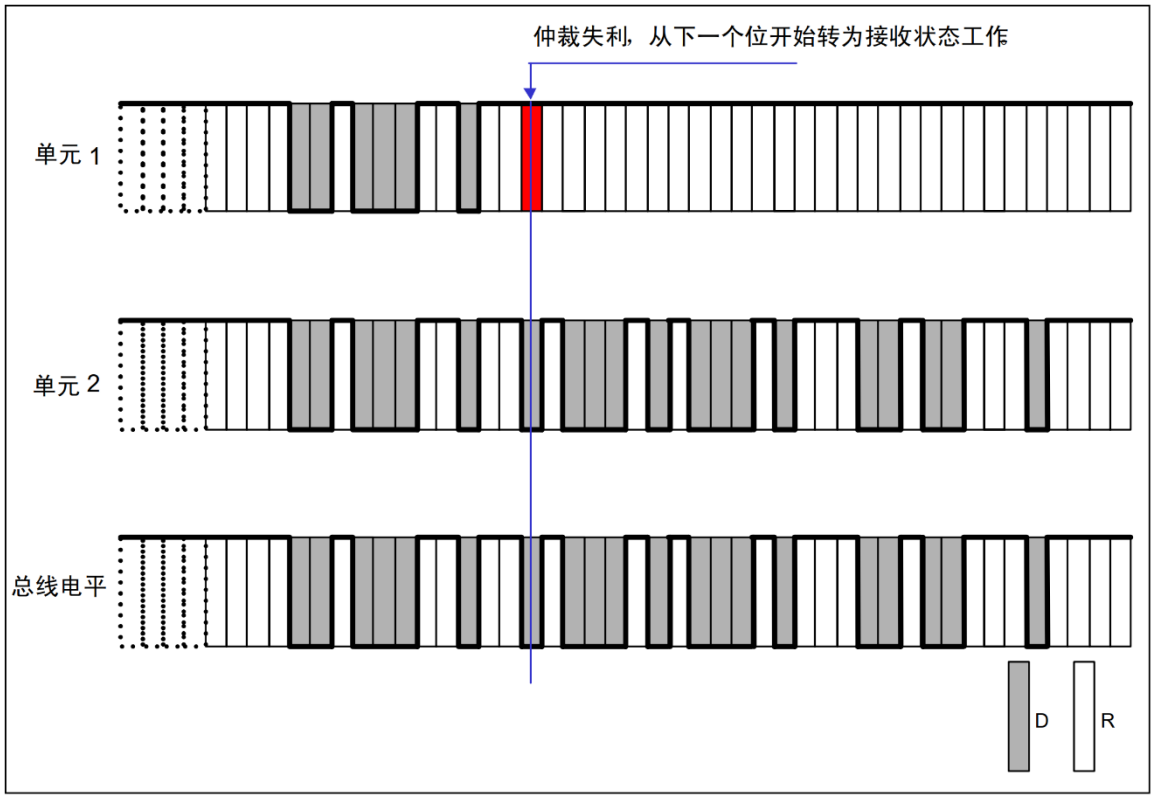


图 49.1.4.3 CAN 总线仲裁过程

图 49.1.4.3 中，单元 1 和单元 2 同时开始向总线发送数据，开始部分他们的数据格式是一样的，故无法区分优先级，直到 T 时刻，单元 1 输出隐性电平，而单元 2 输出显性电平，此时单元 1 仲裁失利，立刻转入接收状态工作，不再与单元 2 竞争，而单元 2 则顺利获得总线使用权，继续发送自己的数据。这就实现了仲裁，让连续发送显性电平多的单元获得总线使用权。

关于 CAN 协议就讲到这里，关于 CAN 协议更详细的内容请参考《CAN 入门教程》。

49.1.5 CAN FD 简介

随着汽车电子的发展，CAN 的速度已经跟不上需求，因此 CAN FD 应运而生。BOSCH 在 2011 年推出了 CAN FD 方案，CAN FD 在 CAN 的基础上发展而来，主要扩展了 CAN 的带宽以及数据长度。

CAN FD 采用两种速率，控制段和总裁段等使用标准的 CAN 速率，但是数据段会切换到更高的通信速率，比如 5Mbps。CAN FD 也对数据长度做了扩展，最大支持 64 字节。CAN FD 数据帧格式如图 49.1.5.1 所示：

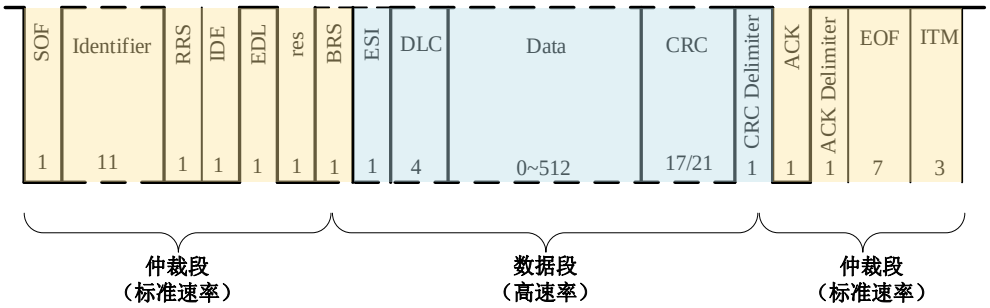


图 49.1.5.1 CAN FD 数据帧

CAN FD 相比于 CAN 改进的地方有一下几点：

1、增加一些控制位

CAN FD 在帧报文里面主要增加了 EDL、BRS 和 ESI 这三位：

EDL：用于指示当前为 CAN 帧还是 CAN FD 帧，标准 CAN 中此位为显性，在 CAN FD 中此位为隐性。

BRS：速率转换开关，为隐性的时候转换为可变速率，此时速率可变，BRS 到 CRC 之间使用可变速率。为显性的时候速率不变。

ESI：错误状态指示位，隐性的时候表示节点处于被动错误状态，显性的时候表示节点处于主动错误状态。

2、优化 CRC 算法

传统的 CAN 协议中，使用位填充的方式来保持同步，但是这种方法会对 CRC 造成干扰，导致错帧漏检。CAN FD 为此对 CRC 算法进行了优化，将填充位加入到差错校验码中进行计算，也就是以填充位的位流进行计算。

关于 CAN FD 就简单介绍这些，它主要的目的就是为了应对车载电子对通信带宽的需求，提供更高的数据速率以及数据长度。

49.1.6 STM32MP1 FDCAN 简介

STM32MP1 带有 CAN 控制器外设，叫做 FDCAN，FDCAN 子系统是有两个 CAN 模块、一个共享消息 RAM 存储器和一个时钟校准单元组成。这两个 CAN 模块(FDCAN1 和 FDCAN2)均符合 CAN2.0 版本 A、B 部分和 CAN FD1.0 版本。此外，第一个 CAN 模块的 FDCAN1 还支持 ISO 11898-4 中规定的时间触发 CAN(TTCAN)。STM32MP1 自带的 FDCAN 模块特性如下：

①、符合 CAN2.0 版 A、B 部分和 ISO 11898-1: 2015, -4
②、CAN FD 最多支持收发 64 个数据字节
③、硬件完全支持 TTCAN 协议级别 1 和级别 2(仅限 FDCAN1)
④、支持事件同步时间触发通讯(仅限 FDCAN1)
⑤、CAN 错误记录
⑥、两个可配置介绍 FIFO
⑦、64 个专用接收缓冲区和 32 个专用发送缓冲区
⑧、可配置发送事件 FIFO
……

FDCAN 支持 5 种模式：正常模式(Normal operation)、受限操作模式(Restricted-operation mode)、总线监控模式(Bus-monitoring mode)、外部环回模式(External loop-back mode)、内部环回模式(Internal loop-back mode)。除了正常模式外其它的模式都要配置 CCCR 寄存器的 TEST 位

为 1。

①、正常模式

把 CCCR 寄存器的 INIT 位和 TEST 位清零后，FDCAN 会正常接收或发送消息帧，所有的 CAN 协议功能都使能。

②、受限操作模式

当 CCCR 寄存器 ASM 位置 1 的时候使能此模式，在此模式下能够正常接收数据帧、接收远程帧和确认有效帧，不支持发送数据帧、发送远程帧和活动错误帧或发送过载帧。

③、总线监控模式

当 CCCR 寄存器 MON 位置 1 的时候使能此模式，在此模式下能够正常接收有效的数据帧和接收有效的远程帧，不支持确认有效帧。

④、外部环回模式

把 TEST 中的 LBCK 位置 1，并且 CCCR 中的 MON 位置 0，那么 FDCAN 将进入外部环回模式，此模式用于硬件自检。

⑤、内部环回模式

把 TEST 中的 LBCK 位置 1，并且 CCCR 中的 MON 位置 1，那么 FDCAN 将进入内部环回模式，此模式用于硬件自检。

FDCAN 是支持时钟校准单元功能(CCU)，此功能允许用户校准 FDCAN，开启方式为 CCCR 中的 CCE 和 INIT 位都置 1，并且 FDCAN 在 CAN2.0 模式下运行时，才能使用 CCU。

正点原子的 STM32MP1 开发板 FDCAN 接口都支持 CAN_FD，CAN_FD 和 CAN 最大的区别于数据段是可以切换更高的通讯波特率，数据传输率可大于 1Mbit/s，5Mbit/s(STM32MP1 目前最大支持 5Mbit/s)。

关于 STM32MP1 的 FDCAN 控制器就讲解到这里，如果想更加详细的了解 FDCAN，请参考《STM32MP1 参考手册》的“59 Controller area network with flexible data rate(FDCAN)”章节。

49.2 硬件原理图分析

正点原子 STM32MP1 开发板 FD CAN 接口原理图如图 49.2.1 所示：

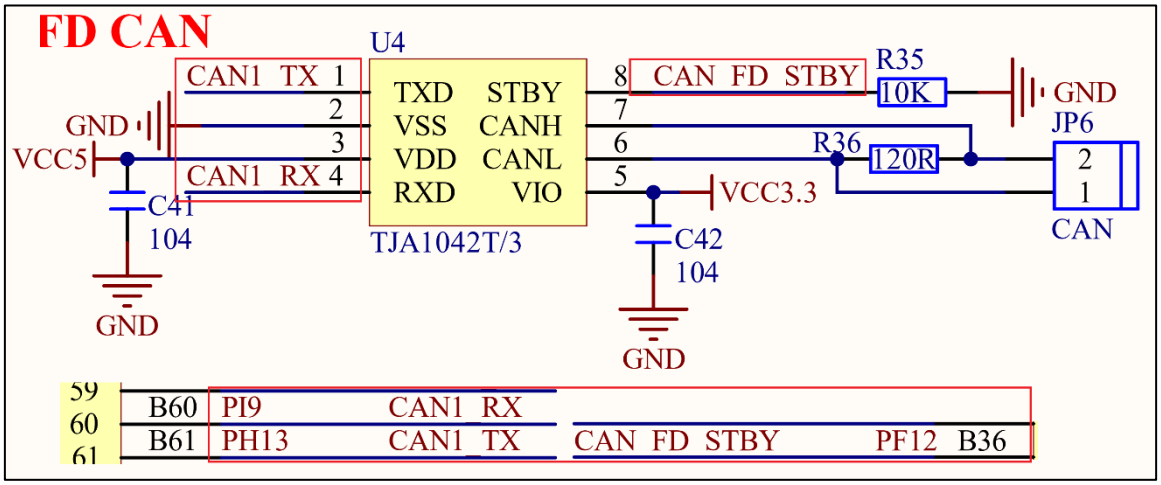


图 49.2.1 CAN 原理图

图 49.2.1 中 CAN1_TX 和 CAN1_RX 是 STM3MP1 FDCAN 的发送和接收引脚，对应 STM32MP1 的 PH13 和 PI9 这个两个引脚。正点原子 STM32MP1 开发板上的 CAN FD 收发芯片有三种：MCP2562FDT、TJA1042T/3 或者 SIT1042T/3，其中 SIT1042T/3 为国产 FD CAN 芯片，也是正点原子开发板主推的！这三种芯片 Pin to Pin 兼容，使用起来没有区别，具体芯片以

实际为准。这里就有 SIT1042T/3 为例，通过此收发器向外界提供 CAN_H 和 CAN_L 总线，R36 是一个 120 欧的端接匹配电阻。CAN_FD_STBY 是 SIT1042T/3 的高速与待机模式选择引脚，低电平为高速模式，连接在了 STM32MP157 的 PF12 引脚上，在此实验中没有用到 CAN_FD_STBY。

49.3 实验程序编写

49.3.1 修改设备树

ST 原厂提供的设备树已经配置好了 FDCAN1 的节点信息，但是我们还是要来看一下如何配置 STM32MP1 的 CAN1 节点。首先看一下 STM32MP1 的 FDCAN 设备树绑定文档，打开 Documentation/devicetree/bindings/net/can/m_can.txt，此文档描述了 FDCAN 节点下的相关属性信息，这里就不做介绍了，大家自行查阅。

1、FDCAN1 pinctrl 配置

首先我们肯定是配置 CAN1 的引脚配置，打开 stm32mp15-pinctrl.dtsi，找到如下所示内容：

示例代码 49.3.1.1 CAN1 的 pinctrl 配置

```
1   m_can1_pins_a: m-can1-0 {
2       pins1 {
3           pinmux = <STM32_PINMUX('H', 13, AF9)>;       /* CAN1_TX */
4           slew-rate = <1>;
5           drive-push-pull;
6           bias-disable;
7       };
8       pins2 {
9           pinmux = <STM32_PINMUX('I', 9, AF9)>;        /* CAN1_RX */
10          bias-disable;
11      };
12  };
13
14  m_can1_sleep_pins_a: m_can1-sleep-0 {
15      pins {
16          pinmux = <STM32_PINMUX('H', 13, ANALOG)>,    /* CAN1_TX */
17               <STM32_PINMUX('I', 9, ANALOG)>;         /* CAN1_RX */
18      };
19  };
```

可以看出，这里设置了 CAN1 的 default 和 sleep 两种模式下的引脚配置，其中 CAN1_TX 和 CAN1_RX 的引脚分别为 PH13 和 PI9，复位为 AF9，也就是复用为 CAN 引脚。这个配置和我们开发板上的引脚一致，所以无需修改，如果使用了其他引脚作为 CAN1 的收发引脚就需要自行修改。

2、FDCAN1 控制器节点信息

打开 stm32mp153.dtsi 文件，找到名为“m_can1”的节点，内容如下：

示例代码 49.3.1.2 m_can1 节点信息

```
1   m_can1: can@4400e000 {
```

```
2       compatible = "bosch,m_can";
3       reg = <0x4400e000 0x400>, <0x44011000 0x1400>;
4       reg-names = "m_can", "message_ram";
5       interrupts = <GIC_SPI 19 IRQ_TYPE_LEVEL_HIGH>,
6                    <GIC_SPI 21 IRQ_TYPE_LEVEL_HIGH>;
7       interrupt-names = "int0", "int1";
8       clocks = <&scmi0_clk CK_SCMI0_HSE>, <&rcc FDCAN_K>;
9       clock-names = "hclk", "cclk";
10      bosch,mram-cfg = <0x0 0 0 32 0 0 2 2>;
11      status = "disabled";
12    };
```

根据第 2 行的 compatible 属性就可以找到 STM32MP1 的 FD CAN 驱动源文件，驱动文名为 drivers/net/can/m_can/m_can_platform.c。第 11 行的 status 属性为 disabled，所以 FDCAN1 默认关闭的。在 stm32mp157d-atk.dts 中添加使能 FDCAN1 的相关操作，添加如下所示代码：

示例代码 49.3.1.3 stm32mp157d-atk.dts 要追加的 m_can1 节点

```
1   &m_can1 {
2           pinctrl-names = "default", "sleep";
3           pinctrl-0 = <&m_can1_pins_a>;
4           pinctrl-1 = <&m_can1_sleep_pins_a>;
5           status = "okay";
6   };
```

第 3-4 行指定 FDCAN1 两个状态的 pinctrl 节点分别为：m_can1_pins_a 和 m_can1_sleep_pins_a，也就是示例代码 49.3.1.1 中的两个 pinctrl 节点。

第 5 行将 m_can1 节点的 status 属性改为“okay”，也就是使能 FDCAN1。

重新编译设备树。

49.3.2 修复 m_can_platform.c

在做实验的时候，发现 ST 官方 5.4.31 版本内核下的 CAN 驱动不能进行收发数据，想进官网找答案，官网居然在维护。我们只能够使用 ST 官方提供 4.19 版本内核下的 CAN 驱动，此驱动的路径：开发板光盘：1、程序源码/2、Linux 驱动例程/24_can/m_can_platform.c。

只需要用此驱动替换掉 5.4.31 内核中的 drivers/net/can/m_can/m_can_platform.c 即可。替换好以后重新编译内核，然后使用新的内核和设备树启动开发板。

49.3.3 使能 Linux 内核自带的 FDCAN 驱动

正常来说还需要配置 Linux 内核，使能内核里面的 FDCAN 驱动，这些 ST 官方已经帮我们做好了，不需要我们重新配置内核。但是这里为了学习，还是来看一下如何配置内核，使能 FDCAN 驱动。

1、使能 CAN 总线

首先打开 CAN 总线子系统，在 Linux 下 CAN 总线是作为网络子系统的，配置路径如下：

```
-> Networking support
    -> <*> CAN bus subsystem support     //打开 CAN 总线子系统
```

如图 49.3.3.1 所示：

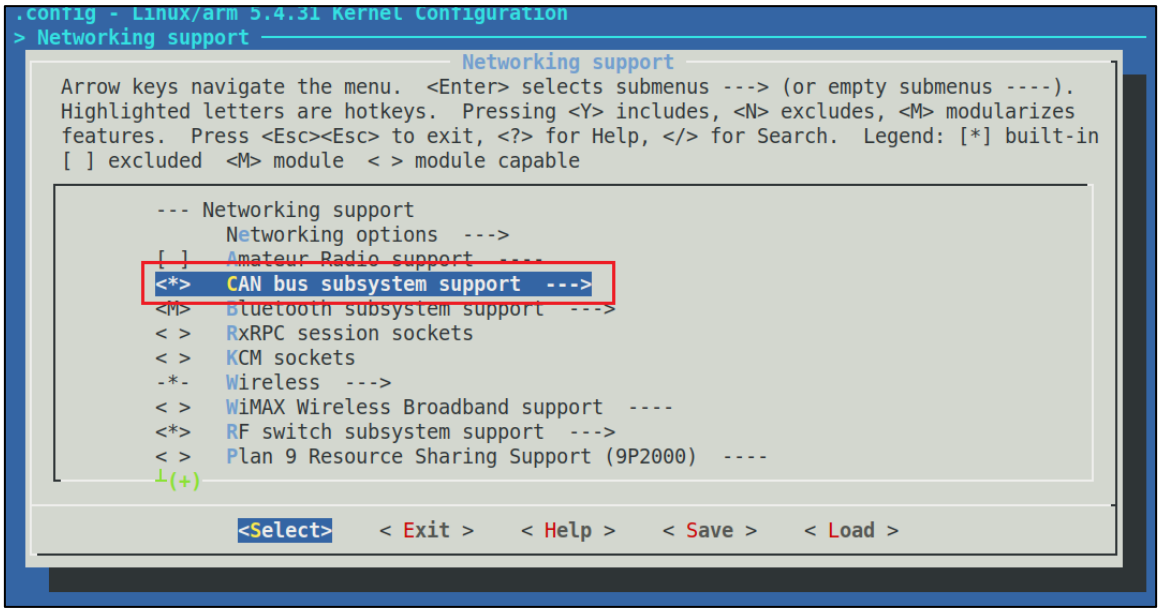


图 49.3.3.1 使能 Linux 内核 CAN 总线子系统

2、使能 STM32MP1 的 FDCAN 外设驱动

接着使能 STM32MP1 的 FDCAN 外设驱动，配置路径如下：

-> Networking support
-> CAN bus subsystem support
-> CAN Device Drivers
-> Platform CAN drivers with Netlink support
-> Bosch M_CAN support
-> <*> Bosch M_CAN support for io-mapped devices(选中)

配置如图 49.3.3.2 所示：

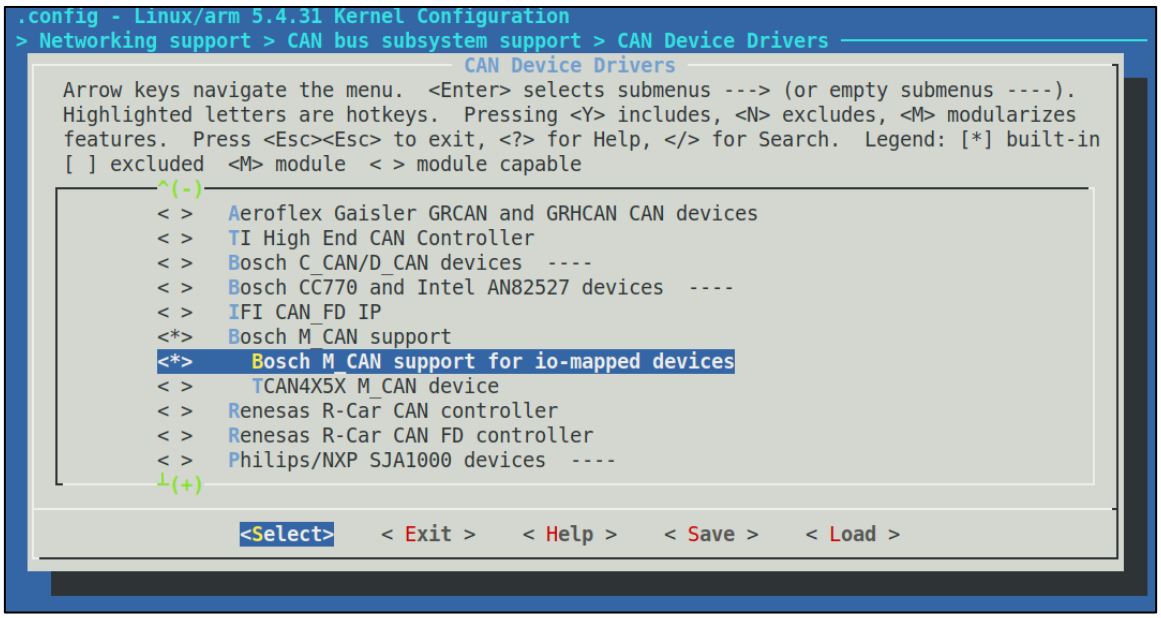


图 49.3.3.2 使能 STM32MP1 的 FDCAN 驱动

49.4 CAN 测试

49.4.1 检查 CAN 网卡设备是否存在

使用新编译的设备树启动开发板，然后输入如下命令：

```
ifconfig -a                    //查看所有网卡
```

前面我们说了，Linux 系统中把 CAN 总线接口设备作为网络设备进行统一管理，因此如果 FDCAN 驱动工作正常的话就会看到 CAN 对应的网卡接口，如图 49.4.1.1 所示：

```
[root@ATK-stm32mp1]:~$ ifconfig -a
can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:50

eth0      Link encap:Ethernet  HWaddr B8:AE:1D:01:01:00
          inet addr:192.168.1.33  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:82636 errors:0 dropped:1949 overruns:0 frame:0
          TX packets:3594 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:15823640 (15.0 MiB)  TX bytes:521428 (509.2 KiB)
          Interrupt:47 Base address:0x4000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[root@ATK-stm32mp1]:~$
```

图 49.4.1.1 当前系统网卡信息

从图 49.4.1.1 可以看出，有一个名为“can0”的网卡，这个就是正点原子的 STM32MP1 开发板上的 CAN1 接口对应的 can 网卡设备。如果使能了 STM32MP1 上的 FDCAN2 的话也会出现一个名为“can1”的 can 网卡设备。

49.4.2 移植 iproute2 和 can-utils 工具

1、移植 iproute2

iproute2 是 Linux 上有关 TCP/IP 网络的一系列工具。我们需要此工具进行配置 CAN 网络设备。还是在 buildroot 的源码目录下，运行以下命令进入图形化界面配置：

```
make menuconfig
```

按照以下路径进入配置我们的 iproute2：

```
-> Target packages
    -> Networking applications
        -> [*] iproute2(选中)
```

结果如图 49.4.2.1 所示：

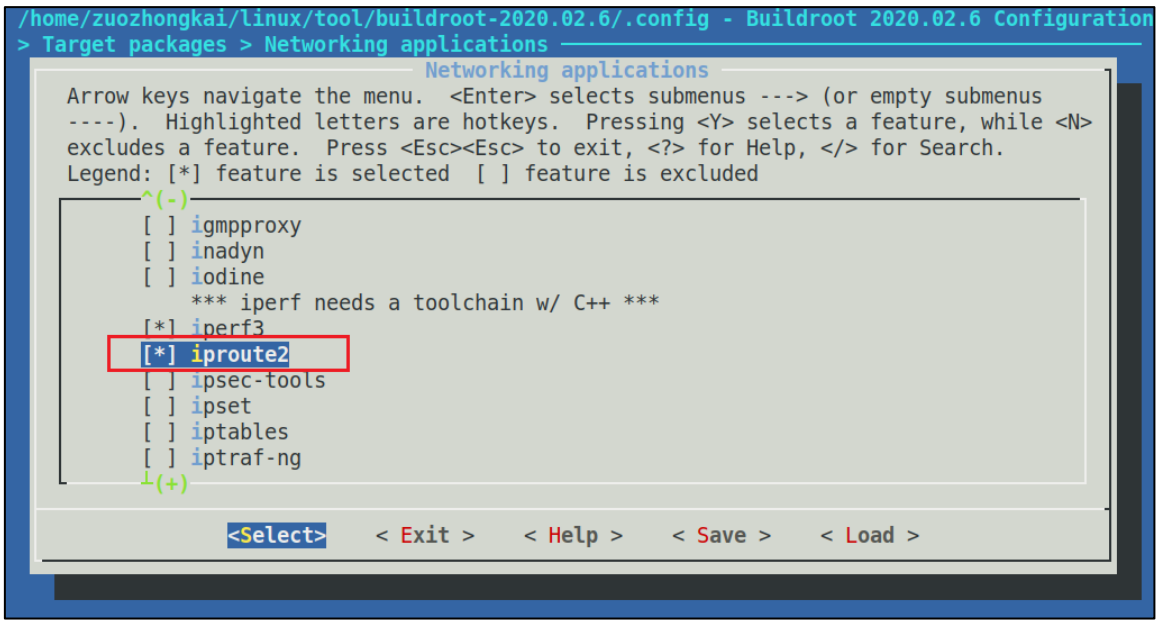


图 49.4.2.1 要配置的 iproute2

2、移植 can-utils 工具

can0 网卡已经出现了，但是工作正不正常还不知道，必须要进行数据收发测试。这里我们使用 can-utils 这个工具来对 can0 网卡进行测试。

按照以下路径进入配置我们的 can-utils：

```
-> Target packages
    -> Networking applications
        ->  [*] can-utils (选中)
```

结果如图 49.4.2.2 所示：

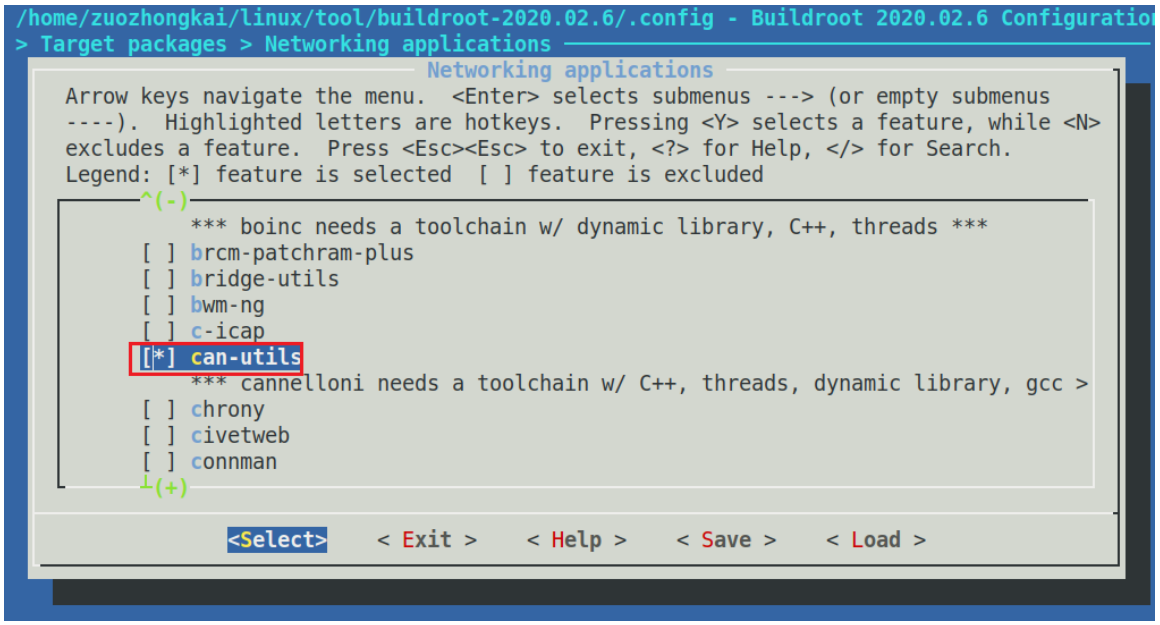


图 49.4.2.2 配置 can-utils 工具

保存和退出配置选项，然后运行以下命令重新编译 buildroot：

```
sudo make
```

编译完成后，进入 output/images 目录，运行以下命令把文件系统替换进去：

```
cd output/images/                                          //进入到 output/images 目录
sudo tar -axvf rootfs.tar -C /home/zuozhongkai/linux/nfs/rootfs  //解压到 nfsroot 目录
```

上述命令将 buildroot 中 output/images/rootfs.tar 这个压缩包解压到/home/zuozhongkai/linux/nfs/rootfs 这个目录中，这个目录就是我当前 nfsroot 目录，大家根据自己的实际情况解压到对应的目录文件中。

重启开发板。

49.4.3 CAN 通信测试

正点原子的 STM32MP1 开发板上只有一个 CAN 接口，因此我们还需要另外一个 CAN 设备，可以使用另一块 STM32MP1 开发板或者 USB 转 CAN 设备。在 49.1.5 小节里提到 STM32MP1 的 CAN 设备支持正常的 CAN 协议和 CAN FD 协议，这里两个都测试一下：

1、CAN 测试

准备一块 STM32MP1 开发板和一个 USB 转 CAN 设备，然后将者 CAN 接口连接起来，STM32MP1 开发板上 CAN 接线端子如图 49.4.3.1 所示：

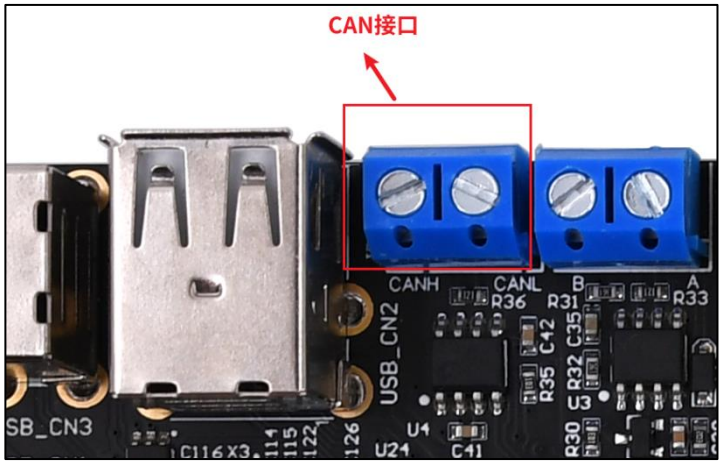


图 49.4.3.1 STM32MP1 开发板的 FDCAN1 接口

将两个开发板的 CAN 接口连接起来(CAN 设备和开发板连接)，注意，CAN_H 接 CAN_H，CAN_L 接 CAN_L！

MCP2560FDT 所允许的最小速率为 14.4Kbps，TJA1042 和 SIT1042 所允许的最低速率为 40kbps！

①、**收发测试**

首先使用 ip 命令设置两个开发板的 CAN 接口，首先设置 CAN 接口的速度，输入如下所示命令：

```
ip link set can0 type can bitrate 1000000
```

上述命令设置 can0 速度为 1000Kbit/S，两个 CAN 设备的速度要设置为一样的！速度设置好以后打开 can0 网卡，命令如下：

```
ifconfig can0 up          //打开 can0
```

can0 打开以后就可以使用 can-utils 里面的小工具进行数据收发测试了。开发板输入如下命令进行接收：

```
candump can0              //接收数据
```

通过 USB 转 CAN 设备发送 8 个字节的数据：0X00、0X01、0X02、0X03、0X04、0X05、

0X06、0X07。

此时开发板就会接收到这 8 个字节的数据，如图 49.4.3.2 所示：

```
[root@ATK-stm32mp1]:~$ candump can0
  can0  000   [8]  00 01 02 03 04 05 06 07
```

图 49.4.3.2 开发板接收到数据

接下来测试一下开发板向 USB 转 CAN 设备发送设备，在开发板上输入如下命令：

```
cansend can0 5A1#11.22.33.44.55.66.77.88
```

cansend 命令用于发送 can 数据，“5A1”是帧 ID，“#”号后面的“11.22.33.44.55.66.77.88”就是要发送的数据，十六进制。CAN2.0 一次最多发送 8 个字节的数据，8 个字节的数据之间用“.”隔开。

如果 USB 转 CAN 设备工作正常的话接收端就会接收到上面发送过来的这 8 个字节的数据，如图 49.4.3.3 所示：

序号	时间标识	源通道	帧ID	CAN类型	方向	长度	数据
				全部	全部		
0	0.000000	0	0x5A1	CAN	Rx	8	11 22 33 44 55 66 77 88

图 49.4.3.3 接收到的数据

从图 49.4.3.3 可以看出，USB 转 CAN 设备接收到了 8 个字节的数据，帧 ID 为 5A1，说明 CAN 驱动工作正常。

如果要关闭 can0 的话输入如下命令：

```
ifconfig can0 down
```

2、CAN FD 协议测试

CAN FD 的波特率最高为 1000 kBit/s，数据波特率为 5000 kBit/s。

这里我们使用 USB 转 CAN FD 设备，首先设置开发板的 CAN0 接口，配置 CAN FD 的速率，三选一，只需要选择下面其中一条指令来配置即可(这里只是测到这个三个波特率能使用，其它波特率各位自行测试)。

```
ip link set can0 up type can bitrate 1000000 dbitrate 5000000 fd on
ip link set can0 up type can bitrate 200000 dbitrate 1000000 fd on
ip link set can0 up type can bitrate 100000 dbitrate 500000 fd on
```

接着我们运行以下命令使能开发板的接收功能：

```
candump can0                                    //candump 接收数据
```

上位机可以开始发送数据，我们开发板就能够接收数据，接收到如下所示：

```
[root@ATK-stm32mp1]:~$ candump can0
  can0  000   [64]  11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF 11 22 33 44 55 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66
  can0  000   [64]  11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF 11 22 33 44 55 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66
```

图 49.4.3.4 CAN FD 接收数据测试

图 49.4.3.4 中接收到的数据为 64 位，刚好是 CAN FD 能够最大接收数据。

接下来测试从开发板发送数据到 USB 转 CAN 设备，输入如下命令：

```
cansend can0 123##300.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff
```

USB 转 CAN 设备接收到数据如下图所示：

序号	帧间隔时间us	名称	帧ID	帧类型	帧格式	DLC	数据	帧数量
00000001	66494.802.765	接收	123	FD.BRS	STANDARD	64	00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF ...	1
00000002	000.796.213	接收	123	FD.BRS	STANDARD	64	00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF ...	1

图 49.4.3.5 CAN 设备接收数据

关于 CAN 驱动就讲解到这里，如果要编写 CAN 总线应用的话就直接使用 Linux 提供的 CAN 接口，使用方法类似网络通信，本教程不讲解应用编程。

49.4.4 CAN 500K 收发异常处理方法

大家在测试标准 CAN 的时候会发现 500K 波特率的时候开发板接收有问题，这个并不是板子硬件有问题，而是需要修改 TF-A 下的 PLL4 频率。打开 TF-A 中 fdts/stm32mp157d-atk.dtsi 文件，找到 pll4 的时钟配置，如图 49.4.4.1 所示：

```
207     /* VCO = 594.0 MHz => P = 99, Q = 74, R = 74 */
208     pll4: st,pll@3 {
209         compatible = "st,stm32mp1-pll";
210         reg = <3>;
211         cfg = <3 98 5 7 7 PQR(1,1,1)>;
212     };
```

图 49.4.4.1 pll4 默认配置

图 49.4.4.1 中是 TF-A 下的 PLL4 的时钟配置，具体配置方法参考 TF-A 下的绑定文档：docs/devicetree/bindings/clock/st,stm32mp1-rcc.txt。ST 默认配置 FDCAN 的时钟源为 PLL4R，根据 st,stm32mp1-rcc.txt 文档描述，可以算出默认配置下 PLL4R=74.25MHz。这里我们需要将 PLL4R 的频率改为 60MHz，也就是将 FDCAN 的时钟源设置为 60MHz。将图 49.4.4.1 中第 211 行的 cfg 改为：

```
cfg = <1 34 3 4 6 PQR(1,1,1)>;
```

修改以后如图 49.4.4.2 所示：

```
207     /* VCO = 594.0 MHz => P = 99, Q = 74, R = 74 */
208     pll4: st,pll@3 {
209         compatible = "st,stm32mp1-pll";
210         reg = <3>;
211         /* cfg = <3 98 5 7 7 PQR(1,1,1)>; */
212         cfg = <1 34 3 4 6 PQR(1,1,1)>;
213     };
214 };
```

图 49.4.4.2 修改后的 PLL4

经过上面修改以后 PLL4P=105MHz PLL4Q=84MHz，PLL4R=60MHz。重新编译 TF-A，然后使用新的 TF-A 启动开发板。

系统启动以后再次测试标准 CAN 500KHz 的时候数据收发，此时收发就正常了！

第五十章 Linux USB 驱动实验

USB 是很常用的接口，目前大多数的设备都是 USB 接口的，比如鼠标、键盘、USB 摄像头等，我们在实际开发中也常常遇到 USB 接口的设备，本章我们就来学习一下如何使能 Linux 内核自带的 USB 驱动。注意！本章并不讲解具体的 USB 开发，因为 USB 接口很复杂，不同的设备其协议也不同，这不是简简单单一章内容就能说完的，USB 驱动开发本身就是一门复杂的课程。所以，如果想要学习如何编写代码开发一个全新的 USB 设备那就可以跳过本章。

50.1 USB 接口简介

关于 USB 详细的协议内容请参考《USB2.0 协议中文版.pdf》和《USB3.0 协议中文版.pdf》，这两份文档已经放到了开发板光盘中，存放在“4、参考资料” 中。

50.1.1 什么是 USB?

USB 全称为 Universal Serial Bus，翻译过来就是通用串行总线。由英特尔与众多电脑公司提出来，用于规范电脑与外部设备的连接与通讯。目前 USB 接口已经得到了大范围的应用，已经是电脑、手机等终端设备的必配接口，甚至取代了大量的其他接口。比如最新的智能手机均采用 USB Typec 取到了传统的 3.5mm 耳机接口，苹果最新的 MacBook 只有 USB Typec 接口，至于其他的 HDMI、网口等均可以通过 USB Typec 扩展坞来扩展。

按照大版本划分，USB 目前可以划分为 USB1.0、USB2.0、USB3.0 以及正在即将到来的 USB4.0。

USB1.0：USB 规范于 1995 年第一次发布，由 Inter、IBM、Microsoft 等公司组成的 USB-IF(USB Implement Forum)组织提出。USB-IF 与 1996 年正式发布 USB1.0，理论速度为 1.5Mbps。1998 年 USBIF 在 USB1.0 的基础上提出了 USB1.1 规范。

USB2.0：USB2.0 依旧由 Inter、IBM、Microsoft 等公司提出并发布，USB2.0 分为两个版本:Full-Speed 和 High-Speed，也就是全速(FS)和高速(HS)。USB2.0 FS 的速度为 12Mbps，USB2.0 HS 速度为 480Mbps。目前大多数单片机以及低端 Cortex-A 芯片配置的都是 USB2.0 接口，比如 STM32MP157，USB2.0 全面兼容 USB1.0 标准。

USB3.0：USB3.0 同样由 Inter 等公司发起的，USB3.0 最大理论传输速度为 5.0Gbps，USB3.0 引入了全双工数据传输，USB2.0 的 480Mbps 为半双工。USB3.0 中两根线用于发送数据，另外两根用于接收数据。在 USB3.0 的基础上又提出了 USB3.1、USB3.2 等规范，USB3.1 理论传输速度提升到了 10Gbps，USB3.2 理论传输速度为 20Gbps。为了规范 USB3.0 标准的命名，USB-IF 公布了最新的 USB 命名规范，原来的 USB3.0 和 USB3.1 命名将不会采用，所有的 3.0 版本的 USB 都命名为 USB3.2，以前的 USB3.0、USB3.1 和 USB3.2 分别叫做 USB3.2 Gen1、USB3.2 Gen2、USB3.2 Gen 2X2。

USB4.0：目前还在标准定制中，目前还没有设备搭载，据说是在 Inter 的雷电 3 接口上改进而来。USB4.0 的速度将提升到了 40Gbps，最高支持 100W 的供电能力，只需要一根线就可以完成数据传输与供电，极大的简化了设备之间的链接线数，期待 USB4.0 设备上市。

如果按照接口类型划分的话 USB 就要分为很多种了，最常见的就是 USB A 插头和插座，如图 50.1.1.1 所示：

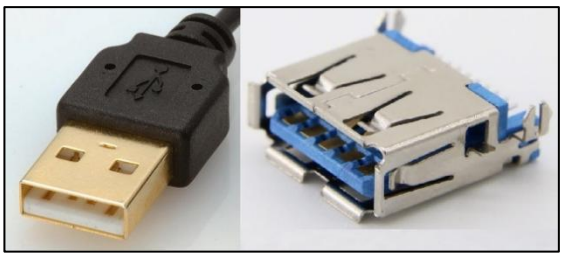


图 50.1.1.1 USB A 插头(左)和插座(右)

使用过 JLINK 调试器的朋友应该还见过 USB B 插头和插座，USB B 插头和插座如图 50.1.1.2 所示：

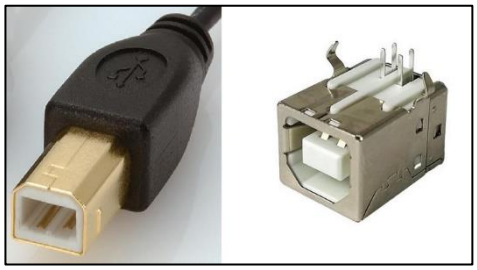


图 50.1.1.2 USB B 插头(左)和插座(右)

USB 插头在不断的缩小，由此产生了 Mini USB 接口， Mini USB 插头和插座如图 50.1.1.3 所示：

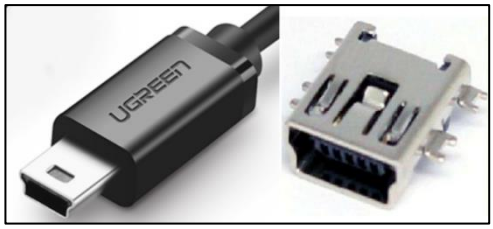


图 50.1.1.3 Mini USB 插头(左)和插座(右)

比 Mini USB 更小的就是 Micro USB 接口了，以前的智能手机基本都是 Micro USB 接口的，Micro USB 插头和插座如图 50.1.1.4 所示：

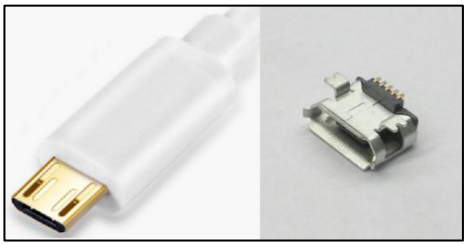


图 50.1.1.4 Micro USB 插头(左)和插座(右)

现在最流行的就是 USB Typec 了，正点原子的 STM32MP1 开发板使用的是 USB Typec 接口，USB Typec 插头和插座如图 50.1.1.5 所示：

图 50.1.1.5 USB Typec 插头(左)和插座(右)

50.1.2 USB 电气特性

1、Mini USB 电气属性

我们先以 Mini USB 为例讲解一下 USB 的基本电气属性。Mini USB 线一般都是一头为 USB A 插头，一头为 Mini USB 插头。一共有四个触点，也就是 4 根线，这四根线的顺序如图 50.1.2.1 所示：

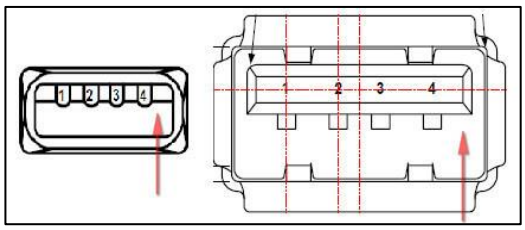


图 50.1.2.1 USB A 插头线序

如图 50.1.2.1 所示，USB A 插头从左到右线序依次为 1,2,3,4，第 1 根线为 VBUS，电压为 5V，第 2 根线为 D-，第 3 根线为 D+，第 4 根线为 GND。USB 采用差分信号来传输数据，因此有 D-和 D+两根差分信号线。大家仔细观察的话会发现 USB A 插头的 1 和 4 这两个触点比较长，2 和 3 这两个触点比较短。1 和 4 分别为 VBUS 和 GND，也就是供电引脚，当插入 USB 的时候会先供电，然后再接通数据线。拔出的时候先断开数据线，然后再断开电源线。

大家再观察一下 Mini USB 插头，会发现 Mini USB 插头有 5 个触点，也就是 5 根线，线序从左往右依次是 1~5。第 1 根线为 VCC(5V)，第 2 根线为 D-，第 3 根线为 D+，第 4 根线为 ID，第 5 根线为 GND。可以看出 Mini USB 插头相比 USB A 插头多了一个 ID 线，这个 ID 线用于实现 OTG 功能，通过 ID 线来判断当前连接的是主设备(HOST)还是从设备(SLAVE)。

USB 是一种支持热插拔的总线接口，使用差分线(D-和 D+)来传输数据，USB 支持两种供电模式：总线供电和自供电，总线供电就是由 USB 接口为外部设备供电，在 USB2.0 下，总线供电最大可以提供 500mA 的电流。

50.1.3 USB 拓扑结构

USB 是主从结构的，也就是分为主机和从机两部分，一般主机叫做 Host，从机叫做 Device。主机通过 USB A 插座来连接外部的设备，比如电脑作为主机，对外提供 USB A 插座，我们可以通过 USB 线来连接一些 USB 设备，比如声卡、手机等。因此电脑带的 USB A 插座数量就决定了你能外接多少个 USB 设备，如果不够用的话我们可以购买 USB 集线器来扩展电脑的 USB 插口，USB 集线器也叫做 USB HUB，USB HUB 如图 50.1.3.1 所示：

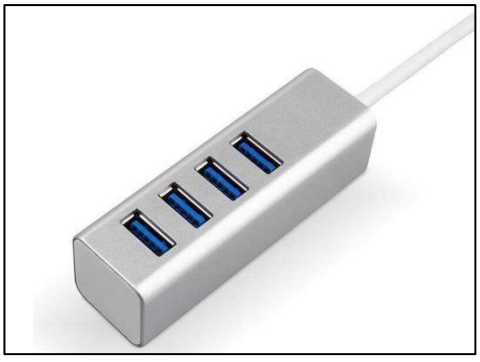


图 50.1.3.1 USB HUB

图 50.1.3.1 是一个一拖四的 USB HUB，也就是将一个 USB 接口扩展为 4 个。主机一般会带几个原生的 USB 主控制器，比如 STM32MP1 就有两个原生的 USB 主控制器，因此 STM32MP1 对外提供两个 USB 接口，这两个接口肯定不够用，正点原子的 STM32MP1 开发板上有 6 个 HOST 接口，六路都是 USB2 通过 USB HUB 芯片扩展出来的，稍后我们会讲解其原理图。

虽然我们可以对原生的 USB 口数量进行扩展，但是我们不能对原生 USB 口的带宽进行扩展，比如 STM32MP1 的两个原生 USB 口都是 USB2.0 的，带宽最大为 480Mbps，因此接到下面的所有 USB 设备总带宽最大为 480Mbps。

USB 只能主机与设备之间进行数据通信，USB 主机与主机、设备与设备之间是不能通信的。因此两个正常通信的 USB 接口之间必定有一个主机，一个设备。为此使用了不同的插头和插座来区分主机与设备，比如主机提供 USB A 插座，从机提供 Mini USB、Micro USB 等插座。在一个 USB 系统中，仅有一个 USB 主机，但是可以有多个 USB 设备，包括 USB 功能设备和 USB HUB，最多支持 127 个设备。一个 USB 主控制器支持 128 个地址，地址 0 是默认地址，只有在设备枚举的时候才会使用，地址 0 不会分配给任何一个设备。所以一个 USB 主控制器最多可以分配 127 个地址。整个 USB 的拓扑结构就是一个分层的金字塔形，如图 50.1.3.2 所示(参考自 USB2.0 协议中文版.pdf)：

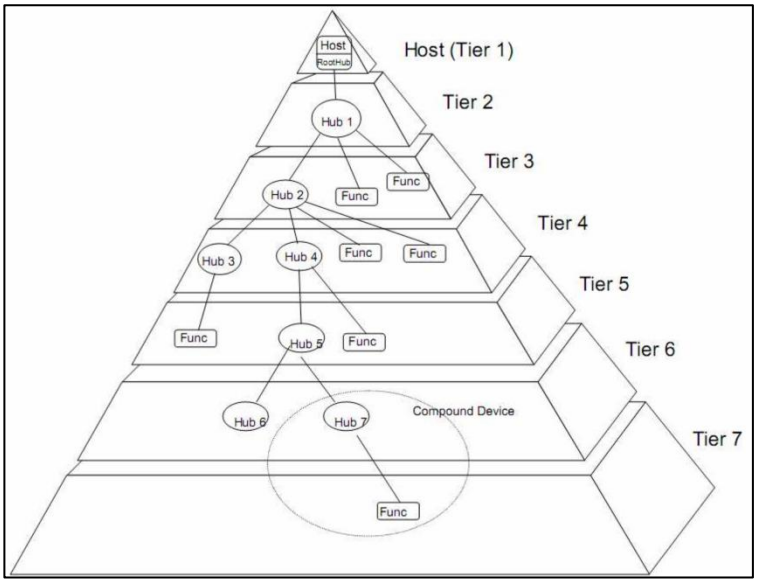


图 50.1.3.2 USB 金字塔拓扑

图 50.1.3.2 中可以看出从 Root Hub 开始，一共有 7 层，金字塔顶部是 Root Hub，这个是 USB 控制器内部的。图中的 Hub 就是连接的 USB 集线器，Func 就是具体的 USB 设备。

USB 主机和从机之间的通信通过管道(Pipe)来完成，管道是一个逻辑概念，任何一个 USB 设备一旦上电就会存在一个管道，也就是默认管道，USB 主机通过管道来获取从机的描述符、配置等信息。在主机端管道其实就是一组缓冲区，用来存放主机数据，在设备端管道对应一个特定的端点。

50.1.4 什么是 USB OTG?

前面我们讲了，USB 分为 HOST(主机)和从机(或 DEVICE)，有些设备可能有时候需要做 HOST，有时候又需要做 DEVICE，配两个 USB 口当然可以实现，但是太浪费资源了。如果一个 USB 接口既可以做 HOST 又可以做 DEVICE 那就太好了，使用起来就方便很多。为此，USB OTG 应运而生，OTG 是 On-The-Go 的缩写，支持 USB OTG 功能的 USB 接口既可以做 HOST，也可以做 DEVICE。那么问题来了，一个 USB 接口如何知道应该工作在 HOST 还是 DEVICE 呢？这里就引入了 ID 线这个概念，前面讲解 USB 电气属性的时候已经说过了，Mini USB 插头有 5 根线，其中一条就是 ID 线。ID 线的高低电平表示 USB 口工作在 HOST 还是 DEVICE 模式：

ID=1：OTG 设备工作在从机模式。

ID=0：OTG 设备工作在主机模式。

支持 OTG 模式的 USB 接口一般都是 Mini USB、Micro USB 等这些带有 ID 线的接口。正点原子的 STM32MP1 开发板 OTG 模式是使用 USB Type C 做接口，没有 ID 线。USB Type C 有

自己的识别方法，稍后会讲解 TypeC 接口电气属性。正点原子的 STM32MP157 开发板 USB_OTG 连接到了 STM32MP1 的 USB1 接口上。如果要使用 OTG 的主机模式，那么就需要一根 OTG 线，TypeC 接口的 OTG 线如图 50.1.4.1 所示：

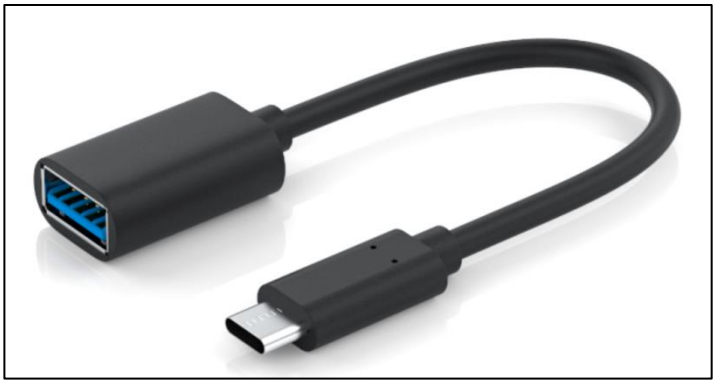


图 50.1.4.1 Typec OTG 线

可以看出，TypeC OTG 线一头是 USB A 插座，一头是 Typec 插头，将 TypeC 插头插入机器的 TypeC 口上，需要连接的 USB 设备插到另一端的 USB A 插座上，比如 U 盘啥的。TypeC 的 CC 引脚检查到 USB 设备已经接入，就会建立 USB 设备为主模式，机器就知道自己要做为一个主机，用来连接外部的从机设备(U 盘)。

50.1.5 STM32MP1 USB 接口简介

STM32MP157 提供了两个 USB2.0 接口，这两个 USB 接口都是支持高速模式，也就是 480Mbit/S，这两个 USB 接口都内置了高速 PHY。其中 USB2 支持 OTG 功能，正点原子 STM32MP157 开发板上的 USB OTG 接口就是链接到 USB2 接口上的。USB1 接口连接了一个 HUB 芯片，实现了 USB Host 接口扩展。

STM32MP1 内部集成了三个跟 USB 相关的控制器名字分别为：USB OTG 控制器、USB Host 控制器和 USB HS PHY 控制器，提供一个简称方便书写：OTG、USBH 和 PHY。接着我们分析一下这三个控制器是如何工作的。

1、PHY 控制器

PHY（英语：Port Physical Layer），中文可称之为端口物理层，是一个对 OSI 模型物理层的共同简称。PHY 控制器主要是提供两个端口，端口 1 已经规定分配给 USB Host 控制器，端口 2 可以分配给 USB OTG 和 USB Host。

2、OTG 控制器

此控制器是支持 OTG 功能，它的内部特性如下所示：

①、此控制器有一个独立的内核 USB OTG HS。

②、该控制器支持 HS、FS 和 LS 模式，不管是主机还是从机模式都支持 HS/FS/LS，硬件支持 OTG 信号、会话请求协议和主机协商协议，支持 8 个双向端点，还支持 PHY 接口。

③、内嵌一个 DMA。

④、支持片上全速 PHY、连接外部全速 PHY 的 I2C 接口和连接外部高速 PHY 的 ULPL 接口。

⑤、具有采用高级 FIFO 控制的 4 KB 专用 RAM。

OTG 控制器有两个模式：正常模式(normal mode)和低功耗模式(low power mode)。 OTG 控制器都可以运行在高速模式(HS 480Mbps)、全速模式(LS 12Mbps)和低速模式(1.5Mbps)。正常模

式下每个 OTG 控制器都可以工作在主机(HOST)或从机(DEVICE)模式下，每个 USB 控制器都有其对应的接口。低功耗模式顾名思义就是为了节省功耗，USB2.0 协议中要求，设备在上行端口检测到空闲状态以后就可以进入挂起状态。在从机(DEVICE)模式下，端口停止活动 3ms 以后 OTG 控制器内核进入挂起状态。在主机(HOST)模式下，OTG 控制器内核不会自动进入挂起状态，但是可以通过软件设置。不管是本地还是远端的 USB 主从机都可以通过产生唤醒序列来重新开始 USB 通信。

3、USBH 控制器

USBH 控制器这是一个主机控制器，此控制器由 EHCI 控制器和 OHCI 控制器组成。USBH 控制器只能做主机模式。

这里我们简单提一下 OHCI、UHCI、EHCI 和 xHCI，这三个是用来描述 USB 控制器规格的，区别如下：

OHCI：全称为 Open Host Controller Interface，这是一种 USB 控制器标准，厂商在设计 USB 控制器的时候需要遵循此标准，用于 USB1.1 标准。OHCI 不仅仅用于 USB，也支持一些其他的接口，比如苹果的 Firewire 等，OHCI 由于硬件比较难，所以软件要求就降低了，软件相对来说比较简单。OHCI 主要用于非 X86 的 USB，比如扩展卡、嵌入式 USB 控制器。

UHCI：全称是 Universal Host Controller Interface，UHCI 是 Inter 主导的一个用于 USB1.0/1.1 的标准，与 OHCI 不兼容。与 OHCI 相比 UHCI 硬件要求低，但是软件要求相应就高了，因此硬件成本上就比较低。

EHCI：全称是 Enhanced Host Controller Interface，是 Inter 主导的一个用于 USB2.0 的 USB 控制器标准。EHCI 仅提供 USB2.0 的高速功能，至于全速和低速功能就由 OHCI 或 UHCI 来提供。

xHCI：全称是 eXtensible Host Controller Interface，是目前最流行的 USB3.0 控制器标准，在速度、能效和虚拟化等方面比前三个都有较大的提高。xHCI 支持所有速度种类的 USB 设备，xHCI 出现的目的就是为了替换前面三个。

通俗来讲，OHCI 就是 FS 模式，也就是低速模式，EHCI 是 HS 模式，也就是高速模式。关于 STM32MP1 的 USB 控制器就简单的讲解到这里，至于更详细的内容请参考 STM32MP1 参考手册中的 60-62 章节。

50.1.6 USB Typec 电气属性

1、USB TypeC 接口引脚定义

STM32MP157 有两个 USB2 接口，所以可以直接使用 Mini USB 或者 Micro USB。但是目前 USB TypeC 接口非常普及，大家一般都有很多 USB TypeC 线材，相反，Mini USB 或者 Micro USB 的线材基本上都没有了，所以为了方便大家使用，正点原子 STM32MP157 开发板上的 USB 接口采用了 TypeC。

接下来就看一下 USB TypeC 接口(根据 USB 协议，也叫做 USB3.1 接口)电气属性，由于 TypeC 功能比较复杂，比如支持 PD 充电、显示、音频等，这里我们只讲解一下 TypeC 的数据通信部分。首先来看一下 TypeC 的接口定义，这里我们只看母头的引脚定义，如图 50.1.6.1 所示：

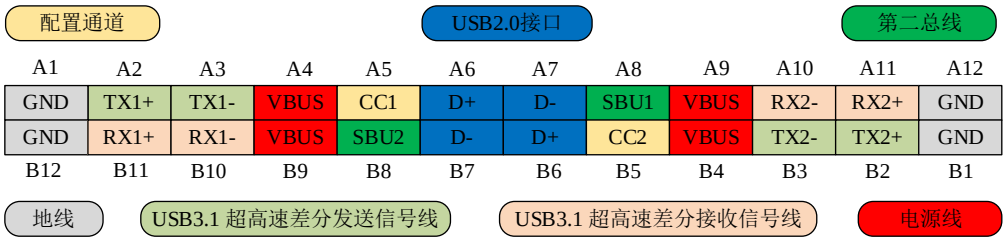


图 50.1.6.1 标准 USB TypeC 母头引脚

从图 50.1.6.1 中可以看出，标准的 USB TypeC 有 24 根线，分为上下两部分，明显比前面讲的 Mini USB 要多不少引脚，这些引脚按照功能分类：

A1、A12、B1、B12：这 4 个引脚为 GND。

A2、A3、B2、B3：这 4 个引脚是 USB3.1 特有的，为超高速差分发送信号线，这个是 USB2.0 和 USB1.0 所没有的。

A10、A11、B10、B11：这 4 个引脚也是 USB3.1 特有的，为超高速差分接收信号线，这个也是 USB2.0 和 USB1.0 所没有的。

A4、A9、B4、B9：VBUS 引脚，这个 USB2.0 也有。

A5、B5：CC1 和 CC2 这 2 个引脚为 USB3.1 特有的配置通道引脚。Type-C 的插座中有两个 CC 脚，USB 通信中的角色检测都是通过 CC 脚进行的。对于 TypeC 插头或者线缆正常只有一个 CC 引脚，两个端口连接在一起之后，只存在一个 CC 引脚连接，通过检测哪一个 CC 有连接，就可以判断连接的方向。如果 USB 线缆中有需供电的器件，其中一个 CC 引脚将作为 VCONN 供电。

对于本章教程来说，我们主要使用 CC1 和 CC2 引脚来实现 USB OTG 功能，稍后会讲解 USB3.1 下的设备类型。

A8、B8：SBU1 和 SBU2 引脚，这两个并不是 USB 信号，而是用作其他功能的，比如 TypeC 作为 DP 接口使用的时候 SBU 用作音频传输通道。

A6、A7、B6、B7：这 4 个引脚大家应该就很熟悉了，就是 USB2.0 的 D+和 D-，因为 USB3.1 要兼容 USB2.0 和 USB1.0 接口，所以必须要有 D+和 D-。

大家仔细观察图 50.1.3.1 会发现，上下两排引脚是对应的，比如(A1,B1)、(A2、B2)等，一直到(A12,B12)。这是因为 USB3.1 要有能够正反插，因此上下两排引脚肯定要是一样的，这样才能在正反插的时候都能正常工作。

图 50.1.3.1 是 TypeC 接口标准的 24P 母头引脚，大家在进行 TypeC 母头选型的时候会发现也有 16P 甚至 6P 封装的接口。比如正点原子 STM32MP157 开发板上使用的就是 16P 的 TypeC 母座，16P 的母座引脚结构如图 50.1.3.2 所示：

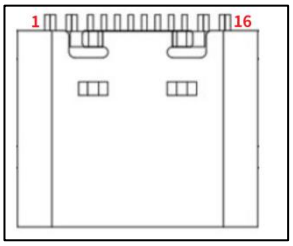


图 50.1.6.2 16P TypeC 引脚示意图

图 50.1.6.2 就是 16P 的 TypeC 引脚示意图，从左到右依次是 1~16 脚，注意 1,2 脚、3,4 脚、13,14 脚和 15,16 脚离得很近，看起来像是一个引脚，其实他们是两个引脚。有些封装为了方便，会将这些离的近的引脚画在一起，看起来就是 12P 的座子，这里一定要注意这点，尤其是自己做板子，画 TypeC 的时候。16P 对应的引脚功能如表 50.1.6.1 所示：

编号	USB3.1 引脚	功能	编号	USB3.1 引脚	功能
1	A1	GND	**9**	A7	D-
2	B12	GND	**10**	B6	D+
3	A4	VBUS	**11**	A8	SBU1
4	B9	VBUS	**12**	B5	CC2
5	B8	SBU2	**13**	B4	VBUS
6	A5	CC1	**14**	A9	VBUS
7	B7	D-	**15**	B1	GND
8	A6	D+	**16**	A12	GND

表 50.1.6.1 16P 母座引脚定义

可以看出，相比于标准 24P 引脚定义，16P 的母座少了 8 根 USB3.1 高速差分收发数据线，这个在本章是用不到的。

2、USB TypeC 的 Data Role

USB2.0 根据数据传输方向定义了 HOST/Device/OTG 这三种类型的设备角色，其中 OTG 既可以做 HOST 也可以做 Device，USB2.0 的 OTG 设备接口通过 ID 线来区分 HOST 还是 Device。在 TypeC 领域，也有类似的概念，但是名字变了：

①、DFP

DFP 全称是 Downstream Facing Port，也就是下行端口，可以理解为 HOST，DFP 提供 VBUS、VCONN，可以接收数据。

②、UFP

UFP 全称是 Upstream Facing Port，也就是上行端口，可以理解为 Device，UFP 从 VBUS 中取电源，UFP 设备也可以传输数据，比如 U 盘，键盘鼠标等。

③、DRP

DRP 全称是 Dual Role Port，也就是双角色端口，可以理解为 OTG，DRP 既可以做 DFP，也可以做 UFP。也可以在 DFP 和 UFP 之间进行动态切换。这个切换过程就用到了 CC 引脚，具体识别过程比较复杂，这里就不讲解了。如果要在 USB TypeC 接口上实现 DRP 功能，那么就需要使用到外置的 TypeC 芯片！正点原子 STM32MP157 开发板使用了 STUSB1600 或 FUSB302MPX 这两种 TypeC 芯片。这两种芯片都有一个 IIC 配置接口，也就是我们需要编写驱动程序，否则的话 DRP 功能无法实现。

50.2 硬件原理图分析

正点原子的 STM32MP1 开发板 USB 部分原理图可以分为两部分：USB HUB 以及 USB OTG。

USB TypeC 本来是给 USB3.1 准备的，但是由于现在大家手里有很多的 TypeC 线材，所以正点原子 STM32MP157 开发板也使用了 TypeC 接口，但是本质上还是 USB2.0 协议，所以大家不要看到 TypeC 就以为支持 USB3.0！

另外，使用 TypeC 接口实现 OTG 功能的话就需要外界 TypeC 芯片！通过专用的 TypeC 芯片来控制 CC 引脚实现 USB 的主从切换。V1.3 版本以前的底板使用 STUSB1600 这颗 TypeC 芯片，V1.5 版本以后的底板都使用 FUSB302MPX。

我们依次来看一下这两部分的硬件原理图。

50.2.1 USB HUB 原理图分析

首先来看一下 USB HUB 原理图，STM32MP1 使用 FE2.1 这个 HUB 芯片将 STM32MP1 的 USB2 扩展成了 7 路 HOST 接口，其中一路供 4G 模块使用，因此就剩下了 6 个通用的 USB A 插座，原理图如图 50.2.1.1 所示：

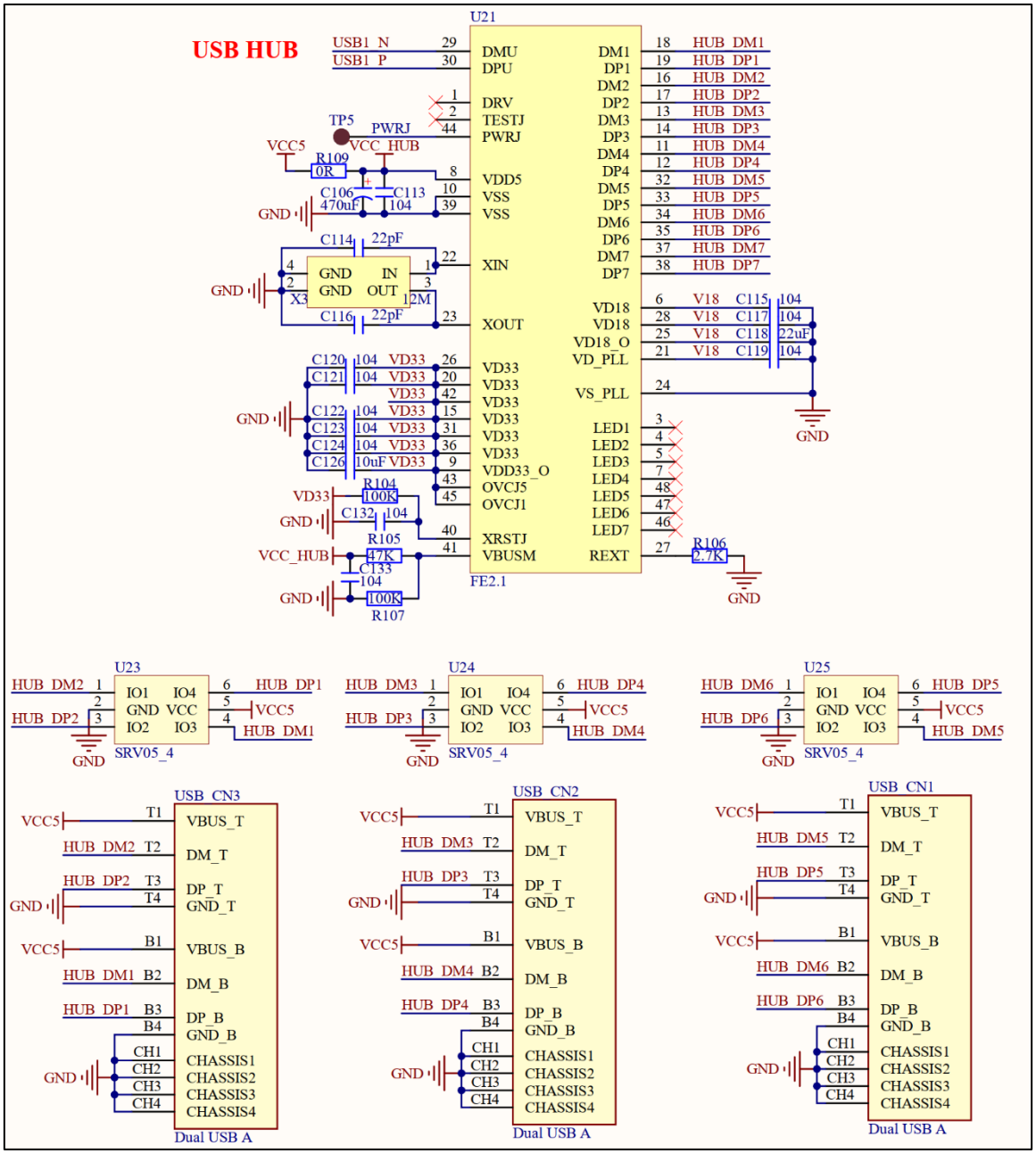


图 50.2.1.1 USB HUB 原理图

图 50.2.1.1 中 U21 就是 USB HUB 芯片 FE2.1，FE2.1 是一款符合 USB2.0 标准的 USB HUB 芯片，支持一拖七扩展，可以将一路 USB 扩展为 7 路 USB HOST 接口。这里我们将 STM32MP1 的 USB1 扩展出了 7 路 USB HOST 接口，分别为 HUB_DP1/DM1、HUB_DP2/DM2、HUB_DP3/DM3、HUB_DP4/DM4、HUB_DP5/DM5、HUB_DP6/DM6 和 HUB_DP7/DM7。其中 HUB_DP7/DM7 用于 4G 模块，因此对外提供的只有六个 USB HOST 接口，这三个 USB HOST 接口如图 50.2.1.3 所示：

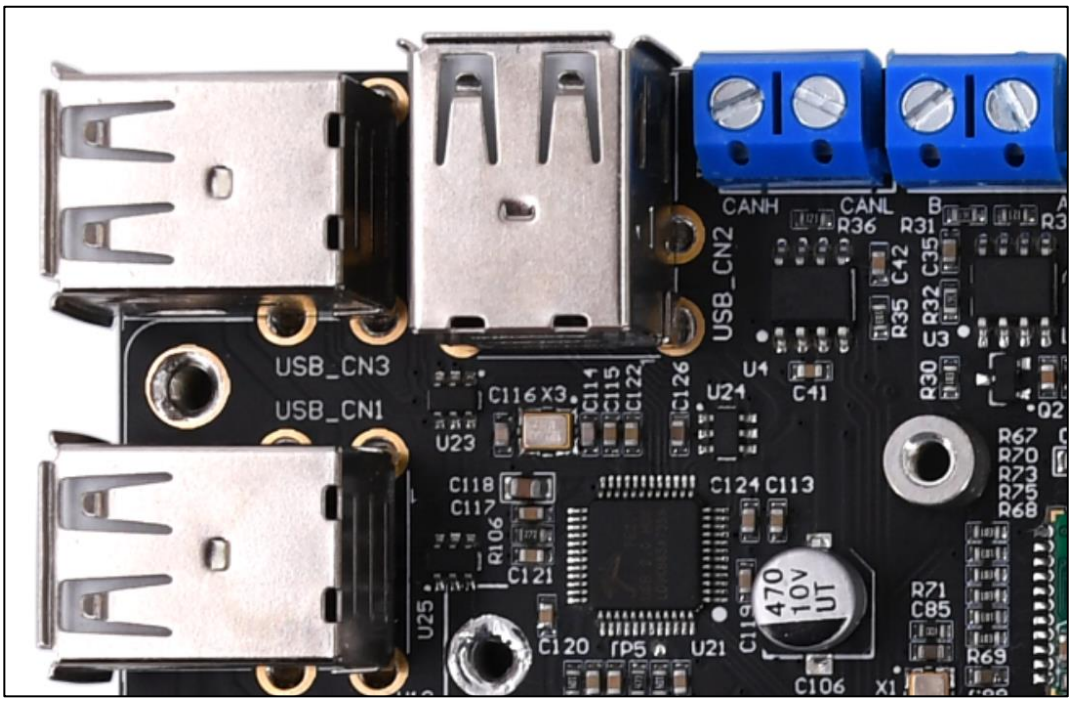


图 50.2.1.3 扩展出来的 USB HSOT

注意，使用 FE2.1 扩展出来的 7 路 USB 接口只能用作 HOST！

50.2.2 USB OTG 原理图分析

正点原子的 STM32MP1 开发板上还有一路 USB OTG 接口，使用 STM32MP1 的 USB OTG 接口。此路 USB OTG 既可以作为主机(HOST)，也可以作为从机(DEVICE)，从而实现完整的 OTG 功能。V1.4 版本以前的底板上 TypeC 芯片使用 STUSB1600，V1.5 及以后版本的底板使用 FUSB302MPX 这颗芯片。

1、V1.4 版本以前底板 USB OTG 原理图

原理图如图 50.2.2.1 所示：

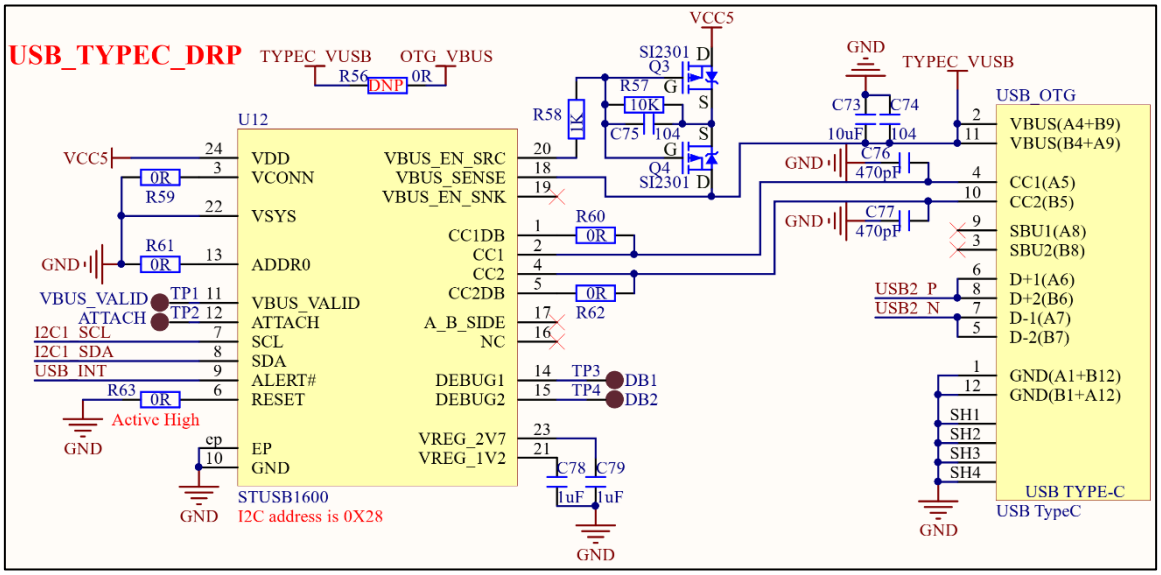


图 50.2.2.1 V1.4 版本的 USB OTG 原理图

在 V1.4 版本以前的底板中 USB OTG 接口使用的是 STUSB1600 做控制，此控制器主要的功能就是负责切换主机和从机模式。I2C1_SCL 和 I2C1_SDA 的引脚分别为：PF14 和 PF15。USB 属于热插拔设备。USB_INT 这个中断引脚主要是检测有没有设备接入，有设备接入电平就会被拉低，驱动进入中断函数，使用 I2C 去读取寄存器配置开发板的主从模式，USB_INT 的引脚为 PG2。

2、V1.5 版本的 USB OTG 原理图分析

在 V1.5 版本及以后的底板里，开发板 USB OTG 是使用 FUSB302MPX 做控制。原理图如 50.2.2.2 所示：

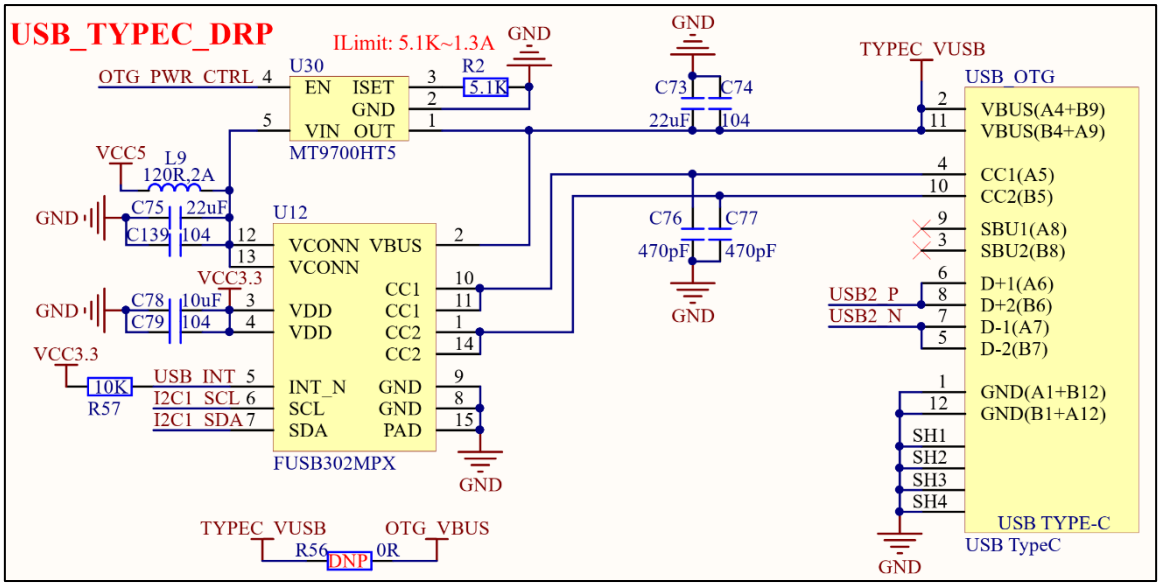


图 50.2.2.2 FUSB302MPX

FUSB302PMX 也是负责控制切换主机和从机模式的，MT9700HT5 是负载开关，用来控制 VBUS 输出，当 OTG_PWR_CTRL 输出高电平的时候 OUT 引脚就输出 5V 电压，也就是 VBUS 变为 5V。当 OTG_PWR_CTRL 输出低电平的时候 OUT 输出 0V，相当于 VBUS 关闭。

随意当开发板上的 TypeC 接口作为主设备的时候，OTG_PWR_CTRL 要输出高电平，VBUS 输出 5V，为外部 USB 设备供电。当 TypeC 接口作为从设备的时候，OTG_PWR_CTRL 输出低电平。OTG_PWR_CTRL 对应的 GPIO 引脚为 PZ6。

50.3 USB 协议简析

USB 协议中有很多的基础概念，本节就来看一下这些概念。

50.3.1 USB 描述符

顾名思义，USB 描述符就是用来描述 USB 信息的，描述符就是一串按照一定规则构建的字符串，USB 设备使用描述符来向主机报告自己的相关属性信息，常用的描述符如表 50.3.1.1 所示：

描述符类型	名字	值
Device Descriptor	设备描述符	1
Configuration Descriptor	配置描述符	2
String Descriptor	字符串描述符	3
Interface Descriptor	接口字符串	4

Endpoint Descriptor	端点描述符	5

表 50.3.1.1 USB 设备常用描述符

我们依次来看一下表 50.3.1.1 中这 5 个描述符的含义：

1、设备描述符

设备描述符用于描述 USB 设备的一般信息，USB 设备只有一个设备描述符。设备描述符里面记录了设备的 USB 版本号、设备类型、VID(厂商 ID)、PID(产品 ID)、设备序列号等。设备描述符结构如表 50.3.1.2 所示：

偏移	域	大小(B)	值类型	描述
0	bLength	1	数字	此设备描述符长度，18 个字节
1	bDescriptorType	1	常量	描述符类型，为 0X01
2	bcdUSB	2	BCD 码	USB 版本号
4	bDeviceClass	1	类	设备类
5	bDeviceSubClass	1	子类	设备子类
6	bDevicePortocol	1	协议	设备协议
7	bMaxPacketSize0	1	数字	端点 0 的最大包长度
8	idVendor	2	ID	厂商 ID
10	idProduct	2	ID	产品 ID
12	bcdDevice	2	BCD 码	设备版本号
14	iManufacturer	1	索引	厂商信息字符串描述符索引值
15	iProduct	1	索引	产品信息字符串描述符索引值
16	iSerialNumber	1	索引	产品序列号字符串描述符索引值
17	bNumConfigurations	1	索引	可能的配置描述符数目

表 50.3.1.2 设备描述符结构

2、配置描述符

设备描述符的 bNumConfigurations 域定义了一个 USB 设备的配置描述符数量，一个 USB 设备至少有一个配置描述符。配置描述符描述了设备可提供的接口(Interface)数量、配置编号、供电信息等，配置描述符结构如表 50.3.1.3 所示：

偏移	域	大小(B)	值类型	描述
0	bLength	1	数字	此配置描述符长度，9 个字节
1	bDescriptorType	1	常量	配置描述符类型，为 0X02
2	wTotalLength	2	数字	整个配置信息总长度(包括配置、接口、端点、设备类和厂家定义的描述符)
4	bNumInterfaces	1	数字	此配置所支持的接口数
5	bConfigurationValue	1	数字	该配置的值，一个设备支持多种配置，通过配置值来区分不同的配置。
6	bConfiguration	1	数字	描述此配置的字符串描述索引
7	bmAttributes	1	数字	该设备的属性： D7：保留 D6：自给电源 D5：远程唤醒

				D4:0：保留
8	bMaxPower	1	数字	此配置下所需的总线电流(单位 2mA)

表 50.3.1.3 配置描述符结构

3、字符串描述符

字符串描述符是可选的，字符串描述符用于描述一些方便人们阅读的信息，比如制造商、设备名称啥的。如果一个设备没有字符串描述符，那么其他描述符中和字符串有关的索引值都必须为 0，字符串描述符结构如表 50.3.1.4 所示：

偏移	域	大小(B)	值类型	描述
0	bLength	1	N+2	此字符串描述符长度
1	bDescriptorType	1	常量	字符串描述符类型，为 0X03
2	wLANGID[0]	2	数字	语言标识码 0
…	…	2	数字	…
N	wLANGID[x]	2	数字	语言标识码 x

表 50.3.1.4 字符串描述符结构

wLANGID[0]~wLANGID[x]指明了设备支持的语言，具体含义要查阅文档《USB_LANGIDs.pdf》，此文档已经放到了开发板光盘中，路径为：4、参考资料->USB_LANGIDs.pdf。

主机会再次根据自己所需的语言向设备请求字符串描述符，这次主机会指明要得到的字符串索引值和语言。设备返回 Unicode 编码的字符串描述符，结构如表 50.3.1.5 所示：

偏移	域	大小(B)	值类型	描述
0	bLength	1	N+2	此字符串描述符长度
1	bDescriptorType	1	常量	字符串描述符类型，为 0X03
2	bString	N	数字	Unicode 编码的字符串

表 50.3.1.5 Unicode 编码的字符串描述符结构

4、接口描述符

配置描述符中指定了该配置下的接口数量，配置可以提供一个或多个接口，接口描述符用于描述接口属性。接口描述符中一般记录接口编号、接口对应的端点数量、接口所述的类等，接口描述符结构如表 50.3.1.6 所示：

偏移	域	大小(B)	值类型	描述
0	bLength	1	数字	此接口描述符长度，9 个字节
1	bDescriptorType	1	常量	描述符类型，为 0X04
2	bInterfaceNumber	1	数字	当前接口编号，从 0 开始
3	bAlternateSetting	1	数字	当前接口备用编号
4	bNumEndpoints	1	数字	当前接口的端点数量
5	bInterfaceClass	1	类	当前接口所属的类
6	bInterfaceSubClass	1	子类	当前接口所属的子类
7	bInterfaceProtocol	1	协议	当前接口所使用的协议
8	iInterface	1	索引	当前接口字符串的索引值

表 50.3.1.6 接口描述符结构

5、端口描述符

接口描述符定义了其端点数量，端点是设备与主机之间进行数据传输的逻辑接口，除了端

点 0 是双向端口，其他的端口都是单向的。端点描述符描述了树传输类型、方向、数据包大小、端点号等信息，端点描述符结构如表 50.3.1.7 所示：

偏移	域	大小(B)	值类型	描述
0	bLength	1	数字	此端点描述符长度，7 个字节
1	bDescriptorType	1	常量	描述符类型，为 0X05
2	bEndpointAddress	1	数字	端点地址和方向： bit3:0：端点号 bit6:4：保留，为零。 bit7：方向，0 输出端点(主机到设备)，1 输入端点(设备到主机)
3	bmAttributes	1	数字	端点属性，bit1:0 表示传输类型： 00：控制传输 01：同步传输 10：批量传输 11：中断传输 其他位保留
4	wMaxPacketSize	2	数字	端点能发送或接收的最大数据包长度
6	bInterval	1	子类	端点数据传输中周期时间间隙值，此域对于批量传输和控制传输无效，同步传输的话此域必须为 1ms，中断传输此域可以设置 1ms~255ms。

表 50.3.1.7 端点描述符结构

50.3.2 USB 数据包类型

USB 是串行通信，需要一位一位的去传输数据，USB 传输的时候先将原始数据进行打包，所以 USB 中传输的基本单元就是数据包。根据用途的不同，USB 协议定义了 4 种不同的包结构：令牌(Token)包、数据(Data)包、握手(Handshake)包和特殊(Special)包。这四种包通过包标识符 PID 来区分，PID 共有 8 位，USB 协议使用低 4 位 PID3~PID0，另外的高四位 PID7~PID4 是 PID3~PID0 的取反，传输顺序是 PID0、PID1、PID2、PID3…PID7。令牌包的 PID1~0 为 01，数据包的 PID1~0 为 11，握手包的 PID1~0 为 10，特殊包的 PID1~0 为 00。每种类型的包又有多种具体的包，如表 50.3.2.1 所示：

PID 类型	PID 名字	PID<3:0>	描述
令牌包	OUT	0001	主机向从机通知将要输出数据
	IN	1001	主机向从机通知将要输入数据
	SOF	0101	帧起始包
	SETUP	1101	开始一个控制传输
数据包	DATA0	0011	偶数据包
	DATA1	1011	奇数据包
	DATA2	0111	适用于高速和等时传输的数据包
	MDATA	1111	
握手包	ACK	0010	确认包，传输正确
	NAK	1010	不确认包，传输错误

	STALL	1110	设备部支持该请求，或者端点挂起
	NYET	0110	接收未响应，未准备好
特殊包	PRE	1100	前导，令牌包
	ERR	1100	错误，握手包
	SPLIT	1000	分裂传输(Split)，令牌包
	PING	0100	PING 测试，令牌包
	Reserved	0000	保留

表 50.3.2.1 数据包结构

一个完整的包分为多个域，所有的数据包都是以同步域(SYNC)开始，后面紧跟着包标识符(PID)，最终都以包结束(EOP)信号结束。不同的数据包中间位域不同，一般有包目标地址(ADDR)、包目标端点(ENDP)、数据、帧索引、CRC 等，这个要具体数据包具体分析。接下来简单看一下这些数据包的结构。

1、令牌包

令牌包结构如图 50.3.2.1 所示：

域	SYNC	SETUP(8b)	ADDR(7b)	ENDP(4b)	CRC5(5b)	EOP
示例	00000001	0XB4	0	0	0X08	

图 50.3.2.1 SETUP 令牌包

图 50.3.2.1 是一个 SETUP 令牌包结构，首先是 SYNC 同步域，包同步域为 00000001，也就是连续 7 个 0，后面跟一个 1，如果是高速设备的话就是 31 个 0 后面跟一个 1。紧跟着是 PID，这里是 SETUP 包，为 0XB4，大家可能会好奇为什么不是 0X2D(00101101)，0XB4 的原因如下：

①、SETUP 包的 PID3~PID0 为 1101，因此对应的 PID7~PID4 就是 0010。

②、PID 传输顺序为 PID0、PID1、PID2…PID7，因此按照传输顺序排列的话此处的 PID 就是 10110100=0XB4，并不是 0X2D。

PID 后面跟着地址域(ADDR)和端点域(ENDP)，为目标设备的地址和端点号。CRC5 域是 5 位 CRC 值，是 ADDR 和 ENDP 这两个域的校验值。最后就是包结束域(EOP)，标记本数据包结束。其他令牌包的结构和 SETUP 基本类似，只是 SOF 令包中间没有 ADDR 和 ENDP 这两个域，而是只有一个 11 位的帧号域。

2、数据包

数据包结构如图 50.3.2.2 所示：

域	SYNC	DATA0(8b)	DATA	DATA	DATA	DATA	CRC16	EOP
示例	00000001	0XC3	字节0	字节1	……	字节N	0	

图 50.3.3.2 数据包

数据包比较简单，同样的，数据包从 SYNC 同步域开始，然后紧跟着是 PID，这里就是 DATA0，PID 值为 0XC3。接下来就是具体的数据，数据完了以后就是 16 位的 CRC 校验值，最后是 EOP。

3、握手包

握手包结构如图 50.3.2.3 所示：

域	SYNC	ACK(8b)	EOP
示例	00000001	0X4B	

图 67.3.3.3 ACK 握手包

图 50.3.2.3 是 ACK 握手包，很简单，首先是 SYNC 同步域，然后就是 ACK 包的 PID，为 0X4B，最后就是 EOP。其他的 NAK、STALL、NYET 和 ERR 握手包结构都是一样的，只是其中的 PID 不同而已。

50.3.3 USB 传输类型

在端点描述符中 bmAttributes 指定了端点的传输类型，一共有 4 种，本节我们来看一下这四种传输类型的区别。

1、控制传输

控制传输一般用于特定的请求，比如枚举过程就是全部由控制传输来完成的，比如获取描述符、设置地址、设置配置等。控制传输分为三个阶段：建立阶段(SETUP)、数据阶段(DATA)和状态阶段(STATUS)，其中数据阶段是可选的。建立阶段使用 SETUP 令牌包，SETUP 使用 DATA0 包。数据阶段是 0 个、1 个或多个输入(IN)/输出(OUT)事务，数据阶段的所有输入事务必须是同一个方向的，比如都为 IN 或都为 OUT。数据阶段的第一个数据包必须是 DATA1，每次正确传输以后就在 DATA0 和 DATA1 之间进行切换。数据阶段完成以后就是状态阶段，状态阶段的传输方向要和数据阶段相反，比如数据阶段为 IN 的话状态阶段就要为 OUT，状态阶段使用 DATA1 包。比如一个读控制传输格式如图 50.3.3.1 所示：

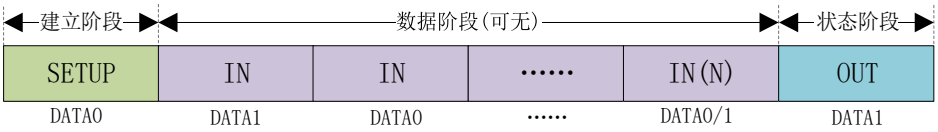


图 50.3.3.1 读控制传输阶段

2、同步传输

同步传输用于周期性、低时延、数据量大的场合，比如音视频传输，这些场合对于时延要求很高，但是不要求数据 100%正确，允许有少量的错误。因此，同步传输没有握手阶段，即使数据传输出错了也不会重传。

3、批量传输

提起“批量”，我们第一反应就是“多”、“大”等，因此，批量传输就是用于大批量传输大块数据的，这些数据对实时性没有要求，比如 MSD 类设备(存储设备)，U 盘之类的。批量传输分为批量读(输入)和批量写(输出)，如果是批量读的话第一阶段就是 IN 令牌包，如果是批量写那么第一阶段就是 OUT 令牌包。

我们就以批量写为例简单介绍一下批量传输过程：

①、主机发出 OUT 令牌包，令牌包里面包含了设备地址、端点等信息。

②、如果 OUT 令牌包正确的话，也就是设备地址和端点号匹配，主机就会向设备发送一个数据(DATA)包，发送完成以后主机进入接收模式，等待设备返回握手包，一切都正确的话设备就会向主机返回一个 ACK 握手信号。

批量读的过程刚好相反：

①、主机发出 IN 令牌包，令牌包里面包含了设备地址、端点等信息。发送完成以后主机就进入到数据接收状态，等待设备返回数据。

②、如果 IN 令牌包正确的话，设备就会将一个 DATA 包放到总线上发送给主机。主机收到这个 DATA 包以后就会向设备发送一个 ACK 握手信号。

4、中断传输

这里的中断传输并不是我们传统意义上的硬件中断，而是一种保持一定频率的传输，中断传输适用于传输数据量小、具有周期性并且要求响应速度快的数据，比如键盘、鼠标等。中断的端点会在端点描述符中报告自己的查询时间间隔，对于时间要求严格的设备可以采用中断传输。

50.3.4 USB 枚举

当 USB 设备与 USB 主机连接以后主机就会对 USB 设备进行枚举，通过枚举来获取设备的描述符信息，主机得到这些信息以后就知道该加载什么样的驱动、如何进行通信等。USB 枚举过程如下：

①、第一回合，当 USB 主机检测到 USB 设备插入以后机会发出总线复位信号来复位设备。USB 设备复位完成以后地址为 0，主机向地址 0 的端点 0 发送数据，请求设备的描述符。设备得到请求以后就会按照主机的要求将设备描述符发送给主机，主机得到设备发送过来的设备描述符以后，如果确认无误就会向设备返回一个确认数据包(ACK)。

②、第二回合，主机再次复位设备，进入地址设置阶段。主机向地址 0 的端点 0 发送设置地址请求数据包，新的设备地址就包含在这个数据包中，因此没有数据过程。设备进入状态过程，等待主机请求状态返回，收到以后设备就会向主机发送一个 0 字节状态数据包，表明设备已经设置好地址了，主机收到这个 0 字节状态数据包以后会返回一个确认包(ACK)。设备收到主机发送的 ACK 包以后就会使用这个新的设备地址，至此设备就得到了一个唯一的地址。

③、第三回合，主机向新的设备地址端点 0 发送请求设备描述符数据包，这一次主机要获取整个设备描述符，一共是 18 个字节。

④、和第③步类似，接下来依次获取配置描述符、配置集合、字符串描述符等等。

50.4 STM32MP1 USB HOST 驱动编写

接下来我们就开始编写 USB HOST 的驱动。USB 子系统是一个标准和复杂的接口，所以驱动基本我们不用写，都是内核里有现成的，我们只需要在设备树提供对应的设备节点即可。

在 50.1.5 小节里告诉各位了 USBH 是只能主机模式，要编写 USB HOST 就用 USBH 控制器即可。ST 官方的 STM3MP157C-DK2 开发板已经配置好了 USBH 的节点信息，所以我们直接参考此节点即可，这样开发板就能够使用 USB Host 模式。

1、USBH 控制器节点信息

打开“stm32mp151.dtsi”文件，找到 USBH 两个控制器的节点信息，名字分别为“usbh_ohci”和“usbh_ehci”。如下示例代码所示：

示例代码 50.4.1 USBH 下两个控制器节点信息

```
1  usbh_ohci: usbh-ohci@5800c000 {
2      compatible = "generic-ohci";
3      reg = <0x5800c000 0x1000>;
4      clocks = <&rcc USBH>;
5      resets = <&rcc USBH_R>;
```

```
6       interrupts = <GIC_SPI 74 IRQ_TYPE_LEVEL_HIGH>;
7       status = "disabled";
8   };
9
10  usbh_ehci: usbh-ehci@5800d000 {
11      compatible = "generic-ehci";
12      reg = <0x5800d000 0x1000>;
13      clocks = <&rcc USBH>;
14      resets = <&rcc USBH_R>;
15      interrupts-extended = <&exti 43 IRQ_TYPE_LEVEL_HIGH>;
16      companion = <&usbh_ohci>;
17      power-domains = <&pd_core>;
18      wakeup-source;
19      status = "disabled";
20  };
```

从上面的代码可以知道 USBH 是支持 USB2.0 和 USB1.1。使用 USB2.0 就要配置 usbh_ehci 节点，使用 USB1.1 就要配置 usbh_ohci 节点。这两个节点的信息我们是不需要修改的，这是 STM32MP1 一些通用配置信息。需要做的就是在 stm32mp157d-atk.dts 文件中追加对应属性信息，然后 status 属性值改为“okay”，还要配置 USBH 使用哪个 PHY 端口。根据两个节点的 compatible 属性找到对应的驱动路径为：drivers/usb/host/ohci-platform.c 和 drivers/usb/host/ehci-platform.c。

2、配置 PHY 控制器

我们先去了解一下 PHY 控制器，一些通用配置，打开 stm32mp151.dtsi 文件，找的如下内容所示：

示例代码 50.4.2 usbphyc 控制器节点信息

```
1   usbphyc: usbphyc@5a006000 {
2       #address-cells = <1>;
3       #size-cells = <0>;
4       #clock-cells = <0>;
5       compatible = "st,stm32mp1-usbphyc";
6       reg = <0x5a006000 0x1000>;
7       clocks = <&rcc USBPHY_K>;
8       resets = <&rcc USBPHY_R>;
9       vdda1v1-supply = <&reg11>;
10      vdda1v8-supply = <&reg18>;
11      status = "disabled";
12
13      usbphyc_port0: usb-phy@0 {
14          #phy-cells = <0>;
15          reg = <0>;
16      };
17
18      usbphyc_port1: usb-phy@1 {
```

```
19          #phy-cells = <1>;
20          reg = <1>;
21      };
22  };
```

示例代码 50.4.1.2 usbphyc 节点就是 STM32MP1 的 USB PHY。我已经知道 PHY 控制器有两个端口，刚好 usbphyc 节点里有两个子节点名字分别为：usbphyc_port0 和 usbphyc_port1，这个两个子节点就是 PHY 控制器的两个端口，其中 usbphyc_port0 只能分配给 USB Host。注意：“#phy-cells”属性和“#gpio-cells”属性作用是一样的，如果#phy-cells 为 1，表示一个 cell，此 cell 表示端口做 USBH 的 PHY 端口还是 OTG 的 PHY 端口，0 表示做 OTG 的 PHY 端口，1 表示做 USBH 的 PHY 端口。

打开“stm32mp15xx-dkx.dtsi”文件找到“usb_phy_tuning”节点，把此节点的内容拷贝到 stm32mp157d-atk.dts 的根目录下，拷贝内容如下所示：

示例代码 50.4.3 添加的 usb_phy_tuning 节点信息

```
1   usb_phy_tuning: usb-phy-tuning {
2       st,hs-dc-level = <2>;
3       st,fs-rftime-tuning;
4       st,hs-rftime-reduction;
5       st,hs-current-trim = <15>;
6       st,hs-impedance-trim = <1>;
7       st,squelch-level = <3>;
8       st,hs-rx-offset = <2>;
9       st,no-lsfs-sc;
10  };
```

usb_phy_tuning 此节点负责调整 PHY 的配置，对于此节点的属性内容感兴趣的可以去看 Documentation/devicetree/bindings/phy/phy-stm32-usbphyc.yaml 文件。接着我还是在 stm32mp157d-atk.dts 文件中使能 usbphyc 以及向 usbphyc_port0 节点追加的内容，要修改的如下所示：

示例代码 50.4.4 使能 usbphyc 和追加 usbphyc_port0 的内容

```
1   &usbphyc {
2       status = "okay";
3   };
4
5   &usbphyc_port0 {
6       phy-supply = <&v3v3>;
7       st,phy-tuning = <&usb_phy_tuning>;
8   };
```

第 2 行，这里我们把 usbphyc 的 status 属性修改为“okay”，使能 usbphyc。

第 6 行，给 usbphyc_port0 节点追加 phy-supply 属性，添加一个电源管理属性。

第 7 行，把我们要修改的 PHY 配置，添加到 usbphyc_port0 节点里。

3、配置 usbh_ehci

最后我们在 stm32mp157d-atk.dts 文件，使能 usbh_ehci 和指定 PHY 端口。

示例代码 50.4.5 追加 usbh_ehci 节点内容

```
1   &usbh_ehci {
```

```
2       phys = <&usbphyc_port0>;
3       status = "okay";
4   };
```

这里的内容很简单，就是修改 status 属性为“okay”和指定使用 usbphyc_port0 端口。修改完就重新编译设备树，使用新的 stm32mp157d-atk.dtb 设备树重新启动开发板。就会有以下输出所示：

```
ehci-platform 5800d000.usbh-ehci: EHCI Host Controller
ehci-platform 5800d000.usbh-ehci: new USB bus registered, assigned bus number 1
ehci-platform 5800d000.usbh-ehci: irq 57, io mem 0x5800d000
ehci-platform 5800d000.usbh-ehci: USB 2.0 started, EHCI 1.00
hub 1-0:1.0: USB hub found
hub 1-0:1.0: 2 ports detected
```

图 50.4.1 usbh-ehci 的打印信息

如果有图 50.4.1 这些输出信息，说明我们的 usb host 驱动加载到内核了。

50.5 Linux 内核自带 HOST 实验

50.5.1 USB 鼠标键盘测试

首先做一下 USB HOST 试验，也就是正点原子的 STM32MP1 开发板做 USB 主机，然后外接 USB 设备，比如 USB 鼠标键盘、USB 转 TTL 串口线、U 盘等设备。Linux 内核已经集成了大量的 USB 设备驱动，尤其是我们常见的 USB 鼠标键盘、U 盘等，本节我们就来学习一下如何使能 Linux 内核常见的 USB 设备驱动。

1、USB 鼠标键盘驱动使能

注意，NXP 官方的 Linux 内核默认已经使能了 USB 键盘鼠标驱动！

USB 鼠标键盘属于 HID 设备，内核已经集成了相应的驱动，ST 官方提供的 linux 内核默认已经使能了 USB 鼠标键盘驱动，但是我们还要学习一下如何手动使能这些驱动。输入“make menuconfig”，打开 linux 内核配置界面，首先打开 HID 驱动，按照如下路径到相应的配置项目：

```
-> Device Drivers
    -> HID support
        -> HID bus support
            ->  <*>  Generic HID driver          //使能通用 HID 驱动
```

使能以后如图 50.5.1.1 所示：

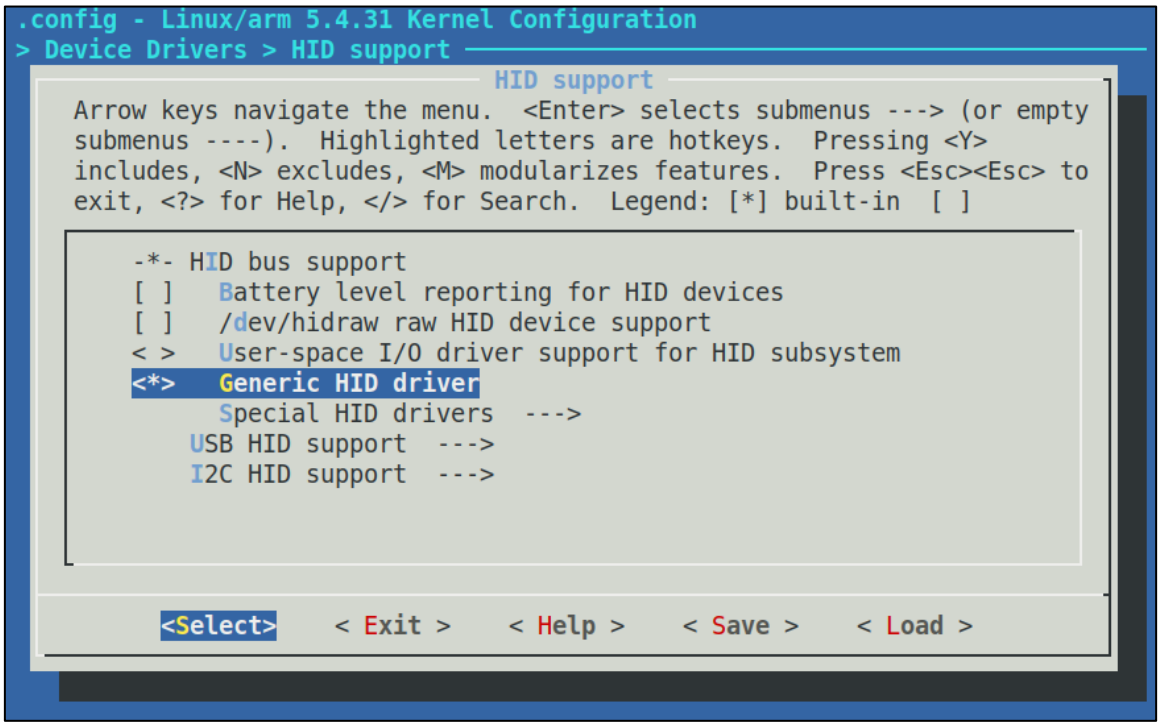


图 50.5.1.1 使能 HID 驱动

接下来需要使能 USB 键盘和鼠标驱动，配置路径如下：

```
-> Device Drivers
    -> HID support
        -> USB HID support
            ->  <*> USB HID transport layer     //USB 键盘鼠标等 HID 设备驱动
```

使能以后如图 50.5.1.2 所示：

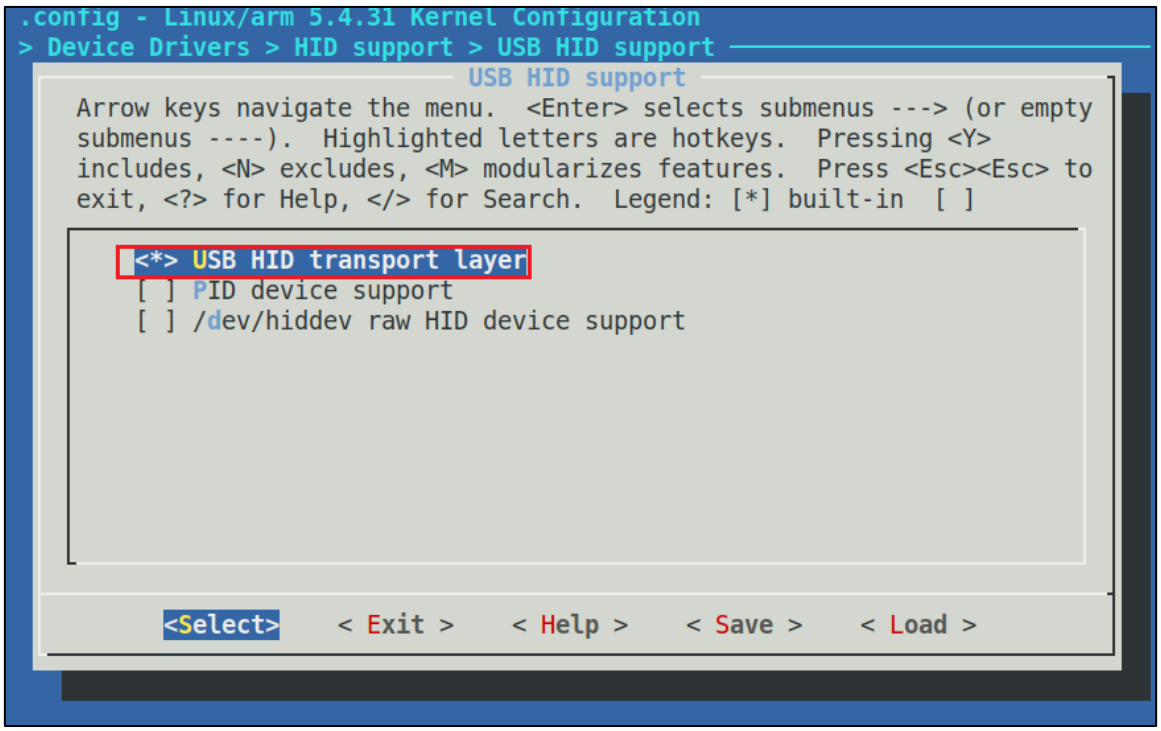


图 50.5.1.2 使能 USB 鼠标键盘驱动

大家可以将光标放到图 50.5.1.2 中“USB HID Transport layer”这一行，然后按下“?”键打开对应的帮助信息就可以看到对于这个配置项的描述，简单总结一下：

此选项对应配置项就是 CONFIG_USB_HID，也就是 USB 接口的 HID 设备。如果要使用 USB 接口的 keyboards(键盘)、mice(鼠标)、joysticks(摇杆)、graphic tablets(绘图板)等其他的 HID 设备，那么就需要选中“USB HID Transport layer”。但是要注意一点，此驱动和 HIDBP(Boot Protocol)键盘、鼠标的驱动不能一起使用！所以大家要是在网上查阅 linux 内核 USB 键盘鼠标驱动的时候，发现推荐使用“USB HIDBP Keyboard (simple Boot) support”和“USB HIDBP Mouse (simple Boot) support”这两个配置项的时候也不要觉得教程这里写错了。

2、测试 USB 鼠标和键盘

完成以后重新编译 linux 内核并且使用得到的 zImage 启动开发板。启动以后插入 USB 鼠标，会有如图 50.5.1.3 所示的提示信息：

```
[root@ATK-stm32mp1]:~$ usb 1-1.5: new low-speed USB device number 5 using ehci-platform
input: USB OPTICAL MOUSE  as /devices/platform/soc/5800d000.usbh-ehci/usb1/1-1/1-1.5/1-1.5:1.0/0003:275D
:0BA6.0007/input/input14
hid-generic 0003:275D:0BA6.0007: input: USB HID v1.11 Mouse [USB OPTICAL MOUSE ] on usb-5800d000.usbh-eh
ci-1.5/input0

[root@ATK-stm32mp1]:~$
```

图 50.5.1.3 USB 鼠标 log 信息

从图 50.5.1.3 可以看出，系统检测到了鼠标，如果成功驱动的话就会在/dev/input 目录下生成一个名为 eventX(X=0,1,2,3…)的文件，这个就是我们前面讲的输入子系统，鼠标和键盘都是作为输入子系统设备的。笔者这里对应的就是/dev/input/event1 这个设备，使用如下命令查看鼠标的原始输入值，结果如图 50.5.1.4 所示：

```
[root@ATK-stm32mp1]:~$ hexdump /dev/input/event1
0000000 047b 0000 f314 000c 0004 0004 0001 0009
0000010 047b 0000 f314 000c 0001 0110 0001 0000
0000020 047b 0000 f314 000c 0000 0000 0000 0000
0000030 047c 0000 6038 0000 0004 0004 0001 0009
0000040 047c 0000 6038 0000 0001 0110 0000 0000
0000050 047c 0000 6038 0000 0000 0000 0000 0000
```

图 50.5.1.4 鼠标原始输入值

图 50.5.1.4 就是鼠标作为输入子系统设备的原始输入值，这里就不去分析了，我们在移植 GUI 图形库以后就可以直接使用鼠标，比如 QT 等。

最后再来测试一下 USB 键盘，屏幕已经驱动起来了，所以我们可以直接将屏幕作为终端，然后接上键盘直接输入命令来进行各种操作。首先将屏幕设置为控制台，打开开发板根文件系统中的/etc/inittab 文件，然后在里面加入下面这一行：

```
tty1::askfirst:-/bin/sh
```

完成以后重启开发板，此时屏幕就会作为终端控制台，会有“Please press Enter to activate this console.”这样提示，如图 50.5.1.5 所示：

图 50.5.1.5 LCD 屏幕作为终端

接上键盘，然后根据图 50.5.1.5 中的提示，按下键盘上的 Enter(回车)键即可使能 LCD 屏幕控制台，然后我们就可以输入各种命令来执行相应的操作，如图 50.5.1.6 所示：

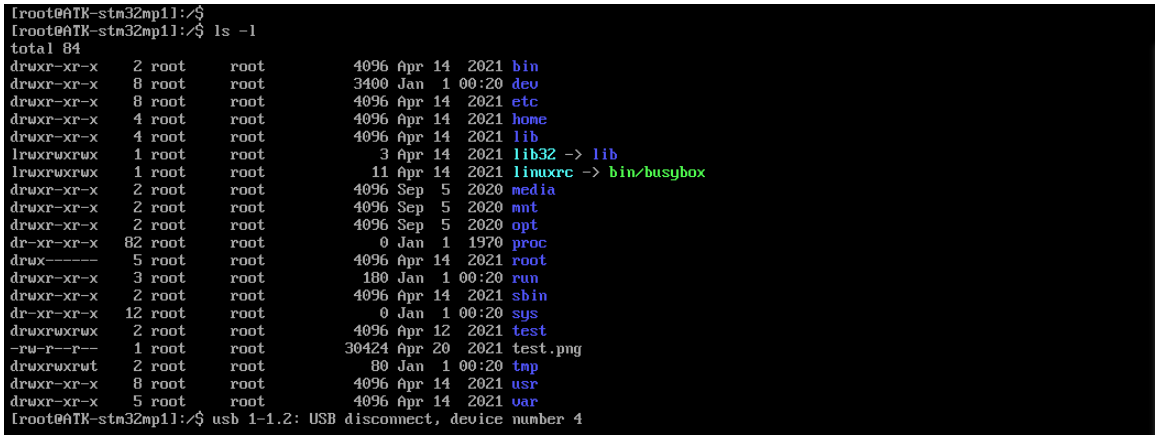


图 50.5.1.6 键盘操作终端

50.5.2 U 盘实验

注意，ST 官方的 Linux 内核默认已经使能了 U 盘！

ST 提供的 Linux 内核默认也已经使能了 U 盘驱动，因此我们可以直接插上去使用。但是我们还是需要学习一下如何手动配置 Linux 内核，使能 U 盘驱动。

1、使能 U 盘驱动

U 盘使用 SCSI 协议，因此要先使能 Linux 内核中的 SCSI 协议，配置路径如下：

```
-> Device Drivers
    -> SCSI device support
        -> <*> SCSI disk support      //选中此选项
```

结果如图 50.5.2.1 所示：

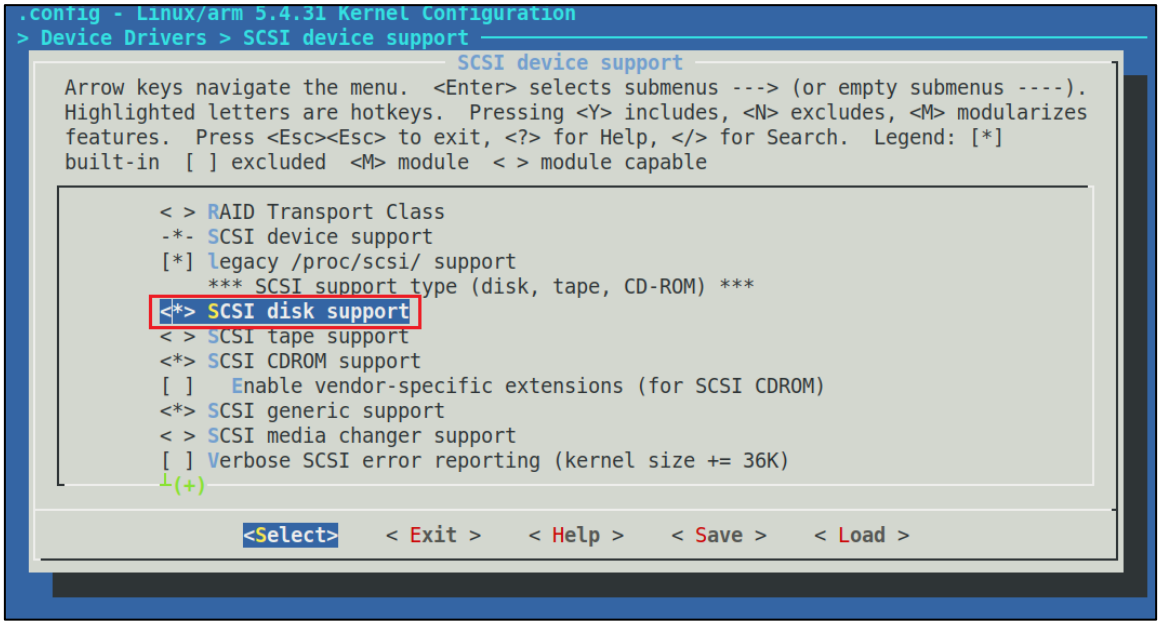


图 50.5.2.1 使能 SCSI

我们还需要使能 USB Mass Storage，也就是 USB 接口的大容量存储设备，配置路径如下：

-> Device Drivers
-> USB support
-> USB Gadget Support
-> USB Gadget functions configurable through configfs
-> [*] Mass storage //选中

结果如图 50.5.2.2 所示：

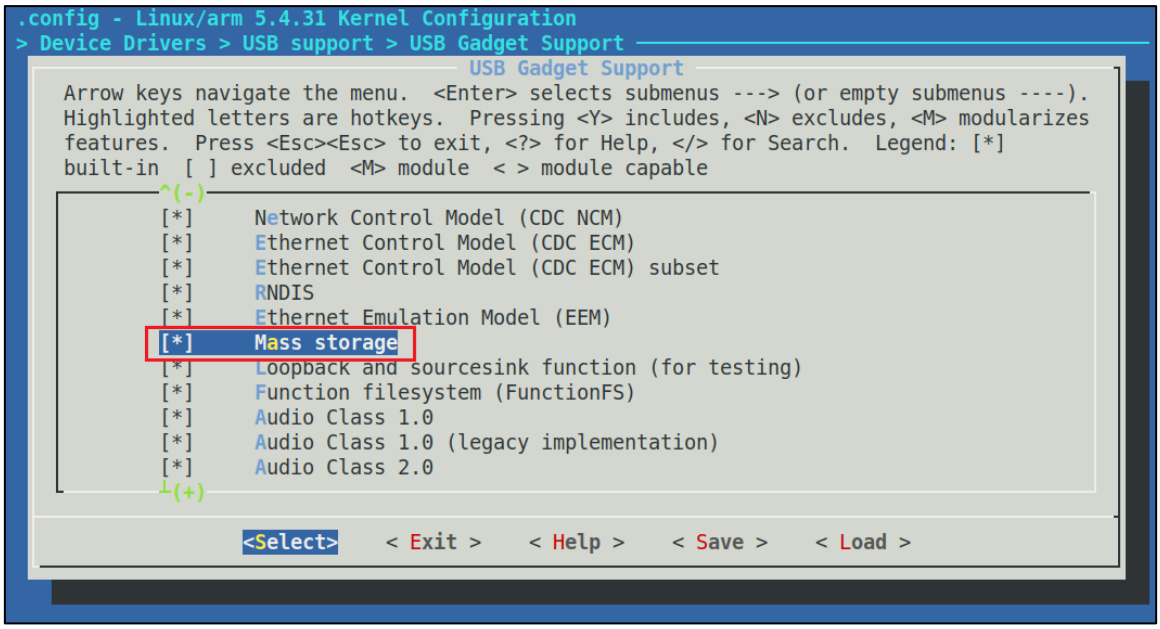


图 50.5.2.2 使能 USB 大容量存储设备

2、U 盘测试

准备好一个 U 盘，注意 U 盘要为 FAT32 格式的!NTFS 和 exFAT 由于版权问题所以在 Linux 下支持的不完善，操作的话可能会有问题，比如只能读，不能写或者无法识别等。准备好以后

将 U 盘插入到开发板 USB HUB 扩展出来的 HOST 接口上，此时会输出如图 50.5.2.3 所示信息：

```
[root@ATK-stm32mp1]:/$ usb 1-1.5: new high-speed USB device number 5 using ehci-platform
usb-storage 1-1.5:1.0: USB Mass Storage device detected
scsi host0: usb-storage 1-1.5:1.0
scsi 0:0:0:0: Direct-Access              USB DISK 2.0     PMAP PQ: 0 ANSI: 6
sd 0:0:0:0: Attached scsi generic sg0 type 0
sd 0:0:0:0: [sda] 31129600 512-byte logical blocks: (15.9 GB/14.8 GiB)
sd 0:0:0:0: [sda] Write Protect is off
sd 0:0:0:0: [sda] No Caching mode page found
sd 0:0:0:0: [sda] Assuming drive cache: write through
 sda: sda1
sd 0:0:0:0: [sda] Attached SCSI removable disk

[root@ATK-stm32mp1]:/$
```

图 50.5.2.3 U 盘 log 信息

从图 50.5.2.3 可以看出，系统检测到 U 盘插入，大小为 16GB 对应的设备文件为/dev/sda 和/dev/sda1，大家可以查看一下/dev 目录下有没有 sda 和 sda1 这两个文件。/dev/sda 是整个 U 盘，/dev/sda1 是 U 盘的第一个分区，我们一般使用 U 盘的时候都是只有一个分区。要想访问 U 盘我们需要先对 U 盘进行挂载，理论上挂载到任意一个目录下都可以，这里我创建一个/mnt/usb_disk 目录，然后将 U 盘挂载到/mnt/usb_disk 目录下，命令如下：

```
mkdir /mnt/usb_disk -p                                  //创建目录
mount /dev/sda1 /mnt/usb_disk/ -t vfat -o iocharset=utf8    //挂载
```

-t 指定挂载所使用的文件系统类型，这里设置为 vfat，也就是 FAT 文件系统，“-o iocharset”设置硬盘编码格式为 utf8，否则的话 U 盘里面的中文会显示乱码！

挂载成功以后进入到/mnt/usb_disk 目录下，输入 ls 命令查看 U 盘文件，如图 50.5.2.4 所示：

```
[root@ATK-stm32mp1]:/mnt/usb_disk$ ls
SIM900A.jpg                 test.txt
SIM900A2.jpg                实例图片.jpg
System Volume Information   测试用文件
readme.txt
[root@ATK-stm32mp1]:/mnt/usb_disk$
```

图 50.5.2.4 U 盘文件

至此 U 盘就能正常读写操作了，直接对/mnt/usb_disk 目录进行操作就行了。如果要拔出 U 盘要执行一个 sync 命令进行同步，然后在使用 unmount 进行 U 盘卸载，命令如下所示：

```
sync                    //同步
cd /                    //如果处于/mnt/usb_disk 目录的话先退出来，否则卸载的时候提示设
                        //备忙，导致卸载失败，切记！
umount /mnt/usb_disk    //卸载
```

50.6 Linux 内核自带 USB OTG 实验

注意：因为正点原子 STM32MP1 开发板底板上的 USB OTG 有两个版本，所以两种修改方法都提供，V1.4 及以前的请看 50.6.1 小节，V1.5 及其以后的两个小节都要看。

50.6.1 STUSB1600 设备树编写

STUSB1600 是 ST 出的一款 TypeC 芯片，也是 ST 官方开发板搭配 STM32MP1 的。STUSB1600 的驱动设备树编写参考文档“Documentation/devicetree/bindings/usb/st,typec-stusb.txt”。此文档描述了 STUSB1600 设备相关信息。这里就不做介绍了，大家自行查阅。

1、USB OTG 控制器节点信息

进入到 Linux 内核源码目录，打开 arch/arm/boot/dts/stm32mp151.dtsi 设备树文件，在这个设备树文件有一个 usbotg_hs 节点，此节点就是 USB OTG 控制器节点，内容如下：

示例代码 50.6.1.1 usbotg_hs 控制器

```
1  usbotg_hs: usb-otg@49000000 {
2      compatible = "st,stm32mp1-hsotg", "snps,dwc2";
3      reg = <0x49000000 0x10000>;
4      clocks = <&rcc USBO_K>;
5      clock-names = "otg";
6      resets = <&rcc USBO_R>;
7      reset-names = "dwc2";
8      interrupts-extended = <&exti 44 IRQ_TYPE_LEVEL_HIGH>;
9      g-rx-fifo-size = <512>;
10     g-np-tx-fifo-size = <32>;
11     g-tx-fifo-size = <256 16 16 16 16 16 16 16>;
12     dr_mode = "otg";
13     usb33d-supply = <&usb33>;
14     power-domains = <&pd_core>;
15     wakeup-source;
16     status = "disabled";
17 };
```

示例代码 50.6.1.1 中的 usbotg_hs 节点不需要我们修改，这里只是告诉大家 usbotg_hs 完整节点信息。根据第 2 行的 compatible 属性就是可以找到 STM32MP1 的 usbotg_hs 驱动源文件，驱动文件名为 drivers/usb/dwc2/params.c。第 12 行，dr_mode 属性用来配置控制器做 otg 功能还是 host 功能，不配置此属性默认为 otg 功能。第 16 行的 status 属性为 disabled，所以 usbotg_hs 默认关闭的，如果要使能就是要修改 status 属性为 okay、配置一个 PHY 端口和设置使用那个控制器去控制 usbotg_hs。

2、使能 usbotg_hs 节点

中添加使能 usbotg_hs 的相关操作。

首先我们先去配置 PHY 接口，在 stm32mp157d-atk.dts 文件中追加 usbphyc_port1 节点，内容如下所示：

示例代码 50.6.1.2 usbphyc_port1 节点配置

```
1  &usbphyc_port1 {
2      phy-supply = <&vdd_usb>;
3      st,phy-tuning = <&usb_phy_tuning>;
4  };
```

注意：要先使能 usbphyc 节点，这里在 USB HOST 实验已经使能就不用配置。

接着我们去追加 usbotg_hs 的相关属性信息。追加的内容如下所示：

示例代码 50.6.1.3 usbotg_hs 节点内容

```
1  &usbotg_hs {
2      phys = <&usbphyc_port1 0>;
3      phy-names = "usb2-phy";
4      usb-role-switch;
5      status = "okay";
```

```
6
7       port {
8           usbotg_hs_ep: endpoint {
9               remote-endpoint = <&con_usbotg_hs_ep>;
10          };
11      };
12  };
```

第 2 行，配置 usbotg_hs 的 PHY 接口，这里 0 表示为 OTG USB 的 PHY 端口。

第 5 行，把 status 属性改为 okay，使能 usbotg_hs。

第 7-11 行，添加了一个 port 节点，第 9 行指定 usbotg_hs 节点使用 con_usbotg_hs_ep 做控制器，con_usbotg_hs_ep 会在 STUSB1600 节点里创建。

最后还需要添加 vdd_usb 电源节点，因为示例代码 50.6.1.2 中的 usbphyc_port1 节点需要用到 vdd_usb 节点，在 stm32mp157d-atk.dts 中添加 vdd_usb 电源节点，在根节点下添加内容如下：

示例代码 50.6.1.4 vdd_usb 电源节点

```
1 vdd_usb: regulator-vdd-usb {
2     compatible = "regulator-fixed";
3     regulator-name = "vdd_usb";
4     regulator-min-microvolt = <3300000>;
5     regulator-max-microvolt = <3300000>;
6     regulator-always-on;
7     regulator-boot-on;
8 };
```

3、使能 I2C1 节点

控制 Typec 的芯片是 STUSB1600，此芯片是使用 I2C 协议和 CPU 进行通讯，所以要使能 I2C1。把示例代码 50.6.1.4 拷贝到我们的 stm32mp157d-atk.dts 文件里，代码如下所示：

示例代码 50.6.1.5 使能 i2c1 节点

```
1   &i2c1 {
2       pinctrl-names = "default", "sleep";
3       pinctrl-0 = <&i2c1_pins_b>;
4       pinctrl-1 = <&i2c1_pins_sleep_b>;
5       status = "okay";
6   };
```

上面的代码没啥可以说的，正点原子 STM32MP157 开发板 I2C1_SCL 和 I2C1_SDA 这两个引脚为 PF14 和 PF15。这两个引脚的 pinctrl 配置已经在 stm32mp15-pinctrl.dtsi 文件里面提供了，如果大家使用的其他引脚需要自行配置。

4、配置 STUSB1600 节点

先配置 STUSB1600 的中断引脚电气属性和电源管理，STUSB1600 中断引脚为 PG2，在 stm32mp15-pinctrl.dtsi 文件中输入如下内容：

示例代码 50.6.1.6 stusb1600 中断电气属性

```
1    stusb1600_pins_b: stusb1600-0 {
2        pins {
3            pinmux = <STM32_PINMUX('G', 2, ANALOG)>;
```

```
4           bias-pull-up;
5       };
6   };
```

把 STUSB1600 的中断引脚改为内部上拉。因为 USB OTG 需要 5V 电源管理，前面没有此配置，所以要一个 5V 的电源配置，在根节点下添加如下示例代码所示：

示例代码 50.6.1.7 5V 的 vin 节点

```
1   vin: regulator-vin {
2       compatible = "regulator-fixed";
3       regulator-name = "vin";
4       regulator-min-microvolt = <5000000>;
5       regulator-max-microvolt = <5000000>;
6       regulator-always-on;
7       regulator-boot-on;
8   };
```

接下来配置 STUSB1600 节点，此节点属于 I2C1 的子节点。STUSB1600 的配置内容如下所示：

示例代码 50.6.1.8 stusb1600 节点信息

```
1    stusb1600@28 {
2        compatible = "st,stusb1600";
3        reg = <0x28>;
4        interrupts = <2 IRQ_TYPE_EDGE_FALLING>;
5        interrupt-parent = <&gpiog>;
6        pinctrl-names = "default";
7        pinctrl-0 = <&stusb1600_pins_b>;
8        status = "okay";
9        vdd-supply = <&vin>;
10
11       connector {
12           compatible = "usb-c-connector";
13           label = "USB-C";
14           power-role = "dual";
15           power-opmode = "default";
16
17           port {
18               con_usbotg_hs_ep: endpoint {
19                   remote-endpoint = <&usbotg_hs_ep>;
20               };
21           };
22       };
23   };
```

示例代码 50.6.1.7 代码中可以分为两个部分：第 1-9 行属于控制 STUSB1600 芯片相关属性，第 11-22 行属于 Typec 端口相关属性。

第 2 行，compatible 的属性值为“st,stusb1600”，在 Linux 源码目录下，搜索此属性值，会

找到 stusb1600 的驱动源码为 drivers/usb/typec/typec_stusb.c。

第 3 行，reg 属性值为“0x28”，stusb1600 芯片通讯地址为 0x28。

第 4-5 行，设置中断相关配置，中断触发为下降沿触发。

第 6-7 行，设置中断的电气属性。

第 9 行，设置电源配置。

第 12-15 行，就不给大家讲解了，可以查看 Documentation/devicetree/bindings/connector/usb-connector.txt 文件。

第 17-20 行，定义了一个 con_usbotg_hs_ep 端口，同时指定此端口连接到 usbotg_hs 节点的 usbotg_hs_ep 端口。

我们重新编译设备树，使用新的 stm32mp157d-atk.dtb 文件去启动开发板。

50.6.2 FUSB302 驱动移植

在 50.6.1 里已经完成了一部分的设备树配置，由于只是改了 USB TypeC 的芯片，所以我们只需要把 STUSB1600 相关的部分改为 FUSB302 即可。其实就是 50.6.1 前面的 1-3 小点不用修改，只需要改第 4 小点。

首先配置 FUSB302 的中断电气属性，配置如下所示：

示例代码 50.6.2.1 fusb302 中断电气属性

```
1  fusb302_pins_a: fusb302-0 {
2      pins {
3          pinmux = <STM32_PINMUX('G', 2, ANALOG)>;
4          bias-pull-up;
5      };
6  };
```

在 stm32mp167d-atk.dts 文件中添加头文件“dt-bindings/usb/pd.h”，如图 50.6.2.1 所示：

```
15  #include "dt-bindings/input/input.h"
16  #include <dt-bindings/usb/pd.h>
17
```

图 50.6.2.1 添加“dt-bindings/usb/pd.h”头文件

接着配置 FUSB302 节点，配置的代码如下所示：

示例代码 50.6.2.2 fusb302 节点信息

```
1  fusb302@22 {
2      compatible = "fcs,fusb302","fairchild,fusb302";
3      reg = <0x22>;
4      pinctrl-names = "default";
5      pinctrl-0 = <&fusb302_pins_a>;
6      int-n-gpios = <&gpiog 2 GPIO_ACTIVE_HIGH>;
7      vbus-5v-gpios = <&gpioz 6 GPIO_ACTIVE_HIGH>;
8      status = "okay";
9
10     connector {
11         compatible = "usb-c-connector";
12         label = "USB-C";
13         power-role = "dual";
```

```
            power-opmode = "default";

            try-power-role = "sink";
            source-pdos = <PDO_FIXED(5000, 3000, PDO_FIXED_USB_COMM)>;
            sink-pdos = <PDO_FIXED(5000, 3000, PDO_FIXED_USB_COMM)
                         PDO_VAR(3000, 12000, 3000)
                         PDO_PPS_APDO(3000, 11000, 3000)>;
             op-sink-microwatt = <10000000>;
             port {
                  con_usbotg_hs_ep: endpoint {
                      remote-endpoint = <&usbotg_hs_ep>;
                  };
            };
       };
   };
```

fusb302 的设备树没有使用电源管理，而是用 PZ6 引脚去控制电压。其它的属性和 stusb1600 一样。

最后就是去改驱动了，Linux 内核提供的 fusb302 驱动是没读取 connector 相关的属性，所以此驱动是不能使用。辛亏在 github 里找到了可用的 fusb302 驱动，已经放到了开发板光盘中，路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→25_fusb302。改动的部分是如何读取 connector 节点和使用 PZ6 引脚控制电压。大家想看如何修改的，可以用文件对比软件，就能知道修改了那些内容。将 25_fusb302 目录的 fusb302.c 和 fusb302.h 文件拷贝到 linux 源码下的 drivers/usb/typec/tcpm 目录里，注意要把原来的 fusb302.c 覆盖掉，拷贝结果如下 50.6.2.2 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31/drivers/usb/typec/tcpm$ ls
fusb302.c  fusb302_reg.h  Makefile  tcpci.h          tcpm.c
fusb302.h  Kconfig        tcpci.c   tcpci_rt1711h.c  wcove.c
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31/drivers/usb/typec/tcpm$
```

图 50.6.2.2 拷贝 fusb302 驱动的 tcpm 目录

拷贝完成后，进入内核的配置选项图形界面，配置路径如下：

```
-> Device Drivers
    -> USB support
        -> USB Type-C Support
            -> USB Type-C Port Controller Manager
                -> <*> Fairchild FUSB302 Type-C chip driver    //选中
```

如图 50.6.2.2 所示：

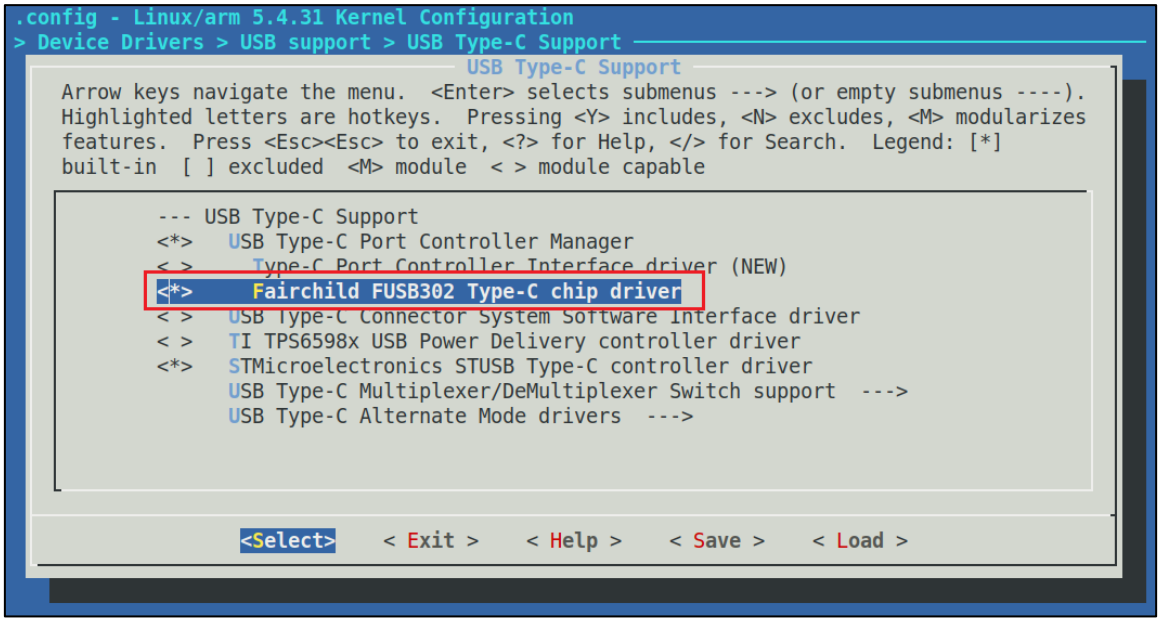


图 50.6.2.2 FUSB302 的驱动使能

重新编译设备树和内核，使用新的设备树 stm32mp157d-atk.dtb 和内核 uImage 去启动开发板，有图 50.6.2.3 打印信息，说明加载驱动成功。如图所示：

```
fusb302 0-0022: Can't get property of role, set role to default DRP
fusb302 0-0022: port 0 probe success with role ROLE_MODE_DRP, try_role ROLE_MODE_NONE
```

图 50.6.2.3 加载 FUSB302 驱动打印信息

50.6.3 OTG 主机实验

系统重启成功以后就可以正常使用 USB_OTG 接口，OTG 既可以做主机，也可以做从机，做主机的话测试方法和 50.5 小节一模一样，直接在正点原子的 USB_OTG 接口上使用 Typec OTG 线接入鼠标键盘、U 盘等设备。USB_OTG 接口如下所示：

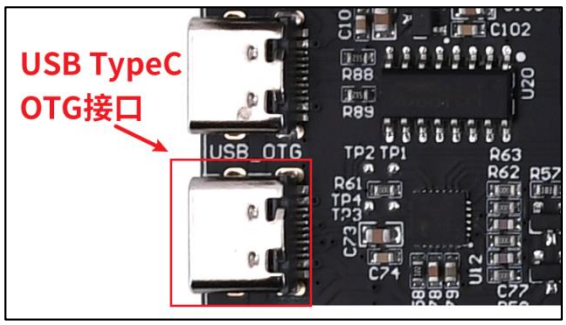


图 50.6.3.1 USB_OTG 接口

50.6.4 OTG 从机实验

OTG 从机就是将开发板作为一个 USB 设备连接到其他的主机上，这里我们来做两个 USB 从机实验：模拟 U 盘以及 USB 声卡。

1、模拟 U 盘实验

模拟 U 盘实验就是将开发板当做一个 U 盘，可以将开发板上的 U 盘或者 TF 卡挂载到 PC 上去，首先需要配置 Linux，配置路径如下：

-> Device Drivers
-> USB support
-> USB Gadget Support
-> < > USB Gadget functions configurable through configfs //不要编译进内核
-> USB Gadget precomposed configurations (<choice> [=m])
-> <M> Mass Storage Gadget //大容量存储

如图 50.6.4.1 所示：

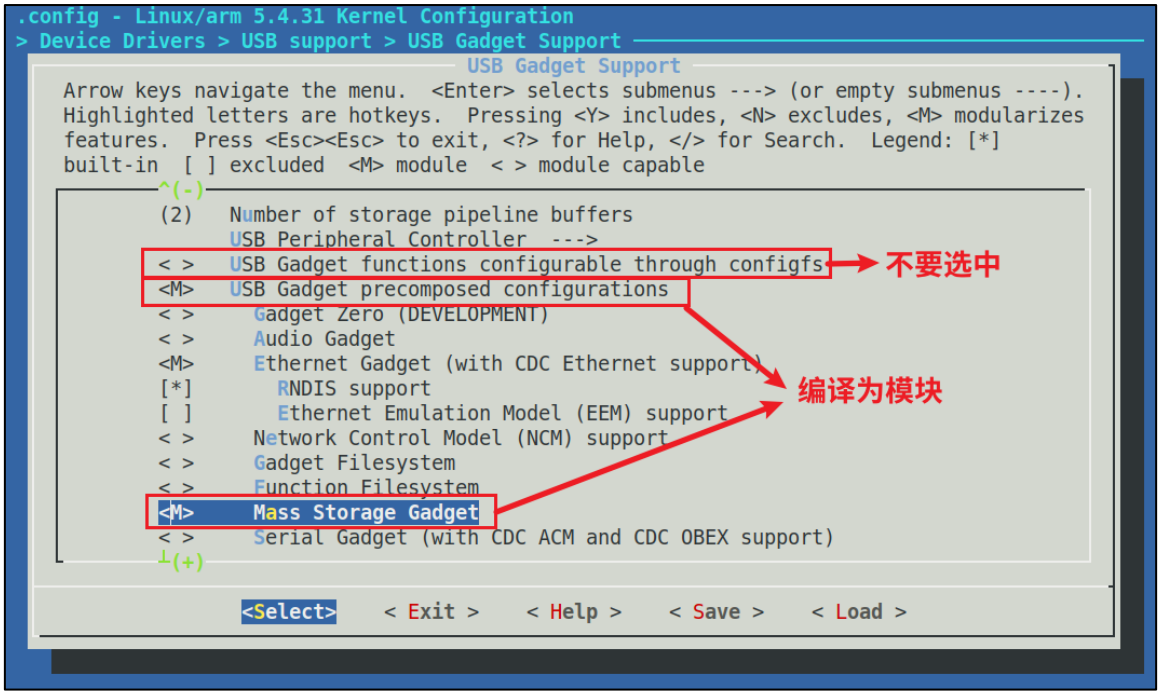


图 50.6.4.1 USB Gadget 中大容量存储设备驱动

这里我们需要将驱动编译为模块！使用的时候直接输入命令加载驱动模块即可。配置好以后执行下面这些命令重新编译 Linux 内核、编译模块：

```
make uImage LOADADDR=0XC2000040 -j32
make modules -j32
```

编译完成之后会得到 3 个.ko 内核模块文件，它们对应的路径为：

```
drivers/usb/gadget/libcomposite.ko
drivers/usb/gadget/function/usb_f_mass_storage.ko
drivers/usb/gadget/legacy/g_mass_storage.ko
```

将上述三个.ko 模块拷贝到开发板根文件系统/lib/modules/5.4.31 目录下，如图所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls
g_mass_storage.ko      modules.alias          modules.symbols
libcomposite.ko        modules.dep            usb_f_mass_storage.ko
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 50.6.4.2 将内核模块拷贝到文件系统

拷贝完成以后使用新编译出来的 uImage 启动开发板，在开发板上插入一个 U 盘，记住这个 U 盘对应的设备文件，比如我们这里是/dev/sda 和/dev/sda1，以后要将/dev/sda1 挂载到 PC 上，也就是把/dev/sda1 作为模拟 U 盘的存储区域。

使用 TypeC 数据线将开发板和电脑连接，连接好以后依次加载 libcomposite.ko、usb_f_mass_storage.ko 和 g_mass_storage.ko 这三个驱动文件，顺序不能错了！命令如下：

```
depmod
modprobe libcomposite.ko
modprobe usb_f_mass_storage.ko
modprobe g_mass_storage.ko file=/dev/sda1 removable=1
```

加载 g_mass_storage.ko 的时候使用 file 参数指定使用的大容量存储设备，我这里使用 U 盘对应的/dev/sda1。如果加载成功的话电脑就会出现一个 U 盘，这个 U 盘就是我们开发板模拟的，如图 50.6.4.3 所示：

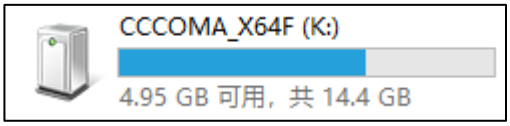


图 50.6.4.3 电脑识别出来的 U 盘

我们可以直接在电脑上对这个 U 盘进行读写操作，实际上操作的就是插在开发板上的 U 盘。操作完成以后要退出的话执行如下命令：

```
rmmod g_mass_storage.ko
rmmod usb_f_mass_storage.ko
rmmod libcomposite.ko
```

注意！不要将开发板上的 EMMC 或者 NAND 作为模拟 U 盘的存储区域，因为 linux 下 EMMC 和 NAND 使用的文件系统一般都是 EXT3/EXT4 和 UBIFS，这些文件系统类型和 windows 下的不兼容，如果挂载的话就会在 windows 下提示要你格式化 U 盘！

2、USB 声卡实验

USB 声卡就是 USB 接口的外置声卡，一般电脑内部都自带了声卡，但是内部自带的声卡效果相对来说比较差，不能满足很多 HIFI 玩家的需求。USB 声卡通过 USB 接口来传递音频数据，具体的 ADC 和 DAC 过程由声卡完成，摆脱了电脑主板体积的限制，外置 USB 声卡就可以做的很好。STM32MP1 开发板板载了音频解码芯片，因此可以将 STM32MP1 开发板作为一个外置 USB 声卡，配置 Linux 内核，配置路径如下：

```
-> Device Drivers
    -> USB suppor
        -> USB Gadget Support
            -> USB Gadget precomposed configurations
                -> Audio Gadget              //选中音频，编译为模块
                    -> UAC 1.0               //选中
                        -> [*] UAC 1.0 (Legacy)    //选中 UAC
```

配置如图 50.6.4.4 所示

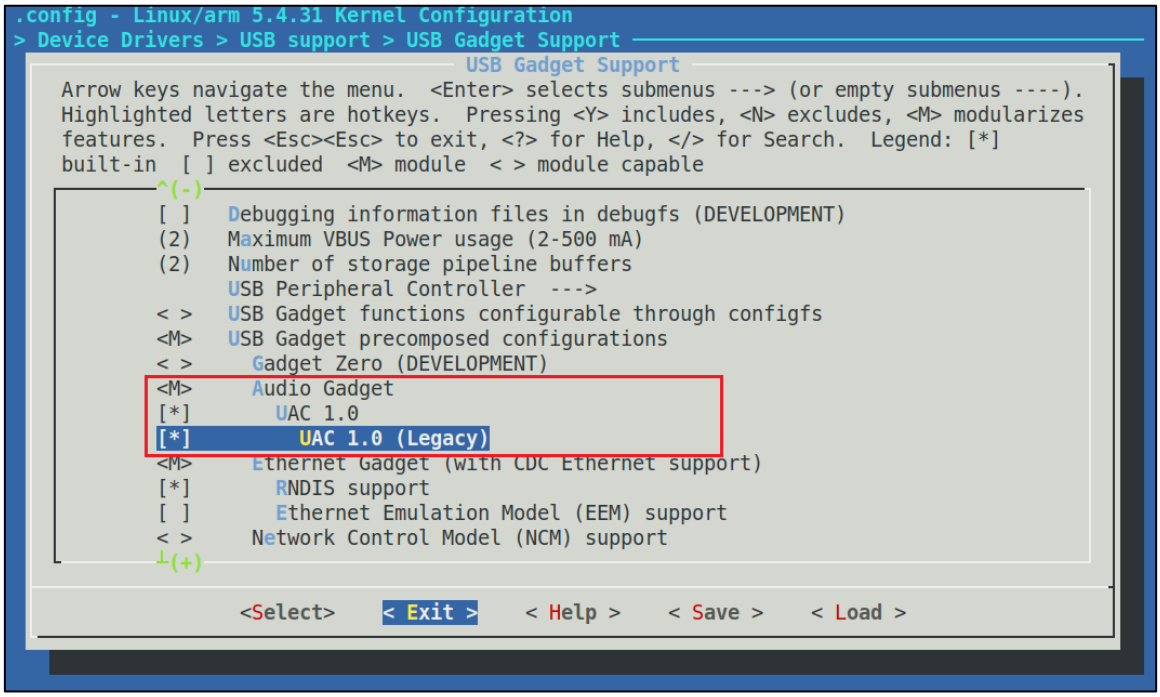


图 50.6.4.4 使能 USB Gadget 下的音频驱动

注意，这里也是编译为驱动模块，配置完成以后重新编译内核、编译模块，我们会得到 3 个驱动模块文件，模块文件路径如下：

```
drivers/usb/gadget/libcomposite.ko
drivers/usb/gadget/function/usb_f_uac1_legacy.ko
drivers/usb/gadget/legacy/g_audio.ko
```

将上述三个.ko 模块文件拷贝到开发板根文件系统/lib/modules/5.4.31 目录下，如下所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ls
g_audio.ko              modules.alias          usb_f_mass_storage.ko
g_mass_storage.ko       modules.dep            usb_f_uac1_legacy.ko
libcomposite.ko         modules.symbols
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 50.6.4.5 拷贝音频相关模块到内核

贝完成以后使用新编译出来的 zImage 启动开发板，首先按照 48.5.2 小节讲解的方法配置 STM32MP1 的声卡，保证声卡播放正常！使用 Typec 数据线将开发板与电脑连接起来，最后依次加载 libcomposite.ko、usb_f_uac1_legacy.ko 和 g_audio.ko 这三个驱动模块，命令如下：

```
depmod
modprobe libcomposite.ko
modprobe usb_f_uac1_legacy.ko
modprobe g_audio.ko
```

加载完成以后稍等一会虚拟出一个 USB 声卡，打开电脑的设备管理器，选择“声音、视频和游戏控制器”，会发现有一个名为“AC Interface”设备，如图 50.6.4.6 所示：

图 50.6.4.6 模拟出来的 USB 声卡

图 50.6.4.6 中的“AC Interface”就是开发板模拟出来的 USB 声卡，设置 windows，选择音频输出使用“AC Interface”，Windows10 设置如图 50.6.4.7 所示：

图 50.6.4.7 声卡输出设置为 AC Interface

一切设置好以后就可以从开发板上听到电脑输出的声音，此时开发板就完全是一个 USB 声卡设备了。

关于 USB 驱动就讲解到这里，本章并没有深入到 USB 驱动具体编写方式，只是对 USB 的协议做了简单的介绍，后面讲解了一下 Linux 内核自带的 USB HOST 和 DEVICE 驱动的使用，Linux 内核已经集成了大量的 USB 设备驱动，至于其他特殊的就需要具体情况具体分析了，比如本教程后面讲解的 4G 模块驱动。

第五十一章 Linux 块设备驱动实验

前面我们都是在学习字符设备驱动，本章我们来学习一下块设备驱动框架，块设备驱动是 Linux 三大驱动类型之一。块设备驱动要远比字符设备驱动复杂得多，不同类型的存储设备又对应不同的驱动子系统，本章我们重点学习一下块设备相关驱动概念，不涉及到具体的存储设备。最后，我们使用 STM32MP1 开发板板载 RAM 模拟一个块设备，学习块设备驱动框架的使用。

51.1 什么是块设备

块设备是针对存储设备的，比如 SD 卡、EMMC、NAND Flash、Nor Flash、SPI Flash、机械硬盘、固态硬盘等。因此块设备驱动其实就是这些存储设备驱动，块设备驱动相比字符设备驱动的主要区别如下：

①、块设备只能以块为单位进行读写访问，块是 linux 虚拟文件系统(VFS)基本的数据传输单位。字符设备是以字节为单位进行数据传输的，不需要缓冲。

②、块设备在结构上是可以进行随机访问的，对于这些设备的读写都是按块进行的，块设备使用缓冲区来暂时存放数据，等到条件成熟以后再一次性将缓冲区中的数据写入块设备中。这么做的目的为了提高块设备寿命，大家如果仔细观察的话就会发现有些硬盘或者 NAND Flash 就会标明擦除次数(flash 的特性，写之前要先擦除)，比如擦除 100000 次等。因此，为了提高块设备寿命而引入了缓冲区，数据先写入到缓冲区中，等满足一定条件后再一次性写入到真正的物理存储设备中，这样就减少了对块设备的擦除次数，提高了块设备寿命。

字符设备是顺序的数据流设备，字符设备是按照字节进行读写访问的。字符设备不需要缓冲区，对于字符设备的访问都是实时的，而且也不需要按照固定的块大小进行访问。

块设备结构的不同其 I/O 算法也会不同，比如对于 EMMC、SD 卡、NAND Flash 这类没有任何机械设备的存储设备就可以任意读写任何的扇区(块设备物理存储单元)。但是对于机械硬盘这样带有磁头的设备，读取不同的盘面或者磁道里面的数据，磁头都需要进行移动，因此对于机械硬盘而言，将那些杂乱的访问按照一定的顺序进行排列可以有效提高磁盘性能，linux 里面针对不同的存储设备实现了不同的 I/O 调度算法。

Linux 内核使用 block_device 表示块设备，block_device 为一个结构体，定义在 include/linux/fs.h 文件中，结构体内容如下所示：

51.2 块设备驱动框架

51.2.1 block_device 结构体

Linux 内核使用 block_device 表示块设备，block_device 为一个结构体，定义在 include/linux/fs.h 文件中，结构体内容如下所示：

示例代码 51.2.1.1 block_device 结构体

```
1  struct block_device {
2      dev_t           bd_dev;  /* not a kdev_t - it's a search key */
3      int                     bd_openers;
4      struct inode *          bd_inode;   /* will die */
5      struct super_block *    bd_super;
6      struct mutex            bd_mutex;   /* open/close mutex */
7      void *                  bd_claiming;
8      void *                  bd_holder;
9      int                     bd_holders;
10     bool                    bd_write_holder;
11 #ifdef CONFIG_SYSFS
12     struct list_head        bd_holder_disks;
13 #endif
14     struct block_device *   bd_contains;
```

```
15      unsigned              bd_block_size;
16      u8                    bd_partno;
17      struct hd_struct *    bd_part;
18  /* number of times partitions within this device have been opened. */
19      unsigned              bd_part_count;
20      int                   bd_invalidated;
21      struct gendisk *      bd_disk;
22      struct request_queue *  bd_queue;
23      struct backing_dev_info *bd_bdi;
24      struct list_head      bd_list;
25      /*
26       * Private data.  You must have bd_claim'ed the block_device
27       * to use this.  NOTE:  bd_claim allows an owner to claim
28       * the same device multiple times, the owner must take special
29       * care to not mess up bd_private for that case.
30       */
31      unsigned long       bd_private;
32
33      /* The counter of freeze processes */
34      int         bd_fsfreeze_count;
35      /* Mutex for freeze */
36      struct mutex        bd_fsfreeze_mutex;
37  } __randomize_layout;
```

对于 block_device 结构体，我们重点关注一下第 21 行的 bd_disk 成员变量，此成员变量为 gendisk 结构体指针类型。内核使用 block_device 来表示一个具体的块设备对象，比如一个硬盘或者分区，如果是硬盘的话 bd_disk 就指向通用磁盘结构 gendisk。

1 、注册块设备

和字符设备驱动一样，我们需要向内核注册新的块设备、申请设备号，块设备注册函数为 register_blkdev，函数原型如下：

```
int register_blkdev(unsigned int major, const char *name)
```

函数参数和返回值含义如下：

major ： 主设备号。

name ： 块设备名字。

返回值：如果参数 major 在 1~(BLKDEV_MAJOR_MAX-1)之间的话表示自定义主设备号，那么返回 0 表示注册成功，如果返回负值的话表示注册失败。如果 major 为 0 的话表示由系统自动分配主设备号，那么返回值就是系统分配的主设备号，如果返回负值那就表示注册失败。

2 、注销块设备

和字符设备驱动一样，如果不使用某个块设备了，那么就需要注销掉，函数为 unregister_blkdev，函数原型如下：

```
void unregister_blkdev(unsigned int major, const char *name)
```

函数参数和返回值含义如下：

major ： 要注销的块设备主设备号。

name： ： 要注销的块设备名字。

返回值：无。

51.2.2 gendisk 结构体

Linux 内核使用 gendisk 来描述一个磁盘设备，这是一个结构体，定义在 include/linux/genhd.h 中，内容如下所示：

示例代码 51.2.2.1 gendisk 结构体

```
1  struct gendisk {
2      /* major, first_minor and minors are input parameters only,
3       * don't use directly.  Use disk_devt() and disk_max_parts().
4       */
5      int major;          /* major number of driver */
6      int first_minor;
7      int minors;         /* maximum number of minors, =1 for
8                           * disks that can't be partitioned. */
9
10     char disk_name[DISK_NAME_LEN];  /* name of major driver */
11     char *(*devnode)(struct gendisk *gd, umode_t *mode);
12
13     unsigned short events;   /* supported events */
14     unsigned short event_flags;
15
16     /* Array of pointers to partitions indexed by partno.
17      * Protected with matching bdev lock but stat and other
18      * non-critical accesses use RCU.  Always access through
19      * helpers.
20      */
21     struct disk_part_tbl __rcu *part_tbl;
22     struct hd_struct part0;
23
24     const struct block_device_operations *fops;
25     struct request_queue *queue;
26     void *private_data;
27
28     int flags;
29     struct rw_semaphore lookup_sem;
30     struct kobject *slave_dir;
31
32     struct timer_rand_state *random;
33     atomic_t sync_io;       /* RAID */
34     struct disk_events *ev;
35 #ifdef  CONFIG_BLK_DEV_INTEGRITY
36     struct kobject integrity_kobj;
```

```
37  #endif  /* CONFIG_BLK_DEV_INTEGRITY */
38      int node_id;
39      struct badblocks *bb;
40      struct lockdep_map lockdep_map;
41  };
```

我们简单看一下 gendisk 结构体中比较重要的几个成员变量：

第 5 行，major 为磁盘设备的主设备号。

第 6 行，first_minor 为磁盘的第一个次设备号。

第 7 行，minors 为磁盘的此设备号数量，也就是磁盘的分区数量，这些分区的主设备号一样，此设备号不同。

第 21 行，part_tbl 为磁盘对应的分区表，为结构体 disk_part_tbl 类型，disk_part_tbl 的核心是一个 hd_struct 结构体指针数组，此数组每一项都对应一个分区信息。

第 24 行，fops 为块设备操作集，为 block_device_operations 结构体类型。和字符设备操作集 file_operations 一样，是块设备驱动中的重点！

第 25 行，queue 为磁盘对应的请求队列，所以针对该磁盘设备的请求都放到此队列中，驱动程序需要处理此队列中的所有请求。

编写块的设备驱动的时候需要分配并初始化一个 gendisk，linux 内核提供了一组 gendisk 操作函数，我们来看一下一些常用的 API 函数。

1、申请 gendisk

使用 gendisk 之前要先申请，allo_disk 函数用于申请一个 gendisk，函数原型如下：

```
struct gendisk *alloc_disk(int minors)
```

函数参数和返回值含义如下：

minors：次设备号数量，也就是 gendisk 对应的分区数量。

返回值：成功：返回申请到的 gendisk，失败：NULL。

2、删除 gendisk

如果要删除 gendisk 的话可以使用函数 del_gendisk，函数原型如下：

```
void del_gendisk(struct gendisk *gp)
```

函数参数和返回值含义如下：

gp：删除的 gendisk。

返回值：无。

3、将 gendisk 添加到内核

使用 alloc_disk 申请到 gendisk 以后系统还不能使用，必须使用 add_disk 函数将申请到的 gendisk 添加到内核中，add_disk 函数原型如下：

```
void add_disk(struct gendisk *disk)
```

函数参数和返回值含义如下：

disk：添加到内核的 gendisk。

返回值：无。

4、设置 gendisk 容量

每一个磁盘都有容量，所以在初始化 gendisk 的时候也需要设置其容量，使用函数 set_capacity，函数原型如下：

```
void set_capacity(struct gendisk *disk, sector_t size)
```

函数参数和返回值含义如下：

disk： 设置容量的 gendisk。

size： 盘容量大小，注意这里是扇区数量。块设备中最小的可寻址单元是扇区，一个扇区一般是 512 字节，有些设备的物理扇区可能不是 512 字节。不管物理扇区是多少，内核和块设备驱动之间的扇区都是 512 字节。所以 set_capacity 函数设置的大小就是块设备实际容量除以 512 字节得到的扇区数量。比如一个 2MB 的磁盘，其扇区数量就是(2*1024*1024)/512=4096。

返回值： 无。

5、调整 gendisk 引用计数

内核会通过 get_disk_and_module 和 put_disk 这两个函数来调整 gendisk 的引用计数，根据名字就可以知道，get_disk_and_module 是增加 gendisk 的引用计数，put_disk 是减少 gendisk 的引用计数，这两个函数原型如下所示：

```
truct kobject * get_disk_and_module (struct gendisk *disk)
void put_disk(struct gendisk *disk)
```

51.2.3 block_device_operations 结构体

和字符设备的 file_operations 一样，块设备也有操作集，为结构体 block_device_operations，此结构体定义在 include/linux/blkdev.h 中，结构体内容如下：

示例代码 51.2.3.1 block_device_operations 结构体

```
1  struct block_device_operations {
2      int (*open) (struct block_device *, fmode_t);
3      void (*release) (struct gendisk *, fmode_t);
4      int (*rw_page)(struct block_device *, sector_t, struct page *,
                      unsigned int);
5      int (*ioctl) (struct block_device *, fmode_t, unsigned,
                     unsigned long);
6      int (*compat_ioctl) (struct block_device *, fmode_t, unsigned,
                            unsigned long);
7      unsigned int (*check_events) (struct gendisk *disk,
8                      unsigned int clearing);
9    /* ->media_changed() is DEPRECATED, use ->check_events() instead */
10     int (*media_changed) (struct gendisk *);
11     void (*unlock_native_capacity) (struct gendisk *);
12     int (*revalidate_disk) (struct gendisk *);
13     int (*getgeo)(struct block_device *, struct hd_geometry *);
14
15     void (*swap_slot_free_notify) (struct block_device *,
                                      unsigned long);
16     int (*report_zones)(struct gendisk *, sector_t sector,
17                 struct blk_zone *zones, unsigned int *nr_zones);
18     struct module *owner;
19     const struct pr_ops *pr_ops;
20 };
```

可以看出，block_device_operations 结构体里面的操作集函数和字符设备的 file_operations 操作集基本类似，但是块设备的操作集函数比较少，我们来看一下其中比较重要的几个成员函数：

第 2 行，open 函数用于打开指定的块设备。

第 3 行，release 函数用于关闭(释放)指定的块设备。

第 4 行，rw_page 函数用于读写指定的页。

第 5 行，ioctl 函数用于块设备的 I/O 控制。

第 6 行，compat_ioctl 函数和 ioctl 函数一样，都是用于块设备的 I/O 控制。区别在于在 64 位系统上，32 位应用程序的 ioctl 会调用 compat_iotl 函数。在 32 位系统上运行的 32 位应用程序调用的就是 ioctl 函数。

第 13 行，getgeo 函数用于获取磁盘信息，包括磁头、柱面和扇区等信息。

第 18 行，owner 表示此结构体属于哪个模块，一般直接设置为 THIS_MODULE。

51.2.4 块设备 I/O 请求过程

大家如果仔细观察的话会在 block_device_operations 结构体中并没有找到 read 和 write 这样的读写函数，那么块设备是怎么从物理块设备中读写数据？这里就引处理块设备驱动中非常重要的 request_queue、request 和 bio。

1、请求队列 request_queue

内核将对块设备的读写都发送到请求队列 request_queue 中，request_queue 中是大量的 request(请求结构体)，而 request 又包含了 bio，bio 保存了读写相关数据，比如从块设备的哪个地址开始读取、读取的数据长度，读取到哪里，如果是写的话还包括要写入的数据等。我们先来看一下 request_queue，这是一个结构体，定义在文件 include/linux/blkdev.h 中，由于 request_queue 结构体比较长，这里就不列出来了。大家回过头看一下示例代码 51.2.2.1 的 gendisk 结构体就会发现里面有一个 request_queue 结构体指针类型成员变量 queue，也就说在编写块设备驱动的时候，每个磁盘(gendisk)都要分配一个 request_queue。

①、初始化请求队列

初始化请求队列可以分为两部分，第一部分是创建 blk_mq_tag_set 结构体，然后使用 blk_mq_alloc_tag_set 函数初始化 blk_mq_tag_set 对象。第二部分使用 blk_mq_init_queue 函数获取 request_queue。

blk_mq_tag_set 结构体定义在 include/linux/blk-mq.h 中，如下示例代码所示：

示例代码 51.2.4.1 blk_mq_tag_set 结构体

```
1  struct blk_mq_tag_set {
2      /*
3       * map[] holds ctx -> hctx mappings, one map exists for each type
4       * that the driver wishes to support. There are no restrictions
5       * on maps being of the same size, and it's perfectly legal to
6       * share maps between types.
7       */
8      struct blk_mq_queue_map map[HCTX_MAX_TYPES];
9      unsigned int   nr_maps;    /* nr entries in map[] */
10     const struct   blk_mq_ops *ops;
11     unsigned int   nr_hw_queues;   /* nr hw queues across maps */
```

```
12      unsigned int  queue_depth;    /* max hw supported */
13      unsigned int  reserved_tags;
14      unsigned int  cmd_size;   /* per-request extra data */
15      int           numa_node;
16      unsigned int  timeout;
17      unsigned int  flags;      /* BLK_MQ_F_* */
18      void          *driver_data;
19
20      struct blk_mq_tags  **tags;
21
22      struct mutex         tag_list_lock;
23      struct list_head     tag_list;
24  };
```

第 8 行，map[]数组为软硬件队列映射表。

第 9 行，nr_maps 为映射表数量

第 10 行，ops 为驱动实现的操作集合，会被 request_queue 继承。

第 11 行，nr_hw_queues 为硬件队列个数。

第 12 行，queue_depth 为队列深度。

第 15 行，numa_node 为所在 numa 节点。

第 17 行，flags 为标志位，一般为 BLK_MQ_F_SHOULD_MERGE，想了解跟多的标志位可以去看 include/linux/blk-mq.h 文件。

接着我们去看看 blk_mq_ops 结构体，结构体原型如下所示(有省略)：

示例代码 51.2.4.2 blk_mq_ops 结构体

```
1   struct blk_mq_ops {
2       queue_rq_fn     *queue_rq;
3   }
```

这里只是列出 queue_rq 成员，因为本章例程只用到它。queue_rq 这是一个 queue_rq_fn 类型的指针，queue_rq_fn 类型定义如下：

```
typedef blk_status_t (queue_rq_fn)(struct blk_mq_hw_ctx *,const struct blk_mq_queue_data *);
```

queue_rq 请求处理函数指针，此函数需要驱动人员自行实现。注意：函数的两个形参就不要管了，只要知道可以通过第二形参获取 request 结构体。

编写块设备的请求队列驱动的时候需要分配并初始化一个 blk_mq_tag_set，linux 内核提供了一组 blk_mq_tag_set 操作相关的函数，我们来看一下一些常用的 API 函数。

1)、为一个或多个请求队列分配 tag 集合

给 blk_mq_tag_set 对象赋值后，要使用 blk_mq_alloc_tag_set 函数为一个或多个请求队列分配 tag 和 request 集合，函数原型如下：

```
int  blk_mq_alloc_tag_set(struct  blk_mq_tag_set  *set);
```

函数参数和返回值含义如下：

set：需要分配 tag 集合的 blk_mq_tag_set。

返回值：0 表示成功，非 0 表示失败。

2)、释放请求队列中的 tag 集合

如果要释放请求队列中的 tag 集合，可以使用 blk_mq_free_tag_set，函数原型如下：

```
void blk_mq_free_tag_set(struct blk_mq_tag_set *set);
```

函数参数和返回值含义如下：

set：要释放 tag 集合的 blk_mq_tag_set。

返回值：无。

最后我们需要通过 blk_mq_init_queue 函数来初始化 IO 请求队列 request_queue，此函数会申请 request_queue，然后返回，函数原型如下：

```
struct request_queue *blk_mq_init_queue(struct blk_mq_tag_set *);
```

函数参数和返回值含义如下：

set：blk_mq_tag_set 对象。

返回值：初始化以后的 request_queue 的地址。

Linux 内核也提供了一步创建 request_queue 队列的函数：blk_mq_init_sq_queue，使用此函数可以一步创建请求队列，函数原型如下：

```
struct request_queue *blk_mq_init_sq_queue(struct blk_mq_tag_set *set,
                                           const struct blk_mq_ops *ops,
                                           unsigned int queue_depth,
                                           unsigned int set_flags)
```

函数参数和返回值含义如下：

set：blk_mq_tag_set 对象。

ops：操作函数。

queue_depth：队列深度。

set_flags：标志。

返回值：request_queue 的地址。

②、删除请求队列

当卸载块设备驱动的时候我们还需要删除掉前面申请到的 request_queue，删除请求队列使用函数 blk_cleanup_queue，函数原型如下：

```
void blk_cleanup_queue(struct request_queue      *q)
```

函数参数和返回值含义如下：

q：需要删除的请求队列。

返回值：无。

③、分配请求队列并绑定制造请求函数

blk_mq_init_queue 函数完成了请求队列的申请以及请求处理函数的绑定，这个一般用于像机械硬盘这样的存储设备，需要 I/O 调度器来优化数据读写过程。但是对于 EMMC、SD 卡这样的非机械设备，可以进行完全随机访问，所以就不需要复杂的 I/O 调度器了。对于非机械设备我们可以先申请 request_queue，然后将申请到的 request_queue 与“制造请求”函数绑定在一起。先来看一下 request_queue 申请函数 blk_alloc_queue，函数原型如下：

```
struct request_queue *blk_alloc_queue(gfp_t       gfp_mask)
```

函数参数和返回值含义如下：

gfp_mask：内存分配掩码，具体可选择的掩码值请参考 include/linux/gfp.h 中的相关宏定义，一般为 GFP_KERNEL。

返回值：申请到的无 I/O 调度的 request_queue。

我们需要为申请到的请求队列绑定一个“制造请求”函数(其他参考资料将其直接翻译为“制造请求”函数)。这里我们需要用到函数 blk_queue_make_request，函数原型如下：

```
void blk_queue_make_request(struct request_queue     *q, make_request_fn      *mfn)
```

函数参数和返回值含义如下：

q：需要绑定的请求队列，也就是 blk_alloc_queue 申请到的请求队列。

mfn：需要绑定的“制造”请求函数，函数原型如下：

```
void (make_request_fn) (struct request_queue     *q, struct bio   *bio)
```

“制造请求”函数需要驱动编写人员实现。

返回值：无。

一般 blk_alloc_queue 和 blk_queue_make_request 是搭配在一起使用的，用于那些非机械的存储设备、无需 I/O 调度器，比如 EMMC、SD 卡等。blk_init_queue 函数会给请求队列分配一个 I/O 调度器，用于机械存储设备，比如机械硬盘等。

2、请求 request

请求队列(request_queue)里面包含的就是一系列的请求(request)，request 是一个结构体，定义在 include/linux/blkdev.h 里面，这里就不展开 request 结构体了，太长了。request 里面有一个名为“bio”的成员变量，类型为 bio 结构体指针。前面说了，真正的数据就保存在 bio 里面，所以我们需要从 request_queue 中取出一个一个的 request，然后再从每个 request 里面取出 bio，最后根据 bio 的描述讲数据写入到块设备，或者从块设备中读取数据。

①、开启请求

当有请求处理的时候，我们要用 blk_mq_start_request 函数开启请求处理，函数原型如下：

```
void blk_mq_start_request(struct request *rq);
```

函数参数和返回值含义如下：

rq：指定 request_queue。

返回值：无。

②、结束请求

我们不用处理请求的时候，要使用 blk_mq_end_request 函数结束请求处理，函数原型如下：

```
void blk_mq_end_request(struct request *rq, blk_status_t error);
```

函数参数和返回值含义如下：

rq：需要结束的请求(request)。

error：0 表示正确的退出结束请求处理，非 0 错误退出结束请求处理。

返回值：无。

这里我们先总结一下是如何使用这个两个函数去处理请求数据的，示例代码如下所示：

示例代码 51.2.4.3 处理请求模型

```
1  static int ramdisk_transfer(struct request *req)
2  {
3      /* 此函数要实现把数据拷贝到硬盘 */
4      reutrn 0;
5  }
6
7  static blk_status_t _queue_rq(struct blk_mq_hw_ctx *hctx,
                                 const struct blk_mq_queue_data* bd)
8  {
9      struct request *req = bd->rq;
10     int ret;
11     /* 开启请求处理 */
```

```
12      blk_mq_start_request(req);
13
14      /* 处理请求 */
15      ret = ramdisk_transfer(req);
16
17      /* 结束请求处理 */
18      blk_mq_end_request(req, ret);
19
20      return 0;
21  }
```

3、bio 结构

每个 request 里面里面会有多个 bio，bio 保存着最终要读写的数据、地址等信息。上层应用程序对于块设备的读写会被构造成一个或多个 bio 结构，bio 结构描述了要读写的起始扇区、要读写的扇区数量、是读取还是写入、页便宜、数据长度等等信息。上层会将 bio 提交给 I/O 调度器，I/O 调度器会将这些 bio 构造成 request 结构，request_queue 里面顺序存放着一系列的 request。新产生的 bio 可能被合并到 request_queue 里现有的 request 中，也可能产生新的 request，然后插入到 request_queue 中合适的位置，这一切都是由 I/O 调度器来完成的。request_queue、request 和 bio 之间的关系如图 51.2.4.1 所示：

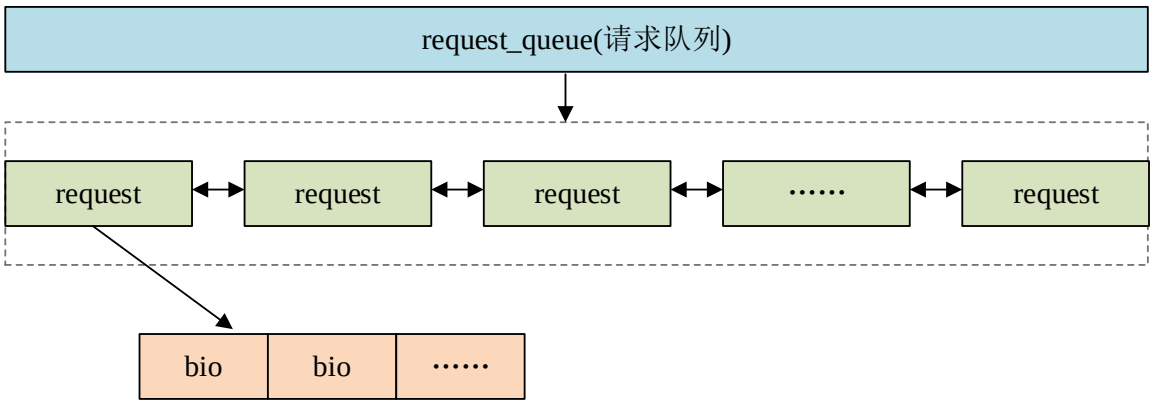


图 51.2.4.1 request_queue、request 和 bio 之间的关系

bio 是个结构体，定义在 include/linux/blk_types.h 中，结构体内容如下(有缩减)：

示例代码 51.2.4.4 bio 结构体

```
1  struct bio {
2       struct bio          *bi_next;   /* 请求队列的下一个 bio */
3       struct gendisk      *bi_disk;
4       unsigned int        bi_opf;     /* bottom bits req flags,
5                           * top bits REQ_OP. Use
6                           * accessors.
7                           */
8       unsigned short      bi_flags;   /* bio 状态等信息          */
9       unsigned short      bi_ioprio;
10      unsigned short      bi_write_hint;
11      blk_status_t        bi_status;
```

```
12      u8                bi_partno;
13      struct bvec_iter  bi_iter;    /* I/O 操作,读或写        */
14      atomic_t          __bi_remaining;
15      bio_end_io_t      *bi_end_io;
16      void              *bi_private;
17      unsigned short    bi_vcnt;    /* bio_vec 列表中元素数量  */
18
19  /*
20   * Everything starting with bi_max_vecs will be preserved by
       bio_reset()
21   */
22
23  unsigned short        bi_max_vecs;  /* bio_vec 列表长度    */
24  atomic_t              __bi_cnt;     /* pin count           */
25  struct bio_vec        *bi_io_vec;   /* bio_vec  列表       */
26  struct bio_set        *bi_pool;
27
28  /*
29   * We can inline a number of vecs at the end of the bio, to avoid
30   * double allocations for a small number of bio_vecs. This member
31   * MUST obviously be kept at the very end of the bio.
32   */
33  struct bio_vec      bi_inline_vecs[0];
34  };
```

重点来看一下第 13 行和第 25 行，第 13 行为 bvec_iter 结构体类型的成员变量，第 25 行为 bio_vec 结构体指针类型的成员变量。

bvec_iter 结构体描述了要操作的设备扇区等信息，结构体内容如下：

示例代码 51.2.4.5 bvec_iter 结构体

```
1   struct bvec_iter {
2       sector_t bi_sector;          /* I/O 请求的设备起始扇区(512 字节) */
3       unsigned int bi_size;        /* 剩余的 I/O 数量                  */
4       unsigned int bi_idx;         /* blv_vec 中当前索引               */
5       unsigned int bi_bvec_done;   /* 当前 bvec 中已经处理完成的字节数  */
6   };
```

bio_vec 结构体描述了内容如下：

示例代码 51.2.4.6 bio_vec 结构体

```
1   struct bio_vec {
2       struct page *bv_page;    /* 页   */
3       unsigned int bv_len;     /* 长度 */
4       unsigned int bv_offset;  /* 偏移 */
5   }
```

可以看出 bio_vec 就是“page,offset,len”组合，page 指定了所在的物理页，offset 表示所处页的偏移地址，len 就是数据长度。

我们对于物理存储设备的操作不外乎就是将 RAM 中的数据写入到物理存储设备中，或者将物理设备中的数据读取到 RAM 中去处理。数据传输三个要求：数据源、数据长度以及数据目的地，也就是你要从物理存储设备的哪个地址开始读取、读取到 RAM 中的哪个地址处、读取的数据长度是多少。既然 bio 是块设备最小的数据传输单元，那么 bio 就有必要描述清楚这些信息，其中 bi_iter 这个结构体成员变量就用于描述物理存储设备地址信息，比如要操作的扇区地址。bi_io_vec 指向 bio_vec 数组首地址，bio_vec 数组就是 RAM 信息，比如页地址、页偏移以及长度，“页地址”是 linux 内核里面内存管理相关的概念，这里我们不深究 linux 内存管理，我们只需要知道对于 RAM 的操作最终会转换为页相关操作。

bio、bvec_iter 以及 bio_vec 这三个结构体之间的关系如图 51.2.4.2 所示：

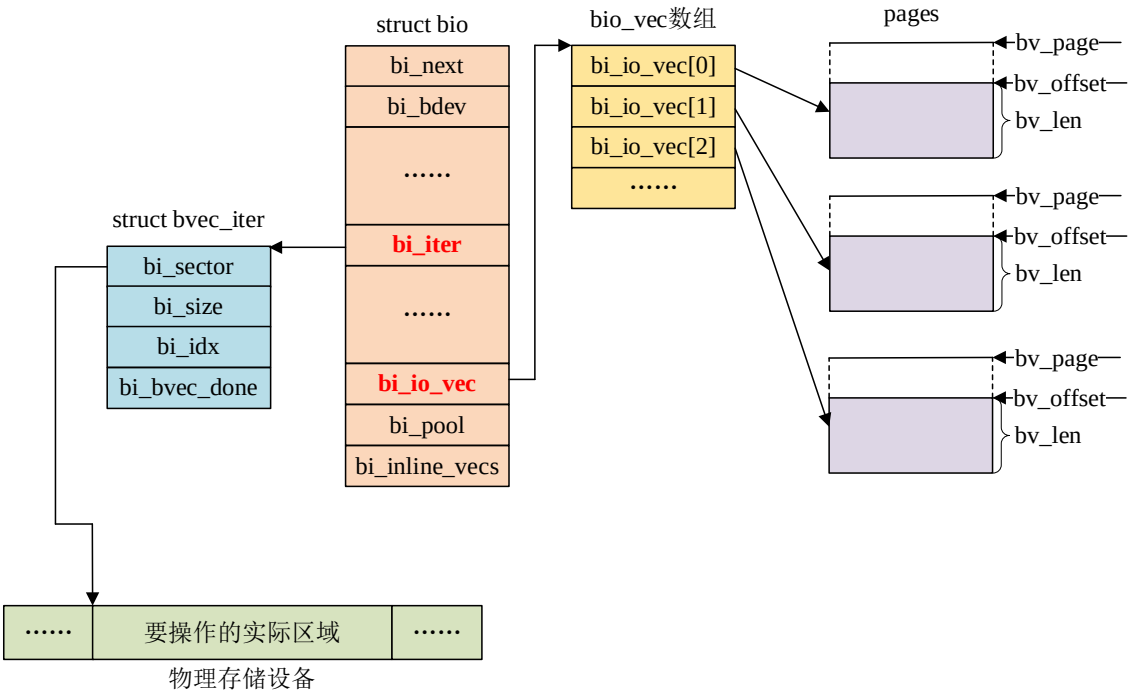


图 51.2.4.2 bio、bio_iter 与 bio_vec 之间的关系

①、遍历请求中的 bio

前面说了，请求中包含有大量的 bio，因此就涉及到遍历请求中所有 bio 并进行处理。遍历请求中的 bio 使用函数__rq_for_each_bio，这是一个宏，内容如下：

示例代码 51.2.4.7 __rq_for_each_bio 函数

```
#define __rq_for_each_bio(_bio, rq) \
    if ((rq->bio))          \
        for (_bio = (rq)->bio; _bio; _bio = _bio->bi_next)
```

_bio 就是遍历出来的每个 bio，rq 是要进行遍历操作的请求，_bio 参数为 bio 结构体指针类型，rq 参数为 request 结构体指针类型。

②、遍历 bio 中的所有段

bio 包含了最终要操作的数据，因此还需要遍历 bio 中的所有段，这里要用到 bio_for_each_segment 函数，此函数也是一个宏，内容如下：

示例代码 51.2.4.8 bio_for_each_segment 函数

```
#define bio_for_each_segment(bvl, bio, iter) \
```

```
__bio_for_each_segment(bvl, bio, iter, (bio)->bi_iter)
```

第一个 bvl 参数就是遍历出来的每个 bio_vec，第二个 bio 参数就是要遍历的 bio，类型为 bio 结构体指针，第三个 iter 参数保存要遍历的 bio 中 bio_iter 成员变量。

③、通知 bio 处理结束

如果使用“制造请求”，也就是抛开 I/O 调度器直接处理 bio 的话，在 bio 处理完成以后要通过内核 bio 处理完成，使用 bio_endio 函数，函数原型如下：

```
bvoid bio_endio(struct bio *bio, int error)
```

函数参数和返回值含义如下：

bio：要结束的 bio。

error：如果 bio 处理成功的话就直接填 0，如果失败的话就填个负值，比如-EIO。

返回值：无

51.3 使用请求队列实验

关于块设备架构就讲解这些，接下来我们使用开发板上的 RAM 模拟一段块设备，也就是 ramdisk，然后编写块设备驱动。

51.3.1 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→26_ramdisk_withrequest。

首先这个实验为多队列请求实验，在内核 5.0 之后才有的。本实验是使用 ROM 模拟块设备的空间，没有使用硬件队列相关的函数。由于实验程序稍微有点长，因此我们就分步骤来讲解一下，实验是参数自 linux 内核 drivers/block/z2ram.c。我们先来看一下相关的宏定义和结构体，代码如下：

示例代码 51.3.1.1 宏定义和结构体

```
11  #include <linux/module.h>
......
24  #include <linux/genhd.h>
25  #include <linux/blk-mq.h>
26  #include <linux/buffer_head.h>
27  #include <linux/bio.h>
28
29  #define RAMDISK_SIZE    (2 * 1024 * 1024)      /* 容量大小为 2MB    */
30  #define RAMDISK_NAME    "ramdisk"              /* 名字              */
31  #define RADMISK_MINOR   3       /* 表示有三个磁盘分区！不是次设备号为 3！ */
32
33  /* ramdisk 设备结构体 */
34  struct ramdisk_dev{
35      int major;                      /* 主设备号                      */
36      unsigned char *ramdiskbuf;      /* ramdisk 内存空间,用于模拟块设备 */
37      struct gendisk *gendisk;        /* gendisk                       */
38      struct request_queue *queue;    /* 请求队列                      */
39      struct blk_mq_tag_set tag_set;  /* blk_mq_tag_set                */
```

```
40      spinlock_t lock;                /* 自旋锁                          */
41  };
42
43  struct ramdisk_dev *ramdisk = NULL;     /* ramdisk 设备指针 */
```

第 29~31 行，实验相关宏定义，RAMDISK_SIZE 就是模拟块设备的大小，这里设置为 2MB，也就是说本实验中的虚拟块设备大小为 2MB。RAMDISK_NAME 为本实验名字，RADMISK_MINOR 是本实验此设备号数量，注意不是次设备号！此设备号数量决定了本块设备的磁盘分区数量。

第 34~41 行，ramdisk 的设备结构体。

第 43 行，定义一个全局变量 ramdisk_dev 类型的指针。

接下来看一下驱动模块的加载与卸载，内容如下：

示例代码 51.3.1.2 驱动加载与卸载

```
222 static int __init ramdisk_init(void)
223 {
224     int ret = 0;
225     struct ramdisk_dev * dev;
226     printk("ramdisk init\r\n");
227
228     /* 1、申请内存 */
229     dev = kzalloc(sizeof(*dev), GFP_KERNEL);
230     if(dev == NULL) {
231         return -ENOMEM;
232     }
233
234     dev->ramdiskbuf = kmalloc(RAMDISK_SIZE, GFP_KERNEL);
235     if(dev->ramdiskbuf == NULL) {
236         printk(KERN_WARNING "dev->ramdiskbuf: vmalloc failure.\n");
237         return -ENOMEM;
238     }
239     ramdisk = dev;
240
241     /* 2、初始化自旋锁 */
242     spin_lock_init(&dev->lock);
243
244     /* 3、注册块设备 */
245     dev->major = register_blkdev(0, RAMDISK_NAME);
246     if(dev->major < 0) {
247         goto register_blkdev_fail;
248     }
249
250     /* 4、创建多队列 */
251     dev->queue = create_req_queue(dev);
252     if(dev->queue == NULL) {
```

```
253         goto create_queue_fail;
254     }
255 
256     /* 5、创建块设备 */
257     ret = create_req_gendisk(dev);
258     if(ret < 0)
259         goto create_gendisk_fail;
260 
261     return 0;
262 
263 create_gendisk_fail:
264     blk_cleanup_queue(dev->queue);
265     blk_mq_free_tag_set(&dev->tag_set);
266 create_queue_fail:
267     unregister_blkdev(dev->major, RAMDISK_NAME);
268 register_blkdev_fail:
269     kfree(dev->ramdiskbuf);
270     kfree(dev);
271     return -ENOMEM;
272 }
273 
274 /*
275  * @description   : 驱动出口函数
276  * @param         : 无
277  * @return        : 无
278  */
279 static void __exit ramdisk_exit(void)
280 {
281 
282     printk("ramdisk exit\r\n");
283     /* 释放 gendisk */
284     del_gendisk(ramdisk->gendisk);
285     put_disk(ramdisk->gendisk);
286 
287     /* 清除请求队列 */
288     blk_cleanup_queue(ramdisk->queue);
289 
290     /* 释放 blk_mq_tag_set */
291     blk_mq_free_tag_set(&ramdisk->tag_set);
292 
293     /* 注销块设备 */
294     unregister_blkdev(ramdisk->major, RAMDISK_NAME);
295 
```

```
296     /* 释放内存 */
297     kfree(ramdisk->ramdiskbuf);
298     kfree(ramdisk);
299 }
```

ramdisk_init 和 ramdisk_exit 这两个函数分别为驱动入口以及出口函数，我们依次来看一下这两个函数。

第 229 行，先给自定义的 ramdisk 结构体申请空间。

第 234 行，因为本实验是使用一块内存模拟真实的块设备，因此这里先使用 kzalloc 函数申请用于 ramdisk 实验的内存，大小为 2MB。

第 242 行，初始化一个自旋锁，用于在队列操作的时候做保护。

第 245 行，使用 register_blkdev 函数向内核注册一个块设备，返回值就是注册成功的块设备主设备号。这里我们让内核自动分配一个主设备号，因此 register_blkdev 函数的第一个参数为 0。

第 251 行，create_req_queue 函数用于创建一个多队列。这一部分主要用于操作块设备，后面会讲解此函数。

第 257 行，create_req_gendisk 函数用于创建一个块设备和提供一些接口给应用层调用，后面会讲解此函数。

ramdisk_exit 函数就比较简单了，在卸载块设备驱动模块的时候需要将前面注册的对象进行卸载、实例化的对象要进行释放。代码当中注释已经非常清楚了，这里就不给大家解释了。

接着我们就先看看 create_req_queue 函数是如何初始化队列的，如下示例代码所示：

示例代码 51.3.1.3 create_req_queue 函数

```
153 static struct request_queue * create_req_queue(struct ramdisk_dev
*dev)
154 {
155     struct request_queue *q;
156     struct blk_mq_alloc_tag_set *set = &dev->tag_set;
157 #if 0
158
159     /*
160      *这里是使用了 blk_mq_init_sq_queue 函数
161      *进行初始化的。
162      */
163     q = blk_mq_init_sq_queue(set, &mq_ops, 2,BLK_MQ_F_SHOULD_MERGE);
164
165 #else
166     int ret;
167
168     memset(set, 0, sizeof(*set));
169     set->ops = &mq_ops;                     /* 操作函数 */
170     set->nr_hw_queues = 2;                  /* 硬件队列 */
171     set->queue_depth = 2;                   /* 队列深度 */
172     set->numa_node = NUMA_NO_NODE;          /* numa 节点 */
173     set->flags =  BLK_MQ_F_SHOULD_MERGE;    /* 标记在 bio 下发时需要合并 */
```

```
174
175     ret = blk_mq_alloc_tag_set(set);    /* 使用函数进行再次初始化 */
176     if (ret) {
177         printk(KERN_WARNING "sblkdev: unable to allocate tag set\n");
178         return ERR_PTR(ret);
179     }
180
181     q = blk_mq_init_queue(set);         /* 分配请求队列 */
182     if(IS_ERR(q)) {
183         blk_mq_free_tag_set(set);
184         return q;
185     }
186 #endif
187
188     return q;
189 }
```

第 166~185 行，初始化多队列。在 169 行到 173 行里先设置多队列的重要参数，比如一些操作函数、队列深度、硬件队列个数和标志位等等；其中第 169 行设置 blk_mq_tag_set 的 ops 成员变量，这就是块设备的队列操作集，这里设置为 mq_ops，需要驱动开发人员自行编写实现，后面讲解。使用 blk_mq_alloc_tag_set 函数进行再次初始化 blk_mq_tag_set 对象，最后我们根据此对象分配请求队列。我们也可以使用 blk_mq_init_sq_queue 函数一步到位，第一个参数为 blk_mq_tag_set 对象、第二个参数为操作函数集合、第三个参数为硬件队列个数，第四个参数为标志位。

有了多队列后，我们就可以使用 gendisk 进行初始化块设备了，初始化块设备的函数如下所示：

示例代码 51.3.1.4 create_req_gendisk 函数

```
196 static int create_req_gendisk(struct ramdisk_dev *set)
197 {
198     struct ramdisk_dev *dev = set;
199
200     /* 1、分配并初始化 gendisk */
201     dev->gendisk = alloc_disk(RADMISK_MINOR);
202     if(dev == NULL)
203         return -ENOMEM;
204
205     /* 2、添加(注册)disk */
206     dev->gendisk->major = ramdisk->major;  /* 主设备号     */
207     dev->gendisk->first_minor = 0;         /* 起始次设备号 */
208     dev->gendisk->fops = &ramdisk_fops;    /* 操作函数     */
209     dev->gendisk->private_data = set;      /* 私有数据     */
210     dev->gendisk->queue = dev->queue;      /* 请求队列     */
211     sprintf(dev->gendisk->disk_name, RAMDISK_NAME); /* 名字 */
212     set_capacity(dev->gendisk, RAMDISK_SIZE/512); /* 设备容量(扇区)*/
```

```
213     add_disk(dev->gendisk);
214     return 0;
215 }
```

第 201 行，使用 alloc_disk 分配一个 gendisk。

第 206~211 行，初始化申请到的 gendisk 对象，重点第 208 行设置 geddisk 的 fops 成员变量，fops 负责设置块设备的操作集。第 210 行设置我们的多队列。

第 212 行，使用 set_capacity 函数设置本块设备容量大小，注意这里的大小是扇区数，不是字节数，一个扇区是 512 字节。

第 213 行，gendisk 初始化完成以后就可以使用 add_disk 函数将 gendisk 添加到内核中，也就是向内核添加一个磁盘设备。

块设备的初始化和多队列的初始化都有自己的操作集，我们依次来看下这两个操作集的具体内容如下：

①、gendisk 的 fops 操作集

示例代码 51.3.1.5 gendisk 的 fops 操作集

```
99  /*
100  * @description   : 打开块设备
101  * @param - dev   : 块设备
102  * @param - mode  : 打开模式
103  * @return        : 0 成功;其他 失败
104  */
105 int ramdisk_open(struct block_device *dev, fmode_t mode)
106 {
107     printk("ramdisk open\r\n");
108     return 0;
109 }
110
111 /*
112  * @description   : 释放块设备
113  * @param - disk  : gendisk
114  * @param - mode  : 模式
115  * @return        : 0 成功;其他 失败
116  */
117 void ramdisk_release(struct gendisk *disk, fmode_t mode)
118 {
119     printk("ramdisk release\r\n");
120 }
121
122 /*
123  * @description   : 获取磁盘信息
124  * @param - dev   : 块设备
125  * @param - geo   : 模式
126  * @return        : 0 成功;其他 失败
127  */
```

```
128 int ramdisk_getgeo(struct block_device *dev,
                      struct hd_geometry *geo)
129 {
130     /* 这是相对于机械硬盘的概念 */
131     geo->heads = 2;          /* 磁头 */
132     geo->cylinders = 32;     /* 柱面 */
133     geo->sectors = RAMDISK_SIZE / (2 * 32 *512);/* 磁道上的扇区数量 */
134     return 0;
135 }
136
137 /*
138  * 块设备操作函数
139  */
140 static struct block_device_operations ramdisk_fops =
141 {
142     .owner     = THIS_MODULE,
143     .open      = ramdisk_open,
144     .release   = ramdisk_release,
145     .getgeo    = ramdisk_getgeo,
146 };
```

第 140~126 行就是块设备的操作集 block_device_operations，本例程实现的比较简单，仅仅实现了 open、release 和 getgeo，其中 open 和 release 函数都是空函数，重点是 getgeo 函数，第 128~134 行就是 getgeo 的具体实现，此函数眼红红获取磁盘信息，信息保存在参数 geo 中，geo 的类型为 hd_geometry 结构体，如下：

示例代码 51.3.1.6 hd_geometty 结构体内容

```
1  struct hd_geometry {
2      unsigned char      heads;          /* 磁头                  */
3      unsigned char      sectors;        /*一个磁道上的扇区数量   */
4      unsigned short     cylinders;      /* 柱面                  */
5      unsigned long      start;
6  };
```

本例程中设置 ramdisk 有 2 个磁头(head)、一共 32 个柱面(cylinderr)。知道磁盘总容量、磁头数、柱面数以后我们就可以计算出一共磁道上有多少个扇区了，也就是 hd_geometry 中的 sectors 成员变量。

②、blk_mq_tag_set 的 ops 操作集

最后 blk_mq_tag_set 的 ops 就是请求处理函数集合，ops 的操作集如下内容所示：

示例代码 51.3.1.7 blk_mq_tag_set 的 ops 操作集

```
45 /*
46  * @description   : 处理传输过程
47  * @param-req     : 请求
48  * @return        : 0，成功；其它表示失败
49  */
50 static int ramdisk_transfer(struct request *req)
```

```
51  {
52      unsigned long start = blk_rq_pos(req) << 9; /* blk_rq_pos 获取到的
                                          是扇区地址，左移 9 位转换为字节地址 */
53      unsigned long len  = blk_rq_cur_bytes(req);   /* 大小   */
54
55
56      /* bio 中的数据缓冲区
57       * 读：从磁盘读取到的数据存放到 buffer 中
58       * 写：buffer 保存这要写入磁盘的数据
59       */
60      void *buffer = bio_data(req->bio);
61
62      if(rq_data_dir(req) == READ)            /* 读数据 */
63          memcpy(buffer, ramdisk->ramdiskbuf + start, len);
64      else if(rq_data_dir(req) == WRITE)      /* 写数据 */
65          memcpy(ramdisk->ramdiskbuf + start, buffer, len);
66
67      return 0;
68
69  }
70
71  /*
72   * @description   : 开始处理传输数据的队列
73   * @hctx          : 硬件相关的队列结构体
74   * @bd            : 数据相关的结构体
75   * @return        : 0，成功；其它值为失败
76   */
77  static blk_status_t _queue_rq(struct blk_mq_hw_ctx *hctx,
                                  const struct blk_mq_queue_data* bd)
78  {
79      struct request *req = bd->rq; /* 通过 bd 获取到 request 队列*/
80      struct ramdisk_dev *dev = req->rq_disk->private_data;
81      int ret;
82
83      blk_mq_start_request(req);      /* 开启处理队列 */
84      spin_lock(&dev->lock);
85      ret = ramdisk_transfer(req);    /* 处理数据     */
86      blk_mq_end_request(req, ret);   /* 结束处理队列 */
87      spin_unlock(&dev->lock);
88
89      return BLK_STS_OK;
90
91  }
```

```
92  /*
93   * 队列操作函数
94   */
95  static struct blk_mq_ops mq_ops = {
96      .queue_rq = _queue_rq,
97  };
```

第 95~97 行就是多队列的操作集 blk_mq_os，这里我们只是实现了 queue_rq，第 77~91 行就是 queue_rq 的具体实现；第 79 行通过 blk_mq_queue_data 的 rq 成员获取到 request，第 83 行使用 blk_mq_start_request 函数开启多队列处理，第 85 行就是我们自定义的处理数据函数 ramdisk_transfer，第 86 行就是我们通过 blk_mq_end_request 函数去结束多队列处理。

ramdisk_transfer 数据处理函数，第 52 行 blk_rq_pos 函数获取要操作的块设备扇区地址，第 53 行使用 blk_rq_cur_bytes 函数获取请求要操作的数据长度，第 60 行使用 bio_data 函数获取请求中的 bio 保存的数据。第 62~65 行 rq_data_dio 函数判断当前是读还是写，如果是写的话就将 bio 中的数据拷贝到 ramdisk 指定地址(扇区)，如果是读的话就从 ramdisk 中的指定打字(扇区)读取数据放到 bio 中。

这里就简单总结一下块设备的编写步骤，首先有两个重要的结构体：blk_mq_tag_set 和 gendisk。可以把 blk_mq_tag_set 看作真正的 IO 读写操作(ops 操作集就是 IO 操作)，有了底层操作还不行，还需要 gendisk 结构体为上层提供接口调用(fops 就是实现上层调用的操作)。

51.3.2 运行测试

1、编译驱动程序

编写 Makefile 文件，将 22_spi 实验目录下的 Makefile 文件拷贝到本实验目录下，打开改 Makefile 文件，将 obj-m 变量的值改为“ramdisk.o”，Makefile 内容如下所示：

示例代码 51.4.1.1 Makefile 文件

```
1  KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4  obj-m := ramdisk.o
......
11 clean:
12  $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“ramdisk.o”。

输入如下命令编译出驱动模块文件：

```
make -j32
```

编译成功以后就会生成一个名为“ramdisk.ko”的驱动模块文件。

2、使能 mkfs.vfat 命令

我们还需要要在 buildroot 目录下，打开 busybox 的图形化配置界面，使能 mkfs.vfat。命令如下所示：

```
sudo make busybox-menuconfig
```

按照以下路径使能 mkfs.vfat 命令：

```
-> Linux System Utilities
    -> [*] mkfs.vfat (7.2 kb)
```

如图 51.3.2.1 所示：

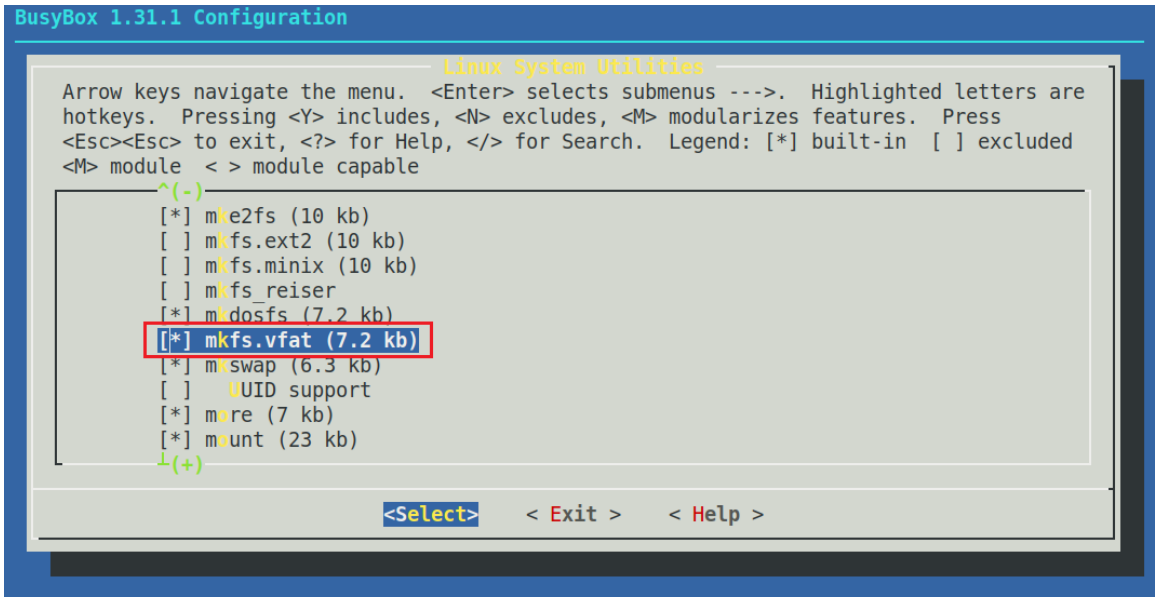


图 51.3.2.1 使能 mkfs.vfat 命令

保存 busybox 的配置，重新编译 busybox，运行以下命令：

```
sudo make busybox
```

编译完成 busybox 后我们还需要重新编译 buildroot，命令如下：

```
sudo make
```

编译完成后，进入 output/images 目录，运行以下命令把文件系统替换进去：

```
cd output/images/                                          //进入到 output/images 目录
sudo tar -axvf rootfs.tar -C /home/zuozhongkai/linux/nfs/rootfs  //解压到 nfsroot 目录
```

上述命令将 buildroot 中 output/images/rootfs.tar 这个压缩包解压到/home/zuozhongkai/linux/nfs/rootfs 这个目录中，这个目录就是我当前 nfsroot 目录，大家根据自己的实际情况解压到对应的目录文件中。

将前面编译出来的 ramdisk.ko 文件拷贝到 rootfs/lib/modules/5.3.41 目录中，重启开发板，进入到目录 lib/modules/5.3.41 中。输入如下命令加载 ramdisk.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的实验需要运行此命令
modprobe ramdisk.ko     //加载驱动模块
```

正常加载驱动就会有如下所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe ramdisk.ko
ramdisk init
ramdisk open
ramdisk release
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 51.3.2.2 加载 ramdisk.ko 驱动

3、查看 ramdisk 磁盘

驱动加载成功以后就会在/dev/目录下生成一个名为“ramdisk”的设备，输入如下命令查看 ramdisk 磁盘信息：

```
fdisk -l        //查看磁盘信息
```

上述命令会将当前系统中所有的磁盘信息都打印出来，其中就包括了 ramdisk 设备，如图 51.4.2.3 所示：

```
Number  Start (sector)    End (sector)  Size Name
     1            1024            5119 2048K ssbl
     2            5120          136191 64.0M boot
     3          136192        15268863 7389M rootfs
Disk /dev/ramdisk: 2 MB, 2097152 bytes, 4096 sectors
32 cylinders, 2 heads, 64 sectors/track
Units: sectors of 1 * 512 = 512 bytes

Disk /dev/ramdisk doesn't contain a valid partition table
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 51.4.2.3 ramdisk 磁盘信息

从同 51.4.2.3 可以看出，ramdisk 已经识别出来了，大小为 2MB，但是同时也提示/dev/ramdisk 没有分区表，因为我们还没有格式化/dev/ramdisk。

4、格式化/dev/ramdisk

使用 mkfs.vfat 命令格式化/dev/ramdisk，将其格式化成 vfat 格式，输入如下命令：

```
mkfs.vfat /dev/ramdisk
```

格式化完成以后就可以挂载/dev/ramdisk 来访问了，挂载点可以自定义，这里笔者就将其挂载到/mnt 目录下，输入如下命令：

```
mkdir /mnt/ram_disk -P                  //创建 ramdisk 挂载目录
mount /dev/ramdisk /mnt/ram_disk        //挂载 ramdisk
```

挂载成功以后就可以通过/mnt 来访问 ramdisk 这个磁盘了，进入到/mnt 目录中，可以通过 vi 命令新建一个 txt 文件来测试磁盘访问是否正常。

51.4 不使用请求队列实验

51.4.1 实验程序编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→27_ramdisk_norequest。

前面我们学习了如何使用请求队列，请求队列会用到 I/O 调度器，适合机械硬盘这种存储设备。对于 EMMC、SD、ramdisk 这样没有机械结构的存储设备，我们可以直接访问任意一个扇区，因此可以不需要 I/O 调度器，也就不需要请求队列了，这个我们前面已经说过了。本实验就来学习一下如何使用“制造请求”方法，本实验在上一个实验的基础上修改而来，参考了 linux 内核 drivers/block/zram/zram_drv.c。首先是驱动入口函数 ramdisk_init，ramdisk_init 函数大部分和上一个实验相同，我们只需要把 blk_mq_tag_set 相关的都删除掉，然后修改 create_req_queue 函数即可，在此函数里使用 create_req_queue 函数设置“制造请求”函数，修改后的 create_req_queue 函数内容如下：

示例代码 51.4.1.1 create_req_queue 函数

```
130 static struct request_queue * create_req_queue(struct ramdisk_dev
*dev)
131 {
132     struct request_queue *q;
133     /* 分配请求队列 */
134     q = blk_alloc_queue(GFP_KERNEL);
135
136     /* 设置“制造请求”函数 */
```

```
137     blk_queue_make_request(q, ramdisk_make_request_fn);
138
139     /* 保存 dev 方便在 ramdisk_make_request_fn 函数使用 */
140     q->queuedata = dev;
141     return q;
142 }
```

create_req_queue 函数与上一个实验不同的地方在于，这里使用 blk_alloc_queue 和 blk_queue_make_request 这两个函数取代了上一个实验中 blk_mq_tag_set 结构体相关的操作。

第 134 行，使用 blk_alloc_queue 函数申请一个请求队列。

第 137 行，使用 blk_queue_make_request 函数设置“制造请求”函数，这里设置的制造请求函数为 ramdisk_make_request_fn，这个需要驱动编写人员去实现，稍后讲解。

第 140 行，保存我们自定义的 ramdisk_dev 结构体。

接下来重点看一下“制造请求”函数 ramdisk_make_request_fn，函数内容如下：

示例代码 51.5.1.1 ramdisk_make_request_fn 函数

```
99  static blk_qc_t ramdisk_make_request_fn(struct request_queue *q,
                                            struct bio *bio)
100 {
101     int offset;
102     struct bio_vec bvec;
103     struct bvec_iter iter;
104     unsigned long len = 0;
105     struct ramdisk_dev *dev = q->queuedata;
106
107     offset = (bio->bi_iter.bi_sector) << 9;/* 获取偏移地址 */
108     spin_lock(&dev->lock);
109     /* 处理 bio 中的每个段 */
110     bio_for_each_segment(bvec, bio, iter){
111         char *ptr = page_address(bvec.bv_page) + bvec.bv_offset;
112         len = bvec.bv_len;
113
114         if(bio_data_dir(bio) == READ)           /* 读数据 */
115             memcpy(ptr, dev->ramdiskbuf + offset, len);
116         else if(bio_data_dir(bio) == WRITE)     /* 写数据 */
117             memcpy(dev->ramdiskbuf + offset, ptr, len);
118         offset += len;
119     }
120     spin_unlock(&dev->lock);
121     bio_endio(bio);
122     return BLK_QC_T_NONE;
123 }
```

虽然 ramdisk_make_request_fn 函数第一个参数依旧是请求队列，但是实际上这个请求队列不包含真正的请求，所有的处理内容都在第二个 bio 参数里面，所以 ramdisk_make_request_fn 函数里面是全部是对 bio 的操作。

第 103 行，直接读取 bio 的 bi_iter 成员变量的 bi_sector 来获取要操作的设备地址(扇区)。

第 110~119 行，使用 bio_for_each_segment 函数循环获取 bio 中的每个段，然后对其每个段进行处理。

第 111 行，根据 bio_vec 中页地址以及偏移地址转为真正的数据起始地址。

第 112 行，获取要出来的数据长度，也就是 bio_vec 的 bv_len 成员变量。

第 114~117 行，和上一个实验一样，要操作的块设备起始地址找到了，数据的存放地址以及长度也知道，接下来就是根据读写操作将数据从块设备中读出来，或者将数据写入到块设备中。

第 121 行，调用 bio_endio 函数，结束 bio。

51.4.2 运行测试

测试方法和上一个实验一样，参考 51.3 小节即可。

第五十二章 Linux 网络设备驱动实验

网络驱动是 linux 里面驱动三巨头之一，linux 下的网络功能非常强大，嵌入式 linux 中也常常用到网络功能。前面我们已经讲过了字符设备驱动和块设备驱动，本章我们就来学习一下 linux 里面的网络设备驱动。

52.1 嵌入式网络简介

52.1.1 嵌入式下的网络硬件接口

本章节讨论的都是有线网络！

提起网络，我们一般想到的硬件就是“网卡”，“网卡”这个概念最早从电脑领域传出来，顾名思义就是能上网的卡。在电脑领域的“原始社会”，网卡是独立的硬件，如果电脑要上网就得买个网卡插上去，类似现在的显卡一样。但是大家现在观察自己的笔记本或者台式机主板会发现并没有类似显卡一样的网卡设备，原因是随着技术的不断发展，现在只需要一个芯片就可以实现有线网卡功能，因此网卡芯片都直接放到了主板上。所以大家在接触嵌入式的时候听到“网卡”这两个字，不要急着在开发板上找“卡”一样的东西。

既然现在网卡已经是通过一个芯片来完成了，那么是什么样的芯片呢？这个就要先了解一下嵌入式中的网络硬件方案。首先，嵌入式网络硬件分为两部分：MAC 和 PHY，大家都是通过看数据手册来判断一款 SoC 是否支持网络，如果一款芯片数据手册说自己支持网络，一般都是说的这款 SoC 内置 MAC，MAC 类似 I2C 控制器、SPI 控制器一样的外设。但是光有 MAC 还不能直接驱动网络，还需要另外一个芯片：PHY，因此对于内置 MAC 的 SoC，其外部必须搭配一个 PHY 芯片。但是有些 SoC 内部没有 MAC，那也就没法搭配 PHY 芯片了，这些内部没有网络 MAC 的芯片如何上网呢？这里就要牵扯出常见的两个嵌入式网络硬件方案了。

1、SOC 内部没有网络 MAC 外设

我们一般说某个 SoC 不支持网络，说的就是它没有网络 MAC。那么这个芯片就不能上网了吗？显然不是的，既然没有内部 MAC，那么可以找个外置的 MAC 芯片啊，不过一般这种外置的网络芯片都是 MAC+PHY 一体的。比如三星 linux 开发板里面用的最多的 DM9000，因为三星的芯片基本没有内部 MAC(比如 S3C2440、S5PV210，4412 等)，所以三星的开发板都是通过外置的 DM9000 来完成有线网络功能的，DM9000 对 SoC 提供了一个 SRAM 接口，SOC 会以 SRAM 的方式操作 DM9000。

有些外置的网络芯片更强大，内部甚至集成了硬件 TCP/IP 协议栈，对外提供一个 SPI 接口，比如 W5500。这个一般用于单片机领域，单片机通过 SPI 接口与 W5500 进行通信，由于 W5500 内置了硬件 TCP/IP 协议栈，因此单片机就不需要移植负责的软件协议栈，直接通过 SPI 来操作 W5500，简化了单片机联网方案。

这种方案的优点就是让不支持网络的 SoC 能够另辟蹊径，实现网络功能，但是缺点就是网络效率不高，因为一般芯片内置的 MAC 会有网络加速引擎，比如网络专用 DMA，网络处理效率会很高。而且此类芯片网速都不快，基本就是 10/100M。另外，相比 PHY 芯片而言，此类芯片的成本也比较高，可选择比较少。

SoC 与外部 MAC+PHY 芯片的连接如图 52.1.1.1 所示：

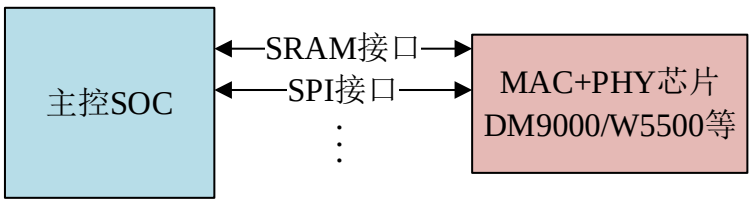


图 52.1.1.1 主控 SOC 与外置 MAC+PHY 芯片连接

2、SOC 内部集成网络 MAC 外设

我们一般说某个 SoC 支持网络，说的就是他内部集成网络 MAC 外设，此时我们还需要外

接一个网络 PHY 芯片。此时就有朋友会有疑问，PHY 芯片不能也集成进 SoC 吗？笔者目前还没见过将 PHY 也集成到芯片里面的 SoC。

一般常见的通用 SoC 都会集成网络 MAC 外设，比如 STM32F4/F7/H7 系列、NXP 的 I.MX 系列以及 STM32MP1 系列，内部集成网络 MAC 的优点如下：

①、内部 MAC 外设会有专用的加速模块，比如专用的 DMA，加速网速数据的处理。

②、网速快，可以支持 10/100/1000M 网速。

③、外接 PHY 可选择性多，成本低。

内部的 MAC 外设会通过相应的接口来连接外部 PHY 芯片，根据数据传输模式不同，大致可以分为以下两类：

①、MII/RMII 接口：支持 10Mbit/s 和 100Mbit/s 数据传输模式；

②、GMII/RGMII 接口：支持 10Mbit/s、100Mbit/s 以及 1000Mbit/s 数据传输模式。

从这里可以知道，MII/RMII 接口最大传输速率为 100Mbit/s，而 GMII/RGMII 接口最大传输速率可达 1000Mbit/s；所以笔者一般把 MII/RMII 称为百兆以太网接口，而把 GMII/RGMII 称为千兆以太网接口。

关于这两组接口更加详细的内容会在后面给大家进行介绍，MII/RMII 或 GMII/RGMII 接口是用来传输网络数据的，另外主控 SoC 需要配置或读取 PHY 芯片，也就是读写 PHY 的内部寄存器，所以还需要一个控制接口，叫做 MIDO，MDIO 很类似 IIC，也是两根线，一根数据线叫做 MDIO，一根时钟线叫做 MDC。

SoC 内部 MAC 外设与外部 PHY 芯片的连接如图 52.1.1.2、图 52.1.1.3 所示：

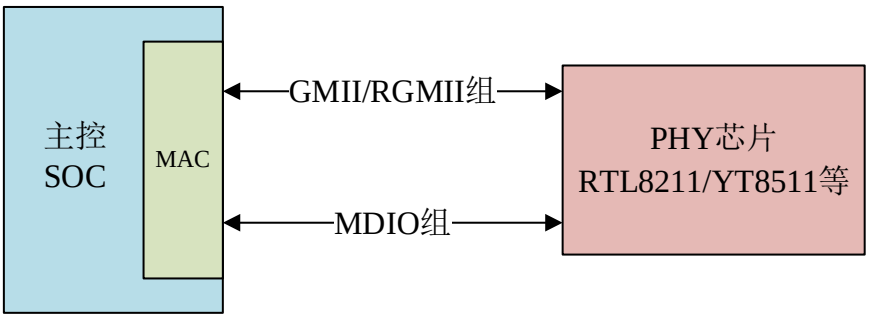


图 52.1.1.2 内部 MAC 与外部 PHY 之间的连接(千兆以太网)

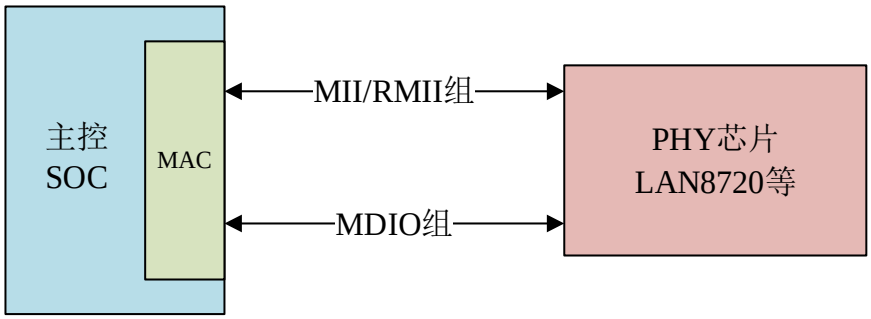


图 52.1.1.3 内部 MAC 与外部 PHY 之间的连接(百兆以太网)

大家在做项目的时候，如果要用到网络功能，强烈建议大家选择内部带有网络 MAC 外设的主控 SOC！比如 STM32MP1 就有一颗 10M/100M/1000M 的网络 MAC 外设，正点原子的 STM32MP1 开发板板载了一颗 PHY 芯片，V1.2 版本及其以前的核心板使用 RTL8211F-CG 这颗 PHY 芯片， V1.3 版本及其以后核心板使用 YT8511C/H 这颗 PHY 芯片。

因此，本章节只讲解 SOC 内部 MAC+外置 PHY 芯片这种方案。

52.1.2 MII/RMII、GMII/RGMII 接口

前面我们说了，内部 MAC 通过 MII/RMII 接口或者 GMII/RGMII 接口来与外部的 PHY 芯片连接，完成网络数据传输，本节我们就来学习一下什么是 MII/GMII、GMII/RGMII 接口。

1、MII 接口

MII 全称是 Media Independent Interface，直译过来就是介质独立接口，它是 IEEE-802.3 定义的以太网标准接口，MII 接口用于以太网 MAC 连接 PHY 芯片，连接示意图如图 52.1.2.1 所示：

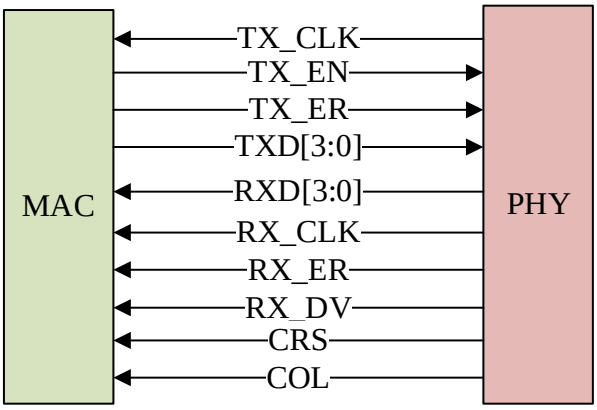


图 52.1.2.1 MII 接口

MII 接口一共有 16 根信号线，含义如下：

TX_CLK：发送时钟，如果网速为 100M 的话时钟频率为 25MHz，10M 网速的话时钟频率为 2.5MHz，此时钟由 PHY 产生并发送给 MAC。

TX_EN：发送使能信号。

TX_ER：发送错误信号，高电平有效，表示 TX_ER 有效期内传输的数据无效。10Mpbs 网速下 TX_ER 不起作用。

TXD[3:0]：发送数据信号线，一共 4 根。

RXD[3:0]：接收数据信号线，一共 4 根。

RX_CLK：接收时钟信号，如果网速为 100M 的话时钟频率为 25MHz，10M 网速的话时钟频率为 2.5MHz，RX_CLK 也是由 PHY 产生的。

RX_ER：接收错误信号，高电平有效，表示 RX_ER 有效期内传输的数据无效。10Mpbs 网速下 RX_ER 不起作用。

RX_DV：接收数据有效，作用类似 TX_EN。

CRS：载波侦听信号。

COL：冲突检测信号。

MII 接口的缺点就是所需信号线太多，这还没有算 MDIO 和 MDC 这两根管理接口的数据线，因此 MII 接口使用已经越来越少了。

2、RMII 接口

RMII 全称是 Reduced Media Independent Interface，翻译过来就是精简的介质独立接口，也就是 MII 接口的精简版本。RMII 接口只需要 7 根数据线，相比 MII 直接减少了 9 根，极大的方便了板子布线，RMII 接口连接 PHY 芯片的示意图如图 52.1.2.2 所示：

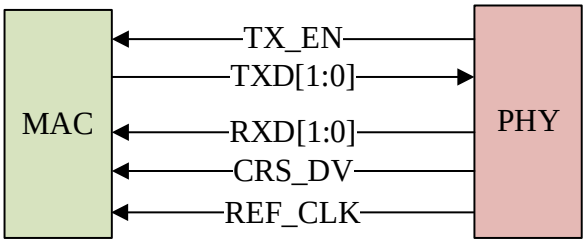


图 52.1.2.2 RMII 接口

TX_EN：发送使能信号。

TXD[1:0]：发送数据信号线，一共 2 根。

RXD[1:0]：接收数据信号线，一共 2 根。

CRS_DV：相当于 MII 接口中的 RX_DV 和 CRS 这两个信号的混合。

REF_CLK：参考时钟，由外部时钟源提供，频率为 50MHz。这里与 MII 不同，MII 的接收和发送时钟是独立分开的，而且都是由 PHY 芯片提供的。

3、GMII 接口

GMII（Gigabit Media Independant Interface），千兆 MII 接口。GMII 采用 8 位接口数据，工作时钟 125MHz，因此传输速率可达 1000Mbps；同时兼容 MII 所规定的 10/100Mbps 工作方式。GMII 接口数据结构符合 IEEE 以太网标准，该接口定义见 IEEE 802.3-2000。信号定义如下：

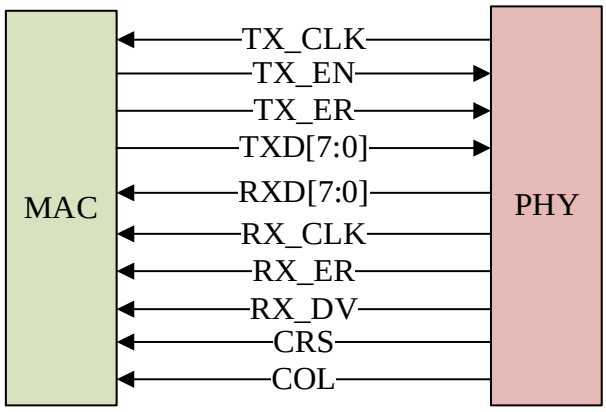


图 52.1.2.3 GMII 接口

GTX_CLK：1000M 工作模式下的发送时钟（125MHz）。

TX_EN：发送使能信号。

TX_ER：发送错误信号，高电平有效，表示 TX_ER 有效期内传输的数据无效。

TXD[7:0]：发送数据信号线，一共 8 根。

RXD[7:0]：接收数据信号线，一共 8 根。

RX_CLK：接收时钟信号。

RX_ER：接收错误信号，高电平有效，表示 RX_ER 有效期内传输的数据无效。

RX_DV：接收数据有效，作用类似 TX_EN。

CRS：载波侦听信号。

COL：冲突检测信号。

与 MII 接口相比，GMII 的数据宽度由 4 位变为 8 位，GMII 接口中的控制信号如 TX_ER、TX_EN、RX_ER、RX_DV、CRS 和 COL 的作用同 MII 接口中的一样，发送参考时钟 GTX_CLK 和接收参考时钟 RX_CLK 的频率均为 125MHz(在 1000Mbps 工作模式下)。

在实际应用中，绝大多数 GMII 接口都是兼容 MII 接口的，所以，一般的 GMII 接口都有

两个发送参考时钟：TX_CLK 和 GTX_CLK(两者的方向是不一样的，前面已经说过了)，在用作 MII 模式时，使用 TX_CLK 和 8 根数据线中的 4 根。

4、RGMII 接口

RGMII(Reduced Gigabit Media Independant Interface),精简版 GMII 接口。将接口信号线数量从 24 根减少到 14 根(COL/CRS 端口状态指示信号，这里没有画出)，时钟频率仍旧为 125MHz，TX/RX 数据宽度从 8 为变为 4 位，为了保持 1000Mbps 的传输速率不变，RGMII 接口在时钟的上升沿和下降沿都采样数据，在参考时钟的上升沿发送 GMII 接口中的 TXD[3:0]/RXD[3:0]，在参考时钟的下降沿发送 GMII 接口中的 TXD[7:4]/RXD[7:4]。RGMII 同时也兼容 100Mbps 和 10Mbps 两种速率，此时参考时钟速率分别为 25MHz 和 2.5MHz。

TX_EN 信号线上传送 TX_EN 和 TX_ER 两种信息，在 TX_CLK 的上升沿发送 TX_EN，下降沿发送 TX_ER；同样的，RX_DV 信号线上也传送 RX_DV 和 RX_ER 两种信息，在 RX_CLK 的上升沿发送 RX_DV，下降沿发送 RX_ER。

RGMII 接口定义如下所示：

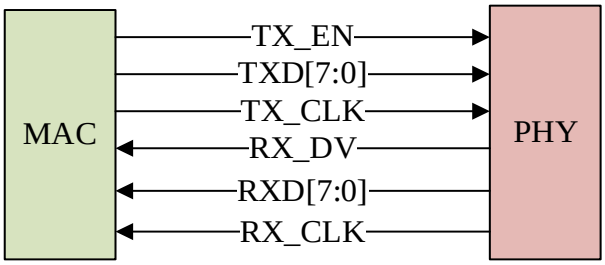


图 52.1.2.4 RGMII 接口

关于这些接口定义相关的内容我们就讲到这里了，除了上面说到 4 种接口以外，还有其他接口，比如 SMII、SSMII 和 SGMII 等，关于其他接口基本都是大同小异的，这里就不做讲解了。正点原子的 STM32MP1 开发板上的网口是采用 RGMII 接口来连接 MAC 与外部 PHY 芯片。

52.1.3 MDIO 接口

MDIO 全称是 Management Data Input/Output，直译过来就是管理数据输入输出接口，是一个简单的两线串行接口，一根 MDIO 数据线，一根 MDC 时钟线。驱动程序可以通过 MDIO 和 MDC 这两根线访问 PHY 芯片的任意一个寄存器。MDIO 接口支持多达 32 个 PHY。同一时刻内只能对一个 PHY 进行操作，那么如何区分这 32 个 PHY 芯片呢？和 IIC 一样，使用器件地址即可。同一 MDIO 接口下的所有 PHY 芯片，其器件地址不能冲突，必须保证唯一，具体器件地址值要查阅相应的 PHY 数据手册。

因此，MAC 和外部 PHY 芯片进行连接的时候主要是 MII/RMII（百兆网）或 GMII/RGMII（千兆网）和 MDIO 接口，另外可能还需要复位、中断等其他引脚。

52.1.4 RJ45 接口

网络设备是通过网线连接起来的，插入网线的叫做 RJ45 座，如图 52.1.4.1 所示：

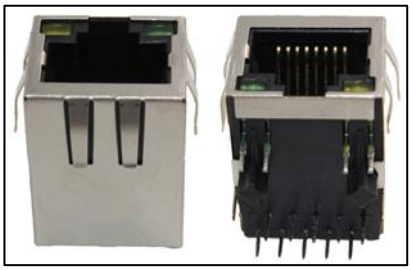


图 52.1.4.1 RJ45 座子

RJ45 座要与 PHY 芯片连接在一起，但是中间需要一个网络变压器，网络编译器用于隔离以及滤波等，网络变压器也是一个芯片，外形一般如图 52.1.4.2 所示：

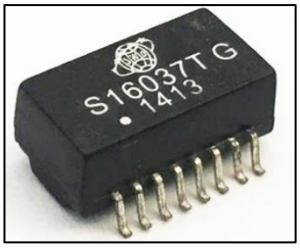


图 52.1.4.2 网络变压器

但是现在很多 RJ45 座子内部已经集成了网络变压器，比如正点原子的 STM32MP1 开发板所使用的 ATK911130A 就是内置网络变压器的 RJ45 座。内置网络变压器的 RJ45 座和不内置的引脚一样，但是一般不内置的 RJ45 座会短一点。因此，大家在画板的时候一定要考虑你所使用的 RJ45 座是否内置网络变压器，如果不内置的话就要自行添加网络变压器部分电路！同理，如果你所设计的硬件是需要内置网络变压器的 RJ45 座，肯定不能随便焊接一个不内置变压器的 RJ45 座，否则网络工作不正常！

RJ45 座子上一般有两个灯，一个黄色(橙色)，一个绿色，一般绿色亮的话表示网络连接正常，黄色闪烁的话说明当前正在进行网络通信，当然了有时候两个灯的状态会反过来，大家以实际为准。这两个灯由 PHY 芯片控制，对于千 M 网络 PHY 芯片，一般 PHY 芯片会有 3 个 LED 灯引脚，多了一个千 M 网络状态指示灯。PHY 芯片会通过这几个 LED 灯引脚来连接 RJ45 座上的这两个灯(千 M PHY 会有 3 个 LED 灯引脚，一般硬件设计人员会自行选择将其中的哪两个连接到 RJ45 座上)。由于正点原子 STM32MP157 开发板采用的千 M 网络 PHY，所以后面只讲千 M 网络。内部 MAC+外部 PHY+RJ45 座(内置网络变压器)就组成了一个完整的嵌入式网络接口硬件，如图 52.1.4.3 所示：

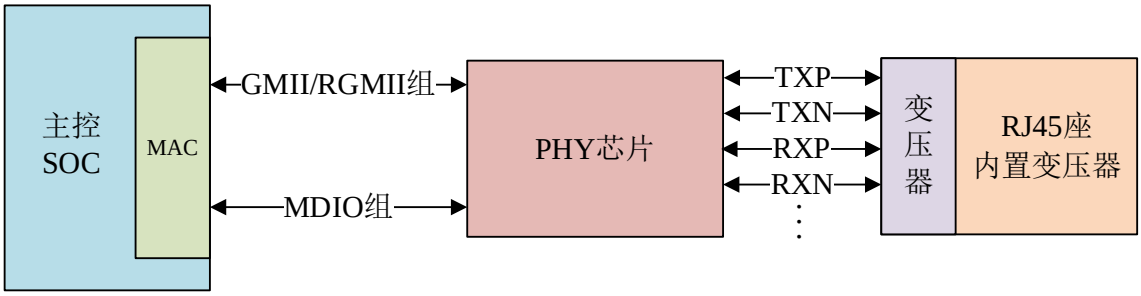


图 52.1.4.3 嵌入式网络硬件接口示意图

52.1.5 STM32MP1 GMAC 接口简介

STM32MP1 内核集成了一个 10M/100M/1000M 的网络 MAC，符合 IEEE802.3-2002 标准，

MAC 层支持双工或者半双工模式下运行。MAC 可编程，有直接存储器接口的专用 DMA，介质访问控制器(MAC)和支持多种格式的 PHY 接口模块。

STM32MP1 内部 ENET 外设主要特性如下：

①、支持全工和半双工操作。

②、全双工流控制操作(IEEE 802.3X 暂停包和优先级流控制)

③、报头和帧起始数据(SFD)在发送模式下自动插入、在接收中自动删除。

④、可逐帧控制 CRC 和 pad 自动生成

⑤、可编程数据包长度，支持标准以太网数据包或高达 16KB 的巨型以太网数据包

⑥、可编程数据包间隙

⑦、两组 FIFO：一个具有可编程阈值功能的 4096 字节发送 FIFO 和一个具有可配置阈值功能的 4096 字节接收 FIFO。

……

STM32MP1 的 GMAC 外设内容比较多，详细的介绍请查阅《STM32MP1 参考手册》的“Ethernet (ETH): Gigabit media access control(GMAC) with DMA controller”章节。我们在编写驱动的时候其实并不需要关注 GMAC 控制器外设的具体内容，因为这部分驱动是 SoC 厂商写的，我们重点关注的是更换 PHY 芯片以后哪里需要调整。

52.2 PHY 芯片详解

52.2.1 PHY 基础知识简介

PHY 是 IEEE 802.3 规定的一个标准模块，前面说了，SoC 可以对 PHY 进行配置或者读取 PHY 相关状态，这个就需要 PHY 内部寄存器去实现了。PHY 芯片寄存器地址空间为 5 位，地址 0~31 共 32 个寄存器，IEEE 定义了 0~15 这 16 个寄存器的功能，16~31 这 16 个寄存器由厂商自行实现。也就是说不管你用的哪个厂家的 PHY 芯片，其中 0~15 这 16 个寄存器是一模一样的。仅靠这 16 个寄存器是完全可以驱动起 PHY 芯片的，至少能保证基本的网络数据通信，因此 Linux 内核有通用 PHY 驱动，按道理来讲，不管你使用的哪个厂家的 PHY 芯片，都可以使用 Linux 的这个通用 PHY 驱动来验证网络工作是否正常。事实上在实际开发中可能会遇到一些其他的问题导致 Linux 内核的通用 PHY 驱动工作不正常，这个时候就需要驱动开发人员去调试了。但是，随着现在的 PHY 芯片性能越来越强大，32 个寄存器可能满足不了厂商的需求，因此很多厂商采用分页技术来扩展寄存器地址空间，以求定义更多的寄存器。这些多出来的寄存器可以用于实现厂商特有的一些技术，因此 Linux 内核的通用 PHY 驱动就无法驱动这些特色功能了，这个时候就需要 PHY 厂商提供相应的驱动源码了，所以大家也会在 Linux 内核里面看到很多具体的 PHY 芯片驱动源码。不管你的 PHY 芯片有多少特色功能，按道理来讲，Linux 内核的通用 PHY 驱动是绝对可以让你这 PHY 芯片实现基本的网络通信，因此大家也不用担心更换 PHY 芯片以后网络驱动编写是不是会很复杂。

IEEE802.3 协议英文原版已经放到了开发板光盘中，路径为：4、参考资料→802.3 协议英文原版_2018 年.pdf 打开此文档，此文档有 5600 页，按照 SECTION 进行分类，一共 8 个 SECTION。选中“802.3-2018_SECTION2”，找到“22.2.4 Management functions”章节，此章节对 PHY 的前 16 个寄存器功能进行了规定，如图 52.2.1.1 所示：

Register address	Register name	Basic/Extended	
		MII	GMII
0	Control	B	B
1	Status	B	B
2,3	PHY Identifier	E	E
4	Auto-Negotiation Advertisement	E	E
5	Auto-Negotiation Link Partner Base Page Ability	E	E
6	Auto-Negotiation Expansion	E	E
7	Auto-Negotiation Next Page Transmit	E	E
8	Auto-Negotiation Link Partner Received Next Page	E	E
9	MASTER-SLAVE Control Register	E	E
10	MASTER-SLAVE Status Register	E	E
11	PSE Control register	E	E
12	PSE Status register	E	E
13	MMD Access Control Register	E	E
14	MMD Access Address Data Register	E	E
15	Extended Status	Reserved	B
16 through 31	Vendor Specific	E	E

图 52.2.1.1 IEEE 规定的前 16 个寄存器

关于这 16 个寄存器的内容协议里面也进行了详细的讲解，这里就不分析了。我们后面会以正点原子 STM32MP1 开发板所使用的 RTL8211F-CG（YT8511C/H）这个 PHY 为例，详细的分析一下 PHY 芯片的寄存器。

52.2.2 RTL8211F-CG 详解

此小节是针对核心板为 V1.2 版本的 PHY 讲解，RTL8211F-CG 这颗 PHY 芯片是由 realtek 公司出品的，它的数据手册在开发板资料包里，路径为：6、硬件资料→1、芯片资料→RTL8211F-CG.pdf。

本小节我们以 RTL8211F-CG 为例给大家做一个简单第介绍，虽然本小节讲解的是 RTL8211F-CG 这颗 PHY，但是前面说了，IEEE 规定了 PHY 的前 16 个寄存器的功能，因此如果你所使用的 板子用的其它厂家的 PHY 芯片，也是可以看本小节的。

1、RTL8211F-CG 简介

Realtek RTL8211F-CG 是高度集成的以太网收发器，符合 10Base-T、100Base-TX 和 1000Base-T IEEE 802.3 标准。它提供了所有通过 CAT.5 UTP 电缆收发以太网数据包所需的必要物理层功能。

RTL8211F 使用最新的 DSP 技术和模拟前端（AFE），可以通过 UTP 电缆进行高速数据传输和接收。RTL8211F 中实现了交叉检测和自动校正、极性校正、自适应均衡、串扰消除、回声消除、定时恢复和错误校正等功能，以提供 10Mbps，100Mbps 或 1000Mbps 的强大收发功能。

MAC 和 PHY 之间的数据传输是通过 RGMII 接口进行的，RTL8211E 支持 RGMII 的 1.5V 信号。

RTL8211E 的主要特点如下：

- 兼容 1000Base-T IEEE 802.3ab 标准。
- 兼容 100Base-TX IEEE 802.3u 标准。
- 兼容 10Base-T IEEE 802.3 标准。
- 支持 GMII、RGMII 接口。
- 支持 IEEE 802.3az-2010（节能以太网）。
- 内置 LAN 唤醒（WOL）。
- 支持中断、并行检测、交叉检测、自动校正、自动极性校正。
- 支持 120m 的 1000Base-T 的 CAT.5 类电缆。
- 支持 RGMII 的 1.5V 信号。

……

RTL8211F 千兆 PHY 的系统应用场景如下：

- 数字电视（DTV）。
- 媒体访问单元（MAU）。
- 通讯和网络提升板（CNR）。
- 游戏机。
- 打印机和办公机器。
- DVD 播放机和刻录机。
- 以太网集线器、交换机。

RTL8211F 功能框图如下所示：

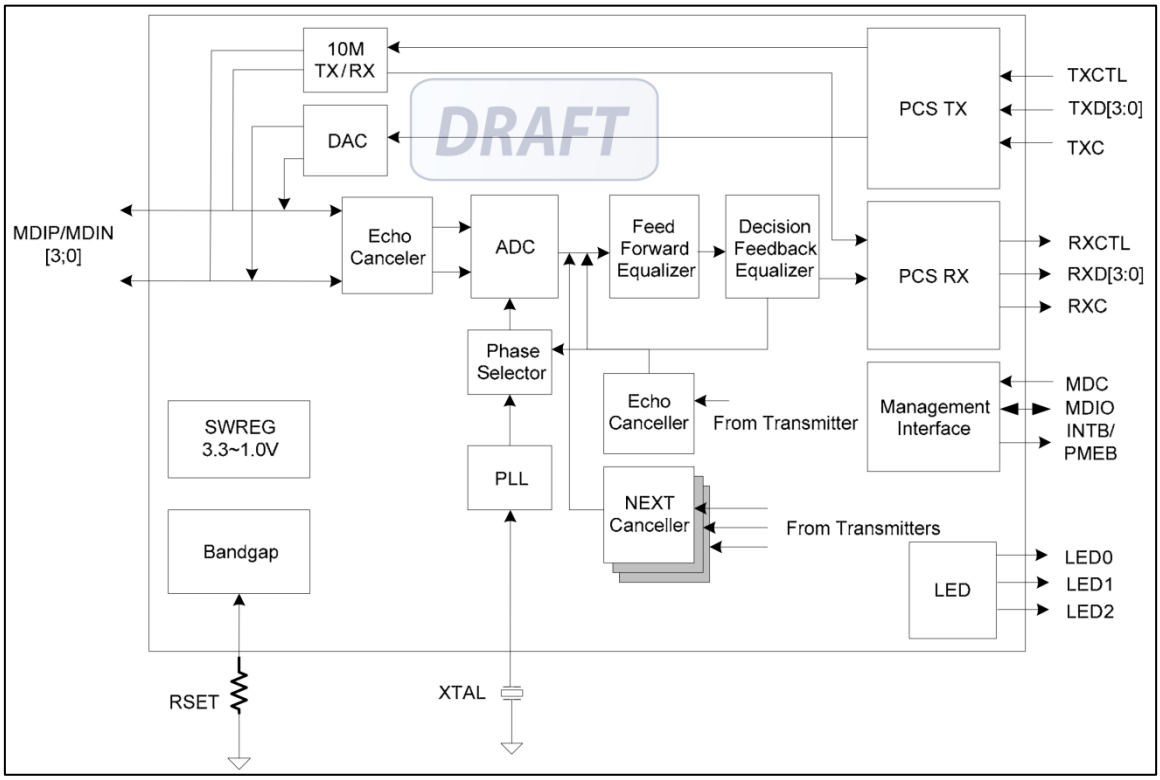


图 52.2.2.1 RTL8211F-CG 详解

2、节能以太网(EEE)

RTL8211F 支持 IEEE 802.3az-2010（也称为节能以太网（EEE）），速率为 10Mbps、100Mbps 或 1000Mbps 时它提供了一个协议，可根据链路利用率协调进出较低功耗级别（低功耗空闲模

式）的转换，当没有数据包传输时，系统进入低功耗空闲模式以降低功耗，一旦需要发送数据包，系统将返回正常模式，并且无需更改链接状态和丢弃/破坏帧即可进行此操作。

为了节省功率，当系统处于低功耗空闲模式时，大多数电路功能都被禁用；但是，低功耗空闲模式的过渡时间保持足够小，以使得对上层协议和应用程序透明。

3、中断管理

每当 RTL8211F 检测到介质状态发生变化时，它就会将中断引脚（INTB）拉低发出中断事件。SoC MAC 端会感应到状态更改，并通过 MDC/MDIO 接口访问相关的寄存器。

一旦 MAC 通过 MDC/MDIO 读取了这些状态寄存器，就会将 INTB 置为无效。所以不需要通过轮训的方式去反复查询状态寄存器的变化。

4、PHY 地址设置

MAC 层通过 MDIO/MDC 总线对 PHY 进行读写操作，MDIO 最多可以控制 32 个 PHY 芯片，通过不同的 PHY 芯片地址来对不同的 PHY 操作。RTL8211F 通过配置 PHYAD[2:0]这三个引脚来设置 PHY 地址，其中 PHYAD2 与 RXCTL 共用一个引脚、PHYAD1 与 RXC 共用一个引脚、PHYAD0 与 RXD3 共用一个引脚。可设置的 PHY 地址范围为：0X001~0X111，如表 52.2.2.1 所示。

PHYAD2(RXCTL) 引脚电平	PHYAD1(RXC) 引脚电平	PHYAD0(RXD3) 引脚电平	PHY 地址
0	0	1	0x001
0	1	0	0x010
1	0	0	0x100
0	1	1	0x011
……			0xXXX

表 52.2.2.1 PHY 地址设置

正点原子 STM32MP1 开发板上 RTL8211F 地址为 0x001。

5、自动协商

RTL8211F 可以开始启动协商功能，将 PHYSR 寄存器的 bit12 置 1 即可使能自动写上，如下图所示：

8.4.18. PHYSR (PHY Specific Status Register, Address 0x1A)

Table 39. PHYSR (PHY Specific Status Register, Address 0x1A)

Bit	Name	Type	Default	Description
26.15	RSVD	RO	0	Reserved.
26.14	ALDPS State	RO	0	Link Down Power Saving Mode. 1: Reflects local device entered Link Down Power Saving Mode, i.e., cable not plugged in (reflected after 3 sec). 0: With cable plugged in
26.13	MDI Plug	RO	0	MDI Status. 1: Plugged 0: Unplugged 为1使能自动协商功能
26.12	NWay Enable	RO	1	Auto-Negotiation (NWay) Status. 1: Enable 0: Disable
26.11	Master Mode	RO	0	Device is in Master/Slave Mode. 1: Master mode 0: Slave mode

图 52.2.2.2 使能自动协商

6、RGMII 接口电平配置

RTL8211F 支持 RGMII 接口，我们可以通过 CFG_EXT、CFG_LDO0 和 CFG_LDO1 这三个

引脚去配置 RGMII 的电平信号，配置方法如表 52.2.2.2 所示：

CFG_EXT 引脚电平	CFG_LDO0 引脚电平	CFG_LDO1 引脚电平	RGMII 电压信号
0	0	0	Reserved
0	0	1	2.5V
0	1	0	1.8V
0	1	1	1.5V
1	0	0	3.3V
1	0	1	2.5V
1	1	0	1.8V
1	1	1	1.5V

表 52.2.2.2 RGMII 的电平信号设置

正点原子 V1.2 版本之前的核心板 CFG_EXT 引脚外接 3.3V 上拉电阻，CFG_LDO0 和 CFG_LDO1 都是接到低了，因此 RGMII 电压信号为 3.3V。

7、RTL8211F 内部寄存器

RTL8211F 的前 16 个寄存器满足 IEEE 的要求，在这里我们只介绍几个常用的寄存器，首先是 BMCR(Basic Mode Control Register)寄存器，地址为 0，BMCR 寄存器各位如表 52.2.2.3 所示：

位	描述	类型
15	**软件复位** 1：软件复位 0：普通操作模式	R/W
14	**回环模式** 0：禁止 PCS 回环模式 1：使能 PCS 回环模式	R/W
13	**速度选择位 speed[0]** 该位将与 speed[1]共同来配置速度 注意：当使用自动协商功能时此位失能	R/W
12	**自动协商功能** 0：关闭自动协商功能 1：打开自动协商功能	R/W
11	**掉电（power down）** 0：正常运行 1：进入掉电模式 注意:进入掉电模式前自动协商必须失能	R/W
10	**隔离** 0：正常运行 1：PHY 的 RGMII/GMII 接口电气隔离	R/W
9	**重启自动协商功能** 0：正常运行 1：重启自动协商功能 注意:此位会被自动清零	R/W SC
8	**双工模式**	R/W

	0：半双工 1：全双工 注意:开启自动协商功能后此位失效	
7	**碰撞测试** 1：碰撞测试使能 0：正常运行	RW
6	**速度选择位 speed[1]** 该位将与 speed[0]共同来配置速度	RW
5:0	**保留**	RO

表 52.2.2.3 BMCR 寄存器

我们说的配置 PHY 芯片，重点就是配置 BMCR 寄存器。

接下来看一下 BMSR(Basic Mode Status Register)寄存器，地址为 1。此寄存器为 PHY 的状态寄存器，通过此寄存器可以获取到 PHY 芯片的工作状态，BMSR 寄存器各位如表 52.2.2.4 所示：

位	描述	类型
15	**100BAST-T4 功能** RTL8211E 不支持该功能，该位始终为 0	RO
14	**100BAST-TX 全双工功能** 0：不支持 TX 全双工 1：支持 TX 全双工	RO
13	**100BAST-TX 半双工功能** 0：不支持 TX 半双工 1：支持 TX 半双工	RO
12	**10BAST-T 全双工功能** 0：不支持 10Mbps 全双工 1：支持 10Mbps 全双工	RO
11	**10BAST-T 半双工功能** 0：不支持 10Mbps 半双工 1：支持 10Mbps 半双工	RO
10	**10Base-T2 全双工功能** 0：不支持 10Base-T2 全双工 1：支持 10Base-T2 全双工	RO
9	**10Base-T2 半双工功能** 0：不支持 10Base-T2 半双工 1：支持 10Base-T2 半双工	RO
8	**1000Base-T 扩展状态** 0：设备不支持扩展状态寄存器 0x0F 1：设备支持扩展状态寄存器 0x0F	RO
7	**保留**	RO
6	**前导抑制功能** 该位为 1，表示永久打开前导抑制功能	RO
5	**自协商功能完成**	RO

	0：自动协商未完成 1：自动协商完成	
4	**远端错误** 0：无远端错误 1：检测到远端错误	RC*
3	**自协商功能** 0：不能执行自协商功能 1：可以执行自协商功能	RO
2	**连接状态** 0：连接断开 1：连接建立	RO/LL
1	**Jabber 检测** 0：未检测到 jabber 1：检测到 jabber	RO/LH
0	**扩展功能** 0：不支持扩展寄存器 1：支持扩展寄存器	RO

表 52.2.2.4 BMCR 寄存器

从表 52.2.2.4 中可以看出，不管什么 PHY 芯片，只要它实现了的位和 IEEE 规定相符就行。通过读取 BMSR 寄存器的值我们可以得到当前的连接速度、双工状态和连接状态等。

接下来看一下 RTL8211F 的 PHY ID 1 和 PHY ID 2 这两个寄存器，这两个寄存器的地址分别为 2 和 3，后面就把这两个寄存器叫做寄存器 2 和寄存器 3 了。IEEE 规定寄存器 2 和寄存器 3 为 PHY 的 ID 寄存器，这两个寄存器组成一个 32 位的唯一 ID 值。IEEE 规定了一个叫做 OUI 的 ID 组成方式，全称是 Organizationally Unique Identifier，OUI 一共 32 位，分为三部分：22 位的 ID+6 位厂商型号 ID+4 位厂商版本 ID，组成如图 52.2.2.3 所示：

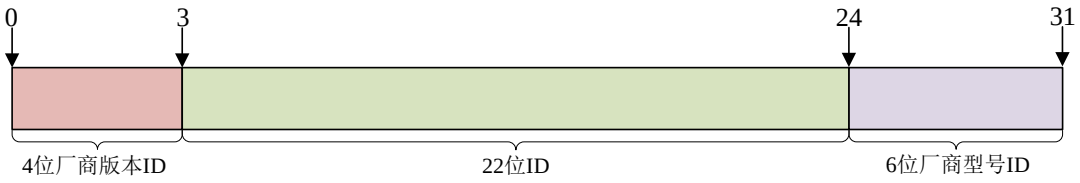


图 52.2.2.3 OUI 组成方式

RTL8211F 的 ID 寄存器 2 如表 52.2.2.5 所示：

位	描述	类型	默认值
15:0	22 位 ID 号中的低 16 位 也就是对应 OUI 的 bit[18:3]	RO	0000000000011100

表 52.2.2.5 PHY ID 寄存器 2

ID 寄存器 3 如表 52.2.2.6 所示：

位	描述	类型	默认值
15:10	22 为 ID 号中的高 6 位 也就是对应 OUI 的 bit[24:19]	RO	110010
9:4	厂商型号 ID	RO	010001
3:0	厂商版本 ID	RO	0101

表 52.2.2.6 PHY ID 寄存器 3

接着我们来看一下 RTL8211F 的 GBCR（1000Base-T Control Register）寄存器，寄存器地址为 0x9；该寄存器定义了 1000Base-T 功能的相关控制位，寄存器描述信息如下所示：

位	描述	类型	默认值
15:13	**测试模式选择：** 000：普通模式 001：测试模式 1--传输抖动测试 010：测试模式 2--传输抖动测试（主模式） 011：测试模式 3—传输抖动测试（从模式） 100：测试模式 4--传输失真测试 101、110、111：保留	RW	0
12	**启用手动主/从配置：** 0：自动主/从 1：手动主/从配置	RW	0
11	**主/从配置值设置：** 0：手动配置为从模式 1：手动配置为主模式	RW	0
10	**端口类型：** 0：首选单端口设备（SLAVE） 1：首选多端口设备（MASTER）	RW	0
9	**1000Base-T 全双工：** 0：不宣传 1：宣传	RW	1
8	**保留**	RW	0
7:0	**保留**	RO	0

表 52.2.2.7 GBCR 寄存器

关于 RTL8211F 这个 PHY 就讲解到这里了，更加详细的内容大家可以查看 RTL8211F 对应的参考手册 RTL8211F-CG.pdf。

52.2.3 YT8511C 详解

此小节针对 V1.3 版本及以后核心板，YT8511C 这颗 PHY 芯片是由苏州裕太车通电子科技有限公司出品的，它的数据手册已经放到了开发板资料包里，路径为：6、硬件资料→1、芯片资料→ YT8511_Datasheet_v1.10_release.pdf。

前面说了，PHY 芯片的前 16 个寄存器都是一样的，而且只需要使用这前 16 个寄存器基本就可以驱动起来 PHY 芯片。这里就不详细讲解了，YT8511C 和 RTL8211F 区别不大，此章节主要是讲一下 YT8511C 的对于 RTL8211F 差异。

1、PHY 地址设置

YT8511 通过 PHYADDR[2:0] 这三个引脚来确定 PHY 芯片地址，其中 PHYADDR2~PHYADDR0 分别对应 LED_AC、RXD1 和 RXD0 这三个引脚，配置如图 52.2.3.1 所示：

PHY Pin	PHY Core Config	Description	Default
RXD0	PHYADDRESS0	LED_ACT, RXD[1:0] sets the lower three bits of the physical address. The upper two bits of the physical address are set to the default, “00”	0
RXD1	PHYADDRESS1	LED_ACT, RXD[1:0] sets the lower three bits of the physical address. The upper two bits of the physical address are set to the default, “00”	0
LED_ACT	PHYADDRESS2	LED_ACT, RXD[1:0] sets the lower three bits of the physical address. The upper two bits of the physical address are set to the default, “00”	1

图 52.2.3.1 YT8511C PHY 地址设置

正点原子 STM32MP157 开发板上 YT8511 的地址为 0X00。

2、低功率模式

YT8511C 的低功率模式是由 RXD3 引脚控制，配置如下图所示：

RXD3	Low Power Mode selection	0= Force Low Power mode for shorter cable application 1= Normal cable application	0

图 52.2.3.2 YT8511C 低功率模式配置

从图 52.2.3.2 中可以看出来，RXD3 为低电平的时候 YT8511C 进入低功率模式，RXD3 为高电平的话 YT8511C 芯片就为正常功率模式。正点原子开发板 RXD3 连接高电平，所以 YT8511C 芯片为正常功率模式。

3、工作模式

YT8511C 的 LED_1000 引脚用来控制工作模式，配置如下图所示：

LED_1000	TEST Mode	0=Test Mode only for internal use 1=Normal Mode, Must external pull up

图 52.2.3.3 正常模式配置

从图 52.2.3.3 中可以看出来，LED_1000 为低电平的话就进入测试模式，为高电平的话就是正常模式。正点原子开发板 LED_1000 连接高电平，所以 YT8511C 芯片为正常模式。

4、RGMII 电压配置

YT8511C 的 RGMII 电压是由 RX_DV 引脚控制，如下图所示：

RX_DV	RGMII IO power selection	0= RGMII IO 3.3V 1= RGMII IO 2.5V

图 52.2.3.4 RGMII 的电压配置

从图 52.2.3.4 可以看出如果 RX_DV 为低电平的时候 RGMII 电平为 3.3V，RX_DV 为高电平的时候 RGMII 电平为 2.5V。正点原子开发板 RX_DV 连接高电平，所以 YT8511C 的引脚电平为 3.3V。

5、125M 时钟使能

STM32MP157 千 M 网络工作的时候需要和 PHY 之间有一个 125MHz 的时钟，在这里 YT8511C 提供这个 125MHz 的时钟。但是默认情况下，YT8511C 这个 125MHz 时钟没有使能，

所以需要配置寄存器，使能这个 125MHz。这里需要用到 YT8511C 的时钟控制寄存器，这是个扩展寄存器，地址为 0X0C，这个寄存器是 YT8511C 官方自定义的，不属于 IEEE 规定的寄存器。时钟控制寄存器的 bit2 和 bit1 用于设置时钟输入，如下图所示：

2:01	Clk_25m_sel	RW	0x1	select which clock is output to clk_25m pad 2'b00, 25m from pll; 2'b01, 25m from xtl; 2'b10: 62.5m from pll; 2'b11: 125m from pll

图 52.2.3.5 时钟控制寄存器

从图 52.2.3.5 可以看出，当 bit2:1 为 11 的时候，输出 125M 时钟。

芯片 YT8511C 还有很多设置，这边就不一一列出了，大家直接查看芯片原理图就知道了。

52.3 Linux 内核网络驱动框架

52.3.1 net_device 结构体

Linux 内核使用 net_device 结构体表示一个具体的网络设备，net_device 是整个网络驱动的灵魂。网络驱动的核心就是初始化 net_device 结构体中的各个成员变量，然后将初始化完成以后的 net_device 注册到 Linux 内核中。net_device 结构体定义在 include/linux/netdevice.h 中，net_device 是一个庞大的结构体，内容如下(有缩减)：

示例代码 52.3.1.1 net_device 结构体

```
1783  struct net_device {
1784      char            name[IFNAMSIZ];
1785      struct hlist_node   name_hlist;
1786      struct dev_ifalias  __rcu *ifalias;
1787      /*
1788       *  I/O specific fields
1789       *  FIXME: Merge these and struct ifmap into one
1790       */
1791      unsigned long       mem_end;
1792      unsigned long       mem_start;
1793      unsigned long       base_addr;
1794      int         irq;
1795
1796      /*
1797       *  Some hardware also needs these fields (state,dev_list,
1798       *  napi_list,unreg_list,close_list) but they are not
1799       *  part of the usual set specified in Space.c.
1800       */
1801
1802      unsigned long       state;
1803
1804      struct list_head    dev_list;
1805      struct list_head    napi_list;
```

```
1806        struct list_head    unreg_list;
1807        struct list_head    close_list;
......
1841        const struct net_device_ops *netdev_ops;
1842        const struct ethtool_ops *ethtool_ops;
1843    #ifdef CONFIG_NET_L3_MASTER_DEV
1844        const struct l3mdev_ops *l3mdev_ops;
1845    #endif
1846    #if IS_ENABLED(CONFIG_IPV6)
1847        const struct ndisc_ops *ndisc_ops;
1848    #endif
1849
1850    #ifdef CONFIG_XFRM_OFFLOAD
1851        const struct xfrmdev_ops *xfrmdev_ops;
1852    #endif
1853
1854    #if IS_ENABLED(CONFIG_TLS_DEVICE)
1855        const struct tlsdev_ops *tlsdev_ops;
1856    #endif
1857
1858        const struct header_ops *header_ops;
1859
1860        unsigned int        flags;
......
1869        unsigned char       if_port;
1870        unsigned char       dma;
1871
1872        /* Note : dev->mtu is often read without holding a lock.
1873         * Writers usually hold RTNL.
1874         * It is recommended to use READ_ONCE() to annotate the
1875         * reads, and to use WRITE_ONCE() to annotate the writes.
1876         */
1877        unsigned int        mtu;
1878        unsigned int        min_mtu;
1879        unsigned int        max_mtu;
1880        unsigned short      type;
1881        unsigned short      hard_header_len;
1882        unsigned char       min_header_len;
1883
1884        unsigned short      needed_headroom;
1885        unsigned short      needed_tailroom;
1886
1887        /* Interface address info. */
```

```
1888        unsigned char       perm_addr[MAX_ADDR_LEN];
1889        unsigned char       addr_assign_type;
1890        unsigned char       addr_len;
.....
1938    /*
1939     * Cache lines mostly used on receive path (including
1940     eth_type_trans()) */
1941        /* Interface address info used in eth_type_trans() */
1942        unsigned char       *dev_addr;
1943
1944        struct netdev_rx_queue  *_rx;
1945        unsigned int        num_rx_queues;
1946        unsigned int        real_num_rx_queues;
......
1967    /*
1968     * Cache lines mostly used on transmit path
1969     */
1970        struct netdev_queue *_tx ____cacheline_aligned_in_smp;
1971        unsigned int        num_tx_queues;
1972        unsigned int        real_num_tx_queues;
1973        struct Qdisc        *qdisc;
1974    #ifdef CONFIG_NET_SCHED
1975        DECLARE_HASHTABLE   (qdisc_hash, 4);
1976    #endif
1977        unsigned int        tx_queue_len;
1978        spinlock_t      tx_global_lock;
1979        int         watchdog_timeo;
.....
2061        struct phy_device   *phydev;
.....
2069    };
```

下面介绍一些关键的成员变量，如下：

第 1784 行：name 是网络设备的名字。

第 1791 行：mem_end 是共享内存结束地址。

第 1792 行：mem_start 是共享内存起始地址。

第 1793 行：base_addr 是网络设备 I/O 地址。

第 1794 行：irq 是网络设备的中断号。

第 1804 行：dev_list 是全局网络设备列表。

第 1805 行：napi_list 是 napi 网络设备的列表入口。

第 1806 行：unreg_list 是注销(unregister)的网络设备列表入口。

第 1807 行：close_list 是关闭的网络设备列表入口。

第 1841 行：netdev_ops 是网络设备的操作集函数，包含了一系列的网络设备操作回调函数，类似字符设备中的 file_operations，稍后会讲解 netdev_ops 结构体。

第 1842 行：ethtool_ops 是网络管理工具相关函数集，用户空间网络管理工具会调用此结构体中的相关函数获取网卡状态或者配置网卡。

第 1858 行：header_ops 是头部的相关操作函数集，比如创建、解析、缓冲等。

第 1860 行：flags 是网络接口标志，标志类型定义在 include/uapi/linux/if.h 文件中，为一个枚举类型，内容如下：

示例代码 52.3.1.2 网络标志类型

```
81  enum net_device_flags {
82  /* for compatibility with glibc net/if.h */
83  #if __UAPI_DEF_IF_NET_DEVICE_FLAGS
84      IFF_UP              = 1<<0,  /* sysfs */
85      IFF_BROADCAST       = 1<<1,  /* volatile */
86      IFF_DEBUG           = 1<<2,  /* sysfs */
87      IFF_LOOPBACK        = 1<<3,  /* volatile */
88      IFF_POINTOPOINT     = 1<<4,  /* volatile */
89      IFF_NOTRAILERS      = 1<<5,  /* sysfs */
90      IFF_RUNNING         = 1<<6,  /* volatile */
91      IFF_NOARP           = 1<<7,  /* sysfs */
92      IFF_PROMISC         = 1<<8,  /* sysfs */
93      IFF_ALLMULTI        = 1<<9,  /* sysfs */
94      IFF_MASTER          = 1<<10, /* volatile */
95      IFF_SLAVE           = 1<<11, /* volatile */
96      IFF_MULTICAST       = 1<<12, /* sysfs */
97      IFF_PORTSEL         = 1<<13, /* sysfs */
98      IFF_AUTOMEDIA       = 1<<14, /* sysfs */
99      IFF_DYNAMIC         = 1<<15, /* sysfs */
100 #endif /* __UAPI_DEF_IF_NET_DEVICE_FLAGS */
101 #if __UAPI_DEF_IF_NET_DEVICE_FLAGS_LOWER_UP_DORMANT_ECHO
102     IFF_LOWER_UP        = 1<<16, /* volatile */
103     IFF_DORMANT         = 1<<17, /* volatile */
104     IFF_ECHO            = 1<<18, /* volatile */
105 #endif /* __UAPI_DEF_IF_NET_DEVICE_FLAGS_LOWER_UP_DORMANT_ECHO */
106 };
```

继续回到示例代码 52.3.1.1 中，接着看 net_device 结构体。

第 1869 行：if_port 指定接口的端口类型，如果设备支持多端口的话就通过 if_port 来指定所使用的端口类型。可选的端口类型定义在 include/uapi/linux/netdevice.h 中，为一个枚举类型，如下所示：

示例代码 52.3.1.3 端口类型

```
49 enum {
50      IF_PORT_UNKNOWN = 0,
51      IF_PORT_10BASE2,
52      IF_PORT_10BASET,
53      IF_PORT_AUI,
54      IF_PORT_100BASET,
```

```
55      IF_PORT_100BASETX,
56      IF_PORT_100BASEFX
57  };
```

第 1870 行：dma 是网络设备所使用的 DMA 通道，不是所有的设备都会用到 DMA。

第 1877 行：mtu 是网络最大传输单元，为 1500。

第 1880 行：type 用于指定 ARP 模块的类型，以太网的 ARP 接口为 ARPHRD_ETHER，Linux 内核所支持的 ARP 协议定义在 include/uapi/linux/if_arp.h 中，大家自行查阅。

第 1888 行：perm_addr 是永久的硬件地址，如果某个网卡设备有永久的硬件地址，那么就会填充 perm_addr。

第 1890 行：addr_len 是硬件地址长度。

第 1942 行：dev_addr 也是硬件地址，是当前分配的 MAC 地址，可以通过软件修改。

第 1944 行：_rx 是接收队列。

第 1945 行：num_rx_queues 是接收队列数量，在调用 register_netdev 注册网络设备的时候会分配指定数量的接收队列。

第 1946 行：real_num_rx_queues 是当前活动的队列数量。

第 1970 行：_tx 是发送队列。

第 1971 行：num_tx_queues 是发送队列数量，通过 alloc_netdev_mq 函数分配指定数量的发送队列。

第 1972 行：real_num_tx_queues 是当前有效的发送队列数量。

第 2061 行：phydev 是对应的 PHY 设备。

1、申请 net_device

编写网络驱动的时候首先要申请 net_device，使用 alloc_netdev 函数来申请 net_device，这是一个宏，宏定义如下：

```
#define alloc_netdev(sizeof_priv, name, name_assign_type, setup) \
        alloc_netdev_mqs(sizeof_priv, name, name_assign_type, setup, 1, 1)
```

可以看出 alloc_netdev 的本质是 alloc_netdev_mqs 函数，此函数原型如下

```
struct net_device *alloc_netdev_mqs(int sizeof_priv, const char *name,
                    unsigned char name_assign_type,
                    void (*setup)(struct net_device *),
                    unsigned int txqs, unsigned int rxqs)
```

函数参数和返回值含义如下：

sizeof_priv：私有数据块大小。

name：设备名字。

name_assign_type：设备名字的来源。

setup：回调函数，初始化设备的设备后调用此函数。

txqs：分配的发送队列数量。

rxqs：分配的接收队列数量。

返回值：如果申请成功的话就返回申请到的 net_device 指针，失败的话就返回 NULL。

事实上网络设备有多种，大家不要以为就只有以太网一种。Linux 内核支持的网络接口有很多，比如光纤分布式数据接口(FDDI)、以太网设备(Ethernet)、红外数据接口(InDA)、高性能并行接口(HPPI)、CAN 网络等。内核针对不同的网络设备在 alloc_netdev 的基础上提供了一层封装，比如我们本章讲解的以太网，针对以太网封装的 net_device 申请函数是 alloc_etherdev 和 alloc_etherdev_mq，这也是一个宏，内容如下：

```
#define alloc_etherdev(sizeof_priv) alloc_etherdev_mq(sizeof_priv, 1)
#define alloc_etherdev_mq(sizeof_priv, count) alloc_etherdev_mqs(sizeof_priv, count, count)
```

可以看出，alloc_etherdev 最终依靠的是 alloc_etherdev_mqs 函数，此函数就是对 alloc_netdev_mqs 的简单封装，函数内容如下：

示例代码 52.3.1.4 alloc_etherdev_mqs 函数

```
411 struct net_device *alloc_etherdev_mqs(int sizeof_priv,
                                        unsigned int txqs,
412                                     unsigned int rxqs)
413 {
414     return alloc_netdev_mqs(sizeof_priv, "eth%d", NET_NAME_UNKNOWN,
415                 ether_setup, txqs, rxqs);
416 }
```

第 414 行调用 alloc_netdev_mqs 来申请 net_device，注意这里设置网卡的名字为“eth%d”，这是格式化字符串，大家进入开发板的 linux 系统以后看到的“eth0”、“eth1”这样的网卡名字就是从这里来的。同样的，这里设置了以太网的 setup 函数为 ether_setup，不同的网络设备其 setup 函数不同，比如 CAN 网络里面 setup 函数就是 can_setup。

ether_setup 函数会对 net_device 做初步的初始化，函数内容如下所示：

示例代码 52.3.1.5 ether_setup 函数

```
377 void ether_setup(struct net_device *dev)
378 {
379     dev->header_ops      = &eth_header_ops;
380     dev->type            = ARPHRD_ETHER;
381     dev->hard_header_len = ETH_HLEN;
382     dev->min_header_len  = ETH_HLEN;
383     dev->mtu             = ETH_DATA_LEN;
384     dev->min_mtu         = ETH_MIN_MTU;
385     dev->max_mtu         = ETH_DATA_LEN;
386     dev->addr_len        = ETH_ALEN;
387     dev->tx_queue_len    = DEFAULT_TX_QUEUE_LEN;
388     dev->flags           = IFF_BROADCAST|IFF_MULTICAST;
389     dev->priv_flags      |= IFF_TX_SKB_SHARING;
390
391     eth_broadcast_addr(dev->broadcast);
392
393 }
```

关于 net_device 的申请就讲解到这里，对于网络设备而言，使用 alloc_etherdev 或 alloc_etherdev_mqs 来申请 net_device。ST 官方编写的网络驱动就是采用 alloc_etherdev_mqs 来申请 net_device。

2、删除 net_device

当我们注销网络驱动的时候需要释放掉前面已经申请到的 net_device，释放函数为 free_netdev，函数原型如下：

```
void free_netdev(struct net_device *dev)
```

函数参数和返回值含义如下：

dev：要释放掉的 net_device 指针。

返回值：无。

3、注册 net_device

net_device 申请并初始化完成以后就需要向内核注册 net_device，要用到函数 register_netdev，函数原型如下：

```
int register_netdev(struct net_device *dev)
```

函数参数和返回值含义如下：

dev：要注册的 net_device 指针。

返回值：0 注册成功，负值 注册失败。

3、注销 net_device

既然有注册，那么必然有注销，注销 net_device 使用函数 unregister_netdev，函数原型如下：

```
void unregister_netdev(struct net_device *dev)
```

函数参数和返回值含义如下：

dev：要注销的 net_device 指针。

返回值：无。

52.3.2 net_device_ops 结构体

net_device 有个非常重要的成员变量：netdev_ops，为 net_device_ops 结构体指针类型，这就是网络设备的操作集。net_device_ops 结构体定义在 include/linux/netdevice.h 文件中，net_device_ops 结构体里面都是一些以“ndo_”开头的函数，这些函数就需要网络驱动编写人员去实现，不需要全部都实现，根据实际驱动情况实现其中一部分即可。结构体内容如下所示(结构体成员变量比较多，这里有缩减)：

示例代码 52.3.2.1 net_device_ops 结构体内容

```
1249  struct net_device_ops {
1250      int     (*ndo_init)(struct net_device *dev);
1251      void    (*ndo_uninit)(struct net_device *dev);
1252      int     (*ndo_open)(struct net_device *dev);
1253      int     (*ndo_stop)(struct net_device *dev);
1254      netdev_tx_t     (*ndo_start_xmit)(struct sk_buff *skb,
1255                                        struct net_device *dev);
1256      netdev_features_t  (*ndo_features_check)(struct sk_buff *skb,
1257                                        struct net_device *dev,
1258                                        netdev_features_t features);
1259      u16     (*ndo_select_queue)(struct net_device *dev,
1260                                  struct sk_buff *skb,
1261                                  struct net_device *sb_dev);
1262      void    (*ndo_change_rx_flags)(struct net_device *dev,
1263                                     int flags);
1264      void    (*ndo_set_rx_mode)(struct net_device *dev);
1265      int     (*ndo_set_mac_address)(struct net_device *dev,
1266                                     void *addr);
1267      int     (*ndo_validate_addr)(struct net_device *dev);
```

```
1268        int     (*ndo_do_ioctl)(struct net_device *dev,
1269                                 struct ifreq *ifr, int cmd);
1270        int     (*ndo_set_config)(struct net_device *dev,
1271                                  struct ifmap *map);
1272        int     (*ndo_change_mtu)(struct net_device *dev,
1273                                  int new_mtu);
1274        int     (*ndo_neigh_setup)(struct net_device *dev,
1275                                   struct neigh_parms *);
1276        void    (*ndo_tx_timeout) (struct net_device *dev);
......
1290    #ifdef CONFIG_NET_POLL_CONTROLLER
1291        void    (*ndo_poll_controller)(struct net_device *dev);
1292        int     (*ndo_netpoll_setup)(struct net_device *dev,
1293                                     struct netpoll_info *info);
1294        void    (*ndo_netpoll_cleanup)(struct net_device *dev);
1295    #endif
......
1368    int         (*ndo_set_features)(struct net_device *dev,
1369                                    netdev_features_t features);
......
1445    };
```

第 1250 行：ndo_init 函数，当第一次注册网络设备的时候此函数会执行，设备可以在此函数中做一些需要推后初始化的内容，不过一般驱动中不使用此函数，虚拟网络设备可能会使用。

第 1251 行：ndo_uninit 函数，卸载网络设备的时候此函数会执行。

第 1252 行：ndo_open 函数，打开网络设备的时候此函数会执行，网络驱动程序需要实现此函数，非常重要！以 STM32MP1 系列 SoC 网络驱动为例，会在此函数中做如下工作：

- 初始化接收缓冲区。
- 如果使用 NAPI 的话要使能 NAPI 模块，通过 napi_enable 函数来使能。
- STM32MP1 MAC 控制器硬件相关初始化。
- 开启 PHY。
- 调用 netif_tx_start_all_queues 来使能传输队列，也可能调用 netif_start_queue 函数。
- ……

第 1253 行：ndo_stop 函数，关闭网络设备的时候此函数会执行，网络驱动程序也需要实现此函数。以 STM32MP1 系列 SoC 网络驱动为例，会在此函数中做如下工作：

- 停止 PHY。
- 停止 NAPI 功能。
- 停止发送功能。
- 关闭 MAC。
- 断开 PHY 连接。
- 释放数据缓冲区。
- ……

第 1254 行：ndo_start_xmit 函数，当需要发送数据的时候此函数就会执行，此函数有一个参数为 sk_buff 结构体指针，sk_buff 结构体在 Linux 的网络驱动中非常重要，sk_buff 保存了上

层传递给网络驱动层的数据。也就是说，要发送出去的数据都存在了 sk_buff 中，关于 sk_buff 稍后会做详细的讲解。如果发送成功的话此函数返回 NETDEV_TX_OK，如果发送失败了就返回 NETDEV_TX_BUSY，如果发送失败了我们就需要停止队列。

第 1259 行：do_select_queue 函数，当设备支持多传输队列的时候选择使用哪个队列。

第 1264 行：ndo_set_rx_mode 函数，此函数用于改变地址过滤列表，根据 net_device 的 flags 成员变量来设置 SoC 的网络外设寄存器。比如 flags 可能为 IFF_PROMISC、IFF_ALLMULTI 或 IFF_MULTICAST，分别表示混杂模式、单播模式或多播模式。

第 1265 行：ndo_set_mac_address 函数，此函数用于修改网卡的 MAC 地址，设置 net_device 的 dev_addr 成员变量，并且将 MAC 地址写入到网络外设的硬件寄存器中。

第 1267 行：ndo_validate_addr 函数，验证 MAC 地址是否合法，也即是验证 net_device 的 dev_addr 中的 MAC 地址是否合法，直接调用 is_valid_ether_addr 函数。

第 1268 行：ndo_do_ioctl 函数，用户程序调用 ioctl 的时候此函数就会执行，比如 PHY 芯片相关的命令操作，一般会直接调用 phy_mii_ioctl 函数。

第 1272 行：ndo_change_mtu 函数，更改 MTU 大小。

第 1276 行：ndo_tx_timeout 函数，当发送超时的时候产生会执行，一般都是网络出问题了导致发送超时。一般可能会重启 MAC 和 PHY，重新开始数据发送等。

第 1291 行：ndo_poll_controller 函数，使用查询方式来处理网卡数据的收发。

第 1368 行：ndo_set_features 函数，修改 net_device 的 features 属性，设置相应的硬件属性。

52.3.3 sk_buff 结构体

网络是分层的，对于应用层而言不用关心具体的底层是如何工作的，只需要按照协议将要发送或接收的数据打包好即可。打包好以后都通过 dev_queue_xmit 函数将数据发送出去，接收数据的话使用 netif_rx 函数即可，我们依次来看一下这两个函数。

1、dev_queue_xmit 函数

此函数用于将网络数据发送出去，函数定义在 include/linux/netdevice.h 中，函数原型如下：

```
int dev_queue_xmit(struct sk_buff       *skb)
```

函数参数和返回值含义如下：

skb：要发送的数据，这是一个 sk_buff 结构体指针，sk_buff 是 Linux 网络驱动中一个非常重要的结构体，网络数据就是以 sk_buff 保存的，各个协议层在 sk_buff 中添加自己的协议头，最终由底层驱动将 sk_buff 中的数据发送出去。网络数据的接收过程恰好相反，网络底层驱动将接收到的原始数据打包成 sk_buff，然后发送给上层协议，上层会取掉相应的头部，然后将最终的数据发送给用户。

返回值：0，发送成功；负值，发送失败。

dev_queue_xmit 函数太长，这里就不详细的分析了，dev_queue_xmit 函数最终是通过 net_device_ops 操作集里面的 ndo_start_xmit 函数来完成最终发送了，ndo_start_xmit 就是网络驱动编写人员去实现的，整个路程如图 52.3.3.1 所示：

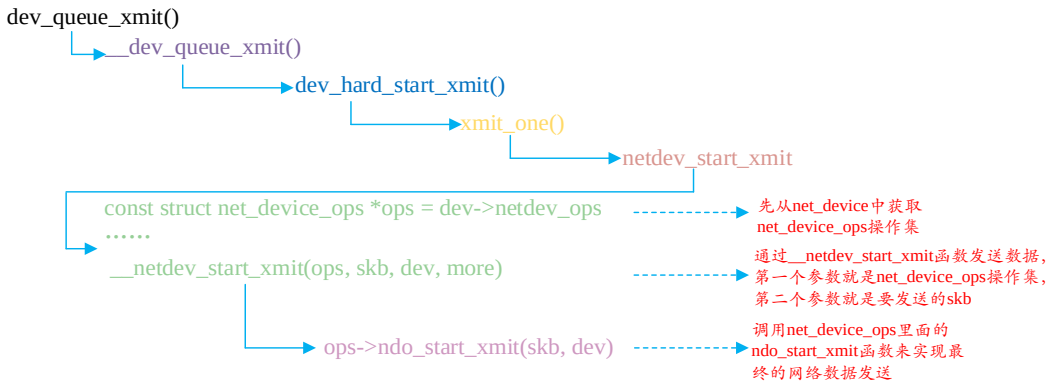


图 52.3.3.1 dev_queue_xmit 执行流程

2、netif_rx 函数

上层接收数据的话使用 netif_rx 函数，但是最原始的网络数据一般是通过轮询、中断或 NAPI 的方式来接收。netif_rx 函数定义在 net/core/dev.c 中，函数原型如下：

```
int netif_rx(struct sk_buff *skb)
```

函数参数和返回值含义如下：

skb：保存接收数据的 sk_buff。

返回值：NET_RX_SUCCESS，成功；NET_RX_DROP，数据包丢弃。

我们重点来看一下 sk_buff 这个结构体，sk_buff 是 Linux 网络重要的数据结构，用于管理接收或发送数据包，sk_buff 结构体定义在 include/linux/skbuff.h 中，结构体内容如下(由于结构体比较大，为了缩小篇幅只列出部分重要的内容)：

示例代码 52.3.3.1 sk_buff 结构体

```
685 struct sk_buff {
686     union {
687         struct {
688             /* These two members must be first. */
689             struct sk_buff      *next;
690             struct sk_buff      *prev;
691 
692             union {
693                 struct net_device   *dev;
694                 /* Some protocols might use this space to store
695                  * information, while device pointer would be NULL.
696                  * UDP receive path is one user.
697                  */
698                 unsigned long       dev_scratch;
699             };
700         };
701         struct rb_node      rbnode;
702         struct list_head    list;
703     };
704 
705     union {
```

```
706         struct sock     *sk;
707         int             ip_defrag_offset;
708     };
709
710     union {
711         ktime_t     tstamp;
712         u64     skb_mstamp_ns; /* earliest departure time */
713     };
714     /*
715      * This is the control buffer. It is free to use for every
716      * layer. Please put your private variables there. If you
717      * want to keep them across layers you have to do a skb_clone()
718      * first. This is owned by whoever has the skb queued ATM.
719      */
720     char            cb[48] __aligned(8);
721
722     union {
723         struct {
724             unsigned long   _skb_refdst;
725             void        (*destructor)(struct sk_buff *skb);
726         };
727         struct list_head    tcp_tsorted_anchor;
728     };
.....
733     unsigned int        len,
734                         data_len;
735     __u16               mac_len,
736                         hdr_len;
......
868     __be16              protocol;
869     __u16               transport_header;
870     __u16               network_header;
871     __u16               mac_header;
872
873     /* private: */
874     __u32               headers_end[0];
875     /* public: */
876
877     /* These elements must be at the end, see alloc_skb() for
           details.  */
878     sk_buff_data_t      tail;
879     sk_buff_data_t      end;
880     unsigned char       *head,
```

```
881                     *data;
882     unsigned int    truesize;
883     refcount_t      users;
884
885 #ifdef CONFIG_SKB_EXTENSIONS
886     /* only useable after checking ->active_extensions != 0 */
887     struct skb_ext      *extensions;
888 #endif
889 };
```

第 687~700 行：next 和 prev 分别指向下一个和前一个 sk_buff，构成一个双向链表。

第 693 行：dev 表示当前 sk_buff 从哪个设备接收到或者发出的。

第 706 行：sk 表示当前 sk_buff 所属的 Socket。

第 711 行：tstamp 表示数据包接收时或准备发送时的时间戳。

第 720 行：cb 为控制缓冲区，不管哪个层都可以自由使用此缓冲区，用于放置私有数据。

第 725 行：destructor 函数，当释放缓冲区的时候可以在此函数里面完成某些动作。

第 733~734 行：len 为实际的数据长度，包括主缓冲区中数据长度和分片中的数据长度。data_len 为数据长度，只计算分片中数据的长度。

第 735~736 行：mac_len 为连接层头部长度，也就是 MAC 头的长度。

第 868 行：protocol 协议。

第 869 行：transport_header 为传输层头部。

第 870 行：network_header 为网络层头部

第 871 行：mac_header 为链接层头部。

第 878 行：tail 指向实际数据的尾部。

第 879 行：end 指向缓冲区的尾部。

第 880 行：head 指向缓冲区的头部，data 指向实际数据的头部。data 和 tail 指向实际数据的头部和尾部，head 和 end 指向缓冲区的头部和尾部。结构如图 52.3.3.1 所示：

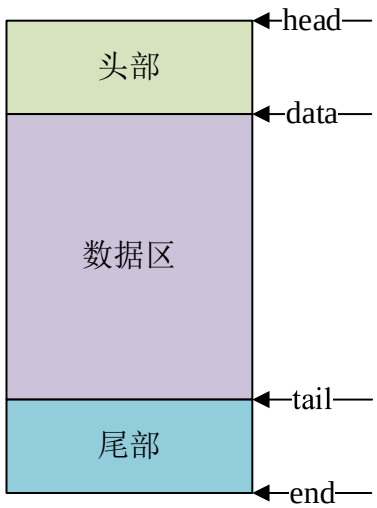


图 52.3.3.1 sk_buff 数据区结构示意图

针对 sk_buff 内核提供了一系列的操作与管理函数，我们简单看一些常见的 API 函数

1、分配 sk_buff

要使用 sk_buff 必须先分配，首先来看一下 alloc_skb 这个函数，此函数定义在

include/linux/skbuff.h 中，函数原型如下：

```
static inline struct sk_buff *alloc_skb(unsigned int      size,
                                        gfp_t             priority)
```

函数参数和返回值含义如下：

size：要分配的大小，也就是 skb 数据段大小。

priority：为 GFP MASK 宏，比如 GFP_KERNEL、GFP_ATOMIC 等。

返回值：分配成功的话就返回申请到的 sk_buff 首地址，失败的话就返回 NULL。

在网络设备驱动中常常使用 netdev_alloc_skb 来为某个设备申请一个用于接收的 skb_buff，此函数也定义在 include/linux/skbuff.h 中，函数原型如下：

```
static inline struct sk_buff *netdev_alloc_skb(struct net_device   *dev,
                                               unsigned int        length)
```

函数参数和返回值含义如下：

dev：要给哪个设备分配 sk_buff。

length：要分配的大小。

返回值：分配成功的话就返回申请到的 sk_buff 首地址，失败的话就返回 NULL。

2、释放 sk_buff

当使用完成以后就要释放掉 sk_buff，释放函数可以使用 kfree_skb，函数定义在 include/linux/skbuff.c 中，函数原型如下：

```
void kfree_skb(struct sk_buff *skb)
```

函数参数和返回值含义如下：

skb：要释放的 sk_buff。

返回值：无。

对于网络设备而言最好使用如下所示释放函数：

```
void dev_kfree_skb (struct sk_buff      *skb)
```

函数只要一个参数 skb，就是要释放的 sk_buff。

3、skb_put、skb_push、sbk_pull 和 skb_reserve

这四个函数用于变更 sk_buff，先来看一下 skb_put 函数，此函数用于在尾部扩展 skb_buff 的数据区，也就将 skb_buff 的 tail 后移 n 个字节，从而导致 skb_buff 的 len 增加 n 个字节，原型如下：

```
unsigned char *skb_put(struct sk_buff  *skb, unsigned int   len)
```

函数参数和返回值含义如下：

skb：要操作的 sk_buff。

len：要增加多少个字节。

返回值：扩展出来的那一段数据区首地址。

skb_put 操作之前和操作之后的数据区如图 52.3.3.2 所示：

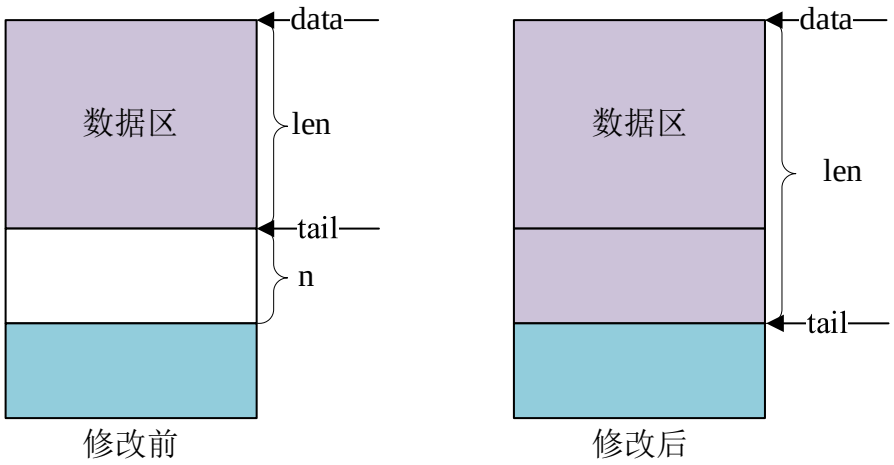


图 52.3.3.2 skb_put 函数操作前后对比

skb_push 函数用于在头部扩展 skb_buff 的数据区，函数原型如下所示：

```
unsigned char *skb_push(struct sk_buff      *skb, unsigned int   len)
```

函数参数和返回值含义如下：

skb：要操作的 sk_buff。

len：要增加多少个字节。

返回值：扩展完成以后新的数据区首地址。

skb_push 操作之前和操作之后的数据区如图 52.3.3.3 所示：

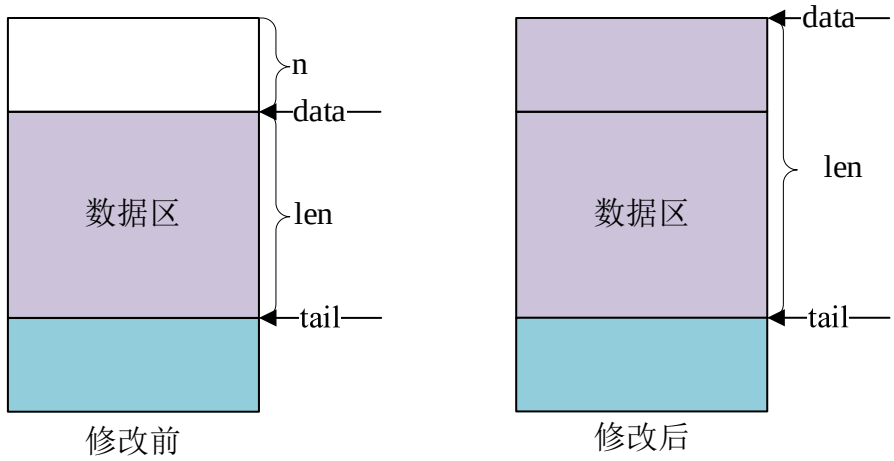


图 52.3.3.3 skb_push 函数操作前后对比

sbk_pull 函数用于从 sk_buff 的数据区起始位置删除数据，函数原型如下所示：

```
unsigned char *skb_pull(struct sk_buff      *skb, unsigned int   len)
```

函数参数和返回值含义如下：

skb：要操作的 sk_buff。

len：要删除的字节数。

返回值：删除以后新的数据区首地址。

skb_pull 操作之前和操作之后的数据区如 52.3.3.4 所示：

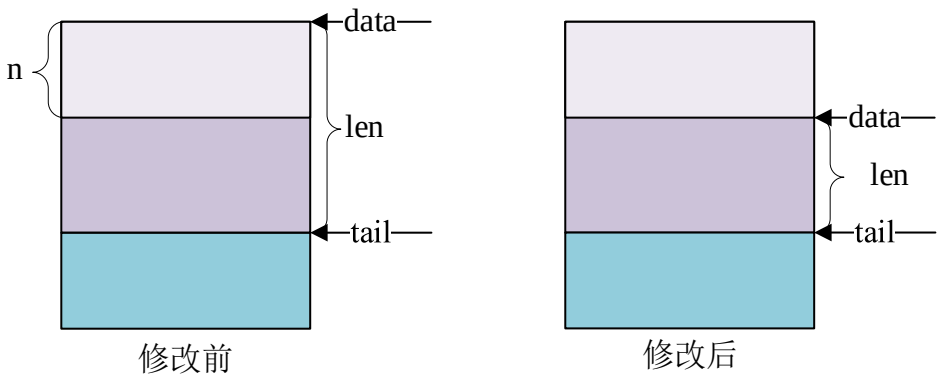


图 52.3.3.4 skb_pull 函数操作前后对比

sbk_reserve 函数用于调整缓冲区的头部大小，方法很简单，将 skb_buff 的 data 和 tail 同时后移 n 个字节即可，函数原型如下所示：

```
static inline void skb_reserve(struct sk_buff  *skb, int      len)
```

函数参数和返回值含义如下：

skb：要操作的 sk_buff。

len：要增加的缓冲区头部大小。

返回值：无。

52.3.4 网络 NAPI 处理机制

如果玩过单片机的话应该都知道，像 IIC、SPI、网络等这些通信接口，接收数据有两种方法：轮询或中断。Linux 里面的网络数据接收也有轮询和中断两种，中断的好处就是响应快，数据量小的时候处理及时，速度快，但是一旦当数据量大，而且都是短帧的时候会导致中断频繁发生，消耗大量的 CPU 处理时间在中断自身处理上。轮询恰好相反，响应没有中断及时，但是在处理大量数据的时候不需要消耗过多的 CPU 处理时间。Linux 在这两个处理方式的基础上提出了另外一种网络数据接收的处理方法：NAPI(New API)，NAPI 是一种高效的网络处理技术。NAPI 的核心思想就是不全部采用中断来读取网络数据，而是采用中断来唤醒数据接收服务程序，在接收服务程序中采用 POLL 的方法来轮询处理数据。这种方法的好处就是可以提高短数据包的接收效率，减少中断处理的时间。目前 NAPI 已经在 Linux 的网络驱动中得到了大量的应用，ST 官方编写的网络驱动都是采用的 NAPI 机制。

关于 NAPI 详细的处理过程本章节不讨论，本章节就简单讲解一下如何在驱动中使用 NAPI，Linux 内核使用结构体 napi_struct 表示 NAPI，在使用 NAPI 之前要先初始化一个 napi_struct 实例。

1、初始化 NAPI

首先要初始化一个 napi_struct 实例，使用 netif_napi_add 函数，此函数定义在 net/core/dev.c 中，函数原型如下：

```
void netif_napi_add(struct net_device   *dev,
                    struct napi_struct  *napi,
                    int (*poll)(struct napi_struct *, int),
                    int                 weight)
```

函数参数和返回值含义如下：

dev：每个 NAPI 必须关联一个网络设备，此参数指定 NAPI 要关联的网络设备。

napi：要初始化的 NAPI 实例。

poll：NAPI 所使用的轮询函数，非常重要，一般在此轮询函数中完成网络数据接收的工作。

weight：NAPI 默认权重(weight)，一般为 NAPI_POLL_WEIGHT。

返回值：无。

2、删除 NAPI

如果要删除 NAPI，使用 netif_napi_del 函数即可，函数原型如下：

```
void netif_napi_del(struct napi_struct  *napi)
```

函数参数和返回值含义如下：

napi：要删除的 NAPI。

返回值：无。

3、使能 NAPI

初始化完 NAPI 以后，必须使能才能使用，使用函数 napi_enable，函数原型如下：

```
void napi_enable(struct napi_struct      *n)
```

函数参数和返回值含义如下：

n：要使能的 NAPI。

返回值：无。

4、关闭 NAPI

关闭 NAPI 使用 napi_disable 函数即可，函数原型如下：

```
void napi_disable(struct napi_struct *n)
```

函数参数和返回值含义如下：

n：要关闭的 NAPI。

返回值：无。

5、检查 NAPI 是否可以进行调度

使用 napi_schedule_prep 函数检查 NAPI 是否可以进行调度，函数原型如下：

```
inline bool napi_schedule_prep(struct napi_struct *n)
```

函数参数和返回值含义如下：

n：要检查的 NAPI。

返回值：如果可以调度就返回真，如果不可调度就返回假。

6、NAPI 调度

如果可以调度的话就进行调度，使用__napi_schedule 函数完成 NAPI 调度，函数原型如下：

```
void __napi_schedule(struct napi_struct *n)
```

函数参数和返回值含义如下：

n：要调度的 NAPI。

返回值：无。

我们也可以使用 napi_schedule 函数来一次完成 napi_schedule_prep 和__napi_schedule 这两个函数的工作，napi_schedule 函数内容如下所示：

示例代码 52.3.4.1 napi_schedule 函数代码

```
442 static inline void napi_schedule(struct napi_struct *n)
443 {
444     if (napi_schedule_prep(n))
445         __napi_schedule(n);
```

```
446 }
```

从示例代码 52.3.4.1 可以看出，napi_schedule 函数就是对 napi_schedule_prep 和 __napi_schedule 的简单封装，一次完成判断和调度。

7、NAPI 处理完成

NAPI 处理完成以后需要调用 napi_complete 函数来标记 NAPI 处理完成，函数原型如下：

```
inline void napi_complete(struct napi_struct *n)
```

函数参数和返回值含义如下：

n：处理完成的 NAPI。

返回值：无。

52.4 原理图介绍

正点原子的 STM32MP1 开发板网络相关的硬件有两个版本分别为：V1.2 版本以前的核心板和 V1.3 版本以后的核心板。

1、V1.2 版本以前的核心板

V1.2 版本以前的核心板的原理图，如下所示：

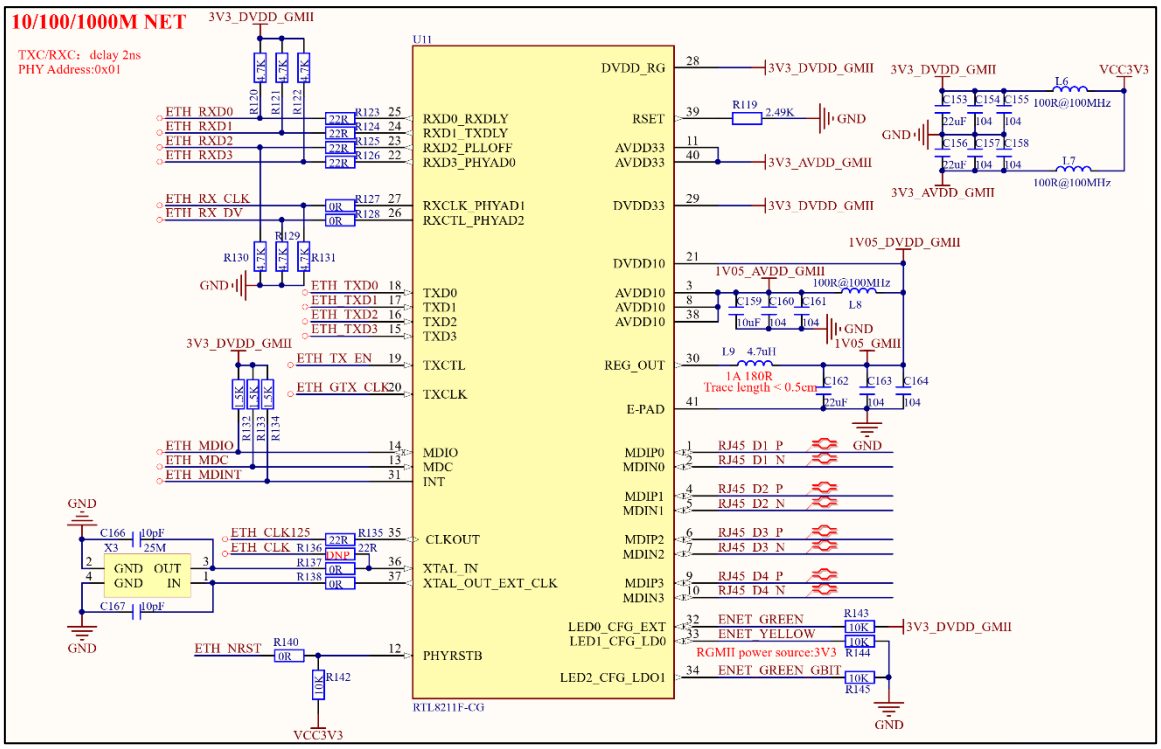


图 52.4.1 V1.2 版本以前的核心板的原理图

在 52.2.2 小节已经知道了 PHY 地址是由 RXCTL、RXC 和 RXD3 控制的，从图 52.4.1 中知道了这三个引脚为 0、0 和 1，PHY 的地址为 0x1，RGMII 的电平为 3.3V。

2、V1.3 版本以后的核心板

V1.3 版本以后的核心板原理图，如下所示：

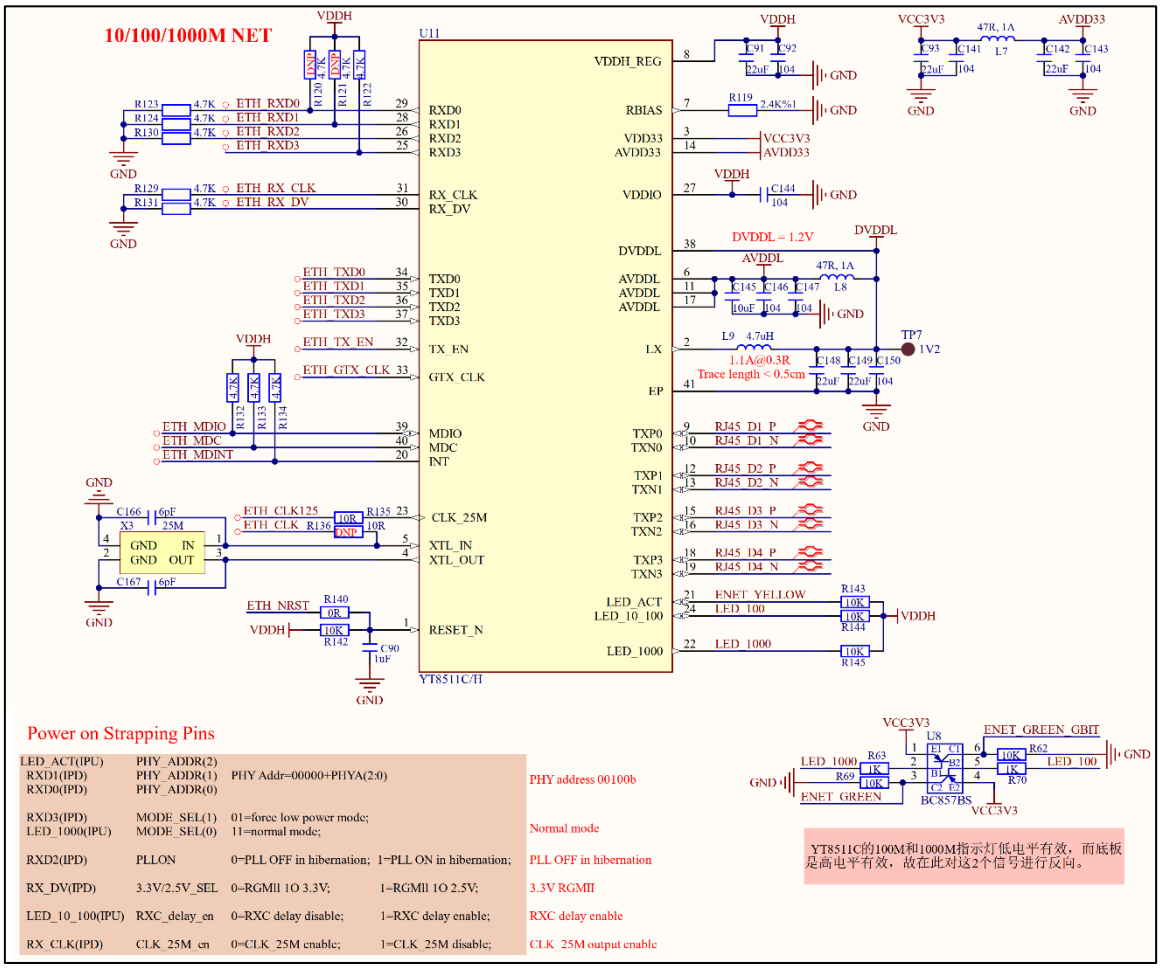


图 52.4.2 V1.3 版本的核心板原理图

52.5 STM32MP1 网络驱动简介

52.5.1 STM32MP1 网络外设设备树

上一小节我们对 Linux 的网络驱动框架进行了一个简单的介绍，本节我们就来简单分析一下 STM32MP1 的网络驱动源码。我们肯定是先分析设备树，STM32MP1 系列 SoC 网络绑定文档为 Documentation/devicetree/bindings/net/stm32-dwmac.txt，此绑定文档描述了 STM32MP1 系列 SoC 网络设备树节点的要求。此外还有一份文档 Documentation/devicetree/bindings/net/ethernet.txt，该文档描述了网络设备节点的一些通用属性。

在内核源码目录 arch/arm/boot/dts/stm32mp152.dtsi 设备树文件中，定义了网络设备节点，如下所示：

示例代码 52.5.1.1 ethernet0 节点

```
1604    ethernet0: ethernet@5800a000 {
1605        compatible = "st,stm32mp1-dwmac", "snps,dwmac-4.20a";
1606        reg = <0x5800a000 0x2000>;
1607        reg-names = "stmmaceth";
1608        interrupts-extended = <&intc GIC_SPI 61 IRQ_TYPE_LEVEL_HIGH>,
1609                              <&exti 70 IRQ_TYPE_LEVEL_HIGH>;
```

```
1610        interrupt-names = "macirq",
1611                          "eth_wake_irq";
1612        clock-names = "stmmaceth",
1613                      "mac-clk-tx",
1614                      "mac-clk-rx",
1615                      "ethstp";
1616        clocks =  <&rcc ETHMAC>,
1617                  <&rcc ETHTX>,
1618                  <&rcc ETHRX>,
1619                  <&rcc ETHSTP>;
1620        st,syscon = <&syscfg 0x4>;
1621        snps,mixed-burst;
1622        snps,pbl = <2>;
1623        snps,en-tx-lpi-clockgating;
1624        snps,axi-config = <&stmmac_axi_config_0>;
1625        snps,tso;
1626        power-domains = <&pd_core>;
1627        status = "disabled";
1628    };
```

第 1605 行，compatible 属性有两个分别为：st,stm32mp1-dwmac 和 snps,dwmac-4.20a。在内核源码目录里搜索这两个属性值，我们就能找 drivers/net/ethernet/stmicro/stmmac/dwmac-stm32.c 文件。dwmac-stm32.c 就是 stm32mp1 的驱动源码，后面在分析。

第 1606 行，reg 属性值表示网络的物理地址大小。

第 1621 行，snps,mixed-burst 属性，表示 DMA 使用混合突发模式。

第 1622 行，snps,pbl 属性值为 2，表示突发长度为 2。

第 1623 行，snps,en-tx-lpi-clockgating 属性，在 TX 低功耗模式下，使能 MAC TX 门控时钟。

第 1625 行，snps,tso 属性，表示启动 tso 功能，tso 全称为 TCP Segment Offload，也就是利用网卡的少量处理能力，降低 CPU 发送数据包负载的技术，需要网卡硬件及驱动支持。

示例代码 52.5.1.1 是 ST 官方编写的，我们不需要去修改，此示例代码 52.5.1.1 是不能正常工作，还需要根据实际情况添加或者修改一些属性。打开 stm32mp157d-atk.dtsi，找到如下内容：

示例代码 52.5.1.2 stm32mp157d-atk.dtsi 中的网络设备

```
1   &ethernet0 {
2       status = "okay";
3       pinctrl-0 = <&ethernet0_rgmii_pins_a>;
4       pinctrl-1 = <&ethernet0_rgmii_pins_sleep_a>;
5       pinctrl-names = "default", "sleep";
6       phy-mode = "rgmii-id";
7       max-speed = <1000>;
8       phy-handle = <&phy0>;
9
10      mdio0 {
11          #address-cells = <1>;
```

```
12          #size-cells = <0>;
13          compatible = "snps,dwmac-mdio";
14          phy0: ethernet-phy@0 {
15              reg = <0>;
16          };
17      };
18  };
```

从示例代码 52.5.1.2 中可以看到，ethernet0 就是我们要追加的内容，接下来我们来看看这些追加属性表示什么意思。

status：这个属性就不用说了，把 status 属性值修改为“okay”，使能网络。

pinctrl-XXX：这些属性不用说了，就是定义了网络的引脚复用功能。在 stm32mp15-pinctrl.dtsi 里定义了 ethernet0_rgmii_pins_a 和 ethernet0_rgmii_pins_sleep_a。

phy-mode：该属性值是一个字符串，表示网络所使用的 PHY 接口模式，是 MII、RMII、GMII 还是 RGMII 等，它是一个标准的属性，支持的字符串值有‘mii’、‘gmii’、‘sgmii’、‘rmii’、‘rgmii’、‘rgmii-id’、‘rgmii-rxid’、‘rgmii-txid’、‘xgmii’ 等等。第 6 行中，phy-mode 属性的值等于‘rgmii-id’表示他使用的是 RGMII 接口模式，由 PHY 提供 RX 和 TX 延迟，MAC 不需要添加 RX 和 TX 延时内容。与 RGMII 接口相关的值有‘rgmii’、‘rgmii-id’、‘rgmii-rxid’、‘rgmii-txid’这些，ethernet.txt 文件里面详细讲解了这些属性值的含义，这里就不详细介绍了。

max-speed：PHY 支持的最高速度，比如 10、100 或 1000。

phy- handle：连接到此网络设备的 PHY 芯片句柄。

mdio：第 10~17 行中定义了一个 mdio 子节点，该节点用于描述 MDIO 总线。#address-cell 属性和#size-cells 属性用于描述 mdio 节点的子节点中所定义的 reg 属性，这个 reg 属性其实就表示 PHY 芯片的地址；

52.5.2 STM32MP1 网络驱动源码简析

1、重要的结构体

每个驱动都会自定义自己的结构体，要了解驱动的注册过程，就先从自定义结构体看起。前面我们说了驱动文件为：drivers/net/ethernet/stmicro/stmmac/dwmac-stm32.c，打开 dwmac-stm32.c，找到如下所示内容：

示例代码 52.5.2.1 stm32_dwmac 结构体

```
90  struct stm32_dwmac {
91      struct clk *clk_tx;
92      struct clk *clk_rx;
93      struct clk *clk_eth_ck;
94      struct clk *clk_ethstp;
95      struct clk *syscfg_clk;
96      int eth_clk_sel_reg;
97      int eth_ref_clk_sel_reg;
98      u32 mode_reg;                /* MAC glue-logic mode register */
99      struct regmap *regmap;          /* 网络的基地址 */
100     u32 speed;                      /* 网络的速度 */
101     const struct stm32_ops *ops;    /* 网络的操作集 */
```

```
102     struct device *dev;
103 };
```

示例代码 52.5.2.1 中，主要是第 101 行，ops 成员，表示网络相关的操作集，驱动人员要实现的，ST 官方已经写好了；其它属性成员都是时钟相关的，肯定是通过设备树配置时钟。去了解一下 stm32_ops 结构体代码如下所示：

示例代码 52.5.2.2 stm32_ops 结构体

```
struct stm32_ops {
    int (*set_mode)(struct plat_stmmacenet_data *plat_dat);
    int (*clk_prepare)(struct stm32_dwmac *dwmac, bool prepare);
    int (*suspend)(struct stm32_dwmac *dwmac);
    void (*resume)(struct stm32_dwmac *dwmac);
    int (*parse_data)(struct stm32_dwmac *dwmac,
              struct device *dev);
    u32 syscfg_eth_mask;
};
```

大家都知道单片机要使用网络的时候，需要初始化网络相关的寄存器，stm32_ops 就是根据设备树参数去设置网络相关的寄存器，后面会分析 stm32_ops。

plat_stmmacenet_data 结构体很重要，将网络设备注册到内核的时候需要使用到此结构体。此结构体定义在 include/linux/stmmac.h，plat_stmmacenet_data 结构体原型如下示例代码所示：

示例代码 52.5.2.3 plat_stmmacenet_data 结构体

```
1   struct plat_stmmacenet_data {
2       int bus_id;                         /* 网络的 ID          */
3       int phy_addr;                       /* PHY 地址           */
4       int interface;
5       int phy_interface;                  /* PHY 的接口模式     */
6       struct stmmac_mdio_bus_data *mdio_bus_data;
7       struct device_node *phy_node;       /* PHY 的句柄         */
8       struct device_node *phylink_node;
9       struct device_node *mdio_node;      /* mdio 节点          */
10      struct stmmac_dma_cfg *dma_cfg;
11      int clk_csr;
12      int has_gmac;
13      int enh_desc;
14      int tx_coe;
15      int rx_coe;
16      int bugged_jumbo;
17      int pmt;
18      int force_sf_dma_mode;
19      int force_thresh_dma_mode;
20      int riwt_off;
21      int max_speed;                      /* 支持最大的速率     */
.....
53  };
```

示例代码 52.5.2.3 中的内容比较多，有省略，只列出了我们驱动要用到的一些参数。stm32_dwmac 和 stm32_ops 这两个结构体主要是用作初始化网络，plat_stmmacenet_data 结构体主要用于注册网络设备。

2、stm32_dwmac_probe 函数

STM32MP1 网络驱动主要分两部分：STM32MP1 网络外设 MAC 驱动以及 PHY 芯片驱动，MAC 驱动是 ST 编写的，PHY 芯片有通用驱动文件，有些 PHY 芯片厂商还会针对自己的芯片编写对应的 PHY 驱动。总体来说，SOC 内置网络 MAC+外置 PHY 芯片这种方案我们是不需要编写什么驱动的，基本可以直接使用。但是为了学习，我们还是要简单分析一下具体的网络驱动编写过程。打开 dwmac-stm32.c，找到如下所示内容：

示例代码 52.5.2.4 STM32 系列 SOC 网络平台驱动匹配表

```
526 static const struct of_device_id stm32_dwmac_match[] = {
527     { .compatible = "st,stm32-dwmac", .data = &stm32mcu_dwmac_data},
528     { .compatible = "st,stm32mp1-dwmac", .data =
            &stm32mp1_dwmac_data},
529     { }
530 };
531 MODULE_DEVICE_TABLE(of, stm32_dwmac_match);
532
533 static struct platform_driver stm32_dwmac_driver = {
534     .probe  = stm32_dwmac_probe,
535     .remove = stm32_dwmac_remove,
536     .driver = {
537         .name           = "stm32-dwmac",
538         .pm     = &stm32_dwmac_pm_ops,
539         .of_match_table = stm32_dwmac_match,
540     },
541 };
542 module_platform_driver(stm32_dwmac_driver);
```

第 528 行，匹配表包含“st,stm32mp1-dwmac”，因此设备树和驱动匹配上，当匹配成功以后第 534 行的 stm32_dwmac_probe 函数就会执行，我们简单分析一下 stm32_dwmac_probe 函数，函数内容如下：

示例代码 52.5.2.5 stm32_dwmac_probe 函数

```
360 static int stm32_dwmac_probe(struct platform_device *pdev)
361 {
362     struct plat_stmmacenet_data *plat_dat;
363     struct stmmac_resources stmmac_res;
364     struct stm32_dwmac *dwmac;
365     const struct stm32_ops *data;
366     int ret;
367
368     ret = stmmac_get_platform_resources(pdev, &stmmac_res);
369     if (ret)
370         return ret;
```

```
371
372     plat_dat = stmmac_probe_config_dt(pdev, &stmmac_res.mac);
373     if (IS_ERR(plat_dat))
374         return PTR_ERR(plat_dat);
375
376     dwmac = devm_kzalloc(&pdev->dev, sizeof(*dwmac), GFP_KERNEL);
377     if (!dwmac) {
378         ret = -ENOMEM;
379         goto err_remove_config_dt;
380     }
381
382     data = of_device_get_match_data(&pdev->dev);
383     if (!data) {
384         dev_err(&pdev->dev, "no of match data provided\n");
385         ret = -EINVAL;
386         goto err_remove_config_dt;
387     }
388
389     dwmac->ops = data;
390     dwmac->dev = &pdev->dev;
391
392     ret = stm32_dwmac_parse_data(dwmac, &pdev->dev);
393     if (ret) {
394         dev_err(&pdev->dev, "Unable to parse OF data\n");
395         goto err_remove_config_dt;
396     }
397
398     if (stmmac_res.wol_irq && !dwmac->clk_eth_ck) {
399         ret = stm32_dwmac_wake_init(&pdev->dev, &stmmac_res);
400         if (ret)
401             return ret;
402     }
403
404     plat_dat->bsp_priv = dwmac;
405
406     ret = stm32_dwmac_init(plat_dat);
407     if (ret)
408         goto err_remove_config_dt;
409
410     ret = stmmac_dvr_probe(&pdev->dev, plat_dat, &stmmac_res);
411     if (ret)
412         goto err_clk_disable;
413
```

```
414     return 0;
415
416 err_clk_disable:
417     stm32_dwmac_clk_disable(dwmac);
418 err_remove_config_dt:
419     stmmac_remove_config_dt(pdev, plat_dat);
420
421     return ret;
422 }
```

第 368 行，使用 stmmac_get_platform_resources 函数获取设备树上的资源，保存到 struct stmmac_resources 类型的结构体里。

第 372 行，plat_dat 变量指针类型为 plat_stmmacenet_data 结构体，plat_dat 结构体主要是保存网络设备的参数，根据 plat_dat 的参数将网络设备注册进内核。stmmac_probe_config_dt 函数作用是根据设备树上的属性值去填充 plat_dat 各个成员。stmmac_probe_config_dt 函数内容如下示例代码所示(有省略)：

示例代码 52.5.2.6 stmmac_probe_config_dt 函数

```
395 struct plat_stmmacenet_data *
396 stmmac_probe_config_dt(struct platform_device *pdev,
                           const char **mac)
397 {
398     struct device_node *np = pdev->dev.of_node;
399     struct plat_stmmacenet_data *plat;
400     struct stmmac_dma_cfg *dma_cfg;
401     int rc;
402
403     plat = devm_kzalloc(&pdev->dev, sizeof(*plat), GFP_KERNEL);
404     if (!plat)
405         return ERR_PTR(-ENOMEM);
406
407     *mac = of_get_mac_address(np);
408     if (IS_ERR(*mac)) {
409         if (PTR_ERR(*mac) == -EPROBE_DEFER)
410             return ERR_CAST(*mac);
411
412         *mac = NULL;
413     }
414
415     plat->phy_interface = of_get_phy_mode(np);
416     if (plat->phy_interface < 0)
417         return ERR_PTR(plat->phy_interface);
.....
425     plat->phy_node = of_parse_phandle(np, "phy-handle", 0);
426
```

```
427     /* PHYLINK automatically parses the phy-handle property */
428     plat->phylink_node = np;
429
430     /* Get max speed of operation from device tree */
431     if (of_property_read_u32(np, "max-speed", &plat->max_speed))
432         plat->max_speed = -1;
433
434     plat->bus_id = of_alias_get_id(np, "ethernet");
435     if (plat->bus_id < 0)
436         plat->bus_id = 0;
437
438     /* Default to phy auto-detection */
439     plat->phy_addr = -1;
......
454     rc = stmmac_dt_phy(plat, np, &pdev->dev);
......
607     return plat;
608
609 error_hw_init:
610     clk_disable_unprepare(plat->pclk);
611 error_pclk_get:
612     clk_disable_unprepare(plat->stmmac_clk);
613
614     return ERR_PTR(-EPROBE_DEFER);
615 }
```

stmmac_probe_config_dt 函数会新建一个 plat_dat 指针变量，从设备树获取数据初始化 palt_dat 里的成员，返回 plat_dat 的指针地址。

第 407 行，使用 of_get_mac_address 函数从设备树获取 MAC 地址，这里我们没有定义。

第 415 行，使用 of_get_phy_mode 函数从设备树获取接口模式，获取 phy-mode 属性值。这里属性值为“rgmii-id”。

第 425 行，使用 of_parse_phandle 函数从设备树获取 PHY 句柄。

第 431 行，使用 of_property_read_u32 函数从设备树获取 max-speed 属性值。这里属性值为“1000”，表示我们的网络为 1000M 网络。

第 434 行，获取网络的别名。

第 439 行，设置 plat 结构体里的 phy_addr 成员，phy_addr 设置为“-1”，phy_addr 表示 PHY 的地址。设置为-1 表示自动检测，一个 MDIO 接口最多可以连接 32 个 PHY 芯片，设置为-1 就是自动遍历所有的 PHY 地址，看看哪个地址上有 PHY 芯片，然后将对应的 PHY 地址重新赋值给 phy_addr。

第 454 行，使用 stmmac_dt_phy 函数去获取“mdio”节点中的信息。

我们继续返回示例代码 52.5.2.4 中的进行解释。

第 376 行，给 dwmac 结构体开空间。

第 382 行，获取 of_device_id 里的 data 成员值，获取 ST 官方自定义的 stm32_ops 操作集。

第 389 行，把我们获取的 stm32_ops 地址，赋值到 stm32_dwmac 结构体里的 ops 成员，我

们初始化网络的时候，可以调用 ops 就能操作到网络相关的寄存器。

第 392 行，使用 stm32_dwmac_parse_data 函数主要是负责从设备树里获取时钟和基地址。

第 405 行，调用 stm32_dwmac_init 函数进行，初始化网络设备，函数的原型如下示例代码所示：

示例代码 52.5.2.7 stm32_dwmac_init 函数

```
115 static int stm32_dwmac_init(struct plat_stmmacenet_data *plat_dat)
116 {
117     struct stm32_dwmac *dwmac = plat_dat->bsp_priv;
118     int ret;
119
120     if (dwmac->ops->set_mode) {
121         ret = dwmac->ops->set_mode(plat_dat);
122         if (ret)
123             return ret;
124     }
125
126     ret = clk_prepare_enable(dwmac->clk_tx);
127     if (ret)
128         return ret;
129
130     if (!dwmac->dev->power.is_suspended) {
131         ret = clk_prepare_enable(dwmac->clk_rx);
132         if (ret) {
133             clk_disable_unprepare(dwmac->clk_tx);
134             return ret;
135         }
136     }
137
138     if (dwmac->ops->clk_prepare) {
139         ret = dwmac->ops->clk_prepare(dwmac, true);
140         if (ret) {
141             clk_disable_unprepare(dwmac->clk_rx);
142             clk_disable_unprepare(dwmac->clk_tx);
143         }
144     }
145
146     return ret;
147 }
```

第 121 行，调用我们的 stm32_ops 结构体里的 set_mode 函数进行寄存器的初始化。

第 126 行，使能时钟。

我们继续返回示例代码 52.5.2.5 中 stm32_dwmac_probe 函数。

stm32_dwmac_probe 函数第 410 行，调用 stmmac_dvr_probe 函数进行网络注册，同时完成 GMII/RGMII 接口初始化。

stmmac_dvr_probe 函数调用 netif_napi_add 函数来设置 poll 函数，如图 52.5.2.1 所示：

```
4638        if (queue < priv->plat->rx_queues_to_use) {
4639            netif_napi_add(ndev, &ch->rx_napi, stmmac_napi_poll_rx,
4640                           NAPI_POLL_WEIGHT);
4641        }
4642        if (queue < priv->plat->tx_queues_to_use) {
4643            netif_tx_napi_add(ndev, &ch->tx_napi,
4644                              stmmac_napi_poll_tx,
4645                              NAPI_POLL_WEIGHT);
4646        }
```

图 52.5.2.1 设置 poll 函数

从图 52.5.2.1 可以知道，通过 netif_napi_add 函数向网卡加了一个 napi 示例，使用 NAPI 驱动要提供一个 poll 函数来轮询处理接收数据，发送和接收的 poll 函数分别为：“stmmac_napi_poll_tx”和“stmmac_napi_poll_rx”。后面分析网络数据接收处理流程的时候详细讲解这个两个函数。

继续分析 stmmac_dvr_probe 函数，此函数会调用 stmmac_mdio_register 函数来向内核注册 MDIO 总线，stmmac_mdio_register 函数重点是图 52.5.2.2 中的两行代码：

```
346         new_bus->read = &stmmac_mdio_read;
347         new_bus->write = &stmmac_mdio_write;
348         max_addr = PHY_MAX_ADDR;
349     }
```

图 52.5.2.2 mdio 读写函数

new_bus 下的 read 和 write 这两个成员变量分别是读/写 PHY 寄存器的操作函数，这设置为 stmmac_mdio_read 和 stmmac_mdio_write，这两个函数就是 STM32MP1 系列 SOC 读写 PHY 内部寄存器的函数。读写 PHY 寄存器都会通过这两个 MDIO 总线函数完成。stmmac_mdio_register 函数最终会向 Linux 内核注册 MDIO 总线，相关代码如下所示：

示例代码 52.5.2.8 stmmac_mdio_register 函数注册 mdio 总线

```
1  err = of_mdiobus_register(new_bus, mdio_node);
2  if (err != 0) {
3      dev_err(dev, "Cannot register the MDIO bus\n");
4      goto bus_register_fail;
5  }
6
7  if (priv->plat->phy_node || mdio_node)
8      goto bus_register_done;
9
10 found = 0;
11 for (addr = 0; addr < max_addr; addr++) {
12     struct phy_device *phydev = mdiobus_get_phy(new_bus, addr);
13
14     if (!phydev)
15         continue;
16
17     if (!mdio_bus_data->irqs &&
18         (mdio_bus_data->probed_phy_irq > 0)) {
```

```
19          new_bus->irq[addr] = mdio_bus_data->probed_phy_irq;
20          phydev->irq = mdio_bus_data->probed_phy_irq;
21      }
22
23      if (priv->plat->phy_addr == -1)
24          priv->plat->phy_addr = addr;
25
26      phy_attached_info(phydev);
27      found = 1;
28  }
```

第 1 行，通过 of_mdiobus_register 向内核注册 MDIO 总线，如果设备树定义了 mdio 节点和 PHY 句柄还会注册 PHY 设备。

第 7 行，判断 PHY 设备是否注册成功，没有就遍历所有的 PHY 地址找到对应的 PHY 设备。

第 10~28 行，如果设备树里没有 mdio 节点和 PHY 句柄，通过遍历所有的 PHY 地址，找到硬件所对应的 PHY 设备。

最后 stmmac_dvr_probe 函数调用 register_netdev 函数进行网络设备注册。如下图所示：

```
4683        ret = register_netdev(ndev);
```

图 52.5.2.3 register_netdev 函数

3、MDIO 总线注册

MDIO 我们讲了很多次了，就是用来管理 PHY 芯片的，分为 MDIO 和 MDC 两根线，Linux 内核专门为 MDIO 准备一个总线，叫做 MDIO 总线，采用 mii_bus 结构体表示，定义在 include/linux/phy.h 文件中，mii_bus 结构体如下所示(限于篇幅，有省略)：

示例代码 52.5.2.9 mii_bus 结构体

```
212 struct mii_bus {
213     struct module *owner;
214     const char *name;
215     char id[MII_BUS_ID_SIZE];
216     void *priv;
217     int (*read)(struct mii_bus *bus, int addr, int regnum);
218     int (*write)(struct mii_bus *bus, int addr, int regnum,
                    u16 val);
219     int (*reset)(struct mii_bus *bus);
.....
255 };
```

重点是第 217、218 两行的 read 和 write 函数，这两个函数就是读/写 PHY 芯片的操作函数，不同的 SoC 其 MDIO 主控部分是不一样的，因此需要驱动编写人员去编写。我们前面在分析 stm32_dwmac_probe 函数的时候已经讲过了，stm32_dwmac_probe 函数会调用 stmmac_mdio_register 函数完成 MII 接口的初始化，其中就包括初始化 mii_bus 下的 read 和 write 这两个函数。最终通过 of_mdiobus_register 或者 mdiobus_register 函数将初始化以后的 mii_bus 注册到 Linux 内核，of_mdiobus_register 函数其实也是调用的 mdiobus_register 函数来完成 mii_bus 注册的。我们先看下 of_mdiobus_register 函数，如下示例代码所示：

示例代码 52.5.2.10 of_mdiobus_register 函数

```
199 int of_mdiobus_register(struct mii_bus *mdio,
                          struct device_node *np)
200 {
201     struct device_node *child;
202     bool scanphys = false;
203     int addr, rc;
204
205     if (!np)
206         return mdiobus_register(mdio);
207
208     /* Do not continue if the node is disabled */
209     if (!of_device_is_available(np))
210         return -ENODEV;
211
212     /* Mask out all PHYs from auto probing.  Instead the PHYs listed
213      * in the device tree are populated after the bus has been
       registered */
214     mdio->phy_mask = ~0;
215
216     mdio->dev.of_node = np;
217     mdio->dev.fwnode = of_fwnode_handle(np);
218
219     /* Get bus level PHY reset GPIO details */
220     mdio->reset_delay_us = DEFAULT_GPIO_RESET_DELAY;
221     of_property_read_u32(np, "reset-delay-us",
                          &mdio->reset_delay_us);
222
223     /* Register the MDIO bus */
224     rc = mdiobus_register(mdio);
225     if (rc)
226         return rc;
227
228   /* Loop over the child nodes and register a phy_device for each
      phy */
229     for_each_available_child_of_node(np, child) {
230         addr = of_mdio_parse_addr(&mdio->dev, child);
231         if (addr < 0) {
232             scanphys = true;
233             continue;
234         }
235
236         if (of_mdiobus_child_is_phy(child))
```

```
237             rc = of_mdiobus_register_phy(mdio, child, addr);
238         else
239             rc = of_mdiobus_register_device(mdio, child, addr);
240
241         if (rc == -ENODEV)
242             dev_err(&mdio->dev,
243                 "MDIO device at address %d is missing.\n",
244                 addr);
245         else if (rc)
246             goto unregister;
247     }
.....
281 }
```

of_mdiobus_register 函数主要是注册 mii 总线(mii_bus)，然后遍历所有的 PHY 地址，当找到对应的 PHY 芯片以后就会创建 PHY 设备，最后将这个 PHY 设备注册到内核中。使用 mdiobus_scan 函数进行注册 PHY 设备。

第 206 行，设备节点不存在的时候此行代码执行，本实验设备节点肯定存在，所以之类不会运行。

第 224 行，当设备树中 mdio 节点存在，就会将相关属性信息赋值给 mdio 参数，然后调用 mdiobus_register 向内核注册此 mii 总线。

第 229 行，轮询 mdio 节点下的所有 phy 子节点，比如示例代码 52.5.1.2 中的“phy0:ethernet-phy@0”这个子节点，它描述的是 PHY 芯片信息。

第 237 行，如果找到一个 PHY 子节点，就说明找到了一个 PHY 芯片，那么就调用 of_mdiobus_register_phy 函数向内核注册此 PHY 设备。

接下来简单分析一下 of_mdiobus_register_phy 函数，看看如何向 Linux 内核注册 PHY 设备的，of_mdiobus_register_phy 函数内容如下所示：

示例代码 52.5.2.11 of_mdiobus_register_phy 函数

```
1  static int of_mdiobus_register_phy(struct mii_bus *mdio,
2                     struct device_node *child, u32 addr)
3  {
4      struct phy_device *phy;
5      bool is_c45;
6      int rc;
7      u32 phy_id;
8
9      is_c45 = of_device_is_compatible(child,
10                     "ethernet-phy-ieee802.3-c45");
11
12     if (!is_c45 && !of_get_phy_id(child, &phy_id))
13         phy = phy_device_create(mdio, addr, phy_id, 0, NULL);
14     else
15         phy = get_phy_device(mdio, addr, is_c45);
16     if (IS_ERR(phy))
```

```
17          return PTR_ERR(phy);
18
19      rc = of_irq_get(child, 0);
20      if (rc == -EPROBE_DEFER) {
21          phy_device_free(phy);
22          return rc;
23      }
24      if (rc > 0) {
25          phy->irq = rc;
26          mdio->irq[addr] = rc;
27      } else {
28          phy->irq = mdio->irq[addr];
29      }
30
31      if (of_property_read_bool(child, "broken-turn-around"))
32          mdio->phy_ignore_ta_mask |= 1 << addr;
33
34      of_property_read_u32(child, "reset-assert-us",
35                   &phy->mdio.reset_assert_delay);
36      of_property_read_u32(child, "reset-deassert-us",
37                   &phy->mdio.reset_deassert_delay);
38
39      /* Associate the OF node with the device structure so it
40       * can be looked up later */
41      of_node_get(child);
42      phy->mdio.dev.of_node = child;
43      phy->mdio.dev.fwnode = of_fwnode_handle(child);
44
45      /* All data is now stored in the phy struct;
46       * register it */
47      rc = phy_device_register(phy);
48      if (rc) {
49          phy_device_free(phy);
50          of_node_put(child);
51          return rc;
52      }
53
54      dev_dbg(&mdio->dev, "registered phy %pOFn at address %i\n",
55          child, addr);
56      return 0;
57  }
```

第 9 行，使用函数 of_device_is_compatible 检查 PHY 节点的 compatible 属性是否为“ethernet-phy-ieee802.3-c45”，如果是的话要做其他的处理，这里没有设置此属性。

第 15 行，调用 get_phy_device 函数获取 PHY 设备，此函数里面会调用 phy_device_create 来创建一个 phy_device 设备并返回。

第 19 行，获取 PHY 芯片的中断信息，本章节并未用到。

第 47 行，调用 phy_device_register 函数向 Linux 内核注册 PHY 设备。

从上面的分析可以看出，向 Linux 内核注册 MDIO 总线的时候也会同时向 Linux 内核注册 PHY 设备，流程如图 52.5.2.2 所示：

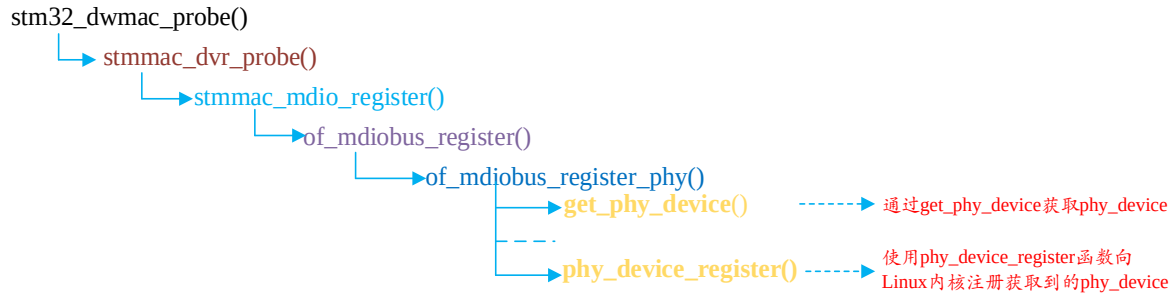


图 52.5.2.2 MDIO 总线注册流程

注册 MDIO 总线的时候会先从设备树中查找 PHY 设备，没有就会遍历所有的 PHY 设备，然后通过 phy_device_register 函数向内核注册 PHY 设备.

4、stm32_dwmac_remove 函数简析

卸载 STM32MP1 网络驱动的时候 stm32_dwmac_remove 函数就会执行，函数内容如下所示：

示例代码 52.5.2.12 stm32_dwmac_remove 函数

```
424 static int stm32_dwmac_remove(struct platform_device *pdev)
425 {
426     struct net_device *ndev = platform_get_drvdata(pdev);
427     struct stmmac_priv *priv = netdev_priv(ndev);
428     int ret = stmmac_dvr_remove(&pdev->dev);
......
437
438     return ret;
439 }
```

第 428 行调用了 stmmac_dvr_remove 函数进行卸载，此函数内容如下所示：

示例代码 52.5.2.13 stmmac_dvi_remove 函数

```
1  int stmmac_dvr_remove(struct device *dev)
2  {
3      struct net_device *ndev = dev_get_drvdata(dev);
4      struct stmmac_priv *priv = netdev_priv(ndev);
5
6      netdev_info(priv->dev, "%s: removing driver", __func__);
7
8      stmmac_stop_all_dma(priv);
9
10     stmmac_mac_set(priv, priv->ioaddr, false);
11     netif_carrier_off(ndev);
```

```
12      unregister_netdev(ndev);
13  #ifdef CONFIG_DEBUG_FS
14      stmmac_exit_fs(ndev);
15  #endif
16      phylink_destroy(priv->phylink);
17      if (priv->plat->stmmac_rst)
18          reset_control_assert(priv->plat->stmmac_rst);
19      clk_disable_unprepare(priv->plat->pclk);
20      clk_disable_unprepare(priv->plat->stmmac_clk);
21      if (priv->hw->pcs != STMMAC_PCS_RGMII &&
22          priv->hw->pcs != STMMAC_PCS_TBI &&
23          priv->hw->pcs != STMMAC_PCS_RTBI)
24          stmmac_mdio_unregister(ndev);
25      destroy_workqueue(priv->wq);
26      mutex_destroy(&priv->lock);
27
28      return 0;
29  }
```

第 8 行，调用 stmmac_stop_all_dma 函数结束所有的 DMA。

第 12 行，调用 unregister_netdev 函数注销前面注册的 net_device。

第 24 行，调用 stmmac_mdio_unregister 函数来移除掉 MDIO 总线相关的内容，此函数会调用 mdiobus_unregister 来注销掉 mii_bus，并且通过函数 mdiobus_free 释放掉 mii_bus。

第 25 行，释放工作队列。

52.5.3 stmmac_netdev_ops 操作集

stmmac_dvr_probe 函数设置了网卡驱动的 net_dev_ops 操作集为 stmmac_netdev_ops，stmmac_netdev_ops 内容如下：

示例代码 52.5.3.1 stmmac_netdev_ops 操作集

```
1   static const struct net_device_ops stmmac_netdev_ops = {
2       .ndo_open          = stmmac_open,
3       .ndo_start_xmit    = stmmac_xmit,
4       .ndo_stop          = stmmac_release,
5       .ndo_change_mtu    = stmmac_change_mtu,
6       .ndo_fix_features  = stmmac_fix_features,
7       .ndo_set_features  = stmmac_set_features,
8       .ndo_set_rx_mode   = stmmac_set_rx_mode,
9       .ndo_tx_timeout    = stmmac_tx_timeout,
10      .ndo_do_ioctl      = stmmac_ioctl,
11      .ndo_setup_tc      = stmmac_setup_tc,
12      .ndo_select_queue  = stmmac_select_queue,
13  #ifdef CONFIG_NET_POLL_CONTROLLER
14      .ndo_poll_controller     = stmmac_poll_controller,
15  #endif
```

```
16      .ndo_set_mac_address     = stmmac_set_mac_address,
17      .ndo_vlan_rx_add_vid     = stmmac_vlan_rx_add_vid,
18      .ndo_vlan_rx_kill_vid    = stmmac_vlan_rx_kill_vid,
19  };
```

1、stmmac_ppen 函数简析

打开一个网卡的时候 stmmac_open 函数就会执行，函数源码如下所示(限于篇幅原因，有省略)：

示例代码 52.5.3.1 stmmac_open 函数

```
1   static int stmmac_open(struct net_device *dev)
2   {
3       struct stmmac_priv *priv = netdev_priv(dev);
4       int bfsize = 0;
5       u32 chan;
6       int ret;
7
8       if (priv->hw->pcs != STMMAC_PCS_RGMII &&
9           priv->hw->pcs != STMMAC_PCS_TBI &&
10          priv->hw->pcs != STMMAC_PCS_RTBI) {
11          ret = stmmac_init_phy(dev);
12          if (ret) {
13              netdev_err(priv->dev,
14                         "%s: Cannot attach to PHY (error: %d)\n",
15                         __func__, ret);
16              return ret;
17          }
18      }
......
36      ret = alloc_dma_desc_resources(priv);
37      if (ret < 0) {
38          netdev_err(priv->dev, "%s: DMA descriptors allocation
                                   failed\n",
39                     __func__);
40          goto dma_desc_error;
41      }
42
43      ret = init_dma_desc_rings(dev, GFP_KERNEL);
44      if (ret < 0) {
45          netdev_err(priv->dev, "%s: DMA descriptors initialization
                                   failed\n",
46                     __func__);
47          goto init_error;
48      }
49
```

```
50      ret = stmmac_hw_setup(dev, true);
51      if (ret < 0) {
52          netdev_err(priv->dev, "%s: Hw setup failed\n", __func__);
53          goto init_error;
54      }
55
56      stmmac_init_coalesce(priv);
57
58      phylink_start(priv->phylink);
59
60      /* Request the IRQ lines */
61          ret = request_irq(dev->irq, stmmac_interrupt,
62                    IRQF_SHARED, dev->name, dev);
63          if (unlikely(ret < 0)) {
64              netdev_err(priv->dev,
65                     "%s: ERROR: allocating the IRQ %d (error: %d)\n",
66                     __func__, dev->irq, ret);
67              goto irq_error;
68          }
69
70     /* Request the Wake IRQ in case of another line is used for WoL */
71          if (priv->wol_irq != dev->irq) {
72              ret = request_irq(priv->wol_irq, stmmac_interrupt,
73                        IRQF_SHARED, dev->name, dev);
74              if (unlikely(ret < 0)) {
75                  netdev_err(priv->dev,
76                         "%s: ERROR: allocating the WoL IRQ %d (%d)\n",
77                         __func__, priv->wol_irq, ret);
78                  goto wolirq_error;
79              }
80          }
81
82          /* Request the IRQ lines */
83          if (priv->lpi_irq > 0) {
84              ret = request_irq(priv->lpi_irq, stmmac_interrupt,
                                  IRQF_SHARED,
85                                dev->name, dev);
86              if (unlikely(ret < 0)) {
87                  netdev_err(priv->dev,
88                         "%s: ERROR: allocating the LPI IRQ %d (%d)\n",
89                         __func__, priv->lpi_irq, ret);
90                  goto lpiirq_error;
91              }
```

```
92          }
93
94      stmmac_enable_all_queues(priv);
95      stmmac_start_all_queues(priv);
96
97      return 0;
98  };
```

第 11 行，看名字就知道了，初始化 phy 设备。
第 36 行，分配 DMA 资源，分为 TX 和 RX 两个 DMA 资源。
第 43 行，初始化 TX 和 RX DMA 描述符，并且分配 socket 缓冲区。
第 50 行，设置 mac 为可用状态，配置 mac 核心寄存器，然后 DMA 数据准备接收和发送。
第 58 行，启动 phylink。
第 60~92 行，申请中断，中断函数为 stmmac_interrupt，重点，后面会解析。
第 94~95 行，使能队列和开启队列。

2、stmmac_release 函数简析

关闭网卡的时候 stmmac_release 函数就会执行，函数内容如下：

示例代码 52.5.3.2 stmmac_release 函数

```
1   static int stmmac_release(struct net_device *dev)
2   {
3       struct stmmac_priv *priv = netdev_priv(dev);
4       u32 chan;
5
6       if (priv->eee_enabled)
7           del_timer_sync(&priv->eee_ctrl_timer);
8
9       /* Stop and disconnect the PHY */
10      phylink_stop(priv->phylink);
11      phylink_disconnect_phy(priv->phylink);
12
13      stmmac_stop_all_queues(priv);
14
15      stmmac_disable_all_queues(priv);
16
17      for (chan = 0; chan < priv->plat->tx_queues_to_use; chan++)
18          del_timer_sync(&priv->tx_queue[chan].txtimer);
19
20      /* Free the IRQ lines */
21      free_irq(dev->irq, dev);
22      if (priv->wol_irq != dev->irq)
23          free_irq(priv->wol_irq, dev);
24      if (priv->lpi_irq > 0)
25          free_irq(priv->lpi_irq, dev);
26
```

```
27      /* Stop TX/RX DMA and clear the descriptors */
28      stmmac_stop_all_dma(priv);
29
30      /* Release and free the Rx/Tx resources */
31      free_dma_desc_resources(priv);
32
33      /* Disable the MAC Rx/Tx */
34      stmmac_mac_set(priv, priv->ioaddr, false);
35
36      netif_carrier_off(dev);
37
38      stmmac_release_ptp(priv);
39
40      return 0;
41  }
```

第 10 行，停止 phylink。
第 11 行，断开 phylink。
第 13 行，停止工作队列。
第 15 行，关闭工作队列。
第 28 行，停止 TX/RX 的 DMA。
第 31 行，释放 TX/RX DMA 资源。
第 34 行，关闭 MAC TX/RX。
第 36 行，关闭网络。

3、stmmac_xmit 函数简析

STM32MP1 的网络数据发送是通过 stmmac_xmit 函数来完成，这个函数将上层传递过来的 sk_buff 中的数据通过 DMA 发送出去，函数源码如下所示(限于篇幅原因，有省略)：

示例代码 52.5.3.3 stmmac_xmit 函数

```
1   static netdev_tx_t stmmac_xmit(struct sk_buff *skb,
                                   struct net_device *dev)
2   {
3       struct stmmac_priv *priv = netdev_priv(dev);
4       unsigned int nopaged_len = skb_headlen(skb);
5       int i, csum_insertion = 0, is_jumbo = 0;
6       u32 queue = skb_get_queue_mapping(skb);
7       int nfrags = skb_shinfo(skb)->nr_frags;
......
164     /* Ready to fill the first descriptor and set the OWN bit w/o
165      * any problems because all the descriptors are actually ready
166      * to be passed to the DMA engine.
167      */
168     if (likely(!is_jumbo)) {
169         bool last_segment = (nfrags == 0);
170
```

```
171         des = dma_map_single(priv->device, skb->data,
172                              nopaged_len, DMA_TO_DEVICE);
173         if (dma_mapping_error(priv->device, des))
174             goto dma_map_err;
175
176         tx_q->tx_skbuff_dma[first_entry].buf = des;
177
......
190         /* Prepare the first descriptor setting the OWN bit too */
191         stmmac_prepare_tx_desc(priv, first, 1, nopaged_len,
192                                csum_insertion, priv->mode, 1, last_segment,
193                                skb->len);
194     } else {
195         stmmac_set_tx_owner(priv, first);
196     }
197
198     /* The own bit must be the latest setting done when prepare the
199      * descriptor and then barrier is needed to make sure that
200      * all is coherent before granting the DMA engine.
201      */
202     wmb();
203
204     netdev_tx_sent_queue(netdev_get_tx_queue(dev, queue), skb->len);
205
206     stmmac_enable_dma_transmission(priv, priv->ioaddr);
207
208     tx_q->tx_tail_addr = tx_q->dma_tx_phy + (tx_q->cur_tx *
                                              sizeof(*desc));
209     stmmac_set_tx_tail_ptr(priv, priv->ioaddr, tx_q->tx_tail_addr,
                               queue);
210     stmmac_tx_timer_arm(priv, queue);
211
212     return NETDEV_TX_OK;
......
219 }
```

第 171 行，调用 dma_map_single 函数，进行 DMA 映射，返回值就是 DMA 映射后的虚拟地址。

第 176 行，保存 DMA 的虚拟地址，在 stmmac_interrupt 中断函数里使用。

第 191 行，调用 stmmac_prepare_tx_desc 函数，填充描述符其它信息，让 DMA 拥有该描述符。

第 209 行，设置尾指针，启动 DMA 发送。

4、stmmac_interrupt 中断函数简析

当 DMA 发送完数据或者接收数据都会产生中断，调用 stmmac_interrupt 函数就行处理；

stmmac_interrupt 函数通过调用 stmmac_dma_interrupt 函数处理 DMA 相关中断，函数的源码如下所示(有省略)：

示例代码 52.5.3.4 stmmac_dma_interrupt 函数

```
1  static void stmmac_dma_interrupt(struct stmmac_priv *priv)
2  {
3      u32 tx_channel_count = priv->plat->tx_queues_to_use;
4      u32 rx_channel_count = priv->plat->rx_queues_to_use;
5      u32 channels_to_check = tx_channel_count > rx_channel_count ?
6                  tx_channel_count : rx_channel_count;
7      u32 chan;
8      int status[max_t(u32, MTL_MAX_TX_QUEUES, MTL_MAX_RX_QUEUES)];
9
10     /* Make sure we never check beyond our status buffer. */
11     if (WARN_ON_ONCE(channels_to_check > ARRAY_SIZE(status)))
12         channels_to_check = ARRAY_SIZE(status);
13
14     for (chan = 0; chan < channels_to_check; chan++)
15         status[chan] = stmmac_napi_check(priv, chan);
.......
39 }
```

第 15 行，调用 stmmac_napi_check 函数设置 NAPI 软中断标志位，当触发 NAPI 机制软中断，这个时候 napi 的 poll 函数就会执行，如果是接收数据的就会触发 stmmac_napi_poll_rx 函数，是发送数据的就会触发 stmmac_napi_poll_tx 函数。

5、stmmac_napi_poll_rx 函数简析

stmmac_napi_poll_rx 函数内容如下所示：

示例代码 52.5.3.5 stmmac_napi_poll_rx 函数

```
1  static int stmmac_napi_poll_rx(struct napi_struct *napi, int budget)
2  {
3      struct stmmac_channel *ch =
4          container_of(napi, struct stmmac_channel, rx_napi);
5      struct stmmac_priv *priv = ch->priv_data;
6      u32 chan = ch->index;
7      int work_done;
8
9      priv->xstats.napi_poll++;
10
11     work_done = stmmac_rx(priv, budget, chan);
12     if (work_done < budget && napi_complete_done(napi, work_done))
13         stmmac_enable_dma_irq(priv, priv->ioaddr, chan);
14     return work_done;
15 }
```

第 11 行，stmmac_rx 函数是用来接收数据的。

第 12 行，napi_complete_done 函数结束 NAPI 轮询。

第 13 行，重新开启 dma 的中断。

6、stmmac_napi_poll_tx 函数简析

stmmac_napi_poll_tx 函数内容如下所示：

示例代码 52.5.3.6 stmmac_napi_poll_tx 函数

```
1  static int stmmac_napi_poll_tx(struct napi_struct *napi, int budget)
2  {
3      struct stmmac_channel *ch =
4          container_of(napi, struct stmmac_channel, tx_napi);
5      struct stmmac_priv *priv = ch->priv_data;
6      struct stmmac_tx_queue *tx_q;
7      u32 chan = ch->index;
8      int work_done;
9
10     priv->xstats.napi_poll++;
11
12     work_done = stmmac_tx_clean(priv, DMA_TX_SIZE, chan);
13     work_done = min(work_done, budget);
14
15     if (work_done < budget)
16         napi_complete_done(napi, work_done);
17
18     /* Force transmission restart */
19     tx_q = &priv->tx_queue[chan];
20     if (tx_q->cur_tx != tx_q->dirty_tx) {
21         stmmac_enable_dma_transmission(priv, priv->ioaddr);
22         stmmac_set_tx_tail_ptr(priv, priv->ioaddr,
           tx_q->tx_tail_addr,
23         chan);
24     }
25
26     return work_done;
27 }
```

第 12 行，调用 stmmac_tx_clean 进行数据传输，传输完成以后会进行资源回收。

第 19~24 行，重启发送数据。

52.5.4 Linux 内核 PHY 子系统与 MDIO 总线简析

上一小节在讲解 MDIO 总线的时候讲过，注册 MDIO 总线的时候也会向内核注册 PHY 设备，本节我们就来简单了解一下 PHY 子系统。PHY 子系统就是用于 PHY 设备相关内容的，分为 PHY 设备和 PHY 驱动，和 platform 总线一样，PHY 子系统也是一个设备、总线和驱动模型。

1、PHY 设备

首先看一下 PHY 设备，Linux 内核使用 phy_device 结构体来表示 PHY 设备，结构体定义在 include/linux/phy.h，结构体内容如下(为了缩小篇幅，有省略)：

示例代码 52.5.4.1 phy_device 结构体

```
360 struct phy_device {
361     struct mdio_device mdio;     /* mdio 设备       */
362
363     /* Information about the PHY type */
364     /* And management functions */
365     struct phy_driver *drv;      /* PHY 设备驱动     */
366
367     u32 phy_id;                  /* PHY ID          */
......
398     int speed;                   /* 速度            */
399     int duplex;                  /* 双共模式        */
400     int pause;
401     int asym_pause;
402
......
421
422     /*
423      * Interrupt number for this PHY
424      * -1 means no interrupt
425      */
426     int irq;                     /* 中断号          */
427
428     /* private data pointer */
429     /* For use by PHYs to maintain extra state */
430     void *priv;                  /* 私有数据        */
431
432     /* Interrupt and Polling infrastructure */
433     struct delayed_work state_queue;
434
435     struct mutex lock;
436
437     struct phylink *phylink;
438     struct net_device *attached_dev;    /* PHY 芯片对应的网络设备 */
......
445 };
```

一个 PHY 设备对应一个 phy_device 实例，然后需要向 Linux 内核注册这个实例。使用 phy_device_register 函数完成 PHY 设备的注册，函数原型如下：

```
int phy_device_register(struct phy_device *phy)
```

函数参数和返回值含义如下：

phy：需要注册的 PHY 设备。

返回值：0 成功，负值 失败。

PHY 设备的注册过程一般是先调用 get_phy_device 函数获取 PHY 设备，此函数内容如下：

示例代码 52.5.4.2 get_phy_device 函数

```
818 struct phy_device *get_phy_device(struct mii_bus *bus, int addr,
                                     bool is_c45)
819 {
820     struct phy_c45_device_ids c45_ids;
821     u32 phy_id = 0;
822     int r;
823
824     c45_ids.devices_in_package = 0;
825     memset(c45_ids.device_ids, 0xff, sizeof(c45_ids.device_ids));
826
827     r = get_phy_id(bus, addr, &phy_id, is_c45, &c45_ids);
828     if (r)
829         return ERR_PTR(r);
830
831     /* If the phy_id is mostly Fs, there is no device there */
832     if ((phy_id & 0x1fffffff) == 0x1fffffff)
833         return ERR_PTR(-ENODEV);
834
835     return phy_device_create(bus, addr, phy_id, is_c45, &c45_ids);
836 }
```

第 827 行，调用 get_phy_id 函数获取 PHY ID，也就是读取 PHY 芯片的那两个 ID 寄存器，得到 PHY 芯片 ID 信息。

第 835 行，调用 phy_device_create 函数创建 phy_device，此函数先申请 phy_device 内存，然后初始化 phy_device 的各个结构体成员，最终返回创建好的 phy_device。phy_device_register 函数注册的就是这个创建好的 phy_device。

2、PHY 驱动

PHY 驱动使用结构体 phy_driver 表示，结构体也定义在 include/linux/phy.h 文件中，结构体内容如下(为了缩小篇幅，省略了注释部分)：

示例代码 52.5.4.3 phy_driver 结构体

```
1  struct phy_driver {
2      struct mdio_driver_common mdiodrv;
3      u32 phy_id;           /* PHY ID     */
4      char *name;
5      u32 phy_id_mask;      /* PHY ID 掩码 */
6      const unsigned long * const features;
7      u32 flags;
8      const void *driver_data;
9
10     int (*soft_reset)(struct phy_device *phydev);
11     int (*config_init)(struct phy_device *phydev);
12     int (*probe)(struct phy_device *phydev);
13     int (*get_features)(struct phy_device *phydev);
```

```
14      int (*suspend)(struct phy_device *phydev);
15      int (*resume)(struct phy_device *phydev);
16      int (*config_aneg)(struct phy_device *phydev);
17      int (*aneg_done)(struct phy_device *phydev);
18      int (*read_status)(struct phy_device *phydev);
19      int (*ack_interrupt)(struct phy_device *phydev);
20      int (*config_intr)(struct phy_device *phydev);
21      int (*did_interrupt)(struct phy_device *phydev);
22      int (*handle_interrupt)(struct phy_device *phydev);
23      void (*remove)(struct phy_device *phydev);
24      int (*match_phy_device)(struct phy_device *phydev);
25      int (*ts_info)(struct phy_device *phydev,
                       struct ethtool_ts_info *ti);
26      int  (*hwtstamp)(struct phy_device *phydev, struct ifreq *ifr);
27      bool (*rxtstamp)(struct phy_device *dev, struct sk_buff *skb,
                       int type);
28      void (*txtstamp)(struct phy_device *dev, struct sk_buff *skb,
                         int type);
29      int (*set_wol)(struct phy_device *dev,
                       struct ethtool_wolinfo *wol);
30      void (*get_wol)(struct phy_device *dev,
                        struct ethtool_wolinfo *wol);
31      void (*link_change_notify)(struct phy_device *dev);
32      int (*read_mmd)(struct phy_device *dev, int devnum, u16 regnum);
33      int (*write_mmd)(struct phy_device *dev, int devnum,
                         u16 regnum,u16 val);
34      int (*read_page)(struct phy_device *dev);
35      int (*write_page)(struct phy_device *dev, int page);
36      int (*module_info)(struct phy_device *dev,
37                struct ethtool_modinfo *modinfo);
38      int (*module_eeprom)(struct phy_device *dev,
39                 struct ethtool_eeprom *ee, u8 *data);
40      int (*get_sset_count)(struct phy_device *dev);
41      void (*get_strings)(struct phy_device *dev, u8 *data);
42      void (*get_stats)(struct phy_device *dev,
43              struct ethtool_stats *stats, u64 *data);
44      int (*get_tunable)(struct phy_device *dev,
45                struct ethtool_tunable *tuna, void *data);
46      int (*set_tunable)(struct phy_device *dev,
47                 struct ethtool_tunable *tuna,
48                 const void *data);
49      int (*set_loopback)(struct phy_device *dev, bool enable);
50  };
```

可以看出，phy_driver 重点是大量的函数，编写 PHY 驱动的主要工作就是实现这些函数，但是不一定全部实现，稍后我们会简单分析一下 Linux 内核通用 PHY 驱动。

①、注册 PHY 驱动

phy_driver 结构体初始化完成以后，就需要向 Linux 内核注册，PHY 驱动的注册使用 phy_driver_register 函数，注册 phy 驱动时候会设置驱动的总线为 mdio_bus_type，也就是 MDIO 总线，关于 MDIO 总线稍后会讲解，函数原型如下：

```
int phy_driver_register(struct phy_driver *new_driver, struct module *owner);
```

函数参数和返回值含义如下：

new_driver：需要注册的 PHY 驱动。

owner：驱动模块所属的 PHY 设备。

返回值：0 成功，负值 失败。

②、连续注册多个 PHY 驱动

一个厂家会生产多种 PHY 芯片，这些 PHY 芯片内部差别一般不大，如果一个个的去注册驱动将会导致一堆重复的驱动文件，因此 Linux 内核提供了一个连续注册多个 PHY 驱动的函数 phy_drivers_register。首先准备一个 phy_driver 数组，一个数组元素就表示一个 PHY 芯片的驱动，然后调用 phy_drivers_register 一次性注册整个数组中的所有驱动，函数原型如下：

```
int phy_drivers_register(struct phy_driver    *new_driver,
                         int                  n,
                         struct module        *owner);
```

函数参数和返回值含义如下：

new_driver：需要注册的多个 PHY 驱动数组。

n：要注册的驱动数量。

owner：驱动模块所属的 PHY 设备。

返回值：0 成功，负值 失败。

②、卸载 PHY 驱动

卸载 PHY 驱动的话使用 phy_driver_unregister 函数，函数原型如下：

```
void phy_driver_unregister(struct phy_driver *drv)
```

函数参数和返回值含义如下：

drv：需要卸载的 PHY 驱动。

返回值：无。

3、MDIO 总线

前面说了，PHY 子系统也是遵循设备、总线、驱动模型的，设备和驱动就是 phy_device 和 phy_driver。总线就是 MDIO 总线，因为 PHY 芯片是通过 MIDO 接口来管理的，MDIO 总线最主要的工作就是匹配 PHY 设备和 PHY 驱动。在文件 drivers/net/phy/mdio_bus.c 中有如下定义：

示例代码 52.5.4.4 mdio 总线

```
1  struct bus_type mdio_bus_type = {
2      .name      = "mdio_bus",
3      .match     = mdio_bus_match,
4      .uevent    = mdio_uevent,
5  };
```

示例代码 52.5.4.4 定义了一个名为“mdio_bus_type”的总线，这个就是 MDIO 总线，总线

的名字为“mdio_bus”，重点是总线的匹配函数为 mdio_bus_match。此函数内容如下：

示例代码 52.5.4.5 mdio_bus_match 函数

```
1  static int mdio_bus_match(struct device *dev,
                             struct device_driver *drv)
2  {
3      struct mdio_device *mdio = to_mdio_device(dev);
4
5      if (of_driver_match_device(dev, drv))
6          return 1;
7
8      if (mdio->bus_match)
9          return mdio->bus_match(dev, drv);
10
11     return 0;
12 }
```

第 5 行，采用设备树的话先尝试使用 of_driver_match_device 来对设备和驱动进行匹配，也就是检查 compatible 属性值与匹配表 of_match_table 里面的内容是否一致。但是对于本章教程而言，并不是通过 of_driver_match_device 来完成 PHY 驱动和设备匹配的。

第 8~9 行，使用 PHY 驱动的匹配方法，会调用 phy_bus_match 函数，函数源码如下所示：

示例代码 52.5.4.6 phy_bus_match 函数

```
1  static int phy_bus_match(struct device *dev,
                            struct device_driver *drv)
2  {
3      struct phy_device *phydev = to_phy_device(dev);
4      struct phy_driver *phydrv = to_phy_driver(drv);
5      const int num_ids = ARRAY_SIZE(phydev->c45_ids.device_ids);
6      int i;
7
8      if (!(phydrv->mdiodrv.flags & MDIO_DEVICE_IS_PHY))
9          return 0;
10
11     if (phydrv->match_phy_device)
12         return phydrv->match_phy_device(phydev);
13
14     if (phydev->is_c45) {
15         for (i = 1; i < num_ids; i++) {
16             if (phydev->c45_ids.device_ids[i] == 0xffffffff)
17                 continue;
18
19             if ((phydrv->phy_id & phydrv->phy_id_mask) ==
20                 (phydev->c45_ids.device_ids[i] &
21                  phydrv->phy_id_mask))
22                 return 1;
```

```
23          }
24          return 0;
25      } else {
26          return (phydrv->phy_id & phydrv->phy_id_mask) ==
27              (phydev->phy_id & phydrv->phy_id_mask);
28      }
29  }
```

第 11、12 行，检查 PHY 驱动有没有提供匹配函数 match_phy_device，如果有的话就直接调用 PHY 驱动提供的匹配函数完成与设备的匹配。

第 26、27 行，phy_driver 里面有两个成员变量 phy_id 和 phy_id_mask，表示此驱动所匹配的 PHY 芯片 ID 以及 ID 掩码，PHY 驱动编写人员需要给这两个成员变量赋值。phy_device 也有一个 phy_id 成员变量，表示此 PHY 芯片的 ID，phy_device 里面的 phy_id 是在注册 PHY 设备的时候调用 get_phy_id 函数直接读取 PHY 芯片内部 ID 寄存器得到的！很明显 PHY 驱动和 PHY 设备中的 ID 要一样，这样才能匹配起来。所以最后一种方法就是对比 PHY 驱动和 PHY 设备中的 phy_id 是否一致，这里需要与 PHY 驱动里面的 phy_id_mask 进行与运算，如果结果一致的话就说明驱动和设备匹配。

如果 PHY 设备和 PHY 驱动匹配，那么就使用指定的 PHY 驱动，如果不匹配的话就使用 Linux 内核自带的通用 PHY 驱动。

3、通用 PHY 驱动

前面多次提到 Linux 内核已经集成了通用 PHY 驱动，通用 PHY 驱动名字为“Generic PHY”，打开 drivers/net/phy/phy_device.c，找到 phy_init 函数，内容如下：

示例代码 52.5.4.7 phy_init 函数

```
1   static int __init phy_init(void)
2   {
3       int rc;
4
5       rc = mdio_bus_init();
6       if (rc)
7           return rc;
8
9       features_init();
10
11      rc = phy_driver_register(&genphy_c45_driver, THIS_MODULE);
12      if (rc)
13          goto err_c45;
14
15      rc = phy_driver_register(&genphy_driver, THIS_MODULE);
16      if (rc) {
17          phy_driver_unregister(&genphy_c45_driver);
18  err_c45:
19          mdio_bus_exit();
20      }
21
```

```
22      return rc;
23  }
```

phy_init 是整个 PHY 子系统的入口函数，第 11 行和 15 行都会调用 phy_drivers_register 函数向内核直接注册一个通用 PHY 驱动：genphy_c45_driver 和 genphy_driver；genphy_c45_driver 为 10G 网络，genphy_driver 为 10/100/1000M 网络。它们都是是通用 PHY 驱动，也就是说 Linux 系统启动以后默认就已经存在了通用 PHY 驱动。

genphy_c45_driver 和 genphy_driver 这两个结构体，它们定义分别在 drivers/net/phy/phy_device.c 和 drivers/net/phy/phy-c46.c 这两个文件中，内容如下所示：

示例代码 52.5.4.8 通用 phy 设备对象

```
1   struct phy_driver genphy_c45_driver = {
2       .phy_id         = 0xffffffff,
3       .phy_id_mask    = 0xffffffff,
4       .name           = "Generic Clause 45 PHY",
5       .soft_reset     = genphy_no_soft_reset,
6       .read_status    = genphy_c45_read_status,
7   };
8
9   static struct phy_driver genphy_driver = {
10      .phy_id         = 0xffffffff,
11      .phy_id_mask    = 0xffffffff,
12      .name           = "Generic PHY",
13      .soft_reset     = genphy_no_soft_reset,
14      .get_features   = genphy_read_abilities,
15      .aneg_done      = genphy_aneg_done,
16      .suspend        = genphy_suspend,
17      .resume         = genphy_resume,
18      .set_loopback   = genphy_loopback,
19  };
```

genphy_c45_driver 为 10G 的 PHY 驱动，名字为“Generic Clause 45 PHY”，genphy_driver 为 10/100/1000M 的 PHY 驱动，名字为“Generic PHY”。注意，很多专用 PHY 芯片的驱动程序中也会用到通用 PHY 驱动的一些函数。

52.5.5 RTL8211F PHY 驱动

RTL8211F 芯片所对应的驱动文件为 driver/net/phy/realtek.c，这个文件是针对 realtek 公司自己的一些 PHY 芯片编写的驱动文件，其中就包含了 RTL8211F 这个 PHY 芯片。

打开 realtek.c 驱动源码文件，找到如下所示内容(减轻篇幅，有省略)：

示例代码 52.5.5.1 realtek 公司 PHY 驱动对象

```
451 static struct phy_driver realtek_drvs[] = {
452     {
......
510     }, {
511         PHY_ID_MATCH_EXACT(0x001cc916),
512         .name               = "RTL8211F Gigabit Ethernet",
```

```
513         .config_init      = &rtl8211f_config_init,
514         .ack_interrupt    = &rtl8211f_ack_interrupt,
515         .config_intr      = &rtl8211f_config_intr,
516         .suspend          = genphy_suspend,
517         .resume           = genphy_resume,
518         .read_page        = rtl821x_read_page,
519         .write_page       = rtl821x_write_page,
520     }, {
......
555 };
556
557 module_phy_driver(realtek_drvs);
```

第 511 行， PHY ID 为 0x001cc916。

第 512 行，驱动名字为"RTL8211F Gigabit Ethernet"，系统在启动过程中，加载网络设备驱动时候就会提示当前 PHY 驱动名字为"RTL8211F Gigabit Ethernet"。

最后，第 557 行使用 module_phy_driver(本质是一个宏)来完成 realtek_drvs 的注册。此驱动里的成员有一些是 realtek 驱动工程师自己编写的，有一些是直接使用了通用 PHY 驱动的，一 genphy 开头的函数都是通用 PHY 驱动。

默认情况下，realtek.c 驱动是被使能的，我们还是要教一下如何配置此驱动。打看内核的图形配置界面，打开此驱动配置选项，配置路径如下：

```
-> Device Drivers
    -> Network device support (NETDEVICES [=y])
        ->  PHY Device support and infrastructure (PHYLIB [=y])
            -> <*> Realtek PHYs          //选中
```

配置界面如图 52.5.5.1 所示：

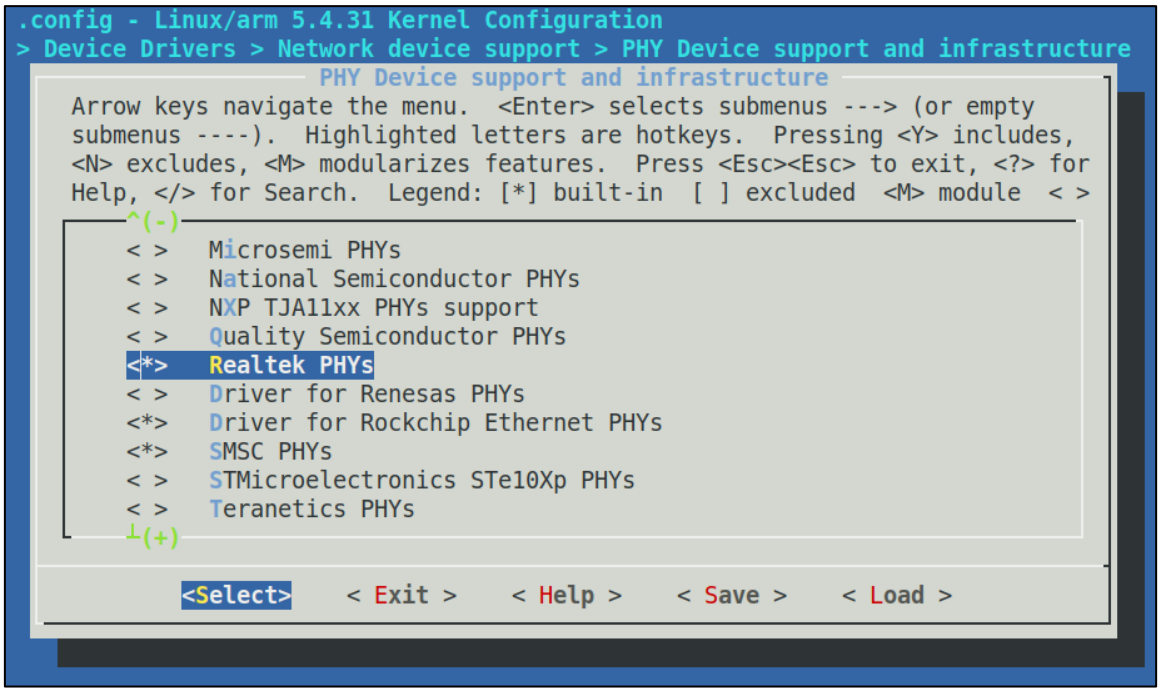


图 52.5.5.1 使能 RTL8211F 驱动

52.5.6 YT8511 PHY 驱动

如果是使用 V1.3 版本以后的核心板，网络的 PHY 芯片为 YT8511 芯片，此芯片为国产的网络芯片，在 Linux 内核没有对应的 PHY 驱动，但是厂家会给我们提供 linux 下的 PHY 驱动。不过这个原厂驱动有点小问题，那就是没有使能 YT8511 的 125M 时钟输出，导致驱动工作不正常。正点原子已经修改好了这个小问题，把修改好的 PHY 驱动放到：开发板光盘→1、程序源码→2、Linux 驱动例程→ 28_YT8511C_PHY 目录下。

在 17.2.3 小节移植 Linux 驱动的时候已经详细讲解过了如何在 Linux 内核里面添加 YT8511 驱动，但是为了教程的连贯性，这里我们再讲解一遍！

1、将驱动文件添加到 Linux 内核

首先需要将 28_YT8511C_PHY 目录里面的 linux 驱动文件拷贝到 Linux 内核源码相应的目录中。这里一共有 motorcomm.c 和 motorcomm_phy.h 这两个文件，一个是驱动 C 文件，一个是头文件。我们把 motorcomm.c 拷贝到 drivers/net/phy 下，把 motorcomm_phy.h 拷贝到 include/linux 目录下。

这样我们就把 YT8511 的驱动文件和头文件添加到了 Linux 内核源码中。

2、修改 Makefile

前面我们已经将 YT8511 的驱动文件添加到 Linux 内核源码里面了，但是还不能编译，因为还没有添加到 Makefile 里面。在 phy 驱动的源码目录 drivers/net/phy，打开 Makefile 文件，添加如下代码：

```
obj-$(CONFIG_MOTORCOMM_PHY) += motorcomm.o
```

添加结果如下所示：

```
96  obj-$(CONFIG_XILINX_GMII2RGMII) += xilinx_gmii2rgmii.o
97  obj-$(CONFIG_MOTORCOMM_PHY) += motorcomm.o
98
```

图 52.5.6.1 在 Makefile 添加 YT8511 驱动文件

3、修改 Kconfig

最后修改一下 drivers/net/phy/Kconfig 这个文件，这样我们以后就可以通过图形化界面来使能或者禁止 YT8511 驱动了。在 Kconfig 里面添加如下内容：

```
config MOTORCOMM_PHY
    tristate "Motorcomm PHYs"
    ---help---
      Supports the YT8010, YT8510, YT8511, YT8512 PHYs.
```

添加结果如下所示：

```
487 config XILINX_GMII2RGMII
488     tristate "Xilinx GMII2RGMII converter driver"
489     ---help---
490       This driver support xilinx GMII to RGMII IP core it provides
491       the Reduced Gigabit Media Independent Interface(RGMII) between
492       Ethernet physical media devices and the Gigabit Ethernet controller.
493
494 config MOTORCOMM_PHY
495     tristate "Motorcomm PHYs"
496     ---help---
497       Supports the YT8010, YT8510, YT8511, YT8512 PHYs.
498
499 endif # PHYLIB
```

图 52.5.6.2 在 Kconfig 添加 YT8511C

3、使能 YT8511C PHY 驱动

最后就要进入内核的图形界面去使能 YT8511 驱动，配置路径如下：

```
-> Device Drivers
    -> Network device support (NETDEVICES [=y])
        -> PHY Device support and infrastructure (PHYLIB [=y])
            -> <*> Motorcomm PHYs    //选中
```

配置界面如图所示：

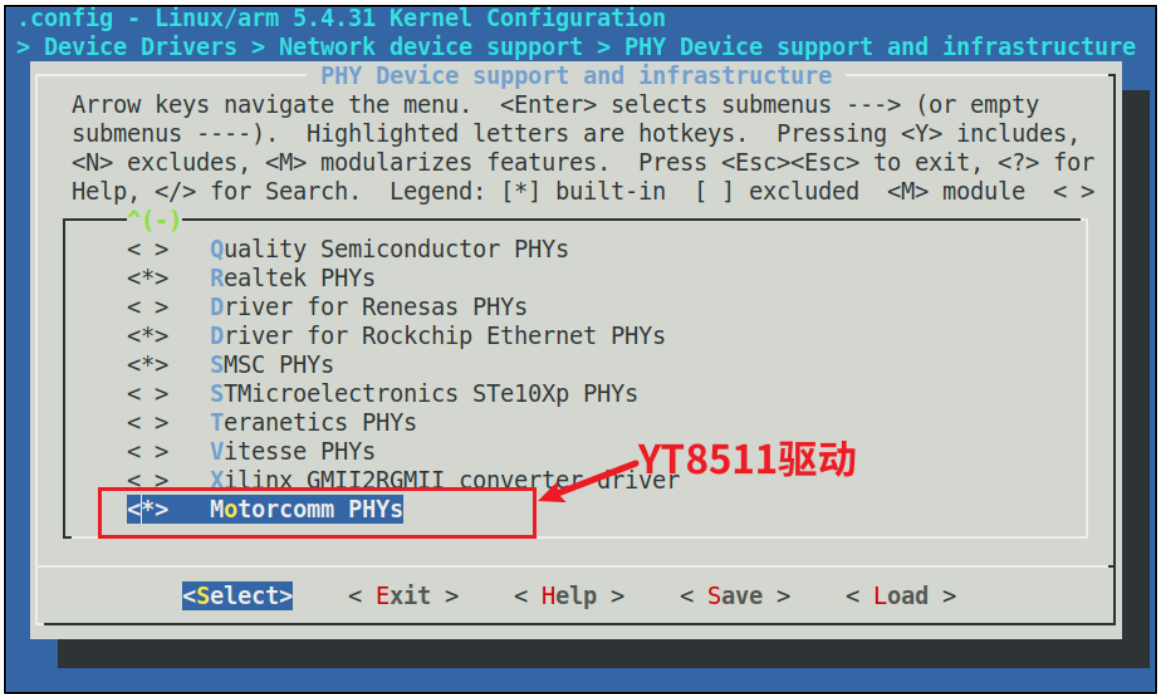


图 52.5.6.3 使能 YT8511 驱动

使能以后就可以重新编译 Linux 内核，然后使用新的 Linux 内核启动开发板。

4、YT8511C 驱动源码简析

打开 motorcomm.c 文件，找到 YT8511 驱动结构体，代码如下所示：

示例代码 52.5.6.1 YT8511 PHY 驱动结构体

```
1  static struct phy_driver ytphy_drvs[] = {
.....
20      }, {
21          .phy_id         = PHY_ID_YT8511,
22          .name           = "YT8511 Gigabit Ethernet",
23          .phy_id_mask    = MOTORCOMM_PHY_ID_MASK,
24          .features       = PHY_GBIT_FEATURES,
25          //.flags        = PHY_HAS_INTERRUPT,
26          .config_aneg    = genphy_config_aneg,
27  #if GMAC_CLOCK_INPUT_NEEDED
28          .config_init    = yt8511_config_init,
29  #else
30          .config_init    = genphy_config_init,
```

```
31  #endif
32          .read_status    = genphy_read_status,
33          .suspend        = genphy_suspend,
34          .resume         = genphy_resume,
35      }, {
......
71  };
72  module_phy_driver(ytphy_drvs);
```

第 21 行，PHY_ID_YT8511 是一个宏，定义为 0x0000010a，是 PHY ID。

第 22 行，驱动名字为"YT8511 Gigabit Ethernet"，系统启动过程中，加载网络设备驱动的时候就会提示电气 PHY 驱动文字为"YT8511 Gigabit Ethernet"。

第 23 行，PHY 的 ID 掩码，MOTORCOMM_PHY_ID_MASK 是一个宏，定义为 0x00000fff，也就是前 12 位有效，在进行匹配的时候只需要比较前 12 位。

最后，第 72 行使用 module_phy_driver(本质是一个宏)来完成 ytphy_drvs 的注册，以 genphy 开头的函数都是通用 PHY 驱动。

52.6 网络驱动实验测试

52.6.1 RTL8211F PHY 驱动测试

前面我们已经大概了解多网络框架，PHY 驱动内核已经使能了，我们也不需要做任何的修改；系统启动以后就会打印当前 PHY 驱动名字为“RTL8211F Gigabit Ethernet”，如下图所示：

```
rtc-pcf8563 4-0051: hctosys: unable to read the hardware clock
stm32-dwmac 5800a000.ethernet eth0: PHY [stmmac-0:00] driver [RTL8211F Gigabit Ethernet]
dwmac4: Master AXI performs any burst length
stm32-dwmac 5800a000.ethernet eth0: No Safety Features support found
stm32-dwmac 5800a000.ethernet eth0: IEEE 1588-2008 Advanced Timestamp supported
stm32-dwmac 5800a000.ethernet eth0: registered PTP clock
stm32-dwmac 5800a000.ethernet eth0: configuring for phy/rgmii-id link mode
fusb302 0-0022: PD disabled
stm32-dwmac 5800a000.ethernet eth0: Link is Up - 1Gbps/Full - flow control rx/tx
IP-Config: Complete:
```

图 52.5.6.1 TRL8211F PHY 驱动信息

从图 52.5.6.1 可以知道，此时 PHY 网络使用的是“RTL8211F Gigabit Ethernet”。由于我的开发板连接到了千 M 路由器上，因此此时的网速为 1Gpbs，也就是 1000M 网络。测试网络就很简单了，我们一直都是用网络在挂载文件系统的，一直在测试中。

52.6.2 YT8511C PHY 驱动测试

在 52.5.6 小节笔者带大家如何把 YT8511C PHY 驱动加进入内核里，运行以下命令重新编译内核：

```
make uImage LOADADDR=0XC2000040 -j8
```

使用新的 uImage 内核，重新启动开发板，当系统驱动以后就打印出当前 PHY 驱动名字为“YT8511 Gigabit Ethernet”，如下图所示：

```
rtc-pcf8563 4-0051: hctosys: unable to read the hardware clock
stm32-dwmac 5800a000.ethernet eth0: PHY [stmmac-0:00] driver [YT8511 Gigabit Ethernet]
dwmac4: Master AXI performs any burst length
stm32-dwmac 5800a000.ethernet eth0: No Safety Features support found
stm32-dwmac 5800a000.ethernet eth0: IEEE 1588-2008 Advanced Timestamp supported
stm32-dwmac 5800a000.ethernet eth0: registered PTP clock
stm32-dwmac 5800a000.ethernet eth0: configuring for phy/rgmii-id link mode
fusb302 0-0022: PD disabled
stm32-dwmac 5800a000.ethernet eth0: Link is Up - 1Gbps/Full - flow control rx/tx
IP-Config: Complete:
```

图 52.5.6.2 YT8511C PHY 驱动信息

从图 52.5.6.2 可以知道，此时 PHY 网络使用的是“YT8511 Gigabit Ethernet”。由于我的开发板连接到了千 M 路由器上，因此此时的网速为 1Gpbs，也就是 1000M 网络。测试网络就很简单了，我们一直都是用网络在挂载文件系统的，一直在测试中。

52.6.3 千兆以太网网速测试

注意：这里要连接千兆路由器或者千兆交换机，没有这两个设备可以开发板和电脑直接连接。

STM32MP1 网络接口为千兆网络，理论最大通信速率可达 1000mb/s，RTL8211 和 YT8511 也是千 M PHY 芯片。本小节我们就来测试下这个网口的速率，此时过程我们使用开发板和 PC 机直连方式，使用千兆网线，比如 CAT-5E 类网线或 CAT-6 类网线。

Linux 下网络速率测试可以使用 iper3 工具，iperf3 是一个网络性能测试工具，可以测试 TCP 和 UDP 的带宽质量、测量 TCP 最大带宽、报告带宽、延迟抖动和数据丢包等；这里我们仅仅只是使用 iperf3 工具测试开发板上网口的带宽，其它的一些参数特性就不给大家进行详细测试了。

iperf3 的测试方法需要有一台主机作为服务器端，另一台主机作为客户端，客户端向服务器端发送数据，这里我们可以将 Ubuntu 系统作为服务器端，而开发板作为客户端进行测试。

1、使能 iperf3 命令

跳转到 buildroot 源码目录下，启动到图形配置选项，按照以下配置去是能 iperf3 命令：

```
-> Target packages
    -> Networking applications
        ->  [*] iperf3  //选中
```

配置如下图所示：

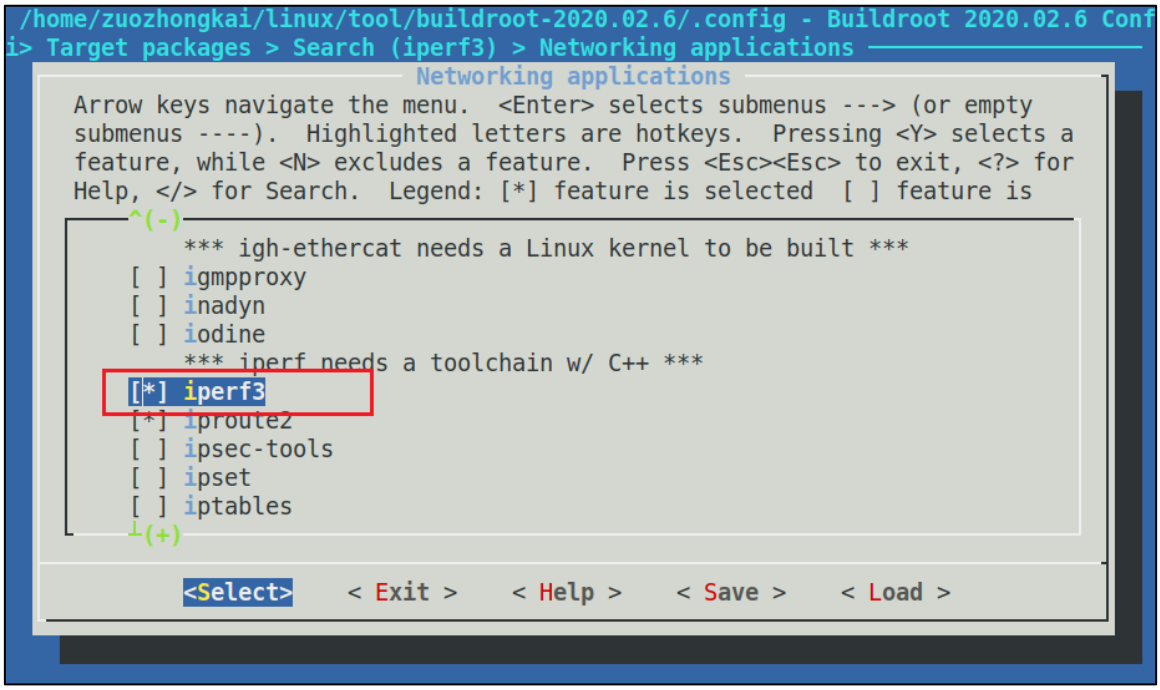


图 52.6.3.1 使能 iperf3 命令

保存配置项，重新编译新的文件系统，使用新的文件系统启动开发板。

2、Ubuntu 下安装 iperf3 工具

前面我们已经将 iperf3 工具移植到开发板了，接下来我们需要在 Ubuntu 下也安装 iperf3 工具，因为 Ubuntu 系统默认没有安装这个工具，在 Ubuntu 系统中执行下面这条命令安装 iperf3：

```
sudo apt-get install iperf3
```

安装完成之后就可以进行测试了！

3、测试

在 Ubuntu 系统中执行下面这条命令，将 Ubuntu 系统作为 iperf3 的服务器，如下所示：

```
iperf3 -s
```

命令执行如下图所示：

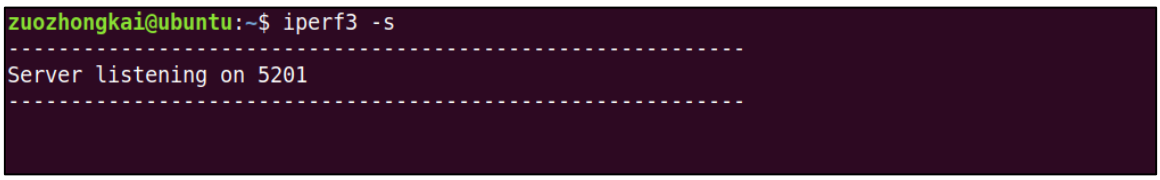


图 52.6.3.2 将 Ubuntu 作为服务器

-s 表示将其作为服务器端。

接下来我们需要在开发板终端执行下面这条命令，将开发板作为客户端，并进行测试：

```
iperf3 -c 192.168.1.250
```

-c 选项表示将其作为客户端，后面紧跟着的就是服务器主机的 IP 地址，在这里也就是 Ubuntu 系统的 IP 地址，所以这个 IP 地址大家根据自己 Ubuntu 系统的 IP 进行填写即可，测试结果如下图所示：

```
[root@ATK-stm32mp1]:~$ iperf3 -c 192.168.1.250
Connecting to host 192.168.1.250, port 5201
[  5] local 192.168.1.130 port 58090 connected to 192.168.1.250 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec   112 MBytes   939 Mbits/sec  169    584 KBytes
[  5]   1.00-2.00   sec   112 MBytes   938 Mbits/sec    0    669 KBytes
[  5]   2.00-3.00   sec   112 MBytes   941 Mbits/sec    0    735 KBytes
[  5]   3.00-4.00   sec   110 MBytes   923 Mbits/sec   72    560 KBytes
[  5]   4.00-5.00   sec   112 MBytes   943 Mbits/sec    0    594 KBytes
[  5]   5.00-6.00   sec   112 MBytes   936 Mbits/sec    0    618 KBytes
[  5]   6.00-7.00   sec   110 MBytes   925 Mbits/sec    0    626 KBytes
[  5]   7.00-8.00   sec   113 MBytes   944 Mbits/sec    0    631 KBytes
[  5]   8.00-9.00   sec   112 MBytes   941 Mbits/sec    0    641 KBytes
[  5]   9.00-10.00  sec   110 MBytes   922 Mbits/sec    0    680 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  1.09 GBytes   935 Mbits/sec  241             sender
[  5]   0.00-10.00  sec  1.09 GBytes   934 Mbits/sec                  receiver

iperf Done.
[root@ATK-stm32mp1]:~$
```

图 52.6.3.3 开发板作为客户端测试结果

图 52.6.3.3 中执行命令共打印出了 10 次带宽测试报告，每一秒钟报告一次带宽，耗费时间为 10 秒钟。最终测试出来发送和接收带宽分别为 935M 和 934M，基本接近 1000M 的带宽，说明千 M 网络工作正常。

第五十三章 Linux WIFI 驱动实验

WIFI 的使用已经很常见了，手机、平板、汽车等等，虽然可以使用有线网络，但是有时候很多设备存在布线困难的情况，此时 WIFI 就是一个不错的选择。正点原子 STM32MP1 开发板支持 USB 和 SDIO 这两种接口的 WIFI，本章我们就来学习一下如何在 STM32MP1 开发板上使用 USB 和 SDIO 这两种 WIFI。

53.1 硬件原理图分析

正点原子的 STM32MP1 开发板板载了 SDIO WIFI 使用芯片的为 RTL8723DS，原理图如下所示：

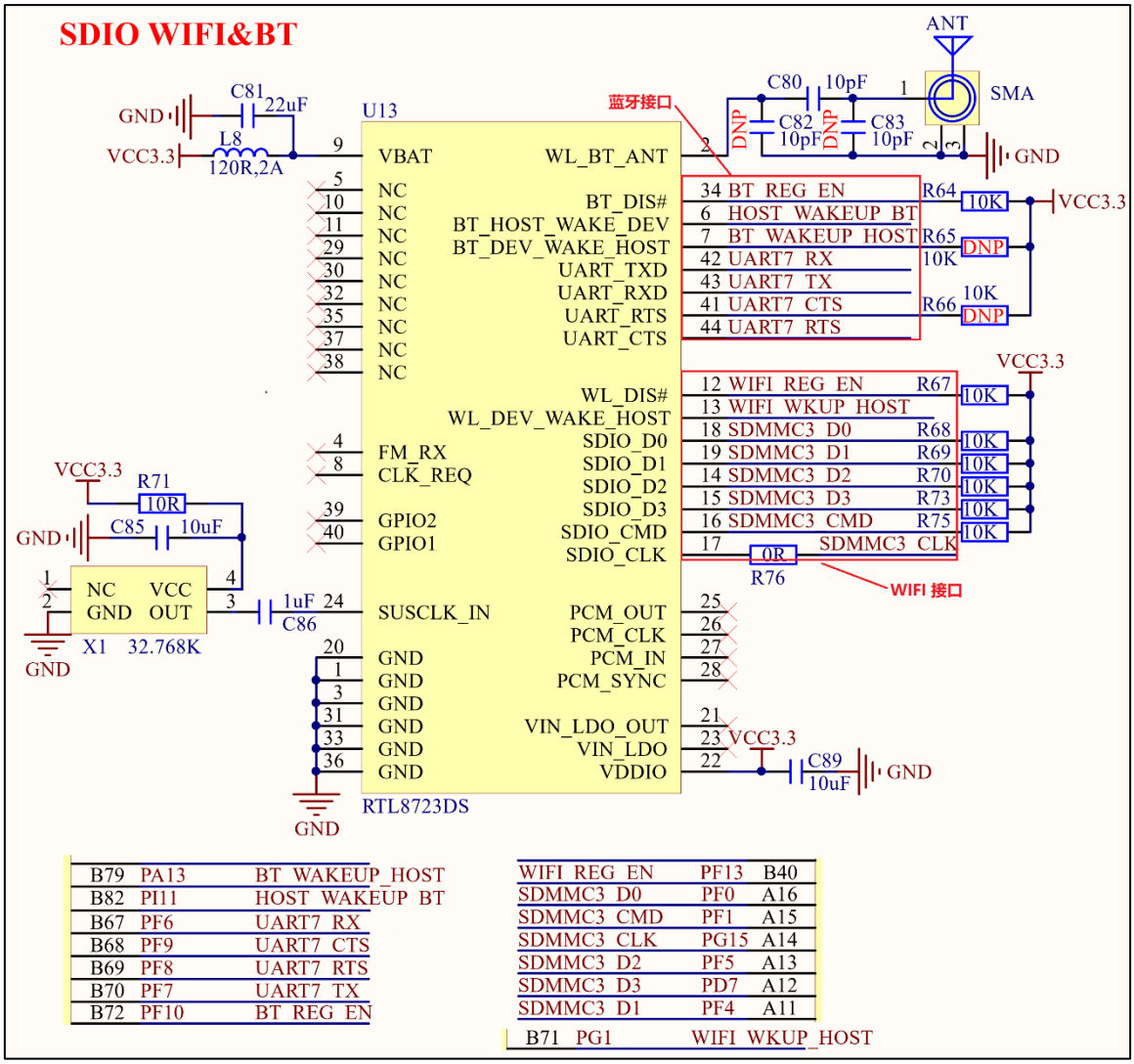


图 53.1.1 RTL8732DS 原理图

从图 53.1.1 中可以看出，板载的 WIFI 为 SDIO 接口，D0~4 所对应的硬件引脚为：PF0、PF4、PF5 和 PD7，CMD 对应的硬件引脚为 PF1，CLK 所对应的硬件引脚为 PG15。

53.2 WIFI 驱动添加与编译

正点原子的 STM32MP1 开发板目前支持两种接口的 WIFI：USB 和 SDIO，其中 USB WIFI 使用的芯片为 RTL8188EUS，SDIO 接口的 WIFI 使用芯片为 RTL8723DS，这两个都是 realtek 公司出品的 WIFI 芯片。WIFI 驱动不需要我们编写，因为 realtek 公司提供了 WIFI 驱动源码，因此我们只需要将 WIFI 驱动源码添加到 Linux 内核中，然后通过图形界面配置，选择将其编译成模块即可。

正点原子的 STM32MP1 开发板上有一个板载 RTL8723DS WIFI，接口为 SDIO。RTL8723DS 芯片是 WIFI 和蓝牙二合一，这一章只讲解 WIFI。正点原子 STM32MP157 开发板上的

RTL8723DS 模组有两种封装形式的，如下图所示：

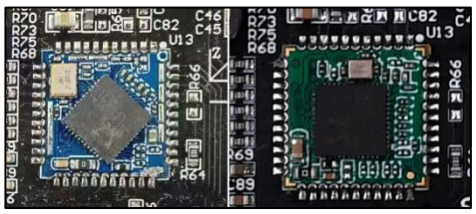


图 53.2.1 RTL8723DS 板载 SDIO WIFI

图 53.2.1 中这两个都是 RTL8723DS 模组，封装、功能一模一样，只是元器件布局不同而已。另外，正点原子还有一款采用 RTL8188EUS 芯片的 USB WIFI，如下图所示：

图 53.2.2 RTL8188 USB WIFI

53.2.1 向 Linux 内核添加 WIFI 驱动

WIFI 驱动源码已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料→1、程序源码→2、Linux 驱动例程→29_WIFI。这个目录只是存放着 RTL8723DS 这个芯片的驱动源码，RTL8188EUS 是直接使用内核自带的驱动源码即可。

1、驱动移植

将 29_WIFI 目录下的 rtl8723ds 目录拷贝到 ubuntu 下 Linux 内核源码中的 drivers/net/wireless/realtek/目录下，此目录存放着 Linux 主线上的所有 realtek 公司的 WIFI 驱动文件，拷贝完成以后此目录如图 53.2.1.1 所示：

```
zuozhongkai@ubuntu:~/linux/atk-mp1/linux/my_linux/linux-5.4.31/drivers/net/wireless/realtek$ ls
built-in.a  Makefile         modules.order  rtl8723ds  rtlwifi
Kconfig     modules.builtin  rtl818x        rtl8xxxu   rtw88
```

图 53.2.1.1 拷贝后的 realtek 目录

2、修改 drivers/net/wireless/realtek/Kconfig

打开 drivers/net/wireless/ realtek/Kconfig，在里面加入下面这一行内容：

```
source "drivers/net/wireless/realtek/rtl8723ds/Kconfig"
```

添加完成以后的如图 53.2.1.2 所示：

```
18  source "drivers/net/wireless/realtek/rtw88/Kconfig"
19  source "drivers/net/wireless/realtek/rtl8723ds/Kconfig"
20
21  endif # WLAN_VENDOR_REALTEK
22
```

图 53.2.1.2 修改后的 Kconfig

3、修改 drivers/net/wireless/realtek/Makefile

打开 drivers/net/wireless/realtek/Makefile，在里面加入下面以后内容：

```
obj-$(CONFIG_RTL8723DS)        += rtl8723ds/
```

修改以后的 Makefile 文件如图 53.2.1.3 所示：

```
10  obj-$(CONFIG_RTW88)      += rtw88/
11  obj-$(CONFIG_RTL8723DS)      += rtl8723ds/
12
```

图 53.2.1.3 修改后的 Makefile

图 53.2.1.3 中第 11 行，这里新添加一个内核配置，会根据 CONFIG_RTL8723DS 这个宏来决定 8723ds WIFI 驱动是编译进内核还是模块。

53.2.2 配置 USB 支持设备

在编译 RTL8188 和 RTL8723 驱动之前需要先配置 Linux 内核。

1、配置 USB 支持设备

USB 配置默认已经选中了，这里只是给大家讲一下都有哪些要配置，大家也可以检查一下，配置路径如下：

```
-> Device Drivers
    -> <*> USB support
        -> <*> Support for Host-side USB                    //选中
            -> <*> EHCI HCD (USB 2.0) support               //选中
            -> <*> OHCI HCD (USB 1.1) support               //选中
            -> <*> ChipIdea Highspeed Dual Role Controller  //选中
                ->  [*] ChipIdea device controller->        //选中
                ->  [*] ChipIdea host controller            //选中
```

2、配置支持 WIFI 设备

配置路径如下：

```
-> Device Drivers
    -> [*] Network device support
        -> [*] Wireless LAN
            -> <*> IEEE 802.11 for Host AP (Prism2/2.5/3 and WEP/TKIP/CCMP)    //选中
                -> [*] Support downloading firmware images with Host AP driver  //选中
                -> [*] Support for non-volatile firmware download               //选中
```

配置完如图 53.2.2.1 所示：

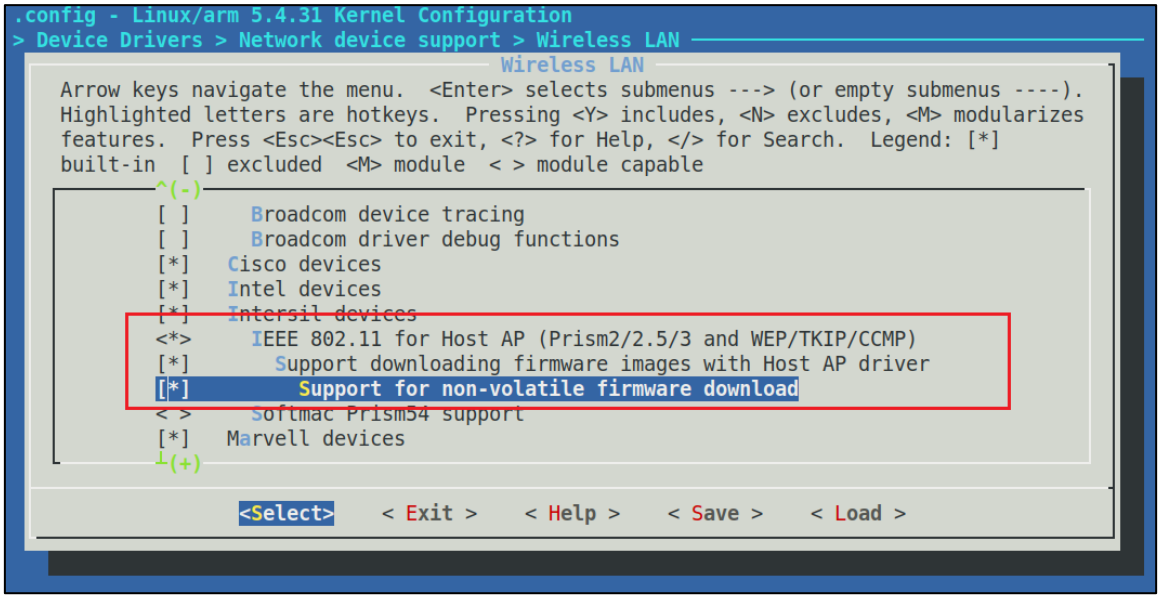


图 53.2.2.1 配置支持 WIFI 设备

3、配置支持 IEEE 802.11

配置路径如下：(注意：是配置成模块，不是编译进内核)

-> Networking support (NET [=y])
-> Wireless (WIRELESS [=y])
-> <M> cfg80211 - wireless configuration API //编译成模块

配置完如图 53.2.2.2 所示：

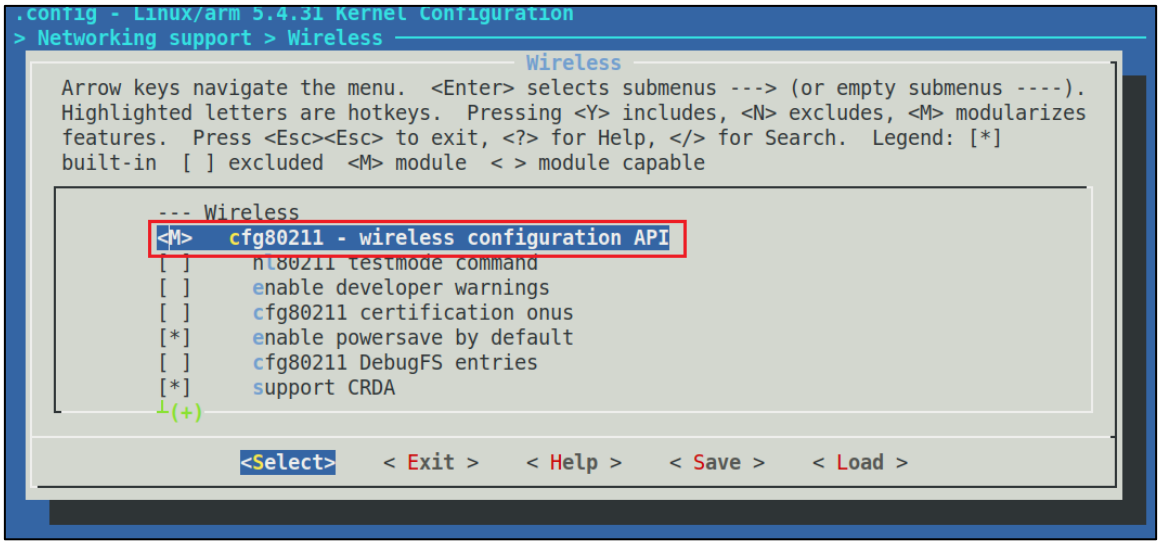


图 53.2.2.2 IEEE 802.11 配置项

4、使能 STAGING 配置

配置路径如下：

-> Device Drivers
-> [*] Staging drivers //选中

配置完如图 53.2.2.3 所示：

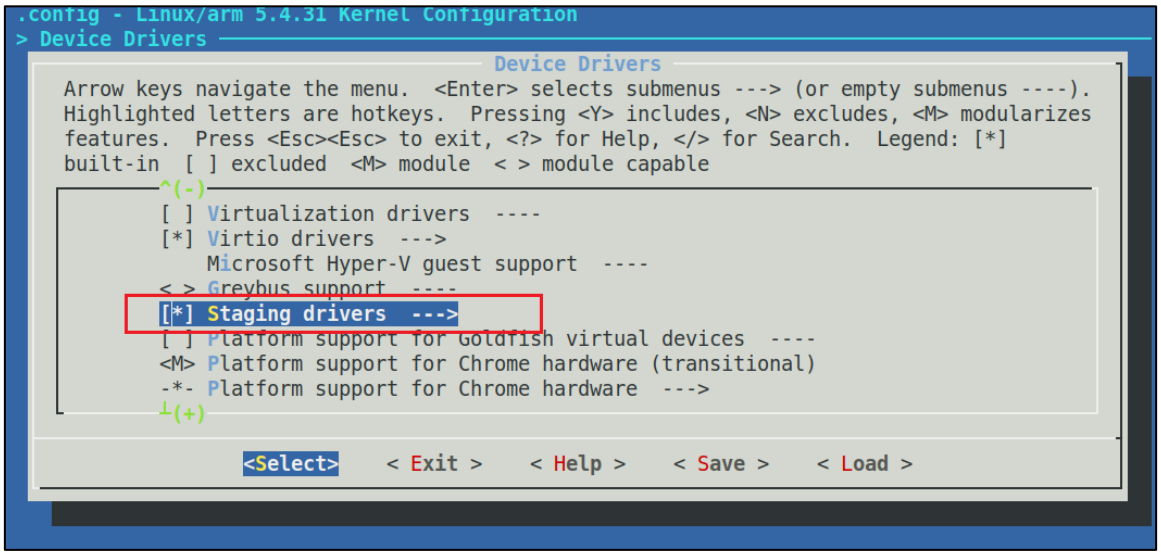


图 53.2.2.3 使能 STAGING 配置

配置好以后重新编译一下 Linux 内核，得到新的 uImage，后面使用新的编译出来的 uImage 启动系统。

53.2.3 设备树的配置

1、pinctrl 的配置

首先肯定是 sdmmc3 引脚配置信息，打开 stm32mp15-pinctrl.dtsi，找到如下所示内容：

示例代码 53.2.3.1 sdmmc3 引脚配置

```
1   sdmmc3_b4_pins_a: sdmmc3-b4-0 {
2       pins1 {
3           pinmux = <STM32_PINMUX('F', 0, AF9)>,/* SDMMC3_D0  */
4                <STM32_PINMUX('F', 4, AF9)>,     /* SDMMC3_D1  */
5                <STM32_PINMUX('F', 5, AF9)>,     /* SDMMC3_D2  */
6                <STM32_PINMUX('D', 7, AF10)>,    /* SDMMC3_D3  */
7                <STM32_PINMUX('F', 1, AF9)>;     /* SDMMC3_CMD */
8           slew-rate = <1>;
9           drive-push-pull;
10          bias-pull-up;
11      };
12      pins2 {
13          pinmux = <STM32_PINMUX('G', 15, AF10)>; /* SDMMC3_CK */
14          slew-rate = <2>;
15          drive-push-pull;
16          bias-pull-up;
17      };
18  };
```

示例代码 53.1.3.1 中就是 sdmmc3 的默认引脚配置，这个是 ST 官方根据自己的 EVK 开发板编写的，和正点原子 STM32MP157 开发板上的 SDMMC3 引脚一直，所以不需要修改！

2、sdmmc3 控制器节点信息

打开 stm32mp151.dtsi 文件，找到名为“sdmmc3”的节点，内容如下：

示例代码 53.2.3.2 sdmmc3 节点

```
1  sdmmc3: sdmmc@48004000 {
2      compatible = "arm,pl18x", "arm,primecell";
3      arm,primecell-periphid = <0x00253180>;
4      reg = <0x48004000 0x400>, <0x48005000 0x400>;
5      interrupts = <GIC_SPI 137 IRQ_TYPE_LEVEL_HIGH>;
6      interrupt-names = "cmd_irq";
7      clocks = <&rcc SDMMC3_K>;
8      clock-names = "apb_pclk";
9      resets = <&rcc SDMMC3_R>;
10     cap-sd-highspeed;
11     cap-mmc-highspeed;
12     max-frequency = <120000000>;
13     status = "disabled";
14 };
```

示例代码 53.1.3.2 中的 sdmmc3 节点不需要我们修改，这里只是告诉大家 sdmmc3 完整节点信息。从第 13 行可以看出，sdmmc3 设备默认是关闭的。

3、使能 sdmmc3 节点

打开 stm32mp157d-atk.dts 文件，用追加代码的方式去使能 sdmmc3 节点，追加的内容如下所示：

示例代码 53.2.3.3 要追加的 sdmmc3 节点

```
1  &sdmmc3 {
2      pinctrl-names = "default", "opendrain", "sleep";
3      pinctrl-0 = <&sdmmc3_b4_pins_a>;
4      pinctrl-1 = <&sdmmc3_b4_od_pins_a>;
5      pinctrl-2 = <&sdmmc3_b4_sleep_pins_a>;
6      non-removable;
7      st,neg-edge;
8      bus-width = <4>;
9      vmmc-supply = <&v3v3>;
10     status = "okay";
11     keep-power-in-suspend;
12 };
```

示例代码 53.2.3.3 给 sdmmc3 节点追加了 pinctrl 配置和使能了 sdmmc3。

重点来了，后面说的是笔者遇到一个坑。

大家都知道 STM32MP1 有三个 SDMMC 接口，它们的物理地址分别为：0x58005000~0x58005FFF、0x58007000~0x58007FFF 和 0x48004000~0x480043FF。从它们的物理地址可以知道，它们的初始化顺序是根据物理地址：SDMMC3→SDMMC1→SDMMC2。假如三个 SDMMC 接口都使能，那么在 Linux 里所对应的操作文件为：/dev/mmcblk0 (SDMMC3)、/dev/mmcblk1 (SDMMC1)和/dev/mmcblk2 (SDMMC2)。要注意，出厂系统 uboot 启动 linux 的时

候，启动 EMMC 上的文件系统使用的是 mmcblk2。但是在本教程前面讲解系统移植的时候说过挂载 EMMC 里面文件系统的时候使用 mmcblk1，那是因为那个时候还没有使能 SDMMC3 这个接口，所以 mmcblk1 就是 EMMC。

53.2.4 编译 WIFI 驱动

执行“make menuconfig”命令，打开 Linux 内核配置界面，然后按照如下路径选择将 rtl8723ds 和 rtl8188eus 驱动编译成模块。

1、rtl8723ds 驱动配置

配置路径如下：

```
-> Device Drivers
    -> Network device support (NETDEVICES [=y])
        -> Wireless LAN (WLAN [=y])
            -> Realtek devices (WLAN_VENDOR_REALTEK [=y])
                -> <M> Realtek 8723D SDIO or SPI WiFi  //编译为模块
```

配置完如图 53.2.4.1 所示：

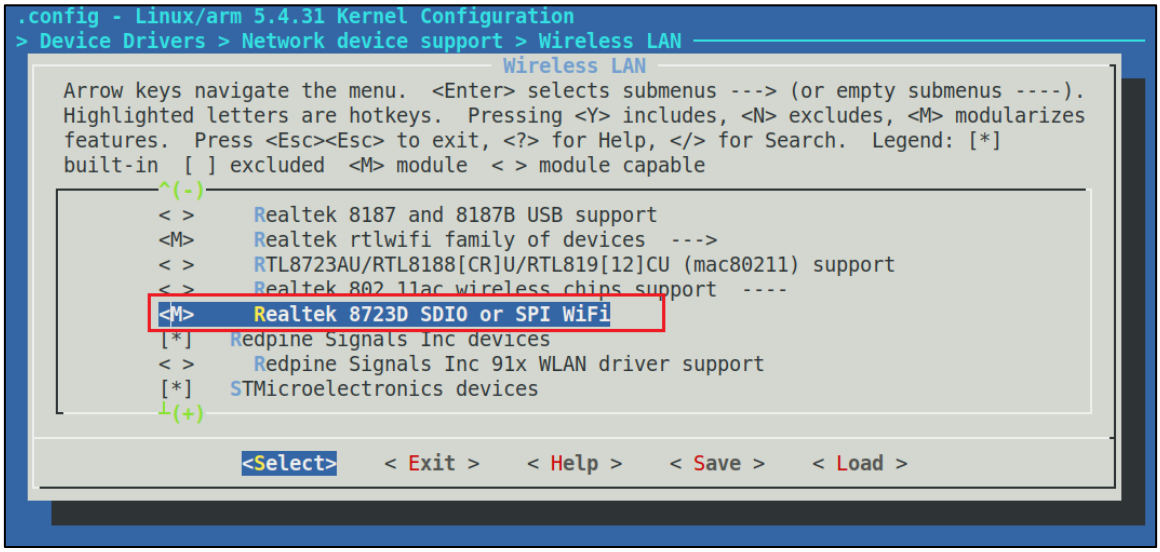


图 53.2.4.1 把 rtl8723ds 配置成模块

2、rtl8188eus 驱动配置

配置路径如下：

```
-> Device Drivers
    -> Staging drivers (STAGING [=y])
        -> <M> Realtek RTL8188EU Wireless LAN NIC driver
```

配置完如图 53.2.4.2 所示：

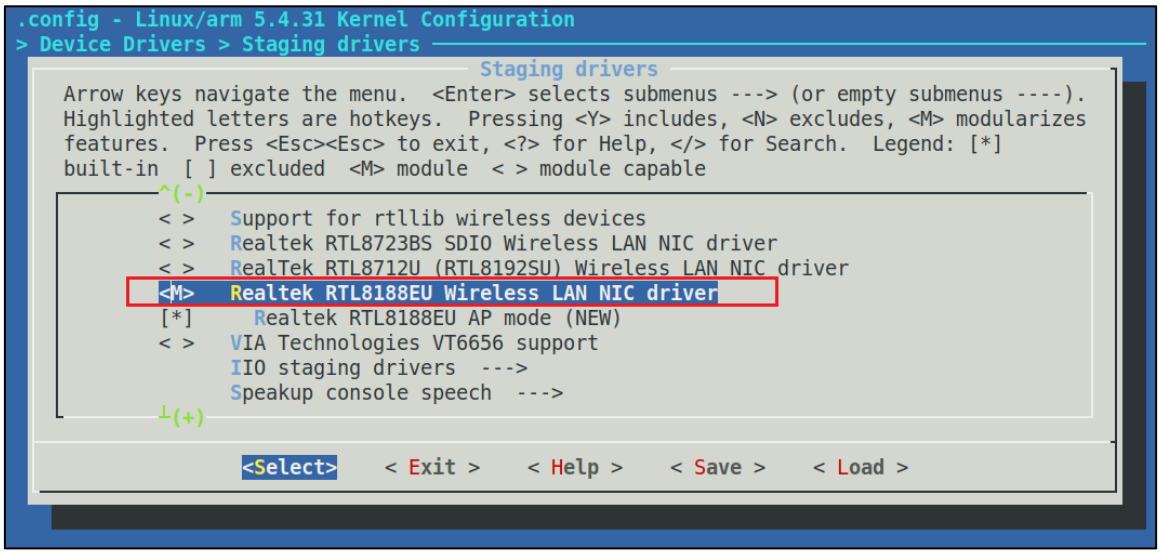


图 53.2.4.2 把 rtl8188eus 配置成模块

我们已经两种 WIFI 都配置成模块，接着就用以下命令去编译成模块：

```
make dtbs                  //重新编译设备树
make modules -j12          //编译驱动模块
```

在本章节里配置了三处为模块，分别是：IEEE 802.11、rtl8723ds 和 rtl8188eus。它们所对应的模块名为：cfg8011.ko、8723ds.ko 和 r8188eu.ko。

编译完成后在所对应的目录下生成模块文件。比如：rtl8188eus WIFI 驱动源码在 linux 源码下 drivers/staging/rtl8188eu/目录里，所以 rtl8188eus 的模块文件就在此目录下，如下图所示：

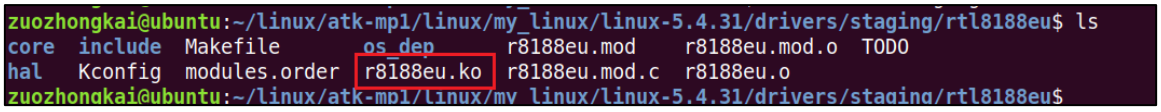


图 53.2.4.3 rtl8188eus 编译结果

在内核的源码目录下，把三个模块拷贝到开发板 nfs 根目录下，拷贝命令如下：

```
sudo cp drivers/staging/rtl8188eu/r8188eu.ko ~/linux/nfs/rootfs/lib/modules/5.4.31/
sudo cp net/wireless/cfg80211.ko ~/linux/nfs/rootfs/lib/modules/5.4.31/
sudo cp drivers/net/wireless/realtek/rtl8723ds/8723ds.ko ~/linux/nfs/rootfs/lib/modules/5.4.31/
```

因为我们重新配置过 Linux 内核和设备树，因此也需要使用新的 uImage 启动和设备树，将新编译出来的 uImage 镜像文件和 stm32mp157d-atk.dtb 拷贝到 ubuntu 中的 tftpboot 目录下。

53.2.5 驱动加载测试

1、固件的拷贝

有些模块是需要固件配合使用的，比如 cfg80211.ko 驱动模块要读取/lib/firmware/regulatory.db 文件。把本章节要用到的固件都开发板光盘中，路径：开发板光盘 A-基础资料→1、程序源码→2、Linux 驱动例程→29_WIFI，如下图所示：

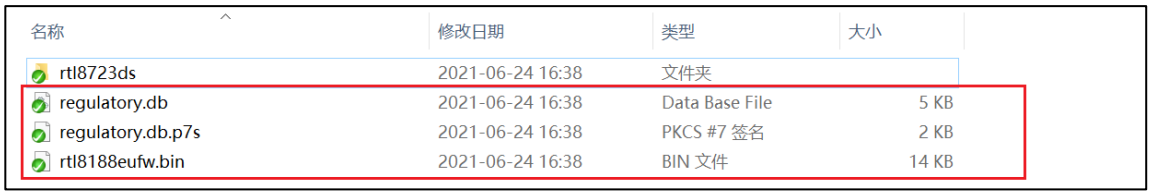


图 53.2.5.1 固件的路径

regulatory.db 固件和 regulatory.db.p7s 固件都是 cfg80211.ko 模块需要加载的。rtl8188wufw.bin 固件是 r8188.ko 模块需要加载的。

在开发板根文件系统中创建存放固件的文件夹，这里我们创建“lib/firmware/rtlwifi”目录来存放固件，nfs 挂载的文件系统就直接在 ubuntu 系统下运行以下命令：

```
sudo mkdir ~/linux/nfs/rootfs/lib/firmware/rtlwifi -p
```

把 regulatory.db 和 regulatory.db.p7s 固件文件拷贝到刚刚创建的 firmware 目录下，把 rtl8188wufw.bin 固件拷贝 rtlwifi 目录下。拷贝完结果如下所示：

```
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib/firmware$ ls
regulatory.db  regulatory.db.p7s  rtlwifi
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib/firmware$ ls rtlwifi/
rtl8188eufw.bin
zuozhongkai@ubuntu:~/linux/nfs/rootfs/lib/firmware$
```

图 53.2.5.2 拷贝固件后的结果

2、RTL8723 SDIO WIFI 驱动测试

重启开发板以后，首先我们先测试一下板载的 SDIO WIFI。进入到目录/lib/modules/5.4.31 中，运行以下命令进行加载 8723ds.ko 这个驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe cfg80211.ko    //先加载 cfg80211.ko，IEEE 协议
modprobe 8723ds.ko      //RTL8723DS 模块加载 8723ds.ko 模块
```

如果驱动加载成功的话如图所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe cfg80211.ko
cfg80211: Loading compiled-in X.509 certificates for regulatory database
cfg80211: Loaded X.509 cert 'sforshee: 00b28ddf47aef9cea7'
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe 8723ds.ko
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 53.2.5.3 RTL8723DS 驱动加载成功

在加载 cfg80211.ko 模块，没有固件就会报以下图警告，还是可以使用的：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe cfg80211.ko
cfg80211: Loading compiled-in X.509 certificates for regulatory database
cfg80211: Loaded X.509 cert 'sforshee: 00b28ddf47aef9cea7'
platform regulatory.0: Direct firmware load for regulatory.db failed with error -2
cfg80211: failed to load regulatory.db
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 53.2.5.4 加载 cfg80211.ko 模块报的警告

输入“ifconfig -a”命令，查看 wlanX(X=0…n)网卡是否存在，一般都是 wlan0，除非板子上有多个 WIFI 模块在工作，结果如图 53.1.5.5 所示：

```
[root@ATK-stm32mp1]:~$ ifconfig -a
can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:51

eth0      Link encap:Ethernet  HWaddr B8:AE:1D:01:01:00
          inet addr:192.168.1.33  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:14987 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7943 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:20493454 (19.5 MiB)  TX bytes:1178986 (1.1 MiB)
          Interrupt:48 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

wlan0     Link encap:Ethernet  HWaddr 30:4A:26:46:E3:4A
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[root@ATK-stm32mp1]:~$
```

图 53.2.5.5 当前开发板所有网卡

从图 53.2.5.5 中可以看出，当前开发板有一个叫做“wlan0”的网卡，这个就是 RTL8723DS 对应的网卡。

注意！后面重启开发板的话会自动加载 RTL8723 驱动，不需要我们再手动加载！！

3、RTL8188EUS USB WIFI 驱动测试

接着我们把 RTL8188 USB WIFI 接到开发板上，运行以下命令进行加载：

```
depmod
modprobe r8188eu.ko
```

如果驱动加载成功的话如图 53.2.5.6 所示：

```
r8188eu: module is from the staging directory, the quality is unknown, you have been warned.
Chip Version Info: CHIP_8188E_Normal_Chip_TSMC_D_CUT_1T1R_RomVer(0)
usbcore: registered new interface driver r8188eu
```

图 53.2.5.6 RTL8188EUS 驱动加载成功

从 53.2.5.6 可以看出，RTL8188EUS USB WIFI 驱动加载成功，同样使用“ifconfig -a”命令查看一下是否有 wlanX(X=0…n)网卡存在，如果有的话就说明 RTL8188 USB WIFI 驱动工作正常。

同样的，以后重启开发板的话会自动加载 RTL8188 驱动，不需要我们再手动加载！！

不管是 RTL8188 USB WIFI 还是 RTL8723 SDIO WIFI，驱动测试都工作正常，但是我们得能联网啊，不能联网的话要他有什么用呢？WIFI 要想联网，需要移植一些其他第三方组件，否则无法连接路由器。

53.3 WIFI 联网测试

53.3.1 buildroot WIFI 工具配置

首先要给文件系统添加一些操作 WIFI 的工具。进入 buildroot 源码目录里，接着运行“make menuconfig”进入图形配置界面，配置如下：

```
→ Target packages
    → Networking applications
        → [*] wireless tools                                    //选中
            →[*]    Install shared library                      //选中
        → [*] wpa_supplicant                                    //选中
            → [*]   Enable nl80211 support                      //选中
            →[*]    Enable AP mode                              //选中
                → [*] Enable Wi-Fi Display                      //选中
                → [*] Enable mesh networking                    //选中
            → [*] Enable autoscan                               //选中
            → [*] Enable EAP                                    //选中
            → [*] Enable HS20                                   //选中
            → [*] Enable syslog support                         //选中
            → [*] Enable WPS                                    //选中
            → [*] Enable WPA3 support                           //选中
            → [*] Install wpa_cli binary                        //选中
            → [*] Install wpa_client shared library             //选中
            → [*] Install wpa_passphrase binary                 //选中
            → [*] Enable support for the DBus control interface //选中
            → [*] Introspection support                         //选中
```

配置完成如图 53.3.1.1 所示：

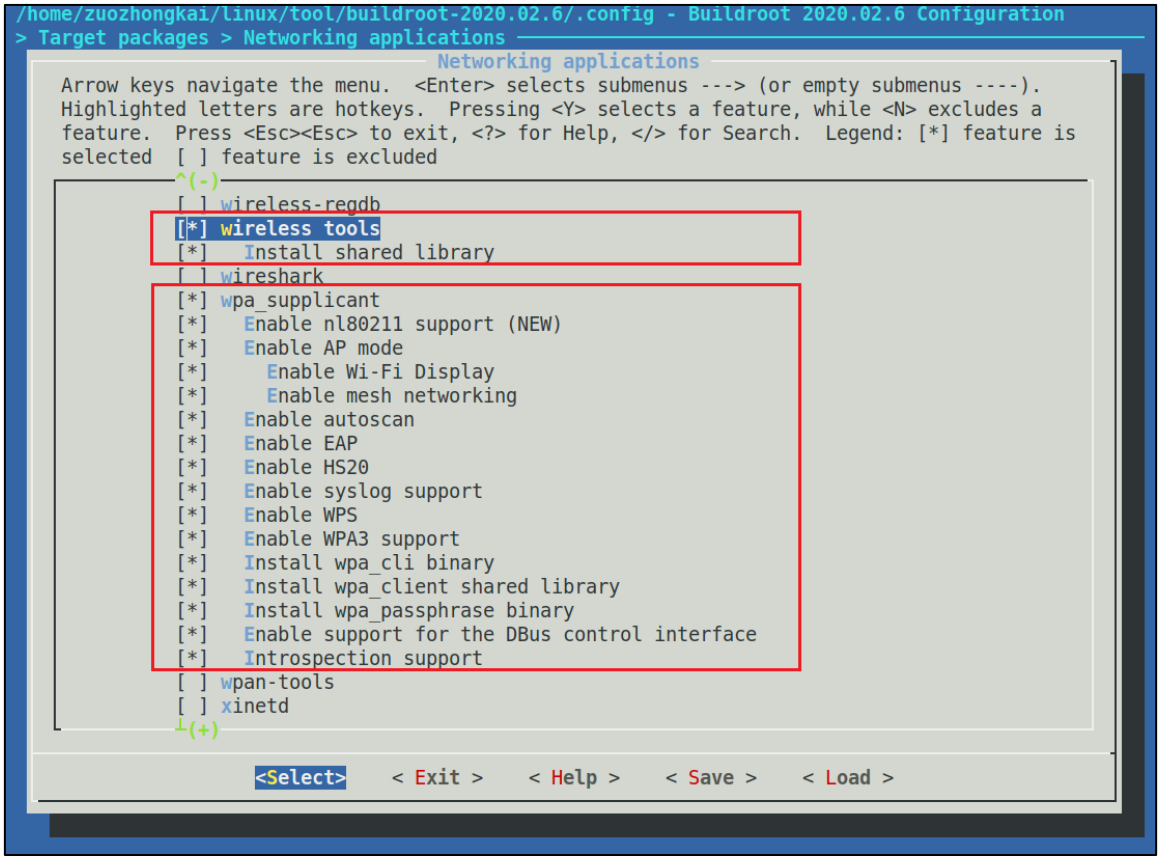


图 53.1.3.1 buildroot WIFI 工具配置

配置完成后重新编译文件系统，编译完成后，进入 output/images 目录，运行以下命令把文件系统替换进去：

```
cd output/images/                                          //进入到 output/images 目录
sudo tar -axvf rootfs.tar -C /home/zuozhongkai/linux/nfs/rootfs  //解压到 nfsroot 目录
```

上述命令将 buildroot 中 output/images/rootfs.tar 这个压缩包解压到/home/zuozhongkai/linux/nfs/rootfs 这个目录中，这个目录就是我当前 nfsroot 目录，大家根据自己的实际情况解压到对应的目录文件中。

重启开发板。

53.3.2 WIFI 工具测试

我们先看下 iwlist 命令的使用的方法，命令如下：

```
iwlist --help
```

iwlist 使用方法如下图：

```
[root@ATK-stm32mp1]:~$ iwlist --help
Usage: iwlist [interface] scanning [essid NNN] [last]
              [interface] frequency
              [interface] channel
              [interface] bitrate
              [interface] rate
              [interface] encryption
              [interface] keys
              [interface] power
              [interface] txpower
              [interface] retry
              [interface] ap
              [interface] accesspoints
              [interface] peers
              [interface] event
              [interface] auth
              [interface] wpakeys
              [interface] genie
              [interface] modulation
[root@ATK-stm32mp1]:~$
```

图 53.3.2.1 iwlist 命令使用方法

iwlist 命令测试 WIFI 之前得先让 WIFI 模块工作起来。这里就以板载的 RTL8723 WIFI 为例。先加载 RTL8723 驱动模块 8723ds.ko，因为在 53.2.5 小节中使用了 depmod 命令去分析了模块之间的依赖，所以重启系统会自动加载模块。运行以下命令打开 WIFI 网卡：

```
ifconfig wlan0 up
```

SDIO WIFI 已经启动了，现在可以使用 iwlist 去扫描网络(一定要接天线!!)，输入如下命令扫描当前存在的无线网络:

```
iwlist wlan0 scanning
```

上述命令就会搜索当前环境下的所有 WIFI 热点，然后将这些热点的信息全部打印出来，包括 MAC 地址、ESSID(WIFI 名字)、频率、速率，信号质量等等，如图 53.3.2.2 所示：

```
[root@ATK-stm32mp1]:~$ iwlist wlan0 scanning
wlan0     Scan completed :
          Cell 01 - Address: 88:F8:72:8D:F3:18
                    ESSID:"ZZK"
                    Protocol:IEEE 802.11bgn
                    Mode:Master
                    Frequency:2.412 GHz (Channel 1)
                    Encryption key:on
                    Bit Rates:300 Mb/s
                    Extra:rsn_ie=30140100000fac040100000fac040100000fac020000
                    IE: IEEE 802.11i/WPA2 Version 1
                        Group Cipher : CCMP
                        Pairwise Ciphers (1) : CCMP
                        Authentication Suites (1) : PSK
                    IE: Unknown: DD880050F204104A000110104400010210 3B0001031047001063041253101920061228 88F
8728DF31C1021000648756177656910230004575378781024000732303137 2D31311042000F313233343536373839303132333437 1
054000800060050F204000110110011414453 4C204D6F64656D2F526F757465721008000206801049000600372A000120
                    Quality=0/100  Signal level=80/100
                    Extra:fm=0003
          Cell 02 - Address: 48:EE:0C:44:C7:CA
                    ESSID:"stone"
                    Protocol:IEEE 802.11bgn
                    Mode:Master
                    Frequency:2.412 GHz (Channel 1)
                    Encryption key:on
                    Bit Rates:270 Mb/s
                    Extra:rsn_ie=30140100000fac040100000fac040100000fac020000
                    IE: IEEE 802.11i/WPA2 Version 1
                        Group Cipher : CCMP
                        Pairwise Ciphers (1) : CCMP
                        Authentication Suites (1) : PSK
                    IE: Unknown: DD180050F204104A000110104400010210 49000600372A000120
                    Quality=0/100  Signal level=36/100
                    Extra:fm=0001
```

图 53.3.2.2 扫描到的 WIFI 信息

图 53.3.2.2 只是扫描出来的一部分截图，可以看出有两个 WIFI 热点，一个名为“ZZK”，一个名为“stone”。这两个都是 2.4G 频段的 WIFI，RTL8723 只支持 2.4G 频段 WIFI，因此扫描不出来 5G 频段 WIFI，这点要注意一下。

接下来我们使用 wpa_supplicant 这个工具来连接 WIFI 热点。简单介绍一下 wpa_supplicant 命令的使用，这个命令的参数有很多，下面就列出常用的参数：

-D：指定设备，有三个设备分别为：nl80211、wext 和 wired。设备是固定的根据驱动源码决定使用那个设备。

-c：指定 wpa_supplicant 配置文件。

-i：指定网口，比如我们的 WiFi 网口名为 wlan0。

-B：表示在后台运行。

53.3.3 RTL8723 SDIO WIFI 联网测试

确保 RTL8723 能扫描出要连接的 WIFI 热点，比如我们这里连接图 53.3.2.2 中“ZZK”这个热点

要连接的 WIFI 热点扫描到以后就可以连接了，默认情况下，开发板根文件系统/etc 目录下会有一个名为“wpa_supplicant.conf”的配置文件，没有的话就自行创建一下。此文件用于配置要连接的 WIFI 热点以及对应的秘密，比如我要连接到“ZZK”这个热点上，因此 wpa_supplicant.conf 文件内容如下所示：

示例代码 53.3.3.1 wpa_supplicant.conf 文件内容

```
1  ctrl_interface=/var/run/wpa_supplicant
2  ctrl_interface_group=0
3  update_config=1
4
5  network={
```

```
6           ssid="ZZK"
7           psk="xxxxxxxxxxxxx"
8   }
```

第 6 行，ssid 是要连接的 WIFI 热点名字，这里我要连接的是“ZZK”这个 WIFI 热点。

第 7 行，psk 就是要连接的 WIFI 热点密码，根据自己的实际情况填写即可。

注意，wpa_supplicant.conf 文件对于格式要求比较严格，“=”前后一定不能有空格，缩进应该采用空格，否则的话会出现 wpa_supplicant.conf 文件解析错误！最重要的一点！wpa_supplicant.conf 文件内容要自己手动输入，不要偷懒复制粘贴!!!

wpa_supplicant.conf 文件编写好以后再在开发板根文件系统下创建一个“/var/run/wpa_supplicant”目录，wpa_supplicant 工具要用到此目录！命令如下：

```
mkdir /var/run/wpa_supplicant -p
```

一切准备好以后就可以使用 wpa_supplicant 工具让 RTL8723 SDIO WIFI 连接到热点上，输入如下命令：

```
wpa_supplicant -Dnl80211 -c /etc/wpa_supplicant.conf -i wlan0 &
```

当 RTL8723 连接上 WIFI 热点以后会输出如图 53.3.3.3 所示的信息：

```
[root@ATK-stm32mp1]:/etc$ Successfully initialized wpa_supplicant
wlan0: Trying to associate with 88:f8:72:8d:f3:18 (SSID='ZZK' freq=2412 MHz)
wlan0: Associated with 88:f8:72:8d:f3:18
wlan0: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
wlan0: CTRL-EVENT-REGDOM-CHANGE init=COUNTRY_IE type=COUNTRY alpha2=CN
wlan0: WPA: Key negotiation completed with 88:f8:72:8d:f3:18 [PTK=CCMP GTK=CCMP]
wlan0: CTRL-EVENT-CONNECTED - Connection to 88:f8:72:8d:f3:18 completed [id=0 id_str=]
[root@ATK-stm32mp1]:/etc$
```

连接成功

图 53.3.3.3 连接成功

从图 53.3.3.3 可以看出，当 RTL8723 连接到 WIFI 热点上以后会输出“wlan0: CTRL-EVENT-CONNECTED”字样。接下来就是最后一步了，设置 wlan0 的 IP 地址，这里使用 udhcpc 命令从路由器申请 IP 地址，输入如下命令：

```
udhcpc -i wlan0    //从路由器获取 IP 地址
```

IP 地址获取成功以后会输出如图 53.3.3.4 所示信息：

```
[root@ATK-stm32mp1]:/etc$ udhcpc -i wlan0
udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending select for 192.168.1.196
udhcpc: lease of 192.168.1.196 obtained, lease time 86400
deleting routers
adding dns 114.114.114.114
[root@ATK-stm32mp1]:/etc$
```

图 53.3.3.4 wlan0 网卡详细信息

可以通过电脑 ping 一下 wlan0 的 192.168.1.196 这个 IP 地址，如果能 ping 通就说明 RTL8723 SDIO WIFI 工作正常。也可以直接在开发板上使用 wlan0 来 ping 一下百度网站，输入如下命令：

```
ping -I 192.168.1.196 www.baidu.com
```

-I 是指定执行 ping 操作的网卡 IP 地址，我们要使用 wlan0 去 ping 百度网站，因此要通过“-I”指定 wlan0 的 IP 地址。如果 WIFI 工作正常的话就可以 ping 通百度网站，如图 53.3.3.5 所示：

```
[root@ATK-stm32mp1]:/etc$ ping -I 192.168.1.196 www.baidu.com
PING www.baidu.com (14.215.177.39) from 192.168.1.196: 56 data bytes
64 bytes from 14.215.177.39: seq=0 ttl=56 time=9.102 ms
64 bytes from 14.215.177.39: seq=1 ttl=56 time=18.512 ms
64 bytes from 14.215.177.39: seq=2 ttl=56 time=28.120 ms
64 bytes from 14.215.177.39: seq=3 ttl=56 time=17.876 ms
64 bytes from 14.215.177.39: seq=4 ttl=56 time=17.649 ms
64 bytes from 14.215.177.39: seq=5 ttl=56 time=17.392 ms
64 bytes from 14.215.177.39: seq=6 ttl=56 time=27.079 ms
^C
--- www.baidu.com ping statistics ---
7 packets transmitted, 7 packets received, 0% packet loss
round-trip min/avg/max = 9.102/19.390/28.120 ms
[root@ATK-stm32mp1]:/etc$
```

图 53.3.3.5 百度网站 ping 成功

至此 RTL8723 SDIO WIFI 我们就完全驱动起来了，大家就可以使用 WIFI 来进行网络通信了。

53.3.4 RTL8188 USB WIFI 联网测试

RTL8188 USB WIFI 测试和 RTL8723 的测试方法基本一致。我们直接把 WIFI 接到 USB HOST 接口上。由于 RTL8723 默认就会开启，因此 RTL8188 此时就会是 wlan1 这个网卡，输入如下命令打开 wlan1 接口：

```
ifconfig wlan1 up
```

RTL8188 USB WIFI 同样使用 wap_supplicant 来完成热点连接工作，还是连接“ZZK”这个热点。我们可以直接使用 53.3.3 中创建的/etc/wpa_supplicant.conf 文件，输入以下命令来完成 WIFI 热点连接：

```
wpa_supplicant -D wext -c /etc/wpa_supplicant.conf -i wlan1 &
```

注意：这里的设备指定为“wext”，设备跟驱动源码有关。

当 RTL8188 连接上 WIFI 热点以后会输出如图 53.3.3.6 所示的信息：

```
[root@ATK-stm32mp1]:~$ Successfully initialized wpa_supplicant
rfkill: Cannot get wiphy information
MAC Address = 00:13:ef:f2:22:a0
ioctl[SIOCSIWAP]: Operation not permitted
ioctl[SIOCSIWENCODEEXT]: Invalid argument
ioctl[SIOCSIWENCODEEXT]: Invalid argument
wlan1: Trying to associate with 88:f8:72:8d:f3:18 (SSID='ZZK' freq=2412 MHz)
Failed to add supported operating classes IE
wlan1: Association request to the driver failed
R8188EU: assoc success
wlan1: Associated with 88:f8:72:8d:f3:18
wlan1: WPA: Key negotiation completed with 88:f8:72:8d:f3:18 [PTK=CCMP GTK=CCMP]
wlan1: CTRL-EVENT-CONNECTED - Connection to 88:f8:72:8d:f3:18 completed [id=0 id_str=]
```

图 53.3.3.6 RTL8188 USB WIFI 连接成功

使用 udhcpc 命令获取 IP 地址，命令如下：

```
udhcpc -i wlan1
```

IP 地址获取过程如图 53.3.3.7 所示：

```
[root@ATK-stm32mp1]:~$ udhcpc -i wlan1
udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending select for 192.168.1.107
udhcpc: lease of 192.168.1.107 obtained, lease time 86400
deleting routers
adding dns 114.114.114.114
[root@ATK-stm32mp1]:~$
```

图 53.3.3.7 udhcpc 获取 IP 地址过程

从图 53.3.3.7 可以看出，wlan1 的 IP 地址为 192.168.1.107。大家可以使用“ifconfig wlan1”查看一下 wlan1 网卡的详细信息。同样的，可以通过电脑 ping 一下 192.168.1.107 测试 WIFI 是否工作正常，或者在开发板上使用 wlan0 网卡 ping 一下百度网址来测试一下 WIFI 工作是否正常，输入如下命令：

```
ping -I 192.168.1.107 www.baidu.com
```

如果 ping 成功的话结果如图 53.3.3.7 所示：

```
[root@ATK-stm32mp1]:~$ ping -I 192.168.1.107 www.baidu.com
PING www.baidu.com (14.215.177.38) from 192.168.1.107: 56 data bytes
64 bytes from 14.215.177.38: seq=0 ttl=55 time=8.318 ms
64 bytes from 14.215.177.38: seq=1 ttl=55 time=12.898 ms
64 bytes from 14.215.177.38: seq=2 ttl=55 time=7.615 ms
64 bytes from 14.215.177.38: seq=3 ttl=55 time=13.223 ms
^C
--- www.baidu.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 7.615/10.513/13.223 ms
[root@ATK-stm32mp1]:~$
```

图 53.3.3.7 ping 百度网站测试成功

至此，如何在 STM32MP1 开发板上使用 WIFI 就全部讲解完了，包括 USB WIFI 和 SDIO WIFI。其实不管是在 STM32MP1 上，还是在其他的 SOC 上，USB WIFI 和 SDIO WIFI 的驱动都是类似的，大家可以参考本章教程讲 RTL8188、RTL8723 这两款 WIFI 的驱动移植到芯片或者开发板上。

第五十四章 Linux 4G 通信实验

前面我们学习了如何在 Linux 中使用有线网络或者 WIFI，但是使用有线网络或者 WIFI 有很多限制，因为要布线，即使是 WIFI 你也得先布线，然后再接个无线路由器。有很多场合是不方便布线的，这个时候就是 4G 大显身手的时候，产品可以直接通过 4G 连接到网络，实现无人值守。本章我们就来学一下如何在 STM32MP157 开发板中使用 4G 来实现联网功能。

54.1 4G 网络连接简介

提起 4G 网络连接，大家可能会觉得是个很难的东西，其实对于嵌入式 Linux 而言，4G 网络连接恰恰相反——不难！大家可以看一下其他的嵌入式 Linux 或者 Android 开发板，4G 模块都是 MiniPCIE 接口的，包括很多 4G 模块都是 MiniPCIE 接口的。但是大家稍微深入研究一下就会发现，这些 4G 模块虽然用了 MiniPCIE 接口，但是实际上的通信接口都是 USB，所以 4G 模块的驱动就转换为了 USB 驱动。而这些 4G 模块厂商都提供了详细的文档讲解如何在 Linux 下使用 4G 模块，以及如何修改 Linux 内核加入 4G 模块驱动。正点原子的 STM32MP157 开发板也有一个 MiniPCIE 形式的 4G 模块接口，虽然外形是 MiniPCIE 的，但是内心却是 USB 的。STM32MP157 开发板的 4G 模块原理图如图 54.1.1 所示：

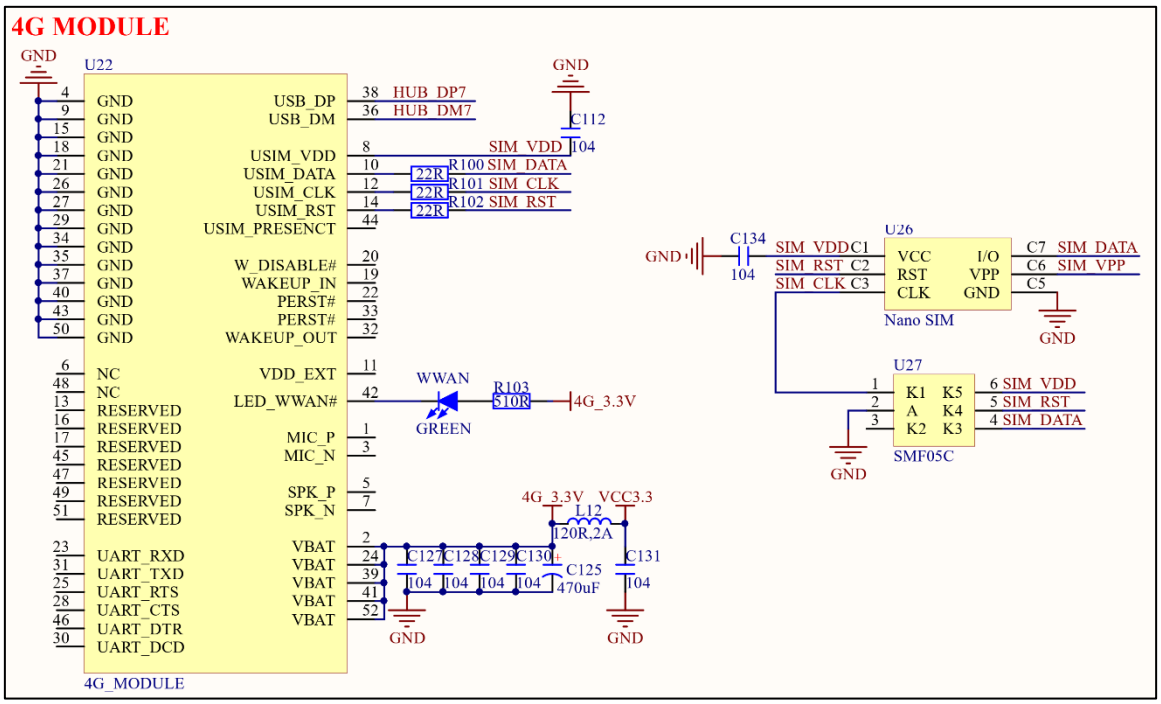


图 54.1.1 4G 模块原理图

图 54.4.1 中的 U22 就是 MiniPCIE 接口，MiniPCIE 接口连接到了 FE2.1 这个 HUB 芯片的 DP7 和 DM7。U26 就是 Nano SIM 接口，STM32MP157 开发板使用 Nano SIM 卡，这样大家就可以直接拿自己的手机卡进行测试。STM32MP157 开发板的 4G 接口位置如图 54.1.2 所示：

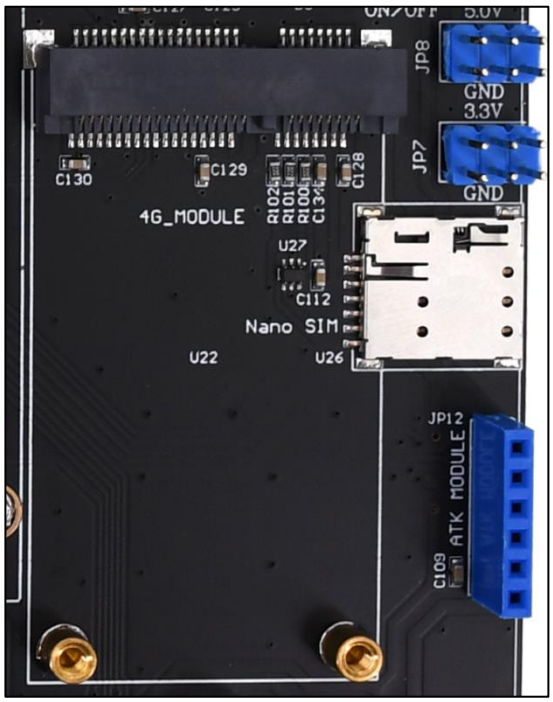


图 54.1.2 ALPHA 开发板 4G 接口

在使用之前需要将 4G 模块插入到图 54.1.2 所示的 MiniPCIE 接口上，然后上紧两边的螺丝，Nano SIM 卡插入到图 54.1.2 中的 Nano SIM 座里面，注意 Nano SIM 卡的方向！

Nano SIM 卡的金属触点朝下！

理论上所有的 MiniPCIE 接口的 4G 模块都可以连接到正点原子的 STM32MP157 开发板上，因为这些 4G 模块都遵循同样的接口标准，但是大家在使用的时候还是要详细的看一下 4G 模块的接口引脚描述。不同的 4G 模块其驱动形式也不同，本章我们讲解两款 4G 模块在开发板上的使用，一个是上海移远公司的 EC20，另外一个是高新兴物联的 ME3630，这两款 4G 模块都有 MiniPCIE 接口的，这两个 4G 模块如图 54.4.2 所示：

图 54.4.2 4G 模块

图 54.4.2 中左侧的是高新兴物联的 ME3630-W 4G 模块，右侧的是上海移远的 EC20 4G 模块。本章我们就分别来讲解一下如何在开发板上使用 EC20 和 ME3630 这两个 4G 模块。

4G 模块工作是需要天线的，因此在选购 4G 模块的时候一定要记得购买天线，否则无法进行测试。一般 MiniPCIE 接口的 4G 模块留出来的天线接口为 IPEX 座，因此购买天线的时候也要选择 IPEX 接口的，或者使用 IPEX 转 SMA 线来转接。推荐大家到正电原子官方店铺购买 4G 模块和相应的天线。

54.2 高新兴 ME3630 4G 模块实验

54.2.1 ME3630 4G 模块简介

ME3630 4G 模块是正点原子官方推荐的 4G 通信模块，ME3630 4G 模块是深圳高新兴物联出品的 4G LTE 模块，前身是中兴物联，正点原子是高新兴物联官方代理商，因此在模块质量以及售后服务这一块是有保证的。

ME3630 是一款 LTE Cat.4 七模全网通 4G 模块，在 LTE 模式下可以提供 50Mbps 上行速率以及 150Mbps 的下行速率，并支持回退到 3G 或 2G 网络。此模组支持分集接收、分集接收是终端产品支持双天线以提高通信质量和通信可靠性的无线连接技术。ME3630 支持多种网络协议，比如 PAP、CHAP、PPP 等，拥有多种功能，比如 GNSS、Remote wakeup、SMS、支持 FoTA 空中升级等。ME3630 4G 模块广泛应用于智能抄表、安防信息采集、工业路由器、车载通信以及监控等等 M2M 领域。ME3630 4G 模组特性如下：

①、一路 USB2.0 接口。

②、一路 UART 接口。

③、SIM 卡接口支持 1.8/3.0V。

④、内置 TCP、UDP、FTP 和 HTTP 等协议。

⑤、支持 RAS/ECM/NDIS。

⑥、支持 AT 指令。

ME3630 4G 模块有多种配置，比如纯数据版本、集成 GNSS 版本、全网通版本等等，本节教程我们主要使用到 ME3630 的数据通信功能，因此推荐大家购买全网通数据版，如果想要定位功能的话就购买全网通数据+GNSS 版本，至于其他的版本大家根据自己的实际需求选择即可。在正式使用 ME3630 4G 模块之前，请先将其插入到开发板的 MiniPCIE 座上、上紧螺、插入 Nano SIM 卡、接上天线，如图 54.2.1.1 所示：

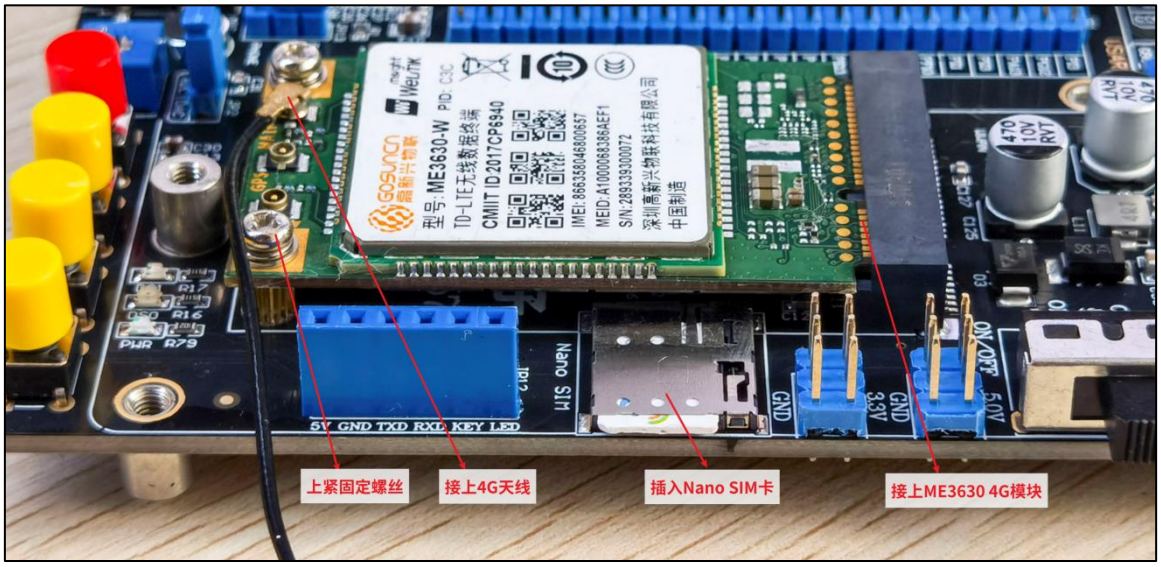


图 54.2.1.1 开发板连接 ME3630 模块

一切准备就绪以后就可以开始驱动 ME3630 4G 模块了。

54.2.2 ME3630 4G 模块驱动修改

1、添加 USB 设备信息

我们需要先在 Linux 内核中添加 ME3630 的 USB 设备信息，因为我们前面说了，ME3630 4G 模块用的 USB 接口。打开 Linux 源码的 drivers/usb/serial/option.c 文件，找到 option_ids 数组，然后在里面添加 ME3630 的 PID 和 VID， 要添加的内容如下：

示例代码 54.2.2.1 ME3630 PID 和 VID 信息

```
1 { USB_DEVICE(0x19d2, 0x0117) }, /* ME3630*/
2 { USB_DEVICE(0x19d2, 0x0199) },
3 { USB_DEVICE(0x19d2, 0x1476) },
```

完成以后的 option_ids 数组如图 54.2.2.1 所示：

```
575  static const struct usb device id option ids[] = {
576      { USB_DEVICE(0x19d2, 0x0117) },     /* ME3630 */
577      { USB_DEVICE(0x19d2, 0x0199) },
578      { USB_DEVICE(0x19d2, 0x1476) },
579      { USB_DEVICE(OPTION_VENDOR_ID, OPTION_PRODUCT_COLT) },
580      { USB_DEVICE(OPTION_VENDOR_ID, OPTION_PRODUCT_RICOLA) },
```

图 54.2.2.1 添加 PID 和 VID 后的 option_ids 数组

2、添加 ECM 支持程序

ME3630 支持 ECM 接口，可以通过 ECM 接口轻松联网，如果要使用 ECM 接口的话需要修改 drivers/usb/serial/option.c 文件里面的 option_probe 函数。找到此函数，然后在里面输入如下内容：

示例代码 54.2.2.2 option_probe 函数需要添加的内容

```
1  /* GOSUNCN 4G modems */
2  printk("idVendor=%x, idProduct=%x, bInterfaceNumber =%d\r\n",
3          serial->dev->descriptor.idVendor,
4          serial->dev->descriptor.idProduct,
```

```
5          serial->interface->cur_altsetting->desc. bInterfaceNumber);
6
7  if (serial->dev->descriptor.idVendor == 0x19d2 &&
8      serial->dev->descriptor.idProduct == 0x1476 &&
9      serial->interface->cur_altsetting->desc. bInterfaceNumber == 3)
10             return -ENODEV;
11
12 if (serial->dev->descriptor.idVendor == 0x19d2 &&
13     serial->dev->descriptor.idProduct == 0x1476 &&
14     serial->interface->cur_altsetting->desc. bInterfaceNumber == 4)
15             return -ENODEV;
16
17 if (serial->dev->descriptor.idVendor == 0x19d2 &&
18     serial->dev->descriptor.idProduct == 0x1509 &&
19     serial->interface->cur_altsetting->desc. bInterfaceNumber == 4)
20             return -ENODEV;
21
22 if (serial->dev->descriptor.idVendor == 0x19d2 &&
23     serial->dev->descriptor.idProduct == 0x1509 &&
24     serial->interface->cur_altsetting->desc. bInterfaceNumber == 5)
25
```

添加完成以后的 option_probe 函数如图 54.2.2.2 所示：

```
2081    /* GOSUNCN 4G modems */
2082    printk("idVendor=%x, idProduct=%x, bInterfaceNumber =%d\r\n",
2083                                    serial->dev->descriptor.idVendor,
2084                                    serial->dev->descriptor.idProduct,
2085                                    serial->interface->cur_altsetting->desc. bInterfaceNumber);
2086
2087    if (serial->dev->descriptor.idVendor == 0x19d2 &&
2088                    serial->dev->descriptor.idProduct == 0x1476 &&
2089                    serial->interface->cur_altsetting->desc. bInterfaceNumber == 3)
2090                    return -ENODEV;
2091
2092    if (serial->dev->descriptor.idVendor == 0x19d2 &&
2093                    serial->dev->descriptor.idProduct == 0x1476 &&
2094                    serial->interface->cur_altsetting->desc. bInterfaceNumber == 4)
2095                    return -ENODEV;
2096
2097    if (serial->dev->descriptor.idVendor == 0x19d2 &&
2098                    serial->dev->descriptor.idProduct == 0x1509 &&
2099                    serial->interface->cur_altsetting->desc. bInterfaceNumber == 4)
2100                    return -ENODEV;
2101
2102    if (serial->dev->descriptor.idVendor == 0x19d2 &&
2103                    serial->dev->descriptor.idProduct == 0x1509 &&
2104                    serial->interface->cur_altsetting->desc. bInterfaceNumber == 5)
2105                    return -ENODEV;
2106
2107    /* Never bind to the CD-Rom emulation interface */
2108    if (iface_desc->bInterfaceClass == USB_CLASS_MASS_STORAGE)
2109        return -ENODEV;
```

图 54.2.2.2 向 option_probe 添加的内容

3、配置 Linux 内核

我们需要配置 Linux 内核，首先使能 USBNET 功能，路径如下：

```
-> Device Drivers
```

-> [*]Network device support
-> USB Network Adapters
-> <*> Multi-purpose USB Networking Framework

配置如图 54.2.2.3 所示：

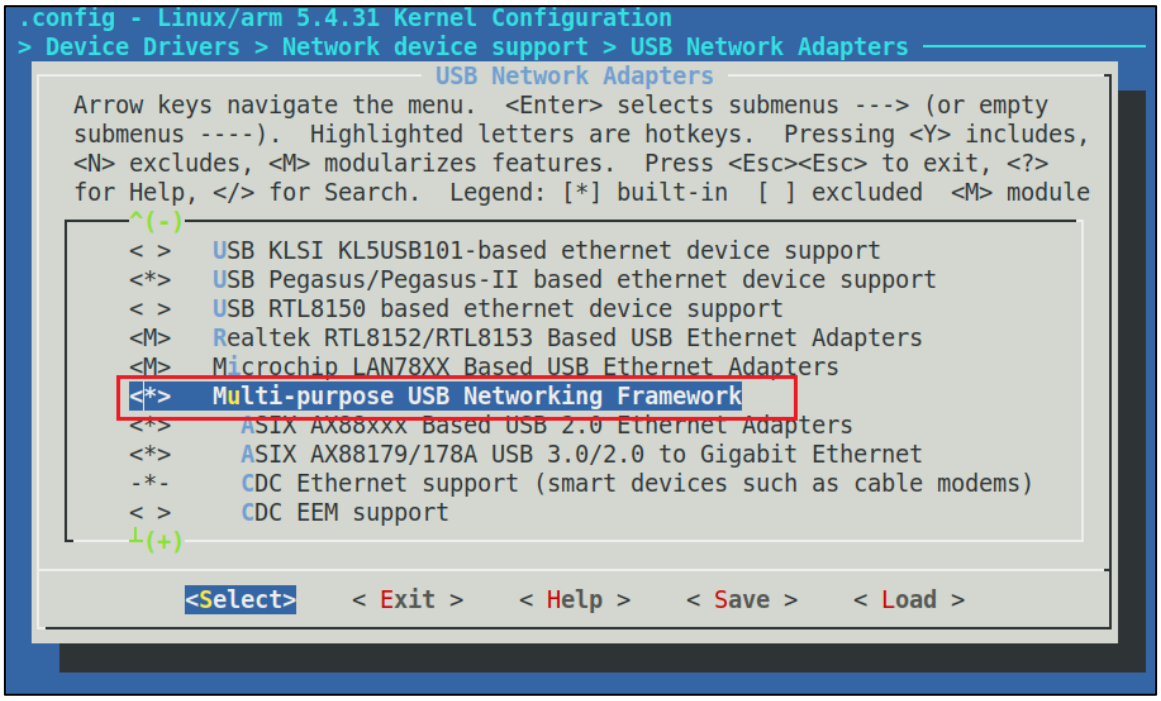


图 54.2.2.3 使能 USB 网络

接下来我们还需要使能 USB 串口 GSM、CDMA 驱动，配置路径如下：

-> Device Drivers
-> [*] USB support
-> <*> USB Serial Converter support //选中
-> <*> USB driver for GSM and CDMA modems //选中

配置如图 54.2.2.4 所示：

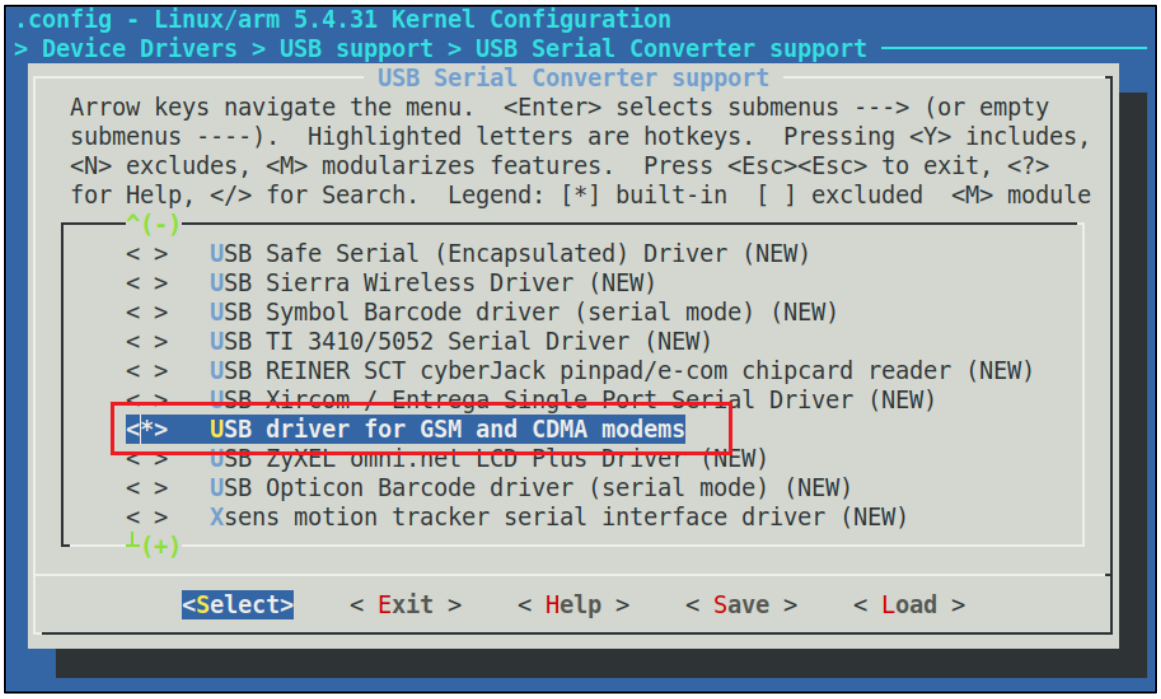


图 54.2.2.4 USB GSM 和 CDMA

继续配置 Linux 内核，使能 USB 的 CDC ACM 模式，配置路径如下：

-> Device Drivers
-> [*] USB support
-> <*> Support for Host-side USB
-> <*> USB Modem (CDC ACM) support

配置如图 54.2.2.5 所示：

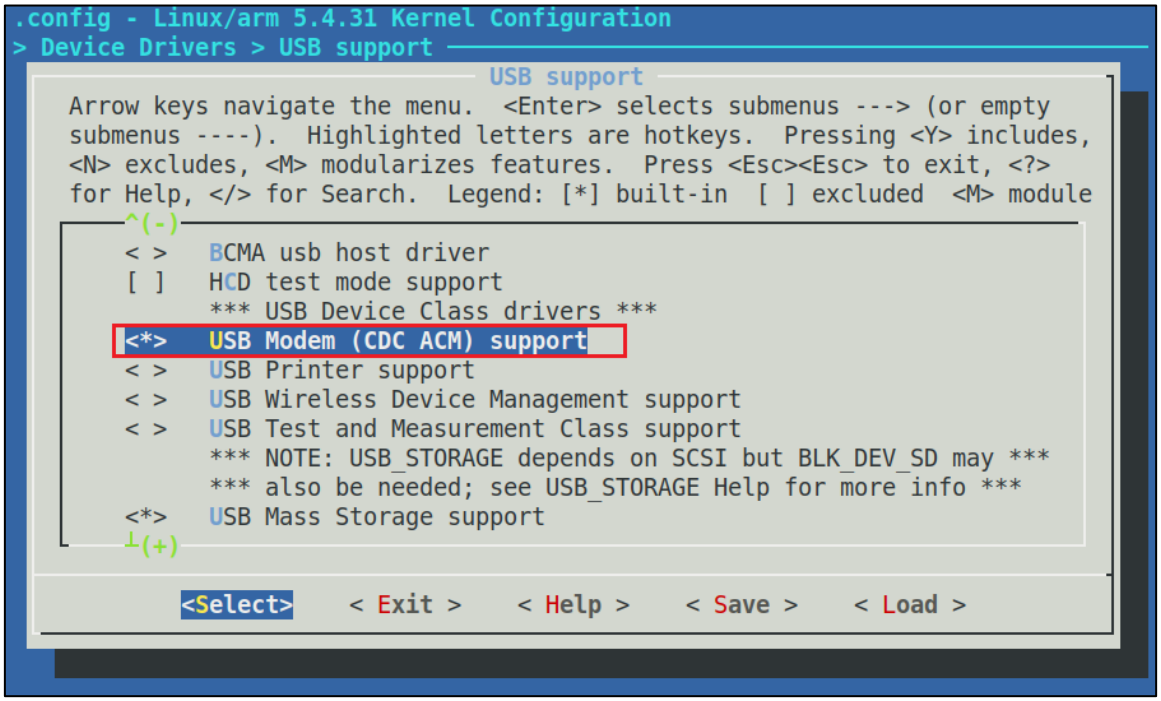


图 54.2.2.5 使能 USB 的 CDC ACM 功能

关于 Linux 内核的配置就到此为止，编译一下 Linux 内核，然后使用新的 uImage 启动开发

板。如果 ME3630 已经插上的话，系统启动以后就会输出如图 54.2.2.6 所示的信息：

```
usb 2-1.7: new high-speed USB device number 3 using ehci-platform
idVendor=19d2, idProduct=1476, bInterfaceNumber =0
option 2-1.7:1.0: GSM modem (1-port) converter detected
usb 2-1.7: GSM modem (1-port) converter now attached to ttyUSB0
idVendor=19d2, idProduct=1476, bInterfaceNumber =1
option 2-1.7:1.1: GSM modem (1-port) converter detected
usb 2-1.7: GSM modem (1-port) converter now attached to ttyUSB1
idVendor=19d2, idProduct=1476, bInterfaceNumber =2
option 2-1.7:1.2: GSM modem (1-port) converter detected
usb 2-1.7: GSM modem (1-port) converter now attached to ttyUSB2
cdc_ether 2-1.7:1.3 usb0: register 'cdc_ether' at usb-5800d000.usbh-ehci-1.7, ZTE CDC Ethernet Device, 82:
c6:e6:c5:e6:da
```

图 54.2.2.6 ME3630 虚拟 USB 信息

从图 54.2.2.6 可以看出，ME3630 虚拟出了 3 个 USB 设备，分别为 ttyUSB0~ttyUSB2。对于支持 ECM 接口的 4G 模块来讲，比如 ZM5330/ZM8620/ME3620/ME3630。如果模块工作在 ECM 模式下，可以通过运行“ifconfig -a”命令查看对应的网卡，网卡的名字可能为 usbX/ecmX/ethX 等，X 是具体的数字，如果存在的话就说明 ECM 接口驱动加载成功。输入“ifconfig -a”命令，会发现多了一个名为“usb0”的网卡，图 54.2.2.7 所示：

```
usb0      Link encap:Ethernet  HWaddr 82:C6:E6:C5:E6:DA
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

图 54.2.2.7 ME3630 对应的 usb0 网卡

54.2.3 ME3630 4G 模块 ppp 联网测试

1、使能 Linux 内核 ppp 功能

ME3630 支持通过 ppp 拨号上网，也支持使用 ECM 接口上网，我们先来学习一下如何通过 ppp 拨号上网。首先我们需要配置 Linux 内核，打开 Linux 内核的 ppp 功能，配置路径如下：

```
-> Device Drivers
    -> [*] Network device support
        -> <*> PPP (point-to-point protocol) support
            -> <*> PPP BSD-Compress compression
            -> <*> PPP Deflate compression
            -> [*] PPP filtering
            -> <*> PPP MPPE compression (encryption)
            -> [*] PPP multilink support
            -> <*> PPP over Ethernet
            -> <*> PPP support for async serial ports
            -> <*> PPP support for sync tty ports
```

配置完成以后如图 54.2.3.1 所示：

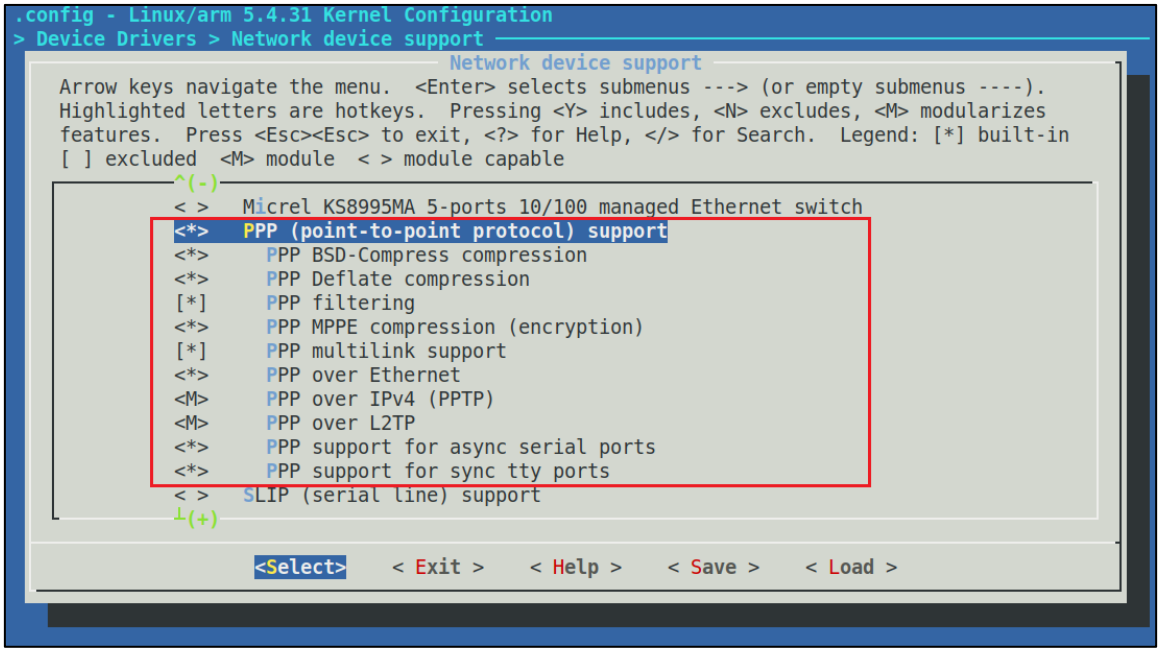


图 54.2.3.1 Linux 内核 PPP 使能

配置完成以后重新编译一下 Linux 内核，得到新的 uImage 镜像文件，然后使用新的 uImage 镜像文件启动开发板。

2、移植 pppd 软件

我们需要通过 pppd 这个软件来实现 ppp 拨号上网，直接在 buildroot 里面使能即可，配置路径如下：

```
-> Target packages
        -> Networking applications
            -> pppd
                -> [*] filtering
                -> [*] radius
                -> [*] overwrite /etc/resolv.conf (NEW)
```

配置如图 54.2.3.2 所示：

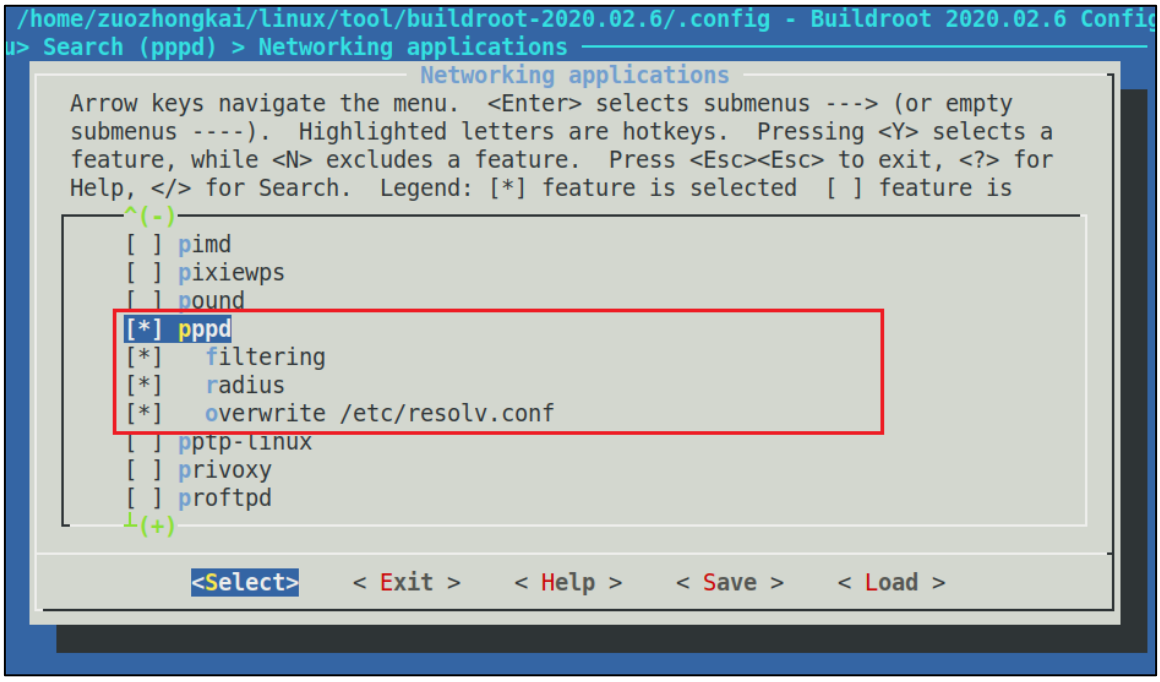


图 54.2.3.2 pppd 工具

配置完成后重新编译文件系统，编译完成后，进入 output/images 目录，运行以下命令把文件系统替换进去：

```
cd output/images/                                          //进入到 output/images 目录
sudo tar -axvf rootfs.tar -C /home/zuozhongkai/linux/nfs/rootfs  //解压到 nfsroot 目录
```

上述命令将 buildroot 中 output/images/rootfs.tar 这个压缩包解压到/home/zuozhongkai/linux/nfs/rootfs 这个目录中，这个目录就是我当前 nfsroot 目录，大家根据自己的实际情况解压到对应的目录文件中。

重启开发板，重启以后输入“pppd -v”查看一下 pppd 的版本号，如果 pppd 版本号显示正常的话就说明 pppd 移植成功，如图 54.2.3.3 所示：

```
[root@ATK-stm32mp1]:~$ pppd -v
pppd: unrecognized option '-v'
pppd version 2.4.8
Usage: pppd [ options ], where options are:
        <device>        Communicate over the named device
        <speed>         Set the baud rate to <speed>
        <loc>:<rem>     Set the local and/or remote interface IP
                        addresses.  Either one may be omitted.
        asyncmap <n>    Set the desired async map to hex <n>
        auth            Require authentication from peer
        connect <p>     Invoke shell command <p> to set up the serial line
        crtscts         Use hardware RTS/CTS flow control
        defaultroute    Add default route through interface
        file <f>        Take options from file <f>
        modem           Use modem control lines
        mru <n>         Set MRU value to <n> for negotiation
See pppd(8) for more options.
[root@ATK-stm32mp1]:~$
```

图 54.2.3.3 pppd 版本号信息

3、ppp 上网测试

在使用 pppd 进行拨号上网之前需要先创建 4 个文件，这个 4 个文件必须放到同一个目录下。在开发板根文件系统下创建/etc/ppp/gosuncn 目录，进入到刚刚创建的/etc/ppp/gosuncn 目录下，然后新建一个名为“ppp-on”的 shell 脚本文件，在 ppp-on 文件里面输入如下所示内容：

示例代码 54.2.3.1 ppp-on 文件内容

```
1 #!/bin/sh
2 clear
3 OPTION_FILE="gosuncn_options"
4 DIALER_SCRIPT=$(pwd)/gosuncn_ppp_dialer
5 exec pppd file $OPTION_FILE connect "chat -v -f ${DIALER_SCRIPT}"
```

再新建一个名为“gosuncn_options”的文件，在文件里面输入如下所示内容：

示例代码 54.2.3.2 gosuncn_options 文件内容

```
1  /dev/ttyUSB2
2  115200
3  crtscts
4  modem
5  persist
6  lock
7  noauth
8  noipdefault
9  debug
10 nodetach
11 user Anyname
12 password Anypassword
13 ipcp-accept-local
14 ipcp-accept-remote
15 defaultroute
16 usepeerdns
17 noccp
18 nobsdcomp
19 novj
20 dump
```

第 1 行，如果是联通或移动的卡就是用 ttyUSB2。

再新建一个名为“gosuncn_ppp_dialer”的文件，输入如下所示内容：

示例代码 54.2.3.3 gosuncn_ppp_dialer 文件内容

```
1  ABORT          "NO CARRIER"
2  ABORT          "ERROR"
3  TIMEOUT        120
4  ""             ATE
5  SAY            "ATE"
6  ECHO           ON
7  OK             ATH
8  OK             ATP
9  OK             AT+CGDCONT=1,\"IP\",\"3GNET\"
10 OK             ATD*99#
11 CONNECT
```

第 9、10 行是网络 APN 码，如果是联通卡，那么第 9，10 行内容如下：

```
OK                  AT+CGDCONT=1,\"IP\",\"3GNET\"
OK                  ATD*99#
```

如果是电信卡，那么第 9，10 行内容如下：

```
OK                  AT+CGDCONT=1,\"IP\",\"CTNET\"
OK                  ATD*99#
```

如果是移动卡，那么第 9,10 行内容如下：

```
OK                  AT+CGDCONT=1,\"IP\",\"CMNET\"
OK                  ATD*99#
```

最后新建一个名为“disconnect”的 shell 脚本，输入如下所示内容：

示例代码 54.2.3.4 disconnect 文件内容

```
1 #!/bin/sh
2 killall pppd
```

这四个文件编写完成以后要给予 ppp-on 和 disconnect 这两个文件可执行权限，命令如下：

```
chmod 777 ppp-on disconnect
```

完成以后输入如下命令连接 4G 网络：

```
./ppp-on &
```

如果 ME3630 连接 4G 网络成功，那么就会自动申请 IP 地址，并且打印出对应的地址信息，如图 54.2.3.4 所示：

```
CONNECT
Script chat -v -f /etc/gosuncn/gosuncn_ppp_dialer finished (pid 280), status = 0x0
Serial connection established.
using channel 1
Using interface ppp0
Connect: ppp0 <--> /dev/ttyUSB2
sent [LCP ConfReq id=0x1 <asyncmap 0x0> <magic 0x2ff62fe1> <pcomp> <accomp>]
rcvd [LCP ConfReq id=0x1 <asyncmap 0x0> <magic 0x384ad6a3> <pcomp> <accomp>]
sent [LCP ConfAck id=0x1 <asyncmap 0x0> <magic 0x384ad6a3> <pcomp> <accomp>]
rcvd [LCP ConfAck id=0x1 <asyncmap 0x0> <magic 0x2ff62fe1> <pcomp> <accomp>]
sent [IPCP ConfReq id=0x1 <addr 0.0.0.0> <ms-dns1 0.0.0.0> <ms-dns2 0.0.0.0>]
rcvd [IPCP ConfReq id=0x1 <addr 10.40.133.154>]
sent [IPCP ConfAck id=0x1 <addr 10.40.133.154>]
rcvd [IPV6CP ConfReq id=0x1 <addr fe80::8c41:dbb1:ce41:6c86>]
Unsupported protocol 'IPv6 Control Protocol' (0x8057) received
sent [LCP ProtRej id=0x2 80 57 01 01 00 0e 01 0a 8c 41 db b1 ce 41 6c 86]
rcvd [IPCP ConfNak id=0x1 <addr 10.40.133.153> <ms-dns1 120.80.80.80> <ms-dns2 120.80.80.80>]
sent [IPCP ConfReq id=0x2 <addr 10.40.133.153> <ms-dns1 120.80.80.80> <ms-dns2 120.80.80.80>]
rcvd [IPCP ConfAck id=0x2 <addr 10.40.133.153> <ms-dns1 120.80.80.80> <ms-dns2 120.80.80.80>]
not replacing existing default route via 192.168.1.1 with metric -1
local  IP address 10.40.133.153
remote IP address 10.40.133.154
primary   DNS address 120.80.80.80
secondary DNS address 120.80.80.80
```

图 54.2.3.4 4G 连接过程

从图 54.2.3.4 可以看出，4G 网络连接成功，IP 地址为 10.40.133.153。ppp 拨号成功以后就会生成一个名为“ppp0”的网卡，如图 54.2.3.5 所示：

```
ppp0      Link encap:Point-to-Point Protocol
          inet addr:10.40.133.153  P-t-P:10.40.133.154  Mask:255.255.255.255
          UP POINTOPOINT RUNNING NOARP MULTICAST  MTU:1500  Metric:1
          RX packets:4 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:3
          RX bytes:68 (68.0 B)  TX bytes:54 (54.0 B)
```

图 54.2.3.5 ppp0 网卡信息

4G 网络测试需要关闭其他网卡，否则的话网络测试可能有问题，但是我们现在是通过网络

启动的系统，并且通过 NFS 挂载的根文件系统，因此我们没法关闭其他的网卡，比如 eth0。为了解决这个问题，我们只能将 uboot、Linux kernel、.dtb 设备树和根文件系统都烧写到板子的 EMMC 里面，然后直接启动 EMMC 上的系统即可，这样就不需要其他网卡工作了。

烧写完成以后重启启动开发板，确保当前开发板只有一个 ME3630 对应的 ppp0 网卡，最后直接 ping 百度官网即可，结果如图 54.2.3.6 所示：

```
root@ATK-MP157:~/shell/me3630-w# ping www.baidu.com
PING www.a.shifen.com (163.177.151.109) 56(84) bytes of data.
64 bytes from 163.177.151.109 (163.177.151.109): icmp_seq=1 ttl=55 time=40.9 ms
64 bytes from 163.177.151.109 (163.177.151.109): icmp_seq=2 ttl=55 time=23.8 ms
64 bytes from 163.177.151.109 (163.177.151.109): icmp_seq=3 ttl=55 time=28.8 ms
64 bytes from 163.177.151.109 (163.177.151.109): icmp_seq=4 ttl=55 time=31.2 ms
64 bytes from 163.177.151.109 (163.177.151.109): icmp_seq=5 ttl=55 time=25.8 ms
64 bytes from 163.177.151.109 (163.177.151.109): icmp_seq=6 ttl=55 time=26.6 ms
```

图 54.2.3.6 ME3630 4G 模块 ping 百度官网

54.2.4 ME3630 4G 模块 ECM 联网测试

对于支持 ECM 接口的模块可以直接通过 ECM 上网，ME3630 模块支持 ECM 接口，重启开发板，输入“ifconfig -a”命令可以看到有一个名为“usb0”的网卡，如图 54.2.4.1 所示：

```
usb0      Link encap:Ethernet  HWaddr 86:72:86:CE:A0:49
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

图 54.2.4.1 usb0 网卡

这个 usb0 网卡就是 ECM 接口对应的网卡，在开发板根文件系统的/etc/ppp/gosuncn(上一小节创建的)目录下创建一个名为“ecm_on”的 shell 脚本，内容如下：

示例代码 54.2.4.1 ecm_on 文件内容

```
1 #!/bin/sh
2 clear
3 OPTION_FILE="gosuncn_options"
4 DIALER_SCRIPT=$(pwd)/gosuncn_ecm_dialer
5 exec pppd file $OPTION_FILE connect "chat -v -f ${DIALER_SCRIPT}"
```

最后再创建一个名为“gosuncn_ecm_dialer”的文件，文件内容为：

示例代码 54.2.4.2 gosuncn_ecm_dialer 文件内容

```
1  ABORT           "NO CARRIER"
2  ABORT           "ERROR"
3  TIMEOUT         120
4  ""              ATE
5  SAY             "ATE"
6  ECHO            ON
7  OK              ATH
8  OK              ATP
9  OK              AT+ZSWITCH=L
10 OK              AT+ZECMCALL=1
11 OK              AT+CGDCONT=1,"IP","3GNET"
```

```
12 OK                ATD*99#
13 CONNECT
```

第 11、12 行是网络 APN 码，如果是联通卡，那么第 11、12 行内容如下：

```
OK                   AT+CGDCONT=1,\"IP\",\"3GNET\"
OK                   ATD*99#
```

如果是电信卡，那么第 11、12 行内容如下：

```
OK                   AT+CGDCONT=1,\"IP\",\"CTNET\"
OK                   ATD*99#
```

如果是移动卡，那么第 11、12 行内容如下：

```
OK                   AT+CGDCONT=1,\"IP\",\"CMNET\"
OK                   ATD*99#
```

最后给予 ecm-on 这个文件可执行权限，命令如下：

```
chmod 777 ecm-on
```

完成以后输入如下命令连接 4G 网络：

```
./ecm-on &
```

如果 ME3630 连接 4G 网络成功会出书如图 54.2.4.2 所示信息：

```
CONNECT
Script chat -v -f /etc/ppp/gosuncn/gosuncn_ecm_dialer finished (pid 257), status = 0x0
Serial connection established.
using channel 1
Using interface ppp0
Connect: ppp0 <--> /dev/ttyUSB2
Modem hangup
Connection terminated.

[root@ATK-stm32mp1]:/etc/ppp/gosuncn$
```

图 54.2.4.2 ECM 网络连接成功

连接成功以后打开 usb0 网卡，命令如下：

```
ifconfig usb0 up          //打开 usb0 网卡
```

usb0 网卡打开以后输入如下命令获取 IP 地址：

```
udhcpc -i usb0            //获取 IP 地址
```

IP 地址获取过程如图 54.2.4.3 所示：

```
[root@ATK-stm32mp1]:/etc/ppp/gosuncn$ ifconfig usb0 up
[root@ATK-stm32mp1]:/etc/ppp/gosuncn$ udhcpc -i usb0
udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending discover
udhcpc: sending select for 10.155.147.186
udhcpc: lease of 10.155.147.186 obtained, lease time 43200
deleting routers
adding dns 120.80.80.80
adding dns 221.5.88.88
[root@ATK-stm32mp1]:/etc/ppp/gosuncn$
```

图 54.2.4.3 usb0 网卡获取 IP 地址过程

从图 54.2.4.3 可以看出，usb0 网卡获取到的 IP 地址为 10.35.243.53，然后 ping 一下百度官网，如果能 ping 通就说明 ME3630 的 ECM 接口联网成功。

至此 ME3630 4G 模块的网络连接就已经全部测试完成，大家可以在正点原子的 STM32MP157 开发板上使用 4G 上网了。

54.2.5 ME3630 4G 模块 GNSS 定位测试

注意！ME3630-C3C 的 GNSS 要用无源天线，不能使用有源天线，否则无法定位！

有些型号的 ME3630 带有 GNSS 功能，也就是 GPS 定位。在 54.3.2 小节移植 ME3630 驱动的时候我们知道最终会出现 3 个 ttyUSB 设备，分别为 ttyUSB0~ttyUSB2，其中 ttyUSB1 为 GNSS 接口。如果以前用过其他的 GPS 模块的话应该知道，GPS 模块是串口输出的，ME3630 也一样的，只不过 ME3630 是 USB 转串口，ttyUSB1 就是 ME3630 转出来的 GNSS 串口。所以我们可以直接使用 minicom 来查看 ttyUSB1 输出的 GNSS 信息，minicom 的移植请参考 46.5 小节，其他串口软件也可以，根据自己的需求选择即可。ME3630 的 GNSS 定位使用 ttyUSB1 接口，波特率为 115200，因此本小节中的 minicom 配置如图 54.2.5.1 所示：

```
A -    Serial Device      : /dev/ttyUSB1
B - Lockfile Location     : /var/lock
C -   Callin Program      :
D -  Callout Program      :
E -    Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : No
G - Software Flow Control : No

   Change which setting?
```

图 54.2.5.1 minicom 配置

1、使用 AT 指令配置 ME3630

ME3630 默认关闭了 GNSS 定位输出功能的，必须先使用 AT 指令配置，ME3630 的详细 AT 指令请参考《高新兴物联 ME3630&ME3630-W 模组 AT 指令手册_V3.1.pdf》的“11.GPS 相关指令”小节。此文档已经放到了开发板光盘中，路径为：开发板光盘->1、例程源码->5、模块驱动源码->3、4G 模块->高新兴物联->ME3630。

AT 指令配置过程如下，其中深色是 AT 指令，“OK”为 ME3630 返回值：

54.2.5.1 ME3630-C3C AT 指令配置 GPS

```
ATI                              //查看固件信息

Manufacturer: GOSUNCNWELINK
Model: ME3630-W
Revision: ME3630C3CV1.0B03
IMEI: 864863045876287

OK

AT+ZGINIT                        //初始化 GPS

OK

AT+ZGMODE=3                      //设置定位模式

OK
```

```
AT+ZGPORT=0                    //定位信息从 AT、MODEM、UART 三个口同时上报

OK

AT+ZGNMEA=31                   //设置 GPS 数据为 NMEA 格式

OK

AT+ZGPSR=1                     //使能 ZGPSR 数据

OK

AT+ZGRUN=2                     //连续定位模式

OK
```

AT 指令配置完成以后 ME3630 就会开始搜星，注意，GPS 天线一定要放到室外！等 ME3630 搜星结束以后就会输出 NEMA 格式的定位信息，如图 54.2.5.2 所示：

```
$GLGSV,3,1,10,78,21,213,16,82,08,305,18,79,29,270,19,65,23,037,34*69
$GLGSV,3,2,10,81,34,350,31,66,55,116,,80,05,322,,88,28,053,*6B
$GLGSV,3,3,10,87,04,085,,67,23,177,*63
$GPGSV,4,1,14,02,35,299,32,04,10,061,23,11,,,27,12,17,316,28*43
$GPGSV,4,2,14,19,59,045,25,05,50,043,,09,,,,13,01,052,*42
$GPGSV,4,3,14,17,57,064,,20,86,064,,21,12,057,,29,10,112,*79
$GPGSV,4,4,14,31,00,279,,32,76,149,*7E
$BDGSV,3,1,09,11,26,317,20,12,51,019,37,22,29,047,44,01,43,122,,0,4*60
$BDGSV,3,2,09,10,70,240,,13,51,295,,14,09,201,,16,08,170,,0,4*6A
$BDGSV,3,3,09,25,22,178,,0,4*5C
$GPGGA,033153.00,2318.130320,N,11319.737000,E,1,03,3.2,25.2,M,-5.0,M,,*47
$GPVTG,211.7,T,214.0,M,0.0,N,0.0,K,A*21
$GPRMC,033153.00,A,2318.130320,N,11319.737000,E,0.0,211.7,310721,2.4,W,A*26
$GPGSA,A,2,02,12,19,,,,,,,,,,3.3,3.2,0.9*32
```

图 54.2.5.2 GNSS 定位信息

在实际的应用开发中大家直接解析 NEMA 格式数据，然后得到具体的经纬度、速度、高度、UTC 时间等信息。

54.3 EC20 4G 模块实验

54.3.1 EC20 4G 模块简介

关于 EC20 4G 模块的详细资料请找卖家索要，正点原子没有代理 EC20 模块，所以不提供任何 EC20 的模块资料！

EC20 有多种不同的配置，比如全网通纯数据版本、语音版、带 GNSS 版等等，建议大家购买的时候至少要选择全网通数据版，因为我们使用 4G 模块主要还是用于数据通信的。移远的 EC20 4G 模块采用 LTE 3GPP Rel.11 技术，支持最大下行速率 150Mbps，最大上行速率 50Mbps。EC20 4G 模块特性如下：

①、一路 USB2.0 高速接口，最高可达 480Mbps。

②、一组模拟语音接口(可选)。

③、1.8V/3.0V SIM 接口。

④、1 个 UART 接口。

⑤、W_DISABLE#(飞行模式控制)。

⑥、LED_WWAN#(网络状态指示)。

EC20 也支持 AT 指令，本教程不讲 AT 指令，关于 AT 指令的使用请参考 EC20 的相关文档。在正式使用 EC20 4G 模块之前，请先将其插入到开发板的 MiniPCIE 座上、上紧螺、插入 Nano SIM 卡、接上天线，如图 54.3.1.1 所示：

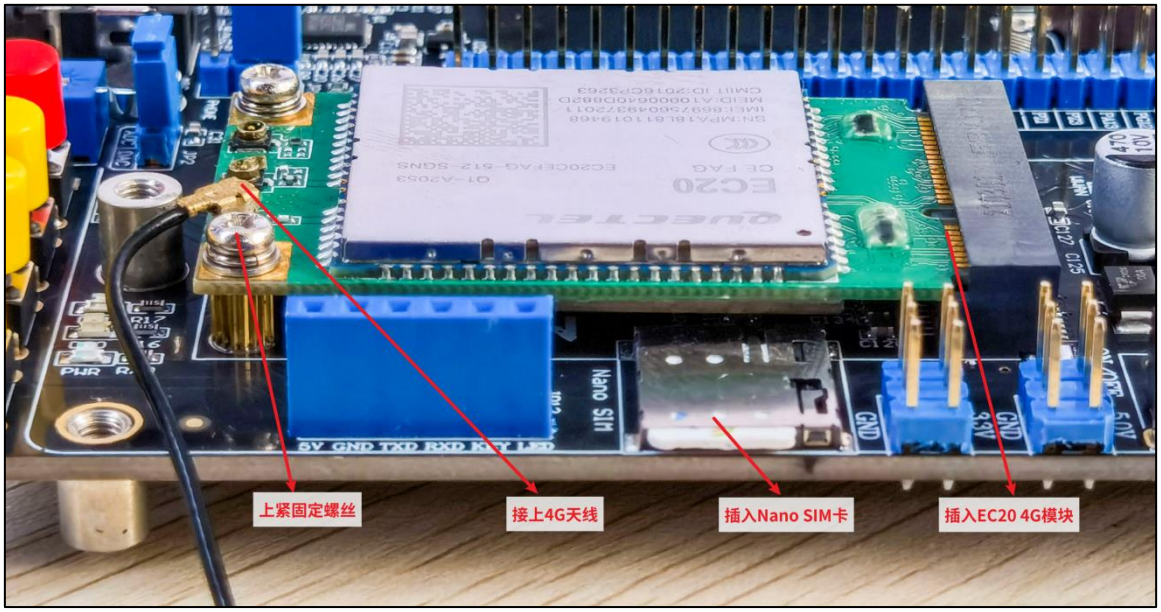


图 54.3.1.1 连接好 4G 模块

一切准备就绪以后就可以开始驱动 EC20 4G 模块了。

54.3.2 EC20 4G 模块驱动修改

1、添加 USB 设备信息

我们需要先在 Linux 内核中添加 EC20 的 USB 设备信息，因为我们前面说了，EC20 4G 模块用的 USB 接口。打开 Linux 源码的 drivers/usb/serial/option.c 文件，找到 option_ids 数组，然后在里面添加 ME3630 的 PID 和 VID，要添加的内容如下：

示例代码 54.3.2.1 EC20 4G 模块 ID

```
1 { USB_DEVICE(0x2C7C, 0x0125) },     /* EC20 */
```

完成以后如图 54.3.2.1 所示：

```
578     { USB_DEVICE(0x19d2, 0x1476) },
579     { USB_DEVICE(0x2C7C, 0x0125) },     /* EC20 */
580     { USB_DEVICE(OPTION_VENDOR_ID, OPTION_PRODUCT_COLT) },
```

图 54.3.2.1 修改后的 option_ids 数组

继续在 drivers/usb/serial/option.c 文件里面找到 option_probe 函数，在此函数里面添加如下内容：

示例代码 54.3.2.3 option_probe 函数添加的代码

```
1  /* EC20  */
2  if (serial->dev->descriptor.idVendor == 0x05c6 &&
3      serial->dev->descriptor.idProduct == 0x9003 &&
```

```
4       serial->interface->cur_altsetting->desc. bInterfaceNumber >= 4)
5       return -ENODEV;
6
7   if (serial->dev->descriptor.idVendor == 0x05c6 &&
8       serial->dev->descriptor.idProduct == 0x9215 &&
9       serial->interface->cur_altsetting->desc. bInterfaceNumber >= 4)
10      return -ENODEV;
11
12  if (serial->dev->descriptor.idVendor == 0x2c7c &&
13      serial->interface->cur_altsetting->desc. bInterfaceNumber >= 4)
14              return -ENODEV;
```

添加完成以后的 option_probe 函数如图 54.3.2.2 所示：

```
2107    /* EC20  */
2108    if (serial->dev->descriptor.idVendor == 0x05c6 &&
2109                    serial->dev->descriptor.idProduct == 0x9003 &&
2110                    serial->interface->cur_altsetting->desc. bInterfaceNumber >= 4)
2111                    return -ENODEV;
2112
2113    if (serial->dev->descriptor.idVendor == 0x05c6 &&
2114                    serial->dev->descriptor.idProduct == 0x9215 &&
2115                    serial->interface->cur_altsetting->desc. bInterfaceNumber >= 4)
2116                    return -ENODEV;
2117
2118    if (serial->dev->descriptor.idVendor == 0x2c7c &&
2119                    serial->interface->cur_altsetting->desc. bInterfaceNumber >= 4)
2120                    return -ENODEV;
2121
2122    /* Never bind to the CD-Rom emulation interface */
2123    if (iface_desc->bInterfaceClass == USB_CLASS_MASS_STORAGE)
2124        return -ENODEV;
```

图 54.3.2.2 修改后的 option_probe 函数

继续在 drivers/usb/serial/option.c 文件里面找到 option_1port_device 结构体变量，在里面加入休眠后唤醒接口，如图 54.3.2.3 所示：

```
2062        .read_int_callback = option_instat_callback,
2063    #ifdef CONFIG_PM
2064        .suspend           = usb_wwan_suspend,
2065        .resume            = usb_wwan_resume,
2066        .reset_resume      = usb_wwan_resume,
2067    #endif
2068    };
```

添加的休眠唤醒接口

图 54.3.2.3 添加的休眠唤醒接口

打开 drivers/usb/serial/usb_wwan.c 文件，在 usb_wwan_setup_urb 函数中添加零包处理代码，完成后的 usb_wwan_setup_urb 函数如下所示：

示例代码 54.3.2.4 修改后的 usb_wwan_setup_urb 函数

```
1   static struct urb *usb_wwan_setup_urb(struct usb_serial_port *port,
2                   int endpoint,
3                   int dir, void *ctx, char *buf, int len,
4                   void (*callback) (struct urb *))
5   {
6       struct usb_serial *serial = port->serial;
7       struct usb_wwan_intf_private *intfdata =
                                usb_get_serial_data(serial);
8       struct urb *urb;
```

```
9
10      urb = usb_alloc_urb(0, GFP_KERNEL); /* No ISO */
11      if (!urb)
12          return NULL;
13
14      usb_fill_bulk_urb(urb, serial->dev,
15           usb_sndbulkpipe(serial->dev, endpoint) | dir,
16           buf, len, callback, ctx);
17
18      if (intfdata->use_zlp && dir == USB_DIR_OUT)
19          urb->transfer_flags |= URB_ZERO_PACKET;
20
21      /* EC20 */
22      if (dir == USB_DIR_OUT) {
23          struct usb_device_descriptor *desc =
24          &serial->dev->descriptor;
25
26          if (desc->idVendor == cpu_to_le16(0x05c6) &&
27              desc->iProduct == cpu_to_le16(0x9090))
28              urb->transfer_flags |= URB_ZERO_PACKET;
29
30          if (desc->idVendor == cpu_to_le16(0x05c6) &&
31              desc->iProduct == cpu_to_le16(0x9003))
32              urb->transfer_flags |= URB_ZERO_PACKET;
33
34          if (desc->idVendor == cpu_to_le16(0x05c6) &&
35              desc->iProduct == cpu_to_le16(0x9215))
36              urb->transfer_flags |= URB_ZERO_PACKET;
37
38          if (desc->idVendor == cpu_to_le16(0x2c7c))
39              urb->transfer_flags |= URB_ZERO_PACKET;
40      }
41
42  return urb;
43  }
```

第 21~40 行就是要添加到 usb_wwan_setup_urb 函数里面的零包处理代码。至此，Linux 内核需要修改的地方就完了。编译一下 Linux 内核，检查一下代码修改有没有问题，如果修改正确的话就是没有任何问题的。

2、配置 Linux 内核

我们需要配置 Linux 内核，使能 USB NET、GSM、CDMA 驱动等，配置方法和我们在 54.2.2 小节讲解的 ME3630 4G 模块的配置方法一样，大家参考 54.2.2 小节即可。配置完成以后编译一下 Linux 内核，然后使用新的 zImage 启动开发板。如果 EC20 已经插上的话，系统启动以后就会输出如图 54.3.2.4 所示的信息：

```
usb 2-1.7: not running at top speed; connect to a high speed hub
idVendor=2c7c, idProduct=125, bInterfaceNumber =0
option 2-1.7:1.0: GSM modem (1-port) converter detected
usb 2-1.7: GSM modem (1-port) converter now attached to ttyUSB0
idVendor=2c7c, idProduct=125, bInterfaceNumber =1
option 2-1.7:1.1: GSM modem (1-port) converter detected
usb 2-1.7: GSM modem (1-port) converter now attached to ttyUSB1
idVendor=2c7c, idProduct=125, bInterfaceNumber =2
option 2-1.7:1.2: GSM modem (1-port) converter detected
usb 2-1.7: GSM modem (1-port) converter now attached to ttyUSB2
idVendor=2c7c, idProduct=125, bInterfaceNumber =3
option 2-1.7:1.3: GSM modem (1-port) converter detected
usb 2-1.7: GSM modem (1-port) converter now attached to ttyUSB3
idVendor=2c7c, idProduct=125, bInterfaceNumber =4
```

图 54.3.2.4 EC20 虚拟出的 USB 接口

从图 54.3.2.4 可以看出，多了 ttyUSB0~ttyUSB3，这 4 个 tty 接口就是 EC20 虚拟出来的，可以查看一下/dev/ttyUSB0~ttyUSB3 是否存在，结果如图 54.3.2.5 所示：

```
[root@ATK-stm32mp1]:~$ ls /dev/ttyUSB*
/dev/ttyUSB0  /dev/ttyUSB1  /dev/ttyUSB2  /dev/ttyUSB3
[root@ATK-stm32mp1]:~$
```

图 54.3.2.5 EC20 虚拟出来的 tty 接口

图 54.3.2.5 中的这 4 路 ttyUSB 的含义见表 54.3.2.1：

ttyUSB	描述
ttyUSB0	DM
ttyUSB1	GPS 的 NMEA 信息输出接口
ttyUSB2	AT 指令接口
ttyUSB3	PPP 连接或 AT 指令接口

表 54.3.2.1 四路 ttyUSB 函数

我们用到最多的就是 ttyUSB1 和 ttyUSB2，如果你购买的 EC20 模块带有 GPS 功能，那么就可以通过 ttyUSB1 接口读取 GPS 数据。如果你想使用 EC20 的 AT 指令功能，那么就使用 ttyUSB2 接口即可。

54.3.3 EC20 ppp 拨号上网

ppp 拨号之前要先使能内核一些 ppp 相关的配置，并且在 buildroot 中使能 pppd 工具，方法参考 53.2.3 小节即可。

在使用 pppd 进行拨号上网之前需要先创建 4 个文件，这个 4 个文件必须放到同一个目录下。在开发板根文件系统下创建/etc/ppp/quectel 目录，进入到刚刚创建的/etc/ppp/quectel 目录下，然后新建一个名为“ppp-on”的 shell 脚本文件，在 ppp-on 文件里面输入如下所示内容：

示例代码 54.3.3.1 ppp-on 文件内容

```
1 #!/bin/sh
2 clear
3 OPTION_FILE="quectel_options"
4 DIALER_SCRIPT=$(pwd)/quectel_ppp_dialer
5 exec pppd file $OPTION_FILE connect "chat -v -f ${DIALER_SCRIPT}"
```

再新建一个名为“quectel_options”的文件，在文件里面输入如下所示内容：

示例代码 54.3.3.2 quectel_options 文件内容

```
1 /dev/ttyUSB2
2 115200
```

```
3  crtscts
4  modem
5  persist
6  lock
7  noauth
8  noipdefault
9  debug
10 nodetach
11 user Anyname
12 password Anypassword
13 ipcp-accept-local
14 ipcp-accept-remote
15 defaultroute
16 usepeerdns
17 noccp
18 nobsdcomp
19 novj
20 dump
```

第 1 行，ppp 拨号接口使用 ttyUSB2。

再新建一个名为“quectel_ppp_dialer”的文件，输入如下所示内容：

示例代码 54.3.3.3 quectel_ppp_dialer 文件内容

```
1  ABORT          "NO CARRIER"
2  ABORT          "ERROR"
3  TIMEOUT        120
4  ""             ATE
5  SAY            "ATE"
6  ECHO           ON
7  OK             ATH
8  OK             ATP
9  OK             AT+CGDCONT=1,\"IP\",\"3GNET\"
10 OK             ATD*99#
11 CONNECT
```

第 9、10 行是网络 APN 码，如果是联通卡，那么第 9，10 行内容如下：

```
OK              AT+CGDCONT=1,\"IP\",\"3GNET\"
OK              ATD*99#
```

如果是电信卡，那么第 9，10 行内容如下：

```
OK              AT+CGDCONT=1,\"IP\",\"CTNET\"
OK              ATD*99#
```

如果是移动卡，那么第 9,10 行内容如下：

```
OK              AT+CGDCONT=1,\"IP\",\"CMNET\"
OK              ATD*99#
```

最后新建一个名为“disconnect”的 shell 脚本，输入如下所示内容：

示例代码 54.3.3.4 disconnect 文件内容

```
#!/bin/sh
killall pppd
```

这四个文件编写完成以后要给予 ppp-on 和 disconnect 这两个文件可执行权限，命令如下：

```
chmod 777 ppp-on disconnect
```

完成以后输入如下命令连接 4G 网络：

```
./ppp-on &
```

如果 EC20 连接 4G 网络成功，那么就会自动申请 IP 地址，并且打印出对应的地址信息，如图 54.3.3.1 所示：

```
CONNECT
Script chat -v -f /home/root/shell/me3630-w/gosuncn_ppp_dialer_10086 finished (pid 989), status = 0x0
Serial connection established.
using channel 1
Using interface ppp0
Connect: ppp0 <--> /dev/ttyUSB2
sent [LCP ConfReq id=0x1 <asyncmap 0x0> <magic 0x8d04ea7d> <pcomp> <accomp>]
rcvd [LCP ConfReq id=0x6 <asyncmap 0x0> <auth pap> <magic 0xd1e901d4> <pcomp> <accomp>]
sent [LCP ConfAck id=0x6 <asyncmap 0x0> <auth pap> <magic 0xd1e901d4> <pcomp> <accomp>]
rcvd [LCP ConfAck id=0x1 <asyncmap 0x0> <magic 0x8d04ea7d> <pcomp> <accomp>]
sent [PAP AuthReq id=0x1 user="Anyname" password=<hidden>]
rcvd [LCP DiscReq id=0x7 magic=0xd1e901d4]
rcvd [PAP AuthAck id=0x1 ""]
PAP authentication succeeded
sent [IPCP ConfReq id=0x1 <addr 0.0.0.0> <ms-dns1 0.0.0.0> <ms-dns2 0.0.0.0>]
rcvd [IPCP ConfReq id=0x6]
sent [IPCP ConfNak id=0x6 <addr 0.0.0.0>]
rcvd [IPCP ConfNak id=0x1 <addr 10.236.58.172> <ms-dns1 120.80.80.80> <ms-dns2 221.5.88.88>]
sent [IPCP ConfReq id=0x2 <addr 10.236.58.172> <ms-dns1 120.80.80.80> <ms-dns2 221.5.88.88>]
rcvd [IPCP ConfReq id=0x7]
sent [IPCP ConfAck id=0x7]
rcvd [IPCP ConfAck id=0x2 <addr 10.236.58.172> <ms-dns1 120.80.80.80> <ms-dns2 221.5.88.88>]
Could not determine remote IP address: defaulting to 10.64.64.64
local  IP address 10.236.58.172
remote IP address 10.64.64.64
primary   DNS address 120.80.80.80
secondary DNS address 221.5.88.88
```

图 54.3.3.1 4G 连接过程

从图 54.3.3.1 可以看出，4G 网络连接成功，IP 地址为 10.236.58.172。ppp 拨号成功以后就会生成一个名为“ppp0”的网卡，如图 54.3.3.2 所示：

```
ppp0      Link encap:Point-to-Point Protocol
          inet addr:10.236.58.172  P-t-P:10.64.64.64  Mask:255.255.255.255
          UP POINTOPOINT RUNNING NOARP MULTICAST  MTU:1500  Metric:1
          RX packets:4 errors:0 dropped:0 overruns:0 frame:0
          TX packets:9 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:3
          RX bytes:52 (52.0 B)  TX bytes:303 (303.0 B)
```

图 54.3.3.2 ppp0 网卡信息

4G 网络测试需要关闭其他网卡，否则的话网络测试可能有问题，但是我们现在是通过网络启动的系统，并且通过 NFS 挂载的根文件系统，因此我们没法关闭其他的网卡，比如 eth0。为了解决这个问题，我们只能将 uboot、Linux kernel、.dtb 设备树和根文件系统都烧写到板子的 EMMC 里面，然后直接启动 EMMC 上的系统即可，这样就不需要其他网卡工作了。

烧写完成以后重启启动开发板，确保当前开发板只有一个 ME3630 对应的 ppp0 网卡，最后直接 ping 百度官网即可，结果如图 54.3.3.3 所示：

```
root@ATK-MP157:/# ping www.baidu.com
PING www.wshifen.com (104.193.88.123) 56(84) bytes of data.
64 bytes from 104.193.88.123 (104.193.88.123): icmp_seq=1 ttl=50 time=221 ms
64 bytes from 104.193.88.123 (104.193.88.123): icmp_seq=2 ttl=50 time=221 ms
64 bytes from 104.193.88.123 (104.193.88.123): icmp_seq=3 ttl=50 time=230 ms
64 bytes from 104.193.88.123 (104.193.88.123): icmp_seq=4 ttl=50 time=223 ms
```

图 54.3.3.3 EC20 4G 模块 ping 百度官网

从图 54.3.3.3 可以看出，ping 百度地址成功，说明 EC20 ppp 拨号上网成功。

54.3.4 移远 GobiNET 驱动移植

1、GobiNET 移植

EC20 除了可以使用 ppp 拨号上网，也可以使用移远提供的 GobiNnet 驱动，驱动源码我们已经放到了开发板光盘中，路径为：1、例程源码->5、模块驱动源码->3、4G 模块->移远 EC20->EC20_R2.1_Mini_PCIe-C->05 Driver->Linux->GobiNet-> Quectel_Linux&Android_GobiNet_Driver_V1.6.1.zip。将 Quectel_WCDMA&LTE_Linux&Android_GobiNet_Driver_V1.3.0 压缩包里面的所有.c 和.h 文件都拷贝到 Linux 内核中的/driver/net/usb 目录下，也就是图 54.3.4.1 中所示的文件：

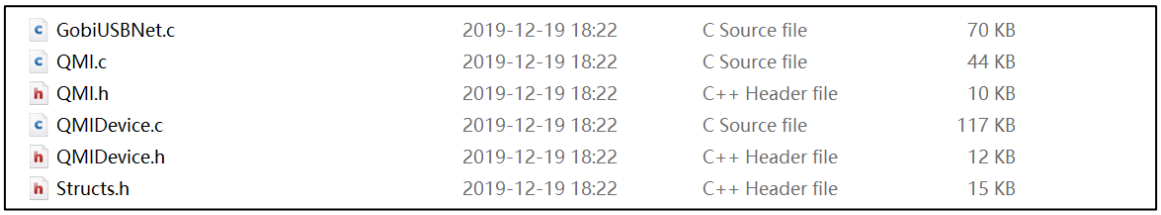


图 54.3.4.1 需要拷贝的文件

拷贝完成以后打开 Linux 内核的 drivers/net/usb/Makefile 文件，在此文件末尾加入如下内容：

示例代码 54.3.4.1 修改后的 drivers/net/usb/Makefile 文件

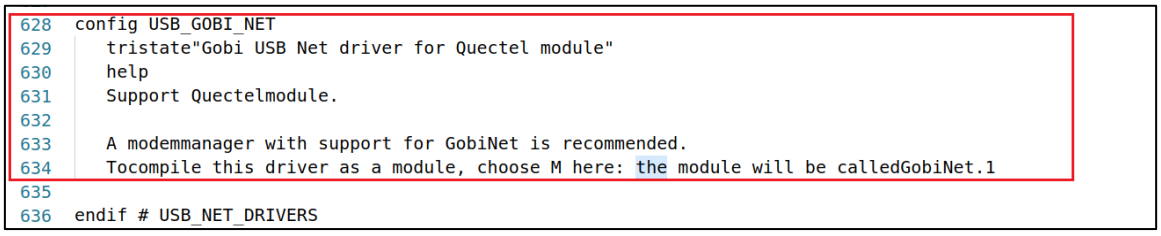
```
1 obj-$(CONFIG_USB_GOBI_NET) += GobiNet.o
2 GobiNet-objs := GobiUSBNet.o QMIDevice.o QMI.o
```

最后在 drivers/net/usb/Kconfig 文件中加入下所示内容：

示例代码 54.3.4.2 drivers/net/usb/Kconfig 要添加的内容

```
1 config USB_GOBI_NET
2   tristate"Gobi USB Net driver for Quectel module"
3   help
4   Support Quectelmodule.
5
6   A modemmanager with support for GobiNet is recommended.
7   Tocompile this driver as a module, choose M here: the module will be
calledGobiNet.1
```

上述内容在 drivers/net/usb/Kconfig 文件中的位置如图 54.3.4.2 所示：



```
628 config USB_GOBI_NET
629   tristate"Gobi USB Net driver for Quectel module"
630   help
631   Support Quectelmodule.
632
633   A modemmanager with support for GobiNet is recommended.
634   Tocompile this driver as a module, choose M here: the module will be calledGobiNet.1
635
636 endif # USB_NET_DRIVERS
```

图 54.3.4.2 内容要添加的位置

完成以后打开 Linux 内核配置界面，使能前面添加的 Gobi 驱动，配置路径如下：

```
-> Device Drivers
    -> [*] Network device support
        -> -*- USB Network Adapters
            -> <*> Gobi USB Net driver for Quectel module
```

配置如图 54.3.4.3 所示：

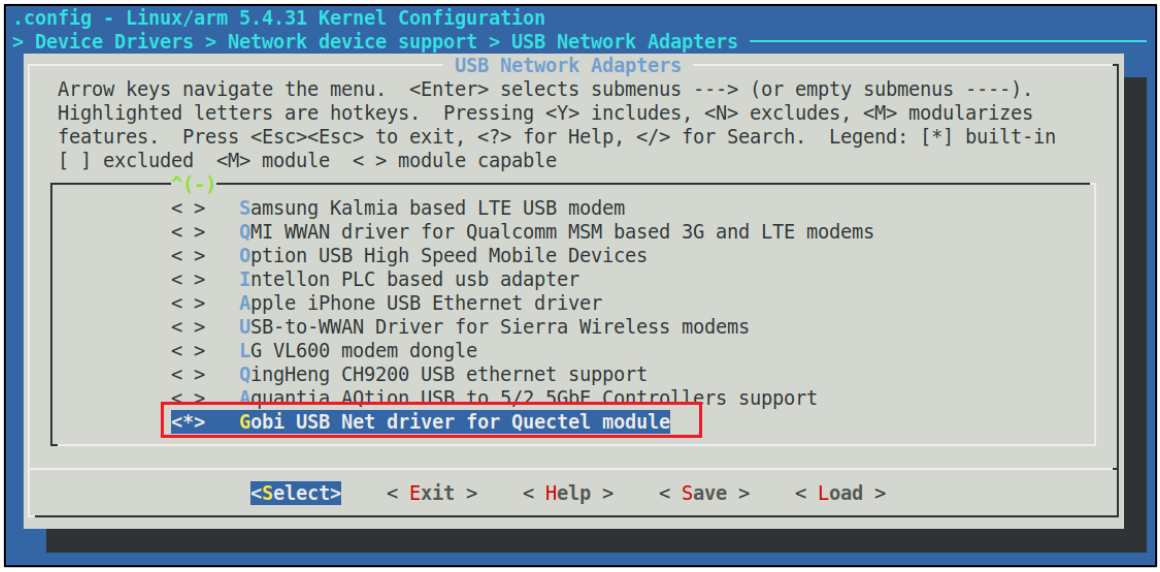


图 54.3.4.3 使能 Gobi 驱动

配置完成以后就重新编译一下 Linux 内核，然后使用新的 uImage 启动开发板。启动以后检查一下“/dev/qcqmi1”这个文件是否存在，如果存在的话就说明 Gobi 驱动工作成功。至此，EC20 的 GobiNET 驱动就已经修改完成了，接下来就是使用 EC20 来实现联网。

2、quectel-CM 移植

GobiNET 移植成功以后如果要想上网，还需要用到移远提供的 quectel-CM 这个软件，这个软件是移远提供的网络管理工具，软件源码已经放到了开发板光盘中，路径为：1、例程源码->5、模块驱动源码->3、4G 模块->移远 EC20->EC20_R2.1_Mini_PCIe-C->05 Driver->Linux->GobiNet->Quectel_QConnectManager_Linux_V2.0.tar.xz。将 Quectel_QConnectManager_Linux_V2.0.tar.xz 这个压缩包拷贝到 Ubuntu 下并解压，得到名为 Quectel_QConnectManager_Linux_V2.0.0 的文件，进入到此文件夹里面，使用如下命令进行交叉编译：

```
make CROSS_COMPILE=arm-none-linux-gnueabihf-
```

编译完成以后得到一个名为“quectel-CM”软件，如图 54.3.4.4 所示：

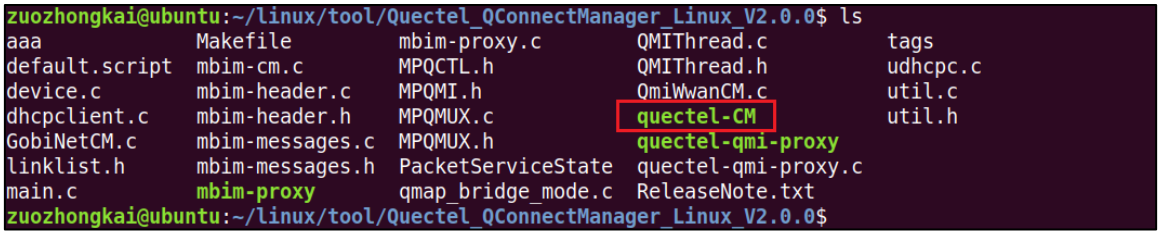


图 54.3.4.4 编译出来的 quectel-CM 软件

将图 54.3.4.4 中编译出来的 quectel-CM 软件拷贝到开发板根文件系统的/usr/bin 目录下，命令如下：

```
sudo cp quectel-CM /home/zuozhongkai/linux/nfs/rootfs/usr/bin/ -f
```

拷贝完成以后就可以使用 quectel-CM 软件来实现 EC20 联网测试了。

54.3.4 EC20 上网测试

使用上一小节移植的 quectel-CM 软件来完成 EC20 的 4G 网络连接，在开发板上输入如下

命令完成 EC20 的 4G 网络连接：

```
quectel-CM -s 3gnet &
```

-s 指定 APN 类型，移动卡的 APN 为 cmnet，联通卡的 APN 为 3gnet，电信卡的 APN 为 cenet。注意，quectel-CM 软件会使用到 udhcpc 来获取 IP 地址。当 4G 网络连接成功以后就会获取到 IP 地址，如图 54.3.4.1 所示：

```
[root@ATK-stm32mp1]:~$ quectel-CM -s 3gnet &
[root@ATK-stm32mp1]:~$ [01-01_00:01:01:422] Quectel_QConnectManager_Linux_V2.0.0
[01-01_00:01:01:425] Find /sys/bus/usb/devices/2-1.7 idVendor=0x2c7c idProduct=0x125
[01-01_00:01:01:425] Auto find qmichannel = /dev/qcqmi1
[01-01_00:01:01:425] Auto find usbnet_adapter = eth1
[01-01_00:01:01:425] qmichannel=/dev/qcqmi1, usbnet_adapter=eth1
[01-01_00:01:01:426] Modem works in QMI mode
[01-01_00:01:01:469] Get clientWDS = 7
[01-01_00:01:01:501] Get clientDMS = 8
[01-01_00:01:01:533] Get clientNAS = 9
[01-01_00:01:01:565] Get clientUIM = 10
[01-01_00:01:01:597] Get clientWDA = 11
[01-01_00:01:01:629] requestBaseBandVersion EC20CEHCR06A03M1G
[01-01_00:01:01:757] requestGetSIMStatus SIMStatus: SIM_READY
[01-01_00:01:01:758] requestSetProfile[1] 3gnet///0
[01-01_00:01:01:823] requestGetProfile[1] 3gnet///0
[01-01_00:01:01:853] requestRegistrationState2 MCC: 460, MNC: 1, PS: Attached, DataCap: LTE
[01-01_00:01:01:885] requestQueryDataCall IPv4ConnectionStatus: DISCONNECTED
[01-01_00:01:01:886] ifconfig eth1 down
[01-01_00:01:01:906] ifconfig eth1 0.0.0.0
[01-01_00:01:01:981] requestRegistrationState2 MCC: 460, MNC: 1, PS: Attached, DataCap: LTE
[01-01_00:01:02:207] requestSetupDataCall WdsConnectionIPv4Handle: 0x86b20a30
[01-01_00:01:02:302] requestQueryDataCall IPv4ConnectionStatus: CONNECTED
[01-01_00:01:02:334] ifconfig eth1 down
[01-01_00:01:02:362] busybox udhcpc -f -n -q -t 5 -i eth1
udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending select for 10.240.151.228
udhcpc: lease of 10.240.151.228 obtained, lease time 7200
[01-01_00:01:02:774] deleting routers
[01-01_00:01:02:839] adding dns 120.80.80.80
[01-01_00:01:02:839] adding dns 221.5.88.88

[root@ATK-stm32mp1]:~$
```

图 54.3.4.1 EC20 4G 网络连接成功并获取到 IP 地址

从图 54.3.4.1 可以看出，EC20 对应的网卡名字为“eth1”，输入“ifconfig eth1”即可查看 eth1 网卡的详细信息，如图 54.3.4.2 所示：

```
[root@ATK-stm32mp1]:~$ ifconfig eth1
eth1      Link encap:Ethernet  HWaddr 92:10:E5:80:9F:57
          inet addr:10.240.5.125  Bcast:10.240.5.127  Mask:255.255.255.252
          UP BROADCAST RUNNING NOARP MULTICAST  MTU:1500  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:612 (612.0 B)  TX bytes:656 (656.0 B)

[root@ATK-stm32mp1]:~$
```

图 54.3.4.2 eth1 网卡详细信息

输入如下命令 ping 百度网址：

```
ping -I 10.240.5.125 www.baidu.com
```

结果如图 54.3.4.3 所示：

```
[root@ATK-stm32mp1]:~$ ping -I 10.240.5.125 www.baidu.com
PING www.baidu.com (163.177.151.110) from 10.240.5.125: 56 data bytes
64 bytes from 163.177.151.110: seq=1 ttl=56 time=50.753 ms
64 bytes from 163.177.151.110: seq=3 ttl=56 time=47.973 ms
64 bytes from 163.177.151.110: seq=5 ttl=56 time=24.388 ms
64 bytes from 163.177.151.110: seq=7 ttl=56 time=47.540 ms
^C
--- www.baidu.com ping statistics ---
8 packets transmitted, 4 packets received, 50% packet loss
round-trip min/avg/max = 24.388/42.663/50.753 ms
[root@ATK-stm32mp1]:~$
```

图 54.3.4.3 ping 百度官网成功

从图 54.3.4.3 可以看出，ping 百度官网没有问题，说明 EC20 4G 模块在 STM32MP157 开发板上工作正常。

第五十五章 Regmap API 实验

我们在前面学习 I2C 和 SPI 驱动的时候，针对 I2C 和 SPI 设备寄存器的操作都是通过相关的 API 函数进行操作的。这样 Linux 内核中就会充斥着大量的重复、冗余代码，但是这些本质上都是对寄存器的操作，所以为了方便内核开发人员统一访问 I2C/SPI 设备的时候，为此引入了 Regmap 子系统，本章我们就来学习一下如何使用 Regmap API 函数来读写 I2C/SPI 设备寄存器。

55.1 Regmap API 简介

55.1.1 什么是 Regmap

Linux 下大部分设备的驱动开发都是操作其内部寄存器，比如 I2C/SPI 设备的本质都是一样的，通过 I2C/SPI 接口读写芯片内部寄存器。芯片内部寄存器也是同样的道理，比如 STM32MP157 的 PWM、TIM 等外设初始化，最终都是要落到寄存器的设置上。

Linux 下使用 i2c_transfer 来读写 I2C 设备中的寄存器，SPI 接口的话使用 spi_write/spi_read 等。I2C/SPI 芯片又非常的多，因此 Linux 内核里面就会充斥了大量的 i2c_transfer 这类的冗余代码，再者，代码的复用性也会降低。比如 icm20608 这个芯片既支持 I2C 接口，也支持 SPI 接口。假设我们在产品设计阶段一开始将 icm20608 设计为 SPI 接口，但是后面发现 SPI 接口不够用，或者 SOC 的引脚不够用，我们需要将 icm20608 改为 I2C 接口。这个时候 icm20608 的驱动就要大改，我们需要将 SPI 接口函数换为 I2C 的，工作量比较大。

基于代码复用的原则，Linux 内核引入了 regmap 模型，regmap 将寄存器访问的共同逻辑抽象出来，驱动开发人员不需要再去纠结使用 SPI 或者 I2C 接口 API 函数，统一使用 regmap API 函数。这样的好处就是统一使用 regmap，降低了代码冗余，提高了驱动的可以移植性。regmap 模型的重点在于：

通过 regmap 模型提供的统一接口函数来访问器件的寄存器，SOC 内部的寄存器也可以使用 regmap 接口函数来访问。

regmap 是 Linux 内核为了减少慢速 I/O 在驱动上的冗余开销，提供了一种通用的接口来操作硬件寄存器。另外，regmap 在驱动和硬件之间添加了 cache，降低了低速 I/O 的操作次数，提高了访问效率，缺点是实时性会降低。

什么情况下会使用 regmap：

①、硬件寄存器操作，比如选用通过 I2C/SPI 接口来读写设备的内部寄存器，或者需要读写 SOC 内部的硬件寄存器。

②、提高代码复用性和驱动一致性，简化驱动开发过程。

③、减少底层 I/O 操作次数，提高访问效率。

本章教程我们就来重点学习一下如何将《第四十五章 Linux SPI 驱动实验》中编写的 SPI 接口的 icm20608 驱动改为使用 regmap API。

55.1.2 Regmap 驱动框架

1、regmap 框架结构

regmap 驱动框架如下图所示：

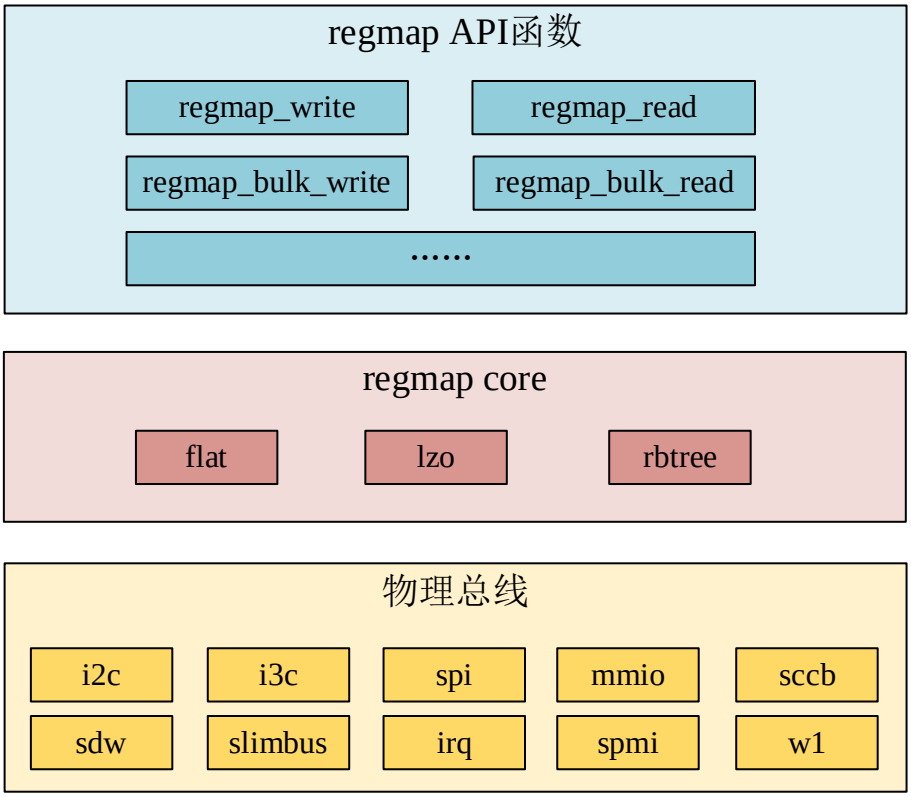


图 55.1.2.1 regmap 框架

regmap 框架分为三层：

①、底层物理总线：regmap 就是对不同的物理总线进行封装，目前 regmap 支持的物理总线有 i2c、i3c、spi、mmio、sccb、sdw、slimbus、irq、spmi 和 w1。

②、regmap 核心层，用于实现 regmap，我们不用关心具体实现。

③、regmap API 抽象层，regmap 向驱动编写人员提供的 API 接口，驱动编写人员使用这些 API 接口来操作具体的芯片设备，也是驱动编写人员重点要掌握的。

2、regmap 结构体

Linux 内核将 regmap 框架抽象为 regmap 结构体，这个结构体定义在文件 include/linux/regmap.h 中，结构体内容如下(有缩减)：

示例代码 55.1.2.1 regmap 结构体

```
49 struct regmap {
50     union {
51         struct mutex mutex;
52         struct {
53             spinlock_t spinlock;
54             unsigned long spinlock_flags;
55         };
56     };
57     regmap_lock lock;
58     regmap_unlock unlock;
59     void *lock_arg; /* This is passed to lock/unlock functions */
60     gfp_t alloc_flags;
```

```
......
89      unsigned int max_register;
90      bool (*writeable_reg)(struct device *dev, unsigned int reg);
91      bool (*readable_reg)(struct device *dev, unsigned int reg);
92      bool (*volatile_reg)(struct device *dev, unsigned int reg);
93      bool (*precious_reg)(struct device *dev, unsigned int reg);
94      bool (*writeable_noinc_reg)(struct device *dev,
                                    unsigned int reg);
95      bool (*readable_noinc_reg)(struct device *dev,
                                   unsigned int reg);
96      const struct regmap_access_table *wr_table;
97      const struct regmap_access_table *rd_table;
98      const struct regmap_access_table *volatile_table;
99      const struct regmap_access_table *precious_table;
100     const struct regmap_access_table *wr_noinc_table;
101     const struct regmap_access_table *rd_noinc_table;
102
103     int (*reg_read)(void *context, unsigned int reg,
                            unsigned int *val);
104     int (*reg_write)(void *context, unsigned int reg,
                         unsigned int val);
105     int (*reg_update_bits)(void *context, unsigned int reg,
106                 unsigned int mask, unsigned int val);
......
159
160     struct rb_root range_tree;
161     void *selector_work_buf;  /* Scratch buffer used for selector */
162
163     struct hwspinlock *hwlock;
164 };
```

要使用 regmap，肯定要先给驱动分配一个具体的 regmap 结构体实例，一会讲解如何分配 regmap 实例。大家可以看到示例代码 55.1.2.1 中第 90~101 行有很多的函数以及 table，这些需要驱动编写人员根据实际情况选择性的初始化，regmap 的初始化通过结构体 regmap_config 来完成。

2、regmap_config 结构体

顾名思义，regmap_config 结构体就是用来初始化 regmap 的，这个结构体也定义在 include/linux/regmap.h 文件中，结构体内容如下：

示例代码 55.1.2.2 regmap_config 结构体

```
352 struct regmap_config {
353     const char *name;
354
355     int reg_bits;
356     int reg_stride;
```

```
357     int pad_bits;
358     int val_bits;
359
360     bool (*writeable_reg)(struct device *dev, unsigned int reg);
361     bool (*readable_reg)(struct device *dev, unsigned int reg);
362     bool (*volatile_reg)(struct device *dev, unsigned int reg);
363     bool (*precious_reg)(struct device *dev, unsigned int reg);
364     bool (*writeable_noinc_reg)(struct device *dev,
                                    unsigned int reg);
365     bool (*readable_noinc_reg)(struct device *dev,
                                   unsigned int reg);
366
367     bool disable_locking;
368     regmap_lock lock;
369     regmap_unlock unlock;
370     void *lock_arg;
371
372     int (*reg_read)(void *context, unsigned int reg,
                        unsigned int *val);
373     int (*reg_write)(void *context, unsigned int reg,
                         unsigned int val);
374
375     bool fast_io;
376
377     unsigned int max_register;
378     const struct regmap_access_table *wr_table;
379     const struct regmap_access_table *rd_table;
380     const struct regmap_access_table *volatile_table;
381     const struct regmap_access_table *precious_table;
382     const struct regmap_access_table *wr_noinc_table;
383     const struct regmap_access_table *rd_noinc_table;
384     const struct reg_default *reg_defaults;
385     unsigned int num_reg_defaults;
386     enum regcache_type cache_type;
387     const void *reg_defaults_raw;
388     unsigned int num_reg_defaults_raw;
389
390     unsigned long read_flag_mask;
391     unsigned long write_flag_mask;
392     bool zero_flag_mask;
393
394     bool use_single_read;
395     bool use_single_write;
```

```
396     bool can_multi_write;
397
398     enum regmap_endian reg_format_endian;
399     enum regmap_endian val_format_endian;
400
401     const struct regmap_range_cfg *ranges;
402     unsigned int num_ranges;
403
404     bool use_hwlock;
405     unsigned int hwlock_id;
406     unsigned int hwlock_mode;
407 };
```

Linux 内核里面已经对 regmap_config 各个成员变量进行了详细的讲解，这里我们只看一些比较重要的：

第 353 行 name：名字。

第 355 行 reg_bits：寄存器地址位数，必填字段。

第 356 行 reg_stride：寄存器地址步长。

第 357 行 pad_bits：寄存器和值之间的填充位数。

第 358 行 val_bits：寄存器值位数，必填字段。

第 360 行 writeable_reg：可选的可写回调函数，寄存器可写的话此回调函数就会被调用，并返回 true。

第 361 行 readable_reg：可选的可读回调函数，寄存器可读的话此回调函数就会被调用，并返回 true。

第 362 行 volatile_reg：可选的回调函数，当寄存器值不能缓存的时候此回调函数就会被调用，并返回 true。

第 363 行 precious_reg：当寄存器值不能被读出来的时候此回调函数会被调用，比如很多中断状态寄存器读清零，读这些寄存器就可以清除中断标志位，但是并没有读出这些寄存器内部的值。

第 372 行 reg_read：可选的读操作回调函数，所有读寄存器的操作此回调函数就会执行。

第 373 行 reg_write：可选的写操作回调函数，所有写寄存器的操作此回调函数就会执行。

第 375 行 fast_io：快速 I/O，使用 spinlock 替代 mutex 来提升锁性能。

第 377 行 max_register：有效的最大寄存器地址，可选。

第 378 行 wr_table：可写的地址范围，为 regmap_access_table 结构体类型。后面的 rd_table、volatile_table、precious_table、wr_noinc_table 和 rd_noinc_table 同理。

第 384 行 reg_defaults：寄存器模式值，为 reg_default 结构体类型，此结构体有两个成员变量：reg 和 def，reg 是寄存器地址，def 是默认值。

第 385 行 num_reg_defaults：默认寄存器表中的元素个数。

第 390 行 read_flag_mask：读标志掩码。

第 391 行 write_flag_mask：写标志掩码。

关于 regmap_config 结构体成员变量就介绍这些，其他没有介绍的自行查阅 Linux 内核中的相关描述。

55.1.3 Regmap 操作函数

1、Regmap 申请与初始化

前面说了，regmap 支持多种物理总线，比如 I2C 和 SPI，我们需要根据所使用的接口来选择合适的 regmap 初始化函数。Linux 内核提供了针对不同接口的 regmap 初始化函数，SPI 接口初始化函数为 regmap_init_spi，函数原型如下：

```
struct regmap * regmap_init_spi(struct spi_device              *spi,
                                const struct regmap_config     *config)
```

函数参数和返回值含义如下：

spi：需要使用 regmap 的 spi_device。

config：regmap_config 结构体，需要程序编写人员初始化一个 regmap_config 实例，然后将其地址赋值给此参数。

返回值：申请到的并进过初始化的 regmap。

I2C 接口的 regmap 初始化函数为 regmap_init_i2c，函数原型如下：

```
struct regmap * regmap_init_i2c(struct i2c_client              *i2c,
                                const struct regmap_config     *config)
```

函数参数和返回值含义如下：

i2c：需要使用 regmap 的 i2c_client。

config：regmap_config 结构体，需要程序编写人员初始化一个 regmap_config 实例，然后将其地址赋值给此参数。

返回值：申请到的并进过初始化的 regmap。

还有很多其他物理接口对应的 regmap 初始化函数，这里就不介绍了，大家直接查阅 Linux 内核即可，基本和 SPI/I2C 的初始化函数相同

在退出驱动的时候需要释放掉申请到的 regmap，不管是什么接口，全部使用 regmap_exit 这个函数来释放 regmap，函数原型如下：

```
void regmap_exit(struct regmap *map)
```

函数参数和返回值含义如下：

map：需要释放的 regmap

返回值：无。

我们一般会在 probe 函数中初始化 regmap_config，然后申请并初始化 regmap。

2、regmap 设备访问 API 函数

不管是 I2C 还是 SPI 等接口，还是 SOC 内部的寄存器，对于寄存器的操作就两种：读和写。regmap 提供了最核心的两个读写操作：regmap_read 和 regmap_write。这两个函数分别用来读/写寄存器，regmap_read 函数原型如下：

```
int regmap_read(struct regmap     *map,
                unsigned int      reg,
                unsigned int      *val)
```

函数参数和返回值含义如下：

map：要操作的 regmap。

reg：要读的寄存器。

val：读到的寄存器值。

返回值：0，读取成功；其他值，读取失败。

regmap_write 函数原型如下：

```
int regmap_write(struct regmap  *map,
                 unsigned int   reg,
                 unsigned int   val)
```

函数参数和返回值含义如下：

map：要操作的 regmap。

reg：要写的寄存器。

val：要写的寄存器值。

返回值：0，写成功；其他值，写失败。

在 regmap_read 和 regmap_write 的基础上还衍生出了其他一些 regmap 的 API 函数，首先是 regmap_update_bits 函数，看名字就知道，此函数用来修改寄存器指定的 bit，函数原型如下：

```
int regmap_update_bits (struct regmap  *map,
                        unsigned int   reg,
                        unsigned int   mask,
                        unsigned int   val,
```

函数参数和返回值含义如下：

map：要操作的 regmap。

reg：要操作的寄存器。

mask：掩码，需要更新的位必须在掩码中设置为 1。

val：需要更新的位值。

返回值：0，写成功；其他值，写失败。

比如要将寄存器的 bit1 和 bit2 置 1，那么 mask 应该设置为 0X00000011，此时 val 的 bit1 和 bit2 应该设置为 1，也就是 0Xxxxxxx11。如果要清除寄存器的 bit4 和 bit7，那么 mask 应该设置为 0X10010000，val 的 bit4 和 bit7 设置为 0，也就是 0X0xx0xxxx。

接下来看一下 regmap_bulk_read 函数，此函数用于读取多个寄存器的值，函数原型如下：

```
int regmap_bulk_read(struct regmap  *map,
                     unsigned int   reg,
                     void           *val,
                     size_t         val_count)
```

函数参数和返回值含义如下：

map：要操作的 regmap。

reg：要读取的第一个寄存器。

val：读取到的数据缓冲区。

val_count：要读取的寄存器数量。

返回值：0，写成功；其他值，读失败。

另外也有多个寄存器写函数 regmap_bulk_write，函数原型如下：

```
int regmap_bulk_write(struct regmap  *map,
                      unsigned int   reg,
                      const void     *val,
                      size_t         val_count)
```

函数参数和返回值含义如下：

map：要操作的 regmap。

reg：要写的第一个寄存器。

val：要写的寄存器数据缓冲区。

val_count：要写的寄存器数量。

返回值：0，写成功；其他值，读失败。

关于 regmap 常用到 API 函数就讲解到这里，还有很多其他功能的 API 函数，大家自行查阅 Linux 内核即可，内核里面对每个 API 函数都有详细的讲解。

55.1.4 regmap_config 掩码设置

结构体 regmap_config 里面有三个关于掩码的成员变量：read_flag_mask、write_flag_mask 和 zero_flag_mask，这三个掩码非常重要，本节我们来学习一下如何使用这三个掩码。我们在学习 icm20608 的时候讲过了，icm20608 支持 i2c 和 spi 接口，但是当使用 spi 接口的时候，读取 icm20608 寄存器的时候地址最高位必须置 1，写内部寄存器的是时候地址最高位要设置为 0。因此这里就涉及到对寄存器地址最高位的操作，在《第四十五章 SPI 驱动实验》中我们在使用 SPI 接口函数读取 icm20608 内部寄存器的时候手动将寄存器地址的最高位置 1，代码如下所示：

示例代码 55.1.4.1 icm20608 驱动

```
1  static int icm20608_read_regs(struct icm20608_dev *dev, u8 reg,
                                 void *buf, int len)
2  {
3
......
21      txdata[0] = reg | 0x80;      /* 写数据的时候首寄存器地址 bit7 要置 1 */
22      t->tx_buf = txdata;          /* 要发送的数据 */
23      t->rx_buf = rxdata;          /* 要读取的数据 */
24      t->len = len+1;              /* t->len=发送的长度+读取的长度 */
25      spi_message_init(&m);        /* 初始化 spi_message */
26      spi_message_add_tail(t, &m);
27      ret = spi_sync(spi, &m);     /* 同步发送 */
......
39      return ret;
40 }
```

示例代码 55.1.4.1 就是标准的 SPI 驱动，其中第 21 行，将寄存器的地址 bit7 置 1，表示这是一个读操作。

当我们使用 regmap 的时候就不需要手动将寄存器地址的 bit7 置 1，在初始化 regmap_config 的时候直接将 read_flag_mask 设置为 0X80 即可，这样通过 regmap 读取 SPI 内部寄存器的时候就会将寄存器地址与 read_flag_mask 进行或运算，结果就是将 bit7 置 1，但是整个过程不需要我们来操作，全部由 regmap 框架来完成的。

同理 write_flag_mask 用法也一样的，只是 write_flag_mask 用于写寄存器操作。

打开 regmap-spi.c 文件，这个文件就是 regmap 的 spi 总线文件，找到如下所示内容：

示例代码 55.1.4.2 regmap-spi.c 代码段

```
101 static const struct regmap_bus regmap_spi = {
102     .write = regmap_spi_write,
103     .gather_write = regmap_spi_gather_write,
104     .async_write = regmap_spi_async_write,
105     .async_alloc = regmap_spi_async_alloc,
```

```
106     .read = regmap_spi_read,
107     .read_flag_mask = 0x80,
108     .reg_format_endian_default = REGMAP_ENDIAN_BIG,
109     .val_format_endian_default = REGMAP_ENDIAN_BIG,
110 };
111
112 struct regmap *__regmap_init_spi(struct spi_device *spi,
113                 const struct regmap_config *config,
114                 struct lock_class_key *lock_key,
115                 const char *lock_name)
116 {
117     return __regmap_init(&spi->dev, &regmap_spi, &spi->dev, config,
118                 lock_key, lock_name);
119 }
```

第 101~110 行初始化了一个 regmap_bus 实例：regmap_spi，我们重点看一下第 107 行中 read_flag_mask 默认为 0X80。注意，这里是将 regmap_bus 的 read_flag_mask 成员变量设置为 0X80。regmap_bus 结构体大家自行查看一下，这里就不讲了。

第 112~119 行__regmap_init_spi 函数，前面说了要想在 spi 总线中使用 regmap 框架，首先要使用 regmap_init_spi 函数用于并申请一个 SPI 总线的 regmap。regmap_init_spi 函数会调用这里的__regmap_init_spi 函数，从第 117 行可以看出__regmap_init_spi 函数只是对__regmap_init 的简单封装，因此最终完成 regmap 申请并初始化的是__regmap_init 函数。在__regmap_init 函数中找到如下所示内容：

示例代码 55.1.4.3 __regmap_init 函数代码段

```
812 if (config->read_flag_mask ||
813     config->write_flag_mask ||
814     config->zero_flag_mask) {
815     map->read_flag_mask = config->read_flag_mask;
816     map->write_flag_mask = config->write_flag_mask;
817 } else if (bus) {
818     map->read_flag_mask = bus->read_flag_mask;
819 }
```

第 812~817 行就是用 regmap_config 中的读写掩码来初始化 regmap_bus 中的掩码。由于 regmap_spi 默认将 read_flag_mask 设置为 0X80，当你所使用的 SPI 设备不需要读掩码，在初始化 regmap_config 的时候一定要将 read_flag_mask 设置为 0X00，并且也要将 zero_flag_mask 设置为 true。

regmap 框架就讲解到这里，接下来学习如何将《第四十五章 Linux SPI 驱动实验》中编写的 icm20608 驱动改为 regmap 框架。

55.2 实验程序编写

本实验不需要对设备树做任何修改。

本实验对应的例程路径为：开发板光盘-> 01、程序源码->02、Linux 驱动例程->30_regmap->spi。

1、修改设备结构体，添加 regmap 和 regmap_config

regmap 框架的核心就是 regmap 和 regmap_config 结构体，我们一般都是在自定义的设备结构体里面添加这两个类型的成员变量，所以我们首先在 icm20608_dev 结构体里面添加 regmap 和 regmap_config，修改完成以后的 icm20608_dev 结构体内容如下：

示例代码 55.2.11 修改后的 icm20608_dev 结构体

```
1  struct icm20608_dev {
2      struct spi_device *spi;      /* spi 设备            */
3      dev_t devid;                 /* 设备号              */
4      struct cdev cdev;            /* cdev                */
5      struct class *class;         /* 类                  */
6      struct device *device;       /* 设备                */
7      struct device_node  *nd;     /* 设备节点            */
8      signed int gyro_x_adc;       /* 陀螺仪 X 轴原始值    */
9      signed int gyro_y_adc;       /* 陀螺仪 Y 轴原始值    */
10     signed int gyro_z_adc;       /* 陀螺仪 Z 轴原始值    */
11     signed int accel_x_adc;      /* 加速度计 X 轴原始值  */
12     signed int accel_y_adc;      /* 加速度计 Y 轴原始值  */
13     signed int accel_z_adc;      /* 加速度计 Z 轴原始值  */
14     signed int temp_adc;         /* 温度原始值          */
15     struct regmap *regmap;
16     struct regmap_config regmap_config;
17 };
```

第 15 行，regmap 指针变量，regmap 我们需要使用 regmap_init_spi 函数来申请和初始化，所以这里是指针类型。

第 16 行，regmap_config 结构体成员变量，从来配置 regmap。

2、初始化 regmap

一般在 probe 函数中初始化 regmap，本章节就是 icm20608_probe 函数，初始化内容如下：

示例代码 55.2.1.2 icm20608_probe 函数

```
1  static int icm20608_probe(struct spi_device *spi)
2  {
3      int ret;
4      struct icm20608_dev *icm20608dev;
5
6      /* 分配 icm20608dev 对象的空间 */
7      icm20608dev = devm_kzalloc(&spi->dev, sizeof(*icm20608dev),
                                  GFP_KERNEL);
8      if(!icm20608dev)
9          return -ENOMEM;
10
11     /* 初始化 regmap_config 设置 */
12     icm20608dev->regmap_config.reg_bits = 8; /* 寄存器长度 8bit */
13     icm20608dev->regmap_config.val_bits = 8;  /* 值长度 8bit */
14     icm20608dev->regmap_config.read_flag_mask = 0x80;  /* 读掩码 */
15
```

```
16      /* 初始化 IIC 接口的 regmap */
17      icm20608dev->regmap = regmap_init_spi(spi,
                                &icm20608dev->regmap_config);
18      if (IS_ERR(icm20608dev->regmap)) {
19          return  PTR_ERR(icm20608dev->regmap);
20      }
21
22      /* 注册字符设备驱动 */
23      /* 1、创建设备号 */
24      ret = alloc_chrdev_region(&icm20608dev->devid, 0, ICM20608_CNT,
                                  ICM20608_NAME);
25      if(ret < 0) {
26          pr_err("%s Couldn't alloc_chrdev_region, ret=%d\r\n",
                    ICM20608_NAME, ret);
27          goto del_regmap;
28      }
......
61
62      return 0;
63  destroy_class:
64      device_destroy(icm20608dev->class, icm20608dev->devid);
65  del_cdev:
66      cdev_del(&icm20608dev->cdev);
67  del_unregister:
68      unregister_chrdev_region(icm20608dev->devid, ICM20608_CNT);
69  del_regmap:
70      regmap_exit(icm20608dev->regmap);
71      return -EIO;
72  }
```

第 11~14 行，regmap_config 的初始化，icm20608 的寄存器地址地址长度为 8bit，寄存器值也是 8bit，因此 reg_bits 和 val_bits 都设置为 8。由于 icm20608 通过 SPI 接口读取的时候地址寄存器最高位要设置为 1，因此 read_flag_mask 设置为 0X80。

第 17 行，通过 regmap_init_spi 函数来申请并初始化 SPI 总线的 regmap。

第 70 行，如果要删除 regmap 就使用 regmap_exit 函数。

同理，在 remove 函数中就要删除 probe 里面申请的 regmap，icm20608_remove 函数内容如下：

示例代码 55.2.1.3 icm20608_remove 函数

```
1  static int icm20608_remove(struct spi_device *spi)
2  {
3      struct icm20608_dev *icm20608dev = spi_get_drvdata(spi);
4
......
12      /* 4、注销类 */
```

```
13      class_destroy(icm20608dev->class);
14      /* 5、删除 regmap */
15      regmap_exit(icm20608dev->regmap);
16      return 0;
17  }
```

第 17 行，卸载驱动的时候使用 regmap_exit 删除掉 probe 函数中申请的 regmap。

3、读写设备内部寄存器

regmap 已经设置好了，接下来就是使用 regmap API 函数来读写 icm20608 内部寄存器了。以前我们使用 spi 驱动框架编写读写函数，现在直接使用 regmap_read、regmap_write 的函数即可，修改后的 icm20608 内部寄存器读写函数如下：

示例代码 55.2.1.4 icm20608 寄存器读写函数

```
1  /*
2   * @description     : 读取 icm20608 指定寄存器值，读取一个寄存器
3   * @param - dev     : icm20608 设备
4   * @param - reg     : 要读取的寄存器
5   * @return          : 读取到的寄存器值
6   */
7  static unsigned char icm20608_read_onereg(struct icm20608_dev *dev,
                                             u8 reg)
8  {
9       u8 ret;
10      unsigned int data;
11
12      ret = regmap_read(dev->regmap, reg, &data);
13      return (u8)data;
14 }
15
16 /*
17  * @description     : 向 icm20608 指定寄存器写入指定的值，写一个寄存器
18  * @param - dev     : icm20608 设备
19  * @param - reg     : 要写的寄存器
20  * @param - data    : 要写入的值
21  * @return          : 无
22  */
23
24 static void icm20608_write_onereg(struct icm20608_dev *dev, u8 reg,
                                     u8 value)
25 {
26      regmap_write(dev->regmap,  reg, value);
27 }
28
29 /*
30  * @description     : 读取 ICM20608 的数据，读取原始数据，包括三轴陀螺仪、
```

```
31  *                   : 三轴加速度计和内部温度。
32  * @param - dev      : ICM20608 设备
33  * @return           : 无。
34  */
35 void icm20608_readdata(struct icm20608_dev *dev)
36 {
37     u8 ret;
38     unsigned char data[14];
39
40     ret = regmap_bulk_read(dev->regmap, ICM20_ACCEL_XOUT_H, data,
                              14);
41
42     dev->accel_x_adc = (signed short)((data[0] << 8) | data[1]);
43     dev->accel_y_adc = (signed short)((data[2] << 8) | data[3]);
44     dev->accel_z_adc = (signed short)((data[4] << 8) | data[5]);
45     dev->temp_adc    = (signed short)((data[6] << 8) | data[7]);
46     dev->gyro_x_adc  = (signed short)((data[8] << 8) | data[9]);
47     dev->gyro_y_adc  = (signed short)((data[10] << 8) | data[11]);
48     dev->gyro_z_adc  = (signed short)((data[12] << 8) | data[13]);
49 }
```

第 7~14 行，icm20608_read_onereg 函数用于读取 icm20608 内部单个寄存器，这里直接使用 regmap_read 函数来完成寄存器读取操作。

第 24~27 行，icm20608_write_onereg 函数用于向 icm20608 指定寄存器写入数据，这里也直接使用 regmap_write 函数来完成写操作。

第 35~49 行，icm20608_readdata 函数用于读取 icm20608 内部陀螺仪、加速度计和温度计的数据，从 ICM20_ACCEL_XOUT_H 寄存器开始，连续读取 14 个寄存器。这里直接使用 regmap_bulk_read 函数来显示多个寄存器的读取。

对比《第四十五章 Linux SPI 驱动实验》中的 icm20608 驱动，采用 regmap API 以后驱动程序精简了很多。具体涉及到 SPI 总线的部分全部由 regmap 来处理了，驱动编写人员不用管，极大的方便了我们的驱动编写。而且驱动的可以执行提高了很多，即使将来更换为 IIC 接口，也只需要更改很少的一部分即可。

55.3 运行测试

测试 APP 直接第四十五章编写的 icm20608App.c 即可。测试方法也和四十五章一样，输入如下命令：

```
depmod                          //第一次加载驱动的时候需要运行此命令
modprobe icm20608.ko            //加载驱动模块
./icm20608App /dev/icm20608     //app 读取内部数据
```

如果 regmap API 工作正常，那么就会正确的初始化 icm20608，并且读出传感器数据，结果和四十五章一样，如图 55.3.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe icm20608
ICM20608 ID = 0XAF
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./icm20608App /dev/icm20608

原始值:
gx = -43, gy = 8, gz = -5
ax = -6, ay = 59, az = 2072
temp = 4039
实际值:act gx = -2.62°/S, act gy = 0.49°/S, act gz = -0.30°/S
act ax = -0.00g, act ay = 0.03g, act az = 1.01g
act temp = 37.28°C
```

图 55.3.1 获取到的 ICM20608 数据

IIC 总线的 regmap 框架基本和 SPI 一样，只是需要使用 regmap_init_i2c 来申请并初始化对应的 regmap，同样都是使用 regmap_read 和 regmap_write 来读写 I2C 设备内部寄存器。这里我们也已经将《第四十一章 Linux I2C 驱动实验》中的 ap3216c 驱动改为了 regmap API 接口的，相应的驱动程序已经放到了开发板光盘中，路径为：本实验对应的例程路径为：开发板光盘->01、程序源码->02、Linux 驱动例程->30_regmap->iic。大家自行查阅，这里就不详细详解了。

第五十六章 Linux IIO 驱动实验

工业场合里面也有大量的模拟量和数字量之间的转换，也就是我们常说的 ADC 和 DAC。而且随着手机、物联网、工业物联网和可穿戴设备的爆发，传感器的需求只持续增强。比如手机或者手环里面的加速度计、光传感器、陀螺仪、气压计、磁力计等，这些传感器本质上都是 ADC，大家注意查看这些传感器的手册，会发现他们内部都会有个 ADC，传感器对外提供 IIC 或者 SPI 接口，SOC 可以通过 IIC 或者 SPI 接口来获取到传感器内部的 ADC 数值，从而得到想要测量的结果。Linux 内核为了管理这些日益增多的 ADC 类传感器，特地推出了 IIO 子系统，本章我们就来学习如何使用 IIO 子系统来编写 ADC 类传感器驱动。

56.1 IIO 子系统简介

IIO 全称是 Industrial I/O，翻译过来就是工业 I/O，大家不要看到“工业”两个字就觉得 IIO 是只用于工业领域的。大家一般在搜索 IIO 子系统的时候，会发现大多数讲的都是 ADC，这是因为 IIO 就是为 ADC 类传感器准备的，当然了 DAC 也是可以的。大家常用的陀螺仪、加速度计、电压/电流测量芯片、光照传感器、压力传感器等内部都是有个 ADC，内部 ADC 将原始的模拟数据转换为数字量，然后通过其他的通信接口，比如 IIC、SPI 等传输给 SOC。

因此，当你使用的传感器本质是 ADC 或 DAC 器件的时候，可以优先考虑使用 IIO 驱动框架。

56.1.1 iio_dev

1、iio_dev 结构体

IIO 子系统使用结构体 iio_dev 来描述一个具体 IIO 设备，此设备结构体定义在 include/linux/iio/iio.h 文件中，结构体内容如下(有省略)：

示例代码 56.1.1.1 iio_dev 结构体

```
525 struct iio_dev {
526     int                         id;
527     struct module               *driver_module;
528
529     int                         modes;
530     int                         currentmode;
531     struct device               dev;
532
533     struct iio_event_interface  *event_interface;
534
535     struct iio_buffer           *buffer;
536     struct list_head            buffer_list;
537     int                         scan_bytes;
538     struct mutex                mlock;
539
540     const unsigned long         *available_scan_masks;
541     unsigned                    masklength;
542     const unsigned long         *active_scan_mask;
543     bool                        scan_timestamp;
544     unsigned                    scan_index_timestamp;
545     struct iio_trigger          *trig;
546     bool                        trig_readonly;
547     struct iio_poll_func        *pollfunc;
548     struct iio_poll_func        *pollfunc_event;
549
550     struct iio_chan_spec const  *channels;
551     int                         num_channels;
552
```

```
553     struct list_head            channel_attr_list;
554     struct attribute_group      chan_attr_group;
555     const char                  *name;
556     const struct iio_info       *info;
557     clockid_t                   clock_id;
558     struct mutex                info_exist_lock;
559     const struct iio_buffer_setup_ops   *setup_ops;
560     struct cdev                 chrdev;
561 };
```

我们来看一下 iio_dev 结构体中几个比较重要的成员变量：

第 529 行，modes 为设备支持的模式，可选择的模如表 56.1.1.1 所示：

模式	描述
INDIO_DIRECT_MODE	提供 sysfs 接口。
INDIO_BUFFER_TRIGGERED	支持硬件缓冲触发。
INDIO_BUFFER_SOFTWARE	支持软件缓冲触发。
INDIO_BUFFER_HARDWARE	支持硬件缓冲区。
INDIO_EVENT_TRIGGERED	支持事件触发。
INDIO_HARDWARE_TRIGGERED	支持硬件触发
INDIO_ALL_BUFFER_MODES	支持所有缓冲模式。
INDIO_ALL_TRIGGERED_MODES	支持所有触发模式。

表 56.1.1.1 模式

第 530 行，currentmode 为当前模式。

第 535 行，buffer 为缓冲区。

第 536 行，buffer_list 为当前匹配的缓冲区列表。

第 537 行，scan_bytes 为捕获到，并且提供给缓冲区的字节数。

第 540 行，available_scan_masks 为可选的扫描位掩码，使用触发缓冲区的时候可以通过设置掩码来确定使能哪些通道，使能以后的通道会将捕获到的数据发送到 IIO 缓冲区。

第 542 行，active_scan_mask 为缓冲区已经开启的通道掩码。只有这些使能了的通道数据才能被发送到缓冲区。

第 543 行，scan_timestamp 为扫描时间戳，如果使能以后会将捕获时间戳放到缓冲区里面。

第 545 行，trig 为 IIO 设备当前触发器，当使用缓冲模式的时候。

第 547 行，pollfunc 为一个函数，在接收到的触发器上运行。

第 550 行，channels 为 IIO 设备通道，为 iio_chan_spec 结构体类型，稍后会详细讲解 IIO 通道。

第 551 行，num_channels 为 IIO 设备的通道数。

第 555 行，name 为 IIO 设备名字。

第 556 行，info 为 iio_info 结构体类型，这个结构体里面有很多函数，需要驱动开发人员编写，非常重要！我们从用户空间读取 IIO 设备内部数据，最终调用的就是 iio_info 里面的函数。稍后会详细讲解 iio_info 结构体。

第 559 行，setup_ops 为 iio_buffer_setup_ops 结构体类型，内容如下：

示例代码 56.1.1.2 iio_buffer_setup_ops 结构体

```
474 struct iio_buffer_setup_ops {
475     int (*preenable)(struct iio_dev *);    /* 缓冲区使能之前调用 */
```

```
476     int (*postenable)(struct iio_dev *);   /* 缓冲区使能之后调用 */
477     int (*predisable)(struct iio_dev *);   /* 缓冲区禁用之前调用 */
478     int (*postdisable)(struct iio_dev *); /* 缓冲区禁用之后调用 */
479     bool (*validate_scan_mask)(struct iio_dev *indio_dev,
480          const unsigned long *scan_mask); /* 检查扫描掩码是否有效 */
481 };
```

可以看出 iio_buffer_setup_ops 里面都是一些回调函数，在使能或禁用缓冲区的时候会调用这些函数。如果未指定的话就默认使用 iio_triggered_buffer_setup_ops。

继续回到示例代码 56.1.1.1 中第 560 行，chrdev 为字符设备，由 IIO 内核创建。

2、iio_dev 申请与释放

在使用之前要先申请 iio_dev，申请函数为 iio_device_alloc，函数原型如下：

```
struct iio_dev *iio_device_alloc(int sizeof_priv)
```

函数参数和返回值含义如下：

sizeof_priv：私有数据内存空间大小，一般我们会将自己定义的设备结构体变量作为 iio_dev 的私有数据，这样可以直接通过 iio_device_alloc 函数同时完成 iio_dev 和设备结构体变量的内存申请。申请成功以后使用 iio_priv 函数来得到自定义的设备结构体变量首地址。

返回值：如果申请成功就返回 iio_dev 首地址，如果失败就返回 NULL。

一般 iio_device_alloc 和 iio_priv 之间的配合使用如下所示：

示例代码 56.1.1.3 iio_device_alloc 和 iio_priv 函数的使用

```
1  struct icm20608_dev *dev;
2  struct iio_dev *indio_dev;
3
4  /*  1、申请 iio_dev 内存 */
5  indio_dev = iio_device_alloc(sizeof(*dev));
6  if (!indio_dev)
7   return -ENOMEM;
8
9  /* 2、获取设备结构体变量地址 */
10 dev = iio_priv(indio_dev);
```

第 1 行，icm20608_dev 是自定义的设备结构体。

第 2 行，indio_dev 是 iio_dev 结构体变量指针。

第 5 行，使用 iio_device_alloc 函数来申请 iio_dev，并且一起申请了 icm2060_dev 的内存。

第 10 行，使用 iio_priv 函数从 iio_dev 中提取出私有数据，也就是 icm2608_dev 这个自定义结构体变量首地址。

如果要释放 iio_dev，需要使用 iio_device_free 函数，函数原型如下：

```
void iio_device_free(struct iio_dev *indio_dev)
```

函数参数和返回值含义如下：

indio_dev：需要释放的 iio_dev。

返回值：无。

也可以使用 devm_iio_device_alloc 来分配 iio_dev，这样就不需要我们手动调用 iio_device_free 函数完成 iio_dev 的释放工作。

3、iio_dev 注册与注销

前面分配好 iio_dev 以后就要初始化各种成员变量，初始化完成以后就需要将 iio_dev 注册到内核中，需要用到 iio_device_register 函数，函数原型如下：

```
int iio_device_register(struct iio_dev *indio_dev)
```

函数参数和返回值含义如下：

indio_dev：需要注册的 iio_dev。

返回值：0，成功；其他值，失败。

如果要注销 iio_dev 使用 iio_device_unregister 函数，函数原型如下：

```
void iio_device_unregister(struct iio_dev *indio_dev)
```

函数参数和返回值含义如下：

indio_dev：需要注销的 iio_dev。

返回值：0，成功；其他值，失败。

56.1.2 iio_info

iio_dev 有个成员变量：info，为 iio_info 结构体指针变量，这个是我们在编写 IIO 驱动的时候需要着重去实现的，因为用户空间对设备的具体操作最终都会反映到 iio_info 里面。iio_info 结构体定义在 include/linux/iio/iio.h 中，结构体定义如下(有省略)：

示例代码 56.1.2.1 iio_info 结构体

```
393 struct iio_info {
394     const struct attribute_group    *event_attrs;
395     const struct attribute_group    *attrs;
396
397     int (*read_raw)(struct iio_dev *indio_dev,
398             struct iio_chan_spec const *chan,
399             int *val,
400             int *val2,
401             long mask);
......
416
417     int (*write_raw)(struct iio_dev *indio_dev,
418              struct iio_chan_spec const *chan,
419              int val,
420              int val2,
421              long mask);
422
423     int (*write_raw_get_fmt)(struct iio_dev *indio_dev,
424              struct iio_chan_spec const *chan,
425              long mask);
......
462 };
```

第 395 行，attrs 是通用的设备属性。

第 397 和 417 行，分别为 read_raw 和 write_raw 函数，这两个函数就是最终读写设备内部数据的操作函数，需要程序编写人员去实现的。比如应用读取一个陀螺仪传感器的原始数据，那么最终完成工作的就是 read_raw 函数，我们需要在 read_raw 函数里面实现对陀螺仪芯片的

读取操作。同理，write_raw 是应用程序向陀螺仪芯片写数据，一般用于配置芯片，比如量程、数据速率等。这两个函数的参数都是一样的，我们依次来看一下：

indio_dev：需要读写的 IIO 设备。

chan：需要读取的通道。

val，val2：对于 read_raw 函数来说 val 和 val2 这两个就是应用程序从内核空间读取到数据，一般就是传感器指定通道值，或者传感器的量程、分辨率等。对于 write_raw 来说就是应用程序向设备写入的数据。val 和 val2 共同组成具体值，val 是整数部分，val2 是小数部分。但是 val2 也是对具体的小数部分扩大 N 倍后的整数值，因为不能直接从内核向应用程序返回一个小数值。比如现在有个值为 1.00236，那么 val 就是 1，vla2 理论上来讲是 0.00236，但是我们需要对 0.00236 扩大 N 倍，使其变为整数，这里我们扩大 1000000 倍，那么 val2 就是 2360。因此 val=1，val2=2360。扩大的倍数我们不能随便设置，而是要使用 Linux 定义的倍数，Linux 内核里面定义的数据扩大倍数，或者说数据组合形式如表 56.1.2.1 所示：

组合宏	描述
IIO_VAL_INT	整数值，没有小数。比如 5000，那么就是 val=5000，不需要设置 val2
IIO_VAL_INT_PLUS_MICRO	小数部分扩大 1000000 倍，比如 1.00236，此时 val=1，val2=2360。
IIO_VAL_INT_PLUS_NANO	小数部分扩大 1000000000 倍，同样是 1.00236，此时 val=1，val2=2360000。
IIO_VAL_INT_PLUS_MICRO_DB	dB 数据，和 IIO_VAL_INT_PLUS_MICRO 数据形式一样，只是在后面添加 db。
IIO_VAL_INT_MULTIPLE	多个整数值，比如一次要传回 6 个整数值，那么 val 和 val2 就不够用了。此宏主要用于 iio_info 的 read_raw_multi 函数。
IIO_VAL_FRACTIONAL	分数值，也就是 val/val2。比如 val=1，val2=4，那么实际值就是 1/4。
IIO_VAL_FRACTIONAL_LOG2	值为 val>>val2，也就是 val 右移 val2 位。比如 val=25600，val2=4，那么真正的值就是 25600 右移 4 位，25600>>4=1600.

表 56.1.2.1 数据组合表

mask：掩码，用于指定我们读取的是什么数据，比如 ICM20608 这样的传感器，他既有原始的测量数据，比如 X,Y,Z 轴的陀螺仪、加速度计等，也有测量范围值，或者分辨率。比如加速度计测量范围设置为±16g，那么分辨率就是 32/65536≈0.000488，我们只有读出原始值以及对应的分辨率(量程)，才能计算出真实的重力加速度。此时就有两种数据值：传感器原始值、分辨率。Linux 内核使用 IIO_CHAN_INFO_RAW 和 IIO_CHAN_INFO_SCALE 这两个宏来表示原始值以及分辨率，这两个宏就是掩码。至于每个通道可以采用哪几种掩码，这个在我们初始化通道的时候需要驱动编写人员设置好。掩码有很多种，稍后讲解 IIO 通道的时候详细讲解！

第 423 行的 write_raw_get_fmt 用于设置用户空间向内核空间写入的数据格式，write_raw_get_fmt 函数决定了 wtite_raw 函数中 val 和 val2 的意义，也就是表 56.1.2.1 中的组合形式。比如我们需要在应用程序中设置 ICM20608 加速度计的量程为±8g，那么分辨率就是 16/65536≈0.000244，我们在 write_raw_get_fmt 函数里面设置加速度计的数据格式为 IIO_VAL_INT_PLUS_MICRO，那么我们在应用程序里面向指定的文件写入 0.000244 以后，最

终传递给内核驱动的就是 0.000244*1000000=244。也就是 write_raw 函数的 val 参数为 0，val2 参数为 244。

56.1.3 iio_chan_spec

IIO 的核心就是通道，一个传感器可能有多路数据，比如一个 ADC 芯片支持 8 路采集，那么这个 ADC 就有 8 个通道。我们本章实验用到的 ICM20608，这是一个六轴传感器，可以输出三轴陀螺仪(X、Y、Z)、三轴加速度计(X、Y、Z)和一路温度，也就是一共有 7 路数据，因此就有 7 个通道。注意，三轴陀螺仪或加速度计的 X、Y、Z 这三个轴，每个轴都算一个通道。

Linux 内核使用 iio_chan_spec 结构体来描述通道，定义在 include/linux/iio/iio.h 文件中，内容如下：

示例代码 56.1.3.1 iio_chan_spec 结构体

```
236 struct iio_chan_spec {
237     enum iio_chan_type   type;
238     int                  channel;
239     int                  channel2;
240     unsigned long        address;
241     int                  scan_index;
242     struct {
243         char    sign;
244         u8      realbits;
245         u8      storagebits;
246         u8      shift;
247         u8      repeat;
248         enum    iio_endian endianness;
249     } scan_type;
250     long           info_mask_separate;
251     long           info_mask_separate_available;
252     long           info_mask_shared_by_type;
253     long           info_mask_shared_by_type_available;
254     long           info_mask_shared_by_dir;
255     long           info_mask_shared_by_dir_available;
256     long           info_mask_shared_by_all;
257     long           info_mask_shared_by_all_available;
258     const struct  iio_event_spec *event_spec;
259     unsigned int  num_event_specs;
260     const struct  iio_chan_spec_ext_info *ext_info;
261     const char    *extend_name;
262     const char    *datasheet_name;
263     unsigned      modified:1;
264     unsigned      indexed:1;
265     unsigned      output:1;
266     unsigned      differential:1;
267 };
```

来看一下 iio_chan_spec 结构体中一些比较重要的成员变量：

第 237 行，type 为通道类型， iio_chan_type 是一个枚举类型，列举出了可以选择的通道类型，定义在 include/uapi/linux/iio/types.h 文件里面，内容如下：

示例代码 56.1.3.2 iio_chan_type

```
14 enum iio_chan_type {
15     IIO_VOLTAGE,                 /* 电压类型                          */
16     IIO_CURRENT,                 /* 电流类型                          */
17     IIO_POWER,                   /* 功率类型                          */
18     IIO_ACCEL,                   /* 加速度类型                        */
19     IIO_ANGL_VEL,                /* 角度类型(陀螺仪)                  */
20     IIO_MAGN,                    /* 电磁类型(磁力计)                  */
21     IIO_LIGHT,                   /* 灯光类型                          */
22     IIO_INTENSITY,               /* 强度类型(光强传感器)              */
23     IIO_PROXIMITY,               /* 接近类型(接近传感器)              */
24     IIO_TEMP,                    /* 温度类型                          */
25     IIO_INCLI,                   /* 倾角类型(倾角测量传感器)          */
26     IIO_ROT,                     /* 旋转角度类型                      */
27     IIO_ANGL,                    /* 转动角度类型(电机旋转角度测量传感器) */
28     IIO_TIMESTAMP,               /* 时间戳类型                        */
29     IIO_CAPACITANCE,             /* 电容类型                          */
30     IIO_ALTVOLTAGE,              /* 频率类型                          */
31     IIO_CCT,                     /* 笔者暂时未知的类型                */
32     IIO_PRESSURE,                /* 压力类型                          */
33     IIO_HUMIDITYRELATIVE,        /* 湿度类型                          */
34     IIO_ACTIVITY,                /* 活动类型(计步传感器)              */
35     IIO_STEPS,                   /* 步数类型                          */
36     IIO_ENERGY,                  /* 能量类型(卡路里)                  */
37     IIO_DISTANCE,                /* 距离类型                          */
38     IIO_VELOCITY,                /* 速度类型                          */
39     IIO_CONCENTRATION,           /* 浓度类型                          */
40     IIO_RESISTANCE,              /* 电阻类型                          */
41     IIO_PH,                      /* PH 类型                           */
42     IIO_UVINDEX,                 /* 紫外线类型                        */
43     IIO_ELECTRICALCONDUCTIVITY,  /* 电导率类型                        */
44     IIO_COUNT,                   /* 计数类型                          */
45     IIO_INDEX,                   /* 笔者暂时未知的类型                */
46     IIO_GRAVITY,                 /* 重力类型                          */
47     IIO_POSITIONRELATIVE,        /* 相对位置类型                      */
48     IIO_PHASE,                   /* 相位类型                          */
49     IIO_MASSCONCENTRATION,       /* 颗粒物浓度类型                    */
50 };
```

从示例代码 56.1.3.2 可以看出，目前 Linux 内核支持的传感器类型非常丰富，而且支持类型也会不断的增加。如果是 ADC，那就是 IIO_VOLTAGE 类型。如果是 ICM20608 这样的多轴

传感器，那么就是复合类型了，陀螺仪部分是 IIO_ANGL_VEL 类型，加速度计部分是 IIO_ACCEL 类型，温度部分就是 IIO_TEMP。

继续来看示例代码 56.1.3.1 中的 iio_chan_spec 结构体，第 238 行，当成员变量 indexed 为 1 时候，channel 为通道索引。

第 239 行，当成员变量 modified 为 1 的时候，channel2 为通道修饰符。Linux 内核给出了可用的通道修饰符，定义在 include/uapi/linux/iio/types.h 文件里面，内容如下(有省略)

示例代码 56.1.3.3 iio_modifier

```
52 enum iio_modifier {
53     IIO_NO_MOD,
54     IIO_MOD_X,              /* X 轴     */
55     IIO_MOD_Y,              /* Y 轴     */
56     IIO_MOD_Z,              /* Z 轴     */
......
94     IIO_MOD_PM10,           /* PM10     */
95     IIO_MOD_ETHANOL,        /* 乙醇     */
96     IIO_MOD_H2,             /* H2       */
97 };
```

比如 ICM20608 的加速度计部分，类型设置为 IIO_ACCEL，X、Y、Z 这三个轴就用 channel2 的通道修饰符来区分。IIO_MOD_X、IIO_MOD_Y、IIO_MOD_Z 就分别对应 X、Y、Z 这三个轴。通道修饰符主要是影响 sysfs 下的通道文件名字，后面我们会讲解 sysfs 下通道文件名字组成形式。

继续回到示例代码 56.1.3.1，第 240 行的 address 用户可以自定义，但是一般会设置为此通道对应的芯片数据寄存器地址。比如 ICM20608 的加速度计 X 轴这个通道，它的数据首地址就是 0X3B。address 也可以用作其他功能，自行选择，也可以不使用 address，一切以实际情况为准。

第 241 行，当使用触发缓冲区的时候，scan_index 是扫描索引。

第 242~249，scan_type 是一个结构体，描述了扫描数据在缓冲区中的存储格式。我们依次来看一下 scan_type 各个成员变量的涵义：

scan_type.sign：如果为‘u’表示数据为无符号类型，为‘s’的话为有符号类型。

scan_type.realbits：数据真实的有效位数，比如很多传感器说的 10 位 ADC，其真实有效数据就是 10 位。

scan_type.storagebits：存储位数，有效位数+填充位。比如有些传感器 ADC 是 12 位的，那么我们存储的话肯定要用到 2 个字节，也就是 16 位，这 16 位就是存储位数。

scan_type.shift:右移位数，也就是存储位数和有效位数不一致的时候，需要右移的位数，这个参数不总是需要，一切以实际芯片的数据手册位数。

scan_type.repeat：实际或存储位的重复数量。

scan_type.endianness:数据的大小端模式，可设置为 IIO_CPU、IIO_BE(大端)或 IIO_LE(小端)。

第 250 行，info_mask_separate 标记某些属性专属于此通道，include/linux/iio/types.h 文件中的 iio_chan_info_enum 枚举类型描述了可选的属性值，如下所示：

示例代码 56.1.3.4 iio_chan_info_enum

```
34 enum iio_chan_info_enum {
35     IIO_CHAN_INFO_RAW = 0,
```

```
36      IIO_CHAN_INFO_PROCESSED,
37      IIO_CHAN_INFO_SCALE,
38      IIO_CHAN_INFO_OFFSET,
......
57      IIO_CHAN_INFO_DEBOUNCE_TIME,
58      IIO_CHAN_INFO_CALIBEMISSIVITY,
59      IIO_CHAN_INFO_OVERSAMPLING_RATIO,
60  };
```

比如 ICM20608 加速度计的 X、Y、Z 这三个轴，在 sysfs 下这三个轴肯定是对应三个不同的文件，我们通过读取这三个文件就能得到每个轴的原始数据。IIO_CHAN_INFO_RAW 这个属性表示原始数据，当我们在配置 X、Y、Z 这三个通道的时候，在 info_mask_separate 中使能 IIO_CHAN_INFO_RAW 这个属性，那么就表示在 sysfs 下生成三个不同的文件分别对应 X、Y、Z 轴，这三个轴的 IIO_CHAN_INFO_RAW 属性是相互独立的。

第 251 行，info_mask_shared_by_type 标记导出的信息由相同类型的通道共享。也就是 iio_chan_spec.type 成员变量相同的通道。比如 ICM20608 加速度计的 X、Y、Z 轴他们的 type 都是 IIO_ACCEL，也就是类型相同。而这三个轴的分辨率(量程)是一样的，那么在配置这三个通道的时候就可以在 info_mask_shared_by_type 中使能 IIO_CHAN_INFO_SCALE 这个属性，表示这三个通道的分辨率是共用的，这样在 sysfs 下就会只生成一个描述分辨率的文件，这三个通道都可以使用这一个分辨率文件。

第 254 行，info_mask_shared_by_dir 标记某些导出的信息由相同方向的通道共享。

第 256 行，info_mask_shared_by_all 表设计某些信息所有的通道共享，无论这些通道的类型、方向如何，全部共享。

第 263 行，modified 为 1 的时候，channel2 为通道修饰符。

第 264 行，indexed 为 1 的时候，channel 为通道索引。

第 265 行，output 表示为输出通道。

第 266 行，differential 表示为差分通道。

56.2 IIO 驱动框架创建

前面我们已经对 IIO 设备、IIO 通道进行了详细的讲解，本节我们就来学习如何搭建 IIO 驱动框架。在上一小节分析 IIO 子系统的时候大家应该看出了，IIO 框架主要用于 ADC 类的传感器，比如陀螺仪、加速度计、磁力计、光强度计等，这些传感器基本都是 IIC 或者 SPI 接口的。因此 IIO 驱动的基础框架就是 IIC 或者 SPI，我们可以在 IIC 或 SPI 驱动里面在加上上一章讲解的 regmap。当然了，有些 SOC 内部的 ADC 也会使用 IIO 框架，那么这个时候驱动的基础框架就是 platfrom。

56.2.1 基础驱动框架建立

我们以 SPI 接口为例，首先是 SPI 驱动框架，如下所示：

示例代码 56.2.1.1 SPI 驱动框架

```
1  /*
2   * @description     : spi 驱动的 probe 函数，当驱动与
3   *                    设备匹配以后此函数就会执行
4   * @param - spi     : spi 设备
5   * @return          : 0,成功；其他值，失败
```

```
6    */
7   static int xxx_probe(struct spi_device *spi)
8   {
9       return 0;
10  }
11
12  /*
13   * @description     : spi 驱动的 remove 函数，移除 spi 驱动的时候此函数会执行
14   * @param - spi     : spi 设备
15   * @return          : 0，成功;其他负值,失败
16   */
17  static int xxx_remove(struct spi_device *spi)
18  {
19      return 0;
20  }
21
22  /* 传统匹配方式 ID 列表 */
23  static const struct spi_device_id xxx_id[] = {
24      {"alientek,xxx", 0},
25      {}
26  };
27
28  /* 设备树匹配列表 */
29  static const struct of_device_id xxx_of_match[] = {
30      { .compatible = "alientek,xxx" },
31      { /* Sentinel */ }
32  };
33
34  /* SPI 驱动结构体 */
35  static struct spi_driver xxx_driver = {
36      .probe = xxx_probe,
37      .remove = xxx_remove,
38      .driver = {
39          .owner = THIS_MODULE,
40          .name = "xxx",
41          .of_match_table = xxx_of_match,
42      },
43      .id_table = xxx_id,
44  };
45
46  /*
47   * @description     : 驱动入口函数
48   * @param           : 无
```

```
49  * @return          : 无
50  */
51 static int __init xxx_init(void)
52 {
53     return spi_register_driver(&xxx_driver);
54 }
55
56 /*
57  * @description    : 驱动出口函数
58  * @param          : 无
59  * @return         : 无
60  */
61 static void __exit xxx_exit(void)
62 {
63     spi_unregister_driver(&xxx_driver);
64 }
65
66 module_init(xxx_init);
67 module_exit(xxx_exit);
68 MODULE_LICENSE("GPL");
69 MODULE_AUTHOR("ALIENTEK");
```

示例代码 56.2.1.1 就是标准的 SPI 驱动框架，如果所使用的传感器是 IIC 接口的，那么就是 IIC 驱动框架，没啥好说的。

56.2.2 IIO 设备申请与初始化

IIO 设备的申请、初始化以及注册在 probe 函数中完成，在注销驱动的时候还需要在 remove 函数中注销掉 IIO 设备、释放掉申请的一些内存。添加完 IIO 框架以后的 probe 和 remove 函数如下所示：

示例代码 56.2.2.1 添加 IIO 框架

```
1  /* 自定义设备结构体 */
2  struct xxx_dev {
3      struct spi_device *spi;              /* spi 设备 */
4      struct regmap *regmap;               /* regmap  */
5      struct regmap_config regmap_config;
6      struct mutex lock;
7  };
8
9  /*
10  * 通道数组
11  */
12 static const struct iio_chan_spec xxx_channels[] = {
13
14 };
```

```
15
16  /*
17    * @description      : 读函数，当读取 sysfs 中的文件的时候最终此函数会执行，
18    *                   : 此函数里面会从传感器里面读取各种数据，然后上传给应用。
19    * @param - indio_dev  : IIO 设备
20    * @param - chan     : 通道
21    * @param - val      : 读取的值，如果是小数值的话，val 是整数部分。
22    * @param - val2     : 读取的值，如果是小数值的话，val2 是小数部分。
23    * @param - mask     : 掩码。
24    * @return           : 0，成功；其他值，错误
25    */
26  static int xxx_read_raw(struct iio_dev *indio_dev,
27                struct iio_chan_spec const *chan,
28                int *val, int *val2, long mask)
29  {
30      return 0;
31  }
32
33  /*
34    * @description          : 写函数，当向 sysfs 中的文件写数据的时候最终此函数
35    *                       : 会执行，一般在此函数里面设置传感器，比如量程等。
36    * @param - indio_dev    : IIO 设备
37    * @param - chan         : 通道
38    * @param - val          : 应用程序写入值，如果是小数的话，val 是整数部分。
39    * @param - val2         : 应用程序写入值，如果是小数的话，val2 是小数部分。
40    * @return               : 0，成功；其他值，错误
41    */
42  static int xxx_write_raw(struct iio_dev *indio_dev,
43                struct iio_chan_spec const *chan,
44                int val, int val2, long mask)
45  {
46      return 0;
47  }
48
49  /*
50    * @description      : 用户空间写数据格式，比如我们在用户空间操作 sysfs 来设
51    *                   : 置传感器的分辨率，如果分辨率带小数，那么这个小数传递到
52    *                   : 内核空间应该扩大多少倍，此函数就是用来设置这个的。
53    * @param - indio_dev  : iio_dev
54    * @param - chan     : 通道
55    * @param - mask     : 掩码
56    * @return           : 0，成功；其他值，错误
57    */
```

```
58  static int xxx_write_raw_get_fmt(struct iio_dev *indio_dev,
59                  struct iio_chan_spec const *chan, long mask)
60  {
61      return 0;
62  }
63
64  /*
65   * iio_info 结构体变量
66   */
67  static const struct iio_info xxx_info = {
68      .read_raw       = xxx_read_raw,
69      .write_raw      = xxx_write_raw,
70      .write_raw_get_fmt = &xxx_write_raw_get_fmt,
71  };
72
73  /*
74    * @description  : spi 驱动的 probe 函数，当驱动与
75    *                 设备匹配以后此函数就会执行
76    * @param - spi  : spi 设备
77    *
78    */
79  static int xxx_probe(struct spi_device *spi)
80  {
81      int ret;
82      struct xxx_dev *data;
83      struct iio_dev *indio_dev;
84
85      /*  1、申请 iio_dev 内存 */
86      indio_dev = devm_iio_device_alloc(&spi->dev, sizeof(*data));
87      if (!indio_dev)
88          return -ENOMEM;
89
90      /* 2、获取 xxx_dev 结构体地址 */
91      data = iio_priv(indio_dev);
92      data->spi = spi;
93      spi_set_drvdata(spi, indio_dev);
94      mutex_init(&data->lock);
95
96      /* 3、初始化 iio_dev 成员变量 */
97      indio_dev->dev.parent = &spi->dev;
98      indio_dev->info = &xxx_info;
99      indio_dev->name = "xxx";
100     indio_dev->modes = INDIO_DIRECT_MODE;   /* 直接模式 /
```

```
101     indio_dev->channels = xxx_channels;
102     indio_dev->num_channels = ARRAY_SIZE(xxx_channels);
103
104     iio_device_register(indio_dev);
105
106     /* 4、regmap 相关设置 */
107
108     /* 5、SPI 相关设置*/
109
110     /* 6、芯片初始化 */
111
112     return 0;
113
114 }
115
116 /*
117  * @description   : spi 驱动的 remove 函数，移除 spi 驱动的时候此函数会执行
118  * @param - spi   : spi 设备
119  * @return        : 0，成功;其他负值,失败
120  */
121 static int xxx_remove(struct spi_device *spi)
122 {
123     struct iio_dev *indio_dev = spi_get_drvdata(spi);
124     struct xxx_dev *data;
125
126     data = iio_priv(indio_dev); ;
127
128     /* 1、其他资源的注销以及释放 */
129
130     /* 2、注销 IIO */
131     iio_device_unregister(indio_dev);
132
133     return 0;
134 }
```

第 2~7 行，用户自定义的设备结构体。

第 12 行，IIO 通道数组。

第 16~71 行， 这部分为 iio_info，当应用程序读取相应的驱动文件的时候，xxx_read_raw 函数就会执行，我们在此函数中会读取传感器数据，然后返回给应用层。当应用层向相应的驱动写数据的时候，xxx_write_raw 函数就会执行。因此 xxx_read_raw 和 xxx_write_raw 这两个函数是非常重要的！需要我们根据具体的传感器来编写，这两个函数是编写 IIO 驱动的核心。

第 79~114 行，xxx_probe 函数，此函数的核心就是分配并初始化 iio_dev，最后向内核注册 iio_dev。第 86 行调用 devm_iio_device_alloc 函数分配 iio_dev 内存，这里连用户自定义的设备结构体变量内存一起申请了。第 91 行调用 iio_priv 函数从 iio_dev 中提取出私有数据，这个私

有数据就是设备结构体变量。第 97~102 行初始化 iio_dev，重点是第 98 行设置 iio_dev 的 info 成员变量。第 101 行设置 iio_dev 的通道。初始化完成以后就要调用 iio_device_register 函数向内核注册 iio_dev。整个过程就是：申请 iio_dev、初始化、注册，和我们前面讲解的其他驱动框架步骤一样。

第 121~134 行，xxx_remove 函数里面需要做的就是释放 xxx_probe 函数申请到的 IIO 相关资源，比如第 131 行，使用 iio_device_unregister 注销掉前面注册的 iio_dev。由于前面我们使用 devm_iio_device_alloc 函数申请的 iio_dev，因此不需要在 remove 函数中手动释放 iio_dev。

IIO 框架示例就讲解到这里，剩下的就是根据所使用的具体传感器，在 IIO 驱动框架里面添加相关的处理，接下来我们就以正点原子 STM32MP157 开发板上的 ICM20608 为例，进行 IIO 驱动实战！

56.3 实验程序编写

接下来我们就直接使用 IIO 驱动框架编写 ICM20608 驱动，ICM20608 驱动核心就是 SPI，上一章我们也讲解了如何在 SPI 总线上使用 regmap。因此本章 ICM20608 驱动底层没啥好讲解的，重点是如何套上 IIO 驱动框架，因此关于 ICM20608 芯片内部寄存器、SPI 驱动、regmap 等不会做讲解，有什么不懂的可以看前面相应的实验章节。

本实验对应的例程路径为：开发板光盘→01、程序源码→02、Linux 驱动例程→31_iio→spi

56.3.1 ICM20608 的 IIO 驱动框架搭建

1、驱动框架搭建

设备树不需要做任何修改！

首先我们基于上一章实验，搭建出 ICM20608 的 IIO 驱动框架，IIO 框架部分我们已经在示例代码 56.2.2.1 中进行了详细的讲解。新建名为 icm20608.c 的驱动文件，搭建好的 ICM20608 IIO 驱动框架内容如下所示：

示例代码 56.3.1.1 ICM20608 驱动框架

```
/***************************************************************
Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
文件名   : icm20608.c
作者     : 正点原子 Linux 团队
版本     : V1.0
描述     : ICM20608 SPI 驱动程序
其他     : 无
论坛     : www.openedv.com
日志     : V1.2 2021/08/13
          使用 IIO 框架，参考 bma220_spi.c
***************************************************************/
1   #include <linux/spi/spi.h>
2   #include <linux/kernel.h>
3   #include <linux/module.h>
4   #include <linux/init.h>
5   #include <linux/delay.h>
6   #include <linux/ide.h>
```

```
7   #include <linux/errno.h>
8   #include <linux/platform_device.h>
9   #include "icm20608reg.h"
10  #include <linux/gpio.h>
11  #include <linux/device.h>
12  #include <asm/uaccess.h>
13  #include <linux/cdev.h>
14  #include <linux/regmap.h>
15  #include <linux/iio/iio.h>
16  #include <linux/iio/sysfs.h>
17  #include <linux/iio/trigger_consumer.h>
18  #include <linux/iio/buffer.h>
19  #include <linux/iio/triggered_buffer.h>
20
21  #define ICM20608_NAME   "icm20608"
22
23  #define ICM20608_CHAN(_type, _channel2, _index)  \
24      {                                            \
25          .type = _type,                           \
26          .modified = 1,                           \
27          .channel2 = _channel2,                   \
28          .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE),    \
29          .info_mask_separate = BIT(IIO_CHAN_INFO_RAW) |           \
30                      BIT(IIO_CHAN_INFO_CALIBBIAS),  \
31          .scan_index = _index,                    \
32          .scan_type = {                           \
33                  .sign = 's',                     \
34                  .realbits = 16,                  \
35                  .storagebits = 16,               \
36                  .shift = 0,                      \
37                  .endianness = IIO_BE,            \
38                   },                              \
39      }
40
41  /*
42   * ICM20608 的扫描元素，3 轴加速度计、
43   * 3 轴陀螺仪、1 路温度传感器，1 路时间戳
44   */
45  enum inv_icm20608_scan {
46      INV_ICM20608_SCAN_ACCL_X,
47      INV_ICM20608_SCAN_ACCL_Y,
48      INV_ICM20608_SCAN_ACCL_Z,
49      INV_ICM20608_SCAN_TEMP,
```

```
50      INV_ICM20608_SCAN_GYRO_X,
51      INV_ICM20608_SCAN_GYRO_Y,
52      INV_ICM20608_SCAN_GYRO_Z,
53      INV_ICM20608_SCAN_TIMESTAMP,
54  };
55
56  struct icm20608_dev {
57      struct spi_device *spi;             /* spi 设备 */
58      struct regmap *regmap;              /* regmap */
59      struct regmap_config regmap_config;
60      struct mutex lock;
61  };
62
63  static const struct iio_chan_spec icm20608_channels[] = {
64      /* 温度通道 */
65      {
66          .type = IIO_TEMP,
67          .info_mask_separate = BIT(IIO_CHAN_INFO_RAW)
68                  | BIT(IIO_CHAN_INFO_OFFSET)
69                  | BIT(IIO_CHAN_INFO_SCALE),
70          .scan_index = INV_ICM20608_SCAN_TEMP,
71          .scan_type = {
72                  .sign = 's',
73                  .realbits = 16,
74                  .storagebits = 16,
75                  .shift = 0,
76                  .endianness = IIO_BE,
77                   },
78      },
79
80      ICM20608_CHAN(IIO_ANGL_VEL, IIO_MOD_X,INV_ICM20608_SCAN_GYRO_X),
81      ICM20608_CHAN(IIO_ANGL_VEL, IIO_MOD_Y,INV_ICM20608_SCAN_GYRO_Y),
82      ICM20608_CHAN(IIO_ANGL_VEL, IIO_MOD_Z,INV_ICM20608_SCAN_GYRO_Z),
83
84      ICM20608_CHAN(IIO_ACCEL, IIO_MOD_Y, INV_ICM20608_SCAN_ACCL_Y),
85      ICM20608_CHAN(IIO_ACCEL, IIO_MOD_X, INV_ICM20608_SCAN_ACCL_X),
86      ICM20608_CHAN(IIO_ACCEL, IIO_MOD_Z, INV_ICM20608_SCAN_ACCL_Z),
87  };
88
89  /*
90   * @description   : 读取 icm20608 指定寄存器值，读取一个寄存器
91   * @param - dev   : icm20608 设备
92   * @param - reg   : 要读取的寄存器
```

```
93   * @return         : 读取到的寄存器值
94   */
95  static unsigned char icm20608_read_onereg(struct icm20608_dev *dev,
                                              u8 reg)
96  {
97      u8 ret;
98      unsigned int data;
99
100     ret = regmap_read(dev->regmap, reg, &data);
101     return (u8)data;
102 }
103
104 /*
105  * @description   : 向 icm20608 指定寄存器写入指定的值，写一个寄存器
106  * @param - dev   : icm20608 设备
107  * @param - reg   : 要写的寄存器
108  * @param - data  : 要写入的值
109  * @return        : 无
110  */
111
112 static void icm20608_write_onereg(struct icm20608_dev *dev, u8 reg,
                                      u8 value)
113 {
114     regmap_write(dev->regmap,  reg, value);
115 }
116
117 /*
118  * ICM20608 内部寄存器初始化函数
119  * @param - spi   : 要操作的设备
120  * @return        : 无
121  */
122 void icm20608_reginit(struct icm20608_dev *dev)
123 {
124     u8 value = 0;
125
126     icm20608_write_onereg(dev, ICM20_PWR_MGMT_1, 0x80);
127     mdelay(50);
128     icm20608_write_onereg(dev, ICM20_PWR_MGMT_1, 0x01);
129     mdelay(50);
130
131     value = icm20608_read_onereg(dev, ICM20_WHO_AM_I);
132     printk("ICM20608 ID = %#X\r\n", value);
133
```

```
134     icm20608_write_onereg(dev, ICM20_SMPLRT_DIV, 0x00);
135     icm20608_write_onereg(dev, ICM20_GYRO_CONFIG, 0x18);
136     icm20608_write_onereg(dev, ICM20_ACCEL_CONFIG, 0x18);
137     icm20608_write_onereg(dev, ICM20_CONFIG, 0x04);
138     icm20608_write_onereg(dev, ICM20_ACCEL_CONFIG2, 0x04);
139     icm20608_write_onereg(dev, ICM20_PWR_MGMT_2, 0x00);
140     icm20608_write_onereg(dev, ICM20_LP_MODE_CFG, 0x00);
141     icm20608_write_onereg(dev, ICM20_FIFO_EN, 0x00);
142 }
143
144 /*
145   * @description      : 读函数，当读取 sysfs 中的文件的时候最终此函数会执行，
146   *                   : 此函数里面会从传感器里面读取各种数据，然后上传给应用。
147   * @param - indio_dev   : iio_dev
148   * @param - chan        : 通道
149   * @param - val         : 读取的值，如果是小数值的话，val 是整数部分。
150   * @param - val2        : 读取的值，如果是小数值的话，val2 是小数部分。
151   * @return              : 0，成功；其他值，错误
152   */
153 static int icm20608_read_raw(struct iio_dev *indio_dev,
154             struct iio_chan_spec const *chan,
155             int *val, int *val2, long mask)
156 {
157     printk("icm20608_read_raw\r\n");
158     return 0;
159 }
160
161 /*
162   * @description      : 写函数，当向 sysfs 中的文件写数据的时候最终此函数会
163   *                   : 执行，一般在此函数里面设置传感器，比如量程等。
164   * @param - indio_dev  : iio_dev
165   * @param - chan     : 通道
166   * @param - val      : 应用程序写入的值，如果是小数值的话，val 是整数部分。
167   * @param - val2     : 应用程序写入的值，如果是小数值的话，val2 是小数部分。
168   * @return           : 0，成功；其他值，错误
169   */
170 static int icm20608_write_raw(struct iio_dev *indio_dev,
171             struct iio_chan_spec const *chan,
172             int val, int val2, long mask)
173 {
174     printk("icm20608_write_raw\r\n");
175     return 0;
176 }
```

```
177
178 /*
179  * @description  : 用户空间写数据格式，比如我们在用户空间操作 sysfs 来
180  *                : 设置传感器的分辨率，如果分辨率带小数，那么这个小数传递
181  *                : 到内核空间应该扩大多少倍，此函数就是用来设置这个的。
182  * @param - indio_dev   : iio_dev
183  * @param - chan        : 通道
184  * @param - mask        : 掩码
185  * @return              : 0，成功；其他值，错误
186  */
187 static int icm20608_write_raw_get_fmt(struct iio_dev *indio_dev,
188                 struct iio_chan_spec const *chan, long mask)
189 {
190     printk("icm20608_write_raw_get_fmt\r\n");
191     return 0;
192 }
193
194 /*
195  * iio_info 结构体变量
196  */
197 static const struct iio_info icm20608_info = {
198     .read_raw       = icm20608_read_raw,
199     .write_raw      = icm20608_write_raw,
200     .write_raw_get_fmt = &icm20608_write_raw_get_fmt,
201 };
202
203 /*
204  * @description  : spi 驱动的 probe 函数，当驱动与
205  *                 设备匹配以后此函数就会执行
206  * @param - spi  : spi 设备
207  *
208  */
209 static int icm20608_probe(struct spi_device *spi)
210 {
211     int ret;
212     struct icm20608_dev *data;
213     struct iio_dev *indio_dev;
214
215     /*  1、申请 iio_dev 内存 */
216     indio_dev = devm_iio_device_alloc(&spi->dev, sizeof(*data));
217     if (!indio_dev)
218         return -ENOMEM;
219
```

```
220     /* 2、获取 icm20608_dev 结构体地址 */
221     data = iio_priv(indio_dev);
222     data->spi = spi;
223     spi_set_drvdata(spi, indio_dev);
224     mutex_init(&data->lock);
225 
226     indio_dev->dev.parent = &spi->dev;
227     indio_dev->info = &icm20608_info;
228     indio_dev->name = ICM20608_NAME;
229     indio_dev->modes = INDIO_DIRECT_MODE;   /* 直接模式 */
230     indio_dev->channels = icm20608_channels;
231     indio_dev->num_channels = ARRAY_SIZE(icm20608_channels);
232 
233     ret = iio_device_register(indio_dev);
234     if (ret < 0) {
235         dev_err(&spi->dev, "iio_device_register failed\n");
236         goto err_iio_register;
237     }
238 
239     /* 2、初始化 regmap_config 设置 */
240     data->regmap_config.reg_bits = 8;           /* 寄存器长度 8bit */
241     data->regmap_config.val_bits = 8;           /* 值长度 8bit */
242     data->regmap_config.read_flag_mask = 0x80;
243 
244     /* 3、初始化 SPI 接口的 regmap */
245     data->regmap = regmap_init_spi(spi, &data->regmap_config);
246     if (IS_ERR(data->regmap)) {
247         ret = PTR_ERR(data->regmap);
248         goto err_regmap_init;
249     }
250 
251 
252     /* 5、初始化 spi_device */
253     spi->mode = SPI_MODE_0; /*MODE0, CPOL=0, CPHA=0*/
254     spi_setup(spi);
255 
256     /* 初始化 ICM20608 内部寄存器 */
257     icm20608_reginit(data);
258 
259     return 0;
260 
261 err_regmap_init:
262     iio_device_unregister(indio_dev);
```

```
263 err_iio_register:
264
265     return ret;
266 }
267
268 /*
269  * @description   : spi 驱动的 remove 函数，移除 spi 驱动的时候此函数会执行
270  * @param - spi   : spi 设备
271  * @return        : 0，成功;其他负值,失败
272  */
273 static int icm20608_remove(struct spi_device *spi)
274 {
275     struct iio_dev *indio_dev = spi_get_drvdata(spi);
276     struct icm20608_dev *data;
277
278     data = iio_priv(indio_dev); ;
279
280     /* 1、删除 regmap */
281     regmap_exit(data->regmap);
282     /* 2、注销 IIO */
283     iio_device_unregister(indio_dev);
284     return 0;
285 }
286
287 /* 传统匹配方式 ID 列表 */
288 static const struct spi_device_id icm20608_id[] = {
289     {"alientek,icm20608", 0},
290     {}
291 };
292
293 /* 设备树匹配列表 */
294 static const struct of_device_id icm20608_of_match[] = {
295     { .compatible = "alientek,icm20608" },
296     { /* Sentinel */ }
297 };
298
299 /* SPI 驱动结构体 */
300 static struct spi_driver icm20608_driver = {
301     .probe = icm20608_probe,
302     .remove = icm20608_remove,
303     .driver = {
304             .owner = THIS_MODULE,
305             .name = "icm20608",
```

```
306             .of_match_table = icm20608_of_match,
307         },
308     .id_table = icm20608_id,
309 };
310
311 /*
312  * @description   : 驱动入口函数
313  * @param         : 无
314  * @return        : 无
315  */
316 static int __init icm20608_init(void)
317 {
318     return spi_register_driver(&icm20608_driver);
319 }
320
321 /*
322  * @description   : 驱动出口函数
323  * @param         : 无
324  * @return        : 无
325  */
326 static void __exit icm20608_exit(void)
327 {
328     spi_unregister_driver(&icm20608_driver);
329 }
330
331 module_init(icm20608_init);
332 module_exit(icm20608_exit);
333 MODULE_LICENSE("GPL");
334 MODULE_AUTHOR("ALIENTEK");
335 MODULE_INFO(intree, "Y");
```

示例代码 56.3.1.1 就是在上一章例程的基础上添加 IIO 驱动框架而来的，这里我们重点讲解一下 IIO 部分。

第 23~39 行，通道宏定义，用于陀螺仪和加速度计，第 26 行 modified 成员变量为 1，所以 channel2 就是通道修饰符，用来指定 X、Y、Z 轴。第 28 行设置相同类型的通道 IIO_CHAN_INFO_SCALE 属性相同，“scale”是比例的意思，在这里就是量程(分辨率)，因为 ICM20608 的陀螺仪和加速度计的量程是可以调整的，量程不同分辨率也就不同。陀螺仪或加速度计的三个轴也是一起设置的，因此对于陀螺仪或加速度计而言，X、Y、Z 这三个轴的量程是共享的。第 29 行，设置每个通道的 IIO_CHAN_INFO_RAW 和 IIO_CHAN_INFO_CALIBBIAS 这两个属性都是独立的，IIO_CHAN_INFO_RAW 表示 ICM20608 每个通道的原始值，这个肯定是每个通道独立的。IIO_CHAN_INFO_CALIBBIAS 是 ICM20608 每个通道的校准值，这个是 ICM20608 的特性，不是所有的传感器都有校准值，一切都要以实际所使用的传感器为准。第 32 行，设置扫描数据类型，也就是 ICM20608 原始数据类型，ICM20608 的陀螺仪和加速度计都是

16 位 ADC，因此这里是通用的：为有符号类型、实际位数 16bit，存储位数 16bit，大端模式(ICM20608 数据寄存器为大端模式)。

第 45~54 行，自定义的扫描索引枚举类型 inv_icm20608_scan，包括陀螺仪、加速度计的 6 个通道，温度计的 1 个通道、以及 1 个 ICM20608 时间戳通道。

第 56~61 行，设备结构体，由于采用了 regmap 和 IIO 框架，因此 ICM20608 的设备结构体非常简单。

第 63~87 行，ICM20608 通道，这里定义了 7 个通道，分别是：1 个温度通道，3 个陀螺仪通道(X、Y、Z)，3 个加速度计通道(X、Y、Z)。温度通道有三个属性，IIO_CHAN_INFO_RAW 为温度通道的原始值，IIO_CHAN_INFO_OFFSET 是 ICM20608 温度 offset 值，这个要查阅数据手册。IIO_CHAN_INFO_SCALE 是 ICM20608 的比例，也就是一个单位的原始值为多少℃，这个也要查阅 ICM20608 的数据手册。从这里可以看出，想要得到 ICM20608 的具体温度值，需要三个数据：原始值、offset 值、比例值，也就是应用程序需要能够从 IIO 驱动框架中的到这三个值，一般是应用程序读取相应的文件，所以这里就要有三个独立的文件分别表示原始值、offset 值、比例值，这就是三个属性的来源。剩下的陀螺仪和加速度计通道设置使用宏 ICM20608_CHAN 即可，IIO_MOD_X、IIO_MOD_Y 和 IIO_MOD_Z 分别是 X、Y、Z 这三个轴的修饰符。

第 153~201 行，这部分就是 iio_info，icm20608_read_raw 为读取函数，应用程序读取相应文件的时候此函数执行，icm20608_write_raw 为写函数，应用程序向相应的文件写数据的时候此函数执行。icm20608_write_raw_get_fmt 函数用来设置应用程序向驱动写入的数据格式，icm20608_info 就是具体的 iio_info 变量，初始化 iio_dev 的时候需要用到。

第 209 行，icm20608_probe 函数，一般在此函数里面申请 iio_dev、初始化并注册，初始化 regmap、初始化 ICM20608 等。第 216 行通过 devm_iio_device_alloc 函数申请 iio_dev 以及自定义设备结构体内存，本章节就是 icm20608_dev。第 226~231 行，初始化 iio_dev，第 233 行，调用 iio_device_register 函数想内核注册 iio_dev。

关于 ICM20608 的 IIO 驱动框架就讲解到这里，接下来就是测试此驱动框架。

2、驱动框架测试

我们已经搭建好了 ICM20608 的 IIO 驱动框架，通道也已经设置好了，虽然还不能直接读取到 ICM20608 的原始数据，但是我们可以通过驱动框架来窥探 IIO 在用户空间的存在方式。编译驱动，得到 icm20608.ko 驱动文件。入如下命令加载 icm20608.ko 这个驱动模块。

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe icm20608.ko  //加载驱动模块
```

我们在 icm20608_probe 函数里面设置了打印 ICM20608 ID 值，因此如果驱动加载成功，SPI 工作正常的话就会读取 ICM20608 的 ID 值并打印出来，如图 56.3.1.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe icm20608.ko
ICM20608 ID = 0XAF
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 56.3.1.1 ICM20608 驱动加载成功

IIO 驱动框架提供了 sysfs 接口，因此加载成功以后我们可以在用户空间访问对应的 sysfs 目录项，进入目录“/sys/bus/iio/devices/”目录里面，此目录下都是 IIO 框架设备，如图 56.3.1.2 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio$ cd /sys/bus/iio/devices/
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ ls
iio:device0
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.3.1.2 IIO 设备

从图 56.3.1.2 可以看出，此时只有一个 IIO 设备“iio:device0”，这个就是 ICM20608，如果有多个 IIO 设备的话就需要大家依次进入到对应的设备目录，查看所对应的具体芯片型号。我们进入图 56.3.1.2 中的“iio:device0”目录，此目录下的内容如图 56.3.1.3 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0$ ls
dev                     in_anglvel_scale        in_temp_raw
in_accel_scale          in_anglvel_x_calibbias  in_temp_scale
in_accel_x_calibbias    in_anglvel_x_raw        name
in_accel_x_raw          in_anglvel_y_calibbias  of_node
in_accel_y_calibbias    in_anglvel_y_raw        power
in_accel_y_raw          in_anglvel_z_calibbias  subsystem
in_accel_z_calibbias    in_anglvel_z_raw        uevent
in_accel_z_raw          in_temp_offset
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0$
```

图 56.3.1.3 iio:device0 设备

从图 56.3.1.3 可以看出，iio:device0 对应 spi0.0 上的设备，也就是 ICM20608，此目录下有很多文件，比如 in_accel_scale、in_accel_x_calibias、in_accel_x_raw 等，这些就是我们设置的通道。in_accel_scale 就是加速度计的比例，也就是分辨率(量程)，in_accel_x_calibias 就是加速度计 X 轴的校准值，in_accel_x_raw 就是加速度计的 X 轴原始值。我们在配置通道的时候，设置了类型相同的所有通道共用 SCALE，所以这里只有一个 in_accel_scale，而 X、Y、Z 轴的原始值和校准值每个轴都有一个文件，陀螺仪和温度计同理。

3、通道文件命名方式

我们来看一下图 56.3.1.3 中这些文件名字组成方式，以 in_accel_x_raw 为例，这是加速度计的 X 轴原始值，驱动代码中此通道的配置内容展开以后如下(演示代码)：

示例代码 56.3.1.2 加速度计 X 轴通道配置演示代码

```
1  .type = IIO_ANGL_VEL,
2  .modified = 1,
3  .channel2 = IIO_MOD_X,
4  .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE),
5  .info_mask_separate = BIT(IIO_CHAN_INFO_RAW) |
6            BIT(IIO_CHAN_INFO_CALIBBIAS),
7  .scan_index = INV_ICM20608_SCAN_GYRO_X,
8  .scan_type = {
9       .sign = 's',
10      .realbits = 16,
11      .storagebits = 16,
12      .shift = 0,
13      .endianness = IIO_BE,
14       },
```

第 5 行设置了此通道有 IIO_CHAN_INFO_RAW 和 IIO_CHAN_INFO_CALIBBIAS 这两个专属属性，因此才会有图 56.3.1.2 中的 in_accel_x_raw 和 in_accel_x_calibias 这两个文件。

通道属性的命名，也就是图 56.3.1.2 中文件的命名模式为：[direction]_[type]_[index]_[modifier]_[info_mask]，我们依次来看一下这些命名组织模块：

direction：为属性对应的方向，iio_direction 结构体定义了方向，内容如下：

示例代码 56.3.1.3 iio_direction 结构体

```
48 static const char * const iio_direction[] = {
49     [0] = "in",
50     [1] = "out",
51 };
```

可以看出，就有两个方向：in 和 out。

type：也就是配置通道的时候 type 值，type 对应的字符可以参考 iio_chan_type_name_spec，如下：

示例代码 56.3.1.4 iio_chan_type_name_spec 结构体

```
53 static const char * const iio_chan_type_name_spec[] = {
54     [IIO_VOLTAGE] = "voltage",
55     [IIO_CURRENT] = "current",
56     [IIO_POWER] = "power",
57     [IIO_ACCEL] = "accel",
58     [IIO_ANGL_VEL] = "anglvel",
59     [IIO_MAGN] = "magn",
......
85     [IIO_GRAVITY]  = "gravity",
86     [IIO_POSITIONRELATIVE]  = "positionrelative",
87     [IIO_PHASE] = "phase",
88     [IIO_MASSCONCENTRATION] = "massconcentration",
89 };
```

所以，当通道的 type 设置为 IIO_ACCEL 的时候，对应的名字就是“accel”。

index：索引，如果配置通道的时候设置了 indexed=1，那么就会使用通道的 channel 成员变量来替代此部分命名。比如，有个 ADC 芯片支持 8 个通道，那么就可以使用 channel 来表示对应的通道，最终在用户空间呈现的每个通道文件名的 index 部分就是通道号。

modifier：当通道的 modified 成员变量为 1 的时候，channel2 就是修饰符，修饰符对应的字符串参考结构体 iio_modifier_names，内容如下：

示例代码 56.3.1.5 iio_modifier_names 结构体

```
91  static const char * const iio_modifier_names[] = {
92     [IIO_MOD_X] = "x",
93     [IIO_MOD_Y] = "y",
94     [IIO_MOD_Z] = "z",
......
131    [IIO_MOD_PM4] = "pm4",
132    [IIO_MOD_PM10] = "pm10",
133 };
```

当通道的修饰符设置为 IIO_MOD_X 的时候，对应的名字就是“x”。

info_mask：属性掩码，也就是属性，不同属性对应的字符如下所示：

示例代码 56.3.1.6 iio_chan_info_postfix 结构体

```
136 static const char * const iio_chan_info_postfix[] = {
137     [IIO_CHAN_INFO_RAW] = "raw",
```

```
138     [IIO_CHAN_INFO_PROCESSED] = "input",
139     [IIO_CHAN_INFO_SCALE] = "scale",
140     [IIO_CHAN_INFO_OFFSET] = "offset",
141     [IIO_CHAN_INFO_CALIBSCALE] = "calibscale",
142     [IIO_CHAN_INFO_CALIBBIAS] = "calibbias",
......
161     [IIO_CHAN_INFO_DEBOUNCE_TIME] = "debounce_time",
162     [IIO_CHAN_INFO_CALIBEMISSIVITY] = "calibemissivity",
163     [IIO_CHAN_INFO_OVERSAMPLING_RATIO] = "oversampling_ratio",
164 };
```

可以看出，IIO_CHAN_INFO_RAW 属性对应的就是“raw”，IIO_CHAN_INFO_SCALE 属性对应的是“scale”。

综上所述，in_accel_x_raw 组成形式如图 56.3.1.4 所示：

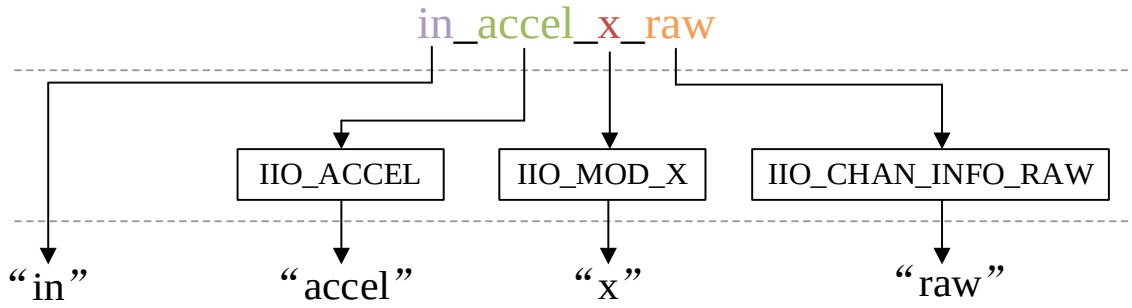


图 56.3.1.4 sysfs 文件命名组成方式

其他文件的命名组成方式类似，大家自行分析即可。

4、文件读测试

我们可以测试一下对图 56.3.1.3 中的这些文件进行读写操作，看看有什么效果，比如 in_accel_x_raw 文件是 ICM20608 的加速度计 X 轴原始值通道，使用 cat 命令查看此文件内容，结果如图 56.3.1.5 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device0/in_accel_x_raw
icm20608_read_raw
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

驱动里面icm206080_read_raw函数执行

图 56.3.1.5 用户空间读取操作

从图 56.3.1.5 可以看出，当用户空间读取相应文件的时候，iio_info 下的 read 函数会被调用，因此关于传感器数据读取的操作都是在此函数中完成的。由于驱动还不完善，大家就先不要测试向指定文件写操作，否则可能会报错，后面再测试写操作。大家也可以读取一下其他文件，比如陀螺仪的 X 轴文件 in_anglvel_x_raw，结果最终都是调用的 icm20608_read_raw 这个函数。

接下来的重点就是完善驱动中的 icm20608_read_raw 函数，实现传感器数据的读取操作。

56.3.2 完善 icm20608_read_raw 函数

应用程序所有的读取操作，最终都会汇总到 iio_info 的 read 函数，这里就是 icm20608_read_raw 函数。由于所有的读取操作都会触发 icm20608_read_raw 函数，比如加速度计、陀螺仪、温度计等，因此我们需要做区分。配置通道的时候设置了 type 值，就可以使用 type

值来区分是陀螺仪、加速度计还是温度计，修改后的 icm20608_read_raw 函数如下：

示例代码 56.3.2.1 修改后的 icm20608_read_raw

```
1  tatic int icm20608_read_raw(struct iio_dev *indio_dev,
2            struct iio_chan_spec const *chan,
3            int *val, int *val2, long mask)
4
5      struct icm20608_dev *dev = iio_priv(indio_dev);
6      int ret = 0;
7      unsigned char regdata = 0;
8      printk("icm20608_read_raw\r\n");
9
10     switch (mask) {
11     case IIO_CHAN_INFO_RAW:  /* 读取加速度计、陀螺仪、温度传感器原始值 */
12         printk("read raw data\r\n");
13         return ret;
14     case IIO_CHAN_INFO_SCALE:
15         switch (chan->type) {
16         case IIO_ANGL_VEL:
17             printk("read gyro sacle\r\n");
18             return ret;
19         case IIO_ACCEL:
20             printk("read accel sacle\r\n");
21             return 0;
22         case IIO_TEMP:
23             printk("read temp sacle\r\n");
24             return ret;
25         default:
26             return -EINVAL;
27     }
28     case IIO_CHAN_INFO_OFFSET:  /* ICM20608 温度传感器 offset 值 */
29         switch (chan->type) {
30         case IIO_TEMP:
31             printk("read temp offset\r\n");
32             return ret;
33         default:
34             return -EINVAL;
35         }
36         return ret;
37     case IIO_CHAN_INFO_CALIBBIAS:   /* ICM20608 加速度计和陀螺仪校准值 */
38         switch (chan->type) {
39         case IIO_ANGL_VEL:          /* 陀螺仪的校准值 */
40             printk("read gyro calibbias\r\n");
41             return ret;
```

```
42          case IIO_ACCEL:             /* 加速度计的校准值 */
43              printk("read accel calibbias\r\n");
44              return ret;
45          default:
46              return -EINVAL;
47          }
48      default:
49          return ret -EINVAL;
50  }
```

示例代码 56.3.2.1 给出了很多分支，这些分支对应不同的文件读操作。比如我们读取 in_accel_scale 这个文件，这个是加速度计的比例文件，对应的就是第 19 行这个分支，结果如图 56.3.2.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device0/in_accel_scale
icm20608_read_raw
read accel sacle

[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.3.2.1 in_accel_sacle 文件读取操作

从图 56.3.2.1 可以看出，输出了“read accel scale”这行字符串，这行字符串正好是第 19 行的分支。我们就可以在这个分支里面读取 ICM20608 的加速度计比例值，也就是量程。其他文件也一样，最终都会对应到相应的分支里面，我们只需要在相应的分支里面做具体的操作就行了。剩下的操作就是读取 ICM20608 的内部寄存器数据，最终修改好的 icm20608_read_raw 函数和其他一些配套函数如下：

示例代码 56.3.2.2 完善的 icm20608_read_raw 函数

```
22  #define ICM20608_TEMP_OFFSET        0
23  #define ICM20608_TEMP_SCALE         326800000
......
65  /* icm20608 陀螺仪分辨率，对应 250、500、1000、2000，计算方法：
66   * 以正负 250 度量程为例，500/2^16=0.007629，扩大 1000000 倍，就是 7629
67   */
68  static const int gyro_scale_icm20608[] = {7629, 15258,
                                              30517, 61035};
69
70  /* icm20608 加速度计分辨率，对应 2、4、8、16 计算方法：
71   * 以正负 2g 量程为例，4/2^16=0.000061035，扩大 1000000000 倍，就是 61035
72   */
73  static const int accel_scale_icm20608[] = {61035, 122070,
                                                244140, 488281};
74
......
155
156 /*
157   * @description  : 读取 ICM20608 传感器数据，可以用于陀螺仪、加速度计、温度
158   * @param - dev  : icm20608 设备
```

```
159  * @param - reg  : 要读取的通道寄存器首地址。
160  * @param - anix : 需要读取的通道，比如 X，Y，Z。
161  * @param - *val : 保存读取到的值。
162  * @return       : 0，成功；其他值，错误
163  */
164 static int icm20608_sensor_show(struct icm20608_dev *dev, int reg,
165                 int axis, int *val)
166 {
167     int ind, result;
168     __be16 d;
169 
170     ind = (axis - IIO_MOD_X) * 2;
171     result = regmap_bulk_read(dev->regmap, reg + ind, (u8 *)&d, 2);
172     if (result)
173         return -EINVAL;
174     *val = (short)be16_to_cpup(&d);
175 
176     return IIO_VAL_INT;
177 }
178 
179 /*
180  * @description         : 读取 ICM20608 陀螺仪、加速度计、温度通道值
181  * @param - indio_dev   : iio 设备
182  * @param - chan        : 通道。
183  * @param - val         : 保存读取到的通道值。
184  * @return              : 0，成功；其他值，错误
185  */
186 static int icm20608_read_channel_data(struct iio_dev *indio_dev,
187                   struct iio_chan_spec const *chan,
188                   int *val)
189 {
190     struct icm20608_dev *dev = iio_priv(indio_dev);
191     int ret = 0;
192 
193     switch (chan->type) {
194     case IIO_ANGL_VEL:  /* 读取陀螺仪数据 */
195         ret = icm20608_sensor_show(dev, ICM20_GYRO_XOUT_H,
                        chan->channel2, val);  /* channel2 为 X、Y、Z 轴 */
196         break;
197     case IIO_ACCEL:     /* 读取加速度计数据 */
198         ret = icm20608_sensor_show(dev, ICM20_ACCEL_XOUT_H,
                          chan->channel2, val); /* channel2 为 X、Y、Z 轴 */
199         break;
```

```
200     case IIO_TEMP:      /* 读取温度 */
201         ret = icm20608_sensor_show(dev, ICM20_TEMP_OUT_H,
                                       IIO_MOD_X, val);
202         break;
203     default:
204         ret = -EINVAL;
205         break;
206     }
207     return ret;
208 }
209
210 /*
211  * @description  : 读函数，当读取 sysfs 中的文件的时候最终此函数会执行，此函
212  *                : 数里面会从传感器里面读取各种数据，然后上传给应用。
213  * @param - indio_dev   : iio_dev
214  * @param - chan        : 通道
215  * @param - val         : 读取的值，如果是小数值的话，val 是整数部分。
216  * @param - val2        : 读取的值，如果是小数值的话，val2 是小数部分。
217  * @param - mask        : 掩码。
218  * @return              : 0，成功；其他值，错误
219  */
220 static int icm20608_read_raw(struct iio_dev *indio_dev,
221             struct iio_chan_spec const *chan,
222             int *val, int *val2, long mask)
223 {
224     struct icm20608_dev *dev = iio_priv(indio_dev);
225     int ret = 0;
226     unsigned char regdata = 0;
227
228     switch (mask) {
229     case IIO_CHAN_INFO_RAW: /* 读取 ICM20608 加速度计、陀螺仪、
                                   温度传感器原始值 */
230         iio_device_claim_direct_mode(indio_dev);/* 保持 direct 模式 */
231         mutex_lock(&dev->lock);                 /* 上锁          */
232         ret = icm20608_read_channel_data(indio_dev, chan, val);
233         mutex_unlock(&dev->lock);               /* 释放锁        */
234         iio_device_release_direct_mode(indio_dev);
235         return ret;
236     case IIO_CHAN_INFO_SCALE:
237         switch (chan->type) {
238         case IIO_ANGL_VEL:
239             mutex_lock(&dev->lock);
240             regdata = (icm20608_read_onereg(dev, ICM20_GYRO_CONFIG)
```

```
                                                & 0X18) >> 3;
241             *val  = 0;
242             *val2 = gyro_scale_icm20608[regdata];
243             mutex_unlock(&dev->lock);
244             return IIO_VAL_INT_PLUS_MICRO;  /* 值为 val+val2/1000000 */
245         case IIO_ACCEL:
246             mutex_lock(&dev->lock);
247             regdata = (icm20608_read_onereg(dev, ICM20_ACCEL_CONFIG)
                                                & 0X18) >> 3;
248             *val = 0;
249             *val2 = accel_scale_icm20608[regdata];;
250             mutex_unlock(&dev->lock);
251             return IIO_VAL_INT_PLUS_NANO;/* 值为 val+val2/1000000000 */
252         case IIO_TEMP:
253             *val = ICM20608_TEMP_SCALE/ 1000000;
254             *val2 = ICM20608_TEMP_SCALE % 1000000;
255             return IIO_VAL_INT_PLUS_MICRO;  /* 值为 val+val2/1000000 */
256         default:
257             return -EINVAL;
258         }
259         return ret;
260     case IIO_CHAN_INFO_OFFSET:      /* ICM20608 温度传感器 offset 值 */
261         switch (chan->type) {
262         case IIO_TEMP:
263             *val = ICM20608_TEMP_OFFSET;
264             return IIO_VAL_INT;
265         default:
266             return -EINVAL;
267         }
268         return ret;
269     case IIO_CHAN_INFO_CALIBBIAS:   /* ICM20608 加速度计和陀螺仪校准值 */
270         switch (chan->type) {
271         case IIO_ANGL_VEL:          /* 陀螺仪的校准值*/
272             mutex_lock(&dev->lock);
273             ret = icm20608_sensor_show(dev, ICM20_XG_OFFS_USRH,
                                           chan->channel2, val);
274             mutex_unlock(&dev->lock);
275             return ret;
276         case IIO_ACCEL:             /* 加速度计的校准值*/
277             mutex_lock(&dev->lock);
278             ret = icm20608_sensor_show(dev, ICM20_XA_OFFSET_H,
                                           chan->channel2, val);
279             mutex_unlock(&dev->lock);
```

```
280             return ret;
281         default:
282             return -EINVAL;
283         }
284
285     default:
286         return ret -EINVAL;
287     }
288 }
```

第 68 行，gyro_scale_icm20608 是 ICM20608 的陀螺仪比例，也就是陀螺仪不同量程对应的分辨率。ICM20608 陀螺仪为 16bit，可选的量程有±250、±500、±1000 和±2000°/s。以±250 这个量程为例，每个数值对应的度数就是 500/2^10≈0.007629°/s。同理，±500 量程对应的度数为 0.015258，±1000 量程对应的度数为 0.030517，±2000 量程为 0.061035。假设现在设置量程为±2000，读取到的原始值为 12540，那么对应的度数就是 12540*0.061035≈765.37°/s。注意，这里扩大了 1000000 倍

第 73 行，accel_scale_icm20608 是 ICM20608 的加速度计比例，也就是加速度计不同量程对应的分辨率，计算方法和陀螺仪一样。这里扩大了 1000000000 倍。

第 164~177 行，icm20608_sensor_show 函数用于读取加速度计、陀螺仪、温度的原始数据的。

第 186~08 行，icm20608_read_channel_data 函数用于读取指定通道的数据，根据 type 类型的不同，给 icm20608_sensor_show 函数传递不同的参数，读取陀螺仪、加速度计或温度数据。

第 220~288 行，修改完善的 icm20608_read_raw 函数，我们就以读取陀螺仪、加速度计或温度原始数据为例，当应用程序读取这几个通道的原始数据的时候第 229 行的分支会执行。第 232 行直接调用 icm20608_read_channel_data 函数来根据通道的 type 值来读取 ICM20608 相应的寄存器。其他分支内容大同小异，这些就是 ICM20608 寄存器读取内容了，这里就不在多讲。

修改完成以后重新编译驱动文件，然后加载新的驱动，测试是否可以正常读取到相应的内容。我们读取一下 in_accel_scale 这个文件，这是加速度计的分辨率，我们默认设置了加速度计量程为±16g，因此分辨率为 0.000488281。结果如图 56.3.2.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device0/in_accel_scale
0.000488281
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.3.2.1 in_accel_scale 内容

从图 56.3.2.1 可以看出，此时 in_accel_scale 为 0.000488281，是正确的。这时候有朋友可能会疑问，在示例代码 56.3.2.2 中，我们设置加速度计±16g 的分辨率为 488281，也就是是扩大了 1000000000 倍，为啥这里读出来的是 0.000488281 这个原始值？大家注意看示例代码 56.3.2.2 中第 245~251 行所在的分支，这部分是读取加速度计的分辨率，读取完成以后在第 251 行返回 IIO_VAL_INT_PLUS_NANO 这个值，这里就是告诉用户空间，小数部分(val2)扩大了 1000000000 倍，因此用户空间得到分辨率以后会除以 1000000000，得到真实的分辨率，488281/1000000000=0.000488281，所以图 56.3.2.1 中得到的值就是 0.000488281。

我们在读取一下 in_accel_z_raw 这个文件，这个文件是加速度计的 Z 轴原始值，静态情况下 Z 轴应该是 1g 的重力加速度计，我们可以读取 in_accel_z_raw 这个文件的值，然后在结合上面读取到的加速度计分辨率，计算一下对应的 Z 轴重力值，看看是不是 1g 左右。

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device0/in_accel_z_raw
2063
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.3.2.2 in_accel_z_raw 值

2063×0.000488281≈1g，此时 Z 轴重力为 1g，结果正确。

56.3.3 完善 icm20608_write_raw 函数

最后我们完善 icm20608_write_raw 函数，用户空间向驱动写数据的时候 icm20608_write_raw 函数会执行。我们可以在用户空间设置陀螺仪、加速度计的量程、校准值等，这时候就需要向驱动写入数据。本章我们简单一点，就只实现设置陀螺仪和加速度计的量程，至于其他的设置项大家可以自行实现，修改后的 icm20608_write_raw 函数和其他相关内容如下：

示例代码 56.3.3.1 完善的 icm20608_write_raw 函数

```
180 /*
181  * @description  : 设置 ICM20608 传感器，可以用于陀螺仪、加速度计设置
182  * @param - dev  : icm20608 设备
183  * @param - reg  : 要设置的通道寄存器首地址。
184  * @param - anix : 要设置的通道，比如 X，Y，Z。
185  * @param - val  : 要设置的值。
186  * @return       : 0，成功；其他值，错误
187  */
188 static int icm20608_sensor_set(struct icm20608_dev *dev, int reg,
189                                int axis, int val)
190 {
191     int ind, result;
192     __be16 d = cpu_to_be16(val);
193
194     ind = (axis - IIO_MOD_X) * 2;
195     result = regmap_bulk_write(dev->regmap, reg + ind, (u8 *)&d, 2);
196     if (result)
197         return -EINVAL;
198
199     return 0;
200 }
......
256 /*
257  * @description  : 设置 ICM20608 的陀螺仪计量程(分辨率)
258  * @param - dev  : icm20608 设备
259  * @param - val  : 量程(分辨率值)。
260  * @return       : 0，成功；其他值，错误
261  */
262 static int icm20608_write_gyro_scale(struct icm20608_dev *dev,
                                         int val)
263 {
```

```
264     int result, i;
265     u8 d;
266
267     for (i = 0; i < ARRAY_SIZE(gyro_scale_icm20608); ++i) {
268         if (gyro_scale_icm20608[i] == val) {
269             d = (i << 3);
270             result = regmap_write(dev->regmap, ICM20_GYRO_CONFIG, d);
271             if (result)
272                 return result;
273             return 0;
274         }
275     }
276     return -EINVAL;
277 }
278
279  /*
280   * @description  : 设置 ICM20608 的加速度计量程(分辨率)
281   * @param - dev  : icm20608 设备
282   * @param - val  : 量程(分辨率值)。
283   * @return       : 0，成功；其他值，错误
284   */
285 static int icm20608_write_accel_scale(struct icm20608_dev *dev,
                                          int val)
286 {
287     int result, i;
288     u8 d;
289
290     for (i = 0; i < ARRAY_SIZE(accel_scale_icm20608); ++i) {
291         if (accel_scale_icm20608[i] == val) {
292             d = (i << 3);
293             result = regmap_write(dev->regmap,
                                      ICM20_ACCEL_CONFIG, d);
294             if (result)
295                 return result;
296             return 0;
297         }
298     }
299     return -EINVAL;
300 }
......
382 /*
383   * @description  : 写函数，当向 sysfs 中的文件写数据的时候最终此函数会执行，
384   *               : 一般在此函数里面设置传感器，比如量程等。
```

```
385   * @param - indio_dev   : iio_dev
386   * @param - chan        : 通道
387   * @param - val      : 应用程序写入的值，如果是小数值的话，val 是整数部分。
388   * @param - val2     : 应用程序写入的值，如果是小数值的话，val2 是小数部分。
389   * @return           : 0，成功；其他值，错误
390   */
391 static int icm20608_write_raw(struct iio_dev *indio_dev,
392               struct iio_chan_spec const *chan,
393               int val, int val2, long mask)
394 {
395     struct icm20608_dev *dev = iio_priv(indio_dev);
396     int ret = 0;
397 
398     iio_device_claim_direct_mode(indio_dev);
399     switch (mask) {
400     case IIO_CHAN_INFO_SCALE:   /* 设置陀螺仪和加速度计的分辨率 */
401         switch (chan->type) {
402         case IIO_ANGL_VEL:      /* 设置陀螺仪 */
403             mutex_lock(&dev->lock);
404             ret = icm20608_write_gyro_scale(dev, val2);
405             mutex_unlock(&dev->lock);
406             break;
407         case IIO_ACCEL:         /* 设置加速度计 */
408             mutex_lock(&dev->lock);
409             ret = icm20608_write_accel_scale(dev, val2);
410             mutex_unlock(&dev->lock);
411             break;
412         default:
413             ret = -EINVAL;
414             break;
415         }
416         break;
417     case IIO_CHAN_INFO_CALIBBIAS:   /* 设置陀螺仪和加速度计的校准值  */
418         switch (chan->type) {
419         case IIO_ANGL_VEL:          /* 设置陀螺仪校准值             */
420             mutex_lock(&dev->lock);
421             ret = icm20608_sensor_set(dev, ICM20_XG_OFFS_USRH,
422                                       chan->channel2, val);
423             mutex_unlock(&dev->lock);
424             break;
425         case IIO_ACCEL:             /* 加速度计校准值               */
426             mutex_lock(&dev->lock);
427             ret = icm20608_sensor_set(dev, ICM20_XA_OFFSET_H,
```

```
428                                     chan->channel2, val);
429             mutex_unlock(&dev->lock);
430             break;
431         default:
432             ret = -EINVAL;
433             break;
434         }
435         break;
436     default:
437         ret = -EINVAL;
438         break;
439     }
440 
441     iio_device_release_direct_mode(indio_dev);
442     return ret;
443 }
444 
445 /*
446  * @description  : 用户空间写数据格式，比如我们在用户空间操作 sysfs 来设置传
447  *                : 感器的分辨率，如果分辨率带小数，那么这个小数传递到内核空间
448  *                : 应该扩大多少倍，此函数就是用来设置这个的。
449  * @param - indio_dev   : iio_dev
450  * @param - chan        : 通道
451  * @param - mask        : 掩码
452  * @return              : 0，成功；其他值，错误
453  */
454 static int icm20608_write_raw_get_fmt(struct iio_dev *indio_dev,
455                 struct iio_chan_spec const *chan, long mask)
456 {
457     switch (mask) {
458     case IIO_CHAN_INFO_SCALE:
459         switch (chan->type) {
460         case IIO_ANGL_VEL:
461             return IIO_VAL_INT_PLUS_MICRO;
462         default:
463             return IIO_VAL_INT_PLUS_NANO;
464         }
465     default:
466         return IIO_VAL_INT_PLUS_MICRO;
467     }
468     return -EINVAL;
469 }
470 
```

```
471 /*
472  * iio_info 结构体变量
473  */
474 static const struct iio_info icm20608_info = {
475     .read_raw      = icm20608_read_raw,
476     .write_raw     = icm20608_write_raw,
477     .write_raw_get_fmt = &icm20608_write_raw_get_fmt,
478 };
```

第 188~200 行，icm20608_sensor_set 函数用于设置指定通道，也就是向 ICM20608 的指定寄存器写入数据，用于设置陀螺仪和加速度计的校准值。

第 262~277 行，icm20608_write_gyro_scale 函数用于设置 ICM20608 陀螺仪量程。

第 285~300 行，icm20608_write_accel_scale 函数用于设置 ICM20608 加速度计量程

第 391~443 行，icm20608_write_raw 函数，也就是 iio_info 的 write 函数，用户空间向驱动程序写入数据以后此函数就会执行，这里我们用来配置 ICM20608 的陀螺仪和加速度量程和校准值。第 400 行的 IIO_CHAN_INFO_SCALE 表示此分支是设置比例(量程的)，第 404 行设置陀螺仪的量程，第 409 行设置加速度计的量程。第 417 行的 IIO_CHAN_INFO_CALIBBIAS 表示此分支是设置校准值的，第 421 行设置陀螺仪各轴校准值，第 427 行设置加速度计各轴校准值。

第 454~469 行，icm20608_write_raw_get_fmt 函数，也就是 iio_info 的 write_raw_get_fmt 函数。此函数用来指定用户空间写入的数据格式，第 458 行的 IIO_CHAN_INFO_SCALE 分支表示要操作的是比例，也就是用户空间要设置陀螺仪和加速度计的量程。第 460 行表示如果写入的是陀螺仪比例，那么小数部分就要扩大 1000000 倍。第 463 行表示如果是其他的就扩 1000000000 倍，比如加速度计。比如我们在用户空间要设置加速度计量程为±4g，只需要向 in_accel_scale 写入 0.000122070，那么最终传入到驱动里面的就是 0.000122070*1000000000=122070。

ICM20608 的 IIO 驱动就已经编写完成了，接下来就是编写测试 APP。

56.4 测试应用程序编写

56.4.1 linux 文件流读取

前面我们都是直接使用 cat 命令读取对应文件的内容，如果要连续不断的读取传感器数据就不能用 cat 命令了，我们需要编写对应的 APP 软件，在编写 APP 之前我们先了解一下所要用到的 API 函数。

首先我们要知道，前面使用 cat 命令读取到的文件内容字符串，虽然看起来像是数字。比如我们使用 cat 命令读取到的 in_accel_scale 如图 56.4.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device0/in_accel_scale
0.000488281
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.4.1 in_accel_scale 文件内容

图 56.4.1 中 in_accel_scale 文件内容为 0.000488281，但是这里的 0.000488281 是字符串，并不是具体的数字，所以我们需要将其转换为对应的数字。另外 in_accel_scale 是流文件，也叫做标准文件 I/O 流，因此打开、读写操作要使用文件流操作函数。

1、打开文件流

打开文件流使用 fopen 函数，函数原型如下：

```
FILE *fopen(const char *pathname, const char *mode)
```

函数参数和返回值含义如下：

pathname：需要打开的文件流路径。

mode：打开方式，可选的打开方式如表 56.4.1 所示：

mode	描述
r	打开只读文件。
r+	打开读写文件。
w	打开只写文件，如文件存在则文件长度清零，如文件不存在就自动创建，文件流指针调整到文件头部。
w+	打开可读写文件，如文件存在则文件长度清零，如文件不存在就自动创建，文件流指针调整到文件头部。
a	以追加的方式打开只写文件，如果文件不存在就新建文件，如果存在就将数据追加到文件末尾。
a+	以追加的方式打开读写文件，如果文件不存在就新建文件，如果存在就将数据追加到文件末尾。

表 56.4.1 打开方式

返回值：NULL，打开错误；其他值，打开成功的文件流指针，为 FILE 类型。

2、关闭文件流

关闭文件流使用函数 fclose，函数原型如下：

```
int fclose(FILE *stream)
```

函数参数和返回值含义如下：

stream：要关闭的文件流指针。

返回值：0，关闭成功；EOF，关闭错误。

3、读取文件流

要读取文件流使用 fread 函数，函数原型如下：

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream)
```

fread 函数用于从给定的输入流中读取最多 nmemb 个对象到数组 ptr 中，函数参数和返回值含义如下：

ptr：要读取的数组中首个对象的指针。

size：每个对象的大小。

nmemb：要读取的对象个数。

stream：要读取的文件流。

返回值：返回读取成功的对象个数，如果出现错误或到文件末尾，那么返回一个短计数值(或者 0)。

4、写文件流

要向文件流写入数据，使用 fwrite 函数，函数原型如下：

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

fwrite 函数用于向给定的文件流中写入最多 nmemb 个对象，函数参数和返回值含义如下：

ptr：要写入的数组中首个对象的指针。

size：每个对象的大小。

nmemb：要写入的对象个数。

stream：要写入的文件流。

返回值：返回成功写入的对象个数，如果出现错误或到文件末尾，那么返回一个短计数值(或者 0)。

5、格式化输入文件流

fscanf 函数用于从一个文件流中格式化读取数据，fscanf 函数在遇到空格和换行符的时候就会结束。前面我们说了 IIO 框架下的 sysfs 文件内容都是字符串，比如 in_accel_scale 文件内容为“0.000488281”，这是一串字符串，并不是具体的数字，因此我们在读取的时候就需要使用字符串读取格式。在这里就可以使用 fscanf 函数来格式化读取文件内容，函数原型如下：

```
int fscanf(FILE *stream, const char *format, ,[argument...])
```

fscanf 用法和 scanf 类似，函数参数和返回值含义如下：

stream：要操作的文件流。

format：格式。

argument：保存读取到的数据。

返回值：成功读取到的数据个数，如果读到文件末尾或者读取错误就返回 EOF。

56.4.2 编写测试 APP

新建名为 icm20608App.c 的文件，输入如下内容：

示例代码 56.4.2.1 icm20608App.c 文件代码段

```
/***************************************************************
Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
文件名       : icm20608.c
作者         : 左忠凯
版本         : V1.0
描述         : icm20608 设备 iio 框架测试程序。
其他         : 无
使用方法     : ./icm20608App
论坛         : www.openedv.com
日志         : 初版 V1.0 2021/8/17 左忠凯创建
***************************************************************/
#include "stdio.h"
#include "unistd.h"
#include "sys/types.h"
#include "sys/stat.h"
#include "sys/ioctl.h"
#include "fcntl.h"
#include "stdlib.h"
#include "string.h"
#include <poll.h>
#include <sys/select.h>
#include <sys/time.h>
#include <signal.h>
```

```
13  #include <fcntl.h>
14  #include <errno.h>
15
16  /* 字符串转数字，将浮点小数字符串转换为浮点数数值 */
17  #define SENSOR_FLOAT_DATA_GET(ret, index, str, member)\
18      ret = file_data_read(file_path[index], str);\
19      dev->member = atof(str);\
20
21  /* 字符串转数字，将整数字符串转换为整数数值 */
22  #define SENSOR_INT_DATA_GET(ret, index, str, member)\
23      ret = file_data_read(file_path[index], str);\
24      dev->member = atoi(str);\
25
26  /* icm20608 iio 框架对应的文件路径 */
27  static char *file_path[] = {
28      "/sys/bus/iio/devices/iio:device0/in_accel_scale",
29      "/sys/bus/iio/devices/iio:device0/in_accel_x_calibbias",
30      "/sys/bus/iio/devices/iio:device0/in_accel_x_raw",
31      "/sys/bus/iio/devices/iio:device0/in_accel_y_calibbias",
32      "/sys/bus/iio/devices/iio:device0/in_accel_y_raw",
33      "/sys/bus/iio/devices/iio:device0/in_accel_z_calibbias",
34      "/sys/bus/iio/devices/iio:device0/in_accel_z_raw",
35      "/sys/bus/iio/devices/iio:device0/in_anglvel_scale",
36      "/sys/bus/iio/devices/iio:device0/in_anglvel_x_calibbias",
37      "/sys/bus/iio/devices/iio:device0/in_anglvel_x_raw",
38      "/sys/bus/iio/devices/iio:device0/in_anglvel_y_calibbias",
39      "/sys/bus/iio/devices/iio:device0/in_anglvel_y_raw",
40      "/sys/bus/iio/devices/iio:device0/in_anglvel_z_calibbias",
41      "/sys/bus/iio/devices/iio:device0/in_anglvel_z_raw",
42      "/sys/bus/iio/devices/iio:device0/in_temp_offset",
43      "/sys/bus/iio/devices/iio:device0/in_temp_raw",
44      "/sys/bus/iio/devices/iio:device0/in_temp_scale",
45  };
46
47  /* 文件路径索引，要和 file_path 里面的文件顺序对应 */
48  enum path_index {
49      IN_ACCEL_SCALE = 0,
50      IN_ACCEL_X_CALIBBIAS,
51      IN_ACCEL_X_RAW,
52      IN_ACCEL_Y_CALIBBIAS,
53      IN_ACCEL_Y_RAW,
54      IN_ACCEL_Z_CALIBBIAS,
55      IN_ACCEL_Z_RAW,
```

```
56      IN_ANGLVEL_SCALE,
57      IN_ANGLVEL_X_CALIBBIAS,
58      IN_ANGLVEL_X_RAW,
59      IN_ANGLVEL_Y_CALIBBIAS,
60      IN_ANGLVEL_Y_RAW,
61      IN_ANGLVEL_Z_CALIBBIAS,
62      IN_ANGLVEL_Z_RAW,
63      IN_TEMP_OFFSET,
64      IN_TEMP_RAW,
65      IN_TEMP_SCALE,
66  };
67
68  /*
69   * icm20608 数据设备结构体
70   */
71  struct icm20608_dev{
72      int accel_x_calibbias, accel_y_calibbias, accel_z_calibbias;
73      int accel_x_raw, accel_y_raw, accel_z_raw;
74
75      int gyro_x_calibbias, gyro_y_calibbias, gyro_z_calibbias;
76      int gyro_x_raw, gyro_y_raw, gyro_z_raw;
77
78      int temp_offset, temp_raw;
79
80      float accel_scale, gyro_scale, temp_scale;
81
82      float gyro_x_act, gyro_y_act, gyro_z_act;
83      float accel_x_act, accel_y_act, accel_z_act;
84      float temp_act;
85  };
86
87  struct icm20608_dev icm20608;
88
89   /*
90   * @description       : 读取指定文件内容
91   * @param - filename : 要读取的文件路径
92   * @param - str       : 读取到的文件字符串
93   * @return            : 0 成功;其他 失败
94   */
95  static int file_data_read(char *filename, char *str)
96  {
97      int ret = 0;
98      FILE *data_stream;
```

```
99
100     data_stream = fopen(filename, "r"); /* 只读打开 */
101     if(data_stream == NULL) {
102         printf("can't open file %s\r\n", filename);
103         return -1;
104     }
105
106     ret = fscanf(data_stream, "%s", str);
107     if(!ret) {
108         printf("file read error!\r\n");
109     } else if(ret == EOF) {
110         /* 读到文件末尾的话将文件指针重新调整到文件头 */
111         fseek(data_stream, 0, SEEK_SET);
112     }
113     fclose(data_stream);    /* 关闭文件 */
114     return 0;
115 }
116
117  /*
118  * @description   : 获取 ICM20608 数据
119  * @param - dev   : 设备结构体
120  * @return        : 0 成功;其他 失败
121  */
122 static int sensor_read(struct icm20608_dev *dev)
123 {
124     int ret = 0;
125     char str[50];
126
127     /* 1、获取陀螺仪原始数据 */
128     SENSOR_FLOAT_DATA_GET(ret, IN_ANGLVEL_SCALE, str, gyro_scale);
129     SENSOR_INT_DATA_GET(ret, IN_ANGLVEL_X_RAW, str, gyro_x_raw);
130     SENSOR_INT_DATA_GET(ret, IN_ANGLVEL_Y_RAW, str, gyro_y_raw);
131     SENSOR_INT_DATA_GET(ret, IN_ANGLVEL_Z_RAW, str, gyro_z_raw);
132
133     /* 2、获取加速度计原始数据 */
134     SENSOR_FLOAT_DATA_GET(ret, IN_ACCEL_SCALE, str, accel_scale);
135     SENSOR_INT_DATA_GET(ret, IN_ACCEL_X_RAW, str, accel_x_raw);
136     SENSOR_INT_DATA_GET(ret, IN_ACCEL_Y_RAW, str, accel_y_raw);
137     SENSOR_INT_DATA_GET(ret, IN_ACCEL_Z_RAW, str, accel_z_raw);
138
139     /* 3、获取温度值 */
140     SENSOR_FLOAT_DATA_GET(ret, IN_TEMP_SCALE, str, temp_scale);
141     SENSOR_INT_DATA_GET(ret, IN_TEMP_OFFSET, str, temp_offset);
```

```
142     SENSOR_INT_DATA_GET(ret, IN_TEMP_RAW, str, temp_raw);
143
144     /* 3、转换为实际数值 */
145     dev->accel_x_act = dev->accel_x_raw * dev->accel_scale;
146     dev->accel_y_act = dev->accel_y_raw * dev->accel_scale;
147     dev->accel_z_act = dev->accel_z_raw * dev->accel_scale;
148
149     dev->gyro_x_act = dev->gyro_x_raw * dev->gyro_scale;
150     dev->gyro_y_act = dev->gyro_y_raw * dev->gyro_scale;
151     dev->gyro_z_act = dev->gyro_z_raw * dev->gyro_scale;
152
153     dev->temp_act = ((dev->temp_raw - dev->temp_offset) /
dev->temp_scale) + 25;
154     return ret;
155 }
156
157 /*
158  * @description   : main 主程序
159  * @param - argc  : argv 数组元素个数
160  * @param - argv  : 具体参数
161  * @return        : 0 成功;其他 失败
162  */
163 int main(int argc, char *argv[])
164 {
165     int ret = 0;
166
167     if (argc != 1) {
168         printf("Error Usage!\r\n");
169         return -1;
170     }
171
172     while (1) {
173         ret = sensor_read(&icm20608);
174         if(ret == 0) {          /* 数据读取成功 */
175             printf("\r\n 原始值:\r\n");
176             printf("gx = %d, gy = %d, gz = %d\r\n",
                        icm20608.gyro_x_raw,
                        icm20608.gyro_y_raw,
                        icm20608.gyro_z_raw);
177             printf("ax = %d, ay = %d, az = %d\r\n",
                        icm20608.accel_x_raw,
                        icm20608.accel_y_raw,
                        icm20608.accel_z_raw);
```

```
178             printf("temp = %d\r\n", icm20608.temp_raw);
179             printf("实际值:");
180             printf("act gx = %.2f°/S, act gy = %.2f°/S,
                        act gz = %.2f°/S\r\n", icm20608.gyro_x_act,
                        icm20608.gyro_y_act, icm20608.gyro_z_act);
181             printf("act ax = %.2fg, act ay = %.2fg,
                        act az = %.2fg\r\n", icm20608.accel_x_act,
                        icm20608.accel_y_act,
                        icm20608.accel_z_act);
182             printf("act temp = %.2f°C\r\n", icm20608.temp_act);
183         }
184         usleep(100000); /*100ms */
185     }
186
187     return 0;
188 }
```

第 17~19 行，SENSOR_FLOAT_DATA_GET 宏用读取指定路径的文件内容，然后将读到的浮点型字符串数据转换为具体的浮点数据。第 19 行使用 atof 函数将浮点字符串转换为具体的浮点数值。

第 22~24 行，SENSOR_INT_DATA_GET 宏用于读取指定路径的文件内容，将读取到的整数型字符串数据转换为具体的整数值，第 24 行使用 atoi 函数将整数字符串转换为具体的整数数值。

atof 和 atoi 这两个函数是标准的 C 库函数，并不需要我们自己编写，添加“stdlib.h”头文件就可以直接调用。这两个函数都只有一个参数，就是要转换的字符串。返回值就是转换成功以后字符串对应的数值。

第 27~45 行，需要操作的文件路径。

第 48~66 行，文件路径索引，顺序要和 file_path 里面的文件路径对应。

第 71~85 行，icm20608_dev 结构体为自定义设备结构体，用于保存 ICM20608 传感器数据。

第 95~115 行，file_data_read 函数用于读取指定文件，第一个参数 filename 是要读取的文件路径。第二个参数 str 为读取到的文件内容，为字符串类型，因为本章例程所读取的文件内容都是字符串。第 100 行调用 fopen 函数打开指定的文件流，第 106 行调用 fscanf 函数进行格式化读取，也就是按照字符串方式读取文件，文件内容保存到 str 参数里面。当读取到文件末尾的时候，第 111 行调用 fseek 函数将读取指针调整到文件头，以备下次重新读取。第 113 行调用 fclose 函数关闭对应的文件流。

第 122~155 行，sensor_read 函数用于读取 ICM20608 传感器数据，包括陀螺仪、加速度和温度计的原始值，还有加速度计和陀螺仪的分辨率等，最后将获取到的原始值转换为具体的数值。

第 163~188 行，main 函数，在 while 循环中调用 sensor_read 函数读取 ICM20608 数据，并将读到的数据打印出来。

56.4.3 运行测试

输入如下命令编译测试 icm20608App.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc icm20608App.c -o icm20608App
```

编译成功以后就会生成 icm20608App 这个应用程序。

将 icm20608.ko 和 icm20608App 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，重启开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 icm20608.ko 驱动模块：

```
depmod                  //第一次加载驱动的时候需要运行此命令
modprobe icm20608.ko  //加载驱动
```

驱动加载成功以后使用如下命令来测试：

```
./icm20608App
```

如果驱动和 APP 工作正常的话就会不断的打印出 ICM20608 数据，包括陀螺仪和加速度计的原始数据和转换后的实际数值、温度等，如图 56.4.3.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./icm20608App

原始值:
gx = -42, gy = 6, gz = -5
ax = -12, ay = -53, az = 2064
temp = 5074
实际值:act gx = -2.56°/S, act gy = 0.37°/S, act gz = -0.31°/S
act ax = -0.01g, act ay = -0.03g, act az = 1.01g
act temp = 40.53°C
```

图 56.4.3.1 读取到的 ICM20608 数值。

56.5 IIO 触发缓冲区

触发缓冲区就是基于某种信号来触发数据采集，这些信号就是触发器，比如：

①、传感器数据就绪中断。

②、周期性中断。

③、用户空间下读取 sysfs 下的指定文件。

触发器肯定是触发数据的采集，数据采集到以后会填充到缓冲区里面，最终会以字符设备形式提供给用户空间，用户空间直接读取缓冲区文件即可。

56.5.1 IIO 触发器

linux 内核使用 iio_trigger 结构体表示触发器，定义在 include/linux/iio/trigger.h 文件中，内容如下：

示例代码 56.5.1.1 iio_trigger 结构体

```
1  struct iio_trigger {
2   const struct iio_trigger_ops    *ops;
3   struct module                   *owner;
4   int                             id;
5   const char                      *name;
6   struct device                   dev;
7
8   struct list_head                list;
9   struct list_head                alloc_list;
10  atomic_t                        use_count;
11
12  struct irq_chip                 subirq_chip;
```

```
13  int                             subirq_base;
14
15  struct iio_subirq subirqs[CONFIG_IIO_CONSUMERS_PER_TRIGGER];
16  unsigned long pool[BITS_TO_LONGS(CONFIG_IIO_CONSUMERS_PER_TRIGGER)];
17  struct mutex                    pool_lock;
18  bool                            attached_own_device;
19 };
```

重点是第 2 行的 iio_trigger_ops，这个是触发器的操作函数结构体，内容如下：

示例代码 56.5.1.2 iio_trigger_ops 结构体

```
32 struct iio_trigger_ops {
33     int (*set_trigger_state)(struct iio_trigger *trig, bool state);
34     int (*try_reenable)(struct iio_trigger *trig);
35     int (*validate_device)(struct iio_trigger *trig,
36             struct iio_dev *indio_dev);
37 };
```

第 33 行，set_trigger_state 函数用于设置触发器状态，也就是打开/关闭触发器。如果用中断作为触发器的话，此函数一般设置传感器的中断使能状态。

第 34 行，try_reenable 函数当用户计数为 0 的时候尝试重新使能触发器。

第 35 行，validate_device 函数用于当触发器改变的时候，使设备生效。

自行创建触发器的话，需要驱动开发人员编写 iio_trigger_ops。

1、申请触发器

首先需要使用 iio_trigger_alloc 函数创建触发器，函数原型如下：

```
struct iio_trigger *iio_trigger_alloc(const char *fmt, ...)
```

可以看出 iio_trigger_alloc 是个可变长度参数函数，但是它的目的只有一个：拼凑触发器名字，和 printf 函数用法类似，比如下面这一行：

```
iio_trigger_alloc("%s-dev%d", indio_dev->name, indio_dev->id)
```

假设 indio_dev->name 为“icm20608”，indio_dev->id 为 0，那么触发器的名字就是“icm20608-dev0”，这个也是我们稍后编写本节例程的时候定义的触发器名字，如图 56.5.1.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat trigger0/name
icm20608-dev0
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.5.1.1 触发器名字

当申请成功以后 iio_trigger_alloc 函数就会返回申请到的 iio_trigger。也可以使用 devm_iio_trigger_alloc 函数申请触发器，这样在卸载驱动的时候就不用手动释放触发器了。

2、释放触发器

如果要释放触发器就使用 iio_trigger_free，函数原型如下：

```
void iio_trigger_free(struct iio_trigger *trig)
```

函数参数和返回值含义如下：

trig：要释放的触发器。

返回值：无。

3、注册触发器

触发器申请并初始化完成以后就需要向内核注册触发器，函数为 devm_iio_trigger_register，函数原型如下：

```
int iio_trigger_register(struct iio_trigger *trig_info)
```

函数参数和返回值含义如下：

trig_info：要注册的触发器。

返回值：0，成功；其他值，失败。

4、注销触发器

注销触发器使用 iio_trigger_unregister 函数，函数原型如下：

```
void iio_trigger_unregister(struct iio_trigger *trig_info)
```

函数参数和返回值含义如下：

trig_info：要注销的触发器。

返回值：无。

56.5.2 IIO 缓冲区

IIO 缓冲区就是保存采集到的数据，用户空间可以直接通过访问字符设备/dev/iio\:deviceX(X=0,1,2……)来读取缓冲区中的数据。编写驱动的时候我们需要先创建缓冲区，函数为 iio_triggered_buffer_setup，函数原型如下：

```
int iio_triggered_buffer_setup( struct iio_dev       *indio_dev,
                                irqreturn_t (*h)(int irq, void *p),
                                irqreturn_t (*thread)(int irq, void *p),
                                const struct iio_buffer_setup_ops *setup_ops)
```

函数参数和返回值含义如下：

indio_dev：需要创建缓冲的 iio_dev。

h：触发器中断上半部，上半部程序一定要简单，执行速度越快越好，一般就是提供捕获时间戳。可以直接使用 IIO 框架提供的 iio_pollfunc_store_time 函数。

thread：触发器中断下半部，重要的处理就在这里面，此函数中我们需要将传感器中的数据和上半部获取到的时间戳一起推送到缓冲区中。

setup_ops：缓冲区操作函数集，iio_buffer_setup_ops 结构体内容如下：

示例代码 56.5.2.1 iio_buffer_setup_ops 结构体

```
474 struct iio_buffer_setup_ops {
475     int (*preenable)(struct iio_dev *);
476     int (*postenable)(struct iio_dev *);
477     int (*predisable)(struct iio_dev *);
478     int (*postdisable)(struct iio_dev *);
479     bool (*validate_scan_mask)(struct iio_dev *indio_dev,
480                    const unsigned long *scan_mask);
481 };
```

如果设置为 NULL，那么就会使用默认的操作集 iio_triggered_buffer_setup_ops。

返回值：无。

一定要在调用 iio_device_register 函数注册 IIO 设备之前设置缓冲区。

56.6 向驱动程序添加触发缓冲功能

56.6.1 修改设备树

中断是最常用的触发方式，因为一般的传感器都有中断功能，当数据准备就绪或者指定事件发生以后就会产生中断通知 SOC，此时 SOC 就可以读取传感器内部数据。首先我们要修改设备树，添加 ICM20608 中断引脚 PA14 的配置，打开 stm32mp15-pinctrl.dtsi 文件，PA14 引脚配置如下：

示例代码 56.6.1.1 ICM20608 中断引脚配置

```
1 icm20608_pins_b: icm20608-0 {
2   pins {
3       pinmux = <STM32_PINMUX('A', 14, ANALOG)>;
4       bias-pull-up;
5     };
6 };
```

另外，打开 stm32mp157d-atk.dts 文件，修改 icm20608 节点，添加中断信息，内容如下：

示例代码 56.6.1.2 icm20608 节点

```
1 spidev: icm20608@0 {
2       compatible = "alientek,icm20608";
3       reg = <0>; /* CS #0 */
4       pinctrl-names = "default";
5       pinctrl-0 = <&icm20608_pins_b>;
6       interrupt-parent = <&gpioa>;
7       interrupts = <14 IRQ_TYPE_LEVEL_HIGH>;
8       spi-max-frequency = <80000000>;
9 };
```

第 6 行，设置中断父节点为 gpioa，第 7 行设置 PA14 为高电平触发。

修改完成以后重新编译设备树，然后使用新的设备树启动 linux 内核。

56.6.2 驱动程序编写

接下来我们就在 56.3 小节里面编写好的 ICM20608 IIO 驱动程序中添加触发缓冲功能，内容如下(仅列出触发缓冲相关功能)：

示例代码 56.6.2.1 触发缓冲程序

```
27  #define ICM20608_BIT_FIFO_OVERFLOW_INT      0x10
28  #define ICM20608_BIT_RAW_DATA_RDY_INT       0x01
......
112 /*
113  * 扫描掩码，两种情况，全启动 0X1111111，或者都不启动 0X0
114  */
115 static const unsigned long icm20608_scan_masks[] = {
116     BIT(INV_ICM20608_SCAN_ACCL_X)
117     | BIT(INV_ICM20608_SCAN_ACCL_Y)
118     | BIT(INV_ICM20608_SCAN_ACCL_Z)
```

```
119     | BIT(INV_ICM20608_SCAN_GYRO_X)
120     | BIT(INV_ICM20608_SCAN_GYRO_Y)
121     | BIT(INV_ICM20608_SCAN_GYRO_Z)
122     | BIT(INV_ICM20608_SCAN_TEMP),
123     0,
124 };
......
480 /*
481  * @description  : 触发器下半部函数，从启动的通道里面读取数据，
482  *                 然后把这些数据送入缓冲区。
483  * @param - irq  : 中断号
484  * @param - p    : iio_poll_func 结构体类型指针变量
485  */
486 irqreturn_t icm20608_trigger_handler(int irq, void *p)
487 {
488     int ret = 0;
489     struct iio_poll_func *pf = p;
490     struct iio_dev *indio_dev = pf->indio_dev;
491     struct icm20608_dev *dev = iio_priv(indio_dev);
492     u8 data[ICM20608_OUTPUT_DATA_SIZE];
493     int int_status = 0;
494
495     mutex_lock(&dev->lock);
496
497     /* 判断数据是否准备就绪 */
498     ret = regmap_read(dev->regmap, ICM20_INT_STATUS, &int_status);
499     if (!(int_status & ICM20608_BIT_RAW_DATA_RDY_INT)) {
500         printk("spurious interrupt with status 0x%x\n", int_status);
501         goto end_session;
502     }
503
504     ret = regmap_bulk_read(dev->regmap, ICM20_ACCEL_XOUT_H,
                               data, 14); /* 一次读取 14 字节的数据 */
505     if (ret)
506             goto end_session;
507     iio_push_to_buffers_with_timestamp(indio_dev, data,
                                           pf->timestamp);
508
509 end_session:
510     mutex_unlock(&dev->lock);
511     iio_trigger_notify_done(indio_dev->trig);
512     return IRQ_HANDLED;
513 }
```

```
514
515 /*
516  * @description : 中断服务函数，对于 iio 触发器来说，一般在此函数
517  *                里面直接调用 iio_trigger_poll
518  */
519 irqreturn_t iio_trigger_generic_data_rdy_poll(int irq,
                                                  void *private)
520 {
521     iio_trigger_poll(private);
522     return IRQ_HANDLED;
523 }
524
525 /*
526   * @description      : 触发器开关，一般在此函数里面实现设备的打开和关闭操作
527   *                     比如使能/关闭中断等
528   * @param - trig     : 触发器
529   * @param - state    : 状态，true，打开；flase，关闭。
530   * @return           : 0,成功；其他值,失败
531   *
532   */
533 static int icm20608_trigger_set_state(struct iio_trigger *trig,
534                       bool state)
535 {
536     int ret;
537     struct iio_dev *indio_dev = iio_trigger_get_drvdata(trig);
538     struct icm20608_dev *dev = iio_priv(indio_dev);
539
540     mutex_lock(&dev->lock);
541     if (state) {
542         ret = regmap_write(dev->regmap, ICM20_INT_ENABLE, 0x01);
543     } else {
544         ret = regmap_write(dev->regmap, ICM20_INT_ENABLE, 0x00);
545     }
546     mutex_unlock(&dev->lock);
547     return ret;
548 }
549
550 /*
551  * 触发器操作函数集
552  */
553 static const struct iio_trigger_ops icm20608_trigger_ops = {
554     .set_trigger_state = &icm20608_trigger_set_state,
555 };
```

```
556
557 /*
558  * @description  : spi 驱动的 probe 函数，当驱动与
559  *                 设备匹配以后此函数就会执行
560  * @param - spi  : spi 设备
561  * @return       : 0,成功；其他值，失败
562  */
563 static int icm20608_probe(struct spi_device *spi)
564 {
565     int ret;
566     struct icm20608_dev *dev;
567     struct iio_dev *indio_dev;
......
592
593     /* 4、iio_dev 的其他成员变量 */
594     indio_dev->dev.parent = &spi->dev;
595     indio_dev->info = &icm20608_info;
596     indio_dev->name = ICM20608_NAME;
597     indio_dev->modes = INDIO_DIRECT_MODE;
598     indio_dev->channels = icm20608_channels;
599     indio_dev->num_channels = ARRAY_SIZE(icm20608_channels);
600     indio_dev->available_scan_masks = icm20608_scan_masks;
601
602     /* 5、触发缓冲区设置 */
603     ret = devm_iio_triggered_buffer_setup(&spi->dev, indio_dev,
604                         iio_pollfunc_store_time,
605                         icm20608_trigger_handler,
606                         NULL);
607
608     /* 6、申请 trigger,并初始化 */
609     dev->trig = devm_iio_trigger_alloc(&indio_dev->dev,
610                       "%s-dev%d",
611                       indio_dev->name,
612                       indio_dev->id);
613     if (!dev->trig){
614         ret = -ENOMEM;
615         goto err_iio_trriger_alloc;
616     }
617
618     dev->trig->dev.parent = regmap_get_device(dev->regmap);
619     dev->trig->ops = &icm20608_trigger_ops;
620     iio_trigger_set_drvdata(dev->trig, indio_dev);
621
```

```
622     /* 7、向内核注册触发器 */
623     ret = devm_iio_trigger_register(&indio_dev->dev, dev->trig);
624     indio_dev->trig = iio_trigger_get(dev->trig);
625
626     /* 8、ICM20608 中断初始化 */
627     ret = devm_request_irq(&indio_dev->dev, spi->irq,
628                 &iio_trigger_generic_data_rdy_poll,
629                 IRQF_TRIGGER_HIGH,
630                 "inv_mpu",
631                 dev->trig);
632
633     /* 9、注册 iio_dev */
634     ret = iio_device_register(indio_dev);
635     if (ret < 0) {
636         dev_err(&spi->dev, "iio_device_register failed\n");
637         goto err_iio_register;
638     }
......
654 }
```

第 115~124 行，icm20608_scan_masks 是扫描掩码，用于初始化 iio_dev 的 available_scan_masks 成员变量。

第 486~513 行，icm20608_trigger_handler 函数的重要工作就是将数据推送到缓冲区中，第 498~506 行判断 ICM20608 数据是否准备就绪，如果准备就绪的话就直接读取从 0X3B 开始的 14 个寄存器的数据，也就是陀螺仪、加速度计、温度原始数据值。最后在第 507 行调用 iio_push_to_buffers_with_timestamp 函数将获取到的数据推送到缓冲区中！

第 519~523 行，iio_trigger_generic_data_rdy_poll 函数是 ICM20608 中断服务函数，此函数很简单，直接调用 iio_trigger_poll。

第 533~548 行，icm20608_trigger_set_state 函数用于设置触发状态，其实就是设置 ICM20608 的中断开关。此函数用于初始化 iio_trigger_ops 的 set_trigger_state 成员变量。

第 553~555 行，icm20608_trigger_ops 为触发器的操作函数集，初始化触发器的时候需要用到。

最后重点在 icm20608_probe 函数，因为触发器的申请、初始化和注册、缓冲区的建立都是在此函数中完成的。第 603 行调用 devm_iio_triggered_buffer_setup 函数设置触发缓冲区，上半部函数为 iio_pollfunc_store_time，下半部为 icm20608_trigger_handler。第 609 行调用 devm_iio_trigger_alloc 函数申请触发器，如果申请成功就初始化触发器，最终在第 623 行调用 devm_iio_trigger_register 函数向内核注册触发器。

第 627~631 行，初始化 ICM20608 中断，中断服务函数为 iio_trigger_generic_data_rdy_poll。

56.7 触发缓冲测试

驱动编写好以后重新编译加载，如果驱动运行正常，那么生成/dev/iio\:deviceX(X=0,1,2…)文件，如图 56.7.1 所示：

```
[root@ATK-stm32mp1]:/dev$ ls /dev/iio\:device0 -l
crw-rw----    1 root     root      249,   0 Jan  1 00:04 /dev/iio:device0
[root@ATK-stm32mp1]:/dev$
```

图 56.7.1 设备文件

图 56.7.1 中的/dev/iio\:device0 就是 ICM20608，我们只需要做简单的配置，就可以通过读取这个文件得到缓冲区中的数据。在读取数据之前，我们要先了解一下其他的配置文件。

56.7.1 缓冲区接口文件

缓冲区接口目录路径为：

/sys/bus/iio/devices/iio:device0/buffer

如图 56.7.1.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cd iio:device0/
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0$ ls
buffer                  in_anglvel_scale        in_temp_scale
dev                     in_anglvel_x_calibbias  name
in_accel_scale          in_anglvel_x_raw        of_node
in_accel_x_calibbias    in_anglvel_y_calibbias  power
in_accel_x_raw          in_anglvel_y_raw        scan_elements
in_accel_y_calibbias    in_anglvel_z_calibbias  subsystem
in_accel_y_raw          in_anglvel_z_raw        trigger
in_accel_z_calibbias    in_temp_offset          uevent
```

缓冲区

图 56.7.1.1 缓冲区目录

进入到图 56.7.1.1 中的 buffer 目录，会有一些属性文件，如图 56.7.1.2 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ ls iio:device0/buffer/
data_available  enable          length          watermark
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.7.1.2 缓冲区属性文件

data_available：指示数据是否有效，为 1 时有效，为 0 时无效。

enable：使能缓冲区，写入 1 使能缓冲区，写 0 关闭缓冲区

length：缓冲区大小，也就是可以存储数据的数量。

watermark：阻塞读取的时候只有数据量大于 watermark 的时候才能读取，非阻塞读取的时候不受 watermark 影响。

我们可以指定哪些通道的数据可以被推送到缓冲区，也就是开启指定的扫描元素。/sys/bus/iio/devices/iio:device0/scan_elements 目录下就是 ICM20608 的扫描元素，如图 56.7.1.3 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ ls iio:device0/scan_elements/
in_accel_x_en       in_accel_z_index    in_anglvel_y_type
in_accel_x_index    in_accel_z_type     in_anglvel_z_en
in_accel_x_type     in_anglvel_x_en     in_anglvel_z_index
in_accel_y_en       in_anglvel_x_index  in_anglvel_z_type
in_accel_y_index    in_anglvel_x_type   in_temp_en
in_accel_y_type     in_anglvel_y_en     in_temp_index
in_accel_z_en       in_anglvel_y_index  in_temp_type
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 56.7.1.3 扫描元素

图 56.7.1.3 中一共有 7 组扫描元素，分别为加速度计的 X、Y、Z 轴，陀螺仪的 X、Y、Z 轴，温度。每个通道有 3 个文件，我们就以加速度计的 X 轴为例：

in_accel_x_en：从后缀的“en”可以看出，这个文件是加速度计 X 轴通道使能文件，写入 1 使能加速度计 X 轴

in_accel_x_index：通道索引值，这个索引值就是我们在驱动中配置通道的时候设置的索引值，比如驱动中加速度 X 轴索引值为 INV_ICM20608_SCAN_ACCL_X=0，查看 in_accel_x_index 文件内容，如图 56.7.1.4 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/scan_elements$ cat in_accel_x_index
0
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/scan_elements$
```

图 56.7.1.4 in_accel_x_index 值

从图 56.7.1.4 可以看出，in_accel_x_index 的值为 0。

in_accel_x_type：此文件描述数据存储格式，这个存储格式就是我们在驱动程序中配置通道的时候设置的存储格式。type 文件内容格式为[be|le]:[s|u]bits/storagebitsXrepeat[>>shift]，各段的含义如下：

be 或 le：be 表示大端、le 表示小端。

s 或 u：s 表示有符号类型，u 表示无符号类型。

bits：有效数据位数。

storagebits：存储位数。

repeat：位/存储重复数量。

shift：屏蔽掉未使用的位之前应该移动的位数，当有效位数等于存储位数的时候 shift 就是 0。

查看 in_accel_x_type 文件内容，如图 56.7.1.5 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/scan_elements$ cat in_accel_x_type
be:s16/16>>0
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/scan_elements$
```

图 56.7.1.5 in_accel_x_type 文件内容

从图 56.7.1.5 可以看出，in_accel_x_type 文件内容为“be:s16/16>>0”，也就是大端模式、有符号类型、有效位数 16 位，存储位数 16 位，右移 0。

56.7.2 触发器接口文件

1、触发器目录

我们在驱动里面创建触发器以后就会有/sys/bus/iio/devices/triggerX(X=0,1,2……)目录，如图 56.7.2.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ ls
iio:device0        iio_sysfs_trigger  trigger0
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

触发器目录

图 56.7.2.1 触发器目录

图 56.7.2.1 中的 trigger0 就是 ICM20608 的中断触发器目录，进入此目录中，目录下的文件如图 56.7.2.2 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/trigger0$ ls
name       power      subsystem  uevent
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/trigger0$
```

图 56.7.2.2 trigger0 触发器目录文件

图 56.7.2.2 中的“name”文件就是此触发器的名字，如下：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/trigger0$ cat name
icm20608-dev0
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/trigger0$
```

触发器名字

图 56.7.2.3 触发器名字

可以看出，此时触发器的名字为“icm20608-dev0”。

2、IIO 设备所使用的触发器

一个 IIO 设备它可以使用多个触发器，只是本章教程我们只给 ICM20608 创建了一个中断触发器，大家如果看过其他资料的话，会发现还有什么定时器触发器、sysfs 触发器等，这些触发器也可以赋给 ICM20608。所以 IIO 设备也会有一个目录来保存自己当前正在使用的触发器，进入 ICM20608 对应的 iio:device0 这个目录中，也就是 ICM20608 的 IIO 设备目录里面，如图 56.7.2.4 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0$ ls
buffer                  in_anglvel_scale        in_temp_scale
dev                     in_anglvel_x_calibbias  name
in_accel_scale          in_anglvel_x_raw        of_node
in_accel_x_calibbias    in_anglvel_y_calibbias  power
in_accel_x_raw          in_anglvel_y_raw        scan_elements
in_accel_y_calibbias    in_anglvel_z_calibbias  subsystem
in_accel_y_raw          in_anglvel_z_raw        trigger   → 设备所使用的触发器
in_accel_z_calibbias    in_temp_offset          uevent
in_accel_z_raw          in_temp_raw
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0$
```

图 56.7.2.4 iio:device0 目录文件

图 56.7.2.4 中也有个名为“trigger”的目录，这个目录就是 ICM20608 正在使用的触发器，此目录下面的文件如图 56.7.2.5 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/trigger$ ls
current_trigger
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/trigger$
```

图 56.7.2.5 trigger 目录

可以看出，只有一个名为“current_trigger”的文件，此文件保存着当前 ICM20608 正在使用的触发器，使用“cat”命令查看其内容，如图 56.7.2.6 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/trigger$ cat current_trigger
icm20608-dev0
[root@ATK-stm32mp1]:/sys/devices/platform/soc/44004000.spi/spi_master/spi0/spi0.0/iio:device0/trigger$
```

图 56.7.2.6 当前正在使用的触发器

从图 56.7.2.6 可以看出，此时 ICM20608 正在使用的触发器是“icm20608-dev0”，也就是我们在驱动里面创建的中断触发器。如果要使用其他的触发器，只需要将对应的触发器名字写入到 current_trigger 文件中即可，比如要将当前触发器改为“test_trigger”，命令如下：

```
echo test_trigger > current_trigger
```

56.7.3 缓冲区数据读取

读取数据之前我们需要配置缓冲区和触发器，首先使能各个扫描元素，也就是通道，输入如下命令：

```
echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_accel_x_en
echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_accel_y_en
echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_accel_z_en
echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_anglvel_x_en
echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_anglvel_y_en
echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_anglvel_z_en
echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_temp_en
```

接下来设置 ICM20608 所使用的触发器，虽然驱动已经默认设置 ICM20608 触发器使用 icm20608-dev0，但是为了完整，流程还是要走一下的，命令如下：

```
echo icm20608-dev0 > /sys/bus/iio/devices/iio:device0/trigger/current_trigger
```

最后就是设置缓冲区长度，并且开启缓冲区，命令如下：

```
echo 14 > /sys/bus/iio/devices/iio:device0/buffer/length
echo 1 > /sys/bus/iio/devices/iio:device0/buffer/enable
```

配置完成以后就可以读取/dev/iio\:device0 文件，得到缓冲区中的数据，这里我们可以使用 cat 命令查看一下原始值，命令如下：

```
hexdump /dev/iio\:device0
```

结果如图 56.7.3.1 所示：

```
[root@ATK-stm32mp1]:/dev$ hexdump /dev/iio\:device0
0000000 ffff dbff ff07 d514 d7ff 0300 fbff ffff
0000010 dbff ff07 d414 d7ff 0400 fbff ffff dcff
0000020 ff07 d314 d7ff 0400 fbff feff dbff ff07
0000030 d214 d7ff 0300 fbff fdff daff 0108 d214
0000040 d7ff 0300 faff feff dbff 0208 d314 d7ff
0000050 0300 faff feff dcff 0408 d414 d7ff 0400
0000060 faff feff dcff 0408 d414 d7ff 0400 faff
```

图 56.7.3.1 缓冲区数据

图 56.7.3.1 中的内容就是 ICM20608 缓冲区数据，现在看起来杂乱无章，接下来我们就编写测试 APP，在 APP 里面处理这些数据。

56.7.4 触发缓冲测试 APP 编写测试

1、触发缓冲区 APP 编写

新建名为 icm20608_triggerAPP.c 文件，输入如下内容：

示例代码 56.7.4.1 icm20608_triggerAPP.c 文件内容

```
/***************************************************************
Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
文件名       : icm20608_triggerApp.c
作者         : 左忠凯
版本         : V1.0
描述         : icm20608 设备 iio 框架触发缓冲测试程序。
其他         : 无
使用方法     : ./icm20608_triggerApp
论坛         : www.openedv.com
日志         : 初版 V1.0 2021/8/21 左忠凯创建
***************************************************************/
1  #include "stdio.h"
2  #include "unistd.h"
3  #include "sys/types.h"
4  #include "sys/stat.h"
5  #include "sys/ioctl.h"
6  #include "fcntl.h"
7  #include "stdlib.h"
8  #include "string.h"
```

```
9   #include <poll.h>
10  #include <sys/select.h>
11  #include <sys/time.h>
12  #include <signal.h>
13  #include <fcntl.h>
14  #include <errno.h>
15
16  /*
17   * icm20608 数据设备结构体
18   */
19  struct icm20608_dev {
20      unsigned char data[14];
21      int accel_x_calibbias, accel_y_calibbias, accel_z_calibbias;
22      int accel_x_raw, accel_y_raw, accel_z_raw;
23
24      int gyro_x_calibbias, gyro_y_calibbias, gyro_z_calibbias;
25      int gyro_x_raw, gyro_y_raw, gyro_z_raw;
26
27      int temp_offset, temp_raw;
28
29      float accel_scale, gyro_scale, temp_scale;
30
31      float gyro_x_act, gyro_y_act, gyro_z_act;
32      float accel_x_act, accel_y_act, accel_z_act;
33      float temp_act;
34  };
35
36  struct icm20608_dev icm20608;
37
38   /*
39   * @description   : 对 icm20608 相关触发进行设置
40   * @param         : 无
41   * @return        : 无
42   */
43  void icm20608_trigger_set(void)
44  {
45      system("echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/
            in_accel_x_en");
46      system("echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/
            in_accel_y_en");
47      system("echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/
            in_accel_z_en");
48      system("echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/
```

```
            in_anglvel_x_en");
49      system("echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/
            in_anglvel_y_en");
50      system("echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/
            in_anglvel_z_en");
51      system("echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/
            in_temp_en");
52
53      system("echo icm20608-dev0 > /sys/bus/iio/devices/iio:device0/
            trigger/current_trigger");
54      system("echo 14 > /sys/bus/iio/devices/iio:device0/buffer/
           length");
55      system("echo 1 > /sys/bus/iio/devices/iio:device0/buffer/
           enable");
56  }
57
58   /*
59   * @description       : 读取指定文件内容
60   * @param - filename : 要读取的文件路径
61   * @param - str       : 读取到的文件字符串
62   * @return            : 0 成功;其他 失败
63   */
64  static int file_data_read(char *filename, char *str)
65  {
66      int ret = 0;
67      FILE *data_stream;
68
69      data_stream = fopen(filename, "r"); /* 只读打开 */
70      if(data_stream == NULL) {
71          printf("can't open file %s\r\n", filename);
72          return -1;
73      }
74
75      ret = fscanf(data_stream, "%s", str);
76      if(!ret) {
77          printf("file read error!\r\n");
78      } else if(ret == EOF) {
79          /* 读到文件末尾的话将文件指针重新调整到文件头 */
80          fseek(data_stream, 0, SEEK_SET);
81      }
82      fclose(data_stream);    /* 关闭文件 */
83      return 0;
84  }
```

```
85
86  /*
87   * @description   : 读取数据
88   * @param - fd    : 文件描述符
89   * @return        : 0 成功;其他 失败
90   */
91  int icm20608_read(int fd, struct icm20608_dev *dev)
92  {
93      int ret = 0;
94      char str[50];
95      int i = 0;
96
97      file_data_read("/sys/bus/iio/devices/iio:device0/
                    in_accel_scale", str);
98      dev->accel_scale = atof(str);
99
100     file_data_read("/sys/bus/iio/devices/iio:device0/
                     in_anglvel_scale", str);
101     dev->gyro_scale = atof(str);
102
103     file_data_read("/sys/bus/iio/devices/iio:device0/
                     in_temp_scale", str);
104     dev->temp_scale = atof(str);
105
106     ret = read(fd, dev->data, 14);
107
108     dev->accel_z_raw    = (signed short)((dev->data[4] << 8) |
                              dev->data[5]);
109     dev->accel_x_raw    = (signed short)((dev->data[0] << 8) |
                              dev->data[1]);
110     dev->accel_y_raw    = (signed short)((dev->data[2] << 8) |
                              dev->data[3]);
111     dev->accel_z_raw    = (signed short)((dev->data[4] << 8) |
                              dev->data[5]);
112     dev->temp_raw       = (signed short)((dev->data[6] << 8) |
                              dev->data[7]);
113     dev->gyro_x_raw     = (signed short)((dev->data[8] << 8) |
                              dev->data[9]);
114     dev->gyro_y_raw     = (signed short)((dev->data[10] << 8) |
                              dev->data[11]);
115     dev->gyro_z_raw     = (signed short)((dev->data[12] << 8) |
                              dev->data[13]);
116
```

```
117     /* 转换为实际数值 */
118     dev->accel_x_act = dev->accel_x_raw * dev->accel_scale;
119     dev->accel_y_act = dev->accel_y_raw * dev->accel_scale;
120     dev->accel_z_act = dev->accel_z_raw * dev->accel_scale;
121 
122     dev->gyro_x_act = dev->gyro_x_raw * dev->gyro_scale;
123     dev->gyro_y_act = dev->gyro_y_raw * dev->gyro_scale;
124     dev->gyro_z_act = dev->gyro_z_raw * dev->gyro_scale;
125 
126     dev->temp_act = ((dev->temp_raw - dev->temp_offset) /
dev->temp_scale) + 25;
127     return ret;
128 }
129 
130 /*
131  * @description   : main 主程序
132  * @param - argc  : argv 数组元素个数
133  * @param - argv  : 具体参数
134  * @return        : 0 成功;其他 失败
135  */
136 int main(int argc, char *argv[])
137 {
138     int fd;
139     int ret = 0;
140 
141     /* 判断传参个数是否正确 */
142     if(2 != argc) {
143         printf("Usage:\n"
144                "\t./icm20608_triggerApp /dev/iio:device0\n"
145               );
146         return -1;
147     }
148 
149     icm20608_trigger_set(); /* 配置好触发缓冲区相关设置 */
150 
151     /* 打开设备 */
152     fd = open(argv[1], O_RDONLY);
153     if(0 > fd) {
154         printf("ERROR: %s file open failed!\n", argv[1]);
155         return -1;
156     }
157 
158     /* 循环轮训读取按键数据 */
```

```
159     while(1) {
160         icm20608_read(fd, &icm20608);
161         if(ret == 0) {          /* 数据读取成功 */
162             printf("\r\n 原始值:\r\n");
163             printf("gx = %d, gy = %d, gz = %d\r\n",
                        icm20608.gyro_x_raw,
                        icm20608.gyro_y_raw,
                        icm20608.gyro_z_raw);
164             printf("ax = %d, ay = %d, az = %d\r\n",
                        icm20608.accel_x_raw,
                        icm20608.accel_y_raw,
                        icm20608.accel_z_raw);
165             printf("temp = %d\r\n", icm20608.temp_raw);
166             printf("实际值:");
167             printf("act gx = %.2f°/S, act gy = %.2f°/S,
                        act gz = %.2f°/S\r\n", icm20608.gyro_x_act,
                        icm20608.gyro_y_act, icm20608.gyro_z_act);
168             printf("act ax = %.2fg, act ay = %.2fg,
                       act az = %.2fg\r\n", icm20608.accel_x_act,
                      icm20608.accel_y_act, icm20608.accel_z_act);
169             printf("act temp = %.2f°C\r\n", icm20608.temp_act);
170         }
171         usleep(100000); /*100ms */
172     }
173 }
```

第 43~56 行，icm20608_trigger_set 函数用于配置触发器和缓冲区，直接使用 system 函数来写入 shell 命令。

第 64~84 行，file_data_read 函数用于读取文件流，主要用于读取 ICM20608 的加速度计、陀螺仪、温度的分辨率等参数。

第 102~139 行，icm20608_read 函数用于读取 ICM20608 缓冲区数据，第 117 行使用 read 函数读取/dev/iio\:device0 文件内容，第 119~126 行对读取到的内容进行解析，提取出加速度计、陀螺仪、温度的原始值。第 129~137 行计算得到具体的数值。

第 147~184 行，main 函数，第 160 行调用 icm20608_trigger_set 函数配置触发器和缓冲区，最后在 while 循环中调用 icm20608_read 函数读取并处理缓冲区数据，最终打印出读取到的数据。

2、测试 APP

最后输入如下命令编译 icm20608_triggerAPP.c 文件：

```
arm-none-linux-gnueabihf-gcc icm20608_triggerAPP.c -o icm20608_triggerAPP
```

编译完成以后将 cm20608_triggerAPP 发送到开发板根文件系统中，加载驱动文件，输入如下命令运行测试 APP：

```
./icm20608_triggerAPP  /dev/iio\:device0
```

如果测试 APP 运行正常，那么就会打印出 ICM20608 的陀螺仪、加速度计、温度的原始值以及实际值，如图 56.7.4.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./icm20608_triggerAPP  /dev/iio\:device
0

原始值:
gx = -47, gy = 4, gz = -5
ax = -3, ay = -37, az = 2048
temp = 5249
实际值:act gx = -2.87°/S, act gy = 0.24°/S, act gz = -0.31°/S
act ax = -0.00g, act ay = -0.02g, act az = 1.00g
act temp = 41.06°C

原始值:
gx = -47, gy = 4, gz = -5
ax = -3, ay = -37, az = 2050
temp = 5250
实际值:act gx = -2.87°/S, act gy = 0.24°/S, act gz = -0.31°/S
act ax = -0.00g, act ay = -0.02g, act az = 1.00g
act temp = 41.06°C
```

图 56.7.4.1 APP 运行结果

从图 56.7.4.1 可以看出，APP 运行成功，驱动也工作成功。

另外，我们也编写 AP3216C 这个光传感器的对应的 IIO 驱动，但是由于 AP3216C 没有数据准备就绪中断，因此 AP3216C 的 IIO 驱动没有实现触发缓冲功能。AP3216C 的 IIO 驱动程序已经放到了开发板光盘中，路径为：开发板光盘->01、程序源码->02、Linux 驱动例程->31_iio->iic。

第五十七章 Linux ADC 驱动实验

上一章我们讲解了如何给 ICM20608 编写 IIO 驱动，ICM20608 本质就是 ADC，因此纯粹的 ADC 驱动也是 IIO 驱动框架的。本章我们就来学习一下如何使用 STM32MP1 内部的 ADC，并且在学习巩固一下 IIO 驱动。

57.1 ADC 简介

57.1.1 什么是 ADC

ADC，Analog to Digital Converter 的缩写，中文名称模数转换器。它可以将外部的模拟信号转化成数字信号。对于 GPIO 口来说高于某个电压值，它读出来的只有高电平，低于就是低电平。假如我想知道具体的电压数值就要借助于 ADC 的帮助，它可以将一个范围内的电压精确的读取出来。假设我们的 GPIO 口只要高于 1.7V 的都认为是高电平，例如，比如某个 IO 口上外接了一个设备它能提供 0-2V 的电压变化，我们在这个 IO 口上使用 GPIO 模式去读取的话我们只能获得 0 和 1 两个数据，但是我们使用 ADC 模式去读取就可以获得 0-2V 之间连续变化的数值。

ADC 有几个比较重要的参数：

测量范围：测量范围对于 ADC 来说就好比尺子的量程，ADC 测量范围决定了你外接的设备其信号输出电压范围，不能超过 ADC 的测量范围。如果所使用的外部传感器输出的电压信号范围和所使用的 ADC 测量范围不符合，那么就需要自行设计相关电压转换电路。

分辨率：就是尺子上的能量出来的最小测量刻度，例如我们常用的厘米尺它的最小刻度就是 1 毫米，表示最小测量精度就是 1 毫米。假如 ADC 的测量范围为 0-5V，分辨率设置为 12 位，那么我们能测出来的最小电压就是 5V 除以 2 的 12 次方，也就是 5/4096=0.00122V。很明显，分辨率越高，采集到的信号越精确，所以分辨率是衡量 ADC 的一个重要指标。

精度：是影响结果准确度的因素之一，比如在厘米尺上我们能测量出大概多少毫米的尺度但是毫米后一点点我们却不能准确的量出。经过计算我们 ADC 在 12 位分辨率下的最小测量值是 0.00122V 但是我们 ADC 的精度最高只能到 11 位也就是 0.00244V。也就是 ADC 测量出 0.00244V 的结果是要比 0.00122V 要可靠，也更准确。

采样时间：当 ADC 在某时刻采集外部电压信号的时候，此时外部的信号应该保持不变，但实际上外部的信号是不停变化的。所以在 ADC 内部有一个保持电路，保持某一时刻的外部信号，这样 ADC 就可以稳定采集了，保持这个信号的时间就是采样时间。

采样率：也就是在一秒的时间内采集多少次。很明显，采样率越高越好，当采样率不够的时候可能会丢失部分信息，所以 ADC 采样率是衡量 ADC 性能的另一个重要指标

总之，只要是需要模拟信号转为数字信号的场合，那么肯定要用到 ADC。很多数字传感器内部会集成 ADC，传感器内部使用 ADC 来处理原始的模拟信号，最终给用户输出数字信号。

57.1.2 STM32MP157 ADC 简介

STM32MP157 有两个 ADC：ADC1 和 ADC2，ADC1 和 ADC2 紧密耦合，可在双重模式下运行(ADC1 为主器件)。每个 ADC 由一个 16 位逐次逼近模数转换器组成，每个 ADC 有 20 个通道，每个通道支持单次、连续、扫描或不连续采样模式。转换结果存储在一个左对齐或右对齐的 32 位数据寄存器中。ADC 主要特性如下：

1)、多达 2 个 ADC，可在双重模式下运行。

2)、可以配置为 16、14、12、10 或 8 位分辨率。

3)、自校准。

4)、可独立配置各通道采样时间。

……

关于 STM32MP157 ADC 更多详细的介绍，请参考《STM32MP1 参考手册》中的 ADC 章节。

57.2 ADC 驱动源码简析

57.2.1 设备树下的 ADC 节点

STM32MP157 有 2 个 ADC，因此对应 2 个 ADC 控制器，所以在设备树里就有 2 个 ADC 控制器节点。这 2 个 ADC 的设备树节点内容都是一样的，除了 reg 属性不同(毕竟不同的控制器，其地址范围不同)。本章实验我们使用 PA5 这个引脚来完成 ADC 实验，而 PA5 就是 ADC1_INP19 通道引脚，所以这里我们就以 ADC1 为例进行讲解，stm32mp151.dtsi 文件中的 adc 节点信息如下：

示例代码 57.2.1.1 adc 节点内容

```
1  adc: adc@48003000 {
2      compatible = "st,stm32mp1-adc-core";
3      reg = <0x48003000 0x400>;
4      interrupts = <GIC_SPI 18 IRQ_TYPE_LEVEL_HIGH>,
5             <GIC_SPI 90 IRQ_TYPE_LEVEL_HIGH>;
6      clocks = <&rcc ADC12>, <&rcc ADC12_K>;
7      clock-names = "bus", "adc";
8      interrupt-controller;
9      st,syscfg = <&syscfg>;
10     #interrupt-cells = <1>;
11     #address-cells = <1>;
12     #size-cells = <0>;
13     status = "disabled";
14
15     adc1: adc@0 {
16         compatible = "st,stm32mp1-adc";
17         #io-channel-cells = <1>;
18         reg = <0x0>;
19         interrupt-parent = <&adc>;
20         interrupts = <0>;
21         dmas = <&dmamux1 9 0x400 0x80000001>;
22         dma-names = "rx";
23         status = "disabled";
24     };
25
26     adc2: adc@100 {
27         compatible = "st,stm32mp1-adc";
28         #io-channel-cells = <1>;
29         reg = <0x100>;
30         interrupt-parent = <&adc>;
31         interrupts = <1>;
32         dmas = <&dmamux1 10 0x400 0x80000001>;
33         dma-names = "rx";
```

```
34          status = "disabled";
35      };
36  };
```

第 2 行，compatible 属性值为“st,stm32mp1-adc-core”，所以在整个 Linux 源码里面搜索这个字符串即可找到 STM32MP157 的 ADC 驱动核心文件，这个文件就是 drivers/iio/adc/stm32-adc-core.c。

第 16、27 行，compatible 属性值“st,stm32mp1-adc”，搜索这个字符串，可以找到 ADC 驱动文件，这个文件就是 drivers/iio/adc/stm32-adc.c。

关于 STM32MP157 的 ADC 节点更为详细的信息请参考对应的绑定文档：Documentation/devicetree/bindings/iio/adc/st,stm32-adc.txt。接下来我们简单分析一下绑定文档，后面我们需要根据绑定文档修改设备树，使能 ADC 对应的通道。

ADC 首先需要一个根节点，adc 根节点属性如下：

①、必要属性

- **compatible**：兼容性属性，必须的，可以设置为“st,stm32mp1-adc-core”。
- **reg**：ADC 控制器寄存器信息。
- **interrupts**：中断属性，ADC1 和 ADC2 各对应一个中断信息。
- **clocks**：时钟属性。
- **clock-names**：时钟名字，可选“adc”或“bus”。
- **interrupt-controller**：中断控制器。
- **vdda-supply**：此属性对应 vdda 输入模拟电压句柄。
- **vref-supply**：此属性对应 vref 参考电压句柄。
- **interrupt-cells**：设置为 1。
- **address-cells**：设置为 1。
- **size-cells**：设置为 0。

②、可选属性：

- ：pinctrl 引脚配置信息。
- **booster-supply**：嵌入式 booster 调节器句柄。
- **vdd-supply**：vdd 输入电压句柄。
- **st,syscfg**：系统配置控制器句柄。
- **st,max-clk-rate-hz**：最大时钟。

STM32MP157 有两个 ADC，每个 ADC 对应一个子节点，ADC 子节点相关属性如下：

①、必要属性

- **compatible**：兼容性属性，必须的，可以设置为“st,stm32mp1-adc”。
- **reg**：不同 ADC 控制器寄存器地址偏移信息。
- **interrupts**：中断线信息，adc@0 为 0，adc@100 为 1。
- **st,adc-channels**：ADC 通道信息，可以设置 0~19，分别对应 20 个通道。
- **st,adc-diff-channels**：ADC 差分通道信息，如果使用差分 ADC 功能的话。
- **io-channel-cells**：设置为 1。

②、可选属性：

- **dmas**：DMA 通道句柄。
- **dma-names**：dma 名字，必须设置成“rx”。

- assigned-resolution-bits：ADC 分辨率，可以设置为 8、10、12,、14 或 16。

- st,min-sample-time-nsecs：最小采样时间，单位 ns。

关于设备树就讲解到这里。

57.2.2 ADC 驱动源码分析

STM32MP157 ADC 驱动文件有两个：stm32-adc-core.c 和 stm32-adc.c。stm32-adc-core.c 是 ADC 核心层，主要用于 ADC 电源等初始化，我们需要重点关注的是 stm32-adc.c 这个文件。stm32-adc.c 主体框架是 platform，配合 IIO 驱动框架实现 ADC 驱动。

1、stm32_adc 结构体

ST 自己将 ADC 外设抽象成了结构体，stm32_adc 就相当于自定义的设备结构体。stm32_adc 结构体贯穿于整个驱动文件，结构体内容如下：

示例代码 57.2.2.1 stm32_adc 结构体

```
196 struct stm32_adc {
197     struct     stm32_adc_common *common;    /* ADC 通用数据          */
198     u32        offset;                      /* ADC 控制器偏移地址    */
199     const struct stm32_adc_cfg  *cfg;       /* 配置信息              */
200     struct completion   completion;         /* 单次转换完成量        */
201     u16        buffer[STM32_ADC_MAX_SQ];    /* 数据缓冲区            */
202     struct clk        *clk;                 /* 时钟                  */
203     int               irq;                  /* 中断                  */
204     spinlock_t        lock;                 /* 自旋锁                */
206     unsigned int      num_conv;             /* 扫描转换编号          */
205     unsigned int      bufi;                 /* 数据缓冲区索引        */
207     u32               res;                  /* 数据分辨率            */
208     u32               trigger_polarity;     /* 外部触发优先级        */
209     struct dma_chan   *dma_chan;            /* dma 通道              */
210     u8                *rx_buf;              /* dma 接收缓冲区        */
211     dma_addr_t        rx_dma_buf;           /* dma 接收缓冲总线地址  */
212     unsigned int      rx_buf_sz;            /* dma 接收缓冲大小      */
213     u32               difsel;               /* 单次结束/差分掩码     */
214     u32               pcsel;                /* 预选通道掩码          */
215     u32               smpr_val[2];          /* 采样时间(smpr1 和 smpr2)*/
216     struct stm32_adc_calib  cal;            /* 校准值                */
217     char   chan_name[STM32_ADC_CH_MAX][STM32_ADC_CH_SZ];/* 通道名字 */
218 };
```

2、stm32_adc_probe 函数

接下来看一下 stm32_adc_probe 函数，内容如下(有省略)：

示例代码 57.2.2.2 stm32_adc_probe 函数

```
1  static int stm32_adc_probe(struct platform_device *pdev)
2  {
3      struct iio_dev *indio_dev;
4      struct device *dev = &pdev->dev;
```

```
5       irqreturn_t (*handler)(int irq, void *p) = NULL;
6       struct stm32_adc *adc;
7       int ret;
8
9       if (!pdev->dev.of_node)
10          return -ENODEV;
11
12      indio_dev = devm_iio_device_alloc(&pdev->dev, sizeof(*adc));
13      if (!indio_dev)
14          return -ENOMEM;
15
16      adc = iio_priv(indio_dev);
17      adc->common = dev_get_drvdata(pdev->dev.parent);
18      spin_lock_init(&adc->lock);
19      init_completion(&adc->completion);
20      adc->cfg = (const struct stm32_adc_cfg *)
21          of_match_device(dev->driver->of_match_table, dev)->data;
22
23      indio_dev->name = dev_name(&pdev->dev);
24      indio_dev->dev.parent = &pdev->dev;
25      indio_dev->dev.of_node = pdev->dev.of_node;
26      indio_dev->info = &stm32_adc_iio_info;
27      indio_dev->modes = INDIO_DIRECT_MODE | INDIO_HARDWARE_TRIGGERED;
28
29      platform_set_drvdata(pdev, adc);
30
31      ret = of_property_read_u32(pdev->dev.of_node, "reg",
                                   &adc->offset);
32      if (ret != 0) {
33          dev_err(&pdev->dev, "missing reg property\n");
34          return -EINVAL;
35      }
36
37      adc->irq = platform_get_irq(pdev, 0);
38      if (adc->irq < 0)
39          return adc->irq;
40
41      ret = devm_request_threaded_irq(&pdev->dev, adc->irq,
                                        stm32_adc_isr,
42                                      stm32_adc_threaded_isr,
43                                      0, pdev->name, adc);
44      if (ret) {
45          dev_err(&pdev->dev, "failed to request IRQ\n");
```

```
46          return ret;
47      }
48
49      adc->clk = devm_clk_get(&pdev->dev, NULL);
50      if (IS_ERR(adc->clk)) {
51          ret = PTR_ERR(adc->clk);
52          if (ret == -ENOENT && !adc->cfg->clk_required) {
53              adc->clk = NULL;
54          } else {
55              dev_err(&pdev->dev, "Can't get clock\n");
56              return ret;
57          }
58      }
59
60      ret = stm32_adc_of_get_resolution(indio_dev);
61      if (ret < 0)
62          return ret;
63
64      ret = stm32_adc_chan_of_init(indio_dev);
65      if (ret < 0)
66          return ret;
67
68      ret = stm32_adc_dma_request(indio_dev);
69      if (ret < 0)
70          return ret;
71
72      if (!adc->dma_chan)
73          handler = &stm32_adc_trigger_handler;
74
75      ret = iio_triggered_buffer_setup(indio_dev,
76                       &iio_pollfunc_store_time, handler,
77                       &stm32_adc_buffer_setup_ops);
78      if (ret) {
79          dev_err(&pdev->dev, "buffer setup failed\n");
80          goto err_dma_disable;
81      }
82
83      /* Get stm32-adc-core PM online */
84      pm_runtime_get_noresume(dev);
85      pm_runtime_set_active(dev);
86      pm_runtime_set_autosuspend_delay(dev,
                                         STM32_ADC_HW_STOP_DELAY_MS);
87      pm_runtime_use_autosuspend(dev);
```

```
88      pm_runtime_enable(dev);
89
90      ret = stm32_adc_hw_start(dev);
91      if (ret)
92          goto err_buffer_cleanup;
93
94      ret = iio_device_register(indio_dev);
95      if (ret) {
96          dev_err(&pdev->dev, "iio dev register failed\n");
97          goto err_hw_stop;
98      }
......
123     return ret;
124 }
```

第 12 行，调用 devm_iio_device_alloc 函数申请 iio_dev，这里也连 stm32_adc 内存一起申请了。

第 16 行，调用 iio_priv 函数从 iio_dev 里面的到 stm32_adc 首地址。

第 23~27 行，初始化 iio_dev，重点是第 26 行的 stm32_adc_iio_info，因为用户空间读取 ADC 数据最终就是由 stm32_adc_iio_info 来完成的。

第 37 行，调用 platform_get_irq 获取中断号。

第 41 行，调用 devm_request_threaded_irq 函数申请中断，这里使用的是中断线程化。

第 60 行，调用 stm32_adc_of_get_resolution 函数获取 ADC 的分辨率。

第 64 行，调用 stm32_adc_chan_of_init 函数初始化 ADC 通道。

第 68 行，调用 stm32_adc_dma_request 函数初始化 DMA。

第 75 行，调用 iio_triggered_buffer_setup 函数设置 IIO 触发缓冲区。

第 90 行，调用 stm32_adc_hw_start 函数开启 ADC。

第 94 行，调用 iio_device_register 函数向内核注册 iio_dev。

可以看出 stm32_adc_probe 函数核心就是初始化 ADC，然后建立 ADC 的 IIO 驱动框架。

3、stm32_adc_iio_info 结构体

stm32_adc_iio_info 结构体内容如下所示：

示例代码 57.2.2.3 stm32_adc_iio_info 结构体

```
1 static const struct iio_info stm32_adc_iio_info = {
2       .read_raw = stm32_adc_read_raw,
3       .validate_trigger = stm32_adc_validate_trigger,
4       .hwfifo_set_watermark = stm32_adc_set_watermark,
5       .update_scan_mode = stm32_adc_update_scan_mode,
6       .debugfs_reg_access = stm32_adc_debugfs_reg_access,
7       .of_xlate = stm32_adc_of_xlate,
8 };
```

我们重点来看一下第 2 行的 stm32_adc_read_raw 函数，因为此函数才是最终向用户空间发送 ADC 原始数据的，函数内容如下：

示例代码 57.2.2.4 stm32_adc_read_raw 函数

```
1  static int stm32_adc_read_raw(struct iio_dev *indio_dev,
2                struct iio_chan_spec const *chan,
3                int *val, int *val2, long mask)
4  {
5      struct stm32_adc *adc = iio_priv(indio_dev);
6      int ret;
7
8      switch (mask) {
9      case IIO_CHAN_INFO_RAW:
10         ret = iio_device_claim_direct_mode(indio_dev);
11         if (ret)
12             return ret;
13         if (chan->type == IIO_VOLTAGE)
14             ret = stm32_adc_single_conv(indio_dev, chan, val);
15         else
16             ret = -EINVAL;
17         iio_device_release_direct_mode(indio_dev);
18         return ret;
19
20     case IIO_CHAN_INFO_SCALE:
21         if (chan->differential) {
22             *val = adc->common->vref_mv * 2;
23             *val2 = chan->scan_type.realbits;
24         } else {
25             *val = adc->common->vref_mv;
26             *val2 = chan->scan_type.realbits;
27         }
28         return IIO_VAL_FRACTIONAL_LOG2;
29
30     case IIO_CHAN_INFO_OFFSET:
31         if (chan->differential)
32             /* ADC_full_scale / 2 */
33             *val = -((1 << chan->scan_type.realbits) / 2);
34         else
35             *val = 0;
36         return IIO_VAL_INT;
37
38     default:
39         return -EINVAL;
40     }
41 }
```

第 9~18 行，读取 ADC 原始数据值，第 18 行 type 值为 IIO_VOLTAGE，也就是读取电压值。第 14 行调用 stm32_adc_single_conv 函数来完成 ADC 单次读取。stm32_adc_single_conv 函

数会设置采样率、配置通道、使用硬件触发、开启转换，最后等待转换完成中断发生。

第 20~28 行，返回 ADC 对应的分辨率。

第 30~36 行，返回差分 ADC 的偏移值。

stm32_adc_read_raw 函数内容还是比较简单的，因为只是读取 ADC 原始值，不像 ICM20608 那么复杂。关于 ADC 驱动源码就讲解到这里，接下来我们学习如何使能 ADC，然后编写应用程序读取 ADC 采集到的值。

57.3 硬件原理图分析

STM32MP157 开发板 ADC 硬件原理图如图 57.3.1 所示：

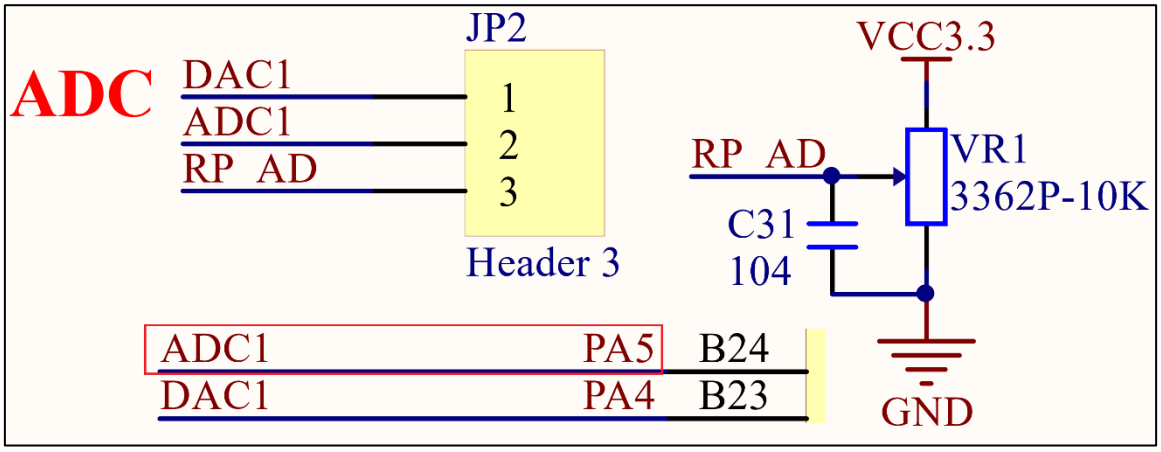


图 57.3.1 ADC 原理图

图 57.3.1 中 JP2 是一个 3P 的排针，1 脚连接到 STM32MP157 的 DAC 引脚上(PA4)，2 脚连接到 ADC 引脚上(PA5)，3 脚连接到 VR1 这个可调电位器上。本章实验我们使用 STM32MP157 的 ADC 来采集 VR1 可调电位器的电压，因此要用跳线帽将 JP1 的 2,3 脚连接起来，如图 57.3.2 所示：

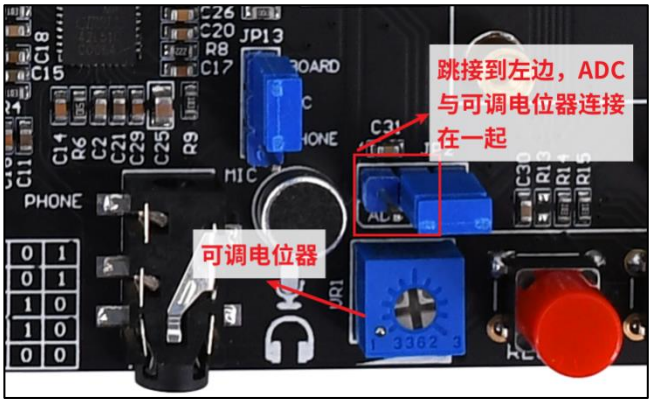


图 57.3.2 开发板 ADC 设置

57.4 ADC 驱动编写

57.4.1 修改设备树

ADC 驱动 ST 已经编写好了，我们只需要修改设备树即可。首先在 stm32mp15-pinctrl.dtsi 文件中添加 ADC 使用的 PA5 引脚配置信息：

示例代码 57.4.1.1 PA5 引脚配置信息

```
1 adc1_in19_pins_a: adc1-in19 {
2     pins {
3         pinmux = <STM32_PINMUX('A', 5, ANALOG)>;
4     };
5 };
```

接下来在 stm32mp157d-atk.dts 文件中向根节点添加 vdd 子节点信息，内容如下：

示例代码 57.4.1.2 vdd 子节点

```
1 vdd: regulator-vdd {
2   compatible = "regulator-fixed";
3   regulator-name = "vdd";
4   regulator-min-microvolt = <3300000>;
5   regulator-max-microvolt = <3300000>;
6   regulator-always-on;
7   regulator-boot-on;
8 };
```

最后在 stm32mp157d-atk.dts 文件中向 adc 节点追加一些内容，内容如下：

示例代码 57.4.1.3 adc 节点

```
1  &adc {
2     pinctrl-names = "default";
3     pinctrl-0 = <&adc1_in19_pins_a>;
4     vdd-supply = <&vdd>;
5     vdda-supply = <&vdd>;
6     vref-supply = <&vdd>;
7     status = "okay";
8
9     adc1: adc@0 {
10        st,adc-channels = <19>;
11        st,min-sample-time-nsecs = <10000>;
12        assigned-resolution-bits = <16>;
13        status = "okay";
14    };
15 };
```

第 3 行，配置 dac 引脚。

第 4~6 行，设置电压属性。

第 9~12 行，adc1 子节点，第 10 行 st,adc-channels 属性设置 adc 通道为 19，第 11 行 st,min-sample-time-nsecs 属性设置最小采样时间为 10000ns，第 12 行设置分辨率为 16 位。

57.4.2 使能 ADC 驱动

ST 官方默认已经使能了 ADC 驱动，所以不需要我们修改，但是为了学习，我们还是学习一下如何使能 Linux 内核自带的 ADC 驱动。打开 Linux 内核配置界面，配置路径如下：

```
-> Device Drivers
    -> Industrial I/O support (IIO [=y])
        -> Analog to digital converters
```

```
-> STMicroelectronics STM32 adc core (STM32_ADC_CORE [=y])
-> <*>STMicroelectronics STM32 adc        //使能 STM32 ADC
```

如图 57.4.2.1 所示：

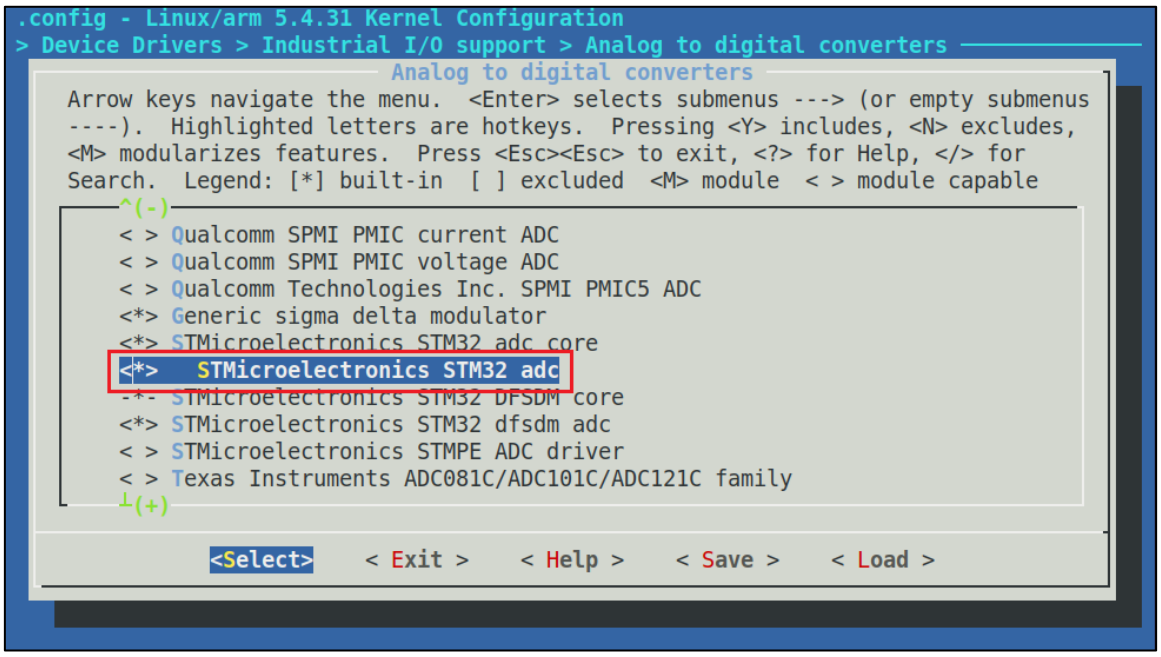


图 57.4.2.1 ADC 配置项

57.4.3 编写测试 APP

编译修改后的设备树，然后使用新的设备树启动系统。进入/sys/bus/iio/devices 目录下，此目录下就有 ADC 对应的 iio 设备：iio:deviceX，本章例程如图 57.4.3.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ ls
iio:device0
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 57.4.3.1 ADC iio 设备

图 57.4.3.1 中的“iio:device0”就是 ADC 设备，因为此时并没有加载其他的 IIO 设备驱动，只有一个 ADC。如果大家还加载了其他 IIO 设备驱动，那么就要依次进入 iio 设备目录，查看一下都对应的是什么设备。

进入“iio:device0”目录，内容如图 57.4.3.2 所示：

```
[root@ATK-stm32mp1]:/sys/devices/platform/soc/48003000.adc/48003000.adc:adc@0/iio:device0$ ls
buffer                  power
dev                     scan_elements
in_voltage19_raw        subsystem
in_voltage_offset       trigger
in_voltage_scale        trigger_polarity
name                    trigger_polarity_available
of_node                 uevent
[root@ATK-stm32mp1]:/sys/devices/platform/soc/48003000.adc/48003000.adc:adc@0/iio:device0$
```

图 57.4.3.2 iio:device0 目录文件

标准的 IIO 设备文件目录，我们只关心三个文件：

in_voltage19_raw：ADC1 通道 19 原始值文件。

in_voltage_offset：ADC1 偏移文件

in_voltage_scale：ADC1 比例文件(分辨率)，单位为 mV。实际电压值(mV)=in_voltage19_raw* in_voltage_scale

我的开发板此时 in_voltage19_raw 和 in_voltage_scale 这两个文件内容如下：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device0/in_voltage19_raw
20779
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device0/in_voltage_scale
0.050354003
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 57.4.3.3 当前电压

经过计算，图 57.4.3.3 中实际电压：20779*0.050354003≈1046.3mV，也就是 1.0463V。

接下来就编写测试 APP，新建 adcApp.c 文件，然后在里面输入如下所示内容：

示例代码 57.4.3.1 adcApp.c 文件内容

```
/***************************************************************
Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
文件名       : adcApp.c
作者         : 正点原子 Linux 团队
版本         : V1.0
描述         : adc 测试应用文件。
其他         : 无
使用方法     : ./adcApp
论坛         : www.openedv.com
日志         : 初版 V1.0 2021/09/24 正点原子 Linux 团队创建
***************************************************************/
#include "stdio.h"
#include "unistd.h"
#include "sys/types.h"
#include "sys/stat.h"
#include "sys/ioctl.h"
#include "fcntl.h"
#include "stdlib.h"
#include "string.h"
#include <poll.h>
#include <sys/select.h>
#include <sys/time.h>
#include <signal.h>
#include <fcntl.h>
#include <errno.h>

/* 字符串转数字，将浮点小数字符串转换为浮点数数值 */
#define SENSOR_FLOAT_DATA_GET(ret, index, str, member)\
    ret = file_data_read(file_path[index], str);\
    dev->member = atof(str);\

/* 字符串转数字，将整数字符串转换为整数数值 */
```

```
22  #define SENSOR_INT_DATA_GET(ret, index, str, member)\
23      ret = file_data_read(file_path[index], str);\
24      dev->member = atoi(str);\
25
26
27  /* adc iio 框架对应的文件路径 */
28  static char *file_path[] = {
29      "/sys/bus/iio/devices/iio:device0/in_voltage_scale",
30      "/sys/bus/iio/devices/iio:device0/in_voltage19_raw",
31  };
32
33  /* 文件路径索引，要和 file_path 里面的文件顺序对应 */
34  enum path_index {
35      IN_VOLTAGE_SCALE = 0,
36      IN_VOLTAGE_RAW,
37  };
38
39  /*
40   * ADC 数据设备结构体
41   */
42  struct adc_dev{
43      int raw;
44      float scale;
45      float act;
46  };
47
48  struct adc_dev stm32adc;
49
50   /*
51   * @description       : 读取指定文件内容
52   * @param - filename  : 要读取的文件路径
53   * @param - str       : 读取到的文件字符串
54   * @return            : 0 成功;其他 失败
55   */
56  static int file_data_read(char *filename, char *str)
57  {
58      int ret = 0;
59      FILE *data_stream;
60
61      data_stream = fopen(filename, "r"); /* 只读打开 */
62      if(data_stream == NULL) {
63          printf("can't open file %s\r\n", filename);
64          return -1;
```

```
65      }
66
67      ret = fscanf(data_stream, "%s", str);
68      if(!ret) {
69          printf("file read error!\r\n");
70      } else if(ret == EOF) {
71          /* 读到文件末尾的话将文件指针重新调整到文件头 */
72          fseek(data_stream, 0, SEEK_SET);
73      }
74      fclose(data_stream);    /* 关闭文件 */
75      return 0;
76  }
77
78   /*
79    * @description   : 获取 ADC 数据
80    * @param - dev   : 设备结构体
81    * @return        : 0 成功;其他 失败
82    */
83  static int adc_read(struct adc_dev *dev)
84  {
85      int ret = 0;
86      char str[50];
87
88      SENSOR_FLOAT_DATA_GET(ret, IN_VOLTAGE_SCALE, str, scale);
89      SENSOR_INT_DATA_GET(ret, IN_VOLTAGE_RAW, str, raw);
90
91      /* 转换得到实际电压值 mV */
92      dev->act = (dev->scale * dev->raw)/1000.f;
93      return ret;
94  }
95
96  /*
97   * @description   : main 主程序
98   * @param - argc  : argv 数组元素个数
99   * @param - argv  : 具体参数
100  * @return        : 0 成功;其他 失败
101  */
102 int main(int argc, char *argv[])
103 {
104     int ret = 0;
105
106     if (argc != 1) {
107         printf("Error Usage!\r\n");
```

```
108         return -1;
109     }
110
111     while (1) {
112         ret = adc_read(&stm32adc);
113         if(ret == 0) {          /* 数据读取成功 */
114             printf("ADC 原始值：%d，电压值：%.3fV\r\n", stm32adc.raw,
                                                    stm32adc.act);
115         }
116         usleep(100000); /*100ms */
117     }
118     return 0;
119 }
```

adcApp.c 就是在上一章的应用程序上修改而来的，由于只读取一路 ADC，因此内容反而更简单，这里就不做介绍了。

57.5 运行测试

57.5.1 编译驱动程序和测试 APP

由于不需要我们编写 ADC 驱动程序，因此也就不需要编译驱动程序。设备树前面已经编译过了，所以这里就只剩下编译测试 APP。由于 adcApp.c 用到了浮点运算，因此我们编译的时候要使能硬件浮点，输入如下编译 adcApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc -march=armv7-a -mfpu=neon -mfloat-abi=hard adcApp.c -o adcApp
```

编译成功以后就会生成 adcApp 这个应用程序。

57.5.2 运行测试

注意，在测试之前一定要先按照图 57.3.2 所示，将 JP2 跳线帽接到左边，也就是将 ADC1_CH19 通道连接到开发板上的可调电位器上！

输入如下命令，使用 adcApp 测试程序：

```
./adcApp
```

测试 APP 会不断的读取 ADC 值并输出到终端，我们可以通过调节开发板上的电位器来改变电压值，如图 57.5.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./adcApp
ADC原始值：40，电压值：0.002V
ADC原始值：0，电压值：0.000V
ADC原始值：0，电压值：0.000V
ADC原始值：72，电压值：0.004V
ADC原始值：6830，电压值：0.344V
ADC原始值：21042，电压值：1.060V
ADC原始值：33203，电压值：1.672V
ADC原始值：46716，电压值：2.352V
ADC原始值：46625，电压值：2.348V
ADC原始值：52412，电压值：2.639V
ADC原始值：65535，电压值：3.300V
ADC原始值：65535，电压值：3.300V
```

图 57.5.2.1 测试结果

从图 57.5.2.1 可以看到 ADC 原始值以及对应的电压值，因为 STM32MP157 的 ADC 可采集电压范围为 0~3.3V，因此当我们扭动开发板上的电位器的时候，电压会在 0~3.3V 之间变化。

第五十八章 Linux DAC 驱动实验

上一章我们讲解了 ADC 驱动，STM32MP157 也有 DAC 外设，DAC 也使用的 IIO 驱动框架。本章我们就来学习一下如下在 Linux 下使用 STM32MP157 上的 DAC。

58.1 DAC 简介

ADC 是模数转换器，负责将外界的模拟信号转换为数字信号。DAC 刚好相反，是数模转换器，负责将 SOC 的数字信号转换为模拟信号。

STM32MP157 的 DAC 模块(数字/模拟转换模块)是 12 位数字输入，电压输出型的 DAC。DAC 可以配置为 8 位或 12 位模式，也可以与 DMA 控制器配合使用。DAC 工作在 12 位模式时，数据可以设置成左对齐或右对齐。DAC 模块有 2 个输出通道，每个通道都有独立的转换器。在双 DAC 模式下，2 个通道可以独立地进行转换，也可以同时进行转换并同步地更新 2 个通道的输出。DAC 可以通过引脚输入参考电压 Vref+（通 ADC 共用）以获得更精确的转换结果。STM32H743 的 DAC 模块主要特点有：

1）、1 个 DAC 接口，最大两个 DAC 输出通道。

2）、12 位模式下数据左对齐或者右对齐。

3）、同步更新功能。

4）、噪声波、三角波形生成。

5）、外部触发。

6）、双 DAC 通道同时或者分别转换。

7）、每个通道都有 DMA 功能。

8）、输入参考电压 V_{REF+}。

……

STM32MP157 DAC 框图如图 58.1.1 所示：

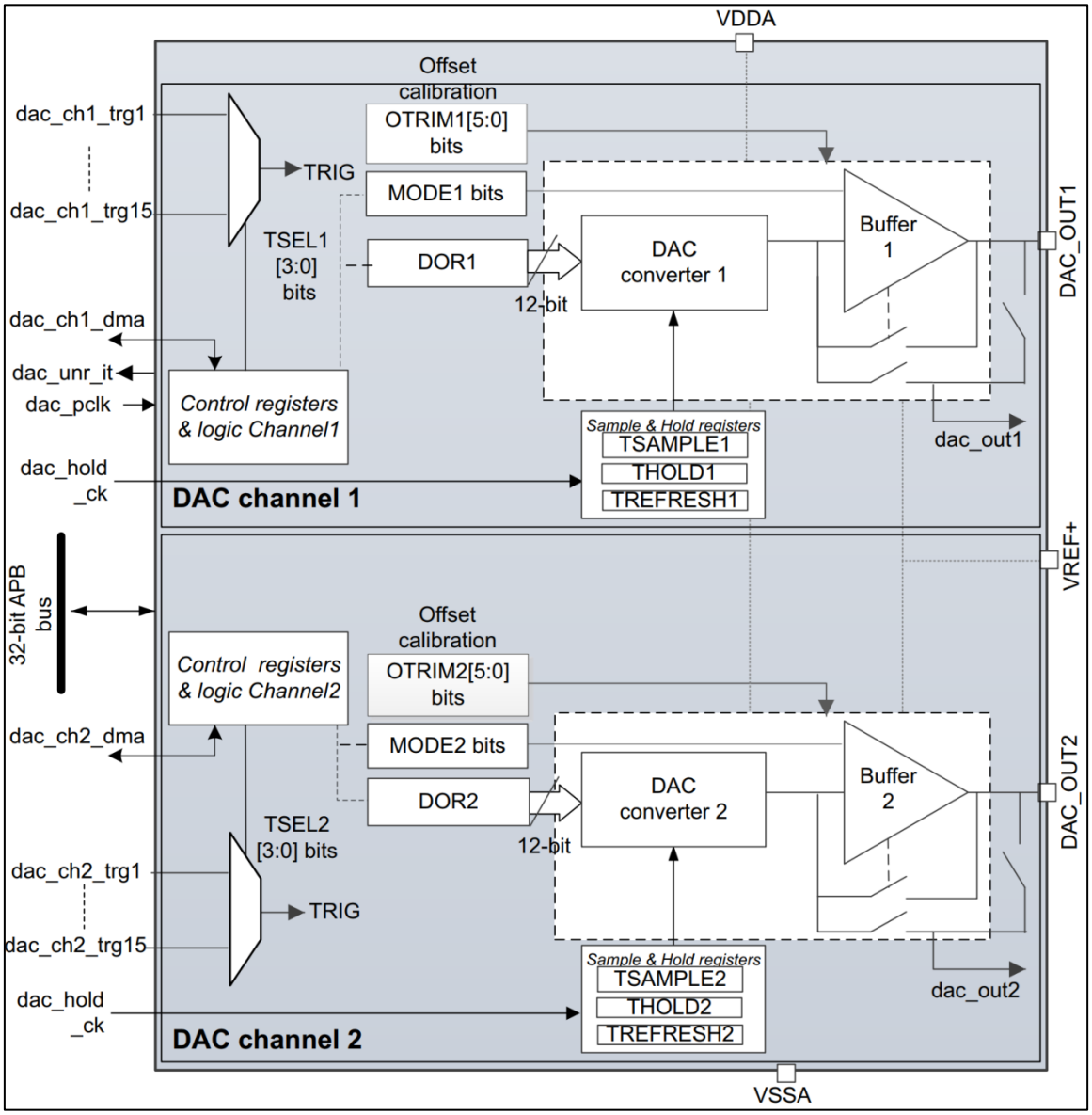


图 58.1.1 DAC 结构框图

图中 VDDA 和 VSSA 为 DAC 模块模拟部分的供电，而 VREF+则是 DAC 模块的参考电压。DAC_OUT1/2 就是 DAC 的两个输出通道了，DAC_OUT1 对应 PA4 引脚，DAC_OUT2 对应 PA5 引脚。正点原子 STM32MP157 开发板使用 DAC_OUT1，引脚为 PA4。

58.2 DAC 驱动源码分析

58.2.1 设备树下的 DAC 节点

stm32mp151.dtsi 文件中的 dac 节点信息如下：

示例代码 58.2.1.1 dac 节点信息

```
1  dac: dac@40017000 {
2       compatible = "st,stm32h7-dac-core";
3       reg = <0x40017000 0x400>;
4       clocks = <&rcc DAC12>;
```

```
5       clock-names = "pclk";
6       #address-cells = <1>;
7       #size-cells = <0>;
8       status = "disabled";
9
10      dac1: dac@1 {
11          compatible = "st,stm32-dac";
12          #io-channel-cells = <1>;
13          reg = <1>;
14          status = "disabled";
15      };
16
17      dac2: dac@2 {
18          compatible = "st,stm32-dac";
19          #io-channel-cells = <1>;
20          reg = <2>;
21          status = "disabled";
22      };
23  };
```

第 2 行，compatible 属性值为“st,stm32h7-dac-core”，所以在整个 Linux 源码里面搜索这个字符串即可找到 STM32MP157 的 DAC 驱动核心文件，这个文件就是 drivers/iio/dac/stm32-dac-core.c。

第 11、18 行，compatible 属性值“st,stm32-dac”，搜索这个字符串，可以找到 ADC 驱动文件，这个文件就是 drivers/iio/dac/stm32-dac.c。

关于 STM32MP157 的 DAC 节点更为详细的信息请参考对应的绑定文档：Documentation/devicetree/bindings/iio/dac/st,stm32-dac.txt。接下来我们简单分析一下绑定文档，后面我们需要根据绑定文档修改设备树，使能 DAC 对应的通道。

DAC 首先需要一个根节点，DAC 根节点属性如下：

①、必要属性

- **compatible**：兼容性属性，必须的，可以设置为“st,stm32h7-dac-core”。
- **reg**：DAC 控制器寄存器信息。
- **clocks**：时钟。
- **clock-names**：时钟名字，必须为“pclk”。
- **vref-supply**：此属性对应 vref 参考电压句柄。
- **address-cells**：设置为 1。
- **size-cells**：设置为 0。

②、可选属性：

- ：pinctrl 引脚配置信息。
- **resets**：复位句柄。

STM32MP157 有两个 DAC 通道，每个 DAC 通道对应一个子节点，DAC 子节点相关属性如下：

- **compatible**：兼容性属性，必须的，可以设置为“st,stm32-dac”。

- reg：不同 ADC 控制器寄存器地址偏移信息。

- io-channel-cells：设置为 1。

关于 DAC 的设备树就讲解到这里。

58.2.2 DAC 驱动源码分析

STM32MP157 DAC 驱动文件也有两个：stm32-dac-core.c 和 stm32-dac.c。stm32-dac-core.c 是 DAC 核心层，主要用于 DAC 时钟、电源等初始化。我们需要重点关注的是 stm32-dac.c 这个文件。stm32-adc.c 主体框架是 platform，配合 IIO 驱动框架实现 DAC 驱动。

1、stm32_dac 结构体

首先来看一下 stm32_dac 结构体，内如如下：

示例代码 58.2.2.1 stm32_dac 结构体

```
1 struct stm32_dac {
2   struct stm32_dac_common *common;
3 };
```

stm32_dac 结构体很简单，比上一章的 stm32_adc 结构体要简单很多，只有一个 stm32_dac_common 成员变量，内容如下：

示例代码 58.2.2.2 stm32_dac_common 结构体

```
1 struct stm32_dac_common {
2       struct regmap *regmap;   /* regmap          */
3       int           vref_mv;   /* 参考电压        */
4       bool          hfsel;     /* 高速总线时钟选择 */
5 };
```

可以看出，DAC 驱动也采用了 regmap API。

2、stm32_adc_probe 函数

接下来看一下 stm32_dac_probe 函数，内容如下(有省略)：

示例代码 58.2.2.3 stm32_dac_probe 函数

```
1  static int stm32_dac_probe(struct platform_device *pdev)
2  {
3       struct device_node *np = pdev->dev.of_node;
4       struct device *dev = &pdev->dev;
5       struct iio_dev *indio_dev;
6       struct stm32_dac *dac;
7       int ret;
8
9       if (!np)
10          return -ENODEV;
11
12      indio_dev = devm_iio_device_alloc(&pdev->dev, sizeof(*dac));
13      if (!indio_dev)
14          return -ENOMEM;
15      platform_set_drvdata(pdev, indio_dev);
16
```

```
17      dac = iio_priv(indio_dev);
18      dac->common = dev_get_drvdata(pdev->dev.parent);
19      indio_dev->name = dev_name(&pdev->dev);
20      indio_dev->dev.parent = &pdev->dev;
21      indio_dev->dev.of_node = pdev->dev.of_node;
22      indio_dev->info = &stm32_dac_iio_info;
23      indio_dev->modes = INDIO_DIRECT_MODE;
24
25      ret = stm32_dac_chan_of_init(indio_dev);
26      if (ret < 0)
27          return ret;
......
36      ret = iio_device_register(indio_dev);
37      if (ret)
38          goto err_pm_put;
......
50      return ret;
51  }
```

第 12 行，调用 devm_iio_device_alloc 函数申请 iio_dev，这里也连 stm32_dac 内存一起申请了。

第 17 行，调用 iio_priv 函数从 iio_dev 里面的到 stm32_dac 首地址。

第 19~23 行，初始化 iio_dev，重点是第 22 行的 stm32_dac_iio_info，因为用户空间读取或设置 DAC 数据最终就是由 stm32_dac_iio_info 来完成的。

第 25 行，调用 stm32_dac_chan_of_init 函数设置 DAC 通道。

第 36 行，调用 iio_device_register 函数向内核注册 iio_dev。

同样的 tm32_dac_probe 函数核心就是初始化 ADC，然后建立 ADC 的 IIO 驱动框架。

3、stm32_dac_iio_info 结构体

stm32_dac_iio_info 结构体内容如下所示：

示例代码 58.2.2.4 stm32_dac_iio_info 结构体

```
1  static const struct iio_info stm32_dac_iio_info = {
2      .read_raw = stm32_dac_read_raw,
3      .write_raw = stm32_dac_write_raw,
4      .debugfs_reg_access = stm32_dac_debugfs_reg_access,
5  };
```

第 2 行，stm32_dac_read_raw 函数用于读取 DAC 信息，读取 DAC 原始数据值、分辨率等。

第 3 行，stm32_dac_write_raw 函数用于设置 DAC 值。

stm32_dac_read_raw 和 stm32_dac_write_raw 函数内容如下：

示例代码 58.2.2.5 stm32_dac_read_raw 和 stm32_dac_write_raw 函数

```
1  static int stm32_dac_read_raw(struct iio_dev *indio_dev,
2                struct iio_chan_spec const *chan,
3                int *val, int *val2, long mask)
4  {
5      struct stm32_dac *dac = iio_priv(indio_dev);
```

```
6
7       switch (mask) {
8           case IIO_CHAN_INFO_RAW:
9               return stm32_dac_get_value(dac, chan->channel, val);
10          case IIO_CHAN_INFO_SCALE:
11              *val = dac->common->vref_mv;
12              *val2 = chan->scan_type.realbits;
13              return IIO_VAL_FRACTIONAL_LOG2;
14          default:
15              return -EINVAL;
16  }
17 }
18
19 static int stm32_dac_write_raw(struct iio_dev *indio_dev,
20                 struct iio_chan_spec const *chan,
21                 int val, int val2, long mask)
22 {
23      struct stm32_dac *dac = iio_priv(indio_dev);
24
25      switch (mask) {
26          case IIO_CHAN_INFO_RAW:
27              return stm32_dac_set_value(dac, chan->channel, val);
28      default:
29          return -EINVAL;
30      }
31 }
```

第 1~17 行，stm32_dac_read_raw 函数，读取 DAC 的原始值以及分辨率，非常简单。

第 19~31 行，stm32_dac_write_raw 函数，向 DAC 写入原始值，也就是设置 DAC。

58.3 硬件原理图分析

DAC 原理图如下所示：

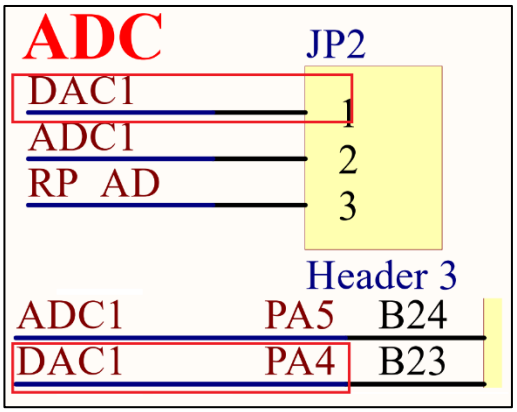


图 58.3.1 DAC 原理图

上一章讲 ADC 的时候，说了 JP2 是一个 3P 的排针，用来设置 ADC 连接可调电位器还是

DAC。本章我们学习使用 DAC，因此可以使用跳线帽将 JP2 的 1,2 引脚连接起来。也就是将 DAC 和 ADC 连接在一起，如下图所示：

图 58.3.2 ADC 跳线帽设置

我们可以编写应用程序设置 DAC，然后再使用 ADC 采集回去。正点原子 STM32MP157 开发板使用了 DAC 通道 1，引脚为 PA4。

58.4 DAC 驱动编写

58.4.1 修改设备树

DAC 驱动 ST 已经编写好了，我们只需要修改设备树即可。首先在 stm32mp15-pinctrl.dtsi 文件中添加 DAC 使用的 PA4 引脚配置信息：

示例代码 58.4.1.1 PA4 引脚配置信息

```
1 dac_ch1_pins_a: dac-ch1 {
2     pins {
3         pinmux = <STM32_PINMUX('A', 4, ANALOG)>;
4     };
5 };
```

接下来在 stm32mp157d-atk.dts 文件中向根节点添加 vdd 子节点信息，内容如下：

示例代码 58.4.1.2 vdd 子节点

```
1 v3v3: regulator-3p3v {
2     compatible = "regulator-fixed";
3     regulator-name = "v3v3";
4     regulator-min-microvolt = <3300000>;
5     regulator-max-microvolt = <3300000>;
6     regulator-always-on;
7     regulator-boot-on;
8 };
```

最后在 stm32mp157d-atk.dts 文件中向 adc 节点追加一些内容，内容如下：

示例代码 58.4.1.3 adc 节点

```
1 &dac {
2     pinctrl-names = "default";
3     pinctrl-0 = <&dac_ch1_pins_a>;
4     vref-supply = <&v3v3>;
5     status = "okay";
```

```
6     dac1: dac@1 {
7         status = "okay";
8     };
9 };
```

第 3 行，配置 dac 引脚。

第 4 行，设置电压属性。

第 6~8 行，dac1 子节点，设置很简单，直接将 status 属性设置为“okay”即可。

58.4.2 使能 DAC 驱动

同样的，使能 Linux 内核中的 ST32MP157 DAC 驱动，打开 Linux 内核配置界面，配置路径如下：

```
-> Device Drivers
    -> Industrial I/O support (IIO [=y])
        -> Digital to analog converters
            -> <*>STMicroelectronics STM32 DAC      //使能 STM32 DAC
```

如图 58.4.2.1 所示：

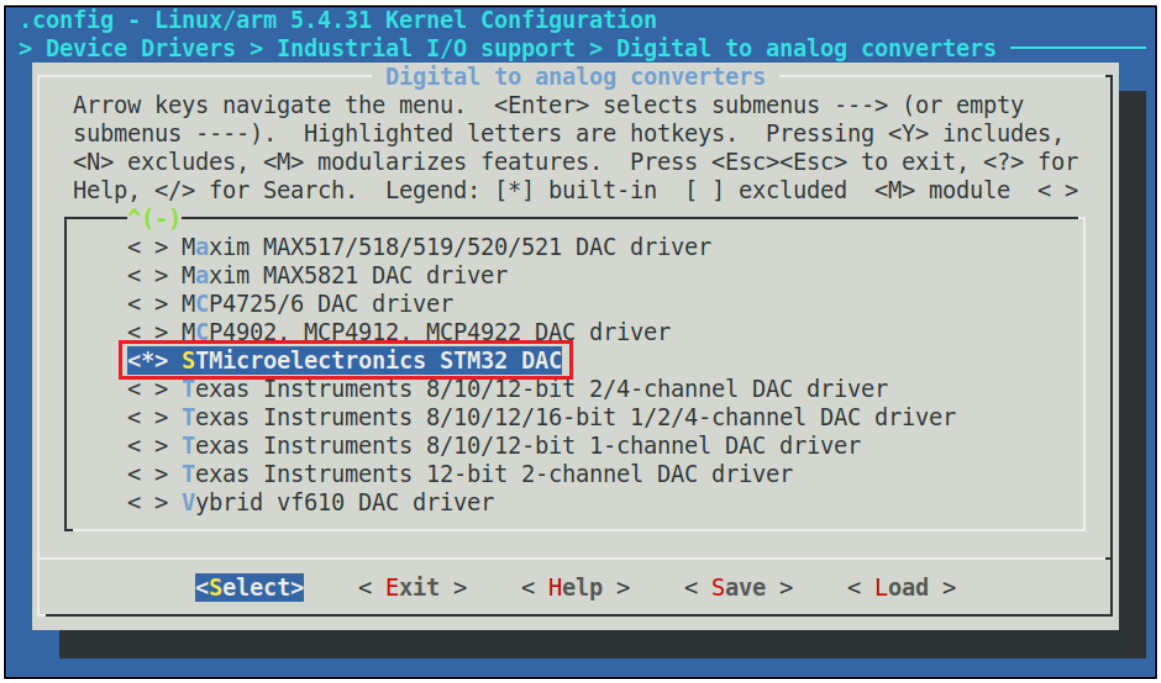


图 58.4.2.1 DAC 配置项

58.4.3 编写测试 APP

编译修改后的设备树，然后使用新的设备树启动系统。进入/sys/bus/iio/devices 目录下，此目录下就有 ADC 对应的 iio 设备：iio:deviceX，本章例程如图 58.4.3.1 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ ls
iio:device0        iio:device1
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 58.4.3.1 ADC iio 设备

图 58.4.3.1 中有两个 IIO 设备：iio:device0 和 iio:device1，大家可以依次进入这两个目录查看分别对应什么外设。笔者当前所使用的开发板中 iio:device0 为 ADC(上一章实验使能的 ADC 驱动)，iio:device1 为本章使能的 DAC 设备。

进入“iio:device1”目录，内容如图 58.4.3.2 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ ls iio:device1
dev                                 out_voltage1_scale
name                                out_voltage_powerdown_mode_available
of_node                             power
out_voltage1_powerdown              subsystem
out_voltage1_powerdown_mode         uevent
out_voltage1_raw
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 58.4.3.2 iio:device1 目录文件

标准的 IIO 设备文件目录，我们只关心三个文件：

out_voltage1_powerdown：DAC 输出使能文件，写 0 打开 DAC，写 1 关闭 DAC，默认为 1，也就是关闭 DAC。

out_voltage1_raw：DAC1 通道 1 原始值文件。

out_voltage1_scale：DAC1 比例文件(分辨率)，单位为 mV。实际输出电压值(mV)= out_voltage1_raw * out_voltage1_scale。out_voltage1_scale 默认值如图 58.4.3.3 所示：

```
[root@ATK-stm32mp1]:/sys/bus/iio/devices$ cat iio:device1/out_voltage1_scale
0.805664062
[root@ATK-stm32mp1]:/sys/bus/iio/devices$
```

图 58.4.3.3 out_voltage1_scale 内容

从图 58.4.3.3 可以看出，out_voltage1_scale 默认为 0.805664062。

ADC1 默认 12 位，因此可设置范围为 0~4095。我们向 out_voltage1_raw 写入 2000，命令如下：

```
echo 0 > /sys/bus/iio/devices/iio:device1/out_voltage1_powerdown      //开启 DAC
echo 2000 > /sys/bus/iio/devices/iio:device1/out_voltage1_raw         //设置 DAC
```

此时 DAC 输出的理论电压值为 2000*0.805664062≈1611.328mV。

那么 DAC 输出是否正确呢？很简单，直接使用上一章编写的 adcAPP.c 读取 DAC 引脚电压值就行了。**一定要先按照图 58.3.2 所示，将 JP2 的右边两根排针连接起来，也就是将 DAC 和 ADC 引脚连接在一起**。

运行上一章的 adcApp.c，结果如图 58.4.3.4 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./adcApp
ADC原始值: 32008, 电压值: 1.612V
ADC原始值: 32041, 电压值: 1.613V
ADC原始值: 32087, 电压值: 1.616V
ADC原始值: 32003, 电压值: 1.611V
```

图 58.4.3.4 ADC 采集结果

从图 58.4.3.4 可以看出，ADC 采集到的电压为 1.61V，和我们设置的 DAC 理论值基本一致。这里要注意，DAC1 是 12 位的，而 ADC 是 16 位的，因此大家可以看到他们的原始值会不一样。

接下来我们编译一个简单的 DAC 测试 APP，APP 等待用户输入 DAC 原始值，一旦用户输入以后就调用 ADC 来采集 DAC 输出的电压值，最后将 DAC 理论值与 ADC 采集到的实际值打印出来，看一下是否正确。新建名为 dacApp.c 的文件，在文件里面加入如下内容：

示例代码 58.4.3.1 dacApp.c 文件内容

```
/****************************************************************
Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
文件名       : dacApp.c
作者         : 正点原子 Linux 团队
版本         : V1.0
描述         : dac 测试应用文件。
其他         : 无
使用方法     : ./dacApp
论坛         : www.openedv.com
日志         : 初版 V1.0 2021/09/24 正点原子 Linux 团队创建
****************************************************************/
#include "stdio.h"
#include "unistd.h"
#include "sys/types.h"
#include "sys/stat.h"
#include "sys/ioctl.h"
#include "fcntl.h"
#include "stdlib.h"
#include "string.h"
#include <poll.h>
#include <sys/select.h>
#include <sys/time.h>
#include <signal.h>
#include <fcntl.h>
#include <errno.h>

/* 字符串转数字，将浮点小数字符串转换为浮点数数值 */
#define SENSOR_FLOAT_DATA_GET(ret, index, str, member)\
    ret = file_data_read(file_path[index], str);\
    dev->member = atof(str);\

/* 字符串转数字，将整数字符串转换为整数数值 */
#define SENSOR_INT_DATA_GET(ret, index, str, member)\
    ret = file_data_read(file_path[index], str);\
    dev->member = atoi(str);\

/* iio 框架对应的文件路径 */
static char *file_path[] = {
    "/sys/bus/iio/devices/iio:device0/in_voltage_scale",
    "/sys/bus/iio/devices/iio:device0/in_voltage19_raw",
    "/sys/bus/iio/devices/iio:device1/out_voltage1_scale",
    "/sys/bus/iio/devices/iio:device1/out_voltage1_raw",
```

```
32  };
33
34  /* 文件路径索引，要和 file_path 里面的文件顺序对应 */
35  enum path_index {
36      IN_VOLTAGE_SCALE = 0,
37      IN_VOLTAGE_RAW,
38      OUT_VOLTAGE1_SCALE,
39      OUT_VOLTAGE1_RAW,
40  };
41
42  /*
43   * dac 数据设备结构体
44   */
45  struct dac_dev{
46      int dac_raw, adc_raw;
47      float dac_scale, adc_scale;
48      float dac_act, adc_act;
49  };
50
51  struct dac_dev stm32dac;
52
53   /*
54   * @description       : 读取指定文件内容
55   * @param - filename : 要读取的文件路径
56   * @param - str       : 读取到的文件字符串
57   * @return            : 0 成功;其他 失败
58   */
59  static int file_data_read(char *filename, char *str)
60  {
61      int ret = 0;
62      FILE *data_stream;
63
64      data_stream = fopen(filename, "r"); /* 只读打开 */
65      if(data_stream == NULL) {
66          printf("can't open file %s\r\n", filename);
67          return -1;
68      }
69
70      ret = fscanf(data_stream, "%s", str);
71      if(!ret) {
72          printf("file read error!\r\n");
73      } else if(ret == EOF) {
74          /* 读到文件末尾的话将文件指针重新调整到文件头 */
```

```
75          fseek(data_stream, 0, SEEK_SET);
76      }
77      fclose(data_stream);    /* 关闭文件 */
78      return 0;
79  }
80
81   /*
82   * @description   : 获取 ADC、DAC 数据
83   * @param - dev   : 设备结构体
84   * @return        : 0 成功;其他 失败
85   */
86  static int dac_add_dac_read(struct dac_dev *dev)
87  {
88      int ret = 0;
89      char str[50];
90
91      /* 1、获取 ADC 值 */
92      SENSOR_FLOAT_DATA_GET(ret, IN_VOLTAGE_SCALE, str, adc_scale);
93      SENSOR_INT_DATA_GET(ret, IN_VOLTAGE_RAW, str, adc_raw);
94
95      /* 转换 ADC 采集到的实际电压值 mV */
96      dev->adc_act = (dev->adc_scale * dev->adc_raw)/1000.f;
97
98      /* 2、获取 DAC 值 */
99      SENSOR_FLOAT_DATA_GET(ret, OUT_VOLTAGE1_SCALE, str, dac_scale);
100     SENSOR_INT_DATA_GET(ret, OUT_VOLTAGE1_RAW, str, dac_raw);
101
102     /* 转换 DAC 理论电压值 mV */
103     dev->dac_act = (dev->dac_scale * dev->dac_raw)/1000.f;
104     return ret;
105 }
106
107  /*
108   * @description   : 使能 DAC
109   * @return        : 无
110   */
111 void dac_enable(void)
112 {
113     system("echo 0 > /sys/bus/iio/devices/iio:device1/out_voltage1_po
werdown");
114 }
115
116 /*
```

```
117  * @description   : 禁止 DAC
118  * @return        : 无
119  */
120 void dac_disable(void)
121 {
122     system("echo 1 > /sys/bus/iio/devices/iio:device1/out_voltage1_po
werdown");
123 }
124 
125 /*
126  * @description      : 设置 DAC
127  * @param - filename : 要写的文件路径
128  * @param - value    : DAC 原始值
129  * @return           : 无
130  */
131 int dac_set(char *filename, int value)
132 {
133     int ret = 0;
134     FILE *data_stream;
135     char str[10];
136 
137     /* 1、将整形变量转换为字符串 */
138     sprintf(str, "%d", value);
139 
140     /* 2、打开文件流 */
141     data_stream = fopen(filename, "w"); /* 只写打开 */
142     if(data_stream == NULL) {
143         printf("can't open file %s\r\n", filename);
144         return -1;
145     }
146 
147     /* 3、将文件指针重新调整到文件头 */
148     fseek(data_stream, 0, SEEK_SET);
149 
150     /* 4、向文件流写入数据 */
151     ret = fwrite(str, sizeof(str), 1, data_stream);
152     if(!ret) {
153         printf("file read error!\r\n");
154     }
155 
156     /* 5、关闭文件 */
157     fclose(data_stream);
158     return 0;
```

```
159 }
160
161 /*
162  * @description   : main 主程序
163  * @param - argc  : argv 数组元素个数
164  * @param - argv  : 具体参数
165  * @return        : 0 成功;其他 失败
166  */
167 int main(int argc, char *argv[])
168 {
169     int ret = 0;
170     unsigned int cmd;
171     unsigned char str[100];
172
173     if (argc != 1) {
174         printf("Error Usage!\r\n");
175         return -1;
176     }
177
178     dac_enable();   /* 是能 DAC */
179     while (1) {
180         printf("请输入 DAC 原始值(0~4095):");
181         ret = scanf("%d", &cmd);
182         if (ret != 1) {                     /* 参数输入错误 */
183             fgets(str, sizeof(str), stdin); /* 防止卡死     */
184         } else {                            /* 参数输入正确 */
185             if((cmd < 0) || (cmd > 4095)) {
186                 printf("输入错误，请正确输入 DAC 值，范围:0~4095!\r\n");
187                 continue;
188             }
189             dac_set(file_path[OUT_VOLTAGE1_RAW], cmd);
190             ret = dac_add_dac_read(&stm32dac);
191             if(ret == 0) {          /* 数据读取成功 */
192                 printf("DAC 原始值: %d，理论电压值: %.3fV\r\n",
                           stm32dac.dac_raw, stm32dac.dac_act);
193                 printf("ADC 原始值: %d，实际电压值: %.3fV\r\n",
                             stm32dac.adc_raw, stm32dac.adc_act);
194                 printf("\r\n");
195             }
196         }
197     }
198     return 0;
199 }
```

本章的 dacApp.c 是在上一章的 dacApp.c 基础上修改而来的。重点是添加了设置 DAC 以及读取 DAC 数据的内容。我们接下来就只看一下与 DAC 设置相关的内容：

第 27~32 行，file_path 为需要操作的文件路径，有 DAC 和 ADC 对应的原始值、分辨率文件。

第 45~49 行，dac_dev 为 DAC 设备结构体。

第 86~105 行，dac_add_adc_read 函数读取 ADC 和 DAC 的原始值以及分辨率，并且计算出对应的实际电压值。

第 111~114 行，dac_enable 函数使能 DAC 输出，其实就是向 out_voltage1_powerdown 文件写 0。

第 120~123 行，dac_disable 函数关闭 DAC 输出，就是向 out_voltage1_powerdown 文件写 1。

第 131~159 行，dac_set 函数用于设置 DAC 原始值。dac_set 的 value 参数就是要设置的 DAC 原始值，但是这是个整形参数。而 out_voltage1_raw 文件需要输入字符串，因此在 138 行先使用 sprintf 函数将整形的 value 值转换为对应的字符串，然后才能调用 fwrite 函数将其写入到 out_voltage1_raw 文件中。

第 167~199 行，main 函数，首先在 178 行调用 dac_enable 函数使能 DAC 输出，然后在 while 循环中等待用户输入要设置的 DAC 值。当输入的 DAC 值处于合法的范围内(0~4095)就调用 dac_set 函数设置 DAC，然后再调用 dac_add_adc_read 函数读取 ADC 采集到的值以及 DAC 理论值。

58.5 运行测试

58.5.1 编译驱动程序和测试 APP

输入如下编译 dacApp.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc -march=armv7-a -mfpu=neon -mfloat-abi=hard dacApp.c -o dacApp
```

编译成功以后就会生成 dacApp 这个应用程序。

58.5.2 运行测试

注意，在测试之前一定要先按照图 57.3.2 所示，将 JP2 跳线帽接到右边，也就是将 ADC1_CH19 通道连接到开发板上 DAC1 引脚上！

输入如下命令，使用 dacApp 测试程序：

```
./dacApp
```

APP 运行以后会等待输入要设置的 DAC 值，每输入一次就会自动打印出 ADC 采集到的实际 ADC 值以及 DAC 的理论值，如图 58.5.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./dacApp
请输入DAC原始值(0~4095):0
DAC原始值：0，理论电压值：0.000V
ADC原始值：895，实际电压值：0.045V

请输入DAC原始值(0~4095):500
DAC原始值：500，理论电压值：0.403V
ADC原始值：8014，实际电压值：0.404V

请输入DAC原始值(0~4095):1000
DAC原始值：1000，理论电压值：0.806V
ADC原始值：16072，实际电压值：0.809V

请输入DAC原始值(0~4095):2000
DAC原始值：2000，理论电压值：1.611V
ADC原始值：31995，实际电压值：1.611V

请输入DAC原始值(0~4095):3000
DAC原始值：3000，理论电压值：2.417V
ADC原始值：48001，实际电压值：2.417V

请输入DAC原始值(0~4095):4095
DAC原始值：4095，理论电压值：3.299V
ADC原始值：64832，实际电压值：3.265V

请输入DAC原始值(0~4095):
```

图 58.5.2.1 DAC 测试结果

图 58.5.2.1 中我们设置了 0、500、1000、2000、3000 和 4095 共 6 个 DAC 原始值，可以看出 DAC 设置的理论值和 ADC 采集到的实际值基本一致。

第五十九章 Linux 单总线驱动实验

单总线的定义是主机和从机通过一根线进行通信，由 DALLAS 半导体公司推出的一种通信技术。优点在于单总线技术具有线路简单，硬件开销少、成本低等等。本章通过 DS18B20 和 DHT11 传感器来学习如何在 Linux 下编写单总线驱动。

59.1 DS18B20 及工作时序简介

59.1.1 DS18B20 简介

DS18B20 为单总线接口的温度传感器。与传统的热敏电阻等测温元件相比，它是一种新型的体积小、适用电压宽、与微处理器接口简单的数字化温度传感器。单总线结构具有简洁且经济的特点，可使用户轻松地组建传感器网络，从而为测量系统的构建引入全新的概念，测试温度范围为-55~+125℃，精度为±0.5℃。现场温度直接以单总线的数字方式传输，大大提高了系统的抗干扰性。它能直接读出被测温度，并且可根据实际要求通过简单的编程实现 9~12 位的数字值读数方式。它工作在 3~5.5V 的电压范围，采用多种封装形式，从而使系统设置灵活、方便，设定分辨率以及用户设定的报警温度存储在 EEPROM 中，掉电后依然保存。其内部结构如图 59.1.1.1 所示：

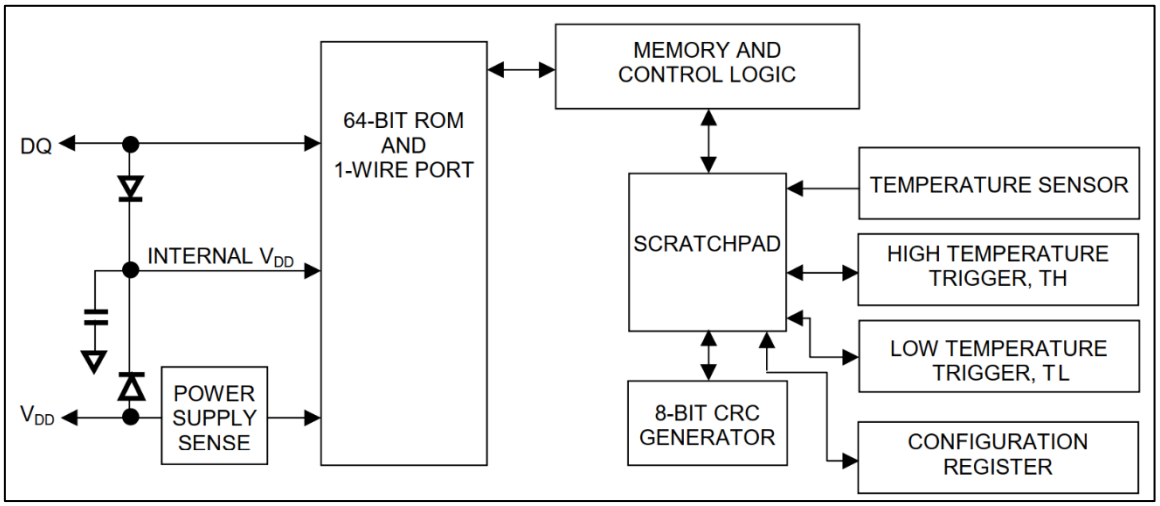


图 59.1.1.1 DS18B20 内部结构图

ROM 中的 64 位序列号是出厂前被标记好的，它可以看作使该 DS18B20 的地址序列码，每个 DS18B20 的 64 位序列号均不相同。64 位 ROM 的排列是：前 8 位是产品家族码，接着 48 位是 DS18B20 的序列号，最后 8 位是前面 56 位的循环冗余校验码(CRC=X8+X5+X4+1）。ROM 作用是使每一个 DS18B20 都各不相同，这样就实现了一根总线上挂载多个 DS18B20 模块同时工作且不会引起冲突。DS18B20 的模块管脚排列如下图所示：

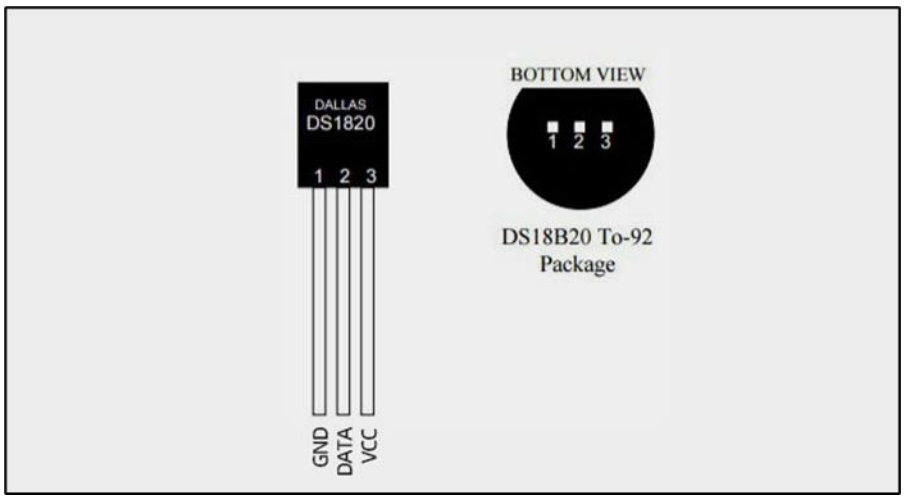


图 59.1.1.2 DS18B20 的管脚排列图

59.1.2 DS18B20 时序简介

所有单总线器件要求采用严格的信号时序，以保证数据的完整性。DS18B20 共有 6 种信号类型：复位脉冲、应答脉冲、写 0、写 1、读 0 和读 1。所有这些信号，除了应答脉冲以外，都是由主机发出同步信号。并且发送所有的命令和数据都是字节的低位在前。这里我们简单介绍这几个信号的时序。

1、复位脉冲和应答脉冲

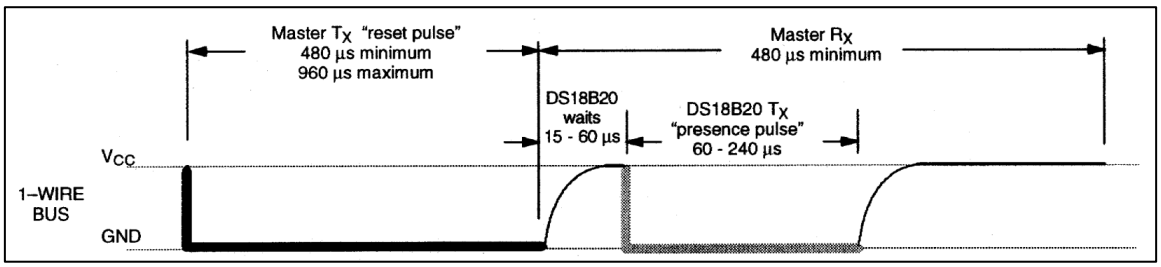


图 59.1.2.1 复位脉冲和应答脉冲时序图

单总线上的所有通信都是以初始化序列开始。主机输出低电平，保持低电平时间至少要在 480us，以产生复位脉冲。接着主机释放总线，4.7K 的上拉电阻将单总线拉高，延时时间要在 15~60us，并进入接收模式(Rx)。接着 DS18B20 拉低总线 60~240us，以产生低电平应答脉冲。

2、写时序

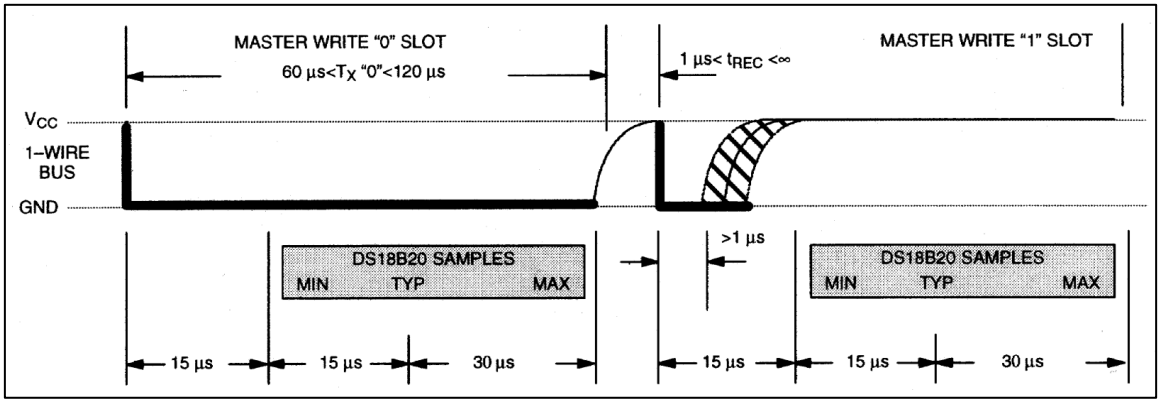


图 59.1.2.2 写时序图

写时序包括写 0 时序和写 1 时序。所有写时序至少需要 60us，且在两次独立的写时序之间至少需要 1us 的恢复时间，两种写时序均起始于主机拉低总线。写 1 时序：主机输出低电平，延时 2us，然后释放总线，延时 60us。写 0 时序：主机输出低电平，延时 60us，然后释放总线延时 2us。

3、读时序

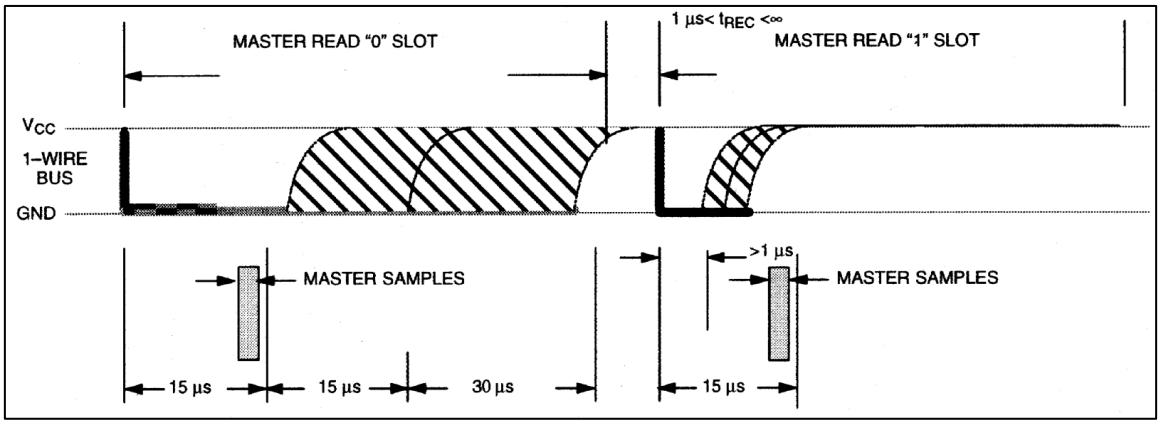


图 59.1.2.3 读时序图

单总线器件仅在主机发出读时序时，才向主机传输数据，所以，在主机发出读数据命令后，必须马上产生读时序，以便从机能够传输数据。所有读时序至少需要 60us，且在 2 次独立的读时序之间至少需要 1us 的恢复时间。每个读时序都由主机发起，至少拉低总线 1us。主机在读时序期间必须释放总线，并且在时序起始后的 15us 之内采样总线状态。典型的读时序过程为：主机输出低电平延时 2us，然后主机转入输入模式延时 12us，然后读取单总线当前的电平，然后延时 50us。

在了解单总线时序之后，我们来看一下 DS18B20 的典型温度读取过程，DS18B20 的典型温度读取过程为：复位→发 SKIP ROM（0xCC）→发开始转换命令（0x44）→延时→复位→发送 SKIP ROM 命令（0xCC）→发送存储器命令（0xBE）→连续读取两个字节数据（即温度）→结束。

59.2 DHT11 及工作时序简介

59.2.1 DHT11 简介

DHT11 是一款温湿度一体化的数字传感器。该传感器包括一个电阻式测湿元件和一个 NTC 测温元件，并与一个高性能 8 位单片机相连接。通过单片机等微处理器简单的电路连接就能够实时的采集本地湿度和温度。DHT11 与单片机之间能采用简单的单总线进行通信，仅仅需要一个 I/O 口。传感器内部湿度和温度数据 40Bit 的数据一次性传给单片机，数据采用校验和方式进行校验，有效的保证数据传输的准确性。DHT11 功耗很低，5V 电源电压下，工作平均最大电流 0.5mA。

DHT11 的技术参数如下：

- 工作电压范围：3.3V ~ 5.5V
- 工作电流：平均 0.5mA
- 输出：单总线数字信号
- 测量范围：湿度 20~90%RH，温度 0~50℃
- 精度：湿度±5%，温度±2℃
- 分辨率：湿度 1%，温度 1℃

DHT11 的管脚排列如图 59.2.1.1 所示：

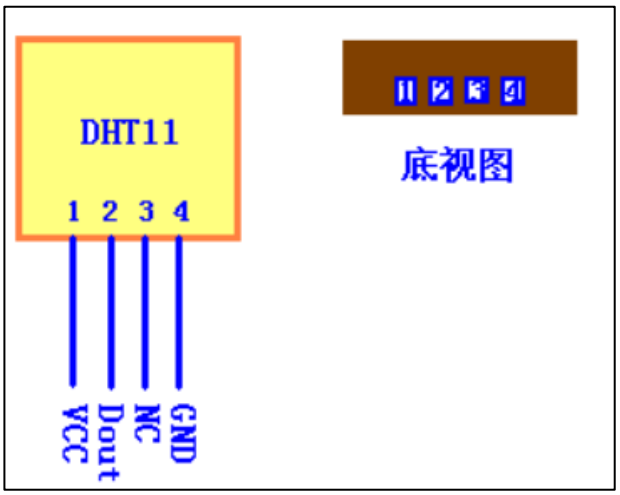


图 59.2.1.1 DHT11 管脚排列图

59.2.2 DHT11 工作时序简介

虽然 DHT11 与 DS18B20 类似，都是单总线访问，但是 DHT11 的访问，相对 DS18B20 来说简单很多。下面我们先来看看 DHT11 的数据结构。

DHT11 数字温湿度传感器采用单总线数据格式。即，单个数据引脚端口完成输入输出双向传输。其数据包由 5byte(40bit)组成。数据分小数部分和整数部分，一次完整的数据传输为 40bit，高位先出。DHT11 的数据格式为：8bit 湿度整数数据+8bit 湿度小数数据+8bit 温度整数数据+8bit 温度小数部分+8bit 校验和。其中校验和数据为前面四个字节相加。

传感器数据输出的是未编码的二进制数据。数据（湿度、温度、整数、小数）之间应该分开处理。例如，某次从 DHT11 读到的数据如图 59.2.2.1 所示：

byte4	byte3	byte2	byte1	byte0
00101101	00000000	00011100	00000000	01001001
整数	小数	整数	小数	校验和
湿度		温度		校验和

图 59.2.2.1 某次读取到 DHT11 数据

由以上数据就可得到湿度和温度的值，计算方法：

湿度 = byte4 . byte3 = 45.0(%RH)

温度 = byte2 . byte1 = 28.0(℃)

校验 = byte4 + byte3 + byte2 + byte1 = 73 (= 湿度 + 温度) （校验正确）

可以看出，DHT11 的数据格式十分简单的，DHT11 和 MCU 的一次通信最大为 3ms 左右，建议主机连续读取时间间隔不要小于 100ms。

下面，我们介绍一下 DHT11 的传输时序。DHT11 的数据发送流程如图 59.2.2.2 所示：

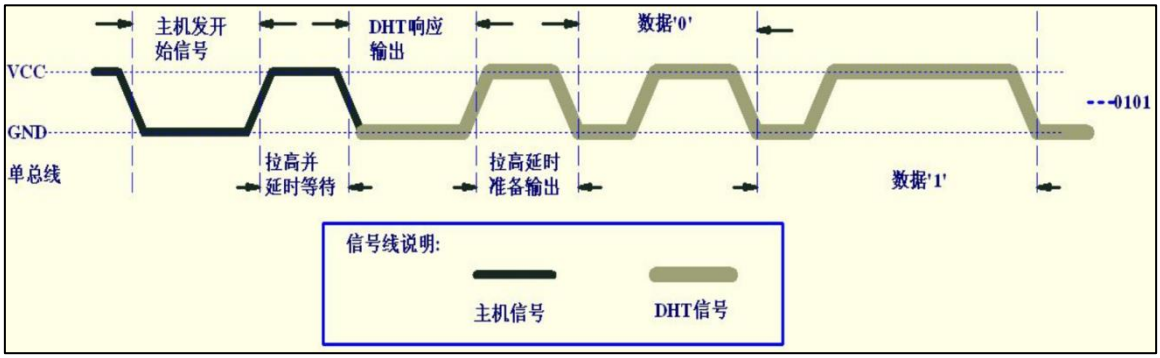


图 59.2.2.2 DHT11 数据发送流程图

首先主机发送开始信号，即：拉低数据线，保持 t1（至少 18ms）时间，然后拉高数据线 t2（20~40us）时间，然后读取 DHT11 的响应，正常的话，DHT11 会拉低数据线，保持 t3（40~50us）时间，作为响应信号，然后 DHT11 拉高数据线，保持 t4（40~50us）时间后，开始输出数据。

DHT11 输出数字‘0’时序如图 59.2.2.3 所示：

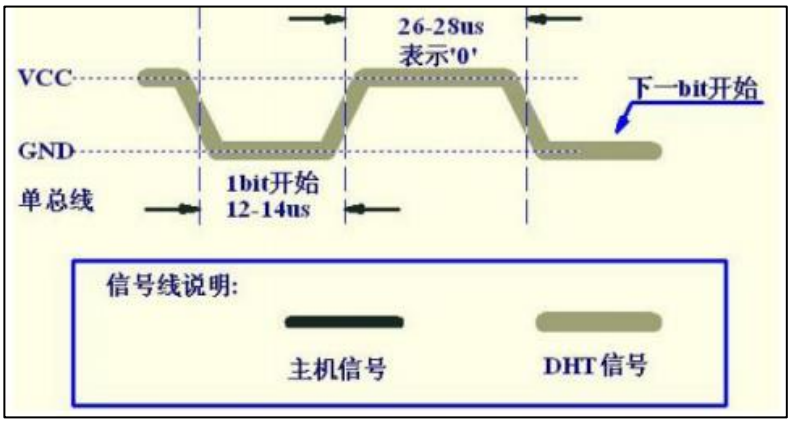


图 59.2.2.3 DHT11 数字‘0’时序图

DHT11 输出数字‘1’的时序如图 59.2.2.4 所示：

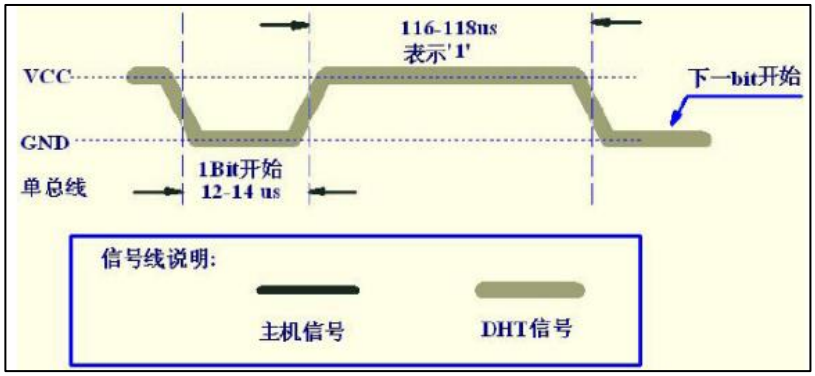


图 59.2.2.4 DHT11 输出数字‘1’时序图

通过以上了解，我们就可以通过 STM32MP157 来实现对 DHT11 的读取了。DHT11 的介绍就到这里，更详细的介绍，请参考 DHT11 数据手册。

59.3 硬件原理图分析

DHT11 或者 DS18B10 的硬件原理如下图所示：

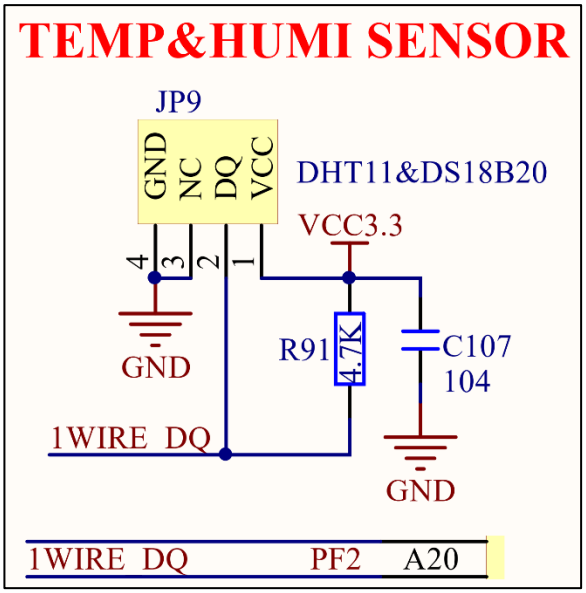


图 59.3.1 STM32MP157 的单总线硬件接口

图 59.3.1 中 JP9 为 DS18B20/DHT11 的接口，其中 DQ 为数据引脚，连接到了 STM32MP157 的 PF2 引脚上。JP2 是一个单排圆孔母座 4Pin 的座子，可用于连接 DHT11 或者 DS18B20，接口如下图所示：

图 59.3.2 DS18B20/DHT11 接口

DHT11 连接方法：正面(有孔面)朝向开发板的外侧。DS18B20 的连接方法：半圆的一面，对准底板上丝印半圆的三个脚，半圆是朝向开发板外侧。

59.4 DS18B20 实验编写

59.4.1 设备树的配置

打开 stm32mp157d-atk.dts 文件，把以下的内容添加到此文件中，内容如下所示：

示例代码 59.4.1.1 ds18b20 节点

```
1  ds18b20 {
2      compatible = "alientek,ds18b20";
3      ds18b20-gpio = <&gpiof 2 GPIO_ACTIVE_LOW>;
4      status = "okay";
5  };
```

59.4.2 ds18b20 驱动编写

本实验对应的例程路径为：开发板光盘：1、程序源码→2、Linux 驱动例程→34_ds18b20。新建名为 34_ds18b20 的文件夹，然后在此文件夹里面创建 vscode 工程，工作区命名为“ds18b20”。工程创建好以后新建一个 ds18b20.c 文件，此文件内容比较多，我们分步骤来写。步骤如下所示：

1、自定义结构体和设置 GPIO 控制函数

单总线是依靠一根 GPIO 来实现收发功能的，肯定是控制 GPIO 的模式和输出高低电平。代码如下所示：

示例代码 59.4.2.1 设置 GPIO 各种控制模式

```
21  struct ds18b20_dev {
22      struct miscdevice mdev;      /* MISC 设备            */
23      int gpio;                    /* GPIO 的编号          */
24      unsigned char data[2];       /* 接收原始数据的 BUFF  */
25      struct timer_list timer;     /* 定时器               */
26      struct work_struct work;     /* 工作队列             */
27  };
28
29  #define HIGH 1
30  #define LOW 0
31
32  struct ds18b20_dev ds18b20_device;
33
34  /*
35   * @description   : 设置 GPIO 的输出值
36   * @param - value : 输出 value 的值
37   * @return        : 无
38   */
39  static void ds18b20_set_output(int value)
40  {
41      if(value)
42          gpio_direction_output(ds18b20_device.gpio, 1);
43      else
44          gpio_direction_output(ds18b20_device.gpio, 0);
45  }
46
47  /*
48   * @description   : 设置 GPIO 为输入模式
49   * @param         : 无
50   * @return        : 无
51   */
52  static void ds18b20_set_input(void)
53  {
```

```
54      gpio_direction_input(ds18b20_device.gpio);
55  }
56
57  /*
58   * @description   : 获取 GPIO 的值
59   * @param         : 无
60   * @return        : GPIO 的电平
61   */
62  static int ds18b20_get_io(void)
63  {
64      return gpio_get_value(ds18b20_device.gpio);
65  }
```

第 25 行，定时器的作用是每一秒去获取温度，触发工作队列。

第 26 行，工作队列去处理 ds18b20 的初始化和获取温度。

2、DS18B20 的初始化函数

接着继续编写 DS18B20 的初始化函数，代码如下：

示例代码 59.4.2.2 复位和应答函数

```
148 /*
149  * @description   : 初始化 DS18B20
150  * @param         : 无
151  * @return        : 0，初始化成功，1，失败
152  */
153 static int ds18b20_init(void)
154 {
155     int ret = -1;
156     ds18b20_set_output(HIGH);    /* 把 DQ 拉高          */
157     udelay(1);                   /* 拉高 1us            */
158     ds18b20_set_output(LOW);     /* 把 DQ 拉低          */
159     udelay(500);                 /* 拉低 500us          */
160     ds18b20_set_output(HIGH);    /* 把 DQ 拉高          */
161     udelay(60);                  /* 拉高 60us           */
162     ds18b20_set_input();         /* 设置为输入模式      */
163     ret = ds18b20_get_io();      /* 获取到低电平做响应 */
164     udelay(240);                 /* 延时 240us          */
165
166     return ret;
167 }
```

上面的 ds18b20_init 函数是根据前面的时序图进行编写的，大家可以对比前面的时序图进行理解。

3、DS18B20 的读写函数

下面看下 DS18B20 的读写函数编写，代码如下所示：

示例代码 59.4.2.3 DS18B20 的读写函数

```
72  static void ds18b20_write_bit(int bit)
73  {
74      local_irq_disable();            /* 关闭处理器的所有中断  */
75      if(bit) {
76          ds18b20_set_output(LOW);    /* 把 DQ 拉低           */
77          udelay(2);                  /* 拉低 2us             */
78          ds18b20_set_output(HIGH);   /* 把 DQ 拉高           */
79          udelay(60);                 /* 拉高 60us            */
80      } else {
81          ds18b20_set_output(LOW);    /* 把 DQ 拉低           */
82          udelay(60);                 /* 拉低 60us            */
83          ds18b20_set_output(HIGH);   /* 把 DQ 拉高           */
84          udelay(2);                  /* 拉高 2us             */
85      }
86      local_irq_enable();             /* 开启处理器的所有中断  */
87  }
88
89  /*
90   * @description   : 读一位数据
91   * @param         : 无
92   * @return        : 返回读取一位的数据
93   */
94  static int ds18b20_read_bit(void)
95  {
96      u8 bit = 0;
97      local_irq_disable();
98      ds18b20_set_output(LOW);    /* 把 DQ 拉低          */
99      udelay(1);                  /* 拉低 1us            */
100
101     ds18b20_set_output(HIGH);   /* 把 DQ 拉高          */
102     udelay(1);                  /* 拉高 1us            */
103
104     ds18b20_set_input();        /* 设置为输入模式，开始接收数据  */
105
106     if(ds18b20_get_io())        /* 获取 DQ 的电平，高为 1，低为 0  */
107         bit = 1;
108     udelay(50);                 /* 延时 50us           */
109      local_irq_enable();
110     return bit;                 /* 返回读取数据        */
111 }
112
113 /*
114  * @description   : 写一个字节到 DS18B20
```

```
115  * @param byte    : 要写入的字节
116  * @return        : 无
117  */
118 static void ds18b20_write_byte(u8 byte)
119 {
120     int i;
121     for(i = 0; i < 8; i++) {
122         if(byte & 0x01)
123             ds18b20_write_bit(1);    /* write 1                */
124         else
125             ds18b20_write_bit(0);    /* write 0                */
126         byte >>= 1;                  /* 右移一位获取高一位的数据  */
127     }
128 }
129
130 /*
131  * @description   : 读取一个字节的数据
132  * @param         : 无
133  * @return        : 读取到的数据
134  */
135 static char ds18b20_read_byte(void)
136 {
137     int i;
138     u8 byte = 0;
139     for(i = 0; i < 8; i++) {     /* 先输出低位数据 ,高位数据后输出 */
140         if(ds18b20_read_bit())
141             byte |= (1 << i);
142         else
143             byte &= ~(1 << i);
144     }
145     return byte;
146 }
```

上面的读写函数都是按照前面分析 DS18B20 的读写时序编写的。这里使用 local_irq_disable 函数去关闭 CPU 的中断在去读写数据，读写数据完成后在使用 local_irq_enable 函数去开启中断。不关闭中断在读写数据时，CPU 有中断产生可能造成错误数据。

4、DS18B20 的读取温度函数

下面来编写读取温度函数，代码如下所示：

示例代码 59.4.2.4 DS18B20 读取温度函数

```
205 /*
206  * @description   : 使用内核的工作队列，获取温度的原始数据
207  * @param - work  : work 的结构体
208  * @return        : 无
209  */
```

```
210 static void ds18b20_work_callback(struct work_struct *work)
211 {
212     int ret = -1;
213      ret = ds18b20_init();        /* 初始化 ds18b20   */
214      if(ret != 0)
215          goto out1;
216     ds18b20_write_byte(0XCC);    /* 跳过 ROM         */
217     ds18b20_write_byte(0X44);    /* 开启温度采集     */
218
219      ret = ds18b20_init();        /* 初始化 ds18b20   */
220      if(ret != 0)
221          goto out1;
222
223     ds18b20_write_byte(0XCC);    /* 跳过 ROM         */
224     ds18b20_write_byte(0XBE);    /* 开启读取温度     */
225
226     ds18b20_device.data[0] = ds18b20_read_byte();   /* 获取低位数据 */
227     ds18b20_device.data[1] = ds18b20_read_byte();   /* 获取高位数据 */
228 out1:
229     return;
230 }
231
232 /*
233  * @description   : 定时器的操作函数，每 1s 去获取一次数据
234  * @param - asg   : 定时器的结构体
235  * @return        : 无
236  */
237 static void ds18b20_timer_callback(struct timer_list *arg)
238 {
239     schedule_work(&ds18b20_device.work); /*工作队列获取温度 */
240     mod_timer(&ds18b20_device.timer, jiffies + (1000 * HZ/1000));
241 }
```

定时器主要作用是定时去触发工作队列，然后在工作队列去读取 ds18b20 的温度值。

在这里简单介绍一下上面用到的指令：

跳过 ROM(0xCC)指令，该指令只适合总线只有一个节点，它允许总线上的主机不提供 64 位 ROM 序列号而直接访问 RAM，节省了操作时间。

温度转换(0x44)指令，启动 DS18B20 进行温度转换，结果存入内部 RAM。

读暂存器(0xBE)指令，读暂存器 9 个字节内容，该指令从 RAM 的第一个字节（字节 0）开始读取，直到九个字节（字节 8，CRC 值）全部被读出为止。如果不需要读出所有字节的内容，那么主机可以在任何时候发出复位信号以中止读操作。

5、MISC 设备框架

下面编写的是 MISC 框架，在内核里获取的温度通过 MISC 拷贝数据到用户层，代码如下：

示例代码 59.4.2.5 MISC 框架

```
177 static int ds18b20_open(struct inode *inode, struct file *filp)
178 {
179     return 0;
180 }
181 /*
182  * @description   : 从设备读取数据
183  * @param - filp  : 要打开的设备文件(文件描述符)
184  * @param - buf   : 返回给用户空间的数据缓冲区
185  * @param - cnt   : 要读取的数据长度
186  * @param - offt  : 相对于文件首地址的偏移
187  * @return        : 读取的字节数，如果为负值，表示读取失败
188  */
189 static ssize_t ds18b20_read(struct file *filp, char __user *buf,
                                size_t cnt, loff_t *offt)
190 {
191     int ret;
192     ret = copy_to_user(buf, &ds18b20_device.data[0], 2);
193     if(ret)
194         return -ENOMEM;
195     return ret;
196 }
197
198
199 static struct file_operations ds18b20_fops = {
200     .owner = THIS_MODULE,
201     .open  = ds18b20_open,
202     .read  = ds18b20_read,
203 };
```

注意：这里只是获取到上一秒的温度值。ds18b20_device.data 数据是通过工作队列去更新的。

6、platform 驱动框架

下面使用 platform 框架去编写 DS18B20 驱动，代码如下所示：

示例代码 59.4.2.6 platform 框架代码段

```
248 static int ds18b20_request_gpio(struct platform_device *pdev)
249 {
250     struct device *dev = &pdev->dev;
251     int ret;
252
253     ds18b20_device.gpio = of_get_named_gpio(dev->of_node,
                                                "ds18b20-gpio", 0);
254     if (!gpio_is_valid(ds18b20_device.gpio)) {
255         dev_err(dev, "Failed to get gpio");
256         return -EINVAL;
```

```
257     }
258
259     ret = devm_gpio_request(dev, ds18b20_device.gpio,
                                "DS18B20 Gpio");
260     if (ret) {
261         dev_err(dev, "Failed to request gpio");
262         return ret;
263     }
264
265     return 0;
266 }
267
268 /*
269  * @description  : 驱动的 probe 函数，当驱动与
270  *                 设备匹配以后此函数就会执行
271  * @param - pdev : pdev 设备
272  * @return       : 0 表示转换成功，其它值表示转换失败
273  */
274 static int ds18b20_probe(struct platform_device *pdev)
275 {
276     struct miscdevice *mdev;
277     int ret;
278
279     dev_info(&pdev->dev, "ds18b20 device and driver matched
                             successfully!\n");
280
281     /* GPIO 的初始化 */
282     ret = ds18b20_request_gpio(pdev);
283     if(ret)
284         return ret;
285     /* 初始化 MISG 设备 */
286     mdev = &ds18b20_device.mdev;
287     mdev->name = "ds18b20";
288     mdev->minor = MISC_DYNAMIC_MINOR;
289     mdev->fops = &ds18b20_fops;
290
291     /* 初始化定时器 */
292     timer_setup(&ds18b20_device.timer, ds18b20_timer_callback, 0);
293     ds18b20_device.timer.expires=jiffies + msecs_to_jiffies(1000);
294     add_timer(&ds18b20_device.timer);
295
296     /* 初始化工作队列 */
297     INIT_WORK(&ds18b20_device.work, ds18b20_work_callback);
```

```
298
299     /* MISG 设备注册 */
300     return misc_register(mdev);
301 }
302
303 /*
304  * @description   : 驱动的 remove 函数，移除驱动的时候此函数会执行
305  * @param - pdev  : pdev 设备
306  * @return        : 0，成功;其他负值,失败
307  */
308 static int ds18b20_remove(struct platform_device *pdev)
309 {
310     dev_info(&pdev->dev, "DS18B20 driver has been removed!\n");
311
312     /* 卸载 MISG 设备 */
313     misc_deregister(&ds18b20_device.mdev);
314     /* 卸载定时器 */
315     del_timer(&ds18b20_device.timer);
316     /* 卸载工作队列 */
317     cancel_work_sync(&ds18b20_device.work);
318     return 0;
319 }
320
321 static const struct of_device_id ds18b20_of_match[] = {
322     { .compatible = "alientek,ds18b20" },
323     { /* Sentinel */ }
324 };
325
326 static struct platform_driver ds18b20_driver = {
327     .driver = {
328         .name           = "ds18b20",
329         .of_match_table = ds18b20_of_match,
330     },
331     .probe          = ds18b20_probe,
332     .remove         = ds18b20_remove,
333 };
334
335 module_platform_driver(ds18b20_driver);
336 MODULE_LICENSE("GPL");
337 MODULE_AUTHOR("ALIENTEK");
338 MODULE_INFO(intree, "Y");
```

platform 平台驱动框架都学过了，就不讲解了。

59.4.3 编写测试 APP

测试 APP 很简单，在 31_ds18b20 目录下新建名为 ds18b20App.c 的文件，然后输入如下所示内容：

示例代码 59.5.3.1 ds18b20App.c

```
12  #include "stdio.h"
13  #include "unistd.h"
14  #include "sys/types.h"
15  #include "sys/stat.h"
16  #include "fcntl.h"
17  #include "stdlib.h"
18  #include "string.h"
19
20  int main()
21  {
22      int fd, ret;
23      unsigned char result[2];
24      int TH, TL;
25      short tmp = 0;
26      float temperature;
27      int flag = 0;
28
29      fd = open("/dev/ds18b20", 0);
30
31      if(fd < 0)
32      {
33          perror("open device failed\n");
34          exit(1);
35      }
36      else
37          printf("Open success!\n");
38
39      while(1)
40      {
41          ret = read(fd, &result, sizeof(result));
42          if(ret == 0) {          /* 读取到数据 */
43              TL = result[0];
44              TH = result[1];
45
46              if((TH == 0XFF) && (TL == 0XFF))/* 如果读取到数据为 0XFFFF
                                                  就跳出本次循序 */
47                  continue;
48              if(TH > 7) {        /* 负数处理      */
```

```
49                  TH = ~TH;
50                  TL = ~TL;
51                  flag = 1;       /* 标记为负数    */
52              }
53
54              tmp = TH;
55              tmp <<= 8;
56              tmp += TL;
57
58              if(flag == 1) {
59                  temperature = (float)(tmp+1)*0.0625;/* 计算负数的温度 */
60                  temperature = -temperature;
61              }else {
62                  temperature = (float)tmp *0.0625;   /* 计算正数的温度 */
63              }
64
65              if(temperature < 125 && temperature > -55) { /* 温度范围 */
66                  printf("Current Temperature: %f\n", temperature);
67              }
68          }
69      flag = 0;
70      sleep(1);
71      }
72      close(fd);  /* 关闭文件 */
73  }
```

59.5 DHT11 实验编写

59.5.1 设备树的配置

打开 stm32mp157d-atk.dts 文件，把以下的内容添加到此文件中，内容如下所示：

示例代码 59.5.1.1 dht11 节点

```
1   dht11 {
2       compatible = "alientek,dht11";
3       dht11-gpio = <&gpiof 2 GPIO_ACTIVE_LOW>;
4       status = "okay";
5   };
```

59.5.2 dht11 驱动编写

本实验对应的例程路径为：开发板光盘→1、程序源码→2、Linux 驱动例程→35_dht11。新建名为 35_dht11 的文件夹，然后在此文件夹里面创建 vscode 工程，工作区命名为“dht11”。工程创建好以后新建一个 dht11.c 文件，代码和 DS18B20 很多相似的，文档只讲解 dht11 相关的代码。步骤如下所示：

1、DHT11 读函数

DHT11 只有读函数，只需要实现读就行，代码如下所示：

示例代码 59.5.2.1 dht11 读一个字节函数

```
71  static unsigned char dht11_read_byte(void)
72  {
73      unsigned char i, time = 0, data = 0;
74      local_irq_disable();
75
76      for(i = 0; i < 8; i++) {
77          time = 0;
78
79          while(dht11_get_io() == 0) {
80              udelay(1);
81              time++;
82              if (time > 100) {
83                  return -EINVAL;
84              }
85          }
86          udelay(45);                     /* 延时 45us                    */
87          if (dht11_get_io() == 1) {  /*高电平，数据为 1，否则就是 0    */
88              data |= 1<<(7 - i);
89              time = 0;
90              while(dht11_get_io() == 1) {
91                  udelay(1);
92                  time++;
93                  if (time > 100)
94                      return -EINVAL;
95              }
96          }
97      }
98      local_irq_enable();
99
100     return data;
101 }
```

上面就是 DHT11 的读函数，根据读的时序来编写的。

2、DHT11 初始化函数

下面我们就编写 DHT11 的初始化函数和应答函数，如下示例代码所示：

示例代码 59.5.2.2 DHT11 的初始化函数

```
108 static int dht11_init(void)
109 {
110     dht11_set_output(HIGH); /* 拉高 Dout    */
111     udelay(30);             /* 拉高 30us    */
```

```
112
113     dht11_set_output(LOW);   /* 拉低 Duot    */
114     mdelay(20);              /* 拉低 20us    */
115
116     dht11_set_output(HIGH);  /* 拉高 Duot    */
117     udelay(30);              /* 拉高 30us    */
118
119     dht11_set_input();       /* 设置 Duot 为输入模式  */
120     udelay(200);             /* 延时 200us          */
121     if(!dht11_get_io()) {    /* 不是高电平，DHT11 就没有响应 */
122         return -ENODEV;
123     }
124     return 0;
125 }
```

上面的 dht11_init 函数就是根据前面的复位脉冲和应答脉冲的时序编写，笔者把它们组成 DHT11 的初始化函数，大家可以对比前面的时序图进行理解。

3、DHT11 读取温度函数

下面我们就要编写工作队列处理函数，代码如下所示：

示例代码 59.5.2.3 MISC 框架

```
166 static void dht11_work_callback(struct work_struct *work)
167 {
168     int i = 0;
169     unsigned char buff[5];
170
171     if(!dht11_init()) {
172         for(i = 0; i < 5; i++) {
173             buff[i] = dht11_read_byte();    /* 获取数据 */
174         }
175             /* 校验数据是否正确 */
176         if((buff[0] + buff[1] + buff[2] + buff[3]) == buff[4]) {
177             memcpy(&dht11_device.data[0], &buff[0], 5);
178         }
179     }
180 }
181
182 /*
183  * @description   : 定时器的操作函数，每 1s 去获取一次数据
184  * @param - asg   : 定时器的结构体
185  * @return        : 无
186  */
187 static void dht11_timer_callback(struct timer_list *arg)
188 {
189     schedule_work(&dht11_device.work);
```

```
190     mod_timer(&dht11_device.timer, jiffies + (1500 * HZ/1000));
191 }
```

dht11_work_callback 函数很简单就是先初始化 DHT11，然后获取 DHT11 的 5byte 数据。其它的代码和 ds18b20 相似就不去讲解了。

59.5.3 编写测试 APP

测试 APP 很简单，在 32_dht11 目录下新建名为 dht11App.c 的文件，然后输入如下所示内容：

示例代码 59.5.3.1 dht11App.c 文件内容

```
12  #include "stdio.h"
13  #include "unistd.h"
14  #include "sys/types.h"
15  #include "sys/stat.h"
16  #include "fcntl.h"
17  #include "stdlib.h"
18  #include "string.h"
19
20  int main()
21  {
22      int fd, ret, i = 0;
23      unsigned char data[5];
24
25
26      fd = open("/dev/dht11", 0);
27
28      if(fd < 0)
29      {
30          perror("open device failed\n");
31          exit(1);
32      }
33      else
34          printf("Open success!\n");
35
36      while(1)
37      {
38          ret = read(fd, &data, sizeof(data));
39          if(ret == 0) {  /* 读取到数据 */
40              if(data[4] == data[0] + data[1] + data[2] + data[3]) {
41                  printf("Temp:%d.%d℃, Humi:%d.%d%\r\n",
                            data[2],data[3], data[0],data[1]);
42              }
43          }
44      sleep(1);
```

```
45      }
46  }
```

59.6 DS18B20 运行测试

59.6.1 编译驱动程序和 APP

1、编译驱动程序

编写 Makefile 文件，将 22_spi 实验目录下的 Makefile 文件拷贝到本实验目录下，打开改 Makefile 文件，将 obj-m 变量的值改为“ds18b20.ko”，Makefile 内容如下所示：

示例代码 59.6.1.1 Makefile 文件

```
1   KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
......
4   obj-m := ds18b20.ko
......
11  clean:
12      $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“ds18b20.ko”。

输入如下命令编译出驱动模块文件：

```
make
```

编译成功以后就会生成一个名为“ds18b20.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 ds18b20App.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc ds18b20App.c -o ds18b20App
```

编译成以后就会生成 ds18b20App 这个应用程序。

59.6.2 运行测试

先把硬件连接到开发板上，接法请看 59.3 小节里有说明。

将上一小节编译出来 ds18b20.ko 和 ds18b20App 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，使用新的设备树启动开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 ds18b20.ko 驱动模块：

```
depmod                  //第一次加载驱动是时候需要运行此命令
modprobe ds18b20.ko     //加载驱动
```

当驱动模块加载成功以后使用 ds18b20App 来测试，输入如下命令：

```
ds18b20App  /dev/ds18b20
```

测试 APP 会不断的从 ds18b20 中读取数据，然后输出到终端上，如图 59.6.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe ds18b20
ds18b20 ds18b20: ds18b20 device and driver matched successfully!
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./ds18b20App /dev/ds18b20
Open success!
Current Temperature: 31.062500
Current Temperature: 31.125000
Current Temperature: 31.125000
Current Temperature: 31.187500
Current Temperature: 31.187500
Current Temperature: 31.375000
Current Temperature: 32.062500
Current Temperature: 32.625000
Current Temperature: 33.062500
Current Temperature: 33.375000
^C
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 59.6.2.1 获取到 ds18b20 的温度

59.7 dht11 运行测试

59.7.1 编译驱动程序和 APP

1、编译驱动程序

编写 Makefile 文件，将 22_spi 实验目录下的 Makefile 文件拷贝到本实验目录下，打开改 Makefile 文件，将 obj-m 变量的值改为“dht11.o”，Makefile 内容如下所示：

示例代码 59.6.1.1 Makefile 文件内容

```
1   KERNELDIR := /home/zuozhongkai/linux/my_linux/linux-5.4.31
.....
4   obj-m := dht11.ko
.....
11  clean:
12      $(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

第 4 行，设置 obj-m 变量的值为“dht11.ko”。

输入如下命令编译出驱动模块文件：

```
make
```

编译成功以后就会生成一个名为“dht11.ko”的驱动模块文件。

2、编译测试 APP

输入如下命令编译测试 dht11App.c 这个测试程序：

```
arm-none-linux-gnueabihf-gcc dht11App.c -o dht11App
```

编译成以后就会生成 dht11App 这个应用程序。

59.7.2 运行测试

先把硬件连接到开发板上，接法请看 59.3 小节里有说明。

将上一小节编译出来 dht11.ko 和 dht11App 这两个文件拷贝到 rootfs/lib/modules/5.4.31 目录中，使用新的设备树启动开发板，进入到目录 lib/modules/5.4.31 中，输入如下命令加载 dht11.ko 驱动模块：

```
depmod              //第一次加载驱动是时候需要运行此命令
modprobe dht11.ko   //加载驱动
```

当驱动模块加载成功以后使用 dht11App 来测试，输入如下命令：

```
dht11App  /dev/dht11
```

测试 APP 会不断的从 dht11 中读取数据，然后输出到终端上，如图 59.7.2.1 所示：

```
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ modprobe dht11
dht11 dht11: dht11 device and driver matched successfully!
[root@ATK-stm32mp1]:/lib/modules/5.4.31$ ./dht11App /dev/dht11
Open success!
Temp:30.9℃,Humi:48.0%
Temp:30.9℃,Humi:52.0%
Temp:30.9℃,Humi:52.0%
Temp:30.9℃,Humi:58.0%
Temp:31.0℃,Humi:64.0%
Temp:31.0℃,Humi:64.0%
^C
[root@ATK-stm32mp1]:/lib/modules/5.4.31$
```

图 59.7.2.1 获取到 dht11 的温度和湿度

图 59.7.2.1 中用手捏着 DHT11，所以温度会逐渐上升，湿度也会增加，因为手上有汗。

未完待续，持续更新中